REALIZATION CALCULUS

Komorebi Math

Technical Manual and Reference

The Python library

Komorebi P.

Finite constructions. Explicit contracts. Inspectable evidence.

Preface

Komorebi Math brings exact algebra, numerical calculation, symbolic expressions and finite computational models into one Python library. This manual develops the mathematical objects behind its interfaces and records their domains, invariants, return values, failure modes and evidence boundaries. It is both a connected exposition and a detailed working reference.

The first parts introduce the shared language and its main constructions. Later chapters connect arithmetic, signals, nonlinear dynamics, integration, quantum models and portable results. The reference appendices retain the stable, beta and research API records, the module inventory, executable examples, command options, schemas, diagnostic recipes and notation. Documentation standards and design specifications follow the operational reference. Specifications marked `[PLANNED]` remain specifications, rather than assertions of an implemented or executed contract.

The distinction between exact algebra and finite evidence matters throughout. A numerical residual, a sampled check, a source-derived inventory and a proved identity answer different questions. Each retains its own hypotheses and limitations. Read each claim with the assumptions and evidence supplied beside it.

The manual is designed for both first use and sustained reference. Begin with the practical guide for environment setup, two small calculations, and a route into your subject. Part-level guides and chapter introductions explain where to start and what to consult next. Detailed contracts and examples remain available when a calculation requires a closer look.

Reference scope

The detailed source-derived records and recorded execution observations describe the reference implementation from which the comprehensive manual was assembled. Older package filenames, version strings, verifier names and failure observations are retained where they identify a literal historical command or an audited record. They must not be read as a fresh qualification of the current release. Current test and verification inventories, with their source binding and methodology, are collected in Appendix O. The practical guide at the beginning of this volume supplies current-source introductory workflows. Literal reference transcripts retain the versions they name.

Reading paths

Start with the practical guide and a small example. Consult Parts I and II for the library's conventions and evidence vocabulary. Continue through the mathematical part relevant to the calculation at hand. Use Appendices A–D to locate a public object or implementation module, E and F for executable examples and command options, and G–N for persistence, diagnostics, dependencies and detailed cross-references. The glossary and notation index are in Appendix M. PDF bookmarks expose the deeper reference hierarchy while the printed contents show the principal divisions.

Source and publication note

The comprehensive reference baseline is the preserved `komorebi_math` source documentation associated with the `1.0.0b4` implementation inventory. The current software release is bound to source commit `5f2b7ddc879c12d2bb00cbe5c487aa0bc7709222`. Historical execution counts and source-derived records remain explicitly scoped to their recorded baseline. Appendix O supplies a separate current-source crosswalk. Neither a static crosswalk nor this editorial work constitutes new hardware qualification or a new proof audit.

The 131 stable, 83 beta and 1,312 research symbol records were compared with the frozen current-release manifests. All 1,526 records agree exactly. This establishes continuity of the recorded interfaces; it does not assert that every historical failure observation or command workflow still describes current implementation behavior.

Original software and documentation are distributed under the MIT License. Third-party notices and separately supplied data terms continue to apply. The source package includes the editable manuscript, typesetting files, original source, preservation checks and proofreading records.

Author: Komorebi P.

Series: Realization Calculus

Subject: Python library, mathematical contracts and technical reference

Contents

Preface . . . i
Source and publication note . . . ii
A practical guide to this manual . . . 1
Reading the evidence . . . 6

Part I. Orientation and first execution **7**
0. Coverage contract . . . 9
1. Editorial and symbolic grammar . . . 12
2. What Komorebi Math is . . . 20
3. Installation and environment profiles . . . 23
4. First verified program . . . 28

Part II. Trust, types, and execution semantics **33**
5. Certificates and result types . . . 35
6. Errors, assumptions, and resource guards . . . 44
7. Public API tiers . . . 52
8. Backends, scalar domains, and coercion . . . 60

Part III. Operator, packet, and refinement foundations **69**
9. Typed operator grammars . . . 71
10. Operator algebras and seed systems . . . 85
11. Packet systems and transfer blocks . . . 99
12. Refinement towers . . . 116
13. Residue, CRT, lattice, and Smith coordinates . . . 135
14. Polyphase systems and refinement . . . 145

Part IV. Coordinate calculus and analysis-synthesis systems **152**
15. Coordinate objects . . . 154
16. Exact and numerical coordinate transforms . . . 163
17. Polynomial and classical coordinate families . . . 178
18. Combinatorial, recurrence, and state-space coordinates . . . 204
19. Spectral, packet, tensor, and operator transport . . . 228
20. Analysis-synthesis and redundant frames . . . 248

Part V. Generating functions, arithmetic, sequences, and analytic packets **268**
21. Formal series and weighted bases . . . 270
22. Normalized generating functions . . . 286
23. Masks, Cartier operators, and digit algebra . . . 309
24. Prime-power, Bell, cyclotomic, and q-Kummer systems . . . 328
25. Local arithmetic models . . . 347
26. Exact sequence execution . . . 371
27. Cyclic analytic systems . . . 394

Part VI. Structured numerical computation and signal processing **411**
28. Packet intermediate representation . . . 413
29. Structured and low-rank operators . . . 419
30. Krylov, precision, compilation, and scheduling . . . 424

31. Signal conventions, windows, and framing . . . 433
32. Time-frequency and spectral analysis . . . 442
33. Filtering and convolution . . . 457
34. Resampling and adaptive filters . . . 467
35. Filter banks, wavelets, and packet trees . . . 475
36. Singular packets and recovery . . . 491
36A. Approximation, nonlinear packet products, and Smith charts . . . 502

Part VII. Nonlinear calculus and finite dynamics **509**
37. Derivative providers and automatic differentiation . . . 511
38. Packet jets and Faà di Bruno composition . . . 522
39. Nonlinear trees and graded operations . . . 537
40. Defects, obstruction, lifting, and holonomy . . . 549
41. Correction and elimination . . . 565
42. Filtered lifting and finite spectral sequences . . . 579
43. Nonlinear execution, streaming, profiling, and GPU paths . . . 593
44. Smooth packet transport . . . 611
45. Matrix dynamics . . . 621
46. Feshbach reduction, memory, and survival . . . 638

Part VIII. Exact finite integration and typed expressions **649**
47. Measures and exact polynomials . . . 651
48. Integration providers and quadrature . . . 658
49. Integral identities and coordinate covectors . . . 666
50. Typed finite expression model . . . 674
51. Expression evaluation and certification . . . 682
52. Expression planning and persistence . . . 689

Part IX. Quantum, deformation, and finite Langlands models **697**
53. Finite quantum foundations . . . 699
54. Circuits, channels, and open dynamics . . . 702
55. Error correction and effective dynamics . . . 706
56. Finite deformation systems . . . 708
57. Local finite Langlands packets . . . 712
58. Finite-image, ramification, and completion models . . . 715

Part X. Export, persistence, extension, and command-line use **719**
59. Mathematical document model . . . 721
60. Renderers and writers . . . 724
61. Portable persistence . . . 727
62. Provenance, registries, and extensions . . . 730
63. Complete CLI reference . . . 732

Part XI. Connected recipes **735**
64. Exact algebra and coordinate recipes . . . 737
65. Arithmetic and sequence recipes . . . 739
66. Signal-processing recipes . . . 741
67. Nonlinear and dynamics recipes . . . 743
68. Integration and expression recipes . . . 745

69. Quantum, deformation, and Langlands recipes . . . 747
70. Export and reproducibility recipes . . . 750

Part XII. Verification, release, and reproducibility **753**

71. Verification suites . . . 755
72. API, semantic, persistence, and upgrade contracts . . . 757
73. Platform and release qualification . . . 760
74. Source publication handoff . . . 762
74A. Source reconciliation . . . 765

Part XIII. Exhaustive reference appendices **766**

Appendix A. Stable API: 131-entry frozen reference . . . 767
Appendix B. Beta API: 83-entry reference . . . 825
Appendix C. Research API: 1,312-entry reference . . . 866
Appendix D. Module reference ledger . . . 1721
Appendix E. Executable-example records . . . 1823
Appendix F. Complete CLI option matrix . . . 1877
Appendix G. Portable schemas and migrations . . . 1928
Appendix H. Certificate and theorem-ID index . . . 1965
Appendix I. Exceptions and diagnostic cookbook . . . 2105
Appendix J. Backend and dependency matrix . . . 2157
Appendix K. Complexity, memory, and resource-budget tables . . . 2189
Appendix L. Migration and deprecation tables . . . 2223
Appendix M. Glossary and notation index . . . 2243
Appendix N. Documentation-source crosswalk . . . 2306
Appendix O. Test and verification crosswalk . . . 2333

Part XIV. Documentation standards and design specifications **2398**

75. Freeze and finalization protocol . . . 2400
76. Symbol-card completion contract . . . 2403
77. Executable-example review . . . 2405
78. Connected tutorial review . . . 2407
79. Warning, limitation, and drift audit . . . 2409
80. Coverage ledger and symbol index . . . 2411
81. Site and PDF build closure . . . 2413
82. Publication requirements and reference architecture . . . 2415
83. Foundations, refinement, coordinates, and analysis-synthesis . . . 2418
84. Generating functions, arithmetic, sequences, and analytic systems . . . 2444
85. Algorithms and signals . . . 2469
86. Nonlinear systems and dynamics . . . 2498
87. Integration, expressions, quantum, deformation, and Langlands . . . 2519
88. Export, persistence, CLI, recipes, verification, and publication . . . 2543

A practical guide to this manual

You can use Komorebi Math without reading this book from beginning to end. Start with one small calculation, learn how to interpret its result, and then follow the chapter for your subject. The detailed reference is there when you need an exact import name, argument, assumption, or failure condition.

The manual has three complementary layers. The chapters explain mathematical objects and show how to use them. The connected recipes combine those objects into workflows. The appendices give detailed lookup records. A useful reading pattern is **example, explanation, reference**: run a small example, read the relevant explanation, and consult the reference before changing its inputs or scale.

Choose a starting point

What you want to do	Start here	Keep nearby
Install the library and complete a first calculation	The setup and examples below, then Chapters 3–4	Chapter 5 for interpreting results
Understand the library's organizing ideas	Chapter 2, then Chapters 9–12	Appendix M for terminology
Work with exact coordinates, bases, or rational matrices	Chapters 15–20, then Chapter 64	Appendix A for stable names
Work with sequences, generating functions, or arithmetic	Chapters 21–27, then Chapter 65	The scalar-domain guidance in Chapter 8
Process signals or build numerical operators	Chapters 28–36A, then Chapter 66	Appendix J for dependencies
Work with derivatives, jets, nonlinear maps, or dynamics	Chapters 37–46, then Chapter 67	Appendix K before scaling up
Integrate finite exact objects or evaluate typed expressions	Chapters 47–52, then Chapter 68	Chapter 5 and the relevant API card
Explore finite quantum, deformation, or Langlands models	Chapters 53–58, then Chapter 69	The local hypotheses and evidence class of each model
Save, export, reload, or extend a calculation	Chapters 59–63, then Chapter 70	Appendix G for portable records
Diagnose an error or unexpected result	Chapter 6 and Appendix I	The failing object's contract and your environment details
Maintain or verify the project	Chapters 71–74A and Appendix O	Part XIV for documentation standards and design specifications

Chapter ranges describe a route, not required reading. For example, a signal-processing reader can start with the signal conventions in Chapter 31 and return to packet algebra when a later operation needs it. The glossary is a reference aid, not an examination to pass before using the library.

What to know before you start

For the introductory Python examples, you should be comfortable importing modules, calling functions, reading lists and tuples, and running a script from a terminal. A terminal command belongs in PowerShell or a POSIX shell. A Python listing belongs in a Python file or interpreter. Output listings show what a program prints and should not be entered as commands.

For the mathematical chapters, the prerequisite depends on the task. Basic vectors and matrices are enough for the first packet and coordinate examples. Later arithmetic, differential, signal, and quantum topics introduce their own

specialized objects. Read the local definitions before relying on a familiar word: a signal frame and a redundant algebraic frame have different meanings, as Appendix M explains.

In a displayed map, the domain describes valid inputs and the codomain describes outputs. In a matrix acting on a column vector, columns correspond to input coordinates and rows to output coordinates. A hypothesis is a condition the construction needs. An invariant is a property that should remain true. A residual measures a discrepancy in a numerical check. A certificate records the checks and assumptions reported by its producer. These terms recur because they help you decide whether a result fits your intended use.

Set up one environment

Use the source checkout or distribution supplied with the release you intend to run. The current-source instructions here refer to the 1.0.0 source identified in the publication note. The detailed historical commands in Chapter 3 remain useful when reproducing their explicitly named reference environment.

The package requires Python 3.11 or newer; the declared support matrix is Python 3.11, 3.12, and 3.13. Use an interpreter from that matrix and check its version before creating an environment. Run the following commands from the **software source directory containing `pyproject.toml`**, rather than from the manual's editable-source directory.

On a POSIX shell:

```
python3 --version
python3 -m venv .venv
. .venv/bin/activate
python -m pip install .
python -m pip check
```

On Windows PowerShell:

```
py --version
py -m venv .venv
$py = (Resolve-Path .\.venv\Scripts\python.exe).Path
& $py -m pip install .
& $py -m pip check
```

If the default interpreter is outside that matrix, select an installed supported version before running the environment-creation command. The Windows form invokes the environment's interpreter directly, so activation is unnecessary. On either platform, installing from source can require downloading build tools and dependencies. An offline installation requires those dependencies to be available locally too.

If you are using a supplied wheel instead, replace the installation target `.` with that wheel's actual path. Use one installation route in a fresh environment. Do not combine an editable checkout and a wheel installation while trying to establish which copy is being imported.

After installation, run this Python check with that same interpreter:

```
import sys
import komorebi_math as km

print("Python:", sys.executable)
print("Version:", km.__version__)
print("Imported from:", km.__file__)
```

For the identified current source, the version should be `1.0.0`. The executable and import paths depend on your machine. If the version is unexpected, check those paths before changing the example. A local file named `komorebi_math.py` or another environment can cause Python to import a different object than intended.

The examples below use the core installation. Optional dependencies are introduced when a particular operation needs them; use Chapter 3 and Appendix J to choose them. Installing every optional backend is unnecessary for a first calculation.

Your first exact calculation

This example changes two coordinates using a rational matrix and then reverses the change. Save the listing as `first_exact.py` and run `python first_exact.py` in the activated POSIX environment, or `& $py .\first_exact.py` in PowerShell.

```
from fractions import Fraction
from komorebi_math.stable import ExactCoordinateTransform, Exactness

transform = ExactCoordinateTransform(
    "A", "B",
    ((1, Fraction(1, 2)), (0, 1)),
    "USER-GUIDE-SHEAR",
    "A rational coordinate example",
)
original = (Fraction(2), Fraction(3))
mapped = transform.apply(original)
restored = transform.inverse().apply(mapped)
certificate = transform.certificate().require()

assert mapped == (Fraction(7, 2), Fraction(3))
assert restored == original
assert certificate.exactness is Exactness.EXACT

print("Mapped:", ", ".join(str(value) for value in mapped))
print("Restored:", ", ".join(str(value) for value in restored))
print("Evidence:", certificate.exactness.value)
```

Expected output:

```
Mapped: 7/2, 3
Restored: 2, 3
Evidence: exact
```

The first output coordinate is two plus half of three. The second is unchanged. `Fraction(1, 2)` supplies a rational value directly. The strings `A` and `B` identify the source and target coordinate systems for this record; they do not ask the library to discover a basis. The identifier `USER-GUIDE-SHEAR` labels this example's contract and does not supply an independent proof.

The assertions check the expected value and inverse roundtrip. The certificate's `.require()` method enforces acceptance of the finite record reported by the producer. In this example, the matrix is invertible; a valid transform record in general need not be invertible. Chapter 16 explains the distinction and the conditions for inversion.

Your first numerical packet calculation

A packet decomposition lets you inspect components and then reconstruct an object. In this two-dimensional example, the first component keeps the first coordinate and the second component keeps the second. Save it as `first_packets.py` and run it with the same interpreter.

```
import numpy as np
from komorebi_math.stable import Exactness, PacketSystem

packets = PacketSystem(
    [np.diag([1.0, 0.0]), np.diag([0.0, 1.0])],
    names=("left", "right"),
)
certificate = packets.certificate().require()
original = np.array([2.0, 3.0])
coordinates = packets.analyze(original)
restored = coordinates.reconstruct()

np.testing.assert_array_equal(restored, original)
assert certificate.exactness is Exactness.NUMERICAL

print("Components:", [value.real.tolist() for value in coordinates.values])
print("Restored:", restored.real.tolist())
print("Evidence:", certificate.exactness.value)
```

Expected output:

```
Components: [[2.0, 0.0], [0.0, 3.0]]
Restored: [2.0, 3.0]
Evidence: numerical
```

Here each component is represented in the original two-dimensional space. Reconstruction adds the components. Other coordinate systems may use different coefficient representations, so read their contracts before transferring this interpretation.

The numerical label remains appropriate even though this simple run returns the expected values exactly. It identifies the floating computation, rather than a rational-arithmetic proof. The equality assertion is suitable for these particular inputs. In a general floating workflow, choose the norm and tolerance required by the operation instead of copying an exact equality check without considering rounding.

Read a result before using it

Start with the returned value, then inspect the information that qualifies it. A useful order is:

1. **Meaning and shape.** What mathematical object is returned, and which axes, coordinates, or units does it use?
2. **Acceptance.** Does its producer report validity? If further work requires acceptance, enforce the documented check.
3. **Evidence class.** Is the result exact, numerical, conditional, heuristic, or planned? API stability is a separate property.
4. **Conditions.** Which assumptions were checked, and which remain the caller's responsibility?
5. **Numerical interpretation.** If there is a residual or tolerance, which quantity does it measure and in which norm?
6. **Next operation.** Does the result meet the input requirements of the operation you intend to call next?

Do not assume that every result object has the same fields or methods. Chapter 5 explains the common certificate model; each API card identifies what its particular operation returns. The literal code above uses methods supported by those two objects.

Find an object without reading every card

If you know the public Python name, search for it in Appendices A, B, and C. These are the stable, beta, and research references. If you know only a task, start with the route table above or the connected recipes in Part XI, then follow their named objects into the reference. If you know an implementation module, use Appendix D.

Read a card in this order: import route, purpose, signature, input contract, return value, failure conditions, and example. Then inspect its evidence and backend notes. The signature tells you how to call an object; the surrounding contract tells you when that call makes sense. An alias can name the same implementation, but a similarly named object need not have the same meaning.

Use the PDF's bookmarks to move directly to a chapter or reference group. The printed contents keep the principal divisions compact. Exact name searches work best when you copy a public name from the source or from a code listing; typeset line wrapping can divide a long identifier visually. Chapter and section numbers remain the preferred references when discussing the book with another reader.

When something goes wrong

Begin with the smallest example that reproduces the problem. Keep the original error message. Changing several settings at once can hide the cause.

Symptom	First useful check	Continue with
Import fails or the version is unexpected	Confirm the interpreter, imported path, installation target, and optional dependency	Chapter 3 and Appendix J
A name or argument is rejected	Compare the exact spelling, namespace, signature, and reference version	Chapter 7 and Appendices A–F
A dimension, domain, or validation error occurs	Check shape, source/target labels, scalar type, and the failing contract	Chapter 6 and Appendix I
A numerical answer differs from an expectation	Check axis, normalization, precision, norm, tolerance, and conditioning	The domain chapter and Chapter 8
A certificate is rejected	Inspect failed checks and assumptions before changing tolerances	Chapter 5 and Appendix H
A calculation uses too much memory or time	Reduce the smallest relevant dimension or order, then inspect the growth estimate	Appendix K
A saved record cannot be loaded	Check schema version, supported codec, integrity information, and migration route	Chapters 61–62 and Appendices G and L

When asking for help, include the package version, interpreter and platform, relevant optional backend versions, a small complete example, its inputs, the full error or unexpected output, and what you expected. For a numerical issue, include the tolerance and precision. For a persistence issue, describe the record type and version. These details usually locate the problem more quickly than a screenshot of a long session.

Continue with confidence

After the first examples, choose one domain chapter and reproduce its smallest working case. Change one input, predict the effect, and inspect both the value and its evidence. Then try the documented failure case so you know what the boundary looks like. Only after those steps should you increase dimension, order, precision, or data volume.

Use the declared source baseline and the local contract to decide whether an example applies to your calculation. The reference distinguishes documented interfaces from executed backend and platform checks, and from mathematical assumptions that remain to be established.

Reading the evidence

The reference records describe the library's public mathematical and computational surfaces. Their inventories and execution observations belong to the documented reference baseline. Source inspection, command-line parsing, finite numerical checks and mathematical proofs remain separate forms of evidence.

The documentation-source crosswalk joins the configured documentation and repository records to API, module, command and example entries. Its recorded checks include finite correction probes and lookup scenarios. Acceptance by an argument parser does not establish that a command handler executed successfully. Lexical code and assertion counts support navigation, rather than blanket execution or proof coverage.

Extension discovery deserves particular care. Loading an entry point or a callable factory runs its code in the caller process. Catching ordinary exceptions does not provide child-process isolation. The default does not activate extensions, but discovery can still import and run factory code.

The recorded documentation checks resolve ordinary local file links within their stated source set. They do not establish that every local fragment or external URL resolves. A lexical Markdown scan can also mistake mathematical punctuation for a link; that limitation is not itself a mathematical or site-build failure.

Reference observations about native platforms, accelerators, PowerShell, certificates, persistence and resource controls retain their stated limitations. Current test and verification inventory is supplied separately in Appendix O. No new Lean proof audit is claimed.

Part I. Orientation and first execution

A practical first route. Begin with Chapter 2 for the purpose of the library, Chapter 3 for the environment, and Chapter 4 for a small computation you can inspect from input to result. Use the label table in Section 1.1 as a reference. Chapters 0 and 1 explain how the full manual is organized; they can be read more closely after your first successful example.

The first examples assume basic Python: imports, function calls, lists, and reading an exception. The numerical packet example also uses small matrices and vectors. The exact coordinate example uses rational numbers. You do not need the advanced operator, signal-processing, or arithmetic chapters before running those examples.

Your immediate goal	Start here	What to look for
Understand the library's purpose	Chapter 2	finite objects, supported workflows, and the limits of each claim
Set up an environment	Chapter 3	the intended interpreter, package identity, and required dependencies
Run a small calculation	Sections 4.3 and 4.4	reconstructed values and the accompanying certificate
Understand a result label	Section 1.1, then Chapter 5	API stability, exactness, validity, and assumptions
Diagnose an unsuccessful run	Section 3.5, then Chapter 6	environment, input, assumption, or resource failure
Find an importable name	Appendices A through C	the exported name, signature, tier, and related examples

Reading the examples. A Python listing contains program text. A shell listing contains commands to enter in the named shell. An output listing shows what a run produced and should not be pasted into the interpreter. Run a complete example, including its imports and setup, before adapting individual lines. When a later listing continues an example, keep the preceding variables in the same Python session.

The recorded examples retain the package identity stated beside them. For a new installation, follow the current package instructions supplied with your release. To reproduce an older transcript exactly, use its named package and dependency profile. A historical version assertion is a reproduction check, not an instruction to change the identity of a newer installation.

0. Coverage contract

The coverage contract states what the guide covers and defines what "covered" means. The contract is tied to the shipped `komorebi-math==1.0.0b4` reference source and its recorded handoff. The handoff revision is distinct from the installed package version.

Coverage is a documentation property. It means that a public object has one canonical place in the guide, its aliases are visible, and its executable or verification routes are mapped. It does **not** mean that every mathematical statement is exact, that every optional backend is installed, or that a finite certificate proves a continuum theorem. Those are separate claims and receive separate evidence labels in Chapter 1.

0.1 Source sets

Komorebi Math exposes three Python API tiers and several non-Python surfaces. The stable tier is the frozen compatibility promise for the 1.x line. The beta tier is curated but still allowed to evolve before promotion. The research tier is the broad additive workspace where implemented finite constructions can appear without immediately becoming compatibility commitments. The command-line interface, examples, and verification suites are part of the user experience, so the guide treats them as first-class source sets rather than appendices.

Let

$$\mathcal{S}_{\text{stable}}, \quad \mathcal{S}_{\beta}, \quad \mathcal{S}_{\text{research}}, \quad \mathcal{C}_{\text{CLI}}, \quad \mathcal{E}_{\text{examples}}, \quad \mathcal{D}_{\text{docs}}, \quad \mathcal{V}_{\text{verification}}$$

denote the frozen stable API, curated beta API, broad research API, command-line interface, executable examples, documentation corpus, and verification/release interfaces.

The guide coverage target is

$$\mathcal{G} = \mathcal{S}_{\text{stable}} \cup \mathcal{S}_{\beta} \cup \mathcal{S}_{\text{research}} \cup \mathcal{C}_{\text{CLI}} \cup \mathcal{E}_{\text{examples}} \cup \mathcal{V}_{\text{verification}}.$$

Measured inventory for the audited reference-source tree:

Surface	Inventory
Stable manifest	131 entries: 58 classes, 61 functions, 12 data symbols
Beta manifest	83 entries: 45 classes, 28 functions, 10 data symbols
Research manifest	1,312 entries: 518 classes, 733 functions, 61 data symbols
Research manifest implementation owners	201: 196 package-owned and 5 standard-library owners
Python implementation modules	300
Executable Python examples	57
Executable PowerShell examples	8
Root PowerShell/shell workflow wrappers	42: 21 PowerShell and 21 shell wrappers
Documentation chapters	71
Verification modules	50
Installed verification-suite names	51
Frozen portable codecs	51 codecs and 1 adjacent migration
Export formats	11
Test modules	130

These numbers describe the recorded source baseline rather than every release of the project. Adding or removing a public symbol in a later release requires both a new manifest and a corresponding guide update. The documentation corpus $\mathcal{D}_{\text{docs}}$ informs the explanations, but it is not included in $\mathcal{G}$ as an independent public execution surface: prose cannot make an unimplemented interface real.

Outline-level placement validation:

$$\text{placed}(\text{manifest implementation owners}) = 201/201, \qquad \text{placed}(\text{shipped examples}) = 65/65.$$

0.2 Canonical implementation map

The same implementation can be re-exported through several supported namespaces. Explaining each re-export as though it were a different class would create duplicate prose that drifts over time. The canonicalization map therefore collapses aliases to the defining implementation while retaining every public import path.

Class and function aliases are mapped to one implementation explanation. Data exports remain keyed by exported name unless source declares an alias:

$$\pi(N, n, k, m, q, \sigma) = \begin{cases} (m, q), & k \in \{\text{class}, \text{function}\}, \\ (N, n), & k = \text{data}. \end{cases}$$

Here $\mathcal{A}$ is the set of exported records, N the namespace, n the exported name, k the manifest kind, $m.q$ the reflected implementation identity, and σ the reflected signature. Every stable exported name still receives its own Appendix A anchor. Deliberate callable aliases cross-link their shared explanation.

The data branch is essential. Several distinct constants reflect as `builtins.int`, `builtins.str`, or `pathlib.Path`, and Python may intern equal immutable values. Neither a shared type owner nor object identity makes those constants semantic aliases.

This prevents duplicate explanations for objects re-exported through `komorebi_math`, domain namespaces, `komorebi_math.beta`, `komorebi_math.stable`, `komorebi_math.v1`, and `komorebi_math.api.v1`.

For example, the stable compatibility namespaces export one shared `CoordinateTransform` object:

```
import komorebi_math.stable as stable
import komorebi_math.v1 as v1
import komorebi_math.api.v1 as api_v1

assert stable.CoordinateTransform is v1.CoordinateTransform
assert v1.CoordinateTransform is api_v1.CoordinateTransform
```

The guide therefore gives `CoordinateTransform` and `ExactCoordinateTransform` distinct export-name anchors, identifies their one shared implementation, and lists the three versioned routes on both cards. Alias identity is an API fact. It does not imply that unrelated names or domain concepts are mathematically interchangeable.

0.3 Coverage invariant

The anchor rule makes documentation drift mechanically detectable. A public manifest entry with no anchor is an omission. Two canonical anchors for one exported name are duplicate entries. Declared aliases that resolve to different implementations indicate a contract conflict that must be repaired before the guide can be declared complete.

For every manifest entry s, exactly one guide anchor must exist:

$$\forall s \in \mathcal{S}_{\text{stable}} \cup \mathcal{S}_{\beta} \cup \mathcal{S}_{\text{research}}, \qquad \#\,\text{Anchor}(s) = 1.$$

For every CLI command c and executable example e:

$$\#\,\mathrm{Recipe}(c) \geq 1, \qquad \#\,\mathrm{GuideMap}(e) \geq 1.$$

The source inventory places all 201 manifest owner strings (196 package-owned and five standard-library owners) and all 65 shipped executable examples. A separate machine-readable coverage ledger and digest remain `[PLANNED]` in this reference baseline. The inventory is an editorial record checked against that source rather than a generated release gate.

What this establishes. Every audited public surface has a destination, aliases have one canonical explanation, and omissions can be named.
What this does not establish. Placement alone does not prove an implementation correct, a numerical result exact, an optional backend usable, or a release qualified on hardware that was not tested.

0.4 Explicit exclusions

The exclusions keep the guide honest and usable. Private helpers are discussed only when a public failure or serialization contract cannot be understood without them. Historical APIs are retained for migration, but they do not compete with the current reference. Most importantly, truth status follows the executed finite path: no amount of documentation polish promotes a result to a stronger class.

- Private names beginning with _ are not reference entries unless needed to explain a public contract.
- Historical APIs remain in migration appendices rather than the main current reference.
- Planned, heuristic, conditional, numerical, and exact claims remain visibly distinct.
- The guide does not elevate finite certificates into unproved global or continuum theorems.

Chapter 0 summary. The object is the versioned public surface. The operation is canonical placement. The evidence is the manifest/module/example audit. The next chapter defines how every placed statement is labeled and read.

1. Editorial and symbolic grammar

Komorebi Math combines exact finite algebra, floating computation, conditional constructions, exploratory methods, and reserved interfaces. A single label such as "verified" is too vague. The evidence grammar below distinguishes what was computed, which hypotheses were checked, and what conclusion the evidence supports.

The labels are local. Read them beside the statement, example, or symbol they qualify. A class can be `[STABLE]` while one method is `[NUMERICAL]`. A `[RESEARCH]` function can still implement an `[EXACT]` finite identity. API stability and mathematical evidence answer different questions.

1.1 Entry labels

Label	Meaning
`[STABLE]`	frozen 1.x compatibility surface
`[BETA]`	curated beta surface
`[RESEARCH]`	additive research surface
`[EXACT]`	exact finite algebra/arithmetic under checked hypotheses
`[NUMERICAL]`	floating computation with residual/tolerance
`[CONDITIONAL]`	conclusion depends on listed unchecked assumptions
`[HEURISTIC]`	exploratory discovery or estimate
`[PLANNED]`	reserved but not production implemented
`[PORTABLE]`	supported by strict portable serialization
`[BACKEND]`	behavior depends on NumPy, SymPy, SciPy, JAX, PyTorch, or CuPy
`[CERT]`	returns or exposes a certificate/report

Several labels may appear together. For example, `[STABLE][EXACT][CERT][PORTABLE]` means that the import path is frozen, the executed finite construction is exact under its checked hypotheses, a machine-readable certificate is available, and a strict serialization route is supported. It says nothing about unchecked continuum assumptions unless a separate theorem supplies them.

1.2 Canonical symbol card

A symbol card is the smallest complete reference unit. It begins with the Python object the reader can import, but it does not stop at a signature. The mathematical type explains what the callable means. Hypotheses bound its domain. Invariants say what the implementation checks. Failures show where the contract rejects an input, and examples connect the abstract map to an executable path.

Every public class, function, method family, and data symbol receives this record:

1. **Identity**

- canonical qualified name
- public aliases
- stability tier
- source module

2. **Signature**

- exact Python signature
- type variables and accepted protocols

3. **Mathematical type**

- domain X
- codomain Y
- map $f : X \to Y$

4. **Definition**

- symbolic formula or finite algorithm

5. **Hypotheses**

- dimensions, fields/rings, positivity, invertibility, regularity, tolerance, resource budget

6. **Invariants**

- identities checked on construction or certification

7. **Exactness class**

- exact, numerical, conditional, heuristic, or planned

8. **Return contract**

- raw value, record, `ComputationResult`, `ConditionalResult`, `Certificate`, or domain report

9. **Failures**

- exception types and invalid inputs

10. **Persistence and export**

- portable tag/version, JSON shape, renderer support

11. **Backend behavior**

- required and optional dependencies

12. **Complexity and budgets**

- finite sizes, asymptotic cost, memory, guards

13. **Examples**

- minimal
- connected workflow
- adversarial/failure
- optional performance/backend example

14. **Related symbols**

- constructors, transforms, inverses, certificates, CLI commands, examples, tests

Not every field needs equal length, but none may be silently inapplicable. If an operation has no portable representation, the card says so. If its complexity has not been established, the card labels the estimate rather than replacing it with a benchmark. This is how the later reference can be exhaustive without pretending that every public object carries the same guarantee.

1.3 Canonical example card

An example is a finite experiment with an explicit claim boundary. The tuple below records not only the API calls and happy-path output, but also the hypotheses, certificate assertions, intentional failure, and reproducibility metadata needed to run it again.

Every example is specified as

$$\mathfrak{E} = (I, H, A, O, C, F, R),$$

where I is input data, H checked and unchecked hypotheses, A API calls, O expected output, C certificate assertions, F intentional failure/adversary, and R reproducibility metadata.

Required example levels:

- E_0: one-screen minimal example
- E_1: realistic connected example
- E_2: certificate inspection
- E_3: invalid/adversarial input
- E_4: backend or scaling variant when relevant

A reproducible rendered example names the package revision, interpreter, backend, tolerance or exact scalar domain, deterministic seed when one exists, input, expected output, and expected failure. Platform-dependent text such as a resolved executable path is described rather than frozen into an incorrect literal output.

1.4 Global mathematical conventions

- Scalars: $\mathbb{Z}, \mathbb{Q}, \mathbb{R}, \mathbb{C}, \mathbb{F}_p, \mathbb{Z}/p^k\mathbb{Z}$
- Vectors are columns unless stated otherwise.
- Matrices act on the left.
- A^* denotes conjugate transpose.
- I_X denotes the identity on X.
- P_i denotes a packet projector or coordinate selector only when the local chapter defines which.
- Signal convolution is signal-first.
- Rational/exact objects are not silently coerced into numerical certificates.
- Tolerances are explicit and never redefine exact equality.

Composition follows execution order. If $T_{B\leftarrow A}$ maps coordinates from A to B and $T_{C\leftarrow B}$ follows it, then

$$T_{C\leftarrow A} = T_{C\leftarrow B}T_{B\leftarrow A}.$$

The distinction between transpose and adjoint is never suppressed, and the word "projector" is reserved for an idempotent map in the local scalar domain. A redundant analysis-synthesis reconstruction component need not itself be an idempotent projector.

1.5 Canonical chapter specification

The chapter record is an authoring contract, not a second runtime API. It keeps prose, formulas, failures, examples, and complexity statements synchronized with one source-backed object set.

Every chapter is expanded as the record

$$\mathfrak{C} = (\Sigma, \mathcal{T}, \mathcal{M}, \mathcal{H}, \mathcal{I}, \mathcal{R}, \mathcal{F}, \mathcal{E}, \mathcal{Q}),$$

with

Field	Symbolic role
Σ	notation and public symbol set
$\mathcal{T}$	mathematical types, shapes, and scalar domains
$\mathcal{M}$	typed maps and defining algorithms
$\mathcal{H}$	preconditions and unverified hypotheses
$\mathcal{I}$	exact identities, invariants, and residual tests
$\mathcal{R}$	result and certificate schemas
$\mathcal{F}$	exceptions, rejected inputs, and conditional branches
$\mathcal{E}$	minimal, connected, certified, adversarial, and backend examples
$\mathcal{Q}$	complexity, memory, precision, and resource guards

The prose stage is therefore a rendering map

$$\text{RenderGuide} : \mathfrak{C} \longmapsto (\text{explanation}, \text{API cards}, \text{code}, \text{outputs}, \text{cross-links}),$$

not a second source of mathematical claims.

The rendering step may improve order and explanation, but it may not strengthen the evidence class, invent a parameter, or hide a rejected branch. When a code path and an older prose statement disagree, the current implementation and its committed contract are audited first. The guide does not choose the more impressive claim.

1.6 Exactness and evidence lattice

The five evidence classes enforce *non-promotion*. The order below is a documentation ranking from stronger finite evidence to weaker or future claims. It is not a theorem that one class automatically converts into another.

Claims occupy the source-backed proof-strength set

$$\mathcal{L} = \{\mathsf{P}, \mathsf{H}, \mathsf{C}, \mathsf{N}, \mathsf{E}\},$$

where

$$\mathsf{P} = \text{planned}, \quad \mathsf{H} = \text{heuristic}, \quad \mathsf{C} = \text{conditional}, \quad \mathsf{N} = \text{numerically certified}, \quad \mathsf{E} = \text{exactly certified}.$$

The guide's strongest-to-weakest editorial order is

$$\mathsf{E} \succ \mathsf{N} \succ \mathsf{C} \succ \mathsf{H} \succ \mathsf{P}.$$

The runtime `Exactness` `StrEnum` records categories but the enum itself implements no comparison or rank operation. The research expression layer separately provides `exactness_strength` and `weakest_exactness` for controlled certificate composition. Chapter 5 states that narrower rule explicitly. The editorial order is also distinct from the unordered `ClaimStatus` taxonomy used for provenance.

Class	Reader test
`exact`	Are all relevant identities decided in the stated finite exact domain?
`numerical`	Is a normed residual bounded by an explicit tolerance?
`conditional`	Which assumptions remain unchecked by the executed construction?
`heuristic`	What is being explored or estimated without a complete guarantee?
`planned`	Is this interface reserved rather than production implemented?

No implicit promotion is allowed. A promotion $a \rightsquigarrow b$ must cite a verification predicate V such that

$$V(x) = \mathsf{true} \Longrightarrow \mathrm{class}(x) = b.$$

In particular, numerical residual bounds do not imply exact equality, finite verification does not imply an infinite theorem, and a portable roundtrip does not prove mathematical validity.

> **Evidence boundary.** Decreasing a floating tolerance does not turn a numerical certificate into exact algebra. Running more finite cases does not prove an infinite statement. Serialization proves that a record survives its codec, not that the record's mathematical claim is true.

1.7 Traceability relations

Traceability answers "where did this statement come from?" The reference links a public symbol to its defining module, one guide anchor, runnable examples, tests, and relevant verification suites. Tests can share symbols and symbols can have many tests, but implementation identity and canonical guide placement are unique.

Let S be public symbols, M implementation modules, X executable examples, T tests, V verification suites, and D guide anchors, with $D_{\mathrm{API}} \subseteq D$ the API-card anchors. The guide records the relations

$$R_{SM} \subseteq S \times M, \quad R_{SD} \subseteq S \times D, \quad R_{SX} \subseteq S \times X, \quad R_{ST} \subseteq S \times T, \quad R_{SV} \subseteq S \times V.$$

Completion requires

$$\begin{aligned} \forall s \in S: &\quad \deg_{R_{SM}}(s) = 1, \quad \deg_{R_{SD}}(s) = 1, \\ \forall x \in X: &\quad \deg_{R_{SX}^{-1}}(x) \geq 1, \\ \forall d \in D_{\mathrm{API}}: &\quad \deg_{R_{SD}^{-1}}(d) \geq 1. \end{aligned}$$

1.8 Shape and domain grammar

Shapes are part of the contract. A matrix described only as "compatible" is not fully documented: the card states the dimension equations, scalar domain, orientation, axis, boundary convention, and batching rule before presenting the formula.

The guide uses the type grammar

$$\tau ::= R \mid R^n \mid R^{m \times n} \mid R[x]_{<n} \mid \mathrm{Seq}_N(R) \mid \mathrm{Signal}_{N \times C}(R) \mid \mathrm{Op}(X, Y) \mid \mathrm{Cert}(P),$$

with R an explicitly declared scalar domain. A callable is documented as

$$f : (x_1 : \tau_1, \ldots, x_k : \tau_k; \theta \in \Theta) \longrightarrow (y : \tau_y, c : \mathrm{Cert}(P)),$$

and every dimension equation is stated before the defining formula. Broadcast, axis, boundary, and batch conventions are part of Θ, never implicit.

1.9 Symbolic example ladder

The ladder keeps a friendly first success and an honest failure branch on the same page. Not every symbol needs a performance example, but every operation with meaningful rejection behavior receives an adversarial or invalid input.

For a public map $f : X \to Y$, the default five-example family is

$$
\begin{array}{lll}
E_0: & x_0 \in X, & y_0 = f(x_0), \\
E_1: & x_1 \in X_{\text{realistic}}, & y_1 = f(x_1), \\
E_2: & (y_2, c_2) = f_{\text{cert}}(x_2), & c_2.\texttt{valid} = \mathsf{true}, \quad c_2.\texttt{require}() = c_2, \\
E_3: & x_3 \notin \mathrm{Dom}(f), & f(x_3) \rightsquigarrow \mathrm{Reject}, \\
E_4: & x_4^{(b)} \in X_b, & f_b(x_4^{(b)}) \approx f_{b'}(x_4^{(b')}).
\end{array}
$$

The last comparison is exact only for exact backends. Otherwise the guide states the norm, tolerance, precision, seed, and device used.

1.10 Function, class, method, and data forms

These forms prevent stateful objects from being documented as if every method were a pure mathematical function. A mutation records both the returned value and the post-state invariant. Public data symbols record identity and mutability, because a shared registry constant and a newly constructed value have different compatibility consequences.

Each manifest kind has a fixed symbolic expansion.

For a function,

$$f_\theta : D_\theta \subseteq X_1 \times \cdots \times X_k \to Y, \qquad (x_1, \ldots, x_k) \mapsto y.$$

For a state-bearing class,

$$\mathcal{O} = (s, V_s, C_s),$$

where s is immutable or mutable state, V_s the construction invariant, and C_s the certificate factory. A pure method has type

$$m : \mathcal{O} \times X \to Y,$$

while a mutating method is shown as a transition

$$m : (\mathcal{O}_t, x) \longmapsto (\mathcal{O}_{t+1}, y), \qquad V_{s_t} \Longrightarrow V_{s_{t+1}}.$$

For a public data symbol a, the card records

$$a \in A, \qquad \mathrm{identity}(a), \qquad \mathrm{mutability}(a), \qquad \mathrm{serialization}(a).$$

1.11 Certificate normal form

The tuple below is the guide's comparison form. It lets certificate-producing families be discussed consistently, but it is not a universal recomputation performed by the `Certificate` data class. The producing API sets `valid` from its own domain rule, and the actual `.require()` implementation remains authoritative.

A certificate is documented through

$$\mathfrak{K} = (v, e, r, \tau, h_c, h_u, d, t, m),$$

where $v \in \{\mathsf{true}, \mathsf{false}\}$, e is the exactness class, $r = (r_i)$ residuals, τ an optional tolerance, h_c checked hypotheses, h_u unchecked assumptions, d diagnostics, t an optional theorem or contract identifier, and m the human-readable status.

Runtime enforcement is intentionally simple:

$$\mathrm{Require}(\mathfrak{K}) = \begin{cases} \mathfrak{K}, & v = \mathsf{true}, \\ \mathrm{ValidationError}, & v = \mathsf{false}. \end{cases}$$

The `Certificate` record does not independently derive `valid` from residuals. Residual and assumption consistency are separate guide-level audits of the producing API's documented exact, numerical, conditional, heuristic, or planned rule. The actual package semantics of `.require()` remain authoritative when a domain-specific certificate specializes this normal form.

A certificate should therefore be read in this order: `valid`, `exactness`, residuals and tolerance, checked assumptions, unchecked assumptions, theorem or contract identifier, diagnostics, and message. Calling `.require()` is the explicit transition from inspection to enforced acceptance.

1.12 Complexity and resource form

Complexity claims identify their evidence. An exact operation count, an asymptotic bound, a measured wall-clock time, and a heuristic estimate are not interchangeable. Resource guards are documented as behavior because rejecting a too-large construction can be part of the public safety contract.

Every nontrivial operation records a resource tuple

$$\mathfrak{Q} = (n_1, \ldots, n_k; T(\mathbf{n}), M(\mathbf{n}), P, B),$$

with input dimensions $\mathbf{n}$, time T, memory M, precision P, and enforced budget B. Claims are tagged as

$$\mathsf{proved\ bound}, \quad \mathsf{implementation\ count}, \quad \mathsf{measured\ benchmark}, \quad \mathsf{heuristic\ estimate}.$$

Hardware timing records never substitute for asymptotic or exact operation counts.

1.13 Persistence and migration form

Portable records are versioned mathematical data, not arbitrary object dumps. The guide records both the canonical encoder and the strict decoder, including unknown-field rejection, depth and size limits, digest scope, and the supported adjacent migration. A roundtrip is checked on the encoded finite record.

For every portable type X, document

$$\mathrm{encode}_v : X \to J_v, \qquad \mathrm{decode}_v : J_v \to X, \qquad \mathrm{decode}_v \circ \mathrm{encode}_v = I_X,$$

plus the adjacent migration

$$\mu_{v \to v+1} : J_v \to J_{v+1}.$$

The symbolic schema includes tag, version, payload grammar, canonical ordering, depth and size limits, digest scope, rejected unknown fields, and tamper cases.

1.14 Fully expanded symbol-card exemplar

The card below defines the format used throughout the function inventory. Notice that the certificate validates the transform record while invertibility remains a separate diagnostic. A singular square transform can be a valid record and still be impossible to invert.

`[STABLE][EXACT][CERT]`

$$\begin{aligned}\text{Name} &= \texttt{komorebi_math.stable.CoordinateTransform},\\ \text{Alias} &= \texttt{ExactCoordinateTransform},\\ \text{Impl} &= \texttt{komorebi_math.coordinates.transform.CoordinateTransform}.\end{aligned}$$

Constructor:

$$\text{CoordinateTransform}(s : \text{Str}, t : \text{Str}, T : \mathbb{Q}^{m\times n}, \vartheta : \text{Str}, \delta : \text{Str} = \texttt{""}) \to \mathcal{T}_{t\leftarrow s}.$$

State and action:

$$c_t = T_{t\leftarrow s} c_s, \qquad \text{apply} : \mathbb{Q}^n \to \mathbb{Q}^m.$$

For $m = n$ and $\det T \neq 0$,

$$\mathcal{T}_{s\leftarrow t} = \mathcal{T}_{t\leftarrow s}^{-1}, \qquad T^{-1}T = TT^{-1} = I_n.$$

Composition is ordered by execution:

$$\mathcal{T}_{t\leftarrow s}.\,\text{then}(\mathcal{T}_{u\leftarrow t}) = \mathcal{T}_{u\leftarrow s}, \qquad T_{u\leftarrow s} = T_{u\leftarrow t} T_{t\leftarrow s}.$$

Construction hypotheses and failures:

$$m, n \geq 1, \quad s \neq \texttt{""}, \quad t \neq \texttt{""};$$

ragged or empty matrices and dimension mismatches raise dimension errors, blank names or incompatible composition labels raise validation errors, and inversion of a rectangular or singular matrix is rejected.

Certificate normal form:

$$\mathfrak{K}_T = (\mathsf{true}, \mathsf{EXACT}, \emptyset, \emptyset, h_T, \emptyset, d, \vartheta, m_T),$$

where h_T records the finite rational matrix and declared dimensions, m_T reads "exact coordinate transform validated,'' and d records $(m, n, m = n, \det T, \text{invertible}(T))$. The certificate validates the transform record. Invertibility remains a separate diagnostic and is not implied for rectangular or singular transforms.

Symbolic example family:

$$T = \begin{bmatrix} 1 & \frac{1}{2} \\ 0 & 1 \end{bmatrix}, \quad c_s = \begin{bmatrix} 2 \\ 3 \end{bmatrix}, \quad c_t = Tc_s = \begin{bmatrix} \frac{7}{2} \\ 3 \end{bmatrix},$$

followed by exact inverse roundtrip, a three-system composition, a source-label composition failure, and a singular-matrix inversion failure. This is the model of detail required for every manifest symbol card.

> **Chapter 1 summary.** The object is a source-backed claim or public symbol. The operation is typed documentation. The evidence is its local label and certificate record. The next chapter shows how those rules organize the library without collapsing its domains into one meaning.

2. What Komorebi Math is

Komorebi Math is a typed Python library for executing finite packet mathematics across operator systems, coordinate transforms, refinement, arithmetic, signals, nonlinear constructions, exact integration, and expression calculus. Its unifying idea is not that these subjects have one meaning. It is that many of their finite computations can expose the same kinds of objects: generators, addresses, operators, packet decompositions, transfer blocks, refinements, constraints, certificates, and reconstructions.

The reference baseline combines a research API with a frozen stable compatibility surface. Use the stable namespace when import and signature continuity matter most. Use beta and research surfaces when the additional construction is worth a looser compatibility promise. In every tier, mathematical truth status is reported separately from API stability.

2.1 Finite execution language

The nine verbs below are an organizing vocabulary. A workflow may use only two or three of them. It does not need to instantiate a universal pipeline.

$$\text{seed} \to \text{address} \to \text{operate} \to \text{project} \to \text{transfer} \to \text{refine} \to \text{constrain} \to \text{complete} \to \text{reconstruct}.$$

Stage	Finite role	Implemented status in `STAGES`
seed	choose finite generating data	yes
address	name legal paths or coordinates	yes
operate	apply a realized operator	yes
project	resolve an object into packets	yes
transfer	expose source-to-target blocks	yes
refine	pass between compatible finite levels	yes
constrain	form defects and obstruction problems	yes
complete	construct compatible inverse limits in general	**no**
reconstruct	return to an ambient finite object	yes

The explicit `no` beside `complete` is a boundary, not a missing footnote. Some modules implement finite completion checks or domain-specific completion data, but `UnifiedSystem` does not claim to construct a general inverse limit.

2.2 What the shared language does and does not identify

The shared language makes composition boundaries inspectable. A packet coordinate object knows its originating system. A transfer block records source and target packets. A refinement requires compatible levels. A certificate says which finite identities were checked. These interfaces let workflows connect without erasing their native mathematics.

- shared finite interfaces
- domain-specific meanings
- exact versus numerical realizations
- finite versus continuum boundaries
- research APIs versus compatibility promises

The language does **not** identify a signal channel with an arithmetic residue, a quantum sector with a polynomial coordinate family, or a small residual with an exact zero. Nor does the presence of a finite algorithm assert convergence, regularity, or a continuum theorem that the implementation does not check.

What this establishes. Komorebi Math has a common finite execution vocabulary and typed cross-domain junctions.
What this does not establish. The domains are not collapsed into one semantics, and the common vocabulary is not a proof of a universal theory.

2.3 Reader pathways

The guide can be entered by task rather than read straight through. Begin with Chapters 0 through 4 and use Appendix A for exact stable-name lookup. Every domain path should include Chapter 5's certificate and result-type reference, Chapter 6's failure contract, the stability and backend boundaries in Chapters 7 and 8 that apply to its examples, and the Part III foundation that matches its finite objects: typed realizations in Chapter 9, generated actions in Chapter 10, packet blocks in Chapter 11, or resolution changes in Chapter 12.

- P_0: install and run a verified example
- P_1: exact algebra and arithmetic
- P_2: coordinates, refinement, and signal processing
- P_3: nonlinear, integration, and typed expressions
- P_4: quantum, deformation, and finite Langlands models
- P_5: extension, export, verification, and release maintenance

Path	Continue with	Typical profile
P_0 first execution	Chapters 3 through 5, then one domain chapter	core
P_1 exact algebra and arithmetic	Parts III, IV, V	core plus `exact` where named
P_2 coordinates, refinement, signals	Parts III, IV, VI	core. Add `scientific` selectively
P_3 nonlinear and expression workflows	Parts VII and VIII	core plus the backend named by the example
P_4 quantum, deformation, finite Langlands	Part IX	research tier. Read all claim boundaries
P_5 extension and release maintenance	Parts X and XII	`publication` or `dev`

Dense symbol inventories remain search-first reference material. The prose path teaches a concept once, then points to the canonical cards rather than repeating the same foundation in every domain.

2.4 `UnifiedSystem` and cross-domain composition boundaries

`UnifiedSystem` is a component-optional container for one finite construction. It can hold an operator grammar and realization, a seed system, a packet system, named ambient operators, a refinement tower, and metadata. `validate()` runs the consistency checks supported by the component combinations that are present. Named operators are dimension-checked when a packet system is available, metadata is descriptive, and absent or presently unchecked components contribute no failure. This makes partial systems useful, but it also means that a metadata-only instance returning a valid certificate has not validated absent mathematics.

Define a finite unified system by

$$\mathcal{U} = (\Gamma, \rho, \mathcal{S}, \mathcal{P}, \{A_\alpha\}_{\alpha\in\Lambda}, \mathcal{T}, \mu),$$

where Γ is an optional operator grammar, ρ its realization, $\mathcal{S}$ a seed system, $\mathcal{P}$ a packet system, $A_\alpha : X \to X$ named ambient operators, $\mathcal{T}$ an optional refinement tower, and μ metadata.

The validation predicate is the conjunction

$$V_{\mathcal{U}} = V_{\rho\vDash\Gamma} \wedge V_{\mathrm{hull}(\mathcal{S})} \wedge V_{\mathcal{P}} \wedge V_{\dim} \wedge V_{\{A_\alpha\}} \wedge V_{\mathcal{T}},$$

with absent components contributing the neutral truth value. Dimension compatibility requires

$$\dim \mathcal{S} = \dim \mathcal{P}, \qquad A_\alpha \in R^{d\times d}, \qquad \mathcal{P} \simeq \mathcal{T}_\ell \text{ for some level } \ell$$

whenever the corresponding components are present.

Public operations are typed as

$$\begin{aligned}
\text{validate} &: \mathcal{U} \to \text{Certificate},\\
\text{analyze} &: X \to \text{PacketCoordinates}(\mathcal{P}),\\
\text{block_operator}_\alpha &: A_\alpha \mapsto (P_i A_\alpha P_j)_{i,j},\\
\text{refine}_{r\to s} &: K_r \to K_s,\\
\text{coarsen}_{s\to r} &: K_s \to K_r,\\
\text{obstruction} &: (L, d) \mapsto [Lc = d].
\end{aligned}$$

The public methods expose those component requirements directly:

Method	Required component	Result
`validate()`	none. Checks whatever is present	`Certificate`
`analyze(x)`	packet system	`PacketCoordinates`
`block_operator(name)`	packet system and named operator	`PacketOperator`
`refine_coordinates(...)`	refinement tower	refined coordinates
`coarsen_coordinates(...)`	refinement tower	coarsened coordinates
`obstruction(L, d)`	correction map and defect	`LinearObstructionProblem`
`summary()`	none	finite dictionary summary

The guide must make the implemented completion boundary explicit:

$$\begin{aligned}
\mathsf{SEED} \to \mathsf{ADDRESS} \to \mathsf{OPERATE} \to \mathsf{PROJECT}\\
\to \mathsf{TRANSFER} \to \mathsf{REFINE} \to \mathsf{CONSTRAIN}\\
\to \mathsf{COMPLETE} \to \mathsf{RECONSTRUCT}.
\end{aligned}$$

where `COMPLETE` is a declared cross-domain stage but is not asserted as a general implemented inverse-limit construction. Missing packet or refinement components, unknown operators, inconsistent dimensions, and failed component certificates receive separate examples.

The shipped `examples/quickstart.py` builds a four-dimensional shift, a cyclic seed hull, even/odd packets, and the associated block operator. On the audited source it reports a valid **numerical** system, hull dimension four, and off-diagonal block norm $\sqrt{2}$. The NumPy backend determines the numerical label even though the displayed matrices contain only zeros and ones.

> **Chapter 2 summary.** The object is a finite typed system. The operation is composition through explicit stages. The evidence is component-level validation. The next chapter binds those operations to a specific interpreter and dependency profile.

3. Installation and environment profiles

Before you start. Choose whether you are installing a delivered package or working on a source checkout. Commands that mention `release/`, `constraints/`, `examples/`, or a repository wrapper require those files in the current working directory. An installed package does not by itself supply a working checkout. Run such commands from the matching source or distribution bundle and confirm that the named file exists before invoking it.

The literal wheel names and version assertions below reproduce the stated reference baseline. Keep the package, constraint file, and expected version from one release together. If you are using the current release, use its supplied installation instructions and artifact names. An optional profile is needed only when the selected example calls for it; the core first program is the simplest starting point.

A Komorebi Math result belongs to a specific interpreter and dependency profile. For that reason, this guide invokes pip through the same Python executable that will run the example. The recorded commands below assume that the reference handoff has been extracted and the shell is at the repository root.

The declared floor is Python 3.11. The recorded qualification matrix covers Python 3.11, 3.12, and 3.13. Python 3.13 matches the recorded Windows replay most closely. The bundled wheel is the canonical package artifact in the handoff. Installation is fully offline only when every required dependency is also available from a local cache or wheelhouse. A name-only `pip install komorebi-math` command assumes an external package-index publication and is therefore not assumed by these installation instructions.

3.1 Core installation

On Windows PowerShell, create a clean environment and retain its resolved interpreter path:

```
py -3.13 -m venv .venv
$py = (Resolve-Path .\.venv\Scripts\python.exe).Path

& $py -m pip install --upgrade pip
& $py -m pip install -c .\constraints\base-py313.txt `
    .\release\dist\komorebi_math-1.0.0b4-py3-none-any.whl

& $py -m pip check
& $py -c "import komorebi_math as km; assert km.__version__ == '1.0.0b4'; print(km.__version__)"
& $py -m komorebi_math info
```

The identity check must print `1.0.0b4`. `komorebi info` then prints the same package identity and the finite stage language. Use `& $py -m ...` rather than an unqualified `python`, `pip`, or `komorebi` command when several Conda or virtual environments are visible in the prompt.

The equivalent POSIX setup is:

```
python3.13 -m venv .venv
. .venv/bin/activate

python -m pip install --upgrade pip
python -m pip install \
  -c constraints/base-py313.txt \
  ./release/dist/komorebi_math-1.0.0b4-py3-none-any.whl

python -m pip check
python -c "import komorebi_math as km; assert km.__version__ == '1.0.0b4'; print(km.__version__)"
python -m komorebi_math info
```

Choose exactly one source-oriented alternative when developing the library. Editable installation keeps imports bound to the checked-out source:

```
& $py -m pip install -e ".[exact,scientific,yaml]"
```

Installing the source distribution exercises a clean PEP 517 rebuild before installation:

```
& $py -m pip install --force-reinstall `
    .\release\dist\komorebi_math-1.0.0b4.tar.gz
```

Do not combine those alternatives in one environment when diagnosing import provenance. A successful import from an editable checkout does not prove that the bundled wheel or source distribution was installed correctly.

3.2 Optional dependency lattice

The core wheel requires NumPy. Optional extras add one feature family at a time. They do not change the package version.

$$\forall x \in \{\text{exact}, \text{scientific}, \text{yaml}, \text{jax}, \text{torch}, \text{publication}\}, \qquad \mathcal{P}_{\text{core}} \subseteq \mathcal{P}_x.$$

Extra	Adds	Use it for
`exact`	SymPy	symbolic/exact backend paths that explicitly request SymPy
`scientific`	SciPy, mpmath	scientific routines and high-precision analytic paths
`plot`	Matplotlib	plotting examples
`yaml`	PyYAML	YAML export/import paths
`jax`	JAX	JAX directional-differentiation adapter
`torch`	PyTorch	PyTorch directional-differentiation adapter
`autodiff`	JAX and PyTorch	both autodiff adapters
`test`	pytest	source test execution
`docs`	MkDocs	documentation build
`build`	build, setuptools, wheel	distribution construction
`publication`	test, build, docs, exact, scientific, YAML	reference publication gate
`dev`	publication tools, Ruff, mypy	repository development
`all`	exact, scientific, plotting, YAML	broad mathematical extras only

Despite its name, `all` does not install JAX, PyTorch, CuPy, test tools, MkDocs, or build tools. CuPy is intentionally not a declared extra because its package must match the local CUDA line.

Append extras to the bundled wheel path before public package-index release:

```
& $py -m pip install `
    ".\release\dist\komorebi_math-1.0.0b4-py3-none-any.whl[exact,scientific,yaml]"
```

Many exact rational facilities, including the `Fraction`-based coordinate transform in Chapter 4, do **not** require SymPy. Install an extra because a named workflow needs it, not because the word “exact” appears in a theorem.

3.3 Windows, Linux, CPU, and CUDA profiles

The base CPU profile is the default and is enough for Chapters 0 through 4. A complete Windows publication replay should use the handoff's isolated launcher rather than adding publication dependencies to the reader's working environment:

```
$hostPython = if ($env:CONDA_PREFIX) {
    Join-Path $env:CONDA_PREFIX "python.exe"
} else {
    (Get-Command python.exe -ErrorAction Stop).Source
}

powershell.exe -NoProfile -ExecutionPolicy Bypass `
    -File .\Run_Block21_Windows_Qualification.ps1 `
    -Python $hostPython
```

The launcher creates a temporary virtual environment, installs the bundled wheel with the `publication` profile, runs `pip check`, requires MkDocs, and executes the publication gate. It removes the temporary environment after success. On failure it retains the environment and prints its location so the same interpreter can be inspected. Internet access, a configured package index, or a populated local wheel cache is required for dependencies not already present.

The `-Quick` switch omits the complete pytest run. It is a diagnostic preflight, not final publication qualification.

On POSIX systems the supplied runner uses the active environment and does not create one:

```
python3.13 -m venv .venv-publication
. .venv-publication/bin/activate
python -m pip install --upgrade pip
python -m pip install \
  './release/dist/komorebi_math-1.0.0b4-py3-none-any.whl[publication]'
python -m pip check

PYTHON="$(command -v python)" ./run_block21_publication_gate.sh
```

GPU qualification is separate from CPU publication qualification. Discovering `cupy` or an installed PyTorch build does not establish that the CUDA runtime, device, and directional primitives are usable. The GPU gate requires a compatible CUDA-specific PyTorch installation and exactly one matching CuPy distribution. It reports its own hardware evidence.

> **Environment boundary.** A green core example proves that example in the resolved interpreter. A green publication gate binds the wider finite release contract. Neither result silently qualifies unavailable hardware.

3.4 `Settings`, `configure`, `get_settings`, and `settings_context`

Runtime settings are immutable records stored in a Python context variable. The default record uses the NumPy backend name, tolerance `1e-10`, and construction validation. `get_settings()` returns the active record. `configure(...)` replaces selected fields in the current context and returns the new record. `settings_context(...)` is the safer choice for examples because it restores the exact previous context even when the body raises:

```
from komorebi_math import get_settings, settings_context

before = get_settings()
try:
    with settings_context(tolerance=1e-14) as scoped:
        assert scoped.tolerance == 1e-14
        raise RuntimeError("demonstrate restoration")
except RuntimeError:
    pass

assert get_settings() == before
print(before)
```

Expected default output:

```
Settings(backend='numpy', tolerance=1e-10, validate_on_construction=True)
```

The tolerance and construction-validation fields are consumed by core objects that consult them. The `backend` field is retained in the settings record, but `get_backend(None)` and core constructors default to NumPy and do not consult that field in this release. Pass `backend="sympy"` or another supported backend explicitly to the operation whose backend matters. `configure(backend="unknown")` stores the string. Rejection can occur later when `get_backend()` is actually called.

3.5 Optional-backend probe and failure isolation

Use the CLI for a human-readable environment report and the Python probe for a small programmatic record:

```
& $py -m komorebi_math doctor
```

```
from komorebi_math.backends import available_backends, get_backend

presence = available_backends()
print(presence)
numpy_backend = get_backend("numpy")
assert numpy_backend.name == "numpy"
```

`available_backends()` reports import presence for NumPy, SymPy, SciPy, mpmath, and CuPy. Presence is not the same as selection or qualification: NumPy is registered by default. SymPy is loaded lazily when its extra is installed. SciPy and mpmath are feature dependencies rather than general linear-algebra backend names, and CuPy presence alone neither registers a backend nor proves a working GPU. `komorebi doctor` similarly reports the environment without installing or certifying an optional package.

Symptom	Check	Recovery
wrong package version	print `sys.executable` and `komorebi_math.__version__`	invoke pip and the module through the resolved interpreter
`BackendUnavailableError`	inspect `available_backends()` and the requested operation	install the named extra or choose an available backend
import succeeds but dependencies conflict	run `python -m pip check`	rebuild a clean environment under the supplied constraint file
publication runner cannot build docs	confirm MkDocs is installed	use the `publication` profile. Do not relabel a skipped docs build as strict
Windows qualification fails	read the retained temporary-environment path	rerun diagnostics with that environment's `python.exe`

The smallest successful import is not a release gate, and the full release gate is not the right first diagnostic. Start with identity, `pip check`, and the smallest certificate relevant to the intended workflow. Escalate only after those pass.

> **Chapter 3 summary.** The object is a resolved Python environment. The operation is profile-specific installation. The evidence is identity, dependency consistency, and the relevant finite gate. The next chapter runs the first mathematical program in that environment.

4. First verified program

What success looks like. By the end of this chapter, you should be able to identify the imported package, reconstruct one numerical vector, perform one exact rational roundtrip, and explain the difference between the two certificates. Run Sections 4.3 and 4.4 unchanged first. Then change one input value at a time and inspect the resulting values and evidence. Section 4.5 is an intentional failure demonstration, so its rejected certificate is the expected outcome.

The first executable path moves from environment identity to two small mathematical checks. The mandatory path uses only the bundled core wheel and NumPy: it constructs a two-packet decomposition, inspects its certificate, analyzes a vector, and reconstructs it. A second path uses Python `Fraction` values to demonstrate a genuinely exact coordinate transform without requiring SymPy.

All code in this chapter was executed against the delivered `1.0.0b4` source. The output blocks omit only platform-dependent executable paths. Run examples with the resolved interpreter from Chapter 3.

4.1 Package identity and `komorebi info`

Begin by proving which package the interpreter imports:

```
& $py -c "import sys, komorebi_math as km; print(sys.executable); print(km.__version__)"
& $py -m komorebi_math info
```

After the local executable path, the version and CLI output are shown below, reflowed for print:

```
1.0.0b4
komorebi-math 1.0.0b4
language:
  seed -> address -> operate -> project -> transfer
  -> refine -> constrain -> complete -> reconstruct
implemented blocks:
  finite dimensional core through Block 17;
  production finite DSP;
  research-tier Blocks 18-20 for frame algebra,
  finite integration, and typed exact expressions
```

The final line is a release summary, not an assertion that every feature is in the frozen stable namespace. The package version is the authoritative identity. The handoff revision identifies the surrounding delivery package.

4.2 Capability discovery and `komorebi features`

The feature registry shows the declared status and scope of each domain:

```
& $py -m komorebi_math features
```

Representative rows include `algorithms implemented`, `coordinates implemented`, `core implemented`, `backends partial`, and `langlands.global partial`, each followed by a finite summary. Status values describe the static project registry. They do not report which optional package is installed in the current environment. Use `komorebi doctor` or `available_backends()` for that question.

Before relying on stable compatibility, the short installed checks are:

```
& $py -m komorebi_math api check --stable
& $py -m komorebi_math verify stable
```

The stable manifest contains 131 symbols. Its committed digest for this release is `23cd9e47a1663f2d4b3c0fef50f9f7d8c88d4af2f8bbc57e7ba76461e6e69276`.

4.3 Minimal packet roundtrip

Let

$$P_{\text{left}} = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}, \qquad P_{\text{right}} = \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}.$$

The projectors are idempotent, annihilate each other, and sum to the identity. For $x = (2, 3)^T$, analysis produces the embedded packet coordinates $(2, 0)^T$ and $(0, 3)^T$. Synthesis adds them back to x.

```python
import numpy as np

from komorebi_math.stable import Exactness, PacketSystem

packets = PacketSystem(
    [np.diag([1.0, 0.0]), np.diag([0.0, 1.0])],
    names=("left", "right"),
)

certificate = packets.certificate().require()
x = np.array([2.0, 3.0])
coordinates = packets.analyze(x)
reconstructed = coordinates.reconstruct()

np.testing.assert_array_equal(reconstructed, x)
assert certificate.exactness is Exactness.NUMERICAL

print("packets:", [value.real.tolist() for value in coordinates.values])
print("reconstructed:", reconstructed.real.tolist())
print("exactness:", certificate.exactness.value)
print("residuals:", certificate.residuals)
```

Expected output:

```
packets: [[2.0, 0.0], [0.0, 3.0]]
reconstructed: [2.0, 3.0]
exactness: numerical
residuals: {'idempotence': 0.0, 'annihilation': 0.0, 'completeness': 0.0}
```

`[NUMERICAL][CERT]` The residuals happen to be exactly representable zeros in this run, but the NumPy execution path remains numerical. The backend determines the certificate's exactness class. The tolerance determines whether numerical residuals are accepted. Visual simplicity of the matrix entries does not promote the result to exact algebra.

This example establishes the finite packet axioms and roundtrip for one input in the resolved environment. It does not establish a continuum decomposition or a backend-independent theorem beyond the implemented finite contract.

4.4 Minimal exact coordinate transform

The stable exact transform stores a finite rational matrix. Use `Fraction`, not a floating literal, to keep the scalar domain $\mathbb{Q}$:

$$T_{B\leftarrow A} = \begin{bmatrix} 1 & \frac{1}{2} \\ 0 & 1 \end{bmatrix}, \qquad T_{B\leftarrow A} \begin{bmatrix} 2 \\ 3 \end{bmatrix} = \begin{bmatrix} \frac{7}{2} \\ 3 \end{bmatrix}.$$

```python
from fractions import Fraction

from komorebi_math.stable import ExactCoordinateTransform, Exactness

Q = Fraction
transform = ExactCoordinateTransform(
    "A",
    "B",
    ((1, Q(1, 2)), (0, 1)),
    "GUIDE-CH4-EXACT-SHEAR",
    "exact rational shear",
)

vector = (Q(2), Q(3))
mapped = transform.apply(vector)
restored = transform.inverse().apply(mapped)
certificate = transform.certificate().require()

assert mapped == (Q(7, 2), Q(3))
assert restored == vector
assert certificate.exactness is Exactness.EXACT
assert transform.roundtrip_valid()

print("mapped:", mapped)
print("restored:", restored)
print("determinant:", transform.determinant)
print("exactness:", certificate.exactness.value)
```

Expected output:

```
mapped: (Fraction(7, 2), Fraction(3, 1))
restored: (Fraction(2, 1), Fraction(3, 1))
determinant: 1
exactness: exact
```

`[EXACT][CERT]` The inverse roundtrip is decided by rational matrix arithmetic. The transform certificate validates the finite record and reports determinant and invertibility diagnostics. Certificate validity alone is not an invertibility theorem for every record: a singular square transform can be a well-formed valid record with the `invertible` diagnostic set to `False`.

4.5 Minimal certificate inspection

A certificate can be inspected before acceptance is enforced with `.require()`. To demonstrate that distinction, intentionally construct an invalid packet family with automatic construction validation disabled. The second projector has eigenvalue $1/2$, so it is neither idempotent nor part of a complete decomposition:

```
import numpy as np

from komorebi_math.stable import PacketSystem, ValidationError

bad = PacketSystem(
    [np.diag([1.0, 0.0]), np.diag([0.0, 0.5])],
    names=("left", "right"),
    validate=False,
)

failed = bad.certificate()
print("valid:", failed.valid)
print("exactness:", failed.exactness.value)
print("residuals:", failed.residuals)
print("message:", failed.message)

try:
    failed.require()
except ValidationError as error:
    print(type(error).__name__ + ":", error)
```

Expected output:

```
valid: False
exactness: numerical
residuals: {'idempotence': 0.25, 'annihilation': 0.0, 'completeness': 0.5}
message: projectors do not form a complete packet system
ValidationError: projectors do not form a complete packet system
```

The `validate=False` argument is appropriate here only because the example's purpose is to inspect the failed record. Under the default settings, the constructor calls `.require()` and rejects the invalid family immediately. In ordinary application code, disabling validation should be an explicit, auditable decision rather than a way to suppress an inconvenient certificate.

For any domain-specific certificate, inspect the fields in this order: `valid`, `exactness`, residuals and tolerance, checked assumptions, unchecked assumptions, theorem or contract identifier, diagnostics, and message. Then call `.require()` when the next operation is permitted only on acceptance.

4.6 Public capability tour

The two examples above are intentionally small. The shipped deterministic tour is the next connected check:

```
& $py .\examples\block21_public_capability_tour.py --json
```

When reviewing the source tree rather than the installed wheel, bind imports to that tree first:

```
$env:PYTHONPATH = Join-Path (Get-Location) "src"
& $py .\examples\block21_public_capability_tour.py --json
```

The finite tour exercises exact polyphase refinement, redundant analysis-synthesis, Euler-to-Fibonacci coordinates, exact finite integration, a rejected false certificate, typed expressions, portable roundtrips, and the frozen 131-symbol stable contract. For the committed inputs its capability record digest is:

```
0e20e32f4ad9004d5248d258813b6acfec7b9e96cc30c001aa6865aba9039fdf
```

`[EXACT][NUMERICAL][CERT]` applies per sub-result, not to the tour as one undifferentiated theorem. The digest binds a deterministic structured record. It is release reproducibility evidence. It does not prove all behavior in all domains or on all backends.

For full Windows qualification, use the isolated launcher from Chapter 3. It adds dependency consistency, strict documentation, API and release contracts, the complete source test suite, and the tour. The guide treats the output as an environment-stamped execution record, not a timeless test-count claim.

> **Chapter 4 summary.** Inspect each finite packet or coordinate result and its certificate. Use the asserted roundtrip as a check on the example. Chapter 5 explains the result and certificate types.

> **Part I execution evidence.** The Chapter 0–4 Python examples were executed against the recorded reference source. Installation and qualification commands were checked against its paths and scripts. Execution claims retain their recorded environment scope.

Part II. Trust, types, and execution semantics

Use this part when deciding whether a result is usable. Start with Chapter 5 when an operation returns a certificate or result wrapper. Use Chapter 6 when it raises an exception or reports a failed check. Chapter 7 explains which imports carry a compatibility promise. Chapter 8 explains which arithmetic and optional dependencies a computation actually uses.

A short review of any result follows four questions:

1. Did the operation accept the input and complete its declared check?
2. Was the executed arithmetic exact, numerical, conditional, or heuristic?
3. Which assumptions and tolerances limit the conclusion?
4. Does the next operation require an accepted certificate, a particular scalar domain, or a stronger hypothesis?

Keep the result and its certificate together while answering these questions. A number alone usually does not preserve enough context to explain how it was obtained. The examples in this part show both accepted and rejected records so that a failure can be recognized and diagnosed without guessing.

5. Certificates and result types

Komorebi Math does not treat a returned value as the whole result of a mathematical computation. Many public operations also return evidence about what was checked, how it was checked, and what remains assumed. The common record for that evidence is `Certificate`. `ComputationResult[T]` and `ConditionalResult[T]` pair such a record with a value.

These types enforce a small runtime contract that is conservative by design. They make evidence visible and provide a standard failure boundary, but they do not re-prove the mathematics recorded by a producing API. In particular, a `Certificate` stores the producer's Boolean `valid`. It does not recompute that Boolean from the residuals, tolerance, or assumptions supplied beside it.

5.1 `Exactness`: classify the executed evidence

`Exactness` is a string-valued enumeration with five members:

Member	Stored value	Reader interpretation
`Exactness.EXACT`	`"exact"`	Finite identities were decided in the declared exact domain.
`Exactness.NUMERICAL`	`"numerical"`	A finite numerical check used an explicit error rule.
`Exactness.CONDITIONAL`	`"conditional"`	The conclusion retains named unchecked assumptions.
`Exactness.HEURISTIC`	`"heuristic"`	The result is exploratory or estimated.
`Exactness.PLANNED`	`"planned"`	The interface or claim is reserved, not established.

The enum member records the path that actually ran. It is not selected by the user's desired conclusion. A zero residual from a NumPy computation remains `NUMERICAL`. Zero floating-point output does not turn that computation into exact algebra. Conversely, an exact finite computation may be invalid because an exact identity failed.

The enum itself defines no ordering methods. The research namespace exposes an explicit ordering for evidence composition:

```
exactness_strength(value: Exactness) -> int
weakest_exactness(*values: Exactness) -> Exactness
```

The implemented ranks are

$$\operatorname{rank}(\text{exact}) = 0, \quad \operatorname{rank}(\text{numerical}) = 1, \quad \operatorname{rank}(\text{conditional}) = 2, \quad \operatorname{rank}(\text{heuristic}) = 3, \quad \operatorname{rank}(\text{planned}) = 4.$$

A larger rank denotes a weaker guarantee. `weakest_exactness()` returns the largest supplied rank, with `EXACT` as the empty meet. Both helpers require real `Exactness` members. They reject raw strings. These helpers belong to the research root API rather than the frozen stable namespace, so Chapter 7's API tier rules apply to them.

Exactness and validity answer different questions:

`exactness` : What kind of evidence was produced?
`valid` : Did the producer's declared finite check pass?

Neither answer determines the other. An invalid exact certificate and a valid numerical certificate are both ordinary states.

5.2 `Certificate`: one machine-readable evidence record

The frozen stable constructor is

```
Certificate(
    valid: bool,
    exactness: Exactness,
    residuals: dict[str, float] = <factory>,
    tolerance: float | None = None,
    assumptions_checked: tuple[str, ...] = (),
    assumptions_unchecked: tuple[str, ...] = (),
    diagnostics: dict[str, Any] = <factory>,
    theorem_id: str | None = None,
    message: str = "",
) -> None
```

In the guide's comparison notation, write

$$C = (v, e, R, \tau, H_{\checkmark}, H_{?}, D, t, m).$$

Field	Declared type	Meaning
`valid`	`bool`	Producer-supplied outcome of its declared check.
`exactness`	`Exactness`	Evidence class of the executed path.
`residuals`	`dict[str, float]`	Named finite discrepancies or counts.
`tolerance`	`float | None`	Numerical comparison scale, when applicable.
`assumptions_checked`	`tuple[str, ...]`	Conditions established by the finite execution.
`assumptions_unchecked`	`tuple[str, ...]`	Conditions retained rather than established.
`diagnostics`	`dict[str, Any]`	Explanatory backend, size, method, or audit data.
`theorem_id`	`str | None`	Optional theorem, contract, or operation label.
`message`	`str`	Human-readable success or failure description.

The declared schema is informative, but the base constructor does not validate or coerce every field. A producing API is responsible for constructing a coherent record. Direct callers should therefore use `Exactness` members, finite float residuals, a nonnegative numerical tolerance when one is needed, and nonempty assumption labels. Passing the raw string `"exact"` instead of `Exactness.EXACT`, for example, creates a malformed record whose later methods need not behave correctly.

For an exact identity check, a typical producer implements

$$v \iff \bigwedge_i R_i = 0, \qquad e = \text{exact}, \qquad \tau = \emptyset.$$

For a numerical check, the rule is producer-specific and should be documented beside the operation. A common form is

$$v \iff \bigwedge_i R_i \le b_i(\tau, \text{scale}), \qquad e = \text{numerical}.$$

There is no universal norm or scaling convention hidden in `Certificate`. Readers must inspect the producing function's contract before interpreting a residual. A count such as `failed_checks` and a norm such as `roundtrip` are both stored as floats, but they do not have the same mathematical units.

The data class is frozen and slot-based: fields cannot be rebound. That immutability is shallow. The `residuals` and `diagnostics` dictionaries remain mutable objects, so code that needs an archival record should avoid modifying them after construction and should serialize the final certificate promptly.

5.3 `ComputationResult[T]` and `ConditionalResult[T]`

`ComputationResult[T]` pairs an arbitrary value with one certificate:

```
ComputationResult(value: T, certificate: Certificate) -> None
```

Its convenience interface is exactly

```
result.valid          # result.certificate.valid
result.exact          # result.certificate.exactness is Exactness.EXACT
result.require()      # require the certificate, then return result.value
```

Symbolically,

$$\text{RequireResult}(x, C) = \begin{cases} x, & C.\text{valid is true}, \\ \text{ValidationError}, & C.\text{valid is false}. \end{cases}$$

The `exact` property classifies evidence only. It does not include a validity test, so `result.exact` can be true while `result.valid` is false. Acceptance code should call `result.require()` rather than use `result.exact` as a proxy for success. The raw `value` also remains directly accessible on an invalid result. The wrapper cannot prevent a caller from bypassing its evidence.

`ConditionalResult[T]` has the same two stored fields:

```
ConditionalResult(value: T, certificate: Certificate) -> None
```

Its constructor enforces one invariant:

$$C.\text{exactness} \textbf{ is } \texttt{Exactness.CONDITIONAL}.$$

A nonconditional certificate raises `ValueError("ConditionalResult requires a conditional certificate")`. Nothing else is inferred. The constructor does not require `valid=True`, does not require a nonempty `assumptions_unchecked`, and does not discharge any assumption. Unlike `ComputationResult`, `ConditionalResult` has no convenience `.valid`, `.exact`, or `.require()` member. Inspect and, when appropriate, enforce `result.certificate` explicitly.

Conditional producers use `valid` in more than one legitimate way. A symbolic plan may be internally well formed and therefore valid while still listing external hypotheses. A completion gate may instead remain invalid until all named inputs have been supplied. The message, checked assumptions, and unchecked assumptions determine which interpretation applies.

5.4 `.require()`, residuals, diagnostics, and serialization

`Certificate.require()` is intentionally small:

```
Certificate.require(self) -> Certificate
```

It returns the same object when `valid` is truthy. When `valid` is false, it raises `ValidationError` with `message`. An empty message uses the fallback `"certificate validation failed"`. It does not compare residuals with the tolerance, inspect assumptions, interpret diagnostics, or resolve the theorem identifier. Those checks must already have been performed by the producer.

Read a certificate in this order:

1. Inspect `valid` and `exactness` together.
2. Identify the formula and norm represented by every residual.
3. Read the tolerance only under that producer's numerical rule.
4. Separate checked conditions from unchecked assumptions.
5. Treat `theorem_id` as a label, not as proof by itself.
6. Use diagnostics to explain the run, never to override `valid`.
7. Call `.require()` at the boundary where invalid evidence must stop work.

The presentation serializers are

```
Certificate.to_dict(self) -> dict[str, Any]
Certificate.to_json(self, *, indent: int = 2) -> str
```

`to_dict()` recursively converts the data-class fields and replaces the enum object by its string value. `to_json()` sorts keys and uses `default=str` for objects that ordinary JSON cannot encode. It is useful for reports, but that fallback can erase type information. It should not be described as a typed roundtrip format.

The portable persistence layer separately registers `Certificate` under codec tag `core.certificate`, version 1. `dumps_portable()` places the encoded record in a canonical, SHA-256-checked envelope. `loads_portable()` validates that envelope before reconstruction. The digest detects an unchanged-versus-tampered payload, not authorship or a trusted signature. Direct portable serialization of `ComputationResult` or `ConditionalResult` is not implemented. Only the certificate itself has this built-in codec. The generic `to_serializable()` presentation export for a result is a different facility.

Worked example: a numerical packet certificate

This example was executed against the recorded reference source. It uses NumPy, so a zero observed residual remains numerical evidence.

```
import numpy as np

from komorebi_math import (
    Certificate,
    ComputationResult,
    Exactness,
    PacketSystem,
    describe,
)
from komorebi_math.stable import ValidationError

system = PacketSystem(
    [np.diag([1, 0]), np.diag([0, 1])],
    names=("left", "right"),
    tolerance=1e-12,
)
certificate = system.certificate()
coordinates = system.analyze(np.array([2.0, 3.0]))
```

```
result = ComputationResult(coordinates.reconstruct(), certificate)

record = describe(PacketSystem)
assert record is not None

print(result.valid, certificate.exactness.value)
print(result.exact, np.allclose(result.require(), [2.0, 3.0]))
print(certificate.residuals)
print(certificate.tolerance)
print(certificate.assumptions_checked)
print(certificate.assumptions_unchecked)
print(certificate.diagnostics)
print(record.status.value, record.identifier == certificate.theorem_id)
```

The same example adds a controlled failure branch:

```


failed = ComputationResult(
    "untrusted value",
    Certificate(
        False,
        Exactness.NUMERICAL,
        residuals={"roundtrip": 1e-4},
        tolerance=1e-12,
        message="roundtrip residual exceeds tolerance",
    ),
)

try:
    failed.require()
except ValidationError as error:
    print(f"{type(error).__name__}: {error}")
```

Expected output:

```
True numerical
False True
{'idempotence': 0.0, 'annihilation': 0.0, 'completeness': 0.0}
1e-12
('common dimension', 'idempotence', 'annihilation', 'completeness')
()
{'backend': 'numpy', 'ambient_dimension': 2, 'packet_count': 2}
proved True
ValidationError: roundtrip residual exceeds tolerance
```

The first Boolean pair says that the result passed but is not exact. The roundtrip assertion is a separate check of the value. The last line shows the enforcement boundary: the invalid wrapper stores a value, but `.require()` does not release it.

5.5 Certificate composition without evidence promotion

The research expression layer provides a source-backed rule for combining certificates:

```
compose_certificates(
    certificates: Iterable[Certificate],
    *,
    operation: str | None = None,
    theorem_id: str | None = None,
    assumptions_checked: Iterable[str] = (),
    assumptions_unchecked: Iterable[str] = (),
    diagnostics: Mapping[str, Any] | None = None,
) -> Certificate
```

For children $C_0, \dots, C_{n-1}$, it implements

$$v_{\text{out}} = \bigwedge_i v_i, \qquad e_{\text{out}} = max_{\text{rank}}\{e_0, \dots, e_{n-1}\}.$$

If any unchecked assumption remains and the computed class is stronger than `CONDITIONAL`, the output is weakened to `CONDITIONAL`. Assumptions are deduplicated in encounter order, and unchecked status wins when the same label appears in both sets. Residual keys receive deterministic `child0:`, `child1:`, and later prefixes. The aggregate tolerance is the maximum non-`None` child tolerance. This is summary metadata, not permission to compare every residual against one universal bound. Child and local diagnostics occupy separate nested namespaces.

The outer theorem identifier is whatever the caller supplies. Composition records child theorem identifiers in diagnostics but does not silently promote one child identifier to the theorem for the whole pipeline.

`exact_internal_certificate()` is a related research helper:

```
exact_internal_certificate(
    operation: str,
    *,
    theorem_id: str | None = None,
    assumptions_checked: Iterable[str] = (),
    diagnostics: Mapping[str, Any] | None = None,
) -> Certificate
```

It constructs valid exact evidence for an operation declared exact by construction. The helper validates its metadata, not the mathematical operation named by the caller. It should therefore be used only inside an implementation whose exact finite rule has already been established.

Worked example: compose, then retain the condition

```
from komorebi_math import compose_certificates
from komorebi_math.stable import (
    Certificate,
    ConditionalResult,
    Exactness,
)

exact = Certificate(
    True,
    Exactness.EXACT,
    residuals={"identity": 0.0},
    assumptions_checked=("finite input", "shared premise"),
    theorem_id="GUIDE.CH5.A",
)
numerical = Certificate(
    True,
```

```
    Exactness.NUMERICAL,
    residuals={"roundtrip": 3e-13},
    tolerance=1e-12,
    assumptions_checked=("bounded input",),
    theorem_id="GUIDE.CH5.B",
)

combined = compose_certificates(
    (exact, numerical),
    operation="demo pipeline",
    theorem_id="GUIDE.CH5.COMPOSE",
    assumptions_unchecked=("shared premise", "external convergence"),
)
conditional = ConditionalResult("pipeline value", combined)

print(conditional.value)
print(combined.valid, combined.exactness.value, combined.tolerance)
print(combined.residuals)
print(combined.assumptions_checked)
print(combined.assumptions_unchecked)
print(combined.diagnostics["composition"]["child_theorem_ids"])
print(combined.require() is combined)

try:
    ConditionalResult(0, exact)
except ValueError as error:
    print(f"{type(error).__name__}: {error}")
```

Expected output:

```
pipeline value
True conditional 1e-12
{'child0:identity': 0.0, 'child1:roundtrip': 3e-13}
('finite input', 'bounded input')
('shared premise', 'external convergence')
('GUIDE.CH5.A', 'GUIDE.CH5.B')
True
ValueError: ConditionalResult requires a conditional certificate
```

The numerical child prevents an exact output, and the unchecked assumptions weaken the composition further to conditional. Although the combined finite record is valid, `external convergence` remains an explicit premise.

5.6 Theorem identifiers and mathematical provenance

`Certificate.theorem_id` is an optional string. In practice it may name a paper theorem, a finite contract, a compatibility check, or an internal operation. The base record does not require it to be present or nonempty, and `.require()` does not look it up. A matching identifier is useful audit evidence only when the producing API and its documentation establish the link.

The research namespace supplies a separate provenance taxonomy:

```
ClaimStatus(*values)

ProvenanceRecord(
    identifier: str,
    title: str,
    status: ClaimStatus,
    source: str,
    assumptions: tuple[str, ...] = (),
    limitations: tuple[str, ...] = (),
```

```
    verification_suite: str | None = None,
) -> None

describe(obj: Any) -> ProvenanceRecord | None
```

`ClaimStatus` contains `definition`, `proved`, `classical input`, `conditional theorem`, `finite certificate`, `heuristic`, `conjecture`, and `planned`. It classifies the documented mathematical source claim. It is not an alternative spelling of execution `Exactness`.

`describe()` accepts an object or a qualified object name. It returns the attached `ProvenanceRecord`, or `None` when no record can be resolved. The CLI uses the same mechanism:

```
komorebi describe komorebi_math.core.packets.PacketSystem
```

For `PacketSystem`, the record identifies `PSC-I-T20`, status `proved`, source `Paper I`, its finite-family assumption, and verification suite `core.packets`. The worked example nevertheless obtains a numerical certificate because it executed through NumPy. This is not a contradiction: the provenance record classifies the documented theorem, while the certificate classifies one concrete run.

The module `komorebi_math.provenance` also contains `documented()`, `register_record()`, and `records()` for package authors and extensions. They are not frozen stable/root reader APIs. The registry is process-local and populated by imports. `records()` should not be presented as a complete static catalog. Registration can replace an existing key, so extension authors must manage identifier collisions themselves.

5.7 Cautions and defining failure cases

- A valid certificate is evidence under its producer's contract, not a universal proof object.
- `valid=True` does not imply exactness, and exactness does not imply validity.
- A zero numerical residual does not promote a computation to exact.
- `.require()` trusts `valid`. It does not independently audit the other fields.
- `ConditionalResult` enforces the label `CONDITIONAL`, not the truth of its assumptions and not the validity of its certificate.
- Unchecked assumptions remain visible even when a conditional certificate is valid.
- Diagnostics explain a run but cannot overturn its Boolean outcome.
- Theorem identifiers and provenance identifiers are not automatically reconciled.
- The frozen data classes contain mutable dictionaries. Freezing is shallow.
- Presentation JSON may stringify diagnostic types. Use the portable codec when a supported typed certificate roundtrip is required.
- A portable SHA-256 digest detects payload inconsistency. It is not a digital signature or a statement of authorship.

5.8 What the shipped tests establish

The certificate contract is exercised at several levels. The focused registry tests assert that a valid certificate returns itself from `.require()`, that JSON contains the string `"exact"`, and that an invalid certificate propagates its message through `ValidationError`. They also reject construction of a `ConditionalResult` from an exact certificate and resolve the `PacketSystem` provenance identifier `PSC-I-T20`.

Export tests show that presentation serialization of a `ComputationResult` retains its nested certificate and theorem identifier. Persistence tests carry certificates inside versioned portable objects, serialize them, reconstruct them, and demand byte-stable re-encoding. The expression-calculus suite then checks the evidence-composition rule at a semantic level: numerical input evidence remains numerical, an unchecked premise imposes conditional status, and invalid child evidence remains invalid. The adversarial verifier repeats those non-promotion checks outside the ordinary unit-test path.

These tests establish the implemented finite contracts. They do not turn an arbitrary hand-written certificate into independently verified evidence. That distinction explains why examples should normally display a certificate re-

turned by a domain operation, then use manually constructed certificates only to demonstrate a controlled failure or composition rule.

> **Chapter 5 summary.** `Exactness` says what kind of finite evidence was produced. `valid` says whether the producer's declared check passed. Residuals, tolerances, and assumptions explain that decision. `ComputationResult.require()` releases a value only after the certificate passes. `ConditionalResult` preserves an explicitly conditional conclusion. Certificate composition may retain or weaken evidence but never strengthen it, and provenance records document source claims without replacing execution certificates.

The rejected result in this chapter still contained inspectable data. Chapter 6 now distinguishes that situation from an exception, where no admissible result record has crossed the public boundary.

6. Errors, assumptions, and resource guards

Komorebi Math distinguishes several kinds of failure because they call for different responses. A malformed object should be corrected. A dimension mismatch should be traced to the map or coordinate space that introduced it. A failed mathematical assumption may require a different theorem or algorithm. A missing backend is an environment problem. A resource-limit failure asks the caller to reduce the problem or explicitly authorize a larger finite budget. An invalid certificate, meanwhile, is evidence returned for inspection rather than necessarily an exception raised at the point of computation.

The central operational rule is:

> Catch only the failures that the current layer can resolve. Preserve the original type and message when passing an unresolved failure upward.

The public contract distinguishes exceptions from invalid certificates, separates checked assumptions from unchecked ones, and states the resource limits used by finite computations. The examples were executed against the delivered `1.0.0b4` source.

6.1 Public exception hierarchy

The canonical definitions live in `komorebi_math.exceptions`. Seven classes are also members of the frozen stable API:

```
from komorebi_math.stable import (
    AssumptionError,
    BackendUnavailableError,
    DimensionError,
    ExactnessError,
    KomorebiMathError,
    ResourceLimitError,
    ValidationError,
)
```

Two additional package exceptions are available from the canonical module but are not frozen stable symbols:

```
from komorebi_math.exceptions import (
    FeatureNotImplementedError,
    RelationError,
)
```

The top-level `komorebi_math` research namespace does not export the exception names. Import from `komorebi_math.stable` when the frozen contract is required, or from `komorebi_math.exceptions` when the complete canonical hierarchy is required.

Let E_K denote `KomorebiMathError`. Let $E_V, E_D, E_R, E_A, E_B, E_X, E_L$ denote validation, dimension, relation, assumption, backend, exactness, and resource-limit errors, and let E_P denote the planned-feature error. With $\preceq$ meaning "is a subclass of," the hierarchy is

$$E_D, E_R, E_L \preceq E_V \preceq E_K, \qquad E_V \preceq \texttt{ValueError},$$

$$E_A, E_B, E_X \preceq E_K, \qquad E_P \preceq E_K, \qquad E_P \preceq \texttt{NotImplementedError}.$$

The classes have these responsibilities.

Class	Meaning	Typical caller response
`KomorebiMathError`	Base of package-defined public errors	Log context and re-raise unless the boundary owns recovery
`ValidationError`	A declared validation rule failed	Correct the value, record, or parameter
`DimensionError`	Matrix, vector, packet, or binding dimensions disagree	Inspect source and target spaces or array shapes
`RelationError`	A typed grammar relation is malformed or violated	Correct endpoints, relation terms, or realization
`AssumptionError`	A required finite mathematical assumption is false	Choose a compatible theorem, transform, or algorithm
`BackendUnavailableError`	A requested optional backend is unknown or unavailable	Install the dependency or explicitly select another backend
`ExactnessError`	An exact operation was requested across an inexact boundary	Select an exact domain or accept a documented numerical path
`ResourceLimitError`	A valid explicit resource contract would be exceeded	Reduce the request or supply a reviewed larger budget
`FeatureNotImplementedError`	A declared feature is still a placeholder	Use implemented functionality. Do not emulate a result silently

The exception classes do not declare custom constructors. They inherit ordinary Python exception arguments, and the stable signature manifest records no package-specific constructor signature. Code should rely on their types and documented meanings, not parse message text as a machine protocol.

6.2 Catch policy

The inheritance structure supports progressively broader handling. Catching `DimensionError` isolates a shape problem. Catching `ValidationError` also catches `DimensionError`, `RelationError`, and `ResourceLimitError`. Because `ValidationError` is also a `ValueError`, a conventional Python value-error boundary catches those four package types. Catching `KomorebiMathError` catches every exception defined in the package hierarchy, including `FeatureNotImplementedError`.

A narrow application boundary can therefore use:

```
from komorebi_math.stable import DimensionError, ResourceLimitError

try:
    result = run_finite_problem()
except DimensionError as error:
    report_shape_problem(error)
except ResourceLimitError as error:
    request_a_smaller_problem(error)
```

A command-line front end may instead catch `KomorebiMathError` once, add the operation name and user-facing guidance, and exit. Library code should generally avoid a broad catch followed by an unrelated default result. In particular, a backend failure must not be converted silently into execution on a different backend: backend selection can change scalar domains, exactness, performance, and available operations.

`KomorebiMathError` is not a total boundary around every possible Python failure. Some APIs intentionally expose ordinary built-ins. A non-expression passed to expression preflight raises `TypeError`. A missing packet coordinate name raises `KeyError`. `OperatorGrammar.paths(max_length=-1)` raises `ValueError`, and a `ConditionalResult` paired with

a nonconditional certificate raises `ValueError`. Catch a built-in only where the surrounding Python operation gives it a well-defined recovery meaning.

6.3 Validation, dimensions, and relations

`ValidationError` covers a failed declared rule that is not better described by a more specific subclass. Examples include an empty packet family, duplicate packet names, a nonpositive budget field, an invalid exact scalar, or a noncanonical portable record. It answers the question "is this admissible input or a valid finite record?"

`DimensionError` specializes that answer when spaces or shapes disagree. A `PacketSystem` requires at least one square projector, all projectors must have one common size, and analyzed vectors must have the ambient dimension. Expression bindings, exact maps, coordinate bases, and matrix chains likewise use the type when their finite dimensions are incompatible.

`RelationError` belongs to the operator-grammar layer developed in Chapter 9. Relation terms must be parallel typed words: every word in one relation has the same source and target. Duplicate relation names, inconsistent endpoints, and a realized operator that violates a registered relation also use this type. By contrast, asking a grammar to form a path from two noncomposable arrows is an input-validation failure and raises `ValidationError` before any relation exists.

A useful abstract routing model is

$$f(x;b) \rightsquigarrow \begin{cases} \uparrow E_D, & \neg D(x), \\ \uparrow E_R, & \neg R(x), \\ \uparrow E_V, & \neg V(x), \\ \uparrow E_A, & \neg A_{\text{checked}}(x), \\ \uparrow E_L, & Q(x) > b, \\ (y, C), & \text{otherwise}, \end{cases}$$

where D, R, V are dimension, relation, and general validation predicates. A_{checked} is a finitely testable assumption. $Q(x)$ is a declared resource measure. b is its budget, and C is a certificate. This is a documentation model, not a promise of one universal check order. Each public function validates the properties relevant to its own contract.

6.4 Exceptions versus invalid certificates

An exception says that the requested operation cannot proceed under its input contract. An invalid certificate says that a finite computation or check was performed and produced inspectable negative evidence. These outcomes are not interchangeable.

For a certificate

$$C = (v, e, r, \tau, H_{\checkmark}, H_{?}, D, T, M),$$

the fields are validity v, exactness class e, residuals r, tolerance τ, checked assumptions $H_{\checkmark}$, unchecked assumptions $H_{?}$, diagnostics D, theorem identifier T, and message M. The strict conversion is

$$\text{require}(C) = \begin{cases} C, & C.\texttt{valid}, \\ \uparrow E_V(M), & \text{otherwise}. \end{cases}$$

`Certificate.require()` returns the same certificate when it is valid and raises `ValidationError` with the certificate message otherwise. `ComputationResult.require()` first enforces its certificate and then returns the contained value.

This distinction supports two legitimate workflows. A strict pipeline calls `.require()` immediately because later stages are meaningful only after acceptance. A diagnostic tool first records residuals, tolerance, assumptions, and diagnostics, then calls `.require()` only if it intends to cross into a strict stage.

Construction validation follows the same pattern. `PacketSystem(..., validate=True)`, the default under the default settings, builds its certificate and requires it. `validate=False` allows an algebraically invalid but structurally well-formed packet record to be inspected. It does not bypass empty-input, nonsquare-matrix, common-dimension, or name-validation checks.

Some iterative algorithms also return an invalid certificate for ordinary nonconvergence. For example, conjugate gradient can return `converged=False` with its final residual and an invalid numerical certificate when the iteration limit is reached. A detected nonpositive generated direction is different: it falsifies a required algorithmic assumption and raises `AssumptionError` immediately.

6.5 Checked assumptions and conditional evidence

An assumption is not automatically an error. The important division is whether the current finite data can decide it.

If a supplied declaration is finitely false, the operation raises `AssumptionError`. The separated-jump recovery routine, for example, can check that the declared minimum separation is at least one sample spacing, that the minimum jump is sufficiently larger than the smooth and noise contrast bound, and that the threshold lies in the required open gap. Failure of one of those finite inequalities prevents the theorem-backed operation from proceeding.

Other hypotheses concern an unknown generating object and cannot be proved from the finite samples alone. The same recovery routine cannot establish globally that the unknown smooth part obeys the declared Lipschitz bound, that all unknown noise obeys the uniform bound, that every unknown jump has the declared minimum magnitude, or that no sample cell contains more than one jump. It therefore returns a valid certificate with exactness `conditional` and records those hypotheses in `assumptions_unchecked`.

Consequently,

$$C.\texttt{valid} = \texttt{True} \quad \not\Rightarrow \quad H_? \text{ is proved.}$$

Validity means the implemented finite checks passed under the certificate's stated semantics. A conditional conclusion remains conditional. Published reports should preserve both the exactness label and the unchecked-assumption list instead of reducing the evidence to one Boolean.

6.6 Executed example: evidence and assumption paths

This compact program shows an inspectable invalid certificate, its conversion to an exception, a valid conditional result, and a finitely false assumption:

```
import numpy as np

from komorebi_math import recover_separated_jumps
from komorebi_math.stable import AssumptionError, PacketSystem, ValidationError

bad = PacketSystem([np.zeros((2, 2))], validate=False)
failed = bad.certificate()
print(failed.valid, failed.message)

try:
    failed.require()
except ValidationError as error:
    print(type(error).__name__ + ":", error)

recovered = recover_separated_jumps(
    [0.0, 0.0, 3.0, 3.0],
    spacing=1.0,
    lipschitz_bound=0.0,
    noise_bound=0.0,
    minimum_jump=2.0,
    minimum_separation=1.0,
)
```

```
print(recovered.cells, recovered.locations.tolist())
print(recovered.certificate.exactness.value)
print(recovered.certificate.assumptions_unchecked[0])
```

The finitely false declaration is a separate negative path:

```
try:
    recover_separated_jumps(
        [0.0, 0.0, 3.0, 3.0],
        spacing=1.0,
        lipschitz_bound=0.0,
        noise_bound=0.0,
        minimum_jump=2.0,
        minimum_separation=0.5,
    )
except AssumptionError as error:
    print(type(error).__name__ + ":", error)
```

Expected output:

```
False projectors do not form a complete packet system
ValidationError: projectors do not form a complete packet system
(1,) [1.5]
conditional
unknown smooth part obeys the declared Lipschitz bound
AssumptionError: minimum_separation must be at least one sample spacing
```

The first object is structurally admissible but fails the packet axioms. The second result passes every implemented finite recovery check, yet its conclusion retains explicit hypotheses about the unknown signal. The final call fails before producing a result because its supplied finite separation declaration is incompatible with the sampling scale.

6.7 Backend and exactness boundaries

`get_backend(name)` raises `BackendUnavailableError` for an unknown or unavailable backend. `available_backends()` reports import presence. It does not select a backend, prove that hardware is usable, or certify numerical behavior. The caller should choose a backend explicitly when scalar domain or exactness matters.

`ExactnessError` is part of the stable hierarchy and reserves the semantic boundary where an exact operation cannot be honored by an inexact backend. In this release, however, exactness is communicated mainly through `Certificate.exactness`, and the audited source contains no active `raise ExactnessError` site. The guide should document the class without inventing a current executable trigger. Invalid "exact" inputs frequently use `ValidationError`. Numerical computations retain `numerical` exactness even when their observed residual happens to print as zero.

`FeatureNotImplementedError` is similarly a real package-defined class but not a frozen stable symbol. The internal planned-feature guard can raise it for a declared placeholder. Every current registry entry is implemented, partial, or experimental, so no ordinary current public path supplies a useful executed example.

6.8 Explicit resource contracts

A resource budget makes a finite computational promise visible before a large allocation or enumeration begins. Exceeding a valid declared budget raises `ResourceLimitError`. Supplying an invalid budget, such as zero or a Boolean where a positive integer is required, raises `ValidationError`, because the contract itself is malformed.

The primary coordinate contract is:

```
CoordinateBudget(
    maximum_matrix_dimension=512,
    maximum_polynomial_degree=512,
    maximum_horizon=4096,
    maximum_exact_entries=1_000_000,
)
```

Its preflight methods are:

```
require_matrix(rows, columns, *, operation) -> None
require_degree(degree, *, operation) -> None
require_horizon(horizon, *, operation) -> None
```

For a matrix request with r rows and c columns, the default contract requires

$$0 \le r, c \le 512, \qquad rc \le 10^6.$$

The dimension bound and exact-entry bound are independent: a caller may reduce one dimension yet still exceed the total-entry contract. The `operation` label is included in the error message so that a report identifies the rejected allocation rather than merely its size.

Two other public budget families cover different domains. `GeneratingBudget` guards degree, coefficient bit length, finite-state matrices, reachable states, local combinations, modulus, serialized bytes, and recursion depth. `ExpressionBudget` preflights an expression DAG and guards node count, depth, traversal steps, polynomial degree, exact entries, coefficient bits, structural bytes, and symbol length. Expression cycles are malformed graphs and therefore raise `ValidationError`. Excess finite size raises `ResourceLimitError`.

The defaults are domain contracts, not estimates of available RAM. In particular, the generating budget uses 64 MiB, or $67{,}108{,}864$ bytes, while the expression budget uses decimal $64{,}000{,}000$ bytes. Do not merge the two values in documentation.

6.9 Executed example: `CoordinateBudget`

```
from komorebi_math import CoordinateBudget
from komorebi_math.stable import ResourceLimitError, ValidationError

budget = CoordinateBudget(
    maximum_matrix_dimension=4,
    maximum_polynomial_degree=3,
    maximum_horizon=8,
    maximum_exact_entries=12,
)

budget.require_matrix(3, 4, operation="guide transform")
print("accepted: 3 by 4")

for action in (
    lambda: budget.require_matrix(4, 4, operation="guide transform"),
    lambda: budget.require_degree(4, operation="guide polynomial"),
    lambda: CoordinateBudget(maximum_horizon=0),
):
    try:
        action()
    except (ResourceLimitError, ValidationError) as error:
        print(type(error).__name__)
        print(str(error))
```

Expected output:

```
accepted: 3 by 4
ResourceLimitError
guide transform requires 16 exact entries, exceeding the coordinate entry limit 12
ResourceLimitError
guide polynomial degree 4 exceeds the coordinate degree limit 3
ValidationError
maximum_horizon must be a positive integer
```

The first request lies exactly on the entry boundary and succeeds. The next two requests are mathematically well-formed but exceed a reviewed resource contract. The final constructor does not create a valid contract at all.

A caller may provide a larger budget when the resulting arithmetic and memory cost are acceptable. Such a change is authorization, not error suppression. It belongs near the operation that needs it and should be visible in an execution record or configuration review.

6.10 Negative contracts

These limits are part of the documentation contract:

- **No universal normalization of Python failures.** Package exceptions cover package-defined semantic failures, but built-in `TypeError`, `ValueError`, and `KeyError` remain possible where ordinary Python semantics are appropriate.
- **No structural bypass through `validate=False`.** The flag can defer selected mathematical axiom enforcement. It cannot make empty, ill-shaped, or inconsistently dimensioned input admissible.
- **No theorem from a Boolean alone.** A valid certificate must be read together with its exactness, tolerance, residuals, and checked and unchecked assumptions.
- **No convergence implication from receiving a result object.** Algorithms may return values and diagnostic histories with invalid certificates.
- **No silent backend substitution.** Availability, selection, exactness, and hardware qualification are separate claims.
- **No guarantee that every historical `max_*` parameter raises `ResourceLimitError`.** The coordinate, generating, and expression budgets use the typed resource contract, but some older domain-specific guards still raise `ValidationError`. Consult the individual function contract.
- **No promise that a resource budget predicts wall-clock time.** It bounds declared structural quantities. Scalar growth, backend behavior, and machine characteristics can still affect cost.
- **No programmatic dependence on message text.** Catch by type and inspect structured result or certificate fields. Messages are for diagnosis and may become more precise without changing the exception contract.

Applications should therefore use this pattern:

1. Validate structural input and dimensions.
2. State the exact, numerical, or conditional domain explicitly.
3. Provide an explicit budget where expensive finite work is possible.
4. Catch only errors that the current layer can repair.
5. Inspect the complete certificate.
6. Call `.require()` before crossing into a stage that requires accepted evidence.
7. Retain the original exception type, certificate, and environment context in diagnostic records.

> **Chapter 6 summary.** The object is an input contract, a finite resource budget, or a returned certificate. The operation either raises a typed failure or returns inspectable evidence. Checked assumptions may fail immediately, while unchecked hypotheses remain explicit in conditional certificates. The safe caller catches narrowly, never treats `validate=False` as a shape bypass, and calls `.require()` only when invalid evidence must become a hard boundary.

Failure behavior is part of an interface's public meaning. Chapter 7 therefore moves from one execution to the longer-lived question: which names, signatures, semantics, and portable records may downstream code rely on across releases?

7. Public API tiers

Komorebi Math makes a deliberate distinction between *availability* and *compatibility commitment*. A name can be importable and useful without being promised for the whole 1.x line. Conversely, a stable import path says that the project has accepted additional obligations: the exported name, broad object kind, documented signature, selected finite behavior, and portable identity must continue to agree with committed contracts.

The recorded `1.0.0b4` reference baseline is a beta package. Its release contract describes the line from `1.0.0b1` through `1.0.0rc1`, with `1.0.0b3` as the direct predecessor. The release observations and literal outputs below belong to that baseline. The word *stable* names its frozen compatibility surface; a tier label alone does not establish the publication or qualification status of any later package.

The public tiers are:

Tier	Public namespace	Current manifest size	Compatibility policy
`[STABLE]`	`komorebi_math.stable` plus two aliases	131	frozen structure, semantics, and persistence
`[BETA]`	`komorebi_math.beta`	83	additive surface with strict signatures
`[RESEARCH]`	top-level and selected domain namespaces	1,312	additive. Signatures non-blocking by default

The release machinery also recognizes an `internal` tier, but internal modules do not form a public compatibility surface. The three documented tiers can be summarized by the implication boundary

$$\text{importable} \not\Rightarrow \text{beta contracted} \not\Rightarrow \text{stable contracted}.$$

This separation allows new finite mathematics to be implemented and tested without silently freezing every experimental spelling. It also gives users a clear choice: exploratory code may use research namespaces, reusable beta pipelines may accept the stricter beta contract, and long-lived integrations should prefer the stable aliases.

A practical dependency rule is to import from the narrowest tier that contains the required operation. If a program only needs stable coordinate transforms, polyphase refinement, certificates, and portable records, importing those names from `komorebi_math.stable` makes its expectations explicit. If it needs the resumable packet-jet stream machinery, `komorebi_math.beta` is the appropriate facade. If it needs `komorebi_math.integration`, `komorebi_math.expression`, or another recently added domain, it is intentionally accepting a research-tier dependency. Mixing tiers is legal, but the weakest selected tier determines the compatibility expectation for that part of the program.

Tier membership is about the exported path, not a second copy of the implementation. A class may be defined in a domain module and re-exported by a curated facade. The guide records both facts: the domain module is the canonical implementation location, while the stable or beta namespace is the public compatibility commitment.

7.1 Research namespace

The research API is the broad executable workspace. Its current manifest is built from explicit `__all__` lists in six namespaces:

$$\mathcal{N}_R = \{\texttt{komorebi_math}, \texttt{komorebi_math.nonlinear}, \texttt{komorebi_math.export}, \texttt{komorebi_math.signals}, \texttt{komorebi_math.generating}, \texttt{komorebi_math.coordinates}\}.$$

The 1,312 manifest entries are namespace/name pairs, so the same implementation may appear more than once when it is intentionally re-exported. Their current distribution is:

Namespace	Manifest entries
`komorebi_math`	516
`komorebi_math.coordinates`	237
`komorebi_math.generating`	181
`komorebi_math.signals`	171
`komorebi_math.nonlinear`	149
`komorebi_math.export`	58

The research stability label is `pre-beta-additive`. In ordinary compatibility mode, a later manifest may add names. It may not remove a committed export or change its broad kind from, for example, a class to a function. Reflected signature changes are reported, but they become compatibility failures only when strict signature checking is requested.

Let M_0 be the reference manifest and M_1 the observed manifest. If s denotes strict-signature mode, the implemented structural decision is

$$\text{Compatible}(M_0, M_1; s) \iff \Delta_{\text{removed}} = \emptyset \wedge \Delta_{\text{kind}} = \emptyset \wedge (\neg s \vee \Delta_{\text{signature}} = \emptyset).$$

Additions Δ_{added} are always reported but do not make this comparison fail. The default research command is therefore useful when asking whether earlier imports still exist:

```
komorebi api check
```

The current publication gate uses the stronger release-time signature check:

```
komorebi api check --strict-signatures
```

This strict command does not turn the research tier into the stable tier. It pins the signatures of the release being qualified. It does not promise that a future research release can never make a reviewed signature change.

The research surface has grown while the curated tiers have remained fixed:

Version	Research entries	Beta entries	Stable entries
`1.0.0b1`	1,179	83	131
`1.0.0b2`	1,191	83	131
`1.0.0b3`	1,229	83	131
`1.0.0b4`	1,312	83	131

This is the intended pattern. The finite measure and integration API and the typed finite expression API are research features. They are tested and documented, but they were not silently inserted into either curated contract.

Research boundary. The manifest checks public structure. It does not prove mathematical equivalence, numerical stability, performance, exception compatibility, thread safety, or the behavior of undocumented internals. Those claims need their own tests, certificates, and domain documentation.

7.2 Beta namespace

`komorebi_math.beta` is a smaller compatibility facade for finite packet-jet execution and persistence. Its 83 exports cover certificates and errors, packet systems, derivative providers, packet jets, sparse tensors, execution plans, snapshots, resumable streams, profiling records, portable codecs, migrations, and compatibility tools.

The beta stability label is `1.0-beta-additive-strict-signatures`. The word *additive* means that new names may be introduced. Existing names may not be removed, change broad kind, or change reflected signature without an explicit compatibility decision. The tier-specific check always enables strict signatures:

```
komorebi api check --beta
```

Structural compatibility is supplemented by a closed behavioral golden fixture. The current fixture executes 15 cases involving packet jets, snapshots, streaming records, migrations, canonical serialization, and related finite behavior. It can be run separately:

```
komorebi golden
```

The golden cases protect the behavior they state. They are not an exhaustive proof of every input or optional backend. In particular, a live arbitrary Python derivative provider cannot acquire a cross-process semantic identity merely because a stream record contains metadata. Frozen snapshots can make a stronger claim because their retained coefficient content is itself fingerprinted.

A user should select the beta tier when the packet-jet and streaming facade is the desired contract but the broader stable workflow is either unnecessary or too restrictive. Importing a beta object from its implementation module remains possible, but importing it from `komorebi_math.beta` states the intended compatibility dependency explicitly.

7.3 Frozen stable namespace and aliases

The stable candidate contains 131 exported names selected to support one complete finite workflow:

$$\text{construct} \to \text{decompose} \to \text{operate} \to \text{refine} \to \text{certify} \to \text{persist} \to \text{render}.$$

It includes core operator and packet objects, exact and numerical coordinate transforms, analysis/synthesis systems, polyphase and residue refinement, certificates and errors, portable persistence, document rendering, settings, and the stable-contract checks themselves. Domain-specific research objects remain available elsewhere without inheriting this promise.

Three import paths expose the same exported objects:

```
import komorebi_math.stable as stable
import komorebi_math.v1 as v1
import komorebi_math.api.v1 as api_v1

print(len(stable.__all__))
print(stable.__all__ == v1.__all__ == api_v1.__all__)
print(all(
    getattr(stable, name) is getattr(v1, name) is getattr(api_v1, name)
    for name in stable.__all__
))
print(stable is v1, stable is api_v1)
```

Expected output:

```
131
True
True
False False
```

The last line matters. These are distinct module objects, not one module assigned three names. Their `__all__` lists agree, and every public value they re-export is object-identical. There is no second implementation hiding behind either versioned path.

Readable polyphase names are also retained as direct object aliases:

```
assert stable.polyphase_refine is stable.refine_polyphase
assert stable.polyphase_coarsen is stable.coarsen_polyphase
assert stable.lattice_polyphase_refine is stable.refine_lattice_polyphase
assert stable.lattice_polyphase_coarsen is stable.coarsen_lattice_polyphase
```

Similarly, a stable class is the same class exported by its defining domain module. The facade selects and names implementations. It does not fork them.

The stable symbol array is exactly unchanged across the committed `0.16`, `1.0.0b1`, `1.0.0b2`, `1.0.0b3`, and `1.0.0b4` manifests. That historical identity is stronger evidence than the count alone: 131 differently defined objects would not satisfy the upgrade contract.

7.4 Manifest identity, reflected signatures, and semantic fixtures

Every API manifest is a deterministic collection of `PublicApiSymbol` records. One symbol has the form

$$S = (N, n, k, m, q, \sigma),$$

where N is the export namespace, n the public name, k the broad kind, m the implementation module, q the qualified implementation name, and σ the reflected callable signature or `null`.

For example, the stable `PolyphaseRefinement` entry records:

```
{
  "implementation_module": "komorebi_math.refinement.polyphase",
  "kind": "class",
  "name": "PolyphaseRefinement",
  "namespace": "komorebi_math.stable",
  "qualname": "PolyphaseRefinement",
  "signature": "(coarse_order: 'int', fine_order: 'int') -> None"
}
```

Manifest construction rejects empty namespace lists, duplicate namespaces, duplicate exports, absent exported attributes, and nondeterministic symbol ordering. It also normalizes two interpreter-dependent cases. Concrete POSIX and Windows path values are recorded as the public `pathlib.Path` abstraction, and generic-alias constructors receive a portable variadic signature across the supported Python 3.11 through 3.13 matrix.

The manifest's `digest` property hashes canonical compact JSON. It is not the same quantity as the SHA-256 hash of the indented JSON file stored on disk. The first binds the validated mapping. The release contract separately binds the exact artifact bytes. Current canonical manifest digests are:

```
research  473c73e1a0563fe0b2569c8be384f7c9f947570bea2f300ef611db351136847a
beta      311f2bf7cbcf0a45bcbe2af433d73a8f597c0fbf7133750882314d6e0edb16b2
stable    23cd9e47a1663f2d4b3c0fef50f9f7d8c88d4af2f8bbc57e7ba76461e6e69276
```

Structural identity alone is not enough for the stable tier. The aggregate contract is

$$C_{\text{stable}} = C_{\text{API}} \wedge C_{\text{semantics}} \wedge C_{\text{persistence}}.$$

`check_stable_api()` performs the strict structural comparison. `run_stable_semantics_fixture()` executes ten committed mathematical cases. `check_portable_codec_contract()` verifies the 51 package-owned codec tags, their represented implementation types, nondecreasing versions, and every required adjacent migration. `check_stable_contract()` combines the three reports into one `StableContractReport`.

```python
from komorebi_math.stable import check_stable_contract

report = check_stable_contract()

print(report.compatible)
print(sum(case.passed for case in report.semantics.results))
print(len(report.semantics.results))
print(report.persistence.current_version)
print(report.certificate.message)
```

Expected output:

```
True
10
10
1.0.0b4
the complete 1.0-candidate compatibility contract passed
```

The semantic fixture covers general analysis/synthesis reconstruction, one-dimensional and lattice polyphase refinement, exact arithmetic polyphase coordinates, an operator relation, exact basis coordinates, a factorial-power series product, document rendering, settings restoration, and a portable refinement-tower roundtrip. Its ten cases are representative and closed, not exhaustive.

Stable semantic case	Principal observation
General analysis/synthesis	reconstruction and transfer-block certificates
Signal polyphase	direct refinement match and coarsening roundtrip
Arithmetic polyphase	exact rational reconstruction
Lattice polyphase	rectangular address set and roundtrip
Operator grammar	exact relation residual
Coordinate basis	exact connection matrix and inverse roundtrip
Normalized series	factorial-power product coefficients
Document rendering	deterministic Markdown and LaTeX digests
Settings scope	active values and restoration after the context
Portable refinement	codec tag, version, type, and equality

The fixture is finite by design and readable. Its certificate lists "fixture coverage is representative rather than exhaustive" and optional hardware equivalence among the unchecked assumptions. A passing fixture cannot be used to infer behavior for an arbitrary dimension, backend, tolerance, or unrepresented mathematical family.

Version identity is also a separate concern. Manifest comparison requires the same namespace set, but its compatibility Boolean is decided from symbol differences rather than from the package-version strings alone. This permits a reference manifest to be compared with a later release. The release contract then checks that runtime `__version__`, the current manifests, citation metadata, and packaged artifacts all identify `1.0.0b4`. In other words, an API checker answers "Is this surface compatible?" A release gate additionally answers "Is this the artifact and version we intended to ship?"

The installed stable verification suite adds two important release assertions: the newly constructed manifest must equal the committed manifest in every recorded field, and all 131 exports must retain tri-alias object identity. This is why a release decision should use the suite rather than treating one structural Boolean as the whole contract:

```
komorebi verify stable
```

The current terminal summary is:

```
[PASS] strict stable API manifest
[PASS] deterministic stable API snapshot
[PASS] closed stable semantic fixture
[PASS] portable codec migration contract
[PASS] aggregate 1.0-candidate contract
[PASS] versioned stable namespace identity
verification passed 6/6 checks
```

The distinction between loose and strict signatures is executable without changing the installed package. This code constructs an in-memory candidate whose `check_stable_contract` signature has been altered:

```
from dataclasses import replace
from komorebi_math.compatibility import (
    PublicApiManifest,
    compare_public_api,
    load_stable_api_manifest,
)

reference = load_stable_api_manifest()
symbols = list(reference.symbols)
index = next(
    i for i, symbol in enumerate(symbols)
    if symbol.name == "check_stable_contract"
)
symbols[index] = replace(
    symbols[index],
    signature="() -> 'StableContractReport'",
)

changed = PublicApiManifest(
    package_version="demonstration",
    namespaces=reference.namespaces,
    symbols=tuple(sorted(symbols)),
    stability_level=reference.stability_level,
)
```

Now compare the same in-memory candidate under both policies:

```


loose = compare_public_api(reference, changed)
strict = compare_public_api(
    reference,
    changed,
    strict_signatures=True,
)
```

```
print("loose:", loose.compatible, len(loose.signature_changed))
print("strict:", strict.compatible, len(strict.signature_changed))
```

Expected output:

```
loose: True 1
strict: False 1
```

This example changes only a manifest value in memory. It does not monkey-patch or overwrite the package. It shows exactly why the beta and stable tier helpers always request strict comparison.

7.5 Deprecation and migration discipline

Compatibility sometimes requires a name to remain available while users move to a replacement. The `deprecated` decorator retains the call, emits `KomorebiDeprecationWarning`, and attaches a machine-readable record

$$D = (\text{since}, \text{replacement}, \text{remove_in}, \text{reason}).$$

```
import warnings
from komorebi_math import KomorebiDeprecationWarning, deprecated

@deprecated(
    since="1.0.0b4",
    replacement="new_square",
    remove_in="2.0",
    reason="demonstration",
)
def old_square(value: int) -> int:
    return value * value

with warnings.catch_warnings(record=True) as caught:
    warnings.simplefilter("always", KomorebiDeprecationWarning)
    print("value:", old_square(5))

record = old_square.__komorebi_deprecation__
print("warning emitted:", len(caught) == 1)
print("since:", record.since)
print("replacement:", record.replacement)
print("remove in:", record.remove_in)
print("reason:", record.reason)
```

Expected output:

```
value: 25
warning emitted: True
since: 1.0.0b4
replacement: new_square
remove in: 2.0
reason: demonstration
```

The example defines its own demonstration function. It does not claim that `old_square` is a package export. At `1.0.0b4`, the exact stable documentation contract covers all 131 stable exports and reports zero deprecated stable names.

Unambiguous spelling or discovery aliases may be retained without a warning. The polyphase aliases above are examples. A compatibility alias must not be used to conceal changed mathematics: the analysis/synthesis methods `projector()` and `projectors()` remain for compatibility, but in a redundant system their returned reconstruction components need not be idempotent. New code should use `component()` and `components()` unless projector hypotheses have been established independently.

Portable data has a separate migration discipline. A Python deprecation does not authorize reinterpretation of serialized records. If a codec advances from version v to $v + k$, every adjacent step must exist:

$$J_v \xrightarrow{\mu_{v\to v+1}} J_{v+1} \xrightarrow{\mu_{v+1\to v+2}} \cdots \xrightarrow{\mu_{v+k-1\to v+k}} J_{v+k}.$$

Unknown tags, unsupported future versions, type-identity changes, decreasing versions, and missing adjacent migrations fail closed. Users should retain original payloads while upgrading and must not hand-edit `__komorebi_type__` or `__version__` fields.

The current upgrade contract checks the stable API against `0.15.0a2`, `0.16.0a2`, `1.0.0b1`, `1.0.0b2`, and `1.0.0b3`. It also validates and loads seven historical portable fixtures. Run it before accepting an upgrade:

```
komorebi release upgrade
```

For a complete current release decision, the useful sequence is:

```
komorebi api check --stable
komorebi api check --beta
komorebi api check --strict-signatures
komorebi verify stable
komorebi release documentation
komorebi release upgrade
komorebi release contract
```

The API commands answer structural questions. `verify stable` adds semantic, persistence, snapshot, and alias checks. The documentation and upgrade commands close their respective inventories. The release contract binds those layers to the current package version and committed artifact hashes.

Maintainers can exercise the focused source contracts directly:

```
python -m pytest -q `
  tests/test_block15_stable_api.py `
  tests/test_block16_contracts.py `
  tests/test_block16_release_contract.py `
  tests/test_block17_upgrade_contract.py
```

These tests cover deterministic manifest construction, platform-neutral path identity, strict signature rejection, all three stable aliases, the aggregate semantic and persistence contract, release artifact counts, frozen CLI route names, predecessor manifests, and historical portable fixtures. The recorded publication tests add strict research-signature portability across the declared Python versions. A focused test pass is useful during development, but the publication decision still belongs to the complete gate because documentation, platform evidence, distribution artifacts, and the rest of the source suite lie outside this chapter's focused selection.

Chapter 7 summary. Use research namespaces to explore the broad executable library, `komorebi_math.beta` when depending on the curated packet-jet and streaming facade, and one of the three stable aliases for long-lived 1.x code. A manifest proves selected public structure, not mathematics. The stable promise is therefore layered: strict API identity, ten committed semantic cases, 51 portable-codec contracts, exact tri-alias identity, one-to-one documentation, and explicit adjacent migrations.

An API tier states how a public interface may evolve. It does not choose the scalar arithmetic used by a particular run. Chapter 8 supplies that second axis: backend selection, coercion, exact domains, and optional runtimes.

8. Backends, scalar domains, and coercion

Komorebi Math uses the word *backend* in more than one setting, but those settings are not interchangeable. The finite-dimensional core has a small linear-algebra backend interface, currently realized by NumPy and, when the optional dependency is installed, SymPy. Other subsystems use optional feature adapters: SciPy supplies selected scientific routines, JAX and PyTorch supply directional automatic differentiation, and CuPy supplies selected device executors. Finally, several exact coordinate and arithmetic modules use Python integers and `fractions.Fraction` directly and do not need any matrix backend at all.

That separation is the central rule of this chapter:

$$\text{core matrix backend} \neq \text{optional feature runtime} \neq \text{exact scalar domain}.$$

Selecting one does not silently select the others. In particular, `get_backend("numpy")` says nothing about whether SciPy is installed, `get_backend("sympy")` does not enable JAX, and the exact rational coordinate layer works without SymPy.

8.1 The core linear-algebra backend contract

The abstract type is `komorebi_math.backends.LinearAlgebraBackend`. A backend declares a name and whether its arithmetic is treated as exact:

```
class LinearAlgebraBackend(ABC):
    name: str
    exact: bool
```

Its operational contract is:

Operation	Signature	Meaning
conversion	`as_matrix(value) -> Any`	convert a matrix-like value
conversion	`as_vector(value) -> Any`	convert and normalize a vector-like value
inspection	`shape(value) -> tuple[int, ...]`	report dimensions
constructors	`zeros(rows, cols)`, `eye(size)`	additive and multiplicative identities
arithmetic	`matmul`, `add`, `subtract`, `scale`	finite linear algebra
dual operations	`transpose`, `adjoint`	transpose and conjugate transpose
inversion	`inverse(value)`	inverse of a nonsingular square matrix
diagnostics	`rank(value, *, tolerance=None)`	exact or thresholded rank
diagnostics	`norm(value) -> float`	a floating diagnostic norm
diagnostics	`is_zero(value, *, tolerance=None)`	exact or tolerance-aware zero test
basis construction	`column_stack(values)`	assemble column vectors
kernel	`nullspace(value, *, tolerance=None)`	a basis for the right nullspace
tensor product	`kron(left, right)`	Kronecker product
copying	`copy(value)`	backend-native copy

The public selection functions are:

```
get_backend(
    backend: str | LinearAlgebraBackend | None = None,
) -> LinearAlgebraBackend

register_backend(
    backend: LinearAlgebraBackend,
    *,
    replace: bool = False,
) -> None

available_backends() -> dict[str, bool]
backend_name(value: Any) -> str
```

The `exact` flag is semantically important. Core objects such as `OperatorAlgebra`, `PacketSystem`, `OperatorRealization`, and `SeedSystem` label their certificates `Exactness.EXACT` when `backend.exact` is true and `Exactness.NUMERICAL` otherwise. For a numerical certificate, the active tolerance is recorded. For an exact certificate, the recorded tolerance is `None`. Chapters 9 through 11 give the full object contracts. Only their backend-dependent evidence behavior is covered here.

This is a declaration by the backend, not an independent proof of its scalar field. A custom backend must therefore set `exact` honestly. Marking floating arithmetic exact would produce misleading certificate labels.

8.2 Registry, discovery, selection, and classification

Four public functions answer four different questions.

Function	Question answered	Important limitation
`get_backend(name)`	Which core linear-algebra backend object will execute?	Only registered names work. `None` means NumPy.
`register_backend(obj)`	Add or replace a process-global registry entry.	No public unregister operation. Registration is not context-local.
`available_backends()`	Which named optional packages can be discovered?	Presence is not registration, successful import, hardware readiness, or execution.
`backend_name(value)`	Does the value's defining module begin with `sympy`?	Everything else is reported as `numpy`. This is a heuristic, not dispatch.

At import time the registry contains one object, a default `NumpyBackend()`. `get_backend()` and `get_backend(None)` always return that NumPy object. The string `"sympy"` is special: if SymPy can be discovered, the library lazily registers a `SympyBackend`. The actual `import sympy` is deferred until the first matrix operation. Backend names are case-sensitive and are not normalized.

`available_backends()` always returns exactly the keys `numpy`, `sympy`, `scipy`, `mpmath`, and `cupy`. Except for NumPy, each Boolean is based on module discovery. Thus this is possible and intentional:

```
from komorebi_math.backends import available_backends, get_backend
from komorebi_math.exceptions import BackendUnavailableError

print(available_backends()["scipy"])  # may be True
try:
    get_backend("scipy")
except BackendUnavailableError as exc:
    print(type(exc).__name__ + ":", exc)
```

Possible output when SciPy is installed:

```
True
BackendUnavailableError: unknown or unavailable backend: scipy
```

There is no contradiction. SciPy is an optional feature dependency, not an implementation of `LinearAlgebraBackend` in this release. The same warning applies to mpmath and CuPy. JAX and PyTorch use their own diagnostic API and do not appear in `available_backends()`.

A deterministic unknown-name failure is useful in examples and installation checks:

```python
from komorebi_math.backends import get_backend
from komorebi_math.exceptions import BackendUnavailableError

try:
    get_backend("not-a-komorebi-backend")
except BackendUnavailableError as exc:
    print(f"{type(exc).__name__}: {exc}")
```

Output:

```
BackendUnavailableError: unknown or unavailable backend: not-a-komorebi-backend
```

Custom registration is intentionally low level. A duplicate name raises `ValueError` unless `replace=True`. Replacement can overwrite even a built-in entry for the remainder of the process. The implementation does not perform a full protocol audit during registration. Extension authors should subclass `LinearAlgebraBackend`, implement every abstract method, use a private unique name, and test in an isolated process.

8.3 NumPy: the default numerical realization

The constructor is:

```
NumpyBackend(*, dtype: Any = numpy.complex128)
```

The default scalar field is therefore represented numerically by complex double precision, even when the input contains only real integers. `as_matrix` requires a genuinely two-dimensional array. `as_vector` accepts a one-dimensional value or a two-dimensional row or column and flattens it. Shape failures raise `DimensionError`.

`transpose(A)` computes A^T. `adjoint(A)` computes $A^* = \overline{A}^T$. They differ for complex matrices. `norm` delegates to `numpy.linalg.norm` without an `ord` argument, so it is the Euclidean norm for a vector and the Frobenius norm for a matrix. Consequently,

$$\text{is_zero}(A;\tau) \iff \|A\| \le \tau,$$

with `1e-10` used when no tolerance is passed directly to `is_zero`. This is a whole-object norm test, not an element-by-element comparison. A large matrix whose individual entries are all below τ may still have Frobenius norm above τ.

NumPy rank and nullspace are numerical decisions. `rank` delegates to `numpy.linalg.matrix_rank`. `nullspace` performs a full singular-value decomposition and treats singular values above the threshold as nonzero. Its default threshold is

$$\tau_{\text{svd}} = \max(m,n)\,\epsilon_{64}\,\sigma_{\max},$$

where the implementation uses `numpy.finfo(float).eps`. An explicit tolerance overrides that threshold.

The rank boundary can be seen with a nearly singular diagonal matrix:

```
from komorebi_math.backends import get_backend

b = get_backend("numpy")
A = b.as_matrix([[1.0, 0.0], [0.0, 1e-12]])

print(b.rank(A))
print(b.rank(A, tolerance=1e-10))
print(b.is_zero(A, tolerance=1e-10))
```

Output:

```
2
1
False
```

The explicit threshold changes the estimated rank. It does not make the matrix exactly singular. The zero test remains false because the matrix norm is approximately one.

NumPy also coerces exact-looking input to the chosen dtype:

```
from fractions import Fraction
from komorebi_math.backends import get_backend

b = get_backend("numpy")
v = b.as_vector([Fraction(1, 3)])
print(v.dtype)
print(v[0])
print(v[0] == Fraction(1, 3))
```

Output:

```
complex128
(0.3333333333333333+0j)
False
```

Inversion and SVD failures are not wrapped in package exceptions. For example, inverting a singular NumPy matrix raises `numpy.linalg.LinAlgError`. The backend conversion boundary also does not reject `NaN` or infinity. Individual higher-level algorithms may impose stronger finite-value checks.

8.4 SymPy: the optional symbolic and exact realization

The supported public selection path is `get_backend("sympy")`. The concrete class is loaded lazily. Install the optional dependency with the package's `exact` extra before relying on it.

```
from fractions import Fraction
from komorebi_math.backends import available_backends, get_backend

if not available_backends()["sympy"]:
    print("SymPy unavailable; install komorebi-math[exact]")
else:
    b = get_backend("sympy")
    A = b.as_matrix([[1, Fraction(1, 2)], [0, 1]])
    print(b.exact)
    print(b.inverse(A))
```

Installed-branch output:

```
True
Matrix([[1, -1/2], [0, 1]])
```

SymPy rank and nullspace are exact symbolic operations. Their `tolerance` arguments are accepted for interface compatibility and ignored. The zero test is structural equality with a SymPy zero matrix. It does not call a general simplifier. A mathematically zero expression that SymPy has not normalized may therefore test false until the caller simplifies it.

There are three more boundaries worth stating explicitly.

First, SymPy and NumPy do not interpret every input shape identically. `NumpyBackend.as_matrix([1, 2])` rejects a one-dimensional list, whereas SymPy's `Matrix([1, 2])` constructs a 2×1 matrix and is accepted. Second, `SympyBackend.norm` ultimately converts a SymPy norm to Python `float`. An unresolved symbolic norm may raise a conversion error. Exact core certificates still store floating residual fields, even though a SymPy-backed zero residual must equal `0.0`. Third, passing an existing mutable SymPy matrix to `as_matrix` returns that matrix rather than an immutable copy. Mutation by the caller can therefore change an object retained by a higher-level construction.

SymPy import readiness is weaker than the crash-isolated JAX and PyTorch checks described below. Discovery can succeed even if a damaged installation later fails during import. A normal missing import is translated to `Backend UnavailableError`. Other third-party initialization failures need not be wrapped.

8.5 Exact rational and modular scalar domains

Some exact computations do not use `LinearAlgebraBackend` at all. The module `komorebi_math.coordinates.exact` represents vectors and matrices as immutable tuples of `Fraction` values:

```
from fractions import Fraction
from komorebi_math.coordinates.exact import (
    as_matrix,
    identity_matrix,
    inverse_matrix,
    matrix_multiply,
)

Q = as_matrix([[1, Fraction(1, 2)], [0, 1]])
Q_inv = inverse_matrix(Q)
print(Q_inv)
print(matrix_multiply(Q, Q_inv) == identity_matrix(2))
```

Output:

```
((Fraction(1, 1), Fraction(-1, 2)), (Fraction(0, 1), Fraction(1, 1)))
True
```

This layer supplies exact rank, determinant, inverse, reduced row-echelon form, linear-system solving, nullspace, powers, and Kronecker products using only the standard library. It rejects booleans as coefficients. Singular inversion raises `ValidationError("matrix is singular")`.

Runtime coercion is slightly broader than the public `int | Fraction` type annotation because it delegates to `Fraction(value)`. That makes strings such as `"1/3"` exact, but it also preserves the binary value of a raw float:

```
from komorebi_math.coordinates.exact import as_vector

print(as_vector(["1/3"]))
print(as_vector([0.1]))
```

Output:

```
(Fraction(1, 3),)
(Fraction(3602879701896397, 36028797018963968),)
```

Use `Fraction(1, 10)` or a deliberate string when decimal intent must remain exact. Do not pass `0.1` and expect $1/10$.

The arithmetic package provides additional exact finite domains: `ModularRing(n)` for $\mathbb{Z}/n\mathbb{Z}$, `PrimeField(p)` for canonical residues in $\mathbb{F}_p$, and `TruncatedPadic(p, k)` for arithmetic modulo p^k. These classes accept integers, reject booleans, and raise `ValidationError` for a nonunit inverse. `TruncatedPadic` is a finite precision residue ring, not an implementation of an infinite complete p-adic field.

8.6 Runtime settings: tolerance is active. Backend is descriptive

The settings API is:

```
Settings(
    backend: str = "numpy",
    tolerance: float = 1e-10,
    validate_on_construction: bool = True,
)

get_settings() -> Settings
configure(*, backend=None, tolerance=None,
          validate_on_construction=None) -> Settings
settings_context(**changes) -> Iterator[Settings]
```

Settings are stored in a `ContextVar`. `configure` updates the current context. `settings_context` temporarily replaces it and restores the previous record, even when the body raises an exception. The record is frozen, so callers replace settings rather than mutating fields.

Only part of the record currently controls core construction. When explicit arguments are absent, `OperatorAlgebra`, `OperatorRealization`, `PacketSystem`, and `SeedSystem` read the active tolerance. `PacketSystem` also reads `validate_on_construction`. No production constructor consults `get_settings().backend`. `get_backend(None)` remains NumPy.

```
from komorebi_math.backends import get_backend
from komorebi_math.config import get_settings, settings_context

with settings_context(backend="sympy"):
    print(get_settings().backend, get_backend().name)
```

Output:

```
sympy numpy
```

To execute with SymPy, pass `backend="sympy"` explicitly to the construction. The backend settings field is retained as configuration metadata, not as a global dispatcher in version `1.0.0b4`.

Tolerance changes do affect constructors that consult them:

```
from komorebi_math.config import get_settings, settings_context
from komorebi_math.core import PacketSystem
from komorebi_math.exceptions import ValidationError

P0 = [[1, 0], [0, 0]]
P1 = [[0, 0], [0, 1 + 1e-8]]

try:
    PacketSystem([P0, P1])
except ValidationError as exc:
```

```
    print(f"{type(exc).__name__}: {exc}")

with settings_context(tolerance=1e-7):
    system = PacketSystem([P0, P1])
    print(system.certificate().valid, system.tolerance)

print(get_settings().tolerance)
```

Output:

```
ValidationError: projectors do not form a complete packet system
True 1e-07
1e-10
```

The settings constructors do not fully validate policy. `configure` converts a tolerance to `float` but accepts negative, infinite, or `NaN` values, and `bool(value)` means the string `"false"` becomes true. `settings_context` uses raw dataclass replacement and performs no conversion. Applications should pass a finite nonnegative tolerance and actual Boolean values.

8.7 Optional feature adapters

Optional feature adapters have their own contracts and must not be passed to `get_backend`.

Dependency	Role in this release	Readiness test
SciPy	selected routines such as `matrix_exponential(..., method="scipy")`	import at point of use. `auto` can fall back to NumPy
mpmath	selected analytic high-precision features	import at point of use
JAX	nested forward-mode directional derivatives	crash-isolated child import plus `jax.jvp` capability
PyTorch	nested `torch.func.jvp` directional derivatives	crash-isolated child import plus `torch.func.jvp` capability
CuPy	selected packet and snapshot device execution	package discovery, then separate CUDA runtime and device checks

JAX and PyTorch expose:

```
ad_framework_diagnostics(*, refresh: bool = False)
available_ad_frameworks() -> dict[str, bool]
```

Provider selection uses:

```


autodiff_provider(
    framework,
    function,
    source_dimension,
    target_dimension,
    *,
    max_order=4,
    **options,
)
```

Provider constructors are:

```
JaxDerivativeProvider(..., max_order=4, dtype=None, ...)
TorchDerivativeProvider(..., max_order=4,
                        dtype="float64", device=None, ...)
```

Provider construction is lazy. The first value or derivative request checks a cached child-process probe before importing the framework into the application process. The probe has a 60-second timeout and records discovery, availability, version, return code, and a bounded diagnostic. Use `ad_framework_diagnostics(refresh=True)` after changing an environment in the same Python process.

Both providers compute higher directional derivatives through nested JVP calls, label results numerical, and convert outputs back to host NumPy arrays. JAX's default `dtype=None` respects the external JAX precision configuration. The PyTorch adapter defaults to `float64` and accepts a device string. The function must be traceable by the selected framework. A scalar-valued map must return a length-one vector, not a zero-dimensional scalar.

```
from komorebi_math.nonlinear import (
    autodiff_provider,
    available_ad_frameworks,
)

def square(x):
    return x * x

for framework, ready in available_ad_frameworks().items():
    if not ready:
        print(framework, "unavailable")
        continue
    provider = autodiff_provider(
        framework, square, source_dimension=2,
        target_dimension=2, max_order=2,
    )
    value = provider.derivative(1, [1.0, 2.0], ([3.0, 4.0],))
    print(framework, value.tolist())
```

Each installed and usable branch prints:

```
jax [6.0, 16.0]
torch [6.0, 16.0]
```

An absent branch prints `jax unavailable` or `torch unavailable` without breaking the NumPy core. Framework tracing errors, unsupported operations, and device failures are not converted into a numerical fallback.

8.8 Scalar and backend decision table

Choose the scalar domain before choosing an optional package. If the input is intended to be rational, construct that rational value explicitly. If the workflow is numerical, choose and record its dtype and tolerance. Then consult the operation's backend field: installing a dependency and selecting it for a particular calculation are separate steps.

Need	Recommended path	Exactness	Main caution
default dense finite linear algebra	`get_backend("numpy")`	numerical	default dtype is `complex128`. Rank depends on thresholds
symbolic or rational matrix identities	`get_backend("sympy")`	declared exact	optional dependency. Structural zero test. Symbolic norms may not convert
immutable rational coordinate algebra	`komorebi_math.coordinates.exact`	exact	construct decimals explicitly with `Fraction`

Need	Recommended path	Exactness	Main caution
finite modular arithmetic	`ModularRing`, `PrimeField`	exact	only units are invertible in a composite-modulus ring
finite local precision modulo p^k	`TruncatedPadic`	exact finite quotient	not an infinite p-adic field
selected scientific matrix routines	feature API with SciPy	numerical	SciPy presence does not register a core backend
directional AD	JAX or PyTorch provider	numerical	runtime capability and traceability are separate from installation
selected CUDA execution	CuPy executor	numerical	package presence does not prove a working device

8.9 Failure boundaries and practical cautions

Keep these distinctions visible in examples and error handling:

1. An unknown or absent core backend raises `BackendUnavailableError`. Duplicate registry insertion raises ordinary `ValueError`.
2. Matrix and vector shape violations raise `DimensionError`, a subclass of `ValidationError`. Low-level NumPy and SymPy algebra failures may retain their framework-native exception types.
3. `available_backends()` reports discovery, not successful execution. `available_ad_frameworks()` is stronger because it performs an isolated capability probe, but even that does not prove a user's particular function is traceable.
4. `backend_name` recognizes SymPy by module name and calls every unfamiliar value NumPy. Python lists, JAX arrays, Torch tensors, and CuPy arrays should not be dispatched through this heuristic.
5. Tolerance changes can alter numerical rank, nullspace dimension, support, closure, and certificate validity. Report the tolerance with every numerical claim.
6. Exactness follows the executed path. A rational number coerced into NumPy is numerical. A `Fraction` tuple in the exact coordinate layer remains exact. A JAX or Torch derivative remains numerical.
7. `ExactnessError` is part of the public exception hierarchy, but the current backend and coercion modules do not raise it. Do not promise it as the generic response to every exact/numerical mismatch.

The focused tests cover NumPy linear algebra and nullspace, SymPy rank and nullspace, the fixed availability-report keys, settings-context restoration, exact rational inverse and solvers, modular-domain units and valuations, crash-isolated optional-runtime aborts and timeouts, and JAX/PyTorch directional derivatives through third order when those runtimes are usable. The current suite does not make custom registry replacement, the `backend_name` heuristic, invalid global tolerance policy, or symbolic simplification completeness part of a strong public contract. Treat those as documented implementation boundaries rather than extension guarantees.

> **Chapter 8 summary.** The core matrix protocol uses NumPy by default and optional SymPy for symbolic/exact matrices. `get_backend` selects execution. `available_backends` reports discovery. `backend_name` is only a two-way module heuristic. SciPy, mpmath, JAX, PyTorch, and CuPy remain feature dependencies, while exact rational coordinates and modular domains are independent scalar systems. Exactness follows the path that actually ran. Numerical claims retain their dtype, tolerance, runtime, and certificate assumptions. Chapter 9 now applies these rules to typed operator realizations and declared path relations.

Part III. Operator, packet, and refinement foundations

Reading route. This part builds a finite operator model in layers. You can enter at the layer that matches the data you already have.

Your starting point	Start here	What to keep for the next step
Named spaces and permitted operations	Chapter 9	A grammar, its realized matrices, and registered relations
Square matrices and candidate generating vectors	Chapter 10	The algebra or orbit basis and the corresponding certificate
A decomposition into labeled subspaces	Chapter 11	Packet names, projectors, and the source/target order of blocks
Several resolutions of one decomposition	Chapter 12	The fine-to-coarse parent maps and compatible endpoint systems
Residues, rational lattices, or modular linear equations	Chapter 13	Address conventions, prime, and exact arithmetic data
Signal or arithmetic channels to refine	Chapter 14	Channel order, original shape, and the compatible refinement plan

For a first pass, read the opening of each chapter and run its first small example. Return to the method tables when adapting that example. The common prerequisites are matrix multiplication on column vectors, finite-dimensional span and rank, and the certificate and backend conventions of Chapters 5 and 8. The arithmetic sections state their additional prime and divisibility requirements locally. Keep the labels attached to matrices and coordinates: matching numerical dimensions alone does not establish that two named objects can be composed.

9. Typed operator grammars

First route. Sections 9.1 to 9.3 separate the declarations from their execution. If you prefer to begin with a complete program, use the numerical factorization in Section 9.4.1, then trace its names back through those sections. Use Section 9.4.2 for the SymPy variant. When a path fails, inspect its endpoints before changing a matrix or a tolerance.

Typed operator grammars are the first executable layer of the guide's operator language. They answer three questions before any matrix calculation is allowed to begin:

1. Which spaces, or *types*, exist?
2. Which named operators may act between those spaces?
3. Which sequences of named operators form legal paths?

Only after those structural questions have been answered does an `OperatorRealization` attach dimensions and matrices. This division is useful because the same formal grammar can have several realizations: an exact rational realization, a floating-point approximation, a lower-dimensional model, or a failed candidate whose declared relations can be inspected.

The implemented picture is a finite directed multigraph, or quiver,

$$\mathsf{G} = (V, A, s, t), \qquad a : s(a) \longrightarrow t(a),$$

together with finitely many linear relations among parallel paths. A realization assigns a finite-dimensional space of dimension d_X to each vertex X, and a matrix

$$\rho(a) \in \mathbb{F}^{d_{t(a)} \times d_{s(a)}}$$

to every arrow. The target dimension is the row count and the source dimension is the column count. This shape convention is what makes the grammar's type checking agree with ordinary matrix action on column vectors.

The `[STABLE]` manifest for the reference version `1.0.0b4` contains the six objects used here:

```
from komorebi_math.stable import (
    Arrow,
    OperatorGrammar,
    OperatorRealization,
    OperatorRelation,
    OperatorWord,
    RelationResidual,
)
```

`Arrow`, `OperatorGrammar`, `OperatorRealization`, `OperatorRelation`, and `OperatorWord` are also top-level research exports. `RelationResidual` is not a top-level export in this release. Import it from `komorebi_math.stable` or `komorebi_math.core`. Long-lived code should use the stable import shown above.

The provenance records distinguish the two layers:

Identifier	Object	Status	Implemented scope
`PSC-I-D01`	`OperatorGrammar`	definition	finite vertices, typed arrows, and finite linear path relations
`PSC-I-T01`	`OperatorRealization`	proved	finite-dimensional matrix realization and verification of the declared relations

These labels do not claim that the software constructs an abstract quotient algebra, finds a normal form, or proves a relation for every possible realization. It checks the supplied finite realization.

9.1 `Arrow`, `OperatorWord`, `OperatorRelation`, `OperatorGrammar`

The small records are frozen, slotted dataclasses. Their stable constructor signatures are:

```
Arrow(name: str, source: str, target: str) -> None

OperatorWord(
    arrows: tuple[str, ...],
    source: str,
    target: str,
) -> None

OperatorRelation(
    name: str,
    terms: tuple[tuple[Any, OperatorWord], ...],
) -> None

RelationResidual(
    name: str,
    residual: float,
    satisfied: bool,
) -> None
```

Their fields have these meanings:

Record	Field	Meaning
`Arrow`	`name`	name used to retrieve and compose the generator
	`source`	input vertex
	`target`	output vertex
`OperatorWord`	`arrows`	arrow names in the order in which they act
	`source`	initial vertex
	`target`	final vertex
`OperatorRelation`	`name`	unique name after registration in a grammar
	`terms`	coefficient-word pairs (c_j, w_j)
`RelationResidual`	`name`	registered relation name
	`residual`	backend norm, returned as a Python `float`
	`satisfied`	result of the backend's zero test

An `OperatorWord` also provides two derived properties:

```
OperatorWord.length: int
OperatorWord.is_identity: bool
```

`length` is the number of arrow names. `is_identity` is true exactly when the arrow tuple is empty and the two stored endpoints are equal. Converting a word to text produces `id_X` for the identity at `X`, and joins nonempty words with the centered-dot string `·`.

The preferred way to obtain all three structural records is through an `OperatorGrammar`, not by assembling dataclasses manually. The grammar checks that arrows use known vertices, paths compose, identities name known vertices, and registered relations contain words that agree with the grammar.

9.1.1 Constructing a grammar

The stable grammar constructor is:

```
OperatorGrammar(
    vertices: Iterable[str],
    arrows: Mapping[str, tuple[str, str]] | Iterable[Arrow],
    relations: Iterable[OperatorRelation] = (),
) -> None
```

The common mapping form is concise:

```
grammar = OperatorGrammar(
    ["X", "Y", "Z"],
    {
        "a": ("X", "Y"),
        "b": ("Y", "Z"),
        "c": ("X", "Z"),
    },
)
```

This defines

$$X \xrightarrow{a} Y \xrightarrow{b} Z, \qquad X \xrightarrow{c} Z.$$

The alternative form is useful when arrows have already been constructed:

```
grammar = OperatorGrammar(
    ["X", "Y", "Z"],
    [
        Arrow("a", "X", "Y"),
        Arrow("b", "Y", "Z"),
        Arrow("c", "X", "Z"),
    ],
)
```

The constructor enforces this finite structural contract:

- At least one vertex is required.
- The normalized vertex names must be nonempty.
- Arrow names must be nonempty and unique.
- Every arrow source and target must occur in the vertex tuple.
- Relations passed to the constructor are registered through the same validation path as later relations.

The public read-only views are:

```
OperatorGrammar.vertices  -> tuple[str, ...]
OperatorGrammar.arrows    -> tuple[Arrow, ...]
OperatorGrammar.relations -> tuple[OperatorRelation, ...]
```

Their order is deterministic under ordinary ordered Python inputs. Vertices preserve first occurrence, arrows preserve mapping or iterable order, and relations preserve registration order.

There is one subtle normalization rule. Vertex inputs are converted with `str`, and repeated normalized values are silently collapsed while preserving the first occurrence. Thus `OperatorGrammar(["X", "X", 1], {})` has vertices `("X", "1")`. The source does **not** reject repeated vertex inputs. Arrow names, by contrast, must be unique and are rejected if repeated. Programs that use names as external identifiers should normalize and reject collisions before calling the constructor rather than depending on this permissive vertex rule.

Individual arrows are retrieved with:

```
OperatorGrammar.arrow(self, name: str) -> Arrow
```

An unknown name raises `ValidationError` rather than returning `None`.

9.2 Composition legality and relation syntax

The grammar creates words through:

```
OperatorGrammar.identity(
    self,
    vertex: str,
) -> OperatorWord

OperatorGrammar.word(
    self,
    *arrow_names: str,
    vertex: str | None = None,
) -> OperatorWord
```

For a nonempty word, the arrow names are listed in **application order**. If

$$a : X \to Y, \qquad b : Y \to Z,$$

then:

```
word = grammar.word("a", "b")
```

stores `word.arrows == ("a", "b")`, has endpoints `X -> Z`, prints as `a · b`, and means "apply `a`, then apply `b`." If the realized matrices are $A = \rho(a)$ and $B = \rho(b)$, evaluation is

$$\rho(\texttt{word("a","b")}) = BA,$$

not AB. More generally, for the stored tuple

$$w = (a_1, a_2, \dots, a_k),$$

the implemented product is

$$\rho(w) = \rho(a_k)\rho(a_{k-1}) \cdots \rho(a_1).$$

The implementation starts with the identity on the source space and left multiplies once for each arrow. This convention is consistent with column vectors:

$$x \longmapsto A_1 x \longmapsto A_2 A_1 x \longmapsto \cdots \longmapsto A_k \cdots A_1 x.$$

Adjacent names are legal exactly when

$$t(a_j) = s(a_{j+1}).$$

A failure is reported at the first noncomposable pair. The `vertex` keyword is not used to override or constrain a nonempty word. It supplies the object for the empty word only. These two expressions are equivalent:

```
grammar.identity("X")
grammar.word(vertex="X")
```

Both return the empty path $1_X : X \to X$. Calling `grammar.word()` without a vertex is ambiguous and raises `ValidationError`.

> **Order rule.** Read `word("a", "b")` from left to right as execution, but read its matrix from right to left as multiplication: first A, then B, hence BA.

9.2.1 Linear path relations

A relation is a named finite linear combination of parallel words:

$$R = \sum_{j=1}^{m} c_j w_j : X \to Y.$$

Every word in the sum must have the same source X and target Y. The factory signature is:

```
OperatorGrammar.relation(
    self,
    name: str,
    terms: Sequence[tuple[Any, OperatorWord]],
    *,
    add: bool = True,
) -> OperatorRelation
```

For the three-arrow grammar above, the factorization relation

$$b \circ a - c = 0$$

is entered as:

```
factorization = grammar.relation(
    "factorization",
    [
        (1, grammar.word("a", "b")),
        (-1, grammar.word("c")),
    ],
)
```

Because the stored words are in application order, the first term evaluates as BA. The factory converts the term sequence to a tuple. By default it also registers the relation in the grammar. `add=False` creates and validates the parallel relation record without registering it:

```
candidate = grammar.relation(
    "candidate",
    [(1, grammar.word("c"))],
    add=False,
)
```

An unattached relation can be evaluated explicitly with `realization.relation_matrix(candidate)`, but it is not included in `grammar.relations`, `relation_residuals()`, constructor validation, or the relation certificate until it is registered.

Existing records are registered through:

```
OperatorGrammar.add_relation(
    self,
    relation: OperatorRelation,
) -> None
```

Registration checks that the name has not already been used and that every word is reconstructible in this grammar with consistent endpoints. Identity terms must name known vertices. A relation must have a nonempty name, at least one term, and parallel endpoints.

Coefficients are intentionally backend-generic and annotated `Any`. The grammar does not prove that a coefficient belongs to a field, coerce it, or reject a zero coefficient. Compatibility is tested only when a realization asks its backend to scale and add matrices. Use ordinary numerical scalars with NumPy and exact SymPy scalars with the SymPy backend. A coefficient accepted by one backend need not be accepted by another.

9.2.2 Enumerating finite paths

Legal paths can be enumerated with:

```
OperatorGrammar.paths(
    self,
    *,
    source: str | None = None,
    target: str | None = None,
    max_length: int = 1,
    include_identities: bool = True,
) -> tuple[OperatorWord, ...]
```

The method performs a finite breadth-by-length enumeration:

1. Accepted identities are emitted first when requested.
2. All length-one arrows are considered in arrow order.
3. Each frontier word is extended by every type-compatible arrow.
4. Source and target filters control which words are returned, not which legal intermediate words may be extended.

For the factorization grammar,

```
grammar.paths(source="X", target="Z", max_length=2)
```

returns the direct path `c` followed by the two-step path `a · b`. An identity is included only when its vertex meets both endpoint filters. With `max_length=0`, the result contains only accepted identities, or is empty when `include_identities=False`.

`max_length` must be nonnegative. Unknown source and target filters raise `ValidationError`. Relations do not alter enumeration: declaring `a · b - c = 0` does not remove either path, identify their `OperatorWord` objects, or choose a canonical representative. In a graph with branching or loops, the number of paths can grow exponentially with `max_length`. This method has no separate path-count resource guard. Keep the length bound small unless growth has been estimated.

9.3 `OperatorRealization` and relation residuals

The realization constructor is:

```
OperatorRealization(
    grammar: OperatorGrammar,
    spaces: Mapping[str, int],
    operators: Mapping[str, Any],
    *,
    backend: str | LinearAlgebraBackend | None = None,
    validate_relations: bool = True,
    tolerance: float | None = None,
) -> None
```

The construction contract is exact at the level of keys and dimensions:

- `spaces` must have exactly the grammar's vertex names as keys.
- Every space dimension must be positive after conversion with `int`.
- `operators` must have exactly the grammar's arrow names as keys.
- The matrix for $a : X \to Y$ must have shape (d_Y, d_X).
- Each matrix is converted by the selected backend.
- If registered relations exist and `validate_relations=True`, all must pass before construction returns.

The selected backend is obtained through the mechanism described in Chapter 8. `backend=None` selects NumPy. A registered backend name or a concrete `LinearAlgebraBackend` instance may also be supplied. The default NumPy backend converts matrices to `complex128`. `backend="sympy"` requests the optional exact SymPy backend.

If `tolerance` is omitted, the realization snapshots the current global setting when it is constructed. An explicit value is converted to `float` and stored as `realization.tolerance`. Changing the settings context afterward does not alter an existing realization.

The main evaluator is:

```
OperatorRealization.evaluate(
    self,
    word: OperatorWord,
) -> Any
```

For an identity it returns the backend identity matrix of the realized vertex dimension. For a nonempty word it reconstructs the arrow sequence through the grammar, verifies the stored endpoints, and left multiplies the matrices in application order.

The evaluator does not infer dimensions or repair a manually forged word. Use `grammar.word(...)` and `grammar.identity(...)` so that the path has already crossed the structural boundary.

9.3.1 Relation matrices, residuals, and certificates

For a parallel relation

$$R = \sum_j c_j w_j : X \to Y,$$

the realized relation matrix is

$$\rho(R) = \sum_j c_j \rho(w_j) \in \mathbb{F}^{d_Y \times d_X}.$$

The relevant methods are:

```
OperatorRealization.relation_matrix(
    self,
    relation: OperatorRelation,
) -> Any

OperatorRealization.relation_residuals(
    self,
) -> tuple[RelationResidual, ...]

OperatorRealization.relation_certificate(
    self,
) -> Certificate
```

`relation_matrix` accepts either a registered relation or an unattached parallel relation. It starts from a backend zero matrix of typed shape and adds each scaled word matrix.

`relation_residuals()` processes only the registered relations, in registration order. For each relation it computes

$$r_R = \|\rho(R)\|$$

through `backend.norm`, then asks `backend.is_zero` whether the matrix is satisfied. The returned residual is always a Python `float`, even when the backend is exact.

The NumPy backend uses `numpy.linalg.norm`, which is the Frobenius norm for the two-dimensional matrices used here, and accepts the relation when

$$r_R \leq \tau.$$

The SymPy backend ignores the tolerance and tests exact equality with the zero matrix. Consequently the two evidence modes are:

Backend path	Certificate exactness	Satisfaction rule	Stored tolerance
NumPy	`[NUMERICAL]`	matrix norm at most τ	explicit float
SymPy	`[EXACT]`	exact zero-matrix equality	`None`

The certificate has:

- `valid=True` exactly when every registered residual record is satisfied.
- Residuals mapped by relation name.
- The fixed assumptions-checked tuple `("operator shapes", "typed path composition", "declared relations")`.
- Diagnostics containing the backend name and relation count.
- Theorem identifier `PSC-I-T01`.
- Message `all grammar relations are satisfied` or `one or more grammar relations fail`.

If the grammar has no registered relations, the certificate is vacuously valid, has an empty residual mapping, and reports a relation count of zero. Such a certificate confirms the typed realization boundary. It does not provide evidence for a relation that was never declared.

With the default `validate_relations=True`, an invalid certificate is converted into `RelationError` inside the constructor. Use `validate_relations=False` only when the intended workflow is to retain a failed candidate and inspect its matrix, residual, and certificate. This flag does not bypass missing keys, nonpositive dimensions, wrong operator shapes, or invalid grammar paths.

9.4 Grammar examples and invalid paths

9.4.1 Worked numerical factorization

This example realizes two routes from X to Z: the direct arrow c and the composed path obtained by applying a and then b. It declares that the routes agree.

```
import numpy as np

from komorebi_math.stable import OperatorGrammar, OperatorRealization

grammar = OperatorGrammar(
    ["X", "Y", "Z"],
    {
        "a": ("X", "Y"),
        "b": ("Y", "Z"),
        "c": ("X", "Z"),
    },
)

ab = grammar.word("a", "b")
grammar.relation(
    "factorization",
    [(1, ab), (-1, grammar.word("c"))],
)

A = np.array([[1, 0], [0, 1], [1, 1]], dtype=float)
B = np.array([[1, 2, 0], [0, 1, 1]], dtype=float)
C = B @ A

realization = OperatorRealization(
    grammar,
    {"X": 2, "Y": 3, "Z": 2},
    {"a": A, "b": B, "c": C},
)

print("word:", str(ab), ab.source, "->", ab.target, "length=", ab.length)
print("B @ A:")
print(np.real_if_close(realization.evaluate(ab)))
print(
    "paths X -> Z, length <= 2:",
    [
        str(word)
        for word in grammar.paths(source="X", target="Z", max_length=2)
    ],
)
print("residuals:", realization.relation_residuals())

certificate = realization.relation_certificate()
print(
    "certificate:",
    certificate.valid,
    certificate.exactness.value,
    certificate.residuals,
    certificate.tolerance,
    certificate.theorem_id,
)
```

Expected output under the shipped default tolerance:

```
word: a · b X -> Z length= 2
B @ A:
[[1. 2.]
 [1. 2.]]
paths X -> Z, length <= 2: ['c', 'a · b']
residuals: (RelationResidual(name='factorization', residual=0.0, satisfied=True),)
certificate: True numerical {'factorization': 0.0} 1e-10 PSC-I-T01
```

Three distinct contracts are visible. The grammar establishes that the two routes are parallel. The realization shape-checks A, B, and C. The certificate then verifies numerically that $BA - C$ has zero residual at the stored tolerance.

9.4.2 Exact nilpotent realization

The same interface can use the optional SymPy backend. Here the single loop $N : X \to X$ obeys $N^2 = 0$:

```python
import sympy as sp

from komorebi_math.stable import OperatorGrammar, OperatorRealization

grammar = OperatorGrammar(["X"], {"N": ("X", "X")})
square = grammar.word("N", "N")
grammar.relation("square_zero", [(1, square)])

realization = OperatorRealization(
    grammar,
    {"X": 2},
    {"N": sp.Matrix([[0, 1], [0, 0]])},
    backend="sympy",
)

print("N^2 =")
print(realization.evaluate(square))
print(
    "loops:",
    [
        str(word)
        for word in grammar.paths(
            source="X",
            target="X",
            max_length=2,
        )
    ],
)
print("residual:", realization.relation_residuals()[0])

certificate = realization.relation_certificate()
print(
    "certificate:",
    certificate.valid,
    certificate.exactness.value,
    certificate.residuals,
    certificate.tolerance,
    certificate.theorem_id,
)
```

Expected output:

```
N^2 =
Matrix([[0, 0], [0, 0]])
loops: ['id_X', 'N', 'N · N']
residual: RelationResidual(name='square_zero', residual=0.0, satisfied=True)
certificate: True exact {'square_zero': 0.0} None PSC-I-T01
```

`[EXACT]` here means that the supplied SymPy matrix satisfies the supplied finite relation by exact matrix equality. It does not mean that every operator called N is nilpotent, or that an abstract nilpotent algebra has been classified.

9.4.3 Tolerance is part of a numerical claim

A numerical relation can pass at one tolerance and fail at another. The realized matrix and its residual do not change. The acceptance boundary does:

```python
import numpy as np

from komorebi_math.stable import OperatorGrammar, OperatorRealization

grammar = OperatorGrammar(["X"], {"T": ("X", "X")})
grammar.relation("approximately_zero", [(1, grammar.word("T"))])
matrix = np.array([[5.0e-9]])

loose = OperatorRealization(
    grammar,
    {"X": 1},
    {"T": matrix},
    tolerance=1.0e-8,
)
strict = OperatorRealization(
    grammar,
    {"X": 1},
    {"T": matrix},
    tolerance=1.0e-10,
    validate_relations=False,
)

for name, realization in (("loose", loose), ("strict", strict)):
    residual = realization.relation_residuals()[0]
    print(name, residual.residual, residual.satisfied)
```

Expected output:

```
loose 5e-09 True
strict 5e-09 False
```

If the strict realization had retained the default `validate_relations=True`, its constructor would have raised `RelationError`. Reporting only "the relation passed" without its backend and tolerance would discard part of the mathematical statement.

9.4.4 Path and loop operator helpers

The realization can evaluate a bounded family of grammar paths in one call:

```
OperatorRealization.path_operators(
    self,
    *,
    source: str | None = None,
    target: str | None = None,
    max_length: int = 1,
    include_identities: bool = True,
) -> dict[OperatorWord, Any]

OperatorRealization.loop_operators(
    self,
    vertex: str,
    *,
    max_length: int,
) -> tuple[Any, ...]
```

`path_operators` preserves the `OperatorWord` objects as dictionary keys, so a caller can retain type and path labels beside each matrix. It delegates all enumeration rules to `grammar.paths` and evaluates every returned word.

`loop_operators` is a narrower convenience method. It returns matrices for all paths from a vertex back to itself through the requested length, always including the identity. It discards the word labels. When labels matter, use `path_operators(source=vertex, target=vertex, ...)` instead.

Neither helper removes duplicate matrices. Distinct formal paths remain distinct dictionary keys even when a declared relation or the particular realization makes their matrices equal. This is the correct boundary for the next chapter: an `OperatorAlgebra` can study the span and closure of realized matrices, while the grammar continues to preserve their formal addresses.

9.4.5 Controlled failures

This example exercises the main structural and relation failures:

```
import numpy as np

from komorebi_math.stable import OperatorGrammar, OperatorRealization


def show_failure(label, action):
    try:
        action()
    except Exception as error:
        print(f"{label}: {type(error).__name__}: {error}")


paths = OperatorGrammar(
    ["X", "Y"],
    {"a": ("X", "Y"), "b": ("X", "Y")},
)

show_failure("noncomposable", lambda: paths.word("a", "b"))
show_failure("empty word", lambda: paths.word())
show_failure(
    "nonparallel",
    lambda: paths.relation(
        "bad",
        [(1, paths.word("a")), (-1, paths.identity("X"))],
    ),
)
```

```
grammar = OperatorGrammar(["X"], {"T": ("X", "X")})
grammar.relation("square_zero", [(1, grammar.word("T", "T"))])

show_failure(
    "failed relation",
    lambda: OperatorRealization(
        grammar,
        {"X": 2},
        {"T": np.eye(2)},
    ),
)

candidate = OperatorRealization(
    grammar,
    {"X": 2},
    {"T": np.eye(2)},
    validate_relations=False,
)
print("inspection:", candidate.relation_residuals(), candidate.relation_certificate().valid)

typed = OperatorGrammar(["X", "Y"], {"a": ("X", "Y")})
show_failure(
    "wrong shape",
    lambda: OperatorRealization(
        typed,
        {"X": 2, "Y": 3},
        {"a": np.eye(2)},
    ),
)
```

Expected output:

```
noncomposable: ValidationError: path is not composable at 'a', 'b': 'Y' != 'X'
empty word: ValidationError: an empty word requires its identity vertex
nonparallel: RelationError: all words in a relation must have the same source and target
failed relation: RelationError: one or more grammar relations fail
inspection: (RelationResidual(name='square_zero', residual=1.4142135623730951, satisfied=False),) False
wrong shape: DimensionError: operator 'a' has shape (2, 2), expected (3, 2)
```

The failure order is intentional. Grammar errors are caught before matrix work. Key and dimension errors are caught while constructing a realization. Declared relation failures are caught last, after matrices have valid typed shapes. `validate_relations=False` changes only that final relation gate.

The relevant exceptions are:

Failure	Exception
empty vertex set, empty vertex name, duplicate arrow, unknown arrow vertex	`ValidationError`
unknown arrow or identity vertex, illegal composition, inconsistent word endpoints	`ValidationError`
empty or nonparallel relation, duplicate relation name, unknown identity vertex, inconsistent stored endpoints	`RelationError`
unknown arrow or illegal composition inside a manually forged relation word	`ValidationError`
nonpositive space dimension or wrong operator shape	`DimensionError`
space or operator key mismatch	`ValidationError`
unavailable backend	`BackendUnavailableError`
failed registered relation during default construction	`RelationError`
negative `max_length`	built-in `ValueError`

Failure	Exception

As in Chapter 6, callers should catch by exception type rather than depending on exact diagnostic strings.

9.4.6 Contract boundaries and source-audit corrections

The implementation is finite by design and smaller than a general symbolic rewriting system. These boundaries prevent common overclaims:

1. **The grammar is not a parser.** It accepts Python objects and named arrows. It does not parse a free-form algebraic expression such as `b*a-c=0`.
2. **Relations are checked, not used as rewrite rules.** There is no Gröbner basis, reduction order, canonical word, or quotient-path enumeration.
3. **A valid certificate concerns one realization.** It verifies the matrices, relations, backend zero test, and tolerance supplied to that object. It does not prove a universal identity over all representations.
4. **Path enumeration is syntactic.** Equal realized matrices do not merge paths, and declared relations do not shorten the enumeration.
5. **Dimensions are supplied, not inferred.** Every vertex dimension and every arrow matrix must be present.
6. **Coefficients are backend-dependent.** The grammar does not validate a scalar domain or transport coefficients between NumPy and SymPy.
7. **Exact symbolic completeness is not promised.** The SymPy path uses exact matrix equality, but arbitrary unsimplified or parameter-dependent symbolic expressions may not reduce to a decidable zero and may not convert to the floating residual required by `RelationResidual`.
8. **A direct dataclass constructor is not the complete validator.** `Arrow` and `OperatorWord` records do not validate themselves. `OperatorRelation` checks name, nonemptiness, and parallel endpoints, while grammar membership is checked only when the relation is added. Prefer grammar factories.
9. **Repeated vertex inputs are normalized away.** Nonempty normalized vertex names are enforced, and duplicates are silently collapsed. Arrow names must be unique.
10. **`RelationResidual` has a narrower top-level import story.** It is stable and exported by `komorebi_math.stable` and `komorebi_math.core`, but not by the top-level `komorebi_math` namespace in `1.0.0b4`.
11. **The theorem identifier is provenance, not a proof transcript.** The certificate records `PSC-I-T01` and finite diagnostics. It does not contain a derivation of the abstract quotient correspondence.
12. **Finite does not mean automatically cheap.** A bounded path call always terminates, but branching and loops can make the result exponentially large in the length bound.

The focused source tests establish typed endpoints and identities, rejection of noncomposable paths and nonparallel relations, a valid numerical cubic relation, constructor rejection of a failed relation, exact SymPy nilpotence, and bounded loop enumeration. The stable semantic fixture additionally freezes one idempotent operator case with a valid certificate, one registered relation, and maximum residual zero. The installed core verifier checks the cubic relation $T^3 - T = 0$ for $T = \mathrm{diag}(-1, 0, 1)$.

The tests do not promise coefficient-field validation, quotient normal forms, large path-enumeration performance, arbitrary symbolic simplification, or special behavior for manually forged dataclass records. Those remain outside the chapter's public claim.

> **Chapter 9 summary.** Vertices are types, arrows are named generators, and `OperatorWord` stores a legal path in application order. Parallel paths form finite `OperatorRelation` records. `OperatorRealization` attaches a positive dimension to every vertex and a target-by-source matrix to every arrow, evaluates `word("a", "b")` as BA, and checks each registered relation exactly with SymPy or numerically with a declared tolerance under NumPy. The certificate is evidence about the supplied finite realization. Formal quotient reduction and generated matrix-algebra closure belong to later layers.

10. Operator algebras and seed systems

Choose the question first. To find the span of products of your matrices, use Section 10.1. To find the vectors reached from supplied seeds, use Section 10.2. These computations return bases of different spaces. Begin with the nilpotent example to see both together, then read Section 10.3 only when your problem meets the structural assumptions of a seed-rank formula.

Chapter 9 separates a symbolic operator grammar from a concrete realization. This chapter takes the next finite step. Once the generators have been realized as square matrices, Komorebi Math can close them under linear combination and multiplication, then ask whether a finite family of vectors generates the whole representation under that algebra.

The two central objects answer different questions:

Object	Finite question
`OperatorAlgebra`	Which independent matrices are produced by multiplying the realized generators?
`SeedSystem`	Which independent vectors are reached by applying the realized operators to the supplied seeds?

Both questions are finite because an algebra of $n \times n$ matrices has dimension at most n^2, while an orbit hull inside K^n has dimension at most n. The numerical backend decides independence relative to a tolerance. The SymPy backend decides it exactly.

> **Implementation map.** Generated matrix algebras are implemented in `komorebi_math.core.algebra`, and seed systems are implemented in `komorebi_math.core.seeds`. Despite its filename, `komorebi_math.core.operators` implements `PacketOperator`. That object belongs to Chapter 11 rather than the generated-algebra layer described here.

10.1 Finite generated algebras

Let K be the scalar domain, let $V = K^n$, and let

$$\mathcal{G} = \{A_1, \dots, A_m\} \subseteq \mathrm{End}_K(V).$$

With the identity included, the realized generated algebra is

$$\mathrm{Alg}_K(\mathcal{G}) = \mathrm{span}_K \left\{ I, A_{i_1}, A_{i_2}A_{i_1}, \dots \right\}.$$

It is the smallest unital linear subspace of $M_n(K)$ that contains the generators and is closed under matrix multiplication. Since

$$\dim_K M_n(K) = n^2,$$

the independent-product search must stabilize after adding at most n^2 basis matrices.

The executable constructor is

```
OperatorAlgebra(
    generators: Mapping[str, Any] | Sequence[Any],
    *,
    backend: str | LinearAlgebraBackend | None = None,
    tolerance: float | None = None,
```

```
    include_identity: bool = True,
) -> None
```

If `generators` is a mapping, its keys are converted to strings and retained as generator names. A sequence is named `A0`, `A1`, and so on. Every value is converted by the selected backend and must have the same square shape. The common row dimension becomes `degree`.

The public state relevant to computation is:

Attribute	Meaning
`backend`	Active `LinearAlgebraBackend` instance
`tolerance`	Explicit tolerance or the active global setting at construction time
`generators`	Dictionary of backend matrices
`degree`	Common matrix size n
`include_identity`	Whether I_n is inserted before closure
`dimension`	Number of matrices in the computed basis

The closure algorithm

`basis()` uses a rank-increment closure rather than enumerating words. It can be written schematically as follows.

1. Start with an empty list B.
2. If `include_identity=True`, add I_n when it is nonzero and independent.
3. Process the generators in insertion order, retaining only rank-increasing matrices.
4. Freeze the current basis as a snapshot and test every product XY with X, Y in that snapshot.
5. Add each product that increases the flattened matrix rank.
6. Repeat from step 4 until a whole sweep adds nothing or $|B| = n^2$.

For numerical matrices, a matrix X is independent of $B = (B_1, \dots, B_d)$ precisely when the backend reports

$$\operatorname{rank}_\tau \begin{bmatrix} \operatorname{vec}(B_1) & \cdots & \operatorname{vec}(B_d) & \operatorname{vec}(X) \end{bmatrix} > \operatorname{rank}_\tau \begin{bmatrix} \operatorname{vec}(B_1) & \cdots & \operatorname{vec}(B_d) \end{bmatrix}.$$

Here τ is `tolerance`. The NumPy backend flattens in row-major order and uses `numpy.linalg.matrix_rank(..., tol=tolerance)`. The SymPy backend reshapes to an $n^2 \times 1$ column and computes exact rank. Its rank calculation ignores the numerical tolerance.

The basis is cached after its first construction. Repeated calls to `basis()` or `dimension` therefore do not repeat the closure search.

Methods

```
basis(self) -> tuple[Any, ...]
contains(self, matrix: Any) -> bool
certificate(self) -> Certificate
seed_system(self, seeds: Sequence[Any]) -> SeedSystem
transfer_graph(self, packets: PacketSystem) -> TransferGraph
```

`contains(X)` tests whether adjoining X changes the flattened basis rank. A shape other than `(degree, degree)` returns `False`. `seed_system(seeds)` uses the computed algebra basis itself as an acting operator family, named `B0`, `B1`, and so on. `transfer_graph(packets)` is the bridge into Chapter 11: it builds packet operators from the original generators, not from every matrix in the closed basis.

Executed example: a nilpotent algebra and a cyclic seed

```python
import numpy as np

from komorebi_math import OperatorAlgebra

N = np.array([[0.0, 1.0], [0.0, 0.0]])
algebra = OperatorAlgebra({"N": N})

basis = [
    np.real_if_close(value).astype(int).tolist()
    for value in algebra.basis()
]

seeds = algebra.seed_system([np.array([0.0, 1.0])])

print("algebra dimension:", algebra.dimension)
print("basis:", basis)
print("contains 3I+4N:", algebra.contains(3 * np.eye(2) + 4 * N))
print("contains N.T:", algebra.contains(N.T))
print("hull dimension:", seeds.hull_dimension())
print("orbit layers:", [len(layer) for layer in seeds.orbit_layers()])
print("algebra certificate:", algebra.certificate().valid)
print("seed certificate:", seeds.generation_certificate().valid)
```

Output:

```
algebra dimension: 2
basis: [[[1, 0], [0, 1]], [[0, 1], [0, 0]]]
contains 3I+4N: True
contains N.T: False
hull dimension: 2
orbit layers: [1, 1]
algebra certificate: True
seed certificate: True
```

Because $N^2 = 0$, the algebra is exactly

$$\mathcal{A} = \operatorname{span}\{I, N\}.$$

The seed e_2 produces $Ne_2 = e_1$, so one seed generates the two-dimensional representation. The NumPy certificates are still *numerical*: validity means the finite rank tests passed at the declared tolerance, not that an exact symbolic calculation was performed.

Exact backend variant

With the optional exact dependency installed, the same algebra can be computed over SymPy:

```
import sympy as sp

from komorebi_math import OperatorAlgebra

N = sp.Matrix([[0, 1], [0, 0]])
algebra = OperatorAlgebra({"N": N}, backend="sympy")

print("dimension:", algebra.dimension)
print("basis:", [value.tolist() for value in algebra.basis()])
print("exactness:", algebra.certificate().exactness.value)
```

Output:

```
dimension: 2
basis: [[[1, 0], [0, 1]], [[0, 1], [0, 0]]]
exactness: exact
```

Matrix units and the full algebra

```
matrix_units(
    size: int,
    *,
    backend: str | LinearAlgebraBackend | None = None,
) -> tuple[Any, ...]
```

`matrix_units(n)` returns the n^2 standard matrices E_{ij}, in row-major `(row, column)` order. They satisfy

$$E_{ij}E_{k\ell} = \delta_{jk}E_{i\ell}$$

and span $M_n(K)$. Consequently,

```
len(matrix_units(2))                        # 4
OperatorAlgebra(matrix_units(2)).dimension   # 4
```

Both values are `4` in the current implementation.

Identity boundary

`include_identity=False` is a genuine semantic switch. It does not merely hide the identity from the returned basis. It asks for the multiplicatively closed linear span of the supplied generators without forcing a unit. For the nilpotent matrix above,

```
nonunital = OperatorAlgebra([N], include_identity=False)
print(nonunital.dimension, nonunital.certificate().valid)
```

prints

```
1 True
```

because span$\{N\}$ is closed under multiplication. Thus the class docstring's phrase "unital … algebra" applies to the default configuration. A certificate with `include_identity=False` does not require that I_n be present.

10.2 `SeedSystem` and cyclic hulls

For an acting operator family

$$\mathcal{T} = \{T_1, \dots, T_m\} \subseteq \mathrm{End}_K(V)$$

and seeds $S = \{s_1, \dots, s_r\} \subseteq V$, the finite cyclic hull is

$$\mathcal{H}_{\mathcal{T}}(S) = \mathrm{span}_K \left\{ T_{i_k} \cdots T_{i_1} s_j : k \geq 0, 1 \leq j \leq r \right\}.$$

The length-zero words include the seeds themselves. The system is generating when

$$\mathcal{H}_{\mathcal{T}}(S) = V.$$

The constructor is

```
SeedSystem(
    operators: Mapping[str, Any] | Sequence[Any],
    seeds: Sequence[Any],
    *,
    backend: str | LinearAlgebraBackend | None = None,
    tolerance: float | None = None,
) -> None
```

At least one seed is required. Empty operator families are allowed. In that case the hull is simply the linear span of the seeds. Mapping keys are converted to strings, while a sequence receives the names `A0`, `A1`, and so on. Every seed must have one common dimension n, and every operator must have shape $n \times n$.

The main state is:

Attribute or property	Meaning
`backend`	Active backend
`tolerance`	Rank and zero threshold captured at construction
`operators`	Named acting matrices
`seeds`	Backend vectors
`dimension`	Ambient vector-space dimension
`exact`	`True` exactly when the backend declares exact arithmetic

Independent frontier closure

`hull_vectors()` carries out a breadth-first independence search.

1. Add each nonzero, rank-increasing seed to a basis and to a queue.
2. Treat the current queue length as one orbit layer.
3. Apply every operator, in insertion order, to every vector in that layer.
4. Add only nonzero vectors that increase the current basis rank.
5. Stop when the queue is empty, the basis reaches dimension n, or the round limit is reached.

If `max_rounds` is omitted, the implemented limit is

$$L = n \max(1, m) + 2,$$

where m is the number of operators. Since every accepted vector increases a rank bounded by n, this is a conservative finite guard. A caller-supplied round limit is a truncation of the orbit search, not a proof that the omitted orbit contains nothing new.

The related methods are

```
hull_vectors(self, *, max_rounds: int | None = None) -> tuple[Any, ...]
hull_basis(self, *, max_rounds: int | None = None) -> Any
hull_dimension(self, *, max_rounds: int | None = None) -> int
is_generating(self) -> bool
generation_certificate(self) -> Certificate
orbit_layers(self, *, max_rounds: int | None = None) -> tuple[tuple[Any, ...], ...]
```

`hull_basis()` stacks accepted vectors as columns. If none survive, it returns a backend zero matrix of shape `(dimension, 0)`. `orbit_layers()` repeats the same rank-increment principle but groups vectors by the round in which they first appear. `is_generating()` and `generation_certificate()` always use the default full closure. They do not inherit a previous truncated `max_rounds` call.

Executed example: one seed for a cyclic shift

```
import numpy as np

from komorebi_math import SeedSystem, cyclic_span


def cyclic_shift(size: int) -> np.ndarray:
    shift = np.zeros((size, size))
    for column in range(size):
        shift[(column + 1) % size, column] = 1.0
    return shift


shift = cyclic_shift(5)
seed = np.eye(5)[:, 0]
system = SeedSystem({"S": shift}, [seed])
result = cyclic_span({"S": shift}, [seed])

print("layers:", [len(layer) for layer in system.orbit_layers()])
print("hull shape:", result.value.shape)
print("valid:", result.certificate.valid)
print("exactness:", result.certificate.exactness.value)
print("diagnostics:", result.certificate.diagnostics)
print("zero rounds:", system.hull_dimension(max_rounds=0))
```

Output:

```
layers: [1, 1, 1, 1, 1]
hull shape: (5, 5)
valid: True
exactness: numerical
diagnostics: {'ambient_dimension': 5, 'hull_dimension': 5, 'operator_count': 1, 'seed_count': 1, 'backend':
    ↪ 'numpy'}
zero rounds: 1
```

The layers are e_1, e_2, e_3, e_4, e_5. The last line is intentionally different: at zero expansion rounds, only the independent starting seed has been admitted.

The convenience wrapper has the exact signature

```
cyclic_span(
    operators: Mapping[str, Any] | Sequence[Any],
    seeds: Sequence[Any],
    *,
    backend: str | LinearAlgebraBackend | None = None,
    tolerance: float | None = None,
) -> ComputationResult[Any]
```

It constructs a `SeedSystem`, places `hull_basis()` in `result.value`, and places `generation_certificate()` in `result.certificate`. A proper hull is therefore a normal returned computation with an invalid certificate, not an exception. Use `result.require()` when a generating hull is a precondition for the next step.

Proper hull

If

$$D = \operatorname{diag}(1, 2, 3), \qquad s = e_1,$$

then $D^k s = e_1$ for every k. The executed result is

```
hull_dimension = 1
certificate.valid = False
```

The computation is still well-defined. What fails is the claim that the chosen seeds generate the full ambient space.

Evaluation presentations

Given a supplied algebra basis or spanning family $B = (B_1, \dots, B_q)$ and seeds $S = (s_1, \dots, s_r)$, the evaluation map is

$$\operatorname{ev}_{B,S} : K^{qr} \longrightarrow K^n, \qquad (c_{ja}) \longmapsto \sum_{j=1}^{r} \sum_{a=1}^{q} c_{ja} B_a s_j.$$

The columns are ordered seed first, then algebra matrix:

$$B_1 s_1, \dots, B_q s_1, B_1 s_2, \dots, B_q s_2, \dots .$$

The method

```
presentation(self, algebra_basis: Sequence[Any]) -> FreePresentation
```

returns the record

Field	Meaning
`evaluation_matrix`	The $n \times qr$ matrix of evaluated seed images
`kernel_basis`	Backend nullspace vectors for the evaluation map
`rank`	Rank of the evaluation matrix
`domain_dimension`	Number qr of evaluation columns
`codomain_dimension`	Ambient dimension n
`backend`	Backend name
`nullity`	Property `domain_dimension - rank`

The caller, not the method, is responsible for supplying an algebra basis or a spanning family. `presentation()` checks only that the sequence is nonempty and that every matrix has the correct shape. It does not check linear independence or multiplicative closure.

For the flip matrix T and seed e_1, the family (I, T) evaluates to (e_1, e_2):

```
flip = np.array([[0.0, 1.0], [1.0, 0.0]])
presentation = SeedSystem(
    [flip], [np.array([1.0, 0.0])]
).presentation([np.eye(2), flip])

print(np.real_if_close(presentation.evaluation_matrix).astype(int).tolist())
print(presentation.rank, presentation.nullity)
print(presentation.domain_dimension, presentation.codomain_dimension)
```

Output:

```
[[1, 0], [0, 1]]
2 0
2 2
```

Smallest standard-basis diagnostic

```
minimum_standard_basis_seeds(
    self,
    *,
    max_dimension: int = 12,
    force: bool = False,
) -> StandardSeedSearchResult
```

This method examines subsets of the ambient standard basis $\{e_1, \dots, e_n\}$ in increasing cardinality and lexicographic combination order. The first generating subset produces

Field	Meaning
`count`	Cardinality of the first generating standard-basis subset
`indices`	Zero-based standard-basis indices
`certificate`	Generation certificate for that subset

It does **not** compute the unrestricted seed rank. For example, a diagonal matrix with three distinct eigenvalues is cyclic: the single vector $(1, 1, 1)^T$ generates under its powers. No single standard basis vector does.

```
diagonal = np.diag([1.0, 2.0, 3.0])
system = SeedSystem([diagonal], [np.ones(3)])
search = system.minimum_standard_basis_seeds()

print("original generating:", system.is_generating())
print("standard subset:", search.count, search.indices, search.certificate.valid)
```

Output:

```
original generating: True
standard subset: 3 (0, 1, 2) True
```

This distinction is essential: `count == 3` describes the restricted standard-coordinate search, while the displayed system already proves that an unrestricted one-vector generating family exists.

10.3 Semisimple, single-operator, direct-sum, and tensor seed ranks

Komorebi Math also exposes finite formulas and bounds that do not require an orbit search. They answer different structural questions and should not be interchanged.

Semisimple seed rank

Suppose a semisimple Artinian algebra acts on a module whose simple block types have dimensions $d_i > 0$ and multiplicities $m_i \geq 0$. The implemented formula is

$$r_{\mathrm{ss}} = \max_i \left\lceil \frac{m_i}{d_i} \right\rceil .$$

```
semisimple_seed_rank(
    block_dimensions: Sequence[int],
    multiplicities: Sequence[int],
) -> int
```

The two sequences must be nonempty and have equal length. Dimensions must be positive and multiplicities nonnegative. The provenance record is `PSC-I-T10`, “Semisimple seed rank formula,” with the assumption of a semisimple Artinian acting algebra.

Executed example:

```
semisimple_seed_rank([2, 4], [5, 9])
```

returns `3`, because

$$\max\{\lceil 5/2\rceil, \lceil 9/4\rceil\} = 3.$$

This function accepts the structural dimensions and multiplicities as data. It does not inspect a matrix representation to prove semisimplicity or infer those values.

Exact seed rank for one rational operator

For one rational matrix $T \in M_n(\mathbb{Q})$, regard $\mathbb{Q}^n$ as a $\mathbb{Q}[t]$-module with $t \cdot v = Tv$. Factor the characteristic polynomial over $\mathbb{Q}$:

$$\chi_T(t) = \prod_p p(t)^{e_p},$$

where the p are distinct monic irreducibles. The number of primary cyclic blocks belonging to p is

$$b_p = \frac{\operatorname{nullity} p(T)}{\deg p},$$

and the implemented seed rank is

$$r_T = \max_p b_p.$$

```
single_operator_seed_rank(operator: Any) -> int
```

The current executable path is exact and rational only. It converts the input to `sympy.Matrix`, requires a square matrix with entries in `sympy.QQ`, factors the characteristic polynomial over $\mathbb{Q}$, evaluates each irreducible factor at T by Horner’s rule, and computes exact ranks. The provenance record is `PSC-I-T13`, “Single operator primary seed rank formula.”

```
import sympy as sp

from komorebi_math import single_operator_seed_rank

jordan2 = sp.Matrix([[4, 1], [0, 4]])
jordan1 = sp.Matrix([[4]])
rotation = sp.Matrix([[0, -1], [1, 0]])

print(single_operator_seed_rank(sp.diag(jordan2, jordan1)))
print(single_operator_seed_rank(sp.diag(rotation, rotation)))
```

Output:

```
2
2
```

In the first case $p(t) = t - 4$, and $T - 4I$ has nullity 2. In the second, $p(t) = t^2 + 1$. Two repeated rational irreducible blocks give nullity $p(T)/2 = 4/2 = 2$.

An empty 0×0 square matrix returns `0`. Missing SymPy support raises `BackendUnavailableError`. Irrational or otherwise nonrational entries raise `ValidationError` rather than silently changing the coefficient field.

Direct-sum and tensor helpers

The remaining helpers are pure integer constructions:

```
shared_direct_sum_seed_bounds(seed_ranks: Sequence[int]) -> tuple[int, int]
independent_direct_sum_seed_rank(seed_ranks: Sequence[int]) -> int
tensor_seed_rank_upper_bound(left_rank: int, right_rank: int) -> int
```

For component ranks $r_1, \dots, r_k$, the declared shared-action bounds are

$$\max_i r_i \le r_{\text{shared}} \le \sum_i r_i.$$

For the library's independent direct-sum construction, the returned rank is

$$r_{\text{independent}} = \sum_i r_i.$$

For tensor products, the function supplies only the product upper bound

$$r_{\otimes} \le r_L r_R.$$

Executed values are:

```
shared_direct_sum_seed_bounds([2, 5, 3])       # (5, 10)
independent_direct_sum_seed_rank([2, 5, 3])    # 10
tensor_seed_rank_upper_bound(3, 4)             # 12
```

These functions do not receive operators, modules, or certificates. Their outputs are valid only after the caller has matched the mathematical situation to the named construction. In particular, the tensor result is an upper bound, not a claim of equality.

10.4 Algebra and seed certificates

The certificate interfaces follow Chapter 5. They record the finite computation that was actually performed, including its exactness boundary.

Algebra certificate

`OperatorAlgebra.certificate()` checks every product of returned basis matrices, all original generators, and, when requested, the identity. Its validity law is

$$\text{valid} = \text{closed} \wedge \text{contains_generators} \wedge \text{contains_required_identity}.$$

The returned fields are populated as follows:

Certificate component	Value
`exactness`	`EXACT` for an exact backend, otherwise `NUMERICAL`
`tolerance`	`None` for exact arithmetic, otherwise the algebra tolerance
`residuals`	Empty mapping in the current implementation
`assumptions_checked`	generator dimensions, linear independence, multiplicative closure
`diagnostics`	degree, algebra dimension, maximum dimension, backend
`theorem_id`	`PSC-I-D05`
success message	`valid generated operator algebra`
failure message	`operator algebra closure failed`

`PSC-I-D05` is a provenance definition, "Realized operator algebra," under the assumption of finite matrix generators.

Seed generation certificate

`SeedSystem.generation_certificate()` certifies only the equality between the computed full hull dimension and the ambient dimension:

$$\text{valid} \iff \dim \mathcal{H}_{\mathcal{T}}(S) = \dim V.$$

Its populated fields are:

Certificate component	Value
`exactness`	`EXACT` for an exact backend, otherwise `NUMERICAL`
`tolerance`	`None` for exact arithmetic, otherwise the seed-system tolerance
`residuals`	Empty mapping in the current implementation
`assumptions_checked`	operator dimensions, seed dimensions, finite orbit closure
`diagnostics`	ambient dimension, hull dimension, operator count, seed count, backend
`theorem_id`	`PSC-I-T02`
success message	`the seeds generate the ambient space`
failure message	`the seed hull is proper`

The provenance theorem `PSC-I-T02`, "Finite seed hull generation," assumes a finite-dimensional realized operator family and explicitly records that numerical rank depends on the declared tolerance.

Neither certificate stores the basis itself. The algebra basis comes from `algebra.basis()`, while the orbit basis comes from `system.hull_basis()` or the value inside `cyclic_span(...)`.

Result records and their exact fields

Three related records appear here:

```
@dataclass(frozen=True, slots=True)
class AlgebraBasisResult:
    basis: tuple[Any, ...]
    dimension: int
    certificate: Certificate

@dataclass(frozen=True, slots=True)
class FreePresentation:
    evaluation_matrix: Any
    kernel_basis: tuple[Any, ...]
    rank: int
    domain_dimension: int
    codomain_dimension: int
    backend: str

    @property
    def nullity(self) -> int: ...

@dataclass(frozen=True, slots=True)
class StandardSeedSearchResult:
    count: int
    indices: tuple[int, ...]
    certificate: Certificate
```

`FreePresentation` and `StandardSeedSearchResult` are returned by the methods documented above and are exported from `komorebi_math.core`. `AlgebraBasisResult` is also exported, including at package top level, but the current `OperatorAlgebra.basis()` method returns `tuple[Any, ...]` directly. The guide therefore does not imply a currently nonexistent `AlgebraBasisResult` return path.

`SeedSystem` is part of the frozen stable namespace. `OperatorAlgebra` and the rank/construction helpers are available from the broader top-level research namespace. Code that requires only the stable contract should import `SeedSystem` from `komorebi_math.stable`.

Failure and caution table

Operation	Boundary	Behavior
`OperatorAlgebra([])`	No generator supplied	`ValidationError`
algebra construction	First generator nonsquare	`DimensionError`
algebra construction	Generator sizes differ	`DimensionError`
`contains(X)`	X has the wrong square size	Returns `False`
`matrix_units(size)`	`size < 1`	`ValueError`
`SeedSystem(..., seeds=[])`	No seed supplied	`ValidationError`
seed construction	Seed dimensions differ	`DimensionError`
seed construction	Operator is not $n \times n$	`DimensionError`
`presentation([])`	No supplied algebra matrix	`ValidationError`
`presentation(B)`	A matrix has the wrong size	`DimensionError`
standard-basis search	$n >$ `max_dimension` and not forced	`ValidationError`
semisimple formula	Empty/mismatched data or invalid dimensions	`ValidationError`
direct-sum helpers	Empty or negative rank data	`ValidationError`
tensor bound	Negative rank	`ValidationError`
single-operator formula	SymPy absent	`BackendUnavailableError`
single-operator formula	Nonsquare matrix	`DimensionError`
single-operator formula	Entry outside $\mathbb{Q}$	`ValidationError`

Operation	Boundary	Behavior
single-operator formula	Primary nullity not divisible by factor degree	`ArithmeticError`

Additional practical boundaries matter:

- `max_rounds=0` intentionally returns only the independent seeds. Negative round values are not rejected by the constructor and also prevent expansion. Callers should use nonnegative values.
- The constructors convert an explicit tolerance with `float(tolerance)` but do not separately validate that it is nonnegative. A negative rank threshold has no useful mathematical interpretation and must not be supplied.
- Numerical closure can merge nearly dependent matrices or vectors, or separate perturbations that should be identified in an ideal exact model. Record the tolerance whenever a dimension claim is reported.
- The SymPy backend makes the rank decisions exact, but exact characteristic polynomial factorization and exact nullspaces can grow expensive as matrix size and coefficient height increase.
- `minimum_standard_basis_seeds(force=True)` removes only the dimension guard. It does not change the exponential search.
- The algebra basis is cached. The exposed generator dictionary and backend matrices should be treated as construction-time data rather than mutated after a basis has been computed.

Complexity and resource profile

Let n be the matrix degree, m the number of operators, $d_A \leq n^2$ the generated-algebra dimension, $d_H \leq n$ the orbit-hull dimension, q the length of a supplied algebra spanning family, and r the seed count.

Computation	Finite scale	Dominant work
Algebra closure	At most $d_A \leq n^2$ accepted matrices	Repeated $n \times n$ products and ranks of flattened $n^2 \times k$ matrices
Algebra storage	d_A matrices	$O(d_A n^2)$, at most $O(n^4)$ scalar entries
Seed hull	At most $d_H \leq n$ accepted vectors	Up to m matrix-vector applications per expanded independent vector, plus repeated ranks
Seed storage	d_H vectors	$O(d_H n)$, at most $O(n^2)$ scalar entries
Presentation	qr evaluated columns	Matrix-vector products followed by rank and nullspace of an $n \times qr$ matrix
Standard-basis search	Up to $2^n - 1$ nonempty subsets	A fresh finite hull computation for each tested subset
Single-operator rank	One exact $n \times n$ rational matrix	Characteristic polynomial, factorization over $\mathbb{Q}$, polynomial evaluation, exact ranks

The algebra and hull searches are bounded by finite-dimensional rank, but that does not make every instance cheap. The explicit `max_dimension=12` guard marks the library's clearest combinatorial boundary in this chapter.

What the shipped tests establish

The focused algebra, seed, and packet-operator replay executed 24 tests with 24 passes. Seventeen directly exercise the algebra and seed layer. The remaining seven confirm the adjacent packet-operator bridge. The Chapter 10 portion establishes at least these finite cases:

- Matrix units generate the full 2×2 algebra.
- A 3×3 Jordan block generates a three-dimensional polynomial algebra.
- A nilpotent generator produces an exact two-dimensional SymPy algebra.
- Algebra generators induce the expected reachable packet transfer graph.
- One seed generates a five-state cyclic shift, layer by layer.
- A coordinate-axis seed for a diagonal matrix has a proper one-dimensional hull.
- An evaluation presentation can have rank two and nullity zero.
- The standard-basis search returns all three coordinate vectors for a three-dimensional diagonal action.
- The dimension guard rejects the default search in dimension 13.
- The semisimple formula matches three supplied block profiles.
- The rational single-operator formula handles repeated linear and irreducible quadratic primary blocks.
- The rational path rejects an irrational SymPy entry.
- The direct-sum and tensor helpers return their declared values.

These tests validate implemented finite contracts and representative examples. They do not prove that a numerical tolerance is appropriate for arbitrary data, infer semisimplicity from a representation, turn the standard-basis diagnostic into an unrestricted seed-rank algorithm, or remove the exact-factorization cost boundary.

> **Chapter 10 summary.** `OperatorAlgebra` closes finite realized generators under multiplication and rank-increasing span. `SeedSystem` closes finite seeds under an acting family. Their certificates report whether those finite closures satisfy the requested algebra or generation condition at the backend's exactness level. Structural rank helpers add exact formulas and named bounds, but only under their stated semisimple, rational single-operator, direct-sum, or tensor assumptions.

11. Packet systems and transfer blocks

First route. Start with analysis and synthesis in Section 11.1.2, then use Section 11.2 to represent an operator between packets. Section 11.3 is the next step when you need reachability or paths. Before interpreting a returned array, identify whether it is an ambient vector, an embedded operator block, or a Boolean adjacency matrix. Their shapes and row conventions differ.

A packet system resolves one finite vector space into labeled pieces. A packet operator then resolves a linear map into every target-by-source transfer block, and a transfer graph forgets the coefficients while retaining which packet transfers are structurally active. These three levels answer different questions:

Level	Stored object	Question answered
packet geometry	projectors P_i	Which component of a vector lies in packet i?
packet operator	blocks $T_{ji} = Q_j T P_i$	How does the operator move source packet i into target packet j?
transfer graph	Boolean edges $i \to j$	Which packet-to-packet transfers are nonzero at the active tolerance?

The implementation is finite dimensional. With the optional SymPy backend it checks matrix equalities exactly. With the default NumPy backend it checks residual norms against a declared tolerance. The graph layer is combinatorial, but a graph extracted from a numerical operator inherits the support decisions made by the numerical zero test.

The main imports are:

```
from komorebi_math import (
    OperatorMorphism,
    PacketCoordinates,
    PacketMorphism,
    PacketOperator,
    PacketSystem,
    SeedMorphism,
    SupportPreorder,
    TransferGraph,
    block_from_paths,
    path_contributions,
)
```

`PacketSystem`, `PacketCoordinates`, `PacketOperator`, `TransferGraph`, `TransferPath`, `block_from_paths`, and `path_contributions` are also members of the frozen `komorebi_math.stable` namespace. `TransferPath` is not exported by the top-level research namespace, although it is the concrete type returned by `path_contributions`. Import that name from `komorebi_math.stable` or `komorebi_math.core` when a direct type reference is needed. `SupportPreorder` is available from the package root and `komorebi_math.core`, but not the stable namespace. The three morphism classes remain research-namespace APIs in `1.0.0b4`.

11.1 `PacketSystem` and `PacketCoordinates`

Let V be an n-dimensional space. A packet system is a nonempty labeled family of endomorphisms

$$P_i : V \to V, \qquad i = 0, \dots, r-1,$$

such that

$$P_i^2 = P_i, \qquad P_i P_j = 0 \quad (i \neq j), \qquad \sum_{i=0}^{r-1} P_i = I_V.$$

These are algebraic projectors. Self-adjointness is not part of the defining packet contract. Consequently, a valid packet decomposition can be oblique.

The constructor is:

```
PacketSystem(
    projectors: Sequence[Any],
    *,
    names: Sequence[str] | None = None,
    backend: str | LinearAlgebraBackend | None = None,
    tolerance: float | None = None,
    validate: bool | None = None,
) -> None
```

The family must be nonempty. Every converted projector must be a square matrix, and all projectors must have one common shape. Omitted names become `P0`, `P1`, and so on. Supplied names are converted to strings, must have the same count as the projectors, and must be unique. An individual packet is allowed to have rank zero. Completeness concerns the family as a whole.

When `tolerance` is omitted, the active setting is copied into the object. When `validate` is omitted, the active `validate_on_construction` setting decides whether `certificate().require()` runs during construction. Its default is `True`. Passing `validate=False` allows an invalid family to exist for diagnostic inspection. It does not make that family valid.

11.1.1 Algebraic and Hilbert certificates

`PacketSystem.certificate()` computes three residuals:

$$\delta_{\mathrm{id}} = \max_i \|P_i^2 - P_i\|,$$
$$\delta_{\mathrm{ann}} = \max_{i \neq j} \|P_i P_j\|,$$
$$\delta_{\mathrm{sum}} = \left\| \sum_i P_i - I \right\|.$$

The returned `Certificate` uses theorem identifier `PSC-I-T20`. Under an exact backend it is valid exactly when all three residuals are zero. Under a numerical backend it is valid when all three are no larger than the packet tolerance. Its diagnostics record the backend name, ambient dimension, and packet count.

`hilbert_certificate()` is a separate check:

$$\delta_* = \max_i \|P_i - P_i^*\|.$$

It returns a valid certificate only when every packet is self-adjoint, exactly or within tolerance. It assumes the algebraic packet axioms. It does not rerun or combine the algebraic certificate. A caller needing orthogonal Hilbert packets should require both records:

```
packets.certificate().require()
packets.hilbert_certificate().require()
```

The distinction is substantive. Pairwise annihilation and completeness do not force self-adjointness.

11.1.2 Analysis, embedded coordinates, and synthesis

Analysis sends a vector to its packet projections:

$$\mathcal{A}_P : V \to V^r, \qquad \mathcal{A}_P(x) = (P_0 x, \dots, P_{r-1} x).$$

Synthesis adds the embedded components:

$$\mathcal{S}_P(c_0, \dots, c_{r-1}) = \sum_i c_i.$$

Completeness gives

$$\mathcal{S}_P \mathcal{A}_P(x) = \sum_i P_i x = x.$$

This API uses **embedded coordinates**: every `PacketCoordinates.values[i]` is still an ambient n-vector. It is not a compressed vector of length `rank(P_i)`. This choice makes reconstruction, block application, and transfer products uniform, but it means storing all packet values can use rn scalars even when the packets are low rank.

```
PacketCoordinates(
    system: PacketSystem,
    values: tuple[Any, ...],
) -> None

PacketSystem.analyze(vector: Any) -> PacketCoordinates
PacketSystem.synthesize(
    coordinates: PacketCoordinates | Sequence[Any] | Mapping[str, Any],
) -> Any
PacketSystem.support(vector: Any, *, tolerance: float | None = None) -> tuple[str, ...]

PacketCoordinates.reconstruct() -> Any
PacketCoordinates.support(*, tolerance: float | None = None) -> tuple[str, ...]
PacketCoordinates.__getitem__(key: int | str) -> Any
PacketCoordinates.as_dict() -> dict[str, Any]
```

`analyze` converts the input through the packet backend and requires ambient dimension n. `synthesize` accepts a coordinate object, a positional sequence, or a mapping whose keys match the packet names exactly. Every value must convert to an ambient n-vector. Coordinates produced by a different system are accepted only when that system is `equivalent` to the receiver.

The `PacketCoordinates` dataclass constructor itself is intentionally shallow: it stores the supplied tuple without checking its count, dimensions, or packet support. `PacketSystem.analyze` is the canonical producer. If coordinates are assembled directly, `reconstruct()` and `synthesize()` check the count and ambient vector dimensions, but they do not check $P_i c_i = c_i$. Only analysis guarantees packet-supported components. Direct callers must establish that stronger invariant themselves.

In practice, begin by obtaining coordinates from `analyze`, and use the same packet system for synthesis. If you later construct components manually, check their packet support as well as their lengths. Correctly sized ambient vectors can still assign a contribution to the wrong packet.

Integer indexing follows ordinary tuple indexing. String indexing resolves a packet name and raises `KeyError` when the name is absent. `support` reports the names whose embedded values are not zero under the selected tolerance. It is a support decision, not a rank or energy threshold.

This example was executed against the delivered source:

```
import numpy as np
from komorebi_math import PacketSystem

packets = PacketSystem(
    [np.diag([1, 0, 0]), np.diag([0, 1, 0]), np.diag([0, 0, 1])],
    names=("a", "b", "c"),
)
x = np.array([2.0, 0.0, -3.0])
coordinates = packets.analyze(x)

print(coordinates.support())
print({name: np.real_if_close(value).tolist()
       for name, value in coordinates.as_dict().items()})
print(np.real_if_close(coordinates.reconstruct()).tolist())
print(packets.packet_dimensions())

coarse = packets.aggregate({"ab": ("a", "b"), "c": ("c",)})
print(coarse.names)
print(coarse.packet_dimensions())
print(coarse.certificate().exactness.value)

A = np.arange(9.0).reshape(3, 3)
print(np.real_if_close(packets.block_diagonal_projection(A)).tolist())
```

Output:

```
('a', 'c')
{'a': [2.0, 0.0, 0.0], 'b': [0.0, 0.0, 0.0], 'c': [0.0, 0.0, -3.0]}
[2.0, 0.0, -3.0]
{'a': 1, 'b': 1, 'c': 1}
('ab', 'c')
{'ab': 2, 'c': 1}
numerical
[[0.0, 0.0, 0.0], [0.0, 4.0, 0.0], [0.0, 0.0, 8.0]]
```

The numerical label is correct even though the matrices contain exact-looking zeros and ones: this execution used the NumPy backend.

11.1.3 Structural operations on packet systems

The remaining packet methods transform or inspect the decomposition.

Method	Implemented map	Main precondition
`index(name)`	packet name to integer index	name must exist
`packet_dimensions()`	`name -> rank(P_i)`	rank follows backend semantics
`aggregate(groups)`	$\widetilde{P}_a = \sum_{i \in G_a} P_i$	groups partition every packet index exactly once
`tensor(other)`	$P_i \otimes Q_j$	both systems use the same backend name
`dual()`	$P_i \mapsto P_i^T$	no additional precondition
`equivalent(other)`	compare backend, dimensions, names, and projectors	numerical comparison uses a tolerance
`transport(C)`	$P_i \mapsto C P_i C^{-1}$	C is an invertible $n \times n$ matrix
`restrict(J,R)`	$P_i \mapsto R P_i J$	$RJ = I$ and $P_i J = J(R P_i J)$
`quotient(Q,S)`	$P_i \mapsto Q P_i S$	$QS = I$ and $Q P_i = (Q P_i S) Q$
`block_diagonal_projection(T)`	$T \mapsto \sum_i P_i T P_i$	T is an $n \times n$ operator

`aggregate` accepts packet names or indices. The values of the mapping must partition the integer index set exactly once: omissions, duplicates, and empty groups fail. Mapping insertion order determines the new packet order and names.

`tensor` produces names such as `left⊗right` in nested left-then-right order. The new tolerance is the maximum of the two input tolerances. Backend identity is checked by backend name.

`dual()` uses **transpose**, not conjugate transpose, despite the generated `name*` labels. It is the algebraic dual packet system. The Hilbert adjoint is used by `hilbert_certificate()` and by `PacketOperator.adjoint()`. For complex objects, do not substitute one operation for the other.

`transport` delegates invertibility to the backend inverse operation. With NumPy, a singular map therefore raises `numpy.linalg.LinAlgError`. It is not wrapped as a package validation exception. Restriction and quotient use declared splitting maps and explicitly verify both the split identity and packet invariance. They retain all packet labels, so a packet can descend to a zero-rank projector.

For example, restricting three coordinate packets to the first two coordinates and quotienting onto the last coordinate preserve all three names:

```
import numpy as np
from komorebi_math import PacketSystem

packets = PacketSystem(
    [np.diag([1, 0, 0]), np.diag([0, 1, 0]), np.diag([0, 0, 1])],
    names=("a", "b", "c"),
)

J = np.array([[1.0, 0.0], [0.0, 1.0], [0.0, 0.0]])
restricted = packets.restrict(J, J.T)

Q = np.array([[0.0, 0.0, 1.0]])
quotient = packets.quotient(Q, Q.T)

print(restricted.packet_dimensions(), restricted.certificate().valid)
print(quotient.packet_dimensions(), quotient.certificate().valid)
```

Output:

```
{'a': 1, 'b': 1, 'c': 0} True
{'a': 0, 'b': 0, 'c': 1} True
```

Finally, `block_diagonal_projection` means diagonal relative to the algebraic packet decomposition. For oblique packets it should not be described as an orthogonal projection in a Hilbert-space norm.

11.1.4 Exact oblique example

This optional-SymPy example separates the algebraic and Hilbert contracts:

```
import sympy as sp
from komorebi_math import PacketSystem

P0 = sp.Matrix([[1, 1], [0, 0]])
P1 = sp.eye(2) - P0
packets = PacketSystem((P0, P1), names=("u", "v"), backend="sympy")
coordinates = packets.analyze(sp.Matrix([2, 3]))
algebraic = packets.certificate()
hilbert = packets.hilbert_certificate()

print(tuple(tuple(value) for value in coordinates.values))
print(tuple(coordinates.reconstruct()))
print(algebraic.valid, algebraic.exactness.value, algebraic.residuals)
```

```
print(hilbert.valid, hilbert.residuals)
print(packets.dual().certificate().valid)
```

Output:

```
((5, 0), (-3, 3))
(2, 3)
True exact {'idempotence': 0.0, 'annihilation': 0.0, 'completeness': 0.0}
False {'self_adjointness': 1.4142135623730951}
True
```

The decomposition and reconstruction are exact. The failed Hilbert certificate does not invalidate the algebraic decomposition. It rejects the stronger claim that the packets are orthogonal.

11.2 `PacketOperator`

Let $P_i : V \to V$ be source packets and $Q_j : W \to W$ target packets. For a linear map $T : V \to W$, define the embedded block

$$T_{ji} = Q_j T P_i : V \to W.$$

Completeness on both sides gives

$$T = \left(\sum_j Q_j\right) T \left(\sum_i P_i\right) = \sum_{j,i} T_{ji}.$$

`PacketOperator` stores blocks as `blocks[target_index][source_index]`:

```
PacketOperator(
    source: PacketSystem,
    target: PacketSystem,
    blocks: tuple[tuple[Any, ...], ...],
) -> None

PacketOperator.from_operator(
    operator: Any,
    source: PacketSystem,
    target: PacketSystem | None = None,
) -> PacketOperator
```

When `target` is omitted, `from_operator` constructs an endomorphism over the source packet system. Source and target must use the same backend name. A rectangular ambient operator is allowed when it has shape `(target.dimension, source.dimension)`. Every stored block nevertheless has that full embedded shape. Blocks are not compressed rank-by-rank matrices.

The direct dataclass constructor verifies the block grid and matrix shapes, but it does not coerce blocks and does not automatically require the block-support certificate. `from_operator` is the canonical constructor for an ordinary ambient matrix. A caller constructing blocks directly should explicitly use

```
packet_operator.certificate().require()
```

before relying on packet support.

11.2.1 Reconstruction and action

The primary methods are:

```
reconstruct() -> Any
certificate() -> Certificate
apply(vector: Any) -> Any
apply_coordinates(coordinates: PacketCoordinates) -> PacketCoordinates
```

`reconstruct()` adds every block. `apply(vector)` reconstructs the ambient matrix and multiplies it by the source vector. `apply_coordinates()` instead performs block matrix multiplication in embedded coordinates:

$$d_j = \sum_i T_{ji} c_i.$$

It requires the coordinate system to be equivalent to the source system and returns a `PacketCoordinates` object tied to the target system.

`certificate()` reconstructs T, reprojects it through `from_operator(T, source, target)`, and checks

$$\delta_{\mathrm{block}} = \max_{j,i} \|B_{ji} - Q_j T P_i\|.$$

The theorem identifier is `PSC-I-T27`. The certificate is exact or numerical according to the backend and uses the maximum of the source and target packet tolerances. This check is especially important for directly supplied blocks.

11.2.2 Composition, adjoints, tensors, and powers

If $T : U \to V$ and $S : V \to W$, then packet composition is ordinary block matrix multiplication:

$$(S \circ T)_{ki} = \sum_j S_{kj} T_{ji}.$$

The method call is intentionally ordered as

```
left.compose(right)   # left after right
```

and requires `right.target.equivalent(left.source)`.

The remaining algebraic methods are:

```
adjoint() -> PacketOperator
tensor(other: PacketOperator) -> PacketOperator
is_endomorphism() -> bool
power(exponent: int) -> PacketOperator
block_norms() -> tuple[tuple[float, ...], ...]
is_block_diagonal(*, tolerance: float | None = None) -> bool
support_mask(*, tolerance: float | None = None) -> tuple[tuple[bool, ...], ...]
```

`adjoint()` conjugate-transposes every block, transposes the block grid, and swaps source with target. For self-adjoint packet systems this is the familiar Hilbert packet adjoint. For oblique packets, the returned direct block object is not automatically reprojected against algebraic dual packet systems. Require its certificate before asserting packet support.

`tensor` forms the Kronecker product of block pairs and uses tensor packet systems on both sides. `power` is defined only for packet endomorphisms and accepts nonnegative exponents. Exponent zero returns the packet-resolved ambient identity. Negative powers raise `ValueError`. Inverse packet powers are not implemented. Positive powers use repeated composition rather than binary exponentiation.

`block_norms` returns backend diagnostic norms as Python floats. Both `is_block_diagonal` and `support_mask` use the selected zero tolerance. `support_mask` retains the storage orientation: rows are targets and columns are sources.

This compact execution exercises the ambient and coordinate actions as well as the composition order:

```
import numpy as np
from komorebi_math import PacketOperator, PacketSystem

packets = PacketSystem(
    (np.diag([1, 0]), np.diag([0, 1])),
    names=("x", "y"),
)
T = np.array([[1.0, 2.0], [3.0, 4.0]])
S = np.array([[0.0, 1.0], [1.0, 0.0]])
t = PacketOperator.from_operator(T, packets)
s = PacketOperator.from_operator(S, packets)
v = np.array([1.0, 2.0])

print(np.real_if_close(t.reconstruct()).tolist())
print(np.real_if_close(t.apply(v)).tolist())
print(np.real_if_close(t.apply_coordinates(packets.analyze(v)).reconstruct()).tolist())
print(t.block_norms())
print(t.support_mask(), t.is_block_diagonal())
print(np.real_if_close(s.compose(t).reconstruct()).tolist())
print(np.real_if_close(t.adjoint().reconstruct()).tolist())
print(np.real_if_close(t.power(0).reconstruct()).tolist())
print(t.certificate().valid, t.certificate().exactness.value)
```

Output:

```
[[1.0, 2.0], [3.0, 4.0]]
[5.0, 11.0]
[5.0, 11.0]
((1.0, 2.0), (3.0, 4.0))
((True, True), (True, True)) False
[[3.0, 4.0], [1.0, 2.0]]
[[1.0, 3.0], [2.0, 4.0]]
[[1.0, 0.0], [0.0, 1.0]]
True numerical
```

11.3 `TransferGraph`, paths, and `TransferPath`

A `TransferGraph` is a finite directed graph. Its adjacency matrix uses the opposite indexing convention from the packet block grid:

$$\texttt{adjacency}[\texttt{source}][\texttt{target}] = \text{edge } source \to target.$$

For a packet endomorphism,

$$i \to j \quad \iff \quad T_{ji} \neq 0$$

under the active exact or tolerance-aware zero test. Thus

```
graph.adjacency[source][target]
    == operator.support_mask()[target][source]
```

The constructors are:

```
TransferGraph(
    adjacency: Sequence[Sequence[bool | int]],
    *,
    names: Sequence[str] | None = None,
) -> None

TransferGraph.from_packet_operator(
    operator: PacketOperator,
    *,
    tolerance: float | None = None,
) -> TransferGraph

TransferGraph.from_operator_family(
    operators: Iterable[PacketOperator],
    *,
    tolerance: float | None = None,
) -> TransferGraph
```

Direct adjacency entries are converted to Boolean values. The matrix must be nonempty and square. Names must be unique and match its order. Extraction from one operator requires a packet endomorphism. Extraction from a family requires a nonempty family using one equivalent packet system and takes the union of the individual support edges.

The graph query API is:

```
successors(vertex) -> tuple[int, ...]
predecessors(vertex) -> tuple[int, ...]
reachable(starts) -> frozenset[int]
paths(source, target, *, length) -> tuple[tuple[int, ...], ...]
strongly_connected_components() -> tuple[tuple[int, ...], ...]
condensation() -> tuple[TransferGraph, tuple[tuple[int, ...], ...]]
source_components() -> tuple[tuple[int, ...], ...]
is_acyclic() -> bool
topological_order() -> tuple[int, ...]
longest_path_vertices() -> int
source_component_certificate() -> ConditionalResult[int]
edges() -> Iterator[tuple[int, int]]
```

`reachable` includes every supplied start vertex. `paths` enumerates walks of **exactly** the requested edge length. Repeated vertices are allowed. At length zero it returns `(source,)` only when source equals target. A negative length raises `ValueError`. Enumeration materializes every matching path, so its time and memory can grow exponentially with length. There is no path-count resource guard in this API.

Strongly connected components are returned as sorted tuples in component order. `condensation()` removes within-component loops and returns the acyclic component graph together with the component membership. A self-loop makes a singleton component cyclic. `topological_order()` rejects every cyclic graph. `longest_path_vertices()` counts vertices rather than edges, so a one-vertex acyclic graph has value one.

`source_component_certificate()` deserves special care. It returns the number of source strongly connected components as a `ConditionalResult[int]` with theorem identifier `PSC-I-T44`. The certificate is intentionally conditional and leaves two assumptions unchecked:

- edge maps are surjective on the declared packet spaces
- each source component is generated by one packet-pure seed

The graph count is therefore not an unconditional seed-rank theorem.

11.3.1 `SupportPreorder`

`SupportPreorder` records a reflexive, transitive Boolean relation on packet labels. It is exported from the package root and `komorebi_math.core`, but not from the frozen stable namespace.

```
SupportPreorder(
    relation: Sequence[Sequence[bool | int]],
) -> None
```

The relation must be nonempty and square. Construction converts each entry to `bool`, then rejects a relation that is not reflexive or transitive. Its public queries are:

```
order -> int
is_reflexive() -> bool
is_transitive() -> bool
equivalence_classes() -> tuple[tuple[int, ...], ...]
strict_class_graph() -> TransferGraph
strict_ideal_nilpotency_bound() -> int
```

Mutual comparability defines the equivalence classes. Collapsing those classes and retaining only strict relations produces an acyclic transfer graph. If its longest class chain contains c vertices, then

$$I^c = 0$$

is the bound returned for the strict transfer ideal. For example, the preorder

$$\begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

has equivalence classes `((0, 1), (2,), (3,))` and nilpotency bound `3`. A merely reflexive but nontransitive matrix is rejected with `ValidationError`. The constructor does not silently take a transitive closure.

`TransferPath` is a frozen record:

```
TransferPath(vertices: tuple[int, ...], product: Any)
```

Its `length` property is `len(vertices) - 1`. The `product` is the full embedded matrix contribution attached to that path. Direct construction does not verify vertex incidence or consistency between the vertices and product. Normally consume these records from `path_contributions()`.

11.4 `block_from_paths` and `path_contributions`

For a packet endomorphism T, source packet a, target packet b, and nonnegative integer ℓ, the transfer-path identity is

$$(T^\ell)_{ba} = \sum_{\pi : a \to b,\ |\pi| = \ell} T_\pi,$$

where a path

$$\pi = (v_0 = a, v_1, \dots, v_\ell = b)$$

has ordered product

$$T_\pi = T_{v_\ell v_{\ell-1}} \cdots T_{v_2 v_1} T_{v_1 v_0}.$$

The two functions separate enumeration from certified summation:

```
path_contributions(
    operator: PacketOperator,
    *,
    source: int,
    target: int,
    length: int,
    tolerance: float | None = None,
) -> tuple[TransferPath, ...]

block_from_paths(
    operator: PacketOperator,
    *,
    source: int,
    target: int,
    length: int,
    tolerance: float | None = None,
) -> ComputationResult[Any]
```

Both require a packet endomorphism. `path_contributions` first extracts a transfer graph at the selected tolerance and returns one record for every structurally active path. At length zero, the only nonempty case is `source == target`, and its product is the source packet projector P_a. Neither function automatically requires `operator.certificate()`. Require that certificate first when the operator was assembled through the direct block constructor rather than `from_operator`.

`block_from_paths` sums the contributions, independently computes `operator.power(length).blocks[target][source]`, and certifies their difference:

$$\delta_{\mathrm{path}} = \left\| \sum_{\pi} T_{\pi} - (T^{\ell})_{ba} \right\|.$$

The resulting `ComputationResult` carries theorem identifier `PSC-I-T29` and records `path_count` and `length` in its diagnostics. Under an exact backend, validity requires zero residual. Under a numerical backend, it requires the residual to be at most the selected tolerance, defaulting to the source packet tolerance.

This certificate matters because the graph extraction can prune a small but nonzero numerical block. The path sum then represents the thresholded support graph, while the comparison target remains the full operator power. A valid certificate says the omitted contribution is within the same declared tolerance. It does not say that the omitted block was algebraically zero.

11.4.1 Diamond transfer example

This endomorphism has two length-two paths from packet zero to packet three:

$$0 \xrightarrow{2} 1 \xrightarrow{5} 3, \qquad 0 \xrightarrow{3} 2 \xrightarrow{7} 3.$$

Their contributions are 10 and 21, so the corresponding block of T^2 is 31.

```
import numpy as np
from komorebi_math import (
    PacketOperator, PacketSystem, TransferGraph,
    block_from_paths, path_contributions,
)

packets = PacketSystem(
    [np.diag([1 if i == j else 0 for i in range(4)]) for j in range(4)],
    names=("s", "u", "v", "t"),
)
T = np.array([
```

```
    [0, 0, 0, 0],
    [2, 0, 0, 0],
    [3, 0, 0, 0],
    [0, 5, 7, 0],
], dtype=float)
operator = PacketOperator.from_operator(T, packets)
graph = TransferGraph.from_packet_operator(operator)

print(operator.support_mask())
print(graph.adjacency)
print(tuple(graph.edges()))
print(graph.paths(0, 3, length=2))

parts = path_contributions(operator, source=0, target=3, length=2)
print([(path.vertices, path.length,
        np.real_if_close(path.product[3, 0]).item()) for path in parts])

result = block_from_paths(operator, source=0, target=3, length=2)
print(np.real_if_close(result.value[3, 0]).item())
print(result.valid, result.certificate.exactness.value,
      result.certificate.diagnostics)
print(np.real_if_close(operator.power(2).reconstruct()[3, 0]).item())
```

Output:

```
((False, False, False, False), (True, False, False, False), (True, False, False, False), (False, True, True, False))
((False, True, True, False), (False, False, False, True), (False, False, False, True), (False, False, False, False))
((0, 1), (0, 2), (1, 3), (2, 3))
((0, 1, 3), (0, 2, 3))
[((0, 1, 3), 2, 10.0), ((0, 2, 3), 2, 21.0)]
31.0
True numerical {'path_count': 2, 'length': 2}
31.0
```

The first two Boolean matrices visibly demonstrate the orientation reversal: the operator mask is target-by-source, while graph adjacency is source-by-target.

11.5 Operator, packet, and seed morphisms

Morphism certificates make declared compatibility equations executable. They do not infer the declaration. The caller supplies the operator-name map, packet-image map, and optional seed images to be checked.

11.5.1 `OperatorMorphism`

For source operators $A_a : V \to V$, target operators $B_b : W \to W$, a linear map $\Phi : V \to W$, and a declared name map $f : a \mapsto b$, the checked equation is

$$\Phi A_a = B_{f(a)} \Phi.$$

Construction should normally use:

```
OperatorMorphism.create(
    source_operators: Mapping[str, Any],
    target_operators: Mapping[str, Any],
    linear_map: Any,
    operator_map: Mapping[str, str] | None = None,
    *,
    backend: str | LinearAlgebraBackend | None = None,
    tolerance: float = 1e-10,
) -> OperatorMorphism
```

The public record can also be instantiated directly with the manifested dataclass signature:

```
OperatorMorphism(
    source_operators,
    target_operators,
    linear_map,
    operator_map,
    backend,
    tolerance,
) -> None
```

That direct path performs no matrix conversion or shape validation. Use `create()` for the validated construction route.

When `operator_map` is omitted, it maps names in the intersection identically. Every explicitly mapped name must exist. The certificate stores one residual `source_name->target_name`, uses theorem identifier `PSC-I-T50`, and is exact or numerical according to the selected backend.

The mapping is allowed to be partial. Passing `{}` does not force an empty map: because `create()` uses the explicit mapping or the shared-name fallback, an empty dictionary still maps shared names identically. The factory produces an empty map only when the operator families have no shared names and no nonempty map is supplied. In that case the certificate has no residuals, and the universal check over the empty set is valid. The per-pair shape check is also vacuous. Therefore a meaningful morphism contract should provide a nonempty map covering every generator whose naturality is being claimed. `create()` converts matrices but does not perform a complete independent audit that every supplied operator is square.

11.5.2 `PacketMorphism`

A packet morphism supplies source packets P_i, target packets Q_j, a linear map Φ, and for every source name a tuple $F(i)$ of admissible target names. The certificate checks image containment:

$$\Phi P_i = \left(\sum_{j \in F(i)} Q_j \right) \Phi P_i.$$

Its constructor is the dataclass signature:

```
PacketMorphism(
    source: PacketSystem,
    target: PacketSystem,
    linear_map: Any,
    packet_map: Mapping[str, tuple[str, ...]],
) -> None
```

Source and target must use one backend. The map Φ must have shape `(target.dimension, source.dimension)`. `packet_map` must name every source packet exactly as a key, and every listed target name must exist. The certificate uses theorem identifier `PSC-I-T52`.

This is containment, not equality of packet ranges and not a claim that Φ is injective, surjective, or invertible. Empty target tuples are allowed and certify a source packet only when its image under Φ is zero. Target packets need not be used exactly once. Repeated names inside one tuple are not rejected structurally, but their projectors are added repeatedly and will normally cause a failed residual. Use duplicate-free tuples.

11.5.3 `SeedMorphism`

`SeedMorphism` combines operator intertwining with optional declared images of the source seeds:

```
SeedMorphism(
    source: SeedSystem,
    target: SeedSystem,
    linear_map: Any,
    operator_map: Mapping[str, str],
    seed_images: Sequence[Any] | None = None,
) -> None
```

It first applies the `OperatorMorphism` equation to the source and target seed systems. If expected seed images are provided, it additionally checks

$$\Phi s_i = \hat{s}_i$$

for every source seed. Its certificate uses theorem identifier `PSC-I-T50`. The expected vectors are caller declarations. The implementation does not independently require each $\hat{s}_i$ to equal a listed seed of the target system. When `seed_images=None`, no seed-vector residual is checked. A complete application-level claim should therefore declare and audit the intended target seed images explicitly.

11.5.4 Executed morphism example

```
import numpy as np
from komorebi_math import OperatorMorphism, PacketMorphism, PacketSystem

operator_morphism = OperatorMorphism.create(
    {"T": np.diag([1.0, 2.0])},
    {"S": np.diag([1.0, 2.0, 3.0])},
    np.array([[1.0, 0.0], [0.0, 1.0], [0.0, 0.0]]),
    {"T": "S"},
)
print(operator_morphism.certificate().valid)
print(operator_morphism.certificate().residuals)

fine = PacketSystem(
    [np.diag([1, 0, 0]), np.diag([0, 1, 0]), np.diag([0, 0, 1])],
    names=("a", "b", "c"),
)
coarse = fine.aggregate({"ab": ("a", "b"), "c": ("c",)})
packet_morphism = PacketMorphism(
    fine,
    coarse,
    np.eye(3),
    {"a": ("ab",), "b": ("ab",), "c": ("c",)},
)
print(packet_morphism.certificate().valid)
print(packet_morphism.certificate().residuals)
```

Output:

```
True
{'T->S': 0.0}
True
{'a': 0.0, 'b': 0.0, 'c': 0.0}
```

The first map embeds the two-dimensional source into the first two target coordinates, preserving its one-dimensional eigenspaces for eigenvalues one and two. The second map certifies that the fine packets `a` and `b` land inside the aggregate packet `ab`, while `c` remains in `c`.

11.5.5 Exactness, tolerances, and failure boundaries

The execution boundary can be summarized as follows.

Operation	Exact backend	NumPy backend	Conditional branch
packet axioms	exact matrix equalities	norm residuals at packet tolerance	none
Hilbert packet check	exact self-adjointness	adjoint residual at packet tolerance	none
packet-operator support	exact reprojection equality	maximum block residual at max source/target tolerance	none
path block identity	exact path sum	path residual at selected tolerance	none
graph support	exact structural zero	tolerance-aware structural zero	none
morphism equations	exact intertwining or containment	residual at morphism/packet tolerance	none
source-component count as seed rank	graph count is finite	graph count is finite	explicitly conditional on propagation assumptions

Exact backends still expose residuals as floating diagnostic values because the backend norm interface returns `float`. The exactness label comes from the backend's declared exact arithmetic and the equality branch used by the producer, not from the Python type of the residual field.

The important failure cases are:

Boundary	Failure behavior
empty packet family. Nonsquare or mixed-size projectors	`ValidationError` or `DimensionError` during construction
invalid packet axioms with ordinary validation enabled	invalid `PSC-I-T20` certificate is required and raises `ValidationError`
invalid packet axioms with `validate=False`	object is returned. `certificate().valid` is `False`
duplicate or wrong-count names	`ValidationError`
wrong analysis vector or synthesis coordinate dimensions	`DimensionError`
coordinate mapping missing or adding packet names	`ValidationError`
coordinates from a nonequivalent packet system	`ValidationError`
aggregate groups omit or repeat a packet	`ValidationError`
singular transport matrix	backend inverse exception, such as `numpy.linalg.LinAlgError`
invalid restriction split or noninvariant subspace	`ValidationError`
invalid quotient split or noninvariant kernel	`ValidationError`
direct packet blocks with wrong grid or embedded shape	`DimensionError`
direct packet blocks with correct shapes but wrong packet support	object exists. `PSC-I-T27` certificate is invalid
composition across nonequivalent intermediate packets	`ValidationError`
negative packet-operator power	`ValueError`
power, graph extraction, or path expansion for a nonendomorphism	`ValidationError`
empty operator family for a graph	`ValidationError`
out-of-range graph vertex	`IndexError`
negative path length	`ValueError`
topological order on a cyclic graph	`ValidationError`
malformed morphism maps or dimensions	`ValidationError` or `DimensionError`
well-shaped but false morphism equation	returned certificate has `valid=False`

A controlled packet failure demonstrates the difference between construction validation and diagnostic construction:

```
import numpy as np
from komorebi_math import PacketSystem
from komorebi_math.exceptions import ValidationError

projectors = (np.diag([1.0, 0.0]), np.diag([1.0, 0.0]))

try:
    PacketSystem(projectors)
except ValidationError as error:
    print(f"{type(error).__name__}: {error}")

diagnostic = PacketSystem(projectors, validate=False)
print(diagnostic.certificate().valid)
print(diagnostic.certificate().residuals)
```

Output:

```
ValidationError: projectors do not form a complete packet system
False
{'idempotence': 0.0, 'annihilation': 1.0, 'completeness': 1.4142135623730951}
```

`validate=False` is appropriate for inspecting why a proposed family failed. It should not appear as a way to suppress a failed mathematical premise in production code.

11.5.6 What the shipped tests establish

The focused packet, operator, transfer, graph, morphism, and quotient tests were executed together while preparing this chapter:

```
34 passed in 1.58s
```

Those tests establish these implemented finite contracts:

- packet analysis reconstructs the input and support names follow projected zero tests
- packet ranks, aggregation, tensors, algebraic duals, oblique-versus-Hilbert certification, and block-diagonal projection behave as declared
- coordinate transport, invariant restriction, and quotient descent produce valid packet systems in the tested cases
- packet operators reconstruct, apply in ambient and packet coordinates, compose in the stated order, take adjoints and tensors, and detect block diagonal structure
- positive powers agree with ambient matrix powers. Zero-length path blocks are packet identities
- fixed-length transfer paths reconstruct the tested operator-power block
- graph reachability, path enumeration, strongly connected components, condensation, topological rejection, and conditional source-component evidence follow the source-by-target convention
- valid and invalid operator intertwiners are distinguished, aggregate packet morphisms certify, and declared seed morphisms certify in the tested cases

These are finite source-level checks. They do not establish that a thresholded numerical edge is algebraically zero, that an arbitrary packet morphism is an isomorphism, that source-component count is an unconditional seed rank, or that enumerating long path families is computationally feasible.

Chapter 11 summary. A `PacketSystem` is an algebraic direct-sum decomposition represented by complete pairwise-annihilating idempotents. Orthogonality is a separate Hilbert certificate. `PacketCoordinates` retain ambient embedded vectors. A `PacketOperator` stores $T_{ji} = Q_j T P_i$ target-by-source and reconstructs by summation. A `TransferGraph` transposes that support mask into source-by-target adjacency. Fixed-length path products reconstruct blocks of operator powers, with `PSC-I-T29` recording the exact or tolerance-aware comparison. Morphism certificates check only the declared intertwining, image-containment, and seed-image equations. They do not infer missing declarations or promote conditional graph evidence into an unconditional theorem.

Chapter 12 refines these finite packet systems across multiple resolutions and adds explicit parent maps, compatibility certificates, common refinements, and weighted transfer structure.

12. Refinement towers

First route. Build and inspect one adjacent refinement in Section 12.1 before constructing a tower. Write down the parent of each fine packet and check a coarse-to-fine-to-coarse coordinate roundtrip. Section 12.2 then adds several levels. Weighted inverses and linear defects in Sections 12.3 to 12.4 are separate operator workflows and can be read when those tasks arise.

A packet system describes one decomposition of an ambient finite-dimensional space. A refinement records how a second packet system resolves every coarse packet into smaller packets without changing that ambient space. A refinement tower then chains several such resolutions.

This is not interpolation, resampling, or a change in vector dimension. If $P = (P_a)_{a \in A}$ is coarse and $Q = (Q_b)_{b \in B}$ is fine, both families act on the same vectors. The parent map

$$p : B \longrightarrow A$$

must satisfy

$$P_a = \sum_{b : p(b) = a} Q_b.$$

The library uses that identity in three related ways:

1. `PacketRefinement` moves packet coordinates and packet operators between two adjacent resolutions.
2. `RefinementTower` composes adjacent refinements, while `common_refinement()` constructs a shared fine level for two commuting packet systems.
3. `WeightedTransferGraph` and `LinearRefinementDefect` describe how operators move across packets or fail to intertwine across levels.

`PacketRefinement`, `RefinementTower`, and `common_refinement` are in the frozen stable namespace. Weighted transfer and defect propagation remain research APIs. Their evidence classes still use the certificate rules from Chapter 5.

12.1 `PacketRefinement`, parent maps, and compatibility

The stable constructor is

```
PacketRefinement(
    coarse: PacketSystem,
    fine: PacketSystem,
    parent_of_fine: tuple[int, ...],
) -> None
```

`parent_of_fine[b]` is the coarse index $p(b)$ assigned to fine packet Q_b. The direction matters: there is one tuple entry per fine packet, not one tuple entry per coarse packet.

Construction enforces these conditions.

Condition	Implemented check
Common scalar implementation	`coarse.backend.name == fine.backend.name`
Common ambient space	`coarse.dimension == fine.dimension`
Total parent map	`len(parent_of_fine) == len(fine)`
Valid parent indices	Every entry lies in `range(len(coarse))`
No empty coarse parent	Every coarse index occurs at least once
Projector reconstruction	$P_a = \sum_{p(b)=a} Q_b$ for every a

Condition	Implemented check

The last condition is checked by `certificate()` and required during `__post_init__`. Under an exact backend, every residual must be exactly zero. Under a numerical backend, each residual must be at most

$$\tau = \max(\tau_P, \tau_Q).$$

The returned certificate has theorem identifier `PSC-II-T03`. Its residual dictionary is keyed by coarse packet name, its checked assumptions are `"packet axioms"`, `"one parent per fine packet"`, and `"coarse projector equals child sum"`, and its exactness is inherited from the backend. A successful NumPy construction is therefore numerical even when every displayed residual is `0.0`.

The frozen record exposes `coarse`, `fine`, and `parent_of_fine` unchanged. Its read-only `backend` property is the coarse system's backend, which is also the fine system's backend after construction succeeds.

Ordinary `PacketSystem` construction validates the packet axioms before a refinement is made. If a caller created either endpoint with `validate=False`, the caller should require those packet certificates separately. `PacketRefinement.certificate()` verifies the parent-sum identity. It does not independently re-run both full packet-system certificates.

Parent and child queries

The address queries are

```
refinement.children(coarse_index: int | str) -> tuple[int, ...]
refinement.parent(fine_index: int | str) -> int
```

A string is resolved through the corresponding packet system's name table. The result of `children()` is an ordered tuple of fine indices in their original packet order. `parent()` always returns the integer coarse index, even when its input is a fine packet name.

Use valid integer indices. String lookups raise `KeyError` for unknown names, but the two methods do not impose a separate uniform integer-range guard: `children()` can return an empty tuple for an out-of-range integer, and normal Python tuple indexing governs integer input to `parent()`.

Coordinate refinement and coarsening

For an ambient vector x, write

$$c_a = P_a x, \qquad f_b = Q_b x.$$

Since each packet system is complete,

$$x = \sum_a c_a = \sum_b f_b.$$

The parent identity gives the exact coordinate law

$$c_a = P_a x = \sum_{b:p(b)=a} Q_b x = \sum_{b:p(b)=a} f_b.$$

The two methods implement that law in opposite directions:

```
refinement.coarsen(coordinates: PacketCoordinates) -> PacketCoordinates
refinement.refine(coordinates: PacketCoordinates) -> PacketCoordinates
```

`coarsen()` requires coordinates belonging to the fine system and explicitly sums the embedded fine coordinate vectors under each parent. `refine()` requires coordinates belonging to the coarse system, reconstructs their ambient vector, and analyzes that vector with the fine packet system:

$$(c_a)_a \xrightarrow{\sum_a} x \xrightarrow{(Q_b x)_b} (f_b)_b.$$

Thus refinement is defined by the projectors, not by arbitrarily dividing a coarse coordinate among its children. Both membership checks use `PacketSystem.equivalent()`. Equivalence includes the backend name, ambient dimension, packet count, packet names, and projector equality at the systems' tolerance. Two mathematically similar systems with different names are not interchangeable coordinate owners.

The coordinate-change claim is documented as `PSC-II-T04`. The methods return `PacketCoordinates` directly because construction already required the refinement certificate. They do not wrap each coordinate conversion in a new `ComputationResult`.

Operator resolution and support reflection

An ambient operator can be represented at either packet resolution:

```
refinement.refine_operator(operator: PacketOperator | Any) -> PacketOperator
refinement.coarsen_operator(operator: PacketOperator | Any) -> PacketOperator
```

If the input is a `PacketOperator`, the method first reconstructs its ambient matrix. Otherwise it treats the input itself as the ambient matrix. It then calls `PacketOperator.from_operator()` with the requested packet system. These methods therefore preserve the reconstructed operator, while the number and arrangement of stored blocks change with the packet resolution.

Support reflection has signature

```
refinement.support_reflection(
    coordinates: PacketCoordinates,
) -> dict[str, tuple[str, ...]]
```

Fine coordinates are used directly. Coarse coordinates are first refined. For each coarse packet with at least one active child, the result maps its name to the ordered names of its nonzero fine children. Zero tests use the fine system's tolerance. Coarse packets with no active fine child are omitted from the dictionary.

Worked example: one valid structural refinement on NumPy

This example was executed against the recorded reference source. Its packet identities have zero observed residual, but NumPy correctly produces a numerical certificate.

```
import numpy as np

from komorebi_math import PacketRefinement, PacketSystem

coarse = PacketSystem(
    [
        np.diag([1, 1, 0, 0]),
        np.diag([0, 0, 1, 1]),
    ],
    names=["left", "right"],
)
fine = PacketSystem(
```

```
    [
        np.diag([1, 0, 0, 0]),
        np.diag([0, 1, 0, 0]),
        np.diag([0, 0, 1, 0]),
        np.diag([0, 0, 0, 1]),
    ],
    names=["x0", "x1", "x2", "x3"],
)

step = PacketRefinement(coarse, fine, (0, 0, 1, 1))
x = np.array([0.0, 2.0, 0.0, 4.0])
fine_coordinates = step.refine(coarse.analyze(x))
back = step.coarsen(fine_coordinates)
certificate = step.certificate()

print("children(left):", step.children("left"))
print("parent(x3):", step.parent("x3"))
print("roundtrip:", np.real_if_close(back.reconstruct()).tolist())
print("support:", step.support_reflection(coarse.analyze(x)))
print(
    "certificate:",
    certificate.valid,
    certificate.exactness.value,
    certificate.residuals,
    certificate.theorem_id,
)
```

Output:

```
children(left): (0, 1)
parent(x3): 1
roundtrip: [0.0, 2.0, 0.0, 4.0]
support: {'left': ('x1',), 'right': ('x3',)}
certificate: True numerical {'left': 0.0, 'right': 0.0} PSC-II-T03
```

12.2 `RefinementTower` and `common_refinement`

Finite towers

`RefinementTower` accepts an ordered nonempty sequence:

```
RefinementTower(
    refinements: tuple[PacketRefinement, ...] | list[PacketRefinement],
) -> None
```

If the tuple is $(R_0, \dots, R_{L-1})$, its levels are

$$\mathcal{P}^{(0)}, \mathcal{P}^{(1)}, \dots, \mathcal{P}^{(L)},$$

where R_ℓ refines $\mathcal{P}^{(\ell)}$ into $\mathcal{P}^{(\ell+1)}$. The public `levels` property returns those $L+1$ packet systems. Construction requires the fine endpoint of every step to be equivalent to the coarse endpoint of the next step. As above, endpoint equivalence includes packet names and order, not merely an isomorphism between decompositions. The normalized tuple of adjacent steps remains available as `tower.refinements`.

The multilevel coordinate operations are

```
tower.refine(
    coordinates,
    *,
    source_level: int,
    target_level: int,
) -> PacketCoordinates

tower.coarsen(
    coordinates,
    *,
    source_level: int,
    target_level: int,
) -> PacketCoordinates
```

Both level indices must lie in `range(len(tower.levels))`. Refinement requires `target_level >= source_level`. Coarsening requires the opposite inequality. The coordinate owner must be equivalent to the declared source level. Equal source and target levels are legal identity operations and return the current coordinate object after the ownership check.

For addresses rather than coordinate values,

```
tower.descendant_address(
    coarse_index: int,
    fine_index: int,
    *,
    coarse_level: int,
    fine_level: int,
) -> bool
```

walks the fine index backward through the adjacent parent maps and reports whether it lands at the requested coarse index. It requires `fine_level > coarse_level`. This is a Boolean ancestry query. It does not return the intervening address tuple.

`tower.certificate()` combines every adjacent `PSC-II-T03` certificate into a `PSC-II-T05` certificate. Residual keys have the form `level_ell:coarse_name`. The tower is exact only when every local certificate is exact. Otherwise its combined exactness is numerical. This certificate establishes finite adjacent coherence. It does not claim that unrelated operators commute with the coordinate maps.

Worked example: a three-level tower

```
import numpy as np

from komorebi_math import PacketRefinement, PacketSystem, RefinementTower

coarsest = PacketSystem([np.eye(4)], names=["all"])
middle = PacketSystem(
    [
        np.diag([1, 1, 0, 0]),
        np.diag([0, 0, 1, 1]),
    ],
    names=["left", "right"],
)
finest = PacketSystem(
    [
        np.diag([1, 0, 0, 0]),
        np.diag([0, 1, 0, 0]),
        np.diag([0, 0, 1, 0]),
        np.diag([0, 0, 0, 1]),
    ],
    names=["x0", "x1", "x2", "x3"],
)

tower = RefinementTower(
    [
        PacketRefinement(coarsest, middle, (0, 0)),
        PacketRefinement(middle, finest, (0, 0, 1, 1)),
    ]
)
x = np.array([5.0, 6.0, 7.0, 8.0])
fine = tower.refine(
    coarsest.analyze(x), source_level=0, target_level=2
)
back = tower.coarsen(fine, source_level=2, target_level=0)

print("packet counts:", [len(level) for level in tower.levels])
print("fine reconstruction:", np.real_if_close(fine.reconstruct()).tolist())
print("coarse reconstruction:", np.real_if_close(back.reconstruct()).tolist())
print(
    "x3 descends from all:",
    tower.descendant_address(0, 3, coarse_level=0, fine_level=2),
)
certificate = tower.certificate()
print("certificate:", certificate.valid, certificate.exactness.value, certificate.theorem_id)
```

Output:

```
packet counts: [1, 2, 4]
fine reconstruction: [5.0, 6.0, 7.0, 8.0]
coarse reconstruction: [5.0, 6.0, 7.0, 8.0]
x3 descends from all: True
certificate: True numerical PSC-II-T05
```

Common refinements of commuting packet systems

Two packet systems on the same ambient space need not be ordered by refinement. When all of their projectors commute, their nonzero pairwise products form a common fine system. The stable function is

```
common_refinement(
    left: PacketSystem,
    right: PacketSystem,
    *,
    names: tuple[str, ...] | None = None,
) -> tuple[PacketSystem, PacketRefinement, PacketRefinement]
```

For left packets P_a and right packets Q_b, define

$$R_{ab} = P_a Q_b.$$

If $P_a Q_b = Q_b P_a$ for every pair, then

$$R_{ab}^2 = P_a^2 Q_b^2 = R_{ab}.$$

Distinct nonzero products annihilate because at least one of their left or right packet factors annihilates. Completeness follows from

$$\sum_{a,b} R_{ab} = \left(\sum_a P_a\right)\left(\sum_b Q_b\right) = I.$$

Finally,

$$\sum_b R_{ab} = P_a, \qquad \sum_a R_{ab} = Q_b,$$

so the result refines both inputs.

The implementation requires one backend name and one ambient dimension. It checks every commutator at the maximum input tolerance, generates products in left-major then right-minor order, and omits zero products. Default names are `"left_name & right_name"`. A supplied `names` tuple must match the number of retained nonzero packets, not the full Cartesian product size.

The return order is

```
(common_system, refinement_from_left, refinement_from_right)
```

The two refinement objects point from the respective input systems to the common system. Zero products do not appear in either parent tuple.

```
import numpy as np

from komorebi_math import PacketSystem, common_refinement

rows = PacketSystem(
    [
        np.diag([1, 1, 0, 0]),
        np.diag([0, 0, 1, 1]),
    ],
    names=["top", "bottom"],
)
columns = PacketSystem(
    [
        np.diag([1, 0, 1, 0]),
        np.diag([0, 1, 0, 1]),
```

```
    ],
    names=["even", "odd"],
)

common, to_rows, to_columns = common_refinement(rows, columns)

print("common names:", common.names)
print("row parents:", to_rows.parent_of_fine)
print("column parents:", to_columns.parent_of_fine)
print("packet count:", len(common))
```

Output:

```
common names: ('top & even', 'top & odd', 'bottom & even', 'bottom & odd')
row parents: (0, 0, 1, 1)
column parents: (0, 1, 0, 1)
packet count: 4
```

12.3 Weighted transfer graphs

A `WeightedTransferGraph` decomposes an ambient operator into a packet-diagonal part and supported directed transfers:

$$T = D + \sum_{e \in E} E_e.$$

For an edge $e : s(e) \to t(e)$, support means

$$E_e = P_{t(e)} E_e P_{s(e)}.$$

Every edge also has a positive integer weight $w(e)$. Path weights add:

$$w(e_k \cdots e_1) = \sum_{j=1}^{k} w(e_j).$$

The weights can be purely declared filtration data. With an exact SymPy backend and a prime p, they can instead be checked as lower bounds on the p-adic valuations of all edge entries.

Data records and constructor

The research interfaces are

```
WeightedTransferEdge(
    source: int,
    target: int,
    weight: int,
    operator: Any,
    name: str = "",
) -> None

WeightedPath(
    vertices: tuple[int, ...],
    edges: tuple[int, ...],
    weight: int,
) -> None

WeightedTransferGraph(
    packet_system: PacketSystem,
    diagonal: Any,
```

```
    edges: tuple[WeightedTransferEdge, ...] | list[WeightedTransferEdge],
    *,
    backend: str | LinearAlgebraBackend | None = None,
    prime: int | None = None,
    validate_weights: bool = True,
) -> None
```

`WeightedPath.edges` contains indices into `graph.edges`. `vertices` records the corresponding directed vertex sequence. Its `.length` property is `len(edges)`. Callers should normally use graph-generated paths rather than constructing inconsistent records manually, since `WeightedPath` itself is a plain frozen data record and does not validate incidence.

Graph construction enforces these structural rules.

Object	Required condition
Backend	Graph and packet system use the same backend name.
D	Ambient square shape, invertible, and packet diagonal, checked exactly on exact backends and at packet tolerance on numerical backends.
Edge address	Source and target are valid packet indices.
Edge matrix	Ambient operator shape and declared packet support, checked exactly on exact backends and at packet tolerance on numerical backends.
Weight	Declared as `int` and required at runtime to satisfy `weight > 0`.
Prime	`None` or a genuine prime.

Here “packet diagonal” means

$$D = \sum_a P_a D P_a.$$

It does not mean that D must be diagonal in the current coordinate basis. The constructor computes D^{-1} immediately. A singular diagonal part therefore fails with the selected backend’s inversion error before an inverse expansion can be requested.

When `prime=p`, `backend="sympy"`, and weight validation is enabled, the constructor additionally requires

$$\min_{i,j} v_p((E_e)_{ij}) \geq w(e)$$

for every edge. It also requires all entries of D and D^{-1} to have nonnegative p-adic valuation, placing both in filtration level zero. Zeros have infinite valuation. The declared weight is a lower bound, so it need not equal the minimum valuation.

The type annotation declares integer weights, but the edge data class does not perform a separate `isinstance(weight, int)` check. Supplying another positive numeric type can pass the positivity guard and then enter path arithmetic. Treat `int` as part of the public contract rather than relying on that runtime gap.

`validate_weights=False` bypasses those up-front valuation declarations, not the matrix-shape, packet-diagonal, or edge-support checks. It is an advanced escape hatch and should not be used to turn an unverified weight assignment into a claimed filtration.

The ambient dimension, reconstructed operator, and a copy of the diagonal inverse are available as properties:

```
graph.dimension
graph.operator
graph.diagonal_inverse
```

`dimension` is the ambient vector-space dimension, not the number of packet vertices.

Paths and minimum weights

`graph.adjacency()` returns one row per source packet. Each row consists of `(target, edge_index)` pairs in the original edge order. Parallel edges and self-loops are permitted.

All nonempty paths with total weight strictly below a threshold are enumerated by

```
graph.paths_below_weight(
    precision: int,
    *,
    source: int | None = None,
    target: int | None = None,
    max_paths: int | None = 1_000_000,
) -> tuple[WeightedPath, ...]
```

The strict inequality is important:

$$\mathcal{P}_{<q} = \{\pi : w(\pi) < q\}.$$

`precision <= 0` returns the empty tuple. Optional source and target filters restrict the returned paths, while the path-count guard limits generated search states. Positive weights make the set below a finite threshold finite, even when the graph has cycles, but its size can still grow rapidly. Results are sorted by weight, then length, vertices, and edge indices.

The least transfer weight is computed with a positive-weight shortest-path search:

```
graph.minimum_path_weight(source: int, target: int) -> int | float
```

It returns `0` when source equals target and `math.inf` when no directed path exists.

Filtered inverse expansion

Set

$$A_e = D^{-1}E_e.$$

Since

$$T = D\left(I + \sum_e A_e\right),$$

the formal inverse expansion is

$$T^{-1} = \sum_{k\geq 0}(-1)^k \left(\sum_e A_e\right)^k D^{-1}.$$

For a path $\pi = (e_1, \dots, e_k)$ in traversal order, the implemented path operator is

$$A_\pi D^{-1} = A_{e_k} \cdots A_{e_1} D^{-1}.$$

`graph.path_operator(path)` returns that matrix. The filtered truncation is

$$S_{<q} = D^{-1} + \sum_{\substack{\pi \neq \emptyset \\ w(\pi) < q}} (-1)^{|\pi|} A_\pi D^{-1}.$$

It is returned by

```
graph.truncated_inverse(
    precision: int,
    *,
    max_paths: int | None = 1_000_000,
) -> FilteredInverseExpansion
```

where `precision` must be positive. The output record contains

```
FilteredInverseExpansion(
    value,
    precision,
    included_paths,
    omitted_minimum_weight,
    certificate,
)
```

`omitted_minimum_weight` is the declared lower bound `precision`. It is not an independent search for the first omitted path. `.require()` requires the certificate and returns `value`.

There are two evidence paths.

Execution path	Certificate behavior
Exact backend plus prime	Computes the ordinary inverse, measures the minimum p-adic valuation of $T^{-1} - S_{<q}$, and requires it to be at least q. Exactness is `EXACT`.
Any other backend profile	Returns a valid `CONDITIONAL` certificate under the unchecked assumption that declared weights realize a complete multiplicative filtration.

The conditional path tries to compute the ordinary matrix inverse for a useful diagnostic. When available, it stores `ordinary_matrix_residual`. When the evaluated operator is singular, it sets `ordinary_inverse_available=False` and still returns the formal conditional truncation. This behavior is deliberate: the filtered word expression may be meaningful under the declared formal assumptions even when one numerical evaluation of T is singular. It must not be reported as an unconditional matrix inverse.

The truncation certificate uses theorem identifier `PSC-II-T10`. Its checked assumptions are packet support, positive path weights, and invertibility of the packet diagonal. In the conditional path it retains `"declared edge weights realize a complete multiplicative filtration"` as an unchecked assumption.

Exact inverse on an acyclic graph

If the directed packet graph is acyclic, no path has more than $N - 1$ edges, where N is the number of packets. The word expansion is then finite. The methods are

```
graph.is_acyclic() -> bool
graph.exact_inverse_if_acyclic(
    *, max_paths: int | None = 1_000_000
) -> ComputationResult[Any]
```

The second method rejects a cyclic graph, enumerates every possible acyclic path, compares the path sum to the backend's ordinary inverse, and returns a `PSC-II-C11` certificate. Exact backends produce exact evidence. Numerical

backends accept an inverse residual at the packet system's tolerance. This method does not require a prime because acyclicity terminates the expansion as an ordinary finite matrix identity.

Scalar transfer-series matrices

The graph also exposes a combinatorial generating matrix:

```
graph.weighted_adjacency(variable)
graph.transfer_series_matrix(variable)
```

Its target-row, source-column entry is

$$\mathcal{A}(z)_{t,s} = \sum_{e:s(e)=s,\ t(e)=t} z^{w(e)},$$

and the transfer series is

$$(I - \mathcal{A}(z))^{-1}.$$

This scalar matrix records graph incidence and weights. It does not contain the ambient edge operators E_e, their numerical coefficients, or the packet-diagonal operator D.

Worked example: an exact 2-adic weighted chain

This exact SymPy example has edge valuations at least their declared weights. All nonempty paths have weight below 4, so the depth-four truncation is the ordinary inverse.

```
import sympy as sp

from komorebi_math import (
    PacketSystem,
    WeightedTransferEdge,
    WeightedTransferGraph,
)

packets = PacketSystem(
    [sp.diag(1, 0, 0), sp.diag(0, 1, 0), sp.diag(0, 0, 1)],
    names=["a", "b", "c"],
    backend="sympy",
)
edges = [
    WeightedTransferEdge(
        0,
        1,
        1,
        sp.Matrix([[0, 0, 0], [2, 0, 0], [0, 0, 0]]),
        name="a_to_b",
    ),
    WeightedTransferEdge(
        1,
        2,
        2,
        sp.Matrix([[0, 0, 0], [0, 0, 0], [0, 4, 0]]),
        name="b_to_c",
    ),
]
graph = WeightedTransferGraph(packets, sp.eye(3), edges, prime=2)
paths = graph.paths_below_weight(4)
expansion = graph.truncated_inverse(4)
z = sp.symbols("z")
```

```
print("paths:", [(path.vertices, path.weight) for path in paths])
print("minimum 0->2:", graph.minimum_path_weight(0, 2))
print("operator:")
print(graph.operator)
print("truncated inverse:")
print(expansion.value)
print(
    "certificate:",
    expansion.certificate.valid,
    expansion.certificate.exactness.value,
)
print("series:")
print(graph.transfer_series_matrix(z))
```

Output:

```
paths: [((0, 1), 1), ((1, 2), 2), ((0, 1, 2), 3)]
minimum 0->2: 3
operator:
Matrix([[1, 0, 0], [2, 1, 0], [0, 4, 1]])
truncated inverse:
Matrix([[1, 0, 0], [-2, 1, 0], [8, -4, 1]])
certificate: True exact
series:
Matrix([[1, 0, 0], [z, 1, 0], [z**3, z**2, 1]])
```

Worked example: a conditional NumPy truncation

Here the only edge has declared weight 2, so no nonempty path is included at depth 2. With no exact prime-valuation profile, the identity matrix is a conditional filtered approximation rather than a certified ordinary inverse.

```
import numpy as np

from komorebi_math import (
    PacketSystem,
    WeightedTransferEdge,
    WeightedTransferGraph,
)

packets = PacketSystem(
    [np.diag([1, 0]), np.diag([0, 1])],
    names=["u", "v"],
)
edge = WeightedTransferEdge(
    0,
    1,
    2,
    np.array([[0.0, 0.0], [0.25, 0.0]]),
    name="u_to_v",
)
graph = WeightedTransferGraph(packets, np.eye(2), [edge])
expansion = graph.truncated_inverse(2)

print("value:", np.real_if_close(expansion.value).tolist())
print("paths:", len(expansion.included_paths))
print("exactness:", expansion.certificate.exactness.value)
print("unchecked:", expansion.certificate.assumptions_unchecked)
print(
    "ordinary residual:",
    expansion.certificate.residuals["ordinary_matrix_residual"],
)
```

Output:

```
value: [[1.0, 0.0], [0.0, 1.0]]
paths: 0
exactness: conditional
unchecked: ('declared edge weights realize a complete multiplicative filtration',)
ordinary residual: 0.25
```

The nonzero ordinary residual does not invalidate the conditional statement. It shows precisely why the result must not be relabeled as an exact or numerical ordinary inverse at this depth.

12.4 Linear refinement defects

A coordinate coarsening map need not intertwine a fine operator with a coarse operator. `LinearRefinementDefect` makes the failure explicit.

Let

$$C : V_f \longrightarrow V_c, \qquad T_f : V_f \longrightarrow V_f, \qquad T_c : V_c \longrightarrow V_c.$$

The defect is

$$\Delta = CT_f - T_cC.$$

Zero defect means that this diagram commutes. It does not mean that the two spaces, operators, or packet systems are equal.

The research constructor is

```
LinearRefinementDefect(
    coarsen: Any,
    fine_operator: Any,
    coarse_operator: Any,
    backend: str | LinearAlgebraBackend | None = None,
) -> None
```

If C has shape $m \times n$, the fine operator must be $n \times n$ and the coarse operator must be $m \times m$. Shape failures raise `DimensionError`. The `.matrix` property computes Δ, and

```
defect.is_exact(*, tolerance: float | None = None) -> bool
```

asks the backend whether that matrix is zero. Despite its name, the method uses the selected backend's zero test. Under a numerical backend the optional tolerance controls numerical zero. It returns only a Boolean, not a certificate.

Propagated identities

The remaining methods return `ComputationResult` objects and independently verify a finite matrix identity.

For a positive integer n, the power defect is

$$CT_f^n - T_c^nC = \sum_{k=0}^{n-1} T_c^{n-1-k}\Delta T_f^k.$$

It is implemented by

```
defect.power(exponent: int) -> ComputationResult[Any]
```

with theorem identifier `PSC-II-T20`. The exponent must be at least one.

If

$$f(z) = \sum_{k=0}^{d} a_k z^k,$$

then

$$Cf(T_f) - f(T_c)C = \sum_{k=1}^{d} a_k \sum_{j=0}^{k-1} T_c^{k-1-j} \Delta T_f^j.$$

The call

```
defect.polynomial(coefficients) -> ComputationResult[Any]
```

interprets `coefficients[k]` as a_k, so coefficients are in ascending power order. The constant term cancels and makes no defect contribution. Each nonzero positive-degree term is assembled through `power(k)`. The resulting certificate is `PSC-II-C21`.

For any z in both resolvent sets, define

$$R_f(z) = (zI_f - T_f)^{-1}, \qquad R_c(z) = (zI_c - T_c)^{-1}.$$

Then

$$CR_f(z) - R_c(z)C = R_c(z)\Delta R_f(z).$$

`defect.resolvent(z)` computes the right side, compares it with the direct difference, and returns a `PSC-II-T22` result. A singular requested resolvent fails through the backend's inversion error. No conditional resolvent is returned.

The exponential identity is the Duhamel formula

$$Ce^{tT_f} - e^{tT_c}C = \int_0^t e^{(t-s)T_c} \Delta e^{sT_f}\, ds.$$

`defect.exponential(t)` obtains the integral as the upper-right block of

$$\exp\left(t \begin{bmatrix} T_c & \Delta \\ 0 & T_f \end{bmatrix} \right)$$

and compares it with the direct exponential difference. The result carries theorem identifier `PSC-II-T24`. SymPy uses its exact matrix exponential. Numerical backends require SciPy's `scipy.linalg.expm`. If SciPy is unavailable, the method raises `BackendUnavailableError` and identifies the `scientific` extra.

The numerical residual bounds are method-specific:

Method	Residual name	Numerical acceptance bound
`power()`	`power_identity`	10^{-10}
`polynomial()`	`polynomial_identity`	10^{-10}
`resolvent()`	`resolvent_identity`	10^{-9}
`exponential()`	`exponential_identity`	10^{-8}

Exact backends require zero residual and store no tolerance. These residuals certify agreement between two finite evaluations of each identity. They do not bound modeling error between a chosen fine model and the phenomenon it is intended to represent.

Multilevel telescoping

For levels $V_0, \dots, V_L$, let

$$C_i : V_{i+1} \to V_i, \qquad T_i : V_i \to V_i, \qquad \Delta_i = C_i T_{i+1} - T_i C_i.$$

With

$$C_{[a,b)} = C_a C_{a+1} \cdots C_{b-1}, \qquad C_{[a,a)} = I,$$

the total defect telescopes as

$$C_{[0,L)} T_L - T_0 C_{[0,L)} = \sum_{i=0}^{L-1} C_{[0,i)} \Delta_i C_{[i+1,L)}.$$

The module-level research function is

```
multilevel_defect(
    coarsenings: Iterable[Any],
    operators: Iterable[Any],
    *,
    backend: str | LinearAlgebraBackend | None = None,
) -> ComputationResult[Any]
```

A chain with L coarsening maps requires exactly $L + 1$ operators. The result uses theorem identifier `PSC-II-T25`. NumPy accepts the telescoping residual at 10^{-9}, while an exact backend requires zero.

Worked example: exact propagation of a nonzero defect

This SymPy run chooses a nonintertwining coarsening map.

```
import sympy as sp

from komorebi_math import LinearRefinementDefect

C = sp.Matrix([[1, 0]])
T_f = sp.Matrix([[1, 2], [3, 4]])
T_c = sp.Matrix([[1]])
defect = LinearRefinementDefect(C, T_f, T_c, backend="sympy")

power_two = defect.power(2)
polynomial = defect.polynomial([3, 2, 1])  # 3 + 2z + z^2
resolvent = defect.resolvent(10)

print("delta:", defect.matrix)
print(
    "power two:",
    power_two.value,
    power_two.certificate.valid,
    power_two.certificate.exactness.value,
)
print("direct power two:", C * T_f**2 - T_c**2 * C)
print("polynomial:", polynomial.value)
print("resolvent:", resolvent.value)
```

Output:

```
delta: Matrix([[0, 2]])
power two: Matrix([[6, 10]]) True exact
direct power two: Matrix([[6, 10]])
polynomial: Matrix([[6, 14]])
resolvent: Matrix([[1/72, 1/24]])
```

The nonzero Δ is not a failed certificate. It is the mathematical quantity being propagated. The certificates validate the power, polynomial, and resolvent identities that express its consequences.

12.4.1 Exactness, result types, and API placement

The chapter's operations use several evidence patterns. Keep them distinct.

Operation	Return form	Possible exactness
`PacketRefinement.certificate()`	`Certificate`	Exact or numerical by backend
`RefinementTower.certificate()`	`Certificate`	Exact if all levels are exact, otherwise numerical
Coordinate and operator conversions	Raw typed value	Rely on the required construction certificate
`WeightedTransferGraph.truncated_inverse()`	`FilteredInverseExpansion`	Exact only for exact backend plus prime. Otherwise conditional
`exact_inverse_if_acyclic()`	`ComputationResult`	Exact or numerical by backend
Defect propagation methods	`ComputationResult`	Exact or numerical by backend

A particularly important boundary is

$$\text{exact matrix backend} \not\Rightarrow \text{exact filtered inverse certificate}.$$

The filtered inverse also needs an exact prime-valuation profile. SymPy without `prime=` takes the conditional path because no executed valuation check binds the declared weights to a complete filtration.

The imports are divided as follows.

Name	`komorebi_math` root	`komorebi_math.stable`	`komorebi_math.refinement`
`PacketRefinement`	Yes	Yes	Yes
`RefinementTower`	Yes	Yes	Yes
`common_refinement`	Yes	Yes	Yes
`WeightedTransferEdge`	Yes	No	Yes
`WeightedTransferGraph`	Yes	No	Yes
`LinearRefinementDefect`	Yes	No	Yes
`WeightedPath`	No	No	Yes
`FilteredInverseExpansion`	No	No	Yes
`multilevel_defect`	No	No	Yes

For a long-lived application that only needs packet refinement and common refinement, import the three stable names from `komorebi_math.stable`. Code using weighted transfer or defect propagation should pin the package version and treat those interfaces as research-tier APIs.

12.4.2 Failure contracts and edge cases

The principal controlled failures are summarized below.

Situation	Failure behavior
Coarse and fine backends differ	`ValidationError`
Ambient dimensions differ	`DimensionError`
Parent tuple has the wrong length	`DimensionError`
Parent is outside the coarse range	`ValueError`
A coarse packet has no child	`ValidationError`
Child sums do not reconstruct a coarse projector	Invalid `PSC-II-T03` certificate is required, raising `ValidationError`
Coordinates belong to the wrong endpoint	`ValidationError`
Empty tower	`ValueError`
Adjacent tower endpoints are not equivalent	`ValidationError`
Refinement requested toward a coarser level, or coarsening toward a finer level	`ValueError`
`refine()` or `coarsen()` level lies outside the tower	`IndexError`
Common-refinement inputs differ in backend or dimension	`ValidationError`
Common-refinement projectors do not commute	`ValidationError`
Weighted edge has negative index or nonpositive weight	`ValueError`
Weighted edge references an unknown packet	`ValueError`
Diagonal or edge has the wrong ambient shape	`DimensionError`
Declared diagonal is not packet diagonal	`ValidationError`
Edge is not supported on its declared source and target	`ValidationError`
Exact p-adic edge valuation falls below its declared weight	`ValidationError`
Composite `prime`	`ValidationError`
Nonpositive truncation precision	`ValueError`
Generated weighted paths exceed `max_paths`	`ValueError`
Exact finite inverse requested on a cyclic graph	`ValidationError`
Defect matrices have incompatible dimensions	`DimensionError`
Power defect exponent is below one	`ValueError`
A requested resolvent is singular	Backend inversion error
Numerical exponential requested without SciPy	`BackendUnavailableError`
L coarsenings are not paired with $L + 1$ operators	`DimensionError`

Four representative messages were reproduced from the delivered source:

```
ValidationError: every coarse packet must have at least one fine child
ValidationError: common refinement requires commuting packet systems
ValidationError: the packet transfer graph contains a directed cycle
ValueError: power defect exponent must be positive
```

Three subtleties deserve explicit treatment.

First, `descendant_address()` requires `fine_level > coarse_level`, but it does not perform the same explicit level and address range checks as `refine()` and `coarsen()`. Python negative indexing, an incidental `IndexError`, or a simple `False` result can therefore occur for malformed indices. Treat valid levels and packet addresses as caller preconditions for that query.

Second, `common_refinement()` omits zero product packets. Its generated names and parent tuples therefore need not have the full Cartesian length `len(left) * len(right)`.

Third, `paths_below_weight()` can safely traverse cycles only because weights are positive and the threshold is strict. The search remains combinatorial. Keep the default `max_paths` guard or choose a smaller task-specific bound when the graph is dense. Setting it to `None` removes the resource guard. It does not improve the mathematical guarantee.

12.4.3 Shipped pipeline and verification boundary

The repository includes `examples/refinement_pipeline.py`. It builds one two-level packet refinement, performs a roundtrip, constructs residue and CRT data used in Chapter 13, and computes an exact finite inverse for an acyclic weighted chain. Executed from the recorded reference repository, it prints

```
refinement levels: 2
CRT address: 29
minimum transfer weight 0 to 2: 3
```

The focused source replay for this chapter ran

```
tests/test_refinement_towers.py
tests/test_weighted_transfer.py
tests/test_linear_defects.py
```

and produced

```
21 passed
```

Those tests cover coordinate and operator roundtrips, support reflection, three-level coherence, commuting and non-commuting common refinements, exact acyclic inverses, weighted path ordering, conditional and exact p-adic truncations, valuation rejection, scalar transfer series, a singular conditional evaluation, path-count guards, composite-prime rejection, and all implemented linear defect identities.

Across Chapters 9 through 12, the combined focused replay contains **71 unique tests** and all 71 pass. The chapter-local replay counts above intentionally overlap where one module exercises both packet and algebra, or packet and transfer, contracts. They must not be added as if they were disjoint test sets.

> **Chapter 12 summary.** A refinement is a certified parent-sum law on packet projectors sharing one ambient space. Towers compose those laws. Commuting products build common fine packets. Weighted transfer expansions are exact only when their executed valuation or acyclicity checks justify that claim, and linear defects quantify rather than conceal a failure of operator intertwining.

13. Residue, CRT, lattice, and Smith coordinates

Choose by the question. Use residue analysis to split a finite list by index. Use nested addresses to describe a second split within each channel. Use CRT to satisfy simultaneous congruences with coprime moduli. Use the lattice and Smith sections when the input is an exact matrix or rational projector. The examples in Sections 13.2 to 13.3 deliberately use the same small numbers to show why nested addressing and CRT return different answers.

Integer addresses support four related but distinct finite coordinate systems. Residue coordinates split a sequence into congruence channels. Nested residue maps refine one such split by positional digits. Chinese-remainder maps identify one address from compatible residues modulo coprime factors. Lattice and Smith coordinates move exact rational packet data and prime-power congruences into bases where denominators and obstructions are visible.

These constructions share arithmetic notation, but they must not be conflated. The nested address $r + Ms$ works for every pair of positive moduli. CRT requires coprime factors and generally produces a different representative. The lattice and Smith paths require the optional SymPy exact dependency. There is no NumPy approximation of Smith reduction or lattice separation in this release.

The broad implementation surface lives in `komorebi_math.refinement`:

```
from komorebi_math.refinement import (
    LatticePacketData,
    SmithCongruenceProblem,
    crt_pair,
    nested_residue_address,
    residue_analyze,
)
```

`residue_analyze` and `crt_pair` are also convenient root exports. The frozen `stable`, `v1`, and `api.v1` namespaces include the residue-coordinate data path, but not the full CRT, lattice, and Smith research surface. Importing from the defining domain makes that boundary explicit.

13.1 Residue analysis and synthesis

$$x_r[k] = x[r + Mk].$$

For a positive modulus M, the channel with residue r contains the entries whose original indices are congruent to r:

$$x_r[k] = x[r + Mk], \qquad x[n] = x_{n \bmod M}\left[\left\lfloor \frac{n}{M} \right\rfloor\right].$$

The public record and functions are

```
ResidueCoordinates(
    modulus: int,
    channels: tuple[np.ndarray, ...],
    length: int,
)

residue_analyze(values: Any, modulus: int) -> ResidueCoordinates

residue_synthesize(
    channels: ResidueCoordinates | tuple[Any, ...],
```

```
    *,
    length: int | None = None,
) -> np.ndarray
```

`ResidueCoordinates.reconstruct()` is the canonical inverse. Its `refine(fine_modulus, copy=True)` and `coarsen(coarse_modulus, copy=True)` methods delegate to the finite polyphase refinement law developed in Chapter 14. `nested(inner_modulus)` instead returns one residue record for every outer channel.

Channel lengths may differ by one. If $L = qM + t$, with $0 \leq t < M$, the first t channels have $q + 1$ entries and the remaining channels have q. The constructor checks a positive modulus, the channel count, and a nonnegative declared length. Reconstruction performs the decisive ragged-length check. A manually assembled record with the wrong shape can therefore exist until it is reconstructed.

When `residue_synthesize` receives raw channels, it infers the output length from their total sizes unless `length=` is supplied. When it receives an existing `ResidueCoordinates`, the record's own length is authoritative.

Executed ragged-channel example

```
import numpy as np
from komorebi_math.refinement import residue_analyze, residue_synthesize

values = np.arange(17)
coordinates = residue_analyze(values, 5)

print(tuple(channel.tolist() for channel in coordinates.channels))
print(tuple(len(channel) for channel in coordinates.channels))
print(residue_synthesize(coordinates).tolist())
```

The audited output is

```
([0, 5, 10, 15], [1, 6, 11, 16], [2, 7, 12],
 [3, 8, 13], [4, 9, 14])
(4, 4, 3, 3, 3)
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16]
```

Residues also have an exact packet-system realization:

```
residue_packet_system(
    length: int,
    modulus: int,
    *,
    backend: str | LinearAlgebraBackend | None = None,
    names: tuple[str, ...] | None = None,
) -> PacketSystem

packet_coordinates_from_residues(
    values: Any,
    modulus: int,
    *,
    backend: str | LinearAlgebraBackend | None = None,
) -> PacketCoordinates
```

For ambient length L, the residue projector keeps standard coordinates with indices $n \equiv r \pmod{M}$. The packet certificate verifies idempotence, pairwise annihilation, and completeness on that finite ambient space. It does not turn an arbitrary infinite sequence into a finite object.

13.2 Nested residue addresses

Let r select a class modulo M, then let s select a class modulo K inside that outer channel. The flattened address is

$$a = r + Ms, \qquad 0 \le r < M, \quad 0 \le s < K.$$

The implementation is

```
nested_residue_address(
    outer_residue: int,
    outer_modulus: int,
    inner_residue: int,
    inner_modulus: int,
) -> int

nested_residue_analysis(
    values: Any,
    outer_modulus: int,
    inner_modulus: int,
) -> ResidueCoordinates
```

The returned flattened record has modulus MK. Its channel at $a = r + Ms$ equals the inner-s channel of the outer-r channel, so nested analysis is exactly direct residue analysis modulo MK. No coprimality condition is needed because this is positional refinement, not simultaneous congruence solving.

For example,

```
assert nested_residue_address(1, 4, 2, 3) == 9
```

The address 9 records outer digit 1 and inner digit 2. In the next section, the CRT solution for the congruences $x \equiv 1 \pmod 4$ and $x \equiv 2 \pmod 3$ is 5, not 9. That contrast is the simplest way to remember which map is being used.

13.3 Chinese remainder maps

$$x \equiv r_i \pmod{m_i}.$$

For positive pairwise-coprime moduli m_i, the Chinese remainder theorem provides one normalized address

$$0 \le x < \prod_i m_i, \qquad x \equiv r_i \pmod{m_i}.$$

The immutable result record is

```
CRTAddress(
    residue: int,
    modulus: int,
    residues: tuple[int, ...],
    moduli: tuple[int, ...],
)
```

Use the checked factories rather than constructing that record directly:

```
crt_pair(
    a: int,
    modulus_a: int,
    b: int,
    modulus_b: int,
) -> CRTAddress

crt_many(
```

```
    residues: Iterable[int],
    moduli: Iterable[int],
) -> CRTAddress

factor_residue(
    residue: int,
    moduli: Iterable[int],
) -> CRTAddress

dyadic_odd_factor(
    residue: int,
    dyadic_power: int,
    odd_modulus: int,
) -> CRTAddress
```

`crt_pair` and `crt_many` validate pairwise coprimality and normalize both the component residues and the combined residue. `factor_residue` starts from an integer, normalizes it modulo the product, and reports its residues. `dyadic_odd_factor(x, a, m)` is the specialized factorization modulo $2^a m$, with $a \geq 0$ and m positive and odd.

```python
from komorebi_math.refinement import crt_many, crt_pair, factor_residue

pair = crt_pair(1, 4, 2, 3)
assert pair.residue == 5
assert pair.modulus == 12
assert pair.residues == (1, 2)

many = crt_many((2, 3, 1), (3, 5, 7))
assert tuple(many.residue % m for m in many.moduli) == many.residues

assert factor_residue(53, (8, 5)).residues == (5, 3)
```

A direct `CRTAddress(...)` checks only the record's local consistency. It does not independently establish that its moduli are coprime. The factory functions are the public theorem-bearing path. Noncoprime moduli are rejected rather than silently changed into a generalized CRT problem.

13.4 Lattice packet data and exact separation

$$w_\alpha = \max\left(0, -\min_{i,j} v_p((B^{-1}P_\alpha B)_{ij})\right).$$

Residue channels organize addresses. `LatticePacketData` instead measures how an exact rational packet decomposition interacts with a chosen integer lattice. It is a SymPy-exact construction:

```
LatticePacketData(
    lattice_basis: Any,
    projectors: Any,
    *,
    prime: int,
    split_basis: Any | None = None,
)
```

If the columns of B are a lattice basis and P_α is a packet projector, then

$$\widetilde{P}_\alpha = B^{-1}P_\alpha B$$

is the projector in lattice coordinates. For prime p, zeros are assigned $v_p(0) = \infty$, and the denominator weight is

$$w_\alpha = \max\left(0, -\min_{i,j} v_p((\widetilde{P}_\alpha)_{ij})\right).$$

The finite packet hypotheses are checked exactly:

$$P_\alpha^2 = P_\alpha, \qquad P_\alpha P_\beta = 0 \quad (\alpha \neq \beta), \qquad \sum_\alpha P_\alpha = I.$$

Self-adjointness is not required. Oblique rational projectors are permitted. The main interface is

Member	Meaning
`dimension`	ambient lattice dimension
`certificate()`	exact packet/lattice construction certificate
`projector_in_lattice_basis(i)`	$B^{-1}P_iB$
`projector_weights()`	all w_i
`separation_weight`	$\max_i w_i$
`precision_delays()`	one `PrecisionDelay` per projector
`inclusion_matrix()`	$B^{-1}S$ for a supplied split basis S
`gluing_module()`	Smith invariants of the integral inclusion
`gluing_weight_certificate()`	exact comparison of gluing and separation weights

The valuation helpers are

```
padic_valuation(value: int, prime: int) -> int | float
rational_padic_valuation(value: Any, prime: int) -> int | float
```

Import these from `komorebi_math.refinement`. The root package has a different arithmetic `padic_valuation` export, so a domain-qualified import avoids a name collision.

Executed denominator example

```
import sympy as sp
from komorebi_math.refinement import (
    LatticePacketData,
    rational_padic_valuation,
)

p0 = sp.Matrix([[1, sp.Rational(1, 8)], [0, 0]])
p1 = sp.eye(2) - p0
data = LatticePacketData(sp.eye(2), [p0, p1], prime=2)

print(rational_padic_valuation(sp.Rational(3, 8), 2))
print(data.projector_weights())
print(data.separation_weight)
print(data.precision_delays()[0].input_depth(10))
```

The audited values are

```
-3
(3, 3)
3
13
```

The code assumes rational exact entries when weights are requested. Packet axioms alone do not exclude an algebraic irrational such as `sqrt(2)`. Such an entry can survive construction and then fail valuation conversion. Rationality is therefore a caller-visible hypothesis, not an inferred theorem.

13.5 Smith congruence problems

Smith coordinates diagonalize an integer matrix up to unimodular changes of basis. The shipped decomposition has

```
SmithDecomposition(
    matrix,
    diagonal,
    left,
    right,
    invariant_factors,
)

smith_decomposition(matrix: Any) -> SmithDecomposition
```

and satisfies

$$LAR = D.$$

`rank` is the number of nonzero Smith factors. `certificate()` directly checks the displayed matrix identity. The underlying SymPy algorithm supplies the unimodularity and invariant-factor divisibility properties. The certificate's assumption text names them, but this release does not independently recompute both properties inside the certificate.

Prime-power systems use

```
SmithCongruenceProblem(
    matrix: Any,
    rhs: Any,
    *,
    prime: int,
```

```
    depth: int,
)

solve() -> SmithCongruenceSolution
enumerate_solutions(*, limit: int = 100000) -> tuple[tuple[int, ...], ...]
```

For

$$Ax \equiv b \pmod{p^k}, \qquad b' = Lb,$$

let r be the Smith rank and

$$e_i = \min(v_p(d_i), k).$$

The transformed divisibility conditions are

$$b'_i \equiv 0 \pmod{p^{e_i}} \quad (i < r),$$

and every transformed zero row must vanish modulo p^k. With n unknowns, the exact solution count is

$$p^{(n-r)k+\sum_{i<r} e_i}.$$

The result record exposes all evidence needed to audit that conclusion:

```
solvable                prime
depth                   modulus
rank                    smith_factors
smith_exponents         transformed_rhs
obstruction_rows        solution_count
particular_solution     kernel_generators
certificate
```

An obstructed problem returns a valid exact certificate: the certificate establishes the obstruction. `result.require()` first requires that certificate and then raises because there is no solution.

Executed prime-power example

```
from komorebi_math.refinement import SmithCongruenceProblem

problem = SmithCongruenceProblem(
    [[2, 0, 0], [0, 4, 0], [0, 0, 0]],
    [2, 4, 0],
    prime=2,
    depth=3,
)
solution = problem.solve().require()

print(solution.smith_factors)
print(solution.smith_exponents)
print(solution.solution_count)
print(solution.particular_solution)
print(solution.kernel_generators)
```

Output:

```
(2, 4)
(1, 2)
64
(1, 1, 0)
((4, 0, 0), (0, 2, 0), (0, 0, 1))
```

Signed diagonal entries matter when the particular solution is reconstructed. The regression contract is

$$-2x \equiv 2 \pmod 8 \quad \Longrightarrow \quad x \in \{3, 7\}.$$

`enumerate_solutions` first rejects a known solution count above `limit`. For some dependent generator configurations its fallback can still inspect a much larger ambient modular space. The limit is a solution-count guard, not a strict bound on every internal loop.

13.6 Gluing modules, precision delays, and refinement-weight monotonicity

When a split basis S is supplied, `LatticePacketData` forms the inclusion matrix

$$C = B^{-1}S.$$

It must be integral and nonsingular. If the nonzero Smith factors are d_i, the `GluingModule(invariant_factors, prime)` record reports

$$|G| = \prod_i d_i, \qquad \text{length}_p(G) = \sum_i v_p(d_i), \qquad w_G = \max_i v_p(d_i).$$

Its public properties are `prime_exponents`, `separation_weight`, `length`, and `order`. This is not an arbitrary lattice-intersection algorithm: the caller provides the split basis, and the implementation checks the resulting integral containment and Smith data.

`PrecisionDelay(weight, prime, projector_index=None)` translates a requested output precision into the required input depth:

$$L_{\text{input}} = L_{\text{output}} + w.$$

The gluing example shipped in `examples/exact_lattice_separation.py` uses

$$P_0 = \begin{pmatrix} 1 & 1/4 \\ 0 & 0 \end{pmatrix}, \qquad P_1 = I - P_0, \qquad S = \begin{pmatrix} 1 & -1 \\ 0 & 4 \end{pmatrix},$$

and prints

```
projector weights: (2, 2)
gluing factors: (1, 4)
congruence solutions: 64
```

For a genuine fine decomposition of a coarse one, `refinement_weight_monotonicity(coarse, fine)` first finds the unique parent of every nonzero fine projector using two-sided containment,

$$P_\alpha Q_\beta = Q_\beta P_\alpha = Q_\beta,$$

then verifies that each coarse projector is exactly reconstructed by its children. Only after those checks does it certify

$$w_{\text{coarse}} \leq w_{\text{fine}}.$$

An unrelated decomposition cannot pass merely because its denominator weights happen to be larger. Zero fine projectors receive parent diagnostic `-1` and do not falsify the reconstruction law.

13.7 Exact, numerical, and failure examples

Residue analysis accepts NumPy data and can therefore carry integers, floats, or complex values. Its reconstruction is a finite reindexing, so it introduces no arithmetic error by itself. Packet certificates still follow the backend rules of Chapter 11. CRT, lattice valuation, gluing, and Smith congruence records are integer or rational constructions. The lattice and Smith branches require SymPy.

The main controlled failures are:

Condition	Public outcome
nonpositive residue modulus	validation failure
wrong residue channel count or ragged length	construction or reconstruction failure
noncoprime CRT moduli	checked factory raises
negative dyadic exponent or nonpositive/even odd factor	checked factory raises
singular lattice basis	validation failure
composite value supplied as `prime`	validation failure
nonprojector, overlapping, or incomplete packet family	exact packet validation failure
nonintegral or singular split inclusion	gluing validation failure
nonrational lattice entry during valuation	exact conversion failure
inconsistent Smith congruence	solvable is false. Obstruction is certified
exact solution count above enumeration limit	guarded enumeration failure
unrelated coarse and fine packet decompositions	refinement-parent validation failure

Python integers and SymPy rationals are unbounded. There is no explicit denominator-overflow exception or precision budget in this subsystem. Very large exact matrices can instead incur large time and memory costs. Callers that need a resource envelope should impose one before decomposition or enumeration.

The installed provenance identifiers used here are:

Identifier	Executed contract
`PSC-II-T06`	exact residue analysis/synthesis
`PSC-II-T07`	nested residue address
`PSC-II-T08`	two-modulus CRT
`PSC-II-C09`	many-modulus CRT
`PSC-II-D12`	lattice packet definition
`PSC-II-T13`	denominator/gluing weight facets
`PSC-II-T15`	refinement-weight monotonicity
`PSC-II-SNF`	integer Smith decomposition provenance
`PSC-II-T18`	prime-power Smith solve

`PSC-II-T13` labels two related provenance records. Read them as denominator and gluing facets of the same finite separation result, not as two unrelated theorems.

13.8 Strict portable refinement records and schema rejection

Only `ResidueCoordinates` among the Chapter 13 result types has a registered portable codec. There are no portable codecs in this release for `CRTAddress`, `LatticePacketData`, `GluingModule`, `PrecisionDelay`, `SmithDecomposition`, or `SmithCongruenceSolution`. Serialize those only through an application-defined format that records its own schema.

The Chapter 14 refinement plan objects have strict local dictionaries. For example,

```
PolyphaseRefinement(4, 12).to_dict()
```

returns

```
{
  "schema": "komorebi-math.polyphase-refinement",
  "schema_version": 1,
  "coarse_order": 4,
  "fine_order": 12
}
```

The three local schema names are

```
komorebi-math.polyphase-refinement
komorebi-math.polyphase-refinement-tower
komorebi-math.lattice-polyphase-refinement
```

Local parsers reject missing or extra fields, unknown schema names, and unsupported versions. Only `PolyphaseRefinement` supplies local `to_json`/`from_json`. The tower and lattice plans supply `to_dict`/`from_dict`.

Portable persistence uses the separate digest-checked envelope from Chapter 61. Its outer fields are exactly

```
schema  schema_version  producer_version  payload_sha256  payload
```

with schema `komorebi-math.portable` and version 1. The relevant codec tags are

Codec tag	Payload fields
`refinement.residue_coordinates`	`modulus`, `channels`, `length`
`refinement.polyphase`	`coarse_order`, `fine_order`
`refinement.polyphase_tower`	`orders`
`refinement.lattice_polyphase`	`coarse_strides`, `fine_strides`
`signals.polyphase_coordinates`	`channels`, `order`, `axis`, `original_shape`
`signals.lattice_polyphase_coordinates`	`channels`, `strides`, `original_shape`
`arithmetic.polyphase_coordinates`	`modulus`, `channels`, `length`

The loader rejects a malformed field set, unknown codec, unsupported version, duplicate JSON keys, and a payload digest mismatch before reconstructing the object. “Strict” here principally means field-strict. Some scalar values are coerced by local plan constructors, and `producer_version` is not part of the payload digest. Applications should not treat the envelope as a digital signature or an authorization mechanism.

The focused Chapter 13 replay covers six residue/CRT tests and ten lattice/Smith tests. The larger shipped deep audit executed 11,716 checks in the residue/CRT and Smith-congruence groups with zero failures. The test record supports the implemented finite contracts. It does not establish asymptotic performance or a numerical lattice backend.

> **Chapter 13 summary.** Residue coordinates split and exactly reassemble finite arrays. Nested digits and CRT solve different address problems. Rational packet projectors expose prime-adic denominator delays, while Smith coordinates turn a prime-power linear congruence into explicit obstruction, count, particular-solution, and kernel data. Portable coverage is intentionally narrower than the complete research surface.

14. Polyphase systems and refinement

Before the first refinement. Identify the record type in Section 14.1 and retain its original length or shape. For a one-dimensional plan, check that the fine order is divisible by the coarse order. For a lattice plan, check divisibility along every axis. Work through the address table in Section 14.2 before changing channel data, then compare refinement with direct fine analysis using the certificate described in Section 14.5.

Polyphase coordinates are residue coordinates with an explicit signal, arithmetic, or lattice realization. A refinement plan changes the channel order without first rebuilding the ambient object. Every implementation in this chapter is finite: it proves array reindexing and finite tower coherence, not convergence of an infinite refinement limit or existence of a continuum wavelet.

The complete surface is split across three domain namespaces:

```
import komorebi_math.signals as signals
import komorebi_math.arithmetic as arithmetic
import komorebi_math.refinement as refinement
```

The frozen `stable`, `v1`, and `api.v1` namespaces re-export the supported records and plan objects. The package root exposes only `PolyphaseSystem` and the signal `polyphase_analyze` from this family, so examples that need a refinement plan should import from `komorebi_math.refinement` or a versioned namespace.

14.1 Signal and arithmetic polyphase coordinates

There are four coordinate records, three one-dimensional and one lattice:

Record	Payload	Reconstruction target
`PolyphaseCoordinates`	NumPy channels, order, axis, original shape	NumPy array
`ResidueCoordinates`	NumPy channels, modulus, length	one-dimensional NumPy array
`ArithmeticPolyphase`	exact/general-number tuple channels, modulus, length	Python tuple
`LatticePolyphaseCoordinates`	address-to-array map, strides, original shape	NumPy array

The one-dimensional refinement union is exactly

```
PolyphaseCoordinateRecord = (
    PolyphaseCoordinates | ResidueCoordinates | ArithmeticPolyphase
)
```

It does not include `LatticePolyphaseCoordinates`. Lattice refinement has a separate class because its addresses are tuples.

Signal analysis and synthesis are

```
polyphase_analyze(
    signal: Any,
    order: int,
    *,
    axis: int = -1,
    copy: bool = True,
) -> PolyphaseCoordinates
```

```
polyphase_synthesize(
    coordinates: PolyphaseCoordinates,
) -> np.ndarray

PolyphaseSystem(order: int, axis: int = -1)
```

`PolyphaseSystem.analyze` and `.synthesize` bind the order and axis. A record exposes `reconstruct()`, `channel_lengths()`, `refine()`, `coarsen()`, `certificate(original, tolerance=...)`, and `dtype`. The analyzed axis can be any valid NumPy axis. The other dimensions remain attached to each channel.

The exact arithmetic path is

```
ArithmeticPolyphase(
    modulus: int,
    channels: tuple[tuple[Number, ...], ...],
    length: int,
)

arithmetic.polyphase_analyze(
    values: Iterable[Number],
    modulus: int,
) -> ArithmeticPolyphase

arithmetic.polyphase_synthesize(
    channels: Iterable[Iterable[Number]],
    length: int | None = None,
) -> tuple[Number, ...]
```

`ArithmeticPolyphase.reconstruct`, `.nested`, `.refine`, and `.coarsen` preserve the Python scalar values. Construct it through the analyzer when possible: the dataclass constructor does not have a post-initialization validator, whereas the analyzer enforces the public modulus/channel conventions.

For an M-channel record, signal and arithmetic channels obey the same law:

$$x_r[k] = x[r + Mk], \qquad 0 \le r < M.$$

With `copy=False`, signal refinement may retain NumPy slices where the layout permits. Coarsening necessarily allocates interleaved outputs in many cases. The flag is not a general promise of view semantics.

14.2 `PolyphaseRefinement` and `PolyphaseRefinementTower`

$$m_c \mid m_f, \qquad r_f = r_c + m_c q.$$

The canonical finite plan is

```
PolyphaseRefinement(
    coarse_order: int,
    fine_order: int,
)
```

Both orders must be positive non-Boolean integers and the coarse order must divide the fine order. Write $M = m_c$, $N = m_f$, and $K = N/M$. A fine address is split uniquely as

$$a = r + Ms, \qquad r = a \bmod M, \qquad s = \left\lfloor \frac{a}{M} \right\rfloor, \qquad 0 \le s < K.$$

At the level of channels,

$$\left(x_r^{(M)}\right)_s^{(K)} = x_{r+Ms}^{(N)}.$$

The implementation slices the parent channel `r` by `s::K`. It does not need to reconstruct and reanalyze the full signal. Coarsening performs the inverse interleaving. In projector notation the same law is

$$P_r^{(M)} = \sum_{s=0}^{K-1} P_{r+Ms}^{(N)}.$$

The address and execution surface is

Member	Result
`ratio`	$K = N/M$
`address(r, s)`	$r + Ms$
`split_address(a)`	`(r, s)`
`parent(a)`	coarse residue r
`child_digit(a)`	digit s
`children(r)`	all fine addresses over r
`parent_map`	parent of every fine residue
`refine(record, copy=True)`	fine coordinate record of the same concrete type
`coarsen(record, copy=True)`	coarse coordinate record
`certified_refine(...)`	fine record plus `KM-B15-T01` certificate
`certificate(...)`	refinement/roundtrip evidence
`compose(next_refinement)`	direct endpoint plan

Address-table example

```python
from komorebi_math.refinement import PolyphaseRefinement

plan = PolyphaseRefinement(2, 8)

print(plan.ratio)
print(plan.address(1, 3))
print(plan.split_address(7))
print(plan.children(1))
print(plan.parent_map)
```

Output:

```
4
7
(1, 3)
(1, 3, 5, 7)
(0, 1, 0, 1, 0, 1, 0, 1)
```

Exact residue refinement

```python
import numpy as np
from komorebi_math.refinement import (
    PolyphaseRefinement,
    residue_analyze,
)

values = np.arange(31)
```

```
coarse = residue_analyze(values, 4)
plan = PolyphaseRefinement(4, 12)
fine = plan.refine(coarse)
direct = residue_analyze(values, 12)

assert plan.coarsen(fine).reconstruct().tolist() == values.tolist()
assert all(
    np.array_equal(left, right)
    for left, right in zip(fine.channels, direct.channels, strict=True)
)
plan.certificate(coarse).require()
```

The coarse lengths are `(8, 8, 8, 7)`. The twelve fine channels are

```
(0,12,24), (1,13,25), (2,14,26), (3,15,27),
(4,16,28), (5,17,29), (6,18,30), (7,19),
(8,20), (9,21), (10,22), (11,23)
```

and the certificate is exact and valid.

Finite chains use

```
PolyphaseRefinementTower(
    orders: tuple[int, ...],
)
```

There must be at least two distinct orders and every order must divide the next. The public methods resolve a level, construct a direct refinement between two listed orders, refine or coarsen to a target order, enumerate descendants, evaluate a path address, and produce a coherence certificate.

For

$$M_0 \mid M_1 \mid \cdots \mid M_L,$$

the terminal address is

$$a_L = r + \sum_{j=0}^{L-1} M_j s_j, \qquad 0 \le s_j < M_{j+1}/M_j.$$

```
from komorebi_math.refinement import PolyphaseRefinementTower, residue_analyze

tower = PolyphaseRefinementTower((2, 6, 24))
coarse = residue_analyze(range(31), 2)

print(tower.descendants(1, 2, 24))
print(tower.path_address(1, (2, 3)))
tower.coherence_certificate(coarse).require()
```

The descendants are all odd addresses from 1 through 23, the path address is 23, and the exact sequential-versus-direct residual is zero. `KM-B15-T02` checks that coherence comparison. It does not itself check tower coarsening. Coarsening is exercised separately by the methods and tests.

14.3 Lattice polyphase refinement

For an array with one stride per dimension,

```
lattice_polyphase_analyze(
    signal: Any,
    strides: Iterable[int],
    *,
    copy: bool = True,
) -> LatticePolyphaseCoordinates

lattice_polyphase_synthesize(
    coordinates: LatticePolyphaseCoordinates,
) -> np.ndarray
```

creates one channel for each address tuple. A lattice record exposes `reconstruct`, `refine`, `coarsen`, `certificate`, and `dtype`.

The plan

```
LatticePolyphaseRefinement(
    coarse_strides: tuple[int, ...],
    fine_strides: tuple[int, ...],
)
```

requires equal positive dimension and componentwise divisibility. If $K_j = N_j/M_j$, then addresses split componentwise:

$$\mathbf{a} = \mathbf{r} + \mathbf{M} \odot \mathbf{s}, \qquad 0 \le s_j < K_j.$$

Each coarse address has $\prod_j K_j$ children. The coarse and fine channel counts are $\prod_j M_j$ and $\prod_j N_j$. Methods mirror the one-dimensional plan: `address`, `split_address`, `parent`, `children`, `refine`, `coarsen`, `certificate`, `compose`, `to_dict`, and `from_dict`.

For an array of shape `(6, 9)` and strides `(2, 3) -> (4, 9)`, the executed facts are

```
ratios               = (2, 3)
coarse channel count = 6
fine channel count   = 36
children of (1,2)    = ((1,2),(1,5),(1,8),(3,2),(3,5),(3,8))
roundtrip             = True
KM-B15-T03            = exact and valid
```

There is no lattice refinement tower class in this release. Compose compatible lattice plans explicitly when more than one level is needed.

14.4 Mixed-radix addresses

$$a = d_0 + r_0(d_1 + r_1(\cdots)).$$

Mixed-radix encoding generalizes the nested residue address. Digits are stored least-significant first:

$$a = \sum_{j=0}^{L-1} d_j \prod_{i<j} r_i, \qquad 0 \le d_j < r_j.$$

The signal helpers are

```
signals.mixed_radix_address(digits, radices) -> int
signals.split_mixed_radix_address(address, radices) -> tuple[int, ...]
```

and the arithmetic helpers are

```
arithmetic.mixed_radix_address(digits, radices) -> int
arithmetic.mixed_radix_digits(address, radices) -> tuple[int, ...]
```

The frozen namespace exports the latter pair as `arithmetic_mixed_radix_address` and `arithmetic_mixed_radix_digits`. The signal pair remains a domain/research API.

```
assert signals.mixed_radix_address((1, 2, 3), (2, 3, 5)) == 23
assert arithmetic.mixed_radix_address((2, 1, 3), (4, 3, 5)) == 42
```

Digit and radix counts must agree, every radix must be positive, and each digit must lie in its own half-open range. Decoding rejects an address outside the finite range represented by the declared radices. A scalar has no polyphase or lattice packet decomposition, so zero-dimensional shape shortcuts are not silently introduced.

14.5 Roundtrip and refinement certificates

The certificate family distinguishes basic signal reconstruction from refinement coherence:

Identifier	Checked equality	Evidence class
`PSC-V-T02`	signal analyze then reconstruct	numerical
`PSC-V-T04`	lattice analyze then reconstruct	numerical
`KM-B15-T01`	coarsen-after-refine and refined-vs-direct fine analysis	exact or numerical
`KM-B15-T02`	sequential tower refinement versus direct refinement	exact or numerical
`KM-B15-T03`	lattice coarsen-after-refine and refined-vs-direct analysis	exact or numerical

`KM-B15-T01` and `KM-B15-T03` record residual keys `coarsen_after_refine` and `direct_fine_analysis`. `KM-B15-T02` records `sequential_vs_direct_refinement` only. Arithmetic records and NumPy channels whose dtype is neither floating nor complex can produce `Exactness.EXACT` with zero residual and `tolerance=None`. Floating and complex paths use a norm-scaled tolerance and produce numerical evidence.

The basic `PolyphaseCoordinates.certificate` and `LatticePolyphaseCoordinates.certificate` remain numerical even for integer arrays. This is an API classification decision, not evidence that reindexing changed an integer value.

Only the `KM-B15-T01` implementation explicitly rejects negative, infinite, or NaN tolerance at this release. Callers should still pass a positive finite tolerance to every numerical certificate. The absence of validation in another entry point is not permission to use a nonsensical threshold.

Convenience functions construct the canonical plans:

```
refine_polyphase(coordinates, fine_order, *, copy=True)
coarsen_polyphase(coordinates, coarse_order, *, copy=True)
polyphase_refinement_certificate(coordinates, fine_order, *, tolerance=1e-12)
refine_lattice_polyphase(coordinates, fine_strides, *, copy=True)
coarsen_lattice_polyphase(coordinates, coarse_strides, *, copy=True)
```

The stable aliases `polyphase_refine`, `polyphase_coarsen`, `lattice_polyphase_refine`, and `lattice_polyphase_coarsen` are object-identical to those canonical functions.

Persistence and controlled failures

`PolyphaseRefinement` supports `to_dict`, `from_dict`, `to_json`, and `from_json`. Towers and lattice plans support the first two only. All three also have registered version-1 portable codecs in the digest-checked envelope described in Section 13.8.

Construction and execution reject nonpositive or Boolean orders, nondivisible endpoints, malformed addresses or child digits, wrong coordinate order, inconsistent child shapes, incompatible composition endpoints, duplicate or nondivisible tower orders, dimension/stride mismatch, and missing or extra schema fields. `copy=False` does not weaken any shape or address invariant.

The shipped `examples/block15_polyphase_refinement.py` executes all three plan families and prints

```
Block 15 polyphase refinement verified
```

The recorded polyphase-refinement tests contain 44 parameter-expanded refinement cases, 8 persistence cases, and 9 stable API/codec cases. The independent shipped audit records 39,070 comparisons with zero failures. Those checks establish finite addressing, roundtrip, coherence, and schema behavior. They do not claim an infinite inverse limit, regularity of a limit function, or wavelet convergence.

Chapter 14 summary. A polyphase record is a finite residue split with a concrete scalar and shape convention. A refinement plan factors a fine address into a coarse address and child digit, refines channels without an ambient reconstruction, and certifies equality with direct fine analysis. Towers and lattice plans extend that law finitely and componentwise.

Part IV. Coordinate calculus and analysis-synthesis systems

Reading route. Begin with the object you want to represent. Chapters 15 and 16 establish the common input, matrix-direction, and composition rules. The later chapters apply those rules to particular kinds of data.

Task	First destination	Useful next destination
Express a vector in a finite basis	Section 15.2	Section 15.3 for several named coordinate systems
Change basis or compose coordinate maps	Sections 16.1 to 16.3	Section 19.4 for transporting an operator
Change a polynomial's coefficient family	Section 17.5	Section 16.6 for differentiation and multiplication
Separate a sequence into initial values and recurrence defects	Section 18.3	Section 18.4 for state-space coordinates
Inspect a small spectral matrix or reorder indices	Sections 19.1 to 19.2	Chapter 32 for signal-analysis workflows
Represent a vector with redundant coefficients	Sections 20.1 to 20.2	Section 20.5 for maps and coefficient blocks

Most exact examples need only integer inputs and Python's `Fraction` type. The numerical spectral and frame examples use NumPy. Read the constructor's scalar contract before moving data between these layers. Also distinguish *degree* from *length*: a polynomial through degree four has five coefficient slots. The direction of a coordinate map tells you what its input tuple means, so retain source and target names beside any matrix saved or passed to another routine.

15. Coordinate objects

First route. Normalize exact input as described in Section 15.1, then follow the square-basis example in Section 15.2. If your vectors span only a subspace, use the rectangular example next. An atlas becomes useful when you have several named bases to connect. Affine and barycentric coordinates in Section 15.6 add an origin or simplex and can be read independently of atlas routing.

The coordinate package begins with a small exact algebra: dense, immutable tuples of rational numbers. On top of it, `FiniteBasis` represents a finite independent family, `CoordinateAtlas` routes exact changes of basis, and affine and barycentric objects add origins and simplex coordinates. These are finite computational objects, not symbolic manifold charts or sparse large-scale solvers.

The frozen basis surface is available from `komorebi_math.stable`, `komorebi_math.v1`, and `komorebi_math.api.v1`:

```
import komorebi_math.stable as km

standard = km.standard_basis(2, name="standard")
shear = km.basis_from_matrix("shear", ((1, 1), (0, 1)))
change = standard.connection_to(shear)
```

The complete object, exact-linear-algebra, atlas, budget, affine, and barycentric surface lives in `komorebi_math.coordinates`. `CoordinateSystem`, `CoordinateBudget`, the affine types, and several basis combinators are research APIs even though `FiniteBasis`, `CoordinateAtlas`, `standard_basis`, and `basis_from_matrix` are frozen.

15.1 Scalar, vector, matrix, and polynomial conventions

The exact aliases are

```
Scalar = int | Fraction
Vector = tuple[Fraction, ...]
Matrix = tuple[tuple[Fraction, ...], ...]
Polynomial = tuple[Fraction, ...]
```

A polynomial tuple uses ascending powers:

$$(a_0, a_1, \dots, a_n) \longleftrightarrow a_0 + a_1 x + \cdots + a_n x^n.$$

Polynomial helpers trim trailing zeros and use `(Fraction(0),)` as the canonical zero polynomial. Polynomial families are developed in Chapter 17, and polynomial operators in Section 16.6. Only the scalar and storage conventions are fixed here.

Normalize user inputs at the boundary:

```
from komorebi_math.coordinates import as_matrix, as_vector

v = as_vector((3, -2, 5))
A = as_matrix(((3, 1), (1, 2)))
```

`as_vector` and `as_matrix` reject Boolean coefficients and return immutable `Fraction` tuples. Integers and explicit `Fraction` values are the intended inputs. Although `Fraction(value)` accepts certain other Python values, a float such as `0.1` encodes its exact binary floating value, not the decimal rational $1/10$. Use `Fraction(1, 10)` when that rational is intended.

Low-level elimination functions are typed to an already normalized `Matrix`. Passing raw integer rows straight to such a helper can permit Python `/` to create floating intermediates. High-level constructors normalize automatically, but direct calls to `rref`, `determinant`, `inverse_matrix`, and related helpers should receive the output of `as_matrix`.

The dense exact primitives are

```
matrix_shape(matrix)
zero_matrix(rows, columns)
identity_matrix(size)
transpose(matrix)
matrix_add(left, right)
matrix_subtract(left, right)
matrix_multiply(left, right)
matrix_vector_multiply(matrix, vector)
matrix_power(matrix, exponent)
matrix_from_columns(columns)
```

The deep exact module also exposes `columns(matrix)` and `kronecker(left, right)`. Matrix shape, inner dimension, squareness, and nonnegative-exponent conditions are checked exactly. Some low-level operations admit mathematically meaningful empty matrices. Finite bases and coordinate transforms impose their own nonempty requirements. Avoid a blanket rule that every empty matrix is invalid.

15.2 `FiniteBasis`, standard bases, and basis matrices

```
FiniteBasis(
    name: str,
    vectors: tuple[Vector, ...],
    labels: tuple[str, ...] = (),
    description: str = "",
)
```

The `vectors` argument contains basis vectors. If $B = [b_0 \ \cdots \ b_{d-1}]$, then

$$x = Bc$$

is synthesis and exact analysis solves $Bc = x$. Vectors must be nonempty, share an ambient dimension, and be linearly independent. Missing labels become `b0`, `b1`, and so on. Properties report `dimension`, `ambient_dimension`, `matrix`, and whether the basis is `square`.

The principal methods are

```
synthesize(coordinates) -> Vector
analyze(vector) -> Vector
contains(vector) -> bool
connection_to(target, *, theorem="CC.B1", budget=DEFAULT_COORDINATE_BUDGET)
dual_rows() -> Matrix
gram_matrix() -> Matrix
orthogonal_projector() -> Matrix
certificate() -> Certificate
```

Rectangular bases represent exact subspaces. `analyze` rejects a vector outside the span, while `contains` turns scalar, shape, or span failure into `False`. For a square basis, the dual rows are B^{-1}. For any full-column-rank basis,

$$G = B^{\mathsf{T}}B, \qquad P = B(B^{\mathsf{T}}B)^{-1}B^{\mathsf{T}}.$$

The inner product here is ordinary rational Euclidean transpose, not a complex Hermitian or weighted product.

If square source and target bases have synthesis matrices B_s and B_t, then

$$C_{t \leftarrow s} = B_t^{-1} B_s.$$

For rectangular bases, `connection_to` additionally requires equal ambient dimension, equal basis dimension, and equal exact span. The result is the `CoordinateTransform` of Chapter 16.

Basis constructors

```
standard_basis(dimension: int, *, name: str = "standard")

basis_from_matrix(
    name: str,
    matrix: Iterable[Iterable[Scalar]],
    *,
    labels: Sequence[str] = (),
    description: str = "",
)

direct_sum_basis(left, right, *, name=None)
tensor_product_basis(left, right, *, name=None)
permutation_transform(permutation, *, source=..., target=...)
scaling_transform(scales, *, source=..., target=...)
```

`basis_from_matrix` receives a row-major matrix whose **columns** are the basis vectors. This differs from direct `FiniteBasis(...)`, which receives the vectors themselves.

When adapting input from another program, inspect one basis vector before construction. A list of vectors and a matrix with those vectors as columns have different outer layouts. For a square array, both layouts have the same shape, so a shape check alone will not reveal an accidental transpose.

Standard labels are zero-based `e0`, `e1`, and so forth. Direct-sum labels use `L:` and `R:`. Tensor order is left-outer lexicographic and labels use `⊗`. A permutation has semantics

```
target[i] = source[permutation[i]]
```

and a scaling transform is diagonal with all scale factors nonzero.

Executed square-basis contract

```
standard = standard_basis(2, name="standard")
shear = basis_from_matrix("shear", ((1, 1), (0, 1)))
change = standard.connection_to(shear)

assert change.matrix == ((1, -1), (0, 1))
assert change.apply((3, 5)) == (-2, 5)
change.certificate().require()
assert change.roundtrip_valid()
```

This behavior is frozen in the stable semantic fixture.

Rectangular plane example

```
from fractions import Fraction as Q
from komorebi_math.coordinates import basis_from_matrix

plane = basis_from_matrix(
    "plane",
    ((1, 0), (0, 1), (1, 1)),
    labels=("u", "v"),
```

```
)

print(plane.gram_matrix())
print(plane.orthogonal_projector())
print(plane.analyze((2, 3, 5)))
print(plane.contains((2, 3, 4)))
```

The exact results are

$$G = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}, \qquad P = \begin{pmatrix} 2/3 & -1/3 & 1/3 \\ -1/3 & 2/3 & 1/3 \\ 1/3 & 1/3 & 2/3 \end{pmatrix}.$$

Analysis returns `(2, 3)`, the second membership check is false, and `dual_rows()` raises because the ambient basis is not square. The projector satisfies $P^2 = P$ exactly.

15.3 `CoordinateAtlas`

An atlas is a finite directed graph whose vertices name coordinate systems and whose edges are exact transforms.

```
CoordinateSystem(
    name: str,
    dimension: int,
    kind: str = "exact",
    description: str = "",
)

CoordinateAtlas(name: str = "coordinate atlas")
```

System names must be nonempty, and dimensions are positive non-Boolean integers. `register_basis(basis)` adds a system with kind `finite_basis`. `register_system(system)` adds metadata directly.

```
atlas.register_transform(
    transform,
    *,
    include_inverse: bool = False,
    replace: bool = False,
)

atlas.direct_transform(source, target)
atlas.resolve(source, target) -> AtlasPath
atlas.compose_path(systems) -> AtlasPath
atlas.compare_paths(left, right) -> CommutingDiagramCheck
```

Transforms require registered endpoints with matching dimensions. Repeating identical metadata is idempotent. Conflicting metadata or an existing different edge is rejected unless replacement was requested. `include_inverse=True` requires a square nonsingular transform. One implementation caveat is that the forward edge is stored before inverse construction. If inversion fails, the forward edge can remain registered.

`resolve` uses deterministic breadth-first search: minimum edge count, with neighbors considered in lexical order. It returns

```
AtlasPath(
    systems: tuple[str, ...],
    transform: CoordinateTransform,
)
```

where `length` is the number of edges. Resolving a system to itself returns an identity path with theorem identifier `CC.A0`. `compose_path` follows only the edges explicitly named by the caller. It does not search for missing links.

Two paths with common named endpoints commute exactly when their composed matrices agree. `CommutingDiagramCheck` stores the left and right paths, Boolean `valid`, and the exact matrix `difference`. Its certificate records both routes rather than reducing the comparison to a bare Boolean.

```
from komorebi_math.coordinates import (
    CoordinateAtlas,
    CoordinateSystem,
    CoordinateTransform,
)

atlas = CoordinateAtlas("example")
for name in ("A", "B", "C"):
    atlas.register_system(CoordinateSystem(name, 2))

T_ab = CoordinateTransform("A", "B", ((1, 1), (0, 1)), "demo")
T_bc = CoordinateTransform("B", "C", ((1, 0), (1, 1)), "demo")
T_ac = T_ab.then(T_bc)

atlas.register_transform(T_ab)
atlas.register_transform(T_bc)
atlas.register_transform(T_ac)

check = atlas.compare_paths(("A", "B", "C"), ("A", "C"))
print(check.valid)
print(check.difference)
```

If the direct edge is perturbed in its lower-right entry, a representative diagnostic is

```
valid      = False
difference = ((0, 0), (0, 1))
```

The shipped `examples/general_coordinate_atlas.py` exercises deterministic route resolution and prints

```
path: ('standard', 'scaled')
scaled coordinates: (Fraction(7, 2), Fraction(-1, 1), Fraction(11, 5))
```

15.4 Coordinate budgets and resource guards

Chapter 6 introduced the failure semantics of

```
CoordinateBudget(
    maximum_matrix_dimension: int = 512,
    maximum_polynomial_degree: int = 512,
    maximum_horizon: int = 4096,
    maximum_exact_entries: int = 1_000_000,
)
```

Its methods are

```
require_matrix(rows, columns, *, operation)
require_degree(degree, *, operation)
require_horizon(horizon, *, operation)
```

Constructor limits are positive non-Boolean integers. Degree and horizon requests are nonnegative integers. A malformed request raises `ValidationError`, while a valid request beyond the declared limit raises `ResourceLimitError`.

The budget is explicit rather than ambient. `FiniteBasis.connection_to` checks the dimension-by-dimension output against its supplied budget, and several higher-level polynomial/formal constructors do likewise. Basis construction, analysis, RREF, inversion, projector construction, and the other low-level exact helpers do **not** automatically consult a global budget. A caller building large exact matrices should run the appropriate `require_*` method before invoking them.

This design keeps low-level functions deterministic and context-free, but it also means that importing or configuring a budget does not by itself constrain all exact work.

15.5 Exact finite linear algebra: solve, rank, nullspace, and RREF

The elimination surface is

```
rref(matrix: Matrix) -> tuple[Matrix, tuple[int, ...]]
matrix_rank(matrix: Matrix) -> int
determinant(matrix: Matrix) -> Fraction
inverse_matrix(matrix: Matrix) -> Matrix
solve_linear_system(
    matrix: Matrix,
    right_hand_side: Sequence[Scalar],
) -> ExactLinearSystem
nullspace(matrix: Matrix) -> tuple[Vector, ...]
```

`rref` chooses leftmost pivots. `nullspace` means the right nullspace, with basis vectors ordered by ascending free column. `solve_linear_system` returns

```
consistent
rank
solution
nullspace_basis
pivot_columns
residual
```

Free variables in the chosen particular solution are zero. A consistent system has exact zero residual. For an inconsistent system, `solution` is `None` and `residual` stores the original right-hand side. It is not a least-squares or minimum-norm residual.

Underdetermined worked example

```
from komorebi_math.coordinates import as_matrix, rref, solve_linear_system

A = as_matrix(((1, 1, 0), (0, 1, 1)))
result = solve_linear_system(A, (3, 5))

print(result.consistent)
print(result.rank)
print(result.solution)
print(result.nullspace_basis)
print(result.pivot_columns)
print(result.residual)
print(rref(A))
```

The exact record is

```
consistent      = True
rank            = 2
solution        = (-2, 5, 0)
nullspace_basis = ((1, -1, 1),)
pivot_columns   = (0, 1)
residual        = (0, 0)
```

The corresponding row-reduced matrix is

$$\operatorname{rref}(A) = \begin{pmatrix} 1 & 0 & -1 \\ 0 & 1 & 1 \end{pmatrix}.$$

Thus every solution is

$$x = (-2, 5, 0) + t(1, -1, 1).$$

For $A = ((1,), (1,))$ and $b = (1, 2)$, the result is inconsistent, has rank one, `solution=None`, and `residual=(1, 2)`.

There is no tolerance, pseudoinverse, sparse representation, matrix right-hand side, or backend choice in this exact layer. Those are separate numerical or specialized APIs.

15.6 Affine, local, and barycentric coordinates

An affine coordinate transform is

```
AffineCoordinateTransform(
    source: str,
    target: str,
    matrix: Matrix,
    offset: Vector,
    theorem: str = "CC.F1",
    description: str = "",
)
```

The transform acts by

$$y = Ax + b.$$

The linear matrix must be nonempty, square, and invertible at construction, and the offset must have matching dimension. The inverse is

$$(A, b)^{-1} = (A^{-1}, -A^{-1}b),$$

compatible affine maps compose by

$$(B, c) \circ (A, b) = (BA, Bb + c),$$

and the homogeneous representation is

$$H(A, b) = \begin{pmatrix} A & b \\ 0 & 1 \end{pmatrix}.$$

Methods apply, invert, compose, form the homogeneous matrix, and emit an exact certificate.

There is no separate generic `LocalCoordinateSystem` class. A square local chart is constructed by

```
affine_basis_transform(
    origin,
    basis_vectors,
    *,
    local_name="local affine coordinates",
    ambient_name="ambient Cartesian coordinates",
) -> AffineCoordinateTransform
```

A pure translation is constructed by

```
translation_transform(
    offset,
    *,
    source="original affine coordinates",
    target="translated affine coordinates",
) -> AffineCoordinateTransform
```

For origin `(3, -2)` and basis vectors `((1, 1), (1, -1))`, local coordinates `(5/2, -3/2)` map to Cartesian `(4, 2)`. The homogeneous matrix is

$$\begin{pmatrix} 1 & 1 & 3 \\ 1 & -1 & -2 \\ 0 & 0 & 1 \end{pmatrix},$$

and the inverse roundtrip is exact.

Full simplices use

```
BarycentricCoordinateSystem(
    name: str,
    vertices: tuple[Vector, ...],
)
```

A d-dimensional system requires exactly $d+1$ affinely independent vertices. The methods convert in both directions and certify the finite simplex:

$$x = \sum_i \lambda_i v_i, \qquad \sum_i \lambda_i = 1.$$

`to_cartesian(weights, require_affine=True)` checks the affine sum by default. It does not require $\lambda_i \geq 0$. Negative weights are valid affine coordinates, so this object does not test convex-simplex membership.

For vertices `((0, 0), (2, 0), (0, 3))`, weights `(1/6, 1/3, 1/2)` map to `(2/3, 3/2)` and roundtrip exactly. The weights `(2, -1, 0)` also satisfy the affine-sum rule and map to `(-2, 0)`, outside the triangle.

Evidence, persistence, and failure boundary

The principal exact theorem identifiers are

Identifier	Producer
`CC.B0`	`FiniteBasis.certificate()`
`CC.B1`	default basis connection
`CC.B2`	permutation transform
`CC.B3`	scaling transform
`CC.A0`	atlas identity path
`CC.A1`	composed atlas path
`CC.A2`	commuting-diagram check
`CC.F1`	direct affine transform
`CC.F2`	affine basis transform
`CC.F3`	barycentric system
`CC.F4`	translation transform

These labels identify implementation contracts. They are not links to a separate theorem registry. Exact solve, RREF, nullspace, and budget calls return data or raise. They do not emit certificates.

Portable version-1 codecs exist for budgets, systems, atlases, finite bases, affine transforms, and barycentric systems. Raw matrices and vectors, `ExactLinearSystem`, `AtlasPath`, and `CommutingDiagramCheck` have no standalone coordinate codec.

Controlled failures include ragged matrices, shape disagreement, singular inverse, inconsistent basis spans, a vector outside a rectangular span, duplicate incompatible atlas nodes or edges, a disconnected route, a singular affine matrix, degenerate simplex vertices, and barycentric weights that do not sum to one. Low-level exact operations remain dense and can become expensive. Apply an explicit budget before large work.

The focused source replay covers exact elimination, square and rectangular bases, projectors, atlas routing and commutation, affine and barycentric roundtrips, and portable reconstruction. The guide's perturbed noncommuting atlas path is a derived executable example. It is not separately shipped as an example file.

> **Chapter 15 summary.** Exact coordinate objects use immutable rational tuples. A finite basis synthesizes a vector, analyzes exact span membership, and constructs a change of basis. An atlas composes those changes along a deterministic finite graph. Budgets are explicit, and affine/barycentric objects add origins without claiming convex membership or manifold structure.

16. Exact and numerical coordinate transforms

First route. Choose the exact rational or numerical transform in Sections 16.1 to 16.2, then read the composition convention in Section 16.3 before combining maps. The remaining sections are task references: polynomial rules, normalization, recurrence reconstruction, state recovery, finite fields, and redundant coefficients. If the next operation needs an inverse, check the invertibility contract for your particular transform type.

A coordinate transform records direction, dimensions, coefficients, and provenance. The package has separate exact rational, NumPy numerical, affine, formal, and prime-field realizations. They use compatible composition conventions, but their certificates answer different questions. In particular, an exact transform certificate can validate a singular map as a well-formed linear transformation, while a numerical square-map certificate treats singularity as invalid for its inverse-residual contract.

Use `komorebi_math.coordinates` for the full chapter:

```
from komorebi_math import coordinates
```

`CoordinateTransform`, its true alias `ExactCoordinateTransform`, `NumericCoordinateTransform`, `FormalCoordinateChange`, and the numerical `AnalysisSynthesisSystem` are present on versioned compatibility surfaces as specified below. Several calculus, tensor, state-recovery, and modular helpers remain research APIs.

16.1 `CoordinateTransform` and `ExactCoordinateTransform`

$$[v]_{B_t} = B_t^{-1} B_s [v]_{B_s}.$$

The exact record is

```
CoordinateTransform(
    source: str,
    target: str,
    matrix: Matrix,
    theorem: str,
    description: str = "",
)
```

`ExactCoordinateTransform` is the same class object, not a subclass. It is exported from `komorebi_math.coordinates`, `stable`, `v1`, and `api.v1`, but not from the package root. The matrix is normalized to immutable rational tuples.

The convention is

$$c_t = T c_s, \qquad T \in \mathbb{Q}^{m \times n},$$

so rows count the target dimension and columns count the source dimension. A rectangular map can be applied and composed. Determinant and inverse are square-only.

```
source_dimension
target_dimension
square
determinant
apply(coordinates) -> Vector
inverse(*, theorem=None) -> CoordinateTransform
then(following, *, theorem=None) -> CoordinateTransform
```

```
roundtrip_valid() -> bool
certificate() -> Certificate
```

For synthesis bases B_s and B_t, the change-of-basis formula is the one displayed above. Inputs are intended to be integers or `Fraction` values. A float is converted to its exact binary rational, which is rarely the desired symbolic coefficient.

Exact composition example

```
from fractions import Fraction as Q
from komorebi_math.coordinates import CoordinateTransform

T = CoordinateTransform("A", "B", ((1, 2), (0, 1)), "T")
U = CoordinateTransform("B", "C", ((2, 0), (3, 1)), "U")
x = (Q(3), Q(5))

composed = T.then(U)

print(T.apply(x))
print(composed.matrix)
print(composed.apply(x))
print(composed.inverse().apply(composed.apply(x)))
print(composed.roundtrip_valid())
```

Output:

```
(Fraction(13, 1), Fraction(5, 1))
((Fraction(2, 1), Fraction(4, 1)),
 (Fraction(3, 1), Fraction(7, 1)))
(Fraction(26, 1), Fraction(44, 1))
(Fraction(3, 1), Fraction(5, 1))
True
```

A singular square exact transform is representable. Its certificate remains valid because the record is a valid linear map. Diagnostics report `invertible=False`, `roundtrip_valid()` returns false, and `inverse()` raises. Code that needs an isomorphism must check the diagnostic or roundtrip rather than assuming `certificate().require()` proved invertibility.

16.2 `NumericCoordinateTransform`

```
NumericCoordinateTransform(
    source: str,
    target: str,
    matrix: np.ndarray,
    theorem: str = "CC.NM1",
    description: str = "",
    tolerance: float = 1e-12,
)
```

The constructor copies a finite two-dimensional NumPy matrix, marks it read-only, and requires a positive finite tolerance. `apply`, `inverse`, `then`, `condition_number`, and `certificate` mirror the exact class. Composed maps use the larger of the two tolerances.

For a square nonsingular matrix, the certificate computes

$$r = \max(\|T^{-1}T - I\|_\infty, \|TT^{-1} - I\|_\infty)$$

and accepts the inverse contract when

$$r \le \tau \max(1, \kappa_2(T)).$$

Shape/state	Constructible?	Certificate	Inverse
rectangular finite	yes	valid structural numerical record	dimension failure
square nonsingular	yes	inverse residual and condition diagnostic	numerical inverse
square singular	yes	invalid, with `singular=True`	validation failure
empty, nonmatrix, nonfinite	no	not produced	not available

An ill-conditioned but nonsingular matrix is not rejected solely because of its condition number. Conditioning is recorded so the caller can choose an application threshold.

Do not compare two independent numeric-transform dataclasses with plain `==`: the NumPy array comparison can have an ambiguous truth value. Compare fields and matrices with `np.array_equal` or `np.allclose`.

Spectral constructors

```
discrete_fourier_transform(size, *, unitary=True, inverse=False, tolerance=1e-12)
discrete_cosine_transform(size, *, orthonormal=True, tolerance=1e-12)
walsh_hadamard_transform(size, *, orthonormal=True, tolerance=1e-12)
haar_wavelet_transform(size, *, orthonormal=True, tolerance=1e-12)
modal_coordinate_system(operator, *, state_name=..., modal_name=..., tolerance=1e-10)
```

The unitary DFT uses

$$F_{kn} = N^{-1/2} e^{-2\pi i k n / N},$$

and the orthonormal DCT-II uses

$$C_{kn} = \alpha_k \cos\left(\frac{\pi(n + 1/2)k}{N}\right), \quad \alpha_0 = N^{-1/2}, \quad \alpha_k = (2/N)^{1/2}.$$

Walsh-Hadamard and Haar sizes must be positive powers of two. Modal coordinates require a finite square operator with a complete numerical eigenvector basis. None of these constructors upgrades floating evidence to an exact identity.

16.3 Composition, inversion, and transport

If $T : A \to B$ and $U : B \to C$, then

```
T.then(U)
```

means $U \circ T$ and stores

$$UT,$$

not TU. Labels must agree exactly and the intermediate dimensions must match. Consequently,

$$(UT)^{-1} = T^{-1} U^{-1}.$$

The exact tensor-transport surface includes

```
contragredient_transform(transform)
transport_covector(covector, transform)
transport_operator(operator, transform)
transport_rectangular_operator(operator, domain_transform, codomain_transform)
transport_bilinear_form(form, transform)
tensor_coordinate_transform(transform, variance)
transport_tensor_components(components, transform, variance)
tensor_product_transform(left, right)
direct_sum_transform(left, right)
coordinate_pairing(covector, vector)
pairing_invariant(covector, vector, transform)
```

For $v' = Tv$, the standard laws are

$$\alpha' = T^{-\mathsf{T}}\alpha, \qquad A' = TAT^{-1}, \qquad G' = T^{-\mathsf{T}}GT^{-1}.$$

For a rectangular operator $A : X \to Y$ with separate coordinate changes,

$$A' = T_Y A T_X^{-1}.$$

Each contravariant tensor axis contributes T. Each covariant axis contributes $T^{-\mathsf{T}}$. Ordered Kronecker products act on lexicographically flattened components. The rank-zero variance list produces a 1×1 identity.

```python
from komorebi_math.coordinates import (
    CoordinateTransform,
    coordinate_pairing,
    pairing_invariant,
    transport_bilinear_form,
    transport_covector,
    transport_operator,
)

T = CoordinateTransform("old", "new", ((2, 1), (1, 1)), "T")
v = (3, -2)
alpha = (5, 7)

print(T.apply(v))
print(transport_covector(alpha, T))
print(transport_operator(((1, 2), (3, 4)), T))
print(transport_bilinear_form(((2, 1), (1, 3)), T))
print(coordinate_pairing(alpha, v), pairing_invariant(alpha, v, T))
```

The audited output ends with `1 True`. The transformed vector is `(4, 1)` and the transformed covector is `(-2, 9)`. Covector, operator, form, and positive-rank tensor transport require a square nonsingular exact transform. Even an all-contravariant positive-rank tensor currently computes the inverse during construction, so nonsingularity is an implementation requirement. Parallel NumPy transport helpers are not shipped.

16.4 Affine and formal coordinate changes

The affine class, local-basis helper, translations, and barycentric systems were introduced in Section 15.6. Their composition law remains

$$(B, c) \circ (A, b) = (BA, Bb + c),$$

and an affine map rejects a singular linear part at construction.

Formal changes are finite jets:

```
FormalCoordinateChange(
    name: str,
    coefficients: Vector,
    source: str = "source formal coordinate",
    target: str = "target formal coordinate",
)
```

With ascending coefficients,

$$\phi(z) = a_0 + a_1 z + \cdots + a_N z^N,$$

the constructor enforces

$$a_0 = 0, \qquad a_1 \neq 0, \qquad N \geq 1.$$

The object reports `degree` and `linear_coefficient` and provides

```
truncate(degree)
compose(inner, *, degree=None, name=None)
inverse(*, degree=None, name=None)
pullback_coefficients(coefficients, *, degree=None)
pushforward_coefficients(coefficients, *, degree=None)
substitution_transform(degree=None, *, budget=DEFAULT_COORDINATE_BUDGET)
certificate()
```

`outer.compose(inner)` means

$$\text{outer}(\text{inner}(z)) \pmod{z^{N+1}}.$$

Pullback and pushforward are

$$\phi^* f = f \circ \phi, \qquad \phi_* f = f \circ \phi^{-1}.$$

The substitution-transform columns are the coefficient vectors of $1, \phi, \phi^2, \ldots, \phi^N$. Composition labels must match. A truncation cannot increase stored degree, and requested inversion or substitution degree cannot exceed the available jet. The substitution matrix consults the coordinate budget.

```
compose_ordinary_series(outer, inner, degree)
identity_formal_coordinate(degree, *, name=..., coordinate=...)
formal_coordinate_from_iterable(coefficients, *, name=..., source=..., target=...)
```

are the corresponding helpers. An empty pushforward input returns `()`. A constant-only nonempty input currently requests an impossible degree-zero inverse and raises. Callers should handle constants directly.

The shipped affine/formal example prints

```
Cartesian point: (Fraction(4, 1), Fraction(2, 1))
barycentric point: (Fraction(2, 3), Fraction(3, 2))
formal pullback: (Fraction(3, 1), Fraction(4, 1), Fraction(3, 1), Fraction(-8, 1), Fraction(26, 1))
```

16.5 Portable coordinate records

Coordinate-specific file helpers use path-first argument order:

```
save_transform(path, transform, *, overwrite=False)
load_transform(path) -> CoordinateTransform

save_coordinate_record(path, record, *, overwrite=False)
load_coordinate_record(path) -> RecurrenceCoordinateRecord

save_coordinate_object(path, value, *, overwrite=False)
load_coordinate_object(path) -> Any
```

The generic stable portable helpers use value-first order:

```
dumps_portable(value, *, indent=2, max_depth=128)
loads_portable(text, *, max_depth=128)
save_portable(value, path, *, overwrite=False, indent=2, max_depth=128)
load_portable(path, *, max_depth=128)
portable_digest(value, *, max_depth=128)
```

Every JSON file uses the version-1 envelope

```
{
  "schema": "komorebi-math.portable",
  "schema_version": 1,
  "producer_version": "1.0.0b4",
  "payload_sha256": "...",
  "payload": {}
}
```

Core coordinate codec tags include

Object	Tag
exact transform	`coordinates.transform`
numeric transform	`coordinates.numeric_transform`
affine transform	`coordinates.affine_transform`
formal change	`coordinates.formal_change`
barycentric system	`coordinates.barycentric_system`
recurrence coordinate record	`coordinates.recurrence_record`

Fractions store numerator and denominator. NumPy arrays store dtype, shape, and flattened data. Tagged records and their data dictionaries use exact field sets. The loader rejects unknown tags and versions, duplicate keys, non-finite JSON, invalid array shapes, excess nesting depth, malformed scalar records, and payload-digest mismatch. It reconstructs only registered codecs. A file cannot name an arbitrary Python class for import.

Writes are atomic, parent directories are created, and existing files are refused unless `overwrite=True`. The SHA-256 detects accidental or untrusted payload alteration only when the digest itself has not been rewritten. It is not a signature or message-authentication code.

```
import json
from komorebi_math.coordinates import CoordinateTransform, load_transform, save_transform
from komorebi_math.export import dumps_portable, loads_portable

T = CoordinateTransform("A", "B", ((1, 2), (0, 1)), "CC.X", "shear")
save_transform("transform.km.json", T, overwrite=True)
assert load_transform("transform.km.json") == T
```

```
text = dumps_portable(T)
envelope = json.loads(text)
print(envelope["schema"])
print(envelope["payload"]["__komorebi_type__"])
print(loads_portable(text) == T)
```

Output:

```
komorebi-math.portable
coordinates.transform
True
```

16.6 Polynomial differential and multiplication operators in arbitrary coordinates

`PolynomialCoordinateOperator` compiles a polynomial rule column by column in declared source and target families:

```
PolynomialCoordinateOperator(
    name: str,
    source_family: str,
    target_family: str,
    source_degree: int,
    target_degree: int,
    matrix: Matrix,
    theorem: str = "CC.O1",
)
```

If $P_0, \dots, P_d$ is the source basis, $Q_0, \dots, Q_e$ the target basis, and R the rule, the matrix O is defined by

$$R(P_j) = \sum_{k=0}^{e} O_{kj} Q_k.$$

Equivalently,

$$O = M_Q^{-1} \begin{bmatrix} [R(P_0)]_{\mathrm{mon}} & \cdots & [R(P_d)]_{\mathrm{mon}} \end{bmatrix}.$$

The general compiler and standard constructors are

```
polynomial_operator(source, target, source_degree, target_degree, rule,
                    *, name, theorem="CC.O1", budget=...)
derivative_coordinate_operator(family, degree, *, budget=...)
integral_coordinate_operator(family, degree, *, budget=...)
multiplication_coordinate_operator(family, degree, multiplier, *, budget=...)
affine_substitution_coordinate_operator(family, degree, *, scale=1, shift=0, budget=...)
finite_difference_coordinate_operator(family, degree, *, step=1, budget=...)
transport_polynomial_operator(operator, source_family, target_family, *, budget=...)
operator_commutator(left, right) -> Matrix
```

Derivative maps degree $d \to d$, zero-constant integration maps $d \to d+1$, multiplication by a degree-r polynomial maps $d \to d+r$, and affine substitution and finite difference retain the declared degree. High-degree terms are never silently dropped: if a rule image exceeds `target_degree`, compilation raises.

The object exposes `square`, `apply`, `compose`, and `certificate`. As elsewhere, `operator.compose(following)` stores this matrix after the first:

$$O_{\text{composed}} = O_{\text{following}} O_{\text{self}}.$$

Transport uses domain and codomain connections:

$$O_{\text{target}} = C_{\text{codomain}} O_{\text{source}} C_{\text{domain}}^{-1}.$$

For the Legendre family,

```
from komorebi_math.coordinates import (
    derivative_coordinate_operator,
    multiplication_coordinate_operator,
)
from komorebi_math.coordinates.classical import legendre_polynomial_family

legendre = legendre_polynomial_family()
D = derivative_coordinate_operator(legendre, 3)
X = multiplication_coordinate_operator(legendre, 3, (0, 1))

print(D.apply((0, 0, 0, 1)))
print(X.apply((0, 0, 0, 1)))
```

returns

```
(Fraction(1, 1), Fraction(0, 1), Fraction(5, 1), Fraction(0, 1))
(Fraction(0, 1), Fraction(0, 1), Fraction(3, 7), Fraction(0, 1), Fraction(4, 7))
```

so $P_3' = P_0 + 5P_2$ and $xP_3 = \frac{3}{7}P_2 + \frac{4}{7}P_4$.

Failures include wrong matrix/vector dimensions, an empty name or multiplier, zero finite-difference step, non-composable family labels or degree bounds, a rule image beyond the target, and budget overflow. The certificate validates the exact finite matrix and labels. The original Python callable is not retained or replayed. The version-1 portable tag is `coordinates.polynomial_operator`.

16.7 Normalized coefficient coordinates

`NormalizedBasis` is an alias of `WeightedMonomialBasis`. Such a basis has

$$e_n(z) = \frac{z^n}{\gamma_n}, \qquad \gamma_0 = 1, \qquad \gamma_n \neq 0.$$

If one formal series is written in two normalizations,

$$F(z) = \sum_n a_n \frac{z^n}{\gamma_n} = \sum_n b_n \frac{z^n}{\eta_n},$$

then

$$b_n = \frac{\eta_n}{\gamma_n} a_n.$$

The exact conversion is therefore diagonal:

```
normalization_change_transform(source, target, degree, *, budget=...)
convert_normalized_coefficients(coefficients, source, target, *, budget=...)
apply_lowering_polynomial(coefficients, polynomial)
```

with $T_{nn} = \eta_n/\gamma_n$. Built-in factories exposed from `komorebi_math.coordinates` are

Factory	Weight γ_n
`ordinary_basis()`	1
`exponential_basis()`	$n!$
`fibonacci_factorial_basis()`	$\prod_{j=1}^{n} F_j$

Additional factorial-power, q-factorial, Pochhammer, table, and callback bases live in `komorebi_math.generating`.

```
source = exponential_basis()
target = fibonacci_factorial_basis()
T = normalization_change_transform(source, target, 4)

print(T.apply((1, 0, -1, 0, 5)))
```

The diagonal is `(1, 1, 1/2, 1/3, 1/4)` and the result is

```
(Fraction(1, 1), Fraction(0, 1), Fraction(-1, 2),
 Fraction(0, 1), Fraction(5, 4))
```

The canonical lowering operator satisfies $D_\gamma e_n = e_{n-1}$. For $q(L) = \sum_{j=0}^{r} q_j L^j$,

$$(q(L)a)_n = \sum_{j=0}^{r} q_j a_{n+j}$$

on the available finite interior. Thus

```
apply_lowering_polynomial((1, 2, 3, 4, 5), (-1, 0, 1))
```

returns `(2, 2, 2)`. If the input is no longer than the operator degree, the finite interior is empty and the result is `()`. Callback-defined bases support live finite computation but require a declarative descriptor for persistence. No analytic convergence claim is made.

16.8 Shift-polynomial Bézout identities and defect reconstruction

Shift polynomials store ascending coefficients:

```
ShiftPolynomial(coefficients, name="shift polynomial")
```

They expose `degree`, `apply(values)`, and `multiply(other, name=None)`. The extended Euclidean routine

```
polynomial_extended_gcd(left, right) -> tuple[gcd, u, v]
```

returns a monic polynomial g and multipliers satisfying

$$u(x)q(x) + v(x)r(x) = g(x).$$

When $g = 1$, substitution of the lowering operator gives

$$u(L)q(L)a + v(L)r(L)a = a$$

only on the common finite interior. Each operator composition consumes its degree from the available horizon. This is not a boundary-reconstruction theorem.

```
BezoutDefectIdentity(
    gcd,
    left_multiplier,
    right_multiplier,
    reconstructed,
    expected,
    valid,
)

bezout_defect_identity(values, left, right) -> BezoutDefectIdentity
```

For

$$q = x^2 - x - 1, \qquad r = x^2 - 1,$$

the implementation returns $g = 1$, $u = -x$, and $v = x - 1$. Executing

```
values = (2, 7, -3, 5, 11, 13, 17, 19)
result = bezout_defect_identity(
    values,
    (-1, -1, 1),
    (-1, 0, 1),
)
```

produces

```
reconstructed = (2, 7, -3, 5, 11)
expected      = (2, 7, -3, 5, 11)
valid         = True
```

Only five entries lie in the common interior. Noncoprime shift polynomials are rejected. The input pair must not both be zero: that uncovered edge currently leaks a `ZeroDivisionError` from the polynomial algorithm. A short prefix may yield an empty but formally valid interior. `CC.O2` is used here and also by the derivative-coordinate operator. Theorem identifiers in this area are local implementation labels rather than globally unique keys. Both shift polynomials and defect identities have version-1 portable codecs.

16.9 State recovery, observability, Krylov similarity, and intertwiner spaces

A scalar linear realization is

```
LinearRealization(
    transition: Matrix,
    initial: Vector,
    output: Vector,
    name: str = "linear realization",
)
```

and represents

$$a_n = h^{\mathsf{T}} A^n v.$$

It exposes `dimension`, `state(index)`, `term(index)`, and `prefix(count)`. State recovery uses the exact observability matrix

$$\mathcal{O}_H = \begin{pmatrix} h^{\mathsf{T}} \\ h^{\mathsf{T}}A \\ \vdots \\ h^{\mathsf{T}}A^{H-1} \end{pmatrix}$$

through

```
observability_matrix(transition, output, horizon, *, budget=...)

recover_initial_state(prefix, transition, output, *, budget=...)
    -> StateRecovery
```

`StateRecovery` records `exact`, `unique`, `rank`, `horizon`, `state`, `predicted`, and `residual`. It solves over the rationals. `unique` means full state-column rank. The certificate requires an available state and zero residual but does not require uniqueness. Require $H \geq 1$: a zero-horizon call is an uncovered implementation edge that can return a degenerate empty state marked exact and unique.

For the Fibonacci companion $A = ((0, 1), (1, 1))$ with output `(1, 0)`, the two-row observability matrix is the identity. Fitting eight signed secant-Euler values yields

```
exact     = False
unique    = True
rank      = 2
state     = (1, 0)
predicted = (1, 0, 1, 1, 2, 3, 5, 8)
residual[2] = -2
```

The exact Krylov and intertwiner functions are

```
cyclic_krylov_matrix(transition, vector)
intertwiner_space(source, target) -> IntertwinerSpace
similarity_from_cyclic_vectors(source, target, source_vector, target_vector)
realization_defect(realization, target_coefficients)
```

For a cyclic vector,

$$K_A(v) = [v, Av, \dots, A^{d-1}v].$$

Given cyclic v, w, the similarity is

$$S = K_B(w)K_A(v)^{-1}, \qquad BS = SA,$$

and the identity is checked before return. `intertwiner_space` solves the exact nullspace equations $BS - SA = 0$ and returns source dimension, target dimension, and a matrix basis. Its `contains_invertible_basis_element()` checks only the displayed basis matrices, not arbitrary linear combinations. A span can contain an invertible matrix even when every chosen basis element is singular.

For $S = ((1, 2), (3, 5))$ and $B = SAS^{-1} = ((-1, 1), (-1, 2))$, the shipped routines recover that exact S, report a two-dimensional intertwiner space, and validate the intertwining certificate.

This route is a cyclic-vector similarity method, not a general canonical-form solver. Scalar recovery handles one output covector. The separate `StateSpaceModel`, controllability, block observability, and state-coordinate transform APIs belong to Chapter 18. Those state-space laws are discrete-time:

$$x_{k+1} = Ax_k + Bu_k, \qquad y_k = Cx_k + Du_k.$$

16.10 Exact finite-field coordinate changes and denominator obstructions

Prime-field coordinates use

```
FiniteFieldCoordinateTransform(
    source: str,
    target: str,
    matrix: ModularMatrix,
    prime: int,
    theorem: str = "CC.M1",
    description: str = "",
)
```

with `source_dimension`, `target_dimension`, `square`, `apply`, `inverse`, `then`, and `certificate`. The modulus must be a genuine prime. Matrix and vector entries should be actual integers. Runtime `int(...)` coercion is not a strong validation of noninteger inputs.

Rational reduction is

```
reduce_exact_transform_mod_prime(
    transform: CoordinateTransform,
    prime: int,
) -> FiniteFieldCoordinateTransform
```

For a rational coefficient a/b, reduction sends

$$\frac{a}{b} \longmapsto (a \bmod p)(b \bmod p)^{-1}$$

only when $p \nmid b$. Composition and Gauss-Jordan inversion are exact in $\mathbb{F}_p$.

For

```
R = CoordinateTransform(
    "q",
    "r",
    ((Fraction(1, 2), Fraction(1, 3)),
     (Fraction(2, 3), Fraction(3, 4))),
    "demo",
)
```

reduction modulo five gives

```
matrix          = ((3, 2), (4, 2))
apply((7, -2))  = (2, 4)
certificate     = valid
```

Modulo two, the denominator obstruction raises because 2 is not invertible. A different loss can occur after all denominators reduce: a rational isomorphism whose determinant numerator vanishes modulo p becomes a singular prime-field map. Reduction then succeeds, but the square-map certificate is invalid and inversion fails.

Rectangular maps remain valid maps but are not isomorphisms. Composition requires equal primes, matching labels, and compatible dimensions. This module implements $\mathbb{F}_p$, not extension fields, prime-power rings, or composite-modulus arithmetic. Its portable tag is `coordinates.finite_field_transform`, version 1.

16.11 `komorebi_math.coordinates.analysis_synthesis` canonical implementation page

The stable numerical analysis/synthesis system is

```
AnalysisSynthesisSystem(
    analysis: np.ndarray,
    synthesis: np.ndarray,
    source: str = "source",
    coordinates: str = "coordinates",
    tolerance: float = 1e-12,
    theorem: str = "KM-AS-01",
)
```

It is exported identically from `komorebi_math.coordinates`, `stable`, `v1`, and `api.v1`. Stored arrays are copied and read-only. With

$$A \in \mathbb{F}^{m \times n}, \qquad S \in \mathbb{F}^{n \times m},$$

the defining finite reconstruction law is

$$SA = I_n.$$

Factories are

```
AnalysisSynthesisSystem.from_invertible(
    matrix,
    *,
    source="source",
    coordinates="coordinates",
    tolerance=1e-12,
    theorem="KM-AS-01",
)

AnalysisSynthesisSystem.canonical_dual(
    analysis,
    *,
    source="source",
    coordinates="coordinates",
    tolerance=1e-12,
    theorem="KM-AS-01",
)
```

`canonical_dual` requires full column rank at the declared relative SVD cutoff and computes the Moore-Penrose left inverse. Properties report source and coordinate dimensions, redundancy, and squareness. Methods include

```
frame_operator()
analyze(value)
synthesize(value)
reconstruct(value)
component(indices)
components(groups=None)
operator_blocks(operator, *, groups=None)
certificate()
reconstruction_certificate(value)
```

`projector` and `projectors` remain compatibility aliases for `component` and `components`. The canonical terminology matters: in a redundant system, a singleton component

$$C_I = SE_IA$$

need not be idempotent. For a complete coordinate partition,

$$\sum_I C_I = SA = I.$$

For a source-space operator T, the block identity is

$$T = \sum_{\beta,\alpha} C_\beta T C_\alpha.$$

`AnalysisSynthesisBlocks` stores the system, groups, blocks, and operator and can reconstruct and certify that numerical sum. Construct it through `system.operator_blocks(...)`: direct record construction checks shapes and the summed result, but does not independently prove every supplied block equals $C_\beta T C_\alpha$.

Redundant canonical-dual example

```
import numpy as np
from komorebi_math.coordinates import AnalysisSynthesisSystem

A = np.array([
    [1.0, 0.0],
    [0.0, 1.0],
    [1.0, 1.0],
    [1.0, -1.0],
])
system = AnalysisSynthesisSystem.canonical_dual(A)
x = np.array([3.0, -2.0])

system.certificate().require()
print(system.analyze(x))
print(system.reconstruct(x))
```

The canonical synthesis matrix is

$$S = \begin{pmatrix} 1/3 & 0 & 1/3 & 1/3 \\ 0 & 1/3 & 1/3 & -1/3 \end{pmatrix},$$

and the numerical output is

```
analyze(x)     = [3.0, -2.0, 1.0, 5.0]
reconstruct(x) = [3.0000000000000004, -2.0000000000000004]
```

The four singleton components sum to identity, but each has idempotence residual about `0.22222222`. Calling them projectors would therefore overstate the algebraic property.

The system certificate uses `KM-AS-01`, reports numerical exactness, dimensions, rank, square/redundant state, and the $SA - I$ residual. Operator blocks use `KM-AS-02`. Direct construction does not automatically require $SA = I$, so the documented pattern calls `certificate().require(). from_invertible` also leaves that explicit check to the caller. The portable decoder is stricter: it reconstructs the system and immediately requires the certificate.

Groups must be nonempty, disjoint, and partition every coordinate. Indices must be unique and in range. Source operators must be square. Analysis and synthesis must be finite nonempty matrices with compatible shapes. Names must be nonempty. And tolerance must be positive and finite. `canonical_dual` rejects rank deficiency at the declared cutoff.

This class is NumPy numerical even when input arrays happen to contain integers, and it is not a CUDA/device runtime. The separate `ExactAnalysisSynthesisSystem` in `coordinates.frame_algebra` is a rational exact alternative and is not the stable canonical page documented here. The version-1 portable tag is `coordinates.analysis_synthesis`. Blocks have no independent codec.

The shipped `examples/block16_analysis_synthesis.py` validates a redundant system and operator blocks and reconstructs `[3, -2]`. Across the complete Chapter 16 surface, focused source tests for formal, affine, tensor, modular, calculus, normalization, shift-polynomial, realization, persistence, and analysis/synthesis paths passed in the recorded baseline. The coordinate verifier passed 24/24 checks. The coordinate-calculus verifier passed 168/168 in the baseline profile and 170/170 in the executed SymPy-enabled exact profile. Historical large randomized audit records remain useful provenance but are not relabeled as freshly generated `1.0.0b4` evidence.

> **Chapter 16 summary.** Exact and numerical transform records share a direction and composition convention but have different validity semantics. The same finite language transports tensors, compiles polynomial rules, changes normalization, reconstructs interior shift identities, recovers rational states, detects modular denominator obstructions, and builds redundant numerical analysis/synthesis systems. Each claim stops at its executed finite domain and evidence class.

17. Polynomial and classical coordinate families

First route. Review ascending coefficient order in Section 17.0. For a built-in graded family, choose its catalog name in Section 17.1 and follow the Legendre-to-Chebyshev example in Section 17.5.4. For fixed-degree Bernstein, Newton, or Lagrange coordinates, use the finite-basis route in Section 17.2. Check a small reconstructed polynomial before increasing the degree.

Komorebi Math represents a finite exact polynomial by its coefficients in ascending power order. Thus

$$p(x) = a_0 + a_1 x + \cdots + a_d x^d$$

is stored as

```
(a_0, a_1, ..., a_d)
```

with every entry converted to `fractions.Fraction`. The polynomial layer does not introduce a symbolic variable object. It works with immutable finite coefficient tuples and exact rational matrix operations. This keeps the coordinate convention explicit: a polynomial is the mathematical object, and a family coefficient vector is one representation of that object.

Three related constructions must be kept separate:

1. A `PolynomialFamily`, whose member of index n has exact degree n.
2. A `FiniteBasis`, such as a fixed-degree Bernstein or interpolation basis.
3. A `CoordinateTransform`, which changes coefficient vectors without changing the represented polynomial.

The distinction is operationally important. Graded families can be passed to `connection_transform`. Fixed-degree bases instead use `FiniteBasis.connection_to`. Bell and Touchard polynomial rows are generated in `komorebi_math.generating` and require an explicit finite adapter before they participate in the polynomial-family coordinate calculus.

17.0 Exact polynomial representation

17.0.1 Polynomial storage convention

The complete exact polynomial type is

```
Polynomial = tuple[Fraction, ...]
```

These primitive operations are public from `komorebi_math.coordinates`:

```
polynomial_add(
    left: Sequence[Scalar],
    right: Sequence[Scalar],
) -> Polynomial

polynomial_scale(
    polynomial: Sequence[Scalar],
    scalar: Scalar,
) -> Polynomial

polynomial_shift(
    polynomial: Sequence[Scalar],
```

```
    steps: int = 1,
) -> Polynomial

polynomial_multiply(
    left: Sequence[Scalar],
    right: Sequence[Scalar],
) -> Polynomial

polynomial_evaluate(
    polynomial: Sequence[Scalar],
    value: Scalar,
) -> Fraction

polynomial_derivative(
    polynomial: Sequence[Scalar],
) -> Polynomial

polynomial_subtract(
    left: Sequence[Scalar],
    right: Sequence[Scalar],
) -> Polynomial

constant_polynomial(
    value: Scalar,
) -> Polynomial

affine_polynomial(
    scale: Scalar,
    shift: Scalar,
) -> Polynomial

polynomial_compose_affine(
    polynomial: Sequence[Scalar],
    *,
    scale: Scalar = 1,
    shift: Scalar = 0,
) -> Polynomial
```

`polynomial_shift(p, r)` means multiplication by x^r, not translation of the argument. Translation and scaling of the argument are performed by `polynomial_compose_affine`:

$$p(x) \longmapsto p(\alpha x + \beta).$$

17.0.2 Exact arithmetic example

```
from fractions import Fraction

from komorebi_math.coordinates import (
    polynomial_add,
    polynomial_derivative,
    polynomial_evaluate,
    polynomial_multiply,
)

p = (1, -2, 1)                 # (x - 1)^2
q = (Fraction(1, 2), 3)        # 1/2 + 3x

print(polynomial_add(p, q))
print(polynomial_multiply(p, q))
```

```
print(polynomial_derivative(p))
print(polynomial_evaluate(p, 3))
```

Output:

```
(Fraction(3, 2), Fraction(1, 1), Fraction(1, 1))
(Fraction(1, 2), Fraction(2, 1), Fraction(-11, 2), Fraction(3, 1))
(Fraction(-2, 1), Fraction(2, 1))
4
```

The coefficient order explains the multiplication result:

$$(x-1)^2\left(\frac{1}{2}+3x\right)=\frac{1}{2}+2x-\frac{11}{2}x^2+3x^3.$$

Exact-input practice. Use integers and `Fraction` objects when the intended coefficients are rational. The type contract is `int | Fraction`. Boolean coefficients are rejected. Python does not enforce the scalar type annotation before conversion, so a supplied binary floating-point value can still reach `Fraction` and preserve its exact binary value rather than the short decimal a reader may have intended.

17.1 Family protocol and `get_polynomial_family`

17.1.1 The graded-family invariant

A polynomial family $P=(P_n)_{n\geq 0}$ is admissible when

$$\deg P_n=n, \qquad [x^n]P_n\neq 0.$$

The public record is

```
PolynomialFamily(
    name: str,
    rule: Callable[[int], Polynomial],
) -> None
```

and its public methods are

```
PolynomialFamily.polynomial(
    self,
    degree: int,
) -> Polynomial

PolynomialFamily.coefficient_matrix(
    self,
    degree: int,
) -> Matrix
```

The callback is evaluated lazily. Construction of `PolynomialFamily` alone does not execute or certify the rule. `polynomial(n)` trims trailing zeros and then checks that the result has length $n+1$ with a nonzero final coefficient. Consequently, an invalid callback fails at the first requested degree for which it violates the graded-family invariant.

For a degree bound d, the coefficient matrix is

$$M_P^{(d)} = \begin{bmatrix} | & | & & | \\ [P_0]_{\mathrm{mon}} & [P_1]_{\mathrm{mon}} & \cdots & [P_d]_{\mathrm{mon}} \\ | & | & & | \end{bmatrix}.$$

Rows index monomial power and columns index family degree. Since the column for P_n has no coefficient above degree n, $M_P^{(d)}$ is upper triangular. Its diagonal consists of the nonzero leading coefficients of the family members, so it is invertible over the rationals.

17.1.2 Catalog interface

The catalog record and lookup functions have these exact signatures:

```
PolynomialFamilyCatalog(
    families: tuple[PolynomialFamily, ...],
) -> None

PolynomialFamilyCatalog.names(
    self,
) -> tuple[str, ...]

PolynomialFamilyCatalog.get(
    self,
    name: str,
) -> PolynomialFamily

classical_polynomial_catalog(
) -> PolynomialFamilyCatalog

get_polynomial_family(
    name: str,
) -> PolynomialFamily
```

The built-in catalog contains these 15 graded families.

Short lookup	Canonical family name	Degree-n member
`monomial`	`monomials`	x^n
`euler`	`Euler polynomials`	$E_n(x)$
`fibonacci`	`Fibonacci polynomials`	$F_{n+1}(x)$
`falling`	`falling factorials`	$x^{\underline{n}}$
`rising`	`rising factorials`	$x^{\overline{n}}$
`binomial`	`binomial polynomials`	$\binom{x}{n}$
`bernoulli`	`Bernoulli polynomials`	$B_n(x)$
`lucas`	`Lucas polynomials`	$L_n(x)$
`pell`	`Pell polynomials`	$P_{n+1}(x)$
`chebyshev-t`	`Chebyshev T polynomials`	$T_n(x)$
`chebyshev-u`	`Chebyshev U polynomials`	$U_n(x)$
`legendre`	`Legendre polynomials`	$P_n(x)$
`hermite-probabilists`	`probabilists Hermite polynomials`	$He_n(x)$
`hermite-physicists`	`physicists Hermite polynomials`	$H_n(x)$
`laguerre`	`Laguerre polynomials`	$L_n(x)$

Lookup is case-insensitive and discards nonalphanumeric characters. An exact canonical name also works. For example, each call below returns the same family:

```
from komorebi_math.coordinates import get_polynomial_family

a = get_polynomial_family("chebyshev-t")
b = get_polynomial_family("Chebyshev T")
c = get_polynomial_family("Chebyshev T polynomials")

assert a.name == b.name == c.name == "Chebyshev T polynomials"
```

The catalog does not include aliases for Bernstein, Newton, Lagrange, Bell, or Touchard objects. Their distinct ownership and construction rules are described in Sections 17.2 and 17.4.

17.1.3 Stable and complete import surfaces

The stable family-connection workflow is available from `komorebi_math.stable`:

```
from komorebi_math.stable import (
    connection_transform,
    get_polynomial_family,
)
```

The complete family-construction surface is in `komorebi_math.coordinates`:

```
from komorebi_math.coordinates import (
    PolynomialFamily,
    classical_polynomial_catalog,
    get_polynomial_family,
)
```

`PolynomialFamily` itself is not re-exported by `komorebi_math.stable`. `get_polynomial_family`, `connection_transform`, and their returned `CoordinateTransform` workflow belong to the frozen 1.x stable surface. The additional constructors documented from `komorebi_math.coordinates` are public research-tier APIs: they are covered by the release's public API manifest, but their presence in a public namespace does not give them the same 1.x compatibility promise as `komorebi_math.stable`.

> **Namespace boundary.** The root name `komorebi_math.appell_polynomial` belongs to the analytic Appell subsystem and has a different signature. Use `komorebi_math.coordinates.appell_polynomial` or a namespace-qualified coordinate import for the finite classical-family constructor.

17.1.4 Custom family example

The binomial theorem gives a graded family

$$S_n(x) = (x+1)^n = \sum_{k=0}^{n} \binom{n}{k} x^k.$$

It can be declared directly:

```
from fractions import Fraction
from math import comb

from komorebi_math.coordinates import PolynomialFamily

shifted_monomials = PolynomialFamily(
    "powers of x+1",
    lambda degree: tuple(
```

```
        Fraction(comb(degree, power))
        for power in range(degree + 1)
    ),
)

assert shifted_monomials.polynomial(3) == (1, 3, 3, 1)
assert shifted_monomials.coefficient_matrix(3) == (
    (1, 1, 1, 1),
    (0, 1, 2, 3),
    (0, 0, 1, 3),
    (0, 0, 0, 1),
)
```

The callback is ordinary executable Python. Exactness requires it to be deterministic and to return exact coefficients. The runtime validates the returned polynomial's grade, but it does not attempt to prove purity or inspect the callback's source.

17.1.5 Family construction failures

Condition	Exception	Diagnostic
Negative, Boolean, or noninteger degree	`ValidationError`	`degree must be a nonnegative integer`
Callback result has degree other than n	`ValidationError`	`family polynomials must have exact graded degree`
Callback returns a zero leading coefficient	`ValidationError`	same graded-degree diagnostic
Unknown catalog lookup	`ValidationError`	`unknown polynomial family ...`

For example:

```
from komorebi_math.coordinates import PolynomialFamily

bad = PolynomialFamily("not graded", lambda degree: (1,))

try:
    bad.polynomial(2)
except Exception as error:
    print(type(error).__name__, str(error))
```

Output:

```
ValidationError family polynomials must have exact graded degree
```

17.2 Monomial, falling/rising factorial, Bernstein, Lagrange

17.2.1 Monomial and factorial families

The elementary graded constructors are

```
monomial_polynomial_family(
) -> PolynomialFamily

falling_factorial_polynomial(
    degree: int,
) -> Polynomial

rising_factorial_polynomial(
    degree: int,
) -> Polynomial

binomial_polynomial(
    degree: int,
) -> Polynomial

falling_factorial_polynomial_family(
) -> PolynomialFamily

rising_factorial_polynomial_family(
) -> PolynomialFamily

binomial_polynomial_family(
) -> PolynomialFamily
```

Their defining expressions are

$$m_n(x) = x^n,$$

$$x^{\underline{n}} = \prod_{j=0}^{n-1}(x-j),$$

$$x^{\overline{n}} = \prod_{j=0}^{n-1}(x+j),$$

and

$$\binom{x}{n} = \frac{x^{\underline{n}}}{n!}.$$

The degree-four coefficient tuples make the normalization difference visible.

Family	Degree-four tuple
Monomial	$(0, 0, 0, 0, 1)$
Falling factorial	$(0, -6, 11, -6, 1)$
Rising factorial	$(0, 6, 11, 6, 1)$
Binomial polynomial	$(0, -1/4, 11/24, -1/4, 1/24)$

The falling and rising factorial families are monic. The binomial family has leading coefficient $1/n!$. These leading coefficients determine the diagonal and determinant of every connection involving the family.

17.2.2 Fixed-degree Bernstein coordinates

The Bernstein constructor is

```
bernstein_polynomial(
    degree: int,
    index: int,
) -> Polynomial

bernstein_basis(
    degree: int,
) -> FiniteBasis
```

For fixed degree n,

$$B_{k,n}(x) = \binom{n}{k} x^k (1-x)^{n-k}, \qquad 0 \le k \le n.$$

Every $B_{k,n}$ belongs to the same $(n+1)$-dimensional polynomial space. This is not a graded family indexed by degree, because the family index k does not equal the polynomial degree. `bernstein_basis(n)` therefore returns a `FiniteBasis` whose vectors are coefficient columns in the monomial ambient space.

For $n = 3$:

```
B(3,0) = (1, -3, 3, -1)
B(3,1) = (0, 3, -6, 3)
B(3,2) = (0, 0, 3, -3)
B(3,3) = (0, 0, 0, 1)
```

They satisfy the partition-of-unity identity

$$\sum_{k=0}^{n} B_{k,n}(x) = 1.$$

The implementation-faithful path to monomial coefficients is a finite-basis connection:

```
from komorebi_math.coordinates import bernstein_basis, standard_basis

bernstein = bernstein_basis(3)
monomials = standard_basis(4, name="monomial coefficients")
to_monomials = bernstein.connection_to(monomials)

assert to_monomials.apply((1, 1, 1, 1)) == (1, 0, 0, 0)
assert to_monomials.inverse().apply((1, 0, 0, 0)) == (1, 1, 1, 1)
```

`connection_transform` is not used here because its arguments must both be `PolynomialFamily` objects.

17.2.3 Newton interpolation coordinates

The Newton basis at distinct exact nodes $\xi_0, \dots, \xi_{m-1}$ is

$$N_0(x) = 1, \qquad N_k(x) = \prod_{j=0}^{k-1} (x - \xi_j).$$

Its constructor is

```
newton_interpolation_basis(
    nodes: Sequence[Scalar],
) -> FiniteBasis
```

17.2.4 Lagrange interpolation coordinates

The cardinal basis at the same nodes is

$$L_i(x) = \prod_{\substack{0 \le j < m \\ j \ne i}} \frac{x - \xi_j}{\xi_i - \xi_j},$$

with the exact invariant

$$L_i(\xi_j) = \delta_{ij}.$$

Its constructor is

```
lagrange_interpolation_basis(
    nodes: Sequence[Scalar],
) -> FiniteBasis
```

For the nodes $(-1, 0, 2)$, the two bases are

```
from fractions import Fraction

from komorebi_math.coordinates import (
    lagrange_interpolation_basis,
    newton_interpolation_basis,
)

nodes = (Fraction(-1), Fraction(0), Fraction(2))

print(newton_interpolation_basis(nodes).vectors)
print(lagrange_interpolation_basis(nodes).vectors)
```

Mathematically simplified values:

```
((1, 0, 0), (1, 1, 0), (0, 1, 1))
((0, -2/3, 1/3), (1, 1/2, -1/2), (0, 1/6, 1/6))
```

The coefficient tuples represent

$$N_0 = 1, \qquad N_1 = x + 1, \qquad N_2 = x(x + 1),$$

and

$$L_0 = \frac{x^2 - 2x}{3}, \qquad L_1 = 1 + \frac{x}{2} - \frac{x^2}{2}, \qquad L_2 = \frac{x + x^2}{6}.$$

17.2.5 Evaluation and interpolation transforms

For a graded family and a nonempty node sequence, the public constructors are

```
evaluation_transform(
    family: PolynomialFamily,
    nodes: Sequence[Scalar],
    *,
    budget: CoordinateBudget = DEFAULT_COORDINATE_BUDGET,
) -> CoordinateTransform

interpolation_transform(
    family: PolynomialFamily,
```

```
    nodes: Sequence[Scalar],
    *,
    budget: CoordinateBudget = DEFAULT_COORDINATE_BUDGET,
) -> CoordinateTransform
```

If there are m nodes, the declared polynomial degree is $d = m - 1$. The evaluation matrix is

$$V_{ij} = P_j(\xi_i), \qquad 0 \le i, j \le d.$$

The forward transform maps family coefficients to node values. The interpolation transform is its exact inverse.

```
from fractions import Fraction

from komorebi_math.coordinates import (
    evaluation_transform,
    get_polynomial_family,
)

nodes = (Fraction(-1), Fraction(0), Fraction(2))
legendre = get_polynomial_family("legendre")
evaluate = evaluation_transform(legendre, nodes)

values = evaluate.apply((1, 2, 3))
recovered = evaluate.inverse().apply(values)

print(evaluate.matrix)
print(values)
print(recovered)
```

Mathematically simplified values:

```
((1, -1, 1),
 (1, 0, -1/2),
 (1, 2, 11/2))
(2, -1/2, 43/2)
(1, 2, 3)
```

Distinct nodes are required for Newton and Lagrange basis constructors. `evaluation_transform` detects a nondetermining node set by its zero determinant and raises `ValidationError`.

17.2.6 Fixed-basis failures

Condition	Exception
Bernstein index $k > n$	`DimensionError`
Empty interpolation node sequence	`DimensionError`
Duplicate Newton or Lagrange nodes	`ValidationError`
Singular evaluation matrix	`ValidationError`
Ambient or basis dimension mismatch	`DimensionError`
Dependent vectors in a constructed `FiniteBasis`	`ValidationError`

17.3 Bernoulli, Euler, Hermite, Laguerre, Legendre, Chebyshev

The classical constructors in this section provide exact finite coefficient formulas. Their names do not, by themselves, certify orthogonality integrals, weight functions, normalization constants, zero locations, approximation bounds, or quadrature rules.

17.3.1 Bernoulli polynomials

The public constructors are

```
bernoulli_number(
    index: int,
) -> Fraction

bernoulli_polynomial(
    degree: int,
) -> Polynomial

bernoulli_polynomial_family(
) -> PolynomialFamily
```

The convention is

$$B_0 = 1, \qquad B_1 = -\frac{1}{2}.$$

The implementation obtains successive values from

$$B_n = -\frac{1}{n+1} \sum_{k=0}^{n-1} \binom{n+1}{k} B_k$$

and constructs

$$B_n(x) = \sum_{k=0}^{n} \binom{n}{k} B_{n-k} x^k.$$

For example,

$$B_2(x) = x^2 - x + \frac{1}{6},$$

which is stored as

```
(1/6, -1, 1)
```

17.3.2 Euler polynomials

The public constructors are

```
euler_polynomial_at_zero(
    index: int,
) -> Fraction

euler_polynomial(
    degree: int,
) -> Polynomial

euler_polynomial_family(
) -> PolynomialFamily
```

The implemented convention is determined by

$$\frac{2e^{xt}}{e^t+1}=\sum_{n\ge 0}E_n(x)\frac{t^n}{n!}.$$

At zero,

$$E_0(0)=1,\qquad E_n(0)=-\frac{1}{2}\sum_{k=0}^{n-1}\binom{n}{k}E_k(0),$$

and

$$E_n(x)=\sum_{k=0}^{n}\binom{n}{k}E_{n-k}(0)x^k.$$

The convention matters because several sequences called Euler numbers occur in the literature. The signed secant specialization used elsewhere in the package is recovered from

$$\mathcal{E}_n=2^nE_n\left(\frac{1}{2}\right).$$

17.3.3 Generic finite Appell families

The coordinate-classical Appell API has these exact signatures:

```
appell_polynomial(
    degree: int,
    values_at_zero: Sequence[Scalar],
) -> Polynomial

appell_polynomial_family(
    values_at_zero: Sequence[Scalar],
    *,
    name: str = "custom Appell polynomials",
) -> PolynomialFamily
```

For a supplied seed table $P_n(0)$, it constructs

$$P_n(x)=\sum_{k=0}^{n}\binom{n}{k}P_{n-k}(0)x^k.$$

Hence

$$P_n'(x)=nP_{n-1}(x).$$

The import should remain namespace-qualified:

```
from fractions import Fraction

import komorebi_math.coordinates as coordinates

seed = (
    Fraction(1),
    Fraction(-1, 2),
    Fraction(1, 6),
    Fraction(0),
    Fraction(-1, 30),
```

```
    Fraction(0),
)

bernoulli_appell = coordinates.appell_polynomial_family(
    seed,
    name="Bernoulli via Appell",
)

for degree in range(1, 6):
    derivative = coordinates.polynomial_derivative(
        bernoulli_appell.polynomial(degree)
    )
    expected = tuple(
        degree * coefficient
        for coefficient in bernoulli_appell.polynomial(degree - 1)
    )
    assert derivative == expected
```

The seed must begin with a nonzero value and extend through the highest degree requested later. It is a finite live callback, not an implicitly extensible symbolic generating function.

17.3.4 Chebyshev families

The exact constructors are

```
chebyshev_t_polynomial(degree: int) -> Polynomial
chebyshev_t_polynomial_family() -> PolynomialFamily

chebyshev_u_polynomial(degree: int) -> Polynomial
chebyshev_u_polynomial_family() -> PolynomialFamily
```

They satisfy

$$T_0 = 1, \qquad T_1 = x, \qquad T_{n+1} = 2xT_n - T_{n-1},$$

and

$$U_0 = 1, \qquad U_1 = 2x, \qquad U_{n+1} = 2xU_n - U_{n-1}.$$

For $n \geq 1$, the leading coefficient of T_n is 2^{n-1}, while the leading coefficient of U_n is 2^n.

17.3.5 Legendre polynomials

```
legendre_polynomial(
    degree: int,
) -> Polynomial

legendre_polynomial_family(
) -> PolynomialFamily
```

The implementation uses

$$P_0 = 1, \qquad P_1 = x,$$

$$(n+1)P_{n+1} = (2n+1)xP_n - nP_{n-1}.$$

The leading coefficient is

$$[x^n]P_n(x) = 2^{-n}\binom{2n}{n}.$$

17.3.6 Hermite conventions

The two conventions are exposed separately:

```
hermite_probabilists_polynomial(degree: int) -> Polynomial
hermite_probabilists_polynomial_family() -> PolynomialFamily

hermite_physicists_polynomial(degree: int) -> Polynomial
hermite_physicists_polynomial_family() -> PolynomialFamily
```

Probabilists' Hermite polynomials satisfy

$$He_0 = 1, \qquad He_1 = x, \qquad He_{n+1} = xHe_n - nHe_{n-1}.$$

Physicists' Hermite polynomials satisfy

$$H_0 = 1, \qquad H_1 = 2x, \qquad H_{n+1} = 2xH_n - 2nH_{n-1}.$$

The two families have leading coefficients 1 and 2^n, respectively. The explicit names prevent a silent convention change.

17.3.7 Ordinary Laguerre polynomials

```
laguerre_polynomial(
    degree: int,
) -> Polynomial

laguerre_polynomial_family(
) -> PolynomialFamily
```

Only the ordinary, unparameterized Laguerre family is implemented:

$$L_n(x) = \sum_{k=0}^{n}(-1)^k\binom{n}{k}\frac{x^k}{k!}.$$

Generalized Laguerre, Jacobi, Gegenbauer, and other parameterized orthogonal families are not supplied by this catalog.

17.3.8 Degree-four reference table

All rows below are exact ascending coefficient tuples returned by the installed implementation.

Family	Degree-four tuple
Bernoulli	$(-1/30, 0, 1, -2, 1)$
Euler	$(0, 1, 0, -2, 1)$
Chebyshev T	$(1, 0, -8, 0, 8)$
Chebyshev U	$(1, 0, -12, 0, 16)$
Legendre	$(3/8, 0, -15/4, 0, 35/8)$
Probabilists' Hermite	$(3, 0, -6, 0, 1)$
Physicists' Hermite	$(12, 0, -48, 0, 16)$
Laguerre	$(1, -4, 3, -2/3, 1/24)$

17.3.9 Classical-family boundaries

The implementation constructs these finite coefficient tuples with exact rational arithmetic, and the cited verification suites compare the formulas and coordinate conversions exactly. `PolynomialFamily` itself does not issue a family-level mathematical certificate. A derived `CoordinateTransform` certificate records finite matrix shape and determinant diagnostics. Neither mechanism, in this namespace, certifies:

- An orthogonality measure or interval.
- A norm or normalization integral.
- The location or separation of roots.
- Convergence of an infinite expansion.
- Approximation error for a truncated expansion.
- Gaussian quadrature weights.
- Analytic continuation of a generating function.

Those claims require separate mathematical objects and evidence.

17.4 Fibonacci, Lucas, Bell, Touchard, and combinatorial families

17.4.1 Fibonacci polynomial family

The standard Fibonacci polynomials satisfy

$$F_0(x) = 0, \qquad F_1(x) = 1, \qquad F_{n+1}(x) = xF_n(x) + F_{n-1}(x).$$

Their public constructors are

```
fibonacci_polynomial(
    index: int,
) -> Polynomial

fibonacci_polynomial_family(
) -> PolynomialFamily
```

Since F_n has degree $n-1$ for $n \geq 1$, the graded family is shifted:

$$G_n(x) = F_{n+1}(x).$$

This shift is part of the API contract. The degree-three family member is

$$G_3(x) = F_4(x) = x^3 + 2x,$$

stored as

```
(0, 2, 0, 1)
```

17.4.2 Lucas polynomial family

```
lucas_polynomial(
    index: int,
) -> Polynomial

lucas_polynomial_family(
) -> PolynomialFamily
```

The recurrence is

$$L_0(x) = 2, \qquad L_1(x) = x, \qquad L_n(x) = xL_{n-1}(x) + L_{n-2}(x).$$

Unlike the Fibonacci constructor, this sequence already has degree equal to its index, including a nonzero constant at degree zero. No index shift is required.

17.4.3 Pell polynomial family

```
pell_polynomial(
    index: int,
) -> Polynomial

pell_polynomial_family(
) -> PolynomialFamily
```

The implemented convention is

$$P_0(x) = 0, \qquad P_1(x) = 1, \qquad P_n(x) = 2xP_{n-1}(x) + P_{n-2}(x).$$

As with Fibonacci polynomials, the graded family uses P_{n+1}. Its leading coefficient is 2^n.

17.4.4 Recurrence-family comparison

At degree four, the three installed family members are

Family	Polynomial	Coefficient tuple
Fibonacci graded	$F_5 = x^4 + 3x^2 + 1$	$(1, 0, 3, 0, 1)$
Lucas	$L_4 = x^4 + 4x^2 + 2$	$(2, 0, 4, 0, 1)$
Pell graded	$P_5 = 16x^4 + 12x^2 + 1$	$(1, 0, 12, 0, 16)$

These are polynomial-coordinate families. The recurrence-defect and companion state coordinates of Chapter 18 are different objects even when they use the same family names.

17.4.5 Bell and Touchard ownership boundary

Bell and Touchard coefficient rows are owned by `komorebi_math.generating`, not by the built-in polynomial coordinate catalog. The relevant public signatures are

```
bell_numbers(
    count: int,
) -> tuple[int, ...]

generalized_bell_numbers(
    power: int,
    count: int,
) -> tuple[int, ...]

generalized_bell_polynomials(
    power: int,
    count: int,
) -> tuple[tuple[int, ...], ...]

weighted_generalized_bell_polynomials(
    power: int,
    count: int,
    weights: Iterable[int | Fraction] | Callable[[int], int | Fraction],
) -> tuple[tuple[Fraction, ...], ...]
```

```
weighted_generalized_bell_numbers(
    power: int,
    count: int,
    weights: Iterable[int | Fraction] | Callable[[int], int | Fraction],
) -> tuple[Fraction, ...]

audit_touchard(
    values: Sequence[int],
    prime: int,
) -> RelationAudit
```

The factorial-power Bell polynomials are defined by

$$\exp\left(x\sum_{m\geq 1}\frac{z^m}{(m!)^s}\right)=\sum_{n\geq 0}B_n^{(s)}(x)\frac{z^n}{(n!)^s}.$$

Their exact recurrence is

$$B_n^{(s)}(x)=x\sum_{m=1}^{n}\binom{n-1}{m-1}\binom{n}{m}^{s-1}B_{n-m}^{(s)}(x).$$

When $s=1$, these rows are the ordinary Touchard polynomials. Evaluation at $x=1$ gives the ordinary Bell numbers. The first six rows are

```
(1,)
(0, 1)
(0, 1, 1)
(0, 1, 3, 1)
(0, 1, 7, 6, 1)
(0, 1, 15, 25, 10, 1)
```

and their values at $x=1$ begin

```
1, 1, 2, 5, 15, 52
```

This lookup intentionally fails:

```
from komorebi_math.coordinates import get_polynomial_family

get_polynomial_family("touchard")
```

with

```
ValidationError: unknown polynomial family 'touchard'
```

17.4.6 Finite Touchard adapter

Generated rows can be adapted explicitly when an exact finite family connection is needed:

```
from fractions import Fraction

from komorebi_math.coordinates import (
    PolynomialFamily,
    connection_transform,
    get_polynomial_family,
)
from komorebi_math.generating import generalized_bell_polynomials

maximum_degree = 5
rows = generalized_bell_polynomials(1, maximum_degree + 1)

touchard = PolynomialFamily(
    "Touchard polynomials",
    lambda degree: tuple(Fraction(value) for value in rows[degree]),
)

to_monomials = connection_transform(
    touchard,
    get_polynomial_family("monomial"),
    maximum_degree,
)

assert to_monomials.roundtrip_valid()
print(to_monomials.matrix)
```

Mathematically simplified values:

```
((1, 0, 0, 0, 0, 0),
 (0, 1, 1, 1, 1, 1),
 (0, 0, 1, 3, 7, 15),
 (0, 0, 0, 1, 6, 25),
 (0, 0, 0, 0, 1, 10),
 (0, 0, 0, 0, 0, 1))
```

The captured row table must extend through every degree later requested from the callback. This adapter does not mutate the built-in catalog and does not make the callback-bearing `PolynomialFamily` portable.

17.4.7 Touchard congruence audit

For a prime p, the classical Touchard congruence is

$$B_{n+p} \equiv B_{n+1} + B_n \pmod{p}.$$

The finite audit is

```
from komorebi_math.generating import audit_touchard, bell_numbers

audit = audit_touchard(bell_numbers(30), 5)

print(audit.modulus)
print(audit.tested_cases)
print(audit.failures)
```

Output:

```
5
25
0
```

The function names its second argument `prime`, but it does not internally test primality or impose the number-theoretic hypothesis required by Touchard's theorem. Callers should supply a positive prime. The returned record counts modular failures on exactly the available prefix indices $0 \le n < \text{len}(\texttt{values}) - p$. It is a finite audit of the supplied data, not a new proof of the congruence for all indices.

17.4.8 Bell-family validation boundaries

`generalized_bell_polynomials` and `generalized_bell_numbers` explicitly reject a Boolean, noninteger, or nonpositive factorial power and reject a negative count. A count of zero returns an empty tuple. Their current runtime does not perform a separate explicit integer-type check for `count`. A nonintegral value will fail when Python attempts to use it as a finite range. Weighted families accept either a callable on positive block sizes or a table whose zeroth entry is ignored. A finite table that is too short for the requested coefficient range raises `ValueError`.

The Bell generators use exact integer or rational recurrence arithmetic. They do not require SymPy. The modular audit layer and its hypotheses remain separate from the coordinate-transform layer.

17.5 `connection_transform`

$$p_n(x) = \sum_{k=0}^{n} C_{kn} q_k(x).$$

17.5.1 Exact signature and direction

```
connection_transform(
    source: PolynomialFamily,
    target: PolynomialFamily,
    degree: int,
    *,
    budget: CoordinateBudget = DEFAULT_COORDINATE_BUDGET,
) -> CoordinateTransform
```

Let $P = (p_n)$ be the source family and $Q = (q_n)$ the target family. A source coefficient vector a and target coefficient vector b represent the same polynomial when

$$\sum_{n=0}^{d} a_n p_n(x) = \sum_{k=0}^{d} b_k q_k(x).$$

In monomial coefficient matrices this becomes

$$M_P a = M_Q b.$$

Therefore

$$b = C_{Q \leftarrow P} a, \qquad C_{Q \leftarrow P} = M_Q^{-1} M_P.$$

The returned `CoordinateTransform` maps source coordinates to target coordinates. Its column n is the target coefficient vector of the source basis polynomial p_n. This is the matrix convention expressed by the displayed equation at the beginning of this section.

Choosing the connection method. `PolynomialFamily` has no `.connection_to()` method. Use `connection_transform(source, target, degree)`. The similarly named `FiniteBasis.connection_to(target)` method belongs to the arbitrary finite basis API and is used for Bernstein, Newton, and Lagrange bases.

17.5.2 Triangularity, determinant, and inverse

Let $\lambda_n(P) = [x^n]p_n(x)$ and $\lambda_n(Q) = [x^n]q_n(x)$. Since both coefficient matrices are upper triangular,

$$[C_{Q\leftarrow P}]_{nn} = \frac{\lambda_n(P)}{\lambda_n(Q)}.$$

Consequently,

$$\det C_{Q\leftarrow P} = \prod_{n=0}^{d} \frac{\lambda_n(P)}{\lambda_n(Q)} \neq 0.$$

Every valid graded-family connection is therefore exactly invertible. Its inverse is the connection in the opposite direction:

$$C_{P\leftarrow Q} = C_{Q\leftarrow P}^{-1}.$$

For three compatible families,

$$C_{R\leftarrow P} = C_{R\leftarrow Q}C_{Q\leftarrow P}.$$

The API uses the same order:

```
source_to_middle.then(middle_to_target)
```

forms `middle_to_target.matrix @ source_to_middle.matrix`.

17.5.3 Returned transform interface

The most relevant methods and properties are

```
CoordinateTransform.apply(
    self,
    coordinates: Sequence[Scalar],
) -> Vector

CoordinateTransform.inverse(
    self,
    *,
    theorem: str | None = None,
) -> CoordinateTransform

CoordinateTransform.then(
    self,
    following: CoordinateTransform,
    *,
    theorem: str | None = None,
) -> CoordinateTransform

CoordinateTransform.roundtrip_valid(
    self,
) -> bool

CoordinateTransform.certificate(
```

```
    self,
) -> Certificate
```

and

```
source_dimension
target_dimension
square
determinant
```

For an invertibility claim, use a nonzero determinant, `roundtrip_valid()`, or successful exact inversion. The generic transform certificate records finite shape and determinant diagnostics. Its `valid` flag is not, by itself, a generic nonsingularity assertion for every possible manually constructed square `CoordinateTransform`. A transform returned by `connection_transform` is nonsingular because both validated graded coefficient matrices are nonsingular.

17.5.4 Legendre-to-Chebyshev worked example

```
from fractions import Fraction

from komorebi_math.stable import (
    connection_transform,
    get_polynomial_family,
)

legendre = get_polynomial_family("legendre")
chebyshev = get_polynomial_family("chebyshev-t")

change = connection_transform(legendre, chebyshev, 3)
source = tuple(Fraction(value) for value in (1, 2, 3, 4))
target = change.apply(source)
recovered = change.inverse().apply(target)

print(change.matrix)
print(change.determinant)
print(target)
print(recovered)
print(change.roundtrip_valid())
```

Mathematically simplified values:

```
((1, 0, 1/4, 0),
 (0, 1, 0, 3/8),
 (0, 0, 3/4, 0),
 (0, 0, 0, 5/8))
15/32
(7/4, 7/2, 9/4, 5/2)
(1, 2, 3, 4)
True
```

The first matrix columns encode

$$P_0 = T_0, \qquad P_1 = T_1,$$

$$P_2 = \frac{1}{4}T_0 + \frac{3}{4}T_2,$$

and

$$P_3 = \frac{3}{8}T_1 + \frac{5}{8}T_3.$$

17.5.5 CLI form

The same connection can be emitted as exact JSON:

```
python -m komorebi_math coordinates polynomial --source legendre --target chebyshev-t --degree 3
```

The result includes

```
{
  "degree": 3,
  "determinant": "15/32",
  "source": "Legendre polynomials coordinates through degree 3",
  "target": "Chebyshev T polynomials coordinates through degree 3",
  "verified": true
}
```

The complete JSON also contains the exact string-valued matrix. The committed degree-four behavioral record has determinant

$$\frac{525}{2048}.$$

Available catalog entries can be inspected with

```
python -m komorebi_math coordinates catalog
```

17.5.6 Named Euler-to-Fibonacci specialization

The generic connection mechanism underlies the named specialization

```
euler_to_fibonacci_transform(
    degree: int,
    *,
    budget: CoordinateBudget = DEFAULT_COORDINATE_BUDGET,
) -> CoordinateTransform
```

That constructor independently compares the general matrix with the explicit coefficient formula and raises `ValidationError` if they disagree. Through degree four its matrix is

$$\begin{bmatrix} 1 & -1/2 & -1 & 7/4 & 2 \\ 0 & 1 & -1 & -2 & 5 \\ 0 & 0 & 1 & -3/2 & -3 \\ 0 & 0 & 0 & 1 & -2 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}.$$

Chapter 18's sequence-level Euler-to-Fibonacci construction is a separate map. It analyzes a finite sequence into an initial Fibonacci companion state and a recurrence-defect stream. It is not this polynomial-family connection, nor an alternative interpretation of this connection matrix. The sequence-level construction and its worked state path appear in Chapter 18.

17.5.7 Resource budget

The default finite coordinate budget is

```
CoordinateBudget(
    maximum_matrix_dimension=512,
    maximum_polynomial_degree=512,
    maximum_horizon=4096,
    maximum_exact_entries=1_000_000,
)
```

Before materializing a family connection, the implementation checks both the degree and the $(d+1)\times(d+1)$ exact matrix allocation. Because degree d requires dimension $d+1$, the default matrix-dimension limit makes degree 511 the largest possible connection under the default budget even though the independent degree limit is 512.

A deliberately small budget rejects this request before the matrix is allocated:

```
from komorebi_math.coordinates import (
    CoordinateBudget,
    connection_transform,
    get_polynomial_family,
)

small = CoordinateBudget(maximum_matrix_dimension=4)

connection_transform(
    get_polynomial_family("monomial"),
    get_polynomial_family("legendre"),
    4,
    budget=small,
)
```

Diagnostic:

```
ResourceLimitError: polynomial family connection requires a 5 by 5 matrix,
exceeding the coordinate dimension limit 4
```

The budget is a resource contract, not a performance prediction. Exact rational coefficient growth can make a computation expensive before it reaches the nominal dimension ceiling.

17.5.8 Arithmetic complexity

Let $N = d+1$. The table below counts exact scalar operations and suppresses the potentially dominant bit complexity of growing rational numerators and denominators.

Operation	Arithmetic scale	Stored exact entries
Apply a dense family connection	$O(N^2)$	$O(N)$ output
Store one coefficient matrix	$O(N^2)$	$O(N^2)$
Exact matrix inversion	$O(N^3)$	$O(N^2)$
Dense matrix product	$O(N^3)$	$O(N^2)$
Complete family connection	$O(N^3)$ plus family generation	$O(N^2)$
Newton basis construction	$O(N^2)$	$O(N^2)$
Lagrange basis construction	$O(N^3)$	$O(N^2)$
Bell rows through degree d	$O(N^3)$	$O(N^2)$

Family generation varies. Closed coefficient formulas can fill one polynomial in $O(n)$ or $O(n^2)$ arithmetic, while recurrence constructors that are restarted independently for each degree can add another cubic-scale component

across a complete coefficient matrix. Exact fraction size, rather than matrix dimension alone, often controls practical runtime.

17.5.9 Persistence boundary

`PolynomialFamily` contains an executable callback and has no portable codec. Named families should be reconstructed by catalog name. Custom families should be reconstructed from their declared finite data and rule.

The registered data-only types used here are portable. Portability is not an automatic property of every object derived from a family callback. In particular, a `CoordinateTransform` returned by `connection_transform` can use the specialized transform functions

```
save_transform(
    path: str | Path,
    transform: CoordinateTransform,
    *,
    overwrite: bool = False,
) -> Path

load_transform(
    path: str | Path,
) -> CoordinateTransform
```

For example:

```
from komorebi_math.coordinates import (
    connection_transform,
    get_polynomial_family,
    load_transform,
    save_transform,
)

change = connection_transform(
    get_polynomial_family("legendre"),
    get_polynomial_family("chebyshev-t"),
    8,
)

path = save_transform("legendre_to_chebyshev.json", change)
restored = load_transform(path)

assert restored == change
```

The portable registry includes these four types:

- `FiniteBasis`
- `CoordinateTransform`
- `PolynomialCoordinateOperator`
- `CoordinateBudget`

Objects of these types can be written with `save_coordinate_object` and read with `load_coordinate_object`. A custom callback, an arbitrary closure captured by that callback, and an unregistered derived class are not made portable merely because their outputs are rational.

The version-1 portable envelope records a payload SHA-256 digest. Loading rejects a changed payload or a schema mismatch. This is an integrity mechanism, not a cryptographic signature of authorship or origin.

17.5.10 Connection failure matrix

Condition	Exception	Meaning
Source or target is not `PolynomialFamily`	`ValidationError`	Family protocol not supplied
Negative, Boolean, or noninteger degree	`ValidationError`	Invalid finite truncation
Family callback violates exact grade	`ValidationError`	Coefficient matrix is not a graded basis
Degree or matrix exceeds budget	`ResourceLimitError`	Allocation refused before construction
Applied vector has length other than $d+1$	`DimensionError`	Coordinate dimension mismatch
Composed transform names do not meet	`ValidationError`	Endpoint labels do not compose
Composed transform dimensions differ	`DimensionError`	Matrix shapes do not compose
Requested inverse is rectangular	`DimensionError`	Only square transforms are invertible
Exact square matrix is singular	`ValidationError`	Inverse does not exist
Appell seed is empty or starts at zero	`ValidationError`	Not a valid graded Appell family
Appell seed is too short	`DimensionError`	Requested polynomial is undeclared
Unknown family alias such as `bell`	`ValidationError`	Object is outside the built-in catalog

17.5.11 Verification and evidence map

The implementation is exercised at several levels.

Evidence source	Relevant checks
`tests/test_coordinates_general_classical.py`	Classical formulas, all catalog families, Bernstein partition of unity, Newton/Lagrange cardinality, Appell lowering
`tests/test_coordinates_polynomials.py`	Fibonacci companion identity, monomial/Fibonacci formula, Euler/Fibonacci matrix, composition, operator conjugacy, secant specialization
`tests/test_coordinates_general_calculus.py`	Derivative/integral operators, falling-factorial finite difference, transported operators
`tests/test_coordinates_general_persistence.py`	Direct `FiniteBasis` and `PolynomialCoordinateOperator` round trips, `CoordinateTransform` edges inside an atlas round trip, and tamper rejection for a formal-coordinate record
`tests/test_coordinates_normalized_persistence.py`	Normalization-change `CoordinateTransform` persistence, recurrence-record persistence, and transform-envelope tamper rejection
`tests/test_block14_cli_api.py`	Catalog, default polynomial connection, Legendre/Chebyshev CLI, golden determinant
`verification/block14_general_coordinate_audit.py`	Every catalog family through degree 10 and exact evaluation at 13 rational points
`verification/block14_deep_audit.py`	Independent Fibonacci recurrence and independent Euler/Fibonacci triangular resolution through degree 30
`tests/test_generating_foundation.py`	Bell row integrality, Bell-number evaluation, and finite Touchard audits
`examples/classical_coordinate_families.py`	Executable Legendre/Chebyshev and operator-transport demonstration

The focused source replay for the four principal coordinate test modules reports

```
15 passed
```

and the installed coordinate verification reports

```
verification passed 24/24 checks
```

Relevant named verification checks include:

- `Euler to Fibonacci polynomial connection.`
- `operator conjugation.`
- `portable coordinate records and tamper rejection.`
- `coordinate resource contract.`
- `classical polynomial-family atlas.`
- `coordinate differential and difference calculus.`
- `general portable coordinate records.`

All claims in this chapter are finite. A successful transform proves an exact identity in the declared polynomial space through degree d. A successful prefix audit proves that the checked cases passed. Neither result silently extends its evidence beyond the declared finite boundary.

Chapter 17 summary. A graded `PolynomialFamily` turns exact coefficient rules into upper-triangular basis matrices. Built-in families cover monomial, factorial, Bernoulli, Euler, recurrence, Chebyshev, Legendre, Hermite, and Laguerre coordinates. Bernstein and interpolation objects are fixed `FiniteBasis` records. Bell and Touchard rows remain owned by the generating-function namespace and enter the coordinate layer only through an explicit finite adapter. `connection_transform` maps source coefficient vectors to target coefficient vectors by $M_Q^{-1} M_P$, subject to an explicit resource budget and a data-only persistence boundary.

18. Combinatorial, recurrence, and state-space coordinates

First route. If your input is a finite list, the square-sequence example in Section 18.2 is the simplest exact roundtrip. To study a proposed recurrence, use the direct analysis and reconstruction in Section 18.3.4, then inspect the defects. A nonzero defect is a coordinate of the supplied data. The dense transform and state-space sections become useful when you need to compose coordinate maps or compare internal representations.

Three exact coordinate languages encode finite data. The first replaces a sequence or polynomial coefficient vector by triangular combinatorial coordinates. The second separates a finite sequence into an initial recurrence state and a forcing, or defect, stream. The third changes coordinates inside finite realizations and discrete-time state-space models without changing their observable behavior.

Every matrix in this chapter is finite. Exact constructors convert integer and `Fraction` inputs to immutable rational matrices. They do not invoke a symbolic algebra system, a numerical eigensolver, or a GPU backend. This gives the identities in this chapter exact finite semantics, but it does not turn finite agreement into a theorem about an unspecified infinite sequence.

The matrix direction, composition order, inversion behavior, and certificate semantics of `CoordinateTransform` are defined in Chapter 16. This chapter uses that common contract rather than repeating it. In particular, if

```
target_coordinates = transform.apply(source_coordinates)
```

then rows of `transform.matrix` index target coordinates and columns index source coordinates.

> **API tier.** The complete interface documented here is public through `komorebi_math.coordinates`. A selected convenience subset is re-exported at the `komorebi_math` root. Of the functions in this chapter, `forward_difference_transform` is also part of the frozen `komorebi_math.stable` interface. The remaining names are research-tier public APIs in `1.0.0b4`. Applications using them should pin the package version. The shared `CoordinateTransform` and `FiniteBasis` result types are themselves also available from the stable namespace. This tier statement is about the chapter-specific constructors and algorithms.

> **Chapter 18 summary.** Stirling, Lah, binomial, difference, zeta, and Möbius matrices are exact finite changes of coordinates. A recurrence model maps a prefix to its initial state plus the unique defect stream that makes the recurrence true. Companion Green weights reconstruct the same prefix by variation of constants. Realization and state-space coordinates then expose observability, controllability, intertwiners, and transfer invariance. The implementation also contains one confirmed convenience-method defect, disclosed in Section 18.3, with a fully working lower-level path.

18.1 Binomial and Stirling transforms

18.1.1 Exact triangular conventions

For nonnegative integers n and k, the package uses the Stirling recurrence

$$S(n,k) = S(n-1,k-1) + kS(n-1,k)$$

for the second kind and

$$s(n,k) = s(n-1,k-1) - (n-1)s(n-1,k)$$

for the signed first kind. The unsigned first-kind value is

$$c(n,k) = (-1)^{n-k}s(n,k).$$

The unsigned Lah number uses

$$L(n,k) = \begin{cases} 1, & n = k = 0, \\ \binom{n-1}{k-1}\dfrac{n!}{k!}, & 1 \le k \le n, \\ 0, & \text{otherwise.} \end{cases}$$

These scalar functions are public:

Function	Exact signature	Meaning
`stirling_second`	`stirling_second(n: int, k: int) -> int`	$S(n,k)$
`stirling_first_signed`	`stirling_first_signed(n: int, k: int) -> int`	$s(n,k)$
`stirling_first_unsigned`	`stirling_first_unsigned(n: int, k: int) -> int`	$c(n,k)$
`lah_number`	`lah_number(n: int, k: int) -> int`	$L(n,k)$
`mobius_integer`	`mobius_integer(value: int) -> int`	Classical integer Möbius function $\mu(n)$

For $k > n$, `stirling_second`, `stirling_first_signed`, and `lah_number` return the integer zero. The current `stirling_first_unsigned` implementation multiplies that zero by $(-1)^{n-k}$. Because a negative exponent produces a Python float, this out-of-triangle edge returns `0.0` or `-0.0` despite its declared `-> int` annotation. Values inside the triangle remain integers, and the transform constructors normalize the zero to an exact `Fraction(0)`. At $(0,0)$, both Stirling functions and the Lah function return one. Negative indices and Boolean inputs are rejected. `mobius_integer` requires a positive integer.

The basis identities that fix the matrix orientation are

$$x^n = \sum_{k=0}^{n} S(n,k)x^{\underline{k}}, \qquad x^{\underline{n}} = \sum_{k=0}^{n} s(n,k)x^k,$$

$$x^n = \sum_{k=0}^{n} (-1)^{n-k}S(n,k)x^{\overline{k}}, \qquad x^{\overline{n}} = \sum_{k=0}^{n} c(n,k)x^k,$$

and

$$x^{\overline{n}} = \sum_{k=0}^{n} L(n,k) x^{\underline{k}}.$$

Here

$$x^{\underline{k}} = x(x-1)\cdots(x-k+1), \qquad x^{\overline{k}} = x(x+1)\cdots(x+k-1).$$

If a polynomial has monomial coefficient vector $a = (a_0, \dots, a_N)^T$, the matrix returned by `monomial_to_falling_factorial_transform(N)` has entry

$$T_{k,n} = S(n,k),$$

so `T.apply(a)` is the coefficient vector in the falling-factorial basis. The other constructors follow the same target-row, source-column convention.

18.1.2 Transform constructors

All combinatorial constructors accept an optional `CoordinateBudget`. In the table below, `budget=DEFAULT_COORDINATE_BUDGET` is written as `budget=DEFAULT` to keep signatures readable.

Constructor	Exact public signature	Theorem metadata
`monomial_to_falling_factorial_transform`	`(degree: int, *, budget=DEFAULT) -> CoordinateTransform`	`CC.C1`
`falling_factorial_to_monomial_transform`	`(degree: int, *, budget=DEFAULT) -> CoordinateTransform`	`CC.C2`
`monomial_to_rising_factorial_transform`	`(degree: int, *, budget=DEFAULT) -> CoordinateTransform`	`CC.C3`
`rising_factorial_to_monomial_transform`	`(degree: int, *, budget=DEFAULT) -> CoordinateTransform`	`CC.C4`
`rising_to_falling_factorial_transform`	`(degree: int, *, budget=DEFAULT) -> CoordinateTransform`	`CC.C5`
`binomial_transform`	`(size: int, *, inverse: bool = False, source: str = "sequence values", target: str = "binomial transform", budget=DEFAULT) -> CoordinateTransform`	`CC.C6` or `CC.C7`
`forward_difference_transform`	`(size: int, *, budget=DEFAULT) -> CoordinateTransform`	`CC.C8`
`newton_evaluation_transform`	`(size: int, *, budget=DEFAULT) -> CoordinateTransform`	`CC.C9`
`divisor_zeta_transform`	`(size: int, *, inverse: bool = False, budget=DEFAULT) -> CoordinateTransform`	`CC.C10` or `CC.C11`
`subset_zeta_transform`	`(rank: int, *, inverse: bool = False, budget=DEFAULT) -> CoordinateTransform`	`CC.C12` or `CC.C13`
`transform_sequence`	`(values: Sequence[int | Fraction], transform: CoordinateTransform) -> tuple[Fraction, ...]`	Uses the supplied transform

`degree` means the largest polynomial degree, so a degree-N connection is an $(N+1) \times (N+1)$ matrix. `size` means the number of sequence coordinates, so a size-N transform is $N \times N$.

18.1.3 Binomial coordinates

The forward finite binomial transform is

$$b_n = \sum_{k=0}^{n} \binom{n}{k} a_k, \qquad 0 \le n < N.$$

Its inverse is

$$a_n = \sum_{k=0}^{n} (-1)^{n-k} \binom{n}{k} b_k.$$

The inverse can be requested directly:

```
from komorebi_math.coordinates import binomial_transform

values = (1, 2, 3, 4)
forward = binomial_transform(4)
inverse = binomial_transform(4, inverse=True)

coordinates = forward.apply(values)
recovered = inverse.apply(coordinates)

print("coordinates:", coordinates)
print("recovered:", recovered)
```

Output:

```
coordinates: (Fraction(1, 1), Fraction(3, 1), Fraction(8, 1), Fraction(20, 1))
recovered: (Fraction(1, 1), Fraction(2, 1), Fraction(3, 1), Fraction(4, 1))
```

The input integers are converted to `Fraction` objects. Exact output does not imply that the transformed numbers are integers for an arbitrary rational input. It means the rational result is represented without floating-point roundoff.

18.1.4 Divisor zeta and Möbius coordinates

Indices of the dense divisor transform correspond to the positive integers $1, \dots, N$. The forward map is

$$b_n = \sum_{d \mid n} a_d, \qquad 1 \le n \le N,$$

and its inverse is

$$a_n = \sum_{d \mid n} \mu(n/d) b_d.$$

For example:

```
from komorebi_math.coordinates import divisor_zeta_transform

values = (1, 2, 3, 4, 5, 6)
zeta = divisor_zeta_transform(6)
mobius = divisor_zeta_transform(6, inverse=True)

divisor_sums = zeta.apply(values)
assert mobius.apply(divisor_sums) == values
print(divisor_sums)
```

Output:

```
(Fraction(1, 1), Fraction(3, 1), Fraction(4, 1),
 Fraction(7, 1), Fraction(6, 1), Fraction(12, 1))
```

Although the mathematical divisor matrix is sparse, the current constructor materializes a dense exact matrix. The resource discussion in Section 18.5 is therefore part of the operational contract.

18.1.5 Boolean-lattice zeta and Möbius coordinates

Subsets are encoded by bit masks from 0 through $2^r - 1$. For a rank-r Boolean lattice,

$$b(S) = \sum_{T \subseteq S} a(T),$$

with inverse

$$a(S) = \sum_{T \subseteq S} (-1)^{|S \setminus T|} b(T).$$

At rank two, the input order is $\emptyset, \{0\}, \{1\}, \{0, 1\}$. Thus:

```python
from komorebi_math.coordinates import subset_zeta_transform

values = (1, 2, 3, 4)
zeta = subset_zeta_transform(2)
inverse = subset_zeta_transform(2, inverse=True)

subset_sums = zeta.apply(values)
assert inverse.apply(subset_sums) == values
print(subset_sums)
```

Output:

```
(Fraction(1, 1), Fraction(3, 1), Fraction(4, 1), Fraction(10, 1))
```

The coordinate count is 2^r, so a dense matrix contains 4^r entries. Under the default matrix-dimension budget, rank nine is the largest accepted rank. Rank ten would request a 1024×1024 matrix and is rejected before allocation.

18.2 Forward-difference coordinates

$$\Delta^k a_0 = \sum_{j=0}^{k} (-1)^{k-j} \binom{k}{j} a_j.$$

18.2.1 Analysis and Newton synthesis

For samples

$$a = (f(0), f(1), \dots, f(N-1))^T,$$

`forward_difference_transform(N)` returns the lower-triangular analysis matrix

$$D_{k,j} = \begin{cases} (-1)^{k-j}\binom{k}{j}, & j \le k, \\ 0, & j > k. \end{cases}$$

Its output is

$$(f(0), \Delta f(0), \ldots, \Delta^{N-1} f(0))^T.$$

`newton_evaluation_transform(N)` is the exact inverse. It implements

$$f(n) = \sum_{k=0}^{n} \binom{n}{k} \Delta^k f(0), \qquad 0 \le n < N.$$

The inverse is constructed from the analysis transform and exact rational matrix inversion. Both transforms are square, and their exact determinant is one.

18.2.2 Worked example: the square sequence

A quadratic sequence has no forward differences above order two. The complete round trip is:

```
from komorebi_math.stable import forward_difference_transform
from komorebi_math.coordinates import newton_evaluation_transform

squares = (0, 1, 4, 9, 16, 25)
analysis = forward_difference_transform(6)
synthesis = newton_evaluation_transform(6)

differences = analysis.apply(squares)
recovered = synthesis.apply(differences)

assert recovered == squares
print("differences:", differences)
print("recovered:", recovered)
```

Output:

```
differences: (Fraction(0, 1), Fraction(1, 1), Fraction(2, 1),
              Fraction(0, 1), Fraction(0, 1), Fraction(0, 1))
recovered: (Fraction(0, 1), Fraction(1, 1), Fraction(4, 1),
            Fraction(9, 1), Fraction(16, 1), Fraction(25, 1))
```

The equivalent command-line workflow is:

```
python -m komorebi_math coordinates combinatorial --kind forward-difference --size 6 --values "0,1,4,9,16,25"
```

Its JSON result includes the exact 6×6 matrix, the six output coordinates, source and target names, and `"verified": true`.

18.2.3 Interpretation boundary

Vanishing differences through a finite sample horizon prove a finite matrix identity. If $\Delta^{d+1} f(0) = 0$ throughout a sufficiently long declared sample grid, Newton interpolation returns a degree-at-most-d polynomial on that grid. The package does not infer that an externally supplied infinite sequence must continue to obey the same polynomial law.

`transform_sequence(values, transform)` is a convenience wrapper around `transform.apply(values)`. It requires the sequence length to equal the transform's source dimension. It does not truncate or pad.

18.3 Recurrence defect coordinates

18.3.1 A recurrence as a coordinate system

Let $c = (c_0, \dots, c_{d-1})$ be a nonempty exact coefficient vector. The package convention is

$$a_{n+d} = \sum_{i=0}^{d-1} c_i a_{n+i} + \delta_n.$$

The associated polynomial, stored in ascending coefficient order, is

$$p_c(x) = x^d - \sum_{i=0}^{d-1} c_i x^i,$$

so

```
recurrence_polynomial((c0, c1, ..., c_last))
```

returns

```
(-c0, -c1, ..., -c_last, 1)
```

The consecutive-state companion matrix is

$$C_c = \begin{pmatrix} 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & & \ddots & \ddots & \vdots \\ 0 & 0 & \cdots & 0 & 1 \\ c_0 & c_1 & \cdots & c_{d-2} & c_{d-1} \end{pmatrix}.$$

With

$$x_n = (a_n, a_{n+1}, \dots, a_{n+d-1})^T,$$

the forced state equation is

$$x_{n+1} = C_c x_n + e_d \delta_n, \qquad a_n = e_1^T x_n.$$

For a finite prefix of length $L \geq d$, the unique recurrence coordinates are

$$\mathcal{A}_c(a) = (a_0, \dots, a_{d-1}, \delta_0, \dots, \delta_{L-d-1})^T.$$

The first d rows of the analysis matrix are identity rows. For $0 \leq n < L - d$, row $d + n$ has entries $-c_i$ in columns $n + i$ and a one in column $n + d$. Therefore the full matrix is unit lower triangular:

$$\det \mathcal{A}_c = 1.$$

This is stronger than nonsingularity over $\mathbb{Q}$: the same triangular formula is invertible over every coefficient field containing the recurrence coefficients.

18.3.2 Core recurrence API

`RecurrenceLike` means either an iterable of exact scalars or an arithmetic `LinearRecurrence`. The coordinate module reads only its coefficient vector.

Name	Exact public signature	Result
`recurrence_polynomial`	`(coefficients: RecurrenceLike) -> Vector`	Ascending coefficients of p_c
`companion_matrix`	`(coefficients: RecurrenceLike) -> Matrix`	Exact $d \times d$ companion matrix
`apply_recurrence_defect`	`(values: Sequence[Scalar], coefficients: RecurrenceLike) -> Vector`	The defect stream (δ_n)
`recurrence_analysis_transform`	`(coefficients, length: int, *, name: str | None = None, budget= DEFAULT) -> CoordinateTransform`	Dense analysis matrix, `CC.R1`
`recurrence_synthesis_transform`	`(coefficients, length: int, *, name: str | None = None, budget= DEFAULT) -> CoordinateTransform`	Exact inverse, `CC.R1`
`analyze_recurrence_coordinates`	`(values, coefficients, *, name: str = "recurrence") -> Recurrence CoordinateRecord`	Direct $O(Ld)$ analysis
`synthesize_recurrence_coordinates`	`(record: RecurrenceCoordinate Record) -> Vector`	Direct $O(Ld)$ reconstruction
`coordinate_change_between_recurrences`	`(source_coefficients, target_ coefficients, length, *, source_ name="source recurrence", target_ name="target recurrence", budget= DEFAULT) -> CoordinateTransform`	Exact bridge, `CC.R2`
`defect_green_weights`	`(coefficients, count: int) -> Vector`	Companion impulse-response prefix
`variation_of_constants_prefix`	`(record: RecurrenceCoordinate Record) -> Vector`	Green-kernel reconstruction

The dense transform constructors obey `CoordinateBudget`. The direct record functions do not build an $L \times L$ matrix and therefore are not subject to the matrix-dimension limit. They still materialize the finite input and output vectors in memory.

18.3.3 `RecurrenceCoordinateRecord`

The immutable exact record is

```
RecurrenceCoordinateRecord(
    name: str,
    coefficients: Vector,
    initial: Vector,
    defects: Vector,
) -> None
```

Its fields and derived properties are:

Member	Meaning
`.name`	Nonempty coordinate-system name
`.coefficients`	Exact recurrence vector c
`.initial`	Initial state of length d
`.defects`	Forcing coordinates of length $L - d$
`.order`	d
`.length`	$L = d + \lvert\text{defects}\rvert$
`.homogeneous`	`True` exactly when every defect is zero

Member	Meaning
`.coordinates()`	`.initial + .defects`
`.reconstruct()`	Direct synthesis of the finite prefix
`.certificate()`	Reanalyzes the reconstruction and compares both coordinate parts
`.as_linear_recurrence()`	Homogeneous arithmetic recurrence using the same coefficients and initial state

`.as_linear_recurrence()` intentionally ignores `.defects`. It represents the recorded prefix only when `.homogeneous` is true. Otherwise it returns the unforced comparison model rather than another encoding of the forced record.

The exact certificate has theorem identifier `CC.R1`. Its checked assumptions are a finite sequence prefix and a constant-coefficient recurrence. Diagnostics record the length, order, and homogeneous flag. A valid certificate means the stored coordinates reconstruct and reanalyze exactly. It does not claim that a nonzero defect stream eventually vanishes.

18.3.4 Working analysis and synthesis path

Use this recommended path for the recorded `1.0.0b4` baseline:

```
from komorebi_math.coordinates import (
    analyze_recurrence_coordinates,
    synthesize_recurrence_coordinates,
)

values = (1, 2, 3, 4, 5, 6)
record = analyze_recurrence_coordinates(
    values,
    (1, 1),
    name="Fibonacci companion",
)

recovered = synthesize_recurrence_coordinates(record)
record.certificate().require()

assert recovered == values
print("initial:", record.initial)
print("defects:", record.defects)
print("homogeneous:", record.homogeneous)
```

Output:

```
initial: (Fraction(1, 1), Fraction(2, 1))
defects: (Fraction(0, 1), Fraction(-1, 1),
          Fraction(-2, 1), Fraction(-3, 1))
homogeneous: False
```

Synthesis executes the forced recurrence directly:

$$a_{n+d} = \delta_n + \sum_{i=0}^{d-1} c_i a_{n+i}.$$

It does not invert the dense analysis matrix.

18.3.5 Recorded `RecurrenceModel.synthesize()` erratum

> **Recorded baseline defect.** In the audited `1.0.0b4` source, the convenience method `RecurrenceModel.synthesize(coordinates)` passed an obsolete fifth data argument to the four-field `RecurrenceCoordinateRecord` constructor. Every otherwise valid invocation reached:
>
> ```
> TypeError: RecurrenceCoordinateRecord.__init__() takes 5 positional arguments but 6 were given
> ```
>
> This failure was confirmed by direct execution in the recorded baseline and was not covered by its shipped tests. It was a constructor-call defect, not a failure of recurrence synthesis itself.

For the recorded baseline, the working lower-level path is:

```python
from komorebi_math.coordinates import (
    RecurrenceCoordinateRecord,
    get_recurrence_model,
    synthesize_recurrence_coordinates,
)

model = get_recurrence_model("Fibonacci")
values = (0, 1, 1, 2, 3, 5, 8, 13)
record = model.analyze(values)
coordinates = record.coordinates()

# To synthesize coordinates obtained elsewhere:
rebuilt_record = RecurrenceCoordinateRecord(
    model.name,
    model.coefficients,
    coordinates[: model.order],
    coordinates[model.order :],
)
recovered = synthesize_recurrence_coordinates(rebuilt_record)
assert recovered == values
```

Do not present `model.synthesize(...)` as a passing `1.0.0b4` workflow. The direct function, record method `.reconstruct()`, dense synthesis transform, and coordinate bridges all passed their focused tests.

18.3.6 Change between two recurrence systems

Let A_c denote the analysis matrix of one recurrence and $S_c = A_c^{-1}$ its synthesis matrix. The exact bridge from source recurrence c to target recurrence $\tilde{c}$ is

$$T_{c\to\tilde{c}} = A_{\tilde{c}} S_c.$$

Both factors have determinant one, so

$$\det T_{c\to\tilde{c}} = 1.$$

For every finite raw prefix a,

$$T_{c\to\tilde{c}}\, A_c a = A_{\tilde{c}} a.$$

A runnable bridge is:

```
from komorebi_math.coordinates import (
    analyze_recurrence_coordinates,
    coordinate_change_between_recurrences,
)

values = (1, 2, 3, 4, 5, 6)
source = analyze_recurrence_coordinates(values, (1, 1), name="source")
target = analyze_recurrence_coordinates(values, (2, -1, 3), name="target")

bridge = coordinate_change_between_recurrences(
    (1, 1),
    (2, -1, 3),
    len(values),
)

mapped = bridge.apply(source.coordinates())
assert mapped == target.coordinates()

print("source:", source.coordinates())
print("target:", mapped)
print("determinant:", bridge.determinant)
```

Output:

```
source: (Fraction(1, 1), Fraction(2, 1), Fraction(0, 1),
         Fraction(-1, 1), Fraction(-2, 1), Fraction(-3, 1))
target: (Fraction(1, 1), Fraction(2, 1), Fraction(3, 1),
         Fraction(-5, 1), Fraction(-8, 1), Fraction(-11, 1))
determinant: 1
```

The equivalent CLI command is:

```
python -m komorebi_math coordinates recurrence --values "1,2,3,4,5,6" --source "1,1" --target "2,-1,3"
```

The emitted JSON includes both coordinate vectors, the complete exact bridge matrix, determinant one, and `"verified": true`.

18.3.7 Companion Green weights and variation of constants

For order d, let e_d be the last standard basis vector and e_1^T the first-coordinate covector. The implemented Green weights are

$$g_\ell = e_1^T C_c^{\ell-1} e_d, \qquad 1 \le \ell \le M.$$

If h_n is the homogeneous sequence with the record's initial state, then the finite variation-of-constants formula is

$$a_n = h_n + \sum_{\substack{0 \le j < L-d \\ n-j \ge 1}} g_{n-j}\delta_j.$$

The index condition automatically delays a defect until it can reach the first output coordinate. For Fibonacci coefficients $(1, 1)$,

$$(g_1, g_2, g_3, \ldots) = (0, 1, 1, 2, 3, 5, \ldots).$$

`variation_of_constants_prefix(record)` computes this finite convolution and is independently compared with direct recurrence synthesis in the tests.

18.3.8 Named recurrence models

`RecurrenceModel` is the immutable named wrapper

```
RecurrenceModel(
    name: str,
    coefficients: Vector,
    initial_values: Vector = (),
    description: str = "",
) -> None
```

It provides `.order`, `.companion`, `.sequence()`, `.analyze()`, `.coordinate_transform_to()`, and `.homogeneous_basis()`. Its `.synthesize()` method is subject to the preceding erratum.

The built-in catalog is:

Model	Coefficients	Initial values	First terms
Fibonacci	$(1, 1)$	$(0, 1)$	$0, 1, 1, 2, 3, 5, 8, 13$
Lucas	$(1, 1)$	$(2, 1)$	$2, 1, 3, 4, 7, 11, 18, 29$
Pell	$(1, 2)$	$(0, 1)$	$0, 1, 2, 5, 12, 29, 70, 169$
Jacobsthal	$(2, 1)$	$(0, 1)$	$0, 1, 1, 3, 5, 11, 21, 43$
Tribonacci	$(1, 1, 1)$	$(0, 0, 1)$	$0, 0, 1, 1, 2, 4, 7, 13, 24$
Padovan	$(1, 1, 0)$	$(1, 1, 1)$	$1, 1, 1, 2, 2, 3, 4, 5, 7$

The model helpers are:

```
custom_recurrence_model(
    coefficients: Sequence[Scalar],
    *,
    initial_values: Sequence[Scalar] = (),
    name: str = "custom recurrence",
    description: str = "",
) -> RecurrenceModel

fibonacci_recurrence_model() -> RecurrenceModel
lucas_recurrence_model() -> RecurrenceModel
pell_recurrence_model() -> RecurrenceModel
jacobsthal_recurrence_model() -> RecurrenceModel
tribonacci_recurrence_model() -> RecurrenceModel
padovan_recurrence_model() -> RecurrenceModel
recurrence_model_catalog() -> tuple[RecurrenceModel, ...]
get_recurrence_model(name: str) -> RecurrenceModel
```

`get_recurrence_model` lowercases a name and removes nonalphanumeric characters before matching. It rejects an unknown name rather than silently constructing a custom model.

`model.homogeneous_basis(L)` returns the d fundamental prefixes obtained by starting the recurrence from each standard initial state. These vectors span the homogeneous solution subspace inside the L-dimensional prefix space. The basis dimension is d and its ambient dimension is L.

18.3.9 Pell-to-Tribonacci worked bridge

`sequence_coordinates_between_models(values, source, target)` analyzes one prefix in both named systems, builds the exact bridge, and refuses to return if the bridge does not reproduce the target coordinates.

```
from komorebi_math.coordinates import (
    get_recurrence_model,
    sequence_coordinates_between_models,
)
```

```
pell = get_recurrence_model("Pell")
tribonacci = get_recurrence_model("Tribonacci")
values = pell.sequence(8)

source, target, bridge = sequence_coordinates_between_models(
    values,
    pell,
    tribonacci,
)

assert bridge.apply(source.coordinates()) == target.coordinates()
assert target.reconstruct() == values

print("values:", values)
print("Pell coordinates:", source.coordinates())
print("Tribonacci coordinates:", target.coordinates())
```

Output:

```
values: (Fraction(0, 1), Fraction(1, 1), Fraction(2, 1), Fraction(5, 1),
         Fraction(12, 1), Fraction(29, 1), Fraction(70, 1), Fraction(169, 1))
Pell coordinates: (Fraction(0, 1), Fraction(1, 1), Fraction(0, 1),
                   Fraction(0, 1), Fraction(0, 1), Fraction(0, 1),
                   Fraction(0, 1), Fraction(0, 1))
Tribonacci coordinates: (Fraction(0, 1), Fraction(1, 1), Fraction(2, 1),
                         Fraction(2, 1), Fraction(4, 1), Fraction(10, 1),
                         Fraction(24, 1), Fraction(58, 1))
```

The nonzero Tribonacci defects do not mean the bridge failed. They are the coordinates needed to express a homogeneous Pell prefix in a different recurrence system.

A useful check is to reconstruct the raw prefix from each record and compare those reconstructed values. The initial states and defect streams can differ while representing exactly the same data. Preserve the recurrence coefficients with either record so that the meaning of its coordinates remains clear.

18.4 Companion and state-space realizations

18.4.1 Boundary with Section 16.9

Section 16.9 is the canonical implementation page for the scalar realization layer. It already documents

- `LinearRealization`,
- scalar `observability_matrix`,
- `recover_initial_state` and `StateRecovery`,
- `cyclic_krylov_matrix`,
- `intertwiner_space` and `IntertwinerSpace`,
- `similarity_from_cyclic_vectors`, and
- `realization_defect`.

It also records two important edges: scalar recovery should use a positive horizon, because horizon zero can return a degenerate empty result, and `IntertwinerSpace.contains_invertible_basis_element()` tests only the displayed basis matrices rather than every linear combination in their span.

Those signatures and examples are not repeated here. Their conclusions form the bridge from recurrence coordinates to controlled input-output systems. In that bridge,

$$a_n = h^T A^n v$$

is the no-input, scalar-output specialization, while the state-space layer adds input and feedthrough matrices:

$$x_{k+1} = Ax_k + Bu_k, \qquad y_k = Cx_k + Du_k.$$

The recurrence companion system from Section 18.3 fits the same picture. Its defect stream enters through the last standard basis vector:

$$x_{n+1} = C_c x_n + e_d \delta_n, \qquad a_n = e_1^T x_n.$$

Refer to Section 16.9 for exact state recovery, Krylov similarity, intertwiner-space construction, and realization-defect execution. The remaining subsections document the separate `StateSpaceModel`, controllability, block observability, coordinate transport, and Markov parameter APIs.

18.4.2 Controlled state-space models

`StateSpaceModel` represents the discrete-time exact system

$$x_{k+1} = Ax_k + Bu_k, \qquad y_k = Cx_k + Du_k.$$

The constructor is

```
StateSpaceModel(
    name: str,
    transition: Matrix,
    input_matrix: Matrix,
    output_matrix: Matrix,
    feedthrough: Matrix,
) -> None
```

If A is $d \times d$, then B must have d rows, C must have d columns, and D must have the same output count as C and input count as B. The properties `.state_dimension`, `.input_dimension`, and `.output_dimension` expose those sizes.

For a single input and output, use

```
siso_state_space(
    transition: Iterable[Iterable[Scalar]],
    input_vector: Sequence[Scalar],
    output_covector: Sequence[Scalar],
    *,
    feedthrough: Scalar = 0,
    name: str = "SISO state-space model",
) -> StateSpaceModel
```

It converts a length-d input vector to a $d \times 1$ matrix, a length-d output vector to a $1 \times d$ matrix, and the scalar feedthrough to a 1×1 matrix.

18.4.3 State coordinate changes

Let `transform.matrix` be T and let target coordinates satisfy

$$z = Tx.$$

`model.change_coordinates(transform)` returns

$$A' = TAT^{-1}, \qquad B' = TB, \qquad C' = CT^{-1}, \qquad D' = D.$$

Its signature is

```
model.change_coordinates(
    transform: CoordinateTransform,
    *,
    name: str | None = None,
) -> StateSpaceModel
```

The transform must be square and match the state dimension. Exact inversion then rejects a singular matrix. Coordinate-system names label the transform, but the method does not require a particular literal source name. Dimensional and algebraic validity are the operative checks.

18.4.4 Markov parameters and transfer invariance

The exact Markov parameters are

$$H_0 = D, \qquad H_k = CA^{k-1}B \quad (k \geq 1).$$

They are returned by

```
model.markov_parameter(index: int) -> Matrix
```

and compared by

```
model.transfer_equivalent(
    other: StateSpaceModel,
    *,
    count: int | None = None,
) -> bool
```

The comparison first requires equal input and output dimensions. With no explicit count, it compares

$$2\max(d_{\text{source}}, d_{\text{target}}) + 1$$

parameters, beginning at H_0. This is an exact finite comparison. It is not an approximate frequency-response test.

The coordinate-change factory is

```
certify_state_coordinate_change(
    model: StateSpaceModel,
    transform: CoordinateTransform,
    *,
    markov_count: int | None = None,
) -> StateSpaceCoordinateCertificate
```

Its default count is $2d+1$. The record stores source model, transformed model, transform, comparison count, and the Boolean result. Calling `.certificate()` produces exact theorem metadata `CC.R3` with checked assumptions `"invertible state transform"` and `"exact Markov parameters"`.

Use the factory rather than manually constructing a `StateSpaceCoordinateCertificate`: the dataclass accepts a caller-supplied `valid` field, whereas the factory computes it from the two models.

18.4.5 Controllability coordinates

For a requested number of steps s, the finite controllability matrix is

$$\mathcal{C}_s = [B, AB, \ldots, A^{s-1}B].$$

The public functions are

```
controllability_matrix(
    transition: Iterable[Iterable[Scalar]],
    input_matrix: Iterable[Iterable[Scalar]],
    *,
    steps: int | None = None,
) -> Matrix

controllability_basis(model: StateSpaceModel) -> FiniteBasis

controllability_coordinate_transform(
    model: StateSpaceModel,
) -> CoordinateTransform
```

When `steps` is omitted, the matrix uses $s = d$. The basis factory scans controllability columns in their generated order and retains each column that increases exact rank. It therefore returns a maximal independent reachable basis when at least one reachable direction exists, even when the system is not fully controllable. A completely zero reachable subspace cannot currently be represented by `FiniteBasis`, whose contract requires at least one vector. That edge case raises `DimensionError`.

`controllability_coordinate_transform` requires that basis to be square. If its positive dimension is less than d, it raises `ValidationError("model is not controllable")`. For a controllable model, it returns the inverse of the basis synthesis matrix, mapping original state coordinates to controllability coordinates.

18.4.6 Observability coordinates for multiple outputs

For p outputs and s steps, the block observability matrix is

$$\mathcal{O}_s = \begin{pmatrix} C \\ CA \\ \vdots \\ CA^{s-1} \end{pmatrix}.$$

The functions are

```
observability_block_matrix(
    transition: Iterable[Iterable[Scalar]],
    output_matrix: Iterable[Iterable[Scalar]],
    *,
    steps: int | None = None,
) -> Matrix

observability_coordinate_transform(
    model: StateSpaceModel,
) -> CoordinateTransform
```

With omitted `steps`, the block matrix uses d steps. If it has more than d rows, the transform factory scans rows in order and retains the first rows that increase exact rank. It then requires exactly d independent rows. A rank-deficient model raises `ValidationError("model is not observable")`.

The target coordinate vector is a selected output jet. It is an exact state coordinate system only when the selected observability matrix is square and invertible.

18.4.7 Worked state-space coordinate change

The shipped example uses

$$A = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 2 & -1 & 3 \end{pmatrix}, \quad B = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}, \quad C = \begin{pmatrix} 1 & 2 & -1 \end{pmatrix}, \quad D = \begin{pmatrix} 2 \end{pmatrix}.$$

```
from fractions import Fraction

from komorebi_math.coordinates import (
    certify_state_coordinate_change,
    controllability_coordinate_transform,
    observability_coordinate_transform,
    siso_state_space,
)

model = siso_state_space(
    ((0, 1, 0), (0, 0, 1), (2, -1, 3)),
    (0, 0, 1),
    (1, 2, -1),
    feedthrough=2,
    name="third-order model",
)

controllability = controllability_coordinate_transform(model)
certificate = certify_state_coordinate_change(model, controllability)
certificate.certificate().require()

transformed = model.change_coordinates(controllability)
assert model.transfer_equivalent(transformed, count=12)

observability = observability_coordinate_transform(model)
state = (Fraction(2), Fraction(-3), Fraction(5))
observable_state = observability.apply(state)
assert observability.inverse().apply(observable_state) == state

print("controllability transform:", controllability.matrix)
print("observable state:", observable_state)
print("Markov count:", certificate.markov_count)
print("valid:", certificate.valid)
```

Output:

```
controllability transform:
((Fraction(1, 1), Fraction(-3, 1), Fraction(1, 1)),
 (Fraction(-3, 1), Fraction(1, 1), Fraction(0, 1)),
 (Fraction(1, 1), Fraction(0, 1), Fraction(0, 1)))
observable state: (Fraction(-9, 1), Fraction(-15, 1), Fraction(-6, 1))
Markov count: 7
valid: True
```

The first Markov parameters of the original model are

$$2, -1, -1, -1, -4, -13, -37.$$

The transformed model produces the same exact sequence. This is the relevant invariant. The entries of A, B, C are expected to change.

18.5 Euler-to-Fibonacci worked path

18.5.1 Signed secant Euler numbers

The signed secant Euler numbers in this module are defined by

$$\operatorname{sech}(t) = \sum_{n \geq 0} E_n \frac{t^n}{n!}.$$

The finite helper

```
signed_secant_euler_numbers(count: int) -> tuple[int, ...]
```

uses $E_0 = 1$ and the coefficient recurrence obtained from $\cosh(t)\operatorname{sech}(t) = 1$:

$$E_n = -\sum_{\substack{2 \leq k \leq n \\ k \text{ even}}} \binom{n}{k} E_{n-k}.$$

Consequently,

$$(E_0, E_1, E_2, E_3, E_4, E_5, E_6, E_7, \ldots) = (1, 0, -1, 0, 5, 0, -61, 0, \ldots).$$

This convention includes odd entries explicitly as zeros. It should not be confused with an indexing convention that stores only the nonzero even secant numbers.

18.5.2 Fibonacci defect coordinates

The convenience function

```
euler_fibonacci_coordinates(count: int) -> RecurrenceCoordinateRecord
```

analyzes the first `count` signed secant Euler numbers in the Fibonacci companion recurrence

$$a_{n+2} = a_n + a_{n+1} + \delta_n.$$

It requires `count >= 2`. The first two Euler entries become the initial state

$$(a_0, a_1) = (1, 0),$$

and the defects are

$$\delta_n = E_{n+2} - E_{n+1} - E_n.$$

Their exponential generating function is

$$\sum_{n \geq 0} \delta_n \frac{t^n}{n!} = \operatorname{sech}(t)\tanh(t) - 2\operatorname{sech}^3(t).$$

The identity follows from

$$\operatorname{sech}'(t) = -\operatorname{sech}(t)\tanh(t),$$

and

$$\operatorname{sech}''(t) = \operatorname{sech}(t) - 2\operatorname{sech}^3(t).$$

The complete executed path is:

```
from komorebi_math.coordinates import (
    euler_fibonacci_coordinates,
    variation_of_constants_prefix,
)

record = euler_fibonacci_coordinates(18)
record.certificate().require()

direct = record.reconstruct()
green = variation_of_constants_prefix(record)

assert direct == green

print("initial", tuple(str(value) for value in record.initial))
print("defects", tuple(str(value) for value in record.defects))
print("reconstructed", tuple(str(value) for value in direct))
```

Output:

```
initial ('1', '0')
defects ('-2', '1', '6', '-5', '-66', '61', '1446', '-1385',
         '-51906', '50521', '2753286', '-2702765', '-202063746',
         '199360981', '19590873126', '-19391512145')
reconstructed ('1', '0', '-1', '0', '5', '0', '-61', '0', '1385',
               '0', '-50521', '0', '2702765', '0', '-199360981',
               '0', '19391512145', '0')
```

The same finite prefix has two exact decompositions that agree:

1. direct forced recurrence synthesis, and
2. homogeneous Fibonacci evolution plus convolution with the companion Green weights.

The nonzero defect sequence proves that the signed Euler prefix is not an unforced Fibonacci prefix. It does not imply that the coordinate representation is approximate.

18.5.3 Command-line path

The dedicated command is

```
python -m komorebi_math coordinates euler-fibonacci --count 18
```

It emits JSON fields for system name, count, initial state, defects, reconstruction, homogeneous flag, and the defect generating function. Before printing, the command requires the record certificate and independently reconstructs by variation of constants.

The named recurrence catalog and bridge are available through

```
python -m komorebi_math coordinates catalog
python -m komorebi_math coordinates model --source pell --target tribonacci --count 16
```

There is no separate state-space CLI subcommand in `1.0.0b4`. Use the Python API for controllability, observability, Markov parameters, and transfer certificates.

18.5.4 Portable persistence

The chapter's exact executable records use the portable format described in Section 16.5. The relevant version-one codec tags are:

Type	Portable tag
`CoordinateTransform`	`coordinates.transform`
`RecurrenceCoordinateRecord`	`coordinates.recurrence_record`
`RecurrenceModel`	`coordinates.recurrence_model`
`LinearRealization`	`coordinates.linear_realization`
`StateRecovery`	`coordinates.state_recovery`
`IntertwinerSpace`	`coordinates.intertwiner_space`
`StateSpaceModel`	`coordinates.state_space_model`

The specialized recurrence-record calls are

```
save_coordinate_record(
    path: str | Path,
    record: RecurrenceCoordinateRecord,
    *,
    overwrite: bool = False,
) -> Path

load_coordinate_record(
    path: str | Path,
) -> RecurrenceCoordinateRecord
```

The generic calls are

```
save_coordinate_object(
    path: str | Path,
    value: Any,
    *,
    overwrite: bool = False,
) -> Path

load_coordinate_object(path: str | Path) -> Any
```

A recurrence-record round trip is:

```
from pathlib import Path

from komorebi_math.coordinates import (
    euler_fibonacci_coordinates,
    load_coordinate_record,
    save_coordinate_record,
)

path = Path("euler_fibonacci_coordinates.json")
record = euler_fibonacci_coordinates(18)

save_coordinate_record(path, record)
restored = load_coordinate_record(path)

assert restored == record
restored.certificate().require()
```

Saving is atomic. By default, an existing destination is not overwritten. The JSON envelope has exactly five top-level fields:

```
schema
schema_version
producer_version
payload_sha256
payload
```

The payload is hashed after canonical JSON encoding. Loading verifies the digest before decoding. Duplicate JSON keys, nonfinite numbers, an unknown schema, an unknown codec tag, unsupported future versions, unexpected data fields, and a SHA-256 mismatch are rejected.

The recurrence-record decoder additionally reconstructs and requires its exact certificate. A restored `StateRecovery` is self-contained over its stored state, prediction, and residual, so its zero-residual certificate takes no arguments. It cannot independently rederive those fields from the original transition, output, and prefix because those inputs are not stored. A restored `IntertwinerSpace` stores no source or target transitions. Certify it with `.certificate(source, target)` before relying on its matrix equations.

Combinatorial matrices can be saved with `save_transform` or the generic `save_coordinate_object`. Persistence stores the materialized exact matrix. It does not regenerate the matrix from `degree`, `rank`, or `size`.

18.5.5 Default resource contract

The default budget is

```
CoordinateBudget(
    maximum_matrix_dimension=512,
    maximum_polynomial_degree=512,
    maximum_horizon=4096,
    maximum_exact_entries=1_000_000,
)
```

It is checked before the budget-aware constructors allocate their exact matrices.

Operation	Relevant default limit
Degree-N Stirling or Lah connection	$N + 1 \leq 512$, hence $N \leq 511$
Size-N binomial, difference, or divisor matrix	$N \leq 512$
Rank-r subset zeta matrix	$2^r \leq 512$, hence $r \leq 9$
Dense recurrence analysis or synthesis matrix	$L \leq 512$
Scalar realization observability matrix	Effective default $H \leq 512$ because the matrix-dimension limit binds before the separate 4096 horizon cap

The direct `analyze_recurrence_coordinates` and `synthesize_recurrence_coordinates` functions are not dense matrix constructors. They can represent prefixes longer than 512, subject to ordinary available memory and integer-growth costs.

The controlled `StateSpaceModel` functions do not currently take a `CoordinateBudget`. A caller constructing large exact state models must impose its own dimension and horizon limits.

18.5.6 Complexity and scaling

These bounds describe the current implementation, not the best known algorithm for each mathematical transform. They count exact arithmetic operations and omit coefficient bit complexity.

Operation	Time	Materialized space	Implementation note
Binomial or forward-difference construction, size N	$O(N^2)$	$O(N^2)$	Dense triangular matrix
Stirling connection through degree N	Up to $O(N^4)$	$O(N^2)$	Each matrix entry independently recomputes a dynamic-programming row
Lah connection through degree N	$O(N^2)$ scalar formula calls	$O(N^2)$	Dense exact matrix
Divisor zeta construction, size N	At least $O(N^2)$ dense scan	$O(N^2)$	Does not use a sparse divisor representation
Subset zeta, rank r	$O(4^r)$	$O(4^r)$	Matrix dimension 2^r
Applying any dense size-N transform	$O(N^2)$	$O(N)$ output	Sparsity is not specialized by `CoordinateTransform.apply`
Direct recurrence analysis or synthesis	$O(Ld)$	$O(L)$	d is recurrence order
Dense recurrence analysis construction	$O(L^2)$ allocation	$O(L^2)$	Only $O(Ld)$ entries are nonzero
Dense recurrence inverse or bridge	$O(L^3)$	$O(L^2)$	Generic exact row reduction and matrix multiplication
Green weights, count M	$O(Md^2)$	$O(M + d^2)$	Repeated companion matrix-vector multiplication
Current variation-of-constants reconstruction	$O(L^2 + Ld^2)$	$O(L + d^2)$	Includes L companion Green-weight steps plus the direct defect convolution
`RecurrenceModel.sequence(count)`	$O(Ld)$	$O(L)$	Direct recurrence sweep
`LinearRealization.term(n)`	$O(d^3 \log n)$	$O(d^2)$	Exact matrix exponentiation
Current `LinearRealization.prefix(L)`	Sum of independent term costs	$O(L + d^2)$	Does not reuse the preceding state
Scalar observability construction	$O(Hd^3)$	$O(Hd)$	Iteratively multiplies a full matrix by A
State recovery	$O(Hd^3)$ observability construction plus elimination on an $H \times d$ system	$O(Hd + d^2)$	Current observability path multiplies full $d \times d$ matrices before exact row reduction
Cyclic Krylov matrix	$O(d^3)$	$O(d^2)$	d matrix-vector steps
Intertwiner space	Dense nullspace in $d_t d_s$ variables	$O((d_t d_s)^2)$ matrix	Cubic elimination in the variable count
One Markov parameter H_k	$O(d^3 \log k)$ plus input/output products	$O(d^2)$	Recomputes A^{k-1}

Exact rational coefficient sizes can grow substantially under inversion, matrix powers, and recurrence evolution. Runtime may therefore grow faster than the arithmetic-operation count suggests.

There are no optional SymPy, SciPy, JAX, Torch, or CUDA dependencies in the direct combinatorial, recurrence, realization, or exact state-space modules. The generic persistence layer imports NumPy because the wider portable registry also supports numerical arrays.

18.5.7 Failure contracts and edge cases

The principal controlled failures are:

Situation	Failure behavior
Negative or Boolean Stirling/Lah index	`ValidationError`
Nonpositive transform size	`DimensionError`
Nonpositive Möbius argument	`ValidationError`
Sequence length differs from transform source dimension	`DimensionError`
Dense matrix exceeds dimension or entry budget	`ResourceLimitError` before allocation
Empty recurrence coefficient vector	`ValidationError` or `DimensionError` at the model layer
Prefix shorter than recurrence order	`DimensionError`
Initial state length differs from recurrence order	`DimensionError`
Empty recurrence name	`ValidationError`
Euler-to-Fibonacci count below two	`DimensionError`
Named model lookup fails	`ValidationError`
Model has no complete initial state and `.sequence()` receives none	`DimensionError`
`RecurrenceModel.synthesize()` in the recorded `1.0.0b4` baseline	Confirmed unintended `TypeError`. Use the lower-level record path for that baseline
Realization transition is empty or nonsquare	`DimensionError`
Realization state or output has wrong length	`DimensionError`
Negative realization index or count	`ValidationError`
Scalar state recovery or observability called with horizon zero	Uncovered edge can return a degenerate empty result. Require $H \geq 1$ in caller code
Observability horizon exceeds the budget	`ResourceLimitError`
Source or target transition for an intertwiner is nonsquare	`DimensionError`
Similarity matrices have unequal shape	`DimensionError`
Selected similarity vector is noncyclic	`ValidationError`
State-space A, B, C, D shapes disagree	`DimensionError`
State coordinate transform has wrong dimension	`DimensionError`
State coordinate transform is singular	`ValidationError` during exact inversion
Controllability steps, observability steps, Markov index, or comparison count is invalid	`ValidationError`
Full controllability coordinates requested with positive but deficient reachable rank	`ValidationError`
Controllability basis requested for a zero reachable subspace	`DimensionError`, because `FiniteBasis` is nonempty by contract
Full output-jet coordinates requested for an unobservable model	`ValidationError`
Portable destination exists and `overwrite=False`	`ValidationError`
Portable payload is altered	`ValidationError` reporting SHA-256 or integrity mismatch

Passing Python floating-point values lies outside the declared `Scalar` type, which is `int | Fraction`. Although `Fraction` can convert some other runtime objects, callers seeking the chapter's exact rational semantics should create `Fraction` values explicitly rather than pass binary floats.

18.5.8 Verification evidence

The recorded Chapter 18 audit executed its source baseline rather than relying only on stored reports.

Check	Result
Focused combinatorial tests	Pass
Focused recurrence-coordinate tests	Pass
Named recurrence and state-space tests	Pass

Check	Result
Realization, state-recovery, Krylov, and intertwiner tests	Pass
General coordinate persistence tests	Pass
Normalized recurrence-record persistence tests	Pass
Coordinate and coordinate-calculus CLI/API tests	Pass
Unique focused tests across those files	**26 passed**
`python -m komorebi_math verify coordinates`	**24/24 passed**
`recurrence_state_coordinate_atlas.py`	Executed successfully
`coordinate_calculus.py`	Executed successfully
`euler_to_fibonacci_coordinates.py`	Executed successfully
Direct `RecurrenceModel.synthesize()` probe	**Failed with the disclosed TypeError**

The stored randomized coordinate-calculus audit records additional historical evidence:

Audit category	Checks	Failures
Combinatorial inverse identities	23,450	0
General recurrence-coordinate comparisons	15,732	0
Recurrence coordinate round trips	72,225	0
Companion variation of constants	72,225	0
Cross-recurrence coordinate changes	9,396	0
Unit-triangular recurrence certificates	5,175	0

Those stored counts are provenance from the recorded audit run. They are not relabeled as a fresh execution. The focused tests and 24-check coordinate verifier describe the source baseline used for the recorded chapter audit.

The focused suite's passing result did not erase the convenience-method failure found by the separate probe. Both outcomes belong to the recorded baseline evidence, which is why the erratum appears in Section 18.3.

18.5.9 Theorem metadata and interpretation

The combinatorial constructors use identifiers `CC.C1` through `CC.C13`. Recurrence analysis and cross-system change use `CC.R1` and `CC.R2`. State recovery and intertwiners use `CC.S2` and `CC.S3`, while the state-space coordinate certificate uses `CC.R3`.

Some state coordinate helpers reuse the short labels `CC.R1` and `CC.R2` for controllability and observability transforms. Treat these strings as local implementation metadata, not as globally unique theorem database keys. The object type, source and target names, description, and certificate diagnostics are needed to identify the actual claim.

> **Chapter boundary.** Chapter 18 proves exact finite coordinate identities, exact reconstruction, and exact transfer preservation under the stated hypotheses. It does not infer an infinite recurrence from finite samples, prove minimality from one finite realization, or equate state-coordinate similarity with equality of internal representations. Defects, residuals, noncontrollability, and nonobservability are exposed as mathematical data or controlled failures rather than hidden by an approximate fallback.

19. Spectral, packet, tensor, and operator transport

Choose by the data you need. Section 19.1 provides explicit small spectral matrices. Section 19.2 reorders indices, with order-only helpers when you do not need a dense matrix. Sections 19.3 to 19.4 transport tensors and operators through a declared coordinate change. In every case, compare the represented vector, pairing, or operator action when checking a result; the new coordinate entries are expected to differ.

Finite data can move among four coordinate views without changing the mathematical object represented by that data. Spectral transforms replace samples by Fourier, cosine, Walsh, or Haar coefficients. Packet permutations replace natural index order by a structural order. Tensor transport extends one coordinate change to covectors, forms, operators, and higher tensors. Operator conjugation closes the loop by expressing the same linear map in a new basis.

The implementation has two distinct arithmetic layers:

Layer	Main type	Scalar model	Typical use
numerical spectral	`NumericCoordinateTransform`	NumPy real or complex arrays	Fourier, DCT, Walsh, Haar, and modal coordinates
exact structural	`CoordinateTransform`	immutable `Fraction` matrices	index permutations and tensor or operator transport

Chapter 16 defines the general construction, composition, inversion, certificate, and persistence contracts for these two transform types. This chapter does not repeat that material. It specializes those contracts to the spectral and tensor families and records the conventions that determine every matrix entry.

The recommended public import surface for the complete chapter is `komorebi_math.coordinates`:

```
from komorebi_math.coordinates import (
    NumericCoordinateTransform,
    ModalCoordinateSystem,
    chinese_remainder_transform,
    coordinate_pairing,
    digit_reversal_transform,
    direct_sum_transform,
    discrete_cosine_transform,
    discrete_fourier_transform,
    haar_wavelet_transform,
    modal_coordinate_system,
    pairing_invariant,
    residue_class_transform,
    sorted_coordinate_transform,
    tensor_axis_permutation_transform,
    tensor_coordinate_transform,
    tensor_product_transform,
    transport_bilinear_form,
    transport_covector,
    transport_operator,
    transport_rectangular_operator,
    transport_tensor_components,
    walsh_hadamard_transform,
)
```

Only a smaller convenience subset is re-exported at package top level. Code that uses the full Chapter 19 surface should import from `komorebi_math.coordinates` so that the identity of every function is clear.

19.1 Fourier, cosine, Hadamard, and Haar coordinates

All four constructors return a dense `NumericCoordinateTransform`. They build an explicit matrix and use ordinary dense matrix-vector multiplication on application. They are coordinate constructors, not fast FFT, fast cosine, FWHT, or lifting-scheme executors.

19.1.1 Constructor reference

```
discrete_fourier_transform(
    size: int,
    *,
    unitary: bool = True,
    inverse: bool = False,
    tolerance: float = 1e-12,
) -> NumericCoordinateTransform

discrete_cosine_transform(
    size: int,
    *,
    orthonormal: bool = True,
    tolerance: float = 1e-12,
) -> NumericCoordinateTransform

walsh_hadamard_transform(
    size: int,
    *,
    orthonormal: bool = True,
    tolerance: float = 1e-12,
) -> NumericCoordinateTransform

haar_wavelet_transform(
    size: int,
    *,
    orthonormal: bool = True,
    tolerance: float = 1e-12,
) -> NumericCoordinateTransform

modal_coordinate_system(
    operator: Any,
    *,
    state_name: str = "state coordinates",
    modal_name: str = "modal coordinates",
    tolerance: float = 1e-10,
) -> ModalCoordinateSystem
```

Constructor	Theorem identifier	Admissible size	Default normalization	Output direction
`discrete_fourier_transform`	`CC.S1`	every $N \geq 1$	unitary	samples to Fourier coefficients
`discrete_cosine_transform`	`CC.S2`	every $N \geq 1$	orthonormal DCT-II	samples to cosine coefficients
`walsh_hadamard_transform`	`CC.S3`	positive powers of two	orthonormal	samples to Sylvester Walsh coefficients
`haar_wavelet_transform`	`CC.S4`	positive powers of two	orthonormal	samples to complete Haar coefficients

Constructor	Theorem identifier	Admissible size	Default normalization	Output direction
`modal_coordinate_system`	`CC.S5`	finite square matrix with complete numerical eigenbasis	not applicable	state and modal coordinates in both directions

The returned transforms expose the Chapter 16 numerical interface:

```
transform.apply(coordinates: Any) -> np.ndarray
transform.inverse() -> NumericCoordinateTransform
transform.then(following: NumericCoordinateTransform) -> NumericCoordinateTransform
transform.condition_number() -> float
transform.certificate() -> Certificate
```

The source and target names are part of composition validation. Two matrices with compatible shapes do not compose through `then` unless the first target name exactly equals the second source name.

19.1.2 Discrete Fourier convention

For $N =$ `size`, the default forward matrix is

$$F_{kn} = \frac{1}{\sqrt{N}} \exp\left(-\frac{2\pi i k n}{N}\right), \qquad 0 \le k, n < N.$$

Thus

$$\hat{x}_k = \sum_{n=0}^{N-1} F_{kn} x_n, \qquad F^* F = F F^* = I_N, \qquad \|\hat{x}\|_2 = \|x\|_2.$$

`inverse=True` reverses the sign in the exponential. With the default `unitary=True`, both directions retain the factor $N^{-1/2}$. With `unitary=False`, the forward matrix is unscaled and the inverse constructor uses the factor N^{-1}:

$$(F_0)_{kn} = e^{-2\pi i k n/N}, \qquad (F_0^{-1})_{nk} = \frac{1}{N} e^{2\pi i k n/N}.$$

The `inverse` flag constructs the analytic inverse convention. The inherited `.inverse()` method instead numerically inverts whatever dense matrix is stored. Both routes agree to floating precision for the standard Fourier matrices.

19.1.3 DCT-II convention

The cosine constructor forms

$$C_{kn} = \alpha_k \cos\left(\frac{\pi(n + \frac{1}{2})k}{N}\right),$$

where

$$\alpha_0 = \sqrt{\frac{1}{N}}, \qquad \alpha_k = \sqrt{\frac{2}{N}} \quad (k > 0)$$

when `orthonormal=True`. This gives $CC^T = I_N$. The `orthonormal=False` matrix omits every α_k. There is no separate DCT-III constructor in this API. Call `.inverse()` when the inverse of the constructed matrix is required.

19.1.4 Sylvester Walsh-Hadamard convention

The unnormalized matrices are generated recursively:

$$H_1 = (1), \qquad H_{2N} = \begin{pmatrix} H_N & H_N \\ H_N & -H_N \end{pmatrix}.$$

The default constructor returns $N^{-1/2} H_N$, so its matrix is orthogonal and self-inverse. The row order is the recursive Sylvester order. No Gray-code or sequency reordering is applied.

19.1.5 Complete finite Haar convention

At each active dyadic level the Haar constructor replaces adjacent pairs by a sum and a difference. In the orthonormal case,

$$a_j = \frac{x_{2j} + x_{2j+1}}{\sqrt{2}}, \qquad d_j = \frac{x_{2j} - x_{2j+1}}{\sqrt{2}}.$$

It recurses on the averages until one global coefficient remains. For length four, the explicit matrix is

$$\begin{pmatrix} \frac{1}{2} & \frac{1}{2} & \frac{1}{2} & \frac{1}{2} \\ \frac{1}{2} & \frac{1}{2} & -\frac{1}{2} & -\frac{1}{2} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} & 0 & 0 \\ 0 & 0 & \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \end{pmatrix}.$$

The coefficient order is the global average, followed by the coarsest detail, then successively finer details. Setting `orthonormal=False` replaces every pair scale by one. In that mode the transform remains invertible, but it does not preserve Euclidean energy.

19.1.6 First runnable spectral example

This is the shipped `examples/spectral_packet_coordinates.py` path, reduced to the two spectral transforms:

```
import numpy as np

from komorebi_math.coordinates import (
    discrete_fourier_transform,
    haar_wavelet_transform,
)

samples = np.arange(8, dtype=float)
fourier = discrete_fourier_transform(8)
haar = haar_wavelet_transform(8)

fourier.certificate().require()
haar.certificate().require()

f = fourier.apply(samples)
h = haar.apply(samples)

assert np.allclose(fourier.inverse().apply(f), samples)
assert np.allclose(haar.inverse().apply(h), samples)

print(np.round(f, 8))
print(np.round(h, 8))
```

Output:

```
[ 9.89949494+0.j         -1.41421356+3.41421356j
 -1.41421356+1.41421356j -1.41421356+0.58578644j
 -1.41421356-0.j         -1.41421356-0.58578644j
 -1.41421356-1.41421356j -1.41421356-3.41421356j]
[ 9.89949494 -5.65685425 -2.         -2.
 -0.70710678 -0.70710678 -0.70710678 -0.70710678]
```

The leading value in both arrays is $\sum_{n=0}^{7} n/\sqrt{8} = 28/\sqrt{8}$. The remaining coefficients encode different orthonormal decompositions of the same sample vector.

19.1.7 Modal coordinates

`modal_coordinate_system(A)` uses `np.linalg.eig` and writes the returned eigenvectors as columns of V. Its two transforms are

$$\texttt{synthesis} : z \longmapsto Vz, \qquad \texttt{analysis} : x \longmapsto V^{-1}x.$$

The represented identity is

$$V^{-1}AV = \operatorname{diag}(\lambda_0, \dots, \lambda_{N-1}) + E,$$

with recorded residual

$$\delta_{\text{modal}} = \left\| V^{-1}AV - \operatorname{diag}(\lambda) \right\|_{\infty}.$$

The result record is:

```python
@dataclass(frozen=True, slots=True)
class ModalCoordinateSystem:
    eigenvalues: np.ndarray
    synthesis: NumericCoordinateTransform
    analysis: NumericCoordinateTransform
    diagonal_residual: float

    def certificate(self) -> Certificate: ...
```

A minimal execution is:

```python
import numpy as np
from komorebi_math.coordinates import modal_coordinate_system

A = np.array([
    [2.0, 1.0, 0.0],
    [0.0, 3.0, 1.0],
    [0.0, 0.0, 5.0],
])
modal = modal_coordinate_system(A)
modal.certificate().require()

D = modal.analysis.matrix @ A @ modal.synthesis.matrix
print(np.round(modal.eigenvalues, 12))
print(np.round(D, 12))
print(modal.diagonal_residual)
```

The eigenvalues are $2, 3, 5$, and `D` is diagonal to the declared numerical tolerance. Eigenvalue ordering, eigenvector phases, eigenvector scales, and bases inside repeated eigenspaces are not portable identities. They can differ between NumPy and LAPACK builds. Tests and user code should compare the diagonalization residual and the represented invariant subspaces, not literal eigenvector arrays.

The helper rejects an eigenvector matrix whose numerical rank is less than the operator size under `np.linalg.matrix_rank(eigenvectors, tol=tolerance)`. A large condition number is reported and used in certificate scaling, but it is not a separate constructor rejection threshold.

19.1.8 Numerical certificates

For a square numerical transform C, the certificate computes

$$\delta_L = \|C^{-1}C - I\|_\infty, \qquad \delta_R = \|CC^{-1} - I\|_\infty, \qquad \delta = \max(\delta_L, \delta_R).$$

It is valid when

$$\delta \leq \texttt{tolerance}\ \max\{1, \kappa_2(C)\}.$$

Certificate component	Numerical transform value
exactness	`Exactness.NUMERICAL`
residual	`{"inverse": delta}`
tolerance	transform tolerance
assumptions checked	finite matrix and numerical nonsingularity
diagnostics	source dimension, target dimension, condition number
theorem identifier	constructor theorem such as `CC.S1` through `CC.S4`

A finite rectangular `NumericCoordinateTransform` receives a valid shape certificate without an inverse or full-rank claim. A square singular matrix receives an invalid certificate. This differs from the exact certificate boundary discussed in Section 19.4.

The modal certificate uses

$$\delta_{\text{modal}} \leq \texttt{tolerance}\ \max\{1, \kappa_2(V)\}$$

with theorem identifier `CC.S5`.

19.1.9 Spectral CLI

The command-line route is:

```
python -m komorebi_math coordinates spectral --kind haar --size 4 --values 0,1,2,3
```

The four choices are `dft`, `dct`, `hadamard`, and `haar`. The JSON response contains the source and target names, condition number, dense matrix, optional output vector, and `"verified": true`. Complex values use objects with `real` and `imag` fields.

The CLI exposes only the default direction, normalization, and tolerance. Use the Python constructors when an inverse Fourier convention, unnormalized matrix, or custom tolerance is required.

For the command above, the core output values are:

```
condition_number: 1.0000000000000002
output: [2.999999999999999, -1.9999999999999996,
         -0.7071067811865475, -0.7071067811865475]
verified: true
```

19.2 Residue, digit-reversal, and CRT permutations

Packet coordinate functions reorder entries without changing their values. Every transform is an exact permutation matrix P satisfying

$$(Px)_i = x_{\pi(i)}, \qquad P^{-1} = P^T, \qquad \det P \in \{-1, 1\}.$$

The orientation matters: the stored order tells which source entry becomes each target entry. It does not tell the destination of each source entry.

19.2.1 API reference

```
residue_class_order(length: int, modulus: int) -> tuple[int, ...]
residue_class_transform(length: int, modulus: int) -> CoordinateTransform

digit_reversal_order(length: int, radix: int = 2) -> tuple[int, ...]
digit_reversal_transform(length: int, radix: int = 2) -> CoordinateTransform

tensor_axis_permutation_order(
    shape: Sequence[int],
    axes: Sequence[int],
) -> tuple[int, ...]

tensor_axis_permutation_transform(
    shape: Sequence[int],
    axes: Sequence[int],
) -> CoordinateTransform

chinese_remainder_order(moduli: Sequence[int]) -> tuple[int, ...]
chinese_remainder_transform(moduli: Sequence[int]) -> CoordinateTransform

sorted_coordinate_transform(
    length: int,
    key: Callable[[int], object],
    *,
    source: str = "original index order",
    target: str = "key-sorted index order",
) -> CoordinateTransform
```

Each `*_order` function returns only the integer permutation. Each transform constructor passes that order to `permutation_transform`, which builds a dense exact matrix with theorem identifier `CC.B2`.

19.2.2 Residue lanes

For length N and modulus M, the residue order is

$$0, M, 2M, \dots; \quad 1, 1+M, 1+2M, \dots; \quad \dots \quad ; M-1, 2M-1, \dots,$$

with indices at least N omitted. Equivalently,

$$\pi_{M,N} = (r + kM : 0 \le r < M, 0 \le k, r + kM < N).$$

For $N = 12$ and $M = 4$,

```
(0, 4, 8, 1, 5, 9, 2, 6, 10, 3, 7, 11)
```

This is the finite polyphase order used in Chapter 14. The present function only permutes a finite coordinate vector. It does not construct a filter bank, refinement tower, or packet projector family.

An $M > N$ modulus is legal. Some residue lanes are then empty, while the nonempty lanes still cover every index exactly once.

19.2.3 Fixed-width digit reversal

Suppose $N = b^w$. Write

$$i = \sum_{j=0}^{w-1} d_j b^j, \qquad 0 \le d_j < b.$$

The reversed index is

$$\mathrm{rev}_{b,w}(i) = \sum_{j=0}^{w-1} d_j b^{w-1-j}.$$

For $N = 8$ and $b = 2$,

```
natural:       (0, 1, 2, 3, 4, 5, 6, 7)
bit reversed:  (0, 4, 2, 6, 1, 5, 3, 7)
```

The width is determined from `length`. A length that is not an exact power of the radix raises `DimensionError`.

> **Known `1.0.0b4` defect: do not use radix one.** The implementation's positive-integer validator currently accepts `radix=1`. For every `length > 1`, the width-discovery loop then fails to advance and does not terminate. Use `radix >= 2`. A calling application that accepts an external radix must reject one before invoking `digit_reversal_order` or `digit_reversal_transform`. This guide does not treat radix one as a supported degenerate case.

19.2.4 Tensor-axis order

For a tensor with row-major shape

$$(n_0, n_1, \ldots, n_{r-1}),$$

`tensor_axis_permutation_transform(shape, axes)` reproduces

```
array.reshape(shape).transpose(axes).ravel()
```

as one exact flat-coordinate permutation. The `axes` tuple must contain every axis `0` through `r-1` exactly once.

```python
import numpy as np
from komorebi_math.coordinates import tensor_axis_permutation_transform

array = np.arange(24).reshape(2, 3, 4)
change = tensor_axis_permutation_transform((2, 3, 4), (2, 0, 1))

observed = np.array(change.apply(array.ravel())).reshape(4, 2, 3)
expected = np.transpose(array, (2, 0, 1))
assert np.array_equal(observed, expected)
```

The first part of the stored order is

```
(0, 4, 8, 12, 16, 20, 1, 5, 9, 13, 17, 21, ...)
```

19.2.5 Chinese-remainder order

Given pairwise coprime positive moduli $m_1, \dots, m_r$, let

$$M = \prod_{j=1}^{r} m_j.$$

The implementation sorts each $n \in \{0, \dots, M-1\}$ by the key

$$\rho(n) = (n \bmod m_1, \dots, n \bmod m_r)$$

in lexicographic order. The returned values are still the integer indices. They are not residue tuples and are not the coefficients of a symbolic CRT isomorphism.

For moduli 2 and 3,

n	$\rho(n)$
0	$(0, 0)$
4	$(0, 1)$
2	$(0, 2)$
3	$(1, 0)$
1	$(1, 1)$
5	$(1, 2)$

so the order is

```
(0, 4, 2, 3, 1, 5)
```

Noncoprime moduli raise `ValidationError`. A modulus equal to one is permitted as a trivial factor because it is coprime to every integer.

19.2.6 Stable custom sorting

`sorted_coordinate_transform` evaluates `key(index)` once through Python's sorting machinery and materializes the resulting order as an exact transform. Python sorting is stable, so equal keys retain natural index order.

```python
from komorebi_math.coordinates import sorted_coordinate_transform

change = sorted_coordinate_transform(
    8,
    key=lambda index: (index % 3, index),
    source="natural order",
    target="modulo-three order",
)

print(tuple(int(value) for value in change.apply(tuple(range(8)))))
```

Output:

```
(0, 3, 6, 1, 4, 7, 2, 5)
```

The callable is construction logic, not persisted state. Saving the result stores the materialized permutation matrix and its names. Loading it does not reconstruct the Python callable.

19.2.7 Combined packet example

```
from komorebi_math.coordinates import (
    chinese_remainder_order,
    digit_reversal_transform,
    residue_class_transform,
)

x = tuple(range(8))
lanes = residue_class_transform(8, 2)
bits = digit_reversal_transform(8, 2)

lane_values = lanes.apply(x)
bit_values = bits.apply(x)

assert lanes.inverse().apply(lane_values) == x
assert bits.inverse().apply(bit_values) == x

print(tuple(int(value) for value in lane_values))
print(tuple(int(value) for value in bit_values))
print(chinese_remainder_order((2, 3)))
```

Output:

```
(0, 2, 4, 6, 1, 3, 5, 7)
(0, 4, 2, 6, 1, 5, 3, 7)
(0, 4, 2, 3, 1, 5)
```

19.2.8 Packet CLI

The three command-line choices are:

```
python -m komorebi_math coordinates packet --kind residue --length 12 --modulus 4
python -m komorebi_math coordinates packet --kind digit-reversal --length 8 --radix 2
python -m komorebi_math coordinates packet --kind crt --moduli 2,3
```

The response contains the exact matrix as rational strings, the source and target names, and `"verified": true`. The CLI does not apply the matrix to a user vector. Tensor-axis and custom sorted permutations have no Chapter 19 CLI route and must be constructed through Python.

The radix-one defect also applies to the digit-reversal CLI. Do not pass `--radix 1`.

19.3 Tensor coordinate transport

Let T_V map old vector components to new vector components. On a tensor power, a purely contravariant rank-k tensor uses the Kronecker power

$$T_{V^{\otimes k}} = T_V^{\otimes k}.$$

Mixed tensors require one factor per slot. A contravariant slot uses T_V, while a covariant slot uses the contragredient matrix T_V^{-T}.

19.3.1 Exact transport API

```
contragredient_transform(
    transform: CoordinateTransform,
) -> CoordinateTransform

transport_covector(
    covector: Sequence[Scalar],
    transform: CoordinateTransform,
) -> Vector

tensor_coordinate_transform(
    transform: CoordinateTransform,
    variance: Sequence[Literal["contravariant", "covariant"]],
) -> CoordinateTransform

transport_tensor_components(
    components: Sequence[Scalar],
    transform: CoordinateTransform,
    variance: Sequence[Literal["contravariant", "covariant"]],
) -> Vector

tensor_product_transform(
    left: CoordinateTransform,
    right: CoordinateTransform,
) -> CoordinateTransform

direct_sum_transform(
    left: CoordinateTransform,
    right: CoordinateTransform,
) -> CoordinateTransform

coordinate_pairing(
    covector: Sequence[Scalar],
    vector: Sequence[Scalar],
) -> Fraction

pairing_invariant(
    covector: Sequence[Scalar],
    vector: Sequence[Scalar],
    transform: CoordinateTransform,
) -> bool
```

These functions belong to the exact coordinate layer. Inputs are converted to `Fraction` values and outputs are immutable exact tuples. They are not NumPy tensor routines.

19.3.2 Variance convention

Write the vector-coordinate rule as

$$x' = Cx.$$

For the scalar pairing to remain unchanged, a covector must obey

$$\phi' = C^{-T}\phi,$$

because

$$(\phi')^T x' = (C^{-T}\phi)^T (Cx) = \phi^T x.$$

For a variance word

$$\nu = (\nu_1, \dots, \nu_r), \qquad \nu_j \in \{\text{contravariant}, \text{covariant}\},$$

define

$$K_j = \begin{cases} C, & \nu_j = \text{contravariant}, \\ C^{-T}, & \nu_j = \text{covariant}. \end{cases}$$

Then the flattened component transform is

$$T_\nu = K_1 \otimes \cdots \otimes K_r.$$

Axes use the listed variance order. Components are flattened lexicographically, which agrees with row-major flattening for a tensor whose every axis has dimension n.

An empty variance sequence represents a scalar. Its coordinate transform is the 1×1 identity.

Constructor	Theorem identifier	Result
`contragredient_transform`	`CC.T1`	exact transform on covectors
`tensor_coordinate_transform`	`CC.T2`	exact transform on flattened mixed tensors
`tensor_product_transform`	`CC.T3`	Kronecker product of two transform matrices
`direct_sum_transform`	`CC.T4`	block-diagonal sum of two transform matrices

`transport_covector` and `transport_tensor_components` apply the corresponding transform immediately and return a raw exact vector. They do not return a separate certificate.

19.3.3 Exact pairing example

```
from komorebi_math.coordinates import (
    CoordinateTransform,
    coordinate_pairing,
    pairing_invariant,
    transport_covector,
)

C = CoordinateTransform(
    "old",
    "new",
    ((2, 1), (1, 1)),
    "EXAMPLE.T",
)
x = (3, -2)
phi = (5, 7)

x_new = C.apply(x)
phi_new = transport_covector(phi, C)

print(tuple(map(str, x_new)))
print(tuple(map(str, phi_new)))
print(coordinate_pairing(phi, x))
print(coordinate_pairing(phi_new, x_new))
print(pairing_invariant(phi, x, C))
```

Output:

```
('4', '1')
('-2', '9')
1
1
True
```

The equality is exact. No numerical tolerance enters the comparison.

19.3.4 Mixed tensor example

Use the same C and flatten the matrix

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$$

as $(1, 2, 3, 4)$. One contravariant and one covariant axis gives

$$C \otimes C^{-T}.$$

```
from komorebi_math.coordinates import (
    tensor_coordinate_transform,
    transport_tensor_components,
)

variance = ("contravariant", "covariant")
tensor_change = tensor_coordinate_transform(C, variance)
components = (1, 2, 3, 4)

direct = transport_tensor_components(components, C, variance)
through_object = tensor_change.apply(components)

assert direct == through_object
assert tensor_change.roundtrip_valid()
print(tuple(map(str, direct)))
```

Output:

```
('-3', '11', '-2', '8')
```

With the library's flattening convention, this is also the row-major flattening of the transported operator in Section 19.4.

19.3.5 Tensor products and direct sums

For $C : V \to V'$ and $D : W \to W'$,

$$C \otimes D : V \otimes W \longrightarrow V' \otimes W'$$

is built by `tensor_product_transform(C, D)`. Rectangular transforms are allowed because this constructor does not invert either factor.

For $C : V \to V'$ and $D : W \to W'$,

$$C \oplus D = \begin{pmatrix} C & 0 \\ 0 & D \end{pmatrix}$$

is built by `direct_sum_transform(C, D)`. It is also permitted for rectangular factors. Inversion is available later only when the resulting transform is square and nonsingular.

19.3.6 Tensor scale boundary

If C is $n \times n$ and the tensor rank is r, then

$$T_\nu \in \mathbb{Q}^{n^r \times n^r}.$$

The current implementation stores every entry. Its matrix therefore contains

$$n^{2r}$$

exact rational values. Applying the dense transform has the same scalar-entry scale. The function is suitable for small, finite symbolic tensors. It is not a lazy tensor network, sparse Kronecker operator, or implicit contraction engine.

n	r	component dimension n^r	dense matrix entries n^{2r}
2	2	4	16
2	6	64	4,096
3	4	81	6,561
4	5	1,024	1,048,576

There is no `CoordinateBudget` check inside these tensor constructors. The caller must choose a dimension and rank that fit memory and exact-arithmetic cost.

19.4 Operator conjugation

Suppose B_s and B_t are bases of one finite vector space and T maps source coordinate columns to target coordinate columns. The same endomorphism has matrices related by

$$[A]_{B_t} = T[A]_{B_s}T^{-1}.$$

The implementation follows this convention exactly.

19.4.1 Operator, rectangular-map, and form API

```
transport_operator(
    operator: Iterable[Iterable[Scalar]],
    transform: CoordinateTransform,
) -> Matrix

transport_rectangular_operator(
    operator: Iterable[Iterable[Scalar]],
    domain_transform: CoordinateTransform,
    codomain_transform: CoordinateTransform,
) -> Matrix

transport_bilinear_form(
    form: Iterable[Iterable[Scalar]],
    transform: CoordinateTransform,
) -> Matrix
```

For $x' = Cx$, the three rules are

$$A' = CAC^{-1},$$

$$L' = C_{\text{cod}} L C_{\text{dom}}^{-1},$$

and

$$G' = C^{-T} G C^{-1}.$$

The first applies to an endomorphism. The second applies to a map between different domain and codomain spaces. The third applies to a bilinear form $g(x, y) = x^T G y$.

The transport helpers return raw exact matrices. The theorem-bearing object is the coordinate transform C, not the resulting operator matrix. A caller that needs a reusable named map can wrap the result in an appropriate exact record after transport.

19.4.2 Commuting identities

Operator transport is characterized by

$$A'(Cx) = C(Ax).$$

For a rectangular map $L : V \to W$,

$$L'(C_V x) = C_W(Lx).$$

Bilinear-form transport is characterized by

$$(Cx)^T G'(Cy) = x^T G y.$$

These are component identities. The abstract vector, operator, and bilinear form have not changed.

19.4.3 Exact worked conjugation

```
from komorebi_math.coordinates import (
    CoordinateTransform,
    transport_bilinear_form,
    transport_operator,
)

C = CoordinateTransform(
    "old",
    "new",
    ((2, 1), (1, 1)),
    "EXAMPLE.T",
)
A = ((1, 2), (3, 4))
G = ((2, 1), (1, 3))
x = (3, -2)

A_new = transport_operator(A, C)
G_new = transport_bilinear_form(G, C)
x_new = C.apply(x)

Ax = (
    A[0][0] * x[0] + A[0][1] * x[1],
    A[1][0] * x[0] + A[1][1] * x[1],
```

```
)
A_new_x_new = (
    A_new[0][0] * x_new[0] + A_new[0][1] * x_new[1],
    A_new[1][0] * x_new[0] + A_new[1][1] * x_new[1],
)

assert A_new_x_new == C.apply(Ax)

q_old = sum(G[i][j] * x[i] * x[j] for i in range(2) for j in range(2))
q_new = sum(
    G_new[i][j] * x_new[i] * x_new[j]
    for i in range(2)
    for j in range(2)
)
assert q_new == q_old

print(tuple(tuple(map(str, row)) for row in A_new))
print(tuple(tuple(map(str, row)) for row in G_new))
print(q_old, q_new)
```

Output:

```
(('-3', '11'), ('-2', '8'))
(('3', '-5'), ('-5', '10'))
18 18
```

This example simultaneously verifies the operator commuting square and the quadratic-form invariant.

19.4.4 Exactness and certificate boundary

`CoordinateTransform` converts entries through `Fraction`. Integer and `Fraction` inputs therefore remain literal exact rationals. A Python float is also accepted, but `Fraction(float)` represents its exact binary floating value, which can produce a large denominator. Use integers or explicit `Fraction` values when the intended mathematics is rational.

There is one certificate distinction that must remain explicit:

> `CoordinateTransform.certificate()` validates the finite rational matrix and reports its determinant and `invertible` diagnostic. In `1.0.0b4`, a square singular transform can still have `certificate.valid == True`. The certificate is therefore not an invertibility proof.

For an inverse-dependent operation, use one of these stronger paths:

```
assert transform.roundtrip_valid()
inverse = transform.inverse()
```

`roundtrip_valid()` returns `False` for a nonsquare or singular exact transform. `.inverse()` and the tensor or operator helpers raise when exact inversion fails. Exact permutation transforms are always invertible because their matrices contain every coordinate index exactly once.

Numerical behavior is different. As recorded in Section 19.1, a singular square `NumericCoordinateTransform` receives an invalid numerical certificate.

19.4.5 Failure and edge table

Operation	Invalid boundary	Behavior
numerical transform construction	matrix not two dimensional or empty	`DimensionError`
numerical transform construction	matrix has NaN or infinity	`ValidationError`
numerical transform construction	blank coordinate name	`ValidationError`
numerical transform construction	nonpositive or nonfinite tolerance	`ValidationError`

Operation	Invalid boundary	Behavior
numerical `.apply`	input not a finite vector of exact source length	`DimensionError` or `ValidationError`
numerical `.inverse`	rectangular matrix	`DimensionError`
numerical `.inverse`	singular square matrix	`ValidationError`
spectral constructor	Boolean, noninteger, or nonpositive size	`DimensionError`
Walsh or Haar constructor	size not a power of two	`DimensionError`
modal construction	nonsquare operator	`DimensionError`
modal construction	incomplete numerical eigenvector basis	`ValidationError`
packet input	nonpositive or Boolean length, modulus, radix, or shape entry	`ValidationError`
digit reversal	length not an exact radix power	`DimensionError`
digit reversal	`radix=1`, `length>1`	known nontermination defect, reject before call
tensor-axis order	empty shape	`DimensionError`
tensor-axis order	axes repeat or omit an axis	`ValidationError`
CRT order	empty modulus sequence	`DimensionError`
CRT order	moduli not pairwise coprime	`ValidationError`
exact permutation	order does not contain each index once	`ValidationError`
covector or tensor transport	transform not square or is singular	`DimensionError` or `ValidationError`
tensor transport	variance word contains another string	`ValidationError`
tensor application	flattened component length is not n^r	`DimensionError`
pairing	vector and covector lengths differ	`DimensionError`
operator transport	operator and transform dimensions disagree	`DimensionError`
form transport	form not a matching square matrix	`DimensionError`

Direct construction of `ModalCoordinateSystem` is shallow because it is a frozen result record. Use `modal_coordinate_system(...)` as the validated producer.

19.4.6 Persistence

Chapter 16 gives the complete portable-envelope contract. For the present objects, the relevant helpers are:

```
save_transform(
    path: str | Path,
    transform: CoordinateTransform,
    *,
    overwrite: bool = False,
) -> Path

load_transform(path: str | Path) -> CoordinateTransform

save_coordinate_object(
    path: str | Path,
    value: Any,
    *,
    overwrite: bool = False,
) -> Path

load_coordinate_object(path: str | Path) -> Any
```

Persisted object	Codec tag	Codec version	Recommended helper
exact packet or tensor transform	`coordinates.transform`	1	`save_transform` or `save_coordinate_object`

Persisted object	Codec tag	Codec version	Recommended helper
numerical spectral transform	`coordinates.numeric_transform`	1	`save_coordinate_object`
modal coordinate result	`coordinates.modal_system`	1	`save_coordinate_object`

The outer JSON envelope records schema `komorebi-math.portable`, schema version one, producer version, canonical payload SHA-256, and the encoded payload. Complex numbers, exact fractions, and NumPy arrays with dtype and shape are supported. Writes are atomic and refuse to overwrite an existing path unless `overwrite=True`. Loading rejects an altered digest, duplicate JSON keys, nonfinite values, unsupported future versions, and mismatched codec schemas.

```
from pathlib import Path
from tempfile import TemporaryDirectory

import numpy as np

from komorebi_math.coordinates import (
    NumericCoordinateTransform,
    discrete_fourier_transform,
    load_coordinate_object,
    save_coordinate_object,
)

with TemporaryDirectory() as directory:
    path = Path(directory) / "dft4.json"
    original = discrete_fourier_transform(4)
    save_coordinate_object(path, original)
    restored = load_coordinate_object(path)

    assert isinstance(restored, NumericCoordinateTransform)
    assert restored.source == original.source
    assert restored.target == original.target
    assert restored.tolerance == original.tolerance
    assert np.array_equal(restored.matrix, original.matrix)
    print(restored.certificate().valid)
```

Output:

```
True
```

Persistence proves envelope integrity and schema validity. It does not turn a singular transform into an invertible one. Require the appropriate certificate or roundtrip check after loading when later logic needs inversion.

The raw tuple returned by `transport_operator` or `transport_tensor_components` is not accepted by `save_coordinate_object`, whose type set is closed by design. The broader `save_portable` function can store primitive tuple data when a typed coordinate object is unnecessary.

19.4.7 Complexity and resource profile

Let N be a spectral or permutation length, n a square operator degree, r a tensor rank, $m \times n$ a rectangular operator shape, and $M = \prod_j m_j$ a CRT index count.

Construction or operation	Stored representation	Dominant implementation scale
DFT or DCT construction	dense $N \times N$ NumPy matrix	$O(N^2)$ time and memory
Walsh construction	recursively doubled dense matrix	$O(N^2)$ time and memory
current Haar construction	repeated dense $N \times N$ stage products	nominal $O(N^3 \log N)$ construction, $O(N^2)$ memory
numerical transform application	dense matrix-vector product	$O(N^2)$

Construction or operation	Stored representation	Dominant implementation scale
numerical inverse, condition, or certificate	dense linear algebra	$O(N^3)$
modal construction	eigendecomposition, rank, inverse, residual	$O(n^3)$
residue order only	integer tuple	$O(N)$
digit-reversal order only	integer tuple	$O(N \log_b N)$
tensor-axis order only	integer tuple	$O(Nr)$
CRT order only	keys and stable sort	approximately $O(Mr + M \log M)$
any exact permutation transform	dense $N \times N$ Fraction matrix	$O(N^2)$ time and memory
exact permutation `.apply`	generic dense exact multiply	$O(N^2)$
exact permutation `.inverse`	generic exact inversion	$O(N^3)$ plus rational bit cost
exact $n \times n$ operator or form transport	dense exact inverses and products	$O(n^3)$ arithmetic operations plus rational bit growth
exact rectangular transport	two inverses and two dense products	$O(m^3 + n^3 + m^2n + mn^2)$ at the ordinary dense scale
rank-r tensor transport	dense $n^r \times n^r$ Fraction matrix	$O(n^{2r})$ entries
portable save or load	JSON proportional to stored entries	linear in encoded payload size

The logical permutation operations are linear-time reorderings, but the returned exact transforms intentionally use the common dense `CoordinateTransform` representation. For large arrays, call the `*_order` function and apply that order directly rather than materializing the dense exact matrix.

Likewise, applications needing production FFT, DCT, Walsh, or Haar throughput should use a fast signal backend. The Chapter 19 numerical matrices are most useful for small finite examples, matrix inspection, coordinate composition, certificate testing, and reproducible teaching cases.

19.4.8 What the shipped tests establish

A focused replay of the four most relevant modules completed with 13 passes:

```
tests/test_coordinates_general_numeric_packet.py
tests/test_coordinates_general_tensor_modular.py
tests/test_coordinates_general_persistence.py
tests/test_block14_cli_api.py

13 passed
```

The full coordinate verifier also passed all 24 checks. Three named checks directly cover this chapter:

```
[PASS] Fourier, cosine, Hadamard, and Haar coordinates
[PASS] packet, residue, digit, and CRT coordinates
[PASS] tensor, covector, form, and operator transport
verification passed 24/24 checks
```

The exercised cases establish:

- default DFT, DCT-II, Walsh-Hadamard, and Haar roundtrips on finite vectors
- numerical modal diagonalization of a matrix with a complete eigenbasis
- exact residue, bit-reversal, tensor-axis, and CRT permutation roundtrips
- exact vector-covector pairing invariance
- exact operator and bilinear-form transport identities
- a mixed contravariant-covariant tensor roundtrip
- numeric and modal portable persistence, including digest tamper rejection
- the spectral and packet CLI paths

The focused suite does not directly exercise every option or helper. No direct test was found for the nondefault normalization flags, inverse Fourier flag, `sorted_coordinate_transform`, standalone `contragredient_transform`, `tensor_product_transform`, `direct_sum_transform`, or the standalone rectangular transport path. `transport_rectangular_operator` is exercised indirectly by polynomial operator transport elsewhere in the coordinate suite. These functions have implementation contracts, but the guide does not inflate their present test coverage.

Chapter 19 summary. Spectral constructors provide explicit dense numerical matrices for Fourier, DCT-II, Sylvester Walsh, complete Haar, and modal coordinates. Packet constructors provide exact dense permutations for residue lanes, fixed-width digit reversal, tensor axes, CRT order, and stable custom keys. Exact tensor transport follows variance: vectors use C, covectors use C^{-T}, operators use CAC^{-1}, and bilinear forms use $C^{-T}GC^{-1}$. Pairings and represented maps are invariant, but cost is governed by the stored dense matrices. Keep spectral matrices finite, tensor ranks small, require a real invertibility check when inversion matters, and reject radix one before calling the `1.0.0b4` digit-reversal implementation.

20. Analysis-synthesis and redundant frames

First route. Follow the four-coordinate, two-dimensional example in Sections 20.1 to 20.2 and check reconstruction first. Then read Sections 20.3 to 20.4 before treating individual reconstruction components as projectors. If you need to compose maps in redundant coordinates, continue to the coefficient lifts and bridges in Section 20.5. Keep the physical dimension and the coefficient dimension written separately throughout the calculation.

An analysis/synthesis presentation separates a physical vector from the coefficients used to represent it. In a basis the coefficients are unique. In a redundant frame they are not, and that nonuniqueness changes the algebra: the coefficient identity becomes a projection, coefficient vectors acquire a gauge freedom, and maps between physical spaces require supported lifts and, occasionally, transition bridges.

Section 16.11 is the canonical stable method page for `AnalysisSynthesisSystem` and `AnalysisSynthesisBlocks`. The discussion below uses that construction contract and develops the full redundant-frame algebra. It is authoritative for coefficient gauges, quotient metrics, lifted maps, coefficient blocks, adjoints, direct sums, tensor products, and the exact rational mirror.

The chapter is finite-dimensional. It makes no claim about infinite frame completion, convergence of infinite expansions, measurable direct integrals, or a CUDA/device implementation. Numerical objects use dense NumPy arrays. Exact objects use immutable rational tuples.

20.1 `AnalysisSynthesisSystem`

$$A : X \to C, \qquad S : C \to X, \qquad SA = I_X.$$

20.1.1 Dimensions and data model

Write

$$A \in \mathbb{F}^{m \times n}, \qquad S \in \mathbb{F}^{n \times m}.$$

Here $n = \dim X$ is the source dimension and $m = \dim C$ is the coefficient dimension. The package uses these cases.

Case	Dimension relation	Meaning
square	$m = n$	invertible coordinate presentation when the certificate passes
redundant	$m > n$	more analysis coefficients than source coordinates
undercomplete proposal	$m < n$	constructible only through the bare numerical record. Its left-inverse certificate cannot pass

The stable numerical record has the frozen signature

```
AnalysisSynthesisSystem(
    analysis: np.ndarray,
    synthesis: np.ndarray,
    source: str = "source",
    coordinates: str = "coordinates",
    tolerance: float = 1e-12,
```

```
    theorem: str = "KM-AS-01",
)
```

The matrices are copied, made read-only, and checked for two dimensions, nonempty shape, compatible transposed rectangular shapes, and finite entries. The two labels must be nonempty after stripping. The tolerance must be finite and strictly positive.

Direct construction separates a *well-formed proposal* from a *verified left inverse*. Thus

```
system = AnalysisSynthesisSystem(A, S)
```

can succeed even when $SA \neq I$. Use

```
system.certificate().require()
```

before treating the object as a valid presentation. This separation lets verification code inspect failed proposed systems instead of losing their diagnostics during construction.

> **Terminology.** In this API `frame_operator()` returns the source-space reconstruction operator SA. Some frame-theory texts instead use "frame operator" for A^*A. The method name follows the package's frozen stable contract.

20.1.2 Analysis, synthesis, and roundtrip

For $x \in X$ and $c \in C$,

$$c = Ax, \qquad S(c) = Sc, \qquad \text{reconstruct}(x) = SAx.$$

The core properties and methods are

Member	Return	Contract
`source_dimension`	`int`	n, the number of analysis columns
`coordinate_dimension`	`int`	m, the number of analysis rows
`square`	`bool`	`m == n`
`redundant`	`bool`	`m > n`
`frame_operator()`	`np.ndarray`	$SA \in \mathbb{F}^{n\times n}$
`analyze(value)`	`np.ndarray`	Ax, requiring a finite length-n vector
`synthesize(value)`	`np.ndarray`	Sc, requiring a finite length-m vector
`reconstruct(value)`	`np.ndarray`	SAx
`certificate()`	`Certificate`	global finite left-inverse check
`reconstruction_certificate(value)`	`Certificate`	one concrete roundtrip check

The system certificate computes

$$r_{AS} = \|SA - I_n\|_F$$

and is valid precisely when

$$r_{AS} \leq \texttt{tolerance}\,\max(1, \|I_n\|_F)$$

and `np.linalg.matrix_rank(A) == n`. Its residual key is `analysis_synthesis_left_inverse`. Diagnostics record both dimensions, the observed rank, and the square/redundant flags. The exactness label is `Exactness.NUMERICAL`, even if an input array happens to contain integers.

The pointwise certificate instead records

$$r_x = \|SAx - x\|_2$$

under `analysis_synthesis_roundtrip`. It also requires the global system certificate to be valid.

20.1.3 Minimal stable example

```
import numpy as np
from komorebi_math.stable import AnalysisSynthesisSystem

A = np.array([
    [1.0, 0.0],
    [0.0, 1.0],
    [1.0, 1.0],
    [1.0, -1.0],
])

system = AnalysisSynthesisSystem.canonical_dual(
    A,
    source="signal",
    coordinates="four redundant coefficients",
)
x = np.array([3.0, -2.0])

system.certificate().require()
system.reconstruction_certificate(x).require()

print(system.source_dimension)
print(system.coordinate_dimension)
print(system.redundant)
print(system.analyze(x))
print(system.reconstruct(x))
```

Output, up to ordinary floating representation, is

```
2
4
True
[ 3. -2.  1.  5.]
[ 3. -2.]
```

20.2 Canonical dual synthesis

$$S = (A^*A)^{-1}A^*.$$

20.2.1 Square inverse and Moore-Penrose constructors

Two constructors derive S from A:

```
AnalysisSynthesisSystem.from_invertible(
    matrix,
    *,
    source="source",
    coordinates="coordinates",
    tolerance=1e-12,
    theorem="KM-AS-01",
)

AnalysisSynthesisSystem.canonical_dual(
    analysis,
    *,
    source="source",
    coordinates="coordinates",
    tolerance=1e-12,
    theorem="KM-AS-01",
)
```

`from_invertible` requires a square matrix and calls `np.linalg.inv`. A singular inverse is translated to `ValidationError`. The factory returns the record without implicitly requiring its numerical certificate, so callers retain the standard explicit pattern

```
square = AnalysisSynthesisSystem.from_invertible(A)
square.certificate().require()
```

`canonical_dual` supports square or redundant full-column-rank analysis. Its explicit rank precheck uses the relative SVD threshold

$$r_\tau(A) = \#\{i : \sigma_i > \tau\sigma_{\max}\}, \qquad \tau = \texttt{tolerance}.$$

The method requires $r_\tau(A) = n$ and calls `np.linalg.pinv(A, rcond=tolerance)`, so the explicit precheck and pseudoinverse truncation share the declared relative cutoff. It then constructs the system and separately requires its certificate. That certificate tests the left-inverse residual with the declared tolerance, but its `np.linalg.matrix_rank(A)` call uses NumPy's default rank threshold rather than the declared cutoff. The final certificate is therefore an independent safeguard, not a third use of one universal rank threshold. A matrix can have nonzero entries in every singular direction yet be rejected as numerically rank deficient at the declared precheck resolution or by the subsequent certificate.

For complex NumPy arrays, A^* means conjugate transpose. For the exact rational class in Section 20.6, all coefficients are real rationals and the formula uses the ordinary transpose.

20.2.2 Canonical-dual output

For the four-by-two analysis matrix in Section 20.1,

$$A^*A = 3I_2, \qquad S = \frac{1}{3}A^* = \begin{pmatrix} 1/3 & 0 & 1/3 & 1/3 \\ 0 & 1/3 & 1/3 & -1/3 \end{pmatrix}.$$

The resulting coefficient projection AA^+ is orthogonal. That minimum-Euclidean-norm property belongs to the Moore-Penrose construction. An arbitrary valid left inverse produces the generally oblique gauge algebra of Section 20.5.

20.2.3 Threshold boundary

```
import numpy as np
from komorebi_math.coordinates import AnalysisSynthesisSystem
from komorebi_math.exceptions import ValidationError

near_singular = np.diag([1.0, 9e-16])

try:
    AnalysisSynthesisSystem.canonical_dual(near_singular)
except ValidationError as error:
    print(type(error).__name__)
```

With the default tolerance, output is

```
ValidationError
```

This regression is part of the installed frame-algebra verifier. It prevents rank testing and pseudoinverse truncation from silently using inconsistent thresholds.

20.3 Components and blocks

$$C_i = SE_iA, \qquad \sum_i C_i = I_X.$$

20.3.1 Coordinate selectors and additive reconstruction

For an index set $I \subseteq \{0, \dots, m-1\}$, let E_I be the diagonal coefficient selector. The API constructs

$$C_I = SE_IA \in \mathbb{F}^{n\times n}.$$

```
component(indices)
components(groups=None)
```

An integer selects one coordinate. A sequence selects one nonempty set of unique coordinates. `groups=None` uses the singleton partition. A supplied grouping must be nonempty, disjoint, and cover every coefficient coordinate exactly once.

For a complete partition $\{I_\alpha\}$,

$$\sum_\alpha C_{I_\alpha} = S\left(\sum_\alpha E_{I_\alpha}\right)A = SA = I_n.$$

The aliases `projector()` and `projectors()` call these same methods. They are retained for compatibility. They do not add an idempotence check.

20.3.2 Physical operator blocks

For a source-space endomorphism $T \in \mathbb{F}^{n\times n}$,

$$T = \left(\sum_\beta C_\beta\right) T \left(\sum_\alpha C_\alpha\right) = \sum_{\beta,\alpha} C_\beta T C_\alpha.$$

```
blocks = system.operator_blocks(T, groups=groups)
```

returns the stable record

```
AnalysisSynthesisBlocks(
    system: AnalysisSynthesisSystem,
    groups: tuple[tuple[int, ...], ...],
    blocks: tuple[tuple[np.ndarray, ...], ...],
    operator: np.ndarray,
)
```

Each stored block is a physical $n \times n$ matrix. If there are p groups, `blocks` is a $p \times p$ tuple grid. Its methods are

Member	Meaning
`reconstruct()`	sum all stored physical blocks
`certificate()`	compare that sum with the stored operator

The certificate residual is

$$r_B = \left\| \sum_{\beta,\alpha} B_{\beta\alpha} - T \right\|_F$$

under `operator_block_reconstruction`, with theorem identifier `KM-AS-02`. It also requires the associated system certificate.

Construct blocks through `operator_blocks()`. Directly instantiating `AnalysisSynthesisBlocks` verifies grid and matrix shapes and can certify the sum, but it does not independently prove that arbitrary supplied matrices equal $C_\beta T C_\alpha$.

20.3.3 Grouped block example

```
T = np.array([[2.0, -1.0], [3.0, 4.0]])
groups = ((0, 2), (1, 3))

physical_blocks = system.operator_blocks(T, groups=groups)
physical_blocks.certificate().require()

print(np.allclose(physical_blocks.reconstruct(), T))
print([[block.shape for block in row] for row in physical_blocks.blocks])
```

Output is

```
True
[[(2, 2), (2, 2)], [(2, 2), (2, 2)]]
```

These stable physical blocks should not be confused with the small coefficient blocks in Section 20.5.6.

20.4 Square projectors versus redundant reconstruction components

20.4.1 Square algebra

If $m = n$ and A is invertible, then $S = A^{-1}$ and $AS = I_m$. Hence

$$C_I C_J = S E_I A S E_J A = S E_I E_J A.$$

For disjoint I and J, $C_I C_J = 0$. For $I = J$, $C_I^2 = C_I$. The square components are therefore algebraic projectors transported from the coordinate selectors.

20.4.2 Redundant algebra

In a redundant system,

$$P = AS \neq I_m$$

in general, and

$$C_I C_J = S E_I P E_J A.$$

No selector identity removes P. Individual components may fail both idempotence and pairwise annihilation even though their complete sum remains I_n.

For the four-coordinate canonical-dual example, the singleton components are

$$C_0 = \begin{pmatrix} 1/3 & 0 \\ 0 & 0 \end{pmatrix}, \quad C_1 = \begin{pmatrix} 0 & 0 \\ 0 & 1/3 \end{pmatrix},$$

$$C_2 = \frac{1}{3}\begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}, \quad C_3 = \frac{1}{3}\begin{pmatrix} 1 & -1 \\ -1 & 1 \end{pmatrix}.$$

They sum to I_2, but each has

$$\|C_i^2 - C_i\|_F = \frac{2}{9} \approx 0.22222222.$$

```
components = system.components()

print(np.allclose(sum(components), np.eye(2)))
print([np.linalg.norm(C @ C - C) for C in components])
```

Output is

```
True
[0.22222222, 0.22222222, 0.22222222, 0.22222222]
```

20.4.3 Why one-index physical block multiplication fails

Suppose $T : X \to X$ and $U : X \to X$. The tempting rule

$$C_\beta U T C_\alpha \stackrel{?}{=} \sum_\gamma (C_\beta U C_\gamma)(C_\gamma T C_\alpha)$$

would require the middle components to resolve identity through a single index after multiplication. In a redundant system, the right side contains C_γ^2, not C_γ, and the rule is generally false.

The always-valid physical expansion uses two middle indices:

$$C_\beta U T C_\alpha = \sum_{\gamma,\delta} (C_\beta U C_\gamma)(C_\delta T C_\alpha).$$

Ordinary one-index block multiplication is recovered in coefficient space, where the small blocks are literal submatrices of the coefficient lift.

20.5 Coefficient gauges, quotient metrics, lifts, and bridges

20.5.1 `AnalysisSynthesisGauge`

The research record

```
AnalysisSynthesisGauge(system: AnalysisSynthesisSystem)
```

requires an `AnalysisSynthesisSystem` whose certificate passes. It delegates the public presentation properties and supplies the quotient algebra.

Define

$$P = AS, \qquad Q = I_m - P.$$

Then

$$P^2 = P, \qquad Q^2 = Q, \qquad P + Q = I_m,$$

and every coefficient vector decomposes as

$$c = Pc + Qc, \qquad Qc \in \ker S.$$

The principal methods are

Member	Formula or result
`coefficient_projection()`	$P = AS$
`gauge_projection()`	$Q = I - P$
`canonical_coefficients(c)`	Pc
`gauge_part(c)`	Qc
`gauge_equivalent(c,d)`	tolerance-aware test of $S(c-d) = 0$
`coefficient_metric()`	$H = S^*S$
`coefficient_seminorm(c)`	$\|Sc\|_2$
`dual()`	the presentation (S^*, A^*)
`transition_to(other)`	$A_{\text{other}} S_{\text{self}}$
`direct_sum(other)`	block-diagonal presentation
`tensor_product(other)`	Kronecker presentation
`certificate()`	projection, support, and metric laws

Gauge equivalence is

$$c \sim d \iff S(c-d) = 0.$$

The metric

$$H = S^*S$$

is positive semidefinite and realizes the physical inner product:

$$\langle c, Hd \rangle = \langle Sc, Sd \rangle.$$

It is a seminorm on raw coefficient space and a norm on the quotient $C/\ker S$.

The gauge certificate checks

$$P^2 = P, \quad Q^2 = Q, \quad P + Q = I, \quad PA = A, \quad SP = S, \quad P^*H = H, \quad HP = H.$$

Its residual keys are `projection_idempotence`, `gauge_idempotence`, `projection_complement`, `analysis_support`, `synthesis_support`, `metric_left_support`, and `metric_right_support`. The theorem identifier is `KM-AS-GAUGE-01`.

20.5.2 Oblique gauge example

```
from komorebi_math.coordinates import AnalysisSynthesisGauge

A_oblique = np.array([
    [1.0, 0.0],
    [0.0, 1.0],
    [1.0, 1.0],
])
S_oblique = np.array([
    [0.5, -0.5, 0.5],
    [-0.25, 0.75, 0.25],
])

oblique = AnalysisSynthesisSystem(
    A_oblique,
    S_oblique,
    source="physical",
    coordinates="oblique coefficients",
)
gauge = AnalysisSynthesisGauge(oblique)
gauge.certificate().require()

c = np.array([3.0, -4.0, 5.0])
k = np.array([-2.0, -1.0, 1.0])

print(gauge.coefficient_projection())
print(gauge.canonical_coefficients(c))
print(gauge.gauge_part(c))
print(gauge.gauge_equivalent(c, c + k))
print(gauge.coefficient_seminorm(k))
```

Output is

```
[[ 0.5  -0.5   0.5 ]
 [-0.25  0.75  0.25]
 [ 0.25  0.25  0.75]]
[ 6.  -2.5  3.5]
[-3.  -1.5  1.5]
True
0.0
```

The displayed projection is idempotent but not symmetric. The vector k is a nonzero element of `ker(S_oblique)`, so `c` and `c + k` are distinct arrays representing the same physical vector.

20.5.3 Duals, direct sums, and tensor products

The dual presentation is

$$(A, S)^\vee = (S^*, A^*).$$

It remains valid because

$$A^* S^* = (SA)^* = I.$$

Direct sums use

$$A_{X\oplus Y} = A_X \oplus A_Y, \qquad S_{X\oplus Y} = S_X \oplus S_Y,$$

and tensor products use

$$A_{X\otimes Y} = A_X \otimes A_Y, \qquad S_{X\otimes Y} = S_X \otimes S_Y.$$

The numerical result uses the larger input tolerance. Direct-sum dimensions add. Tensor dimensions multiply. The latter can expand dense storage rapidly.

20.5.4 `AnalysisSynthesisMap` and coefficient lifts

The physical-map record is

```
AnalysisSynthesisMap(
    domain: AnalysisSynthesisSystem,
    codomain: AnalysisSynthesisSystem,
    operator: np.ndarray,
    theorem: str = "KM-AS-MAP-01",
)
```

Both endpoint certificates are required during construction. If $T : X \to Y$, its canonical coefficient lift is

$$\widehat{T} = A_Y T S_X.$$

Let $P_X = A_X S_X$ and $P_Y = A_Y S_Y$. Then

$$P_Y \widehat{T} P_X = \widehat{T}, \qquad S_Y \widehat{T} A_X = T.$$

Consequently,

$$\widehat{T}(I - P_X) = 0, \qquad (I - P_Y)\widehat{T} = 0.$$

The lifted physical identity is P_X, not the full identity on redundant coefficient space.

Member	Contract
`identity(system)`	physical identity map on one system
`tolerance`	maximum endpoint tolerance
`coefficient_lift()`	$A_Y T S_X$
`reconstruct()`	$S_Y \widehat{T} A_X$
`apply_source(x)`	Tx
`apply_coefficients(c)`	$\widehat{T}c$
`then(after)`	`after` $\circ$ `self` on physical operators
`compose(before)`	`self` $\circ$ `before`
`transition_bridge(after)`	bridge between middle presentations

Member	Contract
`adjoint()`	physical adjoint map
`dual_adjoint()`	raw coefficient adjoint on dual presentations
`scaled(scalar)`	finite numerical scalar multiple
`add(other)`	map sum with identical endpoint objects
`blocks(...)`	coefficient-space block record
`certificate()`	lift support and physical reconstruction

The map certificate records `coefficient_sandwich` and `physical_reconstruction`. Addition intentionally requires the same domain and codomain object identities. Merely compatible labels and dimensions are not enough.

20.5.5 Transition bridges and composition

If the first map ends in the same presentation in which the second begins,

$$\widehat{U \circ T} = \widehat{U}\,\widehat{T}.$$

If they use different presentations of the same physical middle space, define

$$R = A_{Y,\mathrm{in}} S_{Y,\mathrm{out}}.$$

Then

$$\widehat{U \circ T} = \widehat{U} R \widehat{T}.$$

Transitions are transitive:

$$R_{Z\leftarrow Y} R_{Y\leftarrow X} = R_{Z\leftarrow X}.$$

A self-transition is $AS = P$, not I_m. Compatibility requires equal physical source dimensions and exactly equal `source` strings. Coordinate labels may differ. The source label therefore participates in runtime identity. It is not only decorative metadata.

20.5.6 `CoefficientPartition` and compositional coefficient blocks

The strict ordered partition is

```
CoefficientPartition(
    dimension: int,
    groups: tuple[tuple[int, ...], ...],
)
```

It rejects Boolean and nonintegral dimensions or indices, nonpositive dimension, empty groups, duplicates, overlap, missing coordinates, and out-of-range indices. Each group is sorted internally. Group order is preserved.

```
CoefficientPartition.singletons(dimension)
CoefficientPartition.one_block(dimension)
fine.refines(coarse)
coarse.coarsens(fine)
```

For a domain partition $\{I_\alpha\}$ and target partition $\{J_\beta\}$, the compositional blocks are the small submatrices

$$L_{\beta\alpha} = \widehat{T}[J_\beta, I_\alpha].$$

They are represented by

```
AnalysisSynthesisCoefficientBlocks(
    mapping: AnalysisSynthesisMap,
    source_partition: CoefficientPartition,
    target_partition: CoefficientPartition,
)
```

Member	Contract
`blocks`	target-by-source tuple of small read-only arrays
`assemble()`	reconstruct the entire coefficient lift
`reblock(source_partition, target_partition)`	derive blocks for any valid new partitions
`physical_block(beta, alpha)`	recover $C_\beta^Y T C_\alpha^X$
`then(after, ...)`	compose underlying physical maps and derive fresh blocks
`certificate()`	certify coefficient-block assembly

The physical block is recovered by

$$S_Y[:, J_\beta] L_{\beta\alpha} A_X[I_\alpha, :] = C_\beta^Y T C_\alpha^X.$$

For an aligned middle partition, the coefficient blocks obey the ordinary one-index rule

$$(L_U L_T)_{\beta\alpha} = \sum_\gamma (L_U)_{\beta\gamma} (L_T)_{\gamma\alpha}.$$

When middle presentations differ, the bridge must appear between their lifts. `then()` does not multiply the stored block arrays. It composes the physical maps and constructs the requested result blocks.

20.5.7 Lift-and-block example

```
from komorebi_math.coordinates import (
    AnalysisSynthesisMap,
    CoefficientPartition,
)

codomain = AnalysisSynthesisSystem.canonical_dual(
    np.array([
        [1.0, 0.0],
        [0.0, 1.0],
        [1.0, -1.0],
        [2.0, 1.0],
    ]),
    source="codomain",
    coordinates="four target coefficients",
)
T = np.array([[2.0, -1.0], [3.0, 4.0]])
mapping = AnalysisSynthesisMap(oblique, codomain, T)

source_partition = CoefficientPartition(3, ((0, 2), (1,)))
target_partition = CoefficientPartition(4, ((0, 3), (1, 2)))
coefficient_blocks = mapping.blocks(source_partition, target_partition)

mapping.certificate().require()
coefficient_blocks.certificate().require()

print(mapping.coefficient_lift())
print([[block.shape for block in row] for row in coefficient_blocks.blocks])
```

```
print(np.allclose(coefficient_blocks.assemble(), mapping.coefficient_lift()))
print(np.allclose(mapping.reconstruct(), T))
```

Output is

```
[[ 1.25 -1.75  0.75]
 [ 0.5   1.5   2.5 ]
 [ 0.75 -3.25 -1.75]
 [ 3.   -2.    4.  ]]
[[(2, 2), (2, 1)], [(2, 2), (2, 1)]]
True
True
```

20.5.8 Physical and dual adjoints

The physical adjoint has lift

$$\widehat{T^*} = A_X T^* S_Y.$$

It is generally not the raw conjugate transpose of the original lift. With $H_X = S_X^* S_X$ and $H_Y = S_Y^* S_Y$,

$$\widehat{T}^* H_Y = H_X \widehat{T^*}.$$

`adjoint()` returns this physical adjoint. `dual_adjoint()` changes the endpoints to the dual presentations and then lifts T^*. Its coefficient lift is literally $\widehat{T}^*$. For Parseval endpoints, $S = A^*$, and the distinction disappears.

20.6 Rational frame algebra and dual/physical adjoints

20.6.1 Exact system signature and scalar domain

The exact rational presentation is

```
ExactAnalysisSynthesisSystem(
    analysis: Matrix,
    synthesis: Matrix,
    source: str = "source",
    coordinates: str = "coordinates",
    theorem: str = "KM-AS-EXACT-01",
)
```

Inputs must be integers or `fractions.Fraction`. Booleans, floats, complex numbers, ragged rows, and empty matrices are rejected. The immutable matrices are normalized to tuples of `Fraction` values. Construction requires the literal equality

$$SA = I_n.$$

Unlike the bare numerical record, an invalid exact left inverse cannot exist as a constructed exact system.

Factories are

```
ExactAnalysisSynthesisSystem.from_invertible(
    analysis,
    *, source="source", coordinates="coordinates",
    theorem="KM-AS-EXACT-01",
)

ExactAnalysisSynthesisSystem.canonical_dual(
    analysis,
```

```
    *, source="source", coordinates="coordinates",
    theorem="KM-AS-EXACT-01",
)
```

For full-column-rank rational A, the latter computes

$$S = (A^T A)^{-1} A^T$$

by exact elimination. There is no tolerance or numerical rank threshold.

20.6.2 Exact system members

Member	Exact result
`source_dimension`	source dimension
`coordinate_dimension`	coefficient dimension
`redundant`	coefficient dimension exceeds source dimension
`frame_operator()`	SA
`coefficient_projection()`	AS
`gauge_projection()`	$I - AS$
`coefficient_metric()`	$S^T S$
`analyze(x)`	exact vector Ax
`synthesize(c)`	exact vector Sc
`canonical_coefficients(c)`	exact ASc
`gauge_part(c)`	exact $(I - AS)c$
`gauge_equivalent(c,d)`	literal test $S(c - d) = 0$
`coefficient_seminorm_squared(c)`	$\sum_i (Sc)_i^2$
`dual()`	(S^T, A^T)
`transition_to(other)`	exact $A_{\text{other}} S_{\text{self}}$
`direct_sum(other)`	exact block-diagonal system
`tensor_product(other)`	exact Kronecker system
`certificate()`	exact reconstruction, projection, and metric identities

The exact certificate has `Exactness.EXACT`, no floating residuals, and checks

$$SA = I, \qquad P^2 = P, \qquad P^T H = H, \qquad HP = H.$$

20.6.3 Exact map signature and members

```
ExactAnalysisSynthesisMap(
    domain: ExactAnalysisSynthesisSystem,
    codomain: ExactAnalysisSynthesisSystem,
    operator: Matrix,
    theorem: str = "KM-AS-EXACT-MAP-01",
)
```

Its public methods are

Member	Exact contract
`identity(system)`	exact physical identity
`coefficient_lift()`	$A_Y T S_X$
`reconstruct()`	$S_Y \widehat{T} A_X = T$
`apply_coefficients(c)`	exact lifted coefficient action

Member	Exact contract
`then(after)`	`after ∘ self`
`compose(before)`	`self ∘ before`
`adjoint()`	transpose physical map
`dual_adjoint()`	transpose lift on dual systems
`scaled(scalar)`	integer or `Fraction` scaling
`add(other)`	exact sum with identical endpoint objects
`blocks(source_partition, target_partition)`	raw nested exact submatrices
`certificate()`	exact support and reconstruction equalities

The exact coefficient domain is rational and real, so "adjoint" means transpose. There is no complex-conjugation branch.

20.6.4 Exact worked example

```
from fractions import Fraction
from komorebi_math.coordinates import ExactAnalysisSynthesisSystem

exact = ExactAnalysisSynthesisSystem.canonical_dual(
    ((1, 0), (0, 1), (1, 1)),
    source="exact physical",
    coordinates="exact frame",
)

c = (3, -4, 5)
k = (-1, -1, 1)

print(exact.synthesis)
print(exact.canonical_coefficients(c))
print(exact.gauge_part(c))
print(exact.gauge_equivalent(c, tuple(a + b for a, b in zip(c, k))))
print(exact.coefficient_seminorm_squared(k))
```

Output is

```
((Fraction(2, 3), Fraction(-1, 3), Fraction(1, 3)),
 (Fraction(-1, 3), Fraction(2, 3), Fraction(1, 3)))
(Fraction(5, 1), Fraction(-2, 1), Fraction(3, 1))
(Fraction(-2, 1), Fraction(-2, 1), Fraction(2, 1))
True
0
```

The exact coefficient projection is

$$P = \begin{pmatrix} 2/3 & -1/3 & 1/3 \\ -1/3 & 2/3 & 1/3 \\ 1/3 & 1/3 & 2/3 \end{pmatrix}.$$

20.6.5 Deliberate numerical/exact asymmetries

The exact layer is not a method-for-method clone of the stable numerical record.

Available only in the numerical convenience layer	Exact-layer behavior
`square` property	infer from dimensions if needed
`reconstruct(x)` on a system	call `synthesize(analyze(x))`
`reconstruction_certificate(x)`	constructor and certificate already use literal identities
system `component(s)` and `operator_blocks()`	construct exact selectors and products explicitly
`apply_source(x)` on a map	multiply by the public exact `operator`
block record with `assemble()` and `physical_block()`	`ExactAnalysisSynthesisMap.blocks()` returns nested tuples
portable codec	no exact frame-algebra codec is registered

Exact arithmetic removes tolerance ambiguity but does not remove resource growth. Dense inversion, Kronecker products, and repeated composition can produce large numerators and denominators.

20.7 `coordinates.frame_algebra` public method and property reference

20.7.1 API tier and import boundaries

The stable surface contains exactly the original numerical system and physical-block record:

```
from komorebi_math.stable import (
    AnalysisSynthesisBlocks,
    AnalysisSynthesisSystem,
)
```

The same class objects are exported by `komorebi_math.v1` and `komorebi_math.api.v1`. Section 16.11 is their canonical stable implementation page.

The six coefficient-frame additions are research APIs:

```
from komorebi_math.coordinates import (
    AnalysisSynthesisCoefficientBlocks,
    AnalysisSynthesisGauge,
    AnalysisSynthesisMap,
    CoefficientPartition,
    ExactAnalysisSynthesisMap,
    ExactAnalysisSynthesisSystem,
)
```

They are also present at the package root. They are absent from both the stable and beta manifests. Top-level availability must not be described as a 1.x compatibility guarantee.

20.7.2 Complete class and member ledger

Class and constructor signature	Public members
`AnalysisSynthesisSystem(analysis, synthesis, source="source", coordinates="coordinates", tolerance=1e-12, theorem="KM-AS-01")`	`analysis`, `synthesis`, `source`, `coordinates`, `tolerance`, `theorem`, `from_invertible`, `canonical_dual`, `source_dimension`, `coordinate_dimension`, `redundant`, `square`, `frame_operator`, `analyze`, `synthesize`, `reconstruct`, `component`, `components`, `projector`, `projectors`, `operator_blocks`, `certificate`, `reconstruction_certificate`
`AnalysisSynthesisBlocks(system, groups, blocks, operator)`	`system`, `groups`, `blocks`, `operator`, `reconstruct`, `certificate`
`CoefficientPartition(dimension, groups)`	`dimension`, `groups`, `singletons`, `one_block`, `refines`, `coarsens`

Class and constructor signature	Public members
`AnalysisSynthesisGauge(system)`	`system`, `analysis`, `synthesis`, `source`, `coordinates`, `source_dimension`, `coordinate_dimension`, `tolerance`, `coefficient_projection`, `gauge_projection`, `coefficient_metric`, `canonical_coefficients`, `gauge_part`, `gauge_equivalent`, `coefficient_seminorm`, `dual`, `transition_to`, `direct_sum`, `tensor_product`, `certificate`
`AnalysisSynthesisMap(domain, codomain, operator, theorem="KM-AS-MAP-01")`	`domain`, `codomain`, `operator`, `theorem`, `identity`, `tolerance`, `coefficient_lift`, `reconstruct`, `apply_source`, `apply_coefficients`, `then`, `compose`, `transition_bridge`, `adjoint`, `dual_adjoint`, `scaled`, `add`, `blocks`, `certificate`
`AnalysisSynthesisCoefficientBlocks(mapping, source_partition, target_partition)`	`mapping`, `source_partition`, `target_partition`, `blocks`, `assemble`, `reblock`, `physical_block`, `then`, `certificate`
`ExactAnalysisSynthesisSystem(analysis, synthesis, source="source", coordinates="coordinates", theorem="KM-AS-EXACT-01")`	`analysis`, `synthesis`, `source`, `coordinates`, `theorem`, `from_invertible`, `canonical_dual`, `source_dimension`, `coordinate_dimension`, `redundant`, `frame_operator`, `coefficient_projection`, `gauge_projection`, `coefficient_metric`, `analyze`, `synthesize`, `canonical_coefficients`, `gauge_part`, `gauge_equivalent`, `coefficient_seminorm_squared`, `dual`, `transition_to`, `direct_sum`, `tensor_product`, `certificate`
`ExactAnalysisSynthesisMap(domain, codomain, operator, theorem="KM-AS-EXACT-MAP-01")`	`domain`, `codomain`, `operator`, `theorem`, `identity`, `coefficient_lift`, `reconstruct`, `apply_coefficients`, `then`, `compose`, `adjoint`, `dual_adjoint`, `scaled`, `add`, `blocks`, `certificate`

20.7.3 Theorem identifiers

Generated objects use these default provenance labels.

Operation	Identifier
numerical system	`KM-AS-01`
stable physical blocks	`KM-AS-02`
numerical gauge certificate	`KM-AS-GAUGE-01`
gauge dual	`KM-AS-GAUGE-DUAL-01`
gauge direct sum	`KM-AS-GAUGE-SUM-01`
gauge tensor product	`KM-AS-GAUGE-TENSOR-01`
numerical map	`KM-AS-MAP-01`
map identity	`KM-AS-MAP-ID-01`
map composition	`KM-AS-MAP-COMPOSE-01`
physical adjoint	`KM-AS-MAP-ADJOINT-01`
dual adjoint	`KM-AS-MAP-DUAL-ADJOINT-01`
map scaling	`KM-AS-MAP-SCALE-01`
map addition	`KM-AS-MAP-ADD-01`
coefficient blocks	`KM-AS-BLOCK-01`
exact system	`KM-AS-EXACT-01`
exact dual	`KM-AS-EXACT-DUAL-01`
exact direct sum	`KM-AS-EXACT-SUM-01`
exact tensor product	`KM-AS-EXACT-TENSOR-01`
exact map	`KM-AS-EXACT-MAP-01`
exact map composition	`KM-AS-EXACT-MAP-COMPOSE-01`
exact physical adjoint	`KM-AS-EXACT-MAP-ADJOINT-01`
exact dual adjoint	`KM-AS-EXACT-MAP-DUAL-ADJOINT-01`
exact map scaling	`KM-AS-EXACT-MAP-SCALE-01`
exact map addition	`KM-AS-EXACT-MAP-ADD-01`

The theorem field is provenance metadata attached to certificates. It is not a lookup into a separate theorem database.

20.7.4 Persistence contract

Only `AnalysisSynthesisSystem` has a registered frame-algebra codec.

Property	Value
tag	`coordinates.analysis_synthesis`
version	1
stable portable contract	yes
separately portable blocks, gauges, maps, partitions, exact systems	no

The payload data contains exactly

```
analysis
synthesis
source
coordinates
tolerance
theorem
```

The decoder rejects missing or extra fields, reconstructs the system, and immediately calls `certificate().require()`. A portable payload cannot silently load an invalid left inverse.

```
from komorebi_math.stable import dumps_portable, loads_portable

text = dumps_portable(system)
restored = loads_portable(text)

restored.certificate().require()
print(type(restored).__name__)
print(np.allclose(restored.reconstruct(x), x))
```

Output is

```
AnalysisSynthesisSystem
True
```

The envelope SHA-256 protects against accidental or untrusted payload changes only when the digest itself has not also been replaced. It is not a digital signature or message-authentication code.

20.7.5 Dense complexity and storage

Let one presentation have source dimension n, coefficient dimension m, and p coordinate groups. These costs describe the current dense implementation, not asymptotically optimized mathematical alternatives.

Operation	Approximate dense arithmetic	Principal storage
construct numerical system	$O(mn)$	$O(mn)$
square inverse	$O(n^3)$	$O(n^2)$
canonical dual, $m \geq n$	$O(mn^2)$	$O(mn + n^2)$
analyze or synthesize	$O(mn)$	output vector
source reconstruction operator	$O(mn^2)$	$O(n^2)$
system certificate with rank work	$O(mn^2)$	$O(mn)$
current dense `component()`	$O(nm^2 + n^2m)$	dense selector plus $n \times n$ result
$p \times p$ physical block grid	approximately $O(p^2n^3)$, plus components	$O(p^2n^2)$
coefficient projection or metric	$O(m^2n)$	$O(m^2)$

Operation	Approximate dense arithmetic	Principal storage
full gauge certificate	$O(m^3)$ dominant	$O(m^2)$
transition $m_X \to m_Y$	$O(m_Y n m_X)$	$O(m_Y m_X)$
lift $A_Y T S_X$	$O(m_Y n_Y n_X + m_Y n_X m_X)$	$O(m_Y m_X)$
reconstruct a lift	$O(n_Y m_Y m_X + n_Y m_X n_X)$	physical matrix
extract coefficient blocks	$O(m_Y m_X)$ copying	$O(m_Y m_X)$
direct sum	output-sized	dimensions add
tensor product	output-sized	dimensions multiply

The current `component()` implementation materializes a dense $m \times m$ selector. Users handling large m should account for that implementation cost rather than assuming a sparse diagonal selector.

Exact operations have the same algebraic operation counts but not constant scalar cost. Numerator and denominator bit lengths can grow under inversion, Kronecker product, and repeated composition.

20.7.6 Failure boundaries

These are controlled rejection boundaries or explicit semantic limits.

Boundary	Result
nonmatrix, empty, or nonfinite numerical data	`DimensionError` or `ValidationError`
incompatible synthesis shape	`DimensionError`
blank labels	`ValidationError`
nonpositive or nonfinite tolerance	`ValidationError`
nonsquare `from_invertible` input	`DimensionError`
singular square inverse	`ValidationError`
rank-deficient canonical-dual input	`ValidationError`
wrong vector length or nonfinite vector	`DimensionError` or `ValidationError`
missing, overlapping, duplicate, empty, or out-of-range groups	`ValidationError`
wrong source-operator shape	`DimensionError`
gauge or map endpoint with failed system certificate	construction fails through `Certificate.require()`
incompatible physical composition labels or dimensions	`ValidationError`
map addition with merely equivalent rather than identical endpoint objects	`ValidationError`
malformed strict coefficient partition	`ValidationError` or `DimensionError`
Boolean, float, or complex exact scalar	`ValidationError`
exact $SA \neq I$	construction-time `ValidationError`
exact transition between differently named physical sources	`ValidationError`
arbitrary callable or class persistence	unsupported. Codecs reconstruct only registered data objects

Component grouping performs historical integer coercion before validating the partition. This should not be used as permission to pass floats or strings. New code should use integer indices, and research coefficient partitions enforce that rule strictly.

`physical_block(target_index, source_index)` is intended for nonnegative valid block indices. The implementation currently inherits Python negative tuple indexing. That incidental behavior is not a documented compatibility promise.

Numerical linear algebra can also propagate a backend `LinAlgError` if an SVD itself fails. Certificates establish the listed finite residual laws only. They do not certify conditioning beyond the declared rank threshold or prove stability of an unbounded sequence of operations.

20.7.7 Verification and executable anchors

The canonical installed check is

```
python -m komorebi_math verify frame-algebra
```

It executes six checks:

1. oblique coefficient gauge
2. coefficient-map closure
3. cross-presentation bridge
4. ordinary coefficient-block multiplication
5. exact rational coefficient closure
6. consistent canonical-dual threshold

Expected final line is

```
verification passed 6/6 checks
```

The focused source replay is

```
python -m pytest -q tests/test_block16_analysis_synthesis.py tests/test_block16_contracts.py
    ↪ tests/test_block18_frame_algebra.py
```

The current chapter audit obtains

```
31 passed
```

The shipped stable worked example is

```
python examples/block16_analysis_synthesis.py
```

It validates the redundant canonical dual, reconstructs `[3, -2]`, certifies grouped physical operator blocks, and roundtrips the numerical system through the portable codec. Its floating operator-block residual is typically near machine precision and must be compared with the tolerance rather than frozen to one platform-specific decimal.

The independent source anchors for this chapter are

```
src/komorebi_math/coordinates/analysis_synthesis.py
src/komorebi_math/coordinates/frame_algebra.py
src/komorebi_math/coordinates/portable.py
src/komorebi_math/verify/frame_algebra.py
tests/test_block16_analysis_synthesis.py
tests/test_block16_contracts.py
tests/test_block18_frame_algebra.py
docs/ANALYSIS_SYNTHESIS_SYSTEMS.md
docs/REDUNDANT_FRAME_ALGEBRA.md
```

Chapter 20 summary. A valid presentation satisfies $SA = I$, but its redundant coefficient identity is the projection $P = AS$. Physical components always add to identity and need not themselves be projectors. Coefficient vectors are therefore gauge classes, physical maps lift to sandwiched coefficient matrices, and mismatched presentations compose through bridges. Small coefficient blocks recover ordinary block multiplication. The rational mirror turns every finite identity into literal equality.

Part V. Generating functions, arithmetic, sequences, and analytic packets

Reading route. Choose a representation before choosing an algorithm. This part offers several ways to describe related data, with different coefficient, index, and arithmetic conventions.

You want to work with	Start here	First decision
A finite coefficient tuple	Chapter 21	Ordinary or normalized coefficients, and which scalar domain
A rule that produces coefficient prefixes	Chapter 22	A declared provider or a candidate inferred from samples
A mask or digit transition system	Chapter 23	Radix, digit order, and the required finite range
Prime-power residues or modular support	Chapter 24	Prime, exponent, and affordable compiler-table size
Local arithmetic coordinates or recurrence packets	Chapter 25	The information retained by the modulus or precision
A catalog sequence or a custom term evaluator	Chapter 26	Prefix, single index, or modular evaluation
Cyclic analytic coefficients or zero diagnostics	Chapter 27	Exact coefficient work or numerical evaluation

Start with a small prefix whose entries you can inspect. Keep the normalization, scalar domain, and finite range with the resulting values. The same tuple can represent different series under different normalizations. Likewise, residue classes of *indices* and residues of *values* answer different questions. Chapter 23's carry and Cartier machines read digits least significant first, while Chapter 26's automatic sequences read them most significant first. Use each producer's documented digit convention when connecting the two subjects.

21. Formal series and weighted bases

Before computing. Decide which tuple you have, which normalization gives it meaning, and which domain allows the required divisions. The rational workflow in Section 21.5.1 is a useful starting point. Use the record comparison in Section 21.4 to choose between the stable rational interface and the explicit-domain interface. Set the desired output degree before a composition or product so the finite result has the intended range.

Komorebi Math represents a truncated generating function by separating three choices that are often conflated:

1. the ordinary coefficient of z^n
2. the normalization used to store that coefficient
3. the exact domain in which arithmetic is performed

This separation is the foundation for the generating-function subsystem. It lets the same finite formal object move between ordinary, exponential, factorial-power, q-factorial, Fibonacci-factorial, Pochhammer, and custom normalizations without changing the underlying polynomial. It also lets a domain-aware series state whether division takes place in $\mathbb{Z}$, $\mathbb{Q}$, a residue ring, a prime field, or a truncated p-adic ring.

The implementation is finite and coefficient based. It does not construct an infinite series object, prove analytic convergence, select a branch of an analytic continuation, or infer a radius of convergence. Numerical evaluation is an explicit final operation on a finite exact truncation.

21.0 API tier and import surface

Surface	Chapter 21 names	Contract
`komorebi_math.stable`	`FormalSeries`, `factorial_power_basis`, `ordinary_generating_basis`	frozen stable surface
`komorebi_math.beta`	no Chapter 21 names	curated beta surface does not include this subsystem
`komorebi_math`	`FormalSeries`, `FiniteNormalizedSeries`, `WeightedMonomialBasis`, `GeneratingBudget`, `ordinary_basis`, `factorial_power_basis`	selected top-level research exports
`komorebi_math.generating`	complete basis, domain, record, certificate, and codec API	domain research surface

The stable namespace renames the ordinary generating basis to `ordinary_generating_basis` so it cannot be confused with coordinate-basis constructors in other subsystems:

```
from komorebi_math.stable import (
    FormalSeries,
    factorial_power_basis,
    ordinary_generating_basis,
)
```

Use the complete research namespace for the explicit domain layer:

```
from komorebi_math.generating import (
    FiniteNormalizedSeries,
    FormalSeries,
    GeneratingBudget,
    PrimeFieldDomain,
    RationalDomain,
    WeightedMonomialBasis,
    exponential_basis,
    factorial_power_basis,
    ordinary_basis,
)
```

21.1 Coefficient domains and exact arithmetic

`FormalSeries` always stores Python `Fraction` values. Its domain is therefore implicitly $\mathbb{Q}$. `FiniteNormalizedSeries` makes the domain an explicit object satisfying `CoefficientDomain`.

For a domain R, the required arithmetic is

$$(R, 0, 1, +, -, \cdot),$$

together with coercion, equality, unit testing, inversion of units, and a closed data-only descriptor. A residue ring is permitted to contain nonunits. The protocol does not pretend that every nonzero element is invertible.

21.1.1 Domain constructors

```
IntegerDomain(
    name: str = "integers",
    zero: int = 0,
    one: int = 1,
)

RationalDomain(
    name: str = "rationals",
    zero: Fraction = Fraction(0, 1),
    one: Fraction = Fraction(1, 1),
)

ResidueRingDomain(
    modulus: int,
    name: str = "residue_ring",
)

PrimeFieldDomain(
    modulus: int,
    name: str = "prime_field",
)

TruncatedPadicDomain(
    prime: int,
    precision: int,
)

domain_from_descriptor(payload: Any) -> CoefficientDomain[Any]
```

`PrimeFieldDomain` names its constructor argument `modulus`, while its validated public property is `prime`. For `TruncatedPadicDomain(p, r)`, the stored modulus is p^r.

Domain	Elements	Units	Descriptor
`IntegerDomain()`	arbitrary integers	$1, -1$	`{"kind": "integers"}`
`RationalDomain()`	exact `Fraction` values	every nonzero value	`{"kind": "rationals"}`
`ResidueRingDomain(M)`	canonical residues modulo M	residues coprime to M	`{"kind": "residue_ring", "modulus": M}`
`PrimeFieldDomain(p)`	canonical residues modulo prime p	every nonzero residue	`{"kind": "prime_field", "prime": p}`
`TruncatedPadicDomain(p, r)`	canonical residues modulo p^r	residues not divisible by p	`{"kind": "truncated_padic", "prime": p, "precision": r}`

Every built-in domain provides these members:

Member	Exact contract
`.zero`, `.one`, `.name`	canonical identities and display name
`.coerce(value)`	normalize a supported input into the domain
`.add(left, right)`	exact domain sum
`.subtract(left, right)`	exact domain difference
`.multiply(left, right)`	exact domain product
`.is_unit(value)`	test whether `.inverse` is defined
`.inverse(value)`	exact inverse or `ValidationError`
`.equal(left, right)`	equality after coercion
`.encode(value)`, `.decode(payload)`	data-only element representation
`.descriptor()`	closed domain description

`TruncatedPadicDomain` additionally provides

```
valuation(value: int) -> int | float
```

and returns positive infinity for the zero residue.

21.1.2 Exact domain example

```
from fractions import Fraction

from komorebi_math.generating import (
    IntegerDomain,
    PrimeFieldDomain,
    RationalDomain,
    ResidueRingDomain,
    TruncatedPadicDomain,
)

integers = IntegerDomain()
rationals = RationalDomain()
residues = ResidueRingDomain(12)
field = PrimeFieldDomain(7)
padic = TruncatedPadicDomain(3, 4)

print(integers.multiply(6, -7))
print(rationals.inverse(Fraction(3, 5)))
print(residues.multiply(7, 5))
print(field.inverse(3))
print(padic.modulus, padic.valuation(27), padic.valuation(0))
```

Output:

```
-42
5/3
11
5
81 3 inf
```

Coercing a Python float into a rational domain preserves the exact binary floating-point value. Use integers, strings accepted by `Fraction`, or explicit `Fraction` objects when a small intended rational denominator matters.

21.2 `WeightedMonomialBasis`

Let $\gamma_0 = 1$ and require $\gamma_n \neq 0$ for every degree that will be used. The normalized basis elements are

$$e_n(z) = \frac{z^n}{\gamma_n}.$$

Equivalently, put $w_n = \gamma_n^{-1}$. Then the same normalized series can be written as:

$$F(z) = \sum_{n \geq 0} a_n w_n z^n.$$

The stored value is a_n, not the ordinary monomial coefficient. The ordinary coefficient is $a_n w_n = a_n / \gamma_n$.

For example, in the exponential basis the stored value at degree two is twice the ordinary quadratic coefficient. The ordinary basis leaves that coefficient unchanged. When comparing outputs from two providers, first compare their ordinary coefficients or explicitly convert them to the same basis.

21.2.1 Constructor and members

```
WeightedMonomialBasis(
    name: str,
    gamma: Callable[[int], int | Fraction],
    *,
    descriptor: BasisDescriptor | None = None,
)
```

Member	Signature or value	Meaning
`.name`	`str`	human-facing basis label
`.descriptor`	`BasisDescriptor | None`	closed portable description, when available
`.portable`	`bool`	whether a descriptor exists
`.gamma(n)`	`(n: int) -> Fraction`	exact γ_n, cached by degree
`.lowering_weight(n)`	`(n: int) -> Fraction`	γ_n / γ_{n-1} for $n \geq 1$
`.structure_constant(n, k)`	`(n: int, k: int) -> Fraction`	$\gamma_n / (\gamma_k \gamma_{n-k})$
`.packet_transfer_weight(...)`	`(modulus: int, residue: int, packet_index: int, target_basis: WeightedMonomialBasis) -> Fraction`	residue-packet normalization ratio
`.semantically_equivalent(other)`	`(other: WeightedMonomialBasis) -> bool`	descriptor equality, or identity for callback bases
`.equivalent_through(other, degree)`	`(other: WeightedMonomialBasis, degree: int) -> bool`	exact finite table comparison
`.finite_certificate(degree)`	`(degree: int) -> BasisCertificate`	finite nonzero and integrality audit
`.require_descriptor()`	`() -> BasisDescriptor`	descriptor or `ValueError` for a live callback

Member	Signature or value	Meaning

The exact product structure constant is

$$C(n,k) = \frac{\gamma_n}{\gamma_k \gamma_{n-k}}.$$

The custom lowering operator is specified on monomials by

$$D_\gamma z^n = \frac{\gamma_n}{\gamma_{n-1}} z^{n-1}, \qquad D_\gamma e_n = e_{n-1}.$$

`BasisCertificate` is the frozen record

```
BasisCertificate(
    name: str,
    degree: int,
    nonzero_normalization: bool,
    integral_structure_constants: bool,
    maximum_denominator: int,
    maximum_numerator_bits: int,
)
```

It is a finite diagnostic through the requested degree. It is not a theorem about all n.

21.2.2 Descriptor model

```
BasisDescriptor(
    name: str,
    kind: str,
    parameters: tuple[tuple[str, BasisParameter], ...] = (),
)

BasisDescriptor.create(
    name: str,
    kind: str,
    parameters: Mapping[str, BasisParameter] | None = None,
) -> BasisDescriptor

basis_from_descriptor(
    descriptor: BasisDescriptor,
) -> WeightedMonomialBasis
```

Descriptor parameter keys are sorted canonically and must be unique. The decoder accepts only the built-in closed kinds listed in Section 21.3. It does not import a callback by module path or execute source text from a file.

21.3 Ordinary, exponential, and factorial-power bases

The research namespace provides seven constructors:

```
ordinary_basis(
    *, name: str = "ordinary",
) -> WeightedMonomialBasis

exponential_basis(
    *, name: str = "exponential",
) -> WeightedMonomialBasis

factorial_power_basis(
    power: int,
    *, name: str | None = None,
) -> WeightedMonomialBasis

q_factorial_basis(
    q: int,
    *, name: str | None = None,
) -> WeightedMonomialBasis

fibonacci_factorial_basis(
    *, name: str = "fibonacci-factorial",
) -> WeightedMonomialBasis

pochhammer_basis(
    alpha: int | Fraction,
    *, name: str | None = None,
) -> WeightedMonomialBasis

basis_from_values(
    name: str,
    values: Iterable[int | Fraction],
) -> WeightedMonomialBasis
```

Constructor	γ_n	Basis element $e_n(z)$	Required parameter
`ordinary_basis()`	1	z^n	none
`exponential_basis()`	$n!$	$z^n/n!$	none
`factorial_power_basis(s)`	$(n!)^s$	$z^n/(n!)^s$	integer $s \geq 1$
`q_factorial_basis(q)`	$[n]_q!$	$z^n/[n]_q!$	integer $q \geq 1$
`fibonacci_factorial_basis()`	$\prod_{j=1}^{n} F_j$	$z^n/\prod_{j=1}^{n} F_j$	none
`pochhammer_basis(alpha)`	$(\alpha)_n$	$z^n/(\alpha)_n$	rational α not a nonpositive integer
`basis_from_values(name, table)`	supplied table entry	z^n/γ_n	finite nonzero table beginning with one

Here the q-factorial convention is

$$[j]_q = \frac{q^j - 1}{q - 1}, \qquad [n]_q! = \prod_{j=1}^{n} [j]_q,$$

with the $q = 1$ branch defined as $n!$. Every constructor in the table returns a descriptor-backed portable basis.

21.3.1 Normalization and integrality example

```
from komorebi_math.generating import factorial_power_basis

basis = factorial_power_basis(2)
audit = basis.finite_certificate(8)

print(tuple(str(basis.gamma(n)) for n in range(5)))
print(basis.structure_constant(5, 2))
print(audit.integral_structure_constants)
```

Output:

```
('1', '1', '4', '36', '576')
100
True
```

For a factorial-power basis,

$$C(n,k) = \binom{n}{k}^{s}.$$

This identity is why a same-basis divided-power product can remain meaningful in characteristic p, even when individual factorial normalizations are not invertible modulo p.

21.3.2 Live callback and finite-table boundary

A callback basis is valid for live finite computation:

```
from komorebi_math.generating import WeightedMonomialBasis

live = WeightedMonomialBasis("linear", lambda n: n + 1)
print(live.portable)
```

Output:

```
False
```

The basis itself cannot be serialized because its callback has no trusted data-only meaning. A `FormalSeries` that uses it can be serialized. The codec captures the finite normalization table only through the series degree. After loading, asking the restored basis for a higher degree raises `IndexError`.

`basis_from_values` has the same finite boundary by construction. It does not extrapolate a missing rule from a normalization prefix.

21.4 `FormalSeries` and finite series records

The package has two related records. Select them by arithmetic contract, not by name similarity.

Record	Coefficient domain	Mutability	Budget	Best use
`FormalSeries`	implicit $\mathbb{Q}$ through `Fraction`	public attributes are mutable	none	stable exact rational series API
`FiniteNormalizedSeries`	explicit `CoefficientDomain`	frozen dataclass	explicit `GeneratingBudget`	modular, field, p-adic, and resource-bounded work

Both are finite. An input iterable of length $d+1$ represents degrees zero through d. An empty iterable becomes the zero series of degree zero.

21.4.1 `FormalSeries` constructor and members

```
FormalSeries(
    basis: WeightedMonomialBasis,
    coefficients: Iterable[int | Fraction],
)
```

Member	Signature	Result
`.degree`	property	`len(coefficients) - 1`
`.truncate(degree)`	`(degree: int) -> FormalSeries`	cut or zero-pad through `degree`
`.ordinary_coefficients(degree=None)`	exact tuple	a_n/γ_n
`.from_ordinary_coefficients(basis, coefficients)`	class method	multiply ordinary coefficient c_n by γ_n
`.convert(basis)`	`(basis) -> FormalSeries`	same finite polynomial in a new normalization
`.add`, `.subtract`, `.scale`	exact finite arithmetic	normalized coefficients
`.multiply`, `.hadamard`	Cauchy or coefficientwise product	`FormalSeries`
`.ordinary_derivative`, `.ordinary_integral`	ordinary calculus	`FormalSeries` in the same basis
`.custom_lower()`	coefficient shift	$D_\gamma F$
`.reciprocal`, `.compose`, `.exp`, `.log`, `.power`	exact truncated formal operations	`FormalSeries`
`.project`, `.packet`, `.packet_identity`	residue-lane operations	series or `PacketIdentity`
`.evaluate(z)`	`(z: complex) -> complex`	numerical evaluation of the finite ordinary polynomial

Operators `+`, `-`, `*`, scalar multiplication, and integer `**` delegate to the named methods. `series[n]` returns the stored normalized coefficient.

The exact named-method signatures are:

```
truncate(degree: int) -> FormalSeries
ordinary_coefficients(degree: int | None = None) -> tuple[Fraction, ...]
from_ordinary_coefficients(
    basis: WeightedMonomialBasis,
    coefficients: Iterable[int | Fraction],
) -> FormalSeries
convert(basis: WeightedMonomialBasis) -> FormalSeries
add(other: FormalSeries, *, degree: int | None = None) -> FormalSeries
subtract(other: FormalSeries, *, degree: int | None = None) -> FormalSeries
scale(scalar: int | Fraction) -> FormalSeries
multiply(other: FormalSeries, *, degree: int | None = None) -> FormalSeries
hadamard(other: FormalSeries, *, degree: int | None = None) -> FormalSeries
ordinary_derivative() -> FormalSeries
ordinary_integral(constant: int | Fraction = 0) -> FormalSeries
custom_lower() -> FormalSeries
reciprocal(*, degree: int | None = None) -> FormalSeries
compose(inner: FormalSeries, *, degree: int | None = None) -> FormalSeries
exp(*, degree: int | None = None) -> FormalSeries
log(*, degree: int | None = None) -> FormalSeries
power(exponent: int, *, degree: int | None = None) -> FormalSeries
project(modulus: int, residue: int) -> FormalSeries
packet(
```

```
    modulus: int,
    residue: int,
    *,
    target_basis: WeightedMonomialBasis,
) -> FormalSeries
packet_identity(
    modulus: int,
    residue: int,
    *,
    target_basis: WeightedMonomialBasis,
) -> PacketIdentity
evaluate(z: complex) -> complex
```

`PacketIdentity` records the modulus, residue, source and target basis names, source and packet degrees, and an exact Boolean `valid` field. The packet system is developed in Chapter 23.

21.4.2 `FiniteNormalizedSeries` constructor and members

```
FiniteNormalizedSeries(
    basis: WeightedMonomialBasis,
    domain: CoefficientDomain[Any],
    coefficients: Iterable[Any],
    *,
    budget: GeneratingBudget = DEFAULT_GENERATING_BUDGET,
)
```

Member	Signature	Result
`.degree`	property	largest represented degree
`.coefficient(index)`	`(index: int) -> Any`	stored value, or domain zero beyond the degree
`.truncate(degree)`	exact cut or zero padding	`FiniteNormalizedSeries`
`.ordinary_coefficients()`	no degree argument	coefficients after division by γ_n in the domain
`.from_ordinary_coefficients(...)`	class method with basis, domain, coefficients, and optional budget	normalized record
`.convert(target_basis)`	returns `(series, SeriesTransferCertificate)`	verified finite basis transfer
`.add`, `.subtract`, `.scale`, `.multiply`	exact domain arithmetic	budget-preserving record
`.reciprocal`, `.compose`, `.exp`, `.log`	formal operations with unit checks	budget-preserving record
`.project(modulus, residue)`	retain one residue lane	record in the same basis and domain
`.custom_lower()`	shift normalized coefficients	record in the same basis and domain
`.certificate()`	`Certificate`	exact construction certificate

`FiniteNormalizedSeries` intentionally has no numerical `.evaluate`, ordinary derivative or integral helper, Hadamard product, packet extraction, or general integer power method in `1.0.0b4`. Convert to a supported representation or compose the available primitives explicitly rather than assuming parity with `FormalSeries`.

Its exact named-method signatures are:

```
coefficient(index: int) -> Any
truncate(degree: int) -> FiniteNormalizedSeries
ordinary_coefficients() -> tuple[Any, ...]
from_ordinary_coefficients(
    basis: WeightedMonomialBasis,
    domain: CoefficientDomain[Any],
```

```
    coefficients: Iterable[Any],
    *,
    budget: GeneratingBudget = DEFAULT_GENERATING_BUDGET,
) -> FiniteNormalizedSeries
convert(
    target_basis: WeightedMonomialBasis,
) -> tuple[FiniteNormalizedSeries, SeriesTransferCertificate]
add(
    other: FiniteNormalizedSeries,
    *,
    degree: int | None = None,
) -> FiniteNormalizedSeries
subtract(
    other: FiniteNormalizedSeries,
    *,
    degree: int | None = None,
) -> FiniteNormalizedSeries
scale(scalar: Any) -> FiniteNormalizedSeries
multiply(
    other: FiniteNormalizedSeries,
    *,
    degree: int | None = None,
) -> FiniteNormalizedSeries
reciprocal(*, degree: int | None = None) -> FiniteNormalizedSeries
compose(
    inner: FiniteNormalizedSeries,
    *,
    degree: int | None = None,
) -> FiniteNormalizedSeries
exp(*, degree: int | None = None) -> FiniteNormalizedSeries
log(*, degree: int | None = None) -> FiniteNormalizedSeries
project(modulus: int, residue: int) -> FiniteNormalizedSeries
custom_lower() -> FiniteNormalizedSeries
certificate() -> Certificate
```

21.4.3 Transfer certificate

```
SeriesTransferCertificate(
    valid: bool,
    degree: int,
    source_basis: str,
    target_basis: str,
    domain: str,
    failures: tuple[int, ...] = (),
)
```

`.require()` raises `ValidationError` when `valid` is false. `.certificate()` lifts the record into the common theorem-bearing `Certificate` format under theorem identifier `NGF.UNIVERSAL_PRODUCT_AND_TRANSFER`.

For a transfer from normalization γ to η,

$$b_n = a_n \frac{\eta_n}{\gamma_n}.$$

The domain-aware implementation also checks the reverse ratio. If a ratio cannot be coerced because its denominator is a nonunit, conversion raises `ValidationError` before it can return a certificate.

21.4.4 Persistence

Import the general portable helpers from `komorebi_math.export` or their stable counterparts:

```
dumps_portable(
    value: Any,
    *,
    indent: int | None = 2,
    max_depth: int = 128,
) -> str

loads_portable(
    text: str | bytes,
    *,
    max_depth: int = 128,
) -> Any

save_portable(
    value: Any,
    path: str | os.PathLike[str],
    *,
    overwrite: bool = False,
    indent: int | None = 2,
    max_depth: int = 128,
) -> Path

load_portable(
    path: str | os.PathLike[str],
    *,
    max_depth: int = 128,
) -> Any
```

Object	Codec tag	Version	Persistence note
`BasisDescriptor`	`generating.basis_descriptor`	1	closed descriptor
`WeightedMonomialBasis`	`generating.weighted_monomial_basis`	1	requires a descriptor
`GeneratingBudget`	`generating.budget`	1	all eight limits retained
`FormalSeries`	`generating.formal_series`	1	callback basis becomes a finite table descriptor
`FiniteNormalizedSeries`	`generating.finite_normalized_series`	1	embeds basis, domain descriptor, coefficients, and budget

Domain objects do not have independent portable codecs in `1.0.0b4`. Their descriptors are persisted inside `FiniteNormalizedSeries`. Calling `dumps_portable(RationalDomain())` directly raises `ValidationError`.

```
from komorebi_math.export import dumps_portable, loads_portable
from komorebi_math.generating import (
    FiniteNormalizedSeries,
    RationalDomain,
    factorial_power_basis,
)

source = FiniteNormalizedSeries(
    factorial_power_basis(2),
    RationalDomain(),
    (1, 2, -3, 5),
```

```
)
restored = loads_portable(dumps_portable(source))

assert restored == source
print(restored.domain.descriptor())
print(restored.budget == source.budget)
```

Output:

```
{'kind': 'rationals'}
True
```

The portable envelope is canonical data with a payload digest. It does not execute arbitrary class names or callback imports while decoding.

21.5 Basis conversion, multiplication, differentiation, and composition

Let

$$F(z) = \sum_{n=0}^{d} a_n e_n(z), \qquad G(z) = \sum_{n=0}^{e} b_n e_n(z).$$

Addition is coefficientwise after basis conversion. Multiplication uses the weighted Cauchy law

$$[FG]_n = \sum_{k=0}^{n} \frac{\gamma_n}{\gamma_k \gamma_{n-k}} a_k b_{n-k}.$$

The ordinary derivative and integral, expressed back in the same normalized basis, are

$$[F']_n = (n+1)\frac{\gamma_n}{\gamma_{n+1}} a_{n+1},$$

and

$$\left[\int F(z)\,dz\right]_{n+1} = a_n \frac{\gamma_{n+1}}{(n+1)\gamma_n}.$$

The custom lowering operator is different from the ordinary derivative unless $\gamma_n = n!$:

$$[D_\gamma F]_n = a_{n+1}.$$

21.5.1 Exact rational workflow

```
from fractions import Fraction

from komorebi_math.generating import (
    FormalSeries,
    exponential_basis,
    factorial_power_basis,
    ordinary_basis,
)

basis = factorial_power_basis(2)
f = FormalSeries(basis, (1, 2, 3))
g = FormalSeries.from_ordinary_coefficients(basis, (0, 1, 1))

product = f.multiply(g, degree=4)
converted = f.convert(exponential_basis())
derivative = f.ordinary_derivative()
integral = f.ordinary_integral(Fraction(1, 2))
```

```
outer = FormalSeries(ordinary_basis(), (1, 2, 3))
inner = FormalSeries(ordinary_basis(), (0, 1, 1))
composed = outer.compose(inner, degree=4)

print(tuple(map(str, f.ordinary_coefficients())))
print(tuple(map(str, product.coefficients)))
print(tuple(map(str, converted.coefficients)))
print(tuple(map(str, derivative.coefficients)))
print(tuple(map(str, integral.coefficients)))
print(tuple(map(str, composed.coefficients)))
```

Output:

```
('1', '2', '3/4')
('0', '1', '12', '99', '432')
('1', '2', '3/2')
('2', '3/2')
('1/2', '1', '4', '9')
('1', '2', '5', '6', '3')
```

The conversion changes stored coordinates but preserves ordinary coefficients:

$$(1, 2, 3)_{(n!)^2} \longmapsto \left(1, 2, \frac{3}{2}\right)_{n!}.$$

Both represent $1 + 2z + \frac{3}{4}z^2$.

21.5.2 Formal reciprocal, exponential, logarithm, and power

For a requested degree D:

- `reciprocal(degree=D)` requires a nonzero constant coefficient in `FormalSeries`, or a unit constant in `FiniteNormalizedSeries`.
- `compose(inner, degree=D)` requires the inner ordinary constant coefficient to be zero.
- `exp(degree=D)` requires ordinary constant coefficient zero.
- `log(degree=D)` requires ordinary constant coefficient one.
- `FormalSeries.power(exponent, degree=D)` accepts an integer exponent. Negative exponents first form the reciprocal.

These are formal identities through a finite degree. No analytic branch of the complex logarithm or exponential is selected.

21.5.3 Positive-characteristic product

```
from komorebi_math.generating import (
    FiniteNormalizedSeries,
    PrimeFieldDomain,
    exponential_basis,
)

field = PrimeFieldDomain(5)
basis = exponential_basis()
left = FiniteNormalizedSeries(basis, field, (1, 2, 3, 4, 0, 1))
right = FiniteNormalizedSeries(basis, field, (2, 1, 0, 3, 2, 4))
product = left.multiply(right, degree=5)

product.certificate().require()
print(product.coefficients)
```

Output:

```
(2, 0, 0, 0, 2, 1)
```

The product uses binomial structure constants and does not divide by $n!$. Calling `.ordinary_coefficients()` at or beyond degree p, however, tries to divide by $p!$ in $\mathbb{F}_p$ and correctly raises `ValidationError`. Formal `exp` and `log` similarly require every integer divisor used by the recurrence to be a unit in the selected domain.

21.5.4 Resource contract

```
GeneratingBudget(
    maximum_degree: int = 100_000,
    maximum_coefficient_bits: int = 4_000_000,
    maximum_matrix_dimension: int = 4_096,
    maximum_reachable_states: int = 100_000,
    maximum_local_combinations: int = 1_000_000,
    maximum_modulus: int = 2_000_000,
    maximum_serialized_bytes: int = 67_108_864,
    maximum_recursion_depth: int = 1_000,
)
```

`FiniteNormalizedSeries` enforces `maximum_degree` and `maximum_coefficient_bits` during construction and preserves the same budget through its operations. The other fields are shared with later generating compilers. `FormalSeries` has no budget parameter.

Two limits must be understood precisely:

1. coefficient size is checked on the completed series record, so exact intermediate arithmetic may already have occurred before an oversized result is rejected
2. `maximum_serialized_bytes` is not consulted by the generic portable dump helpers in `1.0.0b4`

Choose a small explicit truncation degree before multiplication, composition, or exact rational exponentiation. A budget is a guard, not a prediction of wall-clock time.

21.5.5 Complexity and storage

Let D be the requested maximum degree. Counts below describe scalar domain operations and omit exact-integer bit complexity, which can dominate.

Operation	Current implementation scale	Stored result
construct, convert, add, subtract, Hadamard, derivative, integral, lower, project	$O(D)$	$O(D)$
basis finite certificate	$O(D^2)$ structure-constant checks	$O(D)$ cached normalizations plus one record
Cauchy multiplication	$O(D^2)$	$O(D)$
reciprocal	$O(D^2)$	$O(D)$
exponential or logarithm	$O(D^2)$	$O(D)$
integer power	$O(D^2 \log \lvert s \rvert)$ for exponent s	$O(D)$
current formal composition	$O(D^3)$	$O(D)$
residue packet or projection	$O(D)$	$O(D)$
portable save or load	linear in encoded coefficient and descriptor payload	linear in payload

The algorithms are dense finite coefficient algorithms. They do not use FFT multiplication, Newton iteration, relaxed power-series evaluation, sparse support, or GPU kernels.

21.5.6 Failure and edge table

Boundary	Behavior in `1.0.0b4`
empty coefficient iterable	canonical zero series of degree zero
negative or Boolean truncation degree	`ValueError` or `ValidationError`
basis normalization is zero at a requested degree	`ZeroDivisionError`
factorial power is not a positive integer	`ValueError`
q is not a positive integer	`ValueError`
Pochhammer α is a nonpositive integer	`ValueError`
finite normalization table is empty, does not begin with one, or contains zero	`ValueError`
finite-table basis queried beyond its table	`IndexError`
reciprocal constant is zero or a domain nonunit	`ZeroDivisionError` or `ValidationError`
composition inner constant is nonzero	`ValueError` or `ValidationError`
formal exponential constant is nonzero	`ValueError` or `ValidationError`
formal logarithm constant is not one	`ValueError` or `ValidationError`
two finite series have different domain descriptors	`ValidationError`
basis-transfer ratio has a nonunit denominator in the selected domain	`ValidationError`
degree or coefficient bits exceed a declared budget	`ResourceLimitError`
callback basis serialized by itself	`ValueError`
coefficient domain serialized by itself	`ValidationError`

There are three source-level limits worth calling out rather than hiding:

- `FormalSeries.compose(inner)` uses `inner.degree` as its default output bound. For an outer series of higher degree, even composition with $inner(z) = z$ drops the outer tail. Pass `degree=` explicitly whenever the desired bound is not exactly the inner degree. The domain-aware `FiniteNormalizedSeries.compose` instead defaults to the maximum of the two input degrees.
- `FormalSeries` and `WeightedMonomialBasis` are not frozen records. Their public attributes can be rebound after construction and thereby bypass the constructor's invariants. Treat constructed instances as read-only.
- The runtime-checkable `CoefficientDomain` protocol verifies member shape, not algebraic laws. The five built-in domains provide the tested contract. A custom domain is the caller's responsibility and has no automatic portable decoder.

The composition boundary is directly observable:

```
from komorebi_math.generating import FormalSeries, ordinary_basis

outer = FormalSeries(ordinary_basis(), (1, 2, 3, 4, 5))
z = FormalSeries(ordinary_basis(), (0, 1))

print(outer.compose(z).coefficients)
print(outer.compose(z, degree=4).coefficients)
```

Output:

```
(Fraction(1, 1), Fraction(2, 1))
(Fraction(1, 1), Fraction(2, 1), Fraction(3, 1), Fraction(4, 1), Fraction(5, 1))
```

21.5.7 What the shipped tests establish

A recorded formal-series replay covered the basis, rational-series, domain, positive-characteristic, budget, persistence, and stable-surface paths:

```
tests/test_generating_domains.py
15 passed

selected tests from tests/test_generating_foundation.py
12 passed, 56 deselected

tests/test_generating_block13.py plus tests/test_block15_stable_api.py
67 passed

python -m komorebi_math verify generating
verification passed 14/14 checks
```

The 94 passing test cases establish:

- descriptor roundtrips for built-in and finite-table bases
- explicit rejection of direct callback-basis persistence
- finite snapshot persistence for a callback-backed `FormalSeries`
- exact arithmetic and unit behavior in all five built-in domains
- weighted products matching independent ordinary Cauchy products
- same-basis exponential products in positive characteristic
- basis transfer, composition, and formal exponential-logarithm roundtrips
- degree and coefficient-bit resource failures
- portable roundtrips preserving basis, domain, coefficients, and budget
- the frozen stable API contract that includes `FormalSeries` and the two stable basis constructors
- the installed normalized-product, explicit-domain, and portable-record verification paths within the complete 14-check generating verifier

The replay does not prove an infinite normalization law, callback purity, analytic convergence, asymptotic complexity, or custom-domain algebraic laws.

Chapter 21 summary. A weighted monomial basis stores a_n against $e_n = z^n/\gamma_n$, so the ordinary coefficient is a_n/γ_n. `FormalSeries` is the stable exact-rational finite record. `FiniteNormalizedSeries` adds a frozen explicit domain, transfer certificates, and resource bounds. Products use $\gamma_n/(\gamma_k\gamma_{n-k})$, ordinary calculus rescales by adjacent normalizations, and composition is performed after exact conversion to ordinary coefficients. Use descriptor-backed bases for reusable persistence, pass an explicit composition degree, require units before division, and remember that every result is a finite formal truncation rather than an analytic infinite series.

22. Normalized generating functions

First route. If you already know a defining rule, choose a provider in Section 22.1 and reproduce one small prefix. If you have only samples, use the recognition section and retain its fit and holdout information. Both routes need an explicit choice between normalized and ordinary coefficients. When saving a reproducible result, Section 22.3.4 shows how to freeze the evaluated prefix together with its normalization.

Chapter 21 introduced the finite series

$$F(z) = \sum_{n=0}^{d} a_n \frac{z^n}{\gamma_n}, \qquad \gamma_0 = 1, \quad \gamma_n \neq 0,$$

and the distinction between the stored normalized coefficient a_n and the ordinary coefficient $c_n = a_n/\gamma_n$. The next layer adds a declarative provider interface around that finite algebra. A provider states where coefficients come from, a section isolates arithmetic lanes, a monoid series changes the index product, an adapter crosses into numerical signal processing, and a frozen record removes executable callbacks before persistence.

The implementation remains finite even when a mathematical definition uses an infinite product or a formal infinite series. Calling `function.series(d)` constructs exactly the prefix through degree d. It does not install a lazy infinite object and it does not certify any identity beyond that prefix.

> **API tier.** Every Chapter 22 symbol is present in the complete `1.0.0b4` public research manifest. None is in the beta or frozen stable manifest. `CustomFunctionSpec`, `FunctionRegistry`, `FrozenGeneratingFunction`, `GeneratingBudget`, and `freeze_custom_function` are also convenience exports from the package root. The complete surface, including recognition, sections, monoids, and FIR adapters, is `komorebi_math.generating`.

The shortest useful import is

```
from komorebi_math.generating import (
    CustomFunctionSpec,
    FunctionRegistry,
    FrozenGeneratingFunction,
    GeneratingBudget,
    algebraic_function,
    audit_section_cycle,
    compose_functions,
    custom_section,
    dirichlet_convolution,
    euler_product_function,
    freeze_custom_function,
    guess_linear_recurrence,
    rational_function,
)
```

No object in this chapter is a computer algebra expression tree. `CustomFunctionSpec` stores a Python coefficient callback, `FrozenGeneratingFunction` stores a finite data snapshot, and the closed-form guess records store bounded candidates. These three roles must not be interchanged.

22.1 Declarative generating-function objects

22.1.1 Core record and registry

The provider record has the exact constructor and public method

```
CustomFunctionSpec(
    name: str,
    basis: WeightedMonomialBasis,
    builder: CoefficientBuilder,
    source_kind: str = "custom",
    metadata: Mapping[str, object] = <factory>,
) -> None

CustomFunctionSpec.series(
    self,
    degree: int,
) -> FormalSeries
```

Its public data members are:

Member	Meaning
`name`	Human-readable provider name
`basis`	Normalized monomial basis used by the returned coefficients
`builder`	Callback receiving d and returning exactly $d + 1$ coefficients
`source_kind`	Descriptive construction label
`metadata`	User or constructor metadata, not a proof object

`series(d)` rejects a negative degree, executes the callback, checks only the returned length, and constructs a `FormalSeries`. The callback is trusted to be deterministic and mathematically correct. It is not sandboxed, memoized, or symbolically inspected.

The registry is an explicit in-memory name table:

```
FunctionRegistry() -> None

FunctionRegistry.register(
    self,
    function: CustomFunctionSpec,
) -> None

FunctionRegistry.get(
    self,
    name: str,
) -> CustomFunctionSpec

FunctionRegistry.names(
    self,
) -> tuple[str, ...]
```

Registration and lookup strip surrounding whitespace and compare lowercase names. The original `function.name` remains on the provider. A second registration under the same normalized name raises `KeyError` instead of silently replacing the first provider. `names()` returns the normalized names in sorted order.

The registry has no global singleton. Applications create and own registry instances explicitly. A registry is neither portable nor thread synchronized. Persist the selected finite functions rather than the registry itself.

22.1.2 Complete constructor ledger

The declarative constructor surface is:

```
explicit_function(
    name: str,
    basis: WeightedMonomialBasis,
    coefficients: tuple[int | Fraction, ...],
) -> CustomFunctionSpec

linear_recurrence_function(
    name: str,
    coefficients: tuple[int | Fraction, ...],
    initial: tuple[int | Fraction, ...],
    *,
    basis: WeightedMonomialBasis | None = None,
) -> CustomFunctionSpec

hypergeometric_function(
    name: str,
    initial: int | Fraction,
    ratio: Callable[[int], int | Fraction],
    *,
    basis: WeightedMonomialBasis | None = None,
) -> CustomFunctionSpec

rational_function(
    name: str,
    numerator: tuple[int | Fraction, ...],
    denominator: tuple[int | Fraction, ...],
    *,
    basis: WeightedMonomialBasis | None = None,
) -> CustomFunctionSpec

algebraic_function(
    name: str,
    polynomial_terms: Mapping[tuple[int, int], int | Fraction],
    initial_value: int | Fraction,
    *,
    basis: WeightedMonomialBasis | None = None,
) -> CustomFunctionSpec

euler_product_function(
    name: str,
    exponents: Mapping[int, int | Fraction]
        | Callable[[int], int | Fraction],
    *,
    basis: WeightedMonomialBasis | None = None,
) -> CustomFunctionSpec

factorial_power_bell_function(
    power: int,
    *,
    name: str | None = None,
) -> CustomFunctionSpec

weighted_factorial_power_bell_function(
    power: int,
    weights: tuple[int | Fraction, ...]
        | Callable[[int], int | Fraction],
    *,
    name: str | None = None,
) -> CustomFunctionSpec
```

```
compose_functions(
    name: str,
    outer: CustomFunctionSpec,
    inner: CustomFunctionSpec,
) -> CustomFunctionSpec
```

The word *coefficient* has two different meanings across these constructors. The distinction follows directly from the normalized basis law.

Constructor	Input rule	Stored output
`explicit_function`	Declared tuple	Normalized a_n directly
`linear_recurrence_function`	Recurrence on the sequence	Normalized a_n directly
`hypergeometric_function`	Ratio of consecutive terms	Normalized a_n directly
`rational_function`	Ordinary rational function $P(z)/Q(z)$	$a_n = \gamma_n c_n$
`algebraic_function`	Ordinary branch $P(z, F(z)) = 0$	$a_n = \gamma_n c_n$
`euler_product_function`	Ordinary Euler product	$a_n = \gamma_n c_n$
factorial-power Bell constructors	Factorial-power Bell recurrence	Native normalized Bell values
`compose_functions`	Ordinary formal composition	Returned in the outer basis

This table matters whenever `basis` is not ordinary. For example, `linear_recurrence_function(..., basis=exponential_basis())` applies the recurrence to the stored exponential coefficients a_n, not to $a_n/n!$. By contrast, `rational_function` first derives the ordinary coefficients of P/Q and then multiplies coefficient n by $n!$.

22.1.3 Explicit, recurrent, and hypergeometric providers

`explicit_function` copies its tuple into exact `Fraction` values. A request through degree d succeeds only when at least $d + 1$ values were declared. The constructor does not assume that omitted trailing coefficients are zero. This makes the captured boundary explicit.

For a recurrence of order r, the coefficients and seeds mean

$$a_{n+r} = \sum_{j=0}^{r-1} c_j a_{n+j}, \qquad (a_0, \dots, a_{r-1}) = \texttt{initial}.$$

The coefficient and initial tuples must be nonempty and have the same length. Generation through degree d takes $O(rd)$ exact scalar operations after the seeds have been converted to fractions.

The hypergeometric provider implements

$$a_0 = s, \qquad a_{n+1} = a_n R(n).$$

The ratio callback is evaluated once at every index from 0 through $d - 1$. A zero term remains zero unless a later callback operation itself fails. The constructor does not prove that $R(n)$ is rational as a function. Each returned value is converted to `Fraction` at execution time.

22.1.4 Rational and algebraic providers

Let

$$P(z) = \sum_{n \geq 0} p_n z^n, \qquad Q(z) = \sum_{k=0}^{q} q_k z^k, \qquad q_0 \neq 0.$$

The rational builder solves

$$c_n = \frac{1}{q_0}\left(p_n - \sum_{k=1}^{\min(n,q)} q_k c_{n-k}\right)$$

and returns $a_n = \gamma_n c_n$. Numerator and denominator tuples use ascending powers. A finite numerator is extended by zeros. A finite denominator defines the recurrence for every requested degree.

The algebraic builder receives a sparse mapping

```
{
    (z_degree, y_degree): coefficient,
    ...
}
```

and interprets it as

$$P(z,y) = \sum_{i,j} p_{ij} z^i y^j.$$

For the requested branch $y(0) = y_0$, construction requires

$$P(0,y_0) = 0, \qquad P_y(0,y_0) \neq 0.$$

The nonzero derivative is the formal implicit-function condition. Coefficients are then solved exactly, one degree at a time. The implementation recomputes truncated powers of the developing branch. With s the largest supplied y-degree, its straightforward cost is $O(sd^3)$ exact scalar operations in the worst case. Large rational numerators can make the bit cost much higher.

The constructor converts both key coordinates with `int(...)`. Callers should therefore supply genuine nonnegative integer pairs. In `1.0.0b4` a nonintegral numeric key can be silently truncated before validation. This is a source-level input-validation limitation, not a supported coercion rule.

22.1.5 Euler products and factorial-power Bell families

The Euler-product provider represents the finite prefix of

$$F(z) = \prod_{m\geq 1} (1 - z^m)^{-c_m}.$$

If a mapping is supplied, absent positive indices have exponent zero. If a callback is supplied, it is queried on positive indices only. The builder uses the logarithmic-derivative recurrence

$$\ell_k = \sum_{d\mid k} d\, c_d, \qquad n f_n = \sum_{k=1}^{n} \ell_k f_{n-k}, \qquad f_0 = 1.$$

The current divisor scan and coefficient recurrence together use $O(d^2)$ exact scalar operations. No convergence claim is needed because the result is a formal finite prefix.

For a positive integer s, `factorial_power_bell_function(s)` uses

$$\gamma_n = (n!)^s$$

and defines

$$\exp\left(\sum_{m\geq 1} \frac{z^m}{(m!)^s}\right) = \sum_{n\geq 0} B_n^{(s)} \frac{z^n}{(n!)^s}.$$

The exact recurrence is

$$B_n^{(s)} = \sum_{m=1}^{n} \binom{n-1}{m-1} \binom{n}{m}^{s-1} B_{n-m}^{(s)}.$$

The weighted constructor inserts a rational block weight w_m into every summand. A finite weight tuple uses entry zero only as an ignored placeholder and must contain entries through the largest requested block size. A callable is live and is queried as the degree grows. The unweighted path is quadratic in the degree at the recurrence level. The weighted implementation builds full Bell-polynomial rows and has cubic scalar-operation growth in the worst case.

22.1.6 Ordinary formal composition

`compose_functions(name, outer, inner)` defines

$$H(z) = F(G(z)).$$

Both providers are evaluated through the requested output degree. The inner series is converted to the outer basis, ordinary composition is performed exactly, and the result remains in the outer basis. The required formal condition is

$$G(0) = 0.$$

The current repeated-power implementation is cubic in the truncation degree. An `explicit_function` used as either operand must declare coefficients all the way through the requested degree, including explicit trailing zeros. Use a rational provider with denominator (1) when a finite polynomial should be extendable by zero coefficients.

22.1.7 Declarative workflow

This example creates rational, algebraic, Euler-product, and composed providers without constructing a symbolic expression tree:

```
from komorebi_math.generating import (
    FunctionRegistry,
    algebraic_function,
    compose_functions,
    euler_product_function,
    rational_function,
)

fibonacci = rational_function(
    "fibonacci",
    (0, 1),
    (1, -1, -1),
)
catalan = algebraic_function(
    "catalan",
    {(0, 0): 1, (0, 1): -1, (1, 2): 1},
    1,
)
partition = euler_product_function(
    "partition",
    lambda _index: 1,
)
geometric = rational_function("geometric", (1,), (1, -1))
z_plus_z2 = rational_function("z+z^2", (0, 1, 1), (1,))
composed = compose_functions(
    "1/(1-z-z^2)",
    geometric,
    z_plus_z2,
)
```

```
registry = FunctionRegistry()
registry.register(fibonacci)
registry.register(catalan)

print(registry.names())
print(tuple(int(v) for v in fibonacci.series(10).coefficients))
print(tuple(int(v) for v in catalan.series(8).coefficients))
print(tuple(int(v) for v in partition.series(10).coefficients))
print(tuple(int(v) for v in composed.series(7).coefficients))
```

Output:

```
('catalan', 'fibonacci')
(0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55)
(1, 1, 2, 5, 14, 42, 132, 429, 1430)
(1, 1, 2, 3, 5, 7, 11, 15, 22, 30, 42)
(1, 1, 2, 3, 5, 8, 13, 21)
```

The final line is Fibonacci-shifted because

$$\frac{1}{1-(z+z^2)} = 1 + z + 2z^2 + 3z^3 + 5z^4 + \cdots.$$

22.1.8 Construction failures and live-provider limits

Condition	Result in `1.0.0b4`
`series(d)` with $d < 0$	`ValueError: degree must be nonnegative`
Builder returns other than $d + 1$ entries	`ValueError`
Explicit provider is requested past its declared tuple	`ValueError: explicit function has insufficient declared coefficients`
Recurrence seed count differs from recurrence order	`ValueError`
Rational denominator is empty or has zero constant term	`ValueError`
Algebraic polynomial is empty	`ValueError`
Initial algebraic value is not on the curve	`ValueError`
Formal implicit derivative vanishes	`ValueError`
Composition inner constant term is nonzero	`ValueError`
Registry normalized name already exists	`KeyError`

Provider metadata is descriptive. It is not validated as a theorem certificate at construction. A callable ratio, exponent rule, weight rule, or custom builder can depend on mutable external state. Reproducible work should freeze the evaluated prefix described in Section 22.3.

22.2 Normalization maps and closed forms

22.2.1 The normalization-change law

Suppose one finite series is represented in two bases,

$$F(z) = \sum_{n=0}^{d} a_n \frac{z^n}{\gamma_n} = \sum_{n=0}^{d} b_n \frac{z^n}{\eta_n}.$$

Then coefficient equality gives the diagonal map

$$b_n = \frac{\eta_n}{\gamma_n} a_n.$$

`FormalSeries.convert(target_basis)` implements this law through ordinary coefficients. It changes representation, not the polynomial F. For a provider, conversion occurs after a prefix has been built:

```
source = function.series(degree)
converted = source.convert(target_basis)
assert converted.ordinary_coefficients() == source.ordinary_coefficients()
```

The constructor table in Section 22.1 determines whether the provider's input sequence is already normalized or is first interpreted as an ordinary function. There is no separate generic `normalize_function` call.

> **Module-name boundary.** The private implementation module `komorebi_math.generating.normalization` contains carry-linear prime-power kernels used by later chapters. It is not the basis-conversion mechanism described here. Use `FormalSeries.convert` for γ-to-η coefficient changes.

Recognition routines accept plain coefficient iterables. They do not receive a basis. Passing `series.coefficients` asks about the normalized sequence (a_n). Passing `series.ordinary_coefficients()` asks about the ordinary sequence (a_n/γ_n). Neither choice is inferred for the caller.

22.2.2 Recognition function signatures

The complete bounded-recognition surface is:

```
guess_linear_recurrence(
    values: Iterable[int | Fraction],
    *,
    max_order: int = 12,
    holdout: int | None = None,
) -> LinearRecurrenceGuess | None

rational_gf_from_recurrence(
    recurrence: LinearRecurrenceGuess,
    initial: Sequence[int | Fraction],
) -> tuple[tuple[Fraction, ...], tuple[Fraction, ...]]

characteristic_closed_form(
    recurrence: LinearRecurrenceGuess,
    initial: Sequence[int | Fraction],
) -> Any | None

guess_hypergeometric_ratio(
    values: Iterable[int | Fraction],
    *,
    max_numerator_degree: int = 4,
    max_denominator_degree: int = 4,
    holdout: int | None = None,
) -> HypergeometricGuess | None
```

```
guess_p_recursive(
    values: Iterable[int | Fraction],
    *,
    max_order: int = 4,
    max_degree: int = 5,
    holdout: int | None = None,
) -> PRecursiveGuess | None

guess_algebraic_gf(
    values: Iterable[int | Fraction],
    *,
    max_x_degree: int = 4,
    max_y_degree: int = 4,
    holdout: int | None = None,
) -> AlgebraicGFGuess | None

guess_quasipolynomial(
    values: Iterable[int | Fraction],
    *,
    max_period: int = 12,
    max_degree: int = 6,
    holdout: int | None = None,
) -> QuasiPolynomialGuess | None
```

Except for `guess_quasipolynomial` and `rational_gf_from_recurrence`, these workflows require the optional SymPy backend either while fitting or while constructing the symbolic rendering. The import of `komorebi_math.generating` itself does not import SymPy eagerly.

22.2.3 Candidate record ledger

All five result types are frozen slotted data classes.

```
LinearRecurrenceGuess(
    order: int,
    coefficients: tuple[Fraction, ...],
    fit_terms: int,
    holdout_terms: int,
    exact_on_prefix: bool,
) -> None

LinearRecurrenceGuess.equation(self) -> str

HypergeometricGuess(
    numerator: tuple[int, ...],
    denominator: tuple[int, ...],
    fit_relations: int,
    holdout_relations: int,
    exact_on_prefix: bool,
) -> None

HypergeometricGuess.ratio_expression(self) -> Any

PRecursiveGuess(
    order: int,
    degree: int,
    polynomials: tuple[tuple[int, ...], ...],
    fit_relations: int,
    holdout_relations: int,
    exact_on_prefix: bool,
) -> None
```

```
PRecursiveGuess.expression(self) -> Any

AlgebraicGFGuess(
    x_degree: int,
    y_degree: int,
    monomials: tuple[tuple[int, int, int], ...],
    fit_coefficients: int,
    holdout_coefficients: int,
    exact_on_prefix: bool,
) -> None

AlgebraicGFGuess.expression(self) -> Any

QuasiPolynomialGuess(
    period: int,
    degree: int,
    residue_values: tuple[tuple[Fraction, ...], ...],
    fit_terms: int,
    holdout_terms: int,
    exact_on_prefix: bool,
) -> None
```

Their mathematical payloads are:

Record	Candidate law
`LinearRecurrenceGuess`	$a_{n+r} = \sum_{j=0}^{r-1} c_j a_{n+j}$
`HypergeometricGuess`	$a_{n+1}/a_n = P(n)/Q(n)$
`PRecursiveGuess`	$\sum_{j=0}^{r} P_j(n) a_{n+j} = 0$
`AlgebraicGFGuess`	$P(x, F(x)) = 0$
`QuasiPolynomialGuess`	One Newton-form polynomial on each residue class

`monomials` stores triples (i, j, c) for $cx^i y^j$. `polynomials` stores one ascending coefficient tuple for each shift $P_j(n)$. `residue_values` stores forward-difference Newton coefficients, not sampled function values despite the field name.

22.2.4 Fit, holdout, and proof boundary

The fit portion selects a candidate. The holdout portion is then checked, and the implementation finally checks the candidate against the entire supplied prefix. Therefore `exact_on_prefix=True` means exact rational equality at all provided indices covered by the candidate law.

It does **not** mean:

- the law is unique,
- the law holds for the next coefficient,
- the supplied data came from the guessed model,
- an infinite formal identity has been proved.

An infinite claim needs independent information such as a defining recurrence, a functional equation, or a symbolic proof. The candidate record intentionally does not expose `certificate().require()`.

The constant-recurrence bridge forms an ordinary rational generating function. For

$$a_{n+r} = \sum_{j=0}^{r-1} c_j a_{n+j},$$

the denominator is

$$Q(z) = 1 - c_{r-1}z - c_{r-2}z^2 - \cdots - c_0 z^r.$$

The numerator is the truncated product $Q(z)F(z)$ through degree $r-1$. `characteristic_closed_form` separately asks SymPy to solve the recurrence with the supplied initial values. It can return `None` when SymPy has no supported form.

22.2.5 Exact recognition workflow

```
from komorebi_math.generating import (
    guess_algebraic_gf,
    guess_hypergeometric_ratio,
    guess_linear_recurrence,
    rational_gf_from_recurrence,
)

fibonacci = (0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89)
recurrence = guess_linear_recurrence(fibonacci, max_order=4)
assert recurrence is not None
numerator, denominator = rational_gf_from_recurrence(
    recurrence,
    fibonacci,
)

central_binomial = (
    1, 2, 6, 20, 70, 252, 924, 3432, 12870, 48620,
)
ratio = guess_hypergeometric_ratio(central_binomial)
assert ratio is not None

catalan = (
    1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786,
)
algebraic = guess_algebraic_gf(
    catalan,
    max_x_degree=2,
    max_y_degree=3,
)
assert algebraic is not None

print(recurrence.equation())
print(tuple(int(v) for v in numerator))
print(tuple(int(v) for v in denominator))
print(ratio.numerator, ratio.denominator)
print(algebraic.monomials)
```

Output:

```
a[n+2] = (1) a[n+0] + (1) a[n+1]
(0, 1)
(1, -1, -1)
(2, 4) (1, 1)
((0, 0, 1), (0, 1, -1), (1, 2, 1))
```

The last two records encode

$$\frac{a_{n+1}}{a_n} = \frac{4n+2}{n+1}$$

and

$$1 - F(x) + xF(x)^2 = 0.$$

A quasipolynomial uses finite differences on each lane. For example, the sequence

$$a_n = \left\lfloor \frac{n}{3} \right\rfloor^2 + (n \bmod 3)$$

is recovered with period 3, degree 2, and residue tuples

$$(0, 1, 2), \qquad (1, 1, 2), \qquad (2, 1, 2).$$

These are Newton forward differences because

$$q^2 = \binom{q}{1} + 2\binom{q}{2}.$$

22.2.6 Recognition cost and failure behavior

Operation	Principal finite cost
`rational_gf_from_recurrence`	$O(r^2)$ exact arithmetic
`guess_quasipolynomial`	Repeated lane difference tables over tested periods and degrees
Constant recurrence guess	Repeated exact dense linear solves up to `max_order`
Hypergeometric guess	Exact nullspaces across numerator and denominator degree pairs
P-recursive guess	Exact nullspaces of size $(r+1)(s+1)$
Algebraic GF guess	Truncated powers plus exact nullspaces over candidate monomials
`characteristic_closed_form`	Backend-dependent symbolic recurrence solving

The symbolic searches can grow much faster than the prefix length suggests because rational matrix entries grow in bit length. The Chapter 22 guessers do not accept a `GeneratingBudget`. Bound the prefix and all `max_*` parameters before calling them.

The shipped boundary behavior is:

Condition	Result
SymPy is unavailable for a symbolic path	`BackendUnavailableError`
Prefix is shorter than the routine's minimum	`None`
Hypergeometric prefix has a zero before the final term	`None`
No model is found within the declared search box	`None`
SymPy cannot solve a recognized constant recurrence	`None` from `characteristic_closed_form`
Initial tuple is shorter than recurrence order	`ValueError` in rational GF bridge, or an indexing failure in the symbolic bridge

> **Known validation gap.** In `1.0.0b4` the recognition functions do not validate `holdout` or all `max_*` arguments as nonnegative bounded integers. A negative holdout can reach an internal indexing failure, and the result data-class constructors accept inconsistent fields when called directly. Treat constructor defaults or caller-validated positive bounds as the supported path. This does not affect the validated default workflows.

22.3 Sections, monoids, adapters, and portable records

22.3.1 Custom exponential and residue sections

For a declared basis γ, the custom exponential is

$$E_\gamma(z) = \sum_{n=0}^{d} \frac{z^n}{\gamma_n}.$$

Its normalized coefficient tuple is simply $(1, \dots, 1)$. The section with modulus m and residue r is

$$S_{m,r}^{(\gamma)}(z) = \sum_{\substack{0 \le n \le d \\ n \equiv r \pmod m}} \frac{z^n}{\gamma_n}.$$

The signed version multiplies the term $n = mq + r$ by $(-1)^q$.

The complete API is:

```
custom_exponential(
    basis: WeightedMonomialBasis,
    degree: int,
) -> FormalSeries

custom_section(
    basis: WeightedMonomialBasis,
    modulus: int,
    residue: int,
    degree: int,
    *,
    signed: bool = False,
) -> FormalSeries

root_filter_value(
    basis: WeightedMonomialBasis,
    modulus: int,
    residue: int,
    degree: int,
    z: complex,
) -> complex

audit_section_cycle(
    basis: WeightedMonomialBasis,
    *,
    modulus: int,
    degree: int,
    signed: bool = False,
    z: complex = 0.37 + 0.11j,
) -> SectionCycleAudit
```

With

$$\omega = e^{2\pi i/m},$$

the finite root filter is

$$S_{m,r}^{(\gamma)}(z) = \frac{1}{m} \sum_{j=0}^{m-1} \omega^{-rj} E_{\gamma}(\omega^{j} z).$$

`root_filter_value` evaluates the right side in complex floating point. `custom_section(...).evaluate(z)` evaluates the left side using the same finite truncation. Their small residual is numerical evidence for this finite projection, not an exact `Certificate`.

The custom lowering operator from Chapter 21 shifts normalized coefficients. It sends one unsigned section to the preceding residue section, with a wrap from residue zero to residue $m - 1$. One complete unsigned cycle returns the truncated section. A signed cycle returns its negative.

The audit record is

```
SectionCycleAudit(
    basis: str,
    modulus: int,
    degree: int,
    signed: bool,
    one_step_failures: int,
    full_cycle_failures: int,
    root_filter_residual: float,
) -> None
```

The one-step and full-cycle counts cover every residue. The root-filter residual inside `audit_section_cycle` checks the unsigned residue-zero section only, even when `signed=True`. The `signed` audit flag therefore changes the lowering-cycle checks but not the root-filter comparison.

Example:

```
from komorebi_math.generating import (
    audit_section_cycle,
    custom_section,
    factorial_power_basis,
)

basis = factorial_power_basis(2)
section = custom_section(basis, 3, 1, 10)
signed = custom_section(basis, 3, 1, 10, signed=True)
audit = audit_section_cycle(basis, modulus=3, degree=20)

print(tuple(int(v) for v in section.coefficients))
print(tuple(int(v) for v in signed.coefficients))
print(audit.one_step_failures, audit.full_cycle_failures)
print(audit.root_filter_residual < 1e-12)
```

Output:

```
(0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 1)
(0, 1, 0, 0, -1, 0, 0, 1, 0, 0, -1)
0 0
True
```

`custom_section` validates $m \geq 1$, $0 \leq r < m$, and $d \geq 0$. `root_filter_value` does not repeat the modulus and residue validation in `1.0.0b4`. A zero modulus reaches a `ZeroDivisionError`. Validate the same domain before calling the numerical helper directly.

22.3.2 Finite monoid series

A monoid series replaces addition of indices by a caller-declared associative operation $\star$. For finite coefficient maps a and b,

$$(a * b)(u) = \sum_{x \star y = u} a(x) b(y).$$

The generic record and its members are:

```
FiniteMonoidSeries(
    coefficients: Mapping[Index, Fraction],
    combine: Callable[[Index, Index], Index],
    identity: Index,
    name: str = "monoid series",
) -> None

FiniteMonoidSeries.coefficient(
    self,
    index: Index,
) -> Fraction

FiniteMonoidSeries.multiply(
    self,
    other: FiniteMonoidSeries[Index],
    *,
    admissible: Callable[[Index], bool] | None = None,
) -> FiniteMonoidSeries[Index]
```

`admissible` filters combined output indices. It is the explicit finite support boundary for products that would otherwise grow beyond the desired range.

The built-in integer-index helpers are:

```
additive_series(
    values: Iterable[int | Fraction],
) -> FiniteMonoidSeries[int]

multiplicative_series(
    values: Iterable[int | Fraction],
) -> FiniteMonoidSeries[int]

dirichlet_convolution(
    left: Iterable[int | Fraction],
    right: Iterable[int | Fraction],
    *,
    count: int,
) -> tuple[Fraction, ...]

prime_valuation_packets(
    values: Iterable[int | Fraction],
    prime: int,
) -> dict[int, tuple[tuple[int, Fraction], ...]]
```

`additive_series` assigns entries to indices $0, 1, \ldots$ and uses index addition. `multiplicative_series` assigns entries to $1, 2, \ldots$ and uses index multiplication. The direct Dirichlet helper returns

$$(a * b)(n) = \sum_{d \mid n} a(d) b(n/d), \qquad 1 \le n \le N.$$

`prime_valuation_packets` writes an index as $n = p^v u$ and groups (u, a_n) by v.

```
from komorebi_math.generating import (
    additive_series,
    dirichlet_convolution,
    prime_valuation_packets,
)

ordinary = additive_series((1, 2, 3))
square = ordinary.multiply(ordinary, admissible=lambda n: n <= 4)
divisors = dirichlet_convolution(
    (1, 1, 1, 1, 1, 1),
    (1, 1, 1, 1, 1, 1),
    count=6,
)
packets = prime_valuation_packets(
    (10, 20, 30, 40, 50, 60, 70, 80),
    2,
)

print(tuple(square.coefficient(n) for n in range(5)))
print(tuple(int(v) for v in divisors))
print(tuple((v, len(packet)) for v, packet in packets.items()))
```

Output:

```
(Fraction(1, 1), Fraction(4, 1), Fraction(10, 1), Fraction(12, 1), Fraction(9, 1))
(1, 2, 2, 3, 2, 4)
((0, 4), (1, 2), (2, 1), (3, 1))
```

Two source-level boundaries are important:

- `FiniteMonoidSeries` trusts `combine` to be associative and to use the declared identity. `multiply` checks that the two identity values agree, but it does not verify that the two `combine` callables define the same operation. It always uses the left operand's callback.
- `prime_valuation_packets` checks only that its parameter is at least two. It does not perform a primality test. A composite value therefore produces a repeated-divisibility grouping, not a prime-adic valuation theorem.

The coefficient mapping is stored by reference inside a frozen data class. A mutable mapping can still be mutated externally. Use an immutable mapping discipline when the record is shared. These monoid records and callbacks do not have portable codecs.

22.3.3 Formal-series to FIR adapters

A finite ordinary polynomial

$$F(z) = \sum_{n=0}^{d} c_n z^n$$

is interpreted as the causal transfer polynomial

$$H(z) = \sum_{n=0}^{d} c_n z^{-n}.$$

The bridge converts normalized coefficients to ordinary coefficients first. Its complete API is:

```
series_to_fir_taps(
    series: FormalSeries,
    *,
    degree: int | None = None,
    dtype: Any = float,
) -> np.ndarray

series_to_fir_adapter(
    series: FormalSeries,
    *,
    degree: int | None = None,
    dtype: Any = float,
) -> FIRSeriesAdapter

fir_taps_to_series(
    taps: Any,
    *,
    basis: WeightedMonomialBasis | None = None,
    max_denominator: int | None = None,
) -> FormalSeries
```

The adapter record is:

```
FIRSeriesAdapter(
    series: FormalSeries,
    taps: np.ndarray,
    variable_substitution: str =
        "formal z maps to delay z^(-1)",
) -> None

FIRSeriesAdapter.response(
    self,
    *,
    points: int = 512,
    sample_rate: float = 1.0,
    whole: bool = False,
) -> FrequencyResponse

FIRSeriesAdapter.certificate(
    self,
    *,
    tolerance: float = 0.0,
) -> Certificate
```

The `Certificate` has `Exactness.NUMERICAL` and theorem identifier `GF-SIGNAL-FIR-BRIDGE`. It compares the converted taps against floating representations of the ordinary coefficients. A request beyond the stored series degree pads with zeros because the series is explicitly truncated to the selected degree.

The reverse adapter converts a floating value through its decimal string. `max_denominator` then optionally applies `Fraction.limit_denominator`. This is a declared rationalization boundary, not recovery of an unknown exact number.

> **Lossy-dtype boundary.** `series_to_fir_taps` accepts caller-selected NumPy dtypes. An integer dtype can truncate rational coefficients without raising. `series_to_fir_adapter(...).certificate()` detects the resulting residual, but the bare tap conversion has no certificate. Use a floating or complex dtype and require the adapter certificate when fidelity matters.

Complex taps with any nonzero imaginary part cannot be converted to `FormalSeries` because the current exact series scalar domain is rational and real. Empty, multidimensional, or nonfinite tap arrays are rejected.

The normalized coefficients and ordinary taps are visibly different in an exponential basis:

```
from komorebi_math.generating import (
    FormalSeries,
    exponential_basis,
    fir_taps_to_series,
    series_to_fir_adapter,
)

basis = exponential_basis()
source = FormalSeries(basis, (0, 1, 2, 12))
adapter = series_to_fir_adapter(source)
restored = fir_taps_to_series(
    adapter.taps,
    basis=basis,
    max_denominator=1000,
)

print(adapter.taps.tolist())
print(adapter.certificate().valid)
print(restored.coefficients)
print(adapter.response(points=16).response.shape)
```

Output:

```
[0.0, 1.0, 1.0, 2.0]
True
(Fraction(0, 1), Fraction(1, 1), Fraction(2, 1), Fraction(12, 1))
(16,)
```

The tap vector is $(a_n/n!)$, while conversion back into the exponential basis restores the original normalized tuple.

22.3.4 Frozen providers and portable records

Executable callbacks are not portable data. The finite handoff record is:

```
FrozenGeneratingFunction(
    name: str,
    basis_name: str,
    normalization: tuple[Fraction, ...],
    coefficients: tuple[Fraction, ...],
    source_kind: str,
    metadata: Mapping[str, Any] = <factory>,
) -> None

FrozenGeneratingFunction.degree: int

FrozenGeneratingFunction.series(
    self,
    degree: int | None = None,
) -> FormalSeries

freeze_custom_function(
    function: CustomFunctionSpec,
    degree: int,
    *,
    include_metadata: bool = True,
) -> FrozenGeneratingFunction

register_generating_codecs() -> None
```

The frozen record stores both the coefficient prefix and

$$(\gamma_0, \dots, \gamma_d).$$

Reconstruction therefore needs neither the original basis callback nor the function builder. Validation requires nonempty names, at least one coefficient, equal normalization and coefficient lengths, $\gamma_0 = 1$, nonzero normalizations, and portable metadata.

`dumps_portable` and `loads_portable` register the generating codecs automatically on first use. Most applications do not call `register_generating_codecs` themselves. Its public expert hook replaces the built-in registrations idempotently.

```
from komorebi_math.export import dumps_portable, loads_portable
from komorebi_math.generating import (
    algebraic_function,
    freeze_custom_function,
)

catalan = algebraic_function(
    "catalan",
    {(0, 0): 1, (0, 1): -1, (1, 2): 1},
    1,
)
frozen = freeze_custom_function(catalan, 8)
restored = loads_portable(dumps_portable(frozen))

print(restored == frozen)
print(restored.degree)
print(tuple(int(v) for v in restored.series().coefficients))
```

Output:

```
True
8
(1, 1, 2, 5, 14, 42, 132, 429, 1430)
```

The digest-checked portable envelope detects payload modification before reconstruction. It does not prove the generating rule that produced the coefficients.

A frozen record is bounded by design. `restored.series(9)` in the example raises `ValidationError`. Its reconstructed table basis also knows normalizations only through degree eight. Operations that implicitly grow beyond that range can raise `IndexError`. Truncate products and compositions to the captured degree, or freeze a wider prefix first.

22.3.5 Cost and boundary summary

Path	Time	Memory	Evidence class
Custom section	$O(d)$	$O(d)$	Exact coefficients
Root-filter value	$O(md)$	$O(d)$	Numerical complex equality
Section-cycle audit	Up to $O(m^2 d)$	$O(md)$	Finite counts and residual
Monoid product	$O(\|A\|\|B\|)$	Number of admitted output indices	Exact if callback law is correct
Direct Dirichlet prefix	$O(N^2)$ divisor scan	$O(N)$	Exact rational prefix
Series-to-tap conversion	$O(d)$	$O(d)$	Numerical certificate on adapter
Freeze or restore	$O(d)$ before JSON overhead	$O(d)$	Schema and digest validation

22.4 Resource budgets

22.4.1 Budget record and defaults

The shared generating-subsystem resource contract is:

```
GeneratingBudget(
    maximum_degree: int = 100_000,
    maximum_coefficient_bits: int = 4_000_000,
    maximum_matrix_dimension: int = 4_096,
    maximum_reachable_states: int = 100_000,
    maximum_local_combinations: int = 1_000_000,
    maximum_modulus: int = 2_000_000,
    maximum_serialized_bytes: int = 64 * 1024 * 1024,
    maximum_recursion_depth: int = 1_000,
) -> None
```

`DEFAULT_GENERATING_BUDGET` is one instance with those values. Every field must be a positive non-Boolean integer. The degree limit is positive even though degree zero is a valid request.

The complete method ledger is:

```
GeneratingBudget.require_degree(
    self,
    degree: int,
) -> int

GeneratingBudget.require_terms(
    self,
    count: int,
    *,
    label: str = "term count",
) -> int

GeneratingBudget.require_matrix_dimension(
    self,
    dimension: int,
) -> int

GeneratingBudget.require_states(
    self,
    states: int,
) -> int

GeneratingBudget.require_local_combinations(
    self,
    combinations: int,
) -> int

GeneratingBudget.require_modulus(
    self,
    modulus: int,
) -> int

GeneratingBudget.require_serialized_bytes(
    self,
    size: int,
) -> int

GeneratingBudget.require_coefficient_bits(
    self,
    bits: int,
) -> int
```

```
GeneratingBudget.max_modulus_table: int
```

`max_modulus_table` is a compatibility property returning `maximum_modulus`. Each `require_*` method returns the validated input. A malformed value raises `ValidationError`. A well-formed value beyond the configured ceiling raises `ResourceLimitError`.

There is no `require_recursion_depth` method in `1.0.0b4`. `maximum_recursion_depth` is serialized and validated as a positive field, but the Chapter 22 implementation does not enforce it.

22.4.2 Enforcement is operation-specific

A budget is effective only when the called operation accepts it or the caller invokes a guard explicitly.

Resource	Enforced examples
Degree and coefficient bits	`FiniteNormalizedSeries` construction and operations from Chapter 21
Matrix dimension and reachable states	Later carry and Cartier compilers
Local combinations and term counts	Later mask and digit compilers
Modulus	Prime-power and modular table constructors
Serialized bytes	Available as a manual guard
Recursion depth	Field only in `1.0.0b4`

The declarative providers, closed-form guessers, section helpers, monoid helpers, FIR adapters, `freeze_custom_function`, and portable dump/load calls documented in this chapter do **not** accept `budget=`. A large `CustomFunctionSpec.series(d)` request is not stopped by `DEFAULT_GENERATING_BUDGET`.

For those APIs, guard the request before execution:

```
from komorebi_math.generating import (
    GeneratingBudget,
    rational_function,
)

budget = GeneratingBudget(
    maximum_degree=8,
    maximum_coefficient_bits=64,
)
function = rational_function("geometric", (1,), (1, -1))

degree = budget.require_degree(8)
series = function.series(degree)

largest_bits = max(
    max(abs(value.numerator).bit_length(),
        value.denominator.bit_length())
    for value in series.coefficients
)
budget.require_coefficient_bits(largest_bits)
```

For serialization, measure the encoded byte payload and invoke `require_serialized_bytes` explicitly if an application requires that ceiling.

22.4.3 Typed failure example

```
from komorebi_math.exceptions import ResourceLimitError
from komorebi_math.generating import GeneratingBudget

budget = GeneratingBudget(maximum_degree=8)

print(budget.require_degree(8))

try:
    budget.require_degree(9)
except ResourceLimitError as error:
    print(type(error).__name__)
    print(str(error))
```

Output:

```
8
ResourceLimitError
degree 9 exceeds maximum_degree=8
```

The budget is immutable. Create a second record to authorize a larger finite computation. Do not mutate or replace the process-wide default.

22.4.4 Evidence and current limits

The Chapter 22 source audit covered:

Evidence	Result
`tests/test_generating_foundation.py`	Included in focused replay
`tests/test_generating_block13.py`	Included in focused replay
`tests/test_generating_domains.py`	Included in focused replay
Combined focused generating replay	**141 passed**
`examples/custom_generating_functions.py`	Passed
Declarative, recognition, section, monoid, FIR, persistence, and budget examples above	Replayed against `1.0.0b4`

The 141-test replay is broader than Chapter 22 because those source test files also exercise masks, Cartier operators, Bell reductions, and coefficient domains used in later chapters. Relevant Chapter 22 checks include basis sections, Fibonacci recognition, hypergeometric and algebraic guesses, negative low-complexity recognition results, Euler-product partitions, custom registries, algebraic and weighted builders, frozen portable round trips, FIR bridges, lazy optional-SymPy behavior, and typed budget failures.

The supported path is exact on declared finite coefficients and explicit at every conversion boundary. These limits remain in the shipped source:

1. Recognition candidates are prefix identities, not infinite proofs.
2. Symbolic recognition has no integrated `GeneratingBudget`.
3. Holdout and search-bound arguments are not fully validated.
4. Algebraic exponent keys are coerced through `int(...)`.
5. Monoid callback compatibility and associativity are trusted.
6. The direct root-filter helper does not fully validate its address.
7. FIR dtype conversion can be lossy until its certificate is checked.
8. Frozen providers cannot extend beyond their captured normalization table.
9. Recursion depth and portable byte ceilings are not automatically enforced by the Chapter 22 calls.

Chapter 22 summary. A normalized generating function has two independent declarations: the coefficient source and the basis normalization. Declarative providers build finite exact prefixes, diagonal conversion preserves ordinary coefficients, and recognition proposes bounded models without promoting them to theorems. Sections expose residue lanes, monoid records replace the index product, FIR adapters mark the exact to numerical boundary, and frozen records replace live callbacks with digest-checked finite data. Resource ceilings are explicit, but only operations that accept a budget or manually invoke a guard enforce them.

23. Masks, Cartier operators, and digit algebra

First route. Begin with the finite mask expansion in Section 23.1.1. The carry compiler in Section 23.1.2 evaluates the same kind of coefficient data through radix digits. For prime-field rational masks, continue to the Cartier compiler in Sections 23.3 to 23.4. Keep a short direct prefix as a readable comparison when inspecting a compiled plan, and record the radix and digit order with its transition matrices.

A local generating-function rule becomes finite and executable through explicit masks, transition matrices, and coefficient records. A Mahler mask describes a scale equation, carry matrices evaluate its coefficients from radix digits, Cartier sections compile rational masks over a prime field, and digit kernels expose the no-borrow algebra behind Lucas and Kummer phenomena. The implementations are finite by design. Degrees, prefix lengths, matrix dimensions, state counts, and prime-power moduli are declared before expansion.

The chapter covers six implementation modules and one explicit bridge:

Concern	Canonical implementation module
Exact masks, carry plans, Kummer layers, and coboundaries	`komorebi_math.generating.masks`
Polynomial Cartier maps, automata, and linear representations	`komorebi_math.generating.cartier`
Lucas-local kernels and finite closure audits	`komorebi_math.generating.digit_algebra`
Carry-linear valuations and digit-unit twists	`komorebi_math.generating.normalization`
Prime-power factorial and kernel compilers	`komorebi_math.generating.padic_units`
Formal-series and FIR conversion	`komorebi_math.generating.adapters`
Public research imports	`komorebi_math.generating`

All public names documented in this chapter belong to the **research** API in version `1.0.0b4`. None is exported by `komorebi_math.stable` or `komorebi_math.beta`. The complete and least ambiguous import style is

```
from komorebi_math.generating import (
    GeneratingBudget,
    compile_finite_mask_carry_plan,
    compile_rational_mask_automaton,
    mahler_mask_product,
)
```

The package root re-exports a smaller convenience subset, including `FiniteMaskCarryPlan`, `PrimePowerKernelCompiler`, `compile_finite_mask_carry_plan`, `compile_rational_mask_automaton`, `mahler_mask_inverse`, and `mahler_mask_product`. Code that needs the full system should import from `komorebi_math.generating` so that related record types and certificates remain visibly grouped.

> **Finite-claim boundary.** A successful mask certificate proves equality in a stated truncated formal ring. An automaton certificate compares a stated coefficient prefix. A digit-algebra audit exhausts a stated finite digit box. An analytic tail estimate is numerical and is not interval-rigorous. None of these finite records silently upgrades itself to an unrestricted convergence theorem or an unbounded congruence theorem.

23.1 Finite and local Mahler masks

23.1.1 Product, inverse, and functional equation

Let

$$R(z) = \sum_{j=0}^{m} r_j z^j, \qquad r_0 = 1, \qquad b \geq 2.$$

The local Mahler product is

$$\mathcal{M}_b[R](z) = \prod_{\ell \geq 0} R\left(z^{b^\ell}\right).$$

Only finitely many factors affect a finite truncation. In $\mathbb{Q}[z]/(z^{D+1})$, the returned series therefore satisfies

$$F(z) = R(z)F(z^b), \qquad R(z) = \frac{F(z)}{F(z^b)}.$$

The three exact entry points are

```
mahler_mask_product(
    mask: Iterable[int | Fraction],
    radix: int,
    degree: int,
    *,
    budget: GeneratingBudget = DEFAULT_GENERATING_BUDGET,
) -> tuple[Fraction, ...]

mahler_mask_inverse(
    series: Iterable[int | Fraction],
    radix: int,
    degree: int | None = None,
    *,
    budget: GeneratingBudget = DEFAULT_GENERATING_BUDGET,
) -> tuple[Fraction, ...]

certify_mahler_functional_equation(
    series: Iterable[int | Fraction],
    mask: Iterable[int | Fraction],
    radix: int,
    degree: int | None = None,
    *,
    budget: GeneratingBudget = DEFAULT_GENERATING_BUDGET,
) -> Certificate
```

Coefficient tuples use ascending powers. Both the mask and series must have constant coefficient exactly one. `mahler_mask_product` returns exactly `degree + 1` rational coefficients. `mahler_mask_inverse` uses the supplied degree, or the source's final stored degree when `degree=None`. The certificate compares every coefficient through the selected degree and reports at most the first 16 failures in `diagnostics["failures"]`.

```
from fractions import Fraction

from komorebi_math.generating import (
    certify_mahler_functional_equation,
    mahler_mask_inverse,
    mahler_mask_product,
)

mask = (1, 2, -1)
```

```
series = mahler_mask_product(mask, radix=3, degree=20)
recovered = mahler_mask_inverse(series, radix=3, degree=20)
certificate = certify_mahler_functional_equation(
    series,
    mask,
    radix=3,
    degree=20,
)

assert recovered[:3] == tuple(Fraction(value) for value in mask)
assert all(value == 0 for value in recovered[3:])
certificate.require()
```

The arithmetic is exact, but the claim is finite:

$$F(z) - R(z)F(z^3) = 0 \pmod{z^{21}}.$$

Source-audit warning. In `1.0.0b4`, `mahler_mask_product` checks `radix >= 2`, but `mahler_mask_inverse` and `certify_mahler_functional_equation` do not repeat that guard. Their mathematics still requires $b \geq 2$. Callers must validate the radix before invoking either operation. `polynomial_mask_coboundary` inherits the same gap through its use of `mahler_mask_inverse`. Also, the certificate path does not apply the degree budget when `degree=None`. Supply an explicit degree in bounded or adversarial workflows. These are research-surface validation gaps, not additional supported conventions.

23.1.2 Finite carry compiler

For a finite mask, the coefficient of the infinite product can be read through a bounded carry state. If q is the incoming carry, q' the outgoing carry, and d the current radix digit, the transition entry is

$$T_d(q, q') = r_{d+bq'-q},$$

with coefficients outside $0, \ldots, m$ interpreted as zero. The compiler is

```
compile_finite_mask_carry_plan(
    mask: Iterable[int | Fraction],
    radix: int,
    *,
    budget: GeneratingBudget = DEFAULT_GENERATING_BUDGET,
) -> FiniteMaskCarryPlan
```

The immutable result has the following public interface.

Member	Meaning
`radix`	b
`mask`	normalized exact coefficient tuple
`carry_bound`	largest represented carry
`matrices[d]`	exact T_d
`dimension`	`carry_bound + 1`
`evaluate(index)`	one exact coefficient
`sequence(count, *, budget=...)`	finite prefix
`certificate(count, *, budget=...)`	comparison with direct Mahler expansion

Digits are consumed **least significant first**. With a row-vector state v, one step is $v \leftarrow vT_d$, the initial vector is e_0, and the returned coefficient is the final zero-carry component.

For the binary overfull mask $R(z) = 1 + z + z^2$, the plan has carry bound two and

$$T_0 = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 1 \end{bmatrix}, \qquad T_1 = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 0 & 1 & 0 \end{bmatrix}.$$

```python
from komorebi_math.generating import compile_finite_mask_carry_plan

plan = compile_finite_mask_carry_plan((1, 1, 1), radix=2)
plan.certificate(100).require()

# 13 has least-significant-first binary digits (1, 0, 1, 1).
assert plan.evaluate(13) == 3
assert plan.sequence(8) == (1, 1, 2, 1, 3, 2, 3, 1)
```

This prefix is Stern's diatomic sequence shifted by one index. The certificate establishes the coefficient identity for the requested prefix, not a new unbounded identity by itself.

`NormalizedMaskPlan` is an alias of `FiniteMaskCarryPlan`. It is not a second record format. Construct plans with the compiler rather than the raw dataclass constructor, because the compiler validates the unit constant, radix, matrix dimension, and transition construction.

23.1.3 Digit-local Cauchy products

`compile_digit_cauchy_product` compiles the ordinary Cauchy product of several digit-multiplicative sequences. Each input table must contain one weight per radix digit and must assign weight one to digit zero.

```
compile_digit_cauchy_product(
    digit_weights: Sequence[Sequence[int | Fraction]],
    radix: int,
    *,
    budget: GeneratingBudget = DEFAULT_GENERATING_BUDGET,
) -> DigitCauchyProductPlan
```

For h factors, all b^h local digit combinations are first collected by their digit sum. The returned plan has `factor_count == dimension == h`, exact `matrices`, and the methods `evaluate(index)` and `sequence(count, *, budget)`. Unlike `FiniteMaskCarryPlan`, it has no certificate method and no registered portable codec in `1.0.0b4`. A caller that needs evidence should compare its finite prefix with a direct convolution over the same range.

23.1.4 Defect masks, coboundaries, and analytic realization

The remainder of the mask surface separates exact formal identities from one numerical realization.

Function	Finite operation
`frobenius_defect_mask(mask, prime, degree, *, budget=...)`	$R(z^p)/R(z)^p$ through degree D
`first_frobenius_lift(mask, prime, degree)`	$(\mathfrak{D}_p R - 1)/p \pmod p$, after an explicit divisibility check
`polynomial_mask_coboundary(candidate, radix, degree=None)`	$Q(z)/Q(z^b)$ for a declared polynomial unit
`solve_polynomial_coboundary(numerator, denominator, radix, degree_bound, *, budget=...)`	bounded rational solve of $B(z)Q(z) = A(z)Q(z^b)$
`certify_rational_mask_algebraic_relation(numerator, denominator, prime, degree, *, budget=...)`	$A(z)F(z)^{p-1} = B(z) \pmod{(p, z^{D+1})}$
`euler_product_collapse(radix, exponents)`	finite orbit bookkeeping for Euler factors
`analytic_mahler_product(mask, point, radix, terms)`	floating-complex finite product and conservative tail estimate

The Frobenius maps are

$$\mathfrak{D}_p R(z) = \frac{R(z^p)}{R(z)^p}, \qquad \mathfrak{L}_p R(z) = \frac{\mathfrak{D}_p R(z) - 1}{p} \pmod{p}.$$

`first_frobenius_lift` rejects a coefficient whose adjusted defect is not an integer divisible by p. It does not presume the divisibility.

`PolynomialCoboundaryResult` stores `radix`, `degree_bound`, optional `coefficients`, and an exact `residual`. Its convenience properties are `found` and `valid`. `require()` returns the coefficients only when the exact residual vanishes, and `certificate()` returns the finite exact evidence.

```
from fractions import Fraction

from komorebi_math.generating import solve_polynomial_coboundary

result = solve_polynomial_coboundary(
    numerator=(1, 2),
    denominator=(1, 0, 2),
    radix=2,
    degree_bound=4,
)

assert result.require() == (Fraction(1), Fraction(2))
```

Bounded-solver limitation. `PolynomialCoboundaryResult` distinguishes a verified returned solution from no solution found within the declared bound. The `1.0.0b4` record does **not** report rank or nullity, so it cannot expose whether a consistent bounded system was unique or underdetermined. The implementation selects zero for free variables. A finite polynomial solution is also not an unrestricted analytic factorization.

`euler_product_collapse` returns `EulerProductCollapse(radix, mask_exponents, closed_form_exponents, obstruction_orbits)`. The `collapses` property is true exactly when every radix-free orbit has zero terminal exponent sum. `require()` raises when an obstruction orbit remains. This is finite exponent bookkeeping for the supplied map. It makes no unrestricted convergence claim about an analytic Euler product.

`AnalyticMaskApproximation` stores `radix`, `point`, `terms`, `value`, `relative_tail_bound`, and `absolute_error_bound`. Its certificate has `Exactness.NUMERICAL`. It requires $|z| < 1$, but its ordinary complex floating-point product and conservative bound are not interval-rigorous. An infinite relative bound produces an invalid certificate rather than a false convergence claim.

23.2 Carry and digit operators

23.2.1 Kummer borrow layers

For prime p, Kummer's theorem identifies

$$v_p\binom{n}{k} = \text{number of base-}p\text{ borrows in } n - k.$$

The layer polynomial

$$L_{n,p}(t) = \sum_{k=0}^{n} t^{v_p\binom{n}{k}}$$

is represented by `KummerLayerDistribution(index, prime, counts)`. Its public members are

Member	Meaning
`counts[j]`	number of subindices in borrow layer j
`total`	sum of all layer counts, certified equal to `index + 1`

Member	Meaning
`count(borrows)`	one layer count, or zero outside stored support
`support_count(maximum_borrows)`	cumulative count through a layer
`certificate()`	exact exhaustion certificate

The related functions are

```
kummer_layer_distribution(index: int, prime: int) -> KummerLayerDistribution
zero_borrow_count(index: int, prime: int) -> int
one_borrow_count(index: int, prime: int) -> int

factorial_power_support_count(
    index: int,
    prime: int,
    factorial_power: int,
    modulus_exponent: int,
) -> int

factorial_power_support_indices(
    index: int,
    prime: int,
    factorial_power: int,
    modulus_exponent: int,
    *,
    budget: GeneratingBudget = DEFAULT_GENERATING_BUDGET,
) -> tuple[int, ...]
```

For factorial power s modulo p^r, survival uses a **strict** boundary:

$$\binom{n}{k}^s \not\equiv 0 \pmod{p^r} \quad \iff \quad s\, v_p\binom{n}{k} < r.$$

Thus $sv = r - 1$ survives and $sv = r$ does not. The implementation converts this to

$$v_p\binom{n}{k} \le \left\lfloor \frac{r-1}{s} \right\rfloor.$$

`factorial_power_support_count` obtains the answer from the layer counts. `factorial_power_support_indices` explicitly scans $k = 0, \ldots, n$ after a term-budget check.

The two boundary cases can be executed directly. Since $v_2\binom{4}{2} = 1$ and $s = 2$, the weighted valuation is two. It survives modulo 2^3, where it equals $r - 1$, and vanishes modulo 2^2, where it equals r.

```
from komorebi_math.generating import PrimePowerKernelCompiler

surviving = PrimePowerKernelCompiler(2, 3, 2).kernel(4, 2)
vanishing = PrimePowerKernelCompiler(2, 2, 2).kernel(4, 2)

assert surviving.valuation == 2
assert surviving.residue == 4
assert surviving.survives

assert vanishing.valuation == 2
assert vanishing.residue == 0
assert not vanishing.survives
```

23.2.2 Carry-layer convolution

```
factorial_power_convolution_layers(
    left: Sequence[int],
    right: Sequence[int],
    index: int,
    prime: int,
    factorial_power: int,
    modulus_exponent: int,
    *,
    budget: GeneratingBudget = DEFAULT_GENERATING_BUDGET,
) -> CarryLayerConvolution
```

The result stores `index`, `prime`, `exponent`, `factorial_power`, `modulus`, and one modular sum per surviving borrow layer. Its `residue` property adds the layers modulo p^r. The routine requires both coefficient arrays to contain indices through `index`, scans exactly `index + 1` subindices, and constructs a strict `PrimePowerKernelCompiler` for the binomial-power residue.

23.2.3 Lucas-local digit kernels

`DigitKernel` is a finite lower-triangular table. For radix b, modulus m, total digit d, and selected digit e, define

$$K(d, e) = \texttt{table[d][e]} \pmod{m}, \qquad 0 \le e \le d < b.$$

The lifted no-borrow weight is

$$K_{\text{global}}(n, k) = \prod_i K(n_i, k_i),$$

and is zero when any $k_i > n_i$. The record constructor and its methods are

```
DigitKernel(
    radix: int,
    modulus: int,
    table: tuple[tuple[int, ...], ...],
    name: str = "digit kernel",
)

kernel.local_weight(digit, subdigit) -> int
kernel.global_weight(index, subindex) -> int
kernel.lift(digit_values, index) -> int
kernel.local_multiply(left, right) -> tuple[int, ...]
kernel.global_multiply(left, right, *, count) -> tuple[int, ...]
```

The convenience constructor

```
factorial_power_digit_kernel(power: int, prime: int) -> DigitKernel
```

uses

$$K(d, e) = \binom{d}{e}^s \pmod{p}.$$

`base_digits(index, radix)` returns least-significant-first digits and represents zero as `(0,)`. `lift_digit_sequence(digit_values, *, radix, modulus, count)` applies an identity kernel to produce a finite digit-multiplicative prefix.

The finite theorem audit is

```
audit_factorial_power_digit_algebra(
    *,
    prime: int,
    power: int,
    left_digits: Iterable[int],
    right_digits: Iterable[int],
    max_digits: int = 3,
) -> DigitAlgebraAudit
```

For all $0 \leq n < p^D$, it checks the lifted Lucas weights and the closure identity

$$(a \star_s b)_n \equiv \prod_i (a \star_s b)_{n_i} \pmod{p}.$$

The record prints `tested_weights`, `weight_failures`, `tested_coefficients`, `closure_failures`, and `local_product`. It has no `.valid` property or `.require()` method in `1.0.0b4`. Validity means both failure counts are zero.

```
from komorebi_math.generating import audit_factorial_power_digit_algebra

audit = audit_factorial_power_digit_algebra(
    prime=3,
    power=2,
    left_digits=(1, 2, 1),
    right_digits=(1, 1, 2),
    max_digits=2,
)

assert audit.tested_weights == 45
assert audit.tested_coefficients == 9
assert audit.weight_failures == 0
assert audit.closure_failures == 0
assert audit.local_product == (1, 0, 2)
```

Primality guard discrepancy. The Lucas theorem used here requires a prime. `factorial_power_digit_kernel` and `audit_factorial_power_digit_algebra` currently reject values below two but do not test primality. `local_normalized_kernel` and `digit_factorized_normalized_kernel` likewise do not perform a prime test. Composite inputs can therefore return finite modular calculations that are not certified Lucas arithmetic. Validate with a strict prime-aware operation, or with the application's own primality guard, before using those research helpers in a theorem claim.

23.3 Cartier sections

The Cartier operator selects one residue class of exponents. In the exact notation retained by this guide,

$$\Lambda_r\left(\sum_n a_n z^n\right) = \sum_n a_{pn+r} z^n.$$

The implementation names the selected residue `digit` and uses $0 \leq r < p$. All polynomial coefficients are reduced modulo the selected prime and stored in ascending power order.

23.3.1 Polynomial primitives

```
polynomial_multiply_mod(
    left: Sequence[int],
    right: Sequence[int],
    modulus: int,
) -> tuple[int, ...]
```

```
polynomial_power_mod(
    polynomial: Sequence[int],
    exponent: int,
    modulus: int,
) -> tuple[int, ...]

polynomial_cartier(
    polynomial: Sequence[int],
    digit: int,
    prime: int,
) -> tuple[int, ...]

rational_mask_coefficients_mod_prime(
    numerator: Sequence[int],
    denominator: Sequence[int],
    prime: int,
    count: int,
) -> tuple[int, ...]

cartier_degree_bound(
    numerator: Sequence[int],
    denominator: Sequence[int],
    prime: int,
) -> int
```

`polynomial_multiply_mod` is a general modular primitive and only requires a modulus of at least two. `polynomial_cartier`, `rational_mask_coefficients_mod_prime`, and `cartier_degree_bound` perform an actual primality test. The rational-mask recurrence requires

$$A(0) = B(0) \neq 0 \pmod{p}.$$

It returns the unique unit-series prefix satisfying

$$B(z)F(z) = A(z)F(z^p) \quad \text{in } \mathbb{F}_p[[z]].$$

For numerator degree a and denominator degree b, the invariant degree bound is computed from

$$\Delta = \max\left(b, \left\lceil \frac{a + (p-2)b}{p-1} \right\rceil\right).$$

That is a state-polynomial degree bound. It is not the number of reachable automaton states.

23.3.2 Rational-mask algebraic certificate

In characteristic p, the finite rational-mask relation can also be tested as

$$A(z)F(z)^{p-1} = B(z) \pmod{(p, z^{D+1})}.$$

`certify_rational_mask_algebraic_relation` returns `RationalMaskAlgebraicCertificate(prime, degree, failures)`. The record exposes `valid`, `require()`, and `certificate()`. The constructor function calls `require()` internally, so a failed relation raises rather than returning an invalid record.

23.4 Congruence automata and finite snapshots

23.4.1 Reachable Cartier automata

```
compile_rational_mask_automaton(
    numerator: Sequence[int],
    denominator: Sequence[int],
    prime: int,
    *,
    minimize: bool = True,
    max_states: int = 100_000,
    budget: GeneratingBudget = DEFAULT_GENERATING_BUDGET,
) -> CartierAutomaton

minimize_cartier_automaton(
    automaton: CartierAutomaton,
) -> CartierAutomaton
```

The compiler starts from the denominator polynomial, closes the reachable state set under every Cartier digit, and optionally minimizes the resulting output automaton by exact partition refinement. Digits are read least significant first. If

$$N = d_0 + d_1 p + \cdots + d_{L-1} p^{L-1},$$

then evaluation follows the transitions labelled $d_0, d_1, \ldots, d_{L-1}$. Its transition count is $O(\lfloor \log_p N \rfloor + 1)$.

The returned record is

```
CartierAutomaton(
    prime,
    numerator,
    denominator,
    degree_bound,
    transitions,
    outputs,
    state_numerators,
    state_members,
    start_state=0,
)
```

The compiled automaton exposes these members.

Member	Meaning
`number_states`	reachable states after optional minimization
`theoretical_state_bound`	$p^{\Delta+1}$, all possible bounded numerator vectors
`evaluate(index)`	one coefficient modulo p
`sequence(count)`	finite coefficient prefix
`certificate(count)`	finite comparison with the defining recurrence

`AutomatonCertificate` stores only `compared_terms` and a tuple of `(index, observed, expected)` failures. Its `valid` property tests whether the failure tuple is empty, and `require()` raises on the first discrepancy. Reachability, closure, state numerators, merged state members, degree bounds, and outputs belong to `CartierAutomaton`, not to its finite-prefix certificate.

> **Construction rule.** The raw automaton and linear-representation dataclasses do not perform complete `__post_init__` validation. Use the compilers for live construction. Portable decoding performs structural checks, but even a structurally valid snapshot should be replayed with a finite certificate when its mathematical provenance matters.

23.4.2 Large-index linear evaluation

An automaton enumerates the reachable nonlinear state set. A Cartier linear representation instead retains the full bounded polynomial vector space.

```
compile_cartier_linear_representation(
    numerator: Sequence[int],
    denominator: Sequence[int],
    prime: int,
    *,
    budget: GeneratingBudget = DEFAULT_GENERATING_BUDGET,
) -> CartierLinearRepresentation
```

The record stores `prime`, normalized `numerator` and `denominator`, `degree_bound`, one transition matrix per digit, `start_vector`, and `output_scale`. Its `dimension` is `degree_bound + 1`. Its operational methods are `transition(vector, digit)`, `evaluate(index)`, and `sequence(count)`.

```
from komorebi_math.generating import (
    base_digits,
    compile_cartier_linear_representation,
)

index = 10**18 + 12345
representation = compile_cartier_linear_representation(
    numerator=(1, 1),
    denominator=(1, -1),
    prime=5,
)

digits = base_digits(index, 5)
value = representation.evaluate(index)

assert digits == (
    0, 4, 3, 3, 4, 3, 0, 0, 0, 0, 0, 0, 0,
    0, 0, 0, 0, 0, 4, 3, 0, 2, 4, 3, 1, 3,
)
assert representation.dimension == 2
assert value == 0
```

With row-vector convention, the ordered product is

$$v_N = v_0 T_{d_0} T_{d_1} \cdots T_{d_{L-1}}, \qquad f_N = (v_N)_0 B(0)^{-1} \pmod{p}.$$

The scalar evaluation cost is $O(L\Delta^2)$ for dense transition matrices of dimension $\Delta+1$. Compilation stores $p(\Delta+1)^2$ residues. Reachable-automaton evaluation is $O(L)$ table lookups after potentially more expensive state exploration.

23.4.3 Carry and Cartier equality

For a polynomial mask, the carry matrices and the Cartier multiplication matrices are entrywise equal on the compiler's declared state space.

```
finite_mask_carry_matrices(
    mask: Sequence[int],
    prime: int,
    *,
    budget: GeneratingBudget = DEFAULT_GENERATING_BUDGET,
) -> tuple[tuple[tuple[int, ...], ...], ...]

certify_cartier_carry_equivalence(
    mask: Sequence[int],
    prime: int,
```

```
    *,
    budget: GeneratingBudget = DEFAULT_GENERATING_BUDGET,
) -> CartierCarryEquivalence
```

`CartierCarryEquivalence` stores `prime`, `dimension`, and failures of the form `(digit, source, target, observed, expected)`. It provides `valid` and `require()`. The finite claim is matrix equality, not a claimed conjugacy with an arbitrary external automaton.

23.4.4 Portable snapshots

The trusted portable layer registers data-only codecs for these Chapter 23 objects:

Record	Codec tag
`FiniteMaskCarryPlan`	`generating.finite_mask_carry_plan`
`CartierAutomaton`	`generating.cartier_automaton`
`CartierLinearRepresentation`	`generating.cartier_linear_representation`
`KummerLayerDistribution`	`generating.kummer_layer_distribution`
`CarryLayerConvolution`	`generating.carry_layer_convolution`

```
from komorebi_math.export import dumps_portable, loads_portable
from komorebi_math.generating import compile_rational_mask_automaton

source = compile_rational_mask_automaton((1, 1), (1, -1), 5)
payload = dumps_portable(source)
restored = loads_portable(payload)

assert restored.sequence(200) == source.sequence(200)
restored.certificate(200).require()
```

The envelope carries a SHA-256 integrity digest and rejects tampering. This is integrity and closed data reconstruction, not authenticity or authorization. `DigitKernel`, `DigitCauchyProductPlan`, prime-power compiler instances, `FIRSeriesAdapter`, and most certificate records have no registered portable codec in this release. Recompile them from validated parameters or persist a separate application-owned schema.

23.5 Signal adapters

The generating-to-signal bridge interprets a finite ordinary series

$$H(z) = \sum_{n=0}^{D} h_n z^n$$

as the causal FIR transfer polynomial

$$H(z^{-1}) = \sum_{n=0}^{D} h_n z^{-n}.$$

It does not infer an IIR system, an infinite impulse response, a minimum-phase factorization, or stability beyond the finite FIR convention.

```
series_to_fir_taps(
    series: FormalSeries,
    *,
    degree: int | None = None,
    dtype: Any = float,
) -> numpy.ndarray
```

```
series_to_fir_adapter(
    series: FormalSeries,
    *,
    degree: int | None = None,
    dtype: Any = float,
) -> FIRSeriesAdapter

fir_taps_to_series(
    taps: Any,
    *,
    basis: WeightedMonomialBasis | None = None,
    max_denominator: int | None = None,
) -> FormalSeries
```

`series_to_fir_taps` first requests ordinary coefficients from the source basis, then converts them to the selected NumPy dtype. The adapter stores the truncated `FormalSeries`, a copied one-dimensional tap array, and the explicit string `formal z maps to delay z^(-1)`. Its methods are

```
adapter.response(
    *,
    points: int = 512,
    sample_rate: float = 1.0,
    whole: bool = False,
) -> FrequencyResponse

adapter.certificate(
    *,
    tolerance: float = 0.0,
) -> Certificate
```

The certificate has `Exactness.NUMERICAL` because exact rational series coefficients have been converted to a NumPy array. It checks coefficient-to-tap agreement only.

```
from fractions import Fraction

import numpy as np

from komorebi_math.generating import (
    FormalSeries,
    fir_taps_to_series,
    ordinary_basis,
    series_to_fir_adapter,
)

source = FormalSeries(
    ordinary_basis(),
    (1, Fraction(1, 2), Fraction(-1, 4), 2),
)
adapter = series_to_fir_adapter(source)
adapter.certificate().require()

restored = fir_taps_to_series(adapter.taps, max_denominator=1000)
response = adapter.response(points=32)

assert np.allclose(adapter.taps, [1.0, 0.5, -0.25, 2.0])
assert restored.coefficients == source.coefficients
assert response.response.shape == (32,)
```

Floating taps are converted back through their decimal string representation. `max_denominator` requests a bounded rational approximation explicitly. Nonreal complex taps, nonfinite taps, empty or multidimensional arrays, and invalid denominator bounds are rejected.

23.6 Carry-linear normalization and unit twists

23.6.1 Carry-linear valuation profiles

A carry-linear normalization has valuation profile

$$\nu(n) = an + s\, v_p(n!),$$

where $a \in \mathbb{Z}$ and $s \geq 0$. Its convolution defect is

$$\begin{aligned}\nu(n) - \nu(k) - \nu(n-k) &= s\left(v_p(n!) - v_p(k!) - v_p((n-k)!)\right) \\ &= s\, v_p\binom{n}{k}.\end{aligned}$$

The linear term cancels. Carries therefore control the valuation defect.

```
CarryLinearValuation(
    prime: int,
    linear_slope: int,
    carry_weight: int,
)

profile.value(index: int) -> int
profile.defect(index: int, subindex: int) -> int

classify_carry_linear_valuation(
    valuation: Callable[[int], int],
    prime: int,
    checked_through: int,
) -> CarryLinearCertificate
```

The classifier recovers $a = \nu(1)$ and $s = \nu(p) - pa$, then checks every value and every split through `checked_through`. Its result stores `prime`, `linear_slope`, `carry_weight`, `checked_through`, and failures of the form `(index, subindex, observed, expected)`. It supplies `valid` and `require()`. The callback classification is explicitly finite even though the mathematical classification motivating it is unbounded.

```
from komorebi_math.generating import (
    CarryLinearValuation,
    classify_carry_linear_valuation,
)

profile = CarryLinearValuation(prime=3, linear_slope=2, carry_weight=5)
certificate = classify_carry_linear_valuation(profile.value, 3, 40)
certificate.require()

assert certificate.linear_slope == 2
assert certificate.carry_weight == 5
assert profile.defect(5, 2) == 0
```

23.6.2 Digit-unit twists

Let $w_0, \dots, w_{p-1}$ be nonzero residues modulo prime p, and define

$$u(n) = \prod_i w_{n_i} \pmod{p}.$$

The local twisted factorial-power kernel is

$$K_u(d, e) = \binom{d}{e}^s \frac{w_d}{w_e w_{d-e}} \pmod{p},$$

and the digit-factorized global kernel is the product of these local factors.

```
digit_unit(
    index: int,
    prime: int,
    weights: Sequence[int],
) -> int

local_normalized_kernel(
    total_digit: int,
    selected_digit: int,
    prime: int,
    factorial_power: int,
    unit_weights: Sequence[int],
) -> int

digit_factorized_normalized_kernel(
    index: int,
    subindex: int,
    prime: int,
    factorial_power: int,
    unit_weights: Sequence[int],
) -> int
```

`digit_unit` strictly tests that `prime` is prime and that every weight is a unit. The two kernel helpers validate digit range, weight-table length, and invertibility but do not independently test primality. Treat them as prime-domain helpers only after the caller has established the domain.

23.6.3 What the carry and Cartier compilers share

The same least-significant-first digit word drives three finite machines:

1. Exact rational carry matrices for a finite Mahler mask.
2. Prime-field Cartier transitions for a rational mask.
3. Prime-power valuation and unit calculations for normalized kernels.

Their shared digit order does not make their evidence interchangeable. The finite-mask plan proves rational coefficient agreement. The Cartier automaton proves modular prefix agreement. The prime-power compiler returns a valuation, unit, and residue after strict domain validation.

23.7 `generating.padic_units` and strict prime-power domain guards

23.7.1 Valuation primitives

The prime-power implementation module provides

```
factorial_valuation(index: int, prime: int) -> int
integer_valuation(value: int, prime: int) -> int
binomial_valuation(index: int, subindex: int, prime: int) -> int
borrow_count(index: int, subindex: int, prime: int) -> int
```

Each function performs an actual primality test. `factorial_valuation` uses Legendre's sum

$$v_p(n!) = \sum_{j \geq 1} \left\lfloor \frac{n}{p^j} \right\rfloor.$$

`integer_valuation` rejects zero because the record language stores finite integer valuations only. `binomial_valuation` and `borrow_count` require $0 \leq k \leq n$. Their equality is the Kummer bridge used by the layer compiler.

23.7.2 Prime-power binomial compiler

```
PrimePowerUnitCompiler(
    prime: int,
    exponent: int,
    max_modulus: int = 2_000_000,
)
```

Construction checks that `prime` is prime, `exponent` is positive, and $p^r \leq$ `max_modulus`. It builds a prefix table of the p-free factorials modulo p^r. Public operations are

```
compiler.unit_factorial(index: int) -> int
compiler.binomial_data(index: int, subindex: int) -> PrimePowerBinomial
compiler.binomial(index: int, subindex: int) -> int
```

`PrimePowerBinomial` stores `index`, `subindex`, `prime`, `exponent`, `modulus`, `valuation`, `unit`, and `residue`. Its `survives` property is exactly `valuation < exponent`.

```
from komorebi_math.generating import PrimePowerUnitCompiler

compiler = PrimePowerUnitCompiler(prime=3, exponent=4)
entry = compiler.binomial_data(20, 7)

assert compiler.modulus == 81
assert entry.valuation == 1
assert entry.unit == 1
assert entry.residue == 3
assert entry.survives
```

The prefix table costs $O(p^r)$ time and memory at construction. A query uses $O(\log_p n)$ unit-factorial iterations, plus modular exponentiation, and completed unit factorials are memoized.

23.7.3 Normalized prime-power kernels

For

$$\gamma_n = c^n (n!)^s u_n,$$

the geometric factor cancels from the normalized structure constant. The compiler evaluates

$$C_\gamma(n,k) = \binom{n}{k}^s \frac{u_n}{u_k u_{n-k}} \pmod{p^r}.$$

```
PrimePowerKernelCompiler(
    prime: int,
    exponent: int,
    factorial_power: int,
    unit_sequence: Callable[[int], int] | None = None,
    max_modulus: int = 2_000_000,
)

compiler.modulus -> int
compiler.kernel(index: int, subindex: int) -> NormalizedKernelResidue
```

`factorial_power` may be zero here. A missing `unit_sequence` means the constant unit one. Every evaluated callback value must be coprime to p. `NormalizedKernelResidue` stores `index`, `subindex`, `factorial_power`, `prime`, `exponent`, `valuation`, `unit`, and `residue`. `survives` tests the strict condition `valuation < exponent`.

```
from komorebi_math.generating import PrimePowerKernelCompiler

unit = lambda index: 1 + 3 * index
kernel = PrimePowerKernelCompiler(3, 4, 2, unit)
entry = kernel.kernel(20, 7)

assert kernel.modulus == 81
assert entry.valuation == 2
assert entry.unit == 46
assert entry.residue == 9
assert entry.survives
```

The callback is live executable state and is not portable. Persist its application-level identifier and parameters, then reconstruct it in trusted code. Do not serialize an arbitrary callback as though it were a package-owned portable record.

23.7.4 Guard and failure matrix

Request	Guard in `1.0.0b4`	Failure
nonprime `prime` in `padic_units` valuation or compiler APIs	strict trial-division primality test	`ValueError`
nonpositive prime-power exponent	strict	`ValueError`
$p^r >$ `max_modulus`	checked before prefix-table allocation	`ValueError`
zero passed to `integer_valuation`	rejected as having no finite valuation	`ValueError`
subindex outside $[0, n]$	strict	`ValueError` or zero only where explicitly documented
unit callback returns a multiple of p	checked at each kernel query	`ValueError`
nonunit Mahler constant	exact unit-series guard	`ValidationError`
radix below two in mask product or carry compiler	strict	`ValidationError`
matrix, state, term, degree, or modulus request above budget	precondition check	`ResourceLimitError`
composite passed to `factorial_power_digit_kernel`	**not detected**	finite result without a Lucas-theorem guarantee
radix below two in `mahler_mask_inverse` or its functional certificate	**not detected**	unsupported research-surface result

The resource budget relevant to this chapter defaults to degree 100,000, matrix dimension 4,096, reachable states 100,000, local combinations 1,000,000, modulus table 2,000,000, and serialized payload 64 MiB. Raising a budget makes a larger request permissible. It does not change the arithmetic domain or turn a finite audit into an unbounded theorem.

The mask and Cartier implementations check only the budget dimensions they actually allocate. In particular, the exact mask routines do not call `require_coefficient_bits`, and portable dumping does not automatically call a `GeneratingBudget` object's `require_serialized_bytes`. Exact rational coefficient growth and final payload size therefore remain caller-managed resources on these paths. The `maximum_recursion_depth` field is also not used by the iterative Chapter 23 compilers.

23.7.5 Complexity and allocation envelope

Let D be a truncation degree, $m = \deg R$, $c = \lceil m/(b-1) \rceil + 1$ the carry dimension, L a radix-digit length, h the number of digit-local factors, S a reachable automaton state count, and $q = p^r$.

Operation	Time envelope	Stored data	Active precondition
`mahler_mask_product`	worst-case $O(D^2 \log_b D)$ rational operations	$O(D)$ coefficients	degree
`mahler_mask_inverse`	worst-case $O(D^2)$ rational operations	$O(D)$ coefficients	degree
finite-mask compilation	$O(bc^2)$	$O(bc^2)$ rationals	matrix dimension
finite-mask evaluation	$O(Lc^2)$	one c-vector	validated index and compiled plan
digit Cauchy compilation	$O(b^h + bh^2)$	$O(bh^2)$ rationals	local combinations and matrix dimension
support enumeration	$O(n \log_p n)$ borrow work	up to $O(n)$ indices	term count
prime-power compiler construction	$O(q)$	$O(q)$ residues	modulus table
prime-power query	$O(\log_p n)$ iterations plus modular powers	memoized unit factorials	strict prime and unit guards
reachable Cartier compilation	bounded by S states and pS transitions	$O(pS)$ transitions plus state polynomials	degree, matrix dimension, and state guards
reachable automaton evaluation	$O(L)$ table lookups	one state index	nonnegative index
Cartier linear evaluation	$O(L(\Delta+1)^2)$	$p(\Delta+1)^2$ residues	matrix dimension

These are operation-count envelopes for the shipped dense algorithms. Exact `Fraction` cost also depends on numerator and denominator bit growth. Automaton minimization adds partition-refinement work after reachable-state compilation.

23.7.6 Evidence and reproducible checks

The shipped `1.0.0b4` evidence relevant to this chapter includes:

Executed check	Result
`tests/test_generating_foundation.py`	passed in the focused Chapter 23 replay
`tests/test_generating_block13.py`	passed in the focused Chapter 23 replay
`tests/test_generating_cartier_padic.py`	passed in the focused Chapter 23 replay
Combined focused replay	**173 passed**
`examples/mahler_cartier_compilers.py`	passed
`examples/prime_power_generating.py`	passed

The focused tests exercise exact mask inversion, finite carry plans, Cauchy products, Kummer layers, strict support boundaries, Frobenius defects, coboundaries, portable round trips, signal adapters, Cartier automata and minimization, carry/Cartier equality, large prime-power indices, unit twists, and rejection of perturbed finite profiles. A passing finite suite validates the shipped cases and guards. It does not erase the explicitly documented validation gaps above.

> **Chapter 23 summary.** Start with a unit local mask and a radix at least two. Expand it exactly only to a declared degree, or compile it into carry matrices that consume least-significant-first digits. Over a verified prime field, Cartier sections turn a rational Mahler equation into a reachable automaton or a bounded linear representation, both capable of logarithmic digit-length evaluation. Kummer layers and Lucas-local kernels expose the carry mechanism behind factorial-power support. FIR adapters make the formal coefficient-to-delay convention explicit. Carry-linear profiles and unit twists extend the normalized kernel without changing the strict survival boundary. Use registered snapshots only for supported closed records, replay finite certificates after restoration, and never infer prime mathematics from the few low-level research helpers that currently accept composites.

24. Prime-power, Bell, cyclotomic, and q-Kummer systems

First route. Use the kernel example in Section 24.1.4 to see valuation, unit, and residue together. For Bell reductions, compare the complete coefficient rows in Section 24.3 before using a residue automaton. Cyclotomic collapse and q-Kummer valuation are independent branches in Sections 24.4 to 24.5. Choose the branch matching the input object, and inspect its result-field table before interpreting a zero or an obstruction.

Four finite exact-arithmetic workflows share the same local principle: global coefficients are reduced to prime-radix valuations, unit parts, digit sums, or cyclotomic orbits before an exact residue is assembled. The implementations are finite compilers. They do not construct an infinite p-adic field, prove convergence of an infinite product, or infer a congruence from numerical agreement.

The four workflows answer different questions.

Workflow	Primary question	Returned evidence
Prime-power compiler	What is a binomial or normalized kernel modulo p^r?	valuation, unit, residue, support flag
Generalized Bell reduction	Does a factorial-power Bell row reduce to its base-p digit sum?	exact coefficient comparison and certificate
Cyclotomic collapse	Is a finite Mahler mask a cyclotomic coboundary?	closed-form exponents or explicit obstruction orbits
q-Kummer decomposition	What is the p-adic valuation of an integer Gaussian binomial?	mixed-radix branch record and support predicate

API tier and import surfaces.

The complete surface documented here is public from `komorebi_math.generating` and is covered by the research public-API manifest. It is not re-exported by `komorebi_math.stable` or `komorebi_math.beta`. Consequently, the exact signatures are checked in the current release, but they do not carry the frozen 1.x stable-surface promise.

The root namespace provides only the most common entry points:

```
from komorebi_math import (
    BellParityAutomaton,
    BellResidueAutomaton,
    PrimePowerKernelCompiler,
    generalized_bell_digit_reduction,
    q_binomial_valuation_decomposition,
)
```

Use the generating namespace for the full records, primitive valuations, cyclotomic verifiers, weighted Bell reduction, and direct q-Kummer checks:

```
from komorebi_math.generating import (
    BellDigitReduction,
    BellParityAutomaton,
    BellResidueAutomaton,
    CyclotomicCollapse,
    NormalizedKernelResidue,
    PointedSubindexCertificate,
    PrimePowerBinomial,
```

```
    PrimePowerKernelCompiler,
    PrimePowerUnitCompiler,
    QKummerDecomposition,
)
```

Only `normalized_cyclotomic_expression` and `verify_cyclotomic_coboundary` require the optional SymPy backend. The prime-power, Bell, exponent-lattice, and q-Kummer paths otherwise use the Python standard library and Komorebi Math's exact records.

> **Finite-evidence boundary.** A returned exact residue is exact for its selected p^r, index, and parameters. A finite certificate establishes the identity encoded by that certificate. Neither result is an unrestricted theorem about an unexecuted family of inputs.

24.1 Factorial-power prime-power kernels

24.1.1 Normalized structure constants

Let a weighted monomial basis have normalization

$$\gamma_n = c^n (n!)^s u_n,$$

where $s \geq 0$ is an integer and every u_n is a p-adic unit. Its product structure constant at (n, k) is

$$K_{n,k} = \frac{\gamma_n}{\gamma_k \gamma_{n-k}} = \binom{n}{k}^s \frac{u_n}{u_k u_{n-k}}.$$

The geometric factor c^n cancels. If

$$v = v_p\binom{n}{k},$$

then

$$v_p(K_{n,k}) = sv.$$

The kernel survives modulo p^r exactly when $sv < r$. The compiler keeps the valuation separate from the unit so it never attempts to invert a multiple of p.

24.1.2 Prime-power compiler signatures

The raw binomial compiler and its immutable result record are

```
PrimePowerUnitCompiler(
    prime: int,
    exponent: int,
    max_modulus: int = 2_000_000,
) -> None

PrimePowerUnitCompiler.unit_factorial(
    self,
    index: int,
) -> int

PrimePowerUnitCompiler.binomial_data(
    self,
    index: int,
    subindex: int,
) -> PrimePowerBinomial
```

```
PrimePowerUnitCompiler.binomial(
    self,
    index: int,
    subindex: int,
) -> int

PrimePowerBinomial(
    index: int,
    subindex: int,
    prime: int,
    exponent: int,
    modulus: int,
    valuation: int,
    unit: int,
    residue: int,
) -> None

PrimePowerBinomial.survives: bool
```

The normalized-kernel layer is

```
PrimePowerKernelCompiler(
    prime: int,
    exponent: int,
    factorial_power: int,
    unit_sequence: Callable[[int], int] | None = None,
    max_modulus: int = 2_000_000,
) -> None

PrimePowerKernelCompiler.modulus: int

PrimePowerKernelCompiler.kernel(
    self,
    index: int,
    subindex: int,
) -> NormalizedKernelResidue

NormalizedKernelResidue(
    index: int,
    subindex: int,
    factorial_power: int,
    prime: int,
    exponent: int,
    valuation: int,
    unit: int,
    residue: int,
) -> None

NormalizedKernelResidue.survives: bool
```

`unit_sequence=None` selects the constant sequence $u_n = 1$. A custom callback is evaluated at n, k, and $n - k$. All three returned values must be coprime to p.

The implementation module also defines

```
constant_unit_sequence(_index: int) -> int
```

as the callback that always returns one. It is not re-exported in `komorebi_math.generating.__all__`. Use `unit_sequence=None` on the supported umbrella surface.

The compiler data members are intentionally inspectable:

Compiler	Members established at construction
`PrimePowerUnitCompiler`	`prime`, `exponent`, `max_modulus`, `modulus`, `prefix_units`, `block_unit`
`PrimePowerKernelCompiler`	`prime`, `exponent`, `factorial_power`, `unit_sequence`, `max_modulus`, `binomial_compiler`

`prefix_units` is the p-free prefix table and `block_unit` is its final entry. Treat both as read-only implementation data. The unit compiler also maintains a private memoization dictionary named `_cache`.

24.1.3 Record-field interpretation

Field	Meaning
`prime`	selected prime p
`exponent`	modulus exponent r
`modulus`	p^r, present on the raw binomial record or compiler
`valuation`	$v_p\binom{n}{k}$ for `PrimePowerBinomial`, or $sv_p\binom{n}{k}$ for `NormalizedKernelResidue`
`unit`	surviving unit factor modulo p^r
`residue`	reconstructed result modulo p^r
`survives`	whether `valuation < exponent`

When the valuation is at least r, the residue is zero and the implementation stores `unit=0` as a sentinel. In that branch, `unit` is not the mathematical p-free unit of the vanished coefficient.

24.1.4 Exact kernel example

```
from dataclasses import asdict
from math import comb

from komorebi_math.generating import (
    PrimePowerKernelCompiler,
    PrimePowerUnitCompiler,
)

raw = PrimePowerUnitCompiler(3, 5).binomial_data(137, 52)
kernel = PrimePowerKernelCompiler(3, 5, 2).kernel(137, 52)

print(asdict(raw))
print(asdict(kernel))
print(kernel.residue == pow(comb(137, 52), 2, 3**5))
```

Output:

```
{'index': 137, 'subindex': 52, 'prime': 3, 'exponent': 5,
 'modulus': 243, 'valuation': 2, 'unit': 211, 'residue': 198}
{'index': 137, 'subindex': 52, 'factorial_power': 2, 'prime': 3,
 'exponent': 5, 'valuation': 4, 'unit': 52, 'residue': 81}
True
```

The same untwisted kernel can be evaluated from the command line:

```
python -m komorebi_math generating kernel \
  --index 137 --subindex 52 --prime 3 --exponent 5 --power 2
```

24.1.5 Compilation cost and resource guard

Initialization constructs a prefix-unit table of length $p^r + 1$. Its time and memory costs are therefore $O(p^r)$. A completed unit-factorial query uses $O(\log_p n)$ iterations and is memoized by `index`. A binomial query uses three unit-factorial values, one modular inverse, and the valuation calculation.

`max_modulus` guards this table directly. It is not a `GeneratingBudget` object, and the default compiler refuses $p^r > 2{,}000{,}000$. Raising the limit is an explicit memory decision.

Invalid condition	Result
`prime` is composite	`ValueError("prime must be prime")`
`exponent < 1`	`ValueError("exponent must be positive")`
`p**exponent > max_modulus`	`ValueError` before prefix allocation
`factorial_power < 0`	`ValueError`
`subindex` lies outside $[0, n]$	`ValueError`
custom unit is divisible by p	`ValueError("unit_sequence must return p-adic units")`

The primality check is deterministic trial division. It is suitable for the finite moduli targeted here, not a large-prime certification engine.

24.2 p-adic units and valuations

24.2.1 Valuation primitives

The four exact primitives are

```
factorial_valuation(
    index: int,
    prime: int,
) -> int

integer_valuation(
    value: int,
    prime: int,
) -> int

binomial_valuation(
    index: int,
    subindex: int,
    prime: int,
) -> int

borrow_count(
    index: int,
    subindex: int,
    prime: int,
) -> int
```

Legendre's formula is evaluated directly:

$$v_p(n!) = \sum_{j \geq 1} \left\lfloor \frac{n}{p^j} \right\rfloor .$$

The binomial valuation is then

$$v_p\binom{n}{k} = v_p(n!) - v_p(k!) - v_p((n-k)!).$$

Kummer's carry theorem gives the digit interpretation used by `borrow_count`:

$$v_p\binom{n}{k} = \text{number of base-}p\text{ borrows in } n - k.$$

The implementation counts borrows while subtracting `subindex` from `index`. It does not construct the potentially enormous integer $\binom{n}{k}$.

24.2.2 Unit factorials

Write

$$n! = p^{v_p(n!)} U_p(n), \qquad p \nmid U_p(n).$$

`PrimePowerUnitCompiler.unit_factorial(n)` returns $U_p(n) \bmod p^r$. The raw binomial compiler reconstructs

$$\binom{n}{k} \equiv p^{v} U_p(n) \left(U_p(k) U_p(n-k)\right)^{-1} \pmod{p^r}.$$

```
from komorebi_math.generating import (
    binomial_valuation,
    borrow_count,
    factorial_valuation,
    integer_valuation,
)

print(factorial_valuation(137, 3))
print(binomial_valuation(137, 52, 3))
print(borrow_count(137, 52, 3))
print(integer_valuation(3**4 * 10, 3))
```

Output:

```
66
2
2
4
```

24.2.3 Domain boundaries

`integer_valuation` returns only finite valuations. It rejects zero instead of returning an infinity sentinel:

```
integer_valuation(0, 3)
# ValueError: zero has no finite valuation
```

All indices must be nonnegative, every `subindex` must lie between zero and `index`, and every radix supplied to these primitives must be prime. These functions concern ordinary integers. Rational-valued valuation APIs and truncated p-adic coefficient domains are documented with the local arithmetic models in Chapter 25.

24.3 Generalized Bell digit reduction

24.3.1 Factorial-power Bell family

For a positive integer s, Komorebi Math defines the generalized Bell polynomials by

$$\exp\left(x\sum_{m\ge 1}\frac{z^m}{(m!)^s}\right)=\sum_{n\ge 0}B_n^{(s)}(x)\frac{z^n}{(n!)^s}.$$

Their exact recurrence is

$$B_n^{(s)}(x)=x\sum_{m=1}^{n}\binom{n-1}{m-1}\binom{n}{m}^{s-1}B_{n-m}^{(s)}(x).$$

Let $s_p(n)$ denote the sum of the base-p digits of n. When

$$s\ge 2,\qquad s\equiv 1 \pmod{p-1},$$

the implemented reduction compares the two coefficient rows

$$B_n^{(s)}(x)\equiv B_{s_p(n)}^{(1)}(x) \pmod{p}.$$

The comparison is coefficientwise. It is stronger than comparing only the values at $x=1$.

24.3.2 Reduction signatures and result record

```
generalized_bell_digit_reduction(
    index: int,
    prime: int,
    factorial_power: int,
    *,
    budget: GeneratingBudget = DEFAULT_GENERATING_BUDGET,
) -> BellDigitReduction

weighted_bell_digit_reduction(
    index: int,
    prime: int,
    factorial_power: int,
    digit_weights: Iterable[int],
    *,
    budget: GeneratingBudget = DEFAULT_GENERATING_BUDGET,
) -> BellDigitReduction

BellDigitReduction(
    index: int,
    prime: int,
    factorial_power: int,
```

```
    digit_sum: int,
    high_polynomial: tuple[int, ...],
    reduced_polynomial: tuple[int, ...],
) -> None

BellDigitReduction.valid: bool
BellDigitReduction.require(self) -> None
BellDigitReduction.certificate(self) -> Certificate
```

`high_polynomial` stores the degree-n row reduced modulo p. `reduced_polynomial` stores the smaller degree-$s_p(n)$ row. The `valid` property compares them after padding the shorter tuple with zeros.

The weighted form uses an integer table c_j and sets the high-level block weight at m to $c_{s_p(m)}$. It compares that row with the ordinary factorial-power-one Bell row built from the same digit-weight table. The table must contain entries through the maximum digit sum encountered up to `index`.

24.3.3 Pointed digit identity

The Bell proof machinery exposes its pointed subindex identity separately:

```
pointed_subindex_polynomial(
    index: int,
    prime: int,
) -> tuple[int, ...]

certify_pointed_subindex_identity(
    index: int,
    prime: int,
) -> PointedSubindexCertificate

PointedSubindexCertificate(
    index: int,
    prime: int,
    left: tuple[int, ...],
    right: tuple[int, ...],
) -> None

PointedSubindexCertificate.valid: bool
PointedSubindexCertificate.require(self) -> None
PointedSubindexCertificate.certificate(self) -> Certificate
```

For positive n, the coefficient of u^j on the left groups $\binom{n-1}{m-1}$ over positive base-p digitwise subindices m with digit sum $j+1$. The certified identity is

$$\sum_{j\geq 0}\left(\sum_{\substack{m\preceq_p n,\ m>0\\ s_p(m)=j+1}}\binom{n-1}{m-1}\right)u^j\equiv(1+u)^{s_p(n)-1}\pmod p.$$

24.3.4 Bell reduction example

```
from komorebi_math.generating import (
    certify_pointed_subindex_identity,
    generalized_bell_digit_reduction,
)

pointed = certify_pointed_subindex_identity(137, 5)
reduction = generalized_bell_digit_reduction(137, 5, 9)
```

```
print(pointed.left)
print(pointed.right)
print(reduction.digit_sum)
print(reduction.reduced_polynomial)
print(reduction.valid)
```

Output:

```
(1, 4, 1, 4, 1)
(1, 4, 1, 4, 1)
5
(0, 1, 0, 0, 0, 1)
True
```

Both producer functions call `require()` before returning. A successful result has already passed its internal exact comparison. Calling `.certificate()` adapts that comparison to the common `Certificate` interface and records the theorem identifiers `NGF.POINTED_SUBINDEX` or `NGF.BELL_DIGIT_SUM_REDUCTION`.

24.3.5 Parity and residue automata

The parity special case is the three-state binary automaton

```
generalized_bell_parity(
    index: int,
    factorial_power: int,
) -> int

BellParityAutomaton(
    outputs: tuple[int, int, int] = (1, 1, 0),
) -> None

BellParityAutomaton.transition(
    self,
    state: int,
    digit: int,
) -> int

BellParityAutomaton.evaluate(
    self,
    index: int,
) -> int
```

For $s \geq 2$, the returned parity is zero exactly when

$$\operatorname{popcount}(n) \equiv 2 \pmod 3.$$

The general prime automaton uses the period bound and Touchard's recurrence:

$$P_p = \frac{p^p - 1}{p - 1},$$
$$B_{n+p} \equiv B_{n+1} + B_n \pmod p.$$

Its exact interface is

```
bell_period_bound(
    prime: int,
) -> int

BellResidueAutomaton.compile(
    prime: int,
```

```
    factorial_power: int,
    *,
    budget: GeneratingBudget = DEFAULT_GENERATING_BUDGET,
) -> BellResidueAutomaton

BellResidueAutomaton.transition(
    self,
    state: int,
    digit: int,
) -> int

BellResidueAutomaton.evaluate(
    self,
    index: int,
) -> int

BellResidueAutomaton.certificate(
    self,
    count: int,
) -> Certificate
```

The compiled fields are `prime`, `factorial_power`, `period_bound`, and `outputs`. Evaluation reads the base-p digits least significant first and adds them modulo P_p, so the terminal state is the digit sum modulo the period. The output table supplies the corresponding ordinary Bell residue.

```
from komorebi_math.generating import BellResidueAutomaton

automaton = BellResidueAutomaton.compile(3, 5)
print(automaton.period_bound)
print(automaton.outputs)
print(automaton.evaluate(137))
print(automaton.certificate(30).valid)
```

Output:

```
13
(1, 1, 2, 2, 0, 1, 2, 1, 0, 0, 1, 0, 1)
1
True
```

24.3.6 Bell resource limits and construction discipline

The period grows extremely quickly. The default budget permits 100,000 reachable states. Small-prime examples remain manageable, while a prime-seven automaton has 137,257 states and requires an explicitly larger budget:

```
from komorebi_math.generating import BellResidueAutomaton, GeneratingBudget

budget = GeneratingBudget(maximum_reachable_states=140_000)
automaton = BellResidueAutomaton.compile(7, 13, budget=budget)
assert automaton.period_bound == 137_257
```

`generalized_bell_digit_reduction` and its weighted variant enforce `budget.maximum_degree`, but the current path does not apply `maximum_coefficient_bits` to the intermediate exact Bell rows. `BellResidueAutomaton.certificate(count)` also accepts no budget argument and constructs exact generalized Bell rows through `count`. Keep certificate counts bounded by design.

Use `BellResidueAutomaton.compile`, not the dataclass constructor, to establish the prime, power, period, and output-table invariants. Likewise, the default `BellParityAutomaton()` is the theorem-carrying parity machine. Supplying custom outputs creates a different transducer.

24.4 Cyclotomic collapse

24.4.1 Finite Mahler coboundaries

Let $\Phi_m(z)$ be the m-th cyclotomic polynomial, normalized to have constant coefficient one, and let a finite mask be

$$R(z) = \prod_{m \geq 1} \Phi_m(z)^{e_m}.$$

The collapse problem asks whether there is a finite cyclotomic product

$$Q(z) = \prod_{m \geq 1} \Phi_m(z)^{q_m}$$

such that

$$R(z) = \frac{Q(z)}{Q(z^p)}.$$

Write each index uniquely as

$$m = up^j, \qquad p \nmid u.$$

`p_free_orbit` returns `(u, j)`. Within one p-power orbit, a nonzero mask exponent at level zero is the exact obstruction. When all level-zero exponents vanish, the closed-form exponent at level $j - 1$ is the negative tail sum

$$q_{up^{j-1}} = -\sum_{\ell \geq j} e_{up^\ell}.$$

24.4.2 Cyclotomic API and member ledger

```
p_free_orbit(
    index: int,
    prime: int,
) -> tuple[int, int]

cyclotomic_coboundary(
    prime: int,
    exponents: Mapping[int, int],
) -> CyclotomicCollapse

CyclotomicCollapse(
    prime: int,
    mask_exponents: tuple[tuple[int, int], ...],
    closed_form_exponents: tuple[tuple[int, int], ...],
    obstruction_orbits: tuple[int, ...],
) -> None

CyclotomicCollapse.collapses: bool
CyclotomicCollapse.require(self) -> None

verify_cyclotomic_exponent_relation(
    collapse: CyclotomicCollapse,
) -> bool

normalized_cyclotomic_expression(
    exponents: Mapping[int, int],
) -> Any
```

```
verify_cyclotomic_coboundary(
    collapse: CyclotomicCollapse,
) -> bool
```

`mask_exponents` is the sorted nonzero input. `closed_form_exponents` is the sorted finite solution. `obstruction_orbits` lists the p-free orbit labels whose level-zero exponent is nonzero. `collapses` is true exactly when that last tuple is empty.

The exponent-lattice verifier is exact and backend-free. The expression constructor and symbolic verifier import SymPy lazily and raise `BackendUnavailableError` when it is unavailable.

24.4.3 Successful collapse and explicit obstruction

```
from dataclasses import asdict

from komorebi_math.generating import (
    cyclotomic_coboundary,
    verify_cyclotomic_exponent_relation,
)

good = cyclotomic_coboundary(3, {3: 1, 9: -1})
bad = cyclotomic_coboundary(3, {1: 1, 3: -1, 10: 2})

print(asdict(good))
print(verify_cyclotomic_exponent_relation(good))
print(bad.collapses)
print(bad.obstruction_orbits)
```

Output:

```
{'prime': 3, 'mask_exponents': ((3, 1), (9, -1)),
 'closed_form_exponents': ((3, 1),), 'obstruction_orbits': ()}
True
False
(1, 10)
```

`bad.require()` raises

```
ValidationError: p-free cyclotomic obstructions: (1, 10)
```

If any orbit is obstructed, the returned `closed_form_exponents` is empty. The function does not return a partial solution for the unobstructed orbits.

24.4.4 Input and cost boundaries

For M nonzero mask entries, orbit grouping requires approximately $O(M \log_p m_{\max})$ integer divisions. The finite exponent verification is linear in the number of returned exponents. Symbolically expanding cyclotomic polynomials can be much more expensive and should be treated as a separate optional audit.

The public annotation requires an integer-to-integer mapping. The current implementation applies `int()` to every key and exponent instead of rejecting nonintegral numeric inputs. For example, `3.9` is silently converted to `3`. Validate external data as integers before calling `cyclotomic_coboundary`. This also avoids collisions between distinct external keys after coercion.

24.5 Gaussian q-Kummer valuation

24.5.1 Integer Gaussian binomials

Here q is an integer base greater than one, not a symbolic indeterminate. The elementary objects are

$$[n]_q = \frac{q^n - 1}{q - 1}, \qquad [n]_q! = \prod_{j=1}^{n} [j]_q,$$

and

$$\begin{bmatrix} n \\ k \end{bmatrix}_q = \frac{[n]_q!}{[k]_q![n-k]_q!}.$$

The direct constructors are

```
q_integer(index: int, base: int) -> int
q_factorial(index: int, base: int) -> int
gaussian_binomial(index: int, subindex: int, base: int) -> int
q_binomial_valuation_direct(
    index: int,
    subindex: int,
    base: int,
    prime: int,
) -> int
```

They construct exact Python integers and are intended for examples, bounded verification, and small direct computations. A q-integer has $\Theta(n \log q)$ bits, a q-factorial has $\Theta(n^2 \log q)$ bits, and a typical Gaussian binomial has $\Theta(k(n-k) \log q)$ bits. The decomposition below avoids constructing the Gaussian binomial itself.

24.5.2 Mixed-radix valuation formula

For $p \nmid q$, select a block length d, and write

$$n = dN + r, \qquad k = dK + s, \qquad 0 \le r, s < d.$$

Set

$$\delta = \mathbf{1}_{s>r}, \qquad C = v_p\binom{N}{K}, \qquad E = \begin{cases} v_p(N-K), & \delta = 1, \\ 0, & \delta = 0. \end{cases}$$

The implemented formula is

$$v_p\left(\begin{bmatrix} n \\ k \end{bmatrix}_q\right) = C + \delta\lambda + E,$$

with branch-specific d and λ.

Arithmetic branch	Block length d	Lift constant λ
$p \mid q$	1	0, and the complete valuation is 0
odd p, $p \nmid q$	$\mathrm{ord}_p(q)$	$v_p(q^d - 1)$, subtracting $v_p(q-1)$ when $d = 1$
$p = 2$, odd q	2	$v_2(q+1)$

The separate two-adic branch is essential. The multiplicative order modulo two alone would not encode the required parity block.

24.5.3 Exact decomposition example

```
from dataclasses import asdict

from komorebi_math.generating import (
    q_binomial_valuation_decomposition,
    q_binomial_valuation_direct,
)

record = q_binomial_valuation_decomposition(48, 22, 2, 5)

print(asdict(record))
print(q_binomial_valuation_direct(48, 22, 2, 5))
print(record.survives(exponent=2))
print(record.survives(exponent=2, power=2))
```

Output:

```
{'index': 48, 'subindex': 22, 'base': 2, 'prime': 5, 'order': 4,
 'lift_constant': 1, 'quotient_index': 12, 'quotient_subindex': 5,
 'residue_carry': 1, 'quotient_carries': 0,
 'endpoint_valuation': 0, 'valuation': 1}
1
True
False
```

The direct value is useful as an adversarial oracle at this scale. For large indices, the decomposition record is the intended path.

24.6 Congruence certificates and adversarial cases

24.6.1 Evidence capabilities

The four subsystems do not all expose the same certificate type.

Producer	Evidence operation	Exact claim
`certify_pointed_subindex_identity`	`.require()` or `.certificate()`	two finite coefficient tuples agree modulo p
`generalized_bell_digit_reduction`	`.require()` or `.certificate()`	high and digit-reduced Bell rows agree modulo p
`BellResidueAutomaton.compile`	`.certificate(count)`	compiled outputs match the first `count` exact generalized Bell rows
`cyclotomic_coboundary`	`.require()`	no level-zero orbit obstruction remains
`verify_cyclotomic_exponent_relation`	Boolean	returned exponents reconstruct the mask divisor relation
`verify_cyclotomic_coboundary`	Boolean, optional SymPy	symbolic rational-function identity holds
`PrimePowerKernelCompiler.kernel`	no certificate method	exact residue record for one query
`q_binomial_valuation_decomposition`	no certificate method	exact branch record for one query

The immutable dataclasses are transparent records, not unforgeable proof objects. Their public constructors permit a caller to fabricate inconsistent field values. Trust a record as computed evidence only when it comes from the documented producer, and recompute from its defining inputs when reading an external payload.

24.6.2 Adversarial rejection examples

```
from komorebi_math.generating import (
    BellResidueAutomaton,
    PrimePowerKernelCompiler,
    PrimePowerUnitCompiler,
    cyclotomic_coboundary,
    generalized_bell_digit_reduction,
    integer_valuation,
)

checks = (
    lambda: integer_valuation(0, 3),
    lambda: PrimePowerUnitCompiler(101, 4),
    lambda: PrimePowerKernelCompiler(
        3, 3, 2, lambda _index: 3
    ).kernel(4, 2),
    lambda: generalized_bell_digit_reduction(10, 5, 8),
    lambda: BellResidueAutomaton.compile(7, 13),
    lambda: cyclotomic_coboundary(
        3, {1: 1, 3: -1, 10: 2}
    ).require(),
)

for check in checks:
    try:
        check()
    except Exception as error:
        print(type(error).__name__, str(error))
```

Output:

```
ValueError zero has no finite valuation
ValueError modulus 104060401 exceeds max_modulus=2000000
ValueError unit_sequence must return p-adic units
ValidationError factorial_power must be at least two and congruent to one modulo prime-1
ResourceLimitError state request 137257 exceeds maximum_reachable_states=100000
ValidationError p-free cyclotomic obstructions: (1, 10)
```

These failures occur before a successful result is reported. A false `BellDigitReduction` or `PointedSubindexCertificate` created manually is rejected by `.require()`.

24.6.3 Persistence boundary

The records in this chapter do not have registered Komorebi portable codecs in version `1.0.0b4`. Passing `NormalizedKernelResidue`, `BellDigitReduction`, `PointedSubindexCertificate`, `CyclotomicCollapse`, or `QKummerDecomposition` to `dumps_portable` raises a `ValidationError` stating that the type has no portable codec.

For application-owned persistence, store a versioned primitive mapping and recompute the result on load:

```
import json
from dataclasses import asdict

from komorebi_math.generating import q_binomial_valuation_decomposition

record = q_binomial_valuation_decomposition(48, 22, 2, 5)
payload = {
    "schema": "example.q-kummer-query",
    "schema_version": 1,
    "inputs": {
        "index": record.index,
```

```
        "subindex": record.subindex,
        "base": record.base,
        "prime": record.prime,
    },
    "observed": asdict(record),
}
text = json.dumps(payload, sort_keys=True)

loaded = json.loads(text)
recomputed = q_binomial_valuation_decomposition(**loaded["inputs"])
assert asdict(recomputed) == loaded["observed"]
```

Persisting inputs plus observed output is preferable to reconstructing a dataclass directly because the producer reruns all branch and domain checks. Compiler instances should not be persisted as records: they contain prefix tables, caches, and, for custom kernels, an arbitrary callable.

24.6.4 Known limitations and current defects

Boundary	Consequence and safe practice
Research-tier surface	Pin the package version and run the strict public-API check when signatures matter.
Finite prime-power table	Choose `max_modulus` from an explicit memory budget.
Bell state explosion	Inspect `bell_period_bound(p)` before compilation and pass an intentional `GeneratingBudget`.
Bell coefficient growth	Keep exact row and certificate counts bounded. The Bell reducers do not currently enforce the coefficient-bit budget.
Optional symbolic backend	Use `verify_cyclotomic_exponent_relation` when SymPy is unavailable.
Cyclotomic input coercion	Prevalidate integer keys and exponents because nonintegers are currently truncated with `int()`.
Direct q arithmetic	Reserve `gaussian_binomial` and direct valuations for bounded checks. Integer size grows rapidly.
q-factorial boundary inconsistency	`q_factorial(0, base)` currently returns 1 without validating `base`. Validate `base > 1` before every call.
Gaussian out-of-range convention	`gaussian_binomial` returns 0 for `k < 0` or `k > n`, whereas the valuation decomposition raises. Do not mix the two failure conventions.
No portable codecs	Persist versioned inputs and recompute, rather than treating a hand-built record as evidence.

The `q_factorial(0, base)` behavior and cyclotomic numeric coercion are current implementation limitations, not mathematical conventions endorsed by this guide.

24.6.5 Executed evidence for this chapter

The focused replay below was executed against the documented `1.0.0b4` source tree:

```
tests/test_generating_cartier_padic.py
tests/test_generating_domains.py
tests/test_block13_cli_api.py

31 passed, 36 deselected
```

The installed verification commands additionally reported:

```
komorebi_math verify generating   14/14 passed
komorebi_math verify block13      146/146 passed
```

The shipped `examples/prime_power_generating.py` produced:

```
{'kernel_residue': 81, 'kernel_valuation': 4,
 'bell_digit_sum': 5, 'q_binomial_valuation': 1}
```

The generating verifier also exercises portable generating records from other chapters. That pass does not imply that the five chapter-specific record types listed above have portable codecs.

24.7 `generating.q_kummer`: Gaussian-binomial branches and valuation records

24.7.1 Complete q-Kummer callable ledger

The module-level public callables re-exported through `komorebi_math.generating` are

```
multiplicative_order_mod_prime(
    value: int,
    prime: int,
) -> int

q_integer(
    index: int,
    base: int,
) -> int

q_factorial(
    index: int,
    base: int,
) -> int

gaussian_binomial(
    index: int,
    subindex: int,
    base: int,
) -> int

q_binomial_valuation_decomposition(
    index: int,
    subindex: int,
    base: int,
    prime: int,
) -> QKummerDecomposition

q_binomial_valuation_direct(
    index: int,
    subindex: int,
    base: int,
```

```
    prime: int,
) -> int

q_binomial_support(
    index: int,
    base: int,
    prime: int,
    exponent: int,
    power: int = 1,
) -> tuple[int, ...]
```

`multiplicative_order_mod_prime` requires a prime modulus and a value coprime to it. Its simple loop tests successive powers through `prime - 1`. The implementation does not factor $p-1$ or cache orders.

24.7.2 `QKummerDecomposition` member ledger

```
QKummerDecomposition(
    index: int,
    subindex: int,
    base: int,
    prime: int,
    order: int,
    lift_constant: int,
    quotient_index: int,
    quotient_subindex: int,
    residue_carry: int,
    quotient_carries: int,
    endpoint_valuation: int,
    valuation: int,
) -> None

QKummerDecomposition.survives(
    self,
    exponent: int,
    power: int = 1,
) -> bool
```

Member	Interpretation
`index`, `subindex`	n, k
`base`, `prime`	integer Gaussian base q and valuation prime p
`order`	branch block length d
`lift_constant`	branch constant λ
`quotient_index`, `quotient_subindex`	N, K after division by d
`residue_carry`	$\delta = 1$ when the subindex residue exceeds the index residue
`quotient_carries`	$v_p\binom{N}{K}$
`endpoint_valuation`	$v_p(N-K)$ in the residue-carry branch, otherwise zero
`valuation`	complete $v_p(\begin{bmatrix} n \\ k \end{bmatrix}_q)$

`survives(r, a)` returns

$$a\, v_p\left(\begin{bmatrix} n \\ k \end{bmatrix}_q\right) < r,$$

which is equivalent to nonvanishing of $(\begin{bmatrix} n \\ k \end{bmatrix}_q)^a \bmod p^r$.

24.7.3 Support compilation

`q_binomial_support` applies the decomposition to every $k = 0, \dots, n$:

```
from komorebi_math.generating import q_binomial_support

print(q_binomial_support(12, 2, 5, exponent=1))
print(q_binomial_support(20, 3, 2, exponent=2))
```

Output:

```
(0, 4, 8, 12)
(0, 2, 4, 8, 12, 16, 18, 20)
```

The routine is exact, but it recomputes the primality and order branch for each subindex. With the present simple order search, a support sweep is roughly $O(n(p + \log_p n))$ in the coprime odd-prime branch. If many rows share `base` and `prime`, application code should budget accordingly.

24.7.4 q-Kummer failure semantics

Condition	API behavior
composite `prime`	decomposition and order helper raise `ValueError`
`base <= 1`	decomposition and `q_integer` raise `ValueError`
`subindex` outside $[0, n]$	decomposition raises, `gaussian_binomial` returns 0
direct Gaussian value is zero	direct valuation raises `ValueError`
`exponent < 1` or `power < 1`	support and `.survives()` raise `ValueError`
order helper receives a multiple of p	`ValueError("value must be coprime to prime")`

Chapter 24 summary. Prime-power compilers separate valuation from unit arithmetic and make modular support explicit. Bell reducers turn a high index into a certified digit-sum row, with finite automata available only when their state budget is acceptable. Cyclotomic collapse is an exact orbit-coboundary calculation whose obstruction is visible at level zero. q-Kummer records split every integer Gaussian-binomial valuation into one of three arithmetic branches, quotient carries, and an endpoint correction. The results are exact but research-tier, finite, and currently nonportable. Persist defining inputs and recompute them when evidence crosses a process boundary.

25. Local arithmetic models

First route. Read the scalar and precision conventions in Section 25.1. Then choose Hurwitz coefficients, Euler coordinates, Mahler coordinates, or a recurrence packet according to the representation you need. A modulus changes which values can be distinguished: the residue one modulo five, for example, is compatible with the integers one, six, and eleven. Keep the original exact data if a later calculation will need information beyond the chosen precision.

Local arithmetic is the point where an exact global object is viewed through a finite modulus, a prime field, or a bounded p-adic precision. Komorebi Math keeps those views finite. It does not construct an infinite p-adic field or claim analytic convergence from a finite residue calculation. Instead, the package supplies exact residue rings, valuations, finite Hurwitz algebra, certified Euler-coordinate truncations, Mahler transfers, residue packets, and finite congruence certificates.

The main data flow is

$$\text{exact values} \longrightarrow \text{local or residue coordinates} \longrightarrow \text{finite reconstruction or congruence certificate.}$$

The arrow is reversible only where the relevant class supplies and verifies a reconstruction map. Reduction modulo p^r, for example, discards information and is not reversed by asking for a `TruncatedPadic` object at greater precision.

25.0 Scope, API tiers, and imports

The arithmetic package has three distinct public surfaces.

Surface	Contract in `1.0.0b4`	Arithmetic content
`komorebi_math.stable`	Frozen stable signatures, semantics, and persistence	`ArithmeticPolyphase`, stable analyze/synthesize aliases, stable mixed-radix aliases, and `mahler_polyphase`
`komorebi_math`	Pre-beta additive research surface	Ten selected arithmetic names, including `HurwitzSeries`, `MahlerSeries`, `LinearRecurrence`, and `EulerLocalCoordinates`
`komorebi_math.arithmetic`	Complete implemented arithmetic namespace	Every class and function documented in this chapter

`komorebi_math.beta` exports no arithmetic symbol in this release. The ten arithmetic names in the root research manifest are `EulerLocalCoordinates`, `HurwitzSeries`, `LinearRecurrence`, `MahlerSeries`, `StandardWindow`, `TruncatedPadic`, `affine_mahler_kernel`, `fibonacci_recurrence`, `padic_valuation`, and `signed_euler_number`.

Code that needs the complete arithmetic namespace should import it explicitly:

```
from komorebi_math.arithmetic import (
    EulerLocalCoordinates,
    HurwitzSeries,
    LinearRecurrence,
    MahlerSeries,
    StandardWindow,
    TruncatedPadic,
)
```

The stable polyphase spellings are intentionally qualified:

```
from komorebi_math.stable import (
    ArithmeticPolyphase,
    arithmetic_mixed_radix_address,
    arithmetic_mixed_radix_digits,
    arithmetic_polyphase_analyze,
    arithmetic_polyphase_synthesize,
    mahler_polyphase,
)
```

The shorter `polyphase_analyze` name in `komorebi_math.arithmetic` is the same arithmetic operation, but that spelling is not part of the stable namespace. Do not confuse it with array-oriented signal polyphase analysis from `komorebi_math.signals`.

The administrative names `FEATURE_KEY`, `status()`, and `require()` report or confirm arithmetic feature availability. They do not perform arithmetic.

25.1 Scalar domains and valuations

25.1.1 Finite scalar domains

The three domain classes represent canonical integer residues:

$$\mathbb{Z}/M\mathbb{Z}, \qquad \mathbb{F}_p, \qquad \mathbb{Z}/p^r\mathbb{Z}.$$

Their exact constructors are

```
ModularRing(modulus: int) -> None
PrimeField(modulus: int) -> None
TruncatedPadic(prime: int, precision: int) -> None
```

`ModularRing` requires $M \geq 2$. `PrimeField` additionally verifies that its modulus is prime. `TruncatedPadic` requires a prime p and precision $r \geq 1$, then exposes the modulus p^r.

Type	Public member	Meaning
`ModularRing`	`normalize(value)`	Canonical residue in `0, ..., modulus - 1`
	`add(left, right)`	Residue sum
	`subtract(left, right)`	Residue difference
	`multiply(left, right)`	Residue product
	`power(base, exponent)`	Modular nonnegative power
	`is_unit(value)`	Whether the residue is invertible
	`inverse(value)`	Inverse of a unit, otherwise `ValidationError`
`PrimeField`	`prime`	Field characteristic, equal to `modulus`
	inherited ring members	Exact prime-field residue arithmetic
`TruncatedPadic`	`modulus`	p^r
	`ring`	Corresponding `ModularRing`
	`normalize(value)`	Canonical residue modulo p^r
	`is_unit(value)`	Whether $p \nmid value$ at the visible precision
	`inverse(value)`	Inverse of a local unit modulo p^r
	`valuation(value)`	Valuation of the visible nonzero residue
	`refine(additional_digits=1)`	New domain with greater declared precision
	`coarsen(precision)`	New shallower domain

The exact callable signatures behind that table are

```
ModularRing.normalize(self, value: int) -> int
ModularRing.add(self, left: int, right: int) -> int
ModularRing.subtract(self, left: int, right: int) -> int
ModularRing.multiply(self, left: int, right: int) -> int
ModularRing.power(self, base: int, exponent: int) -> int
ModularRing.is_unit(self, value: int) -> bool
ModularRing.inverse(self, value: int) -> int

PrimeField.prime: int

TruncatedPadic.modulus: int
TruncatedPadic.ring: ModularRing
TruncatedPadic.normalize(self, value: int) -> int
TruncatedPadic.is_unit(self, value: int) -> bool
TruncatedPadic.inverse(self, value: int) -> int
TruncatedPadic.valuation(self, value: int) -> int | float
TruncatedPadic.refine(
    self, additional_digits: int = 1,
) -> TruncatedPadic
TruncatedPadic.coarsen(self, precision: int) -> TruncatedPadic
```

The arithmetic methods require integer inputs and reject booleans. Negative integers are accepted where reduction makes them meaningful.

```
from komorebi_math.arithmetic import ModularRing, PrimeField, TruncatedPadic

ring = ModularRing(12)
field = PrimeField(7)
local = TruncatedPadic(3, 4)

print(ring.normalize(-1), ring.multiply(7, 5), ring.inverse(5))
print(field.inverse(3))
print(local.modulus, local.valuation(27), local.refine(2).precision)
```

Output:

```
11 11 5
5
81 3 6
```

Precision boundary. `refine()` changes the declared modulus. It does not infer missing high digits of an already reduced residue. Also, a residue that is zero modulo p^r is reported by `TruncatedPadic.valuation()` as `float("inf")`. That result cannot distinguish exact zero from an unknown integer whose valuation is at least r.

25.1.2 Valuation and combinatorial primitives

The complete function inventory is

```
digit_sum(value: int, base: int) -> int
padic_valuation(value: Rational, prime: int) -> int | float
factorial_valuation(index: int, prime: int) -> int
binomial_valuation(index: int, choose: int, prime: int) -> int
binomial_valuation_from_digits(index: int, choose: int, prime: int) -> int
falling_factorial(value: int, order: int) -> int
carmichael_exponent_prime_power(prime: int, depth: int) -> int
unit_difference_value(
    base: int, index: int, step: int, order: int,
```

```
) -> int
unit_difference_valuation(
    base: int, index: int, step: int, order: int, prime: int,
) -> int | float
verify_binomial_digit_formula(
    index: int, choose: int, prime: int,
) -> bool
exact_binomial(index: int, choose: int) -> int
```

For a rational $x = a/b \neq 0$, normalized with $b > 0$,

$$v_p(x) = v_p(a) - v_p(b), \qquad v_p(0) = +\infty.$$

The factorial and binomial identities are

$$v_p(n!) = \sum_{j \geq 1} \left\lfloor \frac{n}{p^j} \right\rfloor,$$

$$v_p\binom{n}{k} = v_p(n!) - v_p(k!) - v_p((n-k)!) = \frac{s_p(k) + s_p(n-k) - s_p(n)}{p-1}.$$

The two binomial functions implement the two exact formulas independently. The boolean verification helper compares them.

```
from fractions import Fraction

from komorebi_math.arithmetic import (
    binomial_valuation,
    binomial_valuation_from_digits,
    factorial_valuation,
    padic_valuation,
)

print(padic_valuation(Fraction(40, 27), 2))
print(padic_valuation(Fraction(40, 27), 3))
print(factorial_valuation(20, 3))
print(binomial_valuation(20, 8, 3))
print(binomial_valuation_from_digits(20, 8, 3))
```

Output:

```
3
-3
8
1
1
```

`falling_factorial(n, r)` returns zero when $r > n \geq 0$. Likewise, `exact_binomial(n, k)` returns zero outside $0 \leq k \leq n$, rather than raising. The valuation functions reject invalid binomial indices because a valuation of that sentinel zero would have different semantics. All valuation bases must be prime.

The prime-power Carmichael exponent is

$$\lambda(p^d) = \begin{cases} (p-1)p^{d-1}, & p \text{ odd}, \\ 1, & p = 2,\ d = 1, \\ 2, & p = 2,\ d = 2, \\ 2^{d-2}, & p = 2,\ d \geq 3. \end{cases}$$

For the exponential mode $f(n) = u^n$, the implementation uses the closed finite-difference identity

$$\Delta_h^r u^n = u^n (u^h - 1)^r.$$

`unit_difference_valuation` requires u to be a unit at the declared prime. The lower-level `unit_difference_value` only evaluates the integer identity and does not impose that local-unit assumption.

25.1.3 Cost model

Operation	Arithmetic work, excluding big-integer bit cost
Ring add, subtract, multiply	$O(1)$ residue operations
Modular inverse	(O(log M)) arithmetic steps for modulus (M)
Modular nonnegative power	(O(log e)) modular multiplications for exponent (e)
`digit_sum(n, p)`	$O(\log_p n)$
`factorial_valuation(n, p)`	$O(\log_p n)$
Binomial digit valuation	$O(\log_p n)$
Closed exponential difference	Dominated by exponentiation and size of the exact integer

Python integers grow with the result. A formula that takes logarithmically many multiplications can still allocate a very large integer.

25.2 Hurwitz series

25.2.1 Coefficient convention and product

A finite Hurwitz series stores exponential-generating coefficients:

$$A(z) = \sum_{n=0}^{N} a_n \frac{z^n}{n!}.$$

Its product is the binomial convolution

$$(A \star_H B)_n = \sum_{k=0}^{n} \binom{n}{k} a_k b_{n-k}.$$

The low-level signatures are

```
hurwitz_product(
    left: Iterable[Number],
    right: Iterable[Number],
    *,
    degree: int | None = None,
    modulus: int | None = None,
) -> tuple[Number, ...]

hurwitz_inverse(
    coefficients: Iterable[Number],
    degree: int,
    *,
    modulus: int | None = None,
) -> tuple[Number, ...]
```

If `degree` is omitted from `hurwitz_product`, the function returns the complete finite product through degree `len(left) + len(right) - 2`. The inverse solves

$$u_0 = a_0^{-1}, \qquad u_n = -a_0^{-1} \sum_{r=1}^{n} \binom{n}{r} a_r u_{n-r}.$$

Over the rationals, $a_0 \neq 0$ is required. In modular mode, a_0 must be a unit modulo the declared modulus.

25.2.2 `HurwitzSeries`

```
HurwitzSeries(
    coefficients: Iterable[Number],
    modulus: int | None = None,
) -> None
```

Member	Signature or result
`coefficients`	Stored finite coefficient tuple
`modulus`	`None` or residue modulus
`degree`	Highest stored coefficient index
`coefficient(index)`	Stored coefficient, or zero beyond truncation
`truncate(degree)`	New finite series through `degree`
`multiply(other, *, degree=None)`	Hurwitz product
`inverse(degree=None)`	Reciprocal through a finite degree
`evaluate_egf(value)`	Evaluation of the stored EGF polynomial
`identity(degree, *, modulus=None)`	Multiplicative identity
`exponential_mode(value, degree, *, modulus=None)`	Coefficients $value^n$

```
HurwitzSeries.coefficient(self, index: int) -> Number
HurwitzSeries.truncate(self, degree: int) -> HurwitzSeries
HurwitzSeries.multiply(
    self,
    other: HurwitzSeries,
    *,
    degree: int | None = None,
) -> HurwitzSeries
HurwitzSeries.inverse(self, degree: int | None = None) -> HurwitzSeries
HurwitzSeries.evaluate_egf(self, value: Number) -> Number
HurwitzSeries.identity(
    degree: int, *, modulus: int | None = None,
) -> HurwitzSeries
HurwitzSeries.exponential_mode(
    value: int,
    degree: int,
    *,
    modulus: int | None = None,
) -> HurwitzSeries
```

Integer and `Fraction` coefficients become exact `Fraction` objects when no modulus is present. Floating and complex coefficients remain numerical. A modular series accepts integers only and reduces each one canonically.

Exponential modes linearize addition under the Hurwitz product:

$$(a^n)_{n\geq 0} \star_H (b^n)_{n\geq 0} = ((a+b)^n)_{n\geq 0}.$$

```
from komorebi_math.arithmetic import HurwitzSeries

left = HurwitzSeries.exponential_mode(2, 6)
```

```
right = HurwitzSeries.exponential_mode(3, 6)
product = left.multiply(right, degree=6)

print(product.coefficients)
print(HurwitzSeries((1, 1)).inverse(4).coefficients)
```

Output:

```
(Fraction(1, 1), Fraction(5, 1), Fraction(25, 1), Fraction(125, 1), Fraction(625, 1), Fraction(3125, 1),
    ↪ Fraction(15625, 1))
(Fraction(1, 1), Fraction(-1, 1), Fraction(2, 1), Fraction(-6, 1), Fraction(24, 1))
```

Modular evaluation boundary. `evaluate_egf()` performs ordinary numeric division by (n!). It is not a finite-field evaluation routine. Use the stored modular coefficients and explicit finite-field operations when the intended result lies in a residue ring.

25.2.3 Finite-field subgroup selector

For prime p, positive $m \mid p-1$, and the subgroup $\mu_m \subset \mathbb{F}_p^\times$,

$$\frac{1}{m}\sum_{\omega \in \mu_m} \omega^n = \mathbf{1}_{m|n} \pmod p.$$

The corresponding interfaces are

```
subgroup_roots(prime: int, order: int) -> tuple[int, ...]
subgroup_selector(prime: int, order: int, degree: int) -> HurwitzSeries
compressed_hurwitz_inverse(
    prime: int, compression: int, count: int,
) -> tuple[int, ...]
compressed_period(prime: int, compression: int) -> int
```

`count` is the highest compressed index, so `compressed_hurwitz_inverse(p, m, count)` returns `count + 1` entries. Its recurrence is

$$u_0 = 1, \qquad u_n = -\sum_{q<n} \binom{mn}{mq} u_q \pmod p,$$

and its proved positive-tail period is

$$h = \frac{p-1}{m}.$$

```
from komorebi_math.arithmetic import subgroup_roots, subgroup_selector

print(subgroup_roots(13, 4))
print(subgroup_selector(13, 4, 12).coefficients)
```

Output:

```
(1, 5, 8, 12)
(1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1)
```

Subgroup enumeration scans all nonzero residues of the prime field. It is a transparent finite verifier, not a large-prime discrete-log algorithm. Hurwitz multiplication and inversion are quadratic in the requested degree.

25.3 Standard residual windows

25.3.1 Window model

```
StandardWindow(prime: int, compression: int) -> None
```

The window has congruential order mp, where `compression = m`. The uniform tail theorem requires

$$p \text{ odd prime}, \qquad m \mid p-1, \qquad h = (p-1)/m.$$

Member	Meaning
`order`	mp
`has_uniform_period`	Whether the theorem assumptions hold
`period`	Proved positive-tail period h
`reduced_inverse(degree)`	Standard inverse residues through `degree`
`exact_coefficients(address, degree)`	Exact rational address coefficients
`residues(address, degree)`	Exact coefficients reduced modulo p
`rational_generating_function()`	Finite periodic-OGF record
`minimum_period()`	Minimum divisor-period of the computed tail block
`certificate(address, periods=4)`	Address, recurrence, and tail-period check

```
StandardWindow.order: int
StandardWindow.has_uniform_period: bool
StandardWindow.period: int
StandardWindow.reduced_inverse(self, degree: int) -> tuple[int, ...]
StandardWindow.exact_coefficients(
    self, address: int, degree: int,
) -> tuple[Fraction, ...]
StandardWindow.residues(
    self, address: int, degree: int,
) -> tuple[int, ...]
StandardWindow.rational_generating_function(
    self,
) -> PeriodicGeneratingFunction
StandardWindow.minimum_period(self) -> int
StandardWindow.certificate(
    self, address: int, periods: int = 4,
) -> ComputationResult[tuple[int, ...]]
```

The convenience functions are

```
rational_mod(value: Fraction | int, prime: int) -> int
standard_window_inverse(
    prime: int, compression: int, degree: int,
) -> tuple[int, ...]
standard_window_period(prime: int, compression: int) -> int
subgroup_standard_window_inverse(
```

```
    prime: int, compression: int, degree: int,
) -> tuple[int, ...]
```

`rational_mod` requires the rational denominator to be a unit modulo p. The last function reaches the same residue coefficients through the finite Hurwitz subgroup construction.

25.3.2 Positive-tail generating function

`PeriodicGeneratingFunction` is the finite record

```
PeriodicGeneratingFunction(
    prime: int,
    period: int,
    constant: int,
    numerator: tuple[int, ...],
) -> None
```

with methods `coefficient(index)` and `first(count)`.

```
PeriodicGeneratingFunction.coefficient(self, index: int) -> int
PeriodicGeneratingFunction.first(self, count: int) -> tuple[int, ...]
```

If `numerator = (u_1, ..., u_h)`, it represents

$$U(z) = u_0 + \frac{u_1 z + u_2 z^2 + \cdots + u_h z^h}{1 - z^h} \quad \text{in } \mathbb{F}_p[[z]].$$

The exceptional constant coefficient is outside the periodic tail. A statement that the entire sequence is periodic from index zero is stronger and is not what `StandardWindow.period` proves.

```
from komorebi_math.arithmetic import StandardWindow

window = StandardWindow(prime=7, compression=2)
result = window.certificate(address=3, periods=4)
result.certificate.require()

print(window.period, window.minimum_period())
print(window.residues(address=3, degree=12))
print(window.rational_generating_function().first(13))
```

Output:

```
3 3
(1, 6, 5, 2, 6, 5, 2, 6, 5, 2, 6, 5, 2)
(1, 6, 5, 2, 6, 5, 2, 6, 5, 2, 6, 5, 2)
```

The certificate checks that the chosen address reduces to the standard inverse, verifies the defining compressed inverse recurrence, and tests the claimed period through the requested number of repetitions. It records `minimum_period` as a diagnostic. The validity flag does not assert that the theorem period is always minimal.

`StandardWindow(2, 1)` is constructible, but `period`, `rational_generating_function`, `minimum_period`, and `certificate` reject it because the uniform-period theorem requires an odd prime. A compression that does not divide $p - 1$ behaves the same way. Direct construction of `PeriodicGeneratingFunction` performs no schema validation, so normal code should obtain that record from `rational_generating_function()`.

The inverse recurrence and its certificate are quadratic in the requested degree. Since $h = (p - 1)/m$, certification can become costly when p is large and the compression is small.

25.4 Odd and dyadic Euler coordinates

25.4.1 Source sequences and kernels

The signed Euler convention is

$$\operatorname{sech} t = \sum_{n\ge 0} E_n \frac{t^n}{n!},$$

while the nonnegative zigzag numbers satisfy

$$\sec t + \tan t = \sum_{n\ge 0} A_n \frac{t^n}{n!}.$$

The source functions are

```
bernoulli_numbers(degree: int) -> tuple[Fraction, ...]
signed_euler_numbers(degree: int) -> tuple[int, ...]
signed_euler_number(index: int) -> int
euler_zigzag_numbers(degree: int) -> tuple[int, ...]
euler_zigzag_number(index: int) -> int
odd_power_sum(modulus: int, degree: int) -> int
dyadic_source(index: int) -> Fraction
dyadic_beta(index: int) -> Fraction
dyadic_kernel(index: int) -> Fraction
odd_kernel(prime: int, index: int) -> Fraction
dyadic_kernel_valuation(index: int) -> int
dyadic_row_weight(index: int, transfer_index: int) -> int
```

For even n, the local reconstruction is

$$E_n = \sum_{r=0}^{n/2} \binom{n}{2r} \kappa_{p,r} A_{p,n-2r}.$$

The dyadic source and kernel are

$$A_{2,n} = \frac{3^{n+2}-1}{4(n+1)(n+2)},$$

$$\kappa_{2,0} = 1, \qquad \kappa_{2,r} = -2^{4r}(2r-1)B_{2r} \quad (r>0),$$

and the odd-prime kernel is

$$\kappa_{p,r} = (-1)^r A_{2r+1} p^{2r}.$$

For positive r,

$$v_2(\kappa_{2,r}) = 4r - 1.$$

`odd_power_sum(q, n)` accepts any positive odd integer q. The coordinate class itself requires a prime and routes $p = 2$ to the separate dyadic chart.

25.4.2 `EulerLocalCoordinates`

```
EulerLocalCoordinates(prime: int) -> None
```

Member	Signature or result
`is_dyadic`	Whether `prime == 2`
`source(index)`	$A_{p,n}$
`kernel(index)`	$\kappa_{p,r}$
`inverse_kernel(index)`	Inverse transfer coefficient
`reconstruct(index, *, depth=None)`	Full or truncated forward reconstruction
`inverse_reconstruct_source(index)`	Recover the local source from E_n
`kernel_valuation(index)`	$v_p(\kappa_{p,r})$
`row_weight(index, transfer_index)`	Valuation of one complete row term
`formula_row_weight(index, transfer_index)`	Separated valuation formula
`sharp_modulus(depth)`	Uniform omitted-tail modulus
`truncate(index, depth)`	`EulerTruncation` record
`certificate(maximum_index=20)`	Finite forward, inverse, and row-ledger check

```
EulerLocalCoordinates.is_dyadic: bool
EulerLocalCoordinates.source(self, index: int) -> Fraction
EulerLocalCoordinates.kernel(self, index: int) -> Fraction
EulerLocalCoordinates.inverse_kernel(self, index: int) -> Fraction
EulerLocalCoordinates.reconstruct(
    self, index: int, *, depth: int | None = None,
) -> Fraction
EulerLocalCoordinates.inverse_reconstruct_source(
    self, index: int,
) -> Fraction
EulerLocalCoordinates.kernel_valuation(
    self, index: int,
) -> int | float
EulerLocalCoordinates.row_weight(
    self, index: int, transfer_index: int,
) -> int | float
EulerLocalCoordinates.formula_row_weight(
    self, index: int, transfer_index: int,
) -> int | float
EulerLocalCoordinates.sharp_modulus(self, depth: int) -> int
EulerLocalCoordinates.truncate(
    self, index: int, depth: int,
) -> EulerTruncation
EulerLocalCoordinates.certificate(
    self, maximum_index: int = 20,
) -> ComputationResult[tuple[Fraction, ...]]
```

Reconstruction accepts even nonnegative indices only. At depth d, terms with $0 \le r \le d$ are retained, subject to $r \le n/2$. If the row is not complete, the proved moduli are

$$2^{4d+3} \quad \text{and} \quad p^{2d+2}$$

for the dyadic and odd branches respectively.

The result record is

```
EulerTruncation(
    prime: int,
```

```
    index: int,
    depth: int,
    value: Fraction,
    exact_value: int,
    modulus: int | None,
    error: Fraction,
    certificate: Certificate,
) -> None
```

`modulus is None` means the full finite row was retained and equality, rather than a proper congruence, was checked.

```
from komorebi_math.arithmetic import EulerLocalCoordinates, padic_valuation

for prime in (2, 5):
    coordinates = EulerLocalCoordinates(prime)
    result = coordinates.truncate(index=20, depth=2)
    result.certificate.require()
    print(
        prime,
        result.value,
        result.modulus,
        padic_valuation(result.error, prime),
    )
```

Output:

```
2 8804179016755/231 2048 14
5 86777696087933558237525 15625 7
```

The printed error valuations exceed the required bounds (v_2(2048)=11) and (v_5(15625)=6).

25.4.3 Certification and budgets

`EulerLocalCoordinates.certificate(N)` checks every even row through N, both reconstruction directions, and every retained row valuation. Its domain is finite and explicit in `diagnostics["maximum_index"]`. It does not prove a claim for unexecuted indices.

The source functions build Bernoulli, signed Euler, and zigzag prefixes from scratch. They do not install a cross-call cache. Repeated scalar calls such as `signed_euler_number(n)` are therefore less efficient than computing one prefix with `signed_euler_numbers(N)` and reusing it. Exact rational numerator and denominator sizes also grow rapidly. Use modest `maximum_index` values for interactive certificates and raise the bound explicitly in qualification runs.

25.5 Mahler transfer and arithmetic polyphase

25.5.1 Newton-Mahler coordinates

For values $f(0), \dots, f(d)$, the finite Newton expansion is

$$f(x) = \sum_{r=0}^{d} \Delta^r f(0) \binom{x}{r}.$$

The primitive interfaces are

```
generalized_binomial(value: Number, order: int) -> Number
difference_table(
    values: Iterable[Number],
) -> tuple[tuple[Number, ...], ...]
mahler_coefficients(values: Iterable[Number]) -> tuple[Number, ...]
```

`generalized_binomial(x, r)` returns zero for a negative order. Exact integers and `Fraction` values produce exact rational results. Other numeric types use their native arithmetic.

```
MahlerSeries(coefficients: Iterable[Number]) -> None
```

Member	Signature or result
`degree`	Highest stored coefficient index
`from_values(values)`	Consecutive-prefix analysis
`evaluate(value, *, degree=None)`	Finite Newton polynomial evaluation
`first(count)`	Values at $0, \ldots, count - 1$
`difference(order=1)`	Shift coefficients by the difference order
`affine_subsequence(multiplier, shift)`	Coordinates of $f(mx + s)$
`valuation_ledger(prime)`	Exact rational coefficient valuations
`reconstruction_certificate()`	Reanalyze the defining finite prefix

```
MahlerSeries.degree: int
MahlerSeries.from_values(values: Iterable[Number]) -> MahlerSeries
MahlerSeries.evaluate(
    self, value: Number, *, degree: int | None = None,
) -> Number
MahlerSeries.first(self, count: int) -> tuple[Number, ...]
MahlerSeries.difference(self, order: int = 1) -> MahlerSeries
MahlerSeries.affine_subsequence(
    self, multiplier: int, shift: int,
) -> MahlerSeries
MahlerSeries.valuation_ledger(
    self, prime: int,
) -> tuple[int | float, ...]
MahlerSeries.reconstruction_certificate(
    self,
) -> ComputationResult[tuple[Number, ...]]
```

`from_values` preserves a coefficient slot for every input value, including trailing zero differences. `affine_subsequence` removes trailing zero coefficients after transfer.

```
from komorebi_math.arithmetic import MahlerSeries

series = MahlerSeries.from_values([n**3 + 1 for n in range(8)])
branch = series.affine_subsequence(multiplier=4, shift=3)

series.reconstruction_certificate().certificate.require()
print(series.coefficients)
print(branch.first(5))
```

Output:

```
(1, 1, 6, 6, 0, 0, 0, 0)
(Fraction(28, 1), Fraction(344, 1), Fraction(1332, 1), Fraction(3376, 1), Fraction(6860, 1))
```

The reconstruction certificate labels integer and rational input `EXACT` and other numeric types `NUMERICAL`. In the numerical branch it still compares the reanalyzed coefficient tuple directly. It is not a tolerance-based stability or conditioning certificate.

25.5.2 Affine kernel and prime-power sparsity

The transfer kernel is defined by

$$\binom{mq+s}{r} = \sum_{k} K_{m,s}(r,k)\binom{q}{k},$$

$$K_{m,s}(r,k) = \sum_{j=0}^{k} (-1)^{k-j}\binom{k}{j}\binom{mj+s}{r}.$$

Its signatures are

```
affine_mahler_kernel(
    multiplier: int,
    shift: int,
    source_degree: int,
    target_degree: int,
) -> int

affine_kernel_matrix(
    multiplier: int, shift: int, degree: int,
) -> tuple[tuple[int, ...], ...]

affine_kernel_composition_certificate(
    first_multiplier: int,
    first_shift: int,
    second_multiplier: int,
    second_shift: int,
    degree: int,
) -> Certificate
```

Prime-power specializations are exposed separately:

```


prime_power_kernel_residue(
    prime: int,
    exponent: int,
    shift: int,
    source_degree: int,
    target_degree: int,
) -> int

prime_power_kernel_certificate(
    prime: int, exponent: int, shift: int, degree: int,
) -> Certificate
```

Rows of `affine_kernel_matrix` are target degrees and columns are source degrees. Composition obeys

$$K_{m,s}K_{n,t} = K_{mn,mt+s},$$

under that stored orientation. For a prime-power multiplier,

$$K_{p^a,s}(r,k) \equiv \binom{s}{r-p^a k} \pmod{p},$$

where the binomial is zero outside its valid range.

The affine transfer of a degree-d series uses a cubic number of elementary integer operations in the current direct implementation. The exhaustive composition certificate nests additional degree loops and should be treated as a qualification operation, not a hot-path transform for very large d.

25.5.3 Scalar and matrix unit modes

```
UnitModeMahler(base: int, jump: int, residue: int = 0) -> None
```

For nonzero integer u, positive h, and $j \geq 0$,

$$u^{hx+j} = u^j \sum_{r \geq 0} (u^h - 1)^r \binom{x}{r}.$$

Member	Meaning
`increment`	$u^h - 1$
`coefficients(degree)`	$u^j(u^h-1)^r$ through the requested degree
`evaluate(value, *, degree=None)`	Exact mode or finite truncation
`valuation_slope(prime)`	$v_p(u^h - 1)$
`certificate(maximum_value, degree=None)`	Finite-range coordinate check

```
UnitModeMahler.increment: int
UnitModeMahler.coefficients(self, degree: int) -> tuple[int, ...]
UnitModeMahler.evaluate(
    self, value: int, *, degree: int | None = None,
) -> int | Fraction
UnitModeMahler.valuation_slope(self, prime: int) -> int | float
UnitModeMahler.certificate(
    self, maximum_value: int, degree: int | None = None,
) -> Certificate
```

When `degree` is omitted or at least the evaluation value, `evaluate` returns the exact integer mode directly. Otherwise it evaluates the truncated Mahler series.

> **Certificate boundary.** If `degree < maximum_value`, `UnitModeMahler.certificate` checks only through `degree` and records an unchecked assumption for the rest. Its validity flag can still be true. Set `degree >= maximum_value` when full-range validation is required and inspect `assumptions_unchecked` before promoting the result.

Matrix modes replace $u^h - 1$ by $C^h - I$:

```
matrix_mahler_coefficients(
    matrix: np.ndarray,
    jump: int,
    residue: int,
    degree: int,
) -> tuple[np.ndarray, ...]

evaluate_matrix_mahler(
    coefficients: Iterable[np.ndarray],
    value: int,
) -> np.ndarray
```

The matrix constructor requires a square matrix. It uses NumPy object arrays and exact Python scalar operations where the entries are exact. The evaluator requires a common shape but does not independently certify that its supplied coefficient arrays came from one square matrix.

25.5.4 Arithmetic polyphase

For a finite sequence $a = (a_n)_{0 \le n < L}$,

$$a \longmapsto \left((a_{mq+r})_q \right)_{r=0}^{m-1}.$$

The complete signatures are

```
ArithmeticPolyphase(
    modulus: int,
    channels: tuple[tuple[Number, ...], ...],
    length: int,
) -> None

polyphase_analyze(
    values: Iterable[Number], modulus: int,
) -> ArithmeticPolyphase

polyphase_synthesize(
    channels: Iterable[Iterable[Number]],
    length: int | None = None,
) -> tuple[Number, ...]

mahler_polyphase(
    series: MahlerSeries, modulus: int, address: int,
) -> MahlerSeries

mixed_radix_address(
    digits: Iterable[int], radices: Iterable[int],
) -> int

mixed_radix_digits(
    address: int, radices: Iterable[int],
) -> tuple[int, ...]
```

`ArithmeticPolyphase` supplies `reconstruct()`, `nested(inner_modulus)`, `refine(fine_modulus)`, and `coarsen(coarse_modulus)`. Refinement requires the fine modulus to be divisible by the current modulus. Coarsening requires a divisor. The direct constructor does not run a `__post_init__` schema check, so normal code should use `polyphase_analyze` or a trusted portable decoder.

```
ArithmeticPolyphase.reconstruct(self) -> tuple[Number, ...]
ArithmeticPolyphase.nested(
    self, inner_modulus: int,
) -> tuple[ArithmeticPolyphase, ...]
ArithmeticPolyphase.refine(
    self, fine_modulus: int,
) -> ArithmeticPolyphase
ArithmeticPolyphase.coarsen(
    self, coarse_modulus: int,
) -> ArithmeticPolyphase
```

Nested residue digits are least-significant first:

$$j = d_0 + r_0 d_1 + r_0 r_1 d_2 + \cdots .$$

```
from komorebi_math.arithmetic import (
    mixed_radix_address,
    mixed_radix_digits,
    polyphase_analyze,
)

values = tuple(range(13))
coordinates = polyphase_analyze(values, 4)
address = mixed_radix_address((2, 1, 3), (4, 3, 5))

print(coordinates.channels)
print(coordinates.reconstruct() == values)
print(address, mixed_radix_digits(address, (4, 3, 5)))
```

Output:

```
((0, 4, 8, 12), (1, 5, 9), (2, 6, 10), (3, 7, 11))
True
42 (2, 1, 3)
```

Analysis and synthesis are linear in the number of stored values plus the number of channels. Refinement and coarsening likewise rearrange finite values without approximation.

25.5.5 Portable persistence

`ArithmeticPolyphase` is the only Chapter 25 model with a registered portable object codec in `1.0.0b4`. Its tag is `arithmetic.polyphase_coordinates`, version 1. Fractions and other supported portable scalars inside its channels are encoded by the common persistence layer.

```
from fractions import Fraction

from komorebi_math.arithmetic import polyphase_analyze
from komorebi_math.export.persistence import dumps_portable, loads_portable

coordinates = polyphase_analyze(
    (Fraction(n + 1, n + 3) for n in range(9)),
    3,
)
restored = loads_portable(dumps_portable(coordinates))

assert restored == coordinates
assert restored.reconstruct() == coordinates.reconstruct()
```

`HurwitzSeries`, `StandardWindow`, `EulerLocalCoordinates`, `MahlerSeries`, and the recurrence objects do not have registered portable object codecs in this release. Persist their primitive constructor fields under an application-owned schema, then reconstruct and rerun the appropriate certificate. Do not rely on Python `repr`, pickle, or an undocumented dataclass layout as a durable public format.

25.6 Recurrence packets

25.6.1 State convention and matrix primitives

`LinearRecurrence` uses the convention

$$x_{n+d} = \sum_{i=0}^{d-1} c_i x_{n+i}, \qquad X_n = (x_n, \dots, x_{n+d-1})^T, \qquad X_{n+1} = CX_n.$$

The matrix helpers are

```
matrix_power_exact(matrix: np.ndarray, exponent: int) -> np.ndarray
matrix_power_mod(
    matrix: np.ndarray, exponent: int, modulus: int,
) -> np.ndarray
determinant_integer(matrix: np.ndarray) -> int
```

Both power functions use binary exponentiation on square object arrays. `matrix_power_mod` requires integral entries and reduces after every multiply. `determinant_integer` performs exact fraction elimination and rejects a final nonintegral determinant.

25.6.2 `LinearRecurrence`

```
LinearRecurrence(
    coefficients: Iterable[Number],
    initial: Iterable[Number],
    *,
    name: str | None = None,
) -> None
```

The coefficient and initial tuples must be nonempty numeric data of the same length. Booleans are rejected.

Member	Signature or result
`order`	Recurrence order d
`companion_matrix`	Exact object-array transition matrix
`initial_state`	$(x_0, \dots, x_{d-1})$
`term(index)`	Cached sequential evaluation
`term_mod(index, modulus)`	Modular term by binary matrix powering
`first(count)`	Prefix tuple
`state(index)`	State X_n
`jump_matrix(modulus)`	Exact C^m, where the argument is the packet stride m
`residue_packet(modulus, address)`	Validated packet factory
`period_mod(modulus, *, max_steps=None)`	Exact finite-state orbit classification
`characteristic_polynomial()`	Ascending coefficients $(-c_0, \dots, -c_{d-1}, 1)$
`jump_characteristic_polynomial(modulus)`	Exact characteristic polynomial of C^m
`certificate(count=20)`	Recurrence and state-evolution check

```
LinearRecurrence.order: int
LinearRecurrence.companion_matrix: np.ndarray
LinearRecurrence.initial_state: np.ndarray
LinearRecurrence.term(self, index: int) -> Number
LinearRecurrence.term_mod(
    self, index: int, modulus: int,
) -> int
LinearRecurrence.first(self, count: int) -> tuple[Number, ...]
```

```
LinearRecurrence.state(self, index: int) -> np.ndarray
LinearRecurrence.jump_matrix(self, modulus: int) -> np.ndarray
LinearRecurrence.residue_packet(
    self, modulus: int, address: int,
) -> RecurrencePacket
LinearRecurrence.period_mod(
    self, modulus: int, *, max_steps: int | None = None,
) -> OrbitPeriod
LinearRecurrence.characteristic_polynomial(
    self,
) -> tuple[Number, ...]
LinearRecurrence.jump_characteristic_polynomial(
    self, modulus: int,
) -> tuple[Number, ...]
LinearRecurrence.certificate(
    self, count: int = 20,
) -> ComputationResult[tuple[Number, ...]]
```

The parameter name `modulus` has two meanings in this class. In `term_mod` and `period_mod` it is an arithmetic modulus. In `jump_matrix`, `residue_packet`, and `jump_characteristic_polynomial` it is the residue-packet stride m. The latter methods compute C^m over the original scalar domain, not modulo m.

`jump_characteristic_polynomial` requires optional SymPy. Every other member in the table works without SymPy.

`fibonacci_recurrence()` and `lucas_recurrence()` construct

$$F_{n+2} = F_{n+1} + F_n, \quad (F_0, F_1) = (0, 1),$$

$$L_{n+2} = L_{n+1} + L_n, \quad (L_0, L_1) = (2, 1).$$

The complete auxiliary signatures are

```
fibonacci_recurrence() -> LinearRecurrence
lucas_recurrence() -> LinearRecurrence
second_order_trace(
    first: Number, determinant: Number, index: int,
) -> Number
second_order_jump(
    first: Number, determinant: Number, modulus: int,
) -> tuple[Number, Number]
```

For a second-order matrix with trace T and determinant D,

$$V_0 = 2, \qquad V_1 = T, \qquad V_n = TV_{n-1} - DV_{n-2},$$

and the stride-m packet has trace V_m and determinant D^m.

25.6.3 `RecurrencePacket` and modular periods

A validated residue packet obeys

$$X_{mk+r} = (C^m)^k X_r.$$

Its record signature is

```
RecurrencePacket(
    recurrence: LinearRecurrence,
    modulus: int,
```

```
    address: int,
) -> None
```

and its members are `jump_matrix`, `initial_state`, `state(packet_index)`, `term(packet_index)`, `term_mod(packet_index, modulus)`, `first(count)`, and `certificate(count=10)`.

```
RecurrencePacket.jump_matrix: np.ndarray
RecurrencePacket.initial_state: np.ndarray
RecurrencePacket.state(self, packet_index: int) -> np.ndarray
RecurrencePacket.term(self, packet_index: int) -> Number
RecurrencePacket.term_mod(
    self, packet_index: int, modulus: int,
) -> int
RecurrencePacket.first(self, count: int) -> tuple[Number, ...]
RecurrencePacket.certificate(
    self, count: int = 10,
) -> ComputationResult[tuple[Number, ...]]
```

Use `recurrence.residue_packet(m, r)` in normal code. The bare dataclass constructor does not validate that $m > 0$ and $0 \le r < m$. The factory does.

```
from komorebi_math.arithmetic import fibonacci_recurrence

fibonacci = fibonacci_recurrence()
packet = fibonacci.residue_packet(modulus=7, address=3)
packet.certificate(count=6).certificate.require()

print(packet.first(6))
print(fibonacci.term_mod(1_000_000, 1_000_003))
print(fibonacci.period_mod(32).period)
```

Output:

```
(2, 55, 1597, 46368, 1346269, 39088169)
3
48
```

`OrbitPeriod` stores `modulus`, `preperiod`, `period`, `states_checked`, `purely_periodic`, and an exact `certificate`. If the companion determinant is a unit modulo M, the transition is invertible and a valid orbit must have preperiod zero.

```
OrbitPeriod(
    modulus: int,
    preperiod: int,
    period: int,
    states_checked: int,
    purely_periodic: bool,
    certificate: Certificate,
) -> None
```

The default period-search budget is $M^d + 1$ states. Time and memory can therefore grow exponentially in recurrence order d. Set `max_steps` explicitly for untrusted or exploratory moduli. Exhausting that budget raises `ValidationError` rather than returning a guessed period.

Operation	Current direct cost, excluding scalar bit complexity
Cached `term(n)` after a cold start	$O(nd)$
`term_mod(n, M)`	$O(d^3 \log n)$ dense matrix work
Exact `jump_matrix(m)`	$O(d^3 \log m)$
Packet state at k	$O(d^3(\log m + \log k))$ because the jump matrix is rebuilt
`period_mod(M)`	Up to $O(M^d d^2)$ work and $O(M^d d)$ stored state

Operation	Current direct cost, excluding scalar bit complexity

Recurrence and packet certificates label rational data `EXACT` and other numeric data `NUMERICAL`. Numerical validation uses direct equality of computed states. It does not supply tolerances, forward-error bounds, or conditioning analysis.

25.7 Carmichael jumps, local modes, and simultaneous congruences

25.7.1 Scalar finite differences

The exact scalar difference is

$$\Delta_h^r f(n) = \sum_{k=0}^{r} (-1)^{r-k} \binom{r}{k} f(n + kh).$$

The interfaces are

```
finite_difference_value(
    function: Callable[[int], int],
    index: int,
    step: int,
    order: int,
) -> int

certify_difference_congruence(
    function: Callable[[int], int],
    index: int,
    step: int,
    order: int,
    modulus: int,
) -> DifferenceCongruence
```

Every sampled function value must be an integer and cannot be boolean. The record

```
DifferenceCongruence(
    value: int,
    modulus: int,
    certificate: Certificate,
) -> None
```

reports whether the computed difference is divisible by the requested positive modulus. It proves only that finite calculation. It does not infer a uniform theorem over other indices or orders.

25.7.2 Precision-selected unit modes

```
local_unit_jump(prime: int, depth: int) -> int

unit_mode_for_precision(
    base: int,
    prime: int,
    depth: int,
    *,
    residue: int = 0,
) -> UnitModeMahler
```

`local_unit_jump` is the prime-power Carmichael exponent from Section 25.1. `unit_mode_for_precision` additionally verifies that the base is a unit at the prime, then returns the Mahler mode with that jump.

For $h = \lambda(p^d)$,

$$u^h \equiv 1 \pmod{p^d}, \qquad v_p(\Delta_h^r u^n) \geq dr.$$

```
from komorebi_math.arithmetic import (
    certify_difference_congruence,
    local_unit_jump,
    unit_mode_for_precision,
)

mode = unit_mode_for_precision(7, prime=5, depth=2, residue=3)
record = certify_difference_congruence(
    lambda n: 7**n,
    index=2,
    step=mode.jump,
    order=3,
    modulus=5**6,
)
record.certificate.require()

print(local_unit_jump(5, 2), mode.valuation_slope(5))
print(record.value % record.modulus, record.certificate.valid)
```

Output:

```
20 3
0 True
```

The achieved valuation slope can exceed the universal lower bound. Here $v_5(7^{20}-1) = 3$, so an order-three difference gains valuation nine while the requested certificate uses only 5^6.

25.7.3 One jump for several primes

```
simultaneous_unit_difference(
    base: int,
    primes_and_depths: dict[int, int],
    *,
    index: int,
    order: int,
) -> ComputationResult[dict[int, int]]
```

For local requirements ((p_i,d_i)), the function chooses

$$h = \operatorname{lcm}_i \lambda(p_i^{d_i})$$

and evaluates the exact integer

$$u^n(u^h - 1)^r.$$

The result value maps each prime to its achieved valuation. The certificate is valid when

$$v_{p_i}(\Delta_h^r u^n) \geq d_i r$$

at every requested prime.

```
from komorebi_math.arithmetic import simultaneous_unit_difference
```

```
result = simultaneous_unit_difference(
    7,
    {2: 4, 3: 3, 5: 2},
    index=2,
    order=3,
)
result.certificate.require()

print(result.certificate.diagnostics["common_jump"])
print(result.value)
```

Output:

```
180
{2: 15, 3: 9, 5: 9}
```

Every requested prime is validated and the base must be a unit at all of them. A base divisible by even one requested prime raises `ValidationError`.

> **Recorded baseline defect.** In the audited `1.0.0b4` source, `simultaneous_unit_difference` converted each reported valuation to `int`. For $u = 1$, and also for $u = -1$ when the common jump is even, the difference is exactly zero for positive order and its valuation is $+\infty$. That baseline conversion raised `OverflowError: cannot convert float infinity to integer`. When using that baseline, handle these zero-difference modes separately or use `finite_difference_value` with `padic_valuation`, preserving `float("inf")`.

The simultaneous routine constructs the full integer $u^n(u^h - 1)^r$ and stores it in certificate diagnostics. It is exact but can consume substantial time, memory, and report space. Budget the common jump and order before using many primes or deep local precisions. It is not implemented as a modular-only valuation algorithm.

25.7.4 Failure matrix

Request	Result
Composite valuation base or field characteristic	Prime validation failure
Nonunit modular inverse	`ValidationError`
Negative factorial index or invalid valuation binomial indices	`ValidationError`
Modular Hurwitz float coefficient	`ValidationError`
Hurwitz inverse with nonunit constant	`ValidationError`
Window with $p = 2$ or $m \nmid p - 1$, followed by a period-dependent call	`ValidationError`
Odd Euler reconstruction index	`ValidationError`
Mahler valuation ledger with float or complex coefficients	`ValidationError`
Modular recurrence with nonintegral data	`ValidationError`
Recurrence orbit exceeds `max_steps`	`ValidationError`
Jump characteristic polynomial without SymPy	`BackendUnavailableError`
Local unit mode with a base divisible by the prime	`ValidationError`
Simultaneous zero-difference mode $u = \pm 1$, positive order	Recorded baseline `OverflowError` defect

25.7.5 Executed evidence

The discussion was audited against the shipped `1.0.0b4` source, not inferred from the symbolic outline alone. The focused replay covered

```
tests/test_arithmetic_domains_valuation.py
tests/test_arithmetic_hurwitz_windows.py
tests/test_arithmetic_euler.py
tests/test_arithmetic_mahler_polyphase.py
tests/test_arithmetic_recurrence.py
```

and produced

```
31 passed
```

The installed arithmetic verifier produced

```
verification passed 7/7 checks
```

Those seven checks cover Hurwitz multiplication and inversion, the standard window period, odd and dyadic Euler coordinates, Mahler transfer, recurrence packets and modular periods, the compressed inverse recurrence, and simultaneous local valuation gains. The portable `ArithmeticPolyphase` roundtrip is additionally exercised by the recorded polyphase persistence suite.

Chapter 25 summary. Use finite rings to make reduction explicit, valuation functions to measure local divisibility, Hurwitz series for exact EGF convolution, and standard windows only under their odd-prime subgroup assumptions. Euler charts provide distinct odd and dyadic kernels with finite congruence certificates. Mahler coordinates convert values and affine residue branches into exact binomial-basis coefficients. Arithmetic polyphase is the stable, portable residue layout. Recurrence packets replace a fine state step by an exact jump matrix, while modular execution avoids constructing huge recurrence integers. Carmichael jumps certify finite local gains. In the recorded baseline, the simultaneous routine constructs a potentially enormous exact difference and has the zero-difference reporting defect documented above. Every certificate is bounded by its executed indices, degree, precision, or orbit budget.

26. Exact sequence execution

First route. Select a catalog sequence or wrap a custom evaluator, then inspect a small finite prefix. If compiling, require the returned certificate before using the selected model. For a very large index, decide whether you need the full exact term or only its residue, then consult Section 26.6 for the available execution route. A lazy view still needs an explicit stopping point when you turn it into a list or report.

The sequence subsystem turns a rule $a : \mathbb{N} \to \mathcal{R}$ into a finite, inspectable execution object. It supports four distinct ideas that should not be conflated:

1. a lazy evaluator that computes and caches requested terms
2. a declared finite model, such as a recurrence or automaton
3. a compiler that selects a declared model or fits a constant recurrence to a finite prefix
4. a certificate that records exactly what was checked

The implementation is finite even when the mathematical sequence is infinite. Every certificate covers a declared model or a finite prefix. A prefix-fitted recurrence is an extrapolation model, not an automatically proved identity.

The exact workflows in this chapter use integers and `Fraction` values. The `Sequence` constructor itself accepts any Python numeric value, including floating and complex values. Its `exact` field is user-supplied metadata. It does not inspect the evaluator or prove exact scalar arithmetic.

26.1 Sequence protocol and catalog

26.1.1 API tier and import boundary

Sequence execution is part of the **research API** in `1.0.0b4`. No sequence symbol belongs to the frozen `komorebi_math.stable` or `komorebi_math.beta` manifests.

Surface	Role	Sequence coverage
`komorebi_math.sequences`	authoritative sequence namespace	all 36 exported names in this chapter
`komorebi_math`	convenient research root	the principal classes, compiler, and selected catalog constructors
`komorebi_math.beta`	frozen beta surface	none
`komorebi_math.stable`	frozen stable surface	none

The namespace export ledger is complete as follows.

Group	Exported names
feature	`FEATURE_KEY`, `status`, `require`
records and alias	`Sequence`, `AutomaticSequence`, `HolonomicRecurrence`, `CompiledSequence`, `Polynomial`
catalog	`fibonacci_sequence`, `lucas_sequence`, `signed_euler_sequence`, `euler_zigzag_sequence`, `bernoulli_sequence`, `catalan_sequence`, `bell_sequence`, `motzkin_sequence`, `partition_sequence`, `apery_sequence`, `thue_morse_sequence`, `sequence_catalog`, `get_sequence`
automatic	`base_digits`, `thue_morse_automaton`
holonomic	`evaluate_polynomial`, `motzkin_recurrence`, `apery_recurrence`
compiler	`detect_linear_recurrence`, `recurrence_generating_function`, `compile_sequence`

Group	Exported names
transforms	`binomial_transform_values`, `inverse_binomial_transform_values`, `binomial_transform`, `inverse_binomial_transform`, `finite_difference`, `partial_sums`, `cauchy_product`

`FEATURE_KEY` is the literal string `"sequences"`.

The research root exports these sequence objects directly:

```
from komorebi_math import (
    AutomaticSequence,
    CompiledSequence,
    HolonomicRecurrence,
    Sequence,
    apery_sequence,
    compile_sequence,
    fibonacci_sequence,
    get_sequence,
    motzkin_sequence,
    partition_sequence,
    thue_morse_sequence,
)
```

Use the namespace import for the complete facility:

```
from komorebi_math.sequences import (
    Sequence,
    binomial_transform,
    finite_difference,
    lucas_sequence,
    partial_sums,
)
```

> **Name-collision warning.** `komorebi_math.binomial_transform` is the dense coordinate-transform constructor from `komorebi_math.coordinates`, not the lazy sequence transform. `komorebi_math.finite_difference` is the polynomial expression primitive, not the sequence finite difference. Import both sequence functions from `komorebi_math.sequences`.

The feature probe is descriptive:

```
from komorebi_math import sequences

print(sequences.status().status.value)
sequences.require()
```

`status()` returns the registry record for the sequence feature. `require()` returns `None`. Neither function validates a particular sequence or executes a certificate.

26.1.2 `Sequence`

The concrete lazy wrapper has the signature

```
Sequence(
    evaluator: Callable[[int], Number],
    *,
    name: str = "sequence",
    exact: bool = True,
```

```
    metadata: dict[str, Any] | None = None,
) -> None
```

It models

$$a : \mathbb{N} \longrightarrow \mathcal{R}$$

with cached numeric evaluation carrying an explicit exactness flag. The evaluator is called only when an index is absent from the instance cache. Different `Sequence` instances have independent caches, even when they wrap the same function.

Member	Return	Contract
`name`	`str`	display label produced with `str(name)`
`exact`	`bool`	constructor metadata, not a proof
`metadata`	`dict[str, Any]`	shallow copy of the supplied mapping
`term(index)`	`Number`	evaluate and cache one nonnegative integer index
`first(count)`	`tuple[Number, ...]`	terms $a_0, \ldots, a_{N-1}$
`residue(modulus, address)`	`Sequence`	lazy subsequence a_{mk+r}
`modulo(modulus)`	`Sequence`	lazy integral residue $a_k \bmod M$
`map(function, *, name=None, exact=None)`	`Sequence`	lazy scalar map with explicit exactness override
`sequence[index]`	`Number`	alias for `term(index)`
`sequence[start:stop:step]`	`tuple`	finite nonnegative slice with positive step
`iter(sequence)`	iterator	unbounded lazy iteration from index zero

The two primary views are

$$\operatorname{residue}_{m,r}(a)(k) = a_{mk+r}, \qquad \operatorname{modulo}_{M}(a)(k) = a_k \bmod M.$$

`residue(m, r)` requires $m \geq 1$ and $0 \leq r < m$. `modulo(M)` requires $M \geq 2$. Both return new caching wrappers. The residue view delegates to and therefore fills the parent cache. The modular view first obtains a parent term, then requires it to be a rational scalar with denominator one.

`map` propagates the parent's exactness flag unless `exact=` is supplied. Because this is metadata propagation, the declaration below is permitted but mathematically dishonest:

```
Sequence(lambda n: 0.1 * n, exact=True)
```

Callers are responsible for choosing `exact=False` when the evaluator returns floating approximations.

26.1.3 Lazy views and finite slices

```
from komorebi_math.sequences import Sequence

squares = Sequence(lambda n: n * n, name="squares")

print(squares.first(8))
print(squares.residue(3, 1).first(6))
print(squares.modulo(5).first(8))
print(squares[2:10:2])
print(squares.map(lambda x: x + 1, name="square plus one").first(6))
```

Output:

```
(0, 1, 4, 9, 16, 25, 36, 49)
(1, 16, 49, 100, 169, 256)
(0, 1, 4, 4, 1, 0, 1, 4)
(4, 16, 36, 64)
(1, 2, 5, 10, 17, 26)
```

A slice must have a finite stop. Bounds must be nonnegative and the step must be positive. Integer indices, counts, moduli, addresses, and slice components reject booleans and floating coercions. Infinite iteration is intentionally unbounded, so consumers should apply their own limit:

```
from itertools import islice

first_twenty = tuple(islice(squares, 20))
```

26.1.4 Cache and concurrency boundary

The common wrapper has no eviction policy or built-in term budget. If U distinct indices are requested, its cache retains U numeric objects for the lifetime of the wrapper. Exact integers and rationals can grow much larger than the number of cached entries suggests.

`Sequence`, `HolonomicRecurrence`, and `LinearRecurrence` use mutable caches without a synchronization contract. Treat an instance as one execution context. Concurrent applications should use external synchronization or separate instances.

26.2 Fibonacci, Lucas, Catalan, Bell, and built-in families

26.2.1 Constructor catalog

Every catalog constructor takes no arguments and returns a fresh `Sequence`.

```
fibonacci_sequence() -> Sequence
lucas_sequence() -> Sequence
signed_euler_sequence() -> Sequence
euler_zigzag_sequence() -> Sequence
bernoulli_sequence() -> Sequence
catalan_sequence() -> Sequence
bell_sequence() -> Sequence
motzkin_sequence() -> Sequence
partition_sequence() -> Sequence
apery_sequence() -> Sequence
thue_morse_sequence() -> Sequence
```

The defining conventions are:

$$F_{n+2} = F_{n+1} + F_n, \qquad (F_0, F_1) = (0, 1),$$

$$L_{n+2} = L_{n+1} + L_n, \qquad (L_0, L_1) = (2, 1),$$

$$C_n = \frac{1}{n+1}\binom{2n}{n}, \qquad B_{n+1} = \sum_{k=0}^{n} \binom{n}{k} B_k,$$

$$n\,p(n) = \sum_{k=1}^{n} \sigma(k) p(n-k), \qquad p(0) = 1,$$

and

$$t_{2n} = t_n, \qquad t_{2n+1} = 1 - t_n, \qquad t_0 = 0.$$

The signed Euler sequence uses the coefficients of `sech`, the zigzag sequence uses the coefficients of `sec + tan`, and the Bernoulli constructor uses the convention $B_1 = -\frac{1}{2}$:

$$\operatorname{sech}(z) = \sum_{n\geq 0} E_n \frac{z^n}{n!}, \qquad \sec(z) + \tan(z) = \sum_{n\geq 0} A_n \frac{z^n}{n!},$$

$$\sum_{k=0}^{n} \binom{n+1}{k} B_k = 0 \quad (n \geq 1).$$

Motzkin and Apéry numbers are executed through the holonomic models in Section 26.4. Fibonacci and Lucas numbers carry declared constant-recurrence metadata. Thue-Morse carries a declared automatic model. The remaining catalog objects use their direct evaluator.

26.2.2 Exact prefixes

Constructor	First eight terms
`fibonacci_sequence`	`0, 1, 1, 2, 3, 5, 8, 13`
`lucas_sequence`	`2, 1, 3, 4, 7, 11, 18, 29`
`signed_euler_sequence`	`1, 0, -1, 0, 5, 0, -61, 0`
`euler_zigzag_sequence`	`1, 1, 1, 2, 5, 16, 61, 272`
`bernoulli_sequence`	`1, -1/2, 1/6, 0, -1/30, 0, 1/42, 0`
`catalan_sequence`	`1, 1, 2, 5, 14, 42, 132, 429`
`bell_sequence`	`1, 1, 2, 5, 15, 52, 203, 877`
`motzkin_sequence`	`1, 1, 2, 4, 9, 21, 51, 127`
`partition_sequence`	`1, 1, 2, 3, 5, 7, 11, 15`
`apery_sequence`	`1, 5, 73, 1445, 33001, 819005, 21460825, 584307365`
`thue_morse_sequence`	`0, 1, 1, 0, 1, 0, 0, 1`

Motzkin and Apéry terms are represented internally as `Fraction` values, even when their denominators are one. The table prints their mathematical integer values for readability.

26.2.3 Named lookup and aliases

```
sequence_catalog() -> dict[str, Sequence]
get_sequence(name: str) -> Sequence
```

`sequence_catalog()` constructs and returns a new dictionary on every call. Its keys, in insertion order, are

```
apery, bell, bernoulli, catalan, euler, fibonacci, lucas,
motzkin, partition, thue-morse, zigzag
```

`get_sequence` strips the supplied string, lowercases it, and replaces underscores by hyphens. It then applies the aliases below.

Accepted alias	Catalog key
`apery-zeta3`	`apery`
`euler-zigzag`	`zigzag`
`signed-euler`	`euler`
`thue-morse`	`thue-morse`
`thuemorse`	`thue-morse`

Accepted alias	Catalog key

An unknown name raises `KeyError` with the original spelling in the message. Aliases are lookup conveniences only. They do not add keys to `sequence_catalog()`.

```
from komorebi_math.sequences import get_sequence

for name in ("FIBONACCI", "apery_zeta3", "thuemorse"):
    sequence = get_sequence(name)
    print(name, sequence.name, sequence.first(6))
```

Output:

```
FIBONACCI Fibonacci (0, 1, 1, 2, 3, 5)
apery_zeta3 Apery (1, 5, 73, 1445, 33001, 819005)
thuemorse Thue Morse (0, 1, 1, 0, 1, 0)
```

The actual Apéry tuple contains denominator-one `Fraction` objects.

26.2.4 Catalog execution costs

The table below counts high-level arithmetic operations and omits the growing bit cost of exact integers and rationals.

Family	Current execution path	Prefix cost through $N-1$	Stored state
Fibonacci, Lucas	order-two sequential recurrence	$O(N)$	$O(N)$ cached terms
Motzkin, Apéry	order-two holonomic recurrence	$O(N)$	$O(N)$ rational terms
Thue-Morse	two-state DFAO	$O(N \log N)$ through the wrapper	$O(N)$ wrapper cache
Bell	incremental triangular recurrence	$O(N^2)$	$O(N)$
partitions	divisor sums plus convolution	$O(N^2)$	$O(N)$ values and divisor sums
signed Euler, zigzag, Bernoulli	each newly requested term rebuilds a helper prefix	up to $O(N^3)$	$O(N)$ wrapper cache
Catalan	one exact binomial coefficient per term	governed by integer binomial cost	$O(N)$ wrapper cache

The $O(N^3)$ entry describes the current wrapper behavior, not an inherent complexity of those sequences. It arises because each scalar evaluator asks an arithmetic helper to rebuild all coefficients through that index. Applications requiring very long prefixes should call the corresponding bulk arithmetic helper or implement a bounded incremental cache.

No catalog constructor accepts a term, byte, time, or digit budget. Bound the largest index before evaluation. Prefer modular recurrence execution when only a residue of a very large Fibonacci or Lucas index is required.

26.3 Automatic sequences

An automatic sequence is represented by a deterministic finite automaton with numeric output, abbreviated DFAO. For base $b \geq 2$, state set Q, transition map δ, initial state q_0, and output map τ,

$$a_n = \tau\Big(\delta^{*}\big(q_0, \operatorname{digits}_b(n)\big)\Big)\,.$$

Digits are read from most significant to least significant in this namespace. The canonical expansion of zero is the one-digit tuple `(0,)`.

This order differs from the carry and Cartier machines in Chapter 23. Before transferring a transition table or a digit-processing loop between subsystems, check the producer's convention on an index with unequal digits. The binary digits of two, for example, are read as one then zero here and as zero then one by a least-significant-first machine.

26.3.1 Functions and constructor

```
base_digits(index: int, base: int) -> tuple[int, ...]

AutomaticSequence(
    base: int,
    transitions: Iterable[Iterable[int]],
    outputs: Iterable[Number],
    *,
    initial_state: int = 0,
    name: str = "automatic sequence",
) -> None

thue_morse_automaton() -> AutomaticSequence
```

The transition table has one row per state and one column per base digit. The entry in row q, column d is $\delta(q, d)$.

Constructor condition	Failure
base is not an integer or is below two	`ValidationError`
no state rows	`ValidationError`
row length differs from the base	`DimensionError`
output count differs from state count	`DimensionError`
output is Boolean or nonnumeric	`ValidationError`
transition target lies outside the state range	`ValidationError`
initial state lies outside the state range	`ValidationError`
initial state's zero transition is not itself	`ValidationError`

The final condition enforces canonical leading-zero invariance:

$$\delta(q_0, 0) = q_0.$$

It does not require a zero loop at every other state.

26.3.2 Member reference

Field or member	Return	Contract
`base`	`int`	digit radix b
`transitions`	tuple of tuples	immutable state transition table
`outputs`	tuple	one numeric output per state
`initial_state`	`int`	initial row index
`name`	`str`	display label
`state_count`	`int`	number of table rows
`state(index)`	`int`	terminal state after the canonical base digits
`term(index)`	`Number`	output at `state(index)`

Field or member	Return	Contract
`first(count)`	`tuple`	finite prefix
`residue_model(power, address)`	`AutomaticSequence`	model for $a_{b^e n+r}$
`to_sequence()`	`Sequence`	common wrapper with certified automatic metadata
`certificate(count=256)`	`ComputationResult`	finite internal table/evaluation check

For $e \geq 0$ and $0 \leq r < b^e$, the residue model satisfies

$$\text{residue_model}(e, r)(n) = a_{b^e n+r}.$$

The fixed e low digits of the address are applied after the input digits. For `power=0`, the modulus is one and the returned model is equivalent to the original.

26.3.3 Thue-Morse transition card

The built-in binary parity automaton is

State	digit `0`	digit `1`	output
0	0	1	0
1	1	0	1

Equivalently, the labeled transition graph is

$$0 \xrightarrow{0} 0, \qquad 0 \xrightarrow{1} 1, \qquad 1 \xrightarrow{0} 1, \qquad 1 \xrightarrow{1} 0.$$

State zero represents even binary digit parity. State one represents odd parity. The transition table immediately gives

$$t_{2n} = t_n, \qquad t_{2n+1} = 1 - t_n.$$

26.3.4 Automatic execution example

```
from komorebi_math.sequences import base_digits, thue_morse_automaton

model = thue_morse_automaton()
record = model.certificate(1000)
record.certificate.require()

packets = {
    (power, address): model.residue_model(power, address)
    for power in range(5)
    for address in range(2**power)
}

for (power, address), packet in packets.items():
    modulus = 2**power
    assert packet.first(64) == tuple(
        model.term(modulus * n + address) for n in range(64)
    )

print(base_digits(13, 2))
print(model.state_count)
print(model.first(16))
```

```
print(model.residue_model(4, 7).first(20))
print(record.certificate.diagnostics)
print(len(packets))
```

Output:

```
(1, 1, 0, 1)
2
(0, 1, 1, 0, 1, 0, 0, 1, 1, 0, 0, 1, 0, 1, 1, 0)
(1, 0, 0, 1, 0, 1, 1, 0, 0, 1, 1, 0, 1, 0, 0, 1, 0, 1, 1, 0)
{'base': 2, 'states': 2, 'count': 1000}
31
```

There are (1+2+4+8+16=31) residue models below (2^5).

26.3.5 Certificate and cost boundary

The automatic certificate has theorem identifier `PSC-SEQ-AUTOMATIC` and exactness `Exactness.EXACT`. It records the base, state count, and checked prefix length. Constructor validation proves that the finite tables are well-formed. The certificate then checks finite evaluation consistency using those same tables. It does **not** prove that a separately defined infinite sequence is represented by this automaton.

> **Automatic-output exactness caveat.** The constructor accepts floating and complex numeric outputs, while `certificate()` still labels deterministic table execution `Exactness.EXACT` and `to_sequence()` sets `exact=True`. That label establishes exact lookup of the stored output object. It does not turn an approximate floating output into an exact mathematical scalar. Use integers or `Fraction` outputs for exact sequence claims.

For $n > 0$, `term(n)` performs

$$\lfloor \log_b n \rfloor + 1$$

table transitions and uses constant additional memory. Constructing one residue model applies e fixed low digits to each state, then reconstructs and validates the full transition table. Its cost is $O(|Q|(e+b))$, and its stored table and outputs occupy $O(|Q|b)$ entries. Converting it to `Sequence` adds an unbounded term cache.

26.4 Holonomic recurrences

A holonomic sequence satisfies a polynomial-coefficient recurrence

$$\sum_{j=0}^{r} P_j(n) a_{n+j} = 0.$$

Komorebi Math stores every polynomial in ascending powers and every coefficient as `Fraction`. To compute the next value it divides by $P_r(n)$, which must be nonzero at each executed index.

26.4.1 Types, functions, and constructor

```
Polynomial = tuple[Fraction, ...]

evaluate_polynomial(
    coefficients: Iterable[Rational],
    value: int,
) -> Fraction

HolonomicRecurrence(
    polynomials: Iterable[Iterable[Rational]],
    initial: Iterable[Rational],
```

```
    *,
    name: str = "holonomic recurrence",
) -> None

motzkin_recurrence() -> HolonomicRecurrence
apery_recurrence() -> HolonomicRecurrence
```

For

$$P(n) = c_0 + c_1 n + \cdots + c_d n^d,$$

the stored tuple is `(c0, c1, ..., cd)`. Trailing zeros are removed. An empty coefficient iterable is normalized to the zero polynomial `(0,)` rather than rejected. The final recurrence polynomial may not be identically zero.

26.4.2 Member reference

Field or member	Return	Contract
`polynomials`	tuple of `Polynomial`	$P_0, \ldots, P_r$ in ascending powers
`initial`	tuple of `Fraction`	exactly r seed values
`name`	`str`	display label
`order`	`int`	number of initial values and final shift
`coefficient(shift, index)`	`Fraction`	$P_{\text{shift}}(n)$
`term(index)`	`Fraction`	cached exact execution through the requested index
`first(count)`	tuple of `Fraction`	finite exact prefix
`to_sequence()`	`Sequence`	wrapper with certified holonomic metadata
`certificate(count=32)`	`ComputationResult`	exact recurrence residual check on a finite prefix

The update is

$$a_{n+r} = -\frac{1}{P_r(n)} \sum_{j=0}^{r-1} P_j(n) a_{n+j}.$$

26.4.3 Motzkin recurrence from the source tuples

The built-in Motzkin constructor uses

```
P0 = (-3, -3)
P1 = (-5, -2)
P2 = (4, 1)
initial = (1, 1)
```

Therefore its declared equation is

$$(-3 - 3n)M_n + (-5 - 2n)M_{n+1} + (n + 4)M_{n+2} = 0,$$

or equivalently

$$(n + 4)M_{n+2} = (2n + 5)M_{n+1} + 3(n + 1)M_n.$$

Using $M_0 = M_1 = 1$,

$$M_2 = 2, \quad M_3 = 4, \quad M_4 = 9, \quad M_5 = 21.$$

This equation is derived directly from the shipped ascending tuples. It avoids the indexing error that can arise when copying a recurrence stated with M_{n-1} and M_{n-2}.

26.4.4 Apéry recurrence

The Apéry model uses

$$(n+1)^3 A_n - \left(34(n+1)^3 + 51(n+1)^2 + 27(n+1) + 5\right) A_{n+1} + (n+2)^3 A_{n+2} = 0,$$

$$(A_0, A_1) = (1, 5).$$

Its ascending source tuples are

```
P0 = (1, 3, 3, 1)
P1 = (-117, -231, -153, -34)
P2 = (8, 12, 6, 1)
initial = (1, 5)
```

26.4.5 Direct holonomic example

```
from komorebi_math.sequences import apery_recurrence, motzkin_recurrence

motzkin = motzkin_recurrence()
apery = apery_recurrence()

motzkin.certificate(30).certificate.require()
apery.certificate(20).certificate.require()

print(motzkin.first(11))
print(apery.first(8))
print(motzkin.order, apery.order)
```

Output, with denominator-one `Fraction` wrappers suppressed:

```
(1, 1, 2, 4, 9, 21, 51, 127, 323, 835, 2188)
(1, 5, 73, 1445, 33001, 819005, 21460825, 584307365)
2 2
```

26.4.6 Validation and singular-leading-coefficient boundary

Construction rejects fewer than two shift polynomials, a seed count different from the order, Boolean or nonrational coefficients and seeds, and an identically zero final polynomial. A final polynomial that vanishes only at a particular nonnegative index is accepted at construction. Execution raises `ValidationError` when that index is reached.

```
from komorebi_math.sequences import HolonomicRecurrence

# a[n] + n*a[n+1] = 0 has a leading coefficient that vanishes at n = 0
singular = HolonomicRecurrence(((1,), (0, 1)), (1,))
singular.term(1)  # raises ValidationError at executed index 0
```

`certificate(count)` requires `count > order`. It computes the prefix with the same recurrence and then checks every declared residual exactly. It can raise before returning a record if the leading coefficient vanishes. A valid record proves that the computed finite prefix satisfies the declared recurrence. It does not independently identify a user-supplied external sequence.

An order-r prefix through $N-1$ takes $O(rN)$ polynomial evaluations and recurrence scalar operations after seeds, plus polynomial-evaluation cost. The implementation caches all computed terms, so storage is $O(N)$. Exact rational numerator and denominator growth can dominate both counts.

26.5 Exact compiler and strongest-model selection

The compiler does not synthesize every possible description of a sequence. It checks a fixed metadata order, then attempts one kind of finite inference. Its selection relation is

$$\begin{aligned}\text{declared constant recurrence} \succ \text{declared DFAO} \succ \text{declared holonomic model}\\ \succ \text{prefix-fitted recurrence} \succ \text{direct evaluator}.\end{aligned}$$

Here "strongest" means first in this implementation order. It does not mean that the compiler proves a globally minimal or mathematically canonical model.

26.5.1 Function signatures

```
detect_linear_recurrence(
    values: Iterable[Rational],
    *,
    max_order: int | None = None,
) -> LinearRecurrence | None

recurrence_generating_function(
    recurrence: LinearRecurrence,
) -> tuple[tuple[Fraction, ...], tuple[Fraction, ...]]

compile_sequence(
    sequence: Sequence,
    *,
    sample_count: int = 24,
    max_order: int | None = 8,
) -> CompiledSequence
```

`detect_linear_recurrence` accepts only non-Boolean integers and rationals. It requires at least three sample values. With no `max_order`, it searches through half the prefix length. With an explicit limit, it searches through the smaller of that limit and half the prefix length. Orders are tried from one upward, so the first returned recurrence has the smallest supported order under the finite linear-system procedure.

The recurrence convention is

$$a_{n+d} = \sum_{j=0}^{d-1} c_j a_{n+j}.$$

All systems are solved over `Fraction`. When the sample system is underdetermined, the implementation assigns zero to free variables and checks the resulting relation against the entire supplied prefix.

26.5.2 `CompiledSequence` record

The record signature is

```
CompiledSequence(
    sequence: Sequence,
    strategy: str,
    certificate: Certificate,
    recurrence: LinearRecurrence | None = None,
    automatic: AutomaticSequence | None = None,
```

```
    holonomic: HolonomicRecurrence | None = None,
) -> None
```

Applications normally obtain it from `compile_sequence` rather than invoke the dataclass constructor directly.

Field or member	Return	Contract
`sequence`	`Sequence`	original lazy evaluator
`strategy`	`str`	exact selected-strategy label
`certificate`	`Certificate`	finite agreement and provenance record
`recurrence`	`LinearRecurrence | None`	declared or detected constant model
`automatic`	`AutomaticSequence | None`	declared DFAO
`holonomic`	`HolonomicRecurrence | None`	declared polynomial recurrence
`term(index)`	`Number`	dispatch to selected model, else original evaluator
`first(count)`	tuple	dispatch to selected model, else original evaluator
`modulo(index, modulus)`	`int`	modular execution described in Section 26.6
`generating_function()`	pair of rational tuples	rational OGF only when `recurrence` is present

The exact strategy strings are:

```
declared linear recurrence
declared finite automaton
declared holonomic recurrence
detected linear recurrence
direct evaluator
```

26.5.3 Declared-model metadata

`compile_sequence` recognizes these metadata pairs in order:

Model key	Trust marker	Required model type
`recurrence`	`recurrence_certified`	`LinearRecurrence`
`automatic_model`	`automatic_certified`	`AutomaticSequence`
`holonomic_recurrence`	`holonomic_certified`	`HolonomicRecurrence`

For a recognized model, the compiler compares it with the source sequence on a finite prefix. The trust marker changes certificate exactness:

Prefix agrees	Trust marker is `True`	Certificate
yes	yes	valid and `Exactness.EXACT`
yes	no or absent	valid and `Exactness.CONDITIONAL` with an unchecked infinite-identity assumption
no	either	invalid

> **Require before dispatch.** An invalid declared-model certificate does not cause fallback to a weaker strategy. The returned `CompiledSequence` still dispatches `term` and `first` to the mismatching declared model. Call `compiled.certificate.require()` before using compiled execution.

The three `*_certified` values are trusted metadata, not protected credentials. Any caller can set them. Do not mark an externally supplied model certified until its infinite identity has been established independently. The built-in catalog sets the markers only for models constructed as the evaluator's defining implementation.

26.5.4 Rational ordinary generating function

For

$$a_{n+d} = \sum_{j=0}^{d-1} c_j a_{n+j},$$

write

$$Q(z) = 1 - c_{d-1}z - c_{d-2}z^2 - \cdots - c_0 z^d.$$

Then

$$F(z) = \sum_{n \geq 0} a_n z^n = \frac{P(z)}{Q(z)},$$

where the implementation computes the numerator coefficients through degree $d - 1$ from the initial values and trims trailing zeros. Both returned tuples are in ascending powers of z.

For Fibonacci,

```
from komorebi_math.sequences import compile_sequence, fibonacci_sequence

compiled = compile_sequence(fibonacci_sequence())
compiled.certificate.require()
numerator, denominator = compiled.generating_function()

print(numerator)
print(denominator)
```

Output:

```
(Fraction(0, 1), Fraction(1, 1))
(Fraction(1, 1), Fraction(-1, 1), Fraction(-1, 1))
```

Thus $F(z) = z/(1 - z - z^2)$. `generating_function()` raises `ValidationError` for automatic, holonomic, and direct strategies. For a detected recurrence it returns the OGF of the fitted model. Its truth status remains conditional.

26.5.5 Declared-model example

```
from komorebi_math.sequences import (
    apery_sequence,
    compile_sequence,
    fibonacci_sequence,
    thue_morse_sequence,
)

for sequence in (
    fibonacci_sequence(),
    thue_morse_sequence(),
```

```
    apery_sequence(),
):
    compiled = compile_sequence(sequence)
    compiled.certificate.require()
    print(sequence.name, compiled.strategy, compiled.first(12))
```

Output, with denominator-one `Fraction` wrappers in the Apéry row suppressed:

```
Fibonacci declared linear recurrence (0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89)
Thue Morse declared finite automaton (0, 1, 1, 0, 1, 0, 0, 1, 1, 0, 0, 1)
Apery declared holonomic recurrence (1, 5, 73, 1445, 33001, 819005,
  21460825, 584307365, 16367912425, 468690849005,
  13657436403073, 403676083788125)
```

26.5.6 Detection and direct fallback

```
from fractions import Fraction
from komorebi_math.sequences import Sequence, compile_sequence

quartic = Sequence(lambda n: n**4 + 2*n + 1, name="quartic")
reciprocal = Sequence(lambda n: Fraction(1, n + 1), name="reciprocal")

for sequence in (quartic, reciprocal):
    compiled = compile_sequence(sequence)
    print(
        sequence.name,
        compiled.strategy,
        compiled.certificate.exactness.value,
    )

print(compile_sequence(quartic).recurrence.coefficients)
```

Output:

```
quartic detected linear recurrence conditional
reciprocal direct evaluator exact
(Fraction(1, 1), Fraction(-5, 1), Fraction(10, 1),
 Fraction(-10, 1), Fraction(5, 1))
```

The reciprocal evaluator receives `exact` because the source wrapper's default flag is true. That record says the direct callable is the selected execution path. It does not prove a formula for the callable.

26.5.7 Finite-prefix truth boundary

A detected relation establishes only

$$\forall n \in I_{\text{sample}} : \mathcal{R}(a, n) = 0.$$

It does not establish the same relation for every future index. The boundary is observable with a sequence that changes immediately after the default sample:

```
from komorebi_math.sequences import Sequence, compile_sequence

source = Sequence(lambda n: 0 if n < 24 else 1, name="prefix trap")
compiled = compile_sequence(source)

print(compiled.strategy)
print(compiled.certificate.exactness.value)
```

```
print(compiled.certificate.assumptions_unchecked)
print(compiled.term(24), source.term(24))
```

Output:

```
detected linear recurrence
conditional
('finite prefix detection does not prove the recurrence for all future terms',)
0 1
```

This is intended conditional behavior, not a failed exact calculation. A detected model must be tested against independent holdout terms before it is used to extrapolate a user-defined sequence.

The compiler does not infer DFAOs or polynomial-coefficient recurrences. It only consumes declared versions of those models and detects constant rational recurrences.

26.5.8 Compiler cost and resource controls

Let S be `sample_count` and D the effective `max_order`. Declared models incur finite prefix evaluation and comparison. Prefix fitting scans orders one through D and solves exact rational systems. A conservative high-level bound for the current dense elimination path is

$$O(SD^3)$$

scalar rational operations across all trial orders, with $O(SD)$ working storage at one order. Rational numerator and denominator growth can make the bit cost much larger.

`sample_count` must be at least three. An explicit `max_order` must be positive. `max_order=None` permits search through half the sample length. For untrusted input, set both values explicitly and cap the size and digit length of the sampled terms before compilation.

26.6 Modular and indexed evaluation

The subsystem exposes three related but different operations:

$$\text{indexing } a_n, \qquad \text{index packet } a_{mk+r}, \qquad \text{value reduction } a_n \bmod M.$$

An index residue and a value modulus are independent. The first chooses which terms to read. The second reduces the values after, or during, execution.

26.6.1 Indexed access ledger

Operation	Admissible arguments	Behavior
`sequence.term(n)`	integer $n \geq 0$	evaluate one term and cache it
`sequence[n]`	integer $n \geq 0$	same as `term`
`sequence[:N]`	finite $N \geq 0$	tuple $a_0, \ldots, a_{N-1}$
`sequence[i:j:k]`	nonnegative finite bounds, $k \geq 1$	tuple at Python range indices
`sequence.first(N)`	integer $N \geq 0$	first N terms
`sequence.residue(m, r)`	$m \geq 1, 0 \leq r < m$	wrapper $k \mapsto a_{mk+r}$
`automatic.residue_model(e, r)`	$e \geq 0, 0 \leq r < b^e$	new DFAO for $a_{b^e k+r}$

Booleans and floats are not accepted as integer indices. A negative index does not mean Python-style indexing from the end because an infinite sequence has no final element.

26.6.2 Modular dispatch

```
Sequence.modulo(modulus: int) -> Sequence

CompiledSequence.modulo(index: int, modulus: int) -> int
```

The lazy view validates $M \geq 2$ immediately and validates each value only when that term is requested. A rational term must have denominator one.

```
from fractions import Fraction
from komorebi_math.sequences import Sequence

rational = Sequence(lambda n: Fraction(1, n + 1))
residues = rational.modulo(5)

print(residues.term(0))
residues.term(1)  # raises ValidationError: denominator is not one
```

The first request prints `1`. The second fails lazily.

`CompiledSequence.modulo` dispatches as follows.

Selected strategy	Modular route	Index cost, ignoring bit arithmetic
declared integral `LinearRecurrence`	modular companion-matrix exponentiation	$O(d^3 \log n)$
prefix-detected `LinearRecurrence` in `1.0.0b4`	rejected because its denominator-one `Fraction` data fail the integral-type check	unavailable without reconstruction
`AutomaticSequence`	evaluate a_n, then require integral rational output and reduce	$O(\log_b n)$ transitions
`HolonomicRecurrence`	construct the exact rational term, then reduce	sequential through n
direct evaluator	invoke the exact term evaluator, then reduce	evaluator-dependent

Only the constant `LinearRecurrence` branch can avoid constructing the full integer term. It requires coefficients and seeds whose runtime types satisfy `numbers.Integral`, not merely rational values with denominator one.

> **Known `1.0.0b4` defect: detected modular recurrences.** `detect_linear_recurrence` represents every fitted coefficient and seed as a `Fraction`. `LinearRecurrence.term_mod` rejects every `Fraction`, including `Fraction(1, 1)`. Consequently `CompiledSequence.modulo` raises `ValidationError` for a prefix-detected recurrence instead of using the advertised modular matrix route. Built-in Fibonacci and Lucas models store Python integers and are unaffected.

After independent holdout validation, a caller may explicitly reconstruct an integral model when every detected denominator is one:

```
from komorebi_math.arithmetic import LinearRecurrence
from komorebi_math.sequences import Sequence, compile_sequence

source = Sequence(lambda n: n * n, name="squares")
compiled = compile_sequence(source)

# A finite holdout is useful evidence. The known square formula is the
# independent justification in this example.
assert all(compiled.term(n) == source.term(n) for n in range(24, 64))

detected = compiled.recurrence
```

```
data = (*detected.coefficients, *detected.initial)
if all(value.denominator == 1 for value in data):
    integer_model = LinearRecurrence(
        tuple(int(value) for value in detected.coefficients),
        tuple(int(value) for value in detected.initial),
        name=detected.name,
    )
    print(integer_model.term_mod(1000, 97))
```

Output is `27`. This workaround restores the modular type contract. It does not upgrade a finite prefix fit from conditional to proved.

26.6.3 Millionth Fibonacci residue

```
from komorebi_math.sequences import compile_sequence, fibonacci_sequence

compiled = compile_sequence(fibonacci_sequence())
compiled.certificate.require()

print(compiled.modulo(1_000_000, 1_000_003))
```

Output:

```
3
```

This uses logarithmic modular matrix powering. Calling `compiled.term(1_000_000)` instead follows the exact sequential recurrence and caches the enormous integers through that index.

26.6.4 Command-line evaluation

The built-in catalog is available through

```
python -m komorebi_math sequence NAME [--terms N] [--index N] [--modulus M] [--compile]
```

One indexed modular call is

```
python -m komorebi_math sequence fibonacci --index 1000000 --modulus 1000003 --compile
```

Output:

```
3
```

A finite automatic prefix is

```
python -m komorebi_math sequence thue-morse --terms 16 --compile
```

The JSON result contains the 16 values and

```
strategy: declared finite automaton
exactness: exact
```

With `--index`, the command prints one scalar. Without it, the command emits a JSON prefix. `--modulus` reduces every printed value. `--compile` reports the strategy for a prefix and explicitly selects compiled execution. A modular request also compiles internally because it needs `CompiledSequence.modulo`.

The CLI accepts only built-in names. Custom `Sequence` instances require the Python API.

26.6.5 Indexed-execution budgets

There is no global `SequenceBudget` object in `1.0.0b4`. A service boundary should validate at least:

Input	Suggested policy purpose
maximum index	caps sequential work and cache population
maximum prefix count	prevents accidental unbounded reports
maximum modulus bit length	caps modular integer arithmetic
maximum recurrence order	caps dense matrix and detection cost
maximum sample count	caps exact compiler elimination
maximum output digit count	prevents giant exact serialization
wall-clock or cancellation token	limits arbitrary user evaluators

Do not treat the logarithmic exponent count as a byte bound. Even modular matrix products allocate Python integers, while exact nonmodular terms can have a digit count proportional to the index.

26.7 Exact sequence transforms: binomial inverse, finite differences, and Cauchy products

The transform functions are exact when their input scalar arithmetic is exact. They do not coerce floating values to `Fraction`, and the finite tuple helpers do not perform a separate numeric type audit.

26.7.1 Function reference

```
binomial_transform_values(
    values: Iterable[Number],
) -> tuple[Number, ...]

inverse_binomial_transform_values(
    values: Iterable[Number],
) -> tuple[Number, ...]

binomial_transform(sequence: Sequence) -> Sequence
inverse_binomial_transform(sequence: Sequence) -> Sequence

finite_difference(
    sequence: Sequence,
    *,
    step: int = 1,
    order: int = 1,
) -> Sequence

partial_sums(sequence: Sequence) -> Sequence
cauchy_product(left: Sequence, right: Sequence) -> Sequence
```

The finite tuple helpers materialize one finite prefix. The corresponding lazy functions create a new `Sequence` and propagate the input exactness flag. The Cauchy product is exact only if both flags are true.

26.7.2 Binomial transform and inverse

For $0 \le n < N$,

$$b_n = \sum_{k=0}^{n} \binom{n}{k} a_k, \qquad a_n = \sum_{k=0}^{n} (-1)^{n-k} \binom{n}{k} b_k.$$

```
from komorebi_math.sequences import (
    binomial_transform_values,
    inverse_binomial_transform_values,
)

values = (1, 2, 4, 8, 16, 32)
transformed = binomial_transform_values(values)
restored = inverse_binomial_transform_values(transformed)

print(transformed)
print(restored)
```

Output:

```
(1, 3, 9, 27, 81, 243)
(1, 2, 4, 8, 16, 32)
```

Here the identity is the ordinary binomial theorem:

$$\sum_{k=0}^{n} \binom{n}{k} 2^k = 3^n.$$

The lazy form computes the same triangular sum at each requested index. The inverse is algebraically exact for integers, rationals, and any scalar domain supporting the required addition and multiplication laws.

26.7.3 Finite differences

For step $h \geq 0$ and order $r \geq 0$,

$$\Delta_h^r a_n = \sum_{k=0}^{r} (-1)^{r-k} \binom{r}{k} a_{n+kh}.$$

```
from komorebi_math.sequences import Sequence, finite_difference

quartic = Sequence(lambda n: n**4 + 2*n + 1, name="quartic")
fifth = finite_difference(quartic, order=5)

print(fifth.first(12))
```

Output:

```
(0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0)
```

The constructor permits zero step and zero order. Order zero is the identity. For positive order and zero step, the repeated value is multiplied by the alternating binomial sum and the result is zero in ordinary exact arithmetic. Negative step or order raises `ValidationError`.

26.7.4 Partial sums and Cauchy products

The inclusive partial-sum transform and ordinary Cauchy product are

$$s_n = \sum_{k=0}^{n} a_k, \qquad c_n = \sum_{k=0}^{n} a_k b_{n-k}.$$

The Cauchy coefficients are exactly the coefficients of ordinary generating function multiplication:

$$\left(\sum_{n\geq 0} a_n z^n\right)\left(\sum_{n\geq 0} b_n z^n\right) = \sum_{n\geq 0} c_n z^n.$$

```
from komorebi_math.sequences import Sequence, cauchy_product, partial_sums

left_values = (1, 2, 3)
right_values = (4, 5, 6)

left = Sequence(lambda n: left_values[n] if n < 3 else 0, name="left")
right = Sequence(lambda n: right_values[n] if n < 3 else 0, name="right")

product = cauchy_product(left, right)
ones = Sequence(lambda n: 1, name="ones")

print(product.first(6))
print(partial_sums(ones).first(8))
```

Output:

```
(4, 13, 28, 27, 18, 0)
(1, 2, 3, 4, 5, 6, 7, 8)
```

26.7.5 Transform cost ledger

For a requested prefix of length N and finite-difference order r, the current lazy and finite implementations have these high-level costs.

Transform	One term n	Prefix through $N-1$	Additional stored state
finite binomial or inverse	not applicable	$O(N^2)$	$O(N)$ output
lazy binomial or inverse	$O(n)$ summands	$O(N^2)$	$O(N)$ result cache
finite difference	$O(r)$ summands	$O(Nr)$	$O(N)$ result cache
partial sums	amortized $O(1)$ per next index	$O(N)$	$O(N)$ internal and wrapper caches
Cauchy product	$O(n)$ products	$O(N^2)$	$O(N)$ result cache plus input caches

These counts omit exact integer and rational bit complexity. A Cauchy or binomial transform of rapidly growing terms can become memory-bound well before the scalar-operation count is large.

26.7.6 Persistence and reproducible records

The portable codec registry in `1.0.0b4` does not register `Sequence`, `AutomaticSequence`, `HolonomicRecurrence`, `CompiledSequence`, or `LinearRecurrence`. This is intentional for `Sequence`: an arbitrary Python callable cannot be reconstructed safely from portable data. Runtime caches are also execution state, not a portable mathematical identity.

There are two supported documentation patterns.

Persist a finite report. The CLI can export a bounded built-in prefix:

```
python -m komorebi_math export sequence fibonacci --terms 100 --compile --format json --output .\fibonacci.json
    ↪  --force
```

The result preserves the printed values and strategy in a finite mathematical document. It does not serialize the evaluator or cache.

Persist a declarative model card. An application may store and validate the public constructor fields itself:

Model	Fields required for reconstruction
constant recurrence	coefficients, initial values, name
DFAO	base, transition rows, outputs, initial state, name
holonomic recurrence	ascending polynomial tuples, initial values, name
detected model	recurrence fields, sample count, sample values or digest, conditional status

Rational scalars should be encoded as numerator-denominator pairs rather than floating decimals. A reconstructed model must be passed through its constructor validation and a fresh certificate. A persisted prefix does not prove an infinite identity, and a persisted detected recurrence must retain its conditional provenance.

26.7.7 Failure and scope matrix

Situation	Observed boundary
evaluator is not callable	`Sequence` construction raises `ValidationError`
evaluator returns a nonnumeric value	first request for that term raises `ValidationError`
evaluator itself raises	its exception propagates from the lazy request
`exact=True` with floating output	accepted. Exactness remains unproved metadata
slice has no stop	`ValidationError` because the result would be infinite
modular term is nonrational or nonintegral	`ValidationError` at that requested term
malformed DFAO shape or target	constructor raises `DimensionError` or `ValidationError`
DFAO stores floating or complex outputs	deterministic lookup certificate remains `EXACT`. Scalar arithmetic is not thereby exact
holonomic leading polynomial vanishes at execution index	`ValidationError` during term generation
detected recurrence extrapolates beyond its prefix	permitted with a conditional certificate
modular evaluation of a detected recurrence	known `1.0.0b4` denominator-one `Fraction` rejection
declared model disagrees with prefix	compiler returns an invalid certificate and does not fall back
OGF requested without a constant recurrence	`ValidationError`
transform receives floating inputs	floating arithmetic is propagated. No exact coercion occurs
arbitrary sequence object is sent to portable codec	unsupported in `1.0.0b4`
unbounded iteration is consumed without a limit	execution and cache growth continue indefinitely

26.7.8 Verification and executable anchors

The canonical installed verification command is

```
python -m komorebi_math verify sequences
```

It executes five checks:

1. classical exact sequence catalog
2. binomial transform inversion
3. Thue-Morse finite automaton
4. exact holonomic sequence execution
5. declared recurrence compilation

Expected final line:

```
verification passed 5/5 checks
```

The focused source replay is

```
python -m pytest -q tests/test_sequences_execution.py tests/test_sequences_models.py
```

The current chapter audit obtains

```
13 passed
```

The two shipped examples are

```
python examples/sequence_compiler.py
python examples/automatic_holonomic.py
```

Both execute successfully. They cover all three declared compiler strategies, the Thue-Morse residue automaton, and the Apéry holonomic prefix.

A separate boundary probe reproduced the detected-recurrence modular defect: an integer square prefix compiled to denominator-one `Fraction` recurrence data, after which `compiled.modulo(100, 7)` raised the documented integral-data `ValidationError`. Reconstructing the same coefficients and seeds as Python integers returned the expected residue.

The independent source anchors are

```
src/komorebi_math/sequences/base.py
src/komorebi_math/sequences/catalog.py
src/komorebi_math/sequences/automatic.py
src/komorebi_math/sequences/holonomic.py
src/komorebi_math/sequences/compiler.py
src/komorebi_math/sequences/transforms.py
src/komorebi_math/arithmetic/recurrence.py
src/komorebi_math/verify/sequences.py
tests/test_sequences_execution.py
tests/test_sequences_models.py
examples/sequence_compiler.py
examples/automatic_holonomic.py
```

Chapter 26 summary. `Sequence` is a cached lazy numeric wrapper whose exactness flag is a declaration. Built-in constant recurrences, DFAOs, and holonomic recurrences provide stronger executable models. The compiler uses a fixed declared-model order, then fits only a constant rational recurrence to a bounded prefix. Declared built-ins can carry exact certificates, while inferred recurrences remain conditional. Only constant recurrences receive logarithmic modular acceleration. Every transform remains exact only when its scalar inputs are exact, every infinite iterator needs an external budget, and portable artifacts should store bounded reports or declarative model fields rather than Python callables and runtime caches.

27. Cyclic analytic systems

First route. Start with finite packet reconstruction in Section 27.1. For exact coefficient work, continue to the reciprocal, Appell, and logarithmic sections. For numerical zeros, go to Section 27.4 and read the certificate interpretation in Section 27.5 before drawing a conclusion. The combined workflow in Section 27.7 shows how exact symbolic checks and numerical diagnostics can be retained together without merging their claim classes.

The `komorebi_math.analytic` namespace turns a finite cyclic decomposition into an executable analytic calculus. It begins with the exact decomposition of a coefficient sequence into residue classes, builds entire transfer and signed section families, and then derives reciprocal, Appell, theta, logarithmic, and zero-diagnostic constructions. The implementation separates two levels of claim strength:

- Finite coefficient identities can be exact over `Fraction` and SymPy.
- Root locations, residuals, truncated products, and truncated zeta sums are numerical or conditional records.

This distinction is central to the API. A successful zero certificate says that the requested representatives are ordered and have small residuals. It does not assert that every zero has been found or prove a global zero-counting theorem.

The 43 exported names in `komorebi_math.analytic` belong to the research public surface. None is re-exported by `komorebi_math.beta` or `komorebi_math.stable` in version `1.0.0b4`. Pin the package version when these signatures form part of an application contract.

27.1 Cyclic packets and sections

27.1.1 Namespace and mathematical conventions

The public analytic layer is exported from

```
from komorebi_math import analytic
```

or directly from the subpackage:

```
from komorebi_math.analytic import cyclic_packets, signed_section
```

Let $N \geq 1$ be the cyclic order and let

$$f(z) = \sum_{m \geq 0} a_m z^m.$$

For $j \in \{0, \dots, N-1\}$, the j-th residue projection is

$$\Pi_j f(z) = \sum_{q \geq 0} a_{Nq+j} z^{Nq+j}.$$

Write the primitive root of unity as

$$\omega_N = e^{2\pi i/N},$$

the same projection has the root-of-unity form

$$\Pi_j f(z) = \frac{1}{N} \sum_{\ell=0}^{N-1} \omega_N^{-j\ell} f(\omega_N^{\ell} z).$$

The orthogonality relation

$$\frac{1}{N}\sum_{\ell=0}^{N-1}\omega_N^{(m-j)\ell}=\begin{cases}1, & m\equiv j \pmod N,\\ 0, & m\not\equiv j \pmod N\end{cases}$$

explains why the functional and coefficient definitions agree. The complete family reconstructs the original series:

$$f=\sum_{j=0}^{N-1}\Pi_j f.$$

In the finite array API, `cyclic_project_coefficients` retains the positions in one residue class and fills all other positions with zeros. `cyclic_packets` returns every residue packet, and `cyclic_reconstruct` sums equally shaped packets.

```
import numpy as np

from komorebi_math.analytic import (
    cyclic_packets,
    cyclic_projection_certificate,
    cyclic_reconstruct,
)

coefficients = np.arange(10)
packets = cyclic_packets(coefficients, order=3)

assert np.array_equal(
    cyclic_reconstruct(packets),
    coefficients,
)

checked = cyclic_projection_certificate(coefficients, order=3)
checked.certificate.require()
```

The certificate is a `ComputationResult`. Its `.value` is the tuple of packets. Its `.certificate` records the finite reconstruction residual. The result is classified as `Exactness.EXACT`, although the stored residual is represented as a floating norm for compatibility with the common certificate interface.

27.1.2 Packet products

If

$$f=\sum_{a=0}^{N-1}f_a,\qquad g=\sum_{b=0}^{N-1}g_b,$$

where the support of f_a lies in residue a and the support of g_b lies in residue b, then the product packet of residue c is

$$(fg)_c=\sum_{a+b\equiv c \pmod N} f_a g_b.$$

`cyclic_product_packets(left, right)` realizes this cyclic convolution of ordinary coefficient convolutions. Both families must have the same nonzero order and every packet must be one dimensional. The output length is chosen from the longest packet in each input family.

```
left = np.array([1.0, 2.0, 3.0, 4.0])
right = np.array([2.0, -1.0, 5.0])

product_packets = cyclic_product_packets(
    cyclic_packets(left, 3),
    cyclic_packets(right, 3),
```

```
)

assert np.allclose(
    sum(product_packets),
    np.convolve(left, right),
)
```

This is a finite coefficient operation. It does not inspect convergence radii or analytically continue an infinite series.

27.1.3 Transfer and signed sections

The transfer sections are

$$E_{N,j}(z;s)=\sum_{q\ge 0}\frac{(sz)^{Nq+j}}{(Nq+j)!},\qquad 0\le j<N,$$

and the signed sections are

$$S_{N,j}(z)=\sum_{q\ge 0}(-1)^q\frac{z^{Nq+j}}{(Nq+j)!}.$$

They are evaluated by `transfer_section` and `signed_section`. Their complete coordinate vectors are returned by `transfer_sections` and `signed_sections`. The series evaluator stops when the new term satisfies its relative tolerance. Failure to converge within `max_terms` raises `ArithmeticError`.

For order two,

$$E_{2,0}(z;1)=\cosh z,\qquad E_{2,1}(z;1)=\sinh z,$$

and

$$S_{2,0}(z)=\cos z,\qquad S_{2,1}(z)=\sin z.$$

The distinguished signed denominator is

$$D_N(z)=S_{N,0}(z)=\sum_{q\ge 0}(-1)^q\frac{z^{Nq}}{(Nq)!}.$$

The public functions

```
denominator
denominator_root_sum
denominator_derivative
```

evaluate D_N, its finite exponential root representation, and an arbitrary nonnegative derivative order. If

$$\eta_k=e^{(2k+1)\pi i/N},\qquad 0\le k<N,$$

then

$$D_N(z)=\frac{1}{N}\sum_{k=0}^{N-1}e^{\eta_k z}$$

and

$$D_N^{(r)}(z) = \frac{1}{N} \sum_{k=0}^{N-1} \eta_k^r e^{\eta_k z}.$$

`section_certificate` compares the directly summed section vector with the appropriate finite root representation. It reports numerical exactness because both paths use finite-precision complex arithmetic.

27.1.4 Companion realizations and addition

`transfer_shift_matrix(N, scale)` returns the cyclic shift with positive wrap. `signed_shift_matrix(N)` returns the signed shift J_N satisfying

$$J_N^N = -I_N.$$

The first column of the matrix exponential of the shift is the corresponding section vector. `CyclicSectionFamily` packages this realization:

```
from komorebi_math.analytic import CyclicSectionFamily

family = CyclicSectionFamily(
    order=4,
    family="signed",
    scale=1.0,
)

values = family.vector(0.2)
fundamental = family.fundamental_matrix(0.2)

assert np.allclose(fundamental[:, 0], values)
```

Its public members are:

Member	Meaning
`order`	positive cyclic order
`family`	either `"transfer"` or `"signed"`
`scale`	complex scale. The caller must supply a finite value
`evaluate(residue, z)`	evaluate one residue section
`vector(z)`	evaluate all residue sections
`fundamental_matrix(z)`	exponentiate the companion realization

SciPy's `expm` is used when available. Otherwise the package falls back to its internal matrix-exponential implementation.

Because the fundamental matrices form a one-parameter group, section coordinates obey a finite addition law. `cyclic_addition(left, right, signed=...)` performs that law. Both input tuples must have one common nonzero length. In the signed family, products that wrap around the cyclic boundary acquire a minus sign.

27.2 Appell and reciprocal families

27.2.1 Reciprocal coefficients

For a numeric parameter r, define the normalized reciprocal coefficients $P_{N,m}(r)$ through

$$D_N(z)^{-r} = \sum_{m \geq 0} P_{N,m}(r) \frac{z^{Nm}}{(Nm)!}.$$

`reciprocal_coefficients(order, parameter, degree)` returns the finite tuple

$$(P_{N,0}(r), \dots, P_{N,d}(r)).$$

Integer and `Fraction` parameters preserve exact rational arithmetic. Other numeric inputs are converted to complex arithmetic.

```
from fractions import Fraction

from komorebi_math.analytic import reciprocal_coefficients

values = reciprocal_coefficients(
    order=2,
    parameter=Fraction(1),
    degree=5,
)

assert values == (
    Fraction(1),
    Fraction(1),
    Fraction(5),
    Fraction(61),
    Fraction(1385),
    Fraction(50521),
)
```

The order-two, parameter-one sequence is the even-index secant-number family in the normalization used here.

The parameter law is a binomial convolution:

$$P_{N,m}(r+s) = \sum_{q=0}^{m} \binom{Nm}{Nq} P_{N,q}(r) P_{N,m-q}(s).$$

`reciprocal_semigroup_certificate` evaluates both sides through the requested degree. When all participating values are rational and the residual is zero, the certificate is exact. Otherwise it is numerical and uses the supplied tolerance.

`reciprocal_polynomial(order, degree)` returns $P_{N,m}(r)$ as an exact SymPy polynomial in the symbol `r`. It requires the optional SymPy dependency. `reciprocal_value(order, z, parameter)` evaluates the principal branch of $D_N(z)^{-r}$ and raises `ZeroDivisionError` when the denominator vanishes.

27.2.2 Congruential reciprocals

For residue j, introduce the normalized positive section

$$\Phi_{N,j}(w) = \sum_{m \geq 0} \frac{j!}{(Nm+j)!} w^m.$$

Its parameterized inverse is expanded as

$$\Phi_{N,j}(w)^{-r} = \sum_{m \geq 0} C_{N,j,m}(r) \frac{w^m}{(Nm)!}.$$

The finite coefficient tuple is produced by `congruential_reciprocal_coefficients`. The convenience function `congruential_inverse` fixes $r = 1$.

For the parameter-one inverse, the lower-triangular relation is

$$\sum_{q=0}^{m} \binom{Nm + j}{Nq} C_{N,j,q}(1) = \delta_{m0}.$$

The parameter law has the same residue-spaced binomial form:

$$C_{N,j,m}(r + s) = \sum_{q=0}^{m} \binom{Nm}{Nq} C_{N,j,q}(r) C_{N,j,m-q}(s).$$

These functions validate the order, residue, degree, and numeric parameter. Boolean values are not accepted as integer orders, residues, or degrees.

27.2.3 Congruential Appell polynomials

The Appell family attached to the congruential reciprocal coefficients is

$$A^{(r)}_{N,j,m}(x) = \sum_{q=0}^{\lfloor m/N \rfloor} \binom{m}{Nq} C_{N,j,q}(r) x^{m-Nq}.$$

`appell_polynomial` evaluates one polynomial. `appell_coefficients` returns all monomial coefficients in ascending power order, including explicit zeros. `appell_polynomial_symbolic` returns a SymPy expression in `x`.

The family satisfies the Appell derivative law

$$\frac{d}{dx} A^{(r)}_{N,j,m}(x) = m A^{(r)}_{N,j,m-1}(x).$$

```
from komorebi_math.analytic import (
    appell_coefficients,
    appell_polynomial,
)

degree = 8
coefficients = appell_coefficients(3, 1, 2, degree)
x = 0.31

direct = sum(
    coefficient * x**power
    for power, coefficient in enumerate(coefficients)
)

assert abs(
    direct - appell_polynomial(3, 1, 2, degree, x)
) < 1e-12
```

For symbolic checking:

```python
import sympy as sp

from komorebi_math.analytic import appell_polynomial_symbolic

x = sp.Symbol("x")
p5 = appell_polynomial_symbolic(2, 0, 1, 5)
p4 = appell_polynomial_symbolic(2, 0, 1, 4)

assert sp.simplify(sp.diff(p5, x) - 5 * p4) == 0
```

Current validation gap. The numerical Appell functions reach the shared residue validator and reject addresses outside $0 \le j < N$. In version `1.0.0b4`, `appell_polynomial_symbolic` does not call that validator and can accept a residue at least as large as the order. Validate `order` and `residue` explicitly before calling the symbolic helper.

27.2.4 Theta polynomials

`theta_polynomial(order, degree, parameter)` evaluates one transfer theta polynomial. Integer and rational parameters use exact arithmetic. `theta_polynomial_symbolic(order, degree)` returns the exact SymPy polynomial in the symbol `u`.

In the implemented normalization, every positive-degree theta polynomial vanishes at $u = 0$:

$$\Theta_{N,m}(0) = 0, \qquad m \ge 1.$$

For even order it also vanishes at $u = -1$:

$$\Theta_{N,m}(-1) = 0, \qquad N \equiv 0 \pmod 2, \qquad m \ge 1.$$

These are exact coefficient identities, not approximate sampling statements.

27.3 Logarithmic derivatives and coefficient recurrences

27.3.1 Reduced logarithmic coefficients

Set $w = z^N$ and write

$$\widehat{D}_N(w) = \sum_{m \ge 0} (-1)^m \frac{w^m}{(Nm)!}.$$

The exact logarithmic coefficients $\kappa_{N,m}$ are defined by

$$-\log \widehat{D}_N(w) = \sum_{m \ge 1} \kappa_{N,m} \frac{w^m}{m}.$$

`logarithmic_coefficients(order, degree)` returns

$$(\kappa_{N,0}, \kappa_{N,1}, \dots, \kappa_{N,d}),$$

where index zero is the unused zero entry. The implementation uses exact `Fraction` arithmetic.

If

$$\widehat{D}_N(w) = 1 + \sum_{m \ge 1} a_m w^m,$$

coefficient comparison in

$$\widehat{D}'_N(w) = -\widehat{D}_N(w) \sum_{m \geq 1} \kappa_{N,m} w^{m-1}$$

gives the recurrence

$$\kappa_{N,m} = -m a_m - \sum_{q=1}^{m-1} a_q \kappa_{N,m-q}.$$

This recurrence is the shared engine behind the reciprocal, congruential, and theta constructions.

27.3.2 Transfer tangent

The transfer tangent is the logarithmic derivative

$$T_N(z) = -\frac{D'_N(z)}{D_N(z)}.$$

`transfer_tangent(order, z)` evaluates this quotient away from zeros of D_N. Its ordinary coefficient dictionary is returned by `transfer_tangent_coefficients(order, degree)`:

$$T_N(z) = \sum_{m \geq 0} N \kappa_{N,m+1} z^{Nm+N-1}.$$

Only the nonzero residue-spaced exponents appear in the returned dictionary. The keys are ordinary monomial exponents, not compressed series indices.

```
from komorebi_math.analytic import (
    logarithmic_coefficients,
    transfer_tangent,
    transfer_tangent_coefficients,
)

kappa = logarithmic_coefficients(3, 5)
coefficients = transfer_tangent_coefficients(3, 4)

assert coefficients[2] == 3 * kappa[1]
assert coefficients[5] == 3 * kappa[2]

z = 0.1
truncated = sum(
    3 * kappa[index + 1] * z ** (3 * index + 2)
    for index in range(5)
)

assert abs(truncated - transfer_tangent(3, z)) < 1e-10
```

The last assertion is an approximation because the finite series is truncated. The coefficients used to form it are exact.

27.4 Positive zero strings and numerical zero certificates

27.4.1 Distinguished denominator wrapper

`CyclicDenominator(order)` stores one validated cyclic order and exposes:

Member	Result
`__call__(z)`	$D_N(z)$
`derivative(z, derivative_order=1)`	$D_N^{(r)}(z)$
`positive_zeros(count, start=0)`	a `ZeroStringResult`
`zero_count(radius, reduced=False)`	leading count asymptotic

Positive-zero routines require $N \geq 2$. The leading seed for zero index $m \geq 0$ is

$$x_{N,m}^{(0)} = \frac{\pi(m + \frac{1}{2})}{\sin(\pi/N)}.$$

`asymptotic_positive_zero(order, index)` returns this model. For $N = 2$ and $N = 4$, the seed is already the exact positive representative in the implemented families. Other orders are refined by an exponentially scaled Newton iteration.

`count`, `start`, and `tolerance` receive explicit domain checks. In version `1.0.0b4`, `max_iterations` does not. Pass a positive integer explicitly. For orders two and four the exact-seed branch does not consume that argument. For other orders, a nonpositive iteration limit skips refinement and normally yields a failed residual certificate.

27.4.2 `ZeroStringResult`

`positive_zero_string` has the signature

```
positive_zero_string(
    order: int,
    count: int,
    *,
    start: int = 0,
    tolerance: float = 1e-12,
    max_iterations: int = 64,
) -> ZeroStringResult
```

and returns the immutable record

```
ZeroStringResult(
    order: int,
    indices: tuple[int, ...],
    zeros: tuple[float, ...],
    residuals: tuple[float, ...],
    asymptotic_seeds: tuple[float, ...],
    certificate: Certificate,
) -> None

ZeroStringResult.rotational_orbits(
    self,
) -> tuple[tuple[complex, ...], ...]
```

The certificate is numerical. It checks:

1. Positive asymptotic seeds were constructed.
2. The requested representatives are strictly ordered.
3. The maximum scaled function residual is at most twenty times the requested tolerance.

It explicitly leaves unchecked the exhaustion of zeros outside the computed range.

Each positive representative has a rotational orbit

$$\mathcal{O}(x_m) = \{x_m \omega_N^k : 0 \le k < N\}.$$

`ZeroStringResult.rotational_orbits()` materializes these finite tuples.

```python
from komorebi_math.analytic import positive_zero_string

zeros = positive_zero_string(4, count=4)
zeros.certificate.require()

assert zeros.indices == (0, 1, 2, 3)
assert len(zeros.rotational_orbits()) == 4
assert all(len(orbit) == 4 for orbit in zeros.rotational_orbits())
```

27.4.3 Finite Hadamard and coefficient diagnostics

Given positive representatives $x_0, \dots, x_{M-1}$, the finite reduced Hadamard product is

$$H_{N,M}(z) = \prod_{m=0}^{M-1} \left(1 - \frac{z^N}{x_m^N}\right).$$

`hadamard_partial_product` evaluates this expression and requires every representative to be strictly positive. It is a finite approximation to the denominator, not a complete infinite product certificate.

`reciprocal_coefficient_from_zeros` uses finitely many orbits to approximate one reciprocal coefficient:

$$\widetilde{P}_{N,m} = (Nm)! \sum_{x \in X_M} \frac{-N x^{-Nm-1}}{D_N'(x)}.$$

Increasing the finite zero set may improve the diagnostic, but the function does not attach a global truncation bound.

27.4.4 Count and zeta diagnostics

The leading count models are

$$N_N(R) \sim \frac{N \sin(\pi/N)}{\pi} R$$

for the full rotational family, and

$$N_N^{\mathrm{red}}(R) \sim \frac{\sin(\pi/N)}{\pi} R^{1/N}$$

for the reduced convention implemented by `zero_count_asymptotic(..., reduced=True)`.

`reduced_zero_zeta(order, exponent, terms=64, use_computed_zeros=True)` has two explicit modes:

Mode	Value	Certificate
computed zeros	finite sum of x_m^{-Ns}	numerical
asymptotic model	scaled Hurwitz-zeta leading model	conditional

In computed-zero mode,

$$Z_{N,M}(s) = \sum_{m=0}^{M-1} x_m^{-Ns}.$$

The certificate records the maximum zero residual and a tail-scale diagnostic, but it lists the omitted zeta tail as unchecked. In model mode, the function returns

$$\left(\frac{\pi}{\sin(\pi/N)}\right)^{-Ns} \zeta(Ns, \tfrac{1}{2}),$$

classifies it as conditional, and explicitly omits the entire correction term. That mode requires `mpmath` through the scientific dependency profile.

The API accepts an arbitrary complex exponent. It does not enforce the half-plane in which an infinite zero sum would converge, and the model has the Hurwitz-zeta pole when $Ns = 1$. Treat computed mode as a finite sum and model mode as analytic continuation of a conditional leading expression.

27.5 Exact coefficient claims versus numerical root claims

27.5.1 Claim-strength matrix

The table below gives the practical interpretation boundary for this chapter.

Operation	Arithmetic model	Strongest local claim
cyclic coefficient split and reconstruction	finite NumPy arrays	exact structural identity
rational reciprocal coefficients	`Fraction`	exact finite coefficients
reciprocal parameter convolution	rational or complex	exact when rational. Otherwise numerical
symbolic reciprocal/Appell/theta polynomial	SymPy	exact symbolic identity
section root-sum comparison	complex floating point	numerical residual
positive zero string	scaled Newton iteration	ordered requested roots with residuals
finite Hadamard product	complex floating point	finite diagnostic only
reciprocal coefficient from zeros	finite zero truncation	approximation without global tail bound
reduced zero zeta from roots	finite sum	numerical truncation
Hurwitz-zeta model	asymptotic model	conditional leading term

An exact coefficient result does not upgrade a separately computed zero list to an exact or exhaustive list. Conversely, a numerical zero certificate does not weaken an exact rational coefficient identity computed independently.

27.5.2 Certificate reading discipline

For any returned `ComputationResult`, inspect both layers:

```
result = reduced_zero_zeta(3, 1.25, terms=24)

print(result.value)
print(result.certificate.exactness)
print(result.certificate.assumptions_checked)
print(result.certificate.assumptions_unchecked)
print(result.certificate.diagnostics)
```

Calling `result.certificate.require()` means only that the encoded validation conditions passed. It does not erase `assumptions_unchecked` or promote a numerical result to exactness.

27.5.3 Nonclaims

The analytic namespace does not claim:

- Exhaustive enumeration of every zero of D_N.
- A proof of the global zero asymptotic from numerical samples.
- A certified tail bound for every finite Hadamard or zeta truncation.
- Automatic branch reconciliation across zeros for complex reciprocal powers.
- Arbitrary-precision interval isolation of complex zeros.
- Symbolic execution without the optional SymPy dependency.

These are deliberate boundaries, not silent omissions.

27.6 Internal finite-series helpers used by public analytic constructions

The module `komorebi_math.analytic._series` is internal. Its leading underscore is the compatibility warning: applications should call the public analytic functions rather than import these helpers directly. They are documented here to make the recurrence chain auditable, not to enlarge the supported public surface.

Internal helper	Role
`validate_order`	reject nonintegral or too-small orders
`validate_degree`	reject negative or nonintegral truncation degrees
`multiply_series`	truncated ordinary Cauchy product
`inverse_series`	triangular reciprocal with nonzero constant term
`logarithmic_coefficients`	coefficients of $-\log A(w)$ when $A(0) = 1$
`power_series_from_log`	exponentiate a finite logarithmic record
`signed_denominator_coefficients`	exact or floating coefficients of D_N
`positive_section_coefficients`	normalized congruential section coefficients

For two truncated series

$$A(w) = \sum_{m=0}^{d} a_m w^m, \qquad B(w) = \sum_{m=0}^{d} b_m w^m,$$

the multiplication helper returns

$$[w^n](AB) = \sum_{q=0}^{n} a_q b_{n-q}, \qquad 0 \le n \le d.$$

If $a_0 \neq 0$, the inverse recurrence is

$$b_0 = a_0^{-1}, \qquad b_n = -a_0^{-1} \sum_{q=1}^{n} a_q b_{n-q}.$$

If

$$-\log A(w) = \sum_{n \geq 1} \kappa_n \frac{w^n}{n},$$

then `power_series_from_log` constructs the coefficients c_n of

$$C(w) = \exp\left(r \sum_{n \geq 1} \kappa_n \frac{w^n}{n} \right)$$

using

$$c_0 = 1, \qquad c_n = \frac{r}{n} \sum_{q=1}^{n} \kappa_q c_{n-q}.$$

The public reciprocal functions multiply these normalized coefficients by $(Nn)!$ at the boundary. Keeping the normalized internal series separate from the exposed factorial-power coefficients prevents accidental mixing of ordinary and exponential conventions.

27.7 End-to-end symbolic and numerical workflow

This bounded example exercises the chapter's two claim layers without conflating them.

```
from fractions import Fraction

import sympy as sp

from komorebi_math.analytic import (
    CyclicDenominator,
    appell_polynomial_symbolic,
    cyclic_projection_certificate,
    reciprocal_coefficients,
    reciprocal_semigroup_certificate,
    theta_polynomial_symbolic,
)

# Exact finite packet reconstruction.
projection = cyclic_projection_certificate(range(15), order=4)
projection.certificate.require()

# Exact rational reciprocal coefficients.
secant = reciprocal_coefficients(2, Fraction(1), degree=5)
assert secant[:4] == (
    Fraction(1),
    Fraction(1),
    Fraction(5),
    Fraction(61),
)

# Exact finite semigroup identity.
semigroup = reciprocal_semigroup_certificate(
    4,
    Fraction(2, 3),
    Fraction(5, 4),
    degree=6,
```

```
)
semigroup.certificate.require()

# Exact symbolic Appell derivative identity.
x = sp.Symbol("x")
p5 = appell_polynomial_symbolic(2, 0, 1, 5)
p4 = appell_polynomial_symbolic(2, 0, 1, 4)
assert sp.simplify(sp.diff(p5, x) - 5 * p4) == 0

# Exact symbolic theta factors.
u = sp.Symbol("u")
theta = theta_polynomial_symbolic(4, 3)
assert sp.simplify(theta.subs(u, 0)) == 0
assert sp.simplify(theta.subs(u, -1)) == 0

# Numerical zero representatives with an explicit certificate.
zeros = CyclicDenominator(4).positive_zeros(4)
zeros.certificate.require()
assert zeros.certificate.exactness.value == "numerical"
```

The symbolic assertions are literal algebraic equalities. The last assertion checks the zero result's declared claim class. It does not turn its coordinates into exact symbolic numbers.

27.8 Error model and troubleshooting

The following table records behavior in the audited reference baseline. It does not assert that a defect persists in a later package release.

Symptom	Cause	Remedy
`ValidationError` for order	zero, negative, Boolean, or nonintegral order	pass a positive integer. Zero routines require at least two
`ValidationError` for residue	address is outside `0 <= residue < order`	reduce or validate the residue explicitly
`ValidationError` for degree	negative, Boolean, or nonintegral degree	use a nonnegative integer truncation
`ValidationError` for parameter	object is not numeric	use `int`, `Fraction`, float, or complex
`BackendUnavailableError`	SymPy or mpmath-dependent path requested without dependency	install the matching optional dependency profile
`ZeroDivisionError`	reciprocal or tangent evaluated at a denominator zero	choose a point off the zero set or use a limiting analysis
`ArithmeticError`	section series exceeded `max_terms`	increase the limit or review point magnitude and tolerance
failed zero certificate	Newton residual or ordering check exceeded tolerance	inspect seeds and residuals. Do not discard the failure
unexpected complex dtype	packet product promotes to a common complex-compatible dtype	compare values, not an assumed input dtype
symbolic Appell accepts an out-of-range residue	recorded symbolic helper omits the shared residue guard	prevalidate `0 <= residue < order`
nonfinite section scale propagates	`CyclicSectionFamily` validates the family and order, not scale finiteness	require finite real and imaginary parts before construction
invalid `max_iterations` gives `TypeError` or a failed certificate	recorded zero solver does not validate this argument	pass a positive integer

For large $|z|$, the public section sum is mathematically entire but may still be numerically awkward in fixed precision. A section certificate compares two finite-precision realizations. It is not an interval-arithmetic enclosure. `reciprocal_value` and `transfer_tangent` detect an exactly zero computed denominator, not proximity to a pole. Inspect $|D_N(z)|$ when conditioning matters.

27.9 Public API reference

27.9.1 Cyclic projection

Callable	Return	Contract
`cyclic_project_coefficients(coefficients, order, residue)`	`np.ndarray`	retain one residue class
`cyclic_packets(coefficients, order)`	tuple of arrays	produce complete residue family
`cyclic_reconstruct(packets)`	`np.ndarray`	sum equally shaped packets
`cyclic_project_value(function, z, order, residue)`	`complex`	root-of-unity projection
`cyclic_product_packets(left, right)`	tuple of arrays	cyclic convolution of packet products
`cyclic_projection_certificate(coefficients, order, tolerance=...)`	`ComputationResult`	validate finite reconstruction

27.9.2 Sections and companions

Callable	Return	Contract
`transfer_section(order, residue, z, ...)`	`complex`	one positive-wrap entire section
`signed_section(order, residue, z, ...)`	`complex`	one negative-wrap entire section
`transfer_sections(order, z, scale=...)`	tuple	complete transfer vector
`signed_sections(order, z)`	tuple	complete signed vector
`denominator(order, z)`	`complex`	evaluate D_N
`denominator_root_sum(order, z)`	`complex`	finite exponential representation
`denominator_derivative(order, z, derivative_order=1)`	`complex`	arbitrary nonnegative derivative
`transfer_shift_matrix(order, scale=1)`	`np.ndarray`	positive cyclic companion
`signed_shift_matrix(order)`	`np.ndarray`	companion with $J^N = -I$
`cyclic_addition(left, right, signed=...)`	tuple	coordinate addition law
`section_certificate(order, z, family=..., tolerance=...)`	`ComputationResult`	root-sum comparison
`CyclicSectionFamily(...)`	record	family wrapper and fundamental matrix

27.9.3 Reciprocal, Appell, and theta functions

Callable	Return	Contract
`logarithmic_coefficients(order, degree)`	tuple of `Fraction`	exact reduced log coefficients
`reciprocal_coefficients(order, parameter, degree)`	numeric tuple	factorial-power reciprocal coefficients
`reciprocal_polynomial(order, degree)`	SymPy expression	polynomial in reciprocal parameter
`reciprocal_value(order, z, parameter=1)`	`complex`	principal reciprocal branch
`transfer_tangent(order, z)`	`complex`	$-D_N'/D_N$

Callable	Return	Contract
`transfer_tangent_coefficients(order, degree)`	dictionary	ordinary monomial coefficients
`reciprocal_semigroup_certificate(...)`	`ComputationResult`	finite parameter-convolution check
`congruential_reciprocal_coefficients(...)`	numeric tuple	normalized section reciprocal
`congruential_inverse(order, residue, degree)`	numeric tuple	parameter-one inverse
`appell_polynomial(...)`	number	evaluate one Appell polynomial
`appell_coefficients(...)`	numeric tuple	ascending monomial coefficients
`appell_polynomial_symbolic(...)`	SymPy expression	exact polynomial in `x`
`theta_polynomial(order, degree, parameter)`	number	evaluate one theta polynomial
`theta_polynomial_symbolic(order, degree)`	SymPy expression	exact polynomial in `u`

27.9.4 Zero diagnostics

Callable	Return	Contract
`asymptotic_positive_zero(order, index)`	`float`	leading seed model
`positive_zero_string(order, count, ...)`	`ZeroStringResult`	requested numerical representatives
`hadamard_partial_product(order, z, positive_zeros)`	`complex`	finite reduced product
`reciprocal_coefficient_from_zeros(...)`	`complex`	finite-orbit coefficient diagnostic
`zero_count_asymptotic(order, radius, reduced=False)`	`float`	leading count model
`reduced_zero_zeta(order, exponent, ...)`	`ComputationResult`	finite or conditional zeta diagnostic
`CyclicDenominator(order)`	record	denominator evaluation and zero facade
`ZeroStringResult.rotational_orbits()`	tuple of tuples	complete finite cyclic orbit per representative

The namespace also exports `status()` and `require()` for feature discovery. They report the implementation status. They are not mathematical validators.

27.10 Validation evidence

The chapter was checked against the implementation sources

```
src/komorebi_math/analytic/__init__.py
src/komorebi_math/analytic/_series.py
src/komorebi_math/analytic/appell.py
src/komorebi_math/analytic/cyclic.py
src/komorebi_math/analytic/reciprocal.py
src/komorebi_math/analytic/sections.py
src/komorebi_math/analytic/zeros.py
```

and the focused test modules

```
tests/test_analytic_cyclic_reciprocal.py
tests/test_analytic_appell_zeros.py
```

The focused replay completed with

```
19 passed
```

The shipped example `examples/cyclic_analytic.py` also completed successfully. Those executions cover finite packet reconstruction, root-of-unity projection, packet products, section identities, companion matrices, exact reciprocal and Appell formulas, symbolic theta factors, positive-zero special cases, rotational orbits, finite Hadamard diagnostics, reciprocal coefficients from zeros, count models, zeta claim classes, and invalid inputs.

Chapter 27 summary. Cyclic residue projection is an exact finite decomposition. Transfer and signed sections turn it into a finite companion calculus. Rational and symbolic reciprocal/Appell/theta constructions retain exact coefficient arithmetic. Logarithmic coefficients drive the shared recurrences. Positive zero strings, finite Hadamard products, and zeta sums remain explicitly numerical or conditional. Always read the certificate's unchecked assumptions before extending a finite diagnostic into a global analytic claim.

Part VI. Structured numerical computation and signal processing

This part has two entry points. Chapters 28 through 30 document `komorebi_math.algorithms`: packet intermediate representations, structured and low-rank operators, Krylov methods, precision allocation, finite compilation, scheduling, discovery, and optional device execution. Chapters 31 through 36A document the signal-processing tools in `komorebi_math.signals`. Use the namespace named in the relevant chapter when importing its public objects.

The claim labels are: **E** exact finite algebra or combinatorics. **N** a finite numerical result checked at a stated tolerance. **H** a heuristic or data-dependent rule. **M** a declared work, storage, or traffic model. **C** means conditional on the stated hypotheses, and **O** identifies an optional-backend statement. A certificate reports the checks and assumptions recorded by its producer. In particular, a successful finite execution is not a proof of an untested global operator hypothesis.

The API and behavioral statements in this part are grounded in the source modules named in each section. The verification ledger at the end records the tests and examples executed for the recorded reference baseline.

For the algorithms namespace, the controls are `FEATURE_KEY = "algorithms"`, `status() -> FeatureSpec`, which returns the registered feature specification, and `require() -> None`, the implemented-namespace compatibility no-op. Evidence: `src/komorebi_math/algorithms/__init__.py`.

Choose a starting point. Matrix and operator work begins in Chapter 28. Sampled-signal work can begin directly in Chapter 31 after the array and certificate conventions of Part II. The two routes meet in explicit finite operators, but readers need not study every compiler and scheduling API before filtering a signal.

Your task	Start here	Check before interpreting the result
Apply a matrix through named coordinate blocks	Chapters 28 and 29	Source/target layout and any omitted blocks
Solve or plan a finite operator computation	Chapter 30	Solver assumptions, residual, and the declared cost model
Frame a signal or inspect its spectrum	Chapters 31 and 32	Window, hop, padding, frequency scale, and reconstruction interval
Filter or change a sample rate	Chapters 33 and 34	Boundary convention, output length, and continuation state
Decompose or denoise with wavelets	Chapter 35	Transform depth, pruning, and the meaning of the risk certificate
Preserve or estimate jumps	Chapters 36 and 36A	Supplied packet information and unchecked recovery assumptions

In the operator chapters, *RHS* means right-hand side: one vector or several vectors arranged as matrix columns. A *residual* measures the discrepancy in a stated comparison. A *bound* limits a quantity under stated hypotheses. A small residual, a small bound, and a low operation count answer different questions.

28. Packet intermediate representation

A packet layout decomposes a finite coordinate space into named contiguous pieces. `PacketIR` then represents an operator by blocks between those pieces. An address is always `(target, source)`: block `(t, s)` maps source packet `s` to target packet `t`.

Before constructing a layout. Write down the input vector length, the output vector length, and the packet sizes on each side. Source sizes must partition the input, and target sizes must partition the output. A block's row count belongs to its target packet; its column count belongs to its source packet. Keeping this small shape record beside the code makes transposed addresses and mismatched layouts easier to diagnose.

28.1 Layouts, blocks, and `PacketIR`

For

$$X = \bigoplus_{s=0}^{p-1} X_s, \qquad n_s = \dim X_s > 0, \qquad n = \sum_s n_s,$$

`PacketLayout` stores `sizes=(n_0,...,n_{p-1})`. Its offsets and packet intervals are

$$o_0 = 0, \qquad o_{s+1} = o_s + n_s, \qquad I_s = [o_s, o_{s+1}).$$

`names=None` is normalized to `("0", ..., str(p-1))`. Supplied names are stringified, must match the packet count, and must be nonempty and unique. Sizes must be positive integer-index values. Booleans and numeric lookalikes such as `1.5` are rejected.

`PacketLayout` reference

```
PacketLayout(
    sizes: tuple[int, ...],
    names: tuple[str, ...] | None = None,
) -> PacketLayout
```

Member	Exact signature or value	Meaning
`sizes`	`tuple[int, ...]`	Normalized positive packet sizes
`names`	`tuple[str, ...]`	Explicit or generated packet names
`offsets`	`tuple[int, ...]`	`(o_0,...,o_p)`
`slices`	`tuple[slice, ...]`	Contiguous packet coordinate slices
`dimension`	`int`	`sum(sizes)`
`stored_coordinate_count`	property returning `int`	Equal to `dimension`
`equal`	`equal(packet_count: int, packet_size: int, *, prefix: str = "p") -> PacketLayout`	Equal-size named layout
`from_packet_system`	`from_packet_system(packet_system: Any) -> PacketLayout`	Uses `packet_dimensions()` and `names`
`index`	`index(packet: int | str) -> int`	Resolve an integer index or string name
`packet_slice`	`packet_slice(packet: int | str) -> slice`	Resolve `I_s`
`packet_size`	`packet_size(packet: int | str) -> int`	Resolve `n_s`

Member	Exact signature or value	Meaning
`split`	`split(value: Any, *, copy: bool = False) -> tuple[np.ndarray, ...]`	Split a vector or two-dimensional RHS by leading dimension
`merge`	`merge(packets: Sequence[Any]) -> np.ndarray`	Concatenate like-rank packet arrays
`crossed`	`crossed(other: PacketLayout, *, separator: str = "⊗") -> PacketLayout`	Tensor-product packet layout in left-major order
`permutation`	`permutation(order: Sequence[int]) -> np.ndarray`	Coordinate indices for a packet permutation

`split(..., copy=False)` returns views where NumPy slicing permits. `merge` requires all packets to be vectors or all to be matrices. Matrix packets must have a common RHS-column count. For layouts `X` and `Y`, `X.crossed(Y)` has sizes `n_s m_t` and names `X_s⊗Y_t` in nested left-then-right order. **E**

For source layout `X` and target layout `Y`, the stored blocks are

$$A_{ts} = Q_t A P_s \in \mathbb{K}^{m_t \times n_s}, \qquad (t, s) \in \Sigma(A).$$

`PacketIR` reference

```
PacketIR(
    source_layout: PacketLayout,
    target_layout: PacketLayout,
    blocks: Mapping[tuple[int, int], StructuredOperator | Any],
    metadata: Mapping[str, Any] = {},
) -> PacketIR
```

Member	Exact signature or value	Meaning
`square`	`square(layout, blocks, *, metadata=None) -> PacketIR`	Use one layout for source and target
`from_dense`	`from_dense(matrix, source_layout, target_layout=None, *, atol=0.0, rtol=0.0, keep_zeros=False) -> PacketIR`	Compile dense packet blocks by norm threshold
`shape`	property `tuple[int, int]`	`(target.dimension, source.dimension)`
`dtype`	property `np.dtype`	Result type of stored blocks. `float` for empty support
`support`	property `frozenset[tuple[int, int]]`	Stored block addresses
`stored_entries`	property `int`	Sum of each block's `storage_entries()`
`flop_count`	`flop_count(rhs: int = 1) -> int`	Sum of active block models
`block`	`block(target: int | str, source: int | str) -> StructuredOperator | None`	Stored block or `None`
`materialize`	`materialize() -> np.ndarray`	Dense matrix with blocks at packet slices
`apply`	`apply(value: Any) -> np.ndarray`	Apply to a vector or matrix RHS
`adjoint`	`adjoint() -> PacketIR`	Swap layouts and addresses and adjoint each block
`compose`	`compose(right: PacketIR, *, atol: float = 0.0) -> PacketIR`	`self` after `right`
`power`	`power(exponent: int, *, atol: float = 0.0) -> PacketIR`	Nonnegative square power by binary exponentiation
`tensor`	`tensor(other: PacketIR) -> PacketIR`	Blockwise Kronecker product

Member	Exact signature or value	Meaning
`diagonal_part`	`diagonal_part() -> PacketIR`	Retain `(s,s)` blocks of a common square layout
`certificate`	`certificate(*, tolerance: float = 1e-10) -> Certificate`	Compare application to materialization on the identity RHS

The application law is

$$y_t = \sum_{s:(t,s)\in\Sigma(A)} A_{ts}x_s, \qquad y = \mathrm{merge}_Y(y_0, \dots, y_{q-1}).$$

Vector input produces vector output. An `n x r` RHS produces an `m x r` output. `blocks` and `metadata` are copied into read-only mapping proxies at construction. Dense arrays supplied as blocks are wrapped by `DenseOperator`. The certificate computes

$$r_{\mathrm{IR}} = \|A(I_n) - \mathrm{materialize}(A)\|_F$$

because `np.linalg.norm` is called without an `ord`. It is valid when `r_IR <= tolerance`. Its exactness is `NUMERICAL`. **N**

Evidence: `src/komorebi_math/algorithms/layout.py`, `src/komorebi_math/algorithms/ir.py`, and `tests/test_algorithms_layout_ir.py`.

28.2 Dense-to-packet compilation

`PacketIR.from_dense` checks the full dense shape, computes

$$\theta = \tau_{\mathrm{abs}} + \tau_{\mathrm{rel}}\|A\|_2,$$

and stores a packet block exactly when

$$\|A[I_t, I_s]\|_2 > \theta.$$

The comparison is strict: a block exactly on the threshold is omitted. `keep_zeros=True` stores every packet rectangle regardless of norm. The method copies each retained dense block and records `{"support_threshold": theta}` in the IR metadata. Thresholding is numerical. Omission does not prove that a block is algebraically zero. **N/H**

Composition first materializes each contributing structured block, forms

$$C_{ts} = \sum_u A_{tu}B_{us},$$

then retains `C_ts` unless `atol > 0` and `||C_ts||_2 <= atol`. Consequently, the output blocks of `compose` are dense even if the inputs had more specialized block types. With `atol=0`, a reached block is stored even when different paths cancel to the zero matrix. This is intentional: structural reachability and a numerically nonzero block are different facts.

Runnable example (shipped as `examples/packet_ir.py`):

```
import numpy as np
from komorebi_math.algorithms import PacketIR, PacketLayout, packet_cost_estimate

layout = PacketLayout((2, 1), ("bulk", "edge"))
matrix = np.array([[2., 1., 0.],
                   [0., 3., 0.],
                   [4., 0., 5.]])
operator = PacketIR.from_dense(matrix, layout)
```

```
x = np.array([1., 2., 3.])
y = operator.apply(x)
operator.certificate().require()
cost = packet_cost_estimate(layout, layout, operator.support)

print("support", sorted(operator.support))
print("result", y)
print("stored entries", operator.stored_entries)
print("modeled flops", cost.flops)
```

Observed output in the audited environment:

```
support [(0, 0), (1, 0), (1, 1)]
result [ 4.  6. 19.]
stored entries 7
modeled flops 14
```

Evidence: `src/komorebi_math/algorithms/ir.py` and `examples/packet_ir.py`.

28.3 Support propagation

The support module treats addresses as a directed relation, still in `(target, source)` order. Relation composition is

$$L \circ R = \{(t, s) : \exists u \ (t, u) \in L \wedge (u, s) \in R\}.$$

All relation-returning public support functions return immutable `frozenset` relations.

Public API	Exact signature	Contract
`support_product`	`support_product(left: Iterable[Address], right: Iterable[Address]) -> frozenset[Address]`	Relation product `left` after `right`
`support_power`	`support_power(support, exponent: int) -> frozenset[Address]`	Repeated relation product
`support_polynomial`	`support_polynomial(support, coefficients, *, vertex_count: int) -> frozenset[Address]`	Union of supports for nonzero polynomial coefficients
`transitive_closure`	`transitive_closure(vertex_count, support, *, include_identity=True) -> frozenset[Address]`	Finite reachability relation
`inverse_support_if_acyclic`	`inverse_support_if_acyclic(vertex_count, support) -> frozenset[Address]`	Closure inclusion for an acyclic unit-diagonal inverse
`schur_fill_support`	`schur_fill_support(retained, hidden, support) -> frozenset[Address]`	Direct retained edges plus retained-hidden-retained paths
`topological_levels`	`topological_levels(vertex_count, support) -> tuple[tuple[int, ...], ...]`	Sorted executable DAG layers
`longest_path_vertices`	`longest_path_vertices(vertex_count, support) -> int`	Maximum vertex count on a directed path

`support_power(S, 0)` has no ambient vertex count. It returns identity edges only for vertices occurring in `S`, so the zero power of empty support is empty. By contrast, `support_polynomial(..., vertex_count=q)` inserts all `(v,v)` for `0 <= v < q` when the constant coefficient is nonzero, and `PacketIR.power(0)` builds the full packet identity.

For a polynomial `p(A)=sum_k c_k A^k`, the computed relation is an upper inclusion:

$$\Sigma(p(A)) \subseteq \bigcup_{k:c_k\neq 0} \Sigma(A)^k.$$

The inclusion can be strict because sums of block paths can cancel. For a strict acyclic graph on `q` vertices,

$$(I-N)^{-1} = I + N + \cdots + N^{q-1}, \qquad \Sigma((I-N)^{-1}) \subseteq \Sigma(N)^*.$$

The constructor is

```
SupportGraph(vertex_count: int, support: Iterable[Address])
```

`SupportGraph` exposes `targets(source)`, `sources(target)`, `reachable(sources)`, the property `is_acyclic`, and `closure(*, include_identity=True)`. Self-loops are ignored when testing acyclicity and building topological levels. Strict directed cycles are rejected.

Evidence: `src/komorebi_math/algorithms/support.py`, `tests/test_algorithms_support.py`, and the exhaustive four-vertex closure and random Schur-fill groups in `verification/block3_deep_audit.py`.

28.4 Complexity and memory models

For dense active packet blocks, batch size `r`, and scalar size `b` bytes, the declared models are

$$F_{\text{packet}} = 2r \sum_{(t,s)\in\Sigma} m_t n_s, \qquad S_{\text{packet}} = \sum_{(t,s)\in\Sigma} m_t n_s,$$

$$B_{\text{stream}} = b \sum_{(t,s)\in\Sigma} \left(m_t n_s + n_s r + 2 m_t r\right).$$

The traffic formula is a one-pass, no-cache-reuse model. It is not peak memory and does not include Python objects, address maps, executor setup, FFT work buffers, or dense materialization. Arithmetic intensity and the elementary roofline lower bound are

$$I = F/B, \qquad T_{\min} = \max(F/P_{\max}, B/W_{\max}).$$

These are **E/M** under the stated model, not observed runtime claims.

Public API	Exact signature
`dense_apply_flops`	`dense_apply_flops(rows: int, columns: int | None = None, rhs: int = 1) -> int`
`packet_apply_flops`	`packet_apply_flops(source_layout, target_layout, support, rhs: int = 1) -> int`
`packet_dense_storage_entries`	`packet_dense_storage_entries(source_layout, target_layout, support) -> int`
`block_diagonal_low_rank_storage_entries`	`block_diagonal_low_rank_storage_entries(layout, rank: int) -> int`
`block_diagonal_low_rank_flops`	`block_diagonal_low_rank_flops(layout, rank: int, rhs: int = 1) -> int`
`packet_streaming_bytes`	`packet_streaming_bytes(source_layout, target_layout, support, *, rhs: int = 1, itemsize: int = 8) -> int`
`arithmetic_intensity`	`arithmetic_intensity(flops: float, bytes_moved: float) -> float`
`roofline_time_lower_bound`	`roofline_time_lower_bound(flops, bytes_moved, peak_flops_per_second, bandwidth_bytes_per_second) -> float`

Public API	Exact signature
`packet_cost_estimate`	`packet_cost_estimate(source_layout, target_layout, support, *, rhs=1, itemsize=8, peak_flops_per_second=None, bandwidth_bytes_per_second=None) -> CostEstimate`

`CostEstimate` has fields `flops`, `storage_entries`, `streaming_bytes`, `arithmetic_intensity`, `roofline_seconds`, and `metadata`. Its metadata records `rhs`, `itemsize`, and `active_blocks`. Duplicate addresses are normalized. Both device rates must be supplied together. Rates must be finite and positive.

Evidence: `src/komorebi_math/algorithms/complexity.py` and `tests/test_algorithms_complexity_benchmarks.py`.

28.5 Benchmark contracts and observational timing discipline

Timing scopes are `TimingScope.KERNEL`, `DEVICE_RESIDENT`, and `END_TO_END`. A comparison contract is

$$\mathfrak{B} = (\text{case signature}, \text{output contract}, \text{timing scope}, \text{backend}).$$

`BenchmarkContract.create(case_signature, output_contract, timing_scope, backend)` stringifies and sorts signature keys. Collisions after string conversion are rejected. `matches(other, *, allow_backend_difference=True)` requires the same case signature, output contract, and timing scope. Backend equality is optional by that explicit flag.

Type/function	Exact signature or fields
`BenchmarkResult`	`contract`, `samples_seconds`, `median_seconds`, `minimum_seconds`, `maximum_seconds`, `value`
`BenchmarkComparison`	`baseline`, `candidate`, `speedup`
`benchmark_callable`	`benchmark_callable(function, contract, *, warmups=2, repeats=7, synchronize=None) -> BenchmarkResult`
`compare_benchmarks`	`compare_benchmarks(baseline, candidate, *, allow_backend_difference=True) -> BenchmarkComparison`

The benchmark helper runs untimed warmups, synchronizes after each warmup when a callback is supplied, and surrounds every timed call with synchronization. It reports the median and uses

$$\text{speedup} = \tilde{t}_{\text{baseline}} / \tilde{t}_{\text{candidate}}.$$

If the candidate median is nonpositive, the reported speedup is infinity. Timings remain environment-bound observations (**H/O**). The helper does not make them portable performance guarantees.

Evidence: `src/komorebi_math/algorithms/benchmarks.py` and `tests/test_algorithms_complexity_benchmarks.py`.

29. Structured and low-rank operators

Every concrete structured operator accepts vector and two-dimensional RHS input, preserves vector-versus-matrix rank, and supplies `shape`, `dtype`, `apply`, `materialize`, `adjoint`, `storage_entries`, and `flop_count`. `StructuredOperator.solve(value)` is the default dense square solve. `certificate_residual(value, result)` returns `||apply(value)-result||` after normalizing both arguments as RHS matrices.

29.1 Dense, identity, diagonal, block-diagonal

Constructor	Representation	Stored entries	Modeled apply flops for `r` RHS
`DenseOperator(matrix)`	Explicit `m x n` matrix	`mn`	`2mnr`
`IdentityOperator(dimension, scalar=1.0)`	`scalar * I_n`	`1`	`nr`
`DiagonalOperator(diagonal)`	`diag(d)`	`n`	`nr`
`BlockDiagonalOperator(blocks, names=None)`	Direct sum of square structured blocks	Sum of block storage	Sum of block flop models

`DenseOperator` requires a two-dimensional array and copies it. `IdentityOperator` requires a positive integer dimension. `solve` raises `np.linalg.LinAlgError` when `scalar == 0`. `DiagonalOperator` requires a nonempty vector and rejects solve when any diagonal entry equals zero. `BlockDiagonalOperator` requires at least one square block, creates a `PacketLayout` from their sizes, and exposes `block(packet)` by index or name. Its apply and solve paths split the RHS, execute each block, and merge the results. **E/N**, depending on the scalar dtype and called NumPy operation.

The exact public constructor and common method signatures are:

```
DenseOperator(matrix: np.ndarray)
IdentityOperator(dimension: int, scalar: complex = 1.0)
DiagonalOperator(diagonal: np.ndarray)
BlockDiagonalOperator(
    blocks: tuple[StructuredOperator | np.ndarray, ...],
    names: tuple[str, ...] | None = None,
)

operator.apply(value: Any) -> np.ndarray
operator.materialize() -> np.ndarray
operator.adjoint() -> StructuredOperator
operator.storage_entries() -> int
operator.flop_count(rhs: int = 1) -> int
operator.solve(value: Any) -> np.ndarray
```

`as_structured_operator(value)` returns an existing `StructuredOperator` unchanged or wraps `np.asarray(value)` in `DenseOperator`.

Evidence: `src/komorebi_math/algorithms/structured.py` and `tests/test_algorithms_structured.py`.

29.2 Toeplitz, circulant, Kronecker, Walsh-Hadamard

`CirculantOperator(first_column)` uses

$$C_{ij} = c_{(i-j) \bmod n}, \qquad Cx = \mathcal{F}_n^{-1}(\mathcal{F}_n(c) \odot \mathcal{F}_n(x)).$$

It stores `n` scalars and declares `10*n*ceil(log2(max(2,n)))*r` flops. Real operands are passed through `np.real_if_close` after the inverse FFT.

`ToeplitzOperator(first_column, first_row)` may be rectangular. Its corner entries are checked by NumPy `allclose`, not exact equality. Application uses a circulant embedding of length `m+n`. Storage is `m+n-1`, while the modeled work is `10*(m+n)*ceil(log2(max(2,m+n)))*r`. The implementation's one-element zero gap explains why the FFT length is `m+n`, not the minimal `m+n-1`.

For `KroneckerOperator(left, right)` with `left in K^(a x b)` and `right in K^(c x d)`, Fortran-order reshaping implements

$$(L \otimes R)\,\mathrm{vec}_F(X) = \mathrm{vec}_F(RXL^{\mathsf{T}}), \qquad X \in \mathbb{K}^{d \times b}.$$

It stores `ab+cd` factor entries and models, per RHS,

$$2cdb + 2cba$$

operations using the source variable convention `2*c*d*b + 2*c*b*a`.

`WalshHadamardOperator(dimension, normalized=False, dtype_value=float)` requires a positive power-of-two dimension. Its butterfly is

$$(u, v) \mapsto (u + v, u - v).$$

The unnormalized transform satisfies `H^-1 = H/n`. The normalized transform is self-inverse. The source model reports zero stored entries and `n*log2(n)*r` butterfly operations. Normalization promotes an exact integer working dtype to an inexact dtype before division.

Constructor	Shape	Notable members
`CirculantOperator(first_column)`	`(n,n)`	`first_column`
`ToeplitzOperator(first_column, first_row)`	`(len(column), len(row))`	both boundary vectors
`KroneckerOperator(left, right)`	`(a*c,b*d)`	both matrix factors
`WalshHadamardOperator(dimension, normalized=False, dtype_value=float)`	`(n,n)`	`dimension`, `normalized`, `dtype_value`

All four implement the common structured-operator methods listed in 29.1. The FFT and dense NumPy paths are numerical. The representation identities and declared operation counts are exact within their explicit finite models.

Evidence: `src/komorebi_math/algorithms/structured.py`, `tests/test_algorithms_structured.py`, and the `structured_operators` group in `verification/block3_deep_audit.py`.

29.3 Low-rank factors and recompression

$$A \approx UV^*, \qquad \|A - UV^*\| \le \varepsilon.$$

`LowRankOperator(left, right)` represents

$$A = UV^*, \qquad U \in \mathbb{K}^{m\times k}, \quad V \in \mathbb{K}^{n\times k}, \quad \operatorname{rank}(A) \le k.$$

The factors must be matrices with equal column counts. `k=0` is allowed. Application is `U @ (V.conj().T @ x)`, storage is `k(m+n)`, and modeled work is `2kr(m+n)`. `rank_bound`, `shape`, and `dtype` are properties.

The recompression APIs are

```
recompress_core(left, core, right, rank, *, tolerance=1e-10)
    -> RecompressionResult
recompress_factors(left, right, rank, *, tolerance=1e-10)
    -> RecompressionResult
```

For `A=L C R*`, reduced QR and a small SVD give

$$L = Q_L R_L, \quad R = Q_R R_R, \quad R_L C R_R^* = U\Sigma V^*,$$

$$\widetilde{U} = Q_L U_k \Sigma_k^{1/2}, \qquad \widetilde{V} = Q_R V_k \Sigma_k^{1/2}.$$

`RecompressionResult` contains `operator`, `singular_values`, `retained_rank`, `spectral_error`, `frobenius_error`, and `certificate`, plus `left`, `right`, and `materialize()`. In exact arithmetic the tail values are

$$\|A - \widetilde{U}\widetilde{V}^*\|_2 = \sigma_{k+1}, \qquad \|A - \widetilde{U}\widetilde{V}^*\|_F = \left(\sum_{j>k} \sigma_j^2\right)^{1/2}.$$

The returned factors and tails are NumPy numerical realizations. The certificate has `Exactness.NUMERICAL` and checks the actual spectral residual against `spectral_error + tolerance*max(1, ||A||_2)`. Empty factors return a valid rank-zero correction. **N**

Evidence: `src/komorebi_math/algorithms/low_rank.py`, `tests/test_algorithms_low_rank.py`, and the `low_rank` group in `verification/block3_deep_audit.py`.

29.4 Product-chain error accumulation

`BlockDiagonalLowRankOperator(diagonal, correction)` represents

$$A = D + UV^*, \qquad D = \bigoplus_s D_s.$$

The correction must match the square diagonal shape. The complete member surface is:

Member	Exact signature or value
constructor	`BlockDiagonalLowRankOperator(diagonal: BlockDiagonalOperator, correction: LowRankOperator)`
`from_factors`	`from_factors(diagonal, left, right) -> BlockDiagonalLowRankOperator`
properties	`shape`, `dtype`, `rank_bound`
execution	`apply(value)`, `materialize()`, `adjoint()`
models	`storage_entries()`, `flop_count(rhs=1)`
solve	`solve(value) -> np.ndarray`
product	`exact_product(right) -> BlockDiagonalLowRankOperator`
recompression	`recompress(rank, *, tolerance=1e-10) -> tuple[BlockDiagonalLowRankOperator, RecompressionResult]`

For dense square diagonal blocks of sizes `n_s`, its declared representation models are

$$S = \sum_s n_s^2 + 2nk, \qquad F = 2r \sum_s n_s^2 + 4nkr.$$

The object methods instead sum the actual structured block storage and flop models, so these closed forms apply only when each diagonal block is dense.

The Woodbury solve is

$$(D + UV^*)^{-1}b = D^{-1}b - D^{-1}U(I + V^*D^{-1}U)^{-1}V^*D^{-1}b.$$

Each diagonal block and the small middle matrix must be solvable. NumPy `LinAlgError` is allowed to propagate when either condition fails. `exact_product` constructs an unrecompressed BDLR representation before any requested recompression.

For factor lists in application order, `product_chain_error_bound(exact_factors, approximate_factors)` implements the telescoping spectral-norm bound

$$\left\|\widetilde{A}_{q-1}\cdots\widetilde{A}_0 - A_{q-1}\cdots A_0\right\|_2 \le \sum_{j=0}^{q-1}\left(\prod_{i>j}\|\widetilde{A}_i\|_2\right)\|\widetilde{A}_j - A_j\|_2\left(\prod_{i<j}\|A_i\|_2\right).$$

`accumulated_recompression_bound(operator_norms, local_errors, *, initial_error=0.0)` implements the recurrence

$$E_j \le \|A_j\|E_{j-1} + e_j.$$

Both helpers validate dimensions or finite nonnegative scalar inputs. They return bounds, not measured errors.

For a first recompression experiment, keep the original factors, compare the reconstructed operator with that reference, and read the reported retained rank together with the spectral and Frobenius errors. The two error fields summarize different norms; choose the one used by the downstream estimate. When several approximations are composed, use the product-chain bound rather than treating the largest single-step error as the error of the whole chain.

Runnable shipped example (`examples/low_rank_recompression.py`) produced:

```
solve residual 0.0
product rank bound 4
compressed rank 2
compression spectral error 0.0002447567526018822
```

The final number is seed- and NumPy-environment-specific numerical output, not a universal error constant.

Evidence: `src/komorebi_math/algorithms/low_rank.py`, `src/komorebi_math/algorithms/complexity.py`, and `examples/low_rank_recompression.py`.

29.5 Restricted structure discovery

Structure discovery is limited by design to a caller-supplied packet layout. It does not infer a universal representation.

```
discover_packet_support(
    matrix: Any,
    layout: PacketLayout,
    *,
    absolute_tolerance: float = 0.0,
    relative_tolerance: float = 1e-12,
) -> SupportDiscoveryResult
```

It computes

$$\theta = \tau_a + \tau_r \|A\|_2, \qquad \widehat{\Sigma} = \{(t,s) : \|A_{ts}\|_2 > \theta\}.$$

`SupportDiscoveryResult` contains `packet_ir`, every `block_norms` value, `threshold`, and a heuristic `certificate`. The certificate is always marked valid after successful input validation, has `Exactness.HEURISTIC`, reports `||A-materialize(packet_ir)||_2`, and leaves the application-level negligibility of discarded blocks unchecked. A block at the threshold is discarded, so support can change discontinuously under small perturbations. **H/N**

```
discover_block_diagonal_low_rank(
    matrix: Any,
    layout: PacketLayout,
    rank: int,
    *,
    tolerance: float = 1e-10,
) -> BDLRDiscoveryResult
```

This routine fixes the packet diagonal `D`, forms `R=A-D`, and truncates the NumPy SVD of `R` to the requested rank. `BDLRDiscoveryResult` contains `operator`, `singular_values`, `spectral_error`, `frobenius_error`, and a numerical `certificate`. The rank-constrained SVD theorem applies to the fixed residual and fixed layout. The floating realization is **N**, not exact symbolic structure discovery.

Evidence: `src/komorebi_math/algorithms/discovery.py`, `tests/test_algorithms_discovery_gpu.py`, and the `discovery` group in `verification/block3_deep_audit.py`.

30. Krylov, precision, compilation, and scheduling

The Krylov routines accept matrix-free callables. They check the dimensions and scalar pairings encountered during the run, but they cannot infer global linearity, Hermitian symmetry, or positive definiteness from finitely many generated directions.

30.1 Krylov subspaces and conjugate gradients

For nonzero `b in K^n`,

$$\mathcal{K}_k(A, b) = \operatorname{span}\{b, Ab, \dots, A^{k-1}b\}.$$

```
krylov_basis(
    apply: Callable[[np.ndarray], np.ndarray],
    start: Any,
    steps: int,
    *,
    tolerance: float = 1e-12,
) -> KrylovBasisResult
```

Modified Gram-Schmidt returns `KrylovBasisResult(basis, dimension, breakdown, certificate)`. `steps` must lie in `[1, n]`. A start norm at or below tolerance is rejected. Breakdown occurs when a generated vector has norm at or below tolerance after orthogonalization. The certificate checks

$$\|Q^*Q - I\|_F \le 100\,\texttt{tolerance}.$$

The conjugate-gradient API is

```
conjugate_gradient(
    apply,
    right_hand_side,
    *,
    precondition=None,
    initial=None,
    relative_tolerance=1e-10,
    absolute_tolerance=0.0,
    max_iterations=None,
    curvature_tolerance=1e-14,
) -> ConjugateGradientResult
```

It declares `Ax=b` with `A=A*>0` and, when supplied, a positive-definite preconditioner action. The stopping threshold is

$$\tau_{\text{stop}} = \max(\tau_{\text{abs}}, \tau_{\text{rel}}\|b\|_2).$$

With `z_k=M^{-1}r_k`, it uses

$$\alpha_k = \frac{r_k^* z_k}{p_k^* A p_k}, \quad x_{k+1} = x_k + \alpha_k p_k, \quad r_{k+1} = r_k - \alpha_k A p_k,$$

$$\beta_k = \frac{r_{k+1}^* z_{k+1}}{r_k^* z_k}, \qquad p_{k+1} = z_{k+1} + \beta_k p_k.$$

If `max_iterations=None`, the limit is `10*n`. Generated pairings must be real to the configured curvature scale and positive above the numerical floor. Violations raise `AssumptionError`. `ConjugateGradientResult` contains `solution`, `residual_norms`, `iterations`, `converged`, and `certificate`. The certificate compares a freshly evaluated final residual with the stop threshold, also reports the recursive residual, and explicitly leaves global Hermitian positive definiteness unchecked. Reaching the iteration limit returns a nonvalid, nonconverged result rather than raising. **N/C**

A returned solution array is therefore only the start of the decision. Inspect `converged`, the fresh final residual, and the certificate before using it. If the iteration cap was reached, first check the operator and preconditioner assumptions and the requested tolerance. Increasing the cap does not repair a violated positive-definiteness hypothesis.

Evidence: `src/komorebi_math/algorithms/krylov.py` and `tests/test_algorithms_krylov.py`.

30.2 Block Jacobi certificates

`lanczos_tridiagonal(apply, dimension, steps, *, start=None, seed=0, tolerance=1e-12)` performs fully reorthogonalized finite Lanczos iteration. It returns `LanczosResult(tridiagonal, basis, alpha, beta, breakdown, certificate)`. The certificate checks

$$r_{\text{orth}} = \|Q^*Q - I\|_F, \qquad r_{\text{tri}} = \|Q^*AQ - T\|_F,$$

against `1000*tolerance`. A materially complex `q* A q` raises `AssumptionError`. Eigenvalues of the returned `T` are Ritz values. The API does not certify global eigenvalue accuracy. **N/C**

For a square matrix and packet layout, define

$$D = \text{blockdiag}(A), \qquad E = D^{-1/2}(A - D)D^{-1/2}, \qquad \eta = \|E\|_2.$$

If `A` is Hermitian, every packet-diagonal block is positive definite, and `eta < 1`, then

$$\kappa(D^{-1/2}AD^{-1/2}) \le \frac{1+\eta}{1-\eta}.$$

API	Exact signature or fields
`block_jacobi_certificate`	`block_jacobi_certificate(matrix, layout, *, tolerance=1e-10) -> BlockJacobiCertificate`
`BlockJacobiCertificate`	`diagonal`, `normalized_coupling`, `eta`, `condition_bound`, `certificate`
`block_jacobi_preconditioner`	`block_jacobi_preconditioner(diagonal: BlockDiagonalOperator) -> Callable`
`preconditioned_condition_bound`	`preconditioned_condition_bound(eta: float) -> float`
`power_perturbation_bound`	`power_perturbation_bound(left_norm, right_norm, difference_norm, exponent) -> float`

The constructor raises when Hermitian residual exceeds tolerance or a packet block has minimum eigenvalue at or below tolerance. If `eta >= 1`, it does not raise: it returns `condition_bound=None` and a nonvalid certificate that records the missing coupling hypothesis. The preconditioner callable applies the exact packet-block solves represented by `diagonal`.

The power helper returns zero for exponent zero and otherwise

$$\|A^k - B^k\| \le k \max(\|A\|, \|B\|)^{k-1} \|A - B\|.$$

Runnable shipped example (`examples/structured_krylov.py`) produced:

```
iterations 4
residual 9.930136612989092e-16
```

Evidence: `src/komorebi_math/algorithms/krylov.py`, `examples/structured_krylov.py`, and the `krylov` group in `verification/block3_deep_audit.py`.

30.3 Continuous and discrete precision allocation

For sensitivities `s_i>0`, logarithmic costs `c_i>0`, and total error budget `T>0`, the continuous model is

$$\min_{\varepsilon_i > 0} \sum_i c_i \log(1/\varepsilon_i) \quad \text{subject to} \quad \sum_i s_i \varepsilon_i \le T.$$

`optimal_precision_tolerances(sensitivities, costs, total_tolerance)` returns

$$\varepsilon_i^* = \frac{T c_i}{s_i \sum_j c_j}.$$

`clipped_precision_allocation(sensitivities, costs, total_tolerance, maxima)` enforces `0 < epsilon_i <= maxima_i` through

$$\varepsilon_i(\lambda) = \min\left(u_i, \frac{c_i}{\lambda s_i}\right),$$

using bracketing followed by 240 bisection steps. When the supplied total tolerance is at least `dot(sensitivities, maxima)`, it returns `maxima.copy()`. These are floating numerical realizations of the declared allocation model.

For finite precision choices:

```
DTypeChoice(name: str, cost: float, error_units: int, dtype: str | None = None)
assign_discrete_precisions(
    options: Sequence[Sequence[DTypeChoice]],
    budget: int,
) -> PrecisionAssignment | None
```

The dynamic program solves exactly, for its finite additive integer model,

$$\min_{d_i \in \mathcal{D}_i} \sum_i c_{i,d_i} \quad \text{subject to} \quad \sum_i e_{i,d_i} \le B.$$

It returns `None` when infeasible. `PrecisionAssignment` contains `names`, `total_cost`, `error_units`, and an exact certificate. Ties are resolved by the tuple ordering `(total_cost, error_units, names)`.

Additional precision APIs are:

API	Exact signature	Result
`packet_error_bound`	`packet_error_bound(perturbation_norms, source_packet_norms) -> float`	`sum_i error_i*source_norm_i` using `math.fsum`
`precision_certificate`	`precision_certificate(perturbation_norms, source_packet_norms, observed_error, *, tolerance=1e-10) -> Certificate`	Checks observed error against bound plus tolerance
`quantize_array`	`quantize_array(value, dtype) -> np.ndarray`	NumPy cast with complex-partner preservation

API	Exact signature	Result
`quantize_by_packet`	`quantize_by_packet(values, dtypes) -> tuple[np.ndarray, ...]`	One cast per packet
`quantization_error`	`quantization_error(value, dtype, *, norm_order=2) -> float`	Norm of cast error

Complex input cast toward `float16` or `float32` becomes `complex64`. Complex input cast toward `float64` becomes `complex128`. Other target dtypes are left to NumPy's dtype behavior. This prevents silently discarding the imaginary part for the named real floating targets.

The shipped precision/scheduling example produced:

```
precision choices ('low64', 'high32')
total cost 4.0
schedule makespan 5.0
critical path ('analyze', 'high', 'synthesize')
```

Evidence: `src/komorebi_math/algorithms/precision.py`, `tests/test_algorithms_precision.py`, `examples/precision_schedule.py`, and the `precision` group in `verification/block3_deep_audit.py`.

30.4 Exact finite tree compilation

At a node `v`, the compiler compares a keep option `(c_v,e_v)` with split overhead `s_v` plus its children. The public immutable records are:

```
TreeNode(
    name: str,
    keep_cost: float,
    keep_error: float,
    split_cost: float = 0.0,
    children: tuple[TreeNode, ...] = (),
)

TreePlan(
    cost: float,
    error: float,
    leaves: tuple[str, ...],
    decisions: tuple[tuple[str, str], ...],
    objective: float | None = None,
    quantized_error: int | None = None,
    certificate: Certificate | None = None,
)
```

`TreeNode.terminal` is a property and `names()` recursively lists the topology. Names must be nonempty and unique throughout the tree. All model values must be finite and nonnegative. `TreePlan.decision(node)` returns `"keep"`, `"split"`, or `None`.

`compile_tree_lagrangian(root, penalty)` implements

$$J_v^{\text{keep}} = c_v + \lambda e_v, \qquad J_v^{\text{split}} = s_v + \sum_{u \in \text{ch}(v)} J_u^*, \qquad J_v^* = \min(J_v^{\text{keep}}, J_v^{\text{split}}).$$

It returns the exact minimizer of the supplied finite additive floating-value model. Tie order is `(objective, cost, error, leaves)`.

`compile_tree_budget(root, budget, *, error_unit=1.0)` quantizes every selected leaf independently,

$$q_v = \lceil e_v/u - 10^{-14} \rceil, \qquad \sum_{v \in L} q_v \le B,$$

and returns the minimum-cost feasible plan or `None`. The small `1e-14` offset is part of the source behavior at representational boundaries. This compiler does not round the final real error sum once.

Evidence: `src/komorebi_math/algorithms/compiler.py` and `tests/test_algorithms_compiler.py`.

30.5 Pareto frontiers and task scheduling

`pareto_frontier(root, *, tolerance=0.0, max_plans=100000)` enumerates nondominated `(cost,error)` plans. Zero tolerance gives the exact frontier of the supplied Python floating values. Positive tolerance changes dominance and tie semantics. `max_plans` guards the Cartesian product count at each split and raises `ValidationError` before an oversized combination is materialized.

The remaining tree utilities are:

API	Exact signature or fields
`combine_tree_objectives`	`combine_tree_objectives(roots, weights) -> TreeNode`
`DecisionMargin`	`node`, `keep_objective`, `split_objective`, `margin`, `chosen`
`decision_margins`	`decision_margins(root, penalty) -> tuple[DecisionMargin, ...]`
`replace_tree_nodes`	`replace_tree_nodes(root, updates) -> TreeNode`

Objective trees must have identical names and child counts at every node, and weights must be finite, nonnegative, and have positive total. Updates may name only existing nodes and fields `keep_cost`, `keep_error`, and `split_cost`.

The shipped compiler example produced:

```
Lagrangian leaves ('left', 'right')
budget leaves ('left', 'right')
Pareto points [(9.0, 0.0, ('root',)), (7.5, 0.30000000000000004, ('left', 'right')), (5.2, 0.8999999999999999,
    ↪ ('left', 'right_low', 'right_high'))]
```

The printed decimal tails are ordinary binary floating-point representations.

For scheduling, `Task(name, duration=1.0, dependencies=(), shape_key=None)` declares a finite task DAG. Names and dependency names must be nonempty strings. Dependencies must be unique and cannot include the task itself. Durations must be finite and nonnegative. Shape keys must be hashable.

Scheduling API	Exact signature or fields
`ScheduledTask`	`name`, `start`, `finish`, `level`
`TaskSchedule`	`tasks`, `makespan`, `critical_path`, `levels`. Method `by_name()`
`schedule_unlimited`	`schedule_unlimited(tasks: Sequence[Task]) -> TaskSchedule`
`shape_classes`	`shape_classes(tasks: Iterable[Task]) -> dict[Hashable | None, tuple[str, ...]]`
`batched_launch_count`	`batched_launch_count(tasks, *, batching_available=True) -> int`
`verify_schedule`	`verify_schedule(tasks, schedule, *, tolerance=1e-12) -> bool`
`amortization_threshold`	`amortization_threshold(setup_time, baseline_time, candidate_time) -> int | None`
`BreakEvenResult`	`setup_time`, `baseline_time`, `candidate_time`, `reuse_count`, `asymptotic_gain`
`break_even_analysis`	`break_even_analysis(setup_time, baseline_time, candidate_time) -> BreakEvenResult`

Scheduling API	Exact signature or fields

Under unlimited concurrency,

$$t_v^{\text{start}} = \max_{u \to v} t_u^{\text{finish}}, \qquad t_v^{\text{finish}} = t_v^{\text{start}} + d_v.$$

The makespan is the weighted critical-path length for the declared durations. This schedule is exact for that model. It does not model worker limits, launch contention, transfers, or resource occupancy. Shape batching reports the ideal launch count `number of distinct shape_key values` only when batching is available. Otherwise it reports the number of tasks. **E/M**

For setup `S`, baseline per-use time `b`, and candidate per-use time `c`, the strictly profitable reuse count is

$$q_{\min} = \left\lfloor \frac{S}{b - c} \right\rfloor + 1$$

when `b>c`, bounded below by one. If `b<=c`, there is no finite threshold and the result is `None`.

Evidence: `src/komorebi_math/algorithms/compiler.py`, `src/komorebi_math/algorithms/scheduling.py`, `examples/packet_compiler.py`, `examples/precision_schedule.py`, and the `compiler` and `scheduling` groups in `verification/block3_deep_audit.py`.

30.6 NumPy and optional CuPy execution

```
NumpyPacketExecutor(packet_ir: PacketIR)
CupyPacketExecutor(packet_ir: PacketIR)
packet_executor(packet_ir: PacketIR, backend: str = "numpy")
    -> NumpyPacketExecutor | CupyPacketExecutor
cupy_available() -> bool
```

`NumpyPacketExecutor.apply(value)` returns `DeviceExecutionResult(value, backend="numpy", returned_to_host=True, certificate=...)` and delegates to `PacketIR.apply`.

`CupyPacketExecutor` imports CuPy only when constructed and materializes and transfers every stored block once during construction. Its method

```
apply(value, *, return_host: bool = True, input_on_device: bool = False)
    -> DeviceExecutionResult
```

distinguishes input transfer from execution and output transfer. With `return_host=False`, the value remains a CuPy array. `synchronize()` calls the null CUDA stream's synchronization method.

`cupy_available()` checks module discoverability with `find_spec`. It does not prove import success, a working CUDA runtime, a visible device, execution correctness, or performance. `packet_executor` accepts backend names case-insensitively for `numpy` and `cupy`. Other names raise `ValidationError`. There is no silent CPU fallback for a requested CuPy executor. **O**

The recorded audit environment reported `cupy_available() == False`, so it verified the unavailable-backend boundary but did not execute a GPU kernel. Separately, the repository's committed CUDA qualification report records a prior Windows qualification with a 4/4 CUDA gate on an RTX 3070 Ti Laptop GPU and CuPy 14.1.1. That historical environment record is not a timing claim for the recorded audit host.

Evidence: `src/komorebi_math/algorithms/gpu.py`, `tests/test_algorithms_discovery_gpu.py`, `docs/GPU_VALIDATION.md`, and `release/BLOCK17_RELEASE_REPORT.md`.

30.7 Restricted structure discovery, discovery certificates, and false-positive controls

Discovery output must be read in this order:

1. The caller fixes the packet layout.
2. The routine computes finite block norms or a residual SVD.
3. The certificate reports the observed residual and its exactness class.
4. The caller decides whether the threshold or rank is meaningful for the application.

`discover_packet_support` uses a strict norm threshold and labels its certificate `HEURISTIC`. It does not call discarded blocks exact zeros. `discover_block_diagonal_low_rank` certifies a numerical SVD truncation only after fixing the block diagonal. Neither routine searches over layouts, identifies an unknown algebraic family, or protects a near-threshold decision from changing under perturbation. These limitations are the false-positive controls: the API exposes norms, thresholds, residuals, and unchecked assumptions rather than upgrading observed smallness to structure.

The certificate/member tables and exact signatures are in 29.5.

Evidence: `src/komorebi_math/algorithms/discovery.py`, `docs/BLOCK_3_RELEASE.md`, and `tests/test_algorithms_discovery_gpu.py`.

30.8 GPU runtime status, device execution, and hardware qualification

For this algorithms surface, runtime status has several distinct levels:

Level	What establishes it	What it does not establish
Module discoverable	`cupy_available()`	Import success or CUDA usability
Executor constructed	CuPy import plus block transfers complete	Synchronized timing or CPU parity
Device call returned	`CupyPacketExecutor.apply` returns	Timing validity. Its certificate leaves synchronization unchecked
Qualified hardware run	External gate checks runtime, device, synchronized execution, and parity	Performance portability to another host

Use a matched `BenchmarkContract` and pass `executor.synchronize` to `benchmark_callable` before presenting device timing. Compare CPU and GPU values before comparing their medians, and do not compare kernel-only timing with end-to-end timing. The source executor certificate is an execution record, not a hardware qualification record.

Failure and boundary checklist for Chapters 28 through 30

Boundary	Source behavior
Empty layout, nonpositive/noninteger sizes, duplicate names	`ValidationError` or `DimensionError`
Wrong packet leading dimension or mixed RHS ranks	`DimensionError`
Malformed/out-of-range IR address or wrong block shape	`ValidationError` / `DimensionError`
Negative or nonfinite thresholds/tolerances	`ValidationError`
Incompatible composition layouts. Nonsquare power	`DimensionError`
Negative/noninteger exponent or RHS count	`ValidationError`
Singular structured solve	NumPy `LinAlgError`
Toeplitz corner outside NumPy `allclose`	`ValidationError`
Non-power-of-two Walsh dimension	`ValidationError`
Low-rank factor/core mismatch or noncomposable product chain	`DimensionError`
Cyclic inverse/topological request or invalid graph partition	`ValidationError`
Zero Krylov start or invalid step count	`ValidationError`
Generated non-Hermitian/negative CG or Lanczos pairing	`AssumptionError`

Boundary	Source behavior
Block Jacobi `eta >= 1`	Returned nonvalid certificate, not an exception
Empty discrete option list or invalid dtype/budget	`ValidationError`
Infeasible discrete assignment or tree budget	`None`
Duplicate tree/task names, unknown updates/dependencies, DAG cycle	`ValidationError`
Pareto combination count above `max_plans`	`ValidationError`
Unknown executor backend	`ValidationError`
CuPy not installed	`BackendUnavailableError` on CuPy executor construction

Evidence: the validation branches in all thirteen `src/komorebi_math/algorithms/*.py` public modules and the ten focused test modules listed below.

30.9 Verification evidence and known defects

All commands below were run from the recovered reference repository with `PYTHONPATH=src` and the available `.venv-guide` Python environment.

Executed item	Observed result
Focused tests: `test_algorithms_layout_ir.py`, `support.py`, `complexity_benchmarks.py`, `structured.py`, `low_rank.py`, `krylov.py`, `precision.py`, `compiler.py`, `scheduling.py`, `discovery_gpu.py`	`79 passed in 1.22s`
`verification/block3_deep_audit.py`	`25,958` checks, `0` failures
`examples/packet_ir.py`	Completed. Output reproduced in 28.2
`examples/low_rank_recompression.py`	Completed. Output reproduced in 29.4
`examples/structured_krylov.py`	Completed. Output reproduced in 30.2
`examples/precision_schedule.py`	Completed. Output reproduced in 30.3
`examples/packet_compiler.py`	Completed. Output reproduced in 30.5
Local CuPy probe	`cupy_available() == False`. No device execution performed

The deep audit independently covered 1,000 PacketIR checks, 270 structured operator checks, 820 low-rank checks, 5,596 support checks, 792 Krylov checks, 7,787 precision checks, 3,633 compiler checks, 4,560 scheduling checks, and 1,500 discovery checks. Those counts come from the observed JSON printed by the verifier. They are not inferred from test collection.

No implementation failure was observed in the focused tests, examples, or deep audit. The recorded audit identified the following documentation and runtime boundaries:

- `docs/API_INVENTORY.md` lists `PacketBlock` under the algorithms layout/IR surface, but the current `algorithms.__all__` and source modules export no `PacketBlock`. Use `PacketIR` blocks as `StructuredOperator` instances.
- CuPy is absent on the recorded audit host. The local run therefore provides no new GPU correctness or performance evidence, even though a separate committed CUDA qualification report records prior qualified hardware execution.
- Threshold discovery, finite-direction CG/Lanczos checks, roofline numbers, ideal batching, and benchmark medians retain the heuristic, conditional, modeled, or observational boundaries stated above. These are contract limitations, not promoted theorem claims.

Chapters 28 through 30 summary. `PacketLayout` and `PacketIR` make packet coordinates, block addresses, execution, support, storage, and arithmetic explicit. Structured primitives and low-rank factors avoid unnecessary dense storage while retaining a materialized reference path. Krylov certificates validate generated numerical evidence without inferring global hypotheses. Precision and tree compilers are exact only for their declared finite models. Schedules and roofline estimates are resource models. Discovery remains fixed-layout and threshold-aware. GPU claims require an actually qualified runtime and matched timing scope.

31. Signal conventions, windows, and framing

Komorebi Math's signal layer is a finite-array API. In this chapter, a signal is a nonempty one-dimensional NumPy-compatible array

$$x = (x_0, \dots, x_{N-1}) \in \mathbb{K}^N, \qquad \mathbb{K} \in \{\mathbb{R}, \mathbb{C}\},$$

and time is always axis 0. Framed arrays use axis 0 for frame number and axis 1 for the sample within a frame. The routines in Chapters 31 and 32 are exported from `komorebi_math.signals`. Their implementation modules are `signals.windows`, `signals.time_frequency`, `signals.spectral`, and `signals.streaming`.

The word *finite* defines the scope of these operations. Window overlap is evaluated on a finite residue system, framing has explicit left and right padding, and inverse STFT checks only the represented finite interval. None of these operations establishes stationarity, an infinite-dimensional frame theorem, stochastic consistency, or real-time performance.

Signal-processing vocabulary. A frame is a short slice of the signal; the hop is the number of samples between successive frame starts. A window weights the samples within a frame. STFT means short-time Fourier transform, and PSD means power spectral density. COLA means constant overlap-add and NOLA means nonzero overlap-add; their precise tests are given in Section 31.2.3. Keep sample counts distinct from physical time and frequency: the sample rate is what converts between those units.

For a first reconstruction, keep the original signal and the full returned transform record. Reuse its window and framing metadata in synthesis, then compare with the retained original. Changing padding or discarding metadata can change the finite problem even when the coefficient array looks familiar.

31.1 Array, axis, dtype, boundary, and signal-first conventions

This signal research surface follows these rules:

- One-dimensional DSP functions have no `axis` argument. A multidimensional array is rejected where the implementation validates dimensionality.
- Signal-first operations elsewhere in the signal package take the signal before the kernel. The Chapter 31 framing and Chapter 32 Fourier operations have no kernel argument.
- `frame_signal` and `stft` call `np.asarray` but do not reject nonfinite samples. `periodogram`, `welch_psd`, `cross_spectrum`, `parseval_certificate`, and streaming STFT do reject nonfinite input. There is no package-wide finiteness rule.
- Named windows are floating arrays. Explicit array windows are copied with their existing dtype. Framing preserves the signal dtype. Overlap-add uses `np.result_type(frames.dtype, window.dtype)`.
- NumPy's FFT implementation determines transform precision. Fourier coefficients are complex. Frequency and time coordinates are floating arrays. Spectrogram and power-spectrum ordinates are stored as floating arrays. Cross power and H1 transfer may be complex, while coherence is floating.
- `overlap_add` and `istft` call `np.real_if_close(..., tol=1000)` after computation. Consequently, a complex result with only roundoff-sized imaginary parts may be returned as real. This is dtype simplification, not a reconstruction certificate.
- Boundary mode is mathematical input. `"zeros"` pads by zero, `"edge"` repeats endpoints, `"reflect"` mirrors without repeating the edge sample, and `"none"` forbids any required padding.
- Real input defaults to one-sided Fourier output. Complex input defaults to two-sided output and is rejected if one-sided output is explicitly requested.

The public type aliases relevant to these conventions are:

Alias	Declared values
`WindowName`	`"boxcar"`, `"rectangular"`, `"hann"`, `"hamming"`, `"blackman"`, `"bartlett"`, `"kaiser"`, `"tukey"`
`WindowSpec`	`WindowName | str | tuple[str, float] | np.ndarray`
`BoundaryMode`	`"zeros" | "reflect" | "edge" | "none"`

Runtime name normalization also recognizes `rect`, `rectangle`, and `ones` as boxcar, `hanning` as Hann, and `triang` and `triangle` as Bartlett. Case is ignored, surrounding whitespace is stripped, and underscores are changed to hyphens.

The only required runtime dependency for these APIs is NumPy (`numpy>=1.26`). The package requires Python 3.11 or newer. SciPy is not used by the implementation. It is optional and appears only in the independent convention-matched reference audit.

31.2 Deterministic windows and window metrics

31.2.1 Deterministic window construction

```
get_window(
    window: WindowSpec,
    length: int,
    *,
    periodic: bool = True,
) -> np.ndarray
```

`get_window` implements theorem record `KM-DSP-D01`, *Finite deterministic analysis window*. `length` must be a positive integer and a Boolean is not accepted as an integer.

For a symmetric window of requested length $N > 1$, let $j = 0, \dots, N-1$, $D = N - 1$, and $u_j = 2j/D - 1$. The named windows are

$$
\begin{aligned}
w_j^{\text{boxcar}} &= 1, \\
w_j^{\text{Hann}} &= \tfrac{1}{2} - \tfrac{1}{2}\cos(2\pi j/D), \\
w_j^{\text{Hamming}} &= 0.54 - 0.46\cos(2\pi j/D), \\
w_j^{\text{Blackman}} &= 0.42 - 0.5\cos(2\pi j/D) + 0.08\cos(4\pi j/D), \\
w_j^{\text{Bartlett}} &= \frac{2}{D}\left(\frac{D}{2} - \left|j - \frac{D}{2}\right|\right), \\
w_j^{\text{Kaiser}}(\beta) &= \frac{I_0\left(\beta\sqrt{1 - u_j^2}\right)}{I_0(\beta)}.
\end{aligned}
$$

For $0 < \alpha < 1$, the Tukey window is

$$
w_j^{\text{Tukey}}(\alpha) = \begin{cases}
\frac{1}{2}\left[1 + \cos\left(\pi\left(\frac{2j}{\alpha D} - 1\right)\right)\right], & j < \alpha D/2, \\
1, & \alpha D/2 \le j \le D(1 - \alpha/2), \\
\frac{1}{2}\left[1 + \cos\left(\pi\left(\frac{2j}{\alpha D} - \frac{2}{\alpha} + 1\right)\right)\right], & j > D(1 - \alpha/2).
\end{cases}
$$

The endpoint cases are exact branches: $\alpha \le 0$ produces boxcar, and $\alpha \ge 1$ produces Hann. The runtime validator restricts Tukey tuples to $0 \le \alpha \le 1$ and Kaiser tuples to finite $\beta \ge 0$. A bare `"kaiser"` uses $\beta = 14$. A bare `"tukey"` uses $\alpha = 0.5$. Only Kaiser and Tukey accept parameter tuples.

With `periodic=True`, the implementation evaluates the corresponding symmetric formula at length $N+1$, then discards its last sample. Thus the cosine denominator is N, not $N-1$. With `periodic=False`, it directly evaluates the symmetric length-N formula. Every named family returns `[1.0]` at $N=1$.

An explicit array is not regenerated: it must already be one-dimensional, have exactly `length` entries, and contain finite values. `get_window` returns a copy.

Cost. Named construction and explicit-window validation take $\Theta(N)$ time and $\Theta(N)$ output storage.

31.2.2 Gain and equivalent-noise-bandwidth metrics

```
window_metrics(window: Any, *, sample_rate: float = 1.0) -> WindowMetrics
```

For a nonempty finite window $w \in \mathbb{K}^N$, define

$$W_0 = \sum_{n=0}^{N-1} w_n, \qquad G_c = \frac{|W_0|}{N}, \qquad G_p = \frac{1}{N}\sum_{n=0}^{N-1} |w_n|^2.$$

The equivalent noise bandwidth is

$$\mathrm{ENBW}_{\mathrm{bins}} = \frac{N \sum_n |w_n|^2}{|W_0|^2}, \qquad \mathrm{ENBW}_{\mathrm{Hz}} = \frac{f_s}{N}\,\mathrm{ENBW}_{\mathrm{bins}}\,.$$

The implementation rejects a coherent sum satisfying

$$|W_0| \le \epsilon_{64} N,$$

not merely an exactly zero sum. `sample_rate` must be positive and finite. This numerical diagnostic is provenance record `KM-DSP-T01`.

`WindowMetrics` is a frozen, slotted record:

Member	Type	Meaning
`length`	`int`	N
`coherent_gain`	`float`	G_c
`power_gain`	`float`	G_p
`equivalent_noise_bandwidth_bins`	`float`	ENBW in DFT bins
`equivalent_noise_bandwidth_hz`	`float`	ENBW in hertz at the declared sample rate

Cost. One pass over the window, $\Theta(N)$ time and $O(1)$ auxiliary scalar storage apart from NumPy temporaries.

31.2.3 Overlap profiles and COLA/NOLA diagnostics

The public calls are:

```
overlap_profile(
    window: Any,
    hop_length: int,
    *,
    squared: bool = True,
) -> np.ndarray

overlap_add_certificate(
    window: Any,
    hop_length: int,
    *,
    squared: bool = True,
    require_constant: bool = False,
    tolerance: float = 1e-12,
) -> Certificate

canonical_dual_window(
    window: Any,
    hop_length: int,
    *,
    tolerance: float = 1e-12,
) -> ComputationResult[np.ndarray]
```

For $1 \leq H \leq N$, the residue profile has length H. `squared=True` computes

$$p_r^{(2)} = \sum_{\substack{0 \leq n < N \\ n \equiv r \pmod{H}}} |w_n|^2, \qquad r = 0, \dots, H-1,$$

whereas `squared=False` computes

$$p_r^{(1)} = \sum_{\substack{0 \leq n < N \\ n \equiv r \pmod{H}}} w_n.$$

The certificate sets

$$p_{\min} = \min_r |p_r|, \quad \delta = \max_r |p_r - p_0|, \quad s = \max\left(1, \max_r |p_r|\right).$$

Its nonzero-overlap condition is the strict inequality $p_{\min} > \tau s$. Its constant-overlap condition is $\delta \leq \tau s$. The certificate is valid when nonzero overlap holds and, if `require_constant=True`, constant overlap also holds. The diagnostic condition number is

$$\kappa_{\text{overlap}} = \frac{\max_r |p_r|}{p_{\min}},$$

or infinity when the nonzero test fails.

This one function can express the usual diagnostics:

- ordinary COLA: `squared=False, require_constant=True`.
- Same-window squared COLA: `squared=True, require_constant=True`.
- NOLA only: `squared=True, require_constant=False`.

The certificate is numerical, has theorem ID `KM-DSP-C01`, records `overlap_spread` and `minimum_overlap`, and leaves `finite-record boundary coverage` unchecked. Its residue calculation describes a uniform infinite shift lattice away from a finite record's edges. It does not prove that a particular finite framing covers every requested sample.

When $p_r^{(2)}$ passes the nonzero test, the canonical synthesis window is

$$\widetilde{w}_n = \frac{\overline{w_n}}{p^{(2)}_{n \bmod H}}.$$

The implementation then independently evaluates

$$q_r = \sum_{n \equiv r \pmod{H}} w_n \widetilde{w}_n$$

and accepts the result only if

$$\max_r |q_r - 1| \le \tau \max\left(1, \max_r |q_r|\right).$$

`canonical_dual_window` returns a `ComputationResult[np.ndarray]`, not a bare array. If the squared profile vanishes, its value is a zero array, the residual `dual_overlap` is infinity, and the certificate is invalid. Call `.require()` only when failure should become `ValidationError`. The result's numerical certificate is `KM-DSP-T02`. It too leaves finite-record boundary coverage unchecked.

Cost. Each profile and each dual calculation is $\Theta(N+H)$ time. A profile uses $\Theta(H)$ result storage. A dual additionally uses $\Theta(N)$ output storage.

31.3 Frames and overlap-add

31.3.1 Deterministic finite framing

```
frame_signal(
    signal: Any,
    frame_length: int,
    hop_length: int | None = None,
    *,
    center: bool = False,
    pad_end: bool = False,
    boundary: BoundaryMode = "zeros",
) -> FrameBatch
```

Let the original signal length be N, frame length L, and hop H, with $1 \le H \le L$. The default hop is $H = L$. Centering first assigns

$$p_L = p_R = \lfloor L/2 \rfloor;$$

without centering, both are zero. Let $B = N + p_L + p_R$. If $B < L$, `pad_end=False` raises `DimensionError`. With `pad_end=True`, the right pad is increased until $B = L$.

For $B \ge L$, the number of frames is

$$M = \begin{cases} 1 + \left\lfloor \dfrac{B-L}{H} \right\rfloor, & \texttt{pad_end} = \texttt{False}, \\ \max\left(1, 1 + \left\lceil \dfrac{B-L}{H} \right\rceil\right), & \texttt{pad_end} = \texttt{True}. \end{cases}$$

In the second branch, the right pad is extended by exactly

$$((M-1)H + L) - B$$

so the final frame is complete. Frame starts are

$$s_m = mH, \qquad m = 0, \dots, M-1,$$

and the returned array is

$$F_{m,n} = x^{\mathrm{pad}}_{s_m+n}, \qquad F \in \mathbb{K}^{M\times L}.$$

`pad_end=False` discards an incomplete terminal tail. `pad_end=True` adds only enough right padding to include the final complete frame. `boundary="none"` is valid only when these rules request no actual padding. Reflect padding requires at least two source samples.

`FrameBatch` is a frozen, slotted record:

Member	Shape/type	Meaning
`frames`	`(M, L)` ndarray	copied frame data. Axis 0 is frame, axis 1 is local time
`starts`	`(M,)` integer ndarray	padded-array starts mH
`original_length`	`int`	N
`frame_length`	`int`	L
`hop_length`	`int`	H
`pad_left`, `pad_right`	`int`	actual finite padding
`boundary`	`BoundaryMode`	recorded boundary selection
`frame_count`	property	M
`padded_length`	property	$N + p_L + p_R$
`covered_length`	property	$(M-1)H + L$
`certificate()`	method	exact layout certificate `KM-DSP-D02`

The layout certificate checks that `frames` is two-dimensional with second dimension L, that `starts == arange(M) * H`, and that covered length does not exceed padded length. It records `uncovered_tail = padded_length - covered_length`. It certifies indexing and shape, not equality with a separately retained source signal.

Cost. The implementation materializes every frame with slicing and `np.stack`: $\Theta(ML)$ time and $\Theta(ML)$ result storage. Overlap means samples are copied into more than one row.

31.3.2 Raw overlap-add

```
overlap_add(
    frames: Any,
    hop_length: int,
    *,
    synthesis_window: Any | None = None,
    length: int | None = None,
) -> np.ndarray
```

For nonempty `frames` of shape (M, L), and a synthesis window v of length L,

$$y[n] = \sum_{m=0}^{M-1} F_{m,n-mH} v_{n-mH},$$

where only local indices $0 \le n - mH < L$ contribute. If `synthesis_window=None`, $v \equiv 1$. The represented output length is

$$R = (M-1)H + L.$$

`length` must be a positive integer no greater than R and, when supplied, returns the prefix `y[:length]`.

This primitive performs no overlap normalization. Overlapped samples are sums, so `overlap_add(frame_signal(x, L, H).frames, H)` is generally not x when $H < L$. Normalized analysis/synthesis belongs to `istft` in Chapter 32.

Cost. $\Theta(ML)$ time and $\Theta(R)$ output storage.

31.4 Canonical dual windows

The canonical-dual construction in Section 31.2.3 is the bridge from an infinite-lattice overlap diagnostic to a synthesis window:

$$\widetilde{w}_n = \frac{\overline{w_n}}{p^{(2)}_{n \bmod H}}, \qquad p^{(2)}_r = \sum_{n \equiv r \pmod H} |w_n|^2.$$

Its certificate checks the unit cross-profile and leaves finite-record boundary coverage unchecked. It therefore complements, but does not replace, the local-denominator check performed by `istft` on the actual finite reconstruction interval.

31.5 Complete Chapter 31 API and result reference

Public symbol	Exact call or role	Return
`WindowName`	literal alias	type alias
`WindowSpec`	named, parameterized, or explicit window	type alias
`WindowMetrics`	frozen record constructor	metric record
`get_window`	`get_window(window, length, *, periodic=True)`	`(N,)` ndarray
`window_metrics`	`window_metrics(window, *, sample_rate=1.0)`	`WindowMetrics`
`overlap_profile`	`overlap_profile(window, hop_length, *, squared=True)`	`(H,)` ndarray
`overlap_add_certificate`	`overlap_add_certificate(window, hop_length, *, squared=True, require_constant=False, tolerance=1e-12)`	`Certificate`
`canonical_dual_window`	`canonical_dual_window(window, hop_length, *, tolerance=1e-12)`	`ComputationResult[np.ndarray]`
`BoundaryMode`	finite padding choices	type alias
`FrameBatch`	frozen framing record	record with three properties and `certificate()`
`frame_signal`	`frame_signal(signal, frame_length, hop_length=None, *, center=False, pad_end=False, boundary="zeros")`	`FrameBatch`
`overlap_add`	`overlap_add(frames, hop_length, *, synthesis_window=None, length=None)`	ndarray

31.6 Worked window/framing truth table

This example is directly runnable:

```
import numpy as np
from komorebi_math.signals import (
    canonical_dual_window,
    frame_signal,
    get_window,
    overlap_add,
    overlap_add_certificate,
    overlap_profile,
    window_metrics,
)

np.set_printoptions(precision=6, suppress=True)
w = get_window("hann", 8, periodic=True)
metrics = window_metrics(w, sample_rate=800.0)
cola = overlap_add_certificate(
    w, 4, squared=False, require_constant=True
)
dual = canonical_dual_window(w, 2)

batch = frame_signal(
    np.arange(7.0), 4, 3,
    center=True, pad_end=True, boundary="zeros",
)
raw = frame_signal(np.arange(8.0), 4, 2, boundary="none")

print("window", w)
print("metrics", metrics)
print("linear_profile", overlap_profile(w, 4, squared=False))
print("cola", cola.valid, cola.residuals, cola.assumptions_unchecked)
print("dual", dual.valid, dual.certificate.residuals)
print("starts", batch.starts, "padding", (batch.pad_left, batch.pad_right),
      "shape", batch.frames.shape)
print(batch.frames)
print("raw_ola", overlap_add(raw.frames, 2))
```

Observed with package `1.0.0b4`, Python 3.12.13, and NumPy 2.5.2:

```
window [0.       0.146447 0.5      0.853553 1.       0.853553 0.5      0.146447]
metrics WindowMetrics(length=8, coherent_gain=0.5, power_gain=0.37500000000000006,
   ↪  equivalent_noise_bandwidth_bins=1.5000000000000002, equivalent_noise_bandwidth_hz=150.00000000000003)
linear_profile [1. 1. 1. 1.]
cola True {'overlap_spread': 0.0, 'minimum_overlap': 1.0} ('finite-record boundary coverage',)
dual True {'dual_overlap': 0.0, 'minimum_squared_overlap': 1.5}
starts [0 3 6 9] padding (2, 4) shape (4, 4)
[[0. 0. 0. 1.]
 [1. 2. 3. 4.]
 [4. 5. 6. 0.]
 [0. 0. 0. 0.]]
raw_ola [ 0.  1.  4.  6.  8. 10.  6.  7.]
```

The last line is the intended warning: raw overlap-add sums the two interior copies. It is not normalized synthesis.

31.7 Validation and failure behavior

Operation	Failure	Public behavior
`get_window`	nonpositive/nonintegral length	`ValidationError`
`get_window`	explicit array is not 1-D or has wrong length	`DimensionError`
`get_window`	explicit entries are nonfinite. Invalid tuple/parameter/name	`ValidationError`
`window_metrics`	empty/non-1-D window	`DimensionError`
`window_metrics`	nonfinite entries, nonpositive/nonfinite rate, numerically zero coherent sum	`ValidationError`
overlap APIs	empty/non-1-D window	`DimensionError`
overlap APIs	invalid hop or $H > N$	`ValidationError`
`overlap_add_certificate`	negative/nonfinite tolerance	`ValidationError`
`canonical_dual_window`	vanishing squared profile	invalid `ComputationResult`. `.require()` raises `ValidationError`
`frame_signal`	empty/non-1-D input or short unpadded record	`DimensionError`
`frame_signal`	invalid lengths, $H > L$, forbidden/unknown required padding, one-sample reflection	`ValidationError`
`overlap_add`	empty/non-2-D frames or wrong synthesis-window length	`DimensionError`
`overlap_add`	invalid hop, $H > L$, or invalid requested length	`ValidationError`, except an overlong requested output is `DimensionError`

The implementation defects listed in Section 32.9 qualify this table. In particular, some validation occurs only on the execution branch that needs it.

32. Time-frequency and spectral analysis

Chapter 32 applies NumPy's finite DFT convention to the Chapter 31 framing model.

Choose the output by the question. Use an STFT when frame-by-frame complex coefficients or reconstruction are needed, a spectrogram for the corresponding time-frequency magnitudes or powers, a periodogram for one finite-record spectrum, and Welch averaging for a spectrum assembled from segments. Cross spectra add a second aligned signal. These outputs have different scaling and evidence fields, so retain the result record rather than only its plotted array.

For frame length L, hop H, FFT length $K \geq L$, frame count M, and bin count

$$F = \begin{cases} \lfloor K/2 \rfloor + 1, & \text{one-sided real transform,} \\ K, & \text{two-sided transform,} \end{cases}$$

the STFT is a typed finite map

$$\mathrm{STFT}_{w,H,K} : \mathbb{K}^N \longrightarrow \mathbb{C}^{M \times F}.$$

32.1 STFT and inverse STFT

A global shifted-window convention is

$$X[m,k] = \sum_n x[n] w[n - mH] e^{-2\pi i k n / N}.$$

The implementation instead uses local-frame coordinates and K for the FFT length. Translating the preceding display into that convention requires an index change and an explicit frame-dependent phase. Use the local-frame formula in Section 32.1.1 when reproducing the implementation. Comparing complex coefficients from the two conventions without that translation can produce a phase discrepancy even for the same signal.

32.1.1 STFT analysis and coordinates

```
stft(
    signal: Any,
    *,
    sample_rate: float = 1.0,
    frame_length: int = 256,
    hop_length: int | None = None,
    fft_length: int | None = None,
    window: WindowSpec = "hann",
    center: bool = True,
    pad_end: bool = True,
    boundary: BoundaryMode = "zeros",
    onesided: bool | None = None,
    detrend: DetrendMode = "none",
) -> STFTResult
```

The defaults are $H = \max(1, \lfloor L/4 \rfloor)$, $K = L$, a periodic Hann analysis window, centered framing, terminal padding, and zero boundary extension. `stft` delegates its exact framing to `frame_signal`.

For each local frame,

$$f_m[n] = x^{\mathrm{pad}}[mH + n], \qquad 0 \le n < L.$$

With `detrend="constant"`, it first replaces the frame by

$$f_m[n] - \frac{1}{L}\sum_{j=0}^{L-1} f_m[j].$$

It then computes

$$X[m,k] = \sum_{n=0}^{L-1} f_m[n]w[n]e^{-2\pi ikn/K},$$

implicitly zero-padding the windowed frame when $K > L$. The coefficient layout is `(frame, frequency_bin)`, not `(frequency_bin, frame)`.

One-sided frequencies are NumPy `rfftfreq` coordinates,

$$f_k = \frac{kf_s}{K}, \qquad k = 0, \dots, \lfloor K/2 \rfloor.$$

Two-sided frequencies use NumPy's signed `fftfreq` ordering. Frame times are centers in seconds:

$$t_m = \frac{mH + L/2 - p_L}{f_s}.$$

For odd L, these may lie on half-sample coordinates.

`STFTResult` is a frozen, slotted record:

Member	Shape/type	Meaning
`coefficients`	`(M, F)` complex ndarray	finite Fourier coefficients
`frequencies`	`(F,)` float ndarray	hertz
`times`	`(M,)` float ndarray	frame-center seconds
`sample_rate`	`float`	positive finite f_s
`frame_length`, `hop_length`, `fft_length`	`int`	L, H, K
`analysis_window`	`(L,)` ndarray	actual periodic or explicit analysis window
`original_length`	`int`	original N
`pad_left`, `pad_right`	`int`	actual analysis padding
`onesided`	`bool`	Fourier layout selector
`boundary`	`BoundaryMode`	recorded padding convention
`detrend`	`DetrendMode`	`"none"` or `"constant"`
`frame_count`	property	M
`inverse(...)`	method	delegates to `istft`
`roundtrip_certificate(...)`	method	compares normalized synthesis with a supplied source

The STFT definition has provenance record `KM-DSP-D03`. It returns metadata, not a certificate.

Cost. Framing and windowing cost $\Theta(ML)$. Batched transforms cost $\Theta(MK \log K)$. The result occupies $\Theta(MF + M + F + L)$, while peak materialization also includes frame and windowed-frame arrays of order ML.

32.1.2 Normalized inverse STFT and reconstruction certificates

```
istft(
    transform: STFTResult,
    *,
    synthesis_window: Any | None = None,
    length: int | None = None,
    tolerance: float = 1e-11,
) -> ComputationResult[np.ndarray]

STFTResult.inverse(
    self,
    *,
    synthesis_window: Any | None = None,
    length: int | None = None,
    tolerance: float = 1e-11,
) -> ComputationResult[np.ndarray]

STFTResult.roundtrip_certificate(
    self,
    signal: Any,
    *,
    tolerance: float = 1e-10,
) -> Certificate
```

The inverse rejects a transform produced with constant detrending because the removed frame means were not retained. Otherwise it applies `irfft` to one-sided coefficients or `ifft` to two-sided coefficients, truncates each inverse transform to its first L entries, and uses synthesis window

$$v = \overline{w}$$

by default. A caller may supply another length-L window.

For represented padded coordinate n, define

$$a[n] = \sum_m \hat{f}_m[n - mH]v[n - mH], \qquad d[n] = \sum_m w[n - mH]v[n - mH],$$

where only valid local frame indices contribute. The represented length is

$$R = (M - 1)H + L.$$

The requested output defaults to the original length N, cannot exceed N, and is read from padded indices

$$[p_L, p_L + N_{\text{req}}).$$

On that local interval, set

$$s = \max\left(1, \max_n |d[n]|\right), \qquad d_{\min} = \min_n |d[n]|.$$

Synthesis is valid only under the strict test

$$d_{\min} > \tau s.$$

If valid, $\hat{x}[n] = a[n]/d[n]$. The certificate's normalization residual is

$$r_{\text{norm}} = \max_n |\hat{x}[n]d[n] - a[n]|.$$

If invalid, the routine returns a zero array of the requested length paired with an invalid certificate. `.require()` raises `ValidationError`. Certificate `KM-DSP-T03` proves only the finite normalization identity and explicitly leaves source agreement unchecked.

`roundtrip_certificate` performs the separate check. It requires a one-dimensional source of exactly `original_length`, calls the inverse, and tests

$$r_{\text{rt}} = \|\hat{x} - x\|_2 \le \tau \max(1, \|x\|_2).$$

If inverse synthesis is invalid, it returns that inverse certificate unchanged.

Cost. Inverse FFT plus finite overlap-add takes $\Theta(MK \log K + ML)$ time. Peak storage includes all inverse frames, $\Theta(MK)$, and numerator/denominator arrays of length R.

32.2 Spectrograms

```
spectrogram(
    signal: Any,
    *,
    scaling: SpectrogramScaling = "density",
    **stft_options: Any,
) -> SpectrogramResult
```

`SpectrogramScaling` is `"density" | "spectrum" | "magnitude" | "power"`. The remaining keyword arguments are passed directly to `stft`, so misspelled or unsupported options fail according to Python call semantics.

Let $Z_{mk} = |X[m, k]|$,

$$D_{\text{density}} = f_s \sum_n |w_n|^2, \qquad D_{\text{spectrum}} = \left| \sum_n w_n \right|^2.$$

The four values are

$$S_{mk} = \begin{cases} Z_{mk}, & \text{magnitude}, \\ Z_{mk}^2, & \text{power}, \\ Z_{mk}^2 / D_{\text{density}}, & \text{density}, \\ Z_{mk}^2 / D_{\text{spectrum}}, & \text{spectrum}. \end{cases}$$

For one-sided output, `power`, `density`, and `spectrum` double the interior positive-frequency bins. DC is never doubled. For even K, the Nyquist bin is not doubled. `magnitude` is not doubled. Spectrum scaling rejects a denominator at or below machine epsilon. The density branch does not explicitly guard a zero-energy window. See Section 32.9.

`SpectrogramResult` is a frozen, slotted record:

Member	Shape/type	Meaning
`values`	`(M, F)` float ndarray	requested magnitude or power-like values
`frequencies`	`(F,)` float ndarray	same object coordinates as the transform
`times`	`(M,)` float ndarray	same frame times
`scaling`	`SpectrogramScaling`	applied normalization
`sample_rate`	`float`	f_s
`transform`	`STFTResult`	complete underlying STFT and synthesis metadata

Member	Shape/type	Meaning

The spectrogram definition has provenance record `KM-DSP-D04`. The result contains no certificate. Its cost is the STFT cost plus $\Theta(MF)$ scaling work and output storage.

32.3 Periodogram and Welch PSD

32.3.1 Modified periodogram

```
periodogram(
    signal: Any,
    *,
    sample_rate: float = 1.0,
    window: WindowSpec = "boxcar",
    fft_length: int | None = None,
    detrend: DetrendMode = "constant",
    scaling: SpectralScaling = "density",
    onesided: bool | None = None,
) -> PowerSpectrum
```

`SpectralScaling` is `"density" | "spectrum"`. The periodogram validates a finite, nonempty, one-dimensional signal and uses $K = N$ by default. A named window is constructed with `periodic=False`, unlike STFT and Welch.

Writing $\tilde{x} = x$ for no detrending and $\tilde{x} = x - \bar{x}$ for constant detrending,

$$P_{xx}[k] = \frac{|\mathcal{F}_K(\tilde{x}\, w)[k]|^2}{D_w},$$

where

$$D_w = \begin{cases} f_s \sum_n |w_n|^2, & \text{density}, \\ |\sum_n w_n|^2, & \text{spectrum}. \end{cases}$$

One-sided power is doubled on the same interior-bin rule as the spectrogram. The frequency resolution is

$$\Delta f = \frac{f_s}{K}.$$

The returned numerical certificate `KM-DSP-D05` uses fixed tolerance 10^{-12}. It checks that all ordinates are finite, that the minimum real power is no less than $-10^{-12}s$, and that any imaginary part is no greater than $10^{-12}s$, where $s = \max(1, \max|P|)$. It does not certify a stochastic model.

Cost. $\Theta(K \log K + N)$ time and $\Theta(K)$ transform/result storage.

32.3.2 Welch power spectral estimates

```
welch_psd(
    signal: Any,
    *,
    sample_rate: float = 1.0,
    frame_length: int = 256,
    hop_length: int | None = None,
    fft_length: int | None = None,
    window: WindowSpec = "hann",
    detrend: DetrendMode = "constant",
    scaling: SpectralScaling = "density",
    average: SpectralAverage = "mean",
```

```
    onesided: bool | None = None,
) -> PowerSpectrum
```

`SpectralAverage` is `"mean" | "median"`. Welch framing is uncentered, unpadded, and complete-frame only. It requires $N \geq L$, defaults to $H = \max(1, \lfloor L/2 \rfloor)$, and produces

$$M = 1 + \left\lfloor \frac{N - L}{H} \right\rfloor$$

segments. Its named window is periodic. Each segment is detrended before windowing and transformed using $K = L$ by default. If $Q_m[k]$ is the correspondingly scaled segment periodogram, then

$$\widehat{P}_{xx}[k] = \operatorname{Avg}_{m=0}^{M-1} Q_m[k].$$

Mean averaging is arithmetic mean. Median averaging is the elementwise median and includes no statistical median-bias correction. Certificate `KM-DSP-D06` is the same fixed-tolerance finite nonnegativity/finite-value check as the periodogram certificate. Statistical consistency remains unchecked.

Cost. $\Theta(ML + MK \log K)$ time. Peak arrays include M frames and M spectra, so storage is $\Theta(ML + MF)$.

32.3.3 Power-spectrum record and helpers

`PowerSpectrum` is a frozen, slotted record shared by periodogram and Welch:

Member	Shape/type	Meaning
`frequencies`	`(F,)` float ndarray	hertz in one-sided or NumPy signed order
`power`	`(F,)` float ndarray	density or spectrum ordinates
`sample_rate`	`float`	f_s
`scaling`	`SpectralScaling`	`"density"` or `"spectrum"`
`onesided`	`bool`	Fourier layout
`fft_length`	`int`	K
`segment_count`	`int`	1 for periodogram, M for Welch
`frame_length`, `hop_length`	`int`	segment geometry
`window`	`(L,)` ndarray	actual analysis window
`method`	`str`	`"periodogram"`, `"welch-mean"`, or `"welch-median"`
`certificate`	`Certificate`	finite numerical power check
`frequency_resolution`	property	f_s/K

Two helpers are available:

```
PowerSpectrum.dominant_frequency(
    self, *, ignore_dc: bool = False
) -> float

PowerSpectrum.integrated_power(self) -> float
```

`dominant_frequency` returns the coordinate of the largest stored ordinate, optionally starting its search at bin 1. `integrated_power` returns

$$\sum_k P[k]\Delta f$$

for density scaling and $\sum_k P[k]$ for spectrum scaling. These are integrations of the represented finite bins. The helper adds no modeling certificate.

32.4 Cross spectrum, phase, and coherence

```
cross_spectrum(
    input_signal: Any,
    output_signal: Any,
    *,
    sample_rate: float = 1.0,
    frame_length: int = 256,
    hop_length: int | None = None,
    fft_length: int | None = None,
    window: WindowSpec = "hann",
    detrend: DetrendMode = "constant",
    scaling: SpectralScaling = "density",
    average: SpectralAverage = "mean",
    onesided: bool | None = None,
    regularization: float = 0.0,
) -> CrossSpectrum
```

Input and output records must be finite, one-dimensional, nonempty, and equal in length. Framing, window periodicity, detrending, FFT length, one-sided doubling, and averaging match Welch. For paired segment transforms $X_m[k]$ and $Y_m[k]$,

$$P_{xx}[k] = \mathrm{Avg}_m \frac{|X_m[k]|^2 c_k}{D_w},$$
$$P_{yy}[k] = \mathrm{Avg}_m \frac{|Y_m[k]|^2 c_k}{D_w},$$
$$P_{xy}[k] = \mathrm{Avg}_m \frac{\overline{X_m[k]} Y_m[k] c_k}{D_w},$$

where c_k is the one-sided interior-bin factor or one for two-sided spectra. For median averaging, real and imaginary parts of the complex cross-power segments are median-averaged separately. No median-bias correction is applied.

There is no separate phase field. The represented cross phase is obtained from the stored complex array as `np.angle(report.cross_power)`, that is,

$$\phi_{xy}[k] = \arg P_{xy}[k].$$

Phase at a zero or numerically negligible cross-power bin has no stable inferential meaning. The result record does not attach a phase-validity mask.

Given finite ridge $\rho \geq 0$, the implementation computes

$$\gamma^2_{\mathrm{raw}}[k] = \frac{|P_{xy}[k]|^2}{(P_{xx}[k] + \rho)(P_{yy}[k] + \rho)}$$

only where the denominator is positive and uses zero elsewhere. The returned coherence is

$$\gamma^2[k] = \mathrm{clip}(\gamma^2_{\mathrm{raw}}[k], 0, 1).$$

The H1 estimate is

$$H_1[k] = \frac{P_{xy}[k]}{P_{xx}[k] + \rho}$$

where its denominator is positive, and zero elsewhere.

Before clipping, the certificate records

$$r_{\text{bound}} = \max\left(0, \max_k \gamma^2_{\text{raw}}[k] - 1, -\min_k \gamma^2_{\text{raw}}[k]\right).$$

`KM-DSP-D07` is valid only if every returned array is finite and $r_{\text{bound}} \le 10^{-12}$. It records the ridge and segment count. The linear time-invariant data-generating model and statistical consistency are unchecked. High coherence on one finite record is not a causal identification theorem.

`CrossSpectrum` is a frozen, slotted record:

Member	Shape/type	Meaning
`frequencies`	`(F,)` float ndarray	Fourier coordinates in hertz
`cross_power`	`(F,)` ndarray	P_{xy}, generally complex
`input_power`, `output_power`	`(F,)` float ndarray	P_{xx}, P_{yy}
`coherence`	`(F,)` float ndarray	clipped magnitude-squared coherence
`transfer_h1`	`(F,)` complex ndarray	regularized H1 estimate
`sample_rate`	`float`	f_s
`scaling`	`SpectralScaling`	density or spectrum
`onesided`	`bool`	Fourier layout
`fft_length`	`int`	K
`segment_count`	`int`	M
`certificate`	`Certificate`	finite and pre-clipping bound check

Cost. Two framed transforms plus three segment products require $\Theta(ML + MK \log K)$ time with a larger constant than Welch. Peak storage is $\Theta(ML + MF)$.

32.5 Parseval and reconstruction certificates

The Parseval certificate below checks full-DFT energy. The reconstruction certificates are `KM-DSP-T03`: `istft` checks normalized finite overlap, while `STFTResult.roundtrip_certificate` separately compares synthesis with the supplied source, as specified in Section 32.1.2.

32.5.1 Finite Parseval certificate

```
parseval_certificate(
    signal: Any,
    *,
    tolerance: float = 1e-12,
) -> Certificate
```

For the full NumPy DFT,

$$X_k = \sum_{n=0}^{N-1} x_n e^{-2\pi i k n/N},$$

the checked identity is

$$\sum_{n=0}^{N-1} |x_n|^2 = \frac{1}{N} \sum_{k=0}^{N-1} |X_k|^2.$$

With time and frequency energies E_t, E_f, certificate `KM-DSP-T04` records

$$r_P = |E_t - E_f|$$

and is valid if

$$r_P \le \tau \max(1, E_t, E_f).$$

The function validates finite real or complex input and validates a finite nonnegative tolerance. It always uses the complete DFT, independently of the one-sided spectral-estimator APIs.

Cost. $\Theta(N \log N)$ time and $\Theta(N)$ transform storage.

32.5A Stateful and certified streaming STFT

```
StreamingSTFT(
    frame_length: int = 256,
    hop_length: int | None = None,
    fft_length: int | None = None,
    window: WindowSpec = "hann",
    sample_rate: float = 1.0,
    onesided: bool = True,
    detrend: DetrendMode = "none",
)

StreamingSTFT.process(
    self, chunk: Any
) -> StreamingSTFTBatch

StreamingSTFT.flush(
    self, *, pad_end: bool = True
) -> StreamingSTFTBatch

StreamingSTFT.reset(self) -> None

stream_stft(
    signal: Any,
    chunk_sizes: Iterable[int],
    *,
    frame_length: int = 256,
    hop_length: int | None = None,
    fft_length: int | None = None,
    window: WindowSpec = "hann",
    sample_rate: float = 1.0,
    onesided: bool = True,
    detrend: DetrendMode = "none",
    pad_end: bool = True,
    tolerance: float = 1e-11,
) -> ComputationResult[StreamingSTFTBatch]
```

Streaming uses the same default $H = \max(1, \lfloor L/4 \rfloor)$, periodic analysis window, detrending rule, and FFT coordinates as batch STFT. It intentionally matches the *uncentered* batch convention with zero terminal padding.

`process(chunk)` accepts a finite one-dimensional chunk, including an empty chunk, and emits every newly complete frame. After each completed call, the analyzer retains only overlap and unframed samples needed by a future frame. `buffered_samples` exposes that count and `total_samples` exposes the accepted input count. A one-sided analyzer rejects a complex chunk when a frame is transformed.

`flush(pad_end=True)` emits at most one terminal zero-padded frame, closes the analyzer, and releases its buffer. `flush(pad_end=False)` discards an incomplete tail and closes. Repeated flushes return an empty batch. Processing after flush raises `ValidationError`. `reset()` returns the instance to its deterministic empty, open state.

`StreamingSTFTBatch` is a frozen, slotted record:

Member	Shape/type	Meaning
`coefficients`	`(Q, F)` complex ndarray	frames emitted by one operation
`frequencies`	`(F,)` float ndarray	fixed bin coordinates
`times`	`(Q,)` float ndarray	$(s_m + L/2)/f_s$, with no center offset
`starts`	`(Q,)` integer ndarray	absolute source starts
`sample_rate`	`float`	f_s
`frame_length`, `hop_length`, `fft_length`	`int`	L, H, K
`onesided`	`bool`	Fourier layout
`frame_count`	property	Q

The mutable `StreamingSTFT` constructor exposes seven data fields: `frame_length`, `hop_length`, `fft_length`, `window`, `sample_rate`, `onesided`, and `detrend`. Its private buffer, buffer origin, next start, total count, and closed flag are state-machine implementation details rather than public constructor arguments. Its public properties are `frequencies`, `buffered_samples`, and `total_samples`.

The convenience function `stream_stft` requires nonempty finite input and a sequence of *positive* chunk sizes whose sum is exactly N. It feeds those chunks to a `StreamingSTFT`, flushes it, and compares the concatenated result with

```
stft(
    signal,
    center=False,
    pad_end=pad_end,
    boundary="zeros",
    ...matching parameters...
)
```

If coefficient shapes agree, it records

$$r_X = \max_{m,k} |X^{\text{stream}}_{mk} - X^{\text{batch}}_{mk}|, \qquad r_t = \max_{m} |t^{\text{stream}}_{m} - t^{\text{batch}}_{m}|.$$

Certificate `KM-DSP-T07` is valid when

$$r_X \le \tau \max(1, \max |X^{\text{batch}}|) \quad \text{and} \quad r_t \le \tau.$$

A shape mismatch records both residuals as infinity. The certificate checks contiguous declared chunks and matching finite STFT parameters. It does not extend the result to reordered, overlapping, missing, or duplicated chunks.

Cost and memory. For M emitted frames, analysis costs $\Theta(N + MK \log K + ML)$. The persistent analyzer state after a call retains fewer than L signal samples, but a single `process` call first materializes the supplied chunk and returns all frames emitted from it. The convenience `stream_stft` wrapper also retains all batches and constructs a complete batch STFT for comparison, so that certificate-producing wrapper has $\Theta(MF + ML)$ peak storage and is not a bounded-memory substitute for direct `StreamingSTFT` use.

32.6 `signals.time_frequency` complete class, function, and public-member reference

This table includes the adjoining `signals.spectral` and `signals.streaming` symbols needed to complete the reader workflow as well as every public member of `signals.time_frequency`.

Public symbol	Exact call or role	Return
`DetrendMode`	`"none" | "constant"`	type alias
`SpectrogramScaling`	density, spectrum, magnitude, power	type alias
`SpectralScaling`	density or spectrum	type alias
`SpectralAverage`	mean or median	type alias

Public symbol	Exact call or role	Return
`STFTResult`	frozen transform record	coefficients, coordinates, synthesis metadata, methods
`SpectrogramResult`	frozen spectrogram record	values plus underlying `STFTResult`
`PowerSpectrum`	frozen periodogram/Welch record	power, metadata, helpers, certificate
`CrossSpectrum`	frozen paired-spectrum record	auto/cross spectra, coherence, H1, certificate
`StreamingSTFTBatch`	frozen emission record	coefficients, coordinates, starts
`StreamingSTFT`	mutable analyzer	`process`, `flush`, `reset`
`stft`	full signature in Section 32.1	`STFTResult`
`istft`	full signature in Section 32.1	`ComputationResult[np.ndarray]`
`spectrogram`	full signature in Section 32.2	`SpectrogramResult`
`periodogram`	full signature in Section 32.3	`PowerSpectrum`
`welch_psd`	full signature in Section 32.3	`PowerSpectrum`
`cross_spectrum`	full signature in Section 32.4	`CrossSpectrum`
`parseval_certificate`	full signature in Section 32.5	`Certificate`
`stream_stft`	full signature in Section 32.5A	`ComputationResult[StreamingSTFTBatch]`

32.7 End-to-end finite example

This example exercises reconstruction, all spectral result families, Parseval, and adversarial chunk boundaries:

```
import numpy as np
from komorebi_math.signals import (
    cross_spectrum,
    parseval_certificate,
    periodogram,
    spectrogram,
    stft,
    stream_stft,
    welch_psd,
)

fs = 64.0
n = np.arange(64)
x = np.sin(2 * np.pi * 8 * n / fs)

transform = stft(
    x, sample_rate=fs,
    frame_length=16, hop_length=4, fft_length=32,
)
inverse = transform.inverse()
image = spectrogram(
    x, sample_rate=fs,
    frame_length=16, hop_length=4, fft_length=32,
    scaling="density",
)
line = periodogram(
    x, sample_rate=fs, window="boxcar", detrend="none"
)
welch = welch_psd(
    x, sample_rate=fs,
    frame_length=16, hop_length=8, detrend="none",
)
paired = cross_spectrum(
    x, -2.25 * x, sample_rate=fs,
```

```
    frame_length=16, hop_length=8, detrend="none",
)
parseval = parseval_certificate(x)
streamed = stream_stft(
    x, (1, 7, 56), sample_rate=fs,
    frame_length=16, hop_length=4, fft_length=32,
)

tone = np.argmax(paired.input_power)
print("stft", transform.coefficients.shape,
      "times", transform.times[[0, -1]],
      "frequencies", transform.frequencies[[0, -1]])
print("inverse", inverse.valid,
      "max_error", float(np.max(np.abs(inverse.value - x))),
      inverse.certificate.residuals)
print("spectrogram", image.values.shape, image.scaling)
print("periodogram", line.dominant_frequency(),
      line.integrated_power(), line.certificate.valid)
print("welch", welch.segment_count,
      welch.frequency_resolution, welch.certificate.valid)
print("cross_tone", paired.frequencies[tone],
      paired.coherence[tone], paired.transfer_h1[tone],
      paired.certificate.valid)
print("parseval", parseval.valid, parseval.residuals)
print("stream", streamed.value.frame_count,
      streamed.certificate.residuals, streamed.valid)
```

Observed output on the same environment as the Chapter 31 example:

```
stft (17, 17) times [0. 1.] frequencies [ 0. 32.]
inverse True max_error 2.220446049250313e-16 {'normalized_overlap_identity': 1.1102230246251565e-16,
    ↪ 'minimum_overlap': 1.25}
spectrogram (17, 17) density
periodogram 8.0 0.5 True
welch 7 4.0 True
cross_tone 8.0 0.9999999999999994 (-2.249999999999999-2.1028024000228914e-17j) True
parseval True {'parseval': 0.0}
stream 13 {'stream_batch_coefficients': 0.0, 'stream_batch_times': 0.0} True
```

The complex H1 value is numerically real. Its tiny imaginary component is FFT roundoff. The certificate establishes finite computations and bounds, not that arbitrary measured data obey the deterministic gain model used in this example.

32.8 Certificate and exception semantics

Komorebi Math distinguishes three failure channels:

1. Malformed dimensions, parameters, or a violated mandatory assumption raise immediately.
2. A mathematically meaningful computation with a failed numerical condition may return an invalid `Certificate` or `ComputationResult`.
3. Estimators may return a valid finite certificate while leaving stochastic or causal assumptions unchecked.

`Certificate.require()` returns the same certificate when `valid=True` and raises `ValidationError` otherwise. `ComputationResult.require()` first requires its certificate and then returns its value.

ID	Producer	Exactness	What validity means	Explicitly unchecked
`KM-DSP-D01`	`get_window` provenance	classical definition	requested finite window construction completed	no returned certificate

ID	Producer	Exactness	What validity means	Explicitly unchecked
KM-DSP-T01	window_metrics provenance	classical numerical diagnostic	metric preconditions passed	no returned certificate
KM-DSP-C01	overlap_add_certificate	numerical	nonzero residue overlap, plus constant overlap if requested	finite-record boundary coverage
KM-DSP-T02	canonical_dual_window	numerical	nonzero squared profile and unit cross-profile within tolerance	finite-record boundary coverage
KM-DSP-D02	FrameBatch.certificate()	exact	shape, starts, and coverage inclusion agree	source-sample equality is not checked
KM-DSP-D03	stft provenance	classical definition	finite transform completed	no returned certificate
KM-DSP-T03	istft	numerical	requested local overlap denominator is nonzero and normalization identity is represented	source agreement requires roundtrip_certificate
KM-DSP-D04	spectrogram provenance	classical definition	requested scaling completed	no returned certificate
KM-DSP-D05	periodogram	numerical	finite, numerically nonnegative power	stochastic consistency
KM-DSP-D06	welch_psd	numerical	finite, numerically nonnegative averaged power	stochastic consistency
KM-DSP-D07	cross_spectrum	numerical	finite arrays and pre-clipping coherence bound within 10^{-12}	LTI data model and statistical consistency
KM-DSP-T04	parseval_certificate	numerical	finite DFT energy residual satisfies scaled tolerance	none beyond finite DFT convention
KM-DSP-T07	stream_stft	numerical	coefficient and timestamp equality with matching batch STFT	only declared contiguous partition is covered

The principal intentional exceptions are:

API	Condition	Exception or returned state
stft	empty/non-1-D signal	DimensionError
stft	nonpositive/nonfinite rate. Invalid length. $K < L$. Invalid detrend. One-sided complex input. Framing failure	ValidationError or framing DimensionError
istft	input is not STFTResult	ValidationError
istft	transform used constant detrending	AssumptionError
istft	negative/nonfinite tolerance	ValidationError
istft	synthesis window wrong shape. Request exceeds original or represented range	DimensionError
istft	local denominator fails strict nonzero test	invalid result containing zeros. .require() raises ValidationError
spectrogram	unknown scaling. Zero coherent gain in spectrum scaling	ValidationError

API	Condition	Exception or returned state
spectral estimators	empty/non-1-D or nonfinite signal. Paired lengths differ. Record shorter than Welch frame	`DimensionError` or `ValidationError`
spectral estimators	invalid rate, hop, FFT length, detrend, scaling, average, one-sided complex data, or ridge	`ValidationError`
`parseval_certificate`	invalid tolerance	`ValidationError`
`StreamingSTFT`	invalid geometry/rate/detrend. Nonfinite or non-1-D chunk. One-sided complex transformed frame	`ValidationError` or `DimensionError`
`StreamingSTFT.process`	called after flush	`ValidationError`
`stream_stft`	empty/non-1-D/nonfinite source. Nonpositive chunk size. Chunk sum mismatch	`ValidationError` or `DimensionError`

32.9 Source-audit defects and unsupported inferences

These are observed behaviors of the `1.0.0b4` source, not recommended contracts. They should be treated as defects or validation gaps until changed and regression-tested:

1. `get_window("not-a-window", 1)` returns `[1.0]`. The length-one shortcut occurs before unknown bare-string dispatch.
2. `frame_signal(..., boundary="invalid")` accepts and records the invalid string when no padding is needed. `_pad` returns an unpadded copy before validating the boundary. `stft` inherits the same branch behavior.
3. `stft` does not reject NaN or infinity in its input. Nonfinite coefficients can therefore be returned without a certificate. This is an intentional absence of a uniform package guard, but it is hazardous for this transform API.
4. `overlap_profile` does not validate finite window entries. A NaN window produces a NaN profile. `overlap_add_certificate` then becomes invalid, but its `assumptions_checked` still says `finite window`.
5. `canonical_dual_window` does not validate that `tolerance` is finite and nonnegative. For example, `tolerance=-1` is accepted and stored in an invalid certificate instead of raising.
6. Density scaling has no explicit zero-window denominator guard. A zero window can make `spectrogram(..., scaling="density")` return nonfinite values without a certificate. `periodogram` and Welch can likewise divide by zero, but their `PowerSpectrum` certificate becomes invalid because the ordinates are nonfinite.
7. `stream_stft` does not validate that `tolerance` is finite and nonnegative. A negative or NaN tolerance is stored in an invalid batch-equivalence certificate instead of raising.

Additional boundaries are deliberate rather than defects:

- A COLA/NOLA or canonical-dual certificate does not settle finite edge coverage.
- An inverse-STFT normalization certificate does not compare with the original signal.
- Median Welch and cross-spectrum averaging has no bias correction.
- Coherence clipping does not turn a failed pre-clipping bound into a theorem.
- Spectral estimates do not certify stationarity, asymptotic consistency, causality, or the correctness of an LTI model.
- The streaming equality covers uncentered analysis with zero terminal padding and the exact declared chunk partition, not arbitrary stream edits.
- No GPU path exists in these four modules, and no hardware or real-time claim follows from NumPy execution.

32.10 Executed verification evidence

The evidence below was executed against the recovered `1.0.0b4` checkout for the reference baseline:

Evidence	Observed result	Scope
Focused unit tests: `test_block12_windows_time_frequency.py`, `test_block12_spectral_filtering.py`, `test_block12_denoise_streaming.py`	53 passed	windows, metrics, framing, STFT/inverse, spectrogram, Parseval, periodogram, Welch, cross-spectrum, streaming boundaries. The latter two files also contain adjacent DSP tests
`examples/production_dsp.py`	completed, exit 0	reported 73 Hz dominant line, 33 batch STFT frames, 29 streaming frames, and valid certificates across its DSP pipeline
`run_dsp_verification()`	11 of 11 passed	includes window/dual, normalized STFT, spectral/Parseval, coherence/H1, and streaming-STFT checks plus adjacent DSP checks
`verification.block12_deep_audit.run()`	815,779 of 815,779 scalar checks passed	relevant section counts: windows 117, STFT 114,190, streaming STFT 160, spectral 8,460. Remaining checks cover adjacent DSP routines
`verification.block12_scipy_reference_audit.run()`	520,920 of 520,920 comparisons passed	relevant section counts: windows 6,804 and spectral 76,502. Convention-matched comparison only, not API identity
Chapter 31 deterministic example	completed with displayed output	periodic Hann metrics, COLA, canonical dual, centered/end-padded frames, raw overlap-add
Chapter 32 deterministic example	completed with displayed output	STFT/inverse, spectrogram, periodogram, Welch, cross/H1, Parseval, chunk invariance

The focused pytest command initially failed because `pytest` was absent from the primary runtime. It was rerun in the existing `.venv-guide` environment, where Python 3.12.13, NumPy 2.5.2, and pytest 9.1.1 were available. Both independent audits were invoked through their read-only `run()` functions so their committed JSON reports were not rewritten.

The passing evidence establishes the finite cases actually executed. It does not erase the source-audit defects above, expand the supported input domain, or convert estimator diagnostics into stochastic theorems.

> **Chapters 31 through 32 summary.** Construct windows under an explicit periodic or symmetric convention. Read gain and overlap diagnostics as finite numerical records. Keep framing geometry and boundary padding in the result. Use raw overlap-add only when summation is intended. Use `istft` for denominator-normalized synthesis. Distinguish transform metadata from a roundtrip certificate, and treat periodograms, Welch averages, coherence, and H1 as finite estimators whose modeling assumptions remain visible. For streams, use `StreamingSTFT` for bounded retained state and `stream_stft` when a batch-equivalence certificate is worth its full materialization cost.

33. Filtering and convolution

`komorebi_math.signals` implements finite, one-dimensional filtering on NumPy arrays. Two-input signal/kernel application functions take the signal first and the kernel second. Constructors and design routines have their own documented signatures. The routines in this chapter use zero extension unless the mode is explicitly `"circular"`. None silently changes to periodic boundaries. All certificate results in this chapter are numerical, even when the finite indexing identity is mathematically exact.

These APIs are on the complete research namespace:

```
from komorebi_math.signals import fir_filter, fft_convolve
```

They are not all re-exported from the package root, and the production DSP surface is not part of the frozen `komorebi_math.stable` contract in `1.0.0b4`. Import from `komorebi_math.signals` when the names form part of an application interface.

33.1 FIR design and application

For a signal $x \in \mathbb{K}^N$ and a kernel $h \in \mathbb{K}^K$, finite linear convolution means

$$(h * x)[n] = \sum_{j=0}^{K-1} h[j]x[n-j],$$

where samples outside $0 \le n < N$ are zero. The direct FIR entry point is

```
fir_filter(signal: Any, taps: Any, *, mode: FIRMode = "causal") -> np.ndarray
```

`taps` may be a one-dimensional array or an `FIRFilter`. Both the signal and the taps must be nonempty, one dimensional, and finite. `fir_filter` delegates the full calculation to `numpy.convolve` and then applies the package's crop convention.

The design vocabulary is

```
FIRKind = Literal["lowpass", "highpass", "bandpass", "bandstop"]
FIRMode = Literal["causal", "full", "same", "valid"]
```

The public constructors are

```
design_fir(
    kind: FIRKind,
    cutoff: float | tuple[float, float],
    *,
    num_taps: int = 101,
    sample_rate: float = 1.0,
    window: WindowSpec = "hann",
    scale: bool = True,
) -> FIRFilter

design_lowpass(cutoff: float, **options: Any) -> FIRFilter
design_highpass(cutoff: float, **options: Any) -> FIRFilter
design_bandpass(cutoff: tuple[float, float], **options: Any) -> FIRFilter
design_bandstop(cutoff: tuple[float, float], **options: Any) -> FIRFilter
```

The ideal low-pass seed at cutoff f_c and sample rate f_s is

$$h_n^{\text{ideal}} = 2\frac{f_c}{f_s}\,\operatorname{sinc}\left(2\frac{f_c}{f_s}\left[n - \frac{K-1}{2}\right]\right).$$

High-pass and band-stop designs use spectral inversion. A band-pass design is the difference of two low-pass seeds. The selected symmetric window is then applied. With `scale=True`, the implementation normalizes magnitude at DC for low-pass and band-stop, at Nyquist for high-pass, and at the midpoint of the two cutoffs for band-pass. This construction requires an odd `num_taps`, so its group delay is the integer

$$\tau_g = \frac{K-1}{2}.$$

The design record and sampled response record have these exact constructors and public members:

```
FIRFilter(
    taps: np.ndarray,
    kind: FIRKind,
    cutoff: tuple[float, ...],
    sample_rate: float = 1.0,
    window_name: str = "custom",
    reference_frequency: float | None = None,
    reference_gain: float = 1.0,
) -> None

FIRFilter.order -> int
FIRFilter.group_delay_samples -> float
FIRFilter.apply(signal: Any, *, mode: FIRMode = "causal") -> np.ndarray
FIRFilter.response(*, points: int = 512, whole: bool = False) -> FrequencyResponse
FIRFilter.certificate(*, tolerance: float = 1e-10) -> Certificate

FrequencyResponse(
    frequencies: np.ndarray,
    response: np.ndarray,
    sample_rate: float,
) -> None

FrequencyResponse.magnitude -> np.ndarray
FrequencyResponse.phase -> np.ndarray
FrequencyResponse.magnitude_db -> np.ndarray

fir_frequency_response(
    taps: Any,
    *,
    points: int = 512,
    sample_rate: float = 1.0,
    whole: bool = False,
) -> FrequencyResponse
```

The sampled transfer function is

$$H(e^{i\omega}) = \sum_{n=0}^{K-1} h_n e^{-i\omega n}.$$

`whole=False` covers $0 \le f \le f_s/2$, including the endpoint. `whole=True` covers $0 \le f < f_s$. Response values are complex, `phase` is unwrapped, and `magnitude_db` clips magnitude at the smallest positive floating value before taking the logarithm.

`FIRFilter.certificate` checks only tap symmetry and the stored reference gain:

$$r_{\text{sym}} = \max_n |h_n - h_{K-1-n}|, \qquad r_{\text{gain}} = \left| |H(f_{\text{ref}})| - g_{\text{ref}} \right| .$$

It reports theorem `KM-DSP-C02`. It does not certify a transition width, pass-band ripple, stop-band attenuation, or an application-specific filter specification.

Failure cases are explicit. Cutoffs must be finite and strictly inside $(0, f_s/2)$. Band edges must be increasing. Low-pass and high-pass take one cutoff, while band-pass and band-stop take two. The sample rate and point count must be positive, the tap count must be odd, and normalization fails when the selected reference gain is numerically zero.

33.2 Streaming FIR

`StreamingFIR` is the mutable causal executor:

```
StreamingFIR(taps: Any) -> None

StreamingFIR.taps: np.ndarray
StreamingFIR.processed_samples: int
StreamingFIR.process(chunk: Any) -> np.ndarray
StreamingFIR.flush() -> np.ndarray
StreamingFIR.reset() -> None

stream_fir_filter(
    signal: Any,
    taps: Any,
    chunk_sizes: Iterable[int],
    *,
    tolerance: float = 1e-12,
) -> ComputationResult[np.ndarray]
```

The persistent state is the most recent $K - 1$ input samples. `process` accepts an empty chunk and otherwise emits exactly one causal output for every input sample. Therefore, for any ordered complete partition $x = x^{(0)} \| \cdots \| x^{(q-1)}$,

$$\text{concat}_j \, \text{process}(x^{(j)}) = \text{fir_filter}(x, h, \texttt{mode="causal"}).$$

`flush` emits the remaining $K - 1$ full-convolution samples and closes the object. A second flush returns an empty array. Processing after flush raises `ValidationError`. `reset` restores zero history, clears the sample count, and reopens the object.

The `stream_fir_filter` convenience wrapper requires strictly positive chunk sizes whose sum is exactly the signal length. Its `KM-DSP-T05` certificate compares only the concatenated `process` outputs with causal batch filtering. It excludes the full tail because the wrapper does not call `flush`.

For a streaming comparison, decide first whether the desired output is the causal prefix or the complete finite convolution. Compare the concatenated `process` outputs with the causal prefix. Include the single `flush` output when comparing with full convolution. A missing tail in the first comparison is intentional; adding it to only one side makes the comparison inconsistent.

For a chunk of length C, the current direct implementation performs a convolution of the chunk plus history, costing $O((C+K)K)$ arithmetic in the usual direct-convolution model. Persistent memory is $O(K)$, in addition to the current chunk and temporary convolution array. This is chunk-invariant execution, not a hard real-time or fixed-latency guarantee.

33.3 Recursive filtering and stable biquads

`LinearFilter` represents the normalized rational transfer function

$$H(z) = \frac{\sum_{j=0}^{q} b_j z^{-j}}{1 + \sum_{j=1}^{q} a_j z^{-j}}.$$

The constructor divides both coefficient arrays by the first denominator coefficient, pads them to a common length, and stores floating or complex arrays:

```
LinearFilter(numerator: Any, denominator: Any = (1.0,)) -> None

LinearFilter.numerator: np.ndarray
LinearFilter.denominator: np.ndarray
LinearFilter.order -> int
LinearFilter.zeros -> np.ndarray
LinearFilter.poles -> np.ndarray
LinearFilter.process(
    signal: Any,
    *,
    initial_state: Any | None = None,
) -> FilterOutput
LinearFilter.response(
    *,
    points: int = 512,
    sample_rate: float = 1.0,
    whole: bool = False,
) -> FrequencyResponse
LinearFilter.stability_certificate(*, tolerance: float = 1e-12) -> Certificate

FilterOutput(samples: np.ndarray, final_state: np.ndarray) -> None

lfilter(
    numerator: Any,
    denominator: Any,
    signal: Any,
    *,
    initial_state: Any | None = None,
) -> FilterOutput
```

Execution uses direct form II transposed with state $s \in \mathbb{K}^q$:

$$y_n = b_0 x_n + s_0,$$

$$s_j \leftarrow s_{j+1} + b_{j+1} x_n - a_{j+1} y_n, \qquad s_{q-1} \leftarrow b_q x_n - a_q y_n.$$

The returned `final_state` is the exact state to pass as `initial_state` for the next contiguous chunk. Integer coefficients are promoted to floating point. A complex signal or coefficient promotes the recursion to complex arithmetic. `np.real_if_close` may return a real array when a complex result is numerically real.

Zeros and poles are numerical roots of the stored numerator and denominator coefficient polynomials. The `KM-DSP-C04` stability certificate computes

$$r_{\max} = \max_{\lambda \in \text{poles}} |\lambda|, \qquad m = 1 - r_{\max},$$

and requires the strict inequality $m >$ `tolerance`. It is a coefficient-level BIBO diagnostic. It does not cover unrepresented coefficient quantization, arithmetic overflow, limit cycles, or robustness margins under perturbation.

The included biquad constructor is

```
BiquadKind = Literal["lowpass", "highpass", "bandpass", "notch", "allpass"]

design_biquad(
    kind: BiquadKind,
    frequency: float,
    *,
    q: float = np.float64(0.7071067811865475),
    sample_rate: float = 1.0,
) -> LinearFilter
```

It uses

$$\omega_0 = 2\pi f_0 / f_s, \qquad \alpha = \frac{\sin \omega_0}{2Q},$$

and the standard normalized low-pass, high-pass, band-pass, notch, or all-pass coefficient table implemented in `signals.recursive`. This is not a general high-order IIR design facility.

Mutable recursive streaming is available as

```
StreamingLinearFilter(filter: LinearFilter) -> None

StreamingLinearFilter.filter: LinearFilter
StreamingLinearFilter.state: np.ndarray
StreamingLinearFilter.processed_samples: int
StreamingLinearFilter.process(chunk: Any) -> np.ndarray
StreamingLinearFilter.reset() -> None

stream_lfilter(
    filter_: LinearFilter,
    signal: Any,
    chunk_sizes: Any,
    *,
    tolerance: float = 1e-12,
) -> ComputationResult[np.ndarray]
```

There is no recursive-filter flush: a causal recursion has no separately emitted FIR tail. `stream_lfilter` consumes positive requested chunk sizes, clips a chunk that would extend past the end, ignores later sizes after the signal is exhausted, and automatically processes any unlisted remainder. Its `KM-DSP-T09` certificate checks maximum absolute equality with zero-state batch execution.

Filtering costs $O(Nq)$ time and $O(q)$ continuation state. Computing a complete sampled response costs $O(Pq)$ for P frequency points. Invalid inputs include a zero first denominator coefficient, a nonfinite signal or state, a state with length other than `order`, a response denominator that vanishes on the requested grid, $f_0 \notin (0, f_s/2)$, and nonpositive Q. An unstable denominator constructs successfully. Its stability certificate is invalid and callers may enforce it with `.require()`.

Example 33-B: FIR and IIR streaming

This example uses irregular chunks, explicitly includes the FIR tail, and checks a stable biquad.

```
import numpy as np

from komorebi_math.signals import (
    LinearFilter,
    StreamingFIR,
    design_biquad,
    stream_lfilter,
)

x = np.arange(1.0, 8.0)
```

```
fir = StreamingFIR([0.25, 0.50, 0.25])
fir_pieces = [
    fir.process(x[:1]),
    fir.process(x[1:4]),
    fir.process(x[4:]),
    fir.flush(),
]
print("FIR pieces:", [piece.tolist() for piece in fir_pieces])
print("FIR full:", np.concatenate(fir_pieces).tolist())

biquad = design_biquad("lowpass", 100.0, q=0.8, sample_rate=1000.0)
biquad.stability_certificate().require()
iir = stream_lfilter(biquad, x, (1, 3, 3))
iir.certificate.require()
print("IIR:", np.round(iir.value, 8).tolist())
print("IIR stream residual:", iir.certificate.residuals)

unstable = LinearFilter([1.0], [1.0, -1.01])
print("unstable certificate:", unstable.stability_certificate().valid)
```

Observed with NumPy `2.3.5` and Komorebi Math `1.0.0b4`:

```
FIR pieces: [[0.25], [1.0, 2.0, 3.0], [4.0, 5.0, 6.0], [5.0, 1.75]]
FIR full: [0.25, 1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 5.0, 1.75]
IIR: [0.06983611, 0.36198303, 0.95472051, 1.80029766, 2.80599239, 3.88417736, 4.97406059]
IIR stream residual: {'streaming_batch_max_abs': 0.0}
unstable certificate: False
```

33.4 Direct, FFT, and overlap-add convolution

`fft_convolve` pads to

$$L_{\mathrm{FFT}} = 2^{\lceil \log_2 (N+K-1) \rceil}$$

and evaluates

$$y = \mathcal{F}^{-1}_{L_{\mathrm{FFT}}} \left(\mathcal{F}(x) \odot \mathcal{F}(h) \right)_{0:N+K-1}.$$

Real inputs use `numpy.fft.rfft` and `irfft`. Any complex input uses the two-sided complex transform. The public call is

```
fft_convolve(
    signal: Any,
    kernel: Any,
    *,
    mode: FIRMode = "full",
) -> np.ndarray
```

It costs $O(L_{\mathrm{FFT}} \log L_{\mathrm{FFT}})$ time and $O(L_{\mathrm{FFT}})$ temporary memory. No heuristic chooses between direct and FFT convolution. The caller chooses the entry point.

Overlap-add divides the input into blocks of length B and uses

$$L_B = 2^{\lceil \log_2 (B+K-1) \rceil}.$$

Its public surfaces are

```
OverlapAddConvolver(kernel: np.ndarray, block_length: int = 1024) -> None

OverlapAddConvolver.kernel: np.ndarray
OverlapAddConvolver.block_length: int
OverlapAddConvolver.fft_length: int
OverlapAddConvolver.processed_samples: int
OverlapAddConvolver.buffered_samples -> int
OverlapAddConvolver.process(chunk: Any) -> np.ndarray
OverlapAddConvolver.flush() -> np.ndarray
OverlapAddConvolver.reset() -> None

overlap_add_convolve(
    signal: Any,
    kernel: Any,
    *,
    block_length: int = 1024,
    mode: FIRMode = "full",
) -> np.ndarray

certified_overlap_add_convolve(
    signal: Any,
    kernel: Any,
    *,
    block_length: int = 1024,
    mode: FIRMode = "full",
    tolerance: float = 1e-11,
) -> ComputationResult[np.ndarray]
```

The mutable object stores the kernel spectrum, fewer than B untransformed samples after each call, and $K-1$ overlap samples. `process` accepts arbitrary and empty chunks and emits every complete transform block. `flush` transforms a final partial block, emits the convolution tail once, and closes the object. A repeated flush is empty. Processing after flush fails. `reset` restores the open zero-overlap state. A real-kernel processor promotes itself to complex FFTs if a complex chunk arrives.

For $J = \lceil N/B \rceil$, work is $O(JL_B \log L_B)$. Retained processor memory is $O(L_B + B + K)$. The peak for one `process` call also includes the supplied chunk because the implementation concatenates it before draining blocks.

The implementation here is NumPy-only and executes through `numpy.fft`. It does not accept a CUDA device, stream, backend, or workspace parameter. GPU qualification elsewhere in the package does not certify these DSP functions.

33.5 Convolution modes and output lengths

Two mode aliases are intentionally distinct:

```
ConvolutionMode = Literal["full", "same", "valid", "circular"]
FIRMode         = Literal["causal", "full", "same", "valid"]
```

For signal length N and kernel length K, the exact crop conventions are:

Mode	Length	Selection or boundary law
`full`	$N + K - 1$	complete zero-extended convolution
`causal`	N	full indices $0 : N$. FIR APIs only
`same`	N	starts at $\lfloor (K-1)/2 \rfloor$
`valid`	$\max(N - K + 1, 0)$	starts at $K - 1$. Empty when $K > N$
`circular`	N	source index $(n - j)$ mod N. Finite-convolution APIs only

Unlike `numpy.convolve(..., mode="same")`, package `same` is always the signal length, including when $K > N$. `convolution_output_length`, `convolution_matrix`, `convolve_signal`, and the finite polyphase APIs accept `ConvolutionMode`. They reject `"causal"`. `fir_filter`, `fft_convolve`, and the overlap-add APIs accept `FIRMode`. They reject `"circular"`.

The finite matrix APIs are

```
convolution_output_length(
    input_length: int,
    kernel_length: int,
    mode: ConvolutionMode,
) -> int

convolution_matrix(
    kernel: Any,
    input_length: int,
    *,
    mode: ConvolutionMode = "full",
) -> np.ndarray

convolve_signal(
    signal: Any,
    kernel: Any,
    *,
    mode: ConvolutionMode = "full",
) -> np.ndarray
```

They construct $H_h^{\mathrm{mode}} \in \mathbb{K}^{M \times N}$ such that $y = H_h^{\mathrm{mode}} x$. The matrix dtype is `np.result_type(kernel.dtype, float)`, so integer kernels produce floating matrices and complex kernels remain complex. This route stores MN values and matrix application costs $O(MN)$. Use direct or FFT convolution when the dense operator itself is not needed.

For packet order q, define source and target residue indices

$$I_s^X = \{s, s+q, s+2q, \ldots\} \cap [0, N), \qquad I_t^Y = \{t, t+q, t+2q, \ldots\} \cap [0, M).$$

The finite transfer blocks are literal submatrices

$$H_{ts} = H_h^{\mathrm{mode}}[I_t^Y, I_s^X], \qquad y_t = \sum_s H_{ts} x_s.$$

The exact public API is

```
PolyphaseConvolution(
    kernel: np.ndarray,
    input_length: int,
    order: int,
    mode: ConvolutionMode = "full",
) -> None

PolyphaseConvolution.matrix -> np.ndarray
PolyphaseConvolution.output_length -> int
PolyphaseConvolution.source_indices(residue: int) -> np.ndarray
PolyphaseConvolution.target_indices(residue: int) -> np.ndarray
PolyphaseConvolution.blocks() -> tuple[tuple[np.ndarray, ...], ...]
PolyphaseConvolution.apply_coordinates(
    coordinates: PolyphaseCoordinates,
) -> PolyphaseCoordinates
PolyphaseConvolution.apply(signal: Any) -> np.ndarray
PolyphaseConvolution.certificate(
    signal: Any,
    *,
    tolerance: float = 1e-11,
) -> ComputationResult[np.ndarray]
```

```
PolyphaseBlockMatrix(
    blocks: tuple[tuple[np.ndarray, ...], ...],
    order: int,
    input_length: int,
    output_length: int,
) -> None

finite_polyphase_blocks(
    kernel: Any,
    input_length: int,
    order: int,
    *,
    mode: ConvolutionMode = "full",
) -> PolyphaseBlockMatrix

polyphase_convolution(
    signal: Any,
    kernel: Any,
    order: int,
    *,
    mode: ConvolutionMode = "full",
    tolerance: float = 1e-11,
) -> ComputationResult[np.ndarray]
```

`FinitePolyphaseKernel` and `finite_polyphase_filter_kernels` are public aliases of `PolyphaseBlockMatrix` and `finite_polyphase_blocks`. The `PSC-V-T07` certificate compares packet reconstruction with the dense matrix product using the Euclidean residual. It does not compare against an infinite causal kernel. In particular, the tempting formula $h[qm + t - s]$ cannot replace these finite boundary blocks unless boundary equivalence has been proved.

Example 33-A: all convolution modes

Here $K = 4 > N = 3$, so `valid` is intentionally empty and `same` remains length three. FFT and overlap-add are compared only where their `FIRMode` vocabulary matches.

```
import numpy as np

from komorebi_math.signals import (
    convolution_matrix,
    convolve_signal,
    fft_convolve,
    overlap_add_convolve,
    polyphase_convolution,
)

x = np.array([1.0, 2.0, 3.0])
h = np.array([1.0, -1.0, 2.0, 0.5])

for mode in ("full", "same", "valid", "circular"):
    matrix = convolution_matrix(h, x.size, mode=mode)
    direct = convolve_signal(x, h, mode=mode)
    packet = polyphase_convolution(x, h, 2, mode=mode)
    packet.certificate.require()
    print(mode, matrix.shape, direct.tolist(), packet.certificate.residuals)
    if mode != "circular":
        assert np.allclose(fft_convolve(x, h, mode=mode), direct)
        assert np.allclose(
            overlap_add_convolve(x, h, block_length=2, mode=mode), direct
        )
```

Observed output:

```
full (6, 3) [1.0, 1.0, 3.0, 1.5, 7.0, 1.5] {'transfer_law': 0.0}
same (3, 3) [1.0, 3.0, 1.5] {'transfer_law': 0.0}
valid (0, 3) [] {'transfer_law': 0.0}
circular (3, 3) [2.5, 8.0, 4.5] {'transfer_law': 0.0}
```

33.6 Certified overlap-add error

`certified_overlap_add_convolve` computes the overlap-add result and an independent direct `numpy.convolve` reference under the same crop. Its certificate records

$$r_{\mathrm{OLA}} = \|y_{\mathrm{OLA}} - y_{\mathrm{direct}}\|_\infty$$

and accepts when the shapes agree and

$$r_{\mathrm{OLA}} \le \texttt{tolerance}\,\max(1, \|y_{\mathrm{direct}}\|_\infty).$$

The theorem identifier is `KM-DSP-T10`. Checked assumptions are finite ordered input and linear zero-padded convolution. This establishes finite numerical agreement for the supplied arrays, block length, mode, NumPy version, and tolerance. It does not prove exact arithmetic, favorable performance, a frequency-domain specification, real-time scheduling, or GPU execution.

Across the chapter, nonempty one-dimensional finite arrays are required by FIR, FFT, overlap-add, and recursive filtering. The older finite convolution matrix layer requires nonempty one-dimensional arrays but does not perform the same explicit finite-entry check. A nonpositive block length, input length, kernel length, or packet order is invalid. Mode strings outside the relevant alias fail with `ValidationError`.

34. Resampling and adaptive filters

The signal layer contains two finite batch facilities. Rational resampling uses a centered zero-extension plan with explicit polyphase/reference equivalence. Adaptive filtering records every coefficient iterate of an LMS, NLMS, or RLS recursion. Neither facility currently exposes a mutable streaming state class.

34.1 Upsampling, downsampling, interpolation, and decimation

Primitive upsampling inserts zeros:

$$(\uparrow_U x)[n] = \begin{cases} x[n/U], & n \equiv 0 \pmod{U}, \\ 0, & \text{otherwise}, \end{cases}$$

while downsampling retains one residue class:

$$(\downarrow_{D,r} x)[m] = x[r + mD], \qquad 0 \le r < D.$$

The functions are

```
upsample(signal: Any, factor: int) -> np.ndarray
downsample(signal: Any, factor: int, *, phase: int = 0) -> np.ndarray

interpolate(
    signal: Any,
    factor: int,
    *,
    num_taps: int | None = None,
    window: WindowSpec = ("kaiser", 8.0),
) -> np.ndarray

decimate(
    signal: Any,
    factor: int,
    *,
    num_taps: int | None = None,
    window: WindowSpec = ("kaiser", 8.0),
) -> np.ndarray
```

`upsample` and `downsample` are unfiltered primitives and preserve input dtype. `interpolate(x, U)` is exactly the filtered rational call with ratio `(U, 1)`. `decimate(x, D)` is the filtered call with ratio `(1, D)`. Thus the latter two include the finite default low-pass and centered boundary convention. They are not aliases for primitive insertion or selection.

All factors must be positive integers. `phase` must be an integer in $[0, D)$, and the signal must be nonempty, one dimensional, and finite.

34.2 Rational polyphase resampling

For requested integers $u, d > 0$, the implementation first reduces the rate:

$$g = \gcd(u, d), \qquad U = u/g, \qquad D = d/g.$$

Plan metadata, tap design, and output length all use U/D, with

$$N_{\text{out}} = \left\lceil \frac{NU}{D} \right\rceil = \left\lfloor \frac{NU + D - 1}{D} \right\rfloor.$$

The basic design and length calls are

```
resampled_length(input_length: int, up: int, down: int) -> int

design_resample_filter(
    up: int,
    down: int,
    *,
    num_taps: int | None = None,
    window: WindowSpec = ("kaiser", 8.0),
) -> np.ndarray
```

The default odd tap count is

$$K = 20 \max(U, D) + 1.$$

On the upsampled grid, the windowed-sinc seed has cutoff

$$f_c = \frac{1}{2 \max(U, D)}$$

cycles per high-rate sample and is scaled so that

$$\sum_{j=0}^{K-1} h_j = U.$$

The plan and result records expose:

```
RationalResamplingPlan(up: int, down: int, taps: np.ndarray) -> None

RationalResamplingPlan.up: int
RationalResamplingPlan.down: int
RationalResamplingPlan.taps: np.ndarray
RationalResamplingPlan.delay -> int
RationalResamplingPlan.polyphase_components -> tuple[np.ndarray, ...]
RationalResamplingPlan.output_length(input_length: int) -> int
RationalResamplingPlan.apply(signal: Any) -> np.ndarray
RationalResamplingPlan.explicit_reference(signal: Any) -> np.ndarray
RationalResamplingPlan.certificate(
    signal: Any,
    *,
    tolerance: float = 1e-12,
) -> Certificate

ResamplingResult(
    signal: np.ndarray,
    plan: RationalResamplingPlan,
    input_length: int,
```

```
    certificate: Certificate,
) -> None
```

For centered delay $\delta = (K-1)/2$, the U polyphase components are

$$h^{(r)} = (h_r, h_{r+U}, h_{r+2U}, \ldots), \qquad 0 \le r < U.$$

For output sample m, the direct plan sets

$$q_m = \delta + mD, \qquad r_m = q_m \bmod U,$$

then evaluates

$$y_m = \sum_{\substack{0 \le j < K \\ j \equiv r_m \ (U)}} h_j x_{(q_m - j)/U},$$

omitting out-of-range source indices. `explicit_reference` independently inserts zeros, performs full convolution, removes the centered delay, and takes every D-th sample. This is a zero-extension boundary rule, so startup and terminal transients are part of the contract.

The public wrappers are

```
resample_poly_result(
    signal: Any,
    up: int,
    down: int,
    *,
    taps: Any | None = None,
    num_taps: int | None = None,
    window: WindowSpec = ("kaiser", 8.0),
    tolerance: float = 1e-12,
) -> ResamplingResult

resample_poly(
    signal: Any,
    up: int,
    down: int,
    *,
    taps: Any | None = None,
    num_taps: int | None = None,
    window: WindowSpec = ("kaiser", 8.0),
) -> np.ndarray
```

Supplying `taps` bypasses filter design. `num_taps` and `window` then have no effect. A custom plan still requires a nonempty finite one-dimensional odd kernel. It may be applied even when its DC gain is not U, but its certificate will be invalid if the gain residual exceeds tolerance.

`KM-DSP-T06` records

$$r_{\text{poly}} = \|y_{\text{plan}} - y_{\text{explicit}}\|_\infty, \qquad r_{\text{DC}} = \left| \sum_j h_j - U \right|.$$

The first residual is scaled by the maximum reference magnitude. The second is scaled by $\max(1, U)$. The certificate checks the centered zero-extension convention and reduced rate. It explicitly leaves application-specific alias rejection and boundary-transient acceptability unchecked. A passing result is not a universal anti-aliasing or signal-fidelity guarantee.

Designed taps are real `float64`. Primitive up/down sampling preserves dtype. Plan application uses the result type of the signal and taps, supports complex values, and applies `np.real_if_close` to numerically real output. The

direct polyphase loop costs $O(N_{\text{out}}\lceil K/U \rceil)$ time and stores $O(N_{\text{out}} + K)$ values. Materializing `polyphase_components` copies K tap values. The explicit reference additionally materializes NU zero-inserted samples and uses direct `numpy.convolve`. It is principally a verification construction.

Invalid cases include nonpositive or nonintegral rates, nonpositive input length, even tap count, nonfinite signal or taps, invalid downsample phase, and a designed window whose DC sum is numerically zero.

Example 34-A: coprime and reducible resampling

```
import numpy as np

from komorebi_math.signals import (
    downsample,
    resample_poly_result,
    resampled_length,
    upsample,
)

x = np.array([0.0, 1.0, 0.0, -1.0, 0.0])
reducible = resample_poly_result(x, 6, 4)
coprime = resample_poly_result(x, 3, 2)
reducible.certificate.require()
coprime.certificate.require()

print("ratios:", (reducible.plan.up, reducible.plan.down),
      (coprime.plan.up, coprime.plan.down))
print("lengths:", reducible.signal.size, coprime.signal.size,
      resampled_length(x.size, 6, 4))
print("tap count/delay/phases:", reducible.plan.taps.size,
      reducible.plan.delay,
      [part.size for part in reducible.plan.polyphase_components])
print("same taps/output:",
      np.array_equal(reducible.plan.taps, coprime.plan.taps),
      np.array_equal(reducible.signal, coprime.signal))
print("output:", np.round(reducible.signal, 8).tolist())
print("residuals:", reducible.certificate.residuals)
print("upsample:", upsample([1, 2, 3], 3).tolist())
print("downsample:", downsample(np.arange(10), 3, phase=1).tolist())
```

Observed output:

```
ratios: (3, 2) (3, 2)
lengths: 8 8 8
tap count/delay/phases: 61 30 [21, 20, 20]
same taps/output: True True
output: [0.0, 0.72745807, 0.97255322, 0.0, -0.97255322, -0.72745807, -0.0, 0.10426563]
residuals: {'polyphase_reference': 0.0, 'dc_gain': 8.881784197001252e-16}
upsample: [1, 0, 0, 2, 0, 0, 3, 0, 0]
downsample: [1, 4, 7]
```

The final nonzero sample is a finite boundary transient, not a ratio-reduction failure. The certificate intentionally does not judge whether that transient is acceptable for a particular application.

When adapting this example, record the original sample rate and the reduced rate ratio, check the planned output length, and inspect both ends of the output as well as its interior. The direct reference is useful for confirming the implemented finite convention. Application-specific alias rejection and acceptable edge behavior still need their own criteria.

34.3 LMS, NLMS, and RLS

Let the tap count be p, and define the zero-prefixed regressor

$$u_n = (x_n, x_{n-1}, \dots, x_{n-p+1})^{\mathsf{T}}.$$

The routines use

$$y_n = w_n^* u_n, \qquad e_n = d_n - y_n.$$

Real inputs are promoted at least to floating point. Complex inputs use `numpy.vdot` and conjugate updates. Every result has

$$y, e \in \mathbb{K}^N, \qquad w_N \in \mathbb{K}^p, \qquad W_{\text{hist}} \in \mathbb{K}^{(N+1)\times p}.$$

The method alias, result record, and diagnostics are

```
AdaptiveMethod = Literal["lms", "nlms", "rls"]

AdaptiveFilterResult(
    output: np.ndarray,
    error: np.ndarray,
    final_weights: np.ndarray,
    weight_history: np.ndarray,
    method: AdaptiveMethod,
    certificate: Certificate,
) -> None

AdaptiveFilterResult.mean_squared_error -> float
AdaptiveFilterResult.tail_mean_squared_error(fraction: float = 0.25) -> float
```

They compute

$$\text{MSE} = \frac{1}{N}\sum_n |e_n|^2$$

and a tail MSE over `ceil(N * fraction)` final samples. The fraction must lie in $(0, 1]$.

Leaky LMS is

$$w_{n+1} = (1 - \mu\lambda) w_n + \mu u_n \overline{e_n}, \qquad \mu > 0, \quad \lambda \geq 0,$$

with API

```
lms_filter(
    input_signal: Any,
    desired_signal: Any,
    num_taps: int,
    *,
    step_size: float = 0.01,
    leakage: float = 0.0,
    initial_weights: Any | None = None,
) -> AdaptiveFilterResult
```

NLMS uses

$$g_n = \varepsilon + u_n^* u_n,$$

$$w_{n+1} = \left(1 - \frac{\mu\lambda}{g_n}\right) w_n + \frac{\mu}{g_n} u_n \overline{e_n}, \qquad \varepsilon > 0,$$

with API

```
nlms_filter(
    input_signal: Any,
    desired_signal: Any,
    num_taps: int,
    *,
    step_size: float = 0.5,
    epsilon: float = 1e-8,
    leakage: float = 0.0,
    initial_weights: Any | None = None,
) -> AdaptiveFilterResult
```

RLS initializes $P_0 = \delta I$, with $\delta > 0$, and for forgetting factor $0 < \gamma \leq 1$ applies

$$k_n = \frac{P_n u_n}{\gamma + u_n^* P_n u_n},$$

$$w_{n+1} = w_n + k_n \overline{e_n}, \qquad P_{n+1} = \gamma^{-1}(P_n - k_n u_n^* P_n).$$

The implementation symmetrizes each inverse covariance update:

$$P_{n+1} \leftarrow \frac{1}{2}(P_{n+1} + P_{n+1}^*).$$

Its API is

```
rls_filter(
    input_signal: Any,
    desired_signal: Any,
    num_taps: int,
    *,
    forgetting_factor: float = 0.99,
    delta: float = 1.0,
    initial_weights: Any | None = None,
) -> AdaptiveFilterResult
```

LMS and NLMS cost $O(Np)$ time. RLS costs $O(Np^2)$ time and retains the $p \times p$ inverse covariance during execution. All three intentionally store the complete $O(Np)$ weight history. There is no option to disable it.

Input and desired signals must be nonempty, finite, one dimensional, and equal in length. `num_taps` must be a positive integer, and initial weights must be a finite vector of exactly that length. LMS/NLMS require a positive finite step and nonnegative finite leakage. NLMS also requires positive finite `epsilon`. RLS requires `forgetting_factor` in $(0, 1]$, positive finite `delta`, and a nonvanishing gain denominator.

Every adaptive result certificate checks only that the finite recursion and stored trajectory are finite after validating declared parameter ranges. It records initial- and final-quarter MSE, plus method parameters. RLS also records the maximum Hermitian residual. It leaves persistent excitation, stochastic independence, and convergence for the supplied step or forgetting factor unchecked.

There is a source-metadata inconsistency in `1.0.0b4`: provenance identifiers are `KM-DSP-D12`, `KM-DSP-D13`, and `KM-DSP-D14` for LMS, NLMS, and RLS, respectively, but `AdaptiveFilterResult.certificate.theorem_id` is `KM-DSP-D12` for all three. Applications must treat the returned field as the runtime value rather than silently rewriting it.

Example 34-B: finite system identification

```
import numpy as np

from komorebi_math.signals import (
    fir_filter,
    lms_filter,
    nlms_filter,
    rls_filter,
)

rng = np.random.default_rng(12203)
x = rng.normal(size=4000)
target = np.array([0.7, -0.4, 0.2])
d = fir_filter(x, target, mode="causal")

results = {
    "LMS": lms_filter(x, d, 3, step_size=0.01),
    "NLMS": nlms_filter(x, d, 3, step_size=0.8),
    "RLS": rls_filter(x, d, 3, forgetting_factor=0.999, delta=10.0),
}

for name, result in results.items():
    result.certificate.require()
    print(
        name,
        np.round(result.final_weights, 8).tolist(),
        f"error={np.linalg.norm(result.final_weights - target):.3e}",
        f"mse={result.mean_squared_error:.3e}",
        f"tail={result.tail_mean_squared_error(0.2):.3e}",
        result.certificate.theorem_id,
    )
```

Observed output:

```
LMS [0.7, -0.4, 0.2] error=7.855e-16 mse=8.860e-03 tail=1.815e-30 KM-DSP-D12
NLMS [0.7, -0.4, 0.2] error=6.206e-17 mse=3.818e-04 tail=1.071e-32 KM-DSP-D12
RLS [0.69999866, -0.39999929, 0.19999963] error=1.558e-06 mse=2.759e-04 tail=6.082e-12 KM-DSP-D12
```

These numbers are a fixed-seed finite experiment, not a convergence theorem. In particular, lack of excitation and an excessive but still finite step are not parameter-validation failures. For example, NLMS on an all-zero input and all-one desired signal returns zero weights, MSE `1.0`, and a valid finite recursion certificate. In the same environment, twenty LMS samples with `step_size=10.0` produced MSE `9.014e+38` while the certificate remained valid because every stored value was still finite. By contrast, `forgetting_factor=1.1` raises `ValidationError: forgetting_factor must lie in (0, 1]`.

34.4 Streaming and chunk invariance

The state laws across Chapters 32 through 34 are different and should not be interchanged:

Facility	Mutable state	Chunk guarantee	Flush behavior
`StreamingFIR`	$K-1$ input-history samples	`process` concatenation equals causal FIR	emits $K-1$ full tail samples once
`StreamingLinearFilter`	direct-form state $s \in \mathbb{K}^q$	transported state equals batch recursion	no flush
`OverlapAddConvolver`	transform buffer, kernel spectrum, $K-1$ overlap	process plus flush equals full convolution	final block and tail once
`StreamingSTFT`	overlap/unframed sample buffer	emitted frames equal matching batch framing	optional terminal padded frame
`RationalResamplingPlan`	none. Immutable finite plan	no public chunk-invariance contract	batch only
LMS, NLMS, RLS	none exposed. Local recurrence state plus complete returned history	no public chunk-invariance contract	batch only

Splitting a resampling or adaptive-filter input into independent calls resets its finite boundary or recurrence state and is not guaranteed to reproduce one batch call. A caller can build a higher-level streaming system only by defining and testing the missing delay/phase or regressor/weight state itself. The library does not currently provide that contract.

The same device boundary applies here as in Chapter 33. Resampling and adaptive filtering are NumPy host implementations with no CUDA backend, device-array, stream, or workspace parameter. Their finite certificates do not extend to hardware qualification or real-time scheduling.

34.5 Verification evidence and known limitations

The focused unit suites exercise FIR modes and design validation, complex and irregular FIR streaming, independent recursive difference equations, state continuation, every biquad kind, unstable poles, real and complex FFT convolution, overlap-add block sizes from 1 through 256, irregular complex chunks, rational rates including reducible `6/4`, complex resampling, primitive phase conventions, and real and complex adaptive identification.

Against the recovered `1.0.0b4` source, the four directly relevant suites `test_block12_spectral_filtering.py`, `test_block12_recursive_fft.py`, `test_block12_resampling_adaptive.py`, and `test_signals_polyphase_filterbanks.py` pass 68 of 68 tests.

The recorded deterministic signal audit used seed `12012026` and reports `815779` of `815779` scalar checks passing. Relevant section counts are `93170` FIR, `142577` recursive, `413058` FFT-convolution, `27863` resampling, and `240` adaptive checks. Its environment record is Python `3.13.5`, NumPy `2.3.5`, and package version `0.12.0a1`. The algorithms remain present in the `1.0.0b4` source. The independent SciPy convention audit reports `520920` of `520920` comparisons passing and expressly limits its scope to matched finite conventions, not API identity.

Known limitations are therefore part of the contract: no general high-order IIR designer beyond the listed biquads, no automatic direct/FFT cost planner, no streaming rational resampler, no streaming adaptive-filter class, no stochastic convergence theorem, no application-specific alias-rejection certificate, no hard real-time guarantee, and no hardware-validated GPU DSP path.

35. Filter banks, wavelets, and packet trees

The finite signal APIs in this chapter turn analysis/synthesis operators into explicit matrices, pairwise Haar transforms, reversible lifting steps, complete binary packet trees, optimal additive prunings, and deterministic shrinkage pipelines. The common design rule is simple: every transform acts on a finite array under a declared boundary convention. None of these routines silently replaces the finite problem by an infinite convolution theorem.

Choose a representation. Start with one-level Haar analysis to understand low and high channels. Use multilevel Haar decomposition when only the low branch should be refined, and a packet tree when both branches should be refined. Best-basis selection chooses a complete pruning of that packet tree under a supplied additive cost. Explicit finite or periodic filter-bank matrices are most useful when the operator itself must be inspected; consult Section 35.8 before using them at a large signal length.

Three public modules supply the chapter:

```
from komorebi_math.signals import filterbanks, wavelets, denoise
```

Their objects are also re-exported from `komorebi_math.signals`. In version `1.0.0b4` the `signals` feature is implemented, but the names documented here are part of the research public surface rather than the curated `komorebi_math.beta` or `komorebi_math.stable` surfaces. Pin the package version if an application depends on these signatures.

The chapter uses these claim strengths:

- a matrix shape, packet cover, or dynamic-programming choice is an exact finite statement.
- An SVD, pseudoinverse, floating Haar roundtrip, or norm residual is numerical.
- A denoising comparison against a supplied clean array is an observed finite comparison, not a population-risk theorem.
- Universal and SURE thresholds retain their noise-model assumptions.

35.1 Periodic and finite filter banks

35.1.1 Periodic analysis and synthesis

Let the signal length be $N > 0$, let the decimation factor $D > 0$ divide N, and let there be C analysis filters h_c and C synthesis filters g_c. Circular convolution in this module means

$$(C_h x)[n] = \sum_{t=0}^{K-1} h[t]x[(n-t) \bmod N].$$

The downsampler keeps indices $0, D, 2D, \dots, N-D$. If Q_D is its $(N/D) \times N$ matrix and Q_D^{T} is the zero-inserting upsampler, channel c has matrices

$$A_c = Q_D C_{h_c}, \qquad S_c = C_{g_c} Q_D^{\mathsf{T}}.$$

The complete operators are

$$A = \begin{bmatrix} A_0 \\ \vdots \\ A_{C-1} \end{bmatrix} \in \mathbb{K}^{(CN/D)\times N}, \qquad S = \begin{bmatrix} S_0 & \cdots & S_{C-1} \end{bmatrix} \in \mathbb{K}^{N\times(CN/D)}.$$

`PeriodicFilterBank.analyze` computes Ax and returns it as C arrays of length N/D. `synthesize` concatenates those channels in channel order and computes Sc. The declared output length, when supplied, must equal the common channel length times D.

Perfect reconstruction is the finite matrix identity

$$SA = I_N.$$

The certificate stores the absolute spectral-norm defect

$$r_{\mathrm{PR}} = \|SA - I_N\|_2$$

and accepts exactly when $r_{\mathrm{PR}} \leq \tau$. This comparator is absolute. It does not multiply the tolerance by a signal or operator scale.

```
import numpy as np

from komorebi_math.signals import PeriodicFilterBank

bank = PeriodicFilterBank(
    analysis_filters=([1.0],),
    synthesis_filters=([1.0],),
    decimation=1,
)

x = np.arange(4.0)
channels = bank.analyze(x)
reconstructed = bank.synthesize(channels)
certificate = bank.perfect_reconstruction_certificate(length=4)

print(channels)
print(reconstructed)
print(certificate.residuals)
print(bank.frame_bounds(4))
```

Observed with NumPy `2.5.2`:

```
(array([0., 1., 2., 3.]),)
[0. 1. 2. 3.]
{'reconstruction_operator': 0.0}
(1.0, 1.0)
```

This identity bank is intentionally elementary: it exposes channel order and the circular finite convention without hiding phase choices inside a named infinite-wavelet design.

35.1.2 Frame diagnostics and the canonical synthesis matrix

If $A \in \mathbb{K}^{M \times N}$ has full column rank, its frame bounds are

$$\alpha = \sigma_{\min}(A)^2, \qquad \beta = \sigma_{\max}(A)^2,$$

and

$$\alpha \|x\|_2^2 \leq \|Ax\|_2^2 \leq \beta \|x\|_2^2.$$

The canonical dual synthesis operator is the Moore-Penrose pseudoinverse

$$S_{\mathrm{can}} = A^{\dagger} = (A^* A)^{-1} A^*$$

when the displayed full-column-rank formula applies. The method `PeriodicFilterBank.canonical_synthesis_matrix` always returns NumPy's pseudoinverse. It does not first prove full column rank and does not mutate the bank's declared synthesis filters. Callers that need reconstruction must check

$$\|A^{\dagger}A - I_N\|_2$$

or establish the rank separately.

There is an important finite-dimensional reporting boundary. NumPy returns only `min(M, N)` singular values. When $M < N$, the implemented `frame_bounds` method squares the smallest returned value even though the analysis map necessarily has a nontrivial nullspace and its mathematical lower frame bound on all of $\mathbb{K}^N$ is zero. Treat the reported lower value as a singular-value diagnostic until full column rank has been established.

35.1.3 General finite analysis/synthesis pairs

`FiniteFilterBank` removes the convolution structure and accepts an explicit analysis matrix

$$E \in \mathbb{K}^{m \times n}$$

together with an explicit synthesis matrix

$$R \in \mathbb{K}^{n \times m}.$$

It computes

$$c = Ex, \qquad \hat{x} = Rc, \qquad \mathcal{R} = RE.$$

`report()` returns the squares of the smallest and largest singular values returned by NumPy, the absolute spectral defect

$$r = \|RE - I_n\|_2,$$

and a numerical certificate. `canonical_dual()` is stricter than the periodic pseudoinverse helper: it checks `matrix_rank(E) == n` and raises `ValidationError` when the analysis matrix is rank deficient.

For perturbations ΔE and ΔR, the package evaluates

$$\begin{aligned}\|(R + \Delta R)(E + \Delta E) - I\|_2 \le \|RE - I\|_2 + \|\Delta R\|_2\|E\|_2 \\ + \|R\|_2\|\Delta E\|_2 + \|\Delta R\|_2\|\Delta E\|_2.\end{aligned}$$

The method returns the right-hand side. It is an operator-norm upper bound, not a recomputed perturbed reconstruction error.

```
import numpy as np

from komorebi_math.signals import FiniteFilterBank

np.set_printoptions(precision=6, suppress=True)

E = np.array(
    [
        [1.0, 0.0],
        [0.0, 1.0],
        [1.0, 1.0],
    ]
)

bank = FiniteFilterBank(E, np.linalg.pinv(E)).canonical_dual()
report = bank.report()

x = np.array([2.0, -1.0])
print(bank.synthesis_matrix)
print(report.lower_frame_bound, report.upper_frame_bound)
```

```
print(report.reconstruction_residual)
print(bank.synthesize(bank.analyze(x)))
```

Observed output:

```
[[ 0.666667 -0.333333  0.333333]
 [-0.333333  0.666667  0.333333]]
1.0 2.9999999999999996
3.5502063685515195e-16
[ 2. -1.]
```

The rank check and the compatible finite dimensions are exact structural facts. The pseudoinverse entries, singular values, and residual are numerical.

35.2 Haar analysis and synthesis

`HaarFilterBank` acts on adjacent pairs. For (x_{2j}, x_{2j+1}), normalized analysis is

$$a_j = \frac{x_{2j} + x_{2j+1}}{\sqrt{2}}, \qquad d_j = \frac{x_{2j} - x_{2j+1}}{\sqrt{2}},$$

and synthesis is

$$x_{2j} = \frac{a_j + d_j}{\sqrt{2}}, \qquad x_{2j+1} = \frac{a_j - d_j}{\sqrt{2}}.$$

Consequently,

$$\|x\|_2^2 = \|a\|_2^2 + \|d\|_2^2.$$

With `normalized=False`, analysis omits the denominator and synthesis divides by two:

$$a_j = x_{2j} + x_{2j+1}, \qquad d_j = x_{2j} - x_{2j+1}, \qquad (x_{2j}, x_{2j+1}) = \left(\frac{a_j + d_j}{2}, \frac{a_j - d_j}{2} \right).$$

The unnormalized transform reconstructs, but its coefficient energy is twice the signal energy. `HaarFilterBank.certificate` therefore requires the energy residual only when `normalized=True`.

The boundary convention here is finite adjacent pairing, not circular convolution: the input length must be even, and the pair at the right boundary is the final two samples. Low and high channels must be equal one-dimensional arrays for synthesis.

```
import numpy as np

from komorebi_math.signals import HaarFilterBank, haar_analyze, haar_synthesize

np.set_printoptions(precision=6, suppress=True)

x = np.array([3.0, 1.0, -1.0, 5.0, 2.0, 0.0, 4.0, -2.0])
low, high = haar_analyze(x)
checked = HaarFilterBank().certificate(x)

print(low)
print(high)
print(haar_synthesize(low, high))
print(checked.certificate.residuals)
```

Observed output:

```
[2.828427 2.828427 1.414214 1.414214]
[ 1.414214 -4.242641  1.414214  4.242641]
[ 3.  1. -1.  5.  2.  0.  4. -2.]
{'reconstruction': 1.1430445635548515e-15,
 'energy': 1.4210854715202004e-14}
```

`haar_matrix(N)` constructs the one-level analysis matrix by applying the transform to every coordinate vector. `haar_filter_bank(N)` inverts that matrix numerically and returns a `FiniteFilterBank`. These conveniences require a positive even N. The direct `HaarFilterBank.analyze` method checks only one-dimensionality and evenness.

35.3 Lifting schemes

35.3.1 Periodic predict and update

Split an even signal into the subsequences

$$e_j = x_{2j}, \qquad o_j = x_{2j+1}, \qquad 0 \le j < N/2.$$

For a finite filter $p = (p_0, \dots, p_{K_p-1})$, the implementation's circular subsequence filter is

$$(Pe)_j = \sum_{t=0}^{K_p-1} p_t e_{(j-t) \bmod (N/2)}.$$

The update filter U uses the same convention. Analysis performs

$$d = o - Pe, \qquad s = e + Ud.$$

Synthesis reverses the operations in the necessary order:

$$e = s - Ud, \qquad o = d + Pe,$$

and interleaves e, o. This algebra is reversible for arbitrary finite predict and update filters. Floating arithmetic still makes the returned certificate numerical.

`periodic_five_three_lifting()` returns

$$p = (1/2, 1/2), \qquad u = (1/4, 1/4).$$

Thus the implemented phase convention is

$$d_j = o_j - \tfrac{1}{2}e_j - \tfrac{1}{2}e_{j-1}, \qquad s_j = e_j + \tfrac{1}{4}d_j + \tfrac{1}{4}d_{j-1},$$

with indices taken modulo $N/2$. This explicit formula is safer than assuming the phase convention of another library's $5/3$ transform.

```
import numpy as np

from komorebi_math.signals import periodic_five_three_lifting

np.set_printoptions(precision=6, suppress=True)
x = np.array([3.0, 1.0, -1.0, 5.0, 2.0, 0.0, 4.0, -2.0])

scheme = periodic_five_three_lifting()
smooth, detail = scheme.analyze(x)
checked = scheme.certificate(x)
```

```
print(smooth)
print(detail)
print(checked.value)
print(checked.certificate.residuals)
```

For the preceding eight-sample `x`, the observed output is

```
[ 1.125 -0.625  2.875  2.625]
[-2.5  4.  -0.5 -5. ]
[ 3.  1. -1.  5.  2.  0.  4. -2.]
{'lifting_roundtrip': 0.0}
```

`PeriodicLiftingScheme.synthesize` accepts either one iterable containing `(smooth, detail)` or two positional channels. In both forms the channels must be equal one-dimensional arrays.

35.4 Haar packet trees and pruning

35.4.1 Complete trees

A packet path is either a binary string or a tuple of binary integers. The root is the empty string `""`. Appending `"0"` selects the low Haar child and appending `"1"` selects the high child. At depth J, a complete tree has

$$1 + 2 + \cdots + 2^J = 2^{J+1} - 1$$

nodes. Every depth-j level contains N coefficients in total, so the stored coefficient count across the complete tree is $(J + 1)N$.

`HaarPacketTree.analyze(x, J)` requires a nonempty one-dimensional input and

$$2^J \mid N.$$

Unlike an ordinary multilevel discrete wavelet transform, packet analysis splits both the low and high child at every nonterminal node.

The principal tree views are:

Member	Meaning
`signal`	copied root signal
`depth`	complete binary depth
`normalized`	local Haar normalization flag
`nodes`	shallow copy of the complete path-to-array dictionary
`coefficients`	compatibility alias returning the same kind of copy
`coefficient(path)`	coefficient array at one validated path
`paths_at_depth(j)`	sorted paths at depth j
`full_leaves`	all sorted paths at the terminal depth

`HaarPacketTree.decompose` is an alias of `analyze`.

35.4.2 Valid prunings

A selected path set L is a complete pruning exactly when

1. every selected path is a node of the tree.
2. No selected path is the ancestor of another selected path.
3. Every full-depth path has exactly one selected ancestor.

Equivalently, L is a prefix-free complete cover of the terminal leaves. `validate_pruning` deduplicates the supplied paths, sorts them, and returns the canonical tuple. The root alone, `("",)`, and all terminal leaves are both valid prunings.

Reconstruction descends from the root. At a selected path it returns the stored coefficients. At an unselected internal path it recursively reconstructs both children and applies Haar synthesis. `PrunedPacketCoefficients` stores a validated pruning together with its source tree and exposes the chosen coefficient dictionary, reconstruction, energy, and certificate.

For normalized Haar and any complete pruning,

$$\|x\|_2^2 = \sum_{p \in L} \|c_p\|_2^2.$$

The reconstruction certificate stores absolute Euclidean reconstruction and absolute energy residuals. The energy term is required only for a normalized tree.

```
import numpy as np

from komorebi_math.signals import HaarPacketTree, packet_energy

x = np.arange(16.0)
tree = HaarPacketTree.analyze(x, depth=3)
pruned = tree.prune(("0", "10", "110", "111"))

print(tree.full_leaves)
print(pruned.leaves)
print(np.linalg.norm(pruned.reconstruct() - x))
print(packet_energy(pruned), np.vdot(x, x).real)
```

`haar_packet_synthesize(tree, leaves)` and `packet_energy(tree, leaves)` are functional wrappers. When passed a `PrunedPacketCoefficients` object, both wrappers reject an additional `leaves` argument because the pruning is already fixed.

35.5 Best-basis selection and costs

35.5.1 Additive dynamic programming

Let $C(p, c_p)$ be a finite real cost for node p. The minimum additive pruning cost below p satisfies

$$B(p) = \begin{cases} C(p, c_p), & |p| = J, \\ \min\{C(p, c_p), B(p0) + B(p1)\}, & |p| < J. \end{cases}$$

`best_basis(tree, cost)` evaluates this recurrence bottom-up. `cost` can be a mapping containing every visited path, a callable `cost(coefficients, path)`, or a one-argument callable `cost(coefficients)`. Every returned cost must be finite.

The tie policy is deterministic:

$$C(p, c_p) \le B(p0) + B(p1) \quad \Longrightarrow \quad \text{keep } p.$$

The reported margin is

$$\Delta_p = \left| C(p, c_p) - B(p0) - B(p1) \right|$$

for internal nodes and infinity for terminal nodes. A small margin means a small cost perturbation can change the local choice. `BestBasisResult` stores the selected `leaves`, optimum `cost`, every node's `decisions` and `margins`, and an exact certificate for the finite dynamic-programming selection.

That exactness label does not convert floating cost evaluation into exact arithmetic. It means only that, for the finite values supplied to the recurrence, the returned pruning attains its additive minimum with the stated tie policy.

35.5.2 Built-in costs

For coefficient energies $e_j = |c_j|^2$ and total $E = \sum_j e_j$, entropy cost is

$$C_{\text{ent}}(c) = -\sum_{j : q_j > 0} q_j \log q_j, \qquad q_j = e_j / E.$$

The zero vector has cost zero. `entropy_cost` implements this formula. `shannon_entropy_cost(coefficients, path="")` is a compatibility wrapper that calls `entropy_cost`. It is behaviorally equivalent but is not the same function object.

`lp_cost(p)` requires finite $p > 0$ and returns the callable

$$C_p(c) = \sum_j |c_j|^p.$$

The name is a positive-power coefficient cost. It does not claim to be a norm when $0 < p < 1$.

```python
import numpy as np

from komorebi_math.signals import (
    HaarPacketTree,
    best_basis,
    entropy_cost,
    lp_cost,
)

tree = HaarPacketTree.analyze(np.arange(16.0), depth=2)
costs = {path: 10.0 for path in tree.nodes}
costs.update({"": 9.0, "0": 1.0, "10": 1.0, "11": 1.0})

result = best_basis(tree, costs)
entropy_result = best_basis(tree, entropy_cost)
l1_result = best_basis(tree, lp_cost(1.0))

print(result.leaves, result.cost)
print(result.decisions)
print(result.margins)
print(entropy_result.leaves, entropy_result.cost)
print(l1_result.leaves, l1_result.cost)
```

Observed output:

```
('0', '10', '11') 3.0
{'00': 'keep', '01': 'keep', '0': 'keep',
 '10': 'keep', '11': 'keep', '1': 'split', '': 'split'}
{'00': inf, '01': inf, '0': 19.0,
 '10': inf, '11': inf, '1': 8.0, '': 6.0}
('',) 2.309540025247866
('00', '01', '10', '11') 71.99999999999999
```

For the selected mapping-cost basis, reconstruction had Euclidean residual `6.425400768188728e-15`. Selected energy was `1239.9999999999998` against root energy `1240.0`.

35.6 Denoising and risk certificates

35.6.1 Multilevel Haar decomposition

`haar_decompose(x, levels=J)` follows only the low branch. Starting from $a^{(0)} = x$, it computes

$$(a^{(j)}, d^{(j)}) = \mathrm{HaarAnalyze}(a^{(j-1)}), \qquad 1 \le j \le J.$$

The stored detail tuple is ordered from finest to coarsest:

$$(d^{(1)}, d^{(2)}, \dots, d^{(J)}).$$

The final approximation is $a^{(J)}$. The signal must be nonempty, finite, one dimensional, and divisible in length by 2^J, with $J > 0$.

`HaarDecomposition.reconstruct()` traverses the details in reverse order. Supplying a replacement detail tuple requires one correctly shaped array for every level. Its certificate checks

$$r_x = \|\hat{x} - x\|_2$$

and, for normalized Haar,

$$r_E = \left| \|a^{(J)}\|_2^2 + \sum_{j=1}^{J} \|d^{(j)}\|_2^2 - \|x\|_2^2 \right|.$$

Unlike the one-level Haar certificate, this comparator is scaled. Both residuals are compared with

$$\tau \max\{1, \|x\|_2, \|x\|_2^2\}.$$

35.6.2 Noise scale and threshold rules

For a finite vector v,

$$\mathrm{MAD}(v; c) = \mathrm{median}_j \, |v_j - c|.$$

`median_absolute_deviation` uses the sample median as c unless a finite center is supplied. The white-noise scale helper fixes $c = 0$ and returns

$$\hat{\sigma} = \frac{\mathrm{median}_j \, |d_j|}{0.6744897501960817}.$$

The universal threshold for $n > 0$ coefficients is

$$\lambda_{\text{univ}} = \sigma\sqrt{2\log n}.$$

For SURE, let

$$u_{(1)} \le \cdots \le u_{(n)}$$

be the sorted squared normalized magnitudes $u_j = (|d_j|/\sigma)^2$. For candidate rank k, the implementation evaluates

$$R_k = n - 2k + \sum_{j=1}^{k} u_{(j)} + (n-k)u_{(k)}$$

and returns

$$\lambda_{\text{SURE}} = \sigma\sqrt{u_{(k_*)}}, \qquad k_* = \operatorname*{argmin}_k R_k.$$

When $\sigma = 0$, `sure_threshold` returns zero immediately. This is a finite selection among observed coefficient magnitudes. Its usual statistical interpretation requires the relevant noise model.

35.6.3 Shrinkage maps

For $\lambda \ge 0$, the three supported modes are

$$T_{\text{hard}}(z) = z\mathbf{1}_{|z|\ge\lambda},$$

$$T_{\text{soft}}(z) = z\frac{\max(|z| - \lambda, 0)}{|z|},$$

The corresponding garrote rule is

$$T_{\text{garrote}}(z) = z\frac{\max(|z|^2 - \lambda^2, 0)}{|z|^2}.$$

The implementation uses zero-safe division in the last two formulas. For complex coefficients, both maps preserve the phase of retained values.

`wavelet_denoise` never thresholds the final approximation. It thresholds each stored detail array and then synthesizes. Its rule behavior is:

- `"fixed"`: required explicit nonnegative finite `threshold`.
- `"universal"`: noise scale times the universal factor for each detail length.
- `"sure"`: finite SURE selection within each detail array.

When `sigma=None`, the common default scale is estimated from the first, finest detail. With `level_dependent=True`, and only when `sigma` is not supplied, each detail gets its own MAD estimate. A supplied `sigma` is used for every level. An explicit `threshold` is rejected unless `rule="fixed"`.

The returned `WaveletDenoiseResult` contains the reconstructed `signal`, source `decomposition`, `thresholded_details`, one `thresholds` entry per level, `noise_sigma`, normalized `rule` and `mode` strings, and a certificate. Its `removed_energy` property is

$$\sum_{j=1}^{J} \|d^{(j)} - \tilde{d}^{(j)}\|_2^2.$$

The shrinkage certificate checks only that the reconstructed array is finite and that coefficient magnitudes did not increase. Its fixed internal tolerance is `1e-12`. `wavelet_denoise` does not expose that tolerance as a parameter. The certificate intentionally leaves unchecked the white-Gaussian interpretation or the applicability of a supplied scale and optimality beyond the selected finite rule.

35.6.4 Observed risk is not population risk

Given a supplied clean reference x, noisy observation y, and estimate $\hat{x}$, `denoising_risk_certificate` computes

$$e_{\text{noisy}} = \frac{1}{N}\sum_{n=0}^{N-1} |y_n - x_n|^2, \qquad e_{\text{denoised}} = \frac{1}{N}\sum_{n=0}^{N-1} |\hat{x}_n - x_n|^2.$$

It is valid exactly when

$$e_{\text{denoised}} \le e_{\text{noisy}} + \tau, \qquad \tau \ge 0.$$

This is an observed finite error comparison conditioned on the supplied clean array. It does not estimate unseen risk, prove distributional optimality, or certify that another signal will improve.

```
import numpy as np

from komorebi_math.signals import (
    denoising_risk_certificate,
    wavelet_denoise,
)

rng = np.random.default_rng(12303)
clean = np.repeat(np.array([0.0, 2.0, -1.0, 1.5]), 256)
noisy = clean + rng.normal(scale=0.35, size=clean.size)

result = wavelet_denoise(
    noisy,
    levels=5,
    rule="universal",
    mode="soft",
)
risk = denoising_risk_certificate(clean, noisy, result.signal)

print(result.noise_sigma)
print(result.thresholds)
print(result.removed_energy)
print(result.certificate.assumptions_unchecked)
print(risk.valid, risk.residuals)
```

Observed output:

```
0.3803110080909681
(1.3433459777978087, 1.2665187338406718,
 1.1847197939906322, 1.0968373978749237,
 1.0012709745522745)
131.82889808658055
('Gaussian white-noise interpretation',
 'threshold optimality outside the selected finite risk rule')
True {'noisy_mse': 0.1328856705568,
      'denoised_mse': 0.0038218147825770732}
```

The clean reference makes this example reproducible and makes its claim narrow: this one finite realization improved under the displayed metric.

35.7 `signals.filterbanks` periodic, finite, Haar, lifting, canonical-dual, and perturbation APIs

35.7.1 `signals.filterbanks`

The module exports exactly eleven names.

- **`PeriodicFilterBank`:** `(analysis_filters: Iterable[Any], synthesis_filters: Iterable[Any], decimation: int)`. Periodic structured bank.
- **`HaarFilterBank`:** `(normalized: bool = True)`. One-level adjacent-pair Haar bank.
- **`PeriodicLiftingScheme`:** `(predict: Any, update: Any)`. Reversible circular lifting pair.
- **`FiniteFilterBank`:** `(analysis_matrix: Any, synthesis_matrix: Any)`. Explicit finite analysis/synthesis pair.
- **`FilterBankReport`:** immutable diagnostic record with fields `lower_frame_bound`, `upper_frame_bound`, `reconstruction_residual`, and `certificate`.
- **`haar_analyze`:** `(signal: Any, *, normalized: bool = True)`. Returns `(low, high)`.
- **`haar_synthesize`:** `(low: Any, high: Any, *, normalized: bool = True)`. Reconstructed array.
- **`haar_matrix`:** `(length: int, *, normalized: bool = True)`. One-level analysis matrix.
- **`haar_filter_bank`:** `(length: int, *, normalized: bool = True)`. Returns a `FiniteFilterBank`.
- **`periodic_five_three_lifting`:** `()`. Preset `PeriodicLiftingScheme`.
- **`reconstruction_error`:** `(bank: FiniteFilterBank)`. Absolute spectral defect.

`PeriodicFilterBank` members:

- `analysis_filters`, `synthesis_filters`, and `decimation`: stored immutable constructor fields.
- `channels`: number of filters in either bank.
- `analysis_matrix(length: int) -> ndarray`.
- `synthesis_matrix(length: int) -> ndarray`.
- `analyze(signal: Any) -> tuple[ndarray, ...]`.
- `synthesize(coefficients: Iterable[Any], *, length: int | None = None) -> ndarray`.
- `reconstruction_operator(length: int) -> ndarray`.
- `perfect_reconstruction_certificate(length: int, *, tolerance: float = 1e-10) -> Certificate`.
- `frame_bounds(length: int) -> tuple[float, float]`.
- `canonical_synthesis_matrix(length: int, *, rcond: float | None = None) -> ndarray`.
- `reconstruction_error_bound(length: int) -> float`.

`FiniteFilterBank` members:

- `analysis_matrix` and `synthesis_matrix`: stored immutable arrays.
- `input_dimension`: number of analysis columns.
- `coefficient_dimension`: number of analysis rows.
- `analyze(signal: Any) -> ndarray`.
- `synthesize(coefficients: Any) -> ndarray`.
- `reconstruction_operator() -> ndarray`.
- `report(*, tolerance: float = 1e-10) -> FilterBankReport`.
- `canonical_dual(*, rcond: float | None = None) -> FiniteFilterBank`.
- `perturbation_bound(analysis_error: Any, synthesis_error: Any) -> float`.

`HaarFilterBank` supplies `analyze(signal)`, `synthesize(coefficients)`, and `certificate(signal, *, tolerance=1e-12)`. `PeriodicLiftingScheme` supplies `analyze(signal)`, `synthesize(coefficients, detail_channel=None)`, and `certificate(signal, *, tolerance=1e-11)`.

All five records are frozen slotted dataclasses, but stored NumPy arrays remain mutable objects. Dataclass immutability does not make their buffers read-only.

35.7.2 `signals.wavelets`

Public symbol	Signature / role
`PacketPath`	`str | tuple[int, ...]` type alias
`CostFunction`	callable type alias
`HaarPacketTree`	complete tree record. Construct with `analyze` or `decompose`
`PrunedPacketCoefficients`	`(tree, leaves)` validated pruning record
`BestBasisResult`	`(leaves, cost, decisions, margins, certificate)` result record
`best_basis`	`(tree, cost) -> BestBasisResult`
`entropy_cost`	`(coefficients, _="") -> float`
`shannon_entropy_cost`	`(coefficients, path="") -> float` compatibility wrapper
`lp_cost`	`(exponent: float) -> CostFunction`
`haar_packet_analyze`	`(signal, depth, *, normalized=True) -> HaarPacketTree`
`haar_packet_synthesize`	`(tree_or_coefficients, leaves=None) -> ndarray`
`packet_energy`	`(tree_or_coefficients, leaves=None) -> float`

Use `HaarPacketTree.analyze` instead of manually supplying the dataclass's public low-level fields `signal`, `depth`, `normalized`, and `_nodes`. The classmethod constructs the complete invariant that reconstruction assumes.

35.7.3 `signals.denoise`

Public symbol	Exact signature or role
`ThresholdMode`	`Literal["soft", "hard", "garrote"]`
`ThresholdRule`	`Literal["universal", "sure", "fixed"]`
`median_absolute_deviation`	`(values, *, center=None) -> float`
`estimate_white_noise_sigma`	`(detail_coefficients) -> float`
`universal_threshold`	`(sigma: float, coefficient_count: int) -> float`
`sure_threshold`	`(coefficients, sigma: float) -> float`
`threshold_coefficients`	`(coefficients, threshold, *, mode="soft") -> ndarray`
`haar_decompose`	`(signal, levels, *, normalized=True) -> HaarDecomposition`
`wavelet_denoise`	`(signal, *, levels=1, mode="soft", rule="universal", threshold=None, sigma=None, level_dependent=False, normalized=True) -> WaveletDenoiseResult`
`denoising_risk_certificate`	`(clean_signal, noisy_signal, denoised_signal, *, tolerance=0.0) -> Certificate`

`HaarDecomposition` fields are `approximation`, `details`, `normalized`, and `original_length`. Its public members are `levels`, `reconstruct(details=None)`, and `certificate(original, *, tolerance=1e-11)`. `WaveletDenoiseResult` fields are `signal`, `decomposition`, `thresholded_details`, `thresholds`, `noise_sigma`, `rule`, `mode`, and `certificate`. `removed_energy` is its one derived property.

35.7.4 Certificate ledger

- **`PeriodicFilterBank.perfect_reconstruction_certificate`:** theorem `PSC-V-T09`. Numerical. `reconstruction_operator` under an absolute operator norm.
- **`FiniteFilterBank.report`:** theorem `PSC-V-T09`. Numerical. `reconstruction` under an absolute operator norm.
- **`HaarFilterBank.certificate`:** theorem `PSC-V-T10`. Numerical. `reconstruction` and `energy` as absolute residuals.
- **`PeriodicLiftingScheme.certificate`:** theorem `PSC-V-T11`. Numerical. `lifting_roundtrip` as an absolute Euclidean residual.

- **`HaarPacketTree.reconstruction_certificate`:** theorem `PSC-V-T13`. Numerical. `reconstruction` and `energy` as absolute residuals.
- **`best_basis`:** theorem `PSC-V-T14`. Exact finite additive recursion. No residual map.
- **`HaarDecomposition.certificate`:** theorem `KM-DSP-T06`. Numerical. `haar_reconstruction` and `haar_energy` use a scaled comparator.
- **`wavelet_denoise`:** theorem `KM-DSP-D10`. Numerical. `shrinkage_monotonicity` with a fixed internal tolerance.
- **`denoising_risk_certificate`:** theorem `KM-DSP-C03`. Numerical. `noisy_mse` and `denoised_mse` are an observed clean-reference comparison.

The two `PSC-V-T09` producers share a reconstruction theorem identifier but use different residual-key names. Consumers should therefore read certificate fields by producer rather than assuming one global residual schema.

35.8 Cost and storage model

These counts describe the shipped dense algorithms. They are operation and storage models, not timing measurements.

- Build one circular convolution matrix: work $O(NK)$ for K taps. Storage N^2.
- Periodic analysis after matrices exist: work $O(CN^2/D)$. Storage CN/D output values.
- Periodic synthesis after matrices exist: work $O(CN^2/D)$. Storage N output values.
- Periodic perfect-reconstruction operator: dense SA, work $O(CN^3/D)$. Storage N^2.
- SVD or pseudoinverse of an $M \times N$ analysis matrix: work $O(MN \min(M, N))$. Dense factorization-dependent storage.
- Finite-bank analysis or synthesis: work $O(mn)$. Output-only additional storage.
- One-level Haar: work $O(N)$. Storage N coefficients.
- Construct `haar_matrix(N)`: work $O(N^2)$. Storage N^2.
- Construct `haar_filter_bank(N)`: dense-inverse work $O(N^3)$. Storage two N^2 matrices.
- Lifting with K_p, K_u taps: work $O(N(K_p + K_u))$. Storage N coefficients.
- Complete depth-J packet tree: work $O(JN)$. Storage $(J + 1)N$ coefficients plus the node dictionary.
- Best basis with linear coefficient costs: work $O(JN)$. Storage $O(2^J)$ decisions and margins.
- Multilevel low-branch Haar decomposition: geometric-total work $O(N)$. Storage N coefficients.
- Threshold and synthesis: work $O(N)$. Storage $O(N)$ result and coefficient records.

The periodic bank is intentionally an explicit-matrix reference API. It does not use FFT convolution, polyphase kernels, sparse storage, or GPU execution. Large N therefore incurs quadratic matrix storage and dense matrix costs. The Haar, lifting, packet, and denoising routines use direct vector operations.

35.9 Boundaries, failures, and source-observed defects

35.9.1 Declared failure surface

Condition	Public behavior
empty filter bank	`ValidationError`
empty or non-1-D individual filter	`DimensionError`
unequal analysis/synthesis channel counts	`DimensionError`
nonpositive decimation after integer conversion	`ValidationError`
periodic length not divisible by decimation	`DimensionError`
periodic signal or coefficient channel not 1-D	`DimensionError`
wrong channel count, unequal channel lengths, or inconsistent declared output length	`DimensionError`
nonmatrix finite-bank operator or incompatible transpose shapes	`DimensionError`
finite signal or coefficient dimension mismatch	`DimensionError`
rank-deficient finite canonical dual	`ValidationError`
perturbation shape mismatch	`DimensionError`

Condition	Public behavior
odd Haar or lifting analysis length	`DimensionError`
unequal Haar or lifting coefficient shapes	`DimensionError`
noninteger, negative, or Boolean packet depth	`ValidationError`
packet input empty, not 1-D, or not divisible by 2^J	`DimensionError`
nonbinary or unknown packet path	`ValidationError`
empty, overlapping, or incomplete pruning	`ValidationError`
missing mapping cost or nonfinite node cost	`ValidationError`
nonpositive or nonfinite p in `lp_cost`	`ValidationError`
denoising signal empty, non-1-D, or nonfinite	`DimensionError` or `ValidationError`
nonpositive level or incompatible dyadic length	`ValidationError` or `DimensionError`
negative/nonfinite threshold or sigma	`ValidationError`
unknown shrinkage mode or threshold rule	`ValidationError`
fixed rule without threshold, or explicit threshold with another rule	`ValidationError`
unequal clean/noisy/denoised lengths	`DimensionError`
negative/nonfinite risk tolerance	`ValidationError`

`HaarFilterBank.synthesize` and `PeriodicLiftingScheme.synthesize` unpack two channels before their dimensional checks. Supplying other than exactly two can therefore raise Python `ValueError`, not a package `DimensionError`.

35.9.2 Recorded implementation boundaries

These behaviors were reproduced directly against version `1.0.0b4`. They are boundaries to work around, not promises to preserve:

- `PeriodicFilterBank` converts `decimation` with `int()` before checking it. For example `decimation=1.9` is stored as `1`. `haar_matrix(2.9)` similarly constructs a length-two matrix, and `universal_threshold(..., coefficient_count=1.9)` uses count one. Validate integer-valued public inputs before these calls.
- A tuple packet path is converted elementwise with `int()`. Thus `(0.9, 1.9)` is currently read as path `"01"`. Supply actual integers or binary strings.
- The periodic, one-level Haar, lifting, finite-bank, packet-tree, and decomposition certificate/report tolerances are not validated. A negative tolerance merely makes an otherwise exact residual certificate invalid. Nonfinite tolerances can produce misleading comparisons. Pass a finite nonnegative tolerance. The separate denoising-risk helper does validate its tolerance.
- Filter-bank constructors do not reject nonfinite matrix or filter entries. Linear algebra may propagate `NaN`/`Inf` or fail downstream.
- Direct one-level Haar and lifting analysis accept an empty one-dimensional array because zero is even. Their convenience matrix/tree and denoising entry points do enforce nonempty or positive sizes.
- `PeriodicFilterBank.frame_bounds` and `FiniteFilterBank.report` can report a positive lower value for an undercomplete analysis matrix. For example $E = [1\ 0]$ reports `(1.0, 1.0)` even though the true lower frame bound on $\mathbb{R}^2$ is zero. Establish full column rank before interpreting the lower number as a frame bound.
- `FiniteFilterBank.report().certificate` has numerical exactness, but its success message says `"finite filter bank reconstructs exactly"`. Read the `Exactness.NUMERICAL` field and residual, not that wording, as the claim strength.
- `sure_threshold` does not validate coefficient finiteness when called directly: inputs containing `NaN` or `Inf` can return a nonfinite threshold and may emit a NumPy warning. `wavelet_denoise` screens its input signal for finiteness, but direct callers should screen coefficient arrays too.
- The two-argument best-basis callback adapter catches any `TypeError` and then retries the callable with one argument. A `TypeError` raised inside a valid two-argument callback can consequently be masked by a misleading missing-argument error. Keep cost callbacks small and test them independently.

No certificate in this chapter verifies perceptual quality, stationarity, independence, whiteness, Gaussianity, compression quality, application-specific frequency selectivity, or an asymptotic wavelet theorem. Those claims require separate hypotheses and evidence.

35.10 Verification evidence

The chapter's observed examples were executed from the recovered `1.0.0b4` source tree with Python `3.12.13` and NumPy `2.5.2`. Additional checks were also run:

- 25 focused filter-bank, Haar, lifting, packet-tree, best-basis, threshold, and denoising tests passed. Seven unrelated tests in the three selected test files were deselected.
- `verification/block4_deep_audit.py` passed all 18,068 checks, including 360 filter-bank and 480 wavelet checks.
- `verification.block12_deep_audit.run()` passed all 815,779 checks, including 15,944 denoising checks.
- Public signatures were reflected from the three shipped modules and matched the ledgers above.

These executions demonstrate the finite cases covered by the suites. They do not close the source-observed validation gaps in §35.9.2.

> **Chapter 35 summary.** Use `PeriodicFilterBank` when the circular convolution phase and explicit dense matrices are part of the model. Use `FiniteFilterBank` for a general finite analysis/synthesis pair and enforce full column rank before calling a pseudoinverse a canonical dual. Haar and lifting provide linear-time reversible local transforms. Packet trees split both branches, valid prunings are complete prefix-free covers, and `best_basis` exactly minimizes the supplied finite additive costs with a keep-on-tie policy. Denoising thresholds only detail channels, and its risk certificate compares one supplied finite clean reference. Always inspect the residual, exactness, and unchecked assumptions before promoting a finite computation to a stochastic or infinite-dimensional claim.

36. Singular packets and recovery

The singular-packet layer separates a sampled signal into a regular component and a finite family of explicitly represented singular terms. It can then approximate only the regular component and restore either the exact packet or an estimate. The accompanying jump-recovery routines are intentionally conditional: they check a finite contrast gap, but they do not infer the unknown signal's smoothness, noise law, jump size, or separation from the samples.

Choose the information you actually have. With a known packet, use the retained-packet reconstruction route and judge the approximation of the remaining smooth component. With an estimated packet, keep the packet-error term visible. With only samples, the jump-recovery routines need declared smoothness, noise, jump-size, and separation information. Their output cannot establish those declarations for unknown data. The worked example in Section 36.4 uses known synthetic parameters so each comparison has a reference.

These APIs are in the broad research surface:

```
from komorebi_math import (
    SingularPacket1D,
    recover_separated_jumps,
    retained_packet_reconstruct,
)

from komorebi_math.signals import (
    TruncatedPowerTerm,
    recover_vector_jumps,
)
```

Only `SingularPacket1D`, `recover_separated_jumps`, and `retained_packet_reconstruct` are re-exported from `komorebi_math` itself. The complete surface is exported by `komorebi_math.signals`. None of the names in this chapter is promoted into the curated `komorebi_math.stable` or `komorebi_math.beta` namespace in version `1.0.0b4`. Pin the package version if these signatures are part of an application contract.

36.1 Truncated-power packets

For a location $\xi \in \mathbb{R}$, amplitude $a \in \mathbb{C}$, and integer order $r \geq 0$, a packet term is

$$T_{\xi,a,r}(x) = a\frac{(x-\xi)_+^r}{r!}.$$

At order zero the implementation defines the basis separately as $\mathbf{1}_{x\geq\xi}$. Thus the value at the jump is the right-hand value, not zero and not one half. A finite packet is

$$j(x) = \sum_{k=1}^{J} T_{\xi_k,a_k,r_k}(x).$$

The primary types are frozen, slotted dataclasses.

Type	Public members	Meaning
`TruncatedPowerTerm`	`location: float, amplitude: complex, order: int = 0`	One validated finite term. Construction normalizes the amplitude to Python `complex`, the location to `float`, and the order to `int`.
`SingularPacket1D`	`components: tuple[TruncatedPowerTerm, ...]`	A finite, possibly empty sum. Every component must already be a `TruncatedPowerTerm`.

Type	Public members	Meaning
`RetainedPacketResult`	`smooth_component`, `smooth_approximation`, `packet`, `reconstruction`, `error`: arrays. `certificate: Certificate`. `packet_error: array | None`	Full retained-packet execution record.

`TruncatedPowerPacket` is an exact alias of `TruncatedPowerTerm`. `RetainedPacketReconstruction` is an exact alias of `RetainedPacketResult`. Neither name denotes a distinct subclass.

The class surface is:

```
TruncatedPowerTerm(location: float, amplitude: complex, order: int = 0)
    .evaluate(points: Any) -> ndarray
    .location_bound(
        estimated_location: float,
        *, p: float = 2.0,
        interval: tuple[float, float] = (0.0, 1.0),
    ) -> float

SingularPacket1D(components: Iterable[TruncatedPowerTerm])
    .from_jumps(
        locations: Iterable[float],
        amplitudes: Iterable[complex],
        *, orders: Iterable[int] | None = None,
    ) -> SingularPacket1D
    .evaluate(points: Any) -> ndarray
    .remove(signal: Any, points: Any) -> ndarray
    .restore(smooth: Any, points: Any) -> ndarray
    .trace_jumps() -> dict[float, dict[int, complex | float]]
```

`evaluate` converts the grid to `dtype=float`, preserves its shape, and calls `numpy.real_if_close(..., tol=1000)` on the result. A genuinely complex packet therefore returns complex data, while roundoff-level imaginary parts may be discarded. `remove` and `restore` require exact shape equality between the sample array and evaluated packet. They do not broadcast. `trace_jumps` combines equal `(location, order)` entries by addition.

The constructor rejects non-finite locations and amplitudes, negative orders, and nonintegral orders with `ValidationError`. `from_jumps` raises `DimensionError` when the location, amplitude, and optional order sequences do not have equal lengths.

36.1.1 Location perturbation

For two equal-amplitude steps on the whole real line,

$$\left\| a\mathbf{1}_{x\geq\xi} - a\mathbf{1}_{x\geq\hat{\xi}} \right\|_{L^p} = |a|\,|\xi-\hat{\xi}|^{1/p}, \qquad 0 < p < \infty.$$

For $p=\infty$, the value is $|a|$ when the displacement is nonzero. `step_location_lp_error(amplitude, location, estimated_location, *, p=2.0)` implements this whole-line formula.

`TruncatedPowerTerm.location_bound` instead works on the finite declared interval $[L,R]$. For order zero it clips the interval between the two locations to $[L,R]$, so it returns the exact finite-interval norm. This distinction matters when both locations lie outside the interval. For positive order it returns the mean-value bound

$$\left\| T_{\xi,a,r} - T_{\hat{\xi},a,r} \right\|_{L^p([L,R])} \leq \frac{|a|S^{r-1}}{(r-1)!}|\xi-\hat{\xi}|\,(R-L)^{1/p},$$

where

$$S = \max(0, R - \min(\xi, \hat{\xi})).$$

For $p = \infty$, the measure factor is one. Both helpers require positive finite p or `math.inf`. `location_bound` also requires finite increasing interval endpoints. `step_location_lp_error` does not itself reject non-finite locations or amplitudes, so callers needing a finite result must validate those inputs before calling it.

36.2 Retained and estimated reconstruction

Let $f = g + j$, where j is an exactly retained packet and Q is a shape-preserving approximation procedure. Then

$$\hat{f} = Q(f - j) + j = Qg + j$$

and the exact algebraic identity is

$$f - \hat{f} = g - Qg.$$

The callable form is:

```
retained_packet_reconstruct(
    signal: Any,
    packet: Any,
    approximator: Callable[[ndarray], Any],
    *, tolerance: float = 1e-11,
) -> RetainedPacketResult
```

The signal and packet may have any equal shape. The approximator receives `signal - packet` and must return the same shape. The certificate is `Exactness.NUMERICAL` because it measures the algebraic identity with a floating Euclidean norm. The default acceptance condition is residual $\leq 10^{-11}$. This verifies the supplied finite-array identity. It does not show that the retained packet is the true singular component or that Q is an optimal approximation.

For an estimated packet $\hat{j}$ and an explicit $N \times N$ matrix Q,

$$\hat{f} = Q(f - \hat{j}) + \hat{j}, \qquad f - \hat{f} = (I - Q)(f - \hat{j}).$$

If a true decomposition $f = g + j$ is supplied, the implementation also checks

$$f - \hat{f} = (I - Q)g + (I - Q)(j - \hat{j}).$$

The matrix forms are:

```
retained_packet_reconstruction(
    signal: Any,
    estimated_packet: Any,
    approximation_matrix: Any,
    *,
    true_smooth: Any | None = None,
    true_packet: Any | None = None,
    tolerance: float = 1e-10,
) -> RetainedPacketResult

estimated_packet_reconstruct(
    signal: Any,
    estimated_packet: Any,
    approximation_matrix: Any,
    *, tolerance: float = 1e-10,
) -> ComputationResult[ndarray]
```

These forms require one-dimensional signal and packet arrays of equal length; empty arrays are allowed. The matrix must have shape `(N, N)`. Supplying just one of `true_smooth` and `true_packet` raises `ValidationError`. Supplying either true component with a wrong shape raises `DimensionError`. The compatibility wrapper returns only the reconstruction in `.value` while retaining the same certificate.

`RetainedPacketResult.packet` is the packet actually supplied to the routine. In the estimated form, `.packet_error` is `true_packet - estimated_packet` when truth was supplied and is otherwise `None`. `.smooth_component` is the declared true smooth component when available. Otherwise it is `signal - estimated_packet`.

The packet error term can dominate the smooth approximation term. In particular, no bound on

$$\|(I - Q)(j - \hat{j})\|$$

is inferred merely because the algebraic certificate is valid.

36.3 Scalar and vector jump recovery

Suppose the uniform grid is $x_i = x_0 + ih$ and the samples satisfy

$$y_i = g(x_i) + j(x_i) + \eta_i.$$

The declared deterministic assumptions are

$$\mathrm{Lip}(g) \le M, \qquad |\eta_i| \le \varepsilon, \qquad |a_k| \ge a_{\min}.$$

For scalar data, and for vector data after replacing absolute value by the Euclidean norm, an inactive-cell contrast is bounded by

$$B = Mh + 2\varepsilon.$$

The theorem requires

$$a_{\min} > 2B, \qquad B < \theta < a_{\min} - B.$$

A cell is detected when

$$\|y_{i+1} - y_i\| > \theta.$$

Its reported location is the midpoint $x_i + h/2$, its amplitude estimate is the signed scalar contrast or vector contrast, its location-error scale is $h/2$, and its amplitude-error scale is B. Those bounds have theorem meaning only when the unchecked signal assumptions hold and each active cell contains one jump.

36.3.1 Strict scalar API

```
recover_separated_jumps(
    samples: Sequence[float],
    *,
    spacing: float,
    lipschitz_bound: float,
    noise_bound: float,
    minimum_jump: float,
    threshold: float | None = None,
    start: float = 0.0,
    minimum_separation: float | None = None,
) -> JumpRecoveryResult
```

`samples` is converted to a float vector and must contain at least two entries. The default threshold is `minimum_jump / 2`. Invalid finite parameters raise `ValidationError`. A closed contrast gap, an invalid threshold interval, or a declared `minimum_separation < spacing` raises `AssumptionError` rather than returning a failed certificate.

The scalar result types are:

Type	Fields and properties
`JumpDetection`	`cell: int`, `left_position: float`, `right_position: float`, `location: float`, `amplitude: float`
`JumpRecoveryResult`	`detections: tuple[JumpDetection, ...]`, `threshold`, `background_bound`, `location_error_bound`, `amplitude_error_bound`, `certificate`. Derived `.cells: tuple[int, ...]`, `.locations: ndarray`, `.amplitudes: ndarray`

The certificate has `Exactness.CONDITIONAL`. It checks the finite threshold conditions but lists the true Lipschitz bound, noise bound, jump lower bound, and one-jump-per-cell model under `assumptions_unchecked`.

36.3.2 Scalar/vector API

The second implementation accepts samples of shape `(N,)` or `(N, D)` and is exported with disambiguating package aliases:

```
recover_vector_jumps(
    samples: Any,
    *,
    spacing: float,
    lipschitz_bound: float,
    noise_bound: float,
    minimum_jump: float,
    origin: float = 0.0,
    threshold: float | None = None,
) -> VectorJumpRecoveryResult

recover_vector_jumps_subgaussian(
    samples: Any,
    *,
    spacing: float,
    lipschitz_bound: float,
    sigma: float,
    minimum_jump: float,
    failure_probability: float,
    origin: float = 0.0,
) -> VectorJumpRecoveryResult
```

`VectorJumpRecoveryResult` is the package alias for `komorebi_math.signals.edge_recovery.JumpRecoveryResult`. It is a different class from the scalar `JumpRecoveryResult` above.

Member	Scalar input `(N,)`	Vector input `(N, D)`
`.cells`	tuple of contrast indices	same
`.locations`	shape `(K,)`	shape `(K,)`
`.amplitudes`	shape `(K,)`	shape `(K, D)`
`.contrasts`	shape `(N-1,)`	shape `(N-1, D)`
`.packet()`	builds an order-zero `SingularPacket1D`	raises `ValidationError`
Remaining fields	`threshold`, `background_bound`, `location_error_bound`, `amplitude_error_bound`, `certificate`	same

Unlike `recover_separated_jumps`, the vector-capable routine does not raise solely because the contrast-gap condition $a_{\min} > 2B$ fails or the threshold falls outside its allowed interval. It returns the detections with `certificate.valid == False`. Call `result.certificate.require()` when a failed finite theorem gap must stop the workflow.

Adjacent detected cells are recorded in `assumptions_checked`, but the current implementation does not use that observation when setting `certificate.valid`. The actual separation of the unknown jumps remains unchecked in both APIs.

36.3.3 Sub-Gaussian wrappers

For N scalar samples, the scalar helper uses the simultaneous contrast scale

$$B_\eta = 2\sigma\sqrt{\log(2N/\delta)}.$$

```
subgaussian_noise_bound(
    sample_count: int,
    sigma: float,
    delta: float,
) -> float

recover_jumps_subgaussian(
    samples: Sequence[float],
    *,
    spacing: float,
    lipschitz_bound: float,
    sigma: float,
    delta: float,
    minimum_jump: float,
    threshold: float | None = None,
    start: float = 0.0,
    minimum_separation: float | None = None,
) -> JumpRecoveryResult
```

The wrapper passes `noise_bound = B_eta / 2` to the deterministic routine, so its background becomes $Mh + B_\eta$. It reports probability at least $1 - \delta$ only under the unchecked model that the noises are independent, centered, and σ-sub-Gaussian.

For `(N, D)` data, the vector wrapper uses $N - 1$ contrast cells and

$$b_{\mathrm{coord}} = 2\sigma\sqrt{\log\left(\frac{2\max(N-1,1)D}{\delta}\right)}, \qquad B_\eta = \sqrt{D}\, b_{\mathrm{coord}}.$$

It passes `noise_bound = B_eta / 2` to `recover_vector_jumps`. The coordinate and Euclidean bounds are stored under `certificate.residuals["coordinate_noise_bound"]` and `["simultaneous_noise_difference_bound"]`. The probability statement is not an empirical confidence estimate. It depends on the external independence, centering, and tail-scale assumptions.

36.4 Breakpoint bands and edge recovery

For a normalized Haar function on an interval of length L, a Lipschitz background and one step obey

$$|\langle g, \psi_L\rangle| \le \frac{ML^{3/2}}{4}, \qquad |\langle a\mathbf{1}_{x\ge\xi}, \psi_L\rangle| = |a|\sqrt{L}\min(u, 1-u),$$

where $u \in [0, 1]$ is the relative jump location. Two pairs of helpers expose the same formulas with different argument order:

```
haar_lipschitz_bound(lipschitz_constant, support_length) -> float
haar_step_coefficient(amplitude, support_length, relative_location) -> float

smooth_haar_coefficient_bound(lipschitz_bound, interval_length) -> float
step_haar_coefficient(amplitude, relative_location, interval_length) -> float
```

The first pair comes from `signals.recovery`. The second comes from `signals.edge_recovery`. All lengths must be positive and finite, smoothness bounds nonnegative, and relative positions in `[0, 1]`.

`half_shift_distance(point, cell_length)` compares the distance from `point` to the nearest grid boundary with the corresponding half-shifted-grid distance, then returns the **larger** of those two nearest-boundary distances. Despite the docstring phrase "better distance," the implementation uses `max(first, second)`, not `min`.

For forward difference step s,

$$(\Delta_s x)_n = x_{n+s} - x_n, \qquad \Delta_s^q = \underbrace{\Delta_s \circ \cdots \circ \Delta_s}_{q \text{ times}}.$$

The package-level difference routine and breakpoint detector are:

```
finite_difference(
    values: Sequence[Any], *, order: int = 1, step: int = 1,
) -> ndarray

detect_piecewise_polynomial_breaks(
    samples: Sequence[float],
    *, degree: int,
    noise_bound: float,
    threshold: float | None = None,
) -> BreakpointBandResult
```

An order-zero difference returns a copy. A positive order that exhausts the available samples returns an empty array. The detector computes order $q = d + 1$ with unit step. On a degree-d polynomial segment, that response vanishes away from boundaries. Uniform sample noise bounded by ε has worst-case amplification

$$B_q = 2^q \varepsilon.$$

The default threshold is B_q. A supplied threshold below it raises `AssumptionError`. The frozen `BreakpointBandResult` stores `indices: tuple[int, ...]`, `response: ndarray`, `threshold: float`, `noise_amplification: float`, and `certificate: Certificate`.

Each output index is the start of an above-threshold difference response. It is a response band, not automatically a unique physical breakpoint. The piecewise-polynomial model and uniform noise bound remain unchecked.

There is also a module-level compatibility routine:

```
komorebi_math.signals.singular_packets.finite_difference(signal, order=1)
```

It is implemented with `numpy.diff`. The same module also provides `difference_noise_bound(noise_bound, order)`. The former is listed in that module's `__all__`, but it is not the `finite_difference` selected by `komorebi_math.signals`, which exports the keyword-only, arbitrary-step routine from `signals.recovery`.

Exact region-error formulas

For true scalar value a, estimate b, overlap measure μ_I, true-only measure μ_T, and estimate-only measure μ_E,

$$\|a\mathbf{1}_T - b\mathbf{1}_E\|_p^p = |a-b|^p\mu_I + |a|^p\mu_T + |b|^p\mu_E.$$

For labeled partitions with $M_{ij} = \mu(T_i \cap E_j)$,

$$\|f - \hat{f}\|_p = \left(\sum_{i,j} M_{ij}|a_i - b_j|^p\right)^{1/p}.$$

For $p = \infty$, each routine takes the largest mismatch attached to a region of positive measure.

```
scalar_region_lp_error(
    true_value, estimated_value,
    intersection_measure, true_only_measure, estimated_only_measure,
    *, p: float = 2.0,
) -> float

overlap_partition_lp_error(
    true_values, estimated_values, overlap_matrix,
    *, p: float = 2.0,
) -> float

scalar_region_error(
    true_mask, true_value, estimated_mask, estimated_value,
    *, p: float = 2.0, tolerance: float = 1e-12,
) -> ComputationResult[float]

overlap_matrix_error(
    true_labels, true_values, estimated_labels, estimated_values,
    *, p: float = 2.0, tolerance: float = 1e-12,
) -> ComputationResult[float]
```

The first two evaluate formulas from supplied measures. The latter two build empirical count measures from equal-shaped masks or integer label arrays and compare the formula with the direct finite-array norm. Their certificates are `Exactness.NUMERICAL`, even though the underlying finite counting identities are exact, because residual comparison uses floating arithmetic.

Negative or non-finite measures, a wrong overlap-matrix shape, invalid p, unequal mask/label shapes, negative labels, and labels outside the corresponding value table are rejected. Label arrays are coerced with `dtype=int`. Callers should not rely on this function to reject a fractional label before coercion.

Worked recovery and reconstruction

The repository example `examples/singular_packet_recovery.py` constructs two steps on 513 points, with a sinusoidal background and no added noise:

```
import numpy as np

from komorebi_math.signals import (
    SingularPacket1D,
    recover_separated_jumps,
    retained_packet_reconstruction,
)

count = 512
h = 1.0 / count
x = np.arange(count + 1) * h
locations = np.array([0.23, 0.71])
amplitudes = np.array([0.45, -0.35])

smooth = 0.04 * np.sin(2.0 * np.pi * x)
signal = smooth.copy()
for location, amplitude in zip(locations, amplitudes, strict=True):
    signal += amplitude * (x >= location)

recovery = recover_separated_jumps(
    signal,
    spacing=h,
    lipschitz_bound=0.08 * np.pi,
    noise_bound=0.0,
    minimum_jump=float(np.min(np.abs(amplitudes))),
    minimum_separation=0.4,
)
recovery.certificate.require()

packet = SingularPacket1D.from_jumps(
    recovery.locations,
    recovery.amplitudes,
).evaluate(x)

identity = retained_packet_reconstruction(
    signal,
    packet,
    np.eye(signal.size),
    true_smooth=signal - packet,
    true_packet=packet,
)
identity.certificate.require()

print("detected cells:", recovery.cells)
print("estimated locations:", recovery.locations)
print("estimated amplitudes:", recovery.amplitudes)
```

Observed with the recovered `1.0.0b4` source:

```
detected cells: (117, 363)
estimated locations: [0.22949219 0.70996094]
estimated amplitudes: [ 0.45006308 -0.35012219]
```

This run checks the result against known synthetic construction parameters. The valid certificate alone would not establish that the supplied Lipschitz constant or jump lower bound is true for unknown data. The identity example uses `Q = I`, so exact reconstruction is expected for every supplied packet. It is a check of plumbing, not evidence of approximation quality.

36.5 Smoothness, noise, and separation assumptions

Claim or result	Status	What is actually checked
Truncated-power evaluation	definition	finite term parameters and finite sample-grid conversion
Order-zero finite-interval location norm	exact formula	declared interval and values
Positive-order location estimate	deterministic bound	declared term, interval, and displacement
Retained-packet identity	numerical check of exact algebra	shapes and floating residual only
Deterministic jump locations/amplitudes	conditional	parameter gap and observed contrasts. Signal hypotheses remain external
Sub-Gaussian probability	conditional	finite confidence formula. Noise law remains external
Haar helpers	deterministic formulas	supplied scalars only
Breakpoint output	conditional band	difference order and declared noise threshold
Region identities	exact finite formula, numerically certified	input measures or empirical finite counts

36.6 Failure cases that must remain conditional

These cases must not be upgraded into unconditional recovery claims:

- Two jumps in one sample cell can cancel in the adjacent contrast.
- Adjacent active cells violate the separated-cell interpretation.
- A low-amplitude jump below the contrast gap is outside this theorem.
- Incorrect smoothness, noise, jump-size, or probability-model declarations invalidate the theorem interpretation even if execution succeeds.
- A finite-difference band can cover multiple indices and need not identify one exact breakpoint.
- An inaccurate packet can make the packet-estimation term larger than the smooth approximation error.

Implementation boundary notes in `1.0.0b4`:

- `recover_separated_jumps` only checks that a supplied `minimum_separation` is not less than one grid spacing. It does not check the detected-cell gaps against that value. `minimum_separation=float("nan")` also passes that comparison and can yield a valid certificate.
- `recover_vector_jumps` records adjacent detections but does not make them invalidate the certificate. For `[0, 1, 2]` with zero declared background and `minimum_jump=0.5`, the observed result is `cells == (0, 1)` with `certificate.valid == True`.
- The vector API does not validate `origin`. A non-finite origin can therefore produce non-finite reported locations while the certificate remains valid.
- Recovery accepts arrays containing non-finite sample values. Comparisons with `NaN` simply fail to trigger detections. Prevalidate finite sample data when this is not desired.
- Reconstruction tolerances are compared directly but are not validated as finite and nonnegative. Use an explicit finite nonnegative tolerance.

Cost, memory, and scaling

Let P be the number of evaluation points, J the number of packet terms, N the number of samples, D the vector dimension, $q = d + 1$ the difference order, and A, B the numbers of true and estimated regions.

Operation	Arithmetic cost	Additional/output storage
One term evaluation	$O(P)$	$O(P)$
`SingularPacket1D.evaluate`	$O(JP)$	$O(P)$ result plus per-term temporaries

Operation	Arithmetic cost	Additional/output storage
Callable retained reconstruction	cost of `approximator` + $O(P)$	$O(P)$ arrays
Matrix retained reconstruction	dense $O(N^2)$ matvec work	input matrix $O(N^2)$. Identity/matrix temporaries can also be $O(N^2)$
Scalar recovery	$O(N)$	$O(N)$ differences and up to $O(N)$ detections
Vector recovery	$O(ND)$	$O(ND)$ contrasts
Order-q iterative difference	$O(qN)$	$O(N)$ live response, with repeated allocations
Scalar mask error	$O(P)$	$O(P)$ boolean/numeric temporaries
Labeled overlap error	$O(P + AB)$	$O(P + AB)$ including overlap table

There is no streaming recovery interface here, no sparse approximation-matrix specialization, and no GPU backend. NumPy executes all kernels on host memory. The implementation does not impose an explicit resource limit, so callers must budget dense `N x N` approximation matrices and large overlap tables.

36A. Approximation, nonlinear packet products, and Smith charts

This group of tools combines finite Fourier truncation, a narrow positive-contrast heuristic, cyclic coefficient products, real polynomial shift interactions, and normalized impedance coordinates. These tools share a signal namespace, but they do not share a single theorem: each result must be read at its own claim strength.

Public surface.

All names below are exported by `komorebi_math.signals`.

- **`fft_lowpass_1d`:** `(signal: ndarray | Sequence[float], modes: int) -> ndarray`. Finite numerical Fourier truncation.
- **`fft_lowpass_2d`:** `(image: ndarray, cutoff: int) -> ndarray`. Finite numerical Fourier truncation.
- **`otsu_threshold`:** `(values: ndarray | Sequence[float], bins: int = 256) -> float`. Deterministic finite histogram choice.
- **`estimate_positive_packet_1d`:** `(signal, *, bins: int = 256) -> PacketEstimate1D`. Heuristic record.
- **`coefficient_packets`:** `(coefficients: Sequence[complex], order: int) -> tuple[ndarray, ...]`. Exact residue split in array arithmetic.
- **`packet_product`:** `(a: Any, b: Any, order: int) -> ComputationResult[tuple[ndarray, ...]]`. Finite identity with numerical certificate.
- **`nonlinear_shift_filter`:** `(signal: Sequence[float], alpha: float = 0.25, *, shift: int = 1, shifts: Sequence[int] | None = None, coefficients: Sequence[float] | None = None) -> ndarray`. Finite real polynomial map.
- **`gamma_from_impedance`:** `(impedance: ArrayLike) -> ndarray`. Numerical Cayley map.
- **`impedance_from_gamma`:** `(gamma: ArrayLike) -> ndarray`. Numerical inverse Cayley map.
- **`phase_mobius`:** `(impedance: ArrayLike, order: int, phase: int) -> ndarray`. Numerical rational phase action.
- **`smith_sensitivity`:** `(gamma: ArrayLike, *, regularization: float = 0.0) -> ndarray`. Numerical diagnostic.

Here `ArrayLike` means `ndarray | Sequence[complex] | complex`. Even scalar input is returned as a zero-dimensional NumPy array/scalar-like array result, not normalized to a Python `complex`.

36A.1 Fourier low-pass approximation and Gibbs behavior

For a length-N vector, `fft_lowpass_1d` computes the unnormalized NumPy FFT, zeros every integer-frequency bin with magnitude greater than `modes`, and applies the inverse FFT:

$$\hat{x}_k^{(M)} = \hat{x}_k \mathbf{1}_{|\nu_k| \le M}, \qquad x^{(M)} = \mathcal{F}_N^{-1} \hat{x}^{(M)},$$

where `nu = np.fft.fftfreq(N) * N`. The input must be a nonempty 1-D array and `modes` a nonnegative integer. Boolean and fractional cutoffs are rejected. Modes beyond the represented Nyquist range simply retain every bin.

For an `H x W` image, `fft_lowpass_2d` uses `fftshift(fft2(image))` and the rectangular index mask

$$|i - \lfloor H/2 \rfloor| \le M, \qquad |j - \lfloor W/2 \rfloor| \le M.$$

This is a rectangular centered cutoff, not a radial-frequency disk. With even dimensions, the centered Nyquist indexing follows NumPy's `fftshift` convention. Both routines call `real_if_close(tol=1000)` after inverse transformation, so real inputs usually return real arrays and genuinely complex outputs remain complex.

Fourier truncation is a deterministic finite approximation, not an edge-aware method. For a periodic length-32 step and three retained mode pairs, the observed samples around the edge were:

```
[-0.014852, 0.148468, 0.375000, 0.625000, 0.851532, 1.014852]
```

The direct maximum error was `0.37500000000000006`. The undershoot and overshoot are finite-grid Gibbs ringing. Increasing the grid or mode count does not justify a universal pointwise edge-preservation claim.

If the exact step packet is removed first, low-pass filtering a zero smooth remainder and restoring that exact packet gives zero error in this synthetic case:

```python
import numpy as np

from komorebi_math.signals import (
    SingularPacket1D,
    fft_lowpass_1d,
    retained_packet_reconstruct,
)

x = np.r_[np.zeros(16), np.ones(16)]
grid = np.arange(x.size, dtype=float)

packet = SingularPacket1D.from_jumps([16.0], [1.0]).evaluate(grid)
kept = retained_packet_reconstruct(
    x,
    packet,
    lambda smooth: fft_lowpass_1d(smooth, 3),
)

assert np.max(np.abs(kept.error)) == 0.0
kept.certificate.require()
```

This example isolates the value of exact side information. It does not show that an estimated packet always improves a direct Fourier approximation.

36A.2 Otsu thresholding and heuristic positive-packet estimation

Given histogram probabilities p_k, bin centers c_k, cumulative class probability ω_t, cumulative mean μ_t, and total mean μ_T, the implemented Otsu score is

$$\sigma_B^2(t) = \frac{(\mu_T \omega_t - \mu_t)^2}{\omega_t(1-\omega_t)}.$$

Bins with a zero denominator receive score zero. `otsu_threshold` returns the center of the first maximum-scoring bin because it uses `numpy.argmax`. If all values are numerically equal according to `numpy.isclose(low, high)`, it returns that common minimum directly.

`estimate_positive_packet_1d` forms

$$M_i = \mathbf{1}_{x_i > t}$$

and estimates the amplitude by

$$\hat{a} = \operatorname{median}(x_i : M_i = 1) - \operatorname{median}(x_i : M_i = 0).$$

If every point is selected, it uses the selected median alone. If no point is selected, it returns amplitude zero. The frozen result contains:

`PacketEstimate1D` member	Meaning
`mask: ndarray`	Boolean mask selected by strict `value > threshold`
`amplitude: float`	Median contrast heuristic
`threshold: float`	Chosen histogram-bin center
`certificate: Certificate`	Always valid, `Exactness.HEURISTIC`

For 30 zeros followed by 20 ones, the default 256-bin run selected 20 samples, returned threshold `0.001953125`, and amplitude `1.0`. This is consistent with the intended two-class positive-contrast model. It is not a theorem for negative contrasts, multiple regions, heavy gradients, outliers, or unstable histogram binning.

The source checks only `bins < 2`, then calls `int(bins)`. A fractional value such as `2.9` is therefore silently truncated rather than rejected as nonintegral. Non-finite signal values are not explicitly rejected and may fail inside NumPy histogram construction. Validate finite data and integral bin counts at an external boundary when those are contract requirements.

36A.3 Coefficient packets, cyclic products, and nonlinear shift filters

For cyclic order $q \geq 1$, `coefficient_packets` returns the full-length, zero-filled residue arrays

$$a_n^{(r)} = a_n \mathbf{1}_{n \equiv r \pmod q}, \qquad a = \sum_{r=0}^{q-1} a^{(r)}.$$

The input must be one-dimensional and the order a positive non-Boolean integer. The function permits an empty coefficient array and returns `q` empty arrays. `packet_product` is stricter for flat inputs and rejects empty arrays.

`packet_product` accepts exactly one of two representations:

1. two nonempty flat coefficient arrays, or
2. two explicit compact channel families, each with exactly `q` nonempty one-dimensional arrays.

Mixing one flat input with one compact family raises `ValidationError`. Malformed or empty explicit channels raise `DimensionError` when they are recognized as a channel family. A nested sequence whose outer length is not `q` is not recognized as explicit. Its later conversion may consequently fail with a NumPy error rather than a package-specific representation error.

For flat residue-supported arrays, the output packet is

$$c^{(t)} = \sum_{r+s \equiv t \pmod q} a^{(r)} * b^{(s)}.$$

Each returned channel has the full direct-product length $N + M - 1$, remains supported on its routed residue positions, and the direct convolution is reconstructed by summing the channels.

Compact arrays use coarse indices. The channel route must therefore propagate the radix carry

$$t = (r + s) \bmod q, \qquad \kappa = \left\lfloor \frac{r+s}{q} \right\rfloor.$$

The convolution of compact channels r, s is added to output channel t starting at coarse index κ. Omitting that shift gives the wrong polynomial product.

```python
import numpy as np

from komorebi_math.signals import packet_product
```

```
left = (np.array([1.0, 2.0]), np.array([3.0, -1.0]))
right = (np.array([2.0, 1.0]), np.array([4.0]))

result = packet_product(left, right, order=2)
channels = result.require()

assert [channel.tolist() for channel in channels] == [
    [2.0, 17.0, -2.0],
    [10.0, 9.0, -1.0],
]
assert result.certificate.residuals["cyclic_channel_law"] == 0.0
```

The certificate reconstructs both inputs, compares the interleaved output with `numpy.convolve`, and accepts a maximum absolute residual at the fixed tolerance `1e-10`. The coefficient law is an exact finite polynomial identity. Its implementation certificate is `Exactness.NUMERICAL`. The routine promotes working storage to a complex-compatible dtype, then applies `real_if_close` to each output channel. It does not reason about convergence of infinite series.

Nonlinear shift filters

Let $R_s x = \mathrm{roll}(x, s)$ with periodic wraparound. The legacy map is

$$F(x) = x + \alpha x \odot R_s x.$$

The generalized map is

$$F(x) = x + \alpha x \odot \left(\sum_{k=1}^{K} c_k R_{s_k} x \right).$$

```
from komorebi_math.signals import nonlinear_shift_filter

y = nonlinear_shift_filter(
    [0.0, 1.0, 2.0, 3.0],
    alpha=0.1,
    shifts=[0, 1],
    coefficients=[1.0, 0.5],
)
```

The input is coerced with `numpy.asarray(signal, dtype=float)`, so this API is a real-valued map $\mathbb{R}^N \to \mathbb{R}^N$, not a generic complex packet operation. It requires a one-dimensional array but permits the empty vector. Supply either neither generalized list or both. The paired lists must be nonempty and have equal lengths.

`shift` and generalized shifts are converted with `int`, coefficients and `alpha` with `float`. Nonintegral shifts are silently truncated, and no explicit finite-value check is made for `alpha` or the coefficients. Complex input conversion can emit `ComplexWarning` and discard the imaginary part. Callers should validate integer shifts, finite real coefficients, and finite real input when silent coercion is unacceptable.

The operation is a finite polynomial transform. No contractive, energy, invertibility, fixed-point, or nonlinear stability theorem is attached. `packet_product`, in contrast, accepts complex coefficient data and certifies a finite convolution identity.

36A.4 Smith/Cayley impedance coordinates

The normalized impedance-to-reflection map and its inverse are

$$\Gamma = \frac{Z - 1}{Z + 1}, \qquad Z = \frac{1 + \Gamma}{1 - \Gamma}.$$

With their domains made explicit, these are

$$\Gamma = \frac{Z-1}{Z+1}, \qquad Z \neq -1,$$

$$Z = \frac{1+\Gamma}{1-\Gamma}, \qquad \Gamma \neq 1.$$

For cyclic order q and phase index p, let

$$\zeta = e^{2\pi i (p \bmod q)/q}.$$

Disk rotation $\Gamma \mapsto \zeta\Gamma$, pulled back formally to impedance coordinates, gives

$$Z \longmapsto \frac{(1+\zeta)Z + (1-\zeta)}{(1-\zeta)Z + (1+\zeta)}.$$

`phase_mobius` requires a positive integer order, reduces the converted phase modulo that order, and checks its own denominator. Phase zero is the identity. It evaluates the rational formula directly. It does not call `gamma_from_impedance` and therefore accepts some points, such as `Z == -1`, where the two-stage Cayley interpretation itself is undefined.

A finite roundtrip example is:

```
import numpy as np

from komorebi_math.signals import (
    gamma_from_impedance,
    impedance_from_gamma,
    phase_mobius,
    smith_sensitivity,
)

z = np.array([0.5, 1.0, 2.0, 1.0 + 0.2j])
gamma = gamma_from_impedance(z)

assert np.allclose(impedance_from_gamma(gamma), z)
assert np.allclose(phase_mobius(z, order=5, phase=0), z)

sensitivity = smith_sensitivity(gamma, regularization=1e-6)
```

Observed values were:

```
Gamma = [-0.333333+0.j        0.000000+0.j
          0.333333+0.j        0.009901+0.099010j]
maximum roundtrip error = 4.440892098500626e-16
sensitivity = [2.812495, 1.999998, 2.812495, 1.981152]
```

This is a pointwise floating roundtrip, not an exact symbolic identity.

36A.5 Numerical sensitivity and chart singularities

The diagnostic

$$S_\rho(\Gamma) = \frac{1}{|1-\Gamma|^2 + \rho} + \frac{1}{|1+\Gamma|^2 + \rho}, \qquad \rho \geq 0,$$

highlights both chart boundaries. With $\rho = 0$, it diverges at $\Gamma = 1$ and $\Gamma = -1$. Positive regularization caps either individual pole contribution at $1/\rho$, but it changes the diagnostic rather than removing the underlying Cayley singularity. Near $\Gamma = 1$, the inverse derivative

$$\frac{dZ}{d\Gamma} = \frac{2}{(1-\Gamma)^2}$$

also shows quadratic amplification. A small roundtrip residual far from the pole does not imply good conditioning near it.

The implemented practical domain is narrower than exact nonzero-denominator arithmetic. `gamma_from_impedance`, `impedance_from_gamma`, and `phase_mobius` reject a denominator whenever `numpy.isclose(denominator, 0.0)` is true. With NumPy defaults and a zero comparison, this is essentially an absolute `1e-8` exclusion around zero. They raise built-in `ZeroDivisionError`, not `ValidationError`.

`smith_sensitivity` rejects negative regularization but does not check that it is finite. `regularization=float("nan")` returns `nan`. Positive infinity can collapse both terms toward zero. At an unregularized exact pole, NumPy can return `inf` with a runtime warning rather than raise. Inputs to all Smith helpers are converted to complex arrays but are not checked for `NaN` or infinity.

`phase_mobius` validates `order` as an integer but merely applies `int(phase)`. A fractional phase such as `1.9` is silently treated as phase `1`. Validate the phase index externally if truncation is not intended.

Failure and numerical-sensitivity cards

Symptom	Cause	Required interpretation or response
Fourier ringing at a jump	hard spectral cutoff	expected Gibbs behavior. Do not claim edge preservation
Fourier result has tiny imaginary parts removed	`real_if_close(tol=1000)`	compare numerical values and documented dtype behavior
Otsu mask finds the wrong region	heuristic assumes one dominant positive contrast	inspect sign, gradients, region multiplicity, and bin dependence
Mixed flat/compact product rejected	representations have different routing semantics	convert both operands to the same representation
Product channel appears shifted	compact radix carry omitted by external code	include `floor((r+s)/q)` coarse shift
Nonlinear result loses imaginary data	real `dtype=float` coercion	use another implementation for complex transforms
Nonlinear output grows rapidly	polynomial interaction has no stability bound	constrain data/coefficients using an application-specific analysis
`ZeroDivisionError` near a Smith pole	practical `np.isclose` denominator exclusion	change chart or move away from the boundary
Huge inverse impedance perturbation	`Gamma` is close to `1`	report sensitivity. Do not interpret the chart as well-conditioned
Infinite/NaN sensitivity	exact pole, zero regularization, or non-finite input	validate finiteness and choose a declared positive regularization for diagnostics

Cost, memory, and backend behavior

Let N be a 1-D sample count, $P = HW$ a 2-D pixel count, B the histogram bin count, q the cyclic order, $L = N+M-1$ a flat product length, and K the number of nonlinear shifts.

Operation	Arithmetic cost	Storage and traffic
1-D Fourier cutoff	$O(N \log N)$	$O(N)$ spectrum, mask, and output
2-D Fourier cutoff	$O(P \log P)$	$O(P)$ spectrum, coordinate mask, and output
Otsu threshold	$O(N + B)$ after flattening	$O(N + B)$ including converted values and histogram arrays
Positive-packet estimate	Otsu + $O(N)$ medians/mask	$O(N + B)$
Flat `coefficient_packets`	$O(qN)$	q full-length arrays: $O(qN)$

Operation	Arithmetic cost	Storage and traffic
Flat `packet_product`	current zero-filled implementation can do $O(q^2NM)$ direct-convolution work, plus verification	$O(q(N+M+L))$ packet/output storage and convolution temporaries
Compact `packet_product`	sum of all pairwise channel convolutions, plus direct verification. Naive dense worst case is quadratic in reconstructed lengths	output channels, pair terms, reconstructed operands, and direct product are all materialized
Nonlinear shift filter	$O(KN)$, or $O(N)$ in legacy form	$O(N)$ interaction/output, with a rolled temporary per shift
Smith/Cayley pointwise map	$O(N)$	$O(N)$ complex temporaries and output

All implementations use NumPy on host memory. There is no CuPy dispatch, streaming FFT, sparse packet-product kernel, or explicit memory budget. The product certificate always computes a direct convolution reference, so verification is part of runtime and memory cost rather than metadata-only bookkeeping.

Verification evidence and observed defects

The source audit covered the public topology in `signals/__init__.py`, all six defining modules (`singular_packets.py`, `recovery.py`, `edge_recovery.py`, `approximation.py`, `nonlinear.py`, and `smith.py`), the signal release description, the expanded Chapter 36/36A specification, the repository example, and the four focused test files `test_signals_wavelets_singular.py`, `test_signals_recovery.py`, `test_signals_extended.py`, and `test_block4_hardening.py`.

The checked-in singular recovery example ran successfully from source and produced the values printed in Section 36.4. A direct source smoke run also observed zero retained-packet residual for the synthetic exact step, zero compact-product residual, and a Smith roundtrip error of about 4.44×10^{-16}. The exact four-file focused Pytest slice passed **28/28** cases in the qualification environment. The wider Part VI slice passed **212/212**. The recovered runtime's primary interpreter did not itself contain Pytest, so these executions used the declared guide qualification environment.

These reproducible implementation defects or contract weaknesses are preserved in this guide rather than papered over:

1. Scalar `minimum_separation=NaN` can pass validation. Its magnitude is not checked against detected gaps.
2. Vector `origin=NaN` can yield `certificate.valid == True` and `NaN` locations.
3. Adjacent vector detections are diagnosed but do not invalidate the certificate.
4. Non-finite samples and several non-finite tolerance/regularization values lack explicit guards.
5. Otsu bin counts and Smith phase indices are converted with `int` without an integrality check.
6. `nonlinear_shift_filter` silently relies on numeric coercions that can truncate shifts or discard complex components.
7. The direct phase Möbius formula accepts `Z=-1` for tested nonzero phases, although the conceptual route through `Gamma` is undefined there.

Treat these notes as version-specific source observations. They do not expand the accepted mathematical domain and should not be used as guarantees that a future release will preserve the same weak validation.

Part VII. Nonlinear calculus and finite dynamics

This part offers several routes through finite nonlinear and dynamical problems. The common prerequisite is finite-dimensional linear algebra, including linear maps, kernels, images, and projectors. Directional derivatives are introduced in Chapter 37. The finite-field route in Chapter 42 uses exact modular arithmetic instead of floating matrix analysis.

Your task	Reading route	Main result to inspect
Differentiate a finite map	Chapter 37, then Chapter 38 for packet blocks	Provider assumptions, derivative order, and reconstruction
Expand a polynomial fixed-point problem	Chapter 39	Finite tree sum; use the separate majorant for a conditional convergence bound
Decide whether a defect can be corrected	Chapters 40 and 41	Obstruction status and the residual equation that remains
Lift or compute through finite filtration levels	Chapter 42	Exact correction space, branch depth, or spectral page
Execute, persist, or profile packet jets	Chapter 43 after Chapters 37 and 38	Fresh execution plan, complete stored data, and matched timing scope
Follow moving projector ranges	Chapter 44	Derivative, generator, and transport certificates
Evolve or reduce a finite matrix model	Chapters 45 and 46	State convention, finite-time residual, and retained/hidden coupling

Read a result's certificate together with its answer fields. A valid record may certify a nonzero obstruction, an incomplete prefix, or a correctly computed residual. The sections below identify the field that answers each mathematical question.

37. Derivative providers and automatic differentiation

Komorebi Math separates nonlinear differentiation from packet projection. A *derivative provider* owns a finite-dimensional map and its directional derivatives. Chapter 38 then restricts those derivatives to source and target packet ranges. This separation is essential: a packet jet can only be as mathematically reliable as the provider from which it was built.

Throughout this chapter,

$$F : X \to Y, \qquad X \cong \mathbb{K}^{d_X}, \qquad Y \cong \mathbb{K}^{d_Y},$$

where $\mathbb{K}$ may be an exact symbolic or rational domain, or a numerical real or complex domain. A directional derivative is evaluated without materializing a full Jacobian or higher tensor unless the chosen provider already stores such a tensor.

The nonlinear feature record is available from the namespace:

```
from komorebi_math.nonlinear import FEATURE_KEY, require, status
```

- `FEATURE_KEY` is the literal string `"nonlinear"`.
- `status() -> FeatureSpec` returns the registry entry titled *Nonlinear packet calculus*. In release `1.0.0b4` its status is `implemented`.
- `require() -> None` confirms that the base nonlinear package is present. It does not import or validate SymPy, JAX, PyTorch, or CuPy.

There are three practical import tiers.

- The root package, `komorebi_math`, re-exports the common concrete providers, the two external AD adapters, and the main construction functions.
- `komorebi_math.nonlinear` is the complete public nonlinear namespace. Protocol aliases, packet-address records, result records, partition helpers, and package controls should be imported here.
- `ad_framework_diagnostics` is public in `komorebi_math.nonlinear.autodiff` but is not re-exported by `komorebi_math.nonlinear` in this release. Code that needs probe details must import it from the implementation module.

API stability is narrower than namespace visibility. The curated beta surface contains `DerivativeProvider`, `CallableDerivativeProvider`, `FiniteDifferenceDerivativeProvider`, and `PolynomialDerivativeProvider` from Chapter 37. From Chapter 38 it contains `PacketJet`, `PacketJetAddress`, `PacketJetFamily`, `PacketJetSupport`, `packet_jet_support`, `PacketJetCoefficient`, `PacketJetSnapshot`, `snapshot_packet_jet`, `SparsePolynomialDerivativeProvider`, `SparseSymmetricTensor`, and `SparseTensorEntry`. Other Chapter 37 through 38 names are research-surface APIs in this release, and no Part VII name belongs to the frozen stable surface.

The claim boundary is uniform. `Exactness.EXACT` means that the declared coefficients, directions, backend operations, and substitutions are exact. `Exactness.NUMERICAL` permits a residual tolerance. `Exactness.CONDITIONAL` records an incomplete finite audit or a result with visible assumptions. Exactness is partly caller-declared. Selecting `Exactness.EXACT` does not inspect an object array for hidden binary floats, and `SympyDerivativeProvider` cannot prevent a caller from substituting `sympy.Float` values. Use exact labels only with exact input objects.

37.1 Provider protocol

For every provider $\mathfrak{D}_F$, document (d_X, d_Y, e_F, n_F), where e_F is exactness and n_F is the provider name.

$$\mathfrak{D}_F.\,\text{value}(x) = F(x), \qquad \mathfrak{D}_F.\,\text{derivative}(r, x, V) = D^r F(x)[V].$$

The public typing surface is:

```
DirectionalDerivative = Callable[[Any, tuple[Any, ...]], Any]

class DerivativeProvider(Protocol):
    source_dimension: int
    target_dimension: int
    exactness: Exactness
    name: str

    def value(self, point: Any) -> Any: ...

    def derivative(
        self,
        order: int,
        point: Any,
        directions: Sequence[Any],
    ) -> Any: ...

    def supports_order(self, order: int) -> bool: ...
```

`DerivativeProvider` is runtime-checkable. It describes structure, not semantic truth. Passing `isinstance(provider, DerivativeProvider)` establishes that the required attributes are present. It does not prove that `derivative` differentiates `value`.

Universal guards are as follows.

- A point and every direction must be vector-like with length d_X. Row and column vectors are accepted by several providers and normalized locally, but callers should prefer one-dimensional vectors for portability across providers.
- `value` and `derivative` must return a vector-like object with length d_Y.
- A derivative order is positive. Exactly that many directions must be supplied.
- `supports_order(r)` reports availability. A caller should still expect `ValidationError` if an invalid order reaches `derivative`.
- `DimensionError` reports incompatible source or target dimensions. `ValidationError` reports invalid orders, counts, steps, tolerances, or declarations. `BackendUnavailableError` reports an absent or unusable optional framework.
- A certificate returned by a provider is a finite record. Calling `certificate.require()` raises `ValidationError` when `certificate.valid` is false.

The protocol imposes no common dtype or device. A SymPy provider returns a SymPy matrix. A polynomial provider commonly returns a NumPy array, possibly with `dtype=object`. JAX and PyTorch adapters copy their final result to a host NumPy array. Provider interchangeability is therefore mathematical and structural, not a promise of identical Python container types.

Cost model. Let C_F denote one evaluation of `value` and C_{D_r} one provider-specific order-r directional derivative. The protocol itself adds only dimension checks and vector conversion. It does not promise a bound on C_F, C_{D_r}, temporary storage, tracing, or device transfer.

37.2 Callable, polynomial, finite-difference, and SymPy providers

The four provider families in this section expose the same finite directional derivative protocol but obtain their values in different ways: declared callables, stored polynomial tensors, recursive central differences, and symbolic differentiation. Their common interface does not erase their different exactness, storage, and failure boundaries.

For a first provider, choose the representation already available for the map: exact polynomial tensors for a finite polynomial, SymPy expressions for symbolic differentiation, declared callables when derivatives are supplied, or an AD adapter for a function that its framework can trace. Finite differences are a numerical fallback and a small-probe comparison tool. Check one value and one directional derivative before constructing a packet family; this separates provider problems from packet-layout problems.

37.2.1 Declared callable derivatives

`CallableDerivativeProvider` stores a function and a map from positive derivative orders to user-supplied directional callables.

```
CallableDerivativeProvider(
    function: Callable[[Any], Any],
    derivatives: Mapping[int, DirectionalDerivative],
    source_dimension: int,
    target_dimension: int,
    exactness: Exactness = Exactness.NUMERICAL,
    name: str = "declared derivative provider",
)
```

The declaration is

$$\mathcal{D} = \{r \mapsto \mathcal{D}_r\}, \qquad \mathcal{D}_r(x; v_1, \dots, v_r) \overset{\text{declared}}{=} D^r F(x)[v_1, \dots, v_r].$$

The callable at order r receives `(point, directions)`, where `directions` is a tuple of length r. The provider validates the point, direction, and output dimensions, then returns the callable's value without coercing it to a particular backend.

The member surface is:

- `available_orders -> tuple[int, ...]` returns sorted declared orders.
- `supports_order(order: int) -> bool` accepts only an explicitly declared positive integer order. A Boolean is not an order.
- `value(point: Any) -> Any` evaluates the declared map and checks the target dimension.
- `derivative(order, point, directions) -> Any` evaluates one declared directional derivative.
- `validation_certificate(point, directions, *, tolerance=1e-9, check_taylor=False, increment=None) -> Certificate` performs finite probes under theorem record `PSC-III-D02`.

For every available order not exceeding the number of probe directions, validation evaluates all permutations of the selected prefix. Its finite symmetry residual is

$$\rho_{\text{sym}} = \max_{r, \sigma \in S_r} \|\mathcal{D}_r(x; v_{\sigma(1)}, \dots, v_{\sigma(r)}) - \mathcal{D}_r(x; v_1, \dots, v_r)\|.$$

If no declared order fits within the supplied probe prefix, no derivative is checked and the zero residual passes. Supply at least as many probe directions as the largest order that must be audited, and inspect `diagnostics["checked_orders"]` before treating the record as evidence.

With `check_taylor=True`, `increment` is required and the diagnostic polynomial is

$$T_R(x; h) = F(x) + \sum_{r \in R} \mathcal{D}_r(x; h, \dots, h)/r!, \qquad \rho_T = \|F(x+h) - T_R(x; h)\|.$$

The certificate's validity is controlled only by ρ_{sym}. The Taylor residual is recorded under `truncated_taylor` but does not control validity. This permits two important negative cases.

- An asymmetric declaration fails even if it accidentally matches a few scalar Taylor probes.
- A symmetric but mathematically wrong declaration can pass the symmetry gate while producing a large Taylor diagnostic.

The certificate checks finite source and target dimensions and the selected permutations. It leaves global derivative correctness unchecked. If Taylor validation is requested, it also leaves the remainder beyond the largest declared order unchecked. Missing orders, wrong direction counts, nonpositive order keys, noncallable entries, and incompatible dimensions fail immediately.

Cost and storage. For a checked order r, the implementation performs $r!$ derivative calls, including the identity permutation. Across an order set R, the validation cost is

$$\Theta\left(\sum_{r\in R,\ r\le q} r!\, C_{D_r}\right),$$

where q is the number of supplied probe directions. Optional Taylor validation adds one shifted function evaluation and one repeated-direction derivative call per declared order. The provider stores the function, the derivative mapping, and scalar metadata. It does not cache evaluations.

37.2.2 Polynomial tensors

`PolynomialDerivativeProvider` represents a finite polynomial by dense symmetric coefficient tensors.

```
PolynomialDerivativeProvider(
    tensors: Mapping[int, Any],
    *,
    source_dimension: int | None = None,
    target_dimension: int | None = None,
    exactness: Exactness = Exactness.NUMERICAL,
    tolerance: float = 1e-10,
    name: str = "polynomial derivative provider",
)
```

Its factorial convention is

$$F(x) = \sum_{m=0}^{M} \frac{1}{m!} T_m[x^m], \qquad T_m \in Y \otimes (X^*)^{\otimes m}.$$

Thus `T_0` has shape `(d_Y,)`, and `T_m` has shape `(d_Y, d_X, ..., d_X)` with m source axes. The order-r directional derivative is

$$D^r F(x)[v_1, \dots, v_r] = \sum_{m=r}^{M} \frac{1}{(m-r)!} T_m[v_1, \dots, v_r, x^{m-r}].$$

This convention matters when moving formulas into code. For example, the scalar polynomial $3x^2$ is encoded by `T_2 = 6`, not `T_2 = 3`.

The member surface is:

- `degree -> int` is the largest stored tensor order.
- `supports_order(order) -> bool` accepts orders from 1 through `degree`, even when a particular intermediate tensor is absent. Missing tensors contribute zero.
- `value(point) -> Any` contracts every stored tensor against repeated copies of the point and divides by the appropriate factorial.
- `derivative(order, point, directions) -> Any` implements the displayed formula.
- `symmetry_certificate() -> Certificate` checks adjacent source-axis transpositions under `PSC-III-D04`.

For each $m \ge 2$, define

$$\rho_m = \max_{1 \le j < m} \|T_m - \operatorname{swap}_{j,j+1} T_m\|.$$

The certificate is exact when `exactness=Exactness.EXACT` and accepts only zero residual. Otherwise it accepts the maximum residual up to the provider's `tolerance`. Construction checks ranks, common target dimension, equal source axes, and common source dimension, but does not require symmetry. A nonsymmetric tensor can therefore be constructed and evaluated. Call `symmetry_certificate().require()` before using symmetric-derivative theorems.

An order-zero-only map cannot infer d_X. In that case `source_dimension` must be supplied. Empty tensor mappings, negative or Boolean orders, inconsistent ranks, inconsistent target dimensions, and unequal source axes fail during construction.

Exact arithmetic. `Exactness.EXACT` converts tensors to object arrays and uses `Fraction(1, m!)` for scaling. Exactness is preserved when coefficients, points, and directions are exact Python, `Fraction`, or symbolic objects. A binary float stored in an object array remains a binary float, despite the exactness label.

Cost and storage. Dense storage is

$$S_{\text{dense}} = \sum_{m \in \mathcal{M}} d_Y d_X^m$$

scalar entries for the stored order set $\mathcal{M}$. Direct value and derivative evaluation touch each applicable dense tensor. Their leading arithmetic work is proportional to the same applicable entry count, with additional contraction temporaries. Section 38.5.1 gives the orbit-sparse alternative.

37.2.3 Recursive central differences

`FiniteDifferenceDerivativeProvider` supplies a numerical fallback when a function can be evaluated at nearby points.

```
FiniteDifferenceDerivativeProvider(
    function: Callable[[np.ndarray], Any],
    source_dimension: int,
    target_dimension: int,
    step: float = 1e-4,
    max_order: int = 4,
    name: str = "central finite difference provider",
    exactness: Exactness = Exactness.NUMERICAL,
)
```

For a direction v, the actual displacement scale is

$$\eta(v) = h / \max(1, \|v\|), \qquad \Delta_v F(x) = \frac{F(x + \eta v) - F(x - \eta v)}{2\eta}.$$

Higher derivatives are recursive:

$$\Delta_{v_1,\dots,v_r} F = \Delta_{v_r}(\Delta_{v_1,\dots,v_{r-1}} F).$$

The provider accepts a positive finite `step`, a positive `max_order`, and positive dimensions. `value` and `derivative` require finite numeric vectors and finite output. Internally the point and directions are converted to complex arrays before recursion. A user function that cannot accept complex-valued arrays is therefore incompatible even when all original data are real.

The result is always numerical. Error depends on smoothness in the sampled neighborhood, derivative order, displacement scale, floating-point roundoff, cancellation, and the magnitude of the directions. The constructor exposes an `exactness` field and does not force it back to `NUMERICAL`, but changing that field cannot make a finite-difference calculation exact. Retain the default label. The provider does not estimate truncation error, compare multiple step sizes, or return a certificate. A reliable application should perform a step sweep and compare it with an analytic, symbolic, or AD oracle whenever one exists.

At order r, recursion performs exactly 2^r terminal function evaluations. Ignoring function cost, its vector arithmetic is $O(2^r d_X)$, with recursive depth r. This exponential dependence is appropriate for small validation probes, not high-order production differentiation.

The provenance record is `PSC-III-N03`. Its assumptions are sufficient local smoothness and a step that resolves the derivative. A nonpositive or nonfinite step, unsupported order, wrong direction count, nonfinite function output, or dimension mismatch fails explicitly.

37.2.4 SymPy derivatives

`SympyDerivativeProvider` differentiates a finite vector of SymPy expressions.

```
SympyDerivativeProvider(
    expressions: Sequence[Any],
    variables: Sequence[Any],
    *,
    name: str = "SymPy derivative provider",
)
```

For directions $v_1, \dots, v_r$, it computes

$$\nabla_{v_r} \cdots \nabla_{v_1} F = \sum_{i_1,\dots,i_r} v_{1,i_1} \cdots v_{r,i_r} \frac{\partial^r F}{\partial z_{i_r} \cdots \partial z_{i_1}}.$$

The implementation applies each directional differential symbolically, then substitutes the point. It reports `Exactness.EXACT`, supports every positive integer order, and returns a `sympy.Matrix`.

The exact label is justified when expressions, point entries, and direction entries are exact SymPy objects. Substitution of floating SymPy values can produce approximate results even though the class-level exactness field remains `EXACT`. This is a caller-visible semantic boundary, not an automatic coercion.

Construction imports SymPy lazily. An absent installation raises `BackendUnavailableError`. Empty expressions, empty variables, or repeated variables raise `ValidationError`. Points and directions with the wrong number of entries raise `DimensionError`. Symbolic differentiation cost depends on expression-tree growth and simplification behavior. There is no dimension-only complexity guarantee and no internal derivative-expression cache.

The finite symbolic claim is provenance record `PSC-III-E05`. It establishes exact evaluation of the constructed expression when exact symbolic data are used. It does not prove smoothness or equivalence to an independently specified map.

37.3 JAX and PyTorch nested-JVP adapters

The automatic-differentiation surface has a framework-neutral adapter plus lazy JAX and PyTorch adapters.

The JVP primitive is

$$\mathrm{jvp}(G, x, v) = (G(x), DG(x)[v]).$$

Nested construction uses

$$G_0 = F, \qquad G_j(x) = DG_{j-1}(x)[v_j], \qquad G_r(x) = D^r F(x)[v_1, \dots, v_r].$$

37.3.1 Framework-neutral JVP provider

```
AutomaticDifferentiationProvider(
    function: Callable[[Any], Any],
    jvp: DirectionalJVP,
    source_dimension: int,
    target_dimension: int,
    max_order: int = 4,
    exactness: Exactness = Exactness.NUMERICAL,
    name: str = "automatic differentiation provider",
)
```

`DirectionalJVP` is a runtime-checkable protocol for a callable returning `(primal, tangent)`. `AutomaticDifferentiationProvider.derivative` nests that callable. A malformed result that is not a two-tuple raises `ValidationError`.

The member surface is:

- `supports_order(order) -> bool` accepts positive integer orders through `max_order`.
- `value(point) -> Any` evaluates the function and checks target dimension.
- `derivative(order, point, directions) -> Any` performs nested JVPs.
- `validation_certificate(point, directions, *, tolerance=1e-8) -> Certificate` checks finite permutation symmetry.
- `to_dict() -> dict[str, Any]` reports name, dimensions, maximum order, and exactness.

The certificate is `PSC-III-N04A`. It checks only finite dimensions and selected permutation probes. It leaves global correctness of the supplied JVP and tracing outside the probes unchecked. An exactness label supplied here is a declaration about the JVP implementation. It is not inferred.

As with declared-callable validation, an empty direction list checks no derivative and produces a zero symmetry residual. The certificate's `checked_orders` diagnostic is therefore part of the evidence, not optional decoration.

37.3.2 JAX and PyTorch adapters

```
JaxDerivativeProvider(
    function: Callable[[Any], Any],
    source_dimension: int,
    target_dimension: int,
    max_order: int = 4,
    dtype: str | None = None,
    name: str = "JAX derivative provider",
    exactness: Exactness = Exactness.NUMERICAL,
)

TorchDerivativeProvider(
    function: Callable[[Any], Any],
    source_dimension: int,
    target_dimension: int,
    max_order: int = 4,
    dtype: str = "float64",
    device: str | None = None,
    name: str = "PyTorch derivative provider",
    exactness: Exactness = Exactness.NUMERICAL,
)
```

Both adapters force their stored exactness to `NUMERICAL`. Both expose `available()`, `value`, `derivative`, `validation_certificate`, and `to_dict`. JAX uses nested `jax.jvp`. PyTorch uses nested `torch.func.jvp`. Final values are detached or transferred as needed and returned as host NumPy arrays.

Availability states must remain distinct:

$$\text{installed} \overset{?}{\Longrightarrow} \text{importable} \overset{?}{\Longrightarrow} \text{JVP-capable} \overset{?}{\Longrightarrow} \text{function traceable}.$$

Komorebi Math probes JAX and PyTorch in isolated child interpreters. This prevents a failing native import from terminating an application that merely asks about capability. A successful probe establishes import and JVP capability only. The supplied function may still fail because it escapes to NumPy, mutates traced values, uses unsupported data-dependent control flow, requests an unavailable device, or uses an invalid dtype.

The discovery and factory calls are:

```
available_ad_frameworks() -> dict[str, bool]
available_autodiff_backends() -> dict[str, bool]

ad_framework_diagnostics(
    *, refresh: bool = False
) -> dict[str, dict[str, Any]]

autodiff_provider(
    framework: str,
    function: Callable[[Any], Any],
    source_dimension: int,
    target_dimension: int,
    *,
    max_order: int = 4,
    **options: Any,
) -> JaxDerivativeProvider | TorchDerivativeProvider

automatic_derivative_provider = autodiff_provider
```

`available_autodiff_backends` is an alias of `available_ad_frameworks`, and `automatic_derivative_provider` is an alias of `autodiff_provider`. Framework names are case-normalized. `"jax"`, `"torch"`, and `"pytorch"` are accepted. Unknown names raise `ValidationError`. There is no silent numerical fallback after an AD failure.

`ad_framework_diagnostics` returns, for each framework, `discovered`, `available`, `version`, `returncode`, and bounded diagnostic `detail`. Its result describes the active Python interpreter and can change with environment repairs. The Chapter 37 reference run found neither optional framework installed, so its two optional tests were skipped rather than treated as failures.

The JAX class record is `PSC-III-N04J`. The PyTorch class record is `PSC-III-N04T`. Both adapters inherit the framework-neutral `validation_certificate`, whose emitted runtime `theorem_id` is the generic `PSC-III-N04A`. Consumers should therefore qualify an identifier by its emitting class or method rather than treating the string alone as a globally unique evidence key. Device placement, precision, compilation, and derivative cost belong to the external runtime. Nested order r requires r levels of JVP transformation, but tracing and execution may duplicate work. No backend-independent timing or memory bound is claimed.

37.4 Composition of providers

`ComposedDerivativeProvider(outer, inner, *, name=None)` presents $F \circ G$ as another derivative provider when the target dimension of `inner` equals the source dimension of `outer`. Its value is `outer.value(inner.value(point))`. Its order-r derivative is assembled by the finite Faà di Bruno routine documented in Section 38.3. An order is supported only when both component providers supply every order from one through r. The combined exactness is the weaker of the two component declarations. This constructor checks finite-dimensional compatibility and provider availability. It does not independently prove that either provider's declared derivatives are correct.

37.5 Backend-isolated probes

JAX and PyTorch availability is tested in bounded child interpreters before a provider is constructed. The resulting discovery record distinguishes an installed distribution, a successful import, a working JVP primitive, and a user function that is actually traceable. A failed probe never silently selects finite differences or another backend. Applications that make backend availability a requirement should inspect `ad_framework_diagnostics`, then construct the selected provider explicitly.

37.5.1 Cross-provider comparison example

Use

$$F(x_1, x_2) = (x_1^3 + x_1 x_2,\ x_2^2)$$

at $x = (2, 3)$, with directions $u = (1, 2)$, $v = (-1, 4)$, and $w = (3, -2)$. The cross-provider comparison residual is

$$\rho_r^{p,q} = \left\| D_p^r F(x)[V] - D_q^r F(x)[V] \right\|, \qquad r = 1, 2, 3.$$

The dense polynomial tensors use `T_2[0,0,1] = T_2[0,1,0] = 1`, `T_2[1,1,1] = 2`, and `T_3[0,0,0,0] = 6`.

```
from __future__ import annotations

import numpy as np
import sympy as sp

from komorebi_math import Exactness
from komorebi_math.nonlinear import (
    CallableDerivativeProvider,
    FiniteDifferenceDerivativeProvider,
    PolynomialDerivativeProvider,
    SympyDerivativeProvider,
)

t2 = np.zeros((2, 2, 2), dtype=object)
t2[0, 0, 1] = t2[0, 1, 0] = 1
t2[1, 1, 1] = 2
t3 = np.zeros((2, 2, 2, 2), dtype=object)
t3[0, 0, 0, 0] = 6

polynomial = PolynomialDerivativeProvider(
    {2: t2, 3: t3},
    exactness=Exactness.EXACT,
)
x, y = sp.symbols("x y")
symbolic = SympyDerivativeProvider((x**3 + x*y, y**2), (x, y))

point = np.array([2, 3], dtype=object)
directions = (
    np.array([1, 2], dtype=object),
    np.array([-1, 4], dtype=object),
    np.array([3, -2], dtype=object),
)

print("value:", polynomial.value(point).tolist())
for order in range(1, 4):
    dense = polynomial.derivative(order, point, directions[:order])
    exact = symbolic.derivative(
        order,
        sp.Matrix(point),
        tuple(sp.Matrix(v) for v in directions[:order]),
    )
```

```
    print(f"D{order} polynomial:", dense.tolist(), "sympy:", list(exact))

numerical = FiniteDifferenceDerivativeProvider(
    lambda z: np.array([z[0]**3 + z[0]*z[1], z[1]**2]),
    source_dimension=2,
    target_dimension=2,
    step=2e-4,
    max_order=2,
)
print(
    "D2 finite difference:",
    np.round(
        numerical.derivative(2, point, directions[:2]).astype(float),
        8,
    ).tolist(),
)
```

Observed output is:

```
value: [Fraction(14, 1), Fraction(9, 1)]
D1 polynomial: [Fraction(19, 1), Fraction(12, 1)] sympy: [19, 12]
D2 polynomial: [Fraction(-10, 1), Fraction(16, 1)] sympy: [-10, 16]
D3 polynomial: [Fraction(-18, 1), Fraction(0, 1)] sympy: [-18, 0]
D2 finite difference: [-10.00000001, 15.99999993]
```

The exact cross-provider residuals are zero. The finite-difference discrepancy is numerical and step-dependent.

This negative example separates the gating symmetry residual from the non-gating Taylor diagnostic.

```
wrong_asymmetric = CallableDerivativeProvider(
    function=lambda z: np.array([z[0]**3 + z[0]*z[1], z[1]**2]),
    derivatives={
        2: lambda z, d: np.array([d[0][0] * d[1][1], 0.0]),
    },
    source_dimension=2,
    target_dimension=2,
)
bad = wrong_asymmetric.validation_certificate(point, directions[:2])
print("asymmetric valid:", bad.valid)
print("symmetry residual:", bad.residuals["derivative_symmetry"])

wrong_symmetric = CallableDerivativeProvider(
    function=lambda z: np.array([z[0]**3 + z[0]*z[1], z[1]**2]),
    derivatives={
        1: lambda z, d: np.zeros(2),
        2: lambda z, d: np.zeros(2),
    },
    source_dimension=2,
    target_dimension=2,
)
diagnostic = wrong_symmetric.validation_certificate(
    point,
    directions[:2],
    check_taylor=True,
    increment=np.array([1.0, -1.0]),
)
print("symmetric-wrong valid:", diagnostic.valid)
print("Taylor diagnostic:", diagnostic.residuals["truncated_taylor"])
```

Observed output is:

```
asymmetric valid: False
symmetry residual: 6.0
symmetric-wrong valid: True
Taylor diagnostic: 19.6468827043885
```

This is not a contradiction. `PSC-III-D02` certifies selected permutation symmetries. It does not certify that arbitrary declared callables are globally the derivatives of `function`.

38. Packet jets and Faà di Bruno composition

A packet jet is one block of a multilinear derivative after projecting each source direction into a named packet and projecting the output into a named target packet. Packet jets expose sparsity, permit exact derivative reconstruction, and provide the finite building blocks for packet-aware composition.

Read the objects in this order. A provider supplies the derivative. A family fixes its base point, order, and packet systems. An address selects one output packet and an ordered tuple of input packets. A support record classifies which addresses were tested, and a snapshot stores evaluated coefficients for one fixed address. These objects serve different purposes: support discovery does not itself execute a family, and a snapshot cannot be moved to a new base point without recapture.

Let the source and target packet systems satisfy

$$I_X = \sum_{a \in A} P_a, \qquad I_Y = \sum_{b \in B} Q_b.$$

For theorem-level exact reconstruction, these identities must hold exactly in the selected backend. A numerical `PacketSystem` validates them within its tolerance. Packet systems require pairwise-annihilating idempotents, but not necessarily self-adjoint Hilbert-space orthogonal projectors. `hilbert_certificate()` is the separate self-adjointness check. The formulas in this chapter are finite-dimensional. They do not claim convergence of an infinite jet, an analytic Taylor series, or a continuum decomposition.

The recommended imports are `PacketSystem` and `Exactness` from `komorebi_math`, followed by nonlinear objects from `komorebi_math.nonlinear`. Low-level record types such as `PacketJetAddress`, `PacketJetSupport`, `PacketTaylorResult`, `FaaDiBrunoResult`, and `PacketJetCoefficient` are available from the nonlinear namespace even when they are not re-exported by the root package.

38.1 Jet addresses, supports, families, and snapshots

This section establishes the address space used by packet jets. Supports, families, and portable snapshots retain those addresses so a coefficient can be traced back to one output packet, an ordered input-packet tuple, and a derivative order.

38.1.1 Jet addresses and blocks

The public type alias and construction records are:

```
PacketLabel = int | str

PacketJetAddress(
    output: str,
    inputs: tuple[str, ...],
)

PacketJet(
    provider: DerivativeProvider,
    point: Any,
    source: PacketSystem,
    target: PacketSystem,
    output_index: int,
    input_indices: tuple[int, ...],
)
```

```
PacketJetFamily(
    provider: DerivativeProvider,
    point: Any,
    source: PacketSystem,
    target: PacketSystem,
    order: int,
)
```

For order r, the address space is

$$\mathcal{A}_r = B \times A^r, \qquad |\mathcal{A}_r| = |B||A|^r, \qquad \alpha = (b; a_1, \dots, a_r).$$

The corresponding block is

$$J^{(r)}_{b;a_1,\dots,a_r}(x)[v_1, \dots, v_r] = Q_b D^r F(x)[P_{a_1} v_1, \dots, P_{a_r} v_r].$$

`PacketJetAddress` is a frozen, slotted record. Its `order` property is `len(inputs)`, and `to_dict()` returns `order`, `output`, and `inputs`.

`PacketJet` is also frozen and slotted. Its public members are:

- `order -> int`, the number of input packet indices.
- `address -> PacketJetAddress`, using packet names rather than integer indices.
- `exactness -> Exactness`, equal to `EXACT` only when the provider and both packet backends are exact. Conditional, heuristic, or planned provider labels are preserved. Other combinations are numerical.
- `evaluate(directions) -> Any`, also available through `jet(directions)`, projects directions, calls the provider, and projects the result.
- `norm_bound(derivative_norm: float) -> float`, the projector-product bound.
- `symmetry_certificate(directions, *, tolerance=None) -> Certificate`, theorem record `PSC-III-D06`.
- `to_dict() -> dict[str, Any]`, a structural metadata record. It does not serialize the provider callable.

The universal operator-norm statement is

$$\|J^{(r)}_{b;\mathbf{a}}\| \le \|Q_b\| \, \|D^r F(x)\| \prod_{j=1}^{r} \|P_{a_j}\|.$$

Thus, if $\|D^r F(x)\| \le M_r$, the same right-hand side is bounded by $\|Q_b\| M_r \prod_j \|P_{a_j}\|$.

`norm_bound` computes the right-hand side from the caller's declared `derivative_norm`. It does not estimate M_r, validate that the supplied value is a finite nonnegative upper bound, or return a certificate. A negative input can therefore produce a meaningless negative result.

For a symmetric derivative, packet symmetry pairs direction permutations with address permutations:

$$J_{b;a_{\sigma(1)},\dots,a_{\sigma(r)}}[v_{\sigma(1)}, \dots, v_{\sigma(r)}] = J_{b;a_1,\dots,a_r}[v_1, \dots, v_r].$$

The symmetry certificate evaluates all $r!$ paired permutations. It leaves global correctness of a user-supplied provider unchecked.

`PacketJetFamily` fixes a provider, point, source system, target system, and derivative order. Its members are:

- `jet(output, inputs) -> PacketJet`, accepting packet names or integer labels.
- `addresses() -> tuple[PacketJetAddress, ...]`, in deterministic output-major, product-lexicographic input order.
- `reconstruct(directions) -> Any`, the sum over every address.
- `reconstruction_certificate(directions, *, tolerance=None) -> Certificate`.
- `to_dict() -> dict[str, Any]`, including the address count.

Source and target packet systems must use the same backend. Provider dimensions must equal packet-system dimensions. The provider must support the requested order. Invalid labels, wrong direction counts, and wrong base-point dimensions fail explicitly.

Cost. One jet evaluation performs r source projector-vector products, one provider derivative evaluation, and one target projector-vector product. For dense $d_X \times d_X$ and $d_Y \times d_Y$ projectors this adds $O(rd_X^2 + d_Y^2)$ arithmetic around the provider. Sparse or symbolic backends have backend-specific costs.

38.2 Multilinear derivative storage

A packet family stores projected blocks of a multilinear derivative rather than one opaque dense tensor. Exact reconstruction and conservative support discovery are the two structural checks needed before composition.

38.2.1 Reconstruction theorem

Multilinearity and packet completeness give

$$D^r F(x)[V] = \sum_{b \in B} \sum_{\mathbf{a} \in A^r} J^{(r)}_{b;\mathbf{a}}(x)[V].$$

The certificate residual is

$$\rho_{\text{rec}} = \left\| D^r F(x)[V] - \sum_{\alpha \in \mathcal{A}_r} J_\alpha(x)[V] \right\|.$$

`PacketJetFamily.reconstruction_certificate` compares a direct provider evaluation with the reconstructed sum. The theorem record is `PSC-III-T07`, whose checked assumptions are complete input packets, complete output packets, and multilinearity. The certificate reports order, address count, provider name, and residual.

For an exact provider and exact packet backend, only zero residual passes. Otherwise the threshold is the maximum source and target packet tolerance unless the caller supplies `tolerance`.

The reconstruction enumerates all $|B||A|^r$ addresses and does not consult support data. Its cost is therefore exponential in derivative order when the packet count exceeds one:

$$C_{\text{reconstruct}} = |B||A|^r C_J,$$

where C_J is one packet-jet evaluation. This direct path is a transparent reference calculation. Section 38.2.2 classifies support, and later execution APIs compile selected batches, but support does not alter `PacketJetFamily.reconstruct` itself.

38.2.2 Support discovery

`packet_jet_support` exhaustively tests each packet block on bases for the corresponding source packet ranges.

```
PacketJetSupport(
    order: int,
    nonzero: tuple[PacketJetAddress, ...],
    zero: tuple[PacketJetAddress, ...],
    untested: tuple[PacketJetAddress, ...],
    evaluations: int,
    certificate: Certificate,
)

packet_jet_support(
    family: PacketJetFamily,
    *,
    tolerance: float | None = None,
    max_evaluations: int = 100_000,
) -> PacketJetSupport
```

For packet-range bases $\mathcal{B}_a$, a block is certified zero when

$$J_\alpha[e_1, \dots, e_r] = 0 \quad \forall (e_1, \dots, e_r) \in \mathcal{B}_{a_1} \times \cdots \times \mathcal{B}_{a_r}.$$

The resulting partition is

$$\mathcal{A}_r = \mathcal{A}_r^{\neq 0} \sqcup \mathcal{A}_r^{0} \sqcup \mathcal{A}_r^{?}.$$

An exact SymPy backend obtains each packet-range basis from `columnspace()` and tests exact zero. A numerical backend obtains the range basis from an SVD of each source projector. Its rank threshold is the larger of the absolute tolerance and that tolerance times the leading singular value.

The record's `complete` property is true exactly when `untested` is empty. `contains(output, inputs)` tests membership in `nonzero` only. A false result means *not certified nonzero*. It does not distinguish certified zero from untested. Inspect `zero` and `untested` when that distinction matters.

The global `max_evaluations` budget is checked while addresses are processed. Once exhausted, the current and all remaining addresses are placed in `untested`. The certificate remains valid because it truthfully records the finite classification, but its exactness changes to `CONDITIONAL` and its unchecked assumptions mention the untested addresses. An untested address must never be pruned.

Let $k_a = \dim \operatorname{range} P_a$. Apart from basis construction, exhaustive work is bounded by

$$N_{\text{support}} \le \sum_{b \in B} \sum_{(a_1, \dots, a_r) \in A^r} \prod_{j=1}^{r} k_{a_j} = |B| \left(\sum_{a \in A} k_a \right)^r = |B| d_X^r.$$

Nonzero blocks may terminate early after the first nonzero basis value. Zero blocks require their full basis product. `max_evaluations` caps jet evaluations but does not cap SVD or symbolic basis construction.

The theorem record is `PSC-III-C08`. The exact claim is finite multilinear basis classification. Numerical zero is tolerance-dependent and must not be described as structural exact zero.

38.3 Faà di Bruno formula

The finite composition law is organized by set partitions. The packet Taylor construction below fixes the directional and projection conventions before the classical and packet-decorated Faà di Bruno sums are evaluated.

38.3.1 Packet Taylor polynomial

The public result and function are:

```
PacketTaylorResult(
    approximation: Any,
    observed: Any,
    truncation_residual: float,
    order_certificates: tuple[Certificate, ...],
    certificate: Certificate,
)

packet_taylor(
    provider: DerivativeProvider,
    point: Any,
    increment: Any,
    source: PacketSystem,
    target: PacketSystem,
    order: int,
    *,
    tolerance: float | None = None,
) -> PacketTaylorResult
```

It assembles

$$T_nF(x;h) = F(x) + \sum_{r=1}^{n} \frac{1}{r!} \sum_{\alpha \in \mathcal{A}_r} J_\alpha(x)[h^r].$$

The observed diagnostic is

$$\rho_n(x,h) = \|F(x+h) - T_nF(x;h)\|.$$

For every retained order, the implementation constructs a `PacketJetFamily`, obtains its reconstruction certificate, and separately reconstructs the derivative. `certificate.valid` is the conjunction of the retained-order reconstruction certificates. It does not depend on ρ_n. The residual is stored as `observed_truncation_residual`, and the certificate leaves the omitted Taylor remainder and global correctness of user derivatives unchecked.

Order zero is valid and returns `provider.value(point)` as the approximation, with no order certificates. Negative, Boolean, or nonintegral orders fail. Positive orders construct a packet family and therefore enforce provider dimensions and backend compatibility. At order zero no family is constructed, so callers should verify target dimension and backend compatibility explicitly rather than relying on that path as a structural validator.

The theorem record is `PSC-III-C09`. The exact assertion is that the retained Taylor terms were assembled from reconstructed packet derivatives. Equality with $F(x+h)$ occurs for a polynomial when the retained order reaches its degree, but it is not a general certificate condition.

The implementation computes each retained reconstruction twice, once inside its certificate and once for the term. Consequently the transparent cost is approximately

$$2\sum_{r=1}^{n} |B||A|^r C_{J_r} + 2C_F,$$

plus vector additions and factorial scaling. This is a reference implementation rather than a reuse-optimized Taylor compiler.

38.3.2 Formula and partition structure

38.3.2.1 Canonical set partitions and counts

The public combinatoric surface is:

```
Partition = tuple[tuple[int, ...], ...]

set_partitions(order: int) -> tuple[Partition, ...]
stirling_second(order: int, blocks: int) -> int

labeled_composition_term_count(
    order: int,
    intermediate_packets: int,
) -> int
```

`set_partitions(n)` returns a cached deterministic enumeration of partitions of `range(n)`. Entries within a block and blocks within a partition follow their least elements. It returns `((),)` at order zero. The first Bell counts are 1, 1, 2, 5, 15, 52, and 203 for orders zero through six.

The counts are

$$|\Pi_n| = \sum_{k=0}^{n} S(n,k), \qquad N_{\text{labeled}}(n,m) = \sum_{k=1}^{n} S(n,k)m^k.$$

`labeled_composition_term_count` requires positive integer n and m. `set_partitions` strictly validates a nonnegative integer order. `stirling_second` returns zero for negative arguments or $k > n$, but in this release it converts argu-

ments with `int()` without the same strict nonintegral and Boolean guard. Treat integer input as part of its public precondition.

`set_partitions(n)` materializes every partition and retains the immutable tuple in an unbounded process-local cache. Its output storage is $\Theta(B_n n)$ integer positions up to tuple overhead. `stirling_second(n, k)` builds an $(n+1) \times (k+1)$ dynamic-programming table, taking $O(nk)$ time and storage. `labeled_composition_term_count` calls that routine separately for each $k = 1, \ldots, n$, so the current transparent implementation uses $O(n^3)$ aggregate table work and $O(n^2)$ peak table storage before big-integer costs.

38.3.2.2 Classical finite-dimensional composition

For $H = F \circ G$,

$$D^n(F \circ G) = \sum_{\pi \in \Pi_n} D^{|\pi|}F(G) \prod_{B \in \pi} D^{|B|}G.$$

Writing every multilinear argument explicitly gives

$$D^n H(x)[V] = \sum_{\pi \in \Pi_n} D^{|\pi|}F(G(x)) \left[D^{|B|}G(x)[v_i]_{i \in B}\right]_{B \in \pi}.$$

The public records and calls are:

```
FaaDiBrunoTerm(
    partition: Partition,
    intermediate_packets: tuple[str, ...],
    value: Any,
    pruned: bool = False,
    reason: str = "",
)

FaaDiBrunoResult(
    value: Any,
    terms: tuple[FaaDiBrunoTerm, ...],
    raw_term_count: int,
    evaluated_term_count: int,
    pruned_term_count: int,
    certificate: Certificate,
)

faa_di_bruno(
    outer: DerivativeProvider,
    inner: DerivativeProvider,
    point: Any,
    directions: Sequence[Any],
    *,
    direct_provider: DerivativeProvider | None = None,
    tolerance: float = 1e-9,
    retain_terms: bool = True,
) -> FaaDiBrunoResult

ComposedDerivativeProvider(
    outer: DerivativeProvider,
    inner: DerivativeProvider,
    *,
    name: str | None = None,
)
```

At least one direction is required. The inner target dimension must equal the outer source dimension, and both providers must supply *every* derivative order from one through n. Requiring lower orders is essential because a partition can contain blocks of every size through n.

`retain_terms=False` avoids storing term records but does not reduce evaluation work. The raw and evaluated term counts are both the Bell number $|\Pi_n|$, with zero pruned terms.

Without `direct_provider`, the result's certificate has no direct-comparison residual. Its validity records successful finite set-partition assembly, and global correctness of the declared providers remains unchecked. With a compatible direct provider for $F \circ G$, the certificate compares the assembled value with the direct order-n derivative. The exactness is combined across providers. The theorem record is `PSC-III-T10`.

`ComposedDerivativeProvider` exposes the same assembly through the provider protocol. `supports_order(n)` requires all component orders through n. `value(x)` computes `outer.value(inner.value(x))`, and `derivative` delegates to `faa_di_bruno(..., retain_terms=False)`.

Classical composition evaluates one inner derivative per partition block and one outer derivative per partition. Term count grows as a Bell number. Retaining results adds $O(B_n)$ record storage, excluding the potentially large stored values.

38.3.2.3 Packet-decorated composition

For output packet c, source labels a_i, intermediate packet set M, and label map $\lambda : \pi \to M$, the packet identity is

$$Q_c D^n (F \circ G)(x)[P_{a_1} v_1, \dots, P_{a_n} v_n]$$

$$= \sum_{\pi \in \Pi_n} \sum_{\lambda : \pi \to M} J^F_{c;(\lambda(B))_{B \in \pi}} \left[J^G_{\lambda(B);(a_i)_{i \in B}} [v_i]_{i \in B} \right]_{B \in \pi}.$$

The public call is:

```
packet_faa_di_bruno(
    outer: DerivativeProvider,
    inner: DerivativeProvider,
    point: Any,
    directions: Sequence[Any],
    source: PacketSystem,
    intermediate: PacketSystem,
    target: PacketSystem,
    *,
    input_packets: Sequence[PacketLabel],
    output_packet: PacketLabel,
    inner_support: Mapping[int, PacketJetSupport] | None = None,
    outer_support: Mapping[int, PacketJetSupport] | None = None,
    tolerance: float | None = None,
    retain_terms: bool = True,
    max_terms: int = 1_000_000,
) -> FaaDiBrunoResult
```

All three packet systems must share one backend. Provider dimensions must match their source, intermediate, and target spaces. Exactly one source packet is supplied per direction.

The raw labeled count $N_{\text{labeled}}(n, |M|)$ is compared with `max_terms` *before* any support pruning. If it exceeds the limit, the call raises `ValidationError`. Certified zero outer jets prune a term. Certified zero inner jets also prune a term. Missing support, incomplete support, nonzero support, and untested addresses never justify pruning.

The result compares the decorated sum with the target projection of a classical Faà di Bruno evaluation on source-projected directions. Its reconstruction residual gates certificate `PSC-III-T11A`. Diagnostics report raw, evaluated, and pruned terms.

The current over-budget error text suggests supplying structural support or raising the limit. The implementation still rejects on raw count before consulting support, so only reducing order or intermediate packet count, or explicitly raising `max_terms`, can pass that guard. This wording mismatch is documented here rather than hidden.

When `retain_terms=True`, pruned terms are retained with `value=None`, `pruned=True`, and reason `zero outer packet jet` or `zero inner packet jet`. When false, only counts remain. The worst-case evaluation and record count is exactly $N_{\text{labeled}}(n, |M|)$.

38.4 Packet composition and Taylor expansion

`packet_faa_di_bruno` combines the projected inner and outer blocks described above, while `packet_taylor` assembles a finite repeated-direction polynomial. Both are finite identities with explicit support, term-budget, residual, and exactness records. Neither asserts convergence of an infinite analytic expansion.

38.5 Sparse and portable jet records

Dense derivative tensors are not required. Orbit-sparse symmetric tensors and portable coefficient snapshots provide two bounded representations for storage, transfer, digest checking, and later execution.

38.5.1 Sparse symmetric tensors

Dense derivative tensors repeat equal coefficients across every permutation of symmetric source indices. The sparse representation stores one coefficient per nondecreasing orbit.

```
SparseTensorEntry(
    output: int,
    inputs: tuple[int, ...],
    value: Any,
)

SparseSymmetricTensor(
    order: int,
    source_dimension: int,
    target_dimension: int,
    entries: tuple[SparseTensorEntry, ...] = (),
)
```

For a stored orbit $(o; i_1 \le \cdots \le i_r, c)$, contraction is

$$T[v_1, \dots, v_r]_o = \sum_{(o,\mathbf{i},c)} c \sum_{\sigma \in \mathrm{Orb}(\mathbf{i})} \prod_{j=1}^{r} (v_j)_{i_{\sigma(j)}}.$$

The size measures are

$$N_{\mathrm{dense}} = d_Y d_X^r, \qquad \rho_{\mathrm{storage}} = N_{\mathrm{stored}} / N_{\mathrm{dense}}.$$

`SparseSymmetricTensor` exposes:

- `from_mapping(order, source_dimension, target_dimension, entries)`, where keys are `(output, *sorted_inputs)`.
- `from_dense(tensor, *, source_dimension=None, tolerance=0.0, validate_symmetry=True)`.
- `stored_entry_count`, `dense_entry_count`, `expanded_nonzero_count`, and `storage_ratio` properties.
- `contract(vectors, *, exact=False) -> np.ndarray`.
- `to_dense(*, dtype=None) -> np.ndarray`.
- `certificate(*, tolerance=0.0) -> Certificate`, record `PSC-III-D04S`.
- `to_dict() -> dict[str, Any]`.

Construction removes exactly zero stored coefficients, sorts entries, and rejects duplicate orbit keys, out-of-range indices, wrong orbit lengths, and decreasing input tuples. `from_dense` validates all distinct permutations of each representative when `validate_symmetry=True`. A positive tolerance can omit small dense coefficients and accept approximate symmetry, so that compression is numerical.

`SparseTensorEntry` normalizes indices with `int()`. The surrounding tensor validates ranges and ordering but does not separately reject every nonintegral value before conversion. Supply genuine integers rather than relying on coercion.

Contraction cost is

$$O\left(r \sum_{e \in E} |\operatorname{Orb}(e)|\right),$$

with at most $r!$ distinct permutations per stored orbit. `to_dense` writes `expanded_nonzero_count` entries and allocates $d_Y d_X^r$ elements.

The sparse polynomial provider applies the same factorial convention as Section 37.2.2:

```
SparsePolynomialDerivativeProvider(
    tensors: Mapping[
        int,
        SparseSymmetricTensor | Mapping[tuple[int, ...], Any] | Any,
    ],
    *,
    source_dimension: int | None = None,
    target_dimension: int | None = None,
    exactness: Exactness = Exactness.NUMERICAL,
    tolerance: float = 1e-10,
    name: str = "sparse polynomial derivative provider",
)
```

Its public surface includes:

- `from_dense_provider(provider, *, tolerance=None, name=...)`.
- `degree`, `stored_entry_count`, `dense_entry_count`, and `storage_ratio`.
- `supports_order`, `value`, and `derivative`.
- `to_dense_provider() -> PolynomialDerivativeProvider`.
- `storage_certificate(points=(), directions=(), *, max_order=None, tolerance=None) -> Certificate`.
- `to_dict() -> dict[str, Any]`.

`storage_certificate` compares sparse and dense values and derivatives only at supplied probes. With no point probes, its zero residual certifies structural conversion machinery but does not test a function value. With probes, it leaves global equality beyond those probes unchecked. Exact providers require zero residual. Numerical providers use the selected tolerance. The record is `PSC-III-D04P`.

Sparse storage is beneficial only when orbit count is substantially below dense size. A tensor can be symmetric yet dense in orbit coordinates. `storage_ratio` is a representation statistic, not a runtime-speed certificate. Orbit expansion, Python-level iteration, object arithmetic, and dtype conversions can outweigh saved multiplication for small dense problems.

38.5.2 Finite snapshots

A `PacketJet` contains executable provider state and is not safely reconstructible from JSON. A snapshot evaluates one fixed block on the ambient standard basis and stores its nonzero coefficient vectors.

```
PacketJetCoefficient(
    indices: tuple[int, ...],
    value: tuple[Any, ...],
)

PacketJetSnapshot(
    address: PacketJetAddress,
    source_dimension: int,
    target_dimension: int,
    provider_name: str,
    exactness: Exactness,
    point: tuple[Any, ...],
    coefficients: tuple[PacketJetCoefficient, ...],
    capture_certificate: Certificate,
)
```

```
snapshot_packet_jet(
    jet: PacketJet,
    *,
    tolerance: float | None = None,
    max_evaluations: int = 100_000,
) -> PacketJetSnapshot
```

The captured coefficient is

$$c_{o;i_1,\dots,i_r} = (J[e_{i_1},\dots,e_{i_r}])_o,$$

and snapshot evaluation is

$$J[v_1,\dots,v_r]_o = \sum_{i_1,\dots,i_r} c_{o;i_1,\dots,i_r} \prod_{j=1}^{r} (v_j)_{i_j}.$$

`PacketJetSnapshot` exposes:

- `order`, `nnz`, and `dense_coefficient_count` properties.
- `coefficient(indices)`, returning an explicit zero target vector when an address is unstored.
- `evaluate(directions)`, also available through `snapshot(directions)`.
- `to_dense(*, max_elements=10_000_000)`.
- `comparison_certificate(jet, directions, *, tolerance=None)`.
- `to_dict()`.

Capture requires d_X^r jet evaluations and checks this count against `max_evaluations` before starting. Exact snapshots omit only exact zero coefficient vectors. Numerical snapshots omit coefficient vectors whose norm is at most the declared tolerance and record the maximum omitted basis norm.

The largest omitted basis norm is not a uniform error bound for arbitrary large directions. Direction products can amplify many omitted coefficients. The snapshot remains a tolerance-compressed numerical representation whose intended accuracy should be checked at application-relevant probes.

Stored coefficient payload is $O(d_Y \,\mathrm{nnz})$, while dense expansion allocates $d_Y d_X^r$ scalar elements. `to_dense` compares that full element count with `max_elements`. Capture's `max_evaluations` controls basis addresses only and does not multiply the budget by d_Y.

`comparison_certificate` requires matching packet address and dimensions, then compares one supplied probe. Its `assumptions_unchecked` text says that probes were not supplied even though the method does receive directions. That diagnostic wording is a known record defect in `1.0.0b4`. The actual residual is computed from the supplied directions.

The capture record is `PSC-III-D58`. A snapshot fixes order, packet address, dimensions, provider name, exactness, base point, and coefficients. It does not carry executable provider logic and cannot be moved to another base point.

Portable persistence is supplied by `komorebi_math.export.persistence`:

```
dumps_portable(value: Any, *, indent: int | None = 2, max_depth: int = 128) -> str
loads_portable(text: str | bytes, *, max_depth: int = 128) -> Any
portable_digest(value: Any, *, max_depth: int = 128) -> str
```

The portable envelope verifies a SHA-256 payload digest and reconstructs only registered codecs. Digest equality is an integrity statement. It is not a signature, authorship proof, or authenticity claim.

38.5.3 Example ladder

38.5.3.1 Exact packet reconstruction, support, and Taylor assembly

```
import sympy as sp

from komorebi_math import PacketSystem
from komorebi_math.nonlinear import (
    PacketJetFamily,
    SympyDerivativeProvider,
    packet_jet_support,
    packet_taylor,
)

x, y = sp.symbols("x y")
provider = SympyDerivativeProvider((x**2, y**2), (x, y))
packets = PacketSystem(
    (sp.diag(1, 0), sp.diag(0, 1)),
    names=("x", "y"),
    backend="sympy",
)
point = sp.Matrix([1, 2])
directions = (sp.Matrix([2, 3]), sp.Matrix([-1, 4]))

family = PacketJetFamily(provider, point, packets, packets, order=2)
family.reconstruction_certificate(directions).require()
support = packet_jet_support(family)
partial = packet_jet_support(family, max_evaluations=1)
taylor = packet_taylor(
    provider,
    point,
    sp.Matrix([3, -1]),
    packets,
    packets,
    2,
)

print("address count:", len(family.addresses()))
print("nonzero:", support.nonzero)
print("zero blocks:", len(support.zero), "complete:", support.complete)
print(
    "partial evaluations:", partial.evaluations,
    "untested:", len(partial.untested),
    "exactness:", partial.certificate.exactness.value,
)
print(
    "Taylor approximation:", list(taylor.approximation),
    "residual:", taylor.truncation_residual,
)
```

Observed output is:

```
address count: 8
nonzero: (PacketJetAddress(output='x', inputs=('x', 'x')), PacketJetAddress(output='y', inputs=('y', 'y')))
zero blocks: 6 complete: True
partial evaluations: 1 untested: 7 exactness: conditional
Taylor approximation: [16, 1] residual: 0.0
```

The partial support record is valid and conditional. It does not promote its seven untested addresses to zero.

38.5.3.2 Exact Faà di Bruno composition and partition counts

The shipped `examples/nonlinear_jets_composition.py` uses the factorial tensor convention for

$$F(u) = u^2, \qquad G(x) = x + x^2,$$

at $x = 2$ in directions 3 and 4.

```
import numpy as np

from komorebi_math import Exactness
from komorebi_math.nonlinear import (
    PolynomialDerivativeProvider,
    faa_di_bruno,
)
from komorebi_math.nonlinear.composition import set_partitions

outer = PolynomialDerivativeProvider(
    {2: np.array([[[2]]], dtype=object)},
    exactness=Exactness.EXACT,
)
inner = PolynomialDerivativeProvider(
    {
        1: np.array([[1]], dtype=object),
        2: np.array([[[2]]], dtype=object),
    },
    exactness=Exactness.EXACT,
)
result = faa_di_bruno(
    outer,
    inner,
    np.array([2], dtype=object),
    (np.array([3], dtype=object), np.array([4], dtype=object)),
)
result.certificate.require()

print("Bell counts order 0..4:", [len(set_partitions(n)) for n in range(5)])
print("second directional derivative:", result.value[0])
print("set-partition terms:", result.raw_term_count)
```

Observed output is:

```
Bell counts order 0..4: [1, 1, 2, 5, 15]
second directional derivative: 888
set-partition terms: 2
```

The focused exact test suite independently compares classical and packet Faà di Bruno composition through order four with direct SymPy derivatives.

38.5.3.3 Exact sparse contraction

```
from fractions import Fraction

import numpy as np

from komorebi_math import Exactness
from komorebi_math.nonlinear import SparsePolynomialDerivativeProvider

tensor = np.zeros((1, 2, 2), dtype=object)
tensor[0, 0, 0] = Fraction(2, 3)
tensor[0, 0, 1] = tensor[0, 1, 0] = Fraction(1, 5)
```

```
provider = SparsePolynomialDerivativeProvider(
    {2: tensor},
    source_dimension=2,
    exactness=Exactness.EXACT,
)
value = provider.derivative(
    2,
    np.array([0, 0], dtype=object),
    (
        np.array([Fraction(1, 2), Fraction(2, 3)], dtype=object),
        np.array([Fraction(3, 4), Fraction(1, 7)], dtype=object),
    ),
)
provider.storage_certificate(
    points=(np.array([0, 0], dtype=object),),
    directions=(
        np.array([Fraction(1, 2), Fraction(2, 3)], dtype=object),
        np.array([Fraction(3, 4), Fraction(1, 7)], dtype=object),
    ),
).require()

print("sparse exact D2:", value[0])
print(
    "stored/dense:",
    f"{provider.stored_entry_count}/{provider.dense_entry_count}",
)
```

Observed output is:

```
sparse exact D2: 51/140
stored/dense: 2/4
```

38.5.3.4 Snapshot roundtrip, digest, and tamper rejection

```
import json

import numpy as np

from komorebi_math import PacketSystem
from komorebi_math.exceptions import ValidationError
from komorebi_math.export.persistence import (
    dumps_portable,
    loads_portable,
    portable_digest,
)
from komorebi_math.nonlinear import (
    PacketJetFamily,
    PolynomialDerivativeProvider,
    snapshot_packet_jet,
)

packets = PacketSystem(
    (np.diag([1.0, 0.0]), np.diag([0.0, 1.0])),
    names=("x", "y"),
)
tensor = np.zeros((2, 2, 2))
tensor[0, 0, 0] = 2.0
tensor[1, 1, 1] = 2.0
provider = PolynomialDerivativeProvider(
    {2: tensor},
    source_dimension=2,
```

```
    target_dimension=2,
)
jet = PacketJetFamily(
    provider,
    np.zeros(2),
    packets,
    packets,
    2,
).jet("x", ("x", "x"))

snapshot = snapshot_packet_jet(jet)
restored = loads_portable(dumps_portable(snapshot))
probe = (np.array([2.0, 3.0]), np.array([-1.0, 4.0]))
snapshot.comparison_certificate(jet, probe).require()

print("snapshot nnz:", snapshot.nnz)
print("roundtrip:", restored == snapshot)
print("value:", restored.evaluate(probe).tolist())
print("digest:", portable_digest(snapshot))

envelope = json.loads(dumps_portable(snapshot))
envelope["payload_sha256"] = "0" * 64
try:
    loads_portable(json.dumps(envelope))
except ValidationError as exc:
    print("tamper rejection:", type(exc).__name__, str(exc))
```

Observed output is:

```
snapshot nnz: 1
roundtrip: True
value: [(-4+0j), 0j]
digest: 7fbe4ca3a9e166083e4621c3d39a5fc71db307b395a8b159fbbef0d076e170fb
tamper rejection: ValidationError portable payload SHA256 mismatch
```

The digest is deterministic for this exact serialized record and this release's codec. It is not a security signature.

38.5.3.5 Verification and source-truth ledger

The Chapter 37 through 38 focused replay executed:

- `tests/test_nonlinear_jets_composition.py`.
- `tests/test_nonlinear_sparse_autodiff.py`.
- `tests/test_nonlinear_snapshots_persistence.py`.
- `examples/nonlinear_jets_composition.py`.
- Every new code path whose observed output appears above.

The result was **21 passed and 2 skipped**. The skipped cases were isolated JAX and PyTorch adapter tests because neither optional framework was installed in the reference environment. Their skips confirm the optional-backend contract and do not qualify either external runtime.

These source boundaries remain visible:

- Symmetry probes do not establish global derivative correctness.
- Taylor residuals are diagnostics and do not gate provider or packet-Taylor certificates.
- `Exactness.EXACT` relies on genuinely exact caller data.
- Numerical support depends on SVD rank and zero tolerances.
- Untested support addresses are not zero addresses.
- Raw packet-composition term limits are enforced before pruning.
- Sparse storage ratios are not performance benchmarks.
- Snapshot omission tolerance is not an arbitrary-direction error bound.

- Portable digests establish integrity only.
- Several tolerance-bearing low-level certificate calls accept a caller-supplied float without a uniform finite, non-negative guard. Production callers should validate tolerances before passing them.

These boundaries are part of the interface contract. They prevent a finite computation record from being misreported as a global analytic theorem, a backend qualification, or an authenticity claim.

39. Nonlinear trees and graded operations

The finite nonlinear layer turns two familiar infinite-looking constructions into bounded, inspectable computations. Ordered rooted trees encode finite Picard iterates of polynomial fixed-point equations. Finite abelian labels encode packet-selection rules for multilinear operations and finite Maurer-Cartan expressions. The implementation does not infer convergence, multilinearity, grading, or algebraic identities from a successful run. Those hypotheses remain visible in the certificates that use them.

The complete Chapter 39 surface is exported by `komorebi_math.nonlinear`:

```
from komorebi_math.nonlinear import (
    FiniteAbelianGroup,
    GradedPacketOperation,
    OrderedRootedTree,
    PolynomialFixedPointProblem,
    majorant_convergence,
    maurer_cartan_packets,
    packet_tree_reconstruction,
)
```

The principal problem classes and constructor functions are also re-exported from `komorebi_math`, but result records such as `TreeExpansionResult`, `GradingCertificate`, and `MaurerCartanResult` belong to the broader research namespace. None of these names is in the curated `komorebi_math.stable` or `komorebi_math.beta` surface in version `1.0.0b4`. Pin the package version when these signatures form an application contract.

The claim labels used below are:

- **E** for exact finite algebra or combinatorics.
- **N** for a finite numerical identity accepted at a stated tolerance.
- **C** for a conclusion conditional on declared, unchecked hypotheses.
- **M** for a work or storage model rather than a timed benchmark.
- **I** for provenance or digest integrity rather than mathematical truth.

The namespace control card is `FEATURE_KEY = "nonlinear"`. `status() -> FeatureSpec` reports the feature as implemented and `require() -> None` is the implemented-feature compatibility no-op. The registry summary explicitly leaves infinite completion and analytic convergence conditional on supplied hypotheses.

39.1 Ordered rooted trees

An ordered rooted tree is either a leaf or an internal root with an ordered tuple of subtrees:

$$\tau ::= \bullet \mid B_r(\tau_1, \dots, \tau_r), \qquad r \in \mathcal{R} \subset \{2, 3, \dots\}.$$

The order of the children matters. Thus $B_2(\bullet, B_2(\bullet, \bullet))$ and $B_2(B_2(\bullet, \bullet), \bullet)$ are distinct. An internal vertex with zero or one child is not in this grammar.

`OrderedRootedTree` is a frozen, slotted, orderable dataclass with one field:

```
OrderedRootedTree(
    children: tuple[OrderedRootedTree, ...] = (),
)
```

Its finite structural record is:

- `leaf() -> OrderedRootedTree` constructs •.
- `is_leaf -> bool` tests whether `children` is empty.
- `arity -> int` returns the root arity, with zero for a leaf.
- `height -> int` returns zero for a leaf and one plus the maximum child height otherwise.
- `leaf_count -> int`, `internal_count -> int`, and `vertex_count -> int` count the corresponding vertices.
- `arity_profile() -> dict[int, int]` counts internal vertices by arity.
- `prefix() -> str` writes a leaf as `s` and an internal tree as `Br(child,...)`.
- `to_dict() -> dict[str, Any]` recursively records `arity`, `height`, `leaves`, and `children`.

The runtime coerces `children` to a tuple and rejects a non-tree child or an internal root of arity one. These metadata are exact finite combinatorics. **E**

Height-bounded families

For a fixed allowed arity set $\mathcal{R}$, let N_h count all ordered trees of height at most h. The implementation uses the exact recurrence

$$N_0 = 1, \qquad N_h = 1 + \sum_{r \in \mathcal{R}} N_{h-1}^r.$$

The leading one is the leaf. Each other term chooses an allowed root arity and an ordered subtree of height at most $h - 1$ in every child slot.

```
ordered_tree_count(
    max_height: int,
    arities: Sequence[int],
) -> int

ordered_rooted_trees(
    max_height: int,
    arities: Sequence[int],
    *,
    max_trees: int = 100000,
) -> tuple[OrderedRootedTree, ...]
```

`arities` is normalized to a sorted duplicate-free tuple. Every entry must be an integer at least two, booleans are rejected, and the normalized set must be nonempty. The height must be a nonnegative integer. Enumeration first computes the exact count and raises `ValidationError` before materialization when the count exceeds `max_trees`. The returned tuple is cached by height and normalized arity set. **E**

For binary trees the first height-bounded counts are

$$(N_0, N_1, N_2, N_3) = (1, 2, 5, 26).$$

The family therefore grows far faster than its small-height examples suggest. The explicit count guard is a mathematical resource boundary, not a claim that height four or greater is intrinsically unsafe.

Leaf-bounded families and Catalan numbers

The alternative enumerator fixes the number of leaves:

```
ordered_trees_with_leaves(
    leaves: int,
    arities: Sequence[int],
    *,
    max_trees: int = 100000,
) -> tuple[OrderedRootedTree, ...]

catalan_number(index: int) -> int
```

For every allowed root arity r, the routine enumerates every positive composition

$$n = n_1 + \cdots + n_r, \qquad n_j \geq 1,$$

then takes the Cartesian product of the recursively generated child families. For the binary family with n leaves, the exact count is

$$C_{n-1} = \frac{1}{n}\binom{2n-2}{n-1}, \qquad C_k = \frac{1}{k+1}\binom{2k}{k}.$$

The leaf count must be a positive integer and the Catalan index a nonnegative integer.

Before requesting a large family, choose whether height or leaf count is the intended constraint and compute the corresponding count. They describe different finite sets. For evaluating a fixed number of Picard iterations, follow the height convention used by `tree_expansion`; a leaf-count cutoff is not a substitute for that iteration depth.

Unlike `ordered_rooted_trees`, the leaf-bounded routine checks `max_trees` after its cached family has been generated. The limit prevents an oversized result from being returned, but it is not a pre-allocation memory guard for this path. This source-level distinction matters in automated workloads. **E/M**

Minimal structural example

```
from komorebi_math.nonlinear import (
    OrderedRootedTree,
    catalan_number,
    ordered_tree_count,
)

leaf = OrderedRootedTree.leaf()
tree = OrderedRootedTree((leaf, OrderedRootedTree((leaf, leaf))))

print(tuple(ordered_tree_count(h, (2,)) for h in range(4)))
print(tuple(catalan_number(n) for n in range(7)))
print(
    tree.prefix(),
    tree.height,
    tree.leaf_count,
    tree.internal_count,
    tree.vertex_count,
)
```

Observed output:

```
(1, 2, 5, 26)
(1, 1, 2, 5, 14, 42, 132)
B2(s,B2(s,s)) 2 3 2 5
```

39.2 Polynomial fixed-point trees

`PolynomialFixedPointProblem` represents the finite-dimensional equation

$$u = s + \sum_{r \in \mathcal{R}} B_r(u, \dots, u) =: \Phi(u), \qquad u_0 = 0, \qquad u_{m+1} = \Phi(u_m).$$

Every operation is supplied as a callable accepting one tuple of r arguments. The public typing alias is

```
MultilinearOperation = Callable[[tuple[Any, ...]], Any]
```

It describes the expected calling convention. The runtime checks arity and vector dimensions, but it does not prove that a supplied callable is multilinear.

The tree valuation is recursive:

$$\mathrm{val}(\bullet) = s, \qquad \mathrm{val}(B_r(\tau_1, \dots, \tau_r)) = B_r(\mathrm{val}\,\tau_1, \dots, \mathrm{val}\,\tau_r).$$

Distributing each finite Picard iterate gives the exact finite identity

$$u_m = \sum_{h(\tau) < m} \mathrm{val}(\tau).$$

This identity is about a particular finite depth. It neither takes a limit nor asserts that u_m converges.

Problem reference

```
PolynomialFixedPointProblem(
    seed: Any,
    operations: Mapping[int, MultilinearOperation],
    *,
    exactness: Exactness = Exactness.NUMERICAL,
    tolerance: float = 1e-10,
    name: str = "polynomial fixed-point problem",
)
```

The constructor determines one vector dimension from `seed`, normalizes the operation keys to integer arities at least two, requires at least one callable, and rejects a nonfinite or negative tolerance.

Its callable surface is:

- `operation(arity: int, arguments: Sequence[Any]) -> Any` validates the declared arity, argument count, input dimensions, and output dimension.
- `map(value: Any) -> Any` evaluates $\Phi(value)$.
- `residual(value: Any) -> Any` returns $value - \Phi(value)$.
- `picard(iterations: int, *, initial: Any | None = None) -> Any` iterates from the zero-like seed unless an initial value is supplied.
- `tree_value(tree: OrderedRootedTree) -> Any` evaluates one tree recursively.
- `tree_expansion(depth: int, *, max_trees: int = 100000, retain_terms: bool = True) -> TreeExpansionResult` sums all trees of height less than `depth` and compares the sum with the matching Picard iterate.
- `convergence_certificate(operation_norms, radius, *, seed_norm=None) -> MajorantConvergence` delegates to the scalar majorant in Section 39.3.

`depth=0` returns the zero-like seed and compares it to the zeroth Picard iterate. For positive depth, the enumerated family has maximum height `depth - 1`. `retain_terms=False` avoids storing the individual result records, but every tree is still enumerated and evaluated.

The returned frozen, slotted records are:

- `TreeTerm(tree: OrderedRootedTree, value: Any)` for one contribution.
- `TreeExpansionResult(value, picard_value, terms, depth, certificate)` for the finite comparison.

The certificate residual is

$$r_{\text{tree}} = \left\| \sum_{h(\tau)<m} \operatorname{val}(\tau) - u_m \right\|_2 .$$

With `Exactness.EXACT`, validity requires `r_tree == 0.0`. Otherwise it requires `r_tree <= tolerance`. Exactness is caller-declared rather than inferred from the value types. The certificate checks enumeration and recursive evaluation, and explicitly leaves convergence beyond the finite depth unchecked. **E/N**

Work and storage model

Let $T_m = N_{m-1}$ for $m > 0$. The expansion stores T_m tree objects and, when `retain_terms=True`, T_m result records. Each call to `tree_value` recursively evaluates its own tree. Common subtrees across different terms are not memoized at the value level. Consequently a useful structural work model is

$$W_m = \sum_{h(\tau)<m} V(\tau)c(\tau),$$

where $V(\tau)$ is the vertex count and $c(\tau)$ represents the cost of the corresponding multilinear operations. The Picard comparison adds m applications of the polynomial map. This is a model of calls, not a wall-clock guarantee. **M**

39.3 Majorant convergence certificates

Suppose callers have established bounds

$$\|s\| \le a, \qquad \|B_r(x_1, \dots, x_r)\| \le b_r \prod_{j=1}^{r} \|x_j\|.$$

Define the scalar majorant

$$\phi(t) = a + \sum_r b_r t^r.$$

On a declared ball of radius $R > 0$, the runtime checks

$$\phi(R) \le R, \qquad q = \sum_r r b_r R^{r-1} < 1.$$

When the actual operation bounds are valid and the ambient space is complete, the contraction argument supplies the tail estimate

$$\|u - u_m\| \le \frac{q^m a}{1-q}.$$

The conditions and scalar arithmetic are finite numerical checks. Applying them to the nonlinear problem is conditional because the implementation does not measure the multilinear operator norms or prove completeness. **N/C**

```
majorant_convergence(
    seed_norm: float,
    operation_norms: Mapping[int, float],
    radius: float,
    *,
    tolerance: float = 1e-12,
) -> MajorantConvergence
```

All norms and bounds must be finite and nonnegative, the radius finite and positive, every operation label an integer arity at least two, and the operation map nonempty. The tolerance must be finite and nonnegative.

`MajorantConvergence` stores:

- `seed_norm: float` and the normalized `operation_norms: dict[int, float]`.
- `radius: float`.
- `image_bound: float`, equal to $\phi(R)$.
- `contraction: float`, equal to q.
- `certificate: Certificate`.
- `scalar_map(value: float) -> float`.
- `scalar_iterates(iterations: int) -> tuple[float, ...]`, beginning with $t_0 = 0$.
- `picard_tail_bound(iterate: int) -> float`, which first requires the certificate and then evaluates the displayed bound.

The certificate accepts the ball condition with additive tolerance, `image_bound <= radius + tolerance`, but it requires the contraction inequality strictly, `contraction < 1.0`. Thus $q = 1$ is invalid even though the stored `contraction_excess` residual is zero. This is the correct strict boundary for the contraction theorem.

Quadratic Catalan majorant

For

$$u = s + B(u, u), \qquad \|B(x, y)\| \le b\|x\|\|y\|,$$

the small scalar fixed point is

$$R = \frac{1 - \sqrt{1 - 4ab}}{2b}, \qquad 4ab < 1.$$

```
quadratic_catalan_majorant(
    seed_norm: float,
    bilinear_norm: float,
    *,
    tolerance: float = 1e-12,
) -> MajorantConvergence
```

When $a, b > 0$ and $1 - 4ab \le 0$, the function returns a result with candidate radius $1/(2b)$ and an explicitly invalid certificate whose theorem ID is `PSC-III-C15`. It does not raise merely because the strict Catalan condition fails. The cases $a = 0$ or $b = 0$ are handled separately and can produce a valid contraction record.

```
import numpy as np

from komorebi_math.nonlinear import (
    PolynomialFixedPointProblem,
    majorant_convergence,
    quadratic_catalan_majorant,
)

problem = PolynomialFixedPointProblem(
    np.array([0.1]),
```

```
    {2: lambda values: np.array([values[0][0] * values[1][0]])},
)
expansion = problem.tree_expansion(4)
majorant = majorant_convergence(0.1, {2: 1.0}, 0.2)

print(expansion.value.tolist(), expansion.picard_value.tolist())
print(expansion.certificate.valid)
print(majorant.image_bound, majorant.contraction)
print(majorant.picard_tail_bound(4))
print(quadratic_catalan_majorant(1.0, 1.0).certificate.valid)
```

Observed output:

```
[0.11256641000000002] [0.11256641]
True
0.14 0.4
0.004266666666666668
False
```

The shipped `examples/nonlinear_trees_elimination.py` begins with the same tree sum and tail bound before continuing into the correction and elimination APIs documented in Chapter 41.

39.4 Finite abelian gradings

Let Λ be a finite abelian group and let a packet system decompose

$$X = \bigoplus_{g \in \Lambda} X_g.$$

An r-linear operation is grade respecting when

$$\ell_r(X_{g_1}, \dots, X_{g_r}) \subseteq X_{g_1 + \cdots + g_r}.$$

For packet components x_g, the output packet has the finite convolution form

$$(\ell_r \star x)_g = \sum_{g_1 + \cdots + g_r = g} \ell_r(x_{g_1}, \dots, x_{g_r}).$$

The package represents both the group and the packet decomposition explicitly. It does not encode a group label in an array position without checking that the two ordered label sets agree.

Explicit group tables

```
FiniteAbelianGroup(
    labels: Sequence[str],
    addition: Mapping[tuple[str, str], str],
    *,
    zero: str,
    validate: bool = True,
)
```

Labels are stringified, nonempty, and unique. The identity label must be in the label set. The addition mapping must contain every ordered label pair exactly once and every result must remain in the label set. With the default `validate=True`, construction calls an exhaustive certificate that checks:

- left and right identity for every label.
- Existence of a two-sided inverse for every label.
- Commutativity for every ordered pair.
- Associativity for every ordered triple.

The check is exact. For group order q, the table occupies q^2 entries and the exhaustive validation performs $O(q^3)$ additions. **E/M**

The cyclic constructor has the signature

```
cyclic(
    order: int,
    *,
    prefix: str = "",
) -> FiniteAbelianGroup
```

It creates labels `prefix0` through `prefix(order - 1)` and addition modulo the positive integer `order`. The remaining public methods are:

- `add(left: str, right: str) -> str`.
- `sum(labels: Sequence[str]) -> str`, beginning at the declared identity.
- `inverse(label: str) -> str`, found by finite search.
- `certificate() -> Certificate`.
- `to_dict() -> dict[str, Any]`, whose addition keys have the form `g+h`.

`validate=False` permits construction from a closed but otherwise unchecked table. Such an object should not be called a group until `certificate()` has been inspected. `add` raises `ValidationError` for a label outside the table.

Graded operations and finite basis probes

The module-visible callable aliases are:

```
GradedOperationCallable = Callable[[tuple[Any, ...]], Any]
LinearOperation = Callable[[Any], Any]
BracketOperation = Callable[[Any, Any], Any]
```

They are typing conventions in `komorebi_math.nonlinear.graded`, not additional runtime classes and not entries in that module's `__all__` list.

```
GradedPacketOperation(
    group: FiniteAbelianGroup,
    packets: PacketSystem,
    arity: int,
    operation: GradedOperationCallable,
    *,
    exactness: Exactness = Exactness.NUMERICAL,
    tolerance: float | None = None,
    name: str = "graded packet operation",
)
```

The arity is a positive integer. `packets.names` must equal `group.labels` in the same order, and the operation must be callable. `evaluate(arguments)` requires exactly the declared number of vectors and checks every input and the output against the packet dimension.

```
grading_certificate(
    *,
```

```
    max_evaluations: int = 100000,
) -> GradingCertificate
```

This method constructs a basis for every packet range. An exact SymPy backend uses each projector's exact column space. A numerical backend uses left singular vectors whose singular values exceed a tolerance-scaled threshold. For every entered input-label tuple $(g_1, \dots, g_r)$, products of range-basis vectors are evaluated until the budget is exhausted, and every output packet other than $g_1 + \cdots + g_r$ is projected away. The recorded residual is

$$\rho_{\mathrm{grade}} = \max_{\substack{(g_1,\dots,g_r) \\ h \neq g_1+\cdots+g_r}} \|P_h \ell_r(e_1, \dots, e_r)\| .$$

`GradingCertificate` stores:

- `arity: int`.
- `tested_input_labels: int`.
- `forbidden_residual: float`.
- `certificate: Certificate`.

When the finite basis scan completes, this is an exhaustive finite-dimensional multilinearity test provided the callable really is multilinear. If `max_evaluations` stops the scan, `certificate.exactness` becomes `CONDITIONAL` and `complete=False` appears in diagnostics. `tested_input_labels` counts tuples whose scan was entered. The final counted tuple can be only partially evaluated. The record does not identify tuple values individually and reports the remaining work only through a generic unchecked-assumption string. An incomplete valid prefix is not a global grading proof. **E/N/C**

For packet-range dimensions d_g, a complete arity-r scan evaluates

$$N_{\mathrm{basis}} = \sum_{(g_1,\dots,g_r)\in\Lambda^r} \prod_{j=1}^{r} d_{g_j} = \left(\sum_g d_g\right)^r$$

operation calls, with projections onto up to $q-1$ forbidden output packets per call. `max_evaluations` caps operation calls, not projection work. **M**

Packet convolution record

```
GradedPacketOperation.packet_convolution(
    arguments: Sequence[Any],
    output_label: str,
) -> GradedConvolutionResult
```

The method analyzes every full argument into all packet components, enumerates all q^r input-label tuples, retains exactly those whose sum is the requested output label, evaluates and projects those terms, then compares their sum with the same output packet of a direct full-vector evaluation.

For a group, each choice of the first $r-1$ labels determines the last, so there are exactly q^{r-1} retained label terms. The code still enumerates all q^r tuples. The frozen records are:

- `GradedConvolutionTerm(input_labels, value)`.
- `GradedConvolutionResult(output_label, value, direct_value, terms, certificate)`.

The reconstruction residual is

$$\rho_g = \left\| P_g \ell_r(x_1, \dots, x_r) - \sum_{g_1+\cdots+g_r=g} P_g \ell_r(P_{g_1} x_1, \dots, P_{g_r} x_r) \right\| .$$

The certificate checks the finite packet identity at the supplied arguments. It explicitly leaves global grade preservation unchecked. With an exact backend and caller-declared exactness, equality must be exact. Otherwise the declared tolerance is used. **E/N**

39.5 Maurer-Cartan and divided-square packets

For a finite grading and a packet-decomposed element

$$\eta = \sum_{g \in \Lambda} \eta_g,$$

the ordinary finite Maurer-Cartan expression is

$$\mathrm{MC}(\eta) = d\eta + \tfrac{1}{2}[\eta, \eta].$$

Its output packet is assembled as

$$\mathrm{MC}_g = d\eta_g + \tfrac{1}{2} \sum_{a+b=g} [\eta_a, \eta_b].$$

```
maurer_cartan_packets(
    group: FiniteAbelianGroup,
    components: Mapping[str, Any],
    differential: LinearOperation,
    bracket: BracketOperation,
    *,
    exactness: Exactness = Exactness.NUMERICAL,
    tolerance: float = 1e-10,
) -> MaurerCartanResult
```

`components` must contain every group label exactly once. The routine forms each packet equation, sums those equations, independently forms `differential(eta) + bracket(eta, eta)/2`, and compares them. For `Exactness.EXACT`, division by two uses `fractions.Fraction(1, 2)`. For other exactness values it uses the floating scalar `0.5`.

The record `MaurerCartanResult(equations, reconstructed, direct, certificate)` contains both sides. Its certificate checks finite label convolution at the supplied components and leaves linearity of d, bilinearity and grading of the bracket, and any broader differential graded Lie-algebra axioms unchecked. It does not test $d^2 = 0$, graded antisymmetry, or a Jacobi identity. **E/N/C**

Higher finite truncations

For a finite family of declared operations, the implemented expression is

$$\mathrm{MC}^{(R)}(\eta) = \sum_{r=1}^{R} \frac{1}{r!} \ell_r(\eta, \dots, \eta).$$

```
higher_maurer_cartan_packets(
    group: FiniteAbelianGroup,
    components: Mapping[str, Any],
    operations: Mapping[int, GradedOperationCallable],
    *,
    exactness: Exactness = Exactness.NUMERICAL,
    tolerance: float = 1e-10,
) -> HigherMaurerCartanResult
```

Operation labels must be positive integer orders and the mapping must be nonempty. The direct side performs one operation call per declared order. The packet side enumerates q^r ordered label tuples for order r, scales each term by $1/r!$, and places it in the packet indexed by the label sum. The result stores `equations`, `reconstructed`, `direct`, `max_order`, and `certificate`.

This is a finite truncation. It neither supplies nor checks an infinite tail. The certificate lists multilinearity, grading, and convergence of any omitted series as unchecked assumptions. **E/N/C**

Characteristic-sensitive divided squares

When division by two is not appropriate, the package exposes a separate finite polarization path. The caller supplies a map q intended to satisfy

$$q(x+y) = q(x) + q(y) + [x, y].$$

The equation is

$$d\eta + q(\eta).$$

For output grade g, the implementation adds:

$$d\eta_g, \qquad q(\eta_a) \quad \text{when } a + a = g, \qquad [\eta_a, \eta_b] \quad \text{when } a < b \text{ and } a + b = g.$$

The label order used by the final sum is the declared `group.labels` order. Each unordered cross-label pair therefore appears once.

```
divided_square_packets(
    group: FiniteAbelianGroup,
    components: Mapping[str, Any],
    differential: LinearOperation,
    bracket: BracketOperation,
    divided_square: LinearOperation,
    *,
    exactness: Exactness = Exactness.NUMERICAL,
    tolerance: float = 1e-10,
) -> DividedSquareResult
```

`DividedSquareResult` stores `equations`, `reconstructed`, `direct`, and the certificate. The package does not verify the displayed polarization identity or grading compatibility. A successful reconstruction is evidence only at the supplied finite components. **E/N/C**

Exact finite grading example

```
from fractions import Fraction

from komorebi_math import Exactness
from komorebi_math.nonlinear import (
    FiniteAbelianGroup,
    divided_square_packets,
    higher_maurer_cartan_packets,
    maurer_cartan_packets,
)

group = FiniteAbelianGroup.cyclic(3)
components = {"0": Fraction(1), "1": Fraction(2), "2": Fraction(3)}

mc = maurer_cartan_packets(
    group,
    components,
    lambda x: 2 * x,
    lambda x, y: x * y,
    exactness=Exactness.EXACT,
)
higher = higher_maurer_cartan_packets(
    group,
    components,
    {
        1: lambda values: values[0],
```

```
        2: lambda values: values[0] * values[1],
    },
    exactness=Exactness.EXACT,
)
square = divided_square_packets(
    group,
    components,
    lambda x: 3 * x,
    lambda x, y: 2 * x * y,
    lambda x: x * x,
    exactness=Exactness.EXACT,
)

print(group.add("2", "2"), group.inverse("1"), group.certificate().valid)
print(mc.equations, mc.reconstructed, mc.direct, mc.certificate.valid)
print(higher.max_order, higher.reconstructed, higher.direct, higher.certificate.valid)
print(square.reconstructed, square.direct, square.certificate.valid)
```

Observed output:

```
1 2 True
{'0': Fraction(17, 2), '1': Fraction(21, 2), '2': Fraction(11, 1)} 30 30 True
2 24 24 True
54 54 True
```

This example checks exact reconstruction. It is not a claim that the chosen scalar product is a Lie bracket. The distinction illustrates why the result records separate finite reconstruction from unchecked algebraic structure.

Packet-tree reconstruction

The bridge between the tree and grading viewpoints inserts a complete packet identity on every edge of one evaluated tree:

$$\operatorname{val}(\tau) = \sum_{\text{packet labelings } \ell} \operatorname{val}(\tau, \ell).$$

```
packet_tree_reconstruction(
    problem: PolynomialFixedPointProblem,
    tree: OrderedRootedTree,
    packets: PacketSystem,
    *,
    zero_support: Mapping[int, PacketJetSupport] | None = None,
    tolerance: float | None = None,
) -> PacketTreeResult
```

At a leaf, the seed is projected into every packet. At an internal vertex of arity r, the routine recursively obtains packet values for each child, enumerates every input label tuple and every output label, evaluates B_r, and projects the result. A `PacketJetAddress` listed in the supplied `PacketJetSupport.zero` set is skipped. No `unknown` or `untested` address is used for pruning.

`PacketTreeResult` stores `value`, the final `packet_values`, `evaluated_blocks`, `pruned_blocks`, and `certificate`. The certificate compares the synthesized packet value with `problem.tree_value(tree)`. Exactness is `EXACT` only when both the problem declares exactness and the packet backend is exact. It records global correctness of the declared multilinear operations as unchecked. The routine also trusts that any supplied zero-support record is appropriate to the same operation and packet system. It does not verify that provenance relationship itself.

For p packets and an internal vertex of arity r, an unpruned local evaluation performs p^{r+1} projected operation calls. The total depends on the arity profile of the tree. `evaluated_blocks` and `pruned_blocks` provide the realized finite counts. **E/N/M**

40. Defects, obstruction, lifting, and holonomy

One finite diagnostic pattern appears in four settings:

1. Compare a fine nonlinear map with its coarse counterpart.
2. Project a linear correction defect to a cokernel.
3. Solve the residual equation one prime-power layer at a time.
4. Transport local packet fibers around finite graph cycles.

In every setting, a certificate says what was computed and compared. A valid certificate need not say that the computed defect is zero. Solvability is carried by a separate field such as `vanishes`, `lifts`, or the validity of a specific gluing certificate.

The complete records live in `komorebi_math.nonlinear`. The most common constructors are also re-exported from `komorebi_math`:

```
from komorebi_math import (
    LinearLiftProblem,
    LinearObstructionProblem,
    NonlinearPacketMap,
    NonlinearRefinementDefect,
    PolynomialLiftProblem,
    TransportEdge,
    TransportGraph,
    build_lift_tree,
    solve_mod_prime,
)
```

Use the research namespace when type annotations need `ObstructionResult`, `LiftStepResult`, `ModularLinearSolution`, `FundamentalHolonomy`, or `IntertwiningResult`.

40.1 Nonlinear packet maps and refinement defects

This section separates packet-coordinate conjugation, whose certificate checks a reconstructed finite evaluation, from refinement comparison, whose computed defect can be valid evidence without vanishing.

Packet coordinate conjugation

Let source and target packet systems provide analysis and synthesis maps A_X, S_X, A_Y, S_Y. A nonlinear map $F : X \to Y$ has packet-coordinate form

$$\widehat{F} = A_Y \circ F \circ S_X.$$

`NonlinearPacketMap` evaluates precisely this finite conjugation:

```
NonlinearPacketMap(
    function: Callable[[Any], Any],
    source: PacketSystem,
    target: PacketSystem,
    name: str = "nonlinear packet map",
)
```

Source and target must use the same backend name. The method signatures are:

```
__call__(
    coordinates: PacketCoordinates,
) -> PacketCoordinates

apply(vector: Any) -> Any

certificate(vector: Any) -> Certificate

iterate(
    coordinates: PacketCoordinates,
    count: int,
) -> PacketCoordinates

fixed_point_residual(
    coordinates: PacketCoordinates,
) -> ComputationResult[Any]
```

`__call__` reconstructs source coordinates, evaluates the function, validates the target dimension, and analyzes the output. `apply` performs the ambient function evaluation after source and target dimension checks. `certificate` compares direct ambient evaluation with packet-coordinate evaluation. `iterate` and `fixed_point_residual` require equivalent source and target systems.

The coordinate-conjugacy residual is

$$\rho_{\mathrm{coord}} = \left\| F(x) - S_Y \widehat{F}(A_X x) \right\|.$$

It is exact for an exact backend and numerical otherwise, using the larger of the source and target packet tolerances. **E/N**

The fixed-point method returns

$$r = \widehat{F}(\hat{x}) - \hat{x}$$

after reconstruction to ambient coordinates. Its `certificate.valid` is always `True` once the residual has been computed. The norm is stored under `fixed_point_residual_norm`. Validity does not assert $r = 0$ and does not assert that the input is a fixed point.

Refinement defect

For fine and coarse maps

$$F_f : X_f \to Y_f, \qquad F_c : X_c \to Y_c,$$

with linear coarsening maps $C_X : X_f \to X_c$ and $C_Y : Y_f \to Y_c$, the pointwise defect is

$$\delta_{C_X, C_Y}(F)(x) = C_Y F_f(x) - F_c(C_X x).$$

```
NonlinearRefinementDefect(
    fine_map: Callable[[Any], Any],
    coarse_map: Callable[[Any], Any],
    coarsen_source: Any,
    coarsen_target: Any,
)
```

A coarsening map may be callable or matrix-like. The implementation converts the final difference to a NumPy array.

- `evaluate(point: Any) -> np.ndarray` returns the defect vector.

- `norm(point: Any) -> float` returns its Euclidean norm.
- `composition_identity(...) -> ComputationResult[np.ndarray]` evaluates the finite composition law and compares it with an independently evaluated composite defect.
- `iterated_orbit_bound(...) -> float` computes the declared orbit estimate.
- `fixed_point_bound(...) -> float` computes the declared fixed-point estimate.

Their complete signatures are:

```
composition_identity(
    outer_fine: Callable[[Any], Any],
    outer_coarse: Callable[[Any], Any],
    outer_coarsen_target: Any,
    point: Any,
) -> ComputationResult[np.ndarray]

iterated_orbit_bound(
    point: Any,
    *,
    iterations: int,
    coarse_lipschitz: float,
) -> float

fixed_point_bound(
    fine_fixed_point: Any,
    *,
    coarse_contraction: float,
    verify_fixed_point: bool = True,
    tolerance: float = 1e-10,
) -> float
```

For the composition $G \circ F$, the code evaluates

$$\delta(G \circ F)(x) = \delta(G)(F_f(x)) + G_c(F_c(C_X x) + \delta(F)(x)) - G_c(F_c(C_X x))$$

and compares it pointwise with the direct composite defect. The returned certificate is always numerical with tolerance 10^{-10}. This is a finite identity at one point, not a regularity theorem. **N**

For the implemented orbit and fixed-point bounds, take endomaps with a common coarsening $C_X = C_Y =: C$, and write $\delta_C = \delta_{C,C}$. If F_c is L-Lipschitz and

$$\varepsilon_j = \|\delta_C(F)(F_f^j(x))\|,$$

then

$$\|CF_f^n(x) - F_c^n(Cx)\| \le \sum_{j=0}^{n-1} L^{n-1-j}\varepsilon_j.$$

`iterated_orbit_bound` returns exactly the right-hand finite sum. It rejects a negative iteration count or negative declared Lipschitz constant, but it does not test the Lipschitz inequality. **C**

If x_f is a fine fixed point and F_c contracts by $q < 1$, then for a coarse fixed point x_c,

$$\|Cx_f - x_c\| \le \frac{\|\delta_C(F)(x_f)\|}{1-q}.$$

`fixed_point_bound` requires $0 \le q < 1$. By default it checks $\|F_f(x_f) - x_f\| \le$ `tolerance` and raises `ValidationError` when that finite fixed-point test fails. Setting `verify_fixed_point=False` skips that check and makes the conclusion explicitly dependent on the caller's claim. It does not find x_c, test global contraction, or prove uniqueness. **C**

```
import numpy as np

from komorebi_math.nonlinear import NonlinearRefinementDefect

defect = NonlinearRefinementDefect(
    fine_map=lambda x: np.asarray(x),
    coarse_map=lambda y: 0.5 * np.asarray(y),
    coarsen_source=np.array([[1.0, 1.0]]),
    coarsen_target=np.array([[1.0, 1.0]]),
)
point = np.array([2.0, 4.0])

print(defect.evaluate(point).tolist())
print(defect.fixed_point_bound(point, coarse_contraction=0.5))
```

Observed output:

```
[3.0]
6.0
```

40.2 Linear obstruction problems

Given a finite linear correction map $D : X \to Y$ and defect $\delta \in Y$, the correction problem is

$$Du = -\delta.$$

Its obstruction class is the image of δ in the cokernel:

$$\mathrm{ob}(\delta) = \pi_{\operatorname{coker} D}(\delta).$$

Finite-dimensional linear algebra gives

$$\mathrm{ob}(\delta) = 0 \iff \delta \in \operatorname{im} D, \qquad u = u_0 + \ker D.$$

```
LinearObstructionProblem(
    correction_map: Any,
    defect: Any,
    *,
    backend: str | LinearAlgebraBackend | None = None,
    tolerance: float = 1e-10,
)
```

The correction map is converted to a backend matrix, the defect to a backend vector, and their codomain dimensions must agree. `solve() -> ObstructionResult` dispatches to an exact SymPy path when the backend name is `"sympy"`. Other backends use the numerical least-squares path.

`ObstructionResult` is a frozen, slotted record with:

- `vanishes: bool` for the actual finite solvability decision.
- `obstruction_coordinates: tuple[Any, ...]` in a computed left-nullspace basis.
- `cokernel_dimension: int`.
- `correction: Any | None`, present only when `vanishes` is true.
- `kernel_basis: tuple[Any, ...]`.
- `residual: float`.
- `certificate: Certificate`.
- `require_correction() -> Any`, which requires the certificate and then rejects a nonzero obstruction.

Exact SymPy branch

The exact branch compares

$$\operatorname{rank} D \quad \text{and} \quad \operatorname{rank}[D \mid -\delta].$$

It computes a basis z_j for $\ker D^{\mathsf{T}}$ and stores

$$c_j = z_j^{\mathsf{T}} \delta.$$

When solvable, SymPy's parametric Gauss-Jordan result is specialized by setting all free parameters to zero, yielding one correction. The separately stored kernel basis describes the remaining affine freedom. The exact certificate requires equivalence between zero obstruction coordinates and the rank decision, plus zero correction residual. **E**

The obstruction coordinates depend on the basis returned by the backend. Their individual values are not canonical invariants, while the vanishing of all coordinates is.

Numerical branch

The numerical branch computes a least-squares correction, sets

$$r = \|Du + \delta\|_2, \qquad \texttt{vanishes} \iff r \le \tau,$$

and obtains numerical kernel and left-nullspace bases through the selected backend. It records

$$c_{\max} = \max_j |\langle z_j, \delta \rangle|.$$

The certificate verifies that `c_max <= 10 * tolerance` agrees with the least-squares vanishing decision. It can therefore be invalid when the two numerical rank diagnostics disagree near a threshold. A successful certificate records consistency of the finite numerical diagnostics. It does not prove rank stability under perturbation. **N**

Obstruction squares

A correction square consists of horizontal maps $D_s : X_s \to Y_s$ and $D_t : X_t \to Y_t$, together with vertical maps $C_X : X_s \to X_t$ and $C_Y : Y_s \to Y_t$.

It induces a map of obstruction classes only when

$$C_Y D_s = D_t C_X.$$

```
ObstructionSquare(
    source_correction: Any,
    target_correction: Any,
    source_map: Any,
    target_map: Any,
    backend: str | LinearAlgebraBackend | None = None,
    tolerance: float = 1e-10,
)
```

This is a frozen, slotted record. `certificate()` checks all four matrix dimensions and computes

$$\rho_{\square} = \|C_Y D_s - D_t C_X\|.$$

`transport_defect(defect)` first requires that certificate, then returns $C_Y \delta$. An invalid square therefore cannot silently transport a defect. The exact backend requires zero residual. A numerical backend uses the stored tolerance. **E/N**

Exact obstruction example

```
import sympy as sp

from komorebi_math.nonlinear import LinearObstructionProblem

obstructed = LinearObstructionProblem(
    sp.Matrix([[1, 0], [0, 0]]),
    sp.Matrix([0, 1]),
    backend="sympy",
).solve()
solvable = LinearObstructionProblem(
    sp.Matrix([[1, 0], [0, 0]]),
    sp.Matrix([1, 0]),
    backend="sympy",
).solve()

print(obstructed.vanishes, obstructed.obstruction_coordinates)
print(solvable.vanishes, tuple(solvable.require_correction()))
```

Observed output:

```
False (1,)
True (-1, 0)
```

40.3 Linear and polynomial lift problems

Prime-power lifting replaces one large congruence by a sequence of exact correction problems over the residual field $\mathbb{F}_p$. The package requires `prime` to be mathematically prime. Composite moduli are rejected rather than passed to row reduction with noninvertible pivots.

Linear systems over a prime field

```
solve_mod_prime(
    matrix: Any,
    rhs: Any,
    prime: int,
) -> ModularLinearSolution
```

The inputs are converted to exact integral object arrays and reduced modulo p. Nonintegral entries are rejected. Exact Gauss-Jordan elimination then describes

$$Au = b \pmod{p}.$$

When it is solvable, the output has affine form

$$u = u_0 + \sum_{j=1}^{\nu} c_j k_j, \qquad c_j \in \mathbb{F}_p, \qquad N_{\mathrm{sol}} = p^{\nu}.$$

The frozen record

```
ModularLinearSolution(
    prime: int,
    solvable: bool,
    particular: tuple[int, ...] | None,
    kernel_basis: tuple[tuple[int, ...], ...],
    pivot_columns: tuple[int, ...],
```

```
    obstruction_rows: tuple[int, ...],
)
```

also exposes `nullity -> int`, `solution_count -> int`, and `enumerate(*, limit: int = 100000) -> tuple[tuple[int, ...], ...]`. An inconsistent reduced row is recorded in `obstruction_rows`. In that case `particular` is `None`, the kernel record is empty, `solution_count` is zero, and enumeration returns an empty tuple.

For a solvable system, enumeration rejects a solution count above the explicit limit, then traverses all coefficient tuples in lexicographic product order. The row reduction cost is the usual finite dense elimination cost, while full enumeration has unavoidable output size p^{ν}. **E/M**

One linear lift step

Suppose

$$Ax_k = b \pmod{p^k}.$$

Every child modulo p^{k+1} has the form

$$x_{k+1} = x_k + p^k u,$$

and the divided residual is

$$r_k = \frac{Ax_k - b}{p^k} \pmod{p}.$$

Substitution gives the residual-field equation

$$Au = -r_k \pmod{p}.$$

```
LinearLiftProblem(
    matrix: Any,
    rhs: Any,
    point: Any,
    *,
    prime: int,
    depth: int,
)

.step(*, enumerate_limit: int = 100000) -> LiftStepResult
```

Dimensions must agree, the prime must pass the primality guard, and the depth must be positive. `step` first verifies the current congruence modulo p^k. It solves the correction equation, enumerates its affine solution space, constructs all children in canonical residues modulo p^{k+1}, and directly verifies every child.

`LiftStepResult` stores:

- `prime`, `depth`, and the current `point`.
- `residual`, the divided residual modulo p.
- `correction_space: ModularLinearSolution`.
- `children: tuple[tuple[int, ...], ...]`.
- `certificate: Certificate`.
- `lifts -> bool`, an alias for `correction_space.solvable`.

An obstructed step can have no children and still have a valid certificate: the exact certificate says that the complete empty lift space was computed correctly. `lifts`, not certificate validity alone, answers the existence question. **E**

Polynomial one-step law

For an integer polynomial-like callable F and a supplied Jacobian J_F, assume

$$F(x_k) = 0 \pmod{p^k}.$$

The first-order congruence is

$$F(x_k + p^k u) \equiv F(x_k) + p^k J_F(x_k) u \pmod{p^{k+1}}.$$

Therefore the correction equation is

$$J_F(x_k) u = -\frac{F(x_k)}{p^k} \pmod{p}.$$

```
PolynomialLiftProblem(
    function: Callable[[tuple[int, ...]], Iterable[int]],
    jacobian: Callable[[tuple[int, ...]], Any],
    point: Any,
    *,
    prime: int,
    depth: int,
)

.step(
    *,
    enumerate_limit: int = 100000,
    linearization_limit: int = 100000,
    assume_valid_jacobian: bool = False,
) -> LiftStepResult
```

The current function value must be divisible by p^k, and the Jacobian shape must be `number_of_equations x number_of_variables`. Let d be the number of variables. By default the method exhaustively checks the supplied first-order law for all p^d residual corrections. For every $u \in \{0, \dots, p-1\}^d$, it verifies both congruence preservation through depth k and

$$\frac{F(x_k + p^k u) - F(x_k)}{p^k} \equiv J_F(x_k) u \pmod{p}.$$

This exhaustive finite check is exact. A wrong Jacobian or a callable that fails the required congruence law raises `ValidationError` before the lift space is returned. **E**

When $p^d >$ `linearization_limit`, the default path raises `ValueError` and reports the required correction count. Setting `assume_valid_jacobian=True` allows the step to continue. Every computed child is still checked directly modulo p^{k+1}, but the result exactness becomes `CONDITIONAL` and the full Jacobian law appears in `assumptions_unchecked`. This flag is a visible trust boundary, not a faster exact proof. **C**

Lift behavior example

```
import numpy as np

from komorebi_math.nonlinear import PolynomialLiftProblem, solve_mod_prime

affine = solve_mod_prime(
    [[1, 1, 0], [0, 1, 1]],
    [1, 0],
    3,
)
unique = PolynomialLiftProblem(
```

```
    lambda x: (x[0] * x[0] - 2,),
    lambda x: [[2 * x[0]]],
    [3],
    prime=7,
    depth=1,
).step()
branching = PolynomialLiftProblem(
    lambda x: (x[0] * x[0],),
    lambda x: [[2 * x[0]]],
    [0],
    prime=2,
    depth=1,
).step()
conditional = PolynomialLiftProblem(
    lambda x: tuple(x),
    lambda x: np.eye(len(x), dtype=int),
    [0] * 8,
    prime=3,
    depth=1,
).step(
    linearization_limit=100,
    assume_valid_jacobian=True,
)

print(affine.solution_count, affine.enumerate())
print(unique.children, branching.children)
print(
    conditional.certificate.exactness.value,
    conditional.certificate.diagnostics["residual_correction_count"],
)
```

Observed output:

```
3 ((1, 0, 0), (2, 2, 1), (0, 1, 2))
((10,),) ((0,), (2,))
conditional 6561
```

The first polynomial has one lift from $3 \pmod 7$ to $10 \pmod{49}$. The singular binary example branches. The eight-variable identity example shows the explicit conditional boundary without pretending that 6,561 Jacobian tests were executed.

40.4 Prime-power lift trees

Repeated one-step lifting produces a finite rooted forest whose level k contains congruence classes modulo p^k. The data structures are intentionally finite:

```
LiftTreeNode(
    value: tuple[int, ...],
    depth: int,
    parent_index: int | None,
)

FiniteLiftTree(
    prime: int,
    levels: tuple[tuple[LiftTreeNode, ...], ...],
)
```

Every stored node depth must be positive and the tree prime must pass the primality guard. `FiniteLiftTree.max_depth -> int` returns the depth of the first node in the final stored level when that level is nonempty, and zero when

the final stored level is empty. `terminal_branches() -> tuple[tuple[LiftTreeNode, ...], ...]` follows the parent indices of every node in the last level back to a root.

```
build_lift_tree(
    problem_factory: Callable[
        [tuple[int, ...], int],
        LinearLiftProblem | PolynomialLiftProblem,
    ],
    roots: Iterable[tuple[int, ...]],
    *,
    prime: int,
    start_depth: int,
    max_depth: int,
    max_nodes: int = 100000,
) -> FiniteLiftTree
```

The start depth must be positive, the maximum depth cannot precede it, and the node limit must be positive. For each current node the factory receives the node value and depth. Its returned problem must use the same prime. The builder requires the one-step certificate, appends every child with its parent index, and stops early when a level is empty. It raises before returning a forest whose cumulative node count exceeds `max_nodes`.

If a node has correction nullity ν, it can produce p^{ν} children. For level sizes n_k, storage is

$$N_{\text{nodes}} = \sum_{k} n_k,$$

and the exact work includes one residual-field solve plus explicit child verification for every expanded node. `max_nodes` caps stored nodes and is also passed as the per-step correction enumeration limit. **E/M**

The finite conclusion is intentionally narrow:

> A nonempty tree through depth K proves that the displayed residue classes lift through p^K. It does not prove that any branch continues through all depths, converges to a p-adic solution, or is unique beyond the finite tree.

```
from komorebi_math.nonlinear import (
    PolynomialLiftProblem,
    build_lift_tree,
)

F = lambda x: (x[0] * x[0] - 1,)
J = lambda x: [[2 * x[0]]]

tree = build_lift_tree(
    lambda point, depth: PolynomialLiftProblem(
        F, J, point, prime=3, depth=depth
    ),
    [(1,), (2,)],
    prime=3,
    start_depth=1,
    max_depth=4,
)

print(tuple(len(level) for level in tree.levels))
print(len(tree.terminal_branches()))
```

Observed output:

```
(2, 2, 2, 2)
2
```

The shipped `examples/obstruction_lifting.py` extends this calculation through depth five and prints:

```
obstruction coordinates: (1,)
lift levels: 5
terminal branches: 2
```

40.5 Transport graphs and finite holonomy

A `TransportGraph` attaches one finite-dimensional model fiber to every vertex of a connected finite graph. An oriented edge $e : v \to w$ carries an invertible matrix

$$T_e : X_v \to X_w.$$

The graph uses edge orientations for matrix transport but treats each edge as traversable in both directions while finding a spanning tree. Reverse traversal uses T_e^{-1}.

The primary records are:

```
TransportEdge(
    source: str,
    target: str,
    matrix: Any,
    name: str = "",
)

FundamentalHolonomy(
    edge_index: int,
    source: str,
    target: str,
    matrix: Any,
)

IntertwiningResult(
    intertwiners: Mapping[str, Any],
    edge_residuals: Mapping[str, float],
    cycle_residuals: Mapping[str, float],
    certificate: Certificate,
)
```

`TransportEdge.reversed_name() -> str` returns the explicit edge name when present and otherwise the string `target->source`. Certificate keys do not use that helper. They use `TransportGraph.edge_label(index)`, which returns `name[index]` or `edge[index]`. The index keeps parallel or duplicate-named edges distinct.

```
TransportGraph(
    vertices: Iterable[str],
    edges: Iterable[TransportEdge],
    *,
    backend: str | LinearAlgebraBackend | None = None,
    tolerance: float = 1e-10,
)
```

Vertex names are stringified and deduplicated in first-occurrence order. The set must be nonempty. Every edge endpoint must be known, every transport must be square and invertible in the chosen backend, all edge matrices

must share one dimension, and the underlying undirected multigraph must be connected. A single vertex with no edges is accepted with model dimension one. A graph with multiple vertices and no edges is rejected.

Spanning-tree normalization

Choose a root r. Breadth-first traversal produces a parent relation and a tuple of tree-edge indices. Let

$$S_v : X_r \to X_v$$

be the ordered product of edge transports or inverses along the tree path. Then:

- `spanning_tree(root=None)` returns the parent mapping and tree-edge tuple.
- `tree_transports(root=None) -> dict[str, Any]` returns all S_v.
- `tree_gauge(root=None) -> dict[str, Any]` returns S_v^{-1}, which normalizes every tree transport to the identity.

For vertex gauges G_v, `gauge_transform(gauges)` requires one invertible matrix of the common fiber dimension at every vertex and applies

$$T_e \longmapsto G_w T_e G_v^{-1}.$$

The returned graph is a new validated `TransportGraph`. The original is not mutated. **E/N**

Fundamental cycles and gluing

Every non-tree edge $e : v \to w$ closes one based fundamental cycle. Its holonomy at the root is

$$H_e = S_w^{-1} T_e S_v.$$

```
TransportGraph.fundamental_holonomies(
    root: str | None = None,
) -> tuple[FundamentalHolonomy, ...]
```

For a connected multigraph, the number of returned cycles is

$$\beta_1 = |E| - |V| + 1.$$

Parallel edges and self-loops are retained as distinct edges and therefore can contribute cycles. `gluing_certificate(root=None)` compares every fundamental holonomy with the identity:

$$\rho_e = \|H_e - I\|, \qquad \text{glues} \iff \rho_e = 0 \text{ for all non-tree } e.$$

The equality is exact for an exact backend and tolerance-based otherwise. A tree has no non-tree edges, so the empty residual mapping is valid and $\beta_1 = 0$. For a fixed root and traversal tree, the criterion is finite and complete for this transport graph. **E/N**

Under the displayed gauge action, based holonomy transforms by conjugation:

$$H_e \longmapsto G_r H_e G_r^{-1}.$$

Thus identity holonomy and the gluing decision are gauge invariant, although the matrix entries of a nontrivial holonomy depend on the root gauge.

Intertwining two systems

Suppose T_e and T'_e use the same ordered graph, backend, and fiber dimension. Given a root map $R : X_r \to X'_r$, the tree paths propagate vertex maps

$$R_v = S'_v R S_v^{-1}.$$

Global intertwining requires

$$R_w T_e = T'_e R_v$$

on every edge. On the fundamental cycles it equivalently requires

$$R H_e = H'_e R.$$

```
TransportGraph.intertwine(
    other: TransportGraph,
    root_intertwiner: Any,
    *,
    root: str | None = None,
) -> IntertwiningResult
```

The method checks identical ordered vertices, edge counts and orientations, backend names, and dimensions. It stores both per-edge and per-cycle residuals. The certificate is valid only when both sets pass exact or numerical comparison. A root map need not be invertible. It is an intertwiner, not a gauge. **E/N**

Holonomy example

```
import numpy as np

from komorebi_math import TransportEdge, TransportGraph

A = np.array([[1.0, 1.0], [0.0, 1.0]])
B = np.array([[1.0, 0.0], [1.0, 1.0]])
C = B @ A

graph = TransportGraph(
    ("a", "b", "c"),
    (
        TransportEdge("a", "b", A, "ab"),
        TransportEdge("b", "c", B, "bc"),
        TransportEdge("a", "c", C, "ac"),
    ),
)
gauges = {
    "a": np.eye(2),
    "b": np.diag([2.0, 1.0]),
    "c": np.diag([1.0, 3.0]),
}
transformed = graph.gauge_transform(gauges)

broken = TransportGraph(
    ("a", "b", "c"),
    (
        TransportEdge("a", "b", A, "ab"),
        TransportEdge("b", "c", B, "bc"),
        TransportEdge("a", "c", 2.0 * C, "ac"),
    ),
)
```

```
print(len(graph.fundamental_holonomies()), graph.gluing_certificate().valid)
print(transformed.gluing_certificate().valid)
print(broken.gluing_certificate().residuals)
```

Observed output:

```
1 True
True
{'bc[1]': 0.7071067811865476}
```

The non-tree edge is `bc[1]` for the breadth-first spanning tree selected from the root `a`. `ac` is a tree edge in that traversal. The broken cycle is still detected. A fundamental-cycle label describes the chosen tree presentation, not a claim that the named edge alone caused the inconsistency.

The shipped `examples/holonomy.py` exercises the same gluing, gauge, and broken cycle paths and prints:

```
fundamental cycles: 1
gauge invariant gluing: True
broken cycle detected: True
```

40.6 Failure behavior and certificate reading

The most important negative paths are deliberate and typed:

- `ValidationError`: malformed tree arity, invalid group table, a composite declared prime, mismatched packet labels, wrong source coordinates, a non-fixed fine point, a nonzero obstruction passed to `require_correction`, a noncommuting obstruction square, a current point that is not a solution at its declared depth, a bad Jacobian law, a lift-tree prime mismatch, an edge referencing an unknown graph vertex, a disconnected graph, a multivertex graph with no edges, or an incomplete gauge mapping.
- `DimensionError`: nonlinear output dimension mismatch, obstruction codomain mismatch, correction-square shape mismatch, lift matrix/vector mismatch, Jacobian shape mismatch, nonsquare or inconsistent transport dimensions, or a wrong root-intertwiner shape.
- `ValueError`: nonpositive lift depth, an enumerated affine space above its limit, an unverified polynomial linearization above its limit, an invalid depth interval or node limit, a graph with no vertices, or a noncontractive fixed-point bound.
- `KeyError`: an unknown root passed to the spanning-tree routines.
- Backend linear-algebra failure: singular transport or gauge matrices are rejected when the backend inverse is attempted.
- `Certificate.require()`: an invalid finite tree comparison, majorant, grading test, reconstruction, obstruction consistency check, lift-child verification, gluing condition, or intertwining condition raises through the common certificate mechanism.

Several result fields answer questions that certificate validity does not:

- `MajorantConvergence.certificate.valid` says whether the scalar contraction checks pass, still conditional on the declared operator bounds.
- `ComputationResult` from `fixed_point_residual` is valid because the residual was computed, not because it is zero.
- `ObstructionResult.certificate.valid` checks the obstruction computation, while `vanishes` decides solvability.
- `LiftStepResult.certificate.valid` checks the computed child set, while `lifts` decides whether that set is nonempty.
- `TransportGraph.gluing_certificate().valid` itself is the zero-holonomy decision because that particular certificate is defined around vanishing cycle residuals.

40.7 Public records and import tiers

For compact reference, the Chapter 39 records are:

- trees: `OrderedRootedTree`, `TreeTerm`, `TreeExpansionResult`, `MajorantConvergence`, and `PacketTreeResult`.
- Gradings: `FiniteAbelianGroup`, `GradingCertificate`, `GradedConvolutionTerm`, and `GradedConvolutionResult`.
- Maurer-Cartan: `MaurerCartanResult`, `HigherMaurerCartanResult`, and `DividedSquareResult`.

The Chapter 40 records are:

- coordinates and defects: `NonlinearPacketMap` and `NonlinearRefinementDefect`.
- Obstruction: `LinearObstructionProblem`, `ObstructionResult`, and `ObstructionSquare`.
- Lifting: `ModularLinearSolution`, `LiftStepResult`, `LinearLiftProblem`, `PolynomialLiftProblem`, `LiftTreeNode`, and `FiniteLiftTree`.
- Transport: `TransportEdge`, `FundamentalHolonomy`, `IntertwiningResult`, and `TransportGraph`.

Every Chapter 40 record and operation is research-only in `1.0.0b4`. None is in the curated `komorebi_math.beta` or `komorebi_math.stable` manifest. Pin the package version when any of these signatures forms an application contract.

Use these import tiers:

```
# Preferred complete research surface
from komorebi_math import nonlinear

# Convenient promoted constructors
from komorebi_math import PolynomialFixedPointProblem, TransportGraph

# Module-specific typing aliases or implementation-focused inspection
from komorebi_math.nonlinear import graded, lifting, trees
```

`komorebi_math.nonlinear` is the supported broad namespace for this edition. Importing private helpers such as `_normalize_arities`, `_as_int_vector`, or `_oriented_adjacency` binds an application to implementation details and is not recommended.

40.8 Cost and storage summary

These are structural models, not performance promises:

- height-bounded tree enumeration stores N_h trees with $N_h = 1 + \sum_r N_{h-1}^r$.
- A binary leaf-bounded family stores C_{n-1} trees.
- A retained tree expansion stores one result per enumerated tree and reevaluates common subtrees across distinct terms.
- An unpruned packet-tree vertex of arity r over p packets performs p^{r+1} projected operation evaluations.
- A finite group table stores q^2 additions and its exhaustive certificate performs $O(q^3)$ table operations.
- A complete graded basis probe performs d^r operation evaluations for total packet dimension d, plus forbidden-packet projections.
- Higher Maurer-Cartan assembly enumerates q^r label tuples at every retained order r.
- Numerical obstruction uses dense least squares and nullspaces, while the exact path uses symbolic ranks, nullspaces, and a parametric solve.
- One lift step uses finite-field elimination plus up to p^ν explicit children. Polynomial Jacobian validation uses p^d residual corrections.
- A lift tree stores every generated node through the declared depth.
- Spanning-tree transport uses $|V|-1$ tree edges, and fundamental holonomy computes $|E|-|V|+1$ cycle matrices with dense matrix inversions and products.

For large workloads, the decisive controls are `max_trees`, `max_evaluations`, `enumerate_limit`, `linearization_limit`, and `max_nodes`. They guard different stages and are not interchangeable.

40.9 Version-specific source observations

These observations describe version `1.0.0b4`. They are not expanded input contracts:

1. Provenance identifiers collide across modules. `PSC-III-D13` labels both the ordered-tree definition and the linear obstruction datum, while `PSC-III-T14` labels both majorant convergence and the cokernel criterion. Attached `PSC-III-T41` records both nonlinear Schur elimination and the fundamental-holonomy intertwining criterion. Emitted runtime identifiers also collide at `PSC-III-T40` and `PSC-III-T41` between elimination and holonomy. The computations and test suites remain distinct, but consumers should qualify a theorem string by its emitting symbol rather than use the string alone as a globally unique evidence key.
2. `NonlinearPacketMap.iterate` rejects a negative count but does not perform the explicit integral/boolean normalization used by the tree APIs. A nonintegral value therefore reaches Python's `range` behavior, and a boolean acts as zero or one. Pass a genuine nonnegative integer.
3. `NonlinearRefinementDefect` relies on NumPy operations and does not perform a complete constructor-time dimension or finiteness audit. Validate custom callables, matrix shapes, Lipschitz constants, and sampled outputs at the application boundary.
4. `LinearObstructionProblem`, graded constructors, Maurer-Cartan helpers, and `TransportGraph` do not uniformly reject every nonfinite or negative tolerance at construction. Use a finite nonnegative tolerance.
5. The lifting conversion accepts values according to Python's integral equality test, so booleans can pass as zero or one. The lift-problem depth is checked for positivity and then converted with `int`, rather than being guarded with the strict integer check used by tree depth. Supply genuine integers for coefficients, points, and depths.
6. `PolynomialLiftProblem` converts values returned by `function` with `int` without first checking equality with the original values. A nonintegral callable result can therefore be truncated before congruence tests. Ensure that the callable returns genuine integer values.
7. `terminal_branches()` follows only nodes in the final stored level. A branch that dies while another branch continues is not returned. If all branches die, `build_lift_tree` appends an empty final level, `FiniteLiftTree.max_depth` reports zero, and `terminal_branches()` returns no branches even though earlier levels contain nodes. Inspect `levels` and one-step child sets directly when terminal failures are part of the analysis.
8. `ordered_trees_with_leaves` enforces `max_trees` after generating its cached family. Use an independently computed Catalan or recurrence bound before a large leaf request.
9. Exactness on polynomial tree and graded/Maurer-Cartan APIs is caller declared. Marking floating data as `Exactness.EXACT` does not turn it into exact arithmetic or verify multilinearity.
10. A supplied packet zero-support record is used for pruning by matching addresses. The tree routine does not prove that the record was generated from the same operation, point, and packet system.
11. `TransportGraph` chooses a deterministic breadth-first spanning tree from the stored vertex and edge order. Fundamental holonomy matrices and their edge labels can change with this presentation, while the trivial-holonomy gluing decision remains invariant.

Treat these as narrow source-audit notes. Future releases may strengthen the guards without preserving the weak behaviors.

40.10 Executed verification ledger

The chapter was checked against the shipped source rather than only against the symbolic topology.

- Focused tests executed: `test_nonlinear_trees.py`, `test_nonlinear_graded.py`, `test_nonlinear_coordinates_defects.py`, `test_nonlinear_obstruction.py`, `test_lifting.py`, and `test_holonomy.py`.
- Result: **41 passed in 2.99 seconds**.
- Shipped examples executed successfully: `nonlinear_trees_elimination.py`, `obstruction_lifting.py`, and `holonomy.py`.
- The exact and numerical panels printed in this chapter were rerun under CPython `3.12.13`, NumPy `2.5.2`, SymPy `1.14.0`, and pytest `9.1.1`.
- No optional JAX, PyTorch, or CuPy path is required by Chapters 39 through 40.

This evidence establishes parity with the finite shipped implementation on the executed platform. It does not turn the conditional majorant, Lipschitz, Jacobian-trust, or infinite-lift hypotheses into verified global theorems.

41. Correction and elimination

A nonlinear equation can be reduced through four certified routes. The routes are different by design. A Hilbert obstruction projector asks whether a linear defect lies in an image. A right inverse supplies a contraction map for a nonlinear correction. Lyapunov-Schmidt reduction separates kernel and cokernel coordinates near a root. Nonlinear Schur elimination solves a hidden equation and substitutes its local branch into a retained equation. They share matrix operations, but they do not share the same hypotheses or the same meaning of success.

What you know	Route	What remains to be checked
A linear correction map and a defect	Hilbert obstruction projector	Whether `vanishes` is true
A right inverse and a justified Lipschitz bound	Right-inverse correction	The ball hypothesis and returned equation residual
A root and its derivative, possibly rank deficient	Lyapunov-Schmidt reduction	The reduced obstruction after solving the range equation
Retained and hidden equations with an invertible hidden derivative	Nonlinear Schur elimination	The hidden solve and the retained reduced equation

Choose the route from the available hypotheses before choosing a solver. For example, solving the range equation in Lyapunov-Schmidt reduction removes one part of the problem; the remaining obstruction is still an equation to solve, not an error message to suppress.

The implementation is finite-dimensional. Its numerical paths use NumPy complex arrays and singular-value or Newton computations. Its exact Schur path is available when both derivative providers are exact and SymPy is installed. The routines never establish a global implicit branch, a global Lipschitz bound, or rank stability merely by returning a valid finite certificate.

The primary imports are:

```
from komorebi_math.nonlinear import (
    HilbertObstructionProjector,
    LyapunovSchmidtReduction,
    NonlinearSchurElimination,
    RightInverseCorrectionProblem,
)
```

Those four operational classes are also re-exported from `komorebi_math`. Their result-record classes are exported from `komorebi_math.nonlinear`, while the defining modules are `komorebi_math.nonlinear.correction` and `komorebi_math.nonlinear.elimination`. None of the Chapter 41 objects belongs to the curated `komorebi_math.beta` or `komorebi_math.stable` namespace in version `1.0.0b4`. They are implemented research APIs, so applications should pin the package version if reflected signatures are part of their contract.

The chapter uses the common Part VII claim labels:

- **E** means an exact finite algebraic identity.
- **N** means a finite numerical identity accepted relative to a tolerance.
- **C** means a conclusion that still depends on displayed local or analytic assumptions.
- **O** means an optional backend path.
- **I** means integrity or reproducibility evidence, not authorship.

41.1 Hilbert obstruction projectors

This first correction route isolates the component of a declared defect outside the numerically resolved image and reports solvability separately from the validity of the projection computation.

41.1.1 Image, cokernel, and least-norm correction

Let $D : X \to Y$ be represented by an $m \times n$ complex matrix. A full singular-value decomposition resolves a numerical rank r:

$$D = U\Sigma V^*, \qquad U_r = U[:, 0 : r].$$

The implementation forms the orthogonal image and cokernel projectors

$$\Pi_{\text{im}} = U_r U_r^*, \qquad \Pi_{\text{coker}} = I - \Pi_{\text{im}}, \qquad o = \Pi_{\text{coker}}\delta.$$

Here I is the codomain identity I_Y. The obstruction norm is the distance from $-\delta$ to the image of D:

$$\|o\| = \operatorname{dist}(-\delta, \operatorname{im} D), \qquad u_* = -D^+\delta \quad (\|o\| \le \varepsilon).$$

If the resolved obstruction vanishes, the least-norm correction is

$$u_* = -D^+\delta, \qquad Du_* + \delta \approx 0.$$

Here D^+ is NumPy's Moore-Penrose pseudoinverse. The finite-dimensional projection identities are classical. The runtime claim is **N** because the rank, pseudoinverse, and residuals depend on floating singular values and a declared threshold.

`HilbertObstructionProjector` sets

$$\tau_{\text{rank}} = \max\{\tau, \tau\sigma_1\}$$

when singular values exist, and counts values strictly above that threshold. It stores the full `U` and `V` bases because `numpy.linalg.svd` is called with `full_matrices=True`. Consequently, both the kernel and cokernel bases remain available even for a rectangular map.

41.1.2 Result semantics

The result makes obstruction and certificate validity separate facts:

- `vanishes` means $\|o\|_2 \le \tau$.
- `least_norm_correction` is an array only when `vanishes` is true.
- `correction_residual` is $\|Du_* + \delta\|_2$ in the solvable case and $\|o\|_2$ in the obstructed case.
- `certificate.valid` checks the orthogonal distance identity and, when a correction exists, its residual.

An obstructed problem can therefore have a valid certificate. That certificate means that the obstruction was computed consistently. It does not mean that the obstruction vanishes. Code that tests only `result.certificate.valid` will misclassify an honestly certified obstruction as solvable.

The certificate uses theorem identifier `PSC-III-T31`. It records numerical exactness, `correction_residual`, `distance_identity`, the resolved rank, kernel dimension, cokernel dimension, and obstruction norm. Rank stability below the singular-value threshold remains unchecked.

41.1.3 Runnable projection example

```
from komorebi_math.nonlinear import HilbertObstructionProjector

projector = HilbertObstructionProjector(
    [[1.0, 0.0], [0.0, 0.0]],
    tolerance=1e-10,
)

obstructed = projector.solve([0.0, 1.0])
removable = projector.solve([2.0, 0.0])

print(
    obstructed.vanishes,
    obstructed.obstruction_norm,
    obstructed.least_norm_correction,
    obstructed.certificate.valid,
)
print(
    removable.vanishes,
    removable.obstruction_norm,
    removable.least_norm_correction,
    removable.certificate.valid,
)
```

Observed with NumPy `2.5.2`:

```
False 1.0 None True
True 0.0 [-2.+0.j  0.+0.j] True
```

The first line is the important one. The calculation is valid and the equation is obstructed.

41.1.4 Construction and failure boundary

Construction rejects a nonmatrix correction map with `DimensionError`, a negative or nonfinite tolerance with `ValidationError`, and `solve` rejects a defect whose flattened vector dimension is not m. A row or column vector with one singleton axis is flattened. Other two-dimensional shapes are not. Nonfinite defect entries are rejected.

The object does not check that the supplied matrix is a derivative of any nonlinear map. It solves only the displayed finite linear obstruction problem. It also cannot distinguish a true zero singular value from a numerically small one without the caller's rank-scale judgment.

41.2 Right-inverse corrections

41.2.1 Quantitative contraction problem

`RightInverseCorrectionProblem` addresses

$$Dx + N(x) = y,$$

where $D : X \to Y$, $R : Y \to X$, and $N : X \to Y$. Its fixed-point map is

$$\Phi(x) = R(y - N(x)), \qquad x_{m+1} = \Phi(x_m), \qquad x_0 = 0.$$

The constructor receives a radius $R_0 > 0$ and a declared Lipschitz bound $L_N \geq 0$. The hypothesis certificate checks

$$DR = I, \qquad q = \|R\|L_N < 1, \qquad \|R(y - N(0))\| \le (1 - q)R_0.$$

The last inequality places the center displacement inside the contraction ball. The code checks the inequality only at the center. The assertion that L_N bounds N throughout the whole ball is explicitly unchecked, so the existence and uniqueness conclusion remains **C** even though the residual arithmetic is **N**.

The reported target stability estimate is

$$\|x(y) - x(\tilde{y})\| \le \frac{\|R\|}{1 - q}\|y - \tilde{y}\|.$$

`target_stability_bound(target_distance)` evaluates the right-hand side for a caller-supplied nonnegative scalar distance. It does not solve a second target problem or independently verify the distance.

41.2.2 Hypothesis and solution certificates

`hypothesis_certificate()` uses theorem identifier `PSC-III-T32`. Its residual ledger contains:

- `right_inverse`: $\|DR - I_Y\|_2$
- `invariant_ball_excess`: $\max(0, \|R(y - N(0))\|_2 - (1 - q)R_0)$

Its diagnostics include the right-inverse norm, declared Lipschitz bound, contraction factor, center norm, and radius. The certificate is invalid if the right-inverse residual exceeds 10τ, $q \ge 1$, or the ball excess exceeds τ.

`solve(max_iterations=10000)` first calls that certificate's `require()`. Each accepted iterate must remain within $R_0 + 10\tau$. The resulting `RightInverseCorrectionResult` stores the solution, iteration count, contraction factor, equation residual, fixed-point residual, and a second certificate. Reaching the iteration cap does not itself raise an exception. It returns a result whose certificate is invalid when convergence residuals do not pass.

41.2.3 Runnable scalar correction

For

$$x + 0.2x^2 = 0.1,$$

the root near zero is

$$x_* = \frac{-1 + \sqrt{1.08}}{0.4}.$$

```python
import numpy as np

from komorebi_math.nonlinear import RightInverseCorrectionProblem

problem = RightInverseCorrectionProblem(
    np.eye(1),
    np.eye(1),
    lambda x: 0.2 * x * x,
    [0.1],
    radius=0.2,
    lipschitz_bound=0.08,
    tolerance=1e-13,
)

hypotheses = problem.hypothesis_certificate()
result = problem.solve()

print(hypotheses.valid, hypotheses.diagnostics["contraction_factor"])
print(result.solution.tolist())
```

```
print(result.iterations, f"{result.equation_residual:.3e}")
print(problem.target_stability_bound(0.01))
```

Observed output:

```
True 0.08
[0.09807621135331551]
10 4.441e-16
0.010869565217391304
```

41.2.4 Failure example

```
import numpy as np

from komorebi_math.exceptions import ValidationError
from komorebi_math.nonlinear import RightInverseCorrectionProblem

bad = RightInverseCorrectionProblem(
    np.eye(1),
    np.eye(1),
    lambda x: 0.7 * x,
    [0.1],
    radius=1.0,
    lipschitz_bound=1.0,
)

print(bad.hypothesis_certificate().valid)
try:
    bad.solve()
except ValidationError as error:
    print(type(error).__name__, str(error))
```

The certificate is false because $q = 1$. `solve` then raises `ValidationError` with the certificate message. This is an assumption failure, not evidence that the equation has no solution by another method.

41.3 Lyapunov-Schmidt reduction

41.3.1 SVD splitting at a root

Let $F : X \to Y$, let x_0 satisfy $F(x_0) = 0$, and put

$$L = DF(x_0).$$

The numerical singular-value decomposition supplies orthonormal splittings

$$X = \ker L \oplus X_1, \qquad Y = \operatorname{im} L \oplus C.$$

Write the corresponding basis matrices as K, X_1, R, and C. For kernel coordinates κ and complement coordinates ξ, the ambient point is

$$x = x_0 + K\kappa + X_1\xi.$$

The range equation is

$$\Pi_{\mathrm{im}} F(x_0 + K\kappa + X_1\xi) = 0.$$

In the stored orthonormal range coordinates, this is equivalently

$$R^*F(x_0 + K\kappa + X_1\xi) = 0.$$

When a local implicit branch exists, solving it gives

$$\xi = \psi(\kappa), \qquad \kappa_{\text{red}}(\kappa) = \Pi_C F(x_0 + K\kappa + X_1\psi(\kappa)).$$

The stored obstruction coordinates are $C^*F(x_0 + K\kappa + X_1\psi(\kappa))$.

At the base point the resolved kernel and cokernel identities imply

$$D\psi(0) = 0, \qquad D\kappa_{\text{red}}(0) = 0.$$

The identities are classical. This implementation is **N/C** because it resolves rank numerically and assumes that rank and the local branch remain well behaved away from the base point.

41.3.2 Constructor state

`LyapunovSchmidtReduction(provider, base_point, tolerance=1e-10)` requires a positive-dimensional provider with first derivatives. It evaluates the base value and rejects the point unless

$$\|F(x_0)\|_2 \le 10\tau.$$

It constructs a Jacobian by calling the provider once on each standard basis direction, then stores:

- `linearization`
- `rank`
- `kernel_basis` and `kernel_dimension`
- `complement_basis` and `complement_dimension`
- `range_basis`
- `obstruction_basis` and `obstruction_dimension`
- `decomposition`, a `LyapunovSchmidtDecomposition` record

The decomposition certificate has theorem identifier `PSC-III-T33`. It checks that R^*LX_1 is invertible on the resolved rank block. Its unchecked assumption is neighborhood rank stability.

41.3.3 Damped range solve

`solve_range` fixes κ, initializes ξ, and applies Newton's method to the range equation. At every iteration it builds the full provider Jacobian and restricts it to

$$R^*DF(x)X_1.$$

A backtracking factor begins at one and is halved at most twenty times until the range residual decreases. A singular restricted Jacobian or a step that cannot reduce the residual raises `ValidationError`. Exhausting the outer iteration limit returns a `LyapunovSchmidtPoint` with an invalid certificate if the final range residual exceeds 10τ.

The returned `reduced_obstruction` is the cokernel coordinate

$$C^*F(x).$$

It need not vanish. A valid point certificate means the range equation was eliminated. Solutions of the original equation are precisely the computed branch points for which this remaining reduced obstruction also vanishes.

`base_derivative_certificate()` checks the displayed base derivative identities from the SVD splitting. It does not finite-difference the computed branch. `reduced_second_derivative(left, right)` requires provider order two and returns

$$C^* D^2 F(x_0)[Kh, Kk]$$

inside `ComputationResult[np.ndarray]`. The provider's declared second derivative correctness remains unchecked. That result uses theorem identifier `PSC-III-C34`.

41.3.4 Quadratic obstruction example

Consider

$$F(x_1, x_2) = (x_2, x_1^2).$$

At the origin, the kernel is the x_1-axis and the obstruction space is the second target coordinate. The one-coordinate reduced equation is

$$\kappa_{\mathrm{red}}(\kappa) = \kappa^2.$$

```
import numpy as np

from komorebi_math import Exactness
from komorebi_math.nonlinear import (
    CallableDerivativeProvider,
    LyapunovSchmidtReduction,
)

provider = CallableDerivativeProvider(
    lambda x: np.array([x[1], x[0] ** 2]),
    {
        1: lambda x, d: np.array(
            [d[0][1], 2 * x[0] * d[0][0]]
        ),
        2: lambda x, d: np.array(
            [0.0, 2 * d[0][0] * d[1][0]]
        ),
    },
    source_dimension=2,
    target_dimension=2,
    exactness=Exactness.NUMERICAL,
)

reduction = LyapunovSchmidtReduction(
    provider,
    [0.0, 0.0],
    tolerance=1e-12,
)
point = reduction.solve_range([0.3])
quadratic = reduction.reduced_second_derivative([1.0], [1.0])

print(
    reduction.rank,
    reduction.kernel_dimension,
    reduction.obstruction_dimension,
)
print(point.complement_coordinates.tolist())
print(point.reduced_obstruction.tolist(), point.certificate.valid)
print(quadratic.value.tolist(), quadratic.certificate.valid)
```

Observed output:

```
1 1 1
[0.0]
[0.09] True
[2.0] True
```

The valid point at $\kappa = 0.3$ is not a root. It lies on the eliminated range branch and retains the nonzero scalar obstruction 0.09.

41.4 Nonlinear Schur elimination

41.4.1 Retained and hidden equations

Split a variable as $x = (u, v)$, where $u \in \mathbb{K}^{d_P}$ is retained and $v \in \mathbb{K}^{d_Q}$ is hidden. Split the equation as

$$F(u, v) = (F_P(u, v), F_Q(u, v)).$$

At a base point (u_0, v_0), require

$$F_Q(u_0, v_0) = 0, \qquad D_v F_Q(u_0, v_0) \text{ invertible.}$$

The local implicit branch $v = \phi(u)$ then satisfies

$$Dv = -(D_v F_Q)^{-1} D_u F_Q, \qquad F_{\text{red}}(u) = F_P(u, v(u)).$$

Writing that branch as $v = \phi(u)$, the first identity becomes

$$D\phi = -(D_v F_Q)^{-1} D_u F_Q.$$

The reduced retained map

$$F_{\text{red}}(u) = F_P(u, \phi(u))$$

has derivative

$$DF_{\text{red}} = D_u F_P + D_v F_P Dv.$$

Equivalently, in the ϕ notation,

$$DF_{\text{red}} = D_u F_P + D_v F_P D\phi = D_u F_P - D_v F_P (D_v F_Q)^{-1} D_u F_Q.$$

This is the nonlinear analogue of a Schur complement. The finite derivative formula is **E** when both providers are exact SymPy providers, and **N** for numerical providers. The assertion that these derivatives arise from a local branch remains **C**.

41.4.2 Second-order formula

For retained directions h, k, lift them to tangent directions on the branch:

$$\tilde{h} = (h, D\phi[h]), \qquad \tilde{k} = (k, D\phi[k]).$$

Differentiating the hidden equation twice gives

$$D^2 v[h,k] = -(D_v F_Q)^{-1} D^2 F_Q[(h, Dv[h]), (k, Dv[k])],$$

In the ϕ notation this is

$$D^2\phi[h,k] = -(D_v F_Q)^{-1} D^2 F_Q[\tilde{h}, \tilde{k}].$$

Therefore

$$D^2 F_{\text{red}}[h,k] = D^2 F_P[(h, Dv[h]), (k, Dv[k])] + D_v F_P D^2 v[h,k].$$

The equivalent lifted-direction form used by the implementation is

$$D^2 F_{\text{red}}[h,k] = D^2 F_P[\tilde{h}, \tilde{k}] + D_v F_P D^2\phi[h,k].$$

The implementation also assembles the equivalent displayed subtraction form and certifies that both finite expressions agree. This check cannot detect a shared error in the supplied second-derivative providers.

41.4.3 Exact and numerical modes

The constructor decides its mode from the two providers:

- if both providers declare `Exactness.EXACT`, inputs and derivatives are represented as SymPy matrices and the certificate exactness is `exact`
- otherwise, both blocks are evaluated through complex NumPy arrays and the certificate exactness is `numerical`

Both providers must use the concatenated source dimension $d_P + d_Q$. The hidden provider's target dimension must equal d_Q. The retained provider may have an arbitrary positive target dimension.

`first_derivatives()` returns the hidden Jacobian, its inverse, the implicit derivative, the reduced derivative, and a certificate with theorem identifier `PSC-III-T41`. `second_derivative(left, right)` returns the two lifted directions, the implicit second derivative, the reduced second derivative, and a certificate with theorem identifier `PSC-III-T42`.

`solve_hidden` is intentionally numerical only. For a fixed retained value it uses damped Newton iteration on $F_Q(u, v) = 0$, with at most twenty-four backtracking trials per outer iteration. Exact-mode callers must supply or manipulate a symbolic branch externally. `reduced_value` first requires a valid hidden solve and then evaluates F_P on that point. The numerical hidden-solve and reduced-value certificates use the historical theorem identifier `PSC-III-T40`.

41.4.4 Exact second-order example

This example requires the optional SymPy dependency.

```
import sympy as sp

from komorebi_math.nonlinear import (
    NonlinearSchurElimination,
    SympyDerivativeProvider,
)

u, v = sp.symbols("u v")
```

```
retained = SympyDerivativeProvider(
    (3 * u + 2 * v + u * v + v**2,),
    (u, v),
)
hidden = SympyDerivativeProvider(
    (v - 4 * u + u**2 + u * v,),
    (u, v),
)

problem = NonlinearSchurElimination(
    retained,
    hidden,
    1,
    1,
    sp.Matrix([0]),
    sp.Matrix([0]),
)

first = problem.first_derivatives()
second = problem.second_derivative(sp.Matrix([1]), sp.Matrix([1]))

print(list(first.implicit_derivative))
print(list(first.reduced_derivative))
print(list(second.implicit_second_derivative))
print(list(second.reduced_second_derivative))
print(first.certificate.exactness.value)
```

Observed with SymPy `1.14.0`:

```
[4]
[11]
[-10]
[20]
exact
```

41.4.5 Numerical hidden branch

For the same hidden equation, the branch is available explicitly as

$$v(u) = \frac{4u - u^2}{1 + u}$$

near zero. The focused package test evaluates `solve_hidden([0.05])` and matches this expression to absolute tolerance 10^{-12}. A valid numerical certificate checks only the returned hidden residual. It leaves uniqueness, the size of the implicit neighborhood, and absence of remote solutions unchecked.

41.5 Public signature ledger

These signatures are the reflected version `1.0.0b4` contracts. Type annotations are descriptive runtime annotations, not automatic validators.

41.5.1 Correction surface

```
HilbertObstructionProjector(
    correction_map: Any,
    *,
    tolerance: float = 1e-10,
)
    .solve(defect: Any) -> HilbertObstructionResult
```

```
RightInverseCorrectionProblem(
    linear_map: Any,
    right_inverse: Any,
    nonlinear: Callable[[np.ndarray], Any],
    target: Any,
    *,
    radius: float,
    lipschitz_bound: float,
    tolerance: float = 1e-10,
)
    .hypothesis_certificate() -> Certificate
    .solve(
        *,
        max_iterations: int = 10000,
    ) -> RightInverseCorrectionResult
    .target_stability_bound(
        target_distance: float,
    ) -> float
```

```
LyapunovSchmidtReduction(
    provider: DerivativeProvider,
    base_point: Any,
    *,
    tolerance: float = 1e-10,
)
    .solve_range(
        kernel_coordinates: Any,
        *,
        initial_complement: Any | None = None,
        max_iterations: int = 50,
    ) -> LyapunovSchmidtPoint
    .base_derivative_certificate() -> Certificate
    .reduced_second_derivative(
        left: Any,
        right: Any,
    ) -> ComputationResult[np.ndarray]
```

41.5.2 Elimination surface

```
NonlinearSchurElimination(
    retained_provider: DerivativeProvider,
    hidden_provider: DerivativeProvider,
    retained_dimension: int,
    hidden_dimension: int,
    base_retained: Any,
    base_hidden: Any,
    *,
    tolerance: float = 1e-10,
)
    .first_derivatives() -> SchurDerivativeResult
    .second_derivative(
        left: Any,
        right: Any,
    ) -> SchurSecondDerivativeResult
    .solve_hidden(
        retained_value: Any,
        *,
        initial_hidden: Any | None = None,
        max_iterations: int = 50,
```

```
    ) -> HiddenSolveResult
    .reduced_value(
        retained_value: Any,
        *,
        initial_hidden: Any | None = None,
        max_iterations: int = 50,
    ) -> ComputationResult[np.ndarray]
```

41.6 Result-record ledger

All named records below are frozen, slotted dataclasses. Their NumPy arrays remain mutable objects, so `frozen=True` prevents field reassignment but does not make array storage immutable.

- `HilbertObstructionResult`
  - `obstruction`
  - `obstruction_norm`
  - `vanishes`
  - `least_norm_correction`
  - `correction_residual`
  - `certificate`
- `RightInverseCorrectionResult`
  - `solution`
  - `iterations`
  - `contraction_factor`
  - `equation_residual`
  - `fixed_point_residual`
  - `certificate`
- `LyapunovSchmidtDecomposition`
  - `linearization`
  - `kernel_basis`
  - `complement_basis`
  - `range_basis`
  - `obstruction_basis`
  - `rank`
  - `certificate`
- `LyapunovSchmidtPoint`
  - `kernel_coordinates`
  - `complement_coordinates`
  - `point`
  - `reduced_obstruction`
  - `range_residual`
  - `iterations`
  - `certificate`
- `SchurDerivativeResult`
  - `hidden_jacobian`
  - `hidden_inverse`
  - `implicit_derivative`
  - `reduced_derivative`
  - `certificate`
- `SchurSecondDerivativeResult`
  - `lifted_left`

    - `lifted_right`
    - `implicit_second_derivative`
    - `reduced_second_derivative`
    - `certificate`
- `HiddenSolveResult`
    - `hidden`
    - `point`
    - `residual`
    - `iterations`
    - `certificate`

There are no registered package portable codecs for these Chapter 41 records in this release. If they must cross a process boundary, serialize a deliberate schema containing array dtype, shape, real and imaginary components, theorem identifier, and certificate fields. A raw dataclass conversion is not by itself a stable persistence contract.

41.7 Error and certificate interpretation

41.7.1 Exception families

- `DimensionError` reports array dimensionality, vector length, or concatenated-domain shape disagreement.
- `ValidationError` reports nonfinite values, invalid tolerances, missing derivative orders, an uncertified contraction, a bad base root, a singular numerical Jacobian, a failed damped step, or an exact-mode request for the numerical hidden solver.
- Calling `.require()` on an invalid `Certificate` also raises `ValidationError` with the certificate message.
- SymPy is optional. Exact provider construction without it follows that provider's `BackendUnavailableError` path.

An iteration limit is not uniformly an exception boundary. Both the right-inverse solver and the two Newton solvers may return an invalid result certificate after reaching the cap. Singular linear systems and failed backtracking steps raise immediately.

41.7.2 Certificate decision sequence

For safe use, test the facts in this order:

1. Require or inspect the computational certificate.
2. Inspect the semantic flag or residual that represents the mathematical question, such as `vanishes` or `reduced_obstruction`.
3. Inspect `assumptions_unchecked` before promoting a local calculation to a local existence statement.
4. Preserve theorem identifiers and tolerance metadata with reported results.

Neither a valid obstruction certificate nor a valid range-elimination certificate asserts that the original nonlinear equation is solved.

41.8 Cost and storage model

Let $D \in \mathbb{C}^{m\times n}$, let d_X and d_Y be provider dimensions, let r be a resolved rank, and let d_Q be the hidden dimension.

- Hilbert obstruction construction performs one full SVD and one pseudoinverse. Dense arithmetic is on the order of $O(mn\min(m,n))$, while full bases and projectors require $O(m^2+n^2+mn)$ storage.
- One Hilbert `solve` applies dense projectors and, when solvable, the pseudoinverse. It is $O(m^2+mn)$ with the stored matrices.
- A right-inverse hypothesis check takes dense operator norms and matrix products. Each fixed-point iteration performs one nonlinear evaluation and one multiplication by R. Runtime therefore scales linearly with the observed iteration count, apart from the user callback.
- Lyapunov-Schmidt construction calls the first derivative once per source basis vector and performs a full SVD of a $d_Y \times d_X$ Jacobian. Each Newton iteration rebuilds that Jacobian, solves an $r \times r$ system, and may perform up to twenty extra value evaluations for damping.

- First-order Schur assembly makes $d_P + d_Q$ directional derivative calls for each equation provider and inverts a $d_Q \times d_Q$ matrix. The dense inversion cost is $O(d_Q^3)$.
- Second-order Schur assembly adds one second-derivative call per equation block and dense feedback products.
- A numerical hidden solve rebuilds d_Q hidden Jacobian columns per outer iteration, solves one $d_Q \times d_Q$ system, and may make up to twenty-four damping evaluations.

No Chapter 41 routine is sparse-aware. Passing a sparse mathematical problem through these APIs materializes dense NumPy or SymPy matrices.

41.9 Version-specific source observations

These boundaries were reproduced against version `1.0.0b4` and should be treated as source observations rather than desired future guarantees:

1. `HilbertObstructionProjector` permits `tolerance=0`. Its internal distance comparison still uses a 10^{-14} numerical floor, while the emitted certificate tolerance is zero.
2. Obstructed Hilbert and filtered correction results can carry valid certificates. This is intentional but easy to misuse if `vanishes` is not checked.
3. A right-inverse Lipschitz bound is declared, not sampled or proved. The target stability helper accepts a scalar distance and does not compare two solved targets.
4. `base_derivative_certificate()` constructs $D\psi(0) = 0$ from the resolved splitting. It does not differentiate the numerical branch returned by `solve_range`.
5. Schur exactness is selected only when both providers declare exactness. Automatic hidden solving is unavailable in that mode.
6. A singular exact SymPy hidden derivative currently exposes SymPy's `NonInvertibleMatrixError`, while the numerical path normalizes a singular inverse to `ValidationError`.
7. None of the local numerical solvers establishes global uniqueness or finds disconnected solution branches.

41.10 Verification record

The focused correction and elimination test module passed **5/5** tests in the recorded guide qualification environment. The shipped `nonlinear_trees_elimination.py` example also passed and printed:

```
finite tree sum: 0.11256641000000002
Picard tail bound at depth 4: 0.004266666666666668
implicit derivative: [[4.-0.j]]
reduced derivative: [[11.+0.j]]
reduced obstruction: [0.09]
```

The installed advanced nonlinear verifier passed its correction, Lyapunov-Schmidt, and nonlinear Schur checks. These are finite regression results. They do not replace the unchecked local hypotheses displayed above.

42. Filtered lifting and finite spectral sequences

Chapter 42 moves from numerical Hilbert spaces to exact linear algebra over a prime field. A finite decreasing filtration records successive depths of information. A correction problem solves one associated-graded equation. A branch verifier checks compatibility through a finite terminal depth. A filtered cochain complex exposes exact spectral-sequence pages. Only the final projective-limit helper crosses beyond finite data, and it does so through a `ConditionalResult` whose infinite hypotheses remain explicit.

The primary operational imports are:

```
from komorebi_math.nonlinear import (
    FilteredCorrectionProblem,
    FilteredVectorSpace,
    FiniteFilteredCochainComplex,
    conditional_projective_limit_completion,
    verify_finite_filtered_branch,
)
```

All five names are also available from `komorebi_math`. Result records and page records are exported from `komorebi_math.nonlinear`, with definitions in `komorebi_math.nonlinear.filtered`. These objects are not included in the curated beta or stable namespace in version `1.0.0b4`.

All finite filtration, correction, branch, and cochain computations in this chapter use exact row reduction modulo a validated prime p. Their **E** label refers to that finite field computation. It does not mean that an integer, rational, p-adic, or topological object has been reconstructed beyond the explicit finite model.

Keep the two depths separate. A filtration depth identifies a level of the supplied finite space. A spectral-sequence page identifies a stage of the associated cohomology calculation. The examples start with small spaces so both can be inspected directly. Retain the actual basis and differential matrices as well as the displayed dimensions; dimensions alone do not reconstruct the computation.

42.1 Filtered vector spaces

42.1.1 Finite decreasing filtration

A `FilteredVectorSpace` represents

$$V = F^0V \supseteq F^1V \supseteq \cdots \supseteq F^NV = 0, \qquad \mathrm{gr}^k V = F^kV/F^{k+1}V.$$

over $\mathbf{F}_p$. Every level is supplied as an ambient-coordinate basis matrix. Construction reduces its columns to a basis by modular row reduction, then verifies:

- depths are contiguous from zero
- $F^0V = V$
- every $F^{k+1}V$ is contained in F^kV
- the final declared level is the zero subspace
- p is prime

Because quotient elements are represented computationally, the runtime chooses a deterministic complement basis inside the supplied column ordering. It extends a basis of $F^{k+1}V$ with independent columns from F^kV. The resulting `graded_basis(k)` consists of ambient representatives of a quotient basis. It is not a canonical basis independent of the input coordinates and column order.

For $v \in F^k V$, `graded_coordinates(v, k)` solves

$$v = G_k c + B_{k+1} b \pmod{p},$$

where G_k is the selected graded complement and B_{k+1} is a basis of the deeper level. It returns c. Conversely,

$$\text{lift_graded}(c, k) = G_k c \pmod{p}.$$

The lift is a selected representative. Adding any vector from $F^{k+1} V$ represents the same graded class.

42.1.2 Weight constructor

`FilteredVectorSpace.from_weights(weights, prime=p)` assigns a nonnegative integer weight w_i to each coordinate and constructs

$$F^k V = \text{span}\{e_i : w_i \geq k\}.$$

The terminal depth is

$$N = 1 + \max_i w_i,$$

with the empty-coordinate default producing a zero-dimensional space with terminal depth one. The constructor requires every weight to be an actual nonnegative integer and rejects booleans.

For weights `(0, 1)` over $\mathbf{F}_5$,

$$\dim F^0 V = 2, \qquad \dim F^1 V = 1, \qquad \dim F^2 V = 0,$$

and both graded pieces have dimension one.

```
from komorebi_math.nonlinear import FilteredVectorSpace

space = FilteredVectorSpace.from_weights([0, 1], prime=5)

print(tuple(space.dimension_at(k) for k in range(3)))
print(tuple(space.graded_dimension(k) for k in range(2)))
print(space.graded_coordinates([3, 4], 0).tolist())
print(space.graded_coordinates([0, 4], 1).tolist())
print(space.lift_graded([3], 0).tolist())
print(space.lift_graded([4], 1).tolist())
print(space.certificate().exactness.value)
```

Observed output:

```
(2, 1, 0)
(1, 1)
[3]
[4]
[3, 0]
[0, 4]
exact
```

42.1.3 Depth behavior

`basis(depth)` returns a copy. Depths at or below zero select F^0V, while depths at or above the terminal depth select the zero space. Consequently `dimension_at` and `contains` use the same clamped behavior. By contrast, `graded_basis(depth)` returns a zero-column basis outside $0 \le k < N$.

`certificate()` uses theorem identifier `PSC-III-D50`. It has exactness `exact`, no tolerance, and diagnostics for the prime, ambient dimension, terminal depth, and all level dimensions.

42.2 Filtered correction problems

42.2.1 One-depth obstruction equation

Let $D : V \to W$ preserve the relevant filtration levels, and let

$$\delta \in F^k W.$$

A depth-k correction seeks a graded class $[u] \in \mathrm{gr}^k V$ such that

$$\mathrm{gr}^k D([u]) = -[\delta], \qquad \delta + Du \in F^{k+1}W.$$

For any lifted representative $u \in F^k V$, this is equivalent to the second relation in the display.

`FilteredCorrectionProblem` first checks filtration preservation at both depths k and $k+1$:

$$D(F^j V) \subseteq F^j W, \qquad j \in \{k, k+1\}.$$

It then constructs the induced matrix between the selected graded bases, computes the defect's graded coordinates, and calls exact modular row reduction to solve the affine equation.

42.2.2 Correction coset

The solution is a `ModularLinearSolution`:

$$c = c_0 + \sum_{i=1}^{\nu} a_i z_i, \qquad a_i \in \mathbf{F}_p.$$

Its nullity is ν, and its solution count is

$$N_{\mathrm{corr}} = \begin{cases} 0, & \text{if obstructed,} \\ p^{\nu}, & \text{if solvable.} \end{cases}$$

`solve(enumerate_limit=100000)` enumerates the entire correction coset only when this count does not exceed the guard. Every class is lifted to an ambient tuple, and every enumerated representative is checked to improve the defect by one level.

The result's property

```
DepthObstructionResult.vanishes
```

is exactly `correction_space.solvable`. As in Chapter 41, an obstructed result can carry a valid certificate. Its valid certificate means the finite obstruction and empty correction set were computed consistently.

The certificate uses theorem identifier `PSC-III-T51` and records the depth, obstruction status, kernel dimension, solution count, and a child-verification residual.

42.2.3 Solvable and obstructed example

```
import numpy as np

from komorebi_math.nonlinear import (
    FilteredCorrectionProblem,
    FilteredVectorSpace,
)

source = FilteredVectorSpace.from_weights([0, 1], prime=5)
target = FilteredVectorSpace.from_weights([0, 1], prime=5)

blocked = FilteredCorrectionProblem(
    source,
    target,
    np.diag([1, 0]),
    [0, 1],
    1,
).solve()

solved = FilteredCorrectionProblem(
    source,
    target,
    np.eye(2, dtype=int),
    [0, 1],
    1,
).solve()

print(blocked.vanishes, blocked.correction_representatives)
print(blocked.certificate.valid)
print(solved.vanishes, solved.correction_representatives)
print(solved.certificate.valid)
```

Observed output:

```
False ()
True
True ((0, 4),)
True
```

The representative `(0, 4)` changes the defect `(0, 1)` to zero modulo five. The obstructed result is still an exact and valid description of the failed graded equation.

42.2.4 Failure boundary

Construction rejects different source and target primes, a correction matrix with the wrong shape, a defect with the wrong dimension, and a depth outside the common associated-graded range. `induced_map()` raises `ValidationError` when D does not preserve the two required levels. `solve()` raises when the defect is not in its declared depth.

The enumeration guard currently comes from `ModularLinearSolution.enumerate`. If p^{ν} exceeds the limit, it raises built-in `ValueError` rather than the package's `ResourceLimitError`. A caller that expects large correction spaces should inspect `correction_space.solution_count` through a lower-level solve or choose a safe guard explicitly rather than requesting accidental exhaustive enumeration.

42.3 Finite filtered cochain complexes

42.3.1 Complex and filtration contract

A `FiniteFilteredCochainComplex` represents a bounded finite cochain complex

$$\cdots \xrightarrow{d^{n-1}} C^n \xrightarrow{d^n} C^{n+1} \to \cdots, \qquad d^{n+1}d^n = 0, \qquad d(F^pC^n) \subseteq F^pC^{n+1}.$$

The complex is over one prime field and carries a common finite decreasing filtration in every declared degree. Each differential must preserve every filtration level as displayed.

The constructor accepts explicit degree dimensions, any supplied differentials, and one `FilteredVectorSpace` per degree. A missing differential is interpreted as the appropriately shaped zero map when queried through `differential(degree)`.

It rejects inconsistent dimensions, absent filtrations, mixed primes, unequal terminal depths, differential shapes that disagree with adjacent dimensions, failure of $d^2 = 0$, and any filtration-nonpreserving differential.

42.3.2 Implemented finite subquotients

Let the total cochain degree be $n = p + q$. The implementation defines

$$Z_r^{p,n} = \{x \in F^pC^n : dx \in F^{p+r}C^{n+1}\}, \qquad B_r^{p,n} = F^pC^n \cap d(F^{p-r}C^{n-1}).$$

At page zero,

$$E_0^{p,q} = F^pC^{p+q}/F^{p+1}C^{p+q}.$$

For $r \geq 1$, the source implements the finite subquotient

$$E_r^{p,q} = Z_r^{p,p+q} \Big/ \left(Z_{r-1}^{p+1,p+q} + B_{r-1}^{p,p+q}\right).$$

The page differential has bidegree

$$d_r : E_r^{p,q} \to E_r^{p+r,q-r+1}.$$

All subspaces, intersections, preimages, images, quotients, and ranks are computed by exact modular row reduction. The implementation raises `ArithmeticError` if an internally assembled quotient denominator is not contained in its numerator. That exception signals an invariant failure in the page construction, not ordinary user input validation.

42.3.3 Page certificate

For every bidegree, the page checks

$$\dim E_{r+1}^{p,q} = \dim E_r^{p,q} - \operatorname{rank} d_r^{p,q} - \operatorname{rank} d_r^{p-r,q+r-1}.$$

`SpectralSequencePage` stores all nonzero or incident entries, its total dimension, the sum of outgoing differential ranks, and an exact certificate. `page.entry(p, q)` returns the matching entry. If no stored entry exists, it returns a synthetic zero-dimensional entry on that page rather than `None`.

The page certificate uses theorem identifier `PSC-III-S54`. It records the number of failed page-cohomology relations and diagnostics for page number, entry count, total dimension, and total differential rank.

42.3.4 Finite convergence

The runtime constructs pages

$$E_0, E_1, \dots, E_N$$

through the common terminal filtration depth. It computes ordinary cohomology dimensions as

$$\dim H^n(C) = \dim C^n - \operatorname{rank} d^n - \operatorname{rank} d^{n-1},$$

using the already validated relation $d^n d^{n-1} = 0$. The terminal check is

$$\sum_{p+q=n} \dim E_\infty^{p,q} = \dim H^n(C), \qquad \operatorname{rank} d_\infty = 0.$$

In the finite implementation, E_N supplies the limiting page. The `FilteredSpectralSequenceResult` certificate is valid only when every page certificate is valid, all terminal differential ranks vanish, and limiting dimensions match ordinary cohomology dimensions.

This is an exact theorem about the explicit bounded finite filtered complex. It does not claim convergence of an unbounded or topological spectral sequence.

42.3.5 Nonzero differential example

This two-term complex has a nonzero d_1 cancellation.

```
import numpy as np

from komorebi_math.nonlinear import (
    FilteredVectorSpace,
    FiniteFilteredCochainComplex,
)

c0 = FilteredVectorSpace(
    1,
    {
        0: np.eye(1, dtype=int),
        1: np.zeros((1, 0), dtype=int),
        2: np.zeros((1, 0), dtype=int),
    },
    prime=5,
)
c1 = FilteredVectorSpace.from_weights([1], prime=5)

complex_ = FiniteFilteredCochainComplex(
    {0: 1, 1: 1},
    {0: [[1]]},
    {0: c0, 1: c1},
    prime=5,
)

result = complex_.spectral_sequence()

print(
    [
        (page.page, page.total_dimension, page.total_differential_rank)
        for page in result.pages
    ]
)
print(result.total_cohomology_dimensions)
print(result.limiting_dimensions)
print(result.certificate.valid)
```

Observed output:

```
[(0, 2, 0), (1, 2, 1), (2, 0, 0)]
{0: 0, 1: 0}
{0: 0, 1: 0}
True
```

The page-zero rank is zero because d_0 preserves filtration degree. The nonzero map appears at d_1, after which both classes cancel.

42.4 Branch verification

This section verifies a caller-supplied finite branch one filtration level at a time. It does not construct the branch or promote a finite terminal check to an infinite convergence claim.

42.4.1 Finite compatible branches

Given a map $F : V \to W$, a finite branch beginning at depth k_0 is a sequence

$$x_{k_0}, x_{k_0+1}, \dots, x_{k_1}$$

that should satisfy

$$F(x_k) \in F^k W, \qquad x_{k+1} - x_k \in F^k V.$$

`verify_finite_filtered_branch` evaluates the supplied `map_function` at every point. It counts depth failures and compatibility failures exactly modulo the common prime. Its result uses theorem identifier `PSC-III-T52`.

If the last verified depth reaches the target's terminal depth N, then

$$F^N W = 0$$

forces

$$F(x_N) = 0.$$

Accordingly, the certificate is invalid when a terminal branch ends with a nonzero defect. For a prefix ending before N, certificate validity means only that the displayed finite prefix is compatible. The unchecked assumptions explicitly record continuation beyond that prefix.

42.4.2 Runnable terminal branch

```
from komorebi_math.nonlinear import (
    FilteredVectorSpace,
    verify_finite_filtered_branch,
)

source = FilteredVectorSpace.from_weights([0, 1], prime=5)
target = FilteredVectorSpace.from_weights([0, 1], prime=5)

result = verify_finite_filtered_branch(
    source,
    target,
    lambda point: (point[0] - 1, point[1] - 2),
    [(0, 0), (1, 0), (1, 2)],
)

print(result.approximations)
```

```
print(result.limit)
print(result.exact_solution)
print(result.certificate.valid)
print(result.certificate.diagnostics)
```

Observed core output:

```
((0, 0), (1, 0), (1, 2))
(1, 2)
True
True
{'start_depth': 0, 'final_depth': 2, 'terminal_depth': 2, 'terminal_reached': True}
```

The word `limit` in `FiniteFilteredBranchResult` means the last point of this finite branch. It does not mean a topological inverse limit.

42.4.3 Invalid versus exceptional branches

The function raises `ValidationError` for different primes, a noncallable map, an empty approximation sequence, a negative or nonintegral start depth, or a branch extending beyond the finite terminal filtration. Wrong point or defect dimensions also raise.

A well-shaped branch with bad compatibility or defect depth returns a result whose certificate is false. This distinction lets a verifier retain the failed finite record without converting every mathematical counterexample into an exception.

42.5 Conditional projective-limit completion

42.5.1 Finite evidence is not infinite completion

An infinite filtered lifting argument requires more than arbitrarily many finite computations. The helper displays the external requirements as

$$\bigcap_k F^k V = 0, \qquad V \cong \varprojlim V/F^k V,$$

$$F \text{ continuous}, \quad (x_k)_k \text{ compatible and infinite.}$$

Only under those declarations may one conclude conditionally that

$$x = \lim_k x_k, \qquad F(x) = 0.$$

`conditional_projective_limit_completion` does not receive spaces, bonding maps, a topology, a map F, or the branch itself. It records four booleans:

- `separated`
- `complete`
- `continuous`
- `compatible_infinite_branch`

It returns `ConditionalResult[ProjectiveLimitRecord]`. The certificate uses exactness `conditional` and theorem identifier `PSC-III-T53`. A finite `verified_prefix_depth` is diagnostic metadata only. Increasing that integer cannot turn a finite prefix into proof of an infinite branch.

42.5.2 Truth-table example

```
from komorebi_math.nonlinear import (
    conditional_projective_limit_completion,
)

incomplete = conditional_projective_limit_completion(
    7,
    separated=True,
    complete=False,
    continuous=True,
    compatible_infinite_branch=True,
)

declared = conditional_projective_limit_completion(
    7,
    separated=True,
    complete=True,
    continuous=True,
    compatible_infinite_branch=True,
)

print(incomplete.certificate.valid)
print(incomplete.value.conclusion)
print(incomplete.certificate.assumptions_unchecked)
print(declared.certificate.valid)
print(declared.certificate.exactness.value)
print(declared.certificate.assumptions_unchecked)
```

Observed core output:

```
False
the inverse-limit conclusion is not licensed
('complete',)
True
conditional
('the declared infinite branch and topological hypotheses are external inputs, not established by the finite
   ↪  runtime',)
```

Even the all-true record retains the external-input warning. Its valid status means the caller declared every premise. It does not mean the runtime proved those premises.

42.6 Public signature ledger

42.6.1 Filtered spaces and corrections

```
FilteredVectorSpace(
    dimension: int,
    levels: Mapping[int, Any],
    *,
    prime: int,
)
    .from_weights(
        weights: Sequence[int],
        *,
        prime: int,
    ) -> FilteredVectorSpace
    .basis(depth: int) -> np.ndarray
    .dimension_at(depth: int) -> int
    .contains(vector: Any, depth: int) -> bool
```

```
    .graded_basis(depth: int) -> np.ndarray
    .graded_dimension(depth: int) -> int
    .graded_coordinates(
        vector: Any,
        depth: int,
    ) -> np.ndarray
    .lift_graded(
        coordinates: Any,
        depth: int,
    ) -> np.ndarray
    .certificate() -> Certificate
```

```
FilteredCorrectionProblem(
    source: FilteredVectorSpace,
    target: FilteredVectorSpace,
    correction_map: Any,
    defect: Any,
    depth: int,
)
    .induced_map() -> np.ndarray
    .solve(
        *,
        enumerate_limit: int = 100000,
    ) -> DepthObstructionResult
```

42.6.2 Branch and projective helpers

```
verify_finite_filtered_branch(
    source: FilteredVectorSpace,
    target: FilteredVectorSpace,
    map_function: Callable[
        [tuple[int, ...]],
        Sequence[int],
    ],
    approximations: Sequence[Sequence[int]],
    *,
    start_depth: int = 0,
) -> FiniteFilteredBranchResult
```

```
conditional_projective_limit_completion(
    verified_prefix_depth: int,
    *,
    separated: bool,
    complete: bool,
    continuous: bool,
    compatible_infinite_branch: bool,
) -> ConditionalResult[ProjectiveLimitRecord]
```

42.6.3 Filtered complex

```
FiniteFilteredCochainComplex(
    dimensions: Mapping[int, int],
    differentials: Mapping[int, Any],
    filtrations: Mapping[int, FilteredVectorSpace],
    *,
    prime: int,
```

```
)
    .differential(degree: int) -> np.ndarray
    .page(page: int) -> SpectralSequencePage
    .total_cohomology_dimensions() -> dict[int, int]
    .spectral_sequence() -> FilteredSpectralSequenceResult
```

```
SpectralSequencePage
    .entry(
        filtration_degree: int,
        complementary_degree: int,
    ) -> SpectralSequenceEntry
```

42.7 Result-record ledger

The filtered result records are frozen, slotted dataclasses. Their NumPy arrays remain mutable unless a caller copies or protects them.

- `DepthObstructionResult`
  - `depth`
  - `induced_correction_map`
  - `defect_coordinates`
  - `correction_space`
  - `correction_representatives`
  - `certificate`
  - property `vanishes`
- `FiniteFilteredBranchResult`
  - `approximations`
  - `limit`
  - `exact_solution`
  - `certificate`
- `ProjectiveLimitRecord`
  - `verified_prefix_depth`
  - `separated`
  - `complete`
  - `continuous`
  - `compatible_infinite_branch`
  - `conclusion`
- `SpectralSequenceEntry`
  - `page`
  - `filtration_degree`
  - `complementary_degree`
  - `total_degree`
  - `dimension`
  - `outgoing_rank`
  - `incoming_rank`
- `SpectralSequencePage`
  - `page`
  - `entries`
  - `total_dimension`
  - `total_differential_rank`
  - `certificate`

- `FilteredSpectralSequenceResult`
  - `pages`
  - `total_cohomology_dimensions`
  - `limiting_dimensions`
  - `certificate`

`ModularLinearSolution`, used inside `DepthObstructionResult`, stores `prime`, `solvable`, `particular`, `kernel_basis`, `pivot_columns`, and `obstruction_rows`. Its computed properties are `nullity` and `solution_count`, and its `enumerate(limit=100000)` method materializes every affine solution when allowed by the guard.

No Chapter 42 record has a registered package portable codec in this release. For durable storage, use a versioned application schema. Store the prime, ambient dimensions, terminal depth, basis matrices, degree keys, theorem identifiers, and certificates rather than only printed page dimensions.

42.8 Certificate and error map

The theorem identifier ledger is:

- `PSC-III-D50`: finite decreasing filtration definition
- `PSC-III-T51`: one-step associated-graded correction
- `PSC-III-T52`: finite compatible branch
- `PSC-III-T53`: conditional projective-limit completion
- `PSC-III-S54`: finite filtered spectral sequence

The exact finite certificates have no floating tolerance. Their residuals are integer failure counts encoded as floats for the shared certificate schema. Projective completion is the sole `Exactness.CONDITIONAL` result in this chapter.

Input failures are normally `ValidationError` or its `DimensionError` subclass. The principal categories are:

- a composite or otherwise invalid field modulus
- nonintegral modular matrix or vector entries
- noncontiguous, nonnested, incomplete, or nonzero-terminal filtrations
- dimensions or differential shapes that do not agree
- a correction map or differential that does not preserve filtration
- successive differentials with nonzero composition
- an invalid page or branch depth
- an enumeration count beyond its declared materialization limit

The exact modular runtime does not silently coerce a composite modulus into a field computation. `require_prime` rejects it before row reduction.

42.9 Cost and storage model

Let d bound ambient dimensions, N be terminal filtration depth, g the number of cochain degrees, and ν a correction-space nullity.

- Filtration construction row-reduces every supplied level basis and performs nested-subspace tests. With dense square-scale bases, a conservative model is $O(Nd^3)$ field operations and $O(Nd^2)$ stored integers.
- A graded basis or coordinate operation performs further exact rank or linear solves on ambient matrices. The implementation does not cache every quotient computation.
- One filtered correction builds an induced graded map and solves a modular system. Enumerating representatives adds exactly p^ν output vectors, so time and storage become exponential in nullity even though row reduction is polynomial.
- A branch of length L performs L map evaluations, L defect membership tests, and $L-1$ compatibility membership tests. User callback cost is additional.
- One spectral page visits each declared total degree and filtration degree, repeatedly constructing images, preimages, intersections, spans, and quotient complements. The source favors transparent exact recomputation over cached asymptotic optimization.

- `spectral_sequence()` materializes $N+1$ pages. The entry ledger has at most $O(N^2 g)$ bidegree slots across all pages, though zero unincident entries are omitted from stored tuples.
- Projective completion stores only a fixed-size record and performs no limit computation.

These methods use dense NumPy integer arrays for modular bases. They are not designed for very large sparse chain complexes.

42.10 Version-specific source observations

These behaviors were confirmed from the version `1.0.0b4` source and adversarial probes:

1. `FilteredVectorSpace.basis`, `dimension_at`, and related depth accessors call `int(depth)` without checking integrality. For example, depth `1.9` is treated as depth `1`. Constructors and page/branch entry points are stricter in several, but not all, locations.
2. `FiniteFilteredCochainComplex` converts declared dimension values with `int` and does not compare the result back to the input. A value such as `1.9` can be accepted as dimension `1` when the filtration agrees.
3. `conditional_projective_limit_completion` converts each premise with `bool`. A nonempty string such as `"False"` is therefore treated as true. Pass actual booleans only.
4. The filtered correction enumeration guard raises built-in `ValueError` rather than `ResourceLimitError`.
5. `DepthObstructionResult.certificate.valid` can be true while `vanishes` is false. Callers must test both the integrity of the computation and the obstruction status.
6. Quotient complements depend on supplied basis-column order. Dimensions and represented subspaces are invariant, but printed coordinate representatives should not be treated as canonical across arbitrary rebasings.
7. The projective-limit helper checks only declared truth flags. It has no runtime objects from which to establish separation, completeness, continuity, or an infinite compatible branch.

These are documentation boundaries, not invitations to depend on permissive validation. Future releases may reject inputs that this source coerces.

42.11 Integrated finite workflow

The finite correction pipeline can be summarized as

$$\begin{aligned} \delta_k &\longrightarrow [\delta_k] \in \mathrm{gr}^k W \longrightarrow \mathrm{gr}^k D[c_k] = -[\delta_k], \\ x_{k+1} = x_k + \tilde{c}_k &\longrightarrow \delta_{k+1} \in F^{k+1} W. \end{aligned}$$

At every depth, retain the complete `DepthObstructionResult`. If `vanishes=False`, the branch stops with an exact finite obstruction. If it vanishes, choose a representative explicitly, update the approximation, and verify the resulting branch independently. Reaching the terminal zero level gives a finite exact solution. Stopping earlier gives only a verified prefix.

A spectral sequence packages related finite cancellations across an entire cochain complex. It does not replace the branch computation for a nonlinear map. Conversely, a correction branch does not establish the cohomological page identities of a filtered complex unless the complex and its differentials are constructed explicitly.

42.12 Verification record

The focused filtered test module passed **7/7** tests in the recorded guide qualification environment. Together with Chapter 41, the two focused modules passed **12/12**. The installed nonlinear verifier passed all **14/14** checks, including finite filtered lifting and filtered spectral-sequence checks.

The shipped `filtered_transport.py` example also passed. Its Chapter 42 output was:

```
page 0 dimension 2 rank 0
page 1 dimension 2 rank 1
page 2 dimension 0 rank 0
```

The same example continues into Chapter 44 and reported a coarse-to-fine Kato transport difference of approximately 4.54×10^{-15}. That later numerical transport result is independent of the exact finite-field status of the spectral-sequence pages.

42.13 Chapter checklist

Before relying on a filtered computation:

1. Confirm the modulus is prime and every modular entry is integral.
2. Inspect the filtration certificate and level dimensions.
3. Check both `certificate.valid` and the semantic obstruction flag.
4. Bound correction-coset enumeration before materializing it.
5. Preserve the chosen graded representative convention.
6. Verify every finite branch point and its terminal status.
7. Treat a finite prefix as finite even when many depths were checked.
8. Require every page certificate and the terminal convergence certificate.
9. Pass actual booleans to the projective-limit recorder.
10. State separation, completeness, continuity, and infinite compatibility as external assumptions whenever an inverse-limit conclusion is quoted.

43. Nonlinear execution, streaming, profiling, and GPU paths

The packet-jet definitions in Chapter 38 expose every target packet and every ordered source-packet tuple. That explicit family is mathematically useful, but a literal evaluator repeats work: for fixed input packets $(a_1, \dots, a_r)$, every target block begins with the same directional derivative. The execution layer compiles that shared work, the snapshot layer freezes one multilinear block into finite coefficient data, and the streaming layer moves the same data through a bounded-memory directory format.

For a first run, follow the execution ladder in Section 43.5.4.1 and use the small worked plan in Section 43.1.4. Introduce persistence after the direct and planned evaluations agree. Introduce profiling after the workload is fixed. This makes a failed computation, a storage problem, and a timing comparison easier to diagnose independently.

The chapter covers five different kinds of statement. They should not be collapsed into one performance claim:

- execution-plan and complexity fields are exact finite counts.
- Reconstruction certificates compare one retained packet sum with the direct derivative supplied by the same provider.
- Snapshots and streams are finite persisted coefficient records.
- Profile timings and Python allocation peaks are local observations.
- CuPy classes are optional numerical paths whose availability and parity must be established on the machine that executes them.

Most of the CPU execution, snapshot, stream, and profile names in this chapter are part of the curated `komorebi_math.beta` surface in version `1.0.0b4`. They are also available from `komorebi_math.nonlinear` and, for compatibility, many are re-exported at the package root. The CuPy execution names remain on the research surface. None of these names belongs to the smaller frozen `komorebi_math.stable` surface.

The nonlinear feature record is obtained with

```
from komorebi_math import nonlinear

assert nonlinear.FEATURE_KEY == "nonlinear"
assert nonlinear.status().status.value == "implemented"
```

That registry status means the finite implementation is present. It does not assert that an optional CUDA runtime exists or that an arbitrary derivative callable is correct.

43.1 Projection-cached execution

Let the source packet system contain A projectors P_a, the target system contain B projectors Q_b, and the derivative order be r. The complete address family has

$$N_{\text{full}} = BA^r$$

members. For one ordered source tuple $\mathbf{a} = (a_1, \dots, a_r)$, define

$$w_{\mathbf{a}} = D^r F(x)[P_{a_1} v_1, \dots, P_{a_r} v_r].$$

Every active target block with those same inputs is then obtained by the cheap final projection

$$J_{b;\mathbf{a}}(x)[v_1, \dots, v_r] = Q_b w_{\mathbf{a}}.$$

`compile_packet_jet_plan` groups active addresses by the **ordered** tuple **a**. If G is the set of tuples represented by at least one active target address, the planned provider-call count is

$$N_{\text{calls}}^{\text{plan}} = |G|, \qquad N_{\text{calls}}^{\text{naive}} = BA^r.$$

The input order is not symmetrized during grouping. Even when the derivative is mathematically symmetric, `(left, right)` and `(right, left)` remain distinct groups because packet projection is attached to a direction slot.

43.1.1 Conservative support pruning

With no support record, every address stays active. With a supplied `PacketJetSupport`, compilation first requires its certificate, checks that its order agrees with the family, and checks that `nonzero`, `zero`, and `untested` partition the complete family address set. Only addresses in the certified `zero` tuple are removed:

$$\mathcal{A}_{\text{pruned}} \subseteq \mathcal{A}_r^0.$$

An `untested` block remains active. Consequently, an exhausted support search may reduce pruning efficiency, but it cannot justify dropping an untested contribution. `support_complete` records whether any such unresolved addresses remain.

The address ceiling is checked before compilation. For

```
compile_packet_jet_plan(
    family: PacketJetFamily,
    support: PacketJetSupport | None = None,
    *,
    max_addresses: int = 1_000_000,
) -> PacketJetExecutionPlan
```

`max_addresses` must be a positive, non-Boolean integer. The ceiling controls address enumeration. It is not a time or memory guarantee for the derivative provider itself.

43.1.2 Plan and group records

`PacketJetInputGroup(inputs, outputs)` is a frozen slotted record. `inputs` is the ordered tuple of source-packet indices. `outputs` must be a nonempty, sorted tuple of unique nonnegative target indices.

`PacketJetExecutionPlan` has these stored fields:

- `order`, `source_names`, and `target_names`.
- `groups: tuple[PacketJetInputGroup, ...]`.
- `active_addresses` and `pruned_addresses`.
- `support_was_supplied` and `support_complete`.
- `certificate`.

Its derived properties are:

- `active_address_count` and `pruned_address_count`.
- `full_address_count`.
- `derivative_call_count` and `naive_derivative_call_count`.
- `derivative_call_reduction`.

The last is

$$1 - \frac{N_{\text{calls}}^{\text{plan}}}{N_{\text{calls}}^{\text{naive}}},$$

with value one when the denominator is zero. `to_dict()` returns a report mapping.

The method

```
plan.matches(family: PacketJetFamily) -> bool
```

compares only derivative order, source packet names, and target packet names. It does **not** fingerprint the provider, point, packet projector matrices, backend, tolerance, or support proof. A plan is therefore a structural cache, not a self-authenticating executable artifact. Recompile it whenever any semantic input changes.

43.1.3 Execution and reconstruction

The two CPU entry points are

```
execute_packet_jet_plan(
    plan: PacketJetExecutionPlan,
    family: PacketJetFamily,
    directions: Sequence[Any],
    *,
    tolerance: float | None = None,
) -> PacketJetBatchResult

execute_packet_jets(
    family: PacketJetFamily,
    directions: Sequence[Any],
    *,
    support: PacketJetSupport | None = None,
    tolerance: float | None = None,
    max_addresses: int = 1_000_000,
) -> PacketJetBatchResult
```

The convenience form compiles and executes. The explicit form first requires the plan certificate and calls `plan.matches(family)`. Every direction is then converted through the source packet backend and projected once through every source packet. For each group, the provider is called once and the result is projected through the listed target packets.

If $\mathcal{A}_{\text{active}}$ is the retained address set, execution forms

$$\widetilde{D}^r F(x)[V] = \sum_{(b;\mathbf{a})\in\mathcal{A}_{\text{active}}} Q_b D^r F(x)[P_{a_1} v_1, \dots, P_{a_r} v_r]$$

and independently asks the same provider for

$$D^r F(x)[v_1, \dots, v_r].$$

The certificate residual is

$$\rho_{\text{plan}} = \left\| D^r F(x)[V] - \widetilde{D}^r F(x)[V] \right\|.$$

For exact families it must be zero. For numerical families it must not exceed the explicit tolerance, or the maximum source/target packet tolerance when no override is supplied. The certificate theorem identifier is `PSC-III-A10`. It checks the finite reconstruction on the supplied directions. Global correctness of the derivative provider remains unchecked.

`PacketJetEvaluation(address, value)` stores one active address result. `PacketJetBatchResult` stores:

- `evaluations`, `reconstructed`, and `direct`.
- `derivative_calls`, `input_projections`, and `output_projections`.
- `pruned_blocks`, `backend`, and `returned_to_host`.
- `certificate`.

`as_mapping()` creates an address-to-value dictionary. `value(address)` returns one active value and raises `KeyError` for a pruned or otherwise inactive address. `to_dict()` expands the complete report.

43.1.4 Numerical worked plan with exact structural counts

The repository example `examples/scalable_nonlinear.py` builds an exact quadratic provider with coordinate packets. The packet system uses the default NumPy backend, so execution values and certificates are numerical even though the provider coefficients and structural address counts are exact. The full order-two family contains eight blocks. Certified support leaves two:

```
from fractions import Fraction

import numpy as np

from komorebi_math import Exactness, PacketSystem
from komorebi_math.nonlinear import (
    PacketJetFamily,
    PolynomialDerivativeProvider,
    compile_packet_jet_plan,
    execute_packet_jet_plan,
    packet_jet_support,
)

tensor = np.zeros((2, 2, 2), dtype=object)
tensor[0, 0, 0] = Fraction(2)
tensor[1, 1, 1] = Fraction(-3)
provider = PolynomialDerivativeProvider(
    {2: tensor},
    exactness=Exactness.EXACT,
)
packets = PacketSystem(
    (
        np.diag([Fraction(1), Fraction(0)]),
        np.diag([Fraction(0), Fraction(1)]),
    ),
    names=("x", "y"),
)
family = PacketJetFamily(
    provider,
    np.array([Fraction(0), Fraction(0)], dtype=object),
    packets,
    packets,
    order=2,
)
support = packet_jet_support(family)
plan = compile_packet_jet_plan(family, support)

directions = (
    np.array([Fraction(2), Fraction(3)], dtype=object),
    np.array([Fraction(-1), Fraction(4)], dtype=object),
)
result = execute_packet_jet_plan(plan, family, directions)
result.certificate.require()

print(plan.active_address_count, plan.pruned_address_count)
```

```
print(result.derivative_calls)
print(result.reconstructed.tolist())
```

Observed output:

```
2 6
2
[(-4+0j), (-36+0j)]
```

The complex-looking scalars are NumPy's representation after backend normalization. They are numerically equal to integer values, but the resulting family, support, plan, and execution certificate have `Exactness.NUMERICAL`. The two calls are one per active input group. The example does not imply that every provider call has equal cost.

43.1.5 Complexity records

`estimate_packet_jet_complexity` converts a finite family and plan into a deterministic count record:

```
estimate_packet_jet_complexity(
    family: PacketJetFamily,
    support: PacketJetSupport | None = None,
    *,
    plan: PacketJetExecutionPlan | None = None,
    max_addresses: int = 1_000_000,
) -> PacketJetComplexityEstimate
```

When no plan is supplied, the function compiles one under `max_addresses`. When a plan is supplied, it requires that plan's certificate and uses the same name-and-order `matches` check described above.

For source dimension d_X, target dimension d_Y, source packet count A, target packet count B, and order r, the record includes

$$
\begin{aligned}
N_{\text{family}} &= BA^r, \\
N_{\text{basis}} &= d_X^r, \\
N_{\text{dense}} &= d_Y d_X^r, \\
N_{\text{input proj}} &= rA.
\end{aligned}
$$

Its fields are:

- `order`, `source_dimension`, and `target_dimension`.
- `source_packets`, `target_packets`, and `family_addresses`.
- `active_addresses`, `pruned_addresses`, and `input_groups`.
- `naive_derivative_calls` and `planned_derivative_calls`.
- `projected_input_vectors` and `output_projections`.
- `dense_basis_evaluations` and `dense_tensor_elements`.
- `certificate`.

`derivative_call_reduction` is a derived property and `to_dict()` produces the report mapping. The `PSC-III-A61` count certificate has `Exactness.EXACT`. That exactness applies to these structural counts, not to a runtime estimate. It leaves call cost, kernel fusion, caching, and transfer overhead unchecked.

For the three-coordinate sparse quadratic in the shipped example below,

```
examples/scalable_nonlinear_runtime.py
```

the observed ledger was:

```
active packet blocks: 4
pruned packet blocks: 23
planned derivative calls: 4
naive derivative calls: 27
```

The reduction from 27 to 4 provider calls is exact for that plan. A faster wall clock still depends on provider cost, projection cost, allocation, backend, and workload size.

43.2 Packet-jet snapshots and digest identity

A live `PacketJet` contains Python callables and packet-system objects. It is not reconstructed from JSON. `snapshot_packet_jet` instead evaluates one fixed packet block on every ordered ambient standard-basis tuple and stores the resulting coefficients.

For source basis $e_0, \dots, e_{d_X-1}$, address $\alpha = (b; a_1, \dots, a_r)$, and target coordinate o, define

$$c_{o;i_1,\dots,i_r} = \left(J_\alpha[e_{i_1}, \dots, e_{i_r}]\right)_o .$$

The frozen block evaluates as

$$J_\alpha[v_1, \dots, v_r]_o = \sum_{i_1,\dots,i_r} c_{o;i_1,\dots,i_r} \prod_{s=1}^{r} (v_s)_{i_s}.$$

The capture signature is

```
snapshot_packet_jet(
    jet: PacketJet,
    *,
    tolerance: float | None = None,
    max_evaluations: int = 100_000,
) -> PacketJetSnapshot
```

The safety check $d_X^r \le$ `max_evaluations` occurs before any basis evaluation. For exact backends, only coefficients exactly recognized as zero are omitted. For numerical backends, a coefficient vector is omitted when its norm does not exceed the selected tolerance. The capture certificate records the largest omitted norm. Thus numerical sparsification is a declared tolerance operation, not exact support discovery.

43.2.1 Snapshot records and methods

`PacketJetCoefficient(indices, value)` stores one input-index tuple and one target-coordinate tuple. The enclosing snapshot validates order, range, duplicates, target length, and lexicographically sorts the entries.

`PacketJetSnapshot` fields are:

- `address`, `source_dimension`, and `target_dimension`.
- `provider_name`, `exactness`, and `point`.
- `coefficients` and `capture_certificate`.

The derived properties `order`, `nnz`, and `dense_coefficient_count` return r, the stored coefficient-vector count, and d_X^r, respectively. Public methods are:

```
coefficient(indices: Sequence[int]) -> tuple[Any, ...]
evaluate(directions: Sequence[Any]) -> ndarray
to_dense(*, max_elements: int = 10_000_000) -> ndarray
comparison_certificate(
    jet: PacketJet,
    directions: Sequence[Any],
```

```
    *,
    tolerance: float | None = None,
) -> Certificate
to_dict() -> dict[str, Any]
```

`coefficient` returns an explicit zero target vector for an unstored valid index. `evaluate` and the `__call__` alias contract the sparse coefficients. `to_dense` enforces

$$d_Y d_X^r \leq \texttt{max_elements}$$

before allocation. `comparison_certificate` checks a matching address and dimensions, evaluates one supplied direction tuple through the live jet and snapshot, and records residual `snapshot_reconstruction` under theorem `PSC-III-D58`. It is sampled finite parity, not recovery of the live callable.

43.2.2 Portable envelope, digest, and adjacent migration

The portable codec for `PacketJetSnapshot` is version 2 under this tag:

```
nonlinear.packet_jet_snapshot
```

Version 2 names the provider field `provider` and records

```
coefficient_ordering = "lexicographic"
```

before decoding. The one committed adjacent migration transforms version 1 to version 2 by renaming `provider_name` to `provider` and adding the ordering field. A future version, a missing migration link, an unsupported ordering, an unknown codec, duplicate JSON keys, nonfinite JSON constants, or a payload digest mismatch fails closed.

The ordinary persistence path is:

```
from komorebi_math.export import dumps_portable, loads_portable, portable_digest

encoded = dumps_portable(snapshot)
restored = loads_portable(encoded)
assert portable_digest(restored) == portable_digest(snapshot)
```

The SHA-256 value binds canonical payload data. It detects an unchanged digest paired with changed content. It is not a signature, message-authentication code, timestamp, ownership record, or proof that the provider used during capture was mathematically correct.

The shipped `portable_nonlinear_state.py` example produced:

```
snapshot address: PacketJetAddress(output='x', inputs=('x', 'x'))
stored coefficients: 1
portable digest: dae6206f9b99acfa72649825253fd0b748c1e62236109f0a120872767ad97033
evaluation: [(-6+0j), 0j]
tampered payload: rejected
```

That digest is reproducible for this release and example data. It should not be used as a universal digest for all packet-jet snapshots.

43.3 Resumable streams and adjacent migrations

The ordinary portable envelope requires a complete finite snapshot in memory. The stream format stores coefficients in independently digested JSON chunks, commits progress after each completed chunk, and lets readers decode one chunk at a time.

The format constants are:

```
PACKET_JET_STREAM_SCHEMA = "komorebi-math.packet-jet-stream"
PACKET_JET_STREAM_SCHEMA_VERSION = 1
PACKET_JET_CHUNK_SCHEMA = "komorebi-math.packet-jet-stream.chunk"
PACKET_JET_CHUNK_SCHEMA_VERSION = 1
```

`StreamState = Literal["in_progress", "complete"]` and `CaptureMode = Literal["direct", "snapshot"]` are module-visible typing aliases in `komorebi_math.nonlinear.streaming`. They are not exported from the package-level nonlinear namespace. These layout constants are likewise implementation-visible rather than package exports:

- `STREAM_MANIFEST_NAME`.
- `STREAM_CHUNK_DIRECTORY`.
- `STREAM_PARTIAL_SUFFIX`.

For a destination named

```
run.kmstream
```

an interrupted write lives at

```
run.kmstream.partial
```

The directory contains:

- `manifest.json`.
- A `chunks` directory.
- Canonical members such as `chunk-00000000.json` and the successive numbered files.

The state transition is

$$\text{absent} \longrightarrow \texttt{destination.partial} \longrightarrow \text{verified staging} \longrightarrow \texttt{destination}.$$

Manifest and chunks each use the envelope

$$E = (\text{schema}, \text{schema version}, \text{SHA256}(\text{canonical payload}), \text{payload}).$$

Files are written to a same-directory temporary path, flushed, `fsync`ed, and atomically replaced. Promotion uses an operating-system replace operation. An explicit overwrite first moves an existing destination to a temporary backup and restores it if promotion fails. These steps limit torn local updates. They do not turn a multi-file directory into authenticated or adversary-resistant storage.

43.3.1 Direct capture, pause, and resume

```
capture_packet_jet_stream(
    jet: PacketJet,
    path: str | os.PathLike[str],
    *,
    tolerance: float | None = None,
    max_evaluations: int = 10_000_000,
    chunk_size: int = 1024,
    evaluation_budget: int | None = None,
    resume: bool = False,
    overwrite: bool = False,
) -> PacketJetStreamWriteResult
```

`max_evaluations` is a total safety ceiling. `evaluation_budget` is different: it limits only the additional basis evaluations performed by this call. A zero budget is valid and leaves an empty but structurally valid partial stream. If the budget

ends before d_X^r basis tuples have been visited, the write result is incomplete and the staging directory remains in place.

Resume verifies every committed prefix chunk and requires the same chunk size and metadata fingerprint. For direct capture that fingerprint contains

$$(\alpha, d_X, d_Y, n_F, e_F, x, \varepsilon),$$

where α is the packet address, n_F the provider name, e_F the exactness label, x the point, and ε the capture tolerance. It does **not** contain callable bytecode, closure state, packet projector matrices, package source, or a semantic provider hash. Resuming a live provider therefore assumes that all unrecorded behavior is unchanged.

When all basis tuples are complete, the final manifest gains a capture certificate, is reloaded and validated, and the staging directory is promoted. An incomplete write certificate is valid as a progress record but has `Exactness.CONDITIONAL`, residual `incomplete_stream = 1.0`, and an unchecked remaining-work assumption. Do not treat `certificate.valid` alone as proof that the stream is complete. Inspect `completed` or `manifest.complete`.

43.3.2 Streaming an existing snapshot

```
save_packet_jet_snapshot_stream(
    snapshot: PacketJetSnapshot,
    path: str | os.PathLike[str],
    *,
    chunk_size: int = 1024,
    coefficient_budget: int | None = None,
    resume: bool = False,
    overwrite: bool = False,
) -> PacketJetStreamWriteResult
```

Snapshot mode traverses the already stored nonzero coefficients, so `coefficient_budget` limits additional stored coefficient vectors rather than dense basis evaluations. Its resume fingerprint includes the complete ordered coefficient record and capture certificate. This is stronger than the direct callable fingerprint, although it remains an integrity identity rather than an authenticity proof.

Snapshot chunks initially carry coefficient progress. On finalization the last chunk is rewritten to close the complete dense-basis interval. Even a snapshot with zero stored coefficients receives a canonical empty chunk and can be round-tripped as complete.

43.3.3 Manifest and result records

`PacketJetStreamChunkRecord` stores:

- `index`, canonical `filename`, and `byte_count`.
- Half-open `basis_start:basis_stop`.
- Half-open `coefficient_start:coefficient_stop`.
- Lowercase `payload_sha256`.

Its `basis_count` and `coefficient_count` properties subtract the corresponding endpoints. `to_dict()` and `from_dict()` use an exact closed field schema.

`PacketJetStreamManifest` stores:

- `state`, `producer_version`, `capture_mode`, and `chunk_size`.
- Packet address, dimensions, provider name, exactness, point, and tolerance.
- `dense_coefficient_count` and `source_digest`.
- `metadata_sha256` and `stream_sha256`.
- Completed basis and coefficient counts and maximum omitted basis norm.
- The ordered chunk records and optional final capture certificate.

Its `complete` and `order` properties expose state and address order. `metadata_dict()`, `to_payload()`, and `from_payload()` enforce the closed version-1 schema. Chunk indices and ranges must be contiguous. A complete manifest must cover all d_X^r basis tuples and contain a capture certificate. An in-progress manifest must not contain that final certificate.

`PacketJetStreamWriteResult` reports:

- `destination` and `active_path`.
- `completed` and `resumed`.
- Chunk, evaluation, and coefficient counts written by the current call.
- Final `manifest` and `certificate`.

A completed result's active path is the destination. An incomplete result's active path is the `.partial` staging directory.

The stream schema is currently version 1 and has no stream-format migration path. The adjacent version-1-to-version-2 migration described in §43.2.2 is for the ordinary `nonlinear.packet_jet_snapshot` portable codec, not for a stream manifest or chunk. Stream schema/version mismatches are rejected.

43.3.4 Bounded reader, selections, and partial sums

The reader entry points are:

```
PacketJetSnapshotStreamReader(
    path,
    *,
    allow_incomplete: bool = False,
    verify_chunks: bool = False,
)

open_packet_jet_snapshot_stream(
    path,
    *,
    allow_incomplete: bool = False,
    verify_chunks: bool = False,
) -> PacketJetSnapshotStreamReader

load_packet_jet_snapshot_stream(
    path,
    *,
    max_coefficients: int = 1_000_000,
    verify_chunks: bool = True,
) -> PacketJetSnapshot
```

By default, opening an incomplete stream fails. With `allow_incomplete=True`, the reader may resolve the sibling `.partial` directory. `verify_chunks=True` immediately walks and requires the full stream integrity certificate.

Reader methods are:

```
iter_chunks()
iter_coefficients(*, start=0, stop=None)
select(*, start=0, stop=None, max_coefficients=100_000)
evaluate(directions, *, start=0, stop=None)
load_snapshot(*, max_coefficients=1_000_000)
verify()
```

Iteration holds at most one decoded chunk at a time. Selection and evaluation ranges are half-open ordinals in the **stored coefficient sequence**, not dense basis ordinals. If stored coefficients are $(c_0, \dots, c_{m-1})$, range $[a, b)$ evaluates

$$J_{[a,b)}[v_1, \dots, v_r] = \sum_{k=a}^{b-1} c_k \prod_{s=1}^{r} (v_s)_{i_s(k)}.$$

This is the complete snapshot value only when the stream is complete and $[a, b) = [0, m)$. Otherwise the certificate explicitly leaves coefficients outside the selected interval unchecked.

`PacketJetStreamSelection` stores the ordinal bounds, loaded coefficient tuple, `complete_snapshot`, and certificate. `StreamedPacketJetEvaluation` stores the result vector, bounds, total stored coefficient count, `complete_snapshot`, and certificate. `load_snapshot` refuses incomplete streams and enforces its materialization guard before constructing a conventional in-memory `PacketJetSnapshot`.

`verify()` checks manifest, metadata, and stream digests, every chunk envelope and manifest cross-reference, contiguous ranges, exact observed counts, and strict lexicographic coefficient order without duplicates. Its theorem identifier is `PSC-III-D60`. The certificate's unchecked assumptions state plainly that SHA-256 integrity does not establish authenticity or authorship.

43.3.5 Resume excerpt from the shipped example

The shipped example `examples/resumable_packet_jet_stream.py` pauses a quadratic block after two of four basis evaluations, resumes it, verifies every chunk, and evaluates the completed stream. Use this as the central resume excerpt. It assumes the shipped file's preceding imports and its definitions of `jet` and `directions`:

```
with tempfile.TemporaryDirectory() as directory:
    destination = Path(directory) / "quadratic.kmstream"
    partial = capture_packet_jet_stream(
        jet,
        destination,
        chunk_size=1,
        evaluation_budget=2,
    )
    complete = capture_packet_jet_stream(
        jet,
        destination,
        chunk_size=1,
        resume=True,
    )
    reader = open_packet_jet_snapshot_stream(
        destination,
        verify_chunks=True,
    )
    evaluation = reader.evaluate(directions)
    evaluation.certificate.require()

    print(partial.manifest.evaluations_completed)
    print(complete.resumed, len(reader.manifest.chunks))
    print(reader.manifest.coefficients_completed)
    print(evaluation.value.tolist())
```

Observed output:

```
2
True 4
4
[53.0]
```

The example uses a temporary directory. Applications should retain the `.partial` directory between processes if they intend to resume later.

43.4 Cost profiles and budget planners

The profiling module keeps deterministic counts separate from observations. `RuntimeProfileSample(duration_ns, traced_peak_bytes)` is one nonnegative pair. `profile_operation` returns a nonempty `RuntimeStageProfile(name, scope, warmup, samples)` and the record exposes:

- `repetitions`.
- Minimum, median, and maximum duration in nanoseconds.
- Median and maximum traced peak bytes.
- `to_dict()`.

The low-level observer is

```
profile_operation(
    name: str,
    operation: Callable[[], T],
    *,
    repetitions: int = 5,
    warmup: int = 1,
    scope: str =
        "Python call including allocations and synchronization performed by the callable",
) -> RuntimeStageProfile
```

`T` is a typing variable inside `nonlinear.profiling`. It is not a runtime operation exported by the nonlinear package. `repetitions` must be a positive integer, `warmup` a nonnegative integer, and both name and scope nonempty. Warmup results are discarded. Before each measured call, the implementation collects Python garbage, then records `time.perf_counter_ns()` and the `tracemalloc` peak. An already active `tracemalloc` session is preserved.

The complete packet runtime helper is

```
profile_packet_jet_runtime(
    family: PacketJetFamily,
    directions: Sequence[Any],
    *,
    support: PacketJetSupport | None = None,
    jet: PacketJet | None = None,
    repetitions: int = 5,
    warmup: int = 1,
    max_addresses: int = 1_000_000,
    max_snapshot_evaluations: int = 100_000,
    include_stream_write: bool = False,
    stream_chunk_size: int = 1024,
) -> PacketJetProfileReport
```

It always profiles:

- `address_enumeration`.
- `plan_compilation`.
- `plan_execution`.

When `jet` is supplied, it also profiles:

- `snapshot_capture`.
- `portable_serialization`.

When both a jet and stream-write inclusion are supplied, it adds `stream_snapshot_write` in a fresh temporary directory. The stream stage includes JSON encoding, per-chunk digests, flushes, manifest updates, and promotion. It does not profile direct live-provider resume.

`PacketJetProfileReport` contains the exact `estimate`, stage tuple, environment mapping, and a heuristic certificate. The environment records Python version and implementation, platform and processor strings, timer, memory observer, repetition count, and warmup. The certificate theorem is `PSC-III-A61`, with `Exactness.HEURISTIC`. It leaves

scheduler isolation, CPU frequency, cache state, thermals, cross-machine comparison, and native/device allocation visibility unchecked.

`tracemalloc` observes Python-managed allocations. It does not measure all NumPy buffers, linked-library workspaces, process resident memory, CUDA allocations, or device peaks. A callable that performs asynchronous device work must synchronize inside the measured scope if completion time is intended. The profiler does not insert a backend-specific synchronization.

43.4.1 Benchmark contracts are specifications

```
packet_jet_benchmark_contract(
    family: PacketJetFamily,
    *,
    backend: str,
    timing_scope: str = "kernel",
) -> BenchmarkContract
```

This function creates a matched workload description containing order, dimensions, packet counts, address count, provider name, expected output, timing scope, and backend. It does not run a benchmark. In particular,

$$\text{benchmark contract} \neq \text{speedup certificate}.$$

A defensible CPU/GPU comparison should use the same family, directions, precision, output contract, warmup, repetition policy, synchronization rule, and transfer scope. Report transfers explicitly rather than hiding them behind the word "kernel."

43.5 Optional GPU execution

There are two separate CuPy paths. `nonlinear.execution` evaluates a live device derivative provider together with packet projectors. `nonlinear.gpu` evaluates a finite persisted snapshot. Neither path silently falls back to CPU.

`nonlinear_cupy_available()` calls `importlib.util.find_spec("cupy")`. It answers only whether a module can be discovered. The availability ladder is

$$\begin{gathered}\text{discoverable}\\ \Downarrow ?\\ \text{importable}\\ \Downarrow ?\\ \text{runtime usable}\\ \Downarrow ?\\ N_{\text{device}} > 0\\ \Downarrow ?\\ \text{workload valid}.\end{gathered}$$

Only the later observations establish an executable CUDA path.

43.5.1 Device polynomial provider

`CuPyDeviceDerivativeProvider` is the runtime-checkable protocol consumed by the executor. A conforming object supplies `source_dimension`, `target_dimension`, `max_order`, `exactness`, `name`, and `device_backend`, together with:

```
supports_order(order: int) -> bool
device_derivative(
    order: int,
    point: Any,
    directions: Sequence[Any],
) -> Any
```

The preferred package-level spelling is

```
CuPyPolynomialDerivativeProvider(
    tensors: Mapping[int, Any],
    *,
    source_dimension: int | None = None,
    target_dimension: int | None = None,
    tolerance: float = 1e-10,
    name: str = "CuPy polynomial derivative provider",
)
```

`CupyPolynomialDerivativeProvider` is the original compatibility spelling in `komorebi_math.nonlinear.execution`. It is not imported into the package-level nonlinear namespace. Both names denote the same class in that module.

The same module retains these original spellings:

```
CupyDeviceDerivativeProvider
CupyPacketJetExecutor
```

Their preferred `CuPy*` aliases are the names re-exported from the nonlinear package namespace.

Construction first validates the tensors through the CPU polynomial derivative provider, rejects object-dtype tensors, transfers all dense tensors to the active CuPy device, and fixes `exactness = Exactness.NUMERICAL`. Its public members are:

- `degree` and `supports_order`.
- `device_value` and `value`.
- `device_derivative` and `derivative`.
- `synchronize`.

The `device_*` forms retain CuPy arrays. `value` and `derivative` return NumPy host arrays.

The device derivative implements the same factorial tensor convention as the CPU polynomial provider:

$$D^r F(x)[v_1, \dots, v_r] = \sum_{m=r}^{M} \frac{1}{(m-r)!} T_m[v_1, \dots, v_r, x^{m-r}].$$

43.5.2 Device packet executor

```
CuPyPacketJetExecutor(
    family: PacketJetFamily,
    plan: PacketJetExecutionPlan | None = None,
)
```

The family must use NumPy packet systems for both source and target and its provider must satisfy the runtime-checkable device-provider protocol. Projector matrices and the base point are copied to the active CuPy device at construction. A missing plan is compiled on CPU. A supplied plan uses the same structural `matches` boundary as CPU execution.

```
executor.execute(
    directions,
    *,
    return_host: bool = True,
    input_on_device: bool = False,
    tolerance: float | None = None,
) -> PacketJetBatchResult
```

With `input_on_device=False`, directions are explicitly converted by `cupy.asarray`. With `return_host=True`, active values, reconstructed value, and direct derivative are copied back to NumPy. With `return_host=False`, those values remain device arrays inside the ordinary batch record. The certificate uses theorem `PSC-III-A10G` and residual `device_reconstruction`. It checks device reconstruction against the direct **device** derivative, not against a CPU provider.

`synchronize()` waits on the null stream. `hardware_certificate()` queries the active device and returns its identifier, name, compute capability, and global memory. That certificate proves that the metadata query succeeded. It does not certify speed, thermal state, numerical parity, or suitability for a larger workload.

43.5.3 Optional CuPy snapshot execution

`cupy_runtime_status()` is the non-raising runtime probe. It returns

```
CuPyRuntimeStatus(
    installed: bool,
    runtime_available: bool,
    device_count: int,
    runtime_version: int | None,
    error: str | None = None,
)
```

When CuPy is undiscoverable, `installed` and `runtime_available` are false and the error says that CuPy is not installed. An import or CUDA query failure is captured as an error string. A successful import with zero devices remains `runtime_available=False`.

```
CuPySnapshotExecutor(
    snapshot: PacketJetSnapshot,
    *,
    dtype: Any | None = None,
)
```

`CuPyPacketJetSnapshotExecutor` is an exact alias for this class. Construction transfers an `nnz x order` integer index matrix and an `nnz x target_dimension` value matrix. `evaluate_batch` requires shape

$$(N_{\text{batch}}, r, d_X)$$

and returns a NumPy array of shape (N_{batch}, d_Y). A two-dimensional (r, d_X) input is promoted to a one-item batch. The kernel gathers each direction coordinate, multiplies the r gathered factors, then performs the matrix product with stored target values. `evaluate` and `__call__` handle one ordered direction tuple.

Matched CPU/device parity is requested with

```
validate_cupy_snapshot(
    snapshot: PacketJetSnapshot,
    probes: Sequence[Sequence[Any]],
    *,
    tolerance: float = 1e-9,
) -> CuPyValidationResult
```

The result fields are `probe_count`, `maximum_residual`, `runtime`, and `certificate`. The numerical `PSC-III-N59` certificate checks

$$\rho_{\mathrm{GPU}} = \max_j \left\| J_{\mathrm{GPU}}[V_j] - J_{\mathrm{CPU}}[V_j] \right\|_2 \le \varepsilon.$$

It leaves behavior outside the probes and all performance conclusions unchecked. A CPU-only system emits an unavailable status or raises `BackendUnavailableError` when the executor is constructed. It never emits a false hardware pass.

The recorded build machine reported:

```
CuPy installed: False
CuPy runtime available: False
CUDA devices: 0
```

That is an honest no-runtime observation, not a negative statement about a different machine.

43.5.4 Example ladder, resource model, and claim boundaries

This closing execution section orders the recommended workflow before stating its arithmetic model, failure surface, implementation boundaries, and replayed evidence.

43.5.4.1 Recommended execution ladder

For a new finite workload:

1. Build and test the derivative provider independently.
2. Construct the packet family and discover support under an explicit probe budget.
3. Compile a fresh plan from that exact family and support record.
4. Inspect exact counts before execution.
5. Execute and require the reconstruction certificate on representative directions.
6. Snapshot a fixed block only when d_X^r is within the basis budget.
7. Use the portable envelope for a comfortably in-memory snapshot, or the stream format for chunked persistence.
8. Profile only after the workload, timing scope, warmup, synchronization, and output contract are fixed.
9. Enter the CuPy path only after a runtime probe, then compare matched CPU and device outputs.

This ordering separates mathematical correctness, integrity, resource safety, observational profiling, and optional hardware qualification.

43.5.4.2 Arithmetic and storage model

These are dense reference counts. Provider-specific work may dominate.

- Full packet address enumeration stores and visits BA^r addresses.
- Plan execution forms rA input projections, makes $|G|$ derivative calls, and applies one target projection per active address.
- With dense $d_X \times d_X$ and $d_Y \times d_Y$ projectors, projection work is approximately $O(rAd_X^2 + N_{\mathrm{active}} d_Y^2)$, in addition to provider calls.
- A full basis snapshot makes d_X^r jet evaluations. It stores at most d_X^r coefficient vectors, or $d_Y d_X^r$ scalar values.
- Dense snapshot expansion stores $d_Y d_X^r$ elements.
- A stream has the same total coefficient payload plus JSON, digest, manifest, and per-chunk overhead. Iteration retains one decoded chunk, though a selected range or materialized snapshot additionally stores the selected result.
- A CuPy snapshot stores `nnz * r` integer indices and `nnz * d_Y` device values. Batched contraction additionally forms an approximately `batch * nnz` factor matrix.
- A live CuPy packet executor stores every dense source and target projector on device as well as the provider tensors and base point.
- Profiling multiplies the selected stage work by warmups plus repetitions. Stream profiling also rewrites a fresh temporary stream for every call.

No function in this chapter estimates free disk space, device-memory capacity, thermal throttling, filesystem quota, or provider-internal allocations. The `max_*` parameters are local guards, not global schedulers.

43.5.4.3 Failure surface

Expected package failures include:

- `ValidationError` for a nonpositive address, evaluation, element, coefficient, chunk, repetition, or selection ceiling.
- `ValidationError` for an uncertified or nonpartitioning support record, plan name/order mismatch, wrong direction count, or failed plan certificate.
- `DimensionError` for direction, derivative, point, coefficient, or projector dimension mismatch.
- `KeyError` when requesting a batch value at an inactive address.
- `ValidationError` for duplicate snapshot coefficient indices, out-of-range indices, or dense expansion above its guard.
- `ValidationError` for an existing destination without overwrite, a partial destination without resume, a changed resume fingerprint or chunk size, incomplete-as-complete reading, selection outside the stored range, or materialization above its guard.
- `ValidationError` for malformed JSON, duplicate keys, nonfinite constants, closed-schema field drift, unsupported stream schema/version, noncontiguous ranges, changed chunk bytes, or changed manifest content.
- `BackendUnavailableError` when a promised CuPy execution path cannot be constructed.
- `DimensionError` for a malformed CuPy batch or non-NumPy packet systems in the live CuPy executor.

Portable snapshot migration failures and stream failures are distinct. The former report codec-version or migration-path errors. The latter reject any schema other than the current stream version.

43.5.4.4 Source-observed implementation boundaries

These behaviors were reproduced from the `1.0.0b4` source. They are boundaries to design around, not compatibility promises:

- `PacketJetExecutionPlan.matches` fingerprints only order and packet names. A changed provider, point, projector matrix, backend, tolerance, or same-name support semantics can pass `matches`. Reconstruction often detects stale pruning on the supplied directions, but a zero probe need not. Recompile.
- A supplied plan makes `estimate_packet_jet_complexity` ignore the supplied `support` object except for a diagnostic flag. It does not prove that the plan was compiled from that support record.
- `profile_packet_jet_runtime` checks only that an optional snapshot `jet` has the same order as the family. It does not require the same point, address, provider, or packet systems. Treat the caller as responsible for workload identity.
- Direct stream resume fingerprints provider name and structural metadata, not callable semantics or projector matrices. Changing a same-named provider between calls can mix incompatible chunks while satisfying the recorded fingerprint.
- `PacketJetSnapshot.comparison_certificate` checks address and dimensions but not provider name, exactness, base point, or coefficient provenance. Its one supplied probe can pass accidentally.
- The direct snapshot's `to_dict()` leaves `capture_certificate` as a `Certificate` object. Use the registered portable codec rather than assuming this report mapping is directly JSON serializable.
- `input_on_device=True` in `CuPyPacketJetExecutor.execute` is a caller declaration, not a strict array-origin validator. Passing host objects can undermine a transfer-excluding timing claim.
- The live CuPy provider checks module discovery before importing CuPy. A discoverable but broken installation can fail during import rather than producing the non-raising status record. Call `cupy_runtime_status()` first.
- Exact snapshots sent through `CuPySnapshotExecutor` are converted to the requested numerical device dtype. The GPU result is numerical even when the source coefficient record had `Exactness.EXACT`.

No certificate here proves provider semantics, cryptographic authorship, portable speedup, global sparsity, asymptotic complexity improvement, device stability, or behavior outside the supplied finite probes.

43.5.4.5 Verification evidence

In the recorded verification, the six shipped examples `scalable_nonlinear.py`, `scalable_nonlinear_runtime.py`, `portable_nonlinear_state.py`, `resumable_packet_jet_stream.py`, `filtered_transport.py`, and `kato_feshbach.py` all completed from the recovered source tree. A focused set of execution, persistence, stream, migration, profiling, nonlinear transport, and quantum transport tests reported:

```
66 passed in 3.90s
```

The environment was Python `3.12.13` with NumPy `2.5.2`. CuPy was unavailable, so the guide locally verified the explicit unavailable path but made no CUDA hardware claim.

> **Chapter 43 summary.** Compile active blocks by ordered input packet tuple, prune only certified zeros, and interpret the reconstruction certificate as parity with the same provider on the supplied directions. Exact count records are not timings. Snapshots freeze coefficients rather than callables. Direct stream resume has only a structural provider fingerprint, while snapshot resume binds the complete coefficient record. Stream digests provide integrity, not authorship. Profile scopes must name synchronization and transfers. CuPy discovery, runtime availability, device execution, CPU parity, and speed are five separate claims.

44. Smooth packet transport

An ordinary packet system fixes a finite orthogonal decomposition. Here, the projectors vary with a real parameter t. The implementation fixes their count and ambient dimension after a reference sample, but it does not separately certify constant rank or continuity. It validates each sampled packet family, compares declared derivatives with centered finite differences, constructs the canonical Kato generator, and offers two finite transport discretizations.

The implementation is numerical and finite-dimensional. Every projector is a dense NumPy complex matrix. There is no symbolic, exact, sparse, batched, or CuPy transport backend in version `1.0.0b4`. Smoothness between sampled times and convergence under indefinite mesh refinement remain mathematical hypotheses, not runtime facts.

A first transport experiment. Begin with the two-packet example in Section 44.4.2. Keep packet order fixed at every time, check the declared derivatives and generator, and then compare the endpoint projectors with the transported initial projectors. Use mesh refinement to investigate numerical sensitivity. A direct rotation between two samples and an evolution along a sampled path answer related but different questions.

All transport symbols in this chapter are version-pinned research APIs in `1.0.0b4`. They are absent from both the curated beta manifest and the frozen stable manifest. Namespace visibility does not grant either stability tier.

44.1 Moving packet systems

Let $P_1(t), \dots, P_m(t) \in \mathbb{C}^{n\times n}$ satisfy

$$P_a(t)^* = P_a(t), \qquad P_a(t)^2 = P_a(t),$$

$$P_a(t)P_b(t) = 0 \quad (a \neq b), \qquad \sum_{a=1}^{m} P_a(t) = I_n.$$

`MovingPacketSystem` requires the projector and derivative callables separately:

```
MovingPacketSystem(
    projectors: Callable[[float], Iterable[Any]],
    derivatives: Callable[[float], Iterable[Any]],
    *,
    tolerance: float = 1e-9,
    reference_time: float = 0.0,
)
```

Construction requires callable inputs, a positive finite tolerance, and a finite reference time. It evaluates the packet family at that reference, validates the complete orthogonal decomposition, and freezes `dimension` and `count`. The callables themselves remain live. No projector samples are cached.

`packets(time)` converts every returned matrix to complex NumPy form and checks that it is finite, square, self-adjoint, idempotent, pairwise orthogonal, and part of a complete identity decomposition under an absolute tolerance and zero relative tolerance. Later calls must preserve the reference dimension and packet count.

`packet_derivatives(time)` requires one finite square derivative matrix per packet with the same ambient dimension. It does not by itself establish that these matrices are derivatives of `packets`.

44.1.1 Differentiated packet identities

Differentiating idempotence gives

$$\dot{P}_a = \dot{P}_a P_a + P_a \dot{P}_a.$$

Differentiating the partition of unity gives

$$\sum_a \dot{P}_a = 0.$$

The sampled validation method is

```
system.derivative_certificate(
    time: float,
    *,
    step: float = 1e-6,
) -> PacketDerivativeResult
```

It forms centered finite differences

$$\dot{P}_a^{\mathrm{FD}}(t;h) = \frac{P_a(t+h) - P_a(t-h)}{2h}$$

and records

$$\rho_{\mathrm{FD}} = \max_a \|\dot{P}_a - \dot{P}_a^{\mathrm{FD}}\|_2,$$

$$\rho_{\tan} = \max_a \|\dot{P}_a - \dot{P}_a P_a - P_a \dot{P}_a\|_2, \qquad \rho_\Sigma = \left\| \sum_a \dot{P}_a \right\|_2.$$

`PacketDerivativeResult` fields are:

- `time` and `step`.
- `maximum_derivative_residual`.
- `tangent_residual` and `partition_derivative_residual`.
- `certificate`.

The numerical `PSC-III-D56` certificate requires all three residuals to be at most

$$\tau_{\mathrm{derivative}} = \max(50\varepsilon, 50h^2),$$

where ε is the system tolerance. This is sampled numerical evidence. It does not prove differentiability on an interval or bound a third derivative.

44.1.2 Canonical generator

For a differentiable complete orthogonal family, define

$$K(t) = \sum_a \dot{P}_a(t) P_a(t).$$

The differentiated identities imply

$$K(t)^* = -K(t).$$

The combined transport identity is

$$K(t) = \sum_a \dot{P}_a(t) P_a(t) = \frac{1}{2} \sum_a [\dot{P}_a(t), P_a(t)], \qquad \dot{U}(t) = K(t) U(t).$$

They also give the equivalent commutator form

$$K(t) = \frac{1}{2} \sum_a [\dot{P}_a(t), P_a(t)]$$

and the transport equation

$$\dot{P}_a(t) = [K(t), P_a(t)].$$

`canonical_generator(time)` evaluates the first formula directly. It returns a matrix even when the declared derivatives are wrong. The separate `generator_certificate(time)` should therefore be required. That certificate records

$$\rho_{\text{skew}} = \|K + K^*\|_2, \qquad \rho_{\text{comm}} = \max_a \|\dot{P}_a - (K P_a - P_a K)\|_2$$

and compares their maximum with 20ε. Its theorem identifier is `PSC-III-T57`, with numerical exactness.

44.2 Kato transport

The canonical unitary transport solves

$$\dot{U}(t) = K(t) U(t), \qquad U(t_0) = I.$$

Because $K^* = -K$, the exact solution is unitary. The commutator equation implies

$$P_a(t) U(t) = U(t) P_a(t_0),$$

so each initial packet range is carried to its corresponding moving range.

`KatoTransport(system)` accepts only a `MovingPacketSystem`. It exposes three finite numerical paths.

44.2.1 Midpoint exponential evolution

```
transport.evolve(
    start: float,
    stop: float,
    *,
    steps: int = 256,
) -> ComputationResult[ndarray]
```

For N positive integer steps and $\Delta = (t_1 - t_0)/N$, the code uses

$$U_{j+1} = \exp\big(\Delta K\big(t_0 + \big(j + \tfrac{1}{2}\big)\,\Delta\big)\big)\, U_j, \qquad U_0 = I.$$

At every midpoint it counts a generator failure when $\|K + K^*\|_2 > 20\varepsilon$. At the endpoint it records

$$\rho_U = \|U_N^* U_N - I\|_2, \qquad \rho_{\text{int}} = \max_a \|P_a(t_1) U_N - U_N P_a(t_0)\|_2.$$

The `PSC-III-T58` certificate is valid only when no sampled generator failure occurred and both endpoint residuals are at most

$$\tau_N = \max(50\varepsilon, 20|\Delta|^2).$$

Its diagnostics record step count and signed step size. `start > stop` is a supported backward integration because Δ may be negative. Equal endpoints return a sampled identity evolution.

The matrix exponential uses `dynamics.matrix_exponential` with its default `method="auto"`. It selects SciPy's `scipy.linalg.expm` when SciPy imports and otherwise uses the NumPy scaling-and-Taylor fallback. `KatoTransport.evolve` does not expose a method selector, so the exponential backend can differ across environments. Both routes are finite floating computations.

44.2.2 Product of direct rotations

```
transport.direct_rotation_product(
    times: Iterable[float],
) -> ComputationResult[ndarray]
```

The time sequence must contain at least two finite, strictly increasing values. For $t_0 < \cdots < t_N$, let R_j be the direct polar rotation from $P(t_j)$ to $P(t_{j+1})$. The method returns

$$U_{\mathrm{rot}} = R_{N-1} \cdots R_1 R_0.$$

Every step certificate is required immediately. The final numerical `PSC-III-T59` certificate checks only endpoint intertwining against 50ε. Diagnostics include the minimum singular value encountered and the number of partition steps. The product can fail when even one adjacent overlap sum is singular or unresolved.

44.2.3 Two-mesh refinement

```
transport.refinement_certificate(
    start: float,
    stop: float,
    *,
    coarse_steps: int = 64,
    refinement_factor: int = 2,
    tolerance: float | None = None,
) -> TransportRefinementResult
```

For coarse count N and integer factor $q \geq 2$, the function computes midpoint evolutions at N and qN steps. It records

$$\rho_{\mathrm{mesh}} = \|U_{qN} - U_N\|_2$$

and the fine endpoint intertwining residual. With no explicit tolerance, the threshold is twice the larger of the two evolution certificate tolerances. The result is valid only when both evolution certificates are valid and both new residuals are within the selected threshold.

`TransportRefinementResult` fields are `coarse_steps`, `fine_steps`, `coarse`, `fine`, `difference`, `endpoint_intertwining`, and `certificate`. The `PSC-III-C60` certificate is a two-mesh numerical comparison. It does not prove an asymptotic error order, a Cauchy estimate for all finer meshes, or continuum convergence.

44.3 Direct packet rotation

Two complete orthogonal packet systems $(P_1, \ldots, P_m)$ and $(Q_1, \ldots, Q_m)$ are paired by iterable order. Form the overlap sum

$$S = \sum_{a=1}^{m} Q_a P_a.$$

When S is invertible, its polar unitary is

$$U = S(S^*S)^{-1/2}.$$

The shipped implementation obtains this factor from the singular value decomposition $S = W\Sigma V^*$ as

$$U = WV^*.$$

It then satisfies

$$Q_a U = U P_a, \qquad 1 \le a \le m.$$

The entry point is

```
direct_packet_rotation(
    source: Iterable[Any],
    target: Iterable[Any],
    *,
    tolerance: float = 1e-10,
) -> DirectRotationResult
```

Both families are independently checked as complete orthogonal packet systems. They must have the same packet count and matrix shape. If the minimum singular value of S is not greater than the input tolerance, the overlap is treated as singular or unresolved and the call raises `ValidationError`.

`DirectRotationResult` stores `unitary`, `minimum_singular_value`, and `certificate`. The `PSC-III-T55` certificate checks

$$\rho_U = \|U^*U - I\|_2, \qquad \rho_{\text{int}} = \max_a \|Q_a U - U P_a\|_2$$

against 20 times the input tolerance. The SVD and residuals are numerical, even though the polar-intertwining theorem is exact under its hypotheses.

Direct rotation is local in packet-system space. If the overlap sum becomes singular, the routine refuses to choose a transport across that step. It does not relabel packets, solve a permutation assignment, or track rank changes.

44.4 Shared nonlinear and quantum implementation

Smooth packet transport is implemented once in this module:

```
komorebi_math.nonlinear.transport
```

The compatibility module `komorebi_math.quantum.kato` directly imports and re-exports:

- `DirectRotationResult` and `PacketDerivativeResult`.
- `TransportRefinementResult`.
- `MovingPacketSystem` and `KatoTransport`.
- `direct_packet_rotation`.

The identity is literal, not merely behavioral:

```
from komorebi_math.nonlinear import KatoTransport as NonlinearKato
from komorebi_math.quantum import KatoTransport as QuantumKato

assert QuantumKato is NonlinearKato
```

The nonlinear namespace owns the domain-independent geometry. The quantum namespace retains the earlier import path and uses the same object in quantum leakage, effective-Hamiltonian, and Feshbach examples. A quantum import does not change the algorithm, add a Hamiltonian assumption, or promote its numerical certificate to a physical theorem.

44.4.1 API and certificate ledger

The complete public record constructors are:

```
DirectRotationResult(
    unitary,
    minimum_singular_value,
    certificate,
)

PacketDerivativeResult(
    time,
    step,
    maximum_derivative_residual,
    tangent_residual,
    partition_derivative_residual,
    certificate,
)

TransportRefinementResult(
    coarse_steps,
    fine_steps,
    coarse,
    fine,
    difference,
    endpoint_intertwining,
    certificate,
)
```

All three are frozen slotted dataclasses. Their NumPy array fields are still mutable array objects. Dataclass freezing does not mark those buffers read-only.

Certificate producers and meanings are:

- `direct_packet_rotation`: `PSC-III-T55`. Numerical unitarity and per-packet intertwining of one polar factor.
- `MovingPacketSystem.derivative_certificate`: `PSC-III-D56`. Sampled finite difference, tangent identity, and differentiated partition identity.
- `MovingPacketSystem.generator_certificate`: `PSC-III-T57`. Sampled skew adjointness and projector commutator identity.
- `KatoTransport.evolve`: `PSC-III-T58`. Sampled generator skewness and endpoint unitarity/intertwining for one midpoint mesh.
- `KatoTransport.direct_rotation_product`: `PSC-III-T59`. Every local polar step plus final endpoint intertwining.
- `KatoTransport.refinement_certificate`: `PSC-III-C60`. Validity of two midpoint runs, their finite difference, and fine endpoint intertwining.

Every runtime certificate above has `Exactness.NUMERICAL`. The theorem or provenance status attached to the underlying finite identity does not change the fact that SVD, matrix exponentials, norms, and comparisons use floating arithmetic.

44.4.2 Rotating two-packet example

Let

$$G = 0.4\begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}, \qquad P_0 = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix},$$

$$R(t) = e^{tG}, \qquad P_1(t) = R(t)P_0R(t)^*, \qquad P_2(t) = I - P_1(t).$$

Then $\dot{P}_a = [G, P_a]$. The runnable panel below exercises every transport path:

```
import numpy as np

from komorebi_math.dynamics import matrix_exponential
from komorebi_math.nonlinear import (
    KatoTransport,
    MovingPacketSystem,
    direct_packet_rotation,
)

generator = 0.4 * np.array([[0, -1], [1, 0]], dtype=complex)
base = np.diag([1, 0]).astype(complex)

def packets(time):
    rotation = matrix_exponential(time * generator)
    first = rotation @ base @ rotation.conj().T
    return first, np.eye(2) - first

def derivatives(time):
    return tuple(
        generator @ value - value @ generator
        for value in packets(time)
    )

system = MovingPacketSystem(packets, derivatives)
derivative = system.derivative_certificate(0.3, step=1e-5)
generator_check = system.generator_certificate(0.3)
rotation = direct_packet_rotation(
    system.packets(0.0),
    system.packets(0.2),
)

transport = KatoTransport(system)
midpoint = transport.evolve(0.0, 1.0, steps=128)
polar = transport.direct_rotation_product(np.linspace(0.0, 1.0, 129))
refined = transport.refinement_certificate(
    0.0,
    1.0,
    coarse_steps=32,
    tolerance=1e-6,
)

derivative.certificate.require()
generator_check.require()
rotation.certificate.require()
midpoint.certificate.require()
polar.certificate.require()
refined.certificate.require()

print(derivative.maximum_derivative_residual)
print(rotation.minimum_singular_value)
print(midpoint.certificate.residuals)
```

```
print(np.linalg.norm(midpoint.value - polar.value, ord=2))
print(refined.coarse_steps, refined.fine_steps, refined.difference)
```

Observed on the recorded build environment:

```
6.765885324179971e-12
0.9968017063026193
{'intertwining': 1.4412780446994425e-16,
 'unitarity': 1.0484054863159256e-14,
 'generator_failures': 0.0}
1.639684926003546e-14
32 64 4.537702386448892e-15
```

These residuals are platform-sensitive floating observations. The near-machine precision agreement is expected here because the example is generated by one constant skew matrix. It does not establish the same behavior for a general moving packet family.

The repository also supplies:

- `examples/filtered_transport.py`, which combines a finite spectral sequence with the two-mesh Kato check.
- `examples/kato_feshbach.py`, which imports the identical transport objects from `komorebi_math.quantum` and combines them with a quantum Feshbach reduction.

44.4.3 Failures, cost, and source-observed boundaries

44.4.3.1 Declared failure surface

The transport layer intentionally rejects:

- an empty packet family.
- A nonsquare or nonfinite packet or derivative matrix.
- A nonpositive or nonfinite packet tolerance.
- Non-self-adjoint, non-idempotent, nonorthogonal, or incomplete projectors.
- Packet count or dimension drift after the reference time.
- Derivative count or shape inconsistent with the packet family.
- A nonfinite time or nonpositive/nonfinite derivative step.
- Direct-rotation packet systems with different counts or dimensions.
- A direct overlap sum whose minimum singular value is at or below tolerance.
- A non-`MovingPacketSystem` passed to `KatoTransport`.
- A Boolean, nonintegral, or nonpositive evolution step count.
- A direct-rotation time partition with fewer than two points or without strict increase.
- A refinement factor below two or not exactly integral.
- A nonpositive or nonfinite explicit refinement tolerance.

Most contract failures are `ValidationError`. Matrix or family shape failures are `DimensionError`. The NumPy-only Taylor fallback for a matrix exponential can also raise `ArithmeticError` if it does not converge within its internal term ceiling.

44.4.3.2 Dense cost and storage

Let m be packet count, n ambient dimension, and N time steps. Ignoring user-callable cost and lower-order additions:

- validating self-adjointness and idempotence uses $O(mn^3)$ dense products. Pairwise orthogonality raises complete family validation to $O(m^2n^3)$.
- Storing one packet sample uses $O(mn^2)$ complex scalars.
- A derivative certificate evaluates packets at three times, evaluates the declared derivatives once, and performs $O(mn^3)$ tangent products after the family validations.
- Constructing the canonical generator uses m dense matrix products, $O(mn^3)$ work and $O(n^2)$ result storage.
- One direct rotation forms m products, takes one $n \times n$ SVD, and checks m intertwining residuals, for $O((m+1)n^3)$ work and dense $O(mn^2)$ sampled packet storage.
- Midpoint Kato evolution costs approximately $O(N(m+1)n^3)$, including one dense matrix exponential per step.
- A direct-rotation product has the same order with one SVD per interval.
- Refinement at factor q performs $N + qN$ midpoint steps.

No step stores the entire time history, but user callables may cache their own state. There is no sparse-projector specialization or device-memory route.

44.4.3.3 Recorded implementation boundaries

These source-observed behaviors should remain visible:

- Packet labels are absent from the transport API. Source packet `a` is paired with target packet `a` solely by iterable position. A reordered but otherwise valid family changes the intended correspondence without a label error.
- The validator accepts a zero-by-zero projector as a formal complete family. `MovingPacketSystem` can therefore report dimension zero, while `direct_packet_rotation` later reaches NumPy's empty minimum reduction and raises a raw `ValueError`. Require positive ambient dimension externally.
- The derivative threshold grows as $50h^2$. A very large finite-difference step can make the certificate permissive rather than informative. For the rotating example, `step=100.0` produced derivative residual about `0.405`, tolerance `500000.0`, and `certificate.valid == True`. Use a scale-appropriate small step and inspect the residual itself.
- The evolution tolerance similarly contains $20|\Delta|^2$. A coarse mesh can receive a large threshold. Require an explicit refinement tolerance and examine several meshes when convergence matters.
- `canonical_generator` does not call `generator_certificate`. Callers can use a matrix built from inconsistent declared derivatives. `evolve` checks sampled skewness and endpoint behavior, but not the full commutator residual at every midpoint.
- Projector and derivative callables are assumed deterministic and free of incompatible side effects. The runtime may call them repeatedly at the same time and provides no purity or continuity check.
- `KatoTransport.evolve` uses the environment-dependent automatic matrix exponential backend and provides no direct option to force SciPy or the NumPy Taylor path.
- A two-mesh refinement pass is a posteriori agreement only. Two discretizations can agree for the wrong reason, and no certificate bounds unsampled behavior.

No object in this chapter proves global differentiability, constant rank beyond the evaluated packet families, existence of an infinite-dimensional Kato propagator, adiabaticity, a spectral gap, physical realizability, or continuum convergence.

44.4.4 Verification evidence

The focused 66-test run reported in §43.5.4.5 includes all four tests in

```
test_nonlinear_transport.py
```

and the transport path in

```
test_quantum_transport_effective_spectral.py
```

It checked shared object identity, derivative and generator residuals, direct polar rotation, midpoint evolution, direct-rotation products, refinement, and invalid time-partition arguments. Both transport examples completed.

Those tests cover finite two-dimensional systems and declared failure cases. They do not remove the source-observed coarse-threshold and zero-dimensional boundaries above.

Chapter 44 summary. A moving packet system is a sampled family of complete orthogonal projectors plus declared derivatives. Require both the derivative and generator certificates before transport. Direct polar rotation is canonical only while the ordered overlap sum is invertible. Kato evolution uses midpoint exponentials and must be judged by unitarity, endpoint intertwining, and mesh refinement together. The quantum namespace re-exports the exact nonlinear implementation. It does not add a different algorithm or a stronger physical claim.

45. Matrix dynamics

The finite-dynamics layer turns a square matrix into a semigroup, resolves that semigroup into packets, compares it across resolutions, and specializes the same machinery to finite Markov, Bayesian, fractional, Green, and Feynman-Kac models. It is a dense finite-dimensional layer. It does not silently promote floating residuals to exact equalities, a finite state model to an infinite process, or an approximately clustered partition to a sufficient statistic.

The `komorebi_math.dynamics` package exports 43 names used in Chapters 45 and 46. Three additional public names are submodule-only: `MarkovKind` from `dynamics.markov`, `FeynmanKacKind` from `dynamics.feynman_kac`, and `blocks_from_labels` from `dynamics.martingales`. Seven established entry points are also re-exported from the broad top-level namespace:

```
from komorebi_math import (
    BayesianUpdate,
    FeshbachReduction,
    MarkovLumping,
    MatrixSemigroup,
    MemoryKernel,
    PhaseTypeSurvival,
    fractional_power,
)
```

None of those names is part of `komorebi_math.stable` or `komorebi_math.beta` in version `1.0.0b4`. The strict research manifest does not cover the `komorebi_math.dynamics` namespace itself. It freezes only the seven root bindings shown above. Import other package or submodule names from their documented locations, pin the package version, and do not infer a frozen signature merely from namespace visibility.

The namespace controls are:

```
FEATURE_KEY = "dynamics"
status() -> FeatureSpec
```

`status()` returns the registered implemented feature specification. It does not run verification.

The claims fall into these classes.

- **E** is an exact finite algebraic identity or a finite combinatorial fact.
- **N** is a numerical evaluation or residual certificate at a stated tolerance.
- **C** is conditional on hypotheses which the returned object identifies but does not establish.
- **H** is a heuristic or clustering rule with no exact sufficiency claim.
- **M** is an implementation work or storage model.
- **O** concerns an optional backend.

The source provenance identifiers retain the historical `PSC-VI` prefix even though this guide places matrix dynamics in Part VII. The prefix is a provenance identifier, not a chapter-number assertion.

45.1 Semigroups and resolvents

For a finite square generator $A \in \mathbb{C}^{n\times n}$, the semigroup is

$$T(t) = e^{tA}.$$

For $s, t \geq 0$, matrix-exponential algebra gives

$$T(0) = I, \qquad T(s+t) = T(s)T(t), \qquad \frac{d}{dt}T(t) = AT(t) = T(t)A.$$

These are exact finite matrix identities. `MatrixSemigroup` evaluates them with floating dense linear algebra, so its certificate is **N** rather than an exact symbolic proof.

45.1.1 Matrix exponential execution

```
matrix_exponential(
    matrix: Any,
    *,
    tolerance: float = 1e-15,
    max_terms: int = 256,
    method: Literal["auto", "scipy", "taylor"] = "auto",
) -> ndarray
```

The input must be a finite square array. `method="auto"` uses `scipy.linalg.expm` when SciPy is importable. Otherwise it uses the NumPy-only scaling-and-Taylor path. `method="scipy"` makes SciPy mandatory and raises `BackendUnavailableError` when it is absent. `method="taylor"` forces the base-installation path, which is useful for independent qualification.

For the Taylor path, let $m = \lceil \log_2(\|A\|_1/0.5) \rceil_+$ and $B = A/2^m$. The implementation forms

$$E_B = I + \sum_{k=1}^{K} \frac{B^k}{k!}$$

until the one-norm of the latest term is at most `tolerance` times the larger of one and the accumulated one-norm. It then squares the result m times. Failure to converge within `max_terms` raises `ArithmeticError`. A nonpositive or nonfinite tolerance, a nonpositive or nonintegral term budget, or an unknown method raises `ValidationError`. The fallback is a numerical algorithm, not an interval enclosure. **N/O**

45.1.2 `MatrixSemigroup`

```
MatrixSemigroup(generator: ndarray)
    .dimension -> int
    .at(time: float) -> ndarray
    .operator(time: float) -> ndarray
    .apply(time: float, vector: Any) -> ndarray
    .resolvent(value: complex) -> ndarray
    .semigroup_certificate(
        first: float,
        second: float,
        *, tolerance: float = 1e-10,
    ) -> Certificate
```

`operator` is a compatibility synonym for `at`. Both require finite nonnegative time. `apply` accepts exactly one vector of length n. It does not accept a matrix of right-hand sides. The resolvent convention is

$$R(z; A) = (zI - A)^{-1}.$$

`resolvent` delegates singularity detection to `numpy.linalg.inv`. It does not estimate the distance from z to the spectrum or certify conditioning.

The semigroup-law residual is

$$\rho_{\mathrm{sg}} = \|T(s+t) - T(s)T(t)\|_2 .$$

The certificate accepts when

$$\rho_{\mathrm{sg}} \le \varepsilon \max\{1, \|T(s+t)\|_2\}.$$

It has exactness `NUMERICAL`, theorem identifier `PSC-VI-T01`, and checked assumption `finite matrix generator`. The class definition carries provenance identifier `PSC-VI-D01`.

45.1.3 Packet-resolved semigroups

Let P_a and Q_b denote a complete finite packet family. The transfer blocks are

$$T_{ba}(t) = Q_b T(t) P_a.$$

Completeness inserts the intermediate packet resolution and gives

$$T_{ca}(s+t) = \sum_b T_{cb}(s) T_{ba}(t).$$

`PacketSemigroup` stores a `MatrixSemigroup` and a `PacketSystem` of the same dimension. `blocks(t)` constructs a `Packet Operator` from $T(t)$, while `generator_blocks()` resolves A itself. `transfer_graph()` places an edge where a generator block exceeds its numerical support tolerance.

```
PacketSemigroup(semigroup: MatrixSemigroup, packets: PacketSystem)
    .from_generator(generator: Any, packets: PacketSystem)
        -> PacketSemigroup
    .blocks(time: float) -> PacketOperator
    .generator_blocks() -> PacketOperator
    .transfer_graph(*, tolerance: float = 1e-12) -> TransferGraph
    .convolution_certificate(
        first: float,
        second: float,
        *, tolerance: float = 1e-10,
    ) -> Certificate
    .minimum_transfer_order(
        source: int,
        target: int,
        *, tolerance: float = 1e-12,
    ) -> int | None
```

The convolution residual reconstructs the packet products as dense matrices:

$$\rho_{\mathrm{packet}} = \left\| \mathrm{rec}(T(s) \circ T(t)) - \mathrm{rec}(T(s+t)) \right\|_2 .$$

It uses theorem `PSC-VI-T02` and exactness `NUMERICAL`. `minimum_transfer_order(a,b)` returns zero for $a = b$, the first path length found in the thresholded generator graph, or `None`. It is therefore a finite graph statement about numerically active blocks. It is not a proof that an analytically nonzero transfer first appears at that order. **E/N**

45.1.4 Refinement and Duhamel defects

Let $A_f \in \mathbb{C}^{n_f \times n_f}$, $A_c \in \mathbb{C}^{n_c \times n_c}$, and $C \in \mathbb{C}^{n_c \times n_f}$. The generator defect and finite-time defect are

$$E = C A_f - A_c C, \qquad \Delta(t) = C e^{t A_f} - e^{t A_c} C.$$

The exact Duhamel identity is

$$\Delta(t) = \int_0^t e^{(t-s)A_c} E e^{s A_f} \, ds.$$

The implemented direct norm bound is

$$\|\Delta(t)\|_2 \le t\exp\bigl(t\max\{\|A_f\|_2,\|A_c\|_2\}\bigr)\,\|E\|_2.$$

For z in both resolvent sets,

$$CR_f(z)-R_c(z)C=R_c(z)ER_f(z).$$

`SemigroupRefinement.duhamel` evaluates the integral using a declared Gauss-Legendre order and checks it against `direct_defect`. It rejects quadrature orders below two. Its internally selected tolerance is $10^{-9}\max(1,\|\Delta(t)\|_2)$, independent of a caller-supplied certificate tolerance. `resolvent_defect` evaluates both sides of the resolvent identity and uses $10^{-10}\max(1,\|CR_f-R_cC\|_2)$. A singular inverse propagates a NumPy linear-algebra exception.

```
SemigroupRefinement(
    coarsen: ndarray,
    fine_generator: ndarray,
    coarse_generator: ndarray,
)
    .generator_defect -> ndarray
    .direct_defect(time: float) -> ndarray
    .duhamel(time: float, *, quadrature_order: int = 24)
        -> ComputationResult[ndarray]
    .norm_bound(time: float) -> float
    .resolvent_defect(value: complex)
        -> ComputationResult[ndarray]

semigroup_refinement_defect(
    coarsen: Any,
    fine_generator: Any,
    coarse_generator: Any,
    time: float | None = None,
    *, quadrature_order: int = 24,
) -> SemigroupRefinement | ComputationResult[ndarray]
```

The wrapper returns the comparison object when `time is None` and its Duhamel result otherwise. The class definition has identifier `PSC-VI-D07`. The two certificates use `PSC-VI-T08` and `PSC-VI-T10`. **E/N**

45.1.5 Finite semigroup towers

A tower stores generators $A_0,\dots,A_{L-1}$ from coarse to fine and adjacent maps $C_0,\dots,C_{L-2}$, where C_j maps level $j+1$ to level j. For $i<j$,

$$C_{i\leftarrow j}=C_iC_{i+1}\cdots C_{j-1},\qquad \Delta_{i\leftarrow j}(t)=C_{i\leftarrow j}e^{tA_j}-e^{tA_i}C_{i\leftarrow j}.$$

```
FiniteSemigroupTower(
    generators: Sequence[Any],
    coarsenings: Sequence[Any],
)
    .levels -> int
    .adjacent(level: int) -> SemigroupRefinement
    .accumulated_coarsening(
        coarse_level: int,
        fine_level: int,
    ) -> ndarray
    .total_defect(
        coarse_level: int,
        fine_level: int,
        time: float,
    ) -> ndarray
    .certificate(*, tolerance: float = 1e-10) -> Certificate
```

`ProjectiveSemigroupTower` is the same class object, not a subclass. The tower requires at least one generator and exactly one fewer coarsening map. Its certificate records every adjacent generator-defect norm as `level_0`, `level_1`, and so on, under theorem `PSC-VI-T12`. Acceptance says only that the finite adjacent defects do not exceed the requested tolerance.

45.2 Markov lumping

The Markov layer consistently uses column probability vectors. A continuous generator Q and a discrete transition matrix P satisfy

$$Q_{ij} \geq 0 \quad (i \neq j), \qquad \mathbf{1}^T Q = 0,$$

and

$$P_{ij} \geq 0, \qquad \mathbf{1}^T P = \mathbf{1}^T.$$

Thus

$$p(t) = e^{tQ} p_0, \qquad p_k = P^k p_0.$$

Supplying row-stochastic data without transposing it changes the model and is normally rejected by the column-sum checks.

When importing a transition matrix from another program, identify its state ordering and probability-vector convention first. In this chapter a column describes transitions from one source state, and the state vector is a column. The Feynman-Kac section later acts on observables and therefore uses a transpose. That change is part of the mathematical convention, not a change in state labels.

45.2.1 Partition normalization and aggregation

`normalize_partition(partition, count)` accepts either one label per state or explicit nonempty blocks. Label blocks are ordered by first appearance. Explicit blocks must contain every index from zero through `count - 1` exactly once. Labels themselves need not be consecutive because the routine normalizes them.

For blocks $B_0, \dots, B_{b-1}$, the mass map $C \in \mathbb{R}^{b \times n}$ and uniform section $S \in \mathbb{R}^{n \times b}$ are

$$C_{ai} = \mathbf{1}_{i \in B_a}, \qquad S_{ia} = \frac{\mathbf{1}_{i \in B_a}}{|B_a|}, \qquad CS = I_b.$$

The implemented kernel basis N_C has one column $e_i - e_{i_0}$ for each nonanchor state in a block. It spans `ker(C)`.

```
AggregationMap(
    partition: Sequence[int] | Iterable[Iterable[int]],
    state_count: int | None = None,
)
    .blocks -> tuple[tuple[int, ...], ...]
    .state_count -> int
    .block_count -> int
    .labels -> tuple[int, ...]
    .matrix -> ndarray
    .uniform_section -> ndarray
    .kernel_basis() -> ndarray
    .aggregate(values: Any) -> ndarray
    .lift_uniform(block_values: Any) -> ndarray

aggregation_matrix(
    partition: Sequence[int] | Iterable[Iterable[int]],
    count: int | None = None,
) -> ndarray
```

`aggregate` and `lift_uniform` accept one-dimensional vectors only. They check dimensions but do not themselves validate probability mass.

45.2.2 Finite chains

```
FiniteMarkovChain(
    operator: ndarray,
    kind: Literal["generator", "transition"] = "generator",
    tolerance: float = 1e-12,
)
    .states -> int
    .generator -> ndarray
    .transition_matrix -> ndarray
    .evolution(amount: float | int) -> ndarray
    .evolve(distribution: Any, amount: float | int) -> ndarray
    .stationary_distribution() -> ComputationResult[ndarray]

ContinuousTimeMarkovChain(
    generator: ndarray,
    tolerance: float = 1e-12,
)
    .states -> int
    .transition(time: float) -> ndarray
    .evolve(distribution: Any, time: float) -> ndarray
    .stationary_distribution() -> ComputationResult[ndarray]
```

Continuous evolution accepts finite nonnegative real time. Discrete evolution requires a nonnegative integer and rejects booleans. `evolve` requires a vector that is nonnegative and has unit mass within the chain tolerance. It zeros components whose magnitude is less than ten times that tolerance after propagation.

The stationary distribution is a least-squares solution of

$$Q\pi = 0, \quad \mathbf{1}^T\pi = 1$$

or

$$(P - I)\pi = 0, \quad \mathbf{1}^T\pi = 1.$$

The certificate records stationarity, mass, and negativity residuals and uses `max(chain.tolerance, 1e-10)`. It has exactness `NUMERICAL` and theorem identifier `PSC-VI-C22`. It does not certify uniqueness, irreducibility, or a spectral gap. **N**

45.2.3 Exact and approximate lumping

The kernel obstruction and section-based coarse candidate are

$$\mathcal{O} = CAN_C, \qquad \widetilde{A}_c = CAS.$$

The coordinate residual compares the coarse-rate columns CAe_i inside each source block. Exact lumpability requires those columns to agree, which is equivalent in this finite setting to an operator A_c satisfying

$$CA = A_cC.$$

`MarkovLumping.is_exact` requires both the spectral norm of CAN_C and the maximum coordinate discrepancy to be at most the requested tolerance. `exact_coarse_operator` rejects a failed criterion with `AssumptionError`. `approximate_coarse_operator` always returns CAS.

In `1.0.0b4`, the coordinate-residual path reconstructs a supplied chain with the chain constructor's default tolerance rather than preserving a looser tolerance carried by an existing `FiniteMarkovChain` or `ContinuousTimeMarkovChain`. A

chain which was valid under its own looser tolerance can therefore raise during `is_exact()` before a Boolean decision is returned. Treat this as a source-version boundary, not a theorem condition.

For generator mode, the evolution defect is

$$\Delta(t) = Ce^{tA} - e^{t\widetilde{A}_c}C,$$

with the source bound

$$\|\Delta(t)\|_2 \le te^{t\max(\|A\|_2,\|\widetilde{A}_c\|_2)}\|CA - \widetilde{A}_cC\|_2.$$

```
coordinate_lumpability_residual(
    operator: Any,
    partition: Sequence[int] | Iterable[Iterable[int]],
    *, kind: Literal["generator", "transition"] = "generator",
) -> float

MarkovLumping(
    operator: Any | FiniteMarkovChain | ContinuousTimeMarkovChain,
    partition: Sequence[int] | Iterable[Iterable[int]],
    *, kind: Literal["generator", "transition"] | None = None,
)
    .partition -> tuple[int, ...]
    .blocks -> tuple[tuple[int, ...], ...]
    .aggregation -> ndarray
    .obstruction -> ndarray
    .is_exact(*, tolerance: float = 1e-10) -> bool
    .approximate_coarse_operator() -> ndarray
    .exact_coarse_operator(*, tolerance: float = 1e-10) -> ndarray
    .coarse_operator -> ndarray
    .coarse_generator(*, tolerance: float = 1e-10) -> ndarray
    .coarse_transition(*, tolerance: float = 1e-10) -> ndarray
    .coarse_chain(*, tolerance: float = 1e-10)
        -> FiniteMarkovChain | ContinuousTimeMarkovChain
    .certificate(*, tolerance: float = 1e-10) -> Certificate
    .aggregate(distribution: Any) -> ndarray
    .evolution_defect(
        amount: float | int,
        *, exact_coarse: bool = False,
    ) -> ndarray
    .semigroup_defect(
        time: float,
        *, use_exact_only: bool = False,
    ) -> ndarray
    .error_bound(time: float) -> float
```

The lumping certificate is **N**, uses theorem `PSC-VI-T21`, and records `kernel_obstruction` and `coordinate_residual`. The underlying theorem is exact finite mathematics. A certificate which passes at nonzero tolerance means numerically lumpable at that tolerance, not literally equal entries.

`semigroup_defect` and `error_bound` are generator-only. The default `evolution_defect` uses the approximate coarse operator unless `exact_coarse=True`. Callers should make that choice visible in reports.

45.3 Bayesian partitions and updates

Given a finite prior $p_i \ge 0$ and likelihood $L_i \ge 0$, the evidence and posterior are

$$Z = L^T p, \qquad p'_i = \frac{L_i p_i}{Z}.$$

`BayesianUpdate` normalizes the supplied prior before computing Z. Likelihoods are not normalized. They must be finite, nonnegative, and positive on at least one state. Posterior access raises `AssumptionError` when the evidence under the normalized prior is zero.

For a second likelihood $\widetilde{L}$, the implemented perturbation bound is

$$\left\| p'_L - p'_{\widetilde{L}} \right\|_1 \le \frac{2 \sum_i p_i |L_i - \widetilde{L}_i|}{\max(Z_L, Z_{\widetilde{L}})}.$$

Interpret this as a posterior perturbation bound only when both evidences $Z_L > 0$ and $Z_{\widetilde{L}} > 0$, so both posteriors exist. The current method raises when both evidences are zero, but when exactly one is zero it can return a finite number even though one posterior is undefined. Callers should check both evidences explicitly before using the result as a posterior bound.

```
BayesianUpdate(prior: Any, likelihood: Any)
    .prior -> ndarray
    .likelihood -> ndarray
    .evidence -> float
    .posterior -> ndarray
    .perturbation_bound(other_likelihood: Any) -> float
```

The class definition has provenance identifier `PSC-VI-D33`.

45.3.1 Two-stage coarsening

For a partition block B, define

$$\bar{p}_B = \sum_{i \in B} p_i, \qquad L_B^{\mathrm{eff}} = \begin{cases} \dfrac{\sum_{i \in B} L_i p_i}{\bar{p}_B}, & \bar{p}_B > 0, \\ 0, & \bar{p}_B = 0. \end{cases}$$

Then finite arithmetic gives

$$\mathrm{agg}(p') = \mathrm{Bayes}(\mathrm{agg}(p), L^{\mathrm{eff}}).$$

`BayesianPartition.coarse_posterior` evaluates both sides and records their maximum componentwise difference as `two_stage_update`. Its fixed tolerance is 10^{-12}, its exactness is `NUMERICAL`, and its theorem identifier is `PSC-VI-T36`.

```
BayesianPartition(partition: Sequence[int])
    .labels -> tuple[int, ...]
    .blocks -> tuple[tuple[int, ...], ...]
    .block_count -> int
    .aggregate(probability: Any) -> ndarray
    .effective_likelihood(prior: Any, likelihood: Any) -> ndarray
    .coarse_posterior(prior: Any, likelihood: Any)
        -> ComputationResult[ndarray]
    .is_fixed_sufficient(
        likelihoods: Sequence[Any],
        *, tolerance: float = 1e-12,
    ) -> bool
    .fixed_coarse_likelihood(
        likelihood: Any,
        *, tolerance: float = 1e-12,
    ) -> ndarray
    .information_loss(
        prior: Any,
```

```
        likelihood: Any,
        coarse_likelihood: Any,
    ) -> float
```

Labels are normalized in sorted-label order inside this class. This differs from `normalize_partition`, which preserves first-appearance order. The blocks are mathematically the same, but their numeric labels can be permuted.

45.3.2 Exact sufficiency and approximate profile clustering

A partition is fixed sufficient for supplied likelihoods when

$$i, j \in B \Longrightarrow L_i^{(r)} = L_j^{(r)} \quad \text{for every supplied profile } r.$$

```
coarsest_sufficient_partition(
    likelihoods: Sequence[Any],
    *, tolerance: float = 0.0,
) -> BayesianPartition

approximate_likelihood_partition(
    likelihoods: Sequence[Any],
    *, tolerance: float,
) -> BayesianPartition
```

The exact routine requires `tolerance=0`. It groups identical tuples of floating profile values and rejects a positive tolerance. This is deliberate: pairwise closeness is generally not transitive and therefore does not define an equivalence relation.

The approximate routine builds a graph whose vertices are state profiles and whose edges join pairs within the maximum-norm tolerance. It returns connected components. Because connected components take the transitive closure, two profiles in one component can differ by more than the input tolerance. The result is **H** and is not promoted to Bayesian sufficiency. Call `is_fixed_sufficient` separately when an exact conclusion is required.

45.3.3 Relative entropy and finite information splitting

For normalized probability vectors,

$$D(p\|q) = \sum_{i:p_i>0} p_i \log \frac{p_i}{q_i}.$$

The implementation preserves $+\infty$ if any $p_i > 0$ while $q_i = 0$. For a partition,

$$D(p\|q) = D(\bar{p}\|\bar{q}) + \sum_B \bar{p}_B D(p(\cdot \mid B)\|q(\cdot \mid B)) \,.$$

```
relative_entropy(first: Any, second: Any) -> float

relative_entropy_decomposition(
    first: Any,
    second: Any,
    partition: Sequence[int],
) -> ComputationResult[dict[str, Any]]
```

The result dictionary contains `total`, `coarse`, and `conditional_terms`. If the direct and decomposed values are both infinite, the residual is defined to be zero. Otherwise it is their absolute difference. The certificate uses theorem `PSC-VI-T41` and fixed tolerance 10^{-12}. **E/N**

45.3.4 Bayesian flow and hidden Markov filtering

For a score vector s, the exponential-likelihood flow is

$$p_i(t) = \frac{p_i(0)e^{ts_i}}{\sum_j p_j(0)e^{ts_j}}, \qquad \dot{p}_i = p_i(s_i - \mathbb{E}_{p(t)}s).$$

It obeys

$$\frac{d}{dt}\mathbb{E}_{p(t)}s = \mathrm{Var}_{p(t)}\, s \geq 0.$$

`BayesianFlow.at` subtracts the largest shifted score before exponentiation to avoid overflow. It permits finite positive or negative time. The methods return direct floating values and do not attach a derivative certificate.

For a column-stochastic transition P, one hidden-Markov step is

$$\hat{p} = Pp, \qquad p^+ = \mathrm{Bayes}(\hat{p}, L).$$

Exact coarse filtering requires both exact transition lumpability and a block-constant likelihood. If either hypothesis fails, `HiddenMarkovFilter.coarse_step` returns the directly aggregated fine result with a failed `CONDITIONAL` certificate. It does not fabricate an approximate coarse transition. When both pass, it compares the direct aggregation with the reconstructed coarse filter at the caller's tolerance under theorem `PSC-VI-T46`.

```
BayesianFlow(prior: Any, scores: Any)
    .at(time: float) -> ndarray
    .derivative(time: float) -> ndarray
    .expected_score(time: float) -> float
    .score_variance(time: float) -> float

HiddenMarkovFilter(transition: ndarray)
    .step(prior: Any, likelihood: Any) -> ndarray
    .coarse_step(
        prior: Any,
        likelihood: Any,
        partition: Sequence[int],
        *, tolerance: float = 1e-10,
    ) -> ComputationResult[ndarray]
```

45.4 Fractional powers and Green operators

The fractional layer is a finite Hermitian spectral calculus. For $A = U\,\mathrm{diag}(\lambda_i)U^* \succeq 0$ and $\alpha > 0$,

$$A^\alpha = U\,\mathrm{diag}(\lambda_i^\alpha)U^*, \qquad e^{-tA^\alpha} = U\,\mathrm{diag}(e^{-t\lambda_i^\alpha})U^*.$$

The input must be square, Hermitian to the supplied absolute tolerance, and positive semidefinite to that tolerance. Eigenvalues between $-\varepsilon$ and zero are clipped to zero. The operation uses `numpy.linalg.eigh`. It is **N**, even when the displayed spectral identity is exact.

```
fractional_power(
    matrix: Any,
    exponent: float,
    *, tolerance: float = 1e-12,
) -> ndarray

fractional_semigroup(
    matrix: Any,
    exponent: float,
    time: float,
```

```
    *, tolerance: float = 1e-12,
) -> ndarray
```

Both require a positive finite exponent. The semigroup additionally requires finite nonnegative time.

45.4.1 Naturality and the positive-gap defect bound

Exact intertwining gives the finite spectral identity

$$CA_f = A_c C \Longrightarrow CA_f^{\alpha} = A_c^{\alpha} C.$$

The numerical certificate records both

$$\rho_A = \|CA_f - A_c C\|_2, \qquad \rho_\alpha = \|CA_f^{\alpha} - A_c^{\alpha} C\|_2.$$

For $0 < \alpha < 1$ and a common strictly positive spectral gap γ, the implemented theorem bound is

$$\|CA_f^{\alpha} - A_c^{\alpha} C\|_2 \le \alpha \gamma^{\alpha-1} \|CA_f - A_c C\|_2.$$

```
fractional_naturality_certificate(
    coarsen: Any,
    fine_operator: Any,
    coarse_operator: Any,
    exponent: float,
    *, tolerance: float = 1e-10,
) -> Certificate

fractional_defect(
    coarsen: Any,
    fine_operator: Any,
    coarse_operator: Any,
    exponent: float,
    *, tolerance: float = 1e-12,
) -> FractionalDefectResult
```

`FractionalDefectResult` is a frozen, slotted dataclass with fields `defect`, `bound`, `generator_defect_norm`, `spectral_gap`, and `certificate`. The certificate tests the actual norm against the theorem bound plus a numerical allowance. The naturality and defect identifiers are `PSC-VI-T25` and `PSC-VI-T26`. The computed common gap must exceed the caller's tolerance, not merely be positive. A missing certified gap or an exponent outside $(0, 1)$ raises `ValidationError`. The routine does not emit a vacuous infinite bound. **E/N**

The two helper functions do not forward their caller tolerance to every internal `fractional_power` call. Those nested calls use the default 10^{-12} tolerance. Consequently, an operator which passes an initial Hermitian or positivity check under a looser requested tolerance can still raise during the internal power computation. Validate with the stricter default or catch this source-version boundary explicitly.

45.4.2 Discrete Dirichlet Green operator

For $N \ge 1$ interior points and $h = (N+1)^{-1}$,

$$A_h = h^{-2}\,\mathrm{tridiag}(-1, 2, -1).$$

At grid points $x_i = ih$, the continuum Dirichlet kernel is

$$G(x, y) = \min(x, y)(1 - \max(x, y)),$$

and the source implements the exact finite identity

$$G_h = A_h^{-1} = h\,[G(x_i, x_j)]_{i,j}.$$

```
DirichletGreen1D(interior_points: int)
    .spacing -> float
    .grid -> ndarray
    .operator -> ndarray
    .continuum_kernel_samples -> ndarray
    .inverse -> ndarray
    .solve(source: Sequence[float]) -> ndarray
    .eigenvalues() -> ndarray
    .eigenvalue_error_bound(mode: int) -> float
    .certificate(*, tolerance: float = 1e-10) -> Certificate
```

The discrete eigenvalues are

$$\lambda_{k,h} = \frac{4}{h^2}\sin^2\!\left(\frac{k\pi h}{2}\right), \qquad 1 \le k \le N,$$

and the implemented mode bound is

$$0 \le (k\pi)^2 - \lambda_{k,h} \le \frac{(k\pi)^4 h^2}{12}.$$

The certificate checks only $\|A_h G_h - I\|_2$ under theorem `PSC-VI-T49`. It does not check every mode bound. The source materializes dense $N \times N$ arrays. It does not exploit the tridiagonal solve complexity. **E/N/M**

45.5 Finite Feynman-Kac transfer

`FiniteFeynmanKac` acts on observables, while the Markov objects above act on column probability vectors. For a column generator Q and a nonnegative potential V, the backward killed generator and transfer are

$$L_V = Q^T - \operatorname{diag}(V), \qquad K_V(t) = e^{tL_V}.$$

For a column-stochastic transition P, the declared one-step operator is

$$K_V = \operatorname{diag}(e^{-V})P^T.$$

The order is significant: killing multiplies the observable after the transpose transition in the displayed matrix product. Discrete propagation uses integer powers of this one-step operator.

```
FiniteFeynmanKac(
    markov_operator: Any,
    potential: Any,
    *,
    kind: Literal["generator", "transition"] = "generator",
    tolerance: float = 1e-12,
)
    .states -> int
    .backward_generator -> ndarray
    .one_step_operator -> ndarray
    .operator(amount: float | int) -> ndarray
    .apply(observable: Any, amount: float | int) -> ndarray
    .potential_perturbation_bound(
        other_potential: Any,
        time: float,
    ) -> float
    .coarse_model(
        partition: Sequence[int] | Sequence[Sequence[int]],
```

```
        *, tolerance: float = 1e-10,
    ) -> ComputationResult[FiniteFeynmanKac | None]
```

Continuous propagation requires finite nonnegative time. Discrete propagation requires a nonnegative integer and rejects booleans. The continuous perturbation estimate is

$$\|K_V(t) - K_{\widetilde{V}}(t)\|_2 \le t \exp\Big(t(\|Q^T\|_2 + \max(\|V\|_\infty, \|\widetilde{V}\|_\infty))\Big)\, \|V - \widetilde{V}\|_\infty.$$

Exact coarsening requires both a lumpable Markov operator and a potential constant on every block. When either condition fails, `.value` is `None` and the failed certificate has exactness `CONDITIONAL`, residuals `lumping` and `potential_variation`, and unchecked assumptions naming both requirements. When both hold, `.value` is the coarse model and the certificate is valid and `NUMERICAL`. Both paths use theorem identifier `PSC-VI-C57`. The model definition is documented under `PSC-VI-T55`.

The coarsening path also reconstructs its Markov lumping object without preserving a looser source-model tolerance. After a tolerance-based success, the returned certificate hard-codes its `lumping` residual to `0.0`. That field is not the measured coordinate or obstruction residual and must not be cited as evidence of literal zero.

This finite transfer is a matrix implementation of the Feynman-Kac semigroup. It does not construct sample paths or certify an infinite-state stochastic representation. **E/N/C**

45.6 Finite conditional expectations, martingale towers, and energy diagnostics

Let $w_i > 0$ be finite weights, normalized by the constructor, and let $\mathcal{P}$ be a partition. The weighted conditional expectation is

$$(E_{\mathcal{P}} f)_i = \frac{\sum_{j \in B(i)} w_j f_j}{\sum_{j \in B(i)} w_j}, \qquad \langle f, g \rangle_w = \sum_i w_i \overline{f_i} g_i.$$

The dense matrix is constant by rows inside each block. It obeys

$$E_{\mathcal{P}}^2 = E_{\mathcal{P}}, \qquad \operatorname{diag}(w) E_{\mathcal{P}} = E_{\mathcal{P}}^* \operatorname{diag}(w).$$

`ConditionalExpectation.certificate` checks both spectral-norm residuals at the caller's tolerance under theorem `PSC-VI-T17`.

```
ConditionalExpectation(
    partition: Iterable[Iterable[int]],
    weights: Any,
)
    .blocks -> tuple[tuple[int, ...], ...]
    .weights -> ndarray
    .matrix -> ndarray
    .apply(values: Any) -> ndarray
    .inner(left: Any, right: Any) -> complex
    .certificate(*, tolerance: float = 1e-11) -> Certificate

partition_refines(
    fine: tuple[tuple[int, ...], ...],
    coarse: tuple[tuple[int, ...], ...],
) -> bool
```

For partitions becoming finer with level, let E_j be their conditional expectations. The tower law and martingale differences are

$$E_iE_j = E_{\min(i,j)}, \qquad d_0 = E_0 f, \qquad d_j = E_j f - E_{j-1} f.$$

The finite energy decomposition is

$$\|E_m f\|_w^2 = \sum_{j=0}^{m} \|d_j\|_w^2.$$

```
MartingaleTower(
    partitions: Iterable[Iterable[Iterable[int]]],
    weights: Any,
)
    .weights -> ndarray
    .project(values: Any, level: int) -> ndarray
    .differences(values: Any) -> tuple[ndarray, ...]
    .energy_decomposition(values: Any)
        -> ComputationResult[tuple[float, ...]]
    .tower_certificate(*, tolerance: float = 1e-10)
        -> Certificate

ConditionalExpectationTower(
    weights: Any,
    label_partitions: Iterable[Iterable[int]],
)
    .weights -> ndarray
    .expectations -> tuple[ConditionalExpectation, ...]
    .project(values: Any, level: int) -> ndarray
    .differences(values: Any) -> tuple[ndarray, ...]
    .energy(values: Any) -> ComputationResult[tuple[float, ...]]
    .certificate(*, tolerance: float = 1e-10) -> Certificate
```

`MartingaleTower` rejects an empty tower or a pair of partitions which is not nested coarse-to-fine. `ConditionalExpectationTower` converts each label vector to blocks by first-appearance order. The helper `blocks_from_labels(labels)` is public from `komorebi_math.dynamics.martingales`, but is not re-exported by `komorebi_math.dynamics` in this release.

The tower and energy certificates use theorem `PSC-VI-T18` and exactness `NUMERICAL`. Their success establishes the finite weighted identities for the supplied partitions and vector. It does not establish a stochastic-process martingale outside this finite probability space.

45.6.1 Runnable matrix-dynamics panel

```
import numpy as np

from komorebi_math.dynamics import (
    ConditionalExpectationTower,
    DirichletGreen1D,
    FiniteFeynmanKac,
    MatrixSemigroup,
)

semigroup = MatrixSemigroup(np.diag([-1.0, -2.0]))
law = semigroup.semigroup_certificate(0.2, 0.3)
law.require()

tower = ConditionalExpectationTower(
    [1.0, 1.0, 1.0, 1.0],
    [[0, 0, 0, 0], [0, 0, 1, 1], [0, 1, 2, 3]],
)
energy = tower.energy([1.0, 2.0, 4.0, 8.0])
energy.certificate.require()
```

```
green = DirichletGreen1D(4)
green.certificate().require()

transition = np.kron(
    np.array([[0.8, 0.3], [0.2, 0.7]]),
    np.eye(2),
)
feynman_kac = FiniteFeynmanKac(
    transition,
    [0.1, 0.1, 0.4, 0.4],
    kind="transition",
)
coarse = feynman_kac.coarse_model([0, 0, 1, 1])
coarse.certificate.require()

print("semigroup diagonal", np.diag(semigroup.at(0.5)))
print("semigroup residual", law.residuals["semigroup_law"])
print("martingale energies", energy.value)
print("energy residual", energy.certificate.residuals["energy_decomposition"])
print("green spacing", green.spacing)
print("green inverse residual", green.certificate().residuals["inverse_identity"])
print("Feynman-Kac coarse", coarse.certificate.valid, coarse.value.states)
```

Observed output from the audited source tree:

```
semigroup diagonal [0.60653066 0.36787944]
semigroup residual 0.0
martingale energies (14.0625, 5.0625, 2.125)
energy residual 0.0
green spacing 0.2
green inverse residual 1.0924759764801266e-15
Feynman-Kac coarse True 2
```

45.6.2 Claim strength and completion boundary

The strongest justified statements in this chapter are finite.

- Semigroup, Duhamel, resolvent, lumping, Bayesian, entropy, spectral, Green, Feynman-Kac, and martingale formulas are **E** as finite identities under their displayed hypotheses.
- Source certificates are **N** because they evaluate dense floating arrays and compare residuals with tolerances.
- Failed HMM and Feynman-Kac coarsenings are **C** records. They expose the missing hypotheses instead of returning an exact coarse claim.
- Approximate likelihood partitioning is **H**. Graph connectivity is exact for the constructed graph, but sufficiency is not inferred.
- No API here proves convergence of a refinement tower, an infinite-state Markov limit, continuous-path Feynman-Kac, a continuum fractional operator, or an infinite filtration.

45.6.3 Complexity, storage, and execution tiers

Let n be the fine dimension, b a coarse block count, L a tower depth, m a likelihood-profile count, and N a Green grid size.

- Dense matrix exponentials, eigendecompositions, inverses, and stationary least-squares solves have the usual $O(n^3)$ arithmetic and $O(n^2)$ storage model. **M**
- A forced Taylor exponential uses at most `max_terms` dense multiplications plus the scaling squarings. Its upper work model is $O((K + \log\|A\|)n^3)$. **M**
- Explicit packet blocks and conditional-expectation matrices are dense in this implementation. Even blockwise formulas can materialize $O(n^2)$ data. **M**

- Partition normalization and one Bayesian update are linear in the state count. Exact profile grouping is $O(nm)$ expected dictionary work. Approximate profile clustering performs $O(n^2 m)$ pairwise comparisons and stores an $O(n^2)$ worst-case graph. **M**
- The Green implementation stores dense operator, kernel, and inverse arrays, so construction and application use $O(N^2)$ storage/work despite the tridiagonal structure of A_h. **M**
- Discrete Feynman-Kac propagation uses dense matrix powering, conventionally $O(n^3 \log k)$ for step count k. **M**

Tier 0 is NumPy and the base package. Tier 1 optionally uses SciPy only for the matrix exponential. There is no sparse-matrix, exact-rational, streaming, or CUDA path in `komorebi_math.dynamics 1.0.0b4`. Passing a CuPy array through `numpy.asarray` is not a supported device execution contract.

45.6.4 Source-observed boundaries and defects

- `MarkovKind` and `FeynmanKacKind` are public aliases in their submodules but are not re-exported by `komorebi_math.dynamics.__all__`.
- `blocks_from_labels` has the same submodule-only boundary.
- `BayesianPartition` normalizes labels in sorted order, whereas `AggregationMap` uses first appearance. Do not compare raw label numbers across the two classes without aligning blocks.
- Partition helpers coerce supplied labels and explicit state indices with `int(...)`. Numeric values such as `1.5` can therefore be truncated instead of rejected as nonintegral. Supply genuine integer indices.
- `approximate_likelihood_partition` returns connected components, so endpoint profiles in a tolerance chain can differ by more than the tolerance.
- `FiniteSemigroupTower.total_defect` calls the matrix exponential directly and does not apply `MatrixSemigroup`'s finite nonnegative-time validator. Validate tower times at the calling boundary when that convention matters.
- `FiniteMarkovChain.evolve` checks nonnegativity and total mass but does not explicitly check that every distribution entry is finite. A vector containing `NaN` can evade those comparisons and propagate `NaN`. Validate external distributions before calling it.
- `HiddenMarkovFilter` likewise does not explicitly precheck every transition entry for finiteness. A transition containing `NaN` can therefore reach the filtering arithmetic. Validate the transition matrix as finite before construction.
- The likelihood-profile partition helpers require finite profile entries but do not reject negative finite values at their own boundary. Later Bayesian sufficiency routines do reject negative likelihoods. Validate profiles as nonnegative before asking either profile-partition helper to group them.
- `DirichletGreen1D.certificate` checks the inverse identity only. The mode error bounds are separate returned values and are not included in that certificate.
- `FiniteFeynmanKac.potential_perturbation_bound` is a continuous-generator helper. It is not available in transition mode and is not a pathwise error certificate.
- `FiniteFeynmanKac.potential_perturbation_bound` does not explicitly reject a nonfinite `other_potential`. A `NaN` entry can produce a returned `NaN` bound. Validate comparison potentials as finite at the calling boundary.
- All numerical tolerances are absolute unless the formula above explicitly includes a scale. A passing residual must be interpreted together with conditioning and input scale.

45.6.5 Provenance and certificate ledger

These identifiers must be read together with the symbol that emits them. An identifier alone is not a globally unique evidence key. In this source release, runtime records carrying `PSC-VI-T01`, `PSC-VI-T08`, `PSC-VI-T10`, `PSC-VI-T12`, `PSC-VI-C22`, `PSC-VI-T41`, `PSC-VI-T46`, or `PSC-VI-C57` do not all have a matching attached provenance object on the emitting callable. Treat the fully qualified callable, identifier, package version, and certificate payload as the evidence address.

- `MatrixSemigroup` carries definition record `PSC-VI-D01` for the finite matrix semigroup. `MatrixSemigroup.semigroup_certificate` emits `PSC-VI-T01` for the semigroup-law residual.
- `PacketSemigroup.convolution_certificate` emits `PSC-VI-T02` for packet-semigroup convolution.
- `SemigroupRefinement` carries definition record `PSC-VI-D07`. `SemigroupRefinement.duhamel` emits `PSC-VI-T08`. `SemigroupRefinement.resolvent_defect` emits `PSC-VI-T10`, and `FiniteSemigroupTower.certificate` emits `PSC-VI-T12` for adjacent generator defects.

- `FiniteMarkovChain` carries definition record `PSC-VI-D18` for the finite column-convention Markov operator. `FiniteMarkovChain.stationary_distribution` emits `PSC-VI-C22` for a computed stationary distribution, and `MarkovLumping.certificate` emits `PSC-VI-T21` for exact-lumping residuals.
- `BayesianUpdate` carries definition record `PSC-VI-D33`. `BayesianPartition.coarse_posterior` emits `PSC-VI-T36` for two-stage Bayesian coarsening. `relative_entropy_decomposition` emits `PSC-VI-T41`, and `HiddenMarkovFilter.coarse_step` emits `PSC-VI-T46` for exact or conditionally failed coarse filtering.
- `ConditionalExpectation.certificate` emits `PSC-VI-T17`, while `MartingaleTower.energy_decomposition` and `.tower_certificate` emit `PSC-VI-T18` for energy and tower identities.
- `fractional_power` carries attached record `PSC-VI-T24`. `fractional_naturality_certificate` emits `PSC-VI-T25`, and `fractional_defect` emits `PSC-VI-T26` for the positive-gap bound.
- `DirichletGreen1D.certificate` emits `PSC-VI-T49` for the discrete Green inverse.
- `FiniteFeynmanKac` carries attached record `PSC-VI-T55`, while `FiniteFeynmanKac.coarse_model` emits `PSC-VI-C57` for exact or conditionally failed coarsening.

Every returned `Certificate` states exactness, residuals, checked assumptions, and a theorem identifier. A `ComputationResult` preserves its numerical `.value` even when a conditional certificate is false, except that failed Feynman-Kac coarsening uses `value=None`.

45.6.6 Error model and troubleshooting

- `DimensionError`: nonsquare arrays, incompatible coarsening maps, wrong vector lengths, mismatched likelihood profiles, or a malformed Green source.
- `ValidationError`: invalid method/tolerance/budget, negative semigroup time, malformed partitions, invalid Markov signs or column sums, noninteger discrete steps, invalid probabilities or likelihoods, nonnested towers, non-Hermitian or nonpositive fractional operators, missing positive gap, invalid Green mode, or invalid Feynman-Kac kind/potential.
- `AssumptionError`: a zero-evidence posterior, a nonconstant fixed coarse likelihood, or a requested exact Markov coarse operator for a nonlumpable partition.
- `BackendUnavailableError`: `method="scipy"` without SciPy.
- `ArithmeticError`: Taylor fallback exhaustion.
- `numpy.linalg.LinAlgError`: a requested resolvent or other dense inverse is singular.
- A false certificate is not an exception until the caller invokes `certificate.require()` or `result.require()`.

Evidence: `src/komorebi_math/dynamics/_linalg.py`, `semigroups.py`, `markov.py`, `bayes.py`, `martingales.py`, `fractional.py`, `green.py`, and `feynman_kac.py`.

46. Feshbach reduction, memory, and survival

Chapter 46 eliminates a declared hidden coordinate sector from a finite linear system. In the resolvent domain, elimination produces a Schur or Feshbach effective operator. In the time domain, the same block algebra produces a matrix-valued memory kernel and a hidden-source term. Finite phase-type survival is a separate specialization in which a stable transient Markov subgenerator loses mass to absorption.

The shared algebra does not make every memory kernel a survival law. A survival interpretation additionally requires the column-subgenerator, positivity, mass-loss, and stability conditions in Section 46.5.

Choose the question first. Use the resolvent route for dependence on a spectral parameter, the memory route for the effect of hidden dynamics over time, and the survival route only for an admissible absorbing Markov model. Keep the retained index order and any hidden initial state with the calculation. The sign conventions are written side by side in Section 46.2 so that a formula from another reference can be aligned before comparing values.

46.1 Retained and eliminated blocks

Choose a nonempty set P of retained coordinate indices and let Q be its nonempty complement. Reordering only for display gives

$$A = \begin{bmatrix} A_{PP} & A_{PQ} \\ A_{QP} & A_{QQ} \end{bmatrix}, \qquad u = \begin{bmatrix} x \\ y \end{bmatrix}.$$

The implementation does not physically permute the stored matrix. It uses `numpy.ix_` with the declared retained order and the complementary hidden indices in their original increasing order.

```
FeshbachReduction(matrix: Any, retained: Iterable[int])
    .matrix -> ndarray
    .retained -> tuple[int, ...]
    .hidden -> tuple[int, ...]
    .retained_block -> ndarray
    .hidden_block -> ndarray
    .retained_to_hidden -> ndarray
    .hidden_to_retained -> ndarray
    .a_pq -> ndarray
    .a_qp -> ndarray
```

The historical names require care:

- `retained_to_hidden` returns the top-right block A_{PQ}. As a matrix, it maps a hidden vector into the retained equation.
- `hidden_to_retained` returns the bottom-left block A_{QP}. It maps a retained vector into the hidden equation.
- `a_pq` and `a_qp` are unambiguous aliases for those arrays.

The matrix must be finite and square. Retained indices are coerced by Python `int`, after which they must be nonempty, unique, and in range. At least one hidden coordinate must remain. Construction raises `DimensionError` for a nonsquare matrix and `ValidationError` for invalid values or sectors. The definition carries provenance identifier `PSC-VI-D27`.

46.2 Feshbach/Schur effective operator

The guide records both common sign conventions. For the operator $H - zI$, the Feshbach map is

$$F_P(z) = P(H - z)P - PHQ\,[Q(H - z)Q]^{-1}QHP.$$

This is the $H - z$ convention. For the implementation's $zI - H$ convention,

$$F_P(z) = A_{\text{eff}}(z) - zI_P, \qquad P(zI - H)^{-1}P = -F_P(z)^{-1} = (zI_P - A_{\text{eff}}(z))^{-1}.$$

In source notation,

$$\Sigma(z) = A_{PQ}(zI_Q - A_{QQ})^{-1}A_{QP}, \qquad A_{\text{eff}}(z) = A_{PP} + \Sigma(z).$$

Provided both the hidden and full resolvents exist, block inversion gives

$$P(zI - A)^{-1}P = (zI_P - A_{\text{eff}}(z))^{-1}.$$

```
FeshbachReduction
    .self_energy(spectral_parameter: complex) -> ndarray
    .effective_operator(spectral_parameter: complex) -> ndarray
    .effective_resolvent(spectral_parameter: complex) -> ndarray
    .direct_projected_resolvent(
        spectral_parameter: complex,
    ) -> ndarray
    .resolvent_certificate(
        spectral_parameter: complex,
        *, tolerance: float = 1e-10,
    ) -> ComputationResult[ndarray]
```

The certificate returns the effective resolvent as `.value` and compares it with the retained principal block of the full resolvent:

$$\rho_{\text{Fesh}} = \left\|(zI_P - A_{\text{eff}})^{-1} - P(zI - A)^{-1}P\right\|_2.$$

It accepts when

$$\rho_{\text{Fesh}} \le \varepsilon \max\left\{1, \left\|P(zI - A)^{-1}P\right\|_2\right\}.$$

The exact Schur identity is **E** under the two resolvent hypotheses. The source certificate is **N**, names both hypotheses as checked because both inversions completed, and uses theorem `PSC-VI-T28`. It does not report condition numbers. A singular or numerically singular inverse raises `numpy.linalg.LinAlgError` before a failed certificate can be returned.

The minus sign in the displayed `H-z` relation is not optional. If a caller compares `effective_resolvent(z)` directly with $F_P(z)^{-1}$ under that convention, it will introduce exactly one erroneous sign.

46.3 Rational memory kernels

Consider the finite block system

$$\dot{x} = A_{PP}x + A_{PQ}y, \qquad \dot{y} = A_{QP}x + A_{QQ}y.$$

Variation of constants gives

$$y(t) = e^{tA_{QQ}}y(0) + \int_0^t e^{(t-s)A_{QQ}}A_{QP}x(s)\,ds.$$

Substitution into the retained equation yields

$$\dot{x}(t) = A_{PP}x(t) + f_Q(t) + \int_0^t K(t-s)x(s)\,ds,$$

where

$$K(t) = A_{PQ}e^{tA_{QQ}}A_{QP}, \qquad f_Q(t) = A_{PQ}e^{tA_{QQ}}y(0).$$

The hidden initial state therefore contributes an inhomogeneous source. It cannot be discarded unless $y(0) = 0$ or its retained image is independently known to vanish.

More generally, a finite rational memory kernel is

$$K(t) = Be^{tD}C, \qquad \widehat{K}(z) = B(zI - D)^{-1}C.$$

For finite horizon T, source algebra gives

$$\widehat{K}_T(z) = B(zI - D)^{-1}(I - e^{(D-zI)T})C,$$

and

$$\widehat{K}(z) - \widehat{K}_T(z) = B(zI - D)^{-1}e^{(D-zI)T}C.$$

The last two formulas are algebraic resolvent-exponential identities whenever $zI - D$ is invertible. Interpreting $\widehat{K}(z)$ as the convergent integral from zero to infinity additionally requires a spectral half-plane condition, which `MemoryKernel.laplace` does not check.

```
MemoryKernel(output: Any, hidden: Any, input: Any)
    .output -> ndarray
    .hidden -> ndarray
    .input -> ndarray
    .retained_dimension -> int
    .hidden_dimension -> int
    .at(time: float) -> ndarray
    .evaluate(time: float) -> ndarray
    .laplace(spectral_parameter: complex) -> ndarray
    .finite_laplace(
        spectral_parameter: complex,
        horizon: float,
    ) -> ndarray
    .certificate(
        spectral_parameter: complex,
        *,
        horizon: float = 10.0,
        points: int = 1001,
        tolerance: float = 1e-9,
    ) -> Certificate
```

For $B \in \mathbb{C}^{p\times q}$, $D \in \mathbb{C}^{q\times q}$, and $C \in \mathbb{C}^{q\times p}$, the constructor requires the displayed shapes and finite entries. The retained input and output dimension must be the same. `at` and `finite_laplace` require finite nonnegative time or horizon. `evaluate` is a compatibility synonym for `at`.

`MemoryKernel.certificate` checks the finite-plus-tail algebraic decomposition, not quadrature:

$$\rho_{\mathrm{mem}} = \left\| \widehat{K}_T(z) + B(zI - D)^{-1} e^{(D-zI)T} C - \widehat{K}(z) \right\|_2 .$$

The diagnostics record `horizon`, `requested_quadrature_points`, and `tail_norm`. The `points` value must be at least two, but no quadrature is performed and changing it does not change the computed matrices or validity. The certificate is **N** and uses theorem `PSC-VI-T30`.

`RationalMemoryKernel` is exactly the same class object as `MemoryKernel`. It is not a distinct representation.

46.4 Hidden-state realizations

A rational memory kernel can be executed locally by retaining the hidden state:

$$\frac{d}{dt}\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} A & B \\ C & D \end{bmatrix}\begin{bmatrix} x \\ y \end{bmatrix} .$$

Its retained component realizes the Volterra memory equation with the initial hidden-source term included.

```
RationalMemoryRealization(output: Any, hidden: Any, input: Any)
    .output -> ndarray
    .hidden -> ndarray
    .input -> ndarray
    .retained_dimension -> int
    .hidden_dimension -> int
    .kernel() -> MemoryKernel
    .augment(retained_generator: Any) -> ndarray
    .retained_trajectory(
        retained_generator: Any,
        initial_state: Any,
        time: float,
    ) -> ndarray
```

`augment` requires a square retained generator of shape $p \times p$. `retained_trajectory` requires a vector of length $p + q$, exponentiates the full augmented generator, and returns its first p coordinates. It is a dense direct execution, not a Volterra discretization.

46.4.1 Finite local closure

Cayley-Hamilton reduces the closure question to the finite moment list

$$M_j = A_{PQ} A_{QQ}^{j} A_{QP}, \qquad j = 0, \ldots, d_Q - 1.$$

For a hidden dimension d_Q,

$$M_j = 0 \quad \text{for all } j < d_Q \iff K(t) \equiv 0 \iff \text{the retained feedback closes locally.}$$

```
FeshbachReduction
    .memory_kernel() -> MemoryKernel
    .memory_kernel_at(time: float) -> ndarray
    .hidden_source(hidden_initial: Any, time: float) -> ndarray
    .closure_moments() -> tuple[ndarray, ...]
    .local_closure_certificate(
        *, tolerance: float = 1e-10,
    ) -> Certificate
```

`local_closure_certificate` stores one spectral norm per moment as `moment_0`, …, `moment_{d_Q-1}`. It accepts when their maximum is no larger than the tolerance and uses theorem `PSC-VI-T29`. The equivalence at exact zero is **E**. Passing at nonzero tolerance is a numerical near-closure statement, not an exact vanishing proof.

46.4.2 Finite-horizon kernel and trajectory bounds

For two kernels, the implementation samples

$$\delta_K(T) = \int_0^T \|K_1(t) - K_2(t)\|_2 \, dt$$

with trapezoidal quadrature. `kernel_l1_error` returns only that estimate. When both inputs are `MemoryKernel` instances, `memory_kernel_error_bound` augments the estimate by a global derivative allowance and also forms separate exponential-norm bounds. It returns the smaller certified candidate, while retaining the sampled estimate and both allowances in the certificate.

For arbitrary callables, no derivative information is available. The same routine returns only the trapezoidal estimate with exactness `CONDITIONAL` and the unchecked assumption that it upper-bounds the true integral.

```
kernel_l1_error(
    first: MemoryKernel | Callable[[float], Any],
    second: MemoryKernel | Callable[[float], Any],
    horizon: float,
    *, samples: int = 1001,
) -> float

memory_kernel_error_bound(
    first: MemoryKernel | Callable[[float], Any],
    second: MemoryKernel | Callable[[float], Any],
    *, horizon: float,
    samples: int = 1001,
) -> ComputationResult[float]
```

Both require a finite nonnegative horizon and at least two samples. The rational-kernel certificate uses theorem `PSC-VI-T31`, exactness `NUMERICAL`, and residual fields `quadrature_estimate`, `quadrature_allowance`, and `separate_exponential_bound`.

Given declared finite-horizon bounds M for a reference trajectory, a for the retained local generator, and κ for the reference kernel, the source returns

$$\|x_1 - x_2\|_{[0,T]} \le MT\delta_K \exp((a+\kappa)T).$$

```
volterra_trajectory_bound(
    kernel_l1_difference: float,
    horizon: float,
    trajectory_bound: float,
    retained_generator_norm: float | None = None,
    reference_kernel_l1: float | None = None,
    *,
    local_generator_bound: float | None = None,
    kernel_l1_bound: float | None = None,
) -> float
```

The two keyword pairs are compatibility names for the same a and κ. The routine requires one value from each pair and requires every input to be finite and nonnegative. It does not compute or validate the declared trajectory, generator, or reference-kernel bounds. The result is therefore **C** on those caller-supplied hypotheses.

46.5 Phase-type survival models

A phase-type model evolves a column probability vector through a finite transient subgenerator Q. The constructor normalizes its supplied initial vector:

$$p_0 = \frac{p_{\text{supplied}}}{\mathbf{1}^T p_{\text{supplied}}}.$$

For $t \geq 0$,

$$p(t) = e^{tQ} p_0, \qquad S(t) = \mathbf{1}^T p(t), \qquad a = -\mathbf{1}^T Q.$$

The event density and hazard are

$$f(t) = a\, p(t), \qquad h(t) = \frac{f(t)}{S(t)}.$$

The finite admissibility conditions are

$$Q_{ij} \geq 0 \quad (i \neq j), \qquad a_j = -\sum_i Q_{ij} \geq 0, \qquad \max \operatorname{Re} \sigma(Q) < 0.$$

The implementation strengthens the last check to spectral abscissa strictly below `-tolerance`. For cause-specific absorption rows $a^{(c)}$, it requires

$$a_j^{(c)} \geq 0, \qquad \sum_c a_j^{(c)} = a_j$$

within the model tolerance.

The Laplace transform and moments are

$$\mathbb{E}[e^{-sT}] = a(sI - Q)^{-1} p_0,$$

$$\mathbb{E}[T^m] = m!\, \mathbf{1}^T (-Q)^{-m} p_0.$$

Cause densities and the residual initial law are

$$f_c(t) = a^{(c)} p(t), \qquad \sum_c f_c(t) = f(t), \qquad p_t^{\text{res}} = \frac{p(t)}{S(t)}.$$

```
PhaseTypeSurvival(
    subgenerator: Any,
    initial: Any,
    *,
    cause_rates: Any | None = None,
    tolerance: float = 1e-12,
    validate: bool = True,
)
    .subgenerator -> ndarray
    .initial -> ndarray
    .cause_rates -> ndarray | None
    .tolerance -> float
    .states -> int
    .absorption_rates -> ndarray
    .certificate() -> Certificate
    .transient_probability(time: float) -> ndarray
```

```
    .survival(time: float) -> float
    .density(time: float) -> float
    .hazard(time: float) -> float
    .laplace_transform(parameter: float) -> float
    .moment(order: int) -> float
    .cause_density(time: float) -> ndarray
    .residual_initial(time: float) -> ndarray
    .residual_model(time: float) -> PhaseTypeSurvival
    .identities_certificate(
        time: float,
        *, step: float = 1e-6,
    ) -> Certificate
```

By default, construction calls `certificate().require()`. With `validate=False`, it stores even an inadmissible model so that callers can inspect the failed certificate. Subsequent numerical methods are not guarded by an automatic revalidation.

The model certificate has theorem identifier `PSC-VI-T32`, exactness `NUMERICAL`, and residuals:

- `negative_off_diagonal`, the magnitude of the worst forbidden off-diagonal entry.
- `negative_absorption_rate`, the magnitude of the worst negative total absorption rate.
- `cause_rate_sum`, the maximum cause-sum mismatch.
- `spectral_abscissa_failure`, the clipped margin $\max(0, \alpha(Q) + \varepsilon)$, not an exact signed distance to admissibility. At equality $\alpha(Q) = -\varepsilon$, this residual is zero even though the source's strict inequality still makes the certificate invalid.

`hazard` returns positive infinity when survival is no larger than the model tolerance. `laplace_transform` requires a finite nonnegative parameter. `moment` requires a nonnegative integer and returns one at order zero. `cause_density` requires declared causes. A residual restart fails when the survival mass is no larger than tolerance.

`identities_certificate` checks

$$-S'(t) = f(t), \qquad \sum_c f_c(t) = f(t)$$

using a centered difference for the first identity. It requires finite $t \geq h > 0$ and uses a fixed acceptance tolerance 10^{-5}, independent of the model tolerance. The certificate identifier is `PSC-VI-T33`. **E/N**

46.6 Finite completion boundaries

This implementation completes several questions in finite dimension and intentionally leaves stronger questions open.

46.6.1 What is complete

- A declared coordinate split produces all four blocks without ambiguity.
- Whenever the hidden and full resolvents exist, the Schur-Feshbach resolvent identity is exact finite algebra.
- Eliminating a finite hidden linear state yields the displayed source plus Volterra-memory equation.
- Every finite auxiliary realization produces a rational resolvent expression.
- Cayley-Hamilton makes exact local closure decidable from finitely many moments.
- A matrix satisfying the phase-type admissibility conditions defines the finite survival, transform, moment, competing-risk, and restart formulas.

46.6.2 What is not completed by these APIs

- A successful finite resolvent check does not give a uniform spectral region, analytic continuation, or an infinite-dimensional Feshbach theorem.
- The rational expression $B(zI - D)^{-1}C$ is not automatically the Laplace integral on every invertible point. Convergence needs an additional half-plane condition.
- A small closure residual does not prove exact local closure unless the tolerance is zero and arithmetic is exact.
- Finite-horizon kernel estimates do not imply long-time stability.
- The Volterra trajectory formula accepts declared bounds. It does not derive them.
- A generic stable matrix memory kernel is not automatically positive, Markov, or a survival law.
- A phase-type fit does not prove that observed data were generated by that model, identify a unique hidden realization, or establish a continuum lifetime law.
- No minimal-realization, controllability, observability, model-reduction, or statistical-estimation algorithm is supplied in this release.

These boundaries are the correct stopping point for the finite package. A future infinite-dimensional extension would need separate domain, closedness, spectral, convergence, and tightness hypotheses rather than larger finite test sweeps alone.

46.6.3 Runnable Feshbach, memory, and survival panel

The self-contained panel below exercises the finite resolvent, local closure, rational-memory comparison, and phase-type identity paths together.

```
import numpy as np

from komorebi_math.dynamics import (
    FeshbachReduction,
    PhaseTypeSurvival,
    RationalMemoryRealization,
    memory_kernel_error_bound,
)

matrix = np.array([
    [-0.4,  0.2,  0.3,  0.0],
    [ 0.0, -0.6,  0.0, -0.2],
    [ 0.4,  0.0, -1.0,  0.1],
    [ 0.0,  0.5,  0.0, -1.4],
])
reduction = FeshbachReduction(matrix, retained=[0, 1])
resolvent = reduction.resolvent_certificate(2.0 + 0.3j)
resolvent.certificate.require()

first = RationalMemoryRealization([[1.0]], [[-1.0]], [[1.0]])
second = RationalMemoryRealization([[0.9]], [[-1.0]], [[1.0]])
memory_bound = memory_kernel_error_bound(
    first.kernel(),
    second.kernel(),
    horizon=2.0,
    samples=101,
)
memory_bound.certificate.require()

survival = PhaseTypeSurvival(
    [[-0.7, 0.2], [0.3, -0.6]],
    [1.0, 0.0],
)
identity = survival.identities_certificate(0.5)
identity.require()

print("Feshbach residual", resolvent.certificate.residuals["feshbach_resolvent"])
```

```
print("exact local closure", reduction.local_closure_certificate().valid)
print("memory K(1)", first.kernel().at(1.0))
print("memory upper bound", memory_bound.value)
print("survival S(1)", survival.survival(1.0))
print("survival mean", survival.moment(1))
print("identity residuals", identity.residuals)
```

Observed output from the audited source tree:

```
Feshbach residual 1.1259825589158814e-16
exact local closure False
memory K(1) [[0.36787944]]
memory upper bound 0.3672534856322114
survival S(1) 0.6703200460356394
survival mean 2.5
identity residuals {'density_derivative': 9.21618337201835e-12, 'cause_sum': 0.0}
```

The shipped `examples/feshbach_memory.py` additionally prints the complex self-energy, the kernel at zero, and a failed exact-closure result. Its observed output is:

```
self energy at z=2+0.3i:
[[ 0.03960396-0.0039604j   0.00143203-0.00027196j]
 [-0.        +0.j         -0.02918455+0.00257511j]]
memory kernel at zero:
[[ 0.12  0.  ]
 [ 0.   -0.1 ]]
exact local closure: False
```

The failed closure value is expected evidence of retained feedback through the hidden sector, not a failed example.

46.6.4 Complexity and storage

Let p be the retained dimension, q the hidden dimension, $n = p + q$, and S the number of kernel samples.

- One self-energy requires a dense $q \times q$ inverse and the two coupling products, conventionally $O(q^3 + pq^2 + p^2q)$ work and $O(q^2 + pq)$ auxiliary storage. **M**
- The direct projected resolvent in the certificate additionally inverts the full $n \times n$ matrix, so the check is $O(n^3)$. **M**
- One time-domain kernel evaluation requires a hidden matrix exponential and dense products, conventionally $O(q^3)$. **M**
- The closure list contains q retained matrices. The straightforward recurrence stores $O(qp^2)$ output and performs repeated dense products. It is finite but not a Krylov-compressed implementation. **M**
- `kernel_l1_error` performs S evaluations of both kernels. For rational kernels this is $O(Sq^3)$ under a dense-exponential model. **M**
- One phase-type probability or density evaluation uses a dense exponential, while a moment uses a dense inverse and matrix power. No factorization is cached across calls. **M**

All Chapter 46 routines are NumPy CPU routines, except that their shared matrix exponential may use optional SciPy. There is no sparse, GPU, symbolic rational, automatic model-order reduction, or streaming survival tier.

46.6.5 Certificate ledger

As in Chapter 45, pair every theorem identifier with its emitting symbol and package version. An identifier alone is not a complete evidence key.

- `PSC-VI-T28`: Feshbach resolvent. Residual `feshbach_resolvent`. Scaled numerical acceptance.
- `PSC-VI-T29`: local closure. Residuals `moment_j`. Absolute numerical acceptance.
- `PSC-VI-T30`: rational memory transform decomposition. Residual `laplace_decomposition`. Diagnostics include horizon, requested point count, and tail norm.

- PSC-VI-T31: memory_kernel_error_bound emits the rational-kernel upper estimate or callable conditional quadrature, while the attached provenance object carrying this identifier is on volterra_trajectory_bound. Residual fields distinguish the estimate and allowances.
- PSC-VI-T32: phase-type admissibility. Four structural/stability residuals.
- PSC-VI-T33: PhaseTypeSurvival.identities_certificate emits the survival derivative and cause-sum record with fixed 10^{-5} numerical tolerance. The emitting method has no matching attached provenance object in this release.

Every ComputationResult retains both .value and .certificate. .require() returns the value when the certificate is valid and raises the package's certificate failure when it is not. A direct method which needs a matrix inverse can raise before any result record exists.

46.6.6 Error model and troubleshooting

- DimensionError: nonsquare Feshbach matrix, mismatched realization blocks, wrong hidden initial vector, wrong augmented state, wrong retained generator, or wrong survival/cause shape.
- ValidationError: nonfinite matrices, invalid retained indices, no hidden sector, negative/nonfinite memory time or horizon, too few quadrature samples, missing Volterra bounds, invalid survival probability, invalid moment order, unavailable cause rates, or a zero-survival restart. A finite positive constructor tolerance is enforced specifically by PhaseTypeSurvival. Not every certificate method in this chapter validates its tolerance argument.
- numpy.linalg.LinAlgError: a singular hidden, full, transform, or moment inverse.
- Certificate failure: a resolvent residual, local-closure criterion, rational transform decomposition, phase-type admissibility condition, or survival identity exceeds its acceptance rule.
- BackendUnavailableError: only when the shared matrix exponential is forced to method="scipy" without SciPy. Chapter 46 methods call the default auto mode and therefore fall back to Taylor.

46.6.7 Source-observed boundaries and defects

- MemoryKernel.certificate(..., points=...) validates and records points but performs no quadrature. The argument cannot improve this certificate.
- FeshbachReduction.resolvent_certificate, the local-closure certificate, and MemoryKernel.certificate do not consistently reject negative or nonfinite certificate tolerances. Such values can yield invalid records rather than ValidationError. Validate certificate tolerances as finite and positive before these calls.
- FeshbachReduction coerces retained indices with int(...) and does not compare the result with the original value. A numeric value such as 1.5 can be truncated to index one. Supply genuine integers.
- MemoryKernel.certificate forms its horizon expression directly rather than calling finite_laplace. Unlike finite_laplace, it does not explicitly reject a negative or nonfinite horizon. Validate horizon before calling the certificate.
- MemoryKernel.laplace checks invertibility only. It does not check the spectral condition needed to interpret the rational expression as an infinite-horizon Laplace integral.
- RationalMemoryRealization.retained_trajectory and FeshbachReduction.hidden_source do not apply the nonnegative finite-time validation used by MemoryKernel.at. Validate time at the application boundary when forward-time semantics are required.
- PhaseTypeSurvival(validate=False) intentionally bypasses construction-time rejection. Numerical methods remain callable on the invalid object.
- PhaseTypeSurvival does not explicitly prevalidate every subgenerator entry for finiteness. NaN or infinity may reach the eigensolver and raise numpy.linalg.LinAlgError rather than a package validation exception.
- PhaseTypeSurvival.identities_certificate uses a fixed 10^{-5} threshold rather than the model tolerance, and it uses finite differences rather than the available analytic derivative.
- Neither FeshbachReduction nor MemoryKernel estimates resolvent condition numbers. A tiny algebraic residual can coexist with a highly sensitive inverse.
- RationalMemoryKernel and ProjectiveSemigroupTower are aliases. Code should not rely on them to distinguish representations at runtime.

46.6.8 Executed verification evidence

These source-tree checks were executed for the reference baseline with `PYTHONPATH` directed at the delivered repository:

- `test_dynamics_semigroups_markov.py`, `test_dynamics_bayes_martingales.py`, `test_dynamics_fractional_green_feynman_survival.py`, `test_dynamics_feshbach.py`, and `test_block4_hardening.py`: **34 passed**.
- Installed dynamics verification: **11/11 passed**.
- Shipped examples `markov_bayes.py`, `fractional_survival.py`, `feshbach_memory.py`, and `kato_feshbach.py`: **4/4 completed**.

The shipped Markov/Bayesian example reported the coarse generator

$$\begin{bmatrix} -0.1 & 0 \\ 0.1 & 0 \end{bmatrix},$$

coarse posterior `[0.63157895 0.36842105]`, and fixed sufficiency `True`. The fractional/survival example reported square-root spectrum `[1. 2. 3.]`, fractional-transfer diagonal `[0.4965853 0.24659696 0.12245643]`, survival at one `0.6703200460356394`, and mean lifetime `2.5`.

These executions qualify the cited finite source behavior in the audited environment. They do not erase the explicit conditional and infinite-dimensional boundaries above.

Evidence: `src/komorebi_math/dynamics/feshbach.py`, `memory.py`, `survival.py`, `examples/feshbach_memory.py`, `examples/fractional_survival.py`, `examples/markov_bayes.py`, `examples/kato_feshbach.py`, and the focused tests listed above.

Part VIII. Exact finite integration and typed expressions

There are two complementary workflows in this part. Chapters 47 through 49 integrate finite polynomials or sampled callables against explicit measures. Chapters 50 through 52 represent finite rational coordinate calculations as typed expressions that can be checked, evaluated, simplified, planned, and stored. Ordinary integration does not require learning the expression language first.

Your goal	Start here	Keep with the result
Integrate a rational polynomial exactly	Sections 47.1, 47.2, and 48.1	Polynomial, measure, and exact rational inputs
Integrate a callable numerically	Section 48.2	Method, evaluation count, and error-guarantee class
Certify a Simpson error bound	Section 48.3	The matched polynomial, interval, and derivative-bound provenance
Check an integral identity or work in polynomial coordinates	Chapter 49	Both sides of the identity and the basis witness
Build a reusable exact coordinate pipeline	Chapters 50 and 51	Semantic types, exact data, and certificate
Plan or exchange a typed expression	Chapter 52	Budget, canonical expression record, and semantic digest

Use integers and `Fraction` values from the start when rational exactness is intended. A decimal-looking result and an exact rational result may describe similar numbers while carrying different evidence. The worked examples show where that distinction changes the permitted conclusion.

47. Measures and exact polynomials

Integration in Komorebi Math begins with two deliberately small finite objects: a sparse polynomial over $\mathbb{Q}$ and a measure that can return finite monomial moments. This separation is important. A polynomial says what is being integrated. A measure says how monomials are weighted. The dispatcher contracts the two only after their dimensions and certificates agree.

The public objects are exported from `komorebi_math.integration` and from the top-level research API. They are not members of the frozen `komorebi_math.stable` or `komorebi_math.beta` surfaces. All conclusions below are therefore conclusions about the finite integration research API in version `1.0.0b4`.

47.1 `ExactPolynomial`

An `ExactPolynomial` represents

$$p(x_1, \dots, x_d) = \sum_{\alpha \in A} c_\alpha x^\alpha, \qquad x^\alpha = \prod_{j=1}^{d} x_j^{\alpha_j},$$

where $A \subset \mathbb{N}^d$ is finite and every $c_\alpha \in \mathbb{Q}$. The stored representation is immutable, sparse, and canonical: equal exponent tuples are combined, zero coefficients are removed, and the remaining terms are sorted lexicographically.

```
ExactPolynomial(
    dimension: int,
    terms: Mapping[Sequence[int], Rational]
         | Iterable[tuple[Sequence[int], Rational]],
)
```

`Rational` here means `int | fractions.Fraction`. Boolean and floating-point coefficients are rejected. This is not cosmetic type discipline: it prevents an `EXACT` certificate from hiding conversion of a binary float such as `0.1`. The dimension must be positive, each exponent tuple must have that dimension, and every exponent must be a nonnegative integer.

Four constructors cover the common cases:

```
ExactPolynomial.zero(dimension)
ExactPolynomial.constant(value, dimension=1)
ExactPolynomial.monomial(coefficient, exponents)
ExactPolynomial.univariate(coefficients)
```

`univariate((c0, c1, ..., cn))` uses ascending degree order,

$$(c_0, c_1, \dots, c_n) \longmapsto c_0 + c_1 x + \cdots + c_n x^n.$$

An empty coefficient iterable produces the zero polynomial.

For example, an ascending coefficient sequence starts with the constant term, then the linear term, then the quadratic term. Sparse multivariate terms instead pair each coefficient with an exponent tuple whose positions follow the variable order. Confirm that order before comparing a polynomial with a coordinate vector from another basis.

The main query properties are `degree`, `degrees`, and `term_count`. `coefficient(alpha)` returns zero when the canonical term is absent. `to_univariate_coefficients()` returns a dense ascending tuple and rejects a multivariate polynomial.

The finite algebra is exposed by explicit methods rather than Python arithmetic operators:

```
p.add(q)
p.subtract(q)
p.scale(scalar)
p.multiply(q, *, max_terms=100000)
p.power(exponent, *, max_terms=100000)
p.derivative(variable=0, order=1)
p.antiderivative(variable=0, constant=0)
p.evaluate(point)                # p(point) is an alias
```

Addition and multiplication require a common ambient dimension. Powers are nonnegative integers. Differentiation permits order zero. Antidifferentiation adds the supplied constant to the all-zero exponent. The identities are the ordinary finite polynomial identities over $\mathbb{Q}$. In particular,

$$\frac{\partial}{\partial x_j}\left(\sum_{\alpha} c_\alpha x^\alpha\right) = \sum_{\alpha_j > 0} \alpha_j c_\alpha x^{\alpha - e_j}.$$

`multiply`, `power`, and affine pullback enforce `max_terms`. The bound is checked while the sparse expansion is built, so a combinatorial expansion fails explicitly instead of exhausting memory silently. The default budget is 100,000 terms. **E/M**

`integrate_out(variable, lower, upper, *, density=1)` eliminates one variable against a constant-density rational interval. For one monomial it applies

$$\rho \int_a^b x_j^{\alpha_j}\, dx_j = \rho \frac{b^{\alpha_j+1} - a^{\alpha_j+1}}{\alpha_j + 1}.$$

The result is a lower-dimensional `ExactPolynomial`, or a `Fraction` after the final variable is removed. Bounds must satisfy $a \le b$. Oriented bounds are handled separately by `integrate_polynomial_oriented` in Section 48.1.2.

An exact affine pullback is available both as

```
p.pullback_affine(matrix, shift, *, max_terms=100000)
mapping.pullback(p, *, max_terms=100000)
```

For $T(x) = Mx + b$, it constructs $p \circ T$ by substituting one exact linear polynomial for every target coordinate. The matrix shape is `p.dimension x source_dimension`. The output lives in the source dimension.

```
from fractions import Fraction as F

from komorebi_math.integration import ExactPolynomial

p = ExactPolynomial(2, {(2, 0): 1, (0, 1): 3})
q = p.derivative(0)
r = p.antiderivative(1).derivative(1)

print(p((2, 5)))
print(q((2, 5)))
print(r == p)
print(p.multiply(p).degree)
```

Observed output:

```
19
4
True
4
```

The `certificate()` result has theorem identifier `INT.POLY.1` and records dimension, total degree, degree in each variable, and term count. It certifies the canonical finite rational representation, not analytic regularity beyond polynomials. **E**

47.2 Atomic, interval, product, and pushforward measures

47.2.1 The finite `Measure` protocol

The runtime-checkable `Measure` protocol is intentionally minimal:

```
dimension: int
name: str
exactness: Exactness

mass() -> ComputationResult[Any]
moment(exponents: Sequence[int]) -> ComputationResult[Any]
certificate() -> Certificate
```

For a measure μ on a finite-dimensional space, its monomial moment is

$$m_\alpha(\mu) = \int x^\alpha \, d\mu(x).$$

Consequently a sparse polynomial integral is the finite contraction

$$\int p \, d\mu = \sum_{\alpha \in A} c_\alpha m_\alpha(\mu).$$

Implementing the Python protocol is not by itself a proof that a custom object's moments are mathematically correct. `integrate` first requires the structural certificate and propagates its exactness and unchecked assumptions. An exact result requires exact value closure as well as exact, unconditional measure and moment certificates.

47.2.2 Atomic measures

```
AtomicMeasure(
    points: Iterable[Any],
    weights: Iterable[Any],
    *,
    name: str = "atomic measure",
)
```

This object represents the positive finite measure

$$\mu = \sum_{i=1}^{N} w_i \, \delta_{x_i}, \qquad w_i \geq 0.$$

Scalar points define a one-dimensional measure. Tuple-like points define the ambient dimension. The point list must be nonempty, all point dimensions must agree, and the number of weights must equal the number of points. Coordinates and weights may be exact integers/Fractions or finite reals, but negative weights and nonfinite values are rejected.

The basic formulas are finite sums:

$$\mu(X) = \sum_i w_i, \qquad m_\alpha(\mu) = \sum_i w_i x_i^\alpha.$$

When all canonical coordinates and weights are rational, `exactness` is `Exactness.EXACT`. Otherwise it is `NUMERICAL`. `mass()` and `moment()` preserve arbitrary-size exact integers and Fractions. Numerical sums use finite-range checks and accurate scalar summation.

`normalized()` divides every weight by the total mass and rejects a zero-mass measure or numerical underflow. `coalesced()` merges exactly equal canonical point tuples while preserving their first-occurrence order. If zero weights leave no positive atom, it retains one zero-weight atom so the class invariant of nonempty support remains true. Because constructor weights are nonnegative, coalescing never turns this object into a signed functional.

```
from fractions import Fraction as F

from komorebi_math.integration import AtomicMeasure, integrate

mu = AtomicMeasure(
    (-1, 0, 2),
    (F(1, 2), F(1, 3), F(1, 6)),
)
result = integrate(lambda x: (1, x, x * x), mu)

print(result.exact)
print(tuple(result.value))
```

Observed output:

```
True
(Fraction(1, 1), Fraction(-1, 6), Fraction(7, 6))
```

The callable is evaluated once per atom. All outputs must retain one common shape. A scalar at one atom and a vector at another raises `DimensionError`. **E/N**

47.2.3 Interval measures

```
IntervalMeasure(
    lower: Any,
    upper: Any,
    *,
    density: Any = 1,
    name: str = "interval measure",
)
```

This is the positive constant-density measure

$$d\mu(x) = \rho\, dx \quad \text{on } [a, b], \qquad a \le b, \quad \rho \ge 0.$$

Its mass and moments are

$$\mu([a, b]) = \rho(b - a), \qquad m_k(\mu) = \rho \frac{b^{k+1} - a^{k+1}}{k + 1}.$$

Integer or Fraction endpoints and density give exact rational data. Any floating input gives a numerical measure. `structurally_zero` is true when `lower == upper` or `density == 0`. Integration over that measure returns exact zero without evaluating a callable integrand. Reversed endpoints and negative density are rejected because this class models a positive measure, not an oriented integral.

```
from fractions import Fraction as F

from komorebi_math.integration import (
    ExactPolynomial,
    IntervalMeasure,
    integrate,
)

mu = IntervalMeasure(-1, 2, density=F(3, 2))
p = ExactPolynomial.univariate((4, -2, 1))

print(tuple(mu.moment((k,)).require() for k in range(5)))
print(integrate(p, mu).require())
```

Observed output:

```
(Fraction(9, 2), Fraction(9, 4), Fraction(9, 2),
 Fraction(45, 8), Fraction(99, 10))
18
```

`normalized()` replaces the density by $1/(b-a)$, independent of the original nonzero density, and rejects a zero interval. Numerical mass, moment, normalization, and pushforward paths reject detected overflow, underflow, collapsed distinct endpoints, and nonfinite results. **E/N**

47.2.4 Product measures

```
ProductMeasure(
    factors: Iterable[Measure],
    *,
    name: str = "product measure",
)

product_measure(*factors, name="product measure")
```

The factors must form a nonempty finite tuple of `Measure` objects. The dimension is the sum of factor dimensions, and exactness is the weakest factor status. For an exponent tuple split compatibly as $\alpha = (\alpha^{(1)}, \dots, \alpha^{(r)})$,

$$m_\alpha(\mu_1 \otimes \cdots \otimes \mu_r) = \prod_{j=1}^{r} m_{\alpha^{(j)}}(\mu_j).$$

The factor mass, moment, and structural certificates are incorporated into the returned certificate. An invalid or conditional factor cannot be hidden by the product wrapper.

A product of marginals asserts independence. It is not interchangeable with an arbitrary joint atomic law. For the two Bernoulli marginals

$$\mu_X = \mu_Y = \tfrac{1}{2}\delta_0 + \tfrac{1}{2}\delta_1,$$

the product gives $\mathbb{E}[XY] = 1/4$. The joint law $\frac{1}{2}\delta_{(0,0)} + \frac{1}{2}\delta_{(1,1)}$ gives $1/2$. The shipped tests check this distinction directly.

47.2.5 Pushforward measures and exact affine maps

For a map $T : X \to Y$, the pushforward is defined by preimage:

$$(T_{\#}\mu)(B) = \mu(T^{-1}(B)).$$

Its integration law is

$$\int_Y f \, d(T_{\#}\mu) = \int_X f \circ T \, d\mu.$$

The exact map type is

```
ExactAffineMap(
    matrix: Iterable[Iterable[Rational]],
    shift: Iterable[Rational],
    *,
    name: str = "exact affine map",
)
```

and represents $T(x) = Mx + b$ over $\mathbb{Q}$. It exposes `source_dimension`, `target_dimension`, `square`, `apply_point`, callable evaluation, `then`, `pullback`, `determinant`, and `certificate`. Composition `first.then(second)` means `second(first(x))`. `determinant()` requires a square linear part. Rectangular affine maps remain valid typed maps.

```
PushforwardMeasure(
    base: Measure,
    mapping: ExactAffineMap | Callable[[Any], Any],
    *,
    target_dimension: int | None = None,
    name: str = "pushforward measure",
)

pushforward_measure(base, mapping, *, target_dimension=None, name=...)
```

An exact affine map determines its target dimension and preserves the base exactness status. An arbitrary callable requires an explicit positive `target_dimension` and yields at least `CONDITIONAL` status. Its certificate records unchecked global definition, target-dimension, determinism, purity, and finiteness assumptions. Testing the callable on finite atoms does not prove those global properties.

`materialize_atomic()` is available only when the base is atomic. It maps the support once and coalesces equal images. For example, pushing four atoms at $-2, -1, 1, 2$, with source weights $(1/6, 1/3, 1/4, 1/4)$, through $x \mapsto x^2$ produces atoms at 1 and 4 with weights $7/12$ and $5/12$. With a callable map this is explicitly a snapshot. Callable purity and determinism remain unchecked.

For the one-dimensional affine map $y = sx + t$, `IntervalMeasure.pushforward_affine` materializes the image. If $s \neq 0$,

$$[a, b] \longmapsto [\min(sa + t, sb + t), \max(sa + t, sb + t)], \qquad \rho' = \frac{\rho}{|s|}.$$

If $s = 0$, the image is the one-atom measure $\rho(b - a)\delta_t$. Thus mass is preserved in both cases. The absolute Jacobian belongs to the density of the materialized image. It is not an extra factor in the abstract pushforward integration identity.

47.2.6 Two canonical moment ladders

Moment sequences make the finite contract visible before any general polynomial is introduced. For the Bernoulli atomic measure

$$\mu_p = (1-p)\delta_0 + p\delta_1, \qquad 0 \le p \le 1,$$

the exact moments are

$$m_0(\mu_p) = 1, \qquad m_k(\mu_p) = p \quad (k \ge 1).$$

For the constant-density probability measure on $[0, 1]$,

$$m_k = \int_0^1 x^k \, dx = \frac{1}{k+1}.$$

`AtomicMeasure` and `IntervalMeasure` reproduce these ladders as `Fraction` values when p, the endpoints, and the density are rational. A floating parameter produces numerical status instead of being relabelled exact.

47.3 Positive measures versus signed quadrature functionals

An `AtomicMeasure` requires $w_i \ge 0$. A `QuadratureRule`, introduced in Chapter 48, represents the more general linear functional

$$Q[f] = \sum_i w_i f(x_i)$$

and permits negative weights. These notions must not be conflated:

- positivity gives a measure and supports measure semantics.
- Signed weights can still define a valid exact finite functional.
- Exact nodes and weights make evaluation exact but do not prove $Q[f] = \int f \, d\mu$.
- `QuadratureRule.as_atomic_measure()` succeeds only when every weight is nonnegative.

Likewise, reversed bounds are not encoded by a negative interval density. `integrate_polynomial_oriented` records the sign separately while its underlying interval measure stays positive.

47.3.1 Failure contracts, cost, and persistence boundary

Operation	Principal finite cost	Explicit guard or failure
sparse evaluation	proportional to stored term powers	point dimension. Numeric overflow
polynomial product	up to the product of term counts	`max_terms`
affine pullback	expansion-dependent	`max_terms`. Matrix typing
atomic moment	atoms times dimension	point/weight shape. Finite range
interval moment	endpoint exponentiation	ordered endpoints. Finite range
product moment	sum of factor work plus products	factor certificates. Under/overflow
atomic materialization	base atom count	atomic base. Mapped dimension

`ExactPolynomial.to_dict()` emits a plain dictionary, but these integration objects are not among the frozen portable codecs. The integration tests verify that the portable codec contract remains unchanged. Treat ad hoc dictionaries as application data unless a later chapter names a strict codec.

Source anchors: `integration/polynomial.py`, `integration/measures.py`, `integration/quadrature.py`, `tests/test_block19_integration.py`, and `verification/block19_math_verifier.py`.

48. Integration providers and quadrature

The central dispatcher selects a finite algorithm from the integrand and measure types. It does not infer an analytic theorem from successful sampling. Every returned value therefore travels with method, certificate, evaluation count, and an explicitly classified error statement.

Read an integral result in three steps. First inspect or require the certificate. Next inspect `error.guarantee` to distinguish an exact integral, a rigorous bound, an estimate, a conditional statement, and an unknown error. Finally read the value and any bound in the context of the actual integrand and measure. Certificate validity by itself does not mean that a quadrature estimate equals the target integral.

48.1 Exact finite integration

48.1.1 Result and error contracts

```
integrate(
    integrand: ExactPolynomial | Callable[[Any], Any],
    measure: Measure,
    *,
    provider: CompositeSimpson | None = None,
) -> IntegralResult
```

`IntegralResult` contains:

```
value: Any
certificate: Certificate
error: IntegrationError
method: str
evaluations: int
valid -> bool
exact -> bool
require() -> Any
```

`require()` requires certificate validity and then returns `value`. The `exact` property is narrower than certificate validity: it is true only when `error.guarantee` is `ErrorGuarantee.EXACT`.

```
IntegrationError(
    absolute_bound: Fraction | float | None,
    guarantee: ErrorGuarantee,
    sources: tuple[str, ...] = (),
)
```

The guarantee enum is

Value	Meaning
`EXACT`	the library-produced exact path carries zero absolute error
`RIGOROUS`	a proved finite absolute bound, possibly zero, is attached
`CONDITIONAL`	the stated bound depends on unchecked assumptions
`ESTIMATED`	a finite diagnostic estimate, not an enclosure theorem
`UNKNOWN`	no target-integral error bound is supplied

`EXACT` and `RIGOROUS` require a non-`None` nonnegative finite bound. `IntegrationError.exact()` constructs the zero bound used by library-generated exact results. The public dataclass constructor currently does not itself enforce that a caller-labelled `EXACT` record has bound zero, so such a record must not be treated as authoritative merely because it was constructed. The property `error.rigorous` is true for both exact and rigorous guarantees.

48.1.2 Moment contraction, oriented bounds, and coordinate adapters

When the integrand is an `ExactPolynomial` and no provider is supplied, `integrate` contracts its sparse coefficients with measure moments:

$$I = \sum_{\alpha} c_{\alpha} m_{\alpha}(\mu).$$

An exact affine pushforward first constructs the exact pullback and integrates against the base measure. A callable pushforward over an atomic base is materialized as a finite snapshot and retains the callable assumptions. All other supported exact measures provide moments directly.

The result is exact only when every structural and moment certificate is exact and unconditional and all values remain exact. A float-valued moment cannot be upgraded merely because the sparse contraction completes.

```
integrate_polynomial_oriented(
    polynomial: ExactPolynomial,
    lower: int | Fraction,
    upper: int | Fraction,
    *,
    density: int | Fraction = 1,
) -> IntegralResult
```

This helper implements

$$\int_b^a p(x)\rho\,dx = -\int_a^b p(x)\rho\,dx$$

without pretending that a negative density is a positive measure. It accepts only a univariate exact polynomial, rational endpoints, and nonnegative density. Equal endpoints return exact zero.

Two coordinate adapters construct an exact polynomial before using the same moment path:

```
polynomial_from_family_coordinates(family, coefficients)
integrate_polynomial_coordinates(family, coefficients, measure)

polynomial_from_formal_series(series, *, degree=None)
integrate_formal_series(series, measure, *, degree=None)
```

The first expands a finite exact `PolynomialFamily` coordinate vector and rejects coefficients that are not `int` or `Fraction`. The second consumes a finite normalized `FormalSeries`'s already stored `Fraction` coefficients, optionally through the requested degree. It does not re-audit the provenance of values supplied when that series was created: a prior float may already have been converted to its exact binary `Fraction` value. Use explicit `Fraction` inputs when the intended rational value matters.

48.2 Composite quadrature rules

48.2.1 Callable dispatch

For a callable integrand, dispatch is determined by the measure:

Measure	Finite provider
`AtomicMeasure`	one weighted sample per atom
`IntervalMeasure`	composite Simpson, default or supplied
`ProductMeasure`	finite atomic product sum or tensor Simpson construction
`PushforwardMeasure`	atomic image snapshot or composition with the base map

Atomic callable integration can be exact when the nodes, weights, and outputs are exact rational data. Numerical scalar sums use `math.fsum`. Vector or matrix outputs must preserve a common shape.

For product measures, a fully atomic support is enumerated as a finite product sum. Otherwise interval factors generate tensor-product Simpson nodes. The node count grows multiplicatively with factor counts, and `max_evaluations` is checked before evaluation. The tensor fine/coarse difference is `ESTIMATED`. A one-dimensional derivative bound is explicitly rejected as a certificate for a multivariate tensor rule.

For a pushforward, the callable is composed with the mapping and integrated over the base. A derivative bound on f cannot be reused automatically for $f \circ T$. Supplied bounds on pushforward quadrature are rejected because the composed derivative provenance has changed.

```
integrate_derivative_provider(
    provider: Any,
    measure: Measure,
    *,
    quadrature: CompositeSimpson | None = None,
) -> IntegralResult
```

This adapter integrates a provider's `value(point)` map. The provider must expose positive integer `source_dimension` and `target_dimension`. Its source dimension must equal the measure dimension, and each value must have shape `(target_dimension,)`. It does not by itself prove differentiation under an integral sign. That finite fixed-weight contract is addressed in Section 49.6.

48.2.2 Quadrature rules are finite functionals

```
QuadratureRule(points, weights, *, name="quadrature rule")
ExactQuadratureRule(points, weights, *, name="exact quadrature rule")
```

Both classes implement

$$Q[f] = \sum_{i=1}^{N} w_i f(x_i).$$

`ExactQuadratureRule` requires rational nodes and rational signed weights. The base `QuadratureRule` accepts finite numerical data. Important members are:

```
node_count
exact_data
positive
apply(function) -> IntegralResult
moment(exponents) -> Any
integrate_polynomial(polynomial) -> IntegralResult
tensor_product(other) -> QuadratureRule
```

```
as_atomic_measure() -> AtomicMeasure
degree_of_exactness(measure, *, max_degree, tolerance=1e-12)
    -> QuadratureExactness
certify_polynomial_integral(polynomial, measure) -> IntegralResult
certificate() -> Certificate
```

`apply` and `integrate_polynomial` certify only evaluation of the displayed finite functional. Their target-measure error guarantee is `UNKNOWN` until a moment theorem is supplied.

In one dimension, `degree_of_exactness` compares consecutive moments from degree zero through `max_degree`. For exact rule and measure data it uses exact equality. Otherwise it records numerical residuals and the supplied tolerance. Its result stores the last consecutive matching degree, the maximum tested degree, whether the comparison itself was exact, and a certificate.

`certify_polynomial_integral` is stricter: all three objects must be one-dimensional, moment comparison must be exact through the polynomial's degree, and the certified degree must cover that degree. Only then does it return an exact integral result.

Consider Simpson's three-node rule on $[-1, 2]$:

```
from fractions import Fraction as F

from komorebi_math.integration import (
    ExactPolynomial,
    ExactQuadratureRule,
    IntervalMeasure,
    integrate,
)

rule = ExactQuadratureRule(
    (-1, F(1, 2), 2),
    (F(1, 2), 2, F(1, 2)),
    name="exact Simpson",
)
mu = IntervalMeasure(-1, 2)
cubic = ExactPolynomial.univariate((-5, 3, -2, 1))
quartic = ExactPolynomial.univariate((0, 0, 0, 0, 1))

print(rule.degree_of_exactness(mu, max_degree=7).degree)
print(rule.certify_polynomial_integral(cubic, mu).require())
print(rule.integrate_polynomial(quartic).require())
print(integrate(quartic, mu).require())
```

Observed output:

```
3
-51/4
69/8
33/5
```

The exact finite rule evaluates the quartic exactly as a functional, but it does not equal the quartic integral. Calling `certify_polynomial_integral(quartic, mu)` raises `ValidationError`. This is the critical distinction between exact arithmetic and exact quadrature.

An even sharper adversary vanishes at $0, 1/2, 1$ while its exact integral is $1/840$. The shipped verifier confirms that the sampled value is zero, the error guarantee remains `UNKNOWN`, and a false exactness certificate is blocked.

48.2.3 Composite Simpson rules

```
CompositeSimpson(
    subintervals: int = 64,
    derivative_bound: DerivativeBound | None = None,
    max_evaluations: int = 1000000,
)
```

`subintervals` must be at least four and a multiple of four. This ensures that both the fine rule with n panels and the coarse rule with $n/2$ panels are valid Simpson rules. For $h = (b-a)/n$, the fine rule is

$$Q_n[f] = \rho\frac{h}{3}\left(f(a) + f(b) + 4\sum_{j\,\text{odd}} f(a+jh) + 2\sum_{\substack{j\,\text{even}\\0<j<n}} f(a+jh)\right).$$

The implementation evaluates the fine and coarse rules separately. Its recorded evaluation count is

$$(n+1) + (n/2+1) = \frac{3n}{2} + 2.$$

For a general callable with no derivative bound, it returns $Q_n[f]$ and the Richardson diagnostic

$$\widehat{E} = \frac{\|Q_n[f] - Q_{n/2}[f]\|}{15}.$$

This is `ErrorGuarantee.ESTIMATED` and the certificate status is `Exactness.HEURISTIC`. The method does not assert that the diagnostic encloses the true error. Finite samples do not establish smoothness. **H**

In a convergence study, preserve the panel counts and compare successive values on the same integrand and interval. A decreasing fine/coarse difference is useful observed behavior. To report an enclosing error bound, follow the separate provenance-matched construction in Section 48.3 rather than relabelling that observed difference.

For an exact polynomial on an exact rational interval, the same fine and coarse rules are built entirely with `Fraction` nodes and weights. The quadrature value and Richardson difference are therefore exact rational numbers even when the truncation estimate is only heuristic. Again, exact evaluation of Q_n is not automatically an exact integral.

48.3 Rigorous Simpson bounds

For a scalar function with

$$\sup_{x\in[a,b]} |f^{(4)}(x)| \le M_4,$$

the composite Simpson truncation theorem gives

$$\left|\rho\int_a^b f(x)\,dx - Q_n[f]\right| \le \frac{|\rho|(b-a)h^4}{180}M_4.$$

The proof status of M_4 is represented explicitly:

```
DerivativeBound(
    order,
    bound,
    certificate,
    description="derivative supremum bound",
    polynomial=None,
    interval=None,
)
```

```
DerivativeBound.assumed(order, bound, *, description=...)

polynomial_derivative_bound(
    polynomial,
    measure,
    *,
    order=4,
) -> DerivativeBound
```

An internally generated polynomial bound differentiates exactly and uses the coefficient triangle inequality. With $R = \max(|a|, |b|)$ and $p^{(4)}(x) = \sum_k d_k x^k$, it returns

$$M_4 = \sum_k |d_k| R^k.$$

This bound is conservative but exact and replayable. Its provenance includes the exact polynomial and exact rational interval. The Simpson path requires that both match the actual integrand and measure. A bound computed for another polynomial or interval is rejected.

```
from fractions import Fraction as F

from komorebi_math.integration import (
    CompositeSimpson,
    ExactPolynomial,
    IntervalMeasure,
    integrate,
    polynomial_derivative_bound,
)

p = ExactPolynomial.univariate((0, 0, 0, 0, 1))
mu = IntervalMeasure(0, 1)
bound = polynomial_derivative_bound(p, mu)
result = integrate(p, mu, provider=CompositeSimpson(8, bound))

print(result.value)
print(result.error.absolute_bound)
print(result.error.guarantee.value)
print(abs(result.value - F(1, 5)) <= result.error.absolute_bound)
```

Observed output:

```
1229/6144
1/30720
rigorous
True
```

The actual error equals the reported bound in this quartic example. The certificate records 14 evaluations and the exact fine/coarse difference `1/2048`.

`DerivativeBound.assumed` creates a visible conditional boundary. For a callable, the code also records that the bound applies to this callable and interval and that floating node, evaluation, weight, and summation roundoff must lie within the stated bound. It therefore cannot produce an internally `RIGOROUS` result. Passing an internally rigorous polynomial bound with a lambda is rejected: provenance belongs to the exact `ExactPolynomial`, not to observationally similar source code. **C**

48.4 Estimated Richardson error

Without matched theorem provenance, refinement comparison remains useful as a diagnostic:

```
import numpy as np

from komorebi_math.integration import CompositeSimpson, IntervalMeasure, integrate

result = integrate(
    lambda x: np.exp(-x * x),
    IntervalMeasure(0, 1),
    provider=CompositeSimpson(32),
)

print(result.error.guarantee.value)
print(result.certificate.exactness.value)
```

Observed output:

```
estimated
heuristic
```

No amount of agreement between the two finite grids changes this status. A function can vanish on both node sets while differing elsewhere. Use the estimate for diagnostics or convergence studies. Use a provenance-matched bound for a certified enclosure.

48.5 Provenance-matched integrands and bounds

The proof status of a quadrature result follows the complete provenance chain: the concrete integrand, the concrete measure, the derivative bound, and the finite arithmetic path must all match. A bound derived from one polynomial or interval cannot certify another, and an exact functional evaluation cannot by itself certify equality with a target integral.

48.5.1 Provider selection and failure table

Situation	Result or rejection
exact polynomial, no provider	moment contraction
exact polynomial plus Simpson	exact rational Simpson on an exact interval only
callable plus atomic measure	finite weighted sum
callable plus interval	fine/coarse Simpson
callable plus product	atomic product or tensor Simpson
callable plus pushforward	compose with map. Strip unrelated bound
one-dimensional bound on a product	rejected
rigorous polynomial bound on a lambda	rejected
Simpson panels not a multiple of four	rejected
planned evaluations above `max_evaluations`	rejected before evaluation
output shape changes between nodes	rejected
nonfinite output, node, weight, or sum	rejected
structurally zero interval	exact zero. Callable not evaluated

The shipped integration suite also exercises floating underflow and overflow, huge exact integers, complex numerical outputs, invalid custom measure certificates, and adversarial quadrature rules. The point of these guards is not to make finite quadrature universally rigorous. It is to ensure that the result's proof status matches the finite evidence actually available.

48.5.2 Executable exactness and rejection ladder

This compact ladder checks a degree boundary, a tensor moment, an assumed bound, and two rejection paths without turning any of them into a stronger claim:

```
from fractions import Fraction as F

from komorebi_math.exceptions import ValidationError
from komorebi_math.integration import (
    CompositeSimpson,
    DerivativeBound,
    ExactQuadratureRule,
    IntervalMeasure,
    integrate,
)

trapezoid = ExactQuadratureRule((0, 1), (F(1, 2), F(1, 2)))
degree = trapezoid.degree_of_exactness(
    IntervalMeasure(0, 1), max_degree=3
).degree
tensor_moment = trapezoid.tensor_product(trapezoid).moment((1, 1))

assumed = DerivativeBound.assumed(4, 24)
conditional = integrate(
    lambda x: x**4,
    IntervalMeasure(0, 1),
    provider=CompositeSimpson(8, assumed),
)

rejected_panels = []
for count in (2, 6, 5):
    try:
        CompositeSimpson(count)
    except ValidationError:
        rejected_panels.append(count)

signed = ExactQuadratureRule((0, 1), (1, -1))
signed_rejected = False
try:
    signed.as_atomic_measure()
except ValidationError:
    signed_rejected = True

print(degree, tensor_moment)
print(conditional.error.guarantee.value)
print(rejected_panels, signed_rejected)
```

Observed output:

```
1 1/4
conditional
[2, 6, 5] True
```

The same polynomial with a provenance-matched bound from `polynomial_derivative_bound` follows the rigorous path shown in Section 48.3. The assumed callable bound remains conditional by construction.

Source anchors: `integration/quadrature.py`, `integration/rules.py`, `integration/adapters.py`, `tests/test_block19_integration.py`, `verification/block19_math_verifier.py`, and `examples/block19_measure_integration_worked_example.py`.

49. Integral identities and coordinate covectors

Successful finite calculations can be recorded as replayable finite identities. Every identity computes both sides and stores them with a certificate. It does not merely label one side with a theorem name.

Choose an identity constructor by the change being made: antidifferentiation for the finite polynomial FTC, rearranging a product derivative for integration by parts, changing integration order for finite Fubini, and changing variables for pushforward identities. Coordinate covectors express the same integration functional in a chosen basis. The example in Section 49.7.1 brings those coordinate and change-of-variable routes together.

```
IntegralIdentity(
    name: str,
    left: Any,
    right: Any,
    certificate: Certificate,
)

valid -> bool
require() -> IntegralIdentity
```

`require()` requires the certificate and returns the identity record, so callers can inspect both exact sides and diagnostics.

49.1 FTC

The fundamental theorem of calculus identity is

$$p \in \mathbb{Q}[x],\ a, b \in \mathbb{Q} \quad \Longrightarrow \quad \int_a^b p'(x)\,dx = p(b) - p(a).$$

This is the implemented finite rational-polynomial identity, not a claim of a general analytic FTC.

```
fundamental_theorem(
    polynomial: ExactPolynomial,
    measure: IntervalMeasure,
) -> IntegralIdentity
```

The argument named `polynomial` is the integrand. The function constructs its exact zero-constant antiderivative F and checks

$$\int_a^b p(x)\rho\,dx = \rho(F(b) - F(a)).$$

Thus, to express the $\int p' = p(b) - p(a)$ form, pass `primitive.derivative()`.

```
from fractions import Fraction as F

from komorebi_math.integration import (
    ExactPolynomial,
    IntervalMeasure,
    fundamental_theorem,
```

```
)

primitive = ExactPolynomial.univariate((7, -5, F(3, 2), F(-2, 3), F(1, 4)))
identity = fundamental_theorem(
    primitive.derivative(),
    IntervalMeasure(-1, 2),
).require()

print(identity.left)
print(identity.right)
```

Observed output:

```
-51/4
-51/4
```

The function requires a univariate exact polynomial and an unconditionally exact `IntervalMeasure`. Its theorem identifier is `INT.ID.FTC`. **E**

49.2 Integration by parts

```
integration_by_parts(
    left_factor: ExactPolynomial,
    right_factor: ExactPolynomial,
    measure: IntervalMeasure,
) -> IntegralIdentity
```

For exact univariate u, v, the checked identity is

$$\rho \int_a^b u(x)v'(x)\,dx = \rho[u(x)v(x)]_a^b - \rho \int_a^b u'(x)v(x)\,dx.$$

The code builds the polynomial products and derivatives, evaluates both integrals by exact moments, and retains the boundary term in certificate diagnostics. With $u = 1 + x^2$, $v = -2x + x^3$, and $[-1, 2]$, both sides are $84/5$. Omitting the boundary term would fail this exact check.

49.3 Finite Fubini

```
fubini_identity(
    polynomial: ExactPolynomial,
    measure: ProductMeasure,
    *,
    order: tuple[int, ...] | None = None,
) -> IntegralIdentity

all_fubini_orders(
    polynomial: ExactPolynomial,
    measure: ProductMeasure,
    *,
    max_dimensions: int = 7,
) -> tuple[IntegralIdentity, ...]
```

The supported theorem is finite and exact. The polynomial dimension must equal the product dimension, and the factors must be either:

- all exact one-dimensional interval measures, or
- all exact one-dimensional atomic measures.

A mixed interval/atomic product is not accepted by this identity constructor, even though other integration paths may evaluate it. Every factor and the product must have valid, unconditionally exact certificates.

For interval factors, `fubini_identity` repeatedly uses `integrate_out`, tracking how original variable labels move as dimensions disappear. For atomic factors, it evaluates the finite nested weighted sums. The requested `order` must be a permutation of every variable.

For

$$p(x, y) = x^2y + 3xy^2, \qquad (x, y) \in [0, 1] \times [1, 2],$$

both possible iterated orders and the direct moment contraction equal 4. `all_fubini_orders` enumerates all $d!$ permutations and refuses dimensions above `max_dimensions`, which defaults to seven. This is a finite factorial resource guard, not a statement about infinite product spaces. **E/M**

49.4 Affine pushforward and Jacobians

Two related identities keep abstract pushforward semantics separate from a materialized density formula.

```
pushforward_identity(
    polynomial: ExactPolynomial,
    base: Measure,
    mapping: ExactAffineMap,
) -> IntegralIdentity
```

This certifies

$$\int_Y p(y)\, d(T_{\#}\mu)(y) = \int_X p(Tx)\, d\mu(x)$$

by exact affine pullback and exact base moments. There is no extra Jacobian factor: the left side already integrates against the pushforward measure.

```
affine_interval_change_of_variables(
    polynomial: ExactPolynomial,
    base: IntervalMeasure,
    mapping: ExactAffineMap,
) -> IntegralIdentity
```

For $y = sx + t$ with $s \neq 0$, this separately materializes the image interval and its density $\rho/|s|$, then checks equality with the pullback integral. The transformed interval is ordered, so the absolute Jacobian handles a negative scale. A zero scale is valid as a pushforward to an atom but is rejected by the interval Jacobian identity.

The complete worked example uses

$$x \in [0, 1], \qquad y = 2x + 1, \qquad f(y) = y^4 - 2y + 1.$$

The image measure is $\frac{1}{2}dy$ on $[1, 3]$, and

$$f(2x + 1) = 16x^4 + 32x^3 + 24x^2 + 4x.$$

All three exact computations agree:

$$\int f\,d(T_{\#}\mu) = \frac{1}{2}\int_1^3 f(y)\,dy = \int_0^1 f(2x+1)\,dx = \frac{106}{5}.$$

49.5 Coordinate integration covectors

Let $q_0, \dots, q_n$ be an exact graded polynomial basis and write

$$p(x) = \sum_{k=0}^{n} c_k q_k(x).$$

Integration is a covector

$$\ell_\mu = (\ell_0, \dots, \ell_n), \qquad \ell_k = \int q_k\,d\mu, \qquad \int p\,d\mu = \ell_\mu c.$$

```
PolynomialIntegrationCovector(
    family: str,
    degree: int,
    measure: IntervalMeasure,
    weights: tuple[Fraction, ...],
    basis_witness: tuple[tuple[Fraction, ...], ...] | None = None,
)

polynomial_integration_covector(
    family: PolynomialFamily,
    degree: int,
    measure: IntervalMeasure,
) -> PolynomialIntegrationCovector
```

The finite covector has length `degree + 1`. `apply(coefficients)` requires the same length and rational coordinates, then returns the exact pairing.

A subtle but essential provenance rule applies. Direct construction with a family name and weights creates a usable rational functional, but without a `basis_witness` the claim that those weights are moments of the named family is `CONDITIONAL`. The factory `polynomial_integration_covector` obtains every basis polynomial from the actual `PolynomialFamily`, recomputes all interval moments, stores the witness, and returns an unconditionally exact covector. The property `basis_verified` reports this distinction. Directly supplied stored weights must be actual `Fraction` objects. Use the factory for ordinary construction and replayable provenance.

For a family derivative matrix D, endpoint evaluation row vectors `ev_a`, `ev_b`, and constant density ρ,

```
coordinate_fundamental_theorem(family, degree, measure)
```

checks the coordinate FTC

$$\ell_\mu D = \rho(\mathrm{ev}_b - \mathrm{ev}_a).$$

If $C_{s\to t}$ maps source coordinates to target coordinates, the same physical functional transforms contragrediently:

$$c_t = C_{s\to t} c_s, \qquad \ell_t C_{s\to t} = \ell_s.$$

```
coordinate_transport_identity(
    source: PolynomialFamily,
    target: PolynomialFamily,
    degree: int,
    measure: IntervalMeasure,
) -> IntegralIdentity
```

The shipped verifier checks coordinate FTC through degree seven in the monomial, Euler, Fibonacci, and Legendre families, and checks Euler to Fibonacci covector transport exactly.

```
from komorebi_math.coordinates import (
    connection_transform,
    euler_polynomial_family,
    monomial_polynomial_family,
)
from komorebi_math.integration import (
    ExactPolynomial,
    IntervalMeasure,
    polynomial_integration_covector,
)

p = ExactPolynomial.univariate((1, -2, 0, 0, 1))
mu = IntervalMeasure(1, 3, density=1 / 2)  # see exact version below
```

Use `Fraction(1, 2)`, not `1 / 2`, when exactness is required:

```
from fractions import Fraction as F

mu = IntervalMeasure(1, 3, density=F(1, 2))
monomial = monomial_polynomial_family()
euler = euler_polynomial_family()
coordinates = connection_transform(monomial, euler, 4).apply(
    p.to_univariate_coefficients()
)
ell = polynomial_integration_covector(euler, 4, mu)

print(ell.basis_verified)
print(ell.apply(coordinates))
```

Observed output:

```
True
106/5
```

The first short snippet intentionally exposes the Python boundary: `1 / 2` is a float and therefore makes a numerical interval. The factory then rejects it because exact covectors require exact `Fraction` interval data.

49.6 Differentiation through fixed finite measures

For fixed atoms and fixed weights,

$$F(x) = \sum_{i=1}^{N} w_i f_i(x) \quad \Longrightarrow \quad D^k F(x)[h_1, \dots, h_k] = \sum_{i=1}^{N} w_i D^k f_i(x)[h_1, \dots, h_k].$$

```
AtomicIntegratedDerivativeProvider(
    providers: Sequence[Any],
    weights: Sequence[int | Fraction | float],
    name: str = "atomic integrated derivative provider",
)
```

Every provider must expose a common positive `source_dimension` and `target_dimension`, an `Exactness` value, `supports_order(order)`, `value(point)`, and `derivative(order, point, directions)`. The provider and weight lists must be nonempty and have equal length. Weights are finite and nonnegative because the class models positive atomic integration.

`value` and `derivative` compute the fixed weighted sum. Every output must have shape `(target_dimension,)`. Nonfinite numerical data and overflow are rejected. The combined exactness is the weakest provider/weight status. The certificate checks only the finite structure and records the unchecked assumption that the component providers represent globally correct derivatives.

This class intentionally excludes parameter-dependent measures. If $w_i = w_i(x)$, then

$$D\sum_i w_i(x)f_i(x) = \sum_i Dw_i(x)f_i(x) + \sum_i w_i(x)Df_i(x),$$

and the first term is absent from the fixed-weight API. Do not use the class to silently drop it. **E/C**

The independent verifier constructs quadratic providers with scales one and three, weights $1/4$ and $3/4$, point $x = 2$, and direction $h = 5$. The returned first directional derivative is exactly `50`.

49.7 Analysis-synthesis commutation

For a fixed linear analysis map A, synthesis map S, atomic weights w_i, and physical samples v_i, finite linearity predicts

$$A\left(\sum_i w_i v_i\right) = \sum_i w_i A v_i.$$

When the system reconstructs the relevant coefficient vector, it also predicts

$$S\left(\sum_i w_i A v_i\right) = \sum_i w_i v_i.$$

```
analysis_synthesis_integration_certificate(
    system: AnalysisSynthesisSystem,
    measure: AtomicMeasure,
    physical_values: Sequence[Any],
    *,
    tolerance: float = 1e-10,
) -> Certificate
```

The number of physical values must equal the atom count, and every value must be a vector of `system.source_dimension`. The function incorporates both the system and measure certificates and computes three numerical residuals:

1. analysis of the weighted physical sum versus the weighted analyzed sum.
2. Synthesis of the weighted coefficient sum versus the physical sum.
3. The coefficient-gauge residual $(AS)c - c$.

All three must be at most the finite nonnegative tolerance. The returned exactness is at most `NUMERICAL`, even when the atomic weights are Fractions, because the system calculation uses NumPy floating arrays and norms. An invalid analysis-synthesis system certificate or invalid measure certificate cannot be hidden by small commutation residuals. **N**

This certificate is a finite commutation check for the supplied samples and fixed maps. It is not a theorem about Bochner integration, infinite frames, measurability, or parameter-dependent systems.

49.7.1 End-to-end exact example

The shipped `examples/block19_measure_integration_worked_example.py` joins the three chapters:

1. construct $[0, 1]$ and the exact chart $y = 2x + 1$.
2. Integrate $y^4 - 2y + 1$ by pushforward, materialized image density, and pullback.
3. Represent the polynomial in monomial, Euler, and Fibonacci coordinates.
4. Pair every coordinate vector with its verified integration covector.
5. Certify a provenance-matched Simpson enclosure.
6. Reject a polynomial that fools the three Simpson nodes.

Observed summary:

```
chart: y = 2x + 1, x in [0,1]
pushed measure: (1/2) dy on [1,3]
integrand: f(y) = y^4 - 2y + 1
pullback: f(2x+1) = 16x^4 + 32x^3 + 24x^2 + 4x
exact integral: 106/5
monomials pairing: 106/5
Euler polynomials pairing: 106/5
Fibonacci polynomials pairing: 106/5
Simpson: 21.200032552083332, error 3.255e-05,
         rigorous bound 3.255e-05
node adversary: sampled 0, true 1/840,
                false certificate blocked=True
```

The displayed decimal Simpson value is presentation only. The internal certified path keeps the Simpson value and bound as exact Fractions.

49.7.2 Identity boundaries and verification evidence

Claim	Implemented boundary
FTC	exact univariate rational polynomial. Exact constant-density interval
integration by parts	two exact univariate polynomials. Exact interval
Fubini	finite polynomial. All-exact interval factors or all-exact 1-D atomic factors
all Fubini orders	factorial enumeration capped by `max_dimensions`
pushforward identity	exact affine map and exact base moments
interval Jacobian	one-dimensional nonzero exact affine scale
coordinate covector	exact family witness and exact interval for unconditional status
differentiation	fixed finite nonnegative weights. Compatible providers
frame commutation	supplied finite atomic samples. Numerical tolerance

The pytest module `tests/test_block19_integration.py` exercises the complete API, resource guards, invalid-certificate propagation, finite-range failures, and adversarial cases. The independent pytest-free verifier `verification/block19_math_verifier.py` reports **16/16 passed** in the audited source tree. Its named checks include exact interval moments, pushforward and change of variables, Fubini in every finite order, FTC, integration by parts, coordinate covariance, the Simpson degree boundary, the sampled-zero adversary, rigorous Simpson enclosure, fixed-weight derivative commutation, and analysis-synthesis commutation.

These executions qualify the finite source behavior described above. They do not extend the contracts to general analytic measures, arbitrary measurable maps, infinite products, infinite-dimensional integration, or unverified symbolic identities.

Source anchors: `integration/identities.py`, `integration/coordinates.py`, `integration/adapters.py`, `integration/measures.py`, `integration/quadrature.py`, `examples/block19_measure_integration_worked_example.py`, `tests/test_block19_integration.py`, and `verification/block19_math_verifier.py`.

50. Typed finite expression model

The expression package is a typed language for finite, exact rational linear algebra. It turns coordinate vectors, rational matrices, and a small family of replayable polynomial maps into immutable expression trees. The result is not a general computer-algebra system. It does not parse arbitrary formulas, quantify over variables, simplify analytic functions, or prove identities in an infinite-dimensional space. Its deliberately narrow promise is useful: every accepted closed term has a finite rational denotation that can be recomputed, compared entry by entry, planned, and transported without loading user code.

First workflow. Choose the coordinate space, create exact literal data, construct maps with matching source and target types, then evaluate the closed expression. Inspect the certificate before using the returned value. Add symbol binding, simplification, planning, and persistence when the application needs them. The Fibonacci-Euler pipeline in Section 51.2 is the main worked example of this sequence.

The complete public surface lives in `komorebi_math.expression`. It is a research-tier namespace, additive to the package and outside the frozen stable and beta API manifests. Exactness in this chapter therefore describes the implemented arithmetic and evidence path. It does not elevate these names into the stable compatibility contract.

The semantic universe is

$$\mathbb{Q}, \qquad \mathsf{P}_n^B, \qquad \mathsf{Lin}(X, Y),$$

where P_n^B is the $(n+1)$-dimensional coordinate space of univariate polynomials through degree n in one explicit graded basis $B = (b_0, \dots, b_n)$. Vectors inhabit coordinate spaces. Matrices inhabit linear-map types. Arbitrary CAS terms, nonlinear maps, higher-order functions, general measures, analytic identities, and infinite-dimensional operators are outside this model. **E**

50.1 Scalar, vector, matrix, polynomial, operator, and measure types

The model has two coordinate-space classes and one linear-map type.

```
RationalSpace()

PolynomialCoordinateSpace(
    variable: str,
    degree: int,
    basis: tuple[tuple[Fraction, ...], ...],
    label: str = "",
)

LinearMapType(
    source: CoordinateSpace,
    target: CoordinateSpace,
)
```

`RATIONAL_SPACE` is the shared instance of `RationalSpace`. Every `RationalSpace()` has the same semantic identity, dimension one, and key `("Q",)`. An exact scalar is consequently represented internally by a one-entry coordinate vector rather than by a separate untyped node.

`PolynomialCoordinateSpace` snapshots one complete graded basis. Each basis member is stored in ascending monomial coefficients:

$$b_j(x) = \sum_{k=0}^{j} B_{kj} x^k, \qquad B_{jj} \neq 0.$$

The constructor enforces all these conditions:

- `variable` is a nonempty NFC-normalized string.
- `degree` is a nonnegative integer and cannot be a boolean.
- `basis` contains exactly `degree + 1` members.
- Member j has exactly $j + 1$ rational coefficients.
- Each leading coefficient is nonzero.
- Coefficients are integers or `Fraction` values, never binary floats.

The nonzero diagonal makes the triangular basis matrix invertible. It is this checked witness, not a family name, that lets the runtime convert between bases exactly.

```
PolynomialCoordinateSpace.from_family(
    family: Any,
    degree: int,
    *,
    variable: str = "x",
    label: str | None = None,
) -> PolynomialCoordinateSpace

space.dimension -> int
space.semantic_key -> tuple[...]
space.truncate(degree: int) -> PolynomialCoordinateSpace
```

`from_family` calls `family.polynomial(index)` for every index from zero through the requested degree and freezes the resulting rational witnesses. `truncate(r)` returns the compatible prefix through degree r, and rejects an attempt to increase degree. No future mutation of the source family is consulted by the coordinate space.

For a space X, a literal vector is constructed by

```
exact_value(
    space: CoordinateSpace,
    entries: Iterable[Any],
) -> ExactValueLiteral
```

and must have exactly `space.dimension` rational entries. For a map $A : X \to Y$, the constructor

```
exact_map(
    map_type: LinearMapType,
    matrix: Iterable[Iterable[Any]],
    *,
    label: str = "",
) -> ExactMapLiteral
```

requires matrix shape

$$(\dim Y) \times (\dim X).$$

Rows must have a common length and every entry must be an integer or `Fraction`. The optional map label is display metadata, not authority to invoke a polynomial rewrite.

There is no expression-level measure type. Exact interval integration enters the language as a typed linear functional $\mathsf{P}_n^B \to \mathbb{Q}$, documented in Chapter 51. General measure objects and quadrature rules remain in `komorebi_math.integration`. Passing one of them where an expression coordinate space is required is a type error.

Minimal type construction

```
from fractions import Fraction

from komorebi_math.expression import (
    RATIONAL_SPACE,
    LinearMapType,
    PolynomialCoordinateSpace,
    exact_map,
    exact_value,
)

Q = Fraction
monomials_2 = PolynomialCoordinateSpace(
    "x",
    2,
    (
        (Q(1),),
        (Q(0), Q(1)),
        (Q(0), Q(0), Q(1)),
    ),
    "monomials",
)

coordinates = exact_value(monomials_2, (1, -2, 3))
endomorphism = exact_map(
    LinearMapType(monomials_2, monomials_2),
    ((1, 0, 0), (0, 2, 0), (0, 0, 3)),
    label="display-only diagonal map",
)

print(RATIONAL_SPACE.dimension)
print(monomials_2.dimension)
print(coordinates.entries)
print(endomorphism.matrix)
```

Observed output, with `Fraction` values rendered by Python, is

```
1
3
(Fraction(1, 1), Fraction(-2, 1), Fraction(3, 1))
((Fraction(1, 1), Fraction(0, 1), Fraction(0, 1)),
 (Fraction(0, 1), Fraction(2, 1), Fraction(0, 1)),
 (Fraction(0, 1), Fraction(0, 1), Fraction(3, 1)))
```

50.2 Basis witnesses and semantic identity

For a polynomial-coordinate space the semantic key is

$$\mathrm{key}(\mathsf{P}_n^B) = (\texttt{"polynomial"}, x, n, B).$$

The complete rational basis witness B, the physical variable, and the degree participate in equality and hashing. The mutable human idea of a "Fibonacci basis" does not. The `label` field is excluded from dataclass comparison, so two differently labelled snapshots of the same witness are the same type. Conversely, two spaces with the same label and dimension but different basis coefficients are different types.

```
from fractions import Fraction
from komorebi_math.expression import PolynomialCoordinateSpace
```

```
Q = Fraction
left = PolynomialCoordinateSpace(
    "x", 2,
    ((Q(1),), (Q(0), Q(1)), (Q(0), Q(0), Q(1))),
    "custom basis",
)
renamed = PolynomialCoordinateSpace(
    "x", 2,
    left.basis,
    "a new display label",
)
spoof = PolynomialCoordinateSpace(
    "x", 2,
    ((Q(1),), (Q(0), Q(1)), (Q(0), Q(1), Q(1))),
    "custom basis",
)

print(left == renamed, left.semantic_key == renamed.semantic_key)
print(left.label == spoof.label, left.dimension == spoof.dimension)
print(left == spoof, left.semantic_key == spoof.semantic_key)
```

Observed output:

```
True True
True True
False False
```

This distinction closes a common spoofing path. Neither `apply` nor `compose` accepts two endpoints merely because their dimensions or display labels agree. The whole semantic witness must agree:

$$A : X \to Y, \quad v : Z, \quad Z \neq X \quad \Longrightarrow \quad A(v) \text{ is rejected.}$$

The same rule governs basis-aware polynomial primitives. A coordinate-change matrix is recomputed from the source and target witnesses. A caller cannot activate that rule by labelling an ordinary exact matrix "coordinate change." This is why the expression system distinguishes `PolynomialPrimitive` from `ExactMapLiteral` even when their replayed matrices happen to be equal.

A type mismatch between equally sized vectors is therefore worth investigating before changing any labels. Confirm the physical variable, degree, and basis coefficients. If the vectors use different valid bases, insert the explicit coordinate-change map; renaming a space does not perform that conversion.

50.3 Expression nodes and typed constructors

The immutable node grammar is small enough to display completely.

Node	Public constructor	Formation rule
`ExactValueLiteral`	`exact_value(X, entries)`	$v : X$
`ExactMapLiteral`	`exact_map(LinearMapType(X,Y), rows)`	$A : X \to Y$
`Symbol`	`symbol(name, T)`	$s : T$
`Let`	`let(s, value, body)`	`value` must have the declared type of s
`IdentityMap`	`identity(X)`	$I_X : X \to X$
`ZeroMap`	`zero_map(LinearMapType(X,Y))`	$0_{X,Y} : X \to Y$
`Compose`	`compose(B, A)`	$A : X \to Y,\ B : Y \to Z \Rightarrow B \circ A : X \to Z$
`Sum`	`add(a, b, ...)`	every term has one identical expression type
`Scale`	`scale(q, value)`	$q \in \mathbb{Q}$, type is preserved
`Apply`	`apply(A, v)`	$A : X \to Y,\ v : X \Rightarrow A(v) : Y$
`PolynomialPrimitive`	functions in Section 51.2	a branded replayable map

Node	Public constructor	Formation rule

The helper signatures are

```
symbol(name: str, declared_type: ExpressionType) -> Symbol
let(binding: Symbol, value: Expression, body: Expression) -> Let
identity(space: CoordinateSpace) -> IdentityMap
zero_map(map_type: LinearMapType) -> ZeroMap
compose(*maps: Expression) -> Expression
then(*maps: Expression) -> Expression
add(*terms: Expression) -> Expression
scale(scalar: int | Fraction, value: Expression) -> Scale
apply(operator: Expression, argument: Expression) -> Apply
```

`compose` receives factors in mathematical outermost-to-innermost order:

$$\mathrm{compose}(B, A) = B \circ A.$$

`then` receives them in the order in which they act:

$$\mathrm{then}(A, B) = B \circ A.$$

This difference is intentional. For a pipeline that first changes basis and then differentiates, these two expressions are identical:

```
compose(derivative_in_target, source_to_target)
then(source_to_target, derivative_in_target)
```

`compose()` and `add()` reject an empty argument list. With exactly one well-typed argument each returns that argument unchanged. A raw `Compose` or `Sum` dataclass node, by contrast, requires at least two children. These helpers are therefore the convenient way to assemble a variable-length pipeline.

The core formation rules are

$$\frac{v \in X}{\mathrm{Value}(v) : X}, \qquad \frac{A \in \mathsf{Lin}(X, Y)}{\mathrm{Map}(A) : X \to Y},$$

$$\frac{A : X \to Y \quad B : Y \to Z}{B \circ A : X \to Z}, \qquad \frac{A, B : T}{A + B : T},$$

$$\frac{q : \mathbb{Q} \quad e : T}{qe : T}, \qquad \frac{A : X \to Y \quad x : X}{A(x) : Y}.$$

Construction validates these local rules immediately. The later `check_expression` pass replays them from exact built-in classes, which also detects hostile in-memory mutation or subclasses that forge an `expression_type` property.

50.4 Lexical binding and alpha equivalence

`Symbol` uses object identity instead of value equality. Two symbols can have the same name and type and still be different variables. This gives `Let` ordinary lexical scope:

$$\text{let } x = e_1 \text{ in } e_2.$$

Construction requires

$$\text{type}(e_1) = \text{declared}(x).$$

The free variables of the whole term are those of the value plus the free variables of the body after removing occurrences that are the exact binder object. A same-name symbol created separately remains free.

```
check_expression(
    expression: Expression,
    bindings: Mapping[str | Symbol, ExpressionBinding] | None = None,
    *,
    budget: ExpressionBudget | None = None,
) -> CheckedExpression
```

`CheckedExpression` records `expression_type`, structural `stats`, sorted `free_symbols`, and a certificate. Missing values for free symbols do not make the expression ill typed. The checker lists them in diagnostics. Exact normalization does require a compatible binding.

Bindings may be keyed by the particular `Symbol` object or by its name. Object-keyed matches take precedence. Multiple matching entries are accepted only when their `ExpressionBinding` values agree. A binding's declared type and runtime shape are both replayed.

```
exact_binding(
    value: Any,
    declared_type: CoordinateSpace | LinearMapType,
    *,
    theorem_id: str = "EXPR.BINDING.EXACT",
) -> ExpressionBinding
```

The portable semantic digest represents a bound occurrence by lexical distance, not by display name. Consequently alpha-renamed terms have the same digest:

$$\text{let } x = 7 \text{ in } x \equiv_\alpha \text{let } y = 7 \text{ in } y.$$

Shadowing and free-versus-bound identity remain distinct.

```
from komorebi_math.expression import (
    RATIONAL_SPACE,
    exact_binding,
    exact_value,
    expression_digest,
    let,
    normalize_expression,
    symbol,
)

x = symbol("x", RATIONAL_SPACE)
y = symbol("renamed", RATIONAL_SPACE)
seven = exact_value(RATIONAL_SPACE, (7,))

print(expression_digest(let(x, seven, x)) ==
      expression_digest(let(y, seven, y)))
```

```
outer = symbol("same", RATIONAL_SPACE)
inner = symbol("same", RATIONAL_SPACE)
shadow_inner = let(
    outer, exact_value(RATIONAL_SPACE, (1,)),
    let(inner, exact_value(RATIONAL_SPACE, (2,)), inner),
)
shadow_outer = let(
    outer, exact_value(RATIONAL_SPACE, (1,)),
    let(inner, exact_value(RATIONAL_SPACE, (2,)), outer),
)
print(normalize_expression(shadow_inner).data)
print(normalize_expression(shadow_outer).data)

binder = symbol("collision", RATIONAL_SPACE)
same_name_free = symbol("collision", RATIONAL_SPACE)
free_body = let(binder, exact_value(RATIONAL_SPACE, (3,)), same_name_free)
print(normalize_expression(
    free_body,
    {"collision": exact_binding((9,), RATIONAL_SPACE)},
).data)
```

Observed output:

```
True
(Fraction(2, 1),)
(Fraction(1, 1),)
(Fraction(9, 1),)
```

50.5 Resource limits and cycle rejection

Expression validation happens before potentially expensive normalization, planning, or serialization. `ExpressionBudget` supplies one explicit finite envelope:

```
ExpressionBudget(
    max_nodes: int = 10000,
    max_depth: int = 1000,
    max_steps: int = 50000,
    max_degree: int = 512,
    max_entries: int = 1000000,
    max_coefficient_bits: int = 1000000,
    max_serialized_bytes: int = 64000000,
    max_symbol_length: int = 1024,
)

budget.preflight(root: Expression) -> ExpressionStats
budget.require(root: Expression) -> ExpressionStats
```

All limits are positive integers. The returned `ExpressionStats` contains

```
ExpressionStats(
    nodes: int,
    depth: int,
    steps: int,
    exact_entries: int,
    maximum_degree: int,
    maximum_coefficient_bits: int,
    serialized_bytes: int,
)
```

`nodes` counts unique object identities in the in-memory DAG. `steps` is the unique-node count plus traversed edges. `exact_entries` includes literal entries, matrix entries, primitive parameters, and the coefficients of unique semantic basis witnesses. `serialized_bytes` is a deterministic structural accounting used for preflight. It is not a pickle size or a promise that JSON will have exactly that length.

The traversal is iterative and uses a three-color object graph check. A gray node encountered again is a cycle and raises `ValidationError`. This matters even though the public dataclasses are frozen: hostile code can bypass normal construction with `object.__setattr__`, and a decoder must not trust that an object arrived through an honest path.

Data growth is checked again during normalization and execution. A small input DAG can produce a rational numerator with many more bits, so passing the structural preflight does not waive `max_coefficient_bits` for intermediate or final normal forms.

The portable form expands DAG sharing into a JSON tree. Its encoder therefore replays the generated record through the decoder under the same budget. A shared leaf used eleven times may be only one in-memory node, yet it counts as eleven serialized occurrences. `expression_to_dict` refuses to emit a record that its paired decoder would reject.

Failure matrix

Invalid construction or payload	Failure boundary
literal vector length differs from `space.dimension`	`DimensionError`
exact matrix shape differs from target-by-source shape	`DimensionError`
adjacent composition endpoints disagree	`ValidationError`
sum terms have different semantic types	`ValidationError`
application argument differs from operator source	`ValidationError`
`Let` value differs from binder declaration	`ValidationError`
float or boolean used as an exact coefficient	`ValidationError`
singular or incomplete graded basis witness	`ValidationError` or `DimensionError`
object-level expression cycle	`ValidationError`
node, depth, degree, entry, bit, text, or byte limit exceeded	`ResourceLimitError`
unsupported node subclass	`TypeError` during exact-class replay
mutated supported node	`ValidationError`, `DimensionError`, or `TypeError` according to the replayed invariant

These failures are part of the type model. They occur before any claim that a closed expression has an exact normal form.

51. Expression evaluation and certification

Evaluation is finite matrix algebra with evidence propagation. Every closed, well-typed value term normalizes to an exact coordinate vector. Every closed, well-typed map term normalizes to an exact rational matrix. Basis-aware polynomial operations are not accepted as opaque user callbacks. They are branded primitives whose matrices are regenerated from the frozen basis witnesses and rational parameters.

51.1 Exact normal forms

For a closed term $e : T$, the normal form is

$$\mathrm{NF}(e) \in \begin{cases} \mathbb{Q}^{\dim X}, & T = X, \\ \mathbb{Q}^{\dim Y \times \dim X}, & T = \mathsf{Lin}(X, Y). \end{cases}$$

The public operation is

```
normalize_expression(
    expression: Expression,
    bindings: Mapping[str | Symbol, ExpressionBinding] | None = None,
    *,
    budget: ExpressionBudget | None = None,
) -> NormalForm
```

`NormalForm` stores `expression_type`, exact `data`, a semantic `digest`, and the composed `certificate`. The digest is computed from canonical type and rational data, so equality is never delegated to floating tolerance.

Normalization assigns these denotations:

$$\begin{aligned} [\![I_X]\!] &= I_{\dim X}, \\ [\![0_{X,Y}]\!] &= 0_{\dim Y \times \dim X}, \\ [\![B \circ A]\!] &= [\![B]\!][\![A]\!], \\ [\![e_1 + e_2]\!] &= [\![e_1]\!] + [\![e_2]\!], \\ [\![qe]\!] &= q[\![e]\!], \\ [\![A(v)]\!] &= [\![A]\!][\![v]\!]. \end{aligned}$$

Literal data remain exact. `PolynomialPrimitive` nodes call `replay`, and a lexical symbol resolves first to the nearest binder object, then to an external `ExpressionBinding`. Normalizing an unbound free symbol raises `ValidationError`, even though `check_expression` can still report that open term's type.

`normal_form_digest(data, expression_type) -> str` is also public for callers that already possess exact data and the complete type witness. It validates both before hashing. The digest is an integrity identifier, not a signature, authorization token, or proof that data came from a trusted author.

51.2 Typed evaluation

The basis-aware primitive ledger consists of nine exact constructors.

PrimitiveKind	Constructor	Type and meaning
`COORDINATE_CHANGE`	`coordinate_change(source, target)`	$C_{S \to T} : \mathsf{P}_n^S \to \mathsf{P}_n^T$
`DERIVATIVE`	`derivative(source)`	$D_B : \mathsf{P}_n^B \to \mathsf{P}_{n-1}^B$, or a zero endomorphism at degree zero

PrimitiveKind	Constructor	Type and meaning
ANTIDERIVATIVE	antiderivative(source, target, basepoint=0)	$J_{a,B} : \mathsf{P}^B_{n-1} \to \mathsf{P}^B_n$
EVALUATION	evaluation(source, point)	$e_{a,B} : \mathsf{P}^B_n \to \mathbb{Q}$
INTERVAL_INTEGRATION	interval_integration(source, lower, upper, density=1)	$\ell_{B,[a,b],\rho} : \mathsf{P}^B_n \to \mathbb{Q}$
AFFINE_SUBSTITUTION	affine_substitution(source, scale=1, shift=0)	$S_{s,t}p(x) = p(sx+t)$ in the same coordinate space
FINITE_DIFFERENCE	finite_difference(source, step=1)	$\Delta_h p(x) = p(x+h) - p(x)$, re-encoded in P^B_n
BOUNDARY_FUNCTIONAL	boundary_functional(source, lower, upper, density=1)	$\rho(p(b) - p(a))$
CONSTANT_INJECTION	constant_injection(target)	$\iota : \mathbb{Q} \to \mathsf{P}^B_n$

Every constructor accepts only integers or `Fraction` parameters. Coordinate changes require the same physical variable and equal degree. An antiderivative target must have degree exactly one higher and must extend the source basis witness exactly. Interval endpoints obey $a \le b$, density is nonnegative, and finite-difference step is nonzero.

The derivative uses a tight target:

$$D_B : \mathsf{P}^B_n \longrightarrow \mathsf{P}^B_{n-1} \quad (n > 0).$$

At degree zero, it is the zero endomorphism $\mathsf{P}^B_0 \to \mathsf{P}^B_0$. `finite_difference`, by contrast, returns an endomorphism of the original P^B_n, even though its image has degree at most $n-1$. This fixed codomain lets it compose with other endomorphisms without silently changing the declared type.

```
replay(primitive: PolynomialPrimitive) -> Matrix
replay_matrix(primitive: PolynomialPrimitive) -> Matrix

primitive_matrix_equal(
    primitive: PolynomialPrimitive,
    matrix: Iterable[Iterable[Any]],
) -> bool

primitive_certificate(
    primitive: PolynomialPrimitive,
    *,
    claimed_matrix: Iterable[Iterable[Any]] | None = None,
) -> Certificate
```

`replay_matrix` is an alias of `replay`. For coordinate change, replay forms

$$C_{S \to T} = B_T^{-1} B_S.$$

Each `PolynomialPrimitive` also exposes `expression_type`/`map_type`, its enum-valued `kind`, exact `parameters`, a theorem-ID property, and two convenience accessors:

```
primitive.theorem_id -> str
primitive.matrix -> Matrix
primitive.certificate(
    *,
    claimed_matrix: Iterable[Iterable[Any]] | None = None,
) -> Certificate
```

The last two delegate to `replay` and `primitive_certificate`. They do not cache or trust a previously supplied matrix.

Every other matrix is produced by applying its monomial operation to each basis polynomial and converting back through the target basis matrix. The certificate has exactness `EXACT`, tolerance zero, and a theorem identifier specific to the primitive. If `claimed_matrix` is supplied, validity also requires exact entrywise equality with the replayed matrix.

The replayed maps satisfy the finite identities

$$C_{T\to U}C_{S\to T} = C_{S\to U}, \qquad C_{T\to S}C_{S\to T} = I,$$

$$D_T C^{(n)}_{S\to T} = C^{(n-1)}_{S\to T} D_S,$$

$$J_{a,T}C^{(n-1)}_{S\to T} = C^{(n)}_{S\to T}J_{a,S}, \qquad D_B J_{a,B} = I,$$

but

$$J_{a,B}D_B = I - u_B e_{a,B},$$

where $u_B = \iota(1)$ is the coordinate vector of the constant polynomial one. Thus differentiating after basepoint integration is an identity, while integrating after differentiation loses the value at the basepoint. The runtime contains a distinct certified rewrite for this defect and does not replace JD by an identity.

Evaluation and integration covectors transport with basis changes:

$$e_{a,T}C_{S\to T} = e_{a,S}, \qquad \ell_T C_{S\to T} = \ell_S,$$

and the density-weighted finite polynomial FTC is

$$\ell_{n-1}D_n = \rho(e_b - e_a).$$

These are finite rational polynomial identities, not a general analytic fundamental theorem of calculus.

Exact Fibonacci-Euler pipeline

The shipped worked example begins with the Fibonacci-coordinate polynomial

$$G_3(x) = F_4(x) = x^3 + 2x, \qquad c_F = (0,0,0,1)^T.$$

Let `F3`, `F4`, `E3`, and `E4` be the snapshotted Fibonacci and Euler spaces through degrees three and four. Define

$$P = C_{E_3\to F_3}\, D_{E_4}\, C_{F_4\to E_4}\, J_{0,F_3}.$$

The four successive coordinate vectors are

$$\begin{aligned}
J_{0,F_3}c_F &= \left(-\frac{1}{2}, 0, \frac{1}{4}, 0, \frac{1}{4}\right)^T,\\
C_{F_4\to E_4}J_{0,F_3}c_F &= \left(\frac{5}{8}, \frac{3}{2}, \frac{7}{4}, \frac{1}{2}, \frac{1}{4}\right)^T,\\
D_{E_4}C_{F_4\to E_4}J_{0,F_3}c_F &= \left(\frac{3}{2}, \frac{7}{2}, \frac{3}{2}, 1\right)^T,\\
Pc_F &= (0,0,0,1)^T.
\end{aligned}$$

Exact matrix replay gives

$$P = I_4.$$

The Fibonacci integration covector on $[0,1]$ is

$$\ell_F = \left(1, \frac{1}{2}, \frac{4}{3}, \frac{5}{4}\right),$$

so

$$\ell_F P c_F = \frac{5}{4}.$$

This example is executed independently by the installed expression verifier, the focused test suite, and `examples/block20_typed_expression_worked_example.py`.

51.3 Decidable equality

For two closed, well-typed terms, equality in this finite language is

$$e_1 \equiv e_2 \iff \mathrm{type}(e_1) = \mathrm{type}(e_2) \wedge \mathrm{NF}(e_1) = \mathrm{NF}(e_2).$$

The public decision procedure is

```
equivalent(
    left: Expression,
    right: Expression,
    *,
    left_bindings: Mapping[str | Symbol, ExpressionBinding] | None = None,
    right_bindings: Mapping[str | Symbol, ExpressionBinding] | None = None,
    budget: ExpressionBudget | None = None,
) -> EquivalenceResult
```

`EquivalenceResult` stores `equivalent`, optional left and right normal forms, and a certificate. If the semantic types differ, the result is immediately false and both normal-form fields are `None`. If the types agree, the comparison is exact and entrywise. `require_equivalent()` first requires the certificate and then raises `ValidationError` unless the result is true.

A valid comparison certificate can accompany `equivalent=False`: the comparison itself was performed correctly and found a mismatch. Use `require_equivalent()` when equality is required for the next step. Requiring only the comparison certificate does not assert the equality outcome.

```
from komorebi_math.expression import (
    PolynomialCoordinateSpace,
    add,
    equivalent,
    identity,
    scale,
)

M1 = PolynomialCoordinateSpace(
    "x", 1,
    ((1,), (0, 1)),
    "monomials",
)
constructed = add(scale(2, identity(M1)), scale(-1, identity(M1)))
decision = equivalent(constructed, identity(M1))

print(decision.equivalent)
print(decision.left.data)
print(decision.certificate.theorem_id)
```

Observed output:

```
True
((Fraction(1, 1), Fraction(0, 1)),
 (Fraction(0, 1), Fraction(1, 1)))
EXPR.EQUALITY.DECIDABLE
```

This decision rule does not prove a universally quantified identity between open terms. When the semantic types agree, `equivalent()` normalizes both sides and therefore requires compatible bindings for every free symbol. When the semantic types differ, it returns false with no normal forms before normalization. Equality after supplying bindings is equality for those bindings, not for every assignment. Direct node comparison is merely structural and is not quantified equality.

51.4 Certified rewrites and simplification

```
simplify_expression(
    expression: Expression,
    bindings: Mapping[str | Symbol, ExpressionBinding] | None = None,
    *,
    budget: ExpressionBudget | None = None,
) -> SimplificationResult
```

The simplifier is conservative. Before accepting any candidate rewrite it normalizes both sides and requires identical semantic type and exact data. Each accepted step records

```
RewriteStep(
    rule: str,
    theorem_id: str | None,
    before_digest: str,
    after_digest: str,
    side_conditions: tuple[str, ...],
    certificate: Certificate,
)
```

The local algebraic rules include

$$I \circ A \to A, \quad A \circ I \to A, \quad 0 \circ A \to 0, \quad A \circ 0 \to 0,$$

$$A + 0 \to A, \quad 1A \to A, \quad 0A \to 0, \quad q(re) \to (qr)e,$$

plus flattening of nested sums and compositions and literal matrix-vector application. Polynomial-specific proposals include coordinate cocycles, derivative naturality, $DJ = I$, the JD basepoint defect, the density-weighted FTC row, and transport of evaluation, integration, and boundary functionals.

These specialized rules trigger only for `PolynomialPrimitive` instances with matching kinds and exact source/target witnesses. An `ExactMapLiteral` whose label contains `EXPR.REWRITE.DJ` is still an ordinary matrix and cannot trigger the derivative/antiderivative rule.

`SimplificationResult` stores the simplified `expression`, `before` and `after` normal forms, the tuple of `steps`, and a composed certificate. It requires

$$\mathrm{NF}(e_{before}) = \mathrm{NF}(e_{after})$$

for every step and for the whole transformation. A 256-step local convergence guard and the shared `max_steps` budget bound rewrite work.

For the Fibonacci-Euler pipeline above, the observed rule sequence is

```
derivative coordinate naturality
flatten composition and remove identities
coordinate cocycle
flatten composition and remove identities
derivative-antiderivative cancellation
```

and the final node is `IdentityMap`. Both the pre-simplification and post-simplification normal forms are I_4.

51.5 Binding-certificate propagation

External values enter through `ExpressionBinding(value, declared_type, certificate)`. Their evidence cannot be laundered by exact internal arithmetic. Certificate composition uses the public strongest-to-weakest order

$$\text{EXACT} \prec \text{NUMERICAL} \prec \text{CONDITIONAL} \prec \text{HEURISTIC} \prec \text{PLANNED},$$

where the maximum is the weakest guarantee. The empty meet is exact. If any child is invalid, the parent is invalid. Unchecked assumptions are unioned. Their presence imposes at least `CONDITIONAL`, even when all child statuses were `EXACT`.

Formally, for child certificates $C_1, \dots, C_m$,

$$H_?^{out} = \bigcup_{j=1}^{m} H_?(C_j),$$

$$\tau_{out} = \max_{\prec} \left(\{\tau(C_j) : 1 \le j \le m\} \cup \{\text{CONDITIONAL} : H_?^{out} \neq \emptyset\} \right).$$

Residual keys receive deterministic `childN:` prefixes, the output tolerance is the largest child tolerance, and child diagnostics remain in separate namespaces. A key collision therefore cannot make evidence disappear.

The public helpers are

```
exactness_strength(value: Exactness) -> int
weakest_exactness(*values: Exactness) -> Exactness

compose_certificates(
    certificates: Iterable[Certificate],
    *,
    operation: str | None = None,
    theorem_id: str | None = None,
    assumptions_checked: Iterable[str] = (),
    assumptions_unchecked: Iterable[str] = (),
    diagnostics: Mapping[str, Any] | None = None,
) -> Certificate

exact_internal_certificate(
    operation: str,
    *,
    theorem_id: str | None = None,
    assumptions_checked: Iterable[str] = (),
    diagnostics: Mapping[str, Any] | None = None,
) -> Certificate
```

Three boundaries follow immediately:

1. exact parent plus an invalid child gives an invalid result.
2. Exact parent plus a numerical binding remains at most numerical.
3. Exact matrices plus an unchecked premise become conditional.

`evaluate_expression(...).require()` calls the final certificate's `require` method before returning data. Reading `.value` or `.scalar` alone does not perform that explicit validity assertion. `.scalar` is defined only for `RationalSpace`. Applying it to a vector or map result raises `DimensionError`.

52. Expression planning and persistence

The direct normalizer evaluates compositions in their written order. The planning layer adds a second, independently replayed route: it compiles the same expression into a small instruction sequence, optimizes dense matrix chains by dynamic programming, validates a content digest, executes the plan, and compares its final exact data with direct normalization. Persistence then encodes the original typed expression as strict canonical JSON with a semantic SHA-256 digest.

These are separate integrity boundaries. The execution-plan digest content-addresses the canonical execution record rather than every field of the in-memory Python objects. The portable expression digest binds canonical semantic JSON. Neither is a keyed MAC, cryptographic signature, proof of authorship, or sandbox for arbitrary code. **E**

Keep three records distinct when exchanging a calculation. The expression describes the typed computation, its normal form describes the resulting exact vector or matrix, and its execution plan describes one checked route to that result. A content digest identifies the record it was computed from; it does not replace the record or the mathematical type witness needed to interpret it.

52.1 Matrix-chain recurrence

For dimensions

$$(p_0, p_1, \dots, p_n), \qquad A_i \in \mathbb{Q}^{p_i \times p_{i+1}} \quad (0 \le i < n),$$

the planner minimizes classical dense scalar multiplication count over all order-preserving binary parenthesizations. Let $m[i, j]$ be the cost of the half-open matrix interval containing $A_i, \dots, A_{j-1}$. Then

$$m[i, i+1] = 0,$$

and, for $j > i + 1$,

$$m[i, j] = \min_{i<k<j} \left\{ m[i, k] + m[k, j] + p_i p_k p_j \right\}.$$

In the one-based closed-interval notation often used in textbooks, write $\widetilde{A}_i \in \mathbb{Q}^{d_{i-1} \times d_i}$, with $d_r = p_r$. The same recurrence is

$$m[i, j] = \min_{i \le k < j} \{ m[i, k] + m[k + 1, j] + d_{i-1} d_k d_j \}.$$

The implementation records the smallest split index when costs tie. Its theorem is narrow:

> `matrix_chain_plan` is exact and optimal among binary parenthesizations of the supplied order-preserving chain under classical dense scalar multiplication count.

It does not optimize addition count, sparsity, rank structure, memory traffic, parallelism, device transfers, kernel launches, Strassen-like algorithms, rounding stability, or wall-clock time.

```
matrix_chain_plan(
    dimensions: Sequence[int],
) -> MatrixChainPlan

matrix_chain_parenthesization(
    plan: MatrixChainPlan,
```

```
    labels: Sequence[str] | None = None,
) -> str

matrix_chain_multiply(
    matrices: Sequence[Iterable[Iterable[Any]]],
    plan: MatrixChainPlan,
) -> Matrix
```

`dimensions` must contain at least two positive integers. Booleans are rejected. A chain with n matrices explores

$$\frac{n(n-1)(n+1)}{6}$$

candidate splits. A fixed one-million-step planning guard rejects a dimension sequence that would exceed this count before constructing the tables.

`MatrixChainPlan` stores:

- the normalized dimension tuple.
- A `MatrixChainTree` root record.
- Optimal `scalar_operation_cost`.
- `left_associative_cost` and `right_associative_cost`.
- An unconditional exact certificate with theorem ID `EXPR.PLAN.MATRIX_CHAIN_DP`.

Construction of a `MatrixChainPlan` re-runs the dynamic program and rejects a tree or cost that does not match. `matrix_chain_multiply` repeats that replay, validates the number and exact shape of all rational matrices, and evaluates the recorded tree iteratively.

Classical six-matrix example

```
from komorebi_math.expression import (
    matrix_chain_parenthesization,
    matrix_chain_plan,
)

plan = matrix_chain_plan((30, 35, 15, 5, 10, 20, 25))
print(plan.scalar_operation_cost)
print(plan.left_associative_cost)
print(plan.right_associative_cost)
print(matrix_chain_parenthesization(plan))
print(plan.certificate.theorem_id)
```

Observed output:

```
15125
40500
47500
((A0 @ (A1 @ A2)) @ ((A3 @ A4) @ A5))
EXPR.PLAN.MATRIX_CHAIN_DP
```

For a tie such as dimensions `(2, 2, 2, 2)`, the deterministic smallest-split rule produces `(A0 @ (A1 @ A2))`.

52.2 Planned versus direct evaluation

The expression compiler emits only five opcodes:

Opcode	Inputs	Payload	Cost model
`constant`	0	exact vector or matrix	0
`add`	2	none	number of result entries
`scale`	1	exact rational scalar	number of result entries
`matrix_chain`	at least 2 maps	replayable `MatrixChainPlan`	optimal dense multiplication cost
`apply`	map and vector	none	target dimension times source dimension

One instruction is

```
PlanInstruction(
    opcode: str,
    inputs: tuple[int, ...],
    expression_type: CoordinateSpace | LinearMapType,
    payload: Any,
    scalar_operation_cost: int,
    certificate: Certificate,
)
```

and the complete plan is

```
ExpressionPlan(
    expression_digest: str,
    expression_type: CoordinateSpace | LinearMapType,
    instructions: tuple[PlanInstruction, ...],
    output_index: int,
    scalar_operation_cost: int,
    plan_digest: str,
    certificate: Certificate,
)
```

Every input index must name an earlier output. Instruction types, payloads, and costs are re-derived in `ExpressionPlan.__post_init__`. The aggregate cost must equal the sum of instruction costs. The plan digest is

$$d_P = \mathrm{SHA256}(\mathrm{CanonicalJSON}(d_e, T, (I_k), k_{out})),$$

where d_e is the semantic digest of the source expression, T its complete type, (I_k) the instruction records, and k_{out} the output index. The canonical instruction payload includes each opcode, dependency, typed payload, cost, and selected certificate fields. It does not serialize certificate diagnostics or messages, the plan-level certificate, or every other Python object field. Constructor replay separately requires the aggregate cost to equal the sum of instruction costs.

The public pipeline is

```
compile_expression(
    expression: Expression,
    bindings: Mapping[str | Symbol, ExpressionBinding] | None = None,
    *,
    budget: ExpressionBudget | None = None,
) -> ExpressionPlan

execute_plan(
    plan: ExpressionPlan,
```

```
    *,
    budget: ExpressionBudget | None = None,
) -> NormalForm

evaluate_expression(
    expression: Expression,
    bindings: Mapping[str | Symbol, ExpressionBinding] | None = None,
    *,
    budget: ExpressionBudget | None = None,
) -> ExpressionResult
```

`compile_expression` first type-checks and directly normalizes the source. It then emits a self-contained plan, reconstructs the digest, executes the plan, and refuses to return it unless the executed result equals the independent direct normal form.

`execute_plan` calls the plan constructor checks again, recomputes the content digest, verifies topological order, checks every result against the instruction type and budget, and composes the instruction certificates.

`evaluate_expression` performs both routes again and requires

$$\mathrm{NF}_{direct}(e) = \mathrm{NF}_{plan}(e)$$

entry by entry. `ExpressionResult` exposes `normal_form` for the planned result, `direct_normal_form`, the `plan`, and the combined `certificate`. Its `.value` property returns exact data. `.scalar` projects the unique entry only for a rational-space result. `.require()` requires the certificate first.

For the Fibonacci-Euler expression from Chapter 51, the four-map chain has dimensions

$$(4, 4, 5, 5, 4).$$

The selected parenthesization is

$$C_{E_3 \to F_3}\left(D_{E_4}\left(C_{F_4 \to E_4} J_{0,F_3}\right)\right),$$

with dense multiplication cost 244 instead of 260 for a left-associated chain. The full expression plan contains nine instructions and has aggregate cost 264 after the final map-vector application and integration functional. Both routes produce $5/4$.

These isolated mutations fail closed: changing an opcode to an unknown value, introducing a forward dependency, altering an instruction or aggregate cost, or changing either digest without updating its dependent content makes `execute_plan` raise `ValidationError`. These are replay and consistency checks, not authentication. A plan received across a trust boundary still needs an authenticated channel or an external signature.

52.3 Strict expression JSON

Portable expressions use a dedicated research schema:

```
EXPRESSION_SCHEMA = "komorebi_math.expression.v1"
EXPRESSION_SCHEMA_VERSION = 1
EXPRESSION_NODE_VERSION = 1
EXPRESSION_TYPE_VERSION = 1
```

This schema is intentionally isolated from the frozen general portable-codec registry. The expression API can evolve as a research surface without silently changing the stable persistence contract.

```
type_to_dict(
    expression_type: ExpressionType,
    *,
    budget: ExpressionBudget | None = None,
```

```
) -> dict[str, Any]

type_from_dict(
    value: Mapping[str, Any],
    *,
    budget: ExpressionBudget | None = None,
) -> ExpressionType

expression_to_dict(
    expression: Expression,
    *,
    budget: ExpressionBudget | None = None,
) -> dict[str, Any]

expression_from_dict(
    value: Mapping[str, Any],
    *,
    budget: ExpressionBudget | None = None,
) -> Expression

expression_to_json(
    expression: Expression,
    *,
    budget: ExpressionBudget | None = None,
) -> str

expression_from_json(
    value: str | bytes,
    *,
    budget: ExpressionBudget | None = None,
) -> Expression
```

An envelope contains exactly four fields:

```
expression
schema
schema_version
semantic_sha256
```

Every node and type record has an explicit `kind` and `version`. Unknown or missing fields are rejected. Extras are not ignored. Polynomial type records carry the complete variable, degree, basis witness, and transport label. Linear-map records recursively carry source and target types.

Rationals are reduced integer pairs

```
{"denominator":2,"numerator":1}
```

with a positive denominator. Floats, booleans, nulls, nonfinite constants, and unreduced pairs such as `2/4` are invalid exact primitives. Strings must be valid UTF-8 and NFC. JSON output uses sorted keys, no insignificant whitespace, no ASCII escaping requirement for ordinary Unicode text, and no NaN values.

A complete scalar example is

```
from fractions import Fraction
from komorebi_math.expression import (
    RATIONAL_SPACE,
    exact_value,
    expression_to_json,
)
```

```
text = expression_to_json(exact_value(RATIONAL_SPACE, (Fraction(1, 2),)))
print(text)
```

Observed output:

```
{"expression":{"entries":[{"denominator":2,"numerator":1}],"kind":"exact_value","space":{"kind":"rational_space",
    ↪  "version":1},"version":1},"schema":"komorebi_math.expression.v1","schema_version":1,"semantic_sha256":
    ↪  "5111acf0ceae03747c2a8d9a27734664d0de8ecc7ab607fef5658947ad515a4f"}
```

The decoder requires the input text itself to equal that canonical encoding. Pretty-printed JSON can describe the same generic JSON value but is rejected as a noncanonical expression transport. Call `expression_from_dict` when an already parsed mapping is the intended boundary. It validates structure and digest without imposing a previous text representation.

52.4 Digest and canonical text

The semantic digest is

$$d(e) = \mathrm{SHA256}\left(\mathrm{CanonicalUTF8NFCJSON}(\mathrm{semantic_record}(e))\right).$$

```
expression_digest(
    expression: Expression,
    *,
    budget: ExpressionBudget | None = None,
) -> str
```

The semantic record includes every type witness, exact entry, free-symbol name, node kind, and primitive parameter. It intentionally omits display labels. Bound symbol names are replaced by lexical distance. Therefore:

- changing a basis coefficient changes the digest.
- Changing a free symbol's name changes the digest.
- Changing an exact literal or primitive parameter changes the digest.
- Changing only a space or map display label does not change the digest.
- Consistently alpha-renaming a `Let` binder does not change the digest.
- Changing which same-name binder an occurrence refers to does change it.

The transport JSON retains display labels for human reconstruction even though they are absent from the semantic hash. Two differently labelled exact maps may thus have different `expression_to_json` text but identical `expression_digest` values.

On decode, the implementation reconstructs the exact built-in expression, reruns structural and budget preflight, computes a fresh semantic digest, and compares it with `hmac.compare_digest`. This checks internal consistency against the envelope's unkeyed checksum. It detects accidental corruption or a changed semantic tree when the recorded digest is held fixed. Anyone able to alter both can recompute it. It does not authenticate the producer. Use an external signature, MAC, or authenticated channel when provenance matters.

End-to-end roundtrip

```
from fractions import Fraction
from komorebi_math.expression import (
    RATIONAL_SPACE,
    exact_value,
    expression_digest,
    expression_from_json,
    expression_to_json,
    normalize_expression,
)
```

```
source = exact_value(RATIONAL_SPACE, (Fraction(1, 2),))
text = expression_to_json(source)
restored = expression_from_json(text)

print(expression_digest(source))
print(expression_digest(restored))
print(normalize_expression(restored).data)
```

Observed output:

```
5111acf0ceae03747c2a8d9a27734664d0de8ecc7ab607fef5658947ad515a4f
5111acf0ceae03747c2a8d9a27734664d0de8ecc7ab607fef5658947ad515a4f
(Fraction(1, 2),)
```

The worked $5/4$ expression has semantic digest

```
161d64ab695774a8293b9450f88654eabe28a1265a623575a84c73d64c7d9138
```

in the shipped `1.0.0b4` worked-example record.

52.5 Tamper, spoof, cycle, and resource adversaries

The decoder's threat model is restrictive by design. It reconstructs only the known finite node and type grammar. It never imports a class named by the payload, evaluates source text, invokes a callback, resolves an arbitrary module path, or executes a pickle reducer.

It explicitly rejects:

- duplicate object keys.
- Floating-point and nonfinite JSON tokens.
- Booleans and null values at exact-data positions.
- Noncanonical or unreduced rational pairs.
- Non-NFC strings and invalid UTF-8.
- Unknown schema, schema-version, node-version, or type-version values.
- Missing fields and unrecognized extra fields.
- Unknown node, type, or primitive tags.
- A malformed, uppercase, or mismatched semantic digest.
- Dangling bound-symbol indices.
- Incompatible expression endpoints or malformed basis witnesses.
- Circular in-memory containers and expression object cycles.
- Depth, node, step, degree, entry, coefficient-bit, text-length, or byte limits beyond the supplied `ExpressionBudget`.

The graph and transport models have different sharing semantics. An in-memory expression can be a DAG, and normalization caches a shared node. JSON is a tree, so each occurrence is written separately. The encoder preflights the expanded envelope through the strict decoder. This symmetry prevents the encoder from producing a record that is legal only on its side of the API.

Minimal digest-tamper demonstration

```
from copy import deepcopy
from fractions import Fraction

from komorebi_math.exceptions import ValidationError
from komorebi_math.expression import (
    RATIONAL_SPACE,
    exact_value,
```

```
    expression_from_dict,
    expression_to_dict,
)

record = expression_to_dict(
    exact_value(RATIONAL_SPACE, (Fraction(1, 2),))
)
hostile = deepcopy(record)
hostile["semantic_sha256"] = "0" * 64

try:
    expression_from_dict(hostile)
except ValidationError as exc:
    print(type(exc).__name__)
    print(str(exc))
```

Observed output:

```
ValidationError
expression semantic SHA-256 mismatch
```

Changing semantic content and recomputing the unkeyed digest creates a self-consistent record. Treat `semantic_sha256` as a deterministic content identifier and consistency check, not as adversarial tamper protection or authentication.

Operational verification

The installed finite-expression verifier is

```
python -m komorebi_math verify expression
```

For `1.0.0b4` it exercises 22 named checks covering semantic space typing, strict rationals and NFC, primitive replay, coordinate identities, the Fibonacci-Euler and affine examples, exact equality, certified rewrites, matrix-chain optimality and execution, direct/planned agreement, portable roundtrips, lexical alpha equivalence, canonical-text and digest rejection, evidence propagation, budgets, cycles, same-label spoofing, and forged or singular primitives. The expected final line is

```
verification passed 22/22 checks
```

For an end-to-end readable record, run

```
python .\examples\block20_typed_expression_worked_example.py --json
```

from the repository root. The example asserts every intermediate vector, $P = I_4$, the $5/4$ integral, optimal plan cost $244 < 260$, portable roundtrip equality, and the affine/pushforward value $106/5$ before printing the record.

Chapter boundary

The expression layer proves exact facts about the finite typed terms it accepts. It does not convert a finite replay into a general theorem about all polynomials, all bases, all measures, or all programs. The type witness, resource budget, certificate status, and transport digest should travel with the computation whenever the result crosses a process or publication boundary.

Part IX. Quantum, deformation, and finite Langlands models

This part develops three related finite execution layers. The quantum layer turns finite Hilbert spaces, symmetry sectors, circuit paths, channels, moving projectors, and effective operators into packet calculations with residual certificates. The deformation layer replaces a single finite space by explicit lift levels, exact reduction maps, cohomology groups, support loci, and Frobenius trace data. The finite Langlands layer then assembles finite fields, tame packets, Heisenberg models, induced representations, Clifford towers, ramification records, and explicit completion gates.

The word *finite* is essential throughout. A successful computation proves or checks a statement about the supplied matrices, residue levels, groups, characters, or gate record. It does not silently construct an infinite-volume quantum field theory, an infinite prime-power lift, a local or global Langlands correspondence, or an automorphic realization. The certificate class makes the boundary visible:

- exact certificates are finite algebraic or arithmetic identities.
- Numerical certificates record a residual and tolerance for floating matrix calculations.
- Conditional certificates preserve external hypotheses that the runtime does not prove.

The recommended imports are organized by domain:

```
from komorebi_math import quantum
from komorebi_math import deformation
from komorebi_math.langlands import (
    completion,
    finite_image,
    global_,
    local,
    ramification,
)
```

Many promoted names are also available from `komorebi_math`, but the domain imports make the mathematical provenance of each object easier to see.

Reading routes and prerequisites. The quantum route assumes complex linear algebra and begins with Chapter 53. The deformation route begins with Chapter 56 and uses finite sets, reduction maps, and exact linear algebra over prime fields. The local representation route begins with Chapter 57 and uses finite-field arithmetic and finite group representations. Chapter 58 brings those finite records into conditional completion arguments.

Your question	Start here	Meaning of the finite result
Which sectors or transitions occur in a finite quantum model?	Chapters 53 and 54	Matrix, projector, path, or channel diagnostics
Does a declared error family satisfy a code criterion?	Sections 55.1 and 55.2	A check tied to that finite code and error set
How does an eliminated sector affect a retained one?	Section 55.3, with Chapter 46	Effective resolvent and memory feedback
Which lift candidates survive the supplied levels?	Section 56.1	Stable cores and branches through the represented depth
What do finite fields, characters, or group packets compute?	Chapters 57 and 58	Explicit finite arithmetic and representation data
Which external premises are still needed?	Sections 58.5 through 58.7	Conditional support and completion records

The three domains share computational tools, but their objects have different meanings. Keep the domain, scalar field, basis or state ordering, and certificate with a result when moving it into another calculation.

53. Finite quantum foundations

The quantum foundation is a finite-dimensional operator calculus. It starts with a family of observables and seed vectors, resolves the resulting Hilbert space into symmetry packets, and supplies diagnostics for Euclidean transfer, spectral scaling, leakage, and constraint closure. Every matrix is validated for shape and, where appropriate, Hermiticity, unitarity, or projector identities before the calculation begins.

For a first example, choose a small Hilbert space and write down its basis ordering before constructing observables, states, or projectors. The same ordering must be used by every matrix and vector. Tensor products also need a consistent factor order. A shape check can detect incompatible dimensions, but it cannot recover a different physical interpretation of the same-sized array.

53.1 Observable and seed systems

Let $\mathcal{H} \cong \mathbb{C}^d$, let $\mathcal{A} = \langle A_1, \dots, A_r \rangle$ be the finite algebra generated by supplied observables, and let $S = (s_1, \dots, s_k)$ be a finite seed family. The generated orbit space is

$$\mathcal{O}(\mathcal{A}, S) = \operatorname{span}\{as_j : a \in \mathcal{A},\ 1 \le j \le k\}.$$

`QuantumSeedSystem` delegates the closure calculation to the common finite seed engine. `orbit_basis()` returns a maximal independent numerical basis, `orbit_dimension()` returns its rank, and `is_generating()` asks whether that rank is d. The certificate is numerical because independence is decided with a floating tolerance.

```
from komorebi_math.quantum import QuantumSeedSystem

system = QuantumSeedSystem(
    {
        "x": [[0, 1], [1, 0]],
        "z": [[1, 0], [0, -1]],
    },
    ([1, 0],),
)

assert system.dimension == 2
assert system.orbit_dimension() == 2
assert system.is_generating()
system.certificate().require()
```

For n qubits, `computational_basis_generation(n)` constructs the all-zero seed and the 2^n tensor products of Pauli X addresses. It returns the seed, the address operators, and a numerical orthogonality certificate. The construction is explicit and therefore grows exponentially with the qubit count.

The bipartite helpers separate two related ranks. If a pure state is reshaped as a $d_L \times d_R$ matrix M_ψ, then

$$r = \operatorname{rank}(M_\psi)$$

is its numerical Schmidt rank, while the orbit dimension under the full left matrix algebra is

$$\dim(M_{d_L}(\mathbb{C})\psi) = d_L r.$$

`schmidt_rank()` returns r, and `local_orbit_dimension()` returns $d_L r$. Neither routine infers an asymptotic entanglement law. It reports a finite singular-value rank at the requested tolerance.

53.2 Symmetry projectors

For a finite unitary representation U_g and an irreducible character χ of degree d_χ, the central isotypic projector is

$$P_\chi = \frac{d_\chi}{|G|} \sum_{g \in G} \overline{\chi(g)} U_g.$$

`finite_group_projector()` evaluates this expression from aligned mappings of group keys to representation matrices and character values. It checks the unitarity and common dimension of every supplied matrix, but it does not independently prove that the keys and multiplication law form a group or that the supplied values are an irreducible character. Those are declared inputs.

For a unitary V with declared order n, the cyclic packet projectors are

$$P_q = \frac{1}{n} \sum_{m=0}^{n-1} \omega^{-qm} V^m, \qquad \omega = e^{2\pi i/n}, \qquad q = 0, \dots, n-1.$$

`cyclic_symmetry_projectors(V, n)` first verifies $V^n = I$, then returns a `PacketSystem` whose names are the charge labels. The packet certificate checks idempotence, pairwise orthogonality, and completeness numerically.

If an operator A has charge q, then

$$VAV^* = \omega^q A, \qquad P_b A P_a = 0 \quad \text{unless} \quad b \equiv a + q \pmod{n}.$$

`charge_certificate()` checks the first identity, and `charge_selection_certificate()` checks every forbidden packet block in the second. `total_charge_packets(left, right)` groups tensor products according to the addition rule $a+b \equiv q \pmod{n}$. These are useful diagnostics for selection rules because a failure returns the largest forbidden residual instead of only a Boolean.

53.3 Euclidean transfer and spectral diagnostics

`EuclideanTransfer` begins with a finite Hermitian Hamiltonian H, a lattice spacing $a > 0$, and either the computed ground energy or a supplied lower shift E_0. It forms

$$\widetilde{H} = H - E_0 I \succeq 0, \qquad T_m = \exp(-ma\widetilde{H}).$$

For vectors v_i placed at nonnegative integer times t_i, the finite reflection matrix is

$$R_{ij} = \langle v_i, T_{t_i + t_j} v_j \rangle.$$

`reflection_certificate()` symmetrizes this matrix and checks that its smallest eigenvalue is no less than the negative tolerance. This is a finite reflection positivity test for the supplied transfer system. It is not a continuum reconstruction theorem. `EuclideanTransfer.from_transfer()` performs the inverse finite calculation for a positive definite contraction by applying $-a^{-1}$ log to its eigenvalues.

The spectral helpers use a nonnegative finite Hermitian operator L:

$$Z(t) = \operatorname{Tr}(e^{-tL}), \qquad d_s(t) = 2t \, \frac{\operatorname{Tr}(Le^{-tL})}{\operatorname{Tr}(e^{-tL})}.$$

`heat_trace()` evaluates $Z(t)$, and `spectral_dimension()` evaluates the scale-dependent diagnostic $d_s(t)$. In finite dimension, $d_s(t)$ tends to zero as time decreases to zero. Its large-time limit is zero when the operator has a zero eigenvalue. For a positive-definite operator, it instead grows asymptotically linearly with time. A plateau across an intermediate time window can be informative, but the function does not promote that plateau to a continuum dimension theorem.

`periodic_laplacian(shape)` constructs the nearest-neighbor periodic lattice matrix. `periodic_lattice_eigenvalues(shape)` independently returns

$$\lambda_k = \frac{4}{h^2} \sum_{j=1}^{D} \sin^2\left(\frac{\pi k_j}{N_j}\right).$$

`low_mode_dispersion_bound()` returns the discrete eigenvalue, its continuum quadratic approximation, and the quartic upper error expression used by the finite diagnostic.

The shipped replay is:

```
python .\examples\euclidean_spectral.py
```

Expected output:

```
Euclidean and spectral example passed
```

53.4 Leakage and constraint checks

For a fixed orthogonal projector P, $Q = I - P$, and Hermitian H, the finite leakage routine checks

$$\left\| Q e^{-itH} P \right\|_2 \leq |t| \, \|QHP\|_2.$$

`fixed_packet_leakage()` returns the observed norm, the bound, the coupling norm, and a numerical certificate. The bound is most useful when the off-diagonal coupling QHP is small. It can be valid but uninformative when $|t|\|QHP\|_2$ is large.

For a differentiable moving packet family $P_j(t)$, the shared `MovingPacketSystem` computes the Kato generator

$$K(t) = \sum_j \dot{P}_j(t) P_j(t).$$

The geometric coupling for packet P is implemented as

$$Q(t)(H(t) - iK(t))P(t).$$

`moving_packet_leakage_bound()` integrates its norm with sampled trapezoidal quadrature. Its certificate explicitly leaves the between-sample quadrature error unchecked.

Constraint closure is a separate finite diagnostic. Given Hermitian constraints $C_1, \ldots, C_r$, `constraint_closure()` projects each commutator onto their declared complex span:

$$[C_i, C_j] \approx \sum_k f_{ij}^k C_k.$$

The report contains the least-squares coefficients, the residual matrix, and the maximum obstruction. `transported_constraint_defect()` compares fine and coarse constraints through a coarsening map R by returning

$$R C_i^{\text{fine}} - C_i^{\text{coarse}} R.$$

Neither routine infers missing constraints. Closure is only with respect to the declared span, and a small floating residual is a numerical statement at the chosen scale.

54. Circuits, channels, and open dynamics

The next layer moves from static packets to processes. Circuit paths resolve a unitary product into addressed blocks. Quantum channels act on operator space. Lindblad generators evolve density matrices. Kato transport follows moving packet sectors. Finite Hamiltonian towers measure adjacent compatibility.

Choose the state representation before the evolution method. Circuit gates act on state vectors, channels act on matrices, and a Lindblad generator acts on density matrices through its generator equation. The superoperator acts on vectorized operator entries rather than state-vector entries. Preserve the documented column-vectorization convention when comparing direct actions with superoperator calculations.

54.1 Circuit packet expansions

Let $U = U_m \cdots U_1$, and let $\{P_a^{(j)}\}_a$ be a complete packet family before the first gate, between each pair of gates, and after the last gate. Repeated insertion of the identity gives

$$P_{a_m}^{(m)} U P_{a_0}^{(0)} = \sum_{a_1,\ldots,a_{m-1}} P_{a_m}^{(m)} U_m P_{a_{m-1}}^{(m-1)} \cdots U_1 P_{a_0}^{(0)}.$$

`CircuitPacketExpansion` stores the gates and $m+1$ packet systems. `path_blocks(source, target)` returns every internal address path and its operator block. `block()` sums those paths, and `reconstruction_certificate()` sums every source and target block to recover the full unitary.

The number of internal paths is the product of the internal packet counts. The constructor enforces `max_paths`, which defaults to one million. This is a safety limit, not an approximation rule. If the exact enumeration is too large, construction fails before allocating the path family.

For a source vector, `interference()` computes

$$p_{b\leftarrow a} = \left\| \sum_\gamma \psi_\gamma \right\|^2 = \sum_\gamma \|\psi_\gamma\|^2 + 2 \sum_{\gamma<\eta} \mathrm{Re}\langle \psi_\gamma, \psi_\eta \rangle.$$

The report separates the incoherent sum from the interference term and checks their reconstruction of the direct target probability. A negative interference value is allowed. It represents destructive interference, not a negative probability.

```
import numpy as np

from komorebi_math.quantum import (
    CircuitPacketExpansion,
    cyclic_symmetry_projectors,
)

symmetry = np.diag([1, -1]).astype(complex)
packets = cyclic_symmetry_projectors(symmetry, 2)
hadamard = np.array([[1, 1], [1, -1]], dtype=complex) / np.sqrt(2)

circuit = CircuitPacketExpansion(
    (hadamard, hadamard, hadamard),
    (packets, packets, packets, packets),
)
circuit.reconstruction_certificate().certificate.require()
circuit.block_unitarity_certificate().require()
```

```
report = circuit.interference(
    np.array([1, 0], dtype=complex),
    0,
    0,
)
report.certificate.require()
print(report.target_probability)
```

Observed output:

```
0.4999999999999998
```

54.2 Quantum channels

`QuantumChannel` represents a finite completely positive map through a nonempty Kraus family:

$$\Phi(X) = \sum_i K_i X K_i^*, \qquad \sum_i K_i^* K_i = I \quad \text{for trace preservation.}$$

Rectangular Kraus matrices are accepted, so input and output dimensions may differ. The certificate checks the trace-preservation residual and the smallest eigenvalue of the Choi matrix. Because these are floating matrix calculations, the certificate uses `Exactness.NUMERICAL` even though a Kraus representation is an exact mathematical guarantee of complete positivity.

The main methods are:

- `apply(X)` and `apply_density(rho)` for the Schrödinger action.
- `adjoint(Y)` for the Hilbert-Schmidt adjoint.
- `superoperator()` for column-vectorized operator space.
- `choi_matrix()` for the output-tensor-input Choi ordering.
- `compose(previous)` and `tensor(other)` for process composition.
- `covariance_certificate()` for declared input and output symmetries.

Operator packets use both a left and a right address. For projectors P_a,

$$X = \sum_{a,b} P_a X P_b.$$

`operator_packets()` constructs these blocks, `reconstruct_operator_packets()` sums them, and `dephase()` keeps only the diagonal blocks. Channel packet blocks use the induced operator-space projectors and therefore carry four addresses: output left, output right, input left, and input right.

The built-in amplitude-damping constructor uses

$$K_0 = \begin{pmatrix} 1 & 0 \\ 0 & \sqrt{1-\gamma} \end{pmatrix}, \qquad K_1 = \begin{pmatrix} 0 & \sqrt{\gamma} \\ 0 & 0 \end{pmatrix}, \qquad 0 \le \gamma \le 1.$$

```
import numpy as np

from komorebi_math.quantum import amplitude_damping

channel = amplitude_damping(0.2)
channel.certificate().require()

excited = np.diag([0, 1]).astype(complex)
output = channel.apply_density(excited)
assert np.allclose(np.diag(output), [0.2, 0.8])
```

54.3 Lindblad generators

For a Hermitian Hamiltonian H and finite jump family L_j, `LindbladGenerator` implements

$$\mathcal{L}(X) = -i[H, X] + \sum_j \left(L_j X L_j^* - \frac{1}{2} L_j^* L_j X - \frac{1}{2} X L_j^* L_j \right).$$

`apply()` evaluates the generator directly. `superoperator()` returns its column-vectorized matrix, and `evolve(rho, time)` applies the finite matrix exponential before validating and normalizing the resulting density matrix. This is dense finite evolution. It is not a large sparse solver and it does not control model error in a Markovian approximation.

`trace_preservation_certificate()` checks that the trace derivative vanishes on every matrix unit. `coherence_preservation_certificate(V, n)` checks that the superoperator commutes with conjugation by the declared cyclic symmetry. The latter is a direct finite covariance test. It does not assume charges for the jump operators without checking the resulting superoperator relation.

54.4 Kato transport through moving sectors

`MovingPacketSystem` accepts two callables, one for the projector family $P_j(t)$ and one for $\dot{P}_j(t)$. It checks the packet identities at a reference time and can compare the supplied derivative with a finite difference. Its canonical generator is assembled from the moving projectors.

`KatoTransport(system).evolve(start, stop, steps=n)` numerically integrates the transport equation on a uniform mesh. `direct_rotation_product(times)` instead multiplies the canonical direct rotations between successive packet families. `refinement_certificate()` compares a coarse and refined integration. These certificates test internal numerical agreement. They do not bound every higher derivative of an arbitrary moving family.

This shipped example combines moving transport with the effective operator of Chapter 55:

```
python .\examples\kato_feshbach.py
```

Expected output:

```
Kato and Feshbach example passed
```

54.5 Quantum tower diagnostics and finite compatibility

`BoundedHamiltonianTower` stores a finite sequence of Hermitian matrices on one common Hilbert space. Its adjacent differences are

$$\delta_j = \|H_{j+1} - H_j\|_2,$$

and the finite total variation is $\sum_j \delta_j$. For each adjacent pair, the report also checks the Duhamel perturbation bound

$$\|e^{-itH_{j+1}} - e^{-itH_j}\|_2 \le |t|\, \delta_j.$$

The report leaves infinite tail summability unchecked. A small finite total variation is evidence about the supplied levels only. It is not a certificate that an unrepresented infinite tower converges.

Operationally, the entire quantum domain is checked with:

```
python -m komorebi_math verify quantum
```

For `1.0.0b4`, the expected final line is:

```
verification passed 9/9 checks
```

The nine checks cover seeds and Schmidt orbits, cyclic packets, circuit paths, fixed and moving leakage, channels, Lindblad evolution, finite error correction, Feshbach reduction, Euclidean transfer, spectral diagnostics, constraint closure, and bounded towers.

55. Error correction and effective dynamics

A chosen sector can remain meaningful in two ways. A quantum code demands that a declared error family act indistinguishably on the code space. A Feshbach reduction instead permits coupling to a hidden sector and records its exact energy-dependent feedback.

55.1 Quantum codes and error sets

`QuantumCode(projector, errors)` validates an orthogonal projector P of positive rank and a nonempty finite error family $E_1, \dots, E_r$ on the same Hilbert space. The code dimension is the numerical rank of P.

The constructor does not require the raw error family to be a trace-preserving channel. That condition is checked later when the equal-weight error channel is used. This distinction matters. A family can satisfy the compression criterion but fail to define a physical equal-weight noise model.

`encode_density(logical_state)` chooses an orthonormal eigenbasis for the code projector and embeds a logical density matrix into the ambient space. The choice of basis is deterministic for the numerical eigensolver but is not a canonical symbolic encoding.

55.2 Knill-Laflamme finite checks

The finite Knill-Laflamme criterion is

$$PE_i^* E_j P = \alpha_{ij} P \qquad \text{for every } i, j.$$

`knill_laflamme_report()` computes

$$\alpha_{ij} = \frac{\operatorname{Tr}(PE_i^* E_j P)}{\operatorname{rank} P}$$

and records the spectral norm of every compression obstruction. Its `coefficient_matrix`, `residual_matrix`, `maximum_residual`, and certificate make a failed pair directly inspectable.

When the criterion passes, `recovery_channel()` diagonalizes the Hermitian coefficient matrix, constructs active syndrome isometries, and completes the recovery on the unused syndrome complement. `recovered_channel_certificate()` then forms the equal-weight noise channel and checks recovery on every code matrix unit. If the equal-weight errors do not preserve trace, the returned certificate is invalid and names that missing assumption.

The shipped example uses the two-dimensional repetition subspace

$$\mathcal{C} = \operatorname{span}\{|000\rangle, |111\rangle\}$$

and the identity plus one Pauli X error on each qubit:

```
python .\examples\quantum_channel_qec.py
```

Expected output:

```
quantum channel and error correction example passed
```

This example certifies the declared finite error set. It does not claim correction of arbitrary two-qubit errors, a threshold theorem, or fault tolerance under repeated noisy gates.

55.3 Effective operators and Feshbach reduction

Let a Hermitian Hamiltonian be split into retained and hidden coordinates:

$$H = \begin{pmatrix} H_{PP} & H_{PQ} \\ H_{QP} & H_{QQ} \end{pmatrix}.$$

For a spectral parameter z outside the hidden spectrum, `QuantumFeshbachReduction` implements the self-energy

$$\Sigma(z) = H_{PQ}(zI - H_{QQ})^{-1}H_{QP}$$

and the effective Hamiltonian

$$H_{\text{eff}}(z) = H_{PP} + \Sigma(z).$$

The projected resolvent identity is

$$P(zI - H)^{-1}P = \left(zI - H_{\text{eff}}(z)\right)^{-1}.$$

`resolvent_certificate(z)` checks this identity numerically. For $\operatorname{Im} z > 0$, `upper_half_plane_sign_certificate(z)` checks that the Hermitian imaginary part of $\Sigma(z)$ is nonpositive. A spectral parameter on the hidden spectrum is rejected rather than regularized silently.

Eliminating the hidden sector also produces a finite memory kernel. The `memory_kernel()` method returns the shared dynamics object whose Laplace transform reproduces the self-energy with the package's Schrödinger sign convention. `bare_closure_certificate()` checks whether $H_{PQ} = 0$. If that coupling vanishes, retained dynamics closes without memory. Otherwise, the energy-dependent operator is not interchangeable with the bare block H_{PP}.

55.4 Finite-dimensional scope boundaries

These distinctions should remain explicit in downstream work:

1. A numerical projector certificate is not exact symbolic idempotence.
2. A Knill-Laflamme pass covers the declared finite error span, not an unspecified physical noise environment.
3. A constructed recovery channel is tied to the supplied code basis and finite matrix tolerance.
4. A Feshbach map is exact as a finite block identity away from hidden poles, while this implementation verifies it numerically.
5. A finite Hamiltonian tower does not supply an infinite-volume limit.
6. Euclidean reflection matrices do not by themselves establish continuum reconstruction or locality.

These boundaries are not omissions from the certificate. They are part of its meaning. The quantum APIs provide a controlled finite laboratory in which the packet decomposition, coupling, interference, recovery, and hidden feedback can be inspected without claiming a stronger physical theory.

56. Finite deformation systems

The deformation package represents finite inverse systems directly. A node is an explicit lift candidate, a reduction map specifies compatibility between adjacent depths, a stable core records which nodes survive every represented level, and a branch records one compatible choice through the finite forest. Support and cohomology are computed by their own algebraic models. They are not relabelled as spectral projectors.

The chapter answers several different finite questions. A lift forest asks which candidates remain compatible with later supplied levels. A cochain complex computes cohomology of explicit differentials. A module presentation describes relations and fibers. A trace separator distinguishes candidates using a supplied table. Select the object that represents the question; agreement of counts alone does not identify these constructions with one another.

56.1 Residual forests

`FiniteLiftForest(levels, reductions)` expects levels indexed consecutively from one. The reduction at depth m maps level m to level $m-1$. Every image is validated during construction.

For n at depth d, `reduce()` composes reductions to a shallower level, `children()` returns immediate lifts, and `image(source, target)` returns the finite image set. The stable core at depth d is

$$C_d = \bigcap_{m=d}^{M} \operatorname{im}(X_m \longrightarrow X_d),$$

where M is the deepest supplied level. `branches()` enumerates compatible tuples through level M and enforces a branch-count safety limit. The report checks that full finite branches and nonempty stable cores agree.

`residue_forest(values_by_depth, prime=p)` supplies the common reduction

$$x \bmod p^m \longmapsto x \bmod p^{m-1}.$$

Duplicate residues are normalized within each level. The report is exact for the represented finite levels. It does not assert that a branch continues beyond M.

56.2 Visibility and counting charts

`rational_valuation(x, p)` computes the exact prime valuation of a rational number, with $v_p(0) = \infty$. `VisibilityTree(values, prime=p)` uses the separation depth

$$s(i,j) = v_p(x_i - x_j) + 1$$

to form nested finite partitions. `partition(m)` groups coordinates that are indistinguishable at depth m, and `ultrametric_certificate()` exhaustively checks the nonarchimedean separation inequality on every triple.

The counting helpers implement exact formulas for basic local charts. Above one residual root in a smooth d-dimensional chart,

$$N_m^{\mathrm{smooth}} = q^{d(m-1)}.$$

For $x^e = 0$ at depth m,

$$N_m^{\mathrm{nil}} = q^{m-\lceil m/e \rceil}.$$

For the normal crossing $xy = 0$,

$$N_m^{\mathrm{nc}} = q^m + m(q-1)q^{m-1}.$$

`weighted_chart_count()` delays visibility coordinate by coordinate, and `diagonal_nilpotent_count()` multiplies independent nilpotent counts. The corresponding `RationalCountingSeries` objects generate exact `Fraction` coefficients through their denominator recurrences. These formulas describe the implemented basic charts. They are not a general singularity classifier.

56.3 Finite cohomology complexes

`FiniteCochainComplex` stores a bounded cochain complex over $\mathbb{F}_p$:

$$\cdots \longrightarrow C^{i-1} \xrightarrow{d^{i-1}} C^i \xrightarrow{d^i} C^{i+1} \longrightarrow \cdots, \qquad d^i d^{i-1} = 0.$$

All entries are integral and reduced modulo the prime. Matrix shapes and the successive-zero identities are checked exactly at construction. `row_rank_mod` uses modular row reduction, and `cohomology(i)` returns

$$\dim H^i = \dim C^i - \operatorname{rank} d^i - \operatorname{rank} d^{i-1}.$$

`packet_cohomology(projectors)` accepts a complete orthogonal idempotent family in every degree. It requires the same packet order at every degree and checks that the projectors commute with the differentials. The result is the exact dimension data for each packet and degree.

This is genuine finite cohomology over a prime field. It does not infer a deformation ring, obstruction theory, or inverse-limit cohomology that was not supplied.

56.4 Support and Fitting-fiber diagnostics

`PresentedModule` uses a matrix with one row per module generator and one column per relation. Its cokernel is the represented finite module. Symbolic presentation and Fitting operations require the optional SymPy backend.

For a module with g generators, `fitting_generators(k)` returns the distinct $(g-k)$-minors. `fiber(point, prime=None)` evaluates the presentation and computes the exact characteristic-zero rank. With `prime=p`, rational entries are reduced into $\mathbb{F}_p$ when every denominator is a unit. The fiber dimension is

$$\dim M_x = g - \operatorname{rank} A(x).$$

The certificate checks that positive fiber dimension agrees with vanishing of the zeroth Fitting generators at the point. `finite_support()` repeats this calculation over an explicit finite point set.

`nested_subspace_occupancy()` addresses a different question. Given numerical column-space bases, it computes successive intersections using principal angles and returns the dimension sequence, final intersection dimension, and first observed stabilization index. This certificate is numerical. An equal dimension at one step does not prove stabilization for an unrepresented infinite family.

56.5 Frobenius companion systems

`FrobeniusSpine` stores monic polynomials of one common degree at finitely many places. `agrees(other)` compares two spines on their common or selected places. `CompanionForest` couples such a spine to a prime and a finite lift forest.

The trace separator solves a finite identifiability problem. Suppose the row at place v contains traces $t_{v1}, \dots, t_{vr}$ of candidate constituents. `finite_frobenius_separator()` greedily retains rows that increase the exact column rank. If the selected trace matrix T has rank r, then `recover(observed_traces)` solves

$$T\,m = b$$

for the unique multiplicity vector m. Rational data use exact `Fraction` elimination. Nonrational exact data use the optional SymPy backend. The certificate leaves Chebotarev density beyond the supplied table unchecked.

`CompatibleCarrier` adds named capacity bounds. `select(multiplicities)` returns a constituent dictionary only when every recovered multiplicity is nonnegative and no larger than its declared capacity.

56.6 Worked deformation replay and CLI contract

The shipped support example is:

```
import numpy as np

from komorebi_math.deformation import (
    nested_subspace_occupancy,
    normal_crossing_count,
    residue_forest,
)

forest = residue_forest(
    {
        1: (0, 1),
        2: (0, 1, 2, 3),
        3: (0, 1, 4, 5),
    },
    prime=2,
)
report = forest.report()
report.certificate.require()

occupancy = nested_subspace_occupancy(
    (
        np.eye(3),
        np.array([[1, 0], [0, 1], [0, 0]], dtype=float),
        np.array([[1], [0], [0]], dtype=float),
    )
)
occupancy.certificate.require()

print(tuple(len(level) for level in report.levels))
print(tuple(len(core) for core in report.stable_cores))
print(len(report.branches))
print(occupancy.intersection_dimension)
print(normal_crossing_count(3, 2))
```

Observed output:

```
(2, 4, 4)
(2, 2, 4)
4
1
21
```

The finite Frobenius example prints separating places `(2, 3)`, recovered multiplicities `(3, 2)`, and the selected carrier `{'rho1': 3, 'rho2': 2}`.

Run the installed domain verifier with:

```
python -m komorebi_math verify deformation
```

Expected final line:

```
verification passed 4/4 checks
```

The CLI counting entry point is `komorebi deformation count`. It reports only the implemented finite chart formulas and should not be described as a general deformation-ring point counter.

57. Local finite Langlands packets

The local package provides small exact and numerical models for finite field arithmetic, tame Frobenius orbits, Weil relation matrices, Gauss phases, Heisenberg representations, symplectic commutator envelopes, and finite packet bridges. These objects are computational components and diagnostics. They do not implement the local Langlands correspondence.

For the arithmetic examples, keep the field modulus and polynomial-basis convention with every encoded element. An integer code is a representation of an element in that chosen field, not a field-independent value. For the representation examples, also retain the ordered group labels and character data. Those declarations determine which finite matrix relation a certificate is checking.

57.1 Finite fields and Frobenius orbits

`FiniteField(p, degree=n, modulus=f)` represents

$$\mathbb{F}_{p^n} \cong \mathbb{F}_p[x]/(f(x))$$

in the supplied polynomial basis. Coefficients of `modulus` are listed from constant to leading term. The modulus must be monic, have degree n, and pass the package's exact irreducibility test. Extension fields require an explicit modulus.

Elements have canonical integer codes whose base-p digits are the polynomial coefficients. Addition, multiplication, powers, inversion, field trace, norm, primitive-element search, and discrete logarithms are exact finite operations. They enumerate small fields and are not optimized for cryptographic sizes. Zero has no inverse or discrete logarithm and is rejected where appropriate.

`FrobeniusOrbit(q, degree=n, exponent=a)` acts on tame character exponents modulo $q^n - 1$:

$$a \longmapsto qa \pmod{q^n - 1}.$$

The packet is regular when its orbit length equals n. The object also reports the dual exponent and whether the dual lies in the same orbit. `regular_orbit_count(q, n)` uses a Mobius inversion count for full orbits.

57.2 Tame packets and Gauss data

For a regular orbit $(a_0, \dots, a_{n-1})$, `TameWeilPacket` forms a diagonal inertia matrix and a cyclic Frobenius matrix. The certificate checks the tame relation

$$FIF^{-1} = I^q.$$

The orbit is exact integer data, while the relation certificate is numerical because the inertia eigenvalues are complex roots of unity. The Frobenius determinant orientation is

$$\det F = (-1)^{n-1}c,$$

where c is the supplied closing scalar.

`tame_tensor_refinement(m, n)` reports the finite decomposition governed by $\gcd(m, n)$ and $\operatorname{lcm}(m, n)$. `SpecialMonodromyBlock` supplies the finite nilpotent Jordan chain and its compatible Frobenius scaling diagnostic.

For an explicit `FiniteField`, `additive_character()`, `multiplicative_character()`, and `gauss_sum()` compute finite character sums. The returned `GaussSumResult` contains the complex value, magnitude, normalized phase, and a numerical certificate. A magnitude check is a finite character sum identity at the chosen floating tolerance, not an analytic continuation statement.

57.3 Finite Heisenberg models

`HeisenbergModel(prime=p, rank=r)` uses the finite phase space $\mathbb{F}_p^r \times \mathbb{F}_p^r \times \mathbb{F}_p$ and a Schrödinger representation on a space of dimension p^r. The constructor enforces a maximum representation dimension, which defaults to 4096.

In the package convention, coordinates multiply by

$$(x, \xi, z)(y, \eta, w) = (x + y, \xi + \eta, z + w - \eta \cdot x).$$

Translation and modulation matrices satisfy this law up to the declared root of unity phase. `group_law_certificate()` verifies the matrix identity for a chosen pair. `trace_support_certificate()` exhaustively checks that the representation trace vanishes away from the center.

The standard translation selector is the subgroup average

$$P = \frac{1}{p^r} \sum_{x \in \mathbb{F}_p^r} T_x.$$

`selector_certificate()` checks self-adjointness, idempotence, and rank one. This is a concrete finite isotropic selector. It is not a general classification of all Heisenberg representations.

57.4 Symplectic packets and finite diagnostics

`AlternatingForm(matrix, prime=p)` validates an exact alternating matrix over $\mathbb{F}_p$. It reports rank, radical dimension, nondegeneracy, a radical basis, and exact evaluations. In characteristic two, `QuadraticRefinement` stores a finite quadratic function q whose polar form is the declared alternating form:

$$q(x + y) - q(x) - q(y) = B(x, y).$$

The polar certificate exhausts all binary vectors up to the configured dimension limit. For a nondegenerate even-dimensional form, the object returns the integral Gauss sum, Arf invariant, and normalized Gauss sign.

`ProjectiveIntertwinerEnvelope` starts from labelled invertible complex operators. It evaluates their commutators, tests whether each commutator is a scalar p-th root of unity, and recovers the associated alternating form. Because scalar phase recognition is numerical, `EnvelopeRecovery` carries residuals for every basis pair.

`SymplecticBridge.construct()` checks an exact prime-field isometry between two forms. `LocalPacketBridge` wraps a finite transport graph and exposes its fundamental-cycle holonomy certificate. Neither object supplies missing local representation data. They validate a declared comparison.

57.5 Worked local packet replay

The shipped local example is:

```
import numpy as np

from komorebi_math.langlands.local import (
    FrobeniusOrbit,
    HeisenbergModel,
    TameWeilPacket,
)

orbit = FrobeniusOrbit(q=2, degree=3, exponent=1)
orbit.certificate().require()

tame = TameWeilPacket(orbit)
tame.relation_certificate().require()

heisenberg = HeisenbergModel(prime=3, rank=1)
left = heisenberg.element((1,), (2,), 1)
```

```
right = heisenberg.element((2,), (1,), 2)
heisenberg.group_law_certificate(left, right).require()
heisenberg.selector_certificate().require()

print(orbit.elements)
print(tame.dimension)
print(complex(np.linalg.det(tame.frobenius_matrix())))
print(heisenberg.dimension)
print(int(np.linalg.matrix_rank(heisenberg.translation_selector())))
```

Observed output:

```
(1, 2, 4)
3
(1+0j)
3
1
```

The corresponding CLI commands are `komorebi langlands tame-orbit` and `komorebi langlands heisenberg`. They expose these finite diagnostics. They do not claim to attach automorphic representations.

58. Finite-image, ramification, and completion models

The final chapter of Part IX assembles the finite local pieces into explicit group models and conditional global records. The finite-image calculations can be exact or numerically certified. Ramification and automorphic completion remain conditional whenever required phase choices, descent theorems, support comparisons, or realization results are external.

58.1 Finite groups, characters, and orbits

`FiniteGroup` validates an explicit element set, multiplication law, identity, closure, inverses, and, below the configured size limit, associativity. The constructors `cyclic_group(n)` and `symmetric_group(n)` provide common examples. `FiniteSubgroup` validates closure inside one specific parent group object and supports normality, intersection, and conjugation.

`LinearCharacter` requires one complex value on every subgroup element and checks multiplicativity. `InducedRepresentation(G, chi)` chooses deterministic left-coset representatives and constructs the monomial matrices of $\mathrm{Ind}_H^G \chi$. The representation certificate exhaustively checks the matrix law. The group and coset logic is exact, while the complex matrix residual is numerical.

`MonomialAction` decomposes symmetric, exterior, and tensor bases into finite orbits. Every `OrbitPacket` stores a representative, its orbit, its stabilizer, and stabilizer character values. `packet_certificate()` checks dimension conservation. The routines enumerate basis states, so degree and representation dimension should remain modest.

58.2 Mackey and Clifford towers

`mackey_tensor_packets()` uses deterministic double-coset representatives to construct the finite packets for a tensor product of two induced characters. For each representative g, it records

$$H \cap gKg^{-1}$$

and the pulled-back product character. Its exact certificate checks that the sum of induced packet dimensions equals $[G : H][G : K]$.

A `CliffordStep` represents a prime-cyclic quotient. An induction step `I` requires the parent representation degree to be the prime index times the child degree. An extension step `E` keeps the degree unchanged. A higher-rank extension step is a nonabelian descent gate.

`CliffordTower` checks that group orders and degrees form a chain. Its word is the sequence of `I` and `E` letters, and its depth is the number of higher-rank extension gates. The finite report is exact for the declared order and degree data. `automorphic_plan()` is conditional. It records unresolved attached descent assumptions instead of converting an `E` step into an automorphic conclusion.

```
from komorebi_math.langlands.finite_image import (
    CliffordStep,
    CliffordTower,
)

tower = CliffordTower(
    (
        CliffordStep(24, 12, 3, 3, "E"),
        CliffordStep(12, 4, 3, 1, "I"),
        CliffordStep(4, 2, 1, 1, "E"),
```

```
    )
)

report = tower.report()
report.certificate.require()
assert report.word == "EIE"
assert report.depth == 1
assert report.unresolved_extension_steps == (0,)
```

`AutomorphicInductionPlan` and `VirtualCharacterResolution` are symbolic planning objects. The former lists class-field and automorphic-induction steps with external hypotheses. The latter can expose negative virtual terms and render a formal product of labelled L-functions. A formal expression is not a positivity or holomorphy proof.

58.3 Ramification towers

`ProjectiveMultiplicityPacket` stores a prime, quotient rank, alternating commutator form, and optional phase information. Its radical dimension and symplectic rank are exact. The irreducible Heisenberg dimension of the nondegenerate quotient is

$$p^{\operatorname{rank}(B)/2}.$$

At $p = 2$, a matching `QuadraticRefinement` can supply the Arf invariant and Gauss sign. If neither a phase label nor an appropriate dyadic refinement is present, the phase remains unresolved.

A `RamificationStage` records a nonnegative break number, parent and child orders, orbit size, and multiplicity dimension. It enforces

$$\text{orbit size} = \frac{\text{parent order}}{\text{child order}}, \qquad \text{reconstructed dimension} = \text{orbit size} \times \text{multiplicity dimension}.$$

`RamificationCliffordTower` requires strictly increasing breaks and a chained order sequence. Its report is conditional and names unresolved phase stages. `tensor_packet()` combines two aligned towers stage by stage. The direct-sum commutator form is finite data, while multiplicativity of phase normalization remains an explicit condition.

58.4 Dyadic and Hecke packets

`FiniteHeckePackets` validates a complete orthogonal family of finite projectors. When commuting operators are supplied, every projector must commute with them. `packets()` reports numerical ranks, and `occupied()` tests whether a packet meets either the full carrier or a supplied subcarrier.

The term *Hecke packet* here means a finite central-idempotent model on the declared carrier. It is not an automatic construction of a global Hecke eigensystem.

At coefficient prime two, rational sign projectors do not remain distinct integral idempotents. `dyadic_sign_diagnostic()` records the finite replacement in $\mathbb{F}_2[C_2]$: if $X = g - 1$, then

$$X^2 = 0.$$

The returned certificate is exact for this residual group algebra. It does not assert that every dyadic global sign problem is resolved by the same local coordinate.

58.5 Patching plans

`SupportComparison.construct()` compares finite component labels and reports which required source components are missing from a declared image. `PatchingSupportReport` then exposes five support conditions:

1. A patched object exists.
2. It has full support.
3. Support descends to finite level.
4. The comparison preserves required support.
5. A characteristic-zero fiber is nonzero.

The report is complete only when all five Boolean inputs are true. Its certificate remains conditional because these values are supplied results, not the output of a general patching theorem implemented by the package.

58.6 Five formal completion gates

`AutomorphicCompletionReport` uses the canonical gate order

$$\text{residual} \longrightarrow \text{patching} \longrightarrow \text{comparison} \longrightarrow \text{fiber} \longrightarrow \text{realization}.$$

Every gate requires a `GateStatus(satisfied, detail, external=True)`. Missing gate keys are rejected. `complete` is true only if all five statuses are true, `unmet_gates` preserves canonical order, and `external_gates` records which claims depend on imported arithmetic results.

`result()` returns a `ConditionalResult[bool]`. A false value is not a proof that automorphy fails. It means the supplied record does not close every gate. A true value means every declared gate has been supplied, while the certificate still identifies the argument as conditional.

When preparing a completion record, use each gate's detail to identify the specific supporting result and the hypotheses under which it applies. Keep unmet gates explicit. This makes the record useful for review: it distinguishes a missing premise from a failed finite computation and avoids treating five true flags as a substitute for the external arguments they summarize.

`CompatiblePropagationReport` similarly requires a common Frobenius spine, semisimplicity, one automorphic member, and local-global compatibility. It records a one-member propagation argument only after all four assumptions are present.

58.7 Conditional global scope

The package draws a hard line between finite computation and global arithmetic conclusion:

- Finite group laws, residue ranks, orbit counts, and prime-field alternating forms use exact finite arithmetic.
- Complex representation, projector, and phase relations are numerically certified.
- Automorphic induction, attached descent, patching support, compatible propagation, and realization remain conditional when they use external theorems.
- No finite gate record is described as a proof of the local or global Langlands correspondence.

This separation allows a long argument to be represented without erasing its open hypotheses. Downstream code should carry the certificate, especially its `assumptions_unchecked`, with any claimed conclusion.

58.8 End-to-end finite replay

The finite-image example constructs an induced representation of C_4, decomposes its third symmetric power, and inspects an `EIE` Clifford word. Observed output is:

```
{'induced_dimension': 2,
 'symmetric_packet_dimensions': (2, 2),
 'clifford_word': 'EIE',
 'clifford_depth': 1,
 'unresolved_extension_steps': (0,)}
```

The ramification example constructs a dyadic projective packet and leaves the realization gate unmet. Observed output is:

```
{'ramification_breaks': (0, 1),
 'ramification_dimensions': (4, 2),
 'arf_invariant': 0,
 'completion_ready': False,
 'unmet_gates': ('realization',)}
```

Run them with:

```
python .\examples\finite_image_clifford.py
python .\examples\ramification_completion.py
```

The installed domain verifier is:

```
python -m komorebi_math verify langlands
```

For `1.0.0b4`, the expected final line is:

```
verification passed 12/12 checks
```

The twelve checks cover finite fields and Gauss phases, tame packets, Heisenberg selectors, dyadic refinements, projective envelopes, induced and Mackey packets, Clifford depth, finite Hecke packets, support diagnostics, ramification packets, and the five-gate completion record.

58.9 Chapter boundary

Part IX supplies an executable finite bridge from operator packets to residual support and arithmetic representation data. It does not collapse those objects into one type. A quantum symmetry projector, a deformation support component, and a finite Hecke idempotent can interact through explicit maps, but they retain different semantics and different certificate requirements.

The practical rule is simple: exact finite data may be transported exactly, floating matrix relations must retain their tolerances, and imported global theorems must remain visible as conditional gates. The next part turns these typed results into documents, portable records, extensions, and command-line workflows.

Part X. Export, persistence, extension, and command-line use

Part X moves verified mathematical objects across three boundaries. The document layer turns typed values into renderer-neutral reports. The export layer writes those reports and ordinary data to controlled filesystem targets. The persistence layer reconstructs registered objects from a strict, versioned, digest-checked envelope. Provenance records and extensions add discoverability without silently changing the stable core, while the command line exposes the same finite operations to scripts.

These layers are related but not interchangeable. Presentation serialization is intended for readable JSON, tables, and reports. Portable persistence is a reconstruction contract. A rendered equation is not automatically a portable object, and a portable payload is not automatically a polished document. Keeping those roles separate is the main design rule for this part.

Choose the output by what the recipient needs to do with it.

Intended use	Read	Preserve with the output
Read a mathematical report	Chapters 59 and 60	values, equations, certificate explanation, and any figure files
Load supported typed objects again	Chapter 61	the portable envelope and the package or extension codec identity
Inspect the origin of a claim	Chapter 62	the provenance record and its assumptions
Automate a command	Chapter 63 and Appendix F	the exact invocation, exit status, and documented output fields
Share several rendered formats	Sections 60.5 and 70.3	the archive and its manifest

For a first export, start from a small result you already understand. Render it, open the resulting document, and check that the values and certificate are readable. If you also need to resume computation, save a supported portable object separately and test loading it. The report and the reusable state serve different readers and can use different formats.

59. Mathematical document model

The document model is a small typed intermediate representation. A document is an immutable metadata record paired with an ordered, mutable construction buffer of validated blocks:

$$\mathcal{D} = \mathcal{M} \times \mathcal{B}^*.$$

Renderers consume this structure without mutating it. That gives one report a common semantic source even when its Markdown, HTML, text, and LaTeX bytes are different.

59.1 Metadata and block grammar

`DocumentMetadata` stores `title`, `author`, `date`, `abstract`, and a tuple of `keywords`. The title defaults to `Komorebi Math Export` and must contain at least one non-whitespace character. The other fields are optional presentation strings. Metadata is frozen, so a report that needs a different title receives a new metadata value rather than an in-place mutation.

The public block sum type is:

$$\begin{aligned}\mathcal{B} = \mathsf{Heading} \sqcup \mathsf{Paragraph} \sqcup \mathsf{Equation} \sqcup \mathsf{Table} \\ \sqcup \mathsf{CodeBlock} \sqcup \mathsf{ListBlock} \sqcup \mathsf{Figure} \sqcup \mathsf{RawLatex}.\end{aligned}$$

Each constructor owns its local invariant. A `Heading` has nonempty text and a level from 1 through 6. An `Equation` has nonempty LaTeX source and may carry a label, numbering flag, and prose description. A `Table` has at least one column, every row matches the header width, and an optional alignment tuple contains one of `l`, `c`, or `r` for each column. A `ListBlock` is nonempty. A `Figure` normalizes its path and rejects an empty LaTeX width. Paragraph and code contents may be empty because the renderer can represent deliberate spacing or an empty literal block.

`RawLatex` is the deliberate exception to renderer neutrality. It is trusted source for the LaTeX renderer, not a portable mathematical meaning. Use it only when a report truly needs a LaTeX construct that the typed blocks cannot express.

59.2 Fluent `MathDocument` construction

`MathDocument` contains metadata and an ordered `blocks` list. Its `add`, `section`, `paragraph`, `equation`, `table`, `code`, `bullet_list`, and `figure` methods append blocks and return the same document. The blocks themselves are frozen records even though the list is a construction buffer.

```
from fractions import Fraction

from komorebi_math.export import (
    DocumentMetadata,
    MathDocument,
    render_markdown,
)

document = MathDocument(
    DocumentMetadata(
        title="Exact coordinate report",
        author="Komorebi Math",
        keywords=("exact arithmetic", "coordinates"),
    )
)
document.paragraph("The table records exact rational values.")
document.section("Values")
document.equation(r"a_n=\frac{1}{n+1}", numbered=True, label="eq:values")
document.table(
```

```
    ("n", "a_n"),
    tuple((n, Fraction(1, n + 1)) for n in range(4)),
    caption="Four exact values",
)
document.code("assert a_0 == 1", language="python")
document.bullet_list(("typed blocks", "ordered construction"))

markdown = render_markdown(document)
assert len(document.blocks) == 6
assert markdown.startswith("# Exact coordinate report")
print(len(document.blocks), markdown.splitlines()[0])
```

Observed output:

```
6 # Exact coordinate report
```

The same `document` can be sent to every renderer. Rendering must not reorder or replace its blocks. Equality here is semantic: the HTML and LaTeX outputs present the same block sequence, but escaping, markup, styling, and whitespace make byte equality neither expected nor desirable.

59.3 Tables, figures, and renderer boundaries

`Table.from_rows()` converts ordinary header and row sequences into tuples and then applies the same rectangularity checks as the direct constructor. Values inside table cells remain typed until a renderer chooses their presentation. This is why an exact `Fraction(1, 2)` can remain recognizable to JSON export while a text renderer prints a human-readable value.

Figures are path references at construction time. Markdown, HTML, LaTeX, and PDF export localize each figure beside the target document under a deterministic name such as `report_asset_001.png`. The document model does not read the image or prove that the path exists. That filesystem check belongs to the writer.

59.4 Report builders

Three builders cover common conversions:

- `document_from_data(value, ...)` creates a general report from a supported scalar, mapping, table-like object, dataclass, result, or existing document.
- `sequence_document(values, ...)` presents an indexed finite sequence.
- `certificate_document(certificate, ...)` expands certificate fields into a report that keeps validity, exactness, diagnostics, assumptions, and limitations visible.

If `document_from_data` receives an existing `MathDocument`, the writer uses that document rather than wrapping it in a second report. For ordinary data it passes through the presentation serializer and table-shape logic. This route is intentionally more permissive than portable persistence: it produces a report, not authority to reconstruct an arbitrary Python class.

```
from komorebi_math import Certificate, Exactness
from komorebi_math.export import certificate_document, render_text

certificate = Certificate(
    True,
    Exactness.NUMERICAL,
    tolerance=1e-10,
    residuals={"reconstruction": 2e-13},
    assumptions_checked=("finite matrix dimensions",),
)
report = certificate_document(certificate, title="Numerical certificate")
text = render_text(report)
assert "reconstruction" in text
assert "1e-10" in text
print(report.metadata.title, len(report.blocks))
```

The model rejects structural errors early, but it does not certify the mathematical truth of prose or raw equations. Evidence remains the responsibility of the object and certificate used to build the report.

60. Renderers and writers

The export package separates pure renderers from filesystem writers. A renderer maps a document and renderer options to text. A writer resolves a format, enforces destination policy, localizes assets, writes bytes, and returns an `ExportResult` or `ExportBundleResult` with a certificate.

60.1 Text, Markdown, HTML, and LaTeX

`render_text`, `render_markdown`, `render_html`, and `render_latex` cover every typed document block. Text is a low-markup human view. Markdown uses headings, tables, fenced code, lists, display math, and ordinary image references. HTML escapes prose and can include a MathJax loader. LaTeX escapes presentation text, preserves equation source, chooses `tabular` or `longtable`, and emits a complete compilable document.

`HtmlRenderOptions` contains `include_mathjax` and an optional standalone `title`. `LatexRenderOptions` contains the document class, class options, package tuple, table-of-contents flag, section-numbering flag, and geometry. The class and option strings are validated before they can enter the preamble. `latex_escape()` protects ordinary text characters such as `_`, `&`, and `%`, while `latex_label()` normalizes label material. Equation source and `RawLatex` remain trusted LaTeX and are not escaped as prose.

60.2 Eleven on-disk formats

`ExportFormat` defines the supported universe:

Format	Typical suffix	Writer behavior
JSON	`.json`	sorted-key presentation data with finite JSON values
YAML	`.yaml`, `.yml`	safe PyYAML dump. Optional dependency
CSV	`.csv`	rectangular table with comma delimiter
TSV	`.tsv`	rectangular table with tab delimiter
text	`.txt`	renderer-neutral document converted to plain text
Markdown	`.md`, `.markdown`	Markdown report with localized figures
HTML	`.html`, `.htm`	standalone HTML report with localized figures
LaTeX	`.tex`	complete LaTeX source with localized figures
PDF	`.pdf`	generated LaTeX compiled by a local engine
NPY	`.npy`	one NumPy array with `allow_pickle=False`
NPZ	`.npz`	compressed named arrays with pickle disabled

`infer_export_format()` uses the target suffix. Explicit names accept the aliases `md`, `tex`, `txt`, and `yml`. An unknown or absent suffix raises `ValidationError` unless the caller supplies a format.

`ExportOptions` centralizes title, author, date, abstract, overwrite policy, parent creation, type preservation, JSON indentation, certificate inclusion, generated-TeX retention, and HTML, LaTeX, and compiler options. The default is conservative about replacement: an existing output is rejected unless `overwrite=True`.

60.3 Results and path safety

`export_data()` writes one value. `export_document()` is the explicit document route. `export_many()` writes one value under several suffixes, and `export_bundle()` creates a ZIP containing a selected multi-format family. Each single-file result records the absolute path, resolved format, byte count, companion files, and certificate. `result.require()` first requires the certificate and then returns the primary path.

Text and binary writers stage bytes in the destination directory and replace the target after the staged write completes. Parent directories are created by default. `create_parents=False` converts a missing parent into a validation failure. These helpers provide controlled local replacement, not a distributed transaction or a guarantee against every operating-system failure.

Figure localization checks that the source is a regular file. A conflicting localized asset is rejected unless overwrite is enabled. If the existing asset already has exactly the same bytes, it can be reused safely.

```
from pathlib import Path
from tempfile import TemporaryDirectory
from zipfile import ZipFile

from komorebi_math.export import ExportOptions, export_bundle, export_many

value = tuple({"n": n, "square": n * n} for n in range(5))
with TemporaryDirectory() as temporary:
    root = Path(temporary)
    options = ExportOptions(overwrite=True, title="Square table")
    results = export_many(
        value,
        root / "many",
        base_name="squares",
        formats=("json", "csv", "markdown", "latex"),
        options=options,
    )
    assert all(result.valid for result in results)

    bundle = export_bundle(
        value,
        root / "squares.zip",
        base_name="squares",
        formats=("json", "markdown", "latex"),
        options=options,
    )
    bundle.certificate.require()
    with ZipFile(bundle.path) as archive:
        assert "MANIFEST.sha256" in archive.namelist()
    print(tuple(result.format.value for result in results))
    print(bundle.valid, len(bundle.members))
```

Observed output:

```
('json', 'csv', 'markdown', 'latex')
True 4
```

The member count includes the internal `MANIFEST.sha256` record.

60.4 Local LaTeX compilation

PDF export renders a `.tex` companion and invokes a local compiler. The supported engines are `auto`, `pdflatex`, `xelatex`, `lualatex`, and `latexmk`. `available_latex_engines()` reports executable availability, while `resolve_latex_engine()` selects the first supported installed engine for `auto` or rejects a missing requested engine.

`LatexCompileOptions` controls engine, positive pass count, finite positive timeout, auxiliary retention, optional job name, and `shell_escape`. Shell escape is disabled by default. The direct engines receive `-no-shell-escape`. The `latexmk` route uses the corresponding safe compiler configuration. The result records source, PDF, log, engine, command,

return code, elapsed time, and a certificate. A successful process code is not enough: the expected PDF must also exist and be nonempty.

PDF export keeps the generated `.tex` file by default and always records the compiler log as a companion. Set `keep_tex=False` only when the source was not already present and the caller does not need it. Compilation is a local tool contract. Missing engines raise `BackendUnavailableError` rather than silently pretending that a PDF was produced.

60.5 Manifested bundles and their limit

`export_bundle()` writes the requested formats in a temporary directory, computes SHA-256 lines for the staged members, inserts `MANIFEST.sha256` into the ZIP, and writes the same manifest beside the archive as `<archive>.manifest`. Archive member names are derived from the validated base name and supported suffixes.

The bundle certificate establishes that the primary archive was written and is nonempty under the writer's local checks. It is not a cryptographic proof that a future extraction still matches the manifest. A release or handoff workflow should reopen the ZIP, reject unsafe or duplicate paths, verify its CRC, and replay each digest. This manual's own delivery bundle performs that stronger replay separately.

61. Portable persistence

Portable persistence is the strict reconstruction layer. It accepts only finite JSON primitives, explicit structural tags, and registered Komorebi Math codecs. It never imports a class named by untrusted payload data.

61.1 Portable values, codecs, and migrations

The portable grammar begins with `None`, booleans, integers, finite floats, strings, and lists. Tagged values have exactly three fields:

$$\{\texttt{__komorebi_type__} : t, \texttt{__version__} : n, \texttt{data} : v\}.$$

Built-in structural tags cover `Fraction`, complex numbers, paths, bytes, tuples, sets, frozen sets, mappings, and NumPy arrays. Sets and mapping entries are ordered by canonical encoded JSON so that logically equal values have a deterministic digest. NumPy arrays record dtype, shape, and flattened data and are reconstructed without pickle.

`PortableCodec` binds a stable tag and positive version to one Python type, encoder, and decoder. `register_portable_codec()` rejects duplicate types or tags unless explicit replacement is requested. `PortableMigration` advances exactly one version, from n to $n + 1$. Loads run every registered adjacent step until the active codec version is reached. A missing edge is a hard failure, not permission to guess at an older schema.

61.2 Canonical envelope and digest

`dumps_portable()` wraps the encoded payload in exactly five fields:

```
schema, schema_version, producer_version, payload_sha256, payload
```

The schema is `komorebi-math.portable` and the envelope schema version is 1. The payload digest is

$$d(x) = \mathrm{SHA256}\left(\mathrm{UTF8}(\mathrm{CanonicalJSON}(E(x)))\right),$$

where canonical JSON uses sorted keys, compact separators, UTF-8, and rejects NaN and infinity. The pretty-print indentation of the outer envelope does not change the payload digest. `portable_digest(value)` computes the same digest directly from the registered encoding.

`loads_portable()` rejects invalid UTF-8 JSON, duplicate object keys, nonstandard nonfinite constants, missing or additional envelope fields, an unknown schema, an unsupported schema version, and a digest mismatch before it attempts reconstruction.

```python
import json
from fractions import Fraction

import numpy as np

from komorebi_math.exceptions import ValidationError
from komorebi_math.export import dumps_portable, loads_portable, portable_digest

state = {
    "weights": (Fraction(1, 3), Fraction(2, 3)),
    "matrix": np.arange(6).reshape(2, 3),
}
encoded = dumps_portable(state)
restored = loads_portable(encoded)
assert restored["weights"] == state["weights"]
```

```
assert np.array_equal(restored["matrix"], state["matrix"])

envelope = json.loads(encoded)
envelope["payload_sha256"] = "0" * 64
try:
    loads_portable(json.dumps(envelope))
except ValidationError:
    tamper_status = "rejected"
else:
    raise AssertionError("tampered payload was accepted")

print(portable_digest(restored))
print(tamper_status)
```

The digest printed by this example is deterministic for the same package codec contract and value. It should be compared as data, not hard-coded across an intentional schema or codec-version change.

61.3 Decode and resource guards

Decoding validates every structural tag. Fractions require exactly a numerator and denominator and reject zero denominators. Byte strings require valid base64. Sets reject unhashable members. Mappings reject malformed pairs, unhashable keys, and duplicates after reconstruction. NumPy shapes must contain nonnegative dimensions, the flattened data length must equal the declared shape product, and the values must convert to the declared dtype.

Unknown codec tags, future versions, missing migrations, decoder failures, and nonfinite floating or complex components are rejected. The default maximum serialization and deserialization depth is 128. Callers may choose a smaller positive limit for externally supplied material.

These checks make the decoder closed over the active codec registry. They do not make arbitrary data trustworthy, private, or authenticated. SHA-256 detects accidental or adversarial payload modification when the expected digest remains part of the untampered envelope. It is not a keyed signature.

61.4 Atomic save and load

`save_portable()` serializes first, writes a temporary sibling, flushes and `fsync`s the file, and then uses `os.replace` to install the destination. An existing path is rejected unless `overwrite=True`. If failure occurs before replacement, the temporary file is removed and the original destination is left in place. `load_portable()` reads bytes and delegates all schema, digest, depth, and codec checks to `loads_portable()`.

```
from fractions import Fraction
from pathlib import Path
from tempfile import TemporaryDirectory

from komorebi_math.export import load_portable, save_portable

with TemporaryDirectory() as temporary:
    path = Path(temporary) / "state.km.json"
    value = (Fraction(5, 8), {"levels": (2, 4, 8)})
    save_portable(value, path)
    assert load_portable(path) == value
    save_portable(value, path, overwrite=True)
    print(path.name, path.stat().st_size > 0)
```

61.5 Frozen package codec contract

The live registry is dynamic because extensions can add codecs. The release contract separately freezes the package-owned inventory. For `1.0.0b4` it contains 51 Komorebi Math codec entries in deterministic tag order and one registered migration. `nonlinear.packet_jet_snapshot` is at version 2 with the adjacent migration from version 1.

`compare_portable_codec_contract()` reports added and removed tags, changed implementation types, decreased or increased versions, and missing adjacent migrations. Added codecs are compatible. Removed tags, changed types, version decreases, and incomplete migration chains invalidate the certificate.

An extension codec is not silently added to the stable package contract. An extension that promises long-term reconstruction needs its own frozen identity, versioning policy, migration tests, and compatibility record.

62. Provenance, registries, and extensions

The provenance and registry modules answer different questions. Provenance describes the claim attached to a public object. The feature registry describes the implementation status of a mathematical domain. The extension API lets an installed distribution contribute behavior through a standard Python entry point. None of these records substitutes for a mathematical certificate.

62.1 Mathematical provenance

`ClaimStatus` is a claim-kind alphabet:

```
definition, proved, classical input, conditional theorem,
finite certificate, heuristic, conjecture, planned
```

It is not a scalar confidence ordering. In particular, `finite certificate` does not imply a proved global theorem, and `classical input` identifies dependency on established material rather than a lower numerical score.

`ProvenanceRecord` stores an immutable identifier, title, status, source, assumptions, limitations, and optional verification-suite name. The `documented()` decorator attaches a record to a function or class and registers its qualified name. `register_record()` supports explicit records, while `describe()` accepts an object or qualified name and returns a record or `None`. `records()` returns a copy of the registry.

```
import komorebi_math as km

record = km.describe(km.PacketSystem)
assert record is not None
assert record.identifier == "PSC-I-T20"
assert record.status.value == "proved"
print(record.identifier, record.verification_suite)
```

Observed output:

```
PSC-I-T20 core.packets
```

62.2 Feature registry

`FeatureStatus` contains `implemented`, `partial`, `experimental`, `planned`, and `external theorem required`. `FeatureSpec` records a unique key, module, title, status, summary, papers, dependencies, and public object names. `FeatureRegistry.register()` rejects an existing key unless `replace=True`. `all()` returns features in sorted key order, and `by_status()` filters by enum identity.

The package-level `FEATURES` registry backs `register_feature()`, `feature()`, and `features()`. A domain marked implemented means its declared finite API is present. It does not upgrade every associated research program to a theorem.

62.3 Extension contract and discovery

`ExtensionSpec` records name, version, API version, contributed modules, optional dependencies, description, and an optional zero-argument registration callback. `activate()` calls that callback once per explicit activation call. The API does not attempt to infer idempotence, undo registration, or modify the stable namespace automatically.

Installed extensions advertise entry points in the group `komorebi_math.extensions`. `discover_extensions(activate=False)` loads each entry point, accepts either an `ExtensionSpec` or a zero-argument factory that returns one, and produces one `ExtensionLoadResult` per candidate. With `activate=True`, the registration callback runs before the result is marked loaded.

Import, factory, type, and activation failures are caught per entry point. A failed result has `loaded=False` and an error string of the form `ExceptionType: message`. Discovery then continues to the next candidate. This is failure isolation, not a security sandbox. Extension code executes in the current Python process with that process's permissions.

```
from komorebi_math.extensions import ExtensionSpec
from komorebi_math.registry import (
    FeatureRegistry,
    FeatureSpec,
    FeatureStatus,
)

activated = []
extension = ExtensionSpec(
    name="miniature-demo",
    version="0.1",
    api_version="1",
    modules=("miniature_demo",),
    register=lambda: activated.append("ready"),
)
extension.activate()

registry = FeatureRegistry(
    (
        FeatureSpec(
            key="demo",
            module="miniature_demo",
            title="Miniature extension",
            status=FeatureStatus.EXPERIMENTAL,
            summary="A local extension example.",
        ),
    )
)
assert activated == ["ready"]
assert registry.get("demo").status is FeatureStatus.EXPERIMENTAL
print(extension.name, registry.get("demo").status.value)
```

An installable extension would expose a factory under the entry-point group in its own `pyproject.toml`. Merely constructing `ExtensionSpec` does not install or discover anything.

62.4 Stable-core and persistence boundaries

Extensions may register features, adapters, provenance records, or codecs through documented APIs, but those additions do not become stable package symbols by implication. They also do not enter the 51-codec package contract unless a later package release intentionally freezes them. Downstream code should record the extension name, version, API version, and optional dependencies alongside any extension-owned portable state.

The command `komorebi describe QUALIFIED_NAME` exposes a provenance record as JSON and returns a failure status when no record exists. `komorebi doctor` reports optional dependency and extension status. These are diagnostic views. They do not activate extensions unless the underlying command explicitly asks the discovery layer to do so.

63. Complete CLI reference

Using the command reference. Begin with the package identity command from Chapter 4, using the same interpreter that will run the actual operation. Command names in the tables are followed by arguments; names such as `PATH` and `OBJECT` stand for values you supply. Quote paths containing spaces in the syntax of your shell. PowerShell's continuation character is a backtick, while the POSIX examples use a backslash. Copy a command from the matching shell listing rather than mixing those forms.

Choose a new output path for a first run so that a refusal to overwrite is easy to distinguish from a mathematical failure. After a command completes, check its exit status and the expected output file or structured result. For a command's complete option set, use its entry in Appendix F; the short tables here explain purpose and relationships.

The console entry point `komorebi` and `python -m komorebi_math` use the same `build_parser()` and command functions under one interpreter. This equivalence does not cross environments: two shells can resolve different installations, optional backends, or LaTeX executables.

63.1 Identity, verification, API, and release commands

Command	Contract
`info`	print package identity and summary
`doctor`	inspect optional dependencies and discovered extensions
`features`	print registered mathematical domains and statuses
`describe OBJECT`	print one provenance record for a qualified object
`verify SUITE`	run one of the 51 frozen verification-suite names
`api check`	compare research, beta, or stable public API identity
`api snapshot --output PATH`	write a selected API manifest. `--force` replaces
`golden`	run committed behavioral semantics
`release contract`	validate and print the frozen release contract
`release environment`	print the current runtime qualification record
`release qualify`	run installed freeze qualification, optionally probing GPU
`release upgrade`	check predecessor API, fixtures, codecs, and migrations
`release evidence`	validate digest-checked platform evidence
`release documentation`	validate exact stable documentation coverage
`release readiness`	run public-beta or stricter final readiness

`api check` accepts `--manifest`, `--strict-signatures`, and mutually exclusive `--beta` or `--stable` modes. Without a tier flag it checks the research surface. `api snapshot` requires `--output` and uses the same tier selection.

The release commands that can write JSON accept `--output` and refuse an existing destination unless `--force` is present. `--gpu` executes optional CUDA probes. `--require-gpu` makes a successful GPU gate mandatory. The `--final` readiness flag additionally requires current-version evidence across every declared platform family.

63.2 Streams, sequences, and mathematical domains

Family	Subcommands and essential inputs
`stream`	`inspect PATH` with optional incomplete/chunk checks. `verify PATH`
`sequence`	`NAME`, optional term count, index, modulus, and strongest compilation
`coordinates`	`euler-fibonacci`, `recurrence`, `catalog`, `basis`, `model`, `polynomial`, `combinatorial`, `spectral`, `packet`, `formal`

Family	Subcommands and essential inputs
`generating`	`mask`, `cartier`, `kernel`, `bell`
`arithmetic`	`standard-window`, `euler`
`deformation`	`count` for smooth, weighted, nilpotent, or normal-crossing charts
`langlands`	`tame-orbit`, `heisenberg`, `completion`

The parser validates enumerated choices such as spectral coordinate kind, packet index kind, deformation chart, and LaTeX engine. Structured numeric inputs use explicit comma, semicolon, rational, matrix, or JSON decoders in the command function. A syntactically valid string can still fail semantic validation when dimensions, moduli, primes, or mathematical preconditions are invalid.

The completion command accepts a comma-separated subset of the five gate names and returns the conditional completion report. Supplying every gate records that the finite input claims all gates satisfied. It does not independently prove the external automorphic or realization theorems named by those gates.

63.3 Export and LaTeX commands

`export formats` prints the eleven public format names. `export sequence`, `export standard-window`, `export euler`, and `export json` convert their domain values to one requested output. `export bundle` reads JSON and writes a multi-format archive. `--base-name` and comma-separated `--formats` control its members.

All value-export commands share these options:

```
--output, --format, --title, --author, --date, --abstract,
--document-class, --toc, --mathjax, --engine, --passes, --timeout,
--keep-auxiliary, --no-keep-tex, --no-type-metadata, --force
```

The output path is required. Its suffix selects the format unless `--format` overrides it. `--force` is the CLI translation of `overwrite=True`. `--no-type-metadata` affects presentation serialization and is not a switch to portable persistence.

`latex engines` prints availability for `pdflatex`, `xelatex`, `lualatex`, and `latexmk`. `latex compile SOURCE` accepts an optional PDF output, engine, pass count, timeout, and auxiliary-retention flag. A non-PDF output suffix is a usage error.

```
python -m komorebi_math export formats
python -m komorebi_math verify export
python -m komorebi_math describe komorebi_math.core.packets.PacketSystem
python -m komorebi_math export sequence fibonacci --terms 12 --output fibonacci.tex --title "Fibonacci report"
python -m komorebi_math export bundle --input data.json --output report.zip --formats json,markdown,latex --force
```

63.4 Output, exit, and scripting discipline

Commands that promise structured output print JSON followed by a newline. The verification runner also prints one readable line per check and a final count. Scripts should use the exit status as the primary success signal and parse JSON only for fields promised by that command.

The intended exit convention is:

- 0 for a completed command whose requested predicate passed.
- 1 for a completed verification or required capability that failed.
- 2 for parser usage, invalid choices, or command-specific usage errors.

Some lower-level `ValidationError` or backend failures are intentionally not converted into false success records. Callers should capture stderr and a nonzero process status. Filesystem-writing commands refuse replacement unless their `--force` option is present.

`komorebi verify all` is the historical compatibility aggregate. It does not run the entire pytest suite, `verify block20`, or the source publication gate. For release qualification, run the named release command or the bundled handoff wrapper whose scope is explicitly recorded.

63.5 End-to-end command replay

The installed export smoke path for this part is:

```
python -m komorebi_math verify export
```

For `1.0.0b4`, the expected final line is:

```
verification passed 5/5 checks
```

Those checks cover presentation serialization, LaTeX rendering, the portable file formats, manifested multi-format bundles, and local PDF compilation when an engine is installed. The shipped `examples/export_reports.py` adds a fuller report and bundle replay, while `examples/portable_nonlinear_state.py` checks a registered nonlinear snapshot roundtrip and rejects a tampered payload.

Part XI. Connected recipes

The earlier parts document objects one domain at a time. Part XI instead documents complete finite journeys. Every recipe crosses at least two API layers and ends at an observable result, a certificate, or a replay record. This is where the library becomes a working calculus rather than a catalog of constructors.

A recipe has the evidence tuple

$$\mathcal{E} = (I, H, A, O, C, F, R),$$

where I is the input, H the hypotheses, A the ordered actions, O the output, C the certificate predicates, F the failure variants, and R the replay record. A successful numerical output without its checked hypotheses is not a completed recipe. Conversely, an exact finite identity may need no tolerance, but it still needs enough input and version information to be reproduced.

Choose one connected task. These chapters are intended to be entered from a result you want, rather than completed in sequence.

Desired result	Recipe chapter	Useful background
Reconstruct an object after a coordinate change	64	Chapters 9 through 20, as required by the chosen objects
Evaluate a sequence or finite arithmetic model	65	Chapters 21 through 26
Analyze, filter, or resample a signal	66	Chapters 31 through 36
Compose derivatives or reduce finite dynamics	67	Chapters 37 through 46
Integrate or evaluate a typed expression	68	Chapters 47 through 52
Explore a finite quantum, deformation, or Langlands model	69	Chapters 53 through 58
Export and replay a result	70	Chapters 59 through 63

For each recipe, first identify the inputs, dependencies, and expected output. Run the supplied setup and calculation together, then inspect the certificate before changing the example. Make one change at a time, such as a dimension, input sequence, or tolerance. If the change fails, compare the new input with the recipe's stated domain and assumptions before changing another setting. This keeps the example useful as a diagnostic reference.

A printed value is only one checkpoint. Also retain the parameters that define the calculation: for example, the finite degree of a polynomial model or the window and boundary rule of a signal transform. The failure variants explain where a recipe stops applying; they are part of learning the operation, not extra steps required for every ordinary run.

64. Exact algebra and coordinate recipes

Exact algebra recipes begin with a typed domain and end with a reconstruction. The middle objects may be operator paths, seed orbits, basis connections, recurrence defects, or redundant coefficients. The common invariant is that the final physical object agrees with the input under the declared backend.

64.1 Grammar, realization, and seed hull

An `OperatorGrammar` fixes vertices, typed arrows, and parallel path relations. `OperatorRealization` then assigns a dimension to every vertex and a matrix to every arrow. Construction checks shapes and, by default, requires all declared relations. The realized operators can then act on a `SeedSystem`.

For an operator T with $T^3 = T$, a finite recipe is

$$\{T^3 - T = 0\} \longrightarrow T \longrightarrow \operatorname{span}\{T^k v : k \geq 0\}.$$

The seed hull stops when the rank no longer grows or the ambient dimension is reached. A full hull proves that the supplied seeds are cyclic for this finite realization. It does not prove that an abstract infinite-dimensional model is cyclic.

64.2 Transfer paths and basis connections

A packet recipe analyzes an input into source packets, transports those packets along compatible blocks, and synthesizes the target packets. The essential check is global reconstruction, not the appearance of individual blocks:

$$x \xrightarrow{A} (x_r)_r \xrightarrow{T_{sr}} (y_s)_s \xrightarrow{S} \hat{x}, qquad\hat{x} = x.$$

For exact polynomial families the analogous path uses a connection matrix. If B_s and B_t are two bases of the same truncated polynomial space, then the coordinate change $C_{s\to t}$ is accepted only when

$$B_t C_{s\to t} c_s = B_s c_s$$

on the declared degree range. The `connection_transform()` and coordinate family certificates implement this finite statement. Truncation remains part of the hypothesis. A degree-four connection says nothing about a coefficient outside that five-dimensional space.

64.3 Euler values in Fibonacci recurrence coordinates

`euler_fibonacci_coordinates(count)` keeps the first two Euler values as the initial state and stores the defect against the Fibonacci recurrence

$$d_n = E_{n+2} - E_{n+1} - E_n.$$

The reconstruction is a discrete variation-of-constants formula. For `count=18`, the certified record has 16 defects and reconstructs

```
1, 0, -1, 0, 5, 0, -61, 0, 1385, 0,
-50521, 0, 2702765, 0, -199360981, 0, 19391512145, 0
```

exactly. This does not claim that the Euler sequence satisfies the homogeneous Fibonacci recurrence. The defect stream is precisely the information needed because it does not.

64.4 Canonical duals and coefficient gauge

For a full-column-rank analysis matrix A, `AnalysisSynthesisSystem.canonical_dual(A)` constructs

$$S = A^{+}, qquad SA = I.$$

When A is tall, the coordinates are redundant. `AnalysisSynthesisGauge` splits a coefficient vector into its canonical part in $\mathrm{im}(A)$ and its gauge part in $\ker(S)$. Two coefficient vectors are gauge-equivalent when synthesis cannot distinguish them. The physical reconstruction is unique even though the coefficient representative is not.

```
import numpy as np

from komorebi_math import OperatorGrammar, OperatorRealization
from komorebi_math.coordinates import (
    AnalysisSynthesisGauge,
    AnalysisSynthesisSystem,
    euler_fibonacci_coordinates,
    variation_of_constants_prefix,
)
from komorebi_math.core import SeedSystem

grammar = OperatorGrammar(("X",), {"T": ("X", "X")})
grammar.relation(
    "cubic",
    ((1, grammar.word("T", "T", "T")), (-1, grammar.word("T"))),
)
matrix = np.diag([-1.0, 0.0, 1.0])
realization = OperatorRealization(grammar, {"X": 3}, {"T": matrix})
realization.relation_certificate().require()

seeds = SeedSystem({"T": matrix}, (np.ones(3),))
assert seeds.hull_dimension() == 3

record = euler_fibonacci_coordinates(18)
record.certificate().require()
assert record.reconstruct() == variation_of_constants_prefix(record)

analysis = np.array(((1.0, 0.0), (0.0, 1.0), (1.0, 1.0), (1.0, -1.0)))
system = AnalysisSynthesisSystem.canonical_dual(analysis)
gauge = AnalysisSynthesisGauge(system)
gauge.certificate().require()
source = np.array((3.0, -2.0))
canonical = system.analyze(source)
null_shift = gauge.gauge_part(np.array((1.0, 2.0, 3.0, 4.0)))
assert gauge.gauge_equivalent(canonical, canonical + null_shift)
assert np.allclose(system.synthesize(canonical + null_shift), source)
print(seeds.hull_dimension(), len(record.defects), system.redundant)
```

Observed output:

```
3 16 True
```

The principal failure variants are a noncomposable operator word, a relation residual above tolerance, a seed family with a proper hull, a singular analysis matrix, or a coordinate connection used outside its certified degree. Each failure should be preserved as evidence rather than converted into a guessed reconstruction.

65. Arithmetic and sequence recipes

Arithmetic recipes are exact when their inputs are exact. They replace a long prefix or large index by a finite execution model: recurrence, automaton, Cartier representation, carry plan, valuation decomposition, or residue packet. The certificate must distinguish a model declared by the sequence from one merely inferred from a finite sample.

65.1 Catalog to strongest declared model

`compile_sequence()` inspects a `Sequence` in a fixed priority order. A certified declared linear recurrence is used before a declared finite automaton, which is used before a declared holonomic recurrence. Only when no declared model applies does the compiler attempt bounded detection from sample values.

The distinction is visible in the certificate. For a built-in sequence whose metadata declares and certifies a recurrence, the result is exact. For an arbitrary user prefix, agreement on 24 samples remains conditional because a finite prefix cannot identify every infinite continuation.

For the built-in catalog, the shipped compiler recipe reports:

Sequence	Strategy	First terms
Fibonacci	declared linear recurrence	`0, 1, 1, 2, 3, 5, 8, 13`
Thue-Morse	declared finite automaton	`0, 1, 1, 0, 1, 0, 0, 1`
Apéry	declared holonomic recurrence	`1, 5, 73, 1445, 33001`

65.2 Mahler masks and Cartier execution

A Mahler mask represents finite-difference coordinates for a polynomial or integer-valued function. Affine subsequences $f(mx + r)$ remain computable by an exact mask transformation. Cartier sections perform the parallel operation on generating series by selecting coefficients in one residue class.

`compile_finite_mask_carry_plan()` turns a finite mask and base into a carry state system. `compile_cartier_linear_representation()` turns rational series data into a finite linear representation modulo a selected modulus. Its value at a very large index is obtained by reading the base digits rather than materializing every preceding coefficient. The finite representation is the reproducibility record for that evaluation.

65.3 Bell reductions and prime-power kernels

For a normalized factorial family, `PrimePowerKernelCompiler(p, r, s)` records both the p-adic valuation and the surviving unit of

$$\binom{n}{k}^{s} \pmod{p^r}.$$

If the valuation is at least r, the residue is zero without an unnecessary unit computation. `generalized_bell_digit_reduction()` separately reduces a Bell-type index through its declared digit rule and returns a certificate. `q_binomial_valuation_decomposition()` records the mixed-radix carry data for an integer Gaussian binomial.

For $(n, k, p, r, s) = (137, 52, 3, 5, 2)$, the kernel has valuation 4 and residue 81 modulo 3^5. A direct finite assertion against `pow(comb(137, 52), 2, 3**5)` confirms the residue.

65.4 Exact arithmetic polyphase

`arithmetic.polyphase_analyze(values, q)` splits a finite sequence into residue channels

$$a^{(r)} = (a_{qn+r})_n, qquad 0 \le r < q.$$

`ArithmeticPolyphase.reconstruct()` interleaves them exactly. Refinement is allowed only to a divisible finer modulus, and coarsening only to a divisor of the current modulus. This prevents an apparently convenient but undefined channel relabeling.

```
from math import comb

from komorebi_math.arithmetic import polyphase_analyze
from komorebi_math.generating import (
    PrimePowerKernelCompiler,
    generalized_bell_digit_reduction,
)
from komorebi_math.sequences import compile_sequence, fibonacci_sequence

compiled = compile_sequence(fibonacci_sequence())
compiled.certificate.require()
prefix = compiled.first(12)

packet = polyphase_analyze(prefix, 3)
assert packet.reconstruct() == prefix
assert packet.refine(6).coarsen(3).reconstruct() == prefix

kernel = PrimePowerKernelCompiler(3, 5, 2).kernel(137, 52)
assert kernel.residue == pow(comb(137, 52), 2, 3**5)
bell = generalized_bell_digit_reduction(137, 5, 9)
bell.require()
print(compiled.strategy, tuple(map(len, packet.channels)))
print(kernel.valuation, kernel.residue, bell.digit_sum)
```

Observed output:

```
declared linear recurrence (4, 4, 4)
4 81 5
```

Invalid primes, nonpositive exponents, out-of-range subindices, nonunit custom factors, malformed residue addresses, and unjustified detected recurrences are hard boundaries. A congruence certificate applies to the stated modulus and index family only.

66. Signal-processing recipes

Signal recipes combine numerical transforms with finite reconstruction or risk checks. They should record the sampling rate, window, frame and hop lengths, filter coefficients, stream partition, resampling ratio, boundary convention, dtype, and tolerance. Those values are part of the mathematical input, not incidental implementation details.

66.1 Windowed analysis and inverse reconstruction

`stft()` returns frequencies, frame times, complex coefficients, window data, and the exact framing convention needed by its inverse. A roundtrip certificate checks the reconstructed samples against the original signal. Coverage and normalization fail visibly when the chosen window and hop do not support the declared finite reconstruction.

Streaming STFT uses the same frame rule while receiving an explicit chunk partition. Chunk boundaries may change buffering, but they must not change the completed frame values. The shipped production recipe checks both batch and streaming forms on a deterministic 1,024-sample signal.

66.2 Filter design and stateful application

`design_lowpass()` produces an FIR design with a certificate for its declared symmetry and finite response checks. `stream_fir_filter()` then carries the filter state across a supplied sequence of chunk sizes and compares the joined output with batch filtering. Recursive biquad designs use a separate stability certificate and `stream_lfilter()` state replay.

The stream partition belongs in R. A test using only one chunk does not exercise state continuity. Useful adversarial partitions include a one-sample first chunk, uneven interior chunks, and a final chunk shorter than the filter memory.

66.3 Rational resampling

`resample_poly_result(x, L, M)` performs the finite upsample, filter, and downsample path. The result records the rational rate ratio, output signal, and certificate. The recipe must check the expected length and must not describe the operation as lossless inversion: anti-alias filtering and endpoint policy matter, especially when returning to the original rate.

66.4 Packet selection, denoising, and singular recovery

The Haar packet path builds a finite tree, evaluates a declared additive cost, selects a best leaf set, and reconstructs from those leaves. Best means best for that tree, signal, and cost. It is not a universal optimality claim.

`wavelet_denoise()` applies a threshold rule to packet or wavelet coefficients. `denoising_risk_certificate(clean, noisy, estimate)` is an observed finite risk comparison because the clean reference is supplied. It does not certify risk on unseen data. Singular packet recovery is stronger only under its declared separation, sampling, kernel, and sparsity hypotheses. Its result therefore keeps detected cells, locations, amplitudes, and an assumption audit together.

```
import numpy as np

from komorebi_math.signals import (
    denoising_risk_certificate,
    design_lowpass,
    resample_poly_result,
    stft,
    stream_fir_filter,
    wavelet_denoise,
)

rng = np.random.default_rng(1212)
```

```
sample_rate = 1024.0
index = np.arange(1024)
clean = np.sin(2.0 * np.pi * 73.0 * index / sample_rate)
noisy = clean + rng.normal(scale=0.15, size=clean.size)

transform = stft(noisy, sample_rate=sample_rate, frame_length=128, hop_length=32)
transform.roundtrip_certificate(noisy).require()

lowpass = design_lowpass(120.0, num_taps=63, sample_rate=sample_rate)
lowpass.certificate(tolerance=1e-9).require()
streamed = stream_fir_filter(noisy, lowpass, (113, 287, 19, 605))
streamed.certificate.require()

resampled = resample_poly_result(streamed.value, 5, 4)
resampled.certificate.require()

piecewise = np.repeat(np.array((0.0, 2.0, -1.0, 1.5)), 256)
observed = piecewise + rng.normal(scale=0.3, size=piecewise.size)
denoised = wavelet_denoise(observed, levels=5, rule="universal", mode="soft")
denoised.certificate.require()
risk = denoising_risk_certificate(piecewise, observed, denoised.signal)
assert risk.valid
print(transform.frame_count, streamed.value.size, resampled.signal.size, risk.valid)
```

Observed output:

```
33 1024 1280 True
```

The full `examples/production_dsp.py` recipe also checks periodogram frequency, Welch averaging, cross spectra, adaptive NLMS identification, streaming STFT, recursive filtering, and overlap-add convolution. Its deterministic run finds the 73 Hz line, recovers adaptive weights approximately `[0.7, -0.35, 0.15]`, and passes every included certificate.

67. Nonlinear and dynamics recipes

Nonlinear recipes are layered because higher derivatives, correction trees, and reduced dynamics amplify hidden assumptions. The practical rule is to certify each layer before it becomes input to the next one.

67.1 Provider to jet to Faà di Bruno composition

A derivative provider declares source and target dimensions, maximum available order, backend, and exactness. `PacketJetFamily` resolves its multilinear derivatives through source and target packets. `faa_di_bruno()` composes two providers by enumerating set partitions of the requested directions.

For the one-dimensional polynomial providers in the example below, the second directional derivative is exactly 888 and is assembled from two set-partition terms. `packet_jet_support()` separately records which packet addresses are nonzero. Numerical pruning uses the declared tolerance and therefore must not be reported as exact algebraic vanishing.

67.2 Fixed-point trees and majorants

The fixed-point tree workflow expands a finite rooted-tree family, attaches elementary differentials, and compares their norms with a scalar majorant. A majorant certificate gives a finite bound under its supplied coefficient and radius hypotheses. It does not establish convergence beyond the certified order or radius.

67.3 Obstruction, lifting, and holonomy

At each lifting stage the current defect is projected to the obstruction cokernel. A zero obstruction permits the solver to enumerate corrected children. A nonzero obstruction records a stopped branch. The resulting `LiftTree` distinguishes three outcomes that a single success flag would conflate: no lift, one lift, or several compatible lifts.

Holonomy is computed only after a loop of compatible transports exists. The symbolic `examples/obstruction_lifting.py` route uses the optional SymPy profile. Without that dependency, the core NumPy and exact finite verification suites remain available, but that symbolic example should be reported as unavailable rather than silently downgraded.

67.4 Filtered correction and spectral-sequence pages

A filtered complex recipe builds page E_r, computes the induced differential d_r, and advances only after checking the filtration and chain conditions. In the shipped finite example, page dimensions are 2, 2, and 0, with differential ranks 0, 1, and 0. A vanished terminal page is a property of that finite filtered complex, not a general degeneration theorem.

67.5 Markov lumping and Bayesian sufficiency

`MarkovLumping(generator, partition)` checks whether states in one block have the same total rate into every target block. If valid, it produces a coarse generator. `BayesianPartition` uses the same partition and aggregates a prior and likelihood. Fixed sufficiency requires each likelihood to be constant on every block. Thus one partition can support both dynamical and inferential coarse models without identifying the two constructions.

67.6 Feshbach reduction and memory realization

For a block operator

$$H = \begin{bmatrix} A & B \\ C & D \end{bmatrix},$$

the effective operator at spectral parameter z uses the Schur-Feshbach term $A - B(D - zI)^{-1}C$. The eliminated block also defines a memory kernel in time. A complete recipe records the projection, the admissible spectral region, resolvent conditioning, and the state-space realization used for the kernel. Near the spectrum of D, apparent large effects may be conditioning artifacts and must remain visible in the certificate.

```
import numpy as np

from komorebi_math import Exactness, PacketSystem
from komorebi_math.dynamics import BayesianPartition, MarkovLumping
from komorebi_math.nonlinear import PolynomialDerivativeProvider, faa_di_bruno

outer = PolynomialDerivativeProvider(
    {2: np.array([[[2]]], dtype=object)}, exactness=Exactness.EXACT
)
inner = PolynomialDerivativeProvider(
    {1: np.array([[1]], dtype=object), 2: np.array([[[2]]], dtype=object)},
    exactness=Exactness.EXACT,
)
composition = faa_di_bruno(
    outer,
    inner,
    np.array([2], dtype=object),
    (np.array([3], dtype=object), np.array([4], dtype=object)),
)
composition.certificate.require()

generator = np.array(
    ((-0.4, 0.2, 0.0, 0.0), (0.3, -0.3, 0.0, 0.0),
     (0.1, 0.1, -0.6, 0.4), (0.0, 0.0, 0.6, -0.4))
)
partition = (0, 0, 1, 1)
lumping = MarkovLumping(generator, partition)
lumping.certificate().require()
bayes = BayesianPartition(partition)
posterior = bayes.coarse_posterior(
    np.array((0.1, 0.2, 0.3, 0.4)),
    np.array((2.0, 2.0, 0.5, 0.5)),
)
posterior.certificate.require()
assert bayes.is_fixed_sufficient((np.array((2.0, 2.0, 0.5, 0.5)),))
print(composition.value[0], composition.raw_term_count)
print(np.round(posterior.value, 8))
```

Observed output:

```
888 2
[0.63157895 0.36842105]
```

68. Integration and expression recipes

Integration and typed expressions meet at a useful boundary. The integration layer supplies exact measures, polynomial integrals, and error-bounded quadrature. The expression layer records how coordinate changes, derivatives, antiderivatives, substitutions, and integration maps compose. Together they can verify both a scalar result and the operator path that produced it.

68.1 Exact polynomial integration

`IntervalMeasure(a, b)` with rational endpoints and density is an exact finite measure. `ExactPolynomial` stores finite rational coefficients. `integrate(polynomial, measure)` returns an exact `IntegralResult` when exact polynomial coefficients and exact measure moments support the moment contraction.

For

$$f(y) = y^4 - 2y + 1, \qquad \mu = \tfrac{1}{2}\,dy \text{ on } [1, 3],$$

the exact result is $106/5$. Monomial, Euler-polynomial, and Fibonacci-polynomial coordinate pairings all return the same scalar when their connection maps and integration covectors are used consistently.

68.2 Certified Simpson evaluation

`CompositeSimpson(subintervals, derivative_bound)` combines sampled values with a supplied fourth-derivative bound. On the same polynomial, 16 panels produce `21.200032552083332`. The observed error and rigorous absolute bound are both approximately 3.255×10^{-5}.

The derivative provider is part of the hypothesis. The library intentionally tests a degree-six polynomial that vanishes at the three Simpson nodes but has true integral $1/840$. The raw three-node functional returns zero, while an attempt to certify exactness is rejected. Node agreement alone is not an integration theorem.

68.3 Pushforwards and affine change of variables

For $T(x) = 2x + 1$ and Lebesgue measure on $[0, 1]$, `PushforwardMeasure` represents the image measure, whose density is $1/2$ on $[1, 3]$. The recipe checks

$$\int_{[1,3]} f(y)\,\frac{dy}{2} = \int_{[0,1]} f(2x + 1)\,dx.$$

The absolute Jacobian belongs to the pushed measure. Omitting it can preserve the shape of a formula while doubling the integral.

68.4 Typed normalization, planning, and replay

An expression node carries its domain and codomain. Invalid compositions fail before matrix multiplication. `normalize_expression()` computes a canonical finite representation, and `simplify_expression()` applies recorded rewrite steps. Equality is established by matching normal forms under the implemented finite rules, not by comparing display strings.

`compile_expression()` selects executable instructions and uses dynamic programming for matrix chains. In the full worked recipe, the selected cost is 244 scalar multiplications rather than 260 for the left-associated chain. Direct evaluation and planned execution share the same normal-form digest.

68.5 Portable expression boundary

`expression_to_json()` uses the frozen expression schema and `expression_from_json()` validates it before reconstruction. `expression_digest()` binds the normalized portable content. Unknown node types, malformed spaces, invalid rational values, excessive nesting, and digest changes are rejected. As with general portable persistence, the digest detects modification but is not a keyed signature.

```
from fractions import Fraction

from komorebi_math.expression import (
    PolynomialCoordinateSpace,
    affine_substitution,
    apply,
    evaluate_expression,
    exact_value,
    expression_digest,
    expression_from_json,
    expression_to_json,
    interval_integration,
)
from komorebi_math.integration import (
    ExactAffineMap,
    ExactPolynomial,
    IntervalMeasure,
    PushforwardMeasure,
    integrate,
)

Q = Fraction
base = IntervalMeasure(0, 1)
chart = ExactAffineMap(((2,),), (1,))
pushed = PushforwardMeasure(base, chart)
polynomial = ExactPolynomial.univariate((1, -2, 0, 0, 1))
assert integrate(polynomial, pushed).require() == Q(106, 5)
assert integrate(chart.pullback(polynomial), base).require() == Q(106, 5)

monomials = PolynomialCoordinateSpace(
    "y", 4,
    tuple(tuple(Q(int(i == degree)) for i in range(degree + 1))
          for degree in range(5)),
    "monomials",
)
value = exact_value(monomials, (1, -2, 0, 0, 1))
pulled = apply(affine_substitution(monomials, 2, 1), value)
expression = apply(interval_integration(monomials, 0, 1), pulled)
assert evaluate_expression(expression).scalar == Q(106, 5)

encoded = expression_to_json(expression)
restored = expression_from_json(encoded)
assert expression_digest(restored) == expression_digest(expression)
print(evaluate_expression(expression).scalar, len(expression_digest(expression)))
```

Observed output:

```
106/5 64
```

69. Quantum, deformation, and Langlands recipes

These recipes connect finite linear-algebra models while preserving the line between an implemented certificate and an external theorem. The package can verify a finite symmetry decomposition, channel identity, error-correction condition, residual support computation, local packet, or completion gate record. It does not turn a satisfied finite model into an unconditional global Langlands realization.

69.1 Symmetry packets and circuit expansion

A finite group action on $\mathcal{H}$ yields character projectors P_ρ. The symmetry recipe checks projector idempotence, orthogonality, and completeness before converting packet-supported operators into a circuit expansion. The shipped two-qubit example completes with reconstruction error at floating-point scale and reports an overlap of approximately 0.5 for its selected state.

69.2 Channels and Knill-Laflamme conditions

`amplitude_damping(gamma)` constructs a channel from Kraus operators and checks trace preservation. At $\gamma = 0.2$, the excited-state density has diagonal `[0.2, 0.8]` after the channel.

`QuantumCode(projector, errors)` evaluates

$$PE_a^* E_b P = \alpha_{ab} P$$

for every declared error pair. `knill_laflamme_report()` records the scalar matrix and residuals. `recovered_channel_certificate()` then checks the finite recovery construction. Both must pass. A channel certificate alone says nothing about a particular code, and a code certificate covers only the supplied error family.

69.3 Residual forests and visible support

A deformation forest records residual states, children, chart labels, and support data. Visibility selects branches that survive the declared support tests. The result is finite and combinatorial. A missing branch may mean a failed local lift, an excluded chart, or a visibility cutoff, so the diagnostic reason must travel with the support packet.

69.4 Tame packets and Heisenberg models

A Frobenius orbit produces a tame packet whose dimension is the orbit length. Alternating forms and quadratic refinements define finite Heisenberg data and symplectic diagnostics. In the shipped prime-three example, the Frobenius orbit is `(1, 2, 4)`, the tame and Heisenberg dimensions are both 3, and the selected packet projector has rank 1.

These are exact finite representation statements. Naming the objects after local or Langlands structures does not assert a global correspondence beyond the constructors and certificates documented in Part IX.

69.5 Ramification towers and conditional completion

`RamificationCliffordTower` assembles finite stages and reports breaks, dimensions, packet phases, and Clifford data. `AutomorphicCompletionReport` then evaluates five named gates: residual, patching, comparison, fiber, and realization.

The boolean completion result is the conjunction of supplied gate statuses. In the example below, four gates are explicitly supplied as satisfied and the realization gate remains false. The correct result is therefore `completion_ready=False`, with `realization` preserved as the unmet gate. This is the intended boundary: the object tracks what would still be required and does not infer the external theorem.

```
import numpy as np

from komorebi_math.langlands.completion import (
    AutomorphicCompletionReport,
    CompletionGate,
    GateStatus,
)
from komorebi_math.langlands.local import AlternatingForm, QuadraticRefinement
from komorebi_math.langlands.ramification import (
    ProjectiveMultiplicityPacket,
    RamificationCliffordTower,
    RamificationStage,
)
from komorebi_math.quantum import QuantumCode, amplitude_damping

channel = amplitude_damping(0.2)
channel.certificate().require()
excited = np.diag((0, 1)).astype(complex)
assert np.allclose(np.diag(channel.apply_density(excited)), (0.2, 0.8))

identity = np.eye(2)
x = np.array(((0, 1), (1, 0)), dtype=complex)
zero = np.zeros(8)
zero[0] = 1
one = np.zeros(8)
one[-1] = 1
projector = np.outer(zero, zero) + np.outer(one, one)
errors = (
    np.eye(8), np.kron(np.kron(x, identity), identity),
    np.kron(np.kron(identity, x), identity),
    np.kron(np.kron(identity, identity), x),
)
code = QuantumCode(projector, errors)
code.knill_laflamme_report().certificate.require()
code.recovered_channel_certificate().require()

form = AlternatingForm(((0, 1), (1, 0)), prime=2)
packet = ProjectiveMultiplicityPacket(
    2, 2, form, QuadraticRefinement(form, (0, 0))
)
tower = RamificationCliffordTower((
    RamificationStage(0, 8, 4, 2, 2, packet, "wild"),
    RamificationStage(1, 4, 2, 2, 1, None, "deeper"),
))

completion = AutomorphicCompletionReport({
    CompletionGate.RESIDUAL: GateStatus(True, "supplied"),
    CompletionGate.PATCHING: GateStatus(True, "supplied"),
    CompletionGate.COMPARISON: GateStatus(True, "supplied"),
    CompletionGate.FIBER: GateStatus(True, "supplied"),
    CompletionGate.REALIZATION: GateStatus(False, "external"),
})
print(tuple(float(value) for value in np.round(
    np.diag(channel.apply_density(excited)).real, 1
)))
print(tower.report().breaks, bool(completion.result().value),
      tuple(gate.value for gate in completion.unmet_gates))
```

Observed output:

```
(0.2, 0.8)
(0, 1) False ('realization',)
```

70. Export and reproducibility recipes

The final connected recipe moves from an in-memory result to a replayable handoff. It uses several contracts because no single format proves every property.

70.1 Result to document to rendered formats

`document_from_data()` or a domain-specific report builder turns a supported value into `MathDocument`. Text, Markdown, HTML, and LaTeX renderers consume the same ordered block model. PDF export adds a local LaTeX compiler and its log. A rendering certificate establishes that the declared local write completed. It does not certify the mathematical claims inside arbitrary prose.

70.2 Portable object replay

When reconstruction matters, use `dumps_portable()` or `save_portable()` rather than presentation JSON. The portable envelope carries a schema version, producer version, registered typed payload, and SHA-256 payload digest. A roundtrip recipe checks both the reconstructed type and its domain certificate. The active codec contract should be recorded when long-lived files or extension-owned codecs are involved.

70.3 Manifested multi-format bundle

`export_bundle()` renders a selected format family in a staging directory, writes `MANIFEST.sha256`, and packages the members in one ZIP. The manifest is also written beside the archive. A consumer should reopen the ZIP, reject duplicate and unsafe names, verify the CRC, and compare every listed digest. The writer's success certificate alone does not prove that a later copy or extraction is unchanged.

```
from pathlib import Path
from tempfile import TemporaryDirectory
from zipfile import ZipFile

from komorebi_math.export import (
    ExportOptions,
    dumps_portable,
    export_bundle,
    loads_portable,
    portable_digest,
)

rows = tuple({"n": n, "square": n * n} for n in range(6))
with TemporaryDirectory() as temporary:
    root = Path(temporary)
    bundle = export_bundle(
        rows,
        root / "squares.zip",
        base_name="squares",
        formats=("json", "markdown", "latex"),
        options=ExportOptions(overwrite=True, title="Square table"),
    )
    bundle.certificate.require()
    with ZipFile(bundle.path) as archive:
        names = tuple(sorted(archive.namelist()))
        assert "MANIFEST.sha256" in names

    encoded = dumps_portable(rows)
    restored = loads_portable(encoded)
    assert restored == rows
```

```
    assert portable_digest(restored) == portable_digest(rows)
    print(names)
    print(bundle.valid, len(portable_digest(rows)))
```

Observed output:

```
('MANIFEST.sha256', 'squares.json', 'squares.md', 'squares.tex')
True 64
```

70.4 Installed wheel verification

A wheel recipe begins in a fresh environment, installs the bundled wheel and the declared dependency profile, and runs identity plus stable-contract checks:

```
python -m pip install .\komorebi_math-1.0.0b4-py3-none-any.whl
python -m komorebi_math info
python -m komorebi_math verify stable
python -m pip check
```

For the recorded reference baseline, `verify stable` ends with `verification passed 6/6 checks`. This establishes the installed stable surface under that interpreter. It is not a substitute for source tests, strict documentation, platform evidence, or optional GPU qualification.

70.5 Source gate and clean handoff replay

The complete publication path starts from the source tree, runs the source gate, builds the wheel and source distribution, and creates a deterministic handoff archive. The handoff verifier then runs once in staging and again from a new extraction. A release recipe should record at least:

- Source and package version.
- Interpreter, platform, dependency profile, and optional backends.
- Test, documentation, and publication-gate results.
- Artifact paths, byte counts, and SHA-256 digests.
- Archive member count, path-safety audit, and CRC result.
- Deterministic rebuild comparison.
- Clean-extraction verifier result.

The cumulative archive workflow recorded in the reference baseline follows that stronger pattern: it writes the ZIP twice and requires byte equality, audits every member, extracts without trusting member paths, and replays the bundle manifest from the clean root. Such a result applies to the exact archive identified by its record. For the present manual files, consult their supplied build and review records rather than transferring an earlier archive's result to new bytes. Two separately timestamped or intentionally revised editions need not have the same digest.

70.6 Recipe replay matrix

The installed finite smoke suites for Part XI are:

Connected area	Verification command	Expected final line
algebra and coordinates	`python -m komorebi_math verify coordinates`	`verification passed 24/24 checks`
arithmetic sequences	`python -m komorebi_math verify sequences`	`verification passed 5/5 checks`
generating compilers	`python -m komorebi_math verify generating`	`verification passed 14/14 checks`
signal processing	`python -m komorebi_math verify dsp`	`verification passed 11/11 checks`
nonlinear calculus	`python -m komorebi_math verify nonlinear`	`verification passed 14/14 checks`
finite dynamics	`python -m komorebi_math verify dynamics`	`verification passed 11/11 checks`
exact integration	`python -m komorebi_math verify integration`	`verification passed 12/12 checks`
typed expressions	`python -m komorebi_math verify expression`	`verification passed 22/22 checks`
quantum systems	`python -m komorebi_math verify quantum`	`verification passed 9/9 checks`
deformation systems	`python -m komorebi_math verify deformation`	`verification passed 4/4 checks`
finite Langlands models	`python -m komorebi_math verify langlands`	`verification passed 12/12 checks`
export and persistence	`python -m komorebi_math verify export`	`verification passed 5/5 checks`

These suites localize domain regressions. The source publication gate remains the canonical full source release route.

Part XII. Verification, release, and reproducibility

Choose a check that answers your question. Most readers need a focused check while learning a feature. Release maintainers need the wider collection of records described in the later chapters.

Question	Starting point	What a pass applies to
Does this installed feature pass its finite checks?	Chapter 71, named domain suite	the selected suite in the interpreter that ran it
Does the public interface match its contract?	Chapter 72	the selected API, semantic, or persistence contract
Has this package run on the required platforms?	Chapter 73	the version and environment named in each record
Can the delivered source and packages be rebuilt and replayed?	Chapter 74	the source and output bytes identified by the build records
Where is a test or verification program documented?	Appendix O	the static source and reference mapping

This part preserves the reference baseline's commands, digests, counts, and observed outcomes. They explain how the evidence is interpreted. When checking a different release, compare against that release's own contract and records. A changed count or digest is a reason to identify the source version before diagnosing a failure. It is not, by itself, proof that either run is wrong.

For an unsuccessful check, keep its exact invocation, package and interpreter identity, first failing check, and diagnostic. Repeat the smallest relevant check after correcting the cause. This makes failures easier to explain than a single large log with no version or command context.

A trustworthy release claim is a conjunction of narrower claims, not one large green check. Komorebi Math uses five layers: an installed verification suite tests finite package behavior. A frozen contract compares the live package to committed compatibility artifacts. A platform record says where a particular execution occurred. The source publication gate checks the publication-facing source tree, and the handoff builder proves that the distributed bytes can be rebuilt and replayed from a clean extraction. Passing an earlier layer never implies a later one.

In symbols, the strongest local handoff claim has the shape

$$Q_{handoff} = V_{installed} \wedge C_{frozen} \wedge E_{platform} \wedge G_{publication} \wedge A_{replay}.$$

External publication is intentionally outside this expression. A qualified wheel or ZIP is not evidence that a PyPI upload, GitHub release, or authenticated tag exists.

71. Verification suites

71.1 Domain suites

The command-line registry contains exactly 51 accepted names. Each name maps to one callable returning a `VerificationReport`. The final `all` name is retained for compatibility and maps to the historical compatibility runner. The registry divides as follows:

Family	Exact accepted names
finite domain and focused suites	`core`, `refinement`, `nonlinear`, `algorithms`, `signals`, `dynamics`, `analytic`, `quantum`, `arithmetic`, `sequences`, `deformation`, `langlands-local`, `langlands-finite-image`, `langlands-completion`, `langlands`, `export`, `generating`, `coordinates`, `frame-algebra`, `integration`, `expression`, `dsp`, `hardening`, `streaming-beta`
cumulative package suites	`block2`, `block3`, `block4`, `block5`, `block6`, `block7`, `block8`, `block9`, `block10`, `block11`, `block12`, `block13`, `block14`, `block15`, `block16`, `block17`, `block18`, `block19`, `block20`
compatibility and release suites	`stable`, `release`, `gpu`, `upgrade`, `evidence`, `documentation`, `beta`
historical compatibility alias	`all`

A verification check is the immutable triple

$$o = (\text{name}, \text{passed}, \text{detail}),$$

and a `VerificationReport` is a tuple of those checks. Its `passed` property is the conjunction of every check, `passed_count` and `total_count` expose the finite cardinalities, and `certificate()` converts the report to the common certificate vocabulary. The certificate is numerical because many suites mix exact identities with finite floating comparisons. Individual checks remain responsible for their own stronger evidence.

71.2 Cumulative suites

Choose the smallest suite that localizes the question. A domain suite is useful while developing one subsystem. A cumulative `blockN` route checks the graph installed by that development block. Contract routes answer different questions: `stable` combines API, semantic, and persistence checks. `release` checks the beta release contract. `upgrade` replays predecessor relations. `evidence` checks committed platform records. `documentation` checks stable symbol coverage, and `beta` aggregates public-beta readiness. `gpu` is a hardware-facing route and must not be treated as a required base-wheel check.

`komorebi verify all` is a historical compatibility alias. It does not run the complete cumulative verification, source publication gate, or full pytest suite. For the reference installed cumulative graph, use `python -m komorebi_math verify block20`. For the complete source release path, use the source publication gate in full mode.

71.3 Reading verification output

The CLI prints one normalized line per check followed by the aggregate certificate message:

```
[PASS] stable public API manifest
[PASS] stable semantic fixture
...
verification passed 6/6 checks
```

The marker reflects only `check.passed`. A nonempty detail is appended after two spaces and is especially important on failure. The process returns 0 only when the report passes, 1 for a known suite with at least one failed check, and 2 for an unknown suite name. This makes the route safe for shell gates without requiring log scraping.

The recorded qualification environment produced these focused results:

Command suffix after `verify`	Observed result
`stable`	6/6 passed
`release`	10/10 passed
`upgrade`	1/1 passed
`evidence`	1/1 passed
`documentation`	1/1 passed
`beta`	1/1 passed
`block20`	46/46 passed

This installed-Python check uses the report object directly and therefore fails at the same certificate boundary as the CLI:

```python
from komorebi_math.verify import run_stable_verification

report = run_stable_verification()
report.certificate().require()
assert report.passed_count == report.total_count == 6
print(f"stable verification: {report.passed_count}/{report.total_count}")
```

Expected output:

```
stable verification: 6/6
```

71.4 Failure localization

`_capture()` converts an exception raised by one check into a failed `VerificationCheck` whose detail begins with the exception type. This localizes assertion failures, validation errors, stale digests, environment mismatches, and resource refusals without losing the rest of the suite. Native crashes and timeouts require an isolated launcher. They cannot be converted by an in-process `try` block after the interpreter has stopped.

Optional-backend absence is not globally a pass or a failure. A suite may record it as non-gating when the backend was not requested. Once the command or release claim requires that capability, `--require-gpu`, for example, the same absence is a failed requirement. Always read the named check, its detail, the aggregate return code, and the invocation flags together.

> **Chapter 71 boundary.** A passing suite proves the finite checks selected by that suite under the current interpreter. It does not freeze an API, prove a predecessor upgrade, establish another platform, run the full source tests, or qualify an artifact archive.

72. API, semantic, persistence, and upgrade contracts

72.1 Public API manifests

The API checker snapshots exported names, broad object kinds, implementation owners, qualified names, and reflected signatures. It has three intentionally different scopes:

Tier	Namespaces	Entries	Canonical manifest digest
stable	`komorebi_math.stable`	131	`23cd9e47a1663f2d 4b3c0fef50f9f7d8 c88d4af2f8bbc57e 7ba76461e6e69276`
beta	`komorebi_math.beta`	83	`311f2bf7cbcf0a45 bcbe2af433d73a8f 597c0fbf71337508 82314d6e0edb16b2`
research	six broad public namespaces	1,312	`473c73e1a0563fe0 b2569c8be384f7c9 f947570bea2f300e f611db351136847a`

These are canonical object digests calculated from normalized JSON. The release contract separately stores the SHA-256 of each committed JSON file. That file digest also covers whitespace and serialization. Do not compare one kind of digest to the other and call the mismatch a failure.

Stable aliases are stronger than equal spellings. For every stable export `name`, the three supported routes must resolve to the same Python object:

$$\mathrm{id}(stable.name) = \mathrm{id}(v1.name) = \mathrm{id}(api.v1.name).$$

The release and readiness gates check both the identical `__all__` sequences and object identity for every one of the 131 names.

72.2 Closed stable semantic fixture

A structural manifest cannot detect a function that keeps its signature while changing its mathematics. The closed stable fixture therefore executes ten committed cases spanning analysis/synthesis, signal and arithmetic polyphase, lattice refinement, operator grammar, basis coordinates, normalized series, document rendering, settings scope, and portable refinement.

Each result is canonicalized and compared with the committed expected value. The fixture digest is `02d05d0cd59e2e9d 3cd5a72dbeab7223319cf8cbb25b96e738d94d5c9cedd870`. The registry is closed: an unknown case kind is an error, not an ignored extension. The report proves those ten laws only. It is not an exhaustive semantic specification of the research namespace.

72.3 Portable codec contract

The persistence contract freezes the package-owned portable registry at 51 codec tags. For each tag it records the positive codec version and fully qualified implementation type. Comparison rejects a removed tag, a changed implementation type, a version decrease, or a version increase lacking every adjacent migration step.

The live registry contains one adjacent migration: `nonlinear.packet_jet_snapshot` from version 1 to version 2. Additive extension codecs are not silently promoted into the package-owned stable contract. The aggregate stable predicate is therefore

$$C_{stable} = C_{API} \wedge C_{semantics} \wedge C_{persistence}.$$

In the audited tree all three components pass under theorem record `KM-STABLE-CONTRACT-016`.

72.4 Release contract

The current release contract binds package identity, runtime policy, required platforms, stable aliases, CLI routes, and eight committed artifacts. Its artifact rows have the exact shape `(filename, sha256, count)`:

Artifact	Count	Committed file SHA-256
`public_api_stable_1_0.json`	131	`74e94269f875943861cb815f9acdb5eb0289d9a0fab8e8b55ff59e7e15483e67`
`public_api_beta_1_0.json`	83	`c0f4cf27d4a47b86d1ea5b79c6c2fe883368e6ab7f8a640e527f916b271ba000`
`public_api_1_0.json`	1,312	`04a6c1218bb016b2bbd09f551a7866d1aab47efd5efb753710dde322f32f6357`
`stable_semantics_1_0.json`	10	`445668c79fce5eccf4fd3d5f8aebf66a340afff7d58983ec7c48c06201f7aa6e`
`portable_codec_contract_1_0.json`	51	`5f8522edf0dd56c6d3a23be4625e72a33fe0ee7182c13060a9b09f93035a7c05`
`upgrade_contract_1_0.json`	7	`d80de9c2b6913df3386a776519fefffb47b2820a57678b53ed19afae87f3e84d`
`validation_evidence_1_0.json`	2	`7aea0d877dbd1e896567199b9f6a857aa53a0bf40189c6aa17a05827115deaf2`
`stable_documentation_1_0.json`	131	`9ffb806d95c979b77f59334265cc5e045dae949cd25c46a5bd37d2b7e0ca8299`

The verifier first checks file existence, bytes, and the expected array count. Then it re-runs live API, semantics, persistence, upgrade, evidence, and documentation checks. It also verifies alias identity, required CLI routes, and the support matrix. The release-contract digest in this tree is `3f46ed8c65861a043f677dad32c2f84b3e7189f9c122a8489f94ae47ebc8bfd3`.

72.5 Predecessor upgrade records

The accepted predecessor set is

$$\{0.15.0a2, 0.16.0a2, 1.0.0b1, 1.0.0b2, 1.0.0b3\}.$$

The contract contains five stable-API baselines, one `0.16.0a2` stable semantic baseline, one `0.16.0a2` 51-codec baseline, and seven digest-bound portable fixtures. It does not claim one separate semantic or codec snapshot for every predecessor. The `0.15.0a2` API comparison has one documented migration allowance for `create_stable_api_manifest`. The later stable baselines require exact name, kind, and signature continuity.

Every fixture must retain its archive digest and portable-payload digest, load through the current codec graph, produce its expected qualified Python type, and round-trip canonically. The aggregate upgrade-contract digest is `f70d9327fafb6fa743e2934ec0144059219c4fe0f490083c50ef80d19c539bbe`.

This executable check reads the three live manifests and then requires the aggregate stable, release, and upgrade certificates:

```
from komorebi_math.compatibility import (
    create_beta_api_manifest,
    create_public_api_manifest,
    create_stable_api_manifest,
)
from komorebi_math.release import check_release_contract, check_upgrade_contract
from komorebi_math.stable_contract import check_stable_contract

manifests = {
    "stable": create_stable_api_manifest(),
    "beta": create_beta_api_manifest(),
    "research": create_public_api_manifest(),
}
assert {name: len(value.symbols) for name, value in manifests.items()} == {
    "stable": 131,
    "beta": 83,
    "research": 1312,
}
for report in (
    check_stable_contract(),
    check_release_contract(),
    check_upgrade_contract(),
):
    report.certificate.require()
print("stable, release, and upgrade contracts passed")
```

Expected output:

```
stable, release, and upgrade contracts passed
```

> **Chapter 72 boundary.** Compatibility is a versioned relation between the current package and exact committed artifacts. It does not prove performance stability, research-tier stability, platform execution, or artifact reproducibility.

73. Platform and release qualification

73.1 Environment records

`release environment` is descriptive evidence about the interpreter that ran the command. It records the package version, Python executable and version, platform string, installed packages, and isolated optional-backend probes. It does not by itself say that a verification suite passed. Keeping environment and result records separate prevents an installed package list from being misread as executed qualification.

Three predicates must therefore remain distinct:

$$\text{DeclaredSupported}(E) \not\equiv \text{ExecutedSuccessfully}(E, R) \not\equiv \text{HardwareQualified}(E, H).$$

The support matrix declares Python 3.11, 3.12, and 3.13 with NumPy 1.26 or newer. That declaration is policy, not proof that every combination has run.

73.2 Linux and Windows evidence

One validation-evidence record contains an identifier, normalized platform key, raw platform string, package, Python and NumPy versions, a source record and its SHA-256, an inheritance flag, finite results, and optional-backend details. The checker verifies the source file digest separately from version currentness.

The committed evidence set contains two records:

Platform key	Record	Package version	Status in `1.0.0b4` policy
`linux-x86_64`	`linux-1.0.0b4-publication-gate`	`1.0.0b4`	current
`windows-x86_64`	`windows-0.16.0a2-user-qualification`	`0.16.0a2`	inherited through the upgrade contract

Both required platform families are covered, at least one current execution exists, and the inherited predecessor is authorized, so the beta evidence policy passes. Only Linux is current. Consequently,

$$\text{current} = \{\text{linux–!x86_64}\}, \qquad \text{missing_current} = \{\text{windows–!x86_64}\}.$$

The evidence object's canonical digest is `cc7ea963e6799320104b84e2d49231b933ee5a3e379909173fa4df349562474e`.

73.3 CPU and optional CUDA qualification

The base wheel requires NumPy only. JAX, PyTorch, CuPy, SciPy, SymPy, and the documentation toolchain are optional profiles. Backend discovery and hardware qualification are also different events: package metadata can find a module that fails to import, imports can work without the required primitive, and a CUDA library can import without executing the intended GPU kernel.

The GPU qualifier therefore isolates PyTorch and CuPy work in child interpreters and records native import failures, timeouts, abnormal exits, device identity, and numerical residuals. With no GPU flag, beta readiness records that current-host CUDA was not requested. `--gpu` observes hardware without making it mandatory. `--require-gpu` turns a missing or failing GPU record into a gate failure. None of these routes changes the NumPy-only base dependency contract.

73.4 Beta versus release-candidate readiness

Beta readiness permits a required platform to inherit a successful predecessor record when the upgrade contract protects the stable API, semantics, persistence codecs, and portable fixtures. Final release-candidate readiness does not. It sets `require_current_all_platforms=True`, so every declared platform family must supply a current-package record.

Command	Audited exit	Meaning
`python -m komorebi_math release evidence`	0	beta evidence passed. Windows is inherited
`python -m komorebi_math release readiness`	0	public-beta readiness passed
`python -m komorebi_math release evidence --current`	1	current Windows evidence is missing
`python -m komorebi_math release readiness --final`	1	release-candidate readiness is not established

The nonzero final result is expected evidence, not a broken command. This short program makes that asymmetry executable:

```
from komorebi_math.release import check_beta_readiness, check_validation_evidence

beta = check_validation_evidence()
current = check_validation_evidence(require_current_all_platforms=True)
readiness = check_beta_readiness()
final = check_beta_readiness(require_current_platforms=True)

assert beta.passed and readiness.passed
assert not current.passed and not final.passed
assert current.missing_current_platforms == ("windows-x86_64",)
print("beta ready; final missing:", *current.missing_current_platforms)
```

Expected output:

```
beta ready; final missing: windows-x86_64
```

73.5 Externally observed Windows qualification transcript

1148 passed, 4 optional backend skips, 0 failed,

with strict MkDocs and report digest `4aed4f725e7592d0783ede586b7940dc80374fbe67123bf1acd507520c26e2e3`.

This successful transcript was supplied from an external Windows replay, but its JSON record is not committed in this handoff. A provenance filename cited by an earlier manual scaffold, `BLOCK21_WINDOWS_HOTFIX1_PROVENANCE.json`, is also absent from the delivered tree and therefore cannot carry evidence here. The committed validation contract continues to inherit Windows evidence from `0.16.0a2`.

Before describing Windows as current, import the actual transcript JSON, validate its internal self-digest and named checks, bind its file digest into `validation_evidence_1_0.json`, and regenerate the release-contract artifact and dependent handoff records. The separate committed Linux gate has report digest `9ece0eef69115b4519cfcc266eacb16d0881bfb3d3bc4b3b4d5c36e900ef46d5`.

> **Chapter 73 boundary.** The recorded reference package satisfied its committed public-beta policy. Those records do not establish final all-current platform readiness or current Windows qualification for a different package version.

74. Source publication handoff

74.1 Public capability tour

`examples/block21_public_capability_tour.py` is the first deterministic public program for a release reviewer. It stays finite and inexpensive while crossing stable and research-tier features. Its record includes:

- Polyphase refinement from order 4 to order 12 with a 29-sample round trip.
- A redundant two-dimensional analysis/synthesis system with four coordinates and a portable round trip.
- Eighteen Euler-Fibonacci coordinates with verified reconstruction.
- The exact finite integral `106/5` and an adversarial false-certificate guard.
- A typed-expression result `5/4`, identity pipeline, portable normal-form round trip, and matrix-chain costs 244 versus 260.
- The 131-symbol stable manifest and its canonical digest.

Canonical JSON over that record, excluding the digest field itself, produced `0e20e32f4ad9004d5248d258813b6acfec7b9e96cc30c001aa6865aba9039fdf` in the recorded audit. The digest makes altered output detectable. It is not a digital signature or proof of who executed the tour.

74.2 Full publication gate

The gate's base mode contains 14 checks: version/citation identity, required repository layout, archive-safe source paths, strict research, beta, and stable API contracts, stable and cumulative verification, release and upgrade contracts, beta evidence and readiness, the capability tour, and local Markdown links. Full mode adds the complete source test suite. MkDocs is a check when required, an automatic check when importable, or a warning when explicitly skipped or unavailable in automatic mode.

Thus the strict handoff prerequisite is

$$G_{21}^{strict} = G_{base} \wedge G_{full\ tests} \wedge G_{MkDocs}.$$

During the recorded publication audit, an explicit quick invocation with MkDocs skipped passed all 14 base checks. Its report correctly carried two non-gating warnings: the strict documentation build was skipped and the complete source tests were not requested. That observation is smoke evidence, not a new full-gate record.

POSIX source review uses:

```
PYTHON=python ./run_block21_publication_gate.sh
```

The POSIX wrapper always requests full tests. It requires MkDocs when the module is installed and otherwise records an explicit skip, so install MkDocs before using the result as the strict handoff prerequisite.

74.3 Strict MkDocs build

The strict site build checks navigation, local documentation structure, and MkDocs warnings:

```
python -m mkdocs build --strict
```

The site uses `mkdocs.yml` and the `docs/` corpus. The technical manual has a separate Markdown and LaTeX production workflow. The reference-baseline PDF build used Pandoc and `latexmk`; the editable manuscript supplied with this edition includes its own build instructions. Use those instructions when rebuilding these files. A passing site can still have a broken LaTeX page, and a clean PDF can still coexist with a broken site link. Publication evidence must name both workflows separately.

74.4 Reproducible wheel and source distribution

The distribution builder constructs the wheel and source distribution twice under the recorded normalization environment and requires exact byte equality between the first and second build. It then audits wheel `RECORD`, archive paths, source completeness, and clean installation behavior. The shipped artifacts audited for the reference baseline are:

Artifact	SHA-256
`komorebi_math-1.0.0b4-py3-none-any.whl`	`461e0f954aaee3b6b996b0dc0a5aa66f0b15114eaddad7ed3c3767187ef911ff`
`komorebi_math-1.0.0b4.tar.gz`	`744574baca52b13f094d2c2877c42e37c7f45e2affa45fe94038793e631281dc`

Byte reproducibility is intentionally stronger than comparing extracted file sets. It detects metadata, member ordering, compression, or timestamp drift that a payload-only comparison could miss.

74.5 Deterministic repository ZIP

The repository handoff builder first consumes a passing full gate record. It walks the source tree, excludes local caches and build state, and writes a single-root ZIP with normalized timestamps and modes. Every member must be a regular file with a relative NFC path. Traversal components, backslashes, symlinks, case-fold collisions, encrypted entries, and unsafe Windows names are rejected.

The ZIP is built twice and the two SHA-256 digests must match. This proves that the same accepted source and normalization inputs produced identical archive bytes. It does not prove that an independently modified source tree will or should have the same digest.

74.6 Manifest, archive safety, and clean-extraction replay

The handoff manifest records every payload path, normalized mode, size, and SHA-256, plus the embedded wheel and source-distribution identities. After ZIP creation, the verifier audits the central directory before extraction, rejects unlisted or unsafe entries, extracts without trusting member paths, and recomputes the complete inventory.

The clean root then replays the quick publication gate. Replay acceptance requires a full-mode prerequisite in the source record, the expected named checks, a valid report self-digest, matching package version, identical manifest content, payload hashes, and encoded modes. This sequence catches a handoff that passed in its build tree but omitted a required source, example, contract, or documentation file.

74.7 Windows isolated qualification

On Windows, the canonical launcher is:

```
.\Run_Block21_Publication_Gate.ps1 -Python "$env:CONDA_PREFIX\python.exe"
```

Without `-Quick`, it runs full tests and, unless `-SkipMkDocs` is explicitly given, requires strict MkDocs. The outer cumulative-manual wrapper goes further: it verifies the manual bundle manifest, copies the project into a new temporary directory, creates a fresh environment unless told otherwise, runs focused integration and expression tests, and then calls the publication-gate launcher. Failed workspaces are retained for diagnosis. Successful workspaces are removed unless preservation was requested.

After a strict gate passes, the repository handoff is built with:

```
.\Build_Block21_Handoff.ps1 -Python "$env:CONDA_PREFIX\python.exe"
```

These commands qualify source and artifact records. They do not upload to PyPI, create a GitHub release, redistribute source contrary to its recorded license, or create an authenticated tag. Those external actions require separate authority and evidence.

> **Chapter 74 boundary.** A replayable handoff binds finite source, documentation, distributions, archive bytes, and clean extraction. It does not imply external publication or final all-platform readiness.

74A. Source reconciliation

A source review distinguishes historical reproduction from active release instructions. Package identities, command options, support matrices, verifier scope, build workflows, and license descriptions must agree with the declared source version.

A reconciliation record identifies the source finding, governing contract, accepted instruction, and retained historical context. License descriptions must match the license supplied with the declared release.

Packet routes and other coordinate routes may use different option names. Check the declared parser before carrying an option from one route to another.

A maintainer can repeat the focused static review with searches such as:

```
rg -n 'support_matrix_0_16|coordinates packet.*--size' docs examples README.md
rg -n 'verify all' docs README.md
rg -n 'open source|OSI' LICENSE.md CONTRIBUTING.md docs README.md
```

Read search results in context. A historical command may remain valid as versioned reproduction evidence while requiring a different command in active instructions. License claims require review against the applicable release.

Part XIII. Exhaustive reference appendices

Appendix A. Stable API: 131-entry frozen reference

Using the stable reference. Start here when you want an interface with the stable compatibility contract. Find the exported name, read its definition, then check the contract before copying the signature. The signature tells you how arguments are supplied; the contract tells you which values, dimensions, and assumptions make the operation meaningful. Follow the chapter links for construction steps and worked examples.

A reading key for the API cards. Appendices A, B, and C use the same fields. You do not need to memorize their letters. Read each card in the following order when using an unfamiliar object.

Card section	What to look for	Practical question
Identity: `q`, `a`, `t`, `k`	Implementation identity, supported import routes, tier, and kind	Which name should my program import, and is it a class, function, or data value?
Signature: `s`	Parameter order, annotations, defaults, and keyword-only arguments	How do I supply the inputs?
Definition and contract: `D`, `C`, `H`, `R`, `X`	Meaning, input and output types, shapes, units, hypotheses, return variants, and failure conditions	Does this operation apply to my data, and what will I receive?
Guarantees: `E`, `P`, `B`, `K`	Exactness, dependencies and platform, backend behavior, and resource costs	Which guarantees and execution requirements apply to this call?
Evidence and links: `V`, `L`	Relevant checks, certificates, chapters, examples, schemas, and tests	Where can I see the operation in context and understand its evidence?

For example, the card for `CoordinateTransform` describes an exact coordinate map, while `NumericCoordinateTransform` describes a numerical map with a tolerance. Their similar purpose does not make their scalar domains or guarantees interchangeable. The contract and guarantees fields tell you which one matches your calculation. Chapters 15 through 20 explain the construction and use of both kinds.

Reading a reflected signature. An ordinary positional or keyword parameter with no default is required. A parameter followed by a default may be omitted, in which case that default is used. Parameters after a bare `*` must be passed by name; parameters before `/` must be supplied positionally. Entries written as `*args` or `**kwargs` collect additional positional or keyword arguments. They do not by themselves require at least one additional argument. An annotation such as `tuple[...]` describes an expected type; it does not replace the shape and value requirements in the contract. Quotation marks around annotations are part of Python's reflected representation and are not quotation marks to add to your argument values. A class card can show an initializer annotation ending in `-> None`; calling the class still constructs an instance. The literal `null` means that the manifest records no callable signature. Check the kind and definition before treating a name as a function.

An alias names the same object through another route. Choose one documented route consistently in application code. A matching implementation owner alone does not establish aliasing: different data values can share a Python type. The identity section states the supported relationship explicitly.

From a returned value to a usable result. Check whether the card returns a plain value, a result wrapper, a report, or a record that contains a certificate. Read the return contract before accessing fields. Where a certificate is supplied, inspect its validity, exactness, assumptions, and diagnostics as described in Chapter 5. A stable import route describes the compatibility promise; it does not turn a numerical result into exact arithmetic or discharge a conditional assumption.

For a dimension or input failure, begin with Chapter 6 and Appendix I. For an optional backend, consult Chapter 8 and Appendix J. For saved objects, check the card's persistence statement and Appendix G before assuming that a record has a portable codec. This keeps the reference useful both when a call succeeds and when it needs correction.

Appendix A records the exported-symbol ledger for the frozen 1.x surface. It covers exactly 131 manifest names: 58 classes, 61 functions, and 12 data exports. "Exhaustive" here means every exported stable name, not every public method or property on the exported classes. Method behavior remains in the chapters linked by each card.

The canonical committed manifest has digest `23cd9e47a1663f2d4b3c0fef50f9f7d8c88d4af2f8bbc57e7ba76461e6e69276`. The platform-neutral first-line documentation inventory has digest `c0702315e6ddafdb5bfa5a4f973370dab83afba5cf356efeb68280f561cf6803`. Those are canonical-JSON digests. Raw-file SHA-256 values are separate release-artifact identities. The documentation inventory proves one summary record per export but explicitly does not prove narrative completeness. The cards below provide that separate editorial layer.

A.0 Reference contract and alias rules

Every card is keyed by the exported stable name and contains the retained row fields `q, a, t, k, s, D, C, H, R, X, E, P, B, K, V, L`. The exact reflected signature is copied verbatim from the manifest in a fenced `text` block. Literal `null` means the manifest does not claim a callable constructor signature. This is correct for all 12 data exports and the seven stable exception classes.

For each name `n`, these supported routes resolve to the same object:

```
komorebi_math.stable.n is komorebi_math.v1.n
komorebi_math.v1.n is komorebi_math.api.v1.n
```

That gives 393 name-preserving qualified routes. A root or domain re-export may also be identity-equal, but importing there does not itself declare a stable-tier dependency. Five source-declared cross-name alias pairs are retained as separate manifest cards: `CoordinateTransform/ExactCoordinateTransform` and the four verb-order refinement pairs. Data values are never deduplicated by Python `id()` or by reflected `(implementation_module, qualname)`: interned integer `1` and shared runtime types would create false aliases among unrelated constants.

Data summaries are fixed semantic strings rather than docstrings inherited from `PosixPath`, `WindowsPath`, `str`, `int`, or `types.UnionType`. The reference-source contract and its `1.0.0b4` documentation snapshots were updated together. Predecessor snapshots and earlier release evidence retain their original scope. They do not establish execution of the source correction.

The compact profile IDs are editorial inheritance only. Each card repeats the profile's complete operational and evidence text so no required field is hidden behind an external lookup. "No frozen codec" is an explicit non-support statement, and "no tighter public bound" is not replaced by a benchmark.

Section	Classes	Functions	Data	Total
A.1	0	0	11	11
A.2	12	3	0	15
A.3	12	2	0	14
A.4	8	6	0	14
A.5	7	19	1	27
A.6	1	2	0	3
A.7	3	7	0	10
A.8	6	11	0	17
A.9	9	11	0	20
Total	**58**	**61**	**12**	**131**

A.1 Constants and identity records

A.1 · DEFAULT_PORTABLE_CODEC_CONTRACT

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.DEFAULT_PORTABLE_CODEC_CONTRACT`. Name-preserving aliases `komorebi_math.v1.DEFAULT_PORTABLE_CODEC_CONTRACT` and `komorebi_math.api.v1.DEFAULT_PORTABLE_CODEC_CONTRACT`. Tier `[STABLE]`. Manifest kind `data`. Reflected implementation `pathlib.Path`. Profile `A1-I1`.

Manifest signature (s).

```
null
```

Definition. A `pathlib.Path` resolved from the installed package to `data/portable_codec_contract_1_0.json`. The install prefix is intentionally not frozen.

Contract (D, C, H, R, X). The domain is the empty argument tuple and the codomain is the recorded literal or package-relative default path. Import is the only hypothesis. The result is an immutable string, integer, tuple, path, or union-type binding. There is no call-time failure, although dereferencing a default path can raise ordinary filesystem errors.

Guarantees (E, P, B, K). These are exact identity values, not mathematical certificates. Schema constants name persistence formats but are not themselves codec payloads. They use the Python standard library and have constant-time binding lookup. File I/O is outside that bound.

Evidence and links (V, L). The stable API and reference-card parsers verify name, kind, and reflected type owner. The platform-neutral documentation inventory verifies the semantic summary. See Chapters 0, 61, and 72 and the corresponding contract loader.

A.1 · DEFAULT_STABLE_SEMANTICS_FIXTURE

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.DEFAULT_STABLE_SEMANTICS_FIXTURE`. Name-preserving aliases `komorebi_math.v1.DEFAULT_STABLE_SEMANTICS_FIXTURE` and `komorebi_math.api.v1.DEFAULT_STABLE_SEMANTICS_FIXTURE`. Tier `[STABLE]`. Manifest kind `data`. Reflected implementation `pathlib.Path`. Profile `A1-I1`.

Manifest signature (s).

```
null
```

Definition. A `pathlib.Path` resolved from the installed package to `data/stable_semantics_1_0.json`. The install prefix is intentionally not frozen.

Contract (D, C, H, R, X). The domain is the empty argument tuple and the codomain is the recorded literal or package-relative default path. Import is the only hypothesis. The result is an immutable string, integer, tuple, path, or union-type binding. There is no call-time failure, although dereferencing a default path can raise ordinary filesystem errors.

Guarantees (E, P, B, K). These are exact identity values, not mathematical certificates. Schema constants name persistence formats but are not themselves codec payloads. They use the Python standard library and have constant-time binding lookup. File I/O is outside that bound.

Evidence and links (V, L). The stable API and reference-card parsers verify name, kind, and reflected type owner. The platform-neutral documentation inventory verifies the semantic summary. See Chapters 0, 61, and 72 and the corresponding contract loader.

A.1 · PORTABLE_CODEC_CONTRACT_SCHEMA

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.PORTABLE_CODEC_CONTRACT_SCHEMA`. Name-preserving aliases `komorebi_math.v1.PORTABLE_CODEC_CONTRACT_SCHEMA` and `komorebi_math.api.v1.PORTABLE_CODEC_CONTRACT_SCHEMA`. Tier `[STABLE]`. Manifest kind `data`. Reflected implementation `builtins.str`. Profile `A1-I1`.

Manifest signature (`s`).

```
null
```

Definition. The string `komorebi-math.portable-codec-contract`.

Contract (`D, C, H, R, X`). The domain is the empty argument tuple and the codomain is the recorded literal or package-relative default path. Import is the only hypothesis. The result is an immutable string, integer, tuple, path, or union-type binding. There is no call-time failure, although dereferencing a default path can raise ordinary filesystem errors.

Guarantees (`E, P, B, K`). These are exact identity values, not mathematical certificates. Schema constants name persistence formats but are not themselves codec payloads. They use the Python standard library and have constant-time binding lookup. File I/O is outside that bound.

Evidence and links (`V, L`). The stable API and reference-card parsers verify name, kind, and reflected type owner. The platform-neutral documentation inventory verifies the semantic summary. See Chapters 0, 61, and 72 and the corresponding contract loader.

A.1 · `PORTABLE_CODEC_CONTRACT_SCHEMA_VERSION`

Identity (`q, a, t, k`). Canonical manifest name `komorebi_math.stable.PORTABLE_CODEC_CONTRACT_SCHEMA_VERSION`. Name-preserving aliases `komorebi_math.v1.PORTABLE_CODEC_CONTRACT_SCHEMA_VERSION` and `komorebi_math.api.v1.PORTABLE_CODEC_CONTRACT_SCHEMA_VERSION`. Tier `[STABLE]`. Manifest kind `data`. Reflected implementation `builtins.int`. Profile `A1-I1`.

Manifest signature (`s`).

```
null
```

Definition. The integer `1` for the codec-contract schema. Identity with other small integers is not an alias relation.

Contract (`D, C, H, R, X`). The domain is the empty argument tuple and the codomain is the recorded literal or package-relative default path. Import is the only hypothesis. The result is an immutable string, integer, tuple, path, or union-type binding. There is no call-time failure, although dereferencing a default path can raise ordinary filesystem errors.

Guarantees (`E, P, B, K`). These are exact identity values, not mathematical certificates. Schema constants name persistence formats but are not themselves codec payloads. They use the Python standard library and have constant-time binding lookup. File I/O is outside that bound.

Evidence and links (`V, L`). The stable API and reference-card parsers verify name, kind, and reflected type owner. The platform-neutral documentation inventory verifies the semantic summary. See Chapters 0, 61, and 72 and the corresponding contract loader.

A.1 · `PORTABLE_SCHEMA`

Identity (`q, a, t, k`). Canonical manifest name `komorebi_math.stable.PORTABLE_SCHEMA`. Name-preserving aliases `komorebi_math.v1.PORTABLE_SCHEMA` and `komorebi_math.api.v1.PORTABLE_SCHEMA`. Tier `[STABLE]`. Manifest kind `data`. Reflected implementation `builtins.str`. Profile `A1-I1`.

Manifest signature (`s`).

```
null
```

Definition. The string `komorebi-math.portable`.

Contract (`D, C, H, R, X`). The domain is the empty argument tuple and the codomain is the recorded literal or package-relative default path. Import is the only hypothesis. The result is an immutable string, integer, tuple, path, or union-type binding. There is no call-time failure, although dereferencing a default path can raise ordinary filesystem errors.

Guarantees (`E, P, B, K`). These are exact identity values, not mathematical certificates. Schema constants name persistence formats but are not themselves codec payloads. They use the Python standard library and have constant-time binding lookup. File I/O is outside that bound.

Evidence and links (V, L). The stable API and reference-card parsers verify name, kind, and reflected type owner. The platform-neutral documentation inventory verifies the semantic summary. See Chapters 0, 61, and 72 and the corresponding contract loader.

A.1 · PORTABLE_SCHEMA_VERSION

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.PORTABLE_SCHEMA_VERSION`. Name-preserving aliases `komorebi_math.v1.PORTABLE_SCHEMA_VERSION` and `komorebi_math.api.v1.PORTABLE_SCHEMA_VERSION`. Tier `[STABLE]`. Manifest kind `data`. Reflected implementation `builtins.int`. Profile `A1-I1`.

Manifest signature (s).

```
null
```

Definition. The integer `1` for portable envelopes. Identity with other small integers is not an alias relation.

Contract (D, C, H, R, X). The domain is the empty argument tuple and the codomain is the recorded literal or package-relative default path. Import is the only hypothesis. The result is an immutable string, integer, tuple, path, or union-type binding. There is no call-time failure, although dereferencing a default path can raise ordinary filesystem errors.

Guarantees (E, P, B, K). These are exact identity values, not mathematical certificates. Schema constants name persistence formats but are not themselves codec payloads. They use the Python standard library and have constant-time binding lookup. File I/O is outside that bound.

Evidence and links (V, L). The stable API and reference-card parsers verify name, kind, and reflected type owner. The platform-neutral documentation inventory verifies the semantic summary. See Chapters 0, 61, and 72 and the corresponding contract loader.

A.1 · STABLE_API_STABILITY_LEVEL

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.STABLE_API_STABILITY_LEVEL`. Name-preserving aliases `komorebi_math.v1.STABLE_API_STABILITY_LEVEL` and `komorebi_math.api.v1.STABLE_API_STABILITY_LEVEL`. Tier `[STABLE]`. Manifest kind `data`. Reflected implementation `builtins.str`. Profile `A1-I1`.

Manifest signature (s).

```
null
```

Definition. The string `1.0-frozen-strict-signatures-semantics-and-persistence`.

Contract (D, C, H, R, X). The domain is the empty argument tuple and the codomain is the recorded literal or package-relative default path. Import is the only hypothesis. The result is an immutable string, integer, tuple, path, or union-type binding. There is no call-time failure, although dereferencing a default path can raise ordinary filesystem errors.

Guarantees (E, P, B, K). These are exact identity values, not mathematical certificates. Schema constants name persistence formats but are not themselves codec payloads. They use the Python standard library and have constant-time binding lookup. File I/O is outside that bound.

Evidence and links (V, L). The stable API and reference-card parsers verify name, kind, and reflected type owner. The platform-neutral documentation inventory verifies the semantic summary. See Chapters 0, 61, and 72 and the corresponding contract loader.

A.1 · STABLE_PUBLIC_NAMESPACES

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.STABLE_PUBLIC_NAMESPACES`. Name-preserving aliases `komorebi_math.v1.STABLE_PUBLIC_NAMESPACES` and `komorebi_math.api.v1.STABLE_PUBLIC_NAMESPACES`. Tier `[STABLE]`. Manifest kind `data`. Reflected implementation `builtins.tuple`. Profile `A1-I1`.

Manifest signature (s).

```
null
```

Definition. The tuple `('komorebi_math.stable',)` used by the canonical manifest. The two versioned facades are checked separately.

Contract (D, C, H, R, X). The domain is the empty argument tuple and the codomain is the recorded literal or package-relative default path. Import is the only hypothesis. The result is an immutable string, integer, tuple, path, or union-type binding. There is no call-time failure, although dereferencing a default path can raise ordinary filesystem errors.

Guarantees (E, P, B, K). These are exact identity values, not mathematical certificates. Schema constants name persistence formats but are not themselves codec payloads. They use the Python standard library and have constant-time binding lookup. File I/O is outside that bound.

Evidence and links (V, L). The stable API and reference-card parsers verify name, kind, and reflected type owner. The platform-neutral documentation inventory verifies the semantic summary. See Chapters 0, 61, and 72 and the corresponding contract loader.

A.1 · STABLE_SEMANTICS_SCHEMA

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.STABLE_SEMANTICS_SCHEMA`. Name-preserving aliases `komorebi_math.v1.STABLE_SEMANTICS_SCHEMA` and `komorebi_math.api.v1.STABLE_SEMANTICS_SCHEMA`. Tier `[STABLE]`. Manifest kind `data`. Reflected implementation `builtins.str`. Profile `A1-I1`.

Manifest signature (s).

```
null
```

Definition. The string `komorebi-math.stable-semantics`.

Contract (D, C, H, R, X). The domain is the empty argument tuple and the codomain is the recorded literal or package-relative default path. Import is the only hypothesis. The result is an immutable string, integer, tuple, path, or union-type binding. There is no call-time failure, although dereferencing a default path can raise ordinary filesystem errors.

Guarantees (E, P, B, K). These are exact identity values, not mathematical certificates. Schema constants name persistence formats but are not themselves codec payloads. They use the Python standard library and have constant-time binding lookup. File I/O is outside that bound.

Evidence and links (V, L). The stable API and reference-card parsers verify name, kind, and reflected type owner. The platform-neutral documentation inventory verifies the semantic summary. See Chapters 0, 61, and 72 and the corresponding contract loader.

A.1 · STABLE_SEMANTICS_SCHEMA_VERSION

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.STABLE_SEMANTICS_SCHEMA_VERSION`. Name-preserving aliases `komorebi_math.v1.STABLE_SEMANTICS_SCHEMA_VERSION` and `komorebi_math.api.v1.STABLE_SEMANTICS_SCHEMA_VERSION`. Tier `[STABLE]`. Manifest kind `data`. Reflected implementation `builtins.int`. Profile `A1-I1`.

Manifest signature (s).

```
null
```

Definition. The integer `1` for semantic fixtures. Identity with other small integers is not an alias relation.

Contract (D, C, H, R, X). The domain is the empty argument tuple and the codomain is the recorded literal or package-relative default path. Import is the only hypothesis. The result is an immutable string, integer, tuple, path, or union-type binding. There is no call-time failure, although dereferencing a default path can raise ordinary filesystem errors.

Guarantees (E, P, B, K). These are exact identity values, not mathematical certificates. Schema constants name persistence formats but are not themselves codec payloads. They use the Python standard library and have constant-time binding lookup. File I/O is outside that bound.

Evidence and links (V, L). The stable API and reference-card parsers verify name, kind, and reflected type owner. The platform-neutral documentation inventory verifies the semantic summary. See Chapters 0, 61, and 72 and the corresponding contract loader.

A.1 · `__version__`

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.__version__`. Name-preserving aliases `komorebi_math.v1.__version__` and `komorebi_math.api.v1.__version__`. Tier `[STABLE]`. Manifest kind `data`. Reflected implementation `builtins.str`. Profile `A1-I1`.

Manifest signature (s).

```
null
```

Definition. The string `1.0.0b4`.

Contract (D, C, H, R, X). The domain is the empty argument tuple and the codomain is the recorded literal or package-relative default path. Import is the only hypothesis. The result is an immutable string, integer, tuple, path, or union-type binding. There is no call-time failure, although dereferencing a default path can raise ordinary filesystem errors.

Guarantees (E, P, B, K). These are exact identity values, not mathematical certificates. Schema constants name persistence formats but are not themselves codec payloads. They use the Python standard library and have constant-time binding lookup. File I/O is outside that bound.

Evidence and links (V, L). The stable API and reference-card parsers verify name, kind, and reflected type owner. The platform-neutral documentation inventory verifies the semantic summary. See Chapters 0, 61, and 72 and the corresponding contract loader.

A.2 Certificates, settings, and exceptions

A.2 · `AssumptionError`

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.AssumptionError`. Name-preserving aliases `komorebi_math.v1.AssumptionError` and `komorebi_math.api.v1.AssumptionError`. Tier `[STABLE]`. Manifest kind `class`. Reflected implementation `komorebi_math.exceptions.AssumptionError`. Profile `A2-X1`.

Manifest signature (s).

```
null
```

Definition. Raised when a required mathematical assumption is not satisfied.

Contract (D, C, H, R, X). Each export is an exception type with inherited constructor semantics, so the frozen manifest intentionally records `null` rather than inventing a signature. The domain is an error condition and the codomain is a raised exception instance. There is no successful return. The defining predicate is stated in the summary.

Guarantees (E, P, B, K). Exactness, portability, and backend execution are not applicable to exception classes. They use the base runtime. Construction cost is proportional only to supplied exception arguments and traceback creation.

Evidence and links (V, L). Chapter 6 maps each stable exception to producers and recovery actions. Structural API and documentation checks verify the type identity and summary. Domain tests verify the operations that raise them.

A.2 · `BackendUnavailableError`

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.BackendUnavailableError`. Name-preserving aliases `komorebi_math.v1.BackendUnavailableError` and `komorebi_math.api.v1.BackendUnavailableError`. Tier `[STABLE]`. Manifest kind `class`. Reflected implementation `komorebi_math.exceptions.BackendUnavailableError`. Profile `A2-X1`.

Manifest signature (s).

```
null
```

Definition. Raised when an optional computation backend is not installed.

Contract (D, C, H, R, X). Each export is an exception type with inherited constructor semantics, so the frozen manifest intentionally records `null` rather than inventing a signature. The domain is an error condition and the codomain is a raised exception instance. There is no successful return. The defining predicate is stated in the summary.

Guarantees (E, P, B, K). Exactness, portability, and backend execution are not applicable to exception classes. They use the base runtime. Construction cost is proportional only to supplied exception arguments and traceback creation.

Evidence and links (V, L). Chapter 6 maps each stable exception to producers and recovery actions. Structural API and documentation checks verify the type identity and summary. Domain tests verify the operations that raise them.

A.2 · `Certificate`

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.Certificate`. Name-preserving aliases `komorebi_math.v1.Certificate` and `komorebi_math.api.v1.Certificate`. Tier `[STABLE]`. Manifest kind `class`. Reflected implementation `komorebi_math.certificates.Certificate`. Profile `A2-C1`.

Manifest signature (s).

```
(valid: 'bool', exactness: 'Exactness', residuals: 'dict[str, float]' = <factory>, tolerance: 'float | None' =
    ↪ None, assumptions_checked: 'tuple[str, ...]' = (), assumptions_unchecked: 'tuple[str, ...]' = (),
    ↪ diagnostics: 'dict[str, Any]' = <factory>, theorem_id: 'str | None' = None, message: 'str' = '') -> None
```

Definition. Structured validation record.

Contract (D, C, H, R, X). Constructors map typed evidence fields to immutable evidence or result records. `Exactness` maps one of five string values to an enum member. `ConditionalResult` requires a conditional certificate, and `require()` rejects an invalid certificate. Results are records, wrapped values, or enum members. Malformed conditional records raise `ValueError`, while failed certificate requirements raise `ValidationError`.

Guarantees (E, P, B, K). Exactness is recorded, not inferred from arbitrary payloads. `Certificate` alone has the stable `core.certificate` codec. The two generic wrappers have no separate frozen codec. The base path is standard-library-only and storage is linear in recorded residuals, assumptions, and diagnostics.

Evidence and links (V, L). Stable structural verification covers every export. Chapter 5 supplies executable valid, invalid, exact, numerical, and conditional examples. Related symbols are `ValidationError`, contract reports, and certificate-producing domain operations.

A.2 · `ComputationResult`

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.ComputationResult`. Name-preserving aliases `komorebi_math.v1.ComputationResult` and `komorebi_math.api.v1.ComputationResult`. Tier `[STABLE]`. Manifest kind `class`. Reflected implementation `komorebi_math.certificates.ComputationResult`. Profile `A2-C1`.

Manifest signature (s).

```
(value: 'T', certificate: 'Certificate') -> None
```

Definition. A value paired with its execution certificate.

Contract (D, C, H, R, X). Constructors map typed evidence fields to immutable evidence or result records. `Exactness` maps one of five string values to an enum member. `ConditionalResult` requires a conditional certificate, and `require()` rejects an invalid certificate. Results are records, wrapped values, or enum members. Malformed conditional records raise `ValueError`, while failed certificate requirements raise `ValidationError`.

Guarantees (E, P, B, K). Exactness is recorded, not inferred from arbitrary payloads. `Certificate` alone has the stable `core.certificate` codec. The two generic wrappers have no separate frozen codec. The base path is standard-library-only and storage is linear in recorded residuals, assumptions, and diagnostics.

Evidence and links (V, L). Stable structural verification covers every export. Chapter 5 supplies executable valid, invalid, exact, numerical, and conditional examples. Related symbols are `ValidationError`, contract reports, and certificate-producing domain operations.

A.2 · `ConditionalResult`

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.ConditionalResult`. Name-preserving aliases `komorebi_math.v1.ConditionalResult` and `komorebi_math.api.v1.ConditionalResult`. Tier `[STABLE]`. Manifest kind `class`. Reflected implementation `komorebi_math.certificates.ConditionalResult`. Profile `A2-C1`.

Manifest signature (s).

```
(value: 'T', certificate: 'Certificate') -> None
```

Definition. A value whose conclusion depends on explicit unchecked assumptions.

Contract (D, C, H, R, X). Constructors map typed evidence fields to immutable evidence or result records. `Exactness` maps one of five string values to an enum member. `ConditionalResult` requires a conditional certificate, and `require()` rejects an invalid certificate. Results are records, wrapped values, or enum members. Malformed conditional records raise `ValueError`, while failed certificate requirements raise `ValidationError`.

Guarantees (E, P, B, K). Exactness is recorded, not inferred from arbitrary payloads. `Certificate` alone has the stable `core.certificate` codec. The two generic wrappers have no separate frozen codec. The base path is standard-library-only and storage is linear in recorded residuals, assumptions, and diagnostics.

Evidence and links (V, L). Stable structural verification covers every export. Chapter 5 supplies executable valid, invalid, exact, numerical, and conditional examples. Related symbols are `ValidationError`, contract reports, and certificate-producing domain operations.

A.2 · `DimensionError`

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.DimensionError`. Name-preserving aliases `komorebi_math.v1.DimensionError` and `komorebi_math.api.v1.DimensionError`. Tier `[STABLE]`. Manifest kind `class`. Reflected implementation `komorebi_math.exceptions.DimensionError`. Profile `A2-X1`.

Manifest signature (s).

```
null
```

Definition. Raised when matrix, vector, or packet dimensions do not agree.

Contract (D, C, H, R, X). Each export is an exception type with inherited constructor semantics, so the frozen manifest intentionally records `null` rather than inventing a signature. The domain is an error condition and the codomain is a raised exception instance. There is no successful return. The defining predicate is stated in the summary.

Guarantees (E, P, B, K). Exactness, portability, and backend execution are not applicable to exception classes. They use the base runtime. Construction cost is proportional only to supplied exception arguments and traceback creation.

Evidence and links (V, L). Chapter 6 maps each stable exception to producers and recovery actions. Structural API and documentation checks verify the type identity and summary. Domain tests verify the operations that raise them.

A.2 · `Exactness`

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.Exactness`. Name-preserving aliases `komorebi_math.v1.Exactness` and `komorebi_math.api.v1.Exactness`. Tier `[STABLE]`. Manifest kind `class`. Reflected implementation `komorebi_math.certificates.Exactness`. Profile `A2-C1`.

Manifest signature (`s`).

```
(*values)
```

Definition. An enum whose members are strings.

Contract (`D, C, H, R, X`). Constructors map typed evidence fields to immutable evidence or result records. `Exactness` maps one of five string values to an enum member. `ConditionalResult` requires a conditional certificate, and `require()` rejects an invalid certificate. Results are records, wrapped values, or enum members. Malformed conditional records raise `ValueError`, while failed certificate requirements raise `ValidationError`.

Guarantees (`E, P, B, K`). Exactness is recorded, not inferred from arbitrary payloads. `Certificate` alone has the stable `core.certificate` codec. The two generic wrappers have no separate frozen codec. The base path is standard-library-only and storage is linear in recorded residuals, assumptions, and diagnostics.

Evidence and links (`V, L`). Stable structural verification covers every export. Chapter 5 supplies executable valid, invalid, exact, numerical, and conditional examples. Related symbols are `ValidationError`, contract reports, and certificate-producing domain operations.

A.2 · `ExactnessError`

Identity (`q, a, t, k`). Canonical manifest name `komorebi_math.stable.ExactnessError`. Name-preserving aliases `komorebi_math.v1.ExactnessError` and `komorebi_math.api.v1.ExactnessError`. Tier `[STABLE]`. Manifest kind `class`. Reflected implementation `komorebi_math.exceptions.ExactnessError`. Profile `A2-X1`.

Manifest signature (`s`).

```
null
```

Definition. Raised when an exact operation is requested from an inexact backend.

Contract (`D, C, H, R, X`). Each export is an exception type with inherited constructor semantics, so the frozen manifest intentionally records `null` rather than inventing a signature. The domain is an error condition and the codomain is a raised exception instance. There is no successful return. The defining predicate is stated in the summary.

Guarantees (`E, P, B, K`). Exactness, portability, and backend execution are not applicable to exception classes. They use the base runtime. Construction cost is proportional only to supplied exception arguments and traceback creation.

Evidence and links (`V, L`). Chapter 6 maps each stable exception to producers and recovery actions. Structural API and documentation checks verify the type identity and summary. Domain tests verify the operations that raise them.

A.2 · `KomorebiMathError`

Identity (`q, a, t, k`). Canonical manifest name `komorebi_math.stable.KomorebiMathError`. Name-preserving aliases `komorebi_math.v1.KomorebiMathError` and `komorebi_math.api.v1.KomorebiMathError`. Tier `[STABLE]`. Manifest kind `class`. Reflected implementation `komorebi_math.exceptions.KomorebiMathError`. Profile `A2-X1`.

Manifest signature (`s`).

```
null
```

Definition. Base class for all public package exceptions.

Contract (`D, C, H, R, X`). Each export is an exception type with inherited constructor semantics, so the frozen manifest intentionally records `null` rather than inventing a signature. The domain is an error condition and the codomain is a raised exception instance. There is no successful return. The defining predicate is stated in the summary.

Guarantees (`E, P, B, K`). Exactness, portability, and backend execution are not applicable to exception classes. They use the base runtime. Construction cost is proportional only to supplied exception arguments and traceback creation.

Evidence and links (`V, L`). Chapter 6 maps each stable exception to producers and recovery actions. Structural API and documentation checks verify the type identity and summary. Domain tests verify the operations that raise them.

A.2 · ResourceLimitError

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.ResourceLimitError`. Name-preserving aliases `komorebi_math.v1.ResourceLimitError` and `komorebi_math.api.v1.ResourceLimitError`. Tier `[STABLE]`. Manifest kind `class`. Reflected implementation `komorebi_math.exceptions.ResourceLimitError`. Profile `A2-X1`.

Manifest signature (s).

```
null
```

Definition. Raised before a computation exceeds an explicit resource contract.

Contract (D, C, H, R, X). Each export is an exception type with inherited constructor semantics, so the frozen manifest intentionally records `null` rather than inventing a signature. The domain is an error condition and the codomain is a raised exception instance. There is no successful return. The defining predicate is stated in the summary.

Guarantees (E, P, B, K). Exactness, portability, and backend execution are not applicable to exception classes. They use the base runtime. Construction cost is proportional only to supplied exception arguments and traceback creation.

Evidence and links (V, L). Chapter 6 maps each stable exception to producers and recovery actions. Structural API and documentation checks verify the type identity and summary. Domain tests verify the operations that raise them.

A.2 · Settings

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.Settings`. Name-preserving aliases `komorebi_math.v1.Settings` and `komorebi_math.api.v1.Settings`. Tier `[STABLE]`. Manifest kind `class`. Reflected implementation `komorebi_math.config.Settings`. Profile `A2-S1`.

Manifest signature (s).

```
(backend: 'str' = 'numpy', tolerance: 'float' = 1e-10, validate_on_construction: 'bool' = True) -> None
```

Definition. Settings(backend: ‘str’ = ‘numpy’, tolerance: ‘float’ = 1e-10, validate_on_construction: ‘bool’ = True)

Contract (D, C, H, R, X). A context-local `Settings` value is read, replaced, or temporarily installed. Keyword fields must match the dataclass. `configure` converts tolerance to `float`, and the context manager restores the prior value even on exit. Results are settings records or a yielding iterator. Bad replacement fields and failed conversions raise ordinary `TypeError` or `ValueError`.

Guarantees (E, P, B, K). Configuration state is exact process metadata, not a computation certificate and not a frozen portable payload. It uses `contextvars`, has no optional dependency, and each read or replacement is constant-time apart from keyword conversion.

Evidence and links (V, L). The stable semantic fixture checks context restoration. Chapters 3, 5, and 8 show configuration, backend selection, and failure boundaries. Related symbols are `Settings`, `configure`, `get_settings`, and `settings_context`.

A.2 · ValidationError

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.ValidationError`. Name-preserving aliases `komorebi_math.v1.ValidationError` and `komorebi_math.api.v1.ValidationError`. Tier `[STABLE]`. Manifest kind `class`. Reflected implementation `komorebi_math.exceptions.ValidationError`. Profile `A2-X1`.

Manifest signature (s).

```
null
```

Definition. Raised when a mathematical object fails a declared validation rule.

Contract (D, C, H, R, X). Each export is an exception type with inherited constructor semantics, so the frozen manifest intentionally records `null` rather than inventing a signature. The domain is an error condition and the codomain is a raised exception instance. There is no successful return. The defining predicate is stated in the summary.

Guarantees (E, P, B, K). Exactness, portability, and backend execution are not applicable to exception classes. They use the base runtime. Construction cost is proportional only to supplied exception arguments and traceback creation.

Evidence and links (V, L). Chapter 6 maps each stable exception to producers and recovery actions. Structural API and documentation checks verify the type identity and summary. Domain tests verify the operations that raise them.

A.2 · configure

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.configure`. Name-preserving aliases `komorebi_math.v1.configure` and `komorebi_math.api.v1.configure`. Tier `[STABLE]`. Manifest kind `function`. Reflected implementation `komorebi_math.config.configure`. Profile `A2-S1`.

Manifest signature (s).

```
(*, backend: 'str | None' = None, tolerance: 'float | None' = None, validate_on_construction: 'bool | None' = None)
   ↪  -> 'Settings'
```

Definition. Update and return the runtime settings in the current context.

Contract (D, C, H, R, X). A context-local `Settings` value is read, replaced, or temporarily installed. Keyword fields must match the dataclass. `configure` converts tolerance to `float`, and the context manager restores the prior value even on exit. Results are settings records or a yielding iterator. Bad replacement fields and failed conversions raise ordinary `TypeError` or `ValueError`.

Guarantees (E, P, B, K). Configuration state is exact process metadata, not a computation certificate and not a frozen portable payload. It uses `contextvars`, has no optional dependency, and each read or replacement is constant-time apart from keyword conversion.

Evidence and links (V, L). The stable semantic fixture checks context restoration. Chapters 3, 5, and 8 show configuration, backend selection, and failure boundaries. Related symbols are `Settings`, `configure`, `get_settings`, and `settings_context`.

A.2 · get_settings

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.get_settings`. Name-preserving aliases `komorebi_math.v1.get_settings` and `komorebi_math.api.v1.get_settings`. Tier `[STABLE]`. Manifest kind `function`. Reflected implementation `komorebi_math.config.get_settings`. Profile `A2-S1`.

Manifest signature (s).

```
() -> 'Settings'
```

Definition. Return the active runtime settings for the current context.

Contract (D, C, H, R, X). A context-local `Settings` value is read, replaced, or temporarily installed. Keyword fields must match the dataclass. `configure` converts tolerance to `float`, and the context manager restores the prior value even on exit. Results are settings records or a yielding iterator. Bad replacement fields and failed conversions raise ordinary `TypeError` or `ValueError`.

Guarantees (E, P, B, K). Configuration state is exact process metadata, not a computation certificate and not a frozen portable payload. It uses `contextvars`, has no optional dependency, and each read or replacement is constant-time apart from keyword conversion.

Evidence and links (V, L). The stable semantic fixture checks context restoration. Chapters 3, 5, and 8 show configuration, backend selection, and failure boundaries. Related symbols are `Settings`, `configure`, `get_settings`, and `settings_context`.

A.2 · settings_context

Identity (q, a, t, k). Canonical manifest name komorebi_math.stable.settings_context. Name-preserving aliases komorebi_math.v1.settings_context and komorebi_math.api.v1.settings_context. Tier [STABLE]. Manifest kind function. Reflected implementation komorebi_math.config.settings_context. Profile A2-S1.

Manifest signature (s).

```
(**changes: 'object') -> 'Iterator[Settings]'
```

Definition. Temporarily apply runtime setting changes inside a context manager.

Contract (D, C, H, R, X). A context-local Settings value is read, replaced, or temporarily installed. Keyword fields must match the dataclass. configure converts tolerance to float, and the context manager restores the prior value even on exit. Results are settings records or a yielding iterator. Bad replacement fields and failed conversions raise ordinary TypeError or ValueError.

Guarantees (E, P, B, K). Configuration state is exact process metadata, not a computation certificate and not a frozen portable payload. It uses contextvars, has no optional dependency, and each read or replacement is constant-time apart from keyword conversion.

Evidence and links (V, L). The stable semantic fixture checks context restoration. Chapters 3, 5, and 8 show configuration, backend selection, and failure boundaries. Related symbols are Settings, configure, get_settings, and settings_context.

A.3 Operator and packet foundations

A.3 · Arrow

Identity (q, a, t, k). Canonical manifest name komorebi_math.stable.Arrow. Name-preserving aliases komorebi_math.v1.Arrow and komorebi_math.api.v1.Arrow. Tier [STABLE]. Manifest kind class. Reflected implementation komorebi_math.core.grammar.Arrow. Profile A3-G1.

Manifest signature (s).

```
(name: 'str', source: 'str', target: 'str') -> None
```

Definition. Arrow(name: ‘str’, source: ‘str’, target: ‘str’)

Contract (D, C, H, R, X). Finite vertices, typed arrows, paths, and parallel linear relations map to immutable grammar records. Names, sources, targets, and composability must agree. Malformed paths or relations are rejected by validation. Results are structural records rather than executed matrices.

Guarantees (E, P, B, K). The structural layer is exact for its finite labels and coefficients, has no frozen codec on this stable surface, and uses no optional backend. Work and storage scale with the supplied vertices, arrows, path length, and relation terms. No tighter public bound is promised.

Evidence and links (V, L). The typed-operator semantic fixture and Chapters 9 through 10 exercise construction, composition, and rejection paths. Related symbols are OperatorRealization, PacketSystem, and the grammar recipes in Chapter 64.

A.3 · OperatorGrammar

Identity (q, a, t, k). Canonical manifest name komorebi_math.stable.OperatorGrammar. Name-preserving aliases komorebi_math.v1.OperatorGrammar and komorebi_math.api.v1.OperatorGrammar. Tier [STABLE]. Manifest kind class. Reflected implementation komorebi_math.core.grammar.OperatorGrammar. Profile A3-G1.

Manifest signature (s).

```
(vertices: 'Iterable[str]', arrows: 'Mapping[str, tuple[str, str]] | Iterable[Arrow]', relations:
    ↪ 'Iterable[OperatorRelation]' = ()) -> 'None'
```

Definition. A finite quiver with optional linear path relations.

Contract (D, C, H, R, X). Finite vertices, typed arrows, paths, and parallel linear relations map to immutable grammar records. Names, sources, targets, and composability must agree. Malformed paths or relations are rejected by validation. Results are structural records rather than executed matrices.

Guarantees (E, P, B, K). The structural layer is exact for its finite labels and coefficients, has no frozen codec on this stable surface, and uses no optional backend. Work and storage scale with the supplied vertices, arrows, path length, and relation terms. No tighter public bound is promised.

Evidence and links (V, L). The typed-operator semantic fixture and Chapters 9 through 10 exercise construction, composition, and rejection paths. Related symbols are `OperatorRealization`, `PacketSystem`, and the grammar recipes in Chapter 64.

A.3 · `OperatorRealization`

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.OperatorRealization`. Name-preserving aliases `komorebi_math.v1.OperatorRealization` and `komorebi_math.api.v1.OperatorRealization`. Tier `[STABLE]`. Manifest kind `class`. Reflected implementation `komorebi_math.core.realization.OperatorRealization`. Profile `A3-R1`.

Manifest signature (s).

```
(grammar: 'OperatorGrammar', spaces: 'Mapping[str, int]', operators: 'Mapping[str, Any]', *, backend: 'str |
    ↪ LinearAlgebraBackend | None' = None, validate_relations: 'bool' = True, tolerance: 'float | None' = None) ->
    ↪ 'None'
```

Definition. Assign vector space dimensions and matrices to a typed grammar.

Contract (D, C, H, R, X). A typed grammar, declared space dimensions, and arrow matrices map to an `OperatorRealization`. Evaluating declared relations maps to `RelationResidual` records. Parallel shapes and relations must agree, and optional validation may require residuals within tolerance. Results are realized matrices and named residual records. Dimension, backend, or validation failures are explicit.

Guarantees (E, P, B, K). The certificate meaning is exact on an exact backend and tolerance-qualified on a numerical backend. These records have no frozen stable codec. NumPy is the default with optional exact backends. Dense relation evaluation is bounded by the declared matrix products and relation count.

Evidence and links (V, L). The stable typed-operator semantic case and Chapters 9 through 10 verify relation realization. Related symbols are the grammar records, `Certificate`, `PacketSystem`, and `SeedSystem`.

A.3 · `OperatorRelation`

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.OperatorRelation`. Name-preserving aliases `komorebi_math.v1.OperatorRelation` and `komorebi_math.api.v1.OperatorRelation`. Tier `[STABLE]`. Manifest kind `class`. Reflected implementation `komorebi_math.core.grammar.OperatorRelation`. Profile `A3-G1`.

Manifest signature (s).

```
(name: 'str', terms: 'tuple[tuple[Any, OperatorWord], ...]') -> None
```

Definition. A linear relation among parallel operator words.

Contract (D, C, H, R, X). Finite vertices, typed arrows, paths, and parallel linear relations map to immutable grammar records. Names, sources, targets, and composability must agree. Malformed paths or relations are rejected by validation. Results are structural records rather than executed matrices.

Guarantees (E, P, B, K). The structural layer is exact for its finite labels and coefficients, has no frozen codec on this stable surface, and uses no optional backend. Work and storage scale with the supplied vertices, arrows, path length, and relation terms. No tighter public bound is promised.

Evidence and links (V, L). The typed-operator semantic fixture and Chapters 9 through 10 exercise construction, composition, and rejection paths. Related symbols are `OperatorRealization`, `PacketSystem`, and the grammar recipes in Chapter 64.

A.3 · `OperatorWord`

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.OperatorWord`. Name-preserving aliases `komorebi_math.v1.OperatorWord` and `komorebi_math.api.v1.OperatorWord`. Tier `[STABLE]`. Manifest kind `class`. Reflected implementation `komorebi_math.core.grammar.OperatorWord`. Profile `A3-G1`.

Manifest signature (s).

```
(arrows: 'tuple[str, ...]', source: 'str', target: 'str') -> None
```

Definition. A typed path written in the order in which arrows act.

Contract (D, C, H, R, X). Finite vertices, typed arrows, paths, and parallel linear relations map to immutable grammar records. Names, sources, targets, and composability must agree. Malformed paths or relations are rejected by validation. Results are structural records rather than executed matrices.

Guarantees (E, P, B, K). The structural layer is exact for its finite labels and coefficients, has no frozen codec on this stable surface, and uses no optional backend. Work and storage scale with the supplied vertices, arrows, path length, and relation terms. No tighter public bound is promised.

Evidence and links (V, L). The typed-operator semantic fixture and Chapters 9 through 10 exercise construction, composition, and rejection paths. Related symbols are `OperatorRealization`, `PacketSystem`, and the grammar recipes in Chapter 64.

A.3 · `PacketCoordinates`

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.PacketCoordinates`. Name-preserving aliases `komorebi_math.v1.PacketCoordinates` and `komorebi_math.api.v1.PacketCoordinates`. Tier `[STABLE]`. Manifest kind `class`. Reflected implementation `komorebi_math.core.packets.PacketCoordinates`. Profile `A3-P1`.

Manifest signature (s).

```
(system: "'PacketSystem'", values: 'tuple[Any, ...]') -> None
```

Definition. PacketCoordinates(system: “ ‘PacketSystem’ ”, values: ‘tuple[Any, …]’)

Contract (D, C, H, R, X). A complete family of finite projectors, packet coordinates, or a rectangular block operator maps ambient vectors and operators to packet-indexed records. Dimensions, idempotence, annihilation, completeness, names, and source/target systems are checked according to construction options. Results are packet systems, coordinate tuples, or block arrays. Invalid shapes and identities raise validation or dimension errors.

Guarantees (E, P, B, K). Exactness follows the selected backend and tolerance. No stable codec is registered for these three records. NumPy is the default backend, with exact object paths available where supported. Matrix work scales with ambient dimension and packet count.

Evidence and links (V, L). Chapters 8, 11, and 12 provide constructor and transfer examples. Stable semantic cases cover packet transfer through analysis/synthesis. Related symbols are `OperatorRealization`, `TransferPath`, and refinement records.

A.3 · `PacketOperator`

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.PacketOperator`. Name-preserving aliases `komorebi_math.v1.PacketOperator` and `komorebi_math.api.v1.PacketOperator`. Tier `[STABLE]`. Manifest kind `class`. Reflected implementation `komorebi_math.core.operators.PacketOperator`. Profile `A3-P1`.

Manifest signature (`s`).

```
(source: 'PacketSystem', target: 'PacketSystem', blocks: 'tuple[tuple[Any, ...], ...]') -> None
```

Definition. An operator represented by all target by source packet blocks.

Contract (`D, C, H, R, X`). A complete family of finite projectors, packet coordinates, or a rectangular block operator maps ambient vectors and operators to packet-indexed records. Dimensions, idempotence, annihilation, completeness, names, and source/target systems are checked according to construction options. Results are packet systems, coordinate tuples, or block arrays. Invalid shapes and identities raise validation or dimension errors.

Guarantees (`E, P, B, K`). Exactness follows the selected backend and tolerance. No stable codec is registered for these three records. NumPy is the default backend, with exact object paths available where supported. Matrix work scales with ambient dimension and packet count.

Evidence and links (`V, L`). Chapters 8, 11, and 12 provide constructor and transfer examples. Stable semantic cases cover packet transfer through analysis/synthesis. Related symbols are `OperatorRealization`, `TransferPath`, and refinement records.

A.3 · `PacketSystem`

Identity (`q, a, t, k`). Canonical manifest name `komorebi_math.stable.PacketSystem`. Name-preserving aliases `komorebi_math.v1.PacketSystem` and `komorebi_math.api.v1.PacketSystem`. Tier `[STABLE]`. Manifest kind `class`. Reflected implementation `komorebi_math.core.packets.PacketSystem`. Profile `A3-P1`.

Manifest signature (`s`).

```
(projectors: 'Sequence[Any]', *, names: 'Sequence[str] | None' = None, backend: 'str | LinearAlgebraBackend | None'
   ↪  = None, tolerance: 'float | None' = None, validate: 'bool | None' = None) -> 'None'
```

Definition. A complete finite family of pairwise annihilating idempotents.

Contract (`D, C, H, R, X`). A complete family of finite projectors, packet coordinates, or a rectangular block operator maps ambient vectors and operators to packet-indexed records. Dimensions, idempotence, annihilation, completeness, names, and source/target systems are checked according to construction options. Results are packet systems, coordinate tuples, or block arrays. Invalid shapes and identities raise validation or dimension errors.

Guarantees (`E, P, B, K`). Exactness follows the selected backend and tolerance. No stable codec is registered for these three records. NumPy is the default backend, with exact object paths available where supported. Matrix work scales with ambient dimension and packet count.

Evidence and links (`V, L`). Chapters 8, 11, and 12 provide constructor and transfer examples. Stable semantic cases cover packet transfer through analysis/synthesis. Related symbols are `OperatorRealization`, `TransferPath`, and refinement records.

A.3 · `RelationResidual`

Identity (`q, a, t, k`). Canonical manifest name `komorebi_math.stable.RelationResidual`. Name-preserving aliases `komorebi_math.v1.RelationResidual` and `komorebi_math.api.v1.RelationResidual`. Tier `[STABLE]`. Manifest kind `class`. Reflected implementation `komorebi_math.core.realization.RelationResidual`. Profile `A3-R1`.

Manifest signature (`s`).

```
(name: 'str', residual: 'float', satisfied: 'bool') -> None
```

Definition. RelationResidual(name: ‘str’, residual: ‘float’, satisfied: ‘bool’)

Contract (`D, C, H, R, X`). A typed grammar, declared space dimensions, and arrow matrices map to an `OperatorRealization`. Evaluating declared relations maps to `RelationResidual` records. Parallel shapes and relations must agree, and optional validation may require residuals within tolerance. Results are realized matrices and named residual records. Dimension, backend, or validation failures are explicit.

Guarantees (E, P, B, K). The certificate meaning is exact on an exact backend and tolerance-qualified on a numerical backend. These records have no frozen stable codec. NumPy is the default with optional exact backends. Dense relation evaluation is bounded by the declared matrix products and relation count.

Evidence and links (V, L). The stable typed-operator semantic case and Chapters 9 through 10 verify relation realization. Related symbols are the grammar records, `Certificate`, `PacketSystem`, and `SeedSystem`.

A.3 · SeedSystem

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.SeedSystem`. Name-preserving aliases `komorebi_math.v1.SeedSystem` and `komorebi_math.api.v1.SeedSystem`. Tier `[STABLE]`. Manifest kind `class`. Reflected implementation `komorebi_math.core.seeds.SeedSystem`. Profile `A3-SD`.

Manifest signature (s).

```
(operators: 'Mapping[str, Any] | Sequence[Any]', seeds: 'Sequence[Any]', *, backend: 'str | LinearAlgebraBackend |
    ↪ None' = None, tolerance: 'float | None' = None) -> 'None'
```

Definition. A finite family of operators acting on a finite seed family.

Contract (D, C, H, R, X). A finite operator family and finite seed family map to their generated action system. Operator and seed dimensions must agree and backend coercion must succeed. The result stores generated words, vectors, and relations. Invalid dimensions, relations, or unavailable backends fail explicitly.

Guarantees (E, P, B, K). Exactness follows the active backend. The record has no frozen stable codec. Time and memory grow with the finite word horizon, operator count, seed count, and ambient dimension, subject to caller-selected bounds.

Evidence and links (V, L). Chapter 10 supplies generated-action examples and Chapter 64 links them to exact algebra recipes. Stable structural verification covers the symbol. Core and backend tests cover behavior.

A.3 · TransferGraph

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.TransferGraph`. Name-preserving aliases `komorebi_math.v1.TransferGraph` and `komorebi_math.api.v1.TransferGraph`. Tier `[STABLE]`. Manifest kind `class`. Reflected implementation `komorebi_math.core.graphs.TransferGraph`. Profile `A3-GR`.

Manifest signature (s).

```
(adjacency: 'Sequence[Sequence[bool | int]]', *, names: 'Sequence[str] | None' = None) -> 'None'
```

Definition. A finite directed graph with adjacency stored as source by target.

Contract (D, C, H, R, X). A finite adjacency matrix maps to a directed transfer graph. The matrix must be square and optional names must match its size. The result is a finite graph with path and reachability operations. Invalid shapes or names raise validation errors.

Guarantees (E, P, B, K). Graph structure is exact Boolean data, is not assigned a frozen stable codec, and uses the base numerical stack only for array storage. Construction is quadratic in vertex count for a dense adjacency matrix. Later path work depends on requested length.

Evidence and links (V, L). Chapter 11 connects transfer graphs to packet blocks and path contributions. Structural stable checks cover the export. Graph and transfer tests cover semantics and failures.

A.3 · TransferPath

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.TransferPath`. Name-preserving aliases `komorebi_math.v1.TransferPath` and `komorebi_math.api.v1.TransferPath`. Tier `[STABLE]`. Manifest kind `class`. Reflected implementation `komorebi_math.core.transfer.TransferPath`. Profile `A3-T1`.

Manifest signature (s).

```
(vertices: 'tuple[int, ...]', product: 'Any') -> None
```

Definition. TransferPath(vertices: 'tuple[int, …]', product: 'Any')

Contract (D, C, H, R, X). A packet operator plus source, target, and nonnegative path length maps either to all nonzero `TransferPath` contributions or to their summed block in a certified result. Packet compatibility, index ranges, and length are required. Invalid inputs raise validation or dimension errors. Numerical reconstruction can fail its tolerance certificate.

Guarantees (E, P, B, K). Structural enumeration is exact. Summed block certification follows backend exactness and tolerance. No stable codec is registered for `TransferPath` or the returned wrapper. Cost grows with admissible paths and matrix-product dimensions and is therefore exponential in unconstrained path length in the worst case.

Evidence and links (V, L). The analysis/synthesis/transfer semantic case and Chapter 11 verify path-to-block reconstruction. Related symbols are `PacketOperator`, `TransferGraph`, `ComputationResult`, and Chapter 64 transfer recipes.

A.3 · `block_from_paths`

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.block_from_paths`. Name-preserving aliases `komorebi_math.v1.block_from_paths` and `komorebi_math.api.v1.block_from_paths`. Tier `[STABLE]`. Manifest kind `function`. Reflected implementation `komorebi_math.core.transfer.block_from_paths`. Profile `A3-T1`.

Manifest signature (s).

```
(operator: 'PacketOperator', *, source: 'int', target: 'int', length: 'int', tolerance: 'float | None' = None) ->
   ↪ 'ComputationResult[Any]'
```

Definition. Reconstruct one block of an operator power from packet paths.

Contract (D, C, H, R, X). A packet operator plus source, target, and nonnegative path length maps either to all nonzero `TransferPath` contributions or to their summed block in a certified result. Packet compatibility, index ranges, and length are required. Invalid inputs raise validation or dimension errors. Numerical reconstruction can fail its tolerance certificate.

Guarantees (E, P, B, K). Structural enumeration is exact. Summed block certification follows backend exactness and tolerance. No stable codec is registered for `TransferPath` or the returned wrapper. Cost grows with admissible paths and matrix-product dimensions and is therefore exponential in unconstrained path length in the worst case.

Evidence and links (V, L). The analysis/synthesis/transfer semantic case and Chapter 11 verify path-to-block reconstruction. Related symbols are `PacketOperator`, `TransferGraph`, `ComputationResult`, and Chapter 64 transfer recipes.

A.3 · `path_contributions`

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.path_contributions`. Name-preserving aliases `komorebi_math.v1.path_contributions` and `komorebi_math.api.v1.path_contributions`. Tier `[STABLE]`. Manifest kind `function`. Reflected implementation `komorebi_math.core.transfer.path_contributions`. Profile `A3-T1`.

Manifest signature (s).

```
(operator: 'PacketOperator', *, source: 'int', target: 'int', length: 'int', tolerance: 'float | None' = None) ->
   ↪ 'tuple[TransferPath, ...]'
```

Definition. Return every nonzero structural path contribution of fixed length.

Contract (D, C, H, R, X). A packet operator plus source, target, and nonnegative path length maps either to all nonzero `TransferPath` contributions or to their summed block in a certified result. Packet compatibility, index ranges, and length are required. Invalid inputs raise validation or dimension errors. Numerical reconstruction can fail its tolerance certificate.

Guarantees (E, P, B, K). Structural enumeration is exact. Summed block certification follows backend exactness and tolerance. No stable codec is registered for `TransferPath` or the returned wrapper. Cost grows with admissible paths and matrix-product dimensions and is therefore exponential in unconstrained path length in the worst case.

Evidence and links (V, L). The analysis/synthesis/transfer semantic case and Chapter 11 verify path-to-block reconstruction. Related symbols are `PacketOperator`, `TransferGraph`, `ComputationResult`, and Chapter 64 transfer recipes.

A.4 Coordinates and analysis-synthesis

A.4 · `AnalysisSynthesisBlocks`

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.AnalysisSynthesisBlocks`. Name-preserving aliases `komorebi_math.v1.AnalysisSynthesisBlocks` and `komorebi_math.api.v1.AnalysisSynthesisBlocks`. Tier `[STABLE]`. Manifest kind `class`. Reflected implementation `komorebi_math.coordinates.analysis_synthesis.AnalysisSynthesisBlocks`. Profile `A4-AS`.

Manifest signature (s).

```
(system: "'AnalysisSynthesisSystem'", groups: 'tuple[tuple[int, ...], ...]', blocks: 'tuple[tuple[np.ndarray,
   ↪  ...], ...]', operator: 'np.ndarray') -> None
```

Definition. Packet blocks of one operator under a declared analysis/synthesis atlas.

Contract (D, C, H, R, X). Finite analysis and synthesis matrices map ambient vectors/operators to redundant coordinates and packet blocks. Matrix shapes must compose and `S A = I` must hold within the declared tolerance. Results are an analysis/synthesis system or grouped block record. Shape or reconstruction failures raise dimension or validation errors.

Guarantees (E, P, B, K). The implementation is tolerance-qualified numerical linear algebra. `AnalysisSynthesisSystem` has codec `coordinates.analysis_synthesis` version 1. `AnalysisSynthesisBlocks` has no separate codec. NumPy is required, and dense costs follow the displayed matrix products and block extraction.

Evidence and links (V, L). The first stable semantic case exercises analysis, synthesis, and transfer blocks. Chapter 20 gives minimal, redundant, adversarial, and portable examples. Related symbols are `PacketSystem`, `CoordinateAtlas`, and `block_from_paths`.

A.4 · `AnalysisSynthesisSystem`

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.AnalysisSynthesisSystem`. Name-preserving aliases `komorebi_math.v1.AnalysisSynthesisSystem` and `komorebi_math.api.v1.AnalysisSynthesisSystem`. Tier `[STABLE]`. Manifest kind `class`. Reflected implementation `komorebi_math.coordinates.analysis_synthesis.AnalysisSynthesisSystem`. Profile `A4-AS`.

Manifest signature (s).

```
(analysis: 'np.ndarray', synthesis: 'np.ndarray', source: 'str' = 'source', coordinates: 'str' = 'coordinates',
   ↪  tolerance: 'float' = 1e-12, theorem: 'str' = 'KM-AS-01') -> None
```

Definition. A finite analysis map `A` and synthesis map `S` with `S A = I`.

Contract (D, C, H, R, X). Finite analysis and synthesis matrices map ambient vectors/operators to redundant coordinates and packet blocks. Matrix shapes must compose and `S A = I` must hold within the declared tolerance. Results are an analysis/synthesis system or grouped block record. Shape or reconstruction failures raise dimension or validation errors.

Guarantees (E, P, B, K). The implementation is tolerance-qualified numerical linear algebra. `AnalysisSynthesisSystem` has codec `coordinates.analysis_synthesis` version 1. `AnalysisSynthesisBlocks` has no separate codec. NumPy is required, and dense costs follow the displayed matrix products and block extraction.

Evidence and links (V, L). The first stable semantic case exercises analysis, synthesis, and transfer blocks. Chapter 20 gives minimal, redundant, adversarial, and portable examples. Related symbols are `PacketSystem`, `CoordinateAtlas`, and `block_from_paths`.

A.4 · CoordinateAtlas

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.CoordinateAtlas`. Name-preserving aliases `komorebi_math.v1.CoordinateAtlas` and `komorebi_math.api.v1.CoordinateAtlas`. Tier `[STABLE]`. Manifest kind `class`. Reflected implementation `komorebi_math.coordinates.basis.CoordinateAtlas`. Profile `A4-B1`.

Manifest signature (s).

```
(name: 'str' = 'coordinate atlas') -> None
```

Definition. A graph of exact coordinate systems and composable transforms.

Contract (D, C, H, R, X). Rational basis vectors and exact transform edges map to finite bases or an atlas of composable coordinate systems. Vectors must share dimension and be linearly independent. Atlas paths must have compatible endpoints. Results are exact basis and atlas records. Invalid dimensions, dependence, or missing paths fail validation.

Guarantees (E, P, B, K). The finite rational path is exact. `FiniteBasis` and `CoordinateAtlas` have version-1 stable codecs. The base path uses `fractions` and finite matrix operations. Construction and conversion costs are bounded by basis dimension and atlas path length.

Evidence and links (V, L). The exact-basis roundtrip semantic case and Chapters 15 and 20 verify these objects. Related symbols are coordinate transforms, `basis_from_matrix`, `standard_basis`, and exact-coordinate recipes.

A.4 · CoordinateTransform

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.CoordinateTransform`. Name-preserving aliases `komorebi_math.v1.CoordinateTransform` and `komorebi_math.api.v1.CoordinateTransform`. Tier `[STABLE]`. Manifest kind `class`. Reflected implementation `komorebi_math.coordinates.transform.CoordinateTransform`. Profile `A4-T1`. Deliberate cross-name alias: `komorebi_math.stable.ExactCoordinateTransform`. Both export cards remain present.

Manifest signature (s).

```
(source: 'str', target: 'str', matrix: 'Matrix', theorem: 'str', description: 'str' = '') -> None
```

Definition. An exact matrix that maps source coordinates to target coordinates.

Contract (D, C, H, R, X). A labeled finite matrix maps coordinates from a named source to a named target. Exact transforms require exact matrix entries. Numeric transforms use arrays and a tolerance. Results are transform records with composition and application operations. Incompatible dimensions, singular assumptions, or failed tolerance checks raise explicit errors.

Guarantees (E, P, B, K). `CoordinateTransform` and its deliberate alias `ExactCoordinateTransform` share codec `coordinates.transform` version 1. `NumericCoordinateTransform` uses `coordinates.numeric_transform` version 1. Exact paths use rational arithmetic and numerical paths use NumPy. Dense application and composition have ordinary matrix costs.

Evidence and links (V, L). Chapters 15 through 20 provide exact and numerical transform examples and failure cases. The exact-basis semantic fixture covers a roundtrip. Related constructors are connection, difference, formal, and standard-basis functions.

A.4 · ExactCoordinateTransform

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.ExactCoordinateTransform`. Name-preserving aliases `komorebi_math.v1.ExactCoordinateTransform` and `komorebi_math.api.v1.ExactCoordinateTransform`. Tier `[STABLE]`. Manifest kind `class`. Reflected implementation `komorebi_math.coordinates.transform.CoordinateTransform`. Profile `A4-T1`. Deliberate cross-name alias: `komorebi_math.stable.CoordinateTransform`. Both export cards remain present.

Manifest signature (s).

```
(source: 'str', target: 'str', matrix: 'Matrix', theorem: 'str', description: 'str' = '') -> None
```

Definition. An exact matrix that maps source coordinates to target coordinates.

Contract (D, C, H, R, X). A labeled finite matrix maps coordinates from a named source to a named target. Exact transforms require exact matrix entries. Numeric transforms use arrays and a tolerance. Results are transform records with composition and application operations. Incompatible dimensions, singular assumptions, or failed tolerance checks raise explicit errors.

Guarantees (E, P, B, K). `CoordinateTransform` and its deliberate alias `ExactCoordinateTransform` share codec `coordinates.transform` version 1. `NumericCoordinateTransform` uses `coordinates.numeric_transform` version 1. Exact paths use rational arithmetic and numerical paths use NumPy. Dense application and composition have ordinary matrix costs.

Evidence and links (V, L). Chapters 15 through 20 provide exact and numerical transform examples and failure cases. The exact-basis semantic fixture covers a roundtrip. Related constructors are connection, difference, formal, and standard-basis functions.

A.4 · `FiniteBasis`

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.FiniteBasis`. Name-preserving aliases `komorebi_math.v1.FiniteBasis` and `komorebi_math.api.v1.FiniteBasis`. Tier `[STABLE]`. Manifest kind `class`. Reflected implementation `komorebi_math.coordinates.basis.FiniteBasis`. Profile `A4-B1`.

Manifest signature (s).

```
(name: 'str', vectors: 'tuple[Vector, ...]', labels: 'tuple[str, ...]' = (), description: 'str' = '') -> None
```

Definition. A linearly independent finite basis inside a rational ambient space.

Contract (D, C, H, R, X). Rational basis vectors and exact transform edges map to finite bases or an atlas of composable coordinate systems. Vectors must share dimension and be linearly independent. Atlas paths must have compatible endpoints. Results are exact basis and atlas records. Invalid dimensions, dependence, or missing paths fail validation.

Guarantees (E, P, B, K). The finite rational path is exact. `FiniteBasis` and `CoordinateAtlas` have version-1 stable codecs. The base path uses `fractions` and finite matrix operations. Construction and conversion costs are bounded by basis dimension and atlas path length.

Evidence and links (V, L). The exact-basis roundtrip semantic case and Chapters 15 and 20 verify these objects. Related symbols are coordinate transforms, `basis_from_matrix`, `standard_basis`, and exact-coordinate recipes.

A.4 · `FormalCoordinateChange`

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.FormalCoordinateChange`. Name-preserving aliases `komorebi_math.v1.FormalCoordinateChange` and `komorebi_math.api.v1.FormalCoordinateChange`. Tier `[STABLE]`. Manifest kind `class`. Reflected implementation `komorebi_math.coordinates.formal.FormalCoordinateChange`. Profile `A4-F1`.

Manifest signature (s).

```
(name: 'str', coefficients: 'Vector', source: 'str' = 'source formal coordinate', target: 'str' = 'target formal
   ↪ coordinate') -> None
```

Definition. A finite exact jet `phi(z)=a_1 z+a_2 z^2+...`.

Contract (D, C, H, R, X). A finite exact coefficient vector with nonzero linear term maps to a formal coordinate jet. Coefficients must belong to the accepted exact scalar protocol and satisfy invertibility at first order. The result is a `FormalCoordinateChange`. Malformed or noninvertible data raise validation errors.

Guarantees (E, P, B, K). The declared finite-jet algebra is exact and has codec `coordinates.formal_change` version 1. It uses base exact arithmetic. Composition and inversion costs grow with truncation order and coefficient arithmetic.

Evidence and links (V, L). Chapters 15 through 16 give construction, composition, and failure examples. Stable structural checks cover both exports. Coordinate tests verify finite-jet identities and persistence.

A.4 · NumericCoordinateTransform

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.NumericCoordinateTransform`. Name-preserving aliases `komorebi_math.v1.NumericCoordinateTransform` and `komorebi_math.api.v1.NumericCoordinateTransform`. Tier `[STABLE]`. Manifest kind `class`. Reflected implementation `komorebi_math.coordinates.numeric.NumericCoordinateTransform`. Profile `A4-T1`.

Manifest signature (s).

```
(source: 'str', target: 'str', matrix: 'np.ndarray', theorem: 'str' = 'CC.NM1', description: 'str' = '', tolerance:
   ↪ 'float' = 1e-12) -> None
```

Definition. A finite numerical matrix between coordinate systems.

Contract (D, C, H, R, X). A labeled finite matrix maps coordinates from a named source to a named target. Exact transforms require exact matrix entries. Numeric transforms use arrays and a tolerance. Results are transform records with composition and application operations. Incompatible dimensions, singular assumptions, or failed tolerance checks raise explicit errors.

Guarantees (E, P, B, K). `CoordinateTransform` and its deliberate alias `ExactCoordinateTransform` share codec `coordinates.transform` version 1. `NumericCoordinateTransform` uses `coordinates.numeric_transform` version 1. Exact paths use rational arithmetic and numerical paths use NumPy. Dense application and composition have ordinary matrix costs.

Evidence and links (V, L). Chapters 15 through 20 provide exact and numerical transform examples and failure cases. The exact-basis semantic fixture covers a roundtrip. Related constructors are connection, difference, formal, and standard-basis functions.

A.4 · basis_from_matrix

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.basis_from_matrix`. Name-preserving aliases `komorebi_math.v1.basis_from_matrix` and `komorebi_math.api.v1.basis_from_matrix`. Tier `[STABLE]`. Manifest kind `function`. Reflected implementation `komorebi_math.coordinates.basis.basis_from_matrix`. Profile `A4-B1`.

Manifest signature (s).

```
(name: 'str', matrix: 'Iterable[Iterable[Scalar]]', *, labels: 'Sequence[str]' = (), description: 'str' = '') ->
   ↪ 'FiniteBasis'
```

Definition. Construct a basis from a matrix whose columns are basis vectors.

Contract (D, C, H, R, X). Rational basis vectors and exact transform edges map to finite bases or an atlas of composable coordinate systems. Vectors must share dimension and be linearly independent. Atlas paths must have compatible endpoints. Results are exact basis and atlas records. Invalid dimensions, dependence, or missing paths fail validation.

Guarantees (E, P, B, K). The finite rational path is exact. `FiniteBasis` and `CoordinateAtlas` have version-1 stable codecs. The base path uses `fractions` and finite matrix operations. Construction and conversion costs are bounded by basis dimension and atlas path length.

Evidence and links (V, L). The exact-basis roundtrip semantic case and Chapters 15 and 20 verify these objects. Related symbols are coordinate transforms, `basis_from_matrix`, `standard_basis`, and exact-coordinate recipes.

A.4 · connection_transform

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.connection_transform`. Name-preserving aliases `komorebi_math.v1.connection_transform` and `komorebi_math.api.v1.connection_transform`. Tier `[STABLE]`. Manifest kind `function`. Reflected implementation `komorebi_math.coordinates.polynomials.connection_transform`. Profile `A4-C1`.

Manifest signature (s).

```
(source: 'PolynomialFamily', target: 'PolynomialFamily', degree: 'int', *, budget: 'CoordinateBudget' =
    ↪ CoordinateBudget(maximum_matrix_dimension=512, maximum_polynomial_degree=512, maximum_horizon=4096,
    ↪ maximum_exact_entries=1000000)) -> 'CoordinateTransform'
```

Definition. Return target coordinates of source basis vectors through one degree.

Contract (D, C, H, R, X). A family name, matrix, or finite size/degree maps to a built-in polynomial family or exact connection/difference transform. Names must resolve, sizes and degrees must be nonnegative, matrices must define a basis, and coordinate budgets must permit allocation. Results are families, bases, or transforms. Unknown names and exceeded budgets fail explicitly.

Guarantees (E, P, B, K). The intended combinatorial and polynomial transformations are exact. Returned stable basis/transform types are portable through their own codecs. Base exact arithmetic is used, with optional symbolic support inside family implementations. Dense work is bounded by degree and the declared coordinate budget.

Evidence and links (V, L). Chapters 16 through 17 contain connection and forward-difference examples. The exact-basis semantic case and coordinate tests verify roundtrips. Related symbols are `FiniteBasis`, `CoordinateTransform`, and `get_polynomial_family`.

A.4 · formal_coordinate_from_iterable

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.formal_coordinate_from_iterable`. Name-preserving aliases `komorebi_math.v1.formal_coordinate_from_iterable` and `komorebi_math.api.v1.formal_coordinate_from_iterable`. Tier `[STABLE]`. Manifest kind `function`. Reflected implementation `komorebi_math.coordinates.formal.formal_coordinate_from_iterable`. Profile `A4-F1`.

Manifest signature (s).

```
(coefficients: 'Iterable[Scalar]', *, name: 'str' = 'custom formal coordinate', source: 'str' = 'source formal
    ↪ coordinate', target: 'str' = 'target formal coordinate') -> 'FormalCoordinateChange'
```

Definition. Construct a formal coordinate change from exact coefficients.

Contract (D, C, H, R, X). A finite exact coefficient vector with nonzero linear term maps to a formal coordinate jet. Coefficients must belong to the accepted exact scalar protocol and satisfy invertibility at first order. The result is a `FormalCoordinateChange`. Malformed or noninvertible data raise validation errors.

Guarantees (E, P, B, K). The declared finite-jet algebra is exact and has codec `coordinates.formal_change` version 1. It uses base exact arithmetic. Composition and inversion costs grow with truncation order and coefficient arithmetic.

Evidence and links (V, L). Chapters 15 through 16 give construction, composition, and failure examples. Stable structural checks cover both exports. Coordinate tests verify finite-jet identities and persistence.

A.4 · forward_difference_transform

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.forward_difference_transform`. Name-preserving aliases `komorebi_math.v1.forward_difference_transform` and `komorebi_math.api.v1.forward_difference_transform`. Tier `[STABLE]`. Manifest kind `function`. Reflected implementation `komorebi_math.coordinates.combinatorial.forward_difference_transform`. Profile `A4-C1`.

Manifest signature (`s`).

```
(size: 'int', *, budget: 'CoordinateBudget' = CoordinateBudget(maximum_matrix_dimension=512,
    ↪ maximum_polynomial_degree=512, maximum_horizon=4096, maximum_exact_entries=1000000)) ->
    ↪ 'CoordinateTransform'
```

Definition. Map sampled values `f(0),...,f(N)` to forward differences at zero.

Contract (`D, C, H, R, X`). A family name, matrix, or finite size/degree maps to a built-in polynomial family or exact connection/difference transform. Names must resolve, sizes and degrees must be nonnegative, matrices must define a basis, and coordinate budgets must permit allocation. Results are families, bases, or transforms. Unknown names and exceeded budgets fail explicitly.

Guarantees (`E, P, B, K`). The intended combinatorial and polynomial transformations are exact. Returned stable basis/transform types are portable through their own codecs. Base exact arithmetic is used, with optional symbolic support inside family implementations. Dense work is bounded by degree and the declared coordinate budget.

Evidence and links (`V, L`). Chapters 16 through 17 contain connection and forward-difference examples. The exact-basis semantic case and coordinate tests verify roundtrips. Related symbols are `FiniteBasis`, `CoordinateTransform`, and `get_polynomial_family`.

A.4 · `get_polynomial_family`

Identity (`q, a, t, k`). Canonical manifest name `komorebi_math.stable.get_polynomial_family`. Name-preserving aliases `komorebi_math.v1.get_polynomial_family` and `komorebi_math.api.v1.get_polynomial_family`. Tier `[STABLE]`. Manifest kind `function`. Reflected implementation `komorebi_math.coordinates.classical.get_polynomial_family`. Profile `A4-C1`.

Manifest signature (`s`).

```
(name: 'str') -> 'PolynomialFamily'
```

Definition. Return one built-in graded polynomial family by name or short alias.

Contract (`D, C, H, R, X`). A family name, matrix, or finite size/degree maps to a built-in polynomial family or exact connection/difference transform. Names must resolve, sizes and degrees must be nonnegative, matrices must define a basis, and coordinate budgets must permit allocation. Results are families, bases, or transforms. Unknown names and exceeded budgets fail explicitly.

Guarantees (`E, P, B, K`). The intended combinatorial and polynomial transformations are exact. Returned stable basis/transform types are portable through their own codecs. Base exact arithmetic is used, with optional symbolic support inside family implementations. Dense work is bounded by degree and the declared coordinate budget.

Evidence and links (`V, L`). Chapters 16 through 17 contain connection and forward-difference examples. The exact-basis semantic case and coordinate tests verify roundtrips. Related symbols are `FiniteBasis`, `CoordinateTransform`, and `get_polynomial_family`.

A.4 · `standard_basis`

Identity (`q, a, t, k`). Canonical manifest name `komorebi_math.stable.standard_basis`. Name-preserving aliases `komorebi_math.v1.standard_basis` and `komorebi_math.api.v1.standard_basis`. Tier `[STABLE]`. Manifest kind `function`. Reflected implementation `komorebi_math.coordinates.basis.standard_basis`. Profile `A4-B1`.

Manifest signature (`s`).

```
(dimension: 'int', *, name: 'str' = 'standard') -> 'FiniteBasis'
```

Definition. Return the standard basis of a rational vector space.

Contract (`D, C, H, R, X`). Rational basis vectors and exact transform edges map to finite bases or an atlas of composable coordinate systems. Vectors must share dimension and be linearly independent. Atlas paths must have compatible endpoints. Results are exact basis and atlas records. Invalid dimensions, dependence, or missing paths fail validation.

Guarantees (E, P, B, K). The finite rational path is exact. `FiniteBasis` and `CoordinateAtlas` have version-1 stable codecs. The base path uses `fractions` and finite matrix operations. Construction and conversion costs are bounded by basis dimension and atlas path length.

Evidence and links (V, L). The exact-basis roundtrip semantic case and Chapters 15 and 20 verify these objects. Related symbols are coordinate transforms, `basis_from_matrix`, `standard_basis`, and exact-coordinate recipes.

A.5 Refinement, residue, polyphase, and arithmetic polyphase

A.5 · `ArithmeticPolyphase`

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.ArithmeticPolyphase`. Name-preserving aliases `komorebi_math.v1.ArithmeticPolyphase` and `komorebi_math.api.v1.ArithmeticPolyphase`. Tier `[STABLE]`. Manifest kind `class`. Reflected implementation `komorebi_math.arithmetic.polyphase.ArithmeticPolyphase`. Profile `A5-A1`.

Manifest signature (s).

```
(modulus: 'int', channels: 'tuple[tuple[Number, ...], ...]', length: 'int') -> None
```

Definition. Exact residue channels of one finite arithmetic sequence.

Contract (D, C, H, R, X). Finite exact sequences, moduli, residue channels, mixed-radix digits, or Mahler coefficients map to arithmetic polyphase records and exact address conversions. Moduli/radices must be positive, digits and addresses in range, and channel lengths consistent. Results are integers, tuples, channels, or `MahlerSeries`. Invalid arithmetic data raise validation errors.

Guarantees (E, P, B, K). These operations are exact over their accepted number domain. `ArithmeticPolyphase` has codec `arithmetic.polyphase_coordinates` version 1. Returned Mahler series use their own domain support. No numerical backend is required. Analysis/synthesis are linear in sequence length, and mixed-radix conversion is linear in digit count.

Evidence and links (V, L). The exact-arithmetic polyphase semantic case and Chapters 13 through 14, 23, and 25 verify roundtrips and address laws. Related symbols are residue and signal polyphase operations.

A.5 · `LatticePolyphaseRefinement`

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.LatticePolyphaseRefinement`. Name-preserving aliases `komorebi_math.v1.LatticePolyphaseRefinement` and `komorebi_math.api.v1.LatticePolyphaseRefinement`. Tier `[STABLE]`. Manifest kind `class`. Reflected implementation `komorebi_math.refinement.polyphase.LatticePolyphaseRefinement`. Profile `A5-R1`.

Manifest signature (s).

```
(coarse_strides: 'tuple[int, ...]', fine_strides: 'tuple[int, ...]') -> None
```

Definition. Componentwise refinement of rectangular lattice polyphase coordinates.

Contract (D, C, H, R, X). One-dimensional or rectangular polyphase coordinates plus divisible coarse/fine orders map to refined or coarsened coordinate records. Orders and strides must be positive and componentwise divisible, shapes must match, and certificate tolerances must be nonnegative. Results preserve the supported coordinate-record variant. Invalid relations raise validation or dimension errors.

Guarantees (E, P, B, K). Index regrouping is exact. Array comparison certificates are tolerance-qualified. `PolyphaseRefinement`, `LatticePolyphaseRefinement`, and `PolyphaseRefinementTower` have version-1 codecs. NumPy stores signal records. Work and memory are linear in materialized sample count plus channel bookkeeping.

Evidence and links (V, L). Signal, arithmetic, rectangular-lattice, and portable-tower semantic cases cover this family. Chapters 12 through 14 give direct-fine equivalence, alias, and failure examples. Related symbols include analysis/synthesis and `refinement_children`.

A.5 · PacketRefinement

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.PacketRefinement`. Name-preserving aliases `komorebi_math.v1.PacketRefinement` and `komorebi_math.api.v1.PacketRefinement`. Tier `[STABLE]`. Manifest kind `class`. Reflected implementation `komorebi_math.refinement.towers.PacketRefinement`. Profile `A5-T1`.

Manifest signature (s).

```
(coarse: 'PacketSystem', fine: 'PacketSystem', parent_of_fine: 'tuple[int, ...]') -> None
```

Definition. One exact refinement from a coarse packet system to a fine system.

Contract (D, C, H, R, X). Compatible coarse/fine packet systems map to a `PacketRefinement`. A composable finite chain maps to `RefinementTower`. Two commuting systems map to a common refinement and its two arrows. Completeness, commutation, parent maps, and adjacent endpoints must agree. Violations raise validation or dimension errors.

Guarantees (E, P, B, K). Finite projector identities are exact on exact backends and tolerance-qualified otherwise. These packet-tower records have no frozen codec on the stable surface. Dense projector products scale with ambient dimension and packet counts. Tower work sums across levels.

Evidence and links (V, L). Chapter 12 supplies refinement and common-refinement examples and certificates. Core/refinement tests verify identities. Related symbols are `PacketSystem`, `PolyphaseRefinementTower`, and Chapter 64 algebra recipes.

A.5 · PolyphaseCoordinateRecord

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.PolyphaseCoordinateRecord`. Name-preserving aliases `komorebi_math.v1.PolyphaseCoordinateRecord` and `komorebi_math.api.v1.PolyphaseCoordinateRecord`. Tier `[STABLE]`. Manifest kind `data`. Reflected implementation `types.UnionType`. Profile `A5-U1`.

Manifest signature (s).

```
null
```

Definition. The union `PolyphaseCoordinates | ResidueCoordinates | ArithmeticPolyphase`. It is keyed by exported name, not collapsed to `types.UnionType`.

Contract (D, C, H, R, X). This data export is the union of `PolyphaseCoordinates`, `ResidueCoordinates`, and `ArithmeticPolyphase`, accepted by the generic one-dimensional refine/coarsen functions. It takes no call arguments and returns no value by itself. The manifest signature is therefore `null`.

Guarantees (E, P, B, K). The union is exact typing metadata, has no codec of its own, and introduces no backend work or runtime complexity. Each concrete member retains its own persistence and exactness contract.

Evidence and links (V, L). The stable reference-card parser verifies the union binding as a distinct data export rather than collapsing it by runtime type owner. Chapters 13 through 14 and the A.5 cards for the three concrete records provide examples and links.

A.5 · PolyphaseRefinement

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.PolyphaseRefinement`. Name-preserving aliases `komorebi_math.v1.PolyphaseRefinement` and `komorebi_math.api.v1.PolyphaseRefinement`. Tier `[STABLE]`. Manifest kind `class`. Reflected implementation `komorebi_math.refinement.polyphase.PolyphaseRefinement`. Profile `A5-R1`.

Manifest signature (s).

```
(coarse_order: 'int', fine_order: 'int') -> None
```

Definition. One canonical refinement between divisible one-dimensional packet orders.

Contract (D, C, H, R, X). One-dimensional or rectangular polyphase coordinates plus divisible coarse/fine orders map to refined or coarsened coordinate records. Orders and strides must be positive and componentwise divisible, shapes must match, and certificate tolerances must be nonnegative. Results preserve the supported coordinate-record variant. Invalid relations raise validation or dimension errors.

Guarantees (E, P, B, K). Index regrouping is exact. Array comparison certificates are tolerance-qualified. `PolyphaseRefinement`, `LatticePolyphaseRefinement`, and `PolyphaseRefinementTower` have version-1 codecs. NumPy stores signal records. Work and memory are linear in materialized sample count plus channel bookkeeping.

Evidence and links (V, L). Signal, arithmetic, rectangular-lattice, and portable-tower semantic cases cover this family. Chapters 12 through 14 give direct-fine equivalence, alias, and failure examples. Related symbols include analysis/synthesis and `refinement_children`.

A.5 · `PolyphaseRefinementTower`

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.PolyphaseRefinementTower`. Name-preserving aliases `komorebi_math.v1.PolyphaseRefinementTower` and `komorebi_math.api.v1.PolyphaseRefinementTower`. Tier `[STABLE]`. Manifest kind `class`. Reflected implementation `komorebi_math.refinement.polyphase.PolyphaseRefinementTower`. Profile `A5-R1`.

Manifest signature (s).

```
(orders: 'tuple[int, ...]') -> None
```

Definition. A finite chain of compatible polyphase orders.

Contract (D, C, H, R, X). One-dimensional or rectangular polyphase coordinates plus divisible coarse/fine orders map to refined or coarsened coordinate records. Orders and strides must be positive and componentwise divisible, shapes must match, and certificate tolerances must be nonnegative. Results preserve the supported coordinate-record variant. Invalid relations raise validation or dimension errors.

Guarantees (E, P, B, K). Index regrouping is exact. Array comparison certificates are tolerance-qualified. `PolyphaseRefinement`, `LatticePolyphaseRefinement`, and `PolyphaseRefinementTower` have version-1 codecs. NumPy stores signal records. Work and memory are linear in materialized sample count plus channel bookkeeping.

Evidence and links (V, L). Signal, arithmetic, rectangular-lattice, and portable-tower semantic cases cover this family. Chapters 12 through 14 give direct-fine equivalence, alias, and failure examples. Related symbols include analysis/synthesis and `refinement_children`.

A.5 · `RefinementTower`

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.RefinementTower`. Name-preserving aliases `komorebi_math.v1.RefinementTower` and `komorebi_math.api.v1.RefinementTower`. Tier `[STABLE]`. Manifest kind `class`. Reflected implementation `komorebi_math.refinement.towers.RefinementTower`. Profile `A5-T1`.

Manifest signature (s).

```
(refinements: 'tuple[PacketRefinement, ...] | list[PacketRefinement]') -> 'None'
```

Definition. A finite sequence of exact packet refinements.

Contract (D, C, H, R, X). Compatible coarse/fine packet systems map to a `PacketRefinement`. A composable finite chain maps to `RefinementTower`. Two commuting systems map to a common refinement and its two arrows. Completeness, commutation, parent maps, and adjacent endpoints must agree. Violations raise validation or dimension errors.

Guarantees (E, P, B, K). Finite projector identities are exact on exact backends and tolerance-qualified otherwise. These packet-tower records have no frozen codec on the stable surface. Dense projector products scale with ambient dimension and packet counts. Tower work sums across levels.

Evidence and links (V, L). Chapter 12 supplies refinement and common-refinement examples and certificates. Core/refinement tests verify identities. Related symbols are `PacketSystem`, `PolyphaseRefinementTower`, and Chapter 64 algebra recipes.

A.5 · ResidueCoordinates

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.ResidueCoordinates`. Name-preserving aliases `komorebi_math.v1.ResidueCoordinates` and `komorebi_math.api.v1.ResidueCoordinates`. Tier `[STABLE]`. Manifest kind `class`. Reflected implementation `komorebi_math.refinement.residue.ResidueCoordinates`. Profile `A5-Z1`.

Manifest signature (s).

```
(modulus: 'int', channels: 'tuple[np.ndarray, ...]', length: 'int') -> None
```

Definition. Ordinary polyphase channels for one finite sequence.

Contract (D, C, H, R, X). A finite sequence and positive modulus map to ordinary residue channels. Nested residue data map to the product modulus and back. Channel counts, dimensions, lengths, and residue ranges must agree. Results are `ResidueCoordinates`, combined addresses, or reconstructed arrays. Invalid inputs raise validation or dimension errors.

Guarantees (E, P, B, K). Address laws are exact. NumPy reconstruction preserves supplied values subject to array dtype. `ResidueCoordinates` has codec `refinement.residue_coordinates` version 1. Analysis and synthesis are linear in sequence length. Nested addressing is constant-time integer arithmetic.

Evidence and links (V, L). Chapters 13 through 14 verify CRT-style addresses, nesting, synthesis, and failure cases. Stable arithmetic/refinement semantic cases cover roundtrips. Related symbols are arithmetic and signal polyphase functions.

A.5 · arithmetic_mixed_radix_address

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.arithmetic_mixed_radix_address`. Name-preserving aliases `komorebi_math.v1.arithmetic_mixed_radix_address` and `komorebi_math.api.v1.arithmetic_mixed_radix_address`. Tier `[STABLE]`. Manifest kind `function`. Reflected implementation `komorebi_math.arithmetic.polyphase.mixed_radix_address`. Profile `A5-A1`.

Manifest signature (s).

```
(digits: 'Iterable[int]', radices: 'Iterable[int]') -> 'int'
```

Definition. Encode nested residue digits in least significant first order.

Contract (D, C, H, R, X). Finite exact sequences, moduli, residue channels, mixed-radix digits, or Mahler coefficients map to arithmetic polyphase records and exact address conversions. Moduli/radices must be positive, digits and addresses in range, and channel lengths consistent. Results are integers, tuples, channels, or `MahlerSeries`. Invalid arithmetic data raise validation errors.

Guarantees (E, P, B, K). These operations are exact over their accepted number domain. `ArithmeticPolyphase` has codec `arithmetic.polyphase_coordinates` version 1. Returned Mahler series use their own domain support. No numerical backend is required. Analysis/synthesis are linear in sequence length, and mixed-radix conversion is linear in digit count.

Evidence and links (V, L). The exact-arithmetic polyphase semantic case and Chapters 13 through 14, 23, and 25 verify roundtrips and address laws. Related symbols are residue and signal polyphase operations.

A.5 · arithmetic_mixed_radix_digits

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.arithmetic_mixed_radix_digits`. Name-preserving aliases `komorebi_math.v1.arithmetic_mixed_radix_digits` and `komorebi_math.api.v1.arithmetic_mixed_radix_digits`. Tier `[STABLE]`. Manifest kind `function`. Reflected implementation `komorebi_math.arithmetic.polyphase.mixed_radix_digits`. Profile `A5-A1`.

Manifest signature (s).

```
(address: 'int', radices: 'Iterable[int]') -> 'tuple[int, ...]'
```

Definition. Decode one mixed radix address in least significant first order.

Contract (D, C, H, R, X). Finite exact sequences, moduli, residue channels, mixed-radix digits, or Mahler coefficients map to arithmetic polyphase records and exact address conversions. Moduli/radices must be positive, digits and addresses in range, and channel lengths consistent. Results are integers, tuples, channels, or `MahlerSeries`. Invalid arithmetic data raise validation errors.

Guarantees (E, P, B, K). These operations are exact over their accepted number domain. `ArithmeticPolyphase` has codec `arithmetic.polyphase_coordinates` version 1. Returned Mahler series use their own domain support. No numerical backend is required. Analysis/synthesis are linear in sequence length, and mixed-radix conversion is linear in digit count.

Evidence and links (V, L). The exact-arithmetic polyphase semantic case and Chapters 13 through 14, 23, and 25 verify roundtrips and address laws. Related symbols are residue and signal polyphase operations.

A.5 · `arithmetic_polyphase_analyze`

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.arithmetic_polyphase_analyze`. Name-preserving aliases `komorebi_math.v1.arithmetic_polyphase_analyze` and `komorebi_math.api.v1.arithmetic_polyphase_analyze`. Tier `[STABLE]`. Manifest kind `function`. Reflected implementation `komorebi_math.arithmetic.polyphase.polyphase_analyze`. Profile `A5-A1`.

Manifest signature (s).

```
(values: 'Iterable[Number]', modulus: 'int') -> 'ArithmeticPolyphase'
```

Definition. Split a finite sequence into exact residue channels.

Contract (D, C, H, R, X). Finite exact sequences, moduli, residue channels, mixed-radix digits, or Mahler coefficients map to arithmetic polyphase records and exact address conversions. Moduli/radices must be positive, digits and addresses in range, and channel lengths consistent. Results are integers, tuples, channels, or `MahlerSeries`. Invalid arithmetic data raise validation errors.

Guarantees (E, P, B, K). These operations are exact over their accepted number domain. `ArithmeticPolyphase` has codec `arithmetic.polyphase_coordinates` version 1. Returned Mahler series use their own domain support. No numerical backend is required. Analysis/synthesis are linear in sequence length, and mixed-radix conversion is linear in digit count.

Evidence and links (V, L). The exact-arithmetic polyphase semantic case and Chapters 13 through 14, 23, and 25 verify roundtrips and address laws. Related symbols are residue and signal polyphase operations.

A.5 · `arithmetic_polyphase_synthesize`

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.arithmetic_polyphase_synthesize`. Name-preserving aliases `komorebi_math.v1.arithmetic_polyphase_synthesize` and `komorebi_math.api.v1.arithmetic_polyphase_synthesize`. Tier `[STABLE]`. Manifest kind `function`. Reflected implementation `komorebi_math.arithmetic.polyphase.polyphase_synthesize`. Profile `A5-A1`.

Manifest signature (s).

```
(channels: 'Iterable[Iterable[Number]]', length: 'int | None' = None) -> 'tuple[Number, ...]'
```

Definition. Interleave exact residue channels.

Contract (D, C, H, R, X). Finite exact sequences, moduli, residue channels, mixed-radix digits, or Mahler coefficients map to arithmetic polyphase records and exact address conversions. Moduli/radices must be positive, digits and addresses in range, and channel lengths consistent. Results are integers, tuples, channels, or `MahlerSeries`. Invalid arithmetic data raise validation errors.

Guarantees (E, P, B, K). These operations are exact over their accepted number domain. `ArithmeticPolyphase` has codec `arithmetic.polyphase_coordinates` version 1. Returned Mahler series use their own domain support. No numerical backend is required. Analysis/synthesis are linear in sequence length, and mixed-radix conversion is linear in digit count.

Evidence and links (V, L). The exact-arithmetic polyphase semantic case and Chapters 13 through 14, 23, and 25 verify roundtrips and address laws. Related symbols are residue and signal polyphase operations.

A.5 · `coarsen_lattice_polyphase`

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.coarsen_lattice_polyphase`. Name-preserving aliases `komorebi_math.v1.coarsen_lattice_polyphase` and `komorebi_math.api.v1.coarsen_lattice_polyphase`. Tier `[STABLE]`. Manifest kind `function`. Reflected implementation `komorebi_math.refinement.polyphase.coarsen_lattice_polyphase`. Profile `A5-R1`. Deliberate cross-name alias: `komorebi_math.stable.lattice_polyphase_coarsen`. Both export cards remain present.

Manifest signature (s).

```
(coordinates: 'LatticePolyphaseCoordinates', coarse_strides: 'Iterable[int]', *, copy: 'bool' = True) ->
    ↪ 'LatticePolyphaseCoordinates'
```

Definition. Coarsen rectangular lattice coordinates to componentwise coarser strides.

Contract (D, C, H, R, X). One-dimensional or rectangular polyphase coordinates plus divisible coarse/fine orders map to refined or coarsened coordinate records. Orders and strides must be positive and componentwise divisible, shapes must match, and certificate tolerances must be nonnegative. Results preserve the supported coordinate-record variant. Invalid relations raise validation or dimension errors.

Guarantees (E, P, B, K). Index regrouping is exact. Array comparison certificates are tolerance-qualified. `Polyphase Refinement`, `LatticePolyphaseRefinement`, and `PolyphaseRefinementTower` have version-1 codecs. NumPy stores signal records. Work and memory are linear in materialized sample count plus channel bookkeeping.

Evidence and links (V, L). Signal, arithmetic, rectangular-lattice, and portable-tower semantic cases cover this family. Chapters 12 through 14 give direct-fine equivalence, alias, and failure examples. Related symbols include analysis/synthesis and `refinement_children`.

A.5 · `coarsen_polyphase`

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.coarsen_polyphase`. Name-preserving aliases `komorebi_math.v1.coarsen_polyphase` and `komorebi_math.api.v1.coarsen_polyphase`. Tier `[STABLE]`. Manifest kind `function`. Reflected implementation `komorebi_math.refinement.polyphase.coarsen_polyphase`. Profile `A5-R1`. Deliberate cross-name alias: `komorebi_math.stable.polyphase_coarsen`. Both export cards remain present.

Manifest signature (s).

```
(coordinates: 'PolyphaseCoordinateRecord', coarse_order: 'int', *, copy: 'bool' = True) ->
    ↪ 'PolyphaseCoordinateRecord'
```

Definition. Coarsen any supported one-dimensional coordinate record to `coarse_order`.

Contract (D, C, H, R, X). One-dimensional or rectangular polyphase coordinates plus divisible coarse/fine orders map to refined or coarsened coordinate records. Orders and strides must be positive and componentwise divisible, shapes must match, and certificate tolerances must be nonnegative. Results preserve the supported coordinate-record variant. Invalid relations raise validation or dimension errors.

Guarantees (E, P, B, K). Index regrouping is exact. Array comparison certificates are tolerance-qualified. `Polyphase Refinement`, `LatticePolyphaseRefinement`, and `PolyphaseRefinementTower` have version-1 codecs. NumPy stores signal records. Work and memory are linear in materialized sample count plus channel bookkeeping.

Evidence and links (V, L). Signal, arithmetic, rectangular-lattice, and portable-tower semantic cases cover this family. Chapters 12 through 14 give direct-fine equivalence, alias, and failure examples. Related symbols include analysis/synthesis and `refinement_children`.

A.5 · common_refinement

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.common_refinement`. Name-preserving aliases `komorebi_math.v1.common_refinement` and `komorebi_math.api.v1.common_refinement`. Tier `[STABLE]`. Manifest kind `function`. Reflected implementation `komorebi_math.refinement.towers.common_refinement`. Profile `A5-T1`.

Manifest signature (s).

```
(left: 'PacketSystem', right: 'PacketSystem', *, names: 'tuple[str, ...] | None' = None) -> 'tuple[PacketSystem,
   ↪ PacketRefinement, PacketRefinement]'
```

Definition. Construct the common refinement of two commuting packet systems.

Contract (D, C, H, R, X). Compatible coarse/fine packet systems map to a `PacketRefinement`. A composable finite chain maps to `RefinementTower`. Two commuting systems map to a common refinement and its two arrows. Completeness, commutation, parent maps, and adjacent endpoints must agree. Violations raise validation or dimension errors.

Guarantees (E, P, B, K). Finite projector identities are exact on exact backends and tolerance-qualified otherwise. These packet-tower records have no frozen codec on the stable surface. Dense projector products scale with ambient dimension and packet counts. Tower work sums across levels.

Evidence and links (V, L). Chapter 12 supplies refinement and common-refinement examples and certificates. Core/refinement tests verify identities. Related symbols are `PacketSystem`, `PolyphaseRefinementTower`, and Chapter 64 algebra recipes.

A.5 · lattice_polyphase_coarsen

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.lattice_polyphase_coarsen`. Name-preserving aliases `komorebi_math.v1.lattice_polyphase_coarsen` and `komorebi_math.api.v1.lattice_polyphase_coarsen`. Tier `[STABLE]`. Manifest kind `function`. Reflected implementation `komorebi_math.refinement.polyphase.coarsen_lattice_polyphase`. Profile `A5-R1`. Deliberate cross-name alias: `komorebi_math.stable.coarsen_lattice_polyphase`. Both export cards remain present.

Manifest signature (s).

```
(coordinates: 'LatticePolyphaseCoordinates', coarse_strides: 'Iterable[int]', *, copy: 'bool' = True) ->
   ↪ 'LatticePolyphaseCoordinates'
```

Definition. Coarsen rectangular lattice coordinates to componentwise coarser strides.

Contract (D, C, H, R, X). One-dimensional or rectangular polyphase coordinates plus divisible coarse/fine orders map to refined or coarsened coordinate records. Orders and strides must be positive and componentwise divisible, shapes must match, and certificate tolerances must be nonnegative. Results preserve the supported coordinate-record variant. Invalid relations raise validation or dimension errors.

Guarantees (E, P, B, K). Index regrouping is exact. Array comparison certificates are tolerance-qualified. `PolyphaseRefinement`, `LatticePolyphaseRefinement`, and `PolyphaseRefinementTower` have version-1 codecs. NumPy stores signal records. Work and memory are linear in materialized sample count plus channel bookkeeping.

Evidence and links (V, L). Signal, arithmetic, rectangular-lattice, and portable-tower semantic cases cover this family. Chapters 12 through 14 give direct-fine equivalence, alias, and failure examples. Related symbols include analysis/synthesis and `refinement_children`.

A.5 · lattice_polyphase_refine

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.lattice_polyphase_refine`. Name-preserving aliases `komorebi_math.v1.lattice_polyphase_refine` and `komorebi_math.api.v1.lattice_polyphase_refine`. Tier `[STABLE]`. Manifest kind `function`. Reflected implementation `komorebi_math.refinement.polyphase.refine_lattice_polyphase`. Profile `A5-R1`. Deliberate cross-name alias: `komorebi_math.stable.refine_lattice_polyphase`. Both export cards remain present.

Manifest signature (s).

```
(coordinates: 'LatticePolyphaseCoordinates', fine_strides: 'Iterable[int]', *, copy: 'bool' = True) ->
    ↪ 'LatticePolyphaseCoordinates'
```

Definition. Refine rectangular lattice coordinates to componentwise finer strides.

Contract (D, C, H, R, X). One-dimensional or rectangular polyphase coordinates plus divisible coarse/fine orders map to refined or coarsened coordinate records. Orders and strides must be positive and componentwise divisible, shapes must match, and certificate tolerances must be nonnegative. Results preserve the supported coordinate-record variant. Invalid relations raise validation or dimension errors.

Guarantees (E, P, B, K). Index regrouping is exact. Array comparison certificates are tolerance-qualified. `PolyphaseRefinement`, `LatticePolyphaseRefinement`, and `PolyphaseRefinementTower` have version-1 codecs. NumPy stores signal records. Work and memory are linear in materialized sample count plus channel bookkeeping.

Evidence and links (V, L). Signal, arithmetic, rectangular-lattice, and portable-tower semantic cases cover this family. Chapters 12 through 14 give direct-fine equivalence, alias, and failure examples. Related symbols include analysis/synthesis and `refinement_children`.

A.5 · `mahler_polyphase`

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.mahler_polyphase`. Name-preserving aliases `komorebi_math.v1.mahler_polyphase` and `komorebi_math.api.v1.mahler_polyphase`. Tier `[STABLE]`. Manifest kind `function`. Reflected implementation `komorebi_math.arithmetic.polyphase.mahler_polyphase`. Profile `A5-A1`.

Manifest signature (s).

```
(series: 'MahlerSeries', modulus: 'int', address: 'int') -> 'MahlerSeries'
```

Definition. Return the exact Mahler coordinates of `series(modulus*x + address)`.

Contract (D, C, H, R, X). Finite exact sequences, moduli, residue channels, mixed-radix digits, or Mahler coefficients map to arithmetic polyphase records and exact address conversions. Moduli/radices must be positive, digits and addresses in range, and channel lengths consistent. Results are integers, tuples, channels, or `MahlerSeries`. Invalid arithmetic data raise validation errors.

Guarantees (E, P, B, K). These operations are exact over their accepted number domain. `ArithmeticPolyphase` has codec `arithmetic.polyphase_coordinates` version 1. Returned Mahler series use their own domain support. No numerical backend is required. Analysis/synthesis are linear in sequence length, and mixed-radix conversion is linear in digit count.

Evidence and links (V, L). The exact-arithmetic polyphase semantic case and Chapters 13 through 14, 23, and 25 verify roundtrips and address laws. Related symbols are residue and signal polyphase operations.

A.5 · `nested_residue_address`

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.nested_residue_address`. Name-preserving aliases `komorebi_math.v1.nested_residue_address` and `komorebi_math.api.v1.nested_residue_address`. Tier `[STABLE]`. Manifest kind `function`. Reflected implementation `komorebi_math.refinement.residue.nested_residue_address`. Profile `A5-Z1`.

Manifest signature (s).

```
(outer_residue: 'int', outer_modulus: 'int', inner_residue: 'int', inner_modulus: 'int') -> 'int'
```

Definition. Return the combined address `outer_residue + outer_modulus * inner_residue`.

Contract (D, C, H, R, X). A finite sequence and positive modulus map to ordinary residue channels. Nested residue data map to the product modulus and back. Channel counts, dimensions, lengths, and residue ranges must agree. Results are `ResidueCoordinates`, combined addresses, or reconstructed arrays. Invalid inputs raise validation or dimension errors.

Guarantees (E, P, B, K). Address laws are exact. NumPy reconstruction preserves supplied values subject to array dtype. `ResidueCoordinates` has codec `refinement.residue_coordinates` version 1. Analysis and synthesis are linear in sequence length. Nested addressing is constant-time integer arithmetic.

Evidence and links (V, L). Chapters 13 through 14 verify CRT-style addresses, nesting, synthesis, and failure cases. Stable arithmetic/refinement semantic cases cover roundtrips. Related symbols are arithmetic and signal polyphase functions.

A.5 · `nested_residue_analysis`

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.nested_residue_analysis`. Name-preserving aliases `komorebi_math.v1.nested_residue_analysis` and `komorebi_math.api.v1.nested_residue_analysis`. Tier `[STABLE]`. Manifest kind `function`. Reflected implementation `komorebi_math.refinement.residue.nested_residue_analysis`. Profile `A5-Z1`.

Manifest signature (s).

```
(values: 'Any', outer_modulus: 'int', inner_modulus: 'int') -> 'ResidueCoordinates'
```

Definition. Analyze twice and return the equivalent channels modulo the product.

Contract (D, C, H, R, X). A finite sequence and positive modulus map to ordinary residue channels. Nested residue data map to the product modulus and back. Channel counts, dimensions, lengths, and residue ranges must agree. Results are `ResidueCoordinates`, combined addresses, or reconstructed arrays. Invalid inputs raise validation or dimension errors.

Guarantees (E, P, B, K). Address laws are exact. NumPy reconstruction preserves supplied values subject to array dtype. `ResidueCoordinates` has codec `refinement.residue_coordinates` version 1. Analysis and synthesis are linear in sequence length. Nested addressing is constant-time integer arithmetic.

Evidence and links (V, L). Chapters 13 through 14 verify CRT-style addresses, nesting, synthesis, and failure cases. Stable arithmetic/refinement semantic cases cover roundtrips. Related symbols are arithmetic and signal polyphase functions.

A.5 · `polyphase_coarsen`

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.polyphase_coarsen`. Name-preserving aliases `komorebi_math.v1.polyphase_coarsen` and `komorebi_math.api.v1.polyphase_coarsen`. Tier `[STABLE]`. Manifest kind `function`. Reflected implementation `komorebi_math.refinement.polyphase.coarsen_polyphase`. Profile `A5-R1`. Deliberate cross-name alias: `komorebi_math.stable.coarsen_polyphase`. Both export cards remain present.

Manifest signature (s).

```
(coordinates: 'PolyphaseCoordinateRecord', coarse_order: 'int', *, copy: 'bool' = True) ->
    ↪ 'PolyphaseCoordinateRecord'
```

Definition. Coarsen any supported one-dimensional coordinate record to `coarse_order`.

Contract (D, C, H, R, X). One-dimensional or rectangular polyphase coordinates plus divisible coarse/fine orders map to refined or coarsened coordinate records. Orders and strides must be positive and componentwise divisible, shapes must match, and certificate tolerances must be nonnegative. Results preserve the supported coordinate-record variant. Invalid relations raise validation or dimension errors.

Guarantees (E, P, B, K). Index regrouping is exact. Array comparison certificates are tolerance-qualified. `Polyphase Refinement`, `LatticePolyphaseRefinement`, and `PolyphaseRefinementTower` have version-1 codecs. NumPy stores signal records. Work and memory are linear in materialized sample count plus channel bookkeeping.

Evidence and links (V, L). Signal, arithmetic, rectangular-lattice, and portable-tower semantic cases cover this family. Chapters 12 through 14 give direct-fine equivalence, alias, and failure examples. Related symbols include analysis/synthesis and `refinement_children`.

A.5 · polyphase_refine

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.polyphase_refine`. Name-preserving aliases `komorebi_math.v1.polyphase_refine` and `komorebi_math.api.v1.polyphase_refine`. Tier `[STABLE]`. Manifest kind `function`. Reflected implementation `komorebi_math.refinement.polyphase.refine_polyphase`. Profile `A5-R1`. Deliberate cross-name alias: `komorebi_math.stable.refine_polyphase`. Both export cards remain present.

Manifest signature (s).

```
(coordinates: 'PolyphaseCoordinateRecord', fine_order: 'int', *, copy: 'bool' = True) ->
    ↪ 'PolyphaseCoordinateRecord'
```

Definition. Refine any supported one-dimensional coordinate record to `fine_order`.

Contract (D, C, H, R, X). One-dimensional or rectangular polyphase coordinates plus divisible coarse/fine orders map to refined or coarsened coordinate records. Orders and strides must be positive and componentwise divisible, shapes must match, and certificate tolerances must be nonnegative. Results preserve the supported coordinate-record variant. Invalid relations raise validation or dimension errors.

Guarantees (E, P, B, K). Index regrouping is exact. Array comparison certificates are tolerance-qualified. `PolyphaseRefinement`, `LatticePolyphaseRefinement`, and `PolyphaseRefinementTower` have version-1 codecs. NumPy stores signal records. Work and memory are linear in materialized sample count plus channel bookkeeping.

Evidence and links (V, L). Signal, arithmetic, rectangular-lattice, and portable-tower semantic cases cover this family. Chapters 12 through 14 give direct-fine equivalence, alias, and failure examples. Related symbols include analysis/synthesis and `refinement_children`.

A.5 · polyphase_refinement_certificate

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.polyphase_refinement_certificate`. Name-preserving aliases `komorebi_math.v1.polyphase_refinement_certificate` and `komorebi_math.api.v1.polyphase_refinement_certificate`. Tier `[STABLE]`. Manifest kind `function`. Reflected implementation `komorebi_math.refinement.polyphase.polyphase_refinement_certificate`. Profile `A5-R1`.

Manifest signature (s).

```
(coordinates: 'PolyphaseCoordinateRecord', fine_order: 'int', *, tolerance: 'float' = 1e-12) -> 'Certificate'
```

Definition. Certify refinement and direct-analysis equivalence for one coordinate record.

Contract (D, C, H, R, X). One-dimensional or rectangular polyphase coordinates plus divisible coarse/fine orders map to refined or coarsened coordinate records. Orders and strides must be positive and componentwise divisible, shapes must match, and certificate tolerances must be nonnegative. Results preserve the supported coordinate-record variant. Invalid relations raise validation or dimension errors.

Guarantees (E, P, B, K). Index regrouping is exact. Array comparison certificates are tolerance-qualified. `PolyphaseRefinement`, `LatticePolyphaseRefinement`, and `PolyphaseRefinementTower` have version-1 codecs. NumPy stores signal records. Work and memory are linear in materialized sample count plus channel bookkeeping.

Evidence and links (V, L). Signal, arithmetic, rectangular-lattice, and portable-tower semantic cases cover this family. Chapters 12 through 14 give direct-fine equivalence, alias, and failure examples. Related symbols include analysis/synthesis and `refinement_children`.

A.5 · refine_lattice_polyphase

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.refine_lattice_polyphase`. Name-preserving aliases `komorebi_math.v1.refine_lattice_polyphase` and `komorebi_math.api.v1.refine_lattice_polyphase`. Tier `[STABLE]`. Manifest kind `function`. Reflected implementation `komorebi_math.refinement.polyphase.refine_lattice_polyphase`. Profile `A5-R1`. Deliberate cross-name alias: `komorebi_math.stable.lattice_polyphase_refine`. Both export cards remain present.

Manifest signature (s).

```
(coordinates: 'LatticePolyphaseCoordinates', fine_strides: 'Iterable[int]', *, copy: 'bool' = True) ->
    ↪ 'LatticePolyphaseCoordinates'
```

Definition. Refine rectangular lattice coordinates to componentwise finer strides.

Contract (D, C, H, R, X). One-dimensional or rectangular polyphase coordinates plus divisible coarse/fine orders map to refined or coarsened coordinate records. Orders and strides must be positive and componentwise divisible, shapes must match, and certificate tolerances must be nonnegative. Results preserve the supported coordinate-record variant. Invalid relations raise validation or dimension errors.

Guarantees (E, P, B, K). Index regrouping is exact. Array comparison certificates are tolerance-qualified. `Polyphase Refinement`, `LatticePolyphaseRefinement`, and `PolyphaseRefinementTower` have version-1 codecs. NumPy stores signal records. Work and memory are linear in materialized sample count plus channel bookkeeping.

Evidence and links (V, L). Signal, arithmetic, rectangular-lattice, and portable-tower semantic cases cover this family. Chapters 12 through 14 give direct-fine equivalence, alias, and failure examples. Related symbols include analysis/synthesis and `refinement_children`.

A.5 · `refine_polyphase`

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.refine_polyphase`. Name-preserving aliases `komorebi_math.v1.refine_polyphase` and `komorebi_math.api.v1.refine_polyphase`. Tier `[STABLE]`. Manifest kind `function`. Reflected implementation `komorebi_math.refinement.polyphase.refine_polyphase`. Profile `A5-R1`. Deliberate cross-name alias: `komorebi_math.stable.polyphase_refine`. Both export cards remain present.

Manifest signature (s).

```
(coordinates: 'PolyphaseCoordinateRecord', fine_order: 'int', *, copy: 'bool' = True) ->
    ↪ 'PolyphaseCoordinateRecord'
```

Definition. Refine any supported one-dimensional coordinate record to `fine_order`.

Contract (D, C, H, R, X). One-dimensional or rectangular polyphase coordinates plus divisible coarse/fine orders map to refined or coarsened coordinate records. Orders and strides must be positive and componentwise divisible, shapes must match, and certificate tolerances must be nonnegative. Results preserve the supported coordinate-record variant. Invalid relations raise validation or dimension errors.

Guarantees (E, P, B, K). Index regrouping is exact. Array comparison certificates are tolerance-qualified. `Polyphase Refinement`, `LatticePolyphaseRefinement`, and `PolyphaseRefinementTower` have version-1 codecs. NumPy stores signal records. Work and memory are linear in materialized sample count plus channel bookkeeping.

Evidence and links (V, L). Signal, arithmetic, rectangular-lattice, and portable-tower semantic cases cover this family. Chapters 12 through 14 give direct-fine equivalence, alias, and failure examples. Related symbols include analysis/synthesis and `refinement_children`.

A.5 · `residue_analyze`

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.residue_analyze`. Name-preserving aliases `komorebi_math.v1.residue_analyze` and `komorebi_math.api.v1.residue_analyze`. Tier `[STABLE]`. Manifest kind `function`. Reflected implementation `komorebi_math.refinement.residue.residue_analyze`. Profile `A5-Z1`.

Manifest signature (s).

```
(values: 'Any', modulus: 'int') -> 'ResidueCoordinates'
```

Definition. Split a one dimensional sequence into residue channels.

Contract (D, C, H, R, X). A finite sequence and positive modulus map to ordinary residue channels. Nested residue data map to the product modulus and back. Channel counts, dimensions, lengths, and residue ranges must agree. Results are `ResidueCoordinates`, combined addresses, or reconstructed arrays. Invalid inputs raise validation or dimension errors.

Guarantees (E, P, B, K). Address laws are exact. NumPy reconstruction preserves supplied values subject to array dtype. `ResidueCoordinates` has codec `refinement.residue_coordinates` version 1. Analysis and synthesis are linear in sequence length. Nested addressing is constant-time integer arithmetic.

Evidence and links (V, L). Chapters 13 through 14 verify CRT-style addresses, nesting, synthesis, and failure cases. Stable arithmetic/refinement semantic cases cover roundtrips. Related symbols are arithmetic and signal polyphase functions.

A.5 · `residue_synthesize`

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.residue_synthesize`. Name-preserving aliases `komorebi_math.v1.residue_synthesize` and `komorebi_math.api.v1.residue_synthesize`. Tier `[STABLE]`. Manifest kind `function`. Reflected implementation `komorebi_math.refinement.residue.residue_synthesize`. Profile `A5-Z1`.

Manifest signature (s).

```
(channels: 'ResidueCoordinates | tuple[Any, ...]', *, length: 'int | None' = None) -> 'np.ndarray'
```

Definition. Reconstruct a sequence from residue channels.

Contract (D, C, H, R, X). A finite sequence and positive modulus map to ordinary residue channels. Nested residue data map to the product modulus and back. Channel counts, dimensions, lengths, and residue ranges must agree. Results are `ResidueCoordinates`, combined addresses, or reconstructed arrays. Invalid inputs raise validation or dimension errors.

Guarantees (E, P, B, K). Address laws are exact. NumPy reconstruction preserves supplied values subject to array dtype. `ResidueCoordinates` has codec `refinement.residue_coordinates` version 1. Analysis and synthesis are linear in sequence length. Nested addressing is constant-time integer arithmetic.

Evidence and links (V, L). Chapters 13 through 14 verify CRT-style addresses, nesting, synthesis, and failure cases. Stable arithmetic/refinement semantic cases cover roundtrips. Related symbols are arithmetic and signal polyphase functions.

A.6 Generating series

A.6 · `FormalSeries`

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.FormalSeries`. Name-preserving aliases `komorebi_math.v1.FormalSeries` and `komorebi_math.api.v1.FormalSeries`. Tier `[STABLE]`. Manifest kind `class`. Reflected implementation `komorebi_math.generating.series.FormalSeries`. Profile `A6-G1`.

Manifest signature (s).

```
(basis: 'WeightedMonomialBasis', coefficients: 'Iterable[int | Fraction]') -> 'None'
```

Definition. A finite exact series `sum a_n z^n/gamma_n`.

Contract (D, C, H, R, X). A positive factorial power or ordinary-basis request produces a weighted monomial basis. A basis plus finite exact coefficients produces a `FormalSeries`. Orders and coefficients must satisfy the exact scalar and finite-iteration contracts. Results are basis descriptors or finite series. Invalid powers or incompatible bases fail validation.

Guarantees (E, P, B, K). Finite coefficient algebra is exact. `FormalSeries` has codec `generating.formal_series` version 1, while returned weighted bases use `generating.weighted_monomial_basis` version 1. Base exact arithmetic is sufficient. Convolution cost is quadratic in truncation length unless a method documents otherwise.

Evidence and links (V, L). The factorial-power product semantic case and Chapter 21 verify basis construction, multiplication, persistence, and failures. Related symbols are generating-function recipes in Chapters 21 through 25.

A.6 · factorial_power_basis

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.factorial_power_basis`. Name-preserving aliases `komorebi_math.v1.factorial_power_basis` and `komorebi_math.api.v1.factorial_power_basis`. Tier `[STABLE]`. Manifest kind `function`. Reflected implementation `komorebi_math.generating.basis.factorial_power_basis`. Profile `A6-G1`.

Manifest signature (s).

```
(power: 'int', *, name: 'str | None' = None) -> 'WeightedMonomialBasis'
```

Definition. Return `z^n/(n!)^power` for a positive integer power.

Contract (D, C, H, R, X). A positive factorial power or ordinary-basis request produces a weighted monomial basis. A basis plus finite exact coefficients produces a `FormalSeries`. Orders and coefficients must satisfy the exact scalar and finite-iteration contracts. Results are basis descriptors or finite series. Invalid powers or incompatible bases fail validation.

Guarantees (E, P, B, K). Finite coefficient algebra is exact. `FormalSeries` has codec `generating.formal_series` version 1, while returned weighted bases use `generating.weighted_monomial_basis` version 1. Base exact arithmetic is sufficient. Convolution cost is quadratic in truncation length unless a method documents otherwise.

Evidence and links (V, L). The factorial-power product semantic case and Chapter 21 verify basis construction, multiplication, persistence, and failures. Related symbols are generating-function recipes in Chapters 21 through 25.

A.6 · ordinary_generating_basis

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.ordinary_generating_basis`. Name-preserving aliases `komorebi_math.v1.ordinary_generating_basis` and `komorebi_math.api.v1.ordinary_generating_basis`. Tier `[STABLE]`. Manifest kind `function`. Reflected implementation `komorebi_math.generating.basis.ordinary_basis`. Profile `A6-G1`.

Manifest signature (s).

```
(*, name: 'str' = 'ordinary') -> 'WeightedMonomialBasis'
```

Definition. Return the ordinary generating-function basis `z^n`.

Contract (D, C, H, R, X). A positive factorial power or ordinary-basis request produces a weighted monomial basis. A basis plus finite exact coefficients produces a `FormalSeries`. Orders and coefficients must satisfy the exact scalar and finite-iteration contracts. Results are basis descriptors or finite series. Invalid powers or incompatible bases fail validation.

Guarantees (E, P, B, K). Finite coefficient algebra is exact. `FormalSeries` has codec `generating.formal_series` version 1, while returned weighted bases use `generating.weighted_monomial_basis` version 1. Base exact arithmetic is sufficient. Convolution cost is quadratic in truncation length unless a method documents otherwise.

Evidence and links (V, L). The factorial-power product semantic case and Chapter 21 verify basis construction, multiplication, persistence, and failures. Related symbols are generating-function recipes in Chapters 21 through 25.

A.7 Signal polyphase

A.7 · `LatticePolyphaseCoordinates`

Identity (`q, a, t, k`). Canonical manifest name `komorebi_math.stable.LatticePolyphaseCoordinates`. Name-preserving aliases `komorebi_math.v1.LatticePolyphaseCoordinates` and `komorebi_math.api.v1.LatticePolyphaseCoordinates`. Tier `[STABLE]`. Manifest kind `class`. Reflected implementation `komorebi_math.signals.polyphase.LatticePolyphaseCoordinates`. Profile `A7-S1`.

Manifest signature (`s`).

```
(channels: 'dict[tuple[int, ...], np.ndarray]', strides: 'tuple[int, ...]', original_shape: 'tuple[int, ...]') ->
    ↪ None
```

Definition. Rectangular lattice residue channels of a finite multidimensional array.

Contract (`D, C, H, R, X`). A finite NumPy-compatible signal and order/axis or rectangular strides map to residue-channel coordinates. Synthesis maps those records back to the original shape. Orders, axes, strides, channel shapes, and stored original shape must agree. Results are coordinate records, reconstructed arrays, projections, child-index tuples, or certified roundtrips. Invalid shapes raise validation or dimension errors.

Guarantees (`E, P, B, K`). Index splitting and child arithmetic are exact. Floating roundtrip certificates are numerical with explicit tolerance. `PolyphaseCoordinates` and `LatticePolyphaseCoordinates` have version-1 codecs. NumPy is required. Analysis, synthesis, and projection are linear in sample count with channel-view/copy behavior controlled by arguments.

Evidence and links (`V, L`). The signal and rectangular-lattice semantic cases and Chapters 14 and 35 verify roundtrips, refinement children, projections, aliases, and adversarial shapes. Related symbols are A.5 refinement and arithmetic polyphase cards.

A.7 · `PolyphaseCoordinates`

Identity (`q, a, t, k`). Canonical manifest name `komorebi_math.stable.PolyphaseCoordinates`. Name-preserving aliases `komorebi_math.v1.PolyphaseCoordinates` and `komorebi_math.api.v1.PolyphaseCoordinates`. Tier `[STABLE]`. Manifest kind `class`. Reflected implementation `komorebi_math.signals.polyphase.PolyphaseCoordinates`. Profile `A7-S1`.

Manifest signature (`s`).

```
(channels: 'tuple[np.ndarray, ...]', order: 'int', axis: 'int', original_shape: 'tuple[int, ...]') -> None
```

Definition. Residue channels of a finite array along one axis.

Contract (`D, C, H, R, X`). A finite NumPy-compatible signal and order/axis or rectangular strides map to residue-channel coordinates. Synthesis maps those records back to the original shape. Orders, axes, strides, channel shapes, and stored original shape must agree. Results are coordinate records, reconstructed arrays, projections, child-index tuples, or certified roundtrips. Invalid shapes raise validation or dimension errors.

Guarantees (`E, P, B, K`). Index splitting and child arithmetic are exact. Floating roundtrip certificates are numerical with explicit tolerance. `PolyphaseCoordinates` and `LatticePolyphaseCoordinates` have version-1 codecs. NumPy is required. Analysis, synthesis, and projection are linear in sample count with channel-view/copy behavior controlled by arguments.

Evidence and links (`V, L`). The signal and rectangular-lattice semantic cases and Chapters 14 and 35 verify roundtrips, refinement children, projections, aliases, and adversarial shapes. Related symbols are A.5 refinement and arithmetic polyphase cards.

A.7 · PolyphaseSystem

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.PolyphaseSystem`. Name-preserving aliases `komorebi_math.v1.PolyphaseSystem` and `komorebi_math.api.v1.PolyphaseSystem`. Tier `[STABLE]`. Manifest kind `class`. Reflected implementation `komorebi_math.signals.polyphase.PolyphaseSystem`. Profile `A7-S1`.

Manifest signature (s).

```
(order: 'int', axis: 'int' = -1) -> None
```

Definition. Reusable one axis polyphase analysis and synthesis system.

Contract (D, C, H, R, X). A finite NumPy-compatible signal and order/axis or rectangular strides map to residue-channel coordinates. Synthesis maps those records back to the original shape. Orders, axes, strides, channel shapes, and stored original shape must agree. Results are coordinate records, reconstructed arrays, projections, child-index tuples, or certified roundtrips. Invalid shapes raise validation or dimension errors.

Guarantees (E, P, B, K). Index splitting and child arithmetic are exact. Floating roundtrip certificates are numerical with explicit tolerance. `PolyphaseCoordinates` and `LatticePolyphaseCoordinates` have version-1 codecs. NumPy is required. Analysis, synthesis, and projection are linear in sample count with channel-view/copy behavior controlled by arguments.

Evidence and links (V, L). The signal and rectangular-lattice semantic cases and Chapters 14 and 35 verify roundtrips, refinement children, projections, aliases, and adversarial shapes. Related symbols are A.5 refinement and arithmetic polyphase cards.

A.7 · lattice_polyphase_analyze

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.lattice_polyphase_analyze`. Name-preserving aliases `komorebi_math.v1.lattice_polyphase_analyze` and `komorebi_math.api.v1.lattice_polyphase_analyze`. Tier `[STABLE]`. Manifest kind `function`. Reflected implementation `komorebi_math.signals.polyphase.lattice_polyphase_analyze`. Profile `A7-S1`.

Manifest signature (s).

```
(signal: 'Any', strides: 'Iterable[int]', *, copy: 'bool' = True) -> 'LatticePolyphaseCoordinates'
```

Definition. Split an array into channels indexed by a rectangular lattice quotient.

Contract (D, C, H, R, X). A finite NumPy-compatible signal and order/axis or rectangular strides map to residue-channel coordinates. Synthesis maps those records back to the original shape. Orders, axes, strides, channel shapes, and stored original shape must agree. Results are coordinate records, reconstructed arrays, projections, child-index tuples, or certified roundtrips. Invalid shapes raise validation or dimension errors.

Guarantees (E, P, B, K). Index splitting and child arithmetic are exact. Floating roundtrip certificates are numerical with explicit tolerance. `PolyphaseCoordinates` and `LatticePolyphaseCoordinates` have version-1 codecs. NumPy is required. Analysis, synthesis, and projection are linear in sample count with channel-view/copy behavior controlled by arguments.

Evidence and links (V, L). The signal and rectangular-lattice semantic cases and Chapters 14 and 35 verify roundtrips, refinement children, projections, aliases, and adversarial shapes. Related symbols are A.5 refinement and arithmetic polyphase cards.

A.7 · lattice_polyphase_synthesize

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.lattice_polyphase_synthesize`. Name-preserving aliases `komorebi_math.v1.lattice_polyphase_synthesize` and `komorebi_math.api.v1.lattice_polyphase_synthesize`. Tier `[STABLE]`. Manifest kind `function`. Reflected implementation `komorebi_math.signals.polyphase.lattice_polyphase_synthesize`. Profile `A7-S1`.

Manifest signature (`s`).

```
(coordinates: 'LatticePolyphaseCoordinates') -> 'np.ndarray'
```

Definition. Reconstruct an array from rectangular lattice packet coordinates.

Contract (`D, C, H, R, X`). A finite NumPy-compatible signal and order/axis or rectangular strides map to residue-channel coordinates. Synthesis maps those records back to the original shape. Orders, axes, strides, channel shapes, and stored original shape must agree. Results are coordinate records, reconstructed arrays, projections, child-index tuples, or certified roundtrips. Invalid shapes raise validation or dimension errors.

Guarantees (`E, P, B, K`). Index splitting and child arithmetic are exact. Floating roundtrip certificates are numerical with explicit tolerance. `PolyphaseCoordinates` and `LatticePolyphaseCoordinates` have version-1 codecs. NumPy is required. Analysis, synthesis, and projection are linear in sample count with channel-view/copy behavior controlled by arguments.

Evidence and links (`V, L`). The signal and rectangular-lattice semantic cases and Chapters 14 and 35 verify roundtrips, refinement children, projections, aliases, and adversarial shapes. Related symbols are A.5 refinement and arithmetic polyphase cards.

A.7 · `polyphase_roundtrip_certificate`

Identity (`q, a, t, k`). Canonical manifest name `komorebi_math.stable.polyphase_roundtrip_certificate`. Name-preserving aliases `komorebi_math.v1.polyphase_roundtrip_certificate` and `komorebi_math.api.v1.polyphase_roundtrip_certificate`. Tier `[STABLE]`. Manifest kind `function`. Reflected implementation `komorebi_math.signals.polyphase.polyphase_roundtrip_certificate`. Profile `A7-S1`.

Manifest signature (`s`).

```
(signal: 'Any', order: 'int', *, axis: 'int' = -1, tolerance: 'float' = 1e-12) -> 'ComputationResult[np.ndarray]'
```

Definition. Analyze and reconstruct a finite signal with a numerical certificate.

Contract (`D, C, H, R, X`). A finite NumPy-compatible signal and order/axis or rectangular strides map to residue-channel coordinates. Synthesis maps those records back to the original shape. Orders, axes, strides, channel shapes, and stored original shape must agree. Results are coordinate records, reconstructed arrays, projections, child-index tuples, or certified roundtrips. Invalid shapes raise validation or dimension errors.

Guarantees (`E, P, B, K`). Index splitting and child arithmetic are exact. Floating roundtrip certificates are numerical with explicit tolerance. `PolyphaseCoordinates` and `LatticePolyphaseCoordinates` have version-1 codecs. NumPy is required. Analysis, synthesis, and projection are linear in sample count with channel-view/copy behavior controlled by arguments.

Evidence and links (`V, L`). The signal and rectangular-lattice semantic cases and Chapters 14 and 35 verify roundtrips, refinement children, projections, aliases, and adversarial shapes. Related symbols are A.5 refinement and arithmetic polyphase cards.

A.7 · `refinement_children`

Identity (`q, a, t, k`). Canonical manifest name `komorebi_math.stable.refinement_children`. Name-preserving aliases `komorebi_math.v1.refinement_children` and `komorebi_math.api.v1.refinement_children`. Tier `[STABLE]`. Manifest kind `function`. Reflected implementation `komorebi_math.signals.polyphase.refinement_children`. Profile `A7-S1`.

Manifest signature (`s`).

```
(coarse_order: 'int', fine_order: 'int', residue: 'int') -> 'tuple[int, ...]'
```

Definition. Return fine residues whose reduction modulo the coarse order is `residue`.

Contract (`D, C, H, R, X`). A finite NumPy-compatible signal and order/axis or rectangular strides map to residue-channel coordinates. Synthesis maps those records back to the original shape. Orders, axes, strides, channel shapes,

and stored original shape must agree. Results are coordinate records, reconstructed arrays, projections, child-index tuples, or certified roundtrips. Invalid shapes raise validation or dimension errors.

Guarantees (E, P, B, K). Index splitting and child arithmetic are exact. Floating roundtrip certificates are numerical with explicit tolerance. `PolyphaseCoordinates` and `LatticePolyphaseCoordinates` have version-1 codecs. NumPy is required. Analysis, synthesis, and projection are linear in sample count with channel-view/copy behavior controlled by arguments.

Evidence and links (V, L). The signal and rectangular-lattice semantic cases and Chapters 14 and 35 verify roundtrips, refinement children, projections, aliases, and adversarial shapes. Related symbols are A.5 refinement and arithmetic polyphase cards.

A.7 · `residue_project`

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.residue_project`. Name-preserving aliases `komorebi_math.v1.residue_project` and `komorebi_math.api.v1.residue_project`. Tier `[STABLE]`. Manifest kind `function`. Reflected implementation `komorebi_math.signals.polyphase.residue_project`. Profile `A7-S1`.

Manifest signature (s).

```
(signal: 'Any', order: 'int', residue: 'int', *, axis: 'int' = -1) -> 'np.ndarray'
```

Definition. Return the full shaped signal supported on one residue class.

Contract (D, C, H, R, X). A finite NumPy-compatible signal and order/axis or rectangular strides map to residue-channel coordinates. Synthesis maps those records back to the original shape. Orders, axes, strides, channel shapes, and stored original shape must agree. Results are coordinate records, reconstructed arrays, projections, child-index tuples, or certified roundtrips. Invalid shapes raise validation or dimension errors.

Guarantees (E, P, B, K). Index splitting and child arithmetic are exact. Floating roundtrip certificates are numerical with explicit tolerance. `PolyphaseCoordinates` and `LatticePolyphaseCoordinates` have version-1 codecs. NumPy is required. Analysis, synthesis, and projection are linear in sample count with channel-view/copy behavior controlled by arguments.

Evidence and links (V, L). The signal and rectangular-lattice semantic cases and Chapters 14 and 35 verify roundtrips, refinement children, projections, aliases, and adversarial shapes. Related symbols are A.5 refinement and arithmetic polyphase cards.

A.7 · `signal_polyphase_analyze`

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.signal_polyphase_analyze`. Name-preserving aliases `komorebi_math.v1.signal_polyphase_analyze` and `komorebi_math.api.v1.signal_polyphase_analyze`. Tier `[STABLE]`. Manifest kind `function`. Reflected implementation `komorebi_math.signals.polyphase.polyphase_analyze`. Profile `A7-S1`.

Manifest signature (s).

```
(signal: 'Any', order: 'int', *, axis: 'int' = -1, copy: 'bool' = True) -> 'PolyphaseCoordinates'
```

Definition. Split a finite array into residue channels along `axis`.

Contract (D, C, H, R, X). A finite NumPy-compatible signal and order/axis or rectangular strides map to residue-channel coordinates. Synthesis maps those records back to the original shape. Orders, axes, strides, channel shapes, and stored original shape must agree. Results are coordinate records, reconstructed arrays, projections, child-index tuples, or certified roundtrips. Invalid shapes raise validation or dimension errors.

Guarantees (E, P, B, K). Index splitting and child arithmetic are exact. Floating roundtrip certificates are numerical with explicit tolerance. `PolyphaseCoordinates` and `LatticePolyphaseCoordinates` have version-1 codecs. NumPy is required. Analysis, synthesis, and projection are linear in sample count with channel-view/copy behavior controlled by arguments.

Evidence and links (V, L). The signal and rectangular-lattice semantic cases and Chapters 14 and 35 verify roundtrips, refinement children, projections, aliases, and adversarial shapes. Related symbols are A.5 refinement and arithmetic polyphase cards.

A.7 · signal_polyphase_synthesize

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.signal_polyphase_synthesize`. Name-preserving aliases `komorebi_math.v1.signal_polyphase_synthesize` and `komorebi_math.api.v1.signal_polyphase_synthesize`. Tier `[STABLE]`. Manifest kind `function`. Reflected implementation `komorebi_math.signals.polyphase.polyphase_synthesize`. Profile `A7-S1`.

Manifest signature (s).

```
(coordinates: 'PolyphaseCoordinates') -> 'np.ndarray'
```

Definition. Reconstruct a finite array from polyphase coordinates.

Contract (D, C, H, R, X). A finite NumPy-compatible signal and order/axis or rectangular strides map to residue-channel coordinates. Synthesis maps those records back to the original shape. Orders, axes, strides, channel shapes, and stored original shape must agree. Results are coordinate records, reconstructed arrays, projections, child-index tuples, or certified roundtrips. Invalid shapes raise validation or dimension errors.

Guarantees (E, P, B, K). Index splitting and child arithmetic are exact. Floating roundtrip certificates are numerical with explicit tolerance. `PolyphaseCoordinates` and `LatticePolyphaseCoordinates` have version-1 codecs. NumPy is required. Analysis, synthesis, and projection are linear in sample count with channel-view/copy behavior controlled by arguments.

Evidence and links (V, L). The signal and rectangular-lattice semantic cases and Chapters 14 and 35 verify roundtrips, refinement children, projections, aliases, and adversarial shapes. Related symbols are A.5 refinement and arithmetic polyphase cards.

A.8 Document model and portable persistence

A.8 · DocumentMetadata

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.DocumentMetadata`. Name-preserving aliases `komorebi_math.v1.DocumentMetadata` and `komorebi_math.api.v1.DocumentMetadata`. Tier `[STABLE]`. Manifest kind `class`. Reflected implementation `komorebi_math.export.model.DocumentMetadata`. Profile `A8-D1`.

Manifest signature (s).

```
(title: 'str' = 'Komorebi Math Export', author: 'str | None' = None, date: 'str | None' = None, abstract: 'str |
    ↪ None' = None, keywords: 'tuple[str, ...]' = ()) -> None
```

Definition. Metadata used by Markdown, HTML, and LaTeX document renderers.

Contract (D, C, H, R, X). Metadata and typed prose/equation blocks map to a renderer-independent `MathDocument`. Text, labels, numbering flags, and metadata fields must have the declared types. Results are mutable document assemblies or immutable block records. Malformed construction raises ordinary dataclass/type errors and renderer validation errors.

Guarantees (E, P, B, K). Document structure is exact editorial data, not a mathematical certificate. These four records have no frozen portable codec. Their supported public export is rendering. The base model uses the standard library and storage is linear in block and text size.

Evidence and links (V, L). The portable-document-rendering semantic case and Chapters 59 through 60 exercise assembly and output. Related symbols are `render_markdown`, `render_latex`, and the report-builder APIs outside the frozen stable tier.

A.8 · Equation

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.Equation`. Name-preserving aliases `komorebi_math.v1.Equation` and `komorebi_math.api.v1.Equation`. Tier `[STABLE]`. Manifest kind `class`. Reflected implementation `komorebi_math.export.model.Equation`. Profile `A8-D1`.

Manifest signature (s).

```
(latex: 'str', label: 'str | None' = None, numbered: 'bool' = False, description: 'str | None' = None) -> None
```

Definition. One LaTeX mathematical expression.

Contract (D, C, H, R, X). Metadata and typed prose/equation blocks map to a renderer-independent `MathDocument`. Text, labels, numbering flags, and metadata fields must have the declared types. Results are mutable document assemblies or immutable block records. Malformed construction raises ordinary dataclass/type errors and renderer validation errors.

Guarantees (E, P, B, K). Document structure is exact editorial data, not a mathematical certificate. These four records have no frozen portable codec. Their supported public export is rendering. The base model uses the standard library and storage is linear in block and text size.

Evidence and links (V, L). The portable-document-rendering semantic case and Chapters 59 through 60 exercise assembly and output. Related symbols are `render_markdown`, `render_latex`, and the report-builder APIs outside the frozen stable tier.

A.8 · MathDocument

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.MathDocument`. Name-preserving aliases `komorebi_math.v1.MathDocument` and `komorebi_math.api.v1.MathDocument`. Tier `[STABLE]`. Manifest kind `class`. Reflected implementation `komorebi_math.export.model.MathDocument`. Profile `A8-D1`.

Manifest signature (s).

```
(metadata: 'DocumentMetadata' = <factory>, blocks: 'list[DocumentBlock]' = <factory>) -> None
```

Definition. A renderer independent mathematical document.

Contract (D, C, H, R, X). Metadata and typed prose/equation blocks map to a renderer-independent `MathDocument`. Text, labels, numbering flags, and metadata fields must have the declared types. Results are mutable document assemblies or immutable block records. Malformed construction raises ordinary dataclass/type errors and renderer validation errors.

Guarantees (E, P, B, K). Document structure is exact editorial data, not a mathematical certificate. These four records have no frozen portable codec. Their supported public export is rendering. The base model uses the standard library and storage is linear in block and text size.

Evidence and links (V, L). The portable-document-rendering semantic case and Chapters 59 through 60 exercise assembly and output. Related symbols are `render_markdown`, `render_latex`, and the report-builder APIs outside the frozen stable tier.

A.8 · Paragraph

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.Paragraph`. Name-preserving aliases `komorebi_math.v1.Paragraph` and `komorebi_math.api.v1.Paragraph`. Tier `[STABLE]`. Manifest kind `class`. Reflected implementation `komorebi_math.export.model.Paragraph`. Profile `A8-D1`.

Manifest signature (`s`).

```
(text: 'str') -> None
```

Definition. One prose paragraph whose text is escaped by each renderer.

Contract (`D, C, H, R, X`). Metadata and typed prose/equation blocks map to a renderer-independent `MathDocument`. Text, labels, numbering flags, and metadata fields must have the declared types. Results are mutable document assemblies or immutable block records. Malformed construction raises ordinary dataclass/type errors and renderer validation errors.

Guarantees (`E, P, B, K`). Document structure is exact editorial data, not a mathematical certificate. These four records have no frozen portable codec. Their supported public export is rendering. The base model uses the standard library and storage is linear in block and text size.

Evidence and links (`V, L`). The portable-document-rendering semantic case and Chapters 59 through 60 exercise assembly and output. Related symbols are `render_markdown`, `render_latex`, and the report-builder APIs outside the frozen stable tier.

A.8 · `PortableCodec`

Identity (`q, a, t, k`). Canonical manifest name `komorebi_math.stable.PortableCodec`. Name-preserving aliases `komorebi_math.v1.PortableCodec` and `komorebi_math.api.v1.PortableCodec`. Tier `[STABLE]`. Manifest kind `class`. Reflected implementation `komorebi_math.export.persistence.PortableCodec`. Profile `A8-P1`.

Manifest signature (`s`).

```
(tag: 'str', version: 'int', type_: 'type[Any]', encoder: 'Encoder', decoder: 'Decoder') -> None
```

Definition. One explicit type codec in the portable persistence registry.

Contract (`D, C, H, R, X`). Registered codecs map supported objects to versioned payload trees. Envelope functions map payloads to canonical digest-checked JSON or atomic files and back. Tags must be unique, migrations adjacent, schemas exact, depths within budget, digests valid, and overwrite policy explicit. Results are codecs, migrations, payloads, text, paths, or decoded values. Schema, tamper, resource, path, and overwrite failures are explicit.

Guarantees (`E, P, B, K`). Canonical encoding and SHA-256 comparison are exact. Portability exists only for registered types. Unsupported values are rejected. The base route uses JSON, hashing, and filesystem primitives with NumPy used by array codecs. Work is linear in encoded payload size, bounded by `max_depth`, with atomic file replacement for writes.

Evidence and links (`V, L`). Chapter 61 provides envelope, tamper, depth, overwrite, and roundtrip examples. The persistence contract checks all 51 codecs and one migration. Related symbols are A.9 codec-contract records and `Certificate`.

A.8 · `PortableMigration`

Identity (`q, a, t, k`). Canonical manifest name `komorebi_math.stable.PortableMigration`. Name-preserving aliases `komorebi_math.v1.PortableMigration` and `komorebi_math.api.v1.PortableMigration`. Tier `[STABLE]`. Manifest kind `class`. Reflected implementation `komorebi_math.export.persistence.PortableMigration`. Profile `A8-P1`.

Manifest signature (`s`).

```
(tag: 'str', from_version: 'int', to_version: 'int', migrator: 'Migrator') -> None
```

Definition. One explicit, adjacent codec-data migration.

Contract (`D, C, H, R, X`). Registered codecs map supported objects to versioned payload trees. Envelope functions map payloads to canonical digest-checked JSON or atomic files and back. Tags must be unique, migrations adjacent, schemas exact, depths within budget, digests valid, and overwrite policy explicit. Results are codecs, migrations, payloads, text, paths, or decoded values. Schema, tamper, resource, path, and overwrite failures are explicit.

Guarantees (E, P, B, K). Canonical encoding and SHA-256 comparison are exact. Portability exists only for registered types. Unsupported values are rejected. The base route uses JSON, hashing, and filesystem primitives with NumPy used by array codecs. Work is linear in encoded payload size, bounded by `max_depth`, with atomic file replacement for writes.

Evidence and links (V, L). Chapter 61 provides envelope, tamper, depth, overwrite, and roundtrip examples. The persistence contract checks all 51 codecs and one migration. Related symbols are A.9 codec-contract records and `Certificate`.

A.8 · `dumps_portable`

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.dumps_portable`. Name-preserving aliases `komorebi_math.v1.dumps_portable` and `komorebi_math.api.v1.dumps_portable`. Tier `[STABLE]`. Manifest kind `function`. Reflected implementation `komorebi_math.export.persistence.dumps_portable`. Profile `A8-P1`.

Manifest signature (s).

```
(value: 'Any', *, indent: 'int | None' = 2, max_depth: 'int' = 128) -> 'str'
```

Definition. Serialize one value into a complete digest-checked JSON envelope.

Contract (D, C, H, R, X). Registered codecs map supported objects to versioned payload trees. Envelope functions map payloads to canonical digest-checked JSON or atomic files and back. Tags must be unique, migrations adjacent, schemas exact, depths within budget, digests valid, and overwrite policy explicit. Results are codecs, migrations, payloads, text, paths, or decoded values. Schema, tamper, resource, path, and overwrite failures are explicit.

Guarantees (E, P, B, K). Canonical encoding and SHA-256 comparison are exact. Portability exists only for registered types. Unsupported values are rejected. The base route uses JSON, hashing, and filesystem primitives with NumPy used by array codecs. Work is linear in encoded payload size, bounded by `max_depth`, with atomic file replacement for writes.

Evidence and links (V, L). Chapter 61 provides envelope, tamper, depth, overwrite, and roundtrip examples. The persistence contract checks all 51 codecs and one migration. Related symbols are A.9 codec-contract records and `Certificate`.

A.8 · `from_portable_data`

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.from_portable_data`. Name-preserving aliases `komorebi_math.v1.from_portable_data` and `komorebi_math.api.v1.from_portable_data`. Tier `[STABLE]`. Manifest kind `function`. Reflected implementation `komorebi_math.export.persistence.from_portable_data`. Profile `A8-P1`.

Manifest signature (s).

```
(value: 'Any', *, max_depth: 'int' = 128) -> 'Any'
```

Definition. Decode one previously encoded portable payload tree.

Contract (D, C, H, R, X). Registered codecs map supported objects to versioned payload trees. Envelope functions map payloads to canonical digest-checked JSON or atomic files and back. Tags must be unique, migrations adjacent, schemas exact, depths within budget, digests valid, and overwrite policy explicit. Results are codecs, migrations, payloads, text, paths, or decoded values. Schema, tamper, resource, path, and overwrite failures are explicit.

Guarantees (E, P, B, K). Canonical encoding and SHA-256 comparison are exact. Portability exists only for registered types. Unsupported values are rejected. The base route uses JSON, hashing, and filesystem primitives with NumPy used by array codecs. Work is linear in encoded payload size, bounded by `max_depth`, with atomic file replacement for writes.

Evidence and links (V, L). Chapter 61 provides envelope, tamper, depth, overwrite, and roundtrip examples. The persistence contract checks all 51 codecs and one migration. Related symbols are A.9 codec-contract records and `Certificate`.

A.8 · load_portable

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.load_portable`. Name-preserving aliases `komorebi_math.v1.load_portable` and `komorebi_math.api.v1.load_portable`. Tier `[STABLE]`. Manifest kind `function`. Reflected implementation `komorebi_math.export.persistence.load_portable`. Profile `A8-P1`.

Manifest signature (s).

```
(path: 'str | os.PathLike[str]', *, max_depth: 'int' = 128) -> 'Any'
```

Definition. Read and validate a portable envelope from disk.

Contract (D, C, H, R, X). Registered codecs map supported objects to versioned payload trees. Envelope functions map payloads to canonical digest-checked JSON or atomic files and back. Tags must be unique, migrations adjacent, schemas exact, depths within budget, digests valid, and overwrite policy explicit. Results are codecs, migrations, payloads, text, paths, or decoded values. Schema, tamper, resource, path, and overwrite failures are explicit.

Guarantees (E, P, B, K). Canonical encoding and SHA-256 comparison are exact. Portability exists only for registered types. Unsupported values are rejected. The base route uses JSON, hashing, and filesystem primitives with NumPy used by array codecs. Work is linear in encoded payload size, bounded by `max_depth`, with atomic file replacement for writes.

Evidence and links (V, L). Chapter 61 provides envelope, tamper, depth, overwrite, and roundtrip examples. The persistence contract checks all 51 codecs and one migration. Related symbols are A.9 codec-contract records and `Certificate`.

A.8 · loads_portable

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.loads_portable`. Name-preserving aliases `komorebi_math.v1.loads_portable` and `komorebi_math.api.v1.loads_portable`. Tier `[STABLE]`. Manifest kind `function`. Reflected implementation `komorebi_math.export.persistence.loads_portable`. Profile `A8-P1`.

Manifest signature (s).

```
(text: 'str | bytes', *, max_depth: 'int' = 128) -> 'Any'
```

Definition. Validate and deserialize one complete portable JSON envelope.

Contract (D, C, H, R, X). Registered codecs map supported objects to versioned payload trees. Envelope functions map payloads to canonical digest-checked JSON or atomic files and back. Tags must be unique, migrations adjacent, schemas exact, depths within budget, digests valid, and overwrite policy explicit. Results are codecs, migrations, payloads, text, paths, or decoded values. Schema, tamper, resource, path, and overwrite failures are explicit.

Guarantees (E, P, B, K). Canonical encoding and SHA-256 comparison are exact. Portability exists only for registered types. Unsupported values are rejected. The base route uses JSON, hashing, and filesystem primitives with NumPy used by array codecs. Work is linear in encoded payload size, bounded by `max_depth`, with atomic file replacement for writes.

Evidence and links (V, L). Chapter 61 provides envelope, tamper, depth, overwrite, and roundtrip examples. The persistence contract checks all 51 codecs and one migration. Related symbols are A.9 codec-contract records and `Certificate`.

A.8 · portable_codecs

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.portable_codecs`. Name-preserving aliases `komorebi_math.v1.portable_codecs` and `komorebi_math.api.v1.portable_codecs`. Tier `[STABLE]`. Manifest kind `function`. Reflected implementation `komorebi_math.export.persistence.portable_codecs`. Profile `A8-P1`.

Manifest signature (s).

```
() -> 'tuple[PortableCodec, ...]'
```

Definition. Return the active codecs ordered by stable tag.

Contract (D, C, H, R, X). Registered codecs map supported objects to versioned payload trees. Envelope functions map payloads to canonical digest-checked JSON or atomic files and back. Tags must be unique, migrations adjacent, schemas exact, depths within budget, digests valid, and overwrite policy explicit. Results are codecs, migrations, payloads, text, paths, or decoded values. Schema, tamper, resource, path, and overwrite failures are explicit.

Guarantees (E, P, B, K). Canonical encoding and SHA-256 comparison are exact. Portability exists only for registered types. Unsupported values are rejected. The base route uses JSON, hashing, and filesystem primitives with NumPy used by array codecs. Work is linear in encoded payload size, bounded by `max_depth`, with atomic file replacement for writes.

Evidence and links (V, L). Chapter 61 provides envelope, tamper, depth, overwrite, and roundtrip examples. The persistence contract checks all 51 codecs and one migration. Related symbols are A.9 codec-contract records and `Certificate`.

A.8 · `portable_digest`

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.portable_digest`. Name-preserving aliases `komorebi_math.v1.portable_digest` and `komorebi_math.api.v1.portable_digest`. Tier `[STABLE]`. Manifest kind `function`. Reflected implementation `komorebi_math.export.persistence.portable_digest`. Profile `A8-P1`.

Manifest signature (s).

```
(value: 'Any', *, max_depth: 'int' = 128) -> 'str'
```

Definition. Return the canonical SHA256 digest of one encoded payload.

Contract (D, C, H, R, X). Registered codecs map supported objects to versioned payload trees. Envelope functions map payloads to canonical digest-checked JSON or atomic files and back. Tags must be unique, migrations adjacent, schemas exact, depths within budget, digests valid, and overwrite policy explicit. Results are codecs, migrations, payloads, text, paths, or decoded values. Schema, tamper, resource, path, and overwrite failures are explicit.

Guarantees (E, P, B, K). Canonical encoding and SHA-256 comparison are exact. Portability exists only for registered types. Unsupported values are rejected. The base route uses JSON, hashing, and filesystem primitives with NumPy used by array codecs. Work is linear in encoded payload size, bounded by `max_depth`, with atomic file replacement for writes.

Evidence and links (V, L). Chapter 61 provides envelope, tamper, depth, overwrite, and roundtrip examples. The persistence contract checks all 51 codecs and one migration. Related symbols are A.9 codec-contract records and `Certificate`.

A.8 · `portable_migrations`

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.portable_migrations`. Name-preserving aliases `komorebi_math.v1.portable_migrations` and `komorebi_math.api.v1.portable_migrations`. Tier `[STABLE]`. Manifest kind `function`. Reflected implementation `komorebi_math.export.persistence.portable_migrations`. Profile `A8-P1`.

Manifest signature (s).

```
() -> 'tuple[PortableMigration, ...]'
```

Definition. Return registered codec migrations in deterministic order.

Contract (D, C, H, R, X). Registered codecs map supported objects to versioned payload trees. Envelope functions map payloads to canonical digest-checked JSON or atomic files and back. Tags must be unique, migrations adjacent, schemas exact, depths within budget, digests valid, and overwrite policy explicit. Results are codecs, migrations, payloads, text, paths, or decoded values. Schema, tamper, resource, path, and overwrite failures are explicit.

Guarantees (E, P, B, K). Canonical encoding and SHA-256 comparison are exact. Portability exists only for registered types. Unsupported values are rejected. The base route uses JSON, hashing, and filesystem primitives with NumPy used by array codecs. Work is linear in encoded payload size, bounded by `max_depth`, with atomic file replacement for writes.

Evidence and links (V, L). Chapter 61 provides envelope, tamper, depth, overwrite, and roundtrip examples. The persistence contract checks all 51 codecs and one migration. Related symbols are A.9 codec-contract records and `Certificate`.

A.8 · render_latex

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.render_latex`. Name-preserving aliases `komorebi_math.v1.render_latex` and `komorebi_math.api.v1.render_latex`. Tier `[STABLE]`. Manifest kind `function`. Reflected implementation `komorebi_math.export.renderers.render_latex`. Profile `A8-R1`.

Manifest signature (s).

```
(document: 'MathDocument', *, options: 'LatexRenderOptions | None' = None) -> 'str'
```

Definition. Render a complete compilable LaTeX document.

Contract (D, C, H, R, X). A valid `MathDocument` maps to Markdown text or a complete LaTeX document. Content is escaped according to renderer context and LaTeX options. The result is text. Malformed blocks, unsupported content, or invalid renderer options raise validation/export errors.

Guarantees (E, P, B, K). Rendering is deterministic text generation for fixed inputs, not a proof of successful external compilation. It is not a portable codec. The stable renderer path is base-only. Runtime and memory are linear in emitted document size apart from escaping.

Evidence and links (V, L). The portable-document-rendering semantic case and Chapters 59 through 60 compare Markdown and LaTeX output. Related symbols are the four document-model records and external writer/compilation interfaces.

A.8 · render_markdown

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.render_markdown`. Name-preserving aliases `komorebi_math.v1.render_markdown` and `komorebi_math.api.v1.render_markdown`. Tier `[STABLE]`. Manifest kind `function`. Reflected implementation `komorebi_math.export.renderers.render_markdown`. Profile `A8-R1`.

Manifest signature (s).

```
(document: 'MathDocument') -> 'str'
```

Definition. Render a document as Markdown with TeX equations.

Contract (D, C, H, R, X). A valid `MathDocument` maps to Markdown text or a complete LaTeX document. Content is escaped according to renderer context and LaTeX options. The result is text. Malformed blocks, unsupported content, or invalid renderer options raise validation/export errors.

Guarantees (E, P, B, K). Rendering is deterministic text generation for fixed inputs, not a proof of successful external compilation. It is not a portable codec. The stable renderer path is base-only. Runtime and memory are linear in emitted document size apart from escaping.

Evidence and links (V, L). The portable-document-rendering semantic case and Chapters 59 through 60 compare Markdown and LaTeX output. Related symbols are the four document-model records and external writer/compilation interfaces.

A.8 · save_portable

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.save_portable`. Name-preserving aliases `komorebi_math.v1.save_portable` and `komorebi_math.api.v1.save_portable`. Tier `[STABLE]`. Manifest kind `function`. Reflected implementation `komorebi_math.export.persistence.save_portable`. Profile `A8-P1`.

Manifest signature (s).

```
(value: 'Any', path: 'str | os.PathLike[str]', *, overwrite: 'bool' = False, indent: 'int | None' = 2, max_depth:
   ↪  'int' = 128) -> 'Path'
```

Definition. Atomically write a portable envelope to disk.

Contract (D, C, H, R, X). Registered codecs map supported objects to versioned payload trees. Envelope functions map payloads to canonical digest-checked JSON or atomic files and back. Tags must be unique, migrations adjacent, schemas exact, depths within budget, digests valid, and overwrite policy explicit. Results are codecs, migrations, payloads, text, paths, or decoded values. Schema, tamper, resource, path, and overwrite failures are explicit.

Guarantees (E, P, B, K). Canonical encoding and SHA-256 comparison are exact. Portability exists only for registered types. Unsupported values are rejected. The base route uses JSON, hashing, and filesystem primitives with NumPy used by array codecs. Work is linear in encoded payload size, bounded by `max_depth`, with atomic file replacement for writes.

Evidence and links (V, L). Chapter 61 provides envelope, tamper, depth, overwrite, and roundtrip examples. The persistence contract checks all 51 codecs and one migration. Related symbols are A.9 codec-contract records and `Certificate`.

A.8 · to_portable_data

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.to_portable_data`. Name-preserving aliases `komorebi_math.v1.to_portable_data` and `komorebi_math.api.v1.to_portable_data`. Tier `[STABLE]`. Manifest kind `function`. Reflected implementation `komorebi_math.export.persistence.to_portable_data`. Profile `A8-P1`.

Manifest signature (s).

```
(value: 'Any', *, max_depth: 'int' = 128) -> 'Any'
```

Definition. Encode one value into the versioned portable payload tree.

Contract (D, C, H, R, X). Registered codecs map supported objects to versioned payload trees. Envelope functions map payloads to canonical digest-checked JSON or atomic files and back. Tags must be unique, migrations adjacent, schemas exact, depths within budget, digests valid, and overwrite policy explicit. Results are codecs, migrations, payloads, text, paths, or decoded values. Schema, tamper, resource, path, and overwrite failures are explicit.

Guarantees (E, P, B, K). Canonical encoding and SHA-256 comparison are exact. Portability exists only for registered types. Unsupported values are rejected. The base route uses JSON, hashing, and filesystem primitives with NumPy used by array codecs. Work is linear in encoded payload size, bounded by `max_depth`, with atomic file replacement for writes.

Evidence and links (V, L). Chapter 61 provides envelope, tamper, depth, overwrite, and roundtrip examples. The persistence contract checks all 51 codecs and one migration. Related symbols are A.9 codec-contract records and `Certificate`.

A.9 API, semantic, persistence, and aggregate contract functions

A.9 · `ApiCompatibilityReport`

Identity (`q, a, t, k`). Canonical manifest name `komorebi_math.stable.ApiCompatibilityReport`. Name-preserving aliases `komorebi_math.v1.ApiCompatibilityReport` and `komorebi_math.api.v1.ApiCompatibilityReport`. Tier `[STABLE]`. Manifest kind `class`. Reflected implementation `komorebi_math.compatibility.ApiCompatibilityReport`. Profile `A9-A1`.

Manifest signature (`s`).

```
(reference_digest: 'str', current_digest: 'str', added: 'tuple[tuple[str, str], ...]', removed: 'tuple[tuple[str,
   ↪ str], ...]', kind_changed: 'tuple[tuple[str, str, str, str], ...]', signature_changed: 'tuple[tuple[str,
   ↪ str, str | None, str | None], ...]', strict_signatures: 'bool', certificate: 'Certificate') -> None
```

Definition. Structural differences between a reference and current API manifest.

Contract (`D, C, H, R, X`). Stable namespace reflection maps to a deterministic `PublicApiManifest`. Loading and checking map committed JSON plus live reflection to an `ApiCompatibilityReport`. Schemas, package identity, unique names, kinds, owners, and strict signatures must agree. Results are manifests or reports. Malformed JSON and incompatible surfaces raise validation or failed-certificate paths.

Guarantees (`E, P, B, K`). Comparison is exact structural evidence and does not prove every symbol's mathematics. Manifests/reports have no frozen stable codec. Reflection and JSON use the base runtime. Work is linearithmic in export count because entries are normalized and sorted.

Evidence and links (`V, L`). Chapter 72 and `verify stable` cover the 131-name manifest and the three name-preserving alias facades. The stable reference-card parser independently checks every card signature. Related symbols are semantic, persistence, and aggregate contracts.

A.9 · `PortableCodecCompatibilityReport`

Identity (`q, a, t, k`). Canonical manifest name `komorebi_math.stable.PortableCodecCompatibilityReport`. Name-preserving aliases `komorebi_math.v1.PortableCodecCompatibilityReport` and `komorebi_math.api.v1.PortableCodecCompatibilityReport`. Tier `[STABLE]`. Manifest kind `class`. Reflected implementation `komorebi_math.persistence_contract.PortableCodecCompatibilityReport`. Profile `A9-P1`.

Manifest signature (`s`).

```
(reference_version: 'str', current_version: 'str', added: 'tuple[str, ...]', removed: 'tuple[str, ...]',
   ↪ type_changed: 'tuple[tuple[str, str, str], ...]', version_decreased: 'tuple[tuple[str, int, int], ...]',
   ↪ version_increased: 'tuple[tuple[str, int, int], ...]', missing_migrations: 'tuple[tuple[str, int, int],
   ↪ ...]', certificate: 'Certificate') -> None
```

Definition. Comparison of a committed codec contract with the active registry.

Contract (`D, C, H, R, X`). The active codec registry maps to a deterministic contract. Load/save/compare/check map contract JSON and registry state to a compatibility report. Tags, type names, versions, and adjacent migrations must satisfy the frozen schema. Results are entries, contracts, paths, or reports. Malformed files, removed tags, type changes, version regressions, and missing migrations fail validation.

Guarantees (`E, P, B, K`). Compatibility comparison and canonical contract writing are exact. Contract records are metadata about portability, not themselves assigned stable codecs. Base JSON/filesystem operations suffice. Work is linear in the 51 sorted codec entries and migration set.

Evidence and links (`V, L`). Chapters 61 and 72 and the stable verification suite check 51 codecs and one adjacent migration. Related symbols are A.8 registry/envelope functions and the aggregate stable contract.

A.9 · PortableCodecContract

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.PortableCodecContract`. Name-preserving aliases `komorebi_math.v1.PortableCodecContract` and `komorebi_math.api.v1.PortableCodecContract`. Tier `[STABLE]`. Manifest kind `class`. Reflected implementation `komorebi_math.persistence_contract.PortableCodecContract`. Profile `A9-P1`.

Manifest signature (s).

```
(package_version: 'str', entries: 'tuple[PortableCodecContractEntry, ...]', schema: 'str' =
    ↪ 'komorebi-math.portable-codec-contract', schema_version: 'int' = 1) -> None
```

Definition. Deterministic package-owned portable codec inventory.

Contract (D, C, H, R, X). The active codec registry maps to a deterministic contract. Load/save/compare/check map contract JSON and registry state to a compatibility report. Tags, type names, versions, and adjacent migrations must satisfy the frozen schema. Results are entries, contracts, paths, or reports. Malformed files, removed tags, type changes, version regressions, and missing migrations fail validation.

Guarantees (E, P, B, K). Compatibility comparison and canonical contract writing are exact. Contract records are metadata about portability, not themselves assigned stable codecs. Base JSON/filesystem operations suffice. Work is linear in the 51 sorted codec entries and migration set.

Evidence and links (V, L). Chapters 61 and 72 and the stable verification suite check 51 codecs and one adjacent migration. Related symbols are A.8 registry/envelope functions and the aggregate stable contract.

A.9 · PortableCodecContractEntry

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.PortableCodecContractEntry`. Name-preserving aliases `komorebi_math.v1.PortableCodecContractEntry` and `komorebi_math.api.v1.PortableCodecContractEntry`. Tier `[STABLE]`. Manifest kind `class`. Reflected implementation `komorebi_math.persistence_contract.PortableCodecContractEntry`. Profile `A9-P1`.

Manifest signature (s).

```
(tag: 'str', version: 'int', implementation_type: 'str') -> None
```

Definition. One codec identity frozen by the persistence compatibility contract.

Contract (D, C, H, R, X). The active codec registry maps to a deterministic contract. Load/save/compare/check map contract JSON and registry state to a compatibility report. Tags, type names, versions, and adjacent migrations must satisfy the frozen schema. Results are entries, contracts, paths, or reports. Malformed files, removed tags, type changes, version regressions, and missing migrations fail validation.

Guarantees (E, P, B, K). Compatibility comparison and canonical contract writing are exact. Contract records are metadata about portability, not themselves assigned stable codecs. Base JSON/filesystem operations suffice. Work is linear in the 51 sorted codec entries and migration set.

Evidence and links (V, L). Chapters 61 and 72 and the stable verification suite check 51 codecs and one adjacent migration. Related symbols are A.8 registry/envelope functions and the aggregate stable contract.

A.9 · PublicApiManifest

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.PublicApiManifest`. Name-preserving aliases `komorebi_math.v1.PublicApiManifest` and `komorebi_math.api.v1.PublicApiManifest`. Tier `[STABLE]`. Manifest kind `class`. Reflected implementation `komorebi_math.compatibility.PublicApiManifest`. Profile `A9-A1`.

Manifest signature (s).

```
(package_version: 'str', namespaces: 'tuple[str, ...]', symbols: 'tuple[PublicApiSymbol, ...]', schema: 'str' =
    ↪ 'komorebi-math.public-api', schema_version: 'int' = 1, stability_level: 'str' = 'pre-beta-additive') -> None
```

Definition. A deterministic snapshot of selected `__all__` namespaces.

Contract (D, C, H, R, X). Stable namespace reflection maps to a deterministic `PublicApiManifest`. Loading and checking map committed JSON plus live reflection to an `ApiCompatibilityReport`. Schemas, package identity, unique names, kinds, owners, and strict signatures must agree. Results are manifests or reports. Malformed JSON and incompatible surfaces raise validation or failed-certificate paths.

Guarantees (E, P, B, K). Comparison is exact structural evidence and does not prove every symbol's mathematics. Manifests/reports have no frozen stable codec. Reflection and JSON use the base runtime. Work is linearithmic in export count because entries are normalized and sorted.

Evidence and links (V, L). Chapter 72 and `verify stable` cover the 131-name manifest and the three name-preserving alias facades. The stable reference-card parser independently checks every card signature. Related symbols are semantic, persistence, and aggregate contracts.

A.9 · `PublicApiSymbol`

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.PublicApiSymbol`. Name-preserving aliases `komorebi_math.v1.PublicApiSymbol` and `komorebi_math.api.v1.PublicApiSymbol`. Tier `[STABLE]`. Manifest kind `class`. Reflected implementation `komorebi_math.compatibility.PublicApiSymbol`. Profile `A9-A1`.

Manifest signature (s).

```
(namespace: 'str', name: 'str', kind: 'str', implementation_module: 'str', qualname: 'str', signature: 'str |
    ↪ None') -> None
```

Definition. One exported public symbol in a deterministic API manifest.

Contract (D, C, H, R, X). Stable namespace reflection maps to a deterministic `PublicApiManifest`. Loading and checking map committed JSON plus live reflection to an `ApiCompatibilityReport`. Schemas, package identity, unique names, kinds, owners, and strict signatures must agree. Results are manifests or reports. Malformed JSON and incompatible surfaces raise validation or failed-certificate paths.

Guarantees (E, P, B, K). Comparison is exact structural evidence and does not prove every symbol's mathematics. Manifests/reports have no frozen stable codec. Reflection and JSON use the base runtime. Work is linearithmic in export count because entries are normalized and sorted.

Evidence and links (V, L). Chapter 72 and `verify stable` cover the 131-name manifest and the three name-preserving alias facades. The stable reference-card parser independently checks every card signature. Related symbols are semantic, persistence, and aggregate contracts.

A.9 · `StableContractReport`

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.StableContractReport`. Name-preserving aliases `komorebi_math.v1.StableContractReport` and `komorebi_math.api.v1.StableContractReport`. Tier `[STABLE]`. Manifest kind `class`. Reflected implementation `komorebi_math.stable_contract.StableContractReport`. Profile `A9-C1`.

Manifest signature (s).

```
(api: 'ApiCompatibilityReport', semantics: 'StableSemanticReport', persistence:
    ↪ 'PortableCodecCompatibilityReport', certificate: 'Certificate') -> None
```

Definition. Complete 1.0-candidate compatibility report.

Contract (D, C, H, R, X). The API, semantic, and portable-codec reports map to one `StableContractReport`. `check_stable_contract` executes all three with optional alternate committed paths. All child certificates must be valid. The result is an aggregate report. Any child incompatibility is retained and makes the aggregate certificate invalid.

Guarantees (E, P, B, K). The aggregate is exact conjunction over finite contract checks. It is not a codec payload and does not establish platform qualification, narrative completeness, or all research semantics. Runtime is the sum of the three child checks and their bounded fixtures.

Evidence and links (V, L). `verify stable` reports 6/6 when the API, aliases, semantics, persistence, aggregate contract, and facade imports pass. Chapter 72 explains the boundary. Related release readiness and platform evidence remain separate Chapter 73 gates.

A.9 · StableSemanticCaseResult

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.StableSemanticCaseResult`. Name-preserving aliases `komorebi_math.v1.StableSemanticCaseResult` and `komorebi_math.api.v1.StableSemanticCaseResult`. Tier `[STABLE]`. Manifest kind `class`. Reflected implementation `komorebi_math.stable_semantics.StableSemanticCaseResult`. Profile `A9-S1`.

Manifest signature (s).

```
(name: 'str', kind: 'str', expected: 'Any', observed: 'Any', passed: 'bool', detail: 'str' = '') -> None
```

Definition. Observed and expected canonical values for one semantic law.

Contract (D, C, H, R, X). A committed stable-semantics fixture maps to a validated path, JSON mapping, digest, individual case results, and an aggregate report. The fixture schema and ten closed expected values must match, and every case must execute. Results preserve expected/observed values and detail. Malformed fixtures or mismatches produce validation failures or an invalid certificate.

Guarantees (E, P, B, K). The ten comparisons are exact for their declared canonical forms. They remain finite regression laws, not universal theorems. Reports have no frozen stable codec. The base path plus domain operations is used. Cost is the sum of the ten bounded cases.

Evidence and links (V, L). Chapter 72 names all semantic-contract layers. The ten cases cover analysis/synthesis, signal/arithmetic/lattice refinement, operator realization, basis roundtrip, generating products, rendering, settings restoration, and portable refinement towers.

A.9 · StableSemanticReport

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.StableSemanticReport`. Name-preserving aliases `komorebi_math.v1.StableSemanticReport` and `komorebi_math.api.v1.StableSemanticReport`. Tier `[STABLE]`. Manifest kind `class`. Reflected implementation `komorebi_math.stable_semantics.StableSemanticReport`. Profile `A9-S1`.

Manifest signature (s).

```
(source: 'Path', fixture_digest: 'str', results: 'tuple[StableSemanticCaseResult, ...]', certificate:
   ↪ 'Certificate') -> None
```

Definition. Aggregate result for the committed stable semantic fixture.

Contract (D, C, H, R, X). A committed stable-semantics fixture maps to a validated path, JSON mapping, digest, individual case results, and an aggregate report. The fixture schema and ten closed expected values must match, and every case must execute. Results preserve expected/observed values and detail. Malformed fixtures or mismatches produce validation failures or an invalid certificate.

Guarantees (E, P, B, K). The ten comparisons are exact for their declared canonical forms. They remain finite regression laws, not universal theorems. Reports have no frozen stable codec. The base path plus domain operations is used. Cost is the sum of the ten bounded cases.

Evidence and links (V, L). Chapter 72 names all semantic-contract layers. The ten cases cover analysis/synthesis, signal/arithmetic/lattice refinement, operator realization, basis roundtrip, generating products, rendering, settings restoration, and portable refinement towers.

A.9 · check_portable_codec_contract

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.check_portable_codec_contract`. Name-preserving aliases `komorebi_math.v1.check_portable_codec_contract` and `komorebi_math.api.v1.check_portable_codec_contract`. Tier `[STABLE]`. Manifest kind `function`. Reflected implementation `komorebi_math.persistence_contract.check_portable_codec_contract`. Profile `A9-P1`.

Manifest signature (s).

```
(path: 'str | Path | None' = None) -> 'PortableCodecCompatibilityReport'
```

Definition. Compare the active registry with the committed codec contract.

Contract (D, C, H, R, X). The active codec registry maps to a deterministic contract. Load/save/compare/check map contract JSON and registry state to a compatibility report. Tags, type names, versions, and adjacent migrations must satisfy the frozen schema. Results are entries, contracts, paths, or reports. Malformed files, removed tags, type changes, version regressions, and missing migrations fail validation.

Guarantees (E, P, B, K). Compatibility comparison and canonical contract writing are exact. Contract records are metadata about portability, not themselves assigned stable codecs. Base JSON/filesystem operations suffice. Work is linear in the 51 sorted codec entries and migration set.

Evidence and links (V, L). Chapters 61 and 72 and the stable verification suite check 51 codecs and one adjacent migration. Related symbols are A.8 registry/envelope functions and the aggregate stable contract.

A.9 · check_stable_api

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.check_stable_api`. Name-preserving aliases `komorebi_math.v1.check_stable_api` and `komorebi_math.api.v1.check_stable_api`. Tier `[STABLE]`. Manifest kind `function`. Reflected implementation `komorebi_math.compatibility.check_stable_api`. Profile `A9-A1`.

Manifest signature (s).

```
(path: 'str | Path | None' = None) -> 'ApiCompatibilityReport'
```

Definition. Check the 1.0-candidate namespace under strict signature compatibility.

Contract (D, C, H, R, X). Stable namespace reflection maps to a deterministic `PublicApiManifest`. Loading and checking map committed JSON plus live reflection to an `ApiCompatibilityReport`. Schemas, package identity, unique names, kinds, owners, and strict signatures must agree. Results are manifests or reports. Malformed JSON and incompatible surfaces raise validation or failed-certificate paths.

Guarantees (E, P, B, K). Comparison is exact structural evidence and does not prove every symbol's mathematics. Manifests/reports have no frozen stable codec. Reflection and JSON use the base runtime. Work is linearithmic in export count because entries are normalized and sorted.

Evidence and links (V, L). Chapter 72 and `verify stable` cover the 131-name manifest and the three name-preserving alias facades. The stable reference-card parser independently checks every card signature. Related symbols are semantic, persistence, and aggregate contracts.

A.9 · check_stable_contract

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.check_stable_contract`. Name-preserving aliases `komorebi_math.v1.check_stable_contract` and `komorebi_math.api.v1.check_stable_contract`. Tier `[STABLE]`. Manifest kind `function`. Reflected implementation `komorebi_math.stable_contract.check_stable_contract`. Profile `A9-C1`.

Manifest signature (s).

```
(*, api_manifest: 'str | Path | None' = None, semantic_fixture: 'str | Path | None' = None, persistence_contract:
   ↪  'str | Path | None' = None) -> 'StableContractReport'
```

Definition. Run every compatibility layer required by the frozen 1.0 surface.

Contract (D, C, H, R, X). The API, semantic, and portable-codec reports map to one `StableContractReport`. `check_stable_contract` executes all three with optional alternate committed paths. All child certificates must be valid. The result is an aggregate report. Any child incompatibility is retained and makes the aggregate certificate invalid.

Guarantees (E, P, B, K). The aggregate is exact conjunction over finite contract checks. It is not a codec payload and does not establish platform qualification, narrative completeness, or all research semantics. Runtime is the sum of the three child checks and their bounded fixtures.

Evidence and links (V, L). `verify stable` reports 6/6 when the API, aliases, semantics, persistence, aggregate contract, and facade imports pass. Chapter 72 explains the boundary. Related release readiness and platform evidence remain separate Chapter 73 gates.

A.9 · `compare_portable_codec_contract`

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.compare_portable_codec_contract`. Name-preserving aliases `komorebi_math.v1.compare_portable_codec_contract` and `komorebi_math.api.v1.compare_portable_codec_contract`. Tier `[STABLE]`. Manifest kind `function`. Reflected implementation `komorebi_math.persistence_contract.compare_portable_codec_contract`. Profile `A9-P1`.

Manifest signature (s).

```
(reference: 'PortableCodecContract', current: 'PortableCodecContract | None' = None) ->
   ↪  'PortableCodecCompatibilityReport'
```

Definition. Check tag, type, version, and adjacent migration compatibility.

Contract (D, C, H, R, X). The active codec registry maps to a deterministic contract. Load/save/compare/check map contract JSON and registry state to a compatibility report. Tags, type names, versions, and adjacent migrations must satisfy the frozen schema. Results are entries, contracts, paths, or reports. Malformed files, removed tags, type changes, version regressions, and missing migrations fail validation.

Guarantees (E, P, B, K). Compatibility comparison and canonical contract writing are exact. Contract records are metadata about portability, not themselves assigned stable codecs. Base JSON/filesystem operations suffice. Work is linear in the 51 sorted codec entries and migration set.

Evidence and links (V, L). Chapters 61 and 72 and the stable verification suite check 51 codecs and one adjacent migration. Related symbols are A.8 registry/envelope functions and the aggregate stable contract.

A.9 · `create_portable_codec_contract`

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.create_portable_codec_contract`. Name-preserving aliases `komorebi_math.v1.create_portable_codec_contract` and `komorebi_math.api.v1.create_portable_codec_contract`. Tier `[STABLE]`. Manifest kind `function`. Reflected implementation `komorebi_math.persistence_contract.create_portable_codec_contract`. Profile `A9-P1`.

Manifest signature (s).

```
(*, package_version: 'str | None' = None) -> 'PortableCodecContract'
```

Definition. Snapshot every package-owned codec in deterministic tag order.

Contract (D, C, H, R, X). The active codec registry maps to a deterministic contract. Load/save/compare/check map contract JSON and registry state to a compatibility report. Tags, type names, versions, and adjacent migrations must satisfy the frozen schema. Results are entries, contracts, paths, or reports. Malformed files, removed tags, type changes, version regressions, and missing migrations fail validation.

Guarantees (E, P, B, K). Compatibility comparison and canonical contract writing are exact. Contract records are metadata about portability, not themselves assigned stable codecs. Base JSON/filesystem operations suffice. Work is linear in the 51 sorted codec entries and migration set.

Evidence and links (V, L). Chapters 61 and 72 and the stable verification suite check 51 codecs and one adjacent migration. Related symbols are A.8 registry/envelope functions and the aggregate stable contract.

A.9 · `create_stable_api_manifest`

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.create_stable_api_manifest`. Name-preserving aliases `komorebi_math.v1.create_stable_api_manifest` and `komorebi_math.api.v1.create_stable_api_manifest`. Tier `[STABLE]`. Manifest kind `function`. Reflected implementation `komorebi_math.compatibility.create_stable_api_manifest`. Profile `A9-A1`.

Manifest signature (s).

```
(*, package_version: 'str | None' = None) -> 'PublicApiManifest'
```

Definition. Build the curated 1.0-candidate namespace manifest.

Contract (D, C, H, R, X). Stable namespace reflection maps to a deterministic `PublicApiManifest`. Loading and checking map committed JSON plus live reflection to an `ApiCompatibilityReport`. Schemas, package identity, unique names, kinds, owners, and strict signatures must agree. Results are manifests or reports. Malformed JSON and incompatible surfaces raise validation or failed-certificate paths.

Guarantees (E, P, B, K). Comparison is exact structural evidence and does not prove every symbol's mathematics. Manifests/reports have no frozen stable codec. Reflection and JSON use the base runtime. Work is linearithmic in export count because entries are normalized and sorted.

Evidence and links (V, L). Chapter 72 and `verify stable` cover the 131-name manifest and the three name-preserving alias facades. The stable reference-card parser independently checks every card signature. Related symbols are semantic, persistence, and aggregate contracts.

A.9 · `load_portable_codec_contract`

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.load_portable_codec_contract`. Name-preserving aliases `komorebi_math.v1.load_portable_codec_contract` and `komorebi_math.api.v1.load_portable_codec_contract`. Tier `[STABLE]`. Manifest kind `function`. Reflected implementation `komorebi_math.persistence_contract.load_portable_codec_contract`. Profile `A9-P1`.

Manifest signature (s).

```
(path: 'str | Path | None' = None) -> 'PortableCodecContract'
```

Definition. Load and validate the committed codec compatibility contract.

Contract (D, C, H, R, X). The active codec registry maps to a deterministic contract. Load/save/compare/check map contract JSON and registry state to a compatibility report. Tags, type names, versions, and adjacent migrations must satisfy the frozen schema. Results are entries, contracts, paths, or reports. Malformed files, removed tags, type changes, version regressions, and missing migrations fail validation.

Guarantees (E, P, B, K). Compatibility comparison and canonical contract writing are exact. Contract records are metadata about portability, not themselves assigned stable codecs. Base JSON/filesystem operations suffice. Work is linear in the 51 sorted codec entries and migration set.

Evidence and links (V, L). Chapters 61 and 72 and the stable verification suite check 51 codecs and one adjacent migration. Related symbols are A.8 registry/envelope functions and the aggregate stable contract.

A.9 · load_stable_api_manifest

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.load_stable_api_manifest`. Name-preserving aliases `komorebi_math.v1.load_stable_api_manifest` and `komorebi_math.api.v1.load_stable_api_manifest`. Tier `[STABLE]`. Manifest kind `function`. Reflected implementation `komorebi_math.compatibility.load_stable_api_manifest`. Profile `A9-A1`.

Manifest signature (s).

```
(path: 'str | Path | None' = None) -> 'PublicApiManifest'
```

Definition. Load and validate the committed 1.0-candidate API manifest.

Contract (D, C, H, R, X). Stable namespace reflection maps to a deterministic `PublicApiManifest`. Loading and checking map committed JSON plus live reflection to an `ApiCompatibilityReport`. Schemas, package identity, unique names, kinds, owners, and strict signatures must agree. Results are manifests or reports. Malformed JSON and incompatible surfaces raise validation or failed-certificate paths.

Guarantees (E, P, B, K). Comparison is exact structural evidence and does not prove every symbol's mathematics. Manifests/reports have no frozen stable codec. Reflection and JSON use the base runtime. Work is linearithmic in export count because entries are normalized and sorted.

Evidence and links (V, L). Chapter 72 and `verify stable` cover the 131-name manifest and the three name-preserving alias facades. The stable reference-card parser independently checks every card signature. Related symbols are semantic, persistence, and aggregate contracts.

A.9 · load_stable_semantics_fixture

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.load_stable_semantics_fixture`. Name-preserving aliases `komorebi_math.v1.load_stable_semantics_fixture` and `komorebi_math.api.v1.load_stable_semantics_fixture`. Tier `[STABLE]`. Manifest kind `function`. Reflected implementation `komorebi_math.stable_semantics.load_stable_semantics_fixture`. Profile `A9-S1`.

Manifest signature (s).

```
(path: 'str | Path | None' = None) -> 'tuple[Path, Mapping[str, Any], str]'
```

Definition. Load, validate, and digest one stable semantic fixture.

Contract (D, C, H, R, X). A committed stable-semantics fixture maps to a validated path, JSON mapping, digest, individual case results, and an aggregate report. The fixture schema and ten closed expected values must match, and every case must execute. Results preserve expected/observed values and detail. Malformed fixtures or mismatches produce validation failures or an invalid certificate.

Guarantees (E, P, B, K). The ten comparisons are exact for their declared canonical forms. They remain finite regression laws, not universal theorems. Reports have no frozen stable codec. The base path plus domain operations is used. Cost is the sum of the ten bounded cases.

Evidence and links (V, L). Chapter 72 names all semantic-contract layers. The ten cases cover analysis/synthesis, signal/arithmetic/lattice refinement, operator realization, basis roundtrip, generating products, rendering, settings restoration, and portable refinement towers.

A.9 · run_stable_semantics_fixture

Identity (q, a, t, k). Canonical manifest name `komorebi_math.stable.run_stable_semantics_fixture`. Name-preserving aliases `komorebi_math.v1.run_stable_semantics_fixture` and `komorebi_math.api.v1.run_stable_semantics_fixture`. Tier `[STABLE]`. Manifest kind `function`. Reflected implementation `komorebi_math.stable_semantics.run_stable_semantics_fixture`. Profile `A9-S1`.

Manifest signature (`s`).

```
(path: 'str | Path | None' = None) -> 'StableSemanticReport'
```

Definition. Execute every committed 1.0-candidate semantic case.

Contract (`D, C, H, R, X`). A committed stable-semantics fixture maps to a validated path, JSON mapping, digest, individual case results, and an aggregate report. The fixture schema and ten closed expected values must match, and every case must execute. Results preserve expected/observed values and detail. Malformed fixtures or mismatches produce validation failures or an invalid certificate.

Guarantees (`E, P, B, K`). The ten comparisons are exact for their declared canonical forms. They remain finite regression laws, not universal theorems. Reports have no frozen stable codec. The base path plus domain operations is used. Cost is the sum of the ten bounded cases.

Evidence and links (`V, L`). Chapter 72 names all semantic-contract layers. The ten cases cover analysis/synthesis, signal/arithmetic/lattice refinement, operator realization, basis roundtrip, generating products, rendering, settings restoration, and portable refinement towers.

A.9 · `save_portable_codec_contract`

Identity (`q, a, t, k`). Canonical manifest name `komorebi_math.stable.save_portable_codec_contract`. Name-preserving aliases `komorebi_math.v1.save_portable_codec_contract` and `komorebi_math.api.v1.save_portable_codec_contract`. Tier `[STABLE]`. Manifest kind `function`. Reflected implementation `komorebi_math.persistence_contract.save_portable_codec_contract`. Profile `A9-P1`.

Manifest signature (`s`).

```
(contract: 'PortableCodecContract', path: 'str | Path', *, overwrite: 'bool' = False) -> 'Path'
```

Definition. Write a canonical portable-codec contract.

Contract (`D, C, H, R, X`). The active codec registry maps to a deterministic contract. Load/save/compare/check map contract JSON and registry state to a compatibility report. Tags, type names, versions, and adjacent migrations must satisfy the frozen schema. Results are entries, contracts, paths, or reports. Malformed files, removed tags, type changes, version regressions, and missing migrations fail validation.

Guarantees (`E, P, B, K`). Compatibility comparison and canonical contract writing are exact. Contract records are metadata about portability, not themselves assigned stable codecs. Base JSON/filesystem operations suffice. Work is linear in the 51 sorted codec entries and migration set.

Evidence and links (`V, L`). Chapters 61 and 72 and the stable verification suite check 51 codecs and one adjacent migration. Related symbols are A.8 registry/envelope functions and the aggregate stable contract.

> **Stable-reference coverage.** The recorded validation found 131/131 unique export cards, 131/131 unique anchors, exact agreement with all 112 non-null manifest signatures and all 19 manifest `null` values, and runtime identity across all 393 name-preserving versioned routes. This establishes stable exported-symbol coverage only. Appendix B records the curated beta surface, and Appendix C records the research surface.

Appendix B. Beta API: 83-entry reference

Using the beta reference. Use this appendix when the object you need is listed in the curated beta namespace. Read the card-reading key at the start of Appendix A, then use the canonical `komorebi_math.beta` route given on the card. A convenient root re-export does not express the same compatibility dependency in your source code. If the object also has a stable route, compare the stable card and its contract before choosing an import.

The beta compatibility check and a computation's certificate answer different questions. The former compares the public interface with its recorded contract. The latter reports evidence for the computation or validation that produced it. Run and interpret each check for its stated purpose. A compatible manifest does not establish that every input to every function has been tested.

Choose the section by the task.

If you want to…	Begin with	Then read
Inspect certificates, manifests, or regression reports	B.1	Chapters 5, 7, and 72
Build or execute packet jets and understand their work estimates	B.2	Chapters 38 and 43
Capture, resume, or read a finite packet-jet stream	B.3	Chapter 43 and the stream contracts in Appendix G
Encode or recover a supported portable object	B.4	Chapter 61 and Appendix G

For execution, read the input and resource limits before allocating a full family or snapshot. For streaming, distinguish opening a bounded reader from loading the entire snapshot: the cards state their different memory and completion requirements. A partial stream can be useful progress, but it is not a complete snapshot. For persistence, retain the schema and codec version with the data and use the declared decoding route rather than reconstructing an object from a display representation.

The shared card fields are repeated intentionally so that a card can be read on its own. Use the linked chapters when you need the sequence of operations, and return to the card when you need an exact parameter or failure boundary.

Appendix B records the curated beta exported-symbol ledger for the reference `komorebi_math==1.0.0b4` package. It covers exactly 83 manifest names: 45 classes, 28 functions, and 10 data exports. The beta tier is intentionally distinct from the frozen stable tier: it permits additive growth, but the committed `1.0-beta-additive-strict-signatures` contract treats removal, broad kind change, and reflected signature change as release failures.

The canonical committed manifest has compact canonical-JSON digest `311f2bf7cbcf0a45bcbe2af433d73a8f597c0fbf7133750882314d6e0edb16b2`. Its two committed filename variants, `public_api_beta_1_0.json` and `public_api_beta_1_0_0b4.json`, are byte-identical in the reference source and have raw SHA-256 `c0f4cf27d4a47b86d1ea5b79c6c2fe883368e6ab7f8a640e527f916b271ba000`. Canonical and raw digests have different meanings and are not interchangeable.

B.0 Reference contract, maturity rule, and route audit

Every card implements the Chapter 76 fields `q, a, t, k, s, D, C, H, R, X, E, P, B, K, V, L`. Exact manifest signatures appear verbatim in fenced `text` blocks. Literal `null` is preserved for the ten data exports and six exception classes. The manual does not manufacture constructor signatures the manifest does not freeze.

The canonical tier-declaring route is always `komorebi_math.beta.<name>`. The reference root and domain namespaces expose 125 additional identity-equal routes, for 208 audited qualified identities across the 83 names. Those re-exports are convenience routes. Only the `komorebi_math.beta` import explicitly declares the beta compatibility dependency. Unlike the stable tier, there is no `komorebi_math.api.beta` or versioned `v1` beta facade in this source snapshot.

Six names are available only through the canonical beta namespace, 29 have one audited convenience route, and 48 have two.

The cards distinguish structural compatibility, finite regression evidence, mathematical certificates, local runtime observations, persistence integrity, and authenticity. In particular, the strict beta manifest check does not prove behavioral equivalence. The 15-case golden fixture is representative rather than exhaustive. Profiler samples are local observations rather than asymptotic bounds, and SHA-256 stream or portable digests detect corruption but do not authenticate the producer.

Section	Classes	Functions	Data	Total
B.1	16	5	4	25
B.2	19	7	0	26
B.3	8	5	4	17
B.4	2	11	2	15
Total	**45**	**28**	**10**	**83**

A short canonical import pattern is:

```
from komorebi_math import beta

report = beta.check_beta_api()
report.certificate.require()
assert report.strict_signatures
```

B.1 Foundational records, golden fixtures, and compatibility

B.1 · BEHAVIORAL_GOLDEN_SCHEMA

Identity (q, a, t, k). Canonical beta route `komorebi_math.beta.BEHAVIORAL_GOLDEN_SCHEMA`. Identity-equal audited re-exports `komorebi_math.BEHAVIORAL_GOLDEN_SCHEMA`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `data`. Reflected implementation `builtins.str`. Profile `B1-I1`.

Manifest signature (s).

```
null
```

Definition. The immutable runtime value is `komorebi-math.behavioral-golden`.

Contract (D, C, H, R, X). The domain is import-time binding and the codomain is the recorded immutable string or integer. Importing the package is the only precondition. The value is returned by ordinary attribute access. It has no call-time exception path and does not by itself validate a payload, fixture, or compatibility report.

Guarantees (E, P, B, K). The binding is an exact package identity or schema identifier. It requires no optional backend, performs no fallback, and has constant-time lookup. A schema name and version identify a format. They are not proof that arbitrary data conforms to that format.

Evidence and links (V, L). Verified by the beta manifest parser, runtime identity audit, beta regression tests, and the Appendix B reference-card parser. See Chapters 0, 7, 61, and 72 and the related loader or codec functions.

B.1 · BEHAVIORAL_GOLDEN_SCHEMA_VERSION

Identity (q, a, t, k). Canonical beta route `komorebi_math.beta.BEHAVIORAL_GOLDEN_SCHEMA_VERSION`. Identity-equal audited re-exports `komorebi_math.BEHAVIORAL_GOLDEN_SCHEMA_VERSION`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `data`. Reflected implementation `builtins.int`. Profile `B1-I1`.

Manifest signature (s).

```
null
```

Definition. The immutable runtime value is `1`.

Contract (D, C, H, R, X). The domain is import-time binding and the codomain is the recorded immutable string or integer. Importing the package is the only precondition. The value is returned by ordinary attribute access. It has no call-time exception path and does not by itself validate a payload, fixture, or compatibility report.

Guarantees (E, P, B, K). The binding is an exact package identity or schema identifier. It requires no optional backend, performs no fallback, and has constant-time lookup. A schema name and version identify a format. They are not proof that arbitrary data conforms to that format.

Evidence and links (V, L). Verified by the beta manifest parser, runtime identity audit, beta regression tests, and the Appendix B reference-card parser. See Chapters 0, 7, 61, and 72 and the related loader or codec functions.

B.1 · `BETA_API_STABILITY_LEVEL`

Identity (q, a, t, k). Canonical beta route `komorebi_math.beta.BETA_API_STABILITY_LEVEL`. Identity-equal audited re-exports `komorebi_math.BETA_API_STABILITY_LEVEL`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `data`. Reflected implementation `builtins.str`. Profile `B1-I1`.

Manifest signature (s).

```
null
```

Definition. The immutable runtime value is `1.0-beta-additive-strict-signatures`: additions are allowed, while removal, kind drift, and reflected signature drift fail the beta compatibility gate.

Contract (D, C, H, R, X). The domain is import-time binding and the codomain is the recorded immutable string or integer. Importing the package is the only precondition. The value is returned by ordinary attribute access. It has no call-time exception path and does not by itself validate a payload, fixture, or compatibility report.

Guarantees (E, P, B, K). The binding is an exact package identity or schema identifier. It requires no optional backend, performs no fallback, and has constant-time lookup. A schema name and version identify a format. They are not proof that arbitrary data conforms to that format.

Evidence and links (V, L). Verified by the beta manifest parser, runtime identity audit, beta regression tests, and the Appendix B reference-card parser. See Chapters 0, 7, 61, and 72 and the related loader or codec functions.

B.1 · `__version__`

Identity (q, a, t, k). Canonical beta route `komorebi_math.beta.__version__`. Identity-equal audited re-exports `komorebi_math.__version__`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `data`. Reflected implementation `builtins.str`. Profile `B1-I1`.

Manifest signature (s).

```
null
```

Definition. The recorded package version is `1.0.0b4`, as stated in the reference manifest.

Contract (D, C, H, R, X). The domain is import-time binding and the codomain is the recorded immutable string or integer. Importing the package is the only precondition. The value is returned by ordinary attribute access. It has no call-time exception path and does not by itself validate a payload, fixture, or compatibility report.

Guarantees (E, P, B, K). The binding is an exact package identity or schema identifier. It requires no optional backend, performs no fallback, and has constant-time lookup. A schema name and version identify a format. They are not proof that arbitrary data conforms to that format.

Evidence and links (V, L). Verified by the beta manifest parser, runtime identity audit, beta regression tests, and the Appendix B reference-card parser. See Chapters 0, 7, 61, and 72 and the related loader or codec functions.

B.1 · Exactness

Identity (q, a, t, k). Canonical beta route `komorebi_math.beta.Exactness`. Identity-equal audited re-exports `komorebi_math.Exactness`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `class`. Reflected implementation `komorebi_math.certificates.Exactness`. Profile `B1-C1`.

Manifest signature (s).

```
(*values)
```

Definition. A string-valued evidence classification enum. The five string values are `exact`, `numerical`, `conditional`, `heuristic`, and `planned`. Members: `EXACT=exact`, `NUMERICAL=numerical`, `CONDITIONAL=conditional`, `HEURISTIC=heuristic`, `PLANNED=planned`.

Contract (D, C, H, R, X). These records classify and carry validation evidence. Inputs are the declared fields in the manifest signature. The codomain is an immutable enum member or frozen result record. Construction requires type-consistent values. `Certificate.require()` and `ComputationResult.require()` raise `ValidationError` when the certificate is invalid. `ConditionalResult` rejects a certificate whose exactness is not conditional.

Guarantees (E, P, B, K). The classes preserve the supplied evidence exactly and make exact, numerical, conditional, heuristic, and planned states machine-visible. They do not independently prove the theorem named in a record. They use only the Python standard library, have no backend fallback, and require storage linear in the number of residual, assumption, and diagnostic entries.

Evidence and links (V, L). Covered by certificate unit tests, portable certificate round-trips, negative invalid-certificate tests, and aggregate verification reports. See Chapters 1, 5, 6, 37, 43, 61, and 71.

B.1 · Certificate

Identity (q, a, t, k). Canonical beta route `komorebi_math.beta.Certificate`. Identity-equal audited re-exports `komorebi_math.Certificate`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `class`. Reflected implementation `komorebi_math.certificates.Certificate`. Profile `B1-C1`.

Manifest signature (s).

```
(valid: 'bool', exactness: 'Exactness', residuals: 'dict[str, float]' = <factory>, tolerance: 'float | None' =
    ↪ None, assumptions_checked: 'tuple[str, ...]' = (), assumptions_unchecked: 'tuple[str, ...]' = (),
    ↪ diagnostics: 'dict[str, Any]' = <factory>, theorem_id: 'str | None' = None, message: 'str' = '') -> None
```

Definition. Structured validation record. The record carries validity, exactness, residuals, tolerance, checked and unchecked assumptions, diagnostics, an optional theorem identifier, and a message. Stored fields: `valid`, `exactness`, `residuals`, `tolerance`, `assumptions_checked`, `assumptions_unchecked`, `diagnostics`, `theorem_id`, `message`. Runtime-visible operational members: `require`, `to_dict`, `to_json`.

Contract (D, C, H, R, X). These records classify and carry validation evidence. Inputs are the declared fields in the manifest signature. The codomain is an immutable enum member or frozen result record. Construction requires type-consistent values. `Certificate.require()` and `ComputationResult.require()` raise `ValidationError` when the certificate is invalid. `ConditionalResult` rejects a certificate whose exactness is not conditional.

Guarantees (E, P, B, K). The classes preserve the supplied evidence exactly and make exact, numerical, conditional, heuristic, and planned states machine-visible. They do not independently prove the theorem named in a record. They use only the Python standard library, have no backend fallback, and require storage linear in the number of residual, assumption, and diagnostic entries.

Evidence and links (V, L). Covered by certificate unit tests, portable certificate round-trips, negative invalid-certificate tests, and aggregate verification reports. See Chapters 1, 5, 6, 37, 43, 61, and 71.

B.1 · ComputationResult

Identity (q, a, t, k). Canonical beta route `komorebi_math.beta.ComputationResult`. Identity-equal audited re-exports `komorebi_math.ComputationResult`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `class`. Reflected implementation `komorebi_math.certificates.ComputationResult`. Profile `B1-C1`.

Manifest signature (s).

```
(value: 'T', certificate: 'Certificate') -> None
```

Definition. A value paired with its execution certificate. Properties expose certificate validity and exactness. `require()` validates and returns the value. Stored fields: `value`, `certificate`. Runtime-visible operational members: `exact`, `valid`, `require`.

Contract (D, C, H, R, X). These records classify and carry validation evidence. Inputs are the declared fields in the manifest signature. The codomain is an immutable enum member or frozen result record. Construction requires type-consistent values. `Certificate.require()` and `ComputationResult.require()` raise `ValidationError` when the certificate is invalid. `ConditionalResult` rejects a certificate whose exactness is not conditional.

Guarantees (E, P, B, K). The classes preserve the supplied evidence exactly and make exact, numerical, conditional, heuristic, and planned states machine-visible. They do not independently prove the theorem named in a record. They use only the Python standard library, have no backend fallback, and require storage linear in the number of residual, assumption, and diagnostic entries.

Evidence and links (V, L). Covered by certificate unit tests, portable certificate round-trips, negative invalid-certificate tests, and aggregate verification reports. See Chapters 1, 5, 6, 37, 43, 61, and 71.

B.1 · ConditionalResult

Identity (q, a, t, k). Canonical beta route `komorebi_math.beta.ConditionalResult`. Identity-equal audited re-exports `komorebi_math.ConditionalResult`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `class`. Reflected implementation `komorebi_math.certificates.ConditionalResult`. Profile `B1-C1`.

Manifest signature (s).

```
(value: 'T', certificate: 'Certificate') -> None
```

Definition. A value whose conclusion depends on explicit unchecked assumptions. Construction enforces `Exactness.CONDITIONAL`, preventing an unconditional certificate from being wrapped as assumption-dependent output. Stored fields: `value`, `certificate`.

Contract (D, C, H, R, X). These records classify and carry validation evidence. Inputs are the declared fields in the manifest signature. The codomain is an immutable enum member or frozen result record. Construction requires type-consistent values. `Certificate.require()` and `ComputationResult.require()` raise `ValidationError` when the certificate is invalid. `ConditionalResult` rejects a certificate whose exactness is not conditional.

Guarantees (E, P, B, K). The classes preserve the supplied evidence exactly and make exact, numerical, conditional, heuristic, and planned states machine-visible. They do not independently prove the theorem named in a record. They use only the Python standard library, have no backend fallback, and require storage linear in the number of residual, assumption, and diagnostic entries.

Evidence and links (V, L). Covered by certificate unit tests, portable certificate round-trips, negative invalid-certificate tests, and aggregate verification reports. See Chapters 1, 5, 6, 37, 43, 61, and 71.

B.1 · KomorebiMathError

Identity (q, a, t, k). Canonical beta route `komorebi_math.beta.KomorebiMathError`. No identity-equal re-export in the six audited root/domain namespaces. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `class`. Reflected implementation `komorebi_math.exceptions.KomorebiMathError`. Profile `B1-X1`.

Manifest signature (`s`).

```
null
```

Definition. Base class for all public package exceptions.

Contract (`D, C, H, R, X`). The domain is an error condition raised by a public Komorebi operation. The codomain is an exception object carrying the caller-supplied message and ordinary Python exception context. There are no mathematical hypotheses. Constructing an exception does not raise package-specific errors. The originating operation determines when it is emitted.

Guarantees (`E, P, B, K`). The hierarchy provides deterministic error classification, not recovery or proof. It has no optional dependencies or backend behavior and constant construction cost apart from the message and traceback. `ValidationError` also derives from `ValueError`. Specialized classes retain the public `KomorebiMathError` lineage.

Evidence and links (`V, L`). Negative-path tests assert the specific classes for malformed structures, dimension mismatches, unavailable optional backends, exactness violations, and unmet assumptions. See Chapters 6, 8, 37, 43, 61, and Appendix I.

B.1 · `ValidationError`

Identity (`q, a, t, k`). Canonical beta route `komorebi_math.beta.ValidationError`. No identity-equal re-export in the six audited root/domain namespaces. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `class`. Reflected implementation `komorebi_math.exceptions.ValidationError`. Profile `B1-X1`.

Manifest signature (`s`).

```
null
```

Definition. Raised when a mathematical object fails a declared validation rule. This class derives from both `KomorebiMathError` and `ValueError`.

Contract (`D, C, H, R, X`). The domain is an error condition raised by a public Komorebi operation. The codomain is an exception object carrying the caller-supplied message and ordinary Python exception context. There are no mathematical hypotheses. Constructing an exception does not raise package-specific errors. The originating operation determines when it is emitted.

Guarantees (`E, P, B, K`). The hierarchy provides deterministic error classification, not recovery or proof. It has no optional dependencies or backend behavior and constant construction cost apart from the message and traceback. `ValidationError` also derives from `ValueError`. Specialized classes retain the public `KomorebiMathError` lineage.

Evidence and links (`V, L`). Negative-path tests assert the specific classes for malformed structures, dimension mismatches, unavailable optional backends, exactness violations, and unmet assumptions. See Chapters 6, 8, 37, 43, 61, and Appendix I.

B.1 · `DimensionError`

Identity (`q, a, t, k`). Canonical beta route `komorebi_math.beta.DimensionError`. No identity-equal re-export in the six audited root/domain namespaces. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `class`. Reflected implementation `komorebi_math.exceptions.DimensionError`. Profile `B1-X1`.

Manifest signature (`s`).

```
null
```

Definition. Raised when matrix, vector, or packet dimensions do not agree.

Contract (`D, C, H, R, X`). The domain is an error condition raised by a public Komorebi operation. The codomain is an exception object carrying the caller-supplied message and ordinary Python exception context. There are no mathematical hypotheses. Constructing an exception does not raise package-specific errors. The originating operation determines when it is emitted.

Guarantees (E, P, B, K). The hierarchy provides deterministic error classification, not recovery or proof. It has no optional dependencies or backend behavior and constant construction cost apart from the message and traceback. `ValidationError` also derives from `ValueError`. Specialized classes retain the public `KomorebiMathError` lineage.

Evidence and links (V, L). Negative-path tests assert the specific classes for malformed structures, dimension mismatches, unavailable optional backends, exactness violations, and unmet assumptions. See Chapters 6, 8, 37, 43, 61, and Appendix I.

B.1 · `BackendUnavailableError`

Identity (q, a, t, k). Canonical beta route `komorebi_math.beta.BackendUnavailableError`. No identity-equal re-export in the six audited root/domain namespaces. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `class`. Reflected implementation `komorebi_math.exceptions.BackendUnavailableError`. Profile `B1-X1`.

Manifest signature (s).

```
null
```

Definition. Raised when an optional computation backend is not installed.

Contract (D, C, H, R, X). The domain is an error condition raised by a public Komorebi operation. The codomain is an exception object carrying the caller-supplied message and ordinary Python exception context. There are no mathematical hypotheses. Constructing an exception does not raise package-specific errors. The originating operation determines when it is emitted.

Guarantees (E, P, B, K). The hierarchy provides deterministic error classification, not recovery or proof. It has no optional dependencies or backend behavior and constant construction cost apart from the message and traceback. `ValidationError` also derives from `ValueError`. Specialized classes retain the public `KomorebiMathError` lineage.

Evidence and links (V, L). Negative-path tests assert the specific classes for malformed structures, dimension mismatches, unavailable optional backends, exactness violations, and unmet assumptions. See Chapters 6, 8, 37, 43, 61, and Appendix I.

B.1 · `ExactnessError`

Identity (q, a, t, k). Canonical beta route `komorebi_math.beta.ExactnessError`. No identity-equal re-export in the six audited root/domain namespaces. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `class`. Reflected implementation `komorebi_math.exceptions.ExactnessError`. Profile `B1-X1`.

Manifest signature (s).

```
null
```

Definition. Raised when an exact operation is requested from an inexact backend.

Contract (D, C, H, R, X). The domain is an error condition raised by a public Komorebi operation. The codomain is an exception object carrying the caller-supplied message and ordinary Python exception context. There are no mathematical hypotheses. Constructing an exception does not raise package-specific errors. The originating operation determines when it is emitted.

Guarantees (E, P, B, K). The hierarchy provides deterministic error classification, not recovery or proof. It has no optional dependencies or backend behavior and constant construction cost apart from the message and traceback. `ValidationError` also derives from `ValueError`. Specialized classes retain the public `KomorebiMathError` lineage.

Evidence and links (V, L). Negative-path tests assert the specific classes for malformed structures, dimension mismatches, unavailable optional backends, exactness violations, and unmet assumptions. See Chapters 6, 8, 37, 43, 61, and Appendix I.

B.1 · AssumptionError

Identity (q, a, t, k). Canonical beta route `komorebi_math.beta.AssumptionError`. No identity-equal re-export in the six audited root/domain namespaces. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `class`. Reflected implementation `komorebi_math.exceptions.AssumptionError`. Profile `B1-X1`.

Manifest signature (s).

```
null
```

Definition. Raised when a required mathematical assumption is not satisfied.

Contract (D, C, H, R, X). The domain is an error condition raised by a public Komorebi operation. The codomain is an exception object carrying the caller-supplied message and ordinary Python exception context. There are no mathematical hypotheses. Constructing an exception does not raise package-specific errors. The originating operation determines when it is emitted.

Guarantees (E, P, B, K). The hierarchy provides deterministic error classification, not recovery or proof. It has no optional dependencies or backend behavior and constant construction cost apart from the message and traceback. `ValidationError` also derives from `ValueError`. Specialized classes retain the public `KomorebiMathError` lineage.

Evidence and links (V, L). Negative-path tests assert the specific classes for malformed structures, dimension mismatches, unavailable optional backends, exactness violations, and unmet assumptions. See Chapters 6, 8, 37, 43, 61, and Appendix I.

B.1 · PacketSystem

Identity (q, a, t, k). Canonical beta route `komorebi_math.beta.PacketSystem`. Identity-equal audited re-exports `komorebi_math.PacketSystem`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `class`. Reflected implementation `komorebi_math.core.packets.PacketSystem`. Profile `B1-P1`.

Manifest signature (s).

```
(projectors: 'Sequence[Any]', *, names: 'Sequence[str] | None' = None, backend: 'str | LinearAlgebraBackend | None'
    ↪  = None, tolerance: 'float | None' = None, validate: 'bool | None' = None) -> 'None'
```

Definition. A complete finite family of pairwise annihilating idempotents. Runtime-visible operational members: `aggregate`, `analyze`, `block_diagonal_projection`, `certificate`, `dual`, `equivalent`, `hilbert_certificate`, `index`, `packet_dimensions`, `quotient`, `restrict`, `support`, `synthesize`, `tensor`, `transport`.

Contract (D, C, H, R, X). A finite sequence of square projectors, optional names, backend selection, tolerance, and validation policy maps to a complete packet decomposition. Projectors must share dimension, be idempotent, annihilate pairwise, and sum to identity under the selected exact or numerical policy. Invalid shapes, duplicate names, failed algebraic checks, or unavailable backends raise the documented validation, dimension, or backend errors.

Guarantees (E, P, B, K). With exact arithmetic the packet identities are exact. Numerical backends certify residuals against the selected tolerance. The object supports analysis, synthesis, support, restriction, quotient, dual, tensor, transport, and block operations. Construction and validation scale with the packet count and matrix products. Materialized matrices require ambient quadratic storage.

Evidence and links (V, L). The constructor certificate uses theorem identifier `PSC-I-T20`. Packet analysis-synthesis, equivalence, tensor, and transport tests cover the public operations. See Chapters 10 through 12, 38, 43, and 64.

B.1 · PublicApiSymbol

Identity (q, a, t, k). Canonical beta route `komorebi_math.beta.PublicApiSymbol`. Identity-equal audited re-exports `komorebi_math.PublicApiSymbol`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `class`. Reflected implementation `komorebi_math.compatibility.PublicApiSymbol`. Profile `B1-A1`.

Manifest signature (s).

```
(namespace: 'str', name: 'str', kind: 'str', implementation_module: 'str', qualname: 'str', signature: 'str |
   ↪ None') -> None
```

Definition. One exported public symbol in a deterministic API manifest. Stored fields: `namespace`, `name`, `kind`, `implementation_module`, `qualname`, `signature`. Runtime-visible operational members: `to_dict`.

Contract (D, C, H, R, X). The records map reflected public exports to deterministic manifest rows, aggregate rows into a validated manifest, and report structural differences between reference and current manifests. Namespaces must be nonempty and unique. Symbols must be unique, sorted, and inside the declared namespace set. Malformed fields, duplicate symbols, unstable ordering, namespace mismatch, or incompatible changes raise `ValidationError` or produce an invalid report certificate.

Guarantees (E, P, B, K). Canonical dictionaries and SHA-256 digests are deterministic under compact sorted-key JSON. The report policy distinguishes additions, removals, broad kind changes, and reflected signature changes. Structural equality is not mathematical or behavioral equivalence. Work and memory are linear in the number of exported symbols plus sorting cost.

Evidence and links (V, L). The strict beta check is exercised by API tests and `verify streaming-beta`. The reference 83-name manifest reports zero added, removed, kind-changed, or signature-changed entries under theorem identifier `KM-API-010`. See Chapters 7, 72, 75, and 76.

B.1 · `PublicApiManifest`

Identity (q, a, t, k). Canonical beta route `komorebi_math.beta.PublicApiManifest`. Identity-equal audited re-exports `komorebi_math.PublicApiManifest`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `class`. Reflected implementation `komorebi_math.compatibility.PublicApiManifest`. Profile `B1-A1`.

Manifest signature (s).

```
(package_version: 'str', namespaces: 'tuple[str, ...]', symbols: 'tuple[PublicApiSymbol, ...]', schema: 'str' =
   ↪ 'komorebi-math.public-api', schema_version: 'int' = 1, stability_level: 'str' = 'pre-beta-additive') -> None
```

Definition. A deterministic snapshot of selected `__all__` namespaces. The `digest` property hashes canonical compact JSON. `symbol_map()` keys rows by `(namespace, name)`. Stored fields: `package_version`, `namespaces`, `symbols`, `schema`, `schema_version`, `stability_level`. Runtime-visible operational members: `digest`, `symbol_map`, `to_dict`.

Contract (D, C, H, R, X). The records map reflected public exports to deterministic manifest rows, aggregate rows into a validated manifest, and report structural differences between reference and current manifests. Namespaces must be nonempty and unique. Symbols must be unique, sorted, and inside the declared namespace set. Malformed fields, duplicate symbols, unstable ordering, namespace mismatch, or incompatible changes raise `ValidationError` or produce an invalid report certificate.

Guarantees (E, P, B, K). Canonical dictionaries and SHA-256 digests are deterministic under compact sorted-key JSON. The report policy distinguishes additions, removals, broad kind changes, and reflected signature changes. Structural equality is not mathematical or behavioral equivalence. Work and memory are linear in the number of exported symbols plus sorting cost.

Evidence and links (V, L). The strict beta check is exercised by API tests and `verify streaming-beta`. The reference 83-name manifest reports zero added, removed, kind-changed, or signature-changed entries under theorem identifier `KM-API-010`. See Chapters 7, 72, 75, and 76.

B.1 · `ApiCompatibilityReport`

Identity (q, a, t, k). Canonical beta route `komorebi_math.beta.ApiCompatibilityReport`. Identity-equal audited re-exports `komorebi_math.ApiCompatibilityReport`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `class`. Reflected implementation `komorebi_math.compatibility.ApiCompatibilityReport`. Profile `B1-A1`.

Manifest signature (s).

```
(reference_digest: 'str', current_digest: 'str', added: 'tuple[tuple[str, str], ...]', removed: 'tuple[tuple[str,
    ↪ str], ...]', kind_changed: 'tuple[tuple[str, str, str, str], ...]', signature_changed: 'tuple[tuple[str,
    ↪ str, str | None, str | None], ...]', strict_signatures: 'bool', certificate: 'Certificate') -> None
```

Definition. Structural differences between a reference and current API manifest. The `compatible` property mirrors the certificate verdict. `to_dict()` preserves all four change ledgers and both digests. Stored fields: `reference_digest`, `current_digest`, `added`, `removed`, `kind_changed`, `signature_changed`, `strict_signatures`, `certificate`. Runtime-visible operational members: `compatible`, `to_dict`.

Contract (D, C, H, R, X). The records map reflected public exports to deterministic manifest rows, aggregate rows into a validated manifest, and report structural differences between reference and current manifests. Namespaces must be nonempty and unique. Symbols must be unique, sorted, and inside the declared namespace set. Malformed fields, duplicate symbols, unstable ordering, namespace mismatch, or incompatible changes raise `ValidationError` or produce an invalid report certificate.

Guarantees (E, P, B, K). Canonical dictionaries and SHA-256 digests are deterministic under compact sorted-key JSON. The report policy distinguishes additions, removals, broad kind changes, and reflected signature changes. Structural equality is not mathematical or behavioral equivalence. Work and memory are linear in the number of exported symbols plus sorting cost.

Evidence and links (V, L). The strict beta check is exercised by API tests and `verify streaming-beta`. The reference 83-name manifest reports zero added, removed, kind-changed, or signature-changed entries under theorem identifier `KM-API-010`. See Chapters 7, 72, 75, and 76.

B.1 · `create_beta_api_manifest`

Identity (q, a, t, k). Canonical beta route `komorebi_math.beta.create_beta_api_manifest`. Identity-equal audited re-exports `komorebi_math.create_beta_api_manifest`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `function`. Reflected implementation `komorebi_math.compatibility.create_beta_api_manifest`. Profile `B1-A2`.

Manifest signature (s).

```
(*, package_version: 'str | None' = None) -> 'PublicApiManifest'
```

Definition. Build the curated beta namespace manifest with strict signatures.

Contract (D, C, H, R, X). The creation route reflects the curated `komorebi_math.beta.__all__` namespace. The loader reads and validates the committed manifest. The checker compares that reference with the installed package using strict signatures. Optional paths must identify a valid public-API JSON document. Invalid schema, duplicate keys or symbols, wrong namespace or stability level, unreadable files, and incompatible removals, kind changes, or signatures are reported through `ValidationError` or an invalid compatibility certificate.

Guarantees (E, P, B, K). The beta policy permits additive exports while treating removal, broad kind change, and reflected signature change as release failures. Canonical manifest generation is deterministic, but it intentionally leaves behavior and undocumented implementation details unchecked. The implementation uses reflection, JSON, SHA-256, and sorting. Cost is quasilinear in the 83-row inventory.

Evidence and links (V, L). The recorded reference audit reran `check_beta_api()` and required theorem `KM-API-010`, strict signatures, the canonical digest `311f2bf7cbcf0a45bcbe2af433d73a8f597c0fbf7133750882314d6e0edb16b2`, and zero incompatible changes. See Chapters 7, 72, 75, and 76.

B.1 · `load_beta_api_manifest`

Identity (q, a, t, k). Canonical beta route `komorebi_math.beta.load_beta_api_manifest`. Identity-equal audited re-exports `komorebi_math.load_beta_api_manifest`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `function`. Reflected implementation `komorebi_math.compatibility.load_beta_api_manifest`. Profile `B1-A2`.

Manifest signature (s).

```
(path: 'str | Path | None' = None) -> 'PublicApiManifest'
```

Definition. Load and validate the committed curated beta API manifest.

Contract (D, C, H, R, X). The creation route reflects the curated `komorebi_math.beta.__all__` namespace. The loader reads and validates the committed manifest. The checker compares that reference with the installed package using strict signatures. Optional paths must identify a valid public-API JSON document. Invalid schema, duplicate keys or symbols, wrong namespace or stability level, unreadable files, and incompatible removals, kind changes, or signatures are reported through `ValidationError` or an invalid compatibility certificate.

Guarantees (E, P, B, K). The beta policy permits additive exports while treating removal, broad kind change, and reflected signature change as release failures. Canonical manifest generation is deterministic, but it intentionally leaves behavior and undocumented implementation details unchecked. The implementation uses reflection, JSON, SHA-256, and sorting. Cost is quasilinear in the 83-row inventory.

Evidence and links (V, L). The recorded reference audit reran `check_beta_api()` and required theorem `KM-API-010`, strict signatures, the canonical digest `311f2bf7cbcf0a45bcbe2af433d73a8f597c0fbf7133750882314d6e0edb16b2`, and zero incompatible changes. See Chapters 7, 72, 75, and 76.

B.1 · `check_beta_api`

Identity (q, a, t, k). Canonical beta route `komorebi_math.beta.check_beta_api`. Identity-equal audited re-exports `komorebi_math.check_beta_api`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `function`. Reflected implementation `komorebi_math.compatibility.check_beta_api`. Profile `B1-A2`.

Manifest signature (s).

```
(path: 'str | Path | None' = None) -> 'ApiCompatibilityReport'
```

Definition. Check the curated beta namespace under strict signature compatibility. The default reference is the committed curated beta manifest, and strict signatures are always enabled.

Contract (D, C, H, R, X). The creation route reflects the curated `komorebi_math.beta.__all__` namespace. The loader reads and validates the committed manifest. The checker compares that reference with the installed package using strict signatures. Optional paths must identify a valid public-API JSON document. Invalid schema, duplicate keys or symbols, wrong namespace or stability level, unreadable files, and incompatible removals, kind changes, or signatures are reported through `ValidationError` or an invalid compatibility certificate.

Guarantees (E, P, B, K). The beta policy permits additive exports while treating removal, broad kind change, and reflected signature change as release failures. Canonical manifest generation is deterministic, but it intentionally leaves behavior and undocumented implementation details unchecked. The implementation uses reflection, JSON, SHA-256, and sorting. Cost is quasilinear in the 83-row inventory.

Evidence and links (V, L). The recorded reference audit reran `check_beta_api()` and required theorem `KM-API-010`, strict signatures, the canonical digest `311f2bf7cbcf0a45bcbe2af433d73a8f597c0fbf7133750882314d6e0edb16b2`, and zero incompatible changes. See Chapters 7, 72, 75, and 76.

B.1 · `BehavioralGoldenCaseResult`

Identity (q, a, t, k). Canonical beta route `komorebi_math.beta.BehavioralGoldenCaseResult`. Identity-equal audited re-exports `komorebi_math.BehavioralGoldenCaseResult`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `class`. Reflected implementation `komorebi_math.golden.BehavioralGoldenCaseResult`. Profile `B1-G1`.

Manifest signature (`s`).

```
(name: 'str', kind: 'str', observed: 'Any', expected: 'Any', certificate: 'Certificate') -> None
```

Definition. Observed and expected data for one closed golden case. Stored fields: `name`, `kind`, `observed`, `expected`, `certificate`. Runtime-visible operational members: `passed`, `to_dict`.

Contract (`D, C, H, R, X`). A case result stores one closed case name and kind, observed and expected canonical values, and its certificate. A report stores fixture path and digest, the ordered case results, and an aggregate certificate. Inputs must already satisfy the closed fixture evaluator contract. Serialization exposes JSON-compatible mappings. A failed case yields `passed == False`. Malformed fixture data is rejected before these records are returned.

Guarantees (`E, P, B, K`). The records preserve deterministic finite regression evidence. They prove only the enumerated committed cases and explicitly do not make the fixture exhaustive. No optional backend is required for the committed fixture. Storage is linear in the number and size of the case records.

Evidence and links (`V, L`). The reference fixture contains 15 cases and has digest `13bd8c499838593c9d9242ac10210f0695e16a690c9270d68f321c41d85835ea`. The aggregate certificate uses `KM-GOLDEN-011`. See Chapters 72 and 75 and `tests/test_block11_golden_profiling_beta.py`.

B.1 · `BehavioralGoldenReport`

Identity (`q, a, t, k`). Canonical beta route `komorebi_math.beta.BehavioralGoldenReport`. Identity-equal audited re-exports `komorebi_math.BehavioralGoldenReport`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `class`. Reflected implementation `komorebi_math.golden.BehavioralGoldenReport`. Profile `B1-G1`.

Manifest signature (`s`).

```
(fixture_path: 'Path', fixture_digest: 'str', results: 'tuple[BehavioralGoldenCaseResult, ...]', certificate:
    ↪ 'Certificate') -> None
```

Definition. Aggregate deterministic semantic regression report. The `passed` property requires the aggregate certificate and every case result to pass. Stored fields: `fixture_path`, `fixture_digest`, `results`, `certificate`. Runtime-visible operational members: `passed`, `to_dict`.

Contract (`D, C, H, R, X`). A case result stores one closed case name and kind, observed and expected canonical values, and its certificate. A report stores fixture path and digest, the ordered case results, and an aggregate certificate. Inputs must already satisfy the closed fixture evaluator contract. Serialization exposes JSON-compatible mappings. A failed case yields `passed == False`. Malformed fixture data is rejected before these records are returned.

Guarantees (`E, P, B, K`). The records preserve deterministic finite regression evidence. They prove only the enumerated committed cases and explicitly do not make the fixture exhaustive. No optional backend is required for the committed fixture. Storage is linear in the number and size of the case records.

Evidence and links (`V, L`). The reference fixture contains 15 cases and has digest `13bd8c499838593c9d9242ac10210f0695e16a690c9270d68f321c41d85835ea`. The aggregate certificate uses `KM-GOLDEN-011`. See Chapters 72 and 75 and `tests/test_block11_golden_profiling_beta.py`.

B.1 · `load_behavioral_golden_fixture`

Identity (`q, a, t, k`). Canonical beta route `komorebi_math.beta.load_behavioral_golden_fixture`. Identity-equal audited re-exports `komorebi_math.load_behavioral_golden_fixture`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `function`. Reflected implementation `komorebi_math.golden.load_behavioral_golden_fixture`. Profile `B1-G2`.

Manifest signature (s).

```
(path: 'str | Path | None' = None) -> 'tuple[Path, dict[str, Any], str]'
```

Definition. Load and validate the closed behavioral fixture document.

Contract (D, C, H, R, X). The loader accepts the default committed fixture or an explicit path, rejects duplicate JSON keys, validates the fixed schema and closed case document, and returns path, decoded mapping, and canonical digest. The runner dispatches only through the built-in case-kind registry and returns one aggregate report. Unknown kinds, malformed values, failed schema checks, unreadable files, or evaluator errors raise `ValidationError`. Mismatched expected values produce failed case and aggregate certificates.

Guarantees (E, P, B, K). Execution is deterministic for the committed exact and canonicalized numerical forms. It is a representative finite semantic regression suite rather than a proof of all package behavior. The default fixture exercises 15 cases. Time is the sum of those bounded computations and memory is linear in fixture and result size.

Evidence and links (V, L). The recorded reference audit executed all 15 committed cases and required an exact passing `KM-GOLDEN-011` certificate. The fixture spans packet snapshots, sparse and cached execution, migration and streaming, generating-function arithmetic, coordinates, and formal roundtrips. See Chapter 72.

B.1 · `run_behavioral_golden_fixture`

Identity (q, a, t, k). Canonical beta route `komorebi_math.beta.run_behavioral_golden_fixture`. Identity-equal audited re-exports `komorebi_math.run_behavioral_golden_fixture`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `function`. Reflected implementation `komorebi_math.golden.run_behavioral_golden_fixture`. Profile `B1-G2`.

Manifest signature (s).

```
(path: 'str | Path | None' = None) -> 'BehavioralGoldenReport'
```

Definition. Execute every committed semantic fixture through its closed evaluator. The current closed evaluator covers 15 committed case kinds and does not execute import paths supplied by JSON.

Contract (D, C, H, R, X). The loader accepts the default committed fixture or an explicit path, rejects duplicate JSON keys, validates the fixed schema and closed case document, and returns path, decoded mapping, and canonical digest. The runner dispatches only through the built-in case-kind registry and returns one aggregate report. Unknown kinds, malformed values, failed schema checks, unreadable files, or evaluator errors raise `ValidationError`. Mismatched expected values produce failed case and aggregate certificates.

Guarantees (E, P, B, K). Execution is deterministic for the committed exact and canonicalized numerical forms. It is a representative finite semantic regression suite rather than a proof of all package behavior. The default fixture exercises 15 cases. Time is the sum of those bounded computations and memory is linear in fixture and result size.

Evidence and links (V, L). The recorded reference audit executed all 15 committed cases and required an exact passing `KM-GOLDEN-011` certificate. The fixture spans packet snapshots, sparse and cached execution, migration and streaming, generating-function arithmetic, coordinates, and formal roundtrips. See Chapter 72.

B.2 Nonlinear derivatives, jets, sparse storage, execution, and profiling

B.2 · DerivativeProvider

Identity (q, a, t, k). Canonical beta route `komorebi_math.beta.DerivativeProvider`. Identity-equal audited re-exports `komorebi_math.nonlinear.DerivativeProvider`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `class`. Reflected implementation `komorebi_math.nonlinear.derivatives.DerivativeProvider`. Profile `B2-D0`.

Manifest signature (s).

```
(*args, **kwargs)
```

Definition. Protocol implemented by finite dimensional derivative sources. It is a runtime-checkable structural protocol rather than an instantiable mathematical implementation. Runtime-visible operational members: `derivative`, `supports_order`, `value`.

Contract (D, C, H, R, X). This runtime protocol describes finite-dimensional derivative sources with `source_dimension`, `target_dimension`, `exactness`, and `name`, plus `value(point)`, `derivative(order, point, directions)`, and `supports_order(order)`. Implementations must accept an ordered direction tuple of the declared order and return a target-space value. A protocol declaration alone does not validate mathematics. Concrete providers define dimension, order, and exactness failures.

Guarantees (E, P, B, K). The protocol enables structural typing and backend-independent jet construction. It makes no implementation, complexity, persistence, or numerical-accuracy guarantee. Cost and dependencies are those of the concrete provider.

Evidence and links (V, L). Runtime protocol conformance is exercised through callable, finite-difference, polynomial, sparse, automatic-differentiation, and downstream packet-jet tests. See Chapter 37.

B.2 · CallableDerivativeProvider

Identity (q, a, t, k). Canonical beta route `komorebi_math.beta.CallableDerivativeProvider`. Identity-equal audited re-exports `komorebi_math.CallableDerivativeProvider`, `komorebi_math.nonlinear.CallableDerivativeProvider`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `class`. Reflected implementation `komorebi_math.nonlinear.derivatives.CallableDerivativeProvider`. Profile `B2-D1`.

Manifest signature (s).

```
(function: 'Callable[[Any], Any]', derivatives: 'Mapping[int, DirectionalDerivative]', source_dimension: 'int',
   ↪ target_dimension: 'int', exactness: 'Exactness' = <Exactness.NUMERICAL: 'numerical'>, name: 'str' =
   ↪ 'declared derivative provider') -> None
```

Definition. A map with explicitly supplied directional derivative callables. The `available_orders` property is the sorted declared derivative-order set. Stored fields: `function`, `derivatives`, `source_dimension`, `target_dimension`, `exactness`, `name`. Runtime-visible operational members: `available_orders`, `derivative`, `supports_order`, `validation_certificate`, `value`.

Contract (D, C, H, R, X). A function, explicit map from derivative order to directional-derivative callable, positive source and target dimensions, exactness label, and name define a declared derivative source. Each derivative callable receives `(point, directions)`. Unsupported orders, incorrect dimensions, malformed direction tuples, and failed optional validation probes are rejected or certified explicitly.

Guarantees (E, P, B, K). The provider preserves the declared exactness but does not infer correctness from user callables. `validation_certificate` performs only selected finite symmetry and optional Taylor probes. Runtime and memory depend on the supplied callables. No automatic fallback or codec is promised.

Evidence and links (V, L). Derivative-provider tests cover construction, order dispatch, dimension rejection, symmetry probes, and packet-jet use. Certificates from finite probes use `PSC-III-D02`. See Chapters 37 and 38.

B.2 · FiniteDifferenceDerivativeProvider

Identity (q, a, t, k). Canonical beta route `komorebi_math.beta.FiniteDifferenceDerivativeProvider`. Identity-equal audited re-exports `komorebi_math.FiniteDifferenceDerivativeProvider`, `komorebi_math.nonlinear.FiniteDifferenceDerivativeProvider`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `class`. Reflected implementation `komorebi_math.nonlinear.derivatives.FiniteDifferenceDerivativeProvider`. Profile `B2-D2`.

Manifest signature (s).

```
(function: 'Callable[[np.ndarray], Any]', source_dimension: 'int', target_dimension: 'int', step: 'float' =
    ↪ 0.0001, max_order: 'int' = 4, name: 'str' = 'central finite difference provider', exactness: 'Exactness' =
    ↪ <Exactness.NUMERICAL: 'numerical'>) -> None
```

Definition. Numerical directional derivatives by recursive central differences. The default step is `1e-4` and default maximum order is 4. Stored fields: `function`, `source_dimension`, `target_dimension`, `step`, `max_order`, `name`, `exactness`. Runtime-visible operational members: `derivative`, `supports_order`, `value`.

Contract (D, C, H, R, X). A NumPy-compatible function, positive dimensions, positive finite step, nonnegative maximum order, name, and numerical exactness define recursive central directional differences. Points and directions must have the declared source dimension, and the function output must match the target dimension. Unsupported order, invalid step, shape mismatch, or nonfinite setup raises validation or dimension errors.

Guarantees (E, P, B, K). Results are numerical approximations controlled by the supplied step. No exactness or universal truncation-error bound is claimed. The recursive rule uses approximately `2^r` function evaluations for derivative order `r`, before cost inside the user function. It uses NumPy and has no GPU fallback in this provider.

Evidence and links (V, L). Finite-difference tests compare low-order derivatives against known functions and exercise failures and packet projections. See Chapters 8, 37, 38, and 43.

B.2 · PolynomialDerivativeProvider

Identity (q, a, t, k). Canonical beta route `komorebi_math.beta.PolynomialDerivativeProvider`. Identity-equal audited re-exports `komorebi_math.PolynomialDerivativeProvider`, `komorebi_math.nonlinear.PolynomialDerivativeProvider`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `class`. Reflected implementation `komorebi_math.nonlinear.derivatives.PolynomialDerivativeProvider`. Profile `B2-D3`.

Manifest signature (s).

```
(tensors: 'Mapping[int, Any]', *, source_dimension: 'int | None' = None, target_dimension: 'int | None' = None,
    ↪ exactness: 'Exactness' = <Exactness.NUMERICAL: 'numerical'>, tolerance: 'float' = 1e-10, name: 'str' =
    ↪ 'polynomial derivative provider') -> 'None'
```

Definition. A finite polynomial map represented by symmetric coefficient tensors. The `degree` property is the largest retained tensor order. Runtime-visible operational members: `degree`, `derivative`, `supports_order`, `symmetry_certificate`, `value`.

Contract (D, C, H, R, X). A finite map of symmetric coefficient tensors defines `F(x) = sum_r T_r[x,...,x]/r!`. `T_0` is a target vector and order-`r` tensors have one target axis followed by `r` source axes. Dimensions may be inferred or declared. Inconsistent tensor shapes, negative orders, failed symmetry, or invalid tolerance are rejected.

Guarantees (E, P, B, K). Exact input tensors may produce exact values and derivatives. Numerical tensors use tolerance-aware symmetry evidence. Evaluation contracts each retained tensor with the point or direction vectors. Dense storage grows as the sum of `target_dimension * source_dimension^r`. There is no hidden sparse or GPU fallback.

Evidence and links (V, L). Polynomial derivative tests verify value and directional derivatives, symmetry certificates under `PSC-III-D04`, and equivalence with sparse and packet-jet execution. See Chapters 37, 38, and 43.

B.2 · PacketJetAddress

Identity (q, a, t, k). Canonical beta route `komorebi_math.beta.PacketJetAddress`. Identity-equal audited re-exports `komorebi_math.nonlinear.PacketJetAddress`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `class`. Reflected implementation `komorebi_math.nonlinear.jets.PacketJetAddress`. Profile `B2-J1`.

Manifest signature (s).

```
(output: 'str', inputs: 'tuple[str, ...]') -> None
```

Definition. The output and ordered input packet addresses of one jet block. The `order` property is the length of the ordered input packet tuple. Stored fields: `output`, `inputs`. Runtime-visible operational members: `order`, `to_dict`.

Contract (D, C, H, R, X). Address records identify one target packet and an ordered tuple of source packets. A packet jet combines a derivative provider, base point, source and target packet systems, one output index, and ordered input indices. Evaluation projects directions before calling the derivative and projects the result afterward. A family represents every address of one order and reconstructs the full derivative by summation. Invalid packet names or indices, dimensions, order, provider support, or backend compatibility raise explicit validation or dimension errors.

Guarantees (E, P, B, K). Exactness follows the provider and packet backends. Projection identities and family reconstruction are exact for exact systems and tolerance-certified otherwise. One jet evaluation performs one derivative call plus input and output projections. A full family has `target_packets * source_packets^order` addresses unless support pruning is applied.

Evidence and links (V, L). Jet block and family certificates use `PSC-III-D06` and `PSC-III-T07`. Tests verify projection, symmetry, reconstruction, serialization views, and exact/numerical behavior. See Chapters 38, 43, 64, and 67.

B.2 · PacketJet

Identity (q, a, t, k). Canonical beta route `komorebi_math.beta.PacketJet`. Identity-equal audited re-exports `komorebi_math.PacketJet`, `komorebi_math.nonlinear.PacketJet`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `class`. Reflected implementation `komorebi_math.nonlinear.jets.PacketJet`. Profile `B2-J1`.

Manifest signature (s).

```
(provider: 'DerivativeProvider', point: 'Any', source: 'PacketSystem', target: 'PacketSystem', output_index:
    ↪ 'int', input_indices: 'tuple[int, ...]') -> None
```

Definition. One projected multilinear derivative block. Operational members include `evaluate`, `norm_bound`, `symmetry_certificate`, and `to_dict`. Stored fields: `provider`, `point`, `source`, `target`, `output_index`, `input_indices`. Runtime-visible operational members: `address`, `exactness`, `order`, `evaluate`, `norm_bound`, `symmetry_certificate`, `to_dict`.

Contract (D, C, H, R, X). Address records identify one target packet and an ordered tuple of source packets. A packet jet combines a derivative provider, base point, source and target packet systems, one output index, and ordered input indices. Evaluation projects directions before calling the derivative and projects the result afterward. A family represents every address of one order and reconstructs the full derivative by summation. Invalid packet names or indices, dimensions, order, provider support, or backend compatibility raise explicit validation or dimension errors.

Guarantees (E, P, B, K). Exactness follows the provider and packet backends. Projection identities and family reconstruction are exact for exact systems and tolerance-certified otherwise. One jet evaluation performs one derivative call plus input and output projections. A full family has `target_packets * source_packets^order` addresses unless support pruning is applied.

Evidence and links (V, L). Jet block and family certificates use `PSC-III-D06` and `PSC-III-T07`. Tests verify projection, symmetry, reconstruction, serialization views, and exact/numerical behavior. See Chapters 38, 43, 64, and 67.

B.2 · PacketJetFamily

Identity (q, a, t, k). Canonical beta route `komorebi_math.beta.PacketJetFamily`. Identity-equal audited re-exports `komorebi_math.PacketJetFamily`, `komorebi_math.nonlinear.PacketJetFamily`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `class`. Reflected implementation `komorebi_math.nonlinear.jets.PacketJetFamily`. Profile `B2-J1`.

Manifest signature (s).

```
(provider: 'DerivativeProvider', point: 'Any', source: 'PacketSystem', target: 'PacketSystem', order: 'int') ->
   ↪ 'None'
```

Definition. Every packet block of one derivative order at one base point. The family exposes deterministic address enumeration, per-address jet construction, evaluation, and reconstruction of the full derivative. Runtime-visible operational members: `addresses`, `jet`, `reconstruct`, `reconstruction_certificate`, `to_dict`.

Contract (D, C, H, R, X). Address records identify one target packet and an ordered tuple of source packets. A packet jet combines a derivative provider, base point, source and target packet systems, one output index, and ordered input indices. Evaluation projects directions before calling the derivative and projects the result afterward. A family represents every address of one order and reconstructs the full derivative by summation. Invalid packet names or indices, dimensions, order, provider support, or backend compatibility raise explicit validation or dimension errors.

Guarantees (E, P, B, K). Exactness follows the provider and packet backends. Projection identities and family reconstruction are exact for exact systems and tolerance-certified otherwise. One jet evaluation performs one derivative call plus input and output projections. A full family has `target_packets * source_packets^order` addresses unless support pruning is applied.

Evidence and links (V, L). Jet block and family certificates use `PSC-III-D06` and `PSC-III-T07`. Tests verify projection, symmetry, reconstruction, serialization views, and exact/numerical behavior. See Chapters 38, 43, 64, and 67.

B.2 · PacketJetSupport

Identity (q, a, t, k). Canonical beta route `komorebi_math.beta.PacketJetSupport`. Identity-equal audited re-exports `komorebi_math.nonlinear.PacketJetSupport`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `class`. Reflected implementation `komorebi_math.nonlinear.jets.PacketJetSupport`. Profile `B2-J2`.

Manifest signature (s).

```
(order: 'int', nonzero: 'tuple[PacketJetAddress, ...]', zero: 'tuple[PacketJetAddress, ...]', untested:
   ↪ 'tuple[PacketJetAddress, ...]', evaluations: 'int', certificate: 'Certificate') -> None
```

Definition. Finite basis test for zero and nonzero packet derivative blocks. The `complete` property is true exactly when `untested` is empty. `contains()` classifies a specific address. Stored fields: `order`, `nonzero`, `zero`, `untested`, `evaluations`, `certificate`. Runtime-visible operational members: `complete`, `contains`.

Contract (D, C, H, R, X). The classifier exhaustively tests packet-range basis tuples for every family address up to `max_evaluations`, returning disjoint nonzero, zero, and untested address tuples plus a certificate. The family, tolerance, and budget must be valid. Insufficient budget is represented by explicit untested addresses rather than a false zero classification. Invalid tolerance, dimensions, or limits raise validation errors.

Guarantees (E, P, B, K). Multilinearity makes a complete exact-backend classification exact. Numerical backends use a declared tolerance and report bounded finite evidence. The worst-case number of basis evaluations is the sum of packet-range dimension products across addresses and is stopped by the explicit budget.

Evidence and links (V, L). Support tests cover complete, truncated, exact, and numerical classifications under theorem identifiers `PSC-III-C08` and `PSC-III-C09`, then verify that execution prunes only certified zero blocks. See Chapters 38 and 43.

B.2 · packet_jet_support

Identity (q, a, t, k). Canonical beta route `komorebi_math.beta.packet_jet_support`. Identity-equal audited re-exports `komorebi_math.packet_jet_support`, `komorebi_math.nonlinear.packet_jet_support`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `function`. Reflected implementation `komorebi_math.nonlinear.jets.packet_jet_support`. Profile `B2-J2`.

Manifest signature (s).

```
(family: 'PacketJetFamily', *, tolerance: 'float | None' = None, max_evaluations: 'int' = 100000) ->
    ↪ 'PacketJetSupport'
```

Definition. Classify packet jet blocks by exhaustive packet-range basis tests.

Contract (D, C, H, R, X). The classifier exhaustively tests packet-range basis tuples for every family address up to `max_evaluations`, returning disjoint nonzero, zero, and untested address tuples plus a certificate. The family, tolerance, and budget must be valid. Insufficient budget is represented by explicit untested addresses rather than a false zero classification. Invalid tolerance, dimensions, or limits raise validation errors.

Guarantees (E, P, B, K). Multilinearity makes a complete exact-backend classification exact. Numerical backends use a declared tolerance and report bounded finite evidence. The worst-case number of basis evaluations is the sum of packet-range dimension products across addresses and is stopped by the explicit budget.

Evidence and links (V, L). Support tests cover complete, truncated, exact, and numerical classifications under theorem identifiers `PSC-III-C08` and `PSC-III-C09`, then verify that execution prunes only certified zero blocks. See Chapters 38 and 43.

B.2 · SparseTensorEntry

Identity (q, a, t, k). Canonical beta route `komorebi_math.beta.SparseTensorEntry`. Identity-equal audited re-exports `komorebi_math.SparseTensorEntry`, `komorebi_math.nonlinear.SparseTensorEntry`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `class`. Reflected implementation `komorebi_math.nonlinear.sparse.SparseTensorEntry`. Profile `B2-S1`.

Manifest signature (s).

```
(output: 'int', inputs: 'tuple[int, ...]', value: 'Any') -> None
```

Definition. One coefficient of a symmetric tensor orbit. The coefficient applies to every distinct ordered permutation of its nondecreasing source-index tuple. Stored fields: `output`, `inputs`, `value`.

Contract (D, C, H, R, X). A sparse entry names one target coordinate and a nondecreasing source-index orbit. A sparse symmetric tensor validates finite dimensions, order, canonical orbit keys, and unique entries. Contraction expands orbit multiplicities without materializing the full tensor. The sparse polynomial provider collects such tensors by order under the same factorial-normalized polynomial convention as the dense provider. Invalid indices, duplicate or noncanonical entries, inconsistent dimensions, or malformed coefficients are rejected.

Guarantees (E, P, B, K). Exact coefficients preserve exact contractions. Numerical modes use the declared tolerance for storage certificates and dense comparisons. Sparse storage is proportional to retained source-index orbits, while dense expansion would require `target_dimension * source_dimension^order` elements. Conversion to dense is explicit and has no automatic sparse/GPU fallback.

Evidence and links (V, L). Sparse tensor and provider tests verify canonicalization, contractions, storage ratios, dense equivalence, and certificates `PSC-III-D04S` and `PSC-III-D04P`. The behavioral fixture includes a third-order sparse symmetric contraction. See Chapters 37 and 43.

B.2 · SparseSymmetricTensor

Identity (q, a, t, k). Canonical beta route `komorebi_math.beta.SparseSymmetricTensor`. Identity-equal audited re-exports `komorebi_math.SparseSymmetricTensor`, `komorebi_math.nonlinear.SparseSymmetricTensor`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `class`. Reflected implementation `komorebi_math.nonlinear.sparse.SparseSymmetricTensor`. Profile `B2-S1`.

Manifest signature (s).

```
(order: 'int', source_dimension: 'int', target_dimension: 'int', entries: 'tuple[SparseTensorEntry, ...]' = ()) ->
    ↪ None
```

Definition. One finite symmetric tensor stored by source-index orbits. Properties expose stored, expanded-nonzero, and dense entry counts and their storage ratio. Stored fields: `order`, `source_dimension`, `target_dimension`, `entries`. Runtime-visible operational members: `dense_entry_count`, `expanded_nonzero_count`, `storage_ratio`, `stored_entry_count`, `certificate`, `contract`, `to_dense`, `to_dict`.

Contract (D, C, H, R, X). A sparse entry names one target coordinate and a nondecreasing source-index orbit. A sparse symmetric tensor validates finite dimensions, order, canonical orbit keys, and unique entries. Contraction expands orbit multiplicities without materializing the full tensor. The sparse polynomial provider collects such tensors by order under the same factorial-normalized polynomial convention as the dense provider. Invalid indices, duplicate or noncanonical entries, inconsistent dimensions, or malformed coefficients are rejected.

Guarantees (E, P, B, K). Exact coefficients preserve exact contractions. Numerical modes use the declared tolerance for storage certificates and dense comparisons. Sparse storage is proportional to retained source-index orbits, while dense expansion would require `target_dimension * source_dimension^order` elements. Conversion to dense is explicit and has no automatic sparse/GPU fallback.

Evidence and links (V, L). Sparse tensor and provider tests verify canonicalization, contractions, storage ratios, dense equivalence, and certificates `PSC-III-D04S` and `PSC-III-D04P`. The behavioral fixture includes a third-order sparse symmetric contraction. See Chapters 37 and 43.

B.2 · SparsePolynomialDerivativeProvider

Identity (q, a, t, k). Canonical beta route `komorebi_math.beta.SparsePolynomialDerivativeProvider`. Identity-equal audited re-exports `komorebi_math.SparsePolynomialDerivativeProvider`, `komorebi_math.nonlinear.SparsePolynomialDerivativeProvider`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `class`. Reflected implementation `komorebi_math.nonlinear.sparse.SparsePolynomialDerivativeProvider`. Profile `B2-S1`.

Manifest signature (s).

```
(tensors: 'Mapping[int, SparseSymmetricTensor | Mapping[tuple[int, ...], Any] | Any]', *, source_dimension: 'int |
    ↪ None' = None, target_dimension: 'int | None' = None, exactness: 'Exactness' = <Exactness.NUMERICAL:
    ↪ 'numerical'>, tolerance: 'float' = 1e-10, name: 'str' = 'sparse polynomial derivative provider') -> 'None'
```

Definition. Polynomial derivatives evaluated from sparse symmetric tensor orbits. It can emit a dense provider explicitly. It never densifies as an undocumented fallback. Runtime-visible operational members: `degree`, `dense_entry_count`, `storage_ratio`, `stored_entry_count`, `derivative`, `storage_certificate`, `supports_order`, `to_dense_provider`, `to_dict`, `value`.

Contract (D, C, H, R, X). A sparse entry names one target coordinate and a nondecreasing source-index orbit. A sparse symmetric tensor validates finite dimensions, order, canonical orbit keys, and unique entries. Contraction expands orbit multiplicities without materializing the full tensor. The sparse polynomial provider collects such tensors by order under the same factorial-normalized polynomial convention as the dense provider. Invalid indices, duplicate or noncanonical entries, inconsistent dimensions, or malformed coefficients are rejected.

Guarantees (E, P, B, K). Exact coefficients preserve exact contractions. Numerical modes use the declared tolerance for storage certificates and dense comparisons. Sparse storage is proportional to retained source-index orbits, while dense expansion would require `target_dimension * source_dimension^order` elements. Conversion to dense is explicit and has no automatic sparse/GPU fallback.

Evidence and links (V, L). Sparse tensor and provider tests verify canonicalization, contractions, storage ratios, dense equivalence, and certificates `PSC-III-D04S` and `PSC-III-D04P`. The behavioral fixture includes a third-order sparse symmetric contraction. See Chapters 37 and 43.

B.2 · `PacketJetInputGroup`

Identity (q, a, t, k). Canonical beta route `komorebi_math.beta.PacketJetInputGroup`. Identity-equal audited re-exports `komorebi_math.nonlinear.PacketJetInputGroup`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `class`. Reflected implementation `komorebi_math.nonlinear.execution.PacketJetInputGroup`. Profile `B2-E1`.

Manifest signature (s).

```
(inputs: 'tuple[int, ...]', outputs: 'tuple[int, ...]') -> None
```

Definition. Active output packets sharing one ordered input-packet address. Stored fields: `inputs`, `outputs`.

Contract (D, C, H, R, X). Execution records describe groups sharing projected input addresses, a deterministic plan, individual projected evaluations, and the aggregate result including reconstructed and direct derivatives, operation counts, backend, host-transfer flag, and certificate. Records require coherent order, packet-name sets, unique address coverage, and nonnegative counts. Their constructors validate structural consistency. Execution failures arise in the functions that create them.

Guarantees (E, P, B, K). The records preserve deterministic planning and observed execution metadata. A valid batch certificate establishes equality or tolerance agreement for the supplied finite evaluation, not global function equivalence. Storage is linear in active addresses and groups plus the values retained in the result.

Evidence and links (V, L). Planning and execution tests check unique address coverage, deterministic grouping, count identities, reconstruction, direct comparison, and serialization. The execution certificate family uses `PSC-III-A10`. See Chapter 43.

B.2 · `PacketJetExecutionPlan`

Identity (q, a, t, k). Canonical beta route `komorebi_math.beta.PacketJetExecutionPlan`. Identity-equal audited re-exports `komorebi_math.PacketJetExecutionPlan`, `komorebi_math.nonlinear.PacketJetExecutionPlan`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `class`. Reflected implementation `komorebi_math.nonlinear.execution.PacketJetExecutionPlan`. Profile `B2-E1`.

Manifest signature (s).

```
(order: 'int', source_names: 'tuple[str, ...]', target_names: 'tuple[str, ...]', groups:
    ↪ 'tuple[PacketJetInputGroup, ...]', active_addresses: 'tuple[PacketJetAddress, ...]', pruned_addresses:
    ↪ 'tuple[PacketJetAddress, ...]', support_was_supplied: 'bool', support_complete: 'bool', certificate:
    ↪ 'Certificate') -> None
```

Definition. A deterministic grouping of packet-jet blocks by projected inputs. The plan stores active and pruned address tuples, input groups, source and target names, support-completeness flags, and a certificate. Stored fields: `order`, `source_names`, `target_names`, `groups`, `active_addresses`, `pruned_addresses`, `support_was_supplied`, `support_complete`, `certificate`. Runtime-visible operational members: `active_address_count`, `derivative_call_count`, `derivative_call_reduction`, `full_address_count`, `naive_derivative_call_count`, `pruned_address_count`, `matches`, `to_dict`.

Contract (D, C, H, R, X). Execution records describe groups sharing projected input addresses, a deterministic plan, individual projected evaluations, and the aggregate result including reconstructed and direct derivatives, operation counts, backend, host-transfer flag, and certificate. Records require coherent order, packet-name sets, unique address coverage, and nonnegative counts. Their constructors validate structural consistency. Execution failures arise in the functions that create them.

Guarantees (E, P, B, K). The records preserve deterministic planning and observed execution metadata. A valid batch certificate establishes equality or tolerance agreement for the supplied finite evaluation, not global function equivalence. Storage is linear in active addresses and groups plus the values retained in the result.

Evidence and links (V, L). Planning and execution tests check unique address coverage, deterministic grouping, count identities, reconstruction, direct comparison, and serialization. The execution certificate family uses `PSC-III-A10`. See Chapter 43.

B.2 · `PacketJetEvaluation`

Identity (q, a, t, k). Canonical beta route `komorebi_math.beta.PacketJetEvaluation`. Identity-equal audited re-exports `komorebi_math.nonlinear.PacketJetEvaluation`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `class`. Reflected implementation `komorebi_math.nonlinear.execution.PacketJetEvaluation`. Profile `B2-E1`.

Manifest signature (s).

```
(address: 'PacketJetAddress', value: 'Any') -> None
```

Definition. One evaluated packet-jet address and its projected target value. Stored fields: `address`, `value`.

Contract (D, C, H, R, X). Execution records describe groups sharing projected input addresses, a deterministic plan, individual projected evaluations, and the aggregate result including reconstructed and direct derivatives, operation counts, backend, host-transfer flag, and certificate. Records require coherent order, packet-name sets, unique address coverage, and nonnegative counts. Their constructors validate structural consistency. Execution failures arise in the functions that create them.

Guarantees (E, P, B, K). The records preserve deterministic planning and observed execution metadata. A valid batch certificate establishes equality or tolerance agreement for the supplied finite evaluation, not global function equivalence. Storage is linear in active addresses and groups plus the values retained in the result.

Evidence and links (V, L). Planning and execution tests check unique address coverage, deterministic grouping, count identities, reconstruction, direct comparison, and serialization. The execution certificate family uses `PSC-III-A10`. See Chapter 43.

B.2 · `PacketJetBatchResult`

Identity (q, a, t, k). Canonical beta route `komorebi_math.beta.PacketJetBatchResult`. Identity-equal audited re-exports `komorebi_math.PacketJetBatchResult`, `komorebi_math.nonlinear.PacketJetBatchResult`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `class`. Reflected implementation `komorebi_math.nonlinear.execution.PacketJetBatchResult`. Profile `B2-E1`.

Manifest signature (s).

```
(evaluations: 'tuple[PacketJetEvaluation, ...]', reconstructed: 'Any', direct: 'Any', derivative_calls: 'int',
    ↪ input_projections: 'int', output_projections: 'int', pruned_blocks: 'int', backend: 'str', returned_to_host:
    ↪ 'bool', certificate: 'Certificate') -> None
```

Definition. Result of one projection-cached packet-jet execution. `value()` returns the reconstructed derivative and `as_mapping()` indexes individual results by address. Stored fields: `evaluations`, `reconstructed`, `direct`, `derivative_calls`, `input_projections`, `output_projections`, `pruned_blocks`, `backend`, `returned_to_host`, `certificate`. Runtime-visible operational members: `as_mapping`, `to_dict`, `value`.

Contract (D, C, H, R, X). Execution records describe groups sharing projected input addresses, a deterministic plan, individual projected evaluations, and the aggregate result including reconstructed and direct derivatives, operation counts, backend, host-transfer flag, and certificate. Records require coherent order, packet-name sets, unique address coverage, and nonnegative counts. Their constructors validate structural consistency. Execution failures arise in the functions that create them.

Guarantees (E, P, B, K). The records preserve deterministic planning and observed execution metadata. A valid batch certificate establishes equality or tolerance agreement for the supplied finite evaluation, not global function equivalence. Storage is linear in active addresses and groups plus the values retained in the result.

Evidence and links (V, L). Planning and execution tests check unique address coverage, deterministic grouping, count identities, reconstruction, direct comparison, and serialization. The execution certificate family uses `PSC-III-A10`. See Chapter 43.

B.2 · compile_packet_jet_plan

Identity (q, a, t, k). Canonical beta route `komorebi_math.beta.compile_packet_jet_plan`. Identity-equal audited re-exports `komorebi_math.compile_packet_jet_plan`, `komorebi_math.nonlinear.compile_packet_jet_plan`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `function`. Reflected implementation `komorebi_math.nonlinear.execution.compile_packet_jet_plan`. Profile `B2-E2`.

Manifest signature (s).

```
(family: 'PacketJetFamily', support: 'PacketJetSupport | None' = None, *, max_addresses: 'int' = 1000000) ->
    ↪ 'PacketJetExecutionPlan'
```

Definition. Compile a projection-cached plan, pruning only certified zero blocks.

Contract (D, C, H, R, X). Compilation enumerates the finite family, applies an optional support record, prunes only certified zero addresses, and groups active blocks by ordered projected inputs. Execution validates plan/family identity and direction dimensions, caches each projected input group and derivative call, projects outputs, reconstructs the sum, and compares it with direct provider evaluation. The convenience function performs both steps. Invalid plans, incomplete or inconsistent support, excessive address counts, bad directions, backend mismatch, or failed tolerance checks raise validation or dimension errors or return an invalid certificate.

Guarantees (E, P, B, K). Ordering and counts are deterministic. Exact backends compare exact values. Numerical backends use the declared or backend tolerance. The plan caps enumeration with `max_addresses`. Planned derivative calls equal input groups rather than active output addresses, and memory is linear in plan, cached projections, and retained evaluations.

Evidence and links (V, L). Tests and the 15-case golden fixture verify projection caching, support pruning, direct reconstruction, exact count reductions, and the `PSC-III-A10` certificate. See Chapter 43 and the nonlinear execution example.

B.2 · execute_packet_jet_plan

Identity (q, a, t, k). Canonical beta route `komorebi_math.beta.execute_packet_jet_plan`. Identity-equal audited re-exports `komorebi_math.execute_packet_jet_plan`, `komorebi_math.nonlinear.execute_packet_jet_plan`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `function`. Reflected implementation `komorebi_math.nonlinear.execution.execute_packet_jet_plan`. Profile `B2-E2`.

Manifest signature (s).

```
(plan: 'PacketJetExecutionPlan', family: 'PacketJetFamily', directions: 'Sequence[Any]', *, tolerance: 'float |
    ↪ None' = None) -> 'PacketJetBatchResult'
```

Definition. Execute a packet-jet plan with cached input projections.

Contract (D, C, H, R, X). Compilation enumerates the finite family, applies an optional support record, prunes only certified zero addresses, and groups active blocks by ordered projected inputs. Execution validates plan/family identity and direction dimensions, caches each projected input group and derivative call, projects outputs, reconstructs the sum, and compares it with direct provider evaluation. The convenience function performs both steps. Invalid plans, incomplete or inconsistent support, excessive address counts, bad directions, backend mismatch, or failed tolerance checks raise validation or dimension errors or return an invalid certificate.

Guarantees (E, P, B, K). Ordering and counts are deterministic. Exact backends compare exact values. Numerical backends use the declared or backend tolerance. The plan caps enumeration with `max_addresses`. Planned derivative calls equal input groups rather than active output addresses, and memory is linear in plan, cached projections, and retained evaluations.

Evidence and links (V, L). Tests and the 15-case golden fixture verify projection caching, support pruning, direct reconstruction, exact count reductions, and the `PSC-III-A10` certificate. See Chapter 43 and the nonlinear execution example.

B.2 · execute_packet_jets

Identity (q, a, t, k). Canonical beta route `komorebi_math.beta.execute_packet_jets`. Identity-equal audited re-exports `komorebi_math.execute_packet_jets`, `komorebi_math.nonlinear.execute_packet_jets`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `function`. Reflected implementation `komorebi_math.nonlinear.execution.execute_packet_jets`. Profile `B2-E2`.

Manifest signature (s).

```
(family: 'PacketJetFamily', directions: 'Sequence[Any]', *, support: 'PacketJetSupport | None' = None, tolerance:
   ↪ 'float | None' = None, max_addresses: 'int' = 1000000) -> 'PacketJetBatchResult'
```

Definition. Compile and execute one projection-cached packet-jet family.

Contract (D, C, H, R, X). Compilation enumerates the finite family, applies an optional support record, prunes only certified zero addresses, and groups active blocks by ordered projected inputs. Execution validates plan/family identity and direction dimensions, caches each projected input group and derivative call, projects outputs, reconstructs the sum, and compares it with direct provider evaluation. The convenience function performs both steps. Invalid plans, incomplete or inconsistent support, excessive address counts, bad directions, backend mismatch, or failed tolerance checks raise validation or dimension errors or return an invalid certificate.

Guarantees (E, P, B, K). Ordering and counts are deterministic. Exact backends compare exact values. Numerical backends use the declared or backend tolerance. The plan caps enumeration with `max_addresses`. Planned derivative calls equal input groups rather than active output addresses, and memory is linear in plan, cached projections, and retained evaluations.

Evidence and links (V, L). Tests and the 15-case golden fixture verify projection caching, support pruning, direct reconstruction, exact count reductions, and the `PSC-III-A10` certificate. See Chapter 43 and the nonlinear execution example.

B.2 · PacketJetComplexityEstimate

Identity (q, a, t, k). Canonical beta route `komorebi_math.beta.PacketJetComplexityEstimate`. Identity-equal audited re-exports `komorebi_math.PacketJetComplexityEstimate`, `komorebi_math.nonlinear.PacketJetComplexityEstimate`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `class`. Reflected implementation `komorebi_math.nonlinear.profiling.PacketJetComplexityEstimate`. Profile `B2-P1`.

Manifest signature (s).

```
(order: 'int', source_dimension: 'int', target_dimension: 'int', source_packets: 'int', target_packets: 'int',
   ↪ family_addresses: 'int', active_addresses: 'int', pruned_addresses: 'int', input_groups: 'int',
   ↪ naive_derivative_calls: 'int', planned_derivative_calls: 'int', projected_input_vectors: 'int',
   ↪ output_projections: 'int', dense_basis_evaluations: 'int', dense_tensor_elements: 'int', certificate:
   ↪ 'Certificate') -> None
```

Definition. Exact finite counts implied by packet and ambient dimensions. Its reduction property compares naive address-wise calls with planned grouped calls.

Contract (D, C, H, R, X). The estimator accepts a packet-jet family, optional support record, optional precompiled plan, and address budget. It derives exact integer counts for family and active addresses, groups, naive and planned derivative calls, projected vectors, output projections, dense basis evaluations, and dense tensor elements. Inconsistent support or plan identity and excessive address counts are rejected.

Guarantees (E, P, B, K). The counts are exact consequences of finite dimensions and the selected plan under `PSC-III-A61`. They are not elapsed-time, throughput, or memory-allocation measurements. Computation is dominated by finite address enumeration and grouping and uses no optional profiler backend.

Evidence and links (V, L). Unit tests compare every field with hand-counted small systems and check derivative-call reduction. See Chapter 43 and Appendix K.

B.2 · RuntimeProfileSample

Identity (q, a, t, k). Canonical beta route `komorebi_math.beta.RuntimeProfileSample`. Identity-equal audited re-exports `komorebi_math.RuntimeProfileSample`, `komorebi_math.nonlinear.RuntimeProfileSample`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `class`. Reflected implementation `komorebi_math.nonlinear.profiling.RuntimeProfileSample`. Profile `B2-P2`.

Manifest signature (s).

```
(duration_ns: 'int', traced_peak_bytes: 'int') -> None
```

Definition. One observed duration and Python allocation peak. Stored fields: `duration_ns`, `traced_peak_bytes`.

Contract (D, C, H, R, X). Runtime samples record duration in nanoseconds and Python traced peak bytes. Stage profiles aggregate named repeated samples after an explicit warmup. Packet-jet reports join exact complexity counts to selected local stages. Profiling functions require positive repetition counts and valid operations, families, directions, budgets, and optional stream settings. Exceptions raised by the profiled callable propagate rather than becoming benchmark data.

Guarantees (E, P, B, K). Measurements use `perf_counter_ns` and `tracemalloc`. They are environment-specific observations and are never interpreted as asymptotic evidence. Median, minimum, maximum, and peak summaries are deterministic functions of captured samples, but the measurements themselves are not reproducible constants. Profiling adds tracing overhead and does not measure all native or device allocation.

Evidence and links (V, L). Profiling tests validate record arithmetic, scope labels, stage inclusion, exact complexity joins under `PSC-III-A61`, and optional stream-write profiling. See Chapter 43 and Appendix K.

B.2 · RuntimeStageProfile

Identity (q, a, t, k). Canonical beta route `komorebi_math.beta.RuntimeStageProfile`. Identity-equal audited re-exports `komorebi_math.RuntimeStageProfile`, `komorebi_math.nonlinear.RuntimeStageProfile`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `class`. Reflected implementation `komorebi_math.nonlinear.profiling.RuntimeStageProfile`. Profile `B2-P2`.

Manifest signature (s).

```
(name: 'str', scope: 'str', warmup: 'int', samples: 'tuple[RuntimeProfileSample, ...]') -> None
```

Definition. Repeated observations for one precisely named runtime stage. Properties report repetition count and median/minimum/maximum duration plus median/maximum traced peak bytes. Stored fields: `name`, `scope`, `warmup`, `samples`. Runtime-visible operational members: `maximum_duration_ns`, `maximum_peak_bytes`, `median_duration_ns`, `median_peak_bytes`, `minimum_duration_ns`, `repetitions`, `to_dict`.

Contract (D, C, H, R, X). Runtime samples record duration in nanoseconds and Python traced peak bytes. Stage profiles aggregate named repeated samples after an explicit warmup. Packet-jet reports join exact complexity counts to selected local stages. Profiling functions require positive repetition counts and valid operations, families, directions, budgets, and optional stream settings. Exceptions raised by the profiled callable propagate rather than becoming benchmark data.

Guarantees (E, P, B, K). Measurements use `perf_counter_ns` and `tracemalloc`. They are environment-specific observations and are never interpreted as asymptotic evidence. Median, minimum, maximum, and peak summaries are deterministic functions of captured samples, but the measurements themselves are not reproducible constants. Profiling adds tracing overhead and does not measure all native or device allocation.

Evidence and links (V, L). Profiling tests validate record arithmetic, scope labels, stage inclusion, exact complexity joins under `PSC-III-A61`, and optional stream-write profiling. See Chapter 43 and Appendix K.

B.2 · PacketJetProfileReport

Identity (q, a, t, k). Canonical beta route `komorebi_math.beta.PacketJetProfileReport`. Identity-equal audited re-exports `komorebi_math.PacketJetProfileReport`, `komorebi_math.nonlinear.PacketJetProfileReport`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `class`. Reflected implementation `komorebi_math.nonlinear.profiling.PacketJetProfileReport`. Profile `B2-P2`.

Manifest signature (s).

```
(estimate: 'PacketJetComplexityEstimate', stages: 'tuple[RuntimeStageProfile, ...]', environment: 'Mapping[str,
   ↪ Any]', certificate: 'Certificate') -> None
```

Definition. Exact counts paired with environment-specific runtime observations. The report joins exact finite complexity counts to named local runtime stages and an explicit profiling contract. Stored fields: `estimate`, `stages`, `environment`, `certificate`. Runtime-visible operational members: `to_dict`.

Contract (D, C, H, R, X). Runtime samples record duration in nanoseconds and Python traced peak bytes. Stage profiles aggregate named repeated samples after an explicit warmup. Packet-jet reports join exact complexity counts to selected local stages. Profiling functions require positive repetition counts and valid operations, families, directions, budgets, and optional stream settings. Exceptions raised by the profiled callable propagate rather than becoming benchmark data.

Guarantees (E, P, B, K). Measurements use `perf_counter_ns` and `tracemalloc`. They are environment-specific observations and are never interpreted as asymptotic evidence. Median, minimum, maximum, and peak summaries are deterministic functions of captured samples, but the measurements themselves are not reproducible constants. Profiling adds tracing overhead and does not measure all native or device allocation.

Evidence and links (V, L). Profiling tests validate record arithmetic, scope labels, stage inclusion, exact complexity joins under `PSC-III-A61`, and optional stream-write profiling. See Chapter 43 and Appendix K.

B.2 · estimate_packet_jet_complexity

Identity (q, a, t, k). Canonical beta route `komorebi_math.beta.estimate_packet_jet_complexity`. Identity-equal audited re-exports `komorebi_math.estimate_packet_jet_complexity`, `komorebi_math.nonlinear.estimate_packet_jet_complexity`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `function`. Reflected implementation `komorebi_math.nonlinear.profiling.estimate_packet_jet_complexity`. Profile `B2-P1`.

Manifest signature (s).

```
(family: 'PacketJetFamily', support: 'PacketJetSupport | None' = None, *, plan: 'PacketJetExecutionPlan | None' =
   ↪ None, max_addresses: 'int' = 1000000) -> 'PacketJetComplexityEstimate'
```

Definition. Return exact finite operation counts for a packet-jet family.

Contract (D, C, H, R, X). The estimator accepts a packet-jet family, optional support record, optional precompiled plan, and address budget. It derives exact integer counts for family and active addresses, groups, naive and planned derivative calls, projected vectors, output projections, dense basis evaluations, and dense tensor elements. Inconsistent support or plan identity and excessive address counts are rejected.

Guarantees (E, P, B, K). The counts are exact consequences of finite dimensions and the selected plan under `PSC-III-A61`. They are not elapsed-time, throughput, or memory-allocation measurements. Computation is dominated by finite address enumeration and grouping and uses no optional profiler backend.

Evidence and links (V, L). Unit tests compare every field with hand-counted small systems and check derivative-call reduction. See Chapter 43 and Appendix K.

B.2 · profile_operation

Identity (q, a, t, k). Canonical beta route `komorebi_math.beta.profile_operation`. Identity-equal audited re-exports `komorebi_math.profile_operation`, `komorebi_math.nonlinear.profile_operation`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `function`. Reflected implementation `komorebi_math.nonlinear.profiling.profile_operation`. Profile `B2-P2`.

Manifest signature (s).

```
(name: 'str', operation: 'Callable[[], T]', *, repetitions: 'int' = 5, warmup: 'int' = 1, scope: 'str' = 'Python
    ↪ call including allocations and synchronization performed by the callable') -> 'RuntimeStageProfile'
```

Definition. Observe one callable without interpreting timings as asymptotic evidence.

Contract (D, C, H, R, X). Runtime samples record duration in nanoseconds and Python traced peak bytes. Stage profiles aggregate named repeated samples after an explicit warmup. Packet-jet reports join exact complexity counts to selected local stages. Profiling functions require positive repetition counts and valid operations, families, directions, budgets, and optional stream settings. Exceptions raised by the profiled callable propagate rather than becoming benchmark data.

Guarantees (E, P, B, K). Measurements use `perf_counter_ns` and `tracemalloc`. They are environment-specific observations and are never interpreted as asymptotic evidence. Median, minimum, maximum, and peak summaries are deterministic functions of captured samples, but the measurements themselves are not reproducible constants. Profiling adds tracing overhead and does not measure all native or device allocation.

Evidence and links (V, L). Profiling tests validate record arithmetic, scope labels, stage inclusion, exact complexity joins under `PSC-III-A61`, and optional stream-write profiling. See Chapter 43 and Appendix K.

B.2 · profile_packet_jet_runtime

Identity (q, a, t, k). Canonical beta route `komorebi_math.beta.profile_packet_jet_runtime`. Identity-equal audited re-exports `komorebi_math.profile_packet_jet_runtime`, `komorebi_math.nonlinear.profile_packet_jet_runtime`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `function`. Reflected implementation `komorebi_math.nonlinear.profiling.profile_packet_jet_runtime`. Profile `B2-P2`.

Manifest signature (s).

```
(family: 'PacketJetFamily', directions: 'Sequence[Any]', *, support: 'PacketJetSupport | None' = None, jet:
    ↪ 'PacketJet | None' = None, repetitions: 'int' = 5, warmup: 'int' = 1, max_addresses: 'int' = 1000000,
    ↪ max_snapshot_evaluations: 'int' = 100000, include_stream_write: 'bool' = False, stream_chunk_size: 'int' =
    ↪ 1024) -> 'PacketJetProfileReport'
```

Definition. Profile selected packet-jet stages under one explicit local contract.

Contract (D, C, H, R, X). Runtime samples record duration in nanoseconds and Python traced peak bytes. Stage profiles aggregate named repeated samples after an explicit warmup. Packet-jet reports join exact complexity counts to selected local stages. Profiling functions require positive repetition counts and valid operations, families, directions, budgets, and optional stream settings. Exceptions raised by the profiled callable propagate rather than becoming benchmark data.

Guarantees (E, P, B, K). Measurements use `perf_counter_ns` and `tracemalloc`. They are environment-specific observations and are never interpreted as asymptotic evidence. Median, minimum, maximum, and peak summaries are deterministic functions of captured samples, but the measurements themselves are not reproducible constants. Profiling adds tracing overhead and does not measure all native or device allocation.

Evidence and links (V, L). Profiling tests validate record arithmetic, scope labels, stage inclusion, exact complexity joins under `PSC-III-A61`, and optional stream-write profiling. See Chapter 43 and Appendix K.

B.3 Resumable streaming and snapshots

B.3 · PACKET_JET_STREAM_SCHEMA

Identity (q, a, t, k). Canonical beta route `komorebi_math.beta.PACKET_JET_STREAM_SCHEMA`. Identity-equal audited re-exports `komorebi_math.PACKET_JET_STREAM_SCHEMA`, `komorebi_math.nonlinear.PACKET_JET_STREAM_SCHEMA`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `data`. Reflected implementation `builtins.str`. Profile `B3-I1`.

Manifest signature (s).

```
null
```

Definition. The immutable runtime value is `komorebi-math.packet-jet-stream`.

Contract (D, C, H, R, X). The domain is import-time binding and the codomain is the exact packet-jet stream or chunk schema name and integer schema version. There is no call-time failure. These constants are consumed by stream envelope validators and must not be substituted for payload validation.

Guarantees (E, P, B, K). Values are exact immutable identifiers, require no backend, and have constant lookup cost. Version 1 names the current directory-backed manifest/chunk contract. It does not imply authenticity or compatibility with unregistered future versions.

Evidence and links (V, L). Stream tests assert schema/version rejection, canonical encoding, and runtime identity. See Chapters 43 and 61.

B.3 · PACKET_JET_STREAM_SCHEMA_VERSION

Identity (q, a, t, k). Canonical beta route `komorebi_math.beta.PACKET_JET_STREAM_SCHEMA_VERSION`. Identity-equal audited re-exports `komorebi_math.PACKET_JET_STREAM_SCHEMA_VERSION`, `komorebi_math.nonlinear.PACKET_JET_STREAM_SCHEMA_VERSION`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `data`. Reflected implementation `builtins.int`. Profile `B3-I1`.

Manifest signature (s).

```
null
```

Definition. The immutable runtime value is `1`.

Contract (D, C, H, R, X). The domain is import-time binding and the codomain is the exact packet-jet stream or chunk schema name and integer schema version. There is no call-time failure. These constants are consumed by stream envelope validators and must not be substituted for payload validation.

Guarantees (E, P, B, K). Values are exact immutable identifiers, require no backend, and have constant lookup cost. Version 1 names the current directory-backed manifest/chunk contract. It does not imply authenticity or compatibility with unregistered future versions.

Evidence and links (V, L). Stream tests assert schema/version rejection, canonical encoding, and runtime identity. See Chapters 43 and 61.

B.3 · PACKET_JET_CHUNK_SCHEMA

Identity (q, a, t, k). Canonical beta route `komorebi_math.beta.PACKET_JET_CHUNK_SCHEMA`. Identity-equal audited re-exports `komorebi_math.PACKET_JET_CHUNK_SCHEMA`, `komorebi_math.nonlinear.PACKET_JET_CHUNK_SCHEMA`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `data`. Reflected implementation `builtins.str`. Profile `B3-I1`.

Manifest signature (s).

```
null
```

Definition. The immutable runtime value is `komorebi-math.packet-jet-stream.chunk`.

Contract (D, C, H, R, X). The domain is import-time binding and the codomain is the exact packet-jet stream or chunk schema name and integer schema version. There is no call-time failure. These constants are consumed by stream envelope validators and must not be substituted for payload validation.

Guarantees (E, P, B, K). Values are exact immutable identifiers, require no backend, and have constant lookup cost. Version 1 names the current directory-backed manifest/chunk contract. It does not imply authenticity or compatibility with unregistered future versions.

Evidence and links (V, L). Stream tests assert schema/version rejection, canonical encoding, and runtime identity. See Chapters 43 and 61.

B.3 · `PACKET_JET_CHUNK_SCHEMA_VERSION`

Identity (q, a, t, k). Canonical beta route `komorebi_math.beta.PACKET_JET_CHUNK_SCHEMA_VERSION`. Identity-equal audited re-exports `komorebi_math.PACKET_JET_CHUNK_SCHEMA_VERSION`, `komorebi_math.nonlinear.PACKET_JET_CHUNK_SCHEMA_VERSION`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `data`. Reflected implementation `builtins.int`. Profile `B3-I1`.

Manifest signature (s).

```
null
```

Definition. The immutable runtime value is `1`.

Contract (D, C, H, R, X). The domain is import-time binding and the codomain is the exact packet-jet stream or chunk schema name and integer schema version. There is no call-time failure. These constants are consumed by stream envelope validators and must not be substituted for payload validation.

Guarantees (E, P, B, K). Values are exact immutable identifiers, require no backend, and have constant lookup cost. Version 1 names the current directory-backed manifest/chunk contract. It does not imply authenticity or compatibility with unregistered future versions.

Evidence and links (V, L). Stream tests assert schema/version rejection, canonical encoding, and runtime identity. See Chapters 43 and 61.

B.3 · `PacketJetCoefficient`

Identity (q, a, t, k). Canonical beta route `komorebi_math.beta.PacketJetCoefficient`. Identity-equal audited re-exports `komorebi_math.nonlinear.PacketJetCoefficient`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `class`. Reflected implementation `komorebi_math.nonlinear.snapshots.PacketJetCoefficient`. Profile `B3-S1`.

Manifest signature (s).

```
(indices: 'tuple[int, ...]', value: 'tuple[Any, ...]') -> None
```

Definition. One standard-basis coefficient of a persisted packet jet. Stored fields: `indices`, `value`.

Contract (D, C, H, R, X). A coefficient stores one ordered standard-basis multi-index and target tuple. A snapshot stores address, dimensions, provider name, exactness, point, sparse retained coefficients, and a capture certificate, and can evaluate directions or materialize a guarded dense tensor. Capture exhaustively evaluates the jet on standard-basis tuples after preflighting `max_evaluations`. Invalid indices, dimensions, certificate inconsistency, nonpositive limits, or excessive dense materialization raise validation or dimension errors.

Guarantees (E, P, B, K). Exact backends retain every nonzero coefficient. Numerical capture omits coefficients whose norm does not exceed the declared tolerance and records the largest omitted norm. Dense basis work is `source_dimension^order`. Retained sparse storage is linear in stored coefficients, while dense materialization is capped by `max_elements`.

Evidence and links (V, L). Snapshot tests and the golden fixture verify exact contraction, numerical omission evidence, portable migration, and theorem identifier PSC-III-D58. See Chapters 38, 43, and 61.

B.3 · PacketJetSnapshot

Identity (q, a, t, k). Canonical beta route komorebi_math.beta.PacketJetSnapshot. Identity-equal audited re-exports komorebi_math.PacketJetSnapshot, komorebi_math.nonlinear.PacketJetSnapshot. Tier [BETA] under 1.0-beta-additive-strict-signatures. Manifest kind class. Reflected implementation komorebi_math.nonlinear.snapshots.PacketJetSnapshot. Profile B3-S1.

Manifest signature (s).

```
(address: 'PacketJetAddress', source_dimension: 'int', target_dimension: 'int', provider_name: 'str', exactness:
    ↪ 'Exactness', point: 'tuple[Any, ...]', coefficients: 'tuple[PacketJetCoefficient, ...]',
    ↪ capture_certificate: 'Certificate') -> None
```

Definition. A provider-independent sparse coefficient record of one packet jet. Evaluation contracts stored standard-basis coefficients with supplied directions. to_dense() has an explicit element guard. Stored fields: address, source_dimension, target_dimension, provider_name, exactness, point, coefficients, capture_certificate. Runtime-visible operational members: dense_coefficient_count, nnz, order, coefficient, comparison_certificate, evaluate, to_dense, to_dict.

Contract (D, C, H, R, X). A coefficient stores one ordered standard-basis multi-index and target tuple. A snapshot stores address, dimensions, provider name, exactness, point, sparse retained coefficients, and a capture certificate, and can evaluate directions or materialize a guarded dense tensor. Capture exhaustively evaluates the jet on standard-basis tuples after preflighting max_evaluations. Invalid indices, dimensions, certificate inconsistency, nonpositive limits, or excessive dense materialization raise validation or dimension errors.

Guarantees (E, P, B, K). Exact backends retain every nonzero coefficient. Numerical capture omits coefficients whose norm does not exceed the declared tolerance and records the largest omitted norm. Dense basis work is source_dimension^order. Retained sparse storage is linear in stored coefficients, while dense materialization is capped by max_elements.

Evidence and links (V, L). Snapshot tests and the golden fixture verify exact contraction, numerical omission evidence, portable migration, and theorem identifier PSC-III-D58. See Chapters 38, 43, and 61.

B.3 · snapshot_packet_jet

Identity (q, a, t, k). Canonical beta route komorebi_math.beta.snapshot_packet_jet. Identity-equal audited re-exports komorebi_math.snapshot_packet_jet, komorebi_math.nonlinear.snapshot_packet_jet. Tier [BETA] under 1.0-beta-additive-strict-signatures. Manifest kind function. Reflected implementation komorebi_math.nonlinear.snapshots.snapshot_packet_jet. Profile B3-S1.

Manifest signature (s).

```
(jet: 'PacketJet', *, tolerance: 'float | None' = None, max_evaluations: 'int' = 100000) -> 'PacketJetSnapshot'
```

Definition. Capture one packet jet by exhaustive standard-basis evaluation.

Contract (D, C, H, R, X). A coefficient stores one ordered standard-basis multi-index and target tuple. A snapshot stores address, dimensions, provider name, exactness, point, sparse retained coefficients, and a capture certificate, and can evaluate directions or materialize a guarded dense tensor. Capture exhaustively evaluates the jet on standard-basis tuples after preflighting max_evaluations. Invalid indices, dimensions, certificate inconsistency, nonpositive limits, or excessive dense materialization raise validation or dimension errors.

Guarantees (E, P, B, K). Exact backends retain every nonzero coefficient. Numerical capture omits coefficients whose norm does not exceed the declared tolerance and records the largest omitted norm. Dense basis work is source_dimension^order. Retained sparse storage is linear in stored coefficients, while dense materialization is capped by max_elements.

Evidence and links (V, L). Snapshot tests and the golden fixture verify exact contraction, numerical omission evidence, portable migration, and theorem identifier PSC-III-D58. See Chapters 38, 43, and 61.

B.3 · PacketJetStreamChunkRecord

Identity (q, a, t, k). Canonical beta route `komorebi_math.beta.PacketJetStreamChunkRecord`. Identity-equal audited re-exports `komorebi_math.PacketJetStreamChunkRecord`, `komorebi_math.nonlinear.PacketJetStreamChunkRecord`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `class`. Reflected implementation `komorebi_math.nonlinear.streaming.PacketJetStreamChunkRecord`. Profile `B3-R1`.

Manifest signature (s).

```
(index: 'int', filename: 'str', basis_start: 'int', basis_stop: 'int', coefficient_start: 'int', coefficient_stop:
    ↪  'int', payload_sha256: 'str', byte_count: 'int') -> None
```

Definition. One immutable chunk index entry in a packet-jet stream manifest. Stored fields: `index`, `filename`, `basis_start`, `basis_stop`, `coefficient_start`, `coefficient_stop`, `payload_sha256`, `byte_count`. Runtime-visible operational members: `basis_count`, `coefficient_count`, `to_dict`.

Contract (D, C, H, R, X). These immutable records describe one verified chunk, the complete or partial stream manifest, a bounded coefficient selection, write progress, or a chunk-wise evaluation with explicit coefficient coverage. Fields require contiguous nonnegative ranges, coherent counts and digests, valid schema metadata, strict ordering, and certificates matching completion state. Malformed ranges, duplicate or unordered coefficients, digest mismatch, or inconsistent progress is rejected.

Guarantees (E, P, B, K). Records make partial versus complete coverage explicit and preserve integrity metadata. A valid record does not authenticate the producer: an attacker able to replace payloads and digests can construct a different internally consistent stream. Storage is proportional to metadata plus the selected coefficients or evaluation value.

Evidence and links (V, L). Streaming tests validate constructor invariants, incomplete-state labeling, tamper detection, selection budgets, and `PSC-III-D60` coverage certificates. See Chapters 43 and 61.

B.3 · PacketJetStreamManifest

Identity (q, a, t, k). Canonical beta route `komorebi_math.beta.PacketJetStreamManifest`. Identity-equal audited re-exports `komorebi_math.PacketJetStreamManifest`, `komorebi_math.nonlinear.PacketJetStreamManifest`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `class`. Reflected implementation `komorebi_math.nonlinear.streaming.PacketJetStreamManifest`. Profile `B3-R1`.

Manifest signature (s).

```
(state: 'StreamState', producer_version: 'str', address: 'PacketJetAddress', source_dimension: 'int',
    ↪  target_dimension: 'int', provider_name: 'str', exactness: 'Exactness', point: 'tuple[Any, ...]', tolerance:
    ↪  'float', dense_coefficient_count: 'int', capture_mode: 'CaptureMode', source_digest: 'str', metadata_sha256:
    ↪  'str', chunk_size: 'int', evaluations_completed: 'int', coefficients_completed: 'int',
    ↪  maximum_omitted_basis_norm: 'float', chunks: 'tuple[PacketJetStreamChunkRecord, ...]', capture_certificate:
    ↪  'Certificate | None', stream_sha256: 'str') -> None
```

Definition. Validated metadata and progress for one chunked packet-jet stream. The manifest binds source metadata, deterministic chunk records, progress, coefficient totals, completion state, and an integrity digest. Stored fields: `state`, `producer_version`, `address`, `source_dimension`, `target_dimension`, `provider_name`, `exactness`, `point`, `tolerance`, `dense_coefficient_count`, `capture_mode`, `source_digest`, `metadata_sha256`, `chunk_size`, `evaluations_completed`, `coefficients_completed`, `maximum_omitted_basis_norm`, `chunks`, `capture_certificate`, `stream_sha256`. Runtime-visible operational members: `complete`, `order`, `metadata_dict`, `to_payload`.

Contract (D, C, H, R, X). These immutable records describe one verified chunk, the complete or partial stream manifest, a bounded coefficient selection, write progress, or a chunk-wise evaluation with explicit coefficient coverage. Fields require contiguous nonnegative ranges, coherent counts and digests, valid schema metadata, strict ordering, and certificates matching completion state. Malformed ranges, duplicate or unordered coefficients, digest mismatch, or inconsistent progress is rejected.

Guarantees (E, P, B, K). Records make partial versus complete coverage explicit and preserve integrity metadata. A valid record does not authenticate the producer: an attacker able to replace payloads and digests can construct a

different internally consistent stream. Storage is proportional to metadata plus the selected coefficients or evaluation value.

Evidence and links (V, L). Streaming tests validate constructor invariants, incomplete-state labeling, tamper detection, selection budgets, and PSC-III-D60 coverage certificates. See Chapters 43 and 61.

B.3 · PacketJetStreamSelection

Identity (q, a, t, k). Canonical beta route komorebi_math.beta.PacketJetStreamSelection. Identity-equal audited re-exports komorebi_math.PacketJetStreamSelection, komorebi_math.nonlinear.PacketJetStreamSelection. Tier [BETA] under 1.0-beta-additive-strict-signatures. Manifest kind class. Reflected implementation komorebi_math.nonlinear.streaming.PacketJetStreamSelection. Profile B3-R1.

Manifest signature (s).

```
(coefficient_start: 'int', coefficient_stop: 'int', coefficients: 'tuple[PacketJetCoefficient, ...]',
   ↪ complete_snapshot: 'bool', certificate: 'Certificate') -> None
```

Definition. A bounded coefficient selection loaded from a packet-jet stream. The selection identifies a half-open coefficient range and whether it covers the complete snapshot. Stored fields: coefficient_start, coefficient_stop, coefficients, complete_snapshot, certificate.

Contract (D, C, H, R, X). These immutable records describe one verified chunk, the complete or partial stream manifest, a bounded coefficient selection, write progress, or a chunk-wise evaluation with explicit coefficient coverage. Fields require contiguous nonnegative ranges, coherent counts and digests, valid schema metadata, strict ordering, and certificates matching completion state. Malformed ranges, duplicate or unordered coefficients, digest mismatch, or inconsistent progress is rejected.

Guarantees (E, P, B, K). Records make partial versus complete coverage explicit and preserve integrity metadata. A valid record does not authenticate the producer: an attacker able to replace payloads and digests can construct a different internally consistent stream. Storage is proportional to metadata plus the selected coefficients or evaluation value.

Evidence and links (V, L). Streaming tests validate constructor invariants, incomplete-state labeling, tamper detection, selection budgets, and PSC-III-D60 coverage certificates. See Chapters 43 and 61.

B.3 · PacketJetStreamWriteResult

Identity (q, a, t, k). Canonical beta route komorebi_math.beta.PacketJetStreamWriteResult. Identity-equal audited re-exports komorebi_math.PacketJetStreamWriteResult, komorebi_math.nonlinear.PacketJetStreamWriteResult. Tier [BETA] under 1.0-beta-additive-strict-signatures. Manifest kind class. Reflected implementation komorebi_math.nonlinear.streaming.PacketJetStreamWriteResult. Profile B3-R1.

Manifest signature (s).

```
(destination: 'Path', active_path: 'Path', completed: 'bool', resumed: 'bool', chunks_written: 'int',
   ↪ evaluations_written: 'int', coefficients_written: 'int', manifest: 'PacketJetStreamManifest', certificate:
   ↪ 'Certificate') -> None
```

Definition. Progress returned by a direct or snapshot stream write. active_path identifies the final directory after completion or the .partial staging directory while interrupted. Stored fields: destination, active_path, completed, resumed, chunks_written, evaluations_written, coefficients_written, manifest, certificate.

Contract (D, C, H, R, X). These immutable records describe one verified chunk, the complete or partial stream manifest, a bounded coefficient selection, write progress, or a chunk-wise evaluation with explicit coefficient coverage. Fields require contiguous nonnegative ranges, coherent counts and digests, valid schema metadata, strict ordering, and certificates matching completion state. Malformed ranges, duplicate or unordered coefficients, digest mismatch, or inconsistent progress is rejected.

Guarantees (E, P, B, K). Records make partial versus complete coverage explicit and preserve integrity metadata. A valid record does not authenticate the producer: an attacker able to replace payloads and digests can construct a

different internally consistent stream. Storage is proportional to metadata plus the selected coefficients or evaluation value.

Evidence and links (V, L). Streaming tests validate constructor invariants, incomplete-state labeling, tamper detection, selection budgets, and PSC-III-D60 coverage certificates. See Chapters 43 and 61.

B.3 · StreamedPacketJetEvaluation

Identity (q, a, t, k). Canonical beta route komorebi_math.beta.StreamedPacketJetEvaluation. Identity-equal audited re-exports komorebi_math.StreamedPacketJetEvaluation, komorebi_math.nonlinear.StreamedPacketJetEvaluation. Tier [BETA] under 1.0-beta-additive-strict-signatures. Manifest kind class. Reflected implementation komorebi_math.nonlinear.streaming.StreamedPacketJetEvaluation. Profile B3-R1.

Manifest signature (s).

```
(value: 'np.ndarray', coefficient_start: 'int', coefficient_stop: 'int', total_coefficients: 'int',
    ↪ complete_snapshot: 'bool', certificate: 'Certificate') -> None
```

Definition. One chunk-wise packet-jet evaluation with an explicit coverage record. The half-open coefficient range and complete_snapshot flag prevent partial sums from being mistaken for full evaluation. Stored fields: value, coefficient_start, coefficient_stop, total_coefficients, complete_snapshot, certificate.

Contract (D, C, H, R, X). These immutable records describe one verified chunk, the complete or partial stream manifest, a bounded coefficient selection, write progress, or a chunk-wise evaluation with explicit coefficient coverage. Fields require contiguous nonnegative ranges, coherent counts and digests, valid schema metadata, strict ordering, and certificates matching completion state. Malformed ranges, duplicate or unordered coefficients, digest mismatch, or inconsistent progress is rejected.

Guarantees (E, P, B, K). Records make partial versus complete coverage explicit and preserve integrity metadata. A valid record does not authenticate the producer: an attacker able to replace payloads and digests can construct a different internally consistent stream. Storage is proportional to metadata plus the selected coefficients or evaluation value.

Evidence and links (V, L). Streaming tests validate constructor invariants, incomplete-state labeling, tamper detection, selection budgets, and PSC-III-D60 coverage certificates. See Chapters 43 and 61.

B.3 · PacketJetSnapshotStreamReader

Identity (q, a, t, k). Canonical beta route komorebi_math.beta.PacketJetSnapshotStreamReader. Identity-equal audited re-exports komorebi_math.PacketJetSnapshotStreamReader, komorebi_math.nonlinear.PacketJetSnapshotStreamReader. Tier [BETA] under 1.0-beta-additive-strict-signatures. Manifest kind class. Reflected implementation komorebi_math.nonlinear.streaming.PacketJetSnapshotStreamReader. Profile B3-R2.

Manifest signature (s).

```
(path: 'str | os.PathLike[str]', *, allow_incomplete: 'bool' = False, verify_chunks: 'bool' = False) -> 'None'
```

Definition. Bounded-memory reader for a complete or explicitly allowed partial stream. Reader operations include verified chunk iteration, bounded selection, chunk-wise evaluation, full verification, and guarded snapshot materialization. Runtime-visible operational members: complete, evaluate, iter_chunks, iter_coefficients, load_snapshot, select, verify.

Contract (D, C, H, R, X). The reader opens a validated directory-backed stream and exposes manifest metadata, chunk iteration, bounded selection, chunk-wise evaluation, verification, and guarded materialization. Complete streams are required by default. Incomplete streams require explicit opt-in. Chunk verification checks canonical envelopes, SHA-256 digests, contiguous ranges, strict lexicographic coefficient order, and manifest counts. Missing files, malformed JSON, digest tampering, unsupported schema, incomplete disallowed state, and budget overflow raise validation errors.

Guarantees (E, P, B, K). Iteration and evaluation can remain bounded by chunk size. Selection and full materialization obey explicit coefficient limits. A partial evaluation returns a certificate with unchecked coverage assumptions instead of masquerading as a complete jet. Integrity is SHA-256 based and is not producer authentication.

Evidence and links (V, L). Direct-resume, bounded-read, incomplete-policy, tamper, and full-materialization tests exercise theorem identifier PSC-III-D60. See Chapters 43 and 61.

B.3 · capture_packet_jet_stream

Identity (q, a, t, k). Canonical beta route komorebi_math.beta.capture_packet_jet_stream. Identity-equal audited re-exports komorebi_math.capture_packet_jet_stream, komorebi_math.nonlinear.capture_packet_jet_stream. Tier [BETA] under 1.0-beta-additive-strict-signatures. Manifest kind function. Reflected implementation komorebi_math.nonlinear.streaming.capture_packet_jet_stream. Profile B3-O1.

Manifest signature (s).

```
(jet: 'PacketJet', path: 'str | os.PathLike[str]', *, tolerance: 'float | None' = None, max_evaluations: 'int' =
    ↪ 10000000, chunk_size: 'int' = 1024, evaluation_budget: 'int | None' = None, resume: 'bool' = False,
    ↪ overwrite: 'bool' = False) -> 'PacketJetStreamWriteResult'
```

Definition. Capture a packet jet directly into bounded, resumable basis chunks.

Contract (D, C, H, R, X). Direct capture evaluates a packet jet in deterministic standard-basis order and writes verified chunks. Snapshot save emits an existing coefficient sequence. Both stage work in <destination>.partial, support explicit budgets and resumability, and promote only a complete verified staging directory. Paths, chunk sizes, evaluation or coefficient budgets, resume state, and overwrite policy must be valid. Conflicts, incompatible partial state, digest mismatch, malformed chunks, or exhausted budgets return partial progress where specified or raise validation errors.

Guarantees (E, P, B, K). Completed streams preserve exact or tolerance-qualified snapshot semantics and canonical integrity metadata. Resume continues only matching source metadata and contiguous verified progress. Memory is bounded by chunk size plus provider evaluation state. Total work follows dense basis evaluations for direct capture or retained coefficient count for snapshot save.

Evidence and links (V, L). The streaming beta verifier covers direct resume, snapshot streaming, budget interruption, deterministic continuation, tamper detection, and PSC-III-D60. See Chapter 43.

B.3 · save_packet_jet_snapshot_stream

Identity (q, a, t, k). Canonical beta route komorebi_math.beta.save_packet_jet_snapshot_stream. Identity-equal audited re-exports komorebi_math.save_packet_jet_snapshot_stream, komorebi_math.nonlinear.save_packet_jet_snapshot_stream. Tier [BETA] under 1.0-beta-additive-strict-signatures. Manifest kind function. Reflected implementation komorebi_math.nonlinear.streaming.save_packet_jet_snapshot_stream. Profile B3-O1.

Manifest signature (s).

```
(snapshot: 'PacketJetSnapshot', path: 'str | os.PathLike[str]', *, chunk_size: 'int' = 1024, coefficient_budget:
    ↪ 'int | None' = None, resume: 'bool' = False, overwrite: 'bool' = False) -> 'PacketJetStreamWriteResult'
```

Definition. Write an existing snapshot as independently verified coefficient chunks.

Contract (D, C, H, R, X). Direct capture evaluates a packet jet in deterministic standard-basis order and writes verified chunks. Snapshot save emits an existing coefficient sequence. Both stage work in <destination>.partial, support explicit budgets and resumability, and promote only a complete verified staging directory. Paths, chunk sizes, evaluation or coefficient budgets, resume state, and overwrite policy must be valid. Conflicts, incompatible partial state, digest mismatch, malformed chunks, or exhausted budgets return partial progress where specified or raise validation errors.

Guarantees (E, P, B, K). Completed streams preserve exact or tolerance-qualified snapshot semantics and canonical integrity metadata. Resume continues only matching source metadata and contiguous verified progress. Memory is bounded by chunk size plus provider evaluation state. Total work follows dense basis evaluations for direct capture or retained coefficient count for snapshot save.

Evidence and links (V, L). The streaming beta verifier covers direct resume, snapshot streaming, budget interruption, deterministic continuation, tamper detection, and PSC-III-D60. See Chapter 43.

B.3 · open_packet_jet_snapshot_stream

Identity (q, a, t, k). Canonical beta route `komorebi_math.beta.open_packet_jet_snapshot_stream`. Identity-equal audited re-exports `komorebi_math.open_packet_jet_snapshot_stream`, `komorebi_math.nonlinear.open_packet_jet_snapshot_stream`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `function`. Reflected implementation `komorebi_math.nonlinear.streaming.open_packet_jet_snapshot_stream`. Profile `B3-O2`.

Manifest signature (s).

```
(path: 'str | os.PathLike[str]', *, allow_incomplete: 'bool' = False, verify_chunks: 'bool' = False) ->
   ↪ 'PacketJetSnapshotStreamReader'
```

Definition. Open one packet-jet stream with explicit incomplete-stream policy.

Contract (D, C, H, R, X). Open validates stream metadata under an explicit incomplete and verification policy and returns a bounded reader. Load requires a complete stream, optionally verifies every chunk, enforces `max_coefficients`, and materializes a conventional snapshot. Invalid paths, missing or malformed files, unsupported schema, tampering, disallowed incompleteness, or coefficient-budget overflow raise validation errors.

Guarantees (E, P, B, K). Open can defer per-chunk verification when requested. Load defaults to verified chunks and never silently loads a partial stream. Reader iteration is chunk-bounded, while full loading uses memory linear in retained coefficients. There is no network or GPU backend.

Evidence and links (V, L). Tests cover complete and incomplete opening, eager and deferred verification, coefficient guards, exact reconstruction, and tamper rejection under `PSC-III-D60`. See Chapters 43 and 61.

B.3 · load_packet_jet_snapshot_stream

Identity (q, a, t, k). Canonical beta route `komorebi_math.beta.load_packet_jet_snapshot_stream`. Identity-equal audited re-exports `komorebi_math.load_packet_jet_snapshot_stream`, `komorebi_math.nonlinear.load_packet_jet_snapshot_stream`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `function`. Reflected implementation `komorebi_math.nonlinear.streaming.load_packet_jet_snapshot_stream`. Profile `B3-O2`.

Manifest signature (s).

```
(path: 'str | os.PathLike[str]', *, max_coefficients: 'int' = 1000000, verify_chunks: 'bool' = True) ->
   ↪ 'PacketJetSnapshot'
```

Definition. Load a complete stream into a conventional in-memory snapshot.

Contract (D, C, H, R, X). Open validates stream metadata under an explicit incomplete and verification policy and returns a bounded reader. Load requires a complete stream, optionally verifies every chunk, enforces `max_coefficients`, and materializes a conventional snapshot. Invalid paths, missing or malformed files, unsupported schema, tampering, disallowed incompleteness, or coefficient-budget overflow raise validation errors.

Guarantees (E, P, B, K). Open can defer per-chunk verification when requested. Load defaults to verified chunks and never silently loads a partial stream. Reader iteration is chunk-bounded, while full loading uses memory linear in retained coefficients. There is no network or GPU backend.

Evidence and links (V, L). Tests cover complete and incomplete opening, eager and deferred verification, coefficient guards, exact reconstruction, and tamper rejection under `PSC-III-D60`. See Chapters 43 and 61.

B.4 Portable beta records and migrations

B.4 · PORTABLE_SCHEMA

Identity (q, a, t, k). Canonical beta route `komorebi_math.beta.PORTABLE_SCHEMA`. Identity-equal audited re-exports `komorebi_math.export.PORTABLE_SCHEMA`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `data`. Reflected implementation `builtins.str`. Profile `B4-I1`.

Manifest signature (s).

```
null
```

Definition. The immutable runtime value is `komorebi-math.portable`.

Contract (D, C, H, R, X). The domain is import-time binding and the codomain is the exact portable-envelope schema name or integer version. Attribute access cannot validate a payload and has no call-time exception path.

Guarantees (E, P, B, K). The values are exact immutable identifiers with constant lookup cost and no backend dependency. Schema version 1 defines the outer digest-checked JSON envelope. Individual codecs retain their own versions.

Evidence and links (V, L). Persistence tests assert the envelope identifiers, duplicate-key and nonfinite rejection, and runtime identity. See Chapter 61.

B.4 · PORTABLE_SCHEMA_VERSION

Identity (q, a, t, k). Canonical beta route `komorebi_math.beta.PORTABLE_SCHEMA_VERSION`. Identity-equal audited re-exports `komorebi_math.export.PORTABLE_SCHEMA_VERSION`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `data`. Reflected implementation `builtins.int`. Profile `B4-I1`.

Manifest signature (s).

```
null
```

Definition. The immutable runtime value is `1`.

Contract (D, C, H, R, X). The domain is import-time binding and the codomain is the exact portable-envelope schema name or integer version. Attribute access cannot validate a payload and has no call-time exception path.

Guarantees (E, P, B, K). The values are exact immutable identifiers with constant lookup cost and no backend dependency. Schema version 1 defines the outer digest-checked JSON envelope. Individual codecs retain their own versions.

Evidence and links (V, L). Persistence tests assert the envelope identifiers, duplicate-key and nonfinite rejection, and runtime identity. See Chapter 61.

B.4 · PortableCodec

Identity (q, a, t, k). Canonical beta route `komorebi_math.beta.PortableCodec`. Identity-equal audited re-exports `komorebi_math.PortableCodec`, `komorebi_math.export.PortableCodec`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `class`. Reflected implementation `komorebi_math.export.persistence.PortableCodec`. Profile `B4-R1`.

Manifest signature (s).

```
(tag: 'str', version: 'int', type_: 'type[Any]', encoder: 'Encoder', decoder: 'Decoder') -> None
```

Definition. One explicit type codec in the portable persistence registry. The codec version belongs to the stable tag and is independent of the outer portable-envelope schema version. Stored fields: `tag`, `version`, `type_`, `encoder`, `decoder`.

Contract (D, C, H, R, X). Codec records bind a stable tag and positive version to one Python type and explicit encoder/decoder. Migration records bind one adjacent `n -> n+1` safe-data transformation. Registration validates tags, versions, type conflicts, adjacency, and replacement policy. Registry queries return deterministic tag/version order.

Duplicate registrations, nonadjacent migrations, missing migration steps, invalid tags, or incompatible versions raise `ValidationError`.

Guarantees (`E`, `P`, `B`, `K`). The registry is explicit and never imports a class named by persisted data. Encoders and decoders determine mathematical fidelity. The registry provides deterministic dispatch and migration sequencing. The reference package registers 51 codecs and one adjacent packet-jet-snapshot migration. Lookup is dictionary-based. Listing sorts the finite registry.

Evidence and links (`V`, `L`). Persistence, migration, and release-contract tests verify all registered tags, versions, type names, adjacent paths, and roundtrips. See Chapters 61 and 72 and Appendix G.

B.4 · `PortableMigration`

Identity (`q`, `a`, `t`, `k`). Canonical beta route `komorebi_math.beta.PortableMigration`. Identity-equal audited re-exports `komorebi_math.PortableMigration`, `komorebi_math.export.PortableMigration`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `class`. Reflected implementation `komorebi_math.export.persistence.PortableMigration`. Profile `B4-R1`.

Manifest signature (`s`).

```
(tag: 'str', from_version: 'int', to_version: 'int', migrator: 'Migrator') -> None
```

Definition. One explicit, adjacent codec-data migration. Only adjacent migrations are valid, so multi-version upgrades are an auditable sequence of registered steps. Stored fields: `tag`, `from_version`, `to_version`, `migrator`.

Contract (`D`, `C`, `H`, `R`, `X`). Codec records bind a stable tag and positive version to one Python type and explicit encoder/decoder. Migration records bind one adjacent `n -> n+1` safe-data transformation. Registration validates tags, versions, type conflicts, adjacency, and replacement policy. Registry queries return deterministic tag/version order. Duplicate registrations, nonadjacent migrations, missing migration steps, invalid tags, or incompatible versions raise `ValidationError`.

Guarantees (`E`, `P`, `B`, `K`). The registry is explicit and never imports a class named by persisted data. Encoders and decoders determine mathematical fidelity. The registry provides deterministic dispatch and migration sequencing. The reference package registers 51 codecs and one adjacent packet-jet-snapshot migration. Lookup is dictionary-based. Listing sorts the finite registry.

Evidence and links (`V`, `L`). Persistence, migration, and release-contract tests verify all registered tags, versions, type names, adjacent paths, and roundtrips. See Chapters 61 and 72 and Appendix G.

B.4 · `register_portable_codec`

Identity (`q`, `a`, `t`, `k`). Canonical beta route `komorebi_math.beta.register_portable_codec`. Identity-equal audited re-exports `komorebi_math.register_portable_codec`, `komorebi_math.export.register_portable_codec`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `function`. Reflected implementation `komorebi_math.export.persistence.register_portable_codec`. Profile `B4-R1`.

Manifest signature (`s`).

```
(type_: 'type[T]', *, tag: 'str', version: 'int', encoder: 'Callable[[T], Any]', decoder: 'Callable[[Any], T]',
    ↪ replace: 'bool' = False) -> 'None'
```

Definition. Register one stable round-trip codec.

Contract (`D`, `C`, `H`, `R`, `X`). Codec records bind a stable tag and positive version to one Python type and explicit encoder/decoder. Migration records bind one adjacent `n -> n+1` safe-data transformation. Registration validates tags, versions, type conflicts, adjacency, and replacement policy. Registry queries return deterministic tag/version order. Duplicate registrations, nonadjacent migrations, missing migration steps, invalid tags, or incompatible versions raise `ValidationError`.

Guarantees (`E`, `P`, `B`, `K`). The registry is explicit and never imports a class named by persisted data. Encoders and decoders determine mathematical fidelity. The registry provides deterministic dispatch and migration sequencing.

The reference package registers 51 codecs and one adjacent packet-jet-snapshot migration. Lookup is dictionary-based. Listing sorts the finite registry.

Evidence and links (V, L). Persistence, migration, and release-contract tests verify all registered tags, versions, type names, adjacent paths, and roundtrips. See Chapters 61 and 72 and Appendix G.

B.4 · register_portable_migration

Identity (q, a, t, k). Canonical beta route `komorebi_math.beta.register_portable_migration`. Identity-equal audited re-exports `komorebi_math.register_portable_migration`, `komorebi_math.export.register_portable_migration`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `function`. Reflected implementation `komorebi_math.export.persistence.register_portable_migration`. Profile `B4-R1`.

Manifest signature (s).

```
(*, tag: 'str', from_version: 'int', to_version: 'int', migrator: 'Migrator', replace: 'bool' = False) -> 'None'
```

Definition. Register one deterministic adjacent migration for codec data.

Contract (D, C, H, R, X). Codec records bind a stable tag and positive version to one Python type and explicit encoder/decoder. Migration records bind one adjacent `n -> n+1` safe-data transformation. Registration validates tags, versions, type conflicts, adjacency, and replacement policy. Registry queries return deterministic tag/version order. Duplicate registrations, nonadjacent migrations, missing migration steps, invalid tags, or incompatible versions raise `ValidationError`.

Guarantees (E, P, B, K). The registry is explicit and never imports a class named by persisted data. Encoders and decoders determine mathematical fidelity. The registry provides deterministic dispatch and migration sequencing. The reference package registers 51 codecs and one adjacent packet-jet-snapshot migration. Lookup is dictionary-based. Listing sorts the finite registry.

Evidence and links (V, L). Persistence, migration, and release-contract tests verify all registered tags, versions, type names, adjacent paths, and roundtrips. See Chapters 61 and 72 and Appendix G.

B.4 · portable_codecs

Identity (q, a, t, k). Canonical beta route `komorebi_math.beta.portable_codecs`. Identity-equal audited re-exports `komorebi_math.portable_codecs`, `komorebi_math.export.portable_codecs`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `function`. Reflected implementation `komorebi_math.export.persistence.portable_codecs`. Profile `B4-R1`.

Manifest signature (s).

```
() -> 'tuple[PortableCodec, ...]'
```

Definition. Return the active codecs ordered by stable tag.

Contract (D, C, H, R, X). Codec records bind a stable tag and positive version to one Python type and explicit encoder/decoder. Migration records bind one adjacent `n -> n+1` safe-data transformation. Registration validates tags, versions, type conflicts, adjacency, and replacement policy. Registry queries return deterministic tag/version order. Duplicate registrations, nonadjacent migrations, missing migration steps, invalid tags, or incompatible versions raise `ValidationError`.

Guarantees (E, P, B, K). The registry is explicit and never imports a class named by persisted data. Encoders and decoders determine mathematical fidelity. The registry provides deterministic dispatch and migration sequencing. The reference package registers 51 codecs and one adjacent packet-jet-snapshot migration. Lookup is dictionary-based. Listing sorts the finite registry.

Evidence and links (V, L). Persistence, migration, and release-contract tests verify all registered tags, versions, type names, adjacent paths, and roundtrips. See Chapters 61 and 72 and Appendix G.

B.4 · portable_migrations

Identity (q, a, t, k). Canonical beta route `komorebi_math.beta.portable_migrations`. Identity-equal audited re-exports `komorebi_math.portable_migrations`, `komorebi_math.export.portable_migrations`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `function`. Reflected implementation `komorebi_math.export.persistence.portable_migrations`. Profile `B4-R1`.

Manifest signature (s).

```
() -> 'tuple[PortableMigration, ...]'
```

Definition. Return registered codec migrations in deterministic order.

Contract (D, C, H, R, X). Codec records bind a stable tag and positive version to one Python type and explicit encoder/decoder. Migration records bind one adjacent `n -> n+1` safe-data transformation. Registration validates tags, versions, type conflicts, adjacency, and replacement policy. Registry queries return deterministic tag/version order. Duplicate registrations, nonadjacent migrations, missing migration steps, invalid tags, or incompatible versions raise `ValidationError`.

Guarantees (E, P, B, K). The registry is explicit and never imports a class named by persisted data. Encoders and decoders determine mathematical fidelity. The registry provides deterministic dispatch and migration sequencing. The reference package registers 51 codecs and one adjacent packet-jet-snapshot migration. Lookup is dictionary-based. Listing sorts the finite registry.

Evidence and links (V, L). Persistence, migration, and release-contract tests verify all registered tags, versions, type names, adjacent paths, and roundtrips. See Chapters 61 and 72 and Appendix G.

B.4 · to_portable_data

Identity (q, a, t, k). Canonical beta route `komorebi_math.beta.to_portable_data`. Identity-equal audited re-exports `komorebi_math.export.to_portable_data`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `function`. Reflected implementation `komorebi_math.export.persistence.to_portable_data`. Profile `B4-C1`.

Manifest signature (s).

```
(value: 'Any', *, max_depth: 'int' = 128) -> 'Any'
```

Definition. Encode one value into the versioned portable payload tree.

Contract (D, C, H, R, X). Encoding maps supported primitive containers and registered types to a versioned safe payload tree. Decoding validates that tree and applies every registered adjacent migration to the active codec version. Digest computes canonical SHA-256 over encoded payload. Text functions wrap or validate the complete schema/version/payload/digest JSON envelope. Unsupported types, excessive depth, duplicate keys, nonfinite numbers, malformed envelopes, digest mismatch, unknown tags, future versions, or missing migrations raise `ValidationError`.

Guarantees (E, P, B, K). Canonical JSON uses sorted keys, compact separators for digests, UTF-8, and no NaN or infinity. SHA-256 supplies corruption detection, not authentication. Roundtrip fidelity is the codec contract. Work and memory are linear in encoded payload size subject to default depth guard 128. No arbitrary import, pickle, network, or backend fallback occurs.

Evidence and links (V, L). Codec tests cover 51 built-ins, nested containers, deterministic digests, depth limits, malformed and adversarial JSON, v1-to-v2 packet-jet snapshot migration, and exact roundtrips. See Chapters 61 and 72.

B.4 · from_portable_data

Identity (q, a, t, k). Canonical beta route `komorebi_math.beta.from_portable_data`. Identity-equal audited re-exports `komorebi_math.export.from_portable_data`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `function`. Reflected implementation `komorebi_math.export.persistence.from_portable_data`. Profile `B4-C1`.

Manifest signature (`s`).

```
(value: 'Any', *, max_depth: 'int' = 128) -> 'Any'
```

Definition. Decode one previously encoded portable payload tree.

Contract (`D, C, H, R, X`). Encoding maps supported primitive containers and registered types to a versioned safe payload tree. Decoding validates that tree and applies every registered adjacent migration to the active codec version. Digest computes canonical SHA-256 over encoded payload. Text functions wrap or validate the complete schema/version/payload/digest JSON envelope. Unsupported types, excessive depth, duplicate keys, nonfinite numbers, malformed envelopes, digest mismatch, unknown tags, future versions, or missing migrations raise `ValidationError`.

Guarantees (`E, P, B, K`). Canonical JSON uses sorted keys, compact separators for digests, UTF-8, and no NaN or infinity. SHA-256 supplies corruption detection, not authentication. Roundtrip fidelity is the codec contract. Work and memory are linear in encoded payload size subject to default depth guard 128. No arbitrary import, pickle, network, or backend fallback occurs.

Evidence and links (`V, L`). Codec tests cover 51 built-ins, nested containers, deterministic digests, depth limits, malformed and adversarial JSON, v1-to-v2 packet-jet snapshot migration, and exact roundtrips. See Chapters 61 and 72.

B.4 · `portable_digest`

Identity (`q, a, t, k`). Canonical beta route `komorebi_math.beta.portable_digest`. Identity-equal audited re-exports `komorebi_math.portable_digest`, `komorebi_math.export.portable_digest`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `function`. Reflected implementation `komorebi_math.export.persistence.portable_digest`. Profile `B4-C1`.

Manifest signature (`s`).

```
(value: 'Any', *, max_depth: 'int' = 128) -> 'str'
```

Definition. Return the canonical SHA256 digest of one encoded payload. The digest is computed over the encoded payload tree rather than presentation indentation.

Contract (`D, C, H, R, X`). Encoding maps supported primitive containers and registered types to a versioned safe payload tree. Decoding validates that tree and applies every registered adjacent migration to the active codec version. Digest computes canonical SHA-256 over encoded payload. Text functions wrap or validate the complete schema/version/payload/digest JSON envelope. Unsupported types, excessive depth, duplicate keys, nonfinite numbers, malformed envelopes, digest mismatch, unknown tags, future versions, or missing migrations raise `ValidationError`.

Guarantees (`E, P, B, K`). Canonical JSON uses sorted keys, compact separators for digests, UTF-8, and no NaN or infinity. SHA-256 supplies corruption detection, not authentication. Roundtrip fidelity is the codec contract. Work and memory are linear in encoded payload size subject to default depth guard 128. No arbitrary import, pickle, network, or backend fallback occurs.

Evidence and links (`V, L`). Codec tests cover 51 built-ins, nested containers, deterministic digests, depth limits, malformed and adversarial JSON, v1-to-v2 packet-jet snapshot migration, and exact roundtrips. See Chapters 61 and 72.

B.4 · `dumps_portable`

Identity (`q, a, t, k`). Canonical beta route `komorebi_math.beta.dumps_portable`. Identity-equal audited re-exports `komorebi_math.dumps_portable`, `komorebi_math.export.dumps_portable`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `function`. Reflected implementation `komorebi_math.export.persistence.dumps_portable`. Profile `B4-C1`.

Manifest signature (s).

```
(value: 'Any', *, indent: 'int | None' = 2, max_depth: 'int' = 128) -> 'str'
```

Definition. Serialize one value into a complete digest-checked JSON envelope.

Contract (D, C, H, R, X). Encoding maps supported primitive containers and registered types to a versioned safe payload tree. Decoding validates that tree and applies every registered adjacent migration to the active codec version. Digest computes canonical SHA-256 over encoded payload. Text functions wrap or validate the complete schema/version/payload/digest JSON envelope. Unsupported types, excessive depth, duplicate keys, nonfinite numbers, malformed envelopes, digest mismatch, unknown tags, future versions, or missing migrations raise `ValidationError`.

Guarantees (E, P, B, K). Canonical JSON uses sorted keys, compact separators for digests, UTF-8, and no NaN or infinity. SHA-256 supplies corruption detection, not authentication. Roundtrip fidelity is the codec contract. Work and memory are linear in encoded payload size subject to default depth guard 128. No arbitrary import, pickle, network, or backend fallback occurs.

Evidence and links (V, L). Codec tests cover 51 built-ins, nested containers, deterministic digests, depth limits, malformed and adversarial JSON, v1-to-v2 packet-jet snapshot migration, and exact roundtrips. See Chapters 61 and 72.

B.4 · `loads_portable`

Identity (q, a, t, k). Canonical beta route `komorebi_math.beta.loads_portable`. Identity-equal audited re-exports `komorebi_math.loads_portable, komorebi_math.export.loads_portable`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `function`. Reflected implementation `komorebi_math.export.persistence.loads_portable`. Profile `B4-C1`.

Manifest signature (s).

```
(text: 'str | bytes', *, max_depth: 'int' = 128) -> 'Any'
```

Definition. Validate and deserialize one complete portable JSON envelope.

Contract (D, C, H, R, X). Encoding maps supported primitive containers and registered types to a versioned safe payload tree. Decoding validates that tree and applies every registered adjacent migration to the active codec version. Digest computes canonical SHA-256 over encoded payload. Text functions wrap or validate the complete schema/version/payload/digest JSON envelope. Unsupported types, excessive depth, duplicate keys, nonfinite numbers, malformed envelopes, digest mismatch, unknown tags, future versions, or missing migrations raise `ValidationError`.

Guarantees (E, P, B, K). Canonical JSON uses sorted keys, compact separators for digests, UTF-8, and no NaN or infinity. SHA-256 supplies corruption detection, not authentication. Roundtrip fidelity is the codec contract. Work and memory are linear in encoded payload size subject to default depth guard 128. No arbitrary import, pickle, network, or backend fallback occurs.

Evidence and links (V, L). Codec tests cover 51 built-ins, nested containers, deterministic digests, depth limits, malformed and adversarial JSON, v1-to-v2 packet-jet snapshot migration, and exact roundtrips. See Chapters 61 and 72.

B.4 · `save_portable`

Identity (q, a, t, k). Canonical beta route `komorebi_math.beta.save_portable`. Identity-equal audited re-exports `komorebi_math.save_portable, komorebi_math.export.save_portable`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `function`. Reflected implementation `komorebi_math.export.persistence.save_portable`. Profile `B4-F1`.

Manifest signature (`s`).

```
(value: 'Any', path: 'str | os.PathLike[str]', *, overwrite: 'bool' = False, indent: 'int | None' = 2, max_depth:
   ↪  'int' = 128) -> 'Path'
```

Definition. Atomically write a portable envelope to disk. The default is refusal to overwrite and two-space JSON indentation.

Contract (`D, C, H, R, X`). Save encodes one supported value, creates a complete digest-checked envelope, writes a sibling temporary file, and atomically replaces the destination. Overwrite is refused unless explicitly enabled. Load reads UTF-8 text and delegates to strict envelope validation and decoding. Invalid paths, existing destinations, unsupported values, depth violations, I/O errors, malformed JSON, digest mismatch, unknown codecs, or migration gaps raise explicit exceptions.

Guarantees (`E, P, B, K`). A successful save is atomic at the filesystem replacement boundary and deterministic for the same encoded value and indentation policy. It provides integrity but not authenticity or crash-proof guarantees beyond the host filesystem. Runtime and memory are linear in serialized payload size. No remote storage or GPU backend is used.

Evidence and links (`V, L`). Filesystem persistence tests cover overwrite refusal, atomic replacement, exact roundtrip, tamper and truncation rejection, and migration on load. See Chapter 61 and Appendix G.

B.4 · `load_portable`

Identity (`q, a, t, k`). Canonical beta route `komorebi_math.beta.load_portable`. Identity-equal audited re-exports `komorebi_math.load_portable`, `komorebi_math.export.load_portable`. Tier `[BETA]` under `1.0-beta-additive-strict-signatures`. Manifest kind `function`. Reflected implementation `komorebi_math.export.persistence.load_portable`. Profile `B4-F1`.

Manifest signature (`s`).

```
(path: 'str | os.PathLike[str]', *, max_depth: 'int' = 128) -> 'Any'
```

Definition. Read and validate a portable envelope from disk.

Contract (`D, C, H, R, X`). Save encodes one supported value, creates a complete digest-checked envelope, writes a sibling temporary file, and atomically replaces the destination. Overwrite is refused unless explicitly enabled. Load reads UTF-8 text and delegates to strict envelope validation and decoding. Invalid paths, existing destinations, unsupported values, depth violations, I/O errors, malformed JSON, digest mismatch, unknown codecs, or migration gaps raise explicit exceptions.

Guarantees (`E, P, B, K`). A successful save is atomic at the filesystem replacement boundary and deterministic for the same encoded value and indentation policy. It provides integrity but not authenticity or crash-proof guarantees beyond the host filesystem. Runtime and memory are linear in serialized payload size. No remote storage or GPU backend is used.

Evidence and links (`V, L`). Filesystem persistence tests cover overwrite refusal, atomic replacement, exact roundtrip, tamper and truncation rejection, and migration on load. See Chapter 61 and Appendix G.

> **Beta-reference coverage.** The recorded validation found 83/83 unique export cards and 83/83 unique canonical anchors. It matched all 67 non-null manifest signatures and all 16 literal `null` signatures and covered all 208 audited beta/root/domain qualified identities. The strict `KM-API-010` compatibility check reported zero added, removed, kind-changed, or signature-changed entries, and the 15-case `KM-GOLDEN-011` fixture passed. This establishes the curated beta exported-symbol reference only. Appendix C records the research surface, and the following appendices provide its operational crosswalks.

Appendix C. Research API: 1,312-entry reference

Finding an object in the research reference. This appendix is designed for lookup as well as sustained reading. If you know the exported name, go to the alphabetical index in Section C.8 and follow the route for the namespace used by your program. If you know its implementation module, use the owner index in C.7. If you know only the mathematical task, use the namespace map below and the corresponding explanatory chapters.

Task or domain	Reference destination	Explanatory chapters
Certificates, configuration, compatibility, and provenance	Root contracts in C.1A	5 through 8 and 62
Typed expressions, evaluation, and planning	Root expression cards in C.1B	50 through 52
Bases, changes of coordinates, interpolation, and frames	Coordinate cards in C.2	15 through 20
Formal series, normalization, masks, and arithmetic generating functions	Generating cards in C.3	21 through 25
Windows, filters, spectral transforms, resampling, and wavelets	Signal cards in C.4	31 through 36A
Derivatives, packet jets, correction, lifting, and streaming	Nonlinear cards in C.5	37 through 44
Rendering, writing files, and portable objects	Export cards in C.6	59 through 62
Other root-level mathematical families	Root cards in C.1 and the owner index in C.7	Follow the individual card's chapter links

The root namespace and a domain namespace can expose the same runtime object. Their separate cards preserve the import routes that users encounter. Section C.10 collects the alias and tier relationships, so a second matching name does not need to be interpreted as a second algorithm. Conversely, similar names or signatures do not establish equivalence. Read each card's definition, domain, and guarantees before substituting one object for another.

Use the card as a contract. The reading key in Appendix A explains the fields shared by all API cards. Begin with the definition and import route, then read the signature together with the input shapes, hypotheses, and return contract. Follow the evidence links for an example before adapting the operation to larger or different data. Research is a compatibility tier, not a single mathematical truth status: an exact finite algorithm, a tolerance-qualified numerical computation, and a conditional construction can all appear here. The individual guarantees state which one you are using.

When comparing two possible operations, keep five questions beside the cards:

1. Do they accept the same scalar domain and dimensions?
2. Do their labels, basis order, coordinate conventions, and units agree?
3. Do they return the same kind of value or certificate-bearing record?
4. Do they require different backends, hypotheses, or resource budgets?
5. Do their guarantees cover the result you intend to use?

Answering these questions is more reliable than selecting a function because its name sounds closest to the task. Chapter 8 explains scalar and backend choices; the domain chapters supply the ordering conventions and examples.

Reading the indices. Section C.7 groups routes by the module that owns the implementation. Section C.8 groups them by exported name. Section C.9 groups exact reflected signatures, including names for which the manifest

records `null`. Section C.10 groups declared aliases across stability tiers. Each index answers a different lookup question; none replaces the card's operational contract. In particular, a shared signature is a shared calling form, not proof of identical mathematical behavior.

Working from a failure. Keep the exact exported route, the input dimensions and scalar domain, the backend, and the diagnostic together. Compare them with the card's preconditions and failure paragraph. Appendix I provides recovery guidance, Appendix J lists dependency and platform boundaries, and Appendix K explains resource budgets. If the result is conditional, retain the unchecked assumptions when reporting it. If the result is numerical, retain the tolerance and residual information. This makes the result useful to the next reader as well as to the program that produced it.

The signatures and execution observations below belong to the reference baseline stated in the preface. Literal version strings, source identifiers, and recorded outputs are retained for reproducibility. Use the current-source crosswalk when you need to distinguish that record from later source tests.

Appendix C assigns one card to each namespace/name pair in the broad public manifest. The root namespace contributes 516 of the 1,312 entries. Cards are grouped by namespace and contract family rather than by a purely alphabetic order. The records were built from the committed `1.0.0b4` manifest, matched to runtime reflection, and checked against their implementation and focused tests. The evidence applies to the reference baseline and retains the scope stated on each card.

C.1 Root namespace: 516 entries

C.1A Foundational contracts, compatibility, configuration, provenance, and schemas: 46 entries

This foundational root section covers the package metadata and type aliases that govern portable formats, certificate records, context-local settings, deterministic API manifests, deprecation behavior, provenance lookup, and the closed behavioral-golden fixture. Several names are also exposed through curated beta or stable facades. Their stronger facade contracts are stated on each card and do not change the additive policy of the root route.

C.1A · `API_STABILITY_LEVEL`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.API_STABILITY_LEVEL`. Reflected implementation `builtins.str`. No stronger curated facade exports this name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. Stronger facade contracts apply only on the listed facade routes. Manifest kind `data`. Profile `C1-META`.

Manifest signature (`s`).

```
null
```

Definition. The string `pre-beta-additive`, which names the compatibility policy for the broad public manifest.

Contract (`D, C, H, R, X`). Import returns a policy label string. Compatibility functions copy the label into manifests and reports, but the string does not itself run a check or establish semantic equivalence. There is no call-time exception.

Guarantees (`E, P, B, K`). The binding is deterministic package metadata and makes no independent mathematical claim. Import and lookup are constant-time in ordinary Python terms and use no numerical backend, accelerator, network, or filesystem operation. Any later file access or validation belongs to the consuming API.

Evidence and links (`V, L`). The committed manifest, runtime reflection audit, and the manifest source audit and compatibility tests cover this route. See Chapters 4, 7, 43, 52, and 72 as applicable.

C.1A · ApiCompatibilityReport

Identity (q, a, t, k). Public research-manifest route `komorebi_math.ApiCompatibilityReport`. Reflected implementation `komorebi_math.compatibility.ApiCompatibilityReport`. The same object is available through the curated beta route `komorebi_math.beta.ApiCompatibilityReport` and the curated stable routes `komorebi_math.stable.ApiCompatibilityReport`, `komorebi_math.v1.ApiCompatibilityReport`, and `komorebi_math.api.v1.ApiCompatibilityReport`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. Stronger facade contracts apply only on the listed facade routes. Manifest kind `class`. Profile `C1-API-REPORT`.

Manifest signature (s).

```
(reference_digest: 'str', current_digest: 'str', added: 'tuple[tuple[str, str], ...]', removed: 'tuple[tuple[str,
    ↪ str], ...]', kind_changed: 'tuple[tuple[str, str, str, str], ...]', signature_changed: 'tuple[tuple[str,
    ↪ str, str | None, str | None], ...]', strict_signatures: 'bool', certificate: 'Certificate') -> None
```

Definition. An immutable structural diff between a reference API manifest and an observed manifest, together with the compatibility certificate.

Contract (D, C, H, R, X). The constructor stores both digests and the four structural difference sets. `compatible` delegates to the embedded certificate. `to_dict()` returns the report and serialized certificate. The record does not rerun reflection and does not establish behavioral or mathematical equivalence.

Guarantees (E, P, B, K). The records are exact structural metadata, not semantic certificates. Storage is linear in the number and length of recorded symbols. Manifest sorting and canonical digest construction are `O(n log n)` in symbol count plus serialized size. Map construction is linear. The implementation uses the standard library and imports no optional numerical backend.

Evidence and links (V, L). The committed manifest, runtime reflection audit, and `test_block10_compatibility.py`, `test_block11_golden_profiling_beta.py`, and the CLI compatibility tests cover this route. See Chapters 7, 72, and 73.

C.1A · BEHAVIORAL_GOLDEN_SCHEMA

Identity (q, a, t, k). Public research-manifest route `komorebi_math.BEHAVIORAL_GOLDEN_SCHEMA`. Reflected implementation `builtins.str`. The same object is available through the curated beta route `komorebi_math.beta.BEHAVIORAL_GOLDEN_SCHEMA`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. Stronger facade contracts apply only on the listed facade routes. Manifest kind `data`. Profile `C1-META`.

Manifest signature (s).

```
null
```

Definition. The schema identifier `komorebi-math.behavioral-golden` used by the closed semantic fixture document.

Contract (D, C, H, R, X). Import has no arguments and returns the recorded immutable schema string or integer. The constant does not parse or validate a fixture by itself. Validation occurs in `load_behavioral_golden_fixture`. There is no call-time exception.

Guarantees (E, P, B, K). The binding is deterministic package metadata and makes no independent mathematical claim. Import and lookup are constant-time in ordinary Python terms and use no numerical backend, accelerator, network, or filesystem operation. Any later file access or validation belongs to the consuming API.

Evidence and links (V, L). The committed manifest, runtime reflection audit, and `test_block11_golden_profiling_beta.py` cover this route. See Chapters 4, 7, 43, 52, and 72 as applicable.

C.1A · BEHAVIORAL_GOLDEN_SCHEMA_VERSION

Identity (q, a, t, k). Public research-manifest route `komorebi_math.BEHAVIORAL_GOLDEN_SCHEMA_VERSION`. Reflected implementation `builtins.int`. The same object is available through the curated beta route `komorebi_math.beta.BEHAVIORAL_GOLDEN_SCHEMA_VERSION`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. Stronger facade contracts apply only on the listed facade routes. Manifest kind `data`. Profile `C1-META`.

Manifest signature (s).

```
null
```

Definition. The integer `1`, the only behavioral-golden schema version accepted by this release.

Contract (D, C, H, R, X). Import has no arguments and returns the recorded immutable schema string or integer. The constant does not parse or validate a fixture by itself. Validation occurs in `load_behavioral_golden_fixture`. There is no call-time exception.

Guarantees (E, P, B, K). The binding is deterministic package metadata and makes no independent mathematical claim. Import and lookup are constant-time in ordinary Python terms and use no numerical backend, accelerator, network, or filesystem operation. Any later file access or validation belongs to the consuming API.

Evidence and links (V, L). The committed manifest, runtime reflection audit, and `test_block11_golden_profiling_beta.py` cover this route. See Chapters 4, 7, 43, 52, and 72 as applicable.

C.1A · BETA_API_STABILITY_LEVEL

Identity (q, a, t, k). Public research-manifest route `komorebi_math.BETA_API_STABILITY_LEVEL`. Reflected implementation `builtins.str`. The same object is available through the curated beta route `komorebi_math.beta.BETA_API_STABILITY_LEVEL`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. Stronger facade contracts apply only on the listed facade routes. Manifest kind `data`. Profile `C1-META`.

Manifest signature (s).

```
null
```

Definition. The string `1.0-beta-additive-strict-signatures`, which names the curated beta compatibility policy.

Contract (D, C, H, R, X). Import returns a policy label string. Compatibility functions copy the label into manifests and reports, but the string does not itself run a check or establish semantic equivalence. There is no call-time exception.

Guarantees (E, P, B, K). The binding is deterministic package metadata and makes no independent mathematical claim. Import and lookup are constant-time in ordinary Python terms and use no numerical backend, accelerator, network, or filesystem operation. Any later file access or validation belongs to the consuming API.

Evidence and links (V, L). The committed manifest, runtime reflection audit, and the manifest source audit and compatibility tests cover this route. See Chapters 4, 7, 43, 52, and 72 as applicable.

C.1A · BETA_PUBLIC_NAMESPACES

Identity (q, a, t, k). Public research-manifest route `komorebi_math.BETA_PUBLIC_NAMESPACES`. Reflected implementation `builtins.tuple`. No stronger curated facade exports this name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. Stronger facade contracts apply only on the listed facade routes. Manifest kind `data`. Profile `C1-META`.

Manifest signature (s).

```
null
```

Definition. The one-element tuple `('komorebi_math.beta',)` inspected when the curated beta manifest is created or checked.

Contract (D, C, H, R, X). Import returns a tuple of namespace strings. Compatibility helpers consume it as the complete curated beta namespace set. The tuple is data, not an import operation, and raises no call-time exception.

Guarantees (E, P, B, K). The binding is deterministic package metadata and makes no independent mathematical claim. Import and lookup are constant-time in ordinary Python terms and use no numerical backend, accelerator, network, or filesystem operation. Any later file access or validation belongs to the consuming API.

Evidence and links (V, L). The committed manifest, runtime reflection audit, and the manifest source audit and compatibility tests cover this route. See Chapters 4, 7, 43, 52, and 72 as applicable.

C.1A · BehavioralGoldenCaseResult

Identity (q, a, t, k). Public research-manifest route `komorebi_math.BehavioralGoldenCaseResult`. Reflected implementation `komorebi_math.golden.BehavioralGoldenCaseResult`. The same object is available through the curated beta route `komorebi_math.beta.BehavioralGoldenCaseResult`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. Stronger facade contracts apply only on the listed facade routes. Manifest kind `class`. Profile `C1-GOLDEN-CASE`.

Manifest signature (s).

```
(name: 'str', kind: 'str', observed: 'Any', expected: 'Any', certificate: 'Certificate') -> None
```

Definition. An immutable record containing one golden case's name, kind, observed value, expected value, and certificate.

Contract (D, C, H, R, X). The constructor stores observed and expected values plus a certificate. `passed` delegates to the certificate, and `to_dict()` serializes the result and certificate. Equality policy was decided by the runner that created the record. Constructing the record does not recompute it.

Guarantees (E, P, B, K). The record preserves exactness and pass status supplied by its certificate. Storage and serialization are linear in contained values and case count. No runner executes during record construction, and no backend fallback occurs.

Evidence and links (V, L). The committed manifest, runtime reflection audit, and `test_block11_golden_profiling_beta.py` cover this route. See Chapters 7 and 72.

C.1A · BehavioralGoldenReport

Identity (q, a, t, k). Public research-manifest route `komorebi_math.BehavioralGoldenReport`. Reflected implementation `komorebi_math.golden.BehavioralGoldenReport`. The same object is available through the curated beta route `komorebi_math.beta.BehavioralGoldenReport`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. Stronger facade contracts apply only on the listed facade routes. Manifest kind `class`. Profile `C1-GOLDEN-REPORT`.

Manifest signature (s).

```
(fixture_path: 'Path', fixture_digest: 'str', results: 'tuple[BehavioralGoldenCaseResult, ...]', certificate:
    ↪ 'Certificate') -> None
```

Definition. An immutable aggregate of the fixture path, canonical fixture digest, case results, and overall certificate.

Contract (D, C, H, R, X). The constructor stores the fixture path, 64-character digest text, ordered case results, and aggregate certificate. `passed` delegates to that certificate, and `to_dict()` converts the path to text and serializes every case. Construction alone does not rerun the fixture.

Guarantees (E, P, B, K). The record preserves exactness and pass status supplied by its certificate. Storage and serialization are linear in contained values and case count. No runner executes during record construction, and no backend fallback occurs.

Evidence and links (V, L). The committed manifest, runtime reflection audit, and `test_block11_golden_profiling_beta.py` cover this route. See Chapters 7 and 72.

C.1A · Certificate

Identity (q, a, t, k). Public research-manifest route `komorebi_math.Certificate`. Reflected implementation `komorebi_math.certificates.Certificate`. The same object is available through the curated beta route `komorebi_math.beta.Certificate` and the curated stable routes `komorebi_math.stable.Certificate`, `komorebi_math.v1.Certificate`, and `komorebi_math.api.v1.Certificate`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. Stronger facade contracts apply only on the listed facade routes. Manifest kind `class`. Profile `C1-CERT`.

Manifest signature (s).

```
(valid: 'bool', exactness: 'Exactness', residuals: 'dict[str, float]' = <factory>, tolerance: 'float | None' =
    ↪ None, assumptions_checked: 'tuple[str, ...]' = (), assumptions_unchecked: 'tuple[str, ...]' = (),
    ↪ diagnostics: 'dict[str, Any]' = <factory>, theorem_id: 'str | None' = None, message: 'str' = '') -> None
```

Definition. An immutable machine-readable validation record carrying validity, evidence class, residuals, tolerances, assumptions, diagnostics, an optional theorem identifier, and a message.

Contract (D, C, H, R, X). The constructor records the supplied fields without running a theorem or backend. `require()` returns the certificate when `valid` is true and otherwise raises `ValidationError` using `message` or a default failure message. `to_dict()` converts `exactness` to its string value. `to_json()` emits sorted JSON and stringifies otherwise unsupported diagnostic values.

Guarantees (E, P, B, K). These records are backend-neutral and allocate storage proportional to the supplied residual, assumption, and diagnostic payloads. They preserve the declared evidence class but do not strengthen it. Serialization work is linear in the serialized record size. No numerical computation occurs unless caller-supplied diagnostic conversion performs it.

Evidence and links (V, L). The committed manifest, runtime reflection audit, and `test_certificates_registry.py` and `test_export_serialization.py` cover this route. See Chapters 1, 5, 6, and 72.

C.1A · `ClaimStatus`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.ClaimStatus`. Reflected implementation `komorebi_math.provenance.ClaimStatus`. No stronger curated facade exports this name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. Stronger facade contracts apply only on the listed facade routes. Manifest kind `class`. Profile `C1-PROV-ENUM`.

Manifest signature (s).

```
(*values)
```

Definition. A string-valued provenance enumeration distinguishing definition, proved, classical, conditional, finite-certificate, heuristic, conjectural, and planned claims.

Contract (D, C, H, R, X). Construction follows `StrEnum` semantics. The eight string values classify the status of a documented claim and do not themselves verify it. Invalid values raise the standard enum `ValueError`.

Guarantees (E, P, B, K). Provenance metadata is descriptive and does not prove the cited claim. Record lookup is constant-time for registered names and otherwise includes module import plus attribute traversal proportional to path length. The registry is process-local, backend-neutral, and unauthenticated.

Evidence and links (V, L). The committed manifest, runtime reflection audit, and `test_certificates_registry.py` and provenance-bearing domain tests cover this route. See Chapters 5 and 72.

C.1A · `ComputationResult`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.ComputationResult`. Reflected implementation `komorebi_math.certificates.ComputationResult`. The same object is available through the curated beta route `komorebi_math.beta.ComputationResult` and the curated stable routes `komorebi_math.stable.ComputationResult`, `komorebi_math.v1.ComputationResult`, and `komorebi_math.api.v1.ComputationResult`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. Stronger facade contracts apply only on the listed facade routes. Manifest kind `class`. Profile `C1-CERT-RESULT`.

Manifest signature (s).

```
(value: 'T', certificate: 'Certificate') -> None
```

Definition. An immutable generic pair of a computed value and the `Certificate` that qualifies it.

Contract (D, C, H, R, X). The constructor stores an arbitrary value and its certificate. `valid` delegates to `certificate.valid`, `exact` is true only for `Exactness.EXACT`, and `require()` first requires the certificate and then returns the value. Any `ValidationError` from the certificate is propagated.

Guarantees (E, P, B, K). These records are backend-neutral and allocate storage proportional to the supplied residual, assumption, and diagnostic payloads. They preserve the declared evidence class but do not strengthen it. Serialization work is linear in the serialized record size. No numerical computation occurs unless caller-supplied diagnostic conversion performs it.

Evidence and links (V, L). The committed manifest, runtime reflection audit, and `test_certificates_registry.py` and `test_export_serialization.py` cover this route. See Chapters 5 and 61.

C.1A · `ConditionalResult`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.ConditionalResult`. Reflected implementation `komorebi_math.certificates.ConditionalResult`. The same object is available through the curated beta route `komorebi_math.beta.ConditionalResult` and the curated stable routes `komorebi_math.stable.ConditionalResult`, `komorebi_math.v1.ConditionalResult`, and `komorebi_math.api.v1.ConditionalResult`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. Stronger facade contracts apply only on the listed facade routes. Manifest kind `class`. Profile `C1-CERT-COND`.

Manifest signature (s).

```
(value: 'T', certificate: 'Certificate') -> None
```

Definition. An immutable generic pair whose certificate must have `Exactness.CONDITIONAL`.

Contract (D, C, H, R, X). The constructor accepts an arbitrary value and a certificate, then rejects the pair with `ValueError` unless `certificate.exactness` is `Exactness.CONDITIONAL`. It does not independently verify the listed assumptions or the mathematical conclusion.

Guarantees (E, P, B, K). These records are backend-neutral and allocate storage proportional to the supplied residual, assumption, and diagnostic payloads. They preserve the declared evidence class but do not strengthen it. Serialization work is linear in the serialized record size. No numerical computation occurs unless caller-supplied diagnostic conversion performs it.

Evidence and links (V, L). The committed manifest, runtime reflection audit, and `test_certificates_registry.py` and `test_export_serialization.py` cover this route. See Chapters 5 and 6.

C.1A · `CoordinateSpace`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.CoordinateSpace`. Reflected implementation `types.UnionType`. No stronger curated facade exports this name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. Stronger facade contracts apply only on the listed facade routes. Manifest kind `data`. Profile `C1-TYPE`.

Manifest signature (s).

```
null
```

Definition. A PEP 604 union alias for `RationalSpace | PolynomialCoordinateSpace`.

Contract (D, C, H, R, X). Import returns a runtime PEP 604 union object used by annotations and type checks. It is not a factory and has no project-defined constructor contract. Values must be instances of one of the listed exact expression-space classes. Unsupported values are rejected by the consuming expression APIs, not by importing the alias.

Guarantees (E, P, B, K). The binding is deterministic package metadata and makes no independent mathematical claim. Import and lookup are constant-time in ordinary Python terms and use no numerical backend, accelerator, network, or filesystem operation. Any later file access or validation belongs to the consuming API.

Evidence and links (V, L). The committed manifest, runtime reflection audit, and `test_block20_expression_calculus.py` cover this route. See Chapters 50 and 52.

C.1A · EXPRESSION_NODE_VERSION

Identity (q, a, t, k). Public research-manifest route `komorebi_math.EXPRESSION_NODE_VERSION`. Reflected implementation `builtins.int`. No stronger curated facade exports this name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. Stronger facade contracts apply only on the listed facade routes. Manifest kind `data`. Profile `C1-META`.

Manifest signature (s).

```
null
```

Definition. The integer `1`, attached to each supported serialized expression-node record.

Contract (D, C, H, R, X). Import has no arguments and returns the recorded immutable schema string or integer. The binding is metadata for strict expression serialization and does not encode, decode, or validate an expression by itself. There is no call-time exception.

Guarantees (E, P, B, K). The binding is deterministic package metadata and makes no independent mathematical claim. Import and lookup are constant-time in ordinary Python terms and use no numerical backend, accelerator, network, or filesystem operation. Any later file access or validation belongs to the consuming API.

Evidence and links (V, L). The committed manifest, runtime reflection audit, and `test_block20_expression_calculus.py` cover this route. See Chapters 4, 7, 43, 52, and 72 as applicable.

C.1A · EXPRESSION_SCHEMA

Identity (q, a, t, k). Public research-manifest route `komorebi_math.EXPRESSION_SCHEMA`. Reflected implementation `builtins.str`. No stronger curated facade exports this name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. Stronger facade contracts apply only on the listed facade routes. Manifest kind `data`. Profile `C1-META`.

Manifest signature (s).

```
null
```

Definition. The envelope schema identifier `komorebi_math.expression.v1` for strict portable typed expressions.

Contract (D, C, H, R, X). Import has no arguments and returns the recorded immutable schema string or integer. The binding is metadata for strict expression serialization and does not encode, decode, or validate an expression by itself. There is no call-time exception.

Guarantees (E, P, B, K). The binding is deterministic package metadata and makes no independent mathematical claim. Import and lookup are constant-time in ordinary Python terms and use no numerical backend, accelerator, network, or filesystem operation. Any later file access or validation belongs to the consuming API.

Evidence and links (V, L). The committed manifest, runtime reflection audit, and `test_block20_expression_calculus.py` cover this route. See Chapters 4, 7, 43, 52, and 72 as applicable.

C.1A · EXPRESSION_SCHEMA_VERSION

Identity (q, a, t, k). Public research-manifest route `komorebi_math.EXPRESSION_SCHEMA_VERSION`. Reflected implementation `builtins.int`. No stronger curated facade exports this name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. Stronger facade contracts apply only on the listed facade routes. Manifest kind `data`. Profile `C1-META`.

Manifest signature (`s`).

```
null
```

Definition. The integer `1`, the accepted version of the outer expression envelope.

Contract (`D, C, H, R, X`). Import has no arguments and returns the recorded immutable schema string or integer. The binding is metadata for strict expression serialization and does not encode, decode, or validate an expression by itself. There is no call-time exception.

Guarantees (`E, P, B, K`). The binding is deterministic package metadata and makes no independent mathematical claim. Import and lookup are constant-time in ordinary Python terms and use no numerical backend, accelerator, network, or filesystem operation. Any later file access or validation belongs to the consuming API.

Evidence and links (`V, L`). The committed manifest, runtime reflection audit, and `test_block20_expression_calculus.py` cover this route. See Chapters 4, 7, 43, 52, and 72 as applicable.

C.1A · `EXPRESSION_TYPE_VERSION`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.EXPRESSION_TYPE_VERSION`. Reflected implementation `builtins.int`. No stronger curated facade exports this name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. Stronger facade contracts apply only on the listed facade routes. Manifest kind `data`. Profile `C1-META`.

Manifest signature (`s`).

```
null
```

Definition. The integer `1`, attached to supported serialized expression-type records.

Contract (`D, C, H, R, X`). Import has no arguments and returns the recorded immutable schema string or integer. The binding is metadata for strict expression serialization and does not encode, decode, or validate an expression by itself. There is no call-time exception.

Guarantees (`E, P, B, K`). The binding is deterministic package metadata and makes no independent mathematical claim. Import and lookup are constant-time in ordinary Python terms and use no numerical backend, accelerator, network, or filesystem operation. Any later file access or validation belongs to the consuming API.

Evidence and links (`V, L`). The committed manifest, runtime reflection audit, and `test_block20_expression_calculus.py` cover this route. See Chapters 4, 7, 43, 52, and 72 as applicable.

C.1A · `Exactness`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.Exactness`. Reflected implementation `komorebi_math.certificates.Exactness`. The same object is available through the curated beta route `komorebi_math.beta.Exactness` and the curated stable routes `komorebi_math.stable.Exactness`, `komorebi_math.v1.Exactness`, and `komorebi_math.api.v1.Exactness`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. Stronger facade contracts apply only on the listed facade routes. Manifest kind `class`. Profile `C1-CERT-ENUM`.

Manifest signature (`s`).

```
(*values)
```

Definition. A string-valued enumeration with the members `EXACT`, `NUMERICAL`, `CONDITIONAL`, `HEURISTIC`, and `PLANNED`.

Contract (`D, C, H, R, X`). Construction follows `StrEnum` semantics. The accepted values are `exact`, `numerical`, `conditional`, `heuristic`, and `planned`. Consumers compare enum members to prevent accidental evidence promotion. Invalid values raise the standard enum `ValueError`.

Guarantees (`E, P, B, K`). These records are backend-neutral and allocate storage proportional to the supplied residual, assumption, and diagnostic payloads. They preserve the declared evidence class but do not strengthen it. Serialization work is linear in the serialized record size. No numerical computation occurs unless caller-supplied diagnostic conversion performs it.

Evidence and links (V, L). The committed manifest, runtime reflection audit, and `test_certificates_registry.py` and `test_export_serialization.py` cover this route. See Chapters 1, 5, and 6.

C.1A · ExpressionType

Identity (q, a, t, k). Public research-manifest route `komorebi_math.ExpressionType`. Reflected implementation `types.UnionType`. No stronger curated facade exports this name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. Stronger facade contracts apply only on the listed facade routes. Manifest kind `data`. Profile `C1-TYPE`.

Manifest signature (s).

```
null
```

Definition. A PEP 604 union alias for `CoordinateSpace | LinearMapType`.

Contract (D, C, H, R, X). Import returns a runtime PEP 604 union object used by annotations and type checks. It is not a factory and has no project-defined constructor contract. Values must be instances of one of the listed exact expression-space classes. Unsupported values are rejected by the consuming expression APIs, not by importing the alias.

Guarantees (E, P, B, K). The binding is deterministic package metadata and makes no independent mathematical claim. Import and lookup are constant-time in ordinary Python terms and use no numerical backend, accelerator, network, or filesystem operation. Any later file access or validation belongs to the consuming API.

Evidence and links (V, L). The committed manifest, runtime reflection audit, and `test_block20_expression_calculus.py` cover this route. See Chapters 50 and 52.

C.1A · KomorebiDeprecationWarning

Identity (q, a, t, k). Public research-manifest route `komorebi_math.KomorebiDeprecationWarning`. Reflected implementation `komorebi_math.compatibility.KomorebiDeprecationWarning`. No stronger curated facade exports this name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. Stronger facade contracts apply only on the listed facade routes. Manifest kind `class`. Profile `C1-DEPRECATION-WARNING`.

Manifest signature (s).

```
null
```

Definition. The package-specific warning category used for explicit public API deprecations.

Contract (D, C, H, R, X). The class is a `DeprecationWarning` subclass used as a category, not an emitted warning instance. Normal Python warning filters therefore apply. The manifest records a literal `null` signature because the class exposes no project-defined constructor contract.

Guarantees (E, P, B, K). The category adds no runtime dependency beyond Python's warning system. Filtering, display, and escalation to errors follow the caller's warning policy. Class lookup is constant-time and has no backend or platform-specific branch.

Evidence and links (V, L). The committed manifest, runtime reflection audit, and `test_block10_compatibility.py`, `test_block11_golden_profiling_beta.py`, and the CLI compatibility tests cover this route. See Chapters 7 and 72.

C.1A · PACKET_JET_CHUNK_SCHEMA

Identity (q, a, t, k). Public research-manifest route `komorebi_math.PACKET_JET_CHUNK_SCHEMA`. Reflected implementation `builtins.str`. The same object is available through the curated beta route `komorebi_math.beta.PACKET_JET_CHUNK_SCHEMA`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. Stronger facade contracts apply only on the listed facade routes. Manifest kind `data`. Profile `C1-META`.

Manifest signature (`s`).

```
null
```

Definition. The schema identifier `komorebi-math.packet-jet-stream.chunk` for one independently digested packet-jet stream chunk.

Contract (`D, C, H, R, X`). Import has no arguments and returns the recorded immutable schema string or integer. The binding names the accepted stream or chunk envelope but performs no I/O and does not validate a directory. Stream readers and writers enforce the value. There is no call-time exception.

Guarantees (`E, P, B, K`). The binding is deterministic package metadata and makes no independent mathematical claim. Import and lookup are constant-time in ordinary Python terms and use no numerical backend, accelerator, network, or filesystem operation. Any later file access or validation belongs to the consuming API.

Evidence and links (`V, L`). The committed manifest, runtime reflection audit, and `test_block11_streaming_migrations.py` cover this route. See Chapters 4, 7, 43, 52, and 72 as applicable.

C.1A · `PACKET_JET_CHUNK_SCHEMA_VERSION`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.PACKET_JET_CHUNK_SCHEMA_VERSION`. Reflected implementation `builtins.int`. The same object is available through the curated beta route `komorebi_math.beta.PACKET_JET_CHUNK_SCHEMA_VERSION`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. Stronger facade contracts apply only on the listed facade routes. Manifest kind `data`. Profile `C1-META`.

Manifest signature (`s`).

```
null
```

Definition. The integer `1`, the accepted packet-jet chunk schema version.

Contract (`D, C, H, R, X`). Import has no arguments and returns the recorded immutable schema string or integer. The binding names the accepted stream or chunk envelope but performs no I/O and does not validate a directory. Stream readers and writers enforce the value. There is no call-time exception.

Guarantees (`E, P, B, K`). The binding is deterministic package metadata and makes no independent mathematical claim. Import and lookup are constant-time in ordinary Python terms and use no numerical backend, accelerator, network, or filesystem operation. Any later file access or validation belongs to the consuming API.

Evidence and links (`V, L`). The committed manifest, runtime reflection audit, and `test_block11_streaming_migrations.py` cover this route. See Chapters 4, 7, 43, 52, and 72 as applicable.

C.1A · `PACKET_JET_STREAM_SCHEMA`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.PACKET_JET_STREAM_SCHEMA`. Reflected implementation `builtins.str`. The same object is available through the curated beta route `komorebi_math.beta.PACKET_JET_STREAM_SCHEMA`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. Stronger facade contracts apply only on the listed facade routes. Manifest kind `data`. Profile `C1-META`.

Manifest signature (`s`).

```
null
```

Definition. The schema identifier `komorebi-math.packet-jet-stream` for a resumable packet-jet directory manifest.

Contract (`D, C, H, R, X`). Import has no arguments and returns the recorded immutable schema string or integer. The binding names the accepted stream or chunk envelope but performs no I/O and does not validate a directory. Stream readers and writers enforce the value. There is no call-time exception.

Guarantees (`E, P, B, K`). The binding is deterministic package metadata and makes no independent mathematical claim. Import and lookup are constant-time in ordinary Python terms and use no numerical backend, accelerator, network, or filesystem operation. Any later file access or validation belongs to the consuming API.

Evidence and links (V, L). The committed manifest, runtime reflection audit, and `test_block11_streaming_migrations.py` cover this route. See Chapters 4, 7, 43, 52, and 72 as applicable.

C.1A · PACKET_JET_STREAM_SCHEMA_VERSION

Identity (q, a, t, k). Public research-manifest route `komorebi_math.PACKET_JET_STREAM_SCHEMA_VERSION`. Reflected implementation `builtins.int`. The same object is available through the curated beta route `komorebi_math.beta.PACKET_JET_STREAM_SCHEMA_VERSION`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. Stronger facade contracts apply only on the listed facade routes. Manifest kind `data`. Profile `C1-META`.

Manifest signature (s).

```
null
```

Definition. The integer `1`, the accepted packet-jet stream-manifest schema version.

Contract (D, C, H, R, X). Import has no arguments and returns the recorded immutable schema string or integer. The binding names the accepted stream or chunk envelope but performs no I/O and does not validate a directory. Stream readers and writers enforce the value. There is no call-time exception.

Guarantees (E, P, B, K). The binding is deterministic package metadata and makes no independent mathematical claim. Import and lookup are constant-time in ordinary Python terms and use no numerical backend, accelerator, network, or filesystem operation. Any later file access or validation belongs to the consuming API.

Evidence and links (V, L). The committed manifest, runtime reflection audit, and `test_block11_streaming_migrations.py` cover this route. See Chapters 4, 7, 43, 52, and 72 as applicable.

C.1A · ProvenanceRecord

Identity (q, a, t, k). Public research-manifest route `komorebi_math.ProvenanceRecord`. Reflected implementation `komorebi_math.provenance.ProvenanceRecord`. No stronger curated facade exports this name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. Stronger facade contracts apply only on the listed facade routes. Manifest kind `class`. Profile `C1-PROV-RECORD`.

Manifest signature (s).

```
(identifier: 'str', title: 'str', status: 'ClaimStatus', source: 'str', assumptions: 'tuple[str, ...]' = (),
    ↪ limitations: 'tuple[str, ...]' = (), verification_suite: 'str | None' = None) -> None
```

Definition. An immutable provenance record containing an identifier, title, claim status, source, assumptions, limitations, and optional verification suite.

Contract (D, C, H, R, X). The constructor stores provenance metadata without resolving the cited source or running a verification suite. `to_dict()` returns dataclass fields with `status` converted to its string value. Assumptions and limitations remain explicit tuples supplied by the caller.

Guarantees (E, P, B, K). Provenance metadata is descriptive and does not prove the cited claim. Record lookup is constant-time for registered names and otherwise includes module import plus attribute traversal proportional to path length. The registry is process-local, backend-neutral, and unauthenticated.

Evidence and links (V, L). The committed manifest, runtime reflection audit, and `test_certificates_registry.py` and provenance-bearing domain tests cover this route. See Chapters 5 and 72.

C.1A · PublicApiManifest

Identity (q, a, t, k). Public research-manifest route `komorebi_math.PublicApiManifest`. Reflected implementation `komorebi_math.compatibility.PublicApiManifest`. The same object is available through the curated beta route `komorebi_math.beta.PublicApiManifest` and the curated stable routes `komorebi_math.stable.PublicApiManifest`, `komorebi_math.v1.PublicApiManifest`, and `komorebi_math.api.v1.PublicApiManifest`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. Stronger facade contracts apply only on the listed facade routes. Manifest kind `class`. Profile `C1-API-MANIFEST`.

Manifest signature (s).

```
(package_version: 'str', namespaces: 'tuple[str, ...]', symbols: 'tuple[PublicApiSymbol, ...]', schema: 'str' =
    ↪ 'komorebi-math.public-api', schema_version: 'int' = 1, stability_level: 'str' = 'pre-beta-additive') -> None
```

Definition. An immutable deterministic snapshot of selected public namespaces and their sorted `__all__` exports.

Contract (D, C, H, R, X). Construction requires the canonical schema and version, a nonempty unique namespace tuple, deterministically sorted unique symbols, and namespace containment. Violations raise `ValidationError`. `to_dict()` emits canonical manifest data, `from_dict()` validates the six top-level fields, `digest` computes SHA-256 over compact sorted JSON, and `symbol_map()` keys records by namespace and name.

Guarantees (E, P, B, K). The records are exact structural metadata, not semantic certificates. Storage is linear in the number and length of recorded symbols. Manifest sorting and canonical digest construction are `O(n log n)` in symbol count plus serialized size. Map construction is linear. The implementation uses the standard library and imports no optional numerical backend.

Evidence and links (V, L). The committed manifest, runtime reflection audit, and `test_block10_compatibility.py`, `test_block11_golden_profiling_beta.py`, and the CLI compatibility tests cover this route. See Chapters 7, 72, 75, and 76.

C.1A · PublicApiSymbol

Identity (q, a, t, k). Public research-manifest route `komorebi_math.PublicApiSymbol`. Reflected implementation `komorebi_math.compatibility.PublicApiSymbol`. The same object is available through the curated beta route `komorebi_math.beta.PublicApiSymbol` and the curated stable routes `komorebi_math.stable.PublicApiSymbol`, `komorebi_math.v1.PublicApiSymbol`, and `komorebi_math.api.v1.PublicApiSymbol`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. Stronger facade contracts apply only on the listed facade routes. Manifest kind `class`. Profile `C1-API-SYMBOL`.

Manifest signature (s).

```
(namespace: 'str', name: 'str', kind: 'str', implementation_module: 'str', qualname: 'str', signature: 'str |
    ↪ None') -> None
```

Definition. An ordered immutable record for one exported name, its broad kind, implementation owner, qualified name, and reflected signature text.

Contract (D, C, H, R, X). The direct dataclass constructor stores one symbol record. `to_dict()` emits the six schema fields. `from_dict()` requires exactly those fields and requires `signature` to be a string or `null`. Schema violations raise `ValidationError`. Ordering is lexicographic across dataclass fields and is used by manifest sorting.

Guarantees (E, P, B, K). The records are exact structural metadata, not semantic certificates. Storage is linear in the number and length of recorded symbols. Manifest sorting and canonical digest construction are `O(n log n)` in symbol count plus serialized size. Map construction is linear. The implementation uses the standard library and imports no optional numerical backend.

Evidence and links (V, L). The committed manifest, runtime reflection audit, and `test_block10_compatibility.py`, `test_block11_golden_profiling_beta.py`, and the CLI compatibility tests cover this route. See Chapters 7, 72, 75, and 76.

C.1A · STAGES

Identity (q, a, t, k). Public research-manifest route `komorebi_math.STAGES`. Reflected implementation `builtins.tuple`. No stronger curated facade exports this name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. Stronger facade contracts apply only on the listed facade routes. Manifest kind `data`. Profile `C1-STAGE`.

Manifest signature (s).

```
null
```

Definition. A nine-entry tuple of `StageSpec` records describing the common seed, address, operate, project, transfer, refine, constrain, complete, and reconstruct workflow.

Contract (D, C, H, R, X). Import returns a fixed tuple of nine immutable stage records. Each record contains a stage enum, a plain-language meaning, an `implemented_in_alpha` flag, and an owning module string. The `complete` stage is explicitly marked false in this release. The tuple is descriptive and does not execute a pipeline.

Guarantees (E, P, B, K). The binding is deterministic package metadata and makes no independent mathematical claim. Import and lookup are constant-time in ordinary Python terms and use no numerical backend, accelerator, network, or filesystem operation. Any later file access or validation belongs to the consuming API.

Evidence and links (V, L). The committed manifest, runtime reflection audit, and the unified-stage source audit and the Chapter 2 execution-language inventory cover this route. See Chapters 2 and 74.

C.1A · `Settings`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.Settings`. Reflected implementation `komorebi_math.config.Settings`. The same object is available through the curated stable routes `komorebi_math.stable.Settings`, `komorebi_math.v1.Settings`, and `komorebi_math.api.v1.Settings`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. Stronger facade contracts apply only on the listed facade routes. Manifest kind `class`. Profile `C1-SETTINGS`.

Manifest signature (s).

```
(backend: 'str' = 'numpy', tolerance: 'float' = 1e-10, validate_on_construction: 'bool' = True) -> None
```

Definition. An immutable context-local settings record with defaults `backend='numpy'`, `tolerance=1e-10`, and `validate_on_construction=True`.

Contract (D, C, H, R, X). The constructor stores a backend label, numeric tolerance, and validation flag. It is a frozen dataclass and does not validate backend availability or tolerance sign. Runtime helpers place instances in a context variable rather than a process-wide mutable singleton.

Guarantees (E, P, B, K). The settings layer is standard-library only and context-local through `ContextVar`. Reads and replacements are constant-size for the three current fields. The backend label is descriptive here, so no automatic backend selection or fallback occurs. Determinism depends only on the current context and supplied values.

Evidence and links (V, L). The committed manifest, runtime reflection audit, and `test_certificates_registry.py` cover this route. See Chapters 3 and 8.

C.1A · `__version__`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.__version__`. Reflected implementation `builtins.str`. The same object is available through the curated beta route `komorebi_math.beta.__version__` and the curated stable routes `komorebi_math.stable.__version__`, `komorebi_math.v1.__version__`, and `komorebi_math.api.v1.__version__`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. Stronger facade contracts apply only on the listed facade routes. Manifest kind `data`. Profile `C1-META`.

Manifest signature (s).

```
null
```

Definition. The package version string recorded by the reference baseline: `1.0.0b4`.

Contract (D, C, H, R, X). Import returns the installed package version. Release and qualification helpers compare it with committed contracts. The string alone does not prove that a wheel, platform, or documentation build passed. There is no call-time exception.

Guarantees (E, P, B, K). The binding is deterministic package metadata and makes no independent mathematical claim. Import and lookup are constant-time in ordinary Python terms and use no numerical backend, accelerator, network, or filesystem operation. Any later file access or validation belongs to the consuming API.

Evidence and links (V, L). The committed manifest, runtime reflection audit, and release-contract and qualification tests cover this route. See Chapters 4, 7, 43, 52, and 72 as applicable.

C.1A · check_beta_api

Identity (q, a, t, k). Public research-manifest route `komorebi_math.check_beta_api`. Reflected implementation `komorebi_math.compatibility.check_beta_api`. The same object is available through the curated beta route `komorebi_math.beta.check_beta_api`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. Stronger facade contracts apply only on the listed facade routes. Manifest kind `function`. Profile `C1-API-CHECK`.

Manifest signature (s).

```
(path: 'str | Path | None' = None) -> 'ApiCompatibilityReport'
```

Definition. Load the committed beta manifest, create the current beta manifest, and compare them with strict signatures enabled.

Contract (D, C, H, R, X). The function loads the committed or supplied beta manifest, creates the current beta manifest, and compares them with strict signatures enabled. Loading and reflection errors raise `ValidationError` or import errors. Structural incompatibility is represented by a false certificate rather than raised automatically.

Guarantees (E, P, B, K). The report is deterministic relative to the reference file and current imported namespace objects. Cost is the sum of manifest loading, reflection, sorting, and comparison. The check is backend-neutral and establishes structural compatibility only.

Evidence and links (V, L). The committed manifest, runtime reflection audit, and `test_block10_compatibility.py`, `test_block11_golden_profiling_beta.py`, and the CLI compatibility tests cover this route. See Chapters 7, 72, and 73.

C.1A · check_public_api

Identity (q, a, t, k). Public research-manifest route `komorebi_math.check_public_api`. Reflected implementation `komorebi_math.compatibility.check_public_api`. No stronger curated facade exports this name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. Stronger facade contracts apply only on the listed facade routes. Manifest kind `function`. Profile `C1-API-CHECK`.

Manifest signature (s).

```
(path: 'str | Path | None' = None, *, strict_signatures: 'bool' = False) -> 'ApiCompatibilityReport'
```

Definition. Load a reference manifest and compare it with the currently installed exports for the same namespace set.

Contract (D, C, H, R, X). The function loads the chosen reference manifest and compares it with current runtime exports. `strict_signatures` is forwarded unchanged. Loading, reflection, and comparison failures propagate. Otherwise the result may be compatible or incompatible without raising until its certificate is required.

Guarantees (E, P, B, K). The report is deterministic relative to the reference file and current imported namespace objects. Cost is the sum of manifest loading, reflection, sorting, and comparison. The check is backend-neutral and establishes structural compatibility only.

Evidence and links (V, L). The committed manifest, runtime reflection audit, and `test_block10_compatibility.py`, `test_block11_golden_profiling_beta.py`, and the CLI compatibility tests cover this route. See Chapters 7, 72, and 73.

C.1A · compare_public_api

Identity (q, a, t, k). Public research-manifest route `komorebi_math.compare_public_api`. Reflected implementation `komorebi_math.compatibility.compare_public_api`. No stronger curated facade exports this name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. Stronger facade contracts apply only on the listed facade routes. Manifest kind `function`. Profile `C1-API-COMPARE`.

Manifest signature (s).

```
(reference: 'PublicApiManifest', current: 'PublicApiManifest | None' = None, *, strict_signatures: 'bool' = False)
   ↪  -> 'ApiCompatibilityReport'
```

Definition. Compare two manifests under the additive compatibility policy and return an exact structural report.

Contract (D, C, H, R, X). Both operands must be `PublicApiManifest` records covering identical namespaces. When `current` is omitted, the current exports for the reference namespaces are reflected. Additions are compatible. Removals and broad kind changes fail. Signature changes are always reported and fail only when `strict_signatures=True`. The returned exact certificate uses theorem ID `KM-API-010` and explicitly leaves semantic equivalence unchecked.

Guarantees (E, P, B, K). The certificate is exact for the reflected structural policy. Set and map construction are linear in manifest size, sorting differences adds `O(n log n)`, and memory is linear. No mathematical behavior, undocumented implementation detail, or platform qualification is inferred.

Evidence and links (V, L). The committed manifest, runtime reflection audit, and `test_block10_compatibility.py`, `test_block11_golden_profiling_beta.py`, and the CLI compatibility tests cover this route. See Chapters 7, 72, and 75.

C.1A · configure

Identity (q, a, t, k). Public research-manifest route `komorebi_math.configure`. Reflected implementation `komorebi_math.config.configure`. The same object is available through the curated stable routes `komorebi_math.stable.configure`, `komorebi_math.v1.configure`, and `komorebi_math.api.v1.configure`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. Stronger facade contracts apply only on the listed facade routes. Manifest kind `function`. Profile `C1-SETTINGS-SET`.

Manifest signature (s).

```
(*, backend: 'str | None' = None, tolerance: 'float | None' = None, validate_on_construction: 'bool | None' = None)
   ↪  -> 'Settings'
```

Definition. Replace selected fields in the active context's settings and return the new immutable `Settings` record.

Contract (D, C, H, R, X). Each omitted keyword preserves the current field. A supplied tolerance is converted with `float`, and the validation flag is converted with `bool`. The backend value is stored as supplied. The new record becomes active in the current context and is returned. Conversion errors propagate. Backend availability and tolerance range are not checked here.

Guarantees (E, P, B, K). The settings layer is standard-library only and context-local through `ContextVar`. Reads and replacements are constant-size for the three current fields. The backend label is descriptive here, so no automatic backend selection or fallback occurs. Determinism depends only on the current context and supplied values.

Evidence and links (V, L). The committed manifest, runtime reflection audit, and `test_certificates_registry.py` cover this route. See Chapters 3 and 8.

C.1A · create_beta_api_manifest

Identity (q, a, t, k). Public research-manifest route `komorebi_math.create_beta_api_manifest`. Reflected implementation `komorebi_math.compatibility.create_beta_api_manifest`. The same object is available through the curated beta route `komorebi_math.beta.create_beta_api_manifest`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. Stronger facade contracts apply only on the listed facade routes. Manifest kind `function`. Profile `C1-API-CREATE`.

Manifest signature (s).

```
(*, package_version: 'str | None' = None) -> 'PublicApiManifest'
```

Definition. Build the curated `komorebi_math.beta` manifest and label it with the strict beta stability policy.

Contract (D, C, H, R, X). The function delegates reflection to `create_public_api_manifest` for `komorebi_math.beta`, then rebuilds the immutable manifest with `1.0-beta-additive-strict-signatures`. An optional package version overrides only the recorded version string. Reflection and validation failures propagate.

Guarantees (E, P, B, K). The result is deterministic for the same imported namespace objects and supported Python reflection behavior. Runtime is linear in exported names plus `O(n log n)` sorting. Memory is linear in the manifest. Reflection checks structure only. No optional numerical backend is used, though importing selected namespaces may execute their normal import code.

Evidence and links (V, L). The committed manifest, runtime reflection audit, and `test_block10_compatibility.py`, `test_block11_golden_profiling_beta.py`, and the CLI compatibility tests cover this route. See Chapters 7, 72, and 75.

C.1A · `create_public_api_manifest`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.create_public_api_manifest`. Reflected implementation `komorebi_math.compatibility.create_public_api_manifest`. No stronger curated facade exports this name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. Stronger facade contracts apply only on the listed facade routes. Manifest kind `function`. Profile `C1-API-CREATE`.

Manifest signature (s).

```
(namespaces: 'Sequence[str]' = ('komorebi_math', 'komorebi_math.nonlinear', 'komorebi_math.export',
    ↪  'komorebi_math.signals', 'komorebi_math.generating', 'komorebi_math.coordinates'), *, package_version: 'str
    ↪  | None' = None) -> 'PublicApiManifest'
```

Definition. Inspect selected namespaces at runtime and build a deterministically ordered public API manifest.

Contract (D, C, H, R, X). Namespaces are stripped, required to be nonempty and unique, imported in order, and required to define duplicate-free `__all__` names that resolve to attributes. Each export is classified as class, function, module, or data and receives reflected signature text where available. Path objects are normalized to `pathlib.Path`, generic aliases use a cross-version variadic signature, and final records are sorted. Invalid namespace declarations raise `ValidationError`. Import failures propagate.

Guarantees (E, P, B, K). The result is deterministic for the same imported namespace objects and supported Python reflection behavior. Runtime is linear in exported names plus `O(n log n)` sorting. Memory is linear in the manifest. Reflection checks structure only. No optional numerical backend is used, though importing selected namespaces may execute their normal import code.

Evidence and links (V, L). The committed manifest, runtime reflection audit, and `test_block10_compatibility.py`, `test_block11_golden_profiling_beta.py`, and the CLI compatibility tests cover this route. See Chapters 7, 72, and 75.

C.1A · `deprecated`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.deprecated`. Reflected implementation `komorebi_math.compatibility.deprecated`. No stronger curated facade exports this name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. Stronger facade contracts apply only on the listed facade routes. Manifest kind `function`. Profile `C1-DEPRECATION-DECORATOR`.

Manifest signature (s).

```
(*, since: 'str', replacement: 'str | None' = None, remove_in: 'str | None' = None, reason: 'str | None' = None) ->
    ↪  'Callable[[Callable[P, R]], Callable[P, R]]'
```

Definition. Create a decorator that preserves callable metadata, emits one structured deprecation warning on each call, and attaches a machine-readable deprecation record.

Contract (D, C, H, R, X). `since` is stripped and must be nonempty. The returned decorator accepts a callable, preserves its metadata with `functools.wraps`, attaches `__komorebi_deprecation__`, and returns a wrapper. Each invocation emits the structured warning and then returns or raises exactly as the wrapped callable does. Noncallable targets raise `ValidationError`.

Guarantees (E, P, B, K). The mechanism is standard-library only. Warning construction is linear in message length, and the wrapper adds constant bookkeeping plus the wrapped call's own cost. It does not remove an API, schedule deletion, or alter the wrapped result. Warning visibility depends on Python's active filters.

Evidence and links (V, L). The committed manifest, runtime reflection audit, and `test_block10_compatibility.py`, `test_block11_golden_profiling_beta.py`, and the CLI compatibility tests cover this route. See Chapters 7 and 72.

C.1A · describe

Identity (q, a, t, k). Public research-manifest route `komorebi_math.describe`. Reflected implementation `komorebi_math.provenance.describe`. No stronger curated facade exports this name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. Stronger facade contracts apply only on the listed facade routes. Manifest kind `function`. Profile `C1-PROV-DESCRIBE`.

Manifest signature (s).

```
(obj: 'Any') -> 'ProvenanceRecord | None'
```

Definition. Return a provenance record attached to an object or resolve a qualified-name string and inspect the resolved object's provenance attribute.

Contract (D, C, H, R, X). For an object, the function returns its `__komorebi_provenance__` attribute or `None`. For a string, it first checks the internal registry, then partitions the qualified name, imports the module, traverses attributes, and reads the same attribute. Missing modules, attributes, or unqualified strings return `None`. Unrelated import-time exceptions are not suppressed.

Guarantees (E, P, B, K). Provenance metadata is descriptive and does not prove the cited claim. Record lookup is constant-time for registered names and otherwise includes module import plus attribute traversal proportional to path length. The registry is process-local, backend-neutral, and unauthenticated.

Evidence and links (V, L). The committed manifest, runtime reflection audit, and `test_certificates_registry.py` and provenance-bearing domain tests cover this route. See Chapters 5 and 72.

C.1A · get_settings

Identity (q, a, t, k). Public research-manifest route `komorebi_math.get_settings`. Reflected implementation `komorebi_math.config.get_settings`. The same object is available through the curated stable routes `komorebi_math.stable.get_settings`, `komorebi_math.v1.get_settings`, and `komorebi_math.api.v1.get_settings`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. Stronger facade contracts apply only on the listed facade routes. Manifest kind `function`. Profile `C1-SETTINGS-GET`.

Manifest signature (s).

```
() -> 'Settings'
```

Definition. Return the active `Settings` value from the package's `ContextVar` in the current execution context.

Contract (D, C, H, R, X). The function has no arguments and returns the active immutable settings object for the current context. It performs no backend probe, allocation, or validation and raises no project exception under normal use.

Guarantees (E, P, B, K). The settings layer is standard-library only and context-local through `ContextVar`. Reads and replacements are constant-size for the three current fields. The backend label is descriptive here, so no automatic backend selection or fallback occurs. Determinism depends only on the current context and supplied values.

Evidence and links (V, L). The committed manifest, runtime reflection audit, and `test_certificates_registry.py` cover this route. See Chapters 3 and 8.

C.1A · load_behavioral_golden_fixture

Identity (q, a, t, k). Public research-manifest route `komorebi_math.load_behavioral_golden_fixture`. Reflected implementation `komorebi_math.golden.load_behavioral_golden_fixture`. The same object is available through the curated beta route `komorebi_math.beta.load_behavioral_golden_fixture`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. Stronger facade contracts apply only on the listed facade routes. Manifest kind `function`. Profile `C1-GOLDEN-LOAD`.

Manifest signature (s).

```
(path: 'str | Path | None' = None) -> 'tuple[Path, dict[str, Any], str]'
```

Definition. Load the closed behavioral fixture language, reject malformed or noncanonical JSON structures, and return the source path, validated document, and canonical SHA-256 digest.

Contract (D, C, H, R, X). The default is the committed fixture path. The loader requires valid UTF-8 JSON, rejects duplicate object keys and nonfinite constants, requires exactly `schema`, `schema_version`, and `cases`, enforces schema version 1, and requires a nonempty case array with unique nonempty names and at least `name`, `kind`, and `expected`. It returns a canonical SHA-256 digest over compact sorted JSON. File and validation failures raise `ValidationError`.

Guarantees (E, P, B, K). Canonical parsing and hashing are deterministic for the same fixture bytes after JSON normalization. Runtime and memory are linear in fixture size. SHA-256 identifies content but does not authenticate it. The loader executes no callable named by the fixture and uses no network access.

Evidence and links (V, L). The committed manifest, runtime reflection audit, and `test_block11_golden_profiling_beta.py` cover this route. See Chapters 7 and 72.

C.1A · load_beta_api_manifest

Identity (q, a, t, k). Public research-manifest route `komorebi_math.load_beta_api_manifest`. Reflected implementation `komorebi_math.compatibility.load_beta_api_manifest`. The same object is available through the curated beta route `komorebi_math.beta.load_beta_api_manifest`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. Stronger facade contracts apply only on the listed facade routes. Manifest kind `function`. Profile `C1-API-LOAD`.

Manifest signature (s).

```
(path: 'str | Path | None' = None) -> 'PublicApiManifest'
```

Definition. Load a public API manifest and additionally require the curated beta namespace set and beta stability label.

Contract (D, C, H, R, X). The function loads a public manifest, then requires the exact namespace tuple `('komorebi_math.beta',)` and the exact beta stability label. File, JSON, schema, ordering, namespace, and stability failures raise `ValidationError`.

Guarantees (E, P, B, K). Parsing, validation, and storage are linear in manifest size, with deterministic ordering checks. The implementation is standard-library only and performs no backend probe. It validates integrity of structure but provides no authenticity or cryptographic signature.

Evidence and links (V, L). The committed manifest, runtime reflection audit, and `test_block10_compatibility.py`, `test_block11_golden_profiling_beta.py`, and the CLI compatibility tests cover this route. See Chapters 7, 72, and 75.

C.1A · load_public_api_manifest

Identity (q, a, t, k). Public research-manifest route `komorebi_math.load_public_api_manifest`. Reflected implementation `komorebi_math.compatibility.load_public_api_manifest`. No stronger curated facade exports this name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. Stronger facade contracts apply only on the listed facade routes. Manifest kind `function`. Profile `C1-API-LOAD`.

Manifest signature (s).

```
(path: 'str | Path | None' = None) -> 'PublicApiManifest'
```

Definition. Read, parse, and validate one public API manifest from UTF-8 JSON.

Contract (D, C, H, R, X). The default path is the committed public manifest. An explicit path is converted with `Path`. UTF-8 text must contain a JSON object matching the exact manifest fields and nested symbol schema. File errors, malformed JSON, wrong fields, duplicate or unsorted symbols, namespace mismatches, and unsupported schema values raise `ValidationError`.

Guarantees (E, P, B, K). Parsing, validation, and storage are linear in manifest size, with deterministic ordering checks. The implementation is standard-library only and performs no backend probe. It validates integrity of structure but provides no authenticity or cryptographic signature.

Evidence and links (V, L). The committed manifest, runtime reflection audit, and `test_block10_compatibility.py`, `test_block11_golden_profiling_beta.py`, and the CLI compatibility tests cover this route. See Chapters 7, 72, and 75.

C.1A · run_behavioral_golden_fixture

Identity (q, a, t, k). Public research-manifest route `komorebi_math.run_behavioral_golden_fixture`. Reflected implementation `komorebi_math.golden.run_behavioral_golden_fixture`. The same object is available through the curated beta route `komorebi_math.beta.run_behavioral_golden_fixture`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. Stronger facade contracts apply only on the listed facade routes. Manifest kind `function`. Profile `C1-GOLDEN-RUN`.

Manifest signature (s).

```
(path: 'str | Path | None' = None) -> 'BehavioralGoldenReport'
```

Definition. Execute every fixture case through the fixed built-in runner registry and aggregate the case certificates into `KM-GOLDEN-011`.

Contract (D, C, H, R, X). The function loads the validated fixture, dispatches each `kind` through a closed built-in runner mapping, and rejects unknown kinds with `ValidationError`. It records each observed result, gathers failed names, and returns an exact aggregate certificate with theorem ID `KM-GOLDEN-011`. The committed fixture is representative rather than exhaustive.

Guarantees (E, P, B, K). The aggregate certificate is exact for the closed committed cases, not for the entire library. Total work and memory are the sum of the selected built-in case runners and retained results. Some runners use NumPy, but the fixture cannot import arbitrary code or select an external callable.

Evidence and links (V, L). The committed manifest, runtime reflection audit, and `test_block11_golden_profiling_beta.py` cover this route. See Chapters 7 and 72.

C.1A · save_public_api_manifest

Identity (q, a, t, k). Public research-manifest route `komorebi_math.save_public_api_manifest`. Reflected implementation `komorebi_math.compatibility.save_public_api_manifest`. No stronger curated facade exports this name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. Stronger facade contracts apply only on the listed facade routes. Manifest kind `function`. Profile `C1-API-SAVE`.

Manifest signature (s).

```
(manifest: 'PublicApiManifest', path: 'str | Path', *, overwrite: 'bool' = False) -> 'Path'
```

Definition. Write one manifest as sorted, indented UTF-8 JSON with a final newline.

Contract (D, C, H, R, X). The first argument must be a `PublicApiManifest`. Parent directories are created. Existing destinations are rejected unless `overwrite=True`. The file is written as sorted two-space JSON with UTF-8, finite JSON values, LF newlines, and one final newline. The operation is deterministic but not an atomic-replace protocol. Filesystem errors propagate.

Guarantees (E, P, B, K). Output bytes are deterministic for one manifest and Python JSON semantics. Runtime and memory are linear in serialized manifest size. The routine uses the local filesystem only, performs no backend work, and provides neither atomic replacement nor authentication.

Evidence and links (V, L). The committed manifest, runtime reflection audit, and `test_block10_compatibility.py`, `test_block11_golden_profiling_beta.py`, and the CLI compatibility tests cover this route. See Chapters 7 and 72.

C.1A · `settings_context`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.settings_context`. Reflected implementation `komorebi_math.config.settings_context`. The same object is available through the curated stable routes `komorebi_math.stable.settings_context`, `komorebi_math.v1.settings_context`, and `komorebi_math.api.v1.settings_context`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. Stronger facade contracts apply only on the listed facade routes. Manifest kind `function`. Profile `C1-SETTINGS-CTX`.

Manifest signature (s).

```
(**changes: 'object') -> 'Iterator[Settings]'
```

Definition. A context manager that temporarily replaces selected settings fields and restores the prior context token on exit.

Contract (D, C, H, R, X). Keyword names are passed to `dataclasses.replace` on the current settings record. The resulting record is installed, yielded, and the prior context token is restored in `finally`, including exceptional exits. Unknown fields or invalid replacement calls raise the standard `TypeError`.

Guarantees (E, P, B, K). The settings layer is standard-library only and context-local through `ContextVar`. Reads and replacements are constant-size for the three current fields. The backend label is descriptive here, so no automatic backend selection or fallback occurs. Determinism depends only on the current context and supplied values.

Evidence and links (V, L). The committed manifest, runtime reflection audit, and `test_certificates_registry.py` cover this route. See Chapters 3 and 8.

C.1A · `warn_deprecated`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.warn_deprecated`. Reflected implementation `komorebi_math.compatibility.warn_deprecated`. No stronger curated facade exports this name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. Stronger facade contracts apply only on the listed facade routes. Manifest kind `function`. Profile `C1-DEPRECATION-WARN`.

Manifest signature (s).

```
(name: 'str', *, since: 'str', replacement: 'str | None' = None, remove_in: 'str | None' = None, reason: 'str |
    ↪ None' = None, stacklevel: 'int' = 2) -> 'None'
```

Definition. Validate a deprecation name and version, construct a structured message, and emit `KomorebiDeprecationWarning`.

Contract (D, C, H, R, X). `name` and `since` are stripped and must remain nonempty or `ValidationError` is raised. Optional replacement, removal version, and reason clauses are appended to the message. `warnings.warn` receives the package category and requested stack level. The function returns `None`.

Guarantees (E, P, B, K). The mechanism is standard-library only. Warning construction is linear in message length, and the wrapper adds constant bookkeeping plus the wrapped call's own cost. It does not remove an API, schedule deletion, or alter the wrapped result. Warning visibility depends on Python's active filters.

Evidence and links (V, L). The committed manifest, runtime reflection audit, and `test_block10_compatibility.py`, `test_block11_golden_profiling_beta.py`, and the CLI compatibility tests cover this route. See Chapters 7 and 72.

C.1B Typed expression model, exact evaluation, planning, and portable encoding: 77 entries

This root section documents the complete root-export surface owned by the `komorebi_math.expression` implementation family. It covers the finite typed expression grammar, exact rational normal forms, evidence composition, basis-aware polynomial primitives, conservative certified rewrites, matrix-chain planning, a restricted five-opcode execution plan, and strict canonical expression transport. The scope is deliberately finite. It is not a general computer-algebra system, an arbitrary-code serialization format, or a numerical backend abstraction.

C.1B · `Apply`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.Apply`. Reflected implementation `komorebi_math.expression.model.Apply`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `class`. Profile `C1-EXPR-NODE`.

Manifest signature (`s`).

```
(operator: 'Expression', argument: 'Expression') -> None
```

Definition. An immutable expression node for applying a typed linear-map expression to a value expression in its declared source space.

Contract (`D, C, H, R, X`). Both operands must be expressions. The operator must have a `LinearMapType`, and the argument type must equal the operator source by complete semantic witness. The result type is the operator target.

Guarantees (`E, P, B, K`). Formation checks are exact and local. Accepted numeric data are rational, and accepted endpoints use complete semantic type equality. Construction does not claim denotational equality or execute a matrix operation. Those claims arise only from checking, normalization, or replay. Cost is linear in materialized literal data or child count, otherwise constant.

Evidence and links (`V, L`). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover type formation, basis witnesses, lexical scope, local node validation, cycle rejection, and every resource-budget dimension. See Chapters 50 and 68.

C.1B · `CheckedExpression`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.CheckedExpression`. Reflected implementation `komorebi_math.expression.engine.CheckedExpression`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `class`. Profile `C1-EXPR-ENGINE-RECORD`.

Manifest signature (`s`).

```
(expression_type: 'CoordinateSpace | LinearMapType', stats: 'ExpressionStats', free_symbols: 'tuple[Symbol, ...]',
    ↪ certificate: 'Certificate') -> None
```

Definition. The result of structural type checking, carrying the semantic type, bounded graph statistics, sorted free symbols, and an exact checking certificate.

Contract (`D, C, H, R, X`). The frozen record stores already-computed fields. Its direct constructor does not rerun node replay or budget checks. Use `check_expression` for an authoritative record.

Guarantees (`E, P, B, K`). The record is immutable and backend-neutral, but direct dataclass construction is not an authority boundary unless the contract explicitly replays its fields. Authoritative constructors use exact rational data, SHA-256 content checks where stated, and certificates. No tolerance-based equality or floating fallback is used.

Evidence and links (`V, L`). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover structural checking, free bindings, exact normalization, true and false equality decisions, conservative rewrites, dual-path evaluation, plan replay, and tamper rejection. See Chapters 50 through 52 and Chapter 68.

C.1B · Compose

Identity (q, a, t, k). Public research-manifest route `komorebi_math.Compose`. Reflected implementation `komorebi_math.expression.model.Compose`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `class`. Profile `C1-EXPR-NODE`.

Manifest signature (s).

```
(factors: 'tuple[Expression, ...]') -> None
```

Definition. An immutable composition node whose map factors are stored from outermost to innermost.

Contract (D, C, H, R, X). The factor iterable is frozen as a tuple and must contain at least two expressions. Every factor must have a linear-map type. For adjacent outer and inner factors, the inner target must equal the outer source by complete semantic type. The resulting type runs from the innermost source to the outermost target.

Guarantees (E, P, B, K). Formation checks are exact and local. Accepted numeric data are rational, and accepted endpoints use complete semantic type equality. Construction does not claim denotational equality or execute a matrix operation. Those claims arise only from checking, normalization, or replay. Cost is linear in materialized literal data or child count, otherwise constant.

Evidence and links (V, L). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover type formation, basis witnesses, lexical scope, local node validation, cycle rejection, and every resource-budget dimension. See Chapters 50 and 68.

C.1B · EquivalenceResult

Identity (q, a, t, k). Public research-manifest route `komorebi_math.EquivalenceResult`. Reflected implementation `komorebi_math.expression.engine.EquivalenceResult`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `class`. Profile `C1-EXPR-ENGINE-RECORD`.

Manifest signature (s).

```
(equivalent: 'bool', left: 'NormalForm | None', right: 'NormalForm | None', certificate: 'Certificate') -> None
```

Definition. A finite exact equality decision with optional normal forms for both sides and a certificate for the comparison path.

Contract (D, C, H, R, X). The record stores a boolean decision, optional normal forms, and a certificate. `require_equivalent` first requires the certificate and then raises `ValidationError` when the decision is false.

Guarantees (E, P, B, K). The record is immutable and backend-neutral, but direct dataclass construction is not an authority boundary unless the contract explicitly replays its fields. Authoritative constructors use exact rational data, SHA-256 content checks where stated, and certificates. No tolerance-based equality or floating fallback is used.

Evidence and links (V, L). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover structural checking, free bindings, exact normalization, true and false equality decisions, conservative rewrites, dual-path evaluation, plan replay, and tamper rejection. See Chapters 50 through 52 and Chapter 68.

C.1B · ExactMapLiteral

Identity (q, a, t, k). Public research-manifest route `komorebi_math.ExactMapLiteral`. Reflected implementation `komorebi_math.expression.model.ExactMapLiteral`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `class`. Profile `C1-EXPR-NODE`.

Manifest signature (s).

```
(map_type: 'LinearMapType', matrix: 'Matrix', label: 'str' = '') -> None
```

Definition. An exact rational matrix literal with an explicit `LinearMapType` and a display-only label.

Contract (D, C, H, R, X). `map_type` must be a `LinearMapType`. Rows are consumed as finite iterables of integers or `Fraction` values with a common length. Shape must be `(target.dimension, source.dimension)`. The label is NFC-normalized display metadata and cannot activate polynomial rewrite rules.

Guarantees (E, P, B, K). Formation checks are exact and local. Accepted numeric data are rational, and accepted endpoints use complete semantic type equality. Construction does not claim denotational equality or execute a matrix operation. Those claims arise only from checking, normalization, or replay. Cost is linear in materialized literal data or child count, otherwise constant.

Evidence and links (V, L). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover type formation, basis witnesses, lexical scope, local node validation, cycle rejection, and every resource-budget dimension. See Chapters 50 and 68.

C.1B · `ExactValueLiteral`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.ExactValueLiteral`. Reflected implementation `komorebi_math.expression.model.ExactValueLiteral`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `class`. Profile `C1-EXPR-NODE`.

Manifest signature (s).

```
(space: 'CoordinateSpace', entries: 'Vector') -> None
```

Definition. An exact rational coordinate vector inhabiting one supported coordinate space.

Contract (D, C, H, R, X). The space must be supported. Entries are consumed as a finite iterable, converted only from integers or `Fraction` values, and must have length equal to the space dimension. Shape or scalar violations raise `DimensionError` or `ValidationError`.

Guarantees (E, P, B, K). Formation checks are exact and local. Accepted numeric data are rational, and accepted endpoints use complete semantic type equality. Construction does not claim denotational equality or execute a matrix operation. Those claims arise only from checking, normalization, or replay. Cost is linear in materialized literal data or child count, otherwise constant.

Evidence and links (V, L). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover type formation, basis witnesses, lexical scope, local node validation, cycle rejection, and every resource-budget dimension. See Chapters 50 and 68.

C.1B · `Expression`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.Expression`. Reflected implementation `komorebi_math.expression.model.Expression`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `class`. Profile `C1-EXPR-NODE`.

Manifest signature (s).

```
()
```

Definition. The abstract base class for the immutable typed expression grammar.

Contract (D, C, H, R, X). This base class defines the `expression_type` and `children` protocol. It has no public mutable state and is not itself a denotable concrete term. Calling the base `expression_type` property raises `NotImplementedError`.

Guarantees (E, P, B, K). Formation checks are exact and local. Accepted numeric data are rational, and accepted endpoints use complete semantic type equality. Construction does not claim denotational equality or execute a matrix operation. Those claims arise only from checking, normalization, or replay. Cost is linear in materialized literal data or child count, otherwise constant.

Evidence and links (V, L). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover type formation, basis witnesses, lexical scope, local node validation, cycle rejection, and every resource-budget dimension. See Chapters 50 and 68.

C.1B · ExpressionBinding

Identity (q, a, t, k). Public research-manifest route `komorebi_math.ExpressionBinding`. Reflected implementation `komorebi_math.expression.certification.ExpressionBinding`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `class`. Profile `C1-EXPR-CERT-RECORD`.

Manifest signature (s).

```
(value: 'Any', declared_type: 'Any', certificate: 'Certificate') -> None
```

Definition. A runtime value paired with its declared expression type and the certificate that qualifies the supplied evidence.

Contract (D, C, H, R, X). The declared type may not be `None`, and the certificate must be a `Certificate`. The direct constructor does not validate the shape or rationality of `value`. Consumers recheck it. Use `exact_binding` when exact built-in validation is required.

Guarantees (E, P, B, K). Evidence status is never promoted above the weakest child claim. Composition is deterministic and linear in child certificates, residual fields, assumptions, and diagnostics. Certificates remain finite records. They do not authenticate the caller, prove an unchecked assumption, or replace independent theorem verification.

Evidence and links (V, L). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover binding-shape replay, exactness weakening, unchecked-assumption propagation, normal-form evidence, and rewrite certificates. See Chapters 51 and 52.

C.1B · ExpressionBudget

Identity (q, a, t, k). Public research-manifest route `komorebi_math.ExpressionBudget`. Reflected implementation `komorebi_math.expression.model.ExpressionBudget`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `class`. Profile `C1-EXPR-BUDGET`.

Manifest signature (s).

```
(max_nodes: 'int' = 10000, max_depth: 'int' = 1000, max_steps: 'int' = 50000, max_degree: 'int' = 512, max_entries:
    ↪  'int' = 1000000, max_coefficient_bits: 'int' = 1000000, max_serialized_bytes: 'int' = 64000000,
    ↪  max_symbol_length: 'int' = 1024) -> None
```

Definition. A fixed collection of pre-allocation limits for expression graph size, depth, work, degree, exact entries, coefficient bits, encoded size, and symbol length.

Contract (D, C, H, R, X). All eight limits must be positive integers other than `bool`. `preflight` performs iterative cycle detection, counts unique DAG nodes and edges, computes depth, validates child types, tracks polynomial degree, exact entries, coefficient bit size, structural encoded size, and symbol length, and raises `ResourceLimitError` when any bound is exceeded. `require` is an alias of `preflight`.

Guarantees (E, P, B, K). Preflight is deterministic for the in-memory DAG. Traversal is linear in unique nodes plus edges, with additional work linear in exact entries and basis witnesses. Memory is linear in visited nodes and distinct spaces. The limits are denial-of-service guards, not a proof that later rational matrix arithmetic will fit every external resource constraint.

Evidence and links (V, L). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover type formation, basis witnesses, lexical scope, local node validation, cycle rejection, and every resource-budget dimension. See Chapters 50 and 68.

C.1B · ExpressionPlan

Identity (q, a, t, k). Public research-manifest route `komorebi_math.ExpressionPlan`. Reflected implementation `komorebi_math.expression.engine.ExpressionPlan`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `class`. Profile `C1-EXPR-ENGINE-RECORD`.

Manifest signature (s).

```
(expression_digest: 'str', expression_type: 'CoordinateSpace | LinearMapType', instructions:
    ↪ 'tuple[PlanInstruction, ...]', output_index: 'int', scalar_operation_cost: 'int', plan_digest: 'str',
    ↪ certificate: 'Certificate') -> None
```

Definition. A self-contained, content-digested, topologically ordered exact execution plan for one typed expression.

Contract (D, C, H, R, X). The expression digest must be 64 hexadecimal characters. Instructions must be a nonempty tuple of `PlanInstruction` records whose dependencies point only backward. Each opcode's arity, payload, type, and scalar-operation cost is re-derived. Aggregate cost must equal the instruction sum, output index must be valid, and the canonical plan digest must match. Tampering raises `ValidationError`.

Guarantees (E, P, B, K). The record is immutable and backend-neutral, but direct dataclass construction is not an authority boundary unless the contract explicitly replays its fields. Authoritative constructors use exact rational data, SHA-256 content checks where stated, and certificates. No tolerance-based equality or floating fallback is used.

Evidence and links (V, L). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover structural checking, free bindings, exact normalization, true and false equality decisions, conservative rewrites, dual-path evaluation, plan replay, and tamper rejection. See Chapters 50 through 52 and Chapter 68.

C.1B · ExpressionResult

Identity (q, a, t, k). Public research-manifest route `komorebi_math.ExpressionResult`. Reflected implementation `komorebi_math.expression.engine.ExpressionResult`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `class`. Profile `C1-EXPR-ENGINE-RECORD`.

Manifest signature (s).

```
(normal_form: 'NormalForm', direct_normal_form: 'NormalForm', plan: 'ExpressionPlan', certificate: 'Certificate')
    ↪ -> None
```

Definition. The dual-path evaluation record containing planned and direct normal forms, the replayed plan, and their combined certificate.

Contract (D, C, H, R, X). `.value` exposes the planned normal-form data. `.scalar` is available only when the result type is `RationalSpace` and otherwise raises `DimensionError`. `.require()` requires the aggregate certificate before returning the exact data. The direct dataclass constructor does not compare the two paths.

Guarantees (E, P, B, K). The record is immutable and backend-neutral, but direct dataclass construction is not an authority boundary unless the contract explicitly replays its fields. Authoritative constructors use exact rational data, SHA-256 content checks where stated, and certificates. No tolerance-based equality or floating fallback is used.

Evidence and links (V, L). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover structural checking, free bindings, exact normalization, true and false equality decisions, conservative rewrites, dual-path evaluation, plan replay, and tamper rejection. See Chapters 50 through 52 and Chapter 68.

C.1B · `ExpressionStats`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.ExpressionStats`. Reflected implementation `komorebi_math.expression.model.ExpressionStats`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `class`. Profile `C1-EXPR-BUDGET`.

Manifest signature (s).

```
(nodes: 'int', depth: 'int', steps: 'int', exact_entries: 'int', maximum_degree: 'int', maximum_coefficient_bits:
    ↪  'int', serialized_bytes: 'int') -> None
```

Definition. The finite structural totals returned by expression-budget preflight.

Contract (D, C, H, R, X). The frozen record stores seven integer totals. Its direct constructor is a reporting container and does not recompute a graph. Authoritative values are returned by `ExpressionBudget.preflight`.

Guarantees (E, P, B, K). Preflight is deterministic for the in-memory DAG. Traversal is linear in unique nodes plus edges, with additional work linear in exact entries and basis witnesses. Memory is linear in visited nodes and distinct spaces. The limits are denial-of-service guards, not a proof that later rational matrix arithmetic will fit every external resource constraint.

Evidence and links (V, L). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover type formation, basis witnesses, lexical scope, local node validation, cycle rejection, and every resource-budget dimension. See Chapters 50 and 68.

C.1B · `IdentityMap`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.IdentityMap`. Reflected implementation `komorebi_math.expression.model.IdentityMap`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `class`. Profile `C1-EXPR-NODE`.

Manifest signature (s).

```
(space: 'CoordinateSpace') -> None
```

Definition. The typed identity map on one exact coordinate space.

Contract (D, C, H, R, X). The space must be supported. The node's type is `LinearMapType(space, space)`. Matrix data are not stored in the node and are generated during exact normalization.

Guarantees (E, P, B, K). Formation checks are exact and local. Accepted numeric data are rational, and accepted endpoints use complete semantic type equality. Construction does not claim denotational equality or execute a matrix operation. Those claims arise only from checking, normalization, or replay. Cost is linear in materialized literal data or child count, otherwise constant.

Evidence and links (V, L). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover type formation, basis witnesses, lexical scope, local node validation, cycle rejection, and every resource-budget dimension. See Chapters 50 and 68.

C.1B · Let

Identity (q, a, t, k). Public research-manifest route `komorebi_math.Let`. Reflected implementation `komorebi_math.expression.model.Let`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `class`. Profile `C1-EXPR-NODE`.

Manifest signature (s).

```
(symbol: 'Symbol', value: 'Expression', body: 'Expression') -> None
```

Definition. A lexical binding node that associates one identity-distinct `Symbol` with a value inside a typed body expression.

Contract (D, C, H, R, X). The binder must be an exact built-in `Symbol`. The value and body must be expressions, and the value type must equal the binder's declared type. The result type is the body type. Lexical resolution later uses binder identity, not the displayed name.

Guarantees (E, P, B, K). Formation checks are exact and local. Accepted numeric data are rational, and accepted endpoints use complete semantic type equality. Construction does not claim denotational equality or execute a matrix operation. Those claims arise only from checking, normalization, or replay. Cost is linear in materialized literal data or child count, otherwise constant.

Evidence and links (V, L). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover type formation, basis witnesses, lexical scope, local node validation, cycle rejection, and every resource-budget dimension. See Chapters 50 and 68.

C.1B · LinearMapType

Identity (q, a, t, k). Public research-manifest route `komorebi_math.LinearMapType`. Reflected implementation `komorebi_math.expression.model.LinearMapType`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `class`. Profile `C1-EXPR-TYPE`.

Manifest signature (s).

```
(source: 'CoordinateSpace', target: 'CoordinateSpace') -> None
```

Definition. The exact hom-space type from one supported coordinate space to another.

Contract (D, C, H, R, X). Both endpoints must be built-in supported coordinate spaces. The semantic key includes the complete source and target witnesses. Matching dimensions or labels are insufficient. Invalid endpoints raise `TypeError`.

Guarantees (E, P, B, K). The record is immutable and backend-neutral. Semantic equality uses exact witnesses rather than display labels or approximate dimensions. Construction cost is linear in the supplied basis witness for polynomial spaces and constant for rational and map types. No floating arithmetic, user code execution, or GPU path is used.

Evidence and links (V, L). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover type formation, basis witnesses, lexical scope, local node validation, cycle rejection, and every resource-budget dimension. See Chapters 50 and 68.

C.1B · MatrixChainPlan

Identity (q, a, t, k). Public research-manifest route `komorebi_math.MatrixChainPlan`. Reflected implementation `komorebi_math.expression.planning.MatrixChainPlan`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `class`. Profile `C1-EXPR-CHAIN-RECORD`.

Manifest signature (s).

```
(dimensions: 'tuple[int, ...]', tree: 'MatrixChainTree', scalar_operation_cost: 'int', left_associative_cost:
    ↪  'int', right_associative_cost: 'int', certificate: 'Certificate') -> None
```

Definition. The replay-validated optimal binary parenthesization record for one dense matrix-dimension chain.

Contract (D, C, H, R, X). Dimensions are normalized to at least two positive integers. The dynamic program is rerun, and the supplied tree, optimal cost, left cost, and right cost must match exactly. The certificate must be valid, unconditionally exact, assumption-complete, and use theorem ID `EXPR.PLAN.MATRIX_CHAIN_DP`.

Guarantees (E, P, B, K). The optimality claim is exact only for order-preserving binary parenthesizations under classical dense scalar-multiplication count. Planning uses `O(n^3)` time and `O(n^2)` table space for `n` matrices. It does not model additions, sparsity, rank, memory traffic, parallelism, GPU launches, Strassen-like algorithms, rounding error, or wall-clock time.

Evidence and links (V, L). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover matrix-chain optimum, tie breaking, exhaustive small-chain comparison, shape validation, exact multiplication, and plan tamper rejection. See Chapters 52 and 68.

C.1B · MatrixChainTree

Identity (q, a, t, k). Public research-manifest route `komorebi_math.MatrixChainTree`. Reflected implementation `komorebi_math.expression.planning.MatrixChainTree`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `class`. Profile `C1-EXPR-CHAIN-RECORD`.

Manifest signature (s).

```
(start: 'int', end: 'int', split: 'int | None', rows: 'int', columns: 'int', scalar_operation_cost: 'int') -> None
```

Definition. The root interval and split decision for one node of a matrix-chain parenthesization.

Contract (D, C, H, R, X). Start, end, row count, column count, and cost are validated as nonnegative or positive integers as appropriate. A leaf interval has no split and zero cost. A nonleaf split must lie strictly inside its half-open interval. The direct record stores only one root decision, not the full split table.

Guarantees (E, P, B, K). The optimality claim is exact only for order-preserving binary parenthesizations under classical dense scalar-multiplication count. Planning uses `O(n^3)` time and `O(n^2)` table space for `n` matrices. It does not model additions, sparsity, rank, memory traffic, parallelism, GPU launches, Strassen-like algorithms, rounding error, or wall-clock time.

Evidence and links (V, L). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover matrix-chain optimum, tie breaking, exhaustive small-chain comparison, shape validation, exact multiplication, and plan tamper rejection. See Chapters 52 and 68.

C.1B · NormalForm

Identity (q, a, t, k). Public research-manifest route `komorebi_math.NormalForm`. Reflected implementation `komorebi_math.expression.certification.NormalForm`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `class`. Profile `C1-EXPR-CERT-RECORD`.

Manifest signature (s).

```
(expression_type: 'Any', data: 'Any', digest: 'str', certificate: 'Certificate') -> None
```

Definition. Canonical exact vector or matrix data paired with its full semantic type, SHA-256 digest, and derivation certificate.

Contract (D, C, H, R, X). The digest must be a nonempty string and the evidence must be a `Certificate`. The direct constructor does not recompute the digest or revalidate data. Normalization and plan execution create authoritative records through internal checked constructors.

Guarantees (E, P, B, K). Evidence status is never promoted above the weakest child claim. Composition is deterministic and linear in child certificates, residual fields, assumptions, and diagnostics. Certificates remain finite records. They do not authenticate the caller, prove an unchecked assumption, or replace independent theorem verification.

Evidence and links (V, L). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover binding-shape replay, exactness weakening, unchecked-assumption propagation, normal-form evidence, and rewrite certificates. See Chapters 51 and 52.

C.1B · PlanInstruction

Identity (q, a, t, k). Public research-manifest route `komorebi_math.PlanInstruction`. Reflected implementation `komorebi_math.expression.engine.PlanInstruction`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `class`. Profile `C1-EXPR-ENGINE-RECORD`.

Manifest signature (s).

```
(opcode: 'str', inputs: 'tuple[int, ...]', expression_type: 'CoordinateSpace | LinearMapType', payload: 'Any',
    ↪ scalar_operation_cost: 'int', certificate: 'Certificate') -> None
```

Definition. One typed instruction in the restricted exact execution language used by `ExpressionPlan`.

Contract (D, C, H, R, X). Opcode must be one of `constant`, `add`, `scale`, `matrix_chain`, or `apply`. Input indices and cost must be nonnegative integers, the type must be supported, and the evidence must be a `Certificate`. Full payload, dependency, type, and cost consistency is re-derived by `ExpressionPlan`.

Guarantees (E, P, B, K). The record is immutable and backend-neutral, but direct dataclass construction is not an authority boundary unless the contract explicitly replays its fields. Authoritative constructors use exact rational data, SHA-256 content checks where stated, and certificates. No tolerance-based equality or floating fallback is used.

Evidence and links (V, L). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover structural checking, free bindings, exact normalization, true and false equality decisions, conservative rewrites, dual-path evaluation, plan replay, and tamper rejection. See Chapters 50 through 52 and Chapter 68.

C.1B · PolynomialCoordinateSpace

Identity (q, a, t, k). Public research-manifest route `komorebi_math.PolynomialCoordinateSpace`. Reflected implementation `komorebi_math.expression.model.PolynomialCoordinateSpace`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `class`. Profile `C1-EXPR-TYPE`.

Manifest signature (`s`).

```
(variable: 'str', degree: 'int', basis: 'tuple[tuple[Fraction, ...], ...]', label: 'str' = '') -> None
```

Definition. A finite univariate polynomial coordinate space identified by its variable, degree, and complete exact graded-basis witness.

Contract (`D, C, H, R, X`). The variable is NFC-normalized and must be nonempty. Degree must be a nonnegative integer other than `bool`. The basis must contain exactly one finite rational polynomial of every degree from zero through the bound, member `j` must have length `j + 1`, and its leading coefficient must be nonzero. The display label is normalized but excluded from equality. `from_family` snapshots `family.polynomial(i)` exactly, and `truncate` permits only a compatible lower-degree prefix. Invalid witnesses raise `ValidationError`, `DimensionError`, or `TypeError`.

Guarantees (`E, P, B, K`). The record is immutable and backend-neutral. Semantic equality uses exact witnesses rather than display labels or approximate dimensions. Construction cost is linear in the supplied basis witness for polynomial spaces and constant for rational and map types. No floating arithmetic, user code execution, or GPU path is used.

Evidence and links (`V, L`). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover type formation, basis witnesses, lexical scope, local node validation, cycle rejection, and every resource-budget dimension. See Chapters 50 and 68.

C.1B · `PolynomialPrimitive`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.PolynomialPrimitive`. Reflected implementation `komorebi_math.expression.polynomial.PolynomialPrimitive`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `class`. Profile `C1-EXPR-POLY-RECORD`.

Manifest signature (`s`).

```
(kind: 'PrimitiveKind', map_type: 'LinearMapType', parameters: 'tuple[Fraction, ...]' = ()) -> None
```

Definition. A branded, replayable, basis-aware polynomial linear map with an enum kind, exact map type, rational parameters, and theorem identifier.

Contract (`D, C, H, R, X`). `kind` must be `PrimitiveKind`, `map_type` must be `LinearMapType`, and parameters are frozen and converted only from integers or `Fraction`. Kind-specific formation rules validate polynomial and rational endpoints, physical variables, degrees, basis extensions, interval order, density sign, and nonzero difference steps. `.matrix` and `.certificate()` replay rather than trust stored matrix data.

Guarantees (`E, P, B, K`). Every accepted primitive has an exact rational matrix determined by finite basis witnesses and parameters. Replay is deterministic and backend-neutral. Basis inversion and operator conversion use dense exact matrix algebra, typically cubic in coordinate dimension before operation-specific work. The certificate covers the finite polynomial map only, not a general analytic theorem.

Evidence and links (`V, L`). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover the Fibonacci-Euler pipeline, basis spoof resistance, all nine primitive kinds, exact replay, coordinate naturality, the basepoint defect, density-aware FTC, and degree-zero differentiation. See Chapters 50, 51, and 68.

C.1B · `PrimitiveKind`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.PrimitiveKind`. Reflected implementation `komorebi_math.expression.polynomial.PrimitiveKind`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `class`. Profile `C1-EXPR-POLY-RECORD`.

Manifest signature (`s`).

```
(*values)
```

Definition. The nine-member string enumeration of supported replayable polynomial operations.

Contract (`D`, `C`, `H`, `R`, `X`). Construction follows `StrEnum` semantics. Accepted values are coordinate change, derivative, antiderivative, evaluation, interval integration, affine substitution, finite difference, boundary functional, and constant injection. Unknown values raise the standard enum `ValueError`.

Guarantees (`E`, `P`, `B`, `K`). Every accepted primitive has an exact rational matrix determined by finite basis witnesses and parameters. Replay is deterministic and backend-neutral. Basis inversion and operator conversion use dense exact matrix algebra, typically cubic in coordinate dimension before operation-specific work. The certificate covers the finite polynomial map only, not a general analytic theorem.

Evidence and links (`V`, `L`). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover the Fibonacci-Euler pipeline, basis spoof resistance, all nine primitive kinds, exact replay, coordinate naturality, the basepoint defect, density-aware FTC, and degree-zero differentiation. See Chapters 50, 51, and 68.

C.1B · `RATIONAL_SPACE`

Identity (`q`, `a`, `t`, `k`). Public research-manifest route `komorebi_math.RATIONAL_SPACE`. Reflected implementation `komorebi_math.expression.model.RationalSpace`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `data`. Profile `C1-EXPR-TYPE`.

Manifest signature (`s`).

```
null
```

Definition. The shared stateless instance of the one-dimensional exact rational coordinate space.

Contract (`D`, `C`, `H`, `R`, `X`). Import returns the shared `RationalSpace()` value. The class is stateless, so semantic equality and hashing do not depend on this particular object identity. The binding performs no arithmetic and raises no call-time exception.

Guarantees (`E`, `P`, `B`, `K`). The record is immutable and backend-neutral. Semantic equality uses exact witnesses rather than display labels or approximate dimensions. Construction cost is linear in the supplied basis witness for polynomial spaces and constant for rational and map types. No floating arithmetic, user code execution, or GPU path is used.

Evidence and links (`V`, `L`). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover type formation, basis witnesses, lexical scope, local node validation, cycle rejection, and every resource-budget dimension. See Chapters 50 and 68.

C.1B · `RationalSpace`

Identity (`q`, `a`, `t`, `k`). Public research-manifest route `komorebi_math.RationalSpace`. Reflected implementation `komorebi_math.expression.model.RationalSpace`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `class`. Profile `C1-EXPR-TYPE`.

Manifest signature (`s`).

```
() -> None
```

Definition. The stateless one-dimensional coordinate space over the rational numbers.

Contract (`D, C, H, R, X`). Construction takes no arguments and produces a frozen stateless value with dimension one and semantic key `('Q',)`. All instances are semantically equal. The class does not coerce scalars or validate expression data by itself.

Guarantees (`E, P, B, K`). The record is immutable and backend-neutral. Semantic equality uses exact witnesses rather than display labels or approximate dimensions. Construction cost is linear in the supplied basis witness for polynomial spaces and constant for rational and map types. No floating arithmetic, user code execution, or GPU path is used.

Evidence and links (`V, L`). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover type formation, basis witnesses, lexical scope, local node validation, cycle rejection, and every resource-budget dimension. See Chapters 50 and 68.

C.1B · `RewriteStep`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.RewriteStep`. Reflected implementation `komorebi_math.expression.certification.RewriteStep`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `class`. Profile `C1-EXPR-CERT-RECORD`.

Manifest signature (`s`).

```
(rule: 'str', theorem_id: 'str | None', before_digest: 'str', after_digest: 'str', side_conditions: 'tuple[str,
   ↪  ...]', certificate: 'Certificate') -> None
```

Definition. One denotation-checked simplification step with before and after digests, side conditions, theorem ID, and certificate.

Contract (`D, C, H, R, X`). Rule and digest strings must be nonempty. A supplied theorem ID must be nonempty. Side conditions are deduplicated in first-seen order and must be nonempty strings. The certificate type is checked, but the direct constructor does not rerun the before-and-after denotations.

Guarantees (`E, P, B, K`). Evidence status is never promoted above the weakest child claim. Composition is deterministic and linear in child certificates, residual fields, assumptions, and diagnostics. Certificates remain finite records. They do not authenticate the caller, prove an unchecked assumption, or replace independent theorem verification.

Evidence and links (`V, L`). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover binding-shape replay, exactness weakening, unchecked-assumption propagation, normal-form evidence, and rewrite certificates. See Chapters 51 and 52.

C.1B · `Scale`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.Scale`. Reflected implementation `komorebi_math.expression.model.Scale`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `class`. Profile `C1-EXPR-NODE`.

Manifest signature (s).

```
(scalar: 'Fraction', value: 'Expression') -> None
```

Definition. An immutable rational scalar-multiplication node that preserves the type of its child expression.

Contract (D, C, H, R, X). The scalar is converted only from an integer or `Fraction`. Booleans and binary floats are rejected. The child must be an expression. The node preserves the child's exact type.

Guarantees (E, P, B, K). Formation checks are exact and local. Accepted numeric data are rational, and accepted endpoints use complete semantic type equality. Construction does not claim denotational equality or execute a matrix operation. Those claims arise only from checking, normalization, or replay. Cost is linear in materialized literal data or child count, otherwise constant.

Evidence and links (V, L). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover type formation, basis witnesses, lexical scope, local node validation, cycle rejection, and every resource-budget dimension. See Chapters 50 and 68.

C.1B · `SimplificationResult`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.SimplificationResult`. Reflected implementation `komorebi_math.expression.engine.SimplificationResult`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `class`. Profile `C1-EXPR-ENGINE-RECORD`.

Manifest signature (s).

```
(expression: 'Expression', before: 'NormalForm', after: 'NormalForm', steps: 'tuple[RewriteStep, ...]',
   ↪  certificate: 'Certificate') -> None
```

Definition. A conservative rewrite result containing the rewritten expression, exact normal forms before and after, the step ledger, and an aggregate certificate.

Contract (D, C, H, R, X). The record stores the rewritten expression, exact normal forms before and after, the immutable rewrite-step tuple, and the aggregate certificate. Its direct constructor is not a replay boundary. `simplify_expression` performs the authoritative checks.

Guarantees (E, P, B, K). The record is immutable and backend-neutral, but direct dataclass construction is not an authority boundary unless the contract explicitly replays its fields. Authoritative constructors use exact rational data, SHA-256 content checks where stated, and certificates. No tolerance-based equality or floating fallback is used.

Evidence and links (V, L). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover structural checking, free bindings, exact normalization, true and false equality decisions, conservative rewrites, dual-path evaluation, plan replay, and tamper rejection. See Chapters 50 through 52 and Chapter 68.

C.1B · `Sum`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.Sum`. Reflected implementation `komorebi_math.expression.model.Sum`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `class`. Profile `C1-EXPR-NODE`.

Manifest signature (s).

```
(terms: 'tuple[Expression, ...]') -> None
```

Definition. An immutable finite sum node whose terms all have the same exact expression type.

Contract (D, C, H, R, X). The terms are frozen as a tuple and must contain at least two expressions. Every term must have exactly the same semantic expression type. Type disagreement or non-expression children raise a validation or type error.

Guarantees (E, P, B, K). Formation checks are exact and local. Accepted numeric data are rational, and accepted endpoints use complete semantic type equality. Construction does not claim denotational equality or execute a matrix operation. Those claims arise only from checking, normalization, or replay. Cost is linear in materialized literal data or child count, otherwise constant.

Evidence and links (V, L). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover type formation, basis witnesses, lexical scope, local node validation, cycle rejection, and every resource-budget dimension. See Chapters 50 and 68.

C.1B · `Symbol`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.Symbol`. Reflected implementation `komorebi_math.expression.model.Symbol`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `class`. Profile `C1-EXPR-NODE`.

Manifest signature (s).

```
(name: 'str', declared_type: 'ExpressionType') -> None
```

Definition. An identity-distinct lexical or free variable with a normalized display name and an explicit declared expression type.

Contract (D, C, H, R, X). The name is NFC-normalized, stripped, and required to be nonempty. The declared type must be a built-in rational space, polynomial space, or linear-map type. Symbols deliberately use object identity rather than value equality, so same-name instances remain distinct binders.

Guarantees (E, P, B, K). Formation checks are exact and local. Accepted numeric data are rational, and accepted endpoints use complete semantic type equality. Construction does not claim denotational equality or execute a matrix operation. Those claims arise only from checking, normalization, or replay. Cost is linear in materialized literal data or child count, otherwise constant.

Evidence and links (V, L). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover type formation, basis witnesses, lexical scope, local node validation, cycle rejection, and every resource-budget dimension. See Chapters 50 and 68.

C.1B · `ZeroMap`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.ZeroMap`. Reflected implementation `komorebi_math.expression.model.ZeroMap`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `class`. Profile `C1-EXPR-NODE`.

Manifest signature (s).

```
(map_type: 'LinearMapType') -> None
```

Definition. The exact zero linear map of one declared source and target type.

Contract (D, C, H, R, X). The supplied type must be a `LinearMapType`. The node stores no matrix data. Exact zero entries of the declared shape are produced during normalization.

Guarantees (E, P, B, K). Formation checks are exact and local. Accepted numeric data are rational, and accepted endpoints use complete semantic type equality. Construction does not claim denotational equality or execute a matrix operation. Those claims arise only from checking, normalization, or replay. Cost is linear in materialized literal data or child count, otherwise constant.

Evidence and links (V, L). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_`

example.py cover type formation, basis witnesses, lexical scope, local node validation, cycle rejection, and every resource-budget dimension. See Chapters 50 and 68.

C.1B · add

Identity (q, a, t, k). Public research-manifest route komorebi_math.add. Reflected implementation komorebi_math.expression.model.add. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by [RESEARCH] policy pre-beta-additive. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind function. Profile C1-EXPR-CONSTRUCTOR.

Manifest signature (s).

```
(*terms: 'Expression') -> 'Expression'
```

Definition. Construct a typed finite sum, returning its sole term unchanged when exactly one term is supplied.

Contract (D, C, H, R, X). At least one term is required. A sole term is type-checked and returned unchanged. Two or more terms produce Sum, which requires identical semantic types.

Guarantees (E, P, B, K). Formation checks are exact and local. Accepted numeric data are rational, and accepted endpoints use complete semantic type equality. Construction does not claim denotational equality or execute a matrix operation. Those claims arise only from checking, normalization, or replay. Cost is linear in materialized literal data or child count, otherwise constant.

Evidence and links (V, L). The committed research manifest, runtime reflection audit, tests/test_block20_expression_calculus.py, verification/block20_math_verifier.py, and examples/block20_typed_expression_worked_example.py cover type formation, basis witnesses, lexical scope, local node validation, cycle rejection, and every resource-budget dimension. See Chapters 50 and 68.

C.1B · affine_substitution

Identity (q, a, t, k). Public research-manifest route komorebi_math.affine_substitution. Reflected implementation komorebi_math.expression.polynomial.affine_substitution. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by [RESEARCH] policy pre-beta-additive. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind function. Profile C1-EXPR-POLY-CONSTRUCTOR.

Manifest signature (s).

```
(source: 'PolynomialCoordinateSpace', scale: 'int | Fraction' = 1, shift: 'int | Fraction' = 0) ->
   ↪ 'PolynomialPrimitive'
```

Definition. Construct the endomorphism that sends a polynomial to p(scale*x + shift) in the same basis-aware coordinate space.

Contract (D, C, H, R, X). Source must be a polynomial coordinate space. Scale and shift are converted to Fraction. The map is an endomorphism of the same coordinate space, including the zero-scale case.

Guarantees (E, P, B, K). Every accepted primitive has an exact rational matrix determined by finite basis witnesses and parameters. Replay is deterministic and backend-neutral. Basis inversion and operator conversion use dense exact matrix algebra, typically cubic in coordinate dimension before operation-specific work. The certificate covers the finite polynomial map only, not a general analytic theorem.

Evidence and links (V, L). The committed research manifest, runtime reflection audit, tests/test_block20_expression_calculus.py, verification/block20_math_verifier.py, and examples/block20_typed_expression_worked_example.py cover the Fibonacci-Euler pipeline, basis spoof resistance, all nine primitive kinds, exact replay, coordinate naturality, the basepoint defect, density-aware FTC, and degree-zero differentiation. See Chapters 50, 51, and 68.

C.1B · antiderivative

Identity (q, a, t, k). Public research-manifest route `komorebi_math.antiderivative`. Reflected implementation `komorebi_math.expression.polynomial.antiderivative`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `function`. Profile `C1-EXPR-POLY-CONSTRUCTOR`.

Manifest signature (s).

```
(source: 'PolynomialCoordinateSpace', target: 'PolynomialCoordinateSpace', basepoint: 'int | Fraction' = 0) ->
    ↪ 'PolynomialPrimitive'
```

Definition. Construct the basepoint-normalized polynomial antiderivative from a source space to an exact one-degree extension.

Contract (D, C, H, R, X). Source and target must use the same variable. Target degree must be exactly one larger and its truncation must equal the full source witness. The basepoint is converted to `Fraction`.

Guarantees (E, P, B, K). Every accepted primitive has an exact rational matrix determined by finite basis witnesses and parameters. Replay is deterministic and backend-neutral. Basis inversion and operator conversion use dense exact matrix algebra, typically cubic in coordinate dimension before operation-specific work. The certificate covers the finite polynomial map only, not a general analytic theorem.

Evidence and links (V, L). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover the Fibonacci-Euler pipeline, basis spoof resistance, all nine primitive kinds, exact replay, coordinate naturality, the basepoint defect, density-aware FTC, and degree-zero differentiation. See Chapters 50, 51, and 68.

C.1B · apply

Identity (q, a, t, k). Public research-manifest route `komorebi_math.apply`. Reflected implementation `komorebi_math.expression.model.apply`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `function`. Profile `C1-EXPR-CONSTRUCTOR`.

Manifest signature (s).

```
(operator: 'Expression', argument: 'Expression') -> 'Apply'
```

Definition. Construct a checked `Apply` node from a linear-map expression and a compatible value expression.

Contract (D, C, H, R, X). The function delegates to `Apply`. It does not normalize either operand. Source-target compatibility is enforced immediately.

Guarantees (E, P, B, K). Formation checks are exact and local. Accepted numeric data are rational, and accepted endpoints use complete semantic type equality. Construction does not claim denotational equality or execute a matrix operation. Those claims arise only from checking, normalization, or replay. Cost is linear in materialized literal data or child count, otherwise constant.

Evidence and links (V, L). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover type formation, basis witnesses, lexical scope, local node validation, cycle rejection, and every resource-budget dimension. See Chapters 50 and 68.

C.1B · boundary_functional

Identity (q, a, t, k). Public research-manifest route `komorebi_math.boundary_functional`. Reflected implementation `komorebi_math.expression.polynomial.boundary_functional`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `function`. Profile `C1-EXPR-POLY-CONSTRUCTOR`.

Manifest signature (s).

```
(source: 'PolynomialCoordinateSpace', lower: 'int | Fraction', upper: 'int | Fraction', density: 'int | Fraction' =
    ↪ 1) -> 'PolynomialPrimitive'
```

Definition. Construct the exact functional `density * (p(upper) - p(lower))` on a polynomial coordinate space.

Contract (D, C, H, R, X). Source must be a polynomial coordinate space. Lower, upper, and density are converted to `Fraction`. Lower must not exceed upper and density must be nonnegative. The target is `RATIONAL_SPACE`.

Guarantees (E, P, B, K). Every accepted primitive has an exact rational matrix determined by finite basis witnesses and parameters. Replay is deterministic and backend-neutral. Basis inversion and operator conversion use dense exact matrix algebra, typically cubic in coordinate dimension before operation-specific work. The certificate covers the finite polynomial map only, not a general analytic theorem.

Evidence and links (V, L). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover the Fibonacci-Euler pipeline, basis spoof resistance, all nine primitive kinds, exact replay, coordinate naturality, the basepoint defect, density-aware FTC, and degree-zero differentiation. See Chapters 50, 51, and 68.

C.1B · `check_expression`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.check_expression`. Reflected implementation `komorebi_math.expression.engine.check_expression`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `function`. Profile `C1-EXPR-ENGINE-FUNCTION`.

Manifest signature (s).

```
(expression: 'Expression', bindings: 'Mapping[str | Symbol, ExpressionBinding] | None' = None, *, budget:
    ↪ 'ExpressionBudget | None' = None) -> 'CheckedExpression'
```

Definition. Replay the exact node invariants, enforce the resource budget, validate supplied bindings, and report the expression type and free symbols.

Contract (D, C, H, R, X). The root must be an `Expression`, and a supplied budget must be exactly `ExpressionBudget`. Built-in node invariants are replayed, cycles and forged children are rejected, and budget preflight runs. Bindings must map strings or identity-specific symbols to `ExpressionBinding` records. Conflicting, ambiguous, ill-shaped, or mistyped bindings fail. Missing free bindings are reported but do not make an open term ill typed.

Guarantees (E, P, B, K). The check is exact for the finite built-in grammar and supplied bindings. Work is bounded by the graph budget and is linear in the traversed graph plus binding lookup overhead. It establishes structural typing and free-symbol accounting, not a denotation for an open term.

Evidence and links (V, L). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover structural checking, free bindings, exact normalization, true and false equality decisions, conservative rewrites, dual-path evaluation, plan replay, and tamper rejection. See Chapters 50 through 52 and Chapter 68.

C.1B · `compile_expression`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.compile_expression`. Reflected implementation `komorebi_math.expression.engine.compile_expression`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `function`. Profile `C1-EXPR-ENGINE-FUNCTION`.

Manifest signature (s).

```
(expression: 'Expression', bindings: 'Mapping[str | Symbol, ExpressionBinding] | None' = None, *, budget:
    ↪ 'ExpressionBudget | None' = None) -> 'ExpressionPlan'
```

Definition. Compile a closed typed expression into a replayable exact instruction plan and refuse the plan unless it matches independent direct normalization.

Contract (D, C, H, R, X). The source is checked and directly normalized. The compiler emits only the five supported opcodes, uses dynamic programming for flattened map chains, computes source and plan digests, creates a certificate, executes the plan, and compares the result with direct normalization. It returns only after exact type and data agreement.

Guarantees (E, P, B, K). The execution language is closed to five opcodes and uses exact rational operations. Compilation and evaluation include independent replay checks and content digests. Matrix-chain planning is cubic in chain length with quadratic tables. Matrix arithmetic and coefficient growth dominate execution. Plan digests are integrity checks, not signatures or authenticated transport.

Evidence and links (V, L). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover structural checking, free bindings, exact normalization, true and false equality decisions, conservative rewrites, dual-path evaluation, plan replay, and tamper rejection. See Chapters 50 through 52 and Chapter 68.

C.1B · compose

Identity (q, a, t, k). Public research-manifest route `komorebi_math.compose`. Reflected implementation `komorebi_math.expression.model.compose`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `function`. Profile `C1-EXPR-CONSTRUCTOR`.

Manifest signature (s).

```
(*maps: 'Expression') -> 'Expression'
```

Definition. Compose maps supplied in mathematical outermost-to-innermost order.

Contract (D, C, H, R, X). At least one map is required. A sole factor is type-checked as a linear map and returned unchanged. Two or more factors produce `Compose` in the supplied outermost-to-innermost order. Endpoint mismatches and non-map factors fail closed.

Guarantees (E, P, B, K). Formation checks are exact and local. Accepted numeric data are rational, and accepted endpoints use complete semantic type equality. Construction does not claim denotational equality or execute a matrix operation. Those claims arise only from checking, normalization, or replay. Cost is linear in materialized literal data or child count, otherwise constant.

Evidence and links (V, L). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover type formation, basis witnesses, lexical scope, local node validation, cycle rejection, and every resource-budget dimension. See Chapters 50 and 68.

C.1B · compose_certificates

Identity (q, a, t, k). Public research-manifest route `komorebi_math.compose_certificates`. Reflected implementation `komorebi_math.expression.certification.compose_certificates`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `function`. Profile `C1-EXPR-CERT-FUNCTION`.

Manifest signature (`s`).

```
(certificates: 'Iterable[Certificate]', *, operation: 'str | None' = None, theorem_id: 'str | None' = None,
    ↪ assumptions_checked: 'Iterable[str]' = (), assumptions_unchecked: 'Iterable[str]' = (), diagnostics:
    ↪ 'Mapping[str, Any] | None' = None) -> 'Certificate'
```

Definition. Combine child certificates without strengthening their evidence status, while retaining residuals, assumptions, tolerances, diagnostics, and validity.

Contract (`D, C, H, R, X`). Every child must be a `Certificate`. Residual keys receive deterministic child prefixes, tolerance becomes the largest finite child tolerance, diagnostics are kept in separate child and local namespaces, and assumptions are deduplicated. An assumption marked unchecked anywhere remains unchecked. Validity is the conjunction of child validity, and exactness is the weakest child status, weakened to conditional when unchecked assumptions remain.

Guarantees (`E, P, B, K`). Evidence status is never promoted above the weakest child claim. Composition is deterministic and linear in child certificates, residual fields, assumptions, and diagnostics. Certificates remain finite records. They do not authenticate the caller, prove an unchecked assumption, or replace independent theorem verification.

Evidence and links (`V, L`). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover binding-shape replay, exactness weakening, unchecked-assumption propagation, normal-form evidence, and rewrite certificates. See Chapters 51 and 52.

C.1B · `constant_injection`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.constant_injection`. Reflected implementation `komorebi_math.expression.polynomial.constant_injection`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `function`. Profile `C1-EXPR-POLY-CONSTRUCTOR`.

Manifest signature (`s`).

```
(target: 'PolynomialCoordinateSpace') -> 'PolynomialPrimitive'
```

Definition. Construct the exact linear map that embeds a rational scalar as the constant polynomial in a target coordinate space.

Contract (`D, C, H, R, X`). Target must be a polynomial coordinate space. The primitive maps `RATIONAL_SPACE` into the target and takes no rational parameters.

Guarantees (`E, P, B, K`). Every accepted primitive has an exact rational matrix determined by finite basis witnesses and parameters. Replay is deterministic and backend-neutral. Basis inversion and operator conversion use dense exact matrix algebra, typically cubic in coordinate dimension before operation-specific work. The certificate covers the finite polynomial map only, not a general analytic theorem.

Evidence and links (`V, L`). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover the Fibonacci-Euler pipeline, basis spoof resistance, all nine primitive kinds, exact replay, coordinate naturality, the basepoint defect, density-aware FTC, and degree-zero differentiation. See Chapters 50, 51, and 68.

C.1B · `coordinate_change`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinate_change`. Reflected implementation `komorebi_math.expression.polynomial.coordinate_change`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `function`. Profile `C1-EXPR-POLY-CONSTRUCTOR`.

Manifest signature (`s`).

```
(source: 'PolynomialCoordinateSpace', target: 'PolynomialCoordinateSpace') -> 'PolynomialPrimitive'
```

Definition. Construct the exact basis-change map between equal-degree polynomial spaces over the same physical variable.

Contract (`D, C, H, R, X`). Source and target must be polynomial spaces with the same physical variable and equal degree. Their complete basis witnesses may differ. The returned primitive has no rational parameters.

Guarantees (`E, P, B, K`). Every accepted primitive has an exact rational matrix determined by finite basis witnesses and parameters. Replay is deterministic and backend-neutral. Basis inversion and operator conversion use dense exact matrix algebra, typically cubic in coordinate dimension before operation-specific work. The certificate covers the finite polynomial map only, not a general analytic theorem.

Evidence and links (`V, L`). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover the Fibonacci-Euler pipeline, basis spoof resistance, all nine primitive kinds, exact replay, coordinate naturality, the basepoint defect, density-aware FTC, and degree-zero differentiation. See Chapters 50, 51, and 68.

C.1B · `derivative`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.derivative`. Reflected implementation `komorebi_math.expression.polynomial.derivative`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `function`. Profile `C1-EXPR-POLY-CONSTRUCTOR`.

Manifest signature (`s`).

```
(source: 'PolynomialCoordinateSpace') -> 'PolynomialPrimitive'
```

Definition. Construct the tight polynomial derivative, using the compatible degree-minus-one target or a zero endomorphism at degree zero.

Contract (`D, C, H, R, X`). Source must be a polynomial coordinate space. For positive degree the target is the exact compatible truncation through degree minus one. At degree zero the result is the zero endomorphism of the same one-dimensional space.

Guarantees (`E, P, B, K`). Every accepted primitive has an exact rational matrix determined by finite basis witnesses and parameters. Replay is deterministic and backend-neutral. Basis inversion and operator conversion use dense exact matrix algebra, typically cubic in coordinate dimension before operation-specific work. The certificate covers the finite polynomial map only, not a general analytic theorem.

Evidence and links (`V, L`). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover the Fibonacci-Euler pipeline, basis spoof resistance, all nine primitive kinds, exact replay, coordinate naturality, the basepoint defect, density-aware FTC, and degree-zero differentiation. See Chapters 50, 51, and 68.

C.1B · `equivalent`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.equivalent`. Reflected implementation `komorebi_math.expression.engine.equivalent`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `function`. Profile `C1-EXPR-ENGINE-FUNCTION`.

Manifest signature (s).

```
(left: 'Expression', right: 'Expression', *, left_bindings: 'Mapping[str | Symbol, ExpressionBinding] | None' =
    ↪ None, right_bindings: 'Mapping[str | Symbol, ExpressionBinding] | None' = None, budget: 'ExpressionBudget |
    ↪ None' = None) -> 'EquivalenceResult'
```

Definition. Decide equality of two closed typed expressions by exact semantic type and entrywise normal-form comparison.

Contract (D, C, H, R, X). Both operands must be expressions. Each is checked independently. Different semantic types return an exact false decision without normal forms. Matching types are normalized and compared entrywise. Open terms require complete bindings, so the function decides the supplied closed instances rather than a universally quantified identity.

Guarantees (E, P, B, K). The result uses exact `Fraction` arithmetic and complete semantic types. Matrix and vector work follows the finite expression structure. Coefficient growth can dominate runtime and memory, and the budget can reject large intermediates. Equality is entrywise and exact. The scope excludes general symbolic algebra, analytic functions, arbitrary callbacks, and infinite-dimensional operators.

Evidence and links (V, L). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover structural checking, free bindings, exact normalization, true and false equality decisions, conservative rewrites, dual-path evaluation, plan replay, and tamper rejection. See Chapters 50 through 52 and Chapter 68.

C.1B · `evaluate_expression`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.evaluate_expression`. Reflected implementation `komorebi_math.expression.engine.evaluate_expression`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `function`. Profile `C1-EXPR-ENGINE-FUNCTION`.

Manifest signature (s).

```
(expression: 'Expression', bindings: 'Mapping[str | Symbol, ExpressionBinding] | None' = None, *, budget:
    ↪ 'ExpressionBudget | None' = None) -> 'ExpressionResult'
```

Definition. Evaluate an expression through both direct normalization and compiled-plan execution, requiring exact agreement between the two paths.

Contract (D, C, H, R, X). Direct normalization, compilation, and plan execution are all performed. The planned and direct exact data and types must agree entrywise. Disagreement raises `ValidationError`. The result retains both paths and their combined certificate.

Guarantees (E, P, B, K). The execution language is closed to five opcodes and uses exact rational operations. Compilation and evaluation include independent replay checks and content digests. Matrix-chain planning is cubic in chain length with quadratic tables. Matrix arithmetic and coefficient growth dominate execution. Plan digests are integrity checks, not signatures or authenticated transport.

Evidence and links (V, L). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover structural checking, free bindings, exact normalization, true and false equality decisions, conservative rewrites, dual-path evaluation, plan replay, and tamper rejection. See Chapters 50 through 52 and Chapter 68.

C.1B · `evaluation`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.evaluation`. Reflected implementation `komorebi_math.expression.polynomial.evaluation`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `function`. Profile `C1-EXPR-POLY-CONSTRUCTOR`.

Manifest signature (`s`).

```
(source: 'PolynomialCoordinateSpace', point: 'int | Fraction') -> 'PolynomialPrimitive'
```

Definition. Construct the exact point-evaluation functional from a polynomial coordinate space to the rational scalar space.

Contract (`D, C, H, R, X`). Source must be a polynomial coordinate space. The point is converted to `Fraction`, and the target is `RATIONAL_SPACE`.

Guarantees (`E, P, B, K`). Every accepted primitive has an exact rational matrix determined by finite basis witnesses and parameters. Replay is deterministic and backend-neutral. Basis inversion and operator conversion use dense exact matrix algebra, typically cubic in coordinate dimension before operation-specific work. The certificate covers the finite polynomial map only, not a general analytic theorem.

Evidence and links (`V, L`). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover the Fibonacci-Euler pipeline, basis spoof resistance, all nine primitive kinds, exact replay, coordinate naturality, the basepoint defect, density-aware FTC, and degree-zero differentiation. See Chapters 50, 51, and 68.

C.1B · `exact_binding`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.exact_binding`. Reflected implementation `komorebi_math.expression.engine.exact_binding`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `function`. Profile `C1-EXPR-ENGINE-FUNCTION`.

Manifest signature (`s`).

```
(value: 'Any', declared_type: 'CoordinateSpace | LinearMapType', *, theorem_id: 'str' = 'EXPR.BINDING.EXACT') ->
   ↪ 'ExpressionBinding'
```

Definition. Validate exact rational data against a declared coordinate or map type and wrap it in an unconditional exact binding certificate.

Contract (`D, C, H, R, X`). The declared type is replay-validated. A value for a coordinate space must be a finite exact vector of the declared dimension. A value for a map type must be an exact matrix of target-by-source shape. Only integers and `Fraction` values are accepted. The returned certificate uses the supplied theorem ID and has unconditional exact status.

Guarantees (`E, P, B, K`). The result uses exact `Fraction` arithmetic and complete semantic types. Matrix and vector work follows the finite expression structure. Coefficient growth can dominate runtime and memory, and the budget can reject large intermediates. Equality is entrywise and exact. The scope excludes general symbolic algebra, analytic functions, arbitrary callbacks, and infinite-dimensional operators.

Evidence and links (`V, L`). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover binding-shape replay, exactness weakening, unchecked-assumption propagation, normal-form evidence, and rewrite certificates. See Chapters 51 and 52.

C.1B · `exact_internal_certificate`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.exact_internal_certificate`. Reflected implementation `komorebi_math.expression.certification.exact_internal_certificate`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `function`. Profile `C1-EXPR-CERT-FUNCTION`.

Manifest signature (s).

```
(operation: 'str', *, theorem_id: 'str | None' = None, assumptions_checked: 'Iterable[str]' = (), diagnostics:
   ↪  'Mapping[str, Any] | None' = None) -> 'Certificate'
```

Definition. Create an exact certificate for a finite internal operation whose correctness is established by construction and replayed checks.

Contract (D, C, H, R, X). `operation` must be a nonempty string. A supplied theorem ID and every checked assumption must also be nonempty. Diagnostics must be a mapping with nonempty string keys. The returned certificate is valid, exact, and records the internal-operation assumption. It does not execute an external theorem prover.

Guarantees (E, P, B, K). Evidence status is never promoted above the weakest child claim. Composition is deterministic and linear in child certificates, residual fields, assumptions, and diagnostics. Certificates remain finite records. They do not authenticate the caller, prove an unchecked assumption, or replace independent theorem verification.

Evidence and links (V, L). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover binding-shape replay, exactness weakening, unchecked-assumption propagation, normal-form evidence, and rewrite certificates. See Chapters 51 and 52.

C.1B · exact_map

Identity (q, a, t, k). Public research-manifest route `komorebi_math.exact_map`. Reflected implementation `komorebi_math.expression.model.exact_map`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `function`. Profile `C1-EXPR-CONSTRUCTOR`.

Manifest signature (s).

```
(map_type: 'LinearMapType', matrix: 'Iterable[Iterable[Any]]', *, label: 'str' = '') -> 'ExactMapLiteral'
```

Definition. Normalize an iterable of exact rational rows and construct an `ExactMapLiteral` of the declared shape.

Contract (D, C, H, R, X). Rows and their entries are materialized before delegation to `ExactMapLiteral`. The helper does not infer a map type or resize data. Invalid row structure, rational entries, labels, or shape fail in the literal constructor.

Guarantees (E, P, B, K). Formation checks are exact and local. Accepted numeric data are rational, and accepted endpoints use complete semantic type equality. Construction does not claim denotational equality or execute a matrix operation. Those claims arise only from checking, normalization, or replay. Cost is linear in materialized literal data or child count, otherwise constant.

Evidence and links (V, L). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover type formation, basis witnesses, lexical scope, local node validation, cycle rejection, and every resource-budget dimension. See Chapters 50 and 68.

C.1B · exact_value

Identity (q, a, t, k). Public research-manifest route `komorebi_math.exact_value`. Reflected implementation `komorebi_math.expression.model.exact_value`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `function`. Profile `C1-EXPR-CONSTRUCTOR`.

Manifest signature (s).

```
(space: 'CoordinateSpace', entries: 'Iterable[Any]') -> 'ExactValueLiteral'
```

Definition. Normalize an iterable of exact rational entries and construct an `ExactValueLiteral` in the declared space.

Contract (D, C, H, R, X). The iterable is materialized and delegated to `ExactValueLiteral`. Exact scalar conversion and dimension checks occur there. Infinite iterables are outside the contract.

Guarantees (E, P, B, K). Formation checks are exact and local. Accepted numeric data are rational, and accepted endpoints use complete semantic type equality. Construction does not claim denotational equality or execute a matrix operation. Those claims arise only from checking, normalization, or replay. Cost is linear in materialized literal data or child count, otherwise constant.

Evidence and links (V, L). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover type formation, basis witnesses, lexical scope, local node validation, cycle rejection, and every resource-budget dimension. See Chapters 50 and 68.

C.1B · `exactness_strength`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.exactness_strength`. Reflected implementation `komorebi_math.expression.certification.exactness_strength`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `function`. Profile `C1-EXPR-CERT-FUNCTION`.

Manifest signature (s).

```
(value: 'Exactness') -> 'int'
```

Definition. Return the total-order rank used to compare `Exactness` evidence statuses, where a larger rank is weaker.

Contract (D, C, H, R, X). The argument must be an `Exactness` member. The fixed ranks are exact 0, numerical 1, conditional 2, heuristic 3, and planned 4. Passing a string or another value raises `TypeError`.

Guarantees (E, P, B, K). Evidence status is never promoted above the weakest child claim. Composition is deterministic and linear in child certificates, residual fields, assumptions, and diagnostics. Certificates remain finite records. They do not authenticate the caller, prove an unchecked assumption, or replace independent theorem verification.

Evidence and links (V, L). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover binding-shape replay, exactness weakening, unchecked-assumption propagation, normal-form evidence, and rewrite certificates. See Chapters 51 and 52.

C.1B · `execute_plan`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.execute_plan`. Reflected implementation `komorebi_math.expression.engine.execute_plan`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `function`. Profile `C1-EXPR-ENGINE-FUNCTION`.

Manifest signature (s).

```
(plan: 'ExpressionPlan', *, budget: 'ExpressionBudget | None' = None) -> 'NormalForm'
```

Definition. Revalidate and execute a self-contained exact expression plan, producing a certified normal form.

Contract (D, C, H, R, X). The input must be exactly `ExpressionPlan`. Constructor validation is replayed, instruction count is budgeted, and the content digest is recomputed. Dependencies must be topological. Each instruction is executed with exact rational operations, checked against its declared type and resource limits, and combined into a final normal form.

Guarantees (E, P, B, K). The execution language is closed to five opcodes and uses exact rational operations. Compilation and evaluation include independent replay checks and content digests. Matrix-chain planning is cubic in chain length with quadratic tables. Matrix arithmetic and coefficient growth dominate execution. Plan digests are integrity checks, not signatures or authenticated transport.

Evidence and links (V, L). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover structural checking, free bindings, exact normalization, true and false equality decisions, conservative rewrites, dual-path evaluation, plan replay, and tamper rejection. See Chapters 50 through 52 and Chapter 68.

C.1B · `expression_digest`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.expression_digest`. Reflected implementation `komorebi_math.expression.portable.expression_digest`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `function`. Profile `C1-EXPR-PORTABLE`.

Manifest signature (s).

```
(expression: 'Expression', *, budget: 'ExpressionBudget | None' = None) -> 'str'
```

Definition. Compute the SHA-256 digest of canonical semantic expression JSON, omitting display labels and bound-variable names.

Contract (D, C, H, R, X). Budget preflight runs first. Canonical semantic JSON includes complete type witnesses, exact entries, free-symbol names, node kinds, and primitive parameters. Display labels are omitted and bound symbols use lexical distance. The returned SHA-256 is unkeyed.

Guarantees (E, P, B, K). Encoding and decoding are deterministic for the supported finite grammar and selected budget. Work is linear in the serialized tree and exact witness size, aside from SHA-256 and expression preflight. Shared DAG nodes may appear once per occurrence in the portable tree. The decoder never imports payload-named classes, evaluates source, invokes callbacks, or unpickles data. The checksum detects inconsistent content only when the recorded digest is held fixed and does not authenticate the producer.

Evidence and links (V, L). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover all type and node roundtrips, canonical text, alpha equivalence, duplicate keys, floats, schema and digest tampering, dangling binders, cycles, and resource limits. See Chapters 50, 52, and 68.

C.1B · `expression_from_dict`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.expression_from_dict`. Reflected implementation `komorebi_math.expression.portable.expression_from_dict`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `function`. Profile `C1-EXPR-PORTABLE`.

Manifest signature (s).

```
(value: 'Mapping[str, Any]', *, budget: 'ExpressionBudget | None' = None) -> 'Expression'
```

Definition. Validate a versioned expression envelope, reconstruct the exact built-in expression tree, and verify its semantic digest.

Contract (D, C, H, R, X). The mapping must have exactly the four envelope fields and the current schema and version. The strict decoder reconstructs only known built-in types and nodes, applies occurrence and resource limits, reruns expression preflight, recomputes the semantic digest, and compares it with `hmac.compare_digest`.

Guarantees (E, P, B, K). Encoding and decoding are deterministic for the supported finite grammar and selected budget. Work is linear in the serialized tree and exact witness size, aside from SHA-256 and expression preflight. Shared DAG nodes may appear once per occurrence in the portable tree. The decoder never imports payload-named classes, evaluates source, invokes callbacks, or unpickles data. The checksum detects inconsistent content only when the recorded digest is held fixed and does not authenticate the producer.

Evidence and links (V, L). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover all type and node roundtrips, canonical text, alpha equivalence, duplicate keys, floats, schema and digest tampering, dangling binders, cycles, and resource limits. See Chapters 50, 52, and 68.

C.1B · `expression_from_json`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.expression_from_json`. Reflected implementation `komorebi_math.expression.portable.expression_from_json`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `function`. Profile `C1-EXPR-PORTABLE`.

Manifest signature (s).

```
(value: 'str | bytes', *, budget: 'ExpressionBudget | None' = None) -> 'Expression'
```

Definition. Parse strict canonical UTF-8 expression JSON with duplicate-key, float, constant, resource, schema, and digest checks.

Contract (D, C, H, R, X). Input must be UTF-8 `str` or `bytes` within the size budget. JSON parsing rejects duplicate keys, floats, nonfinite constants, malformed syntax, and excessive recursion. The original text must exactly equal the canonical re-encoding before dictionary-level schema and digest validation proceeds.

Guarantees (E, P, B, K). Encoding and decoding are deterministic for the supported finite grammar and selected budget. Work is linear in the serialized tree and exact witness size, aside from SHA-256 and expression preflight. Shared DAG nodes may appear once per occurrence in the portable tree. The decoder never imports payload-named classes, evaluates source, invokes callbacks, or unpickles data. The checksum detects inconsistent content only when the recorded digest is held fixed and does not authenticate the producer.

Evidence and links (V, L). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover all type and node roundtrips, canonical text, alpha equivalence, duplicate keys, floats, schema and digest tampering, dangling binders, cycles, and resource limits. See Chapters 50, 52, and 68.

C.1B · `expression_to_dict`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.expression_to_dict`. Reflected implementation `komorebi_math.expression.portable.expression_to_dict`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `function`. Profile `C1-EXPR-PORTABLE`.

Manifest signature (s).

```
(expression: 'Expression', *, budget: 'ExpressionBudget | None' = None) -> 'dict[str, Any]'
```

Definition. Encode one expression as a strict versioned dictionary envelope with a semantic SHA-256 digest and symmetric decode-budget validation.

Contract (D, C, H, R, X). The expression is budget-preflighted and encoded as a finite tree with exact field sets and versions. The envelope contains `expression`, `schema`, `schema_version`, and `semantic_sha256`. The produced envelope is then decoded under the same budget so the encoder cannot emit data that its paired decoder rejects.

Guarantees (E, P, B, K). Encoding and decoding are deterministic for the supported finite grammar and selected budget. Work is linear in the serialized tree and exact witness size, aside from SHA-256 and expression preflight. Shared DAG nodes may appear once per occurrence in the portable tree. The decoder never imports payload-named classes, evaluates source, invokes callbacks, or unpickles data. The checksum detects inconsistent content only when the recorded digest is held fixed and does not authenticate the producer.

Evidence and links (V, L). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover all type and node roundtrips, canonical text, alpha equivalence, duplicate keys, floats, schema and digest tampering, dangling binders, cycles, and resource limits. See Chapters 50, 52, and 68.

C.1B · expression_to_json

Identity (q, a, t, k). Public research-manifest route `komorebi_math.expression_to_json`. Reflected implementation `komorebi_math.expression.portable.expression_to_json`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `function`. Profile `C1-EXPR-PORTABLE`.

Manifest signature (s).

```
(expression: 'Expression', *, budget: 'ExpressionBudget | None' = None) -> 'str'
```

Definition. Serialize one expression envelope as compact canonical UTF-8 and NFC JSON text.

Contract (D, C, H, R, X). The dictionary envelope is generated first, then encoded with sorted keys, compact separators, NFC text, and no nonfinite values. The byte length must remain within the selected budget.

Guarantees (E, P, B, K). Encoding and decoding are deterministic for the supported finite grammar and selected budget. Work is linear in the serialized tree and exact witness size, aside from SHA-256 and expression preflight. Shared DAG nodes may appear once per occurrence in the portable tree. The decoder never imports payload-named classes, evaluates source, invokes callbacks, or unpickles data. The checksum detects inconsistent content only when the recorded digest is held fixed and does not authenticate the producer.

Evidence and links (V, L). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover all type and node roundtrips, canonical text, alpha equivalence, duplicate keys, floats, schema and digest tampering, dangling binders, cycles, and resource limits. See Chapters 50, 52, and 68.

C.1B · finite_difference

Identity (q, a, t, k). Public research-manifest route `komorebi_math.finite_difference`. Reflected implementation `komorebi_math.expression.polynomial.finite_difference`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `function`. Profile `C1-EXPR-POLY-CONSTRUCTOR`.

Manifest signature (s).

```
(source: 'PolynomialCoordinateSpace', step: 'int | Fraction' = 1) -> 'PolynomialPrimitive'
```

Definition. Construct the endomorphism `p(x) -> p(x + step) - p(x)` for a nonzero rational step.

Contract (D, C, H, R, X). Source must be a polynomial coordinate space. Step is converted to `Fraction` and must be nonzero. The declared target remains the original source space even though the image degree can drop.

Guarantees (E, P, B, K). Every accepted primitive has an exact rational matrix determined by finite basis witnesses and parameters. Replay is deterministic and backend-neutral. Basis inversion and operator conversion use dense exact matrix algebra, typically cubic in coordinate dimension before operation-specific work. The certificate covers the finite polynomial map only, not a general analytic theorem.

Evidence and links (V, L). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover the Fibonacci-Euler pipeline, basis spoof resistance, all nine primitive kinds, exact replay, coordinate naturality, the basepoint defect, density-aware FTC, and degree-zero differentiation. See Chapters 50, 51, and 68.

C.1B · identity

Identity (q, a, t, k). Public research-manifest route `komorebi_math.identity`. Reflected implementation `komorebi_math.expression.model.identity`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `function`. Profile `C1-EXPR-CONSTRUCTOR`.

Manifest signature (`s`).

```
(space: 'CoordinateSpace') -> 'IdentityMap'
```

Definition. Construct the identity-map expression on a supported coordinate space.

Contract (`D, C, H, R, X`). The function delegates to `IdentityMap`. It creates a typed node, not an allocated dense matrix.

Guarantees (`E, P, B, K`). Formation checks are exact and local. Accepted numeric data are rational, and accepted endpoints use complete semantic type equality. Construction does not claim denotational equality or execute a matrix operation. Those claims arise only from checking, normalization, or replay. Cost is linear in materialized literal data or child count, otherwise constant.

Evidence and links (`V, L`). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover type formation, basis witnesses, lexical scope, local node validation, cycle rejection, and every resource-budget dimension. See Chapters 50 and 68.

C.1B · `interval_integration`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.interval_integration`. Reflected implementation `komorebi_math.expression.polynomial.interval_integration`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `function`. Profile `C1-EXPR-POLY-CONSTRUCTOR`.

Manifest signature (`s`).

```
(source: 'PolynomialCoordinateSpace', lower: 'int | Fraction', upper: 'int | Fraction', density: 'int | Fraction' =
    ↪ 1) -> 'PolynomialPrimitive'
```

Definition. Construct the exact density-weighted polynomial integration functional on one closed rational interval.

Contract (`D, C, H, R, X`). Source must be a polynomial coordinate space. Lower, upper, and density are converted to `Fraction`. Lower must not exceed upper and density must be nonnegative. The target is `RATIONAL_SPACE`.

Guarantees (`E, P, B, K`). Every accepted primitive has an exact rational matrix determined by finite basis witnesses and parameters. Replay is deterministic and backend-neutral. Basis inversion and operator conversion use dense exact matrix algebra, typically cubic in coordinate dimension before operation-specific work. The certificate covers the finite polynomial map only, not a general analytic theorem.

Evidence and links (`V, L`). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover the Fibonacci-Euler pipeline, basis spoof resistance, all nine primitive kinds, exact replay, coordinate naturality, the basepoint defect, density-aware FTC, and degree-zero differentiation. See Chapters 50, 51, and 68.

C.1B · `let`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.let`. Reflected implementation `komorebi_math.expression.model.let`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `function`. Profile `C1-EXPR-CONSTRUCTOR`.

Manifest signature (`s`).

```
(binding: 'Symbol', value: 'Expression', body: 'Expression') -> 'Let'
```

Definition. Construct a lexical `Let` node after checking that the bound value matches the symbol declaration.

Contract (`D, C, H, R, X`). The function delegates to `Let`. It does not substitute or evaluate the body at construction time. Binder identity and type compatibility are enforced by the node.

Guarantees (`E, P, B, K`). Formation checks are exact and local. Accepted numeric data are rational, and accepted endpoints use complete semantic type equality. Construction does not claim denotational equality or execute a

matrix operation. Those claims arise only from checking, normalization, or replay. Cost is linear in materialized literal data or child count, otherwise constant.

Evidence and links (V, L). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover type formation, basis witnesses, lexical scope, local node validation, cycle rejection, and every resource-budget dimension. See Chapters 50 and 68.

C.1B · `matrix_chain_multiply`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.matrix_chain_multiply`. Reflected implementation `komorebi_math.expression.planning.matrix_chain_multiply`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `function`. Profile `C1-EXPR-CHAIN-FUNCTION`.

Manifest signature (s).

```
(matrices: 'Sequence[Iterable[Iterable[Any]]]', plan: 'MatrixChainPlan') -> 'Matrix'
```

Definition. Multiply a finite exact rational matrix chain by replaying a validated optimal matrix-chain plan.

Contract (D, C, H, R, X). The plan is replay-validated. The matrix sequence must have the recorded count. Every matrix is finite, nonempty, rectangular, exact rational, and of the declared shape. The recorded split tree is executed iteratively with exact matrix multiplication, and the final shape is checked.

Guarantees (E, P, B, K). The optimality claim is exact only for order-preserving binary parenthesizations under classical dense scalar-multiplication count. Planning uses `O(n^3)` time and `O(n^2)` table space for `n` matrices. It does not model additions, sparsity, rank, memory traffic, parallelism, GPU launches, Strassen-like algorithms, rounding error, or wall-clock time.

Evidence and links (V, L). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover matrix-chain optimum, tie breaking, exhaustive small-chain comparison, shape validation, exact multiplication, and plan tamper rejection. See Chapters 52 and 68.

C.1B · `matrix_chain_parenthesization`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.matrix_chain_parenthesization`. Reflected implementation `komorebi_math.expression.planning.matrix_chain_parenthesization`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `function`. Profile `C1-EXPR-CHAIN-FUNCTION`.

Manifest signature (s).

```
(plan: 'MatrixChainPlan', labels: 'Sequence[str] | None' = None) -> 'str'
```

Definition. Render the deterministic binary tree selected by a validated matrix-chain plan.

Contract (D, C, H, R, X). The plan is fully replay-validated. Optional labels must be a finite sequence with one nonempty NFC-normalized string per matrix. The selected tree is rendered iteratively with `@`. Invalid labels, counts, or plans fail.

Guarantees (E, P, B, K). The optimality claim is exact only for order-preserving binary parenthesizations under classical dense scalar-multiplication count. Planning uses `O(n^3)` time and `O(n^2)` table space for `n` matrices. It does not model additions, sparsity, rank, memory traffic, parallelism, GPU launches, Strassen-like algorithms, rounding error, or wall-clock time.

Evidence and links (V, L). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover matrix-chain optimum, tie breaking, exhaustive small-chain comparison, shape validation, exact multiplication, and plan tamper rejection. See Chapters 52 and 68.

C.1B · matrix_chain_plan

Identity (q, a, t, k). Public research-manifest route `komorebi_math.matrix_chain_plan`. Reflected implementation `komorebi_math.expression.planning.matrix_chain_plan`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `function`. Profile `C1-EXPR-CHAIN-FUNCTION`.

Manifest signature (s).

```
(dimensions: 'Sequence[int]') -> 'MatrixChainPlan'
```

Definition. Find the exact minimum classical dense scalar-multiplication cost over all order-preserving binary parenthesizations of a dimension chain.

Contract (D, C, H, R, X). The dimension sequence must contain at least two positive integers and no booleans. The dynamic program examines every order-preserving binary split, choosing the smallest split index on ties. A fixed one-million-candidate guard raises `ResourceLimitError` before excessive planning.

Guarantees (E, P, B, K). The optimality claim is exact only for order-preserving binary parenthesizations under classical dense scalar-multiplication count. Planning uses `O(n^3)` time and `O(n^2)` table space for `n` matrices. It does not model additions, sparsity, rank, memory traffic, parallelism, GPU launches, Strassen-like algorithms, rounding error, or wall-clock time.

Evidence and links (V, L). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover matrix-chain optimum, tie breaking, exhaustive small-chain comparison, shape validation, exact multiplication, and plan tamper rejection. See Chapters 52 and 68.

C.1B · normal_form_digest

Identity (q, a, t, k). Public research-manifest route `komorebi_math.normal_form_digest`. Reflected implementation `komorebi_math.expression.engine.normal_form_digest`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `function`. Profile `C1-EXPR-ENGINE-FUNCTION`.

Manifest signature (s).

```
(data: 'ExactData', expression_type: 'Any') -> 'str'
```

Definition. Validate typed exact vector or matrix data and hash its canonical normal-form record.

Contract (D, C, H, R, X). The type is validated, data are converted to the exact vector or matrix required by that type, and canonical rational tokens plus the complete type witness are hashed. Shape and scalar violations raise before hashing. The digest is unkeyed metadata, not authentication.

Guarantees (E, P, B, K). The digest is deterministic for identical exact type and data records. Validation and encoding are linear in exact entries and type-witness size. SHA-256 supplies an integrity identifier but no secrecy, origin authentication, or collision-proof mathematical theorem.

Evidence and links (V, L). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover structural checking, free bindings, exact normalization, true and false equality decisions, conservative rewrites, dual-path evaluation, plan replay, and tamper rejection. See Chapters 50 through 52 and Chapter 68.

C.1B · normalize_expression

Identity (q, a, t, k). Public research-manifest route `komorebi_math.normalize_expression`. Reflected implementation `komorebi_math.expression.engine.normalize_expression`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `function`. Profile `C1-EXPR-ENGINE-FUNCTION`.

Manifest signature (s).

```
(expression: 'Expression', bindings: 'Mapping[str | Symbol, ExpressionBinding] | None' = None, *, budget:
   ↪  'ExpressionBudget | None' = None) -> 'NormalForm'
```

Definition. Reduce a closed, well-typed finite expression to its unique exact rational vector or matrix normal form.

Contract (D, C, H, R, X). The expression is first checked under the selected budget. Every free symbol must resolve to a compatible binding. Literals, primitives, lexical bindings, sums, scales, compositions, and applications are evaluated with exact rational vector and matrix operations. Intermediate and final data are budget-checked. Missing bindings, type failures, coefficient growth beyond limits, or unsupported nodes raise explicit exceptions.

Guarantees (E, P, B, K). The result uses exact `Fraction` arithmetic and complete semantic types. Matrix and vector work follows the finite expression structure. Coefficient growth can dominate runtime and memory, and the budget can reject large intermediates. Equality is entrywise and exact. The scope excludes general symbolic algebra, analytic functions, arbitrary callbacks, and infinite-dimensional operators.

Evidence and links (V, L). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover structural checking, free bindings, exact normalization, true and false equality decisions, conservative rewrites, dual-path evaluation, plan replay, and tamper rejection. See Chapters 50 through 52 and Chapter 68.

C.1B · `primitive_certificate`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.primitive_certificate`. Reflected implementation `komorebi_math.expression.polynomial.primitive_certificate`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `function`. Profile `C1-EXPR-POLY-REPLAY`.

Manifest signature (s).

```
(primitive: 'PolynomialPrimitive', *, claimed_matrix: 'Iterable[Iterable[Any]] | None' = None) -> 'Certificate'
```

Definition. Certify exact structural replay of a polynomial primitive and, when supplied, exact equality with a claimed matrix.

Contract (D, C, H, R, X). The argument must be a `PolynomialPrimitive`. Replay is always performed. Without a claim the certificate is valid and exact. With `claimed_matrix`, validity additionally requires exact shape and entrywise equality. The theorem ID is determined by primitive kind and tolerance is zero.

Guarantees (E, P, B, K). Every accepted primitive has an exact rational matrix determined by finite basis witnesses and parameters. Replay is deterministic and backend-neutral. Basis inversion and operator conversion use dense exact matrix algebra, typically cubic in coordinate dimension before operation-specific work. The certificate covers the finite polynomial map only, not a general analytic theorem.

Evidence and links (V, L). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover the Fibonacci-Euler pipeline, basis spoof resistance, all nine primitive kinds, exact replay, coordinate naturality, the basepoint defect, density-aware FTC, and degree-zero differentiation. See Chapters 50, 51, and 68.

C.1B · `primitive_matrix_equal`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.primitive_matrix_equal`. Reflected implementation `komorebi_math.expression.polynomial.primitive_matrix_equal`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `function`. Profile `C1-EXPR-POLY-REPLAY`.

Manifest signature (`s`).

```
(primitive: 'PolynomialPrimitive', matrix: 'Iterable[Iterable[Any]]') -> 'bool'
```

Definition. Validate a claimed exact rational matrix and compare it entrywise with a freshly replayed polynomial primitive.

Contract (`D, C, H, R, X`). The claimed rows are materialized and converted only from integers or `Fraction`. Ragged or invalid data raise. A shape mismatch returns `False`. Otherwise exact tuple equality is compared with a fresh replay.

Guarantees (`E, P, B, K`). Every accepted primitive has an exact rational matrix determined by finite basis witnesses and parameters. Replay is deterministic and backend-neutral. Basis inversion and operator conversion use dense exact matrix algebra, typically cubic in coordinate dimension before operation-specific work. The certificate covers the finite polynomial map only, not a general analytic theorem.

Evidence and links (`V, L`). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover the Fibonacci-Euler pipeline, basis spoof resistance, all nine primitive kinds, exact replay, coordinate naturality, the basepoint defect, density-aware FTC, and degree-zero differentiation. See Chapters 50, 51, and 68.

C.1B · `replay`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.replay`. Reflected implementation `komorebi_math.expression.polynomial.replay`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `function`. Profile `C1-EXPR-POLY-REPLAY`.

Manifest signature (`s`).

```
(primitive: 'PolynomialPrimitive') -> 'Matrix'
```

Definition. Recompute the exact rational matrix of a polynomial primitive solely from its basis witnesses, kind, map type, and parameters.

Contract (`D, C, H, R, X`). The argument must be a `PolynomialPrimitive`. The matrix is recomputed from exact basis matrices and rational parameters for the recorded kind. No callback, cached matrix, display label, or claimed result is trusted. Unsupported or malformed primitives fail closed.

Guarantees (`E, P, B, K`). Every accepted primitive has an exact rational matrix determined by finite basis witnesses and parameters. Replay is deterministic and backend-neutral. Basis inversion and operator conversion use dense exact matrix algebra, typically cubic in coordinate dimension before operation-specific work. The certificate covers the finite polynomial map only, not a general analytic theorem.

Evidence and links (`V, L`). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover the Fibonacci-Euler pipeline, basis spoof resistance, all nine primitive kinds, exact replay, coordinate naturality, the basepoint defect, density-aware FTC, and degree-zero differentiation. See Chapters 50, 51, and 68.

C.1B · `replay_matrix`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.replay_matrix`. Reflected implementation `komorebi_math.expression.polynomial.replay`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `function`. Profile `C1-EXPR-POLY-REPLAY`.

Manifest signature (s).

```
(primitive: 'PolynomialPrimitive') -> 'Matrix'
```

Definition. The public name-preserving alias of `replay` for recomputing a polynomial primitive matrix.

Contract (D, C, H, R, X). The argument must be a `PolynomialPrimitive`. The matrix is recomputed from exact basis matrices and rational parameters for the recorded kind. No callback, cached matrix, display label, or claimed result is trusted. Unsupported or malformed primitives fail closed.

Guarantees (E, P, B, K). Every accepted primitive has an exact rational matrix determined by finite basis witnesses and parameters. Replay is deterministic and backend-neutral. Basis inversion and operator conversion use dense exact matrix algebra, typically cubic in coordinate dimension before operation-specific work. The certificate covers the finite polynomial map only, not a general analytic theorem.

Evidence and links (V, L). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover the Fibonacci-Euler pipeline, basis spoof resistance, all nine primitive kinds, exact replay, coordinate naturality, the basepoint defect, density-aware FTC, and degree-zero differentiation. See Chapters 50, 51, and 68.

C.1B · `scale`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.scale`. Reflected implementation `komorebi_math.expression.model.scale`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `function`. Profile `C1-EXPR-CONSTRUCTOR`.

Manifest signature (s).

```
(scalar: 'int | Fraction', value: 'Expression') -> 'Scale'
```

Definition. Convert an integer or `Fraction` coefficient and construct a typed `Scale` node.

Contract (D, C, H, R, X). The coefficient is converted to an exact `Fraction` and passed to `Scale`. Booleans, floats, and non-expression values are rejected.

Guarantees (E, P, B, K). Formation checks are exact and local. Accepted numeric data are rational, and accepted endpoints use complete semantic type equality. Construction does not claim denotational equality or execute a matrix operation. Those claims arise only from checking, normalization, or replay. Cost is linear in materialized literal data or child count, otherwise constant.

Evidence and links (V, L). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover type formation, basis witnesses, lexical scope, local node validation, cycle rejection, and every resource-budget dimension. See Chapters 50 and 68.

C.1B · `simplify_expression`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.simplify_expression`. Reflected implementation `komorebi_math.expression.engine.simplify_expression`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `function`. Profile `C1-EXPR-ENGINE-FUNCTION`.

Manifest signature (s).

```
(expression: 'Expression', bindings: 'Mapping[str | Symbol, ExpressionBinding] | None' = None, *, budget:
   ↪ 'ExpressionBudget | None' = None) -> 'SimplificationResult'
```

Definition. Apply the finite conservative rewrite system and certify every accepted step by exact before-and-after normalization.

Contract (D, C, H, R, X). The source is normalized first. The restricted rewrite system handles identities, zeros, scale and sum flattening, literal application, coordinate cocycles and transports, derivative-antiderivative laws, the basepoint

defect, and the finite polynomial FTC. Every proposed step is accepted only after exact before-and-after normal forms agree. A 256-step local convergence guard and the global budget prevent unbounded rewriting.

Guarantees (E, P, B, K). The result uses exact `Fraction` arithmetic and complete semantic types. Matrix and vector work follows the finite expression structure. Coefficient growth can dominate runtime and memory, and the budget can reject large intermediates. Equality is entrywise and exact. The scope excludes general symbolic algebra, analytic functions, arbitrary callbacks, and infinite-dimensional operators.

Evidence and links (V, L). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover structural checking, free bindings, exact normalization, true and false equality decisions, conservative rewrites, dual-path evaluation, plan replay, and tamper rejection. See Chapters 50 through 52 and Chapter 68.

C.1B · `symbol`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.symbol`. Reflected implementation `komorebi_math.expression.model.symbol`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `function`. Profile `C1-EXPR-CONSTRUCTOR`.

Manifest signature (s).

```
(name: 'str', declared_type: 'ExpressionType') -> 'Symbol'
```

Definition. Construct an identity-distinct typed `Symbol` with an NFC-normalized nonempty name.

Contract (D, C, H, R, X). The function delegates to `Symbol` and returns a fresh identity-distinct object on every call, even when name and type are equal.

Guarantees (E, P, B, K). Formation checks are exact and local. Accepted numeric data are rational, and accepted endpoints use complete semantic type equality. Construction does not claim denotational equality or execute a matrix operation. Those claims arise only from checking, normalization, or replay. Cost is linear in materialized literal data or child count, otherwise constant.

Evidence and links (V, L). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover type formation, basis witnesses, lexical scope, local node validation, cycle rejection, and every resource-budget dimension. See Chapters 50 and 68.

C.1B · `then`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.then`. Reflected implementation `komorebi_math.expression.model.then`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `function`. Profile `C1-EXPR-CONSTRUCTOR`.

Manifest signature (s).

```
(*maps: 'Expression') -> 'Expression'
```

Definition. Compose maps in the order in which they act, reversing them before delegating to `compose`.

Contract (D, C, H, R, X). At least one map is required. Inputs are interpreted in action order, reversed, and passed to `compose`. Thus `then(A, B)` represents `B` after `A`.

Guarantees (E, P, B, K). Formation checks are exact and local. Accepted numeric data are rational, and accepted endpoints use complete semantic type equality. Construction does not claim denotational equality or execute a matrix operation. Those claims arise only from checking, normalization, or replay. Cost is linear in materialized literal data or child count, otherwise constant.

Evidence and links (V, L). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_`

example.py cover type formation, basis witnesses, lexical scope, local node validation, cycle rejection, and every resource-budget dimension. See Chapters 50 and 68.

C.1B · type_from_dict

Identity (q, a, t, k). Public research-manifest route komorebi_math.type_from_dict. Reflected implementation komorebi_math.expression.portable.type_from_dict. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by [RESEARCH] policy pre-beta-additive. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind function. Profile C1-EXPR-PORTABLE.

Manifest signature (s).

```
(value: 'Mapping[str, Any]', *, budget: 'ExpressionBudget | None' = None) -> 'ExpressionType'
```

Definition. Reconstruct one exact expression type from its strict, versioned transport record under a resource budget.

Contract (D, C, H, R, X). The input mapping is validated recursively under the selected budget. Exact field sets, versions, strings, integers, rational witnesses, and supported type kinds are required. Unknown or extra fields fail closed.

Guarantees (E, P, B, K). Encoding and decoding are deterministic for the supported finite grammar and selected budget. Work is linear in the serialized tree and exact witness size, aside from SHA-256 and expression preflight. Shared DAG nodes may appear once per occurrence in the portable tree. The decoder never imports payload-named classes, evaluates source, invokes callbacks, or unpickles data. The checksum detects inconsistent content only when the recorded digest is held fixed and does not authenticate the producer.

Evidence and links (V, L). The committed research manifest, runtime reflection audit, tests/test_block20_expression_calculus.py, verification/block20_math_verifier.py, and examples/block20_typed_expression_worked_example.py cover all type and node roundtrips, canonical text, alpha equivalence, duplicate keys, floats, schema and digest tampering, dangling binders, cycles, and resource limits. See Chapters 50, 52, and 68.

C.1B · type_to_dict

Identity (q, a, t, k). Public research-manifest route komorebi_math.type_to_dict. Reflected implementation komorebi_math.expression.portable.type_to_dict. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by [RESEARCH] policy pre-beta-additive. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind function. Profile C1-EXPR-PORTABLE.

Manifest signature (s).

```
(expression_type: 'ExpressionType', *, budget: 'ExpressionBudget | None' = None) -> 'dict[str, Any]'
```

Definition. Encode one exact expression type as a strict transport dictionary and replay the decoder against that record.

Contract (D, C, H, R, X). A default or exact ExpressionBudget is selected. The supported type is encoded with explicit kind and version fields, the JSON tree and encoded size are checked, and the decoder is replayed before the dictionary is returned.

Guarantees (E, P, B, K). Encoding and decoding are deterministic for the supported finite grammar and selected budget. Work is linear in the serialized tree and exact witness size, aside from SHA-256 and expression preflight. Shared DAG nodes may appear once per occurrence in the portable tree. The decoder never imports payload-named classes, evaluates source, invokes callbacks, or unpickles data. The checksum detects inconsistent content only when the recorded digest is held fixed and does not authenticate the producer.

Evidence and links (V, L). The committed research manifest, runtime reflection audit, tests/test_block20_expression_calculus.py, verification/block20_math_verifier.py, and examples/block20_typed_expression_worked_example.py cover all type and node roundtrips, canonical text, alpha equivalence, duplicate keys, floats, schema and digest tampering, dangling binders, cycles, and resource limits. See Chapters 50, 52, and 68.

C.1B · weakest_exactness

Identity (q, a, t, k). Public research-manifest route `komorebi_math.weakest_exactness`. Reflected implementation `komorebi_math.expression.certification.weakest_exactness`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `function`. Profile `C1-EXPR-CERT-FUNCTION`.

Manifest signature (s).

```
(*values: 'Exactness') -> 'Exactness'
```

Definition. Return the weakest supplied evidence status, with `EXACT` as the empty meet.

Contract (D, C, H, R, X). With no arguments the function returns `Exactness.EXACT`. Otherwise every argument must be an `Exactness` member, and the one with the largest weakness rank is returned.

Guarantees (E, P, B, K). Evidence status is never promoted above the weakest child claim. Composition is deterministic and linear in child certificates, residual fields, assumptions, and diagnostics. Certificates remain finite records. They do not authenticate the caller, prove an unchecked assumption, or replace independent theorem verification.

Evidence and links (V, L). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover binding-shape replay, exactness weakening, unchecked-assumption propagation, normal-form evidence, and rewrite certificates. See Chapters 51 and 52.

C.1B · zero_map

Identity (q, a, t, k). Public research-manifest route `komorebi_math.zero_map`. Reflected implementation `komorebi_math.expression.model.zero_map`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It is not part of the frozen stable or curated beta compatibility promise. Manifest kind `function`. Profile `C1-EXPR-CONSTRUCTOR`.

Manifest signature (s).

```
(map_type: 'LinearMapType') -> 'ZeroMap'
```

Definition. Construct the exact zero-map expression for a declared `LinearMapType`.

Contract (D, C, H, R, X). The function delegates to `ZeroMap`. It creates a typed node, not an allocated dense matrix.

Guarantees (E, P, B, K). Formation checks are exact and local. Accepted numeric data are rational, and accepted endpoints use complete semantic type equality. Construction does not claim denotational equality or execute a matrix operation. Those claims arise only from checking, normalization, or replay. Cost is linear in materialized literal data or child count, otherwise constant.

Evidence and links (V, L). The committed research manifest, runtime reflection audit, `tests/test_block20_expression_calculus.py`, `verification/block20_math_verifier.py`, and `examples/block20_typed_expression_worked_example.py` cover type formation, basis witnesses, lexical scope, local node validation, cycle rejection, and every resource-budget dimension. See Chapters 50 and 68.

C.1C Coordinate systems, transforms, realizations, and frame algebra: 53 entries

This section documents the complete root-exported surface owned by the `komorebi_math.coordinates` implementation family. It covers finite bases and exact coordinate maps, affine and formal changes, finite-field and tensor transport, recurrence and state-space realizations, polynomial connections, numerical and exact analysis and synthesis systems, coefficient gauges, operator transport, resource guards, and portable coordinate persistence. Exact rational contracts and tolerance-based NumPy contracts are stated separately. A numerical certificate records a finite residual test. It does not convert that test into an exact identity.

C.1C · AffineCoordinateTransform

Identity (q, a, t, k). Public research-manifest route `komorebi_math.AffineCoordinateTransform`. Reflected implementation `komorebi_math.coordinates.affine.AffineCoordinateTransform`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive` and is not part of the frozen stable compatibility promise. Manifest kind `class`. Profile `C1-COORD-AFFINE`.

Manifest signature (s).

```
(source: 'str', target: 'str', matrix: 'Matrix', offset: 'Vector', theorem: 'str' = 'CC.F1', description: 'str' =
   ↪  '') -> None
```

Definition. An exact affine map `y=A x+b` between equal-dimensional spaces.

Contract (D, C, H, R, X). Construction requires a square invertible exact matrix, a matching offset, and nonempty endpoint names. Application, composition, inversion, and homogeneous lifting preserve exact rational arithmetic and reject incompatible dimensions or endpoints. Principal public operations are `apply`, `certificate`, `dimension`, `homogeneous_matrix`, `inverse`, `invertible`, `then`.

Guarantees (E, P, B, K). Structural arithmetic is exact over integers and `Fraction` values. No tolerance, accelerator, network, or symbolic backend is consulted. Runtime and memory are finite and are dominated by matrix materialization, exact elimination, nullspace work, recurrence propagation, or polynomial convolution appropriate to the operation. Dense elimination can require cubic arithmetic-operation cost and exact coefficients may grow.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. `tests/test_coordinates_general_affine_formal.py` and `verification/block14_general_coordinate_audit.py` cover the applicable construction, roundtrip, certificate, failure, and persistence behavior. The source validator for coordinate facades also checks the exact signature, implementation identity, coordinate facade, anchor, and stable aliases. See Chapters 15, 16, and 64.

C.1C · AnalysisSynthesisBlocks

Identity (q, a, t, k). Public research-manifest route `komorebi_math.AnalysisSynthesisBlocks`. Reflected implementation `komorebi_math.coordinates.analysis_synthesis.AnalysisSynthesisBlocks`. The same object is available through the curated stable routes `komorebi_math.stable.AnalysisSynthesisBlocks`, `komorebi_math.v1.AnalysisSynthesisBlocks`, and `komorebi_math.api.v1.AnalysisSynthesisBlocks`. Those facade routes carry the frozen 1.x compatibility promise. The root route remains governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `class`. Profile `C1-COORD-ANALYSIS-SYNTHESIS`.

Manifest signature (s).

```
(system: "'AnalysisSynthesisSystem'", groups: 'tuple[tuple[int, ...], ...]', blocks: 'tuple[tuple[np.ndarray,
   ↪  ...], ...]', operator: 'np.ndarray') -> None
```

Definition. Packet blocks of one operator under a declared analysis and synthesis atlas.

Contract (D, C, H, R, X). The groups must form valid coefficient packets for the supplied system. Every block and the represented operator must have source-space shape. `reconstruct` sums the blocks and `certificate` compares the reconstruction residual with the system tolerance. Principal public operations are `certificate`, `reconstruct`.

Guarantees (E, P, B, K). The implementation uses finite NumPy arithmetic. Residuals, rank, condition numbers, and gauge equivalence are tolerance-based evidence, not exact identities. Dense factorizations and matrix products can require cubic time and quadratic storage.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. `tests/test_block16_analysis_synthesis.py` and `examples/block16_analysis_synthesis.py` cover the applicable construction, roundtrip, certificate, failure, and persistence behavior. The source validator for coordinate facades also checks the exact signature, implementation identity, coordinate facade, anchor, and stable aliases. See Chapters 20, 48, and 64.

C.1C · AnalysisSynthesisCoefficientBlocks

Identity (q, a, t, k). Public research-manifest route `komorebi_math.AnalysisSynthesisCoefficientBlocks`. Reflected implementation `komorebi_math.coordinates.frame_algebra.AnalysisSynthesisCoefficientBlocks`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive` and is not part of the frozen stable compatibility promise. Manifest kind `class`. Profile `C1-COORD-FRAME-ALGEBRA`.

Manifest signature (s).

```
(mapping: 'AnalysisSynthesisMap', source_partition: 'CoefficientPartition', target_partition:
    ↪ 'CoefficientPartition') -> None
```

Definition. Small target-by-source blocks derived from a coefficient lift.

Contract (D, C, H, R, X). Domain and codomain partitions must match the coefficient dimensions of the supplied map. Block lookup and reblocking validate indices and endpoints. Assembly and composition operate through the canonical coefficient lift. Principal public operations are `assemble`, `certificate`, `physical_block`, `reblock`, `then`.

Guarantees (E, P, B, K). Exact variants use rational arithmetic. Numerical variants use finite NumPy arithmetic at an explicit tolerance. Dense lifts, pseudoinverses, projections, direct sums, tensor products, and block assembly can require cubic time and quadratic storage.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. `tests/test_block18_frame_algebra.py` and the independent verifier for coordinate facades cover the applicable construction, roundtrip, certificate, failure, and persistence behavior. The source validator for coordinate facades also checks the exact signature, implementation identity, coordinate facade, anchor, and stable aliases. See Chapters 20, 48, and 64.

C.1C · AnalysisSynthesisGauge

Identity (q, a, t, k). Public research-manifest route `komorebi_math.AnalysisSynthesisGauge`. Reflected implementation `komorebi_math.coordinates.frame_algebra.AnalysisSynthesisGauge`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive` and is not part of the frozen stable compatibility promise. Manifest kind `class`. Profile `C1-COORD-FRAME-ALGEBRA`.

Manifest signature (s).

```
(system: 'AnalysisSynthesisSystem') -> None
```

Definition. Gauge and quotient calculus for one numerical presentation.

Contract (D, C, H, R, X). The constructor accepts one numerical analysis and synthesis system. Its methods split coefficients into canonical and gauge parts, evaluate quotient metrics, and test gauge equivalence at the system tolerance. Principal public operations are `analysis`, `canonical_coefficients`, `certificate`, `coefficient_metric`, `coefficient_projection`, `coefficient_seminorm`, `coordinate_dimension`, `coordinates`, `direct_sum`, `dual`, `gauge_equivalent`, `gauge_part`, and related accessors.

Guarantees (E, P, B, K). Exact variants use rational arithmetic. Numerical variants use finite NumPy arithmetic at an explicit tolerance. Dense lifts, pseudoinverses, projections, direct sums, tensor products, and block assembly can require cubic time and quadratic storage.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. `tests/test_block18_frame_algebra.py` and the independent verifier for coordinate facades cover the applicable construction, roundtrip, certificate, failure, and persistence behavior. The source validator for coordinate facades also checks the exact signature, implementation identity, coordinate facade, anchor, and stable aliases. See Chapters 20, 48, and 64.

C.1C · AnalysisSynthesisMap

Identity (q, a, t, k). Public research-manifest route `komorebi_math.AnalysisSynthesisMap`. Reflected implementation `komorebi_math.coordinates.frame_algebra.AnalysisSynthesisMap`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive` and is not part of the frozen stable compatibility promise. Manifest kind `class`. Profile `C1-COORD-FRAME-ALGEBRA`.

Manifest signature (s).

```
(domain: 'AnalysisSynthesisSystem', codomain: 'AnalysisSynthesisSystem', operator: 'np.ndarray', theorem: 'str' =
    ↪ 'KM-AS-MAP-01') -> None
```

Definition. A physical linear map and its canonical coefficient lift.

Contract (D, C, H, R, X). Domain and codomain presentations must have compatible physical dimensions, and the operator must map between their source spaces. Coefficient lifts, composition, adjoints, bridges, and blocks preserve declared endpoints and validate shapes. Principal public operations are `add`, `adjoint`, `apply_coefficients`, `apply_source`, `blocks`, `certificate`, `coefficient_lift`, `compose`, `dual_adjoint`, `reconstruct`, `scaled`, `then`, and related accessors.

Guarantees (E, P, B, K). Exact variants use rational arithmetic. Numerical variants use finite NumPy arithmetic at an explicit tolerance. Dense lifts, pseudoinverses, projections, direct sums, tensor products, and block assembly can require cubic time and quadratic storage.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. `tests/test_block18_frame_algebra.py` and the independent verifier for coordinate facades cover the applicable construction, roundtrip, certificate, failure, and persistence behavior. The source validator for coordinate facades also checks the exact signature, implementation identity, coordinate facade, anchor, and stable aliases. See Chapters 20, 48, and 64.

C.1C · AnalysisSynthesisSystem

Identity (q, a, t, k). Public research-manifest route `komorebi_math.AnalysisSynthesisSystem`. Reflected implementation `komorebi_math.coordinates.analysis_synthesis.AnalysisSynthesisSystem`. The same object is available through the curated stable routes `komorebi_math.stable.AnalysisSynthesisSystem`, `komorebi_math.v1.AnalysisSynthesisSystem`, and `komorebi_math.api.v1.AnalysisSynthesisSystem`. Those facade routes carry the frozen 1.x compatibility promise. The root route remains governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `class`. Profile `C1-COORD-ANALYSIS-SYNTHESIS`.

Manifest signature (s).

```
(analysis: 'np.ndarray', synthesis: 'np.ndarray', source: 'str' = 'source', coordinates: 'str' = 'coordinates',
    ↪ tolerance: 'float' = 1e-12, theorem: 'str' = 'KM-AS-01') -> None
```

Definition. A finite analysis map `A` and synthesis map `S` with `S A = I`.

Contract (D, C, H, R, X). Analysis and synthesis must have transposed rectangular shapes, names must be nonempty, and tolerance must be positive and finite. Direct construction records a candidate pair. `certificate` checks rank and the residual of `S A - I`, while `canonical_dual` and `from_invertible` construct and require valid systems. Principal public operations are `analyze`, `certificate`, `component`, `components`, `coordinate_dimension`, `frame_operator`, `operator_blocks`, `projector`, `projectors`, `reconstruct`, `reconstruction_certificate`, `redundant`, and related accessors.

Guarantees (E, P, B, K). The implementation uses finite NumPy arithmetic. Residuals, rank, condition numbers, and gauge equivalence are tolerance-based evidence, not exact identities. Dense factorizations and matrix products can require cubic time and quadratic storage.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. `tests/test_block16_analysis_synthesis.py` and `examples/block16_analysis_synthesis.py` cover the applicable construction, roundtrip, certificate, failure, and persistence behavior. The source validator for coordinate facades also checks the exact signature, implementation identity, coordinate facade, anchor, and stable aliases. See Chapters 20, 48, and 64.

C.1C · BezoutDefectIdentity

Identity (q, a, t, k). Public research-manifest route `komorebi_math.BezoutDefectIdentity`. Reflected implementation `komorebi_math.coordinates.operators.BezoutDefectIdentity`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive` and is not part of the frozen stable compatibility promise. Manifest kind `class`. Profile `C1-COORD-OPERATORS`.

Manifest signature (s).

```
(gcd: 'Vector', left_multiplier: 'Vector', right_multiplier: 'Vector', reconstructed: 'Vector', expected:
   ↪  'Vector', valid: 'bool') -> None
```

Definition. A finite prefix reconstruction from two coprime defect streams.

Contract (D, C, H, R, X). The frozen record stores the gcd polynomial, Bezout multipliers, reconstructed prefix, expected prefix, and validity flag. Direct construction does not recompute the algebra. `bezout_defect_identity` is the authoritative constructor and `certificate` reports the finite exact identity. Principal public operations are `certificate`.

Guarantees (E, P, B, K). Structural arithmetic is exact over integers and `Fraction` values. No tolerance, accelerator, network, or symbolic backend is consulted. Runtime and memory are finite and are dominated by matrix materialization, exact elimination, nullspace work, recurrence propagation, or polynomial convolution appropriate to the operation. Dense elimination can require cubic arithmetic-operation cost and exact coefficients may grow.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. `tests/test_coordinates_operators.py` covers the applicable construction, roundtrip, certificate, failure, and persistence behavior. The source validator for coordinate facades also checks the exact signature, implementation identity, coordinate facade, anchor, and stable aliases. See Chapters 10, 18, and 64.

C.1C · CoefficientPartition

Identity (q, a, t, k). Public research-manifest route `komorebi_math.CoefficientPartition`. Reflected implementation `komorebi_math.coordinates.frame_algebra.CoefficientPartition`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive` and is not part of the frozen stable compatibility promise. Manifest kind `class`. Profile `C1-COORD-FRAME-ALGEBRA`.

Manifest signature (s).

```
(dimension: 'int', groups: 'tuple[tuple[int, ...], ...]') -> None
```

Definition. A strict ordered partition of finite coefficient indices.

Contract (D, C, H, R, X). Numerical records require compatible finite arrays and tolerances. Exact records require compatible rational matrices. Direct record construction stores the declared data, while certificate and factory routes provide the authoritative reconstruction, lift, gauge, or block checks. Principal public operations are `coarsens`, `refines`.

Guarantees (E, P, B, K). Exact variants use rational arithmetic. Numerical variants use finite NumPy arithmetic at an explicit tolerance. Dense lifts, pseudoinverses, projections, direct sums, tensor products, and block assembly can require cubic time and quadratic storage.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. `tests/test_block18_frame_algebra.py` and the independent verifier for coordinate facades cover the applicable construction, roundtrip, certificate, failure, and persistence behavior. The source validator for coordinate facades also checks the exact signature, implementation identity, coordinate facade, anchor, and stable aliases. See Chapters 20, 48, and 64.

C.1C · CoordinateAtlas

Identity (q, a, t, k). Public research-manifest route `komorebi_math.CoordinateAtlas`. Reflected implementation `komorebi_math.coordinates.basis.CoordinateAtlas`. The same object is available through the curated stable routes `komorebi_math.stable.CoordinateAtlas`, `komorebi_math.v1.CoordinateAtlas`, and `komorebi_math.api.v1.CoordinateAtlas`. Those facade routes carry the frozen 1.x compatibility promise. The root route remains governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `class`. Profile `C1-COORD-BASIS`.

Manifest signature (s).

```
(name: 'str' = 'coordinate atlas') -> None
```

Definition. A graph of exact coordinate systems and composable transforms.

Contract (D, C, H, R, X). Systems or bases must be registered before edges are added. Edge dimensions and endpoint names must agree. `resolve` searches the finite graph and composes a compatible path, while conflicting edges, missing routes, or noncommuting path comparisons are reported explicitly. Principal public operations are `compare_paths`, `compose_path`, `direct_transform`, `register_basis`, `register_system`, `register_transform`, `resolve`, `systems`, `transforms`.

Guarantees (E, P, B, K). Structural arithmetic is exact over integers and `Fraction` values. No tolerance, accelerator, network, or symbolic backend is consulted. Runtime and memory are finite and are dominated by matrix materialization, exact elimination, nullspace work, recurrence propagation, or polynomial convolution appropriate to the operation. Dense elimination can require cubic arithmetic-operation cost and exact coefficients may grow.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. `tests/test_coordinates_general_basis.py`, `examples/general_coordinate_atlas.py`, and `verification/block14_general_coordinate_audit.py` cover the applicable construction, roundtrip, certificate, failure, and persistence behavior. The source validator for coordinate facades also checks the exact signature, implementation identity, coordinate facade, anchor, and stable aliases. See Chapters 15, 16, and 64.

C.1C · CoordinateBudget

Identity (q, a, t, k). Public research-manifest route `komorebi_math.CoordinateBudget`. Reflected implementation `komorebi_math.coordinates.budget.CoordinateBudget`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive` and is not part of the frozen stable compatibility promise. Manifest kind `class`. Profile `C1-COORD-BUDGET`.

Manifest signature (s).

```
(maximum_matrix_dimension: 'int' = 512, maximum_polynomial_degree: 'int' = 512, maximum_horizon: 'int' = 4096,
    ↪ maximum_exact_entries: 'int' = 1000000) -> None
```

Definition. Finite limits applied before coordinate matrices are materialized.

Contract (D, C, H, R, X). Every limit must be a positive integer. Guard methods reject negative requests as invalid and reject requests above the configured bound before materialization. Principal public operations are `require_degree`, `require_horizon`, `require_matrix`.

Guarantees (E, P, B, K). Checks are deterministic and constant-time in the requested dimensions. Passing a guard does not predict wall-clock time or prove that every permitted computation fits every host.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused coordinate suite and the independent verifier for coordinate facades cover the applicable construction, roundtrip, certificate, failure, and persistence behavior. The source validator for coordinate facades also checks the exact signature, implementation identity, coordinate facade, anchor, and stable aliases. See Chapters 15, 16, and 64.

C.1C · CoordinateSystem

Identity (q, a, t, k). Public research-manifest route `komorebi_math.CoordinateSystem`. Reflected implementation `komorebi_math.coordinates.basis.CoordinateSystem`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive` and is not part of the frozen stable compatibility promise. Manifest kind `class`. Profile `C1-COORD-BASIS`.

Manifest signature (s).

```
(name: 'str', dimension: 'int', kind: 'str' = 'exact', description: 'str' = '') -> None
```

Definition. Metadata for one finite exact coordinate system.

Contract (D, C, H, R, X). The name must contain nonwhitespace text and the dimension must be positive. Kind and description are metadata. The record does not by itself establish a basis or transform.

Guarantees (E, P, B, K). Structural arithmetic is exact over integers and `Fraction` values. No tolerance, accelerator, network, or symbolic backend is consulted. Runtime and memory are finite and are dominated by matrix materialization, exact elimination, nullspace work, recurrence propagation, or polynomial convolution appropriate to the operation. Dense elimination can require cubic arithmetic-operation cost and exact coefficients may grow.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. `tests/test_coordinates_general_basis.py`, `examples/general_coordinate_atlas.py`, and `verification/block14_general_coordinate_audit.py` cover the applicable construction, roundtrip, certificate, failure, and persistence behavior. The source validator for coordinate facades also checks the exact signature, implementation identity, coordinate facade, anchor, and stable aliases. See Chapters 15, 16, and 64.

C.1C · CoordinateTransform

Identity (q, a, t, k). Public research-manifest route `komorebi_math.CoordinateTransform`. Reflected implementation `komorebi_math.coordinates.transform.CoordinateTransform`. The same object is available through the curated stable routes `komorebi_math.stable.CoordinateTransform`, `komorebi_math.v1.CoordinateTransform`, and `komorebi_math.api.v1.CoordinateTransform`. Those facade routes carry the frozen 1.x compatibility promise. The root route remains governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `class`. Profile `C1-COORD-TRANSFORM`.

Manifest signature (s).

```
(source: 'str', target: 'str', matrix: 'Matrix', theorem: 'str', description: 'str' = '') -> None
```

Definition. An exact matrix that maps source coordinates to target coordinates.

Contract (D, C, H, R, X). Endpoint names must be nonempty and the exact matrix nonempty. Application and composition require dimension and endpoint compatibility. Inversion requires a square nonsingular matrix. Principal public operations are `apply`, `certificate`, `determinant`, `inverse`, `roundtrip_valid`, `source_dimension`, `square`, `target_dimension`, `then`.

Guarantees (E, P, B, K). Structural arithmetic is exact over integers and `Fraction` values. No tolerance, accelerator, network, or symbolic backend is consulted. Runtime and memory are finite and are dominated by matrix materialization, exact elimination, nullspace work, recurrence propagation, or polynomial convolution appropriate to the operation. Dense elimination can require cubic arithmetic-operation cost and exact coefficients may grow.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. `tests/test_coordinates_exact.py` and the independent verifier for coordinate facades cover the applicable construction, roundtrip, certificate, failure, and persistence behavior. The source validator for coordinate facades also checks the exact signature, implementation identity, coordinate facade, anchor, and stable aliases. See Chapters 15, 16, and 64.

C.1C · ExactAnalysisSynthesisMap

Identity (q, a, t, k). Public research-manifest route `komorebi_math.ExactAnalysisSynthesisMap`. Reflected implementation `komorebi_math.coordinates.frame_algebra.ExactAnalysisSynthesisMap`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive` and is not part of the frozen stable compatibility promise. Manifest kind `class`. Profile `C1-COORD-FRAME-ALGEBRA`.

Manifest signature (s).

```
(domain: 'ExactAnalysisSynthesisSystem', codomain: 'ExactAnalysisSynthesisSystem', operator: 'Matrix', theorem:
   ↪  'str' = 'KM-AS-EXACT-MAP-01') -> None
```

Definition. An exact rational physical map and coefficient lift.

Contract (D, C, H, R, X). The exact operator must map between compatible physical source spaces. Its coefficient lift, reconstruction, composition, adjoint, addition, scaling, and blocks are exact and endpoint-checked. Principal public operations are `add`, `adjoint`, `apply_coefficients`, `blocks`, `certificate`, `coefficient_lift`, `compose`, `dual_adjoint`, `reconstruct`, `scaled`, `then`.

Guarantees (E, P, B, K). Exact variants use rational arithmetic. Numerical variants use finite NumPy arithmetic at an explicit tolerance. Dense lifts, pseudoinverses, projections, direct sums, tensor products, and block assembly can require cubic time and quadratic storage.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. `tests/test_block18_frame_algebra.py` and the independent verifier for coordinate facades cover the applicable construction, roundtrip, certificate, failure, and persistence behavior. The source validator for coordinate facades also checks the exact signature, implementation identity, coordinate facade, anchor, and stable aliases. See Chapters 20, 48, and 64.

C.1C · ExactAnalysisSynthesisSystem

Identity (q, a, t, k). Public research-manifest route `komorebi_math.ExactAnalysisSynthesisSystem`. Reflected implementation `komorebi_math.coordinates.frame_algebra.ExactAnalysisSynthesisSystem`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive` and is not part of the frozen stable compatibility promise. Manifest kind `class`. Profile `C1-COORD-FRAME-ALGEBRA`.

Manifest signature (s).

```
(analysis: 'Matrix', synthesis: 'Matrix', source: 'str' = 'source', coordinates: 'str' = 'coordinates', theorem:
   ↪  'str' = 'KM-AS-EXACT-01') -> None
```

Definition. An exact rational analysis and synthesis presentation.

Contract (D, C, H, R, X). The exact matrices must have transposed rectangular shapes and satisfy the left-inverse law through the certificate or authoritative constructors. Gauge, dual, transition, direct-sum, and tensor-product operations use exact rational arithmetic. Principal public operations are `analyze`, `canonical_coefficients`, `certificate`, `coefficient_metric`, `coefficient_projection`, `coefficient_seminorm_squared`, `coordinate_dimension`, `direct_sum`, `dual`, `frame_operator`, `gauge_equivalent`, `gauge_part`, and related accessors.

Guarantees (E, P, B, K). Exact variants use rational arithmetic. Numerical variants use finite NumPy arithmetic at an explicit tolerance. Dense lifts, pseudoinverses, projections, direct sums, tensor products, and block assembly can require cubic time and quadratic storage.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. `tests/test_block18_frame_algebra.py` and the independent verifier for coordinate facades cover the applicable construction, roundtrip, certificate, failure, and persistence behavior. The source validator for coordinate facades also checks the exact signature, implementation identity, coordinate facade, anchor, and stable aliases. See Chapters 20, 48, and 64.

C.1C · FiniteBasis

Identity (q, a, t, k). Public research-manifest route `komorebi_math.FiniteBasis`. Reflected implementation `komorebi_math.coordinates.basis.FiniteBasis`. The same object is available through the curated stable routes `komorebi_math.stable.FiniteBasis`, `komorebi_math.v1.FiniteBasis`, and `komorebi_math.api.v1.FiniteBasis`. Those facade routes carry the frozen 1.x compatibility promise. The root route remains governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `class`. Profile `C1-COORD-BASIS`.

Manifest signature (s).

```
(name: 'str', vectors: 'tuple[Vector, ...]', labels: 'tuple[str, ...]' = (), description: 'str' = '') -> None
```

Definition. A linearly independent finite basis inside a rational ambient space.

Contract (D, C, H, R, X). Basis vectors are stored as columns, must share an ambient dimension, and must be linearly independent. Rectangular bases are permitted. Analysis requires membership in the spanned subspace, while synthesis, connections, dual rows, Gram matrices, and projectors use exact arithmetic. Principal public operations are `ambient_dimension`, `analyze`, `certificate`, `connection_to`, `contains`, `dimension`, `dual_rows`, `gram_matrix`, `matrix`, `orthogonal_projector`, `square`, `synthesize`.

Guarantees (E, P, B, K). Structural arithmetic is exact over integers and `Fraction` values. No tolerance, accelerator, network, or symbolic backend is consulted. Runtime and memory are finite and are dominated by matrix materialization, exact elimination, nullspace work, recurrence propagation, or polynomial convolution appropriate to the operation. Dense elimination can require cubic arithmetic-operation cost and exact coefficients may grow.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. `tests/test_coordinates_general_basis.py`, `examples/general_coordinate_atlas.py`, and `verification/block14_general_coordinate_audit.py` cover the applicable construction, roundtrip, certificate, failure, and persistence behavior. The source validator for coordinate facades also checks the exact signature, implementation identity, coordinate facade, anchor, and stable aliases. See Chapters 15, 16, and 64.

C.1C · FiniteFieldCoordinateTransform

Identity (q, a, t, k). Public research-manifest route `komorebi_math.FiniteFieldCoordinateTransform`. Reflected implementation `komorebi_math.coordinates.modular.FiniteFieldCoordinateTransform`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive` and is not part of the frozen stable compatibility promise. Manifest kind `class`. Profile `C1-COORD-MODULAR`.

Manifest signature (s).

```
(source: 'str', target: 'str', matrix: 'ModularMatrix', prime: 'int', theorem: 'str' = 'CC.M1', description: 'str'
    ↪  = '') -> None
```

Definition. An exact linear coordinate transform over `F_prime`.

Contract (D, C, H, R, X). The modulus must be prime, matrix entries are reduced into the finite field, and the square matrix must be invertible modulo that prime. Application, inverse, and composition reject incompatible dimensions or endpoints. Principal public operations are `apply`, `certificate`, `inverse`, `source_dimension`, `square`, `target_dimension`, `then`.

Guarantees (E, P, B, K). Structural arithmetic is exact over integers and `Fraction` values. No tolerance, accelerator, network, or symbolic backend is consulted. Runtime and memory are finite and are dominated by matrix materialization, exact elimination, nullspace work, recurrence propagation, or polynomial convolution appropriate to the operation. Dense elimination can require cubic arithmetic-operation cost and exact coefficients may grow.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. `tests/test_coordinates_general_tensor_modular.py` and `verification/block14_general_coordinate_audit.py` cover the applicable construction, roundtrip, certificate, failure, and persistence behavior. The source validator for coordinate facades also checks the exact signature, implementation identity, coordinate facade, anchor, and stable aliases. See Chapters 13, 16, and 64.

C.1C · FormalCoordinateChange

Identity (q, a, t, k). Public research-manifest route `komorebi_math.FormalCoordinateChange`. Reflected implementation `komorebi_math.coordinates.formal.FormalCoordinateChange`. The same object is available through the curated stable routes `komorebi_math.stable.FormalCoordinateChange`, `komorebi_math.v1.FormalCoordinateChange`, and `komorebi_math.api.v1.FormalCoordinateChange`. Those facade routes carry the frozen 1.x compatibility promise. The root route remains governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `class`. Profile `C1-COORD-FORMAL`.

Manifest signature (s).

```
(name: 'str', coefficients: 'Vector', source: 'str' = 'source formal coordinate', target: 'str' = 'target formal
   ↪ coordinate') -> None
```

Definition. A finite exact jet `phi(z)=a_1 z+a_2 z^2+...`.

Contract (D, C, H, R, X). The constant coefficient must vanish and the linear coefficient must be nonzero. Composition, inversion, truncation, and coefficient pushforward or pullback are performed modulo the first omitted power. Principal public operations are `certificate`, `compose`, `degree`, `inverse`, `linear_coefficient`, `pullback_coefficients`, `pushforward_coefficients`, `substitution_transform`, `truncate`.

Guarantees (E, P, B, K). Structural arithmetic is exact over integers and `Fraction` values. No tolerance, accelerator, network, or symbolic backend is consulted. Runtime and memory are finite and are dominated by matrix materialization, exact elimination, nullspace work, recurrence propagation, or polynomial convolution appropriate to the operation. Dense elimination can require cubic arithmetic-operation cost and exact coefficients may grow.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. `tests/test_coordinates_general_affine_formal.py` and `verification/block14_general_coordinate_audit.py` cover the applicable construction, roundtrip, certificate, failure, and persistence behavior. The source validator for coordinate facades also checks the exact signature, implementation identity, coordinate facade, anchor, and stable aliases. See Chapters 15, 16, and 64.

C.1C · IntertwinerSpace

Identity (q, a, t, k). Public research-manifest route `komorebi_math.IntertwinerSpace`. Reflected implementation `komorebi_math.coordinates.realization.IntertwinerSpace`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive` and is not part of the frozen stable compatibility promise. Manifest kind `class`. Profile `C1-COORD-REALIZATION`.

Manifest signature (s).

```
(source_dimension: 'int', target_dimension: 'int', basis: 'tuple[Matrix, ...]') -> None
```

Definition. Solutions of `B S=S A` from a source to a target realization.

Contract (D, C, H, R, X). The record stores source and target dimensions and a tuple of exact basis matrices. Direct construction does not validate the displayed basis. `certificate` checks matrix shapes and the equations `B S = S A`, and `intertwiner_space` is the authoritative solver route. Principal public operations are `certificate`, `contains_invertible_basis_element`, `dimension`.

Guarantees (E, P, B, K). Structural arithmetic is exact over integers and `Fraction` values. No tolerance, accelerator, network, or symbolic backend is consulted. Runtime and memory are finite and are dominated by matrix materialization, exact elimination, nullspace work, recurrence propagation, or polynomial convolution appropriate to the operation. Dense elimination can require cubic arithmetic-operation cost and exact coefficients may grow.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. `tests/test_coordinates_realization.py`, `examples/recurrence_state_coordinate_atlas.py`, and the independent verifier for coordinate facades cover the applicable construction, roundtrip, certificate, failure, and persistence behavior. The source validator for coordinate facades also checks the exact signature, implementation identity, coordinate facade, anchor, and stable aliases. See Chapters 18, 31, and 64.

C.1C · LinearRealization

Identity (q, a, t, k). Public research-manifest route `komorebi_math.LinearRealization`. Reflected implementation `komorebi_math.coordinates.realization.LinearRealization`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive` and is not part of the frozen stable compatibility promise. Manifest kind `class`. Profile `C1-COORD-REALIZATION`.

Manifest signature (s).

```
(transition: 'Matrix', initial: 'Vector', output: 'Vector', name: 'str' = 'linear realization') -> None
```

Definition. A scalar sequence realization `a_n=h^T A^n v`.

Contract (D, C, H, R, X). Matrices and vectors must have compatible exact dimensions. Observability and recovery are finite-horizon statements. Intertwiner solvers compute exact nullspaces, while record certificates verify displayed basis elements against the intertwining equation. Principal public operations are `dimension`, `prefix`, `state`, `term`.

Guarantees (E, P, B, K). Structural arithmetic is exact over integers and `Fraction` values. No tolerance, accelerator, network, or symbolic backend is consulted. Runtime and memory are finite and are dominated by matrix materialization, exact elimination, nullspace work, recurrence propagation, or polynomial convolution appropriate to the operation. Dense elimination can require cubic arithmetic-operation cost and exact coefficients may grow.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. `tests/test_coordinates_realization.py`, `examples/recurrence_state_coordinate_atlas.py`, and the independent verifier for coordinate facades cover the applicable construction, roundtrip, certificate, failure, and persistence behavior. The source validator for coordinate facades also checks the exact signature, implementation identity, coordinate facade, anchor, and stable aliases. See Chapters 18, 31, and 64.

C.1C · NumericCoordinateTransform

Identity (q, a, t, k). Public research-manifest route `komorebi_math.NumericCoordinateTransform`. Reflected implementation `komorebi_math.coordinates.numeric.NumericCoordinateTransform`. The same object is available through the curated stable routes `komorebi_math.stable.NumericCoordinateTransform`, `komorebi_math.v1.NumericCoordinateTransform`, and `komorebi_math.api.v1.NumericCoordinateTransform`. Those facade routes carry the frozen 1.x compatibility promise. The root route remains governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `class`. Profile `C1-COORD-NUMERIC`.

Manifest signature (s).

```
(source: 'str', target: 'str', matrix: 'np.ndarray', theorem: 'str' = 'CC.NM1', description: 'str' = '', tolerance:
    ↪  'float' = 1e-12) -> None
```

Definition. A finite numerical matrix between coordinate systems.

Contract (D, C, H, R, X). The input is a finite two-dimensional NumPy matrix with nonempty endpoint names and a positive finite tolerance. Inversion requires a square nonsingular matrix. Composition and certificates use tolerance-based numerical tests. Principal public operations are `apply`, `certificate`, `condition_number`, `inverse`, `source_dimension`, `square`, `target_dimension`, `then`.

Guarantees (E, P, B, K). The implementation uses finite NumPy arithmetic and reports tolerance-based residual evidence. Dense inversion or condition estimation can require cubic time and quadratic storage. No GPU or network backend is selected implicitly.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. `tests/test_coordinates_general_numeric_packet.py` and `verification/block14_general_coordinate_audit.py` cover the applicable construction, roundtrip, certificate, failure, and persistence behavior. The source validator for coordinate facades also checks the exact signature, implementation identity, coordinate facade, anchor, and stable aliases. See Chapters 16, 32, and 64.

C.1C · PolynomialCoordinateOperator

Identity (q, a, t, k). Public research-manifest route `komorebi_math.PolynomialCoordinateOperator`. Reflected implementation `komorebi_math.coordinates.calculus.PolynomialCoordinateOperator`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive` and is not part of the frozen stable compatibility promise. Manifest kind `class`. Profile `C1-COORD-CALCULUS`.

Manifest signature (s).

```
(name: 'str', source_family: 'str', target_family: 'str', source_degree: 'int', target_degree: 'int', matrix:
   ↪  'Matrix', theorem: 'str' = 'CC.O1') -> None
```

Definition. A finite polynomial operator matrix between graded family truncations.

Contract (D, C, H, R, X). The matrix shape must match the declared source and target degree truncations. Application and composition require compatible polynomial-family endpoints and use exact finite coefficient arithmetic. Principal public operations are `apply`, `certificate`, `compose`, `square`.

Guarantees (E, P, B, K). Structural arithmetic is exact over integers and `Fraction` values. No tolerance, accelerator, network, or symbolic backend is consulted. Runtime and memory are finite and are dominated by matrix materialization, exact elimination, nullspace work, recurrence propagation, or polynomial convolution appropriate to the operation. Dense elimination can require cubic arithmetic-operation cost and exact coefficients may grow.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. `tests/test_coordinates_general_calculus.py` and `examples/coordinate_calculus.py` cover the applicable construction, roundtrip, certificate, failure, and persistence behavior. The source validator for coordinate facades also checks the exact signature, implementation identity, coordinate facade, anchor, and stable aliases. See Chapters 17 and 64.

C.1C · PolynomialFamily

Identity (q, a, t, k). Public research-manifest route `komorebi_math.PolynomialFamily`. Reflected implementation `komorebi_math.coordinates.polynomials.PolynomialFamily`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive` and is not part of the frozen stable compatibility promise. Manifest kind `class`. Profile `C1-COORD-POLYNOMIALS`.

Manifest signature (s).

```
(name: 'str', rule: 'Callable[[int], Polynomial]') -> None
```

Definition. A graded polynomial basis with one nonzero leading term per degree.

Contract (D, C, H, R, X). The rule must produce one exact polynomial for each nonnegative degree with a nonzero leading coefficient. `polynomial` and `coefficient_matrix` materialize only the requested finite truncation. Principal public operations are `coefficient_matrix`, `polynomial`.

Guarantees (E, P, B, K). Structural arithmetic is exact over integers and `Fraction` values. No tolerance, accelerator, network, or symbolic backend is consulted. Runtime and memory are finite and are dominated by matrix materialization, exact elimination, nullspace work, recurrence propagation, or polynomial convolution appropriate to the operation. Dense elimination can require cubic arithmetic-operation cost and exact coefficients may grow.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. `tests/test_coordinates_polynomials.py` and `examples/euler_to_fibonacci_coordinates.py` cover the applicable construction, roundtrip, certificate, failure, and persistence behavior. The source validator for coordinate facades also checks the exact signature, implementation identity, coordinate facade, anchor, and stable aliases. See Chapters 17 and 64.

C.1C · RecurrenceCoordinateRecord

Identity (q, a, t, k). Public research-manifest route `komorebi_math.RecurrenceCoordinateRecord`. Reflected implementation `komorebi_math.coordinates.recurrence.RecurrenceCoordinateRecord`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive` and is not part of the frozen stable compatibility promise. Manifest kind `class`. Profile `C1-COORD-RECURRENCE`.

Manifest signature (s).

```
(name: 'str', coefficients: 'Vector', initial: 'Vector', defects: 'Vector') -> None
```

Definition. Initial state and forcing coordinates for one recurrence model.

Contract (D, C, H, R, X). The coefficient order and initial-state length must agree, and defects provide one forcing coordinate per later prefix value. Reconstruction, homogeneous propagation, conversion to `LinearRecurrence`, and the certificate all refer to that finite prefix. Principal public operations are `as_linear_recurrence`, `certificate`, `coordinates`, `homogeneous`, `length`, `order`, `reconstruct`.

Guarantees (E, P, B, K). Structural arithmetic is exact over integers and `Fraction` values. No tolerance, accelerator, network, or symbolic backend is consulted. Runtime and memory are finite and are dominated by matrix materialization, exact elimination, nullspace work, recurrence propagation, or polynomial convolution appropriate to the operation. Dense elimination can require cubic arithmetic-operation cost and exact coefficients may grow.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. `tests/test_coordinates_recurrence.py`, `examples/euler_to_fibonacci_coordinates.py`, and `verification/block14_general_coordinate_audit.py` cover the applicable construction, roundtrip, certificate, failure, and persistence behavior. The source validator for coordinate facades also checks the exact signature, implementation identity, coordinate facade, anchor, and stable aliases. See Chapters 18 and 64.

C.1C · RecurrenceModel

Identity (q, a, t, k). Public research-manifest route `komorebi_math.RecurrenceModel`. Reflected implementation `komorebi_math.coordinates.families.RecurrenceModel`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive` and is not part of the frozen stable compatibility promise. Manifest kind `class`. Profile `C1-COORD-FAMILIES`.

Manifest signature (s).

```
(name: 'str', coefficients: 'Vector', initial_values: 'Vector' = (), description: 'str' = '') -> None
```

Definition. A named constant-coefficient recurrence and optional initial state.

Contract (D, C, H, R, X). Coefficients must define positive order, the name must be nonempty, and optional initial values are either absent or have exactly that order. Synthesis requires a complete initial state. Sequence counts and finite transport lengths must be nonnegative. Principal public operations are `analyze`, `companion`, `coordinate_transform_to`, `homogeneous_basis`, `order`, `sequence`, `synthesize`.

Guarantees (E, P, B, K). Structural arithmetic is exact over integers and `Fraction` values. No tolerance, accelerator, network, or symbolic backend is consulted. Runtime and memory are finite and are dominated by matrix materialization, exact elimination, nullspace work, recurrence propagation, or polynomial convolution appropriate to the operation. Dense elimination can require cubic arithmetic-operation cost and exact coefficients may grow.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. `tests/test_coordinates_general_families_state.py` and `examples/recurrence_state_coordinate_atlas.py` cover the applicable construction, roundtrip, certificate, failure, and persistence behavior. The source validator for coordinate facades also checks the exact signature, implementation identity, coordinate facade, anchor, and stable aliases. See Chapters 18 and 64.

C.1C · ShiftPolynomial

Identity (q, a, t, k). Public research-manifest route `komorebi_math.ShiftPolynomial`. Reflected implementation `komorebi_math.coordinates.operators.ShiftPolynomial`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive` and is not part of the frozen stable compatibility promise. Manifest kind `class`. Profile `C1-COORD-OPERATORS`.

Manifest signature (s).

```
(coefficients: 'Iterable[Scalar]', name: 'str' = 'shift polynomial') -> 'None'
```

Definition. A polynomial `q(L)` in the canonical coefficient shift operator.

Contract (D, C, H, R, X). Coefficients are normalized to an exact finite polynomial in the canonical shift. Application requires enough input for the declared degree, and multiplication uses exact convolution. Principal public operations are `apply`, `degree`, `multiply`.

Guarantees (E, P, B, K). Structural arithmetic is exact over integers and `Fraction` values. No tolerance, accelerator, network, or symbolic backend is consulted. Runtime and memory are finite and are dominated by matrix materialization, exact elimination, nullspace work, recurrence propagation, or polynomial convolution appropriate to the operation. Dense elimination can require cubic arithmetic-operation cost and exact coefficients may grow.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. `tests/test_coordinates_operators.py` covers the applicable construction, roundtrip, certificate, failure, and persistence behavior. The source validator for coordinate facades also checks the exact signature, implementation identity, coordinate facade, anchor, and stable aliases. See Chapters 10, 18, and 64.

C.1C · StateRecovery

Identity (q, a, t, k). Public research-manifest route `komorebi_math.StateRecovery`. Reflected implementation `komorebi_math.coordinates.realization.StateRecovery`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive` and is not part of the frozen stable compatibility promise. Manifest kind `class`. Profile `C1-COORD-REALIZATION`.

Manifest signature (s).

```
(exact: 'bool', unique: 'bool', rank: 'int', horizon: 'int', state: 'Vector | None', predicted: 'Vector', residual:
    ↪ 'Vector') -> None
```

Definition. The result of state recovery over a finite horizon.

Contract (D, C, H, R, X). The record stores rank, horizon, predicted prefix, residual, and an optional recovered state. Direct construction stores fields. `recover_initial_state` is the authoritative computation and `certificate` checks the finite reconstruction statement. Principal public operations are `certificate`.

Guarantees (E, P, B, K). Structural arithmetic is exact over integers and `Fraction` values. No tolerance, accelerator, network, or symbolic backend is consulted. Runtime and memory are finite and are dominated by matrix materialization, exact elimination, nullspace work, recurrence propagation, or polynomial convolution appropriate to the operation. Dense elimination can require cubic arithmetic-operation cost and exact coefficients may grow.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. `tests/test_coordinates_realization.py`, `examples/recurrence_state_coordinate_atlas.py`, and the independent verifier for coordinate facades cover the applicable construction, roundtrip, certificate, failure, and persistence behavior. The source validator for coordinate facades also checks the exact signature, implementation identity, coordinate facade, anchor, and stable aliases. See Chapters 18, 31, and 64.

C.1C · `StateSpaceModel`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.StateSpaceModel`. Reflected implementation `komorebi_math.coordinates.state_space.StateSpaceModel`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive` and is not part of the frozen stable compatibility promise. Manifest kind `class`. Profile `C1-COORD-STATE-SPACE`.

Manifest signature (`s`).

```
(name: 'str', transition: 'Matrix', input_matrix: 'Matrix', output_matrix: 'Matrix', feedthrough: 'Matrix') -> None
```

Definition. A finite exact state-space model `x'=Ax+Bu, y=Cx+Du`.

Contract (`D, C, H, R, X`). The transition matrix is square. Input, output, and feedthrough shapes must match the state, input, and output dimensions. Coordinate changes require an invertible state transform. Markov and transfer comparisons are finite-horizon exact statements. Principal public operations are `change_coordinates`, `input_dimension`, `markov_parameter`, `output_dimension`, `state_dimension`, `transfer_equivalent`.

Guarantees (`E, P, B, K`). Structural arithmetic is exact over integers and `Fraction` values. No tolerance, accelerator, network, or symbolic backend is consulted. Runtime and memory are finite and are dominated by matrix materialization, exact elimination, nullspace work, recurrence propagation, or polynomial convolution appropriate to the operation. Dense elimination can require cubic arithmetic-operation cost and exact coefficients may grow.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this root export. `tests/test_coordinates_general_families_state.py` and `examples/recurrence_state_coordinate_atlas.py` cover the applicable construction, roundtrip, certificate, failure, and persistence behavior. The source validator for coordinate facades also checks the exact signature, implementation identity, coordinate facade, anchor, and stable aliases. See Chapters 18 and 64.

C.1C · `analyze_recurrence_coordinates`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.analyze_recurrence_coordinates`. Reflected implementation `komorebi_math.coordinates.recurrence.analyze_recurrence_coordinates`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive` and is not part of the frozen stable compatibility promise. Manifest kind `function`. Profile `C1-COORD-RECURRENCE`.

Manifest signature (`s`).

```
(values: 'Sequence[Scalar]', coefficients: 'RecurrenceLike', *, name: 'str' = 'recurrence') ->
    ↪ 'RecurrenceCoordinateRecord'
```

Definition. Analyze a finite sequence prefix in recurrence defect coordinates.

Contract (`D, C, H, R, X`). The recurrence coefficient vector must have positive order and the prefix must contain enough values for the requested analysis. Defect coordinates, companion evolution, inter-model transforms, and variation of constants use exact finite arithmetic.

Guarantees (`E, P, B, K`). Structural arithmetic is exact over integers and `Fraction` values. No tolerance, accelerator, network, or symbolic backend is consulted. Runtime and memory are finite and are dominated by matrix materialization, exact elimination, nullspace work, recurrence propagation, or polynomial convolution appropriate to the operation. Dense elimination can require cubic arithmetic-operation cost and exact coefficients may grow.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this root export. `tests/test_coordinates_recurrence.py`, `examples/euler_to_fibonacci_coordinates.py`, and `verification/block14_general_coordinate_audit.py` cover the applicable construction, roundtrip, certificate, failure, and persistence behavior. The source validator for coordinate facades also checks the exact signature, implementation identity, coordinate facade, anchor, and stable aliases. See Chapters 18 and 64.

C.1C · basis_from_matrix

Identity (q, a, t, k). Public research-manifest route `komorebi_math.basis_from_matrix`. Reflected implementation `komorebi_math.coordinates.basis.basis_from_matrix`. The same object is available through the curated stable routes `komorebi_math.stable.basis_from_matrix`, `komorebi_math.v1.basis_from_matrix`, and `komorebi_math.api.v1.basis_from_matrix`. Those facade routes carry the frozen 1.x compatibility promise. The root route remains governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C1-COORD-BASIS`.

Manifest signature (s).

```
(name: 'str', matrix: 'Iterable[Iterable[Scalar]]', *, labels: 'Sequence[str]' = (), description: 'str' = '') ->
    ↪ 'FiniteBasis'
```

Definition. Construct a basis from a matrix whose columns are basis vectors.

Contract (D, C, H, R, X). Dimensions and names must be valid, basis columns must be exact and linearly independent, and atlas edges must agree with registered endpoints. Resolution composes only compatible paths and reports missing or conflicting routes explicitly.

Guarantees (E, P, B, K). Structural arithmetic is exact over integers and `Fraction` values. No tolerance, accelerator, network, or symbolic backend is consulted. Runtime and memory are finite and are dominated by matrix materialization, exact elimination, nullspace work, recurrence propagation, or polynomial convolution appropriate to the operation. Dense elimination can require cubic arithmetic-operation cost and exact coefficients may grow.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. `tests/test_coordinates_general_basis.py`, `examples/general_coordinate_atlas.py`, and `verification/block14_general_coordinate_audit.py` cover the applicable construction, roundtrip, certificate, failure, and persistence behavior. The source validator for coordinate facades also checks the exact signature, implementation identity, coordinate facade, anchor, and stable aliases. See Chapters 15, 16, and 64.

C.1C · bezout_defect_identity

Identity (q, a, t, k). Public research-manifest route `komorebi_math.bezout_defect_identity`. Reflected implementation `komorebi_math.coordinates.operators.bezout_defect_identity`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive` and is not part of the frozen stable compatibility promise. Manifest kind `function`. Profile `C1-COORD-OPERATORS`.

Manifest signature (s).

```
(values: 'Sequence[Scalar]', left: 'Sequence[Scalar]', right: 'Sequence[Scalar]') -> 'BezoutDefectIdentity'
```

Definition. Reconstruct the common finite interior from two coprime defects.

Contract (D, C, H, R, X). Polynomial coefficients and finite prefixes are normalized to exact scalars. Shift application, multiplication, and Bezout reconstruction validate lengths and finite support. Computed certificate routes, not unchecked record construction, establish the displayed identity.

Guarantees (E, P, B, K). Structural arithmetic is exact over integers and `Fraction` values. No tolerance, accelerator, network, or symbolic backend is consulted. Runtime and memory are finite and are dominated by matrix materialization, exact elimination, nullspace work, recurrence propagation, or polynomial convolution appropriate to the operation. Dense elimination can require cubic arithmetic-operation cost and exact coefficients may grow.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. `tests/test_coordinates_operators.py` covers the applicable construction, roundtrip, certificate, failure, and persistence behavior. The source validator for coordinate facades also checks the exact signature, implementation identity, coordinate facade, anchor, and stable aliases. See Chapters 10, 18, and 64.

C.1C · `binomial_transform`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.binomial_transform`. Reflected implementation `komorebi_math.coordinates.combinatorial.binomial_transform`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive` and is not part of the frozen stable compatibility promise. Manifest kind `function`. Profile `C1-COORD-COMBINATORIAL`.

Manifest signature (`s`).

```
(size: 'int', *, inverse: 'bool' = False, source: 'str' = 'sequence values', target: 'str' = 'binomial transform',
    ↪ budget: 'CoordinateBudget' = CoordinateBudget(maximum_matrix_dimension=512, maximum_polynomial_degree=512,
    ↪ maximum_horizon=4096, maximum_exact_entries=1000000)) -> 'CoordinateTransform'
```

Definition. Return the finite binomial transform or its inverse.

Contract (`D, C, H, R, X`). Size must be nonnegative and remain within the supplied coordinate budget. The returned lower-triangular exact matrix follows the documented finite transform convention and exposes an exact inverse when requested.

Guarantees (`E, P, B, K`). Structural arithmetic is exact over integers and `Fraction` values. No tolerance, accelerator, network, or symbolic backend is consulted. Runtime and memory are finite and are dominated by matrix materialization, exact elimination, nullspace work, recurrence propagation, or polynomial convolution appropriate to the operation. Dense elimination can require cubic arithmetic-operation cost and exact coefficients may grow.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this root export. `tests/test_coordinates_general_combinatorial.py` and `verification/block14_general_coordinate_audit.py` cover the applicable construction, roundtrip, certificate, failure, and persistence behavior. The source validator for coordinate facades also checks the exact signature, implementation identity, coordinate facade, anchor, and stable aliases. See Chapters 17, 18, and 64.

C.1C · `classical_polynomial_catalog`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.classical_polynomial_catalog`. Reflected implementation `komorebi_math.coordinates.classical.classical_polynomial_catalog`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive` and is not part of the frozen stable compatibility promise. Manifest kind `function`. Profile `C1-COORD-CLASSICAL`.

Manifest signature (`s`).

```
() -> 'PolynomialFamilyCatalog'
```

Definition. Return the built-in exact graded polynomial-family catalog.

Contract (`D, C, H, R, X`). The function takes no arguments and returns the frozen `PolynomialFamilyCatalog` of built-in exact graded families and accepted aliases. The catalog is a lookup surface, not a proof of identities between families.

Guarantees (`E, P, B, K`). Structural arithmetic is exact over integers and `Fraction` values. No tolerance, accelerator, network, or symbolic backend is consulted. Runtime and memory are finite and are dominated by matrix materialization, exact elimination, nullspace work, recurrence propagation, or polynomial convolution appropriate to the operation. Dense elimination can require cubic arithmetic-operation cost and exact coefficients may grow.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this root export. `tests/test_coordinates_general_classical.py` and `examples/classical_coordinate_families.py` cover the applicable construction, roundtrip, certificate, failure, and persistence behavior. The source validator for coordinate facades also checks the exact signature, implementation identity, coordinate facade, anchor, and stable aliases. See Chapters 17 and 64.

C.1C · companion_matrix

Identity (q, a, t, k). Public research-manifest route `komorebi_math.companion_matrix`. Reflected implementation `komorebi_math.coordinates.recurrence.companion_matrix`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive` and is not part of the frozen stable compatibility promise. Manifest kind `function`. Profile `C1-COORD-RECURRENCE`.

Manifest signature (s).

```
(coefficients: 'RecurrenceLike') -> 'Matrix'
```

Definition. Return the exact companion matrix for consecutive sequence states.

Contract (D, C, H, R, X). The recurrence coefficient vector must have positive order and the prefix must contain enough values for the requested analysis. Defect coordinates, companion evolution, inter-model transforms, and variation of constants use exact finite arithmetic.

Guarantees (E, P, B, K). Structural arithmetic is exact over integers and `Fraction` values. No tolerance, accelerator, network, or symbolic backend is consulted. Runtime and memory are finite and are dominated by matrix materialization, exact elimination, nullspace work, recurrence propagation, or polynomial convolution appropriate to the operation. Dense elimination can require cubic arithmetic-operation cost and exact coefficients may grow.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. `tests/test_coordinates_recurrence.py`, `examples/euler_to_fibonacci_coordinates.py`, and `verification/block14_general_coordinate_audit.py` cover the applicable construction, roundtrip, certificate, failure, and persistence behavior. The source validator for coordinate facades also checks the exact signature, implementation identity, coordinate facade, anchor, and stable aliases. See Chapters 18 and 64.

C.1C · connection_transform

Identity (q, a, t, k). Public research-manifest route `komorebi_math.connection_transform`. Reflected implementation `komorebi_math.coordinates.polynomials.connection_transform`. The same object is available through the curated stable routes `komorebi_math.stable.connection_transform`, `komorebi_math.v1.connection_transform`, and `komorebi_math.api.v1.connection_transform`. Those facade routes carry the frozen 1.x compatibility promise. The root route remains governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C1-COORD-POLYNOMIALS`.

Manifest signature (s).

```
(source: 'PolynomialFamily', target: 'PolynomialFamily', degree: 'int', *, budget: 'CoordinateBudget' =
    ↪ CoordinateBudget(maximum_matrix_dimension=512, maximum_polynomial_degree=512, maximum_horizon=4096,
    ↪ maximum_exact_entries=1000000)) -> 'CoordinateTransform'
```

Definition. Return target coordinates of source basis vectors through one degree.

Contract (D, C, H, R, X). Families must be graded with a nonzero leading term in every requested degree. Connection and operator matrices are exact, degree-bounded, and subject to the coordinate budget. Endpoint mismatches are rejected.

Guarantees (E, P, B, K). Structural arithmetic is exact over integers and `Fraction` values. No tolerance, accelerator, network, or symbolic backend is consulted. Runtime and memory are finite and are dominated by matrix materialization, exact elimination, nullspace work, recurrence propagation, or polynomial convolution appropriate to the operation. Dense elimination can require cubic arithmetic-operation cost and exact coefficients may grow.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. `tests/test_coordinates_polynomials.py` and `examples/euler_to_fibonacci_coordinates.py` cover the applicable construction, roundtrip, certificate, failure, and persistence behavior. The source validator for coordinate facades also checks the exact signature, implementation identity, coordinate facade, anchor, and stable aliases. See Chapters 17 and 64.

C.1C · coordinate_change_between_recurrences

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinate_change_between_recurrences`. Reflected implementation `komorebi_math.coordinates.recurrence.coordinate_change_between_recurrences`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive` and is not part of the frozen stable compatibility promise. Manifest kind `function`. Profile `C1-COORD-RECURRENCE`.

Manifest signature (s).

```
(source_coefficients: 'RecurrenceLike', target_coefficients: 'RecurrenceLike', length: 'int', *, source_name:
    ↪ 'str' = 'source recurrence', target_name: 'str' = 'target recurrence', budget: 'CoordinateBudget' =
    ↪ CoordinateBudget(maximum_matrix_dimension=512, maximum_polynomial_degree=512, maximum_horizon=4096,
    ↪ maximum_exact_entries=1000000)) -> 'CoordinateTransform'
```

Definition. Return the exact finite coordinate map between recurrence models.

Contract (D, C, H, R, X). The recurrence coefficient vector must have positive order and the prefix must contain enough values for the requested analysis. Defect coordinates, companion evolution, inter-model transforms, and variation of constants use exact finite arithmetic.

Guarantees (E, P, B, K). Structural arithmetic is exact over integers and `Fraction` values. No tolerance, accelerator, network, or symbolic backend is consulted. Runtime and memory are finite and are dominated by matrix materialization, exact elimination, nullspace work, recurrence propagation, or polynomial convolution appropriate to the operation. Dense elimination can require cubic arithmetic-operation cost and exact coefficients may grow.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. `tests/test_coordinates_recurrence.py`, `examples/euler_to_fibonacci_coordinates.py`, and `verification/block14_general_coordinate_audit.py` cover the applicable construction, roundtrip, certificate, failure, and persistence behavior. The source validator for coordinate facades also checks the exact signature, implementation identity, coordinate facade, anchor, and stable aliases. See Chapters 18 and 64.

C.1C · discrete_fourier_transform

Identity (q, a, t, k). Public research-manifest route `komorebi_math.discrete_fourier_transform`. Reflected implementation `komorebi_math.coordinates.numeric.discrete_fourier_transform`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive` and is not part of the frozen stable compatibility promise. Manifest kind `function`. Profile `C1-COORD-NUMERIC`.

Manifest signature (s).

```
(size: 'int', *, unitary: 'bool' = True, inverse: 'bool' = False, tolerance: 'float' = 1e-12) ->
    ↪ 'NumericCoordinateTransform'
```

Definition. Return the finite discrete Fourier coordinate transform.

Contract (D, C, H, R, X). The input is a finite two-dimensional NumPy matrix with nonempty endpoint names and a positive finite tolerance. Inversion requires a square nonsingular matrix. Composition and certificates use tolerance-based numerical tests.

Guarantees (E, P, B, K). The implementation uses finite NumPy arithmetic and reports tolerance-based residual evidence. Dense inversion or condition estimation can require cubic time and quadratic storage. No GPU or network backend is selected implicitly.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. `tests/test_coordinates_general_numeric_packet.py` and `verification/block14_general_coordinate_audit.py` cover the applicable construction, roundtrip, certificate, failure, and persistence behavior. The source validator for coordinate facades also checks the exact signature, implementation identity, coordinate facade, anchor, and stable aliases. See Chapters 16, 32, and 64.

C.1C · `euler_fibonacci_coordinates`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.euler_fibonacci_coordinates`. Reflected implementation `komorebi_math.coordinates.recurrence.euler_fibonacci_coordinates`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive` and is not part of the frozen stable compatibility promise. Manifest kind `function`. Profile `C1-COORD-RECURRENCE`.

Manifest signature (`s`).

```
(count: 'int') -> 'RecurrenceCoordinateRecord'
```

Definition. Return exact Fibonacci companion coordinates of a signed Euler prefix.

Contract (`D, C, H, R, X`). The recurrence coefficient vector must have positive order and the prefix must contain enough values for the requested analysis. Defect coordinates, companion evolution, inter-model transforms, and variation of constants use exact finite arithmetic.

Guarantees (`E, P, B, K`). Structural arithmetic is exact over integers and `Fraction` values. No tolerance, accelerator, network, or symbolic backend is consulted. Runtime and memory are finite and are dominated by matrix materialization, exact elimination, nullspace work, recurrence propagation, or polynomial convolution appropriate to the operation. Dense elimination can require cubic arithmetic-operation cost and exact coefficients may grow.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this root export. `tests/test_coordinates_recurrence.py`, `examples/euler_to_fibonacci_coordinates.py`, and `verification/block14_general_coordinate_audit.py` cover the applicable construction, roundtrip, certificate, failure, and persistence behavior. The source validator for coordinate facades also checks the exact signature, implementation identity, coordinate facade, anchor, and stable aliases. See Chapters 18 and 64.

C.1C · `euler_to_fibonacci_transform`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.euler_to_fibonacci_transform`. Reflected implementation `komorebi_math.coordinates.polynomials.euler_to_fibonacci_transform`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive` and is not part of the frozen stable compatibility promise. Manifest kind `function`. Profile `C1-COORD-POLYNOMIALS`.

Manifest signature (`s`).

```
(degree: 'int', *, budget: 'CoordinateBudget' = CoordinateBudget(maximum_matrix_dimension=512,
    ↪ maximum_polynomial_degree=512, maximum_horizon=4096, maximum_exact_entries=1000000)) ->
    ↪ 'CoordinateTransform'
```

Definition. Return the Euler polynomial to Fibonacci polynomial connection.

Contract (`D, C, H, R, X`). Families must be graded with a nonzero leading term in every requested degree. Connection and operator matrices are exact, degree-bounded, and subject to the coordinate budget. Endpoint mismatches are rejected.

Guarantees (`E, P, B, K`). Structural arithmetic is exact over integers and `Fraction` values. No tolerance, accelerator, network, or symbolic backend is consulted. Runtime and memory are finite and are dominated by matrix materialization, exact elimination, nullspace work, recurrence propagation, or polynomial convolution appropriate to the operation. Dense elimination can require cubic arithmetic-operation cost and exact coefficients may grow.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this root export. `tests/test_coordinates_polynomials.py` and `examples/euler_to_fibonacci_coordinates.py` cover the applicable construction, roundtrip, certificate, failure, and persistence behavior. The source validator for coordinate facades also checks the exact signature, implementation identity, coordinate facade, anchor, and stable aliases. See Chapters 17 and 64.

C.1C · forward_difference_transform

Identity (q, a, t, k). Public research-manifest route `komorebi_math.forward_difference_transform`. Reflected implementation `komorebi_math.coordinates.combinatorial.forward_difference_transform`. The same object is available through the curated stable routes `komorebi_math.stable.forward_difference_transform`, `komorebi_math.v1.forward_difference_transform`, and `komorebi_math.api.v1.forward_difference_transform`. Those facade routes carry the frozen 1.x compatibility promise. The root route remains governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C1-COORD-COMBINATORIAL`.

Manifest signature (s).

```
(size: 'int', *, budget: 'CoordinateBudget' = CoordinateBudget(maximum_matrix_dimension=512,
    ↪ maximum_polynomial_degree=512, maximum_horizon=4096, maximum_exact_entries=1000000)) ->
    ↪ 'CoordinateTransform'
```

Definition. Map sampled values `f(0),...,f(N)` to forward differences at zero.

Contract (D, C, H, R, X). Size must be nonnegative and remain within the supplied coordinate budget. The returned lower-triangular exact matrix follows the documented finite transform convention and exposes an exact inverse when requested.

Guarantees (E, P, B, K). Structural arithmetic is exact over integers and `Fraction` values. No tolerance, accelerator, network, or symbolic backend is consulted. Runtime and memory are finite and are dominated by matrix materialization, exact elimination, nullspace work, recurrence propagation, or polynomial convolution appropriate to the operation. Dense elimination can require cubic arithmetic-operation cost and exact coefficients may grow.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. `tests/test_coordinates_general_combinatorial.py` and `verification/block14_general_coordinate_audit.py` cover the applicable construction, roundtrip, certificate, failure, and persistence behavior. The source validator for coordinate facades also checks the exact signature, implementation identity, coordinate facade, anchor, and stable aliases. See Chapters 17, 18, and 64.

C.1C · get_polynomial_family

Identity (q, a, t, k). Public research-manifest route `komorebi_math.get_polynomial_family`. Reflected implementation `komorebi_math.coordinates.classical.get_polynomial_family`. The same object is available through the curated stable routes `komorebi_math.stable.get_polynomial_family`, `komorebi_math.v1.get_polynomial_family`, and `komorebi_math.api.v1.get_polynomial_family`. Those facade routes carry the frozen 1.x compatibility promise. The root route remains governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C1-COORD-CLASSICAL`.

Manifest signature (s).

```
(name: 'str') -> 'PolynomialFamily'
```

Definition. Return one built-in graded polynomial family by name or short alias.

Contract (D, C, H, R, X). Names are normalized against the built-in catalog. Unknown names are rejected, and returned families remain exact graded bases rather than numerical approximations.

Guarantees (E, P, B, K). Structural arithmetic is exact over integers and `Fraction` values. No tolerance, accelerator, network, or symbolic backend is consulted. Runtime and memory are finite and are dominated by matrix materialization, exact elimination, nullspace work, recurrence propagation, or polynomial convolution appropriate to the operation. Dense elimination can require cubic arithmetic-operation cost and exact coefficients may grow.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. `tests/test_coordinates_general_classical.py` and `examples/classical_coordinate_families.py` cover the applicable construction, roundtrip, certificate, failure, and persistence behavior. The source validator for coordinate facades also checks the exact signature, implementation identity, coordinate facade, anchor, and stable aliases. See Chapters 17 and 64.

C.1C · get_recurrence_model

Identity (q, a, t, k). Public research-manifest route `komorebi_math.get_recurrence_model`. Reflected implementation `komorebi_math.coordinates.families.get_recurrence_model`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive` and is not part of the frozen stable compatibility promise. Manifest kind `function`. Profile `C1-COORD-FAMILIES`.

Manifest signature (s).

```
(name: 'str') -> 'RecurrenceModel'
```

Definition. Return one built-in recurrence model by normalized name.

Contract (D, C, H, R, X). A recurrence must have positive order and a nonempty name. Initial values are either absent or provide exactly one complete state. Analysis, synthesis, catalog lookup, and inter-model transport operate on finite exact prefixes.

Guarantees (E, P, B, K). Structural arithmetic is exact over integers and `Fraction` values. No tolerance, accelerator, network, or symbolic backend is consulted. Runtime and memory are finite and are dominated by matrix materialization, exact elimination, nullspace work, recurrence propagation, or polynomial convolution appropriate to the operation. Dense elimination can require cubic arithmetic-operation cost and exact coefficients may grow.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. `tests/test_coordinates_general_families_state.py` and `examples/recurrence_state_coordinate_atlas.py` cover the applicable construction, roundtrip, certificate, failure, and persistence behavior. The source validator for coordinate facades also checks the exact signature, implementation identity, coordinate facade, anchor, and stable aliases. See Chapters 18 and 64.

C.1C · intertwiner_space

Identity (q, a, t, k). Public research-manifest route `komorebi_math.intertwiner_space`. Reflected implementation `komorebi_math.coordinates.realization.intertwiner_space`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive` and is not part of the frozen stable compatibility promise. Manifest kind `function`. Profile `C1-COORD-REALIZATION`.

Manifest signature (s).

```
(source: 'Matrix', target: 'Matrix') -> 'IntertwinerSpace'
```

Definition. Return an exact basis of matrices satisfying `target*S=S*source`.

Contract (D, C, H, R, X). Matrices and vectors must have compatible exact dimensions. Observability and recovery are finite-horizon statements. Intertwiner solvers compute exact nullspaces, while record certificates verify displayed basis elements against the intertwining equation.

Guarantees (E, P, B, K). Structural arithmetic is exact over integers and `Fraction` values. No tolerance, accelerator, network, or symbolic backend is consulted. Runtime and memory are finite and are dominated by matrix materialization, exact elimination, nullspace work, recurrence propagation, or polynomial convolution appropriate to the operation. Dense elimination can require cubic arithmetic-operation cost and exact coefficients may grow.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. `tests/test_coordinates_realization.py`, `examples/recurrence_state_coordinate_atlas.py`, and the independent verifier for coordinate facades cover the applicable construction, roundtrip, certificate, failure, and persistence behavior. The source validator for coordinate facades also checks the exact signature, implementation identity, coordinate facade, anchor, and stable aliases. See Chapters 18, 31, and 64.

C.1C · load_coordinate_object

Identity (q, a, t, k). Public research-manifest route `komorebi_math.load_coordinate_object`. Reflected implementation `komorebi_math.coordinates.portable.load_coordinate_object`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive` and is not part of the frozen stable compatibility promise. Manifest kind `function`. Profile `C1-COORD-PORTABLE`.

Manifest signature (s).

```
(path: 'str | Path') -> 'Any'
```

Definition. Load and verify one registered portable coordinate object.

Contract (D, C, H, R, X). The path must contain a registered canonical coordinate envelope. The loader verifies JSON structure, type tag, schema version, and digest before reconstructing the object. Tampered, truncated, unknown, or unsupported records are rejected.

Guarantees (E, P, B, K). Encoding and digest verification are deterministic for the same registered value and schema version. Integrity is checked, but the digest is not an authentication signature. Runtime and memory are linear in encoded payload size apart from reconstruction of contained matrices.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. `tests/test_coordinates_general_persistence.py` and `tests/test_coordinates_normalized_persistence.py` cover the applicable construction, roundtrip, certificate, failure, and persistence behavior. The source validator for coordinate facades also checks the exact signature, implementation identity, coordinate facade, anchor, and stable aliases. See Chapters 61 and 64.

C.1C · normalization_change_transform

Identity (q, a, t, k). Public research-manifest route `komorebi_math.normalization_change_transform`. Reflected implementation `komorebi_math.coordinates.normalized.normalization_change_transform`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive` and is not part of the frozen stable compatibility promise. Manifest kind `function`. Profile `C1-COORD-NORMALIZED`.

Manifest signature (s).

```
(source: 'WeightedMonomialBasis', target: 'WeightedMonomialBasis', degree: 'int', *, budget: 'CoordinateBudget' =
    ↪ CoordinateBudget(maximum_matrix_dimension=512, maximum_polynomial_degree=512, maximum_horizon=4096,
    ↪ maximum_exact_entries=1000000)) -> 'CoordinateTransform'
```

Definition. Return the diagonal map preserving the underlying formal series.

Contract (D, C, H, R, X). Source and target weighted monomial bases must define compatible nonzero weights through the requested degree. The diagonal map preserves the represented formal series and remains subject to the coordinate budget.

Guarantees (E, P, B, K). Structural arithmetic is exact over integers and `Fraction` values. No tolerance, accelerator, network, or symbolic backend is consulted. Runtime and memory are finite and are dominated by matrix materialization, exact elimination, nullspace work, recurrence propagation, or polynomial convolution appropriate to the operation. Dense elimination can require cubic arithmetic-operation cost and exact coefficients may grow.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. `tests/test_coordinates_normalized_persistence.py` covers the applicable construction, roundtrip, certificate, failure, and persistence behavior. The source validator for coordinate facades also checks the exact signature, implementation identity, coordinate facade, anchor, and stable aliases. See Chapters 17, 21, and 64.

C.1C · observability_matrix

Identity (q, a, t, k). Public research-manifest route `komorebi_math.observability_matrix`. Reflected implementation `komorebi_math.coordinates.realization.observability_matrix`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive` and is not part of the frozen stable compatibility promise. Manifest kind `function`. Profile `C1-COORD-REALIZATION`.

Manifest signature (s).

```
(transition: 'Matrix', output: 'Sequence[Scalar]', horizon: 'int', *, budget: 'CoordinateBudget' =
    ↪ CoordinateBudget(maximum_matrix_dimension=512, maximum_polynomial_degree=512, maximum_horizon=4096,
    ↪ maximum_exact_entries=1000000)) -> 'Matrix'
```

Definition. Return rows `h^T A^n` for `0 <= n < horizon`.

Contract (D, C, H, R, X). Matrices and vectors must have compatible exact dimensions. Observability and recovery are finite-horizon statements. Intertwiner solvers compute exact nullspaces, while record certificates verify displayed basis elements against the intertwining equation.

Guarantees (E, P, B, K). Structural arithmetic is exact over integers and `Fraction` values. No tolerance, accelerator, network, or symbolic backend is consulted. Runtime and memory are finite and are dominated by matrix materialization, exact elimination, nullspace work, recurrence propagation, or polynomial convolution appropriate to the operation. Dense elimination can require cubic arithmetic-operation cost and exact coefficients may grow.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. `tests/test_coordinates_realization.py`, `examples/recurrence_state_coordinate_atlas.py`, and the independent verifier for coordinate facades cover the applicable construction, roundtrip, certificate, failure, and persistence behavior. The source validator for coordinate facades also checks the exact signature, implementation identity, coordinate facade, anchor, and stable aliases. See Chapters 18, 31, and 64.

C.1C · operator_change_of_coordinates

Identity (q, a, t, k). Public research-manifest route `komorebi_math.operator_change_of_coordinates`. Reflected implementation `komorebi_math.coordinates.polynomials.operator_change_of_coordinates`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive` and is not part of the frozen stable compatibility promise. Manifest kind `function`. Profile `C1-COORD-POLYNOMIALS`.

Manifest signature (s).

```
(operator_in_source: 'Matrix', source_to_target: 'CoordinateTransform') -> 'Matrix'
```

Definition. Conjugate an operator from source coordinates into target coordinates.

Contract (D, C, H, R, X). Families must be graded with a nonzero leading term in every requested degree. Connection and operator matrices are exact, degree-bounded, and subject to the coordinate budget. Endpoint mismatches are rejected.

Guarantees (E, P, B, K). Structural arithmetic is exact over integers and `Fraction` values. No tolerance, accelerator, network, or symbolic backend is consulted. Runtime and memory are finite and are dominated by matrix materialization, exact elimination, nullspace work, recurrence propagation, or polynomial convolution appropriate to the operation. Dense elimination can require cubic arithmetic-operation cost and exact coefficients may grow.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. `tests/test_coordinates_polynomials.py` and `examples/euler_to_fibonacci_coordinates.py` cover the applicable construction, roundtrip, certificate, failure, and persistence behavior. The source validator for coordinate facades also checks the exact signature, implementation identity, coordinate facade, anchor, and stable aliases. See Chapters 17 and 64.

C.1C · recover_initial_state

Identity (q, a, t, k). Public research-manifest route `komorebi_math.recover_initial_state`. Reflected implementation `komorebi_math.coordinates.realization.recover_initial_state`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive` and is not part of the frozen stable compatibility promise. Manifest kind `function`. Profile `C1-COORD-REALIZATION`.

Manifest signature (s).

```
(prefix: 'Sequence[Scalar]', transition: 'Matrix', output: 'Sequence[Scalar]', *, budget: 'CoordinateBudget' =
    ↪ CoordinateBudget(maximum_matrix_dimension=512, maximum_polynomial_degree=512, maximum_horizon=4096,
    ↪ maximum_exact_entries=1000000)) -> 'StateRecovery'
```

Definition. Recover or test a state against a finite output prefix.

Contract (D, C, H, R, X). Matrices and vectors must have compatible exact dimensions. Observability and recovery are finite-horizon statements. Intertwiner solvers compute exact nullspaces, while record certificates verify displayed basis elements against the intertwining equation.

Guarantees (E, P, B, K). Structural arithmetic is exact over integers and `Fraction` values. No tolerance, accelerator, network, or symbolic backend is consulted. Runtime and memory are finite and are dominated by matrix materialization, exact elimination, nullspace work, recurrence propagation, or polynomial convolution appropriate to the operation. Dense elimination can require cubic arithmetic-operation cost and exact coefficients may grow.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. `tests/test_coordinates_realization.py`, `examples/recurrence_state_coordinate_atlas.py`, and the independent verifier for coordinate facades cover the applicable construction, roundtrip, certificate, failure, and persistence behavior. The source validator for coordinate facades also checks the exact signature, implementation identity, coordinate facade, anchor, and stable aliases. See Chapters 18, 31, and 64.

C.1C · recurrence_model_catalog

Identity (q, a, t, k). Public research-manifest route `komorebi_math.recurrence_model_catalog`. Reflected implementation `komorebi_math.coordinates.families.recurrence_model_catalog`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive` and is not part of the frozen stable compatibility promise. Manifest kind `function`. Profile `C1-COORD-FAMILIES`.

Manifest signature (s).

```
() -> 'tuple[RecurrenceModel, ...]'
```

Definition. Return the built-in named recurrence models.

Contract (D, C, H, R, X). A recurrence must have positive order and a nonempty name. Initial values are either absent or provide exactly one complete state. Analysis, synthesis, catalog lookup, and inter-model transport operate on finite exact prefixes.

Guarantees (E, P, B, K). Structural arithmetic is exact over integers and `Fraction` values. No tolerance, accelerator, network, or symbolic backend is consulted. Runtime and memory are finite and are dominated by matrix materialization, exact elimination, nullspace work, recurrence propagation, or polynomial convolution appropriate to the operation. Dense elimination can require cubic arithmetic-operation cost and exact coefficients may grow.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. `tests/test_coordinates_general_families_state.py` and `examples/recurrence_state_coordinate_atlas.py` cover the applicable construction, roundtrip, certificate, failure, and persistence behavior. The source validator for coordinate facades also checks the exact signature, implementation identity, coordinate facade, anchor, and stable aliases. See Chapters 18 and 64.

C.1C · residue_class_transform

Identity (q, a, t, k). Public research-manifest route `komorebi_math.residue_class_transform`. Reflected implementation `komorebi_math.coordinates.packet.residue_class_transform`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive` and is not part of the frozen stable compatibility promise. Manifest kind `function`. Profile `C1-COORD-PACKET`.

Manifest signature (s).

```
(length: 'int', modulus: 'int') -> 'CoordinateTransform'
```

Definition. Return the exact polyphase permutation into residue-lane order.

Contract (D, C, H, R, X). Length must be nonnegative and modulus positive. The result is an exact permutation between natural order and residue-lane order, with dimensions and endpoints recorded explicitly.

Guarantees (E, P, B, K). Structural arithmetic is exact over integers and `Fraction` values. No tolerance, accelerator, network, or symbolic backend is consulted. Runtime and memory are finite and are dominated by matrix materialization, exact elimination, nullspace work, recurrence propagation, or polynomial convolution appropriate to the operation. Dense elimination can require cubic arithmetic-operation cost and exact coefficients may grow.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. `tests/test_coordinates_general_numeric_packet.py` and `verification/block14_general_coordinate_audit.py` cover the applicable construction, roundtrip, certificate, failure, and persistence behavior. The source validator for coordinate facades also checks the exact signature, implementation identity, coordinate facade, anchor, and stable aliases. See Chapters 13, 18, and 64.

C.1C · save_coordinate_object

Identity (q, a, t, k). Public research-manifest route `komorebi_math.save_coordinate_object`. Reflected implementation `komorebi_math.coordinates.portable.save_coordinate_object`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive` and is not part of the frozen stable compatibility promise. Manifest kind `function`. Profile `C1-COORD-PORTABLE`.

Manifest signature (s).

```
(path: 'str | Path', value: 'Any', *, overwrite: 'bool' = False) -> 'Path'
```

Definition. Save any registered portable coordinate object.

Contract (D, C, H, R, X). The value must have a registered portable coordinate codec. Existing paths are refused unless overwrite is true. The function writes canonical JSON through a temporary file and atomic replacement, then returns the resolved path.

Guarantees (E, P, B, K). Encoding and digest verification are deterministic for the same registered value and schema version. Integrity is checked, but the digest is not an authentication signature. Runtime and memory are linear in encoded payload size apart from reconstruction of contained matrices.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. `tests/test_coordinates_general_persistence.py` and `tests/test_coordinates_normalized_persistence.py` cover the applicable construction, roundtrip, certificate, failure, and persistence behavior. The source validator for coordinate facades also checks the exact signature, implementation identity, coordinate facade, anchor, and stable aliases. See Chapters 61 and 64.

C.1C · standard_basis

Identity (q, a, t, k). Public research-manifest route `komorebi_math.standard_basis`. Reflected implementation `komorebi_math.coordinates.basis.standard_basis`. The same object is available through the curated stable routes `komorebi_math.stable.standard_basis`, `komorebi_math.v1.standard_basis`, and `komorebi_math.api.v1.standard_basis`. Those facade routes carry the frozen 1.x compatibility promise. The root route remains governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C1-COORD-BASIS`.

Manifest signature (s).

```
(dimension: 'int', *, name: 'str' = 'standard') -> 'FiniteBasis'
```

Definition. Return the standard basis of a rational vector space.

Contract (D, C, H, R, X). Dimensions and names must be valid, basis columns must be exact and linearly independent, and atlas edges must agree with registered endpoints. Resolution composes only compatible paths and reports missing or conflicting routes explicitly.

Guarantees (E, P, B, K). Structural arithmetic is exact over integers and `Fraction` values. No tolerance, accelerator, network, or symbolic backend is consulted. Runtime and memory are finite and are dominated by matrix materialization, exact elimination, nullspace work, recurrence propagation, or polynomial convolution appropriate to the operation. Dense elimination can require cubic arithmetic-operation cost and exact coefficients may grow.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. `tests/test_coordinates_general_basis.py`, `examples/general_coordinate_atlas.py`, and `verification/block14_general_coordinate_audit.py` cover the applicable construction, roundtrip, certificate, failure, and persistence behavior. The source validator for coordinate facades also checks the exact signature, implementation identity, coordinate facade, anchor, and stable aliases. See Chapters 15, 16, and 64.

C.1C · `tensor_coordinate_transform`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.tensor_coordinate_transform`. Reflected implementation `komorebi_math.coordinates.tensor.tensor_coordinate_transform`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive` and is not part of the frozen stable compatibility promise. Manifest kind `function`. Profile `C1-COORD-TENSOR`.

Manifest signature (s).

```
(transform: 'CoordinateTransform', variance: 'Sequence[Variance]') -> 'CoordinateTransform'
```

Definition. Return the induced transform on flattened tensor components.

Contract (D, C, H, R, X). Variance labels must be valid and the tensor rank is the length of that declaration. Contravariant axes use the coordinate matrix and covariant axes use its contragredient, with lexicographic flattening.

Guarantees (E, P, B, K). Structural arithmetic is exact over integers and `Fraction` values. No tolerance, accelerator, network, or symbolic backend is consulted. Runtime and memory are finite and are dominated by matrix materialization, exact elimination, nullspace work, recurrence propagation, or polynomial convolution appropriate to the operation. Dense elimination can require cubic arithmetic-operation cost and exact coefficients may grow.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. `tests/test_coordinates_general_tensor_modular.py` and `verification/block14_general_coordinate_audit.py` cover the applicable construction, roundtrip, certificate, failure, and persistence behavior. The source validator for coordinate facades also checks the exact signature, implementation identity, coordinate facade, anchor, and stable aliases. See Chapters 16, 54, and 64.

C.1C · `variation_of_constants_prefix`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.variation_of_constants_prefix`. Reflected implementation `komorebi_math.coordinates.recurrence.variation_of_constants_prefix`. No curated stable or beta facade exports this research name in the reference manifest. The root route is governed by `[RESEARCH]` policy `pre-beta-additive` and is not part of the frozen stable compatibility promise. Manifest kind `function`. Profile `C1-COORD-RECURRENCE`.

Manifest signature (`s`).

```
(record: 'RecurrenceCoordinateRecord') -> 'Vector'
```

Definition. Reconstruct a prefix through the companion Green-kernel formula.

Contract (`D, C, H, R, X`). The recurrence coefficient vector must have positive order and the prefix must contain enough values for the requested analysis. Defect coordinates, companion evolution, inter-model transforms, and variation of constants use exact finite arithmetic.

Guarantees (`E, P, B, K`). Structural arithmetic is exact over integers and `Fraction` values. No tolerance, accelerator, network, or symbolic backend is consulted. Runtime and memory are finite and are dominated by matrix materialization, exact elimination, nullspace work, recurrence propagation, or polynomial convolution appropriate to the operation. Dense elimination can require cubic arithmetic-operation cost and exact coefficients may grow.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this root export. `tests/test_coordinates_recurrence.py`, `examples/euler_to_fibonacci_coordinates.py`, and `verification/block14_general_coordinate_audit.py` cover the applicable construction, roundtrip, certificate, failure, and persistence behavior. The source validator for coordinate facades also checks the exact signature, implementation identity, coordinate facade, anchor, and stable aliases. See Chapters 18 and 64.

C.1D Root core, structured algorithms, and exact integration: 82 entries

This root section documents three connected layers used throughout the manual. The core layer supplies typed operator grammars, finite realizations, packet decompositions, transfer graphs, morphisms, and seed-rank calculations. The algorithm layer turns those structures into packet intermediate representations, structured and low-rank execution, finite tree compilation, precision allocation, Krylov solves, discovery routines, and schedules. The integration layer provides exact measures, sparse polynomials, quadrature rules, finite error records, integral identities, and coordinate covectors.

Contract family	Cards
Core operator systems	25
Structured algorithms	19
Exact integration	38
Total	**82**

The boundary remains finite. Exact certificates apply only when the supplied scalar domain and hypotheses are exact. Numerical backends retain tolerances and residuals. Discovery routines remain heuristic, and callable integration or pushforward paths preserve their unchecked regularity assumptions. Dense materialization, rank tests, singular-value decompositions, path enumeration, and tensor or product growth remain explicit resource boundaries.

Core operator systems (25 cards).

C.1D · `AlgebraBasisResult`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.AlgebraBasisResult`. Reflected implementation `komorebi_math.core.algebra.AlgebraBasisResult`. The same runtime object is also available through `komorebi_math.core.AlgebraBasisResult`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1D-CORE-ALGEBRA`.

Manifest signature (s).

```
(basis: 'tuple[Any, ...]', dimension: 'int', certificate: 'Certificate') -> None
```

Definition. A finite basis, its dimension, and the certificate returned by operator-algebra closure.

Contract (D, C, H, R, X). Generator matrices must act on one finite coordinate space. The selected backend validates common square shape and closes the span under multiplication, optionally adjoining the identity. Exact backends use exact dependence tests. Floating backends use the declared tolerance and therefore return numerical certificates. `matrix_units` requires a positive size, and `AlgebraBasisResult` records the resulting basis, dimension, and certificate without recomputing closure.

Guarantees (E, P, B, K). The generated algebra has dimension at most the square of the matrix dimension, so closure terminates after finitely many independent additions. Matrix products and rank tests dominate cost. Exact scalar growth can dominate exact runs, while conditioning and tolerance selection control floating runs. A numerical containment decision is not promoted to an exact algebraic identity.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_algebra_quotients.py` and `tests/test_block18_frame_algebra.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapters 10 and 64.

C.1D · Arrow

Identity (q, a, t, k). Public research-manifest route `komorebi_math.Arrow`. Reflected implementation `komorebi_math.core.grammar.Arrow`. The same runtime object is also available through `komorebi_math.core.Arrow`, `komorebi_math.stable.Arrow`, `komorebi_math.v1.Arrow`, `komorebi_math.api.v1.Arrow`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1D-CORE-GRAMMAR`.

Manifest signature (s).

```
(name: 'str', source: 'str', target: 'str') -> None
```

Definition. A named typed generator from one grammar vertex to another.

Contract (D, C, H, R, X). Vertices and arrow names must be nonempty and unique, every arrow endpoint must be declared, and each operator word must be composable in application order. Linear relations may combine only parallel words. A realization supplies one positive dimension per vertex and a matrix of the required target-by-source shape for every arrow. Relation validation either produces a certificate or fails closed.

Guarantees (E, P, B, K). Grammar formation is finite and exact at the structural level. Path enumeration is bounded by the requested length and can grow exponentially with branching. Realization evaluation uses the selected linear algebra backend. Exact matrices yield exact relation checks, while floating matrices yield tolerance-dependent residual claims.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_grammar_realization.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapters 9, 10, and 64.

C.1D · OperatorAlgebra

Identity (q, a, t, k). Public research-manifest route `komorebi_math.OperatorAlgebra`. Reflected implementation `komorebi_math.core.algebra.OperatorAlgebra`. The same runtime object is also available through `komorebi_math.core.OperatorAlgebra`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1D-CORE-ALGEBRA`.

Manifest signature (s).

```
(generators: 'Mapping[str, Any] | Sequence[Any]', *, backend: 'str | LinearAlgebraBackend | None' = None,
   ↪ tolerance: 'float | None' = None, include_identity: 'bool' = True) -> 'None'
```

Definition. The unital finite dimensional matrix algebra generated by matrices.

Contract (D, C, H, R, X). Generator matrices must act on one finite coordinate space. The selected backend validates common square shape and closes the span under multiplication, optionally adjoining the identity. Exact backends use exact dependence tests. Floating backends use the declared tolerance and therefore return numerical certificates. `matrix_units` requires a positive size, and `AlgebraBasisResult` records the resulting basis, dimension, and certificate without recomputing closure. Principal public operations and properties are `basis`, `dimension`, `contains`, `certificate`, `seed_system`, `transfer_graph`.

Guarantees (E, P, B, K). The generated algebra has dimension at most the square of the matrix dimension, so closure terminates after finitely many independent additions. Matrix products and rank tests dominate cost. Exact scalar growth can dominate exact runs, while conditioning and tolerance selection control floating runs. A numerical containment decision is not promoted to an exact algebraic identity.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_algebra_quotients.py` and `tests/test_block18_frame_algebra.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapters 10 and 64.

C.1D · OperatorGrammar

Identity (q, a, t, k). Public research-manifest route `komorebi_math.OperatorGrammar`. Reflected implementation `komorebi_math.core.grammar.OperatorGrammar`. The same runtime object is also available through `komorebi_math.core.OperatorGrammar`, `komorebi_math.stable.OperatorGrammar`, `komorebi_math.v1.OperatorGrammar`, `komorebi_math.api.v1.OperatorGrammar`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1D-CORE-GRAMMAR`.

Manifest signature (s).

```
(vertices: 'Iterable[str]', arrows: 'Mapping[str, tuple[str, str]] | Iterable[Arrow]', relations:
   ↪ 'Iterable[OperatorRelation]' = ()) -> 'None'
```

Definition. A finite quiver with optional linear path relations.

Contract (D, C, H, R, X). Vertices and arrow names must be nonempty and unique, every arrow endpoint must be declared, and each operator word must be composable in application order. Linear relations may combine only parallel words. A realization supplies one positive dimension per vertex and a matrix of the required target-by-source shape for every arrow. Relation validation either produces a certificate or fails closed. Principal public operations and properties are `vertices`, `arrows`, `relations`, `arrow`, `identity`, `word`, `relation`, `add_relation`, `paths`.

Guarantees (E, P, B, K). Grammar formation is finite and exact at the structural level. Path enumeration is bounded by the requested length and can grow exponentially with branching. Realization evaluation uses the selected linear algebra backend. Exact matrices yield exact relation checks, while floating matrices yield tolerance-dependent residual claims.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_grammar_realization.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource

paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapters 9, 10, and 64.

C.1D · OperatorMorphism

Identity (q, a, t, k). Public research-manifest route `komorebi_math.OperatorMorphism`. Reflected implementation `komorebi_math.core.morphisms.OperatorMorphism`. The same runtime object is also available through `komorebi_math.core.OperatorMorphism`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1D-CORE-MORPHISM`.

Manifest signature (s).

```
(source_operators: 'Mapping[str, Any]', target_operators: 'Mapping[str, Any]', linear_map: 'Any', operator_map:
    ↪ 'Mapping[str, str]', backend: 'Any', tolerance: 'float') -> None
```

Definition. A finite linear intertwiner between two named operator families.

Contract (D, C, H, R, X). A morphism must match source and target dimensions, map every named operator to an existing target operator, and satisfy the appropriate intertwining equations. Packet morphisms also validate the declared packet image, while seed morphisms validate supplied seed images or derive them through the linear map. Certificates expose all residuals and unchecked assumptions. Principal public operations and properties are `create`, `certificate`.

Guarantees (E, P, B, K). Shape and name checks are exact. Intertwining is exact for exact backends and numerical for floating backends. Verification cost is dominated by finite matrix products and residual norms. A valid finite morphism certificate establishes the listed equations only, not an equivalence of arbitrary external models.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_morphisms.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapters 9 through 11 and Chapter 64.

C.1D · OperatorRealization

Identity (q, a, t, k). Public research-manifest route `komorebi_math.OperatorRealization`. Reflected implementation `komorebi_math.core.realization.OperatorRealization`. The same runtime object is also available through `komorebi_math.core.OperatorRealization`, `komorebi_math.stable.OperatorRealization`, `komorebi_math.v1.OperatorRealization`, `komorebi_math.api.v1.OperatorRealization`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1D-CORE-GRAMMAR`.

Manifest signature (s).

```
(grammar: 'OperatorGrammar', spaces: 'Mapping[str, int]', operators: 'Mapping[str, Any]', *, backend: 'str |
    ↪ LinearAlgebraBackend | None' = None, validate_relations: 'bool' = True, tolerance: 'float | None' = None) ->
    ↪ 'None'
```

Definition. An assignment of vector space dimensions and matrices to a typed grammar.

Contract (D, C, H, R, X). Vertices and arrow names must be nonempty and unique, every arrow endpoint must be declared, and each operator word must be composable in application order. Linear relations may combine only parallel words. A realization supplies one positive dimension per vertex and a matrix of the required target-by-source shape for every arrow. Relation validation either produces a certificate or fails closed. Principal public operations and properties are `evaluate`, `relation_matrix`, `relation_residuals`, `relation_certificate`, `path_operators`, `loop_operators`.

Guarantees (E, P, B, K). Grammar formation is finite and exact at the structural level. Path enumeration is bounded by the requested length and can grow exponentially with branching. Realization evaluation uses the selected lin-

ear algebra backend. Exact matrices yield exact relation checks, while floating matrices yield tolerance-dependent residual claims.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_grammar_realization.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapters 9, 10, and 64.

C.1D · `OperatorRelation`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.OperatorRelation`. Reflected implementation `komorebi_math.core.grammar.OperatorRelation`. The same runtime object is also available through `komorebi_math.core.OperatorRelation`, `komorebi_math.stable.OperatorRelation`, `komorebi_math.v1.OperatorRelation`, `komorebi_math.api.v1.OperatorRelation`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1D-CORE-GRAMMAR`.

Manifest signature (s).

```
(name: 'str', terms: 'tuple[tuple[Any, OperatorWord], ...]') -> None
```

Definition. A linear relation among parallel operator words.

Contract (D, C, H, R, X). Vertices and arrow names must be nonempty and unique, every arrow endpoint must be declared, and each operator word must be composable in application order. Linear relations may combine only parallel words. A realization supplies one positive dimension per vertex and a matrix of the required target-by-source shape for every arrow. Relation validation either produces a certificate or fails closed. Principal public operations and properties are `source`, `target`.

Guarantees (E, P, B, K). Grammar formation is finite and exact at the structural level. Path enumeration is bounded by the requested length and can grow exponentially with branching. Realization evaluation uses the selected linear algebra backend. Exact matrices yield exact relation checks, while floating matrices yield tolerance-dependent residual claims.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_grammar_realization.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapters 9, 10, and 64.

C.1D · `OperatorWord`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.OperatorWord`. Reflected implementation `komorebi_math.core.grammar.OperatorWord`. The same runtime object is also available through `komorebi_math.core.OperatorWord`, `komorebi_math.stable.OperatorWord`, `komorebi_math.v1.OperatorWord`, `komorebi_math.api.v1.OperatorWord`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1D-CORE-GRAMMAR`.

Manifest signature (s).

```
(arrows: 'tuple[str, ...]', source: 'str', target: 'str') -> None
```

Definition. A typed path written in the order in which arrows act.

Contract (D, C, H, R, X). Vertices and arrow names must be nonempty and unique, every arrow endpoint must be declared, and each operator word must be composable in application order. Linear relations may combine only parallel words. A realization supplies one positive dimension per vertex and a matrix of the required target-by-source shape for every arrow. Relation validation either produces a certificate or fails closed. Principal public operations and properties are `length`, `is_identity`.

Guarantees (E, P, B, K). Grammar formation is finite and exact at the structural level. Path enumeration is bounded by the requested length and can grow exponentially with branching. Realization evaluation uses the selected linear algebra backend. Exact matrices yield exact relation checks, while floating matrices yield tolerance-dependent residual claims.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_grammar_realization.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapters 9, 10, and 64.

C.1D · PacketCoordinates

Identity (q, a, t, k). Public research-manifest route `komorebi_math.PacketCoordinates`. Reflected implementation `komorebi_math.core.packets.PacketCoordinates`. The same runtime object is also available through `komorebi_math.core.PacketCoordinates`, `komorebi_math.stable.PacketCoordinates`, `komorebi_math.v1.PacketCoordinates`, `komorebi_math.api.v1.PacketCoordinates`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1D-CORE-PACKET`.

Manifest signature (s).

```
(system: "'PacketSystem'", values: 'tuple[Any, ...]') -> None
```

Definition. One tuple of packet-local coordinate vectors tied to a validated packet system.

Contract (D, C, H, R, X). A packet system is a finite family of square projectors on one ambient space. Validation checks idempotence, pairwise annihilation, and completeness. Coordinates contain one vector per packet with the corresponding packet dimension. Packet operators store every target-by-source block with exact shape, and composition, powers, adjoints, tensor products, analysis, and synthesis preserve declared packet order. Principal public operations and properties are `reconstruct`, `support`, `as_dict`.

Guarantees (E, P, B, K). Structural packet identities are exact on exact backends and tolerance-dependent on numerical backends. Explicit block storage is quadratic in packet count and proportional to the total stored entries. Materialization can require the full ambient matrix. Certificates cover the finite supplied projectors and blocks, not an infinite decomposition.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_packets.py` and `tests/test_operators_transfer.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapters 11, 19, and 64.

C.1D · PacketMorphism

Identity (q, a, t, k). Public research-manifest route `komorebi_math.PacketMorphism`. Reflected implementation `komorebi_math.core.morphisms.PacketMorphism`. The same runtime object is also available through `komorebi_math.core.PacketMorphism`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1D-CORE-MORPHISM`.

Manifest signature (s).

```
(source: 'PacketSystem', target: 'PacketSystem', linear_map: 'Any', packet_map: 'Mapping[str, tuple[str, ...]]')
   ↪  -> None
```

Definition. A linear map between packet systems together with the declared target packets receiving each source packet.

Contract (D, C, H, R, X). A morphism must match source and target dimensions, map every named operator to an existing target operator, and satisfy the appropriate intertwining equations. Packet morphisms also validate the

declared packet image, while seed morphisms validate supplied seed images or derive them through the linear map. Certificates expose all residuals and unchecked assumptions. Principal public operations and properties are `certificate`.

Guarantees (`E, P, B, K`). Shape and name checks are exact. Intertwining is exact for exact backends and numerical for floating backends. Verification cost is dominated by finite matrix products and residual norms. A valid finite morphism certificate establishes the listed equations only, not an equivalence of arbitrary external models.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_morphisms.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapters 9 through 11 and Chapter 64.

C.1D · `PacketOperator`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.PacketOperator`. Reflected implementation `komorebi_math.core.operators.PacketOperator`. The same runtime object is also available through `komorebi_math.core.PacketOperator`, `komorebi_math.stable.PacketOperator`, `komorebi_math.v1.PacketOperator`, `komorebi_math.api.v1.PacketOperator`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1D-CORE-PACKET`.

Manifest signature (`s`).

```
(source: 'PacketSystem', target: 'PacketSystem', blocks: 'tuple[tuple[Any, ...], ...]') -> None
```

Definition. An operator represented by all target by source packet blocks.

Contract (`D, C, H, R, X`). A packet system is a finite family of square projectors on one ambient space. Validation checks idempotence, pairwise annihilation, and completeness. Coordinates contain one vector per packet with the corresponding packet dimension. Packet operators store every target-by-source block with exact shape, and composition, powers, adjoints, tensor products, analysis, and synthesis preserve declared packet order. Principal public operations and properties are `backend`, `from_operator`, `reconstruct`, `certificate`, `apply`, `apply_coordinates`, `compose`, `adjoint`, `tensor`, `is_endomorphism`, `power`, `block_norms`, `is_block_diagonal`, `support_mask`.

Guarantees (`E, P, B, K`). Structural packet identities are exact on exact backends and tolerance-dependent on numerical backends. Explicit block storage is quadratic in packet count and proportional to the total stored entries. Materialization can require the full ambient matrix. Certificates cover the finite supplied projectors and blocks, not an infinite decomposition.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_packets.py` and `tests/test_operators_transfer.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapters 11, 19, and 64.

C.1D · `PacketSystem`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.PacketSystem`. Reflected implementation `komorebi_math.core.packets.PacketSystem`. The same runtime object is also available through `komorebi_math.core.PacketSystem`, `komorebi_math.stable.PacketSystem`, `komorebi_math.beta.PacketSystem`, `komorebi_math.v1.PacketSystem`, `komorebi_math.api.v1.PacketSystem`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1D-CORE-PACKET`.

Manifest signature (s).

```
(projectors: 'Sequence[Any]', *, names: 'Sequence[str] | None' = None, backend: 'str | LinearAlgebraBackend | None'
   ↪  = None, tolerance: 'float | None' = None, validate: 'bool | None' = None) -> 'None'
```

Definition. A complete finite family of pairwise annihilating idempotents.

Contract (D, C, H, R, X). A packet system is a finite family of square projectors on one ambient space. Validation checks idempotence, pairwise annihilation, and completeness. Coordinates contain one vector per packet with the corresponding packet dimension. Packet operators store every target-by-source block with exact shape, and composition, powers, adjoints, tensor products, analysis, and synthesis preserve declared packet order. Principal public operations and properties are `index`, `certificate`, `hilbert_certificate`, `analyze`, `synthesize`, `support`, `packet_dimensions`, `aggregate`, `tensor`, `dual`, `equivalent`, `transport`, `restrict`, `quotient`, `block_diagonal_projection`.

Guarantees (E, P, B, K). Structural packet identities are exact on exact backends and tolerance-dependent on numerical backends. Explicit block storage is quadratic in packet count and proportional to the total stored entries. Materialization can require the full ambient matrix. Certificates cover the finite supplied projectors and blocks, not an infinite decomposition.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_packets.py` and `tests/test_operators_transfer.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapters 11, 19, and 64.

C.1D · SeedMorphism

Identity (q, a, t, k). Public research-manifest route `komorebi_math.SeedMorphism`. Reflected implementation `komorebi_math.core.morphisms.SeedMorphism`. The same runtime object is also available through `komorebi_math.core.SeedMorphism`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1D-CORE-MORPHISM`.

Manifest signature (s).

```
(source: 'SeedSystem', target: 'SeedSystem', linear_map: 'Any', operator_map: 'Mapping[str, str]', seed_images:
   ↪  'Sequence[Any] | None' = None) -> None
```

Definition. An operator intertwiner that also transports a finite seed family.

Contract (D, C, H, R, X). A morphism must match source and target dimensions, map every named operator to an existing target operator, and satisfy the appropriate intertwining equations. Packet morphisms also validate the declared packet image, while seed morphisms validate supplied seed images or derive them through the linear map. Certificates expose all residuals and unchecked assumptions. Principal public operations and properties are `certificate`.

Guarantees (E, P, B, K). Shape and name checks are exact. Intertwining is exact for exact backends and numerical for floating backends. Verification cost is dominated by finite matrix products and residual norms. A valid finite morphism certificate establishes the listed equations only, not an equivalence of arbitrary external models.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_morphisms.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapters 9 through 11 and Chapter 64.

C.1D · SeedSystem

Identity (q, a, t, k). Public research-manifest route `komorebi_math.SeedSystem`. Reflected implementation `komorebi_math.core.seeds.SeedSystem`. The same runtime object is also available through `komorebi_math.core.SeedSystem`, `komorebi_math.stable.SeedSystem`, `komorebi_math.v1.SeedSystem`, `komorebi_math.api.v1.SeedSystem`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1D-CORE-SEED`.

Manifest signature (s).

```
(operators: 'Mapping[str, Any] | Sequence[Any]', seeds: 'Sequence[Any]', *, backend: 'str | LinearAlgebraBackend |
    ↪ None' = None, tolerance: 'float | None' = None) -> 'None'
```

Definition. A finite family of operators acting on a finite seed family.

Contract (D, C, H, R, X). Operators must be square and share one ambient dimension. Seeds must have that dimension. The cyclic hull repeatedly applies every operator until rank stabilizes. Closed-form seed-rank helpers require non-negative integer ranks or valid square matrices and implement the stated direct-sum, semisimple, single-operator, or tensor bound rather than searching an unbounded space. Principal public operations and properties are `exact`, `hull_vectors`, `hull_basis`, `hull_dimension`, `is_generating`, `generation_certificate`, `orbit_layers`, `presentation`, `minimum_standard_basis_seeds`.

Guarantees (E, P, B, K). Finite hull construction adds at most as many independent directions as the ambient dimension. Exact backends produce exact ranks and span certificates. Floating backends depend on numerical rank tolerance. Single-operator rank uses rational primary decomposition and may incur polynomial factorization and exact nullity costs.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_seeds.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapters 10 and 64.

C.1D · SupportPreorder

Identity (q, a, t, k). Public research-manifest route `komorebi_math.SupportPreorder`. Reflected implementation `komorebi_math.core.graphs.SupportPreorder`. The same runtime object is also available through `komorebi_math.core.SupportPreorder`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1D-CORE-GRAPH`.

Manifest signature (s).

```
(relation: 'Sequence[Sequence[bool | int]]') -> 'None'
```

Definition. A reflexive transitive support relation on packet labels.

Contract (D, C, H, R, X). Graph relations are finite square Boolean matrices. Transfer graphs use source-by-target adjacency. Support preorders require reflexivity and transitivity. Path routines honor exact source, target, and length bounds, and packet path reconstruction combines only structurally admissible blocks. Invalid vertices, negative lengths, or incompatible packet operators are rejected. Principal public operations and properties are `order`, `is_reflexive`, `is_transitive`, `equivalence_classes`, `strict_class_graph`, `strict_ideal_nilpotency_bound`.

Guarantees (E, P, B, K). Reachability, strongly connected components, condensation, and topological operations are combinatorial and deterministic. Enumerating all fixed-length paths can grow exponentially. Reconstructed blocks inherit the exactness or tolerance status of the packet operator and backend. Graph certificates describe finite support structure, not analytic convergence.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_graphs.py` and `tests/test_operators_transfer.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms,

and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapters 11 and 64.

C.1D · TransferGraph

Identity (q, a, t, k). Public research-manifest route `komorebi_math.TransferGraph`. Reflected implementation `komorebi_math.core.graphs.TransferGraph`. The same runtime object is also available through `komorebi_math.core.TransferGraph`, `komorebi_math.stable.TransferGraph`, `komorebi_math.v1.TransferGraph`, `komorebi_math.api.v1.TransferGraph`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1D-CORE-GRAPH`.

Manifest signature (s).

```
(adjacency: 'Sequence[Sequence[bool | int]]', *, names: 'Sequence[str] | None' = None) -> 'None'
```

Definition. A finite directed graph with adjacency stored as source by target.

Contract (D, C, H, R, X). Graph relations are finite square Boolean matrices. Transfer graphs use source-by-target adjacency. Support preorders require reflexivity and transitivity. Path routines honor exact source, target, and length bounds, and packet path reconstruction combines only structurally admissible blocks. Invalid vertices, negative lengths, or incompatible packet operators are rejected. Principal public operations and properties are `from_packet_operator`, `from_operator_family`, `successors`, `predecessors`, `reachable`, `paths`, `strongly_connected_components`, `condensation`, `source_components`, `is_acyclic`, `topological_order`, `longest_path_vertices`, `source_component_certificate`, `edges`.

Guarantees (E, P, B, K). Reachability, strongly connected components, condensation, and topological operations are combinatorial and deterministic. Enumerating all fixed-length paths can grow exponentially. Reconstructed blocks inherit the exactness or tolerance status of the packet operator and backend. Graph certificates describe finite support structure, not analytic convergence.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_graphs.py` and `tests/test_operators_transfer.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapters 11 and 64.

C.1D · block_from_paths

Identity (q, a, t, k). Public research-manifest route `komorebi_math.block_from_paths`. Reflected implementation `komorebi_math.core.transfer.block_from_paths`. The same runtime object is also available through `komorebi_math.core.block_from_paths`, `komorebi_math.stable.block_from_paths`, `komorebi_math.v1.block_from_paths`, `komorebi_math.api.v1.block_from_paths`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1D-CORE-GRAPH`.

Manifest signature (s).

```
(operator: 'PacketOperator', *, source: 'int', target: 'int', length: 'int', tolerance: 'float | None' = None) ->
    ↪ 'ComputationResult[Any]'
```

Definition. Reconstruct one block of an operator power from packet paths.

Contract (D, C, H, R, X). Graph relations are finite square Boolean matrices. Transfer graphs use source-by-target adjacency. Support preorders require reflexivity and transitivity. Path routines honor exact source, target, and length bounds, and packet path reconstruction combines only structurally admissible blocks. Invalid vertices, negative lengths, or incompatible packet operators are rejected.

Guarantees (E, P, B, K). Reachability, strongly connected components, condensation, and topological operations are combinatorial and deterministic. Enumerating all fixed-length paths can grow exponentially. Reconstructed blocks inherit the exactness or tolerance status of the packet operator and backend. Graph certificates describe finite support structure, not analytic convergence.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_graphs.py` and `tests/test_operators_transfer.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapters 11 and 64.

C.1D · cyclic_span

Identity (q, a, t, k). Public research-manifest route `komorebi_math.cyclic_span`. Reflected implementation `komorebi_math.core.seeds.cyclic_span`. The same runtime object is also available through `komorebi_math.core.cyclic_span`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1D-CORE-SEED`.

Manifest signature (s).

```
(operators: 'Mapping[str, Any] | Sequence[Any]', seeds: 'Sequence[Any]', *, backend: 'str | LinearAlgebraBackend |
    ↪ None' = None, tolerance: 'float | None' = None) -> 'ComputationResult[Any]'
```

Definition. Compute the finite operator hull generated by the supplied seeds.

Contract (D, C, H, R, X). Operators must be square and share one ambient dimension. Seeds must have that dimension. The cyclic hull repeatedly applies every operator until rank stabilizes. Closed-form seed-rank helpers require nonnegative integer ranks or valid square matrices and implement the stated direct-sum, semisimple, single-operator, or tensor bound rather than searching an unbounded space.

Guarantees (E, P, B, K). Finite hull construction adds at most as many independent directions as the ambient dimension. Exact backends produce exact ranks and span certificates. Floating backends depend on numerical rank tolerance. Single-operator rank uses rational primary decomposition and may incur polynomial factorization and exact nullity costs.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_seeds.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapters 10 and 64.

C.1D · independent_direct_sum_seed_rank

Identity (q, a, t, k). Public research-manifest route `komorebi_math.independent_direct_sum_seed_rank`. Reflected implementation `komorebi_math.core.seeds.independent_direct_sum_seed_rank`. The same runtime object is also available through `komorebi_math.core.independent_direct_sum_seed_rank`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1D-CORE-SEED`.

Manifest signature (s).

```
(seed_ranks: 'Sequence[int]') -> 'int'
```

Definition. Return the exact generator number for independent product actions.

Contract (D, C, H, R, X). Operators must be square and share one ambient dimension. Seeds must have that dimension. The cyclic hull repeatedly applies every operator until rank stabilizes. Closed-form seed-rank helpers require nonnegative integer ranks or valid square matrices and implement the stated direct-sum, semisimple, single-operator, or tensor bound rather than searching an unbounded space.

Guarantees (E, P, B, K). Finite hull construction adds at most as many independent directions as the ambient dimension. Exact backends produce exact ranks and span certificates. Floating backends depend on numerical rank tolerance. Single-operator rank uses rational primary decomposition and may incur polynomial factorization and exact nullity costs.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_seeds.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapters 10 and 64.

C.1D · `matrix_units`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.matrix_units`. Reflected implementation `komorebi_math.core.algebra.matrix_units`. The same runtime object is also available through `komorebi_math.core.matrix_units`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1D-CORE-ALGEBRA`.

Manifest signature (s).

```
(size: 'int', *, backend: 'str | LinearAlgebraBackend | None' = None) -> 'tuple[Any, ...]'
```

Definition. Return the standard matrix units of the requested square size.

Contract (D, C, H, R, X). Generator matrices must act on one finite coordinate space. The selected backend validates common square shape and closes the span under multiplication, optionally adjoining the identity. Exact backends use exact dependence tests. Floating backends use the declared tolerance and therefore return numerical certificates. `matrix_units` requires a positive size, and `AlgebraBasisResult` records the resulting basis, dimension, and certificate without recomputing closure.

Guarantees (E, P, B, K). The generated algebra has dimension at most the square of the matrix dimension, so closure terminates after finitely many independent additions. Matrix products and rank tests dominate cost. Exact scalar growth can dominate exact runs, while conditioning and tolerance selection control floating runs. A numerical containment decision is not promoted to an exact algebraic identity.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_algebra_quotients.py` and `tests/test_block18_frame_algebra.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapters 10 and 64.

C.1D · `path_contributions`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.path_contributions`. Reflected implementation `komorebi_math.core.transfer.path_contributions`. The same runtime object is also available through `komorebi_math.core.path_contributions`, `komorebi_math.stable.path_contributions`, `komorebi_math.v1.path_contributions`, `komorebi_math.api.v1.path_contributions`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1D-CORE-GRAPH`.

Manifest signature (s).

```
(operator: 'PacketOperator', *, source: 'int', target: 'int', length: 'int', tolerance: 'float | None' = None) ->
    ↪ 'tuple[TransferPath, ...]'
```

Definition. Return every nonzero structural path contribution of fixed length.

Contract (D, C, H, R, X). Graph relations are finite square Boolean matrices. Transfer graphs use source-by-target adjacency. Support preorders require reflexivity and transitivity. Path routines honor exact source, target, and length bounds, and packet path reconstruction combines only structurally admissible blocks. Invalid vertices, negative lengths, or incompatible packet operators are rejected.

Guarantees (E, P, B, K). Reachability, strongly connected components, condensation, and topological operations are combinatorial and deterministic. Enumerating all fixed-length paths can grow exponentially. Reconstructed blocks

inherit the exactness or tolerance status of the packet operator and backend. Graph certificates describe finite support structure, not analytic convergence.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_graphs.py` and `tests/test_operators_transfer.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapters 11 and 64.

C.1D · semisimple_seed_rank

Identity (q, a, t, k). Public research-manifest route `komorebi_math.semisimple_seed_rank`. Reflected implementation `komorebi_math.core.seeds.semisimple_seed_rank`. The same runtime object is also available through `komorebi_math.core.semisimple_seed_rank`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1D-CORE-SEED`.

Manifest signature (s).

```
(block_dimensions: 'Sequence[int]', multiplicities: 'Sequence[int]') -> 'int'
```

Definition. Return the generator number from semisimple block multiplicities.

Contract (D, C, H, R, X). Operators must be square and share one ambient dimension. Seeds must have that dimension. The cyclic hull repeatedly applies every operator until rank stabilizes. Closed-form seed-rank helpers require nonnegative integer ranks or valid square matrices and implement the stated direct-sum, semisimple, single-operator, or tensor bound rather than searching an unbounded space.

Guarantees (E, P, B, K). Finite hull construction adds at most as many independent directions as the ambient dimension. Exact backends produce exact ranks and span certificates. Floating backends depend on numerical rank tolerance. Single-operator rank uses rational primary decomposition and may incur polynomial factorization and exact nullity costs.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_seeds.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapters 10 and 64.

C.1D · shared_direct_sum_seed_bounds

Identity (q, a, t, k). Public research-manifest route `komorebi_math.shared_direct_sum_seed_bounds`. Reflected implementation `komorebi_math.core.seeds.shared_direct_sum_seed_bounds`. The same runtime object is also available through `komorebi_math.core.shared_direct_sum_seed_bounds`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1D-CORE-SEED`.

Manifest signature (s).

```
(seed_ranks: 'Sequence[int]') -> 'tuple[int, int]'
```

Definition. Return the general lower and upper bounds for a shared acting algebra.

Contract (D, C, H, R, X). Operators must be square and share one ambient dimension. Seeds must have that dimension. The cyclic hull repeatedly applies every operator until rank stabilizes. Closed-form seed-rank helpers require nonnegative integer ranks or valid square matrices and implement the stated direct-sum, semisimple, single-operator, or tensor bound rather than searching an unbounded space.

Guarantees (E, P, B, K). Finite hull construction adds at most as many independent directions as the ambient dimension. Exact backends produce exact ranks and span certificates. Floating backends depend on numerical rank

tolerance. Single-operator rank uses rational primary decomposition and may incur polynomial factorization and exact nullity costs.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_seeds.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapters 10 and 64.

C.1D · `single_operator_seed_rank`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.single_operator_seed_rank`. Reflected implementation `komorebi_math.core.seeds.single_operator_seed_rank`. The same runtime object is also available through `komorebi_math.core.single_operator_seed_rank`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1D-CORE-SEED`.

Manifest signature (s).

```
(operator: 'Any') -> 'int'
```

Definition. Return the exact generator number for a rational matrix as a Q[t] module.

Contract (D, C, H, R, X). Operators must be square and share one ambient dimension. Seeds must have that dimension. The cyclic hull repeatedly applies every operator until rank stabilizes. Closed-form seed-rank helpers require nonnegative integer ranks or valid square matrices and implement the stated direct-sum, semisimple, single-operator, or tensor bound rather than searching an unbounded space.

Guarantees (E, P, B, K). Finite hull construction adds at most as many independent directions as the ambient dimension. Exact backends produce exact ranks and span certificates. Floating backends depend on numerical rank tolerance. Single-operator rank uses rational primary decomposition and may incur polynomial factorization and exact nullity costs.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_seeds.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapters 10 and 64.

C.1D · `tensor_seed_rank_upper_bound`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.tensor_seed_rank_upper_bound`. Reflected implementation `komorebi_math.core.seeds.tensor_seed_rank_upper_bound`. The same runtime object is also available through `komorebi_math.core.tensor_seed_rank_upper_bound`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1D-CORE-SEED`.

Manifest signature (s).

```
(left_rank: 'int', right_rank: 'int') -> 'int'
```

Definition. Return the product upper bound for tensor product seed rank.

Contract (D, C, H, R, X). Operators must be square and share one ambient dimension. Seeds must have that dimension. The cyclic hull repeatedly applies every operator until rank stabilizes. Closed-form seed-rank helpers require nonnegative integer ranks or valid square matrices and implement the stated direct-sum, semisimple, single-operator, or tensor bound rather than searching an unbounded space.

Guarantees (E, P, B, K). Finite hull construction adds at most as many independent directions as the ambient dimension. Exact backends produce exact ranks and span certificates. Floating backends depend on numerical rank tolerance. Single-operator rank uses rational primary decomposition and may incur polynomial factorization and exact nullity costs.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_seeds.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapters 10 and 64.

Structured algorithms (19 cards).

C.1D · `BlockDiagonalLowRankOperator`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.BlockDiagonalLowRankOperator`. Reflected implementation `komorebi_math.algorithms.low_rank.BlockDiagonalLowRankOperator`. The same runtime object is also available through `komorebi_math.algorithms.BlockDiagonalLowRankOperator`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1D-ALG-LOW-RANK`.

Manifest signature (s).

```
(diagonal: 'BlockDiagonalOperator', correction: 'LowRankOperator') -> None
```

Definition. A square block diagonal operator plus one low rank correction.

Contract (D, C, H, R, X). Low-rank factors must be two-dimensional, share a common rank, and represent `left @ right.conj().T`. A block-diagonal low-rank operator requires a square diagonal part and a correction of matching shape. Recompression validates the requested rank and tolerance, forms a small core, and truncates through singular-value decomposition. Principal public operations and properties are `from_factors`, `shape`, `dtype`, `rank_bound`, `apply`, `materialize`, `adjoint`, `storage_entries`, `flop_count`, `solve`, `exact_product`, `recompress`.

Guarantees (E, P, B, K). Application and storage scale with factor rank rather than the full dense matrix when the rank is small. Recompression and Woodbury-style solves are numerical and depend on singular-value and conditioning thresholds. Materialization remains available but loses the storage advantage. Rank bounds are finite upper bounds, not minimum-rank proofs.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_algorithms_low_rank.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapter 29.

C.1D · `BlockDiagonalOperator`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.BlockDiagonalOperator`. Reflected implementation `komorebi_math.algorithms.structured.BlockDiagonalOperator`. The same runtime object is also available through `komorebi_math.algorithms.BlockDiagonalOperator`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1D-ALG-STRUCTURED`.

Manifest signature (s).

```
(blocks: 'tuple[StructuredOperator | Array, ...]', names: 'tuple[str, ...] | None' = None) -> None
```

Definition. Square direct sum of independently executable structured blocks.

Contract (D, C, H, R, X). Structured operators expose a common finite `shape`, `dtype`, `apply`, `adjoint`, `materialize`, storage, flop, residual, and solve contract. Dense matrices must be rectangular. Block-diagonal operands must be square and dimension-compatible. Circulant operators are determined by one finite first column. Walsh-Hadamard dimensions must be positive powers of two. Principal public operations and properties are `shape`, `dtype`, `apply`, `solve`, `materialize`, `adjoint`, `storage_entries`, `flop_count`, `block`.

Guarantees (E, P, B, K). Applications use the represented structure rather than silently materializing a dense matrix: blockwise execution for direct sums, FFT execution for circulants, and butterfly execution for Walsh-Hadamard

transforms. Explicit `materialize` and generic `solve` may allocate dense matrices. Residual certificates are numerical unless the underlying array arithmetic is exact.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_algorithms_structured.py` and `tests/test_algorithms_complexity_benchmarks.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapter 29.

C.1D · `CirculantOperator`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.CirculantOperator`. Reflected implementation `komorebi_math.algorithms.structured.CirculantOperator`. The same runtime object is also available through `komorebi_math.algorithms.CirculantOperator`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1D-ALG-STRUCTURED`.

Manifest signature (s).

```
(first_column: 'Array') -> None
```

Definition. Circulant matrix applied through a discrete Fourier transform.

Contract (D, C, H, R, X). Structured operators expose a common finite `shape`, `dtype`, `apply`, `adjoint`, `materialize`, storage, flop, residual, and solve contract. Dense matrices must be rectangular. Block-diagonal operands must be square and dimension-compatible. Circulant operators are determined by one finite first column. Walsh-Hadamard dimensions must be positive powers of two. Principal public operations and properties are `shape`, `dtype`, `apply`, `materialize`, `adjoint`, `storage_entries`, `flop_count`.

Guarantees (E, P, B, K). Applications use the represented structure rather than silently materializing a dense matrix: blockwise execution for direct sums, FFT execution for circulants, and butterfly execution for Walsh-Hadamard transforms. Explicit `materialize` and generic `solve` may allocate dense matrices. Residual certificates are numerical unless the underlying array arithmetic is exact.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_algorithms_structured.py` and `tests/test_algorithms_complexity_benchmarks.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapter 29.

C.1D · `DenseOperator`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.DenseOperator`. Reflected implementation `komorebi_math.algorithms.structured.DenseOperator`. The same runtime object is also available through `komorebi_math.algorithms.DenseOperator`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1D-ALG-STRUCTURED`.

Manifest signature (s).

```
(matrix: 'Array') -> None
```

Definition. Explicit dense matrix implementation of the structured contract.

Contract (D, C, H, R, X). Structured operators expose a common finite `shape`, `dtype`, `apply`, `adjoint`, `materialize`, storage, flop, residual, and solve contract. Dense matrices must be rectangular. Block-diagonal operands must be square and dimension-compatible. Circulant operators are determined by one finite first column. Walsh-Hadamard dimensions must be positive powers of two. Principal public operations and properties are `shape`, `dtype`, `apply`, `materialize`, `adjoint`, `storage_entries`, `flop_count`.

Guarantees (E, P, B, K). Applications use the represented structure rather than silently materializing a dense matrix: blockwise execution for direct sums, FFT execution for circulants, and butterfly execution for Walsh-Hadamard

transforms. Explicit `materialize` and generic `solve` may allocate dense matrices. Residual certificates are numerical unless the underlying array arithmetic is exact.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_algorithms_structured.py` and `tests/test_algorithms_complexity_benchmarks.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapter 29.

C.1D · DTypeChoice

Identity (q, a, t, k). Public research-manifest route `komorebi_math.DTypeChoice`. Reflected implementation `komorebi_math.algorithms.precision.DTypeChoice`. The same runtime object is also available through `komorebi_math.algorithms.DTypeChoice`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1D-ALG-PRECISION`.

Manifest signature (s).

```
(name: 'str', cost: 'float', error_units: 'int', dtype: 'str | None' = None) -> None
```

Definition. One discrete precision option with a finite cost, integer error weight, and optional backend dtype label.

Contract (D, C, H, R, X). Each precision choice has a finite cost and a nonnegative integer error weight. Every operation must provide at least one option. The assignment routine solves the finite multiple-choice budget problem and returns `None` when no combination satisfies the integer error budget.

Guarantees (E, P, B, K). Dynamic programming is deterministic and exact for the discretized cost and error model. Runtime and memory scale with the number of choices times the integer budget. The result optimizes the supplied model only. It does not infer rounding error or hardware cost that was not encoded in the options.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_algorithms_precision.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapter 30.

C.1D · LowRankOperator

Identity (q, a, t, k). Public research-manifest route `komorebi_math.LowRankOperator`. Reflected implementation `komorebi_math.algorithms.low_rank.LowRankOperator`. The same runtime object is also available through `komorebi_math.algorithms.LowRankOperator`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1D-ALG-LOW-RANK`.

Manifest signature (s).

```
(left: 'Array', right: 'Array') -> None
```

Definition. A finite operator represented as `left @ right.conj().T`.

Contract (D, C, H, R, X). Low-rank factors must be two-dimensional, share a common rank, and represent `left @ right.conj().T`. A block-diagonal low-rank operator requires a square diagonal part and a correction of matching shape. Recompression validates the requested rank and tolerance, forms a small core, and truncates through singular-value decomposition. Principal public operations and properties are `rank_bound`, `shape`, `dtype`, `apply`, `materialize`, `adjoint`, `storage_entries`, `flop_count`.

Guarantees (E, P, B, K). Application and storage scale with factor rank rather than the full dense matrix when the rank is small. Recompression and Woodbury-style solves are numerical and depend on singular-value and conditioning thresholds. Materialization remains available but loses the storage advantage. Rank bounds are finite upper bounds, not minimum-rank proofs.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_algorithms_low_rank.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapter 29.

C.1D · PacketIR

Identity (q, a, t, k). Public research-manifest route `komorebi_math.PacketIR`. Reflected implementation `komorebi_math.algorithms.ir.PacketIR`. The same runtime object is also available through `komorebi_math.algorithms.PacketIR`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1D-ALG-PACKET-IR`.

Manifest signature (s).

```
(source_layout: 'PacketLayout', target_layout: 'PacketLayout', blocks: 'Mapping[Address, StructuredOperator |
    ↪ Any]', metadata: 'Mapping[str, Any]' = <factory>) -> None
```

Definition. A finite operator as a map from source packets to target packets.

Contract (D, C, H, R, X). Packet layouts require positive packet sizes and unique optional names. Packet-IR block addresses use `(target, source)` order, and every stored block must match the corresponding target-by-source dimensions. Missing blocks are structural zeros. Composition, adjoint, tensor product, powers, and dense conversion preserve the declared layouts. Principal public operations and properties are `square`, `from_dense`, `shape`, `dtype`, `support`, `stored_entries`, `flop_count`, `block`, `materialize`, `apply`, `adjoint`, `compose`, `power`, `tensor`, `diagonal_part`, `certificate`.

Guarantees (E, P, B, K). Sparse packet execution touches stored blocks only, with work proportional to their dimensions and count. Dense materialization scales with the full source and target dimensions. Support and storage counts are exact structural facts. Numerical block values retain their backend precision and do not acquire an exactness claim.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_algorithms_layout_ir.py` and `tests/test_algorithms_support.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapter 28.

C.1D · PacketLayout

Identity (q, a, t, k). Public research-manifest route `komorebi_math.PacketLayout`. Reflected implementation `komorebi_math.algorithms.layout.PacketLayout`. The same runtime object is also available through `komorebi_math.algorithms.PacketLayout`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1D-ALG-PACKET-IR`.

Manifest signature (s).

```
(sizes: 'tuple[int, ...]', names: 'tuple[str, ...] | None' = None) -> None
```

Definition. A contiguous finite packet decomposition of one coordinate space.

Contract (D, C, H, R, X). Packet layouts require positive packet sizes and unique optional names. Packet-IR block addresses use `(target, source)` order, and every stored block must match the corresponding target-by-source dimensions. Missing blocks are structural zeros. Composition, adjoint, tensor product, powers, and dense conversion preserve the declared layouts. Principal public operations and properties are `equal`, `from_packet_system`, `index`, `packet_slice`, `packet_size`, `split`, `merge`, `crossed`, `permutation`, `stored_coordinate_count`.

Guarantees (E, P, B, K). Sparse packet execution touches stored blocks only, with work proportional to their dimensions and count. Dense materialization scales with the full source and target dimensions. Support and storage

counts are exact structural facts. Numerical block values retain their backend precision and do not acquire an exactness claim.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_algorithms_layout_ir.py` and `tests/test_algorithms_support.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapter 28.

C.1D · Task

Identity (q, a, t, k). Public research-manifest route `komorebi_math.Task`. Reflected implementation `komorebi_math.algorithms.scheduling.Task`. The same runtime object is also available through `komorebi_math.algorithms.Task`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1D-ALG-SCHEDULING`.

Manifest signature (s).

```
(name: 'str', duration: 'float' = 1.0, dependencies: 'tuple[str, ...]' = (), shape_key: 'Hashable | None' = None)
   ↪  -> None
```

Definition. One named task with duration, dependencies, and an optional shape key.

Contract (D, C, H, R, X). Task names must be unique, durations finite and nonnegative, and every dependency must name a supplied task. The dependency graph must be acyclic. Unlimited-concurrency scheduling assigns each task its earliest feasible start and computes critical-path information without imposing an unstated worker limit.

Guarantees (E, P, B, K). The schedule is exact for the finite directed acyclic graph and supplied durations. Runtime is linear in tasks and dependency edges after validation. The result does not model resource contention, communication cost, or preemption unless represented separately.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_algorithms_scheduling.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapter 30.

C.1D · TreeNode

Identity (q, a, t, k). Public research-manifest route `komorebi_math.TreeNode`. Reflected implementation `komorebi_math.algorithms.compiler.TreeNode`. The same runtime object is also available through `komorebi_math.algorithms.TreeNode`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1D-ALG-COMPILER`.

Manifest signature (s).

```
(name: 'str', keep_cost: 'float', keep_error: 'float', split_cost: 'float' = 0.0, children: "tuple['TreeNode',
   ↪  ...]" = ()) -> None
```

Definition. One candidate keep or split decision in a finite compiler tree.

Contract (D, C, H, R, X). A compiler tree is finite, has unique node names, and gives every node a keep cost, keep error, split cost, and child tuple. The Lagrangian compiler minimizes cost plus penalty times error by exact tree dynamic programming. The budget compiler converts each keep error independently to an integer weight and minimizes cost subject to the supplied budget. Principal public operations and properties are `terminal`, `names`.

Guarantees (E, P, B, K). Both compilers return a globally optimal plan for the finite tree and stated objective. Lagrangian compilation is linear in tree size after validation. Budget compilation is pseudo-polynomial in tree size and budget. Error quantization is part of the contract and is not a claim about unmodeled numerical error.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_algorithms_compiler.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapter 30.

C.1D · `WalshHadamardOperator`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.WalshHadamardOperator`. Reflected implementation `komorebi_math.algorithms.structured.WalshHadamardOperator`. The same runtime object is also available through `komorebi_math.algorithms.WalshHadamardOperator`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1D-ALG-STRUCTURED`.

Manifest signature (s).

```
(dimension: 'int', normalized: 'bool' = False, dtype_value: 'Any' = <class 'float'>) -> None
```

Definition. Fast Walsh Hadamard transform in power of two dimension.

Contract (D, C, H, R, X). Structured operators expose a common finite `shape`, `dtype`, `apply`, `adjoint`, `materialize`, storage, flop, residual, and solve contract. Dense matrices must be rectangular. Block-diagonal operands must be square and dimension-compatible. Circulant operators are determined by one finite first column. Walsh-Hadamard dimensions must be positive powers of two. Principal public operations and properties are `shape`, `dtype`, `apply`, `materialize`, `adjoint`, `storage_entries`, `flop_count`, `solve`.

Guarantees (E, P, B, K). Applications use the represented structure rather than silently materializing a dense matrix: blockwise execution for direct sums, FFT execution for circulants, and butterfly execution for Walsh-Hadamard transforms. Explicit `materialize` and generic `solve` may allocate dense matrices. Residual certificates are numerical unless the underlying array arithmetic is exact.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_algorithms_structured.py` and `tests/test_algorithms_complexity_benchmarks.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapter 29.

C.1D · `assign_discrete_precisions`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.assign_discrete_precisions`. Reflected implementation `komorebi_math.algorithms.precision.assign_discrete_precisions`. The same runtime object is also available through `komorebi_math.algorithms.assign_discrete_precisions`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1D-ALG-PRECISION`.

Manifest signature (s).

```
(options: 'Sequence[Sequence[DTypeChoice]]', budget: 'int') -> 'PrecisionAssignment | None'
```

Definition. Minimize total cost subject to an exact integer error budget.

Contract (D, C, H, R, X). Each precision choice has a finite cost and a nonnegative integer error weight. Every operation must provide at least one option. The assignment routine solves the finite multiple-choice budget problem and returns `None` when no combination satisfies the integer error budget.

Guarantees (E, P, B, K). Dynamic programming is deterministic and exact for the discretized cost and error model. Runtime and memory scale with the number of choices times the integer budget. The result optimizes the supplied model only. It does not infer rounding error or hardware cost that was not encoded in the options.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_algorithms_precision.py`. The independent verifier for core operators, algorithms,

and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapter 30.

C.1D · compile_tree_budget

Identity (q, a, t, k). Public research-manifest route `komorebi_math.compile_tree_budget`. Reflected implementation `komorebi_math.algorithms.compiler.compile_tree_budget`. The same runtime object is also available through `komorebi_math.algorithms.compile_tree_budget`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1D-ALG-COMPILER`.

Manifest signature (s).

```
(root: 'TreeNode', budget: 'int', *, error_unit: 'float' = 1.0) -> 'TreePlan | None'
```

Definition. Minimize cost under additive integer error weights.

Contract (D, C, H, R, X). A compiler tree is finite, has unique node names, and gives every node a keep cost, keep error, split cost, and child tuple. The Lagrangian compiler minimizes cost plus penalty times error by exact tree dynamic programming. The budget compiler converts each keep error independently to an integer weight and minimizes cost subject to the supplied budget.

Guarantees (E, P, B, K). Both compilers return a globally optimal plan for the finite tree and stated objective. Lagrangian compilation is linear in tree size after validation. Budget compilation is pseudo-polynomial in tree size and budget. Error quantization is part of the contract and is not a claim about unmodeled numerical error.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_algorithms_compiler.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapter 30.

C.1D · compile_tree_lagrangian

Identity (q, a, t, k). Public research-manifest route `komorebi_math.compile_tree_lagrangian`. Reflected implementation `komorebi_math.algorithms.compiler.compile_tree_lagrangian`. The same runtime object is also available through `komorebi_math.algorithms.compile_tree_lagrangian`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1D-ALG-COMPILER`.

Manifest signature (s).

```
(root: 'TreeNode', penalty: 'float') -> 'TreePlan'
```

Definition. Return the exact minimizer of cost plus penalty times error.

Contract (D, C, H, R, X). A compiler tree is finite, has unique node names, and gives every node a keep cost, keep error, split cost, and child tuple. The Lagrangian compiler minimizes cost plus penalty times error by exact tree dynamic programming. The budget compiler converts each keep error independently to an integer weight and minimizes cost subject to the supplied budget.

Guarantees (E, P, B, K). Both compilers return a globally optimal plan for the finite tree and stated objective. Lagrangian compilation is linear in tree size after validation. Budget compilation is pseudo-polynomial in tree size and budget. Error quantization is part of the contract and is not a claim about unmodeled numerical error.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_algorithms_compiler.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapter 30.

C.1D · conjugate_gradient

Identity (q, a, t, k). Public research-manifest route `komorebi_math.conjugate_gradient`. Reflected implementation `komorebi_math.algorithms.krylov.conjugate_gradient`. The same runtime object is also available through `komorebi_math.algorithms.conjugate_gradient`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1D-ALG-KRYLOV`.

Manifest signature (s).

```
(apply: 'Apply', right_hand_side: 'Any', *, precondition: 'Apply | None' = None, initial: 'Any | None' = None,
    ↪ relative_tolerance: 'float' = 1e-10, absolute_tolerance: 'float' = 0.0, max_iterations: 'int | None' = None,
    ↪ curvature_tolerance: 'float' = 1e-14) -> 'ConjugateGradientResult'
```

Definition. Solve a declared Hermitian positive definite system by CG.

Contract (D, C, H, R, X). Conjugate gradient accepts a finite vector, a linear application callable, and an optional preconditioner. Tolerances and iteration limits must be finite and nonnegative. The method assumes a Hermitian positive-definite operator, checks curvature and residuals during execution, and records convergence, breakdown, or iteration exhaustion explicitly.

Guarantees (E, P, B, K). Each iteration performs a bounded number of operator applications and vector operations. Storage is linear in vector dimension. Convergence depends on the declared positive-definite model and conditioning. A small computed residual is numerical evidence, not an exact solution certificate.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_algorithms_krylov.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapter 30.

C.1D · discover_block_diagonal_low_rank

Identity (q, a, t, k). Public research-manifest route `komorebi_math.discover_block_diagonal_low_rank`. Reflected implementation `komorebi_math.algorithms.discovery.discover_block_diagonal_low_rank`. The same runtime object is also available through `komorebi_math.algorithms.discover_block_diagonal_low_rank`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1D-ALG-DISCOVERY`.

Manifest signature (s).

```
(matrix: 'Any', layout: 'PacketLayout', rank: 'int', *, tolerance: 'float' = 1e-10) -> 'BDLRDiscoveryResult'
```

Definition. Project onto a fixed block diagonal part and truncate its residual SVD.

Contract (D, C, H, R, X). Discovery routines operate inside a fixed packet layout. Support discovery thresholds observed block norms using absolute and relative tolerances. Block-diagonal low-rank discovery projects onto the declared diagonal layout, computes a residual singular-value decomposition, and truncates to the requested rank.

Guarantees (E, P, B, K). Outputs are deterministic for fixed numerical inputs and backend routines. They are heuristic model selections accompanied by measured residuals, not proofs that the discovered support or rank is unique or optimal outside the stated projection problem. Dense singular-value decomposition can dominate runtime and memory.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_algorithms_discovery_gpu.py` and `tests/test_algorithms_support.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapters 28 through 30.

C.1D · discover_packet_support

Identity (q, a, t, k). Public research-manifest route `komorebi_math.discover_packet_support`. Reflected implementation `komorebi_math.algorithms.discovery.discover_packet_support`. The same runtime object is also available through `komorebi_math.algorithms.discover_packet_support`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1D-ALG-DISCOVERY`.

Manifest signature (s).

```
(matrix: 'Any', layout: 'PacketLayout', *, absolute_tolerance: 'float' = 0.0, relative_tolerance: 'float' = 1e-12)
   ↪  -> 'SupportDiscoveryResult'
```

Definition. Discover a heuristic block support inside a fixed packet layout.

Contract (D, C, H, R, X). Discovery routines operate inside a fixed packet layout. Support discovery thresholds observed block norms using absolute and relative tolerances. Block-diagonal low-rank discovery projects onto the declared diagonal layout, computes a residual singular-value decomposition, and truncates to the requested rank.

Guarantees (E, P, B, K). Outputs are deterministic for fixed numerical inputs and backend routines. They are heuristic model selections accompanied by measured residuals, not proofs that the discovered support or rank is unique or optimal outside the stated projection problem. Dense singular-value decomposition can dominate runtime and memory.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_algorithms_discovery_gpu.py` and `tests/test_algorithms_support.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapters 28 through 30.

C.1D · recompress_factors

Identity (q, a, t, k). Public research-manifest route `komorebi_math.recompress_factors`. Reflected implementation `komorebi_math.algorithms.low_rank.recompress_factors`. The same runtime object is also available through `komorebi_math.algorithms.recompress_factors`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1D-ALG-LOW-RANK`.

Manifest signature (s).

```
(left: 'Any', right: 'Any', rank: 'int', *, tolerance: 'float' = 1e-10) -> 'RecompressionResult'
```

Definition. Recompress a two factor product through an identity small core.

Contract (D, C, H, R, X). Low-rank factors must be two-dimensional, share a common rank, and represent `left @ right.conj().T`. A block-diagonal low-rank operator requires a square diagonal part and a correction of matching shape. Recompression validates the requested rank and tolerance, forms a small core, and truncates through singular-value decomposition.

Guarantees (E, P, B, K). Application and storage scale with factor rank rather than the full dense matrix when the rank is small. Recompression and Woodbury-style solves are numerical and depend on singular-value and conditioning thresholds. Materialization remains available but loses the storage advantage. Rank bounds are finite upper bounds, not minimum-rank proofs.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_algorithms_low_rank.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapter 29.

C.1D · schedule_unlimited

Identity (q, a, t, k). Public research-manifest route `komorebi_math.schedule_unlimited`. Reflected implementation `komorebi_math.algorithms.scheduling.schedule_unlimited`. The same runtime object is also available through `komorebi_math.algorithms.schedule_unlimited`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1D-ALG-SCHEDULING`.

Manifest signature (s).

```
(tasks: 'Sequence[Task]') -> 'TaskSchedule'
```

Definition. Return the earliest start schedule under unlimited concurrency.

Contract (D, C, H, R, X). Task names must be unique, durations finite and nonnegative, and every dependency must name a supplied task. The dependency graph must be acyclic. Unlimited-concurrency scheduling assigns each task its earliest feasible start and computes critical-path information without imposing an unstated worker limit.

Guarantees (E, P, B, K). The schedule is exact for the finite directed acyclic graph and supplied durations. Runtime is linear in tasks and dependency edges after validation. The result does not model resource contention, communication cost, or preemption unless represented separately.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_algorithms_scheduling.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapter 30.

Exact integration (38 cards).

C.1D · AtomicIntegratedDerivativeProvider

Identity (q, a, t, k). Public research-manifest route `komorebi_math.AtomicIntegratedDerivativeProvider`. Reflected implementation `komorebi_math.integration.adapters.AtomicIntegratedDerivativeProvider`. The same runtime object is also available through `komorebi_math.integration.AtomicIntegratedDerivativeProvider`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1D-INT-ADAPTER`.

Manifest signature (s).

```
(providers: 'Sequence[Any]', weights: 'Sequence[int | Fraction | float]', name: 'str' = 'atomic integrated
    ↪ derivative provider') -> None
```

Definition. A fixed finite weighted sum of compatible derivative providers.

Contract (D, C, H, R, X). Adapters convert finite formal-series or polynomial-family coordinates into `ExactPolynomial`, integrate through the ordinary measure interface, or combine compatible derivative providers with fixed scalar weights. Degree truncation must be explicit. Analysis-synthesis integration checks compare physical and coefficient-side finite sums under one declared system and atomic measure. Principal public operations and properties are `supports_order`, `value`, `derivative`, `certificate`.

Guarantees (E, P, B, K). Conversions are exact when coefficients, family data, and measures are exact. Truncation is reported rather than hidden. Provider sums preserve the weakest child certificate status. The atomic derivative adapter deliberately excludes parameter-dependent measure derivatives, which would require an additional term.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_block19_integration.py` and `tests/test_coordinates_polynomials.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapters 47 through 49 and Chapter 68.

C.1D · AtomicMeasure

Identity (q, a, t, k). Public research-manifest route `komorebi_math.AtomicMeasure`. Reflected implementation `komorebi_math.integration.measures.AtomicMeasure`. The same runtime object is also available through `komorebi_math.integration.AtomicMeasure`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1D-INT-MEASURE`.

Manifest signature (s).

```
(points: 'Iterable[Any]', weights: 'Iterable[Any]', *, name: 'str' = 'atomic measure') -> 'None'
```

Definition. A finite positive atomic measure with exact or floating-point data.

Contract (D, C, H, R, X). Measures expose finite dimension, mass, moments, exactness, and a structural certificate. Atomic measures require compatible finite points and nonnegative finite weights. Interval measures use finite ordered bounds and nonnegative constant density. Products preserve factor order. Exact affine pushforwards validate dimensions and Jacobian data, while arbitrary callable pushforwards retain explicit unchecked regularity and purity assumptions. Principal public operations and properties are `atom_count`, `mass`, `moment`, `normalized`, `coalesced`, `certificate`.

Guarantees (E, P, B, K). Integer and `Fraction` data remain exact. Floating data are checked for finiteness and produce numerical or conditional evidence. Atomic work is linear in atom count, product moments multiply factor results, and affine polynomial pullback can grow combinatorially in term count. Callable pushforwards are definitions with stated assumptions, not global regularity proofs.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_block19_integration.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapters 47 through 49 and Chapter 68.

C.1D · CompositeSimpson

Identity (q, a, t, k). Public research-manifest route `komorebi_math.CompositeSimpson`. Reflected implementation `komorebi_math.integration.quadrature.CompositeSimpson`. The same runtime object is also available through `komorebi_math.integration.CompositeSimpson`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1D-INT-QUADRATURE`.

Manifest signature (s).

```
(subintervals: 'int' = 64, derivative_bound: 'DerivativeBound | None' = None, max_evaluations: 'int' = 1000000) ->
    ↪ None
```

Definition. Composite Simpson quadrature for bounded interval factors.

Contract (D, C, H, R, X). Integration accepts an exact polynomial or a callable and one finite measure. Exact polynomials use moments when available. Atomic callables use finite weighted sums. Interval callables use composite Simpson quadrature with an even positive subinterval count and evaluation budget. Derivative bounds, error guarantees, orientation, and failure status remain explicit in the returned records. Principal public operations and properties are `without_derivative_bound`.

Guarantees (E, P, B, K). Exact polynomial paths return exact values when all required moments are exact. Simpson bounds are rigorous only when a valid fourth-derivative bound applies to the interval. Otherwise the error status is conditional, estimated, or unknown as recorded. Numerical evaluation count is finite and bounded by the provider budget.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_block19_integration.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact

signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapters 47 through 49 and Chapter 68.

C.1D · DerivativeBound

Identity (q, a, t, k). Public research-manifest route `komorebi_math.DerivativeBound`. Reflected implementation `komorebi_math.integration.quadrature.DerivativeBound`. The same runtime object is also available through `komorebi_math.integration.DerivativeBound`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1D-INT-QUADRATURE`.

Manifest signature (s).

```
(order: 'int', bound: 'Fraction | float', certificate: 'Certificate', description: 'str' = 'derivative supremum
    ↪ bound', polynomial: 'ExactPolynomial | None' = None, interval: 'tuple[Fraction, Fraction] | None' = None) ->
    ↪ None
```

Definition. A declared bound `sup |f^(order)| <= bound` with its own proof status.

Contract (D, C, H, R, X). Integration accepts an exact polynomial or a callable and one finite measure. Exact polynomials use moments when available. Atomic callables use finite weighted sums. Interval callables use composite Simpson quadrature with an even positive subinterval count and evaluation budget. Derivative bounds, error guarantees, orientation, and failure status remain explicit in the returned records. Principal public operations and properties are `guarantee`, `applies_to`, `assumed`.

Guarantees (E, P, B, K). Exact polynomial paths return exact values when all required moments are exact. Simpson bounds are rigorous only when a valid fourth-derivative bound applies to the interval. Otherwise the error status is conditional, estimated, or unknown as recorded. Numerical evaluation count is finite and bounded by the provider budget.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_block19_integration.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapters 47 through 49 and Chapter 68.

C.1D · ErrorGuarantee

Identity (q, a, t, k). Public research-manifest route `komorebi_math.ErrorGuarantee`. Reflected implementation `komorebi_math.integration.quadrature.ErrorGuarantee`. The same runtime object is also available through `komorebi_math.integration.ErrorGuarantee`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1D-INT-QUADRATURE`.

Manifest signature (s).

```
(*values)
```

Definition. Strength of an integration error statement.

Contract (D, C, H, R, X). Integration accepts an exact polynomial or a callable and one finite measure. Exact polynomials use moments when available. Atomic callables use finite weighted sums. Interval callables use composite Simpson quadrature with an even positive subinterval count and evaluation budget. Derivative bounds, error guarantees, orientation, and failure status remain explicit in the returned records.

Guarantees (E, P, B, K). Exact polynomial paths return exact values when all required moments are exact. Simpson bounds are rigorous only when a valid fourth-derivative bound applies to the interval. Otherwise the error status is conditional, estimated, or unknown as recorded. Numerical evaluation count is finite and bounded by the provider budget.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_block19_integration.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource

paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapters 47 through 49 and Chapter 68.

C.1D · ExactAffineMap

Identity (q, a, t, k). Public research-manifest route `komorebi_math.ExactAffineMap`. Reflected implementation `komorebi_math.integration.measures.ExactAffineMap`. The same runtime object is also available through `komorebi_math.integration.ExactAffineMap`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1D-INT-MEASURE`.

Manifest signature (s).

```
(matrix: 'Iterable[Iterable[Rational]]', shift: 'Iterable[Rational]', *, name: 'str' = 'exact affine map') ->
   ↪ 'None'
```

Definition. An exact affine map `x -> matrix*x + shift` over `Q`.

Contract (D, C, H, R, X). Measures expose finite dimension, mass, moments, exactness, and a structural certificate. Atomic measures require compatible finite points and nonnegative finite weights. Interval measures use finite ordered bounds and nonnegative constant density. Products preserve factor order. Exact affine pushforwards validate dimensions and Jacobian data, while arbitrary callable pushforwards retain explicit unchecked regularity and purity assumptions. Principal public operations and properties are `identity`, `source_dimension`, `target_dimension`, `square`, `apply_point`, `then`, `pullback`, `determinant`, `certificate`.

Guarantees (E, P, B, K). Integer and `Fraction` data remain exact. Floating data are checked for finiteness and produce numerical or conditional evidence. Atomic work is linear in atom count, product moments multiply factor results, and affine polynomial pullback can grow combinatorially in term count. Callable pushforwards are definitions with stated assumptions, not global regularity proofs.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_block19_integration.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapters 47 through 49 and Chapter 68.

C.1D · ExactPolynomial

Identity (q, a, t, k). Public research-manifest route `komorebi_math.ExactPolynomial`. Reflected implementation `komorebi_math.integration.polynomial.ExactPolynomial`. The same runtime object is also available through `komorebi_math.integration.ExactPolynomial`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1D-INT-POLYNOMIAL`.

Manifest signature (s).

```
(dimension: 'int', terms: 'Mapping[Sequence[int], Rational] | Iterable[tuple[Sequence[int], Rational]]') -> 'None'
```

Definition. A sparse polynomial `sum_exponent c_exponent x^exponent` over `Q`.

Contract (D, C, H, R, X). An exact polynomial has a positive finite dimension and a sparse mapping from nonnegative exponent tuples to integer or `Fraction` coefficients. Construction combines duplicate exponents and removes zeros. Arithmetic, derivatives, antiderivatives, affine pullback, evaluation, and variable integration validate dimensions and enforce explicit term budgets where expansion can grow. Principal public operations and properties are `zero`, `constant`, `monomial`, `univariate`, `degree`, `degrees`, `term_count`, `coefficient`, `evaluate`, `add`, `subtract`, `scale`, `multiply`, `power`, `derivative`, `antiderivative`.

Guarantees (E, P, B, K). All polynomial coefficients and exact evaluations use `Fraction` arithmetic. Sparse costs scale with the number of terms, while multiplication, powers, and affine pullback can expand combinatorially. Certificates cover the finite polynomial representation and exact operations, not convergence of an infinite series.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_block19_integration.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapters 47 through 49 and Chapter 68.

C.1D · `ExactQuadratureRule`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.ExactQuadratureRule`. Reflected implementation `komorebi_math.integration.rules.ExactQuadratureRule`. The same runtime object is also available through `komorebi_math.integration.ExactQuadratureRule`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1D-INT-RULE`.

Manifest signature (s).

```
(points: 'Iterable[Any]', weights: 'Iterable[Any]', *, name: 'str' = 'exact quadrature rule') -> 'None'
```

Definition. A quadrature functional with rational nodes and signed rational weights.

Contract (D, C, H, R, X). A quadrature rule is a finite linear functional with nodes and weights of equal nonzero length. General rules may use signed or floating weights. Exact rules require rational nodes and weights. Moment comparison determines the tested degree of exactness, and tensor products preserve the stated node ordering and finite dimensionality.

Guarantees (E, P, B, K). Exact rules integrate supported exact polynomials through rational arithmetic. Numerical rules preserve measured residuals and do not claim exactness beyond tested moments. Tensor-product node count multiplies across factors and can grow exponentially with dimension.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_block19_integration.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapters 47 through 49 and Chapter 68.

C.1D · `IntegralIdentity`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.IntegralIdentity`. Reflected implementation `komorebi_math.integration.identities.IntegralIdentity`. The same runtime object is also available through `komorebi_math.integration.IntegralIdentity`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1D-INT-IDENTITY`.

Manifest signature (s).

```
(name: 'str', left: 'Any', right: 'Any', certificate: 'Certificate') -> None
```

Definition. Two computed sides of an integral identity and their certificate.

Contract (D, C, H, R, X). Identity constructors compute both sides from exact polynomials, finite measures, and exact affine maps, then compare them under the stated theorem. Fundamental-theorem and integration-by-parts identities require one-dimensional interval measures. Fubini identities require product measures and explicit variable orders. The all-orders routine enforces a factorial dimension budget. Principal public operations and properties are `valid`, `require`.

Guarantees (E, P, B, K). With exact inputs, both sides and equality checks use rational arithmetic and the certificate is exact. Callable pushforward assumptions are not silently discharged. Enumerating every Fubini order grows factorially, so dimensions above the explicit limit are rejected.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_block19_integration.py`. The independent verifier for core operators, algorithms,

and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapters 47 through 49 and Chapter 68.

C.1D · `IntegralResult`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.IntegralResult`. Reflected implementation `komorebi_math.integration.quadrature.IntegralResult`. The same runtime object is also available through `komorebi_math.integration.IntegralResult`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1D-INT-QUADRATURE`.

Manifest signature (`s`).

```
(value: 'Any', certificate: 'Certificate', error: 'IntegrationError', method: 'str', evaluations: 'int') -> None
```

Definition. An integral value with method, evaluation count, certificate, and error.

Contract (`D, C, H, R, X`). Integration accepts an exact polynomial or a callable and one finite measure. Exact polynomials use moments when available. Atomic callables use finite weighted sums. Interval callables use composite Simpson quadrature with an even positive subinterval count and evaluation budget. Derivative bounds, error guarantees, orientation, and failure status remain explicit in the returned records. Principal public operations and properties are `valid`, `exact`, `require`.

Guarantees (`E, P, B, K`). Exact polynomial paths return exact values when all required moments are exact. Simpson bounds are rigorous only when a valid fourth-derivative bound applies to the interval. Otherwise the error status is conditional, estimated, or unknown as recorded. Numerical evaluation count is finite and bounded by the provider budget.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_block19_integration.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapters 47 through 49 and Chapter 68.

C.1D · `IntegrationError`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.IntegrationError`. Reflected implementation `komorebi_math.integration.quadrature.IntegrationError`. The same runtime object is also available through `komorebi_math.integration.IntegrationError`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1D-INT-QUADRATURE`.

Manifest signature (`s`).

```
(absolute_bound: 'Fraction | float | None', guarantee: 'ErrorGuarantee', sources: 'tuple[str, ...]' = ()) -> None
```

Definition. One absolute error statement attached to an integral result.

Contract (`D, C, H, R, X`). Integration accepts an exact polynomial or a callable and one finite measure. Exact polynomials use moments when available. Atomic callables use finite weighted sums. Interval callables use composite Simpson quadrature with an even positive subinterval count and evaluation budget. Derivative bounds, error guarantees, orientation, and failure status remain explicit in the returned records. Principal public operations and properties are `rigorous`, `exact`.

Guarantees (`E, P, B, K`). Exact polynomial paths return exact values when all required moments are exact. Simpson bounds are rigorous only when a valid fourth-derivative bound applies to the interval. Otherwise the error status is conditional, estimated, or unknown as recorded. Numerical evaluation count is finite and bounded by the provider budget.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_block19_integration.py`. The independent verifier for core operators, algorithms,

and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapters 47 through 49 and Chapter 68.

C.1D · `IntervalMeasure`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.IntervalMeasure`. Reflected implementation `komorebi_math.integration.measures.IntervalMeasure`. The same runtime object is also available through `komorebi_math.integration.IntervalMeasure`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1D-INT-MEASURE`.

Manifest signature (`s`).

```
(lower: 'Any', upper: 'Any', *, density: 'Any' = 1, name: 'str' = 'interval measure') -> 'None'
```

Definition. Constant-density Lebesgue measure on one bounded interval.

Contract (`D, C, H, R, X`). Measures expose finite dimension, mass, moments, exactness, and a structural certificate. Atomic measures require compatible finite points and nonnegative finite weights. Interval measures use finite ordered bounds and nonnegative constant density. Products preserve factor order. Exact affine pushforwards validate dimensions and Jacobian data, while arbitrary callable pushforwards retain explicit unchecked regularity and purity assumptions. Principal public operations and properties are `structurally_zero`, `mass`, `moment`, `normalized`, `pushforward_affine`, `certificate`.

Guarantees (`E, P, B, K`). Integer and `Fraction` data remain exact. Floating data are checked for finiteness and produce numerical or conditional evidence. Atomic work is linear in atom count, product moments multiply factor results, and affine polynomial pullback can grow combinatorially in term count. Callable pushforwards are definitions with stated assumptions, not global regularity proofs.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_block19_integration.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapters 47 through 49 and Chapter 68.

C.1D · `Measure`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.Measure`. Reflected implementation `komorebi_math.integration.measures.Measure`. The same runtime object is also available through `komorebi_math.integration.Measure`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1D-INT-MEASURE`.

Manifest signature (`s`).

```
(*args, **kwargs)
```

Definition. Protocol for finite measures usable by `integrate`.

Contract (`D, C, H, R, X`). Measures expose finite dimension, mass, moments, exactness, and a structural certificate. Atomic measures require compatible finite points and nonnegative finite weights. Interval measures use finite ordered bounds and nonnegative constant density. Products preserve factor order. Exact affine pushforwards validate dimensions and Jacobian data, while arbitrary callable pushforwards retain explicit unchecked regularity and purity assumptions. Principal public operations and properties are `mass`, `moment`, `certificate`.

Guarantees (`E, P, B, K`). Integer and `Fraction` data remain exact. Floating data are checked for finiteness and produce numerical or conditional evidence. Atomic work is linear in atom count, product moments multiply factor results, and affine polynomial pullback can grow combinatorially in term count. Callable pushforwards are definitions with stated assumptions, not global regularity proofs.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_block19_integration.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapters 47 through 49 and Chapter 68.

C.1D · PolynomialIntegrationCovector

Identity (q, a, t, k). Public research-manifest route `komorebi_math.PolynomialIntegrationCovector`. Reflected implementation `komorebi_math.integration.coordinates.PolynomialIntegrationCovector`. The same runtime object is also available through `komorebi_math.integration.PolynomialIntegrationCovector`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1D-INT-COORDINATE`.

Manifest signature (s).

```
(family: 'str', degree: 'int', measure: 'IntervalMeasure', weights: 'tuple[Fraction, ...]', basis_witness:
    ↪ 'tuple[tuple[Fraction, ...], ...] | None' = None) -> None
```

Definition. The integral functional in one finite polynomial basis.

Contract (D, C, H, R, X). A polynomial integration covector stores the exact interval moments of one finite polynomial basis. Construction validates family, degree, interval measure, and basis witness. Coordinate identities compare the integration covector with derivative, endpoint-evaluation, or basis-change matrices using exact finite linear algebra. Principal public operations and properties are `basis_verified`, `apply`, `certificate`.

Guarantees (E, P, B, K). A covector built by `polynomial_integration_covector` carries replayable basis provenance and exact rational weights. A manually supplied covector without a basis witness remains a finite linear functional but its family interpretation is conditional. Dense basis conversion can require cubic exact matrix work.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_block19_integration.py` and `tests/test_coordinates_polynomials.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapters 47 through 49 and Chapters 64 and 68.

C.1D · ProductMeasure

Identity (q, a, t, k). Public research-manifest route `komorebi_math.ProductMeasure`. Reflected implementation `komorebi_math.integration.measures.ProductMeasure`. The same runtime object is also available through `komorebi_math.integration.ProductMeasure`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1D-INT-MEASURE`.

Manifest signature (s).

```
(factors: 'Iterable[Measure]', *, name: 'str' = 'product measure') -> 'None'
```

Definition. Finite product of typed component measures.

Contract (D, C, H, R, X). Measures expose finite dimension, mass, moments, exactness, and a structural certificate. Atomic measures require compatible finite points and nonnegative finite weights. Interval measures use finite ordered bounds and nonnegative constant density. Products preserve factor order. Exact affine pushforwards validate dimensions and Jacobian data, while arbitrary callable pushforwards retain explicit unchecked regularity and purity assumptions. Principal public operations and properties are `mass`, `moment`, `certificate`.

Guarantees (E, P, B, K). Integer and `Fraction` data remain exact. Floating data are checked for finiteness and produce numerical or conditional evidence. Atomic work is linear in atom count, product moments multiply factor results,

and affine polynomial pullback can grow combinatorially in term count. Callable pushforwards are definitions with stated assumptions, not global regularity proofs.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_block19_integration.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapters 47 through 49 and Chapter 68.

C.1D · `PushforwardMeasure`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.PushforwardMeasure`. Reflected implementation `komorebi_math.integration.measures.PushforwardMeasure`. The same runtime object is also available through `komorebi_math.integration.PushforwardMeasure`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1D-INT-MEASURE`.

Manifest signature (s).

```
(base: 'Measure', mapping: 'ExactAffineMap | Callable[[Any], Any]', *, target_dimension: 'int | None' = None, name:
   ↪  'str' = 'pushforward measure') -> 'None'
```

Definition. The pushforward `mapping_# base`.

Contract (D, C, H, R, X). Measures expose finite dimension, mass, moments, exactness, and a structural certificate. Atomic measures require compatible finite points and nonnegative finite weights. Interval measures use finite ordered bounds and nonnegative constant density. Products preserve factor order. Exact affine pushforwards validate dimensions and Jacobian data, while arbitrary callable pushforwards retain explicit unchecked regularity and purity assumptions. Principal public operations and properties are `mass`, `map_point`, `materialize_atomic`, `moment`, `certificate`.

Guarantees (E, P, B, K). Integer and `Fraction` data remain exact. Floating data are checked for finiteness and produce numerical or conditional evidence. Atomic work is linear in atom count, product moments multiply factor results, and affine polynomial pullback can grow combinatorially in term count. Callable pushforwards are definitions with stated assumptions, not global regularity proofs.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_block19_integration.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapters 47 through 49 and Chapter 68.

C.1D · `QuadratureExactness`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.QuadratureExactness`. Reflected implementation `komorebi_math.integration.rules.QuadratureExactness`. The same runtime object is also available through `komorebi_math.integration.QuadratureExactness`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1D-INT-RULE`.

Manifest signature (s).

```
(degree: 'int', max_tested_degree: 'int', exact: 'bool', certificate: 'Certificate') -> None
```

Definition. Moment comparison between a quadrature functional and one measure.

Contract (D, C, H, R, X). A quadrature rule is a finite linear functional with nodes and weights of equal nonzero length. General rules may use signed or floating weights. Exact rules require rational nodes and weights. Moment comparison determines the tested degree of exactness, and tensor products preserve the stated node ordering and finite dimensionality.

Guarantees (E, P, B, K). Exact rules integrate supported exact polynomials through rational arithmetic. Numerical rules preserve measured residuals and do not claim exactness beyond tested moments. Tensor-product node count multiplies across factors and can grow exponentially with dimension.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_block19_integration.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapters 47 through 49 and Chapter 68.

C.1D · `QuadratureRule`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.QuadratureRule`. Reflected implementation `komorebi_math.integration.rules.QuadratureRule`. The same runtime object is also available through `komorebi_math.integration.QuadratureRule`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1D-INT-RULE`.

Manifest signature (s).

```
(points: 'Iterable[Any]', weights: 'Iterable[Any]', *, name: 'str' = 'quadrature rule', _require_exact: 'bool' =
   ↪ False) -> 'None'
```

Definition. A finite linear functional `Q[f]=sum_i w_i f(x_i)`.

Contract (D, C, H, R, X). A quadrature rule is a finite linear functional with nodes and weights of equal nonzero length. General rules may use signed or floating weights. Exact rules require rational nodes and weights. Moment comparison determines the tested degree of exactness, and tensor products preserve the stated node ordering and finite dimensionality. Principal public operations and properties are `node_count`, `exact_data`, `positive`, `apply`, `moment`, `integrate_polynomial`, `tensor_product`, `as_atomic_measure`, `degree_of_exactness`, `certify_polynomial_integral`, `certificate`.

Guarantees (E, P, B, K). Exact rules integrate supported exact polynomials through rational arithmetic. Numerical rules preserve measured residuals and do not claim exactness beyond tested moments. Tensor-product node count multiplies across factors and can grow exponentially with dimension.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_block19_integration.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapters 47 through 49 and Chapter 68.

C.1D · `affine_interval_change_of_variables`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.affine_interval_change_of_variables`. Reflected implementation `komorebi_math.integration.identities.affine_interval_change_of_variables`. The same runtime object is also available through `komorebi_math.integration.affine_interval_change_of_variables`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1D-INT-IDENTITY`.

Manifest signature (s).

```
(polynomial: 'ExactPolynomial', base: 'IntervalMeasure', mapping: 'ExactAffineMap') -> 'IntegralIdentity'
```

Definition. Certify the one-dimensional affine Jacobian density law exactly.

Contract (D, C, H, R, X). Identity constructors compute both sides from exact polynomials, finite measures, and exact affine maps, then compare them under the stated theorem. Fundamental-theorem and integration-by-parts identities require one-dimensional interval measures. Fubini identities require product measures and explicit variable orders. The all-orders routine enforces a factorial dimension budget.

Guarantees (E, P, B, K). With exact inputs, both sides and equality checks use rational arithmetic and the certificate is exact. Callable pushforward assumptions are not silently discharged. Enumerating every Fubini order grows factorially, so dimensions above the explicit limit are rejected.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_block19_integration.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapters 47 through 49 and Chapter 68.

C.1D · `all_fubini_orders`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.all_fubini_orders`. Reflected implementation `komorebi_math.integration.identities.all_fubini_orders`. The same runtime object is also available through `komorebi_math.integration.all_fubini_orders`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1D-INT-IDENTITY`.

Manifest signature (s).

```
(polynomial: 'ExactPolynomial', measure: 'ProductMeasure', *, max_dimensions: 'int' = 7) ->
    ↪ 'tuple[IntegralIdentity, ...]'
```

Definition. Certify every variable order within an explicit factorial budget.

Contract (D, C, H, R, X). Identity constructors compute both sides from exact polynomials, finite measures, and exact affine maps, then compare them under the stated theorem. Fundamental-theorem and integration-by-parts identities require one-dimensional interval measures. Fubini identities require product measures and explicit variable orders. The all-orders routine enforces a factorial dimension budget.

Guarantees (E, P, B, K). With exact inputs, both sides and equality checks use rational arithmetic and the certificate is exact. Callable pushforward assumptions are not silently discharged. Enumerating every Fubini order grows factorially, so dimensions above the explicit limit are rejected.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_block19_integration.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapters 47 through 49 and Chapter 68.

C.1D · `analysis_synthesis_integration_certificate`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.analysis_synthesis_integration_certificate`. Reflected implementation `komorebi_math.integration.adapters.analysis_synthesis_integration_certificate`. The same runtime object is also available through `komorebi_math.integration.analysis_synthesis_integration_certificate`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1D-INT-ADAPTER`.

Manifest signature (s).

```
(system: 'AnalysisSynthesisSystem', measure: 'AtomicMeasure', physical_values: 'Sequence[Any]', *, tolerance:
    ↪ 'float' = 1e-10) -> 'Certificate'
```

Definition. Check that a fixed analysis/synthesis system commutes with finite integration.

Contract (D, C, H, R, X). Adapters convert finite formal-series or polynomial-family coordinates into `ExactPolynomial`, integrate through the ordinary measure interface, or combine compatible derivative providers with fixed scalar weights. Degree truncation must be explicit. Analysis-synthesis integration checks compare physical and coefficient-side finite sums under one declared system and atomic measure.

Guarantees (E, P, B, K). Conversions are exact when coefficients, family data, and measures are exact. Truncation is reported rather than hidden. Provider sums preserve the weakest child certificate status. The atomic derivative adapter deliberately excludes parameter-dependent measure derivatives, which would require an additional term.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_block19_integration.py` and `tests/test_coordinates_polynomials.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapters 47 through 49 and Chapter 68.

C.1D · coordinate_fundamental_theorem

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinate_fundamental_theorem`. Reflected implementation `komorebi_math.integration.coordinates.coordinate_fundamental_theorem`. The same runtime object is also available through `komorebi_math.integration.coordinate_fundamental_theorem`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1D-INT-COORDINATE`.

Manifest signature (s).

```
(family: 'PolynomialFamily', degree: 'int', measure: 'IntervalMeasure') -> 'IntegralIdentity'
```

Definition. Certify `ell D = rho(ev_b - ev_a)` in the declared basis.

Contract (D, C, H, R, X). A polynomial integration covector stores the exact interval moments of one finite polynomial basis. Construction validates family, degree, interval measure, and basis witness. Coordinate identities compare the integration covector with derivative, endpoint-evaluation, or basis-change matrices using exact finite linear algebra.

Guarantees (E, P, B, K). A covector built by `polynomial_integration_covector` carries replayable basis provenance and exact rational weights. A manually supplied covector without a basis witness remains a finite linear functional but its family interpretation is conditional. Dense basis conversion can require cubic exact matrix work.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_block19_integration.py` and `tests/test_coordinates_polynomials.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapters 47 through 49 and Chapters 64 and 68.

C.1D · coordinate_transport_identity

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinate_transport_identity`. Reflected implementation `komorebi_math.integration.coordinates.coordinate_transport_identity`. The same runtime object is also available through `komorebi_math.integration.coordinate_transport_identity`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1D-INT-COORDINATE`.

Manifest signature (s).

```
(source: 'PolynomialFamily', target: 'PolynomialFamily', degree: 'int', measure: 'IntervalMeasure') ->
   ↪ 'IntegralIdentity'
```

Definition. Certify contragredient transport `ell_target C = ell_source`.

Contract (D, C, H, R, X). A polynomial integration covector stores the exact interval moments of one finite polynomial basis. Construction validates family, degree, interval measure, and basis witness. Coordinate identities compare the integration covector with derivative, endpoint-evaluation, or basis-change matrices using exact finite linear algebra.

Guarantees (E, P, B, K). A covector built by `polynomial_integration_covector` carries replayable basis provenance and exact rational weights. A manually supplied covector without a basis witness remains a finite linear functional but its family interpretation is conditional. Dense basis conversion can require cubic exact matrix work.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_block19_integration.py` and `tests/test_coordinates_polynomials.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapters 47 through 49 and Chapters 64 and 68.

C.1D · `fubini_identity`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.fubini_identity`. Reflected implementation `komorebi_math.integration.identities.fubini_identity`. The same runtime object is also available through `komorebi_math.integration.fubini_identity`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1D-INT-IDENTITY`.

Manifest signature (s).

```
(polynomial: 'ExactPolynomial', measure: 'ProductMeasure', *, order: 'tuple[int, ...] | None' = None) ->
    ↪ 'IntegralIdentity'
```

Definition. Certify direct versus iterated integration on an exact interval box.

Contract (D, C, H, R, X). Identity constructors compute both sides from exact polynomials, finite measures, and exact affine maps, then compare them under the stated theorem. Fundamental-theorem and integration-by-parts identities require one-dimensional interval measures. Fubini identities require product measures and explicit variable orders. The all-orders routine enforces a factorial dimension budget.

Guarantees (E, P, B, K). With exact inputs, both sides and equality checks use rational arithmetic and the certificate is exact. Callable pushforward assumptions are not silently discharged. Enumerating every Fubini order grows factorially, so dimensions above the explicit limit are rejected.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_block19_integration.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapters 47 through 49 and Chapter 68.

C.1D · `fundamental_theorem`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.fundamental_theorem`. Reflected implementation `komorebi_math.integration.identities.fundamental_theorem`. The same runtime object is also available through `komorebi_math.integration.fundamental_theorem`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1D-INT-IDENTITY`.

Manifest signature (s).

```
(polynomial: 'ExactPolynomial', measure: 'IntervalMeasure') -> 'IntegralIdentity'
```

Definition. Certify `integral p d(rho x) = rho(F(b)-F(a))` exactly.

Contract (D, C, H, R, X). Identity constructors compute both sides from exact polynomials, finite measures, and exact affine maps, then compare them under the stated theorem. Fundamental-theorem and integration-by-parts identities require one-dimensional interval measures. Fubini identities require product measures and explicit variable orders. The all-orders routine enforces a factorial dimension budget.

Guarantees (E, P, B, K). With exact inputs, both sides and equality checks use rational arithmetic and the certificate is exact. Callable pushforward assumptions are not silently discharged. Enumerating every Fubini order grows factorially, so dimensions above the explicit limit are rejected.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_block19_integration.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapters 47 through 49 and Chapter 68.

C.1D · integrate

Identity (q, a, t, k). Public research-manifest route `komorebi_math.integrate`. Reflected implementation `komorebi_math.integration.quadrature.integrate`. The same runtime object is also available through `komorebi_math.integration.integrate`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1D-INT-QUADRATURE`.

Manifest signature (s).

```
(integrand: 'ExactPolynomial | Callable[[Any], Any]', measure: 'Measure', *, provider: 'CompositeSimpson | None' =
    ↪ None) -> 'IntegralResult'
```

Definition. Integrate an exact polynomial or callable against a finite measure.

Contract (D, C, H, R, X). Integration accepts an exact polynomial or a callable and one finite measure. Exact polynomials use moments when available. Atomic callables use finite weighted sums. Interval callables use composite Simpson quadrature with an even positive subinterval count and evaluation budget. Derivative bounds, error guarantees, orientation, and failure status remain explicit in the returned records.

Guarantees (E, P, B, K). Exact polynomial paths return exact values when all required moments are exact. Simpson bounds are rigorous only when a valid fourth-derivative bound applies to the interval. Otherwise the error status is conditional, estimated, or unknown as recorded. Numerical evaluation count is finite and bounded by the provider budget.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_block19_integration.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapters 47 through 49 and Chapter 68.

C.1D · integrate_derivative_provider

Identity (q, a, t, k). Public research-manifest route `komorebi_math.integrate_derivative_provider`. Reflected implementation `komorebi_math.integration.quadrature.integrate_derivative_provider`. The same runtime object is also available through `komorebi_math.integration.integrate_derivative_provider`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1D-INT-QUADRATURE`.

Manifest signature (s).

```
(provider: 'Any', measure: 'Measure', *, quadrature: 'CompositeSimpson | None' = None) -> 'IntegralResult'
```

Definition. Integrate the value map of an existing finite derivative provider.

Contract (D, C, H, R, X). Integration accepts an exact polynomial or a callable and one finite measure. Exact polynomials use moments when available. Atomic callables use finite weighted sums. Interval callables use composite Simpson quadrature with an even positive subinterval count and evaluation budget. Derivative bounds, error guarantees, orientation, and failure status remain explicit in the returned records.

Guarantees (E, P, B, K). Exact polynomial paths return exact values when all required moments are exact. Simpson bounds are rigorous only when a valid fourth-derivative bound applies to the interval. Otherwise the error status is conditional, estimated, or unknown as recorded. Numerical evaluation count is finite and bounded by the provider budget.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_block19_integration.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapters 47 through 49 and Chapter 68.

C.1D · `integrate_formal_series`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.integrate_formal_series`. Reflected implementation `komorebi_math.integration.adapters.integrate_formal_series`. The same runtime object is also available through `komorebi_math.integration.integrate_formal_series`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1D-INT-ADAPTER`.

Manifest signature (s).

```
(series: 'FormalSeries', measure: 'Measure', *, degree: 'int | None' = None) -> 'IntegralResult'
```

Definition. Convert a finite truncation of a normalized formal series to an exact polynomial and integrate it against a finite measure.

Contract (D, C, H, R, X). Adapters convert finite formal-series or polynomial-family coordinates into `ExactPolynomial`, integrate through the ordinary measure interface, or combine compatible derivative providers with fixed scalar weights. Degree truncation must be explicit. Analysis-synthesis integration checks compare physical and coefficient-side finite sums under one declared system and atomic measure.

Guarantees (E, P, B, K). Conversions are exact when coefficients, family data, and measures are exact. Truncation is reported rather than hidden. Provider sums preserve the weakest child certificate status. The atomic derivative adapter deliberately excludes parameter-dependent measure derivatives, which would require an additional term.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_block19_integration.py` and `tests/test_coordinates_polynomials.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapters 47 through 49 and Chapter 68.

C.1D · `integrate_polynomial_coordinates`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.integrate_polynomial_coordinates`. Reflected implementation `komorebi_math.integration.adapters.integrate_polynomial_coordinates`. The same runtime object is also available through `komorebi_math.integration.integrate_polynomial_coordinates`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1D-INT-ADAPTER`.

Manifest signature (s).

```
(family: 'PolynomialFamily', coefficients: 'Iterable[Rational]', measure: 'Measure') -> 'IntegralResult'
```

Definition. Convert coefficients in an exact polynomial family to ordinary polynomial form and integrate the result.

Contract (D, C, H, R, X). Adapters convert finite formal-series or polynomial-family coordinates into `ExactPolynomial`, integrate through the ordinary measure interface, or combine compatible derivative providers with fixed scalar weights. Degree truncation must be explicit. Analysis-synthesis integration checks compare physical and coefficient-side finite sums under one declared system and atomic measure.

Guarantees (E, P, B, K). Conversions are exact when coefficients, family data, and measures are exact. Truncation is reported rather than hidden. Provider sums preserve the weakest child certificate status. The atomic derivative adapter deliberately excludes parameter-dependent measure derivatives, which would require an additional term.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_block19_integration.py` and `tests/test_coordinates_polynomials.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapters 47 through 49 and Chapter 68.

C.1D · `integrate_polynomial_oriented`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.integrate_polynomial_oriented`. Reflected implementation `komorebi_math.integration.quadrature.integrate_polynomial_oriented`. The same runtime object is also available through `komorebi_math.integration.integrate_polynomial_oriented`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1D-INT-QUADRATURE`.

Manifest signature (s).

```
(polynomial: 'ExactPolynomial', lower: 'int | Fraction', upper: 'int | Fraction', *, density: 'int | Fraction' = 1)
   ↪  -> 'IntegralResult'
```

Definition. Integrate over oriented bounds without mislabeling orientation as a measure.

Contract (D, C, H, R, X). Integration accepts an exact polynomial or a callable and one finite measure. Exact polynomials use moments when available. Atomic callables use finite weighted sums. Interval callables use composite Simpson quadrature with an even positive subinterval count and evaluation budget. Derivative bounds, error guarantees, orientation, and failure status remain explicit in the returned records.

Guarantees (E, P, B, K). Exact polynomial paths return exact values when all required moments are exact. Simpson bounds are rigorous only when a valid fourth-derivative bound applies to the interval. Otherwise the error status is conditional, estimated, or unknown as recorded. Numerical evaluation count is finite and bounded by the provider budget.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_block19_integration.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapters 47 through 49 and Chapter 68.

C.1D · `integration_by_parts`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.integration_by_parts`. Reflected implementation `komorebi_math.integration.identities.integration_by_parts`. The same runtime object is also available through `komorebi_math.integration.integration_by_parts`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1D-INT-IDENTITY`.

Manifest signature (s).

```
(left_factor: 'ExactPolynomial', right_factor: 'ExactPolynomial', measure: 'IntervalMeasure') -> 'IntegralIdentity'
```

Definition. Certify `int u v' = rho[uv]_a^b - int u' v` exactly.

Contract (D, C, H, R, X). Identity constructors compute both sides from exact polynomials, finite measures, and exact affine maps, then compare them under the stated theorem. Fundamental-theorem and integration-by-parts identities require one-dimensional interval measures. Fubini identities require product measures and explicit variable orders. The all-orders routine enforces a factorial dimension budget.

Guarantees (E, P, B, K). With exact inputs, both sides and equality checks use rational arithmetic and the certificate is exact. Callable pushforward assumptions are not silently discharged. Enumerating every Fubini order grows factorially, so dimensions above the explicit limit are rejected.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_block19_integration.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapters 47 through 49 and Chapter 68.

C.1D · `polynomial_derivative_bound`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.polynomial_derivative_bound`. Reflected implementation `komorebi_math.integration.quadrature.polynomial_derivative_bound`. The same runtime object is also available through `komorebi_math.integration.polynomial_derivative_bound`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1D-INT-QUADRATURE`.

Manifest signature (s).

```
(polynomial: 'ExactPolynomial', measure: 'IntervalMeasure', *, order: 'int' = 4) -> 'DerivativeBound'
```

Definition. Create a rigorous coefficient-triangle derivative bound.

Contract (D, C, H, R, X). Integration accepts an exact polynomial or a callable and one finite measure. Exact polynomials use moments when available. Atomic callables use finite weighted sums. Interval callables use composite Simpson quadrature with an even positive subinterval count and evaluation budget. Derivative bounds, error guarantees, orientation, and failure status remain explicit in the returned records.

Guarantees (E, P, B, K). Exact polynomial paths return exact values when all required moments are exact. Simpson bounds are rigorous only when a valid fourth-derivative bound applies to the interval. Otherwise the error status is conditional, estimated, or unknown as recorded. Numerical evaluation count is finite and bounded by the provider budget.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_block19_integration.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapters 47 through 49 and Chapter 68.

C.1D · `polynomial_from_family_coordinates`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.polynomial_from_family_coordinates`. Reflected implementation `komorebi_math.integration.adapters.polynomial_from_family_coordinates`. The same runtime object is also available through `komorebi_math.integration.polynomial_from_family_coordinates`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1D-INT-ADAPTER`.

Manifest signature (s).

```
(family: 'PolynomialFamily', coefficients: 'Iterable[Rational]') -> 'ExactPolynomial'
```

Definition. Convert finite coordinates in any exact polynomial family to a polynomial.

Contract (D, C, H, R, X). Adapters convert finite formal-series or polynomial-family coordinates into `ExactPolynomial`, integrate through the ordinary measure interface, or combine compatible derivative providers with fixed scalar weights. Degree truncation must be explicit. Analysis-synthesis integration checks compare physical and coefficient-side finite sums under one declared system and atomic measure.

Guarantees (E, P, B, K). Conversions are exact when coefficients, family data, and measures are exact. Truncation is reported rather than hidden. Provider sums preserve the weakest child certificate status. The atomic derivative adapter deliberately excludes parameter-dependent measure derivatives, which would require an additional term.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_block19_integration.py` and `tests/test_coordinates_polynomials.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapters 47 through 49 and Chapter 68.

C.1D · `polynomial_from_formal_series`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.polynomial_from_formal_series`. Reflected implementation `komorebi_math.integration.adapters.polynomial_from_formal_series`. The same runtime object is also available through `komorebi_math.integration.polynomial_from_formal_series`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1D-INT-ADAPTER`.

Manifest signature (s).

```
(series: 'FormalSeries', *, degree: 'int | None' = None) -> 'ExactPolynomial'
```

Definition. Convert one finite normalized formal series to ordinary polynomial form.

Contract (D, C, H, R, X). Adapters convert finite formal-series or polynomial-family coordinates into `ExactPolynomial`, integrate through the ordinary measure interface, or combine compatible derivative providers with fixed scalar weights. Degree truncation must be explicit. Analysis-synthesis integration checks compare physical and coefficient-side finite sums under one declared system and atomic measure.

Guarantees (E, P, B, K). Conversions are exact when coefficients, family data, and measures are exact. Truncation is reported rather than hidden. Provider sums preserve the weakest child certificate status. The atomic derivative adapter deliberately excludes parameter-dependent measure derivatives, which would require an additional term.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_block19_integration.py` and `tests/test_coordinates_polynomials.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapters 47 through 49 and Chapter 68.

C.1D · `polynomial_integration_covector`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.polynomial_integration_covector`. Reflected implementation `komorebi_math.integration.coordinates.polynomial_integration_covector`. The same runtime object is also available through `komorebi_math.integration.polynomial_integration_covector`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1D-INT-COORDINATE`.

Manifest signature (s).

```
(family: 'PolynomialFamily', degree: 'int', measure: 'IntervalMeasure') -> 'PolynomialIntegrationCovector'
```

Definition. Construct the exact interval-integration functional in a declared finite polynomial basis.

Contract (D, C, H, R, X). A polynomial integration covector stores the exact interval moments of one finite polynomial basis. Construction validates family, degree, interval measure, and basis witness. Coordinate identities compare the integration covector with derivative, endpoint-evaluation, or basis-change matrices using exact finite linear algebra.

Guarantees (E, P, B, K). A covector built by `polynomial_integration_covector` carries replayable basis provenance and exact rational weights. A manually supplied covector without a basis witness remains a finite linear functional but its family interpretation is conditional. Dense basis conversion can require cubic exact matrix work.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_block19_integration.py` and `tests/test_coordinates_polynomials.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapters 47 through 49 and Chapters 64 and 68.

C.1D · `product_measure`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.product_measure`. Reflected implementation `komorebi_math.integration.measures.product_measure`. The same runtime object is also available through `komorebi_math.integration.product_measure`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1D-INT-MEASURE`.

Manifest signature (s).

```
(*factors: 'Measure', name: 'str' = 'product measure') -> 'ProductMeasure'
```

Definition. Construct a finite product measure from the supplied factors.

Contract (D, C, H, R, X). Measures expose finite dimension, mass, moments, exactness, and a structural certificate. Atomic measures require compatible finite points and nonnegative finite weights. Interval measures use finite ordered bounds and nonnegative constant density. Products preserve factor order. Exact affine pushforwards validate dimensions and Jacobian data, while arbitrary callable pushforwards retain explicit unchecked regularity and purity assumptions.

Guarantees (E, P, B, K). Integer and `Fraction` data remain exact. Floating data are checked for finiteness and produce numerical or conditional evidence. Atomic work is linear in atom count, product moments multiply factor results, and affine polynomial pullback can grow combinatorially in term count. Callable pushforwards are definitions with stated assumptions, not global regularity proofs.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_block19_integration.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapters 47 through 49 and Chapter 68.

C.1D · `pushforward_identity`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.pushforward_identity`. Reflected implementation `komorebi_math.integration.identities.pushforward_identity`. The same runtime object is also available through `komorebi_math.integration.pushforward_identity`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1D-INT-IDENTITY`.

Manifest signature (s).

```
(polynomial: 'ExactPolynomial', base: 'Measure', mapping: 'ExactAffineMap') -> 'IntegralIdentity'
```

Definition. Certify `int f d(mapping_# mu) = int (f o mapping) dmu`.

Contract (D, C, H, R, X). Identity constructors compute both sides from exact polynomials, finite measures, and exact affine maps, then compare them under the stated theorem. Fundamental-theorem and integration-by-parts identities require one-dimensional interval measures. Fubini identities require product measures and explicit variable orders. The all-orders routine enforces a factorial dimension budget.

Guarantees (E, P, B, K). With exact inputs, both sides and equality checks use rational arithmetic and the certificate is exact. Callable pushforward assumptions are not silently discharged. Enumerating every Fubini order grows factorially, so dimensions above the explicit limit are rejected.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_block19_integration.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapters 47 through 49 and Chapter 68.

C.1D · `pushforward_measure`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.pushforward_measure`. Reflected implementation `komorebi_math.integration.measures.pushforward_measure`. The same runtime object is also available through `komorebi_math.integration.pushforward_measure`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This card does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1D-INT-MEASURE`.

Manifest signature (s).

```
(base: 'Measure', mapping: 'ExactAffineMap | Callable[[Any], Any]', *, target_dimension: 'int | None' = None, name:
    ↪ 'str' = 'pushforward measure') -> 'PushforwardMeasure'
```

Definition. Construct the pushforward of a finite base measure through an exact affine map or declared callable.

Contract (D, C, H, R, X). Measures expose finite dimension, mass, moments, exactness, and a structural certificate. Atomic measures require compatible finite points and nonnegative finite weights. Interval measures use finite ordered bounds and nonnegative constant density. Products preserve factor order. Exact affine pushforwards validate dimensions and Jacobian data, while arbitrary callable pushforwards retain explicit unchecked regularity and purity assumptions.

Guarantees (E, P, B, K). Integer and `Fraction` data remain exact. Floating data are checked for finiteness and produce numerical or conditional evidence. Atomic work is linear in atom count, product moments multiply factor results, and affine polynomial pullback can grow combinatorially in term count. Callable pushforwards are definitions with stated assumptions, not global regularity proofs.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused evidence is `tests/test_block19_integration.py`. The independent verifier for core operators, algorithms, and integration exercises representative construction, invariant, failure, exactness, numerical-residual, and resource paths for this contract family. The source validator for core operators, algorithms, and integration checks the exact signature, implementation identity, alias routes, card counts, unique anchor, and preservation of the reference cards and formulas. See Chapters 47 through 49 and Chapter 68.

C.1E Root nonlinear calculus, packet-jet, and execution facades: 90 entries

This root section documents the top-level import routes for the nonlinear subsystem documented in Section C.5. These routes expose derivative providers, composition machinery, obstruction and lifting objects, packet jets, sparse execution plans, snapshots, resumable streams, profiling records, optional accelerator boundaries, and ordered rooted-tree expansions directly through `komorebi_math`. Each card retains the domain contract while making the root alias, defining implementation, signature, and stability boundary explicit.

Contract family	Cards
Derivatives and composition	15
Defects, lifting, grading, and transport	27
Packet jets and sparse tensors	7
Execution, snapshots, streaming, and profiling	35
Rooted-tree expansions and majorants	6
Total	**90**

The facade does not create a second implementation. Every callable or class card requires identity with its `komorebi_math.nonlinear` route and its reflected owner. Optional JAX, Torch, and CuPy symbols retain their unavailable-

backend contracts when the corresponding runtime is absent. The recorded CPU-safe validation does not constitute framework or accelerator qualification.

Derivatives and composition (15 cards).

C.1E · AutomaticDifferentiationProvider

Identity (q, a, t, k). Public research-manifest route `komorebi_math.AutomaticDifferentiationProvider`. Reflected implementation `komorebi_math.nonlinear.autodiff.AutomaticDifferentiationProvider`. The same runtime object is also available through `komorebi_math.nonlinear.AutomaticDifferentiationProvider`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1E-DERIVATIVE`.

Manifest signature (s).

```
(function: 'Callable[[Any], Any]', jvp: 'DirectionalJVP', source_dimension: 'int', target_dimension: 'int',
   ↪ max_order: 'int' = 4, exactness: 'Exactness' = <Exactness.NUMERICAL: 'numerical'>, name: 'str' = 'automatic
   ↪ differentiation provider') -> None
```

Definition. Framework-neutral higher derivatives obtained by nested JVP calls.

Contract (D, C, H, R, X). Automatic differentiation adapters require a callable finite dimensional map, positive source and target dimensions, a declared maximum order, and a directional Jacobian vector product primitive supplied either directly or by an optional framework. Direction counts must equal the requested derivative order. Framework discovery is isolated in child processes so that a broken optional runtime cannot terminate the calling process. Principal public operations and properties are `supports_order`, `value`, `derivative`, `validation_certificate`, `to_dict`.

Guarantees (E, P, B, K). A successful provider returns the finite directional derivative computed by the selected JVP recursion and records numerical exactness unless the caller supplies a stronger justified classification. Framework availability means that an isolated probe imported the runtime and found the required JVP capability. It does not assert numerical agreement for every model or device. Work grows with derivative order and the number of nested JVP calls. The provenance registry records `PSC-III-N04A` as Directional automatic-differentiation adapter. Its recorded status is `definition`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_nonlinear_sparse_autodiff.py`, `tests/test_block10_sparse_autodiff.py`, and `tests/test_optional_backend_probe.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 37.

C.1E · CallableDerivativeProvider

Identity (q, a, t, k). Public research-manifest route `komorebi_math.CallableDerivativeProvider`. Reflected implementation `komorebi_math.nonlinear.derivatives.CallableDerivativeProvider`. The same runtime object is also available through `komorebi_math.nonlinear.CallableDerivativeProvider`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1E-DERIVATIVE`.

Manifest signature (s).

```
(function: 'Callable[[Any], Any]', derivatives: 'Mapping[int, DirectionalDerivative]', source_dimension: 'int',
   ↪ target_dimension: 'int', exactness: 'Exactness' = <Exactness.NUMERICAL: 'numerical'>, name: 'str' =
   ↪ 'declared derivative provider') -> None
```

Definition. A map with explicitly supplied directional derivative callables.

Contract (D, C, H, R, X). Derivative providers expose one common finite dimensional interface for map evaluation and directional derivatives. A call must use an order supported by the provider and exactly that many direction vectors of the declared source dimension. Callable and polynomial providers trust explicitly supplied derivative data. The finite difference provider additionally requires a positive finite step. The SymPy provider requires the

optional symbolic dependency. Principal public operations and properties are `available_orders`, `supports_order`, `value`, `derivative`, `validation_certificate`.

Guarantees (E, P, B, K). Polynomial and symbolic providers preserve exact objects where their inputs permit exact arithmetic. Callable providers inherit the exactness declared by the caller. Finite differences return numerical evidence only. Every provider checks dimensions and finite values before evaluation. Tensor contraction cost grows with derivative order and ambient dimension. The provenance registry records `PSC-III-D02` as Declared finite dimensional derivative provider. Its recorded status is `definition`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_nonlinear_jets_composition.py` and `tests/test_nonlinear_sparse_autodiff.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapters 37 and 38.

C.1E · `ComposedDerivativeProvider`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.ComposedDerivativeProvider`. Reflected implementation `komorebi_math.nonlinear.composition.ComposedDerivativeProvider`. The same runtime object is also available through `komorebi_math.nonlinear.ComposedDerivativeProvider`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1E-DERIVATIVE`.

Manifest signature (s).

```
(outer: 'DerivativeProvider', inner: 'DerivativeProvider', *, name: 'str | None' = None) -> 'None'
```

Definition. Expose a composition as another derivative provider.

Contract (D, C, H, R, X). Composition requires compatible inner and outer dimensions and derivative support through the requested order. The Faà di Bruno routines enumerate set partitions of the direction labels. Packet composition also requires compatible source, intermediate, and target packet systems. Partition utilities accept nonnegative finite orders and return canonical immutable records. Principal public operations and properties are `supports_order`, `value`, `derivative`.

Guarantees (E, P, B, K). For the composition operations in this family, the computed derivative value equals the implemented finite Faà di Bruno sum for the supplied providers and directions. Exactness follows the participating providers. The composition result records retained terms, partition counts, and a certificate comparing the composed derivative with the direct implementation when that comparison is available. Enumeration grows according to Bell and Stirling counts, so callers must keep the derivative order bounded. The provenance registry records `PSC-III-D12` as Composed derivative provider. Its recorded status is `definition`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test module is `tests/test_nonlinear_jets_composition.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 38.

Execution, snapshots, streaming, and profiling (35 cards).

C.1E · `CuPyDeviceDerivativeProvider`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.CuPyDeviceDerivativeProvider`. Reflected implementation `komorebi_math.nonlinear.execution.CupyDeviceDerivativeProvider`. The same runtime object is also available through `komorebi_math.nonlinear.CuPyDeviceDerivativeProvider`. The public export name maps to implementation object `CupyDeviceDerivativeProvider`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1E-EXECUTION`.

Manifest signature (`s`).

```
(*args, **kwargs)
```

Definition. Optional protocol consumed by `CupyPacketJetExecutor`.

Contract (`D, C, H, R, X`). Execution plans require a finite packet-jet family and an optional support record for the same order and packet names. Only blocks certified zero may be pruned. Plan compilation enforces an explicit address budget. Execution requires one direction per derivative slot and checks that the plan still matches the family. CuPy execution additionally requires an importable CuPy runtime and a CUDA device. Principal public operations and properties are `supports_order`, `device_derivative`.

Guarantees (`E, P, B, K`). The CPU planner groups active output blocks by common projected inputs, caches those projections, and reports derivative-call and projection counts. Reconstruction is compared with the direct represented derivative. Optional CuPy classes preserve the same finite packet contract when the hardware boundary is available. Runtime discovery alone is not a performance or correctness qualification, and unavailable device paths fail explicitly.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_nonlinear_execution.py` and `tests/test_block10_execution_persistence.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 43.

C.1E · `CuPyPacketJetExecutor`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.CuPyPacketJetExecutor`. Reflected implementation `komorebi_math.nonlinear.execution.CupyPacketJetExecutor`. The same runtime object is also available through `komorebi_math.nonlinear.CuPyPacketJetExecutor`. The public export name maps to implementation object `CupyPacketJetExecutor`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1E-EXECUTION`.

Manifest signature (`s`).

```
(family: 'PacketJetFamily', plan: 'PacketJetExecutionPlan | None' = None) -> 'None'
```

Definition. Device-resident packet projections and polynomial jet reconstruction.

Contract (`D, C, H, R, X`). Execution plans require a finite packet-jet family and an optional support record for the same order and packet names. Only blocks certified zero may be pruned. Plan compilation enforces an explicit address budget. Execution requires one direction per derivative slot and checks that the plan still matches the family. CuPy execution additionally requires an importable CuPy runtime and a CUDA device. Principal public operations and properties are `execute`, `synchronize`, `hardware_certificate`.

Guarantees (`E, P, B, K`). The CPU planner groups active output blocks by common projected inputs, caches those projections, and reports derivative-call and projection counts. Reconstruction is compared with the direct represented derivative. Optional CuPy classes preserve the same finite packet contract when the hardware boundary is available. Runtime discovery alone is not a performance or correctness qualification, and unavailable device paths fail explicitly.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_nonlinear_execution.py` and `tests/test_block10_execution_persistence.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 43.

C.1E · CuPyPacketJetSnapshotExecutor

Identity (q, a, t, k). Public research-manifest route komorebi_math.CuPyPacketJetSnapshotExecutor. Reflected implementation komorebi_math.nonlinear.gpu.CuPySnapshotExecutor. The same runtime object is also available through komorebi_math.nonlinear.CuPyPacketJetSnapshotExecutor. The public export name maps to implementation object CuPySnapshotExecutor. The root route is governed by [RESEARCH] policy pre-beta-additive. This facade does not enlarge the frozen stable compatibility promise. Manifest kind class. Profile C1E-EXECUTION.

Manifest signature (s).

```
(snapshot: 'PacketJetSnapshot', *, dtype: 'Any | None' = None) -> 'None'
```

Definition. Device evaluator for one sparse PacketJetSnapshot.

Contract (D, C, H, R, X). GPU status inspection never requires hardware and returns the observed CuPy import and CUDA device state. Snapshot execution requires a complete finite snapshot, a working CuPy CUDA runtime, and compatible probe shapes. Validation compares the same persisted coefficients on CPU and device under a declared tolerance. Principal public operations and properties are evaluate_batch, evaluate.

Guarantees (E, P, B, K). A successful validation record reports the probe count, maximum residual, runtime state, and certificate. The device executor evaluates only the captured finite snapshot and does not accept arbitrary Python callables. No speed claim follows from API presence. In a runtime without CuPy or a CUDA device, status reporting remains available while execution and validation are not hardware-qualified. The provenance registry records PSC-III-N59 as Sparse CuPy packet-jet execution. Its recorded status is finite certificate. Recorded limitation: no performance claim without matched hardware benchmarking.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are tests/test_block16_gpu_qualification.py and tests/test_nonlinear_execution.py. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 43. The recorded reference environment had no CuPy CUDA device, so GPU evidence is limited to status reporting, explicit unavailability, and CPU-side contract checks.

C.1E · CuPyPolynomialDerivativeProvider

Identity (q, a, t, k). Public research-manifest route komorebi_math.CuPyPolynomialDerivativeProvider. Reflected implementation komorebi_math.nonlinear.execution.CupyPolynomialDerivativeProvider. The same runtime object is also available through komorebi_math.nonlinear.CuPyPolynomialDerivativeProvider. The public export name maps to implementation object CupyPolynomialDerivativeProvider. The root route is governed by [RESEARCH] policy pre-beta-additive. This facade does not enlarge the frozen stable compatibility promise. Manifest kind class. Profile C1E-EXECUTION.

Manifest signature (s).

```
(tensors: 'Mapping[int, Any]', *, source_dimension: 'int | None' = None, target_dimension: 'int | None' = None,
    ↪ tolerance: 'float' = 1e-10, name: 'str' = 'CuPy polynomial derivative provider') -> 'None'
```

Definition. Dense polynomial derivatives resident on one CuPy device.

Contract (D, C, H, R, X). Execution plans require a finite packet-jet family and an optional support record for the same order and packet names. Only blocks certified zero may be pruned. Plan compilation enforces an explicit address budget. Execution requires one direction per derivative slot and checks that the plan still matches the family. CuPy execution additionally requires an importable CuPy runtime and a CUDA device. Principal public operations and properties are degree, supports_order, device_value, value, device_derivative, derivative, synchronize.

Guarantees (E, P, B, K). The CPU planner groups active output blocks by common projected inputs, caches those projections, and reports derivative-call and projection counts. Reconstruction is compared with the direct represented derivative. Optional CuPy classes preserve the same finite packet contract when the hardware boundary is available. Runtime discovery alone is not a performance or correctness qualification, and unavailable device paths fail explicitly.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_nonlinear_execution.py` and `tests/test_block10_execution_persistence.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 43.

C.1E · `CuPyRuntimeStatus`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.CuPyRuntimeStatus`. Reflected implementation `komorebi_math.nonlinear.gpu.CuPyRuntimeStatus`. The same runtime object is also available through `komorebi_math.nonlinear.CuPyRuntimeStatus`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1E-EXECUTION`.

Manifest signature (s).

```
(installed: 'bool', runtime_available: 'bool', device_count: 'int', runtime_version: 'int | None', error: 'str |
    ↪ None' = None) -> None
```

Definition. Observed CuPy import and CUDA device state.

Contract (D, C, H, R, X). GPU status inspection never requires hardware and returns the observed CuPy import and CUDA device state. Snapshot execution requires a complete finite snapshot, a working CuPy CUDA runtime, and compatible probe shapes. Validation compares the same persisted coefficients on CPU and device under a declared tolerance.

Guarantees (E, P, B, K). A successful validation record reports the probe count, maximum residual, runtime state, and certificate. The device executor evaluates only the captured finite snapshot and does not accept arbitrary Python callables. No speed claim follows from API presence. In a runtime without CuPy or a CUDA device, status reporting remains available while execution and validation are not hardware-qualified.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_block16_gpu_qualification.py` and `tests/test_nonlinear_execution.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 43. The recorded reference environment had no CuPy CUDA device, so GPU evidence is limited to status reporting, explicit unavailability, and CPU-side contract checks.

C.1E · `CuPySnapshotExecutor`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.CuPySnapshotExecutor`. Reflected implementation `komorebi_math.nonlinear.gpu.CuPySnapshotExecutor`. The same runtime object is also available through `komorebi_math.nonlinear.CuPySnapshotExecutor`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1E-EXECUTION`.

Manifest signature (s).

```
(snapshot: 'PacketJetSnapshot', *, dtype: 'Any | None' = None) -> 'None'
```

Definition. Device evaluator for one sparse `PacketJetSnapshot`.

Contract (D, C, H, R, X). GPU status inspection never requires hardware and returns the observed CuPy import and CUDA device state. Snapshot execution requires a complete finite snapshot, a working CuPy CUDA runtime, and compatible probe shapes. Validation compares the same persisted coefficients on CPU and device under a declared tolerance. Principal public operations and properties are `evaluate_batch`, `evaluate`.

Guarantees (E, P, B, K). A successful validation record reports the probe count, maximum residual, runtime state, and certificate. The device executor evaluates only the captured finite snapshot and does not accept arbitrary Python callables. No speed claim follows from API presence. In a runtime without CuPy or a CUDA device, status reporting remains available while execution and validation are not hardware-qualified. The provenance registry records `PSC-`

III-N59 as Sparse CuPy packet-jet execution. Its recorded status is finite certificate. Recorded limitation: no performance claim without matched hardware benchmarking.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are tests/test_block16_gpu_qualification.py and tests/test_nonlinear_execution.py. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 43. The recorded reference environment had no CuPy CUDA device, so GPU evidence is limited to status reporting, explicit unavailability, and CPU-side contract checks.

C.1E · CuPyValidationResult

Identity (q, a, t, k). Public research-manifest route komorebi_math.CuPyValidationResult. Reflected implementation komorebi_math.nonlinear.gpu.CuPyValidationResult. The same runtime object is also available through komorebi_math.nonlinear.CuPyValidationResult. The root route is governed by [RESEARCH] policy pre-beta-additive. This facade does not enlarge the frozen stable compatibility promise. Manifest kind class. Profile C1E-EXECUTION.

Manifest signature (s).

```
(probe_count: 'int', maximum_residual: 'float', runtime: 'CuPyRuntimeStatus', certificate: 'Certificate') -> None
```

Definition. Matched CPU and CuPy packet-jet validation data.

Contract (D, C, H, R, X). GPU status inspection never requires hardware and returns the observed CuPy import and CUDA device state. Snapshot execution requires a complete finite snapshot, a working CuPy CUDA runtime, and compatible probe shapes. Validation compares the same persisted coefficients on CPU and device under a declared tolerance.

Guarantees (E, P, B, K). A successful validation record reports the probe count, maximum residual, runtime state, and certificate. The device executor evaluates only the captured finite snapshot and does not accept arbitrary Python callables. No speed claim follows from API presence. In a runtime without CuPy or a CUDA device, status reporting remains available while execution and validation are not hardware-qualified.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are tests/test_block16_gpu_qualification.py and tests/test_nonlinear_execution.py. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 43. The recorded reference environment had no CuPy CUDA device, so GPU evidence is limited to status reporting, explicit unavailability, and CPU-side contract checks.

Derivatives and composition (15 cards).

C.1E · DirectionalJVP

Identity (q, a, t, k). Public research-manifest route komorebi_math.DirectionalJVP. Reflected implementation komorebi_math.nonlinear.autodiff.DirectionalJVP. The same runtime object is also available through komorebi_math.nonlinear.DirectionalJVP. The root route is governed by [RESEARCH] policy pre-beta-additive. This facade does not enlarge the frozen stable compatibility promise. Manifest kind class. Profile C1E-DERIVATIVE.

Manifest signature (s).

```
(*args, **kwargs)
```

Definition. One forward-mode Jacobian-vector product primitive.

Contract (D, C, H, R, X). Automatic differentiation adapters require a callable finite dimensional map, positive source and target dimensions, a declared maximum order, and a directional Jacobian vector product primitive supplied either directly or by an optional framework. Direction counts must equal the requested derivative order. Framework discovery is isolated in child processes so that a broken optional runtime cannot terminate the calling process.

Guarantees (E, P, B, K). A successful provider returns the finite directional derivative computed by the selected JVP recursion and records numerical exactness unless the caller supplies a stronger justified classification. Framework availability means that an isolated probe imported the runtime and found the required JVP capability. It does not assert numerical agreement for every model or device. Work grows with derivative order and the number of nested JVP calls.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_nonlinear_sparse_autodiff.py`, `tests/test_block10_sparse_autodiff.py`, and `tests/test_optional_backend_probe.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 37.

Defects, lifting, grading, and transport (27 cards).

C.1E · `FilteredCorrectionProblem`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.FilteredCorrectionProblem`. Reflected implementation `komorebi_math.nonlinear.filtered.FilteredCorrectionProblem`. The same runtime object is also available through `komorebi_math.nonlinear.FilteredCorrectionProblem`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1E-NONLINEAR-CALCULUS`.

Manifest signature (s).

```
(source: 'FilteredVectorSpace', target: 'FilteredVectorSpace', correction_map: 'Any', defect: 'Any', depth: 'int')
    ↪  -> 'None'
```

Definition. One associated-graded correction problem over a prime field.

Contract (D, C, H, R, X). Filtered contracts require finite dimensional vector spaces over an explicit prime field and decreasing filtration bases with compatible ambient dimensions. Filtered correction is solved one finite depth at a time. A filtered cochain complex requires explicit differentials that square to zero and preserve the declared filtration. Projective limit completion also requires external completeness hypotheses supplied by the caller. Principal public operations and properties are `induced_map`, `solve`.

Guarantees (E, P, B, K). Associated graded coordinates, correction cosets, finite branches, and spectral sequence pages are exact over the represented finite field. The terminal finite filtration permits exact page dimension and stabilization checks. A projective limit record only stores the declared infinite hypotheses and finite compatibility evidence. It does not prove completeness or convergence beyond those assumptions. The provenance registry records `PSC-III-T51` as One-step filtered lifting criterion. Its recorded status is `proved`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test module is `tests/test_nonlinear_filtered.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 42.

C.1E · `FilteredVectorSpace`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.FilteredVectorSpace`. Reflected implementation `komorebi_math.nonlinear.filtered.FilteredVectorSpace`. The same runtime object is also available through `komorebi_math.nonlinear.FilteredVectorSpace`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1E-NONLINEAR-CALCULUS`.

Manifest signature (s).

```
(dimension: 'int', levels: 'Mapping[int, Any]', *, prime: 'int') -> 'None'
```

Definition. A finite decreasing filtration `F^0 V >= ... >= F^N V = 0`.

Contract (D, C, H, R, X). Filtered contracts require finite dimensional vector spaces over an explicit prime field and decreasing filtration bases with compatible ambient dimensions. Filtered correction is solved one finite depth at a time. A filtered cochain complex requires explicit differentials that square to zero and preserve the declared filtration. Projective limit completion also requires external completeness hypotheses supplied by the caller. Principal public operations and properties are `from_weights`, `basis`, `dimension_at`, `contains`, `graded_basis`, `graded_dimension`, `graded_coordinates`, `lift_graded`, `certificate`.

Guarantees (E, P, B, K). Associated graded coordinates, correction cosets, finite branches, and spectral sequence pages are exact over the represented finite field. The terminal finite filtration permits exact page dimension and stabilization checks. A projective limit record only stores the declared infinite hypotheses and finite compatibility evidence. It does not prove completeness or convergence beyond those assumptions. The provenance registry records `PSC-III-D50` as Finite decreasing filtration over a prime field. Its recorded status is `definition`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test module is `tests/test_nonlinear_filtered.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 42.

C.1E · `FiniteAbelianGroup`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.FiniteAbelianGroup`. Reflected implementation `komorebi_math.nonlinear.graded.FiniteAbelianGroup`. The same runtime object is also available through `komorebi_math.nonlinear.FiniteAbelianGroup`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1E-NONLINEAR-CALCULUS`.

Manifest signature (s).

```
(labels: 'Sequence[str]', addition: 'Mapping[tuple[str, str], str]', *, zero: 'str', validate: 'bool' = True) ->
    ↪ 'None'
```

Definition. A finite abelian group represented by an explicit addition table.

Contract (D, C, H, R, X). Graded operations require a finite abelian grading group, finite component records, and multilinear operations whose declared output grade is the group sum of the input grades. Maurer Cartan routines evaluate only the supplied finite arity range. Divided square evaluation requires the supplied differential, bracket, and divided square maps to have compatible component shapes. Principal public operations and properties are `cyclic`, `add`, `sum`, `inverse`, `certificate`, `to_dict`.

Guarantees (E, P, B, K). Grade bookkeeping and finite group arithmetic are exact. Numerical component values inherit the arithmetic of the supplied arrays and operations. Certificates compare the gradewise reconstruction with the direct finite expression and record any failed compatibility law. No unspecified higher operation or infinite Maurer Cartan series is inferred. The provenance registry records `PSC-III-D17` as Finite abelian grading group. Its recorded status is `definition`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test module is `tests/test_nonlinear_graded.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 39.

Derivatives and composition (15 cards).

C.1E · FiniteDifferenceDerivativeProvider

Identity (q, a, t, k). Public research-manifest route `komorebi_math.FiniteDifferenceDerivativeProvider`. Reflected implementation `komorebi_math.nonlinear.derivatives.FiniteDifferenceDerivativeProvider`. The same runtime object is also available through `komorebi_math.nonlinear.FiniteDifferenceDerivativeProvider`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1E-DERIVATIVE`.

Manifest signature (s).

```
(function: 'Callable[[np.ndarray], Any]', source_dimension: 'int', target_dimension: 'int', step: 'float' =
   ↪ 0.0001, max_order: 'int' = 4, name: 'str' = 'central finite difference provider', exactness: 'Exactness' =
   ↪ <Exactness.NUMERICAL: 'numerical'>) -> None
```

Definition. Numerical directional derivatives by recursive central differences.

Contract (D, C, H, R, X). Derivative providers expose one common finite dimensional interface for map evaluation and directional derivatives. A call must use an order supported by the provider and exactly that many direction vectors of the declared source dimension. Callable and polynomial providers trust explicitly supplied derivative data. The finite difference provider additionally requires a positive finite step. The SymPy provider requires the optional symbolic dependency. Principal public operations and properties are `supports_order`, `value`, `derivative`.

Guarantees (E, P, B, K). Polynomial and symbolic providers preserve exact objects where their inputs permit exact arithmetic. Callable providers inherit the exactness declared by the caller. Finite differences return numerical evidence only. Every provider checks dimensions and finite values before evaluation. Tensor contraction cost grows with derivative order and ambient dimension. The provenance registry records `PSC-III-N03` as Recursive central directional differences. Its recorded status is `finite certificate`. The provenance record notes that roundoff and truncation error grow with derivative order.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_nonlinear_jets_composition.py` and `tests/test_nonlinear_sparse_autodiff.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapters 37 and 38.

Defects, lifting, grading, and transport (27 cards).

C.1E · FiniteFilteredCochainComplex

Identity (q, a, t, k). Public research-manifest route `komorebi_math.FiniteFilteredCochainComplex`. Reflected implementation `komorebi_math.nonlinear.filtered.FiniteFilteredCochainComplex`. The same runtime object is also available through `komorebi_math.nonlinear.FiniteFilteredCochainComplex`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1E-NONLINEAR-CALCULUS`.

Manifest signature (s).

```
(dimensions: 'Mapping[int, int]', differentials: 'Mapping[int, Any]', filtrations: 'Mapping[int,
   ↪ FilteredVectorSpace]', *, prime: 'int') -> 'None'
```

Definition. An exact filtered cochain complex over a prime field.

Contract (D, C, H, R, X). Filtered contracts require finite dimensional vector spaces over an explicit prime field and decreasing filtration bases with compatible ambient dimensions. Filtered correction is solved one finite depth at a time. A filtered cochain complex requires explicit differentials that square to zero and preserve the declared filtration. Projective limit completion also requires external completeness hypotheses supplied by the caller. Principal public operations and properties are `differential`, `page`, `total_cohomology_dimensions`, `spectral_sequence`.

Guarantees (E, P, B, K). Associated graded coordinates, correction cosets, finite branches, and spectral sequence pages are exact over the represented finite field. The terminal finite filtration permits exact page dimension and stabilization checks. A projective limit record only stores the declared infinite hypotheses and finite compatibility evidence.

It does not prove completeness or convergence beyond those assumptions. The provenance registry records `PSC-III-S54` as Spectral sequence of a finite filtered complex. Its recorded status is `classical input`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test module is `tests/test_nonlinear_filtered.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 42.

C.1E · FiniteLiftTree

Identity (q, a, t, k). Public research-manifest route `komorebi_math.FiniteLiftTree`. Reflected implementation `komorebi_math.nonlinear.lifting.FiniteLiftTree`. The same runtime object is also available through `komorebi_math.nonlinear.FiniteLiftTree`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1E-NONLINEAR-CALCULUS`.

Manifest signature (s).

```
(prime: 'int', levels: 'tuple[tuple[LiftTreeNode, ...], ...]') -> None
```

Definition. A finite prefix of a prime power lift tree.

Contract (D, C, H, R, X). Linear and polynomial lifting require a prime, a nonnegative depth, compatible finite equations, and a current residue class. One step searches the declared corrections modulo the prime. Tree construction requires an explicit terminal depth and a finite node budget. Principal public operations and properties are `max_depth`, `terminal_branches`.

Guarantees (E, P, B, K). For the lifting operations in this family, every returned child satisfies the represented congruence at the next prime power. Linear solves and branch verification are exact in modular arithmetic. Polynomial lifting uses the declared Jacobian and finite Taylor congruence. Branch counts may grow exponentially with depth, so the tree builder enforces the supplied node limit.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test module is `tests/test_lifting.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 40.

C.1E · GradedPacketOperation

Identity (q, a, t, k). Public research-manifest route `komorebi_math.GradedPacketOperation`. Reflected implementation `komorebi_math.nonlinear.graded.GradedPacketOperation`. The same runtime object is also available through `komorebi_math.nonlinear.GradedPacketOperation`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1E-NONLINEAR-CALCULUS`.

Manifest signature (s).

```
(group: 'FiniteAbelianGroup', packets: 'PacketSystem', arity: 'int', operation: 'GradedOperationCallable', *,
   ↪  exactness: 'Exactness' = <Exactness.NUMERICAL: 'numerical'>, tolerance: 'float | None' = None, name: 'str' =
   ↪  'graded packet operation') -> 'None'
```

Definition. A multilinear operation tested against a finite packet grading.

Contract (D, C, H, R, X). Graded operations require a finite abelian grading group, finite component records, and multilinear operations whose declared output grade is the group sum of the input grades. Maurer Cartan routines evaluate only the supplied finite arity range. Divided square evaluation requires the supplied differential, bracket, and divided square maps to have compatible component shapes. Principal public operations and properties are `evaluate`, `grading_certificate`, `packet_convolution`.

Guarantees (E, P, B, K). Grade bookkeeping and finite group arithmetic are exact. Numerical component values inherit the arithmetic of the supplied arrays and operations. Certificates compare the gradewise reconstruction with the direct finite expression and record any failed compatibility law. No unspecified higher operation or infinite

Maurer Cartan series is inferred. The provenance registry records `PSC-III-T18` as Graded packet convolution. Its recorded status is `proved`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test module is `tests/test_nonlinear_graded.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 39.

C.1E · `HilbertObstructionProjector`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.HilbertObstructionProjector`. Reflected implementation `komorebi_math.nonlinear.correction.HilbertObstructionProjector`. The same runtime object is also available through `komorebi_math.nonlinear.HilbertObstructionProjector`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1E-NONLINEAR-CALCULUS`.

Manifest signature (s).

```
(correction_map: 'Any', *, tolerance: 'float' = 1e-10) -> 'None'
```

Definition. Moore–Penrose correction and orthogonal cokernel projection.

Contract (D, C, H, R, X). Correction contracts operate on finite matrices or derivative providers with explicit dimensions, tolerances, radii, right inverses, and Lipschitz bounds. Hilbert obstruction projection uses the declared linearized operator. Right inverse correction checks the supplied contraction inequalities. Lyapunov Schmidt reduction computes numerical kernel, complement, range, and obstruction coordinates at a specified base point. Principal public operations and properties are `solve`.

Guarantees (E, P, B, K). Certificates report finite residuals, rank decisions, obstruction norms, and the inequalities actually checked. A vanishing obstruction is always relative to the stored tolerance. Local contraction or Lyapunov Schmidt conclusions remain conditional on the supplied finite hypotheses. Matrix factorizations dominate cost and use storage quadratic in the ambient dimensions. The provenance registry records `PSC-III-T31` as Orthogonal Hilbert obstruction projector. Its recorded status is `classical input`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test module is `tests/test_nonlinear_correction_elimination.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 41.

Derivatives and composition (15 cards).

C.1E · `JaxDerivativeProvider`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.JaxDerivativeProvider`. Reflected implementation `komorebi_math.nonlinear.autodiff.JaxDerivativeProvider`. The same runtime object is also available through `komorebi_math.nonlinear.JaxDerivativeProvider`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1E-DERIVATIVE`.

Manifest signature (s).

```
(function: 'Callable[[Any], Any]', source_dimension: 'int', target_dimension: 'int', max_order: 'int' = 4, dtype:
    ↪ 'str | None' = None, name: 'str' = 'JAX derivative provider', exactness: 'Exactness' = <Exactness.NUMERICAL:
    ↪ 'numerical'>) -> None
```

Definition. Nested `jax.jvp` derivatives with lazy framework import.

Contract (D, C, H, R, X). Automatic differentiation adapters require a callable finite dimensional map, positive source and target dimensions, a declared maximum order, and a directional Jacobian vector product primitive supplied either directly or by an optional framework. Direction counts must equal the requested derivative order. Frame-

work discovery is isolated in child processes so that a broken optional runtime cannot terminate the calling process. Principal public operations and properties are `available`, `value`, `derivative`, `to_dict`.

Guarantees (E, P, B, K). A successful provider returns the finite directional derivative computed by the selected JVP recursion and records numerical exactness unless the caller supplies a stronger justified classification. Framework availability means that an isolated probe imported the runtime and found the required JVP capability. It does not assert numerical agreement for every model or device. Work grows with derivative order and the number of nested JVP calls. The provenance registry records `PSC-III-N04J` as Lazy JAX derivative adapter. Its recorded status is `finite certificate`. The provenance record notes that device and precision behavior are controlled by the external JAX runtime.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_nonlinear_sparse_autodiff.py`, `tests/test_block10_sparse_autodiff.py`, and `tests/test_optional_backend_probe.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 37.

Defects, lifting, grading, and transport (27 cards).

C.1E · KatoTransport

Identity (q, a, t, k). Public research-manifest route `komorebi_math.KatoTransport`. Reflected implementation `komorebi_math.nonlinear.transport.KatoTransport`. The same runtime object is also available through `komorebi_math.nonlinear.KatoTransport`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1E-NONLINEAR-CALCULUS`.

Manifest signature (s).

```
(system: 'MovingPacketSystem') -> 'None'
```

Definition. Numerical finite Kato transport for a moving packet system.

Contract (D, C, H, R, X). Smooth transport requires complete finite orthogonal packet families of fixed dimension together with declared packet derivatives. Direct rotation requires two compatible packet families whose overlap sum is numerically invertible. Kato transport requires finite start and end times, a positive step count, and a positive tolerance. Principal public operations and properties are `evolve`, `direct_rotation_product`, `refinement_certificate`.

Guarantees (E, P, B, K). Direct polar rotation returns a finite unitary intertwiner when its residual checks pass. Kato transport integrates the canonical finite generator with midpoint exponential steps and records unitarity, packet transport, and mesh refinement residuals. These certificates are numerical and path specific. They do not prove differentiability of an undeclared family or continuum convergence without external estimates. The provenance registry records `PSC-III-T58` as Finite smooth packet transport. Its recorded status is `proved`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_nonlinear_transport.py`, and `tests/test_quantum_transport_effective_spectral.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapters 40 and 44.

C.1E · LinearLiftProblem

Identity (q, a, t, k). Public research-manifest route `komorebi_math.LinearLiftProblem`. Reflected implementation `komorebi_math.nonlinear.lifting.LinearLiftProblem`. The same runtime object is also available through `komorebi_math.nonlinear.LinearLiftProblem`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1E-NONLINEAR-CALCULUS`.

Manifest signature (s).

```
(matrix: 'Any', rhs: 'Any', point: 'Any', *, prime: 'int', depth: 'int') -> 'None'
```

Definition. Lift a linear congruence one prime power at a time.

Contract (D, C, H, R, X). Linear and polynomial lifting require a prime, a nonnegative depth, compatible finite equations, and a current residue class. One step searches the declared corrections modulo the prime. Tree construction requires an explicit terminal depth and a finite node budget. Principal public operations and properties are `step`.

Guarantees (E, P, B, K). For the lifting operations in this family, every returned child satisfies the represented congruence at the next prime power. Linear solves and branch verification are exact in modular arithmetic. Polynomial lifting uses the declared Jacobian and finite Taylor congruence. Branch counts may grow exponentially with depth, so the tree builder enforces the supplied node limit. The provenance registry records `PSC-III-T28` as One step lifting criterion. Its recorded status is `proved`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test module is `tests/test_lifting.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 40.

C.1E · `LinearObstructionProblem`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.LinearObstructionProblem`. Reflected implementation `komorebi_math.nonlinear.obstruction.LinearObstructionProblem`. The same runtime object is also available through `komorebi_math.nonlinear.LinearObstructionProblem`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1E-NONLINEAR-CALCULUS`.

Manifest signature (s).

```
(correction_map: 'Any', defect: 'Any', *, backend: 'str | LinearAlgebraBackend | None' = None, tolerance: 'float' =
    ↪ 1e-10) -> 'None'
```

Definition. Compute the obstruction to solving `D u = -defect`.

Contract (D, C, H, R, X). A linear obstruction problem requires a finite matrix and a target residual of matching dimension. The obstruction square additionally requires compatible maps whose two routes can be compared. Tolerances must be positive and finite when numerical rank decisions are used. Principal public operations and properties are `solve`.

Guarantees (E, P, B, K). The solver separates the residual into a least squares range component and a cokernel obstruction component. The correction is accepted only when the obstruction norm is within tolerance. Certificates record residuals and rank information for the supplied finite matrix. They do not establish exact vanishing outside the represented arithmetic and tolerance model. The provenance registry records `PSC-III-D13` as Linear correction datum and cokernel obstruction. Its recorded status is `definition`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test module is `tests/test_nonlinear_obstruction.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 40.

C.1E · LyapunovSchmidtReduction

Identity (q, a, t, k). Public research-manifest route `komorebi_math.LyapunovSchmidtReduction`. Reflected implementation `komorebi_math.nonlinear.correction.LyapunovSchmidtReduction`. The same runtime object is also available through `komorebi_math.nonlinear.LyapunovSchmidtReduction`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1E-NONLINEAR-CALCULUS`.

Manifest signature (s).

```
(provider: 'DerivativeProvider', base_point: 'Any', *, tolerance: 'float' = 1e-10) -> 'None'
```

Definition. Numerical SVD realization of local Lyapunov–Schmidt reduction.

Contract (D, C, H, R, X). Correction contracts operate on finite matrices or derivative providers with explicit dimensions, tolerances, radii, right inverses, and Lipschitz bounds. Hilbert obstruction projection uses the declared linearized operator. Right inverse correction checks the supplied contraction inequalities. Lyapunov Schmidt reduction computes numerical kernel, complement, range, and obstruction coordinates at a specified base point. Principal public operations and properties are `solve_range`, `base_derivative_certificate`, `reduced_second_derivative`.

Guarantees (E, P, B, K). Certificates report finite residuals, rank decisions, obstruction norms, and the inequalities actually checked. A vanishing obstruction is always relative to the stored tolerance. Local contraction or Lyapunov Schmidt conclusions remain conditional on the supplied finite hypotheses. Matrix factorizations dominate cost and use storage quadratic in the ambient dimensions. The provenance registry records `PSC-III-T33` as Finite local Lyapunov–Schmidt reduction. Its recorded status is `classical input`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test module is `tests/test_nonlinear_correction_elimination.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 41.

C.1E · MovingPacketSystem

Identity (q, a, t, k). Public research-manifest route `komorebi_math.MovingPacketSystem`. Reflected implementation `komorebi_math.nonlinear.transport.MovingPacketSystem`. The same runtime object is also available through `komorebi_math.nonlinear.MovingPacketSystem`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1E-NONLINEAR-CALCULUS`.

Manifest signature (s).

```
(projectors: 'Callable[[float], Iterable[Any]]', derivatives: 'Callable[[float], Iterable[Any]]', *, tolerance:
    ↪ 'float' = 1e-09, reference_time: 'float' = 0.0) -> 'None'
```

Definition. A differentiable finite family of complete orthogonal projectors.

Contract (D, C, H, R, X). Smooth transport requires complete finite orthogonal packet families of fixed dimension together with declared packet derivatives. Direct rotation requires two compatible packet families whose overlap sum is numerically invertible. Kato transport requires finite start and end times, a positive step count, and a positive tolerance. Principal public operations and properties are `packets`, `packet_derivatives`, `derivative_certificate`, `canonical_generator`, `generator_certificate`.

Guarantees (E, P, B, K). Direct polar rotation returns a finite unitary intertwiner when its residual checks pass. Kato transport integrates the canonical finite generator with midpoint exponential steps and records unitarity, packet transport, and mesh refinement residuals. These certificates are numerical and path specific. They do not prove differentiability of an undeclared family or continuum convergence without external estimates. The provenance registry records `PSC-III-D56` as Smooth finite packet system. Its recorded status is `definition`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_nonlinear_transport.py`, and `tests/test_quantum_transport_effective_spectral.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family.

The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapters 40 and 44.

Rooted-tree expansions and majorants (6 cards).

C.1E · `MultilinearOperation`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.MultilinearOperation`. Reflected implementation `collections.abc.Callable`. The same runtime object is also available through `komorebi_math.nonlinear.MultilinearOperation`. The public export name maps to implementation object `Callable`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `data`. Profile `C1E-ROOTED-TREES`.

Manifest signature (`s`).

```
(*args, **kwargs)
```

Definition. Callable type alias for one finite multilinear operation on a tuple of values.

Contract (`D, C, H, R, X`). Ordered-tree enumeration requires internal arities of at least two, nonnegative height or positive leaf targets, and an explicit tree budget. Polynomial fixed-point problems require a finite seed, compatible multilinear operations, and declared exactness and tolerance. Majorant routines require finite nonnegative norm bounds and a positive candidate radius.

Guarantees (`E, P, B, K`). Tree counts and finite enumerations are exact within the requested boundary. A tree expansion equals the matching finite Picard iterate under the implemented polynomial operations. A convergence certificate is issued only when the supplied scalar majorant maps the declared ball into itself and has contraction factor below one. Packet reconstruction prunes only addresses covered by supplied zero-support certificates. Enumeration can grow exponentially or faster with height and arity.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this root facade. The focused test module is `tests/test_nonlinear_trees.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 39.

Defects, lifting, grading, and transport (27 cards).

C.1E · `NonlinearPacketMap`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.NonlinearPacketMap`. Reflected implementation `komorebi_math.nonlinear.coordinates.NonlinearPacketMap`. The same runtime object is also available through `komorebi_math.nonlinear.NonlinearPacketMap`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1E-NONLINEAR-CALCULUS`.

Manifest signature (`s`).

```
(function: 'Callable[[Any], Any]', source: 'PacketSystem', target: 'PacketSystem', name: 'str' = 'nonlinear packet
   ↪ map') -> None
```

Definition. A nonlinear map evaluated through exact packet analysis and synthesis.

Contract (`D, C, H, R, X`). A nonlinear packet map requires a callable finite map and compatible finite source and target packet systems. Inputs must have the source ambient dimension. Packet evaluation projects the input into declared source channels, evaluates the map, and resolves the output through the target packet family. Principal public operations and properties are `apply`, `certificate`, `iterate`, `fixed_point_residual`.

Guarantees (`E, P, B, K`). For a complete target packet system, the recorded packet outputs reconstruct the direct finite map value within the certificate tolerance. The certificate concerns one supplied input and one implemented packet decomposition. It does not imply global linearity, smoothness, or invertibility. Work is dominated by packet projection and one map evaluation. The provenance registry records `PSC-III-D01` as Nonlinear packet coordinate map. Its recorded status is `definition`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test module is `tests/test_nonlinear_coordinates_defects.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 40.

C.1E · NonlinearRefinementDefect

Identity (q, a, t, k). Public research-manifest route `komorebi_math.NonlinearRefinementDefect`. Reflected implementation `komorebi_math.nonlinear.defects.NonlinearRefinementDefect`. The same runtime object is also available through `komorebi_math.nonlinear.NonlinearRefinementDefect`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1E-NONLINEAR-CALCULUS`.

Manifest signature (s).

```
(fine_map: 'Callable[[Any], Any]', coarse_map: 'Callable[[Any], Any]', coarsen_source: 'Any', coarsen_target:
   ↪  'Any') -> None
```

Definition. Pointwise defect `C_Y F_f(x) - F_c(C_X x)`.

Contract (D, C, H, R, X). A nonlinear refinement defect requires fine and coarse maps together with finite source and target coarsening operators of compatible shape. Iterated bounds additionally require a nonnegative iteration count and a caller supplied coarse Lipschitz bound. Principal public operations and properties are `evaluate`, `norm`, `composition_identity`, `iterated_orbit_bound`, `fixed_point_bound`.

Guarantees (E, P, B, K). The direct defect is the finite discrepancy between coarsening after the fine map and applying the coarse map after source coarsening. Orbit estimates are conditional on the supplied Lipschitz constant and measured finite defects. They do not independently prove a global contraction property. The provenance registry records `PSC-III-D10` as Nonlinear refinement defect. Its recorded status is `definition`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test module is `tests/test_nonlinear_coordinates_defects.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 40.

C.1E · NonlinearSchurElimination

Identity (q, a, t, k). Public research-manifest route `komorebi_math.NonlinearSchurElimination`. Reflected implementation `komorebi_math.nonlinear.elimination.NonlinearSchurElimination`. The same runtime object is also available through `komorebi_math.nonlinear.NonlinearSchurElimination`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1E-NONLINEAR-CALCULUS`.

Manifest signature (s).

```
(retained_provider: 'DerivativeProvider', hidden_provider: 'DerivativeProvider', retained_dimension: 'int',
   ↪  hidden_dimension: 'int', base_retained: 'Any', base_hidden: 'Any', *, tolerance: 'float' = 1e-10) -> 'None'
```

Definition. First and second derivatives after eliminating hidden variables.

Contract (D, C, H, R, X). Nonlinear Schur elimination requires retained and hidden derivative providers with a declared split of visible and hidden coordinates. The hidden derivative at the base point must be square and numerically invertible. Hidden solves require an initial point, a finite iteration limit, and a positive tolerance. Principal public operations and properties are `first_derivatives`, `second_derivative`, `solve_hidden`, `reduced_value`.

Guarantees (E, P, B, K). The first derivative contract implements the finite implicit derivative and reduced Schur derivative. The second derivative contract applies the declared provider derivatives through order two. Exact symbolic inputs may retain exact matrices. Numerical inputs receive residual certificates tied to the selected solve and

base point. These are local finite statements, not global existence claims. The provenance registry records `PSC-III-T41` as Finite nonlinear Schur elimination. Its recorded status is `proved`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test module is `tests/test_nonlinear_correction_elimination.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 41.

Rooted-tree expansions and majorants (6 cards).

C.1E · `OrderedRootedTree`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.OrderedRootedTree`. Reflected implementation `komorebi_math.nonlinear.trees.OrderedRootedTree`. The same runtime object is also available through `komorebi_math.nonlinear.OrderedRootedTree`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1E-ROOTED-TREES`.

Manifest signature (s).

```
(children: "tuple['OrderedRootedTree', ...]" = ()) -> None
```

Definition. An unlabeled ordered rooted tree.

Contract (D, C, H, R, X). Ordered-tree enumeration requires internal arities of at least two, nonnegative height or positive leaf targets, and an explicit tree budget. Polynomial fixed-point problems require a finite seed, compatible multilinear operations, and declared exactness and tolerance. Majorant routines require finite nonnegative norm bounds and a positive candidate radius. Principal public operations and properties are `leaf`, `is_leaf`, `arity`, `height`, `leaf_count`, `internal_count`, `vertex_count`, `arity_profile`, `prefix`, `to_dict`.

Guarantees (E, P, B, K). Tree counts and finite enumerations are exact within the requested boundary. A tree expansion equals the matching finite Picard iterate under the implemented polynomial operations. A convergence certificate is issued only when the supplied scalar majorant maps the declared ball into itself and has contraction factor below one. Packet reconstruction prunes only addresses covered by supplied zero-support certificates. Enumeration can grow exponentially or faster with height and arity. The provenance registry records `PSC-III-D13` as Finite ordered rooted tree. Its recorded status is `definition`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test module is `tests/test_nonlinear_trees.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 39.

Packet jets and sparse tensors (7 cards).

C.1E · `PacketJet`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.PacketJet`. Reflected implementation `komorebi_math.nonlinear.jets.PacketJet`. The same runtime object is also available through `komorebi_math.nonlinear.PacketJet`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1E-PACKET-JET`.

Manifest signature (s).

```
(provider: 'DerivativeProvider', point: 'Any', source: 'PacketSystem', target: 'PacketSystem', output_index:
    ↪ 'int', input_indices: 'tuple[int, ...]') -> None
```

Definition. One projected multilinear derivative block.

Contract (D, C, H, R, X). Packet jets require a derivative provider, compatible finite source and target packet systems, a valid base point, and a positive supported derivative order. Packet labels must resolve inside their systems. Evaluation requires one source direction per input address. Support discovery and Taylor assembly also require positive

finite tolerances and explicit evaluation budgets. Principal public operations and properties are `order`, `address`, `exactness`, `evaluate`, `norm_bound`, `symmetry_certificate`, `to_dict`.

Guarantees (E, P, B, K). A packet jet evaluates the declared derivative after source projection and target projection. A complete family reconstructs the represented finite derivative when both packet systems are complete. Support classifications are exhaustive only when the evaluation budget covers every required packet basis probe. Taylor certificates compare one finite truncation with the observed map value at the supplied increment. Address counts and support work grow combinatorially with order and packet count. The provenance registry records `PSC-III-D06` as Finite packet jet. Its recorded status is `definition`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_nonlinear_jets_composition.py` and `tests/test_nonlinear_execution.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 38.

Execution, snapshots, streaming, and profiling (35 cards).

C.1E · `PacketJetBatchResult`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.PacketJetBatchResult`. Reflected implementation `komorebi_math.nonlinear.execution.PacketJetBatchResult`. The same runtime object is also available through `komorebi_math.nonlinear.PacketJetBatchResult`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1E-EXECUTION`.

Manifest signature (s).

```
(evaluations: 'tuple[PacketJetEvaluation, ...]', reconstructed: 'Any', direct: 'Any', derivative_calls: 'int',
    ↪ input_projections: 'int', output_projections: 'int', pruned_blocks: 'int', backend: 'str', returned_to_host:
    ↪ 'bool', certificate: 'Certificate') -> None
```

Definition. Result of one projection-cached packet-jet execution.

Contract (D, C, H, R, X). Execution plans require a finite packet-jet family and an optional support record for the same order and packet names. Only blocks certified zero may be pruned. Plan compilation enforces an explicit address budget. Execution requires one direction per derivative slot and checks that the plan still matches the family. CuPy execution additionally requires an importable CuPy runtime and a CUDA device. Principal public operations and properties are `as_mapping`, `value`, `to_dict`.

Guarantees (E, P, B, K). The CPU planner groups active output blocks by common projected inputs, caches those projections, and reports derivative-call and projection counts. Reconstruction is compared with the direct represented derivative. Optional CuPy classes preserve the same finite packet contract when the hardware boundary is available. Runtime discovery alone is not a performance or correctness qualification, and unavailable device paths fail explicitly.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_nonlinear_execution.py` and `tests/test_block10_execution_persistence.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 43.

C.1E · `PacketJetComplexityEstimate`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.PacketJetComplexityEstimate`. Reflected implementation `komorebi_math.nonlinear.profiling.PacketJetComplexityEstimate`. The same runtime object is also available through `komorebi_math.nonlinear.PacketJetComplexityEstimate`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1E-EXECUTION`.

Manifest signature (s).

```
(order: 'int', source_dimension: 'int', target_dimension: 'int', source_packets: 'int', target_packets: 'int',
   ↪ family_addresses: 'int', active_addresses: 'int', pruned_addresses: 'int', input_groups: 'int',
   ↪ naive_derivative_calls: 'int', planned_derivative_calls: 'int', projected_input_vectors: 'int',
   ↪ output_projections: 'int', dense_basis_evaluations: 'int', dense_tensor_elements: 'int', certificate:
   ↪ 'Certificate') -> None
```

Definition. Exact finite counts implied by packet and ambient dimensions.

Contract (D, C, H, R, X). Complexity estimation requires a finite family and a compatible optional support record or execution plan. Runtime profiles require positive repetition counts, nonnegative warmup counts, and a precisely stated measurement scope. Packet-jet profiling may include planning, execution, snapshot capture, and optional stream writing under explicit budgets. Principal public operations and properties are `derivative_call_reduction`, `to_dict`.

Guarantees (E, P, B, K). Deterministic estimate fields are exact finite counts derived from the represented dimensions, packets, addresses, and plan. Timing and traced Python allocation samples are environment-dependent observations. They are not asymptotic evidence and do not include unreported device allocation or external process cost. Reports retain the interpreter, platform, scope, repetitions, and certificate needed to interpret the measurements.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_block11_golden_profiling_beta.py` and `tests/test_nonlinear_execution.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 43.

C.1E · `PacketJetExecutionPlan`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.PacketJetExecutionPlan`. Reflected implementation `komorebi_math.nonlinear.execution.PacketJetExecutionPlan`. The same runtime object is also available through `komorebi_math.nonlinear.PacketJetExecutionPlan`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1E-EXECUTION`.

Manifest signature (s).

```
(order: 'int', source_names: 'tuple[str, ...]', target_names: 'tuple[str, ...]', groups:
   ↪ 'tuple[PacketJetInputGroup, ...]', active_addresses: 'tuple[PacketJetAddress, ...]', pruned_addresses:
   ↪ 'tuple[PacketJetAddress, ...]', support_was_supplied: 'bool', support_complete: 'bool', certificate:
   ↪ 'Certificate') -> None
```

Definition. A deterministic grouping of packet-jet blocks by projected inputs.

Contract (D, C, H, R, X). Execution plans require a finite packet-jet family and an optional support record for the same order and packet names. Only blocks certified zero may be pruned. Plan compilation enforces an explicit address budget. Execution requires one direction per derivative slot and checks that the plan still matches the family. CuPy execution additionally requires an importable CuPy runtime and a CUDA device. Principal public operations and properties are `active_address_count`, `pruned_address_count`, `full_address_count`, `derivative_call_count`, `naive_derivative_call_count`, `derivative_call_reduction`, `matches`, `to_dict`.

Guarantees (E, P, B, K). The CPU planner groups active output blocks by common projected inputs, caches those projections, and reports derivative-call and projection counts. Reconstruction is compared with the direct represented derivative. Optional CuPy classes preserve the same finite packet contract when the hardware boundary is available. Runtime discovery alone is not a performance or correctness qualification, and unavailable device paths fail explicitly. The provenance registry records `PSC-III-A10` as Projection-cached packet-jet execution plan. Its recorded status is `proved`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_nonlinear_execution.py` and `tests/test_block10_execution_persistence.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implemen-

tation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 43.

Packet jets and sparse tensors (7 cards).

C.1E · PacketJetFamily

Identity (q, a, t, k). Public research-manifest route `komorebi_math.PacketJetFamily`. Reflected implementation `komorebi_math.nonlinear.jets.PacketJetFamily`. The same runtime object is also available through `komorebi_math.nonlinear.PacketJetFamily`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1E-PACKET-JET`.

Manifest signature (s).

```
(provider: 'DerivativeProvider', point: 'Any', source: 'PacketSystem', target: 'PacketSystem', order: 'int') ->
    ↪ 'None'
```

Definition. Every packet block of one derivative order at one base point.

Contract (D, C, H, R, X). Packet jets require a derivative provider, compatible finite source and target packet systems, a valid base point, and a positive supported derivative order. Packet labels must resolve inside their systems. Evaluation requires one source direction per input address. Support discovery and Taylor assembly also require positive finite tolerances and explicit evaluation budgets. Principal public operations and properties are `jet`, `addresses`, `reconstruct`, `reconstruction_certificate`, `to_dict`.

Guarantees (E, P, B, K). A packet jet evaluates the declared derivative after source projection and target projection. A complete family reconstructs the represented finite derivative when both packet systems are complete. Support classifications are exhaustive only when the evaluation budget covers every required packet basis probe. Taylor certificates compare one finite truncation with the observed map value at the supplied increment. Address counts and support work grow combinatorially with order and packet count. The provenance registry records `PSC-III-T07` as Exact reconstruction of finite packet derivatives. Its recorded status is `proved`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_nonlinear_jets_composition.py` and `tests/test_nonlinear_execution.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 38.

Execution, snapshots, streaming, and profiling (35 cards).

C.1E · PacketJetProfileReport

Identity (q, a, t, k). Public research-manifest route `komorebi_math.PacketJetProfileReport`. Reflected implementation `komorebi_math.nonlinear.profiling.PacketJetProfileReport`. The same runtime object is also available through `komorebi_math.nonlinear.PacketJetProfileReport`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1E-EXECUTION`.

Manifest signature (s).

```
(estimate: 'PacketJetComplexityEstimate', stages: 'tuple[RuntimeStageProfile, ...]', environment: 'Mapping[str,
    ↪ Any]', certificate: 'Certificate') -> None
```

Definition. Exact counts paired with environment-specific runtime observations.

Contract (D, C, H, R, X). Complexity estimation requires a finite family and a compatible optional support record or execution plan. Runtime profiles require positive repetition counts, nonnegative warmup counts, and a precisely stated measurement scope. Packet-jet profiling may include planning, execution, snapshot capture, and optional stream writing under explicit budgets. Principal public operations and properties are `to_dict`.

Guarantees (E, P, B, K). Deterministic estimate fields are exact finite counts derived from the represented dimensions, packets, addresses, and plan. Timing and traced Python allocation samples are environment-dependent observa-

tions. They are not asymptotic evidence and do not include unreported device allocation or external process cost. Reports retain the interpreter, platform, scope, repetitions, and certificate needed to interpret the measurements.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_block11_golden_profiling_beta.py` and `tests/test_nonlinear_execution.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 43.

C.1E · `PacketJetSnapshot`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.PacketJetSnapshot`. Reflected implementation `komorebi_math.nonlinear.snapshots.PacketJetSnapshot`. The same runtime object is also available through `komorebi_math.nonlinear.PacketJetSnapshot`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1E-EXECUTION`.

Manifest signature (s).

```
(address: 'PacketJetAddress', source_dimension: 'int', target_dimension: 'int', provider_name: 'str', exactness:
    ↪ 'Exactness', point: 'tuple[Any, ...]', coefficients: 'tuple[PacketJetCoefficient, ...]',
    ↪ capture_certificate: 'Certificate') -> None
```

Definition. A provider-independent sparse coefficient record of one packet jet.

Contract (D, C, H, R, X). A snapshot captures one fixed packet jet by evaluating the ambient standard-basis tuples through an explicit finite budget. The source and target dimensions, base point, address, provider name, exactness, and sparse coefficients are stored in the record. Snapshot evaluation requires direction vectors of the recorded source dimension. Principal public operations and properties are `order`, `nnz`, `dense_coefficient_count`, `coefficient`, `evaluate`, `to_dense`, `comparison_certificate`, `to_dict`.

Guarantees (E, P, B, K). The snapshot is provider-independent after capture and reconstructs the recorded multilinear block from its stored coefficients. Its comparison certificate checks matched evaluations against the source jet on supplied probes. Exact coefficients preserve exact arithmetic. Numerical coefficients use the stored tolerance and certificate. Dense capture cost grows as the source dimension raised to the jet order. The provenance registry records `PSC-III-D58` as Finite packet-jet snapshot. Its recorded status is `finite certificate`. Recorded limitation: snapshot represents one fixed order, address, and base point.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_nonlinear_snapshots_persistence.py` and `tests/test_block10_execution_persistence.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapters 38 and 43.

C.1E · `PacketJetSnapshotStreamReader`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.PacketJetSnapshotStreamReader`. Reflected implementation `komorebi_math.nonlinear.streaming.PacketJetSnapshotStreamReader`. The same runtime object is also available through `komorebi_math.nonlinear.PacketJetSnapshotStreamReader`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1E-EXECUTION`.

Manifest signature (s).

```
(path: 'str | os.PathLike[str]', *, allow_incomplete: 'bool' = False, verify_chunks: 'bool' = False) -> 'None'
```

Definition. Bounded-memory reader for a complete or explicitly allowed partial stream.

Contract (D, C, H, R, X). Chunked streams use fixed schema identifiers and version 1 envelopes. Writers require a destination, a positive chunk size, and explicit overwrite or resume policy. Direct capture writes to a partial directory and promotes it only after completion. Readers reject duplicate JSON keys, unsupported schemas, inconsistent ranges, digest mismatches, and incomplete streams unless incomplete access is explicitly allowed. Principal public operations and properties are `complete`, `iter_chunks`, `iter_coefficients`, `select`, `evaluate`, `load_snapshot`, `verify`.

Guarantees (E, P, B, K). Each committed chunk has a payload digest and a manifest range. Resume continues only from a verified committed prefix. Selection and evaluation can operate with memory bounded by the configured chunk size. Loading a conventional snapshot requires a complete stream and an explicit coefficient ceiling. Digests provide integrity against accidental corruption, not authentication against an adversary able to replace the entire stream and its hashes.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_nonlinear_snapshots_persistence.py` and `tests/test_block11_streaming_migrations.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 43.

C.1E · `PacketJetStreamChunkRecord`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.PacketJetStreamChunkRecord`. Reflected implementation `komorebi_math.nonlinear.streaming.PacketJetStreamChunkRecord`. The same runtime object is also available through `komorebi_math.nonlinear.PacketJetStreamChunkRecord`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1E-EXECUTION`.

Manifest signature (s).

```
(index: 'int', filename: 'str', basis_start: 'int', basis_stop: 'int', coefficient_start: 'int', coefficient_stop:
    ↪  'int', payload_sha256: 'str', byte_count: 'int') -> None
```

Definition. One immutable chunk index entry in a packet-jet stream manifest.

Contract (D, C, H, R, X). Chunked streams use fixed schema identifiers and version 1 envelopes. Writers require a destination, a positive chunk size, and explicit overwrite or resume policy. Direct capture writes to a partial directory and promotes it only after completion. Readers reject duplicate JSON keys, unsupported schemas, inconsistent ranges, digest mismatches, and incomplete streams unless incomplete access is explicitly allowed. Principal public operations and properties are `coefficient_count`, `basis_count`, `to_dict`, `from_dict`.

Guarantees (E, P, B, K). Each committed chunk has a payload digest and a manifest range. Resume continues only from a verified committed prefix. Selection and evaluation can operate with memory bounded by the configured chunk size. Loading a conventional snapshot requires a complete stream and an explicit coefficient ceiling. Digests provide integrity against accidental corruption, not authentication against an adversary able to replace the entire stream and its hashes.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_nonlinear_snapshots_persistence.py` and `tests/test_block11_streaming_migrations.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 43.

C.1E · PacketJetStreamManifest

Identity (q, a, t, k). Public research-manifest route `komorebi_math.PacketJetStreamManifest`. Reflected implementation `komorebi_math.nonlinear.streaming.PacketJetStreamManifest`. The same runtime object is also available through `komorebi_math.nonlinear.PacketJetStreamManifest`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1E-EXECUTION`.

Manifest signature (s).

```
(state: 'StreamState', producer_version: 'str', address: 'PacketJetAddress', source_dimension: 'int',
   ↪ target_dimension: 'int', provider_name: 'str', exactness: 'Exactness', point: 'tuple[Any, ...]', tolerance:
   ↪ 'float', dense_coefficient_count: 'int', capture_mode: 'CaptureMode', source_digest: 'str', metadata_sha256:
   ↪ 'str', chunk_size: 'int', evaluations_completed: 'int', coefficients_completed: 'int',
   ↪ maximum_omitted_basis_norm: 'float', chunks: 'tuple[PacketJetStreamChunkRecord, ...]', capture_certificate:
   ↪ 'Certificate | None', stream_sha256: 'str') -> None
```

Definition. Validated metadata and progress for one chunked packet-jet stream.

Contract (D, C, H, R, X). Chunked streams use fixed schema identifiers and version 1 envelopes. Writers require a destination, a positive chunk size, and explicit overwrite or resume policy. Direct capture writes to a partial directory and promotes it only after completion. Readers reject duplicate JSON keys, unsupported schemas, inconsistent ranges, digest mismatches, and incomplete streams unless incomplete access is explicitly allowed. Principal public operations and properties are `complete`, `order`, `metadata_dict`, `to_payload`, `from_payload`.

Guarantees (E, P, B, K). Each committed chunk has a payload digest and a manifest range. Resume continues only from a verified committed prefix. Selection and evaluation can operate with memory bounded by the configured chunk size. Loading a conventional snapshot requires a complete stream and an explicit coefficient ceiling. Digests provide integrity against accidental corruption, not authentication against an adversary able to replace the entire stream and its hashes.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_nonlinear_snapshots_persistence.py` and `tests/test_block11_streaming_migrations.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 43.

C.1E · PacketJetStreamSelection

Identity (q, a, t, k). Public research-manifest route `komorebi_math.PacketJetStreamSelection`. Reflected implementation `komorebi_math.nonlinear.streaming.PacketJetStreamSelection`. The same runtime object is also available through `komorebi_math.nonlinear.PacketJetStreamSelection`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1E-EXECUTION`.

Manifest signature (s).

```
(coefficient_start: 'int', coefficient_stop: 'int', coefficients: 'tuple[PacketJetCoefficient, ...]',
   ↪ complete_snapshot: 'bool', certificate: 'Certificate') -> None
```

Definition. A bounded coefficient selection loaded from a packet-jet stream.

Contract (D, C, H, R, X). Chunked streams use fixed schema identifiers and version 1 envelopes. Writers require a destination, a positive chunk size, and explicit overwrite or resume policy. Direct capture writes to a partial directory and promotes it only after completion. Readers reject duplicate JSON keys, unsupported schemas, inconsistent ranges, digest mismatches, and incomplete streams unless incomplete access is explicitly allowed.

Guarantees (E, P, B, K). Each committed chunk has a payload digest and a manifest range. Resume continues only from a verified committed prefix. Selection and evaluation can operate with memory bounded by the configured chunk size. Loading a conventional snapshot requires a complete stream and an explicit coefficient ceiling. Digests provide integrity against accidental corruption, not authentication against an adversary able to replace the entire stream and its hashes.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_nonlinear_snapshots_persistence.py` and `tests/test_block11_streaming_migrations.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 43.

C.1E · `PacketJetStreamWriteResult`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.PacketJetStreamWriteResult`. Reflected implementation `komorebi_math.nonlinear.streaming.PacketJetStreamWriteResult`. The same runtime object is also available through `komorebi_math.nonlinear.PacketJetStreamWriteResult`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1E-EXECUTION`.

Manifest signature (s).

```
(destination: 'Path', active_path: 'Path', completed: 'bool', resumed: 'bool', chunks_written: 'int',
    ↪ evaluations_written: 'int', coefficients_written: 'int', manifest: 'PacketJetStreamManifest', certificate:
    ↪ 'Certificate') -> None
```

Definition. Progress returned by a direct or snapshot stream write.

Contract (D, C, H, R, X). Chunked streams use fixed schema identifiers and version 1 envelopes. Writers require a destination, a positive chunk size, and explicit overwrite or resume policy. Direct capture writes to a partial directory and promotes it only after completion. Readers reject duplicate JSON keys, unsupported schemas, inconsistent ranges, digest mismatches, and incomplete streams unless incomplete access is explicitly allowed.

Guarantees (E, P, B, K). Each committed chunk has a payload digest and a manifest range. Resume continues only from a verified committed prefix. Selection and evaluation can operate with memory bounded by the configured chunk size. Loading a conventional snapshot requires a complete stream and an explicit coefficient ceiling. Digests provide integrity against accidental corruption, not authentication against an adversary able to replace the entire stream and its hashes.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_nonlinear_snapshots_persistence.py` and `tests/test_block11_streaming_migrations.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 43.

Derivatives and composition (15 cards).

C.1E · `PolynomialDerivativeProvider`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.PolynomialDerivativeProvider`. Reflected implementation `komorebi_math.nonlinear.derivatives.PolynomialDerivativeProvider`. The same runtime object is also available through `komorebi_math.nonlinear.PolynomialDerivativeProvider`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1E-DERIVATIVE`.

Manifest signature (s).

```
(tensors: 'Mapping[int, Any]', *, source_dimension: 'int | None' = None, target_dimension: 'int | None' = None,
    ↪ exactness: 'Exactness' = <Exactness.NUMERICAL: 'numerical'>, tolerance: 'float' = 1e-10, name: 'str' =
    ↪ 'polynomial derivative provider') -> 'None'
```

Definition. A finite polynomial map represented by symmetric coefficient tensors.

Contract (D, C, H, R, X). Derivative providers expose one common finite dimensional interface for map evaluation and directional derivatives. A call must use an order supported by the provider and exactly that many direction

vectors of the declared source dimension. Callable and polynomial providers trust explicitly supplied derivative data. The finite difference provider additionally requires a positive finite step. The SymPy provider requires the optional symbolic dependency. Principal public operations and properties are `degree`, `supports_order`, `value`, `derivative`, `symmetry_certificate`.

Guarantees (`E`, `P`, `B`, `K`). Polynomial and symbolic providers preserve exact objects where their inputs permit exact arithmetic. Callable providers inherit the exactness declared by the caller. Finite differences return numerical evidence only. Every provider checks dimensions and finite values before evaluation. Tensor contraction cost grows with derivative order and ambient dimension. The provenance registry records `PSC-III-D04` as Symmetric polynomial derivative tensors. Its recorded status is `definition`.

Evidence and links (`V`, `L`). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_nonlinear_jets_composition.py` and `tests/test_nonlinear_sparse_autodiff.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapters 37 and 38.

Rooted-tree expansions and majorants (6 cards).

C.1E · `PolynomialFixedPointProblem`

Identity (`q`, `a`, `t`, `k`). Public research-manifest route `komorebi_math.PolynomialFixedPointProblem`. Reflected implementation `komorebi_math.nonlinear.trees.PolynomialFixedPointProblem`. The same runtime object is also available through `komorebi_math.nonlinear.PolynomialFixedPointProblem`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1E-ROOTED-TREES`.

Manifest signature (`s`).

```
(seed: 'Any', operations: 'Mapping[int, MultilinearOperation]', *, exactness: 'Exactness' = <Exactness.NUMERICAL:
    ↪ 'numerical'>, tolerance: 'float' = 1e-10, name: 'str' = 'polynomial fixed-point problem') -> 'None'
```

Definition. A finite-dimensional equation `u = s + sum_r B_r(u,...,u)`.

Contract (`D`, `C`, `H`, `R`, `X`). Ordered-tree enumeration requires internal arities of at least two, nonnegative height or positive leaf targets, and an explicit tree budget. Polynomial fixed-point problems require a finite seed, compatible multilinear operations, and declared exactness and tolerance. Majorant routines require finite nonnegative norm bounds and a positive candidate radius. Principal public operations and properties are `operation`, `map`, `residual`, `picard`, `tree_value`, `tree_expansion`, `convergence_certificate`.

Guarantees (`E`, `P`, `B`, `K`). Tree counts and finite enumerations are exact within the requested boundary. A tree expansion equals the matching finite Picard iterate under the implemented polynomial operations. A convergence certificate is issued only when the supplied scalar majorant maps the declared ball into itself and has contraction factor below one. Packet reconstruction prunes only addresses covered by supplied zero-support certificates. Enumeration can grow exponentially or faster with height and arity. The provenance registry records `PSC-III-T14A` as Finite polynomial rooted-tree expansion. Its recorded status is `finite certificate`.

Evidence and links (`V`, `L`). The committed research manifest and runtime reflection audit identify this root facade. The focused test module is `tests/test_nonlinear_trees.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 39.

Defects, lifting, grading, and transport (27 cards).

C.1E · PolynomialLiftProblem

Identity (q, a, t, k). Public research-manifest route `komorebi_math.PolynomialLiftProblem`. Reflected implementation `komorebi_math.nonlinear.lifting.PolynomialLiftProblem`. The same runtime object is also available through `komorebi_math.nonlinear.PolynomialLiftProblem`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1E-NONLINEAR-CALCULUS`.

Manifest signature (s).

```
(function: 'Callable[[tuple[int, ...]], Iterable[int]]', jacobian: 'Callable[[tuple[int, ...]], Any]', point:
   ↪ 'Any', *, prime: 'int', depth: 'int') -> 'None'
```

Definition. Lift a polynomial system by its Jacobian over the residual field.

Contract (D, C, H, R, X). Linear and polynomial lifting require a prime, a nonnegative depth, compatible finite equations, and a current residue class. One step searches the declared corrections modulo the prime. Tree construction requires an explicit terminal depth and a finite node budget. Principal public operations and properties are `step`.

Guarantees (E, P, B, K). For the lifting operations in this family, every returned child satisfies the represented congruence at the next prime power. Linear solves and branch verification are exact in modular arithmetic. Polynomial lifting uses the declared Jacobian and finite Taylor congruence. Branch counts may grow exponentially with depth, so the tree builder enforces the supplied node limit.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test module is `tests/test_lifting.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 40.

C.1E · RightInverseCorrectionProblem

Identity (q, a, t, k). Public research-manifest route `komorebi_math.RightInverseCorrectionProblem`. Reflected implementation `komorebi_math.nonlinear.correction.RightInverseCorrectionProblem`. The same runtime object is also available through `komorebi_math.nonlinear.RightInverseCorrectionProblem`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1E-NONLINEAR-CALCULUS`.

Manifest signature (s).

```
(linear_map: 'Any', right_inverse: 'Any', nonlinear: 'Callable[[np.ndarray], Any]', target: 'Any', *, radius:
   ↪ 'float', lipschitz_bound: 'float', tolerance: 'float' = 1e-10) -> 'None'
```

Definition. Solve `D x + N(x) = y` by the theorem's fixed-point map.

Contract (D, C, H, R, X). Correction contracts operate on finite matrices or derivative providers with explicit dimensions, tolerances, radii, right inverses, and Lipschitz bounds. Hilbert obstruction projection uses the declared linearized operator. Right inverse correction checks the supplied contraction inequalities. Lyapunov Schmidt reduction computes numerical kernel, complement, range, and obstruction coordinates at a specified base point. Principal public operations and properties are `hypothesis_certificate`, `solve`, `target_stability_bound`.

Guarantees (E, P, B, K). Certificates report finite residuals, rank decisions, obstruction norms, and the inequalities actually checked. A vanishing obstruction is always relative to the stored tolerance. Local contraction or Lyapunov Schmidt conclusions remain conditional on the supplied finite hypotheses. Matrix factorizations dominate cost and use storage quadratic in the ambient dimensions. The provenance registry records `PSC-III-T32` as Right-inverse nonlinear correction. Its recorded status is `proved`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test module is `tests/test_nonlinear_correction_elimination.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 41.

Execution, snapshots, streaming, and profiling (35 cards).

C.1E · RuntimeProfileSample

Identity (q, a, t, k). Public research-manifest route `komorebi_math.RuntimeProfileSample`. Reflected implementation `komorebi_math.nonlinear.profiling.RuntimeProfileSample`. The same runtime object is also available through `komorebi_math.nonlinear.RuntimeProfileSample`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1E-EXECUTION`.

Manifest signature (s).

```
(duration_ns: 'int', traced_peak_bytes: 'int') -> None
```

Definition. One observed duration and Python allocation peak.

Contract (D, C, H, R, X). Complexity estimation requires a finite family and a compatible optional support record or execution plan. Runtime profiles require positive repetition counts, nonnegative warmup counts, and a precisely stated measurement scope. Packet-jet profiling may include planning, execution, snapshot capture, and optional stream writing under explicit budgets.

Guarantees (E, P, B, K). Deterministic estimate fields are exact finite counts derived from the represented dimensions, packets, addresses, and plan. Timing and traced Python allocation samples are environment-dependent observations. They are not asymptotic evidence and do not include unreported device allocation or external process cost. Reports retain the interpreter, platform, scope, repetitions, and certificate needed to interpret the measurements.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_block11_golden_profiling_beta.py` and `tests/test_nonlinear_execution.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 43.

C.1E · RuntimeStageProfile

Identity (q, a, t, k). Public research-manifest route `komorebi_math.RuntimeStageProfile`. Reflected implementation `komorebi_math.nonlinear.profiling.RuntimeStageProfile`. The same runtime object is also available through `komorebi_math.nonlinear.RuntimeStageProfile`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1E-EXECUTION`.

Manifest signature (s).

```
(name: 'str', scope: 'str', warmup: 'int', samples: 'tuple[RuntimeProfileSample, ...]') -> None
```

Definition. Repeated observations for one precisely named runtime stage.

Contract (D, C, H, R, X). Complexity estimation requires a finite family and a compatible optional support record or execution plan. Runtime profiles require positive repetition counts, nonnegative warmup counts, and a precisely stated measurement scope. Packet-jet profiling may include planning, execution, snapshot capture, and optional stream writing under explicit budgets. Principal public operations and properties are `repetitions`, `minimum_duration_ns`, `median_duration_ns`, `maximum_duration_ns`, `median_peak_bytes`, `maximum_peak_bytes`, `to_dict`.

Guarantees (E, P, B, K). Deterministic estimate fields are exact finite counts derived from the represented dimensions, packets, addresses, and plan. Timing and traced Python allocation samples are environment-dependent observations. They are not asymptotic evidence and do not include unreported device allocation or external process cost. Reports retain the interpreter, platform, scope, repetitions, and certificate needed to interpret the measurements.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_block11_golden_profiling_beta.py` and `tests/test_nonlinear_execution.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 43.

Packet jets and sparse tensors (7 cards).

C.1E · SparsePolynomialDerivativeProvider

Identity (q, a, t, k). Public research-manifest route `komorebi_math.SparsePolynomialDerivativeProvider`. Reflected implementation `komorebi_math.nonlinear.sparse.SparsePolynomialDerivativeProvider`. The same runtime object is also available through `komorebi_math.nonlinear.SparsePolynomialDerivativeProvider`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1E-PACKET-JET`.

Manifest signature (s).

```
(tensors: 'Mapping[int, SparseSymmetricTensor | Mapping[tuple[int, ...], Any] | Any]', *, source_dimension: 'int |
   ↪ None' = None, target_dimension: 'int | None' = None, exactness: 'Exactness' = <Exactness.NUMERICAL:
   ↪ 'numerical'>, tolerance: 'float' = 1e-10, name: 'str' = 'sparse polynomial derivative provider') -> 'None'
```

Definition. Polynomial derivatives evaluated from sparse symmetric tensor orbits.

Contract (D, C, H, R, X). Sparse tensor records require nonnegative output indices, canonical nondecreasing source-index tuples, positive finite dimensions, and coefficients compatible with the selected exactness model. A sparse polynomial provider requires one finite tensor per declared derivative order and common source and target dimensions. Principal public operations and properties are `from_dense_provider`, `degree`, `stored_entry_count`, `dense_entry_count`, `storage_ratio`, `supports_order`, `value`, `derivative`, `to_dense_provider`, `storage_certificate`, `to_dict`.

Guarantees (E, P, B, K). Orbit storage represents symmetric ordered tensor entries without storing every permutation. Contraction and dense reconstruction agree with the implemented orbit expansion. Exact object arithmetic remains exact when requested and supported. Numerical compression uses the declared tolerance. Storage savings depend on symmetry and sparsity, while dense materialization still requires the full ambient tensor size. The provenance registry records `PSC-III-D04P` as Sparse polynomial derivative provider. Its recorded status is `definition`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test module is `tests/test_nonlinear_sparse_autodiff.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 38.

C.1E · SparseSymmetricTensor

Identity (q, a, t, k). Public research-manifest route `komorebi_math.SparseSymmetricTensor`. Reflected implementation `komorebi_math.nonlinear.sparse.SparseSymmetricTensor`. The same runtime object is also available through `komorebi_math.nonlinear.SparseSymmetricTensor`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1E-PACKET-JET`.

Manifest signature (s).

```
(order: 'int', source_dimension: 'int', target_dimension: 'int', entries: 'tuple[SparseTensorEntry, ...]' = ()) ->
   ↪ None
```

Definition. One finite symmetric tensor stored by source-index orbits.

Contract (D, C, H, R, X). Sparse tensor records require nonnegative output indices, canonical nondecreasing source-index tuples, positive finite dimensions, and coefficients compatible with the selected exactness model. A sparse polynomial provider requires one finite tensor per declared derivative order and common source and target dimensions. Principal public operations and properties are `from_mapping`, `from_dense`, `stored_entry_count`, `dense_entry_count`, `expanded_nonzero_count`, `storage_ratio`, `contract`, `to_dense`, `certificate`, `to_dict`.

Guarantees (E, P, B, K). Orbit storage represents symmetric ordered tensor entries without storing every permutation. Contraction and dense reconstruction agree with the implemented orbit expansion. Exact object arithmetic remains exact when requested and supported. Numerical compression uses the declared tolerance. Storage savings depend on symmetry and sparsity, while dense materialization still requires the full ambient tensor size. The provenance registry records `PSC-III-D04S` as Sparse symmetric derivative tensor. Its recorded status is `definition`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test module is `tests/test_nonlinear_sparse_autodiff.py`. The independent verifier for nonlinear facades

checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 38.

C.1E · SparseTensorEntry

Identity (q, a, t, k). Public research-manifest route `komorebi_math.SparseTensorEntry`. Reflected implementation `komorebi_math.nonlinear.sparse.SparseTensorEntry`. The same runtime object is also available through `komorebi_math.nonlinear.SparseTensorEntry`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1E-PACKET-JET`.

Manifest signature (s).

```
(output: 'int', inputs: 'tuple[int, ...]', value: 'Any') -> None
```

Definition. One coefficient of a symmetric tensor orbit.

Contract (D, C, H, R, X). Sparse tensor records require nonnegative output indices, canonical nondecreasing source-index tuples, positive finite dimensions, and coefficients compatible with the selected exactness model. A sparse polynomial provider requires one finite tensor per declared derivative order and common source and target dimensions.

Guarantees (E, P, B, K). Orbit storage represents symmetric ordered tensor entries without storing every permutation. Contraction and dense reconstruction agree with the implemented orbit expansion. Exact object arithmetic remains exact when requested and supported. Numerical compression uses the declared tolerance. Storage savings depend on symmetry and sparsity, while dense materialization still requires the full ambient tensor size.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test module is `tests/test_nonlinear_sparse_autodiff.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 38.

Execution, snapshots, streaming, and profiling (35 cards).

C.1E · StreamedPacketJetEvaluation

Identity (q, a, t, k). Public research-manifest route `komorebi_math.StreamedPacketJetEvaluation`. Reflected implementation `komorebi_math.nonlinear.streaming.StreamedPacketJetEvaluation`. The same runtime object is also available through `komorebi_math.nonlinear.StreamedPacketJetEvaluation`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1E-EXECUTION`.

Manifest signature (s).

```
(value: 'np.ndarray', coefficient_start: 'int', coefficient_stop: 'int', total_coefficients: 'int',
    ↪ complete_snapshot: 'bool', certificate: 'Certificate') -> None
```

Definition. One chunk-wise packet-jet evaluation with an explicit coverage record.

Contract (D, C, H, R, X). Chunked streams use fixed schema identifiers and version 1 envelopes. Writers require a destination, a positive chunk size, and explicit overwrite or resume policy. Direct capture writes to a partial directory and promotes it only after completion. Readers reject duplicate JSON keys, unsupported schemas, inconsistent ranges, digest mismatches, and incomplete streams unless incomplete access is explicitly allowed.

Guarantees (E, P, B, K). Each committed chunk has a payload digest and a manifest range. Resume continues only from a verified committed prefix. Selection and evaluation can operate with memory bounded by the configured chunk size. Loading a conventional snapshot requires a complete stream and an explicit coefficient ceiling. Digests provide integrity against accidental corruption, not authentication against an adversary able to replace the entire stream and its hashes.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_nonlinear_snapshots_persistence.py` and `tests/test_block11_streaming_migrations.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 43.

Derivatives and composition (15 cards).

C.1E · SympyDerivativeProvider

Identity (q, a, t, k). Public research-manifest route `komorebi_math.SympyDerivativeProvider`. Reflected implementation `komorebi_math.nonlinear.derivatives.SympyDerivativeProvider`. The same runtime object is also available through `komorebi_math.nonlinear.SympyDerivativeProvider`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1E-DERIVATIVE`.

Manifest signature (s).

```
(expressions: 'Sequence[Any]', variables: 'Sequence[Any]', *, name: 'str' = 'SymPy derivative provider') -> 'None'
```

Definition. Exact directional derivatives of a finite SymPy expression vector.

Contract (D, C, H, R, X). Derivative providers expose one common finite dimensional interface for map evaluation and directional derivatives. A call must use an order supported by the provider and exactly that many direction vectors of the declared source dimension. Callable and polynomial providers trust explicitly supplied derivative data. The finite difference provider additionally requires a positive finite step. The SymPy provider requires the optional symbolic dependency. Principal public operations and properties are `supports_order`, `value`, `derivative`.

Guarantees (E, P, B, K). Polynomial and symbolic providers preserve exact objects where their inputs permit exact arithmetic. Callable providers inherit the exactness declared by the caller. Finite differences return numerical evidence only. Every provider checks dimensions and finite values before evaluation. Tensor contraction cost grows with derivative order and ambient dimension. The provenance registry records `PSC-III-E05` as Exact symbolic directional derivatives. Its recorded status is `finite certificate`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_nonlinear_jets_composition.py` and `tests/test_nonlinear_sparse_autodiff.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapters 37 and 38.

C.1E · TorchDerivativeProvider

Identity (q, a, t, k). Public research-manifest route `komorebi_math.TorchDerivativeProvider`. Reflected implementation `komorebi_math.nonlinear.autodiff.TorchDerivativeProvider`. The same runtime object is also available through `komorebi_math.nonlinear.TorchDerivativeProvider`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1E-DERIVATIVE`.

Manifest signature (s).

```
(function: 'Callable[[Any], Any]', source_dimension: 'int', target_dimension: 'int', max_order: 'int' = 4, dtype:
    ↪ 'str' = 'float64', device: 'str | None' = None, name: 'str' = 'PyTorch derivative provider', exactness:
    ↪ 'Exactness' = <Exactness.NUMERICAL: 'numerical'>) -> None
```

Definition. Nested `torch.func.jvp` derivatives with lazy framework import.

Contract (D, C, H, R, X). Automatic differentiation adapters require a callable finite dimensional map, positive source and target dimensions, a declared maximum order, and a directional Jacobian vector product primitive supplied either directly or by an optional framework. Direction counts must equal the requested derivative order. Framework discovery is isolated in child processes so that a broken optional runtime cannot terminate the calling process. Principal public operations and properties are `available`, `value`, `derivative`, `to_dict`.

Guarantees (E, P, B, K). A successful provider returns the finite directional derivative computed by the selected JVP recursion and records numerical exactness unless the caller supplies a stronger justified classification. Framework availability means that an isolated probe imported the runtime and found the required JVP capability. It does not assert numerical agreement for every model or device. Work grows with derivative order and the number of nested JVP calls. The provenance registry records `PSC-III-N04T` as Lazy PyTorch derivative adapter. Its recorded status is `finite certificate`. The provenance record notes that device and precision behavior are controlled by the external PyTorch runtime.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_nonlinear_sparse_autodiff.py`, `tests/test_block10_sparse_autodiff.py`, and `tests/test_optional_backend_probe.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 37.

Defects, lifting, grading, and transport (27 cards).

C.1E · `TransportEdge`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.TransportEdge`. Reflected implementation `komorebi_math.nonlinear.holonomy.TransportEdge`. The same runtime object is also available through `komorebi_math.nonlinear.TransportEdge`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1E-NONLINEAR-CALCULUS`.

Manifest signature (s).

```
(source: 'str', target: 'str', matrix: 'Any', name: 'str' = '') -> None
```

Definition. An invertible transport from `source` to `target`.

Contract (D, C, H, R, X). A transport graph requires a finite vertex set and directed edges carrying compatible square matrices. Path transport requires composable endpoints. Gluing and intertwining checks require a chosen root and finite tolerance. Fundamental holonomy is computed from the selected spanning structure and recorded non tree edges. Principal public operations and properties are `reversed_name`.

Guarantees (E, P, B, K). The graph routines compute exact path products for exact input matrices or numerical products otherwise. Certificates compare finite cycle transports, root gauges, and intertwining residuals. The result describes the supplied finite graph. It is not a continuum connection or topological classification without additional hypotheses.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test module is `tests/test_holonomy.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 40.

C.1E · `TransportGraph`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.TransportGraph`. Reflected implementation `komorebi_math.nonlinear.holonomy.TransportGraph`. The same runtime object is also available through `komorebi_math.nonlinear.TransportGraph`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1E-NONLINEAR-CALCULUS`.

Manifest signature (s).

```
(vertices: 'Iterable[str]', edges: 'Iterable[TransportEdge]', *, backend: 'str | LinearAlgebraBackend | None' =
    ↪ None, tolerance: 'float' = 1e-10) -> 'None'
```

Definition. A connected finite graph with invertible matrix transports.

Contract (D, C, H, R, X). A transport graph requires a finite vertex set and directed edges carrying compatible square matrices. Path transport requires composable endpoints. Gluing and intertwining checks require a chosen root and

finite tolerance. Fundamental holonomy is computed from the selected spanning structure and recorded non tree edges. Principal public operations and properties are `edge_label`, `oriented_matrix`, `spanning_tree`, `tree_transports`, `tree_gauge`, `gauge_transform`, `fundamental_holonomies`, `gluing_certificate`, `intertwine`.

Guarantees (`E`, `P`, `B`, `K`). The graph routines compute exact path products for exact input matrices or numerical products otherwise. Certificates compare finite cycle transports, root gauges, and intertwining residuals. The result describes the supplied finite graph. It is not a continuum connection or topological classification without additional hypotheses. The provenance registry records `PSC-III-D38` as Finite matrix transport system. Its recorded status is `definition`.

Evidence and links (`V`, `L`). The committed research manifest and runtime reflection audit identify this root facade. The focused test module is `tests/test_holonomy.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 40.

Derivatives and composition (15 cards).

C.1E · `autodiff_provider`

Identity (`q`, `a`, `t`, `k`). Public research-manifest route `komorebi_math.autodiff_provider`. Reflected implementation `komorebi_math.nonlinear.autodiff.autodiff_provider`. The same runtime object is also available through `komorebi_math.nonlinear.autodiff_provider`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1E-DERIVATIVE`.

Manifest signature (`s`).

```
(framework: 'str', function: 'Callable[[Any], Any]', source_dimension: 'int', target_dimension: 'int', *,
   ↪  max_order: 'int' = 4, **options: 'Any') -> 'JaxDerivativeProvider | TorchDerivativeProvider'
```

Definition. Construct one supported external automatic-differentiation adapter.

Contract (`D`, `C`, `H`, `R`, `X`). Automatic differentiation adapters require a callable finite dimensional map, positive source and target dimensions, a declared maximum order, and a directional Jacobian vector product primitive supplied either directly or by an optional framework. Direction counts must equal the requested derivative order. Framework discovery is isolated in child processes so that a broken optional runtime cannot terminate the calling process.

Guarantees (`E`, `P`, `B`, `K`). A successful provider returns the finite directional derivative computed by the selected JVP recursion and records numerical exactness unless the caller supplies a stronger justified classification. Framework availability means that an isolated probe imported the runtime and found the required JVP capability. It does not assert numerical agreement for every model or device. Work grows with derivative order and the number of nested JVP calls.

Evidence and links (`V`, `L`). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_nonlinear_sparse_autodiff.py`, `tests/test_block10_sparse_autodiff.py`, and `tests/test_optional_backend_probe.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 37.

C.1E · `automatic_derivative_provider`

Identity (`q`, `a`, `t`, `k`). Public research-manifest route `komorebi_math.automatic_derivative_provider`. Reflected implementation `komorebi_math.nonlinear.autodiff.autodiff_provider`. The same runtime object is also available through `komorebi_math.nonlinear.automatic_derivative_provider`. The public export name maps to implementation object `autodiff_provider`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1E-DERIVATIVE`.

Manifest signature (`s`).

```
(framework: 'str', function: 'Callable[[Any], Any]', source_dimension: 'int', target_dimension: 'int', *,
    ↪ max_order: 'int' = 4, **options: 'Any') -> 'JaxDerivativeProvider | TorchDerivativeProvider'
```

Definition. Construct one supported external automatic-differentiation adapter.

Contract (`D, C, H, R, X`). Automatic differentiation adapters require a callable finite dimensional map, positive source and target dimensions, a declared maximum order, and a directional Jacobian vector product primitive supplied either directly or by an optional framework. Direction counts must equal the requested derivative order. Framework discovery is isolated in child processes so that a broken optional runtime cannot terminate the calling process.

Guarantees (`E, P, B, K`). A successful provider returns the finite directional derivative computed by the selected JVP recursion and records numerical exactness unless the caller supplies a stronger justified classification. Framework availability means that an isolated probe imported the runtime and found the required JVP capability. It does not assert numerical agreement for every model or device. Work grows with derivative order and the number of nested JVP calls.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_nonlinear_sparse_autodiff.py`, `tests/test_block10_sparse_autodiff.py`, and `tests/test_optional_backend_probe.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 37.

C.1E · `available_ad_frameworks`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.available_ad_frameworks`. Reflected implementation `komorebi_math.nonlinear.autodiff.available_ad_frameworks`. The same runtime object is also available through `komorebi_math.nonlinear.available_ad_frameworks`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1E-DERIVATIVE`.

Manifest signature (`s`).

```
() -> 'dict[str, bool]'
```

Definition. Report usable optional AD frameworks without importing them in-process.

Contract (`D, C, H, R, X`). Automatic differentiation adapters require a callable finite dimensional map, positive source and target dimensions, a declared maximum order, and a directional Jacobian vector product primitive supplied either directly or by an optional framework. Direction counts must equal the requested derivative order. Framework discovery is isolated in child processes so that a broken optional runtime cannot terminate the calling process.

Guarantees (`E, P, B, K`). A successful provider returns the finite directional derivative computed by the selected JVP recursion and records numerical exactness unless the caller supplies a stronger justified classification. Framework availability means that an isolated probe imported the runtime and found the required JVP capability. It does not assert numerical agreement for every model or device. Work grows with derivative order and the number of nested JVP calls.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_nonlinear_sparse_autodiff.py`, `tests/test_block10_sparse_autodiff.py`, and `tests/test_optional_backend_probe.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 37.

C.1E · available_autodiff_backends

Identity (q, a, t, k). Public research-manifest route `komorebi_math.available_autodiff_backends`. Reflected implementation `komorebi_math.nonlinear.autodiff.available_ad_frameworks`. The same runtime object is also available through `komorebi_math.nonlinear.available_autodiff_backends`. The public export name maps to implementation object `available_ad_frameworks`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1E-DERIVATIVE`.

Manifest signature (s).

```
() -> 'dict[str, bool]'
```

Definition. Report usable optional AD frameworks without importing them in-process.

Contract (D, C, H, R, X). Automatic differentiation adapters require a callable finite dimensional map, positive source and target dimensions, a declared maximum order, and a directional Jacobian vector product primitive supplied either directly or by an optional framework. Direction counts must equal the requested derivative order. Framework discovery is isolated in child processes so that a broken optional runtime cannot terminate the calling process.

Guarantees (E, P, B, K). A successful provider returns the finite directional derivative computed by the selected JVP recursion and records numerical exactness unless the caller supplies a stronger justified classification. Framework availability means that an isolated probe imported the runtime and found the required JVP capability. It does not assert numerical agreement for every model or device. Work grows with derivative order and the number of nested JVP calls.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_nonlinear_sparse_autodiff.py`, `tests/test_block10_sparse_autodiff.py`, and `tests/test_optional_backend_probe.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 37.

Defects, lifting, grading, and transport (27 cards).

C.1E · build_lift_tree

Identity (q, a, t, k). Public research-manifest route `komorebi_math.build_lift_tree`. Reflected implementation `komorebi_math.nonlinear.lifting.build_lift_tree`. The same runtime object is also available through `komorebi_math.nonlinear.build_lift_tree`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1E-NONLINEAR-CALCULUS`.

Manifest signature (s).

```
(problem_factory: 'Callable[[tuple[int, ...], int], LinearLiftProblem | PolynomialLiftProblem]', roots:
    ↪ 'Iterable[tuple[int, ...]]', *, prime: 'int', start_depth: 'int', max_depth: 'int', max_nodes: 'int' =
    ↪ 100000) -> 'FiniteLiftTree'
```

Definition. Build a finite lift tree from exact one step correction problems.

Contract (D, C, H, R, X). Linear and polynomial lifting require a prime, a nonnegative depth, compatible finite equations, and a current residue class. One step searches the declared corrections modulo the prime. Tree construction requires an explicit terminal depth and a finite node budget.

Guarantees (E, P, B, K). For the lifting operations in this family, every returned child satisfies the represented congruence at the next prime power. Linear solves and branch verification are exact in modular arithmetic. Polynomial lifting uses the declared Jacobian and finite Taylor congruence. Branch counts may grow exponentially with depth, so the tree builder enforces the supplied node limit. The provenance registry records `PSC-III-T30` as Finite lift tree construction. Its recorded status is `proved`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test module is `tests/test_lifting.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe

execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 40.

Execution, snapshots, streaming, and profiling (35 cards).

C.1E · capture_packet_jet_stream

Identity (q, a, t, k). Public research-manifest route `komorebi_math.capture_packet_jet_stream`. Reflected implementation `komorebi_math.nonlinear.streaming.capture_packet_jet_stream`. The same runtime object is also available through `komorebi_math.nonlinear.capture_packet_jet_stream`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1E-EXECUTION`.

Manifest signature (s).

```
(jet: 'PacketJet', path: 'str | os.PathLike[str]', *, tolerance: 'float | None' = None, max_evaluations: 'int' =
    ↪ 10000000, chunk_size: 'int' = 1024, evaluation_budget: 'int | None' = None, resume: 'bool' = False,
    ↪ overwrite: 'bool' = False) -> 'PacketJetStreamWriteResult'
```

Definition. Capture a packet jet directly into bounded, resumable basis chunks.

Contract (D, C, H, R, X). Chunked streams use fixed schema identifiers and version 1 envelopes. Writers require a destination, a positive chunk size, and explicit overwrite or resume policy. Direct capture writes to a partial directory and promotes it only after completion. Readers reject duplicate JSON keys, unsupported schemas, inconsistent ranges, digest mismatches, and incomplete streams unless incomplete access is explicitly allowed.

Guarantees (E, P, B, K). Each committed chunk has a payload digest and a manifest range. Resume continues only from a verified committed prefix. Selection and evaluation can operate with memory bounded by the configured chunk size. Loading a conventional snapshot requires a complete stream and an explicit coefficient ceiling. Digests provide integrity against accidental corruption, not authentication against an adversary able to replace the entire stream and its hashes. The provenance registry records `PSC-III-D60` as Chunked resumable packet-jet capture. Its recorded status is `finite certificate`. Recorded limitations state that integrity digests are not signatures and that resuming assumes the provider behavior still agrees with its recorded identity.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_nonlinear_snapshots_persistence.py` and `tests/test_block11_streaming_migrations.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 43.

C.1E · compile_packet_jet_plan

Identity (q, a, t, k). Public research-manifest route `komorebi_math.compile_packet_jet_plan`. Reflected implementation `komorebi_math.nonlinear.execution.compile_packet_jet_plan`. The same runtime object is also available through `komorebi_math.nonlinear.compile_packet_jet_plan`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1E-EXECUTION`.

Manifest signature (s).

```
(family: 'PacketJetFamily', support: 'PacketJetSupport | None' = None, *, max_addresses: 'int' = 1000000) ->
    ↪ 'PacketJetExecutionPlan'
```

Definition. Compile a projection-cached plan, pruning only certified zero blocks.

Contract (D, C, H, R, X). Execution plans require a finite packet-jet family and an optional support record for the same order and packet names. Only blocks certified zero may be pruned. Plan compilation enforces an explicit address budget. Execution requires one direction per derivative slot and checks that the plan still matches the family. CuPy execution additionally requires an importable CuPy runtime and a CUDA device.

Guarantees (E, P, B, K). The CPU planner groups active output blocks by common projected inputs, caches those projections, and reports derivative-call and projection counts. Reconstruction is compared with the direct represented

derivative. Optional CuPy classes preserve the same finite packet contract when the hardware boundary is available. Runtime discovery alone is not a performance or correctness qualification, and unavailable device paths fail explicitly.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_nonlinear_execution.py` and `tests/test_block10_execution_persistence.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 43.

Defects, lifting, grading, and transport (27 cards).

C.1E · `conditional_projective_limit_completion`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.conditional_projective_limit_completion`. Reflected implementation `komorebi_math.nonlinear.filtered.conditional_projective_limit_completion`. The same runtime object is also available through `komorebi_math.nonlinear.conditional_projective_limit_completion`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1E-NONLINEAR-CALCULUS`.

Manifest signature (s).

```
(verified_prefix_depth: 'int', *, separated: 'bool', complete: 'bool', continuous: 'bool',
    ↪ compatible_infinite_branch: 'bool') -> 'ConditionalResult[ProjectiveLimitRecord]'
```

Definition. Record the conditional infinite filtered-lifting conclusion.

Contract (D, C, H, R, X). Filtered contracts require finite dimensional vector spaces over an explicit prime field and decreasing filtration bases with compatible ambient dimensions. Filtered correction is solved one finite depth at a time. A filtered cochain complex requires explicit differentials that square to zero and preserve the declared filtration. Projective limit completion also requires external completeness hypotheses supplied by the caller.

Guarantees (E, P, B, K). Associated graded coordinates, correction cosets, finite branches, and spectral sequence pages are exact over the represented finite field. The terminal finite filtration permits exact page dimension and stabilization checks. A projective limit record only stores the declared infinite hypotheses and finite compatibility evidence. It does not prove completeness or convergence beyond those assumptions.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test module is `tests/test_nonlinear_filtered.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 42.

Execution, snapshots, streaming, and profiling (35 cards).

C.1E · `cupy_runtime_status`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.cupy_runtime_status`. Reflected implementation `komorebi_math.nonlinear.gpu.cupy_runtime_status`. The same runtime object is also available through `komorebi_math.nonlinear.cupy_runtime_status`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1E-EXECUTION`.

Manifest signature (s).

```
() -> 'CuPyRuntimeStatus'
```

Definition. Inspect CuPy and CUDA availability without raising on absent hardware.

Contract (D, C, H, R, X). GPU status inspection never requires hardware and returns the observed CuPy import and CUDA device state. Snapshot execution requires a complete finite snapshot, a working CuPy CUDA runtime, and

compatible probe shapes. Validation compares the same persisted coefficients on CPU and device under a declared tolerance.

Guarantees (E, P, B, K). A successful validation record reports the probe count, maximum residual, runtime state, and certificate. The device executor evaluates only the captured finite snapshot and does not accept arbitrary Python callables. No speed claim follows from API presence. In a runtime without CuPy or a CUDA device, status reporting remains available while execution and validation are not hardware-qualified.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_block16_gpu_qualification.py` and `tests/test_nonlinear_execution.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 43. The recorded reference environment had no CuPy CUDA device, so GPU evidence is limited to status reporting, explicit unavailability, and CPU-side contract checks.

Defects, lifting, grading, and transport (27 cards).

C.1E · `direct_packet_rotation`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.direct_packet_rotation`. Reflected implementation `komorebi_math.nonlinear.transport.direct_packet_rotation`. The same runtime object is also available through `komorebi_math.nonlinear.direct_packet_rotation`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1E-NONLINEAR-CALCULUS`.

Manifest signature (s).

```
(source: 'Iterable[Any]', target: 'Iterable[Any]', *, tolerance: 'float' = 1e-10) -> 'DirectRotationResult'
```

Definition. Return the polar unitary of `sum target[a] source[a]`.

Contract (D, C, H, R, X). Smooth transport requires complete finite orthogonal packet families of fixed dimension together with declared packet derivatives. Direct rotation requires two compatible packet families whose overlap sum is numerically invertible. Kato transport requires finite start and end times, a positive step count, and a positive tolerance.

Guarantees (E, P, B, K). Direct polar rotation returns a finite unitary intertwiner when its residual checks pass. Kato transport integrates the canonical finite generator with midpoint exponential steps and records unitarity, packet transport, and mesh refinement residuals. These certificates are numerical and path specific. They do not prove differentiability of an undeclared family or continuum convergence without external estimates. The provenance registry records `PSC-III-T55` as Direct rotation of finite packet systems. Its recorded status is `proved`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_nonlinear_transport.py`, and `tests/test_quantum_transport_effective_spectral.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapters 40 and 44.

C.1E · `divided_square_packets`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.divided_square_packets`. Reflected implementation `komorebi_math.nonlinear.graded.divided_square_packets`. The same runtime object is also available through `komorebi_math.nonlinear.divided_square_packets`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1E-NONLINEAR-CALCULUS`.

Manifest signature (s).

```
(group: 'FiniteAbelianGroup', components: 'Mapping[str, Any]', differential: 'LinearOperation', bracket:
    ↪ 'BracketOperation', divided_square: 'LinearOperation', *, exactness: 'Exactness' = <Exactness.NUMERICAL:
    ↪ 'numerical'>, tolerance: 'float' = 1e-10) -> 'DividedSquareResult'
```

Definition. Assemble `d eta + q(eta)` from divided-square packet identities.

Contract (D, C, H, R, X). Graded operations require a finite abelian grading group, finite component records, and multilinear operations whose declared output grade is the group sum of the input grades. Maurer Cartan routines evaluate only the supplied finite arity range. Divided square evaluation requires the supplied differential, bracket, and divided square maps to have compatible component shapes.

Guarantees (E, P, B, K). Grade bookkeeping and finite group arithmetic are exact. Numerical component values inherit the arithmetic of the supplied arrays and operations. Certificates compare the gradewise reconstruction with the direct finite expression and record any failed compatibility law. No unspecified higher operation or infinite Maurer Cartan series is inferred.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test module is `tests/test_nonlinear_graded.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 39.

Execution, snapshots, streaming, and profiling (35 cards).

C.1E · `estimate_packet_jet_complexity`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.estimate_packet_jet_complexity`. Reflected implementation `komorebi_math.nonlinear.profiling.estimate_packet_jet_complexity`. The same runtime object is also available through `komorebi_math.nonlinear.estimate_packet_jet_complexity`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1E-EXECUTION`.

Manifest signature (s).

```
(family: 'PacketJetFamily', support: 'PacketJetSupport | None' = None, *, plan: 'PacketJetExecutionPlan | None' =
    ↪ None, max_addresses: 'int' = 1000000) -> 'PacketJetComplexityEstimate'
```

Definition. Return exact finite operation counts for a packet-jet family.

Contract (D, C, H, R, X). Complexity estimation requires a finite family and a compatible optional support record or execution plan. Runtime profiles require positive repetition counts, nonnegative warmup counts, and a precisely stated measurement scope. Packet-jet profiling may include planning, execution, snapshot capture, and optional stream writing under explicit budgets.

Guarantees (E, P, B, K). Deterministic estimate fields are exact finite counts derived from the represented dimensions, packets, addresses, and plan. Timing and traced Python allocation samples are environment-dependent observations. They are not asymptotic evidence and do not include unreported device allocation or external process cost. Reports retain the interpreter, platform, scope, repetitions, and certificate needed to interpret the measurements.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_block11_golden_profiling_beta.py` and `tests/test_nonlinear_execution.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 43.

C.1E · execute_packet_jet_plan

Identity (q, a, t, k). Public research-manifest route `komorebi_math.execute_packet_jet_plan`. Reflected implementation `komorebi_math.nonlinear.execution.execute_packet_jet_plan`. The same runtime object is also available through `komorebi_math.nonlinear.execute_packet_jet_plan`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1E-EXECUTION`.

Manifest signature (s).

```
(plan: 'PacketJetExecutionPlan', family: 'PacketJetFamily', directions: 'Sequence[Any]', *, tolerance: 'float |
    ↪ None' = None) -> 'PacketJetBatchResult'
```

Definition. Execute a packet-jet plan with cached input projections.

Contract (D, C, H, R, X). Execution plans require a finite packet-jet family and an optional support record for the same order and packet names. Only blocks certified zero may be pruned. Plan compilation enforces an explicit address budget. Execution requires one direction per derivative slot and checks that the plan still matches the family. CuPy execution additionally requires an importable CuPy runtime and a CUDA device.

Guarantees (E, P, B, K). The CPU planner groups active output blocks by common projected inputs, caches those projections, and reports derivative-call and projection counts. Reconstruction is compared with the direct represented derivative. Optional CuPy classes preserve the same finite packet contract when the hardware boundary is available. Runtime discovery alone is not a performance or correctness qualification, and unavailable device paths fail explicitly.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_nonlinear_execution.py` and `tests/test_block10_execution_persistence.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 43.

C.1E · execute_packet_jets

Identity (q, a, t, k). Public research-manifest route `komorebi_math.execute_packet_jets`. Reflected implementation `komorebi_math.nonlinear.execution.execute_packet_jets`. The same runtime object is also available through `komorebi_math.nonlinear.execute_packet_jets`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1E-EXECUTION`.

Manifest signature (s).

```
(family: 'PacketJetFamily', directions: 'Sequence[Any]', *, support: 'PacketJetSupport | None' = None, tolerance:
    ↪ 'float | None' = None, max_addresses: 'int' = 1000000) -> 'PacketJetBatchResult'
```

Definition. Compile and execute one projection-cached packet-jet family.

Contract (D, C, H, R, X). Execution plans require a finite packet-jet family and an optional support record for the same order and packet names. Only blocks certified zero may be pruned. Plan compilation enforces an explicit address budget. Execution requires one direction per derivative slot and checks that the plan still matches the family. CuPy execution additionally requires an importable CuPy runtime and a CUDA device.

Guarantees (E, P, B, K). The CPU planner groups active output blocks by common projected inputs, caches those projections, and reports derivative-call and projection counts. Reconstruction is compared with the direct represented derivative. Optional CuPy classes preserve the same finite packet contract when the hardware boundary is available. Runtime discovery alone is not a performance or correctness qualification, and unavailable device paths fail explicitly.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_nonlinear_execution.py` and `tests/test_block10_execution_persistence.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator

for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 43.

Derivatives and composition (15 cards).

C.1E · faa_di_bruno

Identity (q, a, t, k). Public research-manifest route `komorebi_math.faa_di_bruno`. Reflected implementation `komorebi_math.nonlinear.composition.faa_di_bruno`. The same runtime object is also available through `komorebi_math.nonlinear.faa_di_bruno`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1E-DERIVATIVE`.

Manifest signature (s).

```
(outer: 'DerivativeProvider', inner: 'DerivativeProvider', point: 'Any', directions: 'Sequence[Any]', *,
    ↪ direct_provider: 'DerivativeProvider | None' = None, tolerance: 'float' = 1e-09, retain_terms: 'bool' =
    ↪ True) -> 'FaaDiBrunoResult'
```

Definition. Evaluate the directional higher chain rule for `outer(inner(x))`.

Contract (D, C, H, R, X). Composition requires compatible inner and outer dimensions and derivative support through the requested order. The Faà di Bruno routines enumerate set partitions of the direction labels. Packet composition also requires compatible source, intermediate, and target packet systems. Partition utilities accept nonnegative finite orders and return canonical immutable records.

Guarantees (E, P, B, K). For the composition operations in this family, the computed derivative value equals the implemented finite Faà di Bruno sum for the supplied providers and directions. Exactness follows the participating providers. The composition result records retained terms, partition counts, and a certificate comparing the composed derivative with the direct implementation when that comparison is available. Enumeration grows according to Bell and Stirling counts, so callers must keep the derivative order bounded. The provenance registry records `PSC-III-T10` as Finite dimensional Faà di Bruno formula. Its recorded status is `classical input`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test module is `tests/test_nonlinear_jets_composition.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 38.

Defects, lifting, grading, and transport (27 cards).

C.1E · higher_maurer_cartan_packets

Identity (q, a, t, k). Public research-manifest route `komorebi_math.higher_maurer_cartan_packets`. Reflected implementation `komorebi_math.nonlinear.graded.higher_maurer_cartan_packets`. The same runtime object is also available through `komorebi_math.nonlinear.higher_maurer_cartan_packets`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1E-NONLINEAR-CALCULUS`.

Manifest signature (s).

```
(group: 'FiniteAbelianGroup', components: 'Mapping[str, Any]', operations: 'Mapping[int,
    ↪ GradedOperationCallable]', *, exactness: 'Exactness' = <Exactness.NUMERICAL: 'numerical'>, tolerance:
    ↪ 'float' = 1e-10) -> 'HigherMaurerCartanResult'
```

Definition. Assemble a finite truncation `sum_r ell_r(eta^r)/r!` by labels.

Contract (D, C, H, R, X). Graded operations require a finite abelian grading group, finite component records, and multilinear operations whose declared output grade is the group sum of the input grades. Maurer Cartan routines evaluate only the supplied finite arity range. Divided square evaluation requires the supplied differential, bracket, and divided square maps to have compatible component shapes.

Guarantees (E, P, B, K). Grade bookkeeping and finite group arithmetic are exact. Numerical component values inherit the arithmetic of the supplied arrays and operations. Certificates compare the gradewise reconstruction

with the direct finite expression and record any failed compatibility law. No unspecified higher operation or infinite Maurer Cartan series is inferred. The provenance registry records `PSC-III-P20` as Finite truncated packet equations for higher operations. Its recorded status is `proved`. The provenance record notes that this result does not certify convergence of an infinite Maurer–Cartan series.

Evidence and links (`V`, `L`). The committed research manifest and runtime reflection audit identify this root facade. The focused test module is `tests/test_nonlinear_graded.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 39.

Execution, snapshots, streaming, and profiling (35 cards).

C.1E · `load_packet_jet_snapshot_stream`

Identity (`q`, `a`, `t`, `k`). Public research-manifest route `komorebi_math.load_packet_jet_snapshot_stream`. Reflected implementation `komorebi_math.nonlinear.streaming.load_packet_jet_snapshot_stream`. The same runtime object is also available through `komorebi_math.nonlinear.load_packet_jet_snapshot_stream`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1E-EXECUTION`.

Manifest signature (`s`).

```
(path: 'str | os.PathLike[str]', *, max_coefficients: 'int' = 1000000, verify_chunks: 'bool' = True) ->
   ↪  'PacketJetSnapshot'
```

Definition. Load a complete stream into a conventional in-memory snapshot.

Contract (`D`, `C`, `H`, `R`, `X`). Chunked streams use fixed schema identifiers and version 1 envelopes. Writers require a destination, a positive chunk size, and explicit overwrite or resume policy. Direct capture writes to a partial directory and promotes it only after completion. Readers reject duplicate JSON keys, unsupported schemas, inconsistent ranges, digest mismatches, and incomplete streams unless incomplete access is explicitly allowed.

Guarantees (`E`, `P`, `B`, `K`). Each committed chunk has a payload digest and a manifest range. Resume continues only from a verified committed prefix. Selection and evaluation can operate with memory bounded by the configured chunk size. Loading a conventional snapshot requires a complete stream and an explicit coefficient ceiling. Digests provide integrity against accidental corruption, not authentication against an adversary able to replace the entire stream and its hashes.

Evidence and links (`V`, `L`). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_nonlinear_snapshots_persistence.py` and `tests/test_block11_streaming_migrations.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 43.

Rooted-tree expansions and majorants (6 cards).

C.1E · `majorant_convergence`

Identity (`q`, `a`, `t`, `k`). Public research-manifest route `komorebi_math.majorant_convergence`. Reflected implementation `komorebi_math.nonlinear.trees.majorant_convergence`. The same runtime object is also available through `komorebi_math.nonlinear.majorant_convergence`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1E-ROOTED-TREES`.

Manifest signature (s).

```
(seed_norm: 'float', operation_norms: 'Mapping[int, float]', radius: 'float', *, tolerance: 'float' = 1e-12) ->
    ↪ 'MajorantConvergence'
```

Definition. Validate `phi(R)<=R` and `sum r*b_r*R^(r-1)<1`.

Contract (D, C, H, R, X). Ordered-tree enumeration requires internal arities of at least two, nonnegative height or positive leaf targets, and an explicit tree budget. Polynomial fixed-point problems require a finite seed, compatible multilinear operations, and declared exactness and tolerance. Majorant routines require finite nonnegative norm bounds and a positive candidate radius.

Guarantees (E, P, B, K). Tree counts and finite enumerations are exact within the requested boundary. A tree expansion equals the matching finite Picard iterate under the implemented polynomial operations. A convergence certificate is issued only when the supplied scalar majorant maps the declared ball into itself and has contraction factor below one. Packet reconstruction prunes only addresses covered by supplied zero-support certificates. Enumeration can grow exponentially or faster with height and arity. The provenance registry records `PSC-III-T14` as Polynomial rooted-tree convergence certificate. Its recorded status is `proved`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test module is `tests/test_nonlinear_trees.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 39.

Defects, lifting, grading, and transport (27 cards).

C.1E · `maurer_cartan_packets`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.maurer_cartan_packets`. Reflected implementation `komorebi_math.nonlinear.graded.maurer_cartan_packets`. The same runtime object is also available through `komorebi_math.nonlinear.maurer_cartan_packets`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1E-NONLINEAR-CALCULUS`.

Manifest signature (s).

```
(group: 'FiniteAbelianGroup', components: 'Mapping[str, Any]', differential: 'LinearOperation', bracket:
    ↪ 'BracketOperation', *, exactness: 'Exactness' = <Exactness.NUMERICAL: 'numerical'>, tolerance: 'float' =
    ↪ 1e-10) -> 'MaurerCartanResult'
```

Definition. Assemble the finite packet equations for `d eta + [eta,eta]/2`.

Contract (D, C, H, R, X). Graded operations require a finite abelian grading group, finite component records, and multilinear operations whose declared output grade is the group sum of the input grades. Maurer Cartan routines evaluate only the supplied finite arity range. Divided square evaluation requires the supplied differential, bracket, and divided square maps to have compatible component shapes.

Guarantees (E, P, B, K). Grade bookkeeping and finite group arithmetic are exact. Numerical component values inherit the arithmetic of the supplied arrays and operations. Certificates compare the gradewise reconstruction with the direct finite expression and record any failed compatibility law. No unspecified higher operation or infinite Maurer Cartan series is inferred. The provenance registry records `PSC-III-T19` as Packet Maurer–Cartan equations. Its recorded status is `proved`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test module is `tests/test_nonlinear_graded.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 39.

Execution, snapshots, streaming, and profiling (35 cards).

C.1E · nonlinear_cupy_available

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear_cupy_available`. Reflected implementation `komorebi_math.nonlinear.execution.nonlinear_cupy_available`. The same runtime object is also available through `komorebi_math.nonlinear.nonlinear_cupy_available`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1E-EXECUTION`.

Manifest signature (s).

```
() -> 'bool'
```

Definition. Report whether a CuPy module can be discovered.

Contract (D, C, H, R, X). Execution plans require a finite packet-jet family and an optional support record for the same order and packet names. Only blocks certified zero may be pruned. Plan compilation enforces an explicit address budget. Execution requires one direction per derivative slot and checks that the plan still matches the family. CuPy execution additionally requires an importable CuPy runtime and a CUDA device.

Guarantees (E, P, B, K). The CPU planner groups active output blocks by common projected inputs, caches those projections, and reports derivative-call and projection counts. Reconstruction is compared with the direct represented derivative. Optional CuPy classes preserve the same finite packet contract when the hardware boundary is available. Runtime discovery alone is not a performance or correctness qualification, and unavailable device paths fail explicitly.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_nonlinear_execution.py` and `tests/test_block10_execution_persistence.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 43.

C.1E · open_packet_jet_snapshot_stream

Identity (q, a, t, k). Public research-manifest route `komorebi_math.open_packet_jet_snapshot_stream`. Reflected implementation `komorebi_math.nonlinear.streaming.open_packet_jet_snapshot_stream`. The same runtime object is also available through `komorebi_math.nonlinear.open_packet_jet_snapshot_stream`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1E-EXECUTION`.

Manifest signature (s).

```
(path: 'str | os.PathLike[str]', *, allow_incomplete: 'bool' = False, verify_chunks: 'bool' = False) ->
    ↪ 'PacketJetSnapshotStreamReader'
```

Definition. Open one packet-jet stream with explicit incomplete-stream policy.

Contract (D, C, H, R, X). Chunked streams use fixed schema identifiers and version 1 envelopes. Writers require a destination, a positive chunk size, and explicit overwrite or resume policy. Direct capture writes to a partial directory and promotes it only after completion. Readers reject duplicate JSON keys, unsupported schemas, inconsistent ranges, digest mismatches, and incomplete streams unless incomplete access is explicitly allowed.

Guarantees (E, P, B, K). Each committed chunk has a payload digest and a manifest range. Resume continues only from a verified committed prefix. Selection and evaluation can operate with memory bounded by the configured chunk size. Loading a conventional snapshot requires a complete stream and an explicit coefficient ceiling. Digests provide integrity against accidental corruption, not authentication against an adversary able to replace the entire stream and its hashes.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_nonlinear_snapshots_persistence.py` and `tests/test_block11_streaming_migrations.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family.

The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 43.

Derivatives and composition (15 cards).

C.1E · packet_faa_di_bruno

Identity (q, a, t, k). Public research-manifest route `komorebi_math.packet_faa_di_bruno`. Reflected implementation `komorebi_math.nonlinear.composition.packet_faa_di_bruno`. The same runtime object is also available through `komorebi_math.nonlinear.packet_faa_di_bruno`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1E-DERIVATIVE`.

Manifest signature (s).

```
(outer: 'DerivativeProvider', inner: 'DerivativeProvider', point: 'Any', directions: 'Sequence[Any]', source:
    ↪ 'PacketSystem', intermediate: 'PacketSystem', target: 'PacketSystem', *, input_packets:
    ↪ 'Sequence[PacketLabel]', output_packet: 'PacketLabel', inner_support: 'Mapping[int, PacketJetSupport] |
    ↪ None' = None, outer_support: 'Mapping[int, PacketJetSupport] | None' = None, tolerance: 'float | None' =
    ↪ None, retain_terms: 'bool' = True, max_terms: 'int' = 1000000) -> 'FaaDiBrunoResult'
```

Definition. Evaluate one packet jet of a composition through decorated partitions.

Contract (D, C, H, R, X). Composition requires compatible inner and outer dimensions and derivative support through the requested order. The Faà di Bruno routines enumerate set partitions of the direction labels. Packet composition also requires compatible source, intermediate, and target packet systems. Partition utilities accept nonnegative finite orders and return canonical immutable records.

Guarantees (E, P, B, K). For the composition operations in this family, the computed derivative value equals the implemented finite Faà di Bruno sum for the supplied providers and directions. Exactness follows the participating providers. The composition result records retained terms, partition counts, and a certificate comparing the composed derivative with the direct implementation when that comparison is available. Enumeration grows according to Bell and Stirling counts, so callers must keep the derivative order bounded. The provenance registry records `PSC-III-T11A` as Packet Faà di Bruno formula. Its recorded status is `proved`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test module is `tests/test_nonlinear_jets_composition.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 38.

Execution, snapshots, streaming, and profiling (35 cards).

C.1E · packet_jet_benchmark_contract

Identity (q, a, t, k). Public research-manifest route `komorebi_math.packet_jet_benchmark_contract`. Reflected implementation `komorebi_math.nonlinear.execution.packet_jet_benchmark_contract`. The same runtime object is also available through `komorebi_math.nonlinear.packet_jet_benchmark_contract`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1E-EXECUTION`.

Manifest signature (s).

```
(family: 'PacketJetFamily', *, backend: 'str', timing_scope: 'str' = 'kernel') -> 'Any'
```

Definition. Create a matched benchmark contract without importing algorithms eagerly.

Contract (D, C, H, R, X). Execution plans require a finite packet-jet family and an optional support record for the same order and packet names. Only blocks certified zero may be pruned. Plan compilation enforces an explicit address budget. Execution requires one direction per derivative slot and checks that the plan still matches the family. CuPy execution additionally requires an importable CuPy runtime and a CUDA device.

Guarantees (E, P, B, K). The CPU planner groups active output blocks by common projected inputs, caches those projections, and reports derivative-call and projection counts. Reconstruction is compared with the direct represented

derivative. Optional CuPy classes preserve the same finite packet contract when the hardware boundary is available. Runtime discovery alone is not a performance or correctness qualification, and unavailable device paths fail explicitly.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_nonlinear_execution.py` and `tests/test_block10_execution_persistence.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 43.

Packet jets and sparse tensors (7 cards).

C.1E · packet_jet_support

Identity (q, a, t, k). Public research-manifest route `komorebi_math.packet_jet_support`. Reflected implementation `komorebi_math.nonlinear.jets.packet_jet_support`. The same runtime object is also available through `komorebi_math.nonlinear.packet_jet_support`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1E-PACKET-JET`.

Manifest signature (s).

```
(family: 'PacketJetFamily', *, tolerance: 'float | None' = None, max_evaluations: 'int' = 100000) ->
    ↪ 'PacketJetSupport'
```

Definition. Classify packet jet blocks by exhaustive packet-range basis tests.

Contract (D, C, H, R, X). Packet jets require a derivative provider, compatible finite source and target packet systems, a valid base point, and a positive supported derivative order. Packet labels must resolve inside their systems. Evaluation requires one source direction per input address. Support discovery and Taylor assembly also require positive finite tolerances and explicit evaluation budgets.

Guarantees (E, P, B, K). A packet jet evaluates the declared derivative after source projection and target projection. A complete family reconstructs the represented finite derivative when both packet systems are complete. Support classifications are exhaustive only when the evaluation budget covers every required packet basis probe. Taylor certificates compare one finite truncation with the observed map value at the supplied increment. Address counts and support work grow combinatorially with order and packet count. The provenance registry records `PSC-III-C08` as Finite packet jet support test. Its recorded status is `finite certificate`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_nonlinear_jets_composition.py` and `tests/test_nonlinear_execution.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 38.

C.1E · packet_taylor

Identity (q, a, t, k). Public research-manifest route `komorebi_math.packet_taylor`. Reflected implementation `komorebi_math.nonlinear.jets.packet_taylor`. The same runtime object is also available through `komorebi_math.nonlinear.packet_taylor`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1E-PACKET-JET`.

Manifest signature (s).

```
(provider: 'DerivativeProvider', point: 'Any', increment: 'Any', source: 'PacketSystem', target: 'PacketSystem',
    ↪ order: 'int', *, tolerance: 'float | None' = None) -> 'PacketTaylorResult'
```

Definition. Assemble a finite Taylor polynomial through packet jet reconstruction.

Contract (D, C, H, R, X). Packet jets require a derivative provider, compatible finite source and target packet systems, a valid base point, and a positive supported derivative order. Packet labels must resolve inside their systems. Evalua-

tion requires one source direction per input address. Support discovery and Taylor assembly also require positive finite tolerances and explicit evaluation budgets.

Guarantees (`E, P, B, K`). A packet jet evaluates the declared derivative after source projection and target projection. A complete family reconstructs the represented finite derivative when both packet systems are complete. Support classifications are exhaustive only when the evaluation budget covers every required packet basis probe. Taylor certificates compare one finite truncation with the observed map value at the supplied increment. Address counts and support work grow combinatorially with order and packet count. The provenance registry records `PSC-III-C09` as Finite packet Taylor reconstruction. Its recorded status is `finite certificate`. Recorded limitation: no automatic bound on the omitted Taylor remainder.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_nonlinear_jets_composition.py` and `tests/test_nonlinear_execution.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 38.

Rooted-tree expansions and majorants (6 cards).

C.1E · `packet_tree_reconstruction`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.packet_tree_reconstruction`. Reflected implementation `komorebi_math.nonlinear.trees.packet_tree_reconstruction`. The same runtime object is also available through `komorebi_math.nonlinear.packet_tree_reconstruction`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1E-ROOTED-TREES`.

Manifest signature (`s`).

```
(problem: 'PolynomialFixedPointProblem', tree: 'OrderedRootedTree', packets: 'PacketSystem', *, zero_support:
   ↪ 'Mapping[int, PacketJetSupport] | None' = None, tolerance: 'float | None' = None) -> 'PacketTreeResult'
```

Definition. Reconstruct one tree by summing packet labels at every vertex.

Contract (`D, C, H, R, X`). Ordered-tree enumeration requires internal arities of at least two, nonnegative height or positive leaf targets, and an explicit tree budget. Polynomial fixed-point problems require a finite seed, compatible multilinear operations, and declared exactness and tolerance. Majorant routines require finite nonnegative norm bounds and a positive candidate radius.

Guarantees (`E, P, B, K`). Tree counts and finite enumerations are exact within the requested boundary. A tree expansion equals the matching finite Picard iterate under the implemented polynomial operations. A convergence certificate is issued only when the supplied scalar majorant maps the declared ball into itself and has contraction factor below one. Packet reconstruction prunes only addresses covered by supplied zero-support certificates. Enumeration can grow exponentially or faster with height and arity. The provenance registry records `PSC-III-T16` as Packet-labeled rooted-tree reconstruction. Its recorded status is `proved`.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this root facade. The focused test module is `tests/test_nonlinear_trees.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 39.

Execution, snapshots, streaming, and profiling (35 cards).

C.1E · profile_operation

Identity (q, a, t, k). Public research-manifest route `komorebi_math.profile_operation`. Reflected implementation `komorebi_math.nonlinear.profiling.profile_operation`. The same runtime object is also available through `komorebi_math.nonlinear.profile_operation`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1E-EXECUTION`.

Manifest signature (s).

```
(name: 'str', operation: 'Callable[[], T]', *, repetitions: 'int' = 5, warmup: 'int' = 1, scope: 'str' = 'Python
   ↪  call including allocations and synchronization performed by the callable') -> 'RuntimeStageProfile'
```

Definition. Observe one callable without interpreting timings as asymptotic evidence.

Contract (D, C, H, R, X). Complexity estimation requires a finite family and a compatible optional support record or execution plan. Runtime profiles require positive repetition counts, nonnegative warmup counts, and a precisely stated measurement scope. Packet-jet profiling may include planning, execution, snapshot capture, and optional stream writing under explicit budgets.

Guarantees (E, P, B, K). Deterministic estimate fields are exact finite counts derived from the represented dimensions, packets, addresses, and plan. Timing and traced Python allocation samples are environment-dependent observations. They are not asymptotic evidence and do not include unreported device allocation or external process cost. Reports retain the interpreter, platform, scope, repetitions, and certificate needed to interpret the measurements.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_block11_golden_profiling_beta.py` and `tests/test_nonlinear_execution.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 43.

C.1E · profile_packet_jet_runtime

Identity (q, a, t, k). Public research-manifest route `komorebi_math.profile_packet_jet_runtime`. Reflected implementation `komorebi_math.nonlinear.profiling.profile_packet_jet_runtime`. The same runtime object is also available through `komorebi_math.nonlinear.profile_packet_jet_runtime`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1E-EXECUTION`.

Manifest signature (s).

```
(family: 'PacketJetFamily', directions: 'Sequence[Any]', *, support: 'PacketJetSupport | None' = None, jet:
   ↪  'PacketJet | None' = None, repetitions: 'int' = 5, warmup: 'int' = 1, max_addresses: 'int' = 1000000,
   ↪  max_snapshot_evaluations: 'int' = 100000, include_stream_write: 'bool' = False, stream_chunk_size: 'int' =
   ↪  1024) -> 'PacketJetProfileReport'
```

Definition. Profile selected packet-jet stages under one explicit local contract.

Contract (D, C, H, R, X). Complexity estimation requires a finite family and a compatible optional support record or execution plan. Runtime profiles require positive repetition counts, nonnegative warmup counts, and a precisely stated measurement scope. Packet-jet profiling may include planning, execution, snapshot capture, and optional stream writing under explicit budgets.

Guarantees (E, P, B, K). Deterministic estimate fields are exact finite counts derived from the represented dimensions, packets, addresses, and plan. Timing and traced Python allocation samples are environment-dependent observations. They are not asymptotic evidence and do not include unreported device allocation or external process cost. Reports retain the interpreter, platform, scope, repetitions, and certificate needed to interpret the measurements. The provenance registry records `PSC-III-A61` as Finite packet-jet execution profile. Its recorded status is `heuristic`. Recorded limitations state that timings are environment dependent and that tracemalloc observes Python-managed allocations rather than total device or process memory.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_block11_golden_profiling_beta.py` and `tests/test_nonlinear_execution.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 43.

Rooted-tree expansions and majorants (6 cards).

C.1E · `quadratic_catalan_majorant`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.quadratic_catalan_majorant`. Reflected implementation `komorebi_math.nonlinear.trees.quadratic_catalan_majorant`. The same runtime object is also available through `komorebi_math.nonlinear.quadratic_catalan_majorant`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1E-ROOTED-TREES`.

Manifest signature (s).

```
(seed_norm: 'float', bilinear_norm: 'float', *, tolerance: 'float' = 1e-12) -> 'MajorantConvergence'
```

Definition. Construct the sharp small-root majorant for `u=s+B(u,u)`.

Contract (D, C, H, R, X). Ordered-tree enumeration requires internal arities of at least two, nonnegative height or positive leaf targets, and an explicit tree budget. Polynomial fixed-point problems require a finite seed, compatible multilinear operations, and declared exactness and tolerance. Majorant routines require finite nonnegative norm bounds and a positive candidate radius.

Guarantees (E, P, B, K). Tree counts and finite enumerations are exact within the requested boundary. A tree expansion equals the matching finite Picard iterate under the implemented polynomial operations. A convergence certificate is issued only when the supplied scalar majorant maps the declared ball into itself and has contraction factor below one. Packet reconstruction prunes only addresses covered by supplied zero-support certificates. Enumeration can grow exponentially or faster with height and arity.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test module is `tests/test_nonlinear_trees.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 39.

Execution, snapshots, streaming, and profiling (35 cards).

C.1E · `save_packet_jet_snapshot_stream`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.save_packet_jet_snapshot_stream`. Reflected implementation `komorebi_math.nonlinear.streaming.save_packet_jet_snapshot_stream`. The same runtime object is also available through `komorebi_math.nonlinear.save_packet_jet_snapshot_stream`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1E-EXECUTION`.

Manifest signature (s).

```
(snapshot: 'PacketJetSnapshot', path: 'str | os.PathLike[str]', *, chunk_size: 'int' = 1024, coefficient_budget:
    ↪ 'int | None' = None, resume: 'bool' = False, overwrite: 'bool' = False) -> 'PacketJetStreamWriteResult'
```

Definition. Write an existing snapshot as independently verified coefficient chunks.

Contract (D, C, H, R, X). Chunked streams use fixed schema identifiers and version 1 envelopes. Writers require a destination, a positive chunk size, and explicit overwrite or resume policy. Direct capture writes to a partial directory and promotes it only after completion. Readers reject duplicate JSON keys, unsupported schemas, inconsistent ranges, digest mismatches, and incomplete streams unless incomplete access is explicitly allowed.

Guarantees (E, P, B, K). Each committed chunk has a payload digest and a manifest range. Resume continues only from a verified committed prefix. Selection and evaluation can operate with memory bounded by the configured chunk size. Loading a conventional snapshot requires a complete stream and an explicit coefficient ceiling. Digests provide integrity against accidental corruption, not authentication against an adversary able to replace the entire stream and its hashes.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_nonlinear_snapshots_persistence.py` and `tests/test_block11_streaming_migrations.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 43.

C.1E · `snapshot_packet_jet`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.snapshot_packet_jet`. Reflected implementation `komorebi_math.nonlinear.snapshots.snapshot_packet_jet`. The same runtime object is also available through `komorebi_math.nonlinear.snapshot_packet_jet`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1E-EXECUTION`.

Manifest signature (s).

```
(jet: 'PacketJet', *, tolerance: 'float | None' = None, max_evaluations: 'int' = 100000) -> 'PacketJetSnapshot'
```

Definition. Capture one packet jet by exhaustive standard-basis evaluation.

Contract (D, C, H, R, X). A snapshot captures one fixed packet jet by evaluating the ambient standard-basis tuples through an explicit finite budget. The source and target dimensions, base point, address, provider name, exactness, and sparse coefficients are stored in the record. Snapshot evaluation requires direction vectors of the recorded source dimension.

Guarantees (E, P, B, K). The snapshot is provider-independent after capture and reconstructs the recorded multilinear block from its stored coefficients. Its comparison certificate checks matched evaluations against the source jet on supplied probes. Exact coefficients preserve exact arithmetic. Numerical coefficients use the stored tolerance and certificate. Dense capture cost grows as the source dimension raised to the jet order.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_nonlinear_snapshots_persistence.py` and `tests/test_block10_execution_persistence.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapters 38 and 43.

Defects, lifting, grading, and transport (27 cards).

C.1E · `solve_mod_prime`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.solve_mod_prime`. Reflected implementation `komorebi_math.nonlinear.lifting.solve_mod_prime`. The same runtime object is also available through `komorebi_math.nonlinear.solve_mod_prime`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1E-NONLINEAR-CALCULUS`.

Manifest signature (s).

```
(matrix: 'Any', rhs: 'Any', prime: 'int') -> 'ModularLinearSolution'
```

Definition. Solve a finite linear system modulo a prime by exact row reduction.

Contract (D, C, H, R, X). Linear and polynomial lifting require a prime, a nonnegative depth, compatible finite equations, and a current residue class. One step searches the declared corrections modulo the prime. Tree construction requires an explicit terminal depth and a finite node budget.

Guarantees (E, P, B, K). For the lifting operations in this family, every returned child satisfies the represented congruence at the next prime power. Linear solves and branch verification are exact in modular arithmetic. Polynomial lifting uses the declared Jacobian and finite Taylor congruence. Branch counts may grow exponentially with depth, so the tree builder enforces the supplied node limit. The provenance registry records `PSC-III-LIN-FP` as Linear correction space over a prime field. Its recorded status is `classical input`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test module is `tests/test_lifting.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 40.

Execution, snapshots, streaming, and profiling (35 cards).

C.1E · `validate_cupy_snapshot`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.validate_cupy_snapshot`. Reflected implementation `komorebi_math.nonlinear.gpu.validate_cupy_snapshot`. The same runtime object is also available through `komorebi_math.nonlinear.validate_cupy_snapshot`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1E-EXECUTION`.

Manifest signature (s).

```
(snapshot: 'PacketJetSnapshot', probes: 'Sequence[Sequence[Any]]', *, tolerance: 'float' = 1e-09) ->
   ↪ 'CuPyValidationResult'
```

Definition. Compare matched CPU and CuPy evaluations on explicit probes.

Contract (D, C, H, R, X). GPU status inspection never requires hardware and returns the observed CuPy import and CUDA device state. Snapshot execution requires a complete finite snapshot, a working CuPy CUDA runtime, and compatible probe shapes. Validation compares the same persisted coefficients on CPU and device under a declared tolerance.

Guarantees (E, P, B, K). A successful validation record reports the probe count, maximum residual, runtime state, and certificate. The device executor evaluates only the captured finite snapshot and does not accept arbitrary Python callables. No speed claim follows from API presence. In a runtime without CuPy or a CUDA device, status reporting remains available while execution and validation are not hardware-qualified.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_block16_gpu_qualification.py` and `tests/test_nonlinear_execution.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 43. The recorded reference environment had no CuPy CUDA device, so GPU evidence is limited to status reporting, explicit unavailability, and CPU-side contract checks.

Defects, lifting, grading, and transport (27 cards).

C.1E · `verify_finite_filtered_branch`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.verify_finite_filtered_branch`. Reflected implementation `komorebi_math.nonlinear.filtered.verify_finite_filtered_branch`. The same runtime object is also available through `komorebi_math.nonlinear.verify_finite_filtered_branch`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1E-NONLINEAR-CALCULUS`.

Manifest signature (s).

```
(source: 'FilteredVectorSpace', target: 'FilteredVectorSpace', map_function: 'Callable[[tuple[int, ...]],
    ↪ Sequence[int]]', approximations: 'Sequence[Sequence[int]]', *, start_depth: 'int' = 0) ->
    ↪ 'FiniteFilteredBranchResult'
```

Definition. Verify a compatible branch through a finite terminal filtration.

Contract (D, C, H, R, X). Filtered contracts require finite dimensional vector spaces over an explicit prime field and decreasing filtration bases with compatible ambient dimensions. Filtered correction is solved one finite depth at a time. A filtered cochain complex requires explicit differentials that square to zero and preserve the declared filtration. Projective limit completion also requires external completeness hypotheses supplied by the caller.

Guarantees (E, P, B, K). Associated graded coordinates, correction cosets, finite branches, and spectral sequence pages are exact over the represented finite field. The terminal finite filtration permits exact page dimension and stabilization checks. A projective limit record only stores the declared infinite hypotheses and finite compatibility evidence. It does not prove completeness or convergence beyond those assumptions.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test module is `tests/test_nonlinear_filtered.py`. The independent verifier for nonlinear facades checks root-to-domain object identity, reflected signature, implementation ownership, alias behavior, and representative CPU-safe execution for this facade family. The source validator for nonlinear facades checks exact card order, record counts, unique anchors, and preservation of the reference cards and formulas. See Chapter 42.

Related reference: packet jets, execution, streaming, and rooted trees. The 69 domain-namespace cards are collected in Section C.5B. The following overview explains their scope.

The nonlinear facade contracts include the following public operations. They cover packet-jet addresses, families, support and Taylor reconstruction, sparse symmetric tensors, projection-cached execution, finite snapshots, resumable streams, optional GPU boundaries, observational profiling, and ordered rooted-tree expansions.

Contract family	Cards
Packet jets, supports, and Taylor reconstruction	8
Sparse symmetric tensors	3
Projection-cached and optional CuPy execution	12
Portable packet-jet snapshots	3
Resumable chunked streams and schemas	14
Persisted-snapshot GPU validation	6
Complexity estimates and observational profiles	7
Ordered rooted trees and fixed-point expansions	14
Namespace availability records	2
Total	**69**

The boundary remains finite and explicit. Packet support is certified only over the tested packet bases. Snapshots and streams preserve one captured finite multilinear block. Timing records describe one local environment. GPU contracts require actual CuPy and CUDA availability before device execution can be qualified. Rooted-tree convergence remains conditional on the supplied scalar majorant and contraction inequalities.

C.1F Root generating, signal, and export facades: 62 entries

This root section documents top-level routes for three domain namespaces. The generating cards expose formal-series, basis, automaton, Mahler-mask, Bell-reduction, and prime-power arithmetic contracts. The signal cards expose packet trees, filter banks, polyphase analysis, singular packets, and finite jump recovery. The export cards expose the document model, renderers, portable persistence, serialization, reports, and file-writing interfaces.

Contract family	Cards
Generating functions and arithmetic	21

Contract family	Cards
Signal systems and recovery	7
Document model and rendering	10
Portable persistence and serialization	13
Export writers and reports	11
Total	**62**

These root routes do not create second implementations. Every card requires the root object to be identical to its domain route and to resolve to the implementation owner committed in the research manifest. File-writing and LaTeX contracts retain their explicit filesystem, overwrite, engine, and resource boundaries. These reference records do not claim a new external-toolchain or platform qualification.

Generating functions and arithmetic (21 cards).

C.1F · `BellParityAutomaton`

Identity (`q`, `a`, `t`, `k`). Public research-manifest route `komorebi_math.BellParityAutomaton`. Reflected implementation `komorebi_math.generating.bell.BellParityAutomaton`. The same runtime object is also available through `komorebi_math.generating.BellParityAutomaton`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1F-GENERATING`.

Manifest signature (`s`).

```
(outputs: 'tuple[int, int, int]' = (1, 1, 0)) -> None
```

Definition. The three-state binary automaton for generalized Bell parity.

Contract (`D`, `C`, `H`, `R`, `X`). Bell reductions require a prime, a nonnegative index, and valid factorial-power or weight data. Automata consume base-prime digits in the order stated by each method. Period bounds and pointed-subindex identities are finite arithmetic contracts. Weighted reductions require enough local weights for the requested computation. Resource budgets cap explicit polynomial and state construction. The principal public operations are `transition`, `evaluate`.

Guarantees (`E`, `P`, `B`, `K`). The returned reductions, parity values, residue automata, and pointed polynomial certificates are exact modulo the declared prime. A certificate records the finite comparison it performed. The period bound is an upper bound, not a minimal-period claim. Direct polynomial construction can grow with the digit sum or index, while automaton evaluation is linear in the number of input digits.

Evidence and links (`V`, `L`). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_generating_foundation.py` and `tests/test_generating_cartier_padic.py`. The independent verifier for generating, signal, and export facades checks root-to-domain object identity, reflected signatures, implementation ownership, and representative CPU-safe behavior for the documented facade families. The source validator for generating, signal, and export facades checks exact card order, record counts, unique anchors, and preservation of the reference cards, technical blocks, and formulas. See Chapters 22, 24, and 26.

C.1F · `BellResidueAutomaton`

Identity (`q`, `a`, `t`, `k`). Public research-manifest route `komorebi_math.BellResidueAutomaton`. Reflected implementation `komorebi_math.generating.bell.BellResidueAutomaton`. The same runtime object is also available through `komorebi_math.generating.BellResidueAutomaton`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1F-GENERATING`.

Manifest signature (s).

```
(prime: 'int', factorial_power: 'int', period_bound: 'int', outputs: 'tuple[int, ...]') -> None
```

Definition. A finite digit-sum automaton for factorial-power Bell residues.

Contract (D, C, H, R, X). Bell reductions require a prime, a nonnegative index, and valid factorial-power or weight data. Automata consume base-prime digits in the order stated by each method. Period bounds and pointed-subindex identities are finite arithmetic contracts. Weighted reductions require enough local weights for the requested computation. Resource budgets cap explicit polynomial and state construction. The principal public operations are `compile`, `transition`, `evaluate`, `certificate`.

Guarantees (E, P, B, K). The returned reductions, parity values, residue automata, and pointed polynomial certificates are exact modulo the declared prime. A certificate records the finite comparison it performed. The period bound is an upper bound, not a minimal-period claim. Direct polynomial construction can grow with the digit sum or index, while automaton evaluation is linear in the number of input digits.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_generating_foundation.py` and `tests/test_generating_cartier_padic.py`. The independent verifier for generating, signal, and export facades checks root-to-domain object identity, reflected signatures, implementation ownership, and representative CPU-safe behavior for the documented facade families. The source validator for generating, signal, and export facades checks exact card order, record counts, unique anchors, and preservation of the reference cards, technical blocks, and formulas. See Chapters 22, 24, and 26.

C.1F · `CustomFunctionSpec`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.CustomFunctionSpec`. Reflected implementation `komorebi_math.generating.functions.CustomFunctionSpec`. The same runtime object is also available through `komorebi_math.generating.CustomFunctionSpec`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1F-GENERATING`.

Manifest signature (s).

```
(name: 'str', basis: 'WeightedMonomialBasis', builder: 'CoefficientBuilder', source_kind: 'str' = 'custom',
    ↪ metadata: 'Mapping[str, object]' = <factory>) -> None
```

Definition. A named exact coefficient source with an explicit basis and metadata.

Contract (D, C, H, R, X). A custom function has a nonempty name, an explicit normalized basis, a finite coefficient builder, a source kind, and portable metadata where required. Builders must return exactly `degree+1` coefficients. Recurrence seeds must match the recurrence order. Rational denominators need a nonzero constant. Algebraic branches require `P(0,y0)=0` and a nonzero partial derivative with respect to `y`. Registries reject duplicate names. The principal public operations are `series`.

Guarantees (E, P, B, K). Each call constructs an exact finite `FormalSeries`. Rational, recurrence, hypergeometric, algebraic, Euler product, and Bell builders certify only the generated finite prefix. Runtime is polynomial in the requested degree for the implemented recurrences, but exact coefficient growth can dominate memory and arithmetic cost.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The independent verifier for generating, signal, and export facades checks root-to-domain object identity, reflected signatures, implementation ownership, and representative CPU-safe behavior for the documented facade families. The source validator for generating, signal, and export facades checks exact card order, record counts, unique anchors, and preservation of the reference cards, technical blocks, and formulas. See Chapters 21 through 24.

C.1F · FiniteMaskCarryPlan

Identity (q, a, t, k). Public research-manifest route `komorebi_math.FiniteMaskCarryPlan`. Reflected implementation `komorebi_math.generating.masks.FiniteMaskCarryPlan`. The same runtime object is also available through `komorebi_math.generating.FiniteMaskCarryPlan`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1F-GENERATING`.

Manifest signature (s).

```
(radix: 'int', mask: 'tuple[Fraction, ...]', carry_bound: 'int', matrices: 'tuple[Matrix, ...]') -> None
```

Definition. Exact carry-matrix representation of a finite local mask.

Contract (D, C, H, R, X). Inputs describe a finite local mask, a radix of at least two, and an explicit truncation, modulus, exponent, or resource budget as required by the operation. Unit-series routines require a valid constant coefficient. Prime-power support routines require a prime and positive exponent. Carry plans validate every state and local index before allocation. Bounded solvers return a negative result when the declared search space contains no solution. The principal public operations are `dimension`, `evaluate`, `sequence`, `certificate`.

Guarantees (E, P, B, K). Finite coefficient identities, carry matrices, support layers, and polynomial certificates use exact integer or rational arithmetic. Analytic approximations separate the computed product from a stated tail bound. A finite functional equation or algebraic certificate applies only through its recorded degree. Direct mask products are at least quadratic in the truncation boundary, while carry-state plans depend on the explicit state dimension and budget.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_generating_block13.py` and `tests/test_generating_cartier_padic.py`. The independent verifier for generating, signal, and export facades checks root-to-domain object identity, reflected signatures, implementation ownership, and representative CPU-safe behavior for the documented facade families. The source validator for generating, signal, and export facades checks exact card order, record counts, unique anchors, and preservation of the reference cards, technical blocks, and formulas. See Chapters 24 through 27 and 64.

C.1F · FiniteNormalizedSeries

Identity (q, a, t, k). Public research-manifest route `komorebi_math.FiniteNormalizedSeries`. Reflected implementation `komorebi_math.generating.finite_series.FiniteNormalizedSeries`. The same runtime object is also available through `komorebi_math.generating.FiniteNormalizedSeries`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1F-GENERATING`.

Manifest signature (s).

```
(basis: 'WeightedMonomialBasis', domain: 'CoefficientDomain[Any]', coefficients: 'Iterable[Any]', *, budget:
    ↪ 'GeneratingBudget' = GeneratingBudget(maximum_degree=100000, maximum_coefficient_bits=4000000,
    ↪ maximum_matrix_dimension=4096, maximum_reachable_states=100000, maximum_local_combinations=1000000,
    ↪ maximum_modulus=2000000, maximum_serialized_bytes=67108864, maximum_recursion_depth=1000)) -> 'None'
```

Definition. A finite exact series `sum a_n z^n/gamma_n` over one domain.

Contract (D, C, H, R, X). The basis, coefficient domain, and budget must satisfy their public protocols. Coefficients are coerced into the declared domain. Binary operations require matching domains and compatible bases. Reciprocal requires a unit constant coefficient. Composition, exponential, and logarithm impose the same constant term conditions as ordinary formal calculus. Conversion returns an explicit finite transfer certificate. The principal public operations are `add`, `subtract`, `multiply`, `reciprocal`, `compose`, `exp`, `log`, `project`, `convert`, `certificate`.

Guarantees (E, P, B, K). Results are exact in the selected coefficient domain and bounded by the supplied `Generating Budget`. A valid transfer certificate checks reconstruction coefficient by coefficient through the stored degree. Complexity follows finite polynomial arithmetic, with quadratic convolution and potentially cubic composition at the truncation boundary.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The independent verifier for generating, signal, and export facades checks root-to-domain object identity, reflected signatures, implementation ownership, and representative CPU-safe behavior for the documented facade families.

The source validator for generating, signal, and export facades checks exact card order, record counts, unique anchors, and preservation of the reference cards, technical blocks, and formulas. See Chapters 21, 22, and 61.

C.1F · FormalSeries

Identity (q, a, t, k). Public research-manifest route `komorebi_math.FormalSeries`. Reflected implementation `komorebi_math.generating.series.FormalSeries`. The same runtime object is also available through `komorebi_math.generating.FormalSeries`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1F-GENERATING`.

Manifest signature (s).

```
(basis: 'WeightedMonomialBasis', coefficients: 'Iterable[int | Fraction]') -> 'None'
```

Definition. A finite exact series `sum a_n z^n/gamma_n`.

Contract (D, C, H, R, X). The coefficient tuple is finite and nonempty. Binary operations require compatible basis semantics or an explicit conversion. Reciprocal requires a nonzero constant term. Composition requires an inner zero constant term. Formal exponential requires zero constant term, while formal logarithm requires constant term one. Projectors and packets validate modulus and residue addresses. The principal public operations are `add`, `subtract`, `multiply`, `reciprocal`, `compose`, `exp`, `log`, `project`, `packet`, `convert`, `evaluate`.

Guarantees (E, P, B, K). Every algebraic result is exact over rational coefficients at the requested finite truncation. Multiplication, composition, reciprocal, exponential, and logarithm are finite formal operations and do not assert analytic convergence. Naive multiplication is quadratic in truncation degree, while composition and repeated powers can require cubic work.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The independent verifier for generating, signal, and export facades checks root-to-domain object identity, reflected signatures, implementation ownership, and representative CPU-safe behavior for the documented facade families. The source validator for generating, signal, and export facades checks exact card order, record counts, unique anchors, and preservation of the reference cards, technical blocks, and formulas. See Chapters 21, 22, and 24.

C.1F · FrozenGeneratingFunction

Identity (q, a, t, k). Public research-manifest route `komorebi_math.FrozenGeneratingFunction`. Reflected implementation `komorebi_math.generating.portable.FrozenGeneratingFunction`. The same runtime object is also available through `komorebi_math.generating.FrozenGeneratingFunction`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1F-GENERATING`.

Manifest signature (s).

```
(name: 'str', basis_name: 'str', normalization: 'tuple[Fraction, ...]', coefficients: 'tuple[Fraction, ...]',
   ↪ source_kind: 'str', metadata: 'Mapping[str, Any]' = <factory>) -> None
```

Definition. Provider-independent finite snapshot of a custom generating function.

Contract (D, C, H, R, X). A frozen function requires nonempty names, a nonempty coefficient prefix, normalization of equal length beginning with one, no zero normalization values, and portable metadata. Reconstruction cannot exceed the captured degree. Freezing accepts only `CustomFunctionSpec`. Codec registration installs closed data only decoders and does not permit arbitrary Python object construction. The principal public operations are `degree`, `series`.

Guarantees (E, P, B, K). The frozen record preserves exact rational coefficients and basis normalization independently of the original callback. Roundtrip integrity is delegated to the project portable envelope and its digest checks. Serialization cost is linear in the captured prefix, subject to the global payload budget used by the persistence layer.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The independent verifier for generating, signal, and export facades checks root-to-domain object identity, reflected signatures, implementation ownership, and representative CPU-safe behavior for the documented facade families.

The source validator for generating, signal, and export facades checks exact card order, record counts, unique anchors, and preservation of the reference cards, technical blocks, and formulas. See Chapters 21, 61, and 72.

C.1F · `FunctionRegistry`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.FunctionRegistry`. Reflected implementation `komorebi_math.generating.functions.FunctionRegistry`. The same runtime object is also available through `komorebi_math.generating.FunctionRegistry`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1F-GENERATING`.

Manifest signature (`s`).

```
() -> 'None'
```

Definition. An explicit custom-function registry. Names cannot silently overwrite existing entries.

Contract (`D, C, H, R, X`). A custom function has a nonempty name, an explicit normalized basis, a finite coefficient builder, a source kind, and portable metadata where required. Builders must return exactly `degree+1` coefficients. Recurrence seeds must match the recurrence order. Rational denominators need a nonzero constant. Algebraic branches require `P(0,y0)=0` and a nonzero partial derivative with respect to `y`. Registries reject duplicate names. The principal public operations are `register`, `get`, `names`.

Guarantees (`E, P, B, K`). Each call constructs an exact finite `FormalSeries`. Rational, recurrence, hypergeometric, algebraic, Euler product, and Bell builders certify only the generated finite prefix. Runtime is polynomial in the requested degree for the implemented recurrences, but exact coefficient growth can dominate memory and arithmetic cost.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this root facade. The independent verifier for generating, signal, and export facades checks root-to-domain object identity, reflected signatures, implementation ownership, and representative CPU-safe behavior for the documented facade families. The source validator for generating, signal, and export facades checks exact card order, record counts, unique anchors, and preservation of the reference cards, technical blocks, and formulas. See Chapters 21 through 24.

C.1F · `GeneratingBudget`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.GeneratingBudget`. Reflected implementation `komorebi_math.generating.budget.GeneratingBudget`. The same runtime object is also available through `komorebi_math.generating.GeneratingBudget`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1F-GENERATING`.

Manifest signature (`s`).

```
(maximum_degree: 'int' = 100000, maximum_coefficient_bits: 'int' = 4000000, maximum_matrix_dimension: 'int' = 4096,
    ↪ maximum_reachable_states: 'int' = 100000, maximum_local_combinations: 'int' = 1000000, maximum_modulus:
    ↪ 'int' = 2000000, maximum_serialized_bytes: 'int' = 67108864, maximum_recursion_depth: 'int' = 1000) -> None
```

Definition. Upper bounds checked before potentially expensive operations.

Contract (`D, C, H, R, X`). Every stored limit must be a positive integer. Degree, term, matrix, state, modulus, serialized size, and coefficient bit checks reject malformed requests before the corresponding expansion or allocation begins. The default object is immutable. Exceeding a declared limit raises `ResourceLimitError`, while malformed values raise `ValidationError`. The principal public operations are `require_degree`, `require_terms`, `require_matrix_dimension`, `require_states`, `require_local_combinations`, `require_modulus`, `require_serialized_bytes`, `require_coefficient_bits`.

Guarantees (`E, P, B, K`). The budget performs deterministic scalar validation and does not estimate hidden backend memory. Its checks are constant time. A caller who supplies a larger budget accepts the resulting exact arithmetic, state enumeration, and serialization costs.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this root facade. The independent verifier for generating, signal, and export facades checks root-to-domain object identity, reflected signatures, implementation ownership, and representative CPU-safe behavior for the documented facade families. The source validator for generating, signal, and export facades checks exact card order, record counts, unique anchors, and preservation of the reference cards, technical blocks, and formulas. See Chapters 21, 23, and 64.

C.1F · NormalizedBasis

Identity (q, a, t, k). Public research-manifest route `komorebi_math.NormalizedBasis`. Reflected implementation `komorebi_math.generating.basis.WeightedMonomialBasis`. The same runtime object is also available through `komorebi_math.generating.NormalizedBasis`. The public export name maps to implementation object `WeightedMonomialBasis`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1F-GENERATING`.

Manifest signature (s).

```
(name: 'str', gamma: 'GammaRule', *, descriptor: 'BasisDescriptor | None' = None) -> 'None'
```

Definition. A basis `e_n(z)=z^n/gamma_n` with `gamma_0=1`.

Contract (D, C, H, R, X). A normalized monomial basis is defined by exact nonzero values `gamma_n` with `gamma_0=1`. Names must be nonempty. Indices and finite audit degrees must be nonnegative. Packet addresses must have a positive modulus and a valid residue. Declarative descriptors use a closed registry. Callback bases remain usable but are not portable unless replaced by a finite table descriptor. The principal public operations are `gamma`, `lowering_weight`, `structure_constant`, `packet_transfer_weight`, `finite_certificate`, `require_descriptor`.

Guarantees (E, P, B, K). Normalization, lowering weights, structure constants, packet transfer weights, and finite certificates use exact `Fraction` arithmetic. Finite audits establish only the requested degree range. Computing all structure constants through degree `d` requires a quadratic number of exact ratios, and numerator or denominator sizes can grow quickly.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The independent verifier for generating, signal, and export facades checks root-to-domain object identity, reflected signatures, implementation ownership, and representative CPU-safe behavior for the documented facade families. The source validator for generating, signal, and export facades checks exact card order, record counts, unique anchors, and preservation of the reference cards, technical blocks, and formulas. See Chapters 21, 22, and 24.

C.1F · PrimePowerKernelCompiler

Identity (q, a, t, k). Public research-manifest route `komorebi_math.PrimePowerKernelCompiler`. Reflected implementation `komorebi_math.generating.padic_units.PrimePowerKernelCompiler`. The same runtime object is also available through `komorebi_math.generating.PrimePowerKernelCompiler`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1F-GENERATING`.

Manifest signature (s).

```
(prime: 'int', exponent: 'int', factorial_power: 'int', unit_sequence: 'Callable[[int], int] | None' = None,
    ↪  max_modulus: 'int' = 2000000) -> None
```

Definition. Compile kernels for `gamma_n=c**n (n!)**s u_n` modulo `p**r`. The geometric factor cancels from every structure constant. The supplied `unit_sequence` must take values coprime to `p`.

Contract (D, C, H, R, X). Valuation functions require a prime and valid integer or binomial indices. Prime-power compilers require positive precision and a modulus within the supplied budget. Unit-factorial and binomial methods distinguish valuation from the invertible unit part. Kernel compilers require compatible digit-unit tables and factorial exponents. Borrow counts use nonnegative indices with the subindex not exceeding the index where required. The principal public operations are `modulus`, `kernel`.

Guarantees (E, P, B, K). Valuations, unit parts, residues, and survival decisions are exact for the declared prime power. PrimePowerBinomial separates the p-adic valuation from the unit residue, so zero modulo the prime power is represented without discarding valuation information. Precomputation cost depends on the prime-power modulus. Subsequent digit recursions scale with the number of base-prime digits.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test module is `tests/test_generating_cartier_padic.py`. The independent verifier for generating, signal, and export facades checks root-to-domain object identity, reflected signatures, implementation ownership, and representative CPU-safe behavior for the documented facade families. The source validator for generating, signal,

and export facades checks exact card order, record counts, unique anchors, and preservation of the reference cards, technical blocks, and formulas. See Chapters 24 through 27.

C.1F · `WeightedMonomialBasis`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.WeightedMonomialBasis`. Reflected implementation `komorebi_math.generating.basis.WeightedMonomialBasis`. The same runtime object is also available through `komorebi_math.generating.WeightedMonomialBasis`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1F-GENERATING`.

Manifest signature (`s`).

```
(name: 'str', gamma: 'GammaRule', *, descriptor: 'BasisDescriptor | None' = None) -> 'None'
```

Definition. A basis `e_n(z)=z^n/gamma_n` with `gamma_0=1`.

Contract (`D, C, H, R, X`). A normalized monomial basis is defined by exact nonzero values `gamma_n` with `gamma_0=1`. Names must be nonempty. Indices and finite audit degrees must be nonnegative. Packet addresses must have a positive modulus and a valid residue. Declarative descriptors use a closed registry. Callback bases remain usable but are not portable unless replaced by a finite table descriptor. The principal public operations are `gamma`, `lowering_weight`, `structure_constant`, `packet_transfer_weight`, `finite_certificate`, `require_descriptor`.

Guarantees (`E, P, B, K`). Normalization, lowering weights, structure constants, packet transfer weights, and finite certificates use exact `Fraction` arithmetic. Finite audits establish only the requested degree range. Computing all structure constants through degree `d` requires a quadratic number of exact ratios, and numerator or denominator sizes can grow quickly.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this root facade. The independent verifier for generating, signal, and export facades checks root-to-domain object identity, reflected signatures, implementation ownership, and representative CPU-safe behavior for the documented facade families. The source validator for generating, signal, and export facades checks exact card order, record counts, unique anchors, and preservation of the reference cards, technical blocks, and formulas. See Chapters 21, 22, and 24.

C.1F · `compile_finite_mask_carry_plan`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.compile_finite_mask_carry_plan`. Reflected implementation `komorebi_math.generating.masks.compile_finite_mask_carry_plan`. The same runtime object is also available through `komorebi_math.generating.compile_finite_mask_carry_plan`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1F-GENERATING`.

Manifest signature (`s`).

```
(mask: 'Iterable[Scalar]', radix: 'int', *, budget: 'GeneratingBudget' = GeneratingBudget(maximum_degree=100000,
    ↪ maximum_coefficient_bits=4000000, maximum_matrix_dimension=4096, maximum_reachable_states=100000,
    ↪ maximum_local_combinations=1000000, maximum_modulus=2000000, maximum_serialized_bytes=67108864,
    ↪ maximum_recursion_depth=1000)) -> 'FiniteMaskCarryPlan'
```

Definition. Compile `T_d(q,q')=r_(d+radix*q'-q)` for a finite mask.

Contract (`D, C, H, R, X`). Inputs describe a finite local mask, a radix of at least two, and an explicit truncation, modulus, exponent, or resource budget as required by the operation. Unit-series routines require a valid constant coefficient. Prime-power support routines require a prime and positive exponent. Carry plans validate every state and local index before allocation. Bounded solvers return a negative result when the declared search space contains no solution.

Guarantees (`E, P, B, K`). Finite coefficient identities, carry matrices, support layers, and polynomial certificates use exact integer or rational arithmetic. Analytic approximations separate the computed product from a stated tail bound. A finite functional equation or algebraic certificate applies only through its recorded degree. Direct mask products are at least quadratic in the truncation boundary, while carry-state plans depend on the explicit state dimension and budget.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_generating_block13.py` and `tests/test_generating_cartier_padic.py`. The independent verifier for generating, signal, and export facades checks root-to-domain object identity, reflected signatures, implementation ownership, and representative CPU-safe behavior for the documented facade families. The source validator for generating, signal, and export facades checks exact card order, record counts, unique anchors, and preservation of the reference cards, technical blocks, and formulas. See Chapters 24 through 27 and 64.

C.1F · `compile_rational_mask_automaton`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.compile_rational_mask_automaton`. Reflected implementation `komorebi_math.generating.cartier.compile_rational_mask_automaton`. The same runtime object is also available through `komorebi_math.generating.compile_rational_mask_automaton`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1F-GENERATING`.

Manifest signature (s).

```
(numerator: 'Sequence[int]', denominator: 'Sequence[int]', prime: 'int', *, minimize: 'bool' = True, max_states:
    ↪ 'int' = 100000, budget: 'GeneratingBudget' = GeneratingBudget(maximum_degree=100000,
    ↪ maximum_coefficient_bits=4000000, maximum_matrix_dimension=4096, maximum_reachable_states=100000,
    ↪ maximum_local_combinations=1000000, maximum_modulus=2000000, maximum_serialized_bytes=67108864,
    ↪ maximum_recursion_depth=1000)) -> 'CartierAutomaton'
```

Definition. Compile a rational Mahler mask into a coefficient automaton modulo `prime`. The degree-derived linear dimension and the reachable-state guard are checked before state exploration begins.

Contract (D, C, H, R, X). Polynomial operations require a modulus of at least two. Cartier operators and rational-mask compilers require a prime, a valid digit, compatible nonzero constant coefficients, and explicit state or degree budgets. Automata read least-significant-first prime digits. Minimization preserves output behavior on the compiled finite-state machine. Certificate routines reject mismatched transition systems or coefficient prefixes.

Guarantees (E, P, B, K). Polynomial arithmetic, Cartier transitions, automaton outputs, and linear representations are exact modulo the selected prime. State bounds are finite compilation bounds rather than asymptotic minimality claims. Rational-mask coefficient generation and automaton evaluation agree on the checked prefix. Complexity is polynomial in polynomial degree and compiled state count, with minimization depending on partition refinement over the full transition table.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test module is `tests/test_generating_cartier_padic.py`. The independent verifier for generating, signal, and export facades checks root-to-domain object identity, reflected signatures, implementation ownership, and representative CPU-safe behavior for the documented facade families. The source validator for generating, signal, and export facades checks exact card order, record counts, unique anchors, and preservation of the reference cards, technical blocks, and formulas. See Chapters 24 through 27.

C.1F · `factorial_power_basis`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.factorial_power_basis`. Reflected implementation `komorebi_math.generating.basis.factorial_power_basis`. The same runtime object is also available through `komorebi_math.generating.factorial_power_basis`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1F-GENERATING`.

Manifest signature (s).

```
(power: 'int', *, name: 'str | None' = None) -> 'WeightedMonomialBasis'
```

Definition. Return `z^n/(n!)^power` for a positive integer power.

Contract (D, C, H, R, X). A normalized monomial basis is defined by exact nonzero values `gamma_n` with `gamma_0=1`. Names must be nonempty. Indices and finite audit degrees must be nonnegative. Packet addresses must have a positive modulus and a valid residue. Declarative descriptors use a closed registry. Callback bases remain usable but are not portable unless replaced by a finite table descriptor.

Guarantees (E, P, B, K). Normalization, lowering weights, structure constants, packet transfer weights, and finite certificates use exact `Fraction` arithmetic. Finite audits establish only the requested degree range. Computing all structure constants through degree `d` requires a quadratic number of exact ratios, and numerator or denominator sizes can grow quickly.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The independent verifier for generating, signal, and export facades checks root-to-domain object identity, reflected signatures, implementation ownership, and representative CPU-safe behavior for the documented facade families. The source validator for generating, signal, and export facades checks exact card order, record counts, unique anchors, and preservation of the reference cards, technical blocks, and formulas. See Chapters 21, 22, and 24.

C.1F · `freeze_custom_function`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.freeze_custom_function`. Reflected implementation `komorebi_math.generating.portable.freeze_custom_function`. The same runtime object is also available through `komorebi_math.generating.freeze_custom_function`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1F-GENERATING`.

Manifest signature (s).

```
(function: 'CustomFunctionSpec', degree: 'int', *, include_metadata: 'bool' = True) -> 'FrozenGeneratingFunction'
```

Definition. Capture one custom function into a finite provider-independent record.

Contract (D, C, H, R, X). A frozen function requires nonempty names, a nonempty coefficient prefix, normalization of equal length beginning with one, no zero normalization values, and portable metadata. Reconstruction cannot exceed the captured degree. Freezing accepts only `CustomFunctionSpec`. Codec registration installs closed data only decoders and does not permit arbitrary Python object construction.

Guarantees (E, P, B, K). The frozen record preserves exact rational coefficients and basis normalization independently of the original callback. Roundtrip integrity is delegated to the project portable envelope and its digest checks. Serialization cost is linear in the captured prefix, subject to the global payload budget used by the persistence layer.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The independent verifier for generating, signal, and export facades checks root-to-domain object identity, reflected signatures, implementation ownership, and representative CPU-safe behavior for the documented facade families. The source validator for generating, signal, and export facades checks exact card order, record counts, unique anchors, and preservation of the reference cards, technical blocks, and formulas. See Chapters 21, 61, and 72.

C.1F · `generalized_bell_digit_reduction`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.generalized_bell_digit_reduction`. Reflected implementation `komorebi_math.generating.bell.generalized_bell_digit_reduction`. The same runtime object is also available through `komorebi_math.generating.generalized_bell_digit_reduction`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1F-GENERATING`.

Manifest signature (s).

```
(index: 'int', prime: 'int', factorial_power: 'int', *, budget: 'GeneratingBudget' =
    ↪ GeneratingBudget(maximum_degree=100000, maximum_coefficient_bits=4000000, maximum_matrix_dimension=4096,
    ↪ maximum_reachable_states=100000, maximum_local_combinations=1000000, maximum_modulus=2000000,
    ↪ maximum_serialized_bytes=67108864, maximum_recursion_depth=1000)) -> 'BellDigitReduction'
```

Definition. Compute and certify the unweighted Bell digit theorem at one index.

Contract (D, C, H, R, X). Bell reductions require a prime, a nonnegative index, and valid factorial-power or weight data. Automata consume base-prime digits in the order stated by each method. Period bounds and pointed-subindex identities are finite arithmetic contracts. Weighted reductions require enough local weights for the requested computation. Resource budgets cap explicit polynomial and state construction.

Guarantees (E, P, B, K). The returned reductions, parity values, residue automata, and pointed polynomial certificates are exact modulo the declared prime. A certificate records the finite comparison it performed. The period bound is an upper bound, not a minimal-period claim. Direct polynomial construction can grow with the digit sum or index, while automaton evaluation is linear in the number of input digits.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_generating_foundation.py` and `tests/test_generating_cartier_padic.py`. The independent verifier for generating, signal, and export facades checks root-to-domain object identity, reflected signatures, implementation ownership, and representative CPU-safe behavior for the documented facade families. The source validator for generating, signal, and export facades checks exact card order, record counts, unique anchors, and preservation of the reference cards, technical blocks, and formulas. See Chapters 22, 24, and 26.

C.1F · `mahler_mask_inverse`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.mahler_mask_inverse`. Reflected implementation `komorebi_math.generating.masks.mahler_mask_inverse`. The same runtime object is also available through `komorebi_math.generating.mahler_mask_inverse`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1F-GENERATING`.

Manifest signature (s).

```
(series: 'Iterable[Scalar]', radix: 'int', degree: 'int | None' = None, *, budget: 'GeneratingBudget' =
    ↪ GeneratingBudget(maximum_degree=100000, maximum_coefficient_bits=4000000, maximum_matrix_dimension=4096,
    ↪ maximum_reachable_states=100000, maximum_local_combinations=1000000, maximum_modulus=2000000,
    ↪ maximum_serialized_bytes=67108864, maximum_recursion_depth=1000)) -> 'tuple[Fraction, ...]'
```

Definition. Recover the unique local mask `R(z)=F(z)/F(z**radix)`.

Contract (D, C, H, R, X). Inputs describe a finite local mask, a radix of at least two, and an explicit truncation, modulus, exponent, or resource budget as required by the operation. Unit-series routines require a valid constant coefficient. Prime-power support routines require a prime and positive exponent. Carry plans validate every state and local index before allocation. Bounded solvers return a negative result when the declared search space contains no solution.

Guarantees (E, P, B, K). Finite coefficient identities, carry matrices, support layers, and polynomial certificates use exact integer or rational arithmetic. Analytic approximations separate the computed product from a stated tail bound. A finite functional equation or algebraic certificate applies only through its recorded degree. Direct mask products are at least quadratic in the truncation boundary, while carry-state plans depend on the explicit state dimension and budget.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_generating_block13.py` and `tests/test_generating_cartier_padic.py`. The independent verifier for generating, signal, and export facades checks root-to-domain object identity, reflected signatures, implementation ownership, and representative CPU-safe behavior for the documented facade families. The source validator for generating, signal, and export facades checks exact card order, record counts, unique anchors, and preservation of the reference cards, technical blocks, and formulas. See Chapters 24 through 27 and 64.

C.1F · `mahler_mask_product`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.mahler_mask_product`. Reflected implementation `komorebi_math.generating.masks.mahler_mask_product`. The same runtime object is also available through `komorebi_math.generating.mahler_mask_product`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1F-GENERATING`.

Manifest signature (`s`).

```
(mask: 'Iterable[Scalar]', radix: 'int', degree: 'int', *, budget: 'GeneratingBudget' =
    ↪ GeneratingBudget(maximum_degree=100000, maximum_coefficient_bits=4000000, maximum_matrix_dimension=4096,
    ↪ maximum_reachable_states=100000, maximum_local_combinations=1000000, maximum_modulus=2000000,
    ↪ maximum_serialized_bytes=67108864, maximum_recursion_depth=1000)) -> 'tuple[Fraction, ...]'
```

Definition. Return `prod_(j>=0) R(z**radix**j)` through `z**degree`. Only finitely many mask levels affect a finite truncation. Arithmetic uses exact `fractions.Fraction` values.

Contract (`D, C, H, R, X`). Inputs describe a finite local mask, a radix of at least two, and an explicit truncation, modulus, exponent, or resource budget as required by the operation. Unit-series routines require a valid constant coefficient. Prime-power support routines require a prime and positive exponent. Carry plans validate every state and local index before allocation. Bounded solvers return a negative result when the declared search space contains no solution.

Guarantees (`E, P, B, K`). Finite coefficient identities, carry matrices, support layers, and polynomial certificates use exact integer or rational arithmetic. Analytic approximations separate the computed product from a stated tail bound. A finite functional equation or algebraic certificate applies only through its recorded degree. Direct mask products are at least quadratic in the truncation boundary, while carry-state plans depend on the explicit state dimension and budget.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_generating_block13.py` and `tests/test_generating_cartier_padic.py`. The independent verifier for generating, signal, and export facades checks root-to-domain object identity, reflected signatures, implementation ownership, and representative CPU-safe behavior for the documented facade families. The source validator for generating, signal, and export facades checks exact card order, record counts, unique anchors, and preservation of the reference cards, technical blocks, and formulas. See Chapters 24 through 27 and 64.

C.1F · `ordinary_basis`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.ordinary_basis`. Reflected implementation `komorebi_math.generating.basis.ordinary_basis`. The same runtime object is also available through `komorebi_math.generating.ordinary_basis`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1F-GENERATING`.

Manifest signature (`s`).

```
(*, name: 'str' = 'ordinary') -> 'WeightedMonomialBasis'
```

Definition. Return the ordinary generating-function basis `z^n`.

Contract (`D, C, H, R, X`). A normalized monomial basis is defined by exact nonzero values `gamma_n` with `gamma_0=1`. Names must be nonempty. Indices and finite audit degrees must be nonnegative. Packet addresses must have a positive modulus and a valid residue. Declarative descriptors use a closed registry. Callback bases remain usable but are not portable unless replaced by a finite table descriptor.

Guarantees (`E, P, B, K`). Normalization, lowering weights, structure constants, packet transfer weights, and finite certificates use exact `Fraction` arithmetic. Finite audits establish only the requested degree range. Computing all structure constants through degree `d` requires a quadratic number of exact ratios, and numerator or denominator sizes can grow quickly.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this root facade. The independent verifier for generating, signal, and export facades checks root-to-domain object identity, reflected signatures, implementation ownership, and representative CPU-safe behavior for the documented facade families. The source validator for generating, signal, and export facades checks exact card order, record counts, unique anchors, and preservation of the reference cards, technical blocks, and formulas. See Chapters 21, 22, and 24.

C.1F · q_binomial_valuation_decomposition

Identity (q, a, t, k). Public research-manifest route `komorebi_math.q_binomial_valuation_decomposition`. Reflected implementation `komorebi_math.generating.q_kummer.q_binomial_valuation_decomposition`. The same runtime object is also available through `komorebi_math.generating.q_binomial_valuation_decomposition`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1F-GENERATING`.

Manifest signature (s).

```
(index: 'int', subindex: 'int', base: 'int', prime: 'int') -> 'QKummerDecomposition'
```

Definition. Return the exact mixed-radix valuation decomposition. If `prime` divides `base`, every Gaussian binomial is a unit modulo `prime` and the valuation is zero. For an odd prime coprime to `base`, the block length is `ord_prime(base)` and the usual order-block formula applies. For `prime == 2` and odd `base`, the block length is two and the lift constant is `v_2(base + 1)`.

Contract (D, C, H, R, X). The q-analogue routines require nonnegative indices, a valid subindex, an integer base, and a prime that does not violate the stated denominator or multiplicative-order conditions. Valuation decomposition requires a positive power and exponent where present. Support enumeration is finite and may be bounded by an explicit resource contract.

Guarantees (E, P, B, K). q-integers, q-factorials, Gaussian binomials, direct valuations, mixed-radix decompositions, and support sets are computed with exact integers. The decomposition records all factors used to reconstruct the valuation. Large Gaussian binomials can grow rapidly in bit length, so direct verification is bounded even when the structural decomposition remains compact.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test module is `tests/test_generating_cartier_padic.py`. The independent verifier for generating, signal, and export facades checks root-to-domain object identity, reflected signatures, implementation ownership, and representative CPU-safe behavior for the documented facade families. The source validator for generating, signal, and export facades checks exact card order, record counts, unique anchors, and preservation of the reference cards, technical blocks, and formulas. See Chapters 24 through 27.

Signal systems and recovery (7 cards).

C.1F · HaarPacketTree

Identity (q, a, t, k). Public research-manifest route `komorebi_math.HaarPacketTree`. Reflected implementation `komorebi_math.signals.wavelets.HaarPacketTree`. The same runtime object is also available through `komorebi_math.signals.HaarPacketTree`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1F-SIGNALS`.

Manifest signature (s).

```
(signal: 'np.ndarray', depth: 'int', normalized: 'bool', _nodes: 'dict[str, np.ndarray]') -> None
```

Definition. All Haar packet coefficients through one finite binary depth.

Contract (D, C, H, R, X). Haar packet trees require a finite nonempty one-dimensional signal, a nonnegative depth, and a length divisible by the corresponding power of two. Packet paths contain only binary branches. A pruning must cover the full tree without ancestor and descendant conflicts. Additive cost functions must return finite values. Lp exponents must be positive and finite where required. Principal public operations and properties are `analyze`, `decompose`, `nodes`, `coefficients`, `coefficient`, `paths_at_depth`, `full_leaves`, `validate_pruning`, `reconstruct`, `energy`, `reconstruction_certificate`, `certificate`. Additional public members are recorded in the machine-readable inventory.

Guarantees (E, P, B, K). Normalized Haar packet analysis preserves finite energy to numerical tolerance. Synthesis from a valid full tree or pruning reconstructs the supplied record under the stored tolerance. Best-basis selection is exact dynamic programming for the declared additive node cost. Tree construction and best-basis work scale with the number of stored coefficients across levels. The provenance registry records `PSC-V-D12` as Finite Haar packet tree. Its recorded status is `definition`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test module is `tests/test_signals_wavelets_singular.py`. The independent verifier for generating, signal, and export facades checks root-to-domain object identity, reflected signatures, implementation ownership, and representative CPU-safe behavior for the documented facade families. The source validator for generating, signal, and export facades checks exact card order, record counts, unique anchors, and preservation of the reference cards, technical blocks, and formulas. See Chapter 35.

C.1F · PeriodicFilterBank

Identity (q, a, t, k). Public research-manifest route `komorebi_math.PeriodicFilterBank`. Reflected implementation `komorebi_math.signals.filterbanks.PeriodicFilterBank`. The same runtime object is also available through `komorebi_math.signals.PeriodicFilterBank`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1F-SIGNALS`.

Manifest signature (s).

```
(analysis_filters: 'Iterable[Any]', synthesis_filters: 'Iterable[Any]', decimation: 'int') -> 'None'
```

Definition. A finite periodic analysis and synthesis bank with explicit matrices.

Contract (D, C, H, R, X). Periodic and finite filter banks require compatible nonempty one-dimensional analysis and synthesis filters or compatible explicit matrices. Decimation factors must be positive, channel counts must agree, and periodic signal lengths must satisfy the declared divisibility rules. Lifting schemes require even finite records. Canonical dual construction requires full column rank. Numerical tolerances and matrix conventions remain part of the public contract. Principal public operations and properties are `channels`, `analysis_matrix`, `synthesis_matrix`, `analyze`, `synthesize`, `reconstruction_operator`, `perfect_reconstruction_certificate`, `frame_bounds`, `canonical_synthesis_matrix`, `reconstruction_error_bound`.

Guarantees (E, P, B, K). Analysis, synthesis, frame bounds, and reconstruction reports are finite linear-algebra computations. Perfect-reconstruction certificates compare the implemented finite operator with the identity under the stored tolerance. Haar and five-three lifting constructions use the documented normalization and periodic boundary law. Dense matrix construction, singular values, and pseudoinverses can require quadratic storage and cubic factorization cost. The provenance registry records `PSC-V-D08` as Finite periodic filter bank. Its recorded status is `definition`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test module is `tests/test_signals_polyphase_filterbanks.py`. The independent verifier for generating, signal, and export facades checks root-to-domain object identity, reflected signatures, implementation ownership, and representative CPU-safe behavior for the documented facade families. The source validator for generating, signal, and export facades checks exact card order, record counts, unique anchors, and preservation of the reference cards, technical blocks, and formulas. See Chapter 35.

C.1F · PolyphaseSystem

Identity (q, a, t, k). Public research-manifest route `komorebi_math.PolyphaseSystem`. Reflected implementation `komorebi_math.signals.polyphase.PolyphaseSystem`. The same runtime object is also available through `komorebi_math.signals.PolyphaseSystem`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1F-SIGNALS`.

Manifest signature (s).

```
(order: 'int', axis: 'int' = -1) -> None
```

Definition. A reusable polyphase analysis and synthesis system along one axis.

Contract (D, C, H, R, X). Polyphase orders, strides, and radices must be positive integers. Axis selections must be valid for the supplied array, and scalar arrays are rejected where an axis decomposition is required. Residue indices and mixed-radix addresses must lie in range. Lattice coordinates require one channel for every Cartesian residue address. Refinement requires exact divisibility of the fine stride by the coarse stride. Principal public operations and properties are `analyze`, `synthesize`, `certificate`.

Guarantees (E, P, B, K). Analysis partitions the supplied finite array into explicit residue channels and synthesis restores their interleaving. Roundtrip certificates compare the restored array with the source under the stored tolerance. Mixed-radix addressing is least-significant digit first. Polyphase filtering exposes finite channel kernels with omitted coefficients treated as zero. Work and storage are linear in the represented array size plus the active kernel coefficients.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_signals_polyphase_filterbanks.py` and `tests/test_block15_polyphase_refinement.py`. The independent verifier for generating, signal, and export facades checks root-to-domain object identity, reflected signatures, implementation ownership, and representative CPU-safe behavior for the documented facade families. The source validator for generating, signal, and export facades checks exact card order, record counts, unique anchors, and preservation of the reference cards, technical blocks, and formulas. See Chapters 14 and 35.

C.1F · `SingularPacket1D`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.SingularPacket1D`. Reflected implementation `komorebi_math.signals.singular_packets.SingularPacket1D`. The same runtime object is also available through `komorebi_math.signals.SingularPacket1D`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1F-SIGNALS`.

Manifest signature (s).

```
(components: 'Iterable[TruncatedPowerTerm]') -> 'None'
```

Definition. A finite sum of truncated power terms.

Contract (D, C, H, R, X). Truncated-power terms require finite locations and amplitudes with nonnegative integer order. Packet construction requires compatible term data. Reconstruction helpers require finite arrays of matching shape, and approximation matrices must have compatible dimensions. Region and location error helpers require nonnegative measures, compatible labels, and a positive finite exponent or infinity. Principal public operations and properties are `from_jumps`, `evaluate`, `remove`, `restore`, `trace_jumps`.

Guarantees (E, P, B, K). Singular packets evaluate, remove, and restore explicit truncated-power components. Retained-packet reconstruction records the smooth approximation, packet estimate, full reconstruction, and the exact finite error identity represented by those arrays. Location bounds are exact at jump order zero and use the documented mean-value estimate at positive orders. Runtime follows the number of samples times the number of retained terms plus any supplied approximation-matrix cost. The provenance registry records `PSC-V-D16` as Finite one dimensional singular packet. Its recorded status is `definition`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test module is `tests/test_signals_wavelets_singular.py`. The independent verifier for generating, signal, and export facades checks root-to-domain object identity, reflected signatures, implementation ownership, and representative CPU-safe behavior for the documented facade families. The source validator for generating, signal, and export facades checks exact card order, record counts, unique anchors, and preservation of the reference cards, technical blocks, and formulas. See Chapter 36.

C.1F · `polyphase_analyze`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.polyphase_analyze`. Reflected implementation `komorebi_math.signals.polyphase.polyphase_analyze`. The same runtime object is also available through `komorebi_math.signals.polyphase_analyze`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1F-SIGNALS`.

Manifest signature (s).

```
(signal: 'Any', order: 'int', *, axis: 'int' = -1, copy: 'bool' = True) -> 'PolyphaseCoordinates'
```

Definition. Split a finite array into residue channels along `axis`.

Contract (D, C, H, R, X). Polyphase orders, strides, and radices must be positive integers. Axis selections must be valid for the supplied array, and scalar arrays are rejected where an axis decomposition is required. Residue indices and mixed-radix addresses must lie in range. Lattice coordinates require one channel for every Cartesian residue address. Refinement requires exact divisibility of the fine stride by the coarse stride.

Guarantees (E, P, B, K). Analysis partitions the supplied finite array into explicit residue channels and synthesis restores their interleaving. Roundtrip certificates compare the restored array with the source under the stored tolerance. Mixed-radix addressing is least-significant digit first. Polyphase filtering exposes finite channel kernels with omitted coefficients treated as zero. Work and storage are linear in the represented array size plus the active kernel coefficients. The provenance registry records `PSC-V-T02` as Exact finite polyphase decomposition. Its recorded status is `proved`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_signals_polyphase_filterbanks.py` and `tests/test_block15_polyphase_refinement.py`. The independent verifier for generating, signal, and export facades checks root-to-domain object identity, reflected signatures, implementation ownership, and representative CPU-safe behavior for the documented facade families. The source validator for generating, signal, and export facades checks exact card order, record counts, unique anchors, and preservation of the reference cards, technical blocks, and formulas. See Chapters 14 and 35.

C.1F · `recover_separated_jumps`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.recover_separated_jumps`. Reflected implementation `komorebi_math.signals.recovery.recover_separated_jumps`. The same runtime object is also available through `komorebi_math.signals.recover_separated_jumps`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1F-SIGNALS`.

Manifest signature (s).

```
(samples: 'Sequence[float]', *, spacing: 'float', lipschitz_bound: 'float', noise_bound: 'float', minimum_jump:
    ↪ 'float', threshold: 'float | None' = None, start: 'float' = 0.0, minimum_separation: 'float | None' = None)
    ↪ -> 'JumpRecoveryResult'
```

Definition. Recover jump cells by thresholding adjacent sample contrasts.

Contract (D, C, H, R, X). Scalar recovery requires a finite one-dimensional record with at least two samples. Spacing and minimum-jump parameters must be positive. Lipschitz and noise bounds must be nonnegative. The threshold gap must remain strict, and any minimum-separation value must be at least one sample spacing. Difference order is nonnegative and the finite step size is positive.

Guarantees (E, P, B, K). The recovery theorem is conditional on the stored smoothness, separation, and noise hypotheses. Finite differences and Haar formulas follow their explicit index and normalization conventions. Subgaussian bounds are finite confidence bounds for the declared sigma and delta. Candidate scanning is linear in sample count for fixed difference order. The provenance registry records `PSC-V-T36` as Stable recovery of separated one dimensional jumps. Its recorded status is `conditional theorem`. The provenance record restricts this claim to sampled signals in one dimension with separated value jumps.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test module is `tests/test_signals_recovery.py`. The independent verifier for generating, signal, and export facades checks root-to-domain object identity, reflected signatures, implementation ownership, and representative CPU-safe behavior for the documented facade families. The source validator for generating, signal, and export facades checks exact card order, record counts, unique anchors, and preservation of the reference cards, technical blocks, and formulas. See Chapter 36.

C.1F · retained_packet_reconstruct

Identity (q, a, t, k). Public research-manifest route `komorebi_math.retained_packet_reconstruct`. Reflected implementation `komorebi_math.signals.singular_packets.retained_packet_reconstruct`. The same runtime object is also available through `komorebi_math.signals.retained_packet_reconstruct`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1F-SIGNALS`.

Manifest signature (s).

```
(signal: 'Any', packet: 'Any', approximator: 'Callable[[np.ndarray], Any]', *, tolerance: 'float' = 1e-11) ->
    ↪ 'RetainedPacketResult'
```

Definition. Approximate only the packet removed component and restore the packet.

Contract (D, C, H, R, X). Truncated-power terms require finite locations and amplitudes with nonnegative integer order. Packet construction requires compatible term data. Reconstruction helpers require finite arrays of matching shape, and approximation matrices must have compatible dimensions. Region and location error helpers require nonnegative measures, compatible labels, and a positive finite exponent or infinity.

Guarantees (E, P, B, K). Singular packets evaluate, remove, and restore explicit truncated-power components. Retained-packet reconstruction records the smooth approximation, packet estimate, full reconstruction, and the exact finite error identity represented by those arrays. Location bounds are exact at jump order zero and use the documented mean-value estimate at positive orders. Runtime follows the number of samples times the number of retained terms plus any supplied approximation-matrix cost. The provenance registry records `PSC-V-T17` as Exact retained packet reconstruction identity. Its recorded status is `proved`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test module is `tests/test_signals_wavelets_singular.py`. The independent verifier for generating, signal, and export facades checks root-to-domain object identity, reflected signatures, implementation ownership, and representative CPU-safe behavior for the documented facade families. The source validator for generating, signal, and export facades checks exact card order, record counts, unique anchors, and preservation of the reference cards, technical blocks, and formulas. See Chapter 36.

Document model and rendering (10 cards).

C.1F · DocumentMetadata

Identity (q, a, t, k). Public research-manifest route `komorebi_math.DocumentMetadata`. Reflected implementation `komorebi_math.export.model.DocumentMetadata`. The same runtime object is also available through `komorebi_math.export.DocumentMetadata`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1F-DOCUMENT`.

Manifest signature (s).

```
(title: 'str' = 'Komorebi Math Export', author: 'str | None' = None, date: 'str | None' = None, abstract: 'str |
    ↪ None' = None, keywords: 'tuple[str, ...]' = ()) -> None
```

Definition. Metadata used by Markdown, HTML, and LaTeX document renderers.

Contract (D, C, H, R, X). Document records validate local structure at construction. Headings require nonempty text and levels from 1 through 6. Equations require nonempty LaTeX source. Tables must be rectangular and use one alignment entry per column when alignment is supplied. Lists must be nonempty. Figure paths are normalized, but file existence is checked only by a writer. `MathDocument` appends validated blocks in order and returns itself from its fluent construction methods.

Guarantees (E, P, B, K). The document model preserves block order and separates typed report structure from rendering. Block records are frozen, while the `MathDocument.blocks` list is a deliberate construction buffer. The model does not prove the truth of prose, equations, or certificate claims. `RawLatex` is trusted source and bypasses prose escaping only in the LaTeX renderer.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_export_renderers.py` and `tests/test_export_files.py`. The independent ver-

ifier for generating, signal, and export facades checks root-to-domain object identity, reflected signatures, implementation ownership, and representative CPU-safe behavior for the documented facade families. The source validator for generating, signal, and export facades checks exact card order, record counts, unique anchors, and preservation of the reference cards, technical blocks, and formulas. See Chapter 59.

C.1F · Equation

Identity (q, a, t, k). Public research-manifest route `komorebi_math.Equation`. Reflected implementation `komorebi_math.export.model.Equation`. The same runtime object is also available through `komorebi_math.export.Equation`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1F-DOCUMENT`.

Manifest signature (s).

```
(latex: 'str', label: 'str | None' = None, numbered: 'bool' = False, description: 'str | None' = None) -> None
```

Definition. One LaTeX mathematical expression.

Contract (D, C, H, R, X). Document records validate local structure at construction. Headings require nonempty text and levels from 1 through 6. Equations require nonempty LaTeX source. Tables must be rectangular and use one alignment entry per column when alignment is supplied. Lists must be nonempty. Figure paths are normalized, but file existence is checked only by a writer. `MathDocument` appends validated blocks in order and returns itself from its fluent construction methods.

Guarantees (E, P, B, K). The document model preserves block order and separates typed report structure from rendering. Block records are frozen, while the `MathDocument.blocks` list is a deliberate construction buffer. The model does not prove the truth of prose, equations, or certificate claims. `RawLatex` is trusted source and bypasses prose escaping only in the LaTeX renderer.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_export_renderers.py` and `tests/test_export_files.py`. The independent verifier for generating, signal, and export facades checks root-to-domain object identity, reflected signatures, implementation ownership, and representative CPU-safe behavior for the documented facade families. The source validator for generating, signal, and export facades checks exact card order, record counts, unique anchors, and preservation of the reference cards, technical blocks, and formulas. See Chapter 59.

C.1F · LatexCompileOptions

Identity (q, a, t, k). Public research-manifest route `komorebi_math.LatexCompileOptions`. Reflected implementation `komorebi_math.export.latex.LatexCompileOptions`. The same runtime object is also available through `komorebi_math.export.LatexCompileOptions`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1F-DOCUMENT`.

Manifest signature (s).

```
(engine: 'LatexEngine | str' = <LatexEngine.AUTO: 'auto'>, passes: 'int' = 2, timeout: 'float' = 120.0,
    ↪ shell_escape: 'bool' = False, keep_auxiliary: 'bool' = False, jobname: 'str | None' = None) -> None
```

Definition. Options for one isolated LaTeX compilation.

Contract (D, C, H, R, X). LaTeX compilation resolves an explicit engine or the first installed automatic candidate. Pass counts and timeouts must be positive. Job names must be simple file names. Shell escape is disabled by default, direct engines receive `-no-shell-escape`, and the `latexmk` wrapper rejects shell-escape requests. Source may be an existing `.tex` path or text written into the isolated output directory.

Guarantees (E, P, B, K). The compilation record retains source, PDF, log, engine, commands, return codes, and a certificate. Success requires all invoked commands to return zero and a nonempty PDF to exist. Missing engines raise `BackendUnavailableError`. Compiler success is local environment evidence, not a proof that trusted raw LaTeX is safe or that every external asset remains available later.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_export_files.py` and `tests/test_export_cli.py`. The independent verifier for

generating, signal, and export facades checks root-to-domain object identity, reflected signatures, implementation ownership, and representative CPU-safe behavior for the documented facade families. The source validator for generating, signal, and export facades checks exact card order, record counts, unique anchors, and preservation of the reference cards, technical blocks, and formulas. See Chapter 60. The recorded reference environment exposed pdfLaTeX, XeLaTeX, LuaLaTeX, and latexmk, but engine availability remains an environment observation rather than a package guarantee.

C.1F · LatexEngine

Identity (q, a, t, k). Public research-manifest route `komorebi_math.LatexEngine`. Reflected implementation `komorebi_math.export.latex.LatexEngine`. The same runtime object is also available through `komorebi_math.export.LatexEngine`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1F-DOCUMENT`.

Manifest signature (s).

```
(*values)
```

Definition. Enumeration of automatic, pdfLaTeX, XeLaTeX, LuaLaTeX, and latexmk compiler choices.

Contract (D, C, H, R, X). LaTeX compilation resolves an explicit engine or the first installed automatic candidate. Pass counts and timeouts must be positive. Job names must be simple file names. Shell escape is disabled by default, direct engines receive `-no-shell-escape`, and the `latexmk` wrapper rejects shell-escape requests. Source may be an existing `.tex` path or text written into the isolated output directory.

Guarantees (E, P, B, K). The compilation record retains source, PDF, log, engine, commands, return codes, and a certificate. Success requires all invoked commands to return zero and a nonempty PDF to exist. Missing engines raise `BackendUnavailableError`. Compiler success is local environment evidence, not a proof that trusted raw LaTeX is safe or that every external asset remains available later.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_export_files.py` and `tests/test_export_cli.py`. The independent verifier for generating, signal, and export facades checks root-to-domain object identity, reflected signatures, implementation ownership, and representative CPU-safe behavior for the documented facade families. The source validator for generating, signal, and export facades checks exact card order, record counts, unique anchors, and preservation of the reference cards, technical blocks, and formulas. See Chapter 60. The recorded reference environment exposed pdfLaTeX, XeLaTeX, LuaLaTeX, and latexmk, but engine availability remains an environment observation rather than a package guarantee.

C.1F · LatexRenderOptions

Identity (q, a, t, k). Public research-manifest route `komorebi_math.LatexRenderOptions`. Reflected implementation `komorebi_math.export.renderers.LatexRenderOptions`. The same runtime object is also available through `komorebi_math.export.LatexRenderOptions`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1F-DOCUMENT`.

Manifest signature (s).

```
(document_class: 'str' = 'article', class_options: 'tuple[str, ...]' = ('11pt',), packages: 'tuple[str, ...]' =
    ↪ ('amsmath', 'amssymb', 'booktabs', 'longtable', 'array', 'graphicx', 'hyperref', 'geometry', 'microtype'),
    ↪ include_table_of_contents: 'bool' = False, number_sections: 'bool' = True, geometry: 'str' = 'margin=1in')
    ↪ -> None
```

Definition. Configuration for generated LaTeX documents.

Contract (D, C, H, R, X). Renderers consume one `MathDocument` without modifying it. Ordinary prose, metadata, table cells, labels, and captions are escaped for the target format. Equation bodies and `RawLatex` remain trusted mathematical source. HTML can include an optional MathJax loader. LaTeX options validate document-class and package names before they enter the preamble.

Guarantees (E, P, B, K). Each renderer returns a complete text representation with the original block order. Markdown, HTML, text, and LaTeX outputs are semantically related but are not expected to be byte-identical. Rendering performs no filesystem write and no LaTeX compilation. The HTML MathJax option adds a network reference rather than bundling MathJax locally.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test module is `tests/test_export_renderers.py`. The independent verifier for generating, signal, and export facades checks root-to-domain object identity, reflected signatures, implementation ownership, and representative CPU-safe behavior for the documented facade families. The source validator for generating, signal, and export facades checks exact card order, record counts, unique anchors, and preservation of the reference cards, technical blocks, and formulas. See Chapter 60.

C.1F · `MathDocument`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.MathDocument`. Reflected implementation `komorebi_math.export.model.MathDocument`. The same runtime object is also available through `komorebi_math.export.MathDocument`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1F-DOCUMENT`.

Manifest signature (s).

```
(metadata: 'DocumentMetadata' = <factory>, blocks: 'list[DocumentBlock]' = <factory>) -> None
```

Definition. A renderer independent mathematical document.

Contract (D, C, H, R, X). Document records validate local structure at construction. Headings require nonempty text and levels from 1 through 6. Equations require nonempty LaTeX source. Tables must be rectangular and use one alignment entry per column when alignment is supplied. Lists must be nonempty. Figure paths are normalized, but file existence is checked only by a writer. `MathDocument` appends validated blocks in order and returns itself from its fluent construction methods. Principal public operations and properties are `add`, `section`, `paragraph`, `equation`, `table`, `code`, `bullet_list`, `figure`.

Guarantees (E, P, B, K). The document model preserves block order and separates typed report structure from rendering. Block records are frozen, while the `MathDocument.blocks` list is a deliberate construction buffer. The model does not prove the truth of prose, equations, or certificate claims. `RawLatex` is trusted source and bypasses prose escaping only in the LaTeX renderer.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_export_renderers.py` and `tests/test_export_files.py`. The independent verifier for generating, signal, and export facades checks root-to-domain object identity, reflected signatures, implementation ownership, and representative CPU-safe behavior for the documented facade families. The source validator for generating, signal, and export facades checks exact card order, record counts, unique anchors, and preservation of the reference cards, technical blocks, and formulas. See Chapter 59.

C.1F · `Table`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.Table`. Reflected implementation `komorebi_math.export.model.Table`. The same runtime object is also available through `komorebi_math.export.Table`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1F-DOCUMENT`.

Manifest signature (s).

```
(headers: 'tuple[str, ...]', rows: 'tuple[tuple[Any, ...], ...]', caption: 'str | None' = None, label: 'str | None'
   ↪  = None, alignment: 'tuple[str, ...] | None' = None) -> None
```

Definition. A rectangular table with optional caption and label.

Contract (D, C, H, R, X). Document records validate local structure at construction. Headings require nonempty text and levels from 1 through 6. Equations require nonempty LaTeX source. Tables must be rectangular and use one alignment entry per column when alignment is supplied. Lists must be nonempty. Figure paths are normalized, but

file existence is checked only by a writer. `MathDocument` appends validated blocks in order and returns itself from its fluent construction methods. Principal public operations and properties are `from_rows`.

Guarantees (`E`, `P`, `B`, `K`). The document model preserves block order and separates typed report structure from rendering. Block records are frozen, while the `MathDocument.blocks` list is a deliberate construction buffer. The model does not prove the truth of prose, equations, or certificate claims. `RawLatex` is trusted source and bypasses prose escaping only in the LaTeX renderer.

Evidence and links (`V`, `L`). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_export_renderers.py` and `tests/test_export_files.py`. The independent verifier for generating, signal, and export facades checks root-to-domain object identity, reflected signatures, implementation ownership, and representative CPU-safe behavior for the documented facade families. The source validator for generating, signal, and export facades checks exact card order, record counts, unique anchors, and preservation of the reference cards, technical blocks, and formulas. See Chapter 59.

C.1F · `available_latex_engines`

Identity (`q`, `a`, `t`, `k`). Public research-manifest route `komorebi_math.available_latex_engines`. Reflected implementation `komorebi_math.export.latex.available_latex_engines`. The same runtime object is also available through `komorebi_math.export.available_latex_engines`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1F-DOCUMENT`.

Manifest signature (`s`).

```
() -> 'dict[str, bool]'
```

Definition. Report which supported LaTeX commands are available on PATH.

Contract (`D`, `C`, `H`, `R`, `X`). LaTeX compilation resolves an explicit engine or the first installed automatic candidate. Pass counts and timeouts must be positive. Job names must be simple file names. Shell escape is disabled by default, direct engines receive `-no-shell-escape`, and the `latexmk` wrapper rejects shell-escape requests. Source may be an existing `.tex` path or text written into the isolated output directory.

Guarantees (`E`, `P`, `B`, `K`). The compilation record retains source, PDF, log, engine, commands, return codes, and a certificate. Success requires all invoked commands to return zero and a nonempty PDF to exist. Missing engines raise `BackendUnavailableError`. Compiler success is local environment evidence, not a proof that trusted raw LaTeX is safe or that every external asset remains available later.

Evidence and links (`V`, `L`). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_export_files.py` and `tests/test_export_cli.py`. The independent verifier for generating, signal, and export facades checks root-to-domain object identity, reflected signatures, implementation ownership, and representative CPU-safe behavior for the documented facade families. The source validator for generating, signal, and export facades checks exact card order, record counts, unique anchors, and preservation of the reference cards, technical blocks, and formulas. See Chapter 60. The recorded reference environment exposed pdfLaTeX, XeLaTeX, LuaLaTeX, and latexmk, but engine availability remains an environment observation rather than a package guarantee.

C.1F · `compile_latex`

Identity (`q`, `a`, `t`, `k`). Public research-manifest route `komorebi_math.compile_latex`. Reflected implementation `komorebi_math.export.latex.compile_latex`. The same runtime object is also available through `komorebi_math.export.compile_latex`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1F-DOCUMENT`.

Manifest signature (s).

```
(source: 'str | Path', output_dir: 'str | Path', *, options: 'LatexCompileOptions | None' = None, source_name:
    ↪  'str' = 'document.tex') -> 'LatexCompilation'
```

Definition. Compile LaTeX source to PDF without enabling shell escape by default.

Contract (D, C, H, R, X). LaTeX compilation resolves an explicit engine or the first installed automatic candidate. Pass counts and timeouts must be positive. Job names must be simple file names. Shell escape is disabled by default, direct engines receive `-no-shell-escape`, and the `latexmk` wrapper rejects shell-escape requests. Source may be an existing `.tex` path or text written into the isolated output directory.

Guarantees (E, P, B, K). The compilation record retains source, PDF, log, engine, commands, return codes, and a certificate. Success requires all invoked commands to return zero and a nonempty PDF to exist. Missing engines raise `BackendUnavailableError`. Compiler success is local environment evidence, not a proof that trusted raw LaTeX is safe or that every external asset remains available later.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_export_files.py` and `tests/test_export_cli.py`. The independent verifier for generating, signal, and export facades checks root-to-domain object identity, reflected signatures, implementation ownership, and representative CPU-safe behavior for the documented facade families. The source validator for generating, signal, and export facades checks exact card order, record counts, unique anchors, and preservation of the reference cards, technical blocks, and formulas. See Chapter 60. The recorded reference environment exposed pdfLaTeX, XeLaTeX, LuaLaTeX, and latexmk, but engine availability remains an environment observation rather than a package guarantee.

C.1F · `render_latex`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.render_latex`. Reflected implementation `komorebi_math.export.renderers.render_latex`. The same runtime object is also available through `komorebi_math.export.render_latex`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1F-DOCUMENT`.

Manifest signature (s).

```
(document: 'MathDocument', *, options: 'LatexRenderOptions | None' = None) -> 'str'
```

Definition. Render a complete compilable LaTeX document.

Contract (D, C, H, R, X). Renderers consume one `MathDocument` without modifying it. Ordinary prose, metadata, table cells, labels, and captions are escaped for the target format. Equation bodies and `RawLatex` remain trusted mathematical source. HTML can include an optional MathJax loader. LaTeX options validate document-class and package names before they enter the preamble.

Guarantees (E, P, B, K). Each renderer returns a complete text representation with the original block order. Markdown, HTML, text, and LaTeX outputs are semantically related but are not expected to be byte-identical. Rendering performs no filesystem write and no LaTeX compilation. The HTML MathJax option adds a network reference rather than bundling MathJax locally.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test module is `tests/test_export_renderers.py`. The independent verifier for generating, signal, and export facades checks root-to-domain object identity, reflected signatures, implementation ownership, and representative CPU-safe behavior for the documented facade families. The source validator for generating, signal, and export facades checks exact card order, record counts, unique anchors, and preservation of the reference cards, technical blocks, and formulas. See Chapter 60.

Portable persistence and serialization (13 cards).

C.1F · PortableCodec

Identity (q, a, t, k). Public research-manifest route `komorebi_math.PortableCodec`. Reflected implementation `komorebi_math.export.persistence.PortableCodec`. The same runtime object is also available through `komorebi_math.export.PortableCodec`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1F-PORTABLE`.

Manifest signature (s).

```
(tag: 'str', version: 'int', type_: 'type[Any]', encoder: 'Encoder', decoder: 'Decoder') -> None
```

Definition. One explicit type codec in the portable persistence registry.

Contract (D, C, H, R, X). Portable persistence accepts finite JSON primitives, explicit structural tags, and registered codecs. Codec tags are nonempty and whitespace-free. Versions are positive integers. Migrations advance exactly one version at a time. The envelope has exactly five fields and binds the canonical encoded payload to SHA-256. Loading rejects duplicate JSON keys, nonfinite constants, unknown schemas, unsupported versions, digest mismatches, malformed tags, missing migration edges, and depth-limit violations.

Guarantees (E, P, B, K). Canonical payload ordering makes the digest deterministic for a fixed codec contract and value. Decoding never imports a class named by untrusted data. `save_portable` writes and flushes a temporary sibling before `os.replace`, while `load_portable` delegates all validation to the envelope decoder. SHA-256 provides integrity detection, not encryption, secrecy, or authentication. Extension codecs remain dynamic process state unless separately frozen by a release contract.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_block15_persistence.py`, `tests/test_block10_execution_persistence.py`, `tests/test_block11_streaming_migrations.py`, and `tests/test_nonlinear_snapshots_persistence.py`. The independent verifier for generating, signal, and export facades checks root-to-domain object identity, reflected signatures, implementation ownership, and representative CPU-safe behavior for the documented facade families. The source validator for generating, signal, and export facades checks exact card order, record counts, unique anchors, and preservation of the reference cards, technical blocks, and formulas. See Chapter 61.

C.1F · PortableMigration

Identity (q, a, t, k). Public research-manifest route `komorebi_math.PortableMigration`. Reflected implementation `komorebi_math.export.persistence.PortableMigration`. The same runtime object is also available through `komorebi_math.export.PortableMigration`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1F-PORTABLE`.

Manifest signature (s).

```
(tag: 'str', from_version: 'int', to_version: 'int', migrator: 'Migrator') -> None
```

Definition. One explicit, adjacent codec-data migration.

Contract (D, C, H, R, X). Portable persistence accepts finite JSON primitives, explicit structural tags, and registered codecs. Codec tags are nonempty and whitespace-free. Versions are positive integers. Migrations advance exactly one version at a time. The envelope has exactly five fields and binds the canonical encoded payload to SHA-256. Loading rejects duplicate JSON keys, nonfinite constants, unknown schemas, unsupported versions, digest mismatches, malformed tags, missing migration edges, and depth-limit violations.

Guarantees (E, P, B, K). Canonical payload ordering makes the digest deterministic for a fixed codec contract and value. Decoding never imports a class named by untrusted data. `save_portable` writes and flushes a temporary sibling before `os.replace`, while `load_portable` delegates all validation to the envelope decoder. SHA-256 provides integrity detection, not encryption, secrecy, or authentication. Extension codecs remain dynamic process state unless separately frozen by a release contract.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_block15_persistence.py`, `tests/test_block10_execution_persistence.py`,

tests/test_block11_streaming_migrations.py, and tests/test_nonlinear_snapshots_persistence.py. The independent verifier for generating, signal, and export facades checks root-to-domain object identity, reflected signatures, implementation ownership, and representative CPU-safe behavior for the documented facade families. The source validator for generating, signal, and export facades checks exact card order, record counts, unique anchors, and preservation of the reference cards, technical blocks, and formulas. See Chapter 61.

C.1F · dumps_portable

Identity (q, a, t, k). Public research-manifest route komorebi_math.dumps_portable. Reflected implementation komorebi_math.export.persistence.dumps_portable. The same runtime object is also available through komorebi_math.export.dumps_portable. The root route is governed by [RESEARCH] policy pre-beta-additive. This facade does not enlarge the frozen stable compatibility promise. Manifest kind function. Profile C1F-PORTABLE.

Manifest signature (s).

```
(value: 'Any', *, indent: 'int | None' = 2, max_depth: 'int' = 128) -> 'str'
```

Definition. Serialize one value into a complete digest-checked JSON envelope.

Contract (D, C, H, R, X). Portable persistence accepts finite JSON primitives, explicit structural tags, and registered codecs. Codec tags are nonempty and whitespace-free. Versions are positive integers. Migrations advance exactly one version at a time. The envelope has exactly five fields and binds the canonical encoded payload to SHA-256. Loading rejects duplicate JSON keys, nonfinite constants, unknown schemas, unsupported versions, digest mismatches, malformed tags, missing migration edges, and depth-limit violations.

Guarantees (E, P, B, K). Canonical payload ordering makes the digest deterministic for a fixed codec contract and value. Decoding never imports a class named by untrusted data. save_portable writes and flushes a temporary sibling before os.replace, while load_portable delegates all validation to the envelope decoder. SHA-256 provides integrity detection, not encryption, secrecy, or authentication. Extension codecs remain dynamic process state unless separately frozen by a release contract.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are tests/test_block15_persistence.py, tests/test_block10_execution_persistence.py, tests/test_block11_streaming_migrations.py, and tests/test_nonlinear_snapshots_persistence.py. The independent verifier for generating, signal, and export facades checks root-to-domain object identity, reflected signatures, implementation ownership, and representative CPU-safe behavior for the documented facade families. The source validator for generating, signal, and export facades checks exact card order, record counts, unique anchors, and preservation of the reference cards, technical blocks, and formulas. See Chapter 61.

C.1F · from_serializable

Identity (q, a, t, k). Public research-manifest route komorebi_math.from_serializable. Reflected implementation komorebi_math.export.serialization.from_serializable. The same runtime object is also available through komorebi_math.export.from_serializable. The root route is governed by [RESEARCH] policy pre-beta-additive. This facade does not enlarge the frozen stable compatibility promise. Manifest kind function. Profile C1F-PORTABLE.

Manifest signature (s).

```
(value: 'Any', *, max_depth: 'int' = 64, _depth: 'int' = 0) -> 'Any'
```

Definition. Decode the safe built-in type markers emitted by to_serializable.

Contract (D, C, H, R, X). Presentation serialization accepts supported Python primitives, NumPy values, fractions, complex values, paths, enums, package result records, dataclasses, mappings, sequences, registered adapters, and objects implementing Exportable. Recursion is bounded by max_depth. Registered adapters are selected through the runtime type MRO. Decoding reconstructs only the safe built-in markers emitted by this layer and leaves unknown type markers as ordinary mappings.

Guarantees (E, P, B, K). The result is JSON-compatible presentation data. Type preservation is selective rather than a general object-reconstruction promise. Mapping keys become strings, sets are ordered by repr, and arbitrary class

names in `__type__` are never imported or instantiated. The adapter registry is process-local and a copied registry returned by `export_adapters` cannot mutate the active registry.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test module is `tests/test_export_serialization.py`. The independent verifier for generating, signal, and export facades checks root-to-domain object identity, reflected signatures, implementation ownership, and representative CPU-safe behavior for the documented facade families. The source validator for generating, signal, and export facades checks exact card order, record counts, unique anchors, and preservation of the reference cards, technical blocks, and formulas. See Chapters 59 and 60.

C.1F · `load_portable`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.load_portable`. Reflected implementation `komorebi_math.export.persistence.load_portable`. The same runtime object is also available through `komorebi_math.export.load_portable`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1F-PORTABLE`.

Manifest signature (s).

```
(path: 'str | os.PathLike[str]', *, max_depth: 'int' = 128) -> 'Any'
```

Definition. Read and validate a portable envelope from disk.

Contract (D, C, H, R, X). Portable persistence accepts finite JSON primitives, explicit structural tags, and registered codecs. Codec tags are nonempty and whitespace-free. Versions are positive integers. Migrations advance exactly one version at a time. The envelope has exactly five fields and binds the canonical encoded payload to SHA-256. Loading rejects duplicate JSON keys, nonfinite constants, unknown schemas, unsupported versions, digest mismatches, malformed tags, missing migration edges, and depth-limit violations.

Guarantees (E, P, B, K). Canonical payload ordering makes the digest deterministic for a fixed codec contract and value. Decoding never imports a class named by untrusted data. `save_portable` writes and flushes a temporary sibling before `os.replace`, while `load_portable` delegates all validation to the envelope decoder. SHA-256 provides integrity detection, not encryption, secrecy, or authentication. Extension codecs remain dynamic process state unless separately frozen by a release contract.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_block15_persistence.py`, `tests/test_block10_execution_persistence.py`, `tests/test_block11_streaming_migrations.py`, and `tests/test_nonlinear_snapshots_persistence.py`. The independent verifier for generating, signal, and export facades checks root-to-domain object identity, reflected signatures, implementation ownership, and representative CPU-safe behavior for the documented facade families. The source validator for generating, signal, and export facades checks exact card order, record counts, unique anchors, and preservation of the reference cards, technical blocks, and formulas. See Chapter 61.

C.1F · `loads_portable`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.loads_portable`. Reflected implementation `komorebi_math.export.persistence.loads_portable`. The same runtime object is also available through `komorebi_math.export.loads_portable`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1F-PORTABLE`.

Manifest signature (s).

```
(text: 'str | bytes', *, max_depth: 'int' = 128) -> 'Any'
```

Definition. Validate and deserialize one complete portable JSON envelope.

Contract (D, C, H, R, X). Portable persistence accepts finite JSON primitives, explicit structural tags, and registered codecs. Codec tags are nonempty and whitespace-free. Versions are positive integers. Migrations advance exactly one version at a time. The envelope has exactly five fields and binds the canonical encoded payload to SHA-256. Loading rejects duplicate JSON keys, nonfinite constants, unknown schemas, unsupported versions, digest mismatches, malformed tags, missing migration edges, and depth-limit violations.

Guarantees (E, P, B, K). Canonical payload ordering makes the digest deterministic for a fixed codec contract and value. Decoding never imports a class named by untrusted data. `save_portable` writes and flushes a temporary sibling before `os.replace`, while `load_portable` delegates all validation to the envelope decoder. SHA-256 provides integrity detection, not encryption, secrecy, or authentication. Extension codecs remain dynamic process state unless separately frozen by a release contract.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_block15_persistence.py`, `tests/test_block10_execution_persistence.py`, `tests/test_block11_streaming_migrations.py`, and `tests/test_nonlinear_snapshots_persistence.py`. The independent verifier for generating, signal, and export facades checks root-to-domain object identity, reflected signatures, implementation ownership, and representative CPU-safe behavior for the documented facade families. The source validator for generating, signal, and export facades checks exact card order, record counts, unique anchors, and preservation of the reference cards, technical blocks, and formulas. See Chapter 61.

C.1F · `portable_codecs`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.portable_codecs`. Reflected implementation `komorebi_math.export.persistence.portable_codecs`. The same runtime object is also available through `komorebi_math.export.portable_codecs`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1F-PORTABLE`.

Manifest signature (s).

```
() -> 'tuple[PortableCodec, ...]'
```

Definition. Return the active codecs ordered by stable tag.

Contract (D, C, H, R, X). Portable persistence accepts finite JSON primitives, explicit structural tags, and registered codecs. Codec tags are nonempty and whitespace-free. Versions are positive integers. Migrations advance exactly one version at a time. The envelope has exactly five fields and binds the canonical encoded payload to SHA-256. Loading rejects duplicate JSON keys, nonfinite constants, unknown schemas, unsupported versions, digest mismatches, malformed tags, missing migration edges, and depth-limit violations.

Guarantees (E, P, B, K). Canonical payload ordering makes the digest deterministic for a fixed codec contract and value. Decoding never imports a class named by untrusted data. `save_portable` writes and flushes a temporary sibling before `os.replace`, while `load_portable` delegates all validation to the envelope decoder. SHA-256 provides integrity detection, not encryption, secrecy, or authentication. Extension codecs remain dynamic process state unless separately frozen by a release contract.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_block15_persistence.py`, `tests/test_block10_execution_persistence.py`, `tests/test_block11_streaming_migrations.py`, and `tests/test_nonlinear_snapshots_persistence.py`. The independent verifier for generating, signal, and export facades checks root-to-domain object identity, reflected signatures, implementation ownership, and representative CPU-safe behavior for the documented facade families. The source validator for generating, signal, and export facades checks exact card order, record counts, unique anchors, and preservation of the reference cards, technical blocks, and formulas. See Chapter 61.

C.1F · `portable_digest`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.portable_digest`. Reflected implementation `komorebi_math.export.persistence.portable_digest`. The same runtime object is also available through `komorebi_math.export.portable_digest`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1F-PORTABLE`.

Manifest signature (s).

```
(value: 'Any', *, max_depth: 'int' = 128) -> 'str'
```

Definition. Return the canonical SHA256 digest of one encoded payload.

Contract (D, C, H, R, X). Portable persistence accepts finite JSON primitives, explicit structural tags, and registered codecs. Codec tags are nonempty and whitespace-free. Versions are positive integers. Migrations advance exactly one version at a time. The envelope has exactly five fields and binds the canonical encoded payload to SHA-256. Loading rejects duplicate JSON keys, nonfinite constants, unknown schemas, unsupported versions, digest mismatches, malformed tags, missing migration edges, and depth-limit violations.

Guarantees (E, P, B, K). Canonical payload ordering makes the digest deterministic for a fixed codec contract and value. Decoding never imports a class named by untrusted data. `save_portable` writes and flushes a temporary sibling before `os.replace`, while `load_portable` delegates all validation to the envelope decoder. SHA-256 provides integrity detection, not encryption, secrecy, or authentication. Extension codecs remain dynamic process state unless separately frozen by a release contract.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_block15_persistence.py`, `tests/test_block10_execution_persistence.py`, `tests/test_block11_streaming_migrations.py`, and `tests/test_nonlinear_snapshots_persistence.py`. The independent verifier for generating, signal, and export facades checks root-to-domain object identity, reflected signatures, implementation ownership, and representative CPU-safe behavior for the documented facade families. The source validator for generating, signal, and export facades checks exact card order, record counts, unique anchors, and preservation of the reference cards, technical blocks, and formulas. See Chapter 61.

C.1F · portable_migrations

Identity (q, a, t, k). Public research-manifest route `komorebi_math.portable_migrations`. Reflected implementation `komorebi_math.export.persistence.portable_migrations`. The same runtime object is also available through `komorebi_math.export.portable_migrations`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1F-PORTABLE`.

Manifest signature (s).

```
() -> 'tuple[PortableMigration, ...]'
```

Definition. Return registered codec migrations in deterministic order.

Contract (D, C, H, R, X). Portable persistence accepts finite JSON primitives, explicit structural tags, and registered codecs. Codec tags are nonempty and whitespace-free. Versions are positive integers. Migrations advance exactly one version at a time. The envelope has exactly five fields and binds the canonical encoded payload to SHA-256. Loading rejects duplicate JSON keys, nonfinite constants, unknown schemas, unsupported versions, digest mismatches, malformed tags, missing migration edges, and depth-limit violations.

Guarantees (E, P, B, K). Canonical payload ordering makes the digest deterministic for a fixed codec contract and value. Decoding never imports a class named by untrusted data. `save_portable` writes and flushes a temporary sibling before `os.replace`, while `load_portable` delegates all validation to the envelope decoder. SHA-256 provides integrity detection, not encryption, secrecy, or authentication. Extension codecs remain dynamic process state unless separately frozen by a release contract.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_block15_persistence.py`, `tests/test_block10_execution_persistence.py`, `tests/test_block11_streaming_migrations.py`, and `tests/test_nonlinear_snapshots_persistence.py`. The independent verifier for generating, signal, and export facades checks root-to-domain object identity, reflected signatures, implementation ownership, and representative CPU-safe behavior for the documented facade families. The source validator for generating, signal, and export facades checks exact card order, record counts, unique anchors, and preservation of the reference cards, technical blocks, and formulas. See Chapter 61.

C.1F · register_portable_codec

Identity (q, a, t, k). Public research-manifest route `komorebi_math.register_portable_codec`. Reflected implementation `komorebi_math.export.persistence.register_portable_codec`. The same runtime object is also available through `komorebi_math.export.register_portable_codec`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1F-PORTABLE`.

Manifest signature (s).

```
(type_: 'type[T]', *, tag: 'str', version: 'int', encoder: 'Callable[[T], Any]', decoder: 'Callable[[Any], T]',
   ↪ replace: 'bool' = False) -> 'None'
```

Definition. Register one stable round-trip codec.

Contract (D, C, H, R, X). Portable persistence accepts finite JSON primitives, explicit structural tags, and registered codecs. Codec tags are nonempty and whitespace-free. Versions are positive integers. Migrations advance exactly one version at a time. The envelope has exactly five fields and binds the canonical encoded payload to SHA-256. Loading rejects duplicate JSON keys, nonfinite constants, unknown schemas, unsupported versions, digest mismatches, malformed tags, missing migration edges, and depth-limit violations.

Guarantees (E, P, B, K). Canonical payload ordering makes the digest deterministic for a fixed codec contract and value. Decoding never imports a class named by untrusted data. `save_portable` writes and flushes a temporary sibling before `os.replace`, while `load_portable` delegates all validation to the envelope decoder. SHA-256 provides integrity detection, not encryption, secrecy, or authentication. Extension codecs remain dynamic process state unless separately frozen by a release contract.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_block15_persistence.py`, `tests/test_block10_execution_persistence.py`, `tests/test_block11_streaming_migrations.py`, and `tests/test_nonlinear_snapshots_persistence.py`. The independent verifier for generating, signal, and export facades checks root-to-domain object identity, reflected signatures, implementation ownership, and representative CPU-safe behavior for the documented facade families. The source validator for generating, signal, and export facades checks exact card order, record counts, unique anchors, and preservation of the reference cards, technical blocks, and formulas. See Chapter 61.

C.1F · register_portable_migration

Identity (q, a, t, k). Public research-manifest route `komorebi_math.register_portable_migration`. Reflected implementation `komorebi_math.export.persistence.register_portable_migration`. The same runtime object is also available through `komorebi_math.export.register_portable_migration`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1F-PORTABLE`.

Manifest signature (s).

```
(*, tag: 'str', from_version: 'int', to_version: 'int', migrator: 'Migrator', replace: 'bool' = False) -> 'None'
```

Definition. Register one deterministic adjacent migration for codec data.

Contract (D, C, H, R, X). Portable persistence accepts finite JSON primitives, explicit structural tags, and registered codecs. Codec tags are nonempty and whitespace-free. Versions are positive integers. Migrations advance exactly one version at a time. The envelope has exactly five fields and binds the canonical encoded payload to SHA-256. Loading rejects duplicate JSON keys, nonfinite constants, unknown schemas, unsupported versions, digest mismatches, malformed tags, missing migration edges, and depth-limit violations.

Guarantees (E, P, B, K). Canonical payload ordering makes the digest deterministic for a fixed codec contract and value. Decoding never imports a class named by untrusted data. `save_portable` writes and flushes a temporary sibling before `os.replace`, while `load_portable` delegates all validation to the envelope decoder. SHA-256 provides integrity detection, not encryption, secrecy, or authentication. Extension codecs remain dynamic process state unless separately frozen by a release contract.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_block15_persistence.py`, `tests/test_block10_execution_persistence.py`,

tests/test_block11_streaming_migrations.py, and tests/test_nonlinear_snapshots_persistence.py. The independent verifier for generating, signal, and export facades checks root-to-domain object identity, reflected signatures, implementation ownership, and representative CPU-safe behavior for the documented facade families. The source validator for generating, signal, and export facades checks exact card order, record counts, unique anchors, and preservation of the reference cards, technical blocks, and formulas. See Chapter 61.

C.1F · save_portable

Identity (q, a, t, k). Public research-manifest route komorebi_math.save_portable. Reflected implementation komorebi_math.export.persistence.save_portable. The same runtime object is also available through komorebi_math.export.save_portable. The root route is governed by [RESEARCH] policy pre-beta-additive. This facade does not enlarge the frozen stable compatibility promise. Manifest kind function. Profile C1F-PORTABLE.

Manifest signature (s).

```
(value: 'Any', path: 'str | os.PathLike[str]', *, overwrite: 'bool' = False, indent: 'int | None' = 2, max_depth:
    ↪  'int' = 128) -> 'Path'
```

Definition. Atomically write a portable envelope to disk.

Contract (D, C, H, R, X). Portable persistence accepts finite JSON primitives, explicit structural tags, and registered codecs. Codec tags are nonempty and whitespace-free. Versions are positive integers. Migrations advance exactly one version at a time. The envelope has exactly five fields and binds the canonical encoded payload to SHA-256. Loading rejects duplicate JSON keys, nonfinite constants, unknown schemas, unsupported versions, digest mismatches, malformed tags, missing migration edges, and depth-limit violations.

Guarantees (E, P, B, K). Canonical payload ordering makes the digest deterministic for a fixed codec contract and value. Decoding never imports a class named by untrusted data. save_portable writes and flushes a temporary sibling before os.replace, while load_portable delegates all validation to the envelope decoder. SHA-256 provides integrity detection, not encryption, secrecy, or authentication. Extension codecs remain dynamic process state unless separately frozen by a release contract.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are tests/test_block15_persistence.py, tests/test_block10_execution_persistence.py, tests/test_block11_streaming_migrations.py, and tests/test_nonlinear_snapshots_persistence.py. The independent verifier for generating, signal, and export facades checks root-to-domain object identity, reflected signatures, implementation ownership, and representative CPU-safe behavior for the documented facade families. The source validator for generating, signal, and export facades checks exact card order, record counts, unique anchors, and preservation of the reference cards, technical blocks, and formulas. See Chapter 61.

C.1F · to_serializable

Identity (q, a, t, k). Public research-manifest route komorebi_math.to_serializable. Reflected implementation komorebi_math.export.serialization.to_serializable. The same runtime object is also available through komorebi_math.export.to_serializable. The root route is governed by [RESEARCH] policy pre-beta-additive. This facade does not enlarge the frozen stable compatibility promise. Manifest kind function. Profile C1F-PORTABLE.

Manifest signature (s).

```
(value: 'Any', *, preserve_types: 'bool' = True, max_depth: 'int' = 64, _depth: 'int' = 0) -> 'Any'
```

Definition. Convert package and scientific Python objects to JSON compatible data.

Contract (D, C, H, R, X). Presentation serialization accepts supported Python primitives, NumPy values, fractions, complex values, paths, enums, package result records, dataclasses, mappings, sequences, registered adapters, and objects implementing Exportable. Recursion is bounded by max_depth. Registered adapters are selected through the runtime type MRO. Decoding reconstructs only the safe built-in markers emitted by this layer and leaves unknown type markers as ordinary mappings.

Guarantees (E, P, B, K). The result is JSON-compatible presentation data. Type preservation is selective rather than a general object-reconstruction promise. Mapping keys become strings, sets are ordered by repr, and arbitrary class

names in __type__ are never imported or instantiated. The adapter registry is process-local and a copied registry returned by export_adapters cannot mutate the active registry.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test module is tests/test_export_serialization.py. The independent verifier for generating, signal, and export facades checks root-to-domain object identity, reflected signatures, implementation ownership, and representative CPU-safe behavior for the documented facade families. The source validator for generating, signal, and export facades checks exact card order, record counts, unique anchors, and preservation of the reference cards, technical blocks, and formulas. See Chapters 59 and 60.

Export writers and reports (11 cards).

C.1F · ExportBundleResult

Identity (q, a, t, k). Public research-manifest route komorebi_math.ExportBundleResult. Reflected implementation komorebi_math.export.writers.ExportBundleResult. The same runtime object is also available through komorebi_math.export.ExportBundleResult. The root route is governed by [RESEARCH] policy pre-beta-additive. This facade does not enlarge the frozen stable compatibility promise. Manifest kind class. Profile C1F-EXPORT.

Manifest signature (s).

```
(path: 'Path', members: 'tuple[str, ...]', formats: 'tuple[ExportFormat, ...]', manifest_path: 'Path',
   ↪ certificate: 'Certificate') -> None
```

Definition. A ZIP bundle containing several exports and a hash manifest.

Contract (D, C, H, R, X). Writers resolve one of the 11 public formats, enforce parent-directory and overwrite policy, localize figure files when needed, and install primary outputs through temporary files. YAML requires PyYAML. PDF export requires a supported local LaTeX engine. NPY and NPZ writes disable pickle. Multi-format export uses deterministic suffixes, while bundle export stages files, records SHA-256 values, normalizes ZIP timestamps, and adds MANIFEST.sha256. Principal public operations and properties are valid, require.

Guarantees (E, P, B, K). A valid result certificate establishes that the primary output exists, is nonempty, and matches the observed byte count. Companion paths record localized assets, generated TeX, or compiler logs. A bundle manifest supports later integrity replay, but it is not a keyed signature and does not authenticate the archive against replacement of both data and manifest. Writer certificates do not certify the mathematical truth of rendered content.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are tests/test_export_files.py and tests/test_export_cli.py. The independent verifier for generating, signal, and export facades checks root-to-domain object identity, reflected signatures, implementation ownership, and representative CPU-safe behavior for the documented facade families. The source validator for generating, signal, and export facades checks exact card order, record counts, unique anchors, and preservation of the reference cards, technical blocks, and formulas. See Chapter 60.

C.1F · ExportFormat

Identity (q, a, t, k). Public research-manifest route komorebi_math.ExportFormat. Reflected implementation komorebi_math.export.writers.ExportFormat. The same runtime object is also available through komorebi_math.export.ExportFormat. The root route is governed by [RESEARCH] policy pre-beta-additive. This facade does not enlarge the frozen stable compatibility promise. Manifest kind class. Profile C1F-EXPORT.

Manifest signature (s).

```
(*values)
```

Definition. Enumeration of the 11 supported on-disk formats: JSON, YAML, CSV, TSV, text, Markdown, HTML, LaTeX, PDF, NPY, and NPZ.

Contract (D, C, H, R, X). Writers resolve one of the 11 public formats, enforce parent-directory and overwrite policy, localize figure files when needed, and install primary outputs through temporary files. YAML requires PyYAML. PDF export requires a supported local LaTeX engine. NPY and NPZ writes disable pickle. Multi-format export uses

deterministic suffixes, while bundle export stages files, records SHA-256 values, normalizes ZIP timestamps, and adds `MANIFEST.sha256`.

Guarantees (`E, P, B, K`). A valid result certificate establishes that the primary output exists, is nonempty, and matches the observed byte count. Companion paths record localized assets, generated TeX, or compiler logs. A bundle manifest supports later integrity replay, but it is not a keyed signature and does not authenticate the archive against replacement of both data and manifest. Writer certificates do not certify the mathematical truth of rendered content.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_export_files.py` and `tests/test_export_cli.py`. The independent verifier for generating, signal, and export facades checks root-to-domain object identity, reflected signatures, implementation ownership, and representative CPU-safe behavior for the documented facade families. The source validator for generating, signal, and export facades checks exact card order, record counts, unique anchors, and preservation of the reference cards, technical blocks, and formulas. See Chapter 60.

C.1F · `ExportOptions`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.ExportOptions`. Reflected implementation `komorebi_math.export.writers.ExportOptions`. The same runtime object is also available through `komorebi_math.export.ExportOptions`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1F-EXPORT`.

Manifest signature (`s`).

```
(title: 'str' = 'Komorebi Math Export', author: 'str | None' = None, date: 'str | None' = None, abstract: 'str |
    ↪ None' = None, overwrite: 'bool' = False, create_parents: 'bool' = True, preserve_types: 'bool' = True,
    ↪ json_indent: 'int' = 2, include_certificate: 'bool' = True, keep_tex: 'bool' = True, html:
    ↪ 'HtmlRenderOptions' = <factory>, latex: 'LatexRenderOptions' = <factory>, compiler: 'LatexCompileOptions' =
    ↪ <factory>) -> None
```

Definition. Common export behavior and renderer options.

Contract (`D, C, H, R, X`). Writers resolve one of the 11 public formats, enforce parent-directory and overwrite policy, localize figure files when needed, and install primary outputs through temporary files. YAML requires PyYAML. PDF export requires a supported local LaTeX engine. NPY and NPZ writes disable pickle. Multi-format export uses deterministic suffixes, while bundle export stages files, records SHA-256 values, normalizes ZIP timestamps, and adds `MANIFEST.sha256`.

Guarantees (`E, P, B, K`). A valid result certificate establishes that the primary output exists, is nonempty, and matches the observed byte count. Companion paths record localized assets, generated TeX, or compiler logs. A bundle manifest supports later integrity replay, but it is not a keyed signature and does not authenticate the archive against replacement of both data and manifest. Writer certificates do not certify the mathematical truth of rendered content.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_export_files.py` and `tests/test_export_cli.py`. The independent verifier for generating, signal, and export facades checks root-to-domain object identity, reflected signatures, implementation ownership, and representative CPU-safe behavior for the documented facade families. The source validator for generating, signal, and export facades checks exact card order, record counts, unique anchors, and preservation of the reference cards, technical blocks, and formulas. See Chapter 60.

C.1F · `ExportResult`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.ExportResult`. Reflected implementation `komorebi_math.export.writers.ExportResult`. The same runtime object is also available through `komorebi_math.export.ExportResult`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1F-EXPORT`.

Manifest signature (s).

```
(path: 'Path', format: 'ExportFormat', bytes_written: 'int', companion_files: 'tuple[Path, ...]', certificate:
   ↪ 'Certificate') -> None
```

Definition. One completed file export and its validation certificate.

Contract (D, C, H, R, X). Writers resolve one of the 11 public formats, enforce parent-directory and overwrite policy, localize figure files when needed, and install primary outputs through temporary files. YAML requires PyYAML. PDF export requires a supported local LaTeX engine. NPY and NPZ writes disable pickle. Multi-format export uses deterministic suffixes, while bundle export stages files, records SHA-256 values, normalizes ZIP timestamps, and adds `MANIFEST.sha256`. Principal public operations and properties are `valid`, `require`.

Guarantees (E, P, B, K). A valid result certificate establishes that the primary output exists, is nonempty, and matches the observed byte count. Companion paths record localized assets, generated TeX, or compiler logs. A bundle manifest supports later integrity replay, but it is not a keyed signature and does not authenticate the archive against replacement of both data and manifest. Writer certificates do not certify the mathematical truth of rendered content.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_export_files.py` and `tests/test_export_cli.py`. The independent verifier for generating, signal, and export facades checks root-to-domain object identity, reflected signatures, implementation ownership, and representative CPU-safe behavior for the documented facade families. The source validator for generating, signal, and export facades checks exact card order, record counts, unique anchors, and preservation of the reference cards, technical blocks, and formulas. See Chapter 60.

C.1F · `available_export_formats`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.available_export_formats`. Reflected implementation `komorebi_math.export.writers.available_export_formats`. The same runtime object is also available through `komorebi_math.export.available_export_formats`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1F-EXPORT`.

Manifest signature (s).

```
() -> 'tuple[str, ...]'
```

Definition. Return every supported public format name.

Contract (D, C, H, R, X). Writers resolve one of the 11 public formats, enforce parent-directory and overwrite policy, localize figure files when needed, and install primary outputs through temporary files. YAML requires PyYAML. PDF export requires a supported local LaTeX engine. NPY and NPZ writes disable pickle. Multi-format export uses deterministic suffixes, while bundle export stages files, records SHA-256 values, normalizes ZIP timestamps, and adds `MANIFEST.sha256`.

Guarantees (E, P, B, K). A valid result certificate establishes that the primary output exists, is nonempty, and matches the observed byte count. Companion paths record localized assets, generated TeX, or compiler logs. A bundle manifest supports later integrity replay, but it is not a keyed signature and does not authenticate the archive against replacement of both data and manifest. Writer certificates do not certify the mathematical truth of rendered content.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_export_files.py` and `tests/test_export_cli.py`. The independent verifier for generating, signal, and export facades checks root-to-domain object identity, reflected signatures, implementation ownership, and representative CPU-safe behavior for the documented facade families. The source validator for generating, signal, and export facades checks exact card order, record counts, unique anchors, and preservation of the reference cards, technical blocks, and formulas. See Chapter 60.

C.1F · document_from_data

Identity (q, a, t, k). Public research-manifest route `komorebi_math.document_from_data`. Reflected implementation `komorebi_math.export.report.document_from_data`. The same runtime object is also available through `komorebi_math.export.document_from_data`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1F-EXPORT`.

Manifest signature (s).

```
(value: 'Any', *, title: 'str' = 'Komorebi Math Export', author: 'str | None' = None, date: 'str | None' = None,
    ↪ abstract: 'str | None' = None, include_certificate: 'bool' = True) -> 'MathDocument'
```

Definition. Create a readable mathematical document from common Python data.

Contract (D, C, H, R, X). Report builders convert certificates, finite sequences, and common Python data into the typed document model. Existing `MathDocument` values pass through unchanged. Result records may contribute their certificate when `include_certificate` is enabled. Mappings, short arrays, sequences, and dataclasses are converted to tables when possible, while unsupported table shapes fall back to normalized JSON code.

Guarantees (E, P, B, K). The builders preserve visible validity, exactness, tolerances, residuals, assumptions, and diagnostics when those fields exist. They produce readable reports rather than portable reconstruction records. A report can summarize evidence, but creating the report does not independently validate the source object or certificate.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_export_renderers.py` and `tests/test_export_files.py`. The independent verifier for generating, signal, and export facades checks root-to-domain object identity, reflected signatures, implementation ownership, and representative CPU-safe behavior for the documented facade families. The source validator for generating, signal, and export facades checks exact card order, record counts, unique anchors, and preservation of the reference cards, technical blocks, and formulas. See Chapter 59.

C.1F · export_bundle

Identity (q, a, t, k). Public research-manifest route `komorebi_math.export_bundle`. Reflected implementation `komorebi_math.export.writers.export_bundle`. The same runtime object is also available through `komorebi_math.export.export_bundle`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1F-EXPORT`.

Manifest signature (s).

```
(value: 'Any', path: 'str | Path', *, base_name: 'str' = 'komorebi_export', formats: 'Sequence[ExportFormat | str]'
    ↪ = (<ExportFormat.JSON: 'json'>, <ExportFormat.MARKDOWN: 'markdown'>, <ExportFormat.LATEX: 'latex'>),
    ↪ options: 'ExportOptions | None' = None) -> 'ExportBundleResult'
```

Definition. Create a ZIP archive with several exports and a SHA256 manifest.

Contract (D, C, H, R, X). Writers resolve one of the 11 public formats, enforce parent-directory and overwrite policy, localize figure files when needed, and install primary outputs through temporary files. YAML requires PyYAML. PDF export requires a supported local LaTeX engine. NPY and NPZ writes disable pickle. Multi-format export uses deterministic suffixes, while bundle export stages files, records SHA-256 values, normalizes ZIP timestamps, and adds `MANIFEST.sha256`.

Guarantees (E, P, B, K). A valid result certificate establishes that the primary output exists, is nonempty, and matches the observed byte count. Companion paths record localized assets, generated TeX, or compiler logs. A bundle manifest supports later integrity replay, but it is not a keyed signature and does not authenticate the archive against replacement of both data and manifest. Writer certificates do not certify the mathematical truth of rendered content.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_export_files.py` and `tests/test_export_cli.py`. The independent verifier for generating, signal, and export facades checks root-to-domain object identity, reflected signatures, implementation ownership, and representative CPU-safe behavior for the documented facade families. The source validator for

generating, signal, and export facades checks exact card order, record counts, unique anchors, and preservation of the reference cards, technical blocks, and formulas. See Chapter 60.

C.1F · export_data

Identity (q, a, t, k). Public research-manifest route `komorebi_math.export_data`. Reflected implementation `komorebi_math.export.writers.export_data`. The same runtime object is also available through `komorebi_math.export.export_data`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1F-EXPORT`.

Manifest signature (s).

```
(value: 'Any', path: 'str | Path', *, format: 'ExportFormat | str | None' = None, options: 'ExportOptions | None' =
    ↪ None) -> 'ExportResult'
```

Definition. Export one Python or Komorebi Math value to a selected file format.

Contract (D, C, H, R, X). Writers resolve one of the 11 public formats, enforce parent-directory and overwrite policy, localize figure files when needed, and install primary outputs through temporary files. YAML requires PyYAML. PDF export requires a supported local LaTeX engine. NPY and NPZ writes disable pickle. Multi-format export uses deterministic suffixes, while bundle export stages files, records SHA-256 values, normalizes ZIP timestamps, and adds `MANIFEST.sha256`.

Guarantees (E, P, B, K). A valid result certificate establishes that the primary output exists, is nonempty, and matches the observed byte count. Companion paths record localized assets, generated TeX, or compiler logs. A bundle manifest supports later integrity replay, but it is not a keyed signature and does not authenticate the archive against replacement of both data and manifest. Writer certificates do not certify the mathematical truth of rendered content.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_export_files.py` and `tests/test_export_cli.py`. The independent verifier for generating, signal, and export facades checks root-to-domain object identity, reflected signatures, implementation ownership, and representative CPU-safe behavior for the documented facade families. The source validator for generating, signal, and export facades checks exact card order, record counts, unique anchors, and preservation of the reference cards, technical blocks, and formulas. See Chapter 60.

C.1F · export_document

Identity (q, a, t, k). Public research-manifest route `komorebi_math.export_document`. Reflected implementation `komorebi_math.export.writers.export_document`. The same runtime object is also available through `komorebi_math.export.export_document`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1F-EXPORT`.

Manifest signature (s).

```
(document: 'MathDocument', path: 'str | Path', *, format: 'ExportFormat | str | None' = None, options:
    ↪ 'ExportOptions | None' = None) -> 'ExportResult'
```

Definition. Export one renderer independent mathematical document.

Contract (D, C, H, R, X). Writers resolve one of the 11 public formats, enforce parent-directory and overwrite policy, localize figure files when needed, and install primary outputs through temporary files. YAML requires PyYAML. PDF export requires a supported local LaTeX engine. NPY and NPZ writes disable pickle. Multi-format export uses deterministic suffixes, while bundle export stages files, records SHA-256 values, normalizes ZIP timestamps, and adds `MANIFEST.sha256`.

Guarantees (E, P, B, K). A valid result certificate establishes that the primary output exists, is nonempty, and matches the observed byte count. Companion paths record localized assets, generated TeX, or compiler logs. A bundle manifest supports later integrity replay, but it is not a keyed signature and does not authenticate the archive against replacement of both data and manifest. Writer certificates do not certify the mathematical truth of rendered content.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_export_files.py` and `tests/test_export_cli.py`. The independent verifier for

generating, signal, and export facades checks root-to-domain object identity, reflected signatures, implementation ownership, and representative CPU-safe behavior for the documented facade families. The source validator for generating, signal, and export facades checks exact card order, record counts, unique anchors, and preservation of the reference cards, technical blocks, and formulas. See Chapter 60.

C.1F · export_many

Identity (q, a, t, k). Public research-manifest route `komorebi_math.export_many`. Reflected implementation `komorebi_math.export.writers.export_many`. The same runtime object is also available through `komorebi_math.export.export_many`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1F-EXPORT`.

Manifest signature (s).

```
(value: 'Any', directory: 'str | Path', *, base_name: 'str', formats: 'Sequence[ExportFormat | str]', options:
    ↪  'ExportOptions | None' = None) -> 'tuple[ExportResult, ...]'
```

Definition. Export one value into several formats inside one directory.

Contract (D, C, H, R, X). Writers resolve one of the 11 public formats, enforce parent-directory and overwrite policy, localize figure files when needed, and install primary outputs through temporary files. YAML requires PyYAML. PDF export requires a supported local LaTeX engine. NPY and NPZ writes disable pickle. Multi-format export uses deterministic suffixes, while bundle export stages files, records SHA-256 values, normalizes ZIP timestamps, and adds `MANIFEST.sha256`.

Guarantees (E, P, B, K). A valid result certificate establishes that the primary output exists, is nonempty, and matches the observed byte count. Companion paths record localized assets, generated TeX, or compiler logs. A bundle manifest supports later integrity replay, but it is not a keyed signature and does not authenticate the archive against replacement of both data and manifest. Writer certificates do not certify the mathematical truth of rendered content.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_export_files.py` and `tests/test_export_cli.py`. The independent verifier for generating, signal, and export facades checks root-to-domain object identity, reflected signatures, implementation ownership, and representative CPU-safe behavior for the documented facade families. The source validator for generating, signal, and export facades checks exact card order, record counts, unique anchors, and preservation of the reference cards, technical blocks, and formulas. See Chapter 60.

C.1F · sequence_document

Identity (q, a, t, k). Public research-manifest route `komorebi_math.sequence_document`. Reflected implementation `komorebi_math.export.report.sequence_document`. The same runtime object is also available through `komorebi_math.export.sequence_document`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. This facade does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1F-EXPORT`.

Manifest signature (s).

```
(name: 'str', values: 'Sequence[Any]', *, strategy: 'str | None' = None, modulus: 'int | None' = None, author: 'str
    ↪  | None' = None) -> 'MathDocument'
```

Definition. Build a table report for one executable sequence.

Contract (D, C, H, R, X). Report builders convert certificates, finite sequences, and common Python data into the typed document model. Existing `MathDocument` values pass through unchanged. Result records may contribute their certificate when `include_certificate` is enabled. Mappings, short arrays, sequences, and dataclasses are converted to tables when possible, while unsupported table shapes fall back to normalized JSON code.

Guarantees (E, P, B, K). The builders preserve visible validity, exactness, tolerances, residuals, assumptions, and diagnostics when those fields exist. They produce readable reports rather than portable reconstruction records. A report can summarize evidence, but creating the report does not independently validate the source object or certificate.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root facade. The focused test modules are `tests/test_export_renderers.py` and `tests/test_export_files.py`. The independent verifier for generating, signal, and export facades checks root-to-domain object identity, reflected signatures, implementation ownership, and representative CPU-safe behavior for the documented facade families. The source validator for generating, signal, and export facades checks exact card order, record counts, unique anchors, and preservation of the reference cards, technical blocks, and formulas. See Chapter 59.

C.1G Root namespace closure: 106 entries

This root section documents every research export not covered by the six preceding root sections. It closes the arithmetic, sequence, analytic, refinement, quantum, dynamics, deformation, Langlands, registry, unified workflow, and package-data facades.

Contract family	Cards
Arithmetic and sequence systems	21
Analytic and refinement systems	23
Quantum and dynamical systems	19
Deformation and Langlands systems	34
Registry, unified workflow, and root data	9
Total	**106**

The root names remain aliases or direct exports of the implementation objects recorded in the committed research manifest. Nested Langlands objects are intentionally exported at the package root even when the umbrella `komorebi_math.langlands` module does not repeat the alias. The finite Langlands records and completion gates remain conditional, executable models. They do not assert a proof of the unrestricted global correspondence.

Arithmetic and sequence systems (21 cards).

C.1G · `AutomaticSequence`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.AutomaticSequence`. Reflected implementation `komorebi_math.sequences.automatic.AutomaticSequence`. The same runtime object is also available through `komorebi_math.sequences.AutomaticSequence`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1G-SEQUENCES`.

Manifest signature (s).

```
(base: 'int', transitions: 'Iterable[Iterable[int]]', outputs: 'Iterable[Number]', *, initial_state: 'int' = 0,
   ↪ name: 'str' = 'automatic sequence') -> 'None'
```

Definition. A deterministic finite automaton with numeric output.

Contract (D, C, H, R, X). Sequence indices are nonnegative. Automata must have a finite deterministic transition table, one output per state, and a valid initial state. Holonomic and compiled models require enough initial data to determine every requested term. Public fields are `base`, `transitions`, `outputs`, `initial_state`, `name`. Principal public operations are `state_count`, `state`, `term`, `first`, `residue_model`, `to_sequence`, `certificate`.

Guarantees (E, P, B, K). A fixed model returns deterministic terms and certificates. Lazy caches may change memory use but not the mathematical value. Automatic models scale with the base-digit length, while recurrence and holonomic models scale with the requested index and order.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_sequences_models.py`, `tests/test_sequences_execution.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapter 26.

C.1G · CompiledSequence

Identity (q, a, t, k). Public research-manifest route `komorebi_math.CompiledSequence`. Reflected implementation `komorebi_math.sequences.compiler.CompiledSequence`. The same runtime object is also available through `komorebi_math.sequences.CompiledSequence`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1G-SEQUENCES`.

Manifest signature (s).

```
(sequence: 'Sequence', strategy: 'str', certificate: 'Certificate', recurrence: 'LinearRecurrence | None' = None,
    ↪ automatic: 'AutomaticSequence | None' = None, holonomic: 'HolonomicRecurrence | None' = None) -> None
```

Definition. An executable sequence strategy with an explicit validation certificate.

Contract (D, C, H, R, X). Sequence indices are nonnegative. Automata must have a finite deterministic transition table, one output per state, and a valid initial state. Holonomic and compiled models require enough initial data to determine every requested term. Public fields are `sequence`, `strategy`, `certificate`, `recurrence`, `automatic`, `holonomic`. Principal public operations are `term`, `first`, `modulo`, `generating_function`.

Guarantees (E, P, B, K). A fixed model returns deterministic terms and certificates. Lazy caches may change memory use but not the mathematical value. Automatic models scale with the base-digit length, while recurrence and holonomic models scale with the requested index and order.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_sequences_models.py`, `tests/test_sequences_execution.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapter 26.

C.1G · EulerLocalCoordinates

Identity (q, a, t, k). Public research-manifest route `komorebi_math.EulerLocalCoordinates`. Reflected implementation `komorebi_math.arithmetic.euler.EulerLocalCoordinates`. The same runtime object is also available through `komorebi_math.arithmetic.EulerLocalCoordinates`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1G-ARITHMETIC`.

Manifest signature (s).

```
(prime: 'int') -> None
```

Definition. Exact odd or dyadic local coordinates for signed Euler numbers.

Contract (D, C, H, R, X). Indices, orders, precisions, and moduli must satisfy the bounds stated by the signature. Recurrence coefficients and initial values must have compatible lengths. Local and p-adic operations validate the prime, truncation depth, and denominator conditions before exact expansion. Public fields are `prime`. Principal public operations are `is_dyadic`, `source`, `kernel`, `inverse_kernel`, `reconstruct`, `inverse_reconstruct_source`, `kernel_valuation`, `row_weight`, `formula_row_weight`, `sharp_modulus`, `truncate`, `certificate`.

Guarantees (E, P, B, K). Integer, rational, recurrence, and truncated p-adic calculations are exact within the declared finite order or precision. Coefficient size and execution cost can grow quickly with degree and p-adic depth.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_arithmetic_euler.py`, `tests/test_arithmetic_domains_valuation.py`, `tests/test_arithmetic_hurwitz_windows.py`, `tests/test_arithmetic_recurrence.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 25 and 64.

C.1G · HolonomicRecurrence

Identity (q, a, t, k). Public research-manifest route `komorebi_math.HolonomicRecurrence`. Reflected implementation `komorebi_math.sequences.holonomic.HolonomicRecurrence`. The same runtime object is also available through `komorebi_math.sequences.HolonomicRecurrence`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1G-SEQUENCES`.

Manifest signature (s).

```
(polynomials: 'Iterable[Iterable[Rational]]', initial: 'Iterable[Rational]', *, name: 'str' = 'holonomic
   ↪ recurrence') -> 'None'
```

Definition. An exact recurrence `sum_j P_j(n) a[n+j] = 0`.

Contract (D, C, H, R, X). Sequence indices are nonnegative. Automata must have a finite deterministic transition table, one output per state, and a valid initial state. Holonomic and compiled models require enough initial data to determine every requested term. Public fields are `polynomials`, `initial`, `name`. Principal public operations are `order`, `coefficient`, `term`, `first`, `to_sequence`, `certificate`.

Guarantees (E, P, B, K). A fixed model returns deterministic terms and certificates. Lazy caches may change memory use but not the mathematical value. Automatic models scale with the base-digit length, while recurrence and holonomic models scale with the requested index and order.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_sequences_models.py`, `tests/test_sequences_execution.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapter 26.

C.1G · HurwitzSeries

Identity (q, a, t, k). Public research-manifest route `komorebi_math.HurwitzSeries`. Reflected implementation `komorebi_math.arithmetic.hurwitz.HurwitzSeries`. The same runtime object is also available through `komorebi_math.arithmetic.HurwitzSeries`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1G-ARITHMETIC`.

Manifest signature (s).

```
(coefficients: 'Iterable[Number]', modulus: 'int | None' = None) -> 'None'
```

Definition. A finite exact or modular Hurwitz coefficient system.

Contract (D, C, H, R, X). Indices, orders, precisions, and moduli must satisfy the bounds stated by the signature. Recurrence coefficients and initial values must have compatible lengths. Local and p-adic operations validate the prime, truncation depth, and denominator conditions before exact expansion. Public fields are `coefficients`, `modulus`. Principal public operations are `degree`, `coefficient`, `truncate`, `multiply`, `inverse`, `evaluate_egf`, `identity`, `exponential_mode`.

Guarantees (E, P, B, K). Integer, rational, recurrence, and truncated p-adic calculations are exact within the declared finite order or precision. Coefficient size and execution cost can grow quickly with degree and p-adic depth.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_arithmetic_hurwitz_windows.py`, `tests/test_arithmetic_domains_valuation.py`, `tests/test_arithmetic_recurrence.py`, `tests/test_arithmetic_euler.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 25 and 64.

C.1G · LinearRecurrence

Identity (q, a, t, k). Public research-manifest route `komorebi_math.LinearRecurrence`. Reflected implementation `komorebi_math.arithmetic.recurrence.LinearRecurrence`. The same runtime object is also available through `komorebi_math.arithmetic.LinearRecurrence`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1G-ARITHMETIC`.

Manifest signature (s).

```
(coefficients: 'Iterable[Number]', initial: 'Iterable[Number]', *, name: 'str | None' = None) -> 'None'
```

Definition. An exact constant coefficient recurrence.

Contract (D, C, H, R, X). Indices, orders, precisions, and moduli must satisfy the bounds stated by the signature. Recurrence coefficients and initial values must have compatible lengths. Local and p-adic operations validate the prime, truncation depth, and denominator conditions before exact expansion. Principal public operations are `order`, `companion_matrix`, `initial_state`, `term`, `term_mod`, `first`, `state`, `jump_matrix`, `residue_packet`, `period_mod`, `characteristic_polynomial`, `jump_characteristic_polynomial`, `certificate`.

Guarantees (E, P, B, K). Integer, rational, recurrence, and truncated p-adic calculations are exact within the declared finite order or precision. Coefficient size and execution cost can grow quickly with degree and p-adic depth.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_arithmetic_recurrence.py`, `tests/test_sequences_execution.py`, `tests/test_arithmetic_domains_valuation.py`, `tests/test_arithmetic_hurwitz_windows.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 25 and 64.

C.1G · MahlerSeries

Identity (q, a, t, k). Public research-manifest route `komorebi_math.MahlerSeries`. Reflected implementation `komorebi_math.arithmetic.mahler.MahlerSeries`. The same runtime object is also available through `komorebi_math.arithmetic.MahlerSeries`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1G-ARITHMETIC`.

Manifest signature (s).

```
(coefficients: 'Iterable[Number]') -> 'None'
```

Definition. A finite expansion in the binomial polynomial basis.

Contract (D, C, H, R, X). Indices, orders, precisions, and moduli must satisfy the bounds stated by the signature. Recurrence coefficients and initial values must have compatible lengths. Local and p-adic operations validate the prime, truncation depth, and denominator conditions before exact expansion. Public fields are `coefficients`. Principal public operations are `degree`, `from_values`, `evaluate`, `first`, `difference`, `affine_subsequence`, `valuation_ledger`, `reconstruction_certificate`.

Guarantees (E, P, B, K). Integer, rational, recurrence, and truncated p-adic calculations are exact within the declared finite order or precision. Coefficient size and execution cost can grow quickly with degree and p-adic depth.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_arithmetic_mahler_polyphase.py`, `tests/test_arithmetic_domains_valuation.py`, `tests/test_arithmetic_hurwitz_windows.py`, `tests/test_arithmetic_recurrence.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 25 and 64.

C.1G · Sequence

Identity (q, a, t, k). Public research-manifest route `komorebi_math.Sequence`. Reflected implementation `komorebi_math.sequences.base.Sequence`. The same runtime object is also available through `komorebi_math.sequences.Sequence`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1G-SEQUENCES`.

Manifest signature (s).

```
(evaluator: 'Callable[[int], Number]', *, name: 'str' = 'sequence', exact: 'bool' = True, metadata: 'dict[str, Any]
   ↪  | None' = None) -> 'None'
```

Definition. A cached lazy sequence indexed by nonnegative integers.

Contract (D, C, H, R, X). Sequence indices are nonnegative. Automata must have a finite deterministic transition table, one output per state, and a valid initial state. Holonomic and compiled models require enough initial data to determine every requested term. Principal public operations are `term`, `first`, `residue`, `modulo`, `map`.

Guarantees (E, P, B, K). A fixed model returns deterministic terms and certificates. Lazy caches may change memory use but not the mathematical value. Automatic models scale with the base-digit length, while recurrence and holonomic models scale with the requested index and order.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_sequences_execution.py`, `tests/test_sequences_models.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapter 26.

C.1G · StandardWindow

Identity (q, a, t, k). Public research-manifest route `komorebi_math.StandardWindow`. Reflected implementation `komorebi_math.arithmetic.standard_windows.StandardWindow`. The same runtime object is also available through `komorebi_math.arithmetic.StandardWindow`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1G-ARITHMETIC`.

Manifest signature (s).

```
(prime: 'int', compression: 'int') -> None
```

Definition. A congruential inverse window with exact rational and residual coordinates.

Contract (D, C, H, R, X). Indices, orders, precisions, and moduli must satisfy the bounds stated by the signature. Recurrence coefficients and initial values must have compatible lengths. Local and p-adic operations validate the prime, truncation depth, and denominator conditions before exact expansion. Public fields are `prime`, `compression`. Principal public operations are `order`, `has_uniform_period`, `period`, `reduced_inverse`, `exact_coefficients`, `residues`, `rational_generating_function`, `minimum_period`, `certificate`.

Guarantees (E, P, B, K). Integer, rational, recurrence, and truncated p-adic calculations are exact within the declared finite order or precision. Coefficient size and execution cost can grow quickly with degree and p-adic depth.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_arithmetic_hurwitz_windows.py`, `tests/test_arithmetic_domains_valuation.py`, `tests/test_arithmetic_recurrence.py`, `tests/test_arithmetic_euler.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 25 and 64.

C.1G · `TruncatedPadic`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.TruncatedPadic`. Reflected implementation `komorebi_math.arithmetic.domains.TruncatedPadic`. The same runtime object is also available through `komorebi_math.arithmetic.TruncatedPadic`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1G-ARITHMETIC`.

Manifest signature (`s`).

```
(prime: 'int', precision: 'int') -> None
```

Definition. Arithmetic modulo `prime**precision` with explicit local precision.

Contract (`D, C, H, R, X`). Indices, orders, precisions, and moduli must satisfy the bounds stated by the signature. Recurrence coefficients and initial values must have compatible lengths. Local and p-adic operations validate the prime, truncation depth, and denominator conditions before exact expansion. Public fields are `prime`, `precision`. Principal public operations are `modulus`, `ring`, `normalize`, `is_unit`, `inverse`, `valuation`, `refine`, `coarsen`.

Guarantees (`E, P, B, K`). Integer, rational, recurrence, and truncated p-adic calculations are exact within the declared finite order or precision. Coefficient size and execution cost can grow quickly with degree and p-adic depth.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_arithmetic_domains_valuation.py`, `tests/test_arithmetic_hurwitz_windows.py`, `tests/test_arithmetic_recurrence.py`, `tests/test_arithmetic_euler.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 25 and 64.

C.1G · `affine_mahler_kernel`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.affine_mahler_kernel`. Reflected implementation `komorebi_math.arithmetic.mahler.affine_mahler_kernel`. The same runtime object is also available through `komorebi_math.arithmetic.affine_mahler_kernel`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1G-ARITHMETIC`.

Manifest signature (`s`).

```
(multiplier: 'int', shift: 'int', source_degree: 'int', target_degree: 'int') -> 'int'
```

Definition. Return the exact affine Mahler transfer kernel.

Contract (`D, C, H, R, X`). Indices, orders, precisions, and moduli must satisfy the bounds stated by the signature. Recurrence coefficients and initial values must have compatible lengths. Local and p-adic operations validate the prime, truncation depth, and denominator conditions before exact expansion.

Guarantees (`E, P, B, K`). Integer, rational, recurrence, and truncated p-adic calculations are exact within the declared finite order or precision. Coefficient size and execution cost can grow quickly with degree and p-adic depth.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_arithmetic_mahler_polyphase.py`, `tests/test_arithmetic_domains_valuation.py`, `tests/test_arithmetic_hurwitz_windows.py`, `tests/test_arithmetic_recurrence.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 25 and 64.

C.1G · apery_sequence

Identity (q, a, t, k). Public research-manifest route `komorebi_math.apery_sequence`. Reflected implementation `komorebi_math.sequences.catalog.apery_sequence`. The same runtime object is also available through `komorebi_math.sequences.apery_sequence`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1G-SEQUENCES`.

Manifest signature (s).

```
() -> 'Sequence'
```

Definition. Return the Apéry numbers for `zeta(3)` through their exact recurrence.

Contract (D, C, H, R, X). Sequence indices are nonnegative. Automata must have a finite deterministic transition table, one output per state, and a valid initial state. Holonomic and compiled models require enough initial data to determine every requested term.

Guarantees (E, P, B, K). A fixed model returns deterministic terms and certificates. Lazy caches may change memory use but not the mathematical value. Automatic models scale with the base-digit length, while recurrence and holonomic models scale with the requested index and order.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_sequences_execution.py`, `tests/test_sequences_models.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapter 26.

C.1G · compile_sequence

Identity (q, a, t, k). Public research-manifest route `komorebi_math.compile_sequence`. Reflected implementation `komorebi_math.sequences.compiler.compile_sequence`. The same runtime object is also available through `komorebi_math.sequences.compile_sequence`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1G-SEQUENCES`.

Manifest signature (s).

```
(sequence: 'Sequence', *, sample_count: 'int' = 24, max_order: 'int | None' = 8) -> 'CompiledSequence'
```

Definition. Compile a sequence to a declared or detected finite execution model.

Contract (D, C, H, R, X). Sequence indices are nonnegative. Automata must have a finite deterministic transition table, one output per state, and a valid initial state. Holonomic and compiled models require enough initial data to determine every requested term.

Guarantees (E, P, B, K). A fixed model returns deterministic terms and certificates. Lazy caches may change memory use but not the mathematical value. Automatic models scale with the base-digit length, while recurrence and holonomic models scale with the requested index and order.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_sequences_execution.py`, `tests/test_sequences_models.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapter 26.

C.1G · fibonacci_recurrence

Identity (q, a, t, k). Public research-manifest route `komorebi_math.fibonacci_recurrence`. Reflected implementation `komorebi_math.arithmetic.recurrence.fibonacci_recurrence`. The same runtime object is also available through `komorebi_math.arithmetic.fibonacci_recurrence`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1G-ARITHMETIC`.

Manifest signature (`s`).

```
() -> 'LinearRecurrence'
```

Definition. Return the Fibonacci recurrence.

Contract (`D, C, H, R, X`). Indices, orders, precisions, and moduli must satisfy the bounds stated by the signature. Recurrence coefficients and initial values must have compatible lengths. Local and p-adic operations validate the prime, truncation depth, and denominator conditions before exact expansion.

Guarantees (`E, P, B, K`). Integer, rational, recurrence, and truncated p-adic calculations are exact within the declared finite order or precision. Coefficient size and execution cost can grow quickly with degree and p-adic depth.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_arithmetic_recurrence.py`, `tests/test_arithmetic_domains_valuation.py`, `tests/test_arithmetic_hurwitz_windows.py`, `tests/test_arithmetic_euler.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 25 and 64.

C.1G · `fibonacci_sequence`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.fibonacci_sequence`. Reflected implementation `komorebi_math.sequences.catalog.fibonacci_sequence`. The same runtime object is also available through `komorebi_math.sequences.fibonacci_sequence`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1G-SEQUENCES`.

Manifest signature (`s`).

```
() -> 'Sequence'
```

Definition. Return the Fibonacci sequence.

Contract (`D, C, H, R, X`). Sequence indices are nonnegative. Automata must have a finite deterministic transition table, one output per state, and a valid initial state. Holonomic and compiled models require enough initial data to determine every requested term.

Guarantees (`E, P, B, K`). A fixed model returns deterministic terms and certificates. Lazy caches may change memory use but not the mathematical value. Automatic models scale with the base-digit length, while recurrence and holonomic models scale with the requested index and order.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_sequences_execution.py`, `tests/test_sequences_models.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapter 26.

C.1G · `get_sequence`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.get_sequence`. Reflected implementation `komorebi_math.sequences.catalog.get_sequence`. The same runtime object is also available through `komorebi_math.sequences.get_sequence`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1G-SEQUENCES`.

Manifest signature (`s`).

```
(name: 'str') -> 'Sequence'
```

Definition. Return one built in sequence by a normalized name.

Contract (`D, C, H, R, X`). Sequence indices are nonnegative. Automata must have a finite deterministic transition table, one output per state, and a valid initial state. Holonomic and compiled models require enough initial data to determine every requested term.

Guarantees (`E, P, B, K`). A fixed model returns deterministic terms and certificates. Lazy caches may change memory use but not the mathematical value. Automatic models scale with the base-digit length, while recurrence and holonomic models scale with the requested index and order.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_sequences_execution.py`, `tests/test_sequences_models.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapter 26.

C.1G · `motzkin_sequence`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.motzkin_sequence`. Reflected implementation `komorebi_math.sequences.catalog.motzkin_sequence`. The same runtime object is also available through `komorebi_math.sequences.motzkin_sequence`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1G-SEQUENCES`.

Manifest signature (`s`).

```
() -> 'Sequence'
```

Definition. Return Motzkin numbers through an exact holonomic recurrence.

Contract (`D, C, H, R, X`). Sequence indices are nonnegative. Automata must have a finite deterministic transition table, one output per state, and a valid initial state. Holonomic and compiled models require enough initial data to determine every requested term.

Guarantees (`E, P, B, K`). A fixed model returns deterministic terms and certificates. Lazy caches may change memory use but not the mathematical value. Automatic models scale with the base-digit length, while recurrence and holonomic models scale with the requested index and order.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_sequences_execution.py`, `tests/test_sequences_models.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapter 26.

C.1G · `padic_valuation`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.padic_valuation`. Reflected implementation `komorebi_math.arithmetic.valuation.padic_valuation`. The same runtime object is also available through `komorebi_math.arithmetic.padic_valuation`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1G-ARITHMETIC`.

Manifest signature (s).

```
(value: 'Rational', prime: 'int') -> 'int | float'
```

Definition. Return the exact `prime` adic valuation of an integer or rational.

Contract (D, C, H, R, X). Indices, orders, precisions, and moduli must satisfy the bounds stated by the signature. Recurrence coefficients and initial values must have compatible lengths. Local and p-adic operations validate the prime, truncation depth, and denominator conditions before exact expansion.

Guarantees (E, P, B, K). Integer, rational, recurrence, and truncated p-adic calculations are exact within the declared finite order or precision. Coefficient size and execution cost can grow quickly with degree and p-adic depth.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_arithmetic_domains_valuation.py`, `tests/test_arithmetic_euler.py`, `tests/test_lattice_smith.py`, `tests/test_arithmetic_hurwitz_windows.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 25 and 64.

C.1G · `partition_sequence`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.partition_sequence`. Reflected implementation `komorebi_math.sequences.catalog.partition_sequence`. The same runtime object is also available through `komorebi_math.sequences.partition_sequence`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1G-SEQUENCES`.

Manifest signature (s).

```
() -> 'Sequence'
```

Definition. Return integer partition numbers.

Contract (D, C, H, R, X). Sequence indices are nonnegative. Automata must have a finite deterministic transition table, one output per state, and a valid initial state. Holonomic and compiled models require enough initial data to determine every requested term.

Guarantees (E, P, B, K). A fixed model returns deterministic terms and certificates. Lazy caches may change memory use but not the mathematical value. Automatic models scale with the base-digit length, while recurrence and holonomic models scale with the requested index and order.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_sequences_execution.py`, `tests/test_sequences_models.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapter 26.

C.1G · `signed_euler_number`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signed_euler_number`. Reflected implementation `komorebi_math.arithmetic.euler.signed_euler_number`. The same runtime object is also available through `komorebi_math.arithmetic.signed_euler_number`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1G-ARITHMETIC`.

Manifest signature (s).

```
(index: 'int') -> 'int'
```

Definition. Return one signed Euler number from the `sech` convention.

Contract (D, C, H, R, X). Indices, orders, precisions, and moduli must satisfy the bounds stated by the signature. Recurrence coefficients and initial values must have compatible lengths. Local and p-adic operations validate the prime, truncation depth, and denominator conditions before exact expansion.

Guarantees (E, P, B, K). Integer, rational, recurrence, and truncated p-adic calculations are exact within the declared finite order or precision. Coefficient size and execution cost can grow quickly with degree and p-adic depth.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_arithmetic_domains_valuation.py`, `tests/test_arithmetic_hurwitz_windows.py`, `tests/test_arithmetic_recurrence.py`, `tests/test_arithmetic_euler.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 25 and 64.

C.1G · `thue_morse_sequence`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.thue_morse_sequence`. Reflected implementation `komorebi_math.sequences.catalog.thue_morse_sequence`. The same runtime object is also available through `komorebi_math.sequences.thue_morse_sequence`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1G-SEQUENCES`.

Manifest signature (s).

```
() -> 'Sequence'
```

Definition. Return the zero one Thue Morse automatic sequence.

Contract (D, C, H, R, X). Sequence indices are nonnegative. Automata must have a finite deterministic transition table, one output per state, and a valid initial state. Holonomic and compiled models require enough initial data to determine every requested term.

Guarantees (E, P, B, K). A fixed model returns deterministic terms and certificates. Lazy caches may change memory use but not the mathematical value. Automatic models scale with the base-digit length, while recurrence and holonomic models scale with the requested index and order.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_sequences_execution.py`, `tests/test_sequences_models.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapter 26.

Analytic and refinement systems (23 cards).

C.1G · `CyclicDenominator`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.CyclicDenominator`. Reflected implementation `komorebi_math.analytic.zeros.CyclicDenominator`. The same runtime object is also available through `komorebi_math.analytic.CyclicDenominator`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1G-ANALYTIC`.

Manifest signature (s).

```
(order: 'int') -> None
```

Definition. Executable distinguished cyclic denominator family.

Contract (D, C, H, R, X). Orders and truncation degrees are nonnegative. Denominators and section parameters must avoid their declared singular cases. Numerical zero and tangent diagnostics require finite search windows and a positive tolerance. Public fields are `order`. Principal public operations are `derivative`, `positive_zeros`, `zero_count`.

Guarantees (E, P, B, K). Coefficient transforms are exact when their inputs are exact. Numerical root and tangent records certify only the finite interval, truncation, and tolerance that were actually evaluated. They do not establish an unrestricted analytic theorem.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_analytic_appell_zeros.py`, `tests/test_analytic_cyclic_reciprocal.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapter 27.

C.1G · `CyclicSectionFamily`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.CyclicSectionFamily`. Reflected implementation `komorebi_math.analytic.sections.CyclicSectionFamily`. The same runtime object is also available through `komorebi_math.analytic.CyclicSectionFamily`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1G-ANALYTIC`.

Manifest signature (s).

```
(order: 'int', family: "Literal['transfer', 'signed']" = 'signed', scale: 'complex' = 1.0) -> None
```

Definition. One transfer or signed cyclic differential family.

Contract (D, C, H, R, X). Orders and truncation degrees are nonnegative. Denominators and section parameters must avoid their declared singular cases. Numerical zero and tangent diagnostics require finite search windows and a positive tolerance. Public fields are `order`, `family`, `scale`. Principal public operations are `evaluate`, `vector`, `fundamental_matrix`.

Guarantees (E, P, B, K). Coefficient transforms are exact when their inputs are exact. Numerical root and tangent records certify only the finite interval, truncation, and tolerance that were actually evaluated. They do not establish an unrestricted analytic theorem.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_analytic_cyclic_reciprocal.py`, `tests/test_analytic_appell_zeros.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapter 27.

C.1G · `LatticePacketData`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.LatticePacketData`. Reflected implementation `komorebi_math.refinement.lattices.LatticePacketData`. The same runtime object is also available through `komorebi_math.refinement.LatticePacketData`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1G-REFINEMENT`.

Manifest signature (s).

```
(lattice_basis: 'Any', projectors: 'Any', *, prime: 'int', split_basis: 'Any | None' = None) -> 'None'
```

Definition. A rational packet decomposition measured against one integral lattice.

Contract (D, C, H, R, X). Packet dimensions, lattice bases, transfer matrices, and residue systems must be compatible. CRT operations require valid moduli and compatible congruences. Prime-local records require a prime and finite precision. Principal public operations are `dimension`, `certificate`, `projector_in_lattice_basis`, `projector_weights`, `separation_weight`, `precision_delays`, `inclusion_matrix`, `gluing_module`, `gluing_weight_certificate`.

Guarantees (E, P, B, K). Rational, integral, and finite-field refinement data are computed exactly. Certificates apply to the declared finite matrices, lattices, residues, and tower depth. Matrix and Smith-form costs grow with dimension and coefficient size.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_lattice_smith.py`, `tests/test_linear_defects.py`, `tests/test_residue_crt.py`, `tests/test_refinement_towers.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 17 through 24.

C.1G · `LinearRefinementDefect`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.LinearRefinementDefect`. Reflected implementation `komorebi_math.refinement.defects.LinearRefinementDefect`. The same runtime object is also available through `komorebi_math.refinement.LinearRefinementDefect`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1G-REFINEMENT`.

Manifest signature (s).

```
(coarsen: 'Any', fine_operator: 'Any', coarse_operator: 'Any', backend: 'str | LinearAlgebraBackend | None' =
    ↪ None) -> None
```

Definition. The intertwining defect `C T_f - T_c C`.

Contract (D, C, H, R, X). Packet dimensions, lattice bases, transfer matrices, and residue systems must be compatible. CRT operations require valid moduli and compatible congruences. Prime-local records require a prime and finite precision. Public fields are `coarsen`, `fine_operator`, `coarse_operator`, `backend`. Principal public operations are `matrix`, `is_exact`, `power`, `polynomial`, `resolvent`, `exponential`.

Guarantees (E, P, B, K). Rational, integral, and finite-field refinement data are computed exactly. Certificates apply to the declared finite matrices, lattices, residues, and tower depth. Matrix and Smith-form costs grow with dimension and coefficient size.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_linear_defects.py`, `tests/test_lattice_smith.py`, `tests/test_residue_crt.py`, `tests/test_refinement_towers.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 17 through 24.

C.1G · `PacketRefinement`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.PacketRefinement`. Reflected implementation `komorebi_math.refinement.towers.PacketRefinement`. The same runtime object is also available through `komorebi_math.refinement.PacketRefinement`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1G-REFINEMENT`.

Manifest signature (`s`).

```
(coarse: 'PacketSystem', fine: 'PacketSystem', parent_of_fine: 'tuple[int, ...]') -> None
```

Definition. One exact refinement from a coarse packet system to a fine system.

Contract (`D, C, H, R, X`). Packet dimensions, lattice bases, transfer matrices, and residue systems must be compatible. CRT operations require valid moduli and compatible congruences. Prime-local records require a prime and finite precision. Public fields are `coarse`, `fine`, `parent_of_fine`. Principal public operations are `backend`, `children`, `parent`, `certificate`, `coarsen`, `refine`, `refine_operator`, `coarsen_operator`, `support_reflection`.

Guarantees (`E, P, B, K`). Rational, integral, and finite-field refinement data are computed exactly. Certificates apply to the declared finite matrices, lattices, residues, and tower depth. Matrix and Smith-form costs grow with dimension and coefficient size.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_refinement_towers.py`, `tests/test_linear_defects.py`, `tests/test_lattice_smith.py`, `tests/test_residue_crt.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 17 through 24.

C.1G · `RefinementTower`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.RefinementTower`. Reflected implementation `komorebi_math.refinement.towers.RefinementTower`. The same runtime object is also available through `komorebi_math.refinement.RefinementTower`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1G-REFINEMENT`.

Manifest signature (`s`).

```
(refinements: 'tuple[PacketRefinement, ...] | list[PacketRefinement]') -> 'None'
```

Definition. A finite sequence of exact packet refinements.

Contract (`D, C, H, R, X`). Packet dimensions, lattice bases, transfer matrices, and residue systems must be compatible. CRT operations require valid moduli and compatible congruences. Prime-local records require a prime and finite precision. Principal public operations are `levels`, `refine`, `coarsen`, `descendant_address`, `certificate`.

Guarantees (`E, P, B, K`). Rational, integral, and finite-field refinement data are computed exactly. Certificates apply to the declared finite matrices, lattices, residues, and tower depth. Matrix and Smith-form costs grow with dimension and coefficient size.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_refinement_towers.py`, `tests/test_linear_defects.py`, `tests/test_lattice_smith.py`, `tests/test_residue_crt.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 17 through 24.

C.1G · `SmithCongruenceProblem`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.SmithCongruenceProblem`. Reflected implementation `komorebi_math.refinement.smith.SmithCongruenceProblem`. The same runtime object is also available through `komorebi_math.refinement.SmithCongruenceProblem`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1G-REFINEMENT`.

Manifest signature (s).

```
(matrix: 'Any', rhs: 'Any', *, prime: 'int', depth: 'int') -> 'None'
```

Definition. Solve `matrix * x = rhs` modulo a prime power.

Contract (D, C, H, R, X). Packet dimensions, lattice bases, transfer matrices, and residue systems must be compatible. CRT operations require valid moduli and compatible congruences. Prime-local records require a prime and finite precision. Principal public operations are `solve`, `enumerate_solutions`.

Guarantees (E, P, B, K). Rational, integral, and finite-field refinement data are computed exactly. Certificates apply to the declared finite matrices, lattices, residues, and tower depth. Matrix and Smith-form costs grow with dimension and coefficient size.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_lattice_smith.py`, `tests/test_linear_defects.py`, `tests/test_residue_crt.py`, `tests/test_refinement_towers.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 17 through 24.

C.1G · `WeightedTransferEdge`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.WeightedTransferEdge`. Reflected implementation `komorebi_math.refinement.weighted.WeightedTransferEdge`. The same runtime object is also available through `komorebi_math.refinement.WeightedTransferEdge`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1G-REFINEMENT`.

Manifest signature (s).

```
(source: 'int', target: 'int', weight: 'int', operator: 'Any', name: 'str' = '') -> None
```

Definition. One packet supported transfer with a positive filtration weight.

Contract (D, C, H, R, X). Packet dimensions, lattice bases, transfer matrices, and residue systems must be compatible. CRT operations require valid moduli and compatible congruences. Prime-local records require a prime and finite precision. Public fields are `source`, `target`, `weight`, `operator`, `name`.

Guarantees (E, P, B, K). Rational, integral, and finite-field refinement data are computed exactly. Certificates apply to the declared finite matrices, lattices, residues, and tower depth. Matrix and Smith-form costs grow with dimension and coefficient size.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_weighted_transfer.py`, `tests/test_linear_defects.py`, `tests/test_lattice_smith.py`, `tests/test_residue_crt.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 17 through 24.

C.1G · `WeightedTransferGraph`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.WeightedTransferGraph`. Reflected implementation `komorebi_math.refinement.weighted.WeightedTransferGraph`. The same runtime object is also available through `komorebi_math.refinement.WeightedTransferGraph`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1G-REFINEMENT`.

Manifest signature (s).

```
(packet_system: 'PacketSystem', diagonal: 'Any', edges: 'tuple[WeightedTransferEdge, ...] |
    ↪ list[WeightedTransferEdge]', *, backend: 'str | LinearAlgebraBackend | None' = None, prime: 'int | None' =
    ↪ None, validate_weights: 'bool' = True) -> 'None'
```

Definition. A packet diagonal operator plus weighted directed transfer blocks.

Contract (D, C, H, R, X). Packet dimensions, lattice bases, transfer matrices, and residue systems must be compatible. CRT operations require valid moduli and compatible congruences. Prime-local records require a prime and finite precision. Principal public operations are `dimension`, `operator`, `diagonal_inverse`, `adjacency`, `paths_below_weight`, `minimum_path_weight`, `path_operator`, `truncated_inverse`, `exact_inverse_if_acyclic`, `is_acyclic`, `weighted_adjacency`, `transfer_series_matrix`.

Guarantees (E, P, B, K). Rational, integral, and finite-field refinement data are computed exactly. Certificates apply to the declared finite matrices, lattices, residues, and tower depth. Matrix and Smith-form costs grow with dimension and coefficient size.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_weighted_transfer.py`, `tests/test_linear_defects.py`, `tests/test_lattice_smith.py`, `tests/test_residue_crt.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 17 through 24.

C.1G · `appell_polynomial`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.appell_polynomial`. Reflected implementation `komorebi_math.analytic.appell.appell_polynomial`. The same runtime object is also available through `komorebi_math.analytic.appell_polynomial`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1G-ANALYTIC`.

Manifest signature (s).

```
(order: 'int', residue: 'int', parameter: 'Number', degree: 'int', x: 'Number') -> 'Number'
```

Definition. Evaluate one congruential Appell polynomial.

Contract (D, C, H, R, X). Orders and truncation degrees are nonnegative. Denominators and section parameters must avoid their declared singular cases. Numerical zero and tangent diagnostics require finite search windows and a positive tolerance.

Guarantees (E, P, B, K). Coefficient transforms are exact when their inputs are exact. Numerical root and tangent records certify only the finite interval, truncation, and tolerance that were actually evaluated. They do not establish an unrestricted analytic theorem.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_analytic_appell_zeros.py`, `tests/test_analytic_cyclic_reciprocal.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapter 27.

C.1G · `common_refinement`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.common_refinement`. Reflected implementation `komorebi_math.refinement.towers.common_refinement`. The same runtime object is also available through `komorebi_math.refinement.common_refinement`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1G-REFINEMENT`.

Manifest signature (`s`).

```
(left: 'PacketSystem', right: 'PacketSystem', *, names: 'tuple[str, ...] | None' = None) -> 'tuple[PacketSystem,
   ↪  PacketRefinement, PacketRefinement]'
```

Definition. Construct the common refinement of two commuting packet systems.

Contract (`D, C, H, R, X`). Packet dimensions, lattice bases, transfer matrices, and residue systems must be compatible. CRT operations require valid moduli and compatible congruences. Prime-local records require a prime and finite precision.

Guarantees (`E, P, B, K`). Rational, integral, and finite-field refinement data are computed exactly. Certificates apply to the declared finite matrices, lattices, residues, and tower depth. Matrix and Smith-form costs grow with dimension and coefficient size.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_refinement_towers.py`, `tests/test_linear_defects.py`, `tests/test_lattice_smith.py`, `tests/test_residue_crt.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 17 through 24.

C.1G · `crt_many`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.crt_many`. Reflected implementation `komorebi_math.refinement.crt.crt_many`. The same runtime object is also available through `komorebi_math.refinement.crt_many`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1G-REFINEMENT`.

Manifest signature (`s`).

```
(residues: 'Iterable[int]', moduli: 'Iterable[int]') -> 'CRTAddress'
```

Definition. Combine a finite collection of pairwise coprime residues.

Contract (`D, C, H, R, X`). Packet dimensions, lattice bases, transfer matrices, and residue systems must be compatible. CRT operations require valid moduli and compatible congruences. Prime-local records require a prime and finite precision.

Guarantees (`E, P, B, K`). Rational, integral, and finite-field refinement data are computed exactly. Certificates apply to the declared finite matrices, lattices, residues, and tower depth. Matrix and Smith-form costs grow with dimension and coefficient size.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_residue_crt.py`, `tests/test_linear_defects.py`, `tests/test_lattice_smith.py`, `tests/test_refinement_towers.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 17 through 24.

C.1G · `crt_pair`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.crt_pair`. Reflected implementation `komorebi_math.refinement.crt.crt_pair`. The same runtime object is also available through `komorebi_math.refinement.crt_pair`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1G-REFINEMENT`.

Manifest signature (s).

```
(a: 'int', modulus_a: 'int', b: 'int', modulus_b: 'int') -> 'CRTAddress'
```

Definition. Combine two coprime residue addresses.

Contract (D, C, H, R, X). Packet dimensions, lattice bases, transfer matrices, and residue systems must be compatible. CRT operations require valid moduli and compatible congruences. Prime-local records require a prime and finite precision.

Guarantees (E, P, B, K). Rational, integral, and finite-field refinement data are computed exactly. Certificates apply to the declared finite matrices, lattices, residues, and tower depth. Matrix and Smith-form costs grow with dimension and coefficient size.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_residue_crt.py`, `tests/test_linear_defects.py`, `tests/test_lattice_smith.py`, `tests/test_refinement_towers.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 17 through 24.

C.1G · `cyclic_project_coefficients`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.cyclic_project_coefficients`. Reflected implementation `komorebi_math.analytic.cyclic.cyclic_project_coefficients`. The same runtime object is also available through `komorebi_math.analytic.cyclic_project_coefficients`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1G-ANALYTIC`.

Manifest signature (s).

```
(coefficients: 'Sequence[Any]', order: 'int', residue: 'int') -> 'np.ndarray'
```

Definition. Keep coefficients whose indices equal one residue modulo `order`.

Contract (D, C, H, R, X). Orders and truncation degrees are nonnegative. Denominators and section parameters must avoid their declared singular cases. Numerical zero and tangent diagnostics require finite search windows and a positive tolerance.

Guarantees (E, P, B, K). Coefficient transforms are exact when their inputs are exact. Numerical root and tangent records certify only the finite interval, truncation, and tolerance that were actually evaluated. They do not establish an unrestricted analytic theorem.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_analytic_cyclic_reciprocal.py`, `tests/test_analytic_appell_zeros.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapter 27.

C.1G · `denominator`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.denominator`. Reflected implementation `komorebi_math.analytic.sections.denominator`. The same runtime object is also available through `komorebi_math.analytic.denominator`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1G-ANALYTIC`.

Manifest signature (s).

```
(order: 'int', z: 'complex') -> 'complex'
```

Definition. Evaluate the distinguished signed denominator `D_order(z)`.

Contract (D, C, H, R, X). Orders and truncation degrees are nonnegative. Denominators and section parameters must avoid their declared singular cases. Numerical zero and tangent diagnostics require finite search windows and a positive tolerance.

Guarantees (E, P, B, K). Coefficient transforms are exact when their inputs are exact. Numerical root and tangent records certify only the finite interval, truncation, and tolerance that were actually evaluated. They do not establish an unrestricted analytic theorem.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_analytic_cyclic_reciprocal.py`, `tests/test_arithmetic_euler.py`, `tests/test_block10_execution_persistence.py`, `tests/test_block11_streaming_migrations.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapter 27.

C.1G · positive_zero_string

Identity (q, a, t, k). Public research-manifest route `komorebi_math.positive_zero_string`. Reflected implementation `komorebi_math.analytic.zeros.positive_zero_string`. The same runtime object is also available through `komorebi_math.analytic.positive_zero_string`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1G-ANALYTIC`.

Manifest signature (s).

```
(order: 'int', count: 'int', *, start: 'int' = 0, tolerance: 'float' = 1e-12, max_iterations: 'int' = 64) ->
   ↪ 'ZeroStringResult'
```

Definition. Compute positive real zero representatives from asymptotic seeds.

Contract (D, C, H, R, X). Orders and truncation degrees are nonnegative. Denominators and section parameters must avoid their declared singular cases. Numerical zero and tangent diagnostics require finite search windows and a positive tolerance.

Guarantees (E, P, B, K). Coefficient transforms are exact when their inputs are exact. Numerical root and tangent records certify only the finite interval, truncation, and tolerance that were actually evaluated. They do not establish an unrestricted analytic theorem.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_analytic_appell_zeros.py`, `tests/test_analytic_cyclic_reciprocal.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapter 27.

C.1G · reciprocal_coefficients

Identity (q, a, t, k). Public research-manifest route `komorebi_math.reciprocal_coefficients`. Reflected implementation `komorebi_math.analytic.reciprocal.reciprocal_coefficients`. The same runtime object is also available through `komorebi_math.analytic.reciprocal_coefficients`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1G-ANALYTIC`.

Manifest signature (`s`).

```
(order: 'int', parameter: 'Number', degree: 'int') -> 'tuple[Number, ...]'
```

Definition. Return `P[order,n](parameter)` for `0 <= n <= degree`.

Contract (`D, C, H, R, X`). Orders and truncation degrees are nonnegative. Denominators and section parameters must avoid their declared singular cases. Numerical zero and tangent diagnostics require finite search windows and a positive tolerance.

Guarantees (`E, P, B, K`). Coefficient transforms are exact when their inputs are exact. Numerical root and tangent records certify only the finite interval, truncation, and tolerance that were actually evaluated. They do not establish an unrestricted analytic theorem.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_analytic_appell_zeros.py`, `tests/test_analytic_cyclic_reciprocal.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapter 27.

C.1G · `residue_analyze`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.residue_analyze`. Reflected implementation `komorebi_math.refinement.residue.residue_analyze`. The same runtime object is also available through `komorebi_math.refinement.residue_analyze`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1G-REFINEMENT`.

Manifest signature (`s`).

```
(values: 'Any', modulus: 'int') -> 'ResidueCoordinates'
```

Definition. Split a one dimensional sequence into residue channels.

Contract (`D, C, H, R, X`). Packet dimensions, lattice bases, transfer matrices, and residue systems must be compatible. CRT operations require valid moduli and compatible congruences. Prime-local records require a prime and finite precision.

Guarantees (`E, P, B, K`). Rational, integral, and finite-field refinement data are computed exactly. Certificates apply to the declared finite matrices, lattices, residues, and tower depth. Matrix and Smith-form costs grow with dimension and coefficient size.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_block15_persistence.py`, `tests/test_block15_polyphase_refinement.py`, `tests/test_residue_crt.py`, `tests/test_linear_defects.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 17 through 24.

C.1G · `residue_packet_system`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.residue_packet_system`. Reflected implementation `komorebi_math.refinement.residue.residue_packet_system`. The same runtime object is also available through `komorebi_math.refinement.residue_packet_system`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1G-REFINEMENT`.

Manifest signature (s).

```
(length: 'int', modulus: 'int', *, backend: 'str | LinearAlgebraBackend | None' = None, names: 'tuple[str, ...] |
    ↪ None' = None) -> 'PacketSystem'
```

Definition. Return diagonal projectors selecting coordinate residues modulo `modulus`.

Contract (D, C, H, R, X). Packet dimensions, lattice bases, transfer matrices, and residue systems must be compatible. CRT operations require valid moduli and compatible congruences. Prime-local records require a prime and finite precision.

Guarantees (E, P, B, K). Rational, integral, and finite-field refinement data are computed exactly. Certificates apply to the declared finite matrices, lattices, residues, and tower depth. Matrix and Smith-form costs grow with dimension and coefficient size.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_residue_crt.py`, `tests/test_linear_defects.py`, `tests/test_lattice_smith.py`, `tests/test_refinement_towers.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 17 through 24.

C.1G · `signed_section`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signed_section`. Reflected implementation `komorebi_math.analytic.sections.signed_section`. The same runtime object is also available through `komorebi_math.analytic.signed_section`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1G-ANALYTIC`.

Manifest signature (s).

```
(order: 'int', residue: 'int', z: 'complex', *, tolerance: 'float' = 1e-15, max_terms: 'int' = 512) -> 'complex'
```

Definition. Evaluate `sum (-1)^n z**(order*n+residue)/(order*n+residue)!`.

Contract (D, C, H, R, X). Orders and truncation degrees are nonnegative. Denominators and section parameters must avoid their declared singular cases. Numerical zero and tangent diagnostics require finite search windows and a positive tolerance.

Guarantees (E, P, B, K). Coefficient transforms are exact when their inputs are exact. Numerical root and tangent records certify only the finite interval, truncation, and tolerance that were actually evaluated. They do not establish an unrestricted analytic theorem.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_analytic_cyclic_reciprocal.py`, `tests/test_analytic_appell_zeros.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapter 27.

C.1G · `theta_polynomial`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.theta_polynomial`. Reflected implementation `komorebi_math.analytic.appell.theta_polynomial`. The same runtime object is also available through `komorebi_math.analytic.theta_polynomial`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1G-ANALYTIC`.

Manifest signature (s).

```
(order: 'int', degree: 'int', parameter: 'Number') -> 'Number'
```

Definition. Evaluate the transfer theta polynomial of one degree.

Contract (D, C, H, R, X). Orders and truncation degrees are nonnegative. Denominators and section parameters must avoid their declared singular cases. Numerical zero and tangent diagnostics require finite search windows and a positive tolerance.

Guarantees (E, P, B, K). Coefficient transforms are exact when their inputs are exact. Numerical root and tangent records certify only the finite interval, truncation, and tolerance that were actually evaluated. They do not establish an unrestricted analytic theorem.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_analytic_appell_zeros.py`, `tests/test_analytic_cyclic_reciprocal.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapter 27.

C.1G · `transfer_section`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.transfer_section`. Reflected implementation `komorebi_math.analytic.sections.transfer_section`. The same runtime object is also available through `komorebi_math.analytic.transfer_section`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1G-ANALYTIC`.

Manifest signature (s).

```
(order: 'int', residue: 'int', z: 'complex', *, scale: 'complex' = 1.0, tolerance: 'float' = 1e-15, max_terms:
    ↪  'int' = 512) -> 'complex'
```

Definition. Evaluate `sum (scale*z)**(order*n+residue)/(order*n+residue)!`.

Contract (D, C, H, R, X). Orders and truncation degrees are nonnegative. Denominators and section parameters must avoid their declared singular cases. Numerical zero and tangent diagnostics require finite search windows and a positive tolerance.

Guarantees (E, P, B, K). Coefficient transforms are exact when their inputs are exact. Numerical root and tangent records certify only the finite interval, truncation, and tolerance that were actually evaluated. They do not establish an unrestricted analytic theorem.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_analytic_cyclic_reciprocal.py`, `tests/test_analytic_appell_zeros.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapter 27.

C.1G · `transfer_tangent`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.transfer_tangent`. Reflected implementation `komorebi_math.analytic.reciprocal.transfer_tangent`. The same runtime object is also available through `komorebi_math.analytic.transfer_tangent`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1G-ANALYTIC`.

Manifest signature (`s`).

```
(order: 'int', z: 'complex') -> 'complex'
```

Definition. Evaluate `-D_order'(z)/D_order(z)`.

Contract (`D, C, H, R, X`). Orders and truncation degrees are nonnegative. Denominators and section parameters must avoid their declared singular cases. Numerical zero and tangent diagnostics require finite search windows and a positive tolerance.

Guarantees (`E, P, B, K`). Coefficient transforms are exact when their inputs are exact. Numerical root and tangent records certify only the finite interval, truncation, and tolerance that were actually evaluated. They do not establish an unrestricted analytic theorem.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_analytic_cyclic_reciprocal.py`, `tests/test_analytic_appell_zeros.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapter 27.

Quantum and dynamical systems (19 cards).

C.1G · `BayesianUpdate`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.BayesianUpdate`. Reflected implementation `komorebi_math.dynamics.bayes.BayesianUpdate`. The same runtime object is also available through `komorebi_math.dynamics.BayesianUpdate`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1G-DYNAMICS`.

Manifest signature (`s`).

```
(prior: 'Any', likelihood: 'Any') -> 'None'
```

Definition. A normalized finite Bayesian update.

Contract (`D, C, H, R, X`). State spaces and operators are finite dimensional. Probabilities and likelihoods are nonnegative, stochastic data are normalized, and block reductions require compatible partitions. Fractional and memory operators retain the branch and regularity assumptions stated by their constructors. Public fields are `prior`, `likelihood`. Principal public operations are `evidence`, `posterior`, `perturbation_bound`.

Guarantees (`E, P, B, K`). Results describe the supplied finite model, partition, prior, kernel, or generator. Exact combinatorial parts remain exact and numerical reductions report finite residuals. No asymptotic or continuum claim is inferred beyond the represented system.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_dynamics_bayes_martingales.py`, `tests/test_dynamics_semigroups_markov.py`, `tests/test_dynamics_feshbach.py`, `tests/test_dynamics_fractional_green_feynman_survival.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 44 through 48.

C.1G · `CircuitPacketExpansion`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.CircuitPacketExpansion`. Reflected implementation `komorebi_math.quantum.circuits.CircuitPacketExpansion`. The same runtime object is also available through `komorebi_math.quantum.CircuitPacketExpansion`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1G-QUANTUM`.

Manifest signature (s).

```
(gates: 'Iterable[Any]', packet_systems: 'Iterable[PacketSystem]', *, tolerance: 'float' = 1e-10, max_paths: 'int'
    ↪  = 1000000) -> 'None'
```

Definition. Insert packet resolutions between every gate of a finite circuit.

Contract (D, C, H, R, X). Matrices and packet systems are finite dimensional and shape compatible. Tolerances and path budgets must be positive. Channel, code, circuit, and spectral contracts reject incompatible dimensions, nonfinite entries, and failed normalization conditions. Principal public operations are `dimension`, `unitary`, `path_blocks`, `block`, `reconstruction_certificate`, `interference`, `block_unitarity_certificate`.

Guarantees (E, P, B, K). Results concern the supplied finite matrices and tolerance. Exact symbolic components remain exact, while floating-point certificates report measured residuals. These contracts do not claim hardware execution or a physical realization.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_quantum_symmetry_circuits.py`, `tests/test_quantum_channels_qec.py`, `tests/test_quantum_transport_effective_spectral.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 53 through 55.

C.1G · `EuclideanTransfer`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.EuclideanTransfer`. Reflected implementation `komorebi_math.quantum.euclidean.EuclideanTransfer`. The same runtime object is also available through `komorebi_math.quantum.EuclideanTransfer`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1G-QUANTUM`.

Manifest signature (s).

```
(hamiltonian: 'Any', lattice_spacing: 'float', *, ground_energy: 'float | None' = None, tolerance: 'float' = 1e-10)
    ↪  -> 'None'
```

Definition. A positive Euclidean transfer operator generated by a Hermitian Hamiltonian.

Contract (D, C, H, R, X). Matrices and packet systems are finite dimensional and shape compatible. Tolerances and path budgets must be positive. Channel, code, circuit, and spectral contracts reject incompatible dimensions, nonfinite entries, and failed normalization conditions. Public fields are `hamiltonian`, `lattice_spacing`, `ground_energy`, `tolerance`. Principal public operations are `shifted_hamiltonian`, `operator`, `reflection_matrix`, `reflection_certificate`, `from_transfer`.

Guarantees (E, P, B, K). Results concern the supplied finite matrices and tolerance. Exact symbolic components remain exact, while floating-point certificates report measured residuals. These contracts do not claim hardware execution or a physical realization.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_quantum_transport_effective_spectral.py`, `tests/test_quantum_channels_qec.py`, `tests/test_quantum_symmetry_circuits.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 53 through 55.

C.1G · FeshbachReduction

Identity (q, a, t, k). Public research-manifest route `komorebi_math.FeshbachReduction`. Reflected implementation `komorebi_math.dynamics.feshbach.FeshbachReduction`. The same runtime object is also available through `komorebi_math.dynamics.FeshbachReduction`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1G-DYNAMICS`.

Manifest signature (s).

```
(matrix: 'Any', retained: 'Iterable[int]') -> 'None'
```

Definition. Retained and hidden block reduction of a finite matrix.

Contract (D, C, H, R, X). State spaces and operators are finite dimensional. Probabilities and likelihoods are nonnegative, stochastic data are normalized, and block reductions require compatible partitions. Fractional and memory operators retain the branch and regularity assumptions stated by their constructors. Public fields are `matrix`, `retained`, `hidden`. Principal public operations are `retained_block`, `hidden_block`, `retained_to_hidden`, `hidden_to_retained`, `a_pq`, `a_qp`, `self_energy`, `effective_operator`, `effective_resolvent`, `direct_projected_resolvent`, `resolvent_certificate`, `memory_kernel`, `memory_kernel_at`, `hidden_source`, `closure_moments`, `local_closure_certificate`.

Guarantees (E, P, B, K). Results describe the supplied finite model, partition, prior, kernel, or generator. Exact combinatorial parts remain exact and numerical reductions report finite residuals. No asymptotic or continuum claim is inferred beyond the represented system.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_dynamics_feshbach.py`, `tests/test_dynamics_semigroups_markov.py`, `tests/test_dynamics_bayes_martingales.py`, `tests/test_dynamics_fractional_green_feynman_survival.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 44 through 48.

C.1G · LindbladGenerator

Identity (q, a, t, k). Public research-manifest route `komorebi_math.LindbladGenerator`. Reflected implementation `komorebi_math.quantum.lindblad.LindbladGenerator`. The same runtime object is also available through `komorebi_math.quantum.LindbladGenerator`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1G-QUANTUM`.

Manifest signature (s).

```
(hamiltonian: 'Any', jumps: 'Iterable[Any]', *, tolerance: 'float' = 1e-10) -> 'None'
```

Definition. A finite Gorini Kossakowski Sudarshan Lindblad generator.

Contract (D, C, H, R, X). Matrices and packet systems are finite dimensional and shape compatible. Tolerances and path budgets must be positive. Channel, code, circuit, and spectral contracts reject incompatible dimensions, nonfinite entries, and failed normalization conditions. Principal public operations are `apply`, `superoperator`, `evolve`, `trace_preservation_certificate`, `coherence_preservation_certificate`.

Guarantees (E, P, B, K). Results concern the supplied finite matrices and tolerance. Exact symbolic components remain exact, while floating-point certificates report measured residuals. These contracts do not claim hardware execution or a physical realization.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_quantum_channels_qec.py`, `tests/test_quantum_transport_effective_spectral.py`, `tests/test_quantum_symmetry_circuits.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 53 through 55.

C.1G · MarkovLumping

Identity (q, a, t, k). Public research-manifest route `komorebi_math.MarkovLumping`. Reflected implementation `komorebi_math.dynamics.markov.MarkovLumping`. The same runtime object is also available through `komorebi_math.dynamics.MarkovLumping`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1G-DYNAMICS`.

Manifest signature (s).

```
(operator: 'Any | FiniteMarkovChain | ContinuousTimeMarkovChain', partition: 'Sequence[int] |
    ↪ Iterable[Iterable[int]]', *, kind: 'MarkovKind | None' = None) -> 'None'
```

Definition. A finite state aggregation for a column convention Markov operator.

Contract (D, C, H, R, X). State spaces and operators are finite dimensional. Probabilities and likelihoods are nonnegative, stochastic data are normalized, and block reductions require compatible partitions. Fractional and memory operators retain the branch and regularity assumptions stated by their constructors. Public fields are `chain`, `aggregation_map`. Principal public operations are `partition`, `blocks`, `aggregation`, `obstruction`, `is_exact`, `approximate_coarse_operator`, `exact_coarse_operator`, `coarse_operator`, `coarse_generator`, `coarse_transition`, `coarse_chain`, `certificate`, `aggregate`, `evolution_defect`, `semigroup_defect`, `error_bound`.

Guarantees (E, P, B, K). Results describe the supplied finite model, partition, prior, kernel, or generator. Exact combinatorial parts remain exact and numerical reductions report finite residuals. No asymptotic or continuum claim is inferred beyond the represented system.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_dynamics_semigroups_markov.py`, `tests/test_dynamics_bayes_martingales.py`, `tests/test_dynamics_feshbach.py`, `tests/test_dynamics_fractional_green_feynman_survival.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 44 through 48.

C.1G · MatrixSemigroup

Identity (q, a, t, k). Public research-manifest route `komorebi_math.MatrixSemigroup`. Reflected implementation `komorebi_math.dynamics.semigroups.MatrixSemigroup`. The same runtime object is also available through `komorebi_math.dynamics.MatrixSemigroup`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1G-DYNAMICS`.

Manifest signature (s).

```
(generator: 'np.ndarray') -> None
```

Definition. The finite semigroup generated by one square matrix.

Contract (D, C, H, R, X). State spaces and operators are finite dimensional. Probabilities and likelihoods are nonnegative, stochastic data are normalized, and block reductions require compatible partitions. Fractional and memory operators retain the branch and regularity assumptions stated by their constructors. Public fields are `generator`. Principal public operations are `dimension`, `at`, `operator`, `apply`, `resolvent`, `semigroup_certificate`.

Guarantees (E, P, B, K). Results describe the supplied finite model, partition, prior, kernel, or generator. Exact combinatorial parts remain exact and numerical reductions report finite residuals. No asymptotic or continuum claim is inferred beyond the represented system.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_block4_hardening.py`, `tests/test_dynamics_semigroups_markov.py`, `tests/test_dynamics_bayes_martingales.py`, `tests/test_dynamics_feshbach.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 44 through 48.

C.1G · MemoryKernel

Identity (q, a, t, k). Public research-manifest route `komorebi_math.MemoryKernel`. Reflected implementation `komorebi_math.dynamics.memory.MemoryKernel`. The same runtime object is also available through `komorebi_math.dynamics.MemoryKernel`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1G-DYNAMICS`.

Manifest signature (s).

```
(output: 'Any', hidden: 'Any', input: 'Any') -> 'None'
```

Definition. A matrix kernel `B exp(tD) C` with rational Laplace transform.

Contract (D, C, H, R, X). State spaces and operators are finite dimensional. Probabilities and likelihoods are nonnegative, stochastic data are normalized, and block reductions require compatible partitions. Fractional and memory operators retain the branch and regularity assumptions stated by their constructors. Public fields are `output`, `hidden`, `input`. Principal public operations are `retained_dimension`, `hidden_dimension`, `at`, `evaluate`, `laplace`, `finite_laplace`, `certificate`.

Guarantees (E, P, B, K). Results describe the supplied finite model, partition, prior, kernel, or generator. Exact combinatorial parts remain exact and numerical reductions report finite residuals. No asymptotic or continuum claim is inferred beyond the represented system.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_dynamics_feshbach.py`, `tests/test_dynamics_semigroups_markov.py`, `tests/test_dynamics_bayes_martingales.py`, `tests/test_dynamics_fractional_green_feynman_survival.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 44 through 48.

C.1G · PhaseTypeSurvival

Identity (q, a, t, k). Public research-manifest route `komorebi_math.PhaseTypeSurvival`. Reflected implementation `komorebi_math.dynamics.survival.PhaseTypeSurvival`. The same runtime object is also available through `komorebi_math.dynamics.PhaseTypeSurvival`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1G-DYNAMICS`.

Manifest signature (s).

```
(subgenerator: 'Any', initial: 'Any', *, cause_rates: 'Any | None' = None, tolerance: 'float' = 1e-12, validate:
    ↪ 'bool' = True) -> 'None'
```

Definition. A finite transient subgenerator acting on column probabilities.

Contract (D, C, H, R, X). State spaces and operators are finite dimensional. Probabilities and likelihoods are nonnegative, stochastic data are normalized, and block reductions require compatible partitions. Fractional and memory operators retain the branch and regularity assumptions stated by their constructors. Public fields are `subgenerator`, `initial`, `cause_rates`, `tolerance`. Principal public operations are `states`, `absorption_rates`, `certificate`, `transient_probability`, `survival`, `density`, `hazard`, `laplace_transform`, `moment`, `cause_density`, `residual_initial`, `residual_model`, `identities_certificate`.

Guarantees (E, P, B, K). Results describe the supplied finite model, partition, prior, kernel, or generator. Exact combinatorial parts remain exact and numerical reductions report finite residuals. No asymptotic or continuum claim is inferred beyond the represented system.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_block4_hardening.py`, `tests/test_dynamics_fractional_green_feynman_survival.py`, `tests/test_dynamics_semigroups_markov.py`, `tests/test_dynamics_bayes_martingales.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace

checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 44 through 48.

C.1G · `QuantumChannel`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.QuantumChannel`. Reflected implementation `komorebi_math.quantum.channels.QuantumChannel`. The same runtime object is also available through `komorebi_math.quantum.QuantumChannel`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1G-QUANTUM`.

Manifest signature (`s`).

```
(kraus: 'Iterable[Any]', *, tolerance: 'float' = 1e-10) -> 'None'
```

Definition. A finite completely positive map represented by Kraus operators.

Contract (`D, C, H, R, X`). Matrices and packet systems are finite dimensional and shape compatible. Tolerances and path budgets must be positive. Channel, code, circuit, and spectral contracts reject incompatible dimensions, nonfinite entries, and failed normalization conditions. Principal public operations are `identity`, `apply`, `apply_density`, `adjoint`, `is_trace_preserving`, `is_unital`, `certificate`, `superoperator`, `choi_matrix`, `compose`, `tensor`, `packet_blocks`, `covariance_certificate`.

Guarantees (`E, P, B, K`). Results concern the supplied finite matrices and tolerance. Exact symbolic components remain exact, while floating-point certificates report measured residuals. These contracts do not claim hardware execution or a physical realization.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_quantum_channels_qec.py`, `tests/test_quantum_transport_effective_spectral.py`, `tests/test_quantum_symmetry_circuits.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 53 through 55.

C.1G · `QuantumCode`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.QuantumCode`. Reflected implementation `komorebi_math.quantum.error_correction.QuantumCode`. The same runtime object is also available through `komorebi_math.quantum.QuantumCode`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1G-QUANTUM`.

Manifest signature (`s`).

```
(projector: 'Any', errors: 'Iterable[Any]', *, tolerance: 'float' = 1e-10) -> 'None'
```

Definition. A finite code projector with a declared family of error operators.

Contract (`D, C, H, R, X`). Matrices and packet systems are finite dimensional and shape compatible. Tolerances and path budgets must be positive. Channel, code, circuit, and spectral contracts reject incompatible dimensions, nonfinite entries, and failed normalization conditions. Principal public operations are `knill_laflamme_report`, `recovery_channel`, `recovered_channel_certificate`, `encode_density`.

Guarantees (`E, P, B, K`). Results concern the supplied finite matrices and tolerance. Exact symbolic components remain exact, while floating-point certificates report measured residuals. These contracts do not claim hardware execution or a physical realization.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_quantum_channels_qec.py`, `tests/test_quantum_transport_effective_spectral.py`, `tests/test_quantum_symmetry_circuits.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 53 through 55.

C.1G · QuantumFeshbachReduction

Identity (q, a, t, k). Public research-manifest route `komorebi_math.QuantumFeshbachReduction`. Reflected implementation `komorebi_math.quantum.effective.QuantumFeshbachReduction`. The same runtime object is also available through `komorebi_math.quantum.QuantumFeshbachReduction`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1G-QUANTUM`.

Manifest signature (s).

```
(hamiltonian: 'Any', retained: 'Iterable[int]', *, tolerance: 'float' = 1e-10) -> 'None'
```

Definition. Hermitian specialization of the finite Schur Feshbach reduction.

Contract (D, C, H, R, X). Matrices and packet systems are finite dimensional and shape compatible. Tolerances and path budgets must be positive. Channel, code, circuit, and spectral contracts reject incompatible dimensions, nonfinite entries, and failed normalization conditions. Principal public operations are `self_energy`, `effective_hamiltonian`, `projected_resolvent`, `resolvent_certificate`, `upper_half_plane_sign_certificate`, `memory_kernel`, `bare_closure_certificate`.

Guarantees (E, P, B, K). Results concern the supplied finite matrices and tolerance. Exact symbolic components remain exact, while floating-point certificates report measured residuals. These contracts do not claim hardware execution or a physical realization.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_quantum_transport_effective_spectral.py`, `tests/test_quantum_channels_qec.py`, `tests/test_quantum_symmetry_circuits.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 53 through 55.

C.1G · QuantumSeedSystem

Identity (q, a, t, k). Public research-manifest route `komorebi_math.QuantumSeedSystem`. Reflected implementation `komorebi_math.quantum.seeds.QuantumSeedSystem`. The same runtime object is also available through `komorebi_math.quantum.QuantumSeedSystem`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1G-QUANTUM`.

Manifest signature (s).

```
(observables: 'Mapping[str, Any] | Sequence[Any]', seeds: 'Sequence[Any]', *, tolerance: 'float' = 1e-10) -> 'None'
```

Definition. A finite observable algebra acting on one or more quantum seeds.

Contract (D, C, H, R, X). Matrices and packet systems are finite dimensional and shape compatible. Tolerances and path budgets must be positive. Channel, code, circuit, and spectral contracts reject incompatible dimensions, nonfinite entries, and failed normalization conditions. Principal public operations are `dimension`, `orbit_basis`, `orbit_dimension`, `is_generating`, `certificate`.

Guarantees (E, P, B, K). Results concern the supplied finite matrices and tolerance. Exact symbolic components remain exact, while floating-point certificates report measured residuals. These contracts do not claim hardware execution or a physical realization.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_quantum_symmetry_circuits.py`, `tests/test_quantum_channels_qec.py`, `tests/test_quantum_transport_effective_spectral.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 53 through 55.

C.1G · constraint_closure

Identity (q, a, t, k). Public research-manifest route `komorebi_math.constraint_closure`. Reflected implementation `komorebi_math.quantum.constraints.constraint_closure`. The same runtime object is also available through `komorebi_math.quantum.constraint_closure`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1G-QUANTUM`.

Manifest signature (s).

```
(constraints: 'Iterable[Any]', *, tolerance: 'float' = 1e-10) -> 'ConstraintClosureReport'
```

Definition. Project every commutator onto the declared complex linear constraint span.

Contract (D, C, H, R, X). Matrices and packet systems are finite dimensional and shape compatible. Tolerances and path budgets must be positive. Channel, code, circuit, and spectral contracts reject incompatible dimensions, nonfinite entries, and failed normalization conditions.

Guarantees (E, P, B, K). Results concern the supplied finite matrices and tolerance. Exact symbolic components remain exact, while floating-point certificates report measured residuals. These contracts do not claim hardware execution or a physical realization.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_quantum_transport_effective_spectral.py`, `tests/test_quantum_channels_qec.py`, `tests/test_quantum_symmetry_circuits.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 53 through 55.

C.1G · cyclic_symmetry_projectors

Identity (q, a, t, k). Public research-manifest route `komorebi_math.cyclic_symmetry_projectors`. Reflected implementation `komorebi_math.quantum.symmetry.cyclic_symmetry_projectors`. The same runtime object is also available through `komorebi_math.quantum.cyclic_symmetry_projectors`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1G-QUANTUM`.

Manifest signature (s).

```
(symmetry: 'Any', order: 'int', *, tolerance: 'float' = 1e-10) -> 'PacketSystem'
```

Definition. Return the complete packet system of a finite cyclic unitary symmetry.

Contract (D, C, H, R, X). Matrices and packet systems are finite dimensional and shape compatible. Tolerances and path budgets must be positive. Channel, code, circuit, and spectral contracts reject incompatible dimensions, nonfinite entries, and failed normalization conditions.

Guarantees (E, P, B, K). Results concern the supplied finite matrices and tolerance. Exact symbolic components remain exact, while floating-point certificates report measured residuals. These contracts do not claim hardware execution or a physical realization.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_quantum_channels_qec.py`, `tests/test_quantum_symmetry_circuits.py`, `tests/test_quantum_transport_effective_spectral.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 53 through 55.

C.1G · fixed_packet_leakage

Identity (q, a, t, k). Public research-manifest route `komorebi_math.fixed_packet_leakage`. Reflected implementation `komorebi_math.quantum.leakage.fixed_packet_leakage`. The same runtime object is also available through `komorebi_math.quantum.fixed_packet_leakage`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1G-QUANTUM`.

Manifest signature (s).

```
(hamiltonian: 'Any', projector: 'Any', time: 'float', *, tolerance: 'float' = 1e-10) -> 'LeakageResult'
```

Definition. Evaluate and certify `||(I-P)e^{-itH}P|| <= |t| ||(I-P)HP||`.

Contract (D, C, H, R, X). Matrices and packet systems are finite dimensional and shape compatible. Tolerances and path budgets must be positive. Channel, code, circuit, and spectral contracts reject incompatible dimensions, nonfinite entries, and failed normalization conditions.

Guarantees (E, P, B, K). Results concern the supplied finite matrices and tolerance. Exact symbolic components remain exact, while floating-point certificates report measured residuals. These contracts do not claim hardware execution or a physical realization.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_quantum_transport_effective_spectral.py`, `tests/test_quantum_channels_qec.py`, `tests/test_quantum_symmetry_circuits.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 53 through 55.

C.1G · fractional_power

Identity (q, a, t, k). Public research-manifest route `komorebi_math.fractional_power`. Reflected implementation `komorebi_math.dynamics.fractional.fractional_power`. The same runtime object is also available through `komorebi_math.dynamics.fractional_power`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1G-DYNAMICS`.

Manifest signature (s).

```
(matrix: 'Any', exponent: 'float', *, tolerance: 'float' = 1e-12) -> 'np.ndarray'
```

Definition. Return the positive spectral power of a finite Hermitian matrix.

Contract (D, C, H, R, X). State spaces and operators are finite dimensional. Probabilities and likelihoods are nonnegative, stochastic data are normalized, and block reductions require compatible partitions. Fractional and memory operators retain the branch and regularity assumptions stated by their constructors.

Guarantees (E, P, B, K). Results describe the supplied finite model, partition, prior, kernel, or generator. Exact combinatorial parts remain exact and numerical reductions report finite residuals. No asymptotic or continuum claim is inferred beyond the represented system.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_dynamics_fractional_green_feynman_survival.py`, `tests/test_dynamics_semigroups_markov.py`, `tests/test_dynamics_bayes_martingales.py`, `tests/test_dynamics_feshbach.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 44 through 48.

C.1G · periodic_laplacian

Identity (q, a, t, k). Public research-manifest route `komorebi_math.periodic_laplacian`. Reflected implementation `komorebi_math.quantum.spectral.periodic_laplacian`. The same runtime object is also available through `komorebi_math.quantum.periodic_laplacian`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1G-QUANTUM`.

Manifest signature (s).

```
(shape: 'Sequence[int]', *, spacing: 'float | None' = None) -> 'np.ndarray'
```

Definition. Construct the finite nearest neighbor periodic lattice Laplacian.

Contract (D, C, H, R, X). Matrices and packet systems are finite dimensional and shape compatible. Tolerances and path budgets must be positive. Channel, code, circuit, and spectral contracts reject incompatible dimensions, nonfinite entries, and failed normalization conditions.

Guarantees (E, P, B, K). Results concern the supplied finite matrices and tolerance. Exact symbolic components remain exact, while floating-point certificates report measured residuals. These contracts do not claim hardware execution or a physical realization.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_quantum_transport_effective_spectral.py`, `tests/test_quantum_channels_qec.py`, `tests/test_quantum_symmetry_circuits.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 53 through 55.

C.1G · spectral_dimension

Identity (q, a, t, k). Public research-manifest route `komorebi_math.spectral_dimension`. Reflected implementation `komorebi_math.quantum.spectral.spectral_dimension`. The same runtime object is also available through `komorebi_math.quantum.spectral_dimension`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1G-QUANTUM`.

Manifest signature (s).

```
(operator: 'Any', time: 'float') -> 'float'
```

Definition. Return `2*t*Tr(L exp(-tL))/Tr(exp(-tL))`.

Contract (D, C, H, R, X). Matrices and packet systems are finite dimensional and shape compatible. Tolerances and path budgets must be positive. Channel, code, circuit, and spectral contracts reject incompatible dimensions, nonfinite entries, and failed normalization conditions.

Guarantees (E, P, B, K). Results concern the supplied finite matrices and tolerance. Exact symbolic components remain exact, while floating-point certificates report measured residuals. These contracts do not claim hardware execution or a physical realization.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_quantum_transport_effective_spectral.py`, `tests/test_quantum_channels_qec.py`, `tests/test_quantum_symmetry_circuits.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 53 through 55.

Deformation and Langlands systems (34 cards).

C.1G · AlternatingForm

Identity (q, a, t, k). Public research-manifest route `komorebi_math.AlternatingForm`. Reflected implementation `komorebi_math.langlands.local.symplectic.AlternatingForm`. The root export resolves directly to this nested implementation owner. The umbrella `komorebi_math.langlands` package does not publish a second alias. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1G-LANGLANDS`.

Manifest signature (s).

```
(matrix: 'Any', prime: 'int') -> None
```

Definition. A finite alternating bilinear form over a prime field.

Contract (D, C, H, R, X). Fields, groups, characters, packets, and towers are finite explicit objects. Representation dimensions, actions, cocycles, and local parameters must be compatible. Completion reports require an explicit status for every declared gate. Public fields are `matrix`, `prime`. Principal public operations are `dimension`, `rank`, `radical_dimension`, `nondegenerate`, `radical_basis`, `evaluate`, `certificate`.

Guarantees (E, P, B, K). Finite-field, finite-group, local-packet, ramification, and gate calculations are explicit and reproducible. Global completion objects remain conditional on their recorded gates. This API does not claim a proof of the general Langlands correspondence.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_langlands_local_heisenberg_symplectic.py`, `tests/test_langlands_ramification_global_completion.py`, `tests/test_langlands_finite_image.py`, `tests/test_langlands_local_fields_tame_gauss.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 57 and 58.

C.1G · AutomorphicCompletionReport

Identity (q, a, t, k). Public research-manifest route `komorebi_math.AutomorphicCompletionReport`. Reflected implementation `komorebi_math.langlands.completion.gates.AutomorphicCompletionReport`. The root export resolves directly to this nested implementation owner. The umbrella `komorebi_math.langlands` package does not publish a second alias. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1G-LANGLANDS`.

Manifest signature (s).

```
(gates: 'Mapping[CompletionGate | str, GateStatus]') -> None
```

Definition. A five-gate conditional automorphic completion report.

Contract (D, C, H, R, X). Fields, groups, characters, packets, and towers are finite explicit objects. Representation dimensions, actions, cocycles, and local parameters must be compatible. Completion reports require an explicit status for every declared gate. Public fields are `gates`. Principal public operations are `complete`, `unmet_gates`, `external_gates`, `result`.

Guarantees (E, P, B, K). Finite-field, finite-group, local-packet, ramification, and gate calculations are explicit and reproducible. Global completion objects remain conditional on their recorded gates. This API does not claim a proof of the general Langlands correspondence.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_langlands_ramification_global_completion.py`, `tests/test_langlands_finite_image.py`, `tests/test_langlands_local_heisenberg_symplectic.py`, `tests/test_langlands_local_fields_tame_gauss.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 57 and 58.

C.1G · AutomorphicInductionPlan

Identity (q, a, t, k). Public research-manifest route komorebi_math.AutomorphicInductionPlan. Reflected implementation komorebi_math.langlands.finite_image.plans.AutomorphicInductionPlan. The root export resolves directly to this nested implementation owner. The umbrella komorebi_math.langlands package does not publish a second alias. The root route is governed by [RESEARCH] policy pre-beta-additive. It does not enlarge the frozen stable compatibility promise. Manifest kind class. Profile C1G-LANGLANDS.

Manifest signature (s).

```
(representation: 'InducedRepresentation[Element]', base_field: 'str', inducing_field: 'str', inducing_character:
    ↪ 'str', cyclic_tower: 'tuple[str, ...]' = ()) -> None
```

Definition. A conditional symbolic plan attached to one induced character parameter.

Contract (D, C, H, R, X). Fields, groups, characters, packets, and towers are finite explicit objects. Representation dimensions, actions, cocycles, and local parameters must be compatible. Completion reports require an explicit status for every declared gate. Public fields are representation, base_field, inducing_field, inducing_character, cyclic_tower. Principal public operations are operations.

Guarantees (E, P, B, K). Finite-field, finite-group, local-packet, ramification, and gate calculations are explicit and reproducible. Global completion objects remain conditional on their recorded gates. This API does not claim a proof of the general Langlands correspondence.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include tests/test_langlands_finite_image.py, tests/test_langlands_local_heisenberg_symplectic.py, tests/test_langlands_local_fields_tame_gauss.py, tests/test_langlands_ramification_global_completion.py. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 57 and 58.

C.1G · CliffordStep

Identity (q, a, t, k). Public research-manifest route komorebi_math.CliffordStep. Reflected implementation komorebi_math.langlands.finite_image.clifford.CliffordStep. The root export resolves directly to this nested implementation owner. The umbrella komorebi_math.langlands package does not publish a second alias. The root route is governed by [RESEARCH] policy pre-beta-additive. It does not enlarge the frozen stable compatibility promise. Manifest kind class. Profile C1G-LANGLANDS.

Manifest signature (s).

```
(parent_order: 'int', child_order: 'int', parent_degree: 'int', child_degree: 'int', mode: 'CliffordMode | str',
    ↪ attached_descent: 'bool' = False, label: 'str' = '') -> None
```

Definition. One validated prime-cyclic step in a declared Clifford tower.

Contract (D, C, H, R, X). Fields, groups, characters, packets, and towers are finite explicit objects. Representation dimensions, actions, cocycles, and local parameters must be compatible. Completion reports require an explicit status for every declared gate. Public fields are parent_order, child_order, parent_degree, child_degree, mode, attached_descent, label. Principal public operations are quotient_prime, nonabelian_extension_gate.

Guarantees (E, P, B, K). Finite-field, finite-group, local-packet, ramification, and gate calculations are explicit and reproducible. Global completion objects remain conditional on their recorded gates. This API does not claim a proof of the general Langlands correspondence.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include tests/test_langlands_finite_image.py, tests/test_langlands_local_heisenberg_symplectic.py, tests/test_langlands_local_fields_tame_gauss.py, tests/test_langlands_ramification_global_completion.py. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The

source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 57 and 58.

C.1G · `CliffordTower`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.CliffordTower`. Reflected implementation `komorebi_math.langlands.finite_image.clifford.CliffordTower`. The root export resolves directly to this nested implementation owner. The umbrella `komorebi_math.langlands` package does not publish a second alias. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1G-LANGLANDS`.

Manifest signature (`s`).

```
(steps: 'Iterable[CliffordStep]') -> 'None'
```

Definition. A finite prime-cyclic Clifford tower supplied with dimension data.

Contract (`D, C, H, R, X`). Fields, groups, characters, packets, and towers are finite explicit objects. Representation dimensions, actions, cocycles, and local parameters must be compatible. Completion reports require an explicit status for every declared gate. Principal public operations are `word`, `depth`, `report`, `automorphic_plan`.

Guarantees (`E, P, B, K`). Finite-field, finite-group, local-packet, ramification, and gate calculations are explicit and reproducible. Global completion objects remain conditional on their recorded gates. This API does not claim a proof of the general Langlands correspondence.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_langlands_finite_image.py`, `tests/test_langlands_local_heisenberg_symplectic.py`, `tests/test_langlands_local_fields_tame_gauss.py`, `tests/test_langlands_ramification_global_completion.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 57 and 58.

C.1G · `CompatiblePropagationReport`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.CompatiblePropagationReport`. Reflected implementation `komorebi_math.langlands.completion.gates.CompatiblePropagationReport`. The root export resolves directly to this nested implementation owner. The umbrella `komorebi_math.langlands` package does not publish a second alias. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1G-LANGLANDS`.

Manifest signature (`s`).

```
(common_spine: 'bool', semisimple: 'bool', one_member_automorphic: 'bool', local_global_compatibility: 'bool') ->
    ↪ None
```

Definition. One-member propagation through an already supplied compatible system.

Contract (`D, C, H, R, X`). Fields, groups, characters, packets, and towers are finite explicit objects. Representation dimensions, actions, cocycles, and local parameters must be compatible. Completion reports require an explicit status for every declared gate. Public fields are `common_spine`, `semisimple`, `one_member_automorphic`, `local_global_compatibility`. Principal public operations are `result`.

Guarantees (`E, P, B, K`). Finite-field, finite-group, local-packet, ramification, and gate calculations are explicit and reproducible. Global completion objects remain conditional on their recorded gates. This API does not claim a proof of the general Langlands correspondence.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_langlands_ramification_global_completion.py`, `tests/test_langlands_finite_image.py`, `tests/test_langlands_local_heisenberg_symplectic.py`, `tests/test_langlands_local_fields_tame_gauss.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The

source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 57 and 58.

C.1G · `CompletionGate`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.CompletionGate`. Reflected implementation `komorebi_math.langlands.completion.gates.CompletionGate`. The root export resolves directly to this nested implementation owner. The umbrella `komorebi_math.langlands` package does not publish a second alias. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1G-LANGLANDS`.

Manifest signature (`s`).

```
(*values)
```

Definition. The five formal gates in pointwise automorphic completion.

Contract (`D, C, H, R, X`). Fields, groups, characters, packets, and towers are finite explicit objects. Representation dimensions, actions, cocycles, and local parameters must be compatible. Completion reports require an explicit status for every declared gate.

Guarantees (`E, P, B, K`). Finite-field, finite-group, local-packet, ramification, and gate calculations are explicit and reproducible. Global completion objects remain conditional on their recorded gates. This API does not claim a proof of the general Langlands correspondence.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_langlands_ramification_global_completion.py`, `tests/test_langlands_finite_image.py`, `tests/test_langlands_local_heisenberg_symplectic.py`, `tests/test_langlands_local_fields_tame_gauss.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 57 and 58.

C.1G · `FiniteCochainComplex`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.FiniteCochainComplex`. Reflected implementation `komorebi_math.deformation.complexes.FiniteCochainComplex`. The same runtime object is also available through `komorebi_math.deformation.FiniteCochainComplex`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1G-DEFORMATION`.

Manifest signature (`s`).

```
(dimensions: 'Mapping[int, int]', differentials: 'Mapping[int, Any]', *, prime: 'int') -> 'None'
```

Definition. A bounded finite cochain complex over a prime field.

Contract (`D, C, H, R, X`). Complexes, modules, forests, and Frobenius data are finite. Prime-field operations require a valid prime, differential matrices must match the declared dimensions, and cochain data must satisfy the finite compatibility checks enforced by the implementation. Principal public operations are `differential`, `cohomology`, `certificate`, `packet_cohomology`.

Guarantees (`E, P, B, K`). Cohomology, support, visibility, and companion calculations are exact over the declared finite field or finite presentation. The records do not by themselves prove an unrestricted deformation or lifting theorem.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_deformation_complex_support_companions.py`, `tests/test_deformation_forests_visibility_counting.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapter 56.

C.1G · FiniteField

Identity (q, a, t, k). Public research-manifest route `komorebi_math.FiniteField`. Reflected implementation `komorebi_math.langlands.local.finite_fields.FiniteField`. The root export resolves directly to this nested implementation owner. The umbrella `komorebi_math.langlands` package does not publish a second alias. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1G-LANGLANDS`.

Manifest signature (s).

```
(prime: 'int', degree: 'int' = 1, modulus: 'tuple[int, ...] | None' = None) -> None
```

Definition. The field `F_(p^degree)` in one explicit polynomial basis.

Contract (D, C, H, R, X). Fields, groups, characters, packets, and towers are finite explicit objects. Representation dimensions, actions, cocycles, and local parameters must be compatible. Completion reports require an explicit status for every declared gate. Public fields are `prime`, `degree`, `modulus`. Principal public operations are `order`, `normalize`, `encode`, `decode`, `element`, `coerce`, `add`, `subtract`, `negate`, `multiply`, `power`, `inverse`, `elements`, `trace`, `norm`, `discrete_log`.

Guarantees (E, P, B, K). Finite-field, finite-group, local-packet, ramification, and gate calculations are explicit and reproducible. Global completion objects remain conditional on their recorded gates. This API does not claim a proof of the general Langlands correspondence.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_langlands_local_fields_tame_gauss.py`, `tests/test_langlands_finite_image.py`, `tests/test_langlands_local_heisenberg_symplectic.py`, `tests/test_langlands_ramification_global_completion.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 57 and 58.

C.1G · FiniteGroup

Identity (q, a, t, k). Public research-manifest route `komorebi_math.FiniteGroup`. Reflected implementation `komorebi_math.langlands.finite_image.groups.FiniteGroup`. The root export resolves directly to this nested implementation owner. The umbrella `komorebi_math.langlands` package does not publish a second alias. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1G-LANGLANDS`.

Manifest signature (s).

```
(elements: 'Iterable[Element]', multiply: 'Callable[[Element, Element], Element]', identity: 'Element', *, name:
    ↪ 'str' = 'G', validate: 'bool' = True, associativity_limit: 'int' = 128) -> 'None'
```

Definition. A finite group given by an explicit element set and multiplication law.

Contract (D, C, H, R, X). Fields, groups, characters, packets, and towers are finite explicit objects. Representation dimensions, actions, cocycles, and local parameters must be compatible. Completion reports require an explicit status for every declared gate. Principal public operations are `order`, `inverse`, `power`, `generated_subgroup`, `subgroup`, `left_coset_representatives`, `double_coset_representatives`, `certificate`.

Guarantees (E, P, B, K). Finite-field, finite-group, local-packet, ramification, and gate calculations are explicit and reproducible. Global completion objects remain conditional on their recorded gates. This API does not claim a proof of the general Langlands correspondence.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_langlands_finite_image.py`, `tests/test_langlands_local_heisenberg_symplectic.py`, `tests/test_langlands_local_fields_tame_gauss.py`, `tests/test_langlands_ramification_global_completion.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 57 and 58.

C.1G · FiniteHeckePackets

Identity (q, a, t, k). Public research-manifest route `komorebi_math.FiniteHeckePackets`. Reflected implementation `komorebi_math.langlands.global_.hecke.FiniteHeckePackets`. The root export resolves directly to this nested implementation owner. The umbrella `komorebi_math.langlands` package does not publish a second alias. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1G-LANGLANDS`.

Manifest signature (s).

```
(projectors: 'Mapping[str, Any]', *, commuting_operators: 'Iterable[Any]' = (), tolerance: 'float' = 1e-10) ->
    ↪ 'None'
```

Definition. A validated finite family of central idempotents on one carrier.

Contract (D, C, H, R, X). Fields, groups, characters, packets, and towers are finite explicit objects. Representation dimensions, actions, cocycles, and local parameters must be compatible. Completion reports require an explicit status for every declared gate. Principal public operations are `packets`, `occupied`, `certificate`.

Guarantees (E, P, B, K). Finite-field, finite-group, local-packet, ramification, and gate calculations are explicit and reproducible. Global completion objects remain conditional on their recorded gates. This API does not claim a proof of the general Langlands correspondence.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_langlands_ramification_global_completion.py`, `tests/test_langlands_finite_image.py`, `tests/test_langlands_local_heisenberg_symplectic.py`, `tests/test_langlands_local_fields_tame_gauss.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 57 and 58.

C.1G · FiniteLiftForest

Identity (q, a, t, k). Public research-manifest route `komorebi_math.FiniteLiftForest`. Reflected implementation `komorebi_math.deformation.forests.FiniteLiftForest`. The same runtime object is also available through `komorebi_math.deformation.FiniteLiftForest`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1G-DEFORMATION`.

Manifest signature (s).

```
(levels: 'Mapping[int, Iterable[Node]]', reductions: 'Mapping[int, Callable[[Node], Node] | Mapping[Node, Node]]')
    ↪ -> 'None'
```

Definition. A finite inverse system of lift levels with explicit reduction maps.

Contract (D, C, H, R, X). Complexes, modules, forests, and Frobenius data are finite. Prime-field operations require a valid prime, differential matrices must match the declared dimensions, and cochain data must satisfy the finite compatibility checks enforced by the implementation. Principal public operations are `max_depth`, `level`, `reduce`, `children`, `image`, `stable_core`, `branches`, `report`.

Guarantees (E, P, B, K). Cohomology, support, visibility, and companion calculations are exact over the declared finite field or finite presentation. The records do not by themselves prove an unrestricted deformation or lifting theorem.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_deformation_forests_visibility_counting.py`, `tests/test_deformation_complex_support_companions.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapter 56.

C.1G · FrobeniusOrbit

Identity (q, a, t, k). Public research-manifest route `komorebi_math.FrobeniusOrbit`. Reflected implementation `komorebi_math.langlands.local.tame.FrobeniusOrbit`. The root export resolves directly to this nested implementation owner. The umbrella `komorebi_math.langlands` package does not publish a second alias. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1G-LANGLANDS`.

Manifest signature (s).

```
(q: 'int', degree: 'int', exponent: 'int') -> None
```

Definition. A tame character orbit under multiplication by the residue cardinality.

Contract (D, C, H, R, X). Fields, groups, characters, packets, and towers are finite explicit objects. Representation dimensions, actions, cocycles, and local parameters must be compatible. Completion reports require an explicit status for every declared gate. Public fields are `q`, `degree`, `exponent`. Principal public operations are `modulus`, `elements`, `orbit_length`, `regular`, `dual_exponent`, `self_dual`, `certificate`.

Guarantees (E, P, B, K). Finite-field, finite-group, local-packet, ramification, and gate calculations are explicit and reproducible. Global completion objects remain conditional on their recorded gates. This API does not claim a proof of the general Langlands correspondence.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_langlands_local_fields_tame_gauss.py`, `tests/test_langlands_finite_image.py`, `tests/test_langlands_local_heisenberg_symplectic.py`, `tests/test_langlands_ramification_global_completion.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 57 and 58.

C.1G · FrobeniusSpine

Identity (q, a, t, k). Public research-manifest route `komorebi_math.FrobeniusSpine`. Reflected implementation `komorebi_math.deformation.companions.FrobeniusSpine`. The same runtime object is also available through `komorebi_math.deformation.FrobeniusSpine`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1G-DEFORMATION`.

Manifest signature (s).

```
(polynomials: 'Mapping[Hashable, tuple[Any, ...]]', local_labels: 'Mapping[Hashable, Any] | None' = None) -> None
```

Definition. One finite algebraic spine of monic Frobenius polynomials.

Contract (D, C, H, R, X). Complexes, modules, forests, and Frobenius data are finite. Prime-field operations require a valid prime, differential matrices must match the declared dimensions, and cochain data must satisfy the finite compatibility checks enforced by the implementation. Public fields are `polynomials`, `local_labels`. Principal public operations are `rank`, `agrees`.

Guarantees (E, P, B, K). Cohomology, support, visibility, and companion calculations are exact over the declared finite field or finite presentation. The records do not by themselves prove an unrestricted deformation or lifting theorem.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_deformation_complex_support_companions.py`, `tests/test_deformation_forests_visibility_counting.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapter 56.

C.1G · GateStatus

Identity (q, a, t, k). Public research-manifest route `komorebi_math.GateStatus`. Reflected implementation `komorebi_math.langlands.completion.gates.GateStatus`. The root export resolves directly to this nested implementation owner. The umbrella `komorebi_math.langlands` package does not publish a second alias. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1G-LANGLANDS`.

Manifest signature (s).

```
(satisfied: 'bool', detail: 'str', external: 'bool' = True) -> None
```

Definition. One gate value with a human-readable explanation.

Contract (D, C, H, R, X). Fields, groups, characters, packets, and towers are finite explicit objects. Representation dimensions, actions, cocycles, and local parameters must be compatible. Completion reports require an explicit status for every declared gate. Public fields are `satisfied`, `detail`, `external`.

Guarantees (E, P, B, K). Finite-field, finite-group, local-packet, ramification, and gate calculations are explicit and reproducible. Global completion objects remain conditional on their recorded gates. This API does not claim a proof of the general Langlands correspondence.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_langlands_ramification_global_completion.py`, `tests/test_langlands_finite_image.py`, `tests/test_langlands_local_heisenberg_symplectic.py`, `tests/test_langlands_local_fields_tame_gauss.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 57 and 58.

C.1G · HeisenbergModel

Identity (q, a, t, k). Public research-manifest route `komorebi_math.HeisenbergModel`. Reflected implementation `komorebi_math.langlands.local.heisenberg.HeisenbergModel`. The root export resolves directly to this nested implementation owner. The umbrella `komorebi_math.langlands` package does not publish a second alias. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1G-LANGLANDS`.

Manifest signature (s).

```
(prime: 'int', rank: 'int', *, max_dimension: 'int' = 4096) -> 'None'
```

Definition. The finite Schrödinger representation over `F_p^rank`.

Contract (D, C, H, R, X). Fields, groups, characters, packets, and towers are finite explicit objects. Representation dimensions, actions, cocycles, and local parameters must be compatible. Completion reports require an explicit status for every declared gate. Principal public operations are `element`, `multiply`, `inverse`, `translation_matrix`, `modulation_matrix`, `matrix`, `group_law_certificate`, `trace`, `trace_support_certificate`, `translation_selector`, `selector_certificate`.

Guarantees (E, P, B, K). Finite-field, finite-group, local-packet, ramification, and gate calculations are explicit and reproducible. Global completion objects remain conditional on their recorded gates. This API does not claim a proof of the general Langlands correspondence.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_langlands_local_heisenberg_symplectic.py`, `tests/test_langlands_finite_image.py`, `tests/test_langlands_local_fields_tame_gauss.py`, `tests/test_langlands_ramification_global_completion.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 57 and 58.

C.1G · `InducedRepresentation`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.InducedRepresentation`. Reflected implementation `komorebi_math.langlands.finite_image.characters.InducedRepresentation`. The root export resolves directly to this nested implementation owner. The umbrella `komorebi_math.langlands` package does not publish a second alias. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1G-LANGLANDS`.

Manifest signature (`s`).

```
(group: 'FiniteGroup[Element]', character: 'LinearCharacter[Element]') -> 'None'
```

Definition. The explicit monomial representation induced from a linear character.

Contract (`D, C, H, R, X`). Fields, groups, characters, packets, and towers are finite explicit objects. Representation dimensions, actions, cocycles, and local parameters must be compatible. Completion reports require an explicit status for every declared gate. Principal public operations are `basis_action`, `matrix`, `trace`, `certificate`.

Guarantees (`E, P, B, K`). Finite-field, finite-group, local-packet, ramification, and gate calculations are explicit and reproducible. Global completion objects remain conditional on their recorded gates. This API does not claim a proof of the general Langlands correspondence.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_langlands_finite_image.py`, `tests/test_langlands_local_heisenberg_symplectic.py`, `tests/test_langlands_local_fields_tame_gauss.py`, `tests/test_langlands_ramification_global_completion.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 57 and 58.

C.1G · `LinearCharacter`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.LinearCharacter`. Reflected implementation `komorebi_math.langlands.finite_image.characters.LinearCharacter`. The root export resolves directly to this nested implementation owner. The umbrella `komorebi_math.langlands` package does not publish a second alias. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1G-LANGLANDS`.

Manifest signature (`s`).

```
(subgroup: 'FiniteSubgroup[Element]', values: 'Mapping[Element, complex]', tolerance: 'float' = 1e-10) -> None
```

Definition. A one dimensional complex character of an explicit finite subgroup.

Contract (`D, C, H, R, X`). Fields, groups, characters, packets, and towers are finite explicit objects. Representation dimensions, actions, cocycles, and local parameters must be compatible. Completion reports require an explicit status for every declared gate. Public fields are `subgroup`, `values`, `tolerance`. Principal public operations are `conjugate`.

Guarantees (`E, P, B, K`). Finite-field, finite-group, local-packet, ramification, and gate calculations are explicit and reproducible. Global completion objects remain conditional on their recorded gates. This API does not claim a proof of the general Langlands correspondence.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_langlands_finite_image.py`, `tests/test_langlands_local_heisenberg_symplectic.py`, `tests/test_langlands_local_fields_tame_gauss.py`, `tests/test_langlands_ramification_global_completion.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 57 and 58.

C.1G · MonomialAction

Identity (q, a, t, k). Public research-manifest route `komorebi_math.MonomialAction`. Reflected implementation `komorebi_math.langlands.finite_image.orbits.MonomialAction`. The root export resolves directly to this nested implementation owner. The umbrella `komorebi_math.langlands` package does not publish a second alias. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1G-LANGLANDS`.

Manifest signature (s).

```
(representation: 'InducedRepresentation[Element]') -> 'None'
```

Definition. A finite monomial action described by permutations and basis scalars.

Contract (D, C, H, R, X). Fields, groups, characters, packets, and towers are finite explicit objects. Representation dimensions, actions, cocycles, and local parameters must be compatible. Completion reports require an explicit status for every declared gate. Principal public operations are `symmetric_power_packets`, `exterior_power_packets`, `tensor_packets`, `packet_certificate`.

Guarantees (E, P, B, K). Finite-field, finite-group, local-packet, ramification, and gate calculations are explicit and reproducible. Global completion objects remain conditional on their recorded gates. This API does not claim a proof of the general Langlands correspondence.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_langlands_finite_image.py`, `tests/test_langlands_local_heisenberg_symplectic.py`, `tests/test_langlands_local_fields_tame_gauss.py`, `tests/test_langlands_ramification_global_completion.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 57 and 58.

C.1G · PatchingSupportReport

Identity (q, a, t, k). Public research-manifest route `komorebi_math.PatchingSupportReport`. Reflected implementation `komorebi_math.langlands.global_.patching.PatchingSupportReport`. The root export resolves directly to this nested implementation owner. The umbrella `komorebi_math.langlands` package does not publish a second alias. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1G-LANGLANDS`.

Manifest signature (s).

```
(patched_object_exists: 'bool', patched_full_support: 'bool', finite_level_descent: 'bool',
    ↪ comparison_preserves_support: 'bool', characteristic_zero_fiber_nonzero: 'bool', derived: 'bool' = False) ->
    ↪ None
```

Definition. A conditional support report for one patched module or complex.

Contract (D, C, H, R, X). Fields, groups, characters, packets, and towers are finite explicit objects. Representation dimensions, actions, cocycles, and local parameters must be compatible. Completion reports require an explicit status for every declared gate. Public fields are `patched_object_exists`, `patched_full_support`, `finite_level_descent`, `comparison_preserves_support`, `characteristic_zero_fiber_nonzero`, `derived`. Principal public operations are `complete`, `unmet`, `certificate`.

Guarantees (E, P, B, K). Finite-field, finite-group, local-packet, ramification, and gate calculations are explicit and reproducible. Global completion objects remain conditional on their recorded gates. This API does not claim a proof of the general Langlands correspondence.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_langlands_ramification_global_completion.py`, `tests/test_langlands_finite_image.py`, `tests/test_langlands_local_heisenberg_symplectic.py`, `tests/test_langlands_local_fields_tame_gauss.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The

source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 57 and 58.

C.1G · PresentedModule

Identity (q, a, t, k). Public research-manifest route `komorebi_math.PresentedModule`. Reflected implementation `komorebi_math.deformation.support.PresentedModule`. The same runtime object is also available through `komorebi_math.deformation.PresentedModule`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1G-DEFORMATION`.

Manifest signature (s).

```
(presentation: 'Any', *, variables: 'Iterable[Any]' = ()) -> 'None'
```

Definition. A finite module presented by a matrix of relations.

Contract (D, C, H, R, X). Complexes, modules, forests, and Frobenius data are finite. Prime-field operations require a valid prime, differential matrices must match the declared dimensions, and cochain data must satisfy the finite compatibility checks enforced by the implementation. Principal public operations are `fitting_generators`, `evaluate`, `fiber`, `finite_support`.

Guarantees (E, P, B, K). Cohomology, support, visibility, and companion calculations are exact over the declared finite field or finite presentation. The records do not by themselves prove an unrestricted deformation or lifting theorem.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_deformation_complex_support_companions.py`, `tests/test_deformation_forests_visibility_counting.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapter 56.

C.1G · ProjectiveIntertwinerEnvelope

Identity (q, a, t, k). Public research-manifest route `komorebi_math.ProjectiveIntertwinerEnvelope`. Reflected implementation `komorebi_math.langlands.local.envelopes.ProjectiveIntertwinerEnvelope`. The root export resolves directly to this nested implementation owner. The umbrella `komorebi_math.langlands` package does not publish a second alias. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1G-LANGLANDS`.

Manifest signature (s).

```
(*, prime: 'int', labels: 'Iterable[Iterable[int]]', operators: 'Iterable[Any]', tolerance: 'float' = 1e-08) ->
    ↪ 'None'
```

Definition. Recover a finite commutator pairing from projective intertwiners.

Contract (D, C, H, R, X). Fields, groups, characters, packets, and towers are finite explicit objects. Representation dimensions, actions, cocycles, and local parameters must be compatible. Completion reports require an explicit status for every declared gate. Principal public operations are `recover`.

Guarantees (E, P, B, K). Finite-field, finite-group, local-packet, ramification, and gate calculations are explicit and reproducible. Global completion objects remain conditional on their recorded gates. This API does not claim a proof of the general Langlands correspondence.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_langlands_local_heisenberg_symplectic.py`, `tests/test_langlands_finite_image.py`, `tests/test_langlands_local_fields_tame_gauss.py`, `tests/test_langlands_ramification_global_completion.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 57 and 58.

C.1G · ProjectiveMultiplicityPacket

Identity (q, a, t, k). Public research-manifest route `komorebi_math.ProjectiveMultiplicityPacket`. Reflected implementation `komorebi_math.langlands.ramification.towers.ProjectiveMultiplicityPacket`. The root export resolves directly to this nested implementation owner. The umbrella `komorebi_math.langlands` package does not publish a second alias. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1G-LANGLANDS`.

Manifest signature (s).

```
(prime: 'int', quotient_rank: 'int', form: 'AlternatingForm', phase_label: 'str | None' = None,
    ↪ quadratic_refinement: 'QuadraticRefinement | None' = None) -> None
```

Definition. A finite projective multiplicity packet at one ramification stage.

Contract (D, C, H, R, X). Fields, groups, characters, packets, and towers are finite explicit objects. Representation dimensions, actions, cocycles, and local parameters must be compatible. Completion reports require an explicit status for every declared gate. Public fields are `prime`, `quotient_rank`, `form`, `phase_label`, `quadratic_refinement`. Principal public operations are `radical_dimension`, `symplectic_rank`, `heisenberg_dimension`, `phase_diagnostic`.

Guarantees (E, P, B, K). Finite-field, finite-group, local-packet, ramification, and gate calculations are explicit and reproducible. Global completion objects remain conditional on their recorded gates. This API does not claim a proof of the general Langlands correspondence.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_langlands_ramification_global_completion.py`, `tests/test_langlands_finite_image.py`, `tests/test_langlands_local_heisenberg_symplectic.py`, `tests/test_langlands_local_fields_tame_gauss.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 57 and 58.

C.1G · QuadraticRefinement

Identity (q, a, t, k). Public research-manifest route `komorebi_math.QuadraticRefinement`. Reflected implementation `komorebi_math.langlands.local.symplectic.QuadraticRefinement`. The root export resolves directly to this nested implementation owner. The umbrella `komorebi_math.langlands` package does not publish a second alias. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1G-LANGLANDS`.

Manifest signature (s).

```
(form: 'AlternatingForm', linear: 'tuple[int, ...]' = (), max_dimension: 'int' = 20) -> None
```

Definition. A dyadic quadratic refinement of a finite alternating form.

Contract (D, C, H, R, X). Fields, groups, characters, packets, and towers are finite explicit objects. Representation dimensions, actions, cocycles, and local parameters must be compatible. Completion reports require an explicit status for every declared gate. Public fields are `form`, `linear`, `max_dimension`. Principal public operations are `evaluate`, `polar_certificate`, `gauss_sum`, `arf_invariant`, `normalized_gauss_sign`.

Guarantees (E, P, B, K). Finite-field, finite-group, local-packet, ramification, and gate calculations are explicit and reproducible. Global completion objects remain conditional on their recorded gates. This API does not claim a proof of the general Langlands correspondence.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_langlands_local_heisenberg_symplectic.py`, `tests/test_langlands_ramification_global_completion.py`, `tests/test_langlands_finite_image.py`, `tests/test_langlands_local_fields_tame_gauss.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 57 and 58.

C.1G · RamificationCliffordTower

Identity (q, a, t, k). Public research-manifest route `komorebi_math.RamificationCliffordTower`. Reflected implementation `komorebi_math.langlands.ramification.towers.RamificationCliffordTower`. The root export resolves directly to this nested implementation owner. The umbrella `komorebi_math.langlands` package does not publish a second alias. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1G-LANGLANDS`.

Manifest signature (s).

```
(stages: 'Iterable[RamificationStage]') -> 'None'
```

Definition. A finite ramification-adapted chain of Clifford packet stages.

Contract (D, C, H, R, X). Fields, groups, characters, packets, and towers are finite explicit objects. Representation dimensions, actions, cocycles, and local parameters must be compatible. Completion reports require an explicit status for every declared gate. Principal public operations are `report`, `tensor_packet`.

Guarantees (E, P, B, K). Finite-field, finite-group, local-packet, ramification, and gate calculations are explicit and reproducible. Global completion objects remain conditional on their recorded gates. This API does not claim a proof of the general Langlands correspondence.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_langlands_ramification_global_completion.py`, `tests/test_langlands_finite_image.py`, `tests/test_langlands_local_heisenberg_symplectic.py`, `tests/test_langlands_local_fields_tame_gauss.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 57 and 58.

C.1G · RamificationStage

Identity (q, a, t, k). Public research-manifest route `komorebi_math.RamificationStage`. Reflected implementation `komorebi_math.langlands.ramification.towers.RamificationStage`. The root export resolves directly to this nested implementation owner. The umbrella `komorebi_math.langlands` package does not publish a second alias. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1G-LANGLANDS`.

Manifest signature (s).

```
(break_number: 'int', parent_order: 'int', child_order: 'int', orbit_size: 'int', multiplicity_dimension: 'int',
    ↪ packet: 'ProjectiveMultiplicityPacket | None' = None, label: 'str' = '') -> None
```

Definition. One finite stage in a declared ramification-adapted tower.

Contract (D, C, H, R, X). Fields, groups, characters, packets, and towers are finite explicit objects. Representation dimensions, actions, cocycles, and local parameters must be compatible. Completion reports require an explicit status for every declared gate. Public fields are `break_number`, `parent_order`, `child_order`, `orbit_size`, `multiplicity_dimension`, `packet`, `label`. Principal public operations are `reconstructed_dimension`.

Guarantees (E, P, B, K). Finite-field, finite-group, local-packet, ramification, and gate calculations are explicit and reproducible. Global completion objects remain conditional on their recorded gates. This API does not claim a proof of the general Langlands correspondence.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_langlands_ramification_global_completion.py`, `tests/test_langlands_finite_image.py`, `tests/test_langlands_local_heisenberg_symplectic.py`, `tests/test_langlands_local_fields_tame_gauss.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 57 and 58.

C.1G · SupportComparison

Identity (q, a, t, k). Public research-manifest route `komorebi_math.SupportComparison`. Reflected implementation `komorebi_math.langlands.global_.hecke.SupportComparison`. The root export resolves directly to this nested implementation owner. The umbrella `komorebi_math.langlands` package does not publish a second alias. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1G-LANGLANDS`.

Manifest signature (s).

```
(source_components: 'frozenset[str]', target_components: 'frozenset[str]', image_components: 'frozenset[str]',
   ↪  certificate: 'Certificate') -> None
```

Definition. A support-preserving or support-losing comparison diagnostic.

Contract (D, C, H, R, X). Fields, groups, characters, packets, and towers are finite explicit objects. Representation dimensions, actions, cocycles, and local parameters must be compatible. Completion reports require an explicit status for every declared gate. Public fields are `source_components`, `target_components`, `image_components`, `certificate`. Principal public operations are `construct`.

Guarantees (E, P, B, K). Finite-field, finite-group, local-packet, ramification, and gate calculations are explicit and reproducible. Global completion objects remain conditional on their recorded gates. This API does not claim a proof of the general Langlands correspondence.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_langlands_ramification_global_completion.py`, `tests/test_langlands_finite_image.py`, `tests/test_langlands_local_heisenberg_symplectic.py`, `tests/test_langlands_local_fields_tame_gauss.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 57 and 58.

C.1G · TameWeilPacket

Identity (q, a, t, k). Public research-manifest route `komorebi_math.TameWeilPacket`. Reflected implementation `komorebi_math.langlands.local.tame.TameWeilPacket`. The root export resolves directly to this nested implementation owner. The umbrella `komorebi_math.langlands` package does not publish a second alias. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1G-LANGLANDS`.

Manifest signature (s).

```
(orbit: 'FrobeniusOrbit', frobenius_scalar: 'complex' = 1.0) -> None
```

Definition. A finite matrix model of one regular tame Weil atom.

Contract (D, C, H, R, X). Fields, groups, characters, packets, and towers are finite explicit objects. Representation dimensions, actions, cocycles, and local parameters must be compatible. Completion reports require an explicit status for every declared gate. Public fields are `orbit`, `frobenius_scalar`. Principal public operations are `dimension`, `inertia_matrix`, `frobenius_matrix`, `determinant_orientation`, `relation_certificate`.

Guarantees (E, P, B, K). Finite-field, finite-group, local-packet, ramification, and gate calculations are explicit and reproducible. Global completion objects remain conditional on their recorded gates. This API does not claim a proof of the general Langlands correspondence.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_langlands_local_fields_tame_gauss.py`, `tests/test_langlands_finite_image.py`, `tests/test_langlands_local_heisenberg_symplectic.py`, `tests/test_langlands_ramification_global_completion.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 57 and 58.

C.1G · VisibilityTree

Identity (q, a, t, k). Public research-manifest route `komorebi_math.VisibilityTree`. Reflected implementation `komorebi_math.deformation.visibility.VisibilityTree`. The same runtime object is also available through `komorebi_math.deformation.VisibilityTree`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1G-DEFORMATION`.

Manifest signature (s).

```
(values: 'Iterable[Fraction | int]', *, prime: 'int') -> 'None'
```

Definition. Nested visibility partitions of finitely many exact local coordinates.

Contract (D, C, H, R, X). Complexes, modules, forests, and Frobenius data are finite. Prime-field operations require a valid prime, differential matrices must match the declared dimensions, and cochain data must satisfy the finite compatibility checks enforced by the implementation. Principal public operations are `separation_depth`, `pairwise_depths`, `partition`, `refinement_chain`, `ultrametric_certificate`.

Guarantees (E, P, B, K). Cohomology, support, visibility, and companion calculations are exact over the declared finite field or finite presentation. The records do not by themselves prove an unrestricted deformation or lifting theorem.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_deformation_forests_visibility_counting.py`, `tests/test_deformation_complex_support_companions.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapter 56.

C.1G · cyclic_group

Identity (q, a, t, k). Public research-manifest route `komorebi_math.cyclic_group`. Reflected implementation `komorebi_math.langlands.finite_image.groups.cyclic_group`. The root export resolves directly to this nested implementation owner. The umbrella `komorebi_math.langlands` package does not publish a second alias. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1G-LANGLANDS`.

Manifest signature (s).

```
(order: 'int') -> 'FiniteGroup[int]'
```

Definition. Return the additive cyclic group of the declared order.

Contract (D, C, H, R, X). Fields, groups, characters, packets, and towers are finite explicit objects. Representation dimensions, actions, cocycles, and local parameters must be compatible. Completion reports require an explicit status for every declared gate.

Guarantees (E, P, B, K). Finite-field, finite-group, local-packet, ramification, and gate calculations are explicit and reproducible. Global completion objects remain conditional on their recorded gates. This API does not claim a proof of the general Langlands correspondence.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_langlands_finite_image.py`, `tests/test_langlands_local_heisenberg_symplectic.py`, `tests/test_langlands_local_fields_tame_gauss.py`, `tests/test_langlands_ramification_global_completion.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 57 and 58.

C.1G · dyadic_sign_diagnostic

Identity (q, a, t, k). Public research-manifest route `komorebi_math.dyadic_sign_diagnostic`. Reflected implementation `komorebi_math.langlands.global_.dyadic.dyadic_sign_diagnostic`. The root export resolves directly to this nested implementation owner. The umbrella `komorebi_math.langlands` package does not publish a second alias. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1G-LANGLANDS`.

Manifest signature (s).

```
() -> 'DyadicSignDiagnostic'
```

Definition. Return the exact integral and residual sign packet diagnostic.

Contract (D, C, H, R, X). Fields, groups, characters, packets, and towers are finite explicit objects. Representation dimensions, actions, cocycles, and local parameters must be compatible. Completion reports require an explicit status for every declared gate.

Guarantees (E, P, B, K). Finite-field, finite-group, local-packet, ramification, and gate calculations are explicit and reproducible. Global completion objects remain conditional on their recorded gates. This API does not claim a proof of the general Langlands correspondence.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_langlands_ramification_global_completion.py`, `tests/test_langlands_finite_image.py`, `tests/test_langlands_local_heisenberg_symplectic.py`, `tests/test_langlands_local_fields_tame_gauss.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 57 and 58.

C.1G · finite_frobenius_separator

Identity (q, a, t, k). Public research-manifest route `komorebi_math.finite_frobenius_separator`. Reflected implementation `komorebi_math.deformation.companions.finite_frobenius_separator`. The same runtime object is also available through `komorebi_math.deformation.finite_frobenius_separator`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1G-DEFORMATION`.

Manifest signature (s).

```
(traces: 'Mapping[Hashable, Sequence[Any]]', *, constituent_count: 'int | None' = None) ->
    ↪ 'FrobeniusSeparationResult'
```

Definition. Select finitely many supplied places that separate trace columns.

Contract (D, C, H, R, X). Complexes, modules, forests, and Frobenius data are finite. Prime-field operations require a valid prime, differential matrices must match the declared dimensions, and cochain data must satisfy the finite compatibility checks enforced by the implementation.

Guarantees (E, P, B, K). Cohomology, support, visibility, and companion calculations are exact over the declared finite field or finite presentation. The records do not by themselves prove an unrestricted deformation or lifting theorem.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_deformation_complex_support_companions.py`, `tests/test_deformation_forests_visibility_counting.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapter 56.

C.1G · gauss_sum

Identity (q, a, t, k). Public research-manifest route `komorebi_math.gauss_sum`. Reflected implementation `komorebi_math.langlands.local.gauss.gauss_sum`. The root export resolves directly to this nested implementation owner. The umbrella `komorebi_math.langlands` package does not publish a second alias. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1G-LANGLANDS`.

Manifest signature (s).

```
(field: 'FiniteField', exponent: 'int', *, additive_frequency: 'int' = 1, tolerance: 'float' = 1e-09) ->
   ↪ 'GaussSumResult'
```

Definition. Compute one finite multiplicative and additive Gauss sum.

Contract (D, C, H, R, X). Fields, groups, characters, packets, and towers are finite explicit objects. Representation dimensions, actions, cocycles, and local parameters must be compatible. Completion reports require an explicit status for every declared gate.

Guarantees (E, P, B, K). Finite-field, finite-group, local-packet, ramification, and gate calculations are explicit and reproducible. Global completion objects remain conditional on their recorded gates. This API does not claim a proof of the general Langlands correspondence.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_langlands_local_fields_tame_gauss.py`, `tests/test_langlands_finite_image.py`, `tests/test_langlands_local_heisenberg_symplectic.py`, `tests/test_langlands_ramification_global_completion.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 57 and 58.

C.1G · mackey_tensor_packets

Identity (q, a, t, k). Public research-manifest route `komorebi_math.mackey_tensor_packets`. Reflected implementation `komorebi_math.langlands.finite_image.mackey.mackey_tensor_packets`. The root export resolves directly to this nested implementation owner. The umbrella `komorebi_math.langlands` package does not publish a second alias. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1G-LANGLANDS`.

Manifest signature (s).

```
(group: 'FiniteGroup[Element]', left: 'LinearCharacter[Element]', right: 'LinearCharacter[Element]') ->
   ↪ 'MackeyDecomposition[Element]'
```

Definition. Return the Mackey packets for two character-induced representations.

Contract (D, C, H, R, X). Fields, groups, characters, packets, and towers are finite explicit objects. Representation dimensions, actions, cocycles, and local parameters must be compatible. Completion reports require an explicit status for every declared gate.

Guarantees (E, P, B, K). Finite-field, finite-group, local-packet, ramification, and gate calculations are explicit and reproducible. Global completion objects remain conditional on their recorded gates. This API does not claim a proof of the general Langlands correspondence.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_langlands_finite_image.py`, `tests/test_langlands_local_heisenberg_symplectic.py`, `tests/test_langlands_local_fields_tame_gauss.py`, `tests/test_langlands_ramification_global_completion.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 57 and 58.

Registry, unified workflow, and root data (9 cards).

C.1G · DEFAULT_BEHAVIORAL_GOLDEN_FIXTURE

Identity (q, a, t, k). Public research-manifest route `komorebi_math.DEFAULT_BEHAVIORAL_GOLDEN_FIXTURE`. Reflected implementation `pathlib.Path`. This is a root data export rather than a callable facade. The public name maps to implementation object `Path`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `data`. Profile `C1G-ROOT-DATA`.

Manifest signature (s).

```
null
```

Definition. Default package path for the committed behavioral-golden fixture.

Contract (D, C, H, R, X). The exported value is a read-only package-data location. It may be inspected or passed to the matching loader, but it does not promise that the installation directory is writable.

Guarantees (E, P, B, K). The path resolves relative to the installed package and identifies the committed default resource for that installation. The file contents remain governed by the corresponding loader and compatibility contract.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_block11_golden_profiling_beta.py`, `tests/test_block15_stable_api.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 6 and 61.

C.1G · DEFAULT_BETA_API_MANIFEST

Identity (q, a, t, k). Public research-manifest route `komorebi_math.DEFAULT_BETA_API_MANIFEST`. Reflected implementation `pathlib.Path`. This is a root data export rather than a callable facade. The public name maps to implementation object `Path`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `data`. Profile `C1G-ROOT-DATA`.

Manifest signature (s).

```
null
```

Definition. Default package path for the frozen beta API manifest.

Contract (D, C, H, R, X). The exported value is a read-only package-data location. It may be inspected or passed to the matching loader, but it does not promise that the installation directory is writable.

Guarantees (E, P, B, K). The path resolves relative to the installed package and identifies the committed default resource for that installation. The file contents remain governed by the corresponding loader and compatibility contract.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_block11_golden_profiling_beta.py`, `tests/test_block15_stable_api.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 6 and 61.

C.1G · FeatureSpec

Identity (q, a, t, k). Public research-manifest route `komorebi_math.FeatureSpec`. Reflected implementation `komorebi_math.registry.FeatureSpec`. The same runtime object is also available through `komorebi_math.registry.FeatureSpec`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1G-REGISTRY`.

Manifest signature (s).

```
(key: 'str', module: 'str', title: 'str', status: 'FeatureStatus', summary: 'str', papers: 'tuple[str, ...]' = (),
   ↪ dependencies: 'tuple[str, ...]' = (), public_objects: 'tuple[str, ...]' = ()) -> None
```

Definition. Immutable metadata describing one registered package feature.

Contract (D, C, H, R, X). Feature names and metadata are explicit, finite, and validated before registration or lookup. Registry queries do not infer support that is absent from the committed feature record. Public fields are `key`, `module`, `title`, `status`, `summary`, `papers`, `dependencies`, `public_objects`. Principal public operations are `to_dict`.

Guarantees (E, P, B, K). Registry results reproduce the committed metadata and support status. They are descriptive records, not runtime capability probes or publication evidence.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused test module is `tests/test_certificates_registry.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 6 and 62.

C.1G · `FeatureStatus`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.FeatureStatus`. Reflected implementation `komorebi_math.registry.FeatureStatus`. The same runtime object is also available through `komorebi_math.registry.FeatureStatus`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1G-REGISTRY`.

Manifest signature (s).

```
(*values)
```

Definition. String-valued support status used by the feature registry.

Contract (D, C, H, R, X). Feature names and metadata are explicit, finite, and validated before registration or lookup. Registry queries do not infer support that is absent from the committed feature record.

Guarantees (E, P, B, K). Registry results reproduce the committed metadata and support status. They are descriptive records, not runtime capability probes or publication evidence.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused test module is `tests/test_certificates_registry.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 6 and 62.

C.1G · `StageSpec`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.StageSpec`. Reflected implementation `komorebi_math.unified.StageSpec`. The same runtime object is also available through `komorebi_math.unified.StageSpec`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1G-UNIFIED`.

Manifest signature (s).

```
(stage: 'UnifiedStage', meaning: 'str', implemented_in_alpha: 'bool', module: 'str') -> None
```

Definition. Immutable descriptor for one stage of the unified workflow.

Contract (D, C, H, R, X). Stage records use the fixed unified workflow vocabulary. A unified system delegates to its component stages and therefore requires every supplied component to satisfy its own public contract. Public fields are `stage`, `meaning`, `implemented_in_alpha`, `module`.

Guarantees (E, P, B, K). The unified layer preserves the outputs and limitations of the component stages. It provides one workflow vocabulary rather than a second implementation of the underlying mathematics.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused test module is `tests/test_unified_extensions_cli.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 62 and 64.

C.1G · UnifiedStage

Identity (q, a, t, k). Public research-manifest route `komorebi_math.UnifiedStage`. Reflected implementation `komorebi_math.unified.UnifiedStage`. The same runtime object is also available through `komorebi_math.unified.UnifiedStage`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1G-UNIFIED`.

Manifest signature (s).

```
(*values)
```

Definition. String-valued enumeration of the unified workflow stages.

Contract (D, C, H, R, X). Stage records use the fixed unified workflow vocabulary. A unified system delegates to its component stages and therefore requires every supplied component to satisfy its own public contract.

Guarantees (E, P, B, K). The unified layer preserves the outputs and limitations of the component stages. It provides one workflow vocabulary rather than a second implementation of the underlying mathematics.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused test module is `tests/test_unified_extensions_cli.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 62 and 64.

C.1G · UnifiedSystem

Identity (q, a, t, k). Public research-manifest route `komorebi_math.UnifiedSystem`. Reflected implementation `komorebi_math.unified.UnifiedSystem`. The same runtime object is also available through `komorebi_math.unified.UnifiedSystem`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `class`. Profile `C1G-UNIFIED`.

Manifest signature (s).

```
(grammar: 'OperatorGrammar | None' = None, realization: 'OperatorRealization | None' = None, seed_system:
    ↪  'SeedSystem | None' = None, packet_system: 'PacketSystem | None' = None, operators: 'Mapping[str, Any]' =
    ↪  <factory>, refinement_tower: 'RefinementTower | None' = None, metadata: 'dict[str, Any]' = <factory>) -> None
```

Definition. One finite system expressed in the common seed and packet language.

Contract (D, C, H, R, X). Stage records use the fixed unified workflow vocabulary. A unified system delegates to its component stages and therefore requires every supplied component to satisfy its own public contract. Public fields are `grammar`, `realization`, `seed_system`, `packet_system`, `operators`, `refinement_tower`, `metadata`. Principal public operations are `validate`, `analyze`, `block_operator`, `refine_coordinates`, `coarsen_coordinates`, `obstruction`, `summary`.

Guarantees (E, P, B, K). The unified layer preserves the outputs and limitations of the component stages. It provides one workflow vocabulary rather than a second implementation of the underlying mathematics.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. The focused test module is `tests/test_unified_extensions_cli.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 62 and 64.

C.1G · feature

Identity (q, a, t, k). Public research-manifest route `komorebi_math.feature`. Reflected implementation `komorebi_math.registry.feature`. The same runtime object is also available through `komorebi_math.registry.feature`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1G-REGISTRY`.

Manifest signature (s).

```
(key: 'str') -> 'FeatureSpec'
```

Definition. Return one registered feature specification by key.

Contract (D, C, H, R, X). Feature names and metadata are explicit, finite, and validated before registration or lookup. Registry queries do not infer support that is absent from the committed feature record.

Guarantees (E, P, B, K). Registry results reproduce the committed metadata and support status. They are descriptive records, not runtime capability probes or publication evidence.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_block13_cli_api.py`, `tests/test_block14_cli_api.py`, `tests/test_certificates_registry.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 6 and 62.

C.1G · features

Identity (q, a, t, k). Public research-manifest route `komorebi_math.features`. Reflected implementation `komorebi_math.registry.features`. The same runtime object is also available through `komorebi_math.registry.features`. The root route is governed by `[RESEARCH]` policy `pre-beta-additive`. It does not enlarge the frozen stable compatibility promise. Manifest kind `function`. Profile `C1G-REGISTRY`.

Manifest signature (s).

```
() -> 'tuple[FeatureSpec, ...]'
```

Definition. Return every registered feature specification in key order.

Contract (D, C, H, R, X). Feature names and metadata are explicit, finite, and validated before registration or lookup. Registry queries do not infer support that is absent from the committed feature record.

Guarantees (E, P, B, K). Registry results reproduce the committed metadata and support status. They are descriptive records, not runtime capability probes or publication evidence.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this root export. Focused test modules include `tests/test_certificates_registry.py`, `tests/test_unified_extensions_cli.py`. The independent verifier for the root namespace checks root identity, any published family alias, the recorded implementation owner, the committed signature, and representative construction or invocation. The source validator for the root namespace checks exact card order, record counts, unique anchors, and preservation of the technical content. See Chapters 6 and 62.

C.2 `komorebi_math.coordinates`: 237 entries

C.2A Exact linear algebra, combinatorial transforms, packet orderings, tensor transport, finite fields, and normalization: 60 entries

This domain section documents the exact finite machinery beneath higher coordinate constructions. It covers rational matrix conversion and elimination, Stirling and Lah connection maps, finite binomial and Möbius transforms, deterministic index permutations, tensor and covector transport, prime-field coordinate changes, and normalized generating-function coefficients. The cards distinguish exact algebraic identities from resource limits caused by dense materialization. They also make exponential subset and tensor growth explicit rather than hiding it behind a convenient constructor.

C.2A · `ExactLinearSystem`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinates.ExactLinearSystem`. Reflected implementation `komorebi_math.coordinates.exact.ExactLinearSystem`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `class`. Profile `C2-EXACT`.

Manifest signature (`s`).

```
(consistent: 'bool', rank: 'int', solution: 'Vector | None', nullspace_basis: 'tuple[Vector, ...]', pivot_columns:
   ↪  'tuple[int, ...]', residual: 'Vector') -> None
```

Definition. Exact result for a possibly rectangular linear system.

Contract (`D, C, H, R, X`). The record stores consistency, rank, one affine representative when one exists, a right-nullspace basis, pivot columns, and an exact residual. An inconsistent system has `solution=None`. The record reports the solver result and does not recompute it.

Guarantees (`E, P, B, K`). All arithmetic is exact over `Fraction`. No tolerance, floating backend, accelerator, network, or filesystem access is used. Entrywise work is linear in matrix size, dense multiplication is cubic for square matrices, and elimination-based operations are cubic in arithmetic-operation count. Rational numerator and denominator sizes may grow.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_exact.py` and the independent verifier for exact coordinates cover the applicable construction, inverse, invariant, failure, and resource behavior. The source validator for exact coordinates checks the exact signature, implementation identity, domain facade, alias routes, and anchor. See Chapters 15, 16, and 64.

C.2A · `FiniteFieldCoordinateTransform`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinates.FiniteFieldCoordinateTransform`. Reflected implementation `komorebi_math.coordinates.modular.FiniteFieldCoordinateTransform`. The same runtime object is also available through root route `komorebi_math.FiniteFieldCoordinateTransform`. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `class`. Profile `C2-MODULAR`.

Manifest signature (`s`).

```
(source: 'str', target: 'str', matrix: 'ModularMatrix', prime: 'int', theorem: 'str' = 'CC.M1', description: 'str'
   ↪  = '') -> None
```

Definition. An exact linear coordinate transform over `F_prime`.

Contract (`D, C, H, R, X`). The modulus must be prime, source and target names must be nonempty, and the matrix must be nonempty and rectangular. Entries are reduced modulo the prime. `apply` checks vector length. `inverse` requires a square nonsingular matrix. `then` requires a common prime and matching coordinate names. `certificate` records exact finite-field validation.

Guarantees (E, P, B, K). Arithmetic is exact modulo the selected prime. Certificate validity is based on exact modular products, not a probabilistic numerical test. Primality screening uses trial division, and dense inversion is cubic in matrix dimension in modular arithmetic.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_tensor_modular.py` and the independent verifier for exact coordinates cover the applicable construction, inverse, invariant, failure, and resource behavior. The source validator for exact coordinates checks the exact signature, implementation identity, domain facade, alias routes, and anchor. See Chapters 16 and 64.

C.2A · `apply_lowering_polynomial`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.apply_lowering_polynomial`. Reflected implementation `komorebi_math.coordinates.normalized.apply_lowering_polynomial`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-NORMALIZED`.

Manifest signature (s).

```
(coefficients: 'Sequence[Scalar]', polynomial: 'Iterable[Scalar]') -> 'Vector'
```

Definition. Apply a polynomial in the canonical lowering operator.

Contract (D, C, H, R, X). The lowering polynomial must contain at least one exact coefficient. The finite prefix is shortened by the polynomial degree. If the prefix is too short, the result is empty.

Guarantees (E, P, B, K). Normalization factors and outputs are exact rational values. The explicit diagonal transform is stored as a dense square matrix, while coefficient conversion itself is linear after construction. Applying a lowering polynomial uses work proportional to the output length times the polynomial length.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_normalized_persistence.py` and the independent verifier for exact coordinates cover the applicable construction, inverse, invariant, failure, and resource behavior. The source validator for exact coordinates checks the exact signature, implementation identity, domain facade, alias routes, and anchor. See Chapters 17, 21, and 64.

C.2A · `as_matrix`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.as_matrix`. Reflected implementation `komorebi_math.coordinates.exact.as_matrix`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-EXACT`.

Manifest signature (s).

```
(rows: 'Iterable[Iterable[Scalar]]') -> 'Matrix'
```

Definition. Return an immutable rectangular rational matrix.

Contract (D, C, H, R, X). Rows may be empty, but nonempty input must be rectangular. Every entry must be an exact rational coefficient and Boolean values are rejected. Ragged rows raise `DimensionError`.

Guarantees (E, P, B, K). All arithmetic is exact over `Fraction`. No tolerance, floating backend, accelerator, network, or filesystem access is used. Entrywise work is linear in matrix size, dense multiplication is cubic for square matrices, and elimination-based operations are cubic in arithmetic-operation count. Rational numerator and denominator sizes may grow.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_exact.py` and the independent verifier for exact coordinates cover the applicable construction, inverse, invariant, failure, and resource behavior. The source validator for exact coordinates checks the exact signature, implementation identity, domain facade, alias routes, and anchor. See Chapters 15, 16, and 64.

C.2A · as_vector

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.as_vector`. Reflected implementation `komorebi_math.coordinates.exact.as_vector`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-EXACT`.

Manifest signature (s).

```
(values: 'Iterable[Scalar]') -> 'Vector'
```

Definition. Return an immutable rational vector.

Contract (D, C, H, R, X). The input may be any finite iterable of integers or `Fraction` values. Boolean values and coefficients that cannot be converted to an exact rational raise `ValidationError`. The result is an immutable tuple of `Fraction` values.

Guarantees (E, P, B, K). All arithmetic is exact over `Fraction`. No tolerance, floating backend, accelerator, network, or filesystem access is used. Entrywise work is linear in matrix size, dense multiplication is cubic for square matrices, and elimination-based operations are cubic in arithmetic-operation count. Rational numerator and denominator sizes may grow.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_exact.py` and the independent verifier for exact coordinates cover the applicable construction, inverse, invariant, failure, and resource behavior. The source validator for exact coordinates checks the exact signature, implementation identity, domain facade, alias routes, and anchor. See Chapters 15, 16, and 64.

C.2A · binomial_transform

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.binomial_transform`. Reflected implementation `komorebi_math.coordinates.combinatorial.binomial_transform`. The same runtime object is also available through root route `komorebi_math.binomial_transform`. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-COMBINATORIAL`.

Manifest signature (s).

```
(size: 'int', *, inverse: 'bool' = False, source: 'str' = 'sequence values', target: 'str' = 'binomial transform',
    ↪ budget: 'CoordinateBudget' = CoordinateBudget(maximum_matrix_dimension=512, maximum_polynomial_degree=512,
    ↪ maximum_horizon=4096, maximum_exact_entries=1000000)) -> 'CoordinateTransform'
```

Definition. Return the finite binomial transform or its inverse.

Contract (D, C, H, R, X). The size must be positive and must fit the coordinate budget. `inverse=False` builds the lower-triangular binomial map. `inverse=True` builds its signed inverse and reverses the source and target labels.

Guarantees (E, P, B, K). Coefficients and transforms are exact integers or `Fraction` values. Materialized transforms use quadratic storage in their matrix dimension. For subset transforms the dimension is exponential in rank, so the explicit matrix has `4**rank` entries. The coordinate budget rejects oversized requests before construction.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_combinatorial.py` and `verification/block14_general_coordinate_audit.py` cover the applicable construction, inverse, invariant, failure, and resource behavior. The source validator for exact coordinates checks the exact signature, implementation identity, domain facade, alias routes, and anchor. See Chapters 17, 18, and 64.

C.2A · chinese_remainder_order

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.chinese_remainder_order`. Reflected implementation `komorebi_math.coordinates.packet.chinese_remainder_order`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-PACKET`.

Manifest signature (s).

```
(moduli: 'Sequence[int]') -> 'tuple[int, ...]'
```

Definition. Return flat indices ordered lexicographically by CRT residues.

Contract (D, C, H, R, X). At least one positive modulus is required and all moduli must be pairwise coprime. The output orders integers by their residue tuples in lexicographic order.

Guarantees (E, P, B, K). Every result is an exact finite permutation or its exact `CoordinateTransform`. Construction uses no numerical tolerance or optional backend. Storage is linear in the number of indices. Sorting-based orders require `O(n log n)` comparisons, while direct residue and digit orders require linear work.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_numeric_packet.py` and `verification/block14_general_coordinate_audit.py` cover the applicable construction, inverse, invariant, failure, and resource behavior. The source validator for exact coordinates checks the exact signature, implementation identity, domain facade, alias routes, and anchor. See Chapters 13, 18, and 64.

C.2A · chinese_remainder_transform

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.chinese_remainder_transform`. Reflected implementation `komorebi_math.coordinates.packet.chinese_remainder_transform`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-PACKET`.

Manifest signature (s).

```
(moduli: 'Sequence[int]') -> 'CoordinateTransform'
```

Definition. Return the permutation from integer order to CRT-residue order.

Contract (D, C, H, R, X). At least one positive modulus is required and all moduli must be pairwise coprime. The output orders integers by their residue tuples in lexicographic order.

Guarantees (E, P, B, K). Every result is an exact finite permutation or its exact `CoordinateTransform`. Construction uses no numerical tolerance or optional backend. Storage is linear in the number of indices. Sorting-based orders require `O(n log n)` comparisons, while direct residue and digit orders require linear work.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_numeric_packet.py` and `verification/block14_general_coordinate_audit.py` cover the applicable construction, inverse, invariant, failure, and resource behavior. The source validator for exact coordinates checks the exact signature, implementation identity, domain facade, alias routes, and anchor. See Chapters 13, 18, and 64.

C.2A · contragredient_transform

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.contragredient_transform`. Reflected implementation `komorebi_math.coordinates.tensor.contragredient_transform`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-TENSOR`.

Manifest signature (s).

```
(transform: 'CoordinateTransform') -> 'CoordinateTransform'
```

Definition. Return the induced transform on covector coordinates.

Contract (D, C, H, R, X). The coordinate transform must be square and invertible. Covectors use the inverse transpose of the vector coordinate matrix. Dimension failures propagate as explicit exceptions.

Guarantees (E, P, B, K). Transport is exact over rational coefficients. Inversion and similarity or congruence operations use dense exact elimination and multiplication. Kronecker dimensions multiply, so explicit tensor matrices can grow exponentially with tensor rank. No sparse or lazy representation is implied.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_tensor_modular.py` and the independent verifier for exact coordinates cover the applicable construction, inverse, invariant, failure, and resource behavior. The source validator for exact coordinates checks the exact signature, implementation identity, domain facade, alias routes, and anchor. See Chapters 16, 54, and 64.

C.2A · `convert_normalized_coefficients`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.convert_normalized_coefficients`. Reflected implementation `komorebi_math.coordinates.normalized.convert_normalized_coefficients`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-NORMALIZED`.

Manifest signature (s).

```
(coefficients: 'Sequence[Scalar]', source: 'WeightedMonomialBasis', target: 'WeightedMonomialBasis', *, budget:
    ↪ 'CoordinateBudget' = CoordinateBudget(maximum_matrix_dimension=512, maximum_polynomial_degree=512,
    ↪ maximum_horizon=4096, maximum_exact_entries=1000000)) -> 'Vector'
```

Definition. Convert a finite coefficient vector between normalized bases.

Contract (D, C, H, R, X). The finite coefficient sequence is converted to exact rationals. Empty input returns an empty tuple. Nonempty input applies the degree-matched normalization change.

Guarantees (E, P, B, K). Normalization factors and outputs are exact rational values. The explicit diagonal transform is stored as a dense square matrix, while coefficient conversion itself is linear after construction. Applying a lowering polynomial uses work proportional to the output length times the polynomial length.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_normalized_persistence.py` and the independent verifier for exact coordinates cover the applicable construction, inverse, invariant, failure, and resource behavior. The source validator for exact coordinates checks the exact signature, implementation identity, domain facade, alias routes, and anchor. See Chapters 17, 21, and 64.

C.2A · `coordinate_pairing`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.coordinate_pairing`. Reflected implementation `komorebi_math.coordinates.tensor.coordinate_pairing`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-TENSOR`.

Manifest signature (`S`).

```
(covector: 'Sequence[Scalar]', vector: 'Sequence[Scalar]') -> 'Fraction'
```

Definition. Return the exact natural pairing of coordinate components.

Contract (`D, C, H, R, X`). Vector and covector component sequences must have equal length. Values are converted to exact rationals and a mismatch raises `DimensionError`.

Guarantees (`E, P, B, K`). Transport is exact over rational coefficients. Inversion and similarity or congruence operations use dense exact elimination and multiplication. Kronecker dimensions multiply, so explicit tensor matrices can grow exponentially with tensor rank. No sparse or lazy representation is implied.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_tensor_modular.py` and the independent verifier for exact coordinates cover the applicable construction, inverse, invariant, failure, and resource behavior. The source validator for exact coordinates checks the exact signature, implementation identity, domain facade, alias routes, and anchor. See Chapters 16, 54, and 64.

C.2A · `determinant`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinates.determinant`. Reflected implementation `komorebi_math.coordinates.exact.determinant`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-EXACT`.

Manifest signature (`S`).

```
(matrix: 'Matrix') -> 'Fraction'
```

Definition. Return the exact determinant.

Contract (`D, C, H, R, X`). The matrix must be square. Rectangular input raises `DimensionError`. The determinant of the empty square matrix is one.

Guarantees (`E, P, B, K`). All arithmetic is exact over `Fraction`. No tolerance, floating backend, accelerator, network, or filesystem access is used. Entrywise work is linear in matrix size, dense multiplication is cubic for square matrices, and elimination-based operations are cubic in arithmetic-operation count. Rational numerator and denominator sizes may grow.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_exact.py` and the independent verifier for exact coordinates cover the applicable construction, inverse, invariant, failure, and resource behavior. The source validator for exact coordinates checks the exact signature, implementation identity, domain facade, alias routes, and anchor. See Chapters 15, 16, and 64.

C.2A · `digit_reversal_order`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinates.digit_reversal_order`. Reflected implementation `komorebi_math.coordinates.packet.digit_reversal_order`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-PACKET`.

Manifest signature (`S`).

```
(length: 'int', radix: 'int' = 2) -> 'tuple[int, ...]'
```

Definition. Return the fixed-width base-radix digit-reversal order.

Contract (`D, C, H, R, X`). Length and radix must be positive integers, and the length must be an exact power of the radix. Otherwise `DimensionError` or `ValidationError` is raised.

Guarantees (E, P, B, K). Every result is an exact finite permutation or its exact `CoordinateTransform`. Construction uses no numerical tolerance or optional backend. Storage is linear in the number of indices. Sorting-based orders require `O(n log n)` comparisons, while direct residue and digit orders require linear work.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_numeric_packet.py` and `verification/block14_general_coordinate_audit.py` cover the applicable construction, inverse, invariant, failure, and resource behavior. The source validator for exact coordinates checks the exact signature, implementation identity, domain facade, alias routes, and anchor. See Chapters 13, 18, and 64.

C.2A · `digit_reversal_transform`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.digit_reversal_transform`. Reflected implementation `komorebi_math.coordinates.packet.digit_reversal_transform`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-PACKET`.

Manifest signature (s).

```
(length: 'int', radix: 'int' = 2) -> 'CoordinateTransform'
```

Definition. Return the exact fixed-width digit-reversal permutation.

Contract (D, C, H, R, X). Length and radix must be positive integers, and the length must be an exact power of the radix. Otherwise `DimensionError` or `ValidationError` is raised.

Guarantees (E, P, B, K). Every result is an exact finite permutation or its exact `CoordinateTransform`. Construction uses no numerical tolerance or optional backend. Storage is linear in the number of indices. Sorting-based orders require `O(n log n)` comparisons, while direct residue and digit orders require linear work.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_numeric_packet.py` and `verification/block14_general_coordinate_audit.py` cover the applicable construction, inverse, invariant, failure, and resource behavior. The source validator for exact coordinates checks the exact signature, implementation identity, domain facade, alias routes, and anchor. See Chapters 13, 18, and 64.

C.2A · `direct_sum_transform`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.direct_sum_transform`. Reflected implementation `komorebi_math.coordinates.tensor.direct_sum_transform`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-TENSOR`.

Manifest signature (s).

```
(left: 'CoordinateTransform', right: 'CoordinateTransform') -> 'CoordinateTransform'
```

Definition. Return the block-diagonal direct sum of two coordinate transforms.

Contract (D, C, H, R, X). The function forms the exact block-diagonal direct sum. Rectangular transforms are allowed and the output dimensions are the sums of the input dimensions.

Guarantees (E, P, B, K). Transport is exact over rational coefficients. Inversion and similarity or congruence operations use dense exact elimination and multiplication. Kronecker dimensions multiply, so explicit tensor matrices can grow exponentially with tensor rank. No sparse or lazy representation is implied.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_tensor_modular.py` and the independent verifier for exact coordinates cover the applicable construction, inverse, invariant, failure, and resource behavior. The source validator for exact coordinates

checks the exact signature, implementation identity, domain facade, alias routes, and anchor. See Chapters 16, 54, and 64.

C.2A · divisor_zeta_transform

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.divisor_zeta_transform`. Reflected implementation `komorebi_math.coordinates.combinatorial.divisor_zeta_transform`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-COMBINATORIAL`.

Manifest signature (s).

```
(size: 'int', *, inverse: 'bool' = False, budget: 'CoordinateBudget' =
    ↪ CoordinateBudget(maximum_matrix_dimension=512, maximum_polynomial_degree=512, maximum_horizon=4096,
    ↪ maximum_exact_entries=1000000)) -> 'CoordinateTransform'
```

Definition. Return zeta or Möbius coordinates on the divisibility poset.

Contract (D, C, H, R, X). The size must be positive and must fit the coordinate budget. The forward map uses divisibility incidence. The inverse uses the classical Möbius function on quotient indices.

Guarantees (E, P, B, K). Coefficients and transforms are exact integers or `Fraction` values. Materialized transforms use quadratic storage in their matrix dimension. For subset transforms the dimension is exponential in rank, so the explicit matrix has `4**rank` entries. The coordinate budget rejects oversized requests before construction.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_combinatorial.py` and `verification/block14_general_coordinate_audit.py` cover the applicable construction, inverse, invariant, failure, and resource behavior. The source validator for exact coordinates checks the exact signature, implementation identity, domain facade, alias routes, and anchor. See Chapters 17, 18, and 64.

C.2A · falling_factorial_to_monomial_transform

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.falling_factorial_to_monomial_transform`. Reflected implementation `komorebi_math.coordinates.combinatorial.falling_factorial_to_monomial_transform`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-COMBINATORIAL`.

Manifest signature (s).

```
(degree: 'int', *, budget: 'CoordinateBudget' = CoordinateBudget(maximum_matrix_dimension=512,
    ↪ maximum_polynomial_degree=512, maximum_horizon=4096, maximum_exact_entries=1000000)) ->
    ↪ 'CoordinateTransform'
```

Definition. Return falling-factorial coefficients in the monomial basis.

Contract (D, C, H, R, X). The degree must be a nonnegative integer and the coordinate budget must permit a square matrix of size `degree+1`. The returned `CoordinateTransform` is an exact Stirling or Lah connection matrix with named source and target coefficient systems.

Guarantees (E, P, B, K). Coefficients and transforms are exact integers or `Fraction` values. Materialized transforms use quadratic storage in their matrix dimension. For subset transforms the dimension is exponential in rank, so the explicit matrix has `4**rank` entries. The coordinate budget rejects oversized requests before construction.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_combinatorial.py` and `verification/block14_general_coordinate_audit.py` cover the applicable construction, inverse, invariant, failure, and resource behavior. The source validator for exact coordinates checks the exact signature, implementation identity, domain facade, alias routes, and anchor. See Chapters 17, 18, and 64.

C.2A · forward_difference_transform

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.forward_difference_transform`. Reflected implementation `komorebi_math.coordinates.combinatorial.forward_difference_transform`. The same runtime object is also available through root route `komorebi_math.forward_difference_transform` and stable routes `komorebi_math.stable.forward_difference_transform`, `komorebi_math.v1.forward_difference_transform`, and `komorebi_math.api.v1.forward_difference_transform`. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Only the listed stable routes carry the frozen 1.x compatibility promise. Manifest kind `function`. Profile `C2-COMBINATORIAL`.

Manifest signature (s).

```
(size: 'int', *, budget: 'CoordinateBudget' = CoordinateBudget(maximum_matrix_dimension=512,
    ↪ maximum_polynomial_degree=512, maximum_horizon=4096, maximum_exact_entries=1000000)) ->
    ↪ 'CoordinateTransform'
```

Definition. Map sampled values `f(0),...,f(N)` to forward differences at zero.

Contract (D, C, H, R, X). The size must be positive and must fit the coordinate budget. The analysis map sends consecutive samples to forward differences at zero. The evaluation map is its exact inverse.

Guarantees (E, P, B, K). Coefficients and transforms are exact integers or `Fraction` values. Materialized transforms use quadratic storage in their matrix dimension. For subset transforms the dimension is exponential in rank, so the explicit matrix has `4**rank` entries. The coordinate budget rejects oversized requests before construction.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_combinatorial.py` and `verification/block14_general_coordinate_audit.py` cover the applicable construction, inverse, invariant, failure, and resource behavior. The source validator for exact coordinates checks the exact signature, implementation identity, domain facade, alias routes, and anchor. See Chapters 17, 18, and 64.

C.2A · identity_matrix

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.identity_matrix`. Reflected implementation `komorebi_math.coordinates.exact.identity_matrix`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-EXACT`.

Manifest signature (s).

```
(size: 'int') -> 'Matrix'
```

Definition. Return a rational identity matrix.

Contract (D, C, H, R, X). The size must be a nonnegative integer. Negative sizes raise `DimensionError`. Size zero returns the empty exact identity.

Guarantees (E, P, B, K). All arithmetic is exact over `Fraction`. No tolerance, floating backend, accelerator, network, or filesystem access is used. Entrywise work is linear in matrix size, dense multiplication is cubic for square matrices, and elimination-based operations are cubic in arithmetic-operation count. Rational numerator and denominator sizes may grow.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_exact.py` and the independent verifier for exact coordinates cover the applicable construction, inverse, invariant, failure, and resource behavior. The source validator for exact coordinates checks the exact signature, implementation identity, domain facade, alias routes, and anchor. See Chapters 15, 16, and 64.

C.2A · inverse_matrix

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.inverse_matrix`. Reflected implementation `komorebi_math.coordinates.exact.inverse_matrix`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-EXACT`.

Manifest signature (s).

```
(matrix: 'Matrix') -> 'Matrix'
```

Definition. Return the exact inverse of a nonsingular square matrix.

Contract (D, C, H, R, X). The matrix must be square and nonsingular. Rectangular input raises `DimensionError`. Singular input raises `ValidationError` after exact Gauss-Jordan reduction.

Guarantees (E, P, B, K). All arithmetic is exact over `Fraction`. No tolerance, floating backend, accelerator, network, or filesystem access is used. Entrywise work is linear in matrix size, dense multiplication is cubic for square matrices, and elimination-based operations are cubic in arithmetic-operation count. Rational numerator and denominator sizes may grow.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_exact.py` and the independent verifier for exact coordinates cover the applicable construction, inverse, invariant, failure, and resource behavior. The source validator for exact coordinates checks the exact signature, implementation identity, domain facade, alias routes, and anchor. See Chapters 15, 16, and 64.

C.2A · lah_number

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.lah_number`. Reflected implementation `komorebi_math.coordinates.combinatorial.lah_number`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-COMBINATORIAL`.

Manifest signature (s).

```
(n: 'int', k: 'int') -> 'int'
```

Definition. Return the unsigned Lah number `L(n,k)`.

Contract (D, C, H, R, X). `n` and `k` must be nonnegative integers and Boolean values are invalid. Values with `k>n` return zero, with the conventional exception `L(0,0)=1`. The result is an exact Python integer.

Guarantees (E, P, B, K). Coefficients and transforms are exact integers or `Fraction` values. Materialized transforms use quadratic storage in their matrix dimension. For subset transforms the dimension is exponential in rank, so the explicit matrix has `4**rank` entries. The coordinate budget rejects oversized requests before construction.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_combinatorial.py` and `verification/block14_general_coordinate_audit.py` cover the applicable construction, inverse, invariant, failure, and resource behavior. The source validator for exact coordinates checks the exact signature, implementation identity, domain facade, alias routes, and anchor. See Chapters 17, 18, and 64.

C.2A · matrix_add

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.matrix_add`. Reflected implementation `komorebi_math.coordinates.exact.matrix_add`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-EXACT`.

Manifest signature (s).

```
(left: 'Matrix', right: 'Matrix') -> 'Matrix'
```

Definition. Add matrices with equal shape.

Contract (D, C, H, R, X). Both matrices must have identical shape. A mismatch raises `DimensionError`. Addition is entrywise over exact rational coefficients.

Guarantees (E, P, B, K). All arithmetic is exact over `Fraction`. No tolerance, floating backend, accelerator, network, or filesystem access is used. Entrywise work is linear in matrix size, dense multiplication is cubic for square matrices, and elimination-based operations are cubic in arithmetic-operation count. Rational numerator and denominator sizes may grow.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_exact.py` and the independent verifier for exact coordinates cover the applicable construction, inverse, invariant, failure, and resource behavior. The source validator for exact coordinates checks the exact signature, implementation identity, domain facade, alias routes, and anchor. See Chapters 15, 16, and 64.

C.2A · `matrix_from_columns`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.matrix_from_columns`. Reflected implementation `komorebi_math.coordinates.exact.matrix_from_columns`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-EXACT`.

Manifest signature (s).

```
(columns: 'Sequence[Sequence[Scalar]]') -> 'Matrix'
```

Definition. Build a matrix from column vectors.

Contract (D, C, H, R, X). All column vectors must have the same length and every value must be exactly rational. Unequal lengths raise `DimensionError`. An empty column list returns the empty matrix.

Guarantees (E, P, B, K). All arithmetic is exact over `Fraction`. No tolerance, floating backend, accelerator, network, or filesystem access is used. Entrywise work is linear in matrix size, dense multiplication is cubic for square matrices, and elimination-based operations are cubic in arithmetic-operation count. Rational numerator and denominator sizes may grow.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_exact.py` and the independent verifier for exact coordinates cover the applicable construction, inverse, invariant, failure, and resource behavior. The source validator for exact coordinates checks the exact signature, implementation identity, domain facade, alias routes, and anchor. See Chapters 15, 16, and 64.

C.2A · `matrix_multiply`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.matrix_multiply`. Reflected implementation `komorebi_math.coordinates.exact.matrix_multiply`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-EXACT`.

Manifest signature (`S`).

```
(left: 'Matrix', right: 'Matrix') -> 'Matrix'
```

Definition. Return the exact matrix product.

Contract (`D, C, H, R, X`). The left column count must equal the right row count. A mismatch raises `DimensionError`. Empty and zero-width cases retain their mathematically defined finite shapes.

Guarantees (`E, P, B, K`). All arithmetic is exact over `Fraction`. No tolerance, floating backend, accelerator, network, or filesystem access is used. Entrywise work is linear in matrix size, dense multiplication is cubic for square matrices, and elimination-based operations are cubic in arithmetic-operation count. Rational numerator and denominator sizes may grow.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_exact.py` and the independent verifier for exact coordinates cover the applicable construction, inverse, invariant, failure, and resource behavior. The source validator for exact coordinates checks the exact signature, implementation identity, domain facade, alias routes, and anchor. See Chapters 15, 16, and 64.

C.2A · `matrix_power`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinates.matrix_power`. Reflected implementation `komorebi_math.coordinates.exact.matrix_power`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-EXACT`.

Manifest signature (`S`).

```
(matrix: 'Matrix', exponent: 'int') -> 'Matrix'
```

Definition. Return a nonnegative exact matrix power.

Contract (`D, C, H, R, X`). The matrix must be square and the exponent must be a nonnegative integer that is not Boolean. Invalid shape raises `DimensionError` and an invalid exponent raises `ValidationError`.

Guarantees (`E, P, B, K`). All arithmetic is exact over `Fraction`. No tolerance, floating backend, accelerator, network, or filesystem access is used. Entrywise work is linear in matrix size, dense multiplication is cubic for square matrices, and elimination-based operations are cubic in arithmetic-operation count. Rational numerator and denominator sizes may grow.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_exact.py` and the independent verifier for exact coordinates cover the applicable construction, inverse, invariant, failure, and resource behavior. The source validator for exact coordinates checks the exact signature, implementation identity, domain facade, alias routes, and anchor. See Chapters 15, 16, and 64.

C.2A · `matrix_rank`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinates.matrix_rank`. Reflected implementation `komorebi_math.coordinates.exact.matrix_rank`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-EXACT`.

Manifest signature (`S`).

```
(matrix: 'Matrix') -> 'int'
```

Definition. Return exact rational rank.

Contract (`D, C, H, R, X`). The result is the number of pivot columns in exact reduced row echelon form. No tolerance or numerical rank threshold is used.

Guarantees (`E, P, B, K`). All arithmetic is exact over `Fraction`. No tolerance, floating backend, accelerator, network, or filesystem access is used. Entrywise work is linear in matrix size, dense multiplication is cubic for square matrices,

and elimination-based operations are cubic in arithmetic-operation count. Rational numerator and denominator sizes may grow.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_exact.py` and the independent verifier for exact coordinates cover the applicable construction, inverse, invariant, failure, and resource behavior. The source validator for exact coordinates checks the exact signature, implementation identity, domain facade, alias routes, and anchor. See Chapters 15, 16, and 64.

C.2A · `matrix_shape`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.matrix_shape`. Reflected implementation `komorebi_math.coordinates.exact.matrix_shape`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-EXACT`.

Manifest signature (s).

```
(matrix: 'Matrix') -> 'tuple[int, int]'
```

Definition. Return the matrix shape.

Contract (D, C, H, R, X). The input is an immutable rectangular matrix. The empty matrix has shape `(0, 0)`. The function performs no numerical coercion.

Guarantees (E, P, B, K). All arithmetic is exact over `Fraction`. No tolerance, floating backend, accelerator, network, or filesystem access is used. Entrywise work is linear in matrix size, dense multiplication is cubic for square matrices, and elimination-based operations are cubic in arithmetic-operation count. Rational numerator and denominator sizes may grow.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_exact.py` and the independent verifier for exact coordinates cover the applicable construction, inverse, invariant, failure, and resource behavior. The source validator for exact coordinates checks the exact signature, implementation identity, domain facade, alias routes, and anchor. See Chapters 15, 16, and 64.

C.2A · `matrix_subtract`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.matrix_subtract`. Reflected implementation `komorebi_math.coordinates.exact.matrix_subtract`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-EXACT`.

Manifest signature (s).

```
(left: 'Matrix', right: 'Matrix') -> 'Matrix'
```

Definition. Subtract matrices with equal shape.

Contract (D, C, H, R, X). Both matrices must have identical shape. A mismatch raises `DimensionError`. Subtraction is entrywise over exact rational coefficients.

Guarantees (E, P, B, K). All arithmetic is exact over `Fraction`. No tolerance, floating backend, accelerator, network, or filesystem access is used. Entrywise work is linear in matrix size, dense multiplication is cubic for square matrices, and elimination-based operations are cubic in arithmetic-operation count. Rational numerator and denominator sizes may grow.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_exact.py` and the independent verifier for exact coordinates cover the applicable construction, inverse, invariant, failure, and resource behavior. The source validator for exact coordinates checks the exact signature, implementation identity, domain facade, alias routes, and anchor. See Chapters 15, 16, and 64.

C.2A · matrix_vector_multiply

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.matrix_vector_multiply`. Reflected implementation `komorebi_math.coordinates.exact.matrix_vector_multiply`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-EXACT`.

Manifest signature (s).

```
(matrix: 'Matrix', vector: 'Sequence[Scalar]') -> 'Vector'
```

Definition. Return an exact matrix vector product.

Contract (D, C, H, R, X). The vector length must equal the matrix column count. Inputs are converted to exact rationals and a mismatch raises `DimensionError`.

Guarantees (E, P, B, K). All arithmetic is exact over `Fraction`. No tolerance, floating backend, accelerator, network, or filesystem access is used. Entrywise work is linear in matrix size, dense multiplication is cubic for square matrices, and elimination-based operations are cubic in arithmetic-operation count. Rational numerator and denominator sizes may grow.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_exact.py` and the independent verifier for exact coordinates cover the applicable construction, inverse, invariant, failure, and resource behavior. The source validator for exact coordinates checks the exact signature, implementation identity, domain facade, alias routes, and anchor. See Chapters 15, 16, and 64.

C.2A · mobius_integer

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.mobius_integer`. Reflected implementation `komorebi_math.coordinates.combinatorial.mobius_integer`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-COMBINATORIAL`.

Manifest signature (s).

```
(value: 'int') -> 'int'
```

Definition. Return the classical Möbius function of a positive integer.

Contract (D, C, H, R, X). The argument must be a positive integer that is not Boolean. Invalid input raises `ValidationError`. The result is exactly `-1`, `0`, or `1`.

Guarantees (E, P, B, K). Coefficients and transforms are exact integers or `Fraction` values. Materialized transforms use quadratic storage in their matrix dimension. For subset transforms the dimension is exponential in rank, so the explicit matrix has `4**rank` entries. The coordinate budget rejects oversized requests before construction.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_combinatorial.py` and `verification/block14_general_coordinate_audit.py` cover the applicable construction, inverse, invariant, failure, and resource behavior. The source validator for exact coordinates checks the exact signature, implementation identity, domain facade, alias routes, and anchor. See Chapters 17, 18, and 64.

C.2A · monomial_to_falling_factorial_transform

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.monomial_to_falling_factorial_transform`. Reflected implementation `komorebi_math.coordinates.combinatorial.monomial_to_falling_factorial_transform`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-COMBINATORIAL`.

Manifest signature (s).

```
(degree: 'int', *, budget: 'CoordinateBudget' = CoordinateBudget(maximum_matrix_dimension=512,
    ↪ maximum_polynomial_degree=512, maximum_horizon=4096, maximum_exact_entries=1000000)) ->
    ↪ 'CoordinateTransform'
```

Definition. Return monomial coefficients in the falling-factorial basis.

Contract (D, C, H, R, X). The degree must be a nonnegative integer and the coordinate budget must permit a square matrix of size `degree+1`. The returned `CoordinateTransform` is an exact Stirling or Lah connection matrix with named source and target coefficient systems.

Guarantees (E, P, B, K). Coefficients and transforms are exact integers or `Fraction` values. Materialized transforms use quadratic storage in their matrix dimension. For subset transforms the dimension is exponential in rank, so the explicit matrix has `4**rank` entries. The coordinate budget rejects oversized requests before construction.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_combinatorial.py` and `verification/block14_general_coordinate_audit.py` cover the applicable construction, inverse, invariant, failure, and resource behavior. The source validator for exact coordinates checks the exact signature, implementation identity, domain facade, alias routes, and anchor. See Chapters 17, 18, and 64.

C.2A · `monomial_to_rising_factorial_transform`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.monomial_to_rising_factorial_transform`. Reflected implementation `komorebi_math.coordinates.combinatorial.monomial_to_rising_factorial_transform`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-COMBINATORIAL`.

Manifest signature (s).

```
(degree: 'int', *, budget: 'CoordinateBudget' = CoordinateBudget(maximum_matrix_dimension=512,
    ↪ maximum_polynomial_degree=512, maximum_horizon=4096, maximum_exact_entries=1000000)) ->
    ↪ 'CoordinateTransform'
```

Definition. Return monomial coefficients in the rising-factorial basis.

Contract (D, C, H, R, X). The degree must be a nonnegative integer and the coordinate budget must permit a square matrix of size `degree+1`. The returned `CoordinateTransform` is an exact Stirling or Lah connection matrix with named source and target coefficient systems.

Guarantees (E, P, B, K). Coefficients and transforms are exact integers or `Fraction` values. Materialized transforms use quadratic storage in their matrix dimension. For subset transforms the dimension is exponential in rank, so the explicit matrix has `4**rank` entries. The coordinate budget rejects oversized requests before construction.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_combinatorial.py` and `verification/block14_general_coordinate_audit.py` cover the applicable construction, inverse, invariant, failure, and resource behavior. The source validator for exact coordinates checks the exact signature, implementation identity, domain facade, alias routes, and anchor. See Chapters 17, 18, and 64.

C.2A · `newton_evaluation_transform`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.newton_evaluation_transform`. Reflected implementation `komorebi_math.coordinates.combinatorial.newton_evaluation_transform`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-COMBINATORIAL`.

Manifest signature (s).

```
(size: 'int', *, budget: 'CoordinateBudget' = CoordinateBudget(maximum_matrix_dimension=512,
    ↪ maximum_polynomial_degree=512, maximum_horizon=4096, maximum_exact_entries=1000000)) ->
    ↪ 'CoordinateTransform'
```

Definition. Map forward differences at zero to values at consecutive integers.

Contract (D, C, H, R, X). The size must be positive and must fit the coordinate budget. The analysis map sends consecutive samples to forward differences at zero. The evaluation map is its exact inverse.

Guarantees (E, P, B, K). Coefficients and transforms are exact integers or `Fraction` values. Materialized transforms use quadratic storage in their matrix dimension. For subset transforms the dimension is exponential in rank, so the explicit matrix has `4**rank` entries. The coordinate budget rejects oversized requests before construction.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_combinatorial.py` and `verification/block14_general_coordinate_audit.py` cover the applicable construction, inverse, invariant, failure, and resource behavior. The source validator for exact coordinates checks the exact signature, implementation identity, domain facade, alias routes, and anchor. See Chapters 17, 18, and 64.

C.2A · `normalization_change_transform`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.normalization_change_transform`. Reflected implementation `komorebi_math.coordinates.normalized.normalization_change_transform`. The same runtime object is also available through root route `komorebi_math.normalization_change_transform`. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-NORMALIZED`.

Manifest signature (s).

```
(source: 'WeightedMonomialBasis', target: 'WeightedMonomialBasis', degree: 'int', *, budget: 'CoordinateBudget' =
    ↪ CoordinateBudget(maximum_matrix_dimension=512, maximum_polynomial_degree=512, maximum_horizon=4096,
    ↪ maximum_exact_entries=1000000)) -> 'CoordinateTransform'
```

Definition. Return the diagonal map preserving the underlying formal series.

Contract (D, C, H, R, X). Source and target must be weighted monomial bases. Degree must pass the coordinate budget. The result is the diagonal map with entry `target.gamma(n)/source.gamma(n)`.

Guarantees (E, P, B, K). Normalization factors and outputs are exact rational values. The explicit diagonal transform is stored as a dense square matrix, while coefficient conversion itself is linear after construction. Applying a lowering polynomial uses work proportional to the output length times the polynomial length.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_normalized_persistence.py` and the independent verifier for exact coordinates cover the applicable construction, inverse, invariant, failure, and resource behavior. The source validator for exact coordinates checks the exact signature, implementation identity, domain facade, alias routes, and anchor. See Chapters 17, 21, and 64.

C.2A · `nullspace`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.nullspace`. Reflected implementation `komorebi_math.coordinates.exact.nullspace`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-EXACT`.

Manifest signature (`s`).

```
(matrix: 'Matrix') -> 'tuple[Vector, ...]'
```

Definition. Return an exact basis for the right nullspace.

Contract (`D, C, H, R, X`). The function returns a deterministic exact basis for the right nullspace using free columns from reduced row echelon form. A matrix with zero columns has an empty basis.

Guarantees (`E, P, B, K`). All arithmetic is exact over `Fraction`. No tolerance, floating backend, accelerator, network, or filesystem access is used. Entrywise work is linear in matrix size, dense multiplication is cubic for square matrices, and elimination-based operations are cubic in arithmetic-operation count. Rational numerator and denominator sizes may grow.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_exact.py` and the independent verifier for exact coordinates cover the applicable construction, inverse, invariant, failure, and resource behavior. The source validator for exact coordinates checks the exact signature, implementation identity, domain facade, alias routes, and anchor. See Chapters 15, 16, and 64.

C.2A · `pairing_invariant`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinates.pairing_invariant`. Reflected implementation `komorebi_math.coordinates.tensor.pairing_invariant`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-TENSOR`.

Manifest signature (`s`).

```
(covector: 'Sequence[Scalar]', vector: 'Sequence[Scalar]', transform: 'CoordinateTransform') -> 'bool'
```

Definition. Return whether vector and covector transport preserves their pairing.

Contract (`D, C, H, R, X`). The vector must match the transform source dimension and the covector must have the same length. The result compares source and transported pairings by exact equality.

Guarantees (`E, P, B, K`). Transport is exact over rational coefficients. Inversion and similarity or congruence operations use dense exact elimination and multiplication. Kronecker dimensions multiply, so explicit tensor matrices can grow exponentially with tensor rank. No sparse or lazy representation is implied.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_tensor_modular.py` and the independent verifier for exact coordinates cover the applicable construction, inverse, invariant, failure, and resource behavior. The source validator for exact coordinates checks the exact signature, implementation identity, domain facade, alias routes, and anchor. See Chapters 16, 54, and 64.

C.2A · `reduce_exact_transform_mod_prime`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinates.reduce_exact_transform_mod_prime`. Reflected implementation `komorebi_math.coordinates.modular.reduce_exact_transform_mod_prime`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-MODULAR`.

Manifest signature (s).

```
(transform: 'CoordinateTransform', prime: 'int') -> 'FiniteFieldCoordinateTransform'
```

Definition. Reduce a rational coordinate transform modulo a prime.

Contract (D, C, H, R, X). The modulus must be prime and every rational denominator in the exact transform must be invertible modulo that prime. Failure raises `ValidationError`.

Guarantees (E, P, B, K). Arithmetic is exact modulo the selected prime. Certificate validity is based on exact modular products, not a probabilistic numerical test. Primality screening uses trial division, and dense inversion is cubic in matrix dimension in modular arithmetic.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_tensor_modular.py` and the independent verifier for exact coordinates cover the applicable construction, inverse, invariant, failure, and resource behavior. The source validator for exact coordinates checks the exact signature, implementation identity, domain facade, alias routes, and anchor. See Chapters 16 and 64.

C.2A · `residue_class_order`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.residue_class_order`. Reflected implementation `komorebi_math.coordinates.packet.residue_class_order`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-PACKET`.

Manifest signature (s).

```
(length: 'int', modulus: 'int') -> 'tuple[int, ...]'
```

Definition. Return indices grouped by residue modulo `modulus`.

Contract (D, C, H, R, X). Length and modulus must be positive integers. Indices are grouped by residue class in increasing residue order, with the original order preserved within each lane.

Guarantees (E, P, B, K). Every result is an exact finite permutation or its exact `CoordinateTransform`. Construction uses no numerical tolerance or optional backend. Storage is linear in the number of indices. Sorting-based orders require `O(n log n)` comparisons, while direct residue and digit orders require linear work.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_numeric_packet.py` and `verification/block14_general_coordinate_audit.py` cover the applicable construction, inverse, invariant, failure, and resource behavior. The source validator for exact coordinates checks the exact signature, implementation identity, domain facade, alias routes, and anchor. See Chapters 13, 18, and 64.

C.2A · `residue_class_transform`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.residue_class_transform`. Reflected implementation `komorebi_math.coordinates.packet.residue_class_transform`. The same runtime object is also available through root route `komorebi_math.residue_class_transform`. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-PACKET`.

Manifest signature (`s`).

```
(length: 'int', modulus: 'int') -> 'CoordinateTransform'
```

Definition. Return the exact polyphase permutation into residue-lane order.

Contract (`D, C, H, R, X`). Length and modulus must be positive integers. Indices are grouped by residue class in increasing residue order, with the original order preserved within each lane.

Guarantees (`E, P, B, K`). Every result is an exact finite permutation or its exact `CoordinateTransform`. Construction uses no numerical tolerance or optional backend. Storage is linear in the number of indices. Sorting-based orders require `O(n log n)` comparisons, while direct residue and digit orders require linear work.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_numeric_packet.py` and `verification/block14_general_coordinate_audit.py` cover the applicable construction, inverse, invariant, failure, and resource behavior. The source validator for exact coordinates checks the exact signature, implementation identity, domain facade, alias routes, and anchor. See Chapters 13, 18, and 64.

C.2A · `rising_factorial_to_monomial_transform`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinates.rising_factorial_to_monomial_transform`. Reflected implementation `komorebi_math.coordinates.combinatorial.rising_factorial_to_monomial_transform`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-COMBINATORIAL`.

Manifest signature (`s`).

```
(degree: 'int', *, budget: 'CoordinateBudget' = CoordinateBudget(maximum_matrix_dimension=512,
    ↪ maximum_polynomial_degree=512, maximum_horizon=4096, maximum_exact_entries=1000000)) ->
    ↪ 'CoordinateTransform'
```

Definition. Return rising-factorial coefficients in the monomial basis.

Contract (`D, C, H, R, X`). The degree must be a nonnegative integer and the coordinate budget must permit a square matrix of size `degree+1`. The returned `CoordinateTransform` is an exact Stirling or Lah connection matrix with named source and target coefficient systems.

Guarantees (`E, P, B, K`). Coefficients and transforms are exact integers or `Fraction` values. Materialized transforms use quadratic storage in their matrix dimension. For subset transforms the dimension is exponential in rank, so the explicit matrix has `4**rank` entries. The coordinate budget rejects oversized requests before construction.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_combinatorial.py` and `verification/block14_general_coordinate_audit.py` cover the applicable construction, inverse, invariant, failure, and resource behavior. The source validator for exact coordinates checks the exact signature, implementation identity, domain facade, alias routes, and anchor. See Chapters 17, 18, and 64.

C.2A · `rising_to_falling_factorial_transform`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinates.rising_to_falling_factorial_transform`. Reflected implementation `komorebi_math.coordinates.combinatorial.rising_to_falling_factorial_transform`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-COMBINATORIAL`.

Manifest signature (`s`).

```
(degree: 'int', *, budget: 'CoordinateBudget' = CoordinateBudget(maximum_matrix_dimension=512,
    ↪ maximum_polynomial_degree=512, maximum_horizon=4096, maximum_exact_entries=1000000)) ->
    ↪ 'CoordinateTransform'
```

Definition. Return rising-factorial coordinates in the falling-factorial basis.

Contract (`D, C, H, R, X`). The degree must be a nonnegative integer and the coordinate budget must permit a square matrix of size `degree+1`. The returned `CoordinateTransform` is an exact Stirling or Lah connection matrix with named source and target coefficient systems.

Guarantees (`E, P, B, K`). Coefficients and transforms are exact integers or `Fraction` values. Materialized transforms use quadratic storage in their matrix dimension. For subset transforms the dimension is exponential in rank, so the explicit matrix has `4**rank` entries. The coordinate budget rejects oversized requests before construction.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_combinatorial.py` and `verification/block14_general_coordinate_audit.py` cover the applicable construction, inverse, invariant, failure, and resource behavior. The source validator for exact coordinates checks the exact signature, implementation identity, domain facade, alias routes, and anchor. See Chapters 17, 18, and 64.

C.2A · `rref`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinates.rref`. Reflected implementation `komorebi_math.coordinates.exact.rref`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-EXACT`.

Manifest signature (`s`).

```
(matrix: 'Matrix') -> 'tuple[Matrix, tuple[int, ...]]'
```

Definition. Return reduced row echelon form and pivot columns.

Contract (`D, C, H, R, X`). The input must be rectangular. The function returns exact reduced row echelon form together with pivot-column indices. Pivot selection is deterministic from top to bottom and left to right.

Guarantees (`E, P, B, K`). All arithmetic is exact over `Fraction`. No tolerance, floating backend, accelerator, network, or filesystem access is used. Entrywise work is linear in matrix size, dense multiplication is cubic for square matrices, and elimination-based operations are cubic in arithmetic-operation count. Rational numerator and denominator sizes may grow.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_exact.py` and the independent verifier for exact coordinates cover the applicable construction, inverse, invariant, failure, and resource behavior. The source validator for exact coordinates checks the exact signature, implementation identity, domain facade, alias routes, and anchor. See Chapters 15, 16, and 64.

C.2A · `solve_linear_system`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinates.solve_linear_system`. Reflected implementation `komorebi_math.coordinates.exact.solve_linear_system`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-EXACT`.

Manifest signature (`s`).

```
(matrix: 'Matrix', right_hand_side: 'Sequence[Scalar]') -> 'ExactLinearSystem'
```

Definition. Solve a rational linear system and return one affine representative.

Contract (`D, C, H, R, X`). The right-hand-side length must equal the matrix row count. The solver accepts rectangular systems, returns one representative with free variables set to zero, and returns a nullspace basis. Inconsistency is represented in the result rather than raised as an exception.

Guarantees (`E, P, B, K`). All arithmetic is exact over `Fraction`. No tolerance, floating backend, accelerator, network, or filesystem access is used. Entrywise work is linear in matrix size, dense multiplication is cubic for square matrices, and elimination-based operations are cubic in arithmetic-operation count. Rational numerator and denominator sizes may grow.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_exact.py` and the independent verifier for exact coordinates cover the applicable construction, inverse, invariant, failure, and resource behavior. The source validator for exact coordinates checks the exact signature, implementation identity, domain facade, alias routes, and anchor. See Chapters 15, 16, and 64.

C.2A · `sorted_coordinate_transform`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinates.sorted_coordinate_transform`. Reflected implementation `komorebi_math.coordinates.packet.sorted_coordinate_transform`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-PACKET`.

Manifest signature (`s`).

```
(length: 'int', key: 'Callable[[int], object]', *, source: 'str' = 'original index order', target: 'str' =
    ↪  'key-sorted index order') -> 'CoordinateTransform'
```

Definition. Return a custom exact index permutation obtained from a deterministic key.

Contract (`D, C, H, R, X`). Length must be positive. The key must deterministically order every index from zero through `length-1`. The function returns the exact permutation induced by Python sorting.

Guarantees (`E, P, B, K`). Every result is an exact finite permutation or its exact `CoordinateTransform`. Construction uses no numerical tolerance or optional backend. Storage is linear in the number of indices. Sorting-based orders require `O(n log n)` comparisons, while direct residue and digit orders require linear work.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_numeric_packet.py` and `verification/block14_general_coordinate_audit.py` cover the applicable construction, inverse, invariant, failure, and resource behavior. The source validator for exact coordinates checks the exact signature, implementation identity, domain facade, alias routes, and anchor. See Chapters 13, 18, and 64.

C.2A · `stirling_first_signed`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinates.stirling_first_signed`. Reflected implementation `komorebi_math.coordinates.combinatorial.stirling_first_signed`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-COMBINATORIAL`.

Manifest signature (s).

```
(n: 'int', k: 'int') -> 'int'
```

Definition. Return the signed Stirling number of the first kind `s(n,k)`.

Contract (D, C, H, R, X). `n` and `k` must be nonnegative integers and Boolean values are invalid. Values with `k>n` return zero, with the conventional exception `L(0,0)=1`. The result is an exact Python integer.

Guarantees (E, P, B, K). Coefficients and transforms are exact integers or `Fraction` values. Materialized transforms use quadratic storage in their matrix dimension. For subset transforms the dimension is exponential in rank, so the explicit matrix has `4**rank` entries. The coordinate budget rejects oversized requests before construction.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_combinatorial.py` and `verification/block14_general_coordinate_audit.py` cover the applicable construction, inverse, invariant, failure, and resource behavior. The source validator for exact coordinates checks the exact signature, implementation identity, domain facade, alias routes, and anchor. See Chapters 17, 18, and 64.

C.2A · stirling_first_unsigned

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.stirling_first_unsigned`. Reflected implementation `komorebi_math.coordinates.combinatorial.stirling_first_unsigned`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-COMBINATORIAL`.

Manifest signature (s).

```
(n: 'int', k: 'int') -> 'int'
```

Definition. Return the unsigned Stirling number of the first kind.

Contract (D, C, H, R, X). `n` and `k` must be nonnegative integers and Boolean values are invalid. Values with `k>n` return zero, with the conventional exception `L(0,0)=1`. The result is an exact Python integer.

Guarantees (E, P, B, K). Coefficients and transforms are exact integers or `Fraction` values. Materialized transforms use quadratic storage in their matrix dimension. For subset transforms the dimension is exponential in rank, so the explicit matrix has `4**rank` entries. The coordinate budget rejects oversized requests before construction.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_combinatorial.py` and `verification/block14_general_coordinate_audit.py` cover the applicable construction, inverse, invariant, failure, and resource behavior. The source validator for exact coordinates checks the exact signature, implementation identity, domain facade, alias routes, and anchor. See Chapters 17, 18, and 64.

C.2A · stirling_second

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.stirling_second`. Reflected implementation `komorebi_math.coordinates.combinatorial.stirling_second`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-COMBINATORIAL`.

Manifest signature (s).

```
(n: 'int', k: 'int') -> 'int'
```

Definition. Return the Stirling number of the second kind `S(n,k)`.

Contract (D, C, H, R, X). `n` and `k` must be nonnegative integers and Boolean values are invalid. Values with `k>n` return zero, with the conventional exception `L(0,0)=1`. The result is an exact Python integer.

Guarantees (E, P, B, K). Coefficients and transforms are exact integers or `Fraction` values. Materialized transforms use quadratic storage in their matrix dimension. For subset transforms the dimension is exponential in rank, so the explicit matrix has `4**rank` entries. The coordinate budget rejects oversized requests before construction.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_combinatorial.py` and `verification/block14_general_coordinate_audit.py` cover the applicable construction, inverse, invariant, failure, and resource behavior. The source validator for exact coordinates checks the exact signature, implementation identity, domain facade, alias routes, and anchor. See Chapters 17, 18, and 64.

C.2A · `subset_zeta_transform`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.subset_zeta_transform`. Reflected implementation `komorebi_math.coordinates.combinatorial.subset_zeta_transform`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-COMBINATORIAL`.

Manifest signature (s).

```
(rank: 'int', *, inverse: 'bool' = False, budget: 'CoordinateBudget' =
    ↪ CoordinateBudget(maximum_matrix_dimension=512, maximum_polynomial_degree=512, maximum_horizon=4096,
    ↪ maximum_exact_entries=1000000)) -> 'CoordinateTransform'
```

Definition. Return the subset zeta or Möbius transform on a Boolean lattice.

Contract (D, C, H, R, X). The Boolean-lattice rank must be a nonnegative integer. The materialized dimension is `2**rank`, so the coordinate budget is checked against a square matrix of that size. The inverse flag selects subset Möbius inversion.

Guarantees (E, P, B, K). Coefficients and transforms are exact integers or `Fraction` values. Materialized transforms use quadratic storage in their matrix dimension. For subset transforms the dimension is exponential in rank, so the explicit matrix has `4**rank` entries. The coordinate budget rejects oversized requests before construction.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_combinatorial.py` and `verification/block14_general_coordinate_audit.py` cover the applicable construction, inverse, invariant, failure, and resource behavior. The source validator for exact coordinates checks the exact signature, implementation identity, domain facade, alias routes, and anchor. See Chapters 17, 18, and 64.

C.2A · `tensor_axis_permutation_order`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.tensor_axis_permutation_order`. Reflected implementation `komorebi_math.coordinates.packet.tensor_axis_permutation_order`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-PACKET`.

Manifest signature (s).

```
(shape: 'Sequence[int]', axes: 'Sequence[int]') -> 'tuple[int, ...]'
```

Definition. Return the flat-index order induced by a tensor-axis permutation.

Contract (D, C, H, R, X). The shape must be nonempty with positive entries. `axes` must be a complete permutation of the tensor axes. Flat indices use row-major lexicographic order.

Guarantees (E, P, B, K). Every result is an exact finite permutation or its exact `CoordinateTransform`. Construction uses no numerical tolerance or optional backend. Storage is linear in the number of indices. Sorting-based orders require `O(n log n)` comparisons, while direct residue and digit orders require linear work.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_numeric_packet.py` and `verification/block14_general_coordinate_audit.py` cover the applicable construction, inverse, invariant, failure, and resource behavior. The source validator for exact coordinates checks the exact signature, implementation identity, domain facade, alias routes, and anchor. See Chapters 13, 18, and 64.

C.2A · `tensor_axis_permutation_transform`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.tensor_axis_permutation_transform`. Reflected implementation `komorebi_math.coordinates.packet.tensor_axis_permutation_transform`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-PACKET`.

Manifest signature (s).

```
(shape: 'Sequence[int]', axes: 'Sequence[int]') -> 'CoordinateTransform'
```

Definition. Return the exact coordinate permutation for transposed tensor axes.

Contract (D, C, H, R, X). The shape must be nonempty with positive entries. `axes` must be a complete permutation of the tensor axes. Flat indices use row-major lexicographic order.

Guarantees (E, P, B, K). Every result is an exact finite permutation or its exact `CoordinateTransform`. Construction uses no numerical tolerance or optional backend. Storage is linear in the number of indices. Sorting-based orders require `O(n log n)` comparisons, while direct residue and digit orders require linear work.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_numeric_packet.py` and `verification/block14_general_coordinate_audit.py` cover the applicable construction, inverse, invariant, failure, and resource behavior. The source validator for exact coordinates checks the exact signature, implementation identity, domain facade, alias routes, and anchor. See Chapters 13, 18, and 64.

C.2A · `tensor_coordinate_transform`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.tensor_coordinate_transform`. Reflected implementation `komorebi_math.coordinates.tensor.tensor_coordinate_transform`. The same runtime object is also available through root route `komorebi_math.tensor_coordinate_transform`. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-TENSOR`.

Manifest signature (`s`).

```
(transform: 'CoordinateTransform', variance: 'Sequence[Variance]') -> 'CoordinateTransform'
```

Definition. Return the induced transform on flattened tensor components.

Contract (`D, C, H, R, X`). The base coordinate transform must be square. Each variance entry must be exactly `contravariant` or `covariant`. Tensor axes are flattened lexicographically and a rank-zero tensor uses the one-dimensional identity.

Guarantees (`E, P, B, K`). Transport is exact over rational coefficients. Inversion and similarity or congruence operations use dense exact elimination and multiplication. Kronecker dimensions multiply, so explicit tensor matrices can grow exponentially with tensor rank. No sparse or lazy representation is implied.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_tensor_modular.py` and the independent verifier for exact coordinates cover the applicable construction, inverse, invariant, failure, and resource behavior. The source validator for exact coordinates checks the exact signature, implementation identity, domain facade, alias routes, and anchor. See Chapters 16, 54, and 64.

C.2A · `tensor_product_transform`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinates.tensor_product_transform`. Reflected implementation `komorebi_math.coordinates.tensor.tensor_product_transform`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-TENSOR`.

Manifest signature (`s`).

```
(left: 'CoordinateTransform', right: 'CoordinateTransform') -> 'CoordinateTransform'
```

Definition. Return the Kronecker product of two coordinate transforms.

Contract (`D, C, H, R, X`). The function forms the exact Kronecker product of the two coordinate matrices. Source and target labels are combined in the same left-to-right tensor order.

Guarantees (`E, P, B, K`). Transport is exact over rational coefficients. Inversion and similarity or congruence operations use dense exact elimination and multiplication. Kronecker dimensions multiply, so explicit tensor matrices can grow exponentially with tensor rank. No sparse or lazy representation is implied.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_tensor_modular.py` and the independent verifier for exact coordinates cover the applicable construction, inverse, invariant, failure, and resource behavior. The source validator for exact coordinates checks the exact signature, implementation identity, domain facade, alias routes, and anchor. See Chapters 16, 54, and 64.

C.2A · `transform_sequence`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinates.transform_sequence`. Reflected implementation `komorebi_math.coordinates.combinatorial.transform_sequence`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-COMBINATORIAL`.

Manifest signature (s).

```
(values: 'Sequence[int | Fraction]', transform: 'CoordinateTransform') -> 'tuple[Fraction, ...]'
```

Definition. Apply a coordinate transform to a finite exact sequence.

Contract (D, C, H, R, X). The sequence length must equal the transform source dimension. A mismatch raises `DimensionError`. Values are transported through the exact coordinate map and returned as `Fraction` values.

Guarantees (E, P, B, K). Coefficients and transforms are exact integers or `Fraction` values. Materialized transforms use quadratic storage in their matrix dimension. For subset transforms the dimension is exponential in rank, so the explicit matrix has `4**rank` entries. The coordinate budget rejects oversized requests before construction.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_combinatorial.py` and `verification/block14_general_coordinate_audit.py` cover the applicable construction, inverse, invariant, failure, and resource behavior. The source validator for exact coordinates checks the exact signature, implementation identity, domain facade, alias routes, and anchor. See Chapters 17, 18, and 64.

C.2A · `transport_bilinear_form`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.transport_bilinear_form`. Reflected implementation `komorebi_math.coordinates.tensor.transport_bilinear_form`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-TENSOR`.

Manifest signature (s).

```
(form: 'Iterable[Iterable[Scalar]]', transform: 'CoordinateTransform') -> 'Matrix'
```

Definition. Return the target matrix `C^(-T) G C^(-1)` of a bilinear form.

Contract (D, C, H, R, X). The form and coordinate map must be square with matching dimension. The result uses the exact congruence `C^{-T} G C^{-1}`.

Guarantees (E, P, B, K). Transport is exact over rational coefficients. Inversion and similarity or congruence operations use dense exact elimination and multiplication. Kronecker dimensions multiply, so explicit tensor matrices can grow exponentially with tensor rank. No sparse or lazy representation is implied.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_tensor_modular.py` and the independent verifier for exact coordinates cover the applicable construction, inverse, invariant, failure, and resource behavior. The source validator for exact coordinates checks the exact signature, implementation identity, domain facade, alias routes, and anchor. See Chapters 16, 54, and 64.

C.2A · `transport_covector`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.transport_covector`. Reflected implementation `komorebi_math.coordinates.tensor.transport_covector`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-TENSOR`.

Manifest signature (s).

```
(covector: 'Sequence[Scalar]', transform: 'CoordinateTransform') -> 'Vector'
```

Definition. Transport covector components to the target coordinates.

Contract (D, C, H, R, X). The coordinate transform must be square and invertible. Covectors use the inverse transpose of the vector coordinate matrix. Dimension failures propagate as explicit exceptions.

Guarantees (E, P, B, K). Transport is exact over rational coefficients. Inversion and similarity or congruence operations use dense exact elimination and multiplication. Kronecker dimensions multiply, so explicit tensor matrices can grow exponentially with tensor rank. No sparse or lazy representation is implied.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_tensor_modular.py` and the independent verifier for exact coordinates cover the applicable construction, inverse, invariant, failure, and resource behavior. The source validator for exact coordinates checks the exact signature, implementation identity, domain facade, alias routes, and anchor. See Chapters 16, 54, and 64.

C.2A · `transport_operator`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.transport_operator`. Reflected implementation `komorebi_math.coordinates.tensor.transport_operator`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-TENSOR`.

Manifest signature (s).

```
(operator: 'Iterable[Iterable[Scalar]]', transform: 'CoordinateTransform') -> 'Matrix'
```

Definition. Return `C A C^(-1)` in target coordinates.

Contract (D, C, H, R, X). The operator and coordinate map must be square with matching source dimension. The result is the exact similarity transform `C A C^{-1}`.

Guarantees (E, P, B, K). Transport is exact over rational coefficients. Inversion and similarity or congruence operations use dense exact elimination and multiplication. Kronecker dimensions multiply, so explicit tensor matrices can grow exponentially with tensor rank. No sparse or lazy representation is implied.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_tensor_modular.py` and the independent verifier for exact coordinates cover the applicable construction, inverse, invariant, failure, and resource behavior. The source validator for exact coordinates checks the exact signature, implementation identity, domain facade, alias routes, and anchor. See Chapters 16, 54, and 64.

C.2A · `transport_rectangular_operator`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.transport_rectangular_operator`. Reflected implementation `komorebi_math.coordinates.tensor.transport_rectangular_operator`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-TENSOR`.

Manifest signature (`s`).

```
(operator: 'Iterable[Iterable[Scalar]]', domain_transform: 'CoordinateTransform', codomain_transform:
   ↪  'CoordinateTransform') -> 'Matrix'
```

Definition. Transport a linear map by `C_codomain A C_domain^(-1)`.

Contract (`D, C, H, R, X`). Both domain and codomain coordinate maps must be square. Operator columns must match the domain dimension and rows must match the codomain dimension.

Guarantees (`E, P, B, K`). Transport is exact over rational coefficients. Inversion and similarity or congruence operations use dense exact elimination and multiplication. Kronecker dimensions multiply, so explicit tensor matrices can grow exponentially with tensor rank. No sparse or lazy representation is implied.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_tensor_modular.py` and the independent verifier for exact coordinates cover the applicable construction, inverse, invariant, failure, and resource behavior. The source validator for exact coordinates checks the exact signature, implementation identity, domain facade, alias routes, and anchor. See Chapters 16, 54, and 64.

C.2A · `transport_tensor_components`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinates.transport_tensor_components`. Reflected implementation `komorebi_math.coordinates.tensor.transport_tensor_components`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-TENSOR`.

Manifest signature (`s`).

```
(components: 'Sequence[Scalar]', transform: 'CoordinateTransform', variance: 'Sequence[Variance]') -> 'Vector'
```

Definition. Transport flattened tensor components between coordinates.

Contract (`D, C, H, R, X`). The base coordinate transform must be square. Each variance entry must be exactly `contravariant` or `covariant`. Tensor axes are flattened lexicographically and a rank-zero tensor uses the one-dimensional identity.

Guarantees (`E, P, B, K`). Transport is exact over rational coefficients. Inversion and similarity or congruence operations use dense exact elimination and multiplication. Kronecker dimensions multiply, so explicit tensor matrices can grow exponentially with tensor rank. No sparse or lazy representation is implied.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_tensor_modular.py` and the independent verifier for exact coordinates cover the applicable construction, inverse, invariant, failure, and resource behavior. The source validator for exact coordinates checks the exact signature, implementation identity, domain facade, alias routes, and anchor. See Chapters 16, 54, and 64.

C.2A · `transpose`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinates.transpose`. Reflected implementation `komorebi_math.coordinates.exact.transpose`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-EXACT`.

Manifest signature (`s`).

```
(matrix: 'Matrix') -> 'Matrix'
```

Definition. Return the matrix transpose.

Contract (`D, C, H, R, X`). The input must use the exact rectangular matrix representation. The operation swaps row and column positions and preserves every coefficient exactly.

Guarantees (`E, P, B, K`). All arithmetic is exact over `Fraction`. No tolerance, floating backend, accelerator, network, or filesystem access is used. Entrywise work is linear in matrix size, dense multiplication is cubic for square matrices, and elimination-based operations are cubic in arithmetic-operation count. Rational numerator and denominator sizes may grow.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_exact.py` and the independent verifier for exact coordinates cover the applicable construction, inverse, invariant, failure, and resource behavior. The source validator for exact coordinates checks the exact signature, implementation identity, domain facade, alias routes, and anchor. See Chapters 15, 16, and 64.

C.2A · `zero_matrix`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinates.zero_matrix`. Reflected implementation `komorebi_math.coordinates.exact.zero_matrix`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-EXACT`.

Manifest signature (`s`).

```
(rows: 'int', columns: 'int') -> 'Matrix'
```

Definition. Return a rational zero matrix.

Contract (`D, C, H, R, X`). Row and column counts must be nonnegative integers. Negative dimensions raise `DimensionError`. Zero dimensions are represented by empty tuples in the corresponding direction.

Guarantees (`E, P, B, K`). All arithmetic is exact over `Fraction`. No tolerance, floating backend, accelerator, network, or filesystem access is used. Entrywise work is linear in matrix size, dense multiplication is cubic for square matrices, and elimination-based operations are cubic in arithmetic-operation count. Rational numerator and denominator sizes may grow.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_exact.py` and the independent verifier for exact coordinates cover the applicable construction, inverse, invariant, failure, and resource behavior. The source validator for exact coordinates checks the exact signature, implementation identity, domain facade, alias routes, and anchor. See Chapters 15, 16, and 64.

C.2B Classical polynomial families, interpolation, and exact operator calculus: 73 entries

This section documents the exact polynomial-coordinate layer as one connected subsystem. It starts with immutable coefficient arithmetic and graded family objects, then covers classical and recurrence-defined polynomial families, finite interpolation bases, family-to-family connection matrices, affine substitution, and operator matrices for differentiation, integration, multiplication, finite differences, and coordinate transport. Every card uses the same ascending-degree coefficient convention. The resource notes separate low-order coefficient operations from dense family conversion, where exact matrix inversion is the dominant cost.

C.2B · PolynomialCoordinateOperator

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.PolynomialCoordinateOperator`. Reflected implementation `komorebi_math.coordinates.calculus.PolynomialCoordinateOperator`. The same runtime object is also available through `komorebi_math.PolynomialCoordinateOperator`. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `class`. Profile `C2-POLY-OPERATOR`.

Manifest signature (s).

```
(name: 'str', source_family: 'str', target_family: 'str', source_degree: 'int', target_degree: 'int', matrix:
   ↪  'Matrix', theorem: 'str' = 'CC.01') -> None
```

Definition. Store and apply a finite exact operator matrix between two polynomial-family truncations.

Contract (D, C, H, R, X). The declared source and target degrees determine the required matrix shape. Names must be nonempty. `apply` checks the input dimension through exact matrix multiplication. `compose` requires matching family names and truncation degrees. `certificate` records exact shape validation under the supplied theorem identifier.

Guarantees (E, P, B, K). All coefficients and certificates are exact. Dense family matrices use quadratic storage and inversion or conjugation can require cubic arithmetic-operation cost in the truncation dimension. The coordinate budget limits declared matrix dimensions and polynomial degrees. Exact coefficients may grow.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_calculus.py`, `tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 18, 32, and 64.

C.2B · PolynomialFamily

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.PolynomialFamily`. Reflected implementation `komorebi_math.coordinates.polynomials.PolynomialFamily`. The same runtime object is also available through `komorebi_math.PolynomialFamily`. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `class`. Profile `C2-POLY-FAMILY`.

Manifest signature (s).

```
(name: 'str', rule: 'Callable[[int], Polynomial]') -> None
```

Definition. Represent a graded exact polynomial basis through a degree-indexed construction rule.

Contract (D, C, H, R, X). The name and rule are stored without evaluating the full family. `polynomial(degree)` accepts only nonnegative integer degrees and requires the rule to return a polynomial of exact graded degree with a nonzero leading coefficient. `coefficient_matrix(degree)` places those basis polynomials in monomial-coordinate columns.

Guarantees (E, P, B, K). All coefficients and certificates are exact. Dense family matrices use quadratic storage and inversion or conjugation can require cubic arithmetic-operation cost in the truncation dimension. The coordinate budget limits declared matrix dimensions and polynomial degrees. Exact coefficients may grow.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 17, 18, 21, and 64.

C.2B · `PolynomialFamilyCatalog`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinates.PolynomialFamilyCatalog`. Reflected implementation `komorebi_math.coordinates.classical.PolynomialFamilyCatalog`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `class`. Profile `C2-POLY-FAMILY`.

Manifest signature (`s`).

```
(families: 'tuple[PolynomialFamily, ...]') -> None
```

Definition. Provide deterministic lookup for the built-in exact graded polynomial families.

Contract (`D, C, H, R, X`). The catalog stores an ordered tuple of `PolynomialFamily` objects. `names()` preserves that order. `get(name)` normalizes case and punctuation, accepts the documented short aliases, and raises `ValidationError` for an unknown family.

Guarantees (`E, P, B, K`). All coefficients and certificates are exact. Dense family matrices use quadratic storage and inversion or conjugation can require cubic arithmetic-operation cost in the truncation dimension. The coordinate budget limits declared matrix dimensions and polynomial degrees. Exact coefficients may grow.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_classical.py`, `tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 15, 17, 18, and 64.

C.2B · `affine_polynomial`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinates.affine_polynomial`. Reflected implementation `komorebi_math.coordinates.classical.affine_polynomial`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-PRIMITIVE`.

Manifest signature (`s`).

```
(scale: 'Scalar', shift: 'Scalar') -> 'Polynomial'
```

Definition. Return the affine polynomial `shift + scale*x`.

Contract (`D, C, H, R, X`). Inputs are finite coefficient sequences in ascending degree order and must convert to exact rational values. The result is an immutable coefficient tuple with trailing zeros removed, except that the zero polynomial retains one zero coefficient.

Guarantees (`E, P, B, K`). Arithmetic is exact over `Fraction`. Addition, scaling, shifting, differentiation, integration, and Horner evaluation are linear in coefficient count. No tolerance, symbolic service, accelerator, network, or filesystem is used.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_classical.py`, `tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 15, 17, 18, and 64.

C.2B · affine_substitution_coordinate_operator

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.affine_substitution_coordinate_operator`. Reflected implementation `komorebi_math.coordinates.calculus.affine_substitution_coordinate_operator`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-OPERATOR`.

Manifest signature (s).

```
(family: 'PolynomialFamily', degree: 'int', *, scale: 'Scalar' = 1, shift: 'Scalar' = 0, budget: 'CoordinateBudget'
   ↪ = CoordinateBudget(maximum_matrix_dimension=512, maximum_polynomial_degree=512, maximum_horizon=4096,
   ↪ maximum_exact_entries=1000000)) -> 'PolynomialCoordinateOperator'
```

Definition. Return the degree-preserving substitution `p(x)->p(scale*x+shift)`.

Contract (D, C, H, R, X). The family and degree must satisfy the polynomial-operator contract. Scale and shift are exact rational values. The result is square and represents substitution within the same family.

Guarantees (E, P, B, K). All coefficients and certificates are exact. Dense family matrices use quadratic storage and inversion or conjugation can require cubic arithmetic-operation cost in the truncation dimension. The coordinate budget limits declared matrix dimensions and polynomial degrees. Exact coefficients may grow.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_calculus.py`, `tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 18, 32, and 64.

C.2B · affine_substitution_transform

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.affine_substitution_transform`. Reflected implementation `komorebi_math.coordinates.classical.affine_substitution_transform`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-TRANSFORM`.

Manifest signature (s).

```
(family: 'PolynomialFamily', degree: 'int', *, scale: 'Scalar' = 1, shift: 'Scalar' = 0, budget: 'CoordinateBudget'
   ↪ = CoordinateBudget(maximum_matrix_dimension=512, maximum_polynomial_degree=512, maximum_horizon=4096,
   ↪ maximum_exact_entries=1000000)) -> 'CoordinateTransform'
```

Definition. Return the coordinates of `p(scale*x+shift)` in one family.

Contract (D, C, H, R, X). The family must be valid through the requested nonnegative degree. Scale and shift must convert to exact rational values. The coordinate budget bounds the square matrix, and the result maps coefficients of `p(x)` to coefficients of `p(scale*x+shift)` in the same family.

Guarantees (E, P, B, K). All coefficients and certificates are exact. Dense family matrices use quadratic storage and inversion or conjugation can require cubic arithmetic-operation cost in the truncation dimension. The coordinate budget limits declared matrix dimensions and polynomial degrees. Exact coefficients may grow.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_classical.py`, `tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 15, 17, 18, and 64.

C.2B · appell_polynomial

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.appell_polynomial`. Reflected implementation `komorebi_math.coordinates.classical.appell_polynomial`. The same spelling also appears at `komorebi_math.appell_polynomial`, but that route resolves to `komorebi_math.analytic.appell` and is not an alias of this coordinate-domain object. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-CLASSICAL`.

Manifest signature (s).

```
(degree: 'int', values_at_zero: 'Sequence[Scalar]') -> 'Polynomial'
```

Definition. Return one Appell polynomial from the values `P_n(0)`.

Contract (D, C, H, R, X). Degree must be nonnegative. The seed must contain values through the requested degree and its degree-zero value must be nonzero. The result uses the exact Appell identity based on the values at zero.

Guarantees (E, P, B, K). The construction uses integer and rational arithmetic only. Recurrence and coefficient generation are finite and deterministic for the requested degree. Runtime is polynomial in the requested prefix size, while exact coefficient magnitudes may grow rapidly for some families.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_classical.py`, `tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 15, 17, 18, and 64.

C.2B · appell_polynomial_family

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.appell_polynomial_family`. Reflected implementation `komorebi_math.coordinates.classical.appell_polynomial_family`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-FAMILY`.

Manifest signature (s).

```
(values_at_zero: 'Sequence[Scalar]', *, name: 'str' = 'custom Appell polynomials') -> 'PolynomialFamily'
```

Definition. Return a finite-seed Appell family.

Contract (D, C, H, R, X). The seed must be nonempty and start with a nonzero value. The returned family is finite-seed: requesting a degree outside the stored seed raises `DimensionError`. The optional name labels the coordinate family and does not change the coefficients.

Guarantees (E, P, B, K). The construction uses integer and rational arithmetic only. Recurrence and coefficient generation are finite and deterministic for the requested degree. Runtime is polynomial in the requested prefix size, while exact coefficient magnitudes may grow rapidly for some families.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_classical.py`, `tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 15, 17, 18, and 64.

C.2B · bernoulli_number

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.bernoulli_number`. Reflected implementation `komorebi_math.coordinates.classical.bernoulli_number`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-CLASSICAL`.

Manifest signature (s).

```
(index: 'int') -> 'Fraction'
```

Definition. Return the Bernoulli number with convention `B_1=-1/2`.

Contract (D, C, H, R, X). The index must be a nonnegative integer. The recurrence uses the convention `B_1=-1/2` and returns an exact `Fraction`.

Guarantees (E, P, B, K). The construction uses integer and rational arithmetic only. Recurrence and coefficient generation are finite and deterministic for the requested degree. Runtime is polynomial in the requested prefix size, while exact coefficient magnitudes may grow rapidly for some families.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_classical.py`, `tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 15, 17, 18, and 64.

C.2B · bernoulli_polynomial

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.bernoulli_polynomial`. Reflected implementation `komorebi_math.coordinates.classical.bernoulli_polynomial`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-CLASSICAL`.

Manifest signature (s).

```
(degree: 'int') -> 'Polynomial'
```

Definition. Return the Bernoulli polynomial `B_degree(x)`.

Contract (D, C, H, R, X). The degree must be a nonnegative integer. The returned immutable tuple stores exact coefficients in ascending monomial order and follows the recurrence or closed formula named in the definition.

Guarantees (E, P, B, K). The construction uses integer and rational arithmetic only. Recurrence and coefficient generation are finite and deterministic for the requested degree. Runtime is polynomial in the requested prefix size, while exact coefficient magnitudes may grow rapidly for some families.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_classical.py`, `tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 15, 17, 18, and 64.

C.2B · bernoulli_polynomial_family

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.bernoulli_polynomial_family`. Reflected implementation `komorebi_math.coordinates.classical.bernoulli_polynomial_family`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-FAMILY`.

Manifest signature (s).

```
() -> 'PolynomialFamily'
```

Definition. Return the graded Bernoulli polynomial family.

Contract (D, C, H, R, X). The constructor returns a `PolynomialFamily` whose degree rule is exact and graded. Later calls to `polynomial(degree)` reject negative or noninteger degrees and verify that the leading coefficient is nonzero at the declared degree.

Guarantees (E, P, B, K). The construction uses integer and rational arithmetic only. Recurrence and coefficient generation are finite and deterministic for the requested degree. Runtime is polynomial in the requested prefix size, while exact coefficient magnitudes may grow rapidly for some families.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_classical.py`, `tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 15, 17, 18, and 64.

C.2B · `bernstein_basis`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.bernstein_basis`. Reflected implementation `komorebi_math.coordinates.classical.bernstein_basis`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-INTERPOLATION`.

Manifest signature (s).

```
(degree: 'int') -> 'FiniteBasis'
```

Definition. Return the fixed-degree Bernstein basis of polynomials through `degree`.

Contract (D, C, H, R, X). Degree must be nonnegative. The result contains all degree-fixed Bernstein polynomials from index zero through the degree, padded to one common ambient dimension. It is a `FiniteBasis`, not a graded `PolynomialFamily`.

Guarantees (E, P, B, K). All coefficients and certificates are exact. Dense family matrices use quadratic storage and inversion or conjugation can require cubic arithmetic-operation cost in the truncation dimension. The coordinate budget limits declared matrix dimensions and polynomial degrees. Exact coefficients may grow.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_classical.py`, `tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 15, 17, 18, and 64.

C.2B · `bernstein_polynomial`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.bernstein_polynomial`. Reflected implementation `komorebi_math.coordinates.classical.bernstein_polynomial`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-CLASSICAL`.

Manifest signature (s).

```
(degree: 'int', index: 'int') -> 'Polynomial'
```

Definition. Return `binom(degree,index)x^index(1-x)^(degree-index)`.

Contract (D, C, H, R, X). Degree and index must be nonnegative integers, and the index cannot exceed the degree. The returned coefficient tuple represents the fixed-degree Bernstein polynomial in ascending monomial order.

Guarantees (E, P, B, K). The construction uses integer and rational arithmetic only. Recurrence and coefficient generation are finite and deterministic for the requested degree. Runtime is polynomial in the requested prefix size, while exact coefficient magnitudes may grow rapidly for some families.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_classical.py`, `tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 15, 17, 18, and 64.

C.2B · `binomial_polynomial`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.binomial_polynomial`. Reflected implementation `komorebi_math.coordinates.classical.binomial_polynomial`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-CLASSICAL`.

Manifest signature (s).

```
(degree: 'int') -> 'Polynomial'
```

Definition. Return the binomial polynomial `binom(x,degree)`.

Contract (D, C, H, R, X). The degree must be a nonnegative integer. The returned immutable tuple stores exact coefficients in ascending monomial order and follows the recurrence or closed formula named in the definition.

Guarantees (E, P, B, K). The construction uses integer and rational arithmetic only. Recurrence and coefficient generation are finite and deterministic for the requested degree. Runtime is polynomial in the requested prefix size, while exact coefficient magnitudes may grow rapidly for some families.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_classical.py`, `tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 15, 17, 18, and 64.

C.2B · `binomial_polynomial_family`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.binomial_polynomial_family`. Reflected implementation `komorebi_math.coordinates.classical.binomial_polynomial_family`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-FAMILY`.

Manifest signature (`s`).

```
() -> 'PolynomialFamily'
```

Definition. Return the graded binomial-polynomial family.

Contract (`D, C, H, R, X`). The constructor returns a `PolynomialFamily` whose degree rule is exact and graded. Later calls to `polynomial(degree)` reject negative or noninteger degrees and verify that the leading coefficient is nonzero at the declared degree.

Guarantees (`E, P, B, K`). The construction uses integer and rational arithmetic only. Recurrence and coefficient generation are finite and deterministic for the requested degree. Runtime is polynomial in the requested prefix size, while exact coefficient magnitudes may grow rapidly for some families.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_classical.py, tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 15, 17, 18, and 64.

C.2B · `chebyshev_t_polynomial`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinates.chebyshev_t_polynomial`. Reflected implementation `komorebi_math.coordinates.classical.chebyshev_t_polynomial`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-CLASSICAL`.

Manifest signature (`s`).

```
(degree: 'int') -> 'Polynomial'
```

Definition. Return the Chebyshev polynomial `T_degree`.

Contract (`D, C, H, R, X`). The degree must be a nonnegative integer. The returned immutable tuple stores exact coefficients in ascending monomial order and follows the recurrence or closed formula named in the definition.

Guarantees (`E, P, B, K`). The construction uses integer and rational arithmetic only. Recurrence and coefficient generation are finite and deterministic for the requested degree. Runtime is polynomial in the requested prefix size, while exact coefficient magnitudes may grow rapidly for some families.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_classical.py, tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 15, 17, 18, and 64.

C.2B · `chebyshev_t_polynomial_family`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinates.chebyshev_t_polynomial_family`. Reflected implementation `komorebi_math.coordinates.classical.chebyshev_t_polynomial_family`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-FAMILY`.

Manifest signature (s).

```
() -> 'PolynomialFamily'
```

Definition. Return the graded Chebyshev `T` family.

Contract (D, C, H, R, X). The constructor returns a `PolynomialFamily` whose degree rule is exact and graded. Later calls to `polynomial(degree)` reject negative or noninteger degrees and verify that the leading coefficient is nonzero at the declared degree.

Guarantees (E, P, B, K). The construction uses integer and rational arithmetic only. Recurrence and coefficient generation are finite and deterministic for the requested degree. Runtime is polynomial in the requested prefix size, while exact coefficient magnitudes may grow rapidly for some families.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_classical.py`, `tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 15, 17, 18, and 64.

C.2B · chebyshev_u_polynomial

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.chebyshev_u_polynomial`. Reflected implementation `komorebi_math.coordinates.classical.chebyshev_u_polynomial`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-CLASSICAL`.

Manifest signature (s).

```
(degree: 'int') -> 'Polynomial'
```

Definition. Return the Chebyshev polynomial `U_degree`.

Contract (D, C, H, R, X). The degree must be a nonnegative integer. The returned immutable tuple stores exact coefficients in ascending monomial order and follows the recurrence or closed formula named in the definition.

Guarantees (E, P, B, K). The construction uses integer and rational arithmetic only. Recurrence and coefficient generation are finite and deterministic for the requested degree. Runtime is polynomial in the requested prefix size, while exact coefficient magnitudes may grow rapidly for some families.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_classical.py`, `tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 15, 17, 18, and 64.

C.2B · chebyshev_u_polynomial_family

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.chebyshev_u_polynomial_family`. Reflected implementation `komorebi_math.coordinates.classical.chebyshev_u_polynomial_family`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-FAMILY`.

Manifest signature (s).

```
() -> 'PolynomialFamily'
```

Definition. Return the graded Chebyshev `U` family.

Contract (D, C, H, R, X). The constructor returns a `PolynomialFamily` whose degree rule is exact and graded. Later calls to `polynomial(degree)` reject negative or noninteger degrees and verify that the leading coefficient is nonzero at the declared degree.

Guarantees (E, P, B, K). The construction uses integer and rational arithmetic only. Recurrence and coefficient generation are finite and deterministic for the requested degree. Runtime is polynomial in the requested prefix size, while exact coefficient magnitudes may grow rapidly for some families.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_classical.py`, `tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 15, 17, 18, and 64.

C.2B · `classical_polynomial_catalog`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.classical_polynomial_catalog`. Reflected implementation `komorebi_math.coordinates.classical.classical_polynomial_catalog`. The same runtime object is also available through `komorebi_math.classical_polynomial_catalog`. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-FAMILY`.

Manifest signature (s).

```
() -> 'PolynomialFamilyCatalog'
```

Definition. Return the built-in exact graded polynomial-family catalog.

Contract (D, C, H, R, X). Returns the fixed built-in catalog of fifteen exact graded families. The catalog order is deterministic and includes monomial, Euler, Fibonacci, factorial, Bernoulli, Lucas, Pell, Chebyshev, Legendre, Hermite, and Laguerre families.

Guarantees (E, P, B, K). The result is deterministic and exact over integers and `Fraction` values. Dense coordinate work can require quadratic storage and cubic arithmetic-operation cost. No floating tolerance or optional numerical backend is consulted.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_classical.py`, `tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 15, 17, 18, and 64.

C.2B · `connection_transform`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.connection_transform`. Reflected implementation `komorebi_math.coordinates.polynomials.connection_transform`. The same runtime object is also available through `komorebi_math.connection_transform`. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-TRANSFORM`.

Manifest signature (`S`).

```
(source: 'PolynomialFamily', target: 'PolynomialFamily', degree: 'int', *, budget: 'CoordinateBudget' =
    ↪ CoordinateBudget(maximum_matrix_dimension=512, maximum_polynomial_degree=512, maximum_horizon=4096,
    ↪ maximum_exact_entries=1000000)) -> 'CoordinateTransform'
```

Definition. Return target coordinates of source basis vectors through one degree.

Contract (`D, C, H, R, X`). Source and target must be `PolynomialFamily` objects and degree must satisfy the coordinate budget. Both families must be valid graded bases through that degree. The returned matrix is `M_target^{-1} M_source`.

Guarantees (`E, P, B, K`). All coefficients and certificates are exact. Dense family matrices use quadratic storage and inversion or conjugation can require cubic arithmetic-operation cost in the truncation dimension. The coordinate budget limits declared matrix dimensions and polynomial degrees. Exact coefficients may grow.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 17, 18, 21, and 64.

C.2B · `constant_polynomial`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinates.constant_polynomial`. Reflected implementation `komorebi_math.coordinates.classical.constant_polynomial`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-PRIMITIVE`.

Manifest signature (`S`).

```
(value: 'Scalar') -> 'Polynomial'
```

Definition. Return one exact constant polynomial.

Contract (`D, C, H, R, X`). Inputs are finite coefficient sequences in ascending degree order and must convert to exact rational values. The result is an immutable coefficient tuple with trailing zeros removed, except that the zero polynomial retains one zero coefficient.

Guarantees (`E, P, B, K`). Arithmetic is exact over `Fraction`. Addition, scaling, shifting, differentiation, integration, and Horner evaluation are linear in coefficient count. No tolerance, symbolic service, accelerator, network, or filesystem is used.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_classical.py`, `tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 15, 17, 18, and 64.

C.2B · `derivative_coordinate_operator`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinates.derivative_coordinate_operator`. Reflected implementation `komorebi_math.coordinates.calculus.derivative_coordinate_operator`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-OPERATOR`.

Manifest signature (`s`).

```
(family: 'PolynomialFamily', degree: 'int', *, budget: 'CoordinateBudget' =
    ↪ CoordinateBudget(maximum_matrix_dimension=512, maximum_polynomial_degree=512, maximum_horizon=4096,
    ↪ maximum_exact_entries=1000000)) -> 'PolynomialCoordinateOperator'
```

Definition. Return differentiation in one graded polynomial family.

Contract (`D, C, H, R, X`). The family and degree must satisfy the polynomial-operator contract. The result is a square operator on the declared truncation and carries theorem identifier `CC.02`.

Guarantees (`E, P, B, K`). All coefficients and certificates are exact. Dense family matrices use quadratic storage and inversion or conjugation can require cubic arithmetic-operation cost in the truncation dimension. The coordinate budget limits declared matrix dimensions and polynomial degrees. Exact coefficients may grow.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_calculus.py`, `tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 18, 32, and 64.

C.2B · `derivative_operator_in_family`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinates.derivative_operator_in_family`. Reflected implementation `komorebi_math.coordinates.polynomials.derivative_operator_in_family`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-TRANSFORM`.

Manifest signature (`s`).

```
(family: 'PolynomialFamily', degree: 'int') -> 'Matrix'
```

Definition. Return the derivative operator in one graded polynomial basis.

Contract (`D, C, H, R, X`). The family must be valid through the requested degree. The monomial derivative matrix is conjugated by the family coefficient matrix. The returned dense matrix acts on coefficient vectors of length `degree+1`.

Guarantees (`E, P, B, K`). All coefficients and certificates are exact. Dense family matrices use quadratic storage and inversion or conjugation can require cubic arithmetic-operation cost in the truncation dimension. The coordinate budget limits declared matrix dimensions and polynomial degrees. Exact coefficients may grow.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 17, 18, 21, and 64.

C.2B · `euler_polynomial`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinates.euler_polynomial`. Reflected implementation `komorebi_math.coordinates.polynomials.euler_polynomial`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-CLASSICAL`.

Manifest signature (s).

```
(degree: 'int') -> 'Polynomial'
```

Definition. Return the Euler polynomial from `2e^(xt)/(e^t+1)`.

Contract (D, C, H, R, X). The degree must be a nonnegative integer. The returned immutable tuple stores exact coefficients in ascending monomial order and follows the recurrence or closed formula named in the definition.

Guarantees (E, P, B, K). The construction uses integer and rational arithmetic only. Recurrence and coefficient generation are finite and deterministic for the requested degree. Runtime is polynomial in the requested prefix size, while exact coefficient magnitudes may grow rapidly for some families.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 17, 18, 21, and 64.

C.2B · `euler_polynomial_at_zero`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.euler_polynomial_at_zero`. Reflected implementation `komorebi_math.coordinates.polynomials.euler_polynomial_at_zero`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-CLASSICAL`.

Manifest signature (s).

```
(index: 'int') -> 'Fraction'
```

Definition. Return `E_index(0)` for `2e^(xt)/(e^t+1)`.

Contract (D, C, H, R, X). The index must be nonnegative. The finite recurrence returns the exact value at zero for the Euler-polynomial convention `2e^(xt)/(e^t+1)`.

Guarantees (E, P, B, K). The construction uses integer and rational arithmetic only. Recurrence and coefficient generation are finite and deterministic for the requested degree. Runtime is polynomial in the requested prefix size, while exact coefficient magnitudes may grow rapidly for some families.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 17, 18, 21, and 64.

C.2B · `euler_polynomial_family`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.euler_polynomial_family`. Reflected implementation `komorebi_math.coordinates.polynomials.euler_polynomial_family`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-FAMILY`.

Manifest signature (`s`).

```
() -> 'PolynomialFamily'
```

Definition. Return the graded Euler polynomial family.

Contract (`D, C, H, R, X`). The constructor returns a `PolynomialFamily` whose degree rule is exact and graded. Later calls to `polynomial(degree)` reject negative or noninteger degrees and verify that the leading coefficient is nonzero at the declared degree.

Guarantees (`E, P, B, K`). The construction uses integer and rational arithmetic only. Recurrence and coefficient generation are finite and deterministic for the requested degree. Runtime is polynomial in the requested prefix size, while exact coefficient magnitudes may grow rapidly for some families.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 17, 18, 21, and 64.

C.2B · `euler_to_fibonacci_coefficient`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinates.euler_to_fibonacci_coefficient`. Reflected implementation `komorebi_math.coordinates.polynomials.euler_to_fibonacci_coefficient`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-CLASSICAL`.

Manifest signature (`s`).

```
(euler_degree: 'int', fibonacci_degree: 'int') -> 'Fraction'
```

Definition. Return the exact coefficient in `E_n=sum_k c_(k,n)F_(k+1)`.

Contract (`D, C, H, R, X`). Both degrees must be nonnegative. A Fibonacci degree above the Euler degree returns zero. The function combines exact Euler values at zero with the monomial-to-Fibonacci formula.

Guarantees (`E, P, B, K`). The construction uses integer and rational arithmetic only. Recurrence and coefficient generation are finite and deterministic for the requested degree. Runtime is polynomial in the requested prefix size, while exact coefficient magnitudes may grow rapidly for some families.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 17, 18, 21, and 64.

C.2B · `euler_to_fibonacci_transform`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinates.euler_to_fibonacci_transform`. Reflected implementation `komorebi_math.coordinates.polynomials.euler_to_fibonacci_transform`. The same runtime object is also available through `komorebi_math.euler_to_fibonacci_transform`. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-TRANSFORM`.

Manifest signature (s).

```
(degree: 'int', *, budget: 'CoordinateBudget' = CoordinateBudget(maximum_matrix_dimension=512,
    ↪ maximum_polynomial_degree=512, maximum_horizon=4096, maximum_exact_entries=1000000)) ->
    ↪ 'CoordinateTransform'
```

Definition. Return the Euler polynomial to Fibonacci polynomial connection.

Contract (D, C, H, R, X). Degree must fit the coordinate budget. The general family connection is computed first, then every entry is checked against the explicit coefficient formula. A disagreement raises `ValidationError` instead of returning an unchecked matrix.

Guarantees (E, P, B, K). The connection and its independent entry formula are exact rational calculations. Dense basis inversion and multiplication require cubic arithmetic-operation cost in `degree+1`, with quadratic matrix storage. Exact coefficient sizes may grow.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 17, 18, 21, and 64.

C.2B · `evaluation_transform`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.evaluation_transform`. Reflected implementation `komorebi_math.coordinates.classical.evaluation_transform`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-INTERPOLATION`.

Manifest signature (s).

```
(family: 'PolynomialFamily', nodes: 'Sequence[Scalar]', *, budget: 'CoordinateBudget' =
    ↪ CoordinateBudget(maximum_matrix_dimension=512, maximum_polynomial_degree=512, maximum_horizon=4096,
    ↪ maximum_exact_entries=1000000)) -> 'CoordinateTransform'
```

Definition. Map polynomial-family coefficients to values at exact nodes.

Contract (D, C, H, R, X). Nodes must be nonempty and must determine the polynomial space through degree `len(nodes) -1`. The matrix evaluates each family basis polynomial at each node. A singular evaluation matrix raises `ValidationError`.

Guarantees (E, P, B, K). Evaluation entries are exact rational values. Determinant and inverse-related checks use dense exact algebra with cubic arithmetic-operation cost in the node count and quadratic storage.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_classical.py`, `tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 15, 17, 18, and 64.

C.2B · `falling_factorial_polynomial`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.falling_factorial_polynomial`. Reflected implementation `komorebi_math.coordinates.classical.falling_factorial_polynomial`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-CLASSICAL`.

Manifest signature (`s`).

```
(degree: 'int') -> 'Polynomial'
```

Definition. Return `x^(underline degree)`.

Contract (`D, C, H, R, X`). The degree must be a nonnegative integer. The returned immutable tuple stores exact coefficients in ascending monomial order and follows the recurrence or closed formula named in the definition.

Guarantees (`E, P, B, K`). The construction uses integer and rational arithmetic only. Recurrence and coefficient generation are finite and deterministic for the requested degree. Runtime is polynomial in the requested prefix size, while exact coefficient magnitudes may grow rapidly for some families.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_classical.py, tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 15, 17, 18, and 64.

C.2B · `falling_factorial_polynomial_family`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinates.falling_factorial_polynomial_family`. Reflected implementation `komorebi_math.coordinates.classical.falling_factorial_polynomial_family`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-FAMILY`.

Manifest signature (`s`).

```
() -> 'PolynomialFamily'
```

Definition. Return the graded falling-factorial family.

Contract (`D, C, H, R, X`). The constructor returns a `PolynomialFamily` whose degree rule is exact and graded. Later calls to `polynomial(degree)` reject negative or noninteger degrees and verify that the leading coefficient is nonzero at the declared degree.

Guarantees (`E, P, B, K`). The construction uses integer and rational arithmetic only. Recurrence and coefficient generation are finite and deterministic for the requested degree. Runtime is polynomial in the requested prefix size, while exact coefficient magnitudes may grow rapidly for some families.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_classical.py, tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 15, 17, 18, and 64.

C.2B · `fibonacci_companion_power`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinates.fibonacci_companion_power`. Reflected implementation `komorebi_math.coordinates.polynomials.fibonacci_companion_power`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-CLASSICAL`.

Manifest signature (s).

```
(power: 'int') -> 'tuple[tuple[Polynomial, Polynomial], tuple[Polynomial, Polynomial]]'
```

Definition. Return `[[x,1],[1,0]]^power` as exact polynomial entries.

Contract (D, C, H, R, X). Power must be a nonnegative integer. Binary exponentiation is applied to the polynomial companion matrix `[[x,1],[1,0]]`, with exact polynomial multiplication at every step.

Guarantees (E, P, B, K). All matrix entries are exact polynomials. Binary exponentiation uses logarithmically many 2 by 2 matrix products, while polynomial coefficient work grows with the requested power. Coefficients and polynomial lengths may grow substantially.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 17, 18, 21, and 64.

C.2B · `fibonacci_polynomial`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.fibonacci_polynomial`. Reflected implementation `komorebi_math.coordinates.polynomials.fibonacci_polynomial`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-CLASSICAL`.

Manifest signature (s).

```
(index: 'int') -> 'Polynomial'
```

Definition. Return the standard Fibonacci polynomial `F_index(x)`.

Contract (D, C, H, R, X). The index must be a nonnegative integer. The returned immutable tuple stores exact coefficients in ascending monomial order and follows the recurrence or closed formula named in the definition.

Guarantees (E, P, B, K). The construction uses integer and rational arithmetic only. Recurrence and coefficient generation are finite and deterministic for the requested degree. Runtime is polynomial in the requested prefix size, while exact coefficient magnitudes may grow rapidly for some families.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 17, 18, 21, and 64.

C.2B · `fibonacci_polynomial_family`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.fibonacci_polynomial_family`. Reflected implementation `komorebi_math.coordinates.polynomials.fibonacci_polynomial_family`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-FAMILY`.

Manifest signature (s).

```
() -> 'PolynomialFamily'
```

Definition. Return the graded basis `G_n(x)=F_(n+1)(x)`.

Contract (D, C, H, R, X). The constructor returns a `PolynomialFamily` whose degree rule is exact and graded. Later calls to `polynomial(degree)` reject negative or noninteger degrees and verify that the leading coefficient is nonzero at the declared degree.

Guarantees (E, P, B, K). The construction uses integer and rational arithmetic only. Recurrence and coefficient generation are finite and deterministic for the requested degree. Runtime is polynomial in the requested prefix size, while exact coefficient magnitudes may grow rapidly for some families.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 17, 18, 21, and 64.

C.2B · `finite_difference_coordinate_operator`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.finite_difference_coordinate_operator`. Reflected implementation `komorebi_math.coordinates.calculus.finite_difference_coordinate_operator`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-OPERATOR`.

Manifest signature (s).

```
(family: 'PolynomialFamily', degree: 'int', *, step: 'Scalar' = 1, budget: 'CoordinateBudget' =
    ↪ CoordinateBudget(maximum_matrix_dimension=512, maximum_polynomial_degree=512, maximum_horizon=4096,
    ↪ maximum_exact_entries=1000000)) -> 'PolynomialCoordinateOperator'
```

Definition. Return `Delta_h p(x)=p(x+h)-p(x)` in one family.

Contract (D, C, H, R, X). The step must be an exact nonzero rational value. A zero step raises `ValidationError`. The result is the degree-preserving forward-difference operator `p(x+step)-p(x)`.

Guarantees (E, P, B, K). All coefficients and certificates are exact. Dense family matrices use quadratic storage and inversion or conjugation can require cubic arithmetic-operation cost in the truncation dimension. The coordinate budget limits declared matrix dimensions and polynomial degrees. Exact coefficients may grow.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_calculus.py`, `tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 18, 32, and 64.

C.2B · `get_polynomial_family`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.get_polynomial_family`. Reflected implementation `komorebi_math.coordinates.classical.get_polynomial_family`. The same runtime object is also available through `komorebi_math.get_polynomial_family`. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-FAMILY`.

Manifest signature (s).

```
(name: 'str') -> 'PolynomialFamily'
```

Definition. Return one built-in graded polynomial family by name or short alias.

Contract (D, C, H, R, X). The name is normalized by case and alphanumeric content. Documented short aliases resolve to the corresponding built-in family. Unknown names raise `ValidationError`.

Guarantees (E, P, B, K). The construction uses integer and rational arithmetic only. Recurrence and coefficient generation are finite and deterministic for the requested degree. Runtime is polynomial in the requested prefix size, while exact coefficient magnitudes may grow rapidly for some families.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_classical.py`, `tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 15, 17, 18, and 64.

C.2B · `hermite_physicists_polynomial`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.hermite_physicists_polynomial`. Reflected implementation `komorebi_math.coordinates.classical.hermite_physicists_polynomial`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-CLASSICAL`.

Manifest signature (s).

```
(degree: 'int') -> 'Polynomial'
```

Definition. Return the physicists' Hermite polynomial `H_degree`.

Contract (D, C, H, R, X). The degree must be a nonnegative integer. The returned immutable tuple stores exact coefficients in ascending monomial order and follows the recurrence or closed formula named in the definition.

Guarantees (E, P, B, K). The construction uses integer and rational arithmetic only. Recurrence and coefficient generation are finite and deterministic for the requested degree. Runtime is polynomial in the requested prefix size, while exact coefficient magnitudes may grow rapidly for some families.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_classical.py`, `tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 15, 17, 18, and 64.

C.2B · `hermite_physicists_polynomial_family`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.hermite_physicists_polynomial_family`. Reflected implementation `komorebi_math.coordinates.classical.hermite_physicists_polynomial_family`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-FAMILY`.

Manifest signature (s).

```
() -> 'PolynomialFamily'
```

Definition. Return the physicists' Hermite family.

Contract (D, C, H, R, X). The constructor returns a `PolynomialFamily` whose degree rule is exact and graded. Later calls to `polynomial(degree)` reject negative or noninteger degrees and verify that the leading coefficient is nonzero at the declared degree.

Guarantees (E, P, B, K). The construction uses integer and rational arithmetic only. Recurrence and coefficient generation are finite and deterministic for the requested degree. Runtime is polynomial in the requested prefix size, while exact coefficient magnitudes may grow rapidly for some families.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_classical.py, tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 15, 17, 18, and 64.

C.2B · `hermite_probabilists_polynomial`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.hermite_probabilists_polynomial`. Reflected implementation `komorebi_math.coordinates.classical.hermite_probabilists_polynomial`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-CLASSICAL`.

Manifest signature (s).

```
(degree: 'int') -> 'Polynomial'
```

Definition. Return the probabilists' Hermite polynomial `He_degree`.

Contract (D, C, H, R, X). The degree must be a nonnegative integer. The returned immutable tuple stores exact coefficients in ascending monomial order and follows the recurrence or closed formula named in the definition.

Guarantees (E, P, B, K). The construction uses integer and rational arithmetic only. Recurrence and coefficient generation are finite and deterministic for the requested degree. Runtime is polynomial in the requested prefix size, while exact coefficient magnitudes may grow rapidly for some families.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_classical.py, tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 15, 17, 18, and 64.

C.2B · `hermite_probabilists_polynomial_family`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.hermite_probabilists_polynomial_family`. Reflected implementation `komorebi_math.coordinates.classical.hermite_probabilists_polynomial_family`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-FAMILY`.

Manifest signature (s).

```
() -> 'PolynomialFamily'
```

Definition. Return the probabilists' Hermite family.

Contract (D, C, H, R, X). The constructor returns a `PolynomialFamily` whose degree rule is exact and graded. Later calls to `polynomial(degree)` reject negative or noninteger degrees and verify that the leading coefficient is nonzero at the declared degree.

Guarantees (E, P, B, K). The construction uses integer and rational arithmetic only. Recurrence and coefficient generation are finite and deterministic for the requested degree. Runtime is polynomial in the requested prefix size, while exact coefficient magnitudes may grow rapidly for some families.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_classical.py`, `tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 15, 17, 18, and 64.

C.2B · integral_coordinate_operator

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.integral_coordinate_operator`. Reflected implementation `komorebi_math.coordinates.calculus.integral_coordinate_operator`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-OPERATOR`.

Manifest signature (s).

```
(family: 'PolynomialFamily', degree: 'int', *, budget: 'CoordinateBudget' =
    ↪ CoordinateBudget(maximum_matrix_dimension=512, maximum_polynomial_degree=512, maximum_horizon=4096,
    ↪ maximum_exact_entries=1000000)) -> 'PolynomialCoordinateOperator'
```

Definition. Return zero-constant integration in one graded polynomial family.

Contract (D, C, H, R, X). The family and degree must satisfy the polynomial-operator contract. The result maps degree `d` coefficients to degree `d+1` coefficients using the zero-constant antiderivative and carries theorem identifier `CC.03`.

Guarantees (E, P, B, K). All coefficients and certificates are exact. Dense family matrices use quadratic storage and inversion or conjugation can require cubic arithmetic-operation cost in the truncation dimension. The coordinate budget limits declared matrix dimensions and polynomial degrees. Exact coefficients may grow.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_calculus.py`, `tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 18, 32, and 64.

C.2B · interpolation_transform

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.interpolation_transform`. Reflected implementation `komorebi_math.coordinates.classical.interpolation_transform`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-INTERPOLATION`.

Manifest signature (s).

```
(family: 'PolynomialFamily', nodes: 'Sequence[Scalar]', *, budget: 'CoordinateBudget' =
    ↪ CoordinateBudget(maximum_matrix_dimension=512, maximum_polynomial_degree=512, maximum_horizon=4096,
    ↪ maximum_exact_entries=1000000)) -> 'CoordinateTransform'
```

Definition. Map exact node values to coefficients in a polynomial family.

Contract (D, C, H, R, X). Nodes must satisfy the evaluation-transform contract. The result is the exact inverse map from node values to family coefficients.

Guarantees (E, P, B, K). Interpolation is the exact inverse of the evaluation matrix. Dense exact inversion requires cubic arithmetic-operation cost in the node count and may increase numerator and denominator sizes.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_classical.py`, `tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 15, 17, 18, and 64.

C.2B · `lagrange_interpolation_basis`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.lagrange_interpolation_basis`. Reflected implementation `komorebi_math.coordinates.classical.lagrange_interpolation_basis`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-INTERPOLATION`.

Manifest signature (s).

```
(nodes: 'Sequence[Scalar]') -> 'FiniteBasis'
```

Definition. Return the cardinal Lagrange basis at distinct exact nodes.

Contract (D, C, H, R, X). Nodes must form a nonempty finite exact sequence with no duplicates. The returned basis is cardinal at those nodes, so evaluating basis vector `i` at node `j` gives the Kronecker delta.

Guarantees (E, P, B, K). All coefficients and certificates are exact. Dense family matrices use quadratic storage and inversion or conjugation can require cubic arithmetic-operation cost in the truncation dimension. The coordinate budget limits declared matrix dimensions and polynomial degrees. Exact coefficients may grow.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_classical.py`, `tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 15, 17, 18, and 64.

C.2B · `laguerre_polynomial`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.laguerre_polynomial`. Reflected implementation `komorebi_math.coordinates.classical.laguerre_polynomial`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-CLASSICAL`.

Manifest signature (s).

```
(degree: 'int') -> 'Polynomial'
```

Definition. Return the ordinary Laguerre polynomial `L_degree`.

Contract (D, C, H, R, X). The degree must be a nonnegative integer. The returned immutable tuple stores exact coefficients in ascending monomial order and follows the recurrence or closed formula named in the definition.

Guarantees (E, P, B, K). The construction uses integer and rational arithmetic only. Recurrence and coefficient generation are finite and deterministic for the requested degree. Runtime is polynomial in the requested prefix size, while exact coefficient magnitudes may grow rapidly for some families.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_classical.py`, `tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 15, 17, 18, and 64.

C.2B · `laguerre_polynomial_family`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.laguerre_polynomial_family`. Reflected implementation `komorebi_math.coordinates.classical.laguerre_polynomial_family`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-FAMILY`.

Manifest signature (s).

```
() -> 'PolynomialFamily'
```

Definition. Return the graded ordinary Laguerre family.

Contract (D, C, H, R, X). The constructor returns a `PolynomialFamily` whose degree rule is exact and graded. Later calls to `polynomial(degree)` reject negative or noninteger degrees and verify that the leading coefficient is nonzero at the declared degree.

Guarantees (E, P, B, K). The construction uses integer and rational arithmetic only. Recurrence and coefficient generation are finite and deterministic for the requested degree. Runtime is polynomial in the requested prefix size, while exact coefficient magnitudes may grow rapidly for some families.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_classical.py`, `tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 15, 17, 18, and 64.

C.2B · `legendre_polynomial`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.legendre_polynomial`. Reflected implementation `komorebi_math.coordinates.classical.legendre_polynomial`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-CLASSICAL`.

Manifest signature (`s`).

```
(degree: 'int') -> 'Polynomial'
```

Definition. Return the Legendre polynomial `P_degree`.

Contract (`D, C, H, R, X`). The degree must be a nonnegative integer. The returned immutable tuple stores exact coefficients in ascending monomial order and follows the recurrence or closed formula named in the definition.

Guarantees (`E, P, B, K`). The construction uses integer and rational arithmetic only. Recurrence and coefficient generation are finite and deterministic for the requested degree. Runtime is polynomial in the requested prefix size, while exact coefficient magnitudes may grow rapidly for some families.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_classical.py, tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 15, 17, 18, and 64.

C.2B · `legendre_polynomial_family`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinates.legendre_polynomial_family`. Reflected implementation `komorebi_math.coordinates.classical.legendre_polynomial_family`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-FAMILY`.

Manifest signature (`s`).

```
() -> 'PolynomialFamily'
```

Definition. Return the graded Legendre polynomial family.

Contract (`D, C, H, R, X`). The constructor returns a `PolynomialFamily` whose degree rule is exact and graded. Later calls to `polynomial(degree)` reject negative or noninteger degrees and verify that the leading coefficient is nonzero at the declared degree.

Guarantees (`E, P, B, K`). The construction uses integer and rational arithmetic only. Recurrence and coefficient generation are finite and deterministic for the requested degree. Runtime is polynomial in the requested prefix size, while exact coefficient magnitudes may grow rapidly for some families.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_classical.py, tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 15, 17, 18, and 64.

C.2B · `lucas_polynomial`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinates.lucas_polynomial`. Reflected implementation `komorebi_math.coordinates.classical.lucas_polynomial`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-CLASSICAL`.

Manifest signature (s).

```
(index: 'int') -> 'Polynomial'
```

Definition. Return `L_index(x)` with `L_0=2`, `L_1=x`.

Contract (D, C, H, R, X). The index must be a nonnegative integer. The returned immutable tuple stores exact coefficients in ascending monomial order and follows the recurrence or closed formula named in the definition.

Guarantees (E, P, B, K). The construction uses integer and rational arithmetic only. Recurrence and coefficient generation are finite and deterministic for the requested degree. Runtime is polynomial in the requested prefix size, while exact coefficient magnitudes may grow rapidly for some families.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_classical.py`, `tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 15, 17, 18, and 64.

C.2B · `lucas_polynomial_family`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.lucas_polynomial_family`. Reflected implementation `komorebi_math.coordinates.classical.lucas_polynomial_family`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-FAMILY`.

Manifest signature (s).

```
() -> 'PolynomialFamily'
```

Definition. Return the graded Lucas polynomial family.

Contract (D, C, H, R, X). The constructor returns a `PolynomialFamily` whose degree rule is exact and graded. Later calls to `polynomial(degree)` reject negative or noninteger degrees and verify that the leading coefficient is nonzero at the declared degree.

Guarantees (E, P, B, K). The construction uses integer and rational arithmetic only. Recurrence and coefficient generation are finite and deterministic for the requested degree. Runtime is polynomial in the requested prefix size, while exact coefficient magnitudes may grow rapidly for some families.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_classical.py`, `tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 15, 17, 18, and 64.

C.2B · `monomial_polynomial_family`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.monomial_polynomial_family`. Reflected implementation `komorebi_math.coordinates.polynomials.monomial_polynomial_family`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-FAMILY`.

Manifest signature (`s`).

```
() -> 'PolynomialFamily'
```

Definition. Return the monomial family `1,x,x^2,...`.

Contract (`D, C, H, R, X`). The constructor returns a `PolynomialFamily` whose degree rule is exact and graded. Later calls to `polynomial(degree)` reject negative or noninteger degrees and verify that the leading coefficient is nonzero at the declared degree.

Guarantees (`E, P, B, K`). The construction uses integer and rational arithmetic only. Recurrence and coefficient generation are finite and deterministic for the requested degree. Runtime is polynomial in the requested prefix size, while exact coefficient magnitudes may grow rapidly for some families.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 17, 18, 21, and 64.

C.2B · `monomial_to_fibonacci_coefficient`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinates.monomial_to_fibonacci_coefficient`. Reflected implementation `komorebi_math.coordinates.polynomials.monomial_to_fibonacci_coefficient`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-CLASSICAL`.

Manifest signature (`s`).

```
(degree: 'int', fibonacci_degree: 'int') -> 'int'
```

Definition. Return the coefficient of `F_(k+1)` in `x^degree`.

Contract (`D, C, H, R, X`). Both degrees must be nonnegative. Degree or parity incompatibility returns zero rather than raising. Compatible cases use the exact binomial-difference formula.

Guarantees (`E, P, B, K`). The construction uses integer and rational arithmetic only. Recurrence and coefficient generation are finite and deterministic for the requested degree. Runtime is polynomial in the requested prefix size, while exact coefficient magnitudes may grow rapidly for some families.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 17, 18, 21, and 64.

C.2B · `multiplication_coordinate_operator`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinates.multiplication_coordinate_operator`. Reflected implementation `komorebi_math.coordinates.calculus.multiplication_coordinate_operator`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-OPERATOR`.

Manifest signature (`s`).

```
(family: 'PolynomialFamily', degree: 'int', multiplier: 'Sequence[Scalar]', *, budget: 'CoordinateBudget' =
    ↪ CoordinateBudget(maximum_matrix_dimension=512, maximum_polynomial_degree=512, maximum_horizon=4096,
    ↪ maximum_exact_entries=1000000)) -> 'PolynomialCoordinateOperator'
```

Definition. Return multiplication by a fixed exact polynomial.

Contract (`D, C, H, R, X`). The multiplier must be a nonempty exact coefficient sequence. The target degree increases by the multiplier degree. Matrix size and polynomial degree remain subject to the coordinate budget.

Guarantees (`E, P, B, K`). All coefficients and certificates are exact. Dense family matrices use quadratic storage and inversion or conjugation can require cubic arithmetic-operation cost in the truncation dimension. The coordinate budget limits declared matrix dimensions and polynomial degrees. Exact coefficients may grow.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_calculus.py`, `tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 18, 32, and 64.

C.2B · `newton_interpolation_basis`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinates.newton_interpolation_basis`. Reflected implementation `komorebi_math.coordinates.classical.newton_interpolation_basis`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-INTERPOLATION`.

Manifest signature (`s`).

```
(nodes: 'Sequence[Scalar]') -> 'FiniteBasis'
```

Definition. Return the Newton interpolation basis associated with distinct nodes.

Contract (`D, C, H, R, X`). Nodes must form a nonempty finite exact sequence with no duplicates. The basis contains cumulative products over the preceding nodes and uses the input order as part of the coordinate definition.

Guarantees (`E, P, B, K`). All coefficients and certificates are exact. Dense family matrices use quadratic storage and inversion or conjugation can require cubic arithmetic-operation cost in the truncation dimension. The coordinate budget limits declared matrix dimensions and polynomial degrees. Exact coefficients may grow.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_classical.py`, `tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 15, 17, 18, and 64.

C.2B · `operator_change_of_coordinates`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinates.operator_change_of_coordinates`. Reflected implementation `komorebi_math.coordinates.polynomials.operator_change_of_coordinates`. The same runtime object is also available through `komorebi_math.operator_change_of_coordinates`. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-TRANSFORM`.

Manifest signature (`s`).

```
(operator_in_source: 'Matrix', source_to_target: 'CoordinateTransform') -> 'Matrix'
```

Definition. Conjugate an operator from source coordinates into target coordinates.

Contract (`D, C, H, R, X`). The operator must be square and its dimension must equal the source dimension of the coordinate transform. The result is the exact conjugate `C A C^{-1}` in target coordinates.

Guarantees (`E, P, B, K`). All coefficients and certificates are exact. Dense family matrices use quadratic storage and inversion or conjugation can require cubic arithmetic-operation cost in the truncation dimension. The coordinate budget limits declared matrix dimensions and polynomial degrees. Exact coefficients may grow.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 17, 18, 21, and 64.

C.2B · `operator_commutator`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinates.operator_commutator`. Reflected implementation `komorebi_math.coordinates.calculus.operator_commutator`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-OPERATOR`.

Manifest signature (`s`).

```
(left: 'Matrix', right: 'Matrix') -> 'Matrix'
```

Definition. Return the exact commutator `left*right-right*left`.

Contract (`D, C, H, R, X`). Both matrices must have the same square shape. Unequal or rectangular shapes raise `DimensionError`. The result is computed exactly as `left*right-right*left`.

Guarantees (`E, P, B, K`). All coefficients and certificates are exact. Dense family matrices use quadratic storage and inversion or conjugation can require cubic arithmetic-operation cost in the truncation dimension. The coordinate budget limits declared matrix dimensions and polynomial degrees. Exact coefficients may grow.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_calculus.py`, `tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 18, 32, and 64.

C.2B · `operator_matrix_between_families`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinates.operator_matrix_between_families`. Reflected implementation `komorebi_math.coordinates.classical.operator_matrix_between_families`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-TRANSFORM`.

Manifest signature (`s`).

```
(source: 'PolynomialFamily', target: 'PolynomialFamily', source_degree: 'int', target_degree: 'int', operator:
    ↪ 'Callable[[Polynomial], Polynomial]', *, name: 'str' = 'polynomial operator', budget: 'CoordinateBudget' =
    ↪ CoordinateBudget(maximum_matrix_dimension=512, maximum_polynomial_degree=512, maximum_horizon=4096,
    ↪ maximum_exact_entries=1000000)) -> 'Matrix'
```

Definition. Return an exact matrix for a polynomial operator between two families.

Contract (`D, C, H, R, X`). Source and target degrees must be nonnegative and fit the coordinate budget. The callable is applied to every source basis polynomial. Each image must fit the declared target degree, otherwise `DimensionError` is raised during padding.

Guarantees (`E, P, B, K`). The returned matrix is exact. Compilation materializes every image column and inverts the target family matrix, with cubic dense-algebra cost in the target dimension plus the cost of the supplied rule.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_classical.py`, `tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 15, 17, 18, and 64.

C.2B · `pell_polynomial`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinates.pell_polynomial`. Reflected implementation `komorebi_math.coordinates.classical.pell_polynomial`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-CLASSICAL`.

Manifest signature (`s`).

```
(index: 'int') -> 'Polynomial'
```

Definition. Return `P_index(x)` with `P_0=0`, `P_1=1` and `P_n=2xP_(n-1)+P_(n-2)`.

Contract (`D, C, H, R, X`). The index must be a nonnegative integer. The returned immutable tuple stores exact coefficients in ascending monomial order and follows the recurrence or closed formula named in the definition.

Guarantees (`E, P, B, K`). The construction uses integer and rational arithmetic only. Recurrence and coefficient generation are finite and deterministic for the requested degree. Runtime is polynomial in the requested prefix size, while exact coefficient magnitudes may grow rapidly for some families.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_classical.py`, `tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 15, 17, 18, and 64.

C.2B · `pell_polynomial_family`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinates.pell_polynomial_family`. Reflected implementation `komorebi_math.coordinates.classical.pell_polynomial_family`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-FAMILY`.

Manifest signature (s).

```
() -> 'PolynomialFamily'
```

Definition. Return the graded basis `P_(n+1)(x)` of Pell polynomials.

Contract (D, C, H, R, X). The constructor returns a `PolynomialFamily` whose degree rule is exact and graded. Later calls to `polynomial(degree)` reject negative or noninteger degrees and verify that the leading coefficient is nonzero at the declared degree.

Guarantees (E, P, B, K). The construction uses integer and rational arithmetic only. Recurrence and coefficient generation are finite and deterministic for the requested degree. Runtime is polynomial in the requested prefix size, while exact coefficient magnitudes may grow rapidly for some families.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_classical.py`, `tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 15, 17, 18, and 64.

C.2B · `polynomial_add`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.polynomial_add`. Reflected implementation `komorebi_math.coordinates.polynomials.polynomial_add`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-PRIMITIVE`.

Manifest signature (s).

```
(left: 'Sequence[Scalar]', right: 'Sequence[Scalar]') -> 'Polynomial'
```

Definition. Add exact polynomials stored in ascending degree order.

Contract (D, C, H, R, X). Inputs are finite coefficient sequences in ascending degree order and must convert to exact rational values. The result is an immutable coefficient tuple with trailing zeros removed, except that the zero polynomial retains one zero coefficient.

Guarantees (E, P, B, K). Arithmetic is exact over `Fraction`. Addition, scaling, shifting, differentiation, integration, and Horner evaluation are linear in coefficient count. No tolerance, symbolic service, accelerator, network, or filesystem is used.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 17, 18, 21, and 64.

C.2B · `polynomial_compose_affine`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.polynomial_compose_affine`. Reflected implementation `komorebi_math.coordinates.classical.polynomial_compose_affine`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-PRIMITIVE`.

Manifest signature (`S`).

```
(polynomial: 'Sequence[Scalar]', *, scale: 'Scalar' = 1, shift: 'Scalar' = 0) -> 'Polynomial'
```

Definition. Return `polynomial(scale*x+shift)` exactly.

Contract (`D, C, H, R, X`). Inputs are finite coefficient sequences in ascending degree order and must convert to exact rational values. The result is an immutable coefficient tuple with trailing zeros removed, except that the zero polynomial retains one zero coefficient.

Guarantees (`E, P, B, K`). Arithmetic is exact over `Fraction`. Polynomial convolution or affine composition uses quadratic coefficient work in the represented degrees and linear output storage. No tolerance, symbolic service, accelerator, network, or filesystem is used.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_classical.py`, `tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 15, 17, 18, and 64.

C.2B · `polynomial_derivative`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinates.polynomial_derivative`. Reflected implementation `komorebi_math.coordinates.polynomials.polynomial_derivative`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-PRIMITIVE`.

Manifest signature (`S`).

```
(polynomial: 'Sequence[Scalar]') -> 'Polynomial'
```

Definition. Differentiate an exact polynomial.

Contract (`D, C, H, R, X`). Inputs are finite coefficient sequences in ascending degree order and must convert to exact rational values. The result is an immutable coefficient tuple with trailing zeros removed, except that the zero polynomial retains one zero coefficient.

Guarantees (`E, P, B, K`). Arithmetic is exact over `Fraction`. Addition, scaling, shifting, differentiation, integration, and Horner evaluation are linear in coefficient count. No tolerance, symbolic service, accelerator, network, or filesystem is used.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 17, 18, 21, and 64.

C.2B · `polynomial_evaluate`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinates.polynomial_evaluate`. Reflected implementation `komorebi_math.coordinates.polynomials.polynomial_evaluate`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-PRIMITIVE`.

Manifest signature (`s`).

```
(polynomial: 'Sequence[Scalar]', value: 'Scalar') -> 'Fraction'
```

Definition. Evaluate an exact polynomial by Horner's method.

Contract (`D, C, H, R, X`). The coefficient sequence is interpreted in ascending degree order and the evaluation point must convert to an exact rational value. Horner evaluation returns one exact `Fraction`.

Guarantees (`E, P, B, K`). Arithmetic is exact over `Fraction`. Addition, scaling, shifting, differentiation, integration, and Horner evaluation are linear in coefficient count. No tolerance, symbolic service, accelerator, network, or filesystem is used.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 17, 18, 21, and 64.

C.2B · `polynomial_integral`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinates.polynomial_integral`. Reflected implementation `komorebi_math.coordinates.calculus.polynomial_integral`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-OPERATOR`.

Manifest signature (`s`).

```
(polynomial: 'Sequence[Scalar]', constant: 'Scalar' = 0) -> 'Polynomial'
```

Definition. Return one exact antiderivative.

Contract (`D, C, H, R, X`). The coefficient sequence is interpreted in ascending degree order and converted to exact rational values. The supplied constant becomes the degree-zero coefficient. The output degree is one greater than the input representation.

Guarantees (`E, P, B, K`). Arithmetic is exact over `Fraction`. Addition, scaling, shifting, differentiation, integration, and Horner evaluation are linear in coefficient count. No tolerance, symbolic service, accelerator, network, or filesystem is used.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_calculus.py`, `tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 18, 32, and 64.

C.2B · `polynomial_multiply`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinates.polynomial_multiply`. Reflected implementation `komorebi_math.coordinates.polynomials.polynomial_multiply`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-PRIMITIVE`.

Manifest signature (`s`).

```
(left: 'Sequence[Scalar]', right: 'Sequence[Scalar]') -> 'Polynomial'
```

Definition. Multiply exact polynomials.

Contract (`D, C, H, R, X`). Inputs are finite coefficient sequences in ascending degree order and must convert to exact rational values. The result is an immutable coefficient tuple with trailing zeros removed, except that the zero polynomial retains one zero coefficient.

Guarantees (`E, P, B, K`). Arithmetic is exact over `Fraction`. Polynomial convolution or affine composition uses quadratic coefficient work in the represented degrees and linear output storage. No tolerance, symbolic service, accelerator, network, or filesystem is used.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 17, 18, 21, and 64.

C.2B · `polynomial_operator`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinates.polynomial_operator`. Reflected implementation `komorebi_math.coordinates.calculus.polynomial_operator`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-OPERATOR`.

Manifest signature (`s`).

```
(source: 'PolynomialFamily', target: 'PolynomialFamily', source_degree: 'int', target_degree: 'int', rule:
    ↪ 'Callable[[Polynomial], Polynomial]', *, name: 'str', theorem: 'str' = 'CC.O1', budget: 'CoordinateBudget' =
    ↪ CoordinateBudget(maximum_matrix_dimension=512, maximum_polynomial_degree=512, maximum_horizon=4096,
    ↪ maximum_exact_entries=1000000)) -> 'PolynomialCoordinateOperator'
```

Definition. Compile a polynomial rule into coordinates of two families.

Contract (`D, C, H, R, X`). Source and target families, truncation degrees, and the rule must define images that fit the target space. The name is required. The function compiles the rule through `operator_matrix_between_families` and returns a shape-checked `PolynomialCoordinateOperator`.

Guarantees (`E, P, B, K`). All coefficients and certificates are exact. Dense family matrices use quadratic storage and inversion or conjugation can require cubic arithmetic-operation cost in the truncation dimension. The coordinate budget limits declared matrix dimensions and polynomial degrees. Exact coefficients may grow.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_calculus.py`, `tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 18, 32, and 64.

C.2B · `polynomial_scale`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinates.polynomial_scale`. Reflected implementation `komorebi_math.coordinates.polynomials.polynomial_scale`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-PRIMITIVE`.

Manifest signature (`s`).

```
(polynomial: 'Sequence[Scalar]', scalar: 'Scalar') -> 'Polynomial'
```

Definition. Scale an exact polynomial.

Contract (`D, C, H, R, X`). Inputs are finite coefficient sequences in ascending degree order and must convert to exact rational values. The result is an immutable coefficient tuple with trailing zeros removed, except that the zero polynomial retains one zero coefficient.

Guarantees (`E, P, B, K`). Arithmetic is exact over `Fraction`. Addition, scaling, shifting, differentiation, integration, and Horner evaluation are linear in coefficient count. No tolerance, symbolic service, accelerator, network, or filesystem is used.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 17, 18, 21, and 64.

C.2B · `polynomial_shift`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinates.polynomial_shift`. Reflected implementation `komorebi_math.coordinates.polynomials.polynomial_shift`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-PRIMITIVE`.

Manifest signature (`s`).

```
(polynomial: 'Sequence[Scalar]', steps: 'int' = 1) -> 'Polynomial'
```

Definition. Multiply a polynomial by `x^steps`.

Contract (`D, C, H, R, X`). The coefficient sequence is interpreted in ascending degree order. `steps` must be a nonnegative integer. The output is normalized by removing trailing zero coefficients while retaining a canonical zero polynomial.

Guarantees (`E, P, B, K`). Arithmetic is exact over `Fraction`. Addition, scaling, shifting, differentiation, integration, and Horner evaluation are linear in coefficient count. No tolerance, symbolic service, accelerator, network, or filesystem is used.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 17, 18, 21, and 64.

C.2B · `polynomial_subtract`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinates.polynomial_subtract`. Reflected implementation `komorebi_math.coordinates.classical.polynomial_subtract`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-PRIMITIVE`.

Manifest signature (`s`).

```
(left: 'Sequence[Scalar]', right: 'Sequence[Scalar]') -> 'Polynomial'
```

Definition. Subtract exact polynomials.

Contract (`D, C, H, R, X`). Inputs are finite coefficient sequences in ascending degree order and must convert to exact rational values. The result is an immutable coefficient tuple with trailing zeros removed, except that the zero polynomial retains one zero coefficient.

Guarantees (`E, P, B, K`). Arithmetic is exact over `Fraction`. Addition, scaling, shifting, differentiation, integration, and Horner evaluation are linear in coefficient count. No tolerance, symbolic service, accelerator, network, or filesystem is used.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_classical.py`, `tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 15, 17, 18, and 64.

C.2B · `rising_factorial_polynomial`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinates.rising_factorial_polynomial`. Reflected implementation `komorebi_math.coordinates.classical.rising_factorial_polynomial`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-CLASSICAL`.

Manifest signature (`s`).

```
(degree: 'int') -> 'Polynomial'
```

Definition. Return `x^(overline degree)`.

Contract (`D, C, H, R, X`). The degree must be a nonnegative integer. The returned immutable tuple stores exact coefficients in ascending monomial order and follows the recurrence or closed formula named in the definition.

Guarantees (`E, P, B, K`). The construction uses integer and rational arithmetic only. Recurrence and coefficient generation are finite and deterministic for the requested degree. Runtime is polynomial in the requested prefix size, while exact coefficient magnitudes may grow rapidly for some families.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_classical.py`, `tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 15, 17, 18, and 64.

C.2B · `rising_factorial_polynomial_family`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinates.rising_factorial_polynomial_family`. Reflected implementation `komorebi_math.coordinates.classical.rising_factorial_polynomial_family`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-FAMILY`.

Manifest signature (`s`).

```
() -> 'PolynomialFamily'
```

Definition. Return the graded rising-factorial family.

Contract (`D, C, H, R, X`). The constructor returns a `PolynomialFamily` whose degree rule is exact and graded. Later calls to `polynomial(degree)` reject negative or noninteger degrees and verify that the leading coefficient is nonzero at the declared degree.

Guarantees (`E, P, B, K`). The construction uses integer and rational arithmetic only. Recurrence and coefficient generation are finite and deterministic for the requested degree. Runtime is polynomial in the requested prefix size, while exact coefficient magnitudes may grow rapidly for some families.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_classical.py`, `tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 15, 17, 18, and 64.

C.2B · `secant_euler_from_fibonacci_coordinates`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinates.secant_euler_from_fibonacci_coordinates`. Reflected implementation `komorebi_math.coordinates.polynomials.secant_euler_from_fibonacci_coordinates`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-CLASSICAL`.

Manifest signature (`s`).

```
(index: 'int') -> 'Fraction'
```

Definition. Recover the signed secant Euler number from Fibonacci coordinates.

Contract (`D, C, H, R, X`). Index must be nonnegative through the delegated Euler-to-Fibonacci transform. The function evaluates the Fibonacci expansion at `x=1/2` and rescales by `2^index` to recover the signed secant Euler number.

Guarantees (`E, P, B, K`). The returned value is exact. The path computes a dense degree-index connection transform, so arithmetic-operation cost is cubic in the truncation size before the final evaluation. No floating approximation is used.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 17, 18, 21, and 64.

C.2B · `transport_polynomial_operator`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinates.transport_polynomial_operator`. Reflected implementation `komorebi_math.coordinates.calculus.transport_polynomial_operator`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route is not part of the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-POLY-OPERATOR`.

Manifest signature (s).

```
(operator: 'PolynomialCoordinateOperator', source_family: 'PolynomialFamily', target_family: 'PolynomialFamily',
   ↪ *, budget: 'CoordinateBudget' = CoordinateBudget(maximum_matrix_dimension=512,
   ↪ maximum_polynomial_degree=512, maximum_horizon=4096, maximum_exact_entries=1000000)) ->
   ↪ 'PolynomialCoordinateOperator'
```

Definition. Transport an operator between two graded polynomial-family coordinates.

Contract (`D, C, H, R, X`). The input operator must act within the declared source family. Separate domain and codomain connection transforms are built at the source and target degrees. The rectangular operator is then transported exactly into the target family.

Guarantees (`E, P, B, K`). All coefficients and certificates are exact. Dense family matrices use quadratic storage and inversion or conjugation can require cubic arithmetic-operation cost in the truncation dimension. The coordinate budget limits declared matrix dimensions and polynomial degrees. Exact coefficients may grow.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this domain export. `tests/test_coordinates_general_calculus.py`, `tests/test_coordinates_polynomials.py` and the independent verifier for polynomial coordinates cover the applicable formula, basis, roundtrip, operator, failure, and resource behavior. The source validator for polynomial coordinates checks the exact signature, implementation identity, domain facade, root-name collisions, and anchor. See Chapters 18, 32, and 64.

C.2C Coordinate systems, frames, recurrences, realizations, and persistence: 104 entries

This coordinate section documents the additional public contracts in the `komorebi_math.coordinates` manifest. It starts with the shared representations and exact coordinate abstractions, then records numerical frame systems, recurrence coordinates, finite realizations, state-space changes, and the portable envelope routes. The section completes the namespace without promoting research exports into the frozen stable tier.

Contract family	Cards
Exact coordinate infrastructure	35
Analysis, synthesis, and frame algebra	8
Numerical transforms and modal coordinates	7
Recurrence defects, families, and shift operators	29
Realizations and state-space coordinates	18
Portable coordinate records	7
Total	**104**

The exact and numerical paths are intentionally distinguished. Rational basis, recurrence, realization, and state-space objects make finite exact claims. Analysis and synthesis frames and named numerical transforms use NumPy and state their tolerance-qualified guarantees. Portable loaders validate data envelopes and resource limits before object reconstruction.

`RecurrenceModel.synthesize` uses the four committed fields of `RecurrenceCoordinateRecord`. A regression test checks analysis followed by synthesis on a nonhomogeneous Fibonacci prefix. This guards against the `TypeError` caused by supplying an obsolete fifth constructor field.

C.2C · `AffineCoordinateTransform`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinates.AffineCoordinateTransform`. Reflected implementation `komorebi_math.coordinates.affine.AffineCoordinateTransform`. The same runtime object is also available through `komorebi_math.AffineCoordinateTransform`. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C2-CORE-AFFINE`.

Manifest signature (s).

```
(source: 'str', target: 'str', matrix: 'Matrix', offset: 'Vector', theorem: 'str' = 'CC.F1', description: 'str' =
   ↪  '') -> None
```

Definition. An exact affine map `y=A x+b` between equal-dimensional spaces.

Contract (D, C, H, R, X). All coordinates are converted to exact rational values. Affine matrices must be square with an offset of matching length. Barycentric vertices must form a nondegenerate simplex. Composition, inversion, and conversion reject incompatible dimensions or singular linear parts. The principal public operations are `dimension`, `invertible`, `apply`, `inverse`, `then`, `homogeneous_matrix`, and `certificate`.

Guarantees (E, P, B, K). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_general_affine_formal.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 15, 16, and 64.

C.2C · `AnalysisSynthesisBlocks`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.AnalysisSynthesisBlocks`. Reflected implementation `komorebi_math.coordinates.analysis_synthesis.AnalysisSynthesisBlocks`. The same runtime object is also available through `komorebi_math.AnalysisSynthesisBlocks`, and `komorebi_math.stable.AnalysisSynthesisBlocks`. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C2-FRAME-NUMERIC`.

Manifest signature (s).

```
(system: "'AnalysisSynthesisSystem'", groups: 'tuple[tuple[int, ...], ...]', blocks: 'tuple[tuple[np.ndarray,
   ↪  ...], ...]', operator: 'np.ndarray') -> None
```

Definition. Packet blocks of one operator under a declared analysis and synthesis atlas.

Contract (D, C, H, R, X). Inputs are finite NumPy arrays with compatible rectangular shapes. A system requires the supplied synthesis map to be a left inverse of analysis within the declared tolerance. Partitions must cover valid coefficient indices without overlap, and operator blocks must reconstruct the original operator within tolerance. The principal public operations are `reconstruct` and `certificate`.

Guarantees (E, P, B, K). These paths use finite NumPy linear algebra and report tolerance-qualified numerical evidence rather than exact equality. Dense matrices use quadratic storage and common decompositions require cubic floating-point work. Optional accelerator behavior is not implied by this interface.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_block16_analysis_synthesis.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 15, 32, and 64.

C.2C · `AnalysisSynthesisCoefficientBlocks`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.AnalysisSynthesisCoefficientBlocks`. Reflected implementation `komorebi_math.coordinates.frame_algebra.AnalysisSynthesisCoefficientBlocks`. The same runtime object is also available through `komorebi_math.AnalysisSynthesisCoefficientBlocks`. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C2-FRAME-ALGEBRA`.

Manifest signature (s).

```
(mapping: 'AnalysisSynthesisMap', source_partition: 'CoefficientPartition', target_partition:
   ↪  'CoefficientPartition') -> None
```

Definition. Small target-by-source blocks derived from a coefficient lift.

Contract (D, C, H, R, X). Numerical matrices must have compatible finite shapes and satisfy the declared reconstruction law within tolerance. Gauge projections, coefficient lifts, partitions, adjoints, blocks, sums, and compositions retain explicit domain and codomain systems and reject incompatible presentations. The principal public operations are `assemble`, `reblock`, `physical_block`, `then`, and `certificate`.

Guarantees (E, P, B, K). These paths use finite NumPy linear algebra and report tolerance-qualified numerical evidence rather than exact equality. Dense matrices use quadratic storage and common decompositions require cubic floating-point work. Optional accelerator behavior is not implied by this interface.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_block18_frame_algebra.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 15, 32, and 64.

C.2C · `AnalysisSynthesisGauge`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.AnalysisSynthesisGauge`. Reflected implementation `komorebi_math.coordinates.frame_algebra.AnalysisSynthesisGauge`. The same runtime object is also available through `komorebi_math.AnalysisSynthesisGauge`. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C2-FRAME-ALGEBRA`.

Manifest signature (s).

```
(system: 'AnalysisSynthesisSystem') -> None
```

Definition. Gauge and quotient calculus for one numerical presentation.

Contract (D, C, H, R, X). Numerical matrices must have compatible finite shapes and satisfy the declared reconstruction law within tolerance. Gauge projections, coefficient lifts, partitions, adjoints, blocks, sums, and compositions retain explicit domain and codomain systems and reject incompatible presentations. The principal public operations are `analysis`, `synthesis`, `source`, `coordinates`, `source_dimension`, `coordinate_dimension`, `tolerance`, and `coefficient_projection`. Additional methods remain listed in the class documentation.

Guarantees (E, P, B, K). These paths use finite NumPy linear algebra and report tolerance-qualified numerical evidence rather than exact equality. Dense matrices use quadratic storage and common decompositions require cubic floating-point work. Optional accelerator behavior is not implied by this interface.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_block18_frame_algebra.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 15, 32, and 64.

C.2C · `AnalysisSynthesisMap`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.AnalysisSynthesisMap`. Reflected implementation `komorebi_math.coordinates.frame_algebra.AnalysisSynthesisMap`. The same runtime object is also available through `komorebi_math.AnalysisSynthesisMap`. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C2-FRAME-ALGEBRA`.

Manifest signature (s).

```
(domain: 'AnalysisSynthesisSystem', codomain: 'AnalysisSynthesisSystem', operator: 'np.ndarray', theorem: 'str' =
    ↪  'KM-AS-MAP-01') -> None
```

Definition. A physical linear map and its canonical coefficient lift.

Contract (D, C, H, R, X). Numerical matrices must have compatible finite shapes and satisfy the declared reconstruction law within tolerance. Gauge projections, coefficient lifts, partitions, adjoints, blocks, sums, and compositions retain explicit domain and codomain systems and reject incompatible presentations. The principal public operations are `identity`, `tolerance`, `coefficient_lift`, `reconstruct`, `apply_source`, `apply_coefficients`, `then`, and `compose`. Additional methods remain listed in the class documentation.

Guarantees (E, P, B, K). These paths use finite NumPy linear algebra and report tolerance-qualified numerical evidence rather than exact equality. Dense matrices use quadratic storage and common decompositions require cubic floating-point work. Optional accelerator behavior is not implied by this interface.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_block18_frame_algebra.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 15, 32, and 64.

C.2C · `AnalysisSynthesisSystem`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.AnalysisSynthesisSystem`. Reflected implementation `komorebi_math.coordinates.analysis_synthesis.AnalysisSynthesisSystem`. The same runtime object is also available through `komorebi_math.AnalysisSynthesisSystem`, and `komorebi_math.stable.AnalysisSynthesisSystem`. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C2-FRAME-NUMERIC`.

Manifest signature (s).

```
(analysis: 'np.ndarray', synthesis: 'np.ndarray', source: 'str' = 'source', coordinates: 'str' = 'coordinates',
    ↪  tolerance: 'float' = 1e-12, theorem: 'str' = 'KM-AS-01') -> None
```

Definition. A finite analysis map `A` and synthesis map `S` with `S A = I`.

Contract (D, C, H, R, X). Inputs are finite NumPy arrays with compatible rectangular shapes. A system requires the supplied synthesis map to be a left inverse of analysis within the declared tolerance. Partitions must cover valid coefficient indices without overlap, and operator blocks must reconstruct the original operator within tolerance. The principal public operations are `from_invertible`, `canonical_dual`, `source_dimension`, `coordinate_dimension`, `redundant`, `square`, `frame_operator`, and `analyze`. Additional methods remain listed in the class documentation.

Guarantees (E, P, B, K). These paths use finite NumPy linear algebra and report tolerance-qualified numerical evidence rather than exact equality. Dense matrices use quadratic storage and common decompositions require cubic floating-point work. Optional accelerator behavior is not implied by this interface.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_block16_analysis_synthesis.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 15, 32, and 64.

C.2C · AtlasPath

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.AtlasPath`. Reflected implementation `komorebi_math.coordinates.basis.AtlasPath`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C2-CORE-BASIS`.

Manifest signature (s).

```
(systems: 'tuple[str, ...]', transform: 'CoordinateTransform') -> None
```

Definition. One composed path through a coordinate atlas.

Contract (D, C, H, R, X). Names and dimensions must be consistent. A `FiniteBasis` requires nonempty vectors of one ambient dimension and exact linear independence. Atlas edges must agree with their registered endpoint systems, while path comparison requires common endpoints. The principal public operations are `length`.

Guarantees (E, P, B, K). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_general_basis.py`, `tests/test_coordinates_exact.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 15, 16, and 64.

C.2C · BarycentricCoordinateSystem

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.BarycentricCoordinateSystem`. Reflected implementation `komorebi_math.coordinates.affine.BarycentricCoordinateSystem`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C2-CORE-AFFINE`.

Manifest signature (s).

```
(name: 'str', vertices: 'tuple[Vector, ...]') -> None
```

Definition. Barycentric coordinates of an exact nondegenerate simplex.

Contract (D, C, H, R, X). All coordinates are converted to exact rational values. Affine matrices must be square with an offset of matching length. Barycentric vertices must form a nondegenerate simplex. Composition, inversion, and conversion reject incompatible dimensions or singular linear parts. The principal public operations are `dimension`, `augmented_vertex_matrix`, `to_cartesian`, `from_cartesian`, and `certificate`.

Guarantees (E, P, B, K). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_general_affine_formal.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 15, 16, and 64.

C.2C · `BezoutDefectIdentity`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinates.BezoutDefectIdentity`. Reflected implementation `komorebi_math.coordinates.operators.BezoutDefectIdentity`. The same runtime object is also available through `komorebi_math.BezoutDefectIdentity`. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C2-RECURRENCE-OPERATOR`.

Manifest signature (`s`).

```
(gcd: 'Vector', left_multiplier: 'Vector', right_multiplier: 'Vector', reconstructed: 'Vector', expected:
    ↪ 'Vector', valid: 'bool') -> None
```

Definition. A finite prefix reconstruction from two coprime defect streams.

Contract (`D, C, H, R, X`). Polynomial coefficients are exact and stored in ascending shift degree. Euclidean division rejects the zero polynomial. The Bezout reconstruction requires two defect polynomials with the documented coprimality and finite-prefix compatibility conditions. The principal public operations are `certificate`.

Guarantees (`E, P, B, K`). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_operators.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 16, 21, and 32.

C.2C · `CoefficientPartition`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinates.CoefficientPartition`. Reflected implementation `komorebi_math.coordinates.frame_algebra.CoefficientPartition`. The same runtime object is also available through `komorebi_math.CoefficientPartition`. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C2-FRAME-ALGEBRA`.

Manifest signature (`s`).

```
(dimension: 'int', groups: 'tuple[tuple[int, ...], ...]') -> None
```

Definition. A strict ordered partition of finite coefficient indices.

Contract (`D, C, H, R, X`). Numerical matrices must have compatible finite shapes and satisfy the declared reconstruction law within tolerance. Gauge projections, coefficient lifts, partitions, adjoints, blocks, sums, and compositions retain explicit domain and codomain systems and reject incompatible presentations. The principal public operations are `singletons`, `one_block`, `refines`, and `coarsens`.

Guarantees (`E, P, B, K`). These paths use finite NumPy linear algebra and report tolerance-qualified numerical evidence rather than exact equality. Dense matrices use quadratic storage and common decompositions require cubic floating-point work. Optional accelerator behavior is not implied by this interface.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. `tests/test_block18_frame_algebra.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 15, 32, and 64.

C.2C · CommutingDiagramCheck

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.CommutingDiagramCheck`. Reflected implementation `komorebi_math.coordinates.basis.CommutingDiagramCheck`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C2-CORE-BASIS`.

Manifest signature (s).

```
(left: 'AtlasPath', right: 'AtlasPath', valid: 'bool', difference: 'Matrix') -> None
```

Definition. Exact comparison of two paths with common endpoints.

Contract (D, C, H, R, X). Names and dimensions must be consistent. A `FiniteBasis` requires nonempty vectors of one ambient dimension and exact linear independence. Atlas edges must agree with their registered endpoint systems, while path comparison requires common endpoints. The principal public operations are `certificate`.

Guarantees (E, P, B, K). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_general_basis.py`, `tests/test_coordinates_exact.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 15, 16, and 64.

C.2C · CoordinateAtlas

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.CoordinateAtlas`. Reflected implementation `komorebi_math.coordinates.basis.CoordinateAtlas`. The same runtime object is also available through `komorebi_math.CoordinateAtlas`, and `komorebi_math.stable.CoordinateAtlas`. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C2-CORE-BASIS`.

Manifest signature (s).

```
(name: 'str' = 'coordinate atlas') -> None
```

Definition. A graph of exact coordinate systems and composable transforms.

Contract (D, C, H, R, X). Names and dimensions must be consistent. A `FiniteBasis` requires nonempty vectors of one ambient dimension and exact linear independence. Atlas edges must agree with their registered endpoint systems, while path comparison requires common endpoints. The principal public operations are `register_system`, `register_basis`, `register_transform`, `systems`, `transforms`, `direct_transform`, `resolve`, and `compose_path`. Additional methods remain listed in the class documentation.

Guarantees (E, P, B, K). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_general_basis.py`, `tests/test_coordinates_exact.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 15, 16, and 64.

C.2C · CoordinateBudget

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.CoordinateBudget`. Reflected implementation `komorebi_math.coordinates.budget.CoordinateBudget`. The same runtime object is also available through `komorebi_math.CoordinateBudget`. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C2-CORE-BUDGET`.

Manifest signature (s).

```
(maximum_matrix_dimension: 'int' = 512, maximum_polynomial_degree: 'int' = 512, maximum_horizon: 'int' = 4096,
    ↪  maximum_exact_entries: 'int' = 1000000) -> None
```

Definition. Finite limits applied before coordinate matrices are materialized.

Contract (D, C, H, R, X). Every limit must be a positive integer. The `require_matrix`, `require_degree`, and `require_horizon` methods reject work before an oversized exact object is materialized. The principal public operations are `require_matrix`, `require_degree`, and `require_horizon`.

Guarantees (E, P, B, K). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_general_basis.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 15, 16, and 72.

C.2C · CoordinateSystem

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.CoordinateSystem`. Reflected implementation `komorebi_math.coordinates.basis.CoordinateSystem`. The same runtime object is also available through `komorebi_math.CoordinateSystem`. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C2-CORE-BASIS`.

Manifest signature (s).

```
(name: 'str', dimension: 'int', kind: 'str' = 'exact', description: 'str' = '') -> None
```

Definition. Metadata for one finite exact coordinate system.

Contract (D, C, H, R, X). Names and dimensions must be consistent. A `FiniteBasis` requires nonempty vectors of one ambient dimension and exact linear independence. Atlas edges must agree with their registered endpoint systems, while path comparison requires common endpoints.

Guarantees (E, P, B, K). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_general_basis.py`, `tests/test_coordinates_exact.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 15, 16, and 64.

C.2C · CoordinateTransform

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.CoordinateTransform`. Reflected implementation `komorebi_math.coordinates.transform.CoordinateTransform`. The same runtime object is also available through `komorebi_math.CoordinateTransform`, and `komorebi_math.stable.CoordinateTransform`. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C2-CORE-TRANSFORM`.

Manifest signature (s).

```
(source: 'str', target: 'str', matrix: 'Matrix', theorem: 'str', description: 'str' = '') -> None
```

Definition. An exact matrix that maps source coordinates to target coordinates.

Contract (D, C, H, R, X). The source and target labels must be nonempty and the matrix must be nonempty, square, and exactly coercible to rational entries. Application checks vector length. Inversion rejects a singular matrix, and composition requires matching intermediate labels and dimensions. The principal public operations are `source_dimension`, `target_dimension`, `square`, `determinant`, `apply`, `inverse`, `then`, and `roundtrip_valid`. Additional methods remain listed in the class documentation.

Guarantees (E, P, B, K). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_general_basis.py`, `tests/test_coordinates_exact.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 15, 16, and 64.

C.2C · DEFAULT_COORDINATE_BUDGET

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.DEFAULT_COORDINATE_BUDGET`. Reflected implementation `komorebi_math.coordinates.budget.CoordinateBudget`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `data`. Profile `C2-CORE-BUDGET`.

Manifest signature (s).

```
null
```

Definition. The package default for finite coordinate dimensions, polynomial degrees, recurrence horizons, and exact-entry counts.

Contract (D, C, H, R, X). Import returns one immutable `CoordinateBudget` instance with the committed default limits. Callers may pass it unchanged or replace it with a stricter budget. Mutating the default is not supported.

Guarantees (E, P, B, K). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_general_basis.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 15, 16, and 72.

C.2C · ExactAnalysisSynthesisMap

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.ExactAnalysisSynthesisMap`. Reflected implementation `komorebi_math.coordinates.frame_algebra.ExactAnalysisSynthesisMap`. The same runtime object is also available through `komorebi_math.ExactAnalysisSynthesisMap`. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C2-FRAME-ALGEBRA`.

Manifest signature (s).

```
(domain: 'ExactAnalysisSynthesisSystem', codomain: 'ExactAnalysisSynthesisSystem', operator: 'Matrix', theorem:
   ↪  'str' = 'KM-AS-EXACT-MAP-01') -> None
```

Definition. An exact rational physical map and coefficient lift.

Contract (D, C, H, R, X). Matrices and vectors are converted to exact rational values. Analysis and synthesis dimensions must agree and their product must equal the exact identity. Physical maps, coefficient lifts, partitions, direct sums, tensor products, and compositions reject incompatible source or target systems. The principal public operations are `identity`, `coefficient_lift`, `reconstruct`, `apply_coefficients`, `then`, `compose`, `adjoint`, and `dual_adjoint`. Additional methods remain listed in the class documentation.

Guarantees (E, P, B, K). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_block18_frame_algebra.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 15, 32, and 64.

C.2C · ExactAnalysisSynthesisSystem

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.ExactAnalysisSynthesisSystem`. Reflected implementation `komorebi_math.coordinates.frame_algebra.ExactAnalysisSynthesisSystem`. The same runtime object is also available through `komorebi_math.ExactAnalysisSynthesisSystem`. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C2-FRAME-ALGEBRA`.

Manifest signature (s).

```
(analysis: 'Matrix', synthesis: 'Matrix', source: 'str' = 'source', coordinates: 'str' = 'coordinates', theorem:
   ↪  'str' = 'KM-AS-EXACT-01') -> None
```

Definition. An exact rational analysis and synthesis presentation.

Contract (D, C, H, R, X). Matrices and vectors are converted to exact rational values. Analysis and synthesis dimensions must agree and their product must equal the exact identity. Physical maps, coefficient lifts, partitions, direct sums, tensor products, and compositions reject incompatible source or target systems. The principal public operations are `from_invertible`, `canonical_dual`, `source_dimension`, `coordinate_dimension`, `redundant`, `frame_operator`, `coefficient_projection`, and `gauge_projection`. Additional methods remain listed in the class documentation.

Guarantees (E, P, B, K). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_block18_frame_algebra.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks

the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 15, 32, and 64.

C.2C · ExactCoordinateTransform

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.ExactCoordinateTransform`. Reflected implementation `komorebi_math.coordinates.transform.CoordinateTransform`. The same runtime object is also available through `komorebi_math.stable.ExactCoordinateTransform`. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C2-CORE-TRANSFORM`.

Manifest signature (s).

```
(source: 'str', target: 'str', matrix: 'Matrix', theorem: 'str', description: 'str' = '') -> None
```

Definition. A compatibility alias for `CoordinateTransform` when the matrix entries are exact rational values.

Contract (D, C, H, R, X). The source and target labels must be nonempty and the matrix must be nonempty, square, and exactly coercible to rational entries. Application checks vector length. Inversion rejects a singular matrix, and composition requires matching intermediate labels and dimensions.

Guarantees (E, P, B, K). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_general_basis.py`, `tests/test_coordinates_exact.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 15, 16, and 64.

C.2C · FiniteBasis

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.FiniteBasis`. Reflected implementation `komorebi_math.coordinates.basis.FiniteBasis`. The same runtime object is also available through `komorebi_math.FiniteBasis`, and `komorebi_math.stable.FiniteBasis`. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C2-CORE-BASIS`.

Manifest signature (s).

```
(name: 'str', vectors: 'tuple[Vector, ...]', labels: 'tuple[str, ...]' = (), description: 'str' = '') -> None
```

Definition. A linearly independent finite basis inside a rational ambient space.

Contract (D, C, H, R, X). Names and dimensions must be consistent. A `FiniteBasis` requires nonempty vectors of one ambient dimension and exact linear independence. Atlas edges must agree with their registered endpoint systems, while path comparison requires common endpoints. The principal public operations are `dimension`, `ambient_dimension`, `matrix`, `square`, `synthesize`, `analyze`, `contains`, and `connection_to`. Additional methods remain listed in the class documentation.

Guarantees (E, P, B, K). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_general_basis.py`, `tests/test_coordinates_exact.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 15, 16, and 64.

C.2C · FormalCoordinateChange

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.FormalCoordinateChange`. Reflected implementation `komorebi_math.coordinates.formal.FormalCoordinateChange`. The same runtime object is also available through `komorebi_math.FormalCoordinateChange`, and `komorebi_math.stable.FormalCoordinateChange`. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C2-CORE-FORMAL`.

Manifest signature (s).

```
(name: 'str', coefficients: 'Vector', source: 'str' = 'source formal coordinate', target: 'str' = 'target formal
    ↪ coordinate') -> None
```

Definition. A finite exact jet `phi(z)=a_1 z+a_2 z^2+...`.

Contract (D, C, H, R, X). Coefficients are interpreted as a finite jet through a declared degree. A coordinate change has zero constant term and a nonzero linear coefficient. Composition and inversion truncate only at the explicit degree and reject inconsistent or singular jets. The principal public operations are `degree`, `linear_coefficient`, `truncate`, `compose`, `inverse`, `pullback_coefficients`, `pushforward_coefficients`, and `substitution_transform`. Additional methods remain listed in the class documentation.

Guarantees (E, P, B, K). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_general_affine_formal.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 18, 21, and 64.

C.2C · IntertwinerSpace

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.IntertwinerSpace`. Reflected implementation `komorebi_math.coordinates.realization.IntertwinerSpace`. The same runtime object is also available through `komorebi_math.IntertwinerSpace`. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C2-REALIZATION`.

Manifest signature (s).

```
(source_dimension: 'int', target_dimension: 'int', basis: 'tuple[Matrix, ...]') -> None
```

Definition. Solutions of `B S=S A` from a source to a target realization.

Contract (D, C, H, R, X). Transition matrices are exact and square. Initial vectors and output covectors must match the state dimension. Recovery and similarity constructors require the stated observability or cyclic-rank condition. Intertwiner spaces solve the finite linear system `B S = S A` exactly. The principal public operations are `dimension`, `contains_invertible_basis_element`, and `certificate`.

Guarantees (E, P, B, K). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_realization.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 16, 20, and 32.

C.2C · LinearRealization

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.LinearRealization`. Reflected implementation `komorebi_math.coordinates.realization.LinearRealization`. The same runtime object is also available through `komorebi_math.LinearRealization`. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C2-REALIZATION`.

Manifest signature (s).

```
(transition: 'Matrix', initial: 'Vector', output: 'Vector', name: 'str' = 'linear realization') -> None
```

Definition. A scalar sequence realization `a_n=h^T A^n v`.

Contract (D, C, H, R, X). Transition matrices are exact and square. Initial vectors and output covectors must match the state dimension. Recovery and similarity constructors require the stated observability or cyclic-rank condition. Intertwiner spaces solve the finite linear system `B S = S A` exactly. The principal public operations are `dimension`, `state`, `term`, and `prefix`.

Guarantees (E, P, B, K). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_realization.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 16, 20, and 32.

C.2C · Matrix

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.Matrix`. Reflected implementation `builtins.tuple`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `data`. Profile `C2-TYPE-ALIAS`.

Manifest signature (s).

```
(*args, **kwargs)
```

Definition. The immutable row-major matrix representation used by the exact coordinate routines.

Contract (D, C, H, R, X). This export is a runtime type or typing alias. It accepts no call contract of its own. The functions that consume it perform dimensional, exactness, variance, and domain validation at their public boundaries.

Guarantees (E, P, B, K). The alias adds no computation and no certificate. It preserves the package representation contract at constant lookup cost. Exactness and resource behavior belong to the consuming operation.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_exact.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 1, 15, and 72.

C.2C · ModalCoordinateSystem

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.ModalCoordinateSystem`. Reflected implementation `komorebi_math.coordinates.numeric.ModalCoordinateSystem`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C2-NUMERIC-TRANSFORM`.

Manifest signature (s).

```
(eigenvalues: 'np.ndarray', synthesis: 'NumericCoordinateTransform', analysis: 'NumericCoordinateTransform',
    ↪ diagonal_residual: 'float') -> None
```

Definition. Numerical modal coordinates for a diagonalizable square matrix.

Contract (D, C, H, R, X). The transform matrix must be finite, two-dimensional, and square when inversion is requested. Application and composition check dimensions and labels. Named Fourier, cosine, Walsh, Haar, and modal constructors enforce their size, rank, and diagonalizability conditions. The principal public operations are `certificate`.

Guarantees (E, P, B, K). These paths use finite NumPy linear algebra and report tolerance-qualified numerical evidence rather than exact equality. Dense matrices use quadratic storage and common decompositions require cubic floating-point work. Optional accelerator behavior is not implied by this interface.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_general_numeric_packet.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 15, 28, and 32.

C.2C · ModularMatrix

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.ModularMatrix`. Reflected implementation `builtins.tuple`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `data`. Profile `C2-TYPE-ALIAS`.

Manifest signature (s).

```
(*args, **kwargs)
```

Definition. The immutable row-major integer matrix representation used after reduction modulo a prime.

Contract (D, C, H, R, X). This export is a runtime type or typing alias. It accepts no call contract of its own. The functions that consume it perform dimensional, exactness, variance, and domain validation at their public boundaries.

Guarantees (E, P, B, K). The alias adds no computation and no certificate. It preserves the package representation contract at constant lookup cost. Exactness and resource behavior belong to the consuming operation.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_exact.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 1, 15, and 72.

C.2C · ModularVector

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.ModularVector`. Reflected implementation `builtins.tuple`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `data`. Profile `C2-TYPE-ALIAS`.

Manifest signature (s).

```
(*args, **kwargs)
```

Definition. The immutable integer-vector representation used after reduction modulo a prime.

Contract (D, C, H, R, X). This export is a runtime type or typing alias. It accepts no call contract of its own. The functions that consume it perform dimensional, exactness, variance, and domain validation at their public boundaries.

Guarantees (E, P, B, K). The alias adds no computation and no certificate. It preserves the package representation contract at constant lookup cost. Exactness and resource behavior belong to the consuming operation.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_exact.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 1, 15, and 72.

C.2C · NormalizedBasis

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.NormalizedBasis`. Reflected implementation `komorebi_math.generating.basis.WeightedMonomialBasis`. The same runtime object is also available through `komorebi_math.NormalizedBasis`. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C2-CORE-NORMALIZED`.

Manifest signature (s).

```
(name: 'str', gamma: 'GammaRule', *, descriptor: 'BasisDescriptor | None' = None) -> 'None'
```

Definition. A public alias for `WeightedMonomialBasis`, which records a nonzero weight for each monomial degree.

Contract (D, C, H, R, X). The basis weight at every used degree must be nonzero. Coefficient conversion multiplies or divides by those weights exactly, and basis constructors reject invalid degrees or vanishing normalization factors.

Guarantees (E, P, B, K). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_normalized_persistence.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 17, 18, and 64.

C.2C · NumericCoordinateTransform

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.NumericCoordinateTransform`. Reflected implementation `komorebi_math.coordinates.numeric.NumericCoordinateTransform`. The same runtime object is also available through `komorebi_math.NumericCoordinateTransform`, and `komorebi_math.stable.NumericCoordinateTransform`. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C2-NUMERIC-TRANSFORM`.

Manifest signature (s).

```
(source: 'str', target: 'str', matrix: 'np.ndarray', theorem: 'str' = 'CC.NM1', description: 'str' = '', tolerance:
    ↪  'float' = 1e-12) -> None
```

Definition. A finite numerical matrix between coordinate systems.

Contract (D, C, H, R, X). The transform matrix must be finite, two-dimensional, and square when inversion is requested. Application and composition check dimensions and labels. Named Fourier, cosine, Walsh, Haar, and modal constructors enforce their size, rank, and diagonalizability conditions. The principal public operations are `source_dimension`, `target_dimension`, `square`, `apply`, `inverse`, `then`, `condition_number`, and `certificate`.

Guarantees (E, P, B, K). These paths use finite NumPy linear algebra and report tolerance-qualified numerical evidence rather than exact equality. Dense matrices use quadratic storage and common decompositions require cubic floating-point work. Optional accelerator behavior is not implied by this interface.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_general_numeric_packet.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 15, 28, and 32.

C.2C · `Polynomial`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.Polynomial`. Reflected implementation `builtins.tuple`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `data`. Profile `C2-TYPE-ALIAS`.

Manifest signature (s).

```
(*args, **kwargs)
```

Definition. The immutable ascending-degree coefficient tuple used for exact univariate polynomials.

Contract (D, C, H, R, X). This export is a runtime type or typing alias. It accepts no call contract of its own. The functions that consume it perform dimensional, exactness, variance, and domain validation at their public boundaries.

Guarantees (E, P, B, K). The alias adds no computation and no certificate. It preserves the package representation contract at constant lookup cost. Exactness and resource behavior belong to the consuming operation.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_exact.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 1, 15, and 72.

C.2C · `RecurrenceCoordinateRecord`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.RecurrenceCoordinateRecord`. Reflected implementation `komorebi_math.coordinates.recurrence.RecurrenceCoordinateRecord`. The same runtime object is also available through `komorebi_math.RecurrenceCoordinateRecord`. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C2-RECURRENCE-DEFECT`.

Manifest signature (s).

```
(name: 'str', coefficients: 'Vector', initial: 'Vector', defects: 'Vector') -> None
```

Definition. Initial state and forcing coordinates for one recurrence model.

Contract (D, C, H, R, X). Recurrence coefficients are exact and must define positive order. A sequence prefix must contain at least the initial state when analysis or synthesis needs it. Transform dimensions are tied to the requested finite length, and all defect, Green-kernel, and reconstruction routines reject inconsistent orders or horizons. The principal public operations are `order`, `length`, `homogeneous`, `coordinates`, `reconstruct`, `certificate`, and `as_linear_recurrence`.

Guarantees (E, P, B, K). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_recurrence.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 16, 20, and 21.

C.2C · `RecurrenceLike`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.RecurrenceLike`. Reflected implementation `typing.Union`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `data`. Profile `C2-TYPE-ALIAS`.

Manifest signature (s).

```
(*args, **kwargs)
```

Definition. The accepted recurrence specification, either an iterable of exact coefficients or a `LinearRecurrence` object.

Contract (D, C, H, R, X). This export is a runtime type or typing alias. It accepts no call contract of its own. The functions that consume it perform dimensional, exactness, variance, and domain validation at their public boundaries.

Guarantees (E, P, B, K). The alias adds no computation and no certificate. It preserves the package representation contract at constant lookup cost. Exactness and resource behavior belong to the consuming operation.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_general_tensor_modular.py`, `tests/test_coordinates_recurrence.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 1, 15, and 72.

C.2C · `RecurrenceModel`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.RecurrenceModel`. Reflected implementation `komorebi_math.coordinates.families.RecurrenceModel`. The same runtime object is also available through `komorebi_math.RecurrenceModel`. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C2-RECURRENCE-FAMILY`.

Manifest signature (`s`).

```
(name: 'str', coefficients: 'Vector', initial_values: 'Vector' = (), description: 'str' = '') -> None
```

Definition. A named constant-coefficient recurrence and optional initial state.

Contract (`D, C, H, R, X`). A model has a nonempty name, a positive-order exact coefficient tuple, and an optional initial state of the same order. Generation, analysis, synthesis, homogeneous bases, and model-to-model bridges enforce the requested horizon and coordinate length. Unknown catalog names raise `ValidationError`. The principal public operations are `order`, `companion`, `sequence`, `analyze`, `synthesize`, `coordinate_transform_to`, and `homogeneous_basis`.

Guarantees (`E, P, B, K`). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_general_families_state.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 16, 20, and 21.

C.2C · `Scalar`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinates.Scalar`. Reflected implementation `types.UnionType`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `data`. Profile `C2-TYPE-ALIAS`.

Manifest signature (`s`).

```
null
```

Definition. The exact scalar input union consisting of Python integers and rational `Fraction` values.

Contract (`D, C, H, R, X`). This export is a runtime type or typing alias. It accepts no call contract of its own. The functions that consume it perform dimensional, exactness, variance, and domain validation at their public boundaries.

Guarantees (`E, P, B, K`). The alias adds no computation and no certificate. It preserves the package representation contract at constant lookup cost. Exactness and resource behavior belong to the consuming operation.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_exact.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 1, 15, and 72.

C.2C · `ShiftPolynomial`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinates.ShiftPolynomial`. Reflected implementation `komorebi_math.coordinates.operators.ShiftPolynomial`. The same runtime object is also available through `komorebi_math.ShiftPolynomial`. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C2-RECURRENCE-OPERATOR`.

Manifest signature (s).

```
(coefficients: 'Iterable[Scalar]', name: 'str' = 'shift polynomial') -> 'None'
```

Definition. A polynomial `q(L)` in the canonical coefficient shift operator.

Contract (D, C, H, R, X). Polynomial coefficients are exact and stored in ascending shift degree. Euclidean division rejects the zero polynomial. The Bezout reconstruction requires two defect polynomials with the documented coprimality and finite-prefix compatibility conditions. The principal public operations are `degree`, `apply`, and `multiply`.

Guarantees (E, P, B, K). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_operators.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 16, 21, and 32.

C.2C · `StateRecovery`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.StateRecovery`. Reflected implementation `komorebi_math.coordinates.realization.StateRecovery`. The same runtime object is also available through `komorebi_math.StateRecovery`. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C2-REALIZATION`.

Manifest signature (s).

```
(exact: 'bool', unique: 'bool', rank: 'int', horizon: 'int', state: 'Vector | None', predicted: 'Vector', residual:
   ↪  'Vector') -> None
```

Definition. The result of state recovery over a finite horizon.

Contract (D, C, H, R, X). Transition matrices are exact and square. Initial vectors and output covectors must match the state dimension. Recovery and similarity constructors require the stated observability or cyclic-rank condition. Intertwiner spaces solve the finite linear system `B S = S A` exactly. The principal public operations are `certificate`.

Guarantees (E, P, B, K). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_realization.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 16, 20, and 32.

C.2C · `StateSpaceCoordinateCertificate`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.StateSpaceCoordinateCertificate`. Reflected implementation `komorebi_math.coordinates.state_space.StateSpaceCoordinateCertificate`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C2-STATE-SPACE`.

Manifest signature (`s`).

```
(source: 'StateSpaceModel', target: 'StateSpaceModel', transform: 'CoordinateTransform', markov_count: 'int',
    ↪ valid: 'bool') -> None
```

Definition. Exact invariance record for one state coordinate change.

Contract (`D, C, H, R, X`). The exact matrices `A`, `B`, `C`, and `D` must have compatible state, input, and output dimensions. Controllability and observability coordinate constructors require full state rank. A coordinate-change certificate verifies the transformed realization and finite transfer invariance. The principal public operations are `certificate`.

Guarantees (`E, P, B, K`). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_general_families_state.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 16, 20, and 40.

C.2C · `StateSpaceModel`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinates.StateSpaceModel`. Reflected implementation `komorebi_math.coordinates.state_space.StateSpaceModel`. The same runtime object is also available through `komorebi_math.StateSpaceModel`. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C2-STATE-SPACE`.

Manifest signature (`s`).

```
(name: 'str', transition: 'Matrix', input_matrix: 'Matrix', output_matrix: 'Matrix', feedthrough: 'Matrix') -> None
```

Definition. A finite exact state-space model `x'=Ax+Bu, y=Cx+Du`.

Contract (`D, C, H, R, X`). The exact matrices `A`, `B`, `C`, and `D` must have compatible state, input, and output dimensions. Controllability and observability coordinate constructors require full state rank. A coordinate-change certificate verifies the transformed realization and finite transfer invariance. The principal public operations are `state_dimension`, `input_dimension`, `output_dimension`, `change_coordinates`, `markov_parameter`, and `transfer_equivalent`.

Guarantees (`E, P, B, K`). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_general_families_state.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 16, 20, and 40.

C.2C · `Variance`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinates.Variance`. Reflected implementation `typing.Literal`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `data`. Profile `C2-TYPE-ALIAS`.

Manifest signature (s).

```
(*args, **kwargs)
```

Definition. The tensor-index variance marker with the two accepted values `contravariant` and `covariant`.

Contract (D, C, H, R, X). This export is a runtime type or typing alias. It accepts no call contract of its own. The functions that consume it perform dimensional, exactness, variance, and domain validation at their public boundaries.

Guarantees (E, P, B, K). The alias adds no computation and no certificate. It preserves the package representation contract at constant lookup cost. Exactness and resource behavior belong to the consuming operation.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_general_tensor_modular.py`, `tests/test_coordinates_recurrence.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 1, 15, and 72.

C.2C · `Vector`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.Vector`. Reflected implementation `builtins.tuple`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `data`. Profile `C2-TYPE-ALIAS`.

Manifest signature (s).

```
(*args, **kwargs)
```

Definition. The immutable exact-vector representation whose entries are rational `Fraction` values.

Contract (D, C, H, R, X). This export is a runtime type or typing alias. It accepts no call contract of its own. The functions that consume it perform dimensional, exactness, variance, and domain validation at their public boundaries.

Guarantees (E, P, B, K). The alias adds no computation and no certificate. It preserves the package representation contract at constant lookup cost. Exactness and resource behavior belong to the consuming operation.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_exact.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 1, 15, and 72.

C.2C · `affine_basis_transform`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.affine_basis_transform`. Reflected implementation `komorebi_math.coordinates.affine.affine_basis_transform`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-CORE-AFFINE`.

Manifest signature (s).

```
(origin: 'Sequence[Scalar]', basis_vectors: 'Sequence[Sequence[Scalar]]', *, local_name: 'str' = 'local affine
   ↪  coordinates', ambient_name: 'str' = 'ambient Cartesian coordinates') -> 'AffineCoordinateTransform'
```

Definition. Construct an affine chart from an origin and a square vector basis.

Contract (D, C, H, R, X). All coordinates are converted to exact rational values. Affine matrices must be square with an offset of matching length. Barycentric vertices must form a nondegenerate simplex. Composition, inversion, and conversion reject incompatible dimensions or singular linear parts.

Guarantees (E, P, B, K). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_general_affine_formal.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 15, 16, and 64.

C.2C · `analyze_recurrence_coordinates`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.analyze_recurrence_coordinates`. Reflected implementation `komorebi_math.coordinates.recurrence.analyze_recurrence_coordinates`. The same runtime object is also available through `komorebi_math.analyze_recurrence_coordinates`. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-RECURRENCE-DEFECT`.

Manifest signature (s).

```
(values: 'Sequence[Scalar]', coefficients: 'RecurrenceLike', *, name: 'str' = 'recurrence') ->
    ↪ 'RecurrenceCoordinateRecord'
```

Definition. Analyze a finite sequence prefix in recurrence defect coordinates.

Contract (D, C, H, R, X). Recurrence coefficients are exact and must define positive order. A sequence prefix must contain at least the initial state when analysis or synthesis needs it. Transform dimensions are tied to the requested finite length, and all defect, Green-kernel, and reconstruction routines reject inconsistent orders or horizons.

Guarantees (E, P, B, K). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_recurrence.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 16, 20, and 21.

C.2C · `apply_recurrence_defect`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.apply_recurrence_defect`. Reflected implementation `komorebi_math.coordinates.recurrence.apply_recurrence_defect`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-RECURRENCE-DEFECT`.

Manifest signature (s).

```
(values: 'Sequence[Scalar]', coefficients: 'RecurrenceLike') -> 'Vector'
```

Definition. Compute the unique recurrence defect stream of a finite prefix.

Contract (D, C, H, R, X). Recurrence coefficients are exact and must define positive order. A sequence prefix must contain at least the initial state when analysis or synthesis needs it. Transform dimensions are tied to the requested finite length, and all defect, Green-kernel, and reconstruction routines reject inconsistent orders or horizons.

Guarantees (E, P, B, K). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic

arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_recurrence.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 16, 20, and 21.

C.2C · `basis_from_matrix`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.basis_from_matrix`. Reflected implementation `komorebi_math.coordinates.basis.basis_from_matrix`. The same runtime object is also available through `komorebi_math.basis_from_matrix`, and `komorebi_math.stable.basis_from_matrix`. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-CORE-BASIS`.

Manifest signature (s).

```
(name: 'str', matrix: 'Iterable[Iterable[Scalar]]', *, labels: 'Sequence[str]' = (), description: 'str' = '') ->
    ↪ 'FiniteBasis'
```

Definition. Construct a basis from a matrix whose columns are basis vectors.

Contract (D, C, H, R, X). Inputs are converted to exact rational vectors or matrices before construction. Dimension, independence, nonzero-scale, permutation, and budget conditions are checked before the returned basis or transform is accepted.

Guarantees (E, P, B, K). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_general_basis.py`, `tests/test_coordinates_exact.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 15, 16, and 64.

C.2C · `bezout_defect_identity`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.bezout_defect_identity`. Reflected implementation `komorebi_math.coordinates.operators.bezout_defect_identity`. The same runtime object is also available through `komorebi_math.bezout_defect_identity`. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-RECURRENCE-OPERATOR`.

Manifest signature (s).

```
(values: 'Sequence[Scalar]', left: 'Sequence[Scalar]', right: 'Sequence[Scalar]') -> 'BezoutDefectIdentity'
```

Definition. Reconstruct the common finite interior from two coprime defects.

Contract (D, C, H, R, X). Polynomial coefficients are exact and stored in ascending shift degree. Euclidean division rejects the zero polynomial. The Bezout reconstruction requires two defect polynomials with the documented coprimality and finite-prefix compatibility conditions.

Guarantees (E, P, B, K). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_operators.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 16, 21, and 32.

C.2C · `certify_state_coordinate_change`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.certify_state_coordinate_change`. Reflected implementation `komorebi_math.coordinates.state_space.certify_state_coordinate_change`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-STATE-SPACE`.

Manifest signature (s).

```
(model: 'StateSpaceModel', transform: 'CoordinateTransform', *, markov_count: 'int | None' = None) ->
    ↪ 'StateSpaceCoordinateCertificate'
```

Definition. Certify transfer invariance under an exact state coordinate change.

Contract (D, C, H, R, X). The exact matrices `A`, `B`, `C`, and `D` must have compatible state, input, and output dimensions. Controllability and observability coordinate constructors require full state rank. A coordinate-change certificate verifies the transformed realization and finite transfer invariance.

Guarantees (E, P, B, K). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_general_families_state.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 16, 20, and 40.

C.2C · `companion_matrix`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.companion_matrix`. Reflected implementation `komorebi_math.coordinates.recurrence.companion_matrix`. The same runtime object is also available through `komorebi_math.companion_matrix`. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-RECURRENCE-DEFECT`.

Manifest signature (s).

```
(coefficients: 'RecurrenceLike') -> 'Matrix'
```

Definition. Compute the exact companion matrix for consecutive sequence states.

Contract (D, C, H, R, X). Recurrence coefficients are exact and must define positive order. A sequence prefix must contain at least the initial state when analysis or synthesis needs it. Transform dimensions are tied to the requested finite length, and all defect, Green-kernel, and reconstruction routines reject inconsistent orders or horizons.

Guarantees (E, P, B, K). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_recurrence.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks

the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 16, 20, and 21.

C.2C · compose_ordinary_series

Identity (q, a, t, k). Public research-manifest route komorebi_math.coordinates.compose_ordinary_series. Reflected implementation komorebi_math.coordinates.formal.compose_ordinary_series. No root, stable, or beta facade exports this domain-only name in the reference manifest. The komorebi_math.coordinates route is governed by [RESEARCH] policy pre-beta-additive. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind function. Profile C2-CORE-FORMAL.

Manifest signature (s).

```
(outer: 'Sequence[Scalar]', inner: 'Sequence[Scalar]', degree: 'int') -> 'Vector'
```

Definition. Compute outer(inner(z)) modulo z^(degree+1) exactly.

Contract (D, C, H, R, X). Coefficients are interpreted as a finite jet through a declared degree. A coordinate change has zero constant term and a nonzero linear coefficient. Composition and inversion truncate only at the explicit degree and reject inconsistent or singular jets.

Guarantees (E, P, B, K). Arithmetic is exact over integers and rational Fraction values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. tests/test_coordinates_general_affine_formal.py and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 18, 21, and 64.

C.2C · controllability_basis

Identity (q, a, t, k). Public research-manifest route komorebi_math.coordinates.controllability_basis. Reflected implementation komorebi_math.coordinates.state_space.controllability_basis. No root, stable, or beta facade exports this domain-only name in the reference manifest. The komorebi_math.coordinates route is governed by [RESEARCH] policy pre-beta-additive. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind function. Profile C2-STATE-SPACE.

Manifest signature (s).

```
(model: 'StateSpaceModel') -> 'FiniteBasis'
```

Definition. Compute a maximal independent basis extracted from controllability columns.

Contract (D, C, H, R, X). The exact matrices A, B, C, and D must have compatible state, input, and output dimensions. Controllability and observability coordinate constructors require full state rank. A coordinate-change certificate verifies the transformed realization and finite transfer invariance.

Guarantees (E, P, B, K). Arithmetic is exact over integers and rational Fraction values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. tests/test_coordinates_general_families_state.py and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 16, 20, and 40.

C.2C · controllability_coordinate_transform

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.controllability_coordinate_transform`. Reflected implementation `komorebi_math.coordinates.state_space.controllability_coordinate_transform`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-STATE-SPACE`.

Manifest signature (s).

```
(model: 'StateSpaceModel') -> 'CoordinateTransform'
```

Definition. Map original state coordinates to a full controllability basis.

Contract (D, C, H, R, X). The exact matrices `A`, `B`, `C`, and `D` must have compatible state, input, and output dimensions. Controllability and observability coordinate constructors require full state rank. A coordinate-change certificate verifies the transformed realization and finite transfer invariance.

Guarantees (E, P, B, K). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_general_families_state.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 16, 20, and 40.

C.2C · controllability_matrix

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.controllability_matrix`. Reflected implementation `komorebi_math.coordinates.state_space.controllability_matrix`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-STATE-SPACE`.

Manifest signature (s).

```
(transition: 'Iterable[Iterable[Scalar]]', input_matrix: 'Iterable[Iterable[Scalar]]', *, steps: 'int | None' =
    ↪ None) -> 'Matrix'
```

Definition. Compute the finite controllability block matrix `[B,AB,...]`.

Contract (D, C, H, R, X). The exact matrices `A`, `B`, `C`, and `D` must have compatible state, input, and output dimensions. Controllability and observability coordinate constructors require full state rank. A coordinate-change certificate verifies the transformed realization and finite transfer invariance.

Guarantees (E, P, B, K). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_general_families_state.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 16, 20, and 40.

C.2C · coordinate_change_between_recurrences

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.coordinate_change_between_recurrences`. Reflected implementation `komorebi_math.coordinates.recurrence.coordinate_change_between_recurrences`. The same runtime object is also available through `komorebi_math.coordinate_change_between_recurrences`. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-RECURRENCE-DEFECT`.

Manifest signature (s).

```
(source_coefficients: 'RecurrenceLike', target_coefficients: 'RecurrenceLike', length: 'int', *, source_name:
   ↪ 'str' = 'source recurrence', target_name: 'str' = 'target recurrence', budget: 'CoordinateBudget' =
   ↪ CoordinateBudget(maximum_matrix_dimension=512, maximum_polynomial_degree=512, maximum_horizon=4096,
   ↪ maximum_exact_entries=1000000)) -> 'CoordinateTransform'
```

Definition. Compute the exact finite coordinate map between recurrence models.

Contract (D, C, H, R, X). Recurrence coefficients are exact and must define positive order. A sequence prefix must contain at least the initial state when analysis or synthesis needs it. Transform dimensions are tied to the requested finite length, and all defect, Green-kernel, and reconstruction routines reject inconsistent orders or horizons.

Guarantees (E, P, B, K). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_recurrence.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 16, 20, and 21.

C.2C · custom_recurrence_model

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.custom_recurrence_model`. Reflected implementation `komorebi_math.coordinates.families.custom_recurrence_model`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-RECURRENCE-FAMILY`.

Manifest signature (s).

```
(coefficients: 'Sequence[Scalar]', *, initial_values: 'Sequence[Scalar]' = (), name: 'str' = 'custom recurrence',
   ↪ description: 'str' = '') -> 'RecurrenceModel'
```

Definition. Construct an arbitrary exact constant-coefficient recurrence model.

Contract (D, C, H, R, X). A model has a nonempty name, a positive-order exact coefficient tuple, and an optional initial state of the same order. Generation, analysis, synthesis, homogeneous bases, and model-to-model bridges enforce the requested horizon and coordinate length. Unknown catalog names raise `ValidationError`.

Guarantees (E, P, B, K). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_general_families_state.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 16, 20, and 21.

C.2C · cyclic_krylov_matrix

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.cyclic_krylov_matrix`. Reflected implementation `komorebi_math.coordinates.realization.cyclic_krylov_matrix`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-REALIZATION`.

Manifest signature (s).

```
(transition: 'Matrix', vector: 'Sequence[Scalar]') -> 'Matrix'
```

Definition. Compute the square cyclic Krylov matrix `[v,Av,...]`.

Contract (D, C, H, R, X). Transition matrices are exact and square. Initial vectors and output covectors must match the state dimension. Recovery and similarity constructors require the stated observability or cyclic-rank condition. Intertwiner spaces solve the finite linear system `B S = S A` exactly.

Guarantees (E, P, B, K). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_realization.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 16, 20, and 32.

C.2C · defect_green_weights

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.defect_green_weights`. Reflected implementation `komorebi_math.coordinates.recurrence.defect_green_weights`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-RECURRENCE-DEFECT`.

Manifest signature (s).

```
(coefficients: 'RecurrenceLike', count: 'int') -> 'Vector'
```

Definition. Compute `g_l=e_1^T C^(l-1)e_d` for `1 <= l <= count`.

Contract (D, C, H, R, X). Recurrence coefficients are exact and must define positive order. A sequence prefix must contain at least the initial state when analysis or synthesis needs it. Transform dimensions are tied to the requested finite length, and all defect, Green-kernel, and reconstruction routines reject inconsistent orders or horizons.

Guarantees (E, P, B, K). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_recurrence.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 16, 20, and 21.

C.2C · direct_sum_basis

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.direct_sum_basis`. Reflected implementation `komorebi_math.coordinates.basis.direct_sum_basis`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-CORE-BASIS`.

Manifest signature (s).

```
(left: 'FiniteBasis', right: 'FiniteBasis', *, name: 'str | None' = None) -> 'FiniteBasis'
```

Definition. Compute the external direct sum of two finite bases.

Contract (D, C, H, R, X). Inputs are converted to exact rational vectors or matrices before construction. Dimension, independence, nonzero-scale, permutation, and budget conditions are checked before the returned basis or transform is accepted.

Guarantees (E, P, B, K). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_general_basis.py`, `tests/test_coordinates_exact.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 15, 16, and 64.

C.2C · discrete_cosine_transform

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.discrete_cosine_transform`. Reflected implementation `komorebi_math.coordinates.numeric.discrete_cosine_transform`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-NUMERIC-TRANSFORM`.

Manifest signature (s).

```
(size: 'int', *, orthonormal: 'bool' = True, tolerance: 'float' = 1e-12) -> 'NumericCoordinateTransform'
```

Definition. Compute the type-II discrete cosine transform.

Contract (D, C, H, R, X). The transform matrix must be finite, two-dimensional, and square when inversion is requested. Application and composition check dimensions and labels. Named Fourier, cosine, Walsh, Haar, and modal constructors enforce their size, rank, and diagonalizability conditions.

Guarantees (E, P, B, K). These paths use finite NumPy linear algebra and report tolerance-qualified numerical evidence rather than exact equality. Dense matrices use quadratic storage and common decompositions require cubic floating-point work. Optional accelerator behavior is not implied by this interface.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_general_numeric_packet.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 15, 28, and 32.

C.2C · discrete_fourier_transform

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.discrete_fourier_transform`. Reflected implementation `komorebi_math.coordinates.numeric.discrete_fourier_transform`. The same runtime object is also available through `komorebi_math.discrete_fourier_transform`. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-NUMERIC-TRANSFORM`.

Manifest signature (s).

```
(size: 'int', *, unitary: 'bool' = True, inverse: 'bool' = False, tolerance: 'float' = 1e-12) ->
    ↪ 'NumericCoordinateTransform'
```

Definition. Compute the finite discrete Fourier coordinate transform.

Contract (D, C, H, R, X). The transform matrix must be finite, two-dimensional, and square when inversion is requested. Application and composition check dimensions and labels. Named Fourier, cosine, Walsh, Haar, and modal constructors enforce their size, rank, and diagonalizability conditions.

Guarantees (E, P, B, K). These paths use finite NumPy linear algebra and report tolerance-qualified numerical evidence rather than exact equality. Dense matrices use quadratic storage and common decompositions require cubic floating-point work. Optional accelerator behavior is not implied by this interface.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_general_numeric_packet.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 15, 28, and 32.

C.2C · euler_fibonacci_coordinates

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.euler_fibonacci_coordinates`. Reflected implementation `komorebi_math.coordinates.recurrence.euler_fibonacci_coordinates`. The same runtime object is also available through `komorebi_math.euler_fibonacci_coordinates`. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-RECURRENCE-DEFECT`.

Manifest signature (s).

```
(count: 'int') -> 'RecurrenceCoordinateRecord'
```

Definition. Compute exact Fibonacci companion coordinates of a signed Euler prefix.

Contract (D, C, H, R, X). Recurrence coefficients are exact and must define positive order. A sequence prefix must contain at least the initial state when analysis or synthesis needs it. Transform dimensions are tied to the requested finite length, and all defect, Green-kernel, and reconstruction routines reject inconsistent orders or horizons.

Guarantees (E, P, B, K). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_recurrence.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 16, 20, and 21.

C.2C · exponential_basis

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.exponential_basis`. Reflected implementation `komorebi_math.generating.basis.exponential_basis`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-CORE-NORMALIZED`.

Manifest signature (s).

```
(*, name: 'str' = 'exponential') -> 'WeightedMonomialBasis'
```

Definition. Compute the exponential generating-function basis `z^n/n!`.

Contract (D, C, H, R, X). The basis weight at every used degree must be nonzero. Coefficient conversion multiplies or divides by those weights exactly, and basis constructors reject invalid degrees or vanishing normalization factors.

Guarantees (E, P, B, K). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_normalized_persistence.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 17, 18, and 64.

C.2C · fibonacci_factorial_basis

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.fibonacci_factorial_basis`. Reflected implementation `komorebi_math.generating.basis.fibonacci_factorial_basis`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-CORE-NORMALIZED`.

Manifest signature (s).

```
(*, name: 'str' = 'fibonacci-factorial') -> 'WeightedMonomialBasis'
```

Definition. Compute the fibonomial basis with `gamma_n=prod_(j=1)^n F_j`.

Contract (D, C, H, R, X). The basis weight at every used degree must be nonzero. Coefficient conversion multiplies or divides by those weights exactly, and basis constructors reject invalid degrees or vanishing normalization factors.

Guarantees (E, P, B, K). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_normalized_persistence.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 17, 18, and 64.

C.2C · `fibonacci_numbers`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinates.fibonacci_numbers`. Reflected implementation `komorebi_math.coordinates.recurrence.fibonacci_numbers`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-RECURRENCE-DEFECT`.

Manifest signature (`s`).

```
(count: 'int') -> 'tuple[int, ...]'
```

Definition. Compute Fibonacci numbers beginning with zero.

Contract (`D, C, H, R, X`). Recurrence coefficients are exact and must define positive order. A sequence prefix must contain at least the initial state when analysis or synthesis needs it. Transform dimensions are tied to the requested finite length, and all defect, Green-kernel, and reconstruction routines reject inconsistent orders or horizons.

Guarantees (`E, P, B, K`). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_recurrence.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 16, 20, and 21.

C.2C · `fibonacci_recurrence_model`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinates.fibonacci_recurrence_model`. Reflected implementation `komorebi_math.coordinates.families.fibonacci_recurrence_model`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-RECURRENCE-FAMILY`.

Manifest signature (`s`).

```
() -> 'RecurrenceModel'
```

Definition. Compute the Fibonacci recurrence model.

Contract (`D, C, H, R, X`). A model has a nonempty name, a positive-order exact coefficient tuple, and an optional initial state of the same order. Generation, analysis, synthesis, homogeneous bases, and model-to-model bridges enforce the requested horizon and coordinate length. Unknown catalog names raise `ValidationError`.

Guarantees (`E, P, B, K`). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_general_families_state.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 16, 20, and 21.

C.2C · formal_coordinate_from_iterable

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.formal_coordinate_from_iterable`. Reflected implementation `komorebi_math.coordinates.formal.formal_coordinate_from_iterable`. The same runtime object is also available through `komorebi_math.stable.formal_coordinate_from_iterable`. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-CORE-FORMAL`.

Manifest signature (s).

```
(coefficients: 'Iterable[Scalar]', *, name: 'str' = 'custom formal coordinate', source: 'str' = 'source formal
   ↪  coordinate', target: 'str' = 'target formal coordinate') -> 'FormalCoordinateChange'
```

Definition. Construct a formal coordinate change from exact coefficients.

Contract (D, C, H, R, X). Coefficients are interpreted as a finite jet through a declared degree. A coordinate change has zero constant term and a nonzero linear coefficient. Composition and inversion truncate only at the explicit degree and reject inconsistent or singular jets.

Guarantees (E, P, B, K). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_general_affine_formal.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 18, 21, and 64.

C.2C · get_recurrence_model

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.get_recurrence_model`. Reflected implementation `komorebi_math.coordinates.families.get_recurrence_model`. The same runtime object is also available through `komorebi_math.get_recurrence_model`. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-RECURRENCE-FAMILY`.

Manifest signature (s).

```
(name: 'str') -> 'RecurrenceModel'
```

Definition. Compute one built-in recurrence model by normalized name.

Contract (D, C, H, R, X). A model has a nonempty name, a positive-order exact coefficient tuple, and an optional initial state of the same order. Generation, analysis, synthesis, homogeneous bases, and model-to-model bridges enforce the requested horizon and coordinate length. Unknown catalog names raise `ValidationError`.

Guarantees (E, P, B, K). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_general_families_state.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 16, 20, and 21.

C.2C · haar_wavelet_transform

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.haar_wavelet_transform`. Reflected implementation `komorebi_math.coordinates.numeric.haar_wavelet_transform`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-NUMERIC-TRANSFORM`.

Manifest signature (s).

```
(size: 'int', *, orthonormal: 'bool' = True, tolerance: 'float' = 1e-12) -> 'NumericCoordinateTransform'
```

Definition. Compute a complete finite Haar wavelet transform.

Contract (D, C, H, R, X). The transform matrix must be finite, two-dimensional, and square when inversion is requested. Application and composition check dimensions and labels. Named Fourier, cosine, Walsh, Haar, and modal constructors enforce their size, rank, and diagonalizability conditions.

Guarantees (E, P, B, K). These paths use finite NumPy linear algebra and report tolerance-qualified numerical evidence rather than exact equality. Dense matrices use quadratic storage and common decompositions require cubic floating-point work. Optional accelerator behavior is not implied by this interface.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_general_numeric_packet.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 15, 28, and 32.

C.2C · identity_formal_coordinate

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.identity_formal_coordinate`. Reflected implementation `komorebi_math.coordinates.formal.identity_formal_coordinate`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-CORE-FORMAL`.

Manifest signature (s).

```
(degree: 'int', *, name: 'str' = 'identity formal coordinate', coordinate: 'str' = 'formal coordinate') ->
    ↪ 'FormalCoordinateChange'
```

Definition. Compute the identity formal coordinate jet.

Contract (D, C, H, R, X). Coefficients are interpreted as a finite jet through a declared degree. A coordinate change has zero constant term and a nonzero linear coefficient. Composition and inversion truncate only at the explicit degree and reject inconsistent or singular jets.

Guarantees (E, P, B, K). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_general_affine_formal.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 18, 21, and 64.

C.2C · intertwiner_space

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.intertwiner_space`. Reflected implementation `komorebi_math.coordinates.realization.intertwiner_space`. The same runtime object is also available through `komorebi_math.intertwiner_space`. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-REALIZATION`.

Manifest signature (s).

```
(source: 'Matrix', target: 'Matrix') -> 'IntertwinerSpace'
```

Definition. Compute an exact basis of matrices satisfying `target*S=S*source`.

Contract (D, C, H, R, X). Transition matrices are exact and square. Initial vectors and output covectors must match the state dimension. Recovery and similarity constructors require the stated observability or cyclic-rank condition. Intertwiner spaces solve the finite linear system `B S = S A` exactly.

Guarantees (E, P, B, K). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_realization.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 16, 20, and 32.

C.2C · jacobsthal_recurrence_model

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.jacobsthal_recurrence_model`. Reflected implementation `komorebi_math.coordinates.families.jacobsthal_recurrence_model`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-RECURRENCE-FAMILY`.

Manifest signature (s).

```
() -> 'RecurrenceModel'
```

Definition. Compute the Jacobsthal recurrence model.

Contract (D, C, H, R, X). A model has a nonempty name, a positive-order exact coefficient tuple, and an optional initial state of the same order. Generation, analysis, synthesis, homogeneous bases, and model-to-model bridges enforce the requested horizon and coordinate length. Unknown catalog names raise `ValidationError`.

Guarantees (E, P, B, K). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_general_families_state.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 16, 20, and 21.

C.2C · load_coordinate_object

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.load_coordinate_object`. Reflected implementation `komorebi_math.coordinates.portable.load_coordinate_object`. The same runtime object is also available through `komorebi_math.load_coordinate_object`. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-PORTABLE`.

Manifest signature (s).

```
(path: 'str | Path') -> 'Any'
```

Definition. Load and verify one registered portable coordinate object.

Contract (D, C, H, R, X). Loading verifies the persistence envelope, digest, codec tag, version, required fields, and resource limits before reconstructing an object. Unknown tags, malformed data, unsupported versions, and digest mismatches are rejected.

Guarantees (E, P, B, K). The format is data-only and deterministic for a fixed object, codec version, and serializer. Digest verification detects accidental or adversarial payload changes but is not a cryptographic proof of mathematical truth. Decoding enforces the registered resource limits before large structures are accepted.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_general_persistence.py, tests/test_coordinates_normalized_persistence.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 61, 64, and 72.

C.2C · load_coordinate_record

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.load_coordinate_record`. Reflected implementation `komorebi_math.coordinates.portable.load_coordinate_record`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-PORTABLE`.

Manifest signature (s).

```
(path: 'str | Path') -> 'RecurrenceCoordinateRecord'
```

Definition. Load and verify recurrence defect coordinates.

Contract (D, C, H, R, X). Loading verifies the persistence envelope, digest, codec tag, version, required fields, and resource limits before reconstructing an object. Unknown tags, malformed data, unsupported versions, and digest mismatches are rejected.

Guarantees (E, P, B, K). The format is data-only and deterministic for a fixed object, codec version, and serializer. Digest verification detects accidental or adversarial payload changes but is not a cryptographic proof of mathematical truth. Decoding enforces the registered resource limits before large structures are accepted.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_general_persistence.py, tests/test_coordinates_normalized_persistence.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 61, 64, and 72.

C.2C · load_transform

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.load_transform`. Reflected implementation `komorebi_math.coordinates.portable.load_transform`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-PORTABLE`.

Manifest signature (s).

```
(path: 'str | Path') -> 'CoordinateTransform'
```

Definition. Load and verify one exact coordinate transform.

Contract (D, C, H, R, X). Loading verifies the persistence envelope, digest, codec tag, version, required fields, and resource limits before reconstructing an object. Unknown tags, malformed data, unsupported versions, and digest mismatches are rejected.

Guarantees (E, P, B, K). The format is data-only and deterministic for a fixed object, codec version, and serializer. Digest verification detects accidental or adversarial payload changes but is not a cryptographic proof of mathematical truth. Decoding enforces the registered resource limits before large structures are accepted.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_general_persistence.py, tests/test_coordinates_normalized_persistence.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 61, 64, and 72.

C.2C · lucas_recurrence_model

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.lucas_recurrence_model`. Reflected implementation `komorebi_math.coordinates.families.lucas_recurrence_model`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-RECURRENCE-FAMILY`.

Manifest signature (s).

```
() -> 'RecurrenceModel'
```

Definition. Compute the Lucas recurrence model.

Contract (D, C, H, R, X). A model has a nonempty name, a positive-order exact coefficient tuple, and an optional initial state of the same order. Generation, analysis, synthesis, homogeneous bases, and model-to-model bridges enforce the requested horizon and coordinate length. Unknown catalog names raise `ValidationError`.

Guarantees (E, P, B, K). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_general_families_state.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 16, 20, and 21.

C.2C · modal_coordinate_system

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.modal_coordinate_system`. Reflected implementation `komorebi_math.coordinates.numeric.modal_coordinate_system`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-NUMERIC-TRANSFORM`.

Manifest signature (s).

```
(operator: 'Any', *, state_name: 'str' = 'state coordinates', modal_name: 'str' = 'modal coordinates', tolerance:
    ↪  'float' = 1e-10) -> 'ModalCoordinateSystem'
```

Definition. Construct numerical modal coordinates for a diagonalizable operator.

Contract (D, C, H, R, X). The transform matrix must be finite, two-dimensional, and square when inversion is requested. Application and composition check dimensions and labels. Named Fourier, cosine, Walsh, Haar, and modal constructors enforce their size, rank, and diagonalizability conditions.

Guarantees (E, P, B, K). These paths use finite NumPy linear algebra and report tolerance-qualified numerical evidence rather than exact equality. Dense matrices use quadratic storage and common decompositions require cubic floating-point work. Optional accelerator behavior is not implied by this interface.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_general_numeric_packet.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 15, 28, and 32.

C.2C · observability_block_matrix

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.observability_block_matrix`. Reflected implementation `komorebi_math.coordinates.state_space.observability_block_matrix`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-STATE-SPACE`.

Manifest signature (s).

```
(transition: 'Iterable[Iterable[Scalar]]', output_matrix: 'Iterable[Iterable[Scalar]]', *, steps: 'int | None' =
    ↪  None) -> 'Matrix'
```

Definition. Construct the finite observability block matrix whose block rows are `C`, `C A`, and the successive powers through the requested horizon.

Contract (D, C, H, R, X). The exact matrices `A`, `B`, `C`, and `D` must have compatible state, input, and output dimensions. Controllability and observability coordinate constructors require full state rank. A coordinate-change certificate verifies the transformed realization and finite transfer invariance.

Guarantees (E, P, B, K). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_general_families_state.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 16, 20, and 40.

C.2C · observability_coordinate_transform

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.observability_coordinate_transform`. Reflected implementation `komorebi_math.coordinates.state_space.observability_coordinate_transform`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-STATE-SPACE`.

Manifest signature (s).

```
(model: 'StateSpaceModel') -> 'CoordinateTransform'
```

Definition. Map state coordinates to a full output-jet coordinate system.

Contract (D, C, H, R, X). The exact matrices `A`, `B`, `C`, and `D` must have compatible state, input, and output dimensions. Controllability and observability coordinate constructors require full state rank. A coordinate-change certificate verifies the transformed realization and finite transfer invariance.

Guarantees (E, P, B, K). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_general_families_state.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 16, 20, and 40.

C.2C · observability_matrix

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.observability_matrix`. Reflected implementation `komorebi_math.coordinates.realization.observability_matrix`. The same runtime object is also available through `komorebi_math.observability_matrix`. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-REALIZATION`.

Manifest signature (s).

```
(transition: 'Matrix', output: 'Sequence[Scalar]', horizon: 'int', *, budget: 'CoordinateBudget' =
    ↪ CoordinateBudget(maximum_matrix_dimension=512, maximum_polynomial_degree=512, maximum_horizon=4096,
    ↪ maximum_exact_entries=1000000)) -> 'Matrix'
```

Definition. Compute rows `h^T A^n` for `0 <= n < horizon`.

Contract (D, C, H, R, X). Transition matrices are exact and square. Initial vectors and output covectors must match the state dimension. Recovery and similarity constructors require the stated observability or cyclic-rank condition. Intertwiner spaces solve the finite linear system `B S = S A` exactly.

Guarantees (E, P, B, K). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_realization.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 16, 20, and 32.

C.2C · ordinary_basis

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.ordinary_basis`. Reflected implementation `komorebi_math.generating.basis.ordinary_basis`. The same runtime object is also available through `komorebi_math.ordinary_basis`. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-CORE-NORMALIZED`.

Manifest signature (s).

```
(*, name: 'str' = 'ordinary') -> 'WeightedMonomialBasis'
```

Definition. Compute the ordinary generating-function basis `z^n`.

Contract (D, C, H, R, X). The basis weight at every used degree must be nonzero. Coefficient conversion multiplies or divides by those weights exactly, and basis constructors reject invalid degrees or vanishing normalization factors.

Guarantees (E, P, B, K). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_normalized_persistence.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 17, 18, and 64.

C.2C · padovan_recurrence_model

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.padovan_recurrence_model`. Reflected implementation `komorebi_math.coordinates.families.padovan_recurrence_model`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-RECURRENCE-FAMILY`.

Manifest signature (s).

```
() -> 'RecurrenceModel'
```

Definition. Compute a standard Padovan recurrence model.

Contract (D, C, H, R, X). A model has a nonempty name, a positive-order exact coefficient tuple, and an optional initial state of the same order. Generation, analysis, synthesis, homogeneous bases, and model-to-model bridges enforce the requested horizon and coordinate length. Unknown catalog names raise `ValidationError`.

Guarantees (E, P, B, K). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_general_families_state.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 16, 20, and 21.

C.2C · pell_recurrence_model

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.pell_recurrence_model`. Reflected implementation `komorebi_math.coordinates.families.pell_recurrence_model`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-RECURRENCE-FAMILY`.

Manifest signature (s).

```
() -> 'RecurrenceModel'
```

Definition. Compute the Pell recurrence model.

Contract (D, C, H, R, X). A model has a nonempty name, a positive-order exact coefficient tuple, and an optional initial state of the same order. Generation, analysis, synthesis, homogeneous bases, and model-to-model bridges enforce the requested horizon and coordinate length. Unknown catalog names raise `ValidationError`.

Guarantees (E, P, B, K). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_general_families_state.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 16, 20, and 21.

C.2C · permutation_transform

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.permutation_transform`. Reflected implementation `komorebi_math.coordinates.basis.permutation_transform`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-CORE-BASIS`.

Manifest signature (s).

```
(permutation: 'Sequence[int]', *, source: 'str' = 'original order', target: 'str' = 'permuted order') ->
    ↪ 'CoordinateTransform'
```

Definition. Compute an exact coordinate permutation.

Contract (D, C, H, R, X). Inputs are converted to exact rational vectors or matrices before construction. Dimension, independence, nonzero-scale, permutation, and budget conditions are checked before the returned basis or transform is accepted.

Guarantees (E, P, B, K). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_general_basis.py`, `tests/test_coordinates_exact.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 15, 16, and 64.

C.2C · polynomial_extended_gcd

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.polynomial_extended_gcd`. Reflected implementation `komorebi_math.coordinates.operators.polynomial_extended_gcd`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-RECURRENCE-OPERATOR`.

Manifest signature (s).

```
(left: 'Sequence[Scalar]', right: 'Sequence[Scalar]') -> 'tuple[Vector, Vector, Vector]'
```

Definition. Compute monic `g,u,v` with `u*left+v*right=g`.

Contract (D, C, H, R, X). Polynomial coefficients are exact and stored in ascending shift degree. Euclidean division rejects the zero polynomial. The Bezout reconstruction requires two defect polynomials with the documented coprimality and finite-prefix compatibility conditions.

Guarantees (E, P, B, K). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_operators.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 16, 21, and 32.

C.2C · realization_defect

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.realization_defect`. Reflected implementation `komorebi_math.coordinates.realization.realization_defect`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-REALIZATION`.

Manifest signature (s).

```
(realization: 'LinearRealization', target_coefficients: 'Iterable[Scalar]') -> 'LinearRealization'
```

Definition. Compute a realization of the target recurrence defect sequence.

Contract (D, C, H, R, X). Transition matrices are exact and square. Initial vectors and output covectors must match the state dimension. Recovery and similarity constructors require the stated observability or cyclic-rank condition. Intertwiner spaces solve the finite linear system `B S = S A` exactly.

Guarantees (E, P, B, K). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_realization.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 16, 20, and 32.

C.2C · recover_initial_state

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.recover_initial_state`. Reflected implementation `komorebi_math.coordinates.realization.recover_initial_state`. The same runtime object is also available through `komorebi_math.recover_initial_state`. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-REALIZATION`.

Manifest signature (s).

```
(prefix: 'Sequence[Scalar]', transition: 'Matrix', output: 'Sequence[Scalar]', *, budget: 'CoordinateBudget' =
    ↪ CoordinateBudget(maximum_matrix_dimension=512, maximum_polynomial_degree=512, maximum_horizon=4096,
    ↪ maximum_exact_entries=1000000)) -> 'StateRecovery'
```

Definition. Recover or test a state against a finite output prefix.

Contract (D, C, H, R, X). Transition matrices are exact and square. Initial vectors and output covectors must match the state dimension. Recovery and similarity constructors require the stated observability or cyclic-rank condition. Intertwiner spaces solve the finite linear system `B S = S A` exactly.

Guarantees (E, P, B, K). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_realization.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 16, 20, and 32.

C.2C · recurrence_analysis_transform

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.recurrence_analysis_transform`. Reflected implementation `komorebi_math.coordinates.recurrence.recurrence_analysis_transform`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-RECURRENCE-DEFECT`.

Manifest signature (s).

```
(coefficients: 'RecurrenceLike', length: 'int', *, name: 'str | None' = None, budget: 'CoordinateBudget' =
    ↪ CoordinateBudget(maximum_matrix_dimension=512, maximum_polynomial_degree=512, maximum_horizon=4096,
    ↪ maximum_exact_entries=1000000)) -> 'CoordinateTransform'
```

Definition. Map raw values to initial state followed by recurrence defects.

Contract (D, C, H, R, X). Recurrence coefficients are exact and must define positive order. A sequence prefix must contain at least the initial state when analysis or synthesis needs it. Transform dimensions are tied to the requested finite length, and all defect, Green-kernel, and reconstruction routines reject inconsistent orders or horizons.

Guarantees (E, P, B, K). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_recurrence.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 16, 20, and 21.

C.2C · recurrence_model_catalog

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.recurrence_model_catalog`. Reflected implementation `komorebi_math.coordinates.families.recurrence_model_catalog`. The same runtime object is also available through `komorebi_math.recurrence_model_catalog`. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-RECURRENCE-FAMILY`.

Manifest signature (s).

```
() -> 'tuple[RecurrenceModel, ...]'
```

Definition. Compute the built-in named recurrence models.

Contract (D, C, H, R, X). A model has a nonempty name, a positive-order exact coefficient tuple, and an optional initial state of the same order. Generation, analysis, synthesis, homogeneous bases, and model-to-model bridges enforce the requested horizon and coordinate length. Unknown catalog names raise `ValidationError`.

Guarantees (E, P, B, K). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_general_families_state.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 16, 20, and 21.

C.2C · recurrence_polynomial

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.recurrence_polynomial`. Reflected implementation `komorebi_math.coordinates.recurrence.recurrence_polynomial`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-RECURRENCE-DEFECT`.

Manifest signature (s).

```
(coefficients: 'RecurrenceLike') -> 'Vector'
```

Definition. Compute ascending coefficients of `x^d - sum_i c_i x^i`.

Contract (D, C, H, R, X). Recurrence coefficients are exact and must define positive order. A sequence prefix must contain at least the initial state when analysis or synthesis needs it. Transform dimensions are tied to the requested finite length, and all defect, Green-kernel, and reconstruction routines reject inconsistent orders or horizons.

Guarantees (E, P, B, K). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_recurrence.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 16, 20, and 21.

C.2C · recurrence_synthesis_transform

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.recurrence_synthesis_transform`. Reflected implementation `komorebi_math.coordinates.recurrence.recurrence_synthesis_transform`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-RECURRENCE-DEFECT`.

Manifest signature (s).

```
(coefficients: 'RecurrenceLike', length: 'int', *, name: 'str | None' = None, budget: 'CoordinateBudget' =
    ↪ CoordinateBudget(maximum_matrix_dimension=512, maximum_polynomial_degree=512, maximum_horizon=4096,
    ↪ maximum_exact_entries=1000000)) -> 'CoordinateTransform'
```

Definition. Compute the exact inverse of recurrence defect analysis.

Contract (D, C, H, R, X). Recurrence coefficients are exact and must define positive order. A sequence prefix must contain at least the initial state when analysis or synthesis needs it. Transform dimensions are tied to the requested finite length, and all defect, Green-kernel, and reconstruction routines reject inconsistent orders or horizons.

Guarantees (E, P, B, K). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_recurrence.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 16, 20, and 21.

C.2C · register_coordinate_codecs

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.register_coordinate_codecs`. Reflected implementation `komorebi_math.coordinates.portable.register_coordinate_codecs`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-PORTABLE`.

Manifest signature (s).

```
() -> 'None'
```

Definition. Register the closed data-only coordinate codecs.

Contract (D, C, H, R, X). Registration installs the closed set of coordinate codecs in the package registry. Repeated registration is idempotent and does not broaden the accepted payload grammar.

Guarantees (E, P, B, K). The format is data-only and deterministic for a fixed object, codec version, and serializer. Digest verification detects accidental or adversarial payload changes but is not a cryptographic proof of mathematical truth. Decoding enforces the registered resource limits before large structures are accepted.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_general_persistence.py, tests/test_coordinates_normalized_persistence.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 61, 64, and 72.

C.2C · save_coordinate_object

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.save_coordinate_object`. Reflected implementation `komorebi_math.coordinates.portable.save_coordinate_object`. The same runtime object is also available through `komorebi_math.save_coordinate_object`. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-PORTABLE`.

Manifest signature (s).

```
(path: 'str | Path', value: 'Any', *, overwrite: 'bool' = False) -> 'Path'
```

Definition. Save any registered portable coordinate object.

Contract (D, C, H, R, X). The object must belong to a registered coordinate codec. Saving writes a canonical data-only envelope with schema, version, payload, and digest fields. The destination path must be writable.

Guarantees (E, P, B, K). The format is data-only and deterministic for a fixed object, codec version, and serializer. Digest verification detects accidental or adversarial payload changes but is not a cryptographic proof of mathematical truth. Decoding enforces the registered resource limits before large structures are accepted.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_general_persistence.py, tests/test_coordinates_normalized_persistence.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 61, 64, and 72.

C.2C · save_coordinate_record

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.save_coordinate_record`. Reflected implementation `komorebi_math.coordinates.portable.save_coordinate_record`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-PORTABLE`.

Manifest signature (s).

```
(path: 'str | Path', record: 'RecurrenceCoordinateRecord', *, overwrite: 'bool' = False) -> 'Path'
```

Definition. Save recurrence defect coordinates with the package persistence envelope.

Contract (D, C, H, R, X). The object must belong to a registered coordinate codec. Saving writes a canonical data-only envelope with schema, version, payload, and digest fields. The destination path must be writable.

Guarantees (E, P, B, K). The format is data-only and deterministic for a fixed object, codec version, and serializer. Digest verification detects accidental or adversarial payload changes but is not a cryptographic proof of mathematical truth. Decoding enforces the registered resource limits before large structures are accepted.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_general_persistence.py, tests/test_coordinates_normalized_persistence.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 61, 64, and 72.

C.2C · save_transform

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.save_transform`. Reflected implementation `komorebi_math.coordinates.portable.save_transform`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-PORTABLE`.

Manifest signature (s).

```
(path: 'str | Path', transform: 'CoordinateTransform', *, overwrite: 'bool' = False) -> 'Path'
```

Definition. Save one exact coordinate transform with the package persistence envelope.

Contract (D, C, H, R, X). The object must belong to a registered coordinate codec. Saving writes a canonical data-only envelope with schema, version, payload, and digest fields. The destination path must be writable.

Guarantees (E, P, B, K). The format is data-only and deterministic for a fixed object, codec version, and serializer. Digest verification detects accidental or adversarial payload changes but is not a cryptographic proof of mathematical truth. Decoding enforces the registered resource limits before large structures are accepted.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_general_persistence.py`, `tests/test_coordinates_normalized_persistence.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 61, 64, and 72.

C.2C · `scaling_transform`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.scaling_transform`. Reflected implementation `komorebi_math.coordinates.basis.scaling_transform`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-CORE-BASIS`.

Manifest signature (s).

```
(scales: 'Sequence[Scalar]', *, source: 'str' = 'unscaled coordinates', target: 'str' = 'scaled coordinates') ->
    ↪ 'CoordinateTransform'
```

Definition. Compute an invertible diagonal coordinate scaling.

Contract (D, C, H, R, X). Inputs are converted to exact rational vectors or matrices before construction. Dimension, independence, nonzero-scale, permutation, and budget conditions are checked before the returned basis or transform is accepted.

Guarantees (E, P, B, K). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_general_basis.py`, `tests/test_coordinates_exact.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 15, 16, and 64.

C.2C · `sequence_coordinates_between_models`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.sequence_coordinates_between_models`. Reflected implementation `komorebi_math.coordinates.families.sequence_coordinates_between_models`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-RECURRENCE-FAMILY`.

Manifest signature (s).

```
(values: 'Sequence[Scalar]', source: 'RecurrenceModel', target: 'RecurrenceModel') ->
    ↪ 'tuple[RecurrenceCoordinateRecord, RecurrenceCoordinateRecord, CoordinateTransform]'
```

Definition. Analyze one sequence in two recurrence systems and return the exact bridge.

Contract (D, C, H, R, X). A model has a nonempty name, a positive-order exact coefficient tuple, and an optional initial state of the same order. Generation, analysis, synthesis, homogeneous bases, and model-to-model bridges enforce the requested horizon and coordinate length. Unknown catalog names raise `ValidationError`.

Guarantees (E, P, B, K). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_general_families_state.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 16, 20, and 21.

C.2C · signed_secant_euler_numbers

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.signed_secant_euler_numbers`. Reflected implementation `komorebi_math.coordinates.recurrence.signed_secant_euler_numbers`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-RECURRENCE-DEFECT`.

Manifest signature (s).

```
(count: 'int') -> 'tuple[int, ...]'
```

Definition. Compute `E_n` from `sech(z)=sum E_n z^n/n!`.

Contract (D, C, H, R, X). Recurrence coefficients are exact and must define positive order. A sequence prefix must contain at least the initial state when analysis or synthesis needs it. Transform dimensions are tied to the requested finite length, and all defect, Green-kernel, and reconstruction routines reject inconsistent orders or horizons.

Guarantees (E, P, B, K). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_recurrence.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 16, 20, and 21.

C.2C · similarity_from_cyclic_vectors

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.similarity_from_cyclic_vectors`. Reflected implementation `komorebi_math.coordinates.realization.similarity_from_cyclic_vectors`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-REALIZATION`.

Manifest signature (s).

```
(source: 'Matrix', target: 'Matrix', source_vector: 'Sequence[Scalar]', target_vector: 'Sequence[Scalar]') ->
   ↪ 'Matrix'
```

Definition. Construct `S` with `target*S=S*source` from cyclic vectors.

Contract (D, C, H, R, X). Transition matrices are exact and square. Initial vectors and output covectors must match the state dimension. Recovery and similarity constructors require the stated observability or cyclic-rank condition. Intertwiner spaces solve the finite linear system `B S = S A` exactly.

Guarantees (E, P, B, K). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_realization.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 16, 20, and 32.

C.2C · `siso_state_space`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.siso_state_space`. Reflected implementation `komorebi_math.coordinates.state_space.siso_state_space`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-STATE-SPACE`.

Manifest signature (s).

```
(transition: 'Iterable[Iterable[Scalar]]', input_vector: 'Sequence[Scalar]', output_covector: 'Sequence[Scalar]',
   ↪ *, feedthrough: 'Scalar' = 0, name: 'str' = 'SISO state-space model') -> 'StateSpaceModel'
```

Definition. Construct a single-input single-output exact state-space model.

Contract (D, C, H, R, X). The exact matrices `A`, `B`, `C`, and `D` must have compatible state, input, and output dimensions. Controllability and observability coordinate constructors require full state rank. A coordinate-change certificate verifies the transformed realization and finite transfer invariance.

Guarantees (E, P, B, K). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_general_families_state.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 16, 20, and 40.

C.2C · `standard_basis`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.standard_basis`. Reflected implementation `komorebi_math.coordinates.basis.standard_basis`. The same runtime object is also available through `komorebi_math.standard_basis`, and `komorebi_math.stable.standard_basis`. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-CORE-BASIS`.

Manifest signature (s).

```
(dimension: 'int', *, name: 'str' = 'standard') -> 'FiniteBasis'
```

Definition. Compute the standard basis of a rational vector space.

Contract (D, C, H, R, X). Inputs are converted to exact rational vectors or matrices before construction. Dimension, independence, nonzero-scale, permutation, and budget conditions are checked before the returned basis or transform is accepted.

Guarantees (E, P, B, K). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_general_basis.py`, `tests/test_coordinates_exact.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 15, 16, and 64.

C.2C · `synthesize_recurrence_coordinates`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.synthesize_recurrence_coordinates`. Reflected implementation `komorebi_math.coordinates.recurrence.synthesize_recurrence_coordinates`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-RECURRENCE-DEFECT`.

Manifest signature (s).

```
(record: 'RecurrenceCoordinateRecord') -> 'Vector'
```

Definition. Reconstruct a prefix from initial state and recurrence defects.

Contract (D, C, H, R, X). Recurrence coefficients are exact and must define positive order. A sequence prefix must contain at least the initial state when analysis or synthesis needs it. Transform dimensions are tied to the requested finite length, and all defect, Green-kernel, and reconstruction routines reject inconsistent orders or horizons.

Guarantees (E, P, B, K). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_recurrence.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 16, 20, and 21.

C.2C · `tensor_product_basis`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.coordinates.tensor_product_basis`. Reflected implementation `komorebi_math.coordinates.basis.tensor_product_basis`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-CORE-BASIS`.

Manifest signature (`S`).

```
(left: 'FiniteBasis', right: 'FiniteBasis', *, name: 'str | None' = None) -> 'FiniteBasis'
```

Definition. Compute the tensor-product basis in lexicographic factor order.

Contract (`D, C, H, R, X`). Inputs are converted to exact rational vectors or matrices before construction. Dimension, independence, nonzero-scale, permutation, and budget conditions are checked before the returned basis or transform is accepted.

Guarantees (`E, P, B, K`). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_general_basis.py`, `tests/test_coordinates_exact.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 15, 16, and 64.

C.2C · `translation_transform`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinates.translation_transform`. Reflected implementation `komorebi_math.coordinates.affine.translation_transform`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-CORE-AFFINE`.

Manifest signature (`S`).

```
(offset: 'Sequence[Scalar]', *, source: 'str' = 'original affine coordinates', target: 'str' = 'translated affine
   ↪ coordinates') -> 'AffineCoordinateTransform'
```

Definition. Compute an exact translation coordinate change.

Contract (`D, C, H, R, X`). All coordinates are converted to exact rational values. Affine matrices must be square with an offset of matching length. Barycentric vertices must form a nondegenerate simplex. Composition, inversion, and conversion reject incompatible dimensions or singular linear parts.

Guarantees (`E, P, B, K`). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_general_affine_formal.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 15, 16, and 64.

C.2C · `tribonacci_recurrence_model`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinates.tribonacci_recurrence_model`. Reflected implementation `komorebi_math.coordinates.families.tribonacci_recurrence_model`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-RECURRENCE-FAMILY`.

Manifest signature (`s`).

```
() -> 'RecurrenceModel'
```

Definition. Compute the Tribonacci recurrence model.

Contract (`D, C, H, R, X`). A model has a nonempty name, a positive-order exact coefficient tuple, and an optional initial state of the same order. Generation, analysis, synthesis, homogeneous bases, and model-to-model bridges enforce the requested horizon and coordinate length. Unknown catalog names raise `ValidationError`.

Guarantees (`E, P, B, K`). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_general_families_state.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 16, 20, and 21.

C.2C · `variation_of_constants_prefix`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinates.variation_of_constants_prefix`. Reflected implementation `komorebi_math.coordinates.recurrence.variation_of_constants_prefix`. The same runtime object is also available through `komorebi_math.variation_of_constants_prefix`. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-RECURRENCE-DEFECT`.

Manifest signature (`s`).

```
(record: 'RecurrenceCoordinateRecord') -> 'Vector'
```

Definition. Reconstruct a prefix through the companion Green-kernel formula.

Contract (`D, C, H, R, X`). Recurrence coefficients are exact and must define positive order. A sequence prefix must contain at least the initial state when analysis or synthesis needs it. Transform dimensions are tied to the requested finite length, and all defect, Green-kernel, and reconstruction routines reject inconsistent orders or horizons.

Guarantees (`E, P, B, K`). Arithmetic is exact over integers and rational `Fraction` values. Certificates cover the stated finite identity, rank, reconstruction, or invariance claim only. Dense elimination and inversion can require cubic arithmetic-operation cost and quadratic storage, while exact numerator and denominator sizes may grow. Declared coordinate budgets bound supported dimensions and horizons where the interface exposes them.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_recurrence.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 16, 20, and 21.

C.2C · `walsh_hadamard_transform`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.coordinates.walsh_hadamard_transform`. Reflected implementation `komorebi_math.coordinates.numeric.walsh_hadamard_transform`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.coordinates` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C2-NUMERIC-TRANSFORM`.

Manifest signature (s).

```
(size: 'int', *, orthonormal: 'bool' = True, tolerance: 'float' = 1e-12) -> 'NumericCoordinateTransform'
```

Definition. Compute the Sylvester Walsh-Hadamard transform for a power-of-two size.

Contract (D, C, H, R, X). The transform matrix must be finite, two-dimensional, and square when inversion is requested. Application and composition check dimensions and labels. Named Fourier, cosine, Walsh, Haar, and modal constructors enforce their size, rank, and diagonalizability conditions.

Guarantees (E, P, B, K). These paths use finite NumPy linear algebra and report tolerance-qualified numerical evidence rather than exact equality. Dense matrices use quadratic storage and common decompositions require cubic floating-point work. Optional accelerator behavior is not implied by this interface.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. `tests/test_coordinates_general_numeric_packet.py` and the independent verifier for coordinate systems cover the applicable construction, roundtrip, rank, failure, persistence, or resource behavior. The source validator for coordinate systems checks the exact signature, implementation identity, domain facade, aliases, regression coverage, and unique anchor. See Chapters 15, 28, and 32.

C.3 `komorebi_math.generating`: 181 entries

C.3A Foundational normalized generating functions: 80 entries

This generating-namespace section records the exact finite calculus that the later Mahler, Cartier, congruence, and prime power compilers use. It begins with resource limits and coefficient domains, then documents normalized bases, formal series, custom function builders, named sequences, bounded relation inference, residue sections, monoid indexed products, FIR conversion, and portable snapshots. These contracts remain in the research tier unless a card lists an existing stable facade.

Contract family	Cards
Resource budgets	2
Exact coefficient domains	9
Normalized bases	13
Formal series	2
Finite series over explicit domains	2
Custom generating functions	11
Named exact sequences	12
Bounded relation inference	12
Residue sections and root filters	5
Series over finite monoids	5
FIR adapters	4
Portable generating records	3
Total	**80**

The distinction between exact formal algebra and finite numerical adaptation is explicit. Domain, basis, series, sequence, and closed form records use exact integer or rational arithmetic unless a finite residue domain is selected. Root filters and FIR responses cross into floating point evaluation and therefore state residual or tolerance qualified guarantees. Relation guessers are bounded prefix inference tools. A returned relation is not presented as an infinite sequence theorem without separate proof.

C.3A · AlgebraicGFGuess

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.AlgebraicGFGuess`. Reflected implementation `komorebi_math.generating.closed_forms.AlgebraicGFGuess`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `class`. Profile `C3-FOUND-INFERENCE`.

Manifest signature (s).

```
(x_degree: 'int', y_degree: 'int', monomials: 'tuple[tuple[int, int, int], ...]', fit_coefficients: 'int',
   ↪ holdout_coefficients: 'int', exact_on_prefix: 'bool') -> None
```

Definition. Bounded algebraic relation candidate for an ordinary series.

Contract (D, C, H, R, X). All guessers consume a finite exact prefix and search only within the declared order and degree bounds. A holdout separates fitted relations from later checks when enough data are available. Failure to find a candidate returns `None`. Guess records preserve the fit count, holdout count, and exact prefix status. Symbolic expression methods and characteristic forms require the optional SymPy backend. The principal public operations are `expression`.

Guarantees (E, P, B, K). Linear systems and residual checks use exact rational or integer arithmetic. A returned candidate is evidence for the supplied finite prefix, not a proof that the relation holds for every index. Search cost grows rapidly with the product of recurrence order, polynomial degree, and algebraic bidegree, so the explicit bounds are part of the contract.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test is `tests/test_generating_foundation.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 22, 24, and 26.

C.3A · BasisCertificate

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.BasisCertificate`. Reflected implementation `komorebi_math.generating.basis.BasisCertificate`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `class`. Profile `C3-FOUND-BASIS`.

Manifest signature (s).

```
(name: 'str', degree: 'int', nonzero_normalization: 'bool', integral_structure_constants: 'bool',
   ↪ maximum_denominator: 'int', maximum_numerator_bits: 'int') -> None
```

Definition. Finite exact diagnostics for one normalized monomial basis.

Contract (D, C, H, R, X). A normalized monomial basis is defined by exact nonzero values `gamma_n` with `gamma_0=1`. Names must be nonempty. Indices and finite audit degrees must be nonnegative. Packet addresses must have a positive modulus and a valid residue. Declarative descriptors use a closed registry. Callback bases remain usable but are not portable unless replaced by a finite table descriptor.

Guarantees (E, P, B, K). Normalization, lowering weights, structure constants, packet transfer weights, and finite certificates use exact `Fraction` arithmetic. Finite audits establish only the requested degree range. Computing all structure constants through degree `d` requires a quadratic number of exact ratios, and numerator or denominator sizes can grow quickly.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused tests are `tests/test_generating_foundation.py` and `tests/test_generating_domains.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 21, 22, and 24.

C.3A · BasisDescriptor

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.BasisDescriptor`. Reflected implementation `komorebi_math.generating.basis.BasisDescriptor`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `class`. Profile `C3-FOUND-BASIS`.

Manifest signature (s).

```
(name: 'str', kind: 'str', parameters: 'tuple[tuple[str, BasisParameter], ...]' = ()) -> None
```

Definition. Declarative, portable description of one normalized monomial basis.

Contract (D, C, H, R, X). A normalized monomial basis is defined by exact nonzero values `gamma_n` with `gamma_0=1`. Names must be nonempty. Indices and finite audit degrees must be nonnegative. Packet addresses must have a positive modulus and a valid residue. Declarative descriptors use a closed registry. Callback bases remain usable but are not portable unless replaced by a finite table descriptor. The principal public operations are `create`, `parameter_map`.

Guarantees (E, P, B, K). Normalization, lowering weights, structure constants, packet transfer weights, and finite certificates use exact `Fraction` arithmetic. Finite audits establish only the requested degree range. Computing all structure constants through degree `d` requires a quadratic number of exact ratios, and numerator or denominator sizes can grow quickly.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused tests are `tests/test_generating_foundation.py` and `tests/test_generating_domains.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 21, 22, and 24.

C.3A · CoefficientDomain

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.CoefficientDomain`. Reflected implementation `komorebi_math.generating.domains.CoefficientDomain`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `class`. Profile `C3-FOUND-DOMAIN`.

Manifest signature (s).

```
(*args, **kwargs)
```

Definition. Protocol for exact coefficient arithmetic and safe serialization.

Contract (D, C, H, R, X). A coefficient domain supplies coercion, exact arithmetic, unit tests, inversion, equality, portable encoding, decoding, and a descriptor containing only data. Integer inversion is restricted to `1` and `-1`. Residue ring inversion requires a unit. Prime fields require a prime modulus. Truncated p-adic domains require a prime and positive precision. Descriptor reconstruction accepts only the closed supported schemas. The principal public operations are `coerce`, `add`, `subtract`, `multiply`, `inverse`, `is_unit`, `equal`, `encode`, `decode`, `descriptor`.

Guarantees (E, P, B, K). All arithmetic is exact in the declared finite or rational domain. Encodings are canonical for the represented element. Prime validation is deterministic for the supplied modulus. Operations are constant time in the number of coefficients, although integer bit complexity grows with operand size.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test is `tests/test_generating_domains.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 21, 23, and 61.

C.3A · CustomFunctionSpec

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.CustomFunctionSpec`. Reflected implementation `komorebi_math.generating.functions.CustomFunctionSpec`. The same runtime object is also exported as `komorebi_math.CustomFunctionSpec`. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `class`. Profile `C3-FOUND-FUNCTION`.

Manifest signature (s).

```
(name: 'str', basis: 'WeightedMonomialBasis', builder: 'CoefficientBuilder', source_kind: 'str' = 'custom',
    ↪ metadata: 'Mapping[str, object]' = <factory>) -> None
```

Definition. A named exact coefficient source with an explicit basis and metadata.

Contract (D, C, H, R, X). A custom function has a nonempty name, an explicit normalized basis, a finite coefficient builder, a source kind, and portable metadata where required. Builders must return exactly `degree+1` coefficients. Recurrence seeds must match the recurrence order. Rational denominators need a nonzero constant. Algebraic branches require `P(0,y0)=0` and a nonzero partial derivative with respect to `y`. Registries reject duplicate names. The principal public operations are `series`.

Guarantees (E, P, B, K). Each call constructs an exact finite `FormalSeries`. Rational, recurrence, hypergeometric, algebraic, Euler product, and Bell builders certify only the generated finite prefix. Runtime is polynomial in the requested degree for the implemented recurrences, but exact coefficient growth can dominate memory and arithmetic cost.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test is `tests/test_generating_foundation.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 21 through 24.

C.3A · DEFAULT_GENERATING_BUDGET

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.DEFAULT_GENERATING_BUDGET`. Reflected implementation `komorebi_math.generating.budget.GeneratingBudget`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `data`. Profile `C3-FOUND-BUDGET`.

Manifest signature (s).

```
null
```

Definition. The immutable default resource budget used by generating operations that receive no explicit budget.

Contract (D, C, H, R, X). Every stored limit must be a positive integer. Degree, term, matrix, state, modulus, serialized size, and coefficient bit checks reject malformed requests before the corresponding expansion or allocation begins. The default object is immutable. Exceeding a declared limit raises `ResourceLimitError`, while malformed values raise `ValidationError`.

Guarantees (E, P, B, K). The budget performs deterministic scalar validation and does not estimate hidden backend memory. Its checks are constant time. A caller who supplies a larger budget accepts the resulting exact arithmetic, state enumeration, and serialization costs.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused tests are `tests/test_generating_domains.py` and `tests/test_generating_block13.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 21, 23, and 64.

C.3A · FIRSeriesAdapter

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.FIRSeriesAdapter`. Reflected implementation `komorebi_math.generating.adapters.FIRSeriesAdapter`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `class`. Profile `C3-FOUND-ADAPTER`.

Manifest signature (s).

```
(series: 'FormalSeries', taps: 'np.ndarray', variable_substitution: 'str' = 'formal z maps to delay z^(-1)') ->
    ↪ None
```

Definition. A finite transfer polynomial interpreted under `z -> z**(-1)`.

Contract (D, C, H, R, X). FIR inputs must be a nonempty one dimensional finite real array. Conversion to `FormalSeries` rationalizes taps, optionally limiting denominators. Conversion from a series validates the requested degree and numeric dtype. An adapter requires one tap per stored ordinary coefficient. Frequency responses validate finite sampling parameters and certificates compare the declared coefficient to tap conversion. The principal public operations are `certificate`, `response`.

Guarantees (E, P, B, K). The formal side is exact before conversion to NumPy. Tap arrays and frequency responses use floating point arithmetic, and their certificates are qualified by a stated tolerance. Conversion is linear in the number of taps. Frequency response evaluation delegates to the signal implementation and scales with the chosen point count.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test is `tests/test_generating_block13.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 21, 31, and 66.

C.3A · FiniteMonoidSeries

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.FiniteMonoidSeries`. Reflected implementation `komorebi_math.generating.monoid.FiniteMonoidSeries`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `class`. Profile `C3-FOUND-MONOID`.

Manifest signature (s).

```
(coefficients: 'Mapping[Index, Fraction]', combine: 'Callable[[Index, Index], Index]', identity: 'Index', name:
    ↪ 'str' = 'monoid series') -> None
```

Definition. A finite coefficient map with a declared associative index product.

Contract (D, C, H, R, X). A finite monoid series stores a finite coefficient map, an index combination rule, an identity, and a name. Multiplication requires the same identity and combines only represented pairs, optionally filtered by an admissibility predicate. Additive series use nonnegative indices. Multiplicative series use positive integer indices. Dirichlet convolution validates a nonnegative count, and valuation packets require a prime of at least two. The principal public operations are `coefficient`, `multiply`.

Guarantees (E, P, B, K). Coefficient arithmetic is exact over `Fraction`. The class assumes that the index operation supplied by the caller is associative and does not prove that law. Dense finite multiplication is quadratic in the number of stored indices. Dirichlet convolution uses direct divisor enumeration for the requested prefix.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test is `tests/test_generating_foundation.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 21, 24, and 25.

C.3A · FiniteNormalizedSeries

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.FiniteNormalizedSeries`. Reflected implementation `komorebi_math.generating.finite_series.FiniteNormalizedSeries`. The same runtime object is also exported as `komorebi_math.FiniteNormalizedSeries`. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `class`. Profile `C3-FOUND-DOMAIN-SERIES`.

Manifest signature (s).

```
(basis: 'WeightedMonomialBasis', domain: 'CoefficientDomain[Any]', coefficients: 'Iterable[Any]', *, budget:
    ↪ 'GeneratingBudget' = GeneratingBudget(maximum_degree=100000, maximum_coefficient_bits=4000000,
    ↪ maximum_matrix_dimension=4096, maximum_reachable_states=100000, maximum_local_combinations=1000000,
    ↪ maximum_modulus=2000000, maximum_serialized_bytes=67108864, maximum_recursion_depth=1000)) -> 'None'
```

Definition. A finite exact series `sum a_n z^n/gamma_n` over one domain.

Contract (D, C, H, R, X). The basis, coefficient domain, and budget must satisfy their public protocols. Coefficients are coerced into the declared domain. Binary operations require matching domains and compatible bases. Reciprocal requires a unit constant coefficient. Composition, exponential, and logarithm impose the same constant term conditions as ordinary formal calculus. Conversion returns an explicit finite transfer certificate. The principal public operations are `add`, `subtract`, `multiply`, `reciprocal`, `compose`, `exp`, `log`, `project`, `convert`, `certificate`.

Guarantees (E, P, B, K). Results are exact in the selected coefficient domain and bounded by the supplied `Generating Budget`. A valid transfer certificate checks reconstruction coefficient by coefficient through the stored degree. Complexity follows finite polynomial arithmetic, with quadratic convolution and potentially cubic composition at the truncation boundary.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test is `tests/test_generating_domains.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 21, 22, and 61.

C.3A · FormalSeries

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.FormalSeries`. Reflected implementation `komorebi_math.generating.series.FormalSeries`. The same runtime object is also exported as `komorebi_math.FormalSeries`, `komorebi_math.stable.FormalSeries`, `komorebi_math.v1.FormalSeries`, `komorebi_math.api.v1.FormalSeries`. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. The stable and versioned aliases keep their separate frozen compatibility commitment. Manifest kind `class`. Profile `C3-FOUND-SERIES`.

Manifest signature (s).

```
(basis: 'WeightedMonomialBasis', coefficients: 'Iterable[int | Fraction]') -> 'None'
```

Definition. A finite exact series `sum a_n z^n/gamma_n`.

Contract (D, C, H, R, X). The coefficient tuple is finite and nonempty. Binary operations require compatible basis semantics or an explicit conversion. Reciprocal requires a nonzero constant term. Composition requires an inner zero constant term. Formal exponential requires zero constant term, while formal logarithm requires constant term one. Projectors and packets validate modulus and residue addresses. The principal public operations are `add`, `subtract`, `multiply`, `reciprocal`, `compose`, `exp`, `log`, `project`, `packet`, `convert`, `evaluate`.

Guarantees (E, P, B, K). Every algebraic result is exact over rational coefficients at the requested finite truncation. Multiplication, composition, reciprocal, exponential, and logarithm are finite formal operations and do not assert analytic convergence. Naive multiplication is quadratic in truncation degree, while composition and repeated powers can require cubic work.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused tests are `tests/test_generating_foundation.py` and `tests/test_generating_domains.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foun-

dations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 21, 22, and 24.

C.3A · FrozenGeneratingFunction

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.FrozenGeneratingFunction`. Reflected implementation `komorebi_math.generating.portable.FrozenGeneratingFunction`. The same runtime object is also exported as `komorebi_math.FrozenGeneratingFunction`. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `class`. Profile `C3-FOUND-PORTABLE`.

Manifest signature (s).

```
(name: 'str', basis_name: 'str', normalization: 'tuple[Fraction, ...]', coefficients: 'tuple[Fraction, ...]',
   ↪ source_kind: 'str', metadata: 'Mapping[str, Any]' = <factory>) -> None
```

Definition. Provider-independent finite snapshot of a custom generating function.

Contract (D, C, H, R, X). A frozen function requires nonempty names, a nonempty coefficient prefix, normalization of equal length beginning with one, no zero normalization values, and portable metadata. Reconstruction cannot exceed the captured degree. Freezing accepts only `CustomFunctionSpec`. Codec registration installs closed data only decoders and does not permit arbitrary Python object construction. The principal public operations are `degree`, `series`.

Guarantees (E, P, B, K). The frozen record preserves exact rational coefficients and basis normalization independently of the original callback. Roundtrip integrity is delegated to the project portable envelope and its digest checks. Serialization cost is linear in the captured prefix, subject to the global payload budget used by the persistence layer.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test is `tests/test_generating_block13.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 21, 61, and 72.

C.3A · FunctionRegistry

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.FunctionRegistry`. Reflected implementation `komorebi_math.generating.functions.FunctionRegistry`. The same runtime object is also exported as `komorebi_math.FunctionRegistry`. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `class`. Profile `C3-FOUND-FUNCTION`.

Manifest signature (s).

```
() -> 'None'
```

Definition. An explicit custom-function registry. Names cannot silently overwrite existing entries.

Contract (D, C, H, R, X). A custom function has a nonempty name, an explicit normalized basis, a finite coefficient builder, a source kind, and portable metadata where required. Builders must return exactly `degree+1` coefficients. Recurrence seeds must match the recurrence order. Rational denominators need a nonzero constant. Algebraic branches require `P(0,y0)=0` and a nonzero partial derivative with respect to `y`. Registries reject duplicate names. The principal public operations are `register`, `get`, `names`.

Guarantees (E, P, B, K). Each call constructs an exact finite `FormalSeries`. Rational, recurrence, hypergeometric, algebraic, Euler product, and Bell builders certify only the generated finite prefix. Runtime is polynomial in the requested degree for the implemented recurrences, but exact coefficient growth can dominate memory and arithmetic cost.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test is `tests/test_generating_foundation.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 21 through 24.

C.3A · GeneratingBudget

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.GeneratingBudget`. Reflected implementation `komorebi_math.generating.budget.GeneratingBudget`. The same runtime object is also exported as `komorebi_math.GeneratingBudget`. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `class`. Profile `C3-FOUND-BUDGET`.

Manifest signature (s).

```
(maximum_degree: 'int' = 100000, maximum_coefficient_bits: 'int' = 4000000, maximum_matrix_dimension: 'int' = 4096,
    ↪ maximum_reachable_states: 'int' = 100000, maximum_local_combinations: 'int' = 1000000, maximum_modulus:
    ↪ 'int' = 2000000, maximum_serialized_bytes: 'int' = 67108864, maximum_recursion_depth: 'int' = 1000) -> None
```

Definition. Upper bounds checked before potentially expensive operations.

Contract (D, C, H, R, X). Every stored limit must be a positive integer. Degree, term, matrix, state, modulus, serialized size, and coefficient bit checks reject malformed requests before the corresponding expansion or allocation begins. The default object is immutable. Exceeding a declared limit raises `ResourceLimitError`, while malformed values raise `ValidationError`. The principal public operations are `require_degree`, `require_terms`, `require_matrix_dimension`, `require_states`, `require_local_combinations`, `require_modulus`, `require_serialized_bytes`, `require_coefficient_bits`.

Guarantees (E, P, B, K). The budget performs deterministic scalar validation and does not estimate hidden backend memory. Its checks are constant time. A caller who supplies a larger budget accepts the resulting exact arithmetic, state enumeration, and serialization costs.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused tests are `tests/test_generating_domains.py` and `tests/test_generating_block13.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 21, 23, and 64.

C.3A · HypergeometricGuess

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.HypergeometricGuess`. Reflected implementation `komorebi_math.generating.closed_forms.HypergeometricGuess`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `class`. Profile `C3-FOUND-INFERENCE`.

Manifest signature (s).

```
(numerator: 'tuple[int, ...]', denominator: 'tuple[int, ...]', fit_relations: 'int', holdout_relations: 'int',
    ↪ exact_on_prefix: 'bool') -> None
```

Definition. Bounded rational candidate for a consecutive-term ratio.

Contract (D, C, H, R, X). All guessers consume a finite exact prefix and search only within the declared order and degree bounds. A holdout separates fitted relations from later checks when enough data are available. Failure to find a candidate returns `None`. Guess records preserve the fit count, holdout count, and exact prefix status. Symbolic expression methods and characteristic forms require the optional SymPy backend. The principal public operations are `ratio_expression`.

Guarantees (E, P, B, K). Linear systems and residual checks use exact rational or integer arithmetic. A returned candidate is evidence for the supplied finite prefix, not a proof that the relation holds for every index. Search cost grows rapidly with the product of recurrence order, polynomial degree, and algebraic bidegree, so the explicit bounds are part of the contract.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test is `tests/test_generating_foundation.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 22, 24, and 26.

C.3A · INTEGER_DOMAIN

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.INTEGER_DOMAIN`. Reflected implementation `komorebi_math.generating.domains.IntegerDomain`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `data`. Profile `C3-FOUND-DOMAIN`.

Manifest signature (s).

```
null
```

Definition. The shared exact integer coefficient domain instance.

Contract (D, C, H, R, X). A coefficient domain supplies coercion, exact arithmetic, unit tests, inversion, equality, portable encoding, decoding, and a descriptor containing only data. Integer inversion is restricted to `1` and `-1`. Residue ring inversion requires a unit. Prime fields require a prime modulus. Truncated p-adic domains require a prime and positive precision. Descriptor reconstruction accepts only the closed supported schemas.

Guarantees (E, P, B, K). All arithmetic is exact in the declared finite or rational domain. Encodings are canonical for the represented element. Prime validation is deterministic for the supplied modulus. Operations are constant time in the number of coefficients, although integer bit complexity grows with operand size.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test is `tests/test_generating_domains.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 21, 23, and 61.

C.3A · IntegerDomain

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.IntegerDomain`. Reflected implementation `komorebi_math.generating.domains.IntegerDomain`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `class`. Profile `C3-FOUND-DOMAIN`.

Manifest signature (s).

```
(name: 'str' = 'integers', zero: 'int' = 0, one: 'int' = 1) -> None
```

Definition. The exact integer coefficient domain.

Contract (D, C, H, R, X). A coefficient domain supplies coercion, exact arithmetic, unit tests, inversion, equality, portable encoding, decoding, and a descriptor containing only data. Integer inversion is restricted to `1` and `-1`. Residue ring inversion requires a unit. Prime fields require a prime modulus. Truncated p-adic domains require a prime and positive precision. Descriptor reconstruction accepts only the closed supported schemas. The principal public operations are `coerce`, `add`, `subtract`, `multiply`, `inverse`, `is_unit`, `encode`, `decode`, `descriptor`.

Guarantees (E, P, B, K). All arithmetic is exact in the declared finite or rational domain. Encodings are canonical for the represented element. Prime validation is deterministic for the supplied modulus. Operations are constant time in the number of coefficients, although integer bit complexity grows with operand size.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test is `tests/test_generating_domains.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 21, 23, and 61.

C.3A · LinearRecurrenceGuess

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.LinearRecurrenceGuess`. Reflected implementation `komorebi_math.generating.closed_forms.LinearRecurrenceGuess`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `class`. Profile `C3-FOUND-INFERENCE`.

Manifest signature (s).

```
(order: 'int', coefficients: 'tuple[Fraction, ...]', fit_terms: 'int', holdout_terms: 'int', exact_on_prefix:
   ↪  'bool') -> None
```

Definition. Bounded exact candidate for a constant-coefficient recurrence.

Contract (D, C, H, R, X). All guessers consume a finite exact prefix and search only within the declared order and degree bounds. A holdout separates fitted relations from later checks when enough data are available. Failure to find a candidate returns `None`. Guess records preserve the fit count, holdout count, and exact prefix status. Symbolic expression methods and characteristic forms require the optional SymPy backend. The principal public operations are `equation`.

Guarantees (E, P, B, K). Linear systems and residual checks use exact rational or integer arithmetic. A returned candidate is evidence for the supplied finite prefix, not a proof that the relation holds for every index. Search cost grows rapidly with the product of recurrence order, polynomial degree, and algebraic bidegree, so the explicit bounds are part of the contract.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test is `tests/test_generating_foundation.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 22, 24, and 26.

C.3A · NormalizedBasis

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.NormalizedBasis`. Reflected implementation `komorebi_math.generating.basis.WeightedMonomialBasis`. The same runtime object is also exported as `komorebi_math.NormalizedBasis`. Within that namespace, `komorebi_math.generating.NormalizedMonomialBasis` and `komorebi_math.generating.WeightedMonomialBasis` resolve to the same runtime object. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `class`. Profile `C3-FOUND-BASIS`.

Manifest signature (s).

```
(name: 'str', gamma: 'GammaRule', *, descriptor: 'BasisDescriptor | None' = None) -> 'None'
```

Definition. A basis `e_n(z)=z^n/gamma_n` with `gamma_0=1`.

Contract (D, C, H, R, X). A normalized monomial basis is defined by exact nonzero values `gamma_n` with `gamma_0=1`. Names must be nonempty. Indices and finite audit degrees must be nonnegative. Packet addresses must have a positive modulus and a valid residue. Declarative descriptors use a closed registry. Callback bases remain usable but are not portable unless replaced by a finite table descriptor. The principal public operations are `gamma`, `lowering_weight`, `structure_constant`, `packet_transfer_weight`, `finite_certificate`, `require_descriptor`.

Guarantees (E, P, B, K). Normalization, lowering weights, structure constants, packet transfer weights, and finite certificates use exact `Fraction` arithmetic. Finite audits establish only the requested degree range. Computing all structure constants through degree `d` requires a quadratic number of exact ratios, and numerator or denominator sizes can grow quickly.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused tests are `tests/test_generating_foundation.py` and `tests/test_generating_domains.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 21, 22, and 24.

C.3A · NormalizedMonomialBasis

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.NormalizedMonomialBasis`. Reflected implementation `komorebi_math.generating.basis.WeightedMonomialBasis`. In the reference manifest, this symbol is exported only from the generating namespace. Within that namespace, `komorebi_math.generating.NormalizedBasis` and `komorebi_math.generating.WeightedMonomialBasis` resolve to the same runtime object. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `class`. Profile `C3-FOUND-BASIS`.

Manifest signature (s).

```
(name: 'str', gamma: 'GammaRule', *, descriptor: 'BasisDescriptor | None' = None) -> 'None'
```

Definition. A basis `e_n(z)=z^n/gamma_n` with `gamma_0=1`.

Contract (D, C, H, R, X). A normalized monomial basis is defined by exact nonzero values `gamma_n` with `gamma_0=1`. Names must be nonempty. Indices and finite audit degrees must be nonnegative. Packet addresses must have a positive modulus and a valid residue. Declarative descriptors use a closed registry. Callback bases remain usable but are not portable unless replaced by a finite table descriptor. The principal public operations are `gamma`, `lowering_weight`, `structure_constant`, `packet_transfer_weight`, `finite_certificate`, `require_descriptor`.

Guarantees (E, P, B, K). Normalization, lowering weights, structure constants, packet transfer weights, and finite certificates use exact `Fraction` arithmetic. Finite audits establish only the requested degree range. Computing all structure constants through degree `d` requires a quadratic number of exact ratios, and numerator or denominator sizes can grow quickly.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused tests are `tests/test_generating_foundation.py` and `tests/test_generating_domains.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 21, 22, and 24.

C.3A · PRecursiveGuess

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.PRecursiveGuess`. Reflected implementation `komorebi_math.generating.closed_forms.PRecursiveGuess`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `class`. Profile `C3-FOUND-INFERENCE`.

Manifest signature (s).

```
(order: 'int', degree: 'int', polynomials: 'tuple[tuple[int, ...], ...]', fit_relations: 'int', holdout_relations:
    ↪ 'int', exact_on_prefix: 'bool') -> None
```

Definition. Bounded polynomial-coefficient recurrence candidate.

Contract (D, C, H, R, X). All guessers consume a finite exact prefix and search only within the declared order and degree bounds. A holdout separates fitted relations from later checks when enough data are available. Failure to find a candidate returns `None`. Guess records preserve the fit count, holdout count, and exact prefix status. Symbolic expression methods and characteristic forms require the optional SymPy backend. The principal public operations are `expression`.

Guarantees (E, P, B, K). Linear systems and residual checks use exact rational or integer arithmetic. A returned candidate is evidence for the supplied finite prefix, not a proof that the relation holds for every index. Search cost grows rapidly with the product of recurrence order, polynomial degree, and algebraic bidegree, so the explicit bounds are part of the contract.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test is `tests/test_generating_foundation.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 22, 24, and 26.

C.3A · PacketIdentity

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.PacketIdentity`. Reflected implementation `komorebi_math.generating.series.PacketIdentity`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `class`. Profile `C3-FOUND-SERIES`.

Manifest signature (s).

```
(modulus: 'int', residue: 'int', source_basis: 'str', target_basis: 'str', source_degree: 'int', packet_degree:
   ↪  'int', valid: 'bool') -> None
```

Definition. Exact finite identity for one residue packet transfer.

Contract (D, C, H, R, X). The coefficient tuple is finite and nonempty. Binary operations require compatible basis semantics or an explicit conversion. Reciprocal requires a nonzero constant term. Composition requires an inner zero constant term. Formal exponential requires zero constant term, while formal logarithm requires constant term one. Projectors and packets validate modulus and residue addresses.

Guarantees (E, P, B, K). Every algebraic result is exact over rational coefficients at the requested finite truncation. Multiplication, composition, reciprocal, exponential, and logarithm are finite formal operations and do not assert analytic convergence. Naive multiplication is quadratic in truncation degree, while composition and repeated powers can require cubic work.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused tests are `tests/test_generating_foundation.py` and `tests/test_generating_domains.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 21, 22, and 24.

C.3A · PrimeFieldDomain

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.PrimeFieldDomain`. Reflected implementation `komorebi_math.generating.domains.PrimeFieldDomain`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `class`. Profile `C3-FOUND-DOMAIN`.

Manifest signature (s).

```
(modulus: 'int', name: 'str' = 'prime_field') -> None
```

Definition. The prime field `F_p` represented by canonical integer residues.

Contract (D, C, H, R, X). A coefficient domain supplies coercion, exact arithmetic, unit tests, inversion, equality, portable encoding, decoding, and a descriptor containing only data. Integer inversion is restricted to `1` and `-1`. Residue ring inversion requires a unit. Prime fields require a prime modulus. Truncated p-adic domains require a prime and positive precision. Descriptor reconstruction accepts only the closed supported schemas. The principal public operations are `coerce`, `add`, `subtract`, `multiply`, `inverse`, `is_unit`, `encode`, `decode`, `descriptor`.

Guarantees (E, P, B, K). All arithmetic is exact in the declared finite or rational domain. Encodings are canonical for the represented element. Prime validation is deterministic for the supplied modulus. Operations are constant time in the number of coefficients, although integer bit complexity grows with operand size.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test is `tests/test_generating_domains.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 21, 23, and 61.

C.3A · QuasiPolynomialGuess

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.QuasiPolynomialGuess`. Reflected implementation `komorebi_math.generating.closed_forms.QuasiPolynomialGuess`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `class`. Profile `C3-FOUND-INFERENCE`.

Manifest signature (s).

```
(period: 'int', degree: 'int', residue_values: 'tuple[tuple[Fraction, ...], ...]', fit_terms: 'int',
   ↪ holdout_terms: 'int', exact_on_prefix: 'bool') -> None
```

Definition. Bounded residue-class polynomial candidate.

Contract (D, C, H, R, X). All guessers consume a finite exact prefix and search only within the declared order and degree bounds. A holdout separates fitted relations from later checks when enough data are available. Failure to find a candidate returns `None`. Guess records preserve the fit count, holdout count, and exact prefix status. Symbolic expression methods and characteristic forms require the optional SymPy backend.

Guarantees (E, P, B, K). Linear systems and residual checks use exact rational or integer arithmetic. A returned candidate is evidence for the supplied finite prefix, not a proof that the relation holds for every index. Search cost grows rapidly with the product of recurrence order, polynomial degree, and algebraic bidegree, so the explicit bounds are part of the contract.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test is `tests/test_generating_foundation.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 22, 24, and 26.

C.3A · RATIONAL_DOMAIN

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.RATIONAL_DOMAIN`. Reflected implementation `komorebi_math.generating.domains.RationalDomain`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `data`. Profile `C3-FOUND-DOMAIN`.

Manifest signature (s).

```
null
```

Definition. The shared exact rational coefficient domain instance.

Contract (D, C, H, R, X). A coefficient domain supplies coercion, exact arithmetic, unit tests, inversion, equality, portable encoding, decoding, and a descriptor containing only data. Integer inversion is restricted to `1` and `-1`. Residue ring inversion requires a unit. Prime fields require a prime modulus. Truncated p-adic domains require a prime and positive precision. Descriptor reconstruction accepts only the closed supported schemas.

Guarantees (E, P, B, K). All arithmetic is exact in the declared finite or rational domain. Encodings are canonical for the represented element. Prime validation is deterministic for the supplied modulus. Operations are constant time in the number of coefficients, although integer bit complexity grows with operand size.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test is `tests/test_generating_domains.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 21, 23, and 61.

C.3A · RationalDomain

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.RationalDomain`. Reflected implementation `komorebi_math.generating.domains.RationalDomain`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `class`. Profile `C3-FOUND-DOMAIN`.

Manifest signature (s).

```
(name: 'str' = 'rationals', zero: 'Fraction' = Fraction(0, 1), one: 'Fraction' = Fraction(1, 1)) -> None
```

Definition. The exact rational coefficient field.

Contract (D, C, H, R, X). A coefficient domain supplies coercion, exact arithmetic, unit tests, inversion, equality, portable encoding, decoding, and a descriptor containing only data. Integer inversion is restricted to `1` and `-1`. Residue ring inversion requires a unit. Prime fields require a prime modulus. Truncated p-adic domains require a prime and positive precision. Descriptor reconstruction accepts only the closed supported schemas. The principal public operations are `coerce`, `add`, `subtract`, `multiply`, `inverse`, `is_unit`, `encode`, `decode`, `descriptor`.

Guarantees (E, P, B, K). All arithmetic is exact in the declared finite or rational domain. Encodings are canonical for the represented element. Prime validation is deterministic for the supplied modulus. Operations are constant time in the number of coefficients, although integer bit complexity grows with operand size.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test is `tests/test_generating_domains.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 21, 23, and 61.

C.3A · ResidueRingDomain

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.ResidueRingDomain`. Reflected implementation `komorebi_math.generating.domains.ResidueRingDomain`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `class`. Profile `C3-FOUND-DOMAIN`.

Manifest signature (s).

```
(modulus: 'int', name: 'str' = 'residue_ring') -> None
```

Definition. Exact arithmetic in the finite ring `Z / modulus Z`.

Contract (D, C, H, R, X). A coefficient domain supplies coercion, exact arithmetic, unit tests, inversion, equality, portable encoding, decoding, and a descriptor containing only data. Integer inversion is restricted to `1` and `-1`. Residue ring inversion requires a unit. Prime fields require a prime modulus. Truncated p-adic domains require a prime and positive precision. Descriptor reconstruction accepts only the closed supported schemas. The principal public operations are `coerce`, `add`, `subtract`, `multiply`, `inverse`, `is_unit`, `encode`, `decode`, `descriptor`.

Guarantees (E, P, B, K). All arithmetic is exact in the declared finite or rational domain. Encodings are canonical for the represented element. Prime validation is deterministic for the supplied modulus. Operations are constant time in the number of coefficients, although integer bit complexity grows with operand size.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test is `tests/test_generating_domains.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 21, 23, and 61.

C.3A · SectionCycleAudit

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.SectionCycleAudit`. Reflected implementation `komorebi_math.generating.sections.SectionCycleAudit`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `class`. Profile `C3-FOUND-SECTION`.

Manifest signature (s).

```
(basis: 'str', modulus: 'int', degree: 'int', signed: 'bool', one_step_failures: 'int', full_cycle_failures:
   ↪  'int', root_filter_residual: 'float') -> None
```

Definition. Finite lowering-cycle and root-filter audit summary.

Contract (D, C, H, R, X). The truncation degree must be nonnegative. A section requires a positive modulus and a residue in range. The signed option changes the lowering cycle convention but not the address rules. The audit compares one step, a full cycle, and a finite complex root filter at the supplied evaluation point.

Guarantees (E, P, B, K). Series coefficients and lowering operations are exact. The root filter is a finite floating point evaluation and is reported through a residual rather than exact equality. Construction is linear in the truncation degree, while an audit over all residue classes scales with both degree and modulus.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test is `tests/test_generating_foundation.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 22, 24, and 25.

C.3A · SeriesTransferCertificate

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.SeriesTransferCertificate`. Reflected implementation `komorebi_math.generating.finite_series.SeriesTransferCertificate`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `class`. Profile `C3-FOUND-DOMAIN-SERIES`.

Manifest signature (s).

```
(valid: 'bool', degree: 'int', source_basis: 'str', target_basis: 'str', domain: 'str', failures: 'tuple[int,
   ↪  ...]' = ()) -> None
```

Definition. Exact finite certificate for a normalized basis transfer.

Contract (D, C, H, R, X). The basis, coefficient domain, and budget must satisfy their public protocols. Coefficients are coerced into the declared domain. Binary operations require matching domains and compatible bases. Reciprocal requires a unit constant coefficient. Composition, exponential, and logarithm impose the same constant term conditions as ordinary formal calculus. Conversion returns an explicit finite transfer certificate. The principal public operations are `certificate`, `require`.

Guarantees (E, P, B, K). Results are exact in the selected coefficient domain and bounded by the supplied `Generating Budget`. A valid transfer certificate checks reconstruction coefficient by coefficient through the stored degree. Complexity follows finite polynomial arithmetic, with quadratic convolution and potentially cubic composition at the truncation boundary.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test is `tests/test_generating_domains.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 21, 22, and 61.

C.3A · TruncatedPadicDomain

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.TruncatedPadicDomain`. Reflected implementation `komorebi_math.generating.domains.TruncatedPadicDomain`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `class`. Profile `C3-FOUND-DOMAIN`.

Manifest signature (s).

```
(prime: 'int', precision: 'int') -> 'None'
```

Definition. The finite ring `Z / p^r Z` with declared prime and precision.

Contract (D, C, H, R, X). A coefficient domain supplies coercion, exact arithmetic, unit tests, inversion, equality, portable encoding, decoding, and a descriptor containing only data. Integer inversion is restricted to `1` and `-1`. Residue ring inversion requires a unit. Prime fields require a prime modulus. Truncated p-adic domains require a prime and positive precision. Descriptor reconstruction accepts only the closed supported schemas. The principal public operations are `coerce`, `add`, `subtract`, `multiply`, `inverse`, `valuation`, `encode`, `decode`, `descriptor`.

Guarantees (E, P, B, K). All arithmetic is exact in the declared finite or rational domain. Encodings are canonical for the represented element. Prime validation is deterministic for the supplied modulus. Operations are constant time in the number of coefficients, although integer bit complexity grows with operand size.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test is `tests/test_generating_domains.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 21, 23, and 61.

C.3A · WeightedMonomialBasis

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.WeightedMonomialBasis`. Reflected implementation `komorebi_math.generating.basis.WeightedMonomialBasis`. The same runtime object is also exported as `komorebi_math.WeightedMonomialBasis`. Within that namespace, `komorebi_math.generating.NormalizedBasis` and `komorebi_math.generating.NormalizedMonomialBasis` resolve to the same runtime object. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `class`. Profile `C3-FOUND-BASIS`.

Manifest signature (s).

```
(name: 'str', gamma: 'GammaRule', *, descriptor: 'BasisDescriptor | None' = None) -> 'None'
```

Definition. A basis `e_n(z)=z^n/gamma_n` with `gamma_0=1`.

Contract (D, C, H, R, X). A normalized monomial basis is defined by exact nonzero values `gamma_n` with `gamma_0=1`. Names must be nonempty. Indices and finite audit degrees must be nonnegative. Packet addresses must have a positive modulus and a valid residue. Declarative descriptors use a closed registry. Callback bases remain usable but are not portable unless replaced by a finite table descriptor. The principal public operations are `gamma`, `lowering_weight`, `structure_constant`, `packet_transfer_weight`, `finite_certificate`, `require_descriptor`.

Guarantees (E, P, B, K). Normalization, lowering weights, structure constants, packet transfer weights, and finite certificates use exact `Fraction` arithmetic. Finite audits establish only the requested degree range. Computing all structure constants through degree `d` requires a quadratic number of exact ratios, and numerator or denominator sizes can grow quickly.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused tests are `tests/test_generating_foundation.py` and `tests/test_generating_domains.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 21, 22, and 24.

C.3A · additive_series

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.additive_series`. Reflected implementation `komorebi_math.generating.monoid.additive_series`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-FOUND-MONOID`.

Manifest signature (s).

```
(values: 'Iterable[int | Fraction]') -> 'FiniteMonoidSeries[int]'
```

Definition. Create a finite ordinary series whose index operation is addition.

Contract (D, C, H, R, X). A finite monoid series stores a finite coefficient map, an index combination rule, an identity, and a name. Multiplication requires the same identity and combines only represented pairs, optionally filtered by an admissibility predicate. Additive series use nonnegative indices. Multiplicative series use positive integer indices. Dirichlet convolution validates a nonnegative count, and valuation packets require a prime of at least two.

Guarantees (E, P, B, K). Coefficient arithmetic is exact over `Fraction`. The class assumes that the index operation supplied by the caller is associative and does not prove that law. Dense finite multiplication is quadratic in the number of stored indices. Dirichlet convolution uses direct divisor enumeration for the requested prefix.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test is `tests/test_generating_foundation.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 21, 24, and 25.

C.3A · algebraic_function

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.algebraic_function`. Reflected implementation `komorebi_math.generating.functions.algebraic_function`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-FOUND-FUNCTION`.

Manifest signature (s).

```
(name: 'str', polynomial_terms: 'Mapping[tuple[int, int], int | Fraction]', initial_value: 'int | Fraction', *,
    ↪ basis: 'WeightedMonomialBasis | None' = None) -> 'CustomFunctionSpec'
```

Definition. Create the unique formal branch of `P(z,y)=0` at `y(0)=y0`.

Contract (D, C, H, R, X). A custom function has a nonempty name, an explicit normalized basis, a finite coefficient builder, a source kind, and portable metadata where required. Builders must return exactly `degree+1` coefficients. Recurrence seeds must match the recurrence order. Rational denominators need a nonzero constant. Algebraic branches require `P(0,y0)=0` and a nonzero partial derivative with respect to `y`. Registries reject duplicate names.

Guarantees (E, P, B, K). Each call constructs an exact finite `FormalSeries`. Rational, recurrence, hypergeometric, algebraic, Euler product, and Bell builders certify only the generated finite prefix. Runtime is polynomial in the requested degree for the implemented recurrences, but exact coefficient growth can dominate memory and arithmetic cost.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test is `tests/test_generating_foundation.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 21 through 24.

C.3A · apery_numbers

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.apery_numbers`. Reflected implementation `komorebi_math.generating.sequences.apery_numbers`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-FOUND-SEQUENCE`.

Manifest signature (s).

```
(count: 'int') -> 'tuple[int, ...]'
```

Definition. Return the Apéry numbers for zeta(3) by their finite binomial sum.

Contract (D, C, H, R, X). The requested count must be a nonnegative integer. Factorial power parameters must satisfy the positive order required by the family. Weighted Bell constructors accept either a finite weight source or a callback and require enough weights for the requested prefix. Returned tuples use the convention stated by each sequence, including `B_1=-1/2` and Fibonacci beginning at zero.

Guarantees (E, P, B, K). The functions return deterministic exact integer or rational prefixes. They establish the implemented recurrence or finite sum for those indices only. Most recurrences use quadratic work in the prefix length, while Apéry and weighted Bell sums can incur larger exact integer costs.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test is `tests/test_generating_foundation.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 22, 24, and 26.

C.3A · audit_section_cycle

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.audit_section_cycle`. Reflected implementation `komorebi_math.generating.sections.audit_section_cycle`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-FOUND-SECTION`.

Manifest signature (s).

```
(basis: 'WeightedMonomialBasis', *, modulus: 'int', degree: 'int', signed: 'bool' = False, z: 'complex' =
   ↪  (0.37+0.11j)) -> 'SectionCycleAudit'
```

Definition. Audit lowering cycles and root-filter projection on finite truncations.

Contract (D, C, H, R, X). The truncation degree must be nonnegative. A section requires a positive modulus and a residue in range. The signed option changes the lowering cycle convention but not the address rules. The audit compares one step, a full cycle, and a finite complex root filter at the supplied evaluation point.

Guarantees (E, P, B, K). Series coefficients and lowering operations are exact. The root filter is a finite floating point evaluation and is reported through a residual rather than exact equality. Construction is linear in the truncation degree, while an audit over all residue classes scales with both degree and modulus.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test is `tests/test_generating_foundation.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 22, 24, and 25.

C.3A · basis_from_descriptor

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.basis_from_descriptor`. Reflected implementation `komorebi_math.generating.basis.basis_from_descriptor`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-FOUND-BASIS`.

Manifest signature (s).

```
(descriptor: 'BasisDescriptor') -> 'WeightedMonomialBasis'
```

Definition. Rebuild a basis from the closed declarative descriptor registry.

Contract (D, C, H, R, X). A normalized monomial basis is defined by exact nonzero values `gamma_n` with `gamma_0=1`. Names must be nonempty. Indices and finite audit degrees must be nonnegative. Packet addresses must have a positive modulus and a valid residue. Declarative descriptors use a closed registry. Callback bases remain usable but are not portable unless replaced by a finite table descriptor.

Guarantees (E, P, B, K). Normalization, lowering weights, structure constants, packet transfer weights, and finite certificates use exact `Fraction` arithmetic. Finite audits establish only the requested degree range. Computing all structure constants through degree `d` requires a quadratic number of exact ratios, and numerator or denominator sizes can grow quickly.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused tests are `tests/test_generating_foundation.py` and `tests/test_generating_domains.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 21, 22, and 24.

C.3A · basis_from_values

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.basis_from_values`. Reflected implementation `komorebi_math.generating.basis.basis_from_values`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-FOUND-BASIS`.

Manifest signature (s).

```
(name: 'str', values: 'Iterable[int | Fraction]') -> 'WeightedMonomialBasis'
```

Definition. Create a finite portable basis from an explicit normalization table.

Contract (D, C, H, R, X). A normalized monomial basis is defined by exact nonzero values `gamma_n` with `gamma_0=1`. Names must be nonempty. Indices and finite audit degrees must be nonnegative. Packet addresses must have a positive modulus and a valid residue. Declarative descriptors use a closed registry. Callback bases remain usable but are not portable unless replaced by a finite table descriptor.

Guarantees (E, P, B, K). Normalization, lowering weights, structure constants, packet transfer weights, and finite certificates use exact `Fraction` arithmetic. Finite audits establish only the requested degree range. Computing all structure constants through degree `d` requires a quadratic number of exact ratios, and numerator or denominator sizes can grow quickly.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused tests are `tests/test_generating_foundation.py` and `tests/test_generating_domains.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 21, 22, and 24.

C.3A · bell_numbers

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.bell_numbers`. Reflected implementation `komorebi_math.generating.sequences.bell_numbers`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-FOUND-SEQUENCE`.

Manifest signature (s).

```
(count: 'int') -> 'tuple[int, ...]'
```

Definition. Return the first `count` ordinary Bell numbers by their recurrence.

Contract (D, C, H, R, X). The requested count must be a nonnegative integer. Factorial power parameters must satisfy the positive order required by the family. Weighted Bell constructors accept either a finite weight source or a callback and require enough weights for the requested prefix. Returned tuples use the convention stated by each sequence, including `B_1=-1/2` and Fibonacci beginning at zero.

Guarantees (E, P, B, K). The functions return deterministic exact integer or rational prefixes. They establish the implemented recurrence or finite sum for those indices only. Most recurrences use quadratic work in the prefix length, while Apéry and weighted Bell sums can incur larger exact integer costs.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test is `tests/test_generating_foundation.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 22, 24, and 26.

C.3A · bernoulli_numbers

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.bernoulli_numbers`. Reflected implementation `komorebi_math.generating.sequences.bernoulli_numbers`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-FOUND-SEQUENCE`.

Manifest signature (s).

```
(count: 'int') -> 'tuple[Fraction, ...]'
```

Definition. Return Bernoulli numbers with `B_1=-1/2`.

Contract (D, C, H, R, X). The requested count must be a nonnegative integer. Factorial power parameters must satisfy the positive order required by the family. Weighted Bell constructors accept either a finite weight source or a callback and require enough weights for the requested prefix. Returned tuples use the convention stated by each sequence, including `B_1=-1/2` and Fibonacci beginning at zero.

Guarantees (E, P, B, K). The functions return deterministic exact integer or rational prefixes. They establish the implemented recurrence or finite sum for those indices only. Most recurrences use quadratic work in the prefix length, while Apéry and weighted Bell sums can incur larger exact integer costs.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused tests are `tests/test_generating_foundation.py` and `tests/test_arithmetic_euler.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 22, 24, and 26.

C.3A · catalan_numbers

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.catalan_numbers`. Reflected implementation `komorebi_math.generating.sequences.catalan_numbers`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-FOUND-SEQUENCE`.

Manifest signature (s).

```
(count: 'int') -> 'tuple[int, ...]'
```

Definition. Return the first `count` Catalan numbers.

Contract (D, C, H, R, X). The requested count must be a nonnegative integer. Factorial power parameters must satisfy the positive order required by the family. Weighted Bell constructors accept either a finite weight source or a callback and require enough weights for the requested prefix. Returned tuples use the convention stated by each sequence, including `B_1=-1/2` and Fibonacci beginning at zero.

Guarantees (E, P, B, K). The functions return deterministic exact integer or rational prefixes. They establish the implemented recurrence or finite sum for those indices only. Most recurrences use quadratic work in the prefix length, while Apéry and weighted Bell sums can incur larger exact integer costs.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test is `tests/test_generating_foundation.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 22, 24, and 26.

C.3A · central_binomial_numbers

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.central_binomial_numbers`. Reflected implementation `komorebi_math.generating.sequences.central_binomial_numbers`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-FOUND-SEQUENCE`.

Manifest signature (s).

```
(count: 'int') -> 'tuple[int, ...]'
```

Definition. Return `binomial(2*n, n)` for `0 <= n < count`.

Contract (D, C, H, R, X). The requested count must be a nonnegative integer. Factorial power parameters must satisfy the positive order required by the family. Weighted Bell constructors accept either a finite weight source or a callback and require enough weights for the requested prefix. Returned tuples use the convention stated by each sequence, including `B_1=-1/2` and Fibonacci beginning at zero.

Guarantees (E, P, B, K). The functions return deterministic exact integer or rational prefixes. They establish the implemented recurrence or finite sum for those indices only. Most recurrences use quadratic work in the prefix length, while Apéry and weighted Bell sums can incur larger exact integer costs.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test is `tests/test_generating_foundation.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 22, 24, and 26.

C.3A · characteristic_closed_form

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.characteristic_closed_form`. Reflected implementation `komorebi_math.generating.closed_forms.characteristic_closed_form`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-FOUND-INFERENCE`.

Manifest signature (s).

```
(recurrence: 'LinearRecurrenceGuess', initial: 'Sequence[int | Fraction]') -> 'Any | None'
```

Definition. Ask SymPy for a symbolic closed form of a constant recurrence.

Contract (D, C, H, R, X). All guessers consume a finite exact prefix and search only within the declared order and degree bounds. A holdout separates fitted relations from later checks when enough data are available. Failure to find a candidate returns `None`. Guess records preserve the fit count, holdout count, and exact prefix status. Symbolic expression methods and characteristic forms require the optional SymPy backend.

Guarantees (E, P, B, K). Linear systems and residual checks use exact rational or integer arithmetic. A returned candidate is evidence for the supplied finite prefix, not a proof that the relation holds for every index. Search cost grows rapidly with the product of recurrence order, polynomial degree, and algebraic bidegree, so the explicit bounds are part of the contract.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test is `tests/test_generating_foundation.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 22, 24, and 26.

C.3A · compose_functions

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.compose_functions`. Reflected implementation `komorebi_math.generating.functions.compose_functions`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-FOUND-FUNCTION`.

Manifest signature (s).

```
(name: 'str', outer: 'CustomFunctionSpec', inner: 'CustomFunctionSpec') -> 'CustomFunctionSpec'
```

Definition. Create an exact ordinary-composition specification.

Contract (D, C, H, R, X). A custom function has a nonempty name, an explicit normalized basis, a finite coefficient builder, a source kind, and portable metadata where required. Builders must return exactly `degree+1` coefficients. Recurrence seeds must match the recurrence order. Rational denominators need a nonzero constant. Algebraic branches require `P(0,y0)=0` and a nonzero partial derivative with respect to `y`. Registries reject duplicate names.

Guarantees (E, P, B, K). Each call constructs an exact finite `FormalSeries`. Rational, recurrence, hypergeometric, algebraic, Euler product, and Bell builders certify only the generated finite prefix. Runtime is polynomial in the requested degree for the implemented recurrences, but exact coefficient growth can dominate memory and arithmetic cost.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test is `tests/test_generating_foundation.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 21 through 24.

C.3A · custom_exponential

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.custom_exponential`. Reflected implementation `komorebi_math.generating.sections.custom_exponential`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-FOUND-SECTION`.

Manifest signature (s).

```
(basis: 'WeightedMonomialBasis', degree: 'int') -> 'FormalSeries'
```

Definition. Return `E_gamma(z)=sum z^n/gamma_n` through one degree.

Contract (D, C, H, R, X). The truncation degree must be nonnegative. A section requires a positive modulus and a residue in range. The signed option changes the lowering cycle convention but not the address rules. The audit compares one step, a full cycle, and a finite complex root filter at the supplied evaluation point.

Guarantees (E, P, B, K). Series coefficients and lowering operations are exact. The root filter is a finite floating point evaluation and is reported through a residual rather than exact equality. Construction is linear in the truncation degree, while an audit over all residue classes scales with both degree and modulus.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test is `tests/test_generating_foundation.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 22, 24, and 25.

C.3A · custom_section

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.custom_section`. Reflected implementation `komorebi_math.generating.sections.custom_section`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-FOUND-SECTION`.

Manifest signature (s).

```
(basis: 'WeightedMonomialBasis', modulus: 'int', residue: 'int', degree: 'int', *, signed: 'bool' = False) ->
    ↪ 'FormalSeries'
```

Definition. Return one residue section of the custom exponential.

Contract (D, C, H, R, X). The truncation degree must be nonnegative. A section requires a positive modulus and a residue in range. The signed option changes the lowering cycle convention but not the address rules. The audit compares one step, a full cycle, and a finite complex root filter at the supplied evaluation point.

Guarantees (E, P, B, K). Series coefficients and lowering operations are exact. The root filter is a finite floating point evaluation and is reported through a residual rather than exact equality. Construction is linear in the truncation degree, while an audit over all residue classes scales with both degree and modulus.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test is `tests/test_generating_foundation.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 22, 24, and 25.

C.3A · dirichlet_convolution

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.dirichlet_convolution`. Reflected implementation `komorebi_math.generating.monoid.dirichlet_convolution`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-FOUND-MONOID`.

Manifest signature (s).

```
(left: 'Iterable[int | Fraction]', right: 'Iterable[int | Fraction]', *, count: 'int') -> 'tuple[Fraction, ...]'
```

Definition. Return `(a*b)(n)=sum_(d|n)a(d)b(n/d)` for `1 <= n <= count`.

Contract (D, C, H, R, X). A finite monoid series stores a finite coefficient map, an index combination rule, an identity, and a name. Multiplication requires the same identity and combines only represented pairs, optionally filtered by an admissibility predicate. Additive series use nonnegative indices. Multiplicative series use positive integer indices. Dirichlet convolution validates a nonnegative count, and valuation packets require a prime of at least two.

Guarantees (E, P, B, K). Coefficient arithmetic is exact over `Fraction`. The class assumes that the index operation supplied by the caller is associative and does not prove that law. Dense finite multiplication is quadratic in the number of stored indices. Dirichlet convolution uses direct divisor enumeration for the requested prefix.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test is `tests/test_generating_foundation.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 21, 24, and 25.

C.3A · domain_from_descriptor

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.domain_from_descriptor`. Reflected implementation `komorebi_math.generating.domains.domain_from_descriptor`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-FOUND-DOMAIN`.

Manifest signature (s).

```
(payload: 'Any') -> 'CoefficientDomain[Any]'
```

Definition. Construct one supported exact domain from a data-only descriptor.

Contract (D, C, H, R, X). A coefficient domain supplies coercion, exact arithmetic, unit tests, inversion, equality, portable encoding, decoding, and a descriptor containing only data. Integer inversion is restricted to `1` and `-1`. Residue ring inversion requires a unit. Prime fields require a prime modulus. Truncated p-adic domains require a prime and positive precision. Descriptor reconstruction accepts only the closed supported schemas.

Guarantees (E, P, B, K). All arithmetic is exact in the declared finite or rational domain. Encodings are canonical for the represented element. Prime validation is deterministic for the supplied modulus. Operations are constant time in the number of coefficients, although integer bit complexity grows with operand size.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test is `tests/test_generating_domains.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 21, 23, and 61.

C.3A · euler_product_function

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.euler_product_function`. Reflected implementation `komorebi_math.generating.functions.euler_product_function`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-FOUND-FUNCTION`.

Manifest signature (s).

```
(name: 'str', exponents: 'Mapping[int, int | Fraction] | Callable[[int], int | Fraction]', *, basis:
    ↪ 'WeightedMonomialBasis | None' = None) -> 'CustomFunctionSpec'
```

Definition. Create `prod_(m>=1) (1-z^m)^(-c_m)` by log-derivative recurrence.

Contract (D, C, H, R, X). A custom function has a nonempty name, an explicit normalized basis, a finite coefficient builder, a source kind, and portable metadata where required. Builders must return exactly `degree+1` coefficients. Recurrence seeds must match the recurrence order. Rational denominators need a nonzero constant. Algebraic branches require `P(0,y0)=0` and a nonzero partial derivative with respect to `y`. Registries reject duplicate names.

Guarantees (E, P, B, K). Each call constructs an exact finite `FormalSeries`. Rational, recurrence, hypergeometric, algebraic, Euler product, and Bell builders certify only the generated finite prefix. Runtime is polynomial in the requested degree for the implemented recurrences, but exact coefficient growth can dominate memory and arithmetic cost.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test is `tests/test_generating_foundation.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 21 through 24.

C.3A · explicit_function

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.explicit_function`. Reflected implementation `komorebi_math.generating.functions.explicit_function`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-FOUND-FUNCTION`.

Manifest signature (s).

```
(name: 'str', basis: 'WeightedMonomialBasis', coefficients: 'tuple[int | Fraction, ...]') -> 'CustomFunctionSpec'
```

Definition. Create a finite exact coefficient source in a declared normalized basis.

Contract (D, C, H, R, X). A custom function has a nonempty name, an explicit normalized basis, a finite coefficient builder, a source kind, and portable metadata where required. Builders must return exactly `degree+1` coefficients. Recurrence seeds must match the recurrence order. Rational denominators need a nonzero constant. Algebraic branches require `P(0,y0)=0` and a nonzero partial derivative with respect to `y`. Registries reject duplicate names.

Guarantees (E, P, B, K). Each call constructs an exact finite `FormalSeries`. Rational, recurrence, hypergeometric, algebraic, Euler product, and Bell builders certify only the generated finite prefix. Runtime is polynomial in the requested degree for the implemented recurrences, but exact coefficient growth can dominate memory and arithmetic cost.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test is `tests/test_generating_foundation.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 21 through 24.

C.3A · exponential_basis

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.exponential_basis`. Reflected implementation `komorebi_math.generating.basis.exponential_basis`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-FOUND-BASIS`.

Manifest signature (s).

```
(*, name: 'str' = 'exponential') -> 'WeightedMonomialBasis'
```

Definition. Return the exponential generating-function basis `z^n/n!`.

Contract (D, C, H, R, X). A normalized monomial basis is defined by exact nonzero values `gamma_n` with `gamma_0=1`. Names must be nonempty. Indices and finite audit degrees must be nonnegative. Packet addresses must have a positive modulus and a valid residue. Declarative descriptors use a closed registry. Callback bases remain usable but are not portable unless replaced by a finite table descriptor.

Guarantees (E, P, B, K). Normalization, lowering weights, structure constants, packet transfer weights, and finite certificates use exact `Fraction` arithmetic. Finite audits establish only the requested degree range. Computing all structure constants through degree `d` requires a quadratic number of exact ratios, and numerator or denominator sizes can grow quickly.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused tests are `tests/test_generating_foundation.py`, `tests/test_generating_domains.py`, and `tests/test_coordinates_normalized_persistence.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 21, 22, and 24.

C.3A · factorial_power_basis

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.factorial_power_basis`. Reflected implementation `komorebi_math.generating.basis.factorial_power_basis`. The same runtime object is also exported as `komorebi_math.factorial_power_basis`, `komorebi_math.stable.factorial_power_basis`, `komorebi_math.v1.factorial_power_basis`, `komorebi_math.api.v1.factorial_power_basis`. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. The stable and versioned aliases keep their separate frozen compatibility commitment. Manifest kind `function`. Profile `C3-FOUND-BASIS`.

Manifest signature (s).

```
(power: 'int', *, name: 'str | None' = None) -> 'WeightedMonomialBasis'
```

Definition. Return `z^n/(n!)^power` for a positive integer power.

Contract (D, C, H, R, X). A normalized monomial basis is defined by exact nonzero values `gamma_n` with `gamma_0=1`. Names must be nonempty. Indices and finite audit degrees must be nonnegative. Packet addresses must have a positive modulus and a valid residue. Declarative descriptors use a closed registry. Callback bases remain usable but are not portable unless replaced by a finite table descriptor.

Guarantees (E, P, B, K). Normalization, lowering weights, structure constants, packet transfer weights, and finite certificates use exact `Fraction` arithmetic. Finite audits establish only the requested degree range. Computing all structure constants through degree `d` requires a quadratic number of exact ratios, and numerator or denominator sizes can grow quickly.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused tests are `tests/test_generating_foundation.py` and `tests/test_generating_domains.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 21, 22, and 24.

C.3A · factorial_power_bell_function

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.factorial_power_bell_function`. Reflected implementation `komorebi_math.generating.functions.factorial_power_bell_function`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-FOUND-FUNCTION`.

Manifest signature (s).

```
(power: 'int', *, name: 'str | None' = None) -> 'CustomFunctionSpec'
```

Definition. Create the unweighted factorial-power Bell family of a positive order.

Contract (D, C, H, R, X). A custom function has a nonempty name, an explicit normalized basis, a finite coefficient builder, a source kind, and portable metadata where required. Builders must return exactly `degree+1` coefficients. Recurrence seeds must match the recurrence order. Rational denominators need a nonzero constant. Algebraic branches require `P(0,y0)=0` and a nonzero partial derivative with respect to `y`. Registries reject duplicate names.

Guarantees (E, P, B, K). Each call constructs an exact finite `FormalSeries`. Rational, recurrence, hypergeometric, algebraic, Euler product, and Bell builders certify only the generated finite prefix. Runtime is polynomial in the requested degree for the implemented recurrences, but exact coefficient growth can dominate memory and arithmetic cost.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test is `tests/test_generating_foundation.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 21 through 24.

C.3A · fibonacci_factorial_basis

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.fibonacci_factorial_basis`. Reflected implementation `komorebi_math.generating.basis.fibonacci_factorial_basis`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-FOUND-BASIS`.

Manifest signature (s).

```
(*, name: 'str' = 'fibonacci-factorial') -> 'WeightedMonomialBasis'
```

Definition. Return the fibonomial basis with `gamma_n=prod_(j=1)^n F_j`.

Contract (D, C, H, R, X). A normalized monomial basis is defined by exact nonzero values `gamma_n` with `gamma_0=1`. Names must be nonempty. Indices and finite audit degrees must be nonnegative. Packet addresses must have a positive modulus and a valid residue. Declarative descriptors use a closed registry. Callback bases remain usable but are not portable unless replaced by a finite table descriptor.

Guarantees (E, P, B, K). Normalization, lowering weights, structure constants, packet transfer weights, and finite certificates use exact `Fraction` arithmetic. Finite audits establish only the requested degree range. Computing all structure constants through degree `d` requires a quadratic number of exact ratios, and numerator or denominator sizes can grow quickly.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused tests are `tests/test_generating_foundation.py`, `tests/test_generating_domains.py`, and `tests/test_coordinates_normalized_persistence.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 21, 22, and 24.

C.3A · fibonacci_numbers

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.fibonacci_numbers`. Reflected implementation `komorebi_math.generating.sequences.fibonacci_numbers`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-FOUND-SEQUENCE`.

Manifest signature (s).

```
(count: 'int') -> 'tuple[int, ...]'
```

Definition. Return the first `count` Fibonacci numbers beginning with zero.

Contract (D, C, H, R, X). The requested count must be a nonnegative integer. Factorial power parameters must satisfy the positive order required by the family. Weighted Bell constructors accept either a finite weight source or a callback and require enough weights for the requested prefix. Returned tuples use the convention stated by each sequence, including `B_1=-1/2` and Fibonacci beginning at zero.

Guarantees (E, P, B, K). The functions return deterministic exact integer or rational prefixes. They establish the implemented recurrence or finite sum for those indices only. Most recurrences use quadratic work in the prefix length, while Apéry and weighted Bell sums can incur larger exact integer costs.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test is `tests/test_generating_foundation.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 22, 24, and 26.

C.3A · fir_taps_to_series

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.fir_taps_to_series`. Reflected implementation `komorebi_math.generating.adapters.fir_taps_to_series`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-FOUND-ADAPTER`.

Manifest signature (s).

```
(taps: 'Any', *, basis: 'WeightedMonomialBasis | None' = None, max_denominator: 'int | None' = None) ->
    ↪ 'FormalSeries'
```

Definition. Convert a finite real FIR tap vector into an exact formal series.

Contract (D, C, H, R, X). FIR inputs must be a nonempty one dimensional finite real array. Conversion to `FormalSeries` rationalizes taps, optionally limiting denominators. Conversion from a series validates the requested degree and numeric dtype. An adapter requires one tap per stored ordinary coefficient. Frequency responses validate finite sampling parameters and certificates compare the declared coefficient to tap conversion.

Guarantees (E, P, B, K). The formal side is exact before conversion to NumPy. Tap arrays and frequency responses use floating point arithmetic, and their certificates are qualified by a stated tolerance. Conversion is linear in the number of taps. Frequency response evaluation delegates to the signal implementation and scales with the chosen point count.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test is `tests/test_generating_block13.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 21, 31, and 66.

C.3A · freeze_custom_function

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.freeze_custom_function`. Reflected implementation `komorebi_math.generating.portable.freeze_custom_function`. The same runtime object is also exported as `komorebi_math.freeze_custom_function`. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-FOUND-PORTABLE`.

Manifest signature (s).

```
(function: 'CustomFunctionSpec', degree: 'int', *, include_metadata: 'bool' = True) -> 'FrozenGeneratingFunction'
```

Definition. Capture one custom function into a finite provider-independent record.

Contract (D, C, H, R, X). A frozen function requires nonempty names, a nonempty coefficient prefix, normalization of equal length beginning with one, no zero normalization values, and portable metadata. Reconstruction cannot exceed the captured degree. Freezing accepts only `CustomFunctionSpec`. Codec registration installs closed data only decoders and does not permit arbitrary Python object construction.

Guarantees (E, P, B, K). The frozen record preserves exact rational coefficients and basis normalization independently of the original callback. Roundtrip integrity is delegated to the project portable envelope and its digest checks. Serialization cost is linear in the captured prefix, subject to the global payload budget used by the persistence layer.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test is `tests/test_generating_block13.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 21, 61, and 72.

C.3A · generalized_bell_numbers

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.generalized_bell_numbers`. Reflected implementation `komorebi_math.generating.sequences.generalized_bell_numbers`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-FOUND-SEQUENCE`.

Manifest signature (s).

```
(power: 'int', count: 'int') -> 'tuple[int, ...]'
```

Definition. Return the factorial-power Bell family.

Contract (D, C, H, R, X). The requested count must be a nonnegative integer. Factorial power parameters must satisfy the positive order required by the family. Weighted Bell constructors accept either a finite weight source or a callback and require enough weights for the requested prefix. Returned tuples use the convention stated by each sequence, including `B_1=-1/2` and Fibonacci beginning at zero.

Guarantees (E, P, B, K). The functions return deterministic exact integer or rational prefixes. They establish the implemented recurrence or finite sum for those indices only. Most recurrences use quadratic work in the prefix length, while Apéry and weighted Bell sums can incur larger exact integer costs.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test is `tests/test_generating_foundation.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 22, 24, and 26.

C.3A · generalized_bell_polynomials

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.generalized_bell_polynomials`. Reflected implementation `komorebi_math.generating.sequences.generalized_bell_polynomials`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-FOUND-SEQUENCE`.

Manifest signature (s).

```
(power: 'int', count: 'int') -> 'tuple[tuple[int, ...], ...]'
```

Definition. Return coefficient rows of factorial-power Bell polynomials.

Contract (D, C, H, R, X). The requested count must be a nonnegative integer. Factorial power parameters must satisfy the positive order required by the family. Weighted Bell constructors accept either a finite weight source or a callback and require enough weights for the requested prefix. Returned tuples use the convention stated by each sequence, including `B_1=-1/2` and Fibonacci beginning at zero.

Guarantees (E, P, B, K). The functions return deterministic exact integer or rational prefixes. They establish the implemented recurrence or finite sum for those indices only. Most recurrences use quadratic work in the prefix length, while Apéry and weighted Bell sums can incur larger exact integer costs.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test is `tests/test_generating_foundation.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 22, 24, and 26.

C.3A · guess_algebraic_gf

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.guess_algebraic_gf`. Reflected implementation `komorebi_math.generating.closed_forms.guess_algebraic_gf`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-FOUND-INFERENCE`.

Manifest signature (s).

```
(values: 'Iterable[int | Fraction]', *, max_x_degree: 'int' = 4, max_y_degree: 'int' = 4, holdout: 'int | None' =
    ↪ None) -> 'AlgebraicGFGuess | None'
```

Definition. Guess an exact relation `P(x,F(x))=0` from ordinary coefficients.

Contract (D, C, H, R, X). All guessers consume a finite exact prefix and search only within the declared order and degree bounds. A holdout separates fitted relations from later checks when enough data are available. Failure to find a candidate returns `None`. Guess records preserve the fit count, holdout count, and exact prefix status. Symbolic expression methods and characteristic forms require the optional SymPy backend.

Guarantees (E, P, B, K). Linear systems and residual checks use exact rational or integer arithmetic. A returned candidate is evidence for the supplied finite prefix, not a proof that the relation holds for every index. Search cost grows rapidly with the product of recurrence order, polynomial degree, and algebraic bidegree, so the explicit bounds are part of the contract.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test is `tests/test_generating_foundation.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 22, 24, and 26.

C.3A · guess_hypergeometric_ratio

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.guess_hypergeometric_ratio`. Reflected implementation `komorebi_math.generating.closed_forms.guess_hypergeometric_ratio`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-FOUND-INFERENCE`.

Manifest signature (s).

```
(values: 'Iterable[int | Fraction]', *, max_numerator_degree: 'int' = 4, max_denominator_degree: 'int' = 4,
   ↪ holdout: 'int | None' = None) -> 'HypergeometricGuess | None'
```

Definition. Guess `a[n+1]/a[n]=P(n)/Q(n)` by exact nullspace search.

Contract (D, C, H, R, X). All guessers consume a finite exact prefix and search only within the declared order and degree bounds. A holdout separates fitted relations from later checks when enough data are available. Failure to find a candidate returns `None`. Guess records preserve the fit count, holdout count, and exact prefix status. Symbolic expression methods and characteristic forms require the optional SymPy backend.

Guarantees (E, P, B, K). Linear systems and residual checks use exact rational or integer arithmetic. A returned candidate is evidence for the supplied finite prefix, not a proof that the relation holds for every index. Search cost grows rapidly with the product of recurrence order, polynomial degree, and algebraic bidegree, so the explicit bounds are part of the contract.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test is `tests/test_generating_foundation.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 22, 24, and 26.

C.3A · guess_linear_recurrence

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.guess_linear_recurrence`. Reflected implementation `komorebi_math.generating.closed_forms.guess_linear_recurrence`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-FOUND-INFERENCE`.

Manifest signature (s).

```
(values: 'Iterable[int | Fraction]', *, max_order: 'int' = 12, holdout: 'int | None' = None) ->
   ↪ 'LinearRecurrenceGuess | None'
```

Definition. Guess the smallest exact constant-coefficient recurrence.

Contract (D, C, H, R, X). All guessers consume a finite exact prefix and search only within the declared order and degree bounds. A holdout separates fitted relations from later checks when enough data are available. Failure to find a candidate returns `None`. Guess records preserve the fit count, holdout count, and exact prefix status. Symbolic expression methods and characteristic forms require the optional SymPy backend.

Guarantees (E, P, B, K). Linear systems and residual checks use exact rational or integer arithmetic. A returned candidate is evidence for the supplied finite prefix, not a proof that the relation holds for every index. Search cost grows rapidly with the product of recurrence order, polynomial degree, and algebraic bidegree, so the explicit bounds are part of the contract.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test is `tests/test_generating_foundation.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 22, 24, and 26.

C.3A · guess_p_recursive

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.guess_p_recursive`. Reflected implementation `komorebi_math.generating.closed_forms.guess_p_recursive`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-FOUND-INFERENCE`.

Manifest signature (s).

```
(values: 'Iterable[int | Fraction]', *, max_order: 'int' = 4, max_degree: 'int' = 5, holdout: 'int | None' = None)
   ↪  -> 'PRecursiveGuess | None'
```

Definition. Guess `sum_(j=0)^r P_j(n)a[n+j]=0` exactly on a prefix.

Contract (D, C, H, R, X). All guessers consume a finite exact prefix and search only within the declared order and degree bounds. A holdout separates fitted relations from later checks when enough data are available. Failure to find a candidate returns `None`. Guess records preserve the fit count, holdout count, and exact prefix status. Symbolic expression methods and characteristic forms require the optional SymPy backend.

Guarantees (E, P, B, K). Linear systems and residual checks use exact rational or integer arithmetic. A returned candidate is evidence for the supplied finite prefix, not a proof that the relation holds for every index. Search cost grows rapidly with the product of recurrence order, polynomial degree, and algebraic bidegree, so the explicit bounds are part of the contract.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test is `tests/test_generating_foundation.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 22, 24, and 26.

C.3A · guess_quasipolynomial

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.guess_quasipolynomial`. Reflected implementation `komorebi_math.generating.closed_forms.guess_quasipolynomial`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-FOUND-INFERENCE`.

Manifest signature (s).

```
(values: 'Iterable[int | Fraction]', *, max_period: 'int' = 12, max_degree: 'int' = 6, holdout: 'int | None' =
   ↪  None) -> 'QuasiPolynomialGuess | None'
```

Definition. Guess a quasipolynomial by constant finite differences on each lane.

Contract (D, C, H, R, X). All guessers consume a finite exact prefix and search only within the declared order and degree bounds. A holdout separates fitted relations from later checks when enough data are available. Failure to find a candidate returns `None`. Guess records preserve the fit count, holdout count, and exact prefix status. Symbolic expression methods and characteristic forms require the optional SymPy backend.

Guarantees (E, P, B, K). Linear systems and residual checks use exact rational or integer arithmetic. A returned candidate is evidence for the supplied finite prefix, not a proof that the relation holds for every index. Search cost grows rapidly with the product of recurrence order, polynomial degree, and algebraic bidegree, so the explicit bounds are part of the contract.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test is `tests/test_generating_foundation.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 22, 24, and 26.

C.3A · hypergeometric_function

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.hypergeometric_function`. Reflected implementation `komorebi_math.generating.functions.hypergeometric_function`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-FOUND-FUNCTION`.

Manifest signature (s).

```
(name: 'str', initial: 'int | Fraction', ratio: 'Callable[[int], int | Fraction]', *, basis:
    ↪ 'WeightedMonomialBasis | None' = None) -> 'CustomFunctionSpec'
```

Definition. Create a term sequence with `a[n+1]/a[n]=ratio(n)`.

Contract (D, C, H, R, X). A custom function has a nonempty name, an explicit normalized basis, a finite coefficient builder, a source kind, and portable metadata where required. Builders must return exactly `degree+1` coefficients. Recurrence seeds must match the recurrence order. Rational denominators need a nonzero constant. Algebraic branches require `P(0,y0)=0` and a nonzero partial derivative with respect to `y`. Registries reject duplicate names.

Guarantees (E, P, B, K). Each call constructs an exact finite `FormalSeries`. Rational, recurrence, hypergeometric, algebraic, Euler product, and Bell builders certify only the generated finite prefix. Runtime is polynomial in the requested degree for the implemented recurrences, but exact coefficient growth can dominate memory and arithmetic cost.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test is `tests/test_generating_foundation.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 21 through 24.

C.3A · linear_recurrence_function

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.linear_recurrence_function`. Reflected implementation `komorebi_math.generating.functions.linear_recurrence_function`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-FOUND-FUNCTION`.

Manifest signature (s).

```
(name: 'str', coefficients: 'tuple[int | Fraction, ...]', initial: 'tuple[int | Fraction, ...]', *, basis:
    ↪ 'WeightedMonomialBasis | None' = None) -> 'CustomFunctionSpec'
```

Definition. Create `a[n+d]=sum_j c_j a[n+j]`.

Contract (D, C, H, R, X). A custom function has a nonempty name, an explicit normalized basis, a finite coefficient builder, a source kind, and portable metadata where required. Builders must return exactly `degree+1` coefficients. Recurrence seeds must match the recurrence order. Rational denominators need a nonzero constant. Algebraic branches require `P(0,y0)=0` and a nonzero partial derivative with respect to `y`. Registries reject duplicate names.

Guarantees (E, P, B, K). Each call constructs an exact finite `FormalSeries`. Rational, recurrence, hypergeometric, algebraic, Euler product, and Bell builders certify only the generated finite prefix. Runtime is polynomial in the requested degree for the implemented recurrences, but exact coefficient growth can dominate memory and arithmetic cost.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test is `tests/test_generating_foundation.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 21 through 24.

C.3A · multiplicative_series

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.multiplicative_series`. Reflected implementation `komorebi_math.generating.monoid.multiplicative_series`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-FOUND-MONOID`.

Manifest signature (s).

```
(values: 'Iterable[int | Fraction]') -> 'FiniteMonoidSeries[int]'
```

Definition. Create coefficients indexed by positive integers, starting at one.

Contract (D, C, H, R, X). A finite monoid series stores a finite coefficient map, an index combination rule, an identity, and a name. Multiplication requires the same identity and combines only represented pairs, optionally filtered by an admissibility predicate. Additive series use nonnegative indices. Multiplicative series use positive integer indices. Dirichlet convolution validates a nonnegative count, and valuation packets require a prime of at least two.

Guarantees (E, P, B, K). Coefficient arithmetic is exact over `Fraction`. The class assumes that the index operation supplied by the caller is associative and does not prove that law. Dense finite multiplication is quadratic in the number of stored indices. Dirichlet convolution uses direct divisor enumeration for the requested prefix.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test is `tests/test_generating_foundation.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 21, 24, and 25.

C.3A · ordinary_basis

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.ordinary_basis`. Reflected implementation `komorebi_math.generating.basis.ordinary_basis`. The same runtime object is also exported as `komorebi_math.ordinary_basis`. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-FOUND-BASIS`.

Manifest signature (s).

```
(*, name: 'str' = 'ordinary') -> 'WeightedMonomialBasis'
```

Definition. Return the ordinary generating-function basis `z^n`.

Contract (D, C, H, R, X). A normalized monomial basis is defined by exact nonzero values `gamma_n` with `gamma_0=1`. Names must be nonempty. Indices and finite audit degrees must be nonnegative. Packet addresses must have a positive modulus and a valid residue. Declarative descriptors use a closed registry. Callback bases remain usable but are not portable unless replaced by a finite table descriptor.

Guarantees (E, P, B, K). Normalization, lowering weights, structure constants, packet transfer weights, and finite certificates use exact `Fraction` arithmetic. Finite audits establish only the requested degree range. Computing all structure constants through degree `d` requires a quadratic number of exact ratios, and numerator or denominator sizes can grow quickly.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused tests are `tests/test_generating_foundation.py` and `tests/test_generating_domains.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 21, 22, and 24.

C.3A · partition_numbers

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.partition_numbers`. Reflected implementation `komorebi_math.generating.sequences.partition_numbers`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-FOUND-SEQUENCE`.

Manifest signature (s).

```
(count: 'int') -> 'tuple[int, ...]'
```

Definition. Return partition numbers through Euler's pentagonal recurrence.

Contract (D, C, H, R, X). The requested count must be a nonnegative integer. Factorial power parameters must satisfy the positive order required by the family. Weighted Bell constructors accept either a finite weight source or a callback and require enough weights for the requested prefix. Returned tuples use the convention stated by each sequence, including `B_1=-1/2` and Fibonacci beginning at zero.

Guarantees (E, P, B, K). The functions return deterministic exact integer or rational prefixes. They establish the implemented recurrence or finite sum for those indices only. Most recurrences use quadratic work in the prefix length, while Apéry and weighted Bell sums can incur larger exact integer costs.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test is `tests/test_generating_foundation.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 22, 24, and 26.

C.3A · pochhammer_basis

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.pochhammer_basis`. Reflected implementation `komorebi_math.generating.basis.pochhammer_basis`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-FOUND-BASIS`.

Manifest signature (s).

```
(alpha: 'int | Fraction', *, name: 'str | None' = None) -> 'WeightedMonomialBasis'
```

Definition. Return `z^n/(alpha)_n` for a nonterminating rational path.

Contract (D, C, H, R, X). A normalized monomial basis is defined by exact nonzero values `gamma_n` with `gamma_0=1`. Names must be nonempty. Indices and finite audit degrees must be nonnegative. Packet addresses must have a positive modulus and a valid residue. Declarative descriptors use a closed registry. Callback bases remain usable but are not portable unless replaced by a finite table descriptor.

Guarantees (E, P, B, K). Normalization, lowering weights, structure constants, packet transfer weights, and finite certificates use exact `Fraction` arithmetic. Finite audits establish only the requested degree range. Computing all structure constants through degree `d` requires a quadratic number of exact ratios, and numerator or denominator sizes can grow quickly.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused tests are `tests/test_generating_foundation.py` and `tests/test_generating_domains.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 21, 22, and 24.

C.3A · `prime_valuation_packets`

Identity (`q, a, t, k`). Public research manifest route `komorebi_math.generating.prime_valuation_packets`. Reflected implementation `komorebi_math.generating.monoid.prime_valuation_packets`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-FOUND-MONOID`.

Manifest signature (`s`).

```
(values: 'Iterable[int | Fraction]', prime: 'int') -> 'dict[int, tuple[tuple[int, Fraction], ...]]'
```

Definition. Group multiplicative coefficients by the exact prime-adic index valuation.

Contract (`D, C, H, R, X`). A finite monoid series stores a finite coefficient map, an index combination rule, an identity, and a name. Multiplication requires the same identity and combines only represented pairs, optionally filtered by an admissibility predicate. Additive series use nonnegative indices. Multiplicative series use positive integer indices. Dirichlet convolution validates a nonnegative count, and valuation packets require a prime of at least two.

Guarantees (`E, P, B, K`). Coefficient arithmetic is exact over `Fraction`. The class assumes that the index operation supplied by the caller is associative and does not prove that law. Dense finite multiplication is quadratic in the number of stored indices. Dirichlet convolution uses direct divisor enumeration for the requested prefix.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test is `tests/test_generating_foundation.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 21, 24, and 25.

C.3A · `q_factorial_basis`

Identity (`q, a, t, k`). Public research manifest route `komorebi_math.generating.q_factorial_basis`. Reflected implementation `komorebi_math.generating.basis.q_factorial_basis`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-FOUND-BASIS`.

Manifest signature (`s`).

```
(q: 'int', *, name: 'str | None' = None) -> 'WeightedMonomialBasis'
```

Definition. Return the integer `q`-factorial basis.

Contract (`D, C, H, R, X`). A normalized monomial basis is defined by exact nonzero values `gamma_n` with `gamma_0=1`. Names must be nonempty. Indices and finite audit degrees must be nonnegative. Packet addresses must have a positive modulus and a valid residue. Declarative descriptors use a closed registry. Callback bases remain usable but are not portable unless replaced by a finite table descriptor.

Guarantees (`E, P, B, K`). Normalization, lowering weights, structure constants, packet transfer weights, and finite certificates use exact `Fraction` arithmetic. Finite audits establish only the requested degree range. Computing all structure constants through degree `d` requires a quadratic number of exact ratios, and numerator or denominator sizes can grow quickly.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused tests are `tests/test_generating_foundation.py` and `tests/test_generating_domains.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 21, 22, and 24.

C.3A · rational_function

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.rational_function`. Reflected implementation `komorebi_math.generating.functions.rational_function`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-FOUND-FUNCTION`.

Manifest signature (s).

```
(name: 'str', numerator: 'tuple[int | Fraction, ...]', denominator: 'tuple[int | Fraction, ...]', *, basis:
   ↪  'WeightedMonomialBasis | None' = None) -> 'CustomFunctionSpec'
```

Definition. Create the coefficient sequence of one exact rational function.

Contract (D, C, H, R, X). A custom function has a nonempty name, an explicit normalized basis, a finite coefficient builder, a source kind, and portable metadata where required. Builders must return exactly `degree+1` coefficients. Recurrence seeds must match the recurrence order. Rational denominators need a nonzero constant. Algebraic branches require `P(0,y0)=0` and a nonzero partial derivative with respect to `y`. Registries reject duplicate names.

Guarantees (E, P, B, K). Each call constructs an exact finite `FormalSeries`. Rational, recurrence, hypergeometric, algebraic, Euler product, and Bell builders certify only the generated finite prefix. Runtime is polynomial in the requested degree for the implemented recurrences, but exact coefficient growth can dominate memory and arithmetic cost.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test is `tests/test_generating_foundation.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 21 through 24.

C.3A · rational_gf_from_recurrence

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.rational_gf_from_recurrence`. Reflected implementation `komorebi_math.generating.closed_forms.rational_gf_from_recurrence`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-FOUND-INFERENCE`.

Manifest signature (s).

```
(recurrence: 'LinearRecurrenceGuess', initial: 'Sequence[int | Fraction]') -> 'tuple[tuple[Fraction, ...],
   ↪  tuple[Fraction, ...]]'
```

Definition. Return numerator and denominator of the ordinary rational GF.

Contract (D, C, H, R, X). All guessers consume a finite exact prefix and search only within the declared order and degree bounds. A holdout separates fitted relations from later checks when enough data are available. Failure to find a candidate returns `None`. Guess records preserve the fit count, holdout count, and exact prefix status. Symbolic expression methods and characteristic forms require the optional SymPy backend.

Guarantees (E, P, B, K). Linear systems and residual checks use exact rational or integer arithmetic. A returned candidate is evidence for the supplied finite prefix, not a proof that the relation holds for every index. Search cost grows rapidly with the product of recurrence order, polynomial degree, and algebraic bidegree, so the explicit bounds are part of the contract.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test is `tests/test_generating_foundation.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 22, 24, and 26.

C.3A · register_generating_codecs

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.register_generating_codecs`. Reflected implementation `komorebi_math.generating.portable.register_generating_codecs`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-FOUND-PORTABLE`.

Manifest signature (s).

```
() -> 'None'
```

Definition. Register the trusted data-only generating-function codecs.

Contract (D, C, H, R, X). A frozen function requires nonempty names, a nonempty coefficient prefix, normalization of equal length beginning with one, no zero normalization values, and portable metadata. Reconstruction cannot exceed the captured degree. Freezing accepts only `CustomFunctionSpec`. Codec registration installs closed data only decoders and does not permit arbitrary Python object construction.

Guarantees (E, P, B, K). The frozen record preserves exact rational coefficients and basis normalization independently of the original callback. Roundtrip integrity is delegated to the project portable envelope and its digest checks. Serialization cost is linear in the captured prefix, subject to the global payload budget used by the persistence layer.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test is `tests/test_generating_block13.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 21, 61, and 72.

C.3A · root_filter_value

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.root_filter_value`. Reflected implementation `komorebi_math.generating.sections.root_filter_value`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-FOUND-SECTION`.

Manifest signature (s).

```
(basis: 'WeightedMonomialBasis', modulus: 'int', residue: 'int', degree: 'int', z: 'complex') -> 'complex'
```

Definition. Evaluate the finite root filter of `E_gamma`.

Contract (D, C, H, R, X). The truncation degree must be nonnegative. A section requires a positive modulus and a residue in range. The signed option changes the lowering cycle convention but not the address rules. The audit compares one step, a full cycle, and a finite complex root filter at the supplied evaluation point.

Guarantees (E, P, B, K). Series coefficients and lowering operations are exact. The root filter is a finite floating point evaluation and is reported through a residual rather than exact equality. Construction is linear in the truncation degree, while an audit over all residue classes scales with both degree and modulus.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test is `tests/test_generating_foundation.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 22, 24, and 25.

C.3A · series_to_fir_adapter

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.series_to_fir_adapter`. Reflected implementation `komorebi_math.generating.adapters.series_to_fir_adapter`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-FOUND-ADAPTER`.

Manifest signature (s).

```
(series: 'FormalSeries', *, degree: 'int | None' = None, dtype: 'Any' = <class 'float'>) -> 'FIRSeriesAdapter'
```

Definition. Create an explicit formal-series to FIR adapter record.

Contract (D, C, H, R, X). FIR inputs must be a nonempty one dimensional finite real array. Conversion to `FormalSeries` rationalizes taps, optionally limiting denominators. Conversion from a series validates the requested degree and numeric dtype. An adapter requires one tap per stored ordinary coefficient. Frequency responses validate finite sampling parameters and certificates compare the declared coefficient to tap conversion.

Guarantees (E, P, B, K). The formal side is exact before conversion to NumPy. Tap arrays and frequency responses use floating point arithmetic, and their certificates are qualified by a stated tolerance. Conversion is linear in the number of taps. Frequency response evaluation delegates to the signal implementation and scales with the chosen point count.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test is `tests/test_generating_block13.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 21, 31, and 66.

C.3A · series_to_fir_taps

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.series_to_fir_taps`. Reflected implementation `komorebi_math.generating.adapters.series_to_fir_taps`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-FOUND-ADAPTER`.

Manifest signature (s).

```
(series: 'FormalSeries', *, degree: 'int | None' = None, dtype: 'Any' = <class 'float'>) -> 'np.ndarray'
```

Definition. Map ordinary coefficients to causal taps under `z -> z**(-1)`.

Contract (D, C, H, R, X). FIR inputs must be a nonempty one dimensional finite real array. Conversion to `FormalSeries` rationalizes taps, optionally limiting denominators. Conversion from a series validates the requested degree and numeric dtype. An adapter requires one tap per stored ordinary coefficient. Frequency responses validate finite sampling parameters and certificates compare the declared coefficient to tap conversion.

Guarantees (E, P, B, K). The formal side is exact before conversion to NumPy. Tap arrays and frequency responses use floating point arithmetic, and their certificates are qualified by a stated tolerance. Conversion is linear in the number of taps. Frequency response evaluation delegates to the signal implementation and scales with the chosen point count.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test is `tests/test_generating_block13.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 21, 31, and 66.

C.3A · signed_euler_numbers

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.signed_euler_numbers`. Reflected implementation `komorebi_math.generating.sequences.signed_euler_numbers`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-FOUND-SEQUENCE`.

Manifest signature (s).

```
(count: 'int') -> 'tuple[int, ...]'
```

Definition. Return coefficients `E_n` of `sech(z)=sum E_n z^n/n!`.

Contract (D, C, H, R, X). The requested count must be a nonnegative integer. Factorial power parameters must satisfy the positive order required by the family. Weighted Bell constructors accept either a finite weight source or a callback and require enough weights for the requested prefix. Returned tuples use the convention stated by each sequence, including `B_1=-1/2` and Fibonacci beginning at zero.

Guarantees (E, P, B, K). The functions return deterministic exact integer or rational prefixes. They establish the implemented recurrence or finite sum for those indices only. Most recurrences use quadratic work in the prefix length, while Apéry and weighted Bell sums can incur larger exact integer costs.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused tests are `tests/test_generating_foundation.py` and `tests/test_arithmetic_euler.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 22, 24, and 26.

C.3A · weighted_factorial_power_bell_function

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.weighted_factorial_power_bell_function`. Reflected implementation `komorebi_math.generating.functions.weighted_factorial_power_bell_function`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-FOUND-FUNCTION`.

Manifest signature (s).

```
(power: 'int', weights: 'tuple[int | Fraction, ...] | Callable[[int], int | Fraction]', *, name: 'str | None' =
    ↪ None) -> 'CustomFunctionSpec'
```

Definition. Create a live custom block-weighted factorial-power Bell family.

Contract (D, C, H, R, X). A custom function has a nonempty name, an explicit normalized basis, a finite coefficient builder, a source kind, and portable metadata where required. Builders must return exactly `degree+1` coefficients. Recurrence seeds must match the recurrence order. Rational denominators need a nonzero constant. Algebraic branches require `P(0,y0)=0` and a nonzero partial derivative with respect to `y`. Registries reject duplicate names.

Guarantees (E, P, B, K). Each call constructs an exact finite `FormalSeries`. Rational, recurrence, hypergeometric, algebraic, Euler product, and Bell builders certify only the generated finite prefix. Runtime is polynomial in the requested degree for the implemented recurrences, but exact coefficient growth can dominate memory and arithmetic cost.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test is `tests/test_generating_foundation.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 21 through 24.

C.3A · weighted_generalized_bell_numbers

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.weighted_generalized_bell_numbers`. Reflected implementation `komorebi_math.generating.sequences.weighted_generalized_bell_numbers`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-FOUND-SEQUENCE`.

Manifest signature (s).

```
(power: 'int', count: 'int', weights: 'Iterable[int | Fraction] | Callable[[int], int | Fraction]') ->
   ↪ 'tuple[Fraction, ...]'
```

Definition. Evaluate the custom factorial-power Bell polynomials at `x=1`.

Contract (D, C, H, R, X). The requested count must be a nonnegative integer. Factorial power parameters must satisfy the positive order required by the family. Weighted Bell constructors accept either a finite weight source or a callback and require enough weights for the requested prefix. Returned tuples use the convention stated by each sequence, including `B_1=-1/2` and Fibonacci beginning at zero.

Guarantees (E, P, B, K). The functions return deterministic exact integer or rational prefixes. They establish the implemented recurrence or finite sum for those indices only. Most recurrences use quadratic work in the prefix length, while Apéry and weighted Bell sums can incur larger exact integer costs.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test is `tests/test_generating_foundation.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 22, 24, and 26.

C.3A · weighted_generalized_bell_polynomials

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.weighted_generalized_bell_polynomials`. Reflected implementation `komorebi_math.generating.sequences.weighted_generalized_bell_polynomials`. In the reference manifest, this symbol is exported only from the generating namespace. The `komorebi_math.generating` route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-FOUND-SEQUENCE`.

Manifest signature (s).

```
(power: 'int', count: 'int', weights: 'Iterable[int | Fraction] | Callable[[int], int | Fraction]') ->
   ↪ 'tuple[tuple[Fraction, ...], ...]'
```

Definition. Return factorial-power Bell polynomials with custom block weights.

Contract (D, C, H, R, X). The requested count must be a nonnegative integer. Factorial power parameters must satisfy the positive order required by the family. Weighted Bell constructors accept either a finite weight source or a callback and require enough weights for the requested prefix. Returned tuples use the convention stated by each sequence, including `B_1=-1/2` and Fibonacci beginning at zero.

Guarantees (E, P, B, K). The functions return deterministic exact integer or rational prefixes. They establish the implemented recurrence or finite sum for those indices only. Most recurrences use quadratic work in the prefix length, while Apéry and weighted Bell sums can incur larger exact integer costs.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test is `tests/test_generating_foundation.py`. Together with the independent verifier for generating-function foundations, they exercise the applicable construction, exact identity, roundtrip, failure, optional backend, or resource boundary. The source validator for generating-function foundations checks the exact signature, implementation identity, facade routes, card counts, and unique anchor. See Chapters 22, 24, and 26.

C.3B Arithmetic masks, automata, and prime-power compilers: 101 entries

This section closes the public `komorebi_math.generating` namespace. It records the arithmetic layer built above the normalized series contracts in C.3A. The layer combines local Mahler masks, finite carry plans, congruence audits, Cartier automata, Bell digit reductions, p-adic unit arithmetic, q-Kummer decompositions, normalized kernels, cyclotomic collapse, and digit-kernel convolution. Every contract remains finite and explicit about the range, modulus, truncation, state space, or resource budget that it certifies.

Contract family	Cards
Mahler masks and carry compilers	27
Finite congruence audits	16
Cartier automata and linear representations	14
Bell digit reductions	10
Prime-power unit arithmetic	8
q-Kummer decompositions	8
Normalized digit kernels	6
Cyclotomic collapse	6
Digit-kernel algebra	6
Total	**101**

The exact and finite boundary is central. Integer, rational, finite-field, and prime-power computations are exact in their declared domains. Analytic Mahler products attach a tail bound. Symbolic cyclotomic checks require the optional SymPy backend. Congruence audits, recurrence searches, and compiled finite-state certificates establish only the stored range unless a separate theorem extends the result.

C.3B · `AnalyticMaskApproximation`

Identity (`q, a, t, k`). Public research manifest route `komorebi_math.generating.AnalyticMaskApproximation`. Reflected implementation `komorebi_math.generating.masks.AnalyticMaskApproximation`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `class`. Profile `C3-ARITH-MASK`.

Manifest signature (`s`).

```
(radix: 'int', point: 'complex', terms: 'int', value: 'complex', relative_tail_bound: 'float',
   ↪ absolute_error_bound: 'float') -> None
```

Definition. Finite analytic Mahler product with a conservative tail bound.

Contract (`D, C, H, R, X`). Inputs describe a finite local mask, a radix of at least two, and an explicit truncation, modulus, exponent, or resource budget as required by the operation. Unit-series routines require a valid constant coefficient. Prime-power support routines require a prime and positive exponent. Carry plans validate every state and local index before allocation. Bounded solvers return a negative result when the declared search space contains no solution. The principal public operations are `certificate`.

Guarantees (`E, P, B, K`). Finite coefficient identities, carry matrices, support layers, and polynomial certificates use exact integer or rational arithmetic. Analytic approximations separate the computed product from a stated tail bound. A finite functional equation or algebraic certificate applies only through its recorded degree. Direct mask products are at least quadratic in the truncation boundary, while carry-state plans depend on the explicit state dimension and budget.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_generating_block13.py` and `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27 and 64.

C.3B · `AutomatonCertificate`

Identity (`q, a, t, k`). Public research manifest route `komorebi_math.generating.AutomatonCertificate`. Reflected implementation `komorebi_math.generating.cartier.AutomatonCertificate`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `class`. Profile `C3-ARITH-CARTIER`.

Manifest signature (`s`).

```
(compared_terms: 'int', failures: 'tuple[tuple[int, int, int], ...]') -> None
```

Definition. Finite comparison between compiled and recurrence coefficients.

Contract (`D, C, H, R, X`). Polynomial operations require a modulus of at least two. Cartier operators and rational-mask compilers require a prime, a valid digit, compatible nonzero constant coefficients, and explicit state or degree budgets. Automata read least-significant-first prime digits. Minimization preserves output behavior on the compiled finite-state machine. Certificate routines reject mismatched transition systems or coefficient prefixes. The principal public operations are `valid`, `require`.

Guarantees (`E, P, B, K`). Polynomial arithmetic, Cartier transitions, automaton outputs, and linear representations are exact modulo the selected prime. State bounds are finite compilation bounds rather than asymptotic minimality claims. Rational-mask coefficient generation and automaton evaluation agree on the checked prefix. Complexity is polynomial in polynomial degree and compiled state count, with minimization depending on partition refinement over the full transition table.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27.

C.3B · `BellDigitReduction`

Identity (`q, a, t, k`). Public research manifest route `komorebi_math.generating.BellDigitReduction`. Reflected implementation `komorebi_math.generating.bell.BellDigitReduction`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `class`. Profile `C3-ARITH-BELL`.

Manifest signature (`s`).

```
(index: 'int', prime: 'int', factorial_power: 'int', digit_sum: 'int', high_polynomial: 'tuple[int, ...]',
    ↪ reduced_polynomial: 'tuple[int, ...]') -> None
```

Definition. One coefficientwise factorial-power Bell digit-sum reduction.

Contract (`D, C, H, R, X`). Bell reductions require a prime, a nonnegative index, and valid factorial-power or weight data. Automata consume base-prime digits in the order stated by each method. Period bounds and pointed-subindex identities are finite arithmetic contracts. Weighted reductions require enough local weights for the requested computation. Resource budgets cap explicit polynomial and state construction. The principal public operations are `valid`, `require`, `certificate`.

Guarantees (`E, P, B, K`). The returned reductions, parity values, residue automata, and pointed polynomial certificates are exact modulo the declared prime. A certificate records the finite comparison it performed. The period bound is an upper bound, not a minimal-period claim. Direct polynomial construction can grow with the digit sum or index, while automaton evaluation is linear in the number of input digits.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_generating_foundation.py` and `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 22, 24, and 26.

C.3B · `BellParityAutomaton`

Identity (`q, a, t, k`). Public research manifest route `komorebi_math.generating.BellParityAutomaton`. Reflected implementation `komorebi_math.generating.bell.BellParityAutomaton`. The same runtime object is also exported as `komorebi_math.BellParityAutomaton`. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `class`. Profile `C3-ARITH-BELL`.

Manifest signature (`s`).

```
(outputs: 'tuple[int, int, int]' = (1, 1, 0)) -> None
```

Definition. The three-state binary automaton for generalized Bell parity.

Contract (`D, C, H, R, X`). Bell reductions require a prime, a nonnegative index, and valid factorial-power or weight data. Automata consume base-prime digits in the order stated by each method. Period bounds and pointed-subindex identities are finite arithmetic contracts. Weighted reductions require enough local weights for the requested computation. Resource budgets cap explicit polynomial and state construction. The principal public operations are `transition`, `evaluate`.

Guarantees (`E, P, B, K`). The returned reductions, parity values, residue automata, and pointed polynomial certificates are exact modulo the declared prime. A certificate records the finite comparison it performed. The period bound is an upper bound, not a minimal-period claim. Direct polynomial construction can grow with the digit sum or index, while automaton evaluation is linear in the number of input digits.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_generating_foundation.py` and `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 22, 24, and 26.

C.3B · `BellResidueAutomaton`

Identity (`q, a, t, k`). Public research manifest route `komorebi_math.generating.BellResidueAutomaton`. Reflected implementation `komorebi_math.generating.bell.BellResidueAutomaton`. The same runtime object is also exported as `komorebi_math.BellResidueAutomaton`. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `class`. Profile `C3-ARITH-BELL`.

Manifest signature (`s`).

```
(prime: 'int', factorial_power: 'int', period_bound: 'int', outputs: 'tuple[int, ...]') -> None
```

Definition. A finite digit-sum automaton for factorial-power Bell residues.

Contract (`D, C, H, R, X`). Bell reductions require a prime, a nonnegative index, and valid factorial-power or weight data. Automata consume base-prime digits in the order stated by each method. Period bounds and pointed-subindex identities are finite arithmetic contracts. Weighted reductions require enough local weights for the requested computation. Resource budgets cap explicit polynomial and state construction. The principal public operations are `compile`, `transition`, `evaluate`, `certificate`.

Guarantees (`E, P, B, K`). The returned reductions, parity values, residue automata, and pointed polynomial certificates are exact modulo the declared prime. A certificate records the finite comparison it performed. The period bound is an upper bound, not a minimal-period claim. Direct polynomial construction can grow with the digit sum or index, while automaton evaluation is linear in the number of input digits.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_generating_foundation.py` and `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 22, 24, and 26.

C.3B · `CarryLayerConvolution`

Identity (`q, a, t, k`). Public research manifest route `komorebi_math.generating.CarryLayerConvolution`. Reflected implementation `komorebi_math.generating.masks.CarryLayerConvolution`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `class`. Profile `C3-ARITH-MASK`.

Manifest signature (`s`).

```
(index: 'int', prime: 'int', exponent: 'int', factorial_power: 'int', modulus: 'int', layer_sums: 'tuple[int,
   ↪  ...]') -> None
```

Definition. Prime-power convolution decomposed by exact borrow count.

Contract (`D, C, H, R, X`). Inputs describe a finite local mask, a radix of at least two, and an explicit truncation, modulus, exponent, or resource budget as required by the operation. Unit-series routines require a valid constant coefficient. Prime-power support routines require a prime and positive exponent. Carry plans validate every state and local index before allocation. Bounded solvers return a negative result when the declared search space contains no solution. The principal public operations are `residue`.

Guarantees (`E, P, B, K`). Finite coefficient identities, carry matrices, support layers, and polynomial certificates use exact integer or rational arithmetic. Analytic approximations separate the computed product from a stated tail bound. A finite functional equation or algebraic certificate applies only through its recorded degree. Direct mask products are at least quadratic in the truncation boundary, while carry-state plans depend on the explicit state dimension and budget.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_generating_block13.py` and `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27 and 64.

C.3B · `CarryLinearCertificate`

Identity (`q, a, t, k`). Public research manifest route `komorebi_math.generating.CarryLinearCertificate`. Reflected implementation `komorebi_math.generating.normalization.CarryLinearCertificate`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `class`. Profile `C3-ARITH-NORMALIZATION`.

Manifest signature (`s`).

```
(prime: 'int', linear_slope: 'int', carry_weight: 'int', checked_through: 'int', failures: 'tuple[tuple[int, int,
   ↪  int, int], ...]') -> None
```

Definition. Finite audit of the carry-linear valuation classification.

Contract (`D, C, H, R, X`). Digit-unit and normalized-kernel routines require a prime, nonnegative indices, and unit data compatible with the selected digit system. Carry-linear classification requires a callback with zero value at zero and a finite audit range large enough to recover the slope and carry weight. Invalid unit tables, primes, or audit ranges are rejected. The principal public operations are `valid`, `require`.

Guarantees (`E, P, B, K`). Local and digit-factorized normalized kernels are exact modulo the selected prime power. CarryLinearCertificate records the recovered profile and every finite mismatch. Passing the finite audit confirms the supplied range only. Kernel evaluation scales with the number of base-prime digits, while classification scales with the requested audit boundary.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 21, 24, and 26.

C.3B · `CarryLinearValuation`

Identity (`q, a, t, k`). Public research manifest route `komorebi_math.generating.CarryLinearValuation`. Reflected implementation `komorebi_math.generating.normalization.CarryLinearValuation`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `class`. Profile `C3-ARITH-NORMALIZATION`.

Manifest signature (`s`).

```
(prime: 'int', linear_slope: 'int', carry_weight: 'int') -> None
```

Definition. Profile `nu(n)=a*n+s*v_p(n!)` with defect equal to weighted carries.

Contract (`D, C, H, R, X`). Digit-unit and normalized-kernel routines require a prime, nonnegative indices, and unit data compatible with the selected digit system. Carry-linear classification requires a callback with zero value at zero and a finite audit range large enough to recover the slope and carry weight. Invalid unit tables, primes, or audit ranges are rejected. The principal public operations are `value`, `defect`.

Guarantees (`E, P, B, K`). Local and digit-factorized normalized kernels are exact modulo the selected prime power. CarryLinearCertificate records the recovered profile and every finite mismatch. Passing the finite audit confirms the supplied range only. Kernel evaluation scales with the number of base-prime digits, while classification scales with the requested audit boundary.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 21, 24, and 26.

C.3B · `CartierAutomaton`

Identity (`q, a, t, k`). Public research manifest route `komorebi_math.generating.CartierAutomaton`. Reflected implementation `komorebi_math.generating.cartier.CartierAutomaton`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `class`. Profile `C3-ARITH-CARTIER`.

Manifest signature (`s`).

```
(prime: 'int', numerator: 'tuple[int, ...]', denominator: 'tuple[int, ...]', degree_bound: 'int', transitions:
   ↪  'tuple[tuple[int, ...], ...]', outputs: 'tuple[int, ...]', state_numerators: 'tuple[tuple[int, ...], ...]',
   ↪  state_members: 'tuple[tuple[int, ...], ...]', start_state: 'int' = 0) -> None
```

Definition. Finite output automaton for a rational-mask Mahler series modulo a prime.

Contract (`D, C, H, R, X`). Polynomial operations require a modulus of at least two. Cartier operators and rational-mask compilers require a prime, a valid digit, compatible nonzero constant coefficients, and explicit state or degree budgets. Automata read least-significant-first prime digits. Minimization preserves output behavior on the compiled finite-state machine. Certificate routines reject mismatched transition systems or coefficient prefixes. The principal public operations are `number_states`, `theoretical_state_bound`, `evaluate`, `sequence`, `certificate`.

Guarantees (`E, P, B, K`). Polynomial arithmetic, Cartier transitions, automaton outputs, and linear representations are exact modulo the selected prime. State bounds are finite compilation bounds rather than asymptotic minimality claims. Rational-mask coefficient generation and automaton evaluation agree on the checked prefix. Complexity is polynomial in polynomial degree and compiled state count, with minimization depending on partition refinement over the full transition table.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27.

C.3B · CartierCarryEquivalence

Identity (q, a, t, k). Public research manifest route komorebi_math.generating.CartierCarryEquivalence. Reflected implementation komorebi_math.generating.cartier.CartierCarryEquivalence. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by [RESEARCH] policy pre-beta-additive. Manifest kind class. Profile C3-ARITH-CARTIER.

Manifest signature (s).

```
(prime: 'int', dimension: 'int', failures: 'tuple[tuple[int, int, int, int, int], ...]') -> None
```

Definition. Finite certificate of Cartier/carry matrix equality.

Contract (D, C, H, R, X). Polynomial operations require a modulus of at least two. Cartier operators and rational-mask compilers require a prime, a valid digit, compatible nonzero constant coefficients, and explicit state or degree budgets. Automata read least-significant-first prime digits. Minimization preserves output behavior on the compiled finite-state machine. Certificate routines reject mismatched transition systems or coefficient prefixes. The principal public operations are valid, require.

Guarantees (E, P, B, K). Polynomial arithmetic, Cartier transitions, automaton outputs, and linear representations are exact modulo the selected prime. State bounds are finite compilation bounds rather than asymptotic minimality claims. Rational-mask coefficient generation and automaton evaluation agree on the checked prefix. Complexity is polynomial in polynomial degree and compiled state count, with minimization depending on partition refinement over the full transition table.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is tests/test_generating_cartier_padic.py. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27.

C.3B · CartierLinearRepresentation

Identity (q, a, t, k). Public research manifest route komorebi_math.generating.CartierLinearRepresentation. Reflected implementation komorebi_math.generating.cartier.CartierLinearRepresentation. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by [RESEARCH] policy pre-beta-additive. Manifest kind class. Profile C3-ARITH-CARTIER.

Manifest signature (s).

```
(prime: 'int', numerator: 'tuple[int, ...]', denominator: 'tuple[int, ...]', degree_bound: 'int',
    ↪ transition_matrices: 'tuple[tuple[tuple[int, ...], ...], ...]', start_vector: 'tuple[int, ...]',
    ↪ output_scale: 'int') -> None
```

Definition. Bounded-dimensional linear representation of a rational-mask series.

Contract (D, C, H, R, X). Polynomial operations require a modulus of at least two. Cartier operators and rational-mask compilers require a prime, a valid digit, compatible nonzero constant coefficients, and explicit state or degree budgets. Automata read least-significant-first prime digits. Minimization preserves output behavior on the compiled finite-state machine. Certificate routines reject mismatched transition systems or coefficient prefixes. The principal public operations are dimension, transition, evaluate, sequence.

Guarantees (E, P, B, K). Polynomial arithmetic, Cartier transitions, automaton outputs, and linear representations are exact modulo the selected prime. State bounds are finite compilation bounds rather than asymptotic minimality claims. Rational-mask coefficient generation and automaton evaluation agree on the checked prefix. Complexity is polynomial in polynomial degree and compiled state count, with minimization depending on partition refinement over the full transition table.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is tests/test_generating_cartier_padic.py. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime owner-

ship. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27.

C.3B · `CyclotomicCollapse`

Identity (`q, a, t, k`). Public research manifest route `komorebi_math.generating.CyclotomicCollapse`. Reflected implementation `komorebi_math.generating.cyclotomic.CyclotomicCollapse`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `class`. Profile `C3-ARITH-CYCLOTOMIC`.

Manifest signature (`s`).

```
(prime: 'int', mask_exponents: 'tuple[tuple[int, int], ...]', closed_form_exponents: 'tuple[tuple[int, int],
   ↪  ...]', obstruction_orbits: 'tuple[int, ...]') -> None
```

Definition. Exact finite cyclotomic coboundary analysis.

Contract (`D, C, H, R, X`). Cyclotomic routines require a prime and positive cyclotomic indices. Exponent maps omit zero entries after normalization. Symbolic expression and verification functions require the optional SymPy backend. A collapse result retains every p-free obstruction orbit and raises when the caller demands a closed form that is not present. The principal public operations are `collapses`, `require`.

Guarantees (`E, P, B, K`). Orbit decomposition and exponent-lattice relations are exact. When no obstruction remains, the returned finite cyclotomic quotient is an exact formal collapse of the local Mahler product. Symbolic verification checks the declared relation rather than discovering a new one. Runtime is linear in the number of represented factors apart from optional symbolic simplification.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_generating_cartier_padic.py` and `tests/test_generating_block13.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 26.

C.3B · `DigitAlgebraAudit`

Identity (`q, a, t, k`). Public research manifest route `komorebi_math.generating.DigitAlgebraAudit`. Reflected implementation `komorebi_math.generating.digit_algebra.DigitAlgebraAudit`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `class`. Profile `C3-ARITH-DIGIT`.

Manifest signature (`s`).

```
(radix: 'int', modulus: 'int', power: 'int', tested_weights: 'int', weight_failures: 'int', tested_coefficients:
   ↪  'int', closure_failures: 'int', local_product: 'tuple[int, ...]') -> None
```

Definition. Finite evidence for digitwise factorization and convolution closure.

Contract (`D, C, H, R, X`). Digit kernels require a radix and modulus of at least two and one lower-triangular row for each digit. Global weights return zero when subtraction would borrow. Lifted sequences require one local value per digit. Factorial-power audits require a prime, positive power, and an explicit digit depth.

Guarantees (`E, P, B, K`). Local and global kernel weights, lifted sequences, and convolution audits are exact modulo the declared modulus. A finite DigitAlgebraAudit records the checked range and all failures. Digitwise evaluation is linear in the radix-digit width. Full closure audits scale with the size of the finite range and the number of tested subindices.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_generating_foundation.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 26.

C.3B · DigitCauchyProductPlan

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.DigitCauchyProductPlan`. Reflected implementation `komorebi_math.generating.masks.DigitCauchyProductPlan`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `class`. Profile `C3-ARITH-MASK`.

Manifest signature (s).

```
(radix: 'int', digit_weights: 'tuple[tuple[Fraction, ...], ...]', matrices: 'tuple[Matrix, ...]') -> None
```

Definition. Finite carry representation of an ordinary product of digit-local series.

Contract (D, C, H, R, X). Inputs describe a finite local mask, a radix of at least two, and an explicit truncation, modulus, exponent, or resource budget as required by the operation. Unit-series routines require a valid constant coefficient. Prime-power support routines require a prime and positive exponent. Carry plans validate every state and local index before allocation. Bounded solvers return a negative result when the declared search space contains no solution. The principal public operations are `factor_count`, `dimension`, `evaluate`, `sequence`.

Guarantees (E, P, B, K). Finite coefficient identities, carry matrices, support layers, and polynomial certificates use exact integer or rational arithmetic. Analytic approximations separate the computed product from a stated tail bound. A finite functional equation or algebraic certificate applies only through its recorded degree. Direct mask products are at least quadratic in the truncation boundary, while carry-state plans depend on the explicit state dimension and budget.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_generating_block13.py` and `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27 and 64.

C.3B · DigitKernel

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.DigitKernel`. Reflected implementation `komorebi_math.generating.digit_algebra.DigitKernel`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `class`. Profile `C3-ARITH-DIGIT`.

Manifest signature (s).

```
(radix: 'int', modulus: 'int', table: 'tuple[tuple[int, ...], ...]', name: 'str' = 'digit kernel') -> None
```

Definition. A lower-triangular local convolution table modulo an integer. For digit `d` and subdigit `e`, `table[d][e]` is the local weight. The global no-borrow weight is the product of these local weights across radix digits. Entries above the diagonal are represented implicitly by zero.

Contract (D, C, H, R, X). Digit kernels require a radix and modulus of at least two and one lower-triangular row for each digit. Global weights return zero when subtraction would borrow. Lifted sequences require one local value per digit. Factorial-power audits require a prime, positive power, and an explicit digit depth. The principal public operations are `local_weight`, `global_weight`, `lift`, `local_multiply`, `global_multiply`.

Guarantees (E, P, B, K). Local and global kernel weights, lifted sequences, and convolution audits are exact modulo the declared modulus. A finite DigitAlgebraAudit records the checked range and all failures. Digitwise evaluation is linear in the radix-digit width. Full closure audits scale with the size of the finite range and the number of tested subindices.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_generating_foundation.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 26.

C.3B · `EulerProductCollapse`

Identity (`q, a, t, k`). Public research manifest route `komorebi_math.generating.EulerProductCollapse`. Reflected implementation `komorebi_math.generating.masks.EulerProductCollapse`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `class`. Profile `C3-ARITH-MASK`.

Manifest signature (`s`).

```
(radix: 'int', mask_exponents: 'tuple[tuple[int, int], ...]', closed_form_exponents: 'tuple[tuple[int, int],
   ↪  ...]', obstruction_orbits: 'tuple[int, ...]') -> None
```

Definition. Orbitwise finite-collapse data for an Euler-product local mask.

Contract (`D, C, H, R, X`). Inputs describe a finite local mask, a radix of at least two, and an explicit truncation, modulus, exponent, or resource budget as required by the operation. Unit-series routines require a valid constant coefficient. Prime-power support routines require a prime and positive exponent. Carry plans validate every state and local index before allocation. Bounded solvers return a negative result when the declared search space contains no solution. The principal public operations are `collapses`, `require`.

Guarantees (`E, P, B, K`). Finite coefficient identities, carry matrices, support layers, and polynomial certificates use exact integer or rational arithmetic. Analytic approximations separate the computed product from a stated tail bound. A finite functional equation or algebraic certificate applies only through its recorded degree. Direct mask products are at least quadratic in the truncation boundary, while carry-state plans depend on the explicit state dimension and budget.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_generating_block13.py` and `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27 and 64.

C.3B · `FiniteMaskCarryPlan`

Identity (`q, a, t, k`). Public research manifest route `komorebi_math.generating.FiniteMaskCarryPlan`. Reflected implementation `komorebi_math.generating.masks.FiniteMaskCarryPlan`. The same runtime object is also exported as `komorebi_math.FiniteMaskCarryPlan`. Within the generating namespace, `komorebi_math.generating.NormalizedMaskPlan` resolves to the same runtime object. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `class`. Profile `C3-ARITH-MASK`.

Manifest signature (`s`).

```
(radix: 'int', mask: 'tuple[Fraction, ...]', carry_bound: 'int', matrices: 'tuple[Matrix, ...]') -> None
```

Definition. Exact carry-matrix representation of a finite local mask.

Contract (`D, C, H, R, X`). Inputs describe a finite local mask, a radix of at least two, and an explicit truncation, modulus, exponent, or resource budget as required by the operation. Unit-series routines require a valid constant coefficient. Prime-power support routines require a prime and positive exponent. Carry plans validate every state and local index before allocation. Bounded solvers return a negative result when the declared search space contains no solution. The principal public operations are `dimension`, `evaluate`, `sequence`, `certificate`.

Guarantees (`E, P, B, K`). Finite coefficient identities, carry matrices, support layers, and polynomial certificates use exact integer or rational arithmetic. Analytic approximations separate the computed product from a stated tail bound. A finite functional equation or algebraic certificate applies only through its recorded degree. Direct mask products are at least quadratic in the truncation boundary, while carry-state plans depend on the explicit state dimension and budget.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_generating_block13.py` and `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite cer-

tificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27 and 64.

C.3B · KummerLayerDistribution

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.KummerLayerDistribution`. Reflected implementation `komorebi_math.generating.masks.KummerLayerDistribution`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `class`. Profile `C3-ARITH-MASK`.

Manifest signature (s).

```
(index: 'int', prime: 'int', counts: 'tuple[int, ...]') -> None
```

Definition. Counts of subindices grouped by the number of base-prime borrows.

Contract (D, C, H, R, X). Inputs describe a finite local mask, a radix of at least two, and an explicit truncation, modulus, exponent, or resource budget as required by the operation. Unit-series routines require a valid constant coefficient. Prime-power support routines require a prime and positive exponent. Carry plans validate every state and local index before allocation. Bounded solvers return a negative result when the declared search space contains no solution. The principal public operations are `total`, `count`, `support_count`, `certificate`.

Guarantees (E, P, B, K). Finite coefficient identities, carry matrices, support layers, and polynomial certificates use exact integer or rational arithmetic. Analytic approximations separate the computed product from a stated tail bound. A finite functional equation or algebraic certificate applies only through its recorded degree. Direct mask products are at least quadratic in the truncation boundary, while carry-state plans depend on the explicit state dimension and budget.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_generating_block13.py` and `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27 and 64.

C.3B · NormalizedKernelResidue

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.NormalizedKernelResidue`. Reflected implementation `komorebi_math.generating.padic_units.NormalizedKernelResidue`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `class`. Profile `C3-ARITH-PADIC`.

Manifest signature (s).

```
(index: 'int', subindex: 'int', factorial_power: 'int', prime: 'int', exponent: 'int', valuation: 'int', unit:
   ↪  'int', residue: 'int') -> None
```

Definition. Prime-power residue of a carry-separable normalized-basis kernel.

Contract (D, C, H, R, X). Valuation functions require a prime and valid integer or binomial indices. Prime-power compilers require positive precision and a modulus within the supplied budget. Unit-factorial and binomial methods distinguish valuation from the invertible unit part. Kernel compilers require compatible digit-unit tables and factorial exponents. Borrow counts use nonnegative indices with the subindex not exceeding the index where required. The principal public operations are `survives`.

Guarantees (E, P, B, K). Valuations, unit parts, residues, and survival decisions are exact for the declared prime power. PrimePowerBinomial separates the p-adic valuation from the unit residue, so zero modulo the prime power is represented without discarding valuation information. Precomputation cost depends on the prime-power modulus. Subsequent digit recursions scale with the number of base-prime digits.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the

relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27.

C.3B · `NormalizedMaskPlan`

Identity (`q, a, t, k`). Public research manifest route `komorebi_math.generating.NormalizedMaskPlan`. Reflected implementation `komorebi_math.generating.masks.FiniteMaskCarryPlan`. In the reference manifest, this symbol is exported only from the generating namespace. Within the generating namespace, `komorebi_math.generating.FiniteMaskCarryPlan` resolves to the same runtime object. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `class`. Profile `C3-ARITH-MASK`.

Manifest signature (`s`).

```
(radix: 'int', mask: 'tuple[Fraction, ...]', carry_bound: 'int', matrices: 'tuple[Matrix, ...]') -> None
```

Definition. Exact carry-matrix representation of a finite local mask.

Contract (`D, C, H, R, X`). Inputs describe a finite local mask, a radix of at least two, and an explicit truncation, modulus, exponent, or resource budget as required by the operation. Unit-series routines require a valid constant coefficient. Prime-power support routines require a prime and positive exponent. Carry plans validate every state and local index before allocation. Bounded solvers return a negative result when the declared search space contains no solution. The principal public operations are `dimension`, `evaluate`, `sequence`, `certificate`.

Guarantees (`E, P, B, K`). Finite coefficient identities, carry matrices, support layers, and polynomial certificates use exact integer or rational arithmetic. Analytic approximations separate the computed product from a stated tail bound. A finite functional equation or algebraic certificate applies only through its recorded degree. Direct mask products are at least quadratic in the truncation boundary, while carry-state plans depend on the explicit state dimension and budget.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_generating_block13.py` and `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27 and 64.

C.3B · `PointedSubindexCertificate`

Identity (`q, a, t, k`). Public research manifest route `komorebi_math.generating.PointedSubindexCertificate`. Reflected implementation `komorebi_math.generating.bell.PointedSubindexCertificate`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `class`. Profile `C3-ARITH-BELL`.

Manifest signature (`s`).

```
(index: 'int', prime: 'int', left: 'tuple[int, ...]', right: 'tuple[int, ...]') -> None
```

Definition. Exact finite certificate for the pointed digit identity.

Contract (`D, C, H, R, X`). Bell reductions require a prime, a nonnegative index, and valid factorial-power or weight data. Automata consume base-prime digits in the order stated by each method. Period bounds and pointed-subindex identities are finite arithmetic contracts. Weighted reductions require enough local weights for the requested computation. Resource budgets cap explicit polynomial and state construction. The principal public operations are `valid`, `require`, `certificate`.

Guarantees (`E, P, B, K`). The returned reductions, parity values, residue automata, and pointed polynomial certificates are exact modulo the declared prime. A certificate records the finite comparison it performed. The period bound is an upper bound, not a minimal-period claim. Direct polynomial construction can grow with the digit sum or index, while automaton evaluation is linear in the number of input digits.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_generating_foundation.py` and `tests/test_generating_cartier_padic.py`. The in-

dependent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 22, 24, and 26.

C.3B · PolynomialCoboundaryResult

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.PolynomialCoboundaryResult`. Reflected implementation `komorebi_math.generating.masks.PolynomialCoboundaryResult`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `class`. Profile `C3-ARITH-MASK`.

Manifest signature (s).

```
(radix: 'int', degree_bound: 'int', coefficients: 'tuple[Fraction, ...] | None', residual: 'tuple[Fraction, ...]')
   ↪ -> None
```

Definition. Bounded exact solution of `B Q=A Q(z^radix)` with `Q(0)=1`.

Contract (D, C, H, R, X). Inputs describe a finite local mask, a radix of at least two, and an explicit truncation, modulus, exponent, or resource budget as required by the operation. Unit-series routines require a valid constant coefficient. Prime-power support routines require a prime and positive exponent. Carry plans validate every state and local index before allocation. Bounded solvers return a negative result when the declared search space contains no solution. The principal public operations are `found`, `valid`, `require`, `certificate`.

Guarantees (E, P, B, K). Finite coefficient identities, carry matrices, support layers, and polynomial certificates use exact integer or rational arithmetic. Analytic approximations separate the computed product from a stated tail bound. A finite functional equation or algebraic certificate applies only through its recorded degree. Direct mask products are at least quadratic in the truncation boundary, while carry-state plans depend on the explicit state dimension and budget.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_generating_block13.py` and `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27 and 64.

C.3B · PrimePowerBinomial

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.PrimePowerBinomial`. Reflected implementation `komorebi_math.generating.padic_units.PrimePowerBinomial`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `class`. Profile `C3-ARITH-PADIC`.

Manifest signature (s).

```
(index: 'int', subindex: 'int', prime: 'int', exponent: 'int', modulus: 'int', valuation: 'int', unit: 'int',
   ↪ residue: 'int') -> None
```

Definition. Valuation, unit part, and residue of a binomial coefficient.

Contract (D, C, H, R, X). Valuation functions require a prime and valid integer or binomial indices. Prime-power compilers require positive precision and a modulus within the supplied budget. Unit-factorial and binomial methods distinguish valuation from the invertible unit part. Kernel compilers require compatible digit-unit tables and factorial exponents. Borrow counts use nonnegative indices with the subindex not exceeding the index where required. The principal public operations are `survives`.

Guarantees (E, P, B, K). Valuations, unit parts, residues, and survival decisions are exact for the declared prime power. PrimePowerBinomial separates the p-adic valuation from the unit residue, so zero modulo the prime power is represented without discarding valuation information. Precomputation cost depends on the prime-power modulus. Subsequent digit recursions scale with the number of base-prime digits.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27.

C.3B · PrimePowerKernelCompiler

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.PrimePowerKernelCompiler`. Reflected implementation `komorebi_math.generating.padic_units.PrimePowerKernelCompiler`. The same runtime object is also exported as `komorebi_math.PrimePowerKernelCompiler`. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `class`. Profile `C3-ARITH-PADIC`.

Manifest signature (s).

```
(prime: 'int', exponent: 'int', factorial_power: 'int', unit_sequence: 'Callable[[int], int] | None' = None,
    ↪ max_modulus: 'int' = 2000000) -> None
```

Definition. Compile kernels for `gamma_n=c**n (n!)**s u_n` modulo `p**r`. The geometric factor cancels from every structure constant. The supplied `unit_sequence` must take values coprime to `p`.

Contract (D, C, H, R, X). Valuation functions require a prime and valid integer or binomial indices. Prime-power compilers require positive precision and a modulus within the supplied budget. Unit-factorial and binomial methods distinguish valuation from the invertible unit part. Kernel compilers require compatible digit-unit tables and factorial exponents. Borrow counts use nonnegative indices with the subindex not exceeding the index where required. The principal public operations are `modulus`, `kernel`.

Guarantees (E, P, B, K). Valuations, unit parts, residues, and survival decisions are exact for the declared prime power. PrimePowerBinomial separates the p-adic valuation from the unit residue, so zero modulo the prime power is represented without discarding valuation information. Precomputation cost depends on the prime-power modulus. Subsequent digit recursions scale with the number of base-prime digits.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27.

C.3B · PrimePowerUnitCompiler

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.PrimePowerUnitCompiler`. Reflected implementation `komorebi_math.generating.padic_units.PrimePowerUnitCompiler`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `class`. Profile `C3-ARITH-PADIC`.

Manifest signature (s).

```
(prime: 'int', exponent: 'int', max_modulus: 'int' = 2000000) -> None
```

Definition. Compile p-free factorials and binomial residues modulo `p**r`. The prefix table costs `O(p**r)` memory. Each query then uses `O(log_p(n))` iterations, with memoization of completed unit factorials.

Contract (D, C, H, R, X). Valuation functions require a prime and valid integer or binomial indices. Prime-power compilers require positive precision and a modulus within the supplied budget. Unit-factorial and binomial methods distinguish valuation from the invertible unit part. Kernel compilers require compatible digit-unit tables and factorial exponents. Borrow counts use nonnegative indices with the subindex not exceeding the index where required. The principal public operations are `unit_factorial`, `binomial_data`, `binomial`.

Guarantees (E, P, B, K). Valuations, unit parts, residues, and survival decisions are exact for the declared prime power. PrimePowerBinomial separates the p-adic valuation from the unit residue, so zero modulo the prime power is rep-

resented without discarding valuation information. Precomputation cost depends on the prime-power modulus. Subsequent digit recursions scale with the number of base-prime digits.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27.

C.3B · `QKummerDecomposition`

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.QKummerDecomposition`. Reflected implementation `komorebi_math.generating.q_kummer.QKummerDecomposition`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `class`. Profile `C3-ARITH-QKUMMER`.

Manifest signature (s).

```
(index: 'int', subindex: 'int', base: 'int', prime: 'int', order: 'int', lift_constant: 'int', quotient_index:
   ↪ 'int', quotient_subindex: 'int', residue_carry: 'int', quotient_carries: 'int', endpoint_valuation: 'int',
   ↪ valuation: 'int') -> None
```

Definition. Exact mixed-radix valuation decomposition for a Gaussian binomial.

Contract (D, C, H, R, X). The q-analogue routines require nonnegative indices, a valid subindex, an integer base, and a prime that does not violate the stated denominator or multiplicative-order conditions. Valuation decomposition requires a positive power and exponent where present. Support enumeration is finite and may be bounded by an explicit resource contract. The principal public operations are `survives`.

Guarantees (E, P, B, K). q-integers, q-factorials, Gaussian binomials, direct valuations, mixed-radix decompositions, and support sets are computed with exact integers. The decomposition records all factors used to reconstruct the valuation. Large Gaussian binomials can grow rapidly in bit length, so direct verification is bounded even when the structural decomposition remains compact.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27.

C.3B · `RationalMaskAlgebraicCertificate`

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.RationalMaskAlgebraicCertificate`. Reflected implementation `komorebi_math.generating.masks.RationalMaskAlgebraicCertificate`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `class`. Profile `C3-ARITH-MASK`.

Manifest signature (s).

```
(prime: 'int', degree: 'int', failures: 'tuple[tuple[int, int], ...]') -> None
```

Definition. Finite exact certificate for `A F^(p-1)=B` modulo a prime.

Contract (D, C, H, R, X). Inputs describe a finite local mask, a radix of at least two, and an explicit truncation, modulus, exponent, or resource budget as required by the operation. Unit-series routines require a valid constant coefficient. Prime-power support routines require a prime and positive exponent. Carry plans validate every state and local index before allocation. Bounded solvers return a negative result when the declared search space contains no solution. The principal public operations are `valid`, `require`, `certificate`.

Guarantees (E, P, B, K). Finite coefficient identities, carry matrices, support layers, and polynomial certificates use exact integer or rational arithmetic. Analytic approximations separate the computed product from a stated tail bound. A finite functional equation or algebraic certificate applies only through its recorded degree. Direct mask products

are at least quadratic in the truncation boundary, while carry-state plans depend on the explicit state dimension and budget.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_generating_block13.py` and `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27 and 64.

C.3B · `analytic_mahler_product`

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.analytic_mahler_product`. Reflected implementation `komorebi_math.generating.masks.analytic_mahler_product`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-MASK`.

Manifest signature (s).

```
(mask: 'Sequence[int | float | complex | Fraction]', point: 'complex', radix: 'int', terms: 'int') ->
    ↪ 'AnalyticMaskApproximation'
```

Definition. Evaluate a finite analytic scale product with an explicit tail bound. For `R(z)=1+sum_(k>=1) r_k z^k` and `|z|<1`, the implementation uses `|R(w)-1| <= C|w|` with `C=sum_(k>=1)|r_k|`. The resulting bound is conservative. It is numerical rather than interval-rigorous because the product itself is evaluated with ordinary complex floating arithmetic.

Contract (D, C, H, R, X). Inputs describe a finite local mask, a radix of at least two, and an explicit truncation, modulus, exponent, or resource budget as required by the operation. Unit-series routines require a valid constant coefficient. Prime-power support routines require a prime and positive exponent. Carry plans validate every state and local index before allocation. Bounded solvers return a negative result when the declared search space contains no solution.

Guarantees (E, P, B, K). Finite coefficient identities, carry matrices, support layers, and polynomial certificates use exact integer or rational arithmetic. Analytic approximations separate the computed product from a stated tail bound. A finite functional equation or algebraic certificate applies only through its recorded degree. Direct mask products are at least quadratic in the truncation boundary, while carry-state plans depend on the explicit state dimension and budget.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_generating_block13.py` and `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27 and 64.

C.3B · `audit_apery_supercongruence`

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.audit_apery_supercongruence`. Reflected implementation `komorebi_math.generating.congruences.audit_apery_supercongruence`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-CONGRUENCE`.

Manifest signature (s).

```
(values: 'Sequence[int]', prime: 'int', exponent: 'int' = 3) -> 'RelationAudit'
```

Definition. Verify `A_(p*n)=A_n (mod p^exponent)` on the available prefix.

Contract (D, C, H, R, X). Congruence audits accept finite prefixes, a valid prime or modulus, and explicit index, order, degree, or lane bounds. Rational valuations require denominators that are meaningful for the selected prime. Recurrence discovery and decimation search operate only within the declared finite model class. An empty or insufficient prefix returns an empty result or no candidate according to the individual signature.

Guarantees (E, P, B, K). Every reported residue, valuation, recurrence, lane, or failure tuple is computed exactly on the supplied finite range. A passing audit is evidence for that range and is not an unbounded theorem by itself. Berlekamp-Massey is exact over the selected prime field. Search cost grows with prefix length, lane count, recurrence order, and coefficient search space.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_generating_foundation.py` and `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27.

C.3B · audit_factorial_power_digit_algebra

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.audit_factorial_power_digit_algebra`. Reflected implementation `komorebi_math.generating.digit_algebra.audit_factorial_power_digit_algebra`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-DIGIT`.

Manifest signature (s).

```
(*, prime: 'int', power: 'int', left_digits: 'Iterable[int]', right_digits: 'Iterable[int]', max_digits: 'int' =
   ↪ 3) -> 'DigitAlgebraAudit'
```

Definition. Audit Lucas-compatible closure below `prime^max_digits`. The theorem being tested is `(a star_s b)_n = prod_i (a star_s b)_(n_i) (mod p)` whenever `a_n` and `b_n` are lifted products of their base-`p` digit values and `star_s` uses `binom(n,k)^s`.

Contract (D, C, H, R, X). Digit kernels require a radix and modulus of at least two and one lower-triangular row for each digit. Global weights return zero when subtraction would borrow. Lifted sequences require one local value per digit. Factorial-power audits require a prime, positive power, and an explicit digit depth.

Guarantees (E, P, B, K). Local and global kernel weights, lifted sequences, and convolution audits are exact modulo the declared modulus. A finite DigitAlgebraAudit records the checked range and all failures. Digitwise evaluation is linear in the radix-digit width. Full closure audits scale with the size of the finite range and the number of tested subindices.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_generating_foundation.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 26.

C.3B · audit_factorial_power_packet_convolution

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.audit_factorial_power_packet_convolution`. Reflected implementation `komorebi_math.generating.congruences.audit_factorial_power_packet_convolution`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-CONGRUENCE`.

Manifest signature (s).

```
(left: 'Sequence[int]', right: 'Sequence[int]', *, power: 'int', prime: 'int') -> 'ConvolutionPacketAudit'
```

Definition. Verify the Lucas/polyphase convolution identity modulo a prime.

Contract (D, C, H, R, X). Congruence audits accept finite prefixes, a valid prime or modulus, and explicit index, order, degree, or lane bounds. Rational valuations require denominators that are meaningful for the selected prime. Recurrence discovery and decimation search operate only within the declared finite model class. An empty or insufficient prefix returns an empty result or no candidate according to the individual signature.

Guarantees (E, P, B, K). Every reported residue, valuation, recurrence, lane, or failure tuple is computed exactly on the supplied finite range. A passing audit is evidence for that range and is not an unbounded theorem by itself.

Berlekamp-Massey is exact over the selected prime field. Search cost grows with prefix length, lane count, recurrence order, and coefficient search space.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_generating_foundation.py` and `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27.

C.3B · audit_generalized_bell_digit_sum

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.audit_generalized_bell_digit_sum`. Reflected implementation `komorebi_math.generating.congruences.audit_generalized_bell_digit_sum`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-CONGRUENCE`.

Manifest signature (s).

```
(*, prime: 'int', power: 'int', count: 'int' = 300) -> 'DigitSumCongruenceAudit'
```

Definition. Verify `B_n^(s)(x)=B_(s_p(n))(x) (mod p)` on a finite range.

Contract (D, C, H, R, X). Congruence audits accept finite prefixes, a valid prime or modulus, and explicit index, order, degree, or lane bounds. Rational valuations require denominators that are meaningful for the selected prime. Recurrence discovery and decimation search operate only within the declared finite model class. An empty or insufficient prefix returns an empty result or no candidate according to the individual signature.

Guarantees (E, P, B, K). Every reported residue, valuation, recurrence, lane, or failure tuple is computed exactly on the supplied finite range. A passing audit is evidence for that range and is not an unbounded theorem by itself. Berlekamp-Massey is exact over the selected prime field. Search cost grows with prefix length, lane count, recurrence order, and coefficient search space.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_generating_foundation.py` and `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27.

C.3B · audit_generalized_lucas

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.audit_generalized_lucas`. Reflected implementation `komorebi_math.generating.congruences.audit_generalized_lucas`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-CONGRUENCE`.

Manifest signature (s).

```
(basis: 'WeightedMonomialBasis', *, prime: 'int', high_limit: 'int' = 12) -> 'LucasBasisAudit'
```

Definition. Test `C(pn+r,pk+i)=C(n,k)C(r,i) (mod p)` when integral.

Contract (D, C, H, R, X). Congruence audits accept finite prefixes, a valid prime or modulus, and explicit index, order, degree, or lane bounds. Rational valuations require denominators that are meaningful for the selected prime. Recurrence discovery and decimation search operate only within the declared finite model class. An empty or insufficient prefix returns an empty result or no candidate according to the individual signature.

Guarantees (E, P, B, K). Every reported residue, valuation, recurrence, lane, or failure tuple is computed exactly on the supplied finite range. A passing audit is evidence for that range and is not an unbounded theorem by itself. Berlekamp-Massey is exact over the selected prime field. Search cost grows with prefix length, lane count, recurrence order, and coefficient search space.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_generating_foundation.py` and `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27.

C.3B · audit_partition_lane

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.audit_partition_lane`. Reflected implementation `komorebi_math.generating.congruences.audit_partition_lane`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-CONGRUENCE`.

Manifest signature (s).

```
(values: 'Sequence[int]', prime: 'int', residue: 'int') -> 'RelationAudit'
```

Definition. Audit divisibility by `prime` on indices `prime*n + residue`.

Contract (D, C, H, R, X). Congruence audits accept finite prefixes, a valid prime or modulus, and explicit index, order, degree, or lane bounds. Rational valuations require denominators that are meaningful for the selected prime. Recurrence discovery and decimation search operate only within the declared finite model class. An empty or insufficient prefix returns an empty result or no candidate according to the individual signature.

Guarantees (E, P, B, K). Every reported residue, valuation, recurrence, lane, or failure tuple is computed exactly on the supplied finite range. A passing audit is evidence for that range and is not an unbounded theorem by itself. Berlekamp-Massey is exact over the selected prime field. Search cost grows with prefix length, lane count, recurrence order, and coefficient search space.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_generating_foundation.py` and `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27.

C.3B · audit_q_lucas

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.audit_q_lucas`. Reflected implementation `komorebi_math.generating.congruences.audit_q_lucas`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-CONGRUENCE`.

Manifest signature (s).

```
(*, q: 'int', prime: 'int', high_limit: 'int' = 10) -> 'QLucasAudit'
```

Definition. Audit the order-radix Gaussian-binomial Lucas law. If `d` is the multiplicative order of `q` modulo `p`, the tested identity is `[ad+b choose rd+s]_q = C(a,r) [b choose s]_q (mod p)`.

Contract (D, C, H, R, X). Congruence audits accept finite prefixes, a valid prime or modulus, and explicit index, order, degree, or lane bounds. Rational valuations require denominators that are meaningful for the selected prime. Recurrence discovery and decimation search operate only within the declared finite model class. An empty or insufficient prefix returns an empty result or no candidate according to the individual signature.

Guarantees (E, P, B, K). Every reported residue, valuation, recurrence, lane, or failure tuple is computed exactly on the supplied finite range. A passing audit is evidence for that range and is not an unbounded theorem by itself. Berlekamp-Massey is exact over the selected prime field. Search cost grows with prefix length, lane count, recurrence order, and coefficient search space.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_generating_foundation.py` and `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite

certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27.

C.3B · audit_touchard

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.audit_touchard`. Reflected implementation `komorebi_math.generating.congruences.audit_touchard`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-CONGRUENCE`.

Manifest signature (s).

```
(values: 'Sequence[int]', prime: 'int') -> 'RelationAudit'
```

Definition. Verify `B_(n+p)=B_(n+1)+B_n (mod p)` on a prefix.

Contract (D, C, H, R, X). Congruence audits accept finite prefixes, a valid prime or modulus, and explicit index, order, degree, or lane bounds. Rational valuations require denominators that are meaningful for the selected prime. Recurrence discovery and decimation search operate only within the declared finite model class. An empty or insufficient prefix returns an empty result or no candidate according to the individual signature.

Guarantees (E, P, B, K). Every reported residue, valuation, recurrence, lane, or failure tuple is computed exactly on the supplied finite range. A passing audit is evidence for that range and is not an unbounded theorem by itself. Berlekamp-Massey is exact over the selected prime field. Search cost grows with prefix length, lane count, recurrence order, and coefficient search space.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_generating_foundation.py` and `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27.

C.3B · audit_weighted_generalized_bell_digit_sum

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.audit_weighted_generalized_bell_digit_sum`. Reflected implementation `komorebi_math.generating.congruences.audit_weighted_generalized_bell_digit_sum`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-CONGRUENCE`.

Manifest signature (s).

```
(*, prime: 'int', power: 'int', digit_weights: 'Sequence[int]', count: 'int' = 300) ->
    ↪ 'WeightedDigitSumCongruenceAudit'
```

Definition. Audit a custom weighted factorial-power Bell congruence. Let `c_m = d_(s_p(m))` and define `F(z,x)=exp( x*sum_(m>=1) c_m z^m/(m!)^power)`. When `power >= 2` and `power-1` is divisible by `prime-1`, the normalized coefficient polynomial at index `n` is predicted to equal, modulo `prime`, the ordinary weighted Bell polynomial at `s_p(n)` with block weights `d_t`.

Contract (D, C, H, R, X). Congruence audits accept finite prefixes, a valid prime or modulus, and explicit index, order, degree, or lane bounds. Rational valuations require denominators that are meaningful for the selected prime. Recurrence discovery and decimation search operate only within the declared finite model class. An empty or insufficient prefix returns an empty result or no candidate according to the individual signature.

Guarantees (E, P, B, K). Every reported residue, valuation, recurrence, lane, or failure tuple is computed exactly on the supplied finite range. A passing audit is evidence for that range and is not an unbounded theorem by itself. Berlekamp-Massey is exact over the selected prime field. Search cost grows with prefix length, lane count, recurrence order, and coefficient search space.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_generating_foundation.py` and `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite

certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27.

C.3B · base_digit_sum

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.base_digit_sum`. Reflected implementation `komorebi_math.generating.congruences.base_digit_sum`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-CONGRUENCE`.

Manifest signature (s).

```
(index: 'int', base: 'int') -> 'int'
```

Definition. Return the sum of the nonnegative integer index digits in the declared base.

Contract (D, C, H, R, X). Congruence audits accept finite prefixes, a valid prime or modulus, and explicit index, order, degree, or lane bounds. Rational valuations require denominators that are meaningful for the selected prime. Recurrence discovery and decimation search operate only within the declared finite model class. An empty or insufficient prefix returns an empty result or no candidate according to the individual signature.

Guarantees (E, P, B, K). Every reported residue, valuation, recurrence, lane, or failure tuple is computed exactly on the supplied finite range. A passing audit is evidence for that range and is not an unbounded theorem by itself. Berlekamp-Massey is exact over the selected prime field. Search cost grows with prefix length, lane count, recurrence order, and coefficient search space.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_generating_foundation.py` and `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27.

C.3B · base_digits

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.base_digits`. Reflected implementation `komorebi_math.generating.digit_algebra.base_digits`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-DIGIT`.

Manifest signature (s).

```
(index: 'int', radix: 'int') -> 'tuple[int, ...]'
```

Definition. Return least-significant-first digits, with zero represented by `(0,)`.

Contract (D, C, H, R, X). Digit kernels require a radix and modulus of at least two and one lower-triangular row for each digit. Global weights return zero when subtraction would borrow. Lifted sequences require one local value per digit. Factorial-power audits require a prime, positive power, and an explicit digit depth.

Guarantees (E, P, B, K). Local and global kernel weights, lifted sequences, and convolution audits are exact modulo the declared modulus. A finite DigitAlgebraAudit records the checked range and all failures. Digitwise evaluation is linear in the radix-digit width. Full closure audits scale with the size of the finite range and the number of tested subindices.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_generating_foundation.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 26.

C.3B · `bell_period_bound`

Identity (`q, a, t, k`). Public research manifest route `komorebi_math.generating.bell_period_bound`. Reflected implementation `komorebi_math.generating.bell.bell_period_bound`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-BELL`.

Manifest signature (`s`).

```
(prime: 'int') -> 'int'
```

Definition. Return `(p**p-1)/(p-1)`, a period bound modulo `p`.

Contract (`D, C, H, R, X`). Bell reductions require a prime, a nonnegative index, and valid factorial-power or weight data. Automata consume base-prime digits in the order stated by each method. Period bounds and pointed-subindex identities are finite arithmetic contracts. Weighted reductions require enough local weights for the requested computation. Resource budgets cap explicit polynomial and state construction.

Guarantees (`E, P, B, K`). The returned reductions, parity values, residue automata, and pointed polynomial certificates are exact modulo the declared prime. A certificate records the finite comparison it performed. The period bound is an upper bound, not a minimal-period claim. Direct polynomial construction can grow with the digit sum or index, while automaton evaluation is linear in the number of input digits.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_generating_foundation.py` and `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 22, 24, and 26.

C.3B · `berlekamp_massey`

Identity (`q, a, t, k`). Public research manifest route `komorebi_math.generating.berlekamp_massey`. Reflected implementation `komorebi_math.generating.congruences.berlekamp_massey`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-CONGRUENCE`.

Manifest signature (`s`).

```
(values: 'Sequence[int]', prime: 'int') -> 'tuple[int, tuple[int, ...]]'
```

Definition. Return the minimal recurrence over a prime field for the supplied prefix.

Contract (`D, C, H, R, X`). Congruence audits accept finite prefixes, a valid prime or modulus, and explicit index, order, degree, or lane bounds. Rational valuations require denominators that are meaningful for the selected prime. Recurrence discovery and decimation search operate only within the declared finite model class. An empty or insufficient prefix returns an empty result or no candidate according to the individual signature.

Guarantees (`E, P, B, K`). Every reported residue, valuation, recurrence, lane, or failure tuple is computed exactly on the supplied finite range. A passing audit is evidence for that range and is not an unbounded theorem by itself. Berlekamp-Massey is exact over the selected prime field. Search cost grows with prefix length, lane count, recurrence order, and coefficient search space.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_generating_foundation.py` and `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27.

C.3B · binomial_valuation

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.binomial_valuation`. Reflected implementation `komorebi_math.generating.padic_units.binomial_valuation`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-PADIC`.

Manifest signature (s).

```
(index: 'int', subindex: 'int', prime: 'int') -> 'int'
```

Definition. Return `v_prime(binomial(index,subindex))`.

Contract (D, C, H, R, X). Valuation functions require a prime and valid integer or binomial indices. Prime-power compilers require positive precision and a modulus within the supplied budget. Unit-factorial and binomial methods distinguish valuation from the invertible unit part. Kernel compilers require compatible digit-unit tables and factorial exponents. Borrow counts use nonnegative indices with the subindex not exceeding the index where required.

Guarantees (E, P, B, K). Valuations, unit parts, residues, and survival decisions are exact for the declared prime power. PrimePowerBinomial separates the p-adic valuation from the unit residue, so zero modulo the prime power is represented without discarding valuation information. Precomputation cost depends on the prime-power modulus. Subsequent digit recursions scale with the number of base-prime digits.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27.

C.3B · borrow_count

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.borrow_count`. Reflected implementation `komorebi_math.generating.padic_units.borrow_count`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-PADIC`.

Manifest signature (s).

```
(index: 'int', subindex: 'int', prime: 'int') -> 'int'
```

Definition. Count base-p borrows in subtracting `subindex` from `index`.

Contract (D, C, H, R, X). Valuation functions require a prime and valid integer or binomial indices. Prime-power compilers require positive precision and a modulus within the supplied budget. Unit-factorial and binomial methods distinguish valuation from the invertible unit part. Kernel compilers require compatible digit-unit tables and factorial exponents. Borrow counts use nonnegative indices with the subindex not exceeding the index where required.

Guarantees (E, P, B, K). Valuations, unit parts, residues, and survival decisions are exact for the declared prime power. PrimePowerBinomial separates the p-adic valuation from the unit residue, so zero modulo the prime power is represented without discarding valuation information. Precomputation cost depends on the prime-power modulus. Subsequent digit recursions scale with the number of base-prime digits.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27.

C.3B · `cartier_degree_bound`

Identity (`q, a, t, k`). Public research manifest route `komorebi_math.generating.cartier_degree_bound`. Reflected implementation `komorebi_math.generating.cartier.cartier_degree_bound`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-CARTIER`.

Manifest signature (`s`).

```
(numerator: 'Sequence[int]', denominator: 'Sequence[int]', prime: 'int') -> 'int'
```

Definition. Return a degree bound invariant under every rational-mask transition.

Contract (`D, C, H, R, X`). Polynomial operations require a modulus of at least two. Cartier operators and rational-mask compilers require a prime, a valid digit, compatible nonzero constant coefficients, and explicit state or degree budgets. Automata read least-significant-first prime digits. Minimization preserves output behavior on the compiled finite-state machine. Certificate routines reject mismatched transition systems or coefficient prefixes.

Guarantees (`E, P, B, K`). Polynomial arithmetic, Cartier transitions, automaton outputs, and linear representations are exact modulo the selected prime. State bounds are finite compilation bounds rather than asymptotic minimality claims. Rational-mask coefficient generation and automaton evaluation agree on the checked prefix. Complexity is polynomial in polynomial degree and compiled state count, with minimization depending on partition refinement over the full transition table.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27.

C.3B · `certify_cartier_carry_equivalence`

Identity (`q, a, t, k`). Public research manifest route `komorebi_math.generating.certify_cartier_carry_equivalence`. Reflected implementation `komorebi_math.generating.cartier.certify_cartier_carry_equivalence`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-CARTIER`.

Manifest signature (`s`).

```
(mask: 'Sequence[int]', prime: 'int', *, budget: 'GeneratingBudget' = GeneratingBudget(maximum_degree=100000,
    ↪ maximum_coefficient_bits=4000000, maximum_matrix_dimension=4096, maximum_reachable_states=100000,
    ↪ maximum_local_combinations=1000000, maximum_modulus=2000000, maximum_serialized_bytes=67108864,
    ↪ maximum_recursion_depth=1000)) -> 'CartierCarryEquivalence'
```

Definition. Certify that finite-mask carry matrices are Cartier multiplication matrices.

Contract (`D, C, H, R, X`). Polynomial operations require a modulus of at least two. Cartier operators and rational-mask compilers require a prime, a valid digit, compatible nonzero constant coefficients, and explicit state or degree budgets. Automata read least-significant-first prime digits. Minimization preserves output behavior on the compiled finite-state machine. Certificate routines reject mismatched transition systems or coefficient prefixes.

Guarantees (`E, P, B, K`). Polynomial arithmetic, Cartier transitions, automaton outputs, and linear representations are exact modulo the selected prime. State bounds are finite compilation bounds rather than asymptotic minimality claims. Rational-mask coefficient generation and automaton evaluation agree on the checked prefix. Complexity is polynomial in polynomial degree and compiled state count, with minimization depending on partition refinement over the full transition table.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27.

C.3B · certify_mahler_functional_equation

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.certify_mahler_functional_equation`. Reflected implementation `komorebi_math.generating.masks.certify_mahler_functional_equation`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-MASK`.

Manifest signature (s).

```
(series: 'Iterable[Scalar]', mask: 'Iterable[Scalar]', radix: 'int', degree: 'int | None' = None, *, budget:
    ↪ 'GeneratingBudget' = GeneratingBudget(maximum_degree=100000, maximum_coefficient_bits=4000000,
    ↪ maximum_matrix_dimension=4096, maximum_reachable_states=100000, maximum_local_combinations=1000000,
    ↪ maximum_modulus=2000000, maximum_serialized_bytes=67108864, maximum_recursion_depth=1000)) -> 'Certificate'
```

Definition. Certify `F(z)=R(z)F(z**radix)` on a declared finite truncation.

Contract (D, C, H, R, X). Inputs describe a finite local mask, a radix of at least two, and an explicit truncation, modulus, exponent, or resource budget as required by the operation. Unit-series routines require a valid constant coefficient. Prime-power support routines require a prime and positive exponent. Carry plans validate every state and local index before allocation. Bounded solvers return a negative result when the declared search space contains no solution.

Guarantees (E, P, B, K). Finite coefficient identities, carry matrices, support layers, and polynomial certificates use exact integer or rational arithmetic. Analytic approximations separate the computed product from a stated tail bound. A finite functional equation or algebraic certificate applies only through its recorded degree. Direct mask products are at least quadratic in the truncation boundary, while carry-state plans depend on the explicit state dimension and budget.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_generating_block13.py` and `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27 and 64.

C.3B · certify_pointed_subindex_identity

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.certify_pointed_subindex_identity`. Reflected implementation `komorebi_math.generating.bell.certify_pointed_subindex_identity`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-BELL`.

Manifest signature (s).

```
(index: 'int', prime: 'int') -> 'PointedSubindexCertificate'
```

Definition. Certify the pointed subindex polynomial against `(1+u)^(s_p(n)-1)`.

Contract (D, C, H, R, X). Bell reductions require a prime, a nonnegative index, and valid factorial-power or weight data. Automata consume base-prime digits in the order stated by each method. Period bounds and pointed-subindex identities are finite arithmetic contracts. Weighted reductions require enough local weights for the requested computation. Resource budgets cap explicit polynomial and state construction.

Guarantees (E, P, B, K). The returned reductions, parity values, residue automata, and pointed polynomial certificates are exact modulo the declared prime. A certificate records the finite comparison it performed. The period bound is an upper bound, not a minimal-period claim. Direct polynomial construction can grow with the digit sum or index, while automaton evaluation is linear in the number of input digits.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_generating_foundation.py` and `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 22, 24, and 26.

C.3B · certify_rational_mask_algebraic_relation

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.certify_rational_mask_algebraic_relation`. Reflected implementation `komorebi_math.generating.masks.certify_rational_mask_algebraic_relation`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-MASK`.

Manifest signature (s).

```
(numerator: 'Sequence[int]', denominator: 'Sequence[int]', prime: 'int', degree: 'int', *, budget:
    ↪ 'GeneratingBudget' = GeneratingBudget(maximum_degree=100000, maximum_coefficient_bits=4000000,
    ↪ maximum_matrix_dimension=4096, maximum_reachable_states=100000, maximum_local_combinations=1000000,
    ↪ maximum_modulus=2000000, maximum_serialized_bytes=67108864, maximum_recursion_depth=1000)) ->
    ↪ 'RationalMaskAlgebraicCertificate'
```

Definition. Certify `A(z)F(z)^(p-1)=B(z)` modulo `p` through `degree`.

Contract (D, C, H, R, X). Inputs describe a finite local mask, a radix of at least two, and an explicit truncation, modulus, exponent, or resource budget as required by the operation. Unit-series routines require a valid constant coefficient. Prime-power support routines require a prime and positive exponent. Carry plans validate every state and local index before allocation. Bounded solvers return a negative result when the declared search space contains no solution.

Guarantees (E, P, B, K). Finite coefficient identities, carry matrices, support layers, and polynomial certificates use exact integer or rational arithmetic. Analytic approximations separate the computed product from a stated tail bound. A finite functional equation or algebraic certificate applies only through its recorded degree. Direct mask products are at least quadratic in the truncation boundary, while carry-state plans depend on the explicit state dimension and budget.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_generating_block13.py` and `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27 and 64.

C.3B · classify_carry_linear_valuation

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.classify_carry_linear_valuation`. Reflected implementation `komorebi_math.generating.normalization.classify_carry_linear_valuation`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-NORMALIZATION`.

Manifest signature (s).

```
(valuation: 'Callable[[int], int]', prime: 'int', checked_through: 'int') -> 'CarryLinearCertificate'
```

Definition. Recover the profile parameters and audit a finite range. The mathematical classification is unbounded. This routine is explicitly a finite certificate for a supplied callback.

Contract (D, C, H, R, X). Digit-unit and normalized-kernel routines require a prime, nonnegative indices, and unit data compatible with the selected digit system. Carry-linear classification requires a callback with zero value at zero and a finite audit range large enough to recover the slope and carry weight. Invalid unit tables, primes, or audit ranges are rejected.

Guarantees (E, P, B, K). Local and digit-factorized normalized kernels are exact modulo the selected prime power. CarryLinearCertificate records the recovered profile and every finite mismatch. Passing the finite audit confirms the supplied range only. Kernel evaluation scales with the number of base-prime digits, while classification scales with the requested audit boundary.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime owner-

ship. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 21, 24, and 26.

C.3B · `compile_cartier_linear_representation`

Identity (`q, a, t, k`). Public research manifest route `komorebi_math.generating.compile_cartier_linear_representation`. Reflected implementation `komorebi_math.generating.cartier.compile_cartier_linear_representation`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-CARTIER`.

Manifest signature (`s`).

```
(numerator: 'Sequence[int]', denominator: 'Sequence[int]', prime: 'int', *, budget: 'GeneratingBudget' =
    ↪ GeneratingBudget(maximum_degree=100000, maximum_coefficient_bits=4000000, maximum_matrix_dimension=4096,
    ↪ maximum_reachable_states=100000, maximum_local_combinations=1000000, maximum_modulus=2000000,
    ↪ maximum_serialized_bytes=67108864, maximum_recursion_depth=1000)) -> 'CartierLinearRepresentation'
```

Definition. Compile the denominator-stabilized Cartier transition matrices.

Contract (`D, C, H, R, X`). Polynomial operations require a modulus of at least two. Cartier operators and rational-mask compilers require a prime, a valid digit, compatible nonzero constant coefficients, and explicit state or degree budgets. Automata read least-significant-first prime digits. Minimization preserves output behavior on the compiled finite-state machine. Certificate routines reject mismatched transition systems or coefficient prefixes.

Guarantees (`E, P, B, K`). Polynomial arithmetic, Cartier transitions, automaton outputs, and linear representations are exact modulo the selected prime. State bounds are finite compilation bounds rather than asymptotic minimality claims. Rational-mask coefficient generation and automaton evaluation agree on the checked prefix. Complexity is polynomial in polynomial degree and compiled state count, with minimization depending on partition refinement over the full transition table.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27.

C.3B · `compile_digit_cauchy_product`

Identity (`q, a, t, k`). Public research manifest route `komorebi_math.generating.compile_digit_cauchy_product`. Reflected implementation `komorebi_math.generating.masks.compile_digit_cauchy_product`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-MASK`.

Manifest signature (`s`).

```
(digit_weights: 'Sequence[Sequence[Scalar]]', radix: 'int', *, budget: 'GeneratingBudget' =
    ↪ GeneratingBudget(maximum_degree=100000, maximum_coefficient_bits=4000000, maximum_matrix_dimension=4096,
    ↪ maximum_reachable_states=100000, maximum_local_combinations=1000000, maximum_modulus=2000000,
    ↪ maximum_serialized_bytes=67108864, maximum_recursion_depth=1000)) -> 'DigitCauchyProductPlan'
```

Definition. Compile finite carries for a Cauchy product of digit-local sequences.

Contract (`D, C, H, R, X`). Inputs describe a finite local mask, a radix of at least two, and an explicit truncation, modulus, exponent, or resource budget as required by the operation. Unit-series routines require a valid constant coefficient. Prime-power support routines require a prime and positive exponent. Carry plans validate every state and local index before allocation. Bounded solvers return a negative result when the declared search space contains no solution.

Guarantees (`E, P, B, K`). Finite coefficient identities, carry matrices, support layers, and polynomial certificates use exact integer or rational arithmetic. Analytic approximations separate the computed product from a stated tail bound. A finite functional equation or algebraic certificate applies only through its recorded degree. Direct mask products

are at least quadratic in the truncation boundary, while carry-state plans depend on the explicit state dimension and budget.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_generating_block13.py` and `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27 and 64.

C.3B · `compile_finite_mask_carry_plan`

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.compile_finite_mask_carry_plan`. Reflected implementation `komorebi_math.generating.masks.compile_finite_mask_carry_plan`. The same runtime object is also exported as `komorebi_math.compile_finite_mask_carry_plan`. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-MASK`.

Manifest signature (s).

```
(mask: 'Iterable[Scalar]', radix: 'int', *, budget: 'GeneratingBudget' = GeneratingBudget(maximum_degree=100000,
    ↪ maximum_coefficient_bits=4000000, maximum_matrix_dimension=4096, maximum_reachable_states=100000,
    ↪ maximum_local_combinations=1000000, maximum_modulus=2000000, maximum_serialized_bytes=67108864,
    ↪ maximum_recursion_depth=1000)) -> 'FiniteMaskCarryPlan'
```

Definition. Compile `T_d(q,q')=r_(d+radix*q'-q)` for a finite mask.

Contract (D, C, H, R, X). Inputs describe a finite local mask, a radix of at least two, and an explicit truncation, modulus, exponent, or resource budget as required by the operation. Unit-series routines require a valid constant coefficient. Prime-power support routines require a prime and positive exponent. Carry plans validate every state and local index before allocation. Bounded solvers return a negative result when the declared search space contains no solution.

Guarantees (E, P, B, K). Finite coefficient identities, carry matrices, support layers, and polynomial certificates use exact integer or rational arithmetic. Analytic approximations separate the computed product from a stated tail bound. A finite functional equation or algebraic certificate applies only through its recorded degree. Direct mask products are at least quadratic in the truncation boundary, while carry-state plans depend on the explicit state dimension and budget.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_generating_block13.py` and `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27 and 64.

C.3B · `compile_rational_mask_automaton`

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.compile_rational_mask_automaton`. Reflected implementation `komorebi_math.generating.cartier.compile_rational_mask_automaton`. The same runtime object is also exported as `komorebi_math.compile_rational_mask_automaton`. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-CARTIER`.

Manifest signature (s).

```
(numerator: 'Sequence[int]', denominator: 'Sequence[int]', prime: 'int', *, minimize: 'bool' = True, max_states:
    ↪ 'int' = 100000, budget: 'GeneratingBudget' = GeneratingBudget(maximum_degree=100000,
    ↪ maximum_coefficient_bits=4000000, maximum_matrix_dimension=4096, maximum_reachable_states=100000,
    ↪ maximum_local_combinations=1000000, maximum_modulus=2000000, maximum_serialized_bytes=67108864,
    ↪ maximum_recursion_depth=1000)) -> 'CartierAutomaton'
```

Definition. Compile a rational Mahler mask into a coefficient automaton modulo `prime`. The degree-derived linear dimension and the reachable-state guard are checked before state exploration begins.

Contract (D, C, H, R, X). Polynomial operations require a modulus of at least two. Cartier operators and rational-mask compilers require a prime, a valid digit, compatible nonzero constant coefficients, and explicit state or degree budgets. Automata read least-significant-first prime digits. Minimization preserves output behavior on the compiled finite-state machine. Certificate routines reject mismatched transition systems or coefficient prefixes.

Guarantees (E, P, B, K). Polynomial arithmetic, Cartier transitions, automaton outputs, and linear representations are exact modulo the selected prime. State bounds are finite compilation bounds rather than asymptotic minimality claims. Rational-mask coefficient generation and automaton evaluation agree on the checked prefix. Complexity is polynomial in polynomial degree and compiled state count, with minimization depending on partition refinement over the full transition table.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27.

C.3B · `cyclotomic_coboundary`

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.cyclotomic_coboundary`. Reflected implementation `komorebi_math.generating.cyclotomic.cyclotomic_coboundary`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-CYCLOTOMIC`.

Manifest signature (s).

```
(prime: 'int', exponents: 'Mapping[int, int]') -> 'CyclotomicCollapse'
```

Definition. Solve `R(z)=Q(z)/Q(z**p)` for a finite cyclotomic mask. Factors are normalized to constant coefficient one. The exact obstruction is a nonzero exponent at level zero of a p-power orbit.

Contract (D, C, H, R, X). Cyclotomic routines require a prime and positive cyclotomic indices. Exponent maps omit zero entries after normalization. Symbolic expression and verification functions require the optional SymPy backend. A collapse result retains every p-free obstruction orbit and raises when the caller demands a closed form that is not present.

Guarantees (E, P, B, K). Orbit decomposition and exponent-lattice relations are exact. When no obstruction remains, the returned finite cyclotomic quotient is an exact formal collapse of the local Mahler product. Symbolic verification checks the declared relation rather than discovering a new one. Runtime is linear in the number of represented factors apart from optional symbolic simplification.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_generating_cartier_padic.py` and `tests/test_generating_block13.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 26.

C.3B · `difference_valuation_profile`

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.difference_valuation_profile`. Reflected implementation `komorebi_math.generating.congruences.difference_valuation_profile`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-CONGRUENCE`.

Manifest signature (`s`).

```
(values: 'Sequence[int | Fraction]', *, prime: 'int', step: 'int', max_order: 'int') ->
    ↪ 'DifferenceValuationProfile'
```

Definition. Measure minimum `p`-adic valuations of successive stepped differences.

Contract (`D, C, H, R, X`). Congruence audits accept finite prefixes, a valid prime or modulus, and explicit index, order, degree, or lane bounds. Rational valuations require denominators that are meaningful for the selected prime. Recurrence discovery and decimation search operate only within the declared finite model class. An empty or insufficient prefix returns an empty result or no candidate according to the individual signature.

Guarantees (`E, P, B, K`). Every reported residue, valuation, recurrence, lane, or failure tuple is computed exactly on the supplied finite range. A passing audit is evidence for that range and is not an unbounded theorem by itself. Berlekamp-Massey is exact over the selected prime field. Search cost grows with prefix length, lane count, recurrence order, and coefficient search space.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_generating_foundation.py` and `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27.

C.3B · `digit_factorized_normalized_kernel`

Identity (`q, a, t, k`). Public research manifest route `komorebi_math.generating.digit_factorized_normalized_kernel`. Reflected implementation `komorebi_math.generating.normalization.digit_factorized_normalized_kernel`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-NORMALIZATION`.

Manifest signature (`s`).

```
(index: 'int', subindex: 'int', prime: 'int', factorial_power: 'int', unit_weights: 'Sequence[int]') -> 'int'
```

Definition. Multiply the local normalized kernels over all base-p digits.

Contract (`D, C, H, R, X`). Digit-unit and normalized-kernel routines require a prime, nonnegative indices, and unit data compatible with the selected digit system. Carry-linear classification requires a callback with zero value at zero and a finite audit range large enough to recover the slope and carry weight. Invalid unit tables, primes, or audit ranges are rejected.

Guarantees (`E, P, B, K`). Local and digit-factorized normalized kernels are exact modulo the selected prime power. CarryLinearCertificate records the recovered profile and every finite mismatch. Passing the finite audit confirms the supplied range only. Kernel evaluation scales with the number of base-prime digits, while classification scales with the requested audit boundary.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 21, 24, and 26.

C.3B · `digit_unit`

Identity (`q, a, t, k`). Public research manifest route `komorebi_math.generating.digit_unit`. Reflected implementation `komorebi_math.generating.normalization.digit_unit`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-NORMALIZATION`.

Manifest signature (`s`).

```
(index: 'int', prime: 'int', weights: 'Sequence[int]') -> 'int'
```

Definition. Evaluate a digit-multiplicative p-adic unit profile modulo `prime`.

Contract (`D, C, H, R, X`). Digit-unit and normalized-kernel routines require a prime, nonnegative indices, and unit data compatible with the selected digit system. Carry-linear classification requires a callback with zero value at zero and a finite audit range large enough to recover the slope and carry weight. Invalid unit tables, primes, or audit ranges are rejected.

Guarantees (`E, P, B, K`). Local and digit-factorized normalized kernels are exact modulo the selected prime power. CarryLinearCertificate records the recovered profile and every finite mismatch. Passing the finite audit confirms the supplied range only. Kernel evaluation scales with the number of base-prime digits, while classification scales with the requested audit boundary.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 21, 24, and 26.

C.3B · `euler_product_collapse`

Identity (`q, a, t, k`). Public research manifest route `komorebi_math.generating.euler_product_collapse`. Reflected implementation `komorebi_math.generating.masks.euler_product_collapse`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-MASK`.

Manifest signature (`s`).

```
(radix: 'int', exponents: 'Mapping[int, int]') -> 'EulerProductCollapse'
```

Definition. Compute the finite Euler product represented by a local Euler mask.

Contract (`D, C, H, R, X`). Inputs describe a finite local mask, a radix of at least two, and an explicit truncation, modulus, exponent, or resource budget as required by the operation. Unit-series routines require a valid constant coefficient. Prime-power support routines require a prime and positive exponent. Carry plans validate every state and local index before allocation. Bounded solvers return a negative result when the declared search space contains no solution.

Guarantees (`E, P, B, K`). Finite coefficient identities, carry matrices, support layers, and polynomial certificates use exact integer or rational arithmetic. Analytic approximations separate the computed product from a stated tail bound. A finite functional equation or algebraic certificate applies only through its recorded degree. Direct mask products are at least quadratic in the truncation boundary, while carry-state plans depend on the explicit state dimension and budget.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_generating_block13.py` and `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27 and 64.

C.3B · `factorial_power_convolution_layers`

Identity (`q, a, t, k`). Public research manifest route `komorebi_math.generating.factorial_power_convolution_layers`. Reflected implementation `komorebi_math.generating.masks.factorial_power_convolution_layers`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-MASK`.

Manifest signature (`s`).

```
(left: 'Sequence[int]', right: 'Sequence[int]', index: 'int', prime: 'int', factorial_power: 'int',
    ↪ modulus_exponent: 'int', *, budget: 'GeneratingBudget' = GeneratingBudget(maximum_degree=100000,
    ↪ maximum_coefficient_bits=4000000, maximum_matrix_dimension=4096, maximum_reachable_states=100000,
    ↪ maximum_local_combinations=1000000, maximum_modulus=2000000, maximum_serialized_bytes=67108864,
    ↪ maximum_recursion_depth=1000)) -> 'CarryLayerConvolution'
```

Definition. Evaluate a factorial-power convolution and retain each carry layer.

Contract (`D, C, H, R, X`). Inputs describe a finite local mask, a radix of at least two, and an explicit truncation, modulus, exponent, or resource budget as required by the operation. Unit-series routines require a valid constant coefficient. Prime-power support routines require a prime and positive exponent. Carry plans validate every state and local index before allocation. Bounded solvers return a negative result when the declared search space contains no solution.

Guarantees (`E, P, B, K`). Finite coefficient identities, carry matrices, support layers, and polynomial certificates use exact integer or rational arithmetic. Analytic approximations separate the computed product from a stated tail bound. A finite functional equation or algebraic certificate applies only through its recorded degree. Direct mask products are at least quadratic in the truncation boundary, while carry-state plans depend on the explicit state dimension and budget.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_generating_block13.py` and `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27 and 64.

C.3B · `factorial_power_digit_kernel`

Identity (`q, a, t, k`). Public research manifest route `komorebi_math.generating.factorial_power_digit_kernel`. Reflected implementation `komorebi_math.generating.digit_algebra.factorial_power_digit_kernel`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-DIGIT`.

Manifest signature (`s`).

```
(power: 'int', prime: 'int') -> 'DigitKernel'
```

Definition. Return the Lucas table for `binom(n,k)^power` modulo `prime`.

Contract (`D, C, H, R, X`). Digit kernels require a radix and modulus of at least two and one lower-triangular row for each digit. Global weights return zero when subtraction would borrow. Lifted sequences require one local value per digit. Factorial-power audits require a prime, positive power, and an explicit digit depth.

Guarantees (`E, P, B, K`). Local and global kernel weights, lifted sequences, and convolution audits are exact modulo the declared modulus. A finite DigitAlgebraAudit records the checked range and all failures. Digitwise evaluation is linear in the radix-digit width. Full closure audits scale with the size of the finite range and the number of tested subindices.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_generating_foundation.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 26.

C.3B · factorial_power_support_count

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.factorial_power_support_count`. Reflected implementation `komorebi_math.generating.masks.factorial_power_support_count`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-MASK`.

Manifest signature (s).

```
(index: 'int', prime: 'int', factorial_power: 'int', modulus_exponent: 'int') -> 'int'
```

Definition. Count `k` for which `binomial(index,k)**power` survives modulo `p**r`.

Contract (D, C, H, R, X). Inputs describe a finite local mask, a radix of at least two, and an explicit truncation, modulus, exponent, or resource budget as required by the operation. Unit-series routines require a valid constant coefficient. Prime-power support routines require a prime and positive exponent. Carry plans validate every state and local index before allocation. Bounded solvers return a negative result when the declared search space contains no solution.

Guarantees (E, P, B, K). Finite coefficient identities, carry matrices, support layers, and polynomial certificates use exact integer or rational arithmetic. Analytic approximations separate the computed product from a stated tail bound. A finite functional equation or algebraic certificate applies only through its recorded degree. Direct mask products are at least quadratic in the truncation boundary, while carry-state plans depend on the explicit state dimension and budget.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_generating_block13.py` and `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27 and 64.

C.3B · factorial_power_support_indices

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.factorial_power_support_indices`. Reflected implementation `komorebi_math.generating.masks.factorial_power_support_indices`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-MASK`.

Manifest signature (s).

```
(index: 'int', prime: 'int', factorial_power: 'int', modulus_exponent: 'int', *, budget: 'GeneratingBudget' =
    ↪ GeneratingBudget(maximum_degree=100000, maximum_coefficient_bits=4000000, maximum_matrix_dimension=4096,
    ↪ maximum_reachable_states=100000, maximum_local_combinations=1000000, maximum_modulus=2000000,
    ↪ maximum_serialized_bytes=67108864, maximum_recursion_depth=1000)) -> 'tuple[int, ...]'
```

Definition. Enumerate the surviving subindices under an explicit finite budget.

Contract (D, C, H, R, X). Inputs describe a finite local mask, a radix of at least two, and an explicit truncation, modulus, exponent, or resource budget as required by the operation. Unit-series routines require a valid constant coefficient. Prime-power support routines require a prime and positive exponent. Carry plans validate every state and local index before allocation. Bounded solvers return a negative result when the declared search space contains no solution.

Guarantees (E, P, B, K). Finite coefficient identities, carry matrices, support layers, and polynomial certificates use exact integer or rational arithmetic. Analytic approximations separate the computed product from a stated tail bound. A finite functional equation or algebraic certificate applies only through its recorded degree. Direct mask products are at least quadratic in the truncation boundary, while carry-state plans depend on the explicit state dimension and budget.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_generating_block13.py` and `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite cer-

tificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27 and 64.

C.3B · factorial_valuation

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.factorial_valuation`. Reflected implementation `komorebi_math.generating.padic_units.factorial_valuation`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-PADIC`.

Manifest signature (s).

```
(index: 'int', prime: 'int') -> 'int'
```

Definition. Return `v_prime(index!)` by Legendre's formula.

Contract (D, C, H, R, X). Valuation functions require a prime and valid integer or binomial indices. Prime-power compilers require positive precision and a modulus within the supplied budget. Unit-factorial and binomial methods distinguish valuation from the invertible unit part. Kernel compilers require compatible digit-unit tables and factorial exponents. Borrow counts use nonnegative indices with the subindex not exceeding the index where required.

Guarantees (E, P, B, K). Valuations, unit parts, residues, and survival decisions are exact for the declared prime power. PrimePowerBinomial separates the p-adic valuation from the unit residue, so zero modulo the prime power is represented without discarding valuation information. Precomputation cost depends on the prime-power modulus. Subsequent digit recursions scale with the number of base-prime digits.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27.

C.3B · finite_mask_carry_matrices

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.finite_mask_carry_matrices`. Reflected implementation `komorebi_math.generating.cartier.finite_mask_carry_matrices`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-CARTIER`.

Manifest signature (s).

```
(mask: 'Sequence[int]', prime: 'int', *, budget: 'GeneratingBudget' = GeneratingBudget(maximum_degree=100000,
    ↪ maximum_coefficient_bits=4000000, maximum_matrix_dimension=4096, maximum_reachable_states=100000,
    ↪ maximum_local_combinations=1000000, maximum_modulus=2000000, maximum_serialized_bytes=67108864,
    ↪ maximum_recursion_depth=1000)) -> 'tuple[tuple[tuple[int, ...], ...], ...]'
```

Definition. Return finite-mask carry matrices `T_d(q,q')=r[d+p*q'-q]`.

Contract (D, C, H, R, X). Polynomial operations require a modulus of at least two. Cartier operators and rational-mask compilers require a prime, a valid digit, compatible nonzero constant coefficients, and explicit state or degree budgets. Automata read least-significant-first prime digits. Minimization preserves output behavior on the compiled finite-state machine. Certificate routines reject mismatched transition systems or coefficient prefixes.

Guarantees (E, P, B, K). Polynomial arithmetic, Cartier transitions, automaton outputs, and linear representations are exact modulo the selected prime. State bounds are finite compilation bounds rather than asymptotic minimality claims. Rational-mask coefficient generation and automaton evaluation agree on the checked prefix. Complexity is polynomial in polynomial degree and compiled state count, with minimization depending on partition refinement over the full transition table.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime owner-

ship. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27.

C.3B · `first_frobenius_lift`

Identity (`q, a, t, k`). Public research manifest route `komorebi_math.generating.first_frobenius_lift`. Reflected implementation `komorebi_math.generating.masks.first_frobenius_lift`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-MASK`.

Manifest signature (`s`).

```
(mask: 'Sequence[int]', prime: 'int', degree: 'int') -> 'tuple[int, ...]'
```

Definition. Return `(R(z**p)/R(z)**p-1)/p` modulo `p` when integral.

Contract (`D, C, H, R, X`). Inputs describe a finite local mask, a radix of at least two, and an explicit truncation, modulus, exponent, or resource budget as required by the operation. Unit-series routines require a valid constant coefficient. Prime-power support routines require a prime and positive exponent. Carry plans validate every state and local index before allocation. Bounded solvers return a negative result when the declared search space contains no solution.

Guarantees (`E, P, B, K`). Finite coefficient identities, carry matrices, support layers, and polynomial certificates use exact integer or rational arithmetic. Analytic approximations separate the computed product from a stated tail bound. A finite functional equation or algebraic certificate applies only through its recorded degree. Direct mask products are at least quadratic in the truncation boundary, while carry-state plans depend on the explicit state dimension and budget.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_generating_block13.py` and `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27 and 64.

C.3B · `frobenius_defect_mask`

Identity (`q, a, t, k`). Public research manifest route `komorebi_math.generating.frobenius_defect_mask`. Reflected implementation `komorebi_math.generating.masks.frobenius_defect_mask`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-MASK`.

Manifest signature (`s`).

```
(mask: 'Iterable[Scalar]', prime: 'int', degree: 'int', *, budget: 'GeneratingBudget' =
    ↪ GeneratingBudget(maximum_degree=100000, maximum_coefficient_bits=4000000, maximum_matrix_dimension=4096,
    ↪ maximum_reachable_states=100000, maximum_local_combinations=1000000, maximum_modulus=2000000,
    ↪ maximum_serialized_bytes=67108864, maximum_recursion_depth=1000)) -> 'tuple[Fraction, ...]'
```

Definition. Return `R(z**p)/R(z)**p` through a finite degree.

Contract (`D, C, H, R, X`). Inputs describe a finite local mask, a radix of at least two, and an explicit truncation, modulus, exponent, or resource budget as required by the operation. Unit-series routines require a valid constant coefficient. Prime-power support routines require a prime and positive exponent. Carry plans validate every state and local index before allocation. Bounded solvers return a negative result when the declared search space contains no solution.

Guarantees (`E, P, B, K`). Finite coefficient identities, carry matrices, support layers, and polynomial certificates use exact integer or rational arithmetic. Analytic approximations separate the computed product from a stated tail bound. A finite functional equation or algebraic certificate applies only through its recorded degree. Direct mask products are at least quadratic in the truncation boundary, while carry-state plans depend on the explicit state dimension and budget.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_generating_block13.py` and `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27 and 64.

C.3B · `gaussian_binomial`

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.gaussian_binomial`. Reflected implementation `komorebi_math.generating.q_kummer.gaussian_binomial`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-QKUMMER`.

Manifest signature (s).

```
(index: 'int', subindex: 'int', base: 'int') -> 'int'
```

Definition. Return the integer Gaussian binomial evaluated at `base`.

Contract (D, C, H, R, X). The q-analogue routines require nonnegative indices, a valid subindex, an integer base, and a prime that does not violate the stated denominator or multiplicative-order conditions. Valuation decomposition requires a positive power and exponent where present. Support enumeration is finite and may be bounded by an explicit resource contract.

Guarantees (E, P, B, K). q-integers, q-factorials, Gaussian binomials, direct valuations, mixed-radix decompositions, and support sets are computed with exact integers. The decomposition records all factors used to reconstruct the valuation. Large Gaussian binomials can grow rapidly in bit length, so direct verification is bounded even when the structural decomposition remains compact.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27.

C.3B · `generalized_bell_digit_reduction`

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.generalized_bell_digit_reduction`. Reflected implementation `komorebi_math.generating.bell.generalized_bell_digit_reduction`. The same runtime object is also exported as `komorebi_math.generalized_bell_digit_reduction`. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-BELL`.

Manifest signature (s).

```
(index: 'int', prime: 'int', factorial_power: 'int', *, budget: 'GeneratingBudget' =
    ↪ GeneratingBudget(maximum_degree=100000, maximum_coefficient_bits=4000000, maximum_matrix_dimension=4096,
    ↪ maximum_reachable_states=100000, maximum_local_combinations=1000000, maximum_modulus=2000000,
    ↪ maximum_serialized_bytes=67108864, maximum_recursion_depth=1000)) -> 'BellDigitReduction'
```

Definition. Compute and certify the unweighted Bell digit theorem at one index.

Contract (D, C, H, R, X). Bell reductions require a prime, a nonnegative index, and valid factorial-power or weight data. Automata consume base-prime digits in the order stated by each method. Period bounds and pointed-subindex identities are finite arithmetic contracts. Weighted reductions require enough local weights for the requested computation. Resource budgets cap explicit polynomial and state construction.

Guarantees (E, P, B, K). The returned reductions, parity values, residue automata, and pointed polynomial certificates are exact modulo the declared prime. A certificate records the finite comparison it performed. The period bound is an upper bound, not a minimal-period claim. Direct polynomial construction can grow with the digit sum or index, while automaton evaluation is linear in the number of input digits.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_generating_foundation.py` and `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 22, 24, and 26.

C.3B · generalized_bell_parity

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.generalized_bell_parity`. Reflected implementation `komorebi_math.generating.bell.generalized_bell_parity`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-BELL`.

Manifest signature (s).

```
(index: 'int', factorial_power: 'int') -> 'int'
```

Definition. Return the exact parity predicted by the population-count automaton.

Contract (D, C, H, R, X). Bell reductions require a prime, a nonnegative index, and valid factorial-power or weight data. Automata consume base-prime digits in the order stated by each method. Period bounds and pointed-subindex identities are finite arithmetic contracts. Weighted reductions require enough local weights for the requested computation. Resource budgets cap explicit polynomial and state construction.

Guarantees (E, P, B, K). The returned reductions, parity values, residue automata, and pointed polynomial certificates are exact modulo the declared prime. A certificate records the finite comparison it performed. The period bound is an upper bound, not a minimal-period claim. Direct polynomial construction can grow with the digit sum or index, while automaton evaluation is linear in the number of input digits.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_generating_foundation.py` and `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 22, 24, and 26.

C.3B · integer_valuation

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.integer_valuation`. Reflected implementation `komorebi_math.generating.padic_units.integer_valuation`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-PADIC`.

Manifest signature (s).

```
(value: 'int', prime: 'int') -> 'int'
```

Definition. Return the finite p-adic valuation of a nonzero integer.

Contract (D, C, H, R, X). Valuation functions require a prime and valid integer or binomial indices. Prime-power compilers require positive precision and a modulus within the supplied budget. Unit-factorial and binomial methods distinguish valuation from the invertible unit part. Kernel compilers require compatible digit-unit tables and factorial exponents. Borrow counts use nonnegative indices with the subindex not exceeding the index where required.

Guarantees (E, P, B, K). Valuations, unit parts, residues, and survival decisions are exact for the declared prime power. PrimePowerBinomial separates the p-adic valuation from the unit residue, so zero modulo the prime power is represented without discarding valuation information. Precomputation cost depends on the prime-power modulus. Subsequent digit recursions scale with the number of base-prime digits.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime owner-

ship. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27.

C.3B · kummer_layer_distribution

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.kummer_layer_distribution`. Reflected implementation `komorebi_math.generating.masks.kummer_layer_distribution`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-MASK`.

Manifest signature (s).

```
(index: 'int', prime: 'int') -> 'KummerLayerDistribution'
```

Definition. Return the coefficient polynomial counting `v_p(binomial(index,k))`.

Contract (D, C, H, R, X). Inputs describe a finite local mask, a radix of at least two, and an explicit truncation, modulus, exponent, or resource budget as required by the operation. Unit-series routines require a valid constant coefficient. Prime-power support routines require a prime and positive exponent. Carry plans validate every state and local index before allocation. Bounded solvers return a negative result when the declared search space contains no solution.

Guarantees (E, P, B, K). Finite coefficient identities, carry matrices, support layers, and polynomial certificates use exact integer or rational arithmetic. Analytic approximations separate the computed product from a stated tail bound. A finite functional equation or algebraic certificate applies only through its recorded degree. Direct mask products are at least quadratic in the truncation boundary, while carry-state plans depend on the explicit state dimension and budget.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_generating_block13.py` and `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27 and 64.

C.3B · lift_digit_sequence

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.lift_digit_sequence`. Reflected implementation `komorebi_math.generating.digit_algebra.lift_digit_sequence`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-DIGIT`.

Manifest signature (s).

```
(digit_values: 'Sequence[int]', *, radix: 'int', modulus: 'int', count: 'int') -> 'tuple[int, ...]'
```

Definition. Lift a local table without coupling it to a particular kernel.

Contract (D, C, H, R, X). Digit kernels require a radix and modulus of at least two and one lower-triangular row for each digit. Global weights return zero when subtraction would borrow. Lifted sequences require one local value per digit. Factorial-power audits require a prime, positive power, and an explicit digit depth.

Guarantees (E, P, B, K). Local and global kernel weights, lifted sequences, and convolution audits are exact modulo the declared modulus. A finite DigitAlgebraAudit records the checked range and all failures. Digitwise evaluation is linear in the radix-digit width. Full closure audits scale with the size of the finite range and the number of tested subindices.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_generating_foundation.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 26.

C.3B · `local_normalized_kernel`

Identity (`q, a, t, k`). Public research manifest route `komorebi_math.generating.local_normalized_kernel`. Reflected implementation `komorebi_math.generating.normalization.local_normalized_kernel`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-NORMALIZATION`.

Manifest signature (`s`).

```
(total_digit: 'int', selected_digit: 'int', prime: 'int', factorial_power: 'int', unit_weights: 'Sequence[int]')
   ↪  -> 'int'
```

Definition. Return the no-carry local kernel for a digit-twisted factorial basis.

Contract (`D, C, H, R, X`). Digit-unit and normalized-kernel routines require a prime, nonnegative indices, and unit data compatible with the selected digit system. Carry-linear classification requires a callback with zero value at zero and a finite audit range large enough to recover the slope and carry weight. Invalid unit tables, primes, or audit ranges are rejected.

Guarantees (`E, P, B, K`). Local and digit-factorized normalized kernels are exact modulo the selected prime power. CarryLinearCertificate records the recovered profile and every finite mismatch. Passing the finite audit confirms the supplied range only. Kernel evaluation scales with the number of base-prime digits, while classification scales with the requested audit boundary.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 21, 24, and 26.

C.3B · `mahler_coefficients`

Identity (`q, a, t, k`). Public research manifest route `komorebi_math.generating.mahler_coefficients`. Reflected implementation `komorebi_math.generating.congruences.mahler_coefficients`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-CONGRUENCE`.

Manifest signature (`s`).

```
(values: 'Iterable[int | Fraction]') -> 'tuple[Fraction, ...]'
```

Definition. Return finite Newton/Mahler coefficients `Delta^r f(0)`.

Contract (`D, C, H, R, X`). Congruence audits accept finite prefixes, a valid prime or modulus, and explicit index, order, degree, or lane bounds. Rational valuations require denominators that are meaningful for the selected prime. Recurrence discovery and decimation search operate only within the declared finite model class. An empty or insufficient prefix returns an empty result or no candidate according to the individual signature.

Guarantees (`E, P, B, K`). Every reported residue, valuation, recurrence, lane, or failure tuple is computed exactly on the supplied finite range. A passing audit is evidence for that range and is not an unbounded theorem by itself. Berlekamp-Massey is exact over the selected prime field. Search cost grows with prefix length, lane count, recurrence order, and coefficient search space.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_generating_foundation.py` and `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27.

C.3B · mahler_mask_inverse

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.mahler_mask_inverse`. Reflected implementation `komorebi_math.generating.masks.mahler_mask_inverse`. The same runtime object is also exported as `komorebi_math.mahler_mask_inverse`. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-MASK`.

Manifest signature (s).

```
(series: 'Iterable[Scalar]', radix: 'int', degree: 'int | None' = None, *, budget: 'GeneratingBudget' =
    ↪ GeneratingBudget(maximum_degree=100000, maximum_coefficient_bits=4000000, maximum_matrix_dimension=4096,
    ↪ maximum_reachable_states=100000, maximum_local_combinations=1000000, maximum_modulus=2000000,
    ↪ maximum_serialized_bytes=67108864, maximum_recursion_depth=1000)) -> 'tuple[Fraction, ...]'
```

Definition. Recover the unique local mask `R(z)=F(z)/F(z**radix)`.

Contract (D, C, H, R, X). Inputs describe a finite local mask, a radix of at least two, and an explicit truncation, modulus, exponent, or resource budget as required by the operation. Unit-series routines require a valid constant coefficient. Prime-power support routines require a prime and positive exponent. Carry plans validate every state and local index before allocation. Bounded solvers return a negative result when the declared search space contains no solution.

Guarantees (E, P, B, K). Finite coefficient identities, carry matrices, support layers, and polynomial certificates use exact integer or rational arithmetic. Analytic approximations separate the computed product from a stated tail bound. A finite functional equation or algebraic certificate applies only through its recorded degree. Direct mask products are at least quadratic in the truncation boundary, while carry-state plans depend on the explicit state dimension and budget.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_generating_block13.py` and `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27 and 64.

C.3B · mahler_mask_product

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.mahler_mask_product`. Reflected implementation `komorebi_math.generating.masks.mahler_mask_product`. The same runtime object is also exported as `komorebi_math.mahler_mask_product`. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-MASK`.

Manifest signature (s).

```
(mask: 'Iterable[Scalar]', radix: 'int', degree: 'int', *, budget: 'GeneratingBudget' =
    ↪ GeneratingBudget(maximum_degree=100000, maximum_coefficient_bits=4000000, maximum_matrix_dimension=4096,
    ↪ maximum_reachable_states=100000, maximum_local_combinations=1000000, maximum_modulus=2000000,
    ↪ maximum_serialized_bytes=67108864, maximum_recursion_depth=1000)) -> 'tuple[Fraction, ...]'
```

Definition. Return `prod_(j>=0) R(z**radix**j)` through `z**degree`. Only finitely many mask levels affect a finite truncation. Arithmetic uses exact `fractions.Fraction` values.

Contract (D, C, H, R, X). Inputs describe a finite local mask, a radix of at least two, and an explicit truncation, modulus, exponent, or resource budget as required by the operation. Unit-series routines require a valid constant coefficient. Prime-power support routines require a prime and positive exponent. Carry plans validate every state and local index before allocation. Bounded solvers return a negative result when the declared search space contains no solution.

Guarantees (E, P, B, K). Finite coefficient identities, carry matrices, support layers, and polynomial certificates use exact integer or rational arithmetic. Analytic approximations separate the computed product from a stated tail bound. A finite functional equation or algebraic certificate applies only through its recorded degree. Direct mask products are at least quadratic in the truncation boundary, while carry-state plans depend on the explicit state dimension and budget.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_generating_block13.py` and `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27 and 64.

C.3B · `minimal_prefix_period`

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.minimal_prefix_period`. Reflected implementation `komorebi_math.generating.congruences.minimal_prefix_period`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-CONGRUENCE`.

Manifest signature (s).

```
(values: 'Sequence[int]', modulus: 'int', *, preperiod_limit: 'int' = 0) -> 'tuple[int, int] | None'
```

Definition. Return a finite-prefix preperiod/period certificate when visible.

Contract (D, C, H, R, X). Congruence audits accept finite prefixes, a valid prime or modulus, and explicit index, order, degree, or lane bounds. Rational valuations require denominators that are meaningful for the selected prime. Recurrence discovery and decimation search operate only within the declared finite model class. An empty or insufficient prefix returns an empty result or no candidate according to the individual signature.

Guarantees (E, P, B, K). Every reported residue, valuation, recurrence, lane, or failure tuple is computed exactly on the supplied finite range. A passing audit is evidence for that range and is not an unbounded theorem by itself. Berlekamp-Massey is exact over the selected prime field. Search cost grows with prefix length, lane count, recurrence order, and coefficient search space.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_generating_foundation.py` and `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27.

C.3B · `minimize_cartier_automaton`

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.minimize_cartier_automaton`. Reflected implementation `komorebi_math.generating.cartier.minimize_cartier_automaton`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-CARTIER`.

Manifest signature (s).

```
(automaton: 'CartierAutomaton') -> 'CartierAutomaton'
```

Definition. Minimize an output automaton by exact partition refinement.

Contract (D, C, H, R, X). Polynomial operations require a modulus of at least two. Cartier operators and rational-mask compilers require a prime, a valid digit, compatible nonzero constant coefficients, and explicit state or degree budgets. Automata read least-significant-first prime digits. Minimization preserves output behavior on the compiled finite-state machine. Certificate routines reject mismatched transition systems or coefficient prefixes.

Guarantees (E, P, B, K). Polynomial arithmetic, Cartier transitions, automaton outputs, and linear representations are exact modulo the selected prime. State bounds are finite compilation bounds rather than asymptotic minimality claims. Rational-mask coefficient generation and automaton evaluation agree on the checked prefix. Complexity is polynomial in polynomial degree and compiled state count, with minimization depending on partition refinement over the full transition table.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the

relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27.

C.3B · `multiplicative_order_mod_prime`

Identity (`q, a, t, k`). Public research manifest route `komorebi_math.generating.multiplicative_order_mod_prime`. Reflected implementation `komorebi_math.generating.q_kummer.multiplicative_order_mod_prime`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-QKUMMER`.

Manifest signature (`s`).

```
(value: 'int', prime: 'int') -> 'int'
```

Definition. Return the multiplicative order of `value` modulo `prime`.

Contract (`D, C, H, R, X`). The q-analogue routines require nonnegative indices, a valid subindex, an integer base, and a prime that does not violate the stated denominator or multiplicative-order conditions. Valuation decomposition requires a positive power and exponent where present. Support enumeration is finite and may be bounded by an explicit resource contract.

Guarantees (`E, P, B, K`). q-integers, q-factorials, Gaussian binomials, direct valuations, mixed-radix decompositions, and support sets are computed with exact integers. The decomposition records all factors used to reconstruct the valuation. Large Gaussian binomials can grow rapidly in bit length, so direct verification is bounded even when the structural decomposition remains compact.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27.

C.3B · `normalized_cyclotomic_expression`

Identity (`q, a, t, k`). Public research manifest route `komorebi_math.generating.normalized_cyclotomic_expression`. Reflected implementation `komorebi_math.generating.cyclotomic.normalized_cyclotomic_expression`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-CYCLOTOMIC`.

Manifest signature (`s`).

```
(exponents: 'Mapping[int, int]') -> 'Any'
```

Definition. Return a SymPy expression for normalized cyclotomic factors.

Contract (`D, C, H, R, X`). Cyclotomic routines require a prime and positive cyclotomic indices. Exponent maps omit zero entries after normalization. Symbolic expression and verification functions require the optional SymPy backend. A collapse result retains every p-free obstruction orbit and raises when the caller demands a closed form that is not present.

Guarantees (`E, P, B, K`). Orbit decomposition and exponent-lattice relations are exact. When no obstruction remains, the returned finite cyclotomic quotient is an exact formal collapse of the local Mahler product. Symbolic verification checks the declared relation rather than discovering a new one. Runtime is linear in the number of represented factors apart from optional symbolic simplification.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_generating_cartier_padic.py` and `tests/test_generating_block13.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 26.

C.3B · one_borrow_count

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.one_borrow_count`. Reflected implementation `komorebi_math.generating.masks.one_borrow_count`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-MASK`.

Manifest signature (s).

```
(index: 'int', prime: 'int') -> 'int'
```

Definition. Return the exact number of subindices producing exactly one borrow.

Contract (D, C, H, R, X). Inputs describe a finite local mask, a radix of at least two, and an explicit truncation, modulus, exponent, or resource budget as required by the operation. Unit-series routines require a valid constant coefficient. Prime-power support routines require a prime and positive exponent. Carry plans validate every state and local index before allocation. Bounded solvers return a negative result when the declared search space contains no solution.

Guarantees (E, P, B, K). Finite coefficient identities, carry matrices, support layers, and polynomial certificates use exact integer or rational arithmetic. Analytic approximations separate the computed product from a stated tail bound. A finite functional equation or algebraic certificate applies only through its recorded degree. Direct mask products are at least quadratic in the truncation boundary, while carry-state plans depend on the explicit state dimension and budget.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_generating_block13.py` and `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27 and 64.

C.3B · p_adic_valuation_fraction

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.p_adic_valuation_fraction`. Reflected implementation `komorebi_math.generating.congruences.p_adic_valuation_fraction`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-CONGRUENCE`.

Manifest signature (s).

```
(value: 'int | Fraction', prime: 'int') -> 'int | float'
```

Definition. Return the exact p-adic valuation of a rational, with infinity at zero.

Contract (D, C, H, R, X). Congruence audits accept finite prefixes, a valid prime or modulus, and explicit index, order, degree, or lane bounds. Rational valuations require denominators that are meaningful for the selected prime. Recurrence discovery and decimation search operate only within the declared finite model class. An empty or insufficient prefix returns an empty result or no candidate according to the individual signature.

Guarantees (E, P, B, K). Every reported residue, valuation, recurrence, lane, or failure tuple is computed exactly on the supplied finite range. A passing audit is evidence for that range and is not an unbounded theorem by itself. Berlekamp-Massey is exact over the selected prime field. Search cost grows with prefix length, lane count, recurrence order, and coefficient search space.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_generating_foundation.py` and `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27.

C.3B · p_free_orbit

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.p_free_orbit`. Reflected implementation `komorebi_math.generating.cyclotomic.p_free_orbit`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-CYCLOTOMIC`.

Manifest signature (s).

```
(index: 'int', prime: 'int') -> 'tuple[int, int]'
```

Definition. Return `(u,k)` with `index=u*prime**k` and `p` not dividing `u`.

Contract (D, C, H, R, X). Cyclotomic routines require a prime and positive cyclotomic indices. Exponent maps omit zero entries after normalization. Symbolic expression and verification functions require the optional SymPy backend. A collapse result retains every p-free obstruction orbit and raises when the caller demands a closed form that is not present.

Guarantees (E, P, B, K). Orbit decomposition and exponent-lattice relations are exact. When no obstruction remains, the returned finite cyclotomic quotient is an exact formal collapse of the local Mahler product. Symbolic verification checks the declared relation rather than discovering a new one. Runtime is linear in the number of represented factors apart from optional symbolic simplification.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_generating_cartier_padic.py` and `tests/test_generating_block13.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 26.

C.3B · pointed_subindex_polynomial

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.pointed_subindex_polynomial`. Reflected implementation `komorebi_math.generating.bell.pointed_subindex_polynomial`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-BELL`.

Manifest signature (s).

```
(index: 'int', prime: 'int') -> 'tuple[int, ...]'
```

Definition. Return the grouped left side of the pointed subindex identity. The coefficient of `u**j` is `sum binomial(n-1,m-1)` modulo `prime`, over positive digitwise subindices `m` with digit sum `j+1`.

Contract (D, C, H, R, X). Bell reductions require a prime, a nonnegative index, and valid factorial-power or weight data. Automata consume base-prime digits in the order stated by each method. Period bounds and pointed-subindex identities are finite arithmetic contracts. Weighted reductions require enough local weights for the requested computation. Resource budgets cap explicit polynomial and state construction.

Guarantees (E, P, B, K). The returned reductions, parity values, residue automata, and pointed polynomial certificates are exact modulo the declared prime. A certificate records the finite comparison it performed. The period bound is an upper bound, not a minimal-period claim. Direct polynomial construction can grow with the digit sum or index, while automaton evaluation is linear in the number of input digits.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_generating_foundation.py` and `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 22, 24, and 26.

C.3B · polynomial_cartier

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.polynomial_cartier`. Reflected implementation `komorebi_math.generating.cartier.polynomial_cartier`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-CARTIER`.

Manifest signature (s).

```
(polynomial: 'Sequence[int]', digit: 'int', prime: 'int') -> 'tuple[int, ...]'
```

Definition. Apply `Lambda_digit` to a polynomial modulo `prime`.

Contract (D, C, H, R, X). Polynomial operations require a modulus of at least two. Cartier operators and rational-mask compilers require a prime, a valid digit, compatible nonzero constant coefficients, and explicit state or degree budgets. Automata read least-significant-first prime digits. Minimization preserves output behavior on the compiled finite-state machine. Certificate routines reject mismatched transition systems or coefficient prefixes.

Guarantees (E, P, B, K). Polynomial arithmetic, Cartier transitions, automaton outputs, and linear representations are exact modulo the selected prime. State bounds are finite compilation bounds rather than asymptotic minimality claims. Rational-mask coefficient generation and automaton evaluation agree on the checked prefix. Complexity is polynomial in polynomial degree and compiled state count, with minimization depending on partition refinement over the full transition table.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27.

C.3B · polynomial_mask_coboundary

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.polynomial_mask_coboundary`. Reflected implementation `komorebi_math.generating.masks.polynomial_mask_coboundary`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-MASK`.

Manifest signature (s).

```
(candidate: 'Iterable[Scalar]', radix: 'int', degree: 'int | None' = None) -> 'tuple[Fraction, ...]'
```

Definition. Return the mask `Q(z)/Q(z**radix)` of a declared polynomial unit.

Contract (D, C, H, R, X). Inputs describe a finite local mask, a radix of at least two, and an explicit truncation, modulus, exponent, or resource budget as required by the operation. Unit-series routines require a valid constant coefficient. Prime-power support routines require a prime and positive exponent. Carry plans validate every state and local index before allocation. Bounded solvers return a negative result when the declared search space contains no solution.

Guarantees (E, P, B, K). Finite coefficient identities, carry matrices, support layers, and polynomial certificates use exact integer or rational arithmetic. Analytic approximations separate the computed product from a stated tail bound. A finite functional equation or algebraic certificate applies only through its recorded degree. Direct mask products are at least quadratic in the truncation boundary, while carry-state plans depend on the explicit state dimension and budget.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_generating_block13.py` and `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27 and 64.

C.3B · `polynomial_multiply_mod`

Identity (`q, a, t, k`). Public research manifest route `komorebi_math.generating.polynomial_multiply_mod`. Reflected implementation `komorebi_math.generating.cartier.polynomial_multiply_mod`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-CARTIER`.

Manifest signature (`s`).

```
(left: 'Sequence[int]', right: 'Sequence[int]', modulus: 'int') -> 'tuple[int, ...]'
```

Definition. Multiply coefficient tuples modulo `modulus`.

Contract (`D, C, H, R, X`). Polynomial operations require a modulus of at least two. Cartier operators and rational-mask compilers require a prime, a valid digit, compatible nonzero constant coefficients, and explicit state or degree budgets. Automata read least-significant-first prime digits. Minimization preserves output behavior on the compiled finite-state machine. Certificate routines reject mismatched transition systems or coefficient prefixes.

Guarantees (`E, P, B, K`). Polynomial arithmetic, Cartier transitions, automaton outputs, and linear representations are exact modulo the selected prime. State bounds are finite compilation bounds rather than asymptotic minimality claims. Rational-mask coefficient generation and automaton evaluation agree on the checked prefix. Complexity is polynomial in polynomial degree and compiled state count, with minimization depending on partition refinement over the full transition table.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27.

C.3B · `polynomial_power_mod`

Identity (`q, a, t, k`). Public research manifest route `komorebi_math.generating.polynomial_power_mod`. Reflected implementation `komorebi_math.generating.cartier.polynomial_power_mod`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-CARTIER`.

Manifest signature (`s`).

```
(polynomial: 'Sequence[int]', exponent: 'int', modulus: 'int') -> 'tuple[int, ...]'
```

Definition. Raise a polynomial to a nonnegative power modulo `modulus`.

Contract (`D, C, H, R, X`). Polynomial operations require a modulus of at least two. Cartier operators and rational-mask compilers require a prime, a valid digit, compatible nonzero constant coefficients, and explicit state or degree budgets. Automata read least-significant-first prime digits. Minimization preserves output behavior on the compiled finite-state machine. Certificate routines reject mismatched transition systems or coefficient prefixes.

Guarantees (`E, P, B, K`). Polynomial arithmetic, Cartier transitions, automaton outputs, and linear representations are exact modulo the selected prime. State bounds are finite compilation bounds rather than asymptotic minimality claims. Rational-mask coefficient generation and automaton evaluation agree on the checked prefix. Complexity is polynomial in polynomial degree and compiled state count, with minimization depending on partition refinement over the full transition table.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27.

C.3B · q_binomial_support

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.q_binomial_support`. Reflected implementation `komorebi_math.generating.q_kummer.q_binomial_support`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-QKUMMER`.

Manifest signature (s).

```
(index: 'int', base: 'int', prime: 'int', exponent: 'int', power: 'int' = 1) -> 'tuple[int, ...]'
```

Definition. Return subindices surviving in `GaussianBinomial**power mod p**exponent`.

Contract (D, C, H, R, X). The q-analogue routines require nonnegative indices, a valid subindex, an integer base, and a prime that does not violate the stated denominator or multiplicative-order conditions. Valuation decomposition requires a positive power and exponent where present. Support enumeration is finite and may be bounded by an explicit resource contract.

Guarantees (E, P, B, K). q-integers, q-factorials, Gaussian binomials, direct valuations, mixed-radix decompositions, and support sets are computed with exact integers. The decomposition records all factors used to reconstruct the valuation. Large Gaussian binomials can grow rapidly in bit length, so direct verification is bounded even when the structural decomposition remains compact.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27.

C.3B · q_binomial_valuation_decomposition

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.q_binomial_valuation_decomposition`. Reflected implementation `komorebi_math.generating.q_kummer.q_binomial_valuation_decomposition`. The same runtime object is also exported as `komorebi_math.q_binomial_valuation_decomposition`. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-QKUMMER`.

Manifest signature (s).

```
(index: 'int', subindex: 'int', base: 'int', prime: 'int') -> 'QKummerDecomposition'
```

Definition. Return the exact mixed-radix valuation decomposition. If `prime` divides `base`, every Gaussian binomial is a unit modulo `prime` and the valuation is zero. For an odd prime coprime to `base`, the block length is `ord_prime(base)` and the usual order-block formula applies. For `prime == 2` and odd `base`, the block length is two and the lift constant is `v_2(base + 1)`.

Contract (D, C, H, R, X). The q-analogue routines require nonnegative indices, a valid subindex, an integer base, and a prime that does not violate the stated denominator or multiplicative-order conditions. Valuation decomposition requires a positive power and exponent where present. Support enumeration is finite and may be bounded by an explicit resource contract.

Guarantees (E, P, B, K). q-integers, q-factorials, Gaussian binomials, direct valuations, mixed-radix decompositions, and support sets are computed with exact integers. The decomposition records all factors used to reconstruct the valuation. Large Gaussian binomials can grow rapidly in bit length, so direct verification is bounded even when the structural decomposition remains compact.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27.

C.3B · q_binomial_valuation_direct

Identity (q, a, t, k). Public research manifest route komorebi_math.generating.q_binomial_valuation_direct. Reflected implementation komorebi_math.generating.q_kummer.q_binomial_valuation_direct. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by [RESEARCH] policy pre-beta-additive. Manifest kind function. Profile C3-ARITH-QKUMMER.

Manifest signature (s).

```
(index: 'int', subindex: 'int', base: 'int', prime: 'int') -> 'int'
```

Definition. Compute the valuation directly from the integer Gaussian binomial.

Contract (D, C, H, R, X). The q-analogue routines require nonnegative indices, a valid subindex, an integer base, and a prime that does not violate the stated denominator or multiplicative-order conditions. Valuation decomposition requires a positive power and exponent where present. Support enumeration is finite and may be bounded by an explicit resource contract.

Guarantees (E, P, B, K). q-integers, q-factorials, Gaussian binomials, direct valuations, mixed-radix decompositions, and support sets are computed with exact integers. The decomposition records all factors used to reconstruct the valuation. Large Gaussian binomials can grow rapidly in bit length, so direct verification is bounded even when the structural decomposition remains compact.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is tests/test_generating_cartier_padic.py. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27.

C.3B · q_factorial

Identity (q, a, t, k). Public research manifest route komorebi_math.generating.q_factorial. Reflected implementation komorebi_math.generating.q_kummer.q_factorial. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by [RESEARCH] policy pre-beta-additive. Manifest kind function. Profile C3-ARITH-QKUMMER.

Manifest signature (s).

```
(index: 'int', base: 'int') -> 'int'
```

Definition. Return the integer q-factorial.

Contract (D, C, H, R, X). The q-analogue routines require nonnegative indices, a valid subindex, an integer base, and a prime that does not violate the stated denominator or multiplicative-order conditions. Valuation decomposition requires a positive power and exponent where present. Support enumeration is finite and may be bounded by an explicit resource contract.

Guarantees (E, P, B, K). q-integers, q-factorials, Gaussian binomials, direct valuations, mixed-radix decompositions, and support sets are computed with exact integers. The decomposition records all factors used to reconstruct the valuation. Large Gaussian binomials can grow rapidly in bit length, so direct verification is bounded even when the structural decomposition remains compact.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is tests/test_generating_cartier_padic.py. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27.

C.3B · q_integer

Identity (q, a, t, k). Public research manifest route komorebi_math.generating.q_integer. Reflected implementation komorebi_math.generating.q_kummer.q_integer. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by [RESEARCH] policy pre-beta-additive. Manifest kind function. Profile C3-ARITH-QKUMMER.

Manifest signature (s).

```
(index: 'int', base: 'int') -> 'int'
```

Definition. Return the integer q-analogue [index]_base.

Contract (D, C, H, R, X). The q-analogue routines require nonnegative indices, a valid subindex, an integer base, and a prime that does not violate the stated denominator or multiplicative-order conditions. Valuation decomposition requires a positive power and exponent where present. Support enumeration is finite and may be bounded by an explicit resource contract.

Guarantees (E, P, B, K). q-integers, q-factorials, Gaussian binomials, direct valuations, mixed-radix decompositions, and support sets are computed with exact integers. The decomposition records all factors used to reconstruct the valuation. Large Gaussian binomials can grow rapidly in bit length, so direct verification is bounded even when the structural decomposition remains compact.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is tests/test_generating_cartier_padic.py. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27.

C.3B · rational_mask_coefficients_mod_prime

Identity (q, a, t, k). Public research manifest route komorebi_math.generating.rational_mask_coefficients_mod_prime. Reflected implementation komorebi_math.generating.cartier.rational_mask_coefficients_mod_prime. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by [RESEARCH] policy pre-beta-additive. Manifest kind function. Profile C3-ARITH-CARTIER.

Manifest signature (s).

```
(numerator: 'Sequence[int]', denominator: 'Sequence[int]', prime: 'int', count: 'int') -> 'tuple[int, ...]'
```

Definition. Return the first count coefficients of the Mahler solution modulo prime.

Contract (D, C, H, R, X). Polynomial operations require a modulus of at least two. Cartier operators and rational-mask compilers require a prime, a valid digit, compatible nonzero constant coefficients, and explicit state or degree budgets. Automata read least-significant-first prime digits. Minimization preserves output behavior on the compiled finite-state machine. Certificate routines reject mismatched transition systems or coefficient prefixes.

Guarantees (E, P, B, K). Polynomial arithmetic, Cartier transitions, automaton outputs, and linear representations are exact modulo the selected prime. State bounds are finite compilation bounds rather than asymptotic minimality claims. Rational-mask coefficient generation and automaton evaluation agree on the checked prefix. Complexity is polynomial in polynomial degree and compiled state count, with minimization depending on partition refinement over the full transition table.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is tests/test_generating_cartier_padic.py. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27.

C.3B · scan_residue_lanes

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.scan_residue_lanes`. Reflected implementation `komorebi_math.generating.congruences.scan_residue_lanes`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-CONGRUENCE`.

Manifest signature (s).

```
(values: 'Sequence[int]', *, modulus: 'int', lane_modulus: 'int', minimum_terms: 'int' = 8) ->
    ↪  'tuple[ResidueLaneFinding, ...]'
```

Definition. Find residue lanes that are constant modulo `modulus` on the prefix.

Contract (D, C, H, R, X). Congruence audits accept finite prefixes, a valid prime or modulus, and explicit index, order, degree, or lane bounds. Rational valuations require denominators that are meaningful for the selected prime. Recurrence discovery and decimation search operate only within the declared finite model class. An empty or insufficient prefix returns an empty result or no candidate according to the individual signature.

Guarantees (E, P, B, K). Every reported residue, valuation, recurrence, lane, or failure tuple is computed exactly on the supplied finite range. A passing audit is evidence for that range and is not an unbounded theorem by itself. Berlekamp-Massey is exact over the selected prime field. Search cost grows with prefix length, lane count, recurrence order, and coefficient search space.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_generating_foundation.py` and `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27.

C.3B · search_decimation_relation

Identity (q, a, t, k). Public research manifest route `komorebi_math.generating.search_decimation_relation`. Reflected implementation `komorebi_math.generating.congruences.search_decimation_relation`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-CONGRUENCE`.

Manifest signature (s).

```
(values: 'Sequence[int]', *, prime: 'int', residue: 'int', max_order: 'int' = 8, holdout: 'int' = 12) ->
    ↪  'DecimationRelation | None'
```

Definition. Search `a[p*n+r]=sum_j c_j a[n+j] (mod p)`.

Contract (D, C, H, R, X). Congruence audits accept finite prefixes, a valid prime or modulus, and explicit index, order, degree, or lane bounds. Rational valuations require denominators that are meaningful for the selected prime. Recurrence discovery and decimation search operate only within the declared finite model class. An empty or insufficient prefix returns an empty result or no candidate according to the individual signature.

Guarantees (E, P, B, K). Every reported residue, valuation, recurrence, lane, or failure tuple is computed exactly on the supplied finite range. A passing audit is evidence for that range and is not an unbounded theorem by itself. Berlekamp-Massey is exact over the selected prime field. Search cost grows with prefix length, lane count, recurrence order, and coefficient search space.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_generating_foundation.py` and `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27.

C.3B · `solve_polynomial_coboundary`

Identity (`q, a, t, k`). Public research manifest route `komorebi_math.generating.solve_polynomial_coboundary`. Reflected implementation `komorebi_math.generating.masks.solve_polynomial_coboundary`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-MASK`.

Manifest signature (`s`).

```
(numerator: 'Iterable[Scalar]', denominator: 'Iterable[Scalar]', radix: 'int', degree_bound: 'int', *, budget:
    ↪ 'GeneratingBudget' = GeneratingBudget(maximum_degree=100000, maximum_coefficient_bits=4000000,
    ↪ maximum_matrix_dimension=4096, maximum_reachable_states=100000, maximum_local_combinations=1000000,
    ↪ maximum_modulus=2000000, maximum_serialized_bytes=67108864, maximum_recursion_depth=1000)) ->
    ↪ 'PolynomialCoboundaryResult'
```

Definition. Solve `B(z)Q(z)=A(z)Q(z**radix)` over the rationals.

Contract (`D, C, H, R, X`). Inputs describe a finite local mask, a radix of at least two, and an explicit truncation, modulus, exponent, or resource budget as required by the operation. Unit-series routines require a valid constant coefficient. Prime-power support routines require a prime and positive exponent. Carry plans validate every state and local index before allocation. Bounded solvers return a negative result when the declared search space contains no solution.

Guarantees (`E, P, B, K`). Finite coefficient identities, carry matrices, support layers, and polynomial certificates use exact integer or rational arithmetic. Analytic approximations separate the computed product from a stated tail bound. A finite functional equation or algebraic certificate applies only through its recorded degree. Direct mask products are at least quadratic in the truncation boundary, while carry-state plans depend on the explicit state dimension and budget.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_generating_block13.py` and `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27 and 64.

C.3B · `verify_cyclotomic_coboundary`

Identity (`q, a, t, k`). Public research manifest route `komorebi_math.generating.verify_cyclotomic_coboundary`. Reflected implementation `komorebi_math.generating.cyclotomic.verify_cyclotomic_coboundary`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-CYCLOTOMIC`.

Manifest signature (`s`).

```
(collapse: 'CyclotomicCollapse') -> 'bool'
```

Definition. Symbolically verify the closed form when the obstruction vanishes.

Contract (`D, C, H, R, X`). Cyclotomic routines require a prime and positive cyclotomic indices. Exponent maps omit zero entries after normalization. Symbolic expression and verification functions require the optional SymPy backend. A collapse result retains every p-free obstruction orbit and raises when the caller demands a closed form that is not present.

Guarantees (`E, P, B, K`). Orbit decomposition and exponent-lattice relations are exact. When no obstruction remains, the returned finite cyclotomic quotient is an exact formal collapse of the local Mahler product. Symbolic verification checks the declared relation rather than discovering a new one. Runtime is linear in the number of represented factors apart from optional symbolic simplification.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_generating_cartier_padic.py` and `tests/test_generating_block13.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite cer-

tificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 26.

C.3B · `verify_cyclotomic_exponent_relation`

Identity (`q, a, t, k`). Public research manifest route `komorebi_math.generating.verify_cyclotomic_exponent_relation`. Reflected implementation `komorebi_math.generating.cyclotomic.verify_cyclotomic_exponent_relation`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-CYCLOTOMIC`.

Manifest signature (`s`).

```
(collapse: 'CyclotomicCollapse') -> 'bool'
```

Definition. Verify `R=Q/Q(z**p)` directly in the cyclotomic divisor lattice.

Contract (`D, C, H, R, X`). Cyclotomic routines require a prime and positive cyclotomic indices. Exponent maps omit zero entries after normalization. Symbolic expression and verification functions require the optional SymPy backend. A collapse result retains every p-free obstruction orbit and raises when the caller demands a closed form that is not present.

Guarantees (`E, P, B, K`). Orbit decomposition and exponent-lattice relations are exact. When no obstruction remains, the returned finite cyclotomic quotient is an exact formal collapse of the local Mahler product. Symbolic verification checks the declared relation rather than discovering a new one. Runtime is linear in the number of represented factors apart from optional symbolic simplification.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_generating_cartier_padic.py` and `tests/test_generating_block13.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 26.

C.3B · `weighted_bell_digit_reduction`

Identity (`q, a, t, k`). Public research manifest route `komorebi_math.generating.weighted_bell_digit_reduction`. Reflected implementation `komorebi_math.generating.bell.weighted_bell_digit_reduction`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-BELL`.

Manifest signature (`s`).

```
(index: 'int', prime: 'int', factorial_power: 'int', digit_weights: 'Iterable[int]', *, budget: 'GeneratingBudget'
    ↪ = GeneratingBudget(maximum_degree=100000, maximum_coefficient_bits=4000000, maximum_matrix_dimension=4096,
    ↪ maximum_reachable_states=100000, maximum_local_combinations=1000000, maximum_modulus=2000000,
    ↪ maximum_serialized_bytes=67108864, maximum_recursion_depth=1000)) -> 'BellDigitReduction'
```

Definition. Compute the weighted Bell digit-sum reduction at one index.

Contract (`D, C, H, R, X`). Bell reductions require a prime, a nonnegative index, and valid factorial-power or weight data. Automata consume base-prime digits in the order stated by each method. Period bounds and pointed-subindex identities are finite arithmetic contracts. Weighted reductions require enough local weights for the requested computation. Resource budgets cap explicit polynomial and state construction.

Guarantees (`E, P, B, K`). The returned reductions, parity values, residue automata, and pointed polynomial certificates are exact modulo the declared prime. A certificate records the finite comparison it performed. The period bound is an upper bound, not a minimal-period claim. Direct polynomial construction can grow with the digit sum or index, while automaton evaluation is linear in the number of input digits.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_generating_foundation.py` and `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite

certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 22, 24, and 26.

C.3B · `zero_borrow_count`

Identity (`q, a, t, k`). Public research manifest route `komorebi_math.generating.zero_borrow_count`. Reflected implementation `komorebi_math.generating.masks.zero_borrow_count`. In the reference manifest, this symbol is exported only from the generating namespace. The generating route is governed by `[RESEARCH]` policy `pre-beta-additive`. Manifest kind `function`. Profile `C3-ARITH-MASK`.

Manifest signature (`s`).

```
(index: 'int', prime: 'int') -> 'int'
```

Definition. Return the exact number of no-borrow subindices.

Contract (`D, C, H, R, X`). Inputs describe a finite local mask, a radix of at least two, and an explicit truncation, modulus, exponent, or resource budget as required by the operation. Unit-series routines require a valid constant coefficient. Prime-power support routines require a prime and positive exponent. Carry plans validate every state and local index before allocation. Bounded solvers return a negative result when the declared search space contains no solution.

Guarantees (`E, P, B, K`). Finite coefficient identities, carry matrices, support layers, and polynomial certificates use exact integer or rational arithmetic. Analytic approximations separate the computed product from a stated tail bound. A finite functional equation or algebraic certificate applies only through its recorded degree. Direct mask products are at least quadratic in the truncation boundary, while carry-state plans depend on the explicit state dimension and budget.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_generating_block13.py` and `tests/test_generating_cartier_padic.py`. The independent verifier for arithmetic checks the relevant exact arithmetic identities, compiled representations, finite certificates, rejection paths, and runtime ownership. The source validator for arithmetic checks the exact signature, implementation identity, facade routes, record counts, and unique anchor. See Chapters 24 through 27 and 64.

C.4 `komorebi_math.signals`: 171 entries

C.4A Windows, convolution, filtering, resampling, and spectral transforms: 80 entries

This section establishes the finite signal-processing substrate used by the remaining wavelet, recovery, and packet contracts. It fixes the public type vocabularies, window and overlap conventions, direct and FFT convolution, linear filtering, rational resampling, framing, STFT analysis and synthesis, spectrogram scaling, power spectra, cross spectra, and bounded streaming state. The contracts are numerical and finite. They distinguish exact shape and index conventions from floating-point equality certificates.

Contract family	Cards
Shared type vocabularies and domain status	17
Windows and overlap lattices	6
Finite convolution and block matrices	9
FIR design and streaming FIR	11
Recursive FIR and IIR execution	6
Rational resampling	10
Framing, STFT, inverse STFT, and spectrograms	8
Periodograms, Welch estimates, and cross spectra	6
Incremental STFT execution	3
FFT and overlap-add convolution	4
Total	**80**

The boundary is explicit. A certificate checks one finite array identity, reconstruction, overlap law, stability predicate, or batch-versus-streaming comparison under the stored tolerance. Spectral estimates describe the supplied finite records and selected normalization. They do not infer an unobserved stochastic process. Window, framing, padding, sidedness, detrending, cropping, and rate-conversion conventions remain part of the public contract.

C.4A · `AdaptiveMethod`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.signals.AdaptiveMethod`. Reflected implementation `typing.Literal`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `data`. Profile `C4-SIGNAL-TYPES`.

Manifest signature (`s`).

```
(*args, **kwargs)
```

Definition. Literal method vocabulary accepted by adaptive filters: `lms`, `nlms`, or `rls`.

Contract (`D, C, H, R, X`). These exports are annotation vocabularies rather than executable validators. Callers should pass one of the listed literal values or one of the accepted union members. The consuming function performs the runtime shape, range, and compatibility checks. A type alias may be imported and inspected, but it is not a substitute for invoking the associated signal operation.

Guarantees (`E, P, B, K`). The aliases deterministically describe the accepted public vocabulary in the reference manifest. They make no numerical claim and allocate no signal state. Import and annotation lookup are constant-time in ordinary Python terms. Behavioral guarantees belong to the functions and records that consume each alias.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_block12_windows_time_frequency.py` and `tests/test_block12_spectral_filtering.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapters 31 through 36 and 72.

C.4A · `BiquadKind`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.signals.BiquadKind`. Reflected implementation `typing.Literal`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `data`. Profile `C4-SIGNAL-TYPES`.

Manifest signature (`s`).

```
(*args, **kwargs)
```

Definition. Literal biquad design vocabulary: `lowpass`, `highpass`, `bandpass`, `notch`, or `allpass`.

Contract (`D, C, H, R, X`). These exports are annotation vocabularies rather than executable validators. Callers should pass one of the listed literal values or one of the accepted union members. The consuming function performs the runtime shape, range, and compatibility checks. A type alias may be imported and inspected, but it is not a substitute for invoking the associated signal operation.

Guarantees (`E, P, B, K`). The aliases deterministically describe the accepted public vocabulary in the reference manifest. They make no numerical claim and allocate no signal state. Import and annotation lookup are constant-time in ordinary Python terms. Behavioral guarantees belong to the functions and records that consume each alias.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_block12_windows_time_frequency.py` and `tests/test_block12_spectral_filtering.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapters 31 through 36 and 72.

C.4A · BoundaryMode

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.BoundaryMode`. Reflected implementation `typing.Literal`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `data`. Profile `C4-SIGNAL-TYPES`.

Manifest signature (s).

```
(*args, **kwargs)
```

Definition. Literal framing boundary vocabulary: `zeros`, `reflect`, `edge`, or `none`.

Contract (D, C, H, R, X). These exports are annotation vocabularies rather than executable validators. Callers should pass one of the listed literal values or one of the accepted union members. The consuming function performs the runtime shape, range, and compatibility checks. A type alias may be imported and inspected, but it is not a substitute for invoking the associated signal operation.

Guarantees (E, P, B, K). The aliases deterministically describe the accepted public vocabulary in the reference manifest. They make no numerical claim and allocate no signal state. Import and annotation lookup are constant-time in ordinary Python terms. Behavioral guarantees belong to the functions and records that consume each alias.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_block12_windows_time_frequency.py` and `tests/test_block12_spectral_filtering.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapters 31 through 36 and 72.

C.4A · ConvolutionMode

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.ConvolutionMode`. Reflected implementation `typing.Literal`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `data`. Profile `C4-SIGNAL-TYPES`.

Manifest signature (s).

```
(*args, **kwargs)
```

Definition. Literal finite convolution vocabulary: `full`, `same`, `valid`, or `circular`.

Contract (D, C, H, R, X). These exports are annotation vocabularies rather than executable validators. Callers should pass one of the listed literal values or one of the accepted union members. The consuming function performs the runtime shape, range, and compatibility checks. A type alias may be imported and inspected, but it is not a substitute for invoking the associated signal operation.

Guarantees (E, P, B, K). The aliases deterministically describe the accepted public vocabulary in the reference manifest. They make no numerical claim and allocate no signal state. Import and annotation lookup are constant-time in ordinary Python terms. Behavioral guarantees belong to the functions and records that consume each alias.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_block12_windows_time_frequency.py` and `tests/test_block12_spectral_filtering.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapters 31 through 36 and 72.

C.4A · CrossSpectrum

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.CrossSpectrum`. Reflected implementation `komorebi_math.signals.spectral.CrossSpectrum`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C4-SPECTRAL`.

Manifest signature (s).

```
(frequencies: 'np.ndarray', cross_power: 'np.ndarray', input_power: 'np.ndarray', output_power: 'np.ndarray',
   ↪ coherence: 'np.ndarray', transfer_h1: 'np.ndarray', sample_rate: 'float', scaling: 'SpectralScaling',
   ↪ onesided: 'bool', fft_length: 'int', segment_count: 'int', certificate: 'Certificate') -> None
```

Definition. Averaged auto spectra, cross spectrum, coherence, and H1 transfer estimate.

Contract (D, C, H, R, X). Spectral estimators require finite one-dimensional records, a positive sample rate, a valid window and detrending mode, and compatible frame and overlap lengths. Cross spectra require equal-length inputs. One-sided output is reserved for real data. Regularization must be nonnegative and is applied explicitly where transfer ratios would otherwise be singular.

Guarantees (E, P, B, K). Parseval certificates compare the finite NumPy-normalized DFT energy identity. Periodogram and Welch records expose frequency resolution, dominant frequency, and integrated power under the chosen scaling. Cross-spectrum records contain auto spectra, cross spectrum, coherence, and the H1 estimate. Results are numerical finite estimates, not stochastic consistency theorems. FFT cost dominates.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_spectral_filtering.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapter 32.

C.4A · DetrendMode

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.DetrendMode`. Reflected implementation `typing.Literal`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `data`. Profile `C4-SIGNAL-TYPES`.

Manifest signature (s).

```
(*args, **kwargs)
```

Definition. Literal detrending vocabulary for finite frames: `none` or `constant`.

Contract (D, C, H, R, X). These exports are annotation vocabularies rather than executable validators. Callers should pass one of the listed literal values or one of the accepted union members. The consuming function performs the runtime shape, range, and compatibility checks. A type alias may be imported and inspected, but it is not a substitute for invoking the associated signal operation.

Guarantees (E, P, B, K). The aliases deterministically describe the accepted public vocabulary in the reference manifest. They make no numerical claim and allocate no signal state. Import and annotation lookup are constant-time in ordinary Python terms. Behavioral guarantees belong to the functions and records that consume each alias.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_block12_windows_time_frequency.py` and `tests/test_block12_spectral_filtering.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapters 31 through 36 and 72.

C.4A · FEATURE_KEY

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.FEATURE_KEY`. Reflected implementation `builtins.str`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `data`. Profile `C4-DOMAIN-STATUS`.

Manifest signature (s).

```
null
```

Definition. The registry key `signals` used to query this domain's implementation status.

Contract (D, C, H, R, X). The feature key is the literal string `signals`. `status()` queries the package registry for that key and returns the registered `FeatureSpec`. The call does not run signal processing, import an optional accelerator, or qualify a backend.

Guarantees (E, P, B, K). The result is deterministic package metadata for the installed source tree. Lookup is constant-time in ordinary Python terms and makes no independent mathematical or numerical claim.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The domain registry path is exercised by the independent verifier for signal foundations. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapters 4, 31, and 72.

C.4A · FIRFilter

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.FIRFilter`. Reflected implementation `komorebi_math.signals.filtering.FIRFilter`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C4-FIR`.

Manifest signature (s).

```
(taps: 'np.ndarray', kind: 'FIRKind', cutoff: 'tuple[float, ...]', sample_rate: 'float' = 1.0, window_name: 'str'
   ↪  = 'custom', reference_frequency: 'float | None' = None, reference_gain: 'float' = 1.0) -> None
```

Definition. A finite linear-phase FIR design and its declared design metadata.

Contract (D, C, H, R, X). FIR design requires a positive odd tap count, a positive sample rate, and cutoff frequencies strictly inside the Nyquist interval. Band edges must be ordered. Filtering requires a finite nonempty signal and tap vector. The selected FIR mode fixes cropping. Streaming execution preserves only the bounded causal history and rejects processing after reaching a terminal state where applicable. The principal public operations are `order`, `group_delay_samples`, `apply`, `response`, `certificate`.

Guarantees (E, P, B, K). Windowed ideal-response designs are real and symmetric, with the documented normalization and group delay. Frequency responses are sampled finite Fourier sums. Streaming and batch causal FIR execution agree to numerical precision for the same dtype and chunk order. Direct filtering scales with sample count times tap count and stores tap-count history.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_spectral_filtering.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapter 33.

C.4A · FIRKind

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.FIRKind`. Reflected implementation `typing.Literal`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `data`. Profile `C4-SIGNAL-TYPES`.

Manifest signature (s).

```
(*args, **kwargs)
```

Definition. Literal FIR design vocabulary: `lowpass`, `highpass`, `bandpass`, or `bandstop`.

Contract (D, C, H, R, X). These exports are annotation vocabularies rather than executable validators. Callers should pass one of the listed literal values or one of the accepted union members. The consuming function performs the runtime shape, range, and compatibility checks. A type alias may be imported and inspected, but it is not a substitute for invoking the associated signal operation.

Guarantees (E, P, B, K). The aliases deterministically describe the accepted public vocabulary in the reference manifest. They make no numerical claim and allocate no signal state. Import and annotation lookup are constant-time in ordinary Python terms. Behavioral guarantees belong to the functions and records that consume each alias.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_block12_windows_time_frequency.py` and `tests/test_block12_spectral_filtering.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapters 31 through 36 and 72.

C.4A · FIRMode

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.FIRMode`. Reflected implementation `typing.Literal`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `data`. Profile `C4-SIGNAL-TYPES`.

Manifest signature (s).

```
(*args, **kwargs)
```

Definition. Literal FIR execution vocabulary: `causal`, `full`, `same`, or `valid`.

Contract (D, C, H, R, X). These exports are annotation vocabularies rather than executable validators. Callers should pass one of the listed literal values or one of the accepted union members. The consuming function performs the runtime shape, range, and compatibility checks. A type alias may be imported and inspected, but it is not a substitute for invoking the associated signal operation.

Guarantees (E, P, B, K). The aliases deterministically describe the accepted public vocabulary in the reference manifest. They make no numerical claim and allocate no signal state. Import and annotation lookup are constant-time in ordinary Python terms. Behavioral guarantees belong to the functions and records that consume each alias.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_block12_windows_time_frequency.py` and `tests/test_block12_spectral_filtering.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapters 31 through 36 and 72.

C.4A · FilterOutput

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.FilterOutput`. Reflected implementation `komorebi_math.signals.recursive.FilterOutput`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C4-RECURSIVE`.

Manifest signature (s).

```
(samples: 'np.ndarray', final_state: 'np.ndarray') -> None
```

Definition. Filtered samples paired with the final direct-form state.

Contract (D, C, H, R, X). Numerator and denominator coefficient arrays must be finite and nonempty, and the leading denominator coefficient must be nonzero. Initial state length must equal the normalized filter order. Biquad frequency and quality parameters must satisfy the documented physical range. Streaming execution uses one fixed normalized model until reset.

Guarantees (E, P, B, K). The implementation follows direct-form II transposed recursion after denominator normalization. Batch and chunked execution use the same recurrence and therefore agree to numerical precision. Pole-based stability certificates check the finite coefficient model rather than an external plant. Runtime is linear in sample count times filter order, with linear state storage.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_recursive_fft.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapter 33.

C.4A · FinitePolyphaseKernel

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.FinitePolyphaseKernel`. Reflected implementation `komorebi_math.signals.convolution.PolyphaseBlockMatrix`. No root, stable, or beta facade exports this domain-only name in the reference manifest. Within `komorebi_math.signals`, `PolyphaseBlockMatrix` resolves to the same runtime object. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C4-CONVOLUTION`.

Manifest signature (s).

```
(blocks: 'tuple[tuple[np.ndarray, ...], ...]', order: 'int', input_length: 'int', output_length: 'int') -> None
```

Definition. Finite packet blocks of a convolution matrix.

Contract (D, C, H, R, X). Signals and kernels are finite one-dimensional arrays, with nonempty inputs where convolution is required. The selected mode fixes the output convention. Circular convolution requires a compatible finite length. Matrix and polyphase constructors require positive dimensions and a positive packet order. Every block retains explicit source and target index sets.

Guarantees (E, P, B, K). The direct operator, matrix representation, and finite polyphase blocks implement the same declared finite convolution law up to the numerical behavior of the input dtype. Matrix construction exposes the full linear map and can require quadratic storage. Direct convolution scales with signal length times kernel length, while block application has the corresponding partitioned cost.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_block4_hardening.py` and `tests/test_signals_polyphase_filterbanks.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapter 33.

C.4A · FrameBatch

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.FrameBatch`. Reflected implementation `komorebi_math.signals.time_frequency.FrameBatch`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C4-TIME-FREQUENCY`.

Manifest signature (s).

```
(frames: 'np.ndarray', starts: 'np.ndarray', original_length: 'int', frame_length: 'int', hop_length: 'int',
    ↪ pad_left: 'int', pad_right: 'int', boundary: 'BoundaryMode') -> None
```

Definition. A deterministic finite framing of a one-dimensional signal.

Contract (D, C, H, R, X). Input is a finite one-dimensional signal. Frame length, hop length, and FFT length must be positive, with the hop no larger than the frame and the FFT no shorter than the frame. One-sided transforms require real input. Boundary, centering, padding, detrending, and scaling choices are explicit. Detrended transforms are intentionally noninvertible. Inverse synthesis requires a nonzero overlap profile. The principal public operations are `frame_count`, `padded_length`, `covered_length`, `certificate`.

Guarantees (E, P, B, K). Framing metadata records every finite pad, start, and coverage convention. STFT coefficients use NumPy FFT normalization. Invertible configurations reconstruct the original finite record to numerical tolerance through normalized overlap-add. Spectrogram values are nonnegative for magnitude and power forms. Runtime is proportional to the number of frames times the FFT cost.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_windows_time_frequency.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapters 31 and 32.

C.4A · FrequencyResponse

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.FrequencyResponse`. Reflected implementation `komorebi_math.signals.filtering.FrequencyResponse`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C4-FIR`.

Manifest signature (s).

```
(frequencies: 'np.ndarray', response: 'np.ndarray', sample_rate: 'float') -> None
```

Definition. Sampled FIR transfer values on a physical frequency grid.

Contract (D, C, H, R, X). FIR design requires a positive odd tap count, a positive sample rate, and cutoff frequencies strictly inside the Nyquist interval. Band edges must be ordered. Filtering requires a finite nonempty signal and tap vector. The selected FIR mode fixes cropping. Streaming execution preserves only the bounded causal history and rejects processing after reaching a terminal state where applicable. The principal public operations are `magnitude`, `phase`, `magnitude_db`.

Guarantees (E, P, B, K). Windowed ideal-response designs are real and symmetric, with the documented normalization and group delay. Frequency responses are sampled finite Fourier sums. Streaming and batch causal FIR execution agree to numerical precision for the same dtype and chunk order. Direct filtering scales with sample count times tap count and stores tap-count history.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_spectral_filtering.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapter 33.

C.4A · LinearFilter

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.LinearFilter`. Reflected implementation `komorebi_math.signals.recursive.LinearFilter`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C4-RECURSIVE`.

Manifest signature (s).

```
(numerator: 'Any', denominator: 'Any' = (1.0,)) -> 'None'
```

Definition. Normalized causal FIR/IIR filter in direct-form II transposed form.

Contract (D, C, H, R, X). Numerator and denominator coefficient arrays must be finite and nonempty, and the leading denominator coefficient must be nonzero. Initial state length must equal the normalized filter order. Biquad frequency and quality parameters must satisfy the documented physical range. Streaming execution uses one fixed normalized model until reset. The principal public operations are `order`, `zeros`, `poles`, `process`, `response`, `stability_certificate`.

Guarantees (E, P, B, K). The implementation follows direct-form II transposed recursion after denominator normalization. Batch and chunked execution use the same recurrence and therefore agree to numerical precision. Pole-based stability certificates check the finite coefficient model rather than an external plant. Runtime is linear in sample count times filter order, with linear state storage.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_recursive_fft.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapter 33.

C.4A · OverlapAddConvolver

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.OverlapAddConvolver`. Reflected implementation `komorebi_math.signals.fft_convolution.OverlapAddConvolver`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C4-FFT-CONVOLUTION`.

Manifest signature (s).

```
(kernel: 'np.ndarray', block_length: 'int' = 1024) -> None
```

Definition. Stateful bounded-input-buffer FFT overlap-add convolver.

Contract (D, C, H, R, X). Signal and kernel arrays must be finite, one-dimensional, and nonempty. FFT and overlap-add execution require a positive block length. The output mode applies the same finite cropping convention as direct convolution. A stateful convolver accepts chunks until `flush()`, emits the remaining tail once, and requires `reset()` before reuse after finalization. The principal public operations are `buffered_samples`, `process`, `flush`, `reset`.

Guarantees (E, P, B, K). Zero-padded FFT convolution and bounded-buffer overlap-add agree with direct finite convolution to numerical precision. The certified wrapper records that comparison. FFT execution reduces the asymptotic cost for long records and kernels to blockwise transform cost, while the stateful object stores the kernel spectrum, overlap tail, and one bounded input buffer.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_recursive_fft.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapter 33.

C.4A · PacketPath

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.PacketPath`. Reflected implementation `types.UnionType`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `data`. Profile `C4-SIGNAL-TYPES`.

Manifest signature (s).

```
null
```

Definition. Wavelet packet path identifier represented by a string or a tuple of integer branch indices.

Contract (D, C, H, R, X). These exports are annotation vocabularies rather than executable validators. Callers should pass one of the listed literal values or one of the accepted union members. The consuming function performs the runtime shape, range, and compatibility checks. A type alias may be imported and inspected, but it is not a substitute for invoking the associated signal operation.

Guarantees (E, P, B, K). The aliases deterministically describe the accepted public vocabulary in the reference manifest. They make no numerical claim and allocate no signal state. Import and annotation lookup are constant-time in ordinary Python terms. Behavioral guarantees belong to the functions and records that consume each alias.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_block12_windows_time_frequency.py` and `tests/test_block12_spectral_filtering.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapters 31 through 36 and 72.

C.4A · PolyphaseBlockMatrix

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.PolyphaseBlockMatrix`. Reflected implementation `komorebi_math.signals.convolution.PolyphaseBlockMatrix`. No root, stable, or beta facade exports this domain-only name in the reference manifest. Within `komorebi_math.signals`, `FinitePolyphaseKernel` resolves to the same runtime object. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C4-CONVOLUTION`.

Manifest signature (s).

```
(blocks: 'tuple[tuple[np.ndarray, ...], ...]', order: 'int', input_length: 'int', output_length: 'int') -> None
```

Definition. Finite packet blocks of a convolution matrix.

Contract (D, C, H, R, X). Signals and kernels are finite one-dimensional arrays, with nonempty inputs where convolution is required. The selected mode fixes the output convention. Circular convolution requires a compatible finite length. Matrix and polyphase constructors require positive dimensions and a positive packet order. Every block retains explicit source and target index sets.

Guarantees (E, P, B, K). The direct operator, matrix representation, and finite polyphase blocks implement the same declared finite convolution law up to the numerical behavior of the input dtype. Matrix construction exposes the full linear map and can require quadratic storage. Direct convolution scales with signal length times kernel length, while block application has the corresponding partitioned cost.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_block4_hardening.py` and `tests/test_signals_polyphase_filterbanks.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapter 33.

C.4A · PolyphaseConvolution

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.PolyphaseConvolution`. Reflected implementation `komorebi_math.signals.convolution.PolyphaseConvolution`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C4-CONVOLUTION`.

Manifest signature (s).

```
(kernel: 'np.ndarray', input_length: 'int', order: 'int', mode: 'ConvolutionMode' = 'full') -> None
```

Definition. Exact packet block representation of a finite convolution operator.

Contract (D, C, H, R, X). Signals and kernels are finite one-dimensional arrays, with nonempty inputs where convolution is required. The selected mode fixes the output convention. Circular convolution requires a compatible finite length. Matrix and polyphase constructors require positive dimensions and a positive packet order. Every block retains explicit source and target index sets. The principal public operations are `matrix`, `output_length`, `source_indices`, `target_indices`, `blocks`, `apply_coordinates`, `apply`, `certificate`.

Guarantees (E, P, B, K). The direct operator, matrix representation, and finite polyphase blocks implement the same declared finite convolution law up to the numerical behavior of the input dtype. Matrix construction exposes the full linear map and can require quadratic storage. Direct convolution scales with signal length times kernel length, while block application has the corresponding partitioned cost.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_block4_hardening.py` and `tests/test_signals_polyphase_filterbanks.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapter 33.

C.4A · PowerSpectrum

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.PowerSpectrum`. Reflected implementation `komorebi_math.signals.spectral.PowerSpectrum`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C4-SPECTRAL`.

Manifest signature (s).

```
(frequencies: 'np.ndarray', power: 'np.ndarray', sample_rate: 'float', scaling: 'SpectralScaling', onesided:
    ↪ 'bool', fft_length: 'int', segment_count: 'int', frame_length: 'int', hop_length: 'int', window:
    ↪ 'np.ndarray', method: 'str', certificate: 'Certificate') -> None
```

Definition. A finite periodogram or averaged power spectral estimate.

Contract (D, C, H, R, X). Spectral estimators require finite one-dimensional records, a positive sample rate, a valid window and detrending mode, and compatible frame and overlap lengths. Cross spectra require equal-length inputs. One-sided output is reserved for real data. Regularization must be nonnegative and is applied explicitly where transfer ratios would otherwise be singular. The principal public operations are `frequency_resolution`, `dominant_frequency`, `integrated_power`.

Guarantees (E, P, B, K). Parseval certificates compare the finite NumPy-normalized DFT energy identity. Periodogram and Welch records expose frequency resolution, dominant frequency, and integrated power under the chosen scaling. Cross-spectrum records contain auto spectra, cross spectrum, coherence, and the H1 estimate. Results are numerical finite estimates, not stochastic consistency theorems. FFT cost dominates.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_spectral_filtering.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapter 32.

C.4A · RationalResamplingPlan

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.RationalResamplingPlan`. Reflected implementation `komorebi_math.signals.resampling.RationalResamplingPlan`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C4-RESAMPLING`.

Manifest signature (s).

```
(up: 'int', down: 'int', taps: 'np.ndarray') -> None
```

Definition. A centered zero-extension rational-resampling plan.

Contract (D, C, H, R, X). Upsampling and downsampling factors must be positive integers. A downsampling phase must select a valid residue class. Rational plans reduce the rate ratio, use finite centered zero extension, and require a finite nonempty input. A supplied antialiasing kernel must satisfy the plan's finite shape and gain requirements. Default design uses an odd symmetric low-pass kernel. The principal public operations are `delay`, `polyphase_components`, `output_length`, `apply`, `explicit_reference`, `certificate`.

Guarantees (E, P, B, K). The output length is exactly `ceil(input_length * up / down)`. Plan certificates compare polyphase execution with an explicit zero-insertion, convolution, delay, and decimation reference to numerical tolerance. Primitive upsampling and downsampling are exact array operations. Polyphase runtime scales with output length and the active taps per phase.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_resampling_adaptive.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapter 34.

C.4A · ResamplingResult

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.ResamplingResult`. Reflected implementation `komorebi_math.signals.resampling.ResamplingResult`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C4-RESAMPLING`.

Manifest signature (s).

```
(signal: 'np.ndarray', plan: 'RationalResamplingPlan', input_length: 'int', certificate: 'Certificate') -> None
```

Definition. A finite rationally resampled signal paired with its execution plan and certificate.

Contract (D, C, H, R, X). Upsampling and downsampling factors must be positive integers. A downsampling phase must select a valid residue class. Rational plans reduce the rate ratio, use finite centered zero extension, and require a finite nonempty input. A supplied antialiasing kernel must satisfy the plan's finite shape and gain requirements. Default design uses an odd symmetric low-pass kernel.

Guarantees (E, P, B, K). The output length is exactly `ceil(input_length * up / down)`. Plan certificates compare polyphase execution with an explicit zero-insertion, convolution, delay, and decimation reference to numerical tolerance. Primitive upsampling and downsampling are exact array operations. Polyphase runtime scales with output length and the active taps per phase.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_resampling_adaptive.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapter 34.

C.4A · STFTResult

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.STFTResult`. Reflected implementation `komorebi_math.signals.time_frequency.STFTResult`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C4-TIME-FREQUENCY`.

Manifest signature (s).

```
(coefficients: 'np.ndarray', frequencies: 'np.ndarray', times: 'np.ndarray', sample_rate: 'float', frame_length:
   ↪ 'int', hop_length: 'int', fft_length: 'int', analysis_window: 'np.ndarray', original_length: 'int',
   ↪ pad_left: 'int', pad_right: 'int', onesided: 'bool', boundary: 'BoundaryMode', detrend: 'DetrendMode') ->
   ↪ None
```

Definition. Finite short-time Fourier coefficients and complete synthesis metadata.

Contract (D, C, H, R, X). Input is a finite one-dimensional signal. Frame length, hop length, and FFT length must be positive, with the hop no larger than the frame and the FFT no shorter than the frame. One-sided transforms require real input. Boundary, centering, padding, detrending, and scaling choices are explicit. Detrended transforms are intentionally noninvertible. Inverse synthesis requires a nonzero overlap profile. The principal public operations are `frame_count`, `inverse`, `roundtrip_certificate`.

Guarantees (E, P, B, K). Framing metadata records every finite pad, start, and coverage convention. STFT coefficients use NumPy FFT normalization. Invertible configurations reconstruct the original finite record to numerical tolerance through normalized overlap-add. Spectrogram values are nonnegative for magnitude and power forms. Runtime is proportional to the number of frames times the FFT cost.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_windows_time_frequency.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapters 31 and 32.

C.4A · SpectralAverage

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.SpectralAverage`. Reflected implementation `typing.Literal`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `data`. Profile `C4-SIGNAL-TYPES`.

Manifest signature (s).

```
(*args, **kwargs)
```

Definition. Literal Welch averaging vocabulary: `mean` or `median`.

Contract (D, C, H, R, X). These exports are annotation vocabularies rather than executable validators. Callers should pass one of the listed literal values or one of the accepted union members. The consuming function performs the runtime shape, range, and compatibility checks. A type alias may be imported and inspected, but it is not a substitute for invoking the associated signal operation.

Guarantees (E, P, B, K). The aliases deterministically describe the accepted public vocabulary in the reference manifest. They make no numerical claim and allocate no signal state. Import and annotation lookup are constant-time in ordinary Python terms. Behavioral guarantees belong to the functions and records that consume each alias.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_block12_windows_time_frequency.py` and `tests/test_block12_spectral_filtering.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapters 31 through 36 and 72.

C.4A · SpectralScaling

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.SpectralScaling`. Reflected implementation `typing.Literal`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `data`. Profile `C4-SIGNAL-TYPES`.

Manifest signature (s).

```
(*args, **kwargs)
```

Definition. Literal power spectrum scaling vocabulary: `density` or `spectrum`.

Contract (D, C, H, R, X). These exports are annotation vocabularies rather than executable validators. Callers should pass one of the listed literal values or one of the accepted union members. The consuming function performs the runtime shape, range, and compatibility checks. A type alias may be imported and inspected, but it is not a substitute for invoking the associated signal operation.

Guarantees (E, P, B, K). The aliases deterministically describe the accepted public vocabulary in the reference manifest. They make no numerical claim and allocate no signal state. Import and annotation lookup are constant-time in ordinary Python terms. Behavioral guarantees belong to the functions and records that consume each alias.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_block12_windows_time_frequency.py` and `tests/test_block12_spectral_filtering.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapters 31 through 36 and 72.

C.4A · SpectrogramResult

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.SpectrogramResult`. Reflected implementation `komorebi_math.signals.time_frequency.SpectrogramResult`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C4-TIME-FREQUENCY`.

Manifest signature (s).

```
(values: 'np.ndarray', frequencies: 'np.ndarray', times: 'np.ndarray', scaling: 'SpectrogramScaling', sample_rate:
    ↪ 'float', transform: 'STFTResult') -> None
```

Definition. A finite time-frequency magnitude or power representation.

Contract (D, C, H, R, X). Input is a finite one-dimensional signal. Frame length, hop length, and FFT length must be positive, with the hop no larger than the frame and the FFT no shorter than the frame. One-sided transforms require real input. Boundary, centering, padding, detrending, and scaling choices are explicit. Detrended transforms are intentionally noninvertible. Inverse synthesis requires a nonzero overlap profile.

Guarantees (E, P, B, K). Framing metadata records every finite pad, start, and coverage convention. STFT coefficients use NumPy FFT normalization. Invertible configurations reconstruct the original finite record to numerical tolerance through normalized overlap-add. Spectrogram values are nonnegative for magnitude and power forms. Runtime is proportional to the number of frames times the FFT cost.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_windows_time_frequency.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapters 31 and 32.

C.4A · SpectrogramScaling

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.SpectrogramScaling`. Reflected implementation `typing.Literal`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `data`. Profile `C4-SIGNAL-TYPES`.

Manifest signature (s).

```
(*args, **kwargs)
```

Definition. Literal spectrogram scaling vocabulary: `density`, `spectrum`, `magnitude`, or `power`.

Contract (D, C, H, R, X). These exports are annotation vocabularies rather than executable validators. Callers should pass one of the listed literal values or one of the accepted union members. The consuming function performs the runtime shape, range, and compatibility checks. A type alias may be imported and inspected, but it is not a substitute for invoking the associated signal operation.

Guarantees (E, P, B, K). The aliases deterministically describe the accepted public vocabulary in the reference manifest. They make no numerical claim and allocate no signal state. Import and annotation lookup are constant-time in ordinary Python terms. Behavioral guarantees belong to the functions and records that consume each alias.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_block12_windows_time_frequency.py` and `tests/test_block12_spectral_filtering.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapters 31 through 36 and 72.

C.4A · StreamingFIR

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.StreamingFIR`. Reflected implementation `komorebi_math.signals.filtering.StreamingFIR`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C4-FIR`.

Manifest signature (s).

```
(taps: 'Any') -> 'None'
```

Definition. Stateful causal FIR execution with bounded input history.

Contract (D, C, H, R, X). FIR design requires a positive odd tap count, a positive sample rate, and cutoff frequencies strictly inside the Nyquist interval. Band edges must be ordered. Filtering requires a finite nonempty signal and tap vector. The selected FIR mode fixes cropping. Streaming execution preserves only the bounded causal history and rejects processing after reaching a terminal state where applicable. The principal public operations are `process`, `flush`, `reset`.

Guarantees (E, P, B, K). Windowed ideal-response designs are real and symmetric, with the documented normalization and group delay. Frequency responses are sampled finite Fourier sums. Streaming and batch causal FIR execution agree to numerical precision for the same dtype and chunk order. Direct filtering scales with sample count times tap count and stores tap-count history.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_spectral_filtering.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapter 33.

C.4A · StreamingLinearFilter

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.StreamingLinearFilter`. Reflected implementation `komorebi_math.signals.recursive.StreamingLinearFilter`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C4-RECURSIVE`.

Manifest signature (s).

```
(filter: 'LinearFilter') -> None
```

Definition. Mutable chunked execution of one finite-order linear filter.

Contract (D, C, H, R, X). Numerator and denominator coefficient arrays must be finite and nonempty, and the leading denominator coefficient must be nonzero. Initial state length must equal the normalized filter order. Biquad frequency and quality parameters must satisfy the documented physical range. Streaming execution uses one fixed normalized model until reset. The principal public operations are `process`, `reset`.

Guarantees (E, P, B, K). The implementation follows direct-form II transposed recursion after denominator normalization. Batch and chunked execution use the same recurrence and therefore agree to numerical precision. Pole-based stability certificates check the finite coefficient model rather than an external plant. Runtime is linear in sample count times filter order, with linear state storage.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_recursive_fft.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapter 33.

C.4A · StreamingSTFT

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.StreamingSTFT`. Reflected implementation `komorebi_math.signals.streaming.StreamingSTFT`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C4-STREAMING-STFT`.

Manifest signature (s).

```
(frame_length: 'int' = 256, hop_length: 'int | None' = None, fft_length: 'int | None' = None, window: 'WindowSpec'
   ↪  = 'hann', sample_rate: 'float' = 1.0, onesided: 'bool' = True, detrend: 'DetrendMode' = 'none') -> None
```

Definition. Incremental finite STFT analysis with bounded overlap memory.

Contract (D, C, H, R, X). A streaming STFT instance fixes its frame, hop, FFT, window, boundary, and sidedness configuration at construction. Chunks must be one-dimensional and compatible with the selected real or complex mode. `flush()` finalizes optional end padding. Processing after finalization is rejected until `reset()` is called. The convenience wrapper requires chunk sizes that cover the supplied record exactly. The principal public operations are `frequencies`, `buffered_samples`, `total_samples`, `reset`, `process`, `flush`.

Guarantees (E, P, B, K). The processor stores only the unresolved overlap and framing state. Emitted frames are consecutive and use the same transform convention as batch `stft()`. The wrapper certificate compares concatenated streaming output with the matching batch transform to numerical tolerance. Work per emitted frame is one FFT plus bounded buffer movement.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_denoise_streaming.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapter 32.

C.4A · StreamingSTFTBatch

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.StreamingSTFTBatch`. Reflected implementation `komorebi_math.signals.streaming.StreamingSTFTBatch`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C4-STREAMING-STFT`.

Manifest signature (s).

```
(coefficients: 'np.ndarray', frequencies: 'np.ndarray', times: 'np.ndarray', starts: 'np.ndarray', sample_rate:
   ↪  'float', frame_length: 'int', hop_length: 'int', fft_length: 'int', onesided: 'bool') -> None
```

Definition. Zero or more consecutive STFT frames emitted by one streaming operation.

Contract (D, C, H, R, X). A streaming STFT instance fixes its frame, hop, FFT, window, boundary, and sidedness configuration at construction. Chunks must be one-dimensional and compatible with the selected real or complex mode. `flush()` finalizes optional end padding. Processing after finalization is rejected until `reset()` is called. The convenience wrapper requires chunk sizes that cover the supplied record exactly. The principal public operations are `frame_count`.

Guarantees (E, P, B, K). The processor stores only the unresolved overlap and framing state. Emitted frames are consecutive and use the same transform convention as batch `stft()`. The wrapper certificate compares concatenated streaming output with the matching batch transform to numerical tolerance. Work per emitted frame is one FFT plus bounded buffer movement.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_denoise_streaming.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapter 32.

C.4A · ThresholdMode

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.ThresholdMode`. Reflected implementation `typing.Literal`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `data`. Profile `C4-SIGNAL-TYPES`.

Manifest signature (s).

```
(*args, **kwargs)
```

Definition. Literal coefficient thresholding vocabulary: `soft`, `hard`, or `garrote`.

Contract (D, C, H, R, X). These exports are annotation vocabularies rather than executable validators. Callers should pass one of the listed literal values or one of the accepted union members. The consuming function performs the runtime shape, range, and compatibility checks. A type alias may be imported and inspected, but it is not a substitute for invoking the associated signal operation.

Guarantees (E, P, B, K). The aliases deterministically describe the accepted public vocabulary in the reference manifest. They make no numerical claim and allocate no signal state. Import and annotation lookup are constant-time in ordinary Python terms. Behavioral guarantees belong to the functions and records that consume each alias.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_block12_windows_time_frequency.py` and `tests/test_block12_spectral_filtering.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapters 31 through 36 and 72.

C.4A · ThresholdRule

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.ThresholdRule`. Reflected implementation `typing.Literal`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `data`. Profile `C4-SIGNAL-TYPES`.

Manifest signature (s).

```
(*args, **kwargs)
```

Definition. Literal threshold selection vocabulary: `universal`, `sure`, or `fixed`.

Contract (D, C, H, R, X). These exports are annotation vocabularies rather than executable validators. Callers should pass one of the listed literal values or one of the accepted union members. The consuming function performs the runtime shape, range, and compatibility checks. A type alias may be imported and inspected, but it is not a substitute for invoking the associated signal operation.

Guarantees (E, P, B, K). The aliases deterministically describe the accepted public vocabulary in the reference manifest. They make no numerical claim and allocate no signal state. Import and annotation lookup are constant-time in ordinary Python terms. Behavioral guarantees belong to the functions and records that consume each alias.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_block12_windows_time_frequency.py` and `tests/test_block12_spectral_filtering.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapters 31 through 36 and 72.

C.4A · WindowMetrics

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.WindowMetrics`. Reflected implementation `komorebi_math.signals.windows.WindowMetrics`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C4-WINDOWS`.

Manifest signature (s).

```
(length: 'int', coherent_gain: 'float', power_gain: 'float', equivalent_noise_bandwidth_bins: 'float',
    ↪ equivalent_noise_bandwidth_hz: 'float') -> None
```

Definition. Finite gain and equivalent-noise-bandwidth diagnostics for a window.

Contract (D, C, H, R, X). Window length and hop values must be positive integers. Parameterized Kaiser and Tukey windows validate their finite parameters. An explicit array must be one-dimensional and have exactly the requested length. Overlap certificates require a compatible shift lattice. Canonical dual construction returns an invalid result when the overlap profile contains a zero.

Guarantees (E, P, B, K). Named windows, gain metrics, overlap profiles, and dual windows are deterministic finite NumPy computations. Certificates state finite overlap or constant-overlap identities to a declared tolerance. Window construction and metric evaluation are linear in window length. Overlap profiles also scale with the number of hop residues.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_windows_time_frequency.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapter 31 and Chapter 32.

C.4A · WindowName

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.WindowName`. Reflected implementation `typing.Literal`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `data`. Profile `C4-SIGNAL-TYPES`.

Manifest signature (s).

```
(*args, **kwargs)
```

Definition. Literal named-window vocabulary for boxcar, rectangular, Hann, Hamming, Blackman, Bartlett, Kaiser, and Tukey windows.

Contract (D, C, H, R, X). These exports are annotation vocabularies rather than executable validators. Callers should pass one of the listed literal values or one of the accepted union members. The consuming function performs the runtime shape, range, and compatibility checks. A type alias may be imported and inspected, but it is not a substitute for invoking the associated signal operation.

Guarantees (E, P, B, K). The aliases deterministically describe the accepted public vocabulary in the reference manifest. They make no numerical claim and allocate no signal state. Import and annotation lookup are constant-time in ordinary Python terms. Behavioral guarantees belong to the functions and records that consume each alias.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_block12_windows_time_frequency.py` and `tests/test_block12_spectral_filtering.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapters 31 through 36 and 72.

C.4A · WindowSpec

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.WindowSpec`. Reflected implementation `typing.Union`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `data`. Profile `C4-SIGNAL-TYPES`.

Manifest signature (s).

```
(*args, **kwargs)
```

Definition. Accepted window specification union: a supported name, a string alias, a parameter tuple, or an explicit NumPy array.

Contract (D, C, H, R, X). These exports are annotation vocabularies rather than executable validators. Callers should pass one of the listed literal values or one of the accepted union members. The consuming function performs the runtime shape, range, and compatibility checks. A type alias may be imported and inspected, but it is not a substitute for invoking the associated signal operation.

Guarantees (E, P, B, K). The aliases deterministically describe the accepted public vocabulary in the reference manifest. They make no numerical claim and allocate no signal state. Import and annotation lookup are constant-time in ordinary Python terms. Behavioral guarantees belong to the functions and records that consume each alias.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_block12_windows_time_frequency.py` and `tests/test_block12_spectral_filtering.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapters 31 through 36 and 72.

C.4A · canonical_dual_window

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.canonical_dual_window`. Reflected implementation `komorebi_math.signals.windows.canonical_dual_window`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-WINDOWS`.

Manifest signature (s).

```
(window: 'Any', hop_length: 'int', *, tolerance: 'float' = 1e-12) -> 'ComputationResult[np.ndarray]'
```

Definition. Return the canonical synthesis window for the declared shift lattice.

Contract (D, C, H, R, X). Window length and hop values must be positive integers. Parameterized Kaiser and Tukey windows validate their finite parameters. An explicit array must be one-dimensional and have exactly the requested length. Overlap certificates require a compatible shift lattice. Canonical dual construction returns an invalid result when the overlap profile contains a zero.

Guarantees (E, P, B, K). Named windows, gain metrics, overlap profiles, and dual windows are deterministic finite NumPy computations. Certificates state finite overlap or constant-overlap identities to a declared tolerance. Window construction and metric evaluation are linear in window length. Overlap profiles also scale with the number of hop residues.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_windows_time_frequency.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapter 31 and Chapter 32.

C.4A · certified_overlap_add_convolve

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.certified_overlap_add_convolve`. Reflected implementation `komorebi_math.signals.fft_convolution.certified_overlap_add_convolve`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-FFT-CONVOLUTION`.

Manifest signature (s).

```
(signal: 'Any', kernel: 'Any', *, block_length: 'int' = 1024, mode: 'FIRMode' = 'full', tolerance: 'float' = 1e-11)
   ↪  -> 'ComputationResult[np.ndarray]'
```

Definition. Return overlap-add output with a direct-convolution equality certificate.

Contract (D, C, H, R, X). Signal and kernel arrays must be finite, one-dimensional, and nonempty. FFT and overlap-add execution require a positive block length. The output mode applies the same finite cropping convention as direct convolution. A stateful convolver accepts chunks until `flush()`, emits the remaining tail once, and requires `reset()` before reuse after finalization.

Guarantees (E, P, B, K). Zero-padded FFT convolution and bounded-buffer overlap-add agree with direct finite convolution to numerical precision. The certified wrapper records that comparison. FFT execution reduces the asymptotic cost for long records and kernels to blockwise transform cost, while the stateful object stores the kernel spectrum, overlap tail, and one bounded input buffer.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_recursive_fft.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapter 33.

C.4A · convolution_matrix

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.convolution_matrix`. Reflected implementation `komorebi_math.signals.convolution.convolution_matrix`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-CONVOLUTION`.

Manifest signature (s).

```
(kernel: 'Any', input_length: 'int', *, mode: 'ConvolutionMode' = 'full') -> 'np.ndarray'
```

Definition. Return the matrix implementing a finite convolution convention.

Contract (D, C, H, R, X). Signals and kernels are finite one-dimensional arrays, with nonempty inputs where convolution is required. The selected mode fixes the output convention. Circular convolution requires a compatible finite length. Matrix and polyphase constructors require positive dimensions and a positive packet order. Every block retains explicit source and target index sets.

Guarantees (E, P, B, K). The direct operator, matrix representation, and finite polyphase blocks implement the same declared finite convolution law up to the numerical behavior of the input dtype. Matrix construction exposes the full linear map and can require quadratic storage. Direct convolution scales with signal length times kernel length, while block application has the corresponding partitioned cost.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_block4_hardening.py` and `tests/test_signals_polyphase_filterbanks.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapter 33.

C.4A · convolution_output_length

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.convolution_output_length`. Reflected implementation `komorebi_math.signals.convolution.convolution_output_length`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-CONVOLUTION`.

Manifest signature (s).

```
(input_length: 'int', kernel_length: 'int', mode: 'ConvolutionMode') -> 'int'
```

Definition. Return the finite one dimensional convolution output length.

Contract (D, C, H, R, X). Signals and kernels are finite one-dimensional arrays, with nonempty inputs where convolution is required. The selected mode fixes the output convention. Circular convolution requires a compatible finite length. Matrix and polyphase constructors require positive dimensions and a positive packet order. Every block retains explicit source and target index sets.

Guarantees (E, P, B, K). The direct operator, matrix representation, and finite polyphase blocks implement the same declared finite convolution law up to the numerical behavior of the input dtype. Matrix construction exposes the full linear map and can require quadratic storage. Direct convolution scales with signal length times kernel length, while block application has the corresponding partitioned cost.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_block4_hardening.py` and `tests/test_signals_polyphase_filterbanks.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapter 33.

C.4A · convolve_signal

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.convolve_signal`. Reflected implementation `komorebi_math.signals.convolution.convolve_signal`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-CONVOLUTION`.

Manifest signature (s).

```
(signal: 'Any', kernel: 'Any', *, mode: 'ConvolutionMode' = 'full') -> 'np.ndarray'
```

Definition. Apply the declared finite convolution convention.

Contract (D, C, H, R, X). Signals and kernels are finite one-dimensional arrays, with nonempty inputs where convolution is required. The selected mode fixes the output convention. Circular convolution requires a compatible finite length. Matrix and polyphase constructors require positive dimensions and a positive packet order. Every block retains explicit source and target index sets.

Guarantees (E, P, B, K). The direct operator, matrix representation, and finite polyphase blocks implement the same declared finite convolution law up to the numerical behavior of the input dtype. Matrix construction exposes the full linear map and can require quadratic storage. Direct convolution scales with signal length times kernel length, while block application has the corresponding partitioned cost.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_block4_hardening.py` and `tests/test_signals_polyphase_filterbanks.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapter 33.

C.4A · cross_spectrum

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.cross_spectrum`. Reflected implementation `komorebi_math.signals.spectral.cross_spectrum`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-SPECTRAL`.

Manifest signature (s).

```
(input_signal: 'Any', output_signal: 'Any', *, sample_rate: 'float' = 1.0, frame_length: 'int' = 256, hop_length:
    ↪ 'int | None' = None, fft_length: 'int | None' = None, window: 'WindowSpec' = 'hann', detrend: 'DetrendMode'
    ↪ = 'constant', scaling: 'SpectralScaling' = 'density', average: 'SpectralAverage' = 'mean', onesided: 'bool |
    ↪ None' = None, regularization: 'float' = 0.0) -> 'CrossSpectrum'
```

Definition. Estimate paired auto spectra, cross spectrum, coherence, and H1 transfer.

Contract (D, C, H, R, X). Spectral estimators require finite one-dimensional records, a positive sample rate, a valid window and detrending mode, and compatible frame and overlap lengths. Cross spectra require equal-length inputs. One-sided output is reserved for real data. Regularization must be nonnegative and is applied explicitly where transfer ratios would otherwise be singular.

Guarantees (E, P, B, K). Parseval certificates compare the finite NumPy-normalized DFT energy identity. Periodogram and Welch records expose frequency resolution, dominant frequency, and integrated power under the chosen scaling. Cross-spectrum records contain auto spectra, cross spectrum, coherence, and the H1 estimate. Results are numerical finite estimates, not stochastic consistency theorems. FFT cost dominates.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_spectral_filtering.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapter 32.

C.4A · decimate

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.decimate`. Reflected implementation `komorebi_math.signals.resampling.decimate`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-RESAMPLING`.

Manifest signature (s).

```
(signal: 'Any', factor: 'int', *, num_taps: 'int | None' = None, window: 'WindowSpec' = ('kaiser', 8.0)) ->
    ↪ 'np.ndarray'
```

Definition. Low-pass filter and downsample a finite signal by an integer factor.

Contract (D, C, H, R, X). Upsampling and downsampling factors must be positive integers. A downsampling phase must select a valid residue class. Rational plans reduce the rate ratio, use finite centered zero extension, and require a finite nonempty input. A supplied antialiasing kernel must satisfy the plan's finite shape and gain requirements. Default design uses an odd symmetric low-pass kernel.

Guarantees (E, P, B, K). The output length is exactly `ceil(input_length * up / down)`. Plan certificates compare polyphase execution with an explicit zero-insertion, convolution, delay, and decimation reference to numerical tolerance. Primitive upsampling and downsampling are exact array operations. Polyphase runtime scales with output length and the active taps per phase.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_resampling_adaptive.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapter 34.

C.4A · design_bandpass

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.design_bandpass`. Reflected implementation `komorebi_math.signals.filtering.design_bandpass`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-FIR`.

Manifest signature (s).

```
(cutoff: 'tuple[float, float]', **options: 'Any') -> 'FIRFilter'
```

Definition. Design a bandpass FIR filter.

Contract (D, C, H, R, X). FIR design requires a positive odd tap count, a positive sample rate, and cutoff frequencies strictly inside the Nyquist interval. Band edges must be ordered. Filtering requires a finite nonempty signal and tap vector. The selected FIR mode fixes cropping. Streaming execution preserves only the bounded causal history and rejects processing after reaching a terminal state where applicable.

Guarantees (E, P, B, K). Windowed ideal-response designs are real and symmetric, with the documented normalization and group delay. Frequency responses are sampled finite Fourier sums. Streaming and batch causal FIR execution agree to numerical precision for the same dtype and chunk order. Direct filtering scales with sample count times tap count and stores tap-count history.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_spectral_filtering.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapter 33.

C.4A · design_bandstop

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.design_bandstop`. Reflected implementation `komorebi_math.signals.filtering.design_bandstop`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-FIR`.

Manifest signature (s).

```
(cutoff: 'tuple[float, float]', **options: 'Any') -> 'FIRFilter'
```

Definition. Design a bandstop FIR filter.

Contract (D, C, H, R, X). FIR design requires a positive odd tap count, a positive sample rate, and cutoff frequencies strictly inside the Nyquist interval. Band edges must be ordered. Filtering requires a finite nonempty signal and tap vector. The selected FIR mode fixes cropping. Streaming execution preserves only the bounded causal history and rejects processing after reaching a terminal state where applicable.

Guarantees (E, P, B, K). Windowed ideal-response designs are real and symmetric, with the documented normalization and group delay. Frequency responses are sampled finite Fourier sums. Streaming and batch causal FIR execution agree to numerical precision for the same dtype and chunk order. Direct filtering scales with sample count times tap count and stores tap-count history.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_spectral_filtering.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapter 33.

C.4A · design_biquad

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.design_biquad`. Reflected implementation `komorebi_math.signals.recursive.design_biquad`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-RECURSIVE`.

Manifest signature (s).

```
(kind: 'BiquadKind', frequency: 'float', *, q: 'float' = np.float64(0.7071067811865475), sample_rate: 'float' =
    ↪ 1.0) -> 'LinearFilter'
```

Definition. Design a lowpass, highpass, bandpass, notch, or allpass biquad.

Contract (D, C, H, R, X). Numerator and denominator coefficient arrays must be finite and nonempty, and the leading denominator coefficient must be nonzero. Initial state length must equal the normalized filter order. Biquad frequency and quality parameters must satisfy the documented physical range. Streaming execution uses one fixed normalized model until reset.

Guarantees (E, P, B, K). The implementation follows direct-form II transposed recursion after denominator normalization. Batch and chunked execution use the same recurrence and therefore agree to numerical precision. Pole-based stability certificates check the finite coefficient model rather than an external plant. Runtime is linear in sample count times filter order, with linear state storage.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_recursive_fft.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapter 33.

C.4A · design_fir

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.design_fir`. Reflected implementation `komorebi_math.signals.filtering.design_fir`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-FIR`.

Manifest signature (s).

```
(kind: 'FIRKind', cutoff: 'float | tuple[float, float]', *, num_taps: 'int' = 101, sample_rate: 'float' = 1.0,
    ↪ window: 'WindowSpec' = 'hann', scale: 'bool' = True) -> 'FIRFilter'
```

Definition. Design a real linear-phase FIR filter by a windowed ideal impulse response.

Contract (D, C, H, R, X). FIR design requires a positive odd tap count, a positive sample rate, and cutoff frequencies strictly inside the Nyquist interval. Band edges must be ordered. Filtering requires a finite nonempty signal and tap vector. The selected FIR mode fixes cropping. Streaming execution preserves only the bounded causal history and rejects processing after reaching a terminal state where applicable.

Guarantees (E, P, B, K). Windowed ideal-response designs are real and symmetric, with the documented normalization and group delay. Frequency responses are sampled finite Fourier sums. Streaming and batch causal FIR execution agree to numerical precision for the same dtype and chunk order. Direct filtering scales with sample count times tap count and stores tap-count history.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_spectral_filtering.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapter 33.

C.4A · design_highpass

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.design_highpass`. Reflected implementation `komorebi_math.signals.filtering.design_highpass`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-FIR`.

Manifest signature (s).

```
(cutoff: 'float', **options: 'Any') -> 'FIRFilter'
```

Definition. Design a highpass FIR filter.

Contract (D, C, H, R, X). FIR design requires a positive odd tap count, a positive sample rate, and cutoff frequencies strictly inside the Nyquist interval. Band edges must be ordered. Filtering requires a finite nonempty signal and tap vector. The selected FIR mode fixes cropping. Streaming execution preserves only the bounded causal history and rejects processing after reaching a terminal state where applicable.

Guarantees (E, P, B, K). Windowed ideal-response designs are real and symmetric, with the documented normalization and group delay. Frequency responses are sampled finite Fourier sums. Streaming and batch causal FIR execution agree to numerical precision for the same dtype and chunk order. Direct filtering scales with sample count times tap count and stores tap-count history.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_spectral_filtering.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapter 33.

C.4A · design_lowpass

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.design_lowpass`. Reflected implementation `komorebi_math.signals.filtering.design_lowpass`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-FIR`.

Manifest signature (s).

```
(cutoff: 'float', **options: 'Any') -> 'FIRFilter'
```

Definition. Design a lowpass FIR filter.

Contract (D, C, H, R, X). FIR design requires a positive odd tap count, a positive sample rate, and cutoff frequencies strictly inside the Nyquist interval. Band edges must be ordered. Filtering requires a finite nonempty signal and tap vector. The selected FIR mode fixes cropping. Streaming execution preserves only the bounded causal history and rejects processing after reaching a terminal state where applicable.

Guarantees (E, P, B, K). Windowed ideal-response designs are real and symmetric, with the documented normalization and group delay. Frequency responses are sampled finite Fourier sums. Streaming and batch causal FIR execution agree to numerical precision for the same dtype and chunk order. Direct filtering scales with sample count times tap count and stores tap-count history.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_spectral_filtering.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapter 33.

C.4A · design_resample_filter

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.design_resample_filter`. Reflected implementation `komorebi_math.signals.resampling.design_resample_filter`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-RESAMPLING`.

Manifest signature (s).

```
(up: 'int', down: 'int', *, num_taps: 'int | None' = None, window: 'WindowSpec' = ('kaiser', 8.0)) -> 'np.ndarray'
```

Definition. Design the centered low-pass kernel used by rational resampling.

Contract (D, C, H, R, X). Upsampling and downsampling factors must be positive integers. A downsampling phase must select a valid residue class. Rational plans reduce the rate ratio, use finite centered zero extension, and require a finite nonempty input. A supplied antialiasing kernel must satisfy the plan's finite shape and gain requirements. Default design uses an odd symmetric low-pass kernel.

Guarantees (E, P, B, K). The output length is exactly `ceil(input_length * up / down)`. Plan certificates compare polyphase execution with an explicit zero-insertion, convolution, delay, and decimation reference to numerical tolerance. Primitive upsampling and downsampling are exact array operations. Polyphase runtime scales with output length and the active taps per phase.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_resampling_adaptive.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapter 34.

C.4A · downsample

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.downsample`. Reflected implementation `komorebi_math.signals.resampling.downsample`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-RESAMPLING`.

Manifest signature (s).

```
(signal: 'Any', factor: 'int', *, phase: 'int' = 0) -> 'np.ndarray'
```

Definition. Retain one residue class from a finite signal.

Contract (D, C, H, R, X). Upsampling and downsampling factors must be positive integers. A downsampling phase must select a valid residue class. Rational plans reduce the rate ratio, use finite centered zero extension, and require a finite nonempty input. A supplied antialiasing kernel must satisfy the plan's finite shape and gain requirements. Default design uses an odd symmetric low-pass kernel.

Guarantees (E, P, B, K). The output length is exactly `ceil(input_length * up / down)`. Plan certificates compare polyphase execution with an explicit zero-insertion, convolution, delay, and decimation reference to numerical tolerance. Primitive upsampling and downsampling are exact array operations. Polyphase runtime scales with output length and the active taps per phase.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_resampling_adaptive.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapter 34.

C.4A · fft_convolve

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.fft_convolve`. Reflected implementation `komorebi_math.signals.fft_convolution.fft_convolve`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-FFT-CONVOLUTION`.

Manifest signature (s).

```
(signal: 'Any', kernel: 'Any', *, mode: 'FIRMode' = 'full') -> 'np.ndarray'
```

Definition. Compute finite linear convolution by a zero-padded FFT.

Contract (D, C, H, R, X). Signal and kernel arrays must be finite, one-dimensional, and nonempty. FFT and overlap-add execution require a positive block length. The output mode applies the same finite cropping convention as direct convolution. A stateful convolver accepts chunks until `flush()`, emits the remaining tail once, and requires `reset()` before reuse after finalization.

Guarantees (E, P, B, K). Zero-padded FFT convolution and bounded-buffer overlap-add agree with direct finite convolution to numerical precision. The certified wrapper records that comparison. FFT execution reduces the asymptotic cost for long records and kernels to blockwise transform cost, while the stateful object stores the kernel spectrum, overlap tail, and one bounded input buffer.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_recursive_fft.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapter 33.

C.4A · finite_polyphase_blocks

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.finite_polyphase_blocks`. Reflected implementation `komorebi_math.signals.convolution.finite_polyphase_blocks`. No root, stable, or beta facade exports this domain-only name in the reference manifest. Within `komorebi_math.signals`, `finite_polyphase_filter_kernels` resolves to the same runtime object. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-CONVOLUTION`.

Manifest signature (s).

```
(kernel: 'Any', input_length: 'int', order: 'int', *, mode: 'ConvolutionMode' = 'full') -> 'PolyphaseBlockMatrix'
```

Definition. Return the exact finite packet blocks of a convolution operator.

Contract (D, C, H, R, X). Signals and kernels are finite one-dimensional arrays, with nonempty inputs where convolution is required. The selected mode fixes the output convention. Circular convolution requires a compatible finite length. Matrix and polyphase constructors require positive dimensions and a positive packet order. Every block retains explicit source and target index sets.

Guarantees (E, P, B, K). The direct operator, matrix representation, and finite polyphase blocks implement the same declared finite convolution law up to the numerical behavior of the input dtype. Matrix construction exposes the full linear map and can require quadratic storage. Direct convolution scales with signal length times kernel length, while block application has the corresponding partitioned cost.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_block4_hardening.py` and `tests/test_signals_polyphase_filterbanks.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapter 33.

C.4A · finite_polyphase_filter_kernels

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.finite_polyphase_filter_kernels`. Reflected implementation `komorebi_math.signals.convolution.finite_polyphase_blocks`. No root, stable, or beta facade exports this domain-only name in the reference manifest. Within `komorebi_math.signals`, `finite_polyphase_blocks` resolves to the same runtime object. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-CONVOLUTION`.

Manifest signature (s).

```
(kernel: 'Any', input_length: 'int', order: 'int', *, mode: 'ConvolutionMode' = 'full') -> 'PolyphaseBlockMatrix'
```

Definition. Return the exact finite packet blocks of a convolution operator.

Contract (D, C, H, R, X). Signals and kernels are finite one-dimensional arrays, with nonempty inputs where convolution is required. The selected mode fixes the output convention. Circular convolution requires a compatible finite length. Matrix and polyphase constructors require positive dimensions and a positive packet order. Every block retains explicit source and target index sets.

Guarantees (E, P, B, K). The direct operator, matrix representation, and finite polyphase blocks implement the same declared finite convolution law up to the numerical behavior of the input dtype. Matrix construction exposes the full linear map and can require quadratic storage. Direct convolution scales with signal length times kernel length, while block application has the corresponding partitioned cost.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_block4_hardening.py` and `tests/test_signals_polyphase_filterbanks.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapter 33.

C.4A · fir_filter

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.fir_filter`. Reflected implementation `komorebi_math.signals.filtering.fir_filter`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-FIR`.

Manifest signature (s).

```
(signal: 'Any', taps: 'Any', *, mode: 'FIRMode' = 'causal') -> 'np.ndarray'
```

Definition. Apply finite convolution with a signal-length-preserving `same` convention.

Contract (D, C, H, R, X). FIR design requires a positive odd tap count, a positive sample rate, and cutoff frequencies strictly inside the Nyquist interval. Band edges must be ordered. Filtering requires a finite nonempty signal and tap vector. The selected FIR mode fixes cropping. Streaming execution preserves only the bounded causal history and rejects processing after reaching a terminal state where applicable.

Guarantees (E, P, B, K). Windowed ideal-response designs are real and symmetric, with the documented normalization and group delay. Frequency responses are sampled finite Fourier sums. Streaming and batch causal FIR execution agree to numerical precision for the same dtype and chunk order. Direct filtering scales with sample count times tap count and stores tap-count history.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_spectral_filtering.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapter 33.

C.4A · fir_frequency_response

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.fir_frequency_response`. Reflected implementation `komorebi_math.signals.filtering.fir_frequency_response`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-FIR`.

Manifest signature (s).

```
(taps: 'Any', *, points: 'int' = 512, sample_rate: 'float' = 1.0, whole: 'bool' = False) -> 'FrequencyResponse'
```

Definition. Evaluate a finite impulse response on an equally spaced frequency grid.

Contract (D, C, H, R, X). FIR design requires a positive odd tap count, a positive sample rate, and cutoff frequencies strictly inside the Nyquist interval. Band edges must be ordered. Filtering requires a finite nonempty signal and tap vector. The selected FIR mode fixes cropping. Streaming execution preserves only the bounded causal history and rejects processing after reaching a terminal state where applicable.

Guarantees (E, P, B, K). Windowed ideal-response designs are real and symmetric, with the documented normalization and group delay. Frequency responses are sampled finite Fourier sums. Streaming and batch causal FIR execution agree to numerical precision for the same dtype and chunk order. Direct filtering scales with sample count times tap count and stores tap-count history.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_spectral_filtering.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapter 33.

C.4A · `frame_signal`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.signals.frame_signal`. Reflected implementation `komorebi_math.signals.time_frequency.frame_signal`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-TIME-FREQUENCY`.

Manifest signature (`s`).

```
(signal: 'Any', frame_length: 'int', hop_length: 'int | None' = None, *, center: 'bool' = False, pad_end: 'bool' =
    ↪ False, boundary: 'BoundaryMode' = 'zeros') -> 'FrameBatch'
```

Definition. Split a finite signal into complete uniformly spaced frames.

Contract (`D, C, H, R, X`). Input is a finite one-dimensional signal. Frame length, hop length, and FFT length must be positive, with the hop no larger than the frame and the FFT no shorter than the frame. One-sided transforms require real input. Boundary, centering, padding, detrending, and scaling choices are explicit. Detrended transforms are intentionally noninvertible. Inverse synthesis requires a nonzero overlap profile.

Guarantees (`E, P, B, K`). Framing metadata records every finite pad, start, and coverage convention. STFT coefficients use NumPy FFT normalization. Invertible configurations reconstruct the original finite record to numerical tolerance through normalized overlap-add. Spectrogram values are nonnegative for magnitude and power forms. Runtime is proportional to the number of frames times the FFT cost.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_windows_time_frequency.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapters 31 and 32.

C.4A · `get_window`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.signals.get_window`. Reflected implementation `komorebi_math.signals.windows.get_window`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-WINDOWS`.

Manifest signature (`s`).

```
(window: 'WindowSpec', length: 'int', *, periodic: 'bool' = True) -> 'np.ndarray'
```

Definition. Return a finite real window using an explicit periodic or symmetric convention.

Contract (`D, C, H, R, X`). Window length and hop values must be positive integers. Parameterized Kaiser and Tukey windows validate their finite parameters. An explicit array must be one-dimensional and have exactly the requested length. Overlap certificates require a compatible shift lattice. Canonical dual construction returns an invalid result when the overlap profile contains a zero.

Guarantees (`E, P, B, K`). Named windows, gain metrics, overlap profiles, and dual windows are deterministic finite NumPy computations. Certificates state finite overlap or constant-overlap identities to a declared tolerance. Window construction and metric evaluation are linear in window length. Overlap profiles also scale with the number of hop residues.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_windows_time_frequency.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapter 31 and Chapter 32.

C.4A · interpolate

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.interpolate`. Reflected implementation `komorebi_math.signals.resampling.interpolate`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-RESAMPLING`.

Manifest signature (s).

```
(signal: 'Any', factor: 'int', *, num_taps: 'int | None' = None, window: 'WindowSpec' = ('kaiser', 8.0)) ->
   ↪ 'np.ndarray'
```

Definition. Upsample and anti-image filter a finite signal by an integer factor.

Contract (D, C, H, R, X). Upsampling and downsampling factors must be positive integers. A downsampling phase must select a valid residue class. Rational plans reduce the rate ratio, use finite centered zero extension, and require a finite nonempty input. A supplied antialiasing kernel must satisfy the plan's finite shape and gain requirements. Default design uses an odd symmetric low-pass kernel.

Guarantees (E, P, B, K). The output length is exactly `ceil(input_length * up / down)`. Plan certificates compare polyphase execution with an explicit zero-insertion, convolution, delay, and decimation reference to numerical tolerance. Primitive upsampling and downsampling are exact array operations. Polyphase runtime scales with output length and the active taps per phase.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_resampling_adaptive.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapter 34.

C.4A · istft

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.istft`. Reflected implementation `komorebi_math.signals.time_frequency.istft`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-TIME-FREQUENCY`.

Manifest signature (s).

```
(transform: 'STFTResult', *, synthesis_window: 'Any | None' = None, length: 'int | None' = None, tolerance: 'float'
   ↪ = 1e-11) -> 'ComputationResult[np.ndarray]'
```

Definition. Invert a `STFTResult` by normalized finite overlap-add.

Contract (D, C, H, R, X). Input is a finite one-dimensional signal. Frame length, hop length, and FFT length must be positive, with the hop no larger than the frame and the FFT no shorter than the frame. One-sided transforms require real input. Boundary, centering, padding, detrending, and scaling choices are explicit. Detrended transforms are intentionally noninvertible. Inverse synthesis requires a nonzero overlap profile.

Guarantees (E, P, B, K). Framing metadata records every finite pad, start, and coverage convention. STFT coefficients use NumPy FFT normalization. Invertible configurations reconstruct the original finite record to numerical tolerance through normalized overlap-add. Spectrogram values are nonnegative for magnitude and power forms. Runtime is proportional to the number of frames times the FFT cost.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_windows_time_frequency.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapters 31 and 32.

C.4A · `lfilter`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.signals.lfilter`. Reflected implementation `komorebi_math.signals.recursive.lfilter`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-RECURSIVE`.

Manifest signature (`s`).

```
(numerator: 'Any', denominator: 'Any', signal: 'Any', *, initial_state: 'Any | None' = None) -> 'FilterOutput'
```

Definition. Apply a finite causal rational filter.

Contract (`D, C, H, R, X`). Numerator and denominator coefficient arrays must be finite and nonempty, and the leading denominator coefficient must be nonzero. Initial state length must equal the normalized filter order. Biquad frequency and quality parameters must satisfy the documented physical range. Streaming execution uses one fixed normalized model until reset.

Guarantees (`E, P, B, K`). The implementation follows direct-form II transposed recursion after denominator normalization. Batch and chunked execution use the same recurrence and therefore agree to numerical precision. Pole-based stability certificates check the finite coefficient model rather than an external plant. Runtime is linear in sample count times filter order, with linear state storage.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_recursive_fft.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapter 33.

C.4A · `overlap_add`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.signals.overlap_add`. Reflected implementation `komorebi_math.signals.time_frequency.overlap_add`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-TIME-FREQUENCY`.

Manifest signature (`s`).

```
(frames: 'Any', hop_length: 'int', *, synthesis_window: 'Any | None' = None, length: 'int | None' = None) ->
    ↪ 'np.ndarray'
```

Definition. Overlap and add complete frames with an optional synthesis window.

Contract (`D, C, H, R, X`). Input is a finite one-dimensional signal. Frame length, hop length, and FFT length must be positive, with the hop no larger than the frame and the FFT no shorter than the frame. One-sided transforms require real input. Boundary, centering, padding, detrending, and scaling choices are explicit. Detrended transforms are intentionally noninvertible. Inverse synthesis requires a nonzero overlap profile.

Guarantees (`E, P, B, K`). Framing metadata records every finite pad, start, and coverage convention. STFT coefficients use NumPy FFT normalization. Invertible configurations reconstruct the original finite record to numerical tolerance through normalized overlap-add. Spectrogram values are nonnegative for magnitude and power forms. Runtime is proportional to the number of frames times the FFT cost.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_windows_time_frequency.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapters 31 and 32.

C.4A · overlap_add_certificate

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.overlap_add_certificate`. Reflected implementation `komorebi_math.signals.windows.overlap_add_certificate`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-WINDOWS`.

Manifest signature (s).

```
(window: 'Any', hop_length: 'int', *, squared: 'bool' = True, require_constant: 'bool' = False, tolerance: 'float'
   ↪  = 1e-12) -> 'Certificate'
```

Definition. Certify nonzero overlap and optionally constant overlap on the shift lattice.

Contract (D, C, H, R, X). Window length and hop values must be positive integers. Parameterized Kaiser and Tukey windows validate their finite parameters. An explicit array must be one-dimensional and have exactly the requested length. Overlap certificates require a compatible shift lattice. Canonical dual construction returns an invalid result when the overlap profile contains a zero.

Guarantees (E, P, B, K). Named windows, gain metrics, overlap profiles, and dual windows are deterministic finite NumPy computations. Certificates state finite overlap or constant-overlap identities to a declared tolerance. Window construction and metric evaluation are linear in window length. Overlap profiles also scale with the number of hop residues.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_windows_time_frequency.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapter 31 and Chapter 32.

C.4A · overlap_add_convolve

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.overlap_add_convolve`. Reflected implementation `komorebi_math.signals.fft_convolution.overlap_add_convolve`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-FFT-CONVOLUTION`.

Manifest signature (s).

```
(signal: 'Any', kernel: 'Any', *, block_length: 'int' = 1024, mode: 'FIRMode' = 'full') -> 'np.ndarray'
```

Definition. Compute finite convolution through bounded-buffer FFT overlap-add.

Contract (D, C, H, R, X). Signal and kernel arrays must be finite, one-dimensional, and nonempty. FFT and overlap-add execution require a positive block length. The output mode applies the same finite cropping convention as direct convolution. A stateful convolver accepts chunks until `flush()`, emits the remaining tail once, and requires `reset()` before reuse after finalization.

Guarantees (E, P, B, K). Zero-padded FFT convolution and bounded-buffer overlap-add agree with direct finite convolution to numerical precision. The certified wrapper records that comparison. FFT execution reduces the asymptotic cost for long records and kernels to blockwise transform cost, while the stateful object stores the kernel spectrum, overlap tail, and one bounded input buffer.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_recursive_fft.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapter 33.

C.4A · overlap_profile

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.overlap_profile`. Reflected implementation `komorebi_math.signals.windows.overlap_profile`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-WINDOWS`.

Manifest signature (s).

```
(window: 'Any', hop_length: 'int', *, squared: 'bool' = True) -> 'np.ndarray'
```

Definition. Return the infinite-lattice overlap profile indexed by hop residue.

Contract (D, C, H, R, X). Window length and hop values must be positive integers. Parameterized Kaiser and Tukey windows validate their finite parameters. An explicit array must be one-dimensional and have exactly the requested length. Overlap certificates require a compatible shift lattice. Canonical dual construction returns an invalid result when the overlap profile contains a zero.

Guarantees (E, P, B, K). Named windows, gain metrics, overlap profiles, and dual windows are deterministic finite NumPy computations. Certificates state finite overlap or constant-overlap identities to a declared tolerance. Window construction and metric evaluation are linear in window length. Overlap profiles also scale with the number of hop residues.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_windows_time_frequency.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapter 31 and Chapter 32.

C.4A · parseval_certificate

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.parseval_certificate`. Reflected implementation `komorebi_math.signals.spectral.parseval_certificate`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-SPECTRAL`.

Manifest signature (s).

```
(signal: 'Any', *, tolerance: 'float' = 1e-12) -> 'Certificate'
```

Definition. Certify the finite NumPy-normalized DFT Parseval identity.

Contract (D, C, H, R, X). Spectral estimators require finite one-dimensional records, a positive sample rate, a valid window and detrending mode, and compatible frame and overlap lengths. Cross spectra require equal-length inputs. One-sided output is reserved for real data. Regularization must be nonnegative and is applied explicitly where transfer ratios would otherwise be singular.

Guarantees (E, P, B, K). Parseval certificates compare the finite NumPy-normalized DFT energy identity. Periodogram and Welch records expose frequency resolution, dominant frequency, and integrated power under the chosen scaling. Cross-spectrum records contain auto spectra, cross spectrum, coherence, and the H1 estimate. Results are numerical finite estimates, not stochastic consistency theorems. FFT cost dominates.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_spectral_filtering.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapter 32.

C.4A · periodogram

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.periodogram`. Reflected implementation `komorebi_math.signals.spectral.periodogram`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-SPECTRAL`.

Manifest signature (s).

```
(signal: 'Any', *, sample_rate: 'float' = 1.0, window: 'WindowSpec' = 'boxcar', fft_length: 'int | None' = None,
    ↪ detrend: 'DetrendMode' = 'constant', scaling: 'SpectralScaling' = 'density', onesided: 'bool | None' = None)
    ↪ -> 'PowerSpectrum'
```

Definition. Compute a deterministic finite modified periodogram.

Contract (D, C, H, R, X). Spectral estimators require finite one-dimensional records, a positive sample rate, a valid window and detrending mode, and compatible frame and overlap lengths. Cross spectra require equal-length inputs. One-sided output is reserved for real data. Regularization must be nonnegative and is applied explicitly where transfer ratios would otherwise be singular.

Guarantees (E, P, B, K). Parseval certificates compare the finite NumPy-normalized DFT energy identity. Periodogram and Welch records expose frequency resolution, dominant frequency, and integrated power under the chosen scaling. Cross-spectrum records contain auto spectra, cross spectrum, coherence, and the H1 estimate. Results are numerical finite estimates, not stochastic consistency theorems. FFT cost dominates.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_spectral_filtering.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapter 32.

C.4A · polyphase_convolution

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.polyphase_convolution`. Reflected implementation `komorebi_math.signals.convolution.polyphase_convolution`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-CONVOLUTION`.

Manifest signature (s).

```
(signal: 'Any', kernel: 'Any', order: 'int', *, mode: 'ConvolutionMode' = 'full', tolerance: 'float' = 1e-11) ->
    ↪ 'ComputationResult[np.ndarray]'
```

Definition. Apply a convolution through exact finite polyphase blocks.

Contract (D, C, H, R, X). Signals and kernels are finite one-dimensional arrays, with nonempty inputs where convolution is required. The selected mode fixes the output convention. Circular convolution requires a compatible finite length. Matrix and polyphase constructors require positive dimensions and a positive packet order. Every block retains explicit source and target index sets.

Guarantees (E, P, B, K). The direct operator, matrix representation, and finite polyphase blocks implement the same declared finite convolution law up to the numerical behavior of the input dtype. Matrix construction exposes the full linear map and can require quadratic storage. Direct convolution scales with signal length times kernel length, while block application has the corresponding partitioned cost.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_block4_hardening.py` and `tests/test_signals_polyphase_filterbanks.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapter 33.

C.4A · resample_poly

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.resample_poly`. Reflected implementation `komorebi_math.signals.resampling.resample_poly`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-RESAMPLING`.

Manifest signature (s).

```
(signal: 'Any', up: 'int', down: 'int', *, taps: 'Any | None' = None, num_taps: 'int | None' = None, window:
    ↪ 'WindowSpec' = ('kaiser', 8.0)) -> 'np.ndarray'
```

Definition. Return centered zero-extension rational polyphase resampling output.

Contract (D, C, H, R, X). Upsampling and downsampling factors must be positive integers. A downsampling phase must select a valid residue class. Rational plans reduce the rate ratio, use finite centered zero extension, and require a finite nonempty input. A supplied antialiasing kernel must satisfy the plan's finite shape and gain requirements. Default design uses an odd symmetric low-pass kernel.

Guarantees (E, P, B, K). The output length is exactly `ceil(input_length * up / down)`. Plan certificates compare polyphase execution with an explicit zero-insertion, convolution, delay, and decimation reference to numerical tolerance. Primitive upsampling and downsampling are exact array operations. Polyphase runtime scales with output length and the active taps per phase.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_resampling_adaptive.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapter 34.

C.4A · resample_poly_result

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.resample_poly_result`. Reflected implementation `komorebi_math.signals.resampling.resample_poly_result`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-RESAMPLING`.

Manifest signature (s).

```
(signal: 'Any', up: 'int', down: 'int', *, taps: 'Any | None' = None, num_taps: 'int | None' = None, window:
    ↪ 'WindowSpec' = ('kaiser', 8.0), tolerance: 'float' = 1e-12) -> 'ResamplingResult'
```

Definition. Return rationally resampled data with an explicit finite identity certificate.

Contract (D, C, H, R, X). Upsampling and downsampling factors must be positive integers. A downsampling phase must select a valid residue class. Rational plans reduce the rate ratio, use finite centered zero extension, and require a finite nonempty input. A supplied antialiasing kernel must satisfy the plan's finite shape and gain requirements. Default design uses an odd symmetric low-pass kernel.

Guarantees (E, P, B, K). The output length is exactly `ceil(input_length * up / down)`. Plan certificates compare polyphase execution with an explicit zero-insertion, convolution, delay, and decimation reference to numerical tolerance. Primitive upsampling and downsampling are exact array operations. Polyphase runtime scales with output length and the active taps per phase.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_resampling_adaptive.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapter 34.

C.4A · resampled_length

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.resampled_length`. Reflected implementation `komorebi_math.signals.resampling.resampled_length`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-RESAMPLING`.

Manifest signature (s).

```
(input_length: 'int', up: 'int', down: 'int') -> 'int'
```

Definition. Return the finite output length `ceil(input_length * up / down)`.

Contract (D, C, H, R, X). Upsampling and downsampling factors must be positive integers. A downsampling phase must select a valid residue class. Rational plans reduce the rate ratio, use finite centered zero extension, and require a finite nonempty input. A supplied antialiasing kernel must satisfy the plan's finite shape and gain requirements. Default design uses an odd symmetric low-pass kernel.

Guarantees (E, P, B, K). The output length is exactly `ceil(input_length * up / down)`. Plan certificates compare polyphase execution with an explicit zero-insertion, convolution, delay, and decimation reference to numerical tolerance. Primitive upsampling and downsampling are exact array operations. Polyphase runtime scales with output length and the active taps per phase.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_resampling_adaptive.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapter 34.

C.4A · spectrogram

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.spectrogram`. Reflected implementation `komorebi_math.signals.time_frequency.spectrogram`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-TIME-FREQUENCY`.

Manifest signature (s).

```
(signal: 'Any', *, scaling: 'SpectrogramScaling' = 'density', **stft_options: 'Any') -> 'SpectrogramResult'
```

Definition. Compute a magnitude, power, spectrum, or density spectrogram.

Contract (D, C, H, R, X). Input is a finite one-dimensional signal. Frame length, hop length, and FFT length must be positive, with the hop no larger than the frame and the FFT no shorter than the frame. One-sided transforms require real input. Boundary, centering, padding, detrending, and scaling choices are explicit. Detrended transforms are intentionally noninvertible. Inverse synthesis requires a nonzero overlap profile.

Guarantees (E, P, B, K). Framing metadata records every finite pad, start, and coverage convention. STFT coefficients use NumPy FFT normalization. Invertible configurations reconstruct the original finite record to numerical tolerance through normalized overlap-add. Spectrogram values are nonnegative for magnitude and power forms. Runtime is proportional to the number of frames times the FFT cost.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_windows_time_frequency.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapters 31 and 32.

C.4A · status

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.status`. Reflected implementation `komorebi_math.signals.status`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-DOMAIN-STATUS`.

Manifest signature (s).

```
() -> komorebi_math.registry.FeatureSpec
```

Definition. Return the registered implementation status for this domain.

Contract (D, C, H, R, X). The feature key is the literal string `signals`. `status()` queries the package registry for that key and returns the registered `FeatureSpec`. The call does not run signal processing, import an optional accelerator, or qualify a backend.

Guarantees (E, P, B, K). The result is deterministic package metadata for the installed source tree. Lookup is constant-time in ordinary Python terms and makes no independent mathematical or numerical claim.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The domain registry path is exercised by the independent verifier for signal foundations. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapters 4, 31, and 72.

C.4A · stft

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.stft`. Reflected implementation `komorebi_math.signals.time_frequency.stft`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-TIME-FREQUENCY`.

Manifest signature (s).

```
(signal: 'Any', *, sample_rate: 'float' = 1.0, frame_length: 'int' = 256, hop_length: 'int | None' = None,
    ↪ fft_length: 'int | None' = None, window: 'WindowSpec' = 'hann', center: 'bool' = True, pad_end: 'bool' =
    ↪ True, boundary: 'BoundaryMode' = 'zeros', onesided: 'bool | None' = None, detrend: 'DetrendMode' = 'none')
    ↪ -> 'STFTResult'
```

Definition. Compute a finite STFT with metadata sufficient for normalized synthesis.

Contract (D, C, H, R, X). Input is a finite one-dimensional signal. Frame length, hop length, and FFT length must be positive, with the hop no larger than the frame and the FFT no shorter than the frame. One-sided transforms require real input. Boundary, centering, padding, detrending, and scaling choices are explicit. Detrended transforms are intentionally noninvertible. Inverse synthesis requires a nonzero overlap profile.

Guarantees (E, P, B, K). Framing metadata records every finite pad, start, and coverage convention. STFT coefficients use NumPy FFT normalization. Invertible configurations reconstruct the original finite record to numerical tolerance through normalized overlap-add. Spectrogram values are nonnegative for magnitude and power forms. Runtime is proportional to the number of frames times the FFT cost.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_windows_time_frequency.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapters 31 and 32.

C.4A · stream_fir_filter

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.stream_fir_filter`. Reflected implementation `komorebi_math.signals.filtering.stream_fir_filter`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-FIR`.

Manifest signature (s).

```
(signal: 'Any', taps: 'Any', chunk_sizes: 'Iterable[int]', *, tolerance: 'float' = 1e-12) ->
    ↪ 'ComputationResult[np.ndarray]'
```

Definition. Filter a signal by declared chunks and certify causal batch equivalence.

Contract (D, C, H, R, X). FIR design requires a positive odd tap count, a positive sample rate, and cutoff frequencies strictly inside the Nyquist interval. Band edges must be ordered. Filtering requires a finite nonempty signal and tap vector. The selected FIR mode fixes cropping. Streaming execution preserves only the bounded causal history and rejects processing after reaching a terminal state where applicable.

Guarantees (E, P, B, K). Windowed ideal-response designs are real and symmetric, with the documented normalization and group delay. Frequency responses are sampled finite Fourier sums. Streaming and batch causal FIR execution agree to numerical precision for the same dtype and chunk order. Direct filtering scales with sample count times tap count and stores tap-count history.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_spectral_filtering.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapter 33.

C.4A · stream_lfilter

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.stream_lfilter`. Reflected implementation `komorebi_math.signals.recursive.stream_lfilter`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-RECURSIVE`.

Manifest signature (s).

```
(filter_: 'LinearFilter', signal: 'Any', chunk_sizes: 'Any', *, tolerance: 'float' = 1e-12) ->
    ↪ 'ComputationResult[np.ndarray]'
```

Definition. Execute declared chunks and certify equality with one batch recursion.

Contract (D, C, H, R, X). Numerator and denominator coefficient arrays must be finite and nonempty, and the leading denominator coefficient must be nonzero. Initial state length must equal the normalized filter order. Biquad frequency and quality parameters must satisfy the documented physical range. Streaming execution uses one fixed normalized model until reset.

Guarantees (E, P, B, K). The implementation follows direct-form II transposed recursion after denominator normalization. Batch and chunked execution use the same recurrence and therefore agree to numerical precision. Pole-based stability certificates check the finite coefficient model rather than an external plant. Runtime is linear in sample count times filter order, with linear state storage.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_recursive_fft.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapter 33.

C.4A · stream_stft

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.stream_stft`. Reflected implementation `komorebi_math.signals.streaming.stream_stft`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-STREAMING-STFT`.

Manifest signature (s).

```
(signal: 'Any', chunk_sizes: 'Iterable[int]', *, frame_length: 'int' = 256, hop_length: 'int | None' = None,
    ↪ fft_length: 'int | None' = None, window: 'WindowSpec' = 'hann', sample_rate: 'float' = 1.0, onesided: 'bool'
    ↪ = True, detrend: 'DetrendMode' = 'none', pad_end: 'bool' = True, tolerance: 'float' = 1e-11) ->
    ↪ 'ComputationResult[StreamingSTFTBatch]'
```

Definition. Analyze declared chunks and certify equality with the matching batch STFT.

Contract (D, C, H, R, X). A streaming STFT instance fixes its frame, hop, FFT, window, boundary, and sidedness configuration at construction. Chunks must be one-dimensional and compatible with the selected real or complex mode. `flush()` finalizes optional end padding. Processing after finalization is rejected until `reset()` is called. The convenience wrapper requires chunk sizes that cover the supplied record exactly.

Guarantees (E, P, B, K). The processor stores only the unresolved overlap and framing state. Emitted frames are consecutive and use the same transform convention as batch `stft()`. The wrapper certificate compares concatenated streaming output with the matching batch transform to numerical tolerance. Work per emitted frame is one FFT plus bounded buffer movement.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_denoise_streaming.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapter 32.

C.4A · upsample

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.upsample`. Reflected implementation `komorebi_math.signals.resampling.upsample`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-RESAMPLING`.

Manifest signature (s).

```
(signal: 'Any', factor: 'int') -> 'np.ndarray'
```

Definition. Insert `factor - 1` zeros between consecutive finite samples.

Contract (D, C, H, R, X). Upsampling and downsampling factors must be positive integers. A downsampling phase must select a valid residue class. Rational plans reduce the rate ratio, use finite centered zero extension, and require a finite nonempty input. A supplied antialiasing kernel must satisfy the plan's finite shape and gain requirements. Default design uses an odd symmetric low-pass kernel.

Guarantees (E, P, B, K). The output length is exactly `ceil(input_length * up / down)`. Plan certificates compare polyphase execution with an explicit zero-insertion, convolution, delay, and decimation reference to numerical tolerance. Primitive upsampling and downsampling are exact array operations. Polyphase runtime scales with output length and the active taps per phase.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_resampling_adaptive.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapter 34.

C.4A · welch_psd

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.welch_psd`. Reflected implementation `komorebi_math.signals.spectral.welch_psd`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-SPECTRAL`.

Manifest signature (s).

```
(signal: 'Any', *, sample_rate: 'float' = 1.0, frame_length: 'int' = 256, hop_length: 'int | None' = None,
    ↪ fft_length: 'int | None' = None, window: 'WindowSpec' = 'hann', detrend: 'DetrendMode' = 'constant',
    ↪ scaling: 'SpectralScaling' = 'density', average: 'SpectralAverage' = 'mean', onesided: 'bool | None' = None)
    ↪ -> 'PowerSpectrum'
```

Definition. Average finite modified periodograms over complete overlapping frames.

Contract (D, C, H, R, X). Spectral estimators require finite one-dimensional records, a positive sample rate, a valid window and detrending mode, and compatible frame and overlap lengths. Cross spectra require equal-length inputs. One-sided output is reserved for real data. Regularization must be nonnegative and is applied explicitly where transfer ratios would otherwise be singular.

Guarantees (E, P, B, K). Parseval certificates compare the finite NumPy-normalized DFT energy identity. Periodogram and Welch records expose frequency resolution, dominant frequency, and integrated power under the chosen scaling. Cross-spectrum records contain auto spectra, cross spectrum, coherence, and the H1 estimate. Results are numerical finite estimates, not stochastic consistency theorems. FFT cost dominates.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_spectral_filtering.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapter 32.

C.4A · window_metrics

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.window_metrics`. Reflected implementation `komorebi_math.signals.windows.window_metrics`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-WINDOWS`.

Manifest signature (s).

```
(window: 'Any', *, sample_rate: 'float' = 1.0) -> 'WindowMetrics'
```

Definition. Return coherent gain, power gain, and equivalent noise bandwidth.

Contract (D, C, H, R, X). Window length and hop values must be positive integers. Parameterized Kaiser and Tukey windows validate their finite parameters. An explicit array must be one-dimensional and have exactly the requested length. Overlap certificates require a compatible shift lattice. Canonical dual construction returns an invalid result when the overlap profile contains a zero.

Guarantees (E, P, B, K). Named windows, gain metrics, overlap profiles, and dual windows are deterministic finite NumPy computations. Certificates state finite overlap or constant-overlap identities to a declared tolerance. Window construction and metric evaluation are linear in window length. Overlap profiles also scale with the number of hop residues.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_windows_time_frequency.py`. The independent verifier for signal foundations checks the relevant finite identities, reference calculations, chunk invariance, rejection paths, and runtime ownership. The source validator for signal foundations checks the exact signature, implementation identity, signal facade, cumulative counts, and unique anchor. See Chapter 31 and Chapter 32.

Signal foundations reference. The 80 cards in C.4A cover 20 classes, 44 functions, and 16 data exports. Their recorded audit reconciles the manifest and runtime identities with focused tests, independent finite checks, and reference anchors.

C.4B Wavelets, recovery, filter banks, and packet systems: 91 entries

This section documents the public `komorebi_math.signals` research namespace. It covers the adaptive, approximation, denoising, filter-bank, polyphase, recovery, singular-packet, Smith-plane, wavelet, and nonlinear packet contracts that sit above the finite signal substrate documented in C.4A. The cards separate exact index and shape laws from numerical reconstruction certificates and conditional recovery hypotheses.

Contract family	Cards
Adaptive filtering	4
Approximation and threshold localization	5
Wavelet denoising	10
Vector edge recovery	6
Finite and periodic filter banks	11
Nonlinear coefficient packets	3
Polyphase and lattice coordinates	14
Scalar jump recovery	10
Singular packets and retained reconstruction	14
Smith-plane transforms	4
Haar wavelets and packet trees	10
Total	**91**

The boundary remains finite and explicit. A reconstruction certificate compares one implemented finite operator with its declared reference under a stored tolerance. A recovery certificate remains conditional on the smoothness, separation, jump-size, and noise assumptions recorded by the call. Adaptive and denoising results describe the supplied finite records. They do not establish unobserved stochastic convergence or population optimality.

C.4B · `AdaptiveFilterResult`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.signals.AdaptiveFilterResult`. Reflected implementation `komorebi_math.signals.adaptive.AdaptiveFilterResult`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C4-ADAPTIVE`.

Manifest signature (`s`).

```
(output: 'np.ndarray', error: 'np.ndarray', final_weights: 'np.ndarray', weight_history: 'np.ndarray', method:
    ↪ 'AdaptiveMethod', certificate: 'Certificate') -> None
```

Definition. Finite adaptive-filter outputs, errors, and coefficient trajectory.

Contract (`D, C, H, R, X`). Adaptive filters require paired finite one-dimensional records of equal nonzero length and a positive tap count. Learning rates, leakage, regularization, forgetting factors, and initialization parameters must satisfy the finite ranges stated by the selected recursion. The result records the full output, error, final coefficients, coefficient history, method label, and finite-run certificate. The certificate does not assert persistent excitation, stochastic convergence, or population optimality. Principal public operations and properties are `mean_squared_error`, `tail_mean_squared_error`.

Guarantees (`E, P, B, K`). LMS and normalized LMS use one bounded coefficient update per sample. RLS also updates a dense inverse-correlation matrix. LMS work scales with sample count times tap count. RLS work scales with sample count times the square of the tap count. Stored coefficient histories scale with sample count times tap count.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_resampling_adaptive.py`. The independent verifier for signal recovery and

filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 34.

C.4B · BestBasisResult

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.BestBasisResult`. Reflected implementation `komorebi_math.signals.wavelets.BestBasisResult`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C4-WAVELET-PACKETS`.

Manifest signature (s).

```
(leaves: 'tuple[str, ...]', cost: 'float', decisions: 'dict[str, str]', margins: 'dict[str, float]', certificate:
   ↪ 'Certificate') -> None
```

Definition. Exact additive best basis result for a finite packet tree.

Contract (D, C, H, R, X). Haar packet trees require a finite nonempty one-dimensional signal, a nonnegative depth, and a length divisible by the corresponding power of two. Packet paths contain only binary branches. A pruning must cover the full tree without ancestor and descendant conflicts. Additive cost functions must return finite values. Lp exponents must be positive and finite where required. Principal public operations and properties are `reconstruct`, `pruning`.

Guarantees (E, P, B, K). Normalized Haar packet analysis preserves finite energy to numerical tolerance. Synthesis from a valid full tree or pruning reconstructs the supplied record under the stored tolerance. Best-basis selection is exact dynamic programming for the declared additive node cost. Tree construction and best-basis work scale with the number of stored coefficients across levels.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_signals_wavelets_singular.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 35.

C.4B · BreakpointBandResult

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.BreakpointBandResult`. Reflected implementation `komorebi_math.signals.recovery.BreakpointBandResult`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C4-SCALAR-RECOVERY`.

Manifest signature (s).

```
(indices: 'tuple[int, ...]', response: 'np.ndarray', threshold: 'float', noise_amplification: 'float',
   ↪ certificate: 'Certificate') -> None
```

Definition. Finite difference indices whose magnitude exceeds a declared noise band.

Contract (D, C, H, R, X). Scalar recovery requires a finite one-dimensional record with at least two samples. Spacing and minimum-jump parameters must be positive. Lipschitz and noise bounds must be nonnegative. The threshold gap must remain strict, and any minimum-separation value must be at least one sample spacing. Difference order is nonnegative and the finite step size is positive.

Guarantees (E, P, B, K). The recovery theorem is conditional on the stored smoothness, separation, and noise hypotheses. Finite differences and Haar formulas follow their explicit index and normalization conventions. Subgaussian bounds are finite confidence bounds for the declared sigma and delta. Candidate scanning is linear in sample count for fixed difference order.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_signals_recovery.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 36.

C.4B · FilterBankReport

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.FilterBankReport`. Reflected implementation `komorebi_math.signals.filterbanks.FilterBankReport`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C4-FILTER-BANKS`.

Manifest signature (s).

```
(lower_frame_bound: 'float', upper_frame_bound: 'float', reconstruction_residual: 'float', certificate:
    ↪ 'Certificate') -> None
```

Definition. Finite reconstruction and frame diagnostics for a filter bank.

Contract (D, C, H, R, X). Periodic and finite filter banks require compatible nonempty one-dimensional analysis and synthesis filters or compatible explicit matrices. Decimation factors must be positive, channel counts must agree, and periodic signal lengths must satisfy the declared divisibility rules. Lifting schemes require even finite records. Canonical dual construction requires full column rank. Numerical tolerances and matrix conventions remain part of the public contract.

Guarantees (E, P, B, K). Analysis, synthesis, frame bounds, and reconstruction reports are finite linear-algebra computations. Perfect-reconstruction certificates compare the implemented finite operator with the identity under the stored tolerance. Haar and five-three lifting constructions use the documented normalization and periodic boundary law. Dense matrix construction, singular values, and pseudoinverses can require quadratic storage and cubic factorization cost.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_signals_polyphase_filterbanks.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 35.

C.4B · FiniteFilterBank

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.FiniteFilterBank`. Reflected implementation `komorebi_math.signals.filterbanks.FiniteFilterBank`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C4-FILTER-BANKS`.

Manifest signature (s).

```
(analysis_matrix: 'Any', synthesis_matrix: 'Any') -> 'None'
```

Definition. A finite analysis matrix together with a declared synthesis matrix.

Contract (D, C, H, R, X). Periodic and finite filter banks require compatible nonempty one-dimensional analysis and synthesis filters or compatible explicit matrices. Decimation factors must be positive, channel counts must agree, and periodic signal lengths must satisfy the declared divisibility rules. Lifting schemes require even finite records. Canonical dual construction requires full column rank. Numerical tolerances and matrix conventions remain part of the public contract. Principal public operations and properties are `input_dimension`, `coefficient_dimension`, `analyze`, `synthesize`, `reconstruction_operator`, `report`, `canonical_dual`, `perturbation_bound`.

Guarantees (E, P, B, K). Analysis, synthesis, frame bounds, and reconstruction reports are finite linear-algebra computations. Perfect-reconstruction certificates compare the implemented finite operator with the identity under the stored tolerance. Haar and five-three lifting constructions use the documented normalization and periodic boundary law. Dense matrix construction, singular values, and pseudoinverses can require quadratic storage and cubic factorization cost.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_signals_polyphase_filterbanks.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 35.

C.4B · `HaarDecomposition`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.HaarDecomposition`. Reflected implementation `komorebi_math.signals.denoise.HaarDecomposition`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C4-DENOISE`.

Manifest signature (s).

```
(approximation: 'np.ndarray', details: 'tuple[np.ndarray, ...]', normalized: 'bool', original_length: 'int') ->
    ↪ None
```

Definition. A finite multilevel Haar approximation and ordered detail channels.

Contract (D, C, H, R, X). Denoising functions require finite nonempty one-dimensional records. Haar decomposition requires a positive level count and a length divisible by the corresponding power of two. Threshold values and noise scales must be nonnegative. The fixed rule requires an explicit threshold, while universal and SURE rules derive one from the supplied or estimated scale. Risk certificates compare one supplied finite clean reference and do not establish unobserved statistical risk. Principal public operations and properties are `levels`, `reconstruct`, `certificate`.

Guarantees (E, P, B, K). Normalized Haar analysis preserves finite energy to numerical tolerance. Thresholding is deterministic for the selected mode and rule. Reconstruction uses the stored multilevel coefficient layout. Work is linear in signal length per decomposition and reconstruction pass. Storage is linear in signal length plus the recorded thresholds and coefficient arrays.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_denoise_streaming.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 35.

C.4B · `HaarFilterBank`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.HaarFilterBank`. Reflected implementation `komorebi_math.signals.filterbanks.HaarFilterBank`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C4-FILTER-BANKS`.

Manifest signature (s).

```
(normalized: 'bool' = True) -> None
```

Definition. Exact pairwise orthonormal Haar analysis and synthesis.

Contract (D, C, H, R, X). Periodic and finite filter banks require compatible nonempty one-dimensional analysis and synthesis filters or compatible explicit matrices. Decimation factors must be positive, channel counts must agree, and periodic signal lengths must satisfy the declared divisibility rules. Lifting schemes require even finite records. Canonical dual construction requires full column rank. Numerical tolerances and matrix conventions remain part of the public contract. Principal public operations and properties are `analyze`, `synthesize`, `certificate`.

Guarantees (E, P, B, K). Analysis, synthesis, frame bounds, and reconstruction reports are finite linear-algebra computations. Perfect-reconstruction certificates compare the implemented finite operator with the identity under the stored tolerance. Haar and five-three lifting constructions use the documented normalization and periodic boundary law. Dense matrix construction, singular values, and pseudoinverses can require quadratic storage and cubic factorization cost. The provenance registry records `PSC-V-T10` as Orthonormal Haar reconstruction. Its recorded status is `proved`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_signals_polyphase_filterbanks.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 35.

C.4B · `HaarPacketTree`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.HaarPacketTree`. Reflected implementation `komorebi_math.signals.wavelets.HaarPacketTree`. The same runtime object is also available through `komorebi_math.HaarPacketTree`. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C4-WAVELET-PACKETS`.

Manifest signature (s).

```
(signal: 'np.ndarray', depth: 'int', normalized: 'bool', _nodes: 'dict[str, np.ndarray]') -> None
```

Definition. All Haar packet coefficients through one finite binary depth.

Contract (D, C, H, R, X). Haar packet trees require a finite nonempty one-dimensional signal, a nonnegative depth, and a length divisible by the corresponding power of two. Packet paths contain only binary branches. A pruning must cover the full tree without ancestor and descendant conflicts. Additive cost functions must return finite values. Lp exponents must be positive and finite where required. Principal public operations and properties are `analyze`, `decompose`, `nodes`, `coefficients`, `coefficient`, `paths_at_depth`, `full_leaves`, `validate_pruning`, `reconstruct`, `energy`, `reconstruction_certificate`, `certificate`. Additional public members are recorded in the machine-readable inventory.

Guarantees (E, P, B, K). Normalized Haar packet analysis preserves finite energy to numerical tolerance. Synthesis from a valid full tree or pruning reconstructs the supplied record under the stored tolerance. Best-basis selection is exact dynamic programming for the declared additive node cost. Tree construction and best-basis work scale with the number of stored coefficients across levels. The provenance registry records `PSC-V-D12` as Finite Haar packet tree. Its recorded status is `definition`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_signals_wavelets_singular.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 35.

C.4B · `JumpDetection`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.JumpDetection`. Reflected implementation `komorebi_math.signals.recovery.JumpDetection`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C4-SCALAR-RECOVERY`.

Manifest signature (s).

```
(cell: 'int', left_position: 'float', right_position: 'float', location: 'float', amplitude: 'float') -> None
```

Definition. One detected jump cell with a midpoint location and contrast estimate.

Contract (D, C, H, R, X). Scalar recovery requires a finite one-dimensional record with at least two samples. Spacing and minimum-jump parameters must be positive. Lipschitz and noise bounds must be nonnegative. The threshold

gap must remain strict, and any minimum-separation value must be at least one sample spacing. Difference order is nonnegative and the finite step size is positive.

Guarantees (E, P, B, K). The recovery theorem is conditional on the stored smoothness, separation, and noise hypotheses. Finite differences and Haar formulas follow their explicit index and normalization conventions. Subgaussian bounds are finite confidence bounds for the declared sigma and delta. Candidate scanning is linear in sample count for fixed difference order.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_signals_recovery.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 36.

C.4B · `JumpRecoveryResult`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.JumpRecoveryResult`. Reflected implementation `komorebi_math.signals.recovery.JumpRecoveryResult`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C4-SCALAR-RECOVERY`.

Manifest signature (s).

```
(detections: 'tuple[JumpDetection, ...]', threshold: 'float', background_bound: 'float', location_error_bound:
   ↪  'float', amplitude_error_bound: 'float', certificate: 'Certificate') -> None
```

Definition. Detected jumps together with theorem scale error bounds.

Contract (D, C, H, R, X). Scalar recovery requires a finite one-dimensional record with at least two samples. Spacing and minimum-jump parameters must be positive. Lipschitz and noise bounds must be nonnegative. The threshold gap must remain strict, and any minimum-separation value must be at least one sample spacing. Difference order is nonnegative and the finite step size is positive. Principal public operations and properties are `cells`, `locations`, `amplitudes`.

Guarantees (E, P, B, K). The recovery theorem is conditional on the stored smoothness, separation, and noise hypotheses. Finite differences and Haar formulas follow their explicit index and normalization conventions. Subgaussian bounds are finite confidence bounds for the declared sigma and delta. Candidate scanning is linear in sample count for fixed difference order.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_signals_recovery.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 36.

C.4B · `LatticePolyphaseCoordinates`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.LatticePolyphaseCoordinates`. Reflected implementation `komorebi_math.signals.polyphase.LatticePolyphaseCoordinates`. The same runtime object is also available through `komorebi_math.stable.LatticePolyphaseCoordinates`, `komorebi_math.v1.LatticePolyphaseCoordinates`, `komorebi_math.api.v1.LatticePolyphaseCoordinates`. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C4-POLYPHASE`.

Manifest signature (s).

```
(channels: 'dict[tuple[int, ...], np.ndarray]', strides: 'tuple[int, ...]', original_shape: 'tuple[int, ...]') ->
    ↪ None
```

Definition. Rectangular lattice residue channels of a finite multidimensional array.

Contract (D, C, H, R, X). Polyphase orders, strides, and radices must be positive integers. Axis selections must be valid for the supplied array, and scalar arrays are rejected where an axis decomposition is required. Residue indices and mixed-radix addresses must lie in range. Lattice coordinates require one channel for every Cartesian residue address. Refinement requires exact divisibility of the fine stride by the coarse stride. Principal public operations and properties are `dtype`, `reconstruct`, `refine`, `coarsen`, `certificate`.

Guarantees (E, P, B, K). Analysis partitions the supplied finite array into explicit residue channels and synthesis restores their interleaving. Roundtrip certificates compare the restored array with the source under the stored tolerance. Mixed-radix addressing is least-significant digit first. Polyphase filtering exposes finite channel kernels with omitted coefficients treated as zero. Work and storage are linear in the represented array size plus the active kernel coefficients. The provenance registry records `PSC-V-D03` as Rectangular lattice packet coordinates. Its recorded status is `definition`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_signals_polyphase_filterbanks.py` and `tests/test_block15_polyphase_refinement.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapters 14 and 35.

C.4B · `PacketEstimate1D`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.PacketEstimate1D`. Reflected implementation `komorebi_math.signals.approximation.PacketEstimate1D`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C4-APPROXIMATION`.

Manifest signature (s).

```
(mask: 'np.ndarray', amplitude: 'float', threshold: 'float', certificate: 'Certificate') -> None
```

Definition. One heuristic positive contrast packet estimate.

Contract (D, C, H, R, X). Low-pass approximations require finite nonempty arrays and nonnegative integer mode budgets. The one-dimensional transform retains the declared centered frequency set. The two-dimensional transform retains the declared rectangular centered set. Otsu thresholding requires at least one finite value and two histogram bins. The positive-packet estimator assumes one dominant positive contrast region and is a bounded heuristic rather than a universal edge detector.

Guarantees (E, P, B, K). FFT approximations are deterministic numerical projections for the supplied finite grid and transform convention. Otsu thresholding is a deterministic finite histogram rule. Packet estimates retain their observed interval and reconstruction data together with a heuristic certificate. FFT cost dominates the transform routes. Histogram and scan routes are linear in the input size up to the fixed bin count.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_signals_extended.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 36.

C.4B · PeriodicFilterBank

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.PeriodicFilterBank`. Reflected implementation `komorebi_math.signals.filterbanks.PeriodicFilterBank`. The same runtime object is also available through `komorebi_math.PeriodicFilterBank`. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C4-FILTER-BANKS`.

Manifest signature (s).

```
(analysis_filters: 'Iterable[Any]', synthesis_filters: 'Iterable[Any]', decimation: 'int') -> 'None'
```

Definition. A finite periodic analysis and synthesis bank with explicit matrices.

Contract (D, C, H, R, X). Periodic and finite filter banks require compatible nonempty one-dimensional analysis and synthesis filters or compatible explicit matrices. Decimation factors must be positive, channel counts must agree, and periodic signal lengths must satisfy the declared divisibility rules. Lifting schemes require even finite records. Canonical dual construction requires full column rank. Numerical tolerances and matrix conventions remain part of the public contract. Principal public operations and properties are `channels`, `analysis_matrix`, `synthesis_matrix`, `analyze`, `synthesize`, `reconstruction_operator`, `perfect_reconstruction_certificate`, `frame_bounds`, `canonical_synthesis_matrix`, `reconstruction_error_bound`.

Guarantees (E, P, B, K). Analysis, synthesis, frame bounds, and reconstruction reports are finite linear-algebra computations. Perfect-reconstruction certificates compare the implemented finite operator with the identity under the stored tolerance. Haar and five-three lifting constructions use the documented normalization and periodic boundary law. Dense matrix construction, singular values, and pseudoinverses can require quadratic storage and cubic factorization cost. The provenance registry records `PSC-V-D08` as Finite periodic filter bank. Its recorded status is `definition`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_signals_polyphase_filterbanks.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 35.

C.4B · PeriodicLiftingScheme

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.PeriodicLiftingScheme`. Reflected implementation `komorebi_math.signals.filterbanks.PeriodicLiftingScheme`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C4-FILTER-BANKS`.

Manifest signature (s).

```
(predict: 'Any', update: 'Any') -> 'None'
```

Definition. A reversible predict and update lifting scheme on periodic pairs.

Contract (D, C, H, R, X). Periodic and finite filter banks require compatible nonempty one-dimensional analysis and synthesis filters or compatible explicit matrices. Decimation factors must be positive, channel counts must agree, and periodic signal lengths must satisfy the declared divisibility rules. Lifting schemes require even finite records. Canonical dual construction requires full column rank. Numerical tolerances and matrix conventions remain part of the public contract. Principal public operations and properties are `analyze`, `synthesize`, `certificate`.

Guarantees (E, P, B, K). Analysis, synthesis, frame bounds, and reconstruction reports are finite linear-algebra computations. Perfect-reconstruction certificates compare the implemented finite operator with the identity under the stored tolerance. Haar and five-three lifting constructions use the documented normalization and periodic boundary law. Dense matrix construction, singular values, and pseudoinverses can require quadratic storage and cubic factorization cost. The provenance registry records `PSC-V-T11` as Exact periodic lifting update. Its recorded status is `proved`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_signals_polyphase_filterbanks.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 35.

C.4B · `PolyphaseCoordinates`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.PolyphaseCoordinates`. Reflected implementation `komorebi_math.signals.polyphase.PolyphaseCoordinates`. The same runtime object is also available through `komorebi_math.stable.PolyphaseCoordinates`, `komorebi_math.v1.PolyphaseCoordinates`, `komorebi_math.api.v1.PolyphaseCoordinates`. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C4-POLYPHASE`.

Manifest signature (s).

```
(channels: 'tuple[np.ndarray, ...]', order: 'int', axis: 'int', original_shape: 'tuple[int, ...]') -> None
```

Definition. Residue channels of a finite array along one axis.

Contract (D, C, H, R, X). Polyphase orders, strides, and radices must be positive integers. Axis selections must be valid for the supplied array, and scalar arrays are rejected where an axis decomposition is required. Residue indices and mixed-radix addresses must lie in range. Lattice coordinates require one channel for every Cartesian residue address. Refinement requires exact divisibility of the fine stride by the coarse stride. Principal public operations and properties are `dtype`, `reconstruct`, `channel_lengths`, `refine`, `coarsen`, `certificate`.

Guarantees (E, P, B, K). Analysis partitions the supplied finite array into explicit residue channels and synthesis restores their interleaving. Roundtrip certificates compare the restored array with the source under the stored tolerance. Mixed-radix addressing is least-significant digit first. Polyphase filtering exposes finite channel kernels with omitted coefficients treated as zero. Work and storage are linear in the represented array size plus the active kernel coefficients. The provenance registry records `PSC-V-D01` as One dimensional polyphase coordinates. Its recorded status is `definition`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_signals_polyphase_filterbanks.py` and `tests/test_block15_polyphase_refinement.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapters 14 and 35.

C.4B · `PolyphaseKernel`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.PolyphaseKernel`. Reflected implementation `komorebi_math.signals.polyphase.PolyphaseKernel`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C4-POLYPHASE`.

Manifest signature (s).

```
(target: 'int', source: 'int', coefficients: 'np.ndarray', order: 'int') -> None
```

Definition. One source to target filter kernel in an infinite polyphase law.

Contract (D, C, H, R, X). Polyphase orders, strides, and radices must be positive integers. Axis selections must be valid for the supplied array, and scalar arrays are rejected where an axis decomposition is required. Residue indices and mixed-radix addresses must lie in range. Lattice coordinates require one channel for every Cartesian residue address. Refinement requires exact divisibility of the fine stride by the coarse stride.

Guarantees (E, P, B, K). Analysis partitions the supplied finite array into explicit residue channels and synthesis restores their interleaving. Roundtrip certificates compare the restored array with the source under the stored tolerance. Mixed-radix addressing is least-significant digit first. Polyphase filtering exposes finite channel kernels with

omitted coefficients treated as zero. Work and storage are linear in the represented array size plus the active kernel coefficients.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_signals_polyphase_filterbanks.py` and `tests/test_block15_polyphase_refinement.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapters 14 and 35.

C.4B · PolyphaseSystem

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.PolyphaseSystem`. Reflected implementation `komorebi_math.signals.polyphase.PolyphaseSystem`. The same runtime object is also available through `komorebi_math.PolyphaseSystem`, `komorebi_math.stable.PolyphaseSystem`, `komorebi_math.v1.PolyphaseSystem`, `komorebi_math.api.v1.PolyphaseSystem`. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C4-POLYPHASE`.

Manifest signature (s).

```
(order: 'int', axis: 'int' = -1) -> None
```

Definition. Reusable one axis polyphase analysis and synthesis system.

Contract (D, C, H, R, X). Polyphase orders, strides, and radices must be positive integers. Axis selections must be valid for the supplied array, and scalar arrays are rejected where an axis decomposition is required. Residue indices and mixed-radix addresses must lie in range. Lattice coordinates require one channel for every Cartesian residue address. Refinement requires exact divisibility of the fine stride by the coarse stride. Principal public operations and properties are `analyze`, `synthesize`, `certificate`.

Guarantees (E, P, B, K). Analysis partitions the supplied finite array into explicit residue channels and synthesis restores their interleaving. Roundtrip certificates compare the restored array with the source under the stored tolerance. Mixed-radix addressing is least-significant digit first. Polyphase filtering exposes finite channel kernels with omitted coefficients treated as zero. Work and storage are linear in the represented array size plus the active kernel coefficients.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_signals_polyphase_filterbanks.py` and `tests/test_block15_polyphase_refinement.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapters 14 and 35.

C.4B · PrunedPacketCoefficients

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.PrunedPacketCoefficients`. Reflected implementation `komorebi_math.signals.wavelets.PrunedPacketCoefficients`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C4-WAVELET-PACKETS`.

Manifest signature (s).

```
(tree: 'HaarPacketTree', leaves: 'tuple[str, ...]') -> None
```

Definition. Coefficients selected by one complete pruning of a Haar packet tree.

Contract (D, C, H, R, X). Haar packet trees require a finite nonempty one-dimensional signal, a nonnegative depth, and a length divisible by the corresponding power of two. Packet paths contain only binary branches. A pruning must cover the full tree without ancestor and descendant conflicts. Additive cost functions must return finite values. Lp

exponents must be positive and finite where required. Principal public operations and properties are `coefficients`, `reconstruct`, `energy`, `certificate`.

Guarantees (`E`, `P`, `B`, `K`). Normalized Haar packet analysis preserves finite energy to numerical tolerance. Synthesis from a valid full tree or pruning reconstructs the supplied record under the stored tolerance. Best-basis selection is exact dynamic programming for the declared additive node cost. Tree construction and best-basis work scale with the number of stored coefficients across levels.

Evidence and links (`V`, `L`). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_signals_wavelets_singular.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 35.

C.4B · `RetainedPacketReconstruction`

Identity (`q`, `a`, `t`, `k`). Public research-manifest route `komorebi_math.signals.RetainedPacketReconstruction`. Reflected implementation `komorebi_math.signals.singular_packets.RetainedPacketResult`. No root, stable, or beta facade exports this domain-only name in the reference manifest. Within `komorebi_math.signals`, `RetainedPacketResult` resolves to the same runtime object. The public export name maps to implementation object `RetainedPacketResult`. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C4-SINGULAR-PACKETS`.

Manifest signature (`s`).

```
(smooth_component: 'np.ndarray', smooth_approximation: 'np.ndarray', packet: 'np.ndarray', reconstruction:
    ↪ 'np.ndarray', error: 'np.ndarray', certificate: 'Certificate', packet_error: 'np.ndarray | None' = None) ->
    ↪ None
```

Definition. A smooth approximation plus an exact or estimated singular packet.

Contract (`D`, `C`, `H`, `R`, `X`). Truncated-power terms require finite locations and amplitudes with nonnegative integer order. Packet construction requires compatible term data. Reconstruction helpers require finite arrays of matching shape, and approximation matrices must have compatible dimensions. Region and location error helpers require nonnegative measures, compatible labels, and a positive finite exponent or infinity.

Guarantees (`E`, `P`, `B`, `K`). Singular packets evaluate, remove, and restore explicit truncated-power components. Retained-packet reconstruction records the smooth approximation, packet estimate, full reconstruction, and the exact finite error identity represented by those arrays. Location bounds are exact at jump order zero and use the documented mean-value estimate at positive orders. Runtime follows the number of samples times the number of retained terms plus any supplied approximation-matrix cost.

Evidence and links (`V`, `L`). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_signals_wavelets_singular.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 36.

C.4B · `RetainedPacketResult`

Identity (`q`, `a`, `t`, `k`). Public research-manifest route `komorebi_math.signals.RetainedPacketResult`. Reflected implementation `komorebi_math.signals.singular_packets.RetainedPacketResult`. No root, stable, or beta facade exports this domain-only name in the reference manifest. Within `komorebi_math.signals`, `RetainedPacketReconstruction` resolves to the same runtime object. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C4-SINGULAR-PACKETS`.

Manifest signature (`s`).

```
(smooth_component: 'np.ndarray', smooth_approximation: 'np.ndarray', packet: 'np.ndarray', reconstruction:
   ↪ 'np.ndarray', error: 'np.ndarray', certificate: 'Certificate', packet_error: 'np.ndarray | None' = None) ->
   ↪ None
```

Definition. A smooth approximation plus an exact or estimated singular packet.

Contract (`D, C, H, R, X`). Truncated-power terms require finite locations and amplitudes with nonnegative integer order. Packet construction requires compatible term data. Reconstruction helpers require finite arrays of matching shape, and approximation matrices must have compatible dimensions. Region and location error helpers require nonnegative measures, compatible labels, and a positive finite exponent or infinity.

Guarantees (`E, P, B, K`). Singular packets evaluate, remove, and restore explicit truncated-power components. Retained-packet reconstruction records the smooth approximation, packet estimate, full reconstruction, and the exact finite error identity represented by those arrays. Location bounds are exact at jump order zero and use the documented mean-value estimate at positive orders. Runtime follows the number of samples times the number of retained terms plus any supplied approximation-matrix cost.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_signals_wavelets_singular.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 36.

C.4B · `SingularPacket1D`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.signals.SingularPacket1D`. Reflected implementation `komorebi_math.signals.singular_packets.SingularPacket1D`. The same runtime object is also available through `komorebi_math.SingularPacket1D`. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C4-SINGULAR-PACKETS`.

Manifest signature (`s`).

```
(components: 'Iterable[TruncatedPowerTerm]') -> 'None'
```

Definition. A finite sum of truncated power terms.

Contract (`D, C, H, R, X`). Truncated-power terms require finite locations and amplitudes with nonnegative integer order. Packet construction requires compatible term data. Reconstruction helpers require finite arrays of matching shape, and approximation matrices must have compatible dimensions. Region and location error helpers require nonnegative measures, compatible labels, and a positive finite exponent or infinity. Principal public operations and properties are `from_jumps`, `evaluate`, `remove`, `restore`, `trace_jumps`.

Guarantees (`E, P, B, K`). Singular packets evaluate, remove, and restore explicit truncated-power components. Retained-packet reconstruction records the smooth approximation, packet estimate, full reconstruction, and the exact finite error identity represented by those arrays. Location bounds are exact at jump order zero and use the documented mean-value estimate at positive orders. Runtime follows the number of samples times the number of retained terms plus any supplied approximation-matrix cost. The provenance registry records `PSC-V-D16` as Finite one dimensional singular packet. Its recorded status is `definition`.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_signals_wavelets_singular.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 36.

C.4B · TruncatedPowerPacket

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.TruncatedPowerPacket`. Reflected implementation `komorebi_math.signals.singular_packets.TruncatedPowerTerm`. No root, stable, or beta facade exports this domain-only name in the reference manifest. Within `komorebi_math.signals`, `TruncatedPowerTerm` resolves to the same runtime object. The public export name maps to implementation object `TruncatedPowerTerm`. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C4-SINGULAR-PACKETS`.

Manifest signature (s).

```
(location: 'float', amplitude: 'complex', order: 'int' = 0) -> None
```

Definition. The term `amplitude * (x - location)_+^order / order!`.

Contract (D, C, H, R, X). Truncated-power terms require finite locations and amplitudes with nonnegative integer order. Packet construction requires compatible term data. Reconstruction helpers require finite arrays of matching shape, and approximation matrices must have compatible dimensions. Region and location error helpers require nonnegative measures, compatible labels, and a positive finite exponent or infinity. Principal public operations and properties are `evaluate`, `location_bound`.

Guarantees (E, P, B, K). Singular packets evaluate, remove, and restore explicit truncated-power components. Retained-packet reconstruction records the smooth approximation, packet estimate, full reconstruction, and the exact finite error identity represented by those arrays. Location bounds are exact at jump order zero and use the documented mean-value estimate at positive orders. Runtime follows the number of samples times the number of retained terms plus any supplied approximation-matrix cost. The provenance registry records `PSC-V-D15` as Truncated power singular packet. Its recorded status is `definition`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_signals_wavelets_singular.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 36.

C.4B · TruncatedPowerTerm

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.TruncatedPowerTerm`. Reflected implementation `komorebi_math.signals.singular_packets.TruncatedPowerTerm`. No root, stable, or beta facade exports this domain-only name in the reference manifest. Within `komorebi_math.signals`, `TruncatedPowerPacket` resolves to the same runtime object. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C4-SINGULAR-PACKETS`.

Manifest signature (s).

```
(location: 'float', amplitude: 'complex', order: 'int' = 0) -> None
```

Definition. The term `amplitude * (x - location)_+^order / order!`.

Contract (D, C, H, R, X). Truncated-power terms require finite locations and amplitudes with nonnegative integer order. Packet construction requires compatible term data. Reconstruction helpers require finite arrays of matching shape, and approximation matrices must have compatible dimensions. Region and location error helpers require nonnegative measures, compatible labels, and a positive finite exponent or infinity. Principal public operations and properties are `evaluate`, `location_bound`.

Guarantees (E, P, B, K). Singular packets evaluate, remove, and restore explicit truncated-power components. Retained-packet reconstruction records the smooth approximation, packet estimate, full reconstruction, and the exact finite error identity represented by those arrays. Location bounds are exact at jump order zero and use the documented mean-value estimate at positive orders. Runtime follows the number of samples times the number of retained terms plus any supplied approximation-matrix cost. The provenance registry records `PSC-V-D15` as Truncated power singular packet. Its recorded status is `definition`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_signals_wavelets_singular.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 36.

C.4B · VectorJumpRecoveryResult

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.VectorJumpRecoveryResult`. Reflected implementation `komorebi_math.signals.edge_recovery.JumpRecoveryResult`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The public export name maps to implementation object `JumpRecoveryResult`. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C4-VECTOR-RECOVERY`.

Manifest signature (s).

```
(cells: 'tuple[int, ...]', locations: 'np.ndarray', amplitudes: 'np.ndarray', contrasts: 'np.ndarray', threshold:
   ↪ 'float', background_bound: 'float', location_error_bound: 'float', amplitude_error_bound: 'float',
   ↪ certificate: 'Certificate') -> None
```

Definition. Detected jump cells, locations, amplitudes, and theorem certificate.

Contract (D, C, H, R, X). Vector jump recovery accepts finite scalar or vector samples with at least two sample locations. Spacing and minimum-jump parameters must be positive. Smoothness and noise bounds must be nonnegative. Threshold separation assumptions remain explicit and conditional. Subgaussian wrappers require a positive confidence level strictly below one and return a dimension-aware finite noise bound. Principal public operations and properties are `packet`.

Guarantees (E, P, B, K). Recovery records expose cells, estimated locations, amplitudes, contrasts, and a conditional certificate. Smooth Haar bounds and step coefficients are exact finite formulas under their stated conventions. The recovery guarantee is conditional on the declared smoothness, spacing, separation, and noise hypotheses. Work is linear in the number of candidate cells times the signal dimension.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_signals_recovery.py` and `tests/test_block4_hardening.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 36.

C.4B · WaveletDenoiseResult

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.WaveletDenoiseResult`. Reflected implementation `komorebi_math.signals.denoise.WaveletDenoiseResult`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C4-DENOISE`.

Manifest signature (s).

```
(signal: 'np.ndarray', decomposition: 'HaarDecomposition', thresholded_details: 'tuple[np.ndarray, ...]',
   ↪ thresholds: 'tuple[float, ...]', noise_sigma: 'float', rule: 'ThresholdRule', mode: 'ThresholdMode',
   ↪ certificate: 'Certificate') -> None
```

Definition. A thresholded Haar reconstruction with explicit finite diagnostics.

Contract (D, C, H, R, X). Denoising functions require finite nonempty one-dimensional records. Haar decomposition requires a positive level count and a length divisible by the corresponding power of two. Threshold values and noise scales must be nonnegative. The fixed rule requires an explicit threshold, while universal and SURE rules derive one from the supplied or estimated scale. Risk certificates compare one supplied finite clean reference and do not establish unobserved statistical risk. Principal public operations and properties are `removed_energy`.

Guarantees (`E, P, B, K`). Normalized Haar analysis preserves finite energy to numerical tolerance. Thresholding is deterministic for the selected mode and rule. Reconstruction uses the stored multilevel coefficient layout. Work is linear in signal length per decomposition and reconstruction pass. Storage is linear in signal length plus the recorded thresholds and coefficient arrays.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_denoise_streaming.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 35.

C.4B · `best_basis`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.signals.best_basis`. Reflected implementation `komorebi_math.signals.wavelets.best_basis`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-WAVELET-PACKETS`.

Manifest signature (`s`).

```
(tree: 'HaarPacketTree', cost: 'Mapping[str, float] | CostFunction') -> 'BestBasisResult'
```

Definition. Return the exact minimum cost pruning of a finite Haar packet tree.

Contract (`D, C, H, R, X`). Haar packet trees require a finite nonempty one-dimensional signal, a nonnegative depth, and a length divisible by the corresponding power of two. Packet paths contain only binary branches. A pruning must cover the full tree without ancestor and descendant conflicts. Additive cost functions must return finite values. Lp exponents must be positive and finite where required.

Guarantees (`E, P, B, K`). Normalized Haar packet analysis preserves finite energy to numerical tolerance. Synthesis from a valid full tree or pruning reconstructs the supplied record under the stored tolerance. Best-basis selection is exact dynamic programming for the declared additive node cost. Tree construction and best-basis work scale with the number of stored coefficients across levels. The provenance registry records `PSC-V-T14` as Exact additive best basis recursion. Its recorded status is `proved`.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_signals_wavelets_singular.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 35.

C.4B · `coefficient_packets`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.signals.coefficient_packets`. Reflected implementation `komorebi_math.signals.nonlinear.coefficient_packets`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-NONLINEAR-PACKETS`.

Manifest signature (`s`).

```
(coefficients: 'Sequence[complex]', order: 'int') -> 'tuple[np.ndarray, ...]'
```

Definition. Split a finite coefficient sequence into residue supported packets.

Contract (`D, C, H, R, X`). Coefficient packet construction requires a positive packet order and a finite one-dimensional coefficient record. Packet products require both inputs to use the same flat or nested representation. Channel dimensions must agree with the declared packet order. Nonlinear shift filters require either the supported legacy shift form or paired nonempty shifts and coefficients.

Guarantees (`E, P, B, K`). Packet products implement cyclic channel convolution with the required carry terms and expose a numerical comparison with the direct Cauchy product. Coefficient packets are exact index partitions of

the supplied finite record. Shift filters are deterministic finite sums. Cost follows the number of channels, retained coefficients, and declared shifts.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_signals_extended.py` and `tests/test_block4_hardening.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapters 28 and 35.

C.4B · `denoising_risk_certificate`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.denoising_risk_certificate`. Reflected implementation `komorebi_math.signals.denoise.denoising_risk_certificate`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-DENOISE`.

Manifest signature (s).

```
(clean_signal: 'Any', noisy_signal: 'Any', denoised_signal: 'Any', *, tolerance: 'float' = 0.0) -> 'Certificate'
```

Definition. Compare observed finite squared error before and after denoising.

Contract (D, C, H, R, X). Denoising functions require finite nonempty one-dimensional records. Haar decomposition requires a positive level count and a length divisible by the corresponding power of two. Threshold values and noise scales must be nonnegative. The fixed rule requires an explicit threshold, while universal and SURE rules derive one from the supplied or estimated scale. Risk certificates compare one supplied finite clean reference and do not establish unobserved statistical risk.

Guarantees (E, P, B, K). Normalized Haar analysis preserves finite energy to numerical tolerance. Thresholding is deterministic for the selected mode and rule. Reconstruction uses the stored multilevel coefficient layout. Work is linear in signal length per decomposition and reconstruction pass. Storage is linear in signal length plus the recorded thresholds and coefficient arrays.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_denoise_streaming.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 35.

C.4B · `detect_piecewise_polynomial_breaks`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.detect_piecewise_polynomial_breaks`. Reflected implementation `komorebi_math.signals.recovery.detect_piecewise_polynomial_breaks`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-SCALAR-RECOVERY`.

Manifest signature (s).

```
(samples: 'Sequence[float]', *, degree: 'int', noise_bound: 'float', threshold: 'float | None' = None) ->
    ↪ 'BreakpointBandResult'
```

Definition. Detect the finite bands where a piecewise polynomial may change.

Contract (D, C, H, R, X). Scalar recovery requires a finite one-dimensional record with at least two samples. Spacing and minimum-jump parameters must be positive. Lipschitz and noise bounds must be nonnegative. The threshold gap must remain strict, and any minimum-separation value must be at least one sample spacing. Difference order is nonnegative and the finite step size is positive.

Guarantees (E, P, B, K). The recovery theorem is conditional on the stored smoothness, separation, and noise hypotheses. Finite differences and Haar formulas follow their explicit index and normalization conventions. Subgaussian

bounds are finite confidence bounds for the declared sigma and delta. Candidate scanning is linear in sample count for fixed difference order.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_signals_recovery.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 36.

C.4B · `difference_noise_bound`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.difference_noise_bound`. Reflected implementation `komorebi_math.signals.singular_packets.difference_noise_bound`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-SINGULAR-PACKETS`.

Manifest signature (s).

```
(noise_bound: 'float', order: 'int') -> 'float'
```

Definition. Return the uniform amplification bound `2^order * noise_bound`.

Contract (D, C, H, R, X). Truncated-power terms require finite locations and amplitudes with nonnegative integer order. Packet construction requires compatible term data. Reconstruction helpers require finite arrays of matching shape, and approximation matrices must have compatible dimensions. Region and location error helpers require nonnegative measures, compatible labels, and a positive finite exponent or infinity.

Guarantees (E, P, B, K). Singular packets evaluate, remove, and restore explicit truncated-power components. Retained-packet reconstruction records the smooth approximation, packet estimate, full reconstruction, and the exact finite error identity represented by those arrays. Location bounds are exact at jump order zero and use the documented mean-value estimate at positive orders. Runtime follows the number of samples times the number of retained terms plus any supplied approximation-matrix cost.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_signals_wavelets_singular.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 36.

C.4B · `entropy_cost`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.entropy_cost`. Reflected implementation `komorebi_math.signals.wavelets.entropy_cost`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-WAVELET-PACKETS`.

Manifest signature (s).

```
(coefficients: 'np.ndarray', _: 'str' = '') -> 'float'
```

Definition. Return the Shannon entropy of normalized packet energy.

Contract (D, C, H, R, X). Haar packet trees require a finite nonempty one-dimensional signal, a nonnegative depth, and a length divisible by the corresponding power of two. Packet paths contain only binary branches. A pruning must cover the full tree without ancestor and descendant conflicts. Additive cost functions must return finite values. Lp exponents must be positive and finite where required.

Guarantees (E, P, B, K). Normalized Haar packet analysis preserves finite energy to numerical tolerance. Synthesis from a valid full tree or pruning reconstructs the supplied record under the stored tolerance. Best-basis selection is

exact dynamic programming for the declared additive node cost. Tree construction and best-basis work scale with the number of stored coefficients across levels.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_signals_wavelets_singular.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 35.

C.4B · `estimate_positive_packet_1d`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.estimate_positive_packet_1d`. Reflected implementation `komorebi_math.signals.approximation.estimate_positive_packet_1d`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-APPROXIMATION`.

Manifest signature (s).

```
(signal: 'np.ndarray | Sequence[float]', *, bins: 'int' = 256) -> 'PacketEstimate1D'
```

Definition. Estimate one positive contrast region by Otsu thresholding.

Contract (D, C, H, R, X). Low-pass approximations require finite nonempty arrays and nonnegative integer mode budgets. The one-dimensional transform retains the declared centered frequency set. The two-dimensional transform retains the declared rectangular centered set. Otsu thresholding requires at least one finite value and two histogram bins. The positive-packet estimator assumes one dominant positive contrast region and is a bounded heuristic rather than a universal edge detector.

Guarantees (E, P, B, K). FFT approximations are deterministic numerical projections for the supplied finite grid and transform convention. Otsu thresholding is a deterministic finite histogram rule. Packet estimates retain their observed interval and reconstruction data together with a heuristic certificate. FFT cost dominates the transform routes. Histogram and scan routes are linear in the input size up to the fixed bin count. The provenance registry records `PSC-V-H01` as Positive contrast two class packet estimator. Its recorded status is `heuristic`. The provenance record notes that this is not a universal edge detector.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_signals_extended.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 36.

C.4B · `estimate_white_noise_sigma`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.estimate_white_noise_sigma`. Reflected implementation `komorebi_math.signals.denoise.estimate_white_noise_sigma`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-DENOISE`.

Manifest signature (s).

```
(detail_coefficients: 'Any') -> 'float'
```

Definition. Estimate Gaussian white-noise scale from finest Haar details by MAD.

Contract (D, C, H, R, X). Denoising functions require finite nonempty one-dimensional records. Haar decomposition requires a positive level count and a length divisible by the corresponding power of two. Threshold values and noise scales must be nonnegative. The fixed rule requires an explicit threshold, while universal and SURE rules derive one from the supplied or estimated scale. Risk certificates compare one supplied finite clean reference and do not establish unobserved statistical risk.

Guarantees (E, P, B, K). Normalized Haar analysis preserves finite energy to numerical tolerance. Thresholding is deterministic for the selected mode and rule. Reconstruction uses the stored multilevel coefficient layout. Work is linear in signal length per decomposition and reconstruction pass. Storage is linear in signal length plus the recorded thresholds and coefficient arrays.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_denoise_streaming.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 35.

C.4B · `estimated_packet_reconstruct`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.estimated_packet_reconstruct`. Reflected implementation `komorebi_math.signals.singular_packets.estimated_packet_reconstruct`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-SINGULAR-PACKETS`.

Manifest signature (s).

```
(signal: 'Any', estimated_packet: 'Any', approximation_matrix: 'Any', *, tolerance: 'float' = 1e-10) ->
    ↪ 'ComputationResult[np.ndarray]'
```

Definition. Compatibility wrapper returning only the reconstructed signal.

Contract (D, C, H, R, X). Truncated-power terms require finite locations and amplitudes with nonnegative integer order. Packet construction requires compatible term data. Reconstruction helpers require finite arrays of matching shape, and approximation matrices must have compatible dimensions. Region and location error helpers require nonnegative measures, compatible labels, and a positive finite exponent or infinity.

Guarantees (E, P, B, K). Singular packets evaluate, remove, and restore explicit truncated-power components. Retained-packet reconstruction records the smooth approximation, packet estimate, full reconstruction, and the exact finite error identity represented by those arrays. Location bounds are exact at jump order zero and use the documented mean-value estimate at positive orders. Runtime follows the number of samples times the number of retained terms plus any supplied approximation-matrix cost.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_signals_wavelets_singular.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 36.

C.4B · `fft_lowpass_1d`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.fft_lowpass_1d`. Reflected implementation `komorebi_math.signals.approximation.fft_lowpass_1d`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-APPROXIMATION`.

Manifest signature (s).

```
(signal: 'np.ndarray | Sequence[float]', modes: 'int') -> 'np.ndarray'
```

Definition. Keep discrete Fourier modes whose integer frequency has magnitude at most `modes`.

Contract (D, C, H, R, X). Low-pass approximations require finite nonempty arrays and nonnegative integer mode budgets. The one-dimensional transform retains the declared centered frequency set. The two-dimensional transform retains the declared rectangular centered set. Otsu thresholding requires at least one finite value and two histogram

bins. The positive-packet estimator assumes one dominant positive contrast region and is a bounded heuristic rather than a universal edge detector.

Guarantees (E, P, B, K). FFT approximations are deterministic numerical projections for the supplied finite grid and transform convention. Otsu thresholding is a deterministic finite histogram rule. Packet estimates retain their observed interval and reconstruction data together with a heuristic certificate. FFT cost dominates the transform routes. Histogram and scan routes are linear in the input size up to the fixed bin count.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_signals_extended.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 36.

C.4B · `fft_lowpass_2d`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.fft_lowpass_2d`. Reflected implementation `komorebi_math.signals.approximation.fft_lowpass_2d`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-APPROXIMATION`.

Manifest signature (s).

```
(image: 'np.ndarray', cutoff: 'int') -> 'np.ndarray'
```

Definition. Apply a rectangular centered Fourier cutoff to a finite two dimensional array.

Contract (D, C, H, R, X). Low-pass approximations require finite nonempty arrays and nonnegative integer mode budgets. The one-dimensional transform retains the declared centered frequency set. The two-dimensional transform retains the declared rectangular centered set. Otsu thresholding requires at least one finite value and two histogram bins. The positive-packet estimator assumes one dominant positive contrast region and is a bounded heuristic rather than a universal edge detector.

Guarantees (E, P, B, K). FFT approximations are deterministic numerical projections for the supplied finite grid and transform convention. Otsu thresholding is a deterministic finite histogram rule. Packet estimates retain their observed interval and reconstruction data together with a heuristic certificate. FFT cost dominates the transform routes. Histogram and scan routes are linear in the input size up to the fixed bin count.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_signals_extended.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 36.

C.4B · `finite_difference`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.finite_difference`. Reflected implementation `komorebi_math.signals.recovery.finite_difference`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The same spelling appears at `komorebi_math.finite_difference`, but those routes resolve to different objects. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-SCALAR-RECOVERY`.

Manifest signature (`s`).

```
(values: 'Sequence[Any]', *, order: 'int' = 1, step: 'int' = 1) -> 'np.ndarray'
```

Definition. Return a forward finite difference of arbitrary positive order and step.

Contract (`D, C, H, R, X`). Scalar recovery requires a finite one-dimensional record with at least two samples. Spacing and minimum-jump parameters must be positive. Lipschitz and noise bounds must be nonnegative. The threshold gap must remain strict, and any minimum-separation value must be at least one sample spacing. Difference order is nonnegative and the finite step size is positive.

Guarantees (`E, P, B, K`). The recovery theorem is conditional on the stored smoothness, separation, and noise hypotheses. Finite differences and Haar formulas follow their explicit index and normalization conventions. Subgaussian bounds are finite confidence bounds for the declared sigma and delta. Candidate scanning is linear in sample count for fixed difference order.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_signals_recovery.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 36.

C.4B · `gamma_from_impedance`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.signals.gamma_from_impedance`. Reflected implementation `komorebi_math.signals.smith.gamma_from_impedance`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-SMITH`.

Manifest signature (`s`).

```
(impedance: 'ArrayLike') -> 'np.ndarray'
```

Definition. Map normalized impedance values to reflection coefficients by the Cayley transform.

Contract (`D, C, H, R, X`). Smith-plane transforms accept finite complex scalars or arrays and reject poles in their fractional-linear denominators. Phase rotations require a positive integer order and reduce the phase modulo that order. Sensitivity regularization must be nonnegative.

Guarantees (`E, P, B, K`). Impedance and reflection transforms are inverse pointwise Möbius maps away from their poles. Phase rotation is a deterministic unit-modulus action. Sensitivity is evaluated pointwise with the declared regularization. Work and storage are linear in the number of supplied values.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_signals_extended.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 36.

C.4B · `haar_analyze`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.signals.haar_analyze`. Reflected implementation `komorebi_math.signals.filterbanks.haar_analyze`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-FILTER-BANKS`.

Manifest signature (s).

```
(signal: 'Any', *, normalized: 'bool' = True) -> 'tuple[np.ndarray, np.ndarray]'
```

Definition. Split a finite even length signal into one level Haar low and high channels.

Contract (D, C, H, R, X). Periodic and finite filter banks require compatible nonempty one-dimensional analysis and synthesis filters or compatible explicit matrices. Decimation factors must be positive, channel counts must agree, and periodic signal lengths must satisfy the declared divisibility rules. Lifting schemes require even finite records. Canonical dual construction requires full column rank. Numerical tolerances and matrix conventions remain part of the public contract.

Guarantees (E, P, B, K). Analysis, synthesis, frame bounds, and reconstruction reports are finite linear-algebra computations. Perfect-reconstruction certificates compare the implemented finite operator with the identity under the stored tolerance. Haar and five-three lifting constructions use the documented normalization and periodic boundary law. Dense matrix construction, singular values, and pseudoinverses can require quadratic storage and cubic factorization cost.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_signals_polyphase_filterbanks.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 35.

C.4B · `haar_decompose`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.haar_decompose`. Reflected implementation `komorebi_math.signals.denoise.haar_decompose`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-DENOISE`.

Manifest signature (s).

```
(signal: 'Any', levels: 'int', *, normalized: 'bool' = True) -> 'HaarDecomposition'
```

Definition. Compute a complete finite multilevel Haar decomposition.

Contract (D, C, H, R, X). Denoising functions require finite nonempty one-dimensional records. Haar decomposition requires a positive level count and a length divisible by the corresponding power of two. Threshold values and noise scales must be nonnegative. The fixed rule requires an explicit threshold, while universal and SURE rules derive one from the supplied or estimated scale. Risk certificates compare one supplied finite clean reference and do not establish unobserved statistical risk.

Guarantees (E, P, B, K). Normalized Haar analysis preserves finite energy to numerical tolerance. Thresholding is deterministic for the selected mode and rule. Reconstruction uses the stored multilevel coefficient layout. Work is linear in signal length per decomposition and reconstruction pass. Storage is linear in signal length plus the recorded thresholds and coefficient arrays. The provenance registry records `KM-DSP-T06` as Finite multilevel Haar reconstruction. Its recorded status is `proved`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_denoise_streaming.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 35.

C.4B · haar_filter_bank

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.haar_filter_bank`. Reflected implementation `komorebi_math.signals.filterbanks.haar_filter_bank`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-FILTER-BANKS`.

Manifest signature (s).

```
(length: 'int', *, normalized: 'bool' = True) -> 'FiniteFilterBank'
```

Definition. Construct the finite one level Haar analysis and synthesis matrices.

Contract (D, C, H, R, X). Periodic and finite filter banks require compatible nonempty one-dimensional analysis and synthesis filters or compatible explicit matrices. Decimation factors must be positive, channel counts must agree, and periodic signal lengths must satisfy the declared divisibility rules. Lifting schemes require even finite records. Canonical dual construction requires full column rank. Numerical tolerances and matrix conventions remain part of the public contract.

Guarantees (E, P, B, K). Analysis, synthesis, frame bounds, and reconstruction reports are finite linear-algebra computations. Perfect-reconstruction certificates compare the implemented finite operator with the identity under the stored tolerance. Haar and five-three lifting constructions use the documented normalization and periodic boundary law. Dense matrix construction, singular values, and pseudoinverses can require quadratic storage and cubic factorization cost.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_signals_polyphase_filterbanks.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 35.

C.4B · haar_lipschitz_bound

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.haar_lipschitz_bound`. Reflected implementation `komorebi_math.signals.recovery.haar_lipschitz_bound`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-SCALAR-RECOVERY`.

Manifest signature (s).

```
(lipschitz_constant: 'float', support_length: 'float') -> 'float'
```

Definition. Return the Paper V bound for a smooth Haar coefficient.

Contract (D, C, H, R, X). Scalar recovery requires a finite one-dimensional record with at least two samples. Spacing and minimum-jump parameters must be positive. Lipschitz and noise bounds must be nonnegative. The threshold gap must remain strict, and any minimum-separation value must be at least one sample spacing. Difference order is nonnegative and the finite step size is positive.

Guarantees (E, P, B, K). The recovery theorem is conditional on the stored smoothness, separation, and noise hypotheses. Finite differences and Haar formulas follow their explicit index and normalization conventions. Subgaussian bounds are finite confidence bounds for the declared sigma and delta. Candidate scanning is linear in sample count for fixed difference order.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_signals_recovery.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 36.

C.4B · haar_matrix

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.haar_matrix`. Reflected implementation `komorebi_math.signals.filterbanks.haar_matrix`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-FILTER-BANKS`.

Manifest signature (s).

```
(length: 'int', *, normalized: 'bool' = True) -> 'np.ndarray'
```

Definition. Return the finite one level Haar analysis matrix for an even signal length.

Contract (D, C, H, R, X). Periodic and finite filter banks require compatible nonempty one-dimensional analysis and synthesis filters or compatible explicit matrices. Decimation factors must be positive, channel counts must agree, and periodic signal lengths must satisfy the declared divisibility rules. Lifting schemes require even finite records. Canonical dual construction requires full column rank. Numerical tolerances and matrix conventions remain part of the public contract.

Guarantees (E, P, B, K). Analysis, synthesis, frame bounds, and reconstruction reports are finite linear-algebra computations. Perfect-reconstruction certificates compare the implemented finite operator with the identity under the stored tolerance. Haar and five-three lifting constructions use the documented normalization and periodic boundary law. Dense matrix construction, singular values, and pseudoinverses can require quadratic storage and cubic factorization cost.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_signals_polyphase_filterbanks.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 35.

C.4B · haar_packet_analyze

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.haar_packet_analyze`. Reflected implementation `komorebi_math.signals.wavelets.haar_packet_analyze`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-WAVELET-PACKETS`.

Manifest signature (s).

```
(signal: 'Any', depth: 'int', *, normalized: 'bool' = True) -> 'HaarPacketTree'
```

Definition. Build a complete finite Haar packet tree to the requested depth.

Contract (D, C, H, R, X). Haar packet trees require a finite nonempty one-dimensional signal, a nonnegative depth, and a length divisible by the corresponding power of two. Packet paths contain only binary branches. A pruning must cover the full tree without ancestor and descendant conflicts. Additive cost functions must return finite values. Lp exponents must be positive and finite where required.

Guarantees (E, P, B, K). Normalized Haar packet analysis preserves finite energy to numerical tolerance. Synthesis from a valid full tree or pruning reconstructs the supplied record under the stored tolerance. Best-basis selection is exact dynamic programming for the declared additive node cost. Tree construction and best-basis work scale with the number of stored coefficients across levels.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_signals_wavelets_singular.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 35.

C.4B · haar_packet_synthesize

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.haar_packet_synthesize`. Reflected implementation `komorebi_math.signals.wavelets.haar_packet_synthesize`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-WAVELET-PACKETS`.

Manifest signature (s).

```
(tree_or_coefficients: 'HaarPacketTree | PrunedPacketCoefficients', leaves: 'Iterable[PacketPath] | None' = None)
    ↪  -> 'np.ndarray'
```

Definition. Reconstruct a signal from a complete tree or a pruned packet coefficient set.

Contract (D, C, H, R, X). Haar packet trees require a finite nonempty one-dimensional signal, a nonnegative depth, and a length divisible by the corresponding power of two. Packet paths contain only binary branches. A pruning must cover the full tree without ancestor and descendant conflicts. Additive cost functions must return finite values. Lp exponents must be positive and finite where required.

Guarantees (E, P, B, K). Normalized Haar packet analysis preserves finite energy to numerical tolerance. Synthesis from a valid full tree or pruning reconstructs the supplied record under the stored tolerance. Best-basis selection is exact dynamic programming for the declared additive node cost. Tree construction and best-basis work scale with the number of stored coefficients across levels.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_signals_wavelets_singular.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 35.

C.4B · haar_step_coefficient

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.haar_step_coefficient`. Reflected implementation `komorebi_math.signals.recovery.haar_step_coefficient`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-SCALAR-RECOVERY`.

Manifest signature (s).

```
(amplitude: 'float', support_length: 'float', relative_location: 'float') -> 'float'
```

Definition. Return the magnitude of one normalized Haar coefficient of a step.

Contract (D, C, H, R, X). Scalar recovery requires a finite one-dimensional record with at least two samples. Spacing and minimum-jump parameters must be positive. Lipschitz and noise bounds must be nonnegative. The threshold gap must remain strict, and any minimum-separation value must be at least one sample spacing. Difference order is nonnegative and the finite step size is positive.

Guarantees (E, P, B, K). The recovery theorem is conditional on the stored smoothness, separation, and noise hypotheses. Finite differences and Haar formulas follow their explicit index and normalization conventions. Subgaussian bounds are finite confidence bounds for the declared sigma and delta. Candidate scanning is linear in sample count for fixed difference order.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_signals_recovery.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 36.

C.4B · haar_synthesize

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.haar_synthesize`. Reflected implementation `komorebi_math.signals.filterbanks.haar_synthesize`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-FILTER-BANKS`.

Manifest signature (s).

```
(low: 'Any', high: 'Any', *, normalized: 'bool' = True) -> 'np.ndarray'
```

Definition. Reconstruct a signal from one level Haar low and high channels.

Contract (D, C, H, R, X). Periodic and finite filter banks require compatible nonempty one-dimensional analysis and synthesis filters or compatible explicit matrices. Decimation factors must be positive, channel counts must agree, and periodic signal lengths must satisfy the declared divisibility rules. Lifting schemes require even finite records. Canonical dual construction requires full column rank. Numerical tolerances and matrix conventions remain part of the public contract.

Guarantees (E, P, B, K). Analysis, synthesis, frame bounds, and reconstruction reports are finite linear-algebra computations. Perfect-reconstruction certificates compare the implemented finite operator with the identity under the stored tolerance. Haar and five-three lifting constructions use the documented normalization and periodic boundary law. Dense matrix construction, singular values, and pseudoinverses can require quadratic storage and cubic factorization cost.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_signals_polyphase_filterbanks.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 35.

C.4B · half_shift_distance

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.half_shift_distance`. Reflected implementation `komorebi_math.signals.edge_recovery.half_shift_distance`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-VECTOR-RECOVERY`.

Manifest signature (s).

```
(point: 'float', cell_length: 'float') -> 'float'
```

Definition. Return the better distance to a boundary among a grid and its half cell shift.

Contract (D, C, H, R, X). Vector jump recovery accepts finite scalar or vector samples with at least two sample locations. Spacing and minimum-jump parameters must be positive. Smoothness and noise bounds must be nonnegative. Threshold separation assumptions remain explicit and conditional. Subgaussian wrappers require a positive confidence level strictly below one and return a dimension-aware finite noise bound.

Guarantees (E, P, B, K). Recovery records expose cells, estimated locations, amplitudes, contrasts, and a conditional certificate. Smooth Haar bounds and step coefficients are exact finite formulas under their stated conventions. The recovery guarantee is conditional on the declared smoothness, spacing, separation, and noise hypotheses. Work is linear in the number of candidate cells times the signal dimension.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_signals_recovery.py` and `tests/test_block4_hardening.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 36.

C.4B · impedance_from_gamma

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.impedance_from_gamma`. Reflected implementation `komorebi_math.signals.smith.impedance_from_gamma`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-SMITH`.

Manifest signature (s).

```
(gamma: 'ArrayLike') -> 'np.ndarray'
```

Definition. Invert the normalized impedance Cayley transform.

Contract (D, C, H, R, X). Smith-plane transforms accept finite complex scalars or arrays and reject poles in their fractional-linear denominators. Phase rotations require a positive integer order and reduce the phase modulo that order. Sensitivity regularization must be nonnegative.

Guarantees (E, P, B, K). Impedance and reflection transforms are inverse pointwise Möbius maps away from their poles. Phase rotation is a deterministic unit-modulus action. Sensitivity is evaluated pointwise with the declared regularization. Work and storage are linear in the number of supplied values.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_signals_extended.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 36.

C.4B · lattice_polyphase_analyze

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.lattice_polyphase_analyze`. Reflected implementation `komorebi_math.signals.polyphase.lattice_polyphase_analyze`. The same runtime object is also available through `komorebi_math.stable.lattice_polyphase_analyze`, `komorebi_math.v1.lattice_polyphase_analyze`, `komorebi_math.api.v1.lattice_polyphase_analyze`. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-POLYPHASE`.

Manifest signature (s).

```
(signal: 'Any', strides: 'Iterable[int]', *, copy: 'bool' = True) -> 'LatticePolyphaseCoordinates'
```

Definition. Split an array into channels indexed by a rectangular lattice quotient.

Contract (D, C, H, R, X). Polyphase orders, strides, and radices must be positive integers. Axis selections must be valid for the supplied array, and scalar arrays are rejected where an axis decomposition is required. Residue indices and mixed-radix addresses must lie in range. Lattice coordinates require one channel for every Cartesian residue address. Refinement requires exact divisibility of the fine stride by the coarse stride.

Guarantees (E, P, B, K). Analysis partitions the supplied finite array into explicit residue channels and synthesis restores their interleaving. Roundtrip certificates compare the restored array with the source under the stored tolerance. Mixed-radix addressing is least-significant digit first. Polyphase filtering exposes finite channel kernels with omitted coefficients treated as zero. Work and storage are linear in the represented array size plus the active kernel coefficients. The provenance registry records `PSC-V-T04` as Exact rectangular lattice polyphase decomposition. Its recorded status is `proved`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_signals_polyphase_filterbanks.py` and `tests/test_block15_polyphase_refinement.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapters 14 and 35.

C.4B · lattice_polyphase_synthesize

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.lattice_polyphase_synthesize`. Reflected implementation `komorebi_math.signals.polyphase.lattice_polyphase_synthesize`. The same runtime object is also available through `komorebi_math.stable.lattice_polyphase_synthesize`, `komorebi_math.v1.lattice_polyphase_synthesize`, `komorebi_math.api.v1.lattice_polyphase_synthesize`. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-POLYPHASE`.

Manifest signature (s).

```
(coordinates: 'LatticePolyphaseCoordinates') -> 'np.ndarray'
```

Definition. Reconstruct an array from rectangular lattice packet coordinates.

Contract (D, C, H, R, X). Polyphase orders, strides, and radices must be positive integers. Axis selections must be valid for the supplied array, and scalar arrays are rejected where an axis decomposition is required. Residue indices and mixed-radix addresses must lie in range. Lattice coordinates require one channel for every Cartesian residue address. Refinement requires exact divisibility of the fine stride by the coarse stride.

Guarantees (E, P, B, K). Analysis partitions the supplied finite array into explicit residue channels and synthesis restores their interleaving. Roundtrip certificates compare the restored array with the source under the stored tolerance. Mixed-radix addressing is least-significant digit first. Polyphase filtering exposes finite channel kernels with omitted coefficients treated as zero. Work and storage are linear in the represented array size plus the active kernel coefficients.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_signals_polyphase_filterbanks.py` and `tests/test_block15_polyphase_refinement.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapters 14 and 35.

C.4B · lms_filter

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.lms_filter`. Reflected implementation `komorebi_math.signals.adaptive.lms_filter`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-ADAPTIVE`.

Manifest signature (s).

```
(input_signal: 'Any', desired_signal: 'Any', num_taps: 'int', *, step_size: 'float' = 0.01, leakage: 'float' = 0.0,
    ↪ initial_weights: 'Any | None' = None) -> 'AdaptiveFilterResult'
```

Definition. Run finite real or complex LMS system identification.

Contract (D, C, H, R, X). Adaptive filters require paired finite one-dimensional records of equal nonzero length and a positive tap count. Learning rates, leakage, regularization, forgetting factors, and initialization parameters must satisfy the finite ranges stated by the selected recursion. The result records the full output, error, final coefficients, coefficient history, method label, and finite-run certificate. The certificate does not assert persistent excitation, stochastic convergence, or population optimality.

Guarantees (E, P, B, K). LMS and normalized LMS use one bounded coefficient update per sample. RLS also updates a dense inverse-correlation matrix. LMS work scales with sample count times tap count. RLS work scales with sample count times the square of the tap count. Stored coefficient histories scale with sample count times tap count. The provenance registry records `KM-DSP-D12` as Finite least-mean-squares adaptation. Its recorded status is `classical input`. The provenance record notes that the finite routine does not assert stochastic convergence.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_resampling_adaptive.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant

to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 34.

C.4B · lp_cost

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.lp_cost`. Reflected implementation `komorebi_math.signals.wavelets.lp_cost`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-WAVELET-PACKETS`.

Manifest signature (s).

```
(exponent: 'float') -> 'CostFunction'
```

Definition. Return an additive coefficient cost based on a positive power sum.

Contract (D, C, H, R, X). Haar packet trees require a finite nonempty one-dimensional signal, a nonnegative depth, and a length divisible by the corresponding power of two. Packet paths contain only binary branches. A pruning must cover the full tree without ancestor and descendant conflicts. Additive cost functions must return finite values. Lp exponents must be positive and finite where required.

Guarantees (E, P, B, K). Normalized Haar packet analysis preserves finite energy to numerical tolerance. Synthesis from a valid full tree or pruning reconstructs the supplied record under the stored tolerance. Best-basis selection is exact dynamic programming for the declared additive node cost. Tree construction and best-basis work scale with the number of stored coefficients across levels.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_signals_wavelets_singular.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 35.

C.4B · median_absolute_deviation

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.median_absolute_deviation`. Reflected implementation `komorebi_math.signals.denoise.median_absolute_deviation`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-DENOISE`.

Manifest signature (s).

```
(values: 'Any', *, center: 'float | None' = None) -> 'float'
```

Definition. Return the finite median absolute deviation from a supplied or median center.

Contract (D, C, H, R, X). Denoising functions require finite nonempty one-dimensional records. Haar decomposition requires a positive level count and a length divisible by the corresponding power of two. Threshold values and noise scales must be nonnegative. The fixed rule requires an explicit threshold, while universal and SURE rules derive one from the supplied or estimated scale. Risk certificates compare one supplied finite clean reference and do not establish unobserved statistical risk.

Guarantees (E, P, B, K). Normalized Haar analysis preserves finite energy to numerical tolerance. Thresholding is deterministic for the selected mode and rule. Reconstruction uses the stored multilevel coefficient layout. Work is linear in signal length per decomposition and reconstruction pass. Storage is linear in signal length plus the recorded thresholds and coefficient arrays.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_denoise_streaming.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 35.

C.4B · mixed_radix_address

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.mixed_radix_address`. Reflected implementation `komorebi_math.signals.polyphase.mixed_radix_address`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-POLYPHASE`.

Manifest signature (s).

```
(digits: 'Iterable[int]', radices: 'Iterable[int]') -> 'int'
```

Definition. Return the least significant first mixed radix address.

Contract (D, C, H, R, X). Polyphase orders, strides, and radices must be positive integers. Axis selections must be valid for the supplied array, and scalar arrays are rejected where an axis decomposition is required. Residue indices and mixed-radix addresses must lie in range. Lattice coordinates require one channel for every Cartesian residue address. Refinement requires exact divisibility of the fine stride by the coarse stride.

Guarantees (E, P, B, K). Analysis partitions the supplied finite array into explicit residue channels and synthesis restores their interleaving. Roundtrip certificates compare the restored array with the source under the stored tolerance. Mixed-radix addressing is least-significant digit first. Polyphase filtering exposes finite channel kernels with omitted coefficients treated as zero. Work and storage are linear in the represented array size plus the active kernel coefficients.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_signals_polyphase_filterbanks.py` and `tests/test_block15_polyphase_refinement.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapters 14 and 35.

C.4B · nlms_filter

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.nlms_filter`. Reflected implementation `komorebi_math.signals.adaptive.nlms_filter`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-ADAPTIVE`.

Manifest signature (s).

```
(input_signal: 'Any', desired_signal: 'Any', num_taps: 'int', *, step_size: 'float' = 0.5, epsilon: 'float' =
   ↪ 1e-08, leakage: 'float' = 0.0, initial_weights: 'Any | None' = None) -> 'AdaptiveFilterResult'
```

Definition. Run finite normalized LMS with an explicit denominator floor.

Contract (D, C, H, R, X). Adaptive filters require paired finite one-dimensional records of equal nonzero length and a positive tap count. Learning rates, leakage, regularization, forgetting factors, and initialization parameters must satisfy the finite ranges stated by the selected recursion. The result records the full output, error, final coefficients, coefficient history, method label, and finite-run certificate. The certificate does not assert persistent excitation, stochastic convergence, or population optimality.

Guarantees (E, P, B, K). LMS and normalized LMS use one bounded coefficient update per sample. RLS also updates a dense inverse-correlation matrix. LMS work scales with sample count times tap count. RLS work scales with sample count times the square of the tap count. Stored coefficient histories scale with sample count times tap count. The provenance registry records `KM-DSP-D13` as Finite normalized least-mean-squares adaptation. Its recorded status is `classical input`. The provenance record notes that the finite routine does not assert stochastic convergence.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_resampling_adaptive.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant

to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 34.

C.4B · nonlinear_shift_filter

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.nonlinear_shift_filter`. Reflected implementation `komorebi_math.signals.nonlinear.nonlinear_shift_filter`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-NONLINEAR-PACKETS`.

Manifest signature (s).

```
(signal: 'Sequence[float]', alpha: 'float' = 0.25, *, shift: 'int' = 1, shifts: 'Sequence[int] | None' = None,
    ↪ coefficients: 'Sequence[float] | None' = None) -> 'np.ndarray'
```

Definition. Apply a polynomial shift interaction to a one dimensional signal.

Contract (D, C, H, R, X). Coefficient packet construction requires a positive packet order and a finite one-dimensional coefficient record. Packet products require both inputs to use the same flat or nested representation. Channel dimensions must agree with the declared packet order. Nonlinear shift filters require either the supported legacy shift form or paired nonempty shifts and coefficients.

Guarantees (E, P, B, K). Packet products implement cyclic channel convolution with the required carry terms and expose a numerical comparison with the direct Cauchy product. Coefficient packets are exact index partitions of the supplied finite record. Shift filters are deterministic finite sums. Cost follows the number of channels, retained coefficients, and declared shifts.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_signals_extended.py` and `tests/test_block4_hardening.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapters 28 and 35.

C.4B · otsu_threshold

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.otsu_threshold`. Reflected implementation `komorebi_math.signals.approximation.otsu_threshold`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-APPROXIMATION`.

Manifest signature (s).

```
(values: 'np.ndarray | Sequence[float]', bins: 'int' = 256) -> 'float'
```

Definition. Return the finite histogram threshold that maximizes between class variance.

Contract (D, C, H, R, X). Low-pass approximations require finite nonempty arrays and nonnegative integer mode budgets. The one-dimensional transform retains the declared centered frequency set. The two-dimensional transform retains the declared rectangular centered set. Otsu thresholding requires at least one finite value and two histogram bins. The positive-packet estimator assumes one dominant positive contrast region and is a bounded heuristic rather than a universal edge detector.

Guarantees (E, P, B, K). FFT approximations are deterministic numerical projections for the supplied finite grid and transform convention. Otsu thresholding is a deterministic finite histogram rule. Packet estimates retain their observed interval and reconstruction data together with a heuristic certificate. FFT cost dominates the transform routes. Histogram and scan routes are linear in the input size up to the fixed bin count.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_signals_extended.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this

contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 36.

C.4B · overlap_matrix_error

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.overlap_matrix_error`. Reflected implementation `komorebi_math.signals.singular_packets.overlap_matrix_error`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-SINGULAR-PACKETS`.

Manifest signature (s).

```
(true_labels: 'Any', true_values: 'Sequence[complex]', estimated_labels: 'Any', estimated_values:
   ↪  'Sequence[complex]', *, p: 'float' = 2.0, tolerance: 'float' = 1e-12) -> 'ComputationResult[float]'
```

Definition. Certify the finite overlap matrix formula for piecewise constant data.

Contract (D, C, H, R, X). Truncated-power terms require finite locations and amplitudes with nonnegative integer order. Packet construction requires compatible term data. Reconstruction helpers require finite arrays of matching shape, and approximation matrices must have compatible dimensions. Region and location error helpers require nonnegative measures, compatible labels, and a positive finite exponent or infinity.

Guarantees (E, P, B, K). Singular packets evaluate, remove, and restore explicit truncated-power components. Retained-packet reconstruction records the smooth approximation, packet estimate, full reconstruction, and the exact finite error identity represented by those arrays. Location bounds are exact at jump order zero and use the documented mean-value estimate at positive orders. Runtime follows the number of samples times the number of retained terms plus any supplied approximation-matrix cost.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_signals_wavelets_singular.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 36.

C.4B · overlap_partition_lp_error

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.overlap_partition_lp_error`. Reflected implementation `komorebi_math.signals.singular_packets.overlap_partition_lp_error`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-SINGULAR-PACKETS`.

Manifest signature (s).

```
(true_values: 'Any', estimated_values: 'Any', overlap_matrix: 'Any', *, p: 'float' = 2.0) -> 'float'
```

Definition. Return the exact piecewise constant error from a region overlap matrix.

Contract (D, C, H, R, X). Truncated-power terms require finite locations and amplitudes with nonnegative integer order. Packet construction requires compatible term data. Reconstruction helpers require finite arrays of matching shape, and approximation matrices must have compatible dimensions. Region and location error helpers require nonnegative measures, compatible labels, and a positive finite exponent or infinity.

Guarantees (E, P, B, K). Singular packets evaluate, remove, and restore explicit truncated-power components. Retained-packet reconstruction records the smooth approximation, packet estimate, full reconstruction, and the exact finite error identity represented by those arrays. Location bounds are exact at jump order zero and use the documented mean-value estimate at positive orders. Runtime follows the number of samples times the number of retained terms plus any supplied approximation-matrix cost.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_signals_wavelets_singular.py`. The independent verifier for signal recovery and

filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 36.

C.4B · packet_energy

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.packet_energy`. Reflected implementation `komorebi_math.signals.wavelets.packet_energy`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-WAVELET-PACKETS`.

Manifest signature (s).

```
(tree_or_coefficients: 'HaarPacketTree | PrunedPacketCoefficients', leaves: 'Iterable[PacketPath] | None' = None)
   ↪  -> 'float'
```

Definition. Return total squared coefficient energy for a complete or pruned packet representation.

Contract (D, C, H, R, X). Haar packet trees require a finite nonempty one-dimensional signal, a nonnegative depth, and a length divisible by the corresponding power of two. Packet paths contain only binary branches. A pruning must cover the full tree without ancestor and descendant conflicts. Additive cost functions must return finite values. Lp exponents must be positive and finite where required.

Guarantees (E, P, B, K). Normalized Haar packet analysis preserves finite energy to numerical tolerance. Synthesis from a valid full tree or pruning reconstructs the supplied record under the stored tolerance. Best-basis selection is exact dynamic programming for the declared additive node cost. Tree construction and best-basis work scale with the number of stored coefficients across levels.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_signals_wavelets_singular.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 35.

C.4B · packet_product

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.packet_product`. Reflected implementation `komorebi_math.signals.nonlinear.packet_product`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-NONLINEAR-PACKETS`.

Manifest signature (s).

```
(a: 'Any', b: 'Any', order: 'int') -> 'ComputationResult[tuple[np.ndarray, ...]]'
```

Definition. Multiply finite sequences or already separated cyclic packet channels.

Contract (D, C, H, R, X). Coefficient packet construction requires a positive packet order and a finite one-dimensional coefficient record. Packet products require both inputs to use the same flat or nested representation. Channel dimensions must agree with the declared packet order. Nonlinear shift filters require either the supported legacy shift form or paired nonempty shifts and coefficients.

Guarantees (E, P, B, K). Packet products implement cyclic channel convolution with the required carry terms and expose a numerical comparison with the direct Cauchy product. Coefficient packets are exact index partitions of the supplied finite record. Shift filters are deterministic finite sums. Cost follows the number of channels, retained coefficients, and declared shifts. The provenance registry records `PSC-V-T51` as Cyclic product packet law. Its recorded status is `proved`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_signals_extended.py` and `tests/test_block4_hardening.py`. The independent ver-

ifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapters 28 and 35.

C.4B · `periodic_five_three_lifting`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.signals.periodic_five_three_lifting`. Reflected implementation `komorebi_math.signals.filterbanks.periodic_five_three_lifting`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-FILTER-BANKS`.

Manifest signature (`s`).

```
() -> 'PeriodicLiftingScheme'
```

Definition. Return the floating point periodic five three lifting scheme.

Contract (`D, C, H, R, X`). Periodic and finite filter banks require compatible nonempty one-dimensional analysis and synthesis filters or compatible explicit matrices. Decimation factors must be positive, channel counts must agree, and periodic signal lengths must satisfy the declared divisibility rules. Lifting schemes require even finite records. Canonical dual construction requires full column rank. Numerical tolerances and matrix conventions remain part of the public contract.

Guarantees (`E, P, B, K`). Analysis, synthesis, frame bounds, and reconstruction reports are finite linear-algebra computations. Perfect-reconstruction certificates compare the implemented finite operator with the identity under the stored tolerance. Haar and five-three lifting constructions use the documented normalization and periodic boundary law. Dense matrix construction, singular values, and pseudoinverses can require quadratic storage and cubic factorization cost.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_signals_polyphase_filterbanks.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 35.

C.4B · `phase_mobius`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.signals.phase_mobius`. Reflected implementation `komorebi_math.signals.smith.phase_mobius`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-SMITH`.

Manifest signature (`s`).

```
(impedance: 'ArrayLike', order: 'int', phase: 'int') -> 'np.ndarray'
```

Definition. Apply a cyclic disk rotation pulled back to normalized impedance coordinates.

Contract (`D, C, H, R, X`). Smith-plane transforms accept finite complex scalars or arrays and reject poles in their fractional-linear denominators. Phase rotations require a positive integer order and reduce the phase modulo that order. Sensitivity regularization must be nonnegative.

Guarantees (`E, P, B, K`). Impedance and reflection transforms are inverse pointwise Möbius maps away from their poles. Phase rotation is a deterministic unit-modulus action. Sensitivity is evaluated pointwise with the declared regularization. Work and storage are linear in the number of supplied values.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_signals_extended.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 36.

C.4B · polyphase_analyze

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.polyphase_analyze`. Reflected implementation `komorebi_math.signals.polyphase.polyphase_analyze`. The same runtime object is also available through `komorebi_math.polyphase_analyze`. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-POLYPHASE`.

Manifest signature (s).

```
(signal: 'Any', order: 'int', *, axis: 'int' = -1, copy: 'bool' = True) -> 'PolyphaseCoordinates'
```

Definition. Split a finite array into residue channels along `axis`.

Contract (D, C, H, R, X). Polyphase orders, strides, and radices must be positive integers. Axis selections must be valid for the supplied array, and scalar arrays are rejected where an axis decomposition is required. Residue indices and mixed-radix addresses must lie in range. Lattice coordinates require one channel for every Cartesian residue address. Refinement requires exact divisibility of the fine stride by the coarse stride.

Guarantees (E, P, B, K). Analysis partitions the supplied finite array into explicit residue channels and synthesis restores their interleaving. Roundtrip certificates compare the restored array with the source under the stored tolerance. Mixed-radix addressing is least-significant digit first. Polyphase filtering exposes finite channel kernels with omitted coefficients treated as zero. Work and storage are linear in the represented array size plus the active kernel coefficients. The provenance registry records `PSC-V-T02` as Exact finite polyphase decomposition. Its recorded status is `proved`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_signals_polyphase_filterbanks.py` and `tests/test_block15_polyphase_refinement.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapters 14 and 35.

C.4B · polyphase_filter_kernels

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.polyphase_filter_kernels`. Reflected implementation `komorebi_math.signals.polyphase.polyphase_filter_kernels`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-POLYPHASE`.

Manifest signature (s).

```
(kernel: 'Any', order: 'int') -> 'tuple[tuple[PolyphaseKernel, ...], ...]'
```

Definition. Return the causal kernels `h[Nm + target - source]` for every residue pair.

Contract (D, C, H, R, X). Polyphase orders, strides, and radices must be positive integers. Axis selections must be valid for the supplied array, and scalar arrays are rejected where an axis decomposition is required. Residue indices and mixed-radix addresses must lie in range. Lattice coordinates require one channel for every Cartesian residue address. Refinement requires exact divisibility of the fine stride by the coarse stride.

Guarantees (E, P, B, K). Analysis partitions the supplied finite array into explicit residue channels and synthesis restores their interleaving. Roundtrip certificates compare the restored array with the source under the stored tolerance. Mixed-radix addressing is least-significant digit first. Polyphase filtering exposes finite channel kernels with omitted coefficients treated as zero. Work and storage are linear in the represented array size plus the active kernel coefficients.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_signals_polyphase_filterbanks.py` and `tests/test_block15_polyphase_refinement.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and

filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapters 14 and 35.

C.4B · polyphase_roundtrip_certificate

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.polyphase_roundtrip_certificate`. Reflected implementation `komorebi_math.signals.polyphase.polyphase_roundtrip_certificate`. The same runtime object is also available through `komorebi_math.stable.polyphase_roundtrip_certificate`, `komorebi_math.v1.polyphase_roundtrip_certificate`, `komorebi_math.api.v1.polyphase_roundtrip_certificate`. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-POLYPHASE`.

Manifest signature (s).

```
(signal: 'Any', order: 'int', *, axis: 'int' = -1, tolerance: 'float' = 1e-12) -> 'ComputationResult[np.ndarray]'
```

Definition. Analyze and reconstruct a finite signal with a numerical certificate.

Contract (D, C, H, R, X). Polyphase orders, strides, and radices must be positive integers. Axis selections must be valid for the supplied array, and scalar arrays are rejected where an axis decomposition is required. Residue indices and mixed-radix addresses must lie in range. Lattice coordinates require one channel for every Cartesian residue address. Refinement requires exact divisibility of the fine stride by the coarse stride.

Guarantees (E, P, B, K). Analysis partitions the supplied finite array into explicit residue channels and synthesis restores their interleaving. Roundtrip certificates compare the restored array with the source under the stored tolerance. Mixed-radix addressing is least-significant digit first. Polyphase filtering exposes finite channel kernels with omitted coefficients treated as zero. Work and storage are linear in the represented array size plus the active kernel coefficients.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_signals_polyphase_filterbanks.py` and `tests/test_block15_polyphase_refinement.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapters 14 and 35.

C.4B · polyphase_synthesize

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.polyphase_synthesize`. Reflected implementation `komorebi_math.signals.polyphase.polyphase_synthesize`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-POLYPHASE`.

Manifest signature (s).

```
(coordinates: 'PolyphaseCoordinates') -> 'np.ndarray'
```

Definition. Reconstruct a finite array from polyphase coordinates.

Contract (D, C, H, R, X). Polyphase orders, strides, and radices must be positive integers. Axis selections must be valid for the supplied array, and scalar arrays are rejected where an axis decomposition is required. Residue indices and mixed-radix addresses must lie in range. Lattice coordinates require one channel for every Cartesian residue address. Refinement requires exact divisibility of the fine stride by the coarse stride.

Guarantees (E, P, B, K). Analysis partitions the supplied finite array into explicit residue channels and synthesis restores their interleaving. Roundtrip certificates compare the restored array with the source under the stored tolerance. Mixed-radix addressing is least-significant digit first. Polyphase filtering exposes finite channel kernels with omitted coefficients treated as zero. Work and storage are linear in the represented array size plus the active kernel coefficients.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_signals_polyphase_filterbanks.py` and `tests/test_block15_polyphase_refinement.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapters 14 and 35.

C.4B · `reconstruction_error`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.reconstruction_error`. Reflected implementation `komorebi_math.signals.filterbanks.reconstruction_error`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-FILTER-BANKS`.

Manifest signature (s).

```
(bank: 'FiniteFilterBank') -> 'float'
```

Definition. Return the spectral norm of the finite filter bank reconstruction defect.

Contract (D, C, H, R, X). Periodic and finite filter banks require compatible nonempty one-dimensional analysis and synthesis filters or compatible explicit matrices. Decimation factors must be positive, channel counts must agree, and periodic signal lengths must satisfy the declared divisibility rules. Lifting schemes require even finite records. Canonical dual construction requires full column rank. Numerical tolerances and matrix conventions remain part of the public contract.

Guarantees (E, P, B, K). Analysis, synthesis, frame bounds, and reconstruction reports are finite linear-algebra computations. Perfect-reconstruction certificates compare the implemented finite operator with the identity under the stored tolerance. Haar and five-three lifting constructions use the documented normalization and periodic boundary law. Dense matrix construction, singular values, and pseudoinverses can require quadratic storage and cubic factorization cost.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_signals_polyphase_filterbanks.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 35.

C.4B · `recover_jumps_subgaussian`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.recover_jumps_subgaussian`. Reflected implementation `komorebi_math.signals.recovery.recover_jumps_subgaussian`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-SCALAR-RECOVERY`.

Manifest signature (s).

```
(samples: 'Sequence[float]', *, spacing: 'float', lipschitz_bound: 'float', sigma: 'float', delta: 'float',
    ↪ minimum_jump: 'float', threshold: 'float | None' = None, start: 'float' = 0.0, minimum_separation: 'float |
    ↪ None' = None) -> 'JumpRecoveryResult'
```

Definition. Apply the separated jump theorem with a simultaneous sub Gaussian bound.

Contract (D, C, H, R, X). Scalar recovery requires a finite one-dimensional record with at least two samples. Spacing and minimum-jump parameters must be positive. Lipschitz and noise bounds must be nonnegative. The threshold gap must remain strict, and any minimum-separation value must be at least one sample spacing. Difference order is nonnegative and the finite step size is positive.

Guarantees (E, P, B, K). The recovery theorem is conditional on the stored smoothness, separation, and noise hypotheses. Finite differences and Haar formulas follow their explicit index and normalization conventions. Subgaussian bounds are finite confidence bounds for the declared sigma and delta. Candidate scanning is linear in sample count for fixed difference order.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_signals_recovery.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 36.

C.4B · recover_separated_jumps

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.recover_separated_jumps`. Reflected implementation `komorebi_math.signals.recovery.recover_separated_jumps`. The same runtime object is also available through `komorebi_math.recover_separated_jumps`. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-SCALAR-RECOVERY`.

Manifest signature (s).

```
(samples: 'Sequence[float]', *, spacing: 'float', lipschitz_bound: 'float', noise_bound: 'float', minimum_jump:
    ↪ 'float', threshold: 'float | None' = None, start: 'float' = 0.0, minimum_separation: 'float | None' = None)
    ↪ -> 'JumpRecoveryResult'
```

Definition. Recover jump cells by thresholding adjacent sample contrasts.

Contract (D, C, H, R, X). Scalar recovery requires a finite one-dimensional record with at least two samples. Spacing and minimum-jump parameters must be positive. Lipschitz and noise bounds must be nonnegative. The threshold gap must remain strict, and any minimum-separation value must be at least one sample spacing. Difference order is nonnegative and the finite step size is positive.

Guarantees (E, P, B, K). The recovery theorem is conditional on the stored smoothness, separation, and noise hypotheses. Finite differences and Haar formulas follow their explicit index and normalization conventions. Subgaussian bounds are finite confidence bounds for the declared sigma and delta. Candidate scanning is linear in sample count for fixed difference order. The provenance registry records `PSC-V-T36` as Stable recovery of separated one dimensional jumps. Its recorded status is `conditional theorem`. The provenance record limits the claim to one dimensional sampled signals with separated value jumps.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_signals_recovery.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 36.

C.4B · recover_vector_jumps

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.recover_vector_jumps`. Reflected implementation `komorebi_math.signals.edge_recovery.recover_jumps`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The public export name maps to implementation object `recover_jumps`. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-VECTOR-RECOVERY`.

Manifest signature (s).

```
(samples: 'Any', *, spacing: 'float', lipschitz_bound: 'float', noise_bound: 'float', minimum_jump: 'float',
    ↪ origin: 'float' = 0.0, threshold: 'float | None' = None) -> 'JumpRecoveryResult'
```

Definition. Recover jump cells by adjacent contrast thresholding.

Contract (D, C, H, R, X). Vector jump recovery accepts finite scalar or vector samples with at least two sample locations. Spacing and minimum-jump parameters must be positive. Smoothness and noise bounds must be nonnegative. Threshold separation assumptions remain explicit and conditional. Subgaussian wrappers require a positive confidence level strictly below one and return a dimension-aware finite noise bound.

Guarantees (E, P, B, K). Recovery records expose cells, estimated locations, amplitudes, contrasts, and a conditional certificate. Smooth Haar bounds and step coefficients are exact finite formulas under their stated conventions. The recovery guarantee is conditional on the declared smoothness, spacing, separation, and noise hypotheses. Work is linear in the number of candidate cells times the signal dimension. The provenance registry records `PSC-V-T19` as Deterministic separated jump cell recovery. Its recorded status is `proved`. The provenance record notes that the result does not certify unknown smoothness or unknown minimum jump size from the samples alone.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_signals_recovery.py` and `tests/test_block4_hardening.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 36.

C.4B · `recover_vector_jumps_subgaussian`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.recover_vector_jumps_subgaussian`. Reflected implementation `komorebi_math.signals.edge_recovery.recover_jumps_subgaussian`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The public export name maps to implementation object `recover_jumps_subgaussian`. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-VECTOR-RECOVERY`.

Manifest signature (s).

```
(samples: 'Any', *, spacing: 'float', lipschitz_bound: 'float', sigma: 'float', minimum_jump: 'float',
    ↪ failure_probability: 'float', origin: 'float' = 0.0) -> 'JumpRecoveryResult'
```

Definition. Apply the deterministic theorem with a simultaneous sub Gaussian noise bound.

Contract (D, C, H, R, X). Vector jump recovery accepts finite scalar or vector samples with at least two sample locations. Spacing and minimum-jump parameters must be positive. Smoothness and noise bounds must be nonnegative. Threshold separation assumptions remain explicit and conditional. Subgaussian wrappers require a positive confidence level strictly below one and return a dimension-aware finite noise bound.

Guarantees (E, P, B, K). Recovery records expose cells, estimated locations, amplitudes, contrasts, and a conditional certificate. Smooth Haar bounds and step coefficients are exact finite formulas under their stated conventions. The recovery guarantee is conditional on the declared smoothness, spacing, separation, and noise hypotheses. Work is linear in the number of candidate cells times the signal dimension. The provenance registry records `PSC-V-T20` as Sub Gaussian jump recovery bound. Its recorded status is `proved`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_signals_recovery.py` and `tests/test_block4_hardening.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 36.

C.4B · refinement_children

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.refinement_children`. Reflected implementation `komorebi_math.signals.polyphase.refinement_children`. The same runtime object is also available through `komorebi_math.stable.refinement_children`, `komorebi_math.v1.refinement_children`, `komorebi_math.api.v1.refinement_children`. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-POLYPHASE`.

Manifest signature (s).

```
(coarse_order: 'int', fine_order: 'int', residue: 'int') -> 'tuple[int, ...]'
```

Definition. Return fine residues whose reduction modulo the coarse order is `residue`.

Contract (D, C, H, R, X). Polyphase orders, strides, and radices must be positive integers. Axis selections must be valid for the supplied array, and scalar arrays are rejected where an axis decomposition is required. Residue indices and mixed-radix addresses must lie in range. Lattice coordinates require one channel for every Cartesian residue address. Refinement requires exact divisibility of the fine stride by the coarse stride.

Guarantees (E, P, B, K). Analysis partitions the supplied finite array into explicit residue channels and synthesis restores their interleaving. Roundtrip certificates compare the restored array with the source under the stored tolerance. Mixed-radix addressing is least-significant digit first. Polyphase filtering exposes finite channel kernels with omitted coefficients treated as zero. Work and storage are linear in the represented array size plus the active kernel coefficients.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_signals_polyphase_filterbanks.py` and `tests/test_block15_polyphase_refinement.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapters 14 and 35.

C.4B · residue_project

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.residue_project`. Reflected implementation `komorebi_math.signals.polyphase.residue_project`. The same runtime object is also available through `komorebi_math.stable.residue_project`, `komorebi_math.v1.residue_project`, `komorebi_math.api.v1.residue_project`. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-POLYPHASE`.

Manifest signature (s).

```
(signal: 'Any', order: 'int', residue: 'int', *, axis: 'int' = -1) -> 'np.ndarray'
```

Definition. Return the full shaped signal supported on one residue class.

Contract (D, C, H, R, X). Polyphase orders, strides, and radices must be positive integers. Axis selections must be valid for the supplied array, and scalar arrays are rejected where an axis decomposition is required. Residue indices and mixed-radix addresses must lie in range. Lattice coordinates require one channel for every Cartesian residue address. Refinement requires exact divisibility of the fine stride by the coarse stride.

Guarantees (E, P, B, K). Analysis partitions the supplied finite array into explicit residue channels and synthesis restores their interleaving. Roundtrip certificates compare the restored array with the source under the stored tolerance. Mixed-radix addressing is least-significant digit first. Polyphase filtering exposes finite channel kernels with omitted coefficients treated as zero. Work and storage are linear in the represented array size plus the active kernel coefficients.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_signals_polyphase_filterbanks.py` and `tests/test_block15_polyphase_refinement.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and

filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapters 14 and 35.

C.4B · retained_packet_reconstruct

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.retained_packet_reconstruct`. Reflected implementation `komorebi_math.signals.singular_packets.retained_packet_reconstruct`. The same runtime object is also available through `komorebi_math.retained_packet_reconstruct`. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-SINGULAR-PACKETS`.

Manifest signature (s).

```
(signal: 'Any', packet: 'Any', approximator: 'Callable[[np.ndarray], Any]', *, tolerance: 'float' = 1e-11) ->
    ↪ 'RetainedPacketResult'
```

Definition. Approximate only the packet removed component and restore the packet.

Contract (D, C, H, R, X). Truncated-power terms require finite locations and amplitudes with nonnegative integer order. Packet construction requires compatible term data. Reconstruction helpers require finite arrays of matching shape, and approximation matrices must have compatible dimensions. Region and location error helpers require nonnegative measures, compatible labels, and a positive finite exponent or infinity.

Guarantees (E, P, B, K). Singular packets evaluate, remove, and restore explicit truncated-power components. Retained-packet reconstruction records the smooth approximation, packet estimate, full reconstruction, and the exact finite error identity represented by those arrays. Location bounds are exact at jump order zero and use the documented mean-value estimate at positive orders. Runtime follows the number of samples times the number of retained terms plus any supplied approximation-matrix cost. The provenance registry records `PSC-V-T17` as Exact retained packet reconstruction identity. Its recorded status is `proved`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_signals_wavelets_singular.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 36.

C.4B · retained_packet_reconstruction

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.retained_packet_reconstruction`. Reflected implementation `komorebi_math.signals.singular_packets.retained_packet_reconstruction`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-SINGULAR-PACKETS`.

Manifest signature (s).

```
(signal: 'Any', estimated_packet: 'Any', approximation_matrix: 'Any', *, true_smooth: 'Any | None' = None,
    ↪ true_packet: 'Any | None' = None, tolerance: 'float' = 1e-10) -> 'RetainedPacketResult'
```

Definition. Apply `Q(f - jhat) + jhat` and certify its exact error identity.

Contract (D, C, H, R, X). Truncated-power terms require finite locations and amplitudes with nonnegative integer order. Packet construction requires compatible term data. Reconstruction helpers require finite arrays of matching shape, and approximation matrices must have compatible dimensions. Region and location error helpers require nonnegative measures, compatible labels, and a positive finite exponent or infinity.

Guarantees (E, P, B, K). Singular packets evaluate, remove, and restore explicit truncated-power components. Retained-packet reconstruction records the smooth approximation, packet estimate, full reconstruction, and the exact finite error identity represented by those arrays. Location bounds are exact at jump order zero and use the documented mean-value estimate at positive orders. Runtime follows the number of samples times the number of retained terms plus any supplied approximation-matrix cost.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_signals_wavelets_singular.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 36.

C.4B · rls_filter

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.rls_filter`. Reflected implementation `komorebi_math.signals.adaptive.rls_filter`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-ADAPTIVE`.

Manifest signature (s).

```
(input_signal: 'Any', desired_signal: 'Any', num_taps: 'int', *, forgetting_factor: 'float' = 0.99, delta: 'float'
   ↪  = 1.0, initial_weights: 'Any | None' = None) -> 'AdaptiveFilterResult'
```

Definition. Run finite real or complex recursive least squares.

Contract (D, C, H, R, X). Adaptive filters require paired finite one-dimensional records of equal nonzero length and a positive tap count. Learning rates, leakage, regularization, forgetting factors, and initialization parameters must satisfy the finite ranges stated by the selected recursion. The result records the full output, error, final coefficients, coefficient history, method label, and finite-run certificate. The certificate does not assert persistent excitation, stochastic convergence, or population optimality.

Guarantees (E, P, B, K). LMS and normalized LMS use one bounded coefficient update per sample. RLS also updates a dense inverse-correlation matrix. LMS work scales with sample count times tap count. RLS work scales with sample count times the square of the tap count. Stored coefficient histories scale with sample count times tap count. The provenance registry records `KM-DSP-D14` as Finite recursive least-squares adaptation. Its recorded status is `classical input`. The provenance record notes that the finite routine does not assert a stochastic identification theorem.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_resampling_adaptive.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 34.

C.4B · scalar_region_error

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.scalar_region_error`. Reflected implementation `komorebi_math.signals.singular_packets.scalar_region_error`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-SINGULAR-PACKETS`.

Manifest signature (s).

```
(true_mask: 'Any', true_value: 'complex', estimated_mask: 'Any', estimated_value: 'complex', *, p: 'float' = 2.0,
   ↪  tolerance: 'float' = 1e-12) -> 'ComputationResult[float]'
```

Definition. Certify the exact empirical overlap formula for two scalar regions.

Contract (D, C, H, R, X). Truncated-power terms require finite locations and amplitudes with nonnegative integer order. Packet construction requires compatible term data. Reconstruction helpers require finite arrays of matching shape, and approximation matrices must have compatible dimensions. Region and location error helpers require nonnegative measures, compatible labels, and a positive finite exponent or infinity.

Guarantees (E, P, B, K). Singular packets evaluate, remove, and restore explicit truncated-power components. Retained-packet reconstruction records the smooth approximation, packet estimate, full reconstruction, and the exact finite error identity represented by those arrays. Location bounds are exact at jump order zero and use the

documented mean-value estimate at positive orders. Runtime follows the number of samples times the number of retained terms plus any supplied approximation-matrix cost.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_signals_wavelets_singular.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 36.

C.4B · `scalar_region_lp_error`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.scalar_region_lp_error`. Reflected implementation `komorebi_math.signals.singular_packets.scalar_region_lp_error`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-SINGULAR-PACKETS`.

Manifest signature (s).

```
(true_value: 'complex', estimated_value: 'complex', intersection_measure: 'float', true_only_measure: 'float',
    ↪ estimated_only_measure: 'float', *, p: 'float' = 2.0) -> 'float'
```

Definition. Return the exact scalar region packet error from overlap measures.

Contract (D, C, H, R, X). Truncated-power terms require finite locations and amplitudes with nonnegative integer order. Packet construction requires compatible term data. Reconstruction helpers require finite arrays of matching shape, and approximation matrices must have compatible dimensions. Region and location error helpers require nonnegative measures, compatible labels, and a positive finite exponent or infinity.

Guarantees (E, P, B, K). Singular packets evaluate, remove, and restore explicit truncated-power components. Retained-packet reconstruction records the smooth approximation, packet estimate, full reconstruction, and the exact finite error identity represented by those arrays. Location bounds are exact at jump order zero and use the documented mean-value estimate at positive orders. Runtime follows the number of samples times the number of retained terms plus any supplied approximation-matrix cost.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_signals_wavelets_singular.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 36.

C.4B · `shannon_entropy_cost`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.shannon_entropy_cost`. Reflected implementation `komorebi_math.signals.wavelets.shannon_entropy_cost`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-WAVELET-PACKETS`.

Manifest signature (s).

```
(coefficients: 'np.ndarray', path: 'str' = '') -> 'float'
```

Definition. Compatibility alias for `entropy_cost`.

Contract (D, C, H, R, X). Haar packet trees require a finite nonempty one-dimensional signal, a nonnegative depth, and a length divisible by the corresponding power of two. Packet paths contain only binary branches. A pruning must cover the full tree without ancestor and descendant conflicts. Additive cost functions must return finite values. Lp exponents must be positive and finite where required.

Guarantees (E, P, B, K). Normalized Haar packet analysis preserves finite energy to numerical tolerance. Synthesis from a valid full tree or pruning reconstructs the supplied record under the stored tolerance. Best-basis selection is

exact dynamic programming for the declared additive node cost. Tree construction and best-basis work scale with the number of stored coefficients across levels.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_signals_wavelets_singular.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 35.

C.4B · smith_sensitivity

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.smith_sensitivity`. Reflected implementation `komorebi_math.signals.smith.smith_sensitivity`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-SMITH`.

Manifest signature (s).

```
(gamma: 'ArrayLike', *, regularization: 'float' = 0.0) -> 'np.ndarray'
```

Definition. Return a regularized pointwise sensitivity for the impedance Cayley chart.

Contract (D, C, H, R, X). Smith-plane transforms accept finite complex scalars or arrays and reject poles in their fractional-linear denominators. Phase rotations require a positive integer order and reduce the phase modulo that order. Sensitivity regularization must be nonnegative.

Guarantees (E, P, B, K). Impedance and reflection transforms are inverse pointwise Möbius maps away from their poles. Phase rotation is a deterministic unit-modulus action. Sensitivity is evaluated pointwise with the declared regularization. Work and storage are linear in the number of supplied values.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_signals_extended.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 36.

C.4B · smooth_haar_coefficient_bound

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.smooth_haar_coefficient_bound`. Reflected implementation `komorebi_math.signals.edge_recovery.smooth_haar_coefficient_bound`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-VECTOR-RECOVERY`.

Manifest signature (s).

```
(lipschitz_bound: 'float', interval_length: 'float') -> 'float'
```

Definition. Return the normalized Haar coefficient bound `M L^(3/2) / 4`.

Contract (D, C, H, R, X). Vector jump recovery accepts finite scalar or vector samples with at least two sample locations. Spacing and minimum-jump parameters must be positive. Smoothness and noise bounds must be nonnegative. Threshold separation assumptions remain explicit and conditional. Subgaussian wrappers require a positive confidence level strictly below one and return a dimension-aware finite noise bound.

Guarantees (E, P, B, K). Recovery records expose cells, estimated locations, amplitudes, contrasts, and a conditional certificate. Smooth Haar bounds and step coefficients are exact finite formulas under their stated conventions. The recovery guarantee is conditional on the declared smoothness, spacing, separation, and noise hypotheses. Work is linear in the number of candidate cells times the signal dimension.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_signals_recovery.py` and `tests/test_block4_hardening.py`. The independent ver-

ifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 36.

C.4B · `split_mixed_radix_address`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.signals.split_mixed_radix_address`. Reflected implementation `komorebi_math.signals.polyphase.split_mixed_radix_address`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-POLYPHASE`.

Manifest signature (`s`).

```
(address: 'int', radices: 'Iterable[int]') -> 'tuple[int, ...]'
```

Definition. Recover least significant first mixed radix digits.

Contract (`D, C, H, R, X`). Polyphase orders, strides, and radices must be positive integers. Axis selections must be valid for the supplied array, and scalar arrays are rejected where an axis decomposition is required. Residue indices and mixed-radix addresses must lie in range. Lattice coordinates require one channel for every Cartesian residue address. Refinement requires exact divisibility of the fine stride by the coarse stride.

Guarantees (`E, P, B, K`). Analysis partitions the supplied finite array into explicit residue channels and synthesis restores their interleaving. Roundtrip certificates compare the restored array with the source under the stored tolerance. Mixed-radix addressing is least-significant digit first. Polyphase filtering exposes finite channel kernels with omitted coefficients treated as zero. Work and storage are linear in the represented array size plus the active kernel coefficients.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_signals_polyphase_filterbanks.py` and `tests/test_block15_polyphase_refinement.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapters 14 and 35.

C.4B · `step_haar_coefficient`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.signals.step_haar_coefficient`. Reflected implementation `komorebi_math.signals.edge_recovery.step_haar_coefficient`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-VECTOR-RECOVERY`.

Manifest signature (`s`).

```
(amplitude: 'float', relative_location: 'float', interval_length: 'float') -> 'float'
```

Definition. Return the magnitude of a normalized Haar coefficient of one step.

Contract (`D, C, H, R, X`). Vector jump recovery accepts finite scalar or vector samples with at least two sample locations. Spacing and minimum-jump parameters must be positive. Smoothness and noise bounds must be nonnegative. Threshold separation assumptions remain explicit and conditional. Subgaussian wrappers require a positive confidence level strictly below one and return a dimension-aware finite noise bound.

Guarantees (`E, P, B, K`). Recovery records expose cells, estimated locations, amplitudes, contrasts, and a conditional certificate. Smooth Haar bounds and step coefficients are exact finite formulas under their stated conventions. The recovery guarantee is conditional on the declared smoothness, spacing, separation, and noise hypotheses. Work is linear in the number of candidate cells times the signal dimension.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_signals_recovery.py` and `tests/test_block4_hardening.py`. The independent ver-

ifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 36.

C.4B · step_location_lp_error

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.step_location_lp_error`. Reflected implementation `komorebi_math.signals.singular_packets.step_location_lp_error`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-SINGULAR-PACKETS`.

Manifest signature (s).

```
(amplitude: 'complex', location: 'float', estimated_location: 'float', *, p: 'float' = 2.0) -> 'float'
```

Definition. Return the exact `L^p` distance between equal amplitude steps.

Contract (D, C, H, R, X). Truncated-power terms require finite locations and amplitudes with nonnegative integer order. Packet construction requires compatible term data. Reconstruction helpers require finite arrays of matching shape, and approximation matrices must have compatible dimensions. Region and location error helpers require nonnegative measures, compatible labels, and a positive finite exponent or infinity.

Guarantees (E, P, B, K). Singular packets evaluate, remove, and restore explicit truncated-power components. Retained-packet reconstruction records the smooth approximation, packet estimate, full reconstruction, and the exact finite error identity represented by those arrays. Location bounds are exact at jump order zero and use the documented mean-value estimate at positive orders. Runtime follows the number of samples times the number of retained terms plus any supplied approximation-matrix cost.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_signals_wavelets_singular.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 36.

C.4B · subgaussian_noise_bound

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.subgaussian_noise_bound`. Reflected implementation `komorebi_math.signals.recovery.subgaussian_noise_bound`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-SCALAR-RECOVERY`.

Manifest signature (s).

```
(sample_count: 'int', sigma: 'float', delta: 'float') -> 'float'
```

Definition. Return the simultaneous adjacent contrast bound used by Paper V.

Contract (D, C, H, R, X). Scalar recovery requires a finite one-dimensional record with at least two samples. Spacing and minimum-jump parameters must be positive. Lipschitz and noise bounds must be nonnegative. The threshold gap must remain strict, and any minimum-separation value must be at least one sample spacing. Difference order is nonnegative and the finite step size is positive.

Guarantees (E, P, B, K). The recovery theorem is conditional on the stored smoothness, separation, and noise hypotheses. Finite differences and Haar formulas follow their explicit index and normalization conventions. Subgaussian bounds are finite confidence bounds for the declared sigma and delta. Candidate scanning is linear in sample count for fixed difference order.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_signals_recovery.py`. The independent verifier for signal recovery and filter banks

exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 36.

C.4B · sure_threshold

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.sure_threshold`. Reflected implementation `komorebi_math.signals.denoise.sure_threshold`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-DENOISE`.

Manifest signature (s).

```
(coefficients: 'Any', sigma: 'float') -> 'float'
```

Definition. Return the finite Stein-risk minimizing soft threshold over observed magnitudes.

Contract (D, C, H, R, X). Denoising functions require finite nonempty one-dimensional records. Haar decomposition requires a positive level count and a length divisible by the corresponding power of two. Threshold values and noise scales must be nonnegative. The fixed rule requires an explicit threshold, while universal and SURE rules derive one from the supplied or estimated scale. Risk certificates compare one supplied finite clean reference and do not establish unobserved statistical risk.

Guarantees (E, P, B, K). Normalized Haar analysis preserves finite energy to numerical tolerance. Thresholding is deterministic for the selected mode and rule. Reconstruction uses the stored multilevel coefficient layout. Work is linear in signal length per decomposition and reconstruction pass. Storage is linear in signal length plus the recorded thresholds and coefficient arrays.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_denoise_streaming.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 35.

C.4B · threshold_coefficients

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.threshold_coefficients`. Reflected implementation `komorebi_math.signals.denoise.threshold_coefficients`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-DENOISE`.

Manifest signature (s).

```
(coefficients: 'Any', threshold: 'float', *, mode: 'ThresholdMode' = 'soft') -> 'np.ndarray'
```

Definition. Apply hard, soft, or nonnegative-garrote shrinkage coefficientwise.

Contract (D, C, H, R, X). Denoising functions require finite nonempty one-dimensional records. Haar decomposition requires a positive level count and a length divisible by the corresponding power of two. Threshold values and noise scales must be nonnegative. The fixed rule requires an explicit threshold, while universal and SURE rules derive one from the supplied or estimated scale. Risk certificates compare one supplied finite clean reference and do not establish unobserved statistical risk.

Guarantees (E, P, B, K). Normalized Haar analysis preserves finite energy to numerical tolerance. Thresholding is deterministic for the selected mode and rule. Reconstruction uses the stored multilevel coefficient layout. Work is linear in signal length per decomposition and reconstruction pass. Storage is linear in signal length plus the recorded thresholds and coefficient arrays.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_denoise_streaming.py`. The independent verifier for signal recovery and

filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 35.

C.4B · universal_threshold

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.universal_threshold`. Reflected implementation `komorebi_math.signals.denoise.universal_threshold`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-DENOISE`.

Manifest signature (s).

```
(sigma: 'float', coefficient_count: 'int') -> 'float'
```

Definition. Return the classical finite universal threshold `sigma*sqrt(2 log n)`.

Contract (D, C, H, R, X). Denoising functions require finite nonempty one-dimensional records. Haar decomposition requires a positive level count and a length divisible by the corresponding power of two. Threshold values and noise scales must be nonnegative. The fixed rule requires an explicit threshold, while universal and SURE rules derive one from the supplied or estimated scale. Risk certificates compare one supplied finite clean reference and do not establish unobserved statistical risk.

Guarantees (E, P, B, K). Normalized Haar analysis preserves finite energy to numerical tolerance. Thresholding is deterministic for the selected mode and rule. Reconstruction uses the stored multilevel coefficient layout. Work is linear in signal length per decomposition and reconstruction pass. Storage is linear in signal length plus the recorded thresholds and coefficient arrays.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_denoise_streaming.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 35.

C.4B · wavelet_denoise

Identity (q, a, t, k). Public research-manifest route `komorebi_math.signals.wavelet_denoise`. Reflected implementation `komorebi_math.signals.denoise.wavelet_denoise`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.signals` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C4-DENOISE`.

Manifest signature (s).

```
(signal: 'Any', *, levels: 'int' = 1, mode: 'ThresholdMode' = 'soft', rule: 'ThresholdRule' = 'universal',
    ↪ threshold: 'float | None' = None, sigma: 'float | None' = None, level_dependent: 'bool' = False, normalized:
    ↪ 'bool' = True) -> 'WaveletDenoiseResult'
```

Definition. Denoise a finite signal by thresholding multilevel Haar detail channels.

Contract (D, C, H, R, X). Denoising functions require finite nonempty one-dimensional records. Haar decomposition requires a positive level count and a length divisible by the corresponding power of two. Threshold values and noise scales must be nonnegative. The fixed rule requires an explicit threshold, while universal and SURE rules derive one from the supplied or estimated scale. Risk certificates compare one supplied finite clean reference and do not establish unobserved statistical risk.

Guarantees (E, P, B, K). Normalized Haar analysis preserves finite energy to numerical tolerance. Thresholding is deterministic for the selected mode and rule. Reconstruction uses the stored multilevel coefficient layout. Work is linear in signal length per decomposition and reconstruction pass. Storage is linear in signal length plus the recorded thresholds and coefficient arrays. The provenance registry records `KM-DSP-D10` as Finite Haar coefficient shrinkage.

Its recorded status is `classical input`. The provenance record notes that noise-model optimality is not inferred from one finite signal.

Evidence and links (`V`, `L`). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_block12_denoise_streaming.py`. The independent verifier for signal recovery and filter banks exercises the finite identities, roundtrips, rejection paths, aliases, and implementation ownership relevant to this contract. The source validator for signal recovery and filter banks checks the exact signature, cumulative counts, unique anchor, and preserved prior cards. See Chapter 35.

Signal recovery reference. The 91 cards in C.4B cover 25 classes and 66 functions. Together with C.4A, they document all 171 entries in the signal namespace. Their recorded audit checks the manifest and runtime identities, focused tests, independent finite checks, and reference anchors.

C.5 `komorebi_math.nonlinear`: 149 entries

C.5A Differential calculus, obstruction, lifting, and transport: 80 entries

This section documents the finite nonlinear foundation below packet-jet execution. It covers derivative sources, automatic differentiation adapters, Faà di Bruno composition, nonlinear packet coordinates, obstruction and correction systems, exact prime-power lifting, filtered and graded structures, finite holonomy, and smooth packet transport. The 69 cards in C.5B concern packet jets, sparse tensors, execution plans, GPU boundaries, streaming, profiling, and rooted-tree expansions.

Contract family	Cards
Derivative providers and automatic differentiation	14
Faà di Bruno composition and partition combinatorics	9
Nonlinear coordinates and refinement defects	2
Obstruction, correction, and Schur elimination	14
Filtered correction and spectral sequences	11
Graded operations and Maurer Cartan packets	11
Finite graph holonomy	4
Prime power lifting	8
Smooth packet transport	6
Namespace status	1
Total	**80**

The boundary is deliberately finite. Certificates resolve represented maps, matrices, filtrations, congruences, graphs, and packet families under explicit tolerances or exact finite arithmetic. They do not convert sampled derivatives into global smoothness, finite lift trees into unrestricted p-adic existence, or numerical transport residuals into continuum convergence theorems.

C.5A · `AutomaticDifferentiationProvider`

Identity (`q`, `a`, `t`, `k`). Public research-manifest route `komorebi_math.nonlinear.AutomaticDifferentiationProvider`. Reflected implementation `komorebi_math.nonlinear.autodiff.AutomaticDifferentiationProvider`. The same runtime object is also available through `komorebi_math.AutomaticDifferentiationProvider`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-AUTODIFF`.

Manifest signature (s).

```
(function: 'Callable[[Any], Any]', jvp: 'DirectionalJVP', source_dimension: 'int', target_dimension: 'int',
   ↪ max_order: 'int' = 4, exactness: 'Exactness' = <Exactness.NUMERICAL: 'numerical'>, name: 'str' = 'automatic
   ↪ differentiation provider') -> None
```

Definition. Framework-neutral higher derivatives obtained by nested JVP calls.

Contract (D, C, H, R, X). Automatic differentiation adapters require a callable finite dimensional map, positive source and target dimensions, a declared maximum order, and a directional Jacobian vector product primitive supplied either directly or by an optional framework. Direction counts must equal the requested derivative order. Framework discovery is isolated in child processes so that a broken optional runtime cannot terminate the calling process. Principal public operations and properties are `supports_order`, `value`, `derivative`, `validation_certificate`, `to_dict`.

Guarantees (E, P, B, K). A successful provider returns the finite directional derivative computed by the selected JVP recursion and records numerical exactness unless the caller supplies a stronger justified classification. Framework availability means that an isolated probe imported the runtime and found the required JVP capability. It does not assert numerical agreement for every model or device. Work grows with derivative order and the number of nested JVP calls. The provenance registry records `PSC-III-N04A` as Directional automatic-differentiation adapter. Its recorded status is `definition`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_nonlinear_sparse_autodiff.py`, `tests/test_block10_sparse_autodiff.py`, and `tests/test_optional_backend_probe.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 37.

C.5A · `CallableDerivativeProvider`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.CallableDerivativeProvider`. Reflected implementation `komorebi_math.nonlinear.derivatives.CallableDerivativeProvider`. The same runtime object is also available through `komorebi_math.CallableDerivativeProvider`, `komorebi_math.beta.CallableDerivativeProvider`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-DERIVATIVES`.

Manifest signature (s).

```
(function: 'Callable[[Any], Any]', derivatives: 'Mapping[int, DirectionalDerivative]', source_dimension: 'int',
   ↪ target_dimension: 'int', exactness: 'Exactness' = <Exactness.NUMERICAL: 'numerical'>, name: 'str' =
   ↪ 'declared derivative provider') -> None
```

Definition. A map with explicitly supplied directional derivative callables.

Contract (D, C, H, R, X). Derivative providers expose one common finite dimensional interface for map evaluation and directional derivatives. A call must use an order supported by the provider and exactly that many direction vectors of the declared source dimension. Callable and polynomial providers trust explicitly supplied derivative data. The finite difference provider additionally requires a positive finite step. The SymPy provider requires the optional symbolic dependency. Principal public operations and properties are `available_orders`, `supports_order`, `value`, `derivative`, `validation_certificate`.

Guarantees (E, P, B, K). Polynomial and symbolic providers preserve exact objects where their inputs permit exact arithmetic. Callable providers inherit the exactness declared by the caller. Finite differences return numerical evidence only. Every provider checks dimensions and finite values before evaluation. Tensor contraction cost grows with derivative order and ambient dimension. The provenance registry records `PSC-III-D02` as Declared finite dimensional derivative provider. Its recorded status is `definition`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_nonlinear_jets_composition.py` and `tests/test_nonlinear_sparse_autodiff.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus

foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapters 37 and 38.

C.5A · ComposedDerivativeProvider

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.ComposedDerivativeProvider`. Reflected implementation `komorebi_math.nonlinear.composition.ComposedDerivativeProvider`. The same runtime object is also available through `komorebi_math.ComposedDerivativeProvider`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-COMPOSITION`.

Manifest signature (s).

```
(outer: 'DerivativeProvider', inner: 'DerivativeProvider', *, name: 'str | None' = None) -> 'None'
```

Definition. Expose a composition as another derivative provider.

Contract (D, C, H, R, X). Composition requires compatible inner and outer dimensions and derivative support through the requested order. The Faà di Bruno routines enumerate set partitions of the direction labels. Packet composition also requires compatible source, intermediate, and target packet systems. Partition utilities accept nonnegative finite orders and return canonical immutable records. Principal public operations and properties are `supports_order`, `value`, `derivative`.

Guarantees (E, P, B, K). For the composition operations in this family, the computed derivative value equals the implemented finite Faà di Bruno sum for the supplied providers and directions. Exactness follows the participating providers. The composition result records retained terms, partition counts, and a certificate comparing the composed derivative with the direct implementation when that comparison is available. Enumeration grows according to Bell and Stirling counts, so callers must keep the derivative order bounded. The provenance registry records `PSC-III-D12` as Composed derivative provider. Its recorded status is `definition`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_nonlinear_jets_composition.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 38.

C.5A · DepthObstructionResult

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.DepthObstructionResult`. Reflected implementation `komorebi_math.nonlinear.filtered.DepthObstructionResult`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-FILTERED`.

Manifest signature (s).

```
(depth: 'int', induced_correction_map: 'np.ndarray', defect_coordinates: 'tuple[int, ...]', correction_space:
    ↪ 'ModularLinearSolution', correction_representatives: 'tuple[tuple[int, ...], ...]', certificate:
    ↪ 'Certificate') -> None
```

Definition. One finite filtration-depth obstruction and its correction space.

Contract (D, C, H, R, X). Filtered contracts require finite dimensional vector spaces over an explicit prime field and decreasing filtration bases with compatible ambient dimensions. Filtered correction is solved one finite depth at a time. A filtered cochain complex requires explicit differentials that square to zero and preserve the declared filtration. Projective limit completion also requires external completeness hypotheses supplied by the caller. Principal public operations and properties are `vanishes`.

Guarantees (E, P, B, K). Associated graded coordinates, correction cosets, finite branches, and spectral sequence pages are exact over the represented finite field. The terminal finite filtration permits exact page dimension and stabiliza-

tion checks. A projective limit record only stores the declared infinite hypotheses and finite compatibility evidence. It does not prove completeness or convergence beyond those assumptions.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_nonlinear_filtered.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 42.

C.5A · DerivativeProvider

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.DerivativeProvider`. Reflected implementation `komorebi_math.nonlinear.derivatives.DerivativeProvider`. The same runtime object is also available through `komorebi_math.beta.DerivativeProvider`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-DERIVATIVES`.

Manifest signature (s).

```
(*args, **kwargs)
```

Definition. Protocol implemented by finite dimensional derivative sources.

Contract (D, C, H, R, X). Derivative providers expose one common finite dimensional interface for map evaluation and directional derivatives. A call must use an order supported by the provider and exactly that many direction vectors of the declared source dimension. Callable and polynomial providers trust explicitly supplied derivative data. The finite difference provider additionally requires a positive finite step. The SymPy provider requires the optional symbolic dependency. Principal public operations and properties are `value`, `derivative`, `supports_order`.

Guarantees (E, P, B, K). Polynomial and symbolic providers preserve exact objects where their inputs permit exact arithmetic. Callable providers inherit the exactness declared by the caller. Finite differences return numerical evidence only. Every provider checks dimensions and finite values before evaluation. Tensor contraction cost grows with derivative order and ambient dimension.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_nonlinear_jets_composition.py` and `tests/test_nonlinear_sparse_autodiff.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapters 37 and 38.

C.5A · DirectRotationResult

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.DirectRotationResult`. Reflected implementation `komorebi_math.nonlinear.transport.DirectRotationResult`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-SMOOTH-TRANSPORT`.

Manifest signature (s).

```
(unitary: 'np.ndarray', minimum_singular_value: 'float', certificate: 'Certificate') -> None
```

Definition. Polar direct rotation between two nearby packet systems.

Contract (D, C, H, R, X). Smooth transport requires complete finite orthogonal packet families of fixed dimension together with declared packet derivatives. Direct rotation requires two compatible packet families whose overlap sum is numerically invertible. Kato transport requires finite start and end times, a positive step count, and a positive tolerance.

Guarantees (E, P, B, K). Direct polar rotation returns a finite unitary intertwiner when its residual checks pass. Kato transport integrates the canonical finite generator with midpoint exponential steps and records unitarity, packet

transport, and mesh refinement residuals. These certificates are numerical and path specific. They do not prove differentiability of an undeclared family or continuum convergence without external estimates.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_nonlinear_transport.py` and `tests/test_quantum_transport_effective_spectral.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapters 40 and 44.

C.5A · DirectionalDerivative

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.DirectionalDerivative`. Reflected implementation `collections.abc.Callable`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The public export name maps to implementation object `Callable`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `data`. Profile `C5-DERIVATIVES`.

Manifest signature (s).

```
(*args, **kwargs)
```

Definition. Callable type alias for one finite directional derivative evaluator.

Contract (D, C, H, R, X). Derivative providers expose one common finite dimensional interface for map evaluation and directional derivatives. A call must use an order supported by the provider and exactly that many direction vectors of the declared source dimension. Callable and polynomial providers trust explicitly supplied derivative data. The finite difference provider additionally requires a positive finite step. The SymPy provider requires the optional symbolic dependency.

Guarantees (E, P, B, K). Polynomial and symbolic providers preserve exact objects where their inputs permit exact arithmetic. Callable providers inherit the exactness declared by the caller. Finite differences return numerical evidence only. Every provider checks dimensions and finite values before evaluation. Tensor contraction cost grows with derivative order and ambient dimension.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_nonlinear_jets_composition.py` and `tests/test_nonlinear_sparse_autodiff.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapters 37 and 38.

C.5A · DirectionalJVP

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.DirectionalJVP`. Reflected implementation `komorebi_math.nonlinear.autodiff.DirectionalJVP`. The same runtime object is also available through `komorebi_math.DirectionalJVP`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-AUTODIFF`.

Manifest signature (s).

```
(*args, **kwargs)
```

Definition. One forward-mode Jacobian-vector product primitive.

Contract (D, C, H, R, X). Automatic differentiation adapters require a callable finite dimensional map, positive source and target dimensions, a declared maximum order, and a directional Jacobian vector product primitive supplied either directly or by an optional framework. Direction counts must equal the requested derivative order. Framework discovery is isolated in child processes so that a broken optional runtime cannot terminate the calling process.

Guarantees (E, P, B, K). A successful provider returns the finite directional derivative computed by the selected JVP recursion and records numerical exactness unless the caller supplies a stronger justified classification. Framework

availability means that an isolated probe imported the runtime and found the required JVP capability. It does not assert numerical agreement for every model or device. Work grows with derivative order and the number of nested JVP calls.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_nonlinear_sparse_autodiff.py`, `tests/test_block10_sparse_autodiff.py`, and `tests/test_optional_backend_probe.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 37.

C.5A · DividedSquareResult

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.DividedSquareResult`. Reflected implementation `komorebi_math.nonlinear.graded.DividedSquareResult`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-GRADED`.

Manifest signature (s).

```
(equations: 'dict[str, Any]', reconstructed: 'Any', direct: 'Any', certificate: 'Certificate') -> None
```

Definition. Packet divided-square equations and direct reconstruction result.

Contract (D, C, H, R, X). Graded operations require a finite abelian grading group, finite component records, and multilinear operations whose declared output grade is the group sum of the input grades. Maurer Cartan routines evaluate only the supplied finite arity range. Divided square evaluation requires the supplied differential, bracket, and divided square maps to have compatible component shapes.

Guarantees (E, P, B, K). Grade bookkeeping and finite group arithmetic are exact. Numerical component values inherit the arithmetic of the supplied arrays and operations. Certificates compare the gradewise reconstruction with the direct finite expression and record any failed compatibility law. No unspecified higher operation or infinite Maurer Cartan series is inferred.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_nonlinear_graded.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 39.

C.5A · FaaDiBrunoResult

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.FaaDiBrunoResult`. Reflected implementation `komorebi_math.nonlinear.composition.FaaDiBrunoResult`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-COMPOSITION`.

Manifest signature (s).

```
(value: 'Any', terms: 'tuple[FaaDiBrunoTerm, ...]', raw_term_count: 'int', evaluated_term_count: 'int',
    ↪ pruned_term_count: 'int', certificate: 'Certificate') -> None
```

Definition. A higher composition value with term and validation records.

Contract (D, C, H, R, X). Composition requires compatible inner and outer dimensions and derivative support through the requested order. The Faà di Bruno routines enumerate set partitions of the direction labels. Packet composition also requires compatible source, intermediate, and target packet systems. Partition utilities accept nonnegative finite orders and return canonical immutable records.

Guarantees (E, P, B, K). For the composition operations in this family, the computed derivative value equals the implemented finite Faà di Bruno sum for the supplied providers and directions. Exactness follows the participating providers. The composition result records retained terms, partition counts, and a certificate comparing the composed derivative with the direct implementation when that comparison is available. Enumeration grows according to Bell and Stirling counts, so callers must keep the derivative order bounded.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_nonlinear_jets_composition.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 38.

C.5A · FaaDiBrunoTerm

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.FaaDiBrunoTerm`. Reflected implementation `komorebi_math.nonlinear.composition.FaaDiBrunoTerm`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-COMPOSITION`.

Manifest signature (s).

```
(partition: 'Partition', intermediate_packets: 'tuple[str, ...]', value: 'Any', pruned: 'bool' = False, reason:
   ↪  'str' = '') -> None
```

Definition. One classical or packet-labeled composition term.

Contract (D, C, H, R, X). Composition requires compatible inner and outer dimensions and derivative support through the requested order. The Faà di Bruno routines enumerate set partitions of the direction labels. Packet composition also requires compatible source, intermediate, and target packet systems. Partition utilities accept nonnegative finite orders and return canonical immutable records. Principal public operations and properties are `to_dict`.

Guarantees (E, P, B, K). For the composition operations in this family, the computed derivative value equals the implemented finite Faà di Bruno sum for the supplied providers and directions. Exactness follows the participating providers. The composition result records retained terms, partition counts, and a certificate comparing the composed derivative with the direct implementation when that comparison is available. Enumeration grows according to Bell and Stirling counts, so callers must keep the derivative order bounded.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_nonlinear_jets_composition.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 38.

C.5A · FilteredCorrectionProblem

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.FilteredCorrectionProblem`. Reflected implementation `komorebi_math.nonlinear.filtered.FilteredCorrectionProblem`. The same runtime object is also available through `komorebi_math.FilteredCorrectionProblem`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-FILTERED`.

Manifest signature (s).

```
(source: 'FilteredVectorSpace', target: 'FilteredVectorSpace', correction_map: 'Any', defect: 'Any', depth: 'int')
   ↪  -> 'None'
```

Definition. One associated-graded correction problem over a prime field.

Contract (D, C, H, R, X). Filtered contracts require finite dimensional vector spaces over an explicit prime field and decreasing filtration bases with compatible ambient dimensions. Filtered correction is solved one finite depth at a

time. A filtered cochain complex requires explicit differentials that square to zero and preserve the declared filtration. Projective limit completion also requires external completeness hypotheses supplied by the caller. Principal public operations and properties are `induced_map`, `solve`.

Guarantees (`E`, `P`, `B`, `K`). Associated graded coordinates, correction cosets, finite branches, and spectral sequence pages are exact over the represented finite field. The terminal finite filtration permits exact page dimension and stabilization checks. A projective limit record only stores the declared infinite hypotheses and finite compatibility evidence. It does not prove completeness or convergence beyond those assumptions. The provenance registry records `PSC-III-T51` as One-step filtered lifting criterion. Its recorded status is `proved`.

Evidence and links (`V`, `L`). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_nonlinear_filtered.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 42.

C.5A · `FilteredSpectralSequenceResult`

Identity (`q`, `a`, `t`, `k`). Public research-manifest route `komorebi_math.nonlinear.FilteredSpectralSequenceResult`. Reflected implementation `komorebi_math.nonlinear.filtered.FilteredSpectralSequenceResult`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-FILTERED`.

Manifest signature (`s`).

```
(pages: 'tuple[SpectralSequencePage, ...]', total_cohomology_dimensions: 'dict[int, int]', limiting_dimensions:
   ↪ 'dict[int, int]', certificate: 'Certificate') -> None
```

Definition. Finite spectral-sequence pages and limiting dimension comparison.

Contract (`D`, `C`, `H`, `R`, `X`). Filtered contracts require finite dimensional vector spaces over an explicit prime field and decreasing filtration bases with compatible ambient dimensions. Filtered correction is solved one finite depth at a time. A filtered cochain complex requires explicit differentials that square to zero and preserve the declared filtration. Projective limit completion also requires external completeness hypotheses supplied by the caller.

Guarantees (`E`, `P`, `B`, `K`). Associated graded coordinates, correction cosets, finite branches, and spectral sequence pages are exact over the represented finite field. The terminal finite filtration permits exact page dimension and stabilization checks. A projective limit record only stores the declared infinite hypotheses and finite compatibility evidence. It does not prove completeness or convergence beyond those assumptions.

Evidence and links (`V`, `L`). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_nonlinear_filtered.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 42.

C.5A · `FilteredVectorSpace`

Identity (`q`, `a`, `t`, `k`). Public research-manifest route `komorebi_math.nonlinear.FilteredVectorSpace`. Reflected implementation `komorebi_math.nonlinear.filtered.FilteredVectorSpace`. The same runtime object is also available through `komorebi_math.FilteredVectorSpace`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-FILTERED`.

Manifest signature (s).

```
(dimension: 'int', levels: 'Mapping[int, Any]', *, prime: 'int') -> 'None'
```

Definition. A finite decreasing filtration `F^0 V >= ... >= F^N V = 0`.

Contract (D, C, H, R, X). Filtered contracts require finite dimensional vector spaces over an explicit prime field and decreasing filtration bases with compatible ambient dimensions. Filtered correction is solved one finite depth at a time. A filtered cochain complex requires explicit differentials that square to zero and preserve the declared filtration. Projective limit completion also requires external completeness hypotheses supplied by the caller. Principal public operations and properties are `from_weights`, `basis`, `dimension_at`, `contains`, `graded_basis`, `graded_dimension`, `graded_coordinates`, `lift_graded`, `certificate`.

Guarantees (E, P, B, K). Associated graded coordinates, correction cosets, finite branches, and spectral sequence pages are exact over the represented finite field. The terminal finite filtration permits exact page dimension and stabilization checks. A projective limit record only stores the declared infinite hypotheses and finite compatibility evidence. It does not prove completeness or convergence beyond those assumptions. The provenance registry records `PSC-III-D50` as Finite decreasing filtration over a prime field. Its recorded status is `definition`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_nonlinear_filtered.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 42.

C.5A · FiniteAbelianGroup

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.FiniteAbelianGroup`. Reflected implementation `komorebi_math.nonlinear.graded.FiniteAbelianGroup`. The same runtime object is also available through `komorebi_math.FiniteAbelianGroup`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-GRADED`.

Manifest signature (s).

```
(labels: 'Sequence[str]', addition: 'Mapping[tuple[str, str], str]', *, zero: 'str', validate: 'bool' = True) ->
    ↪ 'None'
```

Definition. A finite abelian group represented by an explicit addition table.

Contract (D, C, H, R, X). Graded operations require a finite abelian grading group, finite component records, and multilinear operations whose declared output grade is the group sum of the input grades. Maurer Cartan routines evaluate only the supplied finite arity range. Divided square evaluation requires the supplied differential, bracket, and divided square maps to have compatible component shapes. Principal public operations and properties are `cyclic`, `add`, `sum`, `inverse`, `certificate`, `to_dict`.

Guarantees (E, P, B, K). Grade bookkeeping and finite group arithmetic are exact. Numerical component values inherit the arithmetic of the supplied arrays and operations. Certificates compare the gradewise reconstruction with the direct finite expression and record any failed compatibility law. No unspecified higher operation or infinite Maurer Cartan series is inferred. The provenance registry records `PSC-III-D17` as Finite abelian grading group. Its recorded status is `definition`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_nonlinear_graded.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 39.

C.5A · FiniteDifferenceDerivativeProvider

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.FiniteDifferenceDerivativeProvider`. Reflected implementation `komorebi_math.nonlinear.derivatives.FiniteDifferenceDerivativeProvider`. The same runtime object is also available through `komorebi_math.FiniteDifferenceDerivativeProvider`, `komorebi_math.beta.FiniteDifferenceDerivativeProvider`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-DERIVATIVES`.

Manifest signature (s).

```
(function: 'Callable[[np.ndarray], Any]', source_dimension: 'int', target_dimension: 'int', step: 'float' =
    ↪ 0.0001, max_order: 'int' = 4, name: 'str' = 'central finite difference provider', exactness: 'Exactness' =
    ↪ <Exactness.NUMERICAL: 'numerical'>) -> None
```

Definition. Numerical directional derivatives by recursive central differences.

Contract (D, C, H, R, X). Derivative providers expose one common finite dimensional interface for map evaluation and directional derivatives. A call must use an order supported by the provider and exactly that many direction vectors of the declared source dimension. Callable and polynomial providers trust explicitly supplied derivative data. The finite difference provider additionally requires a positive finite step. The SymPy provider requires the optional symbolic dependency. Principal public operations and properties are `supports_order`, `value`, `derivative`.

Guarantees (E, P, B, K). Polynomial and symbolic providers preserve exact objects where their inputs permit exact arithmetic. Callable providers inherit the exactness declared by the caller. Finite differences return numerical evidence only. Every provider checks dimensions and finite values before evaluation. Tensor contraction cost grows with derivative order and ambient dimension. The provenance registry records `PSC-III-N03` as Recursive central directional differences. Its recorded status is `finite certificate`. The provenance record notes that roundoff and truncation error grow with derivative order.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_nonlinear_jets_composition.py` and `tests/test_nonlinear_sparse_autodiff.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapters 37 and 38.

C.5A · FiniteFilteredBranchResult

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.FiniteFilteredBranchResult`. Reflected implementation `komorebi_math.nonlinear.filtered.FiniteFilteredBranchResult`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-FILTERED`.

Manifest signature (s).

```
(approximations: 'tuple[tuple[int, ...], ...]', limit: 'tuple[int, ...]', exact_solution: 'bool', certificate:
    ↪ 'Certificate') -> None
```

Definition. Compatibility result for a branch through a terminal finite filtration.

Contract (D, C, H, R, X). Filtered contracts require finite dimensional vector spaces over an explicit prime field and decreasing filtration bases with compatible ambient dimensions. Filtered correction is solved one finite depth at a time. A filtered cochain complex requires explicit differentials that square to zero and preserve the declared filtration. Projective limit completion also requires external completeness hypotheses supplied by the caller.

Guarantees (E, P, B, K). Associated graded coordinates, correction cosets, finite branches, and spectral sequence pages are exact over the represented finite field. The terminal finite filtration permits exact page dimension and stabilization checks. A projective limit record only stores the declared infinite hypotheses and finite compatibility evidence. It does not prove completeness or convergence beyond those assumptions.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_nonlinear_filtered.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 42.

C.5A · FiniteFilteredCochainComplex

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.FiniteFilteredCochainComplex`. Reflected implementation `komorebi_math.nonlinear.filtered.FiniteFilteredCochainComplex`. The same runtime object is also available through `komorebi_math.FiniteFilteredCochainComplex`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-FILTERED`.

Manifest signature (s).

```
(dimensions: 'Mapping[int, int]', differentials: 'Mapping[int, Any]', filtrations: 'Mapping[int,
    ↪ FilteredVectorSpace]', *, prime: 'int') -> 'None'
```

Definition. An exact filtered cochain complex over a prime field.

Contract (D, C, H, R, X). Filtered contracts require finite dimensional vector spaces over an explicit prime field and decreasing filtration bases with compatible ambient dimensions. Filtered correction is solved one finite depth at a time. A filtered cochain complex requires explicit differentials that square to zero and preserve the declared filtration. Projective limit completion also requires external completeness hypotheses supplied by the caller. Principal public operations and properties are `differential`, `page`, `total_cohomology_dimensions`, `spectral_sequence`.

Guarantees (E, P, B, K). Associated graded coordinates, correction cosets, finite branches, and spectral sequence pages are exact over the represented finite field. The terminal finite filtration permits exact page dimension and stabilization checks. A projective limit record only stores the declared infinite hypotheses and finite compatibility evidence. It does not prove completeness or convergence beyond those assumptions. The provenance registry records `PSC-III-S54` as Spectral sequence of a finite filtered complex. Its recorded status is `classical input`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_nonlinear_filtered.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 42.

C.5A · FiniteLiftTree

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.FiniteLiftTree`. Reflected implementation `komorebi_math.nonlinear.lifting.FiniteLiftTree`. The same runtime object is also available through `komorebi_math.FiniteLiftTree`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-LIFTING`.

Manifest signature (s).

```
(prime: 'int', levels: 'tuple[tuple[LiftTreeNode, ...], ...]') -> None
```

Definition. A finite prefix of a prime power lift tree.

Contract (D, C, H, R, X). Linear and polynomial lifting require a prime, a nonnegative depth, compatible finite equations, and a current residue class. One step searches the declared corrections modulo the prime. Tree construction requires an explicit terminal depth and a finite node budget. Principal public operations and properties are `max_depth`, `terminal_branches`.

Guarantees (E, P, B, K). For the lifting operations in this family, every returned child satisfies the represented congruence at the next prime power. Linear solves and branch verification are exact in modular arithmetic. Polynomial lifting uses the declared Jacobian and finite Taylor congruence. Branch counts may grow exponentially with depth, so the tree builder enforces the supplied node limit.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_lifting.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 40.

C.5A · FundamentalHolonomy

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.FundamentalHolonomy`. Reflected implementation `komorebi_math.nonlinear.holonomy.FundamentalHolonomy`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-HOLONOMY`.

Manifest signature (s).

```
(edge_index: 'int', source: 'str', target: 'str', matrix: 'Any') -> None
```

Definition. One edge outside the tree and its based cycle holonomy.

Contract (D, C, H, R, X). A transport graph requires a finite vertex set and directed edges carrying compatible square matrices. Path transport requires composable endpoints. Gluing and intertwining checks require a chosen root and finite tolerance. Fundamental holonomy is computed from the selected spanning structure and recorded edges outside the tree.

Guarantees (E, P, B, K). The graph routines compute exact path products for exact input matrices or numerical products otherwise. Certificates compare finite cycle transports, root gauges, and intertwining residuals. The result describes the supplied finite graph. It is not a continuum connection or topological classification without additional hypotheses.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_holonomy.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 40.

C.5A · GradedConvolutionResult

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.GradedConvolutionResult`. Reflected implementation `komorebi_math.nonlinear.graded.GradedConvolutionResult`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-GRADED`.

Manifest signature (s).

```
(output_label: 'str', value: 'Any', direct_value: 'Any', terms: 'tuple[GradedConvolutionTerm, ...]', certificate:
    ↪ 'Certificate') -> None
```

Definition. Packet-resolved graded convolution and direct reconstruction result.

Contract (D, C, H, R, X). Graded operations require a finite abelian grading group, finite component records, and multilinear operations whose declared output grade is the group sum of the input grades. Maurer Cartan routines evaluate only the supplied finite arity range. Divided square evaluation requires the supplied differential, bracket, and divided square maps to have compatible component shapes.

Guarantees (E, P, B, K). Grade bookkeeping and finite group arithmetic are exact. Numerical component values inherit the arithmetic of the supplied arrays and operations. Certificates compare the gradewise reconstruction with the direct finite expression and record any failed compatibility law. No unspecified higher operation or infinite Maurer Cartan series is inferred.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_nonlinear_graded.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 39.

C.5A · GradedConvolutionTerm

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.GradedConvolutionTerm`. Reflected implementation `komorebi_math.nonlinear.graded.GradedConvolutionTerm`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-GRADED`.

Manifest signature (s).

```
(input_labels: 'tuple[str, ...]', value: 'Any') -> None
```

Definition. One labeled input contribution to a graded packet convolution.

Contract (D, C, H, R, X). Graded operations require a finite abelian grading group, finite component records, and multilinear operations whose declared output grade is the group sum of the input grades. Maurer Cartan routines evaluate only the supplied finite arity range. Divided square evaluation requires the supplied differential, bracket, and divided square maps to have compatible component shapes.

Guarantees (E, P, B, K). Grade bookkeeping and finite group arithmetic are exact. Numerical component values inherit the arithmetic of the supplied arrays and operations. Certificates compare the gradewise reconstruction with the direct finite expression and record any failed compatibility law. No unspecified higher operation or infinite Maurer Cartan series is inferred.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_nonlinear_graded.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 39.

C.5A · GradedPacketOperation

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.GradedPacketOperation`. Reflected implementation `komorebi_math.nonlinear.graded.GradedPacketOperation`. The same runtime object is also available through `komorebi_math.GradedPacketOperation`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-GRADED`.

Manifest signature (s).

```
(group: 'FiniteAbelianGroup', packets: 'PacketSystem', arity: 'int', operation: 'GradedOperationCallable', *,
   ↪ exactness: 'Exactness' = <Exactness.NUMERICAL: 'numerical'>, tolerance: 'float | None' = None, name: 'str' =
   ↪ 'graded packet operation') -> 'None'
```

Definition. A multilinear operation tested against a finite packet grading.

Contract (D, C, H, R, X). Graded operations require a finite abelian grading group, finite component records, and multilinear operations whose declared output grade is the group sum of the input grades. Maurer Cartan routines evaluate only the supplied finite arity range. Divided square evaluation requires the supplied differential, bracket, and divided square maps to have compatible component shapes. Principal public operations and properties are `evaluate`, `grading_certificate`, `packet_convolution`.

Guarantees (E, P, B, K). Grade bookkeeping and finite group arithmetic are exact. Numerical component values inherit the arithmetic of the supplied arrays and operations. Certificates compare the gradewise reconstruction with the direct finite expression and record any failed compatibility law. No unspecified higher operation or infinite

Maurer Cartan series is inferred. The provenance registry records `PSC-III-T18` as Graded packet convolution. Its recorded status is `proved`.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_nonlinear_graded.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 39.

C.5A · `GradingCertificate`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.nonlinear.GradingCertificate`. Reflected implementation `komorebi_math.nonlinear.graded.GradingCertificate`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-GRADED`.

Manifest signature (`s`).

```
(arity: 'int', tested_input_labels: 'int', forbidden_residual: 'float', certificate: 'Certificate') -> None
```

Definition. Finite certificate that a multilinear operation respects the grading.

Contract (`D, C, H, R, X`). Graded operations require a finite abelian grading group, finite component records, and multilinear operations whose declared output grade is the group sum of the input grades. Maurer Cartan routines evaluate only the supplied finite arity range. Divided square evaluation requires the supplied differential, bracket, and divided square maps to have compatible component shapes.

Guarantees (`E, P, B, K`). Grade bookkeeping and finite group arithmetic are exact. Numerical component values inherit the arithmetic of the supplied arrays and operations. Certificates compare the gradewise reconstruction with the direct finite expression and record any failed compatibility law. No unspecified higher operation or infinite Maurer Cartan series is inferred.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_nonlinear_graded.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 39.

C.5A · `HiddenSolveResult`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.nonlinear.HiddenSolveResult`. Reflected implementation `komorebi_math.nonlinear.elimination.HiddenSolveResult`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-SCHUR-ELIMINATION`.

Manifest signature (`s`).

```
(hidden: 'np.ndarray', point: 'np.ndarray', residual: 'float', iterations: 'int', certificate: 'Certificate') ->
    ↪ None
```

Definition. Finite hidden-variable solve with residual and iteration certificate.

Contract (`D, C, H, R, X`). Nonlinear Schur elimination requires retained and hidden derivative providers with a declared split of visible and hidden coordinates. The hidden derivative at the base point must be square and numerically invertible. Hidden solves require an initial point, a finite iteration limit, and a positive tolerance.

Guarantees (`E, P, B, K`). The first derivative contract implements the finite implicit derivative and reduced Schur derivative. The second derivative contract applies the declared provider derivatives through order two. Exact symbolic inputs may retain exact matrices. Numerical inputs receive residual certificates tied to the selected solve and base point. These are local finite statements, not global existence claims.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_nonlinear_correction_elimination.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 41.

C.5A · `HigherMaurerCartanResult`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.HigherMaurerCartanResult`. Reflected implementation `komorebi_math.nonlinear.graded.HigherMaurerCartanResult`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-GRADED`.

Manifest signature (s).

```
(equations: 'dict[str, Any]', reconstructed: 'Any', direct: 'Any', max_order: 'int', certificate: 'Certificate')
   ↪  -> None
```

Definition. Truncated higher packet equations through a declared maximum order.

Contract (D, C, H, R, X). Graded operations require a finite abelian grading group, finite component records, and multilinear operations whose declared output grade is the group sum of the input grades. Maurer Cartan routines evaluate only the supplied finite arity range. Divided square evaluation requires the supplied differential, bracket, and divided square maps to have compatible component shapes.

Guarantees (E, P, B, K). Grade bookkeeping and finite group arithmetic are exact. Numerical component values inherit the arithmetic of the supplied arrays and operations. Certificates compare the gradewise reconstruction with the direct finite expression and record any failed compatibility law. No unspecified higher operation or infinite Maurer Cartan series is inferred.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_nonlinear_graded.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 39.

C.5A · `HilbertObstructionProjector`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.HilbertObstructionProjector`. Reflected implementation `komorebi_math.nonlinear.correction.HilbertObstructionProjector`. The same runtime object is also available through `komorebi_math.HilbertObstructionProjector`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-CORRECTION`.

Manifest signature (s).

```
(correction_map: 'Any', *, tolerance: 'float' = 1e-10) -> 'None'
```

Definition. Moore–Penrose correction and orthogonal cokernel projection.

Contract (D, C, H, R, X). Correction contracts operate on finite matrices or derivative providers with explicit dimensions, tolerances, radii, right inverses, and Lipschitz bounds. Hilbert obstruction projection uses the declared linearized operator. Right inverse correction checks the supplied contraction inequalities. Lyapunov Schmidt reduction computes numerical kernel, complement, range, and obstruction coordinates at a specified base point. Principal public operations and properties are `solve`.

Guarantees (E, P, B, K). Certificates report finite residuals, rank decisions, obstruction norms, and the inequalities actually checked. A vanishing obstruction is always relative to the stored tolerance. Local contraction or Lyapunov Schmidt conclusions remain conditional on the supplied finite hypotheses. Matrix factorizations dominate cost and

use storage quadratic in the ambient dimensions. The provenance registry records `PSC-III-T31` as Orthogonal Hilbert obstruction projector. Its recorded status is `classical input`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_nonlinear_correction_elimination.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 41.

C.5A · `HilbertObstructionResult`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.HilbertObstructionResult`. Reflected implementation `komorebi_math.nonlinear.correction.HilbertObstructionResult`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-CORRECTION`.

Manifest signature (s).

```
(obstruction: 'np.ndarray', obstruction_norm: 'float', vanishes: 'bool', least_norm_correction: 'np.ndarray |
   ↪ None', correction_residual: 'float', certificate: 'Certificate') -> None
```

Definition. Finite orthogonal obstruction projection and least-norm correction data.

Contract (D, C, H, R, X). Correction contracts operate on finite matrices or derivative providers with explicit dimensions, tolerances, radii, right inverses, and Lipschitz bounds. Hilbert obstruction projection uses the declared linearized operator. Right inverse correction checks the supplied contraction inequalities. Lyapunov Schmidt reduction computes numerical kernel, complement, range, and obstruction coordinates at a specified base point.

Guarantees (E, P, B, K). Certificates report finite residuals, rank decisions, obstruction norms, and the inequalities actually checked. A vanishing obstruction is always relative to the stored tolerance. Local contraction or Lyapunov Schmidt conclusions remain conditional on the supplied finite hypotheses. Matrix factorizations dominate cost and use storage quadratic in the ambient dimensions.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_nonlinear_correction_elimination.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 41.

C.5A · `IntertwiningResult`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.IntertwiningResult`. Reflected implementation `komorebi_math.nonlinear.holonomy.IntertwiningResult`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-HOLONOMY`.

Manifest signature (s).

```
(intertwiners: 'Mapping[str, Any]', edge_residuals: 'Mapping[str, float]', cycle_residuals: 'Mapping[str, float]',
   ↪ certificate: 'Certificate') -> None
```

Definition. Vertex intertwiners propagated along a spanning tree.

Contract (D, C, H, R, X). A transport graph requires a finite vertex set and directed edges carrying compatible square matrices. Path transport requires composable endpoints. Gluing and intertwining checks require a chosen root and finite tolerance. Fundamental holonomy is computed from the selected spanning structure and recorded edges outside the tree.

Guarantees (E, P, B, K). The graph routines compute exact path products for exact input matrices or numerical products otherwise. Certificates compare finite cycle transports, root gauges, and intertwining residuals. The result

describes the supplied finite graph. It is not a continuum connection or topological classification without additional hypotheses.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_holonomy.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 40.

C.5A · JaxDerivativeProvider

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.JaxDerivativeProvider`. Reflected implementation `komorebi_math.nonlinear.autodiff.JaxDerivativeProvider`. The same runtime object is also available through `komorebi_math.JaxDerivativeProvider`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-AUTODIFF`.

Manifest signature (s).

```
(function: 'Callable[[Any], Any]', source_dimension: 'int', target_dimension: 'int', max_order: 'int' = 4, dtype:
    ↪ 'str | None' = None, name: 'str' = 'JAX derivative provider', exactness: 'Exactness' = <Exactness.NUMERICAL:
    ↪ 'numerical'>) -> None
```

Definition. Nested `jax.jvp` derivatives with lazy framework import.

Contract (D, C, H, R, X). Automatic differentiation adapters require a callable finite dimensional map, positive source and target dimensions, a declared maximum order, and a directional Jacobian vector product primitive supplied either directly or by an optional framework. Direction counts must equal the requested derivative order. Framework discovery is isolated in child processes so that a broken optional runtime cannot terminate the calling process. Principal public operations and properties are `available`, `value`, `derivative`, `to_dict`.

Guarantees (E, P, B, K). A successful provider returns the finite directional derivative computed by the selected JVP recursion and records numerical exactness unless the caller supplies a stronger justified classification. Framework availability means that an isolated probe imported the runtime and found the required JVP capability. It does not assert numerical agreement for every model or device. Work grows with derivative order and the number of nested JVP calls. The provenance registry records `PSC-III-N04J` as Lazy JAX derivative adapter. Its recorded status is `finite certificate`. The provenance record notes that device and precision behavior are controlled by the external JAX runtime.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_nonlinear_sparse_autodiff.py`, `tests/test_block10_sparse_autodiff.py`, and `tests/test_optional_backend_probe.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 37.

C.5A · KatoTransport

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.KatoTransport`. Reflected implementation `komorebi_math.nonlinear.transport.KatoTransport`. The same runtime object is also available through `komorebi_math.KatoTransport`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-SMOOTH-TRANSPORT`.

Manifest signature (`s`).

```
(system: 'MovingPacketSystem') -> 'None'
```

Definition. Numerical finite Kato transport for a moving packet system.

Contract (`D, C, H, R, X`). Smooth transport requires complete finite orthogonal packet families of fixed dimension together with declared packet derivatives. Direct rotation requires two compatible packet families whose overlap sum is numerically invertible. Kato transport requires finite start and end times, a positive step count, and a positive tolerance. Principal public operations and properties are `evolve`, `direct_rotation_product`, `refinement_certificate`.

Guarantees (`E, P, B, K`). Direct polar rotation returns a finite unitary intertwiner when its residual checks pass. Kato transport integrates the canonical finite generator with midpoint exponential steps and records unitarity, packet transport, and mesh refinement residuals. These certificates are numerical and path specific. They do not prove differentiability of an undeclared family or continuum convergence without external estimates. The provenance registry records `PSC-III-T58` as Finite smooth packet transport. Its recorded status is `proved`.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_nonlinear_transport.py` and `tests/test_quantum_transport_effective_spectral.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapters 40 and 44.

C.5A · `LiftStepResult`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.nonlinear.LiftStepResult`. Reflected implementation `komorebi_math.nonlinear.lifting.LiftStepResult`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-LIFTING`.

Manifest signature (`s`).

```
(prime: 'int', depth: 'int', point: 'tuple[int, ...]', residual: 'tuple[int, ...]', correction_space:
   ↪ 'ModularLinearSolution', children: 'tuple[tuple[int, ...], ...]', certificate: 'Certificate') -> None
```

Definition. One depth obstruction and all finite correction branches.

Contract (`D, C, H, R, X`). Linear and polynomial lifting require a prime, a nonnegative depth, compatible finite equations, and a current residue class. One step searches the declared corrections modulo the prime. Tree construction requires an explicit terminal depth and a finite node budget. Principal public operations and properties are `lifts`.

Guarantees (`E, P, B, K`). For the lifting operations in this family, every returned child satisfies the represented congruence at the next prime power. Linear solves and branch verification are exact in modular arithmetic. Polynomial lifting uses the declared Jacobian and finite Taylor congruence. Branch counts may grow exponentially with depth, so the tree builder enforces the supplied node limit.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_lifting.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 40.

C.5A · `LiftTreeNode`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.nonlinear.LiftTreeNode`. Reflected implementation `komorebi_math.nonlinear.lifting.LiftTreeNode`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-LIFTING`.

Manifest signature (s).

```
(value: 'tuple[int, ...]', depth: 'int', parent_index: 'int | None') -> None
```

Definition. One congruence class in a finite prime power lift tree.

Contract (D, C, H, R, X). Linear and polynomial lifting require a prime, a nonnegative depth, compatible finite equations, and a current residue class. One step searches the declared corrections modulo the prime. Tree construction requires an explicit terminal depth and a finite node budget.

Guarantees (E, P, B, K). For the lifting operations in this family, every returned child satisfies the represented congruence at the next prime power. Linear solves and branch verification are exact in modular arithmetic. Polynomial lifting uses the declared Jacobian and finite Taylor congruence. Branch counts may grow exponentially with depth, so the tree builder enforces the supplied node limit.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_lifting.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 40.

C.5A · `LinearLiftProblem`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.LinearLiftProblem`. Reflected implementation `komorebi_math.nonlinear.lifting.LinearLiftProblem`. The same runtime object is also available through `komorebi_math.LinearLiftProblem`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-LIFTING`.

Manifest signature (s).

```
(matrix: 'Any', rhs: 'Any', point: 'Any', *, prime: 'int', depth: 'int') -> 'None'
```

Definition. Lift a linear congruence one prime power at a time.

Contract (D, C, H, R, X). Linear and polynomial lifting require a prime, a nonnegative depth, compatible finite equations, and a current residue class. One step searches the declared corrections modulo the prime. Tree construction requires an explicit terminal depth and a finite node budget. Principal public operations and properties are `step`.

Guarantees (E, P, B, K). For the lifting operations in this family, every returned child satisfies the represented congruence at the next prime power. Linear solves and branch verification are exact in modular arithmetic. Polynomial lifting uses the declared Jacobian and finite Taylor congruence. Branch counts may grow exponentially with depth, so the tree builder enforces the supplied node limit. The provenance registry records `PSC-III-T28` as One step lifting criterion. Its recorded status is `proved`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_lifting.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 40.

C.5A · `LinearObstructionProblem`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.LinearObstructionProblem`. Reflected implementation `komorebi_math.nonlinear.obstruction.LinearObstructionProblem`. The same runtime object is also available through `komorebi_math.LinearObstructionProblem`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-OBSTRUCTION`.

Manifest signature (`s`).

```
(correction_map: 'Any', defect: 'Any', *, backend: 'str | LinearAlgebraBackend | None' = None, tolerance: 'float' =
   ↪ 1e-10) -> 'None'
```

Definition. Compute the obstruction to solving `D u = -defect`.

Contract (`D, C, H, R, X`). A linear obstruction problem requires a finite matrix and a target residual of matching dimension. The obstruction square additionally requires compatible maps whose two routes can be compared. Tolerances must be positive and finite when numerical rank decisions are used. Principal public operations and properties are `solve`.

Guarantees (`E, P, B, K`). The solver separates the residual into a least squares range component and a cokernel obstruction component. The correction is accepted only when the obstruction norm is within tolerance. Certificates record residuals and rank information for the supplied finite matrix. They do not establish exact vanishing outside the represented arithmetic and tolerance model. The provenance registry records `PSC-III-D13` as Linear correction datum and cokernel obstruction. Its recorded status is `definition`.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_nonlinear_obstruction.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 40.

C.5A · `LyapunovSchmidtDecomposition`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.nonlinear.LyapunovSchmidtDecomposition`. Reflected implementation `komorebi_math.nonlinear.correction.LyapunovSchmidtDecomposition`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-CORRECTION`.

Manifest signature (`s`).

```
(linearization: 'np.ndarray', kernel_basis: 'np.ndarray', complement_basis: 'np.ndarray', range_basis:
   ↪ 'np.ndarray', obstruction_basis: 'np.ndarray', rank: 'int', certificate: 'Certificate') -> None
```

Definition. Resolved kernel, range, complement, and obstruction spaces.

Contract (`D, C, H, R, X`). Correction contracts operate on finite matrices or derivative providers with explicit dimensions, tolerances, radii, right inverses, and Lipschitz bounds. Hilbert obstruction projection uses the declared linearized operator. Right inverse correction checks the supplied contraction inequalities. Lyapunov Schmidt reduction computes numerical kernel, complement, range, and obstruction coordinates at a specified base point.

Guarantees (`E, P, B, K`). Certificates report finite residuals, rank decisions, obstruction norms, and the inequalities actually checked. A vanishing obstruction is always relative to the stored tolerance. Local contraction or Lyapunov Schmidt conclusions remain conditional on the supplied finite hypotheses. Matrix factorizations dominate cost and use storage quadratic in the ambient dimensions.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_nonlinear_correction_elimination.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 41.

C.5A · LyapunovSchmidtPoint

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.LyapunovSchmidtPoint`. Reflected implementation `komorebi_math.nonlinear.correction.LyapunovSchmidtPoint`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-CORRECTION`.

Manifest signature (s).

```
(kernel_coordinates: 'np.ndarray', complement_coordinates: 'np.ndarray', point: 'np.ndarray', reduced_obstruction:
    ↪ 'np.ndarray', range_residual: 'float', iterations: 'int', certificate: 'Certificate') -> None
```

Definition. One computed point on a finite Lyapunov–Schmidt branch.

Contract (D, C, H, R, X). Correction contracts operate on finite matrices or derivative providers with explicit dimensions, tolerances, radii, right inverses, and Lipschitz bounds. Hilbert obstruction projection uses the declared linearized operator. Right inverse correction checks the supplied contraction inequalities. Lyapunov Schmidt reduction computes numerical kernel, complement, range, and obstruction coordinates at a specified base point.

Guarantees (E, P, B, K). Certificates report finite residuals, rank decisions, obstruction norms, and the inequalities actually checked. A vanishing obstruction is always relative to the stored tolerance. Local contraction or Lyapunov Schmidt conclusions remain conditional on the supplied finite hypotheses. Matrix factorizations dominate cost and use storage quadratic in the ambient dimensions.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_nonlinear_correction_elimination.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 41.

C.5A · LyapunovSchmidtReduction

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.LyapunovSchmidtReduction`. Reflected implementation `komorebi_math.nonlinear.correction.LyapunovSchmidtReduction`. The same runtime object is also available through `komorebi_math.LyapunovSchmidtReduction`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-CORRECTION`.

Manifest signature (s).

```
(provider: 'DerivativeProvider', base_point: 'Any', *, tolerance: 'float' = 1e-10) -> 'None'
```

Definition. Numerical SVD realization of local Lyapunov–Schmidt reduction.

Contract (D, C, H, R, X). Correction contracts operate on finite matrices or derivative providers with explicit dimensions, tolerances, radii, right inverses, and Lipschitz bounds. Hilbert obstruction projection uses the declared linearized operator. Right inverse correction checks the supplied contraction inequalities. Lyapunov Schmidt reduction computes numerical kernel, complement, range, and obstruction coordinates at a specified base point. Principal public operations and properties are `solve_range`, `base_derivative_certificate`, `reduced_second_derivative`.

Guarantees (E, P, B, K). Certificates report finite residuals, rank decisions, obstruction norms, and the inequalities actually checked. A vanishing obstruction is always relative to the stored tolerance. Local contraction or Lyapunov Schmidt conclusions remain conditional on the supplied finite hypotheses. Matrix factorizations dominate cost and use storage quadratic in the ambient dimensions. The provenance registry records `PSC-III-T33` as Finite local Lyapunov–Schmidt reduction. Its recorded status is `classical input`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_nonlinear_correction_elimination.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 41.

C.5A · MaurerCartanResult

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.MaurerCartanResult`. Reflected implementation `komorebi_math.nonlinear.graded.MaurerCartanResult`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-GRADED`.

Manifest signature (s).

```
(equations: 'dict[str, Any]', reconstructed: 'Any', direct: 'Any', certificate: 'Certificate') -> None
```

Definition. Finite packet Maurer–Cartan equations and reconstruction certificate.

Contract (D, C, H, R, X). Graded operations require a finite abelian grading group, finite component records, and multilinear operations whose declared output grade is the group sum of the input grades. Maurer Cartan routines evaluate only the supplied finite arity range. Divided square evaluation requires the supplied differential, bracket, and divided square maps to have compatible component shapes.

Guarantees (E, P, B, K). Grade bookkeeping and finite group arithmetic are exact. Numerical component values inherit the arithmetic of the supplied arrays and operations. Certificates compare the gradewise reconstruction with the direct finite expression and record any failed compatibility law. No unspecified higher operation or infinite Maurer Cartan series is inferred.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_nonlinear_graded.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 39.

C.5A · ModularLinearSolution

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.ModularLinearSolution`. Reflected implementation `komorebi_math.nonlinear.lifting.ModularLinearSolution`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-LIFTING`.

Manifest signature (s).

```
(prime: 'int', solvable: 'bool', particular: 'tuple[int, ...] | None', kernel_basis: 'tuple[tuple[int, ...],
   ↪ ...]', pivot_columns: 'tuple[int, ...]', obstruction_rows: 'tuple[int, ...]') -> None
```

Definition. The affine solution space of a linear system over a prime field.

Contract (D, C, H, R, X). Linear and polynomial lifting require a prime, a nonnegative depth, compatible finite equations, and a current residue class. One step searches the declared corrections modulo the prime. Tree construction requires an explicit terminal depth and a finite node budget. Principal public operations and properties are `nullity`, `solution_count`, `enumerate`.

Guarantees (E, P, B, K). For the lifting operations in this family, every returned child satisfies the represented congruence at the next prime power. Linear solves and branch verification are exact in modular arithmetic. Polynomial lifting uses the declared Jacobian and finite Taylor congruence. Branch counts may grow exponentially with depth, so the tree builder enforces the supplied node limit.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_lifting.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 40.

C.5A · MovingPacketSystem

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.MovingPacketSystem`. Reflected implementation `komorebi_math.nonlinear.transport.MovingPacketSystem`. The same runtime object is also available through `komorebi_math.MovingPacketSystem`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-SMOOTH-TRANSPORT`.

Manifest signature (s).

```
(projectors: 'Callable[[float], Iterable[Any]]', derivatives: 'Callable[[float], Iterable[Any]]', *, tolerance:
    ↪  'float' = 1e-09, reference_time: 'float' = 0.0) -> 'None'
```

Definition. A differentiable finite family of complete orthogonal projectors.

Contract (D, C, H, R, X). Smooth transport requires complete finite orthogonal packet families of fixed dimension together with declared packet derivatives. Direct rotation requires two compatible packet families whose overlap sum is numerically invertible. Kato transport requires finite start and end times, a positive step count, and a positive tolerance. Principal public operations and properties are `packets`, `packet_derivatives`, `derivative_certificate`, `canonical_generator`, `generator_certificate`.

Guarantees (E, P, B, K). Direct polar rotation returns a finite unitary intertwiner when its residual checks pass. Kato transport integrates the canonical finite generator with midpoint exponential steps and records unitarity, packet transport, and mesh refinement residuals. These certificates are numerical and path specific. They do not prove differentiability of an undeclared family or continuum convergence without external estimates. The provenance registry records `PSC-III-D56` as Smooth finite packet system. Its recorded status is `definition`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_nonlinear_transport.py` and `tests/test_quantum_transport_effective_spectral.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapters 40 and 44.

C.5A · NonlinearPacketMap

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.NonlinearPacketMap`. Reflected implementation `komorebi_math.nonlinear.coordinates.NonlinearPacketMap`. The same runtime object is also available through `komorebi_math.NonlinearPacketMap`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-NONLINEAR-COORDINATES`.

Manifest signature (s).

```
(function: 'Callable[[Any], Any]', source: 'PacketSystem', target: 'PacketSystem', name: 'str' = 'nonlinear packet
    ↪  map') -> None
```

Definition. A nonlinear map evaluated through exact packet analysis and synthesis.

Contract (D, C, H, R, X). A nonlinear packet map requires a callable finite map and compatible finite source and target packet systems. Inputs must have the source ambient dimension. Packet evaluation projects the input into declared source channels, evaluates the map, and resolves the output through the target packet family. Principal public operations and properties are `apply`, `certificate`, `iterate`, `fixed_point_residual`.

Guarantees (E, P, B, K). For a complete target packet system, the recorded packet outputs reconstruct the direct finite map value within the certificate tolerance. The certificate concerns one supplied input and one implemented packet decomposition. It does not imply global linearity, smoothness, or invertibility. Work is dominated by packet projection and one map evaluation. The provenance registry records `PSC-III-D01` as Nonlinear packet coordinate map. Its recorded status is `definition`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_nonlinear_coordinates_defects.py`. The independent verifier for nonlinear calculus

foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 40.

C.5A · NonlinearRefinementDefect

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.NonlinearRefinementDefect`. Reflected implementation `komorebi_math.nonlinear.defects.NonlinearRefinementDefect`. The same runtime object is also available through `komorebi_math.NonlinearRefinementDefect`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-REFINEMENT-DEFECTS`.

Manifest signature (s).

```
(fine_map: 'Callable[[Any], Any]', coarse_map: 'Callable[[Any], Any]', coarsen_source: 'Any', coarsen_target:
   ↪  'Any') -> None
```

Definition. Pointwise defect `C_Y F_f(x) - F_c(C_X x)`.

Contract (D, C, H, R, X). A nonlinear refinement defect requires fine and coarse maps together with finite source and target coarsening operators of compatible shape. Iterated bounds additionally require a nonnegative iteration count and a coarse Lipschitz bound supplied by the caller. Principal public operations and properties are `evaluate`, `norm`, `composition_identity`, `iterated_orbit_bound`, `fixed_point_bound`.

Guarantees (E, P, B, K). The direct defect is the finite discrepancy between coarsening after the fine map and applying the coarse map after source coarsening. Orbit estimates are conditional on the supplied Lipschitz constant and measured finite defects. They do not independently prove a global contraction property. The provenance registry records `PSC-III-D10` as Nonlinear refinement defect. Its recorded status is `definition`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_nonlinear_coordinates_defects.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 40.

C.5A · NonlinearSchurElimination

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.NonlinearSchurElimination`. Reflected implementation `komorebi_math.nonlinear.elimination.NonlinearSchurElimination`. The same runtime object is also available through `komorebi_math.NonlinearSchurElimination`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-SCHUR-ELIMINATION`.

Manifest signature (s).

```
(retained_provider: 'DerivativeProvider', hidden_provider: 'DerivativeProvider', retained_dimension: 'int',
   ↪  hidden_dimension: 'int', base_retained: 'Any', base_hidden: 'Any', *, tolerance: 'float' = 1e-10) -> 'None'
```

Definition. First and second derivatives after eliminating hidden variables.

Contract (D, C, H, R, X). Nonlinear Schur elimination requires retained and hidden derivative providers with a declared split of visible and hidden coordinates. The hidden derivative at the base point must be square and numerically invertible. Hidden solves require an initial point, a finite iteration limit, and a positive tolerance. Principal public operations and properties are `first_derivatives`, `second_derivative`, `solve_hidden`, `reduced_value`.

Guarantees (E, P, B, K). The first derivative contract implements the finite implicit derivative and reduced Schur derivative. The second derivative contract applies the declared provider derivatives through order two. Exact symbolic inputs may retain exact matrices. Numerical inputs receive residual certificates tied to the selected solve and base point. These are local finite statements, not global existence claims. The provenance registry records `PSC-III-T41` as Finite nonlinear Schur elimination. Its recorded status is `proved`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_nonlinear_correction_elimination.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 41.

C.5A · `ObstructionResult`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.ObstructionResult`. Reflected implementation `komorebi_math.nonlinear.obstruction.ObstructionResult`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-OBSTRUCTION`.

Manifest signature (s).

```
(vanishes: 'bool', obstruction_coordinates: 'tuple[Any, ...]', cokernel_dimension: 'int', correction: 'Any |
    ↪ None', kernel_basis: 'tuple[Any, ...]', residual: 'float', certificate: 'Certificate') -> None
```

Definition. Computed obstruction coordinates and correction space.

Contract (D, C, H, R, X). A linear obstruction problem requires a finite matrix and a target residual of matching dimension. The obstruction square additionally requires compatible maps whose two routes can be compared. Tolerances must be positive and finite when numerical rank decisions are used. Principal public operations and properties are `require_correction`.

Guarantees (E, P, B, K). The solver separates the residual into a least squares range component and a cokernel obstruction component. The correction is accepted only when the obstruction norm is within tolerance. Certificates record residuals and rank information for the supplied finite matrix. They do not establish exact vanishing outside the represented arithmetic and tolerance model.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_nonlinear_obstruction.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 40.

C.5A · `ObstructionSquare`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.ObstructionSquare`. Reflected implementation `komorebi_math.nonlinear.obstruction.ObstructionSquare`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-OBSTRUCTION`.

Manifest signature (s).

```
(source_correction: 'Any', target_correction: 'Any', source_map: 'Any', target_map: 'Any', backend: 'str |
    ↪ LinearAlgebraBackend | None' = None, tolerance: 'float' = 1e-10) -> None
```

Definition. A commuting square inducing a map between cokernels.

Contract (D, C, H, R, X). A linear obstruction problem requires a finite matrix and a target residual of matching dimension. The obstruction square additionally requires compatible maps whose two routes can be compared. Tolerances must be positive and finite when numerical rank decisions are used. Principal public operations and properties are `certificate`, `transport_defect`.

Guarantees (E, P, B, K). The solver separates the residual into a least squares range component and a cokernel obstruction component. The correction is accepted only when the obstruction norm is within tolerance. Certificates record residuals and rank information for the supplied finite matrix. They do not establish exact vanishing outside the represented arithmetic and tolerance model.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_nonlinear_obstruction.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 40.

C.5A · PacketDerivativeResult

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.PacketDerivativeResult`. Reflected implementation `komorebi_math.nonlinear.transport.PacketDerivativeResult`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-SMOOTH-TRANSPORT`.

Manifest signature (s).

```
(time: 'float', step: 'float', maximum_derivative_residual: 'float', tangent_residual: 'float',
   ↪ partition_derivative_residual: 'float', certificate: 'Certificate') -> None
```

Definition. Finite-difference validation of declared moving-packet derivatives.

Contract (D, C, H, R, X). Smooth transport requires complete finite orthogonal packet families of fixed dimension together with declared packet derivatives. Direct rotation requires two compatible packet families whose overlap sum is numerically invertible. Kato transport requires finite start and end times, a positive step count, and a positive tolerance.

Guarantees (E, P, B, K). Direct polar rotation returns a finite unitary intertwiner when its residual checks pass. Kato transport integrates the canonical finite generator with midpoint exponential steps and records unitarity, packet transport, and mesh refinement residuals. These certificates are numerical and path specific. They do not prove differentiability of an undeclared family or continuum convergence without external estimates.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_nonlinear_transport.py` and `tests/test_quantum_transport_effective_spectral.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapters 40 and 44.

C.5A · Partition

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.Partition`. Reflected implementation `builtins.tuple`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The public export name maps to implementation object `tuple`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `data`. Profile `C5-COMPOSITION`.

Manifest signature (s).

```
(*args, **kwargs)
```

Definition. Immutable nested tuple type used to encode a set partition of labeled direction positions.

Contract (D, C, H, R, X). Composition requires compatible inner and outer dimensions and derivative support through the requested order. The Faà di Bruno routines enumerate set partitions of the direction labels. Packet composition also requires compatible source, intermediate, and target packet systems. Partition utilities accept nonnegative finite orders and return canonical immutable records.

Guarantees (E, P, B, K). For the composition operations in this family, the computed derivative value equals the implemented finite Faà di Bruno sum for the supplied providers and directions. Exactness follows the participating providers. The composition result records retained terms, partition counts, and a certificate comparing the com-

posed derivative with the direct implementation when that comparison is available. Enumeration grows according to Bell and Stirling counts, so callers must keep the derivative order bounded.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_nonlinear_jets_composition.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 38.

C.5A · PolynomialDerivativeProvider

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.PolynomialDerivativeProvider`. Reflected implementation `komorebi_math.nonlinear.derivatives.PolynomialDerivativeProvider`. The same runtime object is also available through `komorebi_math.PolynomialDerivativeProvider`, `komorebi_math.beta.PolynomialDerivativeProvider`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-DERIVATIVES`.

Manifest signature (s).

```
(tensors: 'Mapping[int, Any]', *, source_dimension: 'int | None' = None, target_dimension: 'int | None' = None,
    ↪ exactness: 'Exactness' = <Exactness.NUMERICAL: 'numerical'>, tolerance: 'float' = 1e-10, name: 'str' =
    ↪ 'polynomial derivative provider') -> 'None'
```

Definition. A finite polynomial map represented by symmetric coefficient tensors.

Contract (D, C, H, R, X). Derivative providers expose one common finite dimensional interface for map evaluation and directional derivatives. A call must use an order supported by the provider and exactly that many direction vectors of the declared source dimension. Callable and polynomial providers trust explicitly supplied derivative data. The finite difference provider additionally requires a positive finite step. The SymPy provider requires the optional symbolic dependency. Principal public operations and properties are `degree`, `supports_order`, `value`, `derivative`, `symmetry_certificate`.

Guarantees (E, P, B, K). Polynomial and symbolic providers preserve exact objects where their inputs permit exact arithmetic. Callable providers inherit the exactness declared by the caller. Finite differences return numerical evidence only. Every provider checks dimensions and finite values before evaluation. Tensor contraction cost grows with derivative order and ambient dimension. The provenance registry records `PSC-III-D04` as Symmetric polynomial derivative tensors. Its recorded status is `definition`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_nonlinear_jets_composition.py` and `tests/test_nonlinear_sparse_autodiff.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapters 37 and 38.

C.5A · PolynomialLiftProblem

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.PolynomialLiftProblem`. Reflected implementation `komorebi_math.nonlinear.lifting.PolynomialLiftProblem`. The same runtime object is also available through `komorebi_math.PolynomialLiftProblem`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-LIFTING`.

Manifest signature (s).

```
(function: 'Callable[[tuple[int, ...]], Iterable[int]]', jacobian: 'Callable[[tuple[int, ...]], Any]', point:
   ↪ 'Any', *, prime: 'int', depth: 'int') -> 'None'
```

Definition. Lift a polynomial system by its Jacobian over the residual field.

Contract (D, C, H, R, X). Linear and polynomial lifting require a prime, a nonnegative depth, compatible finite equations, and a current residue class. One step searches the declared corrections modulo the prime. Tree construction requires an explicit terminal depth and a finite node budget. Principal public operations and properties are `step`.

Guarantees (E, P, B, K). For the lifting operations in this family, every returned child satisfies the represented congruence at the next prime power. Linear solves and branch verification are exact in modular arithmetic. Polynomial lifting uses the declared Jacobian and finite Taylor congruence. Branch counts may grow exponentially with depth, so the tree builder enforces the supplied node limit.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_lifting.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 40.

C.5A · `ProjectiveLimitRecord`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.ProjectiveLimitRecord`. Reflected implementation `komorebi_math.nonlinear.filtered.ProjectiveLimitRecord`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-FILTERED`.

Manifest signature (s).

```
(verified_prefix_depth: 'int', separated: 'bool', complete: 'bool', continuous: 'bool',
   ↪ compatible_infinite_branch: 'bool', conclusion: 'str') -> None
```

Definition. Declared hypotheses and conclusion for a conditional projective-limit step.

Contract (D, C, H, R, X). Filtered contracts require finite dimensional vector spaces over an explicit prime field and decreasing filtration bases with compatible ambient dimensions. Filtered correction is solved one finite depth at a time. A filtered cochain complex requires explicit differentials that square to zero and preserve the declared filtration. Projective limit completion also requires external completeness hypotheses supplied by the caller.

Guarantees (E, P, B, K). Associated graded coordinates, correction cosets, finite branches, and spectral sequence pages are exact over the represented finite field. The terminal finite filtration permits exact page dimension and stabilization checks. A projective limit record only stores the declared infinite hypotheses and finite compatibility evidence. It does not prove completeness or convergence beyond those assumptions.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_nonlinear_filtered.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 42.

C.5A · `RightInverseCorrectionProblem`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.RightInverseCorrectionProblem`. Reflected implementation `komorebi_math.nonlinear.correction.RightInverseCorrectionProblem`. The same runtime object is also available through `komorebi_math.RightInverseCorrectionProblem`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-CORRECTION`.

Manifest signature (s).

```
(linear_map: 'Any', right_inverse: 'Any', nonlinear: 'Callable[[np.ndarray], Any]', target: 'Any', *, radius:
   ↪  'float', lipschitz_bound: 'float', tolerance: 'float' = 1e-10) -> 'None'
```

Definition. Solve `D x + N(x) = y` by the theorem's fixed-point map.

Contract (D, C, H, R, X). Correction contracts operate on finite matrices or derivative providers with explicit dimensions, tolerances, radii, right inverses, and Lipschitz bounds. Hilbert obstruction projection uses the declared linearized operator. Right inverse correction checks the supplied contraction inequalities. Lyapunov Schmidt reduction computes numerical kernel, complement, range, and obstruction coordinates at a specified base point. Principal public operations and properties are `hypothesis_certificate`, `solve`, `target_stability_bound`.

Guarantees (E, P, B, K). Certificates report finite residuals, rank decisions, obstruction norms, and the inequalities actually checked. A vanishing obstruction is always relative to the stored tolerance. Local contraction or Lyapunov Schmidt conclusions remain conditional on the supplied finite hypotheses. Matrix factorizations dominate cost and use storage quadratic in the ambient dimensions. The provenance registry records `PSC-III-T32` as Right-inverse nonlinear correction. Its recorded status is `proved`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_nonlinear_correction_elimination.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 41.

C.5A · `RightInverseCorrectionResult`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.RightInverseCorrectionResult`. Reflected implementation `komorebi_math.nonlinear.correction.RightInverseCorrectionResult`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-CORRECTION`.

Manifest signature (s).

```
(solution: 'np.ndarray', iterations: 'int', contraction_factor: 'float', equation_residual: 'float',
   ↪  fixed_point_residual: 'float', certificate: 'Certificate') -> None
```

Definition. Result of a finite right-inverse contraction correction.

Contract (D, C, H, R, X). Correction contracts operate on finite matrices or derivative providers with explicit dimensions, tolerances, radii, right inverses, and Lipschitz bounds. Hilbert obstruction projection uses the declared linearized operator. Right inverse correction checks the supplied contraction inequalities. Lyapunov Schmidt reduction computes numerical kernel, complement, range, and obstruction coordinates at a specified base point.

Guarantees (E, P, B, K). Certificates report finite residuals, rank decisions, obstruction norms, and the inequalities actually checked. A vanishing obstruction is always relative to the stored tolerance. Local contraction or Lyapunov Schmidt conclusions remain conditional on the supplied finite hypotheses. Matrix factorizations dominate cost and use storage quadratic in the ambient dimensions.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_nonlinear_correction_elimination.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 41.

C.5A · SchurDerivativeResult

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.SchurDerivativeResult`. Reflected implementation `komorebi_math.nonlinear.elimination.SchurDerivativeResult`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-SCHUR-ELIMINATION`.

Manifest signature (s).

```
(hidden_jacobian: 'Any', hidden_inverse: 'Any', implicit_derivative: 'Any', reduced_derivative: 'Any',
   ↪ certificate: 'Certificate') -> None
```

Definition. First-order implicit and reduced derivatives for nonlinear Schur elimination.

Contract (D, C, H, R, X). Nonlinear Schur elimination requires retained and hidden derivative providers with a declared split of visible and hidden coordinates. The hidden derivative at the base point must be square and numerically invertible. Hidden solves require an initial point, a finite iteration limit, and a positive tolerance.

Guarantees (E, P, B, K). The first derivative contract implements the finite implicit derivative and reduced Schur derivative. The second derivative contract applies the declared provider derivatives through order two. Exact symbolic inputs may retain exact matrices. Numerical inputs receive residual certificates tied to the selected solve and base point. These are local finite statements, not global existence claims.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_nonlinear_correction_elimination.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 41.

C.5A · SchurSecondDerivativeResult

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.SchurSecondDerivativeResult`. Reflected implementation `komorebi_math.nonlinear.elimination.SchurSecondDerivativeResult`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-SCHUR-ELIMINATION`.

Manifest signature (s).

```
(lifted_left: 'Any', lifted_right: 'Any', implicit_second_derivative: 'Any', reduced_second_derivative: 'Any',
   ↪ certificate: 'Certificate') -> None
```

Definition. Second-order implicit and reduced derivatives for nonlinear Schur elimination.

Contract (D, C, H, R, X). Nonlinear Schur elimination requires retained and hidden derivative providers with a declared split of visible and hidden coordinates. The hidden derivative at the base point must be square and numerically invertible. Hidden solves require an initial point, a finite iteration limit, and a positive tolerance.

Guarantees (E, P, B, K). The first derivative contract implements the finite implicit derivative and reduced Schur derivative. The second derivative contract applies the declared provider derivatives through order two. Exact symbolic inputs may retain exact matrices. Numerical inputs receive residual certificates tied to the selected solve and base point. These are local finite statements, not global existence claims.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_nonlinear_correction_elimination.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 41.

C.5A · SpectralSequenceEntry

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.SpectralSequenceEntry`. Reflected implementation `komorebi_math.nonlinear.filtered.SpectralSequenceEntry`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-FILTERED`.

Manifest signature (s).

```
(page: 'int', filtration_degree: 'int', complementary_degree: 'int', total_degree: 'int', dimension: 'int',
   ↪ outgoing_rank: 'int', incoming_rank: 'int') -> None
```

Definition. One bidegree entry on a finite filtered spectral-sequence page.

Contract (D, C, H, R, X). Filtered contracts require finite dimensional vector spaces over an explicit prime field and decreasing filtration bases with compatible ambient dimensions. Filtered correction is solved one finite depth at a time. A filtered cochain complex requires explicit differentials that square to zero and preserve the declared filtration. Projective limit completion also requires external completeness hypotheses supplied by the caller.

Guarantees (E, P, B, K). Associated graded coordinates, correction cosets, finite branches, and spectral sequence pages are exact over the represented finite field. The terminal finite filtration permits exact page dimension and stabilization checks. A projective limit record only stores the declared infinite hypotheses and finite compatibility evidence. It does not prove completeness or convergence beyond those assumptions.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_nonlinear_filtered.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 42.

C.5A · SpectralSequencePage

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.SpectralSequencePage`. Reflected implementation `komorebi_math.nonlinear.filtered.SpectralSequencePage`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-FILTERED`.

Manifest signature (s).

```
(page: 'int', entries: 'tuple[SpectralSequenceEntry, ...]', total_dimension: 'int', total_differential_rank:
   ↪ 'int', certificate: 'Certificate') -> None
```

Definition. One exact finite page with entry and differential-rank data.

Contract (D, C, H, R, X). Filtered contracts require finite dimensional vector spaces over an explicit prime field and decreasing filtration bases with compatible ambient dimensions. Filtered correction is solved one finite depth at a time. A filtered cochain complex requires explicit differentials that square to zero and preserve the declared filtration. Projective limit completion also requires external completeness hypotheses supplied by the caller. Principal public operations and properties are `entry`.

Guarantees (E, P, B, K). Associated graded coordinates, correction cosets, finite branches, and spectral sequence pages are exact over the represented finite field. The terminal finite filtration permits exact page dimension and stabilization checks. A projective limit record only stores the declared infinite hypotheses and finite compatibility evidence. It does not prove completeness or convergence beyond those assumptions.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_nonlinear_filtered.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 42.

C.5A · SympyDerivativeProvider

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.SympyDerivativeProvider`. Reflected implementation `komorebi_math.nonlinear.derivatives.SympyDerivativeProvider`. The same runtime object is also available through `komorebi_math.SympyDerivativeProvider`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-DERIVATIVES`.

Manifest signature (s).

```
(expressions: 'Sequence[Any]', variables: 'Sequence[Any]', *, name: 'str' = 'SymPy derivative provider') -> 'None'
```

Definition. Exact directional derivatives of a finite SymPy expression vector.

Contract (D, C, H, R, X). Derivative providers expose one common finite dimensional interface for map evaluation and directional derivatives. A call must use an order supported by the provider and exactly that many direction vectors of the declared source dimension. Callable and polynomial providers trust explicitly supplied derivative data. The finite difference provider additionally requires a positive finite step. The SymPy provider requires the optional symbolic dependency. Principal public operations and properties are `supports_order`, `value`, `derivative`.

Guarantees (E, P, B, K). Polynomial and symbolic providers preserve exact objects where their inputs permit exact arithmetic. Callable providers inherit the exactness declared by the caller. Finite differences return numerical evidence only. Every provider checks dimensions and finite values before evaluation. Tensor contraction cost grows with derivative order and ambient dimension. The provenance registry records `PSC-III-E05` as Exact symbolic directional derivatives. Its recorded status is `finite certificate`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_nonlinear_jets_composition.py` and `tests/test_nonlinear_sparse_autodiff.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapters 37 and 38.

C.5A · TorchDerivativeProvider

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.TorchDerivativeProvider`. Reflected implementation `komorebi_math.nonlinear.autodiff.TorchDerivativeProvider`. The same runtime object is also available through `komorebi_math.TorchDerivativeProvider`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-AUTODIFF`.

Manifest signature (s).

```
(function: 'Callable[[Any], Any]', source_dimension: 'int', target_dimension: 'int', max_order: 'int' = 4, dtype:
    ↪ 'str' = 'float64', device: 'str | None' = None, name: 'str' = 'PyTorch derivative provider', exactness:
    ↪ 'Exactness' = <Exactness.NUMERICAL: 'numerical'>) -> None
```

Definition. Nested `torch.func.jvp` derivatives with lazy framework import.

Contract (D, C, H, R, X). Automatic differentiation adapters require a callable finite dimensional map, positive source and target dimensions, a declared maximum order, and a directional Jacobian vector product primitive supplied either directly or by an optional framework. Direction counts must equal the requested derivative order. Framework discovery is isolated in child processes so that a broken optional runtime cannot terminate the calling process. Principal public operations and properties are `available`, `value`, `derivative`, `to_dict`.

Guarantees (E, P, B, K). A successful provider returns the finite directional derivative computed by the selected JVP recursion and records numerical exactness unless the caller supplies a stronger justified classification. Framework availability means that an isolated probe imported the runtime and found the required JVP capability. It does not assert numerical agreement for every model or device. Work grows with derivative order and the number of nested JVP calls. The provenance registry records `PSC-III-N04T` as Lazy PyTorch derivative adapter. Its recorded status is

`finite certificate`. The provenance record notes that device and precision behavior are controlled by the external PyTorch runtime.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_nonlinear_sparse_autodiff.py`, `tests/test_block10_sparse_autodiff.py`, and `tests/test_optional_backend_probe.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 37.

C.5A · `TransportEdge`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.TransportEdge`. Reflected implementation `komorebi_math.nonlinear.holonomy.TransportEdge`. The same runtime object is also available through `komorebi_math.TransportEdge`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-HOLONOMY`.

Manifest signature (s).

```
(source: 'str', target: 'str', matrix: 'Any', name: 'str' = '') -> None
```

Definition. An invertible transport from `source` to `target`.

Contract (D, C, H, R, X). A transport graph requires a finite vertex set and directed edges carrying compatible square matrices. Path transport requires composable endpoints. Gluing and intertwining checks require a chosen root and finite tolerance. Fundamental holonomy is computed from the selected spanning structure and recorded edges outside the tree. Principal public operations and properties are `reversed_name`.

Guarantees (E, P, B, K). The graph routines compute exact path products for exact input matrices or numerical products otherwise. Certificates compare finite cycle transports, root gauges, and intertwining residuals. The result describes the supplied finite graph. It is not a continuum connection or topological classification without additional hypotheses.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_holonomy.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 40.

C.5A · `TransportGraph`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.TransportGraph`. Reflected implementation `komorebi_math.nonlinear.holonomy.TransportGraph`. The same runtime object is also available through `komorebi_math.TransportGraph`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-HOLONOMY`.

Manifest signature (s).

```
(vertices: 'Iterable[str]', edges: 'Iterable[TransportEdge]', *, backend: 'str | LinearAlgebraBackend | None' =
    ↪ None, tolerance: 'float' = 1e-10) -> 'None'
```

Definition. A connected finite graph with invertible matrix transports.

Contract (D, C, H, R, X). A transport graph requires a finite vertex set and directed edges carrying compatible square matrices. Path transport requires composable endpoints. Gluing and intertwining checks require a chosen root and finite tolerance. Fundamental holonomy is computed from the selected spanning structure and recorded edges outside the tree. Principal public operations and properties are `edge_label`, `oriented_matrix`, `spanning_tree`, `tree_transports`, `tree_gauge`, `gauge_transform`, `fundamental_holonomies`, `gluing_certificate`, `intertwine`.

Guarantees (E, P, B, K). The graph routines compute exact path products for exact input matrices or numerical products otherwise. Certificates compare finite cycle transports, root gauges, and intertwining residuals. The result describes the supplied finite graph. It is not a continuum connection or topological classification without additional

hypotheses. The provenance registry records `PSC-III-D38` as Finite matrix transport system. Its recorded status is `definition`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_holonomy.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 40.

C.5A · `TransportRefinementResult`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.TransportRefinementResult`. Reflected implementation `komorebi_math.nonlinear.transport.TransportRefinementResult`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-SMOOTH-TRANSPORT`.

Manifest signature (s).

```
(coarse_steps: 'int', fine_steps: 'int', coarse: 'np.ndarray', fine: 'np.ndarray', difference: 'float',
   ↪ endpoint_intertwining: 'float', certificate: 'Certificate') -> None
```

Definition. A posteriori comparison of two midpoint transport resolutions.

Contract (D, C, H, R, X). Smooth transport requires complete finite orthogonal packet families of fixed dimension together with declared packet derivatives. Direct rotation requires two compatible packet families whose overlap sum is numerically invertible. Kato transport requires finite start and end times, a positive step count, and a positive tolerance.

Guarantees (E, P, B, K). Direct polar rotation returns a finite unitary intertwiner when its residual checks pass. Kato transport integrates the canonical finite generator with midpoint exponential steps and records unitarity, packet transport, and mesh refinement residuals. These certificates are numerical and path specific. They do not prove differentiability of an undeclared family or continuum convergence without external estimates.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_nonlinear_transport.py` and `tests/test_quantum_transport_effective_spectral.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapters 40 and 44.

C.5A · `autodiff_provider`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.autodiff_provider`. Reflected implementation `komorebi_math.nonlinear.autodiff.autodiff_provider`. The same runtime object is also available through `komorebi_math.autodiff_provider`. Within `komorebi_math.nonlinear`, `automatic_derivative_provider` resolves to the same runtime object. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C5-AUTODIFF`.

Manifest signature (s).

```
(framework: 'str', function: 'Callable[[Any], Any]', source_dimension: 'int', target_dimension: 'int', *,
   ↪ max_order: 'int' = 4, **options: 'Any') -> 'JaxDerivativeProvider | TorchDerivativeProvider'
```

Definition. Construct one supported external automatic-differentiation adapter.

Contract (D, C, H, R, X). Automatic differentiation adapters require a callable finite dimensional map, positive source and target dimensions, a declared maximum order, and a directional Jacobian vector product primitive supplied either directly or by an optional framework. Direction counts must equal the requested derivative order. Framework discovery is isolated in child processes so that a broken optional runtime cannot terminate the calling process.

Guarantees (E, P, B, K). A successful provider returns the finite directional derivative computed by the selected JVP recursion and records numerical exactness unless the caller supplies a stronger justified classification. Framework availability means that an isolated probe imported the runtime and found the required JVP capability. It does not assert numerical agreement for every model or device. Work grows with derivative order and the number of nested JVP calls.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_nonlinear_sparse_autodiff.py`, `tests/test_block10_sparse_autodiff.py`, and `tests/test_optional_backend_probe.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 37.

C.5A · `automatic_derivative_provider`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.automatic_derivative_provider`. Reflected implementation `komorebi_math.nonlinear.autodiff.autodiff_provider`. The same runtime object is also available through `komorebi_math.automatic_derivative_provider`. Within `komorebi_math.nonlinear`, `autodiff_provider` resolves to the same runtime object. The public export name maps to implementation object `autodiff_provider`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C5-AUTODIFF`.

Manifest signature (s).

```
(framework: 'str', function: 'Callable[[Any], Any]', source_dimension: 'int', target_dimension: 'int', *,
   ↪ max_order: 'int' = 4, **options: 'Any') -> 'JaxDerivativeProvider | TorchDerivativeProvider'
```

Definition. Construct one supported external automatic-differentiation adapter.

Contract (D, C, H, R, X). Automatic differentiation adapters require a callable finite dimensional map, positive source and target dimensions, a declared maximum order, and a directional Jacobian vector product primitive supplied either directly or by an optional framework. Direction counts must equal the requested derivative order. Framework discovery is isolated in child processes so that a broken optional runtime cannot terminate the calling process.

Guarantees (E, P, B, K). A successful provider returns the finite directional derivative computed by the selected JVP recursion and records numerical exactness unless the caller supplies a stronger justified classification. Framework availability means that an isolated probe imported the runtime and found the required JVP capability. It does not assert numerical agreement for every model or device. Work grows with derivative order and the number of nested JVP calls.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_nonlinear_sparse_autodiff.py`, `tests/test_block10_sparse_autodiff.py`, and `tests/test_optional_backend_probe.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 37.

C.5A · `available_ad_frameworks`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.available_ad_frameworks`. Reflected implementation `komorebi_math.nonlinear.autodiff.available_ad_frameworks`. The same runtime object is also available through `komorebi_math.available_ad_frameworks`. Within `komorebi_math.nonlinear`, `available_autodiff_backends` resolves to the same runtime object. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C5-AUTODIFF`.

Manifest signature (`s`).

```
() -> 'dict[str, bool]'
```

Definition. Report usable optional AD frameworks without importing them in-process.

Contract (`D, C, H, R, X`). Automatic differentiation adapters require a callable finite dimensional map, positive source and target dimensions, a declared maximum order, and a directional Jacobian vector product primitive supplied either directly or by an optional framework. Direction counts must equal the requested derivative order. Framework discovery is isolated in child processes so that a broken optional runtime cannot terminate the calling process.

Guarantees (`E, P, B, K`). A successful provider returns the finite directional derivative computed by the selected JVP recursion and records numerical exactness unless the caller supplies a stronger justified classification. Framework availability means that an isolated probe imported the runtime and found the required JVP capability. It does not assert numerical agreement for every model or device. Work grows with derivative order and the number of nested JVP calls.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_nonlinear_sparse_autodiff.py`, `tests/test_block10_sparse_autodiff.py`, and `tests/test_optional_backend_probe.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 37.

C.5A · `available_autodiff_backends`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.nonlinear.available_autodiff_backends`. Reflected implementation `komorebi_math.nonlinear.autodiff.available_ad_frameworks`. The same runtime object is also available through `komorebi_math.available_autodiff_backends`. Within `komorebi_math.nonlinear`, `available_ad_frameworks` resolves to the same runtime object. The public export name maps to implementation object `available_ad_frameworks`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C5-AUTODIFF`.

Manifest signature (`s`).

```
() -> 'dict[str, bool]'
```

Definition. Report usable optional AD frameworks without importing them in-process.

Contract (`D, C, H, R, X`). Automatic differentiation adapters require a callable finite dimensional map, positive source and target dimensions, a declared maximum order, and a directional Jacobian vector product primitive supplied either directly or by an optional framework. Direction counts must equal the requested derivative order. Framework discovery is isolated in child processes so that a broken optional runtime cannot terminate the calling process.

Guarantees (`E, P, B, K`). A successful provider returns the finite directional derivative computed by the selected JVP recursion and records numerical exactness unless the caller supplies a stronger justified classification. Framework availability means that an isolated probe imported the runtime and found the required JVP capability. It does not assert numerical agreement for every model or device. Work grows with derivative order and the number of nested JVP calls.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_nonlinear_sparse_autodiff.py`, `tests/test_block10_sparse_autodiff.py`, and `tests/test_optional_backend_probe.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 37.

C.5A · build_lift_tree

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.build_lift_tree`. Reflected implementation `komorebi_math.nonlinear.lifting.build_lift_tree`. The same runtime object is also available through `komorebi_math.build_lift_tree`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C5-LIFTING`.

Manifest signature (s).

```
(problem_factory: 'Callable[[tuple[int, ...], int], LinearLiftProblem | PolynomialLiftProblem]', roots:
   ↪ 'Iterable[tuple[int, ...]]', *, prime: 'int', start_depth: 'int', max_depth: 'int', max_nodes: 'int' =
   ↪ 100000) -> 'FiniteLiftTree'
```

Definition. Build a finite lift tree from exact one step correction problems.

Contract (D, C, H, R, X). Linear and polynomial lifting require a prime, a nonnegative depth, compatible finite equations, and a current residue class. One step searches the declared corrections modulo the prime. Tree construction requires an explicit terminal depth and a finite node budget.

Guarantees (E, P, B, K). For the lifting operations in this family, every returned child satisfies the represented congruence at the next prime power. Linear solves and branch verification are exact in modular arithmetic. Polynomial lifting uses the declared Jacobian and finite Taylor congruence. Branch counts may grow exponentially with depth, so the tree builder enforces the supplied node limit. The provenance registry records `PSC-III-T30` as Finite lift tree construction. Its recorded status is `proved`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_lifting.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 40.

C.5A · conditional_projective_limit_completion

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.conditional_projective_limit_completion`. Reflected implementation `komorebi_math.nonlinear.filtered.conditional_projective_limit_completion`. The same runtime object is also available through `komorebi_math.conditional_projective_limit_completion`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C5-FILTERED`.

Manifest signature (s).

```
(verified_prefix_depth: 'int', *, separated: 'bool', complete: 'bool', continuous: 'bool',
   ↪ compatible_infinite_branch: 'bool') -> 'ConditionalResult[ProjectiveLimitRecord]'
```

Definition. Record the conditional infinite filtered-lifting conclusion.

Contract (D, C, H, R, X). Filtered contracts require finite dimensional vector spaces over an explicit prime field and decreasing filtration bases with compatible ambient dimensions. Filtered correction is solved one finite depth at a time. A filtered cochain complex requires explicit differentials that square to zero and preserve the declared filtration. Projective limit completion also requires external completeness hypotheses supplied by the caller.

Guarantees (E, P, B, K). Associated graded coordinates, correction cosets, finite branches, and spectral sequence pages are exact over the represented finite field. The terminal finite filtration permits exact page dimension and stabilization checks. A projective limit record only stores the declared infinite hypotheses and finite compatibility evidence. It does not prove completeness or convergence beyond those assumptions.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_nonlinear_filtered.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 42.

C.5A · direct_packet_rotation

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.direct_packet_rotation`. Reflected implementation `komorebi_math.nonlinear.transport.direct_packet_rotation`. The same runtime object is also available through `komorebi_math.direct_packet_rotation`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C5-SMOOTH-TRANSPORT`.

Manifest signature (s).

```
(source: 'Iterable[Any]', target: 'Iterable[Any]', *, tolerance: 'float' = 1e-10) -> 'DirectRotationResult'
```

Definition. Return the polar unitary of `sum target[a] source[a]`.

Contract (D, C, H, R, X). Smooth transport requires complete finite orthogonal packet families of fixed dimension together with declared packet derivatives. Direct rotation requires two compatible packet families whose overlap sum is numerically invertible. Kato transport requires finite start and end times, a positive step count, and a positive tolerance.

Guarantees (E, P, B, K). Direct polar rotation returns a finite unitary intertwiner when its residual checks pass. Kato transport integrates the canonical finite generator with midpoint exponential steps and records unitarity, packet transport, and mesh refinement residuals. These certificates are numerical and path specific. They do not prove differentiability of an undeclared family or continuum convergence without external estimates. The provenance registry records `PSC-III-T55` as Direct rotation of finite packet systems. Its recorded status is `proved`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_nonlinear_transport.py` and `tests/test_quantum_transport_effective_spectral.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapters 40 and 44.

C.5A · divided_square_packets

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.divided_square_packets`. Reflected implementation `komorebi_math.nonlinear.graded.divided_square_packets`. The same runtime object is also available through `komorebi_math.divided_square_packets`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C5-GRADED`.

Manifest signature (s).

```
(group: 'FiniteAbelianGroup', components: 'Mapping[str, Any]', differential: 'LinearOperation', bracket:
    ↪ 'BracketOperation', divided_square: 'LinearOperation', *, exactness: 'Exactness' = <Exactness.NUMERICAL:
    ↪ 'numerical'>, tolerance: 'float' = 1e-10) -> 'DividedSquareResult'
```

Definition. Assemble `d eta + q(eta)` from divided-square packet identities.

Contract (D, C, H, R, X). Graded operations require a finite abelian grading group, finite component records, and multilinear operations whose declared output grade is the group sum of the input grades. Maurer Cartan routines evaluate only the supplied finite arity range. Divided square evaluation requires the supplied differential, bracket, and divided square maps to have compatible component shapes.

Guarantees (E, P, B, K). Grade bookkeeping and finite group arithmetic are exact. Numerical component values inherit the arithmetic of the supplied arrays and operations. Certificates compare the gradewise reconstruction with the direct finite expression and record any failed compatibility law. No unspecified higher operation or infinite Maurer Cartan series is inferred.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_nonlinear_graded.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this con-

tract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 39.

C.5A · faa_di_bruno

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.faa_di_bruno`. Reflected implementation `komorebi_math.nonlinear.composition.faa_di_bruno`. The same runtime object is also available through `komorebi_math.faa_di_bruno`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C5-COMPOSITION`.

Manifest signature (s).

```
(outer: 'DerivativeProvider', inner: 'DerivativeProvider', point: 'Any', directions: 'Sequence[Any]', *,
    ↪ direct_provider: 'DerivativeProvider | None' = None, tolerance: 'float' = 1e-09, retain_terms: 'bool' =
    ↪ True) -> 'FaaDiBrunoResult'
```

Definition. Evaluate the directional higher chain rule for `outer(inner(x))`.

Contract (D, C, H, R, X). Composition requires compatible inner and outer dimensions and derivative support through the requested order. The Faà di Bruno routines enumerate set partitions of the direction labels. Packet composition also requires compatible source, intermediate, and target packet systems. Partition utilities accept nonnegative finite orders and return canonical immutable records.

Guarantees (E, P, B, K). For the composition operations in this family, the computed derivative value equals the implemented finite Faà di Bruno sum for the supplied providers and directions. Exactness follows the participating providers. The composition result records retained terms, partition counts, and a certificate comparing the composed derivative with the direct implementation when that comparison is available. Enumeration grows according to Bell and Stirling counts, so callers must keep the derivative order bounded. The provenance registry records `PSC-III-T10` as Finite dimensional Faà di Bruno formula. Its recorded status is `classical input`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_nonlinear_jets_composition.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 38.

C.5A · higher_maurer_cartan_packets

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.higher_maurer_cartan_packets`. Reflected implementation `komorebi_math.nonlinear.graded.higher_maurer_cartan_packets`. The same runtime object is also available through `komorebi_math.higher_maurer_cartan_packets`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C5-GRADED`.

Manifest signature (s).

```
(group: 'FiniteAbelianGroup', components: 'Mapping[str, Any]', operations: 'Mapping[int,
    ↪ GradedOperationCallable]', *, exactness: 'Exactness' = <Exactness.NUMERICAL: 'numerical'>, tolerance:
    ↪ 'float' = 1e-10) -> 'HigherMaurerCartanResult'
```

Definition. Assemble a finite truncation `sum_r ell_r(eta^r)/r!` by labels.

Contract (D, C, H, R, X). Graded operations require a finite abelian grading group, finite component records, and multilinear operations whose declared output grade is the group sum of the input grades. Maurer Cartan routines evaluate only the supplied finite arity range. Divided square evaluation requires the supplied differential, bracket, and divided square maps to have compatible component shapes.

Guarantees (E, P, B, K). Grade bookkeeping and finite group arithmetic are exact. Numerical component values inherit the arithmetic of the supplied arrays and operations. Certificates compare the gradewise reconstruction with the direct finite expression and record any failed compatibility law. No unspecified higher operation or infinite Maurer Cartan series is inferred. The provenance registry records `PSC-III-P20` as Finite truncated packet equations

for higher operations. Its recorded status is `proved`. The provenance record notes that the result does not certify convergence of an infinite Maurer–Cartan series.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_nonlinear_graded.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 39.

C.5A · `labeled_composition_term_count`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.labeled_composition_term_count`. Reflected implementation `komorebi_math.nonlinear.composition.labeled_composition_term_count`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C5-COMPOSITION`.

Manifest signature (s).

```
(order: 'int', intermediate_packets: 'int') -> 'int'
```

Definition. Return `sum_k S(order,k) * intermediate_packets**k`.

Contract (D, C, H, R, X). Composition requires compatible inner and outer dimensions and derivative support through the requested order. The Faà di Bruno routines enumerate set partitions of the direction labels. Packet composition also requires compatible source, intermediate, and target packet systems. Partition utilities accept nonnegative finite orders and return canonical immutable records.

Guarantees (E, P, B, K). For the composition operations in this family, the computed derivative value equals the implemented finite Faà di Bruno sum for the supplied providers and directions. Exactness follows the participating providers. The composition result records retained terms, partition counts, and a certificate comparing the composed derivative with the direct implementation when that comparison is available. Enumeration grows according to Bell and Stirling counts, so callers must keep the derivative order bounded.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_nonlinear_jets_composition.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 38.

C.5A · `maurer_cartan_packets`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.maurer_cartan_packets`. Reflected implementation `komorebi_math.nonlinear.graded.maurer_cartan_packets`. The same runtime object is also available through `komorebi_math.maurer_cartan_packets`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C5-GRADED`.

Manifest signature (s).

```
(group: 'FiniteAbelianGroup', components: 'Mapping[str, Any]', differential: 'LinearOperation', bracket:
    ↪ 'BracketOperation', *, exactness: 'Exactness' = <Exactness.NUMERICAL: 'numerical'>, tolerance: 'float' =
    ↪ 1e-10) -> 'MaurerCartanResult'
```

Definition. Assemble the finite packet equations for `d eta + [eta,eta]/2`.

Contract (D, C, H, R, X). Graded operations require a finite abelian grading group, finite component records, and multilinear operations whose declared output grade is the group sum of the input grades. Maurer Cartan routines evaluate only the supplied finite arity range. Divided square evaluation requires the supplied differential, bracket, and divided square maps to have compatible component shapes.

Guarantees (E, P, B, K). Grade bookkeeping and finite group arithmetic are exact. Numerical component values inherit the arithmetic of the supplied arrays and operations. Certificates compare the gradewise reconstruction with the direct finite expression and record any failed compatibility law. No unspecified higher operation or infinite Maurer Cartan series is inferred. The provenance registry records `PSC-III-T19` as Packet Maurer–Cartan equations. Its recorded status is `proved`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_nonlinear_graded.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 39.

C.5A · `packet_faa_di_bruno`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.packet_faa_di_bruno`. Reflected implementation `komorebi_math.nonlinear.composition.packet_faa_di_bruno`. The same runtime object is also available through `komorebi_math.packet_faa_di_bruno`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C5-COMPOSITION`.

Manifest signature (s).

```
(outer: 'DerivativeProvider', inner: 'DerivativeProvider', point: 'Any', directions: 'Sequence[Any]', source:
    ↪ 'PacketSystem', intermediate: 'PacketSystem', target: 'PacketSystem', *, input_packets:
    ↪ 'Sequence[PacketLabel]', output_packet: 'PacketLabel', inner_support: 'Mapping[int, PacketJetSupport] |
    ↪ None' = None, outer_support: 'Mapping[int, PacketJetSupport] | None' = None, tolerance: 'float | None' =
    ↪ None, retain_terms: 'bool' = True, max_terms: 'int' = 1000000) -> 'FaaDiBrunoResult'
```

Definition. Evaluate one packet jet of a composition through decorated partitions.

Contract (D, C, H, R, X). Composition requires compatible inner and outer dimensions and derivative support through the requested order. The Faà di Bruno routines enumerate set partitions of the direction labels. Packet composition also requires compatible source, intermediate, and target packet systems. Partition utilities accept nonnegative finite orders and return canonical immutable records.

Guarantees (E, P, B, K). For the composition operations in this family, the computed derivative value equals the implemented finite Faà di Bruno sum for the supplied providers and directions. Exactness follows the participating providers. The composition result records retained terms, partition counts, and a certificate comparing the composed derivative with the direct implementation when that comparison is available. Enumeration grows according to Bell and Stirling counts, so callers must keep the derivative order bounded. The provenance registry records `PSC-III-T11A` as Packet Faà di Bruno formula. Its recorded status is `proved`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_nonlinear_jets_composition.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 38.

C.5A · `set_partitions`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.set_partitions`. Reflected implementation `komorebi_math.nonlinear.composition.set_partitions`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `data`. Profile `C5-COMPOSITION`.

Manifest signature (s).

```
(order: 'int') -> 'tuple[Partition, ...]'
```

Definition. Public finite nonlinear contract reflected from the implementation.

Contract (D, C, H, R, X). Composition requires compatible inner and outer dimensions and derivative support through the requested order. The Faà di Bruno routines enumerate set partitions of the direction labels. Packet composition also requires compatible source, intermediate, and target packet systems. Partition utilities accept nonnegative finite orders and return canonical immutable records.

Guarantees (E, P, B, K). For the composition operations in this family, the computed derivative value equals the implemented finite Faà di Bruno sum for the supplied providers and directions. Exactness follows the participating providers. The composition result records retained terms, partition counts, and a certificate comparing the composed derivative with the direct implementation when that comparison is available. Enumeration grows according to Bell and Stirling counts, so callers must keep the derivative order bounded.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_nonlinear_jets_composition.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 38.

C.5A · solve_mod_prime

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.solve_mod_prime`. Reflected implementation `komorebi_math.nonlinear.lifting.solve_mod_prime`. The same runtime object is also available through `komorebi_math.solve_mod_prime`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C5-LIFTING`.

Manifest signature (s).

```
(matrix: 'Any', rhs: 'Any', prime: 'int') -> 'ModularLinearSolution'
```

Definition. Solve a finite linear system modulo a prime by exact row reduction.

Contract (D, C, H, R, X). Linear and polynomial lifting require a prime, a nonnegative depth, compatible finite equations, and a current residue class. One step searches the declared corrections modulo the prime. Tree construction requires an explicit terminal depth and a finite node budget.

Guarantees (E, P, B, K). For the lifting operations in this family, every returned child satisfies the represented congruence at the next prime power. Linear solves and branch verification are exact in modular arithmetic. Polynomial lifting uses the declared Jacobian and finite Taylor congruence. Branch counts may grow exponentially with depth, so the tree builder enforces the supplied node limit. The provenance registry records `PSC-III-LIN-FP` as Linear correction space over a prime field. Its recorded status is `classical input`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_lifting.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 40.

C.5A · status

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.status`. Reflected implementation `komorebi_math.nonlinear.status`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C5-NAMESPACE`.

Manifest signature (s).

```
() -> komorebi_math.registry.FeatureSpec
```

Definition. Return the feature registry record for this package component.

Contract (D, C, H, R, X). The namespace status route takes no arguments and returns the registered nonlinear feature record for the installed package. It reports implementation availability and package metadata. It does not probe optional automatic differentiation or GPU runtimes.

Guarantees (E, P, B, K). The result is deterministic for one installed source tree. The route performs a registry lookup with constant auxiliary storage. It is an availability record, not a theorem certificate or backend qualification result.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_certificates_registry.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 6.

C.5A · `stirling_second`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.stirling_second`. Reflected implementation `komorebi_math.nonlinear.composition.stirling_second`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C5-COMPOSITION`.

Manifest signature (s).

```
(order: 'int', blocks: 'int') -> 'int'
```

Definition. Return an exact Stirling number of the second kind.

Contract (D, C, H, R, X). Composition requires compatible inner and outer dimensions and derivative support through the requested order. The Faà di Bruno routines enumerate set partitions of the direction labels. Packet composition also requires compatible source, intermediate, and target packet systems. Partition utilities accept nonnegative finite orders and return canonical immutable records.

Guarantees (E, P, B, K). For the composition operations in this family, the computed derivative value equals the implemented finite Faà di Bruno sum for the supplied providers and directions. Exactness follows the participating providers. The composition result records retained terms, partition counts, and a certificate comparing the composed derivative with the direct implementation when that comparison is available. Enumeration grows according to Bell and Stirling counts, so callers must keep the derivative order bounded.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_nonlinear_jets_composition.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 38.

C.5A · `verify_finite_filtered_branch`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.verify_finite_filtered_branch`. Reflected implementation `komorebi_math.nonlinear.filtered.verify_finite_filtered_branch`. The same runtime object is also available through `komorebi_math.verify_finite_filtered_branch`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C5-FILTERED`.

Manifest signature (s).

```
(source: 'FilteredVectorSpace', target: 'FilteredVectorSpace', map_function: 'Callable[[tuple[int, ...]],
   ↪ Sequence[int]]', approximations: 'Sequence[Sequence[int]]', *, start_depth: 'int' = 0) ->
   ↪ 'FiniteFilteredBranchResult'
```

Definition. Verify a compatible branch through a finite terminal filtration.

Contract (D, C, H, R, X). Filtered contracts require finite dimensional vector spaces over an explicit prime field and decreasing filtration bases with compatible ambient dimensions. Filtered correction is solved one finite depth at a time. A filtered cochain complex requires explicit differentials that square to zero and preserve the declared filtration. Projective limit completion also requires external completeness hypotheses supplied by the caller.

Guarantees (E, P, B, K). Associated graded coordinates, correction cosets, finite branches, and spectral sequence pages are exact over the represented finite field. The terminal finite filtration permits exact page dimension and stabilization checks. A projective limit record only stores the declared infinite hypotheses and finite compatibility evidence. It does not prove completeness or convergence beyond those assumptions.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_nonlinear_filtered.py`. The independent verifier for nonlinear calculus foundations exercises representative finite identities, rejection paths, implementation ownership, and alias behavior for this contract family. The source validator for nonlinear calculus foundations checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 42.

Nonlinear foundations reference. The 80 cards in C.5A cover 60 classes, 17 functions, and 3 data exports. Their recorded audit checks manifest and runtime identities, focused tests, independent finite checks, and reference anchors.

C.5B Packet jets, execution, streaming, and rooted trees: 69 entries

This section documents the public nonlinear namespace. It covers packet-jet addresses, families, support and Taylor reconstruction, sparse symmetric tensors, projection-cached execution, finite snapshots, resumable streams, optional GPU boundaries, observational profiling, and ordered rooted-tree expansions.

Contract family	Cards
Packet jets, supports, and Taylor reconstruction	8
Sparse symmetric tensors	3
Projection-cached and optional CuPy execution	12
Portable packet-jet snapshots	3
Resumable chunked streams and schemas	14
Persisted-snapshot GPU validation	6
Complexity estimates and observational profiles	7
Ordered rooted trees and fixed-point expansions	14
Namespace availability records	2
Total	**69**

The boundary remains finite and explicit. Packet support is certified only over the tested packet bases. Snapshots and streams preserve one captured finite multilinear block. Timing records describe one local environment. GPU contracts require actual CuPy and CUDA availability before device execution can be qualified. Rooted-tree convergence remains conditional on the supplied scalar majorant and contraction inequalities.

C.5B · CuPyDeviceDerivativeProvider

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.CuPyDeviceDerivativeProvider`. Reflected implementation `komorebi_math.nonlinear.execution.CupyDeviceDerivativeProvider`. The same runtime object is also available through `komorebi_math.CuPyDeviceDerivativeProvider`. The public export name maps to implementation object `CupyDeviceDerivativeProvider`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-EXECUTION`.

Manifest signature (s).

```
(*args, **kwargs)
```

Definition. Optional protocol consumed by `CupyPacketJetExecutor`.

Contract (D, C, H, R, X). Execution plans require a finite packet-jet family and an optional support record for the same order and packet names. Only blocks certified zero may be pruned. Plan compilation enforces an explicit address budget. Execution requires one direction per derivative slot and checks that the plan still matches the family. CuPy execution additionally requires an importable CuPy runtime and a CUDA device. Principal public operations and properties are `supports_order`, `device_derivative`.

Guarantees (E, P, B, K). The CPU planner groups active output blocks by common projected inputs, caches those projections, and reports derivative-call and projection counts. Reconstruction is compared with the direct represented derivative. Optional CuPy classes preserve the same finite packet contract when the hardware boundary is available. Runtime discovery alone is not a performance or correctness qualification, and unavailable device paths fail explicitly.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_nonlinear_execution.py` and `tests/test_block10_execution_persistence.py`. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 43.

C.5B · CuPyPacketJetExecutor

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.CuPyPacketJetExecutor`. Reflected implementation `komorebi_math.nonlinear.execution.CupyPacketJetExecutor`. The same runtime object is also available through `komorebi_math.CuPyPacketJetExecutor`. The public export name maps to implementation object `CupyPacketJetExecutor`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-EXECUTION`.

Manifest signature (s).

```
(family: 'PacketJetFamily', plan: 'PacketJetExecutionPlan | None' = None) -> 'None'
```

Definition. Device-resident packet projections and polynomial jet reconstruction.

Contract (D, C, H, R, X). Execution plans require a finite packet-jet family and an optional support record for the same order and packet names. Only blocks certified zero may be pruned. Plan compilation enforces an explicit address budget. Execution requires one direction per derivative slot and checks that the plan still matches the family. CuPy execution additionally requires an importable CuPy runtime and a CUDA device. Principal public operations and properties are `execute`, `synchronize`, `hardware_certificate`.

Guarantees (E, P, B, K). The CPU planner groups active output blocks by common projected inputs, caches those projections, and reports derivative-call and projection counts. Reconstruction is compared with the direct represented derivative. Optional CuPy classes preserve the same finite packet contract when the hardware boundary is available. Runtime discovery alone is not a performance or correctness qualification, and unavailable device paths fail explicitly.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_nonlinear_execution.py` and `tests/test_block10_execution_persistence.py`. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 43.

C.5B · `CuPyPacketJetSnapshotExecutor`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.CuPyPacketJetSnapshotExecutor`. Reflected implementation `komorebi_math.nonlinear.gpu.CuPySnapshotExecutor`. The same runtime object is also available through `komorebi_math.CuPyPacketJetSnapshotExecutor`. Within `komorebi_math.nonlinear`, `CuPySnapshotExecutor` resolves to the same runtime object. The public export name maps to implementation object `CuPySnapshotExecutor`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-GPU`.

Manifest signature (s).

```
(snapshot: 'PacketJetSnapshot', *, dtype: 'Any | None' = None) -> 'None'
```

Definition. Device evaluator for one sparse `PacketJetSnapshot`.

Contract (D, C, H, R, X). GPU status inspection never requires hardware and returns the observed CuPy import and CUDA device state. Snapshot execution requires a complete finite snapshot, a working CuPy CUDA runtime, and compatible probe shapes. Validation compares the same persisted coefficients on CPU and device under a declared tolerance. Principal public operations and properties are `evaluate_batch`, `evaluate`.

Guarantees (E, P, B, K). A successful validation record reports the probe count, maximum residual, runtime state, and certificate. The device executor evaluates only the captured finite snapshot and does not accept arbitrary Python callables. No speed claim follows from API presence. In a runtime without CuPy or a CUDA device, status reporting remains available while execution and validation are not hardware-qualified. The provenance registry records `PSC-III-N59` as Sparse CuPy packet-jet execution. Its recorded status is `finite certificate`. Recorded limitation: no performance claim without matched hardware benchmarking.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_block16_gpu_qualification.py` and `tests/test_nonlinear_execution.py`. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 43. The recorded audit runtime had no CuPy CUDA device, so GPU evidence is limited to status reporting, explicit unavailability, and CPU-side contract checks.

C.5B · `CuPyPolynomialDerivativeProvider`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.CuPyPolynomialDerivativeProvider`. Reflected implementation `komorebi_math.nonlinear.execution.CupyPolynomialDerivativeProvider`. The same runtime object is also available through `komorebi_math.CuPyPolynomialDerivativeProvider`. The public export name maps to implementation object `CupyPolynomialDerivativeProvider`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-EXECUTION`.

Manifest signature (s).

```
(tensors: 'Mapping[int, Any]', *, source_dimension: 'int | None' = None, target_dimension: 'int | None' = None,
    ↪ tolerance: 'float' = 1e-10, name: 'str' = 'CuPy polynomial derivative provider') -> 'None'
```

Definition. Dense polynomial derivatives resident on one CuPy device.

Contract (D, C, H, R, X). Execution plans require a finite packet-jet family and an optional support record for the same order and packet names. Only blocks certified zero may be pruned. Plan compilation enforces an explicit address

budget. Execution requires one direction per derivative slot and checks that the plan still matches the family. CuPy execution additionally requires an importable CuPy runtime and a CUDA device. Principal public operations and properties are `degree`, `supports_order`, `device_value`, `value`, `device_derivative`, `derivative`, `synchronize`.

Guarantees (`E`, `P`, `B`, `K`). The CPU planner groups active output blocks by common projected inputs, caches those projections, and reports derivative-call and projection counts. Reconstruction is compared with the direct represented derivative. Optional CuPy classes preserve the same finite packet contract when the hardware boundary is available. Runtime discovery alone is not a performance or correctness qualification, and unavailable device paths fail explicitly.

Evidence and links (`V`, `L`). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_nonlinear_execution.py` and `tests/test_block10_execution_persistence.py`. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 43.

C.5B · `CuPyRuntimeStatus`

Identity (`q`, `a`, `t`, `k`). Public research-manifest route `komorebi_math.nonlinear.CuPyRuntimeStatus`. Reflected implementation `komorebi_math.nonlinear.gpu.CuPyRuntimeStatus`. The same runtime object is also available through `komorebi_math.CuPyRuntimeStatus`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-GPU`.

Manifest signature (`s`).

```
(installed: 'bool', runtime_available: 'bool', device_count: 'int', runtime_version: 'int | None', error: 'str |
    ↪ None' = None) -> None
```

Definition. Observed CuPy import and CUDA device state.

Contract (`D`, `C`, `H`, `R`, `X`). GPU status inspection never requires hardware and returns the observed CuPy import and CUDA device state. Snapshot execution requires a complete finite snapshot, a working CuPy CUDA runtime, and compatible probe shapes. Validation compares the same persisted coefficients on CPU and device under a declared tolerance.

Guarantees (`E`, `P`, `B`, `K`). A successful validation record reports the probe count, maximum residual, runtime state, and certificate. The device executor evaluates only the captured finite snapshot and does not accept arbitrary Python callables. No speed claim follows from API presence. In a runtime without CuPy or a CUDA device, status reporting remains available while execution and validation are not hardware-qualified.

Evidence and links (`V`, `L`). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_block16_gpu_qualification.py` and `tests/test_nonlinear_execution.py`. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 43. The recorded audit runtime had no CuPy CUDA device, so GPU evidence is limited to status reporting, explicit unavailability, and CPU-side contract checks.

C.5B · `CuPySnapshotExecutor`

Identity (`q`, `a`, `t`, `k`). Public research-manifest route `komorebi_math.nonlinear.CuPySnapshotExecutor`. Reflected implementation `komorebi_math.nonlinear.gpu.CuPySnapshotExecutor`. The same runtime object is also available through `komorebi_math.CuPySnapshotExecutor`. Within `komorebi_math.nonlinear`, `CuPyPacketJetSnapshotExecutor` resolves to the same runtime object. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-GPU`.

Manifest signature (s).

```
(snapshot: 'PacketJetSnapshot', *, dtype: 'Any | None' = None) -> 'None'
```

Definition. Device evaluator for one sparse `PacketJetSnapshot`.

Contract (D, C, H, R, X). GPU status inspection never requires hardware and returns the observed CuPy import and CUDA device state. Snapshot execution requires a complete finite snapshot, a working CuPy CUDA runtime, and compatible probe shapes. Validation compares the same persisted coefficients on CPU and device under a declared tolerance. Principal public operations and properties are `evaluate_batch`, `evaluate`.

Guarantees (E, P, B, K). A successful validation record reports the probe count, maximum residual, runtime state, and certificate. The device executor evaluates only the captured finite snapshot and does not accept arbitrary Python callables. No speed claim follows from API presence. In a runtime without CuPy or a CUDA device, status reporting remains available while execution and validation are not hardware-qualified. The provenance registry records `PSC-III-N59` as Sparse CuPy packet-jet execution. Its recorded status is `finite certificate`. Recorded limitation: no performance claim without matched hardware benchmarking.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_block16_gpu_qualification.py` and `tests/test_nonlinear_execution.py`. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 43. The recorded audit runtime had no CuPy CUDA device, so GPU evidence is limited to status reporting, explicit unavailability, and CPU-side contract checks.

C.5B · `CuPyValidationResult`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.CuPyValidationResult`. Reflected implementation `komorebi_math.nonlinear.gpu.CuPyValidationResult`. The same runtime object is also available through `komorebi_math.CuPyValidationResult`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-GPU`.

Manifest signature (s).

```
(probe_count: 'int', maximum_residual: 'float', runtime: 'CuPyRuntimeStatus', certificate: 'Certificate') -> None
```

Definition. Matched CPU and CuPy packet-jet validation data.

Contract (D, C, H, R, X). GPU status inspection never requires hardware and returns the observed CuPy import and CUDA device state. Snapshot execution requires a complete finite snapshot, a working CuPy CUDA runtime, and compatible probe shapes. Validation compares the same persisted coefficients on CPU and device under a declared tolerance.

Guarantees (E, P, B, K). A successful validation record reports the probe count, maximum residual, runtime state, and certificate. The device executor evaluates only the captured finite snapshot and does not accept arbitrary Python callables. No speed claim follows from API presence. In a runtime without CuPy or a CUDA device, status reporting remains available while execution and validation are not hardware-qualified.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_block16_gpu_qualification.py` and `tests/test_nonlinear_execution.py`. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 43. The recorded audit runtime had no CuPy CUDA device, so GPU evidence is limited to status reporting, explicit unavailability, and CPU-side contract checks.

C.5B · FEATURE_KEY

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.FEATURE_KEY`. Reflected implementation `builtins.str`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The public export name maps to implementation object `str`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `data`. Profile `C5-NAMESPACE`.

Manifest signature (s).

```
null
```

Definition. Literal registry key for the nonlinear package component.

Contract (D, C, H, R, X). The namespace marker is the literal feature key used by the registry. The `require` route takes no arguments and returns successfully for this implemented package component. Neither contract imports optional GPU runtimes or changes global state.

Guarantees (E, P, B, K). The feature key is deterministic for the installed source tree. The availability check has constant auxiliary storage and makes no statement about optional devices, performance, or the validity of a particular nonlinear model.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_certificates_registry.py`. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 6.

C.5B · MajorantConvergence

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.MajorantConvergence`. Reflected implementation `komorebi_math.nonlinear.trees.MajorantConvergence`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-ROOTED-TREES`.

Manifest signature (s).

```
(seed_norm: 'float', operation_norms: 'dict[int, float]', radius: 'float', image_bound: 'float', contraction:
    ↪  'float', certificate: 'Certificate') -> None
```

Definition. Scalar majorant data for a polynomial fixed-point problem.

Contract (D, C, H, R, X). Ordered-tree enumeration requires internal arities of at least two, nonnegative height or positive leaf targets, and an explicit tree budget. Polynomial fixed-point problems require a finite seed, compatible multilinear operations, and declared exactness and tolerance. Majorant routines require finite nonnegative norm bounds and a positive candidate radius. Principal public operations and properties are `scalar_map`, `scalar_iterates`, `picard_tail_bound`.

Guarantees (E, P, B, K). Tree counts and finite enumerations are exact within the requested boundary. A tree expansion equals the matching finite Picard iterate under the implemented polynomial operations. A convergence certificate is issued only when the supplied scalar majorant maps the declared ball into itself and has contraction factor below one. Packet reconstruction prunes only addresses covered by supplied zero-support certificates. Enumeration can grow exponentially or faster with height and arity.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_nonlinear_trees.py`. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 39.

C.5B · MultilinearOperation

Identity (q, a, t, k). Public research-manifest route komorebi_math.nonlinear.MultilinearOperation. Reflected implementation collections.abc.Callable. The same runtime object is also available through komorebi_math.MultilinearOperation. The public export name maps to implementation object Callable. The komorebi_math.nonlinear route is governed by [RESEARCH] policy pre-beta-additive. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind data. Profile C5-ROOTED-TREES.

Manifest signature (s).

```
(*args, **kwargs)
```

Definition. Callable type alias for one finite multilinear operation on a tuple of values.

Contract (D, C, H, R, X). Ordered-tree enumeration requires internal arities of at least two, nonnegative height or positive leaf targets, and an explicit tree budget. Polynomial fixed-point problems require a finite seed, compatible multilinear operations, and declared exactness and tolerance. Majorant routines require finite nonnegative norm bounds and a positive candidate radius.

Guarantees (E, P, B, K). Tree counts and finite enumerations are exact within the requested boundary. A tree expansion equals the matching finite Picard iterate under the implemented polynomial operations. A convergence certificate is issued only when the supplied scalar majorant maps the declared ball into itself and has contraction factor below one. Packet reconstruction prunes only addresses covered by supplied zero-support certificates. Enumeration can grow exponentially or faster with height and arity.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is tests/test_nonlinear_trees.py. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 39.

C.5B · OrderedRootedTree

Identity (q, a, t, k). Public research-manifest route komorebi_math.nonlinear.OrderedRootedTree. Reflected implementation komorebi_math.nonlinear.trees.OrderedRootedTree. The same runtime object is also available through komorebi_math.OrderedRootedTree. The komorebi_math.nonlinear route is governed by [RESEARCH] policy pre-beta-additive. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind class. Profile C5-ROOTED-TREES.

Manifest signature (s).

```
(children: "tuple['OrderedRootedTree', ...]" = ()) -> None
```

Definition. An unlabeled ordered rooted tree.

Contract (D, C, H, R, X). Ordered-tree enumeration requires internal arities of at least two, nonnegative height or positive leaf targets, and an explicit tree budget. Polynomial fixed-point problems require a finite seed, compatible multilinear operations, and declared exactness and tolerance. Majorant routines require finite nonnegative norm bounds and a positive candidate radius. Principal public operations and properties are leaf, is_leaf, arity, height, leaf_count, internal_count, vertex_count, arity_profile, prefix, to_dict.

Guarantees (E, P, B, K). Tree counts and finite enumerations are exact within the requested boundary. A tree expansion equals the matching finite Picard iterate under the implemented polynomial operations. A convergence certificate is issued only when the supplied scalar majorant maps the declared ball into itself and has contraction factor below one. Packet reconstruction prunes only addresses covered by supplied zero-support certificates. Enumeration can grow exponentially or faster with height and arity. The provenance registry records PSC-III-D13 as Finite ordered rooted tree. Its recorded status is definition.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is tests/test_nonlinear_trees.py. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity

for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 39.

C.5B · PACKET_JET_CHUNK_SCHEMA

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.PACKET_JET_CHUNK_SCHEMA`. Reflected implementation `builtins.str`. The same runtime object is also available through `komorebi_math.PACKET_JET_CHUNK_SCHEMA`, `komorebi_math.beta.PACKET_JET_CHUNK_SCHEMA`. The public export name maps to implementation object `str`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `data`. Profile `C5-STREAMING`.

Manifest signature (s).

```
null
```

Definition. Canonical schema identifier for one packet-jet coefficient chunk envelope.

Contract (D, C, H, R, X). Chunked streams use fixed schema identifiers and version 1 envelopes. Writers require a destination, a positive chunk size, and explicit overwrite or resume policy. Direct capture writes to a partial directory and promotes it only after completion. Readers reject duplicate JSON keys, unsupported schemas, inconsistent ranges, digest mismatches, and incomplete streams unless incomplete access is explicitly allowed.

Guarantees (E, P, B, K). Each committed chunk has a payload digest and a manifest range. Resume continues only from a verified committed prefix. Selection and evaluation can operate with memory bounded by the configured chunk size. Loading a conventional snapshot requires a complete stream and an explicit coefficient ceiling. Digests provide integrity against accidental corruption, not authentication against an adversary able to replace the entire stream and its hashes.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_nonlinear_snapshots_persistence.py` and `tests/test_block11_streaming_migrations.py`. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 43.

C.5B · PACKET_JET_CHUNK_SCHEMA_VERSION

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.PACKET_JET_CHUNK_SCHEMA_VERSION`. Reflected implementation `builtins.int`. The same runtime object is also available through `komorebi_math.PACKET_JET_CHUNK_SCHEMA_VERSION`, `komorebi_math.beta.PACKET_JET_CHUNK_SCHEMA_VERSION`. Within `komorebi_math.nonlinear`, `PACKET_JET_STREAM_SCHEMA_VERSION` resolves to the same runtime object. The public export name maps to implementation object `int`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `data`. Profile `C5-STREAMING`.

Manifest signature (s).

```
null
```

Definition. Current integer version of the packet-jet chunk schema.

Contract (D, C, H, R, X). Chunked streams use fixed schema identifiers and version 1 envelopes. Writers require a destination, a positive chunk size, and explicit overwrite or resume policy. Direct capture writes to a partial directory and promotes it only after completion. Readers reject duplicate JSON keys, unsupported schemas, inconsistent ranges, digest mismatches, and incomplete streams unless incomplete access is explicitly allowed.

Guarantees (E, P, B, K). Each committed chunk has a payload digest and a manifest range. Resume continues only from a verified committed prefix. Selection and evaluation can operate with memory bounded by the configured chunk size. Loading a conventional snapshot requires a complete stream and an explicit coefficient ceiling. Digests provide integrity against accidental corruption, not authentication against an adversary able to replace the entire stream and its hashes.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_nonlinear_snapshots_persistence.py` and `tests/test_block11_streaming_migrations.py`. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 43.

C.5B · PACKET_JET_STREAM_SCHEMA

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.PACKET_JET_STREAM_SCHEMA`. Reflected implementation `builtins.str`. The same runtime object is also available through `komorebi_math.PACKET_JET_STREAM_SCHEMA`, `komorebi_math.beta.PACKET_JET_STREAM_SCHEMA`. The public export name maps to implementation object `str`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `data`. Profile `C5-STREAMING`.

Manifest signature (s).

```
null
```

Definition. Canonical schema identifier for a packet-jet stream manifest envelope.

Contract (D, C, H, R, X). Chunked streams use fixed schema identifiers and version 1 envelopes. Writers require a destination, a positive chunk size, and explicit overwrite or resume policy. Direct capture writes to a partial directory and promotes it only after completion. Readers reject duplicate JSON keys, unsupported schemas, inconsistent ranges, digest mismatches, and incomplete streams unless incomplete access is explicitly allowed.

Guarantees (E, P, B, K). Each committed chunk has a payload digest and a manifest range. Resume continues only from a verified committed prefix. Selection and evaluation can operate with memory bounded by the configured chunk size. Loading a conventional snapshot requires a complete stream and an explicit coefficient ceiling. Digests provide integrity against accidental corruption, not authentication against an adversary able to replace the entire stream and its hashes.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_nonlinear_snapshots_persistence.py` and `tests/test_block11_streaming_migrations.py`. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 43.

C.5B · PACKET_JET_STREAM_SCHEMA_VERSION

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.PACKET_JET_STREAM_SCHEMA_VERSION`. Reflected implementation `builtins.int`. The same runtime object is also available through `komorebi_math.PACKET_JET_STREAM_SCHEMA_VERSION`, `komorebi_math.beta.PACKET_JET_STREAM_SCHEMA_VERSION`. Within `komorebi_math.nonlinear`, `PACKET_JET_CHUNK_SCHEMA_VERSION` resolves to the same runtime object. The public export name maps to implementation object `int`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `data`. Profile `C5-STREAMING`.

Manifest signature (s).

```
null
```

Definition. Current integer version of the packet-jet stream manifest schema.

Contract (D, C, H, R, X). Chunked streams use fixed schema identifiers and version 1 envelopes. Writers require a destination, a positive chunk size, and explicit overwrite or resume policy. Direct capture writes to a partial directory and promotes it only after completion. Readers reject duplicate JSON keys, unsupported schemas, inconsistent ranges, digest mismatches, and incomplete streams unless incomplete access is explicitly allowed.

Guarantees (E, P, B, K). Each committed chunk has a payload digest and a manifest range. Resume continues only from a verified committed prefix. Selection and evaluation can operate with memory bounded by the configured

chunk size. Loading a conventional snapshot requires a complete stream and an explicit coefficient ceiling. Digests provide integrity against accidental corruption, not authentication against an adversary able to replace the entire stream and its hashes.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_nonlinear_snapshots_persistence.py` and `tests/test_block11_streaming_migrations.py`. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 43.

C.5B · PacketJet

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.PacketJet`. Reflected implementation `komorebi_math.nonlinear.jets.PacketJet`. The same runtime object is also available through `komorebi_math.PacketJet`, `komorebi_math.beta.PacketJet`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-PACKET-JETS`.

Manifest signature (s).

```
(provider: 'DerivativeProvider', point: 'Any', source: 'PacketSystem', target: 'PacketSystem', output_index:
    ↪ 'int', input_indices: 'tuple[int, ...]') -> None
```

Definition. One projected multilinear derivative block.

Contract (D, C, H, R, X). Packet jets require a derivative provider, compatible finite source and target packet systems, a valid base point, and a positive supported derivative order. Packet labels must resolve inside their systems. Evaluation requires one source direction per input address. Support discovery and Taylor assembly also require positive finite tolerances and explicit evaluation budgets. Principal public operations and properties are `order`, `address`, `exactness`, `evaluate`, `norm_bound`, `symmetry_certificate`, `to_dict`.

Guarantees (E, P, B, K). A packet jet evaluates the declared derivative after source projection and target projection. A complete family reconstructs the represented finite derivative when both packet systems are complete. Support classifications are exhaustive only when the evaluation budget covers every required packet basis probe. Taylor certificates compare one finite truncation with the observed map value at the supplied increment. Address counts and support work grow combinatorially with order and packet count. The provenance registry records `PSC-III-D06` as Finite packet jet. Its recorded status is `definition`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_nonlinear_jets_composition.py` and `tests/test_nonlinear_execution.py`. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 38.

C.5B · PacketJetAddress

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.PacketJetAddress`. Reflected implementation `komorebi_math.nonlinear.jets.PacketJetAddress`. The same runtime object is also available through `komorebi_math.beta.PacketJetAddress`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-PACKET-JETS`.

Manifest signature (s).

```
(output: 'str', inputs: 'tuple[str, ...]') -> None
```

Definition. The output and ordered input packet addresses of one jet block.

Contract (D, C, H, R, X). Packet jets require a derivative provider, compatible finite source and target packet systems, a valid base point, and a positive supported derivative order. Packet labels must resolve inside their systems. Evaluation requires one source direction per input address. Support discovery and Taylor assembly also require positive finite tolerances and explicit evaluation budgets. Principal public operations and properties are `order`, `to_dict`.

Guarantees (E, P, B, K). A packet jet evaluates the declared derivative after source projection and target projection. A complete family reconstructs the represented finite derivative when both packet systems are complete. Support classifications are exhaustive only when the evaluation budget covers every required packet basis probe. Taylor certificates compare one finite truncation with the observed map value at the supplied increment. Address counts and support work grow combinatorially with order and packet count.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_nonlinear_jets_composition.py` and `tests/test_nonlinear_execution.py`. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 38.

C.5B · `PacketJetBatchResult`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.PacketJetBatchResult`. Reflected implementation `komorebi_math.nonlinear.execution.PacketJetBatchResult`. The same runtime object is also available through `komorebi_math.PacketJetBatchResult`, `komorebi_math.beta.PacketJetBatchResult`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-EXECUTION`.

Manifest signature (s).

```
(evaluations: 'tuple[PacketJetEvaluation, ...]', reconstructed: 'Any', direct: 'Any', derivative_calls: 'int',
    ↪ input_projections: 'int', output_projections: 'int', pruned_blocks: 'int', backend: 'str', returned_to_host:
    ↪ 'bool', certificate: 'Certificate') -> None
```

Definition. Result of one projection-cached packet-jet execution.

Contract (D, C, H, R, X). Execution plans require a finite packet-jet family and an optional support record for the same order and packet names. Only blocks certified zero may be pruned. Plan compilation enforces an explicit address budget. Execution requires one direction per derivative slot and checks that the plan still matches the family. CuPy execution additionally requires an importable CuPy runtime and a CUDA device. Principal public operations and properties are `as_mapping`, `value`, `to_dict`.

Guarantees (E, P, B, K). The CPU planner groups active output blocks by common projected inputs, caches those projections, and reports derivative-call and projection counts. Reconstruction is compared with the direct represented derivative. Optional CuPy classes preserve the same finite packet contract when the hardware boundary is available. Runtime discovery alone is not a performance or correctness qualification, and unavailable device paths fail explicitly.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_nonlinear_execution.py` and `tests/test_block10_execution_persistence.py`. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 43.

C.5B · PacketJetCoefficient

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.PacketJetCoefficient`. Reflected implementation `komorebi_math.nonlinear.snapshots.PacketJetCoefficient`. The same runtime object is also available through `komorebi_math.beta.PacketJetCoefficient`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-SNAPSHOTS`.

Manifest signature (s).

```
(indices: 'tuple[int, ...]', value: 'tuple[Any, ...]') -> None
```

Definition. One standard-basis coefficient of a persisted packet jet.

Contract (D, C, H, R, X). A snapshot captures one fixed packet jet by evaluating the ambient standard-basis tuples through an explicit finite budget. The source and target dimensions, base point, address, provider name, exactness, and sparse coefficients are stored in the record. Snapshot evaluation requires direction vectors of the recorded source dimension.

Guarantees (E, P, B, K). The snapshot is provider-independent after capture and reconstructs the recorded multilinear block from its stored coefficients. Its comparison certificate checks matched evaluations against the source jet on supplied probes. Exact coefficients preserve exact arithmetic. Numerical coefficients use the stored tolerance and certificate. Dense capture cost grows as the source dimension raised to the jet order.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_nonlinear_snapshots_persistence.py` and `tests/test_block10_execution_persistence.py`. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapters 38 and 43.

C.5B · PacketJetComplexityEstimate

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.PacketJetComplexityEstimate`. Reflected implementation `komorebi_math.nonlinear.profiling.PacketJetComplexityEstimate`. The same runtime object is also available through `komorebi_math.PacketJetComplexityEstimate`, `komorebi_math.beta.PacketJetComplexityEstimate`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-PROFILING`.

Manifest signature (s).

```
(order: 'int', source_dimension: 'int', target_dimension: 'int', source_packets: 'int', target_packets: 'int',
    ↪ family_addresses: 'int', active_addresses: 'int', pruned_addresses: 'int', input_groups: 'int',
    ↪ naive_derivative_calls: 'int', planned_derivative_calls: 'int', projected_input_vectors: 'int',
    ↪ output_projections: 'int', dense_basis_evaluations: 'int', dense_tensor_elements: 'int', certificate:
    ↪ 'Certificate') -> None
```

Definition. Exact finite counts implied by packet and ambient dimensions.

Contract (D, C, H, R, X). Complexity estimation requires a finite family and a compatible optional support record or execution plan. Runtime profiles require positive repetition counts, nonnegative warmup counts, and a precisely stated measurement scope. Packet-jet profiling may include planning, execution, snapshot capture, and optional stream writing under explicit budgets. Principal public operations and properties are `derivative_call_reduction`, `to_dict`.

Guarantees (E, P, B, K). Deterministic estimate fields are exact finite counts derived from the represented dimensions, packets, addresses, and plan. Timing and traced Python allocation samples are environment-dependent observations. They are not asymptotic evidence and do not include unreported device allocation or external process cost. Reports retain the interpreter, platform, scope, repetitions, and certificate needed to interpret the measurements.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_block11_golden_profiling_beta.py` and `tests/test_nonlinear_execution.py`. The

independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 43.

C.5B · PacketJetEvaluation

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.PacketJetEvaluation`. Reflected implementation `komorebi_math.nonlinear.execution.PacketJetEvaluation`. The same runtime object is also available through `komorebi_math.beta.PacketJetEvaluation`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-EXECUTION`.

Manifest signature (s).

```
(address: 'PacketJetAddress', value: 'Any') -> None
```

Definition. One evaluated packet-jet address and its projected target value.

Contract (D, C, H, R, X). Execution plans require a finite packet-jet family and an optional support record for the same order and packet names. Only blocks certified zero may be pruned. Plan compilation enforces an explicit address budget. Execution requires one direction per derivative slot and checks that the plan still matches the family. CuPy execution additionally requires an importable CuPy runtime and a CUDA device.

Guarantees (E, P, B, K). The CPU planner groups active output blocks by common projected inputs, caches those projections, and reports derivative-call and projection counts. Reconstruction is compared with the direct represented derivative. Optional CuPy classes preserve the same finite packet contract when the hardware boundary is available. Runtime discovery alone is not a performance or correctness qualification, and unavailable device paths fail explicitly.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_nonlinear_execution.py` and `tests/test_block10_execution_persistence.py`. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 43.

C.5B · PacketJetExecutionPlan

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.PacketJetExecutionPlan`. Reflected implementation `komorebi_math.nonlinear.execution.PacketJetExecutionPlan`. The same runtime object is also available through `komorebi_math.PacketJetExecutionPlan`, `komorebi_math.beta.PacketJetExecutionPlan`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-EXECUTION`.

Manifest signature (s).

```
(order: 'int', source_names: 'tuple[str, ...]', target_names: 'tuple[str, ...]', groups:
    ↪ 'tuple[PacketJetInputGroup, ...]', active_addresses: 'tuple[PacketJetAddress, ...]', pruned_addresses:
    ↪ 'tuple[PacketJetAddress, ...]', support_was_supplied: 'bool', support_complete: 'bool', certificate:
    ↪ 'Certificate') -> None
```

Definition. A deterministic grouping of packet-jet blocks by projected inputs.

Contract (D, C, H, R, X). Execution plans require a finite packet-jet family and an optional support record for the same order and packet names. Only blocks certified zero may be pruned. Plan compilation enforces an explicit address budget. Execution requires one direction per derivative slot and checks that the plan still matches the family. CuPy execution additionally requires an importable CuPy runtime and a CUDA device. Principal public operations and properties are `active_address_count`, `pruned_address_count`, `full_address_count`, `derivative_call_count`, `naive_derivative_call_count`, `derivative_call_reduction`, `matches`, `to_dict`.

Guarantees (E, P, B, K). The CPU planner groups active output blocks by common projected inputs, caches those projections, and reports derivative-call and projection counts. Reconstruction is compared with the direct represented derivative. Optional CuPy classes preserve the same finite packet contract when the hardware boundary is available. Runtime discovery alone is not a performance or correctness qualification, and unavailable device paths fail explicitly. The provenance registry records `PSC-III-A10` as Projection-cached packet-jet execution plan. Its recorded status is `proved`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_nonlinear_execution.py` and `tests/test_block10_execution_persistence.py`. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 43.

C.5B · `PacketJetFamily`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.PacketJetFamily`. Reflected implementation `komorebi_math.nonlinear.jets.PacketJetFamily`. The same runtime object is also available through `komorebi_math.PacketJetFamily`, `komorebi_math.beta.PacketJetFamily`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-PACKET-JETS`.

Manifest signature (s).

```
(provider: 'DerivativeProvider', point: 'Any', source: 'PacketSystem', target: 'PacketSystem', order: 'int') ->
    ↪ 'None'
```

Definition. Every packet block of one derivative order at one base point.

Contract (D, C, H, R, X). Packet jets require a derivative provider, compatible finite source and target packet systems, a valid base point, and a positive supported derivative order. Packet labels must resolve inside their systems. Evaluation requires one source direction per input address. Support discovery and Taylor assembly also require positive finite tolerances and explicit evaluation budgets. Principal public operations and properties are `jet`, `addresses`, `reconstruct`, `reconstruction_certificate`, `to_dict`.

Guarantees (E, P, B, K). A packet jet evaluates the declared derivative after source projection and target projection. A complete family reconstructs the represented finite derivative when both packet systems are complete. Support classifications are exhaustive only when the evaluation budget covers every required packet basis probe. Taylor certificates compare one finite truncation with the observed map value at the supplied increment. Address counts and support work grow combinatorially with order and packet count. The provenance registry records `PSC-III-T07` as Exact reconstruction of finite packet derivatives. Its recorded status is `proved`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_nonlinear_jets_composition.py` and `tests/test_nonlinear_execution.py`. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 38.

C.5B · `PacketJetInputGroup`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.PacketJetInputGroup`. Reflected implementation `komorebi_math.nonlinear.execution.PacketJetInputGroup`. The same runtime object is also available through `komorebi_math.beta.PacketJetInputGroup`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-EXECUTION`.

Manifest signature (s).

```
(inputs: 'tuple[int, ...]', outputs: 'tuple[int, ...]') -> None
```

Definition. Active output packets sharing one ordered input-packet address.

Contract (D, C, H, R, X). Execution plans require a finite packet-jet family and an optional support record for the same order and packet names. Only blocks certified zero may be pruned. Plan compilation enforces an explicit address budget. Execution requires one direction per derivative slot and checks that the plan still matches the family. CuPy execution additionally requires an importable CuPy runtime and a CUDA device.

Guarantees (E, P, B, K). The CPU planner groups active output blocks by common projected inputs, caches those projections, and reports derivative-call and projection counts. Reconstruction is compared with the direct represented derivative. Optional CuPy classes preserve the same finite packet contract when the hardware boundary is available. Runtime discovery alone is not a performance or correctness qualification, and unavailable device paths fail explicitly.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_nonlinear_execution.py` and `tests/test_block10_execution_persistence.py`. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 43.

C.5B · `PacketJetProfileReport`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.PacketJetProfileReport`. Reflected implementation `komorebi_math.nonlinear.profiling.PacketJetProfileReport`. The same runtime object is also available through `komorebi_math.PacketJetProfileReport`, `komorebi_math.beta.PacketJetProfileReport`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-PROFILING`.

Manifest signature (s).

```
(estimate: 'PacketJetComplexityEstimate', stages: 'tuple[RuntimeStageProfile, ...]', environment: 'Mapping[str,
    ↪ Any]', certificate: 'Certificate') -> None
```

Definition. Exact counts paired with environment-specific runtime observations.

Contract (D, C, H, R, X). Complexity estimation requires a finite family and a compatible optional support record or execution plan. Runtime profiles require positive repetition counts, nonnegative warmup counts, and a precisely stated measurement scope. Packet-jet profiling may include planning, execution, snapshot capture, and optional stream writing under explicit budgets. Principal public operations and properties are `to_dict`.

Guarantees (E, P, B, K). Deterministic estimate fields are exact finite counts derived from the represented dimensions, packets, addresses, and plan. Timing and traced Python allocation samples are environment-dependent observations. They are not asymptotic evidence and do not include unreported device allocation or external process cost. Reports retain the interpreter, platform, scope, repetitions, and certificate needed to interpret the measurements.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_block11_golden_profiling_beta.py` and `tests/test_nonlinear_execution.py`. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 43.

C.5B · PacketJetSnapshot

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.PacketJetSnapshot`. Reflected implementation `komorebi_math.nonlinear.snapshots.PacketJetSnapshot`. The same runtime object is also available through `komorebi_math.PacketJetSnapshot`, `komorebi_math.beta.PacketJetSnapshot`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-SNAPSHOTS`.

Manifest signature (s).

```
(address: 'PacketJetAddress', source_dimension: 'int', target_dimension: 'int', provider_name: 'str', exactness:
    ↪ 'Exactness', point: 'tuple[Any, ...]', coefficients: 'tuple[PacketJetCoefficient, ...]',
    ↪ capture_certificate: 'Certificate') -> None
```

Definition. A provider-independent sparse coefficient record of one packet jet.

Contract (D, C, H, R, X). A snapshot captures one fixed packet jet by evaluating the ambient standard-basis tuples through an explicit finite budget. The source and target dimensions, base point, address, provider name, exactness, and sparse coefficients are stored in the record. Snapshot evaluation requires direction vectors of the recorded source dimension. Principal public operations and properties are `order`, `nnz`, `dense_coefficient_count`, `coefficient`, `evaluate`, `to_dense`, `comparison_certificate`, `to_dict`.

Guarantees (E, P, B, K). The snapshot is provider-independent after capture and reconstructs the recorded multilinear block from its stored coefficients. Its comparison certificate checks matched evaluations against the source jet on supplied probes. Exact coefficients preserve exact arithmetic. Numerical coefficients use the stored tolerance and certificate. Dense capture cost grows as the source dimension raised to the jet order. The provenance registry records `PSC-III-D58` as Finite packet-jet snapshot. Its recorded status is `finite certificate`. Recorded limitation: snapshot represents one fixed order, address, and base point.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_nonlinear_snapshots_persistence.py` and `tests/test_block10_execution_persistence.py`. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapters 38 and 43.

C.5B · PacketJetSnapshotStreamReader

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.PacketJetSnapshotStreamReader`. Reflected implementation `komorebi_math.nonlinear.streaming.PacketJetSnapshotStreamReader`. The same runtime object is also available through `komorebi_math.PacketJetSnapshotStreamReader`, `komorebi_math.beta.PacketJetSnapshotStreamReader`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-STREAMING`.

Manifest signature (s).

```
(path: 'str | os.PathLike[str]', *, allow_incomplete: 'bool' = False, verify_chunks: 'bool' = False) -> 'None'
```

Definition. Bounded-memory reader for a complete or explicitly allowed partial stream.

Contract (D, C, H, R, X). Chunked streams use fixed schema identifiers and version 1 envelopes. Writers require a destination, a positive chunk size, and explicit overwrite or resume policy. Direct capture writes to a partial directory and promotes it only after completion. Readers reject duplicate JSON keys, unsupported schemas, inconsistent ranges, digest mismatches, and incomplete streams unless incomplete access is explicitly allowed. Principal public operations and properties are `complete`, `iter_chunks`, `iter_coefficients`, `select`, `evaluate`, `load_snapshot`, `verify`.

Guarantees (E, P, B, K). Each committed chunk has a payload digest and a manifest range. Resume continues only from a verified committed prefix. Selection and evaluation can operate with memory bounded by the configured chunk size. Loading a conventional snapshot requires a complete stream and an explicit coefficient ceiling. Digests provide integrity against accidental corruption, not authentication against an adversary able to replace the entire stream and its hashes.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_nonlinear_snapshots_persistence.py` and `tests/test_block11_streaming_migrations.py`. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 43.

C.5B · `PacketJetStreamChunkRecord`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.PacketJetStreamChunkRecord`. Reflected implementation `komorebi_math.nonlinear.streaming.PacketJetStreamChunkRecord`. The same runtime object is also available through `komorebi_math.PacketJetStreamChunkRecord`, `komorebi_math.beta.PacketJetStreamChunkRecord`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-STREAMING`.

Manifest signature (s).

```
(index: 'int', filename: 'str', basis_start: 'int', basis_stop: 'int', coefficient_start: 'int', coefficient_stop:
   ↪ 'int', payload_sha256: 'str', byte_count: 'int') -> None
```

Definition. One immutable chunk index entry in a packet-jet stream manifest.

Contract (D, C, H, R, X). Chunked streams use fixed schema identifiers and version 1 envelopes. Writers require a destination, a positive chunk size, and explicit overwrite or resume policy. Direct capture writes to a partial directory and promotes it only after completion. Readers reject duplicate JSON keys, unsupported schemas, inconsistent ranges, digest mismatches, and incomplete streams unless incomplete access is explicitly allowed. Principal public operations and properties are `coefficient_count`, `basis_count`, `to_dict`, `from_dict`.

Guarantees (E, P, B, K). Each committed chunk has a payload digest and a manifest range. Resume continues only from a verified committed prefix. Selection and evaluation can operate with memory bounded by the configured chunk size. Loading a conventional snapshot requires a complete stream and an explicit coefficient ceiling. Digests provide integrity against accidental corruption, not authentication against an adversary able to replace the entire stream and its hashes.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_nonlinear_snapshots_persistence.py` and `tests/test_block11_streaming_migrations.py`. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 43.

C.5B · `PacketJetStreamManifest`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.PacketJetStreamManifest`. Reflected implementation `komorebi_math.nonlinear.streaming.PacketJetStreamManifest`. The same runtime object is also available through `komorebi_math.PacketJetStreamManifest`, `komorebi_math.beta.PacketJetStreamManifest`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-STREAMING`.

Manifest signature (s).

```
(state: 'StreamState', producer_version: 'str', address: 'PacketJetAddress', source_dimension: 'int',
   ↪ target_dimension: 'int', provider_name: 'str', exactness: 'Exactness', point: 'tuple[Any, ...]', tolerance:
   ↪ 'float', dense_coefficient_count: 'int', capture_mode: 'CaptureMode', source_digest: 'str', metadata_sha256:
   ↪ 'str', chunk_size: 'int', evaluations_completed: 'int', coefficients_completed: 'int',
   ↪ maximum_omitted_basis_norm: 'float', chunks: 'tuple[PacketJetStreamChunkRecord, ...]', capture_certificate:
   ↪ 'Certificate | None', stream_sha256: 'str') -> None
```

Definition. Validated metadata and progress for one chunked packet-jet stream.

Contract (D, C, H, R, X). Chunked streams use fixed schema identifiers and version 1 envelopes. Writers require a destination, a positive chunk size, and explicit overwrite or resume policy. Direct capture writes to a partial directory and promotes it only after completion. Readers reject duplicate JSON keys, unsupported schemas, inconsistent ranges, digest mismatches, and incomplete streams unless incomplete access is explicitly allowed. Principal public operations and properties are `complete`, `order`, `metadata_dict`, `to_payload`, `from_payload`.

Guarantees (E, P, B, K). Each committed chunk has a payload digest and a manifest range. Resume continues only from a verified committed prefix. Selection and evaluation can operate with memory bounded by the configured chunk size. Loading a conventional snapshot requires a complete stream and an explicit coefficient ceiling. Digests provide integrity against accidental corruption, not authentication against an adversary able to replace the entire stream and its hashes.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_nonlinear_snapshots_persistence.py` and `tests/test_block11_streaming_migrations.py`. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 43.

C.5B · `PacketJetStreamSelection`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.PacketJetStreamSelection`. Reflected implementation `komorebi_math.nonlinear.streaming.PacketJetStreamSelection`. The same runtime object is also available through `komorebi_math.PacketJetStreamSelection`, `komorebi_math.beta.PacketJetStreamSelection`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-STREAMING`.

Manifest signature (s).

```
(coefficient_start: 'int', coefficient_stop: 'int', coefficients: 'tuple[PacketJetCoefficient, ...]',
   ↪  complete_snapshot: 'bool', certificate: 'Certificate') -> None
```

Definition. A bounded coefficient selection loaded from a packet-jet stream.

Contract (D, C, H, R, X). Chunked streams use fixed schema identifiers and version 1 envelopes. Writers require a destination, a positive chunk size, and explicit overwrite or resume policy. Direct capture writes to a partial directory and promotes it only after completion. Readers reject duplicate JSON keys, unsupported schemas, inconsistent ranges, digest mismatches, and incomplete streams unless incomplete access is explicitly allowed.

Guarantees (E, P, B, K). Each committed chunk has a payload digest and a manifest range. Resume continues only from a verified committed prefix. Selection and evaluation can operate with memory bounded by the configured chunk size. Loading a conventional snapshot requires a complete stream and an explicit coefficient ceiling. Digests provide integrity against accidental corruption, not authentication against an adversary able to replace the entire stream and its hashes.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_nonlinear_snapshots_persistence.py` and `tests/test_block11_streaming_migrations.py`. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 43.

C.5B · `PacketJetStreamWriteResult`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.PacketJetStreamWriteResult`. Reflected implementation `komorebi_math.nonlinear.streaming.PacketJetStreamWriteResult`. The same runtime object is also available through `komorebi_math.PacketJetStreamWriteResult`, `komorebi_math.beta.PacketJetStreamWriteResult`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-STREAMING`.

Manifest signature (`s`).

```
(destination: 'Path', active_path: 'Path', completed: 'bool', resumed: 'bool', chunks_written: 'int',
   ↪ evaluations_written: 'int', coefficients_written: 'int', manifest: 'PacketJetStreamManifest', certificate:
   ↪ 'Certificate') -> None
```

Definition. Progress returned by a direct or snapshot stream write.

Contract (`D, C, H, R, X`). Chunked streams use fixed schema identifiers and version 1 envelopes. Writers require a destination, a positive chunk size, and explicit overwrite or resume policy. Direct capture writes to a partial directory and promotes it only after completion. Readers reject duplicate JSON keys, unsupported schemas, inconsistent ranges, digest mismatches, and incomplete streams unless incomplete access is explicitly allowed.

Guarantees (`E, P, B, K`). Each committed chunk has a payload digest and a manifest range. Resume continues only from a verified committed prefix. Selection and evaluation can operate with memory bounded by the configured chunk size. Loading a conventional snapshot requires a complete stream and an explicit coefficient ceiling. Digests provide integrity against accidental corruption, not authentication against an adversary able to replace the entire stream and its hashes.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_nonlinear_snapshots_persistence.py` and `tests/test_block11_streaming_migrations.py`. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 43.

C.5B · `PacketJetSupport`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.nonlinear.PacketJetSupport`. Reflected implementation `komorebi_math.nonlinear.jets.PacketJetSupport`. The same runtime object is also available through `komorebi_math.beta.PacketJetSupport`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-PACKET-JETS`.

Manifest signature (`s`).

```
(order: 'int', nonzero: 'tuple[PacketJetAddress, ...]', zero: 'tuple[PacketJetAddress, ...]', untested:
   ↪ 'tuple[PacketJetAddress, ...]', evaluations: 'int', certificate: 'Certificate') -> None
```

Definition. Finite basis test for zero and nonzero packet derivative blocks.

Contract (`D, C, H, R, X`). Packet jets require a derivative provider, compatible finite source and target packet systems, a valid base point, and a positive supported derivative order. Packet labels must resolve inside their systems. Evaluation requires one source direction per input address. Support discovery and Taylor assembly also require positive finite tolerances and explicit evaluation budgets. Principal public operations and properties are `complete`, `contains`.

Guarantees (`E, P, B, K`). A packet jet evaluates the declared derivative after source projection and target projection. A complete family reconstructs the represented finite derivative when both packet systems are complete. Support classifications are exhaustive only when the evaluation budget covers every required packet basis probe. Taylor certificates compare one finite truncation with the observed map value at the supplied increment. Address counts and support work grow combinatorially with order and packet count.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_nonlinear_jets_composition.py` and `tests/test_nonlinear_execution.py`. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 38.

C.5B · PacketLabel

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.PacketLabel`. Reflected implementation `types.UnionType`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The public export name maps to implementation object `UnionType`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `data`. Profile `C5-PACKET-JETS`.

Manifest signature (s).

```
null
```

Definition. Public union type for packet indices or packet names.

Contract (D, C, H, R, X). Packet jets require a derivative provider, compatible finite source and target packet systems, a valid base point, and a positive supported derivative order. Packet labels must resolve inside their systems. Evaluation requires one source direction per input address. Support discovery and Taylor assembly also require positive finite tolerances and explicit evaluation budgets.

Guarantees (E, P, B, K). A packet jet evaluates the declared derivative after source projection and target projection. A complete family reconstructs the represented finite derivative when both packet systems are complete. Support classifications are exhaustive only when the evaluation budget covers every required packet basis probe. Taylor certificates compare one finite truncation with the observed map value at the supplied increment. Address counts and support work grow combinatorially with order and packet count.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_nonlinear_jets_composition.py` and `tests/test_nonlinear_execution.py`. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 38.

C.5B · PacketTaylorResult

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.PacketTaylorResult`. Reflected implementation `komorebi_math.nonlinear.jets.PacketTaylorResult`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-PACKET-JETS`.

Manifest signature (s).

```
(approximation: 'Any', observed: 'Any', truncation_residual: 'float', order_certificates: 'tuple[Certificate,
   ↪  ...]', certificate: 'Certificate') -> None
```

Definition. A truncated Taylor value assembled from reconstructed packet jets.

Contract (D, C, H, R, X). Packet jets require a derivative provider, compatible finite source and target packet systems, a valid base point, and a positive supported derivative order. Packet labels must resolve inside their systems. Evaluation requires one source direction per input address. Support discovery and Taylor assembly also require positive finite tolerances and explicit evaluation budgets.

Guarantees (E, P, B, K). A packet jet evaluates the declared derivative after source projection and target projection. A complete family reconstructs the represented finite derivative when both packet systems are complete. Support classifications are exhaustive only when the evaluation budget covers every required packet basis probe. Taylor certificates compare one finite truncation with the observed map value at the supplied increment. Address counts and support work grow combinatorially with order and packet count.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_nonlinear_jets_composition.py` and `tests/test_nonlinear_execution.py`. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation

ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 38.

C.5B · PacketTreeResult

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.PacketTreeResult`. Reflected implementation `komorebi_math.nonlinear.trees.PacketTreeResult`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-ROOTED-TREES`.

Manifest signature (s).

```
(value: 'Any', packet_values: 'tuple[Any, ...]', evaluated_blocks: 'int', pruned_blocks: 'int', certificate:
   ↪ 'Certificate') -> None
```

Definition. Packet-labeled rooted-tree evaluation with pruning statistics.

Contract (D, C, H, R, X). Ordered-tree enumeration requires internal arities of at least two, nonnegative height or positive leaf targets, and an explicit tree budget. Polynomial fixed-point problems require a finite seed, compatible multilinear operations, and declared exactness and tolerance. Majorant routines require finite nonnegative norm bounds and a positive candidate radius.

Guarantees (E, P, B, K). Tree counts and finite enumerations are exact within the requested boundary. A tree expansion equals the matching finite Picard iterate under the implemented polynomial operations. A convergence certificate is issued only when the supplied scalar majorant maps the declared ball into itself and has contraction factor below one. Packet reconstruction prunes only addresses covered by supplied zero-support certificates. Enumeration can grow exponentially or faster with height and arity.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_nonlinear_trees.py`. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 39.

C.5B · PolynomialFixedPointProblem

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.PolynomialFixedPointProblem`. Reflected implementation `komorebi_math.nonlinear.trees.PolynomialFixedPointProblem`. The same runtime object is also available through `komorebi_math.PolynomialFixedPointProblem`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-ROOTED-TREES`.

Manifest signature (s).

```
(seed: 'Any', operations: 'Mapping[int, MultilinearOperation]', *, exactness: 'Exactness' = <Exactness.NUMERICAL:
   ↪ 'numerical'>, tolerance: 'float' = 1e-10, name: 'str' = 'polynomial fixed-point problem') -> 'None'
```

Definition. A finite-dimensional equation `u = s + sum_r B_r(u,...,u)`.

Contract (D, C, H, R, X). Ordered-tree enumeration requires internal arities of at least two, nonnegative height or positive leaf targets, and an explicit tree budget. Polynomial fixed-point problems require a finite seed, compatible multilinear operations, and declared exactness and tolerance. Majorant routines require finite nonnegative norm bounds and a positive candidate radius. Principal public operations and properties are `operation`, `map`, `residual`, `picard`, `tree_value`, `tree_expansion`, `convergence_certificate`.

Guarantees (E, P, B, K). Tree counts and finite enumerations are exact within the requested boundary. A tree expansion equals the matching finite Picard iterate under the implemented polynomial operations. A convergence certificate is issued only when the supplied scalar majorant maps the declared ball into itself and has contraction factor below one. Packet reconstruction prunes only addresses covered by supplied zero-support certificates. Enumeration can grow

exponentially or faster with height and arity. The provenance registry records `PSC-III-T14A` as Finite polynomial rooted-tree expansion. Its recorded status is `finite certificate`.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_nonlinear_trees.py`. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 39.

C.5B · `RuntimeProfileSample`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.nonlinear.RuntimeProfileSample`. Reflected implementation `komorebi_math.nonlinear.profiling.RuntimeProfileSample`. The same runtime object is also available through `komorebi_math.RuntimeProfileSample`, `komorebi_math.beta.RuntimeProfileSample`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-PROFILING`.

Manifest signature (`s`).

```
(duration_ns: 'int', traced_peak_bytes: 'int') -> None
```

Definition. One observed duration and Python allocation peak.

Contract (`D, C, H, R, X`). Complexity estimation requires a finite family and a compatible optional support record or execution plan. Runtime profiles require positive repetition counts, nonnegative warmup counts, and a precisely stated measurement scope. Packet-jet profiling may include planning, execution, snapshot capture, and optional stream writing under explicit budgets.

Guarantees (`E, P, B, K`). Deterministic estimate fields are exact finite counts derived from the represented dimensions, packets, addresses, and plan. Timing and traced Python allocation samples are environment-dependent observations. They are not asymptotic evidence and do not include unreported device allocation or external process cost. Reports retain the interpreter, platform, scope, repetitions, and certificate needed to interpret the measurements.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_block11_golden_profiling_beta.py` and `tests/test_nonlinear_execution.py`. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 43.

C.5B · `RuntimeStageProfile`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.nonlinear.RuntimeStageProfile`. Reflected implementation `komorebi_math.nonlinear.profiling.RuntimeStageProfile`. The same runtime object is also available through `komorebi_math.RuntimeStageProfile`, `komorebi_math.beta.RuntimeStageProfile`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-PROFILING`.

Manifest signature (`s`).

```
(name: 'str', scope: 'str', warmup: 'int', samples: 'tuple[RuntimeProfileSample, ...]') -> None
```

Definition. Repeated observations for one precisely named runtime stage.

Contract (`D, C, H, R, X`). Complexity estimation requires a finite family and a compatible optional support record or execution plan. Runtime profiles require positive repetition counts, nonnegative warmup counts, and a precisely stated measurement scope. Packet-jet profiling may include planning, execution, snapshot capture, and optional stream writing under explicit budgets. Principal public operations and properties are `repetitions`, `minimum_duration_ns`, `median_duration_ns`, `maximum_duration_ns`, `median_peak_bytes`, `maximum_peak_bytes`, `to_dict`.

Guarantees (`E, P, B, K`). Deterministic estimate fields are exact finite counts derived from the represented dimensions, packets, addresses, and plan. Timing and traced Python allocation samples are environment-dependent observa-

tions. They are not asymptotic evidence and do not include unreported device allocation or external process cost. Reports retain the interpreter, platform, scope, repetitions, and certificate needed to interpret the measurements.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_block11_golden_profiling_beta.py` and `tests/test_nonlinear_execution.py`. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 43.

C.5B · SparsePolynomialDerivativeProvider

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.SparsePolynomialDerivativeProvider`. Reflected implementation `komorebi_math.nonlinear.sparse.SparsePolynomialDerivativeProvider`. The same runtime object is also available through `komorebi_math.SparsePolynomialDerivativeProvider`, `komorebi_math.beta.SparsePolynomialDerivativeProvider`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-SPARSE-TENSORS`.

Manifest signature (s).

```
(tensors: 'Mapping[int, SparseSymmetricTensor | Mapping[tuple[int, ...], Any] | Any]', *, source_dimension: 'int |
    ↪ None' = None, target_dimension: 'int | None' = None, exactness: 'Exactness' = <Exactness.NUMERICAL:
    ↪ 'numerical'>, tolerance: 'float' = 1e-10, name: 'str' = 'sparse polynomial derivative provider') -> 'None'
```

Definition. Polynomial derivatives evaluated from sparse symmetric tensor orbits.

Contract (D, C, H, R, X). Sparse tensor records require nonnegative output indices, canonical nondecreasing source-index tuples, positive finite dimensions, and coefficients compatible with the selected exactness model. A sparse polynomial provider requires one finite tensor per declared derivative order and common source and target dimensions. Principal public operations and properties are `from_dense_provider`, `degree`, `stored_entry_count`, `dense_entry_count`, `storage_ratio`, `supports_order`, `value`, `derivative`, `to_dense_provider`, `storage_certificate`, `to_dict`.

Guarantees (E, P, B, K). Orbit storage represents symmetric ordered tensor entries without storing every permutation. Contraction and dense reconstruction agree with the implemented orbit expansion. Exact object arithmetic remains exact when requested and supported. Numerical compression uses the declared tolerance. Storage savings depend on symmetry and sparsity, while dense materialization still requires the full ambient tensor size. The provenance registry records `PSC-III-D04P` as Sparse polynomial derivative provider. Its recorded status is `definition`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_nonlinear_sparse_autodiff.py`. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 38.

C.5B · SparseSymmetricTensor

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.SparseSymmetricTensor`. Reflected implementation `komorebi_math.nonlinear.sparse.SparseSymmetricTensor`. The same runtime object is also available through `komorebi_math.SparseSymmetricTensor`, `komorebi_math.beta.SparseSymmetricTensor`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-SPARSE-TENSORS`.

Manifest signature (s).

```
(order: 'int', source_dimension: 'int', target_dimension: 'int', entries: 'tuple[SparseTensorEntry, ...]' = ()) ->
    ↪ None
```

Definition. One finite symmetric tensor stored by source-index orbits.

Contract (D, C, H, R, X). Sparse tensor records require nonnegative output indices, canonical nondecreasing source-index tuples, positive finite dimensions, and coefficients compatible with the selected exactness model. A sparse polynomial provider requires one finite tensor per declared derivative order and common source and target dimensions. Principal public operations and properties are `from_mapping`, `from_dense`, `stored_entry_count`, `dense_entry_count`, `expanded_nonzero_count`, `storage_ratio`, `contract`, `to_dense`, `certificate`, `to_dict`.

Guarantees (E, P, B, K). Orbit storage represents symmetric ordered tensor entries without storing every permutation. Contraction and dense reconstruction agree with the implemented orbit expansion. Exact object arithmetic remains exact when requested and supported. Numerical compression uses the declared tolerance. Storage savings depend on symmetry and sparsity, while dense materialization still requires the full ambient tensor size. The provenance registry records `PSC-III-D04S` as Sparse symmetric derivative tensor. Its recorded status is `definition`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_nonlinear_sparse_autodiff.py`. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 38.

C.5B · SparseTensorEntry

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.SparseTensorEntry`. Reflected implementation `komorebi_math.nonlinear.sparse.SparseTensorEntry`. The same runtime object is also available through `komorebi_math.SparseTensorEntry`, `komorebi_math.beta.SparseTensorEntry`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-SPARSE-TENSORS`.

Manifest signature (s).

```
(output: 'int', inputs: 'tuple[int, ...]', value: 'Any') -> None
```

Definition. One coefficient of a symmetric tensor orbit.

Contract (D, C, H, R, X). Sparse tensor records require nonnegative output indices, canonical nondecreasing source-index tuples, positive finite dimensions, and coefficients compatible with the selected exactness model. A sparse polynomial provider requires one finite tensor per declared derivative order and common source and target dimensions.

Guarantees (E, P, B, K). Orbit storage represents symmetric ordered tensor entries without storing every permutation. Contraction and dense reconstruction agree with the implemented orbit expansion. Exact object arithmetic remains exact when requested and supported. Numerical compression uses the declared tolerance. Storage savings depend on symmetry and sparsity, while dense materialization still requires the full ambient tensor size.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_nonlinear_sparse_autodiff.py`. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 38.

C.5B · StreamedPacketJetEvaluation

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.StreamedPacketJetEvaluation`. Reflected implementation `komorebi_math.nonlinear.streaming.StreamedPacketJetEvaluation`. The same runtime object is also available through `komorebi_math.StreamedPacketJetEvaluation`, `komorebi_math.beta.StreamedPacketJetEvaluation`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-STREAMING`.

Manifest signature (s).

```
(value: 'np.ndarray', coefficient_start: 'int', coefficient_stop: 'int', total_coefficients: 'int',
   ↪ complete_snapshot: 'bool', certificate: 'Certificate') -> None
```

Definition. One chunk-wise packet-jet evaluation with an explicit coverage record.

Contract (D, C, H, R, X). Chunked streams use fixed schema identifiers and version 1 envelopes. Writers require a destination, a positive chunk size, and explicit overwrite or resume policy. Direct capture writes to a partial directory and promotes it only after completion. Readers reject duplicate JSON keys, unsupported schemas, inconsistent ranges, digest mismatches, and incomplete streams unless incomplete access is explicitly allowed.

Guarantees (E, P, B, K). Each committed chunk has a payload digest and a manifest range. Resume continues only from a verified committed prefix. Selection and evaluation can operate with memory bounded by the configured chunk size. Loading a conventional snapshot requires a complete stream and an explicit coefficient ceiling. Digests provide integrity against accidental corruption, not authentication against an adversary able to replace the entire stream and its hashes.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_nonlinear_snapshots_persistence.py` and `tests/test_block11_streaming_migrations.py`. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 43.

C.5B · TreeExpansionResult

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.TreeExpansionResult`. Reflected implementation `komorebi_math.nonlinear.trees.TreeExpansionResult`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-ROOTED-TREES`.

Manifest signature (s).

```
(value: 'Any', picard_value: 'Any', terms: 'tuple[TreeTerm, ...]', depth: 'int', certificate: 'Certificate') ->
   ↪ None
```

Definition. Finite rooted-tree expansion compared with the matching Picard iterate.

Contract (D, C, H, R, X). Ordered-tree enumeration requires internal arities of at least two, nonnegative height or positive leaf targets, and an explicit tree budget. Polynomial fixed-point problems require a finite seed, compatible multilinear operations, and declared exactness and tolerance. Majorant routines require finite nonnegative norm bounds and a positive candidate radius.

Guarantees (E, P, B, K). Tree counts and finite enumerations are exact within the requested boundary. A tree expansion equals the matching finite Picard iterate under the implemented polynomial operations. A convergence certificate is issued only when the supplied scalar majorant maps the declared ball into itself and has contraction factor below one. Packet reconstruction prunes only addresses covered by supplied zero-support certificates. Enumeration can grow exponentially or faster with height and arity.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_nonlinear_trees.py`. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity

for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 39.

C.5B · `TreeTerm`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.nonlinear.TreeTerm`. Reflected implementation `komorebi_math.nonlinear.trees.TreeTerm`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C5-ROOTED-TREES`.

Manifest signature (`s`).

```
(tree: 'OrderedRootedTree', value: 'Any') -> None
```

Definition. One ordered rooted tree and its evaluated contribution.

Contract (`D, C, H, R, X`). Ordered-tree enumeration requires internal arities of at least two, nonnegative height or positive leaf targets, and an explicit tree budget. Polynomial fixed-point problems require a finite seed, compatible multilinear operations, and declared exactness and tolerance. Majorant routines require finite nonnegative norm bounds and a positive candidate radius.

Guarantees (`E, P, B, K`). Tree counts and finite enumerations are exact within the requested boundary. A tree expansion equals the matching finite Picard iterate under the implemented polynomial operations. A convergence certificate is issued only when the supplied scalar majorant maps the declared ball into itself and has contraction factor below one. Packet reconstruction prunes only addresses covered by supplied zero-support certificates. Enumeration can grow exponentially or faster with height and arity.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_nonlinear_trees.py`. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 39.

C.5B · `capture_packet_jet_stream`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.nonlinear.capture_packet_jet_stream`. Reflected implementation `komorebi_math.nonlinear.streaming.capture_packet_jet_stream`. The same runtime object is also available through `komorebi_math.capture_packet_jet_stream`, `komorebi_math.beta.capture_packet_jet_stream`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C5-STREAMING`.

Manifest signature (`s`).

```
(jet: 'PacketJet', path: 'str | os.PathLike[str]', *, tolerance: 'float | None' = None, max_evaluations: 'int' =
    ↪ 10000000, chunk_size: 'int' = 1024, evaluation_budget: 'int | None' = None, resume: 'bool' = False,
    ↪ overwrite: 'bool' = False) -> 'PacketJetStreamWriteResult'
```

Definition. Capture a packet jet directly into bounded, resumable basis chunks.

Contract (`D, C, H, R, X`). Chunked streams use fixed schema identifiers and version 1 envelopes. Writers require a destination, a positive chunk size, and explicit overwrite or resume policy. Direct capture writes to a partial directory and promotes it only after completion. Readers reject duplicate JSON keys, unsupported schemas, inconsistent ranges, digest mismatches, and incomplete streams unless incomplete access is explicitly allowed.

Guarantees (`E, P, B, K`). Each committed chunk has a payload digest and a manifest range. Resume continues only from a verified committed prefix. Selection and evaluation can operate with memory bounded by the configured chunk size. Loading a conventional snapshot requires a complete stream and an explicit coefficient ceiling. Digests provide integrity against accidental corruption, not authentication against an adversary able to replace the entire stream and its hashes. The provenance registry records `PSC-III-D60` as Chunked resumable packet-jet capture. Its recorded status is `finite certificate`. Recorded limitations are that integrity digests are not signatures and that resuming assumes the provider behavior still agrees with its recorded identity.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_nonlinear_snapshots_persistence.py` and `tests/test_block11_streaming_migrations.py`. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 43.

C.5B · `catalan_number`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.catalan_number`. Reflected implementation `komorebi_math.nonlinear.trees.catalan_number`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C5-ROOTED-TREES`.

Manifest signature (s).

```
(index: 'int') -> 'int'
```

Definition. Return the exact Catalan number `C_index`.

Contract (D, C, H, R, X). Ordered-tree enumeration requires internal arities of at least two, nonnegative height or positive leaf targets, and an explicit tree budget. Polynomial fixed-point problems require a finite seed, compatible multilinear operations, and declared exactness and tolerance. Majorant routines require finite nonnegative norm bounds and a positive candidate radius.

Guarantees (E, P, B, K). Tree counts and finite enumerations are exact within the requested boundary. A tree expansion equals the matching finite Picard iterate under the implemented polynomial operations. A convergence certificate is issued only when the supplied scalar majorant maps the declared ball into itself and has contraction factor below one. Packet reconstruction prunes only addresses covered by supplied zero-support certificates. Enumeration can grow exponentially or faster with height and arity.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_nonlinear_trees.py`. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 39.

C.5B · `compile_packet_jet_plan`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.compile_packet_jet_plan`. Reflected implementation `komorebi_math.nonlinear.execution.compile_packet_jet_plan`. The same runtime object is also available through `komorebi_math.compile_packet_jet_plan, komorebi_math.beta.compile_packet_jet_plan`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C5-EXECUTION`.

Manifest signature (s).

```
(family: 'PacketJetFamily', support: 'PacketJetSupport | None' = None, *, max_addresses: 'int' = 1000000) ->
    ↪ 'PacketJetExecutionPlan'
```

Definition. Compile a projection-cached plan, pruning only certified zero blocks.

Contract (D, C, H, R, X). Execution plans require a finite packet-jet family and an optional support record for the same order and packet names. Only blocks certified zero may be pruned. Plan compilation enforces an explicit address budget. Execution requires one direction per derivative slot and checks that the plan still matches the family. CuPy execution additionally requires an importable CuPy runtime and a CUDA device.

Guarantees (E, P, B, K). The CPU planner groups active output blocks by common projected inputs, caches those projections, and reports derivative-call and projection counts. Reconstruction is compared with the direct represented derivative. Optional CuPy classes preserve the same finite packet contract when the hardware boundary is avail-

able. Runtime discovery alone is not a performance or correctness qualification, and unavailable device paths fail explicitly.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_nonlinear_execution.py` and `tests/test_block10_execution_persistence.py`. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 43.

C.5B · `cupy_runtime_status`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.cupy_runtime_status`. Reflected implementation `komorebi_math.nonlinear.gpu.cupy_runtime_status`. The same runtime object is also available through `komorebi_math.cupy_runtime_status`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C5-GPU`.

Manifest signature (s).

```
() -> 'CuPyRuntimeStatus'
```

Definition. Inspect CuPy and CUDA availability without raising on absent hardware.

Contract (D, C, H, R, X). GPU status inspection never requires hardware and returns the observed CuPy import and CUDA device state. Snapshot execution requires a complete finite snapshot, a working CuPy CUDA runtime, and compatible probe shapes. Validation compares the same persisted coefficients on CPU and device under a declared tolerance.

Guarantees (E, P, B, K). A successful validation record reports the probe count, maximum residual, runtime state, and certificate. The device executor evaluates only the captured finite snapshot and does not accept arbitrary Python callables. No speed claim follows from API presence. In a runtime without CuPy or a CUDA device, status reporting remains available while execution and validation are not hardware-qualified.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_block16_gpu_qualification.py` and `tests/test_nonlinear_execution.py`. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 43. The recorded audit runtime had no CuPy CUDA device, so GPU evidence is limited to status reporting, explicit unavailability, and CPU-side contract checks.

C.5B · `estimate_packet_jet_complexity`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.estimate_packet_jet_complexity`. Reflected implementation `komorebi_math.nonlinear.profiling.estimate_packet_jet_complexity`. The same runtime object is also available through `komorebi_math.estimate_packet_jet_complexity`, `komorebi_math.beta.estimate_packet_jet_complexity`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C5-PROFILING`.

Manifest signature (s).

```
(family: 'PacketJetFamily', support: 'PacketJetSupport | None' = None, *, plan: 'PacketJetExecutionPlan | None' =
    ↪ None, max_addresses: 'int' = 1000000) -> 'PacketJetComplexityEstimate'
```

Definition. Return exact finite operation counts for a packet-jet family.

Contract (D, C, H, R, X). Complexity estimation requires a finite family and a compatible optional support record or execution plan. Runtime profiles require positive repetition counts, nonnegative warmup counts, and a precisely stated measurement scope. Packet-jet profiling may include planning, execution, snapshot capture, and optional stream writing under explicit budgets.

Guarantees (E, P, B, K). Deterministic estimate fields are exact finite counts derived from the represented dimensions, packets, addresses, and plan. Timing and traced Python allocation samples are environment-dependent observations. They are not asymptotic evidence and do not include unreported device allocation or external process cost. Reports retain the interpreter, platform, scope, repetitions, and certificate needed to interpret the measurements.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_block11_golden_profiling_beta.py` and `tests/test_nonlinear_execution.py`. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 43.

C.5B · execute_packet_jet_plan

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.execute_packet_jet_plan`. Reflected implementation `komorebi_math.nonlinear.execution.execute_packet_jet_plan`. The same runtime object is also available through `komorebi_math.execute_packet_jet_plan`, `komorebi_math.beta.execute_packet_jet_plan`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C5-EXECUTION`.

Manifest signature (s).

```
(plan: 'PacketJetExecutionPlan', family: 'PacketJetFamily', directions: 'Sequence[Any]', *, tolerance: 'float |
    ↪ None' = None) -> 'PacketJetBatchResult'
```

Definition. Execute a packet-jet plan with cached input projections.

Contract (D, C, H, R, X). Execution plans require a finite packet-jet family and an optional support record for the same order and packet names. Only blocks certified zero may be pruned. Plan compilation enforces an explicit address budget. Execution requires one direction per derivative slot and checks that the plan still matches the family. CuPy execution additionally requires an importable CuPy runtime and a CUDA device.

Guarantees (E, P, B, K). The CPU planner groups active output blocks by common projected inputs, caches those projections, and reports derivative-call and projection counts. Reconstruction is compared with the direct represented derivative. Optional CuPy classes preserve the same finite packet contract when the hardware boundary is available. Runtime discovery alone is not a performance or correctness qualification, and unavailable device paths fail explicitly.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_nonlinear_execution.py` and `tests/test_block10_execution_persistence.py`. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 43.

C.5B · execute_packet_jets

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.execute_packet_jets`. Reflected implementation `komorebi_math.nonlinear.execution.execute_packet_jets`. The same runtime object is also available through `komorebi_math.execute_packet_jets`, `komorebi_math.beta.execute_packet_jets`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C5-EXECUTION`.

Manifest signature (s).

```
(family: 'PacketJetFamily', directions: 'Sequence[Any]', *, support: 'PacketJetSupport | None' = None, tolerance:
   ↪  'float | None' = None, max_addresses: 'int' = 1000000) -> 'PacketJetBatchResult'
```

Definition. Compile and execute one projection-cached packet-jet family.

Contract (D, C, H, R, X). Execution plans require a finite packet-jet family and an optional support record for the same order and packet names. Only blocks certified zero may be pruned. Plan compilation enforces an explicit address budget. Execution requires one direction per derivative slot and checks that the plan still matches the family. CuPy execution additionally requires an importable CuPy runtime and a CUDA device.

Guarantees (E, P, B, K). The CPU planner groups active output blocks by common projected inputs, caches those projections, and reports derivative-call and projection counts. Reconstruction is compared with the direct represented derivative. Optional CuPy classes preserve the same finite packet contract when the hardware boundary is available. Runtime discovery alone is not a performance or correctness qualification, and unavailable device paths fail explicitly.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_nonlinear_execution.py` and `tests/test_block10_execution_persistence.py`. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 43.

C.5B · `load_packet_jet_snapshot_stream`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.load_packet_jet_snapshot_stream`. Reflected implementation `komorebi_math.nonlinear.streaming.load_packet_jet_snapshot_stream`. The same runtime object is also available through `komorebi_math.load_packet_jet_snapshot_stream`, `komorebi_math.beta.load_packet_jet_snapshot_stream`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C5-STREAMING`.

Manifest signature (s).

```
(path: 'str | os.PathLike[str]', *, max_coefficients: 'int' = 1000000, verify_chunks: 'bool' = True) ->
   ↪  'PacketJetSnapshot'
```

Definition. Load a complete stream into a conventional in-memory snapshot.

Contract (D, C, H, R, X). Chunked streams use fixed schema identifiers and version 1 envelopes. Writers require a destination, a positive chunk size, and explicit overwrite or resume policy. Direct capture writes to a partial directory and promotes it only after completion. Readers reject duplicate JSON keys, unsupported schemas, inconsistent ranges, digest mismatches, and incomplete streams unless incomplete access is explicitly allowed.

Guarantees (E, P, B, K). Each committed chunk has a payload digest and a manifest range. Resume continues only from a verified committed prefix. Selection and evaluation can operate with memory bounded by the configured chunk size. Loading a conventional snapshot requires a complete stream and an explicit coefficient ceiling. Digests provide integrity against accidental corruption, not authentication against an adversary able to replace the entire stream and its hashes.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_nonlinear_snapshots_persistence.py` and `tests/test_block11_streaming_migrations.py`. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 43.

C.5B · majorant_convergence

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.majorant_convergence`. Reflected implementation `komorebi_math.nonlinear.trees.majorant_convergence`. The same runtime object is also available through `komorebi_math.majorant_convergence`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C5-ROOTED-TREES`.

Manifest signature (s).

```
(seed_norm: 'float', operation_norms: 'Mapping[int, float]', radius: 'float', *, tolerance: 'float' = 1e-12) ->
    ↪ 'MajorantConvergence'
```

Definition. Validate `phi(R)<=R` and `sum r*b_r*R^(r-1)<1`.

Contract (D, C, H, R, X). Ordered-tree enumeration requires internal arities of at least two, nonnegative height or positive leaf targets, and an explicit tree budget. Polynomial fixed-point problems require a finite seed, compatible multilinear operations, and declared exactness and tolerance. Majorant routines require finite nonnegative norm bounds and a positive candidate radius.

Guarantees (E, P, B, K). Tree counts and finite enumerations are exact within the requested boundary. A tree expansion equals the matching finite Picard iterate under the implemented polynomial operations. A convergence certificate is issued only when the supplied scalar majorant maps the declared ball into itself and has contraction factor below one. Packet reconstruction prunes only addresses covered by supplied zero-support certificates. Enumeration can grow exponentially or faster with height and arity. The provenance registry records `PSC-III-T14` as Polynomial rooted-tree convergence certificate. Its recorded status is `proved`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_nonlinear_trees.py`. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 39.

C.5B · nonlinear_cupy_available

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.nonlinear_cupy_available`. Reflected implementation `komorebi_math.nonlinear.execution.nonlinear_cupy_available`. The same runtime object is also available through `komorebi_math.nonlinear_cupy_available`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C5-EXECUTION`.

Manifest signature (s).

```
() -> 'bool'
```

Definition. Report whether a CuPy module can be discovered.

Contract (D, C, H, R, X). Execution plans require a finite packet-jet family and an optional support record for the same order and packet names. Only blocks certified zero may be pruned. Plan compilation enforces an explicit address budget. Execution requires one direction per derivative slot and checks that the plan still matches the family. CuPy execution additionally requires an importable CuPy runtime and a CUDA device.

Guarantees (E, P, B, K). The CPU planner groups active output blocks by common projected inputs, caches those projections, and reports derivative-call and projection counts. Reconstruction is compared with the direct represented derivative. Optional CuPy classes preserve the same finite packet contract when the hardware boundary is available. Runtime discovery alone is not a performance or correctness qualification, and unavailable device paths fail explicitly.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_nonlinear_execution.py` and `tests/test_block10_execution_persistence.py`. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear

execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 43.

C.5B · open_packet_jet_snapshot_stream

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.open_packet_jet_snapshot_stream`. Reflected implementation `komorebi_math.nonlinear.streaming.open_packet_jet_snapshot_stream`. The same runtime object is also available through `komorebi_math.open_packet_jet_snapshot_stream`, `komorebi_math.beta.open_packet_jet_snapshot_stream`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C5-STREAMING`.

Manifest signature (s).

```
(path: 'str | os.PathLike[str]', *, allow_incomplete: 'bool' = False, verify_chunks: 'bool' = False) ->
    ↪ 'PacketJetSnapshotStreamReader'
```

Definition. Open one packet-jet stream with explicit incomplete-stream policy.

Contract (D, C, H, R, X). Chunked streams use fixed schema identifiers and version 1 envelopes. Writers require a destination, a positive chunk size, and explicit overwrite or resume policy. Direct capture writes to a partial directory and promotes it only after completion. Readers reject duplicate JSON keys, unsupported schemas, inconsistent ranges, digest mismatches, and incomplete streams unless incomplete access is explicitly allowed.

Guarantees (E, P, B, K). Each committed chunk has a payload digest and a manifest range. Resume continues only from a verified committed prefix. Selection and evaluation can operate with memory bounded by the configured chunk size. Loading a conventional snapshot requires a complete stream and an explicit coefficient ceiling. Digests provide integrity against accidental corruption, not authentication against an adversary able to replace the entire stream and its hashes.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_nonlinear_snapshots_persistence.py` and `tests/test_block11_streaming_migrations.py`. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 43.

C.5B · ordered_rooted_trees

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.ordered_rooted_trees`. Reflected implementation `komorebi_math.nonlinear.trees.ordered_rooted_trees`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C5-ROOTED-TREES`.

Manifest signature (s).

```
(max_height: 'int', arities: 'Sequence[int]', *, max_trees: 'int' = 100000) -> 'tuple[OrderedRootedTree, ...]'
```

Definition. Enumerate all ordered rooted trees up to a height bound.

Contract (D, C, H, R, X). Ordered-tree enumeration requires internal arities of at least two, nonnegative height or positive leaf targets, and an explicit tree budget. Polynomial fixed-point problems require a finite seed, compatible multilinear operations, and declared exactness and tolerance. Majorant routines require finite nonnegative norm bounds and a positive candidate radius.

Guarantees (E, P, B, K). Tree counts and finite enumerations are exact within the requested boundary. A tree expansion equals the matching finite Picard iterate under the implemented polynomial operations. A convergence certificate is issued only when the supplied scalar majorant maps the declared ball into itself and has contraction factor below one. Packet reconstruction prunes only addresses covered by supplied zero-support certificates. Enumeration can grow exponentially or faster with height and arity.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_nonlinear_trees.py`. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 39.

C.5B · ordered_tree_count

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.ordered_tree_count`. Reflected implementation `komorebi_math.nonlinear.trees.ordered_tree_count`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C5-ROOTED-TREES`.

Manifest signature (s).

```
(max_height: 'int', arities: 'Sequence[int]') -> 'int'
```

Definition. Count trees whose height is at most `max_height`.

Contract (D, C, H, R, X). Ordered-tree enumeration requires internal arities of at least two, nonnegative height or positive leaf targets, and an explicit tree budget. Polynomial fixed-point problems require a finite seed, compatible multilinear operations, and declared exactness and tolerance. Majorant routines require finite nonnegative norm bounds and a positive candidate radius.

Guarantees (E, P, B, K). Tree counts and finite enumerations are exact within the requested boundary. A tree expansion equals the matching finite Picard iterate under the implemented polynomial operations. A convergence certificate is issued only when the supplied scalar majorant maps the declared ball into itself and has contraction factor below one. Packet reconstruction prunes only addresses covered by supplied zero-support certificates. Enumeration can grow exponentially or faster with height and arity.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_nonlinear_trees.py`. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 39.

C.5B · ordered_trees_with_leaves

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.ordered_trees_with_leaves`. Reflected implementation `komorebi_math.nonlinear.trees.ordered_trees_with_leaves`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C5-ROOTED-TREES`.

Manifest signature (s).

```
(leaves: 'int', arities: 'Sequence[int]', *, max_trees: 'int' = 100000) -> 'tuple[OrderedRootedTree, ...]'
```

Definition. Enumerate trees with exactly the requested number of leaves.

Contract (D, C, H, R, X). Ordered-tree enumeration requires internal arities of at least two, nonnegative height or positive leaf targets, and an explicit tree budget. Polynomial fixed-point problems require a finite seed, compatible multilinear operations, and declared exactness and tolerance. Majorant routines require finite nonnegative norm bounds and a positive candidate radius.

Guarantees (E, P, B, K). Tree counts and finite enumerations are exact within the requested boundary. A tree expansion equals the matching finite Picard iterate under the implemented polynomial operations. A convergence certificate is issued only when the supplied scalar majorant maps the declared ball into itself and has contraction factor below one. Packet reconstruction prunes only addresses covered by supplied zero-support certificates. Enumeration can grow exponentially or faster with height and arity.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_nonlinear_trees.py`. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 39.

C.5B · packet_jet_benchmark_contract

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.packet_jet_benchmark_contract`. Reflected implementation `komorebi_math.nonlinear.execution.packet_jet_benchmark_contract`. The same runtime object is also available through `komorebi_math.packet_jet_benchmark_contract`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C5-EXECUTION`.

Manifest signature (s).

```
(family: 'PacketJetFamily', *, backend: 'str', timing_scope: 'str' = 'kernel') -> 'Any'
```

Definition. Create a matched benchmark contract without importing algorithms eagerly.

Contract (D, C, H, R, X). Execution plans require a finite packet-jet family and an optional support record for the same order and packet names. Only blocks certified zero may be pruned. Plan compilation enforces an explicit address budget. Execution requires one direction per derivative slot and checks that the plan still matches the family. CuPy execution additionally requires an importable CuPy runtime and a CUDA device.

Guarantees (E, P, B, K). The CPU planner groups active output blocks by common projected inputs, caches those projections, and reports derivative-call and projection counts. Reconstruction is compared with the direct represented derivative. Optional CuPy classes preserve the same finite packet contract when the hardware boundary is available. Runtime discovery alone is not a performance or correctness qualification, and unavailable device paths fail explicitly.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_nonlinear_execution.py` and `tests/test_block10_execution_persistence.py`. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 43.

C.5B · packet_jet_support

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.packet_jet_support`. Reflected implementation `komorebi_math.nonlinear.jets.packet_jet_support`. The same runtime object is also available through `komorebi_math.packet_jet_support`, `komorebi_math.beta.packet_jet_support`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C5-PACKET-JETS`.

Manifest signature (s).

```
(family: 'PacketJetFamily', *, tolerance: 'float | None' = None, max_evaluations: 'int' = 100000) ->
    ↪ 'PacketJetSupport'
```

Definition. Classify packet jet blocks by exhaustive packet-range basis tests.

Contract (D, C, H, R, X). Packet jets require a derivative provider, compatible finite source and target packet systems, a valid base point, and a positive supported derivative order. Packet labels must resolve inside their systems. Evaluation requires one source direction per input address. Support discovery and Taylor assembly also require positive finite tolerances and explicit evaluation budgets.

Guarantees (E, P, B, K). A packet jet evaluates the declared derivative after source projection and target projection. A complete family reconstructs the represented finite derivative when both packet systems are complete. Support classifications are exhaustive only when the evaluation budget covers every required packet basis probe. Taylor

certificates compare one finite truncation with the observed map value at the supplied increment. Address counts and support work grow combinatorially with order and packet count. The provenance registry records `PSC-III-C08` as Finite packet jet support test. Its recorded status is `finite certificate`.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_nonlinear_jets_composition.py` and `tests/test_nonlinear_execution.py`. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 38.

C.5B · packet_taylor

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.packet_taylor`. Reflected implementation `komorebi_math.nonlinear.jets.packet_taylor`. The same runtime object is also available through `komorebi_math.packet_taylor`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C5-PACKET-JETS`.

Manifest signature (s).

```
(provider: 'DerivativeProvider', point: 'Any', increment: 'Any', source: 'PacketSystem', target: 'PacketSystem',
    ↪ order: 'int', *, tolerance: 'float | None' = None) -> 'PacketTaylorResult'
```

Definition. Assemble a finite Taylor polynomial through packet jet reconstruction.

Contract (D, C, H, R, X). Packet jets require a derivative provider, compatible finite source and target packet systems, a valid base point, and a positive supported derivative order. Packet labels must resolve inside their systems. Evaluation requires one source direction per input address. Support discovery and Taylor assembly also require positive finite tolerances and explicit evaluation budgets.

Guarantees (E, P, B, K). A packet jet evaluates the declared derivative after source projection and target projection. A complete family reconstructs the represented finite derivative when both packet systems are complete. Support classifications are exhaustive only when the evaluation budget covers every required packet basis probe. Taylor certificates compare one finite truncation with the observed map value at the supplied increment. Address counts and support work grow combinatorially with order and packet count. The provenance registry records `PSC-III-C09` as Finite packet Taylor reconstruction. Its recorded status is `finite certificate`. Recorded limitation: no automatic bound on the omitted Taylor remainder.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_nonlinear_jets_composition.py` and `tests/test_nonlinear_execution.py`. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 38.

C.5B · packet_tree_reconstruction

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.packet_tree_reconstruction`. Reflected implementation `komorebi_math.nonlinear.trees.packet_tree_reconstruction`. The same runtime object is also available through `komorebi_math.packet_tree_reconstruction`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C5-ROOTED-TREES`.

Manifest signature (`s`).

```
(problem: 'PolynomialFixedPointProblem', tree: 'OrderedRootedTree', packets: 'PacketSystem', *, zero_support:
    ↪ 'Mapping[int, PacketJetSupport] | None' = None, tolerance: 'float | None' = None) -> 'PacketTreeResult'
```

Definition. Reconstruct one tree by summing packet labels at every vertex.

Contract (`D, C, H, R, X`). Ordered-tree enumeration requires internal arities of at least two, nonnegative height or positive leaf targets, and an explicit tree budget. Polynomial fixed-point problems require a finite seed, compatible multilinear operations, and declared exactness and tolerance. Majorant routines require finite nonnegative norm bounds and a positive candidate radius.

Guarantees (`E, P, B, K`). Tree counts and finite enumerations are exact within the requested boundary. A tree expansion equals the matching finite Picard iterate under the implemented polynomial operations. A convergence certificate is issued only when the supplied scalar majorant maps the declared ball into itself and has contraction factor below one. Packet reconstruction prunes only addresses covered by supplied zero-support certificates. Enumeration can grow exponentially or faster with height and arity. The provenance registry records `PSC-III-T16` as Packet-labeled rooted-tree reconstruction. Its recorded status is `proved`.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_nonlinear_trees.py`. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 39.

C.5B · `profile_operation`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.nonlinear.profile_operation`. Reflected implementation `komorebi_math.nonlinear.profiling.profile_operation`. The same runtime object is also available through `komorebi_math.profile_operation`, `komorebi_math.beta.profile_operation`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C5-PROFILING`.

Manifest signature (`s`).

```
(name: 'str', operation: 'Callable[[], T]', *, repetitions: 'int' = 5, warmup: 'int' = 1, scope: 'str' = 'Python
    ↪ call including allocations and synchronization performed by the callable') -> 'RuntimeStageProfile'
```

Definition. Observe one callable without interpreting timings as asymptotic evidence.

Contract (`D, C, H, R, X`). Complexity estimation requires a finite family and a compatible optional support record or execution plan. Runtime profiles require positive repetition counts, nonnegative warmup counts, and a precisely stated measurement scope. Packet-jet profiling may include planning, execution, snapshot capture, and optional stream writing under explicit budgets.

Guarantees (`E, P, B, K`). Deterministic estimate fields are exact finite counts derived from the represented dimensions, packets, addresses, and plan. Timing and traced Python allocation samples are environment-dependent observations. They are not asymptotic evidence and do not include unreported device allocation or external process cost. Reports retain the interpreter, platform, scope, repetitions, and certificate needed to interpret the measurements.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_block11_golden_profiling_beta.py` and `tests/test_nonlinear_execution.py`. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 43.

C.5B · profile_packet_jet_runtime

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.profile_packet_jet_runtime`. Reflected implementation `komorebi_math.nonlinear.profiling.profile_packet_jet_runtime`. The same runtime object is also available through `komorebi_math.profile_packet_jet_runtime`, `komorebi_math.beta.profile_packet_jet_runtime`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C5-PROFILING`.

Manifest signature (s).

```
(family: 'PacketJetFamily', directions: 'Sequence[Any]', *, support: 'PacketJetSupport | None' = None, jet:
   ↪ 'PacketJet | None' = None, repetitions: 'int' = 5, warmup: 'int' = 1, max_addresses: 'int' = 1000000,
   ↪ max_snapshot_evaluations: 'int' = 100000, include_stream_write: 'bool' = False, stream_chunk_size: 'int' =
   ↪ 1024) -> 'PacketJetProfileReport'
```

Definition. Profile selected packet-jet stages under one explicit local contract.

Contract (D, C, H, R, X). Complexity estimation requires a finite family and a compatible optional support record or execution plan. Runtime profiles require positive repetition counts, nonnegative warmup counts, and a precisely stated measurement scope. Packet-jet profiling may include planning, execution, snapshot capture, and optional stream writing under explicit budgets.

Guarantees (E, P, B, K). Deterministic estimate fields are exact finite counts derived from the represented dimensions, packets, addresses, and plan. Timing and traced Python allocation samples are environment-dependent observations. They are not asymptotic evidence and do not include unreported device allocation or external process cost. Reports retain the interpreter, platform, scope, repetitions, and certificate needed to interpret the measurements. The provenance registry records `PSC-III-A61` as Finite packet-jet execution profile. Its recorded status is `heuristic`. Recorded limitations are that timings depend on the environment and that tracemalloc observes Python-managed allocations rather than total device or process memory.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_block11_golden_profiling_beta.py` and `tests/test_nonlinear_execution.py`. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 43.

C.5B · quadratic_catalan_majorant

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.quadratic_catalan_majorant`. Reflected implementation `komorebi_math.nonlinear.trees.quadratic_catalan_majorant`. The same runtime object is also available through `komorebi_math.quadratic_catalan_majorant`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C5-ROOTED-TREES`.

Manifest signature (s).

```
(seed_norm: 'float', bilinear_norm: 'float', *, tolerance: 'float' = 1e-12) -> 'MajorantConvergence'
```

Definition. Construct the sharp small-root majorant for `u=s+B(u,u)`.

Contract (D, C, H, R, X). Ordered-tree enumeration requires internal arities of at least two, nonnegative height or positive leaf targets, and an explicit tree budget. Polynomial fixed-point problems require a finite seed, compatible multilinear operations, and declared exactness and tolerance. Majorant routines require finite nonnegative norm bounds and a positive candidate radius.

Guarantees (E, P, B, K). Tree counts and finite enumerations are exact within the requested boundary. A tree expansion equals the matching finite Picard iterate under the implemented polynomial operations. A convergence certificate is issued only when the supplied scalar majorant maps the declared ball into itself and has contraction factor below one. Packet reconstruction prunes only addresses covered by supplied zero-support certificates. Enumeration can grow exponentially or faster with height and arity.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_nonlinear_trees.py`. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 39.

C.5B · require

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.require`. Reflected implementation `komorebi_math.nonlinear.require`. No root, stable, or beta facade exports this domain-only name in the reference manifest. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C5-NAMESPACE`.

Manifest signature (s).

```
() -> None
```

Definition. Confirm that the implemented nonlinear calculus is available.

Contract (D, C, H, R, X). The namespace marker is the literal feature key used by the registry. The `require` route takes no arguments and returns successfully for this implemented package component. Neither contract imports optional GPU runtimes or changes global state.

Guarantees (E, P, B, K). The feature key is deterministic for the installed source tree. The availability check has constant auxiliary storage and makes no statement about optional devices, performance, or the validity of a particular nonlinear model.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_certificates_registry.py`. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 6.

C.5B · save_packet_jet_snapshot_stream

Identity (q, a, t, k). Public research-manifest route `komorebi_math.nonlinear.save_packet_jet_snapshot_stream`. Reflected implementation `komorebi_math.nonlinear.streaming.save_packet_jet_snapshot_stream`. The same runtime object is also available through `komorebi_math.save_packet_jet_snapshot_stream`, `komorebi_math.beta.save_packet_jet_snapshot_stream`. The `komorebi_math.nonlinear` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C5-STREAMING`.

Manifest signature (s).

```
(snapshot: 'PacketJetSnapshot', path: 'str | os.PathLike[str]', *, chunk_size: 'int' = 1024, coefficient_budget:
    ↪  'int | None' = None, resume: 'bool' = False, overwrite: 'bool' = False) -> 'PacketJetStreamWriteResult'
```

Definition. Write an existing snapshot as independently verified coefficient chunks.

Contract (D, C, H, R, X). Chunked streams use fixed schema identifiers and version 1 envelopes. Writers require a destination, a positive chunk size, and explicit overwrite or resume policy. Direct capture writes to a partial directory and promotes it only after completion. Readers reject duplicate JSON keys, unsupported schemas, inconsistent ranges, digest mismatches, and incomplete streams unless incomplete access is explicitly allowed.

Guarantees (E, P, B, K). Each committed chunk has a payload digest and a manifest range. Resume continues only from a verified committed prefix. Selection and evaluation can operate with memory bounded by the configured chunk size. Loading a conventional snapshot requires a complete stream and an explicit coefficient ceiling. Digests provide integrity against accidental corruption, not authentication against an adversary able to replace the entire stream and its hashes.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_nonlinear_snapshots_persistence.py` and `tests/test_block11_streaming_migratio`

ns.py. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 43.

C.5B · snapshot_packet_jet

Identity (q, a, t, k). Public research-manifest route komorebi_math.nonlinear.snapshot_packet_jet. Reflected implementation komorebi_math.nonlinear.snapshots.snapshot_packet_jet. The same runtime object is also available through komorebi_math.snapshot_packet_jet, komorebi_math.beta.snapshot_packet_jet. The komorebi_math.nonlinear route is governed by [RESEARCH] policy pre-beta-additive. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind function. Profile C5-SNAPSHOTS.

Manifest signature (s).

```
(jet: 'PacketJet', *, tolerance: 'float | None' = None, max_evaluations: 'int' = 100000) -> 'PacketJetSnapshot'
```

Definition. Capture one packet jet by exhaustive standard-basis evaluation.

Contract (D, C, H, R, X). A snapshot captures one fixed packet jet by evaluating the ambient standard-basis tuples through an explicit finite budget. The source and target dimensions, base point, address, provider name, exactness, and sparse coefficients are stored in the record. Snapshot evaluation requires direction vectors of the recorded source dimension.

Guarantees (E, P, B, K). The snapshot is provider-independent after capture and reconstructs the recorded multilinear block from its stored coefficients. Its comparison certificate checks matched evaluations against the source jet on supplied probes. Exact coefficients preserve exact arithmetic. Numerical coefficients use the stored tolerance and certificate. Dense capture cost grows as the source dimension raised to the jet order.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are tests/test_nonlinear_snapshots_persistence.py and tests/test_block10_execution_persistence.py. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapters 38 and 43.

C.5B · validate_cupy_snapshot

Identity (q, a, t, k). Public research-manifest route komorebi_math.nonlinear.validate_cupy_snapshot. Reflected implementation komorebi_math.nonlinear.gpu.validate_cupy_snapshot. The same runtime object is also available through komorebi_math.validate_cupy_snapshot. The komorebi_math.nonlinear route is governed by [RESEARCH] policy pre-beta-additive. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind function. Profile C5-GPU.

Manifest signature (s).

```
(snapshot: 'PacketJetSnapshot', probes: 'Sequence[Sequence[Any]]', *, tolerance: 'float' = 1e-09) ->
    ↪ 'CuPyValidationResult'
```

Definition. Compare matched CPU and CuPy evaluations on explicit probes.

Contract (D, C, H, R, X). GPU status inspection never requires hardware and returns the observed CuPy import and CUDA device state. Snapshot execution requires a complete finite snapshot, a working CuPy CUDA runtime, and compatible probe shapes. Validation compares the same persisted coefficients on CPU and device under a declared tolerance.

Guarantees (E, P, B, K). A successful validation record reports the probe count, maximum residual, runtime state, and certificate. The device executor evaluates only the captured finite snapshot and does not accept arbitrary Python callables. No speed claim follows from API presence. In a runtime without CuPy or a CUDA device, status reporting remains available while execution and validation are not hardware-qualified.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_block16_gpu_qualification.py` and `tests/test_nonlinear_execution.py`. The independent verifier for nonlinear execution exercises representative finite identities, rejection paths, implementation ownership, alias behavior, and persistence integrity for this contract family. The source validator for nonlinear execution checks the exact signature, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 43. The recorded audit runtime had no CuPy CUDA device, so GPU evidence is limited to status reporting, explicit unavailability, and CPU-side contract checks.

> **Nonlinear execution reference.** The 69 cards in C.5B cover 37 classes, 25 functions, and 7 data exports. Together with C.5A, they document all 149 entries in the nonlinear namespace. Their recorded audit includes packet-jet and persistence checks alongside manifest and runtime identity checks.

C.6 `komorebi_math.export`: 58 entries

The export namespace connects finite mathematical values to typed reports, readable renderings, controlled filesystem outputs, local LaTeX compilation, and strict portable reconstruction. These contracts deliberately keep presentation serialization separate from portable persistence. A readable JSON or Markdown export is not automatically a reconstruction format, while a portable envelope is not automatically a polished report.

Contract family	Cards
Typed document model	10
Presentation serialization and adapters	6
Text, Markdown, HTML, and LaTeX renderers	8
File writers, formats, and deterministic bundles	10
Local LaTeX compilation	6
Report builders	3
Portable codecs, migrations, envelopes, and atomic files	15
Total	**58**

The boundary is explicit. Ordinary renderer text is escaped, while equation source and `RawLatex` remain trusted input. File writers enforce destination policy but do not authenticate content. Portable digests detect changes under a fixed codec contract but do not provide encryption or a keyed signature. PDF creation depends on a supported local LaTeX engine.

C.6 · `CodeBlock`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.export.CodeBlock`. Reflected implementation `komorebi_math.export.model.CodeBlock`. No root, stable, beta, or versioned facade exports this domain-only name in the reference manifest. The `komorebi_math.export` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C6-DOCUMENT-MODEL`.

Manifest signature (s).

```
(code: 'str', language: 'str' = 'text', caption: 'str | None' = None) -> None
```

Definition. A literal code block.

Contract (D, C, H, R, X). Document records validate local structure at construction. Headings require nonempty text and levels from 1 through 6. Equations require nonempty LaTeX source. Tables must be rectangular and use one alignment entry per column when alignment is supplied. Lists must be nonempty. Figure paths are normalized, but file existence is checked only by a writer. `MathDocument` appends validated blocks in order and returns itself from its fluent construction methods.

Guarantees (E, P, B, K). The document model preserves block order and separates typed report structure from rendering. Block records are frozen, while the `MathDocument.blocks` list is a deliberate construction buffer. The model does

not prove the truth of prose, equations, or certificate claims. `RawLatex` is trusted source and bypasses prose escaping only in the LaTeX renderer.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_export_renderers.py` and `tests/test_export_files.py`. The independent verifier for export and persistence exercises representative construction, rendering, serialization, file-policy, digest, migration, compilation, and rejection paths for this contract family. The source validator for export and persistence checks the exact signature, implementation identity, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 59.

C.6 · `DocumentMetadata`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.export.DocumentMetadata`. Reflected implementation `komorebi_math.export.model.DocumentMetadata`. The same runtime object is also available through `komorebi_math.DocumentMetadata`, `komorebi_math.stable.DocumentMetadata`, `komorebi_math.v1.DocumentMetadata`, `komorebi_math.api.v1.DocumentMetadata`. The `komorebi_math.export` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C6-DOCUMENT-MODEL`.

Manifest signature (s).

```
(title: 'str' = 'Komorebi Math Export', author: 'str | None' = None, date: 'str | None' = None, abstract: 'str |
   ↪ None' = None, keywords: 'tuple[str, ...]' = ()) -> None
```

Definition. Metadata used by Markdown, HTML, and LaTeX document renderers.

Contract (D, C, H, R, X). Document records validate local structure at construction. Headings require nonempty text and levels from 1 through 6. Equations require nonempty LaTeX source. Tables must be rectangular and use one alignment entry per column when alignment is supplied. Lists must be nonempty. Figure paths are normalized, but file existence is checked only by a writer. `MathDocument` appends validated blocks in order and returns itself from its fluent construction methods.

Guarantees (E, P, B, K). The document model preserves block order and separates typed report structure from rendering. Block records are frozen, while the `MathDocument.blocks` list is a deliberate construction buffer. The model does not prove the truth of prose, equations, or certificate claims. `RawLatex` is trusted source and bypasses prose escaping only in the LaTeX renderer.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_export_renderers.py` and `tests/test_export_files.py`. The independent verifier for export and persistence exercises representative construction, rendering, serialization, file-policy, digest, migration, compilation, and rejection paths for this contract family. The source validator for export and persistence checks the exact signature, implementation identity, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 59.

C.6 · `Equation`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.export.Equation`. Reflected implementation `komorebi_math.export.model.Equation`. The same runtime object is also available through `komorebi_math.Equation`, `komorebi_math.stable.Equation`, `komorebi_math.v1.Equation`, `komorebi_math.api.v1.Equation`. The `komorebi_math.export` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C6-DOCUMENT-MODEL`.

Manifest signature (s).

```
(latex: 'str', label: 'str | None' = None, numbered: 'bool' = False, description: 'str | None' = None) -> None
```

Definition. One LaTeX mathematical expression.

Contract (D, C, H, R, X). Document records validate local structure at construction. Headings require nonempty text and levels from 1 through 6. Equations require nonempty LaTeX source. Tables must be rectangular and use one alignment entry per column when alignment is supplied. Lists must be nonempty. Figure paths are normalized, but

file existence is checked only by a writer. `MathDocument` appends validated blocks in order and returns itself from its fluent construction methods.

Guarantees (`E`, `P`, `B`, `K`). The document model preserves block order and separates typed report structure from rendering. Block records are frozen, while the `MathDocument.blocks` list is a deliberate construction buffer. The model does not prove the truth of prose, equations, or certificate claims. `RawLatex` is trusted source and bypasses prose escaping only in the LaTeX renderer.

Evidence and links (`V`, `L`). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_export_renderers.py` and `tests/test_export_files.py`. The independent verifier for export and persistence exercises representative construction, rendering, serialization, file-policy, digest, migration, compilation, and rejection paths for this contract family. The source validator for export and persistence checks the exact signature, implementation identity, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 59.

C.6 · `ExportBundleResult`

Identity (`q`, `a`, `t`, `k`). Public research-manifest route `komorebi_math.export.ExportBundleResult`. Reflected implementation `komorebi_math.export.writers.ExportBundleResult`. The same runtime object is also available through `komorebi_math.ExportBundleResult`. The `komorebi_math.export` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C6-WRITERS`.

Manifest signature (`s`).

```
(path: 'Path', members: 'tuple[str, ...]', formats: 'tuple[ExportFormat, ...]', manifest_path: 'Path',
   ↪ certificate: 'Certificate') -> None
```

Definition. A ZIP bundle containing several exports and a hash manifest.

Contract (`D`, `C`, `H`, `R`, `X`). Writers resolve one of the 11 public formats, enforce parent-directory and overwrite policy, localize figure files when needed, and install primary outputs through temporary files. YAML requires PyYAML. PDF export requires a supported local LaTeX engine. NPY and NPZ writes disable pickle. Multi-format export uses deterministic suffixes, while bundle export stages files, records SHA-256 values, normalizes ZIP timestamps, and adds `MANIFEST.sha256`. Principal public operations and properties are `valid`, `require`.

Guarantees (`E`, `P`, `B`, `K`). A valid result certificate establishes that the primary output exists, is nonempty, and matches the observed byte count. Companion paths record localized assets, generated TeX, or compiler logs. A bundle manifest supports later integrity replay, but it is not a keyed signature and does not authenticate the archive against replacement of both data and manifest. Writer certificates do not certify the mathematical truth of rendered content.

Evidence and links (`V`, `L`). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_export_files.py` and `tests/test_export_cli.py`. The independent verifier for export and persistence exercises representative construction, rendering, serialization, file-policy, digest, migration, compilation, and rejection paths for this contract family. The source validator for export and persistence checks the exact signature, implementation identity, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 60.

C.6 · `ExportFormat`

Identity (`q`, `a`, `t`, `k`). Public research-manifest route `komorebi_math.export.ExportFormat`. Reflected implementation `komorebi_math.export.writers.ExportFormat`. The same runtime object is also available through `komorebi_math.ExportFormat`. The `komorebi_math.export` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C6-WRITERS`.

Manifest signature (`s`).

```
(*values)
```

Definition. Enumeration of the 11 supported on-disk formats: JSON, YAML, CSV, TSV, text, Markdown, HTML, LaTeX, PDF, NPY, and NPZ.

Contract (`D, C, H, R, X`). Writers resolve one of the 11 public formats, enforce parent-directory and overwrite policy, localize figure files when needed, and install primary outputs through temporary files. YAML requires PyYAML. PDF export requires a supported local LaTeX engine. NPY and NPZ writes disable pickle. Multi-format export uses deterministic suffixes, while bundle export stages files, records SHA-256 values, normalizes ZIP timestamps, and adds `MANIFEST.sha256`.

Guarantees (`E, P, B, K`). A valid result certificate establishes that the primary output exists, is nonempty, and matches the observed byte count. Companion paths record localized assets, generated TeX, or compiler logs. A bundle manifest supports later integrity replay, but it is not a keyed signature and does not authenticate the archive against replacement of both data and manifest. Writer certificates do not certify the mathematical truth of rendered content.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_export_files.py` and `tests/test_export_cli.py`. The independent verifier for export and persistence exercises representative construction, rendering, serialization, file-policy, digest, migration, compilation, and rejection paths for this contract family. The source validator for export and persistence checks the exact signature, implementation identity, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 60.

C.6 · `ExportOptions`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.export.ExportOptions`. Reflected implementation `komorebi_math.export.writers.ExportOptions`. The same runtime object is also available through `komorebi_math.ExportOptions`. The `komorebi_math.export` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C6-WRITERS`.

Manifest signature (`s`).

```
(title: 'str' = 'Komorebi Math Export', author: 'str | None' = None, date: 'str | None' = None, abstract: 'str |
    ↪ None' = None, overwrite: 'bool' = False, create_parents: 'bool' = True, preserve_types: 'bool' = True,
    ↪ json_indent: 'int' = 2, include_certificate: 'bool' = True, keep_tex: 'bool' = True, html:
    ↪ 'HtmlRenderOptions' = <factory>, latex: 'LatexRenderOptions' = <factory>, compiler: 'LatexCompileOptions' =
    ↪ <factory>) -> None
```

Definition. Common export behavior and renderer options.

Contract (`D, C, H, R, X`). Writers resolve one of the 11 public formats, enforce parent-directory and overwrite policy, localize figure files when needed, and install primary outputs through temporary files. YAML requires PyYAML. PDF export requires a supported local LaTeX engine. NPY and NPZ writes disable pickle. Multi-format export uses deterministic suffixes, while bundle export stages files, records SHA-256 values, normalizes ZIP timestamps, and adds `MANIFEST.sha256`.

Guarantees (`E, P, B, K`). A valid result certificate establishes that the primary output exists, is nonempty, and matches the observed byte count. Companion paths record localized assets, generated TeX, or compiler logs. A bundle manifest supports later integrity replay, but it is not a keyed signature and does not authenticate the archive against replacement of both data and manifest. Writer certificates do not certify the mathematical truth of rendered content.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_export_files.py` and `tests/test_export_cli.py`. The independent verifier for export and persistence exercises representative construction, rendering, serialization, file-policy, digest, migration, compilation, and rejection paths for this contract family. The source validator for export and persistence checks the exact signature, implementation identity, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 60.

C.6 · ExportResult

Identity (q, a, t, k). Public research-manifest route `komorebi_math.export.ExportResult`. Reflected implementation `komorebi_math.export.writers.ExportResult`. The same runtime object is also available through `komorebi_math.ExportResult`. The `komorebi_math.export` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C6-WRITERS`.

Manifest signature (s).

```
(path: 'Path', format: 'ExportFormat', bytes_written: 'int', companion_files: 'tuple[Path, ...]', certificate:
    ↪ 'Certificate') -> None
```

Definition. One completed file export and its validation certificate.

Contract (D, C, H, R, X). Writers resolve one of the 11 public formats, enforce parent-directory and overwrite policy, localize figure files when needed, and install primary outputs through temporary files. YAML requires PyYAML. PDF export requires a supported local LaTeX engine. NPY and NPZ writes disable pickle. Multi-format export uses deterministic suffixes, while bundle export stages files, records SHA-256 values, normalizes ZIP timestamps, and adds `MANIFEST.sha256`. Principal public operations and properties are `valid`, `require`.

Guarantees (E, P, B, K). A valid result certificate establishes that the primary output exists, is nonempty, and matches the observed byte count. Companion paths record localized assets, generated TeX, or compiler logs. A bundle manifest supports later integrity replay, but it is not a keyed signature and does not authenticate the archive against replacement of both data and manifest. Writer certificates do not certify the mathematical truth of rendered content.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_export_files.py` and `tests/test_export_cli.py`. The independent verifier for export and persistence exercises representative construction, rendering, serialization, file-policy, digest, migration, compilation, and rejection paths for this contract family. The source validator for export and persistence checks the exact signature, implementation identity, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 60.

C.6 · Exportable

Identity (q, a, t, k). Public research-manifest route `komorebi_math.export.Exportable`. Reflected implementation `komorebi_math.export.serialization.Exportable`. No root, stable, beta, or versioned facade exports this domain-only name in the reference manifest. The `komorebi_math.export` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C6-SERIALIZATION`.

Manifest signature (s).

```
(*args, **kwargs)
```

Definition. Runtime-checkable protocol for objects that provide a recursively normalizable `to_export_data()` representation.

Contract (D, C, H, R, X). Presentation serialization accepts supported Python primitives, NumPy values, fractions, complex values, paths, enums, package result records, dataclasses, mappings, sequences, registered adapters, and objects implementing `Exportable`. Recursion is bounded by `max_depth`. Registered adapters are selected through the runtime type MRO. Decoding reconstructs only the safe built-in markers emitted by this layer and leaves unknown type markers as ordinary mappings. Principal public operations and properties are `to_export_data`.

Guarantees (E, P, B, K). The result is JSON-compatible presentation data. Type preservation is selective rather than a general object-reconstruction promise. Mapping keys become strings, sets are ordered by `repr`, and arbitrary class names in `__type__` are never imported or instantiated. The adapter registry is process-local and a copied registry returned by `export_adapters` cannot mutate the active registry.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_export_serialization.py`. The independent verifier for export and persistence exercises representative construction, rendering, serialization, file-policy, digest, migration, compilation, and rejection

paths for this contract family. The source validator for export and persistence checks the exact signature, implementation identity, cumulative counts, unique anchor, and preservation of earlier cards. See Chapters 59 and 60.

C.6 · Figure

Identity (q, a, t, k). Public research-manifest route `komorebi_math.export.Figure`. Reflected implementation `komorebi_math.export.model.Figure`. No root, stable, beta, or versioned facade exports this domain-only name in the reference manifest. The `komorebi_math.export` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C6-DOCUMENT-MODEL`.

Manifest signature (s).

```
(path: 'Path', caption: 'str | None' = None, label: 'str | None' = None, width: 'str' = '0.9\\textwidth') -> None
```

Definition. A figure file referenced by a document.

Contract (D, C, H, R, X). Document records validate local structure at construction. Headings require nonempty text and levels from 1 through 6. Equations require nonempty LaTeX source. Tables must be rectangular and use one alignment entry per column when alignment is supplied. Lists must be nonempty. Figure paths are normalized, but file existence is checked only by a writer. `MathDocument` appends validated blocks in order and returns itself from its fluent construction methods.

Guarantees (E, P, B, K). The document model preserves block order and separates typed report structure from rendering. Block records are frozen, while the `MathDocument.blocks` list is a deliberate construction buffer. The model does not prove the truth of prose, equations, or certificate claims. `RawLatex` is trusted source and bypasses prose escaping only in the LaTeX renderer.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_export_renderers.py` and `tests/test_export_files.py`. The independent verifier for export and persistence exercises representative construction, rendering, serialization, file-policy, digest, migration, compilation, and rejection paths for this contract family. The source validator for export and persistence checks the exact signature, implementation identity, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 59.

C.6 · Heading

Identity (q, a, t, k). Public research-manifest route `komorebi_math.export.Heading`. Reflected implementation `komorebi_math.export.model.Heading`. No root, stable, beta, or versioned facade exports this domain-only name in the reference manifest. The `komorebi_math.export` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C6-DOCUMENT-MODEL`.

Manifest signature (s).

```
(text: 'str', level: 'int' = 1, label: 'str | None' = None) -> None
```

Definition. One document heading.

Contract (D, C, H, R, X). Document records validate local structure at construction. Headings require nonempty text and levels from 1 through 6. Equations require nonempty LaTeX source. Tables must be rectangular and use one alignment entry per column when alignment is supplied. Lists must be nonempty. Figure paths are normalized, but file existence is checked only by a writer. `MathDocument` appends validated blocks in order and returns itself from its fluent construction methods.

Guarantees (E, P, B, K). The document model preserves block order and separates typed report structure from rendering. Block records are frozen, while the `MathDocument.blocks` list is a deliberate construction buffer. The model does not prove the truth of prose, equations, or certificate claims. `RawLatex` is trusted source and bypasses prose escaping only in the LaTeX renderer.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_export_renderers.py` and `tests/test_export_files.py`. The independent verifier for export and persistence exercises representative construction, rendering, serialization, file-policy, digest, migration, compilation, and rejection paths for this contract family. The source validator for export and persistence checks

the exact signature, implementation identity, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 59.

C.6 · HtmlRenderOptions

Identity (q, a, t, k). Public research-manifest route `komorebi_math.export.HtmlRenderOptions`. Reflected implementation `komorebi_math.export.renderers.HtmlRenderOptions`. No root, stable, beta, or versioned facade exports this domain-only name in the reference manifest. The `komorebi_math.export` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C6-RENDERERS`.

Manifest signature (s).

```
(include_mathjax: 'bool' = False, title: 'str | None' = None) -> None
```

Definition. Configuration for standalone HTML exports.

Contract (D, C, H, R, X). Renderers consume one `MathDocument` without modifying it. Ordinary prose, metadata, table cells, labels, and captions are escaped for the target format. Equation bodies and `RawLatex` remain trusted mathematical source. HTML can include an optional MathJax loader. LaTeX options validate document-class and package names before they enter the preamble.

Guarantees (E, P, B, K). Each renderer returns a complete text representation with the original block order. Markdown, HTML, text, and LaTeX outputs are semantically related but are not expected to be byte-identical. Rendering performs no filesystem write and no LaTeX compilation. The HTML MathJax option adds a network reference rather than bundling MathJax locally.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_export_renderers.py`. The independent verifier for export and persistence exercises representative construction, rendering, serialization, file-policy, digest, migration, compilation, and rejection paths for this contract family. The source validator for export and persistence checks the exact signature, implementation identity, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 60.

C.6 · LatexCompilation

Identity (q, a, t, k). Public research-manifest route `komorebi_math.export.LatexCompilation`. Reflected implementation `komorebi_math.export.latex.LatexCompilation`. No root, stable, beta, or versioned facade exports this domain-only name in the reference manifest. The `komorebi_math.export` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C6-LATEX-COMPILATION`.

Manifest signature (s).

```
(source_path: 'Path', pdf_path: 'Path | None', log_path: 'Path', engine: 'LatexEngine', commands:
   ↪ 'tuple[tuple[str, ...], ...]', return_codes: 'tuple[int, ...]', certificate: 'Certificate') -> None
```

Definition. Paths, logs, and certificate returned by a LaTeX compilation.

Contract (D, C, H, R, X). LaTeX compilation resolves an explicit engine or the first installed automatic candidate. Pass counts and timeouts must be positive. Job names must be simple file names. Shell escape is disabled by default, direct engines receive `-no-shell-escape`, and the `latexmk` wrapper rejects shell-escape requests. Source may be an existing `.tex` path or text written into the isolated output directory. Principal public operations and properties are `valid`, `require`.

Guarantees (E, P, B, K). The compilation record retains source, PDF, log, engine, commands, return codes, and a certificate. Success requires all invoked commands to return zero and a nonempty PDF to exist. Missing engines raise `BackendUnavailableError`. Compiler success is local environment evidence, not a proof that trusted raw LaTeX is safe or that every external asset remains available later.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_export_files.py` and `tests/test_export_cli.py`. The independent verifier for export and persistence exercises representative construction, rendering, serialization, file-policy, digest, migration,

compilation, and rejection paths for this contract family. The source validator for export and persistence checks the exact signature, implementation identity, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 60. The recorded audit runtime exposed pdfLaTeX, XeLaTeX, LuaLaTeX, and latexmk, but engine availability remains an environment observation rather than a package guarantee.

C.6 · `LatexCompileOptions`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.export.LatexCompileOptions`. Reflected implementation `komorebi_math.export.latex.LatexCompileOptions`. The same runtime object is also available through `komorebi_math.LatexCompileOptions`. The `komorebi_math.export` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C6-LATEX-COMPILATION`.

Manifest signature (`s`).

```
(engine: 'LatexEngine | str' = <LatexEngine.AUTO: 'auto'>, passes: 'int' = 2, timeout: 'float' = 120.0,
    ↪ shell_escape: 'bool' = False, keep_auxiliary: 'bool' = False, jobname: 'str | None' = None) -> None
```

Definition. Options for one isolated LaTeX compilation.

Contract (`D, C, H, R, X`). LaTeX compilation resolves an explicit engine or the first installed automatic candidate. Pass counts and timeouts must be positive. Job names must be simple file names. Shell escape is disabled by default, direct engines receive `-no-shell-escape`, and the `latexmk` wrapper rejects shell-escape requests. Source may be an existing `.tex` path or text written into the isolated output directory.

Guarantees (`E, P, B, K`). The compilation record retains source, PDF, log, engine, commands, return codes, and a certificate. Success requires all invoked commands to return zero and a nonempty PDF to exist. Missing engines raise `BackendUnavailableError`. Compiler success is local environment evidence, not a proof that trusted raw LaTeX is safe or that every external asset remains available later.

Evidence and links (`V, L`). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_export_files.py` and `tests/test_export_cli.py`. The independent verifier for export and persistence exercises representative construction, rendering, serialization, file-policy, digest, migration, compilation, and rejection paths for this contract family. The source validator for export and persistence checks the exact signature, implementation identity, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 60. The recorded audit runtime exposed pdfLaTeX, XeLaTeX, LuaLaTeX, and latexmk, but engine availability remains an environment observation rather than a package guarantee.

C.6 · `LatexEngine`

Identity (`q, a, t, k`). Public research-manifest route `komorebi_math.export.LatexEngine`. Reflected implementation `komorebi_math.export.latex.LatexEngine`. The same runtime object is also available through `komorebi_math.LatexEngine`. The `komorebi_math.export` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C6-LATEX-COMPILATION`.

Manifest signature (`s`).

```
(*values)
```

Definition. Enumeration of automatic, pdfLaTeX, XeLaTeX, LuaLaTeX, and latexmk compiler choices.

Contract (`D, C, H, R, X`). LaTeX compilation resolves an explicit engine or the first installed automatic candidate. Pass counts and timeouts must be positive. Job names must be simple file names. Shell escape is disabled by default, direct engines receive `-no-shell-escape`, and the `latexmk` wrapper rejects shell-escape requests. Source may be an existing `.tex` path or text written into the isolated output directory.

Guarantees (`E, P, B, K`). The compilation record retains source, PDF, log, engine, commands, return codes, and a certificate. Success requires all invoked commands to return zero and a nonempty PDF to exist. Missing engines raise `BackendUnavailableError`. Compiler success is local environment evidence, not a proof that trusted raw LaTeX is safe or that every external asset remains available later.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_export_files.py` and `tests/test_export_cli.py`. The independent verifier for export and persistence exercises representative construction, rendering, serialization, file-policy, digest, migration, compilation, and rejection paths for this contract family. The source validator for export and persistence checks the exact signature, implementation identity, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 60. The recorded audit runtime exposed pdfLaTeX, XeLaTeX, LuaLaTeX, and latexmk, but engine availability remains an environment observation rather than a package guarantee.

C.6 · `LatexRenderOptions`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.export.LatexRenderOptions`. Reflected implementation `komorebi_math.export.renderers.LatexRenderOptions`. The same runtime object is also available through `komorebi_math.LatexRenderOptions`. The `komorebi_math.export` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C6-RENDERERS`.

Manifest signature (s).

```
(document_class: 'str' = 'article', class_options: 'tuple[str, ...]' = ('11pt',), packages: 'tuple[str, ...]' =
    ↪ ('amsmath', 'amssymb', 'booktabs', 'longtable', 'array', 'graphicx', 'hyperref', 'geometry', 'microtype'),
    ↪ include_table_of_contents: 'bool' = False, number_sections: 'bool' = True, geometry: 'str' = 'margin=1in')
    ↪ -> None
```

Definition. Configuration for generated LaTeX documents.

Contract (D, C, H, R, X). Renderers consume one `MathDocument` without modifying it. Ordinary prose, metadata, table cells, labels, and captions are escaped for the target format. Equation bodies and `RawLatex` remain trusted mathematical source. HTML can include an optional MathJax loader. LaTeX options validate document-class and package names before they enter the preamble.

Guarantees (E, P, B, K). Each renderer returns a complete text representation with the original block order. Markdown, HTML, text, and LaTeX outputs are semantically related but are not expected to be byte-identical. Rendering performs no filesystem write and no LaTeX compilation. The HTML MathJax option adds a network reference rather than bundling MathJax locally.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_export_renderers.py`. The independent verifier for export and persistence exercises representative construction, rendering, serialization, file-policy, digest, migration, compilation, and rejection paths for this contract family. The source validator for export and persistence checks the exact signature, implementation identity, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 60.

C.6 · `ListBlock`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.export.ListBlock`. Reflected implementation `komorebi_math.export.model.ListBlock`. No root, stable, beta, or versioned facade exports this domain-only name in the reference manifest. The `komorebi_math.export` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C6-DOCUMENT-MODEL`.

Manifest signature (s).

```
(items: 'tuple[str, ...]', ordered: 'bool' = False) -> None
```

Definition. An ordered or unordered list.

Contract (D, C, H, R, X). Document records validate local structure at construction. Headings require nonempty text and levels from 1 through 6. Equations require nonempty LaTeX source. Tables must be rectangular and use one alignment entry per column when alignment is supplied. Lists must be nonempty. Figure paths are normalized, but file existence is checked only by a writer. `MathDocument` appends validated blocks in order and returns itself from its fluent construction methods.

Guarantees (E, P, B, K). The document model preserves block order and separates typed report structure from rendering. Block records are frozen, while the `MathDocument.blocks` list is a deliberate construction buffer. The model does

not prove the truth of prose, equations, or certificate claims. `RawLatex` is trusted source and bypasses prose escaping only in the LaTeX renderer.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_export_renderers.py` and `tests/test_export_files.py`. The independent verifier for export and persistence exercises representative construction, rendering, serialization, file-policy, digest, migration, compilation, and rejection paths for this contract family. The source validator for export and persistence checks the exact signature, implementation identity, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 59.

C.6 · `MathDocument`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.export.MathDocument`. Reflected implementation `komorebi_math.export.model.MathDocument`. The same runtime object is also available through `komorebi_math.MathDocument`, `komorebi_math.stable.MathDocument`, `komorebi_math.v1.MathDocument`, `komorebi_math.api.v1.MathDocument`. The `komorebi_math.export` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C6-DOCUMENT-MODEL`.

Manifest signature (s).

```
(metadata: 'DocumentMetadata' = <factory>, blocks: 'list[DocumentBlock]' = <factory>) -> None
```

Definition. A renderer independent mathematical document.

Contract (D, C, H, R, X). Document records validate local structure at construction. Headings require nonempty text and levels from 1 through 6. Equations require nonempty LaTeX source. Tables must be rectangular and use one alignment entry per column when alignment is supplied. Lists must be nonempty. Figure paths are normalized, but file existence is checked only by a writer. `MathDocument` appends validated blocks in order and returns itself from its fluent construction methods. Principal public operations and properties are `add`, `section`, `paragraph`, `equation`, `table`, `code`, `bullet_list`, `figure`.

Guarantees (E, P, B, K). The document model preserves block order and separates typed report structure from rendering. Block records are frozen, while the `MathDocument.blocks` list is a deliberate construction buffer. The model does not prove the truth of prose, equations, or certificate claims. `RawLatex` is trusted source and bypasses prose escaping only in the LaTeX renderer.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_export_renderers.py` and `tests/test_export_files.py`. The independent verifier for export and persistence exercises representative construction, rendering, serialization, file-policy, digest, migration, compilation, and rejection paths for this contract family. The source validator for export and persistence checks the exact signature, implementation identity, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 59.

C.6 · `PORTABLE_SCHEMA`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.export.PORTABLE_SCHEMA`. Reflected implementation `builtins.str`. The same runtime object is also available through `komorebi_math.stable.PORTABLE_SCHEMA`, `komorebi_math.beta.PORTABLE_SCHEMA`, `komorebi_math.v1.PORTABLE_SCHEMA`, `komorebi_math.api.v1.PORTABLE_SCHEMA`. The public export name maps to implementation object `str`. The `komorebi_math.export` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `data`. Profile `C6-PORTABLE-PERSISTENCE`.

Manifest signature (s).

```
null
```

Definition. Literal portable-envelope schema identifier `komorebi-math.portable`.

Contract (D, C, H, R, X). Portable persistence accepts finite JSON primitives, explicit structural tags, and registered codecs. Codec tags are nonempty and whitespace-free. Versions are positive integers. Migrations advance exactly one version at a time. The envelope has exactly five fields and binds the canonical encoded payload to SHA-256.

Loading rejects duplicate JSON keys, nonfinite constants, unknown schemas, unsupported versions, digest mismatches, malformed tags, missing migration edges, and depth-limit violations.

Guarantees (E, P, B, K). Canonical payload ordering makes the digest deterministic for a fixed codec contract and value. Decoding never imports a class named by untrusted data. `save_portable` writes and flushes a temporary sibling before `os.replace`, while `load_portable` delegates all validation to the envelope decoder. SHA-256 provides integrity detection, not encryption, secrecy, or authentication. Extension codecs remain dynamic process state unless separately frozen by a release contract.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_block15_persistence.py`, `tests/test_block10_execution_persistence.py`, `tests/test_block11_streaming_migrations.py`, and `tests/test_nonlinear_snapshots_persistence.py`. The independent verifier for export and persistence exercises representative construction, rendering, serialization, file-policy, digest, migration, compilation, and rejection paths for this contract family. The source validator for export and persistence checks the exact signature, implementation identity, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 61.

C.6 · `PORTABLE_SCHEMA_VERSION`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.export.PORTABLE_SCHEMA_VERSION`. Reflected implementation `builtins.int`. The same runtime object is also available through `komorebi_math.stable.PORTABLE_SCHEMA_VERSION`, `komorebi_math.beta.PORTABLE_SCHEMA_VERSION`, `komorebi_math.v1.PORTABLE_SCHEMA_VERSION`, `komorebi_math.api.v1.PORTABLE_SCHEMA_VERSION`. The public export name maps to implementation object `int`. The `komorebi_math.export` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `data`. Profile `C6-PORTABLE-PERSISTENCE`.

Manifest signature (s).

```
null
```

Definition. Current integer version of the portable-envelope schema. The committed value is `1`.

Contract (D, C, H, R, X). Portable persistence accepts finite JSON primitives, explicit structural tags, and registered codecs. Codec tags are nonempty and whitespace-free. Versions are positive integers. Migrations advance exactly one version at a time. The envelope has exactly five fields and binds the canonical encoded payload to SHA-256. Loading rejects duplicate JSON keys, nonfinite constants, unknown schemas, unsupported versions, digest mismatches, malformed tags, missing migration edges, and depth-limit violations.

Guarantees (E, P, B, K). Canonical payload ordering makes the digest deterministic for a fixed codec contract and value. Decoding never imports a class named by untrusted data. `save_portable` writes and flushes a temporary sibling before `os.replace`, while `load_portable` delegates all validation to the envelope decoder. SHA-256 provides integrity detection, not encryption, secrecy, or authentication. Extension codecs remain dynamic process state unless separately frozen by a release contract.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_block15_persistence.py`, `tests/test_block10_execution_persistence.py`, `tests/test_block11_streaming_migrations.py`, and `tests/test_nonlinear_snapshots_persistence.py`. The independent verifier for export and persistence exercises representative construction, rendering, serialization, file-policy, digest, migration, compilation, and rejection paths for this contract family. The source validator for export and persistence checks the exact signature, implementation identity, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 61.

C.6 · `Paragraph`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.export.Paragraph`. Reflected implementation `komorebi_math.export.model.Paragraph`. The same runtime object is also available through `komorebi_math.stable.Paragraph`, `komorebi_math.v1.Paragraph`, `komorebi_math.api.v1.Paragraph`. The `komorebi_math.export` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C6-DOCUMENT-MODEL`.

Manifest signature (s).

```
(text: 'str') -> None
```

Definition. One prose paragraph whose text is escaped by each renderer.

Contract (D, C, H, R, X). Document records validate local structure at construction. Headings require nonempty text and levels from 1 through 6. Equations require nonempty LaTeX source. Tables must be rectangular and use one alignment entry per column when alignment is supplied. Lists must be nonempty. Figure paths are normalized, but file existence is checked only by a writer. `MathDocument` appends validated blocks in order and returns itself from its fluent construction methods.

Guarantees (E, P, B, K). The document model preserves block order and separates typed report structure from rendering. Block records are frozen, while the `MathDocument.blocks` list is a deliberate construction buffer. The model does not prove the truth of prose, equations, or certificate claims. `RawLatex` is trusted source and bypasses prose escaping only in the LaTeX renderer.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_export_renderers.py` and `tests/test_export_files.py`. The independent verifier for export and persistence exercises representative construction, rendering, serialization, file-policy, digest, migration, compilation, and rejection paths for this contract family. The source validator for export and persistence checks the exact signature, implementation identity, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 59.

C.6 · `PortableCodec`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.export.PortableCodec`. Reflected implementation `komorebi_math.export.persistence.PortableCodec`. The same runtime object is also available through `komorebi_math.PortableCodec`, `komorebi_math.stable.PortableCodec`, `komorebi_math.beta.PortableCodec`, `komorebi_math.v1.PortableCodec`, `komorebi_math.api.v1.PortableCodec`. The `komorebi_math.export` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C6-PORTABLE-PERSISTENCE`.

Manifest signature (s).

```
(tag: 'str', version: 'int', type_: 'type[Any]', encoder: 'Encoder', decoder: 'Decoder') -> None
```

Definition. One explicit type codec in the portable persistence registry.

Contract (D, C, H, R, X). Portable persistence accepts finite JSON primitives, explicit structural tags, and registered codecs. Codec tags are nonempty and whitespace-free. Versions are positive integers. Migrations advance exactly one version at a time. The envelope has exactly five fields and binds the canonical encoded payload to SHA-256. Loading rejects duplicate JSON keys, nonfinite constants, unknown schemas, unsupported versions, digest mismatches, malformed tags, missing migration edges, and depth-limit violations.

Guarantees (E, P, B, K). Canonical payload ordering makes the digest deterministic for a fixed codec contract and value. Decoding never imports a class named by untrusted data. `save_portable` writes and flushes a temporary sibling before `os.replace`, while `load_portable` delegates all validation to the envelope decoder. SHA-256 provides integrity detection, not encryption, secrecy, or authentication. Extension codecs remain dynamic process state unless separately frozen by a release contract.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_block15_persistence.py`, `tests/test_block10_execution_persistence.py`, `tests/test_block11_streaming_migrations.py`, and `tests/test_nonlinear_snapshots_persistence.py`. The independent verifier for export and persistence exercises representative construction, rendering, serialization, file-policy, digest, migration, compilation, and rejection paths for this contract family. The source validator for export and persistence checks the exact signature, implementation identity, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 61.

C.6 · PortableMigration

Identity (q, a, t, k). Public research-manifest route `komorebi_math.export.PortableMigration`. Reflected implementation `komorebi_math.export.persistence.PortableMigration`. The same runtime object is also available through `komorebi_math.PortableMigration`, `komorebi_math.stable.PortableMigration`, `komorebi_math.beta.PortableMigration`, `komorebi_math.v1.PortableMigration`, `komorebi_math.api.v1.PortableMigration`. The `komorebi_math.export` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C6-PORTABLE-PERSISTENCE`.

Manifest signature (s).

```
(tag: 'str', from_version: 'int', to_version: 'int', migrator: 'Migrator') -> None
```

Definition. One explicit, adjacent codec-data migration.

Contract (D, C, H, R, X). Portable persistence accepts finite JSON primitives, explicit structural tags, and registered codecs. Codec tags are nonempty and whitespace-free. Versions are positive integers. Migrations advance exactly one version at a time. The envelope has exactly five fields and binds the canonical encoded payload to SHA-256. Loading rejects duplicate JSON keys, nonfinite constants, unknown schemas, unsupported versions, digest mismatches, malformed tags, missing migration edges, and depth-limit violations.

Guarantees (E, P, B, K). Canonical payload ordering makes the digest deterministic for a fixed codec contract and value. Decoding never imports a class named by untrusted data. `save_portable` writes and flushes a temporary sibling before `os.replace`, while `load_portable` delegates all validation to the envelope decoder. SHA-256 provides integrity detection, not encryption, secrecy, or authentication. Extension codecs remain dynamic process state unless separately frozen by a release contract.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_block15_persistence.py`, `tests/test_block10_execution_persistence.py`, `tests/test_block11_streaming_migrations.py`, and `tests/test_nonlinear_snapshots_persistence.py`. The independent verifier for export and persistence exercises representative construction, rendering, serialization, file-policy, digest, migration, compilation, and rejection paths for this contract family. The source validator for export and persistence checks the exact signature, implementation identity, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 61.

C.6 · RawLatex

Identity (q, a, t, k). Public research-manifest route `komorebi_math.export.RawLatex`. Reflected implementation `komorebi_math.export.model.RawLatex`. No root, stable, beta, or versioned facade exports this domain-only name in the reference manifest. The `komorebi_math.export` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C6-DOCUMENT-MODEL`.

Manifest signature (s).

```
(source: 'str') -> None
```

Definition. A deliberate trusted-source escape hatch inserted unchanged by the LaTeX renderer and shown as literal code by the other renderers.

Contract (D, C, H, R, X). Document records validate local structure at construction. Headings require nonempty text and levels from 1 through 6. Equations require nonempty LaTeX source. Tables must be rectangular and use one alignment entry per column when alignment is supplied. Lists must be nonempty. Figure paths are normalized, but file existence is checked only by a writer. `MathDocument` appends validated blocks in order and returns itself from its fluent construction methods.

Guarantees (E, P, B, K). The document model preserves block order and separates typed report structure from rendering. Block records are frozen, while the `MathDocument.blocks` list is a deliberate construction buffer. The model does not prove the truth of prose, equations, or certificate claims. `RawLatex` is trusted source and bypasses prose escaping only in the LaTeX renderer.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_export_renderers.py` and `tests/test_export_files.py`. The independent verifier for export and persistence exercises representative construction, rendering, serialization, file-policy, digest, migration, compilation, and rejection paths for this contract family. The source validator for export and persistence checks the exact signature, implementation identity, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 59.

C.6 · `Table`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.export.Table`. Reflected implementation `komorebi_math.export.model.Table`. The same runtime object is also available through `komorebi_math.Table`. The `komorebi_math.export` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `class`. Profile `C6-DOCUMENT-MODEL`.

Manifest signature (s).

```
(headers: 'tuple[str, ...]', rows: 'tuple[tuple[Any, ...], ...]', caption: 'str | None' = None, label: 'str | None'
   ↪  = None, alignment: 'tuple[str, ...] | None' = None) -> None
```

Definition. A rectangular table with optional caption and label.

Contract (D, C, H, R, X). Document records validate local structure at construction. Headings require nonempty text and levels from 1 through 6. Equations require nonempty LaTeX source. Tables must be rectangular and use one alignment entry per column when alignment is supplied. Lists must be nonempty. Figure paths are normalized, but file existence is checked only by a writer. `MathDocument` appends validated blocks in order and returns itself from its fluent construction methods. Principal public operations and properties are `from_rows`.

Guarantees (E, P, B, K). The document model preserves block order and separates typed report structure from rendering. Block records are frozen, while the `MathDocument.blocks` list is a deliberate construction buffer. The model does not prove the truth of prose, equations, or certificate claims. `RawLatex` is trusted source and bypasses prose escaping only in the LaTeX renderer.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_export_renderers.py` and `tests/test_export_files.py`. The independent verifier for export and persistence exercises representative construction, rendering, serialization, file-policy, digest, migration, compilation, and rejection paths for this contract family. The source validator for export and persistence checks the exact signature, implementation identity, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 59.

C.6 · `available_export_formats`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.export.available_export_formats`. Reflected implementation `komorebi_math.export.writers.available_export_formats`. The same runtime object is also available through `komorebi_math.available_export_formats`. The `komorebi_math.export` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C6-WRITERS`.

Manifest signature (s).

```
() -> 'tuple[str, ...]'
```

Definition. Return every supported public format name.

Contract (D, C, H, R, X). Writers resolve one of the 11 public formats, enforce parent-directory and overwrite policy, localize figure files when needed, and install primary outputs through temporary files. YAML requires PyYAML. PDF export requires a supported local LaTeX engine. NPY and NPZ writes disable pickle. Multi-format export uses deterministic suffixes, while bundle export stages files, records SHA-256 values, normalizes ZIP timestamps, and adds `MANIFEST.sha256`.

Guarantees (E, P, B, K). A valid result certificate establishes that the primary output exists, is nonempty, and matches the observed byte count. Companion paths record localized assets, generated TeX, or compiler logs. A bundle

manifest supports later integrity replay, but it is not a keyed signature and does not authenticate the archive against replacement of both data and manifest. Writer certificates do not certify the mathematical truth of rendered content.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_export_files.py` and `tests/test_export_cli.py`. The independent verifier for export and persistence exercises representative construction, rendering, serialization, file-policy, digest, migration, compilation, and rejection paths for this contract family. The source validator for export and persistence checks the exact signature, implementation identity, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 60.

C.6 · `available_latex_engines`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.export.available_latex_engines`. Reflected implementation `komorebi_math.export.latex.available_latex_engines`. The same runtime object is also available through `komorebi_math.available_latex_engines`. The `komorebi_math.export` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C6-LATEX-COMPILATION`.

Manifest signature (s).

```
() -> 'dict[str, bool]'
```

Definition. Report which supported LaTeX commands are available on PATH.

Contract (D, C, H, R, X). LaTeX compilation resolves an explicit engine or the first installed automatic candidate. Pass counts and timeouts must be positive. Job names must be simple file names. Shell escape is disabled by default, direct engines receive `-no-shell-escape`, and the `latexmk` wrapper rejects shell-escape requests. Source may be an existing `.tex` path or text written into the isolated output directory.

Guarantees (E, P, B, K). The compilation record retains source, PDF, log, engine, commands, return codes, and a certificate. Success requires all invoked commands to return zero and a nonempty PDF to exist. Missing engines raise `BackendUnavailableError`. Compiler success is local environment evidence, not a proof that trusted raw LaTeX is safe or that every external asset remains available later.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_export_files.py` and `tests/test_export_cli.py`. The independent verifier for export and persistence exercises representative construction, rendering, serialization, file-policy, digest, migration, compilation, and rejection paths for this contract family. The source validator for export and persistence checks the exact signature, implementation identity, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 60. The recorded audit runtime exposed pdfLaTeX, XeLaTeX, LuaLaTeX, and latexmk, but engine availability remains an environment observation rather than a package guarantee.

C.6 · `certificate_document`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.export.certificate_document`. Reflected implementation `komorebi_math.export.report.certificate_document`. No root, stable, beta, or versioned facade exports this domain-only name in the reference manifest. The `komorebi_math.export` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C6-REPORT-BUILDERS`.

Manifest signature (s).

```
(certificate: 'Certificate', *, title: 'str' = 'Certificate') -> 'MathDocument'
```

Definition. Build a compact document describing one certificate.

Contract (D, C, H, R, X). Report builders convert certificates, finite sequences, and common Python data into the typed document model. Existing `MathDocument` values pass through unchanged. Result records may contribute their certificate when `include_certificate` is enabled. Mappings, short arrays, sequences, and dataclasses are converted to tables when possible, while unsupported table shapes fall back to normalized JSON code.

Guarantees (E, P, B, K). The builders preserve visible validity, exactness, tolerances, residuals, assumptions, and diagnostics when those fields exist. They produce readable reports rather than portable reconstruction records. A report can summarize evidence, but creating the report does not independently validate the source object or certificate.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_export_renderers.py` and `tests/test_export_files.py`. The independent verifier for export and persistence exercises representative construction, rendering, serialization, file-policy, digest, migration, compilation, and rejection paths for this contract family. The source validator for export and persistence checks the exact signature, implementation identity, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 59.

C.6 · `compile_latex`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.export.compile_latex`. Reflected implementation `komorebi_math.export.latex.compile_latex`. The same runtime object is also available through `komorebi_math.compile_latex`. The `komorebi_math.export` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C6-LATEX-COMPILATION`.

Manifest signature (s).

```
(source: 'str | Path', output_dir: 'str | Path', *, options: 'LatexCompileOptions | None' = None, source_name:
    ↪ 'str' = 'document.tex') -> 'LatexCompilation'
```

Definition. Compile LaTeX source to PDF without enabling shell escape by default.

Contract (D, C, H, R, X). LaTeX compilation resolves an explicit engine or the first installed automatic candidate. Pass counts and timeouts must be positive. Job names must be simple file names. Shell escape is disabled by default, direct engines receive `-no-shell-escape`, and the `latexmk` wrapper rejects shell-escape requests. Source may be an existing `.tex` path or text written into the isolated output directory.

Guarantees (E, P, B, K). The compilation record retains source, PDF, log, engine, commands, return codes, and a certificate. Success requires all invoked commands to return zero and a nonempty PDF to exist. Missing engines raise `BackendUnavailableError`. Compiler success is local environment evidence, not a proof that trusted raw LaTeX is safe or that every external asset remains available later.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_export_files.py` and `tests/test_export_cli.py`. The independent verifier for export and persistence exercises representative construction, rendering, serialization, file-policy, digest, migration, compilation, and rejection paths for this contract family. The source validator for export and persistence checks the exact signature, implementation identity, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 60. The recorded audit runtime exposed pdfLaTeX, XeLaTeX, LuaLaTeX, and latexmk, but engine availability remains an environment observation rather than a package guarantee.

C.6 · `document_from_data`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.export.document_from_data`. Reflected implementation `komorebi_math.export.report.document_from_data`. The same runtime object is also available through `komorebi_math.document_from_data`. The `komorebi_math.export` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C6-REPORT-BUILDERS`.

Manifest signature (s).

```
(value: 'Any', *, title: 'str' = 'Komorebi Math Export', author: 'str | None' = None, date: 'str | None' = None,
    ↪ abstract: 'str | None' = None, include_certificate: 'bool' = True) -> 'MathDocument'
```

Definition. Create a readable mathematical document from common Python data.

Contract (D, C, H, R, X). Report builders convert certificates, finite sequences, and common Python data into the typed document model. Existing `MathDocument` values pass through unchanged. Result records may contribute their cer-

tificate when `include_certificate` is enabled. Mappings, short arrays, sequences, and dataclasses are converted to tables when possible, while unsupported table shapes fall back to normalized JSON code.

Guarantees (`E`, `P`, `B`, `K`). The builders preserve visible validity, exactness, tolerances, residuals, assumptions, and diagnostics when those fields exist. They produce readable reports rather than portable reconstruction records. A report can summarize evidence, but creating the report does not independently validate the source object or certificate.

Evidence and links (`V`, `L`). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_export_renderers.py` and `tests/test_export_files.py`. The independent verifier for export and persistence exercises representative construction, rendering, serialization, file-policy, digest, migration, compilation, and rejection paths for this contract family. The source validator for export and persistence checks the exact signature, implementation identity, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 59.

C.6 · `dumps_portable`

Identity (`q`, `a`, `t`, `k`). Public research-manifest route `komorebi_math.export.dumps_portable`. Reflected implementation `komorebi_math.export.persistence.dumps_portable`. The same runtime object is also available through `komorebi_math.dumps_portable`, `komorebi_math.stable.dumps_portable`, `komorebi_math.beta.dumps_portable`, `komorebi_math.v1.dumps_portable`, `komorebi_math.api.v1.dumps_portable`. The `komorebi_math.export` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C6-PORTABLE-PERSISTENCE`.

Manifest signature (`s`).

```
(value: 'Any', *, indent: 'int | None' = 2, max_depth: 'int' = 128) -> 'str'
```

Definition. Serialize one value into a complete digest-checked JSON envelope.

Contract (`D`, `C`, `H`, `R`, `X`). Portable persistence accepts finite JSON primitives, explicit structural tags, and registered codecs. Codec tags are nonempty and whitespace-free. Versions are positive integers. Migrations advance exactly one version at a time. The envelope has exactly five fields and binds the canonical encoded payload to SHA-256. Loading rejects duplicate JSON keys, nonfinite constants, unknown schemas, unsupported versions, digest mismatches, malformed tags, missing migration edges, and depth-limit violations.

Guarantees (`E`, `P`, `B`, `K`). Canonical payload ordering makes the digest deterministic for a fixed codec contract and value. Decoding never imports a class named by untrusted data. `save_portable` writes and flushes a temporary sibling before `os.replace`, while `load_portable` delegates all validation to the envelope decoder. SHA-256 provides integrity detection, not encryption, secrecy, or authentication. Extension codecs remain dynamic process state unless separately frozen by a release contract.

Evidence and links (`V`, `L`). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_block15_persistence.py`, `tests/test_block10_execution_persistence.py`, `tests/test_block11_streaming_migrations.py`, and `tests/test_nonlinear_snapshots_persistence.py`. The independent verifier for export and persistence exercises representative construction, rendering, serialization, file-policy, digest, migration, compilation, and rejection paths for this contract family. The source validator for export and persistence checks the exact signature, implementation identity, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 61.

C.6 · `export_adapters`

Identity (`q`, `a`, `t`, `k`). Public research-manifest route `komorebi_math.export.export_adapters`. Reflected implementation `komorebi_math.export.serialization.export_adapters`. No root, stable, beta, or versioned facade exports this domain-only name in the reference manifest. The `komorebi_math.export` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C6-SERIALIZATION`.

Manifest signature (s).

```
() -> 'dict[type[Any], ExportAdapter]'
```

Definition. Return a copy of the active export adapter registry.

Contract (D, C, H, R, X). Presentation serialization accepts supported Python primitives, NumPy values, fractions, complex values, paths, enums, package result records, dataclasses, mappings, sequences, registered adapters, and objects implementing `Exportable`. Recursion is bounded by `max_depth`. Registered adapters are selected through the runtime type MRO. Decoding reconstructs only the safe built-in markers emitted by this layer and leaves unknown type markers as ordinary mappings.

Guarantees (E, P, B, K). The result is JSON-compatible presentation data. Type preservation is selective rather than a general object-reconstruction promise. Mapping keys become strings, sets are ordered by `repr`, and arbitrary class names in `__type__` are never imported or instantiated. The adapter registry is process-local and a copied registry returned by `export_adapters` cannot mutate the active registry.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_export_serialization.py`. The independent verifier for export and persistence exercises representative construction, rendering, serialization, file-policy, digest, migration, compilation, and rejection paths for this contract family. The source validator for export and persistence checks the exact signature, implementation identity, cumulative counts, unique anchor, and preservation of earlier cards. See Chapters 59 and 60.

C.6 · `export_bundle`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.export.export_bundle`. Reflected implementation `komorebi_math.export.writers.export_bundle`. The same runtime object is also available through `komorebi_math.export_bundle`. The `komorebi_math.export` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C6-WRITERS`.

Manifest signature (s).

```
(value: 'Any', path: 'str | Path', *, base_name: 'str' = 'komorebi_export', formats: 'Sequence[ExportFormat | str]'
    ↪ = (<ExportFormat.JSON: 'json'>, <ExportFormat.MARKDOWN: 'markdown'>, <ExportFormat.LATEX: 'latex'>),
    ↪ options: 'ExportOptions | None' = None) -> 'ExportBundleResult'
```

Definition. Create a ZIP archive with several exports and a SHA256 manifest.

Contract (D, C, H, R, X). Writers resolve one of the 11 public formats, enforce parent-directory and overwrite policy, localize figure files when needed, and install primary outputs through temporary files. YAML requires PyYAML. PDF export requires a supported local LaTeX engine. NPY and NPZ writes disable pickle. Multi-format export uses deterministic suffixes, while bundle export stages files, records SHA-256 values, normalizes ZIP timestamps, and adds `MANIFEST.sha256`.

Guarantees (E, P, B, K). A valid result certificate establishes that the primary output exists, is nonempty, and matches the observed byte count. Companion paths record localized assets, generated TeX, or compiler logs. A bundle manifest supports later integrity replay, but it is not a keyed signature and does not authenticate the archive against replacement of both data and manifest. Writer certificates do not certify the mathematical truth of rendered content.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_export_files.py` and `tests/test_export_cli.py`. The independent verifier for export and persistence exercises representative construction, rendering, serialization, file-policy, digest, migration, compilation, and rejection paths for this contract family. The source validator for export and persistence checks the exact signature, implementation identity, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 60.

C.6 · export_data

Identity (q, a, t, k). Public research-manifest route `komorebi_math.export.export_data`. Reflected implementation `komorebi_math.export.writers.export_data`. The same runtime object is also available through `komorebi_math.export_data`. The `komorebi_math.export` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C6-WRITERS`.

Manifest signature (s).

```
(value: 'Any', path: 'str | Path', *, format: 'ExportFormat | str | None' = None, options: 'ExportOptions | None' =
    ↪ None) -> 'ExportResult'
```

Definition. Export one Python or Komorebi Math value to a selected file format.

Contract (D, C, H, R, X). Writers resolve one of the 11 public formats, enforce parent-directory and overwrite policy, localize figure files when needed, and install primary outputs through temporary files. YAML requires PyYAML. PDF export requires a supported local LaTeX engine. NPY and NPZ writes disable pickle. Multi-format export uses deterministic suffixes, while bundle export stages files, records SHA-256 values, normalizes ZIP timestamps, and adds `MANIFEST.sha256`.

Guarantees (E, P, B, K). A valid result certificate establishes that the primary output exists, is nonempty, and matches the observed byte count. Companion paths record localized assets, generated TeX, or compiler logs. A bundle manifest supports later integrity replay, but it is not a keyed signature and does not authenticate the archive against replacement of both data and manifest. Writer certificates do not certify the mathematical truth of rendered content.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_export_files.py` and `tests/test_export_cli.py`. The independent verifier for export and persistence exercises representative construction, rendering, serialization, file-policy, digest, migration, compilation, and rejection paths for this contract family. The source validator for export and persistence checks the exact signature, implementation identity, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 60.

C.6 · export_document

Identity (q, a, t, k). Public research-manifest route `komorebi_math.export.export_document`. Reflected implementation `komorebi_math.export.writers.export_document`. The same runtime object is also available through `komorebi_math.export_document`. The `komorebi_math.export` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C6-WRITERS`.

Manifest signature (s).

```
(document: 'MathDocument', path: 'str | Path', *, format: 'ExportFormat | str | None' = None, options:
    ↪ 'ExportOptions | None' = None) -> 'ExportResult'
```

Definition. Export one renderer independent mathematical document.

Contract (D, C, H, R, X). Writers resolve one of the 11 public formats, enforce parent-directory and overwrite policy, localize figure files when needed, and install primary outputs through temporary files. YAML requires PyYAML. PDF export requires a supported local LaTeX engine. NPY and NPZ writes disable pickle. Multi-format export uses deterministic suffixes, while bundle export stages files, records SHA-256 values, normalizes ZIP timestamps, and adds `MANIFEST.sha256`.

Guarantees (E, P, B, K). A valid result certificate establishes that the primary output exists, is nonempty, and matches the observed byte count. Companion paths record localized assets, generated TeX, or compiler logs. A bundle manifest supports later integrity replay, but it is not a keyed signature and does not authenticate the archive against replacement of both data and manifest. Writer certificates do not certify the mathematical truth of rendered content.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_export_files.py` and `tests/test_export_cli.py`. The independent verifier for export and persistence exercises representative construction, rendering, serialization, file-policy, digest, migration, compilation, and rejection paths for this contract family. The source validator for export and persistence checks the

exact signature, implementation identity, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 60.

C.6 · export_many

Identity (q, a, t, k). Public research-manifest route `komorebi_math.export.export_many`. Reflected implementation `komorebi_math.export.writers.export_many`. The same runtime object is also available through `komorebi_math.export_many`. The `komorebi_math.export` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C6-WRITERS`.

Manifest signature (s).

```
(value: 'Any', directory: 'str | Path', *, base_name: 'str', formats: 'Sequence[ExportFormat | str]', options:
    ↪ 'ExportOptions | None' = None) -> 'tuple[ExportResult, ...]'
```

Definition. Export one value into several formats inside one directory.

Contract (D, C, H, R, X). Writers resolve one of the 11 public formats, enforce parent-directory and overwrite policy, localize figure files when needed, and install primary outputs through temporary files. YAML requires PyYAML. PDF export requires a supported local LaTeX engine. NPY and NPZ writes disable pickle. Multi-format export uses deterministic suffixes, while bundle export stages files, records SHA-256 values, normalizes ZIP timestamps, and adds `MANIFEST.sha256`.

Guarantees (E, P, B, K). A valid result certificate establishes that the primary output exists, is nonempty, and matches the observed byte count. Companion paths record localized assets, generated TeX, or compiler logs. A bundle manifest supports later integrity replay, but it is not a keyed signature and does not authenticate the archive against replacement of both data and manifest. Writer certificates do not certify the mathematical truth of rendered content.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_export_files.py` and `tests/test_export_cli.py`. The independent verifier for export and persistence exercises representative construction, rendering, serialization, file-policy, digest, migration, compilation, and rejection paths for this contract family. The source validator for export and persistence checks the exact signature, implementation identity, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 60.

C.6 · from_portable_data

Identity (q, a, t, k). Public research-manifest route `komorebi_math.export.from_portable_data`. Reflected implementation `komorebi_math.export.persistence.from_portable_data`. The same runtime object is also available through `komorebi_math.stable.from_portable_data`, `komorebi_math.beta.from_portable_data`, `komorebi_math.v1.from_portable_data`, `komorebi_math.api.v1.from_portable_data`. The `komorebi_math.export` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C6-PORTABLE-PERSISTENCE`.

Manifest signature (s).

```
(value: 'Any', *, max_depth: 'int' = 128) -> 'Any'
```

Definition. Decode one previously encoded portable payload tree.

Contract (D, C, H, R, X). Portable persistence accepts finite JSON primitives, explicit structural tags, and registered codecs. Codec tags are nonempty and whitespace-free. Versions are positive integers. Migrations advance exactly one version at a time. The envelope has exactly five fields and binds the canonical encoded payload to SHA-256. Loading rejects duplicate JSON keys, nonfinite constants, unknown schemas, unsupported versions, digest mismatches, malformed tags, missing migration edges, and depth-limit violations.

Guarantees (E, P, B, K). Canonical payload ordering makes the digest deterministic for a fixed codec contract and value. Decoding never imports a class named by untrusted data. `save_portable` writes and flushes a temporary sibling before `os.replace`, while `load_portable` delegates all validation to the envelope decoder. SHA-256 provides integrity detection, not encryption, secrecy, or authentication. Extension codecs remain dynamic process state unless separately frozen by a release contract.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_block15_persistence.py`, `tests/test_block10_execution_persistence.py`, `tests/test_block11_streaming_migrations.py`, and `tests/test_nonlinear_snapshots_persistence.py`. The independent verifier for export and persistence exercises representative construction, rendering, serialization, file-policy, digest, migration, compilation, and rejection paths for this contract family. The source validator for export and persistence checks the exact signature, implementation identity, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 61.

C.6 · `from_serializable`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.export.from_serializable`. Reflected implementation `komorebi_math.export.serialization.from_serializable`. The same runtime object is also available through `komorebi_math.from_serializable`. The `komorebi_math.export` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C6-SERIALIZATION`.

Manifest signature (s).

```
(value: 'Any', *, max_depth: 'int' = 64, _depth: 'int' = 0) -> 'Any'
```

Definition. Decode the safe built-in type markers emitted by `to_serializable`.

Contract (D, C, H, R, X). Presentation serialization accepts supported Python primitives, NumPy values, fractions, complex values, paths, enums, package result records, dataclasses, mappings, sequences, registered adapters, and objects implementing `Exportable`. Recursion is bounded by `max_depth`. Registered adapters are selected through the runtime type MRO. Decoding reconstructs only the safe built-in markers emitted by this layer and leaves unknown type markers as ordinary mappings.

Guarantees (E, P, B, K). The result is JSON-compatible presentation data. Type preservation is selective rather than a general object-reconstruction promise. Mapping keys become strings, sets are ordered by `repr`, and arbitrary class names in `__type__` are never imported or instantiated. The adapter registry is process-local and a copied registry returned by `export_adapters` cannot mutate the active registry.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_export_serialization.py`. The independent verifier for export and persistence exercises representative construction, rendering, serialization, file-policy, digest, migration, compilation, and rejection paths for this contract family. The source validator for export and persistence checks the exact signature, implementation identity, cumulative counts, unique anchor, and preservation of earlier cards. See Chapters 59 and 60.

C.6 · `infer_export_format`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.export.infer_export_format`. Reflected implementation `komorebi_math.export.writers.infer_export_format`. No root, stable, beta, or versioned facade exports this domain-only name in the reference manifest. The `komorebi_math.export` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C6-WRITERS`.

Manifest signature (s).

```
(path: 'str | Path') -> 'ExportFormat'
```

Definition. Infer an export format from a file suffix.

Contract (D, C, H, R, X). Writers resolve one of the 11 public formats, enforce parent-directory and overwrite policy, localize figure files when needed, and install primary outputs through temporary files. YAML requires PyYAML. PDF export requires a supported local LaTeX engine. NPY and NPZ writes disable pickle. Multi-format export uses deterministic suffixes, while bundle export stages files, records SHA-256 values, normalizes ZIP timestamps, and adds `MANIFEST.sha256`.

Guarantees (E, P, B, K). A valid result certificate establishes that the primary output exists, is nonempty, and matches the observed byte count. Companion paths record localized assets, generated TeX, or compiler logs. A bundle

manifest supports later integrity replay, but it is not a keyed signature and does not authenticate the archive against replacement of both data and manifest. Writer certificates do not certify the mathematical truth of rendered content.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_export_files.py` and `tests/test_export_cli.py`. The independent verifier for export and persistence exercises representative construction, rendering, serialization, file-policy, digest, migration, compilation, and rejection paths for this contract family. The source validator for export and persistence checks the exact signature, implementation identity, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 60.

C.6 · latex_escape

Identity (q, a, t, k). Public research-manifest route `komorebi_math.export.latex_escape`. Reflected implementation `komorebi_math.export.renderers.latex_escape`. No root, stable, beta, or versioned facade exports this domain-only name in the reference manifest. The `komorebi_math.export` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C6-RENDERERS`.

Manifest signature (s).

```
(value: 'Any') -> 'str'
```

Definition. Escape ordinary text for safe inclusion in LaTeX.

Contract (D, C, H, R, X). Renderers consume one `MathDocument` without modifying it. Ordinary prose, metadata, table cells, labels, and captions are escaped for the target format. Equation bodies and `RawLatex` remain trusted mathematical source. HTML can include an optional MathJax loader. LaTeX options validate document-class and package names before they enter the preamble.

Guarantees (E, P, B, K). Each renderer returns a complete text representation with the original block order. Markdown, HTML, text, and LaTeX outputs are semantically related but are not expected to be byte-identical. Rendering performs no filesystem write and no LaTeX compilation. The HTML MathJax option adds a network reference rather than bundling MathJax locally.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_export_renderers.py`. The independent verifier for export and persistence exercises representative construction, rendering, serialization, file-policy, digest, migration, compilation, and rejection paths for this contract family. The source validator for export and persistence checks the exact signature, implementation identity, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 60.

C.6 · latex_label

Identity (q, a, t, k). Public research-manifest route `komorebi_math.export.latex_label`. Reflected implementation `komorebi_math.export.renderers.latex_label`. No root, stable, beta, or versioned facade exports this domain-only name in the reference manifest. The `komorebi_math.export` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C6-RENDERERS`.

Manifest signature (s).

```
(value: 'str') -> 'str'
```

Definition. Normalize a user label to a conservative LaTeX label.

Contract (D, C, H, R, X). Renderers consume one `MathDocument` without modifying it. Ordinary prose, metadata, table cells, labels, and captions are escaped for the target format. Equation bodies and `RawLatex` remain trusted mathematical source. HTML can include an optional MathJax loader. LaTeX options validate document-class and package names before they enter the preamble.

Guarantees (E, P, B, K). Each renderer returns a complete text representation with the original block order. Markdown, HTML, text, and LaTeX outputs are semantically related but are not expected to be byte-identical. Rendering

performs no filesystem write and no LaTeX compilation. The HTML MathJax option adds a network reference rather than bundling MathJax locally.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_export_renderers.py`. The independent verifier for export and persistence exercises representative construction, rendering, serialization, file-policy, digest, migration, compilation, and rejection paths for this contract family. The source validator for export and persistence checks the exact signature, implementation identity, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 60.

C.6 · `load_portable`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.export.load_portable`. Reflected implementation `komorebi_math.export.persistence.load_portable`. The same runtime object is also available through `komorebi_math.load_portable`, `komorebi_math.stable.load_portable`, `komorebi_math.beta.load_portable`, `komorebi_math.v1.load_portable`, `komorebi_math.api.v1.load_portable`. The `komorebi_math.export` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C6-PORTABLE-PERSISTENCE`.

Manifest signature (s).

```
(path: 'str | os.PathLike[str]', *, max_depth: 'int' = 128) -> 'Any'
```

Definition. Read and validate a portable envelope from disk.

Contract (D, C, H, R, X). Portable persistence accepts finite JSON primitives, explicit structural tags, and registered codecs. Codec tags are nonempty and whitespace-free. Versions are positive integers. Migrations advance exactly one version at a time. The envelope has exactly five fields and binds the canonical encoded payload to SHA-256. Loading rejects duplicate JSON keys, nonfinite constants, unknown schemas, unsupported versions, digest mismatches, malformed tags, missing migration edges, and depth-limit violations.

Guarantees (E, P, B, K). Canonical payload ordering makes the digest deterministic for a fixed codec contract and value. Decoding never imports a class named by untrusted data. `save_portable` writes and flushes a temporary sibling before `os.replace`, while `load_portable` delegates all validation to the envelope decoder. SHA-256 provides integrity detection, not encryption, secrecy, or authentication. Extension codecs remain dynamic process state unless separately frozen by a release contract.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_block15_persistence.py`, `tests/test_block10_execution_persistence.py`, `tests/test_block11_streaming_migrations.py`, and `tests/test_nonlinear_snapshots_persistence.py`. The independent verifier for export and persistence exercises representative construction, rendering, serialization, file-policy, digest, migration, compilation, and rejection paths for this contract family. The source validator for export and persistence checks the exact signature, implementation identity, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 61.

C.6 · `loads_portable`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.export.loads_portable`. Reflected implementation `komorebi_math.export.persistence.loads_portable`. The same runtime object is also available through `komorebi_math.loads_portable`, `komorebi_math.stable.loads_portable`, `komorebi_math.beta.loads_portable`, `komorebi_math.v1.loads_portable`, `komorebi_math.api.v1.loads_portable`. The `komorebi_math.export` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C6-PORTABLE-PERSISTENCE`.

Manifest signature (s).

```
(text: 'str | bytes', *, max_depth: 'int' = 128) -> 'Any'
```

Definition. Validate and deserialize one complete portable JSON envelope.

Contract (D, C, H, R, X). Portable persistence accepts finite JSON primitives, explicit structural tags, and registered codecs. Codec tags are nonempty and whitespace-free. Versions are positive integers. Migrations advance exactly one version at a time. The envelope has exactly five fields and binds the canonical encoded payload to SHA-256. Loading rejects duplicate JSON keys, nonfinite constants, unknown schemas, unsupported versions, digest mismatches, malformed tags, missing migration edges, and depth-limit violations.

Guarantees (E, P, B, K). Canonical payload ordering makes the digest deterministic for a fixed codec contract and value. Decoding never imports a class named by untrusted data. `save_portable` writes and flushes a temporary sibling before `os.replace`, while `load_portable` delegates all validation to the envelope decoder. SHA-256 provides integrity detection, not encryption, secrecy, or authentication. Extension codecs remain dynamic process state unless separately frozen by a release contract.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_block15_persistence.py`, `tests/test_block10_execution_persistence.py`, `tests/test_block11_streaming_migrations.py`, and `tests/test_nonlinear_snapshots_persistence.py`. The independent verifier for export and persistence exercises representative construction, rendering, serialization, file-policy, digest, migration, compilation, and rejection paths for this contract family. The source validator for export and persistence checks the exact signature, implementation identity, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 61.

C.6 · `portable_codecs`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.export.portable_codecs`. Reflected implementation `komorebi_math.export.persistence.portable_codecs`. The same runtime object is also available through `komorebi_math.portable_codecs`, `komorebi_math.stable.portable_codecs`, `komorebi_math.beta.portable_codecs`, `komorebi_math.v1.portable_codecs`, `komorebi_math.api.v1.portable_codecs`. The `komorebi_math.export` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C6-PORTABLE-PERSISTENCE`.

Manifest signature (s).

```
() -> 'tuple[PortableCodec, ...]'
```

Definition. Return the active codecs ordered by stable tag.

Contract (D, C, H, R, X). Portable persistence accepts finite JSON primitives, explicit structural tags, and registered codecs. Codec tags are nonempty and whitespace-free. Versions are positive integers. Migrations advance exactly one version at a time. The envelope has exactly five fields and binds the canonical encoded payload to SHA-256. Loading rejects duplicate JSON keys, nonfinite constants, unknown schemas, unsupported versions, digest mismatches, malformed tags, missing migration edges, and depth-limit violations.

Guarantees (E, P, B, K). Canonical payload ordering makes the digest deterministic for a fixed codec contract and value. Decoding never imports a class named by untrusted data. `save_portable` writes and flushes a temporary sibling before `os.replace`, while `load_portable` delegates all validation to the envelope decoder. SHA-256 provides integrity detection, not encryption, secrecy, or authentication. Extension codecs remain dynamic process state unless separately frozen by a release contract.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_block15_persistence.py`, `tests/test_block10_execution_persistence.py`, `tests/test_block11_streaming_migrations.py`, and `tests/test_nonlinear_snapshots_persistence.py`. The independent verifier for export and persistence exercises representative construction, rendering, serialization, file-policy, digest, migration, compilation, and rejection paths for this contract family. The source validator for export and persistence checks the exact signature, implementation identity, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 61.

C.6 · portable_digest

Identity (q, a, t, k). Public research-manifest route `komorebi_math.export.portable_digest`. Reflected implementation `komorebi_math.export.persistence.portable_digest`. The same runtime object is also available through `komorebi_math.portable_digest`, `komorebi_math.stable.portable_digest`, `komorebi_math.beta.portable_digest`, `komorebi_math.v1.portable_digest`, `komorebi_math.api.v1.portable_digest`. The `komorebi_math.export` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C6-PORTABLE-PERSISTENCE`.

Manifest signature (s).

```
(value: 'Any', *, max_depth: 'int' = 128) -> 'str'
```

Definition. Return the canonical SHA256 digest of one encoded payload.

Contract (D, C, H, R, X). Portable persistence accepts finite JSON primitives, explicit structural tags, and registered codecs. Codec tags are nonempty and whitespace-free. Versions are positive integers. Migrations advance exactly one version at a time. The envelope has exactly five fields and binds the canonical encoded payload to SHA-256. Loading rejects duplicate JSON keys, nonfinite constants, unknown schemas, unsupported versions, digest mismatches, malformed tags, missing migration edges, and depth-limit violations.

Guarantees (E, P, B, K). Canonical payload ordering makes the digest deterministic for a fixed codec contract and value. Decoding never imports a class named by untrusted data. `save_portable` writes and flushes a temporary sibling before `os.replace`, while `load_portable` delegates all validation to the envelope decoder. SHA-256 provides integrity detection, not encryption, secrecy, or authentication. Extension codecs remain dynamic process state unless separately frozen by a release contract.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_block15_persistence.py`, `tests/test_block10_execution_persistence.py`, `tests/test_block11_streaming_migrations.py`, and `tests/test_nonlinear_snapshots_persistence.py`. The independent verifier for export and persistence exercises representative construction, rendering, serialization, file-policy, digest, migration, compilation, and rejection paths for this contract family. The source validator for export and persistence checks the exact signature, implementation identity, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 61.

C.6 · portable_migrations

Identity (q, a, t, k). Public research-manifest route `komorebi_math.export.portable_migrations`. Reflected implementation `komorebi_math.export.persistence.portable_migrations`. The same runtime object is also available through `komorebi_math.portable_migrations`, `komorebi_math.stable.portable_migrations`, `komorebi_math.beta.portable_migrations`, `komorebi_math.v1.portable_migrations`, `komorebi_math.api.v1.portable_migrations`. The `komorebi_math.export` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C6-PORTABLE-PERSISTENCE`.

Manifest signature (s).

```
() -> 'tuple[PortableMigration, ...]'
```

Definition. Return registered codec migrations in deterministic order.

Contract (D, C, H, R, X). Portable persistence accepts finite JSON primitives, explicit structural tags, and registered codecs. Codec tags are nonempty and whitespace-free. Versions are positive integers. Migrations advance exactly one version at a time. The envelope has exactly five fields and binds the canonical encoded payload to SHA-256. Loading rejects duplicate JSON keys, nonfinite constants, unknown schemas, unsupported versions, digest mismatches, malformed tags, missing migration edges, and depth-limit violations.

Guarantees (E, P, B, K). Canonical payload ordering makes the digest deterministic for a fixed codec contract and value. Decoding never imports a class named by untrusted data. `save_portable` writes and flushes a temporary sibling before `os.replace`, while `load_portable` delegates all validation to the envelope decoder. SHA-256 provides integrity detection, not encryption, secrecy, or authentication. Extension codecs remain dynamic process state unless separately frozen by a release contract.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_block15_persistence.py`, `tests/test_block10_execution_persistence.py`, `tests/test_block11_streaming_migrations.py`, and `tests/test_nonlinear_snapshots_persistence.py`. The independent verifier for export and persistence exercises representative construction, rendering, serialization, file-policy, digest, migration, compilation, and rejection paths for this contract family. The source validator for export and persistence checks the exact signature, implementation identity, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 61.

C.6 · `register_export_adapter`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.export.register_export_adapter`. Reflected implementation `komorebi_math.export.serialization.register_export_adapter`. No root, stable, beta, or versioned facade exports this domain-only name in the reference manifest. The `komorebi_math.export` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C6-SERIALIZATION`.

Manifest signature (s).

```
(type_: 'type[T]', adapter: 'Callable[[T], Any]', *, replace: 'bool' = False) -> 'None'
```

Definition. Register a normalization adapter for one Python type.

Contract (D, C, H, R, X). Presentation serialization accepts supported Python primitives, NumPy values, fractions, complex values, paths, enums, package result records, dataclasses, mappings, sequences, registered adapters, and objects implementing `Exportable`. Recursion is bounded by `max_depth`. Registered adapters are selected through the runtime type MRO. Decoding reconstructs only the safe built-in markers emitted by this layer and leaves unknown type markers as ordinary mappings.

Guarantees (E, P, B, K). The result is JSON-compatible presentation data. Type preservation is selective rather than a general object-reconstruction promise. Mapping keys become strings, sets are ordered by `repr`, and arbitrary class names in `__type__` are never imported or instantiated. The adapter registry is process-local and a copied registry returned by `export_adapters` cannot mutate the active registry.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_export_serialization.py`. The independent verifier for export and persistence exercises representative construction, rendering, serialization, file-policy, digest, migration, compilation, and rejection paths for this contract family. The source validator for export and persistence checks the exact signature, implementation identity, cumulative counts, unique anchor, and preservation of earlier cards. See Chapters 59 and 60.

C.6 · `register_portable_codec`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.export.register_portable_codec`. Reflected implementation `komorebi_math.export.persistence.register_portable_codec`. The same runtime object is also available through `komorebi_math.register_portable_codec`, `komorebi_math.beta.register_portable_codec`. The `komorebi_math.export` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C6-PORTABLE-PERSISTENCE`.

Manifest signature (s).

```
(type_: 'type[T]', *, tag: 'str', version: 'int', encoder: 'Callable[[T], Any]', decoder: 'Callable[[Any], T]',
   ↪ replace: 'bool' = False) -> 'None'
```

Definition. Register one stable round-trip codec.

Contract (D, C, H, R, X). Portable persistence accepts finite JSON primitives, explicit structural tags, and registered codecs. Codec tags are nonempty and whitespace-free. Versions are positive integers. Migrations advance exactly one version at a time. The envelope has exactly five fields and binds the canonical encoded payload to SHA-256. Loading rejects duplicate JSON keys, nonfinite constants, unknown schemas, unsupported versions, digest mismatches, malformed tags, missing migration edges, and depth-limit violations.

Guarantees (E, P, B, K). Canonical payload ordering makes the digest deterministic for a fixed codec contract and value. Decoding never imports a class named by untrusted data. `save_portable` writes and flushes a temporary sibling before `os.replace`, while `load_portable` delegates all validation to the envelope decoder. SHA-256 provides integrity detection, not encryption, secrecy, or authentication. Extension codecs remain dynamic process state unless separately frozen by a release contract.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_block15_persistence.py`, `tests/test_block10_execution_persistence.py`, `tests/test_block11_streaming_migrations.py`, and `tests/test_nonlinear_snapshots_persistence.py`. The independent verifier for export and persistence exercises representative construction, rendering, serialization, file-policy, digest, migration, compilation, and rejection paths for this contract family. The source validator for export and persistence checks the exact signature, implementation identity, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 61.

C.6 · `register_portable_migration`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.export.register_portable_migration`. Reflected implementation `komorebi_math.export.persistence.register_portable_migration`. The same runtime object is also available through `komorebi_math.register_portable_migration`, `komorebi_math.beta.register_portable_migration`. The `komorebi_math.export` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C6-PORTABLE-PERSISTENCE`.

Manifest signature (s).

```
(*, tag: 'str', from_version: 'int', to_version: 'int', migrator: 'Migrator', replace: 'bool' = False) -> 'None'
```

Definition. Register one deterministic adjacent migration for codec data.

Contract (D, C, H, R, X). Portable persistence accepts finite JSON primitives, explicit structural tags, and registered codecs. Codec tags are nonempty and whitespace-free. Versions are positive integers. Migrations advance exactly one version at a time. The envelope has exactly five fields and binds the canonical encoded payload to SHA-256. Loading rejects duplicate JSON keys, nonfinite constants, unknown schemas, unsupported versions, digest mismatches, malformed tags, missing migration edges, and depth-limit violations.

Guarantees (E, P, B, K). Canonical payload ordering makes the digest deterministic for a fixed codec contract and value. Decoding never imports a class named by untrusted data. `save_portable` writes and flushes a temporary sibling before `os.replace`, while `load_portable` delegates all validation to the envelope decoder. SHA-256 provides integrity detection, not encryption, secrecy, or authentication. Extension codecs remain dynamic process state unless separately frozen by a release contract.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_block15_persistence.py`, `tests/test_block10_execution_persistence.py`, `tests/test_block11_streaming_migrations.py`, and `tests/test_nonlinear_snapshots_persistence.py`. The independent verifier for export and persistence exercises representative construction, rendering, serialization, file-policy, digest, migration, compilation, and rejection paths for this contract family. The source validator for export and persistence checks the exact signature, implementation identity, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 61.

C.6 · `render_html`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.export.render_html`. Reflected implementation `komorebi_math.export.renderers.render_html`. No root, stable, beta, or versioned facade exports this domain-only name in the reference manifest. The `komorebi_math.export` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C6-RENDERERS`.

Manifest signature (s).

```
(document: 'MathDocument', *, options: 'HtmlRenderOptions | None' = None) -> 'str'
```

Definition. Render a standalone HTML document.

Contract (D, C, H, R, X). Renderers consume one `MathDocument` without modifying it. Ordinary prose, metadata, table cells, labels, and captions are escaped for the target format. Equation bodies and `RawLatex` remain trusted mathematical source. HTML can include an optional MathJax loader. LaTeX options validate document-class and package names before they enter the preamble.

Guarantees (E, P, B, K). Each renderer returns a complete text representation with the original block order. Markdown, HTML, text, and LaTeX outputs are semantically related but are not expected to be byte-identical. Rendering performs no filesystem write and no LaTeX compilation. The HTML MathJax option adds a network reference rather than bundling MathJax locally.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_export_renderers.py`. The independent verifier for export and persistence exercises representative construction, rendering, serialization, file-policy, digest, migration, compilation, and rejection paths for this contract family. The source validator for export and persistence checks the exact signature, implementation identity, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 60.

C.6 · `render_latex`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.export.render_latex`. Reflected implementation `komorebi_math.export.renderers.render_latex`. The same runtime object is also available through `komorebi_math.render_latex`, `komorebi_math.stable.render_latex`, `komorebi_math.v1.render_latex`, `komorebi_math.api.v1.render_latex`. The `komorebi_math.export` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C6-RENDERERS`.

Manifest signature (s).

```
(document: 'MathDocument', *, options: 'LatexRenderOptions | None' = None) -> 'str'
```

Definition. Render a complete compilable LaTeX document.

Contract (D, C, H, R, X). Renderers consume one `MathDocument` without modifying it. Ordinary prose, metadata, table cells, labels, and captions are escaped for the target format. Equation bodies and `RawLatex` remain trusted mathematical source. HTML can include an optional MathJax loader. LaTeX options validate document-class and package names before they enter the preamble.

Guarantees (E, P, B, K). Each renderer returns a complete text representation with the original block order. Markdown, HTML, text, and LaTeX outputs are semantically related but are not expected to be byte-identical. Rendering performs no filesystem write and no LaTeX compilation. The HTML MathJax option adds a network reference rather than bundling MathJax locally.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_export_renderers.py`. The independent verifier for export and persistence exercises representative construction, rendering, serialization, file-policy, digest, migration, compilation, and rejection paths for this contract family. The source validator for export and persistence checks the exact signature, implementation identity, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 60.

C.6 · `render_markdown`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.export.render_markdown`. Reflected implementation `komorebi_math.export.renderers.render_markdown`. The same runtime object is also available through `komorebi_math.stable.render_markdown`, `komorebi_math.v1.render_markdown`, `komorebi_math.api.v1.render_markdown`. The `komorebi_math.export` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C6-RENDERERS`.

Manifest signature (s).

```
(document: 'MathDocument') -> 'str'
```

Definition. Render a document as Markdown with TeX equations.

Contract (D, C, H, R, X). Renderers consume one `MathDocument` without modifying it. Ordinary prose, metadata, table cells, labels, and captions are escaped for the target format. Equation bodies and `RawLatex` remain trusted mathematical source. HTML can include an optional MathJax loader. LaTeX options validate document-class and package names before they enter the preamble.

Guarantees (E, P, B, K). Each renderer returns a complete text representation with the original block order. Markdown, HTML, text, and LaTeX outputs are semantically related but are not expected to be byte-identical. Rendering performs no filesystem write and no LaTeX compilation. The HTML MathJax option adds a network reference rather than bundling MathJax locally.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_export_renderers.py`. The independent verifier for export and persistence exercises representative construction, rendering, serialization, file-policy, digest, migration, compilation, and rejection paths for this contract family. The source validator for export and persistence checks the exact signature, implementation identity, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 60.

C.6 · `render_text`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.export.render_text`. Reflected implementation `komorebi_math.export.renderers.render_text`. No root, stable, beta, or versioned facade exports this domain-only name in the reference manifest. The `komorebi_math.export` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C6-RENDERERS`.

Manifest signature (s).

```
(document: 'MathDocument') -> 'str'
```

Definition. Render a document as plain UTF-8 text.

Contract (D, C, H, R, X). Renderers consume one `MathDocument` without modifying it. Ordinary prose, metadata, table cells, labels, and captions are escaped for the target format. Equation bodies and `RawLatex` remain trusted mathematical source. HTML can include an optional MathJax loader. LaTeX options validate document-class and package names before they enter the preamble.

Guarantees (E, P, B, K). Each renderer returns a complete text representation with the original block order. Markdown, HTML, text, and LaTeX outputs are semantically related but are not expected to be byte-identical. Rendering performs no filesystem write and no LaTeX compilation. The HTML MathJax option adds a network reference rather than bundling MathJax locally.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_export_renderers.py`. The independent verifier for export and persistence exercises representative construction, rendering, serialization, file-policy, digest, migration, compilation, and rejection paths for this contract family. The source validator for export and persistence checks the exact signature, implementation identity, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 60.

C.6 · `resolve_latex_engine`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.export.resolve_latex_engine`. Reflected implementation `komorebi_math.export.latex.resolve_latex_engine`. No root, stable, beta, or versioned facade exports this domain-only name in the reference manifest. The `komorebi_math.export` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C6-LATEX-COMPILATION`.

Manifest signature (s).

```
(engine: 'LatexEngine | str' = <LatexEngine.AUTO: 'auto'>) -> 'LatexEngine'
```

Definition. Resolve an explicit or automatic LaTeX engine choice.

Contract (D, C, H, R, X). LaTeX compilation resolves an explicit engine or the first installed automatic candidate. Pass counts and timeouts must be positive. Job names must be simple file names. Shell escape is disabled by default, direct engines receive `-no-shell-escape`, and the `latexmk` wrapper rejects shell-escape requests. Source may be an existing `.tex` path or text written into the isolated output directory.

Guarantees (E, P, B, K). The compilation record retains source, PDF, log, engine, commands, return codes, and a certificate. Success requires all invoked commands to return zero and a nonempty PDF to exist. Missing engines raise `BackendUnavailableError`. Compiler success is local environment evidence, not a proof that trusted raw LaTeX is safe or that every external asset remains available later.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_export_files.py` and `tests/test_export_cli.py`. The independent verifier for export and persistence exercises representative construction, rendering, serialization, file-policy, digest, migration, compilation, and rejection paths for this contract family. The source validator for export and persistence checks the exact signature, implementation identity, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 60. The recorded audit runtime exposed pdfLaTeX, XeLaTeX, LuaLaTeX, and latexmk, but engine availability remains an environment observation rather than a package guarantee.

C.6 · `save_portable`

Identity (q, a, t, k). Public research-manifest route `komorebi_math.export.save_portable`. Reflected implementation `komorebi_math.export.persistence.save_portable`. The same runtime object is also available through `komorebi_math.save_portable`, `komorebi_math.stable.save_portable`, `komorebi_math.beta.save_portable`, `komorebi_math.v1.save_portable`, `komorebi_math.api.v1.save_portable`. The `komorebi_math.export` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C6-PORTABLE-PERSISTENCE`.

Manifest signature (s).

```
(value: 'Any', path: 'str | os.PathLike[str]', *, overwrite: 'bool' = False, indent: 'int | None' = 2, max_depth:
    ↪ 'int' = 128) -> 'Path'
```

Definition. Atomically write a portable envelope to disk.

Contract (D, C, H, R, X). Portable persistence accepts finite JSON primitives, explicit structural tags, and registered codecs. Codec tags are nonempty and whitespace-free. Versions are positive integers. Migrations advance exactly one version at a time. The envelope has exactly five fields and binds the canonical encoded payload to SHA-256. Loading rejects duplicate JSON keys, nonfinite constants, unknown schemas, unsupported versions, digest mismatches, malformed tags, missing migration edges, and depth-limit violations.

Guarantees (E, P, B, K). Canonical payload ordering makes the digest deterministic for a fixed codec contract and value. Decoding never imports a class named by untrusted data. `save_portable` writes and flushes a temporary sibling before `os.replace`, while `load_portable` delegates all validation to the envelope decoder. SHA-256 provides integrity detection, not encryption, secrecy, or authentication. Extension codecs remain dynamic process state unless separately frozen by a release contract.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_block15_persistence.py`, `tests/test_block10_execution_persistence.py`, `tests/test_block11_streaming_migrations.py`, and `tests/test_nonlinear_snapshots_persistence.py`. The independent verifier for export and persistence exercises representative construction, rendering, serialization, file-policy, digest, migration, compilation, and rejection paths for this contract family. The source validator for export and persistence checks the exact signature, implementation identity, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 61.

C.6 · sequence_document

Identity (q, a, t, k). Public research-manifest route `komorebi_math.export.sequence_document`. Reflected implementation `komorebi_math.export.report.sequence_document`. The same runtime object is also available through `komorebi_math.sequence_document`. The `komorebi_math.export` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C6-REPORT-BUILDERS`.

Manifest signature (s).

```
(name: 'str', values: 'Sequence[Any]', *, strategy: 'str | None' = None, modulus: 'int | None' = None, author: 'str
    ↪ | None' = None) -> 'MathDocument'
```

Definition. Build a table report for one executable sequence.

Contract (D, C, H, R, X). Report builders convert certificates, finite sequences, and common Python data into the typed document model. Existing `MathDocument` values pass through unchanged. Result records may contribute their certificate when `include_certificate` is enabled. Mappings, short arrays, sequences, and dataclasses are converted to tables when possible, while unsupported table shapes fall back to normalized JSON code.

Guarantees (E, P, B, K). The builders preserve visible validity, exactness, tolerances, residuals, assumptions, and diagnostics when those fields exist. They produce readable reports rather than portable reconstruction records. A report can summarize evidence, but creating the report does not independently validate the source object or certificate.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_export_renderers.py` and `tests/test_export_files.py`. The independent verifier for export and persistence exercises representative construction, rendering, serialization, file-policy, digest, migration, compilation, and rejection paths for this contract family. The source validator for export and persistence checks the exact signature, implementation identity, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 59.

C.6 · table_rows

Identity (q, a, t, k). Public research-manifest route `komorebi_math.export.table_rows`. Reflected implementation `komorebi_math.export.serialization.table_rows`. No root, stable, beta, or versioned facade exports this domain-only name in the reference manifest. The `komorebi_math.export` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C6-SERIALIZATION`.

Manifest signature (s).

```
(value: 'Any') -> 'tuple[tuple[str, ...], tuple[tuple[Any, ...], ...]]'
```

Definition. Convert a table shaped value to headers and rows.

Contract (D, C, H, R, X). Presentation serialization accepts supported Python primitives, NumPy values, fractions, complex values, paths, enums, package result records, dataclasses, mappings, sequences, registered adapters, and objects implementing `Exportable`. Recursion is bounded by `max_depth`. Registered adapters are selected through the runtime type MRO. Decoding reconstructs only the safe built-in markers emitted by this layer and leaves unknown type markers as ordinary mappings.

Guarantees (E, P, B, K). The result is JSON-compatible presentation data. Type preservation is selective rather than a general object-reconstruction promise. Mapping keys become strings, sets are ordered by `repr`, and arbitrary class names in `__type__` are never imported or instantiated. The adapter registry is process-local and a copied registry returned by `export_adapters` cannot mutate the active registry.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_export_serialization.py`. The independent verifier for export and persistence exercises representative construction, rendering, serialization, file-policy, digest, migration, compilation, and rejection paths for this contract family. The source validator for export and persistence checks the exact signature, implementation identity, cumulative counts, unique anchor, and preservation of earlier cards. See Chapters 59 and 60.

C.6 · to_portable_data

Identity (q, a, t, k). Public research-manifest route `komorebi_math.export.to_portable_data`. Reflected implementation `komorebi_math.export.persistence.to_portable_data`. The same runtime object is also available through `komorebi_math.stable.to_portable_data`, `komorebi_math.beta.to_portable_data`, `komorebi_math.v1.to_portable_data`, `komorebi_math.api.v1.to_portable_data`. The `komorebi_math.export` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C6-PORTABLE-PERSISTENCE`.

Manifest signature (s).

```
(value: 'Any', *, max_depth: 'int' = 128) -> 'Any'
```

Definition. Encode one value into the versioned portable payload tree.

Contract (D, C, H, R, X). Portable persistence accepts finite JSON primitives, explicit structural tags, and registered codecs. Codec tags are nonempty and whitespace-free. Versions are positive integers. Migrations advance exactly one version at a time. The envelope has exactly five fields and binds the canonical encoded payload to SHA-256. Loading rejects duplicate JSON keys, nonfinite constants, unknown schemas, unsupported versions, digest mismatches, malformed tags, missing migration edges, and depth-limit violations.

Guarantees (E, P, B, K). Canonical payload ordering makes the digest deterministic for a fixed codec contract and value. Decoding never imports a class named by untrusted data. `save_portable` writes and flushes a temporary sibling before `os.replace`, while `load_portable` delegates all validation to the envelope decoder. SHA-256 provides integrity detection, not encryption, secrecy, or authentication. Extension codecs remain dynamic process state unless separately frozen by a release contract.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test modules are `tests/test_block15_persistence.py`, `tests/test_block10_execution_persistence.py`, `tests/test_block11_streaming_migrations.py`, and `tests/test_nonlinear_snapshots_persistence.py`. The independent verifier for export and persistence exercises representative construction, rendering, serialization, file-policy, digest, migration, compilation, and rejection paths for this contract family. The source validator for export and persistence checks the exact signature, implementation identity, cumulative counts, unique anchor, and preservation of earlier cards. See Chapter 61.

C.6 · to_serializable

Identity (q, a, t, k). Public research-manifest route `komorebi_math.export.to_serializable`. Reflected implementation `komorebi_math.export.serialization.to_serializable`. The same runtime object is also available through `komorebi_math.to_serializable`. The `komorebi_math.export` route is governed by `[RESEARCH]` policy `pre-beta-additive`. This domain route does not enlarge the frozen stable compatibility surface. Manifest kind `function`. Profile `C6-SERIALIZATION`.

Manifest signature (s).

```
(value: 'Any', *, preserve_types: 'bool' = True, max_depth: 'int' = 64, _depth: 'int' = 0) -> 'Any'
```

Definition. Convert package and scientific Python objects to JSON compatible data.

Contract (D, C, H, R, X). Presentation serialization accepts supported Python primitives, NumPy values, fractions, complex values, paths, enums, package result records, dataclasses, mappings, sequences, registered adapters, and objects implementing `Exportable`. Recursion is bounded by `max_depth`. Registered adapters are selected through the runtime type MRO. Decoding reconstructs only the safe built-in markers emitted by this layer and leaves unknown type markers as ordinary mappings.

Guarantees (E, P, B, K). The result is JSON-compatible presentation data. Type preservation is selective rather than a general object-reconstruction promise. Mapping keys become strings, sets are ordered by `repr`, and arbitrary class names in `__type__` are never imported or instantiated. The adapter registry is process-local and a copied registry returned by `export_adapters` cannot mutate the active registry.

Evidence and links (V, L). The committed research manifest and runtime reflection audit identify this export. The focused test module is `tests/test_export_serialization.py`. The independent verifier for export and persistence exercises representative construction, rendering, serialization, file-policy, digest, migration, compilation, and rejection paths for this contract family. The source validator for export and persistence checks the exact signature, implementation identity, cumulative counts, unique anchor, and preservation of earlier cards. See Chapters 59 and 60.

Research reference coverage. The appendix contains all 1,312 research cards, including 516 root cards and every domain namespace. The four indexes below are derived from those cards and the reference manifests.

C.7 Canonical implementation-owner index across 201 research owners

The research manifest resolves its 1,312 namespace and name pairs to 201 implementation-owner strings. Of these, 196 are owned by the package and 5 come from Python or typing support modules. The package-owned set consists of 194 concrete source modules and the two package initializers `komorebi_math.nonlinear` and `komorebi_math.signals`. This index is scoped to Appendix C. The union across the stable, beta, and research manifests contains 207 distinct owners because six compatibility and exception modules occur only in the curated tiers.

Owner family	Owners	Research cards
`config`	1	4
`certificates`	1	4
`compatibility`	1	14
`golden`	1	4
`provenance`	1	3
`registry`	1	4
`unified`	1	3
`core`	9	25
`algorithms`	9	19
`arithmetic`	7	10
`sequences`	5	11
`analytic`	5	11
`refinement`	7	12
`coordinates`	23	278
`generating`	21	206
`signals`	21	162
`nonlinear`	22	229
`integration`	7	38
`dynamics`	7	7
`quantum`	11	12
`deformation`	5	6
`langlands`	17	28
`expression`	6	77
`export`	7	90
`external owners`	5	55
Total	**201**	**1,312**

Each owner entry lists every research card that resolves to that owner. A card link identifies the exported route. It does not imply that the implementation module itself is a supported public import path.

C.7.1 Config

komorebi_math.config. 4 research cards. Namespace counts: root 4. Cards: C.1A configure, C.1A get_settings, C.1A Settings, C.1A settings_context.

C.7.2 Certificates

komorebi_math.certificates. 4 research cards. Namespace counts: root 4. Cards: C.1A Certificate, C.1A ComputationResult, C.1A ConditionalResult, C.1A Exactness.

C.7.3 Compatibility

komorebi_math.compatibility. 14 research cards. Namespace counts: root 14. Cards: C.1A ApiCompatibilityReport, C.1A check_beta_api, C.1A check_public_api, C.1A compare_public_api, C.1A create_beta_api_manifest, C.1A create_public_api_manifest, C.1A deprecated, C.1A KomorebiDeprecationWarning, C.1A load_beta_api_manifest, C.1A load_public_api_manifest, C.1A PublicApiManifest, C.1A PublicApiSymbol, C.1A save_public_api_manifest, C.1A warn_deprecated.

C.7.4 Golden

komorebi_math.golden. 4 research cards. Namespace counts: root 4. Cards: C.1A BehavioralGoldenCaseResult, C.1A BehavioralGoldenReport, C.1A load_behavioral_golden_fixture, C.1A run_behavioral_golden_fixture.

C.7.5 Provenance

komorebi_math.provenance. 3 research cards. Namespace counts: root 3. Cards: C.1A ClaimStatus, C.1A describe, C.1A ProvenanceRecord.

C.7.6 Registry

komorebi_math.registry. 4 research cards. Namespace counts: root 4. Cards: C.1G feature, C.1G features, C.1G FeatureSpec, C.1G FeatureStatus.

C.7.7 Unified

komorebi_math.unified. 3 research cards. Namespace counts: root 3. Cards: C.1G StageSpec, C.1G UnifiedStage, C.1G UnifiedSystem.

C.7.8 Core

komorebi_math.core.algebra. 3 research cards. Namespace counts: root 3. Cards: C.1D AlgebraBasisResult, C.1D matrix_units, C.1D OperatorAlgebra.

komorebi_math.core.grammar. 4 research cards. Namespace counts: root 4. Cards: C.1D Arrow, C.1D OperatorGrammar, C.1D OperatorRelation, C.1D OperatorWord.

komorebi_math.core.graphs. 2 research cards. Namespace counts: root 2. Cards: C.1D SupportPreorder, C.1D TransferGraph.

komorebi_math.core.morphisms. 3 research cards. Namespace counts: root 3. Cards: C.1D OperatorMorphism, C.1D PacketMorphism, C.1D SeedMorphism.

komorebi_math.core.operators. 1 research card. Namespace counts: root 1. Cards: C.1D PacketOperator.

komorebi_math.core.packets. 2 research cards. Namespace counts: root 2. Cards: C.1D PacketCoordinates, C.1D PacketSystem.

komorebi_math.core.realization. 1 research card. Namespace counts: root 1. Cards: C.1D OperatorRealization.

komorebi_math.core.seeds. 7 research cards. Namespace counts: root 7. Cards: C.1D cyclic_span, C.1D independent_direct_sum_seed_rank, C.1D SeedSystem, C.1D semisimple_seed_rank, C.1D shared_direct_sum_seed_bounds, C.1D single_operator_seed_rank, C.1D tensor_seed_rank_upper_bound.

komorebi_math.core.transfer. 2 research cards. Namespace counts: root 2. Cards: C.1D block_from_paths, C.1D path_contributions.

C.7.9 Algorithms

komorebi_math.algorithms.compiler. 3 research cards. Namespace counts: root 3. Cards: C.1D compile_tree_budget, C.1D compile_tree_lagrangian, C.1D TreeNode.

komorebi_math.algorithms.discovery. 2 research cards. Namespace counts: root 2. Cards: C.1D discover_block_diagonal_low_rank, C.1D discover_packet_support.

komorebi_math.algorithms.ir. 1 research card. Namespace counts: root 1. Cards: C.1D PacketIR.

komorebi_math.algorithms.krylov. 1 research card. Namespace counts: root 1. Cards: C.1D conjugate_gradient.

komorebi_math.algorithms.layout. 1 research card. Namespace counts: root 1. Cards: C.1D PacketLayout.

komorebi_math.algorithms.low_rank. 3 research cards. Namespace counts: root 3. Cards: C.1D BlockDiagonalLowRankOperator, C.1D LowRankOperator, C.1D recompress_factors.

komorebi_math.algorithms.precision. 2 research cards. Namespace counts: root 2. Cards: C.1D assign_discrete_precisions, C.1D DTypeChoice.

komorebi_math.algorithms.scheduling. 2 research cards. Namespace counts: root 2. Cards: C.1D schedule_unlimited, C.1D Task.

komorebi_math.algorithms.structured. 4 research cards. Namespace counts: root 4. Cards: C.1D BlockDiagonalOperator, C.1D CirculantOperator, C.1D DenseOperator, C.1D WalshHadamardOperator.

C.7.10 Arithmetic

komorebi_math.arithmetic.domains. 1 research card. Namespace counts: root 1. Cards: C.1G TruncatedPadic.

komorebi_math.arithmetic.euler. 2 research cards. Namespace counts: root 2. Cards: C.1G EulerLocalCoordinates, C.1G signed_euler_number.

komorebi_math.arithmetic.hurwitz. 1 research card. Namespace counts: root 1. Cards: C.1G HurwitzSeries.

komorebi_math.arithmetic.mahler. 2 research cards. Namespace counts: root 2. Cards: C.1G affine_mahler_kernel, C.1G MahlerSeries.

komorebi_math.arithmetic.recurrence. 2 research cards. Namespace counts: root 2. Cards: C.1G fibonacci_recurrence, C.1G LinearRecurrence.

komorebi_math.arithmetic.standard_windows. 1 research card. Namespace counts: root 1. Cards: C.1G StandardWindow.

komorebi_math.arithmetic.valuation. 1 research card. Namespace counts: root 1. Cards: C.1G padic_valuation.

C.7.11 Sequences

komorebi_math.sequences.automatic. 1 research card. Namespace counts: root 1. Cards: C.1G AutomaticSequence.

komorebi_math.sequences.base. 1 research card. Namespace counts: root 1. Cards: C.1G Sequence.

komorebi_math.sequences.catalog. 6 research cards. Namespace counts: root 6. Cards: C.1G apery_sequence, C.1G fibonacci_sequence, C.1G get_sequence, C.1G motzkin_sequence, C.1G partition_sequence, C.1G thue_morse_sequence.

komorebi_math.sequences.compiler. 2 research cards. Namespace counts: root 2. Cards: C.1G compile_sequence, C.1G CompiledSequence.

komorebi_math.sequences.holonomic. 1 research card. Namespace counts: root 1. Cards: C.1G HolonomicRecurrence.

C.7.12 Analytic

komorebi_math.analytic.appell. 2 research cards. Namespace counts: root 2. Cards: C.1G appell_polynomial, C.1G theta_polynomial.

komorebi_math.analytic.cyclic. 1 research card. Namespace counts: root 1. Cards: C.1G cyclic_project_coefficients.

komorebi_math.analytic.reciprocal. 2 research cards. Namespace counts: root 2. Cards: C.1G reciprocal_coefficients, C.1G transfer_tangent.

komorebi_math.analytic.sections. 4 research cards. Namespace counts: root 4. Cards: C.1G CyclicSectionFamily, C.1G denominator, C.1G signed_section, C.1G transfer_section.

komorebi_math.analytic.zeros. 2 research cards. Namespace counts: root 2. Cards: C.1G CyclicDenominator, C.1G positive_zero_string.

C.7.13 Refinement

komorebi_math.refinement.crt. 2 research cards. Namespace counts: root 2. Cards: C.1G crt_many, C.1G crt_pair.

komorebi_math.refinement.defects. 1 research card. Namespace counts: root 1. Cards: C.1G LinearRefinementDefect.

komorebi_math.refinement.lattices. 1 research card. Namespace counts: root 1. Cards: C.1G LatticePacketData.

komorebi_math.refinement.residue. 2 research cards. Namespace counts: root 2. Cards: C.1G residue_analyze, C.1G residue_packet_system.

komorebi_math.refinement.smith. 1 research card. Namespace counts: root 1. Cards: C.1G SmithCongruenceProblem.

komorebi_math.refinement.towers. 3 research cards. Namespace counts: root 3. Cards: C.1G common_refinement, C.1G PacketRefinement, C.1G RefinementTower.

komorebi_math.refinement.weighted. 2 research cards. Namespace counts: root 2. Cards: C.1G WeightedTransferEdge, C.1G WeightedTransferGraph.

C.7.14 Coordinates

komorebi_math.coordinates.affine. 5 research cards. Namespace counts: coordinates 4, root 1. Cards: C.1C AffineCoordinateTransform, C.2C affine_basis_transform, C.2C AffineCoordinateTransform, C.2C BarycentricCoordinateSystem, C.2C translation_transform.

komorebi_math.coordinates.analysis_synthesis. 4 research cards. Namespace counts: coordinates 2, root 2. Cards: C.1C AnalysisSynthesisBlocks, C.1C AnalysisSynthesisSystem, C.2C AnalysisSynthesisBlocks, C.2C AnalysisSynthesisSystem.

komorebi_math.coordinates.basis. 16 research cards. Namespace counts: coordinates 11, root 5. Cards: C.1C basis_from_matrix, C.1C CoordinateAtlas, C.1C CoordinateSystem, C.1C FiniteBasis, C.1C standard_basis, C.2C AtlasPath, C.2C basis_from_matrix, C.2C CommutingDiagramCheck, C.2C CoordinateAtlas, C.2C CoordinateSystem, C.2C direct_sum_basis, C.2C FiniteBasis, C.2C permutation_transform, C.2C scaling_transform, C.2C standard_basis, C.2C tensor_product_basis.

komorebi_math.coordinates.budget. 3 research cards. Namespace counts: coordinates 2, root 1. Cards: C.1C CoordinateBudget, C.2C CoordinateBudget, C.2C DEFAULT_COORDINATE_BUDGET.

komorebi_math.coordinates.calculus. 11 research cards. Namespace counts: coordinates 10, root 1. Cards: C.1C PolynomialCoordinateOperator, C.2B affine_substitution_coordinate_operator, C.2B derivative_coordinate_operator, C.2B finite_difference_coordinate_operator, C.2B integral_coordinate_operator, C.2B multiplication_coordinate_operator, C.2B operator_commutator, C.2B polynomial_integral, C.2B polynomial_operator, C.2B PolynomialCoordinateOperator, C.2B transport_polynomial_operator.

komorebi_math.coordinates.classical. 44 research cards. Namespace counts: coordinates 42, root 2. Cards: C.1C classical_polynomial_catalog, C.1C get_polynomial_family, C.2B affine_polynomial, C.2B affine_substitution_transform, C.2B appell_polynomial, C.2B appell_polynomial_family, C.2B bernoulli_number, C.2B bernoulli_polynomial, C.2B bernoulli_polynomial_family, C.2B bernstein_basis, C.2B bernstein_polynomial, C.2B binomial_polynomial, C.2B binomial_polynomial_family, C.2B chebyshev_t_polynomial, C.2B chebyshev_t_polynomial_family, C.2B chebyshev_u_polynomial, C.2B chebyshev_u_polynomial_family, C.2B classical_polynomial_catalog, C.2B constant_polynomial, C.2B evaluation_transform, C.2B falling_factorial_polynomial, C.2B falling_factorial_polynomial_family, C.2B get_polynomial_family, C.2B hermite_physicists_polynomial, C.2B hermite_physicists_polynomial_family, C.2B hermite_probabilists_polynomial, C.2B hermite_probabilists_polynomial_family, C.2B interpolation_transform, C.2B lagrange_interpolation_basis, C.2B laguerre_polynomial, C.2B laguerre_polynomial_family, C.2B legendre_polynomial, C.2B legendre_polynomial_family, C.2B lucas_polynomial, C.2B lucas_polynomial_family, C.2B newton_interpolation_basis, C.2B operator_matrix_between_families, C.2B pell_polynomial, C.2B pell_polynomial_family, C.2B polynomial_compose_affine, C.2B polynomial_subtract, C.2B PolynomialFamilyCatalog, C.2B rising_factorial_polynomial, C.2B rising_factorial_polynomial_family.

komorebi_math.coordinates.combinatorial. 18 research cards. Namespace counts: coordinates 16, root 2. Cards: C.1C binomial_transform, C.1C forward_difference_transform, C.2A binomial_transform, C.2A divisor_zeta_transform, C.2A falling_factorial_to_monomial_transform, C.2A forward_difference_transform, C.2A lah_number, C.2A mobius_integer, C.2A monomial_to_falling_factorial_transform, C.2A monomial_to_rising_factorial_transform, C.2A newton_evaluation_transform, C.2A rising_factorial_to_monomial_transform, C.2A rising_to_falling_factorial_transform, C.2A stirling_first_signed, C.2A stirling_first_unsigned, C.2A stirling_second, C.2A subset_zeta_transform, C.2A transform_sequence.

komorebi_math.coordinates.exact. 19 research cards. Namespace counts: coordinates 19. Cards: C.2A as_matrix, C.2A as_vector, C.2A determinant, C.2A ExactLinearSystem, C.2A identity_matrix, C.2A inverse_matrix, C.2A matrix_add, C.2A matrix_from_columns, C.2A matrix_multiply, C.2A matrix_power, C.2A matrix_rank, C.2A matrix_shape, C.2A matrix_subtract, C.2A matrix_vector_multiply, C.2A nullspace, C.2A rref, C.2A solve_linear_system, C.2A transpose, C.2A zero_matrix.

komorebi_math.coordinates.families. 14 research cards. Namespace counts: coordinates 11, root 3. Cards: C.1C get_recurrence_model, C.1C recurrence_model_catalog, C.1C RecurrenceModel, C.2C custom_recurrence_model, C.2C fibonacci_recurrence_model, C.2C get_recurrence_model, C.2C jacobsthal_recurrence_model, C.2C lucas_recurrence_model, C.2C padovan_recurrence_model, C.2C pell_recurrence_model, C.2C recurrence_model_catalog, C.2C RecurrenceModel, C.2C sequence_coordinates_between_models, C.2C tribonacci_recurrence_model.

komorebi_math.coordinates.formal. 5 research cards. Namespace counts: coordinates 4, root 1. Cards: C.1C FormalCoordinateChange, C.2C compose_ordinary_series, C.2C formal_coordinate_from_iterable, C.2C FormalCoordinateChange, C.2C identity_formal_coordinate.

komorebi_math.coordinates.frame_algebra. 12 research cards. Namespace counts: coordinates 6, root 6. Cards: C.1C AnalysisSynthesisCoefficientBlocks, C.1C AnalysisSynthesisGauge, C.1C AnalysisSynthesisMap, C.1C CoefficientPartition, C.1C ExactAnalysisSynthesisMap, C.1C ExactAnalysisSynthesisSystem, C.2C AnalysisSynthesisCoefficientBlocks, C.2C AnalysisSynthesisGauge, C.2C AnalysisSynthesisMap, C.2C CoefficientPartition, C.2C ExactAnalysisSynthesisMap, C.2C ExactAnalysisSynthesisSystem.

komorebi_math.coordinates.modular. 3 research cards. Namespace counts: coordinates 2, root 1. Cards: C.1C FiniteFieldCoordinateTransform, C.2A FiniteFieldCoordinateTransform, C.2A reduce_exact_transform_mod_prime.

komorebi_math.coordinates.normalized. 4 research cards. Namespace counts: coordinates 3, root 1. Cards: C.1C normalization_change_transform, C.2A apply_lowering_polynomial, C.2A convert_normalized_coefficients, C.2A normalization_change_transform.

komorebi_math.coordinates.numeric. 9 research cards. Namespace counts: coordinates 7, root 2. Cards: C.1C discrete_fourier_transform, C.1C NumericCoordinateTransform, C.2C discrete_cosine_transform, C.2C discrete_fourier_transform, C.2C haar_wavelet_transform, C.2C modal_coordinate_system, C.2C ModalCoordinateSystem, C.2C NumericCoordinateTransform, C.2C walsh_hadamard_transform.

komorebi_math.coordinates.operators. 7 research cards. Namespace counts: coordinates 4, root 3. Cards: C.1C bezout_defect_identity, C.1C BezoutDefectIdentity, C.1C ShiftPolynomial, C.2C bezout_defect_identity, C.2C BezoutDefectIdentity, C.2C polynomial_extended_gcd, C.2C ShiftPolynomial.

komorebi_math.coordinates.packet. 10 research cards. Namespace counts: coordinates 9, root 1. Cards: C.1C residue_class_transform, C.2A chinese_remainder_order, C.2A chinese_remainder_transform, C.2A digit_reversal_order, C.2A digit_reversal_transform, C.2A residue_class_order, C.2A residue_class_transform, C.2A sorted_coordinate_transform, C.2A tensor_axis_permutation_order, C.2A tensor_axis_permutation_transform.

komorebi_math.coordinates.polynomials. 25 research cards. Namespace counts: coordinates 21, root 4. Cards: C.1C connection_transform, C.1C euler_to_fibonacci_transform, C.1C operator_change_of_coordinates, C.1C PolynomialFamily, C.2B connection_transform, C.2B derivative_operator_in_family, C.2B euler_polynomial, C.2B euler_polynomial_at_zero, C.2B euler_polynomial_family, C.2B euler_to_fibonacci_coefficient, C.2B euler_to_fibonacci_transform, C.2B fibonacci_companion_power, C.2B fibonacci_polynomial, C.2B fibonacci_polynomial_family, C.2B monomial_polynomial_family, C.2B monomial_to_fibonacci_coefficient, C.2B operator_change_of_coordinates, C.2B polynomial_add, C.2B polynomial_derivative, C.2B polynomial_evaluate, C.2B polynomial_multiply, C.2B polynomial_scale, C.2B polynomial_shift, C.2B PolynomialFamily, C.2B secant_euler_from_fibonacci_coordinates.

komorebi_math.coordinates.portable. 9 research cards. Namespace counts: coordinates 7, root 2. Cards: C.1C load_coordinate_object, C.1C save_coordinate_object, C.2C load_coordinate_object, C.2C load_coordinate_record, C.2C

load_transform, C.2C register_coordinate_codecs, C.2C save_coordinate_object, C.2C save_coordinate_record, C.2C save_transform.

komorebi_math.coordinates.realization. 15 research cards. Namespace counts: coordinates 9, root 6. Cards: C.1C intertwiner_space, C.1C IntertwinerSpace, C.1C LinearRealization, C.1C observability_matrix, C.1C recover_initial_state, C.1C StateRecovery, C.2C cyclic_krylov_matrix, C.2C intertwiner_space, C.2C IntertwinerSpace, C.2C LinearRealization, C.2C observability_matrix, C.2C realization_defect, C.2C recover_initial_state, C.2C similarity_from_cyclic_vectors, C.2C StateRecovery.

komorebi_math.coordinates.recurrence. 20 research cards. Namespace counts: coordinates 14, root 6. Cards: C.1C analyze_recurrence_coordinates, C.1C companion_matrix, C.1C coordinate_change_between_recurrences, C.1C euler_fibonacci_coordinates, C.1C RecurrenceCoordinateRecord, C.1C variation_of_constants_prefix, C.2C analyze_recurrence_coordinates, C.2C apply_recurrence_defect, C.2C companion_matrix, C.2C coordinate_change_between_recurrences, C.2C defect_green_weights, C.2C euler_fibonacci_coordinates, C.2C fibonacci_numbers, C.2C recurrence_analysis_transform, C.2C recurrence_polynomial, C.2C recurrence_synthesis_transform, C.2C RecurrenceCoordinateRecord, C.2C signed_secant_euler_numbers, C.2C synthesize_recurrence_coordinates, C.2C variation_of_constants_prefix.

komorebi_math.coordinates.state_space. 10 research cards. Namespace counts: coordinates 9, root 1. Cards: C.1C StateSpaceModel, C.2C certify_state_coordinate_change, C.2C controllability_basis, C.2C controllability_coordinate_transform, C.2C controllability_matrix, C.2C observability_block_matrix, C.2C observability_coordinate_transform, C.2C siso_state_space, C.2C StateSpaceCoordinateCertificate, C.2C StateSpaceModel.

komorebi_math.coordinates.tensor. 12 research cards. Namespace counts: coordinates 11, root 1. Cards: C.1C tensor_coordinate_transform, C.2A contragredient_transform, C.2A coordinate_pairing, C.2A direct_sum_transform, C.2A pairing_invariant, C.2A tensor_coordinate_transform, C.2A tensor_product_transform, C.2A transport_bilinear_form, C.2A transport_covector, C.2A transport_operator, C.2A transport_rectangular_operator, C.2A transport_tensor_components.

komorebi_math.coordinates.transform. 3 research cards. Namespace counts: coordinates 2, root 1. Cards: C.1C CoordinateTransform, C.2C CoordinateTransform, C.2C ExactCoordinateTransform.

C.7.15 Generating

komorebi_math.generating.adapters. 4 research cards. Namespace counts: generating 4. Cards: C.3A fir_taps_to_series, C.3A FIRSeriesAdapter, C.3A series_to_fir_adapter, C.3A series_to_fir_taps.

komorebi_math.generating.basis. 21 research cards. Namespace counts: coordinates 4, generating 13, root 4. Cards: C.1F factorial_power_basis, C.1F NormalizedBasis, C.1F ordinary_basis, C.1F WeightedMonomialBasis, C.2C exponential_basis, C.2C fibonacci_factorial_basis, C.2C NormalizedBasis, C.2C ordinary_basis, C.3A basis_from_descriptor, C.3A basis_from_values, C.3A BasisCertificate, C.3A BasisDescriptor, C.3A exponential_basis, C.3A factorial_power_basis, C.3A fibonacci_factorial_basis, C.3A NormalizedBasis, C.3A NormalizedMonomialBasis, C.3A ordinary_basis, C.3A pochhammer_basis, C.3A q_factorial_basis, C.3A WeightedMonomialBasis.

komorebi_math.generating.bell. 13 research cards. Namespace counts: generating 10, root 3. Cards: C.1F BellParityAutomaton, C.1F BellResidueAutomaton, C.1F generalized_bell_digit_reduction, C.3B bell_period_bound, C.3B BellDigitReduction, C.3B BellParityAutomaton, C.3B BellResidueAutomaton, C.3B certify_pointed_subindex_identity, C.3B generalized_bell_digit_reduction, C.3B generalized_bell_parity, C.3B pointed_subindex_polynomial, C.3B PointedSubindexCertificate, C.3B weighted_bell_digit_reduction.

komorebi_math.generating.budget. 3 research cards. Namespace counts: generating 2, root 1. Cards: C.1F GeneratingBudget, C.3A DEFAULT_GENERATING_BUDGET, C.3A GeneratingBudget.

komorebi_math.generating.cartier. 15 research cards. Namespace counts: generating 14, root 1. Cards: C.1F compile_rational_mask_automaton, C.3B AutomatonCertificate, C.3B cartier_degree_bound, C.3B CartierAutomaton, C.3B CartierCarryEquivalence, C.3B CartierLinearRepresentation, C.3B certify_cartier_carry_equivalence, C.3B compile_cartier_linear_representation, C.3B compile_rational_mask_automaton, C.3B finite_mask_carry_matrices, C.3B minimize_cartier_automaton, C.3B polynomial_cartier, C.3B polynomial_multiply_mod, C.3B polynomial_power_mod, C.3B rational_mask_coefficients_mod_prime.

komorebi_math.generating.closed_forms. 12 research cards. Namespace counts: generating 12. Cards: C.3A AlgebraicGFGuess, C.3A characteristic_closed_form, C.3A guess_algebraic_gf, C.3A guess_hypergeometric_ratio, C.3A

guess_linear_recurrence, C.3A guess_p_recursive, C.3A guess_quasipolynomial, C.3A HypergeometricGuess, C.3A LinearRecurrenceGuess, C.3A PRecursiveGuess, C.3A QuasiPolynomialGuess, C.3A rational_gf_from_recurrence.

komorebi_math.generating.congruences. 16 research cards. Namespace counts: generating 16. Cards: C.3B audit_apery_supercongruence, C.3B audit_factorial_power_packet_convolution, C.3B audit_generalized_bell_digit_sum, C.3B audit_generalized_lucas, C.3B audit_partition_lane, C.3B audit_q_lucas, C.3B audit_touchard, C.3B audit_weighted_generalized_bell_digit_sum, C.3B base_digit_sum, C.3B berlekamp_massey, C.3B difference_valuation_profile, C.3B mahler_coefficients, C.3B minimal_prefix_period, C.3B p_adic_valuation_fraction, C.3B scan_residue_lanes, C.3B search_decimation_relation.

komorebi_math.generating.cyclotomic. 6 research cards. Namespace counts: generating 6. Cards: C.3B cyclotomic_coboundary, C.3B CyclotomicCollapse, C.3B normalized_cyclotomic_expression, C.3B p_free_orbit, C.3B verify_cyclotomic_coboundary, C.3B verify_cyclotomic_exponent_relation.

komorebi_math.generating.digit_algebra. 6 research cards. Namespace counts: generating 6. Cards: C.3B audit_factorial_power_digit_algebra, C.3B base_digits, C.3B DigitAlgebraAudit, C.3B DigitKernel, C.3B factorial_power_digit_kernel, C.3B lift_digit_sequence.

komorebi_math.generating.domains. 9 research cards. Namespace counts: generating 9. Cards: C.3A CoefficientDomain, C.3A domain_from_descriptor, C.3A INTEGER_DOMAIN, C.3A IntegerDomain, C.3A PrimeFieldDomain, C.3A RATIONAL_DOMAIN, C.3A RationalDomain, C.3A ResidueRingDomain, C.3A TruncatedPadicDomain.

komorebi_math.generating.finite_series. 3 research cards. Namespace counts: generating 2, root 1. Cards: C.1F FiniteNormalizedSeries, C.3A FiniteNormalizedSeries, C.3A SeriesTransferCertificate.

komorebi_math.generating.functions. 13 research cards. Namespace counts: generating 11, root 2. Cards: C.1F CustomFunctionSpec, C.1F FunctionRegistry, C.3A algebraic_function, C.3A compose_functions, C.3A CustomFunctionSpec, C.3A euler_product_function, C.3A explicit_function, C.3A factorial_power_bell_function, C.3A FunctionRegistry, C.3A hypergeometric_function, C.3A linear_recurrence_function, C.3A rational_function, C.3A weighted_factorial_power_bell_function.

komorebi_math.generating.masks. 31 research cards. Namespace counts: generating 27, root 4. Cards: C.1F compile_finite_mask_carry_plan, C.1F FiniteMaskCarryPlan, C.1F mahler_mask_inverse, C.1F mahler_mask_product, C.3B analytic_mahler_product, C.3B AnalyticMaskApproximation, C.3B CarryLayerConvolution, C.3B certify_mahler_functional_equation, C.3B certify_rational_mask_algebraic_relation, C.3B compile_digit_cauchy_product, C.3B compile_finite_mask_carry_plan, C.3B DigitCauchyProductPlan, C.3B euler_product_collapse, C.3B EulerProductCollapse, C.3B factorial_power_convolution_layers, C.3B factorial_power_support_count, C.3B factorial_power_support_indices, C.3B FiniteMaskCarryPlan, C.3B first_frobenius_lift, C.3B frobenius_defect_mask, C.3B kummer_layer_distribution, C.3B KummerLayerDistribution, C.3B mahler_mask_inverse, C.3B mahler_mask_product, C.3B NormalizedMaskPlan, C.3B one_borrow_count, C.3B polynomial_mask_coboundary, C.3B PolynomialCoboundaryResult, C.3B RationalMaskAlgebraicCertificate, C.3B solve_polynomial_coboundary, C.3B zero_borrow_count.

komorebi_math.generating.monoid. 5 research cards. Namespace counts: generating 5. Cards: C.3A additive_series, C.3A dirichlet_convolution, C.3A FiniteMonoidSeries, C.3A multiplicative_series, C.3A prime_valuation_packets.

komorebi_math.generating.normalization. 6 research cards. Namespace counts: generating 6. Cards: C.3B CarryLinearCertificate, C.3B CarryLinearValuation, C.3B classify_carry_linear_valuation, C.3B digit_factorized_normalized_kernel, C.3B digit_unit, C.3B local_normalized_kernel.

komorebi_math.generating.padic_units. 9 research cards. Namespace counts: generating 8, root 1. Cards: C.1F PrimePowerKernelCompiler, C.3B binomial_valuation, C.3B borrow_count, C.3B factorial_valuation, C.3B integer_valuation, C.3B NormalizedKernelResidue, C.3B PrimePowerBinomial, C.3B PrimePowerKernelCompiler, C.3B PrimePowerUnitCompiler.

komorebi_math.generating.portable. 5 research cards. Namespace counts: generating 3, root 2. Cards: C.1F freeze_custom_function, C.1F FrozenGeneratingFunction, C.3A freeze_custom_function, C.3A FrozenGeneratingFunction, C.3A register_generating_codecs.

komorebi_math.generating.q_kummer. 9 research cards. Namespace counts: generating 8, root 1. Cards: C.1F q_binomial_valuation_decomposition, C.3B gaussian_binomial, C.3B multiplicative_order_mod_prime, C.3B q_binomial_support, C.3B q_binomial_valuation_decomposition, C.3B q_binomial_valuation_direct, C.3B q_factorial, C.3B q_integer, C.3B QKummerDecomposition.

komorebi_math.generating.sections. 5 research cards. Namespace counts: generating 5. Cards: C.3A audit_section_cycle, C.3A custom_exponential, C.3A custom_section, C.3A root_filter_value, C.3A SectionCycleAudit.

komorebi_math.generating.sequences. 12 research cards. Namespace counts: generating 12. Cards: C.3A apery_numbers, C.3A bell_numbers, C.3A bernoulli_numbers, C.3A catalan_numbers, C.3A central_binomial_numbers, C.3A fibonacci_numbers, C.3A generalized_bell_numbers, C.3A generalized_bell_polynomials, C.3A partition_numbers, C.3A signed_euler_numbers, C.3A weighted_generalized_bell_numbers, C.3A weighted_generalized_bell_polynomials.

komorebi_math.generating.series. 3 research cards. Namespace counts: generating 2, root 1. Cards: C.1F FormalSeries, C.3A FormalSeries, C.3A PacketIdentity.

C.7.16 Signals

komorebi_math.signals. 1 research card. Namespace counts: signals 1. Cards: C.4A status.

komorebi_math.signals.adaptive. 4 research cards. Namespace counts: signals 4. Cards: C.4B AdaptiveFilterResult, C.4B lms_filter, C.4B nlms_filter, C.4B rls_filter.

komorebi_math.signals.approximation. 5 research cards. Namespace counts: signals 5. Cards: C.4B estimate_positive_packet_1d, C.4B fft_lowpass_1d, C.4B fft_lowpass_2d, C.4B otsu_threshold, C.4B PacketEstimate1D.

komorebi_math.signals.convolution. 9 research cards. Namespace counts: signals 9. Cards: C.4A convolution_matrix, C.4A convolution_output_length, C.4A convolve_signal, C.4A finite_polyphase_blocks, C.4A finite_polyphase_filter_kernels, C.4A FinitePolyphaseKernel, C.4A polyphase_convolution, C.4A PolyphaseBlockMatrix, C.4A PolyphaseConvolution.

komorebi_math.signals.denoise. 10 research cards. Namespace counts: signals 10. Cards: C.4B denoising_risk_certificate, C.4B estimate_white_noise_sigma, C.4B haar_decompose, C.4B HaarDecomposition, C.4B median_absolute_deviation, C.4B sure_threshold, C.4B threshold_coefficients, C.4B universal_threshold, C.4B wavelet_denoise, C.4B WaveletDenoiseResult.

komorebi_math.signals.edge_recovery. 6 research cards. Namespace counts: signals 6. Cards: C.4B half_shift_distance, C.4B recover_vector_jumps, C.4B recover_vector_jumps_subgaussian, C.4B smooth_haar_coefficient_bound, C.4B step_haar_coefficient, C.4B VectorJumpRecoveryResult.

komorebi_math.signals.fft_convolution. 4 research cards. Namespace counts: signals 4. Cards: C.4A certified_overlap_add_convolve, C.4A fft_convolve, C.4A overlap_add_convolve, C.4A OverlapAddConvolver.

komorebi_math.signals.filterbanks. 12 research cards. Namespace counts: root 1, signals 11. Cards: C.1F PeriodicFilterBank, C.4B FilterBankReport, C.4B FiniteFilterBank, C.4B haar_analyze, C.4B haar_filter_bank, C.4B haar_matrix, C.4B haar_synthesize, C.4B HaarFilterBank, C.4B periodic_five_three_lifting, C.4B PeriodicFilterBank, C.4B PeriodicLiftingScheme, C.4B reconstruction_error.

komorebi_math.signals.filtering. 11 research cards. Namespace counts: signals 11. Cards: C.4A design_bandpass, C.4A design_bandstop, C.4A design_fir, C.4A design_highpass, C.4A design_lowpass, C.4A fir_filter, C.4A fir_frequency_response, C.4A FIRFilter, C.4A FrequencyResponse, C.4A stream_fir_filter, C.4A StreamingFIR.

komorebi_math.signals.nonlinear. 3 research cards. Namespace counts: signals 3. Cards: C.4B coefficient_packets, C.4B nonlinear_shift_filter, C.4B packet_product.

komorebi_math.signals.polyphase. 16 research cards. Namespace counts: root 2, signals 14. Cards: C.1F polyphase_analyze, C.1F PolyphaseSystem, C.4B lattice_polyphase_analyze, C.4B lattice_polyphase_synthesize, C.4B LatticePolyphaseCoordinates, C.4B mixed_radix_address, C.4B polyphase_analyze, C.4B polyphase_filter_kernels, C.4B polyphase_roundtrip_certificate, C.4B polyphase_synthesize, C.4B PolyphaseCoordinates, C.4B PolyphaseKernel, C.4B PolyphaseSystem, C.4B refinement_children, C.4B residue_project, C.4B split_mixed_radix_address.

komorebi_math.signals.recovery. 11 research cards. Namespace counts: root 1, signals 10. Cards: C.1F recover_separated_jumps, C.4B BreakpointBandResult, C.4B detect_piecewise_polynomial_breaks, C.4B finite_difference, C.4B haar_lipschitz_bound, C.4B haar_step_coefficient, C.4B JumpDetection, C.4B JumpRecoveryResult, C.4B recover_jumps_subgaussian, C.4B recover_separated_jumps, C.4B subgaussian_noise_bound.

komorebi_math.signals.recursive. 6 research cards. Namespace counts: signals 6. Cards: C.4A design_biquad, C.4A FilterOutput, C.4A lfilter, C.4A LinearFilter, C.4A stream_lfilter, C.4A StreamingLinearFilter.

komorebi_math.signals.resampling. 10 research cards. Namespace counts: signals 10. Cards: C.4A decimate, C.4A design_resample_filter, C.4A downsample, C.4A interpolate, C.4A RationalResamplingPlan, C.4A resample_poly, C.4A resample_poly_result, C.4A resampled_length, C.4A ResamplingResult, C.4A upsample.

komorebi_math.signals.singular_packets. 16 research cards. Namespace counts: root 2, signals 14. Cards: C.1F retained_packet_reconstruct, C.1F SingularPacket1D, C.4B difference_noise_bound, C.4B estimated_packet_reconstruct, C.4B overlap_matrix_error, C.4B overlap_partition_lp_error, C.4B retained_packet_reconstruct, C.4B retained_packet_reconstruction, C.4B RetainedPacketReconstruction, C.4B RetainedPacketResult, C.4B scalar_region_error, C.4B scalar_region_lp_error, C.4B SingularPacket1D, C.4B step_location_lp_error, C.4B TruncatedPowerPacket, C.4B TruncatedPowerTerm.

komorebi_math.signals.smith. 4 research cards. Namespace counts: signals 4. Cards: C.4B gamma_from_impedance, C.4B impedance_from_gamma, C.4B phase_mobius, C.4B smith_sensitivity.

komorebi_math.signals.spectral. 6 research cards. Namespace counts: signals 6. Cards: C.4A cross_spectrum, C.4A CrossSpectrum, C.4A parseval_certificate, C.4A periodogram, C.4A PowerSpectrum, C.4A welch_psd.

komorebi_math.signals.streaming. 3 research cards. Namespace counts: signals 3. Cards: C.4A stream_stft, C.4A StreamingSTFT, C.4A StreamingSTFTBatch.

komorebi_math.signals.time_frequency. 8 research cards. Namespace counts: signals 8. Cards: C.4A frame_signal, C.4A FrameBatch, C.4A istft, C.4A overlap_add, C.4A spectrogram, C.4A SpectrogramResult, C.4A stft, C.4A STFTResult.

komorebi_math.signals.wavelets. 11 research cards. Namespace counts: root 1, signals 10. Cards: C.1F HaarPacketTree, C.4B best_basis, C.4B BestBasisResult, C.4B entropy_cost, C.4B haar_packet_analyze, C.4B haar_packet_synthesize, C.4B HaarPacketTree, C.4B lp_cost, C.4B packet_energy, C.4B PrunedPacketCoefficients, C.4B shannon_entropy_cost.

komorebi_math.signals.windows. 6 research cards. Namespace counts: signals 6. Cards: C.4A canonical_dual_window, C.4A get_window, C.4A overlap_add_certificate, C.4A overlap_profile, C.4A window_metrics, C.4A WindowMetrics.

C.7.17 Nonlinear

komorebi_math.nonlinear. 2 research cards. Namespace counts: nonlinear 2. Cards: C.5B require, C.5A status.

komorebi_math.nonlinear.autodiff. 16 research cards. Namespace counts: nonlinear 8, root 8. Cards: C.1E autodiff_provider, C.1E automatic_derivative_provider, C.1E AutomaticDifferentiationProvider, C.1E available_ad_frameworks, C.1E available_autodiff_backends, C.1E DirectionalJVP, C.1E JaxDerivativeProvider, C.1E TorchDerivativeProvider, C.5A autodiff_provider, C.5A automatic_derivative_provider, C.5A AutomaticDifferentiationProvider, C.5A available_ad_frameworks, C.5A available_autodiff_backends, C.5A DirectionalJVP, C.5A JaxDerivativeProvider, C.5A TorchDerivativeProvider.

komorebi_math.nonlinear.composition. 11 research cards. Namespace counts: nonlinear 8, root 3. Cards: C.1E ComposedDerivativeProvider, C.1E faa_di_bruno, C.1E packet_faa_di_bruno, C.5A ComposedDerivativeProvider, C.5A faa_di_bruno, C.5A FaaDiBrunoResult, C.5A FaaDiBrunoTerm, C.5A labeled_composition_term_count, C.5A packet_faa_di_bruno, C.5A set_partitions, C.5A stirling_second.

komorebi_math.nonlinear.coordinates. 2 research cards. Namespace counts: nonlinear 1, root 1. Cards: C.1E NonlinearPacketMap, C.5A NonlinearPacketMap.

komorebi_math.nonlinear.correction. 10 research cards. Namespace counts: nonlinear 7, root 3. Cards: C.1E HilbertObstructionProjector, C.1E LyapunovSchmidtReduction, C.1E RightInverseCorrectionProblem, C.5A HilbertObstructionProjector, C.5A HilbertObstructionResult, C.5A LyapunovSchmidtDecomposition, C.5A LyapunovSchmidtPoint, C.5A LyapunovSchmidtReduction, C.5A RightInverseCorrectionProblem, C.5A RightInverseCorrectionResult.

komorebi_math.nonlinear.defects. 2 research cards. Namespace counts: nonlinear 1, root 1. Cards: C.1E NonlinearRefinementDefect, C.5A NonlinearRefinementDefect.

komorebi_math.nonlinear.derivatives. 9 research cards. Namespace counts: nonlinear 5, root 4. Cards: C.1E CallableDerivativeProvider, C.1E FiniteDifferenceDerivativeProvider, C.1E PolynomialDerivativeProvider, C.1E

SympyDerivativeProvider, C.5A CallableDerivativeProvider, C.5A DerivativeProvider, C.5A FiniteDifferenceDerivativeProvider, C.5A PolynomialDerivativeProvider, C.5A SympyDerivativeProvider.

komorebi_math.nonlinear.elimination. 5 research cards. Namespace counts: nonlinear 4, root 1. Cards: C.1E NonlinearSchurElimination, C.5A HiddenSolveResult, C.5A NonlinearSchurElimination, C.5A SchurDerivativeResult, C.5A SchurSecondDerivativeResult.

komorebi_math.nonlinear.execution. 22 research cards. Namespace counts: nonlinear 12, root 10. Cards: C.1E compile_packet_jet_plan, C.1E CuPyDeviceDerivativeProvider, C.1E CuPyPacketJetExecutor, C.1E CuPyPolynomialDerivativeProvider, C.1E execute_packet_jet_plan, C.1E execute_packet_jets, C.1E nonlinear_cupy_available, C.1E packet_jet_benchmark_contract, C.1E PacketJetBatchResult, C.1E PacketJetExecutionPlan, C.5B compile_packet_jet_plan, C.5B CuPyDeviceDerivativeProvider, C.5B CuPyPacketJetExecutor, C.5B CuPyPolynomialDerivativeProvider, C.5B execute_packet_jet_plan, C.5B execute_packet_jets, C.5B nonlinear_cupy_available, C.5B packet_jet_benchmark_contract, C.5B PacketJetBatchResult, C.5B PacketJetEvaluation, C.5B PacketJetExecutionPlan, C.5B PacketJetInputGroup.

komorebi_math.nonlinear.filtered. 16 research cards. Namespace counts: nonlinear 11, root 5. Cards: C.1E conditional_projective_limit_completion, C.1E FilteredCorrectionProblem, C.1E FilteredVectorSpace, C.1E FiniteFilteredCochainComplex, C.1E verify_finite_filtered_branch, C.5A conditional_projective_limit_completion, C.5A DepthObstructionResult, C.5A FilteredCorrectionProblem, C.5A FilteredSpectralSequenceResult, C.5A FilteredVectorSpace, C.5A FiniteFilteredBranchResult, C.5A FiniteFilteredCochainComplex, C.5A ProjectiveLimitRecord, C.5A SpectralSequenceEntry, C.5A SpectralSequencePage, C.5A verify_finite_filtered_branch.

komorebi_math.nonlinear.gpu. 12 research cards. Namespace counts: nonlinear 6, root 6. Cards: C.1E cupy_runtime_status, C.1E CuPyPacketJetSnapshotExecutor, C.1E CuPyRuntimeStatus, C.1E CuPySnapshotExecutor, C.1E CuPyValidationResult, C.1E validate_cupy_snapshot, C.5B cupy_runtime_status, C.5B CuPyPacketJetSnapshotExecutor, C.5B CuPyRuntimeStatus, C.5B CuPySnapshotExecutor, C.5B CuPyValidationResult, C.5B validate_cupy_snapshot.

komorebi_math.nonlinear.graded. 16 research cards. Namespace counts: nonlinear 11, root 5. Cards: C.1E divided_square_packets, C.1E FiniteAbelianGroup, C.1E GradedPacketOperation, C.1E higher_maurer_cartan_packets, C.1E maurer_cartan_packets, C.5A divided_square_packets, C.5A DividedSquareResult, C.5A FiniteAbelianGroup, C.5A GradedConvolutionResult, C.5A GradedConvolutionTerm, C.5A GradedPacketOperation, C.5A GradingCertificate, C.5A higher_maurer_cartan_packets, C.5A HigherMaurerCartanResult, C.5A maurer_cartan_packets, C.5A MaurerCartanResult.

komorebi_math.nonlinear.holonomy. 6 research cards. Namespace counts: nonlinear 4, root 2. Cards: C.1E TransportEdge, C.1E TransportGraph, C.5A FundamentalHolonomy, C.5A IntertwiningResult, C.5A TransportEdge, C.5A TransportGraph.

komorebi_math.nonlinear.jets. 11 research cards. Namespace counts: nonlinear 7, root 4. Cards: C.1E packet_jet_support, C.1E packet_taylor, C.1E PacketJet, C.1E PacketJetFamily, C.5B packet_jet_support, C.5B packet_taylor, C.5B PacketJet, C.5B PacketJetAddress, C.5B PacketJetFamily, C.5B PacketJetSupport, C.5B PacketTaylorResult.

komorebi_math.nonlinear.lifting. 13 research cards. Namespace counts: nonlinear 8, root 5. Cards: C.1E build_lift_tree, C.1E FiniteLiftTree, C.1E LinearLiftProblem, C.1E PolynomialLiftProblem, C.1E solve_mod_prime, C.5A build_lift_tree, C.5A FiniteLiftTree, C.5A LiftStepResult, C.5A LiftTreeNode, C.5A LinearLiftProblem, C.5A ModularLinearSolution, C.5A PolynomialLiftProblem, C.5A solve_mod_prime.

komorebi_math.nonlinear.obstruction. 4 research cards. Namespace counts: nonlinear 3, root 1. Cards: C.1E LinearObstructionProblem, C.5A LinearObstructionProblem, C.5A ObstructionResult, C.5A ObstructionSquare.

komorebi_math.nonlinear.profiling. 14 research cards. Namespace counts: nonlinear 7, root 7. Cards: C.1E estimate_packet_jet_complexity, C.1E PacketJetComplexityEstimate, C.1E PacketJetProfileReport, C.1E profile_operation, C.1E profile_packet_jet_runtime, C.1E RuntimeProfileSample, C.1E RuntimeStageProfile, C.5B estimate_packet_jet_complexity, C.5B PacketJetComplexityEstimate, C.5B PacketJetProfileReport, C.5B profile_operation, C.5B profile_packet_jet_runtime, C.5B RuntimeProfileSample, C.5B RuntimeStageProfile.

komorebi_math.nonlinear.snapshots. 5 research cards. Namespace counts: nonlinear 3, root 2. Cards: C.1E PacketJetSnapshot, C.1E snapshot_packet_jet, C.5B PacketJetCoefficient, C.5B PacketJetSnapshot, C.5B snapshot_packet_jet.

komorebi_math.nonlinear.sparse. 6 research cards. Namespace counts: nonlinear 3, root 3. Cards: C.1E SparsePolynomialDerivativeProvider, C.1E SparseSymmetricTensor, C.1E SparseTensorEntry, C.5B SparsePolynomialDerivativeProvider, C.5B SparseSymmetricTensor, C.5B SparseTensorEntry.

komorebi_math.nonlinear.streaming. 20 research cards. Namespace counts: nonlinear 10, root 10. Cards: C.1E capture_packet_jet_stream, C.1E load_packet_jet_snapshot_stream, C.1E open_packet_jet_snapshot_stream, C.1E PacketJetSnapshotStreamReader, C.1E PacketJetStreamChunkRecord, C.1E PacketJetStreamManifest, C.1E PacketJetStreamSelection, C.1E PacketJetStreamWriteResult, C.1E save_packet_jet_snapshot_stream, C.1E StreamedPacketJetEvaluation, C.5B capture_packet_jet_stream, C.5B load_packet_jet_snapshot_stream, C.5B open_packet_jet_snapshot_stream, C.5B PacketJetSnapshotStreamReader, C.5B PacketJetStreamChunkRecord, C.5B PacketJetStreamManifest, C.5B PacketJetStreamSelection, C.5B PacketJetStreamWriteResult, C.5B save_packet_jet_snapshot_stream, C.5B StreamedPacketJetEvaluation.

komorebi_math.nonlinear.transport. 9 research cards. Namespace counts: nonlinear 6, root 3. Cards: C.1E direct_packet_rotation, C.1E KatoTransport, C.1E MovingPacketSystem, C.5A direct_packet_rotation, C.5A DirectRotationResult, C.5A KatoTransport, C.5A MovingPacketSystem, C.5A PacketDerivativeResult, C.5A TransportRefinementResult.

komorebi_math.nonlinear.trees. 18 research cards. Namespace counts: nonlinear 13, root 5. Cards: C.1E majorant_convergence, C.1E OrderedRootedTree, C.1E packet_tree_reconstruction, C.1E PolynomialFixedPointProblem, C.1E quadratic_catalan_majorant, C.5B catalan_number, C.5B majorant_convergence, C.5B MajorantConvergence, C.5B ordered_rooted_trees, C.5B ordered_tree_count, C.5B ordered_trees_with_leaves, C.5B OrderedRootedTree, C.5B packet_tree_reconstruction, C.5B PacketTreeResult, C.5B PolynomialFixedPointProblem, C.5B quadratic_catalan_majorant, C.5B TreeExpansionResult, C.5B TreeTerm.

C.7.18 Integration

komorebi_math.integration.adapters. 6 research cards. Namespace counts: root 6. Cards: C.1D analysis_synthesis_integration_certificate, C.1D AtomicIntegratedDerivativeProvider, C.1D integrate_formal_series, C.1D integrate_polynomial_coordinates, C.1D polynomial_from_family_coordinates, C.1D polynomial_from_formal_series.

komorebi_math.integration.coordinates. 4 research cards. Namespace counts: root 4. Cards: C.1D coordinate_fundamental_theorem, C.1D coordinate_transport_identity, C.1D polynomial_integration_covector, C.1D PolynomialIntegrationCovector.

komorebi_math.integration.identities. 7 research cards. Namespace counts: root 7. Cards: C.1D affine_interval_change_of_variables, C.1D all_fubini_orders, C.1D fubini_identity, C.1D fundamental_theorem, C.1D IntegralIdentity, C.1D integration_by_parts, C.1D pushforward_identity.

komorebi_math.integration.measures. 8 research cards. Namespace counts: root 8. Cards: C.1D AtomicMeasure, C.1D ExactAffineMap, C.1D IntervalMeasure, C.1D Measure, C.1D product_measure, C.1D ProductMeasure, C.1D pushforward_measure, C.1D PushforwardMeasure.

komorebi_math.integration.polynomial. 1 research card. Namespace counts: root 1. Cards: C.1D ExactPolynomial.

komorebi_math.integration.quadrature. 9 research cards. Namespace counts: root 9. Cards: C.1D CompositeSimpson, C.1D DerivativeBound, C.1D ErrorGuarantee, C.1D IntegralResult, C.1D integrate, C.1D integrate_derivative_provider, C.1D integrate_polynomial_oriented, C.1D IntegrationError, C.1D polynomial_derivative_bound.

komorebi_math.integration.rules. 3 research cards. Namespace counts: root 3. Cards: C.1D ExactQuadratureRule, C.1D QuadratureExactness, C.1D QuadratureRule.

C.7.19 Dynamics

komorebi_math.dynamics.bayes. 1 research card. Namespace counts: root 1. Cards: C.1G BayesianUpdate.

komorebi_math.dynamics.feshbach. 1 research card. Namespace counts: root 1. Cards: C.1G FeshbachReduction.

komorebi_math.dynamics.fractional. 1 research card. Namespace counts: root 1. Cards: C.1G fractional_power.

komorebi_math.dynamics.markov. 1 research card. Namespace counts: root 1. Cards: C.1G MarkovLumping.

komorebi_math.dynamics.memory. 1 research card. Namespace counts: root 1. Cards: C.1G MemoryKernel.

komorebi_math.dynamics.semigroups. 1 research card. Namespace counts: root 1. Cards: C.1G MatrixSemigroup.

komorebi_math.dynamics.survival. 1 research card. Namespace counts: root 1. Cards: C.1G PhaseTypeSurvival.

C.7.20 Quantum

komorebi_math.quantum.channels. 1 research card. Namespace counts: root 1. Cards: C.1G QuantumChannel.

komorebi_math.quantum.circuits. 1 research card. Namespace counts: root 1. Cards: C.1G CircuitPacketExpansion.

komorebi_math.quantum.constraints. 1 research card. Namespace counts: root 1. Cards: C.1G constraint_closure.

komorebi_math.quantum.effective. 1 research card. Namespace counts: root 1. Cards: C.1G QuantumFeshbachReduction.

komorebi_math.quantum.error_correction. 1 research card. Namespace counts: root 1. Cards: C.1G QuantumCode.

komorebi_math.quantum.euclidean. 1 research card. Namespace counts: root 1. Cards: C.1G EuclideanTransfer.

komorebi_math.quantum.leakage. 1 research card. Namespace counts: root 1. Cards: C.1G fixed_packet_leakage.

komorebi_math.quantum.lindblad. 1 research card. Namespace counts: root 1. Cards: C.1G LindbladGenerator.

komorebi_math.quantum.seeds. 1 research card. Namespace counts: root 1. Cards: C.1G QuantumSeedSystem.

komorebi_math.quantum.spectral. 2 research cards. Namespace counts: root 2. Cards: C.1G periodic_laplacian, C.1G spectral_dimension.

komorebi_math.quantum.symmetry. 1 research card. Namespace counts: root 1. Cards: C.1G cyclic_symmetry_projectors.

C.7.21 Deformation

komorebi_math.deformation.companions. 2 research cards. Namespace counts: root 2. Cards: C.1G finite_frobenius_separator, C.1G FrobeniusSpine.

komorebi_math.deformation.complexes. 1 research card. Namespace counts: root 1. Cards: C.1G FiniteCochainComplex.

komorebi_math.deformation.forests. 1 research card. Namespace counts: root 1. Cards: C.1G FiniteLiftForest.

komorebi_math.deformation.support. 1 research card. Namespace counts: root 1. Cards: C.1G PresentedModule.

komorebi_math.deformation.visibility. 1 research card. Namespace counts: root 1. Cards: C.1G VisibilityTree.

C.7.22 Langlands

komorebi_math.langlands.completion.gates. 4 research cards. Namespace counts: root 4. Cards: C.1G AutomorphicCompletionReport, C.1G CompatiblePropagationReport, C.1G CompletionGate, C.1G GateStatus.

komorebi_math.langlands.finite_image.characters. 2 research cards. Namespace counts: root 2. Cards: C.1G InducedRepresentation, C.1G LinearCharacter.

komorebi_math.langlands.finite_image.clifford. 2 research cards. Namespace counts: root 2. Cards: C.1G CliffordStep, C.1G CliffordTower.

komorebi_math.langlands.finite_image.groups. 2 research cards. Namespace counts: root 2. Cards: C.1G cyclic_group, C.1G FiniteGroup.

komorebi_math.langlands.finite_image.mackey. 1 research card. Namespace counts: root 1. Cards: C.1G mackey_tensor_packets.

komorebi_math.langlands.finite_image.orbits. 1 research card. Namespace counts: root 1. Cards: C.1G MonomialAction.

komorebi_math.langlands.finite_image.plans. 1 research card. Namespace counts: root 1. Cards: C.1G AutomorphicInductionPlan.

komorebi_math.langlands.global_.dyadic. 1 research card. Namespace counts: root 1. Cards: C.1G dyadic_sign_diagnostic.

komorebi_math.langlands.global_.hecke. 2 research cards. Namespace counts: root 2. Cards: C.1G FiniteHeckePackets, C.1G SupportComparison.

komorebi_math.langlands.global_.patching. 1 research card. Namespace counts: root 1. Cards: C.1G PatchingSupportReport.

komorebi_math.langlands.local.envelopes. 1 research card. Namespace counts: root 1. Cards: C.1G ProjectiveIntertwinerEnvelope.

komorebi_math.langlands.local.finite_fields. 1 research card. Namespace counts: root 1. Cards: C.1G FiniteField.

komorebi_math.langlands.local.gauss. 1 research card. Namespace counts: root 1. Cards: C.1G gauss_sum.

komorebi_math.langlands.local.heisenberg. 1 research card. Namespace counts: root 1. Cards: C.1G HeisenbergModel.

komorebi_math.langlands.local.symplectic. 2 research cards. Namespace counts: root 2. Cards: C.1G AlternatingForm, C.1G QuadraticRefinement.

komorebi_math.langlands.local.tame. 2 research cards. Namespace counts: root 2. Cards: C.1G FrobeniusOrbit, C.1G TameWeilPacket.

komorebi_math.langlands.ramification.towers. 3 research cards. Namespace counts: root 3. Cards: C.1G ProjectiveMultiplicityPacket, C.1G RamificationCliffordTower, C.1G RamificationStage.

C.7.23 Expression

komorebi_math.expression.certification. 7 research cards. Namespace counts: root 7. Cards: C.1B compose_certificates, C.1B exact_internal_certificate, C.1B exactness_strength, C.1B ExpressionBinding, C.1B NormalForm, C.1B RewriteStep, C.1B weakest_exactness.

komorebi_math.expression.engine. 15 research cards. Namespace counts: root 15. Cards: C.1B check_expression, C.1B CheckedExpression, C.1B compile_expression, C.1B EquivalenceResult, C.1B equivalent, C.1B evaluate_expression, C.1B exact_binding, C.1B execute_plan, C.1B ExpressionPlan, C.1B ExpressionResult, C.1B normal_form_digest, C.1B normalize_expression, C.1B PlanInstruction, C.1B SimplificationResult, C.1B simplify_expression.

komorebi_math.expression.model. 28 research cards. Namespace counts: root 28. Cards: C.1B add, C.1B Apply, C.1B apply, C.1B Compose, C.1B compose, C.1B exact_map, C.1B exact_value, C.1B ExactMapLiteral, C.1B ExactValueLiteral, C.1B Expression, C.1B ExpressionBudget, C.1B ExpressionStats, C.1B identity, C.1B IdentityMap, C.1B Let, C.1B let, C.1B LinearMapType, C.1B PolynomialCoordinateSpace, C.1B RATIONAL_SPACE, C.1B RationalSpace, C.1B Scale, C.1B scale, C.1B Sum, C.1B Symbol, C.1B symbol, C.1B then, C.1B zero_map, C.1B ZeroMap.

komorebi_math.expression.planning. 5 research cards. Namespace counts: root 5. Cards: C.1B matrix_chain_multiply, C.1B matrix_chain_parenthesization, C.1B matrix_chain_plan, C.1B MatrixChainPlan, C.1B MatrixChainTree.

komorebi_math.expression.polynomial. 15 research cards. Namespace counts: root 15. Cards: C.1B affine_substitution, C.1B antiderivative, C.1B boundary_functional, C.1B constant_injection, C.1B coordinate_change, C.1B derivative, C.1B evaluation, C.1B finite_difference, C.1B interval_integration, C.1B PolynomialPrimitive, C.1B primitive_certificate, C.1B primitive_matrix_equal, C.1B PrimitiveKind, C.1B replay, C.1B replay_matrix.

komorebi_math.expression.portable. 7 research cards. Namespace counts: root 7. Cards: C.1B expression_digest, C.1B expression_from_dict, C.1B expression_from_json, C.1B expression_to_dict, C.1B expression_to_json, C.1B type_from_dict, C.1B type_to_dict.

C.7.24 Export

komorebi_math.export.latex. 10 research cards. Namespace counts: export 6, root 4. Cards: C.1F available_latex_engines, C.1F compile_latex, C.1F LatexCompileOptions, C.1F LatexEngine, C.6 available_latex_engines, C.6 compile_latex, C.6 LatexCompilation, C.6 LatexCompileOptions, C.6 LatexEngine, C.6 resolve_latex_engine.

komorebi_math.export.model. 14 research cards. Namespace counts: export 10, root 4. Cards: C.1F DocumentMetadata, C.1F Equation, C.1F MathDocument, C.1F Table, C.6 CodeBlock, C.6 DocumentMetadata, C.6 Equation, C.6 Figure, C.6 Heading, C.6 ListBlock, C.6 MathDocument, C.6 Paragraph, C.6 RawLatex, C.6 Table.

komorebi_math.export.persistence. 24 research cards. Namespace counts: export 13, root 11. Cards: C.1F dumps_portable, C.1F load_portable, C.1F loads_portable, C.1F portable_codecs, C.1F portable_digest, C.1F portable_migrations, C.1F PortableCodec, C.1F PortableMigration, C.1F register_portable_codec, C.1F register_portable_

migration, C.1F save_portable, C.6 dumps_portable, C.6 from_portable_data, C.6 load_portable, C.6 loads_portable, C.6 portable_codecs, C.6 portable_digest, C.6 portable_migrations, C.6 PortableCodec, C.6 PortableMigration, C.6 register_portable_codec, C.6 register_portable_migration, C.6 save_portable, C.6 to_portable_data.

komorebi_math.export.renderers. 10 research cards. Namespace counts: export 8, root 2. Cards: C.1F LatexRenderOptions, C.1F render_latex, C.6 HtmlRenderOptions, C.6 latex_escape, C.6 latex_label, C.6 LatexRenderOptions, C.6 render_html, C.6 render_latex, C.6 render_markdown, C.6 render_text.

komorebi_math.export.report. 5 research cards. Namespace counts: export 3, root 2. Cards: C.1F document_from_data, C.1F sequence_document, C.6 certificate_document, C.6 document_from_data, C.6 sequence_document.

komorebi_math.export.serialization. 8 research cards. Namespace counts: export 6, root 2. Cards: C.1F from_serializable, C.1F to_serializable, C.6 export_adapters, C.6 Exportable, C.6 from_serializable, C.6 register_export_adapter, C.6 table_rows, C.6 to_serializable.

komorebi_math.export.writers. 19 research cards. Namespace counts: export 10, root 9. Cards: C.1F available_export_formats, C.1F export_bundle, C.1F export_data, C.1F export_document, C.1F export_many, C.1F ExportBundleResult, C.1F ExportFormat, C.1F ExportOptions, C.1F ExportResult, C.6 available_export_formats, C.6 export_bundle, C.6 export_data, C.6 export_document, C.6 export_many, C.6 ExportBundleResult, C.6 ExportFormat, C.6 ExportOptions, C.6 ExportResult, C.6 infer_export_format.

C.7.25 External Python and typing owners

builtins. 29 research cards. Namespace counts: coordinates 5, export 2, nonlinear 6, root 15, signals 1. Cards: C.1A **version**, C.1A API_STABILITY_LEVEL, C.1A BEHAVIORAL_GOLDEN_SCHEMA, C.1A BEHAVIORAL_GOLDEN_SCHEMA_VERSION, C.1A BETA_API_STABILITY_LEVEL, C.1A BETA_PUBLIC_NAMESPACES, C.1A EXPRESSION_NODE_VERSION, C.1A EXPRESSION_SCHEMA, C.1A EXPRESSION_SCHEMA_VERSION, C.1A EXPRESSION_TYPE_VERSION, C.1A PACKET_JET_CHUNK_SCHEMA, C.1A PACKET_JET_CHUNK_SCHEMA_VERSION, C.1A PACKET_JET_STREAM_SCHEMA, C.1A PACKET_JET_STREAM_SCHEMA_VERSION, C.1A STAGES, C.2C Matrix, C.2C ModularMatrix, C.2C ModularVector, C.2C Polynomial, C.2C Vector, C.6 PORTABLE_SCHEMA, C.6 PORTABLE_SCHEMA_VERSION, C.5B FEATURE_KEY, C.5B PACKET_JET_CHUNK_SCHEMA, C.5B PACKET_JET_CHUNK_SCHEMA_VERSION, C.5B PACKET_JET_STREAM_SCHEMA, C.5B PACKET_JET_STREAM_SCHEMA_VERSION, C.5A Partition, C.4A FEATURE_KEY.

collections.abc. 3 research cards. Namespace counts: nonlinear 2, root 1. Cards: C.1E MultilinearOperation, C.5A DirectionalDerivative, C.5B MultilinearOperation.

pathlib. 2 research cards. Namespace counts: root 2. Cards: C.1G DEFAULT_BEHAVIORAL_GOLDEN_FIXTURE, C.1G DEFAULT_BETA_API_MANIFEST.

types. 5 research cards. Namespace counts: coordinates 1, nonlinear 1, root 2, signals 1. Cards: C.1A CoordinateSpace, C.1A ExpressionType, C.2C Scalar, C.5B PacketLabel, C.4A PacketPath.

typing. 16 research cards. Namespace counts: coordinates 2, signals 14. Cards: C.2C RecurrenceLike, C.2C Variance, C.4A AdaptiveMethod, C.4A BiquadKind, C.4A BoundaryMode, C.4A ConvolutionMode, C.4A DetrendMode, C.4A FIRKind, C.4A FIRMode, C.4A SpectralAverage, C.4A SpectralScaling, C.4A SpectrogramScaling, C.4A ThresholdMode, C.4A ThresholdRule, C.4A WindowName, C.4A WindowSpec.

C.8 Alphabetical research exported-name index

The research manifest contains 1,093 distinct exported names across 1,312 namespace and name pairs. A name can occur in as many as three research namespaces. The 217 repeated-name groups below retain every route instead of collapsing a domain export into its root facade.

Quantity	Count
Distinct research exported names	1,093
Research namespace and name pairs	1,312
Names present in more than one research namespace	217
Maximum research namespaces for one name	3

C.8.1 Names beginning with A

`AdaptiveFilterResult`. 1 research route: signals. `AdaptiveMethod`. 1 research route: signals. `add`. 1 research route: root. `additive_series`. 1 research route: generating. `affine_basis_transform`. 1 research route: coordinates. `affine_interval_change_of_variables`. 1 research route: root. `affine_mahler_kernel`. 1 research route: root. `affine_polynomial`. 1 research route: coordinates. `affine_substitution`. 1 research route: root. `affine_substitution_coordinate_operator`. 1 research route: coordinates. `affine_substitution_transform`. 1 research route: coordinates. `AffineCoordinateTransform`. 2 research routes: root, coordinates. `AlgebraBasisResult`. 1 research route: root. `algebraic_function`. 1 research route: generating. `AlgebraicGFGuess`. 1 research route: generating. `all_fubini_orders`. 1 research route: root. `AlternatingForm`. 1 research route: root. `analysis_synthesis_integration_certificate`. 1 research route: root. `AnalysisSynthesisBlocks`. 2 research routes: root, coordinates. `AnalysisSynthesisCoefficientBlocks`. 2 research routes: root, coordinates. `AnalysisSynthesisGauge`. 2 research routes: root, coordinates. `AnalysisSynthesisMap`. 2 research routes: root, coordinates. `AnalysisSynthesisSystem`. 2 research routes: root, coordinates. `analytic_mahler_product`. 1 research route: generating. `AnalyticMaskApproximation`. 1 research route: generating. `analyze_recurrence_coordinates`. 2 research routes: root, coordinates. `antiderivative`. 1 research route: root. `apery_numbers`. 1 research route: generating. `apery_sequence`. 1 research route: root. `API_STABILITY_LEVEL`. 1 research route: root. `ApiCompatibilityReport`. 1 research route: root. `appell_polynomial`. 2 research routes: root, coordinates. `appell_polynomial_family`. 1 research route: coordinates. `Apply`. 1 research route: root. `apply`. 1 research route: root. `apply_lowering_polynomial`. 1 research route: coordinates. `apply_recurrence_defect`. 1 research route: coordinates. `Arrow`. 1 research route: root. `as_matrix`. 1 research route: coordinates. `as_vector`. 1 research route: coordinates. `assign_discrete_precisions`. 1 research route: root. `AtlasPath`. 1 research route: coordinates. `AtomicIntegratedDerivativeProvider`. 1 research route: root. `AtomicMeasure`. 1 research route: root. `audit_apery_supercongruence`. 1 research route: generating. `audit_factorial_power_digit_algebra`. 1 research route: generating. `audit_factorial_power_packet_convolution`. 1 research route: generating. `audit_generalized_bell_digit_sum`. 1 research route: generating. `audit_generalized_lucas`. 1 research route: generating. `audit_partition_lane`. 1 research route: generating. `audit_q_lucas`. 1 research route: generating. `audit_section_cycle`. 1 research route: generating. `audit_touchard`. 1 research route: generating. `audit_weighted_generalized_bell_digit_sum`. 1 research route: generating. `autodiff_provider`. 2 research routes: root, nonlinear. `automatic_derivative_provider`. 2 research routes: root, nonlinear. `AutomaticDifferentiationProvider`. 2 research routes: root, nonlinear. `AutomaticSequence`. 1 research route: root. `AutomatonCertificate`. 1 research route: generating. `AutomorphicCompletionReport`. 1 research route: root. `AutomorphicInductionPlan`. 1 research route: root. `available_ad_frameworks`. 2 research routes: root, nonlinear. `available_autodiff_backends`. 2 research routes: root, nonlinear. `available_export_formats`. 2 research routes: root, export. `available_latex_engines`. 2 research routes: root, export.

C.8.2 Names beginning with B

`BarycentricCoordinateSystem`. 1 research route: coordinates. `base_digit_sum`. 1 research route: generating. `base_digits`. 1 research route: generating. `basis_from_descriptor`. 1 research route: generating. `basis_from_matrix`. 2 research routes: root, coordinates. `basis_from_values`. 1 research route: generating. `BasisCertificate`. 1 research route: generating. `BasisDescriptor`. 1 research route: generating. `BayesianUpdate`. 1 research route: root. `BEHAVIORAL_GOLDEN_SCHEMA`. 1 research route: root. `BEHAVIORAL_GOLDEN_SCHEMA_VERSION`. 1 research route: root. `BehavioralGoldenCaseResult`. 1 research route: root. `BehavioralGoldenReport`. 1 research route: root. `bell_numbers`. 1 research route: generating. `bell_period_bound`. 1 research route: generating. `BellDigitReduction`. 1 research route: generating. `BellParityAutomaton`. 2 research routes: root, generating. `BellResidueAutomaton`. 2 research routes: root, generating. `berlekamp_massey`. 1 research route: generating. `bernoulli_number`. 1 research route: coordinates. `bernoulli_numbers`. 1 research route: generating. `bernoulli_polynomial`. 1 research route: coordinates. `bernoulli_polynomial_family`. 1 research route: coordinates. `bernstein_basis`. 1 research route: coordinates. `bernstein_polynomial`. 1 research route: coordinates. `best_basis`. 1 research route: signals. `BestBasisResult`. 1 research route: signals. `BETA_API_STABILITY_LEVEL`. 1 research route: root. `BETA_PUBLIC_NAMESPACES`. 1 research route: root. `bezout_defect_identity`. 2 research routes: root, coordinates. `BezoutDefectIdentity`. 2 research routes: root, coordinates. `binomial_polynomial`. 1 research route: coordinates. `binomial_polynomial_family`. 1 research route: coordinates. `binomial_transform`. 2 research routes: root, coordinates. `binomial_valuation`. 1 research route: generating. `BiquadKind`. 1 research route: signals. `block_from_paths`. 1 research route: root. `BlockDiagonalLowRankOperator`. 1 research route: root. `BlockDiagonalOperator`. 1 research route: root. `borrow_count`. 1 research route: generating. `boundary_functional`. 1 research route: root. `BoundaryMode`. 1 research route: signals. `BreakpointBandResult`. 1 research route: signals. `build_lift_tree`. 2 research routes: root, nonlinear.

C.8.3 Names beginning with C

`CallableDerivativeProvider`. 2 research routes: root, nonlinear. `canonical_dual_window`. 1 research route: signals. `capture_packet_jet_stream`. 2 research routes: root, nonlinear. `CarryLayerConvolution`. 1 research route: generating. `CarryLinearCertificate`. 1 research route: generating. `CarryLinearValuation`. 1 research route: generating. `cartier_degree_bound`. 1 research route: generating. `CartierAutomaton`. 1 research route: generating. `CartierCarryEquivalence`. 1 research route: generating. `CartierLinearRepresentation`. 1 research route: generating. `catalan_number`. 1 research route: nonlinear. `catalan_numbers`. 1 research route: generating. `central_binomial_numbers`. 1 research route: generating. `Certificate`. 1 research route: root. `certificate_document`. 1 research route: export. `certified_overlap_add_convolve`. 1 research route: signals. `certify_cartier_carry_equivalence`. 1 research route: generating. `certify_mahler_functional_equation`. 1 research route: generating. `certify_pointed_subindex_identity`. 1 research route: generating. `certify_rational_mask_algebraic_relation`. 1 research route: generating. `certify_state_coordinate_change`. 1 research route: coordinates. `characteristic_closed_form`. 1 research route: generating. `chebyshev_t_polynomial`. 1 research route: coordinates. `chebyshev_t_polynomial_family`. 1 research route: coordinates. `chebyshev_u_polynomial`. 1 research route: coordinates. `chebyshev_u_polynomial_family`. 1 research route: coordinates. `check_beta_api`. 1 research route: root. `check_expression`. 1 research route: root. `check_public_api`. 1 research route: root. `CheckedExpression`. 1 research route: root. `chinese_remainder_order`. 1 research route: coordinates. `chinese_remainder_transform`. 1 research route: coordinates. `CircuitPacketExpansion`. 1 research route: root. `CirculantOperator`. 1 research route: root. `ClaimStatus`. 1 research route: root. `classical_polynomial_catalog`. 2 research routes: root, coordinates. `classify_carry_linear_valuation`. 1 research route: generating. `CliffordStep`. 1 research route: root. `CliffordTower`. 1 research route: root. `CodeBlock`. 1 research route: export. `coefficient_packets`. 1 research route: signals. `CoefficientDomain`. 1 research route: generating. `CoefficientPartition`. 2 research routes: root, coordinates. `common_refinement`. 1 research route: root. `CommutingDiagramCheck`. 1 research route: coordinates. `companion_matrix`. 2 research routes: root, coordinates. `compare_public_api`. 1 research route: root. `CompatiblePropagationReport`. 1 research route: root. `compile_cartier_linear_representation`. 1 research route: generating. `compile_digit_cauchy_product`. 1 research route: generating. `compile_expression`. 1 research route: root. `compile_finite_mask_carry_plan`. 2 research routes: root, generating. `compile_latex`. 2 research routes: root, export. `compile_packet_jet_plan`. 2 research routes: root, nonlinear. `compile_rational_mask_automaton`. 2 research routes: root, generating. `compile_sequence`. 1 research route: root. `compile_tree_budget`. 1 research route: root. `compile_tree_lagrangian`. 1 research route: root. `CompiledSequence`. 1 research route: root. `CompletionGate`. 1 research route: root. `Compose`. 1 research route: root. `compose`. 1 research route: root. `compose_certificates`. 1 research route: root. `compose_functions`. 1 research route: generating. `compose_ordinary_series`. 1 research route: coordinates. `ComposedDerivativeProvider`. 2 research routes: root, nonlinear. `CompositeSimpson`. 1 research route: root. `ComputationResult`. 1 research route: root. `conditional_projective_limit_completion`. 2 research routes: root, nonlinear. `ConditionalResult`. 1 research route: root. `configure`. 1 research route: root. `conjugate_gradient`. 1 research route: root. `connection_transform`. 2 research routes: root, coordinates. `constant_injection`. 1 research route: root. `constant_polynomial`. 1 research route: coordinates. `constraint_closure`. 1 research route: root. `contragredient_transform`. 1 research route: coordinates. `controllability_basis`. 1 research route: coordinates. `controllability_coordinate_transform`. 1 research route: coordinates. `controllability_matrix`. 1 research route: coordinates. `convert_normalized_coefficients`. 1 research route: coordinates. `convolution_matrix`. 1 research route: signals. `convolution_output_length`. 1 research route: signals. `ConvolutionMode`. 1 research route: signals. `convolve_signal`. 1 research route: signals. `coordinate_change`. 1 research route: root. `coordinate_change_between_recurrences`. 2 research routes: root, coordinates. `coordinate_fundamental_theorem`. 1 research route: root. `coordinate_pairing`. 1 research route: coordinates. `coordinate_transport_identity`. 1 research route: root. `CoordinateAtlas`. 2 research routes: root, coordinates. `CoordinateBudget`. 2 research routes: root, coordinates. `CoordinateSpace`. 1 research route: root. `CoordinateSystem`. 2 research routes: root, coordinates. `CoordinateTransform`. 2 research routes: root, coordinates. `create_beta_api_manifest`. 1 research route: root. `create_public_api_manifest`. 1 research route: root. `cross_spectrum`. 1 research route: signals. `CrossSpectrum`. 1 research route: signals. `crt_many`. 1 research route: root. `crt_pair`. 1 research route: root. `cupy_runtime_status`. 2 research routes: root, nonlinear. `CuPyDeviceDerivativeProvider`. 2 research routes: root, nonlinear. `CuPyPacketJetExecutor`. 2 research routes: root, nonlinear. `CuPyPacketJetSnapshotExecutor`. 2 research routes: root, nonlinear. `CuPyPolynomialDerivativeProvider`. 2 research routes: root, nonlinear. `CuPyRuntimeStatus`. 2 research routes: root, nonlinear. `CuPySnapshotExecutor`. 2 research routes: root, nonlinear. `CuPyValidationResult`. 2 research routes: root, nonlinear. `custom_exponential`. 1 research route: generating. `custom_recurrence_model`. 1 research route: coordinates. `custom_section`. 1 research route: generating. `CustomFunctionSpec`. 2 research routes: root, generating. `cyclic_group`. 1 research route: root. `cyclic_krylov_matrix`. 1 research route: coor-

dinates. `cyclic_project_coefficients`. 1 research route: root. `cyclic_span`. 1 research route: root. `cyclic_symmetry_projectors`. 1 research route: root. `CyclicDenominator`. 1 research route: root. `CyclicSectionFamily`. 1 research route: root. `cyclotomic_coboundary`. 1 research route: generating. `CyclotomicCollapse`. 1 research route: generating.

C.8.4 Names beginning with D

`decimate`. 1 research route: signals. `DEFAULT_BEHAVIORAL_GOLDEN_FIXTURE`. 1 research route: root. `DEFAULT_BETA_API_MANIFEST`. 1 research route: root. `DEFAULT_COORDINATE_BUDGET`. 1 research route: coordinates. `DEFAULT_GENERATING_BUDGET`. 1 research route: generating. `defect_green_weights`. 1 research route: coordinates. `denoising_risk_certificate`. 1 research route: signals. `denominator`. 1 research route: root. `DenseOperator`. 1 research route: root. `deprecated`. 1 research route: root. `DepthObstructionResult`. 1 research route: nonlinear. `derivative`. 1 research route: root. `derivative_coordinate_operator`. 1 research route: coordinates. `derivative_operator_in_family`. 1 research route: coordinates. `DerivativeBound`. 1 research route: root. `DerivativeProvider`. 1 research route: nonlinear. `describe`. 1 research route: root. `design_bandpass`. 1 research route: signals. `design_bandstop`. 1 research route: signals. `design_biquad`. 1 research route: signals. `design_fir`. 1 research route: signals. `design_highpass`. 1 research route: signals. `design_lowpass`. 1 research route: signals. `design_resample_filter`. 1 research route: signals. `detect_piecewise_polynomial_breaks`. 1 research route: signals. `determinant`. 1 research route: coordinates. `DetrendMode`. 1 research route: signals. `difference_noise_bound`. 1 research route: signals. `difference_valuation_profile`. 1 research route: generating. `digit_factorized_normalized_kernel`. 1 research route: generating. `digit_reversal_order`. 1 research route: coordinates. `digit_reversal_transform`. 1 research route: coordinates. `digit_unit`. 1 research route: generating. `DigitAlgebraAudit`. 1 research route: generating. `DigitCauchyProductPlan`. 1 research route: generating. `DigitKernel`. 1 research route: generating. `direct_packet_rotation`. 2 research routes: root, nonlinear. `direct_sum_basis`. 1 research route: coordinates. `direct_sum_transform`. 1 research route: coordinates. `DirectionalDerivative`. 1 research route: nonlinear. `DirectionalJVP`. 2 research routes: root, nonlinear. `DirectRotationResult`. 1 research route: nonlinear. `dirichlet_convolution`. 1 research route: generating. `discover_block_diagonal_low_rank`. 1 research route: root. `discover_packet_support`. 1 research route: root. `discrete_cosine_transform`. 1 research route: coordinates. `discrete_fourier_transform`. 2 research routes: root, coordinates. `divided_square_packets`. 2 research routes: root, nonlinear. `DividedSquareResult`. 1 research route: nonlinear. `divisor_zeta_transform`. 1 research route: coordinates. `document_from_data`. 2 research routes: root, export. `DocumentMetadata`. 2 research routes: root, export. `domain_from_descriptor`. 1 research route: generating. `downsample`. 1 research route: signals. `DTypeChoice`. 1 research route: root. `dumps_portable`. 2 research routes: root, export. `dyadic_sign_diagnostic`. 1 research route: root.

C.8.5 Names beginning with E

`entropy_cost`. 1 research route: signals. `Equation`. 2 research routes: root, export. `EquivalenceResult`. 1 research route: root. `equivalent`. 1 research route: root. `ErrorGuarantee`. 1 research route: root. `estimate_packet_jet_complexity`. 2 research routes: root, nonlinear. `estimate_positive_packet_1d`. 1 research route: signals. `estimate_white_noise_sigma`. 1 research route: signals. `estimated_packet_reconstruct`. 1 research route: signals. `EuclideanTransfer`. 1 research route: root. `euler_fibonacci_coordinates`. 2 research routes: root, coordinates. `euler_polynomial`. 1 research route: coordinates. `euler_polynomial_at_zero`. 1 research route: coordinates. `euler_polynomial_family`. 1 research route: coordinates. `euler_product_collapse`. 1 research route: generating. `euler_product_function`. 1 research route: generating. `euler_to_fibonacci_coefficient`. 1 research route: coordinates. `euler_to_fibonacci_transform`. 2 research routes: root, coordinates. `EulerLocalCoordinates`. 1 research route: root. `EulerProductCollapse`. 1 research route: generating. `evaluate_expression`. 1 research route: root. `evaluation`. 1 research route: root. `evaluation_transform`. 1 research route: coordinates. `exact_binding`. 1 research route: root. `exact_internal_certificate`. 1 research route: root. `exact_map`. 1 research route: root. `exact_value`. 1 research route: root. `ExactAffineMap`. 1 research route: root. `ExactAnalysisSynthesisMap`. 2 research routes: root, coordinates. `ExactAnalysisSynthesisSystem`. 2 research routes: root, coordinates. `ExactCoordinateTransform`. 1 research route: coordinates. `ExactLinearSystem`. 1 research route: coordinates. `ExactMapLiteral`. 1 research route: root. `Exactness`. 1 research route: root. `exactness_strength`. 1 research route: root. `ExactPolynomial`. 1 research route: root. `ExactQuadratureRule`. 1 research route: root. `ExactValueLiteral`. 1 research route: root. `execute_packet_jet_plan`. 2 research routes: root, nonlinear. `execute_packet_jets`. 2 research routes: root, nonlinear. `execute_plan`. 1 research route: root. `explicit_function`. 1 research route: generating. `exponential_basis`. 2 research routes: coordinates, generating. `export_adapters`. 1 research route: export. `export_bundle`. 2 research routes: root, export. `export_data`. 2 research routes: root, export. `export_document`. 2 research routes: root, export. `export_many`. 2 research routes: root, export. `Exportable`. 1 research route: export. `ExportBundleResult`. 2 research routes:

root, export. `ExportFormat`. 2 research routes: root, export. `ExportOptions`. 2 research routes: root, export. `ExportResult`. 2 research routes: root, export. `Expression`. 1 research route: root. `expression_digest`. 1 research route: root. `expression_from_dict`. 1 research route: root. `expression_from_json`. 1 research route: root. `EXPRESSION_NODE_VERSION`. 1 research route: root. `EXPRESSION_SCHEMA`. 1 research route: root. `EXPRESSION_SCHEMA_VERSION`. 1 research route: root. `expression_to_dict`. 1 research route: root. `expression_to_json`. 1 research route: root. `EXPRESSION_TYPE_VERSION`. 1 research route: root. `ExpressionBinding`. 1 research route: root. `ExpressionBudget`. 1 research route: root. `ExpressionPlan`. 1 research route: root. `ExpressionResult`. 1 research route: root. `ExpressionStats`. 1 research route: root. `ExpressionType`. 1 research route: root.

C.8.6 Names beginning with F

`faa_di_bruno`. 2 research routes: root, nonlinear. `FaaDiBrunoResult`. 1 research route: nonlinear. `FaaDiBrunoTerm`. 1 research route: nonlinear. `factorial_power_basis`. 2 research routes: root, generating. `factorial_power_bell_function`. 1 research route: generating. `factorial_power_convolution_layers`. 1 research route: generating. `factorial_power_digit_kernel`. 1 research route: generating. `factorial_power_support_count`. 1 research route: generating. `factorial_power_support_indices`. 1 research route: generating. `factorial_valuation`. 1 research route: generating. `falling_factorial_polynomial`. 1 research route: coordinates. `falling_factorial_polynomial_family`. 1 research route: coordinates. `falling_factorial_to_monomial_transform`. 1 research route: coordinates. `feature`. 1 research route: root. `FEATURE_KEY`. 2 research routes: nonlinear, signals. `features`. 1 research route: root. `FeatureSpec`. 1 research route: root. `FeatureStatus`. 1 research route: root. `FeshbachReduction`. 1 research route: root. `fft_convolve`. 1 research route: signals. `fft_lowpass_1d`. 1 research route: signals. `fft_lowpass_2d`. 1 research route: signals. `fibonacci_companion_power`. 1 research route: coordinates. `fibonacci_factorial_basis`. 2 research routes: coordinates, generating. `fibonacci_numbers`. 2 research routes: coordinates, generating. `fibonacci_polynomial`. 1 research route: coordinates. `fibonacci_polynomial_family`. 1 research route: coordinates. `fibonacci_recurrence`. 1 research route: root. `fibonacci_recurrence_model`. 1 research route: coordinates. `fibonacci_sequence`. 1 research route: root. `Figure`. 1 research route: export. `FilterBankReport`. 1 research route: signals. `FilteredCorrectionProblem`. 2 research routes: root, nonlinear. `FilteredSpectralSequenceResult`. 1 research route: nonlinear. `FilteredVectorSpace`. 2 research routes: root, nonlinear. `FilterOutput`. 1 research route: signals. `finite_difference`. 2 research routes: root, signals. `finite_difference_coordinate_operator`. 1 research route: coordinates. `finite_frobenius_separator`. 1 research route: root. `finite_mask_carry_matrices`. 1 research route: generating. `finite_polyphase_blocks`. 1 research route: signals. `finite_polyphase_filter_kernels`. 1 research route: signals. `FiniteAbelianGroup`. 2 research routes: root, nonlinear. `FiniteBasis`. 2 research routes: root, coordinates. `FiniteCochainComplex`. 1 research route: root. `FiniteDifferenceDerivativeProvider`. 2 research routes: root, nonlinear. `FiniteField`. 1 research route: root. `FiniteFieldCoordinateTransform`. 2 research routes: root, coordinates. `FiniteFilterBank`. 1 research route: signals. `FiniteFilteredBranchResult`. 1 research route: nonlinear. `FiniteFilteredCochainComplex`. 2 research routes: root, nonlinear. `FiniteGroup`. 1 research route: root. `FiniteHeckePackets`. 1 research route: root. `FiniteLiftForest`. 1 research route: root. `FiniteLiftTree`. 2 research routes: root, nonlinear. `FiniteMaskCarryPlan`. 2 research routes: root, generating. `FiniteMonoidSeries`. 1 research route: generating. `FiniteNormalizedSeries`. 2 research routes: root, generating. `FinitePolyphaseKernel`. 1 research route: signals. `fir_filter`. 1 research route: signals. `fir_frequency_response`. 1 research route: signals. `fir_taps_to_series`. 1 research route: generating. `FIRFilter`. 1 research route: signals. `FIRKind`. 1 research route: signals. `FIRMode`. 1 research route: signals. `FIRSeriesAdapter`. 1 research route: generating. `first_frobenius_lift`. 1 research route: generating. `fixed_packet_leakage`. 1 research route: root. `formal_coordinate_from_iterable`. 1 research route: coordinates. `FormalCoordinateChange`. 2 research routes: root, coordinates. `FormalSeries`. 2 research routes: root, generating. `forward_difference_transform`. 2 research routes: root, coordinates. `fractional_power`. 1 research route: root. `frame_signal`. 1 research route: signals. `FrameBatch`. 1 research route: signals. `freeze_custom_function`. 2 research routes: root, generating. `FrequencyResponse`. 1 research route: signals. `frobenius_defect_mask`. 1 research route: generating. `FrobeniusOrbit`. 1 research route: root. `FrobeniusSpine`. 1 research route: root. `from_portable_data`. 1 research route: export. `from_serializable`. 2 research routes: root, export. `FrozenGeneratingFunction`. 2 research routes: root, generating. `fubini_identity`. 1 research route: root. `FunctionRegistry`. 2 research routes: root, generating. `fundamental_theorem`. 1 research route: root. `FundamentalHolonomy`. 1 research route: nonlinear.

C.8.7 Names beginning with G

`gamma_from_impedance`. 1 research route: signals. `GateStatus`. 1 research route: root. `gauss_sum`. 1 research route: root. `gaussian_binomial`. 1 research route: generating. `generalized_bell_digit_reduction`. 2 research routes: root, generating. `generalized_bell_numbers`. 1 research route: generating. `generalized_bell_parity`. 1 research route: generating. `generalized_bell_polynomials`. 1 research route: generating. `GeneratingBudget`. 2 research routes: root, generating. `get_polynomial_family`. 2 research routes: root, coordinates. `get_recurrence_model`. 2 research routes: root, coordinates. `get_sequence`. 1 research route: root. `get_settings`. 1 research route: root. `get_window`. 1 research route: signals. `GradedConvolutionResult`. 1 research route: nonlinear. `GradedConvolutionTerm`. 1 research route: nonlinear. `GradedPacketOperation`. 2 research routes: root, nonlinear. `GradingCertificate`. 1 research route: nonlinear. `guess_algebraic_gf`. 1 research route: generating. `guess_hypergeometric_ratio`. 1 research route: generating. `guess_linear_recurrence`. 1 research route: generating. `guess_p_recursive`. 1 research route: generating. `guess_quasipolynomial`. 1 research route: generating.

C.8.8 Names beginning with H

`haar_analyze`. 1 research route: signals. `haar_decompose`. 1 research route: signals. `haar_filter_bank`. 1 research route: signals. `haar_lipschitz_bound`. 1 research route: signals. `haar_matrix`. 1 research route: signals. `haar_packet_analyze`. 1 research route: signals. `haar_packet_synthesize`. 1 research route: signals. `haar_step_coefficient`. 1 research route: signals. `haar_synthesize`. 1 research route: signals. `haar_wavelet_transform`. 1 research route: coordinates. `HaarDecomposition`. 1 research route: signals. `HaarFilterBank`. 1 research route: signals. `HaarPacketTree`. 2 research routes: root, signals. `half_shift_distance`. 1 research route: signals. `Heading`. 1 research route: export. `HeisenbergModel`. 1 research route: root. `hermite_physicists_polynomial`. 1 research route: coordinates. `hermite_physicists_polynomial_family`. 1 research route: coordinates. `hermite_probabilists_polynomial`. 1 research route: coordinates. `hermite_probabilists_polynomial_family`. 1 research route: coordinates. `HiddenSolveResult`. 1 research route: nonlinear. `higher_maurer_cartan_packets`. 2 research routes: root, nonlinear. `HigherMaurerCartanResult`. 1 research route: nonlinear. `HilbertObstructionProjector`. 2 research routes: root, nonlinear. `HilbertObstructionResult`. 1 research route: nonlinear. `HolonomicRecurrence`. 1 research route: root. `HtmlRenderOptions`. 1 research route: export. `HurwitzSeries`. 1 research route: root. `hypergeometric_function`. 1 research route: generating. `HypergeometricGuess`. 1 research route: generating.

C.8.9 Names beginning with I

`identity`. 1 research route: root. `identity_formal_coordinate`. 1 research route: coordinates. `identity_matrix`. 1 research route: coordinates. `IdentityMap`. 1 research route: root. `impedance_from_gamma`. 1 research route: signals. `independent_direct_sum_seed_rank`. 1 research route: root. `InducedRepresentation`. 1 research route: root. `infer_export_format`. 1 research route: export. `INTEGER_DOMAIN`. 1 research route: generating. `integer_valuation`. 1 research route: generating. `IntegerDomain`. 1 research route: generating. `integral_coordinate_operator`. 1 research route: coordinates. `IntegralIdentity`. 1 research route: root. `IntegralResult`. 1 research route: root. `integrate`. 1 research route: root. `integrate_derivative_provider`. 1 research route: root. `integrate_formal_series`. 1 research route: root. `integrate_polynomial_coordinates`. 1 research route: root. `integrate_polynomial_oriented`. 1 research route: root. `integration_by_parts`. 1 research route: root. `IntegrationError`. 1 research route: root. `interpolate`. 1 research route: signals. `interpolation_transform`. 1 research route: coordinates. `intertwiner_space`. 2 research routes: root, coordinates. `IntertwinerSpace`. 2 research routes: root, coordinates. `IntertwiningResult`. 1 research route: nonlinear. `interval_integration`. 1 research route: root. `IntervalMeasure`. 1 research route: root. `inverse_matrix`. 1 research route: coordinates. `istft`. 1 research route: signals.

C.8.10 Names beginning with J

`jacobsthal_recurrence_model`. 1 research route: coordinates. `JaxDerivativeProvider`. 2 research routes: root, nonlinear. `JumpDetection`. 1 research route: signals. `JumpRecoveryResult`. 1 research route: signals.

C.8.11 Names beginning with K

`KatoTransport`. 2 research routes: root, nonlinear. `KomorebiDeprecationWarning`. 1 research route: root. `kummer_layer_distribution`. 1 research route: generating. `KummerLayerDistribution`. 1 research route: generating.

C.8.12 Names beginning with L

`labeled_composition_term_count`. 1 research route: nonlinear. `lagrange_interpolation_basis`. 1 research route: coordinates. `laguerre_polynomial`. 1 research route: coordinates. `laguerre_polynomial_family`. 1 research route: coordinates. `lah_number`. 1 research route: coordinates. `latex_escape`. 1 research route: export. `latex_label`. 1 research route: export. `LatexCompilation`. 1 research route: export. `LatexCompileOptions`. 2 research routes: root, export. `LatexEngine`. 2 research routes: root, export. `LatexRenderOptions`. 2 research routes: root, export. `lattice_polyphase_analyze`. 1 research route: signals. `lattice_polyphase_synthesize`. 1 research route: signals. `LatticePacketData`. 1 research route: root. `LatticePolyphaseCoordinates`. 1 research route: signals. `legendre_polynomial`. 1 research route: coordinates. `legendre_polynomial_family`. 1 research route: coordinates. `Let`. 1 research route: root. `let`. 1 research route: root. `lfilter`. 1 research route: signals. `lift_digit_sequence`. 1 research route: generating. `LiftStepResult`. 1 research route: nonlinear. `LiftTreeNode`. 1 research route: nonlinear. `LindbladGenerator`. 1 research route: root. `linear_recurrence_function`. 1 research route: generating. `LinearCharacter`. 1 research route: root. `LinearFilter`. 1 research route: signals. `LinearLiftProblem`. 2 research routes: root, nonlinear. `LinearMapType`. 1 research route: root. `LinearObstructionProblem`. 2 research routes: root, nonlinear. `LinearRealization`. 2 research routes: root, coordinates. `LinearRecurrence`. 1 research route: root. `LinearRecurrenceGuess`. 1 research route: generating. `LinearRefinementDefect`. 1 research route: root. `ListBlock`. 1 research route: export. `lms_filter`. 1 research route: signals. `load_behavioral_golden_fixture`. 1 research route: root. `load_beta_api_manifest`. 1 research route: root. `load_coordinate_object`. 2 research routes: root, coordinates. `load_coordinate_record`. 1 research route: coordinates. `load_packet_jet_snapshot_stream`. 2 research routes: root, nonlinear. `load_portable`. 2 research routes: root, export. `load_public_api_manifest`. 1 research route: root. `load_transform`. 1 research route: coordinates. `loads_portable`. 2 research routes: root, export. `local_normalized_kernel`. 1 research route: generating. `LowRankOperator`. 1 research route: root. `lp_cost`. 1 research route: signals. `lucas_polynomial`. 1 research route: coordinates. `lucas_polynomial_family`. 1 research route: coordinates. `lucas_recurrence_model`. 1 research route: coordinates. `LyapunovSchmidtDecomposition`. 1 research route: nonlinear. `LyapunovSchmidtPoint`. 1 research route: nonlinear. `LyapunovSchmidtReduction`. 2 research routes: root, nonlinear.

C.8.13 Names beginning with M

`mackey_tensor_packets`. 1 research route: root. `mahler_coefficients`. 1 research route: generating. `mahler_mask_inverse`. 2 research routes: root, generating. `mahler_mask_product`. 2 research routes: root, generating. `MahlerSeries`. 1 research route: root. `majorant_convergence`. 2 research routes: root, nonlinear. `MajorantConvergence`. 1 research route: nonlinear. `MarkovLumping`. 1 research route: root. `MathDocument`. 2 research routes: root, export. `Matrix`. 1 research route: coordinates. `matrix_add`. 1 research route: coordinates. `matrix_chain_multiply`. 1 research route: root. `matrix_chain_parenthesization`. 1 research route: root. `matrix_chain_plan`. 1 research route: root. `matrix_from_columns`. 1 research route: coordinates. `matrix_multiply`. 1 research route: coordinates. `matrix_power`. 1 research route: coordinates. `matrix_rank`. 1 research route: coordinates. `matrix_shape`. 1 research route: coordinates. `matrix_subtract`. 1 research route: coordinates. `matrix_units`. 1 research route: root. `matrix_vector_multiply`. 1 research route: coordinates. `MatrixChainPlan`. 1 research route: root. `MatrixChainTree`. 1 research route: root. `MatrixSemigroup`. 1 research route: root. `maurer_cartan_packets`. 2 research routes: root, nonlinear. `MaurerCartanResult`. 1 research route: nonlinear. `Measure`. 1 research route: root. `median_absolute_deviation`. 1 research route: signals. `MemoryKernel`. 1 research route: root. `minimal_prefix_period`. 1 research route: generating. `minimize_cartier_automaton`. 1 research route: generating. `mixed_radix_address`. 1 research route: signals. `mobius_integer`. 1 research route: coordinates. `modal_coordinate_system`. 1 research route: coordinates. `ModalCoordinateSystem`. 1 research route: coordinates. `ModularLinearSolution`. 1 research route: nonlinear. `ModularMatrix`. 1 research route: coordinates. `ModularVector`. 1 research route: coordinates. `monomial_polynomial_family`. 1 research route: coordinates. `monomial_to_falling_factorial_transform`. 1 research route: coordinates. `monomial_to_fibonacci_coefficient`. 1 research route: coordinates. `monomial_to_rising_factorial_transform`. 1 research route: coordinates. `MonomialAction`. 1 research route: root. `motzkin_sequence`. 1 research route: root. `MovingPacketSystem`. 2 research routes: root, nonlinear. `MultilinearOperation`. 2 research routes: root, nonlinear. `multiplication_coordinate_operator`. 1 research route: coordinates. `multiplicative_order_mod_prime`. 1 research route: generating. `multiplicative_series`. 1 research route: generating.

C.8.14 Names beginning with N

`newton_evaluation_transform`. 1 research route: coordinates. `newton_interpolation_basis`. 1 research route: coordinates. `nlms_filter`. 1 research route: signals. `nonlinear_cupy_available`. 2 research routes: root, nonlinear. `nonlinear_shift_filter`. 1 research route: signals. `NonlinearPacketMap`. 2 research routes: root, nonlinear. `NonlinearRefinementDefect`. 2 research routes: root, nonlinear. `NonlinearSchurElimination`. 2 research routes: root, nonlinear. `normal_form_digest`. 1 research route: root. `NormalForm`. 1 research route: root. `normalization_change_transform`. 2 research routes: root, coordinates. `normalize_expression`. 1 research route: root. `normalized_cyclotomic_expression`. 1 research route: generating. `NormalizedBasis`. 3 research routes: root, coordinates, generating. `NormalizedKernelResidue`. 1 research route: generating. `NormalizedMaskPlan`. 1 research route: generating. `NormalizedMonomialBasis`. 1 research route: generating. `nullspace`. 1 research route: coordinates. `NumericCoordinateTransform`. 2 research routes: root, coordinates.

C.8.15 Names beginning with O

`observability_block_matrix`. 1 research route: coordinates. `observability_coordinate_transform`. 1 research route: coordinates. `observability_matrix`. 2 research routes: root, coordinates. `ObstructionResult`. 1 research route: nonlinear. `ObstructionSquare`. 1 research route: nonlinear. `one_borrow_count`. 1 research route: generating. `open_packet_jet_snapshot_stream`. 2 research routes: root, nonlinear. `operator_change_of_coordinates`. 2 research routes: root, coordinates. `operator_commutator`. 1 research route: coordinates. `operator_matrix_between_families`. 1 research route: coordinates. `OperatorAlgebra`. 1 research route: root. `OperatorGrammar`. 1 research route: root. `OperatorMorphism`. 1 research route: root. `OperatorRealization`. 1 research route: root. `OperatorRelation`. 1 research route: root. `OperatorWord`. 1 research route: root. `ordered_rooted_trees`. 1 research route: nonlinear. `ordered_tree_count`. 1 research route: nonlinear. `ordered_trees_with_leaves`. 1 research route: nonlinear. `OrderedRootedTree`. 2 research routes: root, nonlinear. `ordinary_basis`. 3 research routes: root, coordinates, generating. `otsu_threshold`. 1 research route: signals. `overlap_add`. 1 research route: signals. `overlap_add_certificate`. 1 research route: signals. `overlap_add_convolve`. 1 research route: signals. `overlap_matrix_error`. 1 research route: signals. `overlap_partition_lp_error`. 1 research route: signals. `overlap_profile`. 1 research route: signals. `OverlapAddConvolver`. 1 research route: signals.

C.8.16 Names beginning with P

`p_adic_valuation_fraction`. 1 research route: generating. `p_free_orbit`. 1 research route: generating. `packet_energy`. 1 research route: signals. `packet_faa_di_bruno`. 2 research routes: root, nonlinear. `packet_jet_benchmark_contract`. 2 research routes: root, nonlinear. `PACKET_JET_CHUNK_SCHEMA`. 2 research routes: root, nonlinear. `PACKET_JET_CHUNK_SCHEMA_VERSION`. 2 research routes: root, nonlinear. `PACKET_JET_STREAM_SCHEMA`. 2 research routes: root, nonlinear. `PACKET_JET_STREAM_SCHEMA_VERSION`. 2 research routes: root, nonlinear. `packet_jet_support`. 2 research routes: root, nonlinear. `packet_product`. 1 research route: signals. `packet_taylor`. 2 research routes: root, nonlinear. `packet_tree_reconstruction`. 2 research routes: root, nonlinear. `PacketCoordinates`. 1 research route: root. `PacketDerivativeResult`. 1 research route: nonlinear. `PacketEstimate1D`. 1 research route: signals. `PacketIdentity`. 1 research route: generating. `PacketIR`. 1 research route: root. `PacketJet`. 2 research routes: root, nonlinear. `PacketJetAddress`. 1 research route: nonlinear. `PacketJetBatchResult`. 2 research routes: root, nonlinear. `PacketJetCoefficient`. 1 research route: nonlinear. `PacketJetComplexityEstimate`. 2 research routes: root, nonlinear. `PacketJetEvaluation`. 1 research route: nonlinear. `PacketJetExecutionPlan`. 2 research routes: root, nonlinear. `PacketJetFamily`. 2 research routes: root, nonlinear. `PacketJetInputGroup`. 1 research route: nonlinear. `PacketJetProfileReport`. 2 research routes: root, nonlinear. `PacketJetSnapshot`. 2 research routes: root, nonlinear. `PacketJetSnapshotStreamReader`. 2 research routes: root, nonlinear. `PacketJetStreamChunkRecord`. 2 research routes: root, nonlinear. `PacketJetStreamManifest`. 2 research routes: root, nonlinear. `PacketJetStreamSelection`. 2 research routes: root, nonlinear. `PacketJetStreamWriteResult`. 2 research routes: root, nonlinear. `PacketJetSupport`. 1 research route: nonlinear. `PacketLabel`. 1 research route: nonlinear. `PacketLayout`. 1 research route: root. `PacketMorphism`. 1 research route: root. `PacketOperator`. 1 research route: root. `PacketPath`. 1 research route: signals. `PacketRefinement`. 1 research route: root. `PacketSystem`. 1 research route: root. `PacketTaylorResult`. 1 research route: nonlinear. `PacketTreeResult`. 1 research route: nonlinear. `padic_valuation`. 1 research route: root. `padovan_recurrence_model`. 1 research route: coordinates. `pairing_invariant`. 1 research route: coordinates. `Paragraph`. 1 research route: export. `parseval_certificate`. 1 research route: signals. `Partition`. 1 research route: nonlinear. `partition_numbers`. 1 research route: generating. `partition_sequence`. 1 research route: root. `PatchingSupportReport`. 1 research route: root. `path_contributions`. 1 research route: root. `pell_polynomial`. 1 research route: coordinates. `pell_polynomial_family`. 1 research route: coordinates. `pell_recurrence_model`. 1 research route:

coordinates. `periodic_five_three_lifting`. 1 research route: signals. `periodic_laplacian`. 1 research route: root. `PeriodicFilterBank`. 2 research routes: root, signals. `PeriodicLiftingScheme`. 1 research route: signals. `periodogram`. 1 research route: signals. `permutation_transform`. 1 research route: coordinates. `phase_mobius`. 1 research route: signals. `PhaseTypeSurvival`. 1 research route: root. `PlanInstruction`. 1 research route: root. `pochhammer_basis`. 1 research route: generating. `pointed_subindex_polynomial`. 1 research route: generating. `PointedSubindexCertificate`. 1 research route: generating. `Polynomial`. 1 research route: coordinates. `polynomial_add`. 1 research route: coordinates. `polynomial_cartier`. 1 research route: generating. `polynomial_compose_affine`. 1 research route: coordinates. `polynomial_derivative`. 1 research route: coordinates. `polynomial_derivative_bound`. 1 research route: root. `polynomial_evaluate`. 1 research route: coordinates. `polynomial_extended_gcd`. 1 research route: coordinates. `polynomial_from_family_coordinates`. 1 research route: root. `polynomial_from_formal_series`. 1 research route: root. `polynomial_integral`. 1 research route: coordinates. `polynomial_integration_covector`. 1 research route: root. `polynomial_mask_coboundary`. 1 research route: generating. `polynomial_multiply`. 1 research route: coordinates. `polynomial_multiply_mod`. 1 research route: generating. `polynomial_operator`. 1 research route: coordinates. `polynomial_power_mod`. 1 research route: generating. `polynomial_scale`. 1 research route: coordinates. `polynomial_shift`. 1 research route: coordinates. `polynomial_subtract`. 1 research route: coordinates. `PolynomialCoboundaryResult`. 1 research route: generating. `PolynomialCoordinateOperator`. 2 research routes: root, coordinates. `PolynomialCoordinateSpace`. 1 research route: root. `PolynomialDerivativeProvider`. 2 research routes: root, nonlinear. `PolynomialFamily`. 2 research routes: root, coordinates. `PolynomialFamilyCatalog`. 1 research route: coordinates. `PolynomialFixedPointProblem`. 2 research routes: root, nonlinear. `PolynomialIntegrationCovector`. 1 research route: root. `PolynomialLiftProblem`. 2 research routes: root, nonlinear. `PolynomialPrimitive`. 1 research route: root. `polyphase_analyze`. 2 research routes: root, signals. `polyphase_convolution`. 1 research route: signals. `polyphase_filter_kernels`. 1 research route: signals. `polyphase_roundtrip_certificate`. 1 research route: signals. `polyphase_synthesize`. 1 research route: signals. `PolyphaseBlockMatrix`. 1 research route: signals. `PolyphaseConvolution`. 1 research route: signals. `PolyphaseCoordinates`. 1 research route: signals. `PolyphaseKernel`. 1 research route: signals. `PolyphaseSystem`. 2 research routes: root, signals. `portable_codecs`. 2 research routes: root, export. `portable_digest`. 2 research routes: root, export. `portable_migrations`. 2 research routes: root, export. `PORTABLE_SCHEMA`. 1 research route: export. `PORTABLE_SCHEMA_VERSION`. 1 research route: export. `PortableCodec`. 2 research routes: root, export. `PortableMigration`. 2 research routes: root, export. `positive_zero_string`. 1 research route: root. `PowerSpectrum`. 1 research route: signals. `PRecursiveGuess`. 1 research route: generating. `PresentedModule`. 1 research route: root. `prime_valuation_packets`. 1 research route: generating. `PrimeFieldDomain`. 1 research route: generating. `PrimePowerBinomial`. 1 research route: generating. `PrimePowerKernelCompiler`. 2 research routes: root, generating. `PrimePowerUnitCompiler`. 1 research route: generating. `primitive_certificate`. 1 research route: root. `primitive_matrix_equal`. 1 research route: root. `PrimitiveKind`. 1 research route: root. `product_measure`. 1 research route: root. `ProductMeasure`. 1 research route: root. `profile_operation`. 2 research routes: root, nonlinear. `profile_packet_jet_runtime`. 2 research routes: root, nonlinear. `ProjectiveIntertwinerEnvelope`. 1 research route: root. `ProjectiveLimitRecord`. 1 research route: nonlinear. `ProjectiveMultiplicityPacket`. 1 research route: root. `ProvenanceRecord`. 1 research route: root. `PrunedPacketCoefficients`. 1 research route: signals. `PublicApiManifest`. 1 research route: root. `PublicApiSymbol`. 1 research route: root. `pushforward_identity`. 1 research route: root. `pushforward_measure`. 1 research route: root. `PushforwardMeasure`. 1 research route: root.

C.8.17 Names beginning with Q

`q_binomial_support`. 1 research route: generating. `q_binomial_valuation_decomposition`. 2 research routes: root, generating. `q_binomial_valuation_direct`. 1 research route: generating. `q_factorial`. 1 research route: generating. `q_factorial_basis`. 1 research route: generating. `q_integer`. 1 research route: generating. `QKummerDecomposition`. 1 research route: generating. `quadratic_catalan_majorant`. 2 research routes: root, nonlinear. `QuadraticRefinement`. 1 research route: root. `QuadratureExactness`. 1 research route: root. `QuadratureRule`. 1 research route: root. `QuantumChannel`. 1 research route: root. `QuantumCode`. 1 research route: root. `QuantumFeshbachReduction`. 1 research route: root. `QuantumSeedSystem`. 1 research route: root. `QuasiPolynomialGuess`. 1 research route: generating.

C.8.18 Names beginning with R

`RamificationCliffordTower`. 1 research route: root. `RamificationStage`. 1 research route: root. `RATIONAL_DOMAIN`. 1 research route: generating. `rational_function`. 1 research route: generating. `rational_gf_from_recurrence`. 1 research route: generating. `rational_mask_coefficients_mod_prime`. 1 research route: generating. `RATIONAL_SPACE`. 1 research route: root. `RationalDomain`. 1 research route: generating. `RationalMaskAlgebraicCertificate`. 1 research route: generating. `RationalResamplingPlan`. 1 research route: signals. `RationalSpace`. 1 research route: root. `RawLatex`. 1 research route: export. `realization_defect`. 1 research route: coordinates. `reciprocal_coefficients`. 1 research route: root. `recompress_factors`. 1 research route: root. `reconstruction_error`. 1 research route: signals. `recover_initial_state`. 2 research routes: root, coordinates. `recover_jumps_subgaussian`. 1 research route: signals. `recover_separated_jumps`. 2 research routes: root, signals. `recover_vector_jumps`. 1 research route: signals. `recover_vector_jumps_subgaussian`. 1 research route: signals. `recurrence_analysis_transform`. 1 research route: coordinates. `recurrence_model_catalog`. 2 research routes: root, coordinates. `recurrence_polynomial`. 1 research route: coordinates. `recurrence_synthesis_transform`. 1 research route: coordinates. `RecurrenceCoordinateRecord`. 2 research routes: root, coordinates. `RecurrenceLike`. 1 research route: coordinates. `RecurrenceModel`. 2 research routes: root, coordinates. `reduce_exact_transform_mod_prime`. 1 research route: coordinates. `refinement_children`. 1 research route: signals. `RefinementTower`. 1 research route: root. `register_coordinate_codecs`. 1 research route: coordinates. `register_export_adapter`. 1 research route: export. `register_generating_codecs`. 1 research route: generating. `register_portable_codec`. 2 research routes: root, export. `register_portable_migration`. 2 research routes: root, export. `render_html`. 1 research route: export. `render_latex`. 2 research routes: root, export. `render_markdown`. 1 research route: export. `render_text`. 1 research route: export. `replay`. 1 research route: root. `replay_matrix`. 1 research route: root. `require`. 1 research route: nonlinear. `resample_poly`. 1 research route: signals. `resample_poly_result`. 1 research route: signals. `resampled_length`. 1 research route: signals. `ResamplingResult`. 1 research route: signals. `residue_analyze`. 1 research route: root. `residue_class_order`. 1 research route: coordinates. `residue_class_transform`. 2 research routes: root, coordinates. `residue_packet_system`. 1 research route: root. `residue_project`. 1 research route: signals. `ResidueRingDomain`. 1 research route: generating. `resolve_latex_engine`. 1 research route: export. `retained_packet_reconstruct`. 2 research routes: root, signals. `retained_packet_reconstruction`. 1 research route: signals. `RetainedPacketReconstruction`. 1 research route: signals. `RetainedPacketResult`. 1 research route: signals. `RewriteStep`. 1 research route: root. `RightInverseCorrectionProblem`. 2 research routes: root, nonlinear. `RightInverseCorrectionResult`. 1 research route: nonlinear. `rising_factorial_polynomial`. 1 research route: coordinates. `rising_factorial_polynomial_family`. 1 research route: coordinates. `rising_factorial_to_monomial_transform`. 1 research route: coordinates. `rising_to_falling_factorial_transform`. 1 research route: coordinates. `rls_filter`. 1 research route: signals. `root_filter_value`. 1 research route: generating. `rref`. 1 research route: coordinates. `run_behavioral_golden_fixture`. 1 research route: root. `RuntimeProfileSample`. 2 research routes: root, nonlinear. `RuntimeStageProfile`. 2 research routes: root, nonlinear.

C.8.19 Names beginning with S

`save_coordinate_object`. 2 research routes: root, coordinates. `save_coordinate_record`. 1 research route: coordinates. `save_packet_jet_snapshot_stream`. 2 research routes: root, nonlinear. `save_portable`. 2 research routes: root, export. `save_public_api_manifest`. 1 research route: root. `save_transform`. 1 research route: coordinates. `Scalar`. 1 research route: coordinates. `scalar_region_error`. 1 research route: signals. `scalar_region_lp_error`. 1 research route: signals. `Scale`. 1 research route: root. `scale`. 1 research route: root. `scaling_transform`. 1 research route: coordinates. `scan_residue_lanes`. 1 research route: generating. `schedule_unlimited`. 1 research route: root. `SchurDerivativeResult`. 1 research route: nonlinear. `SchurSecondDerivativeResult`. 1 research route: nonlinear. `search_decimation_relation`. 1 research route: generating. `secant_euler_from_fibonacci_coordinates`. 1 research route: coordinates. `SectionCycleAudit`. 1 research route: generating. `SeedMorphism`. 1 research route: root. `SeedSystem`. 1 research route: root. `semisimple_seed_rank`. 1 research route: root. `Sequence`. 1 research route: root. `sequence_coordinates_between_models`. 1 research route: coordinates. `sequence_document`. 2 research routes: root, export. `series_to_fir_adapter`. 1 research route: generating. `series_to_fir_taps`. 1 research route: generating. `SeriesTransferCertificate`. 1 research route: generating. `set_partitions`. 1 research route: nonlinear. `Settings`. 1 research route: root. `settings_context`. 1 research route: root. `shannon_entropy_cost`. 1 research route: signals. `shared_direct_sum_seed_bounds`. 1 research route: root. `ShiftPolynomial`. 2 research routes: root, coordinates. `signed_euler_number`. 1 research route: root. `signed_euler_numbers`. 1 research route: generating. `signed_secant_euler_numbers`. 1 research route: coordinates. `signed_section`. 1 research route: root. `similarity_from_cyclic_vectors`. 1 research route: coordinates. `SimplificationResult`. 1 research route: root. `simplify_expression`. 1 research route: root. `single_operator_seed_rank`. 1 research route: root.

`SingularPacket1D`. 2 research routes: root, signals. `siso_state_space`. 1 research route: coordinates. `smith_sensitivity`. 1 research route: signals. `SmithCongruenceProblem`. 1 research route: root. `smooth_haar_coefficient_bound`. 1 research route: signals. `snapshot_packet_jet`. 2 research routes: root, nonlinear. `solve_linear_system`. 1 research route: coordinates. `solve_mod_prime`. 2 research routes: root, nonlinear. `solve_polynomial_coboundary`. 1 research route: generating. `sorted_coordinate_transform`. 1 research route: coordinates. `SparsePolynomialDerivativeProvider`. 2 research routes: root, nonlinear. `SparseSymmetricTensor`. 2 research routes: root, nonlinear. `SparseTensorEntry`. 2 research routes: root, nonlinear. `spectral_dimension`. 1 research route: root. `SpectralAverage`. 1 research route: signals. `SpectralScaling`. 1 research route: signals. `SpectralSequenceEntry`. 1 research route: nonlinear. `SpectralSequencePage`. 1 research route: nonlinear. `spectrogram`. 1 research route: signals. `SpectrogramResult`. 1 research route: signals. `SpectrogramScaling`. 1 research route: signals. `split_mixed_radix_address`. 1 research route: signals. `STAGES`. 1 research route: root. `StageSpec`. 1 research route: root. `standard_basis`. 2 research routes: root, coordinates. `StandardWindow`. 1 research route: root. `StateRecovery`. 2 research routes: root, coordinates. `StateSpaceCoordinateCertificate`. 1 research route: coordinates. `StateSpaceModel`. 2 research routes: root, coordinates. `status`. 2 research routes: nonlinear, signals. `step_haar_coefficient`. 1 research route: signals. `step_location_lp_error`. 1 research route: signals. `stft`. 1 research route: signals. `STFTResult`. 1 research route: signals. `stirling_first_signed`. 1 research route: coordinates. `stirling_first_unsigned`. 1 research route: coordinates. `stirling_second`. 2 research routes: coordinates, nonlinear. `stream_fir_filter`. 1 research route: signals. `stream_lfilter`. 1 research route: signals. `stream_stft`. 1 research route: signals. `StreamedPacketJetEvaluation`. 2 research routes: root, nonlinear. `StreamingFIR`. 1 research route: signals. `StreamingLinearFilter`. 1 research route: signals. `StreamingSTFT`. 1 research route: signals. `StreamingSTFTBatch`. 1 research route: signals. `subgaussian_noise_bound`. 1 research route: signals. `subset_zeta_transform`. 1 research route: coordinates. `Sum`. 1 research route: root. `SupportComparison`. 1 research route: root. `SupportPreorder`. 1 research route: root. `sure_threshold`. 1 research route: signals. `Symbol`. 1 research route: root. `symbol`. 1 research route: root. `SympyDerivativeProvider`. 2 research routes: root, nonlinear. `synthesize_recurrence_coordinates`. 1 research route: coordinates.

C.8.20 Names beginning with T

`Table`. 2 research routes: root, export. `table_rows`. 1 research route: export. `TameWeilPacket`. 1 research route: root. `Task`. 1 research route: root. `tensor_axis_permutation_order`. 1 research route: coordinates. `tensor_axis_permutation_transform`. 1 research route: coordinates. `tensor_coordinate_transform`. 2 research routes: root, coordinates. `tensor_product_basis`. 1 research route: coordinates. `tensor_product_transform`. 1 research route: coordinates. `tensor_seed_rank_upper_bound`. 1 research route: root. `then`. 1 research route: root. `theta_polynomial`. 1 research route: root. `threshold_coefficients`. 1 research route: signals. `ThresholdMode`. 1 research route: signals. `ThresholdRule`. 1 research route: signals. `thue_morse_sequence`. 1 research route: root. `to_portable_data`. 1 research route: export. `to_serializable`. 2 research routes: root, export. `TorchDerivativeProvider`. 2 research routes: root, nonlinear. `transfer_section`. 1 research route: root. `transfer_tangent`. 1 research route: root. `TransferGraph`. 1 research route: root. `transform_sequence`. 1 research route: coordinates. `translation_transform`. 1 research route: coordinates. `transport_bilinear_form`. 1 research route: coordinates. `transport_covector`. 1 research route: coordinates. `transport_operator`. 1 research route: coordinates. `transport_polynomial_operator`. 1 research route: coordinates. `transport_rectangular_operator`. 1 research route: coordinates. `transport_tensor_components`. 1 research route: coordinates. `TransportEdge`. 2 research routes: root, nonlinear. `TransportGraph`. 2 research routes: root, nonlinear. `TransportRefinementResult`. 1 research route: nonlinear. `transpose`. 1 research route: coordinates. `TreeExpansionResult`. 1 research route: nonlinear. `TreeNode`. 1 research route: root. `TreeTerm`. 1 research route: nonlinear. `tribonacci_recurrence_model`. 1 research route: coordinates. `TruncatedPadic`. 1 research route: root. `TruncatedPadicDomain`. 1 research route: generating. `TruncatedPowerPacket`. 1 research route: signals. `TruncatedPowerTerm`. 1 research route: signals. `type_from_dict`. 1 research route: root. `type_to_dict`. 1 research route: root.

C.8.21 Names beginning with U

`UnifiedStage`. 1 research route: root. `UnifiedSystem`. 1 research route: root. `universal_threshold`. 1 research route: signals. `upsample`. 1 research route: signals.

C.8.22 Names beginning with V

validate_cupy_snapshot. 2 research routes: root, nonlinear. **Variance**. 1 research route: coordinates. **variation_of_constants_prefix**. 2 research routes: root, coordinates. **Vector**. 1 research route: coordinates. **VectorJumpRecoveryResult**. 1 research route: signals. **verify_cyclotomic_coboundary**. 1 research route: generating. **verify_cyclotomic_exponent_relation**. 1 research route: generating. **verify_finite_filtered_branch**. 2 research routes: root, nonlinear. **VisibilityTree**. 1 research route: root.

C.8.23 Names beginning with W

walsh_hadamard_transform. 1 research route: coordinates. **WalshHadamardOperator**. 1 research route: root. **warn_deprecated**. 1 research route: root. **wavelet_denoise**. 1 research route: signals. **WaveletDenoiseResult**. 1 research route: signals. **weakest_exactness**. 1 research route: root. **weighted_bell_digit_reduction**. 1 research route: generating. **weighted_factorial_power_bell_function**. 1 research route: generating. **weighted_generalized_bell_numbers**. 1 research route: generating. **weighted_generalized_bell_polynomials**. 1 research route: generating. **WeightedMonomialBasis**. 2 research routes: root, generating. **WeightedTransferEdge**. 1 research route: root. **WeightedTransferGraph**. 1 research route: root. **welch_psd**. 1 research route: signals. **window_metrics**. 1 research route: signals. **WindowMetrics**. 1 research route: signals. **WindowName**. 1 research route: signals. **WindowSpec**. 1 research route: signals.

C.8.24 Names beginning with Z

zero_borrow_count. 1 research route: generating. **zero_map**. 1 research route: root. **zero_matrix**. 1 research route: coordinates. **ZeroMap**. 1 research route: root.

C.8.25 Names beginning with Symbols and underscores

__version__. 1 research route: root.

C.9 Exact research signature index

The 1,312 research cards contain 923 exact manifest-signature values. The index treats literal `null` as one recorded value and preserves every quote, default, annotation, positional marker, and return annotation in the non-null signatures. Signature equality is textual manifest equality. It is not a claim that two implementations share behavior.

Signature class	Groups	Research cards
Literal `null`	1	36
Shared non-null signatures	244	598
Non-null signatures used by one card	678	678
Total	**923**	**1,312**

C.9.1 Literal null signature

SIG-NULL. 36 research cards. Kinds: `class` and `data`.

```
null
```

Cards: C.1A **version**, C.1A API_STABILITY_LEVEL, C.1A BEHAVIORAL_GOLDEN_SCHEMA, C.1A BEHAVIORAL_GOLDEN_SCHEMA_VERSION, C.1A BETA_API_STABILITY_LEVEL, C.1A BETA_PUBLIC_NAMESPACES, C.1A CoordinateSpace, C.1G DEFAULT_BEHAVIORAL_GOLDEN_FIXTURE, C.1G DEFAULT_BETA_API_MANIFEST, C.2C DEFAULT_COORDINATE_BUDGET, C.3A DEFAULT_GENERATING_BUDGET, C.1A EXPRESSION_NODE_VERSION, C.1A EXPRESSION_SCHEMA, C.1A EXPRESSION_SCHEMA_VERSION, C.1A EXPRESSION_TYPE_VERSION, C.1A ExpressionType, C.5B FEATURE_KEY, C.4A FEATURE_KEY, C.3A INTEGER_DOMAIN, C.1A KomorebiDeprecationWarning, C.1A PACKET_JET_CHUNK_SCHEMA, C.5B PACKET_JET_CHUNK_SCHEMA, C.1A PACKET_JET_CHUNK_SCHEMA_VERSION, C.5B PACKET_JET_CHUNK_SCHEMA_VERSION, C.1A PACKET_JET_STREAM_SCHEMA, C.5B PACKET_JET_STREAM_SCHEMA, C.1A

PACKET_JET_STREAM_SCHEMA_VERSION, C.5B PACKET_JET_STREAM_SCHEMA_VERSION, C.5B PacketLabel, C.4A PacketPath, C.6 PORTABLE_SCHEMA, C.6 PORTABLE_SCHEMA_VERSION, C.3A RATIONAL_DOMAIN, C.1B RATIONAL_SPACE, C.2C Scalar, C.1A STAGES.

C.9.2 Shared non-null signatures

SIG-0002. 2 research cards. Kinds: `function`.

```
() -> 'bool'
```

Cards: C.1E nonlinear_cupy_available, C.5B nonlinear_cupy_available.

SIG-0003. 2 research cards. Kinds: `function`.

```
() -> 'CuPyRuntimeStatus'
```

Cards: C.1E cupy_runtime_status, C.5B cupy_runtime_status.

SIG-0004. 6 research cards. Kinds: `function`.

```
() -> 'dict[str, bool]'
```

Cards: C.1E available_ad_frameworks, C.5A available_ad_frameworks, C.1E available_autodiff_backends, C.5A available_autodiff_backends, C.1F available_latex_engines, C.6 available_latex_engines.

SIG-0008. 4 research cards. Kinds: `class` and `function`.

```
() -> 'None'
```

Cards: C.1F FunctionRegistry, C.3A FunctionRegistry, C.2C register_coordinate_codecs, C.3A register_generating_codecs.

SIG-0010. 15 research cards. Kinds: `function`.

```
() -> 'PolynomialFamily'
```

Cards: C.2B bernoulli_polynomial_family, C.2B binomial_polynomial_family, C.2B chebyshev_t_polynomial_family, C.2B chebyshev_u_polynomial_family, C.2B euler_polynomial_family, C.2B falling_factorial_polynomial_family, C.2B fibonacci_polynomial_family, C.2B hermite_physicists_polynomial_family, C.2B hermite_probabilists_polynomial_family, C.2B laguerre_polynomial_family, C.2B legendre_polynomial_family, C.2B lucas_polynomial_family, C.2B monomial_polynomial_family, C.2B pell_polynomial_family, C.2B rising_factorial_polynomial_family.

SIG-0011. 2 research cards. Kinds: `function`.

```
() -> 'PolynomialFamilyCatalog'
```

Cards: C.1C classical_polynomial_catalog, C.2B classical_polynomial_catalog.

SIG-0012. 6 research cards. Kinds: `function`.

```
() -> 'RecurrenceModel'
```

Cards: C.2C fibonacci_recurrence_model, C.2C jacobsthal_recurrence_model, C.2C lucas_recurrence_model, C.2C padovan_recurrence_model, C.2C pell_recurrence_model, C.2C tribonacci_recurrence_model.

SIG-0013. 5 research cards. Kinds: `function`.

```
() -> 'Sequence'
```

Cards: C.1G apery_sequence, C.1G fibonacci_sequence, C.1G motzkin_sequence, C.1G partition_sequence, C.1G thue_morse_sequence.

SIG-0016. 2 research cards. Kinds: `function`.

```
() -> 'tuple[PortableCodec, ...]'
```

Cards: C.1F portable_codecs, C.6 portable_codecs.

SIG-0017. 2 research cards. Kinds: `function`.

```
() -> 'tuple[PortableMigration, ...]'
```

Cards: C.1F portable_migrations, C.6 portable_migrations.

SIG-0018. 2 research cards. Kinds: `function`.

```
() -> 'tuple[RecurrenceModel, ...]'
```

Cards: C.1C recurrence_model_catalog, C.2C recurrence_model_catalog.

SIG-0019. 2 research cards. Kinds: `function`.

```
() -> 'tuple[str, ...]'
```

Cards: C.1F available_export_formats, C.6 available_export_formats.

SIG-0020. 2 research cards. Kinds: `function`.

```
() -> komorebi_math.registry.FeatureSpec
```

Cards: C.5A status, C.4A status.

SIG-0021. 2 research cards. Kinds: `class` and `function`.

```
() -> None
```

Cards: C.1B RationalSpace, C.5B require.

SIG-0024. 2 research cards. Kinds: `function`.

```
(*, name: 'str' = 'exponential') -> 'WeightedMonomialBasis'
```

Cards: C.2C exponential_basis, C.3A exponential_basis.

SIG-0025. 2 research cards. Kinds: `function`.

```
(*, name: 'str' = 'fibonacci-factorial') -> 'WeightedMonomialBasis'
```

Cards: C.2C fibonacci_factorial_basis, C.3A fibonacci_factorial_basis.

SIG-0026. 3 research cards. Kinds: `function`.

```
(*, name: 'str' = 'ordinary') -> 'WeightedMonomialBasis'
```

Cards: C.1F ordinary_basis, C.2C ordinary_basis, C.3A ordinary_basis.

SIG-0034. 2 research cards. Kinds: `function`.

```
(*, tag: 'str', from_version: 'int', to_version: 'int', migrator: 'Migrator', replace: 'bool' = False) -> 'None'
```

Cards: C.1F register_portable_migration, C.6 register_portable_migration.

SIG-0035. 33 research cards. Kinds: `class` and `data`.

```
(*args, **kwargs)
```

Cards: C.4A AdaptiveMethod, C.4A BiquadKind, C.4A BoundaryMode, C.3A CoefficientDomain, C.4A ConvolutionMode, C.1E CuPyDeviceDerivativeProvider, C.5B CuPyDeviceDerivativeProvider, C.5A DerivativeProvider, C.4A DetrendMode, C.5A DirectionalDerivative, C.1E DirectionalJVP, C.5A DirectionalJVP, C.6 Exportable, C.4A FIRKind, C.4A FIRMode, C.2C Matrix, C.1D Measure, C.2C ModularMatrix, C.2C ModularVector, C.1E MultilinearOperation, C.5B MultilinearOperation, C.5A Partition, C.2C Polynomial, C.2C RecurrenceLike, C.4A SpectralAverage, C.4A SpectralScaling, C.4A SpectrogramScaling, C.4A ThresholdMode, C.4A ThresholdRule, C.2C Variance, C.2C Vector, C.4A WindowName, C.4A WindowSpec.

SIG-0037. 2 research cards. Kinds: `function`.

```
(*maps: 'Expression') -> 'Expression'
```

Cards: C.1B compose, C.1B then.

SIG-0039. 11 research cards. Kinds: `class`.

```
(*values)
```

Cards: C.1A ClaimStatus, C.1G CompletionGate, C.1D ErrorGuarantee, C.1A Exactness, C.1F ExportFormat, C.6 ExportFormat, C.1G FeatureStatus, C.1F LatexEngine, C.6 LatexEngine, C.1B PrimitiveKind, C.1G UnifiedStage.

SIG-0045. 2 research cards. Kinds: `class`.

```
(address: 'PacketJetAddress', source_dimension: 'int', target_dimension: 'int', provider_name: 'str', exactness:
   ↪ 'Exactness', point: 'tuple[Any, ...]', coefficients: 'tuple[PacketJetCoefficient, ...]',
   ↪ capture_certificate: 'Certificate') -> None
```

Cards: C.1E PacketJetSnapshot, C.5B PacketJetSnapshot.

SIG-0052. 2 research cards. Kinds: `class`.

```
(analysis: 'Matrix', synthesis: 'Matrix', source: 'str' = 'source', coordinates: 'str' = 'coordinates', theorem:
   ↪ 'str' = 'KM-AS-EXACT-01') -> None
```

Cards: C.1C ExactAnalysisSynthesisSystem, C.2C ExactAnalysisSynthesisSystem.

SIG-0053. 2 research cards. Kinds: `class`.

```
(analysis: 'np.ndarray', synthesis: 'np.ndarray', source: 'str' = 'source', coordinates: 'str' = 'coordinates',
   ↪ tolerance: 'float' = 1e-12, theorem: 'str' = 'KM-AS-01') -> None
```

Cards: C.1C AnalysisSynthesisSystem, C.2C AnalysisSynthesisSystem.

SIG-0054. 2 research cards. Kinds: `class`.

```
(analysis_filters: 'Iterable[Any]', synthesis_filters: 'Iterable[Any]', decimation: 'int') -> 'None'
```

Cards: C.1F PeriodicFilterBank, C.4B PeriodicFilterBank.

SIG-0072. 2 research cards. Kinds: `class`.

```
(basis: 'WeightedMonomialBasis', coefficients: 'Iterable[int | Fraction]') -> 'None'
```

Cards: C.1F FormalSeries, C.3A FormalSeries.

SIG-0074. 2 research cards. Kinds: `class`.

```
(basis: 'WeightedMonomialBasis', domain: 'CoefficientDomain[Any]', coefficients: 'Iterable[Any]', *, budget:
    ↪ 'GeneratingBudget' = GeneratingBudget(maximum_degree=100000, maximum_coefficient_bits=4000000,
    ↪ maximum_matrix_dimension=4096, maximum_reachable_states=100000, maximum_local_combinations=1000000,
    ↪ maximum_modulus=2000000, maximum_serialized_bytes=67108864, maximum_recursion_depth=1000)) -> 'None'
```

Cards: C.1F FiniteNormalizedSeries, C.3A FiniteNormalizedSeries.

SIG-0080. 2 research cards. Kinds: `class`.

```
(blocks: 'tuple[tuple[np.ndarray, ...], ...]', order: 'int', input_length: 'int', output_length: 'int') -> None
```

Cards: C.4A FinitePolyphaseKernel, C.4A PolyphaseBlockMatrix.

SIG-0089. 2 research cards. Kinds: `class`.

```
(children: "tuple['OrderedRootedTree', ...]" = ()) -> None
```

Cards: C.1E OrderedRootedTree, C.5B OrderedRootedTree.

SIG-0096. 2 research cards. Kinds: `class`.

```
(coefficient_start: 'int', coefficient_stop: 'int', coefficients: 'tuple[PacketJetCoefficient, ...]',
    ↪ complete_snapshot: 'bool', certificate: 'Certificate') -> None
```

Cards: C.1E PacketJetStreamSelection, C.5B PacketJetStreamSelection.

SIG-0103. 2 research cards. Kinds: `class`.

```
(coefficients: 'Iterable[Scalar]', name: 'str' = 'shift polynomial') -> 'None'
```

Cards: C.1C ShiftPolynomial, C.2C ShiftPolynomial.

SIG-0109. 2 research cards. Kinds: `function`.

```
(coefficients: 'RecurrenceLike') -> 'Matrix'
```

Cards: C.1C companion_matrix, C.2C companion_matrix.

SIG-0112. 2 research cards. Kinds: `function`.

```
(coefficients: 'RecurrenceLike', length: 'int', *, name: 'str | None' = None, budget: 'CoordinateBudget' =
    ↪ CoordinateBudget(maximum_matrix_dimension=512, maximum_polynomial_degree=512, maximum_horizon=4096,
    ↪ maximum_exact_entries=1000000)) -> 'CoordinateTransform'
```

Cards: C.2C recurrence_analysis_transform, C.2C recurrence_synthesis_transform.

SIG-0118. 2 research cards. Kinds: `function`.

```
(collapse: 'CyclotomicCollapse') -> 'bool'
```

Cards: C.3B verify_cyclotomic_coboundary, C.3B verify_cyclotomic_exponent_relation.

SIG-0122. 2 research cards. Kinds: `class`.

```
(components: 'Iterable[TruncatedPowerTerm]') -> 'None'
```

Cards: C.1F SingularPacket1D, C.4B SingularPacket1D.

SIG-0128. 2 research cards. Kinds: `class`.

```
(correction_map: 'Any', *, tolerance: 'float' = 1e-10) -> 'None'
```

Cards: C.1E HilbertObstructionProjector, C.5A HilbertObstructionProjector.

SIG-0129. 2 research cards. Kinds: `class`.

```
(correction_map: 'Any', defect: 'Any', *, backend: 'str | LinearAlgebraBackend | None' = None, tolerance: 'float' =
    ↪ 1e-10) -> 'None'
```

Cards: C.1E LinearObstructionProblem, C.5A LinearObstructionProblem.

SIG-0130. 2 research cards. Kinds: `function`.

```
(count: 'int') -> 'RecurrenceCoordinateRecord'
```

Cards: C.1C euler_fibonacci_coordinates, C.2C euler_fibonacci_coordinates.

SIG-0132. 9 research cards. Kinds: `function`.

```
(count: 'int') -> 'tuple[int, ...]'
```

Cards: C.3A apery_numbers, C.3A bell_numbers, C.3A catalan_numbers, C.3A central_binomial_numbers, C.2C fibonacci_numbers, C.3A fibonacci_numbers, C.3A partition_numbers, C.3A signed_euler_numbers, C.2C signed_secant_euler_numbers.

SIG-0136. 2 research cards. Kinds: `function`.

```
(cutoff: 'float', **options: 'Any') -> 'FIRFilter'
```

Cards: C.4A design_highpass, C.4A design_lowpass.

SIG-0137. 2 research cards. Kinds: `function`.

```
(cutoff: 'tuple[float, float]', **options: 'Any') -> 'FIRFilter'
```

Cards: C.4A design_bandpass, C.4A design_bandstop.

SIG-0140. 11 research cards. Kinds: `function`.

```
(degree: 'int') -> 'Polynomial'
```

Cards: C.2B bernoulli_polynomial, C.2B binomial_polynomial, C.2B chebyshev_t_polynomial, C.2B chebyshev_u_polynomial, C.2B euler_polynomial, C.2B falling_factorial_polynomial, C.2B hermite_physicists_polynomial, C.2B hermite_probabilists_polynomial, C.2B laguerre_polynomial, C.2B legendre_polynomial, C.2B rising_factorial_polynomial.

SIG-0141. 7 research cards. Kinds: `function`.

```
(degree: 'int', *, budget: 'CoordinateBudget' = CoordinateBudget(maximum_matrix_dimension=512,
    ↪ maximum_polynomial_degree=512, maximum_horizon=4096, maximum_exact_entries=1000000)) ->
    ↪ 'CoordinateTransform'
```

Cards: C.1C euler_to_fibonacci_transform, C.2B euler_to_fibonacci_transform, C.2A falling_factorial_to_monomial_transform, C.2A monomial_to_falling_factorial_transform, C.2A monomial_to_rising_factorial_transform, C.2A rising_factorial_to_monomial_transform, C.2A rising_to_falling_factorial_transform.

SIG-0149. 2 research cards. Kinds: `class`.

```
(destination: 'Path', active_path: 'Path', completed: 'bool', resumed: 'bool', chunks_written: 'int',
    ↪ evaluations_written: 'int', coefficients_written: 'int', manifest: 'PacketJetStreamManifest', certificate:
    ↪ 'Certificate') -> None
```

Cards: C.1E PacketJetStreamWriteResult, C.5B PacketJetStreamWriteResult.

SIG-0156. 2 research cards. Kinds: `function`.

```
(dimension: 'int', *, name: 'str' = 'standard') -> 'FiniteBasis'
```

Cards: C.1C standard_basis, C.2C standard_basis.

SIG-0157. 2 research cards. Kinds: `class`.

```
(dimension: 'int', groups: 'tuple[tuple[int, ...], ...]') -> None
```

Cards: C.1C CoefficientPartition, C.2C CoefficientPartition.

SIG-0158. 2 research cards. Kinds: `class`.

```
(dimension: 'int', levels: 'Mapping[int, Any]', *, prime: 'int') -> 'None'
```

Cards: C.1E FilteredVectorSpace, C.5A FilteredVectorSpace.

SIG-0162. 2 research cards. Kinds: `class`.

```
(dimensions: 'Mapping[int, int]', differentials: 'Mapping[int, Any]', filtrations: 'Mapping[int,
    ↪ FilteredVectorSpace]', *, prime: 'int') -> 'None'
```

Cards: C.1E FiniteFilteredCochainComplex, C.5A FiniteFilteredCochainComplex.

SIG-0165. 2 research cards. Kinds: `function`.

```
(document: 'MathDocument') -> 'str'
```

Cards: C.6 render_markdown, C.6 render_text.

SIG-0167. 2 research cards. Kinds: `function`.

```
(document: 'MathDocument', *, options: 'LatexRenderOptions | None' = None) -> 'str'
```

Cards: C.1F render_latex, C.6 render_latex.

SIG-0168. 2 research cards. Kinds: `function`.

```
(document: 'MathDocument', path: 'str | Path', *, format: 'ExportFormat | str | None' = None, options:
    ↪ 'ExportOptions | None' = None) -> 'ExportResult'
```

Cards: C.1F export_document, C.6 export_document.

SIG-0169. 2 research cards. Kinds: `class`.

```
(document_class: 'str' = 'article', class_options: 'tuple[str, ...]' = ('11pt',), packages: 'tuple[str, ...]' =
    ↪ ('amsmath', 'amssymb', 'booktabs', 'longtable', 'array', 'graphicx', 'hyperref', 'geometry', 'microtype'),
    ↪ include_table_of_contents: 'bool' = False, number_sections: 'bool' = True, geometry: 'str' = 'margin=1in')
    ↪ -> None
```

Cards: C.1F LatexRenderOptions, C.6 LatexRenderOptions.

SIG-0170. 2 research cards. Kinds: `class`.

```
(domain: 'AnalysisSynthesisSystem', codomain: 'AnalysisSynthesisSystem', operator: 'np.ndarray', theorem: 'str' =
   ↪ 'KM-AS-MAP-01') -> None
```

Cards: C.1C AnalysisSynthesisMap, C.2C AnalysisSynthesisMap.

SIG-0171. 2 research cards. Kinds: `class`.

```
(domain: 'ExactAnalysisSynthesisSystem', codomain: 'ExactAnalysisSynthesisSystem', operator: 'Matrix', theorem:
   ↪ 'str' = 'KM-AS-EXACT-MAP-01') -> None
```

Cards: C.1C ExactAnalysisSynthesisMap, C.2C ExactAnalysisSynthesisMap.

SIG-0172. 2 research cards. Kinds: `class`.

```
(duration_ns: 'int', traced_peak_bytes: 'int') -> None
```

Cards: C.1E RuntimeProfileSample, C.5B RuntimeProfileSample.

SIG-0177. 2 research cards. Kinds: `class`.

```
(engine: 'LatexEngine | str' = <LatexEngine.AUTO: 'auto'>, passes: 'int' = 2, timeout: 'float' = 120.0,
   ↪ shell_escape: 'bool' = False, keep_auxiliary: 'bool' = False, jobname: 'str | None' = None) -> None
```

Cards: C.1F LatexCompileOptions, C.6 LatexCompileOptions.

SIG-0178. 2 research cards. Kinds: `class`.

```
(equations: 'dict[str, Any]', reconstructed: 'Any', direct: 'Any', certificate: 'Certificate') -> None
```

Cards: C.5A DividedSquareResult, C.5A MaurerCartanResult.

SIG-0181. 2 research cards. Kinds: `class`.

```
(estimate: 'PacketJetComplexityEstimate', stages: 'tuple[RuntimeStageProfile, ...]', environment: 'Mapping[str,
   ↪ Any]', certificate: 'Certificate') -> None
```

Cards: C.1E PacketJetProfileReport, C.5B PacketJetProfileReport.

SIG-0183. 2 research cards. Kinds: `class`.

```
(evaluations: 'tuple[PacketJetEvaluation, ...]', reconstructed: 'Any', direct: 'Any', derivative_calls: 'int',
   ↪ input_projections: 'int', output_projections: 'int', pruned_blocks: 'int', backend: 'str', returned_to_host:
   ↪ 'bool', certificate: 'Certificate') -> None
```

Cards: C.1E PacketJetBatchResult, C.5B PacketJetBatchResult.

SIG-0185. 2 research cards. Kinds: `class`.

```
(exact: 'bool', unique: 'bool', rank: 'int', horizon: 'int', state: 'Vector | None', predicted: 'Vector', residual:
   ↪ 'Vector') -> None
```

Cards: C.1C StateRecovery, C.2C StateRecovery.

SIG-0189. 2 research cards. Kinds: `function`.

```
(expression: 'Expression', *, budget: 'ExpressionBudget | None' = None) -> 'str'
```

Cards: C.1B expression_digest, C.1B expression_to_json.

SIG-0200. 2 research cards. Kinds: `class`.

```
(expressions: 'Sequence[Any]', variables: 'Sequence[Any]', *, name: 'str' = 'SymPy derivative provider') -> 'None'
```

Cards: C.1E SympyDerivativeProvider, C.5A SympyDerivativeProvider.

SIG-0204. 2 research cards. Kinds: `function`.

```
(family: 'PacketJetFamily', *, backend: 'str', timing_scope: 'str' = 'kernel') -> 'Any'
```

Cards: C.1E packet_jet_benchmark_contract, C.5B packet_jet_benchmark_contract.

SIG-0205. 2 research cards. Kinds: `function`.

```
(family: 'PacketJetFamily', *, tolerance: 'float | None' = None, max_evaluations: 'int' = 100000) ->
    ↪ 'PacketJetSupport'
```

Cards: C.1E packet_jet_support, C.5B packet_jet_support.

SIG-0206. 2 research cards. Kinds: `function`.

```
(family: 'PacketJetFamily', directions: 'Sequence[Any]', *, support: 'PacketJetSupport | None' = None, jet:
    ↪ 'PacketJet | None' = None, repetitions: 'int' = 5, warmup: 'int' = 1, max_addresses: 'int' = 1000000,
    ↪ max_snapshot_evaluations: 'int' = 100000, include_stream_write: 'bool' = False, stream_chunk_size: 'int' =
    ↪ 1024) -> 'PacketJetProfileReport'
```

Cards: C.1E profile_packet_jet_runtime, C.5B profile_packet_jet_runtime.

SIG-0207. 2 research cards. Kinds: `function`.

```
(family: 'PacketJetFamily', directions: 'Sequence[Any]', *, support: 'PacketJetSupport | None' = None, tolerance:
    ↪ 'float | None' = None, max_addresses: 'int' = 1000000) -> 'PacketJetBatchResult'
```

Cards: C.1E execute_packet_jets, C.5B execute_packet_jets.

SIG-0208. 2 research cards. Kinds: `class`.

```
(family: 'PacketJetFamily', plan: 'PacketJetExecutionPlan | None' = None) -> 'None'
```

Cards: C.1E CuPyPacketJetExecutor, C.5B CuPyPacketJetExecutor.

SIG-0209. 2 research cards. Kinds: `function`.

```
(family: 'PacketJetFamily', support: 'PacketJetSupport | None' = None, *, max_addresses: 'int' = 1000000) ->
    ↪ 'PacketJetExecutionPlan'
```

Cards: C.1E compile_packet_jet_plan, C.5B compile_packet_jet_plan.

SIG-0210. 2 research cards. Kinds: `function`.

```
(family: 'PacketJetFamily', support: 'PacketJetSupport | None' = None, *, plan: 'PacketJetExecutionPlan | None' =
    ↪ None, max_addresses: 'int' = 1000000) -> 'PacketJetComplexityEstimate'
```

Cards: C.1E estimate_packet_jet_complexity, C.5B estimate_packet_jet_complexity.

SIG-0214. 2 research cards. Kinds: `function`.

```
(family: 'PolynomialFamily', degree: 'int', *, budget: 'CoordinateBudget' =
    ↪ CoordinateBudget(maximum_matrix_dimension=512, maximum_polynomial_degree=512, maximum_horizon=4096,
    ↪ maximum_exact_entries=1000000)) -> 'PolynomialCoordinateOperator'
```

Cards: C.2B derivative_coordinate_operator, C.2B integral_coordinate_operator.

SIG-0221. 2 research cards. Kinds: `function`.

```
(family: 'PolynomialFamily', nodes: 'Sequence[Scalar]', *, budget: 'CoordinateBudget' =
    ↪ CoordinateBudget(maximum_matrix_dimension=512, maximum_polynomial_degree=512, maximum_horizon=4096,
    ↪ maximum_exact_entries=1000000)) -> 'CoordinateTransform'
```

Cards: C.2B evaluation_transform, C.2B interpolation_transform.

SIG-0226. 2 research cards. Kinds: `class`.

```
(fine_map: 'Callable[[Any], Any]', coarse_map: 'Callable[[Any], Any]', coarsen_source: 'Any', coarsen_target:
    ↪ 'Any') -> None
```

Cards: C.1E NonlinearRefinementDefect, C.5A NonlinearRefinementDefect.

SIG-0234. 4 research cards. Kinds: `function`.

```
(framework: 'str', function: 'Callable[[Any], Any]', source_dimension: 'int', target_dimension: 'int', *,
    ↪ max_order: 'int' = 4, **options: 'Any') -> 'JaxDerivativeProvider | TorchDerivativeProvider'
```

Cards: C.1E autodiff_provider, C.5A autodiff_provider, C.1E automatic_derivative_provider, C.5A automatic_derivative_provider.

SIG-0238. 2 research cards. Kinds: `class`.

```
(function: 'Callable[[Any], Any]', derivatives: 'Mapping[int, DirectionalDerivative]', source_dimension: 'int',
    ↪ target_dimension: 'int', exactness: 'Exactness' = <Exactness.NUMERICAL: 'numerical'>, name: 'str' =
    ↪ 'declared derivative provider') -> None
```

Cards: C.1E CallableDerivativeProvider, C.5A CallableDerivativeProvider.

SIG-0239. 2 research cards. Kinds: `class`.

```
(function: 'Callable[[Any], Any]', jvp: 'DirectionalJVP', source_dimension: 'int', target_dimension: 'int',
    ↪ max_order: 'int' = 4, exactness: 'Exactness' = <Exactness.NUMERICAL: 'numerical'>, name: 'str' = 'automatic
    ↪ differentiation provider') -> None
```

Cards: C.1E AutomaticDifferentiationProvider, C.5A AutomaticDifferentiationProvider.

SIG-0240. 2 research cards. Kinds: `class`.

```
(function: 'Callable[[Any], Any]', source: 'PacketSystem', target: 'PacketSystem', name: 'str' = 'nonlinear packet
    ↪ map') -> None
```

Cards: C.1E NonlinearPacketMap, C.5A NonlinearPacketMap.

SIG-0241. 2 research cards. Kinds: `class`.

```
(function: 'Callable[[Any], Any]', source_dimension: 'int', target_dimension: 'int', max_order: 'int' = 4, dtype:
    ↪ 'str | None' = None, name: 'str' = 'JAX derivative provider', exactness: 'Exactness' = <Exactness.NUMERICAL:
    ↪ 'numerical'>) -> None
```

Cards: C.1E JaxDerivativeProvider, C.5A JaxDerivativeProvider.

SIG-0242. 2 research cards. Kinds: `class`.

```
(function: 'Callable[[Any], Any]', source_dimension: 'int', target_dimension: 'int', max_order: 'int' = 4, dtype:
    ↪ 'str' = 'float64', device: 'str | None' = None, name: 'str' = 'PyTorch derivative provider', exactness:
    ↪ 'Exactness' = <Exactness.NUMERICAL: 'numerical'>) -> None
```

Cards: C.1E TorchDerivativeProvider, C.5A TorchDerivativeProvider.

SIG-0243. 2 research cards. Kinds: `class`.

```
(function: 'Callable[[np.ndarray], Any]', source_dimension: 'int', target_dimension: 'int', step: 'float' =
    ↪ 0.0001, max_order: 'int' = 4, name: 'str' = 'central finite difference provider', exactness: 'Exactness' =
    ↪ <Exactness.NUMERICAL: 'numerical'>) -> None
```

Cards: C.1E FiniteDifferenceDerivativeProvider, C.5A FiniteDifferenceDerivativeProvider.

SIG-0244. 2 research cards. Kinds: `class`.

```
(function: 'Callable[[tuple[int, ...]], Iterable[int]]', jacobian: 'Callable[[tuple[int, ...]], Any]', point:
    ↪ 'Any', *, prime: 'int', depth: 'int') -> 'None'
```

Cards: C.1E PolynomialLiftProblem, C.5A PolynomialLiftProblem.

SIG-0245. 2 research cards. Kinds: `function`.

```
(function: 'CustomFunctionSpec', degree: 'int', *, include_metadata: 'bool' = True) -> 'FrozenGeneratingFunction'
```

Cards: C.1F freeze_custom_function, C.3A freeze_custom_function.

SIG-0250. 2 research cards. Kinds: `class`.

```
(gcd: 'Vector', left_multiplier: 'Vector', right_multiplier: 'Vector', reconstructed: 'Vector', expected:
    ↪ 'Vector', valid: 'bool') -> None
```

Cards: C.1C BezoutDefectIdentity, C.2C BezoutDefectIdentity.

SIG-0255. 2 research cards. Kinds: `function`.

```
(group: 'FiniteAbelianGroup', components: 'Mapping[str, Any]', differential: 'LinearOperation', bracket:
    ↪ 'BracketOperation', *, exactness: 'Exactness' = <Exactness.NUMERICAL: 'numerical'>, tolerance: 'float' =
    ↪ 1e-10) -> 'MaurerCartanResult'
```

Cards: C.1E maurer_cartan_packets, C.5A maurer_cartan_packets.

SIG-0256. 2 research cards. Kinds: `function`.

```
(group: 'FiniteAbelianGroup', components: 'Mapping[str, Any]', differential: 'LinearOperation', bracket:
    ↪ 'BracketOperation', divided_square: 'LinearOperation', *, exactness: 'Exactness' = <Exactness.NUMERICAL:
    ↪ 'numerical'>, tolerance: 'float' = 1e-10) -> 'DividedSquareResult'
```

Cards: C.1E divided_square_packets, C.5A divided_square_packets.

SIG-0257. 2 research cards. Kinds: `function`.

```
(group: 'FiniteAbelianGroup', components: 'Mapping[str, Any]', operations: 'Mapping[int,
    ↪ GradedOperationCallable]', *, exactness: 'Exactness' = <Exactness.NUMERICAL: 'numerical'>, tolerance:
    ↪ 'float' = 1e-10) -> 'HigherMaurerCartanResult'
```

Cards: C.1E higher_maurer_cartan_packets, C.5A higher_maurer_cartan_packets.

SIG-0258. 2 research cards. Kinds: `class`.

```
(group: 'FiniteAbelianGroup', packets: 'PacketSystem', arity: 'int', operation: 'GradedOperationCallable', *,
    ↪ exactness: 'Exactness' = <Exactness.NUMERICAL: 'numerical'>, tolerance: 'float | None' = None, name: 'str' =
    ↪ 'graded packet operation') -> 'None'
```

Cards: C.1E GradedPacketOperation, C.5A GradedPacketOperation.

SIG-0265. 2 research cards. Kinds: `class`.

```
(headers: 'tuple[str, ...]', rows: 'tuple[tuple[Any, ...], ...]', caption: 'str | None' = None, label: 'str | None'
   ↪  = None, alignment: 'tuple[str, ...] | None' = None) -> None
```

Cards: C.1F Table, C.6 Table.

SIG-0273. 3 research cards. Kinds: `function`.

```
(index: 'int') -> 'Fraction'
```

Cards: C.2B bernoulli_number, C.2B euler_polynomial_at_zero, C.2B secant_euler_from_fibonacci_coordinates.

SIG-0274. 2 research cards. Kinds: `function`.

```
(index: 'int') -> 'int'
```

Cards: C.5B catalan_number, C.1G signed_euler_number.

SIG-0275. 3 research cards. Kinds: `function`.

```
(index: 'int') -> 'Polynomial'
```

Cards: C.2B fibonacci_polynomial, C.2B lucas_polynomial, C.2B pell_polynomial.

SIG-0276. 3 research cards. Kinds: `function`.

```
(index: 'int', base: 'int') -> 'int'
```

Cards: C.3B base_digit_sum, C.3B q_factorial, C.3B q_integer.

SIG-0279. 2 research cards. Kinds: `class`.

```
(index: 'int', filename: 'str', basis_start: 'int', basis_stop: 'int', coefficient_start: 'int', coefficient_stop:
   ↪  'int', payload_sha256: 'str', byte_count: 'int') -> None
```

Cards: C.1E PacketJetStreamChunkRecord, C.5B PacketJetStreamChunkRecord.

SIG-0280. 3 research cards. Kinds: `function`.

```
(index: 'int', prime: 'int') -> 'int'
```

Cards: C.3B factorial_valuation, C.3B one_borrow_count, C.3B zero_borrow_count.

SIG-0287. 2 research cards. Kinds: `function`.

```
(index: 'int', prime: 'int', factorial_power: 'int', *, budget: 'GeneratingBudget' =
   ↪  GeneratingBudget(maximum_degree=100000, maximum_coefficient_bits=4000000, maximum_matrix_dimension=4096,
   ↪  maximum_reachable_states=100000, maximum_local_combinations=1000000, maximum_modulus=2000000,
   ↪  maximum_serialized_bytes=67108864, maximum_recursion_depth=1000)) -> 'BellDigitReduction'
```

Cards: C.1F generalized_bell_digit_reduction, C.3B generalized_bell_digit_reduction.

SIG-0297. 2 research cards. Kinds: `function`.

```
(index: 'int', subindex: 'int', base: 'int', prime: 'int') -> 'QKummerDecomposition'
```

Cards: C.1F q_binomial_valuation_decomposition, C.3B q_binomial_valuation_decomposition.

SIG-0300. 2 research cards. Kinds: `function`.

```
(index: 'int', subindex: 'int', prime: 'int') -> 'int'
```

Cards: C.3B binomial_valuation, C.3B borrow_count.

SIG-0313. 2 research cards. Kinds: `class`.

```
(installed: 'bool', runtime_available: 'bool', device_count: 'int', runtime_version: 'int | None', error: 'str |
    ↪ None' = None) -> None
```

Cards: C.1E CuPyRuntimeStatus, C.5B CuPyRuntimeStatus.

SIG-0317. 2 research cards. Kinds: `function`.

```
(jet: 'PacketJet', *, tolerance: 'float | None' = None, max_evaluations: 'int' = 100000) -> 'PacketJetSnapshot'
```

Cards: C.1E snapshot_packet_jet, C.5B snapshot_packet_jet.

SIG-0318. 2 research cards. Kinds: `function`.

```
(jet: 'PacketJet', path: 'str | os.PathLike[str]', *, tolerance: 'float | None' = None, max_evaluations: 'int' =
    ↪ 10000000, chunk_size: 'int' = 1024, evaluation_budget: 'int | None' = None, resume: 'bool' = False,
    ↪ overwrite: 'bool' = False) -> 'PacketJetStreamWriteResult'
```

Cards: C.1E capture_packet_jet_stream, C.5B capture_packet_jet_stream.

SIG-0320. 2 research cards. Kinds: `function`.

```
(kernel: 'Any', input_length: 'int', order: 'int', *, mode: 'ConvolutionMode' = 'full') -> 'PolyphaseBlockMatrix'
```

Cards: C.4A finite_polyphase_blocks, C.4A finite_polyphase_filter_kernels.

SIG-0331. 2 research cards. Kinds: `class`.

```
(labels: 'Sequence[str]', addition: 'Mapping[tuple[str, str], str]', *, zero: 'str', validate: 'bool' = True) ->
    ↪ 'None'
```

Cards: C.1E FiniteAbelianGroup, C.5A FiniteAbelianGroup.

SIG-0332. 2 research cards. Kinds: `class`.

```
(latex: 'str', label: 'str | None' = None, numbered: 'bool' = False, description: 'str | None' = None) -> None
```

Cards: C.1F Equation, C.6 Equation.

SIG-0339. 2 research cards. Kinds: `function`.

```
(left: 'CoordinateTransform', right: 'CoordinateTransform') -> 'CoordinateTransform'
```

Cards: C.2A direct_sum_transform, C.2A tensor_product_transform.

SIG-0341. 2 research cards. Kinds: `function`.

```
(left: 'FiniteBasis', right: 'FiniteBasis', *, name: 'str | None' = None) -> 'FiniteBasis'
```

Cards: C.2C direct_sum_basis, C.2C tensor_product_basis.

SIG-0343. 4 research cards. Kinds: `function`.

```
(left: 'Matrix', right: 'Matrix') -> 'Matrix'
```

Cards: C.2A matrix_add, C.2A matrix_multiply, C.2A matrix_subtract, C.2B operator_commutator.

SIG-0348. 3 research cards. Kinds: `function`.

```
(left: 'Sequence[Scalar]', right: 'Sequence[Scalar]') -> 'Polynomial'
```

Cards: C.2B polynomial_add, C.2B polynomial_multiply, C.2B polynomial_subtract.

SIG-0356. 2 research cards. Kinds: `function`.

```
(length: 'int', modulus: 'int') -> 'CoordinateTransform'
```

Cards: C.1C residue_class_transform, C.2A residue_class_transform.

SIG-0363. 2 research cards. Kinds: `class`.

```
(linear_map: 'Any', right_inverse: 'Any', nonlinear: 'Callable[[np.ndarray], Any]', target: 'Any', *, radius:
   ↪ 'float', lipschitz_bound: 'float', tolerance: 'float' = 1e-10) -> 'None'
```

Cards: C.1E RightInverseCorrectionProblem, C.5A RightInverseCorrectionProblem.

SIG-0367. 2 research cards. Kinds: `class`.

```
(location: 'float', amplitude: 'complex', order: 'int' = 0) -> None
```

Cards: C.4B TruncatedPowerPacket, C.4B TruncatedPowerTerm.

SIG-0376. 2 research cards. Kinds: `class`.

```
(mapping: 'AnalysisSynthesisMap', source_partition: 'CoefficientPartition', target_partition:
   ↪ 'CoefficientPartition') -> None
```

Cards: C.1C AnalysisSynthesisCoefficientBlocks, C.2C AnalysisSynthesisCoefficientBlocks.

SIG-0378. 2 research cards. Kinds: `function`.

```
(mask: 'Iterable[Scalar]', radix: 'int', *, budget: 'GeneratingBudget' = GeneratingBudget(maximum_degree=100000,
   ↪ maximum_coefficient_bits=4000000, maximum_matrix_dimension=4096, maximum_reachable_states=100000,
   ↪ maximum_local_combinations=1000000, maximum_modulus=2000000, maximum_serialized_bytes=67108864,
   ↪ maximum_recursion_depth=1000)) -> 'FiniteMaskCarryPlan'
```

Cards: C.1F compile_finite_mask_carry_plan, C.3B compile_finite_mask_carry_plan.

SIG-0379. 2 research cards. Kinds: `function`.

```
(mask: 'Iterable[Scalar]', radix: 'int', degree: 'int', *, budget: 'GeneratingBudget' =
   ↪ GeneratingBudget(maximum_degree=100000, maximum_coefficient_bits=4000000, maximum_matrix_dimension=4096,
   ↪ maximum_reachable_states=100000, maximum_local_combinations=1000000, maximum_modulus=2000000,
   ↪ maximum_serialized_bytes=67108864, maximum_recursion_depth=1000)) -> 'tuple[Fraction, ...]'
```

Cards: C.1F mahler_mask_product, C.3B mahler_mask_product.

SIG-0392. 2 research cards. Kinds: `class`.

```
(matrix: 'Any', rhs: 'Any', point: 'Any', *, prime: 'int', depth: 'int') -> 'None'
```

Cards: C.1E LinearLiftProblem, C.5A LinearLiftProblem.

SIG-0393. 2 research cards. Kinds: `function`.

```
(matrix: 'Any', rhs: 'Any', prime: 'int') -> 'ModularLinearSolution'
```

Cards: C.1E solve_mod_prime, C.5A solve_mod_prime.

SIG-0398. 2 research cards. Kinds: `function`.

```
(matrix: 'Matrix') -> 'Matrix'
```

Cards: C.2A inverse_matrix, C.2A transpose.

SIG-0408. 2 research cards. Kinds: `class`.

```
(maximum_degree: 'int' = 100000, maximum_coefficient_bits: 'int' = 4000000, maximum_matrix_dimension: 'int' = 4096,
    ↪ maximum_reachable_states: 'int' = 100000, maximum_local_combinations: 'int' = 1000000, maximum_modulus:
    ↪ 'int' = 2000000, maximum_serialized_bytes: 'int' = 67108864, maximum_recursion_depth: 'int' = 1000) -> None
```

Cards: C.1F GeneratingBudget, C.3A GeneratingBudget.

SIG-0409. 2 research cards. Kinds: `class`.

```
(maximum_matrix_dimension: 'int' = 512, maximum_polynomial_degree: 'int' = 512, maximum_horizon: 'int' = 4096,
    ↪ maximum_exact_entries: 'int' = 1000000) -> None
```

Cards: C.1C CoordinateBudget, C.2C CoordinateBudget.

SIG-0410. 2 research cards. Kinds: `class`.

```
(metadata: 'DocumentMetadata' = <factory>, blocks: 'list[DocumentBlock]' = <factory>) -> None
```

Cards: C.1F MathDocument, C.6 MathDocument.

SIG-0411. 2 research cards. Kinds: `function`.

```
(model: 'StateSpaceModel') -> 'CoordinateTransform'
```

Cards: C.2C controllability_coordinate_transform, C.2C observability_coordinate_transform.

SIG-0420. 4 research cards. Kinds: `function`.

```
(n: 'int', k: 'int') -> 'int'
```

Cards: C.2A lah_number, C.2A stirling_first_signed, C.2A stirling_first_unsigned, C.2A stirling_second.

SIG-0421. 2 research cards. Kinds: `class`.

```
(name: 'str' = 'coordinate atlas') -> None
```

Cards: C.1C CoordinateAtlas, C.2C CoordinateAtlas.

SIG-0424. 2 research cards. Kinds: `function`.

```
(name: 'str') -> 'PolynomialFamily'
```

Cards: C.1C get_polynomial_family, C.2B get_polynomial_family.

SIG-0425. 2 research cards. Kinds: `function`.

```
(name: 'str') -> 'RecurrenceModel'
```

Cards: C.1C get_recurrence_model, C.2C get_recurrence_model.

SIG-0428. 2 research cards. Kinds: `class`.

```
(name: 'str', basis: 'WeightedMonomialBasis', builder: 'CoefficientBuilder', source_kind: 'str' = 'custom',
   ↪ metadata: 'Mapping[str, object]' = <factory>) -> None
```

Cards: C.1F CustomFunctionSpec, C.3A CustomFunctionSpec.

SIG-0430. 2 research cards. Kinds: `class`.

```
(name: 'str', basis_name: 'str', normalization: 'tuple[Fraction, ...]', coefficients: 'tuple[Fraction, ...]',
   ↪ source_kind: 'str', metadata: 'Mapping[str, Any]' = <factory>) -> None
```

Cards: C.1F FrozenGeneratingFunction, C.3A FrozenGeneratingFunction.

SIG-0432. 2 research cards. Kinds: `class`.

```
(name: 'str', coefficients: 'Vector', initial: 'Vector', defects: 'Vector') -> None
```

Cards: C.1C RecurrenceCoordinateRecord, C.2C RecurrenceCoordinateRecord.

SIG-0433. 2 research cards. Kinds: `class`.

```
(name: 'str', coefficients: 'Vector', initial_values: 'Vector' = (), description: 'str' = '') -> None
```

Cards: C.1C RecurrenceModel, C.2C RecurrenceModel.

SIG-0434. 2 research cards. Kinds: `class`.

```
(name: 'str', coefficients: 'Vector', source: 'str' = 'source formal coordinate', target: 'str' = 'target formal
   ↪ coordinate') -> None
```

Cards: C.1C FormalCoordinateChange, C.2C FormalCoordinateChange.

SIG-0439. 2 research cards. Kinds: `class`.

```
(name: 'str', dimension: 'int', kind: 'str' = 'exact', description: 'str' = '') -> None
```

Cards: C.1C CoordinateSystem, C.2C CoordinateSystem.

SIG-0442. 6 research cards. Kinds: `class`.

```
(name: 'str', gamma: 'GammaRule', *, descriptor: 'BasisDescriptor | None' = None) -> 'None'
```

Cards: C.1F NormalizedBasis, C.2C NormalizedBasis, C.3A NormalizedBasis, C.3A NormalizedMonomialBasis, C.1F WeightedMonomialBasis, C.3A WeightedMonomialBasis.

SIG-0448. 2 research cards. Kinds: `function`.

```
(name: 'str', matrix: 'Iterable[Iterable[Scalar]]', *, labels: 'Sequence[str]' = (), description: 'str' = '') ->
   ↪ 'FiniteBasis'
```

Cards: C.1C basis_from_matrix, C.2C basis_from_matrix.

SIG-0450. 2 research cards. Kinds: `function`.

```
(name: 'str', operation: 'Callable[[], T]', *, repetitions: 'int' = 5, warmup: 'int' = 1, scope: 'str' = 'Python
   ↪ call including allocations and synchronization performed by the callable') -> 'RuntimeStageProfile'
```

Cards: C.1E profile_operation, C.5B profile_operation.

SIG-0453. 2 research cards. Kinds: `class`.

```
(name: 'str', rule: 'Callable[[int], Polynomial]') -> None
```

Cards: C.1C PolynomialFamily, C.2B PolynomialFamily.

SIG-0454. 2 research cards. Kinds: `class`.

```
(name: 'str', scope: 'str', warmup: 'int', samples: 'tuple[RuntimeProfileSample, ...]') -> None
```

Cards: C.1E RuntimeStageProfile, C.5B RuntimeStageProfile.

SIG-0456. 2 research cards. Kinds: `class`.

```
(name: 'str', source_family: 'str', target_family: 'str', source_degree: 'int', target_degree: 'int', matrix:
   ↪ 'Matrix', theorem: 'str' = 'CC.O1') -> None
```

Cards: C.1C PolynomialCoordinateOperator, C.2B PolynomialCoordinateOperator.

SIG-0458. 2 research cards. Kinds: `class`.

```
(name: 'str', transition: 'Matrix', input_matrix: 'Matrix', output_matrix: 'Matrix', feedthrough: 'Matrix') -> None
```

Cards: C.1C StateSpaceModel, C.2C StateSpaceModel.

SIG-0460. 2 research cards. Kinds: `function`.

```
(name: 'str', values: 'Sequence[Any]', *, strategy: 'str | None' = None, modulus: 'int | None' = None, author: 'str
   ↪ | None' = None) -> 'MathDocument'
```

Cards: C.1F sequence_document, C.6 sequence_document.

SIG-0461. 2 research cards. Kinds: `class`.

```
(name: 'str', vectors: 'tuple[Vector, ...]', labels: 'tuple[str, ...]' = (), description: 'str' = '') -> None
```

Cards: C.1C FiniteBasis, C.2C FiniteBasis.

SIG-0466. 2 research cards. Kinds: `function`.

```
(nodes: 'Sequence[Scalar]') -> 'FiniteBasis'
```

Cards: C.2B lagrange_interpolation_basis, C.2B newton_interpolation_basis.

SIG-0475. 2 research cards. Kinds: `function`.

```
(numerator: 'Sequence[int]', denominator: 'Sequence[int]', prime: 'int', *, minimize: 'bool' = True, max_states:
   ↪ 'int' = 100000, budget: 'GeneratingBudget' = GeneratingBudget(maximum_degree=100000,
   ↪ maximum_coefficient_bits=4000000, maximum_matrix_dimension=4096, maximum_reachable_states=100000,
   ↪ maximum_local_combinations=1000000, maximum_modulus=2000000, maximum_serialized_bytes=67108864,
   ↪ maximum_recursion_depth=1000)) -> 'CartierAutomaton'
```

Cards: C.1F compile_rational_mask_automaton, C.3B compile_rational_mask_automaton.

SIG-0496. 2 research cards. Kinds: `function`.

```
(operator_in_source: 'Matrix', source_to_target: 'CoordinateTransform') -> 'Matrix'
```

Cards: C.1C operator_change_of_coordinates, C.2B operator_change_of_coordinates.

SIG-0504. 2 research cards. Kinds: `class`.

```
(order: 'int', axis: 'int' = -1) -> None
```

Cards: C.1F PolyphaseSystem, C.4B PolyphaseSystem.

SIG-0518. 2 research cards. Kinds: `class`.

```
(order: 'int', source_dimension: 'int', target_dimension: 'int', entries: 'tuple[SparseTensorEntry, ...]' = ()) ->
   ↪ None
```

Cards: C.1E SparseSymmetricTensor, C.5B SparseSymmetricTensor.

SIG-0519. 2 research cards. Kinds: `class`.

```
(order: 'int', source_dimension: 'int', target_dimension: 'int', source_packets: 'int', target_packets: 'int',
   ↪ family_addresses: 'int', active_addresses: 'int', pruned_addresses: 'int', input_groups: 'int',
   ↪ naive_derivative_calls: 'int', planned_derivative_calls: 'int', projected_input_vectors: 'int',
   ↪ output_projections: 'int', dense_basis_evaluations: 'int', dense_tensor_elements: 'int', certificate:
   ↪ 'Certificate') -> None
```

Cards: C.1E PacketJetComplexityEstimate, C.5B PacketJetComplexityEstimate.

SIG-0520. 2 research cards. Kinds: `class`.

```
(order: 'int', source_names: 'tuple[str, ...]', target_names: 'tuple[str, ...]', groups:
   ↪ 'tuple[PacketJetInputGroup, ...]', active_addresses: 'tuple[PacketJetAddress, ...]', pruned_addresses:
   ↪ 'tuple[PacketJetAddress, ...]', support_was_supplied: 'bool', support_complete: 'bool', certificate:
   ↪ 'Certificate') -> None
```

Cards: C.1E PacketJetExecutionPlan, C.5B PacketJetExecutionPlan.

SIG-0521. 2 research cards. Kinds: `function`.

```
(order: 'int', z: 'complex') -> 'complex'
```

Cards: C.1G denominator, C.1G transfer_tangent.

SIG-0523. 2 research cards. Kinds: `class`.

```
(outer: 'DerivativeProvider', inner: 'DerivativeProvider', *, name: 'str | None' = None) -> 'None'
```

Cards: C.1E ComposedDerivativeProvider, C.5A ComposedDerivativeProvider.

SIG-0524. 2 research cards. Kinds: `function`.

```
(outer: 'DerivativeProvider', inner: 'DerivativeProvider', point: 'Any', directions: 'Sequence[Any]', *,
   ↪ direct_provider: 'DerivativeProvider | None' = None, tolerance: 'float' = 1e-09, retain_terms: 'bool' =
   ↪ True) -> 'FaaDiBrunoResult'
```

Cards: C.1E faa_di_bruno, C.5A faa_di_bruno.

SIG-0525. 2 research cards. Kinds: `function`.

```
(outer: 'DerivativeProvider', inner: 'DerivativeProvider', point: 'Any', directions: 'Sequence[Any]', source:
   ↪ 'PacketSystem', intermediate: 'PacketSystem', target: 'PacketSystem', *, input_packets:
   ↪ 'Sequence[PacketLabel]', output_packet: 'PacketLabel', inner_support: 'Mapping[int, PacketJetSupport] |
   ↪ None' = None, outer_support: 'Mapping[int, PacketJetSupport] | None' = None, tolerance: 'float | None' =
   ↪ None, retain_terms: 'bool' = True, max_terms: 'int' = 1000000) -> 'FaaDiBrunoResult'
```

Cards: C.1E packet_faa_di_bruno, C.5A packet_faa_di_bruno.

SIG-0528. 2 research cards. Kinds: `class`.

```
(output: 'int', inputs: 'tuple[int, ...]', value: 'Any') -> None
```

Cards: C.1E SparseTensorEntry, C.5B SparseTensorEntry.

SIG-0532. 2 research cards. Kinds: `class`.

```
(outputs: 'tuple[int, int, int]' = (1, 1, 0)) -> None
```

Cards: C.1F BellParityAutomaton, C.3B BellParityAutomaton.

SIG-0542. 2 research cards. Kinds: `class`.

```
(path: 'Path', format: 'ExportFormat', bytes_written: 'int', companion_files: 'tuple[Path, ...]', certificate:
   ↪ 'Certificate') -> None
```

Cards: C.1F ExportResult, C.6 ExportResult.

SIG-0543. 2 research cards. Kinds: `class`.

```
(path: 'Path', members: 'tuple[str, ...]', formats: 'tuple[ExportFormat, ...]', manifest_path: 'Path',
   ↪ certificate: 'Certificate') -> None
```

Cards: C.1F ExportBundleResult, C.6 ExportBundleResult.

SIG-0544. 2 research cards. Kinds: `class`.

```
(path: 'str | os.PathLike[str]', *, allow_incomplete: 'bool' = False, verify_chunks: 'bool' = False) -> 'None'
```

Cards: C.1E PacketJetSnapshotStreamReader, C.5B PacketJetSnapshotStreamReader.

SIG-0545. 2 research cards. Kinds: `function`.

```
(path: 'str | os.PathLike[str]', *, allow_incomplete: 'bool' = False, verify_chunks: 'bool' = False) ->
   ↪ 'PacketJetSnapshotStreamReader'
```

Cards: C.1E open_packet_jet_snapshot_stream, C.5B open_packet_jet_snapshot_stream.

SIG-0546. 2 research cards. Kinds: `function`.

```
(path: 'str | os.PathLike[str]', *, max_coefficients: 'int' = 1000000, verify_chunks: 'bool' = True) ->
   ↪ 'PacketJetSnapshot'
```

Cards: C.1E load_packet_jet_snapshot_stream, C.5B load_packet_jet_snapshot_stream.

SIG-0547. 2 research cards. Kinds: `function`.

```
(path: 'str | os.PathLike[str]', *, max_depth: 'int' = 128) -> 'Any'
```

Cards: C.1F load_portable, C.6 load_portable.

SIG-0550. 2 research cards. Kinds: `function`.

```
(path: 'str | Path | None' = None) -> 'PublicApiManifest'
```

Cards: C.1A load_beta_api_manifest, C.1A load_public_api_manifest.

SIG-0553. 2 research cards. Kinds: `function`.

```
(path: 'str | Path') -> 'Any'
```

Cards: C.1C load_coordinate_object, C.2C load_coordinate_object.

SIG-0559. 2 research cards. Kinds: `function`.

```
(path: 'str | Path', value: 'Any', *, overwrite: 'bool' = False) -> 'Path'
```

Cards: C.1C save_coordinate_object, C.2C save_coordinate_object.

SIG-0565. 2 research cards. Kinds: `function`.

```
(plan: 'PacketJetExecutionPlan', family: 'PacketJetFamily', directions: 'Sequence[Any]', *, tolerance: 'float |
   ↪ None' = None) -> 'PacketJetBatchResult'
```

Cards: C.1E execute_packet_jet_plan, C.5B execute_packet_jet_plan.

SIG-0589. 2 research cards. Kinds: `function`.

```
(power: 'int', *, name: 'str | None' = None) -> 'WeightedMonomialBasis'
```

Cards: C.1F factorial_power_basis, C.3A factorial_power_basis.

SIG-0597. 2 research cards. Kinds: `function`.

```
(prefix: 'Sequence[Scalar]', transition: 'Matrix', output: 'Sequence[Scalar]', *, budget: 'CoordinateBudget' =
   ↪ CoordinateBudget(maximum_matrix_dimension=512, maximum_polynomial_degree=512, maximum_horizon=4096,
   ↪ maximum_exact_entries=1000000)) -> 'StateRecovery'
```

Cards: C.1C recover_initial_state, C.2C recover_initial_state.

SIG-0606. 2 research cards. Kinds: `class`.

```
(prime: 'int', exponent: 'int', factorial_power: 'int', unit_sequence: 'Callable[[int], int] | None' = None,
   ↪ max_modulus: 'int' = 2000000) -> None
```

Cards: C.1F PrimePowerKernelCompiler, C.3B PrimePowerKernelCompiler.

SIG-0609. 2 research cards. Kinds: `class`.

```
(prime: 'int', factorial_power: 'int', period_bound: 'int', outputs: 'tuple[int, ...]') -> None
```

Cards: C.1F BellResidueAutomaton, C.3B BellResidueAutomaton.

SIG-0610. 2 research cards. Kinds: `class`.

```
(prime: 'int', levels: 'tuple[tuple[LiftTreeNode, ...], ...]') -> None
```

Cards: C.1E FiniteLiftTree, C.5A FiniteLiftTree.

SIG-0621. 2 research cards. Kinds: `function`.

```
(primitive: 'PolynomialPrimitive') -> 'Matrix'
```

Cards: C.1B replay, C.1B replay_matrix.

SIG-0625. 2 research cards. Kinds: `class`.

```
(probe_count: 'int', maximum_residual: 'float', runtime: 'CuPyRuntimeStatus', certificate: 'Certificate') -> None
```

Cards: C.1E CuPyValidationResult, C.5B CuPyValidationResult.

SIG-0626. 2 research cards. Kinds: `function`.

```
(problem: 'PolynomialFixedPointProblem', tree: 'OrderedRootedTree', packets: 'PacketSystem', *, zero_support:
   ↪ 'Mapping[int, PacketJetSupport] | None' = None, tolerance: 'float | None' = None) -> 'PacketTreeResult'
```

Cards: C.1E packet_tree_reconstruction, C.5B packet_tree_reconstruction.

SIG-0627. 2 research cards. Kinds: `function`.

```
(problem_factory: 'Callable[[tuple[int, ...], int], LinearLiftProblem | PolynomialLiftProblem]', roots:
   ↪ 'Iterable[tuple[int, ...]]', *, prime: 'int', start_depth: 'int', max_depth: 'int', max_nodes: 'int' =
   ↪ 100000) -> 'FiniteLiftTree'
```

Cards: C.1E build_lift_tree, C.5A build_lift_tree.

SIG-0629. 2 research cards. Kinds: `class`.

```
(projectors: 'Callable[[float], Iterable[Any]]', derivatives: 'Callable[[float], Iterable[Any]]', *, tolerance:
   ↪ 'float' = 1e-09, reference_time: 'float' = 0.0) -> 'None'
```

Cards: C.1E MovingPacketSystem, C.5A MovingPacketSystem.

SIG-0633. 2 research cards. Kinds: `class`.

```
(provider: 'DerivativeProvider', base_point: 'Any', *, tolerance: 'float' = 1e-10) -> 'None'
```

Cards: C.1E LyapunovSchmidtReduction, C.5A LyapunovSchmidtReduction.

SIG-0634. 2 research cards. Kinds: `function`.

```
(provider: 'DerivativeProvider', point: 'Any', increment: 'Any', source: 'PacketSystem', target: 'PacketSystem',
   ↪ order: 'int', *, tolerance: 'float | None' = None) -> 'PacketTaylorResult'
```

Cards: C.1E packet_taylor, C.5B packet_taylor.

SIG-0635. 2 research cards. Kinds: `class`.

```
(provider: 'DerivativeProvider', point: 'Any', source: 'PacketSystem', target: 'PacketSystem', order: 'int') ->
   ↪ 'None'
```

Cards: C.1E PacketJetFamily, C.5B PacketJetFamily.

SIG-0636. 2 research cards. Kinds: `class`.

```
(provider: 'DerivativeProvider', point: 'Any', source: 'PacketSystem', target: 'PacketSystem', output_index:
   ↪ 'int', input_indices: 'tuple[int, ...]') -> None
```

Cards: C.1E PacketJet, C.5B PacketJet.

SIG-0643. 3 research cards. Kinds: `class`.

```
(radix: 'int', mask: 'tuple[Fraction, ...]', carry_bound: 'int', matrices: 'tuple[Matrix, ...]') -> None
```

Cards: C.1F FiniteMaskCarryPlan, C.3B FiniteMaskCarryPlan, C.3B NormalizedMaskPlan.

SIG-0650. 3 research cards. Kinds: `function`.

```
(record: 'RecurrenceCoordinateRecord') -> 'Vector'
```

Cards: C.2C synthesize_recurrence_coordinates, C.1C variation_of_constants_prefix, C.2C variation_of_constants_prefix.

SIG-0660. 2 research cards. Kinds: `class`.

```
(retained_provider: 'DerivativeProvider', hidden_provider: 'DerivativeProvider', retained_dimension: 'int',
    ↪ hidden_dimension: 'int', base_retained: 'Any', base_hidden: 'Any', *, tolerance: 'float' = 1e-10) -> 'None'
```

Cards: C.1E NonlinearSchurElimination, C.5A NonlinearSchurElimination.

SIG-0671. 2 research cards. Kinds: `function`.

```
(samples: 'Sequence[float]', *, spacing: 'float', lipschitz_bound: 'float', noise_bound: 'float', minimum_jump:
    ↪ 'float', threshold: 'float | None' = None, start: 'float' = 0.0, minimum_separation: 'float | None' = None)
    ↪ -> 'JumpRecoveryResult'
```

Cards: C.1F recover_separated_jumps, C.4B recover_separated_jumps.

SIG-0678. 2 research cards. Kinds: `class`.

```
(seed: 'Any', operations: 'Mapping[int, MultilinearOperation]', *, exactness: 'Exactness' = <Exactness.NUMERICAL:
    ↪ 'numerical'>, tolerance: 'float' = 1e-10, name: 'str' = 'polynomial fixed-point problem') -> 'None'
```

Cards: C.1E PolynomialFixedPointProblem, C.5B PolynomialFixedPointProblem.

SIG-0679. 2 research cards. Kinds: `function`.

```
(seed_norm: 'float', bilinear_norm: 'float', *, tolerance: 'float' = 1e-12) -> 'MajorantConvergence'
```

Cards: C.1E quadratic_catalan_majorant, C.5B quadratic_catalan_majorant.

SIG-0681. 2 research cards. Kinds: `function`.

```
(seed_norm: 'float', operation_norms: 'Mapping[int, float]', radius: 'float', *, tolerance: 'float' = 1e-12) ->
    ↪ 'MajorantConvergence'
```

Cards: C.1E majorant_convergence, C.5B majorant_convergence.

SIG-0692. 2 research cards. Kinds: `function`.

```
(series: 'Iterable[Scalar]', radix: 'int', degree: 'int | None' = None, *, budget: 'GeneratingBudget' =
    ↪ GeneratingBudget(maximum_degree=100000, maximum_coefficient_bits=4000000, maximum_matrix_dimension=4096,
    ↪ maximum_reachable_states=100000, maximum_local_combinations=1000000, maximum_modulus=2000000,
    ↪ maximum_serialized_bytes=67108864, maximum_recursion_depth=1000)) -> 'tuple[Fraction, ...]'
```

Cards: C.1F mahler_mask_inverse, C.3B mahler_mask_inverse.

SIG-0709. 2 research cards. Kinds: `function`.

```
(signal: 'Any', factor: 'int', *, num_taps: 'int | None' = None, window: 'WindowSpec' = ('kaiser', 8.0)) ->
    ↪ 'np.ndarray'
```

Cards: C.4A decimate, C.4A interpolate.

SIG-0718. 2 research cards. Kinds: `function`.

```
(signal: 'Any', order: 'int', *, axis: 'int' = -1, copy: 'bool' = True) -> 'PolyphaseCoordinates'
```

Cards: C.1F polyphase_analyze, C.4B polyphase_analyze.

SIG-0721. 2 research cards. Kinds: `function`.

```
(signal: 'Any', packet: 'Any', approximator: 'Callable[[np.ndarray], Any]', *, tolerance: 'float' = 1e-11) ->
    ↪ 'RetainedPacketResult'
```

Cards: C.1F retained_packet_reconstruct, C.4B retained_packet_reconstruct.

SIG-0730. 2 research cards. Kinds: `class`.

```
(signal: 'np.ndarray', depth: 'int', normalized: 'bool', _nodes: 'dict[str, np.ndarray]') -> None
```

Cards: C.1F HaarPacketTree, C.4B HaarPacketTree.

SIG-0735. 3 research cards. Kinds: `function`.

```
(size: 'int', *, budget: 'CoordinateBudget' = CoordinateBudget(maximum_matrix_dimension=512,
    ↪ maximum_polynomial_degree=512, maximum_horizon=4096, maximum_exact_entries=1000000)) ->
    ↪ 'CoordinateTransform'
```

Cards: C.1C forward_difference_transform, C.2A forward_difference_transform, C.2A newton_evaluation_transform.

SIG-0737. 2 research cards. Kinds: `function`.

```
(size: 'int', *, inverse: 'bool' = False, source: 'str' = 'sequence values', target: 'str' = 'binomial transform',
    ↪ budget: 'CoordinateBudget' = CoordinateBudget(maximum_matrix_dimension=512, maximum_polynomial_degree=512,
    ↪ maximum_horizon=4096, maximum_exact_entries=1000000)) -> 'CoordinateTransform'
```

Cards: C.1C binomial_transform, C.2A binomial_transform.

SIG-0738. 3 research cards. Kinds: `function`.

```
(size: 'int', *, orthonormal: 'bool' = True, tolerance: 'float' = 1e-12) -> 'NumericCoordinateTransform'
```

Cards: C.2C discrete_cosine_transform, C.2C haar_wavelet_transform, C.2C walsh_hadamard_transform.

SIG-0739. 2 research cards. Kinds: `function`.

```
(size: 'int', *, unitary: 'bool' = True, inverse: 'bool' = False, tolerance: 'float' = 1e-12) ->
    ↪ 'NumericCoordinateTransform'
```

Cards: C.1C discrete_fourier_transform, C.2C discrete_fourier_transform.

SIG-0741. 2 research cards. Kinds: `class`.

```
(smooth_component: 'np.ndarray', smooth_approximation: 'np.ndarray', packet: 'np.ndarray', reconstruction:
    ↪ 'np.ndarray', error: 'np.ndarray', certificate: 'Certificate', packet_error: 'np.ndarray | None' = None) ->
    ↪ None
```

Cards: C.4B RetainedPacketReconstruction, C.4B RetainedPacketResult.

SIG-0742. 4 research cards. Kinds: `class`.

```
(snapshot: 'PacketJetSnapshot', *, dtype: 'Any | None' = None) -> 'None'
```

Cards: C.1E CuPyPacketJetSnapshotExecutor, C.5B CuPyPacketJetSnapshotExecutor, C.1E CuPySnapshotExecutor, C.5B CuPySnapshotExecutor.

SIG-0743. 2 research cards. Kinds: `function`.

```
(snapshot: 'PacketJetSnapshot', path: 'str | os.PathLike[str]', *, chunk_size: 'int' = 1024, coefficient_budget:
    ↪ 'int | None' = None, resume: 'bool' = False, overwrite: 'bool' = False) -> 'PacketJetStreamWriteResult'
```

Cards: C.1E save_packet_jet_snapshot_stream, C.5B save_packet_jet_snapshot_stream.

SIG-0744. 2 research cards. Kinds: `function`.

```
(snapshot: 'PacketJetSnapshot', probes: 'Sequence[Sequence[Any]]', *, tolerance: 'float' = 1e-09) ->
    ↪ 'CuPyValidationResult'
```

Cards: C.1E validate_cupy_snapshot, C.5B validate_cupy_snapshot.

SIG-0747. 2 research cards. Kinds: `class`.

```
(source: 'FilteredVectorSpace', target: 'FilteredVectorSpace', correction_map: 'Any', defect: 'Any', depth: 'int')
    ↪ -> 'None'
```

Cards: C.1E FilteredCorrectionProblem, C.5A FilteredCorrectionProblem.

SIG-0748. 2 research cards. Kinds: `function`.

```
(source: 'FilteredVectorSpace', target: 'FilteredVectorSpace', map_function: 'Callable[[tuple[int, ...]],
    ↪ Sequence[int]]', approximations: 'Sequence[Sequence[int]]', *, start_depth: 'int' = 0) ->
    ↪ 'FiniteFilteredBranchResult'
```

Cards: C.1E verify_finite_filtered_branch, C.5A verify_finite_filtered_branch.

SIG-0750. 2 research cards. Kinds: `function`.

```
(source: 'Iterable[Any]', target: 'Iterable[Any]', *, tolerance: 'float' = 1e-10) -> 'DirectRotationResult'
```

Cards: C.1E direct_packet_rotation, C.5A direct_packet_rotation.

SIG-0751. 2 research cards. Kinds: `function`.

```
(source: 'Matrix', target: 'Matrix') -> 'IntertwinerSpace'
```

Cards: C.1C intertwiner_space, C.2C intertwiner_space.

SIG-0756. 2 research cards. Kinds: `function`.

```
(source: 'PolynomialCoordinateSpace', lower: 'int | Fraction', upper: 'int | Fraction', density: 'int | Fraction' =
    ↪ 1) -> 'PolynomialPrimitive'
```

Cards: C.1B boundary_functional, C.1B interval_integration.

SIG-0762. 2 research cards. Kinds: `function`.

```
(source: 'PolynomialFamily', target: 'PolynomialFamily', degree: 'int', *, budget: 'CoordinateBudget' =
    ↪ CoordinateBudget(maximum_matrix_dimension=512, maximum_polynomial_degree=512, maximum_horizon=4096,
    ↪ maximum_exact_entries=1000000)) -> 'CoordinateTransform'
```

Cards: C.1C connection_transform, C.2B connection_transform.

SIG-0768. 2 research cards. Kinds: `function`.

```
(source: 'str | Path', output_dir: 'str | Path', *, options: 'LatexCompileOptions | None' = None, source_name:
    ↪ 'str' = 'document.tex') -> 'LatexCompilation'
```

Cards: C.1F compile_latex, C.6 compile_latex.

SIG-0770. 2 research cards. Kinds: `class`.

```
(source: 'str', target: 'str', matrix: 'Any', name: 'str' = '') -> None
```

Cards: C.1E TransportEdge, C.5A TransportEdge.

SIG-0771. 2 research cards. Kinds: `class`.

```
(source: 'str', target: 'str', matrix: 'Matrix', offset: 'Vector', theorem: 'str' = 'CC.F1', description: 'str' =
    ↪ '') -> None
```

Cards: C.1C AffineCoordinateTransform, C.2C AffineCoordinateTransform.

SIG-0772. 3 research cards. Kinds: `class`.

```
(source: 'str', target: 'str', matrix: 'Matrix', theorem: 'str', description: 'str' = '') -> None
```

Cards: C.1C CoordinateTransform, C.2C CoordinateTransform, C.2C ExactCoordinateTransform.

SIG-0773. 2 research cards. Kinds: `class`.

```
(source: 'str', target: 'str', matrix: 'ModularMatrix', prime: 'int', theorem: 'str' = 'CC.M1', description: 'str'
    ↪ = '') -> None
```

Cards: C.1C FiniteFieldCoordinateTransform, C.2A FiniteFieldCoordinateTransform.

SIG-0774. 2 research cards. Kinds: `class`.

```
(source: 'str', target: 'str', matrix: 'np.ndarray', theorem: 'str' = 'CC.NM1', description: 'str' = '', tolerance:
    ↪ 'float' = 1e-12) -> None
```

Cards: C.1C NumericCoordinateTransform, C.2C NumericCoordinateTransform.

SIG-0775. 2 research cards. Kinds: `function`.

```
(source: 'WeightedMonomialBasis', target: 'WeightedMonomialBasis', degree: 'int', *, budget: 'CoordinateBudget' =
    ↪ CoordinateBudget(maximum_matrix_dimension=512, maximum_polynomial_degree=512, maximum_horizon=4096,
    ↪ maximum_exact_entries=1000000)) -> 'CoordinateTransform'
```

Cards: C.1C normalization_change_transform, C.2A normalization_change_transform.

SIG-0776. 2 research cards. Kinds: `function`.

```
(source_coefficients: 'RecurrenceLike', target_coefficients: 'RecurrenceLike', length: 'int', *, source_name:
    ↪ 'str' = 'source recurrence', target_name: 'str' = 'target recurrence', budget: 'CoordinateBudget' =
    ↪ CoordinateBudget(maximum_matrix_dimension=512, maximum_polynomial_degree=512, maximum_horizon=4096,
    ↪ maximum_exact_entries=1000000)) -> 'CoordinateTransform'
```

Cards: C.1C coordinate_change_between_recurrences, C.2C coordinate_change_between_recurrences.

SIG-0779. 2 research cards. Kinds: `class`.

```
(source_dimension: 'int', target_dimension: 'int', basis: 'tuple[Matrix, ...]') -> None
```

Cards: C.1C IntertwinerSpace, C.2C IntertwinerSpace.

SIG-0790. 2 research cards. Kinds: `class`.

```
(state: 'StreamState', producer_version: 'str', address: 'PacketJetAddress', source_dimension: 'int',
   ↪ target_dimension: 'int', provider_name: 'str', exactness: 'Exactness', point: 'tuple[Any, ...]', tolerance:
   ↪ 'float', dense_coefficient_count: 'int', capture_mode: 'CaptureMode', source_digest: 'str', metadata_sha256:
   ↪ 'str', chunk_size: 'int', evaluations_completed: 'int', coefficients_completed: 'int',
   ↪ maximum_omitted_basis_norm: 'float', chunks: 'tuple[PacketJetStreamChunkRecord, ...]', capture_certificate:
   ↪ 'Certificate | None', stream_sha256: 'str') -> None
```

Cards: C.1E PacketJetStreamManifest, C.5B PacketJetStreamManifest.

SIG-0797. 2 research cards. Kinds: `class`.

```
(system: "'AnalysisSynthesisSystem'", groups: 'tuple[tuple[int, ...], ...]', blocks: 'tuple[tuple[np.ndarray,
   ↪ ...], ...]', operator: 'np.ndarray') -> None
```

Cards: C.1C AnalysisSynthesisBlocks, C.2C AnalysisSynthesisBlocks.

SIG-0799. 2 research cards. Kinds: `class`.

```
(system: 'AnalysisSynthesisSystem') -> None
```

Cards: C.1C AnalysisSynthesisGauge, C.2C AnalysisSynthesisGauge.

SIG-0801. 2 research cards. Kinds: `class`.

```
(system: 'MovingPacketSystem') -> 'None'
```

Cards: C.1E KatoTransport, C.5A KatoTransport.

SIG-0803. 2 research cards. Kinds: `class`.

```
(tag: 'str', from_version: 'int', to_version: 'int', migrator: 'Migrator') -> None
```

Cards: C.1F PortableMigration, C.6 PortableMigration.

SIG-0804. 2 research cards. Kinds: `class`.

```
(tag: 'str', version: 'int', type_: 'type[Any]', encoder: 'Encoder', decoder: 'Decoder') -> None
```

Cards: C.1F PortableCodec, C.6 PortableCodec.

SIG-0812. 2 research cards. Kinds: `class`.

```
(tensors: 'Mapping[int, Any]', *, source_dimension: 'int | None' = None, target_dimension: 'int | None' = None,
   ↪ exactness: 'Exactness' = <Exactness.NUMERICAL: 'numerical'>, tolerance: 'float' = 1e-10, name: 'str' =
   ↪ 'polynomial derivative provider') -> 'None'
```

Cards: C.1E PolynomialDerivativeProvider, C.5A PolynomialDerivativeProvider.

SIG-0813. 2 research cards. Kinds: `class`.

```
(tensors: 'Mapping[int, Any]', *, source_dimension: 'int | None' = None, target_dimension: 'int | None' = None,
   ↪ tolerance: 'float' = 1e-10, name: 'str' = 'CuPy polynomial derivative provider') -> 'None'
```

Cards: C.1E CuPyPolynomialDerivativeProvider, C.5B CuPyPolynomialDerivativeProvider.

SIG-0814. 2 research cards. Kinds: `class`.

```
(tensors: 'Mapping[int, SparseSymmetricTensor | Mapping[tuple[int, ...], Any] | Any]', *, source_dimension: 'int |
   ↪ None' = None, target_dimension: 'int | None' = None, exactness: 'Exactness' = <Exactness.NUMERICAL:
   ↪ 'numerical'>, tolerance: 'float' = 1e-10, name: 'str' = 'sparse polynomial derivative provider') -> 'None'
```

Cards: C.1E SparsePolynomialDerivativeProvider, C.5B SparsePolynomialDerivativeProvider.

SIG-0816. 2 research cards. Kinds: `function`.

```
(text: 'str | bytes', *, max_depth: 'int' = 128) -> 'Any'
```

Cards: C.1F loads_portable, C.6 loads_portable.

SIG-0820. 2 research cards. Kinds: `class`.

```
(title: 'str' = 'Komorebi Math Export', author: 'str | None' = None, date: 'str | None' = None, abstract: 'str |
   ↪ None' = None, keywords: 'tuple[str, ...]' = ()) -> None
```

Cards: C.1F DocumentMetadata, C.6 DocumentMetadata.

SIG-0821. 2 research cards. Kinds: `class`.

```
(title: 'str' = 'Komorebi Math Export', author: 'str | None' = None, date: 'str | None' = None, abstract: 'str |
   ↪ None' = None, overwrite: 'bool' = False, create_parents: 'bool' = True, preserve_types: 'bool' = True,
   ↪ json_indent: 'int' = 2, include_certificate: 'bool' = True, keep_tex: 'bool' = True, html:
   ↪ 'HtmlRenderOptions' = <factory>, latex: 'LatexRenderOptions' = <factory>, compiler: 'LatexCompileOptions' =
   ↪ <factory>) -> None
```

Cards: C.1F ExportOptions, C.6 ExportOptions.

SIG-0826. 2 research cards. Kinds: `function`.

```
(transform: 'CoordinateTransform', variance: 'Sequence[Variance]') -> 'CoordinateTransform'
```

Cards: C.1C tensor_coordinate_transform, C.2A tensor_coordinate_transform.

SIG-0831. 2 research cards. Kinds: `class`.

```
(transition: 'Matrix', initial: 'Vector', output: 'Vector', name: 'str' = 'linear realization') -> None
```

Cards: C.1C LinearRealization, C.2C LinearRealization.

SIG-0832. 2 research cards. Kinds: `function`.

```
(transition: 'Matrix', output: 'Sequence[Scalar]', horizon: 'int', *, budget: 'CoordinateBudget' =
   ↪ CoordinateBudget(maximum_matrix_dimension=512, maximum_polynomial_degree=512, maximum_horizon=4096,
   ↪ maximum_exact_entries=1000000)) -> 'Matrix'
```

Cards: C.1C observability_matrix, C.2C observability_matrix.

SIG-0843. 2 research cards. Kinds: `function`.

```
(type_: 'type[T]', *, tag: 'str', version: 'int', encoder: 'Callable[[T], Any]', decoder: 'Callable[[Any], T]',
   ↪ replace: 'bool' = False) -> 'None'
```

Cards: C.1F register_portable_codec, C.6 register_portable_codec.

SIG-0853. 2 research cards. Kinds: `function`.

```
(value: 'Any', *, indent: 'int | None' = 2, max_depth: 'int' = 128) -> 'str'
```

Cards: C.1F dumps_portable, C.6 dumps_portable.

SIG-0854. 2 research cards. Kinds: `function`.

```
(value: 'Any', *, max_depth: 'int' = 128) -> 'Any'
```

Cards: C.6 from_portable_data, C.6 to_portable_data.

SIG-0855. 2 research cards. Kinds: `function`.

```
(value: 'Any', *, max_depth: 'int' = 128) -> 'str'
```

Cards: C.1F portable_digest, C.6 portable_digest.

SIG-0856. 2 research cards. Kinds: `function`.

```
(value: 'Any', *, max_depth: 'int' = 64, _depth: 'int' = 0) -> 'Any'
```

Cards: C.1F from_serializable, C.6 from_serializable.

SIG-0857. 2 research cards. Kinds: `function`.

```
(value: 'Any', *, preserve_types: 'bool' = True, max_depth: 'int' = 64, _depth: 'int' = 0) -> 'Any'
```

Cards: C.1F to_serializable, C.6 to_serializable.

SIG-0858. 2 research cards. Kinds: `function`.

```
(value: 'Any', *, title: 'str' = 'Komorebi Math Export', author: 'str | None' = None, date: 'str | None' = None,
   ↪ abstract: 'str | None' = None, include_certificate: 'bool' = True) -> 'MathDocument'
```

Cards: C.1F document_from_data, C.6 document_from_data.

SIG-0862. 2 research cards. Kinds: `function`.

```
(value: 'Any', directory: 'str | Path', *, base_name: 'str', formats: 'Sequence[ExportFormat | str]', options:
   ↪ 'ExportOptions | None' = None) -> 'tuple[ExportResult, ...]'
```

Cards: C.1F export_many, C.6 export_many.

SIG-0864. 2 research cards. Kinds: `function`.

```
(value: 'Any', path: 'str | os.PathLike[str]', *, overwrite: 'bool' = False, indent: 'int | None' = 2, max_depth:
   ↪ 'int' = 128) -> 'Path'
```

Cards: C.1F save_portable, C.6 save_portable.

SIG-0865. 2 research cards. Kinds: `function`.

```
(value: 'Any', path: 'str | Path', *, base_name: 'str' = 'komorebi_export', formats: 'Sequence[ExportFormat | str]'
   ↪ = (<ExportFormat.JSON: 'json'>, <ExportFormat.MARKDOWN: 'markdown'>, <ExportFormat.LATEX: 'latex'>),
   ↪ options: 'ExportOptions | None' = None) -> 'ExportBundleResult'
```

Cards: C.1F export_bundle, C.6 export_bundle.

SIG-0866. 2 research cards. Kinds: `function`.

```
(value: 'Any', path: 'str | Path', *, format: 'ExportFormat | str | None' = None, options: 'ExportOptions | None' =
    ↪ None) -> 'ExportResult'
```

Cards: C.1F export_data, C.6 export_data.

SIG-0872. 2 research cards. Kinds: `function`.

```
(value: 'int', prime: 'int') -> 'int'
```

Cards: C.3B integer_valuation, C.3B multiplicative_order_mod_prime.

SIG-0875. 2 research cards. Kinds: `class`.

```
(value: 'np.ndarray', coefficient_start: 'int', coefficient_stop: 'int', total_coefficients: 'int',
    ↪ complete_snapshot: 'bool', certificate: 'Certificate') -> None
```

Cards: C.1E StreamedPacketJetEvaluation, C.5B StreamedPacketJetEvaluation.

SIG-0880. 2 research cards. Kinds: `class`.

```
(value: 'T', certificate: 'Certificate') -> None
```

Cards: C.1A ComputationResult, C.1A ConditionalResult.

SIG-0885. 2 research cards. Kinds: `function`.

```
(values: 'Iterable[int | Fraction]') -> 'FiniteMonoidSeries[int]'
```

Cards: C.3A additive_series, C.3A multiplicative_series.

SIG-0907. 2 research cards. Kinds: `function`.

```
(values: 'Sequence[Scalar]', coefficients: 'RecurrenceLike', *, name: 'str' = 'recurrence') ->
    ↪ 'RecurrenceCoordinateRecord'
```

Cards: C.1C analyze_recurrence_coordinates, C.2C analyze_recurrence_coordinates.

SIG-0908. 2 research cards. Kinds: `function`.

```
(values: 'Sequence[Scalar]', left: 'Sequence[Scalar]', right: 'Sequence[Scalar]') -> 'BezoutDefectIdentity'
```

Cards: C.1C bezout_defect_identity, C.2C bezout_defect_identity.

SIG-0913. 2 research cards. Kinds: `function`.

```
(verified_prefix_depth: 'int', *, separated: 'bool', complete: 'bool', continuous: 'bool',
    ↪ compatible_infinite_branch: 'bool') -> 'ConditionalResult[ProjectiveLimitRecord]'
```

Cards: C.1E conditional_projective_limit_completion, C.5A conditional_projective_limit_completion.

SIG-0916. 2 research cards. Kinds: `class`.

```
(vertices: 'Iterable[str]', edges: 'Iterable[TransportEdge]', *, backend: 'str | LinearAlgebraBackend | None' =
    ↪ None, tolerance: 'float' = 1e-10) -> 'None'
```

Cards: C.1E TransportGraph, C.5A TransportGraph.

C.9.3 Non-null signatures used by one card

SIG-0001. 1 research card. Kinds: `class`.

```
()
```

Cards: C.1B Expression.

SIG-0005. 1 research card. Kinds: `function`.

```
() -> 'dict[type[Any], ExportAdapter]'
```

Cards: C.6 export_adapters.

SIG-0006. 1 research card. Kinds: `function`.

```
() -> 'DyadicSignDiagnostic'
```

Cards: C.1G dyadic_sign_diagnostic.

SIG-0007. 1 research card. Kinds: `function`.

```
() -> 'LinearRecurrence'
```

Cards: C.1G fibonacci_recurrence.

SIG-0009. 1 research card. Kinds: `function`.

```
() -> 'PeriodicLiftingScheme'
```

Cards: C.4B periodic_five_three_lifting.

SIG-0014. 1 research card. Kinds: `function`.

```
() -> 'Settings'
```

Cards: C.1A get_settings.

SIG-0015. 1 research card. Kinds: `function`.

```
() -> 'tuple[FeatureSpec, ...]'
```

Cards: C.1G features.

SIG-0022. 1 research card. Kinds: `function`.

```
(**changes: 'object') -> 'Iterator[Settings]'
```

Cards: C.1A settings_context.

SIG-0023. 1 research card. Kinds: `function`.

```
(*, backend: 'str | None' = None, tolerance: 'float | None' = None, validate_on_construction: 'bool | None' = None)
   ↪  -> 'Settings'
```

Cards: C.1A configure.

SIG-0027. 1 research card. Kinds: `function`.

```
(*, package_version: 'str | None' = None) -> 'PublicApiManifest'
```

Cards: C.1A create_beta_api_manifest.

SIG-0028. 1 research card. Kinds: `class`.

```
(*, prime: 'int', labels: 'Iterable[Iterable[int]]', operators: 'Iterable[Any]', tolerance: 'float' = 1e-08) ->
   ↪ 'None'
```

Cards: C.1G ProjectiveIntertwinerEnvelope.

SIG-0029. 1 research card. Kinds: `function`.

```
(*, prime: 'int', power: 'int', count: 'int' = 300) -> 'DigitSumCongruenceAudit'
```

Cards: C.3B audit_generalized_bell_digit_sum.

SIG-0030. 1 research card. Kinds: `function`.

```
(*, prime: 'int', power: 'int', digit_weights: 'Sequence[int]', count: 'int' = 300) ->
   ↪ 'WeightedDigitSumCongruenceAudit'
```

Cards: C.3B audit_weighted_generalized_bell_digit_sum.

SIG-0031. 1 research card. Kinds: `function`.

```
(*, prime: 'int', power: 'int', left_digits: 'Iterable[int]', right_digits: 'Iterable[int]', max_digits: 'int' =
   ↪ 3) -> 'DigitAlgebraAudit'
```

Cards: C.3B audit_factorial_power_digit_algebra.

SIG-0032. 1 research card. Kinds: `function`.

```
(*, q: 'int', prime: 'int', high_limit: 'int' = 10) -> 'QLucasAudit'
```

Cards: C.3B audit_q_lucas.

SIG-0033. 1 research card. Kinds: `function`.

```
(*, since: 'str', replacement: 'str | None' = None, remove_in: 'str | None' = None, reason: 'str | None' = None) ->
   ↪ 'Callable[[Callable[P, R]], Callable[P, R]]'
```

Cards: C.1A deprecated.

SIG-0036. 1 research card. Kinds: `function`.

```
(*factors: 'Measure', name: 'str' = 'product measure') -> 'ProductMeasure'
```

Cards: C.1D product_measure.

SIG-0038. 1 research card. Kinds: `function`.

```
(*terms: 'Expression') -> 'Expression'
```

Cards: C.1B add.

SIG-0040. 1 research card. Kinds: `function`.

```
(*values: 'Exactness') -> 'Exactness'
```

Cards: C.1B weakest_exactness.

SIG-0041. 1 research card. Kinds: `function`.

```
(a: 'Any', b: 'Any', order: 'int') -> 'ComputationResult[tuple[np.ndarray, ...]]'
```

Cards: C.4B packet_product.

SIG-0042. 1 research card. Kinds: `function`.

```
(a: 'int', modulus_a: 'int', b: 'int', modulus_b: 'int') -> 'CRTAddress'
```

Cards: C.1G crt_pair.

SIG-0043. 1 research card. Kinds: `class`.

```
(absolute_bound: 'Fraction | float | None', guarantee: 'ErrorGuarantee', sources: 'tuple[str, ...]' = ()) -> None
```

Cards: C.1D IntegrationError.

SIG-0044. 1 research card. Kinds: `function`.

```
(address: 'int', radices: 'Iterable[int]') -> 'tuple[int, ...]'
```

Cards: C.4B split_mixed_radix_address.

SIG-0046. 1 research card. Kinds: `class`.

```
(address: 'PacketJetAddress', value: 'Any') -> None
```

Cards: C.5B PacketJetEvaluation.

SIG-0047. 1 research card. Kinds: `class`.

```
(adjacency: 'Sequence[Sequence[bool | int]]', *, names: 'Sequence[str] | None' = None) -> 'None'
```

Cards: C.1D TransferGraph.

SIG-0048. 1 research card. Kinds: `function`.

```
(alpha: 'int | Fraction', *, name: 'str | None' = None) -> 'WeightedMonomialBasis'
```

Cards: C.3A pochhammer_basis.

SIG-0049. 1 research card. Kinds: `function`.

```
(amplitude: 'complex', location: 'float', estimated_location: 'float', *, p: 'float' = 2.0) -> 'float'
```

Cards: C.4B step_location_lp_error.

SIG-0050. 1 research card. Kinds: `function`.

```
(amplitude: 'float', relative_location: 'float', interval_length: 'float') -> 'float'
```

Cards: C.4B step_haar_coefficient.

SIG-0051. 1 research card. Kinds: `function`.

```
(amplitude: 'float', support_length: 'float', relative_location: 'float') -> 'float'
```

Cards: C.4B haar_step_coefficient.

SIG-0055. 1 research card. Kinds: `class`.

```
(analysis_matrix: 'Any', synthesis_matrix: 'Any') -> 'None'
```

Cards: C.4B FiniteFilterBank.

SIG-0056. 1 research card. Kinds: `function`.

```
(apply: 'Apply', right_hand_side: 'Any', *, precondition: 'Apply | None' = None, initial: 'Any | None' = None,
   ↪ relative_tolerance: 'float' = 1e-10, absolute_tolerance: 'float' = 0.0, max_iterations: 'int | None' = None,
   ↪ curvature_tolerance: 'float' = 1e-14) -> 'ConjugateGradientResult'
```

Cards: C.1D conjugate_gradient.

SIG-0057. 1 research card. Kinds: `class`.

```
(approximation: 'Any', observed: 'Any', truncation_residual: 'float', order_certificates: 'tuple[Certificate,
   ↪ ...]', certificate: 'Certificate') -> None
```

Cards: C.5B PacketTaylorResult.

SIG-0058. 1 research card. Kinds: `class`.

```
(approximation: 'np.ndarray', details: 'tuple[np.ndarray, ...]', normalized: 'bool', original_length: 'int') ->
   ↪ None
```

Cards: C.4B HaarDecomposition.

SIG-0059. 1 research card. Kinds: `class`.

```
(approximations: 'tuple[tuple[int, ...], ...]', limit: 'tuple[int, ...]', exact_solution: 'bool', certificate:
   ↪ 'Certificate') -> None
```

Cards: C.5A FiniteFilteredBranchResult.

SIG-0060. 1 research card. Kinds: `class`.

```
(arity: 'int', tested_input_labels: 'int', forbidden_residual: 'float', certificate: 'Certificate') -> None
```

Cards: C.5A GradingCertificate.

SIG-0061. 1 research card. Kinds: `class`.

```
(arrows: 'tuple[str, ...]', source: 'str', target: 'str') -> None
```

Cards: C.1D OperatorWord.

SIG-0062. 1 research card. Kinds: `function`.

```
(automaton: 'CartierAutomaton') -> 'CartierAutomaton'
```

Cards: C.3B minimize_cartier_automaton.

SIG-0063. 1 research card. Kinds: `class`.

```
(backend: 'str' = 'numpy', tolerance: 'float' = 1e-10, validate_on_construction: 'bool' = True) -> None
```

Cards: C.1A Settings.

SIG-0064. 1 research card. Kinds: `function`.

```
(bank: 'FiniteFilterBank') -> 'float'
```

Cards: C.4B reconstruction_error.

SIG-0065. 1 research card. Kinds: `class`.

```
(base: 'int', transitions: 'Iterable[Iterable[int]]', outputs: 'Iterable[Number]', *, initial_state: 'int' = 0,
   ↪ name: 'str' = 'automatic sequence') -> 'None'
```

Cards: C.1G AutomaticSequence.

SIG-0066. 1 research card. Kinds: `class`.

```
(base: 'Measure', mapping: 'ExactAffineMap | Callable[[Any], Any]', *, target_dimension: 'int | None' = None, name:
   ↪ 'str' = 'pushforward measure') -> 'None'
```

Cards: C.1D PushforwardMeasure.

SIG-0067. 1 research card. Kinds: `function`.

```
(base: 'Measure', mapping: 'ExactAffineMap | Callable[[Any], Any]', *, target_dimension: 'int | None' = None, name:
   ↪ 'str' = 'pushforward measure') -> 'PushforwardMeasure'
```

Cards: C.1D pushforward_measure.

SIG-0068. 1 research card. Kinds: `class`.

```
(basis: 'str', modulus: 'int', degree: 'int', signed: 'bool', one_step_failures: 'int', full_cycle_failures:
   ↪ 'int', root_filter_residual: 'float') -> None
```

Cards: C.3A SectionCycleAudit.

SIG-0069. 1 research card. Kinds: `class`.

```
(basis: 'tuple[Any, ...]', dimension: 'int', certificate: 'Certificate') -> None
```

Cards: C.1D AlgebraBasisResult.

SIG-0070. 1 research card. Kinds: `function`.

```
(basis: 'WeightedMonomialBasis', *, modulus: 'int', degree: 'int', signed: 'bool' = False, z: 'complex' =
   ↪ (0.37+0.11j)) -> 'SectionCycleAudit'
```

Cards: C.3A audit_section_cycle.

SIG-0071. 1 research card. Kinds: `function`.

```
(basis: 'WeightedMonomialBasis', *, prime: 'int', high_limit: 'int' = 12) -> 'LucasBasisAudit'
```

Cards: C.3B audit_generalized_lucas.

SIG-0073. 1 research card. Kinds: `function`.

```
(basis: 'WeightedMonomialBasis', degree: 'int') -> 'FormalSeries'
```

Cards: C.3A custom_exponential.

SIG-0075. 1 research card. Kinds: `function`.

```
(basis: 'WeightedMonomialBasis', modulus: 'int', residue: 'int', degree: 'int', *, signed: 'bool' = False) ->
    ↪ 'FormalSeries'
```

Cards: C.3A custom_section.

SIG-0076. 1 research card. Kinds: `function`.

```
(basis: 'WeightedMonomialBasis', modulus: 'int', residue: 'int', degree: 'int', z: 'complex') -> 'complex'
```

Cards: C.3A root_filter_value.

SIG-0077. 1 research card. Kinds: `function`.

```
(binding: 'Symbol', value: 'Expression', body: 'Expression') -> 'Let'
```

Cards: C.1B let.

SIG-0078. 1 research card. Kinds: `function`.

```
(block_dimensions: 'Sequence[int]', multiplicities: 'Sequence[int]') -> 'int'
```

Cards: C.1D semisimple_seed_rank.

SIG-0079. 1 research card. Kinds: `class`.

```
(blocks: 'tuple[StructuredOperator | Array, ...]', names: 'tuple[str, ...] | None' = None) -> None
```

Cards: C.1D BlockDiagonalOperator.

SIG-0081. 1 research card. Kinds: `class`.

```
(break_number: 'int', parent_order: 'int', child_order: 'int', orbit_size: 'int', multiplicity_dimension: 'int',
    ↪ packet: 'ProjectiveMultiplicityPacket | None' = None, label: 'str' = '') -> None
```

Cards: C.1G RamificationStage.

SIG-0082. 1 research card. Kinds: `function`.

```
(candidate: 'Iterable[Scalar]', radix: 'int', degree: 'int | None' = None) -> 'tuple[Fraction, ...]'
```

Cards: C.3B polynomial_mask_coboundary.

SIG-0083. 1 research card. Kinds: `class`.

```
(cell: 'int', left_position: 'float', right_position: 'float', location: 'float', amplitude: 'float') -> None
```

Cards: C.4B JumpDetection.

SIG-0084. 1 research card. Kinds: `class`.

```
(cells: 'tuple[int, ...]', locations: 'np.ndarray', amplitudes: 'np.ndarray', contrasts: 'np.ndarray', threshold:
    ↪ 'float', background_bound: 'float', location_error_bound: 'float', amplitude_error_bound: 'float',
    ↪ certificate: 'Certificate') -> None
```

Cards: C.4B VectorJumpRecoveryResult.

SIG-0085. 1 research card. Kinds: `function`.

```
(certificate: 'Certificate', *, title: 'str' = 'Certificate') -> 'MathDocument'
```

Cards: C.6 certificate_document.

SIG-0086. 1 research card. Kinds: `function`.

```
(certificates: 'Iterable[Certificate]', *, operation: 'str | None' = None, theorem_id: 'str | None' = None,
   ↪ assumptions_checked: 'Iterable[str]' = (), assumptions_unchecked: 'Iterable[str]' = (), diagnostics:
   ↪ 'Mapping[str, Any] | None' = None) -> 'Certificate'
```

Cards: C.1B compose_certificates.

SIG-0087. 1 research card. Kinds: `class`.

```
(channels: 'dict[tuple[int, ...], np.ndarray]', strides: 'tuple[int, ...]', original_shape: 'tuple[int, ...]') ->
   ↪ None
```

Cards: C.4B LatticePolyphaseCoordinates.

SIG-0088. 1 research card. Kinds: `class`.

```
(channels: 'tuple[np.ndarray, ...]', order: 'int', axis: 'int', original_shape: 'tuple[int, ...]') -> None
```

Cards: C.4B PolyphaseCoordinates.

SIG-0090. 1 research card. Kinds: `function`.

```
(clean_signal: 'Any', noisy_signal: 'Any', denoised_signal: 'Any', *, tolerance: 'float' = 0.0) -> 'Certificate'
```

Cards: C.4B denoising_risk_certificate.

SIG-0091. 1 research card. Kinds: `class`.

```
(coarse: 'PacketSystem', fine: 'PacketSystem', parent_of_fine: 'tuple[int, ...]') -> None
```

Cards: C.1G PacketRefinement.

SIG-0092. 1 research card. Kinds: `function`.

```
(coarse_order: 'int', fine_order: 'int', residue: 'int') -> 'tuple[int, ...]'
```

Cards: C.4B refinement_children.

SIG-0093. 1 research card. Kinds: `class`.

```
(coarse_steps: 'int', fine_steps: 'int', coarse: 'np.ndarray', fine: 'np.ndarray', difference: 'float',
   ↪ endpoint_intertwining: 'float', certificate: 'Certificate') -> None
```

Cards: C.5A TransportRefinementResult.

SIG-0094. 1 research card. Kinds: `class`.

```
(coarsen: 'Any', fine_operator: 'Any', coarse_operator: 'Any', backend: 'str | LinearAlgebraBackend | None' =
   ↪ None) -> None
```

Cards: C.1G LinearRefinementDefect.

SIG-0095. 1 research card. Kinds: `class`.

```
(code: 'str', language: 'str' = 'text', caption: 'str | None' = None) -> None
```

Cards: C.6 CodeBlock.

SIG-0097. 1 research card. Kinds: `function`.

```
(coefficients: 'Any', sigma: 'float') -> 'float'
```

Cards: C.4B sure_threshold.

SIG-0098. 1 research card. Kinds: `function`.

```
(coefficients: 'Any', threshold: 'float', *, mode: 'ThresholdMode' = 'soft') -> 'np.ndarray'
```

Cards: C.4B threshold_coefficients.

SIG-0099. 1 research card. Kinds: `class`.

```
(coefficients: 'Iterable[Number]') -> 'None'
```

Cards: C.1G MahlerSeries.

SIG-0100. 1 research card. Kinds: `class`.

```
(coefficients: 'Iterable[Number]', initial: 'Iterable[Number]', *, name: 'str | None' = None) -> 'None'
```

Cards: C.1G LinearRecurrence.

SIG-0101. 1 research card. Kinds: `class`.

```
(coefficients: 'Iterable[Number]', modulus: 'int | None' = None) -> 'None'
```

Cards: C.1G HurwitzSeries.

SIG-0102. 1 research card. Kinds: `function`.

```
(coefficients: 'Iterable[Scalar]', *, name: 'str' = 'custom formal coordinate', source: 'str' = 'source formal
    ↪ coordinate', target: 'str' = 'target formal coordinate') -> 'FormalCoordinateChange'
```

Cards: C.2C formal_coordinate_from_iterable.

SIG-0104. 1 research card. Kinds: `class`.

```
(coefficients: 'Mapping[Index, Fraction]', combine: 'Callable[[Index, Index], Index]', identity: 'Index', name:
    ↪ 'str' = 'monoid series') -> None
```

Cards: C.3A FiniteMonoidSeries.

SIG-0105. 1 research card. Kinds: `function`.

```
(coefficients: 'np.ndarray', _: 'str' = '') -> 'float'
```

Cards: C.4B entropy_cost.

SIG-0106. 1 research card. Kinds: `class`.

```
(coefficients: 'np.ndarray', frequencies: 'np.ndarray', times: 'np.ndarray', sample_rate: 'float', frame_length:
    ↪ 'int', hop_length: 'int', fft_length: 'int', analysis_window: 'np.ndarray', original_length: 'int',
    ↪ pad_left: 'int', pad_right: 'int', onesided: 'bool', boundary: 'BoundaryMode', detrend: 'DetrendMode') ->
    ↪ None
```

Cards: C.4A STFTResult.

SIG-0107. 1 research card. Kinds: `class`.

```
(coefficients: 'np.ndarray', frequencies: 'np.ndarray', times: 'np.ndarray', starts: 'np.ndarray', sample_rate:
   ↪ 'float', frame_length: 'int', hop_length: 'int', fft_length: 'int', onesided: 'bool') -> None
```

Cards: C.4A StreamingSTFTBatch.

SIG-0108. 1 research card. Kinds: `function`.

```
(coefficients: 'np.ndarray', path: 'str' = '') -> 'float'
```

Cards: C.4B shannon_entropy_cost.

SIG-0110. 1 research card. Kinds: `function`.

```
(coefficients: 'RecurrenceLike') -> 'Vector'
```

Cards: C.2C recurrence_polynomial.

SIG-0111. 1 research card. Kinds: `function`.

```
(coefficients: 'RecurrenceLike', count: 'int') -> 'Vector'
```

Cards: C.2C defect_green_weights.

SIG-0113. 1 research card. Kinds: `function`.

```
(coefficients: 'Sequence[Any]', order: 'int', residue: 'int') -> 'np.ndarray'
```

Cards: C.1G cyclic_project_coefficients.

SIG-0114. 1 research card. Kinds: `function`.

```
(coefficients: 'Sequence[complex]', order: 'int') -> 'tuple[np.ndarray, ...]'
```

Cards: C.4B coefficient_packets.

SIG-0115. 1 research card. Kinds: `function`.

```
(coefficients: 'Sequence[Scalar]', *, initial_values: 'Sequence[Scalar]' = (), name: 'str' = 'custom recurrence',
   ↪ description: 'str' = '') -> 'RecurrenceModel'
```

Cards: C.2C custom_recurrence_model.

SIG-0116. 1 research card. Kinds: `function`.

```
(coefficients: 'Sequence[Scalar]', polynomial: 'Iterable[Scalar]') -> 'Vector'
```

Cards: C.2A apply_lowering_polynomial.

SIG-0117. 1 research card. Kinds: `function`.

```
(coefficients: 'Sequence[Scalar]', source: 'WeightedMonomialBasis', target: 'WeightedMonomialBasis', *, budget:
   ↪ 'CoordinateBudget' = CoordinateBudget(maximum_matrix_dimension=512, maximum_polynomial_degree=512,
   ↪ maximum_horizon=4096, maximum_exact_entries=1000000)) -> 'Vector'
```

Cards: C.2A convert_normalized_coefficients.

SIG-0119. 1 research card. Kinds: `function`.

```
(columns: 'Sequence[Sequence[Scalar]]') -> 'Matrix'
```

Cards: C.2A matrix_from_columns.

SIG-0120. 1 research card. Kinds: `class`.

```
(common_spine: 'bool', semisimple: 'bool', one_member_automorphic: 'bool', local_global_compatibility: 'bool') ->
    ↪ None
```

Cards: C.1G CompatiblePropagationReport.

SIG-0121. 1 research card. Kinds: `class`.

```
(compared_terms: 'int', failures: 'tuple[tuple[int, int, int], ...]') -> None
```

Cards: C.3B AutomatonCertificate.

SIG-0123. 1 research card. Kinds: `function`.

```
(components: 'Sequence[Scalar]', transform: 'CoordinateTransform', variance: 'Sequence[Variance]') -> 'Vector'
```

Cards: C.2A transport_tensor_components.

SIG-0124. 1 research card. Kinds: `class`.

```
(consistent: 'bool', rank: 'int', solution: 'Vector | None', nullspace_basis: 'tuple[Vector, ...]', pivot_columns:
    ↪ 'tuple[int, ...]', residual: 'Vector') -> None
```

Cards: C.2A ExactLinearSystem.

SIG-0125. 1 research card. Kinds: `function`.

```
(constraints: 'Iterable[Any]', *, tolerance: 'float' = 1e-10) -> 'ConstraintClosureReport'
```

Cards: C.1G constraint_closure.

SIG-0126. 1 research card. Kinds: `function`.

```
(coordinates: 'LatticePolyphaseCoordinates') -> 'np.ndarray'
```

Cards: C.4B lattice_polyphase_synthesize.

SIG-0127. 1 research card. Kinds: `function`.

```
(coordinates: 'PolyphaseCoordinates') -> 'np.ndarray'
```

Cards: C.4B polyphase_synthesize.

SIG-0131. 1 research card. Kinds: `function`.

```
(count: 'int') -> 'tuple[Fraction, ...]'
```

Cards: C.3A bernoulli_numbers.

SIG-0133. 1 research card. Kinds: `function`.

```
(covector: 'Sequence[Scalar]', transform: 'CoordinateTransform') -> 'Vector'
```

Cards: C.2A transport_covector.

SIG-0134. 1 research card. Kinds: `function`.

```
(covector: 'Sequence[Scalar]', vector: 'Sequence[Scalar]') -> 'Fraction'
```

Cards: C.2A coordinate_pairing.

SIG-0135. 1 research card. Kinds: `function`.

```
(covector: 'Sequence[Scalar]', vector: 'Sequence[Scalar]', transform: 'CoordinateTransform') -> 'bool'
```

Cards: C.2A pairing_invariant.

SIG-0138. 1 research card. Kinds: `function`.

```
(data: 'ExactData', expression_type: 'Any') -> 'str'
```

Cards: C.1B normal_form_digest.

SIG-0139. 1 research card. Kinds: `function`.

```
(degree: 'int') -> 'FiniteBasis'
```

Cards: C.2B bernstein_basis.

SIG-0142. 1 research card. Kinds: `function`.

```
(degree: 'int', *, name: 'str' = 'identity formal coordinate', coordinate: 'str' = 'formal coordinate') ->
    ↪ 'FormalCoordinateChange'
```

Cards: C.2C identity_formal_coordinate.

SIG-0143. 1 research card. Kinds: `function`.

```
(degree: 'int', fibonacci_degree: 'int') -> 'int'
```

Cards: C.2B monomial_to_fibonacci_coefficient.

SIG-0144. 1 research card. Kinds: `function`.

```
(degree: 'int', index: 'int') -> 'Polynomial'
```

Cards: C.2B bernstein_polynomial.

SIG-0145. 1 research card. Kinds: `class`.

```
(degree: 'int', max_tested_degree: 'int', exact: 'bool', certificate: 'Certificate') -> None
```

Cards: C.1D QuadratureExactness.

SIG-0146. 1 research card. Kinds: `function`.

```
(degree: 'int', values_at_zero: 'Sequence[Scalar]') -> 'Polynomial'
```

Cards: C.2B appell_polynomial.

SIG-0147. 1 research card. Kinds: `class`.

```
(depth: 'int', induced_correction_map: 'np.ndarray', defect_coordinates: 'tuple[int, ...]', correction_space:
    ↪ 'ModularLinearSolution', correction_representatives: 'tuple[tuple[int, ...], ...]', certificate:
    ↪ 'Certificate') -> None
```

Cards: C.5A DepthObstructionResult.

SIG-0148. 1 research card. Kinds: `function`.

```
(descriptor: 'BasisDescriptor') -> 'WeightedMonomialBasis'
```

Cards: C.3A basis_from_descriptor.

SIG-0150. 1 research card. Kinds: `function`.

```
(detail_coefficients: 'Any') -> 'float'
```

Cards: C.4B estimate_white_noise_sigma.

SIG-0151. 1 research card. Kinds: `class`.

```
(detections: 'tuple[JumpDetection, ...]', threshold: 'float', background_bound: 'float', location_error_bound:
    ↪ 'float', amplitude_error_bound: 'float', certificate: 'Certificate') -> None
```

Cards: C.4B JumpRecoveryResult.

SIG-0152. 1 research card. Kinds: `class`.

```
(diagonal: 'BlockDiagonalOperator', correction: 'LowRankOperator') -> None
```

Cards: C.1D BlockDiagonalLowRankOperator.

SIG-0153. 1 research card. Kinds: `function`.

```
(digit_values: 'Sequence[int]', *, radix: 'int', modulus: 'int', count: 'int') -> 'tuple[int, ...]'
```

Cards: C.3B lift_digit_sequence.

SIG-0154. 1 research card. Kinds: `function`.

```
(digit_weights: 'Sequence[Sequence[Scalar]]', radix: 'int', *, budget: 'GeneratingBudget' =
    ↪ GeneratingBudget(maximum_degree=100000, maximum_coefficient_bits=4000000, maximum_matrix_dimension=4096,
    ↪ maximum_reachable_states=100000, maximum_local_combinations=1000000, maximum_modulus=2000000,
    ↪ maximum_serialized_bytes=67108864, maximum_recursion_depth=1000)) -> 'DigitCauchyProductPlan'
```

Cards: C.3B compile_digit_cauchy_product.

SIG-0155. 1 research card. Kinds: `function`.

```
(digits: 'Iterable[int]', radices: 'Iterable[int]') -> 'int'
```

Cards: C.4B mixed_radix_address.

SIG-0159. 1 research card. Kinds: `class`.

```
(dimension: 'int', normalized: 'bool' = False, dtype_value: 'Any' = <class 'float'>) -> None
```

Cards: C.1D WalshHadamardOperator.

SIG-0160. 1 research card. Kinds: `class`.

```
(dimension: 'int', terms: 'Mapping[Sequence[int], Rational] | Iterable[tuple[Sequence[int], Rational]]') -> 'None'
```

Cards: C.1D ExactPolynomial.

SIG-0161. 1 research card. Kinds: `class`.

```
(dimensions: 'Mapping[int, int]', differentials: 'Mapping[int, Any]', *, prime: 'int') -> 'None'
```

Cards: C.1G FiniteCochainComplex.

SIG-0163. 1 research card. Kinds: `function`.

```
(dimensions: 'Sequence[int]') -> 'MatrixChainPlan'
```

Cards: C.1B matrix_chain_plan.

SIG-0164. 1 research card. Kinds: `class`.

```
(dimensions: 'tuple[int, ...]', tree: 'MatrixChainTree', scalar_operation_cost: 'int', left_associative_cost:
   ↪ 'int', right_associative_cost: 'int', certificate: 'Certificate') -> None
```

Cards: C.1B MatrixChainPlan.

SIG-0166. 1 research card. Kinds: `function`.

```
(document: 'MathDocument', *, options: 'HtmlRenderOptions | None' = None) -> 'str'
```

Cards: C.6 render_html.

SIG-0173. 1 research card. Kinds: `class`.

```
(edge_index: 'int', source: 'str', target: 'str', matrix: 'Any') -> None
```

Cards: C.5A FundamentalHolonomy.

SIG-0174. 1 research card. Kinds: `class`.

```
(eigenvalues: 'np.ndarray', synthesis: 'NumericCoordinateTransform', analysis: 'NumericCoordinateTransform',
   ↪ diagonal_residual: 'float') -> None
```

Cards: C.2C ModalCoordinateSystem.

SIG-0175. 1 research card. Kinds: `class`.

```
(elements: 'Iterable[Element]', multiply: 'Callable[[Element, Element], Element]', identity: 'Element', *, name:
   ↪ 'str' = 'G', validate: 'bool' = True, associativity_limit: 'int' = 128) -> 'None'
```

Cards: C.1G FiniteGroup.

SIG-0176. 1 research card. Kinds: `function`.

```
(engine: 'LatexEngine | str' = <LatexEngine.AUTO: 'auto'>) -> 'LatexEngine'
```

Cards: C.6 resolve_latex_engine.

SIG-0179. 1 research card. Kinds: `class`.

```
(equations: 'dict[str, Any]', reconstructed: 'Any', direct: 'Any', max_order: 'int', certificate: 'Certificate')
   ↪ -> None
```

Cards: C.5A HigherMaurerCartanResult.

SIG-0180. 1 research card. Kinds: `class`.

```
(equivalent: 'bool', left: 'NormalForm | None', right: 'NormalForm | None', certificate: 'Certificate') -> None
```

Cards: C.1B EquivalenceResult.

SIG-0182. 1 research card. Kinds: `function`.

```
(euler_degree: 'int', fibonacci_degree: 'int') -> 'Fraction'
```

Cards: C.2B euler_to_fibonacci_coefficient.

SIG-0184. 1 research card. Kinds: `class`.

```
(evaluator: 'Callable[[int], Number]', *, name: 'str' = 'sequence', exact: 'bool' = True, metadata: 'dict[str, Any]
   ↪  | None' = None) -> 'None'
```

Cards: C.1G Sequence.

SIG-0186. 1 research card. Kinds: `function`.

```
(exponent: 'float') -> 'CostFunction'
```

Cards: C.4B lp_cost.

SIG-0187. 1 research card. Kinds: `function`.

```
(exponents: 'Mapping[int, int]') -> 'Any'
```

Cards: C.3B normalized_cyclotomic_expression.

SIG-0188. 1 research card. Kinds: `function`.

```
(expression: 'Expression', *, budget: 'ExpressionBudget | None' = None) -> 'dict[str, Any]'
```

Cards: C.1B expression_to_dict.

SIG-0190. 1 research card. Kinds: `class`.

```
(expression: 'Expression', before: 'NormalForm', after: 'NormalForm', steps: 'tuple[RewriteStep, ...]',
   ↪  certificate: 'Certificate') -> None
```

Cards: C.1B SimplificationResult.

SIG-0191. 1 research card. Kinds: `function`.

```
(expression: 'Expression', bindings: 'Mapping[str | Symbol, ExpressionBinding] | None' = None, *, budget:
   ↪  'ExpressionBudget | None' = None) -> 'CheckedExpression'
```

Cards: C.1B check_expression.

SIG-0192. 1 research card. Kinds: `function`.

```
(expression: 'Expression', bindings: 'Mapping[str | Symbol, ExpressionBinding] | None' = None, *, budget:
   ↪  'ExpressionBudget | None' = None) -> 'ExpressionPlan'
```

Cards: C.1B compile_expression.

SIG-0193. 1 research card. Kinds: `function`.

```
(expression: 'Expression', bindings: 'Mapping[str | Symbol, ExpressionBinding] | None' = None, *, budget:
   ↪ 'ExpressionBudget | None' = None) -> 'ExpressionResult'
```

Cards: C.1B evaluate_expression.

SIG-0194. 1 research card. Kinds: `function`.

```
(expression: 'Expression', bindings: 'Mapping[str | Symbol, ExpressionBinding] | None' = None, *, budget:
   ↪ 'ExpressionBudget | None' = None) -> 'NormalForm'
```

Cards: C.1B normalize_expression.

SIG-0195. 1 research card. Kinds: `function`.

```
(expression: 'Expression', bindings: 'Mapping[str | Symbol, ExpressionBinding] | None' = None, *, budget:
   ↪ 'ExpressionBudget | None' = None) -> 'SimplificationResult'
```

Cards: C.1B simplify_expression.

SIG-0196. 1 research card. Kinds: `class`.

```
(expression_digest: 'str', expression_type: 'CoordinateSpace | LinearMapType', instructions:
   ↪ 'tuple[PlanInstruction, ...]', output_index: 'int', scalar_operation_cost: 'int', plan_digest: 'str',
   ↪ certificate: 'Certificate') -> None
```

Cards: C.1B ExpressionPlan.

SIG-0197. 1 research card. Kinds: `class`.

```
(expression_type: 'Any', data: 'Any', digest: 'str', certificate: 'Certificate') -> None
```

Cards: C.1B NormalForm.

SIG-0198. 1 research card. Kinds: `class`.

```
(expression_type: 'CoordinateSpace | LinearMapType', stats: 'ExpressionStats', free_symbols: 'tuple[Symbol, ...]',
   ↪ certificate: 'Certificate') -> None
```

Cards: C.1B CheckedExpression.

SIG-0199. 1 research card. Kinds: `function`.

```
(expression_type: 'ExpressionType', *, budget: 'ExpressionBudget | None' = None) -> 'dict[str, Any]'
```

Cards: C.1B type_to_dict.

SIG-0201. 1 research card. Kinds: `class`.

```
(factors: 'Iterable[Measure]', *, name: 'str' = 'product measure') -> 'None'
```

Cards: C.1D ProductMeasure.

SIG-0202. 1 research card. Kinds: `class`.

```
(factors: 'tuple[Expression, ...]') -> None
```

Cards: C.1B Compose.

SIG-0203. 1 research card. Kinds: `class`.

```
(families: 'tuple[PolynomialFamily, ...]') -> None
```

Cards: C.2B PolynomialFamilyCatalog.

SIG-0211. 1 research card. Kinds: `function`.

```
(family: 'PolynomialFamily', coefficients: 'Iterable[Rational]') -> 'ExactPolynomial'
```

Cards: C.1D polynomial_from_family_coordinates.

SIG-0212. 1 research card. Kinds: `function`.

```
(family: 'PolynomialFamily', coefficients: 'Iterable[Rational]', measure: 'Measure') -> 'IntegralResult'
```

Cards: C.1D integrate_polynomial_coordinates.

SIG-0213. 1 research card. Kinds: `function`.

```
(family: 'PolynomialFamily', degree: 'int') -> 'Matrix'
```

Cards: C.2B derivative_operator_in_family.

SIG-0215. 1 research card. Kinds: `function`.

```
(family: 'PolynomialFamily', degree: 'int', *, scale: 'Scalar' = 1, shift: 'Scalar' = 0, budget: 'CoordinateBudget'
   ↪  = CoordinateBudget(maximum_matrix_dimension=512, maximum_polynomial_degree=512, maximum_horizon=4096,
   ↪  maximum_exact_entries=1000000)) -> 'CoordinateTransform'
```

Cards: C.2B affine_substitution_transform.

SIG-0216. 1 research card. Kinds: `function`.

```
(family: 'PolynomialFamily', degree: 'int', *, scale: 'Scalar' = 1, shift: 'Scalar' = 0, budget: 'CoordinateBudget'
   ↪  = CoordinateBudget(maximum_matrix_dimension=512, maximum_polynomial_degree=512, maximum_horizon=4096,
   ↪  maximum_exact_entries=1000000)) -> 'PolynomialCoordinateOperator'
```

Cards: C.2B affine_substitution_coordinate_operator.

SIG-0217. 1 research card. Kinds: `function`.

```
(family: 'PolynomialFamily', degree: 'int', *, step: 'Scalar' = 1, budget: 'CoordinateBudget' =
   ↪  CoordinateBudget(maximum_matrix_dimension=512, maximum_polynomial_degree=512, maximum_horizon=4096,
   ↪  maximum_exact_entries=1000000)) -> 'PolynomialCoordinateOperator'
```

Cards: C.2B finite_difference_coordinate_operator.

SIG-0218. 1 research card. Kinds: `function`.

```
(family: 'PolynomialFamily', degree: 'int', measure: 'IntervalMeasure') -> 'IntegralIdentity'
```

Cards: C.1D coordinate_fundamental_theorem.

SIG-0219. 1 research card. Kinds: `function`.

```
(family: 'PolynomialFamily', degree: 'int', measure: 'IntervalMeasure') -> 'PolynomialIntegrationCovector'
```

Cards: C.1D polynomial_integration_covector.

SIG-0220. 1 research card. Kinds: `function`.

```
(family: 'PolynomialFamily', degree: 'int', multiplier: 'Sequence[Scalar]', *, budget: 'CoordinateBudget' =
    ↪ CoordinateBudget(maximum_matrix_dimension=512, maximum_polynomial_degree=512, maximum_horizon=4096,
    ↪ maximum_exact_entries=1000000)) -> 'PolynomialCoordinateOperator'
```

Cards: C.2B multiplication_coordinate_operator.

SIG-0222. 1 research card. Kinds: `class`.

```
(family: 'str', degree: 'int', measure: 'IntervalMeasure', weights: 'tuple[Fraction, ...]', basis_witness:
    ↪ 'tuple[tuple[Fraction, ...], ...] | None' = None) -> None
```

Cards: C.1D PolynomialIntegrationCovector.

SIG-0223. 1 research card. Kinds: `function`.

```
(field: 'FiniteField', exponent: 'int', *, additive_frequency: 'int' = 1, tolerance: 'float' = 1e-09) ->
    ↪ 'GaussSumResult'
```

Cards: C.1G gauss_sum.

SIG-0224. 1 research card. Kinds: `class`.

```
(filter: 'LinearFilter') -> None
```

Cards: C.4A StreamingLinearFilter.

SIG-0225. 1 research card. Kinds: `function`.

```
(filter_: 'LinearFilter', signal: 'Any', chunk_sizes: 'Any', *, tolerance: 'float' = 1e-12) ->
    ↪ 'ComputationResult[np.ndarray]'
```

Cards: C.4A stream_lfilter.

SIG-0227. 1 research card. Kinds: `class`.

```
(first_column: 'Array') -> None
```

Cards: C.1D CirculantOperator.

SIG-0228. 1 research card. Kinds: `class`.

```
(fixture_path: 'Path', fixture_digest: 'str', results: 'tuple[BehavioralGoldenCaseResult, ...]', certificate:
    ↪ 'Certificate') -> None
```

Cards: C.1A BehavioralGoldenReport.

SIG-0229. 1 research card. Kinds: `class`.

```
(form: 'AlternatingForm', linear: 'tuple[int, ...]' = (), max_dimension: 'int' = 20) -> None
```

Cards: C.1G QuadraticRefinement.

SIG-0230. 1 research card. Kinds: `function`.

```
(form: 'Iterable[Iterable[Scalar]]', transform: 'CoordinateTransform') -> 'Matrix'
```

Cards: C.2A transport_bilinear_form.

SIG-0231. 1 research card. Kinds: `class`.

```
(frame_length: 'int' = 256, hop_length: 'int | None' = None, fft_length: 'int | None' = None, window: 'WindowSpec'
   ↪  = 'hann', sample_rate: 'float' = 1.0, onesided: 'bool' = True, detrend: 'DetrendMode' = 'none') -> None
```

Cards: C.4A StreamingSTFT.

SIG-0232. 1 research card. Kinds: `function`.

```
(frames: 'Any', hop_length: 'int', *, synthesis_window: 'Any | None' = None, length: 'int | None' = None) ->
   ↪  'np.ndarray'
```

Cards: C.4A overlap_add.

SIG-0233. 1 research card. Kinds: `class`.

```
(frames: 'np.ndarray', starts: 'np.ndarray', original_length: 'int', frame_length: 'int', hop_length: 'int',
   ↪  pad_left: 'int', pad_right: 'int', boundary: 'BoundaryMode') -> None
```

Cards: C.4A FrameBatch.

SIG-0235. 1 research card. Kinds: `class`.

```
(frequencies: 'np.ndarray', cross_power: 'np.ndarray', input_power: 'np.ndarray', output_power: 'np.ndarray',
   ↪  coherence: 'np.ndarray', transfer_h1: 'np.ndarray', sample_rate: 'float', scaling: 'SpectralScaling',
   ↪  onesided: 'bool', fft_length: 'int', segment_count: 'int', certificate: 'Certificate') -> None
```

Cards: C.4A CrossSpectrum.

SIG-0236. 1 research card. Kinds: `class`.

```
(frequencies: 'np.ndarray', power: 'np.ndarray', sample_rate: 'float', scaling: 'SpectralScaling', onesided:
   ↪  'bool', fft_length: 'int', segment_count: 'int', frame_length: 'int', hop_length: 'int', window:
   ↪  'np.ndarray', method: 'str', certificate: 'Certificate') -> None
```

Cards: C.4A PowerSpectrum.

SIG-0237. 1 research card. Kinds: `class`.

```
(frequencies: 'np.ndarray', response: 'np.ndarray', sample_rate: 'float') -> None
```

Cards: C.4A FrequencyResponse.

SIG-0246. 1 research card. Kinds: `function`.

```
(gamma: 'ArrayLike') -> 'np.ndarray'
```

Cards: C.4B impedance_from_gamma.

SIG-0247. 1 research card. Kinds: `function`.

```
(gamma: 'ArrayLike', *, regularization: 'float' = 0.0) -> 'np.ndarray'
```

Cards: C.4B smith_sensitivity.

SIG-0248. 1 research card. Kinds: `class`.

```
(gates: 'Iterable[Any]', packet_systems: 'Iterable[PacketSystem]', *, tolerance: 'float' = 1e-10, max_paths: 'int'
   ↪  = 1000000) -> 'None'
```

Cards: C.1G CircuitPacketExpansion.

SIG-0249. 1 research card. Kinds: `class`.

```
(gates: 'Mapping[CompletionGate | str, GateStatus]') -> None
```

Cards: C.1G AutomorphicCompletionReport.

SIG-0251. 1 research card. Kinds: `class`.

```
(generator: 'np.ndarray') -> None
```

Cards: C.1G MatrixSemigroup.

SIG-0252. 1 research card. Kinds: `class`.

```
(generators: 'Mapping[str, Any] | Sequence[Any]', *, backend: 'str | LinearAlgebraBackend | None' = None,
   ↪ tolerance: 'float | None' = None, include_identity: 'bool' = True) -> 'None'
```

Cards: C.1D OperatorAlgebra.

SIG-0253. 1 research card. Kinds: `class`.

```
(grammar: 'OperatorGrammar | None' = None, realization: 'OperatorRealization | None' = None, seed_system:
   ↪ 'SeedSystem | None' = None, packet_system: 'PacketSystem | None' = None, operators: 'Mapping[str, Any]' =
   ↪ <factory>, refinement_tower: 'RefinementTower | None' = None, metadata: 'dict[str, Any]' = <factory>) -> None
```

Cards: C.1G UnifiedSystem.

SIG-0254. 1 research card. Kinds: `class`.

```
(grammar: 'OperatorGrammar', spaces: 'Mapping[str, int]', operators: 'Mapping[str, Any]', *, backend: 'str |
   ↪ LinearAlgebraBackend | None' = None, validate_relations: 'bool' = True, tolerance: 'float | None' = None) ->
   ↪ 'None'
```

Cards: C.1D OperatorRealization.

SIG-0259. 1 research card. Kinds: `class`.

```
(group: 'FiniteGroup[Element]', character: 'LinearCharacter[Element]') -> 'None'
```

Cards: C.1G InducedRepresentation.

SIG-0260. 1 research card. Kinds: `function`.

```
(group: 'FiniteGroup[Element]', left: 'LinearCharacter[Element]', right: 'LinearCharacter[Element]') ->
   ↪ 'MackeyDecomposition[Element]'
```

Cards: C.1G mackey_tensor_packets.

SIG-0261. 1 research card. Kinds: `class`.

```
(hamiltonian: 'Any', jumps: 'Iterable[Any]', *, tolerance: 'float' = 1e-10) -> 'None'
```

Cards: C.1G LindbladGenerator.

SIG-0262. 1 research card. Kinds: `class`.

```
(hamiltonian: 'Any', lattice_spacing: 'float', *, ground_energy: 'float | None' = None, tolerance: 'float' = 1e-10)
   ↪ -> 'None'
```

Cards: C.1G EuclideanTransfer.

SIG-0263. 1 research card. Kinds: `function`.

```
(hamiltonian: 'Any', projector: 'Any', time: 'float', *, tolerance: 'float' = 1e-10) -> 'LeakageResult'
```

Cards: C.1G fixed_packet_leakage.

SIG-0264. 1 research card. Kinds: `class`.

```
(hamiltonian: 'Any', retained: 'Iterable[int]', *, tolerance: 'float' = 1e-10) -> 'None'
```

Cards: C.1G QuantumFeshbachReduction.

SIG-0266. 1 research card. Kinds: `class`.

```
(hidden: 'np.ndarray', point: 'np.ndarray', residual: 'float', iterations: 'int', certificate: 'Certificate') ->
   ↪ None
```

Cards: C.5A HiddenSolveResult.

SIG-0267. 1 research card. Kinds: `class`.

```
(hidden_jacobian: 'Any', hidden_inverse: 'Any', implicit_derivative: 'Any', reduced_derivative: 'Any',
   ↪ certificate: 'Certificate') -> None
```

Cards: C.5A SchurDerivativeResult.

SIG-0268. 1 research card. Kinds: `class`.

```
(identifier: 'str', title: 'str', status: 'ClaimStatus', source: 'str', assumptions: 'tuple[str, ...]' = (),
   ↪ limitations: 'tuple[str, ...]' = (), verification_suite: 'str | None' = None) -> None
```

Cards: C.1A ProvenanceRecord.

SIG-0269. 1 research card. Kinds: `function`.

```
(image: 'np.ndarray', cutoff: 'int') -> 'np.ndarray'
```

Cards: C.4B fft_lowpass_2d.

SIG-0270. 1 research card. Kinds: `function`.

```
(impedance: 'ArrayLike') -> 'np.ndarray'
```

Cards: C.4B gamma_from_impedance.

SIG-0271. 1 research card. Kinds: `function`.

```
(impedance: 'ArrayLike', order: 'int', phase: 'int') -> 'np.ndarray'
```

Cards: C.4B phase_mobius.

SIG-0272. 1 research card. Kinds: `class`.

```
(include_mathjax: 'bool' = False, title: 'str | None' = None) -> None
```

Cards: C.6 HtmlRenderOptions.

SIG-0277. 1 research card. Kinds: `function`.

```
(index: 'int', base: 'int', prime: 'int', exponent: 'int', power: 'int' = 1) -> 'tuple[int, ...]'
```

Cards: C.3B q_binomial_support.

SIG-0278. 1 research card. Kinds: `function`.

```
(index: 'int', factorial_power: 'int') -> 'int'
```

Cards: C.3B generalized_bell_parity.

SIG-0281. 1 research card. Kinds: `function`.

```
(index: 'int', prime: 'int') -> 'KummerLayerDistribution'
```

Cards: C.3B kummer_layer_distribution.

SIG-0282. 1 research card. Kinds: `function`.

```
(index: 'int', prime: 'int') -> 'PointedSubindexCertificate'
```

Cards: C.3B certify_pointed_subindex_identity.

SIG-0283. 1 research card. Kinds: `function`.

```
(index: 'int', prime: 'int') -> 'tuple[int, ...]'
```

Cards: C.3B pointed_subindex_polynomial.

SIG-0284. 1 research card. Kinds: `function`.

```
(index: 'int', prime: 'int') -> 'tuple[int, int]'
```

Cards: C.3B p_free_orbit.

SIG-0285. 1 research card. Kinds: `class`.

```
(index: 'int', prime: 'int', counts: 'tuple[int, ...]') -> None
```

Cards: C.3B KummerLayerDistribution.

SIG-0286. 1 research card. Kinds: `class`.

```
(index: 'int', prime: 'int', exponent: 'int', factorial_power: 'int', modulus: 'int', layer_sums: 'tuple[int,
   ↪ ...]') -> None
```

Cards: C.3B CarryLayerConvolution.

SIG-0288. 1 research card. Kinds: `class`.

```
(index: 'int', prime: 'int', factorial_power: 'int', digit_sum: 'int', high_polynomial: 'tuple[int, ...]',
   ↪ reduced_polynomial: 'tuple[int, ...]') -> None
```

Cards: C.3B BellDigitReduction.

SIG-0289. 1 research card. Kinds: `function`.

```
(index: 'int', prime: 'int', factorial_power: 'int', digit_weights: 'Iterable[int]', *, budget: 'GeneratingBudget'
   ↪ = GeneratingBudget(maximum_degree=100000, maximum_coefficient_bits=4000000, maximum_matrix_dimension=4096,
   ↪ maximum_reachable_states=100000, maximum_local_combinations=1000000, maximum_modulus=2000000,
   ↪ maximum_serialized_bytes=67108864, maximum_recursion_depth=1000)) -> 'BellDigitReduction'
```

Cards: C.3B weighted_bell_digit_reduction.

SIG-0290. 1 research card. Kinds: `function`.

```
(index: 'int', prime: 'int', factorial_power: 'int', modulus_exponent: 'int') -> 'int'
```

Cards: C.3B factorial_power_support_count.

SIG-0291. 1 research card. Kinds: `function`.

```
(index: 'int', prime: 'int', factorial_power: 'int', modulus_exponent: 'int', *, budget: 'GeneratingBudget' =
    ↪ GeneratingBudget(maximum_degree=100000, maximum_coefficient_bits=4000000, maximum_matrix_dimension=4096,
    ↪ maximum_reachable_states=100000, maximum_local_combinations=1000000, maximum_modulus=2000000,
    ↪ maximum_serialized_bytes=67108864, maximum_recursion_depth=1000)) -> 'tuple[int, ...]'
```

Cards: C.3B factorial_power_support_indices.

SIG-0292. 1 research card. Kinds: `class`.

```
(index: 'int', prime: 'int', left: 'tuple[int, ...]', right: 'tuple[int, ...]') -> None
```

Cards: C.3B PointedSubindexCertificate.

SIG-0293. 1 research card. Kinds: `function`.

```
(index: 'int', prime: 'int', weights: 'Sequence[int]') -> 'int'
```

Cards: C.3B digit_unit.

SIG-0294. 1 research card. Kinds: `function`.

```
(index: 'int', radix: 'int') -> 'tuple[int, ...]'
```

Cards: C.3B base_digits.

SIG-0295. 1 research card. Kinds: `function`.

```
(index: 'int', subindex: 'int', base: 'int') -> 'int'
```

Cards: C.3B gaussian_binomial.

SIG-0296. 1 research card. Kinds: `function`.

```
(index: 'int', subindex: 'int', base: 'int', prime: 'int') -> 'int'
```

Cards: C.3B q_binomial_valuation_direct.

SIG-0298. 1 research card. Kinds: `class`.

```
(index: 'int', subindex: 'int', base: 'int', prime: 'int', order: 'int', lift_constant: 'int', quotient_index:
    ↪ 'int', quotient_subindex: 'int', residue_carry: 'int', quotient_carries: 'int', endpoint_valuation: 'int',
    ↪ valuation: 'int') -> None
```

Cards: C.3B QKummerDecomposition.

SIG-0299. 1 research card. Kinds: `class`.

```
(index: 'int', subindex: 'int', factorial_power: 'int', prime: 'int', exponent: 'int', valuation: 'int', unit:
    ↪ 'int', residue: 'int') -> None
```

Cards: C.3B NormalizedKernelResidue.

SIG-0301. 1 research card. Kinds: `class`.

```
(index: 'int', subindex: 'int', prime: 'int', exponent: 'int', modulus: 'int', valuation: 'int', unit: 'int',
    ↪ residue: 'int') -> None
```

Cards: C.3B PrimePowerBinomial.

SIG-0302. 1 research card. Kinds: `function`.

```
(index: 'int', subindex: 'int', prime: 'int', factorial_power: 'int', unit_weights: 'Sequence[int]') -> 'int'
```

Cards: C.3B digit_factorized_normalized_kernel.

SIG-0303. 1 research card. Kinds: `class`.

```
(indices: 'tuple[int, ...]', response: 'np.ndarray', threshold: 'float', noise_amplification: 'float',
    ↪ certificate: 'Certificate') -> None
```

Cards: C.4B BreakpointBandResult.

SIG-0304. 1 research card. Kinds: `class`.

```
(indices: 'tuple[int, ...]', value: 'tuple[Any, ...]') -> None
```

Cards: C.5B PacketJetCoefficient.

SIG-0305. 1 research card. Kinds: `class`.

```
(input_labels: 'tuple[str, ...]', value: 'Any') -> None
```

Cards: C.5A GradedConvolutionTerm.

SIG-0306. 1 research card. Kinds: `function`.

```
(input_length: 'int', kernel_length: 'int', mode: 'ConvolutionMode') -> 'int'
```

Cards: C.4A convolution_output_length.

SIG-0307. 1 research card. Kinds: `function`.

```
(input_length: 'int', up: 'int', down: 'int') -> 'int'
```

Cards: C.4A resampled_length.

SIG-0308. 1 research card. Kinds: `function`.

```
(input_signal: 'Any', desired_signal: 'Any', num_taps: 'int', *, forgetting_factor: 'float' = 0.99, delta: 'float'
    ↪ = 1.0, initial_weights: 'Any | None' = None) -> 'AdaptiveFilterResult'
```

Cards: C.4B rls_filter.

SIG-0309. 1 research card. Kinds: `function`.

```
(input_signal: 'Any', desired_signal: 'Any', num_taps: 'int', *, step_size: 'float' = 0.01, leakage: 'float' = 0.0,
    ↪ initial_weights: 'Any | None' = None) -> 'AdaptiveFilterResult'
```

Cards: C.4B lms_filter.

SIG-0310. 1 research card. Kinds: `function`.

```
(input_signal: 'Any', desired_signal: 'Any', num_taps: 'int', *, step_size: 'float' = 0.5, epsilon: 'float' =
    ↪ 1e-08, leakage: 'float' = 0.0, initial_weights: 'Any | None' = None) -> 'AdaptiveFilterResult'
```

Cards: C.4B nlms_filter.

SIG-0311. 1 research card. Kinds: `function`.

```
(input_signal: 'Any', output_signal: 'Any', *, sample_rate: 'float' = 1.0, frame_length: 'int' = 256, hop_length:
    ↪ 'int | None' = None, fft_length: 'int | None' = None, window: 'WindowSpec' = 'hann', detrend: 'DetrendMode'
    ↪ = 'constant', scaling: 'SpectralScaling' = 'density', average: 'SpectralAverage' = 'mean', onesided: 'bool |
    ↪ None' = None, regularization: 'float' = 0.0) -> 'CrossSpectrum'
```

Cards: C.4A cross_spectrum.

SIG-0312. 1 research card. Kinds: `class`.

```
(inputs: 'tuple[int, ...]', outputs: 'tuple[int, ...]') -> None
```

Cards: C.5B PacketJetInputGroup.

SIG-0314. 1 research card. Kinds: `function`.

```
(integrand: 'ExactPolynomial | Callable[[Any], Any]', measure: 'Measure', *, provider: 'CompositeSimpson | None' =
    ↪ None) -> 'IntegralResult'
```

Cards: C.1D integrate.

SIG-0315. 1 research card. Kinds: `class`.

```
(intertwiners: 'Mapping[str, Any]', edge_residuals: 'Mapping[str, float]', cycle_residuals: 'Mapping[str, float]',
    ↪ certificate: 'Certificate') -> None
```

Cards: C.5A IntertwiningResult.

SIG-0316. 1 research card. Kinds: `class`.

```
(items: 'tuple[str, ...]', ordered: 'bool' = False) -> None
```

Cards: C.6 ListBlock.

SIG-0319. 1 research card. Kinds: `function`.

```
(kernel: 'Any', input_length: 'int', *, mode: 'ConvolutionMode' = 'full') -> 'np.ndarray'
```

Cards: C.4A convolution_matrix.

SIG-0321. 1 research card. Kinds: `function`.

```
(kernel: 'Any', order: 'int') -> 'tuple[tuple[PolyphaseKernel, ...], ...]'
```

Cards: C.4B polyphase_filter_kernels.

SIG-0322. 1 research card. Kinds: `class`.

```
(kernel: 'np.ndarray', block_length: 'int' = 1024) -> None
```

Cards: C.4A OverlapAddConvolver.

SIG-0323. 1 research card. Kinds: `class`.

```
(kernel: 'np.ndarray', input_length: 'int', order: 'int', mode: 'ConvolutionMode' = 'full') -> None
```

Cards: C.4A PolyphaseConvolution.

SIG-0324. 1 research card. Kinds: `class`.

```
(kernel_coordinates: 'np.ndarray', complement_coordinates: 'np.ndarray', point: 'np.ndarray', reduced_obstruction:
    ↪ 'np.ndarray', range_residual: 'float', iterations: 'int', certificate: 'Certificate') -> None
```

Cards: C.5A LyapunovSchmidtPoint.

SIG-0325. 1 research card. Kinds: `function`.

```
(key: 'str') -> 'FeatureSpec'
```

Cards: C.1G feature.

SIG-0326. 1 research card. Kinds: `class`.

```
(key: 'str', module: 'str', title: 'str', status: 'FeatureStatus', summary: 'str', papers: 'tuple[str, ...]' = (),
    ↪ dependencies: 'tuple[str, ...]' = (), public_objects: 'tuple[str, ...]' = ()) -> None
```

Cards: C.1G FeatureSpec.

SIG-0327. 1 research card. Kinds: `function`.

```
(kind: 'BiquadKind', frequency: 'float', *, q: 'float' = np.float64(0.7071067811865475), sample_rate: 'float' =
    ↪ 1.0) -> 'LinearFilter'
```

Cards: C.4A design_biquad.

SIG-0328. 1 research card. Kinds: `function`.

```
(kind: 'FIRKind', cutoff: 'float | tuple[float, float]', *, num_taps: 'int' = 101, sample_rate: 'float' = 1.0,
    ↪ window: 'WindowSpec' = 'hann', scale: 'bool' = True) -> 'FIRFilter'
```

Cards: C.4A design_fir.

SIG-0329. 1 research card. Kinds: `class`.

```
(kind: 'PrimitiveKind', map_type: 'LinearMapType', parameters: 'tuple[Fraction, ...]' = ()) -> None
```

Cards: C.1B PolynomialPrimitive.

SIG-0330. 1 research card. Kinds: `class`.

```
(kraus: 'Iterable[Any]', *, tolerance: 'float' = 1e-10) -> 'None'
```

Cards: C.1G QuantumChannel.

SIG-0333. 1 research card. Kinds: `class`.

```
(lattice_basis: 'Any', projectors: 'Any', *, prime: 'int', split_basis: 'Any | None' = None) -> 'None'
```

Cards: C.1G LatticePacketData.

SIG-0334. 1 research card. Kinds: `function`.

```
(leaves: 'int', arities: 'Sequence[int]', *, max_trees: 'int' = 100000) -> 'tuple[OrderedRootedTree, ...]'
```

Cards: C.5B ordered_trees_with_leaves.

SIG-0335. 1 research card. Kinds: `class`.

```
(leaves: 'tuple[str, ...]', cost: 'float', decisions: 'dict[str, str]', margins: 'dict[str, float]', certificate:
    ↪ 'Certificate') -> None
```

Cards: C.4B BestBasisResult.

SIG-0336. 1 research card. Kinds: `function`.

```
(left: 'Any', right: 'Any', rank: 'int', *, tolerance: 'float' = 1e-10) -> 'RecompressionResult'
```

Cards: C.1D recompress_factors.

SIG-0337. 1 research card. Kinds: `class`.

```
(left: 'Array', right: 'Array') -> None
```

Cards: C.1D LowRankOperator.

SIG-0338. 1 research card. Kinds: `class`.

```
(left: 'AtlasPath', right: 'AtlasPath', valid: 'bool', difference: 'Matrix') -> None
```

Cards: C.2C CommutingDiagramCheck.

SIG-0340. 1 research card. Kinds: `function`.

```
(left: 'Expression', right: 'Expression', *, left_bindings: 'Mapping[str | Symbol, ExpressionBinding] | None' =
    ↪ None, right_bindings: 'Mapping[str | Symbol, ExpressionBinding] | None' = None, budget: 'ExpressionBudget |
    ↪ None' = None) -> 'EquivalenceResult'
```

Cards: C.1B equivalent.

SIG-0342. 1 research card. Kinds: `function`.

```
(left: 'Iterable[int | Fraction]', right: 'Iterable[int | Fraction]', *, count: 'int') -> 'tuple[Fraction, ...]'
```

Cards: C.3A dirichlet_convolution.

SIG-0344. 1 research card. Kinds: `function`.

```
(left: 'PacketSystem', right: 'PacketSystem', *, names: 'tuple[str, ...] | None' = None) -> 'tuple[PacketSystem,
    ↪ PacketRefinement, PacketRefinement]'
```

Cards: C.1G common_refinement.

SIG-0345. 1 research card. Kinds: `function`.

```
(left: 'Sequence[int]', right: 'Sequence[int]', *, power: 'int', prime: 'int') -> 'ConvolutionPacketAudit'
```

Cards: C.3B audit_factorial_power_packet_convolution.

SIG-0346. 1 research card. Kinds: `function`.

```
(left: 'Sequence[int]', right: 'Sequence[int]', index: 'int', prime: 'int', factorial_power: 'int',
    ↪ modulus_exponent: 'int', *, budget: 'GeneratingBudget' = GeneratingBudget(maximum_degree=100000,
    ↪ maximum_coefficient_bits=4000000, maximum_matrix_dimension=4096, maximum_reachable_states=100000,
    ↪ maximum_local_combinations=1000000, maximum_modulus=2000000, maximum_serialized_bytes=67108864,
    ↪ maximum_recursion_depth=1000)) -> 'CarryLayerConvolution'
```

Cards: C.3B factorial_power_convolution_layers.

SIG-0347. 1 research card. Kinds: `function`.

```
(left: 'Sequence[int]', right: 'Sequence[int]', modulus: 'int') -> 'tuple[int, ...]'
```

Cards: C.3B polynomial_multiply_mod.

SIG-0349. 1 research card. Kinds: `function`.

```
(left: 'Sequence[Scalar]', right: 'Sequence[Scalar]') -> 'tuple[Vector, Vector, Vector]'
```

Cards: C.2C polynomial_extended_gcd.

SIG-0350. 1 research card. Kinds: `function`.

```
(left_factor: 'ExactPolynomial', right_factor: 'ExactPolynomial', measure: 'IntervalMeasure') -> 'IntegralIdentity'
```

Cards: C.1D integration_by_parts.

SIG-0351. 1 research card. Kinds: `function`.

```
(left_rank: 'int', right_rank: 'int') -> 'int'
```

Cards: C.1D tensor_seed_rank_upper_bound.

SIG-0352. 1 research card. Kinds: `function`.

```
(length: 'int', *, normalized: 'bool' = True) -> 'FiniteFilterBank'
```

Cards: C.4B haar_filter_bank.

SIG-0353. 1 research card. Kinds: `function`.

```
(length: 'int', *, normalized: 'bool' = True) -> 'np.ndarray'
```

Cards: C.4B haar_matrix.

SIG-0354. 1 research card. Kinds: `class`.

```
(length: 'int', coherent_gain: 'float', power_gain: 'float', equivalent_noise_bandwidth_bins: 'float',
    ↪ equivalent_noise_bandwidth_hz: 'float') -> None
```

Cards: C.4A WindowMetrics.

SIG-0355. 1 research card. Kinds: `function`.

```
(length: 'int', key: 'Callable[[int], object]', *, source: 'str' = 'original index order', target: 'str' =
    ↪ 'key-sorted index order') -> 'CoordinateTransform'
```

Cards: C.2A sorted_coordinate_transform.

SIG-0357. 1 research card. Kinds: `function`.

```
(length: 'int', modulus: 'int') -> 'tuple[int, ...]'
```

Cards: C.2A residue_class_order.

SIG-0358. 1 research card. Kinds: `function`.

```
(length: 'int', modulus: 'int', *, backend: 'str | LinearAlgebraBackend | None' = None, names: 'tuple[str, ...] |
   ↪ None' = None) -> 'PacketSystem'
```

Cards: C.1G residue_packet_system.

SIG-0359. 1 research card. Kinds: `function`.

```
(length: 'int', radix: 'int' = 2) -> 'CoordinateTransform'
```

Cards: C.2A digit_reversal_transform.

SIG-0360. 1 research card. Kinds: `function`.

```
(length: 'int', radix: 'int' = 2) -> 'tuple[int, ...]'
```

Cards: C.2A digit_reversal_order.

SIG-0361. 1 research card. Kinds: `class`.

```
(levels: 'Mapping[int, Iterable[Node]]', reductions: 'Mapping[int, Callable[[Node], Node] | Mapping[Node, Node]]')
   ↪ -> 'None'
```

Cards: C.1G FiniteLiftForest.

SIG-0362. 1 research card. Kinds: `class`.

```
(lifted_left: 'Any', lifted_right: 'Any', implicit_second_derivative: 'Any', reduced_second_derivative: 'Any',
   ↪ certificate: 'Certificate') -> None
```

Cards: C.5A SchurSecondDerivativeResult.

SIG-0364. 1 research card. Kinds: `class`.

```
(linearization: 'np.ndarray', kernel_basis: 'np.ndarray', complement_basis: 'np.ndarray', range_basis:
   ↪ 'np.ndarray', obstruction_basis: 'np.ndarray', rank: 'int', certificate: 'Certificate') -> None
```

Cards: C.5A LyapunovSchmidtDecomposition.

SIG-0365. 1 research card. Kinds: `function`.

```
(lipschitz_bound: 'float', interval_length: 'float') -> 'float'
```

Cards: C.4B smooth_haar_coefficient_bound.

SIG-0366. 1 research card. Kinds: `function`.

```
(lipschitz_constant: 'float', support_length: 'float') -> 'float'
```

Cards: C.4B haar_lipschitz_bound.

SIG-0368. 1 research card. Kinds: `function`.

```
(low: 'Any', high: 'Any', *, normalized: 'bool' = True) -> 'np.ndarray'
```

Cards: C.4B haar_synthesize.

SIG-0369. 1 research card. Kinds: `class`.

```
(lower: 'Any', upper: 'Any', *, density: 'Any' = 1, name: 'str' = 'interval measure') -> 'None'
```

Cards: C.1D IntervalMeasure.

SIG-0370. 1 research card. Kinds: `class`.

```
(lower_frame_bound: 'float', upper_frame_bound: 'float', reconstruction_residual: 'float', certificate:
    ↪ 'Certificate') -> None
```

Cards: C.4B FilterBankReport.

SIG-0371. 1 research card. Kinds: `function`.

```
(manifest: 'PublicApiManifest', path: 'str | Path', *, overwrite: 'bool' = False) -> 'Path'
```

Cards: C.1A save_public_api_manifest.

SIG-0372. 1 research card. Kinds: `function`.

```
(map_type: 'LinearMapType') -> 'ZeroMap'
```

Cards: C.1B zero_map.

SIG-0373. 1 research card. Kinds: `class`.

```
(map_type: 'LinearMapType') -> None
```

Cards: C.1B ZeroMap.

SIG-0374. 1 research card. Kinds: `function`.

```
(map_type: 'LinearMapType', matrix: 'Iterable[Iterable[Any]]', *, label: 'str' = '') -> 'ExactMapLiteral'
```

Cards: C.1B exact_map.

SIG-0375. 1 research card. Kinds: `class`.

```
(map_type: 'LinearMapType', matrix: 'Matrix', label: 'str' = '') -> None
```

Cards: C.1B ExactMapLiteral.

SIG-0377. 1 research card. Kinds: `function`.

```
(mask: 'Iterable[Scalar]', prime: 'int', degree: 'int', *, budget: 'GeneratingBudget' =
    ↪ GeneratingBudget(maximum_degree=100000, maximum_coefficient_bits=4000000, maximum_matrix_dimension=4096,
    ↪ maximum_reachable_states=100000, maximum_local_combinations=1000000, maximum_modulus=2000000,
    ↪ maximum_serialized_bytes=67108864, maximum_recursion_depth=1000)) -> 'tuple[Fraction, ...]'
```

Cards: C.3B frobenius_defect_mask.

SIG-0380. 1 research card. Kinds: `class`.

```
(mask: 'np.ndarray', amplitude: 'float', threshold: 'float', certificate: 'Certificate') -> None
```

Cards: C.4B PacketEstimate1D.

SIG-0381. 1 research card. Kinds: `function`.

```
(mask: 'Sequence[int | float | complex | Fraction]', point: 'complex', radix: 'int', terms: 'int') ->
    ↪ 'AnalyticMaskApproximation'
```

Cards: C.3B analytic_mahler_product.

SIG-0382. 1 research card. Kinds: `function`.

```
(mask: 'Sequence[int]', prime: 'int', *, budget: 'GeneratingBudget' = GeneratingBudget(maximum_degree=100000,
    ↪ maximum_coefficient_bits=4000000, maximum_matrix_dimension=4096, maximum_reachable_states=100000,
    ↪ maximum_local_combinations=1000000, maximum_modulus=2000000, maximum_serialized_bytes=67108864,
    ↪ maximum_recursion_depth=1000)) -> 'CartierCarryEquivalence'
```

Cards: C.3B certify_cartier_carry_equivalence.

SIG-0383. 1 research card. Kinds: `function`.

```
(mask: 'Sequence[int]', prime: 'int', *, budget: 'GeneratingBudget' = GeneratingBudget(maximum_degree=100000,
    ↪ maximum_coefficient_bits=4000000, maximum_matrix_dimension=4096, maximum_reachable_states=100000,
    ↪ maximum_local_combinations=1000000, maximum_modulus=2000000, maximum_serialized_bytes=67108864,
    ↪ maximum_recursion_depth=1000)) -> 'tuple[tuple[tuple[int, ...], ...], ...]'
```

Cards: C.3B finite_mask_carry_matrices.

SIG-0384. 1 research card. Kinds: `function`.

```
(mask: 'Sequence[int]', prime: 'int', degree: 'int') -> 'tuple[int, ...]'
```

Cards: C.3B first_frobenius_lift.

SIG-0385. 1 research card. Kinds: `function`.

```
(matrices: 'Sequence[Iterable[Iterable[Any]]]', plan: 'MatrixChainPlan') -> 'Matrix'
```

Cards: C.1B matrix_chain_multiply.

SIG-0386. 1 research card. Kinds: `function`.

```
(matrix: 'Any', exponent: 'float', *, tolerance: 'float' = 1e-12) -> 'np.ndarray'
```

Cards: C.1G fractional_power.

SIG-0387. 1 research card. Kinds: `function`.

```
(matrix: 'Any', layout: 'PacketLayout', *, absolute_tolerance: 'float' = 0.0, relative_tolerance: 'float' = 1e-12)
    ↪ -> 'SupportDiscoveryResult'
```

Cards: C.1D discover_packet_support.

SIG-0388. 1 research card. Kinds: `function`.

```
(matrix: 'Any', layout: 'PacketLayout', rank: 'int', *, tolerance: 'float' = 1e-10) -> 'BDLRDiscoveryResult'
```

Cards: C.1D discover_block_diagonal_low_rank.

SIG-0389. 1 research card. Kinds: `class`.

```
(matrix: 'Any', prime: 'int') -> None
```

Cards: C.1G AlternatingForm.

SIG-0390. 1 research card. Kinds: `class`.

```
(matrix: 'Any', retained: 'Iterable[int]') -> 'None'
```

Cards: C.1G FeshbachReduction.

SIG-0391. 1 research card. Kinds: `class`.

```
(matrix: 'Any', rhs: 'Any', *, prime: 'int', depth: 'int') -> 'None'
```

Cards: C.1G SmithCongruenceProblem.

SIG-0394. 1 research card. Kinds: `class`.

```
(matrix: 'Array') -> None
```

Cards: C.1D DenseOperator.

SIG-0395. 1 research card. Kinds: `class`.

```
(matrix: 'Iterable[Iterable[Rational]]', shift: 'Iterable[Rational]', *, name: 'str' = 'exact affine map') ->
    ↪ 'None'
```

Cards: C.1D ExactAffineMap.

SIG-0396. 1 research card. Kinds: `function`.

```
(matrix: 'Matrix') -> 'Fraction'
```

Cards: C.2A determinant.

SIG-0397. 1 research card. Kinds: `function`.

```
(matrix: 'Matrix') -> 'int'
```

Cards: C.2A matrix_rank.

SIG-0399. 1 research card. Kinds: `function`.

```
(matrix: 'Matrix') -> 'tuple[int, int]'
```

Cards: C.2A matrix_shape.

SIG-0400. 1 research card. Kinds: `function`.

```
(matrix: 'Matrix') -> 'tuple[Matrix, tuple[int, ...]]'
```

Cards: C.2A rref.

SIG-0401. 1 research card. Kinds: `function`.

```
(matrix: 'Matrix') -> 'tuple[Vector, ...]'
```

Cards: C.2A nullspace.

SIG-0402. 1 research card. Kinds: `function`.

```
(matrix: 'Matrix', exponent: 'int') -> 'Matrix'
```

Cards: C.2A matrix_power.

SIG-0403. 1 research card. Kinds: `function`.

```
(matrix: 'Matrix', right_hand_side: 'Sequence[Scalar]') -> 'ExactLinearSystem'
```

Cards: C.2A solve_linear_system.

SIG-0404. 1 research card. Kinds: `function`.

```
(matrix: 'Matrix', vector: 'Sequence[Scalar]') -> 'Vector'
```

Cards: C.2A matrix_vector_multiply.

SIG-0405. 1 research card. Kinds: `function`.

```
(max_height: 'int', arities: 'Sequence[int]') -> 'int'
```

Cards: C.5B ordered_tree_count.

SIG-0406. 1 research card. Kinds: `function`.

```
(max_height: 'int', arities: 'Sequence[int]', *, max_trees: 'int' = 100000) -> 'tuple[OrderedRootedTree, ...]'
```

Cards: C.5B ordered_rooted_trees.

SIG-0407. 1 research card. Kinds: `class`.

```
(max_nodes: 'int' = 10000, max_depth: 'int' = 1000, max_steps: 'int' = 50000, max_degree: 'int' = 512, max_entries:
   ↪ 'int' = 1000000, max_coefficient_bits: 'int' = 1000000, max_serialized_bytes: 'int' = 64000000,
   ↪ max_symbol_length: 'int' = 1024) -> None
```

Cards: C.1B ExpressionBudget.

SIG-0412. 1 research card. Kinds: `function`.

```
(model: 'StateSpaceModel') -> 'FiniteBasis'
```

Cards: C.2C controllability_basis.

SIG-0413. 1 research card. Kinds: `function`.

```
(model: 'StateSpaceModel', transform: 'CoordinateTransform', *, markov_count: 'int | None' = None) ->
   ↪ 'StateSpaceCoordinateCertificate'
```

Cards: C.2C certify_state_coordinate_change.

SIG-0414. 1 research card. Kinds: `function`.

```
(moduli: 'Sequence[int]') -> 'CoordinateTransform'
```

Cards: C.2A chinese_remainder_transform.

SIG-0415. 1 research card. Kinds: `function`.

```
(moduli: 'Sequence[int]') -> 'tuple[int, ...]'
```

Cards: C.2A chinese_remainder_order.

SIG-0416. 1 research card. Kinds: `class`.

```
(modulus: 'int', name: 'str' = 'prime_field') -> None
```

Cards: C.3A PrimeFieldDomain.

SIG-0417. 1 research card. Kinds: `class`.

```
(modulus: 'int', name: 'str' = 'residue_ring') -> None
```

Cards: C.3A ResidueRingDomain.

SIG-0418. 1 research card. Kinds: `class`.

```
(modulus: 'int', residue: 'int', source_basis: 'str', target_basis: 'str', source_degree: 'int', packet_degree:
   ↪  'int', valid: 'bool') -> None
```

Cards: C.3A PacketIdentity.

SIG-0419. 1 research card. Kinds: `function`.

```
(multiplier: 'int', shift: 'int', source_degree: 'int', target_degree: 'int') -> 'int'
```

Cards: C.1G affine_mahler_kernel.

SIG-0422. 1 research card. Kinds: `class`.

```
(name: 'str' = 'integers', zero: 'int' = 0, one: 'int' = 1) -> None
```

Cards: C.3A IntegerDomain.

SIG-0423. 1 research card. Kinds: `class`.

```
(name: 'str' = 'rationals', zero: 'Fraction' = Fraction(0, 1), one: 'Fraction' = Fraction(1, 1)) -> None
```

Cards: C.3A RationalDomain.

SIG-0426. 1 research card. Kinds: `function`.

```
(name: 'str') -> 'Sequence'
```

Cards: C.1G get_sequence.

SIG-0427. 1 research card. Kinds: `function`.

```
(name: 'str', *, since: 'str', replacement: 'str | None' = None, remove_in: 'str | None' = None, reason: 'str |
   ↪  None' = None, stacklevel: 'int' = 2) -> 'None'
```

Cards: C.1A warn_deprecated.

SIG-0429. 1 research card. Kinds: `function`.

```
(name: 'str', basis: 'WeightedMonomialBasis', coefficients: 'tuple[int | Fraction, ...]') -> 'CustomFunctionSpec'
```

Cards: C.3A explicit_function.

SIG-0431. 1 research card. Kinds: `function`.

```
(name: 'str', coefficients: 'tuple[int | Fraction, ...]', initial: 'tuple[int | Fraction, ...]', *, basis:
   ↪ 'WeightedMonomialBasis | None' = None) -> 'CustomFunctionSpec'
```

Cards: C.3A linear_recurrence_function.

SIG-0435. 1 research card. Kinds: `class`.

```
(name: 'str', cost: 'float', error_units: 'int', dtype: 'str | None' = None) -> None
```

Cards: C.1D DTypeChoice.

SIG-0436. 1 research card. Kinds: `function`.

```
(name: 'str', declared_type: 'ExpressionType') -> 'Symbol'
```

Cards: C.1B symbol.

SIG-0437. 1 research card. Kinds: `class`.

```
(name: 'str', declared_type: 'ExpressionType') -> None
```

Cards: C.1B Symbol.

SIG-0438. 1 research card. Kinds: `class`.

```
(name: 'str', degree: 'int', nonzero_normalization: 'bool', integral_structure_constants: 'bool',
   ↪ maximum_denominator: 'int', maximum_numerator_bits: 'int') -> None
```

Cards: C.3A BasisCertificate.

SIG-0440. 1 research card. Kinds: `class`.

```
(name: 'str', duration: 'float' = 1.0, dependencies: 'tuple[str, ...]' = (), shape_key: 'Hashable | None' = None)
   ↪ -> None
```

Cards: C.1D Task.

SIG-0441. 1 research card. Kinds: `function`.

```
(name: 'str', exponents: 'Mapping[int, int | Fraction] | Callable[[int], int | Fraction]', *, basis:
   ↪ 'WeightedMonomialBasis | None' = None) -> 'CustomFunctionSpec'
```

Cards: C.3A euler_product_function.

SIG-0443. 1 research card. Kinds: `function`.

```
(name: 'str', initial: 'int | Fraction', ratio: 'Callable[[int], int | Fraction]', *, basis:
   ↪ 'WeightedMonomialBasis | None' = None) -> 'CustomFunctionSpec'
```

Cards: C.3A hypergeometric_function.

SIG-0444. 1 research card. Kinds: `class`.

```
(name: 'str', keep_cost: 'float', keep_error: 'float', split_cost: 'float' = 0.0, children: "tuple['TreeNode',
   ↪ ...]" = ()) -> None
```

Cards: C.1D TreeNode.

SIG-0445. 1 research card. Kinds: `class`.

```
(name: 'str', kind: 'str', observed: 'Any', expected: 'Any', certificate: 'Certificate') -> None
```

Cards: C.1A BehavioralGoldenCaseResult.

SIG-0446. 1 research card. Kinds: `class`.

```
(name: 'str', kind: 'str', parameters: 'tuple[tuple[str, BasisParameter], ...]' = ()) -> None
```

Cards: C.3A BasisDescriptor.

SIG-0447. 1 research card. Kinds: `class`.

```
(name: 'str', left: 'Any', right: 'Any', certificate: 'Certificate') -> None
```

Cards: C.1D IntegralIdentity.

SIG-0449. 1 research card. Kinds: `function`.

```
(name: 'str', numerator: 'tuple[int | Fraction, ...]', denominator: 'tuple[int | Fraction, ...]', *, basis:
   ↪ 'WeightedMonomialBasis | None' = None) -> 'CustomFunctionSpec'
```

Cards: C.3A rational_function.

SIG-0451. 1 research card. Kinds: `function`.

```
(name: 'str', outer: 'CustomFunctionSpec', inner: 'CustomFunctionSpec') -> 'CustomFunctionSpec'
```

Cards: C.3A compose_functions.

SIG-0452. 1 research card. Kinds: `function`.

```
(name: 'str', polynomial_terms: 'Mapping[tuple[int, int], int | Fraction]', initial_value: 'int | Fraction', *,
   ↪ basis: 'WeightedMonomialBasis | None' = None) -> 'CustomFunctionSpec'
```

Cards: C.3A algebraic_function.

SIG-0455. 1 research card. Kinds: `class`.

```
(name: 'str', source: 'str', target: 'str') -> None
```

Cards: C.1D Arrow.

SIG-0457. 1 research card. Kinds: `class`.

```
(name: 'str', terms: 'tuple[tuple[Any, OperatorWord], ...]') -> None
```

Cards: C.1D OperatorRelation.

SIG-0459. 1 research card. Kinds: `function`.

```
(name: 'str', values: 'Iterable[int | Fraction]') -> 'WeightedMonomialBasis'
```

Cards: C.3A basis_from_values.

SIG-0462. 1 research card. Kinds: `class`.

```
(name: 'str', vertices: 'tuple[Vector, ...]') -> None
```

Cards: C.2C BarycentricCoordinateSystem.

SIG-0463. 1 research card. Kinds: `class`.

```
(namespace: 'str', name: 'str', kind: 'str', implementation_module: 'str', qualname: 'str', signature: 'str |
    ↪ None') -> None
```

Cards: C.1A PublicApiSymbol.

SIG-0464. 1 research card. Kinds: `function`.

```
(namespaces: 'Sequence[str]' = ('komorebi_math', 'komorebi_math.nonlinear', 'komorebi_math.export',
    ↪ 'komorebi_math.signals', 'komorebi_math.generating', 'komorebi_math.coordinates'), *, package_version: 'str
    ↪ | None' = None) -> 'PublicApiManifest'
```

Cards: C.1A create_public_api_manifest.

SIG-0465. 1 research card. Kinds: `class`.

```
(nodes: 'int', depth: 'int', steps: 'int', exact_entries: 'int', maximum_degree: 'int', maximum_coefficient_bits:
    ↪ 'int', serialized_bytes: 'int') -> None
```

Cards: C.1B ExpressionStats.

SIG-0467. 1 research card. Kinds: `function`.

```
(noise_bound: 'float', order: 'int') -> 'float'
```

Cards: C.4B difference_noise_bound.

SIG-0468. 1 research card. Kinds: `class`.

```
(normal_form: 'NormalForm', direct_normal_form: 'NormalForm', plan: 'ExpressionPlan', certificate: 'Certificate')
    ↪ -> None
```

Cards: C.1B ExpressionResult.

SIG-0469. 1 research card. Kinds: `class`.

```
(normalized: 'bool' = True) -> None
```

Cards: C.4B HaarFilterBank.

SIG-0470. 1 research card. Kinds: `class`.

```
(numerator: 'Any', denominator: 'Any' = (1.0,)) -> 'None'
```

Cards: C.4A LinearFilter.

SIG-0471. 1 research card. Kinds: `function`.

```
(numerator: 'Any', denominator: 'Any', signal: 'Any', *, initial_state: 'Any | None' = None) -> 'FilterOutput'
```

Cards: C.4A lfilter.

SIG-0472. 1 research card. Kinds: `function`.

```
(numerator: 'Iterable[Scalar]', denominator: 'Iterable[Scalar]', radix: 'int', degree_bound: 'int', *, budget:
    ↪ 'GeneratingBudget' = GeneratingBudget(maximum_degree=100000, maximum_coefficient_bits=4000000,
    ↪ maximum_matrix_dimension=4096, maximum_reachable_states=100000, maximum_local_combinations=1000000,
    ↪ maximum_modulus=2000000, maximum_serialized_bytes=67108864, maximum_recursion_depth=1000)) ->
    ↪ 'PolynomialCoboundaryResult'
```

Cards: C.3B solve_polynomial_coboundary.

SIG-0473. 1 research card. Kinds: `function`.

```
(numerator: 'Sequence[int]', denominator: 'Sequence[int]', prime: 'int') -> 'int'
```

Cards: C.3B cartier_degree_bound.

SIG-0474. 1 research card. Kinds: `function`.

```
(numerator: 'Sequence[int]', denominator: 'Sequence[int]', prime: 'int', *, budget: 'GeneratingBudget' =
    ↪ GeneratingBudget(maximum_degree=100000, maximum_coefficient_bits=4000000, maximum_matrix_dimension=4096,
    ↪ maximum_reachable_states=100000, maximum_local_combinations=1000000, maximum_modulus=2000000,
    ↪ maximum_serialized_bytes=67108864, maximum_recursion_depth=1000)) -> 'CartierLinearRepresentation'
```

Cards: C.3B compile_cartier_linear_representation.

SIG-0476. 1 research card. Kinds: `function`.

```
(numerator: 'Sequence[int]', denominator: 'Sequence[int]', prime: 'int', count: 'int') -> 'tuple[int, ...]'
```

Cards: C.3B rational_mask_coefficients_mod_prime.

SIG-0477. 1 research card. Kinds: `function`.

```
(numerator: 'Sequence[int]', denominator: 'Sequence[int]', prime: 'int', degree: 'int', *, budget:
    ↪ 'GeneratingBudget' = GeneratingBudget(maximum_degree=100000, maximum_coefficient_bits=4000000,
    ↪ maximum_matrix_dimension=4096, maximum_reachable_states=100000, maximum_local_combinations=1000000,
    ↪ maximum_modulus=2000000, maximum_serialized_bytes=67108864, maximum_recursion_depth=1000)) ->
    ↪ 'RationalMaskAlgebraicCertificate'
```

Cards: C.3B certify_rational_mask_algebraic_relation.

SIG-0478. 1 research card. Kinds: `class`.

```
(numerator: 'tuple[int, ...]', denominator: 'tuple[int, ...]', fit_relations: 'int', holdout_relations: 'int',
    ↪ exact_on_prefix: 'bool') -> None
```

Cards: C.3A HypergeometricGuess.

SIG-0479. 1 research card. Kinds: `function`.

```
(obj: 'Any') -> 'ProvenanceRecord | None'
```

Cards: C.1A describe.

SIG-0480. 1 research card. Kinds: `class`.

```
(observables: 'Mapping[str, Any] | Sequence[Any]', seeds: 'Sequence[Any]', *, tolerance: 'float' = 1e-10) -> 'None'
```

Cards: C.1G QuantumSeedSystem.

SIG-0481. 1 research card. Kinds: `class`.

```
(obstruction: 'np.ndarray', obstruction_norm: 'float', vanishes: 'bool', least_norm_correction: 'np.ndarray |
    ↪ None', correction_residual: 'float', certificate: 'Certificate') -> None
```

Cards: C.5A HilbertObstructionResult.

SIG-0482. 1 research card. Kinds: `function`.

```
(offset: 'Sequence[Scalar]', *, source: 'str' = 'original affine coordinates', target: 'str' = 'translated affine
   ↪ coordinates') -> 'AffineCoordinateTransform'
```

Cards: C.2C translation_transform.

SIG-0483. 1 research card. Kinds: `class`.

```
(opcode: 'str', inputs: 'tuple[int, ...]', expression_type: 'CoordinateSpace | LinearMapType', payload: 'Any',
   ↪ scalar_operation_cost: 'int', certificate: 'Certificate') -> None
```

Cards: C.1B PlanInstruction.

SIG-0484. 1 research card. Kinds: `function`.

```
(operation: 'str', *, theorem_id: 'str | None' = None, assumptions_checked: 'Iterable[str]' = (), diagnostics:
   ↪ 'Mapping[str, Any] | None' = None) -> 'Certificate'
```

Cards: C.1B exact_internal_certificate.

SIG-0485. 1 research card. Kinds: `class`.

```
(operator: 'Any | FiniteMarkovChain | ContinuousTimeMarkovChain', partition: 'Sequence[int] |
   ↪ Iterable[Iterable[int]]', *, kind: 'MarkovKind | None' = None) -> 'None'
```

Cards: C.1G MarkovLumping.

SIG-0486. 1 research card. Kinds: `function`.

```
(operator: 'Any') -> 'int'
```

Cards: C.1D single_operator_seed_rank.

SIG-0487. 1 research card. Kinds: `function`.

```
(operator: 'Any', *, state_name: 'str' = 'state coordinates', modal_name: 'str' = 'modal coordinates', tolerance:
   ↪ 'float' = 1e-10) -> 'ModalCoordinateSystem'
```

Cards: C.2C modal_coordinate_system.

SIG-0488. 1 research card. Kinds: `function`.

```
(operator: 'Any', time: 'float') -> 'float'
```

Cards: C.1G spectral_dimension.

SIG-0489. 1 research card. Kinds: `function`.

```
(operator: 'Expression', argument: 'Expression') -> 'Apply'
```

Cards: C.1B apply.

SIG-0490. 1 research card. Kinds: `class`.

```
(operator: 'Expression', argument: 'Expression') -> None
```

Cards: C.1B Apply.

SIG-0491. 1 research card. Kinds: `function`.

```
(operator: 'Iterable[Iterable[Scalar]]', domain_transform: 'CoordinateTransform', codomain_transform:
    ↪ 'CoordinateTransform') -> 'Matrix'
```

Cards: C.2A transport_rectangular_operator.

SIG-0492. 1 research card. Kinds: `function`.

```
(operator: 'Iterable[Iterable[Scalar]]', transform: 'CoordinateTransform') -> 'Matrix'
```

Cards: C.2A transport_operator.

SIG-0493. 1 research card. Kinds: `function`.

```
(operator: 'PacketOperator', *, source: 'int', target: 'int', length: 'int', tolerance: 'float | None' = None) ->
    ↪ 'ComputationResult[Any]'
```

Cards: C.1D block_from_paths.

SIG-0494. 1 research card. Kinds: `function`.

```
(operator: 'PacketOperator', *, source: 'int', target: 'int', length: 'int', tolerance: 'float | None' = None) ->
    ↪ 'tuple[TransferPath, ...]'
```

Cards: C.1D path_contributions.

SIG-0495. 1 research card. Kinds: `function`.

```
(operator: 'PolynomialCoordinateOperator', source_family: 'PolynomialFamily', target_family: 'PolynomialFamily',
    ↪ *, budget: 'CoordinateBudget' = CoordinateBudget(maximum_matrix_dimension=512,
    ↪ maximum_polynomial_degree=512, maximum_horizon=4096, maximum_exact_entries=1000000)) ->
    ↪ 'PolynomialCoordinateOperator'
```

Cards: C.2B transport_polynomial_operator.

SIG-0497. 1 research card. Kinds: `function`.

```
(operators: 'Mapping[str, Any] | Sequence[Any]', seeds: 'Sequence[Any]', *, backend: 'str | LinearAlgebraBackend |
    ↪ None' = None, tolerance: 'float | None' = None) -> 'ComputationResult[Any]'
```

Cards: C.1D cyclic_span.

SIG-0498. 1 research card. Kinds: `class`.

```
(operators: 'Mapping[str, Any] | Sequence[Any]', seeds: 'Sequence[Any]', *, backend: 'str | LinearAlgebraBackend |
    ↪ None' = None, tolerance: 'float | None' = None) -> 'None'
```

Cards: C.1D SeedSystem.

SIG-0499. 1 research card. Kinds: `function`.

```
(options: 'Sequence[Sequence[DTypeChoice]]', budget: 'int') -> 'PrecisionAssignment | None'
```

Cards: C.1D assign_discrete_precisions.

SIG-0500. 1 research card. Kinds: `class`.

```
(orbit: 'FrobeniusOrbit', frobenius_scalar: 'complex' = 1.0) -> None
```

Cards: C.1G TameWeilPacket.

SIG-0501. 1 research card. Kinds: `function`.

```
(order: 'int') -> 'FiniteGroup[int]'
```

Cards: C.1G cyclic_group.

SIG-0502. 1 research card. Kinds: `data`.

```
(order: 'int') -> 'tuple[Partition, ...]'
```

Cards: C.5A set_partitions.

SIG-0503. 1 research card. Kinds: `class`.

```
(order: 'int') -> None
```

Cards: C.1G CyclicDenominator.

SIG-0505. 1 research card. Kinds: `function`.

```
(order: 'int', blocks: 'int') -> 'int'
```

Cards: C.5A stirling_second.

SIG-0506. 1 research card. Kinds: `class`.

```
(order: 'int', bound: 'Fraction | float', certificate: 'Certificate', description: 'str' = 'derivative supremum
   ↪ bound', polynomial: 'ExactPolynomial | None' = None, interval: 'tuple[Fraction, Fraction] | None' = None) ->
   ↪ None
```

Cards: C.1D DerivativeBound.

SIG-0507. 1 research card. Kinds: `class`.

```
(order: 'int', coefficients: 'tuple[Fraction, ...]', fit_terms: 'int', holdout_terms: 'int', exact_on_prefix:
   ↪ 'bool') -> None
```

Cards: C.3A LinearRecurrenceGuess.

SIG-0508. 1 research card. Kinds: `function`.

```
(order: 'int', count: 'int', *, start: 'int' = 0, tolerance: 'float' = 1e-12, max_iterations: 'int' = 64) ->
   ↪ 'ZeroStringResult'
```

Cards: C.1G positive_zero_string.

SIG-0509. 1 research card. Kinds: `function`.

```
(order: 'int', degree: 'int', parameter: 'Number') -> 'Number'
```

Cards: C.1G theta_polynomial.

SIG-0510. 1 research card. Kinds: `class`.

```
(order: 'int', degree: 'int', polynomials: 'tuple[tuple[int, ...], ...]', fit_relations: 'int', holdout_relations:
   ↪ 'int', exact_on_prefix: 'bool') -> None
```

Cards: C.3A PRecursiveGuess.

SIG-0511. 1 research card. Kinds: `class`.

```
(order: 'int', family: "Literal['transfer', 'signed']" = 'signed', scale: 'complex' = 1.0) -> None
```

Cards: C.1G CyclicSectionFamily.

SIG-0512. 1 research card. Kinds: `function`.

```
(order: 'int', intermediate_packets: 'int') -> 'int'
```

Cards: C.5A labeled_composition_term_count.

SIG-0513. 1 research card. Kinds: `class`.

```
(order: 'int', nonzero: 'tuple[PacketJetAddress, ...]', zero: 'tuple[PacketJetAddress, ...]', untested:
   ↪ 'tuple[PacketJetAddress, ...]', evaluations: 'int', certificate: 'Certificate') -> None
```

Cards: C.5B PacketJetSupport.

SIG-0514. 1 research card. Kinds: `function`.

```
(order: 'int', parameter: 'Number', degree: 'int') -> 'tuple[Number, ...]'
```

Cards: C.1G reciprocal_coefficients.

SIG-0515. 1 research card. Kinds: `function`.

```
(order: 'int', residue: 'int', parameter: 'Number', degree: 'int', x: 'Number') -> 'Number'
```

Cards: C.1G appell_polynomial.

SIG-0516. 1 research card. Kinds: `function`.

```
(order: 'int', residue: 'int', z: 'complex', *, scale: 'complex' = 1.0, tolerance: 'float' = 1e-15, max_terms:
   ↪ 'int' = 512) -> 'complex'
```

Cards: C.1G transfer_section.

SIG-0517. 1 research card. Kinds: `function`.

```
(order: 'int', residue: 'int', z: 'complex', *, tolerance: 'float' = 1e-15, max_terms: 'int' = 512) -> 'complex'
```

Cards: C.1G signed_section.

SIG-0522. 1 research card. Kinds: `function`.

```
(origin: 'Sequence[Scalar]', basis_vectors: 'Sequence[Sequence[Scalar]]', *, local_name: 'str' = 'local affine
   ↪ coordinates', ambient_name: 'str' = 'ambient Cartesian coordinates') -> 'AffineCoordinateTransform'
```

Cards: C.2C affine_basis_transform.

SIG-0526. 1 research card. Kinds: `function`.

```
(outer: 'Sequence[Scalar]', inner: 'Sequence[Scalar]', degree: 'int') -> 'Vector'
```

Cards: C.2C compose_ordinary_series.

SIG-0527. 1 research card. Kinds: `class`.

```
(output: 'Any', hidden: 'Any', input: 'Any') -> 'None'
```

Cards: C.1G MemoryKernel.

SIG-0529. 1 research card. Kinds: `class`.

```
(output: 'np.ndarray', error: 'np.ndarray', final_weights: 'np.ndarray', weight_history: 'np.ndarray', method:
    ↪ 'AdaptiveMethod', certificate: 'Certificate') -> None
```

Cards: C.4B AdaptiveFilterResult.

SIG-0530. 1 research card. Kinds: `class`.

```
(output: 'str', inputs: 'tuple[str, ...]') -> None
```

Cards: C.5B PacketJetAddress.

SIG-0531. 1 research card. Kinds: `class`.

```
(output_label: 'str', value: 'Any', direct_value: 'Any', terms: 'tuple[GradedConvolutionTerm, ...]', certificate:
    ↪ 'Certificate') -> None
```

Cards: C.5A GradedConvolutionResult.

SIG-0533. 1 research card. Kinds: `class`.

```
(package_version: 'str', namespaces: 'tuple[str, ...]', symbols: 'tuple[PublicApiSymbol, ...]', schema: 'str' =
    ↪ 'komorebi-math.public-api', schema_version: 'int' = 1, stability_level: 'str' = 'pre-beta-additive') -> None
```

Cards: C.1A PublicApiManifest.

SIG-0534. 1 research card. Kinds: `class`.

```
(packet_system: 'PacketSystem', diagonal: 'Any', edges: 'tuple[WeightedTransferEdge, ...] |
    ↪ list[WeightedTransferEdge]', *, backend: 'str | LinearAlgebraBackend | None' = None, prime: 'int | None' =
    ↪ None, validate_weights: 'bool' = True) -> 'None'
```

Cards: C.1G WeightedTransferGraph.

SIG-0535. 1 research card. Kinds: `class`.

```
(page: 'int', entries: 'tuple[SpectralSequenceEntry, ...]', total_dimension: 'int', total_differential_rank:
    ↪ 'int', certificate: 'Certificate') -> None
```

Cards: C.5A SpectralSequencePage.

SIG-0536. 1 research card. Kinds: `class`.

```
(page: 'int', filtration_degree: 'int', complementary_degree: 'int', total_degree: 'int', dimension: 'int',
    ↪ outgoing_rank: 'int', incoming_rank: 'int') -> None
```

Cards: C.5A SpectralSequenceEntry.

SIG-0537. 1 research card. Kinds: `class`.

```
(pages: 'tuple[SpectralSequencePage, ...]', total_cohomology_dimensions: 'dict[int, int]', limiting_dimensions:
    ↪ 'dict[int, int]', certificate: 'Certificate') -> None
```

Cards: C.5A FilteredSpectralSequenceResult.

SIG-0538. 1 research card. Kinds: `class`.

```
(parent_order: 'int', child_order: 'int', parent_degree: 'int', child_degree: 'int', mode: 'CliffordMode | str',
    ↪ attached_descent: 'bool' = False, label: 'str' = '') -> None
```

Cards: C.1G CliffordStep.

SIG-0539. 1 research card. Kinds: `class`.

```
(partition: 'Partition', intermediate_packets: 'tuple[str, ...]', value: 'Any', pruned: 'bool' = False, reason:
    ↪ 'str' = '') -> None
```

Cards: C.5A FaaDiBrunoTerm.

SIG-0540. 1 research card. Kinds: `class`.

```
(patched_object_exists: 'bool', patched_full_support: 'bool', finite_level_descent: 'bool',
    ↪ comparison_preserves_support: 'bool', characteristic_zero_fiber_nonzero: 'bool', derived: 'bool' = False) ->
    ↪ None
```

Cards: C.1G PatchingSupportReport.

SIG-0541. 1 research card. Kinds: `class`.

```
(path: 'Path', caption: 'str | None' = None, label: 'str | None' = None, width: 'str' = '0.9\\textwidth') -> None
```

Cards: C.6 Figure.

SIG-0548. 1 research card. Kinds: `function`.

```
(path: 'str | Path | None' = None) -> 'ApiCompatibilityReport'
```

Cards: C.1A check_beta_api.

SIG-0549. 1 research card. Kinds: `function`.

```
(path: 'str | Path | None' = None) -> 'BehavioralGoldenReport'
```

Cards: C.1A run_behavioral_golden_fixture.

SIG-0551. 1 research card. Kinds: `function`.

```
(path: 'str | Path | None' = None) -> 'tuple[Path, dict[str, Any], str]'
```

Cards: C.1A load_behavioral_golden_fixture.

SIG-0552. 1 research card. Kinds: `function`.

```
(path: 'str | Path | None' = None, *, strict_signatures: 'bool' = False) -> 'ApiCompatibilityReport'
```

Cards: C.1A check_public_api.

SIG-0554. 1 research card. Kinds: `function`.

```
(path: 'str | Path') -> 'CoordinateTransform'
```

Cards: C.2C load_transform.

SIG-0555. 1 research card. Kinds: `function`.

```
(path: 'str | Path') -> 'ExportFormat'
```

Cards: C.6 infer_export_format.

SIG-0556. 1 research card. Kinds: `function`.

```
(path: 'str | Path') -> 'RecurrenceCoordinateRecord'
```

Cards: C.2C load_coordinate_record.

SIG-0557. 1 research card. Kinds: `function`.

```
(path: 'str | Path', record: 'RecurrenceCoordinateRecord', *, overwrite: 'bool' = False) -> 'Path'
```

Cards: C.2C save_coordinate_record.

SIG-0558. 1 research card. Kinds: `function`.

```
(path: 'str | Path', transform: 'CoordinateTransform', *, overwrite: 'bool' = False) -> 'Path'
```

Cards: C.2C save_transform.

SIG-0560. 1 research card. Kinds: `function`.

```
(payload: 'Any') -> 'CoefficientDomain[Any]'
```

Cards: C.3A domain_from_descriptor.

SIG-0561. 1 research card. Kinds: `class`.

```
(period: 'int', degree: 'int', residue_values: 'tuple[tuple[Fraction, ...], ...]', fit_terms: 'int',
    ↪ holdout_terms: 'int', exact_on_prefix: 'bool') -> None
```

Cards: C.3A QuasiPolynomialGuess.

SIG-0562. 1 research card. Kinds: `function`.

```
(permutation: 'Sequence[int]', *, source: 'str' = 'original order', target: 'str' = 'permuted order') ->
    ↪ 'CoordinateTransform'
```

Cards: C.2C permutation_transform.

SIG-0563. 1 research card. Kinds: `function`.

```
(plan: 'ExpressionPlan', *, budget: 'ExpressionBudget | None' = None) -> 'NormalForm'
```

Cards: C.1B execute_plan.

SIG-0564. 1 research card. Kinds: `function`.

```
(plan: 'MatrixChainPlan', labels: 'Sequence[str] | None' = None) -> 'str'
```

Cards: C.1B matrix_chain_parenthesization.

SIG-0566. 1 research card. Kinds: `function`.

```
(point: 'float', cell_length: 'float') -> 'float'
```

Cards: C.4B half_shift_distance.

SIG-0567. 1 research card. Kinds: `class`.

```
(points: 'Iterable[Any]', weights: 'Iterable[Any]', *, name: 'str' = 'atomic measure') -> 'None'
```

Cards: C.1D AtomicMeasure.

SIG-0568. 1 research card. Kinds: `class`.

```
(points: 'Iterable[Any]', weights: 'Iterable[Any]', *, name: 'str' = 'exact quadrature rule') -> 'None'
```

Cards: C.1D ExactQuadratureRule.

SIG-0569. 1 research card. Kinds: `class`.

```
(points: 'Iterable[Any]', weights: 'Iterable[Any]', *, name: 'str' = 'quadrature rule', _require_exact: 'bool' =
   ↪ False) -> 'None'
```

Cards: C.1D QuadratureRule.

SIG-0570. 1 research card. Kinds: `function`.

```
(polynomial: 'ExactPolynomial', base: 'IntervalMeasure', mapping: 'ExactAffineMap') -> 'IntegralIdentity'
```

Cards: C.1D affine_interval_change_of_variables.

SIG-0571. 1 research card. Kinds: `function`.

```
(polynomial: 'ExactPolynomial', base: 'Measure', mapping: 'ExactAffineMap') -> 'IntegralIdentity'
```

Cards: C.1D pushforward_identity.

SIG-0572. 1 research card. Kinds: `function`.

```
(polynomial: 'ExactPolynomial', lower: 'int | Fraction', upper: 'int | Fraction', *, density: 'int | Fraction' = 1)
   ↪ -> 'IntegralResult'
```

Cards: C.1D integrate_polynomial_oriented.

SIG-0573. 1 research card. Kinds: `function`.

```
(polynomial: 'ExactPolynomial', measure: 'IntervalMeasure') -> 'IntegralIdentity'
```

Cards: C.1D fundamental_theorem.

SIG-0574. 1 research card. Kinds: `function`.

```
(polynomial: 'ExactPolynomial', measure: 'IntervalMeasure', *, order: 'int' = 4) -> 'DerivativeBound'
```

Cards: C.1D polynomial_derivative_bound.

SIG-0575. 1 research card. Kinds: `function`.

```
(polynomial: 'ExactPolynomial', measure: 'ProductMeasure', *, max_dimensions: 'int' = 7) ->
   ↪ 'tuple[IntegralIdentity, ...]'
```

Cards: C.1D all_fubini_orders.

SIG-0576. 1 research card. Kinds: `function`.

```
(polynomial: 'ExactPolynomial', measure: 'ProductMeasure', *, order: 'tuple[int, ...] | None' = None) ->
   ↪  'IntegralIdentity'
```

Cards: C.1D fubini_identity.

SIG-0577. 1 research card. Kinds: `function`.

```
(polynomial: 'Sequence[int]', digit: 'int', prime: 'int') -> 'tuple[int, ...]'
```

Cards: C.3B polynomial_cartier.

SIG-0578. 1 research card. Kinds: `function`.

```
(polynomial: 'Sequence[int]', exponent: 'int', modulus: 'int') -> 'tuple[int, ...]'
```

Cards: C.3B polynomial_power_mod.

SIG-0579. 1 research card. Kinds: `function`.

```
(polynomial: 'Sequence[Scalar]') -> 'Polynomial'
```

Cards: C.2B polynomial_derivative.

SIG-0580. 1 research card. Kinds: `function`.

```
(polynomial: 'Sequence[Scalar]', *, scale: 'Scalar' = 1, shift: 'Scalar' = 0) -> 'Polynomial'
```

Cards: C.2B polynomial_compose_affine.

SIG-0581. 1 research card. Kinds: `function`.

```
(polynomial: 'Sequence[Scalar]', constant: 'Scalar' = 0) -> 'Polynomial'
```

Cards: C.2B polynomial_integral.

SIG-0582. 1 research card. Kinds: `function`.

```
(polynomial: 'Sequence[Scalar]', scalar: 'Scalar') -> 'Polynomial'
```

Cards: C.2B polynomial_scale.

SIG-0583. 1 research card. Kinds: `function`.

```
(polynomial: 'Sequence[Scalar]', steps: 'int' = 1) -> 'Polynomial'
```

Cards: C.2B polynomial_shift.

SIG-0584. 1 research card. Kinds: `function`.

```
(polynomial: 'Sequence[Scalar]', value: 'Scalar') -> 'Fraction'
```

Cards: C.2B polynomial_evaluate.

SIG-0585. 1 research card. Kinds: `class`.

```
(polynomials: 'Iterable[Iterable[Rational]]', initial: 'Iterable[Rational]', *, name: 'str' = 'holonomic
   ↪  recurrence') -> 'None'
```

Cards: C.1G HolonomicRecurrence.

SIG-0586. 1 research card. Kinds: `class`.

```
(polynomials: 'Mapping[Hashable, tuple[Any, ...]]', local_labels: 'Mapping[Hashable, Any] | None' = None) -> None
```

Cards: C.1G FrobeniusSpine.

SIG-0587. 1 research card. Kinds: `function`.

```
(power: 'int') -> 'tuple[tuple[Polynomial, Polynomial], tuple[Polynomial, Polynomial]]'
```

Cards: C.2B fibonacci_companion_power.

SIG-0588. 1 research card. Kinds: `function`.

```
(power: 'int', *, name: 'str | None' = None) -> 'CustomFunctionSpec'
```

Cards: C.3A factorial_power_bell_function.

SIG-0590. 1 research card. Kinds: `function`.

```
(power: 'int', count: 'int') -> 'tuple[int, ...]'
```

Cards: C.3A generalized_bell_numbers.

SIG-0591. 1 research card. Kinds: `function`.

```
(power: 'int', count: 'int') -> 'tuple[tuple[int, ...], ...]'
```

Cards: C.3A generalized_bell_polynomials.

SIG-0592. 1 research card. Kinds: `function`.

```
(power: 'int', count: 'int', weights: 'Iterable[int | Fraction] | Callable[[int], int | Fraction]') ->
   ↪ 'tuple[Fraction, ...]'
```

Cards: C.3A weighted_generalized_bell_numbers.

SIG-0593. 1 research card. Kinds: `function`.

```
(power: 'int', count: 'int', weights: 'Iterable[int | Fraction] | Callable[[int], int | Fraction]') ->
   ↪ 'tuple[tuple[Fraction, ...], ...]'
```

Cards: C.3A weighted_generalized_bell_polynomials.

SIG-0594. 1 research card. Kinds: `function`.

```
(power: 'int', prime: 'int') -> 'DigitKernel'
```

Cards: C.3B factorial_power_digit_kernel.

SIG-0595. 1 research card. Kinds: `function`.

```
(power: 'int', weights: 'tuple[int | Fraction, ...] | Callable[[int], int | Fraction]', *, name: 'str | None' =
   ↪ None) -> 'CustomFunctionSpec'
```

Cards: C.3A weighted_factorial_power_bell_function.

SIG-0596. 1 research card. Kinds: `class`.

```
(predict: 'Any', update: 'Any') -> 'None'
```

Cards: C.4B PeriodicLiftingScheme.

SIG-0598. 1 research card. Kinds: `class`.

```
(presentation: 'Any', *, variables: 'Iterable[Any]' = ()) -> 'None'
```

Cards: C.1G PresentedModule.

SIG-0599. 1 research card. Kinds: `function`.

```
(prime: 'int') -> 'int'
```

Cards: C.3B bell_period_bound.

SIG-0600. 1 research card. Kinds: `class`.

```
(prime: 'int') -> None
```

Cards: C.1G EulerLocalCoordinates.

SIG-0601. 1 research card. Kinds: `class`.

```
(prime: 'int', compression: 'int') -> None
```

Cards: C.1G StandardWindow.

SIG-0602. 1 research card. Kinds: `class`.

```
(prime: 'int', degree: 'int' = 1, modulus: 'tuple[int, ...] | None' = None) -> None
```

Cards: C.1G FiniteField.

SIG-0603. 1 research card. Kinds: `class`.

```
(prime: 'int', degree: 'int', failures: 'tuple[tuple[int, int], ...]') -> None
```

Cards: C.3B RationalMaskAlgebraicCertificate.

SIG-0604. 1 research card. Kinds: `class`.

```
(prime: 'int', depth: 'int', point: 'tuple[int, ...]', residual: 'tuple[int, ...]', correction_space:
    ↪ 'ModularLinearSolution', children: 'tuple[tuple[int, ...], ...]', certificate: 'Certificate') -> None
```

Cards: C.5A LiftStepResult.

SIG-0605. 1 research card. Kinds: `class`.

```
(prime: 'int', dimension: 'int', failures: 'tuple[tuple[int, int, int, int, int], ...]') -> None
```

Cards: C.3B CartierCarryEquivalence.

SIG-0607. 1 research card. Kinds: `class`.

```
(prime: 'int', exponent: 'int', max_modulus: 'int' = 2000000) -> None
```

Cards: C.3B PrimePowerUnitCompiler.

SIG-0608. 1 research card. Kinds: `function`.

```
(prime: 'int', exponents: 'Mapping[int, int]') -> 'CyclotomicCollapse'
```

Cards: C.3B cyclotomic_coboundary.

SIG-0611. 1 research card. Kinds: `class`.

```
(prime: 'int', linear_slope: 'int', carry_weight: 'int') -> None
```

Cards: C.3B CarryLinearValuation.

SIG-0612. 1 research card. Kinds: `class`.

```
(prime: 'int', linear_slope: 'int', carry_weight: 'int', checked_through: 'int', failures: 'tuple[tuple[int, int,
   ↪ int, int], ...]') -> None
```

Cards: C.3B CarryLinearCertificate.

SIG-0613. 1 research card. Kinds: `class`.

```
(prime: 'int', mask_exponents: 'tuple[tuple[int, int], ...]', closed_form_exponents: 'tuple[tuple[int, int],
   ↪ ...]', obstruction_orbits: 'tuple[int, ...]') -> None
```

Cards: C.3B CyclotomicCollapse.

SIG-0614. 1 research card. Kinds: `class`.

```
(prime: 'int', numerator: 'tuple[int, ...]', denominator: 'tuple[int, ...]', degree_bound: 'int',
   ↪ transition_matrices: 'tuple[tuple[tuple[int, ...], ...], ...]', start_vector: 'tuple[int, ...]',
   ↪ output_scale: 'int') -> None
```

Cards: C.3B CartierLinearRepresentation.

SIG-0615. 1 research card. Kinds: `class`.

```
(prime: 'int', numerator: 'tuple[int, ...]', denominator: 'tuple[int, ...]', degree_bound: 'int', transitions:
   ↪ 'tuple[tuple[int, ...], ...]', outputs: 'tuple[int, ...]', state_numerators: 'tuple[tuple[int, ...], ...]',
   ↪ state_members: 'tuple[tuple[int, ...], ...]', start_state: 'int' = 0) -> None
```

Cards: C.3B CartierAutomaton.

SIG-0616. 1 research card. Kinds: `class`.

```
(prime: 'int', precision: 'int') -> 'None'
```

Cards: C.3A TruncatedPadicDomain.

SIG-0617. 1 research card. Kinds: `class`.

```
(prime: 'int', precision: 'int') -> None
```

Cards: C.1G TruncatedPadic.

SIG-0618. 1 research card. Kinds: `class`.

```
(prime: 'int', quotient_rank: 'int', form: 'AlternatingForm', phase_label: 'str | None' = None,
   ↪ quadratic_refinement: 'QuadraticRefinement | None' = None) -> None
```

Cards: C.1G ProjectiveMultiplicityPacket.

SIG-0619. 1 research card. Kinds: `class`.

```
(prime: 'int', rank: 'int', *, max_dimension: 'int' = 4096) -> 'None'
```

Cards: C.1G HeisenbergModel.

SIG-0620. 1 research card. Kinds: `class`.

```
(prime: 'int', solvable: 'bool', particular: 'tuple[int, ...] | None', kernel_basis: 'tuple[tuple[int, ...],
    ↪  ...]', pivot_columns: 'tuple[int, ...]', obstruction_rows: 'tuple[int, ...]') -> None
```

Cards: C.5A ModularLinearSolution.

SIG-0622. 1 research card. Kinds: `function`.

```
(primitive: 'PolynomialPrimitive', *, claimed_matrix: 'Iterable[Iterable[Any]] | None' = None) -> 'Certificate'
```

Cards: C.1B primitive_certificate.

SIG-0623. 1 research card. Kinds: `function`.

```
(primitive: 'PolynomialPrimitive', matrix: 'Iterable[Iterable[Any]]') -> 'bool'
```

Cards: C.1B primitive_matrix_equal.

SIG-0624. 1 research card. Kinds: `class`.

```
(prior: 'Any', likelihood: 'Any') -> 'None'
```

Cards: C.1G BayesianUpdate.

SIG-0628. 1 research card. Kinds: `class`.

```
(projector: 'Any', errors: 'Iterable[Any]', *, tolerance: 'float' = 1e-10) -> 'None'
```

Cards: C.1G QuantumCode.

SIG-0630. 1 research card. Kinds: `class`.

```
(projectors: 'Mapping[str, Any]', *, commuting_operators: 'Iterable[Any]' = (), tolerance: 'float' = 1e-10) ->
    ↪  'None'
```

Cards: C.1G FiniteHeckePackets.

SIG-0631. 1 research card. Kinds: `class`.

```
(projectors: 'Sequence[Any]', *, names: 'Sequence[str] | None' = None, backend: 'str | LinearAlgebraBackend | None'
    ↪  = None, tolerance: 'float | None' = None, validate: 'bool | None' = None) -> 'None'
```

Cards: C.1D PacketSystem.

SIG-0632. 1 research card. Kinds: `function`.

```
(provider: 'Any', measure: 'Measure', *, quadrature: 'CompositeSimpson | None' = None) -> 'IntegralResult'
```

Cards: C.1D integrate_derivative_provider.

SIG-0637. 1 research card. Kinds: `class`.

```
(providers: 'Sequence[Any]', weights: 'Sequence[int | Fraction | float]', name: 'str' = 'atomic integrated
   ↪ derivative provider') -> None
```

Cards: C.1D AtomicIntegratedDerivativeProvider.

SIG-0638. 1 research card. Kinds: `function`.

```
(q: 'int', *, name: 'str | None' = None) -> 'WeightedMonomialBasis'
```

Cards: C.3A q_factorial_basis.

SIG-0639. 1 research card. Kinds: `class`.

```
(q: 'int', degree: 'int', exponent: 'int') -> None
```

Cards: C.1G FrobeniusOrbit.

SIG-0640. 1 research card. Kinds: `class`.

```
(radix: 'int', degree_bound: 'int', coefficients: 'tuple[Fraction, ...] | None', residual: 'tuple[Fraction, ...]')
   ↪ -> None
```

Cards: C.3B PolynomialCoboundaryResult.

SIG-0641. 1 research card. Kinds: `class`.

```
(radix: 'int', digit_weights: 'tuple[tuple[Fraction, ...], ...]', matrices: 'tuple[Matrix, ...]') -> None
```

Cards: C.3B DigitCauchyProductPlan.

SIG-0642. 1 research card. Kinds: `function`.

```
(radix: 'int', exponents: 'Mapping[int, int]') -> 'EulerProductCollapse'
```

Cards: C.3B euler_product_collapse.

SIG-0644. 1 research card. Kinds: `class`.

```
(radix: 'int', mask_exponents: 'tuple[tuple[int, int], ...]', closed_form_exponents: 'tuple[tuple[int, int],
   ↪ ...]', obstruction_orbits: 'tuple[int, ...]') -> None
```

Cards: C.3B EulerProductCollapse.

SIG-0645. 1 research card. Kinds: `class`.

```
(radix: 'int', modulus: 'int', power: 'int', tested_weights: 'int', weight_failures: 'int', tested_coefficients:
   ↪ 'int', closure_failures: 'int', local_product: 'tuple[int, ...]') -> None
```

Cards: C.3B DigitAlgebraAudit.

SIG-0646. 1 research card. Kinds: `class`.

```
(radix: 'int', modulus: 'int', table: 'tuple[tuple[int, ...], ...]', name: 'str' = 'digit kernel') -> None
```

Cards: C.3B DigitKernel.

SIG-0647. 1 research card. Kinds: `class`.

```
(radix: 'int', point: 'complex', terms: 'int', value: 'complex', relative_tail_bound: 'float',
    ↪ absolute_error_bound: 'float') -> None
```

Cards: C.3B AnalyticMaskApproximation.

SIG-0648. 1 research card. Kinds: `function`.

```
(rank: 'int', *, inverse: 'bool' = False, budget: 'CoordinateBudget' =
    ↪ CoordinateBudget(maximum_matrix_dimension=512, maximum_polynomial_degree=512, maximum_horizon=4096,
    ↪ maximum_exact_entries=1000000)) -> 'CoordinateTransform'
```

Cards: C.2A subset_zeta_transform.

SIG-0649. 1 research card. Kinds: `function`.

```
(realization: 'LinearRealization', target_coefficients: 'Iterable[Scalar]') -> 'LinearRealization'
```

Cards: C.2C realization_defect.

SIG-0651. 1 research card. Kinds: `function`.

```
(recurrence: 'LinearRecurrenceGuess', initial: 'Sequence[int | Fraction]') -> 'Any | None'
```

Cards: C.3A characteristic_closed_form.

SIG-0652. 1 research card. Kinds: `function`.

```
(recurrence: 'LinearRecurrenceGuess', initial: 'Sequence[int | Fraction]') -> 'tuple[tuple[Fraction, ...],
    ↪ tuple[Fraction, ...]]'
```

Cards: C.3A rational_gf_from_recurrence.

SIG-0653. 1 research card. Kinds: `function`.

```
(reference: 'PublicApiManifest', current: 'PublicApiManifest | None' = None, *, strict_signatures: 'bool' = False)
    ↪ -> 'ApiCompatibilityReport'
```

Cards: C.1A compare_public_api.

SIG-0654. 1 research card. Kinds: `class`.

```
(reference_digest: 'str', current_digest: 'str', added: 'tuple[tuple[str, str], ...]', removed: 'tuple[tuple[str,
    ↪ str], ...]', kind_changed: 'tuple[tuple[str, str, str, str], ...]', signature_changed: 'tuple[tuple[str,
    ↪ str, str | None, str | None], ...]', strict_signatures: 'bool', certificate: 'Certificate') -> None
```

Cards: C.1A ApiCompatibilityReport.

SIG-0655. 1 research card. Kinds: `class`.

```
(refinements: 'tuple[PacketRefinement, ...] | list[PacketRefinement]') -> 'None'
```

Cards: C.1G RefinementTower.

SIG-0656. 1 research card. Kinds: `class`.

```
(relation: 'Sequence[Sequence[bool | int]]') -> 'None'
```

Cards: C.1D SupportPreorder.

SIG-0657. 1 research card. Kinds: `class`.

```
(representation: 'InducedRepresentation[Element]') -> 'None'
```

Cards: C.1G MonomialAction.

SIG-0658. 1 research card. Kinds: `class`.

```
(representation: 'InducedRepresentation[Element]', base_field: 'str', inducing_field: 'str', inducing_character:
   ↪ 'str', cyclic_tower: 'tuple[str, ...]' = ()) -> None
```

Cards: C.1G AutomorphicInductionPlan.

SIG-0659. 1 research card. Kinds: `function`.

```
(residues: 'Iterable[int]', moduli: 'Iterable[int]') -> 'CRTAddress'
```

Cards: C.1G crt_many.

SIG-0661. 1 research card. Kinds: `function`.

```
(root: 'TreeNode', budget: 'int', *, error_unit: 'float' = 1.0) -> 'TreePlan | None'
```

Cards: C.1D compile_tree_budget.

SIG-0662. 1 research card. Kinds: `function`.

```
(root: 'TreeNode', penalty: 'float') -> 'TreePlan'
```

Cards: C.1D compile_tree_lagrangian.

SIG-0663. 1 research card. Kinds: `function`.

```
(rows: 'int', columns: 'int') -> 'Matrix'
```

Cards: C.2A zero_matrix.

SIG-0664. 1 research card. Kinds: `function`.

```
(rows: 'Iterable[Iterable[Scalar]]') -> 'Matrix'
```

Cards: C.2A as_matrix.

SIG-0665. 1 research card. Kinds: `class`.

```
(rule: 'str', theorem_id: 'str | None', before_digest: 'str', after_digest: 'str', side_conditions: 'tuple[str,
   ↪ ...]', certificate: 'Certificate') -> None
```

Cards: C.1B RewriteStep.

SIG-0666. 1 research card. Kinds: `function`.

```
(sample_count: 'int', sigma: 'float', delta: 'float') -> 'float'
```

Cards: C.4B subgaussian_noise_bound.

SIG-0667. 1 research card. Kinds: `function`.

```
(samples: 'Any', *, spacing: 'float', lipschitz_bound: 'float', noise_bound: 'float', minimum_jump: 'float',
   ↪ origin: 'float' = 0.0, threshold: 'float | None' = None) -> 'JumpRecoveryResult'
```

Cards: C.4B recover_vector_jumps.

SIG-0668. 1 research card. Kinds: `function`.

```
(samples: 'Any', *, spacing: 'float', lipschitz_bound: 'float', sigma: 'float', minimum_jump: 'float',
    ↪ failure_probability: 'float', origin: 'float' = 0.0) -> 'JumpRecoveryResult'
```

Cards: C.4B recover_vector_jumps_subgaussian.

SIG-0669. 1 research card. Kinds: `class`.

```
(samples: 'np.ndarray', final_state: 'np.ndarray') -> None
```

Cards: C.4A FilterOutput.

SIG-0670. 1 research card. Kinds: `function`.

```
(samples: 'Sequence[float]', *, degree: 'int', noise_bound: 'float', threshold: 'float | None' = None) ->
    ↪ 'BreakpointBandResult'
```

Cards: C.4B detect_piecewise_polynomial_breaks.

SIG-0672. 1 research card. Kinds: `function`.

```
(samples: 'Sequence[float]', *, spacing: 'float', lipschitz_bound: 'float', sigma: 'float', delta: 'float',
    ↪ minimum_jump: 'float', threshold: 'float | None' = None, start: 'float' = 0.0, minimum_separation: 'float |
    ↪ None' = None) -> 'JumpRecoveryResult'
```

Cards: C.4B recover_jumps_subgaussian.

SIG-0673. 1 research card. Kinds: `class`.

```
(satisfied: 'bool', detail: 'str', external: 'bool' = True) -> None
```

Cards: C.1G GateStatus.

SIG-0674. 1 research card. Kinds: `class`.

```
(scalar: 'Fraction', value: 'Expression') -> None
```

Cards: C.1B Scale.

SIG-0675. 1 research card. Kinds: `function`.

```
(scalar: 'int | Fraction', value: 'Expression') -> 'Scale'
```

Cards: C.1B scale.

SIG-0676. 1 research card. Kinds: `function`.

```
(scale: 'Scalar', shift: 'Scalar') -> 'Polynomial'
```

Cards: C.2B affine_polynomial.

SIG-0677. 1 research card. Kinds: `function`.

```
(scales: 'Sequence[Scalar]', *, source: 'str' = 'unscaled coordinates', target: 'str' = 'scaled coordinates') ->
    ↪ 'CoordinateTransform'
```

Cards: C.2C scaling_transform.

SIG-0680. 1 research card. Kinds: `class`.

```
(seed_norm: 'float', operation_norms: 'dict[int, float]', radius: 'float', image_bound: 'float', contraction:
    ↪  'float', certificate: 'Certificate') -> None
```

Cards: C.5B MajorantConvergence.

SIG-0682. 1 research card. Kinds: `function`.

```
(seed_ranks: 'Sequence[int]') -> 'int'
```

Cards: C.1D independent_direct_sum_seed_rank.

SIG-0683. 1 research card. Kinds: `function`.

```
(seed_ranks: 'Sequence[int]') -> 'tuple[int, int]'
```

Cards: C.1D shared_direct_sum_seed_bounds.

SIG-0684. 1 research card. Kinds: `function`.

```
(sequence: 'Sequence', *, sample_count: 'int' = 24, max_order: 'int | None' = 8) -> 'CompiledSequence'
```

Cards: C.1G compile_sequence.

SIG-0685. 1 research card. Kinds: `class`.

```
(sequence: 'Sequence', strategy: 'str', certificate: 'Certificate', recurrence: 'LinearRecurrence | None' = None,
    ↪  automatic: 'AutomaticSequence | None' = None, holonomic: 'HolonomicRecurrence | None' = None) -> None
```

Cards: C.1G CompiledSequence.

SIG-0686. 1 research card. Kinds: `function`.

```
(series: 'FormalSeries', *, degree: 'int | None' = None) -> 'ExactPolynomial'
```

Cards: C.1D polynomial_from_formal_series.

SIG-0687. 1 research card. Kinds: `function`.

```
(series: 'FormalSeries', *, degree: 'int | None' = None, dtype: 'Any' = <class 'float'>) -> 'FIRSeriesAdapter'
```

Cards: C.3A series_to_fir_adapter.

SIG-0688. 1 research card. Kinds: `function`.

```
(series: 'FormalSeries', *, degree: 'int | None' = None, dtype: 'Any' = <class 'float'>) -> 'np.ndarray'
```

Cards: C.3A series_to_fir_taps.

SIG-0689. 1 research card. Kinds: `function`.

```
(series: 'FormalSeries', measure: 'Measure', *, degree: 'int | None' = None) -> 'IntegralResult'
```

Cards: C.1D integrate_formal_series.

SIG-0690. 1 research card. Kinds: `class`.

```
(series: 'FormalSeries', taps: 'np.ndarray', variable_substitution: 'str' = 'formal z maps to delay z^(-1)') ->
    ↪  None
```

Cards: C.3A FIRSeriesAdapter.

SIG-0691. 1 research card. Kinds: `function`.

```
(series: 'Iterable[Scalar]', mask: 'Iterable[Scalar]', radix: 'int', degree: 'int | None' = None, *, budget:
    ↪ 'GeneratingBudget' = GeneratingBudget(maximum_degree=100000, maximum_coefficient_bits=4000000,
    ↪ maximum_matrix_dimension=4096, maximum_reachable_states=100000, maximum_local_combinations=1000000,
    ↪ maximum_modulus=2000000, maximum_serialized_bytes=67108864, maximum_recursion_depth=1000)) -> 'Certificate'
```

Cards: C.3B certify_mahler_functional_equation.

SIG-0693. 1 research card. Kinds: `function`.

```
(shape: 'Sequence[int]', *, spacing: 'float | None' = None) -> 'np.ndarray'
```

Cards: C.1G periodic_laplacian.

SIG-0694. 1 research card. Kinds: `function`.

```
(shape: 'Sequence[int]', axes: 'Sequence[int]') -> 'CoordinateTransform'
```

Cards: C.2A tensor_axis_permutation_transform.

SIG-0695. 1 research card. Kinds: `function`.

```
(shape: 'Sequence[int]', axes: 'Sequence[int]') -> 'tuple[int, ...]'
```

Cards: C.2A tensor_axis_permutation_order.

SIG-0696. 1 research card. Kinds: `function`.

```
(sigma: 'float', coefficient_count: 'int') -> 'float'
```

Cards: C.4B universal_threshold.

SIG-0697. 1 research card. Kinds: `function`.

```
(signal: 'Any', *, levels: 'int' = 1, mode: 'ThresholdMode' = 'soft', rule: 'ThresholdRule' = 'universal',
    ↪ threshold: 'float | None' = None, sigma: 'float | None' = None, level_dependent: 'bool' = False, normalized:
    ↪ 'bool' = True) -> 'WaveletDenoiseResult'
```

Cards: C.4B wavelet_denoise.

SIG-0698. 1 research card. Kinds: `function`.

```
(signal: 'Any', *, normalized: 'bool' = True) -> 'tuple[np.ndarray, np.ndarray]'
```

Cards: C.4B haar_analyze.

SIG-0699. 1 research card. Kinds: `function`.

```
(signal: 'Any', *, sample_rate: 'float' = 1.0, frame_length: 'int' = 256, hop_length: 'int | None' = None,
    ↪ fft_length: 'int | None' = None, window: 'WindowSpec' = 'hann', center: 'bool' = True, pad_end: 'bool' =
    ↪ True, boundary: 'BoundaryMode' = 'zeros', onesided: 'bool | None' = None, detrend: 'DetrendMode' = 'none')
    ↪ -> 'STFTResult'
```

Cards: C.4A stft.

SIG-0700. 1 research card. Kinds: `function`.

```
(signal: 'Any', *, sample_rate: 'float' = 1.0, frame_length: 'int' = 256, hop_length: 'int | None' = None,
    ↪ fft_length: 'int | None' = None, window: 'WindowSpec' = 'hann', detrend: 'DetrendMode' = 'constant',
    ↪ scaling: 'SpectralScaling' = 'density', average: 'SpectralAverage' = 'mean', onesided: 'bool | None' = None)
    ↪ -> 'PowerSpectrum'
```

Cards: C.4A welch_psd.

SIG-0701. 1 research card. Kinds: `function`.

```
(signal: 'Any', *, sample_rate: 'float' = 1.0, window: 'WindowSpec' = 'boxcar', fft_length: 'int | None' = None,
   ↪ detrend: 'DetrendMode' = 'constant', scaling: 'SpectralScaling' = 'density', onesided: 'bool | None' = None)
   ↪ -> 'PowerSpectrum'
```

Cards: C.4A periodogram.

SIG-0702. 1 research card. Kinds: `function`.

```
(signal: 'Any', *, scaling: 'SpectrogramScaling' = 'density', **stft_options: 'Any') -> 'SpectrogramResult'
```

Cards: C.4A spectrogram.

SIG-0703. 1 research card. Kinds: `function`.

```
(signal: 'Any', *, tolerance: 'float' = 1e-12) -> 'Certificate'
```

Cards: C.4A parseval_certificate.

SIG-0704. 1 research card. Kinds: `function`.

```
(signal: 'Any', chunk_sizes: 'Iterable[int]', *, frame_length: 'int' = 256, hop_length: 'int | None' = None,
   ↪ fft_length: 'int | None' = None, window: 'WindowSpec' = 'hann', sample_rate: 'float' = 1.0, onesided: 'bool'
   ↪ = True, detrend: 'DetrendMode' = 'none', pad_end: 'bool' = True, tolerance: 'float' = 1e-11) ->
   ↪ 'ComputationResult[StreamingSTFTBatch]'
```

Cards: C.4A stream_stft.

SIG-0705. 1 research card. Kinds: `function`.

```
(signal: 'Any', depth: 'int', *, normalized: 'bool' = True) -> 'HaarPacketTree'
```

Cards: C.4B haar_packet_analyze.

SIG-0706. 1 research card. Kinds: `function`.

```
(signal: 'Any', estimated_packet: 'Any', approximation_matrix: 'Any', *, tolerance: 'float' = 1e-10) ->
   ↪ 'ComputationResult[np.ndarray]'
```

Cards: C.4B estimated_packet_reconstruct.

SIG-0707. 1 research card. Kinds: `function`.

```
(signal: 'Any', estimated_packet: 'Any', approximation_matrix: 'Any', *, true_smooth: 'Any | None' = None,
   ↪ true_packet: 'Any | None' = None, tolerance: 'float' = 1e-10) -> 'RetainedPacketResult'
```

Cards: C.4B retained_packet_reconstruction.

SIG-0708. 1 research card. Kinds: `function`.

```
(signal: 'Any', factor: 'int') -> 'np.ndarray'
```

Cards: C.4A upsample.

SIG-0710. 1 research card. Kinds: `function`.

```
(signal: 'Any', factor: 'int', *, phase: 'int' = 0) -> 'np.ndarray'
```

Cards: C.4A downsample.

SIG-0711. 1 research card. Kinds: `function`.

```
(signal: 'Any', frame_length: 'int', hop_length: 'int | None' = None, *, center: 'bool' = False, pad_end: 'bool' =
    ↪ False, boundary: 'BoundaryMode' = 'zeros') -> 'FrameBatch'
```

Cards: C.4A frame_signal.

SIG-0712. 1 research card. Kinds: `function`.

```
(signal: 'Any', kernel: 'Any', *, block_length: 'int' = 1024, mode: 'FIRMode' = 'full') -> 'np.ndarray'
```

Cards: C.4A overlap_add_convolve.

SIG-0713. 1 research card. Kinds: `function`.

```
(signal: 'Any', kernel: 'Any', *, block_length: 'int' = 1024, mode: 'FIRMode' = 'full', tolerance: 'float' = 1e-11)
    ↪ -> 'ComputationResult[np.ndarray]'
```

Cards: C.4A certified_overlap_add_convolve.

SIG-0714. 1 research card. Kinds: `function`.

```
(signal: 'Any', kernel: 'Any', *, mode: 'ConvolutionMode' = 'full') -> 'np.ndarray'
```

Cards: C.4A convolve_signal.

SIG-0715. 1 research card. Kinds: `function`.

```
(signal: 'Any', kernel: 'Any', *, mode: 'FIRMode' = 'full') -> 'np.ndarray'
```

Cards: C.4A fft_convolve.

SIG-0716. 1 research card. Kinds: `function`.

```
(signal: 'Any', kernel: 'Any', order: 'int', *, mode: 'ConvolutionMode' = 'full', tolerance: 'float' = 1e-11) ->
    ↪ 'ComputationResult[np.ndarray]'
```

Cards: C.4A polyphase_convolution.

SIG-0717. 1 research card. Kinds: `function`.

```
(signal: 'Any', levels: 'int', *, normalized: 'bool' = True) -> 'HaarDecomposition'
```

Cards: C.4B haar_decompose.

SIG-0719. 1 research card. Kinds: `function`.

```
(signal: 'Any', order: 'int', *, axis: 'int' = -1, tolerance: 'float' = 1e-12) -> 'ComputationResult[np.ndarray]'
```

Cards: C.4B polyphase_roundtrip_certificate.

SIG-0720. 1 research card. Kinds: `function`.

```
(signal: 'Any', order: 'int', residue: 'int', *, axis: 'int' = -1) -> 'np.ndarray'
```

Cards: C.4B residue_project.

SIG-0722. 1 research card. Kinds: `function`.

```
(signal: 'Any', strides: 'Iterable[int]', *, copy: 'bool' = True) -> 'LatticePolyphaseCoordinates'
```

Cards: C.4B lattice_polyphase_analyze.

SIG-0723. 1 research card. Kinds: `function`.

```
(signal: 'Any', taps: 'Any', *, mode: 'FIRMode' = 'causal') -> 'np.ndarray'
```

Cards: C.4A fir_filter.

SIG-0724. 1 research card. Kinds: `function`.

```
(signal: 'Any', taps: 'Any', chunk_sizes: 'Iterable[int]', *, tolerance: 'float' = 1e-12) ->
   ↪ 'ComputationResult[np.ndarray]'
```

Cards: C.4A stream_fir_filter.

SIG-0725. 1 research card. Kinds: `function`.

```
(signal: 'Any', up: 'int', down: 'int', *, taps: 'Any | None' = None, num_taps: 'int | None' = None, window:
   ↪ 'WindowSpec' = ('kaiser', 8.0)) -> 'np.ndarray'
```

Cards: C.4A resample_poly.

SIG-0726. 1 research card. Kinds: `function`.

```
(signal: 'Any', up: 'int', down: 'int', *, taps: 'Any | None' = None, num_taps: 'int | None' = None, window:
   ↪ 'WindowSpec' = ('kaiser', 8.0), tolerance: 'float' = 1e-12) -> 'ResamplingResult'
```

Cards: C.4A resample_poly_result.

SIG-0727. 1 research card. Kinds: `function`.

```
(signal: 'np.ndarray | Sequence[float]', *, bins: 'int' = 256) -> 'PacketEstimate1D'
```

Cards: C.4B estimate_positive_packet_1d.

SIG-0728. 1 research card. Kinds: `function`.

```
(signal: 'np.ndarray | Sequence[float]', modes: 'int') -> 'np.ndarray'
```

Cards: C.4B fft_lowpass_1d.

SIG-0729. 1 research card. Kinds: `class`.

```
(signal: 'np.ndarray', decomposition: 'HaarDecomposition', thresholded_details: 'tuple[np.ndarray, ...]',
   ↪ thresholds: 'tuple[float, ...]', noise_sigma: 'float', rule: 'ThresholdRule', mode: 'ThresholdMode',
   ↪ certificate: 'Certificate') -> None
```

Cards: C.4B WaveletDenoiseResult.

SIG-0731. 1 research card. Kinds: `class`.

```
(signal: 'np.ndarray', plan: 'RationalResamplingPlan', input_length: 'int', certificate: 'Certificate') -> None
```

Cards: C.4A ResamplingResult.

SIG-0732. 1 research card. Kinds: `function`.

```
(signal: 'Sequence[float]', alpha: 'float' = 0.25, *, shift: 'int' = 1, shifts: 'Sequence[int] | None' = None,
    ↪ coefficients: 'Sequence[float] | None' = None) -> 'np.ndarray'
```

Cards: C.4B nonlinear_shift_filter.

SIG-0733. 1 research card. Kinds: `function`.

```
(size: 'int') -> 'Matrix'
```

Cards: C.2A identity_matrix.

SIG-0734. 1 research card. Kinds: `function`.

```
(size: 'int', *, backend: 'str | LinearAlgebraBackend | None' = None) -> 'tuple[Any, ...]'
```

Cards: C.1D matrix_units.

SIG-0736. 1 research card. Kinds: `function`.

```
(size: 'int', *, inverse: 'bool' = False, budget: 'CoordinateBudget' =
    ↪ CoordinateBudget(maximum_matrix_dimension=512, maximum_polynomial_degree=512, maximum_horizon=4096,
    ↪ maximum_exact_entries=1000000)) -> 'CoordinateTransform'
```

Cards: C.2A divisor_zeta_transform.

SIG-0740. 1 research card. Kinds: `class`.

```
(sizes: 'tuple[int, ...]', names: 'tuple[str, ...] | None' = None) -> None
```

Cards: C.1D PacketLayout.

SIG-0745. 1 research card. Kinds: `class`.

```
(solution: 'np.ndarray', iterations: 'int', contraction_factor: 'float', equation_residual: 'float',
    ↪ fixed_point_residual: 'float', certificate: 'Certificate') -> None
```

Cards: C.5A RightInverseCorrectionResult.

SIG-0746. 1 research card. Kinds: `class`.

```
(source: 'CoordinateSpace', target: 'CoordinateSpace') -> None
```

Cards: C.1B LinearMapType.

SIG-0749. 1 research card. Kinds: `class`.

```
(source: 'int', target: 'int', weight: 'int', operator: 'Any', name: 'str' = '') -> None
```

Cards: C.1G WeightedTransferEdge.

SIG-0752. 1 research card. Kinds: `function`.

```
(source: 'Matrix', target: 'Matrix', source_vector: 'Sequence[Scalar]', target_vector: 'Sequence[Scalar]') ->
    ↪ 'Matrix'
```

Cards: C.2C similarity_from_cyclic_vectors.

SIG-0753. 1 research card. Kinds: `class`.

```
(source: 'PacketSystem', target: 'PacketSystem', blocks: 'tuple[tuple[Any, ...], ...]') -> None
```

Cards: C.1D PacketOperator.

SIG-0754. 1 research card. Kinds: `class`.

```
(source: 'PacketSystem', target: 'PacketSystem', linear_map: 'Any', packet_map: 'Mapping[str, tuple[str, ...]]')
   ↪  -> None
```

Cards: C.1D PacketMorphism.

SIG-0755. 1 research card. Kinds: `function`.

```
(source: 'PolynomialCoordinateSpace') -> 'PolynomialPrimitive'
```

Cards: C.1B derivative.

SIG-0757. 1 research card. Kinds: `function`.

```
(source: 'PolynomialCoordinateSpace', point: 'int | Fraction') -> 'PolynomialPrimitive'
```

Cards: C.1B evaluation.

SIG-0758. 1 research card. Kinds: `function`.

```
(source: 'PolynomialCoordinateSpace', scale: 'int | Fraction' = 1, shift: 'int | Fraction' = 0) ->
   ↪  'PolynomialPrimitive'
```

Cards: C.1B affine_substitution.

SIG-0759. 1 research card. Kinds: `function`.

```
(source: 'PolynomialCoordinateSpace', step: 'int | Fraction' = 1) -> 'PolynomialPrimitive'
```

Cards: C.1B finite_difference.

SIG-0760. 1 research card. Kinds: `function`.

```
(source: 'PolynomialCoordinateSpace', target: 'PolynomialCoordinateSpace') -> 'PolynomialPrimitive'
```

Cards: C.1B coordinate_change.

SIG-0761. 1 research card. Kinds: `function`.

```
(source: 'PolynomialCoordinateSpace', target: 'PolynomialCoordinateSpace', basepoint: 'int | Fraction' = 0) ->
   ↪  'PolynomialPrimitive'
```

Cards: C.1B antiderivative.

SIG-0763. 1 research card. Kinds: `function`.

```
(source: 'PolynomialFamily', target: 'PolynomialFamily', degree: 'int', measure: 'IntervalMeasure') ->
   ↪  'IntegralIdentity'
```

Cards: C.1D coordinate_transport_identity.

SIG-0764. 1 research card. Kinds: `function`.

```
(source: 'PolynomialFamily', target: 'PolynomialFamily', source_degree: 'int', target_degree: 'int', operator:
    ↪ 'Callable[[Polynomial], Polynomial]', *, name: 'str' = 'polynomial operator', budget: 'CoordinateBudget' =
    ↪ CoordinateBudget(maximum_matrix_dimension=512, maximum_polynomial_degree=512, maximum_horizon=4096,
    ↪ maximum_exact_entries=1000000)) -> 'Matrix'
```

Cards: C.2B operator_matrix_between_families.

SIG-0765. 1 research card. Kinds: `function`.

```
(source: 'PolynomialFamily', target: 'PolynomialFamily', source_degree: 'int', target_degree: 'int', rule:
    ↪ 'Callable[[Polynomial], Polynomial]', *, name: 'str', theorem: 'str' = 'CC.O1', budget: 'CoordinateBudget' =
    ↪ CoordinateBudget(maximum_matrix_dimension=512, maximum_polynomial_degree=512, maximum_horizon=4096,
    ↪ maximum_exact_entries=1000000)) -> 'PolynomialCoordinateOperator'
```

Cards: C.2B polynomial_operator.

SIG-0766. 1 research card. Kinds: `class`.

```
(source: 'SeedSystem', target: 'SeedSystem', linear_map: 'Any', operator_map: 'Mapping[str, str]', seed_images:
    ↪ 'Sequence[Any] | None' = None) -> None
```

Cards: C.1D SeedMorphism.

SIG-0767. 1 research card. Kinds: `class`.

```
(source: 'StateSpaceModel', target: 'StateSpaceModel', transform: 'CoordinateTransform', markov_count: 'int',
    ↪ valid: 'bool') -> None
```

Cards: C.2C StateSpaceCoordinateCertificate.

SIG-0769. 1 research card. Kinds: `class`.

```
(source: 'str') -> None
```

Cards: C.6 RawLatex.

SIG-0777. 1 research card. Kinds: `class`.

```
(source_components: 'frozenset[str]', target_components: 'frozenset[str]', image_components: 'frozenset[str]',
    ↪ certificate: 'Certificate') -> None
```

Cards: C.1G SupportComparison.

SIG-0778. 1 research card. Kinds: `class`.

```
(source_correction: 'Any', target_correction: 'Any', source_map: 'Any', target_map: 'Any', backend: 'str |
    ↪ LinearAlgebraBackend | None' = None, tolerance: 'float' = 1e-10) -> None
```

Cards: C.5A ObstructionSquare.

SIG-0780. 1 research card. Kinds: `class`.

```
(source_layout: 'PacketLayout', target_layout: 'PacketLayout', blocks: 'Mapping[Address, StructuredOperator |
    ↪ Any]', metadata: 'Mapping[str, Any]' = <factory>) -> None
```

Cards: C.1D PacketIR.

SIG-0781. 1 research card. Kinds: `class`.

```
(source_operators: 'Mapping[str, Any]', target_operators: 'Mapping[str, Any]', linear_map: 'Any', operator_map:
   ↪ 'Mapping[str, str]', backend: 'Any', tolerance: 'float') -> None
```

Cards: C.1D OperatorMorphism.

SIG-0782. 1 research card. Kinds: `class`.

```
(source_path: 'Path', pdf_path: 'Path | None', log_path: 'Path', engine: 'LatexEngine', commands:
   ↪ 'tuple[tuple[str, ...], ...]', return_codes: 'tuple[int, ...]', certificate: 'Certificate') -> None
```

Cards: C.6 LatexCompilation.

SIG-0783. 1 research card. Kinds: `function`.

```
(space: 'CoordinateSpace') -> 'IdentityMap'
```

Cards: C.1B identity.

SIG-0784. 1 research card. Kinds: `class`.

```
(space: 'CoordinateSpace') -> None
```

Cards: C.1B IdentityMap.

SIG-0785. 1 research card. Kinds: `function`.

```
(space: 'CoordinateSpace', entries: 'Iterable[Any]') -> 'ExactValueLiteral'
```

Cards: C.1B exact_value.

SIG-0786. 1 research card. Kinds: `class`.

```
(space: 'CoordinateSpace', entries: 'Vector') -> None
```

Cards: C.1B ExactValueLiteral.

SIG-0787. 1 research card. Kinds: `class`.

```
(stage: 'UnifiedStage', meaning: 'str', implemented_in_alpha: 'bool', module: 'str') -> None
```

Cards: C.1G StageSpec.

SIG-0788. 1 research card. Kinds: `class`.

```
(stages: 'Iterable[RamificationStage]') -> 'None'
```

Cards: C.1G RamificationCliffordTower.

SIG-0789. 1 research card. Kinds: `class`.

```
(start: 'int', end: 'int', split: 'int | None', rows: 'int', columns: 'int', scalar_operation_cost: 'int') -> None
```

Cards: C.1B MatrixChainTree.

SIG-0791. 1 research card. Kinds: `class`.

```
(steps: 'Iterable[CliffordStep]') -> 'None'
```

Cards: C.1G CliffordTower.

SIG-0792. 1 research card. Kinds: `class`.

```
(subgenerator: 'Any', initial: 'Any', *, cause_rates: 'Any | None' = None, tolerance: 'float' = 1e-12, validate:
    ↪  'bool' = True) -> 'None'
```

Cards: C.1G PhaseTypeSurvival.

SIG-0793. 1 research card. Kinds: `class`.

```
(subgroup: 'FiniteSubgroup[Element]', values: 'Mapping[Element, complex]', tolerance: 'float' = 1e-10) -> None
```

Cards: C.1G LinearCharacter.

SIG-0794. 1 research card. Kinds: `class`.

```
(subintervals: 'int' = 64, derivative_bound: 'DerivativeBound | None' = None, max_evaluations: 'int' = 1000000) ->
    ↪  None
```

Cards: C.1D CompositeSimpson.

SIG-0795. 1 research card. Kinds: `class`.

```
(symbol: 'Symbol', value: 'Expression', body: 'Expression') -> None
```

Cards: C.1B Let.

SIG-0796. 1 research card. Kinds: `function`.

```
(symmetry: 'Any', order: 'int', *, tolerance: 'float' = 1e-10) -> 'PacketSystem'
```

Cards: C.1G cyclic_symmetry_projectors.

SIG-0798. 1 research card. Kinds: `class`.

```
(system: "'PacketSystem'", values: 'tuple[Any, ...]') -> None
```

Cards: C.1D PacketCoordinates.

SIG-0800. 1 research card. Kinds: `function`.

```
(system: 'AnalysisSynthesisSystem', measure: 'AtomicMeasure', physical_values: 'Sequence[Any]', *, tolerance:
    ↪  'float' = 1e-10) -> 'Certificate'
```

Cards: C.1D analysis_synthesis_integration_certificate.

SIG-0802. 1 research card. Kinds: `class`.

```
(systems: 'tuple[str, ...]', transform: 'CoordinateTransform') -> None
```

Cards: C.2C AtlasPath.

SIG-0805. 1 research card. Kinds: `class`.

```
(taps: 'Any') -> 'None'
```

Cards: C.4A StreamingFIR.

SIG-0806. 1 research card. Kinds: `function`.

```
(taps: 'Any', *, basis: 'WeightedMonomialBasis | None' = None, max_denominator: 'int | None' = None) ->
    ↪ 'FormalSeries'
```

Cards: C.3A fir_taps_to_series.

SIG-0807. 1 research card. Kinds: `function`.

```
(taps: 'Any', *, points: 'int' = 512, sample_rate: 'float' = 1.0, whole: 'bool' = False) -> 'FrequencyResponse'
```

Cards: C.4A fir_frequency_response.

SIG-0808. 1 research card. Kinds: `class`.

```
(taps: 'np.ndarray', kind: 'FIRKind', cutoff: 'tuple[float, ...]', sample_rate: 'float' = 1.0, window_name: 'str'
    ↪ = 'custom', reference_frequency: 'float | None' = None, reference_gain: 'float' = 1.0) -> None
```

Cards: C.4A FIRFilter.

SIG-0809. 1 research card. Kinds: `class`.

```
(target: 'int', source: 'int', coefficients: 'np.ndarray', order: 'int') -> None
```

Cards: C.4B PolyphaseKernel.

SIG-0810. 1 research card. Kinds: `function`.

```
(target: 'PolynomialCoordinateSpace') -> 'PolynomialPrimitive'
```

Cards: C.1B constant_injection.

SIG-0811. 1 research card. Kinds: `function`.

```
(tasks: 'Sequence[Task]') -> 'TaskSchedule'
```

Cards: C.1D schedule_unlimited.

SIG-0815. 1 research card. Kinds: `class`.

```
(terms: 'tuple[Expression, ...]') -> None
```

Cards: C.1B Sum.

SIG-0817. 1 research card. Kinds: `class`.

```
(text: 'str') -> None
```

Cards: C.6 Paragraph.

SIG-0818. 1 research card. Kinds: `class`.

```
(text: 'str', level: 'int' = 1, label: 'str | None' = None) -> None
```

Cards: C.6 Heading.

SIG-0819. 1 research card. Kinds: `class`.

```
(time: 'float', step: 'float', maximum_derivative_residual: 'float', tangent_residual: 'float',
    ↪ partition_derivative_residual: 'float', certificate: 'Certificate') -> None
```

Cards: C.5A PacketDerivativeResult.

SIG-0822. 1 research card. Kinds: `function`.

```
(total_digit: 'int', selected_digit: 'int', prime: 'int', factorial_power: 'int', unit_weights: 'Sequence[int]')
   ↪  -> 'int'
```

Cards: C.3B local_normalized_kernel.

SIG-0823. 1 research card. Kinds: `function`.

```
(traces: 'Mapping[Hashable, Sequence[Any]]', *, constituent_count: 'int | None' = None) ->
   ↪  'FrobeniusSeparationResult'
```

Cards: C.1G finite_frobenius_separator.

SIG-0824. 1 research card. Kinds: `function`.

```
(transform: 'CoordinateTransform') -> 'CoordinateTransform'
```

Cards: C.2A contragredient_transform.

SIG-0825. 1 research card. Kinds: `function`.

```
(transform: 'CoordinateTransform', prime: 'int') -> 'FiniteFieldCoordinateTransform'
```

Cards: C.2A reduce_exact_transform_mod_prime.

SIG-0827. 1 research card. Kinds: `function`.

```
(transform: 'STFTResult', *, synthesis_window: 'Any | None' = None, length: 'int | None' = None, tolerance: 'float'
   ↪  = 1e-11) -> 'ComputationResult[np.ndarray]'
```

Cards: C.4A istft.

SIG-0828. 1 research card. Kinds: `function`.

```
(transition: 'Iterable[Iterable[Scalar]]', input_matrix: 'Iterable[Iterable[Scalar]]', *, steps: 'int | None' =
   ↪  None) -> 'Matrix'
```

Cards: C.2C controllability_matrix.

SIG-0829. 1 research card. Kinds: `function`.

```
(transition: 'Iterable[Iterable[Scalar]]', input_vector: 'Sequence[Scalar]', output_covector: 'Sequence[Scalar]',
   ↪  *, feedthrough: 'Scalar' = 0, name: 'str' = 'SISO state-space model') -> 'StateSpaceModel'
```

Cards: C.2C siso_state_space.

SIG-0830. 1 research card. Kinds: `function`.

```
(transition: 'Iterable[Iterable[Scalar]]', output_matrix: 'Iterable[Iterable[Scalar]]', *, steps: 'int | None' =
   ↪  None) -> 'Matrix'
```

Cards: C.2C observability_block_matrix.

SIG-0833. 1 research card. Kinds: `function`.

```
(transition: 'Matrix', vector: 'Sequence[Scalar]') -> 'Matrix'
```

Cards: C.2C cyclic_krylov_matrix.

SIG-0834. 1 research card. Kinds: `function`.

```
(tree: 'HaarPacketTree', cost: 'Mapping[str, float] | CostFunction') -> 'BestBasisResult'
```

Cards: C.4B best_basis.

SIG-0835. 1 research card. Kinds: `class`.

```
(tree: 'HaarPacketTree', leaves: 'tuple[str, ...]') -> None
```

Cards: C.4B PrunedPacketCoefficients.

SIG-0836. 1 research card. Kinds: `class`.

```
(tree: 'OrderedRootedTree', value: 'Any') -> None
```

Cards: C.5B TreeTerm.

SIG-0837. 1 research card. Kinds: `function`.

```
(tree_or_coefficients: 'HaarPacketTree | PrunedPacketCoefficients', leaves: 'Iterable[PacketPath] | None' = None)
   ↪ -> 'float'
```

Cards: C.4B packet_energy.

SIG-0838. 1 research card. Kinds: `function`.

```
(tree_or_coefficients: 'HaarPacketTree | PrunedPacketCoefficients', leaves: 'Iterable[PacketPath] | None' = None)
   ↪ -> 'np.ndarray'
```

Cards: C.4B haar_packet_synthesize.

SIG-0839. 1 research card. Kinds: `function`.

```
(true_labels: 'Any', true_values: 'Sequence[complex]', estimated_labels: 'Any', estimated_values:
   ↪ 'Sequence[complex]', *, p: 'float' = 2.0, tolerance: 'float' = 1e-12) -> 'ComputationResult[float]'
```

Cards: C.4B overlap_matrix_error.

SIG-0840. 1 research card. Kinds: `function`.

```
(true_mask: 'Any', true_value: 'complex', estimated_mask: 'Any', estimated_value: 'complex', *, p: 'float' = 2.0,
   ↪ tolerance: 'float' = 1e-12) -> 'ComputationResult[float]'
```

Cards: C.4B scalar_region_error.

SIG-0841. 1 research card. Kinds: `function`.

```
(true_value: 'complex', estimated_value: 'complex', intersection_measure: 'float', true_only_measure: 'float',
   ↪ estimated_only_measure: 'float', *, p: 'float' = 2.0) -> 'float'
```

Cards: C.4B scalar_region_lp_error.

SIG-0842. 1 research card. Kinds: `function`.

```
(true_values: 'Any', estimated_values: 'Any', overlap_matrix: 'Any', *, p: 'float' = 2.0) -> 'float'
```

Cards: C.4B overlap_partition_lp_error.

SIG-0844. 1 research card. Kinds: `function`.

```
(type_: 'type[T]', adapter: 'Callable[[T], Any]', *, replace: 'bool' = False) -> 'None'
```

Cards: C.6 register_export_adapter.

SIG-0845. 1 research card. Kinds: `class`.

```
(unitary: 'np.ndarray', minimum_singular_value: 'float', certificate: 'Certificate') -> None
```

Cards: C.5A DirectRotationResult.

SIG-0846. 1 research card. Kinds: `function`.

```
(up: 'int', down: 'int', *, num_taps: 'int | None' = None, window: 'WindowSpec' = ('kaiser', 8.0)) -> 'np.ndarray'
```

Cards: C.4A design_resample_filter.

SIG-0847. 1 research card. Kinds: `class`.

```
(up: 'int', down: 'int', taps: 'np.ndarray') -> None
```

Cards: C.4A RationalResamplingPlan.

SIG-0848. 1 research card. Kinds: `class`.

```
(valid: 'bool', degree: 'int', source_basis: 'str', target_basis: 'str', domain: 'str', failures: 'tuple[int,
    ↪ ...]' = ()) -> None
```

Cards: C.3A SeriesTransferCertificate.

SIG-0849. 1 research card. Kinds: `class`.

```
(valid: 'bool', exactness: 'Exactness', residuals: 'dict[str, float]' = <factory>, tolerance: 'float | None' =
    ↪ None, assumptions_checked: 'tuple[str, ...]' = (), assumptions_unchecked: 'tuple[str, ...]' = (),
    ↪ diagnostics: 'dict[str, Any]' = <factory>, theorem_id: 'str | None' = None, message: 'str' = '') -> None
```

Cards: C.1A Certificate.

SIG-0850. 1 research card. Kinds: `function`.

```
(valuation: 'Callable[[int], int]', prime: 'int', checked_through: 'int') -> 'CarryLinearCertificate'
```

Cards: C.3B classify_carry_linear_valuation.

SIG-0851. 1 research card. Kinds: `function`.

```
(value: 'Any') -> 'str'
```

Cards: C.6 latex_escape.

SIG-0852. 1 research card. Kinds: `function`.

```
(value: 'Any') -> 'tuple[tuple[str, ...], tuple[tuple[Any, ...], ...]]'
```

Cards: C.6 table_rows.

SIG-0859. 1 research card. Kinds: `class`.

```
(value: 'Any', certificate: 'Certificate', error: 'IntegrationError', method: 'str', evaluations: 'int') -> None
```

Cards: C.1D IntegralResult.

SIG-0860. 1 research card. Kinds: `class`.

```
(value: 'Any', declared_type: 'Any', certificate: 'Certificate') -> None
```

Cards: C.1B ExpressionBinding.

SIG-0861. 1 research card. Kinds: `function`.

```
(value: 'Any', declared_type: 'CoordinateSpace | LinearMapType', *, theorem_id: 'str' = 'EXPR.BINDING.EXACT') ->
    ↪ 'ExpressionBinding'
```

Cards: C.1B exact_binding.

SIG-0863. 1 research card. Kinds: `class`.

```
(value: 'Any', packet_values: 'tuple[Any, ...]', evaluated_blocks: 'int', pruned_blocks: 'int', certificate:
    ↪ 'Certificate') -> None
```

Cards: C.5B PacketTreeResult.

SIG-0867. 1 research card. Kinds: `class`.

```
(value: 'Any', picard_value: 'Any', terms: 'tuple[TreeTerm, ...]', depth: 'int', certificate: 'Certificate') ->
    ↪ None
```

Cards: C.5B TreeExpansionResult.

SIG-0868. 1 research card. Kinds: `class`.

```
(value: 'Any', terms: 'tuple[FaaDiBrunoTerm, ...]', raw_term_count: 'int', evaluated_term_count: 'int',
    ↪ pruned_term_count: 'int', certificate: 'Certificate') -> None
```

Cards: C.5A FaaDiBrunoResult.

SIG-0869. 1 research card. Kinds: `function`.

```
(value: 'Exactness') -> 'int'
```

Cards: C.1B exactness_strength.

SIG-0870. 1 research card. Kinds: `function`.

```
(value: 'int | Fraction', prime: 'int') -> 'int | float'
```

Cards: C.3B p_adic_valuation_fraction.

SIG-0871. 1 research card. Kinds: `function`.

```
(value: 'int') -> 'int'
```

Cards: C.2A mobius_integer.

SIG-0873. 1 research card. Kinds: `function`.

```
(value: 'Mapping[str, Any]', *, budget: 'ExpressionBudget | None' = None) -> 'Expression'
```

Cards: C.1B expression_from_dict.

SIG-0874. 1 research card. Kinds: `function`.

```
(value: 'Mapping[str, Any]', *, budget: 'ExpressionBudget | None' = None) -> 'ExpressionType'
```

Cards: C.1B type_from_dict.

SIG-0876. 1 research card. Kinds: `function`.

```
(value: 'Rational', prime: 'int') -> 'int | float'
```

Cards: C.1G padic_valuation.

SIG-0877. 1 research card. Kinds: `function`.

```
(value: 'Scalar') -> 'Polynomial'
```

Cards: C.2B constant_polynomial.

SIG-0878. 1 research card. Kinds: `function`.

```
(value: 'str | bytes', *, budget: 'ExpressionBudget | None' = None) -> 'Expression'
```

Cards: C.1B expression_from_json.

SIG-0879. 1 research card. Kinds: `function`.

```
(value: 'str') -> 'str'
```

Cards: C.6 latex_label.

SIG-0881. 1 research card. Kinds: `class`.

```
(value: 'tuple[int, ...]', depth: 'int', parent_index: 'int | None') -> None
```

Cards: C.5A LiftTreeNode.

SIG-0882. 1 research card. Kinds: `function`.

```
(values: 'Any', *, center: 'float | None' = None) -> 'float'
```

Cards: C.4B median_absolute_deviation.

SIG-0883. 1 research card. Kinds: `function`.

```
(values: 'Any', modulus: 'int') -> 'ResidueCoordinates'
```

Cards: C.1G residue_analyze.

SIG-0884. 1 research card. Kinds: `class`.

```
(values: 'Iterable[Fraction | int]', *, prime: 'int') -> 'None'
```

Cards: C.1G VisibilityTree.

SIG-0886. 1 research card. Kinds: `function`.

```
(values: 'Iterable[int | Fraction]') -> 'tuple[Fraction, ...]'
```

Cards: C.3B mahler_coefficients.

SIG-0887. 1 research card. Kinds: `function`.

```
(values: 'Iterable[int | Fraction]', *, max_numerator_degree: 'int' = 4, max_denominator_degree: 'int' = 4,
   ↪ holdout: 'int | None' = None) -> 'HypergeometricGuess | None'
```

Cards: C.3A guess_hypergeometric_ratio.

SIG-0888. 1 research card. Kinds: `function`.

```
(values: 'Iterable[int | Fraction]', *, max_order: 'int' = 12, holdout: 'int | None' = None) ->
   ↪ 'LinearRecurrenceGuess | None'
```

Cards: C.3A guess_linear_recurrence.

SIG-0889. 1 research card. Kinds: `function`.

```
(values: 'Iterable[int | Fraction]', *, max_order: 'int' = 4, max_degree: 'int' = 5, holdout: 'int | None' = None)
   ↪ -> 'PRecursiveGuess | None'
```

Cards: C.3A guess_p_recursive.

SIG-0890. 1 research card. Kinds: `function`.

```
(values: 'Iterable[int | Fraction]', *, max_period: 'int' = 12, max_degree: 'int' = 6, holdout: 'int | None' =
   ↪ None) -> 'QuasiPolynomialGuess | None'
```

Cards: C.3A guess_quasipolynomial.

SIG-0891. 1 research card. Kinds: `function`.

```
(values: 'Iterable[int | Fraction]', *, max_x_degree: 'int' = 4, max_y_degree: 'int' = 4, holdout: 'int | None' =
   ↪ None) -> 'AlgebraicGFGuess | None'
```

Cards: C.3A guess_algebraic_gf.

SIG-0892. 1 research card. Kinds: `function`.

```
(values: 'Iterable[int | Fraction]', prime: 'int') -> 'dict[int, tuple[tuple[int, Fraction], ...]]'
```

Cards: C.3A prime_valuation_packets.

SIG-0893. 1 research card. Kinds: `function`.

```
(values: 'Iterable[Scalar]') -> 'Vector'
```

Cards: C.2A as_vector.

SIG-0894. 1 research card. Kinds: `function`.

```
(values: 'np.ndarray | Sequence[float]', bins: 'int' = 256) -> 'float'
```

Cards: C.4B otsu_threshold.

SIG-0895. 1 research card. Kinds: `class`.

```
(values: 'np.ndarray', frequencies: 'np.ndarray', times: 'np.ndarray', scaling: 'SpectrogramScaling', sample_rate:
   ↪ 'float', transform: 'STFTResult') -> None
```

Cards: C.4A SpectrogramResult.

SIG-0896. 1 research card. Kinds: `function`.

```
(values: 'Sequence[Any]', *, order: 'int' = 1, step: 'int' = 1) -> 'np.ndarray'
```

Cards: C.4B finite_difference.

SIG-0897. 1 research card. Kinds: `function`.

```
(values: 'Sequence[int | Fraction]', *, prime: 'int', step: 'int', max_order: 'int') ->
   ↪ 'DifferenceValuationProfile'
```

Cards: C.3B difference_valuation_profile.

SIG-0898. 1 research card. Kinds: `function`.

```
(values: 'Sequence[int | Fraction]', transform: 'CoordinateTransform') -> 'tuple[Fraction, ...]'
```

Cards: C.2A transform_sequence.

SIG-0899. 1 research card. Kinds: `function`.

```
(values: 'Sequence[int]', *, modulus: 'int', lane_modulus: 'int', minimum_terms: 'int' = 8) ->
   ↪ 'tuple[ResidueLaneFinding, ...]'
```

Cards: C.3B scan_residue_lanes.

SIG-0900. 1 research card. Kinds: `function`.

```
(values: 'Sequence[int]', *, prime: 'int', residue: 'int', max_order: 'int' = 8, holdout: 'int' = 12) ->
   ↪ 'DecimationRelation | None'
```

Cards: C.3B search_decimation_relation.

SIG-0901. 1 research card. Kinds: `function`.

```
(values: 'Sequence[int]', modulus: 'int', *, preperiod_limit: 'int' = 0) -> 'tuple[int, int] | None'
```

Cards: C.3B minimal_prefix_period.

SIG-0902. 1 research card. Kinds: `function`.

```
(values: 'Sequence[int]', prime: 'int') -> 'RelationAudit'
```

Cards: C.3B audit_touchard.

SIG-0903. 1 research card. Kinds: `function`.

```
(values: 'Sequence[int]', prime: 'int') -> 'tuple[int, tuple[int, ...]]'
```

Cards: C.3B berlekamp_massey.

SIG-0904. 1 research card. Kinds: `function`.

```
(values: 'Sequence[int]', prime: 'int', exponent: 'int' = 3) -> 'RelationAudit'
```

Cards: C.3B audit_apery_supercongruence.

SIG-0905. 1 research card. Kinds: `function`.

```
(values: 'Sequence[int]', prime: 'int', residue: 'int') -> 'RelationAudit'
```

Cards: C.3B audit_partition_lane.

SIG-0906. 1 research card. Kinds: `function`.

```
(values: 'Sequence[Scalar]', coefficients: 'RecurrenceLike') -> 'Vector'
```

Cards: C.2C apply_recurrence_defect.

SIG-0909. 1 research card. Kinds: `function`.

```
(values: 'Sequence[Scalar]', source: 'RecurrenceModel', target: 'RecurrenceModel') ->
    ↪ 'tuple[RecurrenceCoordinateRecord, RecurrenceCoordinateRecord, CoordinateTransform]'
```

Cards: C.2C sequence_coordinates_between_models.

SIG-0910. 1 research card. Kinds: `function`.

```
(values_at_zero: 'Sequence[Scalar]', *, name: 'str' = 'custom Appell polynomials') -> 'PolynomialFamily'
```

Cards: C.2B appell_polynomial_family.

SIG-0911. 1 research card. Kinds: `class`.

```
(vanishes: 'bool', obstruction_coordinates: 'tuple[Any, ...]', cokernel_dimension: 'int', correction: 'Any |
    ↪ None', kernel_basis: 'tuple[Any, ...]', residual: 'float', certificate: 'Certificate') -> None
```

Cards: C.5A ObstructionResult.

SIG-0912. 1 research card. Kinds: `class`.

```
(variable: 'str', degree: 'int', basis: 'tuple[tuple[Fraction, ...], ...]', label: 'str' = '') -> None
```

Cards: C.1B PolynomialCoordinateSpace.

SIG-0914. 1 research card. Kinds: `class`.

```
(verified_prefix_depth: 'int', separated: 'bool', complete: 'bool', continuous: 'bool',
    ↪ compatible_infinite_branch: 'bool', conclusion: 'str') -> None
```

Cards: C.5A ProjectiveLimitRecord.

SIG-0915. 1 research card. Kinds: `class`.

```
(vertices: 'Iterable[str]', arrows: 'Mapping[str, tuple[str, str]] | Iterable[Arrow]', relations:
    ↪ 'Iterable[OperatorRelation]' = ()) -> 'None'
```

Cards: C.1D OperatorGrammar.

SIG-0917. 1 research card. Kinds: `function`.

```
(window: 'Any', *, sample_rate: 'float' = 1.0) -> 'WindowMetrics'
```

Cards: C.4A window_metrics.

SIG-0918. 1 research card. Kinds: `function`.

```
(window: 'Any', hop_length: 'int', *, squared: 'bool' = True) -> 'np.ndarray'
```

Cards: C.4A overlap_profile.

SIG-0919. 1 research card. Kinds: `function`.

```
(window: 'Any', hop_length: 'int', *, squared: 'bool' = True, require_constant: 'bool' = False, tolerance: 'float'
   ↪  = 1e-12) -> 'Certificate'
```

Cards: C.4A overlap_add_certificate.

SIG-0920. 1 research card. Kinds: `function`.

```
(window: 'Any', hop_length: 'int', *, tolerance: 'float' = 1e-12) -> 'ComputationResult[np.ndarray]'
```

Cards: C.4A canonical_dual_window.

SIG-0921. 1 research card. Kinds: `function`.

```
(window: 'WindowSpec', length: 'int', *, periodic: 'bool' = True) -> 'np.ndarray'
```

Cards: C.4A get_window.

SIG-0922. 1 research card. Kinds: `class`.

```
(x_degree: 'int', y_degree: 'int', monomials: 'tuple[tuple[int, int, int], ...]', fit_coefficients: 'int',
   ↪  holdout_coefficients: 'int', exact_on_prefix: 'bool') -> None
```

Cards: C.3A AlgebraicGFGuess.

C.10 Alias and stability-tier crosswalk

This crosswalk joins all 1,526 card routes to the additional public routes named by their affirmative card identities. Callable cards that resolve to the same implementation object are placed in one alias group. Data exports are joined only when a card explicitly records the route relation. Negative same-spelling statements are excluded. This prevents distinct functions and equal or interned immutable values from becoming accidental aliases.

The strongest tier shown for a group applies only to the routes that carry that tier. A stable route does not promote a root, beta, or research route merely because both names resolve to the same Python object. The route-specific contract remains controlling.

Tier pattern in alias group	Groups	Card routes	All listed public routes
`stable+beta+research`	22	75	119
`stable+beta`	6	12	24
`stable+research`	51	126	230
`stable`	47	51	153
`beta+research`	55	149	149
`research`	959	1113	1113
Total	**1140**	**1,526**	**1,788**

C.10.1 Stable + Beta + Research route groups

AG-0001 `__version__`. Strongest listed tier `[STABLE]`. Kinds: `data`. Owners: `builtins`. Stable cards: A.1 **version**. Beta cards: B.1 **version**. Research cards: C.1A **version**. Additional public routes: `komorebi_math.api.v1.__version__`, `komorebi_math.v1.__version__`.

AG-0004 `ApiCompatibilityReport`. Strongest listed tier `[STABLE]`. Kinds: `class`. Owners: `komorebi_math.compatibility`. Stable cards: A.9 ApiCompatibilityReport. Beta cards: B.1 ApiCompatibilityReport. Research cards: C.1A ApiCompatibilityReport. Additional public routes: `komorebi_math.api.v1.ApiCompatibilityReport`, `komorebi_math.v1.ApiCompatibilityReport`.

AG-0015 `Certificate`. Strongest listed tier `[STABLE]`. Kinds: `class`. Owners: `komorebi_math.certificates`. Stable cards: A.2 Certificate. Beta cards: B.1 Certificate. Research cards: C.1A Certificate. Additional public routes: `komorebi_math.api.v1.Certificate`, `komorebi_math.v1.Certificate`.

AG-0023 `ComputationResult`. Strongest listed tier `[STABLE]`. Kinds: `class`. Owners: `komorebi_math.certificates`. Stable cards: A.2 ComputationResult. Beta cards: B.1 ComputationResult. Research cards: C.1A ComputationResult. Additional public routes: `komorebi_math.api.v1.ComputationResult`, `komorebi_math.v1.ComputationResult`.

AG-0024 `ConditionalResult`. Strongest listed tier `[STABLE]`. Kinds: `class`. Owners: `komorebi_math.certificates`. Stable cards: A.2 ConditionalResult. Beta cards: B.1 ConditionalResult. Research cards: C.1A ConditionalResult. Additional public routes: `komorebi_math.api.v1.ConditionalResult`, `komorebi_math.v1.ConditionalResult`.

AG-0035 `dumps_portable`. Strongest listed tier `[STABLE]`. Kinds: `function`. Owners: `komorebi_math.export.persistence`. Stable cards: A.8 dumps_portable. Beta cards: B.4 dumps_portable. Research cards: C.1F dumps_portable, C.6 dumps_portable. Additional public routes: `komorebi_math.api.v1.dumps_portable`, `komorebi_math.v1.dumps_portable`.

AG-0037 `Exactness`. Strongest listed tier `[STABLE]`. Kinds: `class`. Owners: `komorebi_math.certificates`. Stable cards: A.2 Exactness. Beta cards: B.1 Exactness. Research cards: C.1A Exactness. Additional public routes: `komorebi_math.api.v1.Exactness`, `komorebi_math.v1.Exactness`.

AG-0045 `from_portable_data`. Strongest listed tier `[STABLE]`. Kinds: `function`. Owners: `komorebi_math.export.persistence`. Stable cards: A.8 from_portable_data. Beta cards: B.4 from_portable_data. Research cards: C.6 from_portable_data. Additional public routes: `komorebi_math.api.v1.from_portable_data`, `komorebi_math.v1.from_portable_data`.

AG-0054 `load_portable`. Strongest listed tier `[STABLE]`. Kinds: `function`. Owners: `komorebi_math.export.persistence`. Stable cards: A.8 load_portable. Beta cards: B.4 load_portable. Research cards: C.6 load_portable, C.1F load_portable. Additional public routes: `komorebi_math.api.v1.load_portable`, `komorebi_math.v1.load_portable`.

AG-0058 `loads_portable`. Strongest listed tier `[STABLE]`. Kinds: `function`. Owners: `komorebi_math.export.persistence`. Stable cards: A.8 loads_portable. Beta cards: B.4 loads_portable. Research cards: C.6 loads_portable, C.1F loads_portable. Additional public routes: `komorebi_math.api.v1.loads_portable`, `komorebi_math.v1.loads_portable`.

AG-0072 `PacketSystem`. Strongest listed tier `[STABLE]`. Kinds: `class`. Owners: `komorebi_math.core.packets`. Stable cards: A.3 PacketSystem. Beta cards: B.1 PacketSystem. Research cards: C.1D PacketSystem. Additional public routes: `komorebi_math.api.v1.PacketSystem`, `komorebi_math.v1.PacketSystem`.

AG-0085 `portable_codecs`. Strongest listed tier `[STABLE]`. Kinds: `function`. Owners: `komorebi_math.export.persistence`. Stable cards: A.8 portable_codecs. Beta cards: B.4 portable_codecs. Research cards: C.6 portable_codecs, C.1F portable_codecs. Additional public routes: `komorebi_math.api.v1.portable_codecs`, `komorebi_math.v1.portable_codecs`.

AG-0086 `portable_digest`. Strongest listed tier `[STABLE]`. Kinds: `function`. Owners: `komorebi_math.export.persistence`. Stable cards: A.8 portable_digest. Beta cards: B.4 portable_digest. Research cards: C.6 portable_digest, C.1F portable_digest. Additional public routes: `komorebi_math.api.v1.portable_digest`, `komorebi_math.v1.portable_digest`.

AG-0087 `portable_migrations`. Strongest listed tier `[STABLE]`. Kinds: `function`. Owners: `komorebi_math.export.persistence`. Stable cards: A.8 portable_migrations. Beta cards: B.4 portable_migrations. Research cards: C.6 portable_migrations, C.1F portable_migrations. Additional public routes: `komorebi_math.api.v1.portable_migrations`, `komorebi_math.v1.portable_migrations`.

AG-0088 `PORTABLE_SCHEMA`. Strongest listed tier `[STABLE]`. Kinds: `data`. Owners: `builtins`. Stable cards: A.1 PORTABLE_SCHEMA. Beta cards: B.4 PORTABLE_SCHEMA. Research cards: C.6 PORTABLE_SCHEMA. Additional public routes: `komorebi_math.api.v1.PORTABLE_SCHEMA`, `komorebi_math.v1.PORTABLE_SCHEMA`.

AG-0089 **`PORTABLE_SCHEMA_VERSION`**. Strongest listed tier `[STABLE]`. Kinds: `data`. Owners: `builtins`. Stable cards: A.1 PORTABLE_SCHEMA_VERSION. Beta cards: B.4 PORTABLE_SCHEMA_VERSION. Research cards: C.6 PORTABLE_SCHEMA_VERSION. Additional public routes: `komorebi_math.api.v1.PORTABLE_SCHEMA_VERSION`, `komorebi_math.v1.PORTABLE_SCHEMA_VERSION`.

AG-0090 **`PortableCodec`**. Strongest listed tier `[STABLE]`. Kinds: `class`. Owners: `komorebi_math.export.persistence`. Stable cards: A.8 PortableCodec. Beta cards: B.4 PortableCodec. Research cards: C.1F PortableCodec, C.6 PortableCodec. Additional public routes: `komorebi_math.api.v1.PortableCodec`, `komorebi_math.v1.PortableCodec`.

AG-0094 **`PortableMigration`**. Strongest listed tier `[STABLE]`. Kinds: `class`. Owners: `komorebi_math.export.persistence`. Stable cards: A.8 PortableMigration. Beta cards: B.4 PortableMigration. Research cards: C.1F PortableMigration, C.6 PortableMigration. Additional public routes: `komorebi_math.api.v1.PortableMigration`, `komorebi_math.v1.PortableMigration`.

AG-0095 **`PublicApiManifest`**. Strongest listed tier `[STABLE]`. Kinds: `class`. Owners: `komorebi_math.compatibility`. Stable cards: A.9 PublicApiManifest. Beta cards: B.1 PublicApiManifest. Research cards: C.1A PublicApiManifest. Additional public routes: `komorebi_math.api.v1.PublicApiManifest`, `komorebi_math.v1.PublicApiManifest`.

AG-0096 **`PublicApiSymbol`**. Strongest listed tier `[STABLE]`. Kinds: `class`. Owners: `komorebi_math.compatibility`. Stable cards: A.9 PublicApiSymbol. Beta cards: B.1 PublicApiSymbol. Research cards: C.1A PublicApiSymbol. Additional public routes: `komorebi_math.api.v1.PublicApiSymbol`, `komorebi_math.v1.PublicApiSymbol`.

AG-0108 **`save_portable`**. Strongest listed tier `[STABLE]`. Kinds: `function`. Owners: `komorebi_math.export.persistence`. Stable cards: A.8 save_portable. Beta cards: B.4 save_portable. Research cards: C.6 save_portable, C.1F save_portable. Additional public routes: `komorebi_math.api.v1.save_portable`, `komorebi_math.v1.save_portable`.

AG-0123 **`to_portable_data`**. Strongest listed tier `[STABLE]`. Kinds: `function`. Owners: `komorebi_math.export.persistence`. Stable cards: A.8 to_portable_data. Beta cards: B.4 to_portable_data. Research cards: C.6 to_portable_data. Additional public routes: `komorebi_math.api.v1.to_portable_data`, `komorebi_math.v1.to_portable_data`.

C.10.2 Stable + Beta route groups

AG-0011 **`AssumptionError`**. Strongest listed tier `[STABLE]`. Kinds: `class`. Owners: `komorebi_math.exceptions`. Stable cards: A.2 AssumptionError. Beta cards: B.1 AssumptionError. Additional public routes: `komorebi_math.api.v1.AssumptionError`, `komorebi_math.v1.AssumptionError`.

AG-0012 **`BackendUnavailableError`**. Strongest listed tier `[STABLE]`. Kinds: `class`. Owners: `komorebi_math.exceptions`. Stable cards: A.2 BackendUnavailableError. Beta cards: B.1 BackendUnavailableError. Additional public routes: `komorebi_math.api.v1.BackendUnavailableError`, `komorebi_math.v1.BackendUnavailableError`.

AG-0033 **`DimensionError`**. Strongest listed tier `[STABLE]`. Kinds: `class`. Owners: `komorebi_math.exceptions`. Stable cards: A.2 DimensionError. Beta cards: B.1 DimensionError. Additional public routes: `komorebi_math.api.v1.DimensionError`, `komorebi_math.v1.DimensionError`.

AG-0038 **`ExactnessError`**. Strongest listed tier `[STABLE]`. Kinds: `class`. Owners: `komorebi_math.exceptions`. Stable cards: A.2 ExactnessError. Beta cards: B.1 ExactnessError. Additional public routes: `komorebi_math.api.v1.ExactnessError`, `komorebi_math.v1.ExactnessError`.

AG-0048 **`KomorebiMathError`**. Strongest listed tier `[STABLE]`. Kinds: `class`. Owners: `komorebi_math.exceptions`. Stable cards: A.2 KomorebiMathError. Beta cards: B.1 KomorebiMathError. Additional public routes: `komorebi_math.api.v1.KomorebiMathError`, `komorebi_math.v1.KomorebiMathError`.

AG-0126 **`ValidationError`**. Strongest listed tier `[STABLE]`. Kinds: `class`. Owners: `komorebi_math.exceptions`. Stable cards: A.2 ValidationError. Beta cards: B.1 ValidationError. Additional public routes: `komorebi_math.api.v1.ValidationError`, `komorebi_math.v1.ValidationError`.

C.10.3 Stable + Research route groups

AG-0002 `AnalysisSynthesisBlocks`. Strongest listed tier `[STABLE]`. Kinds: `class`. Owners: `komorebi_math.coordinates.analysis_synthesis`. Stable cards: A.4 AnalysisSynthesisBlocks. Research cards: C.1C AnalysisSynthesisBlocks, C.2C AnalysisSynthesisBlocks. Additional public routes: `komorebi_math.api.v1.AnalysisSynthesisBlocks`, `komorebi_math.v1.AnalysisSynthesisBlocks`.

AG-0003 `AnalysisSynthesisSystem`. Strongest listed tier `[STABLE]`. Kinds: `class`. Owners: `komorebi_math.coordinates.analysis_synthesis`. Stable cards: A.4 AnalysisSynthesisSystem. Research cards: C.1C AnalysisSynthesisSystem, C.2C AnalysisSynthesisSystem. Additional public routes: `komorebi_math.api.v1.AnalysisSynthesisSystem`, `komorebi_math.v1.AnalysisSynthesisSystem`.

AG-0010 `Arrow`. Strongest listed tier `[STABLE]`. Kinds: `class`. Owners: `komorebi_math.core.grammar`. Stable cards: A.3 Arrow. Research cards: C.1D Arrow. Additional public routes: `komorebi_math.api.v1.Arrow`, `komorebi_math.v1.Arrow`.

AG-0013 `basis_from_matrix`. Strongest listed tier `[STABLE]`. Kinds: `function`. Owners: `komorebi_math.coordinates.basis`. Stable cards: A.4 basis_from_matrix. Research cards: C.1C basis_from_matrix, C.2C basis_from_matrix. Additional public routes: `komorebi_math.api.v1.basis_from_matrix`, `komorebi_math.v1.basis_from_matrix`.

AG-0014 `block_from_paths`. Strongest listed tier `[STABLE]`. Kinds: `function`. Owners: `komorebi_math.core.transfer`. Stable cards: A.3 block_from_paths. Research cards: C.1D block_from_paths. Additional public routes: `komorebi_math.api.v1.block_from_paths`, `komorebi_math.v1.block_from_paths`.

AG-0021 `common_refinement`. Strongest listed tier `[STABLE]`. Kinds: `function`. Owners: `komorebi_math.refinement.towers`. Stable cards: A.5 common_refinement. Research cards: C.1G common_refinement. Additional public routes: `komorebi_math.api.v1.common_refinement`, `komorebi_math.v1.common_refinement`.

AG-0025 `configure`. Strongest listed tier `[STABLE]`. Kinds: `function`. Owners: `komorebi_math.config`. Stable cards: A.2 configure. Research cards: C.1A configure. Additional public routes: `komorebi_math.api.v1.configure`, `komorebi_math.v1.configure`.

AG-0026 `connection_transform`. Strongest listed tier `[STABLE]`. Kinds: `function`. Owners: `komorebi_math.coordinates.polynomials`. Stable cards: A.4 connection_transform. Research cards: C.1C connection_transform, C.2B connection_transform. Additional public routes: `komorebi_math.api.v1.connection_transform`, `komorebi_math.v1.connection_transform`.

AG-0027 `CoordinateAtlas`. Strongest listed tier `[STABLE]`. Kinds: `class`. Owners: `komorebi_math.coordinates.basis`. Stable cards: A.4 CoordinateAtlas. Research cards: C.1C CoordinateAtlas, C.2C CoordinateAtlas. Additional public routes: `komorebi_math.api.v1.CoordinateAtlas`, `komorebi_math.v1.CoordinateAtlas`.

AG-0028 `CoordinateTransform`. Strongest listed tier `[STABLE]`. Kinds: `class`. Owners: `komorebi_math.coordinates.transform`. Stable cards: A.4 CoordinateTransform, A.4 ExactCoordinateTransform. Research cards: C.1C CoordinateTransform, C.2C CoordinateTransform, C.2C ExactCoordinateTransform. Additional public routes: `komorebi_math.api.v1.CoordinateTransform`, `komorebi_math.api.v1.ExactCoordinateTransform`, `komorebi_math.v1.CoordinateTransform`, `komorebi_math.v1.ExactCoordinateTransform`.

AG-0034 `DocumentMetadata`. Strongest listed tier `[STABLE]`. Kinds: `class`. Owners: `komorebi_math.export.model`. Stable cards: A.8 DocumentMetadata. Research cards: C.1F DocumentMetadata, C.6 DocumentMetadata. Additional public routes: `komorebi_math.api.v1.DocumentMetadata`, `komorebi_math.v1.DocumentMetadata`.

AG-0036 `Equation`. Strongest listed tier `[STABLE]`. Kinds: `class`. Owners: `komorebi_math.export.model`. Stable cards: A.8 Equation. Research cards: C.1F Equation, C.6 Equation. Additional public routes: `komorebi_math.api.v1.Equation`, `komorebi_math.v1.Equation`.

AG-0039 `factorial_power_basis`. Strongest listed tier `[STABLE]`. Kinds: `function`. Owners: `komorebi_math.generating.basis`. Stable cards: A.6 factorial_power_basis. Research cards: C.1F factorial_power_basis, C.3A factorial_power_basis. Additional public routes: `komorebi_math.api.v1.factorial_power_basis`, `komorebi_math.v1.factorial_power_basis`.

AG-0040 `FiniteBasis`. Strongest listed tier `[STABLE]`. Kinds: `class`. Owners: `komorebi_math.coordinates.basis`. Stable cards: A.4 FiniteBasis. Research cards: C.1C FiniteBasis, C.2C FiniteBasis. Additional public routes: `komorebi_math.api.v1.FiniteBasis`, `komorebi_math.v1.FiniteBasis`.

AG-0041 `formal_coordinate_from_iterable`. Strongest listed tier `[STABLE]`. Kinds: `function`. Owners: `komorebi_math.coordinates.formal`. Stable cards: A.4 formal_coordinate_from_iterable. Research cards: C.2C formal_coordinate_

from_iterable. Additional public routes: `komorebi_math.api.v1.formal_coordinate_from_iterable`, `komorebi_math.v1.formal_coordinate_from_iterable`.

AG-0042 **`FormalCoordinateChange`**. Strongest listed tier `[STABLE]`. Kinds: `class`. Owners: `komorebi_math.coordinates.formal`. Stable cards: A.4 FormalCoordinateChange. Research cards: C.1C FormalCoordinateChange, C.2C FormalCoordinateChange. Additional public routes: `komorebi_math.api.v1.FormalCoordinateChange`, `komorebi_math.v1.FormalCoordinateChange`.

AG-0043 **`FormalSeries`**. Strongest listed tier `[STABLE]`. Kinds: `class`. Owners: `komorebi_math.generating.series`. Stable cards: A.6 FormalSeries. Research cards: C.1F FormalSeries, C.3A FormalSeries. Additional public routes: `komorebi_math.api.v1.FormalSeries`, `komorebi_math.v1.FormalSeries`.

AG-0044 **`forward_difference_transform`**. Strongest listed tier `[STABLE]`. Kinds: `function`. Owners: `komorebi_math.coordinates.combinatorial`. Stable cards: A.4 forward_difference_transform. Research cards: C.2A forward_difference_transform, C.1C forward_difference_transform. Additional public routes: `komorebi_math.api.v1.forward_difference_transform`, `komorebi_math.v1.forward_difference_transform`.

AG-0046 **`get_polynomial_family`**. Strongest listed tier `[STABLE]`. Kinds: `function`. Owners: `komorebi_math.coordinates.classical`. Stable cards: A.4 get_polynomial_family. Research cards: C.2B get_polynomial_family, C.1C get_polynomial_family. Additional public routes: `komorebi_math.api.v1.get_polynomial_family`, `komorebi_math.v1.get_polynomial_family`.

AG-0047 **`get_settings`**. Strongest listed tier `[STABLE]`. Kinds: `function`. Owners: `komorebi_math.config`. Stable cards: A.2 get_settings. Research cards: C.1A get_settings. Additional public routes: `komorebi_math.api.v1.get_settings`, `komorebi_math.v1.get_settings`.

AG-0049 **`lattice_polyphase_analyze`**. Strongest listed tier `[STABLE]`. Kinds: `function`. Owners: `komorebi_math.signals.polyphase`. Stable cards: A.7 lattice_polyphase_analyze. Research cards: C.4B lattice_polyphase_analyze. Additional public routes: `komorebi_math.api.v1.lattice_polyphase_analyze`, `komorebi_math.v1.lattice_polyphase_analyze`.

AG-0051 **`lattice_polyphase_synthesize`**. Strongest listed tier `[STABLE]`. Kinds: `function`. Owners: `komorebi_math.signals.polyphase`. Stable cards: A.7 lattice_polyphase_synthesize. Research cards: C.4B lattice_polyphase_synthesize. Additional public routes: `komorebi_math.api.v1.lattice_polyphase_synthesize`, `komorebi_math.v1.lattice_polyphase_synthesize`.

AG-0052 **`LatticePolyphaseCoordinates`**. Strongest listed tier `[STABLE]`. Kinds: `class`. Owners: `komorebi_math.signals.polyphase`. Stable cards: A.7 LatticePolyphaseCoordinates. Research cards: C.4B LatticePolyphaseCoordinates. Additional public routes: `komorebi_math.api.v1.LatticePolyphaseCoordinates`, `komorebi_math.v1.LatticePolyphaseCoordinates`.

AG-0060 **`MathDocument`**. Strongest listed tier `[STABLE]`. Kinds: `class`. Owners: `komorebi_math.export.model`. Stable cards: A.8 MathDocument. Research cards: C.1F MathDocument, C.6 MathDocument. Additional public routes: `komorebi_math.api.v1.MathDocument`, `komorebi_math.v1.MathDocument`.

AG-0063 **`NumericCoordinateTransform`**. Strongest listed tier `[STABLE]`. Kinds: `class`. Owners: `komorebi_math.coordinates.numeric`. Stable cards: A.4 NumericCoordinateTransform. Research cards: C.1C NumericCoordinateTransform, C.2C NumericCoordinateTransform. Additional public routes: `komorebi_math.api.v1.NumericCoordinateTransform`, `komorebi_math.v1.NumericCoordinateTransform`.

AG-0064 **`OperatorGrammar`**. Strongest listed tier `[STABLE]`. Kinds: `class`. Owners: `komorebi_math.core.grammar`. Stable cards: A.3 OperatorGrammar. Research cards: C.1D OperatorGrammar. Additional public routes: `komorebi_math.api.v1.OperatorGrammar`, `komorebi_math.v1.OperatorGrammar`.

AG-0065 **`OperatorRealization`**. Strongest listed tier `[STABLE]`. Kinds: `class`. Owners: `komorebi_math.core.realization`. Stable cards: A.3 OperatorRealization. Research cards: C.1D OperatorRealization. Additional public routes: `komorebi_math.api.v1.OperatorRealization`, `komorebi_math.v1.OperatorRealization`.

AG-0066 **`OperatorRelation`**. Strongest listed tier `[STABLE]`. Kinds: `class`. Owners: `komorebi_math.core.grammar`. Stable cards: A.3 OperatorRelation. Research cards: C.1D OperatorRelation. Additional public routes: `komorebi_math.api.v1.OperatorRelation`, `komorebi_math.v1.OperatorRelation`.

AG-0067 **`OperatorWord`**. Strongest listed tier `[STABLE]`. Kinds: `class`. Owners: `komorebi_math.core.grammar`. Stable cards: A.3 OperatorWord. Research cards: C.1D OperatorWord. Additional public routes: `komorebi_math.api.v1.OperatorWord`, `komorebi_math.v1.OperatorWord`.

AG-0068 **`ordinary_generating_basis`**. Strongest listed tier `[STABLE]`. Kinds: `function`. Owners: `komorebi_math.generating.basis`. Stable cards: A.6 ordinary_generating_basis. Research cards: C.2C ordinary_basis, C.3A ordinary_basis, C.1F ordinary_basis. Additional public routes: `komorebi_math.api.v1.ordinary_generating_basis`, `komorebi_math.v1.ordinary_generating_basis`.

AG-0069 **`PacketCoordinates`**. Strongest listed tier `[STABLE]`. Kinds: `class`. Owners: `komorebi_math.core.packets`. Stable cards: A.3 PacketCoordinates. Research cards: C.1D PacketCoordinates. Additional public routes: `komorebi_math.api.v1.PacketCoordinates`, `komorebi_math.v1.PacketCoordinates`.

AG-0070 **`PacketOperator`**. Strongest listed tier `[STABLE]`. Kinds: `class`. Owners: `komorebi_math.core.operators`. Stable cards: A.3 PacketOperator. Research cards: C.1D PacketOperator. Additional public routes: `komorebi_math.api.v1.PacketOperator`, `komorebi_math.v1.PacketOperator`.

AG-0071 **`PacketRefinement`**. Strongest listed tier `[STABLE]`. Kinds: `class`. Owners: `komorebi_math.refinement.towers`. Stable cards: A.5 PacketRefinement. Research cards: C.1G PacketRefinement. Additional public routes: `komorebi_math.api.v1.PacketRefinement`, `komorebi_math.v1.PacketRefinement`.

AG-0073 **`Paragraph`**. Strongest listed tier `[STABLE]`. Kinds: `class`. Owners: `komorebi_math.export.model`. Stable cards: A.8 Paragraph. Research cards: C.6 Paragraph. Additional public routes: `komorebi_math.api.v1.Paragraph`, `komorebi_math.v1.Paragraph`.

AG-0074 **`path_contributions`**. Strongest listed tier `[STABLE]`. Kinds: `function`. Owners: `komorebi_math.core.transfer`. Stable cards: A.3 path_contributions. Research cards: C.1D path_contributions. Additional public routes: `komorebi_math.api.v1.path_contributions`, `komorebi_math.v1.path_contributions`.

AG-0077 **`polyphase_roundtrip_certificate`**. Strongest listed tier `[STABLE]`. Kinds: `function`. Owners: `komorebi_math.signals.polyphase`. Stable cards: A.7 polyphase_roundtrip_certificate. Research cards: C.4B polyphase_roundtrip_certificate. Additional public routes: `komorebi_math.api.v1.polyphase_roundtrip_certificate`, `komorebi_math.v1.polyphase_roundtrip_certificate`.

AG-0079 **`PolyphaseCoordinates`**. Strongest listed tier `[STABLE]`. Kinds: `class`. Owners: `komorebi_math.signals.polyphase`. Stable cards: A.7 PolyphaseCoordinates. Research cards: C.4B PolyphaseCoordinates. Additional public routes: `komorebi_math.api.v1.PolyphaseCoordinates`, `komorebi_math.v1.PolyphaseCoordinates`.

AG-0082 **`PolyphaseSystem`**. Strongest listed tier `[STABLE]`. Kinds: `class`. Owners: `komorebi_math.signals.polyphase`. Stable cards: A.7 PolyphaseSystem. Research cards: C.1F PolyphaseSystem, C.4B PolyphaseSystem. Additional public routes: `komorebi_math.api.v1.PolyphaseSystem`, `komorebi_math.v1.PolyphaseSystem`.

AG-0097 **`refinement_children`**. Strongest listed tier `[STABLE]`. Kinds: `function`. Owners: `komorebi_math.signals.polyphase`. Stable cards: A.7 refinement_children. Research cards: C.4B refinement_children. Additional public routes: `komorebi_math.api.v1.refinement_children`, `komorebi_math.v1.refinement_children`.

AG-0098 **`RefinementTower`**. Strongest listed tier `[STABLE]`. Kinds: `class`. Owners: `komorebi_math.refinement.towers`. Stable cards: A.5 RefinementTower. Research cards: C.1G RefinementTower. Additional public routes: `komorebi_math.api.v1.RefinementTower`, `komorebi_math.v1.RefinementTower`.

AG-0100 **`render_latex`**. Strongest listed tier `[STABLE]`. Kinds: `function`. Owners: `komorebi_math.export.renderers`. Stable cards: A.8 render_latex. Research cards: C.6 render_latex, C.1F render_latex. Additional public routes: `komorebi_math.api.v1.render_latex`, `komorebi_math.v1.render_latex`.

AG-0101 **`render_markdown`**. Strongest listed tier `[STABLE]`. Kinds: `function`. Owners: `komorebi_math.export.renderers`. Stable cards: A.8 render_markdown. Research cards: C.6 render_markdown. Additional public routes: `komorebi_math.api.v1.render_markdown`, `komorebi_math.v1.render_markdown`.

AG-0102 **`residue_analyze`**. Strongest listed tier `[STABLE]`. Kinds: `function`. Owners: `komorebi_math.refinement.residue`. Stable cards: A.5 residue_analyze. Research cards: C.1G residue_analyze. Additional public routes: `komorebi_math.api.v1.residue_analyze`, `komorebi_math.v1.residue_analyze`.

AG-0103 **`residue_project`**. Strongest listed tier `[STABLE]`. Kinds: `function`. Owners: `komorebi_math.signals.polyphase`. Stable cards: A.7 residue_project. Research cards: C.4B residue_project. Additional public routes: `komorebi_math.api.v1.residue_project`, `komorebi_math.v1.residue_project`.

AG-0110 **`SeedSystem`**. Strongest listed tier `[STABLE]`. Kinds: `class`. Owners: `komorebi_math.core.seeds`. Stable cards: A.3 SeedSystem. Research cards: C.1D SeedSystem. Additional public routes: `komorebi_math.api.v1.SeedSystem`, `komorebi_math.v1.SeedSystem`.

AG-0111 **`Settings`**. Strongest listed tier `[STABLE]`. Kinds: `class`. Owners: `komorebi_math.config`. Stable cards: A.2 Settings. Research cards: C.1A Settings. Additional public routes: `komorebi_math.api.v1.Settings`, `komorebi_math.v1.Settings`.

AG-0112 **`settings_context`**. Strongest listed tier `[STABLE]`. Kinds: `function`. Owners: `komorebi_math.config`. Stable cards: A.2 settings_context. Research cards: C.1A settings_context. Additional public routes: `komorebi_math.api.v1.settings_context`, `komorebi_math.v1.settings_context`.

AG-0113 **`signal_polyphase_analyze`**. Strongest listed tier `[STABLE]`. Kinds: `function`. Owners: `komorebi_math.signals.polyphase`. Stable cards: A.7 signal_polyphase_analyze. Research cards: C.1F polyphase_analyze, C.4B polyphase_analyze. Additional public routes: `komorebi_math.api.v1.signal_polyphase_analyze`, `komorebi_math.v1.signal_polyphase_analyze`.

AG-0114 **`signal_polyphase_synthesize`**. Strongest listed tier `[STABLE]`. Kinds: `function`. Owners: `komorebi_math.signals.polyphase`. Stable cards: A.7 signal_polyphase_synthesize. Research cards: C.4B polyphase_synthesize. Additional public routes: `komorebi_math.api.v1.signal_polyphase_synthesize`, `komorebi_math.v1.signal_polyphase_synthesize`.

AG-0122 **`standard_basis`**. Strongest listed tier `[STABLE]`. Kinds: `function`. Owners: `komorebi_math.coordinates.basis`. Stable cards: A.4 standard_basis. Research cards: C.2C standard_basis, C.1C standard_basis. Additional public routes: `komorebi_math.api.v1.standard_basis`, `komorebi_math.v1.standard_basis`.

AG-0124 **`TransferGraph`**. Strongest listed tier `[STABLE]`. Kinds: `class`. Owners: `komorebi_math.core.graphs`. Stable cards: A.3 TransferGraph. Research cards: C.1D TransferGraph. Additional public routes: `komorebi_math.api.v1.TransferGraph`, `komorebi_math.v1.TransferGraph`.

C.10.4 Stable route groups

AG-0005 **`arithmetic_mixed_radix_address`**. Strongest listed tier `[STABLE]`. Kinds: `function`. Owners: `komorebi_math.arithmetic.polyphase`. Stable cards: A.5 arithmetic_mixed_radix_address. Additional public routes: `komorebi_math.api.v1.arithmetic_mixed_radix_address`, `komorebi_math.v1.arithmetic_mixed_radix_address`.

AG-0006 **`arithmetic_mixed_radix_digits`**. Strongest listed tier `[STABLE]`. Kinds: `function`. Owners: `komorebi_math.arithmetic.polyphase`. Stable cards: A.5 arithmetic_mixed_radix_digits. Additional public routes: `komorebi_math.api.v1.arithmetic_mixed_radix_digits`, `komorebi_math.v1.arithmetic_mixed_radix_digits`.

AG-0007 **`arithmetic_polyphase_analyze`**. Strongest listed tier `[STABLE]`. Kinds: `function`. Owners: `komorebi_math.arithmetic.polyphase`. Stable cards: A.5 arithmetic_polyphase_analyze. Additional public routes: `komorebi_math.api.v1.arithmetic_polyphase_analyze`, `komorebi_math.v1.arithmetic_polyphase_analyze`.

AG-0008 **`arithmetic_polyphase_synthesize`**. Strongest listed tier `[STABLE]`. Kinds: `function`. Owners: `komorebi_math.arithmetic.polyphase`. Stable cards: A.5 arithmetic_polyphase_synthesize. Additional public routes: `komorebi_math.api.v1.arithmetic_polyphase_synthesize`, `komorebi_math.v1.arithmetic_polyphase_synthesize`.

AG-0009 **`ArithmeticPolyphase`**. Strongest listed tier `[STABLE]`. Kinds: `class`. Owners: `komorebi_math.arithmetic.polyphase`. Stable cards: A.5 ArithmeticPolyphase. Additional public routes: `komorebi_math.api.v1.ArithmeticPolyphase`, `komorebi_math.v1.ArithmeticPolyphase`.

AG-0016 **`check_portable_codec_contract`**. Strongest listed tier `[STABLE]`. Kinds: `function`. Owners: `komorebi_math.persistence_contract`. Stable cards: A.9 check_portable_codec_contract. Additional public routes: `komorebi_math.api.v1.check_portable_codec_contract`, `komorebi_math.v1.check_portable_codec_contract`.

AG-0017 **`check_stable_api`**. Strongest listed tier `[STABLE]`. Kinds: `function`. Owners: `komorebi_math.compatibility`. Stable cards: A.9 check_stable_api. Additional public routes: `komorebi_math.api.v1.check_stable_api`, `komorebi_math.v1.check_stable_api`.

AG-0018 **`check_stable_contract`**. Strongest listed tier `[STABLE]`. Kinds: `function`. Owners: `komorebi_math.stable_contract`. Stable cards: A.9 check_stable_contract. Additional public routes: `komorebi_math.api.v1.check_stable_contract`, `komorebi_math.v1.check_stable_contract`.

AG-0019 **`coarsen_lattice_polyphase`**. Strongest listed tier `[STABLE]`. Kinds: `function`. Owners: `komorebi_math.refinement.polyphase`. Stable cards: A.5 coarsen_lattice_polyphase, A.5 lattice_polyphase_coarsen. Additional public routes: `komorebi_math.api.v1.coarsen_lattice_polyphase`, `komorebi_math.api.v1.lattice_polyphase_coarsen`, `komorebi_math.v1.coarsen_lattice_polyphase`, `komorebi_math.v1.lattice_polyphase_coarsen`.

AG-0020 **`coarsen_polyphase`**. Strongest listed tier `[STABLE]`. Kinds: `function`. Owners: `komorebi_math.refinement.polyphase`. Stable cards: A.5 coarsen_polyphase, A.5 polyphase_coarsen. Additional public routes: `komorebi_math.api.v1.coarsen_polyphase`, `komorebi_math.api.v1.polyphase_coarsen`, `komorebi_math.v1.coarsen_polyphase`, `komorebi_math.v1.polyphase_coarsen`.

AG-0022 **`compare_portable_codec_contract`**. Strongest listed tier `[STABLE]`. Kinds: `function`. Owners: `komorebi_math.persistence_contract`. Stable cards: A.9 compare_portable_codec_contract. Additional public routes: `komorebi_math.api.v1.compare_portable_codec_contract`, `komorebi_math.v1.compare_portable_codec_contract`.

AG-0029 **`create_portable_codec_contract`**. Strongest listed tier `[STABLE]`. Kinds: `function`. Owners: `komorebi_math.persistence_contract`. Stable cards: A.9 create_portable_codec_contract. Additional public routes: `komorebi_math.api.v1.create_portable_codec_contract`, `komorebi_math.v1.create_portable_codec_contract`.

AG-0030 **`create_stable_api_manifest`**. Strongest listed tier `[STABLE]`. Kinds: `function`. Owners: `komorebi_math.compatibility`. Stable cards: A.9 create_stable_api_manifest. Additional public routes: `komorebi_math.api.v1.create_stable_api_manifest`, `komorebi_math.v1.create_stable_api_manifest`.

AG-0031 **`DEFAULT_PORTABLE_CODEC_CONTRACT`**. Strongest listed tier `[STABLE]`. Kinds: `data`. Owners: `pathlib`. Stable cards: A.1 DEFAULT_PORTABLE_CODEC_CONTRACT. Additional public routes: `komorebi_math.api.v1.DEFAULT_PORTABLE_CODEC_CONTRACT`, `komorebi_math.v1.DEFAULT_PORTABLE_CODEC_CONTRACT`.

AG-0032 **`DEFAULT_STABLE_SEMANTICS_FIXTURE`**. Strongest listed tier `[STABLE]`. Kinds: `data`. Owners: `pathlib`. Stable cards: A.1 DEFAULT_STABLE_SEMANTICS_FIXTURE. Additional public routes: `komorebi_math.api.v1.DEFAULT_STABLE_SEMANTICS_FIXTURE`, `komorebi_math.v1.DEFAULT_STABLE_SEMANTICS_FIXTURE`.

AG-0050 **`lattice_polyphase_refine`**. Strongest listed tier `[STABLE]`. Kinds: `function`. Owners: `komorebi_math.refinement.polyphase`. Stable cards: A.5 lattice_polyphase_refine, A.5 refine_lattice_polyphase. Additional public routes: `komorebi_math.api.v1.lattice_polyphase_refine`, `komorebi_math.api.v1.refine_lattice_polyphase`, `komorebi_math.v1.lattice_polyphase_refine`, `komorebi_math.v1.refine_lattice_polyphase`.

AG-0053 **`LatticePolyphaseRefinement`**. Strongest listed tier `[STABLE]`. Kinds: `class`. Owners: `komorebi_math.refinement.polyphase`. Stable cards: A.5 LatticePolyphaseRefinement. Additional public routes: `komorebi_math.api.v1.LatticePolyphaseRefinement`, `komorebi_math.v1.LatticePolyphaseRefinement`.

AG-0055 **`load_portable_codec_contract`**. Strongest listed tier `[STABLE]`. Kinds: `function`. Owners: `komorebi_math.persistence_contract`. Stable cards: A.9 load_portable_codec_contract. Additional public routes: `komorebi_math.api.v1.load_portable_codec_contract`, `komorebi_math.v1.load_portable_codec_contract`.

AG-0056 **`load_stable_api_manifest`**. Strongest listed tier `[STABLE]`. Kinds: `function`. Owners: `komorebi_math.compatibility`. Stable cards: A.9 load_stable_api_manifest. Additional public routes: `komorebi_math.api.v1.load_stable_api_manifest`, `komorebi_math.v1.load_stable_api_manifest`.

AG-0057 **`load_stable_semantics_fixture`**. Strongest listed tier `[STABLE]`. Kinds: `function`. Owners: `komorebi_math.stable_semantics`. Stable cards: A.9 load_stable_semantics_fixture. Additional public routes: `komorebi_math.api.v1.load_stable_semantics_fixture`, `komorebi_math.v1.load_stable_semantics_fixture`.

AG-0059 **`mahler_polyphase`**. Strongest listed tier `[STABLE]`. Kinds: `function`. Owners: `komorebi_math.arithmetic.polyphase`. Stable cards: A.5 mahler_polyphase. Additional public routes: `komorebi_math.api.v1.mahler_polyphase`, `komorebi_math.v1.mahler_polyphase`.

AG-0061 **`nested_residue_address`**. Strongest listed tier `[STABLE]`. Kinds: `function`. Owners: `komorebi_math.refinement.residue`. Stable cards: A.5 nested_residue_address. Additional public routes: `komorebi_math.api.v1.nested_residue_address`, `komorebi_math.v1.nested_residue_address`.

AG-0062 **`nested_residue_analysis`**. Strongest listed tier `[STABLE]`. Kinds: `function`. Owners: `komorebi_math.refinement.residue`. Stable cards: A.5 nested_residue_analysis. Additional public routes: `komorebi_math.api.v1.nested_residue_analysis`, `komorebi_math.v1.nested_residue_analysis`.

AG-0075 **`polyphase_refine`**. Strongest listed tier `[STABLE]`. Kinds: `function`. Owners: `komorebi_math.refinement.polyphase`. Stable cards: A.5 polyphase_refine, A.5 refine_polyphase. Additional public routes: `komorebi_math.api.v1.polyphase_refine`, `komorebi_math.api.v1.refine_polyphase`, `komorebi_math.v1.polyphase_refine`, `komorebi_math.v1.refine_polyphase`.

AG-0076 **`polyphase_refinement_certificate`**. Strongest listed tier `[STABLE]`. Kinds: `function`. Owners: `komorebi_math.refinement.polyphase`. Stable cards: A.5 polyphase_refinement_certificate. Additional public routes: `komorebi_math.api.v1.polyphase_refinement_certificate`, `komorebi_math.v1.polyphase_refinement_certificate`.

AG-0078 **`PolyphaseCoordinateRecord`**. Strongest listed tier `[STABLE]`. Kinds: `data`. Owners: `types`. Stable cards: A.5 PolyphaseCoordinateRecord. Additional public routes: `komorebi_math.api.v1.PolyphaseCoordinateRecord`, `komorebi_math.v1.PolyphaseCoordinateRecord`.

AG-0080 **`PolyphaseRefinement`**. Strongest listed tier `[STABLE]`. Kinds: `class`. Owners: `komorebi_math.refinement.polyphase`. Stable cards: A.5 PolyphaseRefinement. Additional public routes: `komorebi_math.api.v1.PolyphaseRefinement`, `komorebi_math.v1.PolyphaseRefinement`.

AG-0081 **`PolyphaseRefinementTower`**. Strongest listed tier `[STABLE]`. Kinds: `class`. Owners: `komorebi_math.refinement.polyphase`. Stable cards: A.5 PolyphaseRefinementTower. Additional public routes: `komorebi_math.api.v1.PolyphaseRefinementTower`, `komorebi_math.v1.PolyphaseRefinementTower`.

AG-0083 **`PORTABLE_CODEC_CONTRACT_SCHEMA`**. Strongest listed tier `[STABLE]`. Kinds: `data`. Owners: `builtins`. Stable cards: A.1 PORTABLE_CODEC_CONTRACT_SCHEMA. Additional public routes: `komorebi_math.api.v1.PORTABLE_CODEC_CONTRACT_SCHEMA`, `komorebi_math.v1.PORTABLE_CODEC_CONTRACT_SCHEMA`.

AG-0084 **`PORTABLE_CODEC_CONTRACT_SCHEMA_VERSION`**. Strongest listed tier `[STABLE]`. Kinds: `data`. Owners: `builtins`. Stable cards: A.1 PORTABLE_CODEC_CONTRACT_SCHEMA_VERSION. Additional public routes: `komorebi_math.api.v1.PORTABLE_CODEC_CONTRACT_SCHEMA_VERSION`, `komorebi_math.v1.PORTABLE_CODEC_CONTRACT_SCHEMA_VERSION`.

AG-0091 **`PortableCodecCompatibilityReport`**. Strongest listed tier `[STABLE]`. Kinds: `class`. Owners: `komorebi_math.persistence_contract`. Stable cards: A.9 PortableCodecCompatibilityReport. Additional public routes: `komorebi_math.api.v1.PortableCodecCompatibilityReport`, `komorebi_math.v1.PortableCodecCompatibilityReport`.

AG-0092 **`PortableCodecContract`**. Strongest listed tier `[STABLE]`. Kinds: `class`. Owners: `komorebi_math.persistence_contract`. Stable cards: A.9 PortableCodecContract. Additional public routes: `komorebi_math.api.v1.PortableCodecContract`, `komorebi_math.v1.PortableCodecContract`.

AG-0093 **`PortableCodecContractEntry`**. Strongest listed tier `[STABLE]`. Kinds: `class`. Owners: `komorebi_math.persistence_contract`. Stable cards: A.9 PortableCodecContractEntry. Additional public routes: `komorebi_math.api.v1.PortableCodecContractEntry`, `komorebi_math.v1.PortableCodecContractEntry`.

AG-0099 **`RelationResidual`**. Strongest listed tier `[STABLE]`. Kinds: `class`. Owners: `komorebi_math.core.realization`. Stable cards: A.3 RelationResidual. Additional public routes: `komorebi_math.api.v1.RelationResidual`, `komorebi_math.v1.RelationResidual`.

AG-0104 **`residue_synthesize`**. Strongest listed tier `[STABLE]`. Kinds: `function`. Owners: `komorebi_math.refinement.residue`. Stable cards: A.5 residue_synthesize. Additional public routes: `komorebi_math.api.v1.residue_synthesize`, `komorebi_math.v1.residue_synthesize`.

AG-0105 **`ResidueCoordinates`**. Strongest listed tier `[STABLE]`. Kinds: `class`. Owners: `komorebi_math.refinement.residue`. Stable cards: A.5 ResidueCoordinates. Additional public routes: `komorebi_math.api.v1.ResidueCoordinates`, `komorebi_math.v1.ResidueCoordinates`.

AG-0106 **`ResourceLimitError`**. Strongest listed tier `[STABLE]`. Kinds: `class`. Owners: `komorebi_math.exceptions`. Stable cards: A.2 ResourceLimitError. Additional public routes: `komorebi_math.api.v1.ResourceLimitError`, `komorebi_math.v1.ResourceLimitError`.

AG-0107 **`run_stable_semantics_fixture`**. Strongest listed tier `[STABLE]`. Kinds: `function`. Owners: `komorebi_math.stable_semantics`. Stable cards: A.9 run_stable_semantics_fixture. Additional public routes: `komorebi_math.api.v1.run_stable_semantics_fixture`, `komorebi_math.v1.run_stable_semantics_fixture`.

AG-0109 **`save_portable_codec_contract`**. Strongest listed tier `[STABLE]`. Kinds: `function`. Owners: `komorebi_math.persistence_contract`. Stable cards: A.9 save_portable_codec_contract. Additional public routes: `komorebi_math.api.v1.save_portable_codec_contract`, `komorebi_math.v1.save_portable_codec_contract`.

AG-0115 **`STABLE_API_STABILITY_LEVEL`**. Strongest listed tier `[STABLE]`. Kinds: `data`. Owners: `builtins`. Stable cards: A.1 STABLE_API_STABILITY_LEVEL. Additional public routes: `komorebi_math.api.v1.STABLE_API_STABILITY_LEVEL`, `komorebi_math.v1.STABLE_API_STABILITY_LEVEL`.

AG-0116 **`STABLE_PUBLIC_NAMESPACES`**. Strongest listed tier `[STABLE]`. Kinds: `data`. Owners: `builtins`. Stable cards: A.1 STABLE_PUBLIC_NAMESPACES. Additional public routes: `komorebi_math.api.v1.STABLE_PUBLIC_NAMESPACES`, `komorebi_math.v1.STABLE_PUBLIC_NAMESPACES`.

AG-0117 **`STABLE_SEMANTICS_SCHEMA`**. Strongest listed tier `[STABLE]`. Kinds: `data`. Owners: `builtins`. Stable cards: A.1 STABLE_SEMANTICS_SCHEMA. Additional public routes: `komorebi_math.api.v1.STABLE_SEMANTICS_SCHEMA`, `komorebi_math.v1.STABLE_SEMANTICS_SCHEMA`.

AG-0118 **`STABLE_SEMANTICS_SCHEMA_VERSION`**. Strongest listed tier `[STABLE]`. Kinds: `data`. Owners: `builtins`. Stable cards: A.1 STABLE_SEMANTICS_SCHEMA_VERSION. Additional public routes: `komorebi_math.api.v1.STABLE_SEMANTICS_SCHEMA_VERSION`, `komorebi_math.v1.STABLE_SEMANTICS_SCHEMA_VERSION`.

AG-0119 **`StableContractReport`**. Strongest listed tier `[STABLE]`. Kinds: `class`. Owners: `komorebi_math.stable_contract`. Stable cards: A.9 StableContractReport. Additional public routes: `komorebi_math.api.v1.StableContractReport`, `komorebi_math.v1.StableContractReport`.

AG-0120 **`StableSemanticCaseResult`**. Strongest listed tier `[STABLE]`. Kinds: `class`. Owners: `komorebi_math.stable_semantics`. Stable cards: A.9 StableSemanticCaseResult. Additional public routes: `komorebi_math.api.v1.StableSemanticCaseResult`, `komorebi_math.v1.StableSemanticCaseResult`.

AG-0121 **`StableSemanticReport`**. Strongest listed tier `[STABLE]`. Kinds: `class`. Owners: `komorebi_math.stable_semantics`. Stable cards: A.9 StableSemanticReport. Additional public routes: `komorebi_math.api.v1.StableSemanticReport`, `komorebi_math.v1.StableSemanticReport`.

AG-0125 **`TransferPath`**. Strongest listed tier `[STABLE]`. Kinds: `class`. Owners: `komorebi_math.core.transfer`. Stable cards: A.3 TransferPath. Additional public routes: `komorebi_math.api.v1.TransferPath`, `komorebi_math.v1.TransferPath`.

C.10.5 Beta + Research route groups

AG-0127 **`BEHAVIORAL_GOLDEN_SCHEMA`**. Strongest listed tier `[BETA]`. Kinds: `data`. Owners: `builtins`. Beta cards: B.1 BEHAVIORAL_GOLDEN_SCHEMA. Research cards: C.1A BEHAVIORAL_GOLDEN_SCHEMA.

AG-0128 **`BEHAVIORAL_GOLDEN_SCHEMA_VERSION`**. Strongest listed tier `[BETA]`. Kinds: `data`. Owners: `builtins`. Beta cards: B.1 BEHAVIORAL_GOLDEN_SCHEMA_VERSION. Research cards: C.1A BEHAVIORAL_GOLDEN_SCHEMA_VERSION.

AG-0129 **`BehavioralGoldenCaseResult`**. Strongest listed tier `[BETA]`. Kinds: `class`. Owners: `komorebi_math.golden`. Beta cards: B.1 BehavioralGoldenCaseResult. Research cards: C.1A BehavioralGoldenCaseResult.

AG-0130 **`BehavioralGoldenReport`**. Strongest listed tier `[BETA]`. Kinds: `class`. Owners: `komorebi_math.golden`. Beta cards: B.1 BehavioralGoldenReport. Research cards: C.1A BehavioralGoldenReport.

AG-0131 **`BETA_API_STABILITY_LEVEL`**. Strongest listed tier `[BETA]`. Kinds: `data`. Owners: `builtins`. Beta cards: B.1 BETA_API_STABILITY_LEVEL. Research cards: C.1A BETA_API_STABILITY_LEVEL.

AG-0132 **`CallableDerivativeProvider`**. Strongest listed tier `[BETA]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.derivatives`. Beta cards: B.2 CallableDerivativeProvider. Research cards: C.1E CallableDerivativeProvider, C.5A CallableDerivativeProvider.

AG-0133 **`capture_packet_jet_stream`**. Strongest listed tier `[BETA]`. Kinds: `function`. Owners: `komorebi_math.nonlinear.streaming`. Beta cards: B.3 capture_packet_jet_stream. Research cards: C.1E capture_packet_jet_stream, C.5B capture_packet_jet_stream.

AG-0134 **`check_beta_api`**. Strongest listed tier `[BETA]`. Kinds: `function`. Owners: `komorebi_math.compatibility`. Beta cards: B.1 check_beta_api. Research cards: C.1A check_beta_api.

AG-0135 **`compile_packet_jet_plan`**. Strongest listed tier `[BETA]`. Kinds: `function`. Owners: `komorebi_math.nonlinear.execution`. Beta cards: B.2 compile_packet_jet_plan. Research cards: C.1E compile_packet_jet_plan, C.5B compile_packet_jet_plan.

AG-0136 **`create_beta_api_manifest`**. Strongest listed tier `[BETA]`. Kinds: `function`. Owners: `komorebi_math.compatibility`. Beta cards: B.1 create_beta_api_manifest. Research cards: C.1A create_beta_api_manifest.

AG-0137 **`DerivativeProvider`**. Strongest listed tier `[BETA]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.derivatives`. Beta cards: B.2 DerivativeProvider. Research cards: C.5A DerivativeProvider.

AG-0138 **`estimate_packet_jet_complexity`**. Strongest listed tier `[BETA]`. Kinds: `function`. Owners: `komorebi_math.nonlinear.profiling`. Beta cards: B.2 estimate_packet_jet_complexity. Research cards: C.1E estimate_packet_jet_complexity, C.5B estimate_packet_jet_complexity.

AG-0139 **`execute_packet_jet_plan`**. Strongest listed tier `[BETA]`. Kinds: `function`. Owners: `komorebi_math.nonlinear.execution`. Beta cards: B.2 execute_packet_jet_plan. Research cards: C.1E execute_packet_jet_plan, C.5B execute_packet_jet_plan.

AG-0140 **`execute_packet_jets`**. Strongest listed tier `[BETA]`. Kinds: `function`. Owners: `komorebi_math.nonlinear.execution`. Beta cards: B.2 execute_packet_jets. Research cards: C.1E execute_packet_jets, C.5B execute_packet_jets.

AG-0141 **`FiniteDifferenceDerivativeProvider`**. Strongest listed tier `[BETA]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.derivatives`. Beta cards: B.2 FiniteDifferenceDerivativeProvider. Research cards: C.1E FiniteDifferenceDerivativeProvider, C.5A FiniteDifferenceDerivativeProvider.

AG-0142 **`load_behavioral_golden_fixture`**. Strongest listed tier `[BETA]`. Kinds: `function`. Owners: `komorebi_math.golden`. Beta cards: B.1 load_behavioral_golden_fixture. Research cards: C.1A load_behavioral_golden_fixture.

AG-0143 **`load_beta_api_manifest`**. Strongest listed tier `[BETA]`. Kinds: `function`. Owners: `komorebi_math.compatibility`. Beta cards: B.1 load_beta_api_manifest. Research cards: C.1A load_beta_api_manifest.

AG-0144 **`load_packet_jet_snapshot_stream`**. Strongest listed tier `[BETA]`. Kinds: `function`. Owners: `komorebi_math.nonlinear.streaming`. Beta cards: B.3 load_packet_jet_snapshot_stream. Research cards: C.1E load_packet_jet_snapshot_stream, C.5B load_packet_jet_snapshot_stream.

AG-0145 **`open_packet_jet_snapshot_stream`**. Strongest listed tier `[BETA]`. Kinds: `function`. Owners: `komorebi_math.nonlinear.streaming`. Beta cards: B.3 open_packet_jet_snapshot_stream. Research cards: C.5B open_packet_jet_snapshot_stream, C.1E open_packet_jet_snapshot_stream.

AG-0146 **`PACKET_JET_CHUNK_SCHEMA`**. Strongest listed tier `[BETA]`. Kinds: `data`. Owners: `builtins`. Beta cards: B.3 PACKET_JET_CHUNK_SCHEMA. Research cards: C.1A PACKET_JET_CHUNK_SCHEMA, C.5B PACKET_JET_CHUNK_SCHEMA.

AG-0147 **`PACKET_JET_CHUNK_SCHEMA_VERSION`**. Strongest listed tier `[BETA]`. Kinds: `data`. Owners: `builtins`. Beta cards: B.3 PACKET_JET_CHUNK_SCHEMA_VERSION. Research cards: C.1A PACKET_JET_CHUNK_SCHEMA_VERSION, C.5B PACKET_JET_CHUNK_SCHEMA_VERSION.

AG-0148 **`PACKET_JET_STREAM_SCHEMA`**. Strongest listed tier `[BETA]`. Kinds: `data`. Owners: `builtins`. Beta cards: B.3 PACKET_JET_STREAM_SCHEMA. Research cards: C.1A PACKET_JET_STREAM_SCHEMA, C.5B PACKET_JET_STREAM_SCHEMA.

AG-0149 **`PACKET_JET_STREAM_SCHEMA_VERSION`**. Strongest listed tier `[BETA]`. Kinds: `data`. Owners: `builtins`. Beta cards: B.3 PACKET_JET_STREAM_SCHEMA_VERSION. Research cards: C.1A PACKET_JET_STREAM_SCHEMA_VERSION, C.5B PACKET_JET_STREAM_SCHEMA_VERSION.

AG-0150 **`packet_jet_support`**. Strongest listed tier `[BETA]`. Kinds: `function`. Owners: `komorebi_math.nonlinear.jets`. Beta cards: B.2 packet_jet_support. Research cards: C.5B packet_jet_support, C.1E packet_jet_support.

AG-0151 **`PacketJet`**. Strongest listed tier `[BETA]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.jets`. Beta cards: B.2 PacketJet. Research cards: C.1E PacketJet, C.5B PacketJet.

AG-0152 **`PacketJetAddress`**. Strongest listed tier `[BETA]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.jets`. Beta cards: B.2 PacketJetAddress. Research cards: C.5B PacketJetAddress.

AG-0153 **`PacketJetBatchResult`**. Strongest listed tier `[BETA]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.execution`. Beta cards: B.2 PacketJetBatchResult. Research cards: C.1E PacketJetBatchResult, C.5B PacketJetBatchResult.

AG-0154 **`PacketJetCoefficient`**. Strongest listed tier `[BETA]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.snapshots`. Beta cards: B.3 PacketJetCoefficient. Research cards: C.5B PacketJetCoefficient.

AG-0155 **`PacketJetComplexityEstimate`**. Strongest listed tier `[BETA]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.profiling`. Beta cards: B.2 PacketJetComplexityEstimate. Research cards: C.1E PacketJetComplexityEstimate, C.5B PacketJetComplexityEstimate.

AG-0156 **`PacketJetEvaluation`**. Strongest listed tier `[BETA]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.execution`. Beta cards: B.2 PacketJetEvaluation. Research cards: C.5B PacketJetEvaluation.

AG-0157 `PacketJetExecutionPlan`. Strongest listed tier `[BETA]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.execution`. Beta cards: B.2 PacketJetExecutionPlan. Research cards: C.1E PacketJetExecutionPlan, C.5B PacketJetExecutionPlan.

AG-0158 `PacketJetFamily`. Strongest listed tier `[BETA]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.jets`. Beta cards: B.2 PacketJetFamily. Research cards: C.1E PacketJetFamily, C.5B PacketJetFamily.

AG-0159 `PacketJetInputGroup`. Strongest listed tier `[BETA]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.execution`. Beta cards: B.2 PacketJetInputGroup. Research cards: C.5B PacketJetInputGroup.

AG-0160 `PacketJetProfileReport`. Strongest listed tier `[BETA]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.profiling`. Beta cards: B.2 PacketJetProfileReport. Research cards: C.1E PacketJetProfileReport, C.5B PacketJetProfileReport.

AG-0161 `PacketJetSnapshot`. Strongest listed tier `[BETA]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.snapshots`. Beta cards: B.3 PacketJetSnapshot. Research cards: C.1E PacketJetSnapshot, C.5B PacketJetSnapshot.

AG-0162 `PacketJetSnapshotStreamReader`. Strongest listed tier `[BETA]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.streaming`. Beta cards: B.3 PacketJetSnapshotStreamReader. Research cards: C.1E PacketJetSnapshotStreamReader, C.5B PacketJetSnapshotStreamReader.

AG-0163 `PacketJetStreamChunkRecord`. Strongest listed tier `[BETA]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.streaming`. Beta cards: B.3 PacketJetStreamChunkRecord. Research cards: C.1E PacketJetStreamChunkRecord, C.5B PacketJetStreamChunkRecord.

AG-0164 `PacketJetStreamManifest`. Strongest listed tier `[BETA]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.streaming`. Beta cards: B.3 PacketJetStreamManifest. Research cards: C.1E PacketJetStreamManifest, C.5B PacketJetStreamManifest.

AG-0165 `PacketJetStreamSelection`. Strongest listed tier `[BETA]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.streaming`. Beta cards: B.3 PacketJetStreamSelection. Research cards: C.1E PacketJetStreamSelection, C.5B PacketJetStreamSelection.

AG-0166 `PacketJetStreamWriteResult`. Strongest listed tier `[BETA]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.streaming`. Beta cards: B.3 PacketJetStreamWriteResult. Research cards: C.1E PacketJetStreamWriteResult, C.5B PacketJetStreamWriteResult.

AG-0167 `PacketJetSupport`. Strongest listed tier `[BETA]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.jets`. Beta cards: B.2 PacketJetSupport. Research cards: C.5B PacketJetSupport.

AG-0168 `PolynomialDerivativeProvider`. Strongest listed tier `[BETA]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.derivatives`. Beta cards: B.2 PolynomialDerivativeProvider. Research cards: C.1E PolynomialDerivativeProvider, C.5A PolynomialDerivativeProvider.

AG-0169 `profile_operation`. Strongest listed tier `[BETA]`. Kinds: `function`. Owners: `komorebi_math.nonlinear.profiling`. Beta cards: B.2 profile_operation. Research cards: C.5B profile_operation, C.1E profile_operation.

AG-0170 `profile_packet_jet_runtime`. Strongest listed tier `[BETA]`. Kinds: `function`. Owners: `komorebi_math.nonlinear.profiling`. Beta cards: B.2 profile_packet_jet_runtime. Research cards: C.5B profile_packet_jet_runtime, C.1E profile_packet_jet_runtime.

AG-0171 `register_portable_codec`. Strongest listed tier `[BETA]`. Kinds: `function`. Owners: `komorebi_math.export.persistence`. Beta cards: B.4 register_portable_codec. Research cards: C.6 register_portable_codec, C.1F register_portable_codec.

AG-0172 `register_portable_migration`. Strongest listed tier `[BETA]`. Kinds: `function`. Owners: `komorebi_math.export.persistence`. Beta cards: B.4 register_portable_migration. Research cards: C.6 register_portable_migration, C.1F register_portable_migration.

AG-0173 `run_behavioral_golden_fixture`. Strongest listed tier `[BETA]`. Kinds: `function`. Owners: `komorebi_math.golden`. Beta cards: B.1 run_behavioral_golden_fixture. Research cards: C.1A run_behavioral_golden_fixture.

AG-0174 `RuntimeProfileSample`. Strongest listed tier `[BETA]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.profiling`. Beta cards: B.2 RuntimeProfileSample. Research cards: C.1E RuntimeProfileSample, C.5B RuntimeProfileSample.

AG-0175 `RuntimeStageProfile`. Strongest listed tier `[BETA]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.profiling`. Beta cards: B.2 RuntimeStageProfile. Research cards: C.1E RuntimeStageProfile, C.5B RuntimeStageProfile.

AG-0176 **`save_packet_jet_snapshot_stream`**. Strongest listed tier `[BETA]`. Kinds: `function`. Owners: `komorebi_math.nonlinear.streaming`. Beta cards: B.3 save_packet_jet_snapshot_stream. Research cards: C.5B save_packet_jet_snapshot_stream, C.1E save_packet_jet_snapshot_stream.

AG-0177 **`snapshot_packet_jet`**. Strongest listed tier `[BETA]`. Kinds: `function`. Owners: `komorebi_math.nonlinear.snapshots`. Beta cards: B.3 snapshot_packet_jet. Research cards: C.5B snapshot_packet_jet, C.1E snapshot_packet_jet.

AG-0178 **`SparsePolynomialDerivativeProvider`**. Strongest listed tier `[BETA]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.sparse`. Beta cards: B.2 SparsePolynomialDerivativeProvider. Research cards: C.1E SparsePolynomialDerivativeProvider, C.5B SparsePolynomialDerivativeProvider.

AG-0179 **`SparseSymmetricTensor`**. Strongest listed tier `[BETA]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.sparse`. Beta cards: B.2 SparseSymmetricTensor. Research cards: C.1E SparseSymmetricTensor, C.5B SparseSymmetricTensor.

AG-0180 **`SparseTensorEntry`**. Strongest listed tier `[BETA]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.sparse`. Beta cards: B.2 SparseTensorEntry. Research cards: C.1E SparseTensorEntry, C.5B SparseTensorEntry.

AG-0181 **`StreamedPacketJetEvaluation`**. Strongest listed tier `[BETA]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.streaming`. Beta cards: B.3 StreamedPacketJetEvaluation. Research cards: C.1E StreamedPacketJetEvaluation, C.5B StreamedPacketJetEvaluation.

C.10.6 Research route groups

AG-0182 **`AdaptiveFilterResult`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.signals.adaptive`. Research cards: C.4B AdaptiveFilterResult.

AG-0183 **`AdaptiveMethod`**. Strongest listed tier `[RESEARCH]`. Kinds: `data`. Owners: `typing`. Research cards: C.4A AdaptiveMethod.

AG-0184 **`add`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.expression.model`. Research cards: C.1B add.

AG-0185 **`additive_series`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.monoid`. Research cards: C.3A additive_series.

AG-0186 **`affine_basis_transform`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.affine`. Research cards: C.2C affine_basis_transform.

AG-0187 **`affine_interval_change_of_variables`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.integration.identities`. Research cards: C.1D affine_interval_change_of_variables.

AG-0188 **`affine_mahler_kernel`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.arithmetic.mahler`. Research cards: C.1G affine_mahler_kernel.

AG-0189 **`affine_polynomial`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.classical`. Research cards: C.2B affine_polynomial.

AG-0190 **`affine_substitution`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.expression.polynomial`. Research cards: C.1B affine_substitution.

AG-0191 **`affine_substitution_coordinate_operator`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.calculus`. Research cards: C.2B affine_substitution_coordinate_operator.

AG-0192 **`affine_substitution_transform`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.classical`. Research cards: C.2B affine_substitution_transform.

AG-0193 **`AffineCoordinateTransform`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.coordinates.affine`. Research cards: C.1C AffineCoordinateTransform, C.2C AffineCoordinateTransform.

AG-0194 **`AlgebraBasisResult`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.core.algebra`. Research cards: C.1D AlgebraBasisResult.

AG-0195 **`algebraic_function`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.functions`. Research cards: C.3A algebraic_function.

AG-0196 **`AlgebraicGFGuess`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.generating.closed_forms`. Research cards: C.3A AlgebraicGFGuess.

AG-0197 `all_fubini_orders`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.integration.identities`. Research cards: C.1D all_fubini_orders.

AG-0198 `AlternatingForm`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.langlands.local.symplectic`. Research cards: C.1G AlternatingForm.

AG-0199 `analysis_synthesis_integration_certificate`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.integration.adapters`. Research cards: C.1D analysis_synthesis_integration_certificate.

AG-0200 `AnalysisSynthesisCoefficientBlocks`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.coordinates.frame_algebra`. Research cards: C.1C AnalysisSynthesisCoefficientBlocks, C.2C AnalysisSynthesisCoefficientBlocks.

AG-0201 `AnalysisSynthesisGauge`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.coordinates.frame_algebra`. Research cards: C.1C AnalysisSynthesisGauge, C.2C AnalysisSynthesisGauge.

AG-0202 `AnalysisSynthesisMap`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.coordinates.frame_algebra`. Research cards: C.1C AnalysisSynthesisMap, C.2C AnalysisSynthesisMap.

AG-0203 `analytic_mahler_product`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.masks`. Research cards: C.3B analytic_mahler_product.

AG-0204 `AnalyticMaskApproximation`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.generating.masks`. Research cards: C.3B AnalyticMaskApproximation.

AG-0205 `analyze_recurrence_coordinates`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.recurrence`. Research cards: C.1C analyze_recurrence_coordinates, C.2C analyze_recurrence_coordinates.

AG-0206 `antiderivative`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.expression.polynomial`. Research cards: C.1B antiderivative.

AG-0207 `apery_numbers`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.sequences`. Research cards: C.3A apery_numbers.

AG-0208 `apery_sequence`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.sequences.catalog`. Research cards: C.1G apery_sequence.

AG-0209 `API_STABILITY_LEVEL`. Strongest listed tier `[RESEARCH]`. Kinds: `data`. Owners: `builtins`. Research cards: C.1A API_STABILITY_LEVEL.

AG-0210 `appell_polynomial`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.analytic.appell`. Research cards: C.1G appell_polynomial.

AG-0211 `appell_polynomial`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.classical`. Research cards: C.2B appell_polynomial.

AG-0212 `appell_polynomial_family`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.classical`. Research cards: C.2B appell_polynomial_family.

AG-0213 `Apply`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.expression.model`. Research cards: C.1B Apply.

AG-0214 `apply`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.expression.model`. Research cards: C.1B apply.

AG-0215 `apply_lowering_polynomial`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.normalized`. Research cards: C.2A apply_lowering_polynomial.

AG-0216 `apply_recurrence_defect`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.recurrence`. Research cards: C.2C apply_recurrence_defect.

AG-0217 `as_matrix`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.exact`. Research cards: C.2A as_matrix.

AG-0218 `as_vector`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.exact`. Research cards: C.2A as_vector.

AG-0219 `assign_discrete_precisions`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.algorithms.precision`. Research cards: C.1D assign_discrete_precisions.

AG-0220 **`AtlasPath`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.coordinates.basis`. Research cards: C.2C AtlasPath.

AG-0221 **`AtomicIntegratedDerivativeProvider`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.integration.adapters`. Research cards: C.1D AtomicIntegratedDerivativeProvider.

AG-0222 **`AtomicMeasure`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.integration.measures`. Research cards: C.1D AtomicMeasure.

AG-0223 **`audit_apery_supercongruence`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.congruences`. Research cards: C.3B audit_apery_supercongruence.

AG-0224 **`audit_factorial_power_digit_algebra`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.digit_algebra`. Research cards: C.3B audit_factorial_power_digit_algebra.

AG-0225 **`audit_factorial_power_packet_convolution`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.congruences`. Research cards: C.3B audit_factorial_power_packet_convolution.

AG-0226 **`audit_generalized_bell_digit_sum`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.congruences`. Research cards: C.3B audit_generalized_bell_digit_sum.

AG-0227 **`audit_generalized_lucas`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.congruences`. Research cards: C.3B audit_generalized_lucas.

AG-0228 **`audit_partition_lane`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.congruences`. Research cards: C.3B audit_partition_lane.

AG-0229 **`audit_q_lucas`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.congruences`. Research cards: C.3B audit_q_lucas.

AG-0230 **`audit_section_cycle`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.sections`. Research cards: C.3A audit_section_cycle.

AG-0231 **`audit_touchard`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.congruences`. Research cards: C.3B audit_touchard.

AG-0232 **`audit_weighted_generalized_bell_digit_sum`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.congruences`. Research cards: C.3B audit_weighted_generalized_bell_digit_sum.

AG-0233 **`autodiff_provider`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.nonlinear.autodiff`. Research cards: C.1E autodiff_provider, C.1E automatic_derivative_provider, C.5A autodiff_provider, C.5A automatic_derivative_provider.

AG-0234 **`AutomaticDifferentiationProvider`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.autodiff`. Research cards: C.1E AutomaticDifferentiationProvider, C.5A AutomaticDifferentiationProvider.

AG-0235 **`AutomaticSequence`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.sequences.automatic`. Research cards: C.1G AutomaticSequence.

AG-0236 **`AutomatonCertificate`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.generating.cartier`. Research cards: C.3B AutomatonCertificate.

AG-0237 **`AutomorphicCompletionReport`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.langlands.completion.gates`. Research cards: C.1G AutomorphicCompletionReport.

AG-0238 **`AutomorphicInductionPlan`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.langlands.finite_image.plans`. Research cards: C.1G AutomorphicInductionPlan.

AG-0239 **`available_ad_frameworks`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.nonlinear.autodiff`. Research cards: C.1E available_ad_frameworks, C.1E available_autodiff_backends, C.5A available_ad_frameworks, C.5A available_autodiff_backends.

AG-0240 **`available_export_formats`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.export.writers`. Research cards: C.1F available_export_formats, C.6 available_export_formats.

AG-0241 **`available_latex_engines`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.export.latex`. Research cards: C.1F available_latex_engines, C.6 available_latex_engines.

AG-0242 **`BarycentricCoordinateSystem`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.coordinates.affine`. Research cards: C.2C BarycentricCoordinateSystem.

AG-0243 **`base_digit_sum`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.congruences`. Research cards: C.3B base_digit_sum.

AG-0244 **`base_digits`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.digit_algebra`. Research cards: C.3B base_digits.

AG-0245 **`basis_from_descriptor`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.basis`. Research cards: C.3A basis_from_descriptor.

AG-0246 **`basis_from_values`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.basis`. Research cards: C.3A basis_from_values.

AG-0247 **`BasisCertificate`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.generating.basis`. Research cards: C.3A BasisCertificate.

AG-0248 **`BasisDescriptor`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.generating.basis`. Research cards: C.3A BasisDescriptor.

AG-0249 **`BayesianUpdate`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.dynamics.bayes`. Research cards: C.1G BayesianUpdate.

AG-0250 **`bell_numbers`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.sequences`. Research cards: C.3A bell_numbers.

AG-0251 **`bell_period_bound`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.bell`. Research cards: C.3B bell_period_bound.

AG-0252 **`BellDigitReduction`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.generating.bell`. Research cards: C.3B BellDigitReduction.

AG-0253 **`BellParityAutomaton`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.generating.bell`. Research cards: C.1F BellParityAutomaton, C.3B BellParityAutomaton.

AG-0254 **`BellResidueAutomaton`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.generating.bell`. Research cards: C.1F BellResidueAutomaton, C.3B BellResidueAutomaton.

AG-0255 **`berlekamp_massey`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.congruences`. Research cards: C.3B berlekamp_massey.

AG-0256 **`bernoulli_number`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.classical`. Research cards: C.2B bernoulli_number.

AG-0257 **`bernoulli_numbers`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.sequences`. Research cards: C.3A bernoulli_numbers.

AG-0258 **`bernoulli_polynomial`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.classical`. Research cards: C.2B bernoulli_polynomial.

AG-0259 **`bernoulli_polynomial_family`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.classical`. Research cards: C.2B bernoulli_polynomial_family.

AG-0260 **`bernstein_basis`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.classical`. Research cards: C.2B bernstein_basis.

AG-0261 **`bernstein_polynomial`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.classical`. Research cards: C.2B bernstein_polynomial.

AG-0262 **`best_basis`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.wavelets`. Research cards: C.4B best_basis.

AG-0263 **`BestBasisResult`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.signals.wavelets`. Research cards: C.4B BestBasisResult.

AG-0264 **`BETA_PUBLIC_NAMESPACES`**. Strongest listed tier `[RESEARCH]`. Kinds: `data`. Owners: `builtins`. Research cards: C.1A BETA_PUBLIC_NAMESPACES.

AG-0265 **`bezout_defect_identity`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.operators`. Research cards: C.1C bezout_defect_identity, C.2C bezout_defect_identity.

AG-0266 **`BezoutDefectIdentity`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.coordinates.operators`. Research cards: C.1C BezoutDefectIdentity, C.2C BezoutDefectIdentity.

AG-0267 **`binomial_polynomial`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.classical`. Research cards: C.2B binomial_polynomial.

AG-0268 **`binomial_polynomial_family`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.classical`. Research cards: C.2B binomial_polynomial_family.

AG-0269 **`binomial_transform`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.combinatorial`. Research cards: C.1C binomial_transform, C.2A binomial_transform.

AG-0270 **`binomial_valuation`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.padic_units`. Research cards: C.3B binomial_valuation.

AG-0271 **`BiquadKind`**. Strongest listed tier `[RESEARCH]`. Kinds: `data`. Owners: `typing`. Research cards: C.4A BiquadKind.

AG-0272 **`BlockDiagonalLowRankOperator`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.algorithms.low_rank`. Research cards: C.1D BlockDiagonalLowRankOperator.

AG-0273 **`BlockDiagonalOperator`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.algorithms.structured`. Research cards: C.1D BlockDiagonalOperator.

AG-0274 **`borrow_count`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.padic_units`. Research cards: C.3B borrow_count.

AG-0275 **`boundary_functional`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.expression.polynomial`. Research cards: C.1B boundary_functional.

AG-0276 **`BoundaryMode`**. Strongest listed tier `[RESEARCH]`. Kinds: `data`. Owners: `typing`. Research cards: C.4A BoundaryMode.

AG-0277 **`BreakpointBandResult`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.signals.recovery`. Research cards: C.4B BreakpointBandResult.

AG-0278 **`build_lift_tree`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.nonlinear.lifting`. Research cards: C.1E build_lift_tree, C.5A build_lift_tree.

AG-0279 **`canonical_dual_window`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.windows`. Research cards: C.4A canonical_dual_window.

AG-0280 **`CarryLayerConvolution`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.generating.masks`. Research cards: C.3B CarryLayerConvolution.

AG-0281 **`CarryLinearCertificate`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.generating.normalization`. Research cards: C.3B CarryLinearCertificate.

AG-0282 **`CarryLinearValuation`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.generating.normalization`. Research cards: C.3B CarryLinearValuation.

AG-0283 **`cartier_degree_bound`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.cartier`. Research cards: C.3B cartier_degree_bound.

AG-0284 **`CartierAutomaton`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.generating.cartier`. Research cards: C.3B CartierAutomaton.

AG-0285 **`CartierCarryEquivalence`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.generating.cartier`. Research cards: C.3B CartierCarryEquivalence.

AG-0286 **`CartierLinearRepresentation`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.generating.cartier`. Research cards: C.3B CartierLinearRepresentation.

AG-0287 **`catalan_number`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.nonlinear.trees`. Research cards: C.5B catalan_number.

AG-0288 **`catalan_numbers`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.sequences`. Research cards: C.3A catalan_numbers.

AG-0289 **`central_binomial_numbers`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.sequences`. Research cards: C.3A central_binomial_numbers.

AG-0290 `certificate_document`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.export.report`. Research cards: C.6 certificate_document.

AG-0291 `certified_overlap_add_convolve`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.fft_convolution`. Research cards: C.4A certified_overlap_add_convolve.

AG-0292 `certify_cartier_carry_equivalence`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.cartier`. Research cards: C.3B certify_cartier_carry_equivalence.

AG-0293 `certify_mahler_functional_equation`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.masks`. Research cards: C.3B certify_mahler_functional_equation.

AG-0294 `certify_pointed_subindex_identity`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.bell`. Research cards: C.3B certify_pointed_subindex_identity.

AG-0295 `certify_rational_mask_algebraic_relation`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.masks`. Research cards: C.3B certify_rational_mask_algebraic_relation.

AG-0296 `certify_state_coordinate_change`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.state_space`. Research cards: C.2C certify_state_coordinate_change.

AG-0297 `characteristic_closed_form`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.closed_forms`. Research cards: C.3A characteristic_closed_form.

AG-0298 `chebyshev_t_polynomial`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.classical`. Research cards: C.2B chebyshev_t_polynomial.

AG-0299 `chebyshev_t_polynomial_family`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.classical`. Research cards: C.2B chebyshev_t_polynomial_family.

AG-0300 `chebyshev_u_polynomial`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.classical`. Research cards: C.2B chebyshev_u_polynomial.

AG-0301 `chebyshev_u_polynomial_family`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.classical`. Research cards: C.2B chebyshev_u_polynomial_family.

AG-0302 `check_expression`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.expression.engine`. Research cards: C.1B check_expression.

AG-0303 `check_public_api`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.compatibility`. Research cards: C.1A check_public_api.

AG-0304 `CheckedExpression`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.expression.engine`. Research cards: C.1B CheckedExpression.

AG-0305 `chinese_remainder_order`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.packet`. Research cards: C.2A chinese_remainder_order.

AG-0306 `chinese_remainder_transform`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.packet`. Research cards: C.2A chinese_remainder_transform.

AG-0307 `CircuitPacketExpansion`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.quantum.circuits`. Research cards: C.1G CircuitPacketExpansion.

AG-0308 `CirculantOperator`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.algorithms.structured`. Research cards: C.1D CirculantOperator.

AG-0309 `ClaimStatus`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.provenance`. Research cards: C.1A ClaimStatus.

AG-0310 `classical_polynomial_catalog`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.classical`. Research cards: C.1C classical_polynomial_catalog, C.2B classical_polynomial_catalog.

AG-0311 `classify_carry_linear_valuation`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.normalization`. Research cards: C.3B classify_carry_linear_valuation.

AG-0312 `CliffordStep`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.langlands.finite_image.clifford`. Research cards: C.1G CliffordStep.

AG-0313 `CliffordTower`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.langlands.finite_image.clifford`. Research cards: C.1G CliffordTower.

AG-0314 **`CodeBlock`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.export.model`. Research cards: C.6 CodeBlock.

AG-0315 **`coefficient_packets`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.nonlinear`. Research cards: C.4B coefficient_packets.

AG-0316 **`CoefficientDomain`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.generating.domains`. Research cards: C.3A CoefficientDomain.

AG-0317 **`CoefficientPartition`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.coordinates.frame_algebra`. Research cards: C.1C CoefficientPartition, C.2C CoefficientPartition.

AG-0318 **`CommutingDiagramCheck`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.coordinates.basis`. Research cards: C.2C CommutingDiagramCheck.

AG-0319 **`companion_matrix`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.recurrence`. Research cards: C.1C companion_matrix, C.2C companion_matrix.

AG-0320 **`compare_public_api`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.compatibility`. Research cards: C.1A compare_public_api.

AG-0321 **`CompatiblePropagationReport`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.langlands.completion.gates`. Research cards: C.1G CompatiblePropagationReport.

AG-0322 **`compile_cartier_linear_representation`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.cartier`. Research cards: C.3B compile_cartier_linear_representation.

AG-0323 **`compile_digit_cauchy_product`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.masks`. Research cards: C.3B compile_digit_cauchy_product.

AG-0324 **`compile_expression`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.expression.engine`. Research cards: C.1B compile_expression.

AG-0325 **`compile_finite_mask_carry_plan`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.masks`. Research cards: C.1F compile_finite_mask_carry_plan, C.3B compile_finite_mask_carry_plan.

AG-0326 **`compile_latex`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.export.latex`. Research cards: C.1F compile_latex, C.6 compile_latex.

AG-0327 **`compile_rational_mask_automaton`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.cartier`. Research cards: C.1F compile_rational_mask_automaton, C.3B compile_rational_mask_automaton.

AG-0328 **`compile_sequence`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.sequences.compiler`. Research cards: C.1G compile_sequence.

AG-0329 **`compile_tree_budget`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.algorithms.compiler`. Research cards: C.1D compile_tree_budget.

AG-0330 **`compile_tree_lagrangian`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.algorithms.compiler`. Research cards: C.1D compile_tree_lagrangian.

AG-0331 **`CompiledSequence`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.sequences.compiler`. Research cards: C.1G CompiledSequence.

AG-0332 **`CompletionGate`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.langlands.completion.gates`. Research cards: C.1G CompletionGate.

AG-0333 **`Compose`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.expression.model`. Research cards: C.1B Compose.

AG-0334 **`compose`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.expression.model`. Research cards: C.1B compose.

AG-0335 **`compose_certificates`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.expression.certification`. Research cards: C.1B compose_certificates.

AG-0336 **`compose_functions`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.functions`. Research cards: C.3A compose_functions.

AG-0337 **`compose_ordinary_series`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.formal`. Research cards: C.2C compose_ordinary_series.

AG-0338 **`ComposedDerivativeProvider`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.composition`. Research cards: C.1E ComposedDerivativeProvider, C.5A ComposedDerivativeProvider.

AG-0339 **`CompositeSimpson`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.integration.quadrature`. Research cards: C.1D CompositeSimpson.

AG-0340 **`conditional_projective_limit_completion`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.nonlinear.filtered`. Research cards: C.1E conditional_projective_limit_completion, C.5A conditional_projective_limit_completion.

AG-0341 **`conjugate_gradient`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.algorithms.krylov`. Research cards: C.1D conjugate_gradient.

AG-0342 **`constant_injection`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.expression.polynomial`. Research cards: C.1B constant_injection.

AG-0343 **`constant_polynomial`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.classical`. Research cards: C.2B constant_polynomial.

AG-0344 **`constraint_closure`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.quantum.constraints`. Research cards: C.1G constraint_closure.

AG-0345 **`contragredient_transform`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.tensor`. Research cards: C.2A contragredient_transform.

AG-0346 **`controllability_basis`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.state_space`. Research cards: C.2C controllability_basis.

AG-0347 **`controllability_coordinate_transform`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.state_space`. Research cards: C.2C controllability_coordinate_transform.

AG-0348 **`controllability_matrix`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.state_space`. Research cards: C.2C controllability_matrix.

AG-0349 **`convert_normalized_coefficients`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.normalized`. Research cards: C.2A convert_normalized_coefficients.

AG-0350 **`convolution_matrix`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.convolution`. Research cards: C.4A convolution_matrix.

AG-0351 **`convolution_output_length`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.convolution`. Research cards: C.4A convolution_output_length.

AG-0352 **`ConvolutionMode`**. Strongest listed tier `[RESEARCH]`. Kinds: `data`. Owners: `typing`. Research cards: C.4A ConvolutionMode.

AG-0353 **`convolve_signal`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.convolution`. Research cards: C.4A convolve_signal.

AG-0354 **`coordinate_change`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.expression.polynomial`. Research cards: C.1B coordinate_change.

AG-0355 **`coordinate_change_between_recurrences`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.recurrence`. Research cards: C.1C coordinate_change_between_recurrences, C.2C coordinate_change_between_recurrences.

AG-0356 **`coordinate_fundamental_theorem`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.integration.coordinates`. Research cards: C.1D coordinate_fundamental_theorem.

AG-0357 **`coordinate_pairing`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.tensor`. Research cards: C.2A coordinate_pairing.

AG-0358 **`coordinate_transport_identity`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.integration.coordinates`. Research cards: C.1D coordinate_transport_identity.

AG-0359 **`CoordinateBudget`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.coordinates.budget`. Research cards: C.1C CoordinateBudget, C.2C CoordinateBudget.

AG-0360 `CoordinateSpace`. Strongest listed tier `[RESEARCH]`. Kinds: `data`. Owners: `types`. Research cards: C.1A CoordinateSpace.

AG-0361 `CoordinateSystem`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.coordinates.basis`. Research cards: C.1C CoordinateSystem, C.2C CoordinateSystem.

AG-0362 `create_public_api_manifest`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.compatibility`. Research cards: C.1A create_public_api_manifest.

AG-0363 `cross_spectrum`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.spectral`. Research cards: C.4A cross_spectrum.

AG-0364 `CrossSpectrum`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.signals.spectral`. Research cards: C.4A CrossSpectrum.

AG-0365 `crt_many`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.refinement.crt`. Research cards: C.1G crt_many.

AG-0366 `crt_pair`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.refinement.crt`. Research cards: C.1G crt_pair.

AG-0367 `cupy_runtime_status`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.nonlinear.gpu`. Research cards: C.1E cupy_runtime_status, C.5B cupy_runtime_status.

AG-0368 `CuPyDeviceDerivativeProvider`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.execution`. Research cards: C.1E CuPyDeviceDerivativeProvider, C.5B CuPyDeviceDerivativeProvider.

AG-0369 `CuPyPacketJetExecutor`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.execution`. Research cards: C.1E CuPyPacketJetExecutor, C.5B CuPyPacketJetExecutor.

AG-0370 `CuPyPacketJetSnapshotExecutor`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.gpu`. Research cards: C.1E CuPyPacketJetSnapshotExecutor, C.1E CuPySnapshotExecutor, C.5B CuPyPacketJetSnapshotExecutor, C.5B CuPySnapshotExecutor.

AG-0371 `CuPyPolynomialDerivativeProvider`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.execution`. Research cards: C.1E CuPyPolynomialDerivativeProvider, C.5B CuPyPolynomialDerivativeProvider.

AG-0372 `CuPyRuntimeStatus`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.gpu`. Research cards: C.1E CuPyRuntimeStatus, C.5B CuPyRuntimeStatus.

AG-0373 `CuPyValidationResult`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.gpu`. Research cards: C.1E CuPyValidationResult, C.5B CuPyValidationResult.

AG-0374 `custom_exponential`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.sections`. Research cards: C.3A custom_exponential.

AG-0375 `custom_recurrence_model`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.families`. Research cards: C.2C custom_recurrence_model.

AG-0376 `custom_section`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.sections`. Research cards: C.3A custom_section.

AG-0377 `CustomFunctionSpec`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.generating.functions`. Research cards: C.1F CustomFunctionSpec, C.3A CustomFunctionSpec.

AG-0378 `cyclic_group`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.langlands.finite_image.groups`. Research cards: C.1G cyclic_group.

AG-0379 `cyclic_krylov_matrix`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.realization`. Research cards: C.2C cyclic_krylov_matrix.

AG-0380 `cyclic_project_coefficients`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.analytic.cyclic`. Research cards: C.1G cyclic_project_coefficients.

AG-0381 `cyclic_span`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.core.seeds`. Research cards: C.1D cyclic_span.

AG-0382 `cyclic_symmetry_projectors`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.quantum.symmetry`. Research cards: C.1G cyclic_symmetry_projectors.

AG-0383 `CyclicDenominator`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.analytic.zeros`. Research cards: C.1G CyclicDenominator.

AG-0384 `CyclicSectionFamily`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.analytic.sections`. Research cards: C.1G CyclicSectionFamily.

AG-0385 `cyclotomic_coboundary`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.cyclotomic`. Research cards: C.3B cyclotomic_coboundary.

AG-0386 `CyclotomicCollapse`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.generating.cyclotomic`. Research cards: C.3B CyclotomicCollapse.

AG-0387 `decimate`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.resampling`. Research cards: C.4A decimate.

AG-0388 `DEFAULT_BEHAVIORAL_GOLDEN_FIXTURE`. Strongest listed tier `[RESEARCH]`. Kinds: `data`. Owners: `pathlib`. Research cards: C.1G DEFAULT_BEHAVIORAL_GOLDEN_FIXTURE.

AG-0389 `DEFAULT_BETA_API_MANIFEST`. Strongest listed tier `[RESEARCH]`. Kinds: `data`. Owners: `pathlib`. Research cards: C.1G DEFAULT_BETA_API_MANIFEST.

AG-0390 `DEFAULT_COORDINATE_BUDGET`. Strongest listed tier `[RESEARCH]`. Kinds: `data`. Owners: `komorebi_math.coordinates.budget`. Research cards: C.2C DEFAULT_COORDINATE_BUDGET.

AG-0391 `DEFAULT_GENERATING_BUDGET`. Strongest listed tier `[RESEARCH]`. Kinds: `data`. Owners: `komorebi_math.generating.budget`. Research cards: C.3A DEFAULT_GENERATING_BUDGET.

AG-0392 `defect_green_weights`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.recurrence`. Research cards: C.2C defect_green_weights.

AG-0393 `denoising_risk_certificate`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.denoise`. Research cards: C.4B denoising_risk_certificate.

AG-0394 `denominator`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.analytic.sections`. Research cards: C.1G denominator.

AG-0395 `DenseOperator`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.algorithms.structured`. Research cards: C.1D DenseOperator.

AG-0396 `deprecated`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.compatibility`. Research cards: C.1A deprecated.

AG-0397 `DepthObstructionResult`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.filtered`. Research cards: C.5A DepthObstructionResult.

AG-0398 `derivative`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.expression.polynomial`. Research cards: C.1B derivative.

AG-0399 `derivative_coordinate_operator`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.calculus`. Research cards: C.2B derivative_coordinate_operator.

AG-0400 `derivative_operator_in_family`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.polynomials`. Research cards: C.2B derivative_operator_in_family.

AG-0401 `DerivativeBound`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.integration.quadrature`. Research cards: C.1D DerivativeBound.

AG-0402 `describe`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.provenance`. Research cards: C.1A describe.

AG-0403 `design_bandpass`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.filtering`. Research cards: C.4A design_bandpass.

AG-0404 `design_bandstop`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.filtering`. Research cards: C.4A design_bandstop.

AG-0405 `design_biquad`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.recursive`. Research cards: C.4A design_biquad.

AG-0406 `design_fir`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.filtering`. Research cards: C.4A design_fir.

AG-0407 **`design_highpass`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.filtering`. Research cards: C.4A design_highpass.

AG-0408 **`design_lowpass`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.filtering`. Research cards: C.4A design_lowpass.

AG-0409 **`design_resample_filter`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.resampling`. Research cards: C.4A design_resample_filter.

AG-0410 **`detect_piecewise_polynomial_breaks`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.recovery`. Research cards: C.4B detect_piecewise_polynomial_breaks.

AG-0411 **`determinant`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.exact`. Research cards: C.2A determinant.

AG-0412 **`DetrendMode`**. Strongest listed tier `[RESEARCH]`. Kinds: `data`. Owners: `typing`. Research cards: C.4A DetrendMode.

AG-0413 **`difference_noise_bound`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.singular_packets`. Research cards: C.4B difference_noise_bound.

AG-0414 **`difference_valuation_profile`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.congruences`. Research cards: C.3B difference_valuation_profile.

AG-0415 **`digit_factorized_normalized_kernel`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.normalization`. Research cards: C.3B digit_factorized_normalized_kernel.

AG-0416 **`digit_reversal_order`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.packet`. Research cards: C.2A digit_reversal_order.

AG-0417 **`digit_reversal_transform`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.packet`. Research cards: C.2A digit_reversal_transform.

AG-0418 **`digit_unit`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.normalization`. Research cards: C.3B digit_unit.

AG-0419 **`DigitAlgebraAudit`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.generating.digit_algebra`. Research cards: C.3B DigitAlgebraAudit.

AG-0420 **`DigitCauchyProductPlan`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.generating.masks`. Research cards: C.3B DigitCauchyProductPlan.

AG-0421 **`DigitKernel`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.generating.digit_algebra`. Research cards: C.3B DigitKernel.

AG-0422 **`direct_packet_rotation`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.nonlinear.transport`. Research cards: C.1E direct_packet_rotation, C.5A direct_packet_rotation.

AG-0423 **`direct_sum_basis`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.basis`. Research cards: C.2C direct_sum_basis.

AG-0424 **`direct_sum_transform`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.tensor`. Research cards: C.2A direct_sum_transform.

AG-0425 **`DirectionalDerivative`**. Strongest listed tier `[RESEARCH]`. Kinds: `data`. Owners: `collections.abc`. Research cards: C.5A DirectionalDerivative.

AG-0426 **`DirectionalJVP`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.autodiff`. Research cards: C.1E DirectionalJVP, C.5A DirectionalJVP.

AG-0427 **`DirectRotationResult`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.transport`. Research cards: C.5A DirectRotationResult.

AG-0428 **`dirichlet_convolution`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.monoid`. Research cards: C.3A dirichlet_convolution.

AG-0429 **`discover_block_diagonal_low_rank`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.algorithms.discovery`. Research cards: C.1D discover_block_diagonal_low_rank.

AG-0430 **`discover_packet_support`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.algorithms.discovery`. Research cards: C.1D discover_packet_support.

AG-0431 `discrete_cosine_transform`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.numeric`. Research cards: C.2C discrete_cosine_transform.

AG-0432 `discrete_fourier_transform`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.numeric`. Research cards: C.2C discrete_fourier_transform, C.1C discrete_fourier_transform.

AG-0433 `divided_square_packets`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.nonlinear.graded`. Research cards: C.1E divided_square_packets, C.5A divided_square_packets.

AG-0434 `DividedSquareResult`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.graded`. Research cards: C.5A DividedSquareResult.

AG-0435 `divisor_zeta_transform`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.combinatorial`. Research cards: C.2A divisor_zeta_transform.

AG-0436 `document_from_data`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.export.report`. Research cards: C.1F document_from_data, C.6 document_from_data.

AG-0437 `domain_from_descriptor`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.domains`. Research cards: C.3A domain_from_descriptor.

AG-0438 `downsample`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.resampling`. Research cards: C.4A downsample.

AG-0439 `DTypeChoice`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.algorithms.precision`. Research cards: C.1D DTypeChoice.

AG-0440 `dyadic_sign_diagnostic`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.langlands.global_.dyadic`. Research cards: C.1G dyadic_sign_diagnostic.

AG-0441 `entropy_cost`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.wavelets`. Research cards: C.4B entropy_cost.

AG-0442 `EquivalenceResult`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.expression.engine`. Research cards: C.1B EquivalenceResult.

AG-0443 `equivalent`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.expression.engine`. Research cards: C.1B equivalent.

AG-0444 `ErrorGuarantee`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.integration.quadrature`. Research cards: C.1D ErrorGuarantee.

AG-0445 `estimate_positive_packet_1d`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.approximation`. Research cards: C.4B estimate_positive_packet_1d.

AG-0446 `estimate_white_noise_sigma`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.denoise`. Research cards: C.4B estimate_white_noise_sigma.

AG-0447 `estimated_packet_reconstruct`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.singular_packets`. Research cards: C.4B estimated_packet_reconstruct.

AG-0448 `EuclideanTransfer`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.quantum.euclidean`. Research cards: C.1G EuclideanTransfer.

AG-0449 `euler_fibonacci_coordinates`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.recurrence`. Research cards: C.2C euler_fibonacci_coordinates, C.1C euler_fibonacci_coordinates.

AG-0450 `euler_polynomial`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.polynomials`. Research cards: C.2B euler_polynomial.

AG-0451 `euler_polynomial_at_zero`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.polynomials`. Research cards: C.2B euler_polynomial_at_zero.

AG-0452 `euler_polynomial_family`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.polynomials`. Research cards: C.2B euler_polynomial_family.

AG-0453 `euler_product_collapse`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.masks`. Research cards: C.3B euler_product_collapse.

AG-0454 `euler_product_function`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.functions`. Research cards: C.3A euler_product_function.

AG-0455 **`euler_to_fibonacci_coefficient`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.polynomials`. Research cards: C.2B euler_to_fibonacci_coefficient.

AG-0456 **`euler_to_fibonacci_transform`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.polynomials`. Research cards: C.2B euler_to_fibonacci_transform, C.1C euler_to_fibonacci_transform.

AG-0457 **`EulerLocalCoordinates`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.arithmetic.euler`. Research cards: C.1G EulerLocalCoordinates.

AG-0458 **`EulerProductCollapse`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.generating.masks`. Research cards: C.3B EulerProductCollapse.

AG-0459 **`evaluate_expression`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.expression.engine`. Research cards: C.1B evaluate_expression.

AG-0460 **`evaluation`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.expression.polynomial`. Research cards: C.1B evaluation.

AG-0461 **`evaluation_transform`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.classical`. Research cards: C.2B evaluation_transform.

AG-0462 **`exact_binding`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.expression.engine`. Research cards: C.1B exact_binding.

AG-0463 **`exact_internal_certificate`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.expression.certification`. Research cards: C.1B exact_internal_certificate.

AG-0464 **`exact_map`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.expression.model`. Research cards: C.1B exact_map.

AG-0465 **`exact_value`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.expression.model`. Research cards: C.1B exact_value.

AG-0466 **`ExactAffineMap`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.integration.measures`. Research cards: C.1D ExactAffineMap.

AG-0467 **`ExactAnalysisSynthesisMap`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.coordinates.frame_algebra`. Research cards: C.1C ExactAnalysisSynthesisMap, C.2C ExactAnalysisSynthesisMap.

AG-0468 **`ExactAnalysisSynthesisSystem`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.coordinates.frame_algebra`. Research cards: C.1C ExactAnalysisSynthesisSystem, C.2C ExactAnalysisSynthesisSystem.

AG-0469 **`ExactLinearSystem`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.coordinates.exact`. Research cards: C.2A ExactLinearSystem.

AG-0470 **`ExactMapLiteral`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.expression.model`. Research cards: C.1B ExactMapLiteral.

AG-0471 **`exactness_strength`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.expression.certification`. Research cards: C.1B exactness_strength.

AG-0472 **`ExactPolynomial`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.integration.polynomial`. Research cards: C.1D ExactPolynomial.

AG-0473 **`ExactQuadratureRule`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.integration.rules`. Research cards: C.1D ExactQuadratureRule.

AG-0474 **`ExactValueLiteral`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.expression.model`. Research cards: C.1B ExactValueLiteral.

AG-0475 **`execute_plan`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.expression.engine`. Research cards: C.1B execute_plan.

AG-0476 **`explicit_function`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.functions`. Research cards: C.3A explicit_function.

AG-0477 **`exponential_basis`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.basis`. Research cards: C.2C exponential_basis, C.3A exponential_basis.

AG-0478 **`export_adapters`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.export.serialization`. Research cards: C.6 export_adapters.

AG-0479 **`export_bundle`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.export.writers`. Research cards: C.6 export_bundle, C.1F export_bundle.

AG-0480 **`export_data`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.export.writers`. Research cards: C.6 export_data, C.1F export_data.

AG-0481 **`export_document`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.export.writers`. Research cards: C.6 export_document, C.1F export_document.

AG-0482 **`export_many`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.export.writers`. Research cards: C.6 export_many, C.1F export_many.

AG-0483 **`Exportable`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.export.serialization`. Research cards: C.6 Exportable.

AG-0484 **`ExportBundleResult`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.export.writers`. Research cards: C.1F ExportBundleResult, C.6 ExportBundleResult.

AG-0485 **`ExportFormat`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.export.writers`. Research cards: C.1F ExportFormat, C.6 ExportFormat.

AG-0486 **`ExportOptions`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.export.writers`. Research cards: C.1F ExportOptions, C.6 ExportOptions.

AG-0487 **`ExportResult`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.export.writers`. Research cards: C.1F ExportResult, C.6 ExportResult.

AG-0488 **`Expression`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.expression.model`. Research cards: C.1B Expression.

AG-0489 **`expression_digest`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.expression.portable`. Research cards: C.1B expression_digest.

AG-0490 **`expression_from_dict`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.expression.portable`. Research cards: C.1B expression_from_dict.

AG-0491 **`expression_from_json`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.expression.portable`. Research cards: C.1B expression_from_json.

AG-0492 **`EXPRESSION_NODE_VERSION`**. Strongest listed tier `[RESEARCH]`. Kinds: `data`. Owners: `builtins`. Research cards: C.1A EXPRESSION_NODE_VERSION.

AG-0493 **`EXPRESSION_SCHEMA`**. Strongest listed tier `[RESEARCH]`. Kinds: `data`. Owners: `builtins`. Research cards: C.1A EXPRESSION_SCHEMA.

AG-0494 **`EXPRESSION_SCHEMA_VERSION`**. Strongest listed tier `[RESEARCH]`. Kinds: `data`. Owners: `builtins`. Research cards: C.1A EXPRESSION_SCHEMA_VERSION.

AG-0495 **`expression_to_dict`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.expression.portable`. Research cards: C.1B expression_to_dict.

AG-0496 **`expression_to_json`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.expression.portable`. Research cards: C.1B expression_to_json.

AG-0497 **`EXPRESSION_TYPE_VERSION`**. Strongest listed tier `[RESEARCH]`. Kinds: `data`. Owners: `builtins`. Research cards: C.1A EXPRESSION_TYPE_VERSION.

AG-0498 **`ExpressionBinding`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.expression.certification`. Research cards: C.1B ExpressionBinding.

AG-0499 **`ExpressionBudget`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.expression.model`. Research cards: C.1B ExpressionBudget.

AG-0500 **`ExpressionPlan`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.expression.engine`. Research cards: C.1B ExpressionPlan.

AG-0501 **`ExpressionResult`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.expression.engine`. Research cards: C.1B ExpressionResult.

AG-0502 `ExpressionStats`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.expression.model`. Research cards: C.1B ExpressionStats.

AG-0503 `ExpressionType`. Strongest listed tier `[RESEARCH]`. Kinds: `data`. Owners: `types`. Research cards: C.1A ExpressionType.

AG-0504 `faa_di_bruno`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.nonlinear.composition`. Research cards: C.1E faa_di_bruno, C.5A faa_di_bruno.

AG-0505 `FaaDiBrunoResult`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.composition`. Research cards: C.5A FaaDiBrunoResult.

AG-0506 `FaaDiBrunoTerm`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.composition`. Research cards: C.5A FaaDiBrunoTerm.

AG-0507 `factorial_power_bell_function`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.functions`. Research cards: C.3A factorial_power_bell_function.

AG-0508 `factorial_power_convolution_layers`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.masks`. Research cards: C.3B factorial_power_convolution_layers.

AG-0509 `factorial_power_digit_kernel`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.digit_algebra`. Research cards: C.3B factorial_power_digit_kernel.

AG-0510 `factorial_power_support_count`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.masks`. Research cards: C.3B factorial_power_support_count.

AG-0511 `factorial_power_support_indices`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.masks`. Research cards: C.3B factorial_power_support_indices.

AG-0512 `factorial_valuation`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.padic_units`. Research cards: C.3B factorial_valuation.

AG-0513 `falling_factorial_polynomial`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.classical`. Research cards: C.2B falling_factorial_polynomial.

AG-0514 `falling_factorial_polynomial_family`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.classical`. Research cards: C.2B falling_factorial_polynomial_family.

AG-0515 `falling_factorial_to_monomial_transform`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.combinatorial`. Research cards: C.2A falling_factorial_to_monomial_transform.

AG-0516 `feature`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.registry`. Research cards: C.1G feature.

AG-0517 `FEATURE_KEY`. Strongest listed tier `[RESEARCH]`. Kinds: `data`. Owners: `builtins`. Research cards: C.5B FEATURE_KEY.

AG-0518 `FEATURE_KEY`. Strongest listed tier `[RESEARCH]`. Kinds: `data`. Owners: `builtins`. Research cards: C.4A FEATURE_KEY.

AG-0519 `features`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.registry`. Research cards: C.1G features.

AG-0520 `FeatureSpec`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.registry`. Research cards: C.1G FeatureSpec.

AG-0521 `FeatureStatus`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.registry`. Research cards: C.1G FeatureStatus.

AG-0522 `FeshbachReduction`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.dynamics.feshbach`. Research cards: C.1G FeshbachReduction.

AG-0523 `fft_convolve`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.fft_convolution`. Research cards: C.4A fft_convolve.

AG-0524 `fft_lowpass_1d`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.approximation`. Research cards: C.4B fft_lowpass_1d.

AG-0525 `fft_lowpass_2d`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.approximation`. Research cards: C.4B fft_lowpass_2d.

AG-0526 `fibonacci_companion_power`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.polynomials`. Research cards: C.2B fibonacci_companion_power.

AG-0527 `fibonacci_factorial_basis`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.basis`. Research cards: C.2C fibonacci_factorial_basis, C.3A fibonacci_factorial_basis.

AG-0528 `fibonacci_numbers`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.recurrence`. Research cards: C.2C fibonacci_numbers.

AG-0529 `fibonacci_numbers`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.sequences`. Research cards: C.3A fibonacci_numbers.

AG-0530 `fibonacci_polynomial`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.polynomials`. Research cards: C.2B fibonacci_polynomial.

AG-0531 `fibonacci_polynomial_family`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.polynomials`. Research cards: C.2B fibonacci_polynomial_family.

AG-0532 `fibonacci_recurrence`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.arithmetic.recurrence`. Research cards: C.1G fibonacci_recurrence.

AG-0533 `fibonacci_recurrence_model`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.families`. Research cards: C.2C fibonacci_recurrence_model.

AG-0534 `fibonacci_sequence`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.sequences.catalog`. Research cards: C.1G fibonacci_sequence.

AG-0535 `Figure`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.export.model`. Research cards: C.6 Figure.

AG-0536 `FilterBankReport`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.signals.filterbanks`. Research cards: C.4B FilterBankReport.

AG-0537 `FilteredCorrectionProblem`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.filtered`. Research cards: C.1E FilteredCorrectionProblem, C.5A FilteredCorrectionProblem.

AG-0538 `FilteredSpectralSequenceResult`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.filtered`. Research cards: C.5A FilteredSpectralSequenceResult.

AG-0539 `FilteredVectorSpace`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.filtered`. Research cards: C.1E FilteredVectorSpace, C.5A FilteredVectorSpace.

AG-0540 `FilterOutput`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.signals.recursive`. Research cards: C.4A FilterOutput.

AG-0541 `finite_difference`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.expression.polynomial`. Research cards: C.1B finite_difference.

AG-0542 `finite_difference`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.recovery`. Research cards: C.4B finite_difference.

AG-0543 `finite_difference_coordinate_operator`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.calculus`. Research cards: C.2B finite_difference_coordinate_operator.

AG-0544 `finite_frobenius_separator`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.deformation.companions`. Research cards: C.1G finite_frobenius_separator.

AG-0545 `finite_mask_carry_matrices`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.cartier`. Research cards: C.3B finite_mask_carry_matrices.

AG-0546 `finite_polyphase_blocks`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.convolution`. Research cards: C.4A finite_polyphase_blocks, C.4A finite_polyphase_filter_kernels.

AG-0547 `FiniteAbelianGroup`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.graded`. Research cards: C.1E FiniteAbelianGroup, C.5A FiniteAbelianGroup.

AG-0548 `FiniteCochainComplex`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.deformation.complexes`. Research cards: C.1G FiniteCochainComplex.

AG-0549 `FiniteField`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.langlands.local.finite_fields`. Research cards: C.1G FiniteField.

AG-0550 **`FiniteFieldCoordinateTransform`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.coordinates.modular`. Research cards: C.1C FiniteFieldCoordinateTransform, C.2A FiniteFieldCoordinateTransform.

AG-0551 **`FiniteFilterBank`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.signals.filterbanks`. Research cards: C.4B FiniteFilterBank.

AG-0552 **`FiniteFilteredBranchResult`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.filtered`. Research cards: C.5A FiniteFilteredBranchResult.

AG-0553 **`FiniteFilteredCochainComplex`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.filtered`. Research cards: C.1E FiniteFilteredCochainComplex, C.5A FiniteFilteredCochainComplex.

AG-0554 **`FiniteGroup`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.langlands.finite_image.groups`. Research cards: C.1G FiniteGroup.

AG-0555 **`FiniteHeckePackets`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.langlands.global_.hecke`. Research cards: C.1G FiniteHeckePackets.

AG-0556 **`FiniteLiftForest`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.deformation.forests`. Research cards: C.1G FiniteLiftForest.

AG-0557 **`FiniteLiftTree`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.lifting`. Research cards: C.1E FiniteLiftTree, C.5A FiniteLiftTree.

AG-0558 **`FiniteMaskCarryPlan`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.generating.masks`. Research cards: C.1F FiniteMaskCarryPlan, C.3B FiniteMaskCarryPlan, C.3B NormalizedMaskPlan.

AG-0559 **`FiniteMonoidSeries`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.generating.monoid`. Research cards: C.3A FiniteMonoidSeries.

AG-0560 **`FiniteNormalizedSeries`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.generating.finite_series`. Research cards: C.1F FiniteNormalizedSeries, C.3A FiniteNormalizedSeries.

AG-0561 **`FinitePolyphaseKernel`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.signals.convolution`. Research cards: C.4A FinitePolyphaseKernel, C.4A PolyphaseBlockMatrix.

AG-0562 **`fir_filter`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.filtering`. Research cards: C.4A fir_filter.

AG-0563 **`fir_frequency_response`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.filtering`. Research cards: C.4A fir_frequency_response.

AG-0564 **`fir_taps_to_series`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.adapters`. Research cards: C.3A fir_taps_to_series.

AG-0565 **`FIRFilter`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.signals.filtering`. Research cards: C.4A FIRFilter.

AG-0566 **`FIRKind`**. Strongest listed tier `[RESEARCH]`. Kinds: `data`. Owners: `typing`. Research cards: C.4A FIRKind.

AG-0567 **`FIRMode`**. Strongest listed tier `[RESEARCH]`. Kinds: `data`. Owners: `typing`. Research cards: C.4A FIRMode.

AG-0568 **`FIRSeriesAdapter`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.generating.adapters`. Research cards: C.3A FIRSeriesAdapter.

AG-0569 **`first_frobenius_lift`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.masks`. Research cards: C.3B first_frobenius_lift.

AG-0570 **`fixed_packet_leakage`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.quantum.leakage`. Research cards: C.1G fixed_packet_leakage.

AG-0571 **`fractional_power`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.dynamics.fractional`. Research cards: C.1G fractional_power.

AG-0572 **`frame_signal`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.time_frequency`. Research cards: C.4A frame_signal.

AG-0573 **`FrameBatch`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.signals.time_frequency`. Research cards: C.4A FrameBatch.

AG-0574 **`freeze_custom_function`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.portable`. Research cards: C.1F freeze_custom_function, C.3A freeze_custom_function.

AG-0575 **`FrequencyResponse`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.signals.filtering`. Research cards: C.4A FrequencyResponse.

AG-0576 **`frobenius_defect_mask`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.masks`. Research cards: C.3B frobenius_defect_mask.

AG-0577 **`FrobeniusOrbit`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.langlands.local.tame`. Research cards: C.1G FrobeniusOrbit.

AG-0578 **`FrobeniusSpine`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.deformation.companions`. Research cards: C.1G FrobeniusSpine.

AG-0579 **`from_serializable`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.export.serialization`. Research cards: C.6 from_serializable, C.1F from_serializable.

AG-0580 **`FrozenGeneratingFunction`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.generating.portable`. Research cards: C.1F FrozenGeneratingFunction, C.3A FrozenGeneratingFunction.

AG-0581 **`fubini_identity`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.integration.identities`. Research cards: C.1D fubini_identity.

AG-0582 **`FunctionRegistry`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.generating.functions`. Research cards: C.1F FunctionRegistry, C.3A FunctionRegistry.

AG-0583 **`fundamental_theorem`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.integration.identities`. Research cards: C.1D fundamental_theorem.

AG-0584 **`FundamentalHolonomy`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.holonomy`. Research cards: C.5A FundamentalHolonomy.

AG-0585 **`gamma_from_impedance`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.smith`. Research cards: C.4B gamma_from_impedance.

AG-0586 **`GateStatus`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.langlands.completion.gates`. Research cards: C.1G GateStatus.

AG-0587 **`gauss_sum`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.langlands.local.gauss`. Research cards: C.1G gauss_sum.

AG-0588 **`gaussian_binomial`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.q_kummer`. Research cards: C.3B gaussian_binomial.

AG-0589 **`generalized_bell_digit_reduction`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.bell`. Research cards: C.1F generalized_bell_digit_reduction, C.3B generalized_bell_digit_reduction.

AG-0590 **`generalized_bell_numbers`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.sequences`. Research cards: C.3A generalized_bell_numbers.

AG-0591 **`generalized_bell_parity`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.bell`. Research cards: C.3B generalized_bell_parity.

AG-0592 **`generalized_bell_polynomials`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.sequences`. Research cards: C.3A generalized_bell_polynomials.

AG-0593 **`GeneratingBudget`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.generating.budget`. Research cards: C.1F GeneratingBudget, C.3A GeneratingBudget.

AG-0594 **`get_recurrence_model`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.families`. Research cards: C.2C get_recurrence_model, C.1C get_recurrence_model.

AG-0595 **`get_sequence`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.sequences.catalog`. Research cards: C.1G get_sequence.

AG-0596 **`get_window`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.windows`. Research cards: C.4A get_window.

AG-0597 **`GradedConvolutionResult`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.graded`. Research cards: C.5A GradedConvolutionResult.

AG-0598 **`GradedConvolutionTerm`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.graded`. Research cards: C.5A GradedConvolutionTerm.

AG-0599 **`GradedPacketOperation`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.graded`. Research cards: C.1E GradedPacketOperation, C.5A GradedPacketOperation.

AG-0600 **`GradingCertificate`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.graded`. Research cards: C.5A GradingCertificate.

AG-0601 **`guess_algebraic_gf`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.closed_forms`. Research cards: C.3A guess_algebraic_gf.

AG-0602 **`guess_hypergeometric_ratio`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.closed_forms`. Research cards: C.3A guess_hypergeometric_ratio.

AG-0603 **`guess_linear_recurrence`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.closed_forms`. Research cards: C.3A guess_linear_recurrence.

AG-0604 **`guess_p_recursive`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.closed_forms`. Research cards: C.3A guess_p_recursive.

AG-0605 **`guess_quasipolynomial`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.closed_forms`. Research cards: C.3A guess_quasipolynomial.

AG-0606 **`haar_analyze`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.filterbanks`. Research cards: C.4B haar_analyze.

AG-0607 **`haar_decompose`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.denoise`. Research cards: C.4B haar_decompose.

AG-0608 **`haar_filter_bank`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.filterbanks`. Research cards: C.4B haar_filter_bank.

AG-0609 **`haar_lipschitz_bound`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.recovery`. Research cards: C.4B haar_lipschitz_bound.

AG-0610 **`haar_matrix`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.filterbanks`. Research cards: C.4B haar_matrix.

AG-0611 **`haar_packet_analyze`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.wavelets`. Research cards: C.4B haar_packet_analyze.

AG-0612 **`haar_packet_synthesize`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.wavelets`. Research cards: C.4B haar_packet_synthesize.

AG-0613 **`haar_step_coefficient`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.recovery`. Research cards: C.4B haar_step_coefficient.

AG-0614 **`haar_synthesize`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.filterbanks`. Research cards: C.4B haar_synthesize.

AG-0615 **`haar_wavelet_transform`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.numeric`. Research cards: C.2C haar_wavelet_transform.

AG-0616 **`HaarDecomposition`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.signals.denoise`. Research cards: C.4B HaarDecomposition.

AG-0617 **`HaarFilterBank`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.signals.filterbanks`. Research cards: C.4B HaarFilterBank.

AG-0618 **`HaarPacketTree`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.signals.wavelets`. Research cards: C.1F HaarPacketTree, C.4B HaarPacketTree.

AG-0619 **`half_shift_distance`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.edge_recovery`. Research cards: C.4B half_shift_distance.

AG-0620 **`Heading`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.export.model`. Research cards: C.6 Heading.

AG-0621 **`HeisenbergModel`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.langlands.local.heisenberg`. Research cards: C.1G HeisenbergModel.

AG-0622 **`hermite_physicists_polynomial`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.classical`. Research cards: C.2B hermite_physicists_polynomial.

AG-0623 **`hermite_physicists_polynomial_family`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.classical`. Research cards: C.2B hermite_physicists_polynomial_family.

AG-0624 **`hermite_probabilists_polynomial`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.classical`. Research cards: C.2B hermite_probabilists_polynomial.

AG-0625 **`hermite_probabilists_polynomial_family`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.classical`. Research cards: C.2B hermite_probabilists_polynomial_family.

AG-0626 **`HiddenSolveResult`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.elimination`. Research cards: C.5A HiddenSolveResult.

AG-0627 **`higher_maurer_cartan_packets`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.nonlinear.graded`. Research cards: C.1E higher_maurer_cartan_packets, C.5A higher_maurer_cartan_packets.

AG-0628 **`HigherMaurerCartanResult`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.graded`. Research cards: C.5A HigherMaurerCartanResult.

AG-0629 **`HilbertObstructionProjector`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.correction`. Research cards: C.1E HilbertObstructionProjector, C.5A HilbertObstructionProjector.

AG-0630 **`HilbertObstructionResult`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.correction`. Research cards: C.5A HilbertObstructionResult.

AG-0631 **`HolonomicRecurrence`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.sequences.holonomic`. Research cards: C.1G HolonomicRecurrence.

AG-0632 **`HtmlRenderOptions`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.export.renderers`. Research cards: C.6 HtmlRenderOptions.

AG-0633 **`HurwitzSeries`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.arithmetic.hurwitz`. Research cards: C.1G HurwitzSeries.

AG-0634 **`hypergeometric_function`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.functions`. Research cards: C.3A hypergeometric_function.

AG-0635 **`HypergeometricGuess`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.generating.closed_forms`. Research cards: C.3A HypergeometricGuess.

AG-0636 **`identity`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.expression.model`. Research cards: C.1B identity.

AG-0637 **`identity_formal_coordinate`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.formal`. Research cards: C.2C identity_formal_coordinate.

AG-0638 **`identity_matrix`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.exact`. Research cards: C.2A identity_matrix.

AG-0639 **`IdentityMap`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.expression.model`. Research cards: C.1B IdentityMap.

AG-0640 **`impedance_from_gamma`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.smith`. Research cards: C.4B impedance_from_gamma.

AG-0641 **`independent_direct_sum_seed_rank`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.core.seeds`. Research cards: C.1D independent_direct_sum_seed_rank.

AG-0642 **`InducedRepresentation`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.langlands.finite_image.characters`. Research cards: C.1G InducedRepresentation.

AG-0643 **`infer_export_format`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.export.writers`. Research cards: C.6 infer_export_format.

AG-0644 **`INTEGER_DOMAIN`**. Strongest listed tier `[RESEARCH]`. Kinds: `data`. Owners: `komorebi_math.generating.domains`. Research cards: C.3A INTEGER_DOMAIN.

AG-0645 **`integer_valuation`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.padic_units`. Research cards: C.3B integer_valuation.

AG-0646 **`IntegerDomain`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.generating.domains`. Research cards: C.3A IntegerDomain.

AG-0647 **`integral_coordinate_operator`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.calculus`. Research cards: C.2B integral_coordinate_operator.

AG-0648 **`IntegralIdentity`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.integration.identities`. Research cards: C.1D IntegralIdentity.

AG-0649 **`IntegralResult`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.integration.quadrature`. Research cards: C.1D IntegralResult.

AG-0650 **`integrate`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.integration.quadrature`. Research cards: C.1D integrate.

AG-0651 **`integrate_derivative_provider`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.integration.quadrature`. Research cards: C.1D integrate_derivative_provider.

AG-0652 **`integrate_formal_series`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.integration.adapters`. Research cards: C.1D integrate_formal_series.

AG-0653 **`integrate_polynomial_coordinates`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.integration.adapters`. Research cards: C.1D integrate_polynomial_coordinates.

AG-0654 **`integrate_polynomial_oriented`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.integration.quadrature`. Research cards: C.1D integrate_polynomial_oriented.

AG-0655 **`integration_by_parts`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.integration.identities`. Research cards: C.1D integration_by_parts.

AG-0656 **`IntegrationError`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.integration.quadrature`. Research cards: C.1D IntegrationError.

AG-0657 **`interpolate`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.resampling`. Research cards: C.4A interpolate.

AG-0658 **`interpolation_transform`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.classical`. Research cards: C.2B interpolation_transform.

AG-0659 **`intertwiner_space`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.realization`. Research cards: C.2C intertwiner_space, C.1C intertwiner_space.

AG-0660 **`IntertwinerSpace`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.coordinates.realization`. Research cards: C.1C IntertwinerSpace, C.2C IntertwinerSpace.

AG-0661 **`IntertwiningResult`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.holonomy`. Research cards: C.5A IntertwiningResult.

AG-0662 **`interval_integration`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.expression.polynomial`. Research cards: C.1B interval_integration.

AG-0663 **`IntervalMeasure`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.integration.measures`. Research cards: C.1D IntervalMeasure.

AG-0664 **`inverse_matrix`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.exact`. Research cards: C.2A inverse_matrix.

AG-0665 **`istft`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.time_frequency`. Research cards: C.4A istft.

AG-0666 **`jacobsthal_recurrence_model`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.families`. Research cards: C.2C jacobsthal_recurrence_model.

AG-0667 **`JaxDerivativeProvider`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.autodiff`. Research cards: C.1E JaxDerivativeProvider, C.5A JaxDerivativeProvider.

AG-0668 **`JumpDetection`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.signals.recovery`. Research cards: C.4B JumpDetection.

AG-0669 **`JumpRecoveryResult`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.signals.recovery`. Research cards: C.4B JumpRecoveryResult.

AG-0670 **`KatoTransport`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.transport`. Research cards: C.1E KatoTransport, C.5A KatoTransport.

AG-0671 **`KomorebiDeprecationWarning`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.compatibility`. Research cards: C.1A KomorebiDeprecationWarning.

AG-0672 **`kummer_layer_distribution`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.masks`. Research cards: C.3B kummer_layer_distribution.

AG-0673 **`KummerLayerDistribution`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.generating.masks`. Research cards: C.3B KummerLayerDistribution.

AG-0674 **`labeled_composition_term_count`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.nonlinear.composition`. Research cards: C.5A labeled_composition_term_count.

AG-0675 **`lagrange_interpolation_basis`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.classical`. Research cards: C.2B lagrange_interpolation_basis.

AG-0676 **`laguerre_polynomial`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.classical`. Research cards: C.2B laguerre_polynomial.

AG-0677 **`laguerre_polynomial_family`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.classical`. Research cards: C.2B laguerre_polynomial_family.

AG-0678 **`lah_number`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.combinatorial`. Research cards: C.2A lah_number.

AG-0679 **`latex_escape`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.export.renderers`. Research cards: C.6 latex_escape.

AG-0680 **`latex_label`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.export.renderers`. Research cards: C.6 latex_label.

AG-0681 **`LatexCompilation`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.export.latex`. Research cards: C.6 LatexCompilation.

AG-0682 **`LatexCompileOptions`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.export.latex`. Research cards: C.1F LatexCompileOptions, C.6 LatexCompileOptions.

AG-0683 **`LatexEngine`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.export.latex`. Research cards: C.1F LatexEngine, C.6 LatexEngine.

AG-0684 **`LatexRenderOptions`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.export.renderers`. Research cards: C.1F LatexRenderOptions, C.6 LatexRenderOptions.

AG-0685 **`LatticePacketData`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.refinement.lattices`. Research cards: C.1G LatticePacketData.

AG-0686 **`legendre_polynomial`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.classical`. Research cards: C.2B legendre_polynomial.

AG-0687 **`legendre_polynomial_family`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.classical`. Research cards: C.2B legendre_polynomial_family.

AG-0688 **`Let`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.expression.model`. Research cards: C.1B Let.

AG-0689 **`let`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.expression.model`. Research cards: C.1B let.

AG-0690 **`lfilter`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.recursive`. Research cards: C.4A lfilter.

AG-0691 **`lift_digit_sequence`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.digit_algebra`. Research cards: C.3B lift_digit_sequence.

AG-0692 **`LiftStepResult`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.lifting`. Research cards: C.5A LiftStepResult.

AG-0693 **`LiftTreeNode`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.lifting`. Research cards: C.5A LiftTreeNode.

AG-0694 **`LindbladGenerator`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.quantum.lindblad`. Research cards: C.1G LindbladGenerator.

AG-0695 **`linear_recurrence_function`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.functions`. Research cards: C.3A linear_recurrence_function.

AG-0696 **`LinearCharacter`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.langlands.finite_image.characters`. Research cards: C.1G LinearCharacter.

AG-0697 **`LinearFilter`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.signals.recursive`. Research cards: C.4A LinearFilter.

AG-0698 **`LinearLiftProblem`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.lifting`. Research cards: C.1E LinearLiftProblem, C.5A LinearLiftProblem.

AG-0699 **`LinearMapType`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.expression.model`. Research cards: C.1B LinearMapType.

AG-0700 **`LinearObstructionProblem`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.obstruction`. Research cards: C.1E LinearObstructionProblem, C.5A LinearObstructionProblem.

AG-0701 **`LinearRealization`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.coordinates.realization`. Research cards: C.1C LinearRealization, C.2C LinearRealization.

AG-0702 **`LinearRecurrence`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.arithmetic.recurrence`. Research cards: C.1G LinearRecurrence.

AG-0703 **`LinearRecurrenceGuess`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.generating.closed_forms`. Research cards: C.3A LinearRecurrenceGuess.

AG-0704 **`LinearRefinementDefect`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.refinement.defects`. Research cards: C.1G LinearRefinementDefect.

AG-0705 **`ListBlock`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.export.model`. Research cards: C.6 ListBlock.

AG-0706 **`lms_filter`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.adaptive`. Research cards: C.4B lms_filter.

AG-0707 **`load_coordinate_object`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.portable`. Research cards: C.2C load_coordinate_object, C.1C load_coordinate_object.

AG-0708 **`load_coordinate_record`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.portable`. Research cards: C.2C load_coordinate_record.

AG-0709 **`load_public_api_manifest`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.compatibility`. Research cards: C.1A load_public_api_manifest.

AG-0710 **`load_transform`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.portable`. Research cards: C.2C load_transform.

AG-0711 **`local_normalized_kernel`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.normalization`. Research cards: C.3B local_normalized_kernel.

AG-0712 **`LowRankOperator`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.algorithms.low_rank`. Research cards: C.1D LowRankOperator.

AG-0713 **`lp_cost`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.wavelets`. Research cards: C.4B lp_cost.

AG-0714 **`lucas_polynomial`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.classical`. Research cards: C.2B lucas_polynomial.

AG-0715 **`lucas_polynomial_family`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.classical`. Research cards: C.2B lucas_polynomial_family.

AG-0716 **`lucas_recurrence_model`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.families`. Research cards: C.2C lucas_recurrence_model.

AG-0717 **`LyapunovSchmidtDecomposition`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.correction`. Research cards: C.5A LyapunovSchmidtDecomposition.

AG-0718 **`LyapunovSchmidtPoint`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.correction`. Research cards: C.5A LyapunovSchmidtPoint.

AG-0719 **`LyapunovSchmidtReduction`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.correction`. Research cards: C.1E LyapunovSchmidtReduction, C.5A LyapunovSchmidtReduction.

AG-0720 **`mackey_tensor_packets`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.langlands.finite_image.mackey`. Research cards: C.1G mackey_tensor_packets.

AG-0721 **`mahler_coefficients`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.congruences`. Research cards: C.3B mahler_coefficients.

AG-0722 **`mahler_mask_inverse`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.masks`. Research cards: C.3B mahler_mask_inverse, C.1F mahler_mask_inverse.

AG-0723 **`mahler_mask_product`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.masks`. Research cards: C.3B mahler_mask_product, C.1F mahler_mask_product.

AG-0724 **`MahlerSeries`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.arithmetic.mahler`. Research cards: C.1G MahlerSeries.

AG-0725 **`majorant_convergence`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.nonlinear.trees`. Research cards: C.1E majorant_convergence, C.5B majorant_convergence.

AG-0726 **`MajorantConvergence`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.trees`. Research cards: C.5B MajorantConvergence.

AG-0727 **`MarkovLumping`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.dynamics.markov`. Research cards: C.1G MarkovLumping.

AG-0728 **`Matrix`**. Strongest listed tier `[RESEARCH]`. Kinds: `data`. Owners: `builtins`. Research cards: C.2C Matrix.

AG-0729 **`matrix_add`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.exact`. Research cards: C.2A matrix_add.

AG-0730 **`matrix_chain_multiply`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.expression.planning`. Research cards: C.1B matrix_chain_multiply.

AG-0731 **`matrix_chain_parenthesization`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.expression.planning`. Research cards: C.1B matrix_chain_parenthesization.

AG-0732 **`matrix_chain_plan`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.expression.planning`. Research cards: C.1B matrix_chain_plan.

AG-0733 **`matrix_from_columns`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.exact`. Research cards: C.2A matrix_from_columns.

AG-0734 **`matrix_multiply`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.exact`. Research cards: C.2A matrix_multiply.

AG-0735 **`matrix_power`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.exact`. Research cards: C.2A matrix_power.

AG-0736 **`matrix_rank`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.exact`. Research cards: C.2A matrix_rank.

AG-0737 **`matrix_shape`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.exact`. Research cards: C.2A matrix_shape.

AG-0738 **`matrix_subtract`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.exact`. Research cards: C.2A matrix_subtract.

AG-0739 **`matrix_units`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.core.algebra`. Research cards: C.1D matrix_units.

AG-0740 **`matrix_vector_multiply`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.exact`. Research cards: C.2A matrix_vector_multiply.

AG-0741 **`MatrixChainPlan`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.expression.planning`. Research cards: C.1B MatrixChainPlan.

AG-0742 **`MatrixChainTree`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.expression.planning`. Research cards: C.1B MatrixChainTree.

AG-0743 **`MatrixSemigroup`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.dynamics.semigroups`. Research cards: C.1G MatrixSemigroup.

AG-0744 **`maurer_cartan_packets`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.nonlinear.graded`. Research cards: C.1E maurer_cartan_packets, C.5A maurer_cartan_packets.

AG-0745 **`MaurerCartanResult`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.graded`. Research cards: C.5A MaurerCartanResult.

AG-0746 **`Measure`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.integration.measures`. Research cards: C.1D Measure.

AG-0747 **`median_absolute_deviation`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.denoise`. Research cards: C.4B median_absolute_deviation.

AG-0748 **`MemoryKernel`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.dynamics.memory`. Research cards: C.1G MemoryKernel.

AG-0749 **`minimal_prefix_period`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.congruences`. Research cards: C.3B minimal_prefix_period.

AG-0750 **`minimize_cartier_automaton`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.cartier`. Research cards: C.3B minimize_cartier_automaton.

AG-0751 **`mixed_radix_address`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.polyphase`. Research cards: C.4B mixed_radix_address.

AG-0752 **`mobius_integer`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.combinatorial`. Research cards: C.2A mobius_integer.

AG-0753 **`modal_coordinate_system`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.numeric`. Research cards: C.2C modal_coordinate_system.

AG-0754 **`ModalCoordinateSystem`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.coordinates.numeric`. Research cards: C.2C ModalCoordinateSystem.

AG-0755 **`ModularLinearSolution`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.lifting`. Research cards: C.5A ModularLinearSolution.

AG-0756 **`ModularMatrix`**. Strongest listed tier `[RESEARCH]`. Kinds: `data`. Owners: `builtins`. Research cards: C.2C ModularMatrix.

AG-0757 **`ModularVector`**. Strongest listed tier `[RESEARCH]`. Kinds: `data`. Owners: `builtins`. Research cards: C.2C ModularVector.

AG-0758 **`monomial_polynomial_family`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.polynomials`. Research cards: C.2B monomial_polynomial_family.

AG-0759 **`monomial_to_falling_factorial_transform`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.combinatorial`. Research cards: C.2A monomial_to_falling_factorial_transform.

AG-0760 **`monomial_to_fibonacci_coefficient`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.polynomials`. Research cards: C.2B monomial_to_fibonacci_coefficient.

AG-0761 **`monomial_to_rising_factorial_transform`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.combinatorial`. Research cards: C.2A monomial_to_rising_factorial_transform.

AG-0762 **`MonomialAction`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.langlands.finite_image.orbits`. Research cards: C.1G MonomialAction.

AG-0763 **`motzkin_sequence`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.sequences.catalog`. Research cards: C.1G motzkin_sequence.

AG-0764 **`MovingPacketSystem`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.transport`. Research cards: C.1E MovingPacketSystem, C.5A MovingPacketSystem.

AG-0765 **`MultilinearOperation`**. Strongest listed tier `[RESEARCH]`. Kinds: `data`. Owners: `collections.abc`. Research cards: C.1E MultilinearOperation, C.5B MultilinearOperation.

AG-0766 **`multiplication_coordinate_operator`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.calculus`. Research cards: C.2B multiplication_coordinate_operator.

AG-0767 **`multiplicative_order_mod_prime`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.q_kummer`. Research cards: C.3B multiplicative_order_mod_prime.

AG-0768 **`multiplicative_series`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.monoid`. Research cards: C.3A multiplicative_series.

AG-0769 **`newton_evaluation_transform`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.combinatorial`. Research cards: C.2A newton_evaluation_transform.

AG-0770 **`newton_interpolation_basis`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.classical`. Research cards: C.2B newton_interpolation_basis.

AG-0771 **`nlms_filter`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.adaptive`. Research cards: C.4B nlms_filter.

AG-0772 **`nonlinear_cupy_available`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.nonlinear.execution`. Research cards: C.5B nonlinear_cupy_available, C.1E nonlinear_cupy_available.

AG-0773 **`nonlinear_shift_filter`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.nonlinear`. Research cards: C.4B nonlinear_shift_filter.

AG-0774 **`NonlinearPacketMap`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.coordinates`. Research cards: C.1E NonlinearPacketMap, C.5A NonlinearPacketMap.

AG-0775 **`NonlinearRefinementDefect`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.defects`. Research cards: C.1E NonlinearRefinementDefect, C.5A NonlinearRefinementDefect.

AG-0776 **`NonlinearSchurElimination`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.elimination`. Research cards: C.1E NonlinearSchurElimination, C.5A NonlinearSchurElimination.

AG-0777 **`normal_form_digest`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.expression.engine`. Research cards: C.1B normal_form_digest.

AG-0778 **`NormalForm`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.expression.certification`. Research cards: C.1B NormalForm.

AG-0779 **`normalization_change_transform`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.normalized`. Research cards: C.2A normalization_change_transform, C.1C normalization_change_transform.

AG-0780 **`normalize_expression`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.expression.engine`. Research cards: C.1B normalize_expression.

AG-0781 **`normalized_cyclotomic_expression`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.cyclotomic`. Research cards: C.3B normalized_cyclotomic_expression.

AG-0782 **`NormalizedBasis`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.generating.basis`. Research cards: C.1F NormalizedBasis, C.1F WeightedMonomialBasis, C.2C NormalizedBasis, C.3A NormalizedBasis, C.3A NormalizedMonomialBasis, C.3A WeightedMonomialBasis.

AG-0783 **`NormalizedKernelResidue`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.generating.padic_units`. Research cards: C.3B NormalizedKernelResidue.

AG-0784 **`nullspace`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.exact`. Research cards: C.2A nullspace.

AG-0785 **`observability_block_matrix`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.state_space`. Research cards: C.2C observability_block_matrix.

AG-0786 **`observability_coordinate_transform`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.state_space`. Research cards: C.2C observability_coordinate_transform.

AG-0787 **`observability_matrix`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.realization`. Research cards: C.2C observability_matrix, C.1C observability_matrix.

AG-0788 **`ObstructionResult`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.obstruction`. Research cards: C.5A ObstructionResult.

AG-0789 **`ObstructionSquare`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.obstruction`. Research cards: C.5A ObstructionSquare.

AG-0790 **`one_borrow_count`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.masks`. Research cards: C.3B one_borrow_count.

AG-0791 **`operator_change_of_coordinates`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.polynomials`. Research cards: C.2B operator_change_of_coordinates, C.1C operator_change_of_coordinates.

AG-0792 **`operator_commutator`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.calculus`. Research cards: C.2B operator_commutator.

AG-0793 **`operator_matrix_between_families`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.classical`. Research cards: C.2B operator_matrix_between_families.

AG-0794 **`OperatorAlgebra`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.core.algebra`. Research cards: C.1D OperatorAlgebra.

AG-0795 **`OperatorMorphism`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.core.morphisms`. Research cards: C.1D OperatorMorphism.

AG-0796 **`ordered_rooted_trees`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.nonlinear.trees`. Research cards: C.5B ordered_rooted_trees.

AG-0797 **`ordered_tree_count`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.nonlinear.trees`. Research cards: C.5B ordered_tree_count.

AG-0798 **`ordered_trees_with_leaves`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.nonlinear.trees`. Research cards: C.5B ordered_trees_with_leaves.

AG-0799 **`OrderedRootedTree`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.trees`. Research cards: C.1E OrderedRootedTree, C.5B OrderedRootedTree.

AG-0800 **`otsu_threshold`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.approximation`. Research cards: C.4B otsu_threshold.

AG-0801 **`overlap_add`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.time_frequency`. Research cards: C.4A overlap_add.

AG-0802 **`overlap_add_certificate`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.windows`. Research cards: C.4A overlap_add_certificate.

AG-0803 **`overlap_add_convolve`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.fft_convolution`. Research cards: C.4A overlap_add_convolve.

AG-0804 **`overlap_matrix_error`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.singular_packets`. Research cards: C.4B overlap_matrix_error.

AG-0805 **`overlap_partition_lp_error`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.singular_packets`. Research cards: C.4B overlap_partition_lp_error.

AG-0806 **`overlap_profile`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.windows`. Research cards: C.4A overlap_profile.

AG-0807 **`OverlapAddConvolver`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.signals.fft_convolution`. Research cards: C.4A OverlapAddConvolver.

AG-0808 **`p_adic_valuation_fraction`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.congruences`. Research cards: C.3B p_adic_valuation_fraction.

AG-0809 **`p_free_orbit`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.cyclotomic`. Research cards: C.3B p_free_orbit.

AG-0810 **`packet_energy`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.wavelets`. Research cards: C.4B packet_energy.

AG-0811 **`packet_faa_di_bruno`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.nonlinear.composition`. Research cards: C.5A packet_faa_di_bruno, C.1E packet_faa_di_bruno.

AG-0812 `packet_jet_benchmark_contract`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.nonlinear.execution`. Research cards: C.5B packet_jet_benchmark_contract, C.1E packet_jet_benchmark_contract.

AG-0813 `packet_product`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.nonlinear`. Research cards: C.4B packet_product.

AG-0814 `packet_taylor`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.nonlinear.jets`. Research cards: C.5B packet_taylor, C.1E packet_taylor.

AG-0815 `packet_tree_reconstruction`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.nonlinear.trees`. Research cards: C.5B packet_tree_reconstruction, C.1E packet_tree_reconstruction.

AG-0816 `PacketDerivativeResult`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.transport`. Research cards: C.5A PacketDerivativeResult.

AG-0817 `PacketEstimate1D`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.signals.approximation`. Research cards: C.4B PacketEstimate1D.

AG-0818 `PacketIdentity`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.generating.series`. Research cards: C.3A PacketIdentity.

AG-0819 `PacketIR`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.algorithms.ir`. Research cards: C.1D PacketIR.

AG-0820 `PacketLabel`. Strongest listed tier `[RESEARCH]`. Kinds: `data`. Owners: `types`. Research cards: C.5B PacketLabel.

AG-0821 `PacketLayout`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.algorithms.layout`. Research cards: C.1D PacketLayout.

AG-0822 `PacketMorphism`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.core.morphisms`. Research cards: C.1D PacketMorphism.

AG-0823 `PacketPath`. Strongest listed tier `[RESEARCH]`. Kinds: `data`. Owners: `types`. Research cards: C.4A PacketPath.

AG-0824 `PacketTaylorResult`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.jets`. Research cards: C.5B PacketTaylorResult.

AG-0825 `PacketTreeResult`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.trees`. Research cards: C.5B PacketTreeResult.

AG-0826 `padic_valuation`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.arithmetic.valuation`. Research cards: C.1G padic_valuation.

AG-0827 `padovan_recurrence_model`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.families`. Research cards: C.2C padovan_recurrence_model.

AG-0828 `pairing_invariant`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.tensor`. Research cards: C.2A pairing_invariant.

AG-0829 `parseval_certificate`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.spectral`. Research cards: C.4A parseval_certificate.

AG-0830 `Partition`. Strongest listed tier `[RESEARCH]`. Kinds: `data`. Owners: `builtins`. Research cards: C.5A Partition.

AG-0831 `partition_numbers`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.sequences`. Research cards: C.3A partition_numbers.

AG-0832 `partition_sequence`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.sequences.catalog`. Research cards: C.1G partition_sequence.

AG-0833 `PatchingSupportReport`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.langlands.global_.patching`. Research cards: C.1G PatchingSupportReport.

AG-0834 `pell_polynomial`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.classical`. Research cards: C.2B pell_polynomial.

AG-0835 `pell_polynomial_family`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.classical`. Research cards: C.2B pell_polynomial_family.

AG-0836 `pell_recurrence_model`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.families`. Research cards: C.2C pell_recurrence_model.

AG-0837 **`periodic_five_three_lifting`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.filterbanks`. Research cards: C.4B periodic_five_three_lifting.

AG-0838 **`periodic_laplacian`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.quantum.spectral`. Research cards: C.1G periodic_laplacian.

AG-0839 **`PeriodicFilterBank`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.signals.filterbanks`. Research cards: C.1F PeriodicFilterBank, C.4B PeriodicFilterBank.

AG-0840 **`PeriodicLiftingScheme`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.signals.filterbanks`. Research cards: C.4B PeriodicLiftingScheme.

AG-0841 **`periodogram`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.spectral`. Research cards: C.4A periodogram.

AG-0842 **`permutation_transform`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.basis`. Research cards: C.2C permutation_transform.

AG-0843 **`phase_mobius`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.smith`. Research cards: C.4B phase_mobius.

AG-0844 **`PhaseTypeSurvival`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.dynamics.survival`. Research cards: C.1G PhaseTypeSurvival.

AG-0845 **`PlanInstruction`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.expression.engine`. Research cards: C.1B PlanInstruction.

AG-0846 **`pochhammer_basis`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.basis`. Research cards: C.3A pochhammer_basis.

AG-0847 **`pointed_subindex_polynomial`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.bell`. Research cards: C.3B pointed_subindex_polynomial.

AG-0848 **`PointedSubindexCertificate`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.generating.bell`. Research cards: C.3B PointedSubindexCertificate.

AG-0849 **`Polynomial`**. Strongest listed tier `[RESEARCH]`. Kinds: `data`. Owners: `builtins`. Research cards: C.2C Polynomial.

AG-0850 **`polynomial_add`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.polynomials`. Research cards: C.2B polynomial_add.

AG-0851 **`polynomial_cartier`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.cartier`. Research cards: C.3B polynomial_cartier.

AG-0852 **`polynomial_compose_affine`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.classical`. Research cards: C.2B polynomial_compose_affine.

AG-0853 **`polynomial_derivative`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.polynomials`. Research cards: C.2B polynomial_derivative.

AG-0854 **`polynomial_derivative_bound`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.integration.quadrature`. Research cards: C.1D polynomial_derivative_bound.

AG-0855 **`polynomial_evaluate`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.polynomials`. Research cards: C.2B polynomial_evaluate.

AG-0856 **`polynomial_extended_gcd`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.operators`. Research cards: C.2C polynomial_extended_gcd.

AG-0857 **`polynomial_from_family_coordinates`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.integration.adapters`. Research cards: C.1D polynomial_from_family_coordinates.

AG-0858 **`polynomial_from_formal_series`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.integration.adapters`. Research cards: C.1D polynomial_from_formal_series.

AG-0859 **`polynomial_integral`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.calculus`. Research cards: C.2B polynomial_integral.

AG-0860 **`polynomial_integration_covector`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.integration.coordinates`. Research cards: C.1D polynomial_integration_covector.

AG-0861 `polynomial_mask_coboundary`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.masks`. Research cards: C.3B polynomial_mask_coboundary.

AG-0862 `polynomial_multiply`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.polynomials`. Research cards: C.2B polynomial_multiply.

AG-0863 `polynomial_multiply_mod`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.cartier`. Research cards: C.3B polynomial_multiply_mod.

AG-0864 `polynomial_operator`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.calculus`. Research cards: C.2B polynomial_operator.

AG-0865 `polynomial_power_mod`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.cartier`. Research cards: C.3B polynomial_power_mod.

AG-0866 `polynomial_scale`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.polynomials`. Research cards: C.2B polynomial_scale.

AG-0867 `polynomial_shift`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.polynomials`. Research cards: C.2B polynomial_shift.

AG-0868 `polynomial_subtract`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.classical`. Research cards: C.2B polynomial_subtract.

AG-0869 `PolynomialCoboundaryResult`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.generating.masks`. Research cards: C.3B PolynomialCoboundaryResult.

AG-0870 `PolynomialCoordinateOperator`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.coordinates.calculus`. Research cards: C.1C PolynomialCoordinateOperator, C.2B PolynomialCoordinateOperator.

AG-0871 `PolynomialCoordinateSpace`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.expression.model`. Research cards: C.1B PolynomialCoordinateSpace.

AG-0872 `PolynomialFamily`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.coordinates.polynomials`. Research cards: C.1C PolynomialFamily, C.2B PolynomialFamily.

AG-0873 `PolynomialFamilyCatalog`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.coordinates.classical`. Research cards: C.2B PolynomialFamilyCatalog.

AG-0874 `PolynomialFixedPointProblem`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.trees`. Research cards: C.1E PolynomialFixedPointProblem, C.5B PolynomialFixedPointProblem.

AG-0875 `PolynomialIntegrationCovector`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.integration.coordinates`. Research cards: C.1D PolynomialIntegrationCovector.

AG-0876 `PolynomialLiftProblem`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.lifting`. Research cards: C.1E PolynomialLiftProblem, C.5A PolynomialLiftProblem.

AG-0877 `PolynomialPrimitive`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.expression.polynomial`. Research cards: C.1B PolynomialPrimitive.

AG-0878 `polyphase_convolution`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.convolution`. Research cards: C.4A polyphase_convolution.

AG-0879 `polyphase_filter_kernels`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.polyphase`. Research cards: C.4B polyphase_filter_kernels.

AG-0880 `PolyphaseConvolution`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.signals.convolution`. Research cards: C.4A PolyphaseConvolution.

AG-0881 `PolyphaseKernel`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.signals.polyphase`. Research cards: C.4B PolyphaseKernel.

AG-0882 `positive_zero_string`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.analytic.zeros`. Research cards: C.1G positive_zero_string.

AG-0883 `PowerSpectrum`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.signals.spectral`. Research cards: C.4A PowerSpectrum.

AG-0884 `PRecursiveGuess`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.generating.closed_forms`. Research cards: C.3A PRecursiveGuess.

AG-0885 **`PresentedModule`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.deformation.support`. Research cards: C.1G PresentedModule.

AG-0886 **`prime_valuation_packets`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.monoid`. Research cards: C.3A prime_valuation_packets.

AG-0887 **`PrimeFieldDomain`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.generating.domains`. Research cards: C.3A PrimeFieldDomain.

AG-0888 **`PrimePowerBinomial`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.generating.padic_units`. Research cards: C.3B PrimePowerBinomial.

AG-0889 **`PrimePowerKernelCompiler`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.generating.padic_units`. Research cards: C.1F PrimePowerKernelCompiler, C.3B PrimePowerKernelCompiler.

AG-0890 **`PrimePowerUnitCompiler`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.generating.padic_units`. Research cards: C.3B PrimePowerUnitCompiler.

AG-0891 **`primitive_certificate`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.expression.polynomial`. Research cards: C.1B primitive_certificate.

AG-0892 **`primitive_matrix_equal`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.expression.polynomial`. Research cards: C.1B primitive_matrix_equal.

AG-0893 **`PrimitiveKind`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.expression.polynomial`. Research cards: C.1B PrimitiveKind.

AG-0894 **`product_measure`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.integration.measures`. Research cards: C.1D product_measure.

AG-0895 **`ProductMeasure`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.integration.measures`. Research cards: C.1D ProductMeasure.

AG-0896 **`ProjectiveIntertwinerEnvelope`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.langlands.local.envelopes`. Research cards: C.1G ProjectiveIntertwinerEnvelope.

AG-0897 **`ProjectiveLimitRecord`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.filtered`. Research cards: C.5A ProjectiveLimitRecord.

AG-0898 **`ProjectiveMultiplicityPacket`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.langlands.ramification.towers`. Research cards: C.1G ProjectiveMultiplicityPacket.

AG-0899 **`ProvenanceRecord`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.provenance`. Research cards: C.1A ProvenanceRecord.

AG-0900 **`PrunedPacketCoefficients`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.signals.wavelets`. Research cards: C.4B PrunedPacketCoefficients.

AG-0901 **`pushforward_identity`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.integration.identities`. Research cards: C.1D pushforward_identity.

AG-0902 **`pushforward_measure`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.integration.measures`. Research cards: C.1D pushforward_measure.

AG-0903 **`PushforwardMeasure`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.integration.measures`. Research cards: C.1D PushforwardMeasure.

AG-0904 **`q_binomial_support`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.q_kummer`. Research cards: C.3B q_binomial_support.

AG-0905 **`q_binomial_valuation_decomposition`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.q_kummer`. Research cards: C.3B q_binomial_valuation_decomposition, C.1F q_binomial_valuation_decomposition.

AG-0906 **`q_binomial_valuation_direct`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.q_kummer`. Research cards: C.3B q_binomial_valuation_direct.

AG-0907 **`q_factorial`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.q_kummer`. Research cards: C.3B q_factorial.

AG-0908 `q_factorial_basis`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.basis`. Research cards: C.3A q_factorial_basis.

AG-0909 `q_integer`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.q_kummer`. Research cards: C.3B q_integer.

AG-0910 `QKummerDecomposition`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.generating.q_kummer`. Research cards: C.3B QKummerDecomposition.

AG-0911 `quadratic_catalan_majorant`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.nonlinear.trees`. Research cards: C.5B quadratic_catalan_majorant, C.1E quadratic_catalan_majorant.

AG-0912 `QuadraticRefinement`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.langlands.local.symplectic`. Research cards: C.1G QuadraticRefinement.

AG-0913 `QuadratureExactness`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.integration.rules`. Research cards: C.1D QuadratureExactness.

AG-0914 `QuadratureRule`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.integration.rules`. Research cards: C.1D QuadratureRule.

AG-0915 `QuantumChannel`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.quantum.channels`. Research cards: C.1G QuantumChannel.

AG-0916 `QuantumCode`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.quantum.error_correction`. Research cards: C.1G QuantumCode.

AG-0917 `QuantumFeshbachReduction`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.quantum.effective`. Research cards: C.1G QuantumFeshbachReduction.

AG-0918 `QuantumSeedSystem`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.quantum.seeds`. Research cards: C.1G QuantumSeedSystem.

AG-0919 `QuasiPolynomialGuess`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.generating.closed_forms`. Research cards: C.3A QuasiPolynomialGuess.

AG-0920 `RamificationCliffordTower`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.langlands.ramification.towers`. Research cards: C.1G RamificationCliffordTower.

AG-0921 `RamificationStage`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.langlands.ramification.towers`. Research cards: C.1G RamificationStage.

AG-0922 `RATIONAL_DOMAIN`. Strongest listed tier `[RESEARCH]`. Kinds: `data`. Owners: `komorebi_math.generating.domains`. Research cards: C.3A RATIONAL_DOMAIN.

AG-0923 `rational_function`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.functions`. Research cards: C.3A rational_function.

AG-0924 `rational_gf_from_recurrence`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.closed_forms`. Research cards: C.3A rational_gf_from_recurrence.

AG-0925 `rational_mask_coefficients_mod_prime`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.cartier`. Research cards: C.3B rational_mask_coefficients_mod_prime.

AG-0926 `RATIONAL_SPACE`. Strongest listed tier `[RESEARCH]`. Kinds: `data`. Owners: `komorebi_math.expression.model`. Research cards: C.1B RATIONAL_SPACE.

AG-0927 `RationalDomain`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.generating.domains`. Research cards: C.3A RationalDomain.

AG-0928 `RationalMaskAlgebraicCertificate`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.generating.masks`. Research cards: C.3B RationalMaskAlgebraicCertificate.

AG-0929 `RationalResamplingPlan`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.signals.resampling`. Research cards: C.4A RationalResamplingPlan.

AG-0930 `RationalSpace`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.expression.model`. Research cards: C.1B RationalSpace.

AG-0931 `RawLatex`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.export.model`. Research cards: C.6 RawLatex.

AG-0932 **realization_defect**. Strongest listed tier [RESEARCH]. Kinds: function. Owners: komorebi_math.coordinates.realization. Research cards: C.2C realization_defect.

AG-0933 **reciprocal_coefficients**. Strongest listed tier [RESEARCH]. Kinds: function. Owners: komorebi_math.analytic.reciprocal. Research cards: C.1G reciprocal_coefficients.

AG-0934 **recompress_factors**. Strongest listed tier [RESEARCH]. Kinds: function. Owners: komorebi_math.algorithms.low_rank. Research cards: C.1D recompress_factors.

AG-0935 **reconstruction_error**. Strongest listed tier [RESEARCH]. Kinds: function. Owners: komorebi_math.signals.filterbanks. Research cards: C.4B reconstruction_error.

AG-0936 **recover_initial_state**. Strongest listed tier [RESEARCH]. Kinds: function. Owners: komorebi_math.coordinates.realization. Research cards: C.2C recover_initial_state, C.1C recover_initial_state.

AG-0937 **recover_jumps_subgaussian**. Strongest listed tier [RESEARCH]. Kinds: function. Owners: komorebi_math.signals.recovery. Research cards: C.4B recover_jumps_subgaussian.

AG-0938 **recover_separated_jumps**. Strongest listed tier [RESEARCH]. Kinds: function. Owners: komorebi_math.signals.recovery. Research cards: C.1F recover_separated_jumps, C.4B recover_separated_jumps.

AG-0939 **recover_vector_jumps**. Strongest listed tier [RESEARCH]. Kinds: function. Owners: komorebi_math.signals.edge_recovery. Research cards: C.4B recover_vector_jumps.

AG-0940 **recover_vector_jumps_subgaussian**. Strongest listed tier [RESEARCH]. Kinds: function. Owners: komorebi_math.signals.edge_recovery. Research cards: C.4B recover_vector_jumps_subgaussian.

AG-0941 **recurrence_analysis_transform**. Strongest listed tier [RESEARCH]. Kinds: function. Owners: komorebi_math.coordinates.recurrence. Research cards: C.2C recurrence_analysis_transform.

AG-0942 **recurrence_model_catalog**. Strongest listed tier [RESEARCH]. Kinds: function. Owners: komorebi_math.coordinates.families. Research cards: C.2C recurrence_model_catalog, C.1C recurrence_model_catalog.

AG-0943 **recurrence_polynomial**. Strongest listed tier [RESEARCH]. Kinds: function. Owners: komorebi_math.coordinates.recurrence. Research cards: C.2C recurrence_polynomial.

AG-0944 **recurrence_synthesis_transform**. Strongest listed tier [RESEARCH]. Kinds: function. Owners: komorebi_math.coordinates.recurrence. Research cards: C.2C recurrence_synthesis_transform.

AG-0945 **RecurrenceCoordinateRecord**. Strongest listed tier [RESEARCH]. Kinds: class. Owners: komorebi_math.coordinates.recurrence. Research cards: C.1C RecurrenceCoordinateRecord, C.2C RecurrenceCoordinateRecord.

AG-0946 **RecurrenceLike**. Strongest listed tier [RESEARCH]. Kinds: data. Owners: typing. Research cards: C.2C RecurrenceLike.

AG-0947 **RecurrenceModel**. Strongest listed tier [RESEARCH]. Kinds: class. Owners: komorebi_math.coordinates.families. Research cards: C.1C RecurrenceModel, C.2C RecurrenceModel.

AG-0948 **reduce_exact_transform_mod_prime**. Strongest listed tier [RESEARCH]. Kinds: function. Owners: komorebi_math.coordinates.modular. Research cards: C.2A reduce_exact_transform_mod_prime.

AG-0949 **register_coordinate_codecs**. Strongest listed tier [RESEARCH]. Kinds: function. Owners: komorebi_math.coordinates.portable. Research cards: C.2C register_coordinate_codecs.

AG-0950 **register_export_adapter**. Strongest listed tier [RESEARCH]. Kinds: function. Owners: komorebi_math.export.serialization. Research cards: C.6 register_export_adapter.

AG-0951 **register_generating_codecs**. Strongest listed tier [RESEARCH]. Kinds: function. Owners: komorebi_math.generating.portable. Research cards: C.3A register_generating_codecs.

AG-0952 **render_html**. Strongest listed tier [RESEARCH]. Kinds: function. Owners: komorebi_math.export.renderers. Research cards: C.6 render_html.

AG-0953 **render_text**. Strongest listed tier [RESEARCH]. Kinds: function. Owners: komorebi_math.export.renderers. Research cards: C.6 render_text.

AG-0954 **replay**. Strongest listed tier [RESEARCH]. Kinds: function. Owners: komorebi_math.expression.polynomial. Research cards: C.1B replay, C.1B replay_matrix.

AG-0955 **require**. Strongest listed tier [RESEARCH]. Kinds: function. Owners: komorebi_math.nonlinear. Research cards: C.5B require.

AG-0956 **`resample_poly`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.resampling`. Research cards: C.4A resample_poly.

AG-0957 **`resample_poly_result`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.resampling`. Research cards: C.4A resample_poly_result.

AG-0958 **`resampled_length`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.resampling`. Research cards: C.4A resampled_length.

AG-0959 **`ResamplingResult`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.signals.resampling`. Research cards: C.4A ResamplingResult.

AG-0960 **`residue_class_order`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.packet`. Research cards: C.2A residue_class_order.

AG-0961 **`residue_class_transform`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.packet`. Research cards: C.2A residue_class_transform, C.1C residue_class_transform.

AG-0962 **`residue_packet_system`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.refinement.residue`. Research cards: C.1G residue_packet_system.

AG-0963 **`ResidueRingDomain`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.generating.domains`. Research cards: C.3A ResidueRingDomain.

AG-0964 **`resolve_latex_engine`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.export.latex`. Research cards: C.6 resolve_latex_engine.

AG-0965 **`retained_packet_reconstruct`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.singular_packets`. Research cards: C.1F retained_packet_reconstruct, C.4B retained_packet_reconstruct.

AG-0966 **`retained_packet_reconstruction`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.singular_packets`. Research cards: C.4B retained_packet_reconstruction.

AG-0967 **`RetainedPacketReconstruction`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.signals.singular_packets`. Research cards: C.4B RetainedPacketReconstruction, C.4B RetainedPacketResult.

AG-0968 **`RewriteStep`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.expression.certification`. Research cards: C.1B RewriteStep.

AG-0969 **`RightInverseCorrectionProblem`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.correction`. Research cards: C.1E RightInverseCorrectionProblem, C.5A RightInverseCorrectionProblem.

AG-0970 **`RightInverseCorrectionResult`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.correction`. Research cards: C.5A RightInverseCorrectionResult.

AG-0971 **`rising_factorial_polynomial`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.classical`. Research cards: C.2B rising_factorial_polynomial.

AG-0972 **`rising_factorial_polynomial_family`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.classical`. Research cards: C.2B rising_factorial_polynomial_family.

AG-0973 **`rising_factorial_to_monomial_transform`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.combinatorial`. Research cards: C.2A rising_factorial_to_monomial_transform.

AG-0974 **`rising_to_falling_factorial_transform`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.combinatorial`. Research cards: C.2A rising_to_falling_factorial_transform.

AG-0975 **`rls_filter`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.adaptive`. Research cards: C.4B rls_filter.

AG-0976 **`root_filter_value`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.sections`. Research cards: C.3A root_filter_value.

AG-0977 **`rref`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.exact`. Research cards: C.2A rref.

AG-0978 **`save_coordinate_object`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.portable`. Research cards: C.2C save_coordinate_object, C.1C save_coordinate_object.

AG-0979 **`save_coordinate_record`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.portable`. Research cards: C.2C save_coordinate_record.

AG-0980 **`save_public_api_manifest`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.compatibility`. Research cards: C.1A save_public_api_manifest.

AG-0981 **`save_transform`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.portable`. Research cards: C.2C save_transform.

AG-0982 **`Scalar`**. Strongest listed tier `[RESEARCH]`. Kinds: `data`. Owners: `types`. Research cards: C.2C Scalar.

AG-0983 **`scalar_region_error`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.singular_packets`. Research cards: C.4B scalar_region_error.

AG-0984 **`scalar_region_lp_error`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.singular_packets`. Research cards: C.4B scalar_region_lp_error.

AG-0985 **`Scale`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.expression.model`. Research cards: C.1B Scale.

AG-0986 **`scale`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.expression.model`. Research cards: C.1B scale.

AG-0987 **`scaling_transform`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.basis`. Research cards: C.2C scaling_transform.

AG-0988 **`scan_residue_lanes`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.congruences`. Research cards: C.3B scan_residue_lanes.

AG-0989 **`schedule_unlimited`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.algorithms.scheduling`. Research cards: C.1D schedule_unlimited.

AG-0990 **`SchurDerivativeResult`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.elimination`. Research cards: C.5A SchurDerivativeResult.

AG-0991 **`SchurSecondDerivativeResult`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.elimination`. Research cards: C.5A SchurSecondDerivativeResult.

AG-0992 **`search_decimation_relation`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.congruences`. Research cards: C.3B search_decimation_relation.

AG-0993 **`secant_euler_from_fibonacci_coordinates`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.polynomials`. Research cards: C.2B secant_euler_from_fibonacci_coordinates.

AG-0994 **`SectionCycleAudit`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.generating.sections`. Research cards: C.3A SectionCycleAudit.

AG-0995 **`SeedMorphism`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.core.morphisms`. Research cards: C.1D SeedMorphism.

AG-0996 **`semisimple_seed_rank`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.core.seeds`. Research cards: C.1D semisimple_seed_rank.

AG-0997 **`Sequence`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.sequences.base`. Research cards: C.1G Sequence.

AG-0998 **`sequence_coordinates_between_models`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.families`. Research cards: C.2C sequence_coordinates_between_models.

AG-0999 **`sequence_document`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.export.report`. Research cards: C.6 sequence_document, C.1F sequence_document.

AG-1000 **`series_to_fir_adapter`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.adapters`. Research cards: C.3A series_to_fir_adapter.

AG-1001 **`series_to_fir_taps`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.adapters`. Research cards: C.3A series_to_fir_taps.

AG-1002 **`SeriesTransferCertificate`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.generating.finite_series`. Research cards: C.3A SeriesTransferCertificate.

AG-1003 **`set_partitions`**. Strongest listed tier `[RESEARCH]`. Kinds: `data`. Owners: `komorebi_math.nonlinear.composition`. Research cards: C.5A set_partitions.

AG-1004 **`shannon_entropy_cost`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.wavelets`. Research cards: C.4B shannon_entropy_cost.

AG-1005 **`shared_direct_sum_seed_bounds`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.core.seeds`. Research cards: C.1D shared_direct_sum_seed_bounds.

AG-1006 **`ShiftPolynomial`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.coordinates.operators`. Research cards: C.1C ShiftPolynomial, C.2C ShiftPolynomial.

AG-1007 **`signed_euler_number`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.arithmetic.euler`. Research cards: C.1G signed_euler_number.

AG-1008 **`signed_euler_numbers`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.sequences`. Research cards: C.3A signed_euler_numbers.

AG-1009 **`signed_secant_euler_numbers`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.recurrence`. Research cards: C.2C signed_secant_euler_numbers.

AG-1010 **`signed_section`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.analytic.sections`. Research cards: C.1G signed_section.

AG-1011 **`similarity_from_cyclic_vectors`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.realization`. Research cards: C.2C similarity_from_cyclic_vectors.

AG-1012 **`SimplificationResult`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.expression.engine`. Research cards: C.1B SimplificationResult.

AG-1013 **`simplify_expression`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.expression.engine`. Research cards: C.1B simplify_expression.

AG-1014 **`single_operator_seed_rank`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.core.seeds`. Research cards: C.1D single_operator_seed_rank.

AG-1015 **`SingularPacket1D`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.signals.singular_packets`. Research cards: C.1F SingularPacket1D, C.4B SingularPacket1D.

AG-1016 **`siso_state_space`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.state_space`. Research cards: C.2C siso_state_space.

AG-1017 **`smith_sensitivity`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.smith`. Research cards: C.4B smith_sensitivity.

AG-1018 **`SmithCongruenceProblem`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.refinement.smith`. Research cards: C.1G SmithCongruenceProblem.

AG-1019 **`smooth_haar_coefficient_bound`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.edge_recovery`. Research cards: C.4B smooth_haar_coefficient_bound.

AG-1020 **`solve_linear_system`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.exact`. Research cards: C.2A solve_linear_system.

AG-1021 **`solve_mod_prime`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.nonlinear.lifting`. Research cards: C.5A solve_mod_prime, C.1E solve_mod_prime.

AG-1022 **`solve_polynomial_coboundary`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.masks`. Research cards: C.3B solve_polynomial_coboundary.

AG-1023 **`sorted_coordinate_transform`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.packet`. Research cards: C.2A sorted_coordinate_transform.

AG-1024 **`spectral_dimension`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.quantum.spectral`. Research cards: C.1G spectral_dimension.

AG-1025 **`SpectralAverage`**. Strongest listed tier `[RESEARCH]`. Kinds: `data`. Owners: `typing`. Research cards: C.4A SpectralAverage.

AG-1026 **`SpectralScaling`**. Strongest listed tier `[RESEARCH]`. Kinds: `data`. Owners: `typing`. Research cards: C.4A SpectralScaling.

AG-1027 **`SpectralSequenceEntry`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.filtered`. Research cards: C.5A SpectralSequenceEntry.

AG-1028 `SpectralSequencePage`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.filtered`. Research cards: C.5A SpectralSequencePage.

AG-1029 `spectrogram`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.time_frequency`. Research cards: C.4A spectrogram.

AG-1030 `SpectrogramResult`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.signals.time_frequency`. Research cards: C.4A SpectrogramResult.

AG-1031 `SpectrogramScaling`. Strongest listed tier `[RESEARCH]`. Kinds: `data`. Owners: `typing`. Research cards: C.4A SpectrogramScaling.

AG-1032 `split_mixed_radix_address`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.polyphase`. Research cards: C.4B split_mixed_radix_address.

AG-1033 `STAGES`. Strongest listed tier `[RESEARCH]`. Kinds: `data`. Owners: `builtins`. Research cards: C.1A STAGES.

AG-1034 `StageSpec`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.unified`. Research cards: C.1G StageSpec.

AG-1035 `StandardWindow`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.arithmetic.standard_windows`. Research cards: C.1G StandardWindow.

AG-1036 `StateRecovery`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.coordinates.realization`. Research cards: C.1C StateRecovery, C.2C StateRecovery.

AG-1037 `StateSpaceCoordinateCertificate`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.coordinates.state_space`. Research cards: C.2C StateSpaceCoordinateCertificate.

AG-1038 `StateSpaceModel`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.coordinates.state_space`. Research cards: C.1C StateSpaceModel, C.2C StateSpaceModel.

AG-1039 `status`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.nonlinear`. Research cards: C.5A status.

AG-1040 `status`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals`. Research cards: C.4A status.

AG-1041 `step_haar_coefficient`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.edge_recovery`. Research cards: C.4B step_haar_coefficient.

AG-1042 `step_location_lp_error`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.singular_packets`. Research cards: C.4B step_location_lp_error.

AG-1043 `stft`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.time_frequency`. Research cards: C.4A stft.

AG-1044 `STFTResult`. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.signals.time_frequency`. Research cards: C.4A STFTResult.

AG-1045 `stirling_first_signed`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.combinatorial`. Research cards: C.2A stirling_first_signed.

AG-1046 `stirling_first_unsigned`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.combinatorial`. Research cards: C.2A stirling_first_unsigned.

AG-1047 `stirling_second`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.combinatorial`. Research cards: C.2A stirling_second.

AG-1048 `stirling_second`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.nonlinear.composition`. Research cards: C.5A stirling_second.

AG-1049 `stream_fir_filter`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.filtering`. Research cards: C.4A stream_fir_filter.

AG-1050 `stream_lfilter`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.recursive`. Research cards: C.4A stream_lfilter.

AG-1051 `stream_stft`. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.streaming`. Research cards: C.4A stream_stft.

AG-1052 **`StreamingFIR`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.signals.filtering`. Research cards: C.4A StreamingFIR.

AG-1053 **`StreamingLinearFilter`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.signals.recursive`. Research cards: C.4A StreamingLinearFilter.

AG-1054 **`StreamingSTFT`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.signals.streaming`. Research cards: C.4A StreamingSTFT.

AG-1055 **`StreamingSTFTBatch`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.signals.streaming`. Research cards: C.4A StreamingSTFTBatch.

AG-1056 **`subgaussian_noise_bound`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.recovery`. Research cards: C.4B subgaussian_noise_bound.

AG-1057 **`subset_zeta_transform`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.combinatorial`. Research cards: C.2A subset_zeta_transform.

AG-1058 **`Sum`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.expression.model`. Research cards: C.1B Sum.

AG-1059 **`SupportComparison`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.langlands.global_.hecke`. Research cards: C.1G SupportComparison.

AG-1060 **`SupportPreorder`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.core.graphs`. Research cards: C.1D SupportPreorder.

AG-1061 **`sure_threshold`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.denoise`. Research cards: C.4B sure_threshold.

AG-1062 **`Symbol`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.expression.model`. Research cards: C.1B Symbol.

AG-1063 **`symbol`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.expression.model`. Research cards: C.1B symbol.

AG-1064 **`SympyDerivativeProvider`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.derivatives`. Research cards: C.1E SympyDerivativeProvider, C.5A SympyDerivativeProvider.

AG-1065 **`synthesize_recurrence_coordinates`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.recurrence`. Research cards: C.2C synthesize_recurrence_coordinates.

AG-1066 **`Table`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.export.model`. Research cards: C.1F Table, C.6 Table.

AG-1067 **`table_rows`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.export.serialization`. Research cards: C.6 table_rows.

AG-1068 **`TameWeilPacket`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.langlands.local.tame`. Research cards: C.1G TameWeilPacket.

AG-1069 **`Task`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.algorithms.scheduling`. Research cards: C.1D Task.

AG-1070 **`tensor_axis_permutation_order`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.packet`. Research cards: C.2A tensor_axis_permutation_order.

AG-1071 **`tensor_axis_permutation_transform`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.packet`. Research cards: C.2A tensor_axis_permutation_transform.

AG-1072 **`tensor_coordinate_transform`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.tensor`. Research cards: C.2A tensor_coordinate_transform, C.1C tensor_coordinate_transform.

AG-1073 **`tensor_product_basis`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.basis`. Research cards: C.2C tensor_product_basis.

AG-1074 **`tensor_product_transform`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.tensor`. Research cards: C.2A tensor_product_transform.

AG-1075 **`tensor_seed_rank_upper_bound`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.core.seeds`. Research cards: C.1D tensor_seed_rank_upper_bound.

AG-1076 **`then`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.expression.model`. Research cards: C.1B then.

AG-1077 **`theta_polynomial`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.analytic.appell`. Research cards: C.1G theta_polynomial.

AG-1078 **`threshold_coefficients`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.denoise`. Research cards: C.4B threshold_coefficients.

AG-1079 **`ThresholdMode`**. Strongest listed tier `[RESEARCH]`. Kinds: `data`. Owners: `typing`. Research cards: C.4A ThresholdMode.

AG-1080 **`ThresholdRule`**. Strongest listed tier `[RESEARCH]`. Kinds: `data`. Owners: `typing`. Research cards: C.4A ThresholdRule.

AG-1081 **`thue_morse_sequence`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.sequences.catalog`. Research cards: C.1G thue_morse_sequence.

AG-1082 **`to_serializable`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.export.serialization`. Research cards: C.6 to_serializable, C.1F to_serializable.

AG-1083 **`TorchDerivativeProvider`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.autodiff`. Research cards: C.1E TorchDerivativeProvider, C.5A TorchDerivativeProvider.

AG-1084 **`transfer_section`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.analytic.sections`. Research cards: C.1G transfer_section.

AG-1085 **`transfer_tangent`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.analytic.reciprocal`. Research cards: C.1G transfer_tangent.

AG-1086 **`transform_sequence`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.combinatorial`. Research cards: C.2A transform_sequence.

AG-1087 **`translation_transform`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.affine`. Research cards: C.2C translation_transform.

AG-1088 **`transport_bilinear_form`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.tensor`. Research cards: C.2A transport_bilinear_form.

AG-1089 **`transport_covector`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.tensor`. Research cards: C.2A transport_covector.

AG-1090 **`transport_operator`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.tensor`. Research cards: C.2A transport_operator.

AG-1091 **`transport_polynomial_operator`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.calculus`. Research cards: C.2B transport_polynomial_operator.

AG-1092 **`transport_rectangular_operator`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.tensor`. Research cards: C.2A transport_rectangular_operator.

AG-1093 **`transport_tensor_components`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.tensor`. Research cards: C.2A transport_tensor_components.

AG-1094 **`TransportEdge`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.holonomy`. Research cards: C.1E TransportEdge, C.5A TransportEdge.

AG-1095 **`TransportGraph`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.holonomy`. Research cards: C.1E TransportGraph, C.5A TransportGraph.

AG-1096 **`TransportRefinementResult`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.transport`. Research cards: C.5A TransportRefinementResult.

AG-1097 **`transpose`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.exact`. Research cards: C.2A transpose.

AG-1098 **`TreeExpansionResult`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.trees`. Research cards: C.5B TreeExpansionResult.

AG-1099 **`TreeNode`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.algorithms.compiler`. Research cards: C.1D TreeNode.

AG-1100 **`TreeTerm`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.nonlinear.trees`. Research cards: C.5B TreeTerm.

AG-1101 **`tribonacci_recurrence_model`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.families`. Research cards: C.2C tribonacci_recurrence_model.

AG-1102 **`TruncatedPadic`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.arithmetic.domains`. Research cards: C.1G TruncatedPadic.

AG-1103 **`TruncatedPadicDomain`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.generating.domains`. Research cards: C.3A TruncatedPadicDomain.

AG-1104 **`TruncatedPowerPacket`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.signals.singular_packets`. Research cards: C.4B TruncatedPowerPacket, C.4B TruncatedPowerTerm.

AG-1105 **`type_from_dict`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.expression.portable`. Research cards: C.1B type_from_dict.

AG-1106 **`type_to_dict`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.expression.portable`. Research cards: C.1B type_to_dict.

AG-1107 **`UnifiedStage`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.unified`. Research cards: C.1G UnifiedStage.

AG-1108 **`UnifiedSystem`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.unified`. Research cards: C.1G UnifiedSystem.

AG-1109 **`universal_threshold`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.denoise`. Research cards: C.4B universal_threshold.

AG-1110 **`upsample`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.resampling`. Research cards: C.4A upsample.

AG-1111 **`validate_cupy_snapshot`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.nonlinear.gpu`. Research cards: C.5B validate_cupy_snapshot, C.1E validate_cupy_snapshot.

AG-1112 **`Variance`**. Strongest listed tier `[RESEARCH]`. Kinds: `data`. Owners: `typing`. Research cards: C.2C Variance.

AG-1113 **`variation_of_constants_prefix`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.recurrence`. Research cards: C.2C variation_of_constants_prefix, C.1C variation_of_constants_prefix.

AG-1114 **`Vector`**. Strongest listed tier `[RESEARCH]`. Kinds: `data`. Owners: `builtins`. Research cards: C.2C Vector.

AG-1115 **`VectorJumpRecoveryResult`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.signals.edge_recovery`. Research cards: C.4B VectorJumpRecoveryResult.

AG-1116 **`verify_cyclotomic_coboundary`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.cyclotomic`. Research cards: C.3B verify_cyclotomic_coboundary.

AG-1117 **`verify_cyclotomic_exponent_relation`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.cyclotomic`. Research cards: C.3B verify_cyclotomic_exponent_relation.

AG-1118 **`verify_finite_filtered_branch`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.nonlinear.filtered`. Research cards: C.5A verify_finite_filtered_branch, C.1E verify_finite_filtered_branch.

AG-1119 **`VisibilityTree`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.deformation.visibility`. Research cards: C.1G VisibilityTree.

AG-1120 **`walsh_hadamard_transform`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.numeric`. Research cards: C.2C walsh_hadamard_transform.

AG-1121 **`WalshHadamardOperator`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.algorithms.structured`. Research cards: C.1D WalshHadamardOperator.

AG-1122 **`warn_deprecated`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.compatibility`. Research cards: C.1A warn_deprecated.

AG-1123 **`wavelet_denoise`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.denoise`. Research cards: C.4B wavelet_denoise.

AG-1124 **`WaveletDenoiseResult`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.signals.denoise`. Research cards: C.4B WaveletDenoiseResult.

AG-1125 **`weakest_exactness`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.expression.certification`. Research cards: C.1B weakest_exactness.

AG-1126 **`weighted_bell_digit_reduction`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.bell`. Research cards: C.3B weighted_bell_digit_reduction.

AG-1127 **`weighted_factorial_power_bell_function`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.functions`. Research cards: C.3A weighted_factorial_power_bell_function.

AG-1128 **`weighted_generalized_bell_numbers`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.sequences`. Research cards: C.3A weighted_generalized_bell_numbers.

AG-1129 **`weighted_generalized_bell_polynomials`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.sequences`. Research cards: C.3A weighted_generalized_bell_polynomials.

AG-1130 **`WeightedTransferEdge`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.refinement.weighted`. Research cards: C.1G WeightedTransferEdge.

AG-1131 **`WeightedTransferGraph`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.refinement.weighted`. Research cards: C.1G WeightedTransferGraph.

AG-1132 **`welch_psd`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.spectral`. Research cards: C.4A welch_psd.

AG-1133 **`window_metrics`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.signals.windows`. Research cards: C.4A window_metrics.

AG-1134 **`WindowMetrics`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.signals.windows`. Research cards: C.4A WindowMetrics.

AG-1135 **`WindowName`**. Strongest listed tier `[RESEARCH]`. Kinds: `data`. Owners: `typing`. Research cards: C.4A WindowName.

AG-1136 **`WindowSpec`**. Strongest listed tier `[RESEARCH]`. Kinds: `data`. Owners: `typing`. Research cards: C.4A WindowSpec.

AG-1137 **`zero_borrow_count`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.generating.masks`. Research cards: C.3B zero_borrow_count.

AG-1138 **`zero_map`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.expression.model`. Research cards: C.1B zero_map.

AG-1139 **`zero_matrix`**. Strongest listed tier `[RESEARCH]`. Kinds: `function`. Owners: `komorebi_math.coordinates.exact`. Research cards: C.2A zero_matrix.

AG-1140 **`ZeroMap`**. Strongest listed tier `[RESEARCH]`. Kinds: `class`. Owners: `komorebi_math.expression.model`. Research cards: C.1B ZeroMap.

> **Reference connections.** The 1,312 research cards and four derived indexes connect public routes and implementation owners. Appendix D joins those owners and routes to the source-module ledger. These indexes record the reference baseline and do not by themselves qualify a new release.

Appendix D. Module reference ledger

Using the module ledger. Start with an API card when you need to call an operation. Use this appendix when you need to locate its implementation, understand its neighboring modules, or trace a dependency or diagnostic. The family summary narrows the search; the module card then gives the source path and links to the API names it owns.

Module-card field	How to use it
Purpose and source	Identify the module and the source revision represented by the record.
Surface and ownership	Distinguish an implementation owner from a facade or internal support module; follow the linked API card for a public contract.
Direct package dependencies	See the modules this module imports directly in the recorded analysis.
Reverse package dependents	See which package modules import this module directly.
Examples, tests, and documentation	Find files with the recorded direct references, then read them in context.

A facade is a convenient import surface that exposes objects implemented elsewhere. For instance, a stable route can lead to an implementation in a domain module. Follow the stable card when deciding which compatibility promise your program relies on; follow the implementation path when reading the source. Importability by itself does not make an internal module a supported application interface.

Dependency lists in these cards describe direct references. They are useful for tracing an import, but they are not a complete installation recipe or a measurement of runtime memory. Consult Appendix J for dependencies and Appendix K for resource guidance. Likewise, a zero in an example or test column means that the particular static join found no direct match. It does not establish that no code path ever exercises the module.

The printed cards abbreviate long file and edge lists. Section D.4 identifies the machine-readable ledger containing the complete arrays. Use Appendix O for the separate current-source test and verification inventory. Keeping these two inventories distinct prevents a reference-source count from being mistaken for a count of a later source tree.

Appendix D records every Python source module in the reference package under `src/komorebi_math`. It separates public API ownership from importability, package facades, internal support, verification, release tooling, command-line implementation, backend adapters, and extension infrastructure. A module can be importable without being covered by a stable, beta, or research compatibility promise.

$$M = (module, source, role, public, owner, domain, exports, tiers, deps, examples, tests, docs, status).$$

The inventory is derived from the reference filesystem, the three committed API manifests, runtime imports, module docstrings, declared export lists, direct import edges, and exact references found in examples, tests, and documentation. Evidence file lists are static direct-reference joins. A zero in that join does not prove that a module is behaviorally untested through another facade. Purpose descriptions summarize source docstrings. Exactness and qualification remain governed by the object contracts and recorded evidence, rather than by a module label.

D.1 Inventory, roles, and closure rules

Inventory	Count
Package source modules	300
Package initializers	30
Ordinary module files	270
Package modules owning at least one API card	202
Package modules owning at least one research card	196
Package modules with no manifest-owned card	98
Implementation-owner strings across all tiers	207
Research implementation-owner strings	201
External standard-library owner strings	5
Direct package dependency edges	1,591

Module role	Modules
public implementation owner	200
installed verification module	28
importable package facade outside API manifests	17
unmanifested implementation support	17
internal support module	10
canonical API namespace facade	9
release infrastructure module	7
backend adapter module	4
command line implementation module	3
extension infrastructure module	3
canonical API namespace facade and owner	2
Total	**300**

A module is marked as an API implementation owner only when at least one committed manifest card names it as `implementation_module`. Canonical namespace facades are recorded separately because they re-export objects whose implementations usually live elsewhere. Modules with a private path segment remain internal even when they define `__all__`. Verification and release modules are shipped operational surfaces, not compatibility tiers.

D.1.1 Family summary

Family	Modules	Initializers	API owners	Research owners
`package root`	22	1	11	7
`api`	2	1	0	0
`beta`	1	1	0	0
`backends`	4	1	0	0
`core`	10	1	9	9
`algorithms`	15	1	9	9
`arithmetic`	10	1	8	7
`sequences`	7	1	5	5
`analytic`	7	1	5	5
`refinement`	10	1	8	7
`coordinates`	24	1	23	23
`generating`	22	1	21	21
`signals`	21	1	21	21
`nonlinear`	22	1	22	22

Family	Modules	Initializers	API owners	Research owners
`integration`	8	1	7	7
`dynamics`	12	1	7	7
`quantum`	15	1	11	11
`deformation`	7	1	5	5
`langlands`	24	6	17	17
`expression`	7	1	6	6
`export`	8	1	7	7
`extensions`	3	1	0	0
`cli`	3	1	0	0
`release`	8	1	0	0
`verify`	28	1	0	0
Total package modules	**300**	**30**	**202**	**196**

Family	Stable cards	Beta cards	Research cards
`package root`	35	20	36
`api`	0	0	0
`beta`	0	0	0
`backends`	0	0	0
`core`	14	1	25
`algorithms`	0	0	19
`arithmetic`	6	0	10
`sequences`	0	0	11
`analytic`	0	0	11
`refinement`	20	0	12
`coordinates`	14	0	278
`generating`	3	0	206
`signals`	10	0	162
`nonlinear`	0	39	229
`integration`	0	0	38
`dynamics`	0	0	7
`quantum`	0	0	12
`deformation`	0	0	6
`langlands`	0	0	28
`expression`	0	0	77
`export`	17	13	90
`extensions`	0	0	0
`cli`	0	0	0
`release`	0	0	0
`verify`	0	0	0
Package-owned API cards	**119**	**73**	**1257**

The family table counts cards owned by package modules. The five external owner rows account for the remaining manifest cards.

Ownership source	Stable cards	Beta cards	Research cards	Total
Package source modules	119	73	1257	1449
External owner rows	12	10	55	77
All manifest cards	**131**	**83**	**1312**	**1526**

Ownership source	Stable cards	Beta cards	Research cards	Total

D.2 Package source-module cards

The 300 cards below are ordered by package family and fully qualified module name. Each card states the module role, exact source path, source size and digest, manifest ownership, declared module-level names, direct dependencies, reverse package dependents, and direct example, test, and documentation references. Full lists are retained in the JSON and CSV ledgers when a printed card abbreviates a long field.

D.2.1 Package Root: 22 modules

D.2.1.1 `komorebi_math`

Purpose and source. Executable mathematics for the Komorebi Math research program. It is the package initializer for the `package root` family. The ledger classifies it as a canonical API namespace facade. Source `src/komorebi_math/__init__.py` contains 1,072 lines and 27,312 bytes. Source SHA-256 begins `b427cf245f4876c3`.

Surface and ownership. API classification: canonical public namespace without implementation ownership. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its `__all__` declaration contains 516 names: `__version__`, `add`, `affine_interval_change_of_variables`, `affine_mahler_kernel`, `affine_substitution`, `AffineCoordinateTransform`, `Algebra BasisResult`, `all_fubini_orders`, `AlternatingForm`, `analysis_synthesis_integration_certificate`, and 506 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (96): `komorebi_math._features`, `komorebi_math._version`, `komorebi_math.algorithms`, `komorebi_math.analytic`, `komorebi_math.arithmetic`, `komorebi_math.certificates`, and 90 more in the machine-readable ledger. Reverse package dependents (0): none. Examples (65): `automatic_holonomic.py`, `block11_semantic_profile.py`, `block15_polyphase_refinement.py`, `block16_analysis_synthesis.py`, `block19_measure_integration_worked_example.py`, and 60 more in the machine-readable ledger. Tests or test support files (126): `optional_backend_support.py`, `test_algebra_quotients.py`, `test_algorithms_compiler.py`, `test_algorithms_complexity_benchmarks.py`, `test_algorithms_discovery_gpu.py`, and 121 more in the machine-readable ledger. Documentation files with literal module or public-route references (43): `ANALYSIS_SYNTHESIS_SYSTEMS.md`, `API_COMPATIBILITY_0.11.md`, `API_COMPATIBILITY_0.12.md`, `API_COMPATIBILITY_0.13.md`, `API_COMPATIBILITY_0.14.md`, and 38 more in the machine-readable ledger. AST parsing and runtime import both passed in the recorded module audit.

D.2.1.2 `komorebi_math.__main__`

Purpose and source. Package command entry point. It is an ordinary module in the `package root` family. The ledger classifies it as an internal support module. Source `src/komorebi_math/__main__.py` contains 4 lines and 84 bytes. Source SHA-256 begins `39a0ee4edd1039df`.

Surface and ownership. API classification: internal by module name. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Static analysis found no top-level public definitions.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (1): `komorebi_math.cli.main`. Reverse package dependents (0): none. Examples (8): `Test-KomorebiBlock10.ps1`, `Test-KomorebiBlock11.ps1`, `Test-KomorebiBlock12.ps1`, `Test-KomorebiBlock13.ps1`, `Test-KomorebiBlock14.ps1`, and 3 more in the machine-readable ledger. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.1.3 komorebi_math._features

Purpose and source. Built-in feature declarations. It is an ordinary module in the package root family. The ledger classifies it as an internal support module. Source src/komorebi_math/_features.py contains 370 lines and 15,118 bytes. Source SHA-256 begins 87e70493395fae52.

Surface and ownership. API classification: internal by module name. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Static analysis found 1 top-level public definition: register_builtin_features.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (1): komorebi_math.registry. Reverse package dependents (1): komorebi_math. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.1.4 komorebi_math._number_theory

Purpose and source. Small exact number-theoretic validation helpers used across the package. It is an ordinary module in the package root family. The ledger classifies it as an internal support module. Source src/komorebi_math/_number_theory.py contains 52 lines and 1,439 bytes. Source SHA-256 begins d125f065a96f24bb.

Surface and ownership. API classification: internal by module name. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its __all__ declaration contains 3 names: is_prime, require_integer, require_prime.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (1): komorebi_math.exceptions. Reverse package dependents (33): komorebi_math.arithmetic.domains, komorebi_math.arithmetic.euler, komorebi_math.arithmetic.hurwitz, komorebi_math.arithmetic.local_modes, komorebi_math.arithmetic.mahler, komorebi_math.arithmetic.polyphase, and 27 more in the machine-readable ledger. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.1.5 komorebi_math._planned

Purpose and source. Shared utilities for declared but unimplemented feature modules. It is an ordinary module in the package root family. The ledger classifies it as an internal support module. Source src/komorebi_math/_planned.py contains 20 lines and 705 bytes. Source SHA-256 begins 660a23f170c6fe9d.

Surface and ownership. API classification: internal by module name. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Static analysis found 2 top-level public definitions: development_status, require_implemented.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (2): komorebi_math.exceptions, komorebi_math.registry. Reverse package dependents (12): komorebi_math.analytic, komorebi_math.arithmetic, komorebi_math.deformation, komorebi_math.dynamics, komorebi_math.langlands.completion, komorebi_math.langlands.finite_image, and 6 more in the machine-readable ledger. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.1.6 komorebi_math._version

Purpose and source. Installed package version constant. It is an ordinary module in the package root family. The ledger classifies it as an internal support module. Source src/komorebi_math/_version.py contains 1 lines and 24 bytes. Source SHA-256 begins bfdd9f8d756a3db5.

Surface and ownership. API classification: internal by module name. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Static analysis found no top-level public definitions.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (0): none. Reverse package dependents (13): `komorebi_math`, `komorebi_math.beta`, `komorebi_math.compatibility`, `komorebi_math.export.persistence`, `komorebi_math.nonlinear.streaming`, `komorebi_math.persistence_contract`, and 7 more in the machine-readable ledger. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.1.7 `komorebi_math.certificates`

Purpose and source. Typed certificates and result wrappers. It is an ordinary module in the `package root` family. The ledger classifies it as a public implementation owner. Source `src/komorebi_math/certificates.py` contains 86 lines and 2,435 bytes. Source SHA-256 begins `a437769b196f24ba`.

Surface and ownership. API classification: API-manifested implementation owner. It owns 12 manifest cards: 4 stable, 4 beta, and 4 research. Linked cards: Certificate, ComputationResult, ConditionalResult, Exactness, Certificate, ComputationResult, ConditionalResult, Exactness, Certificate, ComputationResult, and 2 more in the machine-readable ledger. Static analysis found 4 top-level public definitions: `Certificate`, `ComputationResult`, `ConditionalResult`, `Exactness`.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (1): `komorebi_math.exceptions`. Reverse package dependents (169): `komorebi_math`, `komorebi_math.algorithms.compiler`, `komorebi_math.algorithms.discovery`, `komorebi_math.algorithms.gpu`, `komorebi_math.algorithms.ir`, `komorebi_math.algorithms.krylov`, and 163 more in the machine-readable ledger. Examples (2): `nonlinear_jets_composition.py`, `scalable_nonlinear.py`. Tests or test support files (15): `test_analytic_appell_zeros.py`, `test_block10_execution_persistence.py`, `test_block10_sparse_autodiff.py`, `test_block11_streaming_migrations.py`, `test_block19_integration.py`, and 10 more in the machine-readable ledger. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.1.8 `komorebi_math.compatibility`

Purpose and source. Deterministic public-API manifests and additive compatibility checks. It is an ordinary module in the `package root` family. The ledger classifies it as a public implementation owner. Source `src/komorebi_math/compatibility.py` contains 653 lines and 23,790 bytes. Source SHA-256 begins `9422447e4d0c996c`.

Surface and ownership. API classification: API-manifested implementation owner. It owns 26 manifest cards: 6 stable, 6 beta, and 14 research. Linked cards: ApiCompatibilityReport, check_stable_api, create_stable_api_manifest, load_stable_api_manifest, PublicApiManifest, PublicApiSymbol, ApiCompatibilityReport, check_beta_api, create_beta_api_manifest, load_beta_api_manifest, and 16 more in the machine-readable ledger. Its `__all__` declaration contains 29 names: `API_STABILITY_LEVEL`, `ApiCompatibilityReport`, `BETA_API_STABILITY_LEVEL`, `BETA_PUBLIC_NAMESPACES`, `check_beta_api`, `check_public_api`, `check_stable_api`, `compare_public_api`, `create_beta_api_manifest`, `create_public_api_manifest`, and 19 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (3): `komorebi_math._version`, `komorebi_math.certificates`, `komorebi_math.exceptions`. Reverse package dependents (13): `komorebi_math`, `komorebi_math.beta`, `komorebi_math.cli.main`, `komorebi_math.qualification`, `komorebi_math.release.contract`, `komorebi_math.release.documentation`, and 7 more in the machine-readable ledger. Examples (1): `block21_public_capability_tour.py`. Tests or test support files (13): `test_block10_cli.py`, `test_block10_compatibility.py`, `test_block11_golden_profiling_beta.py`, `test_block12_cli_api.py`, `test_block13_cli_api.py`, and 8 more in the machine-readable ledger. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.1.9 `komorebi_math.config`

Purpose and source. Runtime settings for backend and validation defaults. It is an ordinary module in the `package root` family. The ledger classifies it as a public implementation owner. Source `src/komorebi_math/config.py` contains 60 lines and 1,643 bytes. Source SHA-256 begins `b6a5591e3d32ed49`.

Surface and ownership. API classification: API-manifested implementation owner. It owns 8 manifest cards: 4 stable, 0 beta, and 4 research. Linked cards: configure, get_settings, Settings, settings_context, configure, get_settings, Settings, settings_context. Static analysis found 4 top-level public definitions: `configure`, `get_settings`, `Settings`, `settings_context`.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (0): none. Reverse package dependents (8): `komorebi_math`, `komorebi_math.core.algebra`, `komorebi_math.core.packets`, `komorebi_math.core.realization`, `komorebi_math.core.seeds`, `komorebi_math.qualification`, and 2 more in the machine-readable ledger. Examples (0): none. Tests or test support files (1): `test_certificates_registry.py`. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.1.10 `komorebi_math.exceptions`

Purpose and source. Package-specific exception hierarchy. It is an ordinary module in the `package root` family. The ledger classifies it as a public implementation owner. Source `src/komorebi_math/exceptions.py` contains 39 lines and 1,157 bytes. Source SHA-256 begins `65aee29fea129eec`.

Surface and ownership. API classification: API-manifested implementation owner. It owns 13 manifest cards: 7 stable, 6 beta, and 0 research. Linked cards: AssumptionError, BackendUnavailableError, DimensionError, ExactnessError, KomorebiMathError, ResourceLimitError, ValidationError, AssumptionError, BackendUnavailableError, DimensionError, and 3 more in the machine-readable ledger. Static analysis found 9 top-level public definitions: `AssumptionError`, `BackendUnavailableError`, `DimensionError`, `ExactnessError`, `FeatureNotImplementedError`, `KomorebiMathError`, `RelationError`, `ResourceLimitError`, `ValidationError`.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (0): none. Reverse package dependents (214): `komorebi_math._number_theory`, `komorebi_math._planned`, `komorebi_math.algorithms._validation`, `komorebi_math.algorithms.benchmarks`, `komorebi_math.algorithms.compiler`, `komorebi_math.algorithms.complexity`, and 208 more in the machine-readable ledger. Examples (2): `block19_measure_integration_worked_example.py`, `portable_nonlinear_state.py`. Tests or test support files (70): `test_algorithms_compiler.py`, `test_algorithms_complexity_benchmarks.py`, `test_algorithms_discovery_gpu.py`, `test_algorithms_krylov.py`, `test_algorithms_layout_ir.py`, and 65 more in the machine-readable ledger. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.1.11 `komorebi_math.golden`

Purpose and source. Deterministic behavioral fixtures for release-level semantic regression. It is an ordinary module in the `package root` family. The ledger classifies it as a public implementation owner. Source `src/komorebi_math/golden.py` contains 625 lines and 23,737 bytes. Source SHA-256 begins `af7f39387bb8aa35`.

Surface and ownership. API classification: API-manifested implementation owner. It owns 8 manifest cards: 0 stable, 4 beta, and 4 research. Linked cards: BehavioralGoldenCaseResult, BehavioralGoldenReport, load_behavioral_golden_fixture, run_behavioral_golden_fixture, BehavioralGoldenCaseResult, BehavioralGoldenReport, load_behavioral_golden_fixture, run_behavioral_golden_fixture. Its `__all__` declaration contains 7 names: `BEHAVIORAL_GOLDEN_SCHEMA`, `BEHAVIORAL_GOLDEN_SCHEMA_VERSION`, `BehavioralGoldenCaseResult`, `BehavioralGoldenReport`, `DEFAULT_BEHAVIORAL_GOLDEN_FIXTURE`, `load_behavioral_golden_fixture`, `run_behavioral_golden_fixture`.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (22): `komorebi_math.certificates`, `komorebi_math.coordinates`, `komorebi_math.coordinates.basis`, `komorebi_math.coordinates.classical`, `komorebi_math.coordinates.combinatorial`, `komorebi_math.coordinates.formal`, and 16 more in the machine-readable ledger. Reverse package dependents (4): `komorebi_math`, `komorebi_math.beta`, `komorebi_math.cli.main`, `komorebi_math.verify.streaming_beta`. Examples (1): `block11_semantic_profile.py`. Tests or test support files (1): `test_block11_golden_profiling_beta.py`. Documentation files with literal module or public-route references

(1): BEHAVIORAL_GOLDEN_AND_PROFILING.md. AST parsing and runtime import both passed in the recorded module audit.

D.2.1.12 komorebi_math.persistence_contract

Purpose and source. Frozen portable-codec inventory and migration-path compatibility checks. It is an ordinary module in the package root family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/persistence_contract.py contains 333 lines and 13,146 bytes. Source SHA-256 begins f70374c8b7101775.

Surface and ownership. API classification: API-manifested implementation owner. It owns 8 manifest cards: 8 stable, 0 beta, and 0 research. Linked cards: check_portable_codec_contract, compare_portable_codec_contract, create_portable_codec_contract, load_portable_codec_contract, PortableCodecCompatibilityReport, PortableCodecContract, PortableCodecContractEntry, save_portable_codec_contract. Its __all__ declaration contains 11 names: check_portable_codec_contract, compare_portable_codec_contract, create_portable_codec_contract, DEFAULT_PORTABLE_CODEC_CONTRACT, load_portable_codec_contract, PORTABLE_CODEC_CONTRACT_SCHEMA, PORTABLE_CODEC_CONTRACT_SCHEMA_VERSION, PortableCodecCompatibilityReport, PortableCodecContract, PortableCodecContractEntry, and 1 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (4): komorebi_math._version, komorebi_math.certificates, komorebi_math.exceptions, komorebi_math.export.persistence. Reverse package dependents (6): komorebi_math.release.contract, komorebi_math.release.upgrade, komorebi_math.stable, komorebi_math.stable_contract, komorebi_math.verify.release, komorebi_math.verify.stabilization. Examples (0): none. Tests or test support files (2): test_block16_contracts.py, test_block19_integration.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.1.13 komorebi_math.provenance

Purpose and source. Mathematical provenance registry for public objects. It is an ordinary module in the package root family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/provenance.py contains 78 lines and 2,216 bytes. Source SHA-256 begins 64c95e2cf499a0c9.

Surface and ownership. API classification: API-manifested implementation owner. It owns 3 manifest cards: 0 stable, 0 beta, and 3 research. Linked cards: ClaimStatus, describe, ProvenanceRecord. Static analysis found 6 top-level public definitions: ClaimStatus, describe, documented, ProvenanceRecord, records, register_record.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (0): none. Reverse package dependents (100): komorebi_math, komorebi_math.algorithms.compiler, komorebi_math.algorithms.complexity, komorebi_math.algorithms.discovery, komorebi_math.algorithms.ir, komorebi_math.algorithms.krylov, and 94 more in the machine-readable ledger. Examples (0): none. Tests or test support files (1): test_certificates_registry.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.1.14 komorebi_math.qualification

Purpose and source. Release qualification and stable semantic regression for Komorebi Math. It is an ordinary module in the package root family. The ledger classifies it as unmanifested implementation support. Source src/komorebi_math/qualification.py contains 431 lines and 16,046 bytes. Source SHA-256 begins bbc2a27a6c3f797f.

Surface and ownership. API classification: importable package module outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its __all__ declaration contains 10 names: gpu_diagnostics, QualificationCheck, QualificationVerificationReport, RELEASE_CANDIDATE_VERSION, RELEASE_QUALIFICATION_SCHEMA, RELEASE_QUALIFICATION_SCHEMA_VERSION, ReleaseQualificationReport, run_release_qualification, run_stable_semantic_verification, runtime_inventory.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (20): komorebi_math._version, komorebi_math.api, komorebi_math.api.v1, komorebi_math.arithmetic, komorebi_math.certificates, komorebi_math.compatibility, and 14 more in the machine-readable ledger. Reverse package dependents (2): komorebi_math.cli.main, komorebi_math.verify.gpu. Examples (0): none. Tests or test support files (1): test_

block16_release_qualification.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.1.15 komorebi_math.registry

Purpose and source. Feature registry for implemented and planned mathematical domains. It is an ordinary module in the package root family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/registry.py contains 72 lines and 2,058 bytes. Source SHA-256 begins e4e93781427a808f.

Surface and ownership. API classification: API-manifested implementation owner. It owns 4 manifest cards: 0 stable, 0 beta, and 4 research. Linked cards: feature, features, FeatureSpec, FeatureStatus. Static analysis found 6 top-level public definitions: feature, FeatureRegistry, features, FeatureSpec, FeatureStatus, register_feature.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (0): none. Reverse package dependents (19): komorebi_math, komorebi_math._features, komorebi_math._planned, komorebi_math.algorithms, komorebi_math.analytic, komorebi_math.arithmetic, and 13 more in the machine-readable ledger. Examples (0): none. Tests or test support files (3): test_block13_cli_api.py, test_block14_cli_api.py, test_certificates_registry.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.1.16 komorebi_math.stable

Purpose and source. Curated 1.0 freeze-candidate API for Komorebi Math. It is an ordinary module in the package root family. The ledger classifies it as a canonical API namespace facade. Source src/komorebi_math/stable.py contains 298 lines and 8,473 bytes. Source SHA-256 begins 8fc11ed02b8971c9.

Surface and ownership. API classification: canonical public namespace without implementation ownership. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its __all__ declaration contains 131 names: __version__, AnalysisSynthesisBlocks, AnalysisSynthesisSystem, ApiCompatibilityReport, arithmetic_mixed_radix_address, arithmetic_mixed_radix_digits, arithmetic_polyphase_analyze, arithmetic_polyphase_synthesize, ArithmeticPolyphase, Arrow, and 121 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (30): komorebi_math._version, komorebi_math.arithmetic.polyphase, komorebi_math.certificates, komorebi_math.compatibility, komorebi_math.config, komorebi_math.coordinates, and 24 more in the machine-readable ledger. Reverse package dependents (7): komorebi_math.api.v1, komorebi_math.qualification, komorebi_math.release.readiness, komorebi_math.stable_semantics, komorebi_math.v1, komorebi_math.verify.release, and 1 more in the machine-readable ledger. Examples (2): block16_analysis_synthesis.py, block21_public_capability_tour.py. Tests or test support files (8): test_block15_polyphase_refinement.py, test_block15_stable_api.py, test_block16_analysis_synthesis.py, test_block16_companion_crosswalk.py, test_block16_contracts.py, and 3 more in the machine-readable ledger. Documentation files with literal module or public-route references (10): ANALYSIS_SYNTHESIS_SYSTEMS.md, API_COMPATIBILITY_0.15.md, API_INVENTORY.md, BLOCK_15_RELEASE.md, BLOCK_16_RELEASE.md, and 5 more in the machine-readable ledger. AST parsing and runtime import both passed in the recorded module audit.

D.2.1.17 komorebi_math.stable_contract

Purpose and source. Aggregate structural, semantic, and persistence compatibility for 1.0. It is an ordinary module in the package root family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/stable_contract.py contains 87 lines and 3,104 bytes. Source SHA-256 begins 80cf6fe3febe6803.

Surface and ownership. API classification: API-manifested implementation owner. It owns 2 manifest cards: 2 stable, 0 beta, and 0 research. Linked cards: check_stable_contract, StableContractReport. Its __all__ declaration contains 2 names: check_stable_contract, StableContractReport.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (4): komorebi_math.certificates, komorebi_math.compatibility, komorebi_math.persistence_contract, komorebi_math.stable_semantics. Reverse package dependents

(4): komorebi_math.qualification, komorebi_math.release.readiness, komorebi_math.stable, komorebi_math.verify.stabilization. Examples (0): none. Tests or test support files (1): test_block16_contracts.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.1.18 komorebi_math.stable_semantics

Purpose and source. Closed semantic regression fixture for the 1.0-candidate API. It is an ordinary module in the package root family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/stable_semantics.py contains 452 lines and 17,237 bytes. Source SHA-256 begins 739e76603fef9645.

Surface and ownership. API classification: API-manifested implementation owner. It owns 4 manifest cards: 4 stable, 0 beta, and 0 research. Linked cards: load_stable_semantics_fixture, run_stable_semantics_fixture, StableSemanticCaseResult, StableSemanticReport. Its __all__ declaration contains 7 names: DEFAULT_STABLE_SEMANTICS_FIXTURE, load_stable_semantics_fixture, run_stable_semantics_fixture, STABLE_SEMANTICS_SCHEMA, STABLE_SEMANTICS_SCHEMA_VERSION, StableSemanticCaseResult, StableSemanticReport.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (16): komorebi_math.arithmetic.polyphase, komorebi_math.certificates, komorebi_math.config, komorebi_math.coordinates.analysis_synthesis, komorebi_math.coordinates.basis, komorebi_math.core.grammar, and 10 more in the machine-readable ledger. Reverse package dependents (6): komorebi_math.release.contract, komorebi_math.release.upgrade, komorebi_math.stable, komorebi_math.stable_contract, komorebi_math.verify.release, komorebi_math.verify.stabilization. Examples (0): none. Tests or test support files (2): test_block16_contracts.py, test_block18_frame_algebra.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.1.19 komorebi_math.support

Purpose and source. Machine-readable support policy and runtime environment inspection. It is an ordinary module in the package root family. The ledger classifies it as unmanifested implementation support. Source src/komorebi_math/support.py contains 201 lines and 7,232 bytes. Source SHA-256 begins 34a2beb47434175b.

Surface and ownership. API classification: importable package module outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its __all__ declaration contains 7 names: DEFAULT_SUPPORT_MATRIX, load_support_matrix, runtime_environment, SUPPORT_MATRIX_SCHEMA, SUPPORT_MATRIX_SCHEMA_VERSION, SupportMatrix, ValidatedEnvironment.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (2): komorebi_math.exceptions, komorebi_math.nonlinear.autodiff. Reverse package dependents (4): komorebi_math.qualification, komorebi_math.release.contract, komorebi_math.release.readiness, komorebi_math.verify.release. Examples (0): none. Tests or test support files (1): test_block16_contracts.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.1.20 komorebi_math.typing

Purpose and source. Shared type aliases used by the public API. It is an ordinary module in the package root family. The ledger classifies it as unmanifested implementation support. Source src/komorebi_math/typing.py contains 16 lines and 507 bytes. Source SHA-256 begins 70a5dcaba944a26d.

Surface and ownership. API classification: importable package module outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Static analysis found no top-level public definitions.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (0): none. Reverse package dependents (0): none. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.1.21 `komorebi_math.unified`

Purpose and source. The common executable language used across Komorebi Math domains. It is an ordinary module in the `package root` family. The ledger classifies it as a public implementation owner. Source `src/komorebi_math/unified.py` contains 199 lines and 9,187 bytes. Source SHA-256 begins `025c94a00ccd651a`.

Surface and ownership. API classification: API-manifested implementation owner. It owns 3 manifest cards: 0 stable, 0 beta, and 3 research. Linked cards: StageSpec, UnifiedStage, UnifiedSystem. Static analysis found 3 top-level public definitions: `StageSpec`, `UnifiedStage`, `UnifiedSystem`.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (9): `komorebi_math.certificates`, `komorebi_math.core.grammar`, `komorebi_math.core.operators`, `komorebi_math.core.packets`, `komorebi_math.core.realization`, `komorebi_math.core.seeds`, and 3 more in the machine-readable ledger. Reverse package dependents (1): `komorebi_math`. Examples (1): `quickstart.py`. Tests or test support files (1): `test_unified_extensions_cli.py`. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.1.22 `komorebi_math.v1`

Purpose and source. Convenience alias for the curated version 1 API. It is an ordinary module in the `package root` family. The ledger classifies it as a canonical API namespace facade. Source `src/komorebi_math/v1.py` contains 6 lines and 143 bytes. Source SHA-256 begins `c2d39f24ed6249f6`.

Surface and ownership. API classification: canonical public namespace without implementation ownership. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its `__all__` declaration contains 131 names: `__version__`, `AnalysisSynthesisBlocks`, `AnalysisSynthesisSystem`, `ApiCompatibilityReport`, `arithmetic_mixed_radix_address`, `arithmetic_mixed_radix_digits`, `arithmetic_polyphase_analyze`, `arithmetic_polyphase_synthesize`, `ArithmeticPolyphase`, `Arrow`, and 121 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (1): `komorebi_math.stable`. Reverse package dependents (3): `komorebi_math.qualification`, `komorebi_math.release.readiness`, `komorebi_math.verify.stabilization`. Examples (0): none. Tests or test support files (5): `test_block15_polyphase_refinement.py`, `test_block15_stable_api.py`, `test_block16_analysis_synthesis.py`, `test_block16_release_contract.py`, `test_block18_frame_algebra.py`. Documentation files with literal module or public-route references (8): `API_COMPATIBILITY_0.15.md`, `API_INVENTORY.md`, `BLOCK_15_RELEASE.md`, `BLOCK_16_RELEASE.md`, `IMPLEMENTATION_ROADMAP.md`, and 3 more in the machine-readable ledger. AST parsing and runtime import both passed in the recorded module audit.

D.2.2 Api: 2 modules

D.2.2.1 `komorebi_math.api`

Purpose and source. Versioned public API namespaces. It is the package initializer for the `api` family. The ledger classifies it as a canonical API namespace facade. Source `src/komorebi_math/api/__init__.py` contains 5 lines and 75 bytes. Source SHA-256 begins `844fd5e337598266`.

Surface and ownership. API classification: canonical public namespace without implementation ownership. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its `__all__` declaration contains 1 name: `v1`.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (1): `komorebi_math.api.v1`. Reverse package dependents (3): `komorebi_math.qualification`, `komorebi_math.release.readiness`, `komorebi_math.verify.stabilization`. Examples (0): none. Tests or test support files (5): `test_block15_polyphase_refinement.py`, `test_block15_stable_api.py`, `test_block16_analysis_synthesis.py`, `test_block16_release_contract.py`, `test_block18_frame_algebra.py`. Documentation files with literal module or public-route references (9): `API_COMPATIBILITY_0.15.md`, `API_INVENTORY.md`, `BLOCK_15_RELEASE.md`, `BLOCK_16_RELEASE.md`, `IMPLEMENTATION_ROADMAP.md`, and 4 more in the machine-readable ledger. AST parsing and runtime import both passed in the recorded module audit.

D.2.2.2 komorebi_math.api.v1

Purpose and source. Version 1 compatibility alias for komorebi_math.stable. It is an ordinary module in the api family. The ledger classifies it as a canonical API namespace facade. Source src/komorebi_math/api/v1.py contains 6 lines and 159 bytes. Source SHA-256 begins 053a732923c0c611.

Surface and ownership. API classification: canonical public namespace without implementation ownership. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its __all__ declaration contains 131 names: __version__, AnalysisSynthesisBlocks, AnalysisSynthesisSystem, ApiCompatibilityReport, arithmetic_mixed_radix_address, arithmetic_mixed_radix_digits, arithmetic_polyphase_analyze, arithmetic_polyphase_synthesize, ArithmeticPolyphase, Arrow, and 121 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (1): komorebi_math.stable. Reverse package dependents (4): komorebi_math.api, komorebi_math.qualification, komorebi_math.release.readiness, komorebi_math.verify.stabilization. Examples (0): none. Tests or test support files (5): test_block15_polyphase_refinement.py, test_block15_stable_api.py, test_block16_analysis_synthesis.py, test_block16_release_contract.py, test_block18_frame_algebra.py. Documentation files with literal module or public-route references (8): API_COMPATIBILITY_0.15.md, API_INVENTORY.md, BLOCK_15_RELEASE.md, BLOCK_16_RELEASE.md, IMPLEMENTATION_ROADMAP.md, and 3 more in the machine-readable ledger. AST parsing and runtime import both passed in the recorded module audit.

D.2.3 Beta: 1 module

D.2.3.1 komorebi_math.beta

Purpose and source. Curated beta-compatibility surface for finite packet execution and persistence. It is the package initializer for the beta family. The ledger classifies it as a canonical API namespace facade. Source src/komorebi_math/beta/__init__.py contains 197 lines and 5,520 bytes. Source SHA-256 begins b56d0c7448fdcdfb.

Surface and ownership. API classification: canonical public namespace without implementation ownership. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its __all__ declaration contains 83 names: __version__, ApiCompatibilityReport, AssumptionError, BackendUnavailableError, BEHAVIORAL_GOLDEN_SCHEMA, BEHAVIORAL_GOLDEN_SCHEMA_VERSION, BehavioralGoldenCaseResult, BehavioralGoldenReport, BETA_API_STABILITY_LEVEL, CallableDerivativeProvider, and 73 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (14): komorebi_math._version, komorebi_math.certificates, komorebi_math.compatibility, komorebi_math.core, komorebi_math.exceptions, komorebi_math.export.persistence, and 8 more in the machine-readable ledger. Reverse package dependents (0): none. Examples (0): none. Tests or test support files (3): test_block11_golden_profiling_beta.py, test_block12_cli_api.py, test_block19_integration.py. Documentation files with literal module or public-route references (10): API_COMPATIBILITY_0.11.md, API_COMPATIBILITY_0.12.md, API_COMPATIBILITY_0.13.md, API_COMPATIBILITY_0.14.md, API_INVENTORY.md, and 5 more in the machine-readable ledger. AST parsing and runtime import both passed in the recorded module audit.

D.2.4 Backends: 4 modules

D.2.4.1 `komorebi_math.backends`

Purpose and source. Backend discovery and selection. It is the package initializer for the `backends` family. The ledger classifies it as a backend adapter module. Source `src/komorebi_math/backends/__init__.py` contains 63 lines and 1,857 bytes. Source SHA-256 begins `b5a016ff2b71593a`.

Surface and ownership. API classification: backend implementation outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its `__all__` declaration contains 6 names: `available_backends`, `backend_name`, `get_backend`, `LinearAlgebraBackend`, `NumpyBackend`, `register_backend`.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (4): `komorebi_math.backends.base`, `komorebi_math.backends.numpy`, `komorebi_math.backends.sympy`, `komorebi_math.exceptions`. Reverse package dependents (11): `komorebi_math.cli.main`, `komorebi_math.core.algebra`, `komorebi_math.core.morphisms`, `komorebi_math.core.packets`, `komorebi_math.core.realization`, `komorebi_math.core.seeds`, and 5 more in the machine-readable ledger. Examples (0): none. Tests or test support files (2): `test_backends.py`, `test_morphisms.py`. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.4.2 `komorebi_math.backends.base`

Purpose and source. Linear algebra backend interface used by the finite-dimensional core. It is an ordinary module in the `backends` family. The ledger classifies it as a backend adapter module. Source `src/komorebi_math/backends/base.py` contains 68 lines and 1,696 bytes. Source SHA-256 begins `08139171a8fa5660`.

Surface and ownership. API classification: backend implementation outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Static analysis found 1 top-level public definition: `LinearAlgebraBackend`.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (0): none. Reverse package dependents (3): `komorebi_math.backends`, `komorebi_math.backends.numpy`, `komorebi_math.backends.sympy`. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.4.3 `komorebi_math.backends.numpy`

Purpose and source. NumPy backend. It is an ordinary module in the `backends` family. The ledger classifies it as a backend adapter module. Source `src/komorebi_math/backends/numpy.py` contains 96 lines and 3,410 bytes. Source SHA-256 begins `ff1acb35928ecc8b`.

Surface and ownership. API classification: backend implementation outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Static analysis found 1 top-level public definition: `NumpyBackend`.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (2): `komorebi_math.backends.base`, `komorebi_math.exceptions`. Reverse package dependents (1): `komorebi_math.backends`. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.4.4 `komorebi_math.backends.sympy`

Purpose and source. Optional exact SymPy backend. It is an ordinary module in the `backends` family. The ledger classifies it as a backend adapter module. Source `src/komorebi_math/backends/sympy.py` contains 104 lines and 3,003 bytes. Source SHA-256 begins `8f5082c8e05db32e`.

Surface and ownership. API classification: backend implementation outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Static analysis found 1 top-level public definition: `SympyBackend`.

Dependencies and evidence. Direct dependency profile: optional `exact` extra. Direct package dependencies (2): `komorebi_math.backends.base`, `komorebi_math.exceptions`. Reverse package dependents (1): `komorebi_math.backends`. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.5 Core: 10 modules

D.2.5.1 `komorebi_math.core`

Purpose and source. Finite-dimensional executable foundation. It is the package initializer for the `core` family. The ledger classifies it as an importable package facade outside API manifests. Source `src/komorebi_math/core/__init__.py` contains 53 lines and 1,545 bytes. Source SHA-256 begins `cc9b9d613fed58b7`.

Surface and ownership. API classification: importable package module outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its `__all__` declaration contains 29 names: `AlgebraBasisResult`, `Arrow`, `block_from_paths`, `cyclic_span`, `FreePresentation`, `independent_direct_sum_seed_rank`, `matrix_units`, `OperatorAlgebra`, `OperatorGrammar`, `OperatorMorphism`, and 19 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (9): `komorebi_math.core.algebra`, `komorebi_math.core.grammar`, `komorebi_math.core.graphs`, `komorebi_math.core.morphisms`, `komorebi_math.core.operators`, `komorebi_math.core.packets`, and 3 more in the machine-readable ledger. Reverse package dependents (17): `komorebi_math`, `komorebi_math.beta`, `komorebi_math.dynamics.semigroups`, `komorebi_math.golden`, `komorebi_math.qualification`, `komorebi_math.quantum.channels`, and 11 more in the machine-readable ledger. Examples (0): none. Tests or test support files (2): `test_quantum_symmetry_circuits.py`, `test_unified_extensions_cli.py`. Documentation files with literal module or public-route references (1): `API_INVENTORY.md`. AST parsing and runtime import both passed in the recorded module audit.

D.2.5.2 `komorebi_math.core.algebra`

Purpose and source. Finite matrix algebras generated by operator families. It is an ordinary module in the `core` family. The ledger classifies it as a public implementation owner. Source `src/komorebi_math/core/algebra.py` contains 180 lines and 7,007 bytes. Source SHA-256 begins `e62691c9a68bd448`.

Surface and ownership. API classification: API-manifested implementation owner. It owns 3 manifest cards: 0 stable, 0 beta, and 3 research. Linked cards: AlgebraBasisResult, matrix_units, OperatorAlgebra. Static analysis found 3 top-level public definitions: `AlgebraBasisResult`, `matrix_units`, `OperatorAlgebra`.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (9): `komorebi_math.backends`, `komorebi_math.certificates`, `komorebi_math.config`, `komorebi_math.core.graphs`, `komorebi_math.core.operators`, `komorebi_math.core.packets`, and 3 more in the machine-readable ledger. Reverse package dependents (1): `komorebi_math.core`. Examples (1): `exact_backend.py`. Tests or test support files (1): `test_algebra_quotients.py`. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.5.3 `komorebi_math.core.grammar`

Purpose and source. Typed operator grammars and formal path relations. It is an ordinary module in the `core` family. The ledger classifies it as a public implementation owner. Source `src/komorebi_math/core/grammar.py` contains 231 lines and 8,368 bytes. Source SHA-256 begins `003a8bb22f6dfeac`.

Surface and ownership. API classification: API-manifested implementation owner. It owns 8 manifest cards: 4 stable, 0 beta, and 4 research. Linked cards: Arrow, OperatorGrammar, OperatorRelation, OperatorWord, Arrow, OperatorGrammar, OperatorRelation, OperatorWord. Static analysis found 4 top-level public definitions: `Arrow`, `OperatorGrammar`, `OperatorRelation`, `OperatorWord`.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (2): `komorebi_math.exceptions`, `komorebi_math.provenance`. Reverse package dependents (4): `komorebi_math.core`, `komorebi_math.core.realization`, `komorebi_math.stable_semantic`

s, komorebi_math.unified. Examples (1): typed_grammar.py. Tests or test support files (3): test_block16_analysis_synthesis.py, test_grammar_realization.py, test_unified_extensions_cli.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.5.4 komorebi_math.core.graphs

Purpose and source. Transfer graphs, strongly connected components, and support preorders. It is an ordinary module in the core family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/core/graphs.py contains 282 lines and 11,961 bytes. Source SHA-256 begins b565ad292c4dc647.

Surface and ownership. API classification: API-manifested implementation owner. It owns 3 manifest cards: 1 stable, 0 beta, and 2 research. Linked cards: TransferGraph, SupportPreorder, TransferGraph. Static analysis found 2 top-level public definitions: SupportPreorder, TransferGraph.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (3): komorebi_math.certificates, komorebi_math.exceptions, komorebi_math.provenance. Reverse package dependents (3): komorebi_math.core, komorebi_math.core.algebra, komorebi_math.core.transfer. Examples (0): none. Tests or test support files (1): test_graphs.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.5.5 komorebi_math.core.morphisms

Purpose and source. Intertwining and packet-compatible morphisms. It is an ordinary module in the core family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/core/morphisms.py contains 181 lines and 8,067 bytes. Source SHA-256 begins 40280cd87fd19f9d.

Surface and ownership. API classification: API-manifested implementation owner. It owns 3 manifest cards: 0 stable, 0 beta, and 3 research. Linked cards: OperatorMorphism, PacketMorphism, SeedMorphism. Static analysis found 3 top-level public definitions: OperatorMorphism, PacketMorphism, SeedMorphism.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (6): komorebi_math.backends, komorebi_math.certificates, komorebi_math.core.packets, komorebi_math.core.seeds, komorebi_math.exceptions, komorebi_math.provenance. Reverse package dependents (1): komorebi_math.core. Examples (0): none. Tests or test support files (1): test_morphisms.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.5.6 komorebi_math.core.operators

Purpose and source. Packet-resolved linear operators. It is an ordinary module in the core family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/core/operators.py contains 192 lines and 8,652 bytes. Source SHA-256 begins 5d9a3794e8fc56f6.

Surface and ownership. API classification: API-manifested implementation owner. It owns 2 manifest cards: 1 stable, 0 beta, and 1 research. Linked cards: PacketOperator, PacketOperator. Static analysis found 1 top-level public definition: PacketOperator.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (4): komorebi_math.certificates, komorebi_math.core.packets, komorebi_math.exceptions, komorebi_math.provenance. Reverse package dependents (5): komorebi_math.core, komorebi_math.core.algebra, komorebi_math.core.transfer, komorebi_math.refinement.towers, komorebi_math.unified. Examples (1): quickstart.py. Tests or test support files (1): test_operators_transfer.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.5.7 komorebi_math.core.packets

Purpose and source. Exact finite packet systems and packet coordinates. It is an ordinary module in the core family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/core/packets.py contains 355 lines and 16,054 bytes. Source SHA-256 begins 02427ff27728818c.

Surface and ownership. API classification: API-manifested implementation owner. It owns 5 manifest cards: 2 stable, 1 beta, and 2 research. Linked cards: PacketCoordinates, PacketSystem, PacketSystem, PacketCoordinates, PacketSystem. Static analysis found 2 top-level public definitions: PacketCoordinates, PacketSystem.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (5): komorebi_math.backends, komorebi_math.certificates, komorebi_math.config, komorebi_math.exceptions, komorebi_math.provenance. Reverse package dependents (13): komorebi_math.core, komorebi_math.core.algebra, komorebi_math.core.morphisms, komorebi_math.core.operators, komorebi_math.nonlinear.composition, komorebi_math.nonlinear.coordinates, and 7 more in the machine-readable ledger. Examples (9): block11_semantic_profile.py, exact_backend.py, nonlinear_jets_composition.py, portable_nonlinear_state.py, quickstart.py, and 4 more in the machine-readable ledger. Tests or test support files (19): test_algebra_quotients.py, test_block10_execution_persistence.py, test_block11_cli.py, test_block11_golden_profiling_beta.py, test_block11_streaming_migrations.py, and 14 more in the machine-readable ledger. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.5.8 komorebi_math.core.realization

Purpose and source. Matrix realizations of typed operator grammars. It is an ordinary module in the core family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/core/realization.py contains 147 lines and 6,210 bytes. Source SHA-256 begins 9992284c22390fbc.

Surface and ownership. API classification: API-manifested implementation owner. It owns 3 manifest cards: 2 stable, 0 beta, and 1 research. Linked cards: OperatorRealization, RelationResidual, OperatorRealization. Static analysis found 2 top-level public definitions: OperatorRealization, RelationResidual.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (6): komorebi_math.backends, komorebi_math.certificates, komorebi_math.config, komorebi_math.core.grammar, komorebi_math.exceptions, komorebi_math.provenance. Reverse package dependents (3): komorebi_math.core, komorebi_math.stable_semantics, komorebi_math.unified. Examples (1): typed_grammar.py. Tests or test support files (2): test_grammar_realization.py, test_unified_extensions_cli.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.5.9 komorebi_math.core.seeds

Purpose and source. Finite-dimensional seed systems and generator calculations. It is an ordinary module in the core family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/core/seeds.py contains 385 lines and 15,228 bytes. Source SHA-256 begins 1962a589cce8d354.

Surface and ownership. API classification: API-manifested implementation owner. It owns 8 manifest cards: 1 stable, 0 beta, and 7 research. Linked cards: SeedSystem, cyclic_span, independent_direct_sum_seed_rank, SeedSystem, semisimple_seed_rank, shared_direct_sum_seed_bounds, single_operator_seed_rank, tensor_seed_rank_upper_bound. Static analysis found 9 top-level public definitions: cyclic_span, FreePresentation, independent_direct_sum_seed_rank, SeedSystem, semisimple_seed_rank, shared_direct_sum_seed_bounds, single_operator_seed_rank, StandardSeedSearchResult, tensor_seed_rank_upper_bound.

Dependencies and evidence. Direct dependency profile: optional exact extra. Direct package dependencies (5): komorebi_math.backends, komorebi_math.certificates, komorebi_math.config, komorebi_math.exceptions, komorebi_math.provenance. Reverse package dependents (4): komorebi_math.core, komorebi_math.core.algebra, komorebi_math.core.morphisms, komorebi_math.unified. Examples (2): exact_backend.py, quickstart.py. Tests or test support files (4): test_algebra_quotients.py, test_morphisms.py, test_seeds.py, test_unified_extensions_cli.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.5.10 komorebi_math.core.transfer

Purpose and source. Path expansions for packet transfer blocks. It is an ordinary module in the core family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/core/transfer.py contains 86 lines and 3,075 bytes. Source SHA-256 begins 87bf51f7a29f7918.

Surface and ownership. API classification: API-manifested implementation owner. It owns 5 manifest cards: 3 stable, 0 beta, and 2 research. Linked cards: block_from_paths, path_contributions, TransferPath, block_from_paths, path_contributions. Static analysis found 3 top-level public definitions: block_from_paths, path_contributions, TransferPath.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (4): komorebi_math.certificates, komorebi_math.core.graphs, komorebi_math.core.operators, komorebi_math.exceptions. Reverse package dependents (1): komorebi_math.core. Examples (0): none. Tests or test support files (1): test_operators_transfer.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.6 Algorithms: 15 modules

D.2.6.1 komorebi_math.algorithms

Purpose and source. Structured operators, packet compilation, precision planning, and scheduling. It is the package initializer for the algorithms family. The ledger classifies it as an importable package facade outside API manifests. Source src/komorebi_math/algorithms/__init__.py contains 240 lines and 5,734 bytes. Source SHA-256 begins 914568810861185f.

Surface and ownership. API classification: importable package module outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its __all__ declaration contains 97 names: accumulated_recompression_bound, Address, amortization_threshold, arithmetic_intensity, as_structured_operator, assign_discrete_precisions, batched_launch_count, BDLRDiscoveryResult, benchmark_callable, BenchmarkComparison, and 87 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (14): komorebi_math.algorithms.benchmarks, komorebi_math.algorithms.compiler, komorebi_math.algorithms.complexity, komorebi_math.algorithms.discovery, komorebi_math.algorithms.gpu, komorebi_math.algorithms.ir, and 8 more in the machine-readable ledger. Reverse package dependents (3): komorebi_math, komorebi_math.release._gpu_worker, komorebi_math.verify.algorithms. Examples (5): low_rank_recompression.py, packet_compiler.py, packet_ir.py, precision_schedule.py, structured_krylov.py. Tests or test support files (10): test_algorithms_compiler.py, test_algorithms_complexity_benchmarks.py, test_algorithms_discovery_gpu.py, test_algorithms_krylov.py, test_algorithms_layout_ir.py, and 5 more in the machine-readable ledger. Documentation files with literal module or public-route references (2): API_INVENTORY.md, PUBLIC_API.md. AST parsing and runtime import both passed in the recorded module audit.

D.2.6.2 komorebi_math.algorithms._validation

Purpose and source. Internal validation helpers for structured algorithm APIs. It is an ordinary module in the algorithms family. The ledger classifies it as an internal support module. Source src/komorebi_math/algorithms/_validation.py contains 59 lines and 1,768 bytes. Source SHA-256 begins 71adc32e6fdfcbbc.

Surface and ownership. API classification: internal by module name. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its __all__ declaration contains 0 names: none.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (1): komorebi_math.exceptions. Reverse package dependents (10): komorebi_math.algorithms.benchmarks, komorebi_math.algorithms.compiler, komorebi_math.algorithms.complexity, komorebi_math.algorithms.ir, komorebi_math.algorithms.krylov, komorebi_math.algorithms.low_rank, and 4 more in the machine-readable ledger. Examples (0):

none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.6.3 `komorebi_math.algorithms.benchmarks`

Purpose and source. Matched benchmark contracts and lightweight timing helpers. It is an ordinary module in the `algorithms` family. The ledger classifies it as unmanifested implementation support. Source `src/komorebi_math/algorithms/benchmarks.py` contains 147 lines and 4,524 bytes. Source SHA-256 begins `9f1bbd4dd9644ff5`.

Surface and ownership. API classification: importable package module outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its `__all__` declaration contains 6 names: `benchmark_callable`, `BenchmarkComparison`, `BenchmarkContract`, `BenchmarkResult`, `compare_benchmarks`, `TimingScope`.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (2): `komorebi_math.algorithms._validation`, `komorebi_math.exceptions`. Reverse package dependents (2): `komorebi_math.algorithms`, `komorebi_math.nonlinear.execution`. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.6.4 `komorebi_math.algorithms.compiler`

Purpose and source. Exact finite compilers for additive packet representation trees. It is an ordinary module in the `algorithms` family. The ledger classifies it as a public implementation owner. Source `src/komorebi_math/algorithms/compiler.py` contains 413 lines and 15,314 bytes. Source SHA-256 begins `2bf95f61c84f3fd3`.

Surface and ownership. API classification: API-manifested implementation owner. It owns 3 manifest cards: 0 stable, 0 beta, and 3 research. Linked cards: compile_tree_budget, compile_tree_lagrangian, TreeNode. Its `__all__` declaration contains 9 names: `combine_tree_objectives`, `compile_tree_budget`, `compile_tree_lagrangian`, `decision_margins`, `DecisionMargin`, `pareto_frontier`, `replace_tree_nodes`, `TreeNode`, `TreePlan`.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (4): `komorebi_math.algorithms._validation`, `komorebi_math.certificates`, `komorebi_math.exceptions`, `komorebi_math.provenance`. Reverse package dependents (1): `komorebi_math.algorithms`. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.6.5 `komorebi_math.algorithms.complexity`

Purpose and source. Declared arithmetic, storage, data movement, and roofline cost models. It is an ordinary module in the `algorithms` family. The ledger classifies it as unmanifested implementation support. Source `src/komorebi_math/algorithms/complexity.py` contains 229 lines and 7,965 bytes. Source SHA-256 begins `2728e130cc780dd3`.

Surface and ownership. API classification: importable package module outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its `__all__` declaration contains 11 names: `Address`, `arithmetic_intensity`, `block_diagonal_low_rank_flops`, `block_diagonal_low_rank_storage_entries`, `CostEstimate`, `dense_apply_flops`, `packet_apply_flops`, `packet_cost_estimate`, `packet_dense_storage_entries`, `packet_streaming_bytes`, and 1 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (4): `komorebi_math.algorithms._validation`, `komorebi_math.algorithms.layout`, `komorebi_math.exceptions`, `komorebi_math.provenance`. Reverse package dependents (1): `komorebi_math.algorithms`. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.6.6 komorebi_math.algorithms.discovery

Purpose and source. Restricted discovery of packet support and block-diagonal plus low-rank structure. It is an ordinary module in the algorithms family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/algorithms/discovery.py contains 171 lines and 6,929 bytes. Source SHA-256 begins 2d80ca238d92a8b5.

Surface and ownership. API classification: API-manifested implementation owner. It owns 2 manifest cards: 0 stable, 0 beta, and 2 research. Linked cards: discover_block_diagonal_low_rank, discover_packet_support. Its __all__ declaration contains 4 names: BDLRDiscoveryResult, discover_block_diagonal_low_rank, discover_packet_support, SupportDiscoveryResult.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (7): komorebi_math.algorithms.ir, komorebi_math.algorithms.layout, komorebi_math.algorithms.low_rank, komorebi_math.algorithms.structured, komorebi_math.certificates, komorebi_math.exceptions, and 1 more in the machine-readable ledger. Reverse package dependents (1): komorebi_math.algorithms. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.6.7 komorebi_math.algorithms.gpu

Purpose and source. Optional device executors for packet intermediate representations. It is an ordinary module in the algorithms family. The ledger classifies it as unmanifested implementation support. Source src/komorebi_math/algorithms/gpu.py contains 134 lines and 4,498 bytes. Source SHA-256 begins 83aa2b3d85369e02.

Surface and ownership. API classification: importable package module outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its __all__ declaration contains 5 names: cupy_available, CupyPacketExecutor, DeviceExecutionResult, NumpyPacketExecutor, packet_executor.

Dependencies and evidence. Direct dependency profile: optional CuPy accelerator outside the declared project extras. Direct package dependencies (3): komorebi_math.algorithms.ir, komorebi_math.certificates, komorebi_math.exceptions. Reverse package dependents (1): komorebi_math.algorithms. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.6.8 komorebi_math.algorithms.ir

Purpose and source. Packet operator intermediate representation. It is an ordinary module in the algorithms family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/algorithms/ir.py contains 276 lines and 11,889 bytes. Source SHA-256 begins 437053d4666cfb7f.

Surface and ownership. API classification: API-manifested implementation owner. It owns 1 manifest card: 0 stable, 0 beta, and 1 research. Linked cards: PacketIR. Its __all__ declaration contains 2 names: Address, PacketIR.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (6): komorebi_math.algorithms._validation, komorebi_math.algorithms.layout, komorebi_math.algorithms.structured, komorebi_math.certificates, komorebi_math.exceptions, komorebi_math.provenance. Reverse package dependents (3): komorebi_math.algorithms, komorebi_math.algorithms.discovery, komorebi_math.algorithms.gpu. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.6.9 komorebi_math.algorithms.krylov

Purpose and source. Krylov methods and packet block preconditioning. It is an ordinary module in the algorithms family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/algorithms/krylov.py contains 517 lines and 19,903 bytes. Source SHA-256 begins d6c5ae7cdd4ac9be.

Surface and ownership. API classification: API-manifested implementation owner. It owns 1 manifest card: 0 stable, 0 beta, and 1 research. Linked cards: conjugate_gradient. Its __all__ declaration contains 11 names: block_jacobi_certificate, block_jacobi_preconditioner, BlockJacobiCertificate, conjugate_gradient, ConjugateGradientResult, krylov_basis, KrylovBasisResult, lanczos_tridiagonal, LanczosResult, power_perturbation_bound, and 1 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (6): `komorebi_math.algorithms._validation`, `komorebi_math.algorithms.layout`, `komorebi_math.algorithms.structured`, `komorebi_math.certificates`, `komorebi_math.exceptions`, `komorebi_math.provenance`. Reverse package dependents (1): `komorebi_math.algorithms`. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.6.10 `komorebi_math.algorithms.layout`

Purpose and source. Packet layouts for structured numerical operators. It is an ordinary module in the `algorithms` family. The ledger classifies it as a public implementation owner. Source `src/komorebi_math/algorithms/layout.py` contains 195 lines and 8,058 bytes. Source SHA-256 begins `455a8d3718c57d06`.

Surface and ownership. API classification: API-manifested implementation owner. It owns 1 manifest card: 0 stable, 0 beta, and 1 research. Linked cards: PacketLayout. Its `__all__` declaration contains 1 name: `PacketLayout`.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (2): `komorebi_math.exceptions`, `komorebi_math.provenance`. Reverse package dependents (6): `komorebi_math.algorithms`, `komorebi_math.algorithms.complexity`, `komorebi_math.algorithms.discovery`, `komorebi_math.algorithms.ir`, `komorebi_math.algorithms.krylov`, `komorebi_math.algorithms.structured`. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.6.11 `komorebi_math.algorithms.low_rank`

Purpose and source. Low-rank algebra, Woodbury solves, and certified recompression. It is an ordinary module in the `algorithms` family. The ledger classifies it as a public implementation owner. Source `src/komorebi_math/algorithms/low_rank.py` contains 409 lines and 15,582 bytes. Source SHA-256 begins `63f9c6af6c821014`.

Surface and ownership. API classification: API-manifested implementation owner. It owns 3 manifest cards: 0 stable, 0 beta, and 3 research. Linked cards: BlockDiagonalLowRankOperator, LowRankOperator, recompress_factors. Its `__all__` declaration contains 7 names: `accumulated_recompression_bound`, `BlockDiagonalLowRankOperator`, `LowRankOperator`, `product_chain_error_bound`, `recompress_core`, `recompress_factors`, `RecompressionResult`.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (5): `komorebi_math.algorithms._validation`, `komorebi_math.algorithms.structured`, `komorebi_math.certificates`, `komorebi_math.exceptions`, `komorebi_math.provenance`. Reverse package dependents (2): `komorebi_math.algorithms`, `komorebi_math.algorithms.discovery`. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.6.12 `komorebi_math.algorithms.precision`

Purpose and source. Packetwise error certificates and precision allocation. It is an ordinary module in the `algorithms` family. The ledger classifies it as a public implementation owner. Source `src/komorebi_math/algorithms/precision.py` contains 291 lines and 10,775 bytes. Source SHA-256 begins `424ffa52c460863e`.

Surface and ownership. API classification: API-manifested implementation owner. It owns 2 manifest cards: 0 stable, 0 beta, and 2 research. Linked cards: assign_discrete_precisions, DTypeChoice. Its `__all__` declaration contains 10 names: `assign_discrete_precisions`, `clipped_precision_allocation`, `DTypeChoice`, `optimal_precision_tolerances`, `packet_error_bound`, `precision_certificate`, `PrecisionAssignment`, `quantization_error`, `quantize_array`, `quantize_by_packet`.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (4): `komorebi_math.algorithms._validation`, `komorebi_math.certificates`, `komorebi_math.exceptions`, `komorebi_math.provenance`. Reverse package dependents (1): `komorebi_math.algorithms`. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.6.13 `komorebi_math.algorithms.scheduling`

Purpose and source. Finite task graph scheduling and amortization models. It is an ordinary module in the `algorithms` family. The ledger classifies it as a public implementation owner. Source `src/komorebi_math/algorithms/scheduling.py` contains 243 lines and 8,289 bytes. Source SHA-256 begins `4e82dc70675b6a01`.

Surface and ownership. API classification: API-manifested implementation owner. It owns 2 manifest cards: 0 stable, 0 beta, and 2 research. Linked cards: schedule_unlimited, Task. Its `__all__` declaration contains 10 names: `amortization_threshold`, `batched_launch_count`, `break_even_analysis`, `BreakEvenResult`, `schedule_unlimited`, `ScheduledTask`, `shape_classes`, `Task`, `TaskSchedule`, `verify_schedule`.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (3): `komorebi_math.algorithms._validation`, `komorebi_math.exceptions`, `komorebi_math.provenance`. Reverse package dependents (1): `komorebi_math.algorithms`. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.6.14 `komorebi_math.algorithms.structured`

Purpose and source. Structured finite linear operators used by the packet compiler. It is an ordinary module in the `algorithms` family. The ledger classifies it as a public implementation owner. Source `src/komorebi_math/algorithms/structured.py` contains 556 lines and 19,395 bytes. Source SHA-256 begins `a6a8edebd8429fed`.

Surface and ownership. API classification: API-manifested implementation owner. It owns 4 manifest cards: 0 stable, 0 beta, and 4 research. Linked cards: BlockDiagonalOperator, CirculantOperator, DenseOperator, Walsh-HadamardOperator. Its `__all__` declaration contains 10 names: `as_structured_operator`, `BlockDiagonalOperator`, `CirculantOperator`, `DenseOperator`, `DiagonalOperator`, `IdentityOperator`, `KroneckerOperator`, `StructuredOperator`, `ToeplitzOperator`, `WalshHadamardOperator`.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (4): `komorebi_math.algorithms._validation`, `komorebi_math.algorithms.layout`, `komorebi_math.exceptions`, `komorebi_math.provenance`. Reverse package dependents (5): `komorebi_math.algorithms`, `komorebi_math.algorithms.discovery`, `komorebi_math.algorithms.ir`, `komorebi_math.algorithms.krylov`, `komorebi_math.algorithms.low_rank`. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.6.15 `komorebi_math.algorithms.support`

Purpose and source. Structural support algebra for packet operators. It is an ordinary module in the `algorithms` family. The ledger classifies it as unmanifested implementation support. Source `src/komorebi_math/algorithms/support.py` contains 327 lines and 12,315 bytes. Source SHA-256 begins `349fe33f7f780527`.

Surface and ownership. API classification: importable package module outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its `__all__` declaration contains 10 names: `Address`, `inverse_support_if_acyclic`, `longest_path_vertices`, `schur_fill_support`, `support_polynomial`, `support_power`, `support_product`, `SupportGraph`, `topological_levels`, `transitive_closure`.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (3): `komorebi_math.algorithms._validation`, `komorebi_math.exceptions`, `komorebi_math.provenance`. Reverse package dependents (1): `komorebi_math.algorithms`. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.7 Arithmetic: 10 modules

D.2.7.1 `komorebi_math.arithmetic`

Purpose and source. Exact local arithmetic packets, Euler coordinates, Mahler transfer, and recurrences. It is the package initializer for the `arithmetic` family. The ledger classifies it as an importable package facade outside API manifests. Source `src/komorebi_math/arithmetic/__init__.py` contains 189 lines and 4,614 bytes. Source SHA-256 begins `d2e6e0a6edba21d0`.

Surface and ownership. API classification: importable package module outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its `__all__` declaration contains 78 names: `affine_kernel_composition_certificate`, `affine_kernel_matrix`, `affine_mahler_kernel`, `ArithmeticPolyphase`, `bernoulli_numbers`, `binomial_valuation`, `binomial_valuation_from_digits`, `carmichael_exponent_prime_power`, `certify_difference_congruence`, `compressed_hurwitz_inverse`, and 68 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (11): `komorebi_math._planned`, `komorebi_math.arithmetic.domains`, `komorebi_math.arithmetic.euler`, `komorebi_math.arithmetic.hurwitz`, `komorebi_math.arithmetic.local_modes`, `komorebi_math.arithmetic.mahler`, and 5 more in the machine-readable ledger. Reverse package dependents (5): `komorebi_math`, `komorebi_math.cli.main`, `komorebi_math.qualification`, `komorebi_math.verify.arithmetic`, `komorebi_math.verify.stabilization`. Examples (3): `local_euler_coordinates.py`, `recurrence_packets.py`, `standard_window_mahler.py`. Tests or test support files (8): `test_arithmetic_domains_valuation.py`, `test_arithmetic_euler.py`, `test_arithmetic_hurwitz_windows.py`, `test_arithmetic_mahler_polyphase.py`, `test_arithmetic_recurrence.py`, and 3 more in the machine-readable ledger. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.7.2 `komorebi_math.arithmetic.domains`

Purpose and source. Finite modular and truncated local arithmetic domains. It is an ordinary module in the `arithmetic` family. The ledger classifies it as a public implementation owner. Source `src/komorebi_math/arithmetic/domains.py` contains 159 lines and 5,117 bytes. Source SHA-256 begins `47d71569e4ead7e0`.

Surface and ownership. API classification: API-manifested implementation owner. It owns 1 manifest card: 0 stable, 0 beta, and 1 research. Linked cards: TruncatedPadic. Its `__all__` declaration contains 3 names: `ModularRing`, `PrimeField`, `TruncatedPadic`.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (2): `komorebi_math._number_theory`, `komorebi_math.exceptions`. Reverse package dependents (2): `komorebi_math.arithmetic`, `komorebi_math.arithmetic.hurwitz`. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.7.3 `komorebi_math.arithmetic.euler`

Purpose and source. Odd and dyadic Euler coordinate systems with exact local certificates. It is an ordinary module in the `arithmetic` family. The ledger classifies it as a public implementation owner. Source `src/komorebi_math/arithmetic/euler.py` contains 354 lines and 13,261 bytes. Source SHA-256 begins `0e0ae8608e6fa652`.

Surface and ownership. API classification: API-manifested implementation owner. It owns 2 manifest cards: 0 stable, 0 beta, and 2 research. Linked cards: EulerLocalCoordinates, signed_euler_number. Its `__all__` declaration contains 14 names: `bernoulli_numbers`, `dyadic_beta`, `dyadic_kernel`, `dyadic_kernel_valuation`, `dyadic_row_weight`, `dyadic_source`, `euler_zigzag_number`, `euler_zigzag_numbers`, `EulerLocalCoordinates`, `EulerTruncation`, and 4 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (5): `komorebi_math._number_theory`, `komorebi_math.arithmetic.valuation`, `komorebi_math.certificates`, `komorebi_math.exceptions`, `komorebi_math.provenance`. Reverse package dependents (2): `komorebi_math.arithmetic`, `komorebi_math.sequences.catalog`. Examples (1): `export_reports.py`. Tests

or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.7.4 `komorebi_math.arithmetic.hurwitz`

Purpose and source. Hurwitz series, exponential modes, and finite field subgroup inversion. It is an ordinary module in the `arithmetic` family. The ledger classifies it as a public implementation owner. Source `src/komorebi_math/arithmetic/hurwitz.py` contains 263 lines and 9,646 bytes. Source SHA-256 begins `1d1c43b85d0e39f0`.

Surface and ownership. API classification: API-manifested implementation owner. It owns 1 manifest card: 0 stable, 0 beta, and 1 research. Linked cards: HurwitzSeries. Its `__all__` declaration contains 7 names: `compressed_hurwitz_inverse`, `compressed_period`, `hurwitz_inverse`, `hurwitz_product`, `HurwitzSeries`, `subgroup_roots`, `subgroup_selector`.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (4): `komorebi_math._number_theory`, `komorebi_math.arithmetic.domains`, `komorebi_math.exceptions`, `komorebi_math.provenance`. Reverse package dependents (2): `komorebi_math.arithmetic`, `komorebi_math.arithmetic.standard_windows`. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.7.5 `komorebi_math.arithmetic.local_modes`

Purpose and source. Carmichael jumps, scalar local modes, and finite simultaneous congruences. It is an ordinary module in the `arithmetic` family. The ledger classifies it as unmanifested implementation support. Source `src/komorebi_math/arithmetic/local_modes.py` contains 151 lines and 5,269 bytes. Source SHA-256 begins `db9e8b16776da56c`.

Surface and ownership. API classification: importable package module outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its `__all__` declaration contains 6 names: `certify_difference_congruence`, `DifferenceCongruence`, `finite_difference_value`, `local_unit_jump`, `simultaneous_unit_difference`, `unit_mode_for_precision`.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (5): `komorebi_math._number_theory`, `komorebi_math.arithmetic.mahler`, `komorebi_math.arithmetic.valuation`, `komorebi_math.certificates`, `komorebi_math.exceptions`. Reverse package dependents (1): `komorebi_math.arithmetic`. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.7.6 `komorebi_math.arithmetic.mahler`

Purpose and source. Finite Mahler coordinates, affine reindexing, and local unit modes. It is an ordinary module in the `arithmetic` family. The ledger classifies it as a public implementation owner. Source `src/komorebi_math/arithmetic/mahler.py` contains 427 lines and 16,377 bytes. Source SHA-256 begins `3f70a036ab857b89`.

Surface and ownership. API classification: API-manifested implementation owner. It owns 2 manifest cards: 0 stable, 0 beta, and 2 research. Linked cards: affine_mahler_kernel, MahlerSeries. Its `__all__` declaration contains 12 names: `affine_kernel_composition_certificate`, `affine_kernel_matrix`, `affine_mahler_kernel`, `difference_table`, `evaluate_matrix_mahler`, `generalized_binomial`, `mahler_coefficients`, `MahlerSeries`, `matrix_mahler_coefficients`, `prime_power_kernel_certificate`, and 2 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (5): `komorebi_math._number_theory`, `komorebi_math.arithmetic.valuation`, `komorebi_math.certificates`, `komorebi_math.exceptions`, `komorebi_math.provenance`. Reverse package dependents (3): `komorebi_math.arithmetic`, `komorebi_math.arithmetic.local_modes`, `komorebi_math.arithmetic.polyphase`. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.7.7 komorebi_math.arithmetic.polyphase

Purpose and source. Exact discrete and Mahler polyphase refinement for arithmetic sequences. It is an ordinary module in the arithmetic family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/arithmetic/polyphase.py contains 135 lines and 5,407 bytes. Source SHA-256 begins a03e0b8499db987c.

Surface and ownership. API classification: API-manifested implementation owner. It owns 6 manifest cards: 6 stable, 0 beta, and 0 research. Linked cards: arithmetic_mixed_radix_address, arithmetic_mixed_radix_digits, arithmetic_polyphase_analyze, arithmetic_polyphase_synthesize, ArithmeticPolyphase, mahler_polyphase. Its __all__ declaration contains 6 names: ArithmeticPolyphase, mahler_polyphase, mixed_radix_address, mixed_radix_digits, polyphase_analyze, polyphase_synthesize.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (4): komorebi_math._number_theory, komorebi_math.arithmetic.mahler, komorebi_math.exceptions, komorebi_math.refinement.polyphase. Reverse package dependents (5): komorebi_math.arithmetic, komorebi_math.refinement.polyphase, komorebi_math.refinement.portable, komorebi_math.stable, komorebi_math.stable_semantics. Examples (0): none. Tests or test support files (1): test_block15_polyphase_refinement.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.7.8 komorebi_math.arithmetic.recurrence

Purpose and source. Exact linear recurrences, residue packets, and modular orbit diagnostics. It is an ordinary module in the arithmetic family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/arithmetic/recurrence.py contains 444 lines and 17,704 bytes. Source SHA-256 begins 3374acced7d63e28.

Surface and ownership. API classification: API-manifested implementation owner. It owns 2 manifest cards: 0 stable, 0 beta, and 2 research. Linked cards: fibonacci_recurrence, LinearRecurrence. Its __all__ declaration contains 10 names: determinant_integer, fibonacci_recurrence, LinearRecurrence, lucas_recurrence, matrix_power_exact, matrix_power_mod, OrbitPeriod, RecurrencePacket, second_order_jump, second_order_trace.

Dependencies and evidence. Direct dependency profile: optional exact extra, required NumPy dependency. Direct package dependencies (3): komorebi_math.certificates, komorebi_math.exceptions, komorebi_math.provenance. Reverse package dependents (4): komorebi_math.arithmetic, komorebi_math.coordinates.recurrence, komorebi_math.sequences.catalog, komorebi_math.sequences.compiler. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.7.9 komorebi_math.arithmetic.standard_windows

Purpose and source. Finite-field standard windows and congruential inverse coordinates. It is an ordinary module in the arithmetic family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/arithmetic/standard_windows.py contains 223 lines and 8,478 bytes. Source SHA-256 begins a255d07bb27261ab.

Surface and ownership. API classification: API-manifested implementation owner. It owns 1 manifest card: 0 stable, 0 beta, and 1 research. Linked cards: StandardWindow. Its __all__ declaration contains 6 names: PeriodicGeneratingFunction, rational_mod, standard_window_inverse, standard_window_period, StandardWindow, subgroup_standard_window_inverse.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (6): komorebi_math._number_theory, komorebi_math.analytic, komorebi_math.arithmetic.hurwitz, komorebi_math.certificates, komorebi_math.exceptions, komorebi_math.provenance. Reverse package dependents (1): komorebi_math.arithmetic. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.7.10 komorebi_math.arithmetic.valuation

Purpose and source. Exact valuations, digit sums, and local precision helpers. It is an ordinary module in the arithmetic family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/arithmetic/valuation.py contains 201 lines and 6,362 bytes. Source SHA-256 begins b5b73ce2d861abfd.

Surface and ownership. API classification: API-manifested implementation owner. It owns 1 manifest card: 0 stable, 0 beta, and 1 research. Linked cards: padic_valuation. Its __all__ declaration contains 11 names: binomial_valuation, binomial_valuation_from_digits, carmichael_exponent_prime_power, digit_sum, exact_binomial, factorial_valuation, falling_factorial, padic_valuation, unit_difference_valuation, unit_difference_value, and 1 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (3): komorebi_math._number_theory, komorebi_math.exceptions, komorebi_math.provenance. Reverse package dependents (4): komorebi_math.arithmetic, komorebi_math.arithmetic.euler, komorebi_math.arithmetic.local_modes, komorebi_math.arithmetic.mahler. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.8 Sequences: 7 modules

D.2.8.1 komorebi_math.sequences

Purpose and source. Lazy exact sequences, automatic models, holonomic recurrences, and compilation. It is the package initializer for the sequences family. The ledger classifies it as an importable package facade outside API manifests. Source src/komorebi_math/sequences/__init__.py contains 92 lines and 2,241 bytes. Source SHA-256 begins ce0573d804d3d36a.

Surface and ownership. API classification: importable package module outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its __all__ declaration contains 36 names: apery_recurrence, apery_sequence, AutomaticSequence, base_digits, bell_sequence, bernoulli_sequence, binomial_transform, binomial_transform_values, catalan_sequence, cauchy_product, and 26 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (8): komorebi_math._planned, komorebi_math.registry, komorebi_math.sequences.automatic, komorebi_math.sequences.base, komorebi_math.sequences.catalog, komorebi_math.sequences.compiler, and 2 more in the machine-readable ledger. Reverse package dependents (3): komorebi_math, komorebi_math.cli.main, komorebi_math.verify.sequences. Examples (3): automatic_holonomic.py, export_reports.py, sequence_compiler.py. Tests or test support files (2): test_sequences_execution.py, test_sequences_models.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.8.2 komorebi_math.sequences.automatic

Purpose and source. Finite automata with output for exact automatic sequence execution. It is an ordinary module in the sequences family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/sequences/automatic.py contains 181 lines and 6,938 bytes. Source SHA-256 begins 00eb49222a7e5cde.

Surface and ownership. API classification: API-manifested implementation owner. It owns 1 manifest card: 0 stable, 0 beta, and 1 research. Linked cards: AutomaticSequence. Its __all__ declaration contains 3 names: AutomaticSequence, base_digits, thue_morse_automaton.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (4): komorebi_math._number_theory, komorebi_math.certificates, komorebi_math.exceptions, komorebi_math.sequences.base. Reverse package dependents (3): komorebi_math.sequences, komorebi_math.sequences.catalog, komorebi_math.sequences.compiler. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.8.3 komorebi_math.sequences.base

Purpose and source. Lazy exact sequence objects and common sequence views. It is an ordinary module in the sequences family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/sequences/base.py contains 126 lines and 4,965 bytes. Source SHA-256 begins 7fbc0c79c09ed5d7.

Surface and ownership. API classification: API-manifested implementation owner. It owns 1 manifest card: 0 stable, 0 beta, and 1 research. Linked cards: Sequence. Its __all__ declaration contains 1 name: Sequence.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (2): komorebi_math._number_theory, komorebi_math.exceptions. Reverse package dependents (6): komorebi_math.sequences, komorebi_math.sequences.automatic, komorebi_math.sequences.catalog, komorebi_math.sequences.compiler, komorebi_math.sequences.holonomic, komorebi_math.sequences.transforms. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.8.4 komorebi_math.sequences.catalog

Purpose and source. A small exact catalog of classical integer sequences. It is an ordinary module in the sequences family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/sequences/catalog.py contains 162 lines and 4,501 bytes. Source SHA-256 begins 655764bb651a0f1c.

Surface and ownership. API classification: API-manifested implementation owner. It owns 6 manifest cards: 0 stable, 0 beta, and 6 research. Linked cards: apery_sequence, fibonacci_sequence, get_sequence, motzkin_sequence, partition_sequence, thue_morse_sequence. Its __all__ declaration contains 13 names: apery_sequence, bell_sequence, bernoulli_sequence, catalan_sequence, euler_zigzag_sequence, fibonacci_sequence, get_sequence, lucas_sequence, motzkin_sequence, partition_sequence, and 3 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (5): komorebi_math.arithmetic.euler, komorebi_math.arithmetic.recurrence, komorebi_math.sequences.automatic, komorebi_math.sequences.base, komorebi_math.sequences.holonomic. Reverse package dependents (1): komorebi_math.sequences. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.8.5 komorebi_math.sequences.compiler

Purpose and source. Finite sequence analysis and executable model compilation. It is an ordinary module in the sequences family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/sequences/compiler.py contains 293 lines and 12,180 bytes. Source SHA-256 begins 7c8a2dbf9deeded3.

Surface and ownership. API classification: API-manifested implementation owner. It owns 2 manifest cards: 0 stable, 0 beta, and 2 research. Linked cards: compile_sequence, CompiledSequence. Its __all__ declaration contains 4 names: compile_sequence, CompiledSequence, detect_linear_recurrence, recurrence_generating_function.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (7): komorebi_math._number_theory, komorebi_math.arithmetic.recurrence, komorebi_math.certificates, komorebi_math.exceptions, komorebi_math.sequences.automatic, komorebi_math.sequences.base, and 1 more in the machine-readable ledger. Reverse package dependents (1): komorebi_math.sequences. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.8.6 komorebi_math.sequences.holonomic

Purpose and source. Exact polynomial coefficient recurrences for holonomic sequences. It is an ordinary module in the sequences family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/sequences/holonomic.py contains 178 lines and 6,725 bytes. Source SHA-256 begins 59d3301e58c9c239.

Surface and ownership. API classification: API-manifested implementation owner. It owns 1 manifest card: 0 stable, 0 beta, and 1 research. Linked cards: HolonomicRecurrence. Its __all__ declaration contains 5 names: apery_recurrence, evaluate_polynomial, HolonomicRecurrence, motzkin_recurrence, Polynomial.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (4): `komorebi_math._number_theory`, `komorebi_math.certificates`, `komorebi_math.exceptions`, `komorebi_math.sequences.base`. Reverse package dependents (3): `komorebi_math.sequences`, `komorebi_math.sequences.catalog`, `komorebi_math.sequences.compiler`. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.8.7 `komorebi_math.sequences.transforms`

Purpose and source. Exact finite and lazy transformations of integer sequences. It is an ordinary module in the `sequences` family. The ledger classifies it as unmanifested implementation support. Source `src/komorebi_math/sequences/transforms.py` contains 103 lines and 3,416 bytes. Source SHA-256 begins `53d6359493989a4a`.

Surface and ownership. API classification: importable package module outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its `__all__` declaration contains 7 names: `binomial_transform`, `binomial_transform_values`, `cauchy_product`, `finite_difference`, `inverse_binomial_transform`, `inverse_binomial_transform_values`, `partial_sums`.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (3): `komorebi_math._number_theory`, `komorebi_math.exceptions`, `komorebi_math.sequences.base`. Reverse package dependents (1): `komorebi_math.sequences`. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.9 Analytic: 7 modules

D.2.9.1 `komorebi_math.analytic`

Purpose and source. Cyclic analytic functions, reciprocal families, and zero diagnostics. It is the package initializer for the `analytic` family. The ledger classifies it as an importable package facade outside API manifests. Source `src/komorebi_math/analytic/__init__.py` contains 115 lines and 2,835 bytes. Source SHA-256 begins `a66468c72a4eeda3`.

Surface and ownership. API classification: importable package module outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its `__all__` declaration contains 43 names: `appell_coefficients`, `appell_polynomial`, `appell_polynomial_symbolic`, `asymptotic_positive_zero`, `congruential_inverse`, `congruential_reciprocal_coefficients`, `cyclic_addition`, `cyclic_packets`, `cyclic_product_packets`, `cyclic_project_coefficients`, and 33 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (7): `komorebi_math._planned`, `komorebi_math.analytic.appell`, `komorebi_math.analytic.cyclic`, `komorebi_math.analytic.reciprocal`, `komorebi_math.analytic.sections`, `komorebi_math.analytic.zeros`, and 1 more in the machine-readable ledger. Reverse package dependents (3): `komorebi_math`, `komorebi_math.arithmetic.standard_windows`, `komorebi_math.verify.analytic`. Examples (1): `cyclic_analytic.py`. Tests or test support files (2): `test_analytic_appell_zeros.py`, `test_analytic_cyclic_reciprocal.py`. Documentation files with literal module or public-route references (2): `API_INVENTORY.md`, `PUBLIC_API.md`. AST parsing and runtime import both passed in the recorded module audit.

D.2.9.2 `komorebi_math.analytic._series`

Purpose and source. Finite formal power series helpers for the analytic package. It is an ordinary module in the `analytic` family. The ledger classifies it as an internal support module. Source `src/komorebi_math/analytic/_series.py` contains 137 lines and 4,660 bytes. Source SHA-256 begins `c2741faf01809dd7`.

Surface and ownership. API classification: internal by module name. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its `__all__` declaration contains 8 names: `inverse_series`, `logarithmic_coefficients`, `multi-`

ply_series, positive_section_coefficients, power_series_from_log, signed_denominator_coefficients, validate_degree, validate_order.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (1): komorebi_math.exceptions. Reverse package dependents (5): komorebi_math.analytic.appell, komorebi_math.analytic.cyclic, komorebi_math.analytic.reciprocal, komorebi_math.analytic.sections, komorebi_math.analytic.zeros. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.9.3 komorebi_math.analytic.appell

Purpose and source. Congruential reciprocal, Appell, and theta polynomial families. It is an ordinary module in the analytic family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/analytic/appell.py contains 161 lines and 5,577 bytes. Source SHA-256 begins 77e09c343a4ea948.

Surface and ownership. API classification: API-manifested implementation owner. It owns 2 manifest cards: 0 stable, 0 beta, and 2 research. Linked cards: appell_polynomial, theta_polynomial. Its __all__ declaration contains 7 names: appell_coefficients, appell_polynomial, appell_polynomial_symbolic, congruential_inverse, congruential_reciprocal_coefficients, theta_polynomial, theta_polynomial_symbolic.

Dependencies and evidence. Direct dependency profile: optional exact extra. Direct package dependencies (3): komorebi_math.analytic._series, komorebi_math.exceptions, komorebi_math.provenance. Reverse package dependents (1): komorebi_math.analytic. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.9.4 komorebi_math.analytic.cyclic

Purpose and source. Cyclic projection and residue packet operations for analytic series. It is an ordinary module in the analytic family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/analytic/cyclic.py contains 140 lines and 4,844 bytes. Source SHA-256 begins 1fe389400de98f65.

Surface and ownership. API classification: API-manifested implementation owner. It owns 1 manifest card: 0 stable, 0 beta, and 1 research. Linked cards: cyclic_project_coefficients. Its __all__ declaration contains 6 names: cyclic_packets, cyclic_product_packets, cyclic_project_coefficients, cyclic_project_value, cyclic_projection_certificate, cyclic_reconstruct.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (4): komorebi_math.analytic._series, komorebi_math.certificates, komorebi_math.exceptions, komorebi_math.provenance. Reverse package dependents (1): komorebi_math.analytic. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.9.5 komorebi_math.analytic.reciprocal

Purpose and source. Reciprocal powers, logarithmic coefficients, and transfer tangent data. It is an ordinary module in the analytic family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/analytic/reciprocal.py contains 170 lines and 6,173 bytes. Source SHA-256 begins 50e0b6e36c23385f.

Surface and ownership. API classification: API-manifested implementation owner. It owns 2 manifest cards: 0 stable, 0 beta, and 2 research. Linked cards: reciprocal_coefficients, transfer_tangent. Its __all__ declaration contains 7 names: logarithmic_coefficients, reciprocal_coefficients, reciprocal_polynomial, reciprocal_semigroup_certificate, reciprocal_value, transfer_tangent, transfer_tangent_coefficients.

Dependencies and evidence. Direct dependency profile: optional exact extra, required NumPy dependency. Direct package dependencies (5): komorebi_math.analytic._series, komorebi_math.analytic.sections, komorebi_math.certificates, komorebi_math.exceptions, komorebi_math.provenance. Reverse package dependents (1): komorebi_math.analytic. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.9.6 `komorebi_math.analytic.sections`

Purpose and source. Transfer exponential sections and signed cyclic differential bases. It is an ordinary module in the `analytic` family. The ledger classifies it as a public implementation owner. Source `src/komorebi_math/analytic/sections.py` contains 287 lines and 9,125 bytes. Source SHA-256 begins `d33b4dcff5f0c442`.

Surface and ownership. API classification: API-manifested implementation owner. It owns 4 manifest cards: 0 stable, 0 beta, and 4 research. Linked cards: CyclicSectionFamily, denominator, signed_section, transfer_section. Its `__all__` declaration contains 12 names: `cyclic_addition`, `CyclicSectionFamily`, `denominator`, `denominator_derivative`, `denominator_root_sum`, `section_certificate`, `signed_section`, `signed_sections`, `signed_shift_matrix`, `transfer_section`, and 2 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: optional `scientific` extra, required NumPy dependency. Direct package dependencies (5): `komorebi_math.analytic._series`, `komorebi_math.certificates`, `komorebi_math.dynamics._linalg`, `komorebi_math.exceptions`, `komorebi_math.provenance`. Reverse package dependents (3): `komorebi_math.analytic`, `komorebi_math.analytic.reciprocal`, `komorebi_math.analytic.zeros`. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.9.7 `komorebi_math.analytic.zeros`

Purpose and source. Numerical zero strings, Hadamard products, and zeta diagnostics. It is an ordinary module in the `analytic` family. The ledger classifies it as a public implementation owner. Source `src/komorebi_math/analytic/zeros.py` contains 275 lines and 10,246 bytes. Source SHA-256 begins `f39a5a958e9dd837`.

Surface and ownership. API classification: API-manifested implementation owner. It owns 2 manifest cards: 0 stable, 0 beta, and 2 research. Linked cards: CyclicDenominator, positive_zero_string. Its `__all__` declaration contains 8 names: `asymptotic_positive_zero`, `CyclicDenominator`, `hadamard_partial_product`, `positive_zero_string`, `reciprocal_coefficient_from_zeros`, `reduced_zero_zeta`, `zero_count_asymptotic`, `ZeroStringResult`.

Dependencies and evidence. Direct dependency profile: optional `scientific` extra, required NumPy dependency. Direct package dependencies (5): `komorebi_math.analytic._series`, `komorebi_math.analytic.sections`, `komorebi_math.certificates`, `komorebi_math.exceptions`, `komorebi_math.provenance`. Reverse package dependents (1): `komorebi_math.analytic`. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.10 Refinement: 10 modules

D.2.10.1 `komorebi_math.refinement`

Purpose and source. Refinement towers, weighted transfer, integral separation, and linear defects. It is the package initializer for the `refinement` family. The ledger classifies it as an importable package facade outside API manifests. Source `src/komorebi_math/refinement/__init__.py` contains 99 lines and 2,688 bytes. Source SHA-256 begins `86c29af7a7991527`.

Surface and ownership. API classification: importable package module outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its `__all__` declaration contains 43 names: `coarsen_lattice_polyphase`, `coarsen_polyphase`, `common_refinement`, `crt_many`, `crt_pair`, `CRTAddress`, `dyadic_odd_factor`, `factor_residue`, `FEATURE_KEY`, `FilteredInverseExpansion`, and 33 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (9): `komorebi_math.refinement.crt`, `komorebi_math.refinement.defects`, `komorebi_math.refinement.lattices`, `komorebi_math.refinement.polyphase`, `komorebi_math.refinement.residue`, `komorebi_math.refinement.smith`, and 3 more in the machine-readable ledger. Reverse package dependents (4): `komorebi_math`, `komorebi_math.qualification`, `komorebi_math.verify.refinement`, `komorebi_math.verify.stabilization`. Examples (3): `block15_polyphase_refinement.py`, `block21_public_capability_tour.py`, `refinement_pipeline.py`. Tests or test support files (10): `test_block15_persistence.py`, `test_block15_polyphase_refinement.py`, `test_block15_stable_api.py`, `test_block16_release_qualification.py`, `test_block17_upgrade_contract.py`, and 5 more

in the machine-readable ledger. Documentation files with literal module or public-route references (2): `API_INVENTORY.md`, `BLOCK_15_RELEASE.md`. AST parsing and runtime import both passed in the recorded module audit.

D.2.10.2 `komorebi_math.refinement.crt`

Purpose and source. Chinese remainder addresses for residue packet systems. It is an ordinary module in the `refinement` family. The ledger classifies it as a public implementation owner. Source `src/komorebi_math/refinement/crt.py` contains 128 lines and 5,280 bytes. Source SHA-256 begins `72cf5291bf5a9e82`.

Surface and ownership. API classification: API-manifested implementation owner. It owns 2 manifest cards: 0 stable, 0 beta, and 2 research. Linked cards: crt_many, crt_pair. Its `__all__` declaration contains 5 names: `crt_many`, `crt_pair`, `CRTAddress`, `dyadic_odd_factor`, `factor_residue`.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (1): `komorebi_math.provenance`. Reverse package dependents (1): `komorebi_math.refinement`. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.10.3 `komorebi_math.refinement.defects`

Purpose and source. Linear refinement defects and their propagated identities. It is an ordinary module in the `refinement` family. The ledger classifies it as a public implementation owner. Source `src/komorebi_math/refinement/defects.py` contains 302 lines and 13,618 bytes. Source SHA-256 begins `d256db55d9bb6fa3`.

Surface and ownership. API classification: API-manifested implementation owner. It owns 1 manifest card: 0 stable, 0 beta, and 1 research. Linked cards: LinearRefinementDefect. Its `__all__` declaration contains 2 names: `LinearRefinementDefect`, `multilevel_defect`.

Dependencies and evidence. Direct dependency profile: optional `exact` extra, optional `scientific` extra, required NumPy dependency. Direct package dependencies (4): `komorebi_math.backends`, `komorebi_math.certificates`, `komorebi_math.exceptions`, `komorebi_math.provenance`. Reverse package dependents (1): `komorebi_math.refinement`. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.10.4 `komorebi_math.refinement.lattices`

Purpose and source. Integral packet separation and lattice gluing modules. It is an ordinary module in the `refinement` family. The ledger classifies it as a public implementation owner. Source `src/komorebi_math/refinement/lattices.py` contains 290 lines and 11,522 bytes. Source SHA-256 begins `65fa26dd374a3c39`.

Surface and ownership. API classification: API-manifested implementation owner. It owns 1 manifest card: 0 stable, 0 beta, and 1 research. Linked cards: LatticePacketData. Its `__all__` declaration contains 4 names: `GluingModule`, `LatticePacketData`, `PrecisionDelay`, `refinement_weight_monotonicity`.

Dependencies and evidence. Direct dependency profile: optional `exact` extra. Direct package dependencies (5): `komorebi_math._number_theory`, `komorebi_math.certificates`, `komorebi_math.exceptions`, `komorebi_math.provenance`, `komorebi_math.refinement.smith`. Reverse package dependents (1): `komorebi_math.refinement`. Examples (1): `exact_lattice_separation.py`. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.10.5 `komorebi_math.refinement.polyphase`

Purpose and source. Canonical finite polyphase refinement and coarsening. It is an ordinary module in the `refinement` family. The ledger classifies it as a public implementation owner. Source `src/komorebi_math/refinement/polyphase.py` contains 997 lines and 40,482 bytes. Source SHA-256 begins `013f8f23a15f2443`.

Surface and ownership. API classification: API-manifested implementation owner. It owns 12 manifest cards: 12 stable, 0 beta, and 0 research. Linked cards: coarsen_lattice_polyphase, coarsen_polyphase, lattice_polyphase_coarsen, lattice_polyphase_refine, LatticePolyphaseRefinement, polyphase_coarsen, polyphase_refine, polyphase_refinement_certificate, PolyphaseRefinement, PolyphaseRefinementTower, and 2 more in the machine-readable ledger. Its `__all__` declaration contains 9 names: `coarsen_lattice_polyphase`, `coarsen_polyphase`, `LatticePolyphase`

Refinement, polyphase_refinement_certificate, PolyphaseCoordinateRecord, PolyphaseRefinement, PolyphaseRefinementTower, refine_lattice_polyphase, refine_polyphase.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (5): komorebi_math.arithmetic.polyphase, komorebi_math.certificates, komorebi_math.exceptions, komorebi_math.refinement.residue, komorebi_math.signals.polyphase. Reverse package dependents (7): komorebi_math.arithmetic.polyphase, komorebi_math.refinement, komorebi_math.refinement.portable, komorebi_math.refinement.residue, komorebi_math.signals.polyphase, komorebi_math.stable, and 1 more in the machine-readable ledger. Examples (0): none. Tests or test support files (2): test_block15_polyphase_refinement.py, test_block15_stable_api.py. Documentation files with literal module or public-route references (1): BLOCK_15_RELEASE.md. AST parsing and runtime import both passed in the recorded module audit.

D.2.10.6 komorebi_math.refinement.portable

Purpose and source. Trusted portable codecs for finite polyphase and refinement records. It is an ordinary module in the refinement family. The ledger classifies it as unmanifested implementation support. Source src/komorebi_math/refinement/portable.py contains 177 lines and 5,979 bytes. Source SHA-256 begins 7b2bd87214fe500d.

Surface and ownership. API classification: importable package module outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its __all__ declaration contains 1 name: register_refinement_codecs.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (6): komorebi_math.arithmetic.polyphase, komorebi_math.exceptions, komorebi_math.export.persistence, komorebi_math.refinement.polyphase, komorebi_math.refinement.residue, komorebi_math.signals.polyphase. Reverse package dependents (1): komorebi_math.export.persistence. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.10.7 komorebi_math.refinement.residue

Purpose and source. Residue and polyphase packet coordinates. It is an ordinary module in the refinement family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/refinement/residue.py contains 192 lines and 7,728 bytes. Source SHA-256 begins 8287237612523129.

Surface and ownership. API classification: API-manifested implementation owner. It owns 7 manifest cards: 5 stable, 0 beta, and 2 research. Linked cards: nested_residue_address, nested_residue_analysis, residue_analyze, residue_synthesize, ResidueCoordinates, residue_analyze, residue_packet_system. Its __all__ declaration contains 7 names: nested_residue_address, nested_residue_analysis, packet_coordinates_from_residues, residue_analyze, residue_packet_system, residue_synthesize, ResidueCoordinates.

Dependencies and evidence. Direct dependency profile: optional exact extra, required NumPy dependency. Direct package dependencies (5): komorebi_math.backends, komorebi_math.core.packets, komorebi_math.exceptions, komorebi_math.provenance, komorebi_math.refinement.polyphase. Reverse package dependents (4): komorebi_math.refinement, komorebi_math.refinement.polyphase, komorebi_math.refinement.portable, komorebi_math.stable. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.10.8 komorebi_math.refinement.smith

Purpose and source. Smith coordinates and congruence solving over prime-power rings. It is an ordinary module in the refinement family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/refinement/smith.py contains 324 lines and 12,783 bytes. Source SHA-256 begins fa6238011f016a44.

Surface and ownership. API classification: API-manifested implementation owner. It owns 1 manifest card: 0 stable, 0 beta, and 1 research. Linked cards: SmithCongruenceProblem. Its __all__ declaration contains 6 names: padic_valuation, rational_padic_valuation, smith_decomposition, SmithCongruenceProblem, SmithCongruenceSolution, SmithDecomposition.

Dependencies and evidence. Direct dependency profile: optional exact extra. Direct package dependencies (4): komorebi_math._number_theory, komorebi_math.certificates, komorebi_math.exceptions, komorebi_math.provenance. Re-

verse package dependents (3): komorebi_math.refinement, komorebi_math.refinement.lattices, komorebi_math.refinement.weighted. Examples (1): exact_lattice_separation.py. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.10.9 komorebi_math.refinement.towers

Purpose and source. Exact packet refinements and finite refinement towers. It is an ordinary module in the refinement family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/refinement/towers.py contains 266 lines and 12,635 bytes. Source SHA-256 begins 03928347574f0f76.

Surface and ownership. API classification: API-manifested implementation owner. It owns 6 manifest cards: 3 stable, 0 beta, and 3 research. Linked cards: common_refinement, PacketRefinement, RefinementTower, common_refinement, PacketRefinement, RefinementTower. Its __all__ declaration contains 3 names: common_refinement, PacketRefinement, RefinementTower.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (5): komorebi_math.certificates, komorebi_math.core.operators, komorebi_math.core.packets, komorebi_math.exceptions, komorebi_math.provenance. Reverse package dependents (3): komorebi_math.refinement, komorebi_math.stable, komorebi_math.unified. Examples (1): refinement_pipeline.py. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.10.10 komorebi_math.refinement.weighted

Purpose and source. Weighted packet transfers and filtered inverse expansions. It is an ordinary module in the refinement family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/refinement/weighted.py contains 403 lines and 17,163 bytes. Source SHA-256 begins 94c13dd4ed26909c.

Surface and ownership. API classification: API-manifested implementation owner. It owns 2 manifest cards: 0 stable, 0 beta, and 2 research. Linked cards: WeightedTransferEdge, WeightedTransferGraph. Its __all__ declaration contains 4 names: FilteredInverseExpansion, WeightedPath, WeightedTransferEdge, WeightedTransferGraph.

Dependencies and evidence. Direct dependency profile: optional exact extra, required NumPy dependency. Direct package dependencies (7): komorebi_math._number_theory, komorebi_math.backends, komorebi_math.certificates, komorebi_math.core.packets, komorebi_math.exceptions, komorebi_math.provenance, and 1 more in the machine-readable ledger. Reverse package dependents (1): komorebi_math.refinement. Examples (1): refinement_pipeline.py. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.11 Coordinates: 24 modules

D.2.11.1 komorebi_math.coordinates

Purpose and source. General exact and numerical coordinate calculus across Komorebi Math. It is the package initializer for the coordinates family. The ledger classifies it as a canonical API namespace facade. Source src/komorebi_math/coordinates/__init__.py contains 521 lines and 14,575 bytes. Source SHA-256 begins 94c38b190c95bbea.

Surface and ownership. API classification: canonical public namespace without implementation ownership. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its __all__ declaration contains 237 names: affine_basis_transform, affine_polynomial, affine_substitution_coordinate_operator, affine_substitution_transform, AffineCoordinateTransform, AnalysisSynthesisBlocks, AnalysisSynthesisCoefficientBlocks, AnalysisSynthesisGauge, AnalysisSynthesisMap, AnalysisSynthesisSystem, and 227 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (23): komorebi_math.coordinates.affine, komorebi_math.coordinates.analysis_synthesis, komorebi_math.coordinates.basis, komorebi_math.coordinates.budget, komorebi_math.coordinates.calculus, komorebi_math.coordinates.classical, and 17 more in the machine-readable ledger. Reverse package dependents (9): komorebi_math, komorebi_math.cli.main, komorebi_math.golden, komorebi_math.qualification,

komorebi_math.stable, komorebi_math.verify.coordinates, and 3 more in the machine-readable ledger. Examples (9): block19_measure_integration_worked_example.py, block21_public_capability_tour.py, classical_coordinate_families.py, coordinate_calculus.py, euler_to_fibonacci_coordinates.py, and 4 more in the machine-readable ledger. Tests or test support files (20): test_block14_cli_api.py, test_block16_analysis_synthesis.py, test_block16_contracts.py, test_block18_frame_algebra.py, test_block19_integration.py, and 15 more in the machine-readable ledger. Documentation files with literal module or public-route references (6): API_COMPATIBILITY_0.14.md, API_INVENTORY.md, BLOCK_14_RELEASE.md, COMPANION_COORDINATE_CALCULUS.md, GENERAL_COORDINATE_ATLAS.md, and 1 more in the machine-readable ledger. AST parsing and runtime import both passed in the recorded module audit.

D.2.11.2 komorebi_math.coordinates.affine

Purpose and source. Exact affine, homogeneous, and barycentric coordinate changes. It is an ordinary module in the coordinates family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/coordinates/affine.py contains 278 lines and 10,891 bytes. Source SHA-256 begins 758d3b5f1ea53c60.

Surface and ownership. API classification: API-manifested implementation owner. It owns 5 manifest cards: 0 stable, 0 beta, and 5 research. Linked cards: AffineCoordinateTransform, affine_basis_transform, AffineCoordinateTransform, BarycentricCoordinateSystem, translation_transform. Its __all__ declaration contains 4 names: affine_basis_transform, AffineCoordinateTransform, BarycentricCoordinateSystem, translation_transform.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (3): komorebi_math.certificates, komorebi_math.coordinates.exact, komorebi_math.exceptions. Reverse package dependents (4): komorebi_math, komorebi_math.coordinates, komorebi_math.coordinates.portable, komorebi_math.verify.coordinates. Examples (1): formal_affine_coordinate_changes.py. Tests or test support files (2): test_coordinates_general_affine_formal.py, test_coordinates_general_persistence.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.11.3 komorebi_math.coordinates.analysis_synthesis

Purpose and source. General finite analysis-synthesis systems. It is an ordinary module in the coordinates family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/coordinates/analysis_synthesis.py contains 384 lines and 15,752 bytes. Source SHA-256 begins 08f1edf9d69ee112.

Surface and ownership. API classification: API-manifested implementation owner. It owns 6 manifest cards: 2 stable, 0 beta, and 4 research. Linked cards: AnalysisSynthesisBlocks, AnalysisSynthesisSystem, AnalysisSynthesisBlocks, AnalysisSynthesisSystem, AnalysisSynthesisBlocks, AnalysisSynthesisSystem. Its __all__ declaration contains 2 names: AnalysisSynthesisBlocks, AnalysisSynthesisSystem.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (2): komorebi_math.certificates, komorebi_math.exceptions. Reverse package dependents (10): komorebi_math, komorebi_math.coordinates, komorebi_math.coordinates.frame_algebra, komorebi_math.coordinates.portable, komorebi_math.integration.adapters, komorebi_math.qualification, and 4 more in the machine-readable ledger. Examples (2): block16_analysis_synthesis.py, block21_public_capability_tour.py. Tests or test support files (4): test_block16_analysis_synthesis.py, test_block16_contracts.py, test_block18_frame_algebra.py, test_block19_integration.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.11.4 komorebi_math.coordinates.basis

Purpose and source. Arbitrary finite bases and exact coordinate atlases. It is an ordinary module in the coordinates family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/coordinates/basis.py contains 535 lines and 21,004 bytes. Source SHA-256 begins 9f7351fd968c2c8c.

Surface and ownership. API classification: API-manifested implementation owner. It owns 20 manifest cards: 4 stable, 0 beta, and 16 research. Linked cards: basis_from_matrix, CoordinateAtlas, FiniteBasis, standard_basis, basis_from_matrix, CoordinateAtlas, CoordinateSystem, FiniteBasis, standard_basis, AtlasPath, and 10 more in the machine-readable ledger. Its __all__ declaration contains 11 names: AtlasPath, basis_from_matrix, CommutingDiagramCheck, CoordinateAtlas, CoordinateSystem, direct_sum_basis, FiniteBasis, permutation_transform, scaling_transform, standard_basis, and 1 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (5): `komorebi_math.certificates`, `komorebi_math.coordinates.budget`, `komorebi_math.coordinates.exact`, `komorebi_math.coordinates.transform`, `komorebi_math.exceptions`. Reverse package dependents (12): `komorebi_math`, `komorebi_math.cli.main`, `komorebi_math.coordinates`, `komorebi_math.coordinates.classical`, `komorebi_math.coordinates.families`, `komorebi_math.coordinates.packet`, and 6 more in the machine-readable ledger. Examples (1): `general_coordinate_atlas.py`. Tests or test support files (2): `test_coordinates_general_basis.py`, `test_coordinates_general_persistence.py`. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.11.5 `komorebi_math.coordinates.budget`

Purpose and source. Explicit resource contracts for finite coordinate computations. It is an ordinary module in the `coordinates` family. The ledger classifies it as a public implementation owner. Source `src/komorebi_math/coordinates/budget.py` contains 71 lines and 3,029 bytes. Source SHA-256 begins `0d9e7122304df293`.

Surface and ownership. API classification: API-manifested implementation owner. It owns 3 manifest cards: 0 stable, 0 beta, and 3 research. Linked cards: CoordinateBudget, CoordinateBudget, DEFAULT_COORDINATE_BUDGET. Its `__all__` declaration contains 2 names: `CoordinateBudget`, `DEFAULT_COORDINATE_BUDGET`.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (1): `komorebi_math.exceptions`. Reverse package dependents (13): `komorebi_math`, `komorebi_math.coordinates`, `komorebi_math.coordinates.basis`, `komorebi_math.coordinates.calculus`, `komorebi_math.coordinates.classical`, `komorebi_math.coordinates.combinatorial`, and 7 more in the machine-readable ledger. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.11.6 `komorebi_math.coordinates.calculus`

Purpose and source. Operator matrices in arbitrary polynomial coordinates. It is an ordinary module in the `coordinates` family. The ledger classifies it as a public implementation owner. Source `src/komorebi_math/coordinates/calculus.py` contains 314 lines and 10,152 bytes. Source SHA-256 begins `7156398767a48211`.

Surface and ownership. API classification: API-manifested implementation owner. It owns 11 manifest cards: 0 stable, 0 beta, and 11 research. Linked cards: PolynomialCoordinateOperator, affine_substitution_coordinate_operator, derivative_coordinate_operator, finite_difference_coordinate_operator, integral_coordinate_operator, multiplication_coordinate_operator, operator_commutator, polynomial_integral, polynomial_operator, PolynomialCoordinateOperator, and 1 more in the machine-readable ledger. Its `__all__` declaration contains 10 names: `affine_substitution_coordinate_operator`, `derivative_coordinate_operator`, `finite_difference_coordinate_operator`, `integral_coordinate_operator`, `multiplication_coordinate_operator`, `operator_commutator`, `polynomial_integral`, `polynomial_operator`, `PolynomialCoordinateOperator`, `transport_polynomial_operator`.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (7): `komorebi_math.certificates`, `komorebi_math.coordinates.budget`, `komorebi_math.coordinates.classical`, `komorebi_math.coordinates.exact`, `komorebi_math.coordinates.polynomials`, `komorebi_math.coordinates.tensor`, and 1 more in the machine-readable ledger. Reverse package dependents (5): `komorebi_math`, `komorebi_math.coordinates`, `komorebi_math.coordinates.portable`, `komorebi_math.integration.coordinates`, `komorebi_math.verify.coordinates`. Examples (1): `classical_coordinate_families.py`. Tests or test support files (2): `test_coordinates_general_calculus.py`, `test_coordinates_general_persistence.py`. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.11.7 `komorebi_math.coordinates.classical`

Purpose and source. Classical polynomial families and interpolation coordinate systems. It is an ordinary module in the `coordinates` family. The ledger classifies it as a public implementation owner. Source `src/komorebi_math/coordinates/classical.py` contains 657 lines and 21,840 bytes. Source SHA-256 begins `4a44df81afb429df`.

Surface and ownership. API classification: API-manifested implementation owner. It owns 45 manifest cards: 1 stable, 0 beta, and 44 research. Linked cards: get_polynomial_family, classical_polynomial_catalog, get_polynomial_family, affine_polynomial, affine_substitution_transform, appell_polynomial, appell_polynomial_

family, bernoulli_number, bernoulli_polynomial, bernoulli_polynomial_family, and 35 more in the machine-readable ledger. Its `__all__` declaration contains 42 names: `affine_polynomial`, `affine_substitution_transform`, `appell_polynomial`, `appell_polynomial_family`, `bernoulli_number`, `bernoulli_polynomial`, `bernoulli_polynomial_family`, `bernstein_basis`, `bernstein_polynomial`, `binomial_polynomial`, and 32 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (6): `komorebi_math.coordinates.basis`, `komorebi_math.coordinates.budget`, `komorebi_math.coordinates.exact`, `komorebi_math.coordinates.polynomials`, `komorebi_math.coordinates.transform`, `komorebi_math.exceptions`. Reverse package dependents (8): `komorebi_math`, `komorebi_math.cli.main`, `komorebi_math.coordinates`, `komorebi_math.coordinates.calculus`, `komorebi_math.golden`, `komorebi_math.stable`, and 2 more in the machine-readable ledger. Examples (1): `classical_coordinate_families.py`. Tests or test support files (3): `test_block19_integration.py`, `test_coordinates_general_calculus.py`, `test_coordinates_general_classical.py`. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.11.8 `komorebi_math.coordinates.combinatorial`

Purpose and source. Exact combinatorial coordinate transforms for finite sequences. It is an ordinary module in the `coordinates` family. The ledger classifies it as a public implementation owner. Source `src/komorebi_math/coordinates/combinatorial.py` contains 381 lines and 12,049 bytes. Source SHA-256 begins `bb0a73e44deecfd2`.

Surface and ownership. API classification: API-manifested implementation owner. It owns 19 manifest cards: 1 stable, 0 beta, and 18 research. Linked cards: forward_difference_transform, binomial_transform, forward_difference_transform, binomial_transform, divisor_zeta_transform, falling_factorial_to_monomial_transform, forward_difference_transform, lah_number, mobius_integer, monomial_to_falling_factorial_transform, and 9 more in the machine-readable ledger. Its `__all__` declaration contains 16 names: `binomial_transform`, `divisor_zeta_transform`, `falling_factorial_to_monomial_transform`, `forward_difference_transform`, `lah_number`, `mobius_integer`, `monomial_to_falling_factorial_transform`, `monomial_to_rising_factorial_transform`, `newton_evaluation_transform`, `rising_factorial_to_monomial_transform`, and 6 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (4): `komorebi_math.coordinates.budget`, `komorebi_math.coordinates.exact`, `komorebi_math.coordinates.transform`, `komorebi_math.exceptions`. Reverse package dependents (6): `komorebi_math`, `komorebi_math.cli.main`, `komorebi_math.coordinates`, `komorebi_math.golden`, `komorebi_math.stable`, `komorebi_math.verify.coordinates`. Examples (0): none. Tests or test support files (2): `test_coordinates_general_combinatorial.py`, `test_coordinates_general_tensor_modular.py`. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.11.9 `komorebi_math.coordinates.exact`

Purpose and source. Small exact linear algebra over the rational numbers. It is an ordinary module in the `coordinates` family. The ledger classifies it as a public implementation owner. Source `src/komorebi_math/coordinates/exact.py` contains 357 lines and 11,758 bytes. Source SHA-256 begins `730ffa5075436ef2`.

Surface and ownership. API classification: API-manifested implementation owner. It owns 19 manifest cards: 0 stable, 0 beta, and 19 research. Linked cards: as_matrix, as_vector, determinant, ExactLinearSystem, identity_matrix, inverse_matrix, matrix_add, matrix_from_columns, matrix_multiply, matrix_power, and 9 more in the machine-readable ledger. Its `__all__` declaration contains 24 names: `as_matrix`, `as_vector`, `columns`, `determinant`, `ExactLinearSystem`, `identity_matrix`, `inverse_matrix`, `kronecker`, `Matrix`, `matrix_add`, and 14 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (1): `komorebi_math.exceptions`. Reverse package dependents (27): `komorebi_math.coordinates`, `komorebi_math.coordinates.affine`, `komorebi_math.coordinates.basis`, `komorebi_math.coordinates.calculus`, `komorebi_math.coordinates.classical`, `komorebi_math.coordinates.combinatorial`, and 21 more in the machine-readable ledger. Examples (0): none. Tests or test support files (9): `test_block18_frame_algebra.py`, `test_coordinates_exact.py`, `test_coordinates_general_affine_formal.py`, `test_coordinates_general_classical.py`, `test_coordinates_general_combinatorial.py`, and 4 more in the machine-readable ledger. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.11.10 komorebi_math.coordinates.families

Purpose and source. Named and custom recurrence coordinate families. It is an ordinary module in the coordinates family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/coordinates/families.py contains 238 lines and 8,746 bytes. Source SHA-256 begins 13eb428d864cbd31.

Surface and ownership. API classification: API-manifested implementation owner. It owns 14 manifest cards: 0 stable, 0 beta, and 14 research. Linked cards: get_recurrence_model, recurrence_model_catalog, RecurrenceModel, custom_recurrence_model, fibonacci_recurrence_model, get_recurrence_model, jacobsthal_recurrence_model, lucas_recurrence_model, padovan_recurrence_model, pell_recurrence_model, and 4 more in the machine-readable ledger. Its __all__ declaration contains 11 names: custom_recurrence_model, fibonacci_recurrence_model, get_recurrence_model, jacobsthal_recurrence_model, lucas_recurrence_model, padovan_recurrence_model, pell_recurrence_model, recurrence_model_catalog, RecurrenceModel, sequence_coordinates_between_models, and 1 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (5): komorebi_math.coordinates.basis, komorebi_math.coordinates.exact, komorebi_math.coordinates.recurrence, komorebi_math.coordinates.transform, komorebi_math.exceptions. Reverse package dependents (5): komorebi_math, komorebi_math.cli.main, komorebi_math.coordinates, komorebi_math.coordinates.portable, komorebi_math.verify.coordinates. Examples (1): recurrence_state_coordinate_atlas.py. Tests or test support files (2): test_coordinates_general_families_state.py, test_coordinates_general_persistence.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.11.11 komorebi_math.coordinates.formal

Purpose and source. Exact formal changes of variable and finite jet coordinates. It is an ordinary module in the coordinates family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/coordinates/formal.py contains 275 lines and 11,000 bytes. Source SHA-256 begins 2c12755281dc4433.

Surface and ownership. API classification: API-manifested implementation owner. It owns 7 manifest cards: 2 stable, 0 beta, and 5 research. Linked cards: formal_coordinate_from_iterable, FormalCoordinateChange, FormalCoordinateChange, compose_ordinary_series, formal_coordinate_from_iterable, FormalCoordinateChange, identity_formal_coordinate. Its __all__ declaration contains 4 names: compose_ordinary_series, formal_coordinate_from_iterable, FormalCoordinateChange, identity_formal_coordinate.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (5): komorebi_math.certificates, komorebi_math.coordinates.budget, komorebi_math.coordinates.exact, komorebi_math.coordinates.transform, komorebi_math.exceptions. Reverse package dependents (7): komorebi_math, komorebi_math.cli.main, komorebi_math.coordinates, komorebi_math.coordinates.portable, komorebi_math.golden, komorebi_math.stable, and 1 more in the machine-readable ledger. Examples (1): formal_affine_coordinate_changes.py. Tests or test support files (2): test_coordinates_general_affine_formal.py, test_coordinates_general_persistence.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.11.12 komorebi_math.coordinates.frame_algebra

Purpose and source. Finite coefficient-space closure for analysis-synthesis systems. It is an ordinary module in the coordinates family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/coordinates/frame_algebra.py contains 1,024 lines and 41,467 bytes. Source SHA-256 begins cb9f91235c866b83.

Surface and ownership. API classification: API-manifested implementation owner. It owns 12 manifest cards: 0 stable, 0 beta, and 12 research. Linked cards: AnalysisSynthesisCoefficientBlocks, AnalysisSynthesisGauge, AnalysisSynthesisMap, CoefficientPartition, ExactAnalysisSynthesisMap, ExactAnalysisSynthesisSystem, AnalysisSynthesisCoefficientBlocks, AnalysisSynthesisGauge, AnalysisSynthesisMap, CoefficientPartition, and 2 more in the machine-readable ledger. Its __all__ declaration contains 6 names: AnalysisSynthesisCoefficientBlocks, AnalysisSynthesisGauge, AnalysisSynthesisMap, CoefficientPartition, ExactAnalysisSynthesisMap, ExactAnalysisSynthesisSystem.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (4): `komorebi_math.certificates`, `komorebi_math.coordinates.analysis_synthesis`, `komorebi_math.coordinates.exact`, `komorebi_math.exceptions`. Reverse package dependents (3): `komorebi_math`, `komorebi_math.coordinates`, `komorebi_math.verify.frame_algebra`. Examples (0): none. Tests or test support files (1): `test_block18_frame_algebra.py`. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.11.13 `komorebi_math.coordinates.modular`

Purpose and source. Coordinate transforms over finite prime fields. It is an ordinary module in the `coordinates` family. The ledger classifies it as a public implementation owner. Source `src/komorebi_math/coordinates/modular.py` contains 249 lines and 9,569 bytes. Source SHA-256 begins `529be182bb6f4436`.

Surface and ownership. API classification: API-manifested implementation owner. It owns 3 manifest cards: 0 stable, 0 beta, and 3 research. Linked cards: FiniteFieldCoordinateTransform, FiniteFieldCoordinateTransform, reduce_exact_transform_mod_prime. Its `__all__` declaration contains 4 names: `FiniteFieldCoordinateTransform`, `ModularMatrix`, `ModularVector`, `reduce_exact_transform_mod_prime`.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (4): `komorebi_math.certificates`, `komorebi_math.coordinates.exact`, `komorebi_math.coordinates.transform`, `komorebi_math.exceptions`. Reverse package dependents (4): `komorebi_math`, `komorebi_math.coordinates`, `komorebi_math.coordinates.portable`, `komorebi_math.verify.coordinates`. Examples (0): none. Tests or test support files (2): `test_coordinates_general_persistence.py`, `test_coordinates_general_tensor_modular.py`. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.11.14 `komorebi_math.coordinates.normalized`

Purpose and source. Coordinate changes for normalized monomial generating functions. It is an ordinary module in the `coordinates` family. The ledger classifies it as a public implementation owner. Source `src/komorebi_math/coordinates/normalized.py` contains 120 lines and 3,376 bytes. Source SHA-256 begins `e92de85740e231a3`.

Surface and ownership. API classification: API-manifested implementation owner. It owns 4 manifest cards: 0 stable, 0 beta, and 4 research. Linked cards: normalization_change_transform, apply_lowering_polynomial, convert_normalized_coefficients, normalization_change_transform. Its `__all__` declaration contains 7 names: `apply_lowering_polynomial`, `convert_normalized_coefficients`, `exponential_basis`, `fibonacci_factorial_basis`, `normalization_change_transform`, `NormalizedBasis`, `ordinary_basis`.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (6): `komorebi_math.coordinates.budget`, `komorebi_math.coordinates.exact`, `komorebi_math.coordinates.transform`, `komorebi_math.exceptions`, `komorebi_math.generating`, `komorebi_math.generating.basis`. Reverse package dependents (5): `komorebi_math`, `komorebi_math.coordinates`, `komorebi_math.coordinates.operators`, `komorebi_math.coordinates.recurrence`, `komorebi_math.verify.coordinates`. Examples (0): none. Tests or test support files (1): `test_coordinates_normalized_persistence.py`. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.11.15 `komorebi_math.coordinates.numeric`

Purpose and source. Numerical spectral and orthogonal coordinate transforms. It is an ordinary module in the `coordinates` family. The ledger classifies it as a public implementation owner. Source `src/komorebi_math/coordinates/numeric.py` contains 352 lines and 13,128 bytes. Source SHA-256 begins `868983da6260f318`.

Surface and ownership. API classification: API-manifested implementation owner. It owns 10 manifest cards: 1 stable, 0 beta, and 9 research. Linked cards: NumericCoordinateTransform, discrete_fourier_transform, NumericCoordinateTransform, discrete_cosine_transform, discrete_fourier_transform, haar_wavelet_transform, modal_coordinate_system, ModalCoordinateSystem, NumericCoordinateTransform, walsh_hadamard_transform. Its `__all__` declaration contains 7 names: `discrete_cosine_transform`, `discrete_fourier_transform`, `haar_wavelet_transform`, `modal_coordinate_system`, `ModalCoordinateSystem`, `NumericCoordinateTransform`, `walsh_hadamard_transform`.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (2): `komorebi_math.certificates`, `komorebi_math.exceptions`. Reverse package dependents (6): `komorebi_math`, `komorebi_math.cli.main`, `komorebi_math.coordinates`, `komorebi_math.coordinates.portable`, `komorebi_math.stable`, `komorebi_math.verify.coordinates`. Examples (1): `spectral_packet_coordinates.py`. Tests or test support files (2): `test_coordinates_general_numeric_packet.py`, `test_coordinates_general_persistence.py`. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.11.16 `komorebi_math.coordinates.operators`

Purpose and source. Polynomial identities for lowering and recurrence defect operators. It is an ordinary module in the `coordinates` family. The ledger classifies it as a public implementation owner. Source `src/komorebi_math/coordinates/operators.py` contains 148 lines and 5,396 bytes. Source SHA-256 begins `339b6623ba6283c9`.

Surface and ownership. API classification: API-manifested implementation owner. It owns 7 manifest cards: 0 stable, 0 beta, and 7 research. Linked cards: bezout_defect_identity, BezoutDefectIdentity, ShiftPolynomial, bezout_defect_identity, BezoutDefectIdentity, polynomial_extended_gcd, ShiftPolynomial. Its `__all__` declaration contains 4 names: `bezout_defect_identity`, `BezoutDefectIdentity`, `polynomial_extended_gcd`, `ShiftPolynomial`.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (5): `komorebi_math.certificates`, `komorebi_math.coordinates.exact`, `komorebi_math.coordinates.normalized`, `komorebi_math.coordinates.polynomials`, `komorebi_math.exceptions`. Reverse package dependents (4): `komorebi_math`, `komorebi_math.coordinates`, `komorebi_math.coordinates.portable`, `komorebi_math.verify.coordinates`. Examples (0): none. Tests or test support files (1): `test_coordinates_operators.py`. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.11.17 `komorebi_math.coordinates.packet`

Purpose and source. Exact packet, residue, and tensor-index coordinate permutations. It is an ordinary module in the `coordinates` family. The ledger classifies it as a public implementation owner. Source `src/komorebi_math/coordinates/packet.py` contains 178 lines and 6,100 bytes. Source SHA-256 begins `e3dd0f89f6266680`.

Surface and ownership. API classification: API-manifested implementation owner. It owns 10 manifest cards: 0 stable, 0 beta, and 10 research. Linked cards: residue_class_transform, chinese_remainder_order, chinese_remainder_transform, digit_reversal_order, digit_reversal_transform, residue_class_order, residue_class_transform, sorted_coordinate_transform, tensor_axis_permutation_order, tensor_axis_permutation_transform. Its `__all__` declaration contains 9 names: `chinese_remainder_order`, `chinese_remainder_transform`, `digit_reversal_order`, `digit_reversal_transform`, `residue_class_order`, `residue_class_transform`, `sorted_coordinate_transform`, `tensor_axis_permutation_order`, `tensor_axis_permutation_transform`.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (3): `komorebi_math.coordinates.basis`, `komorebi_math.coordinates.transform`, `komorebi_math.exceptions`. Reverse package dependents (4): `komorebi_math`, `komorebi_math.cli.main`, `komorebi_math.coordinates`, `komorebi_math.verify.coordinates`. Examples (1): `spectral_packet_coordinates.py`. Tests or test support files (1): `test_coordinates_general_numeric_packet.py`. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.11.18 `komorebi_math.coordinates.polynomials`

Purpose and source. Connection coefficients for graded polynomial families. It is an ordinary module in the `coordinates` family. The ledger classifies it as a public implementation owner. Source `src/komorebi_math/coordinates/polynomials.py` contains 389 lines and 13,184 bytes. Source SHA-256 begins `4b891dbe1541765e`.

Surface and ownership. API classification: API-manifested implementation owner. It owns 26 manifest cards: 1 stable, 0 beta, and 25 research. Linked cards: connection_transform, connection_transform, euler_to_fibonacci_transform, operator_change_of_coordinates, PolynomialFamily, connection_transform, derivative_operator_in_family, euler_polynomial, euler_polynomial_at_zero, euler_polynomial_family, and 16 more in the machine-readable ledger. Its `__all__` declaration contains 21 names: `connection_transform`, `derivative_operator_in_family`,

euler_polynomial, euler_polynomial_at_zero, euler_to_fibonacci_coefficient, euler_to_fibonacci_transform, fibonacci_companion_power, fibonacci_polynomial, fibonacci_polynomial_family, monomial_polynomial_family, and 11 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (4): komorebi_math.coordinates.budget, komorebi_math.coordinates.exact, komorebi_math.coordinates.transform, komorebi_math.exceptions. Reverse package dependents (13): komorebi_math, komorebi_math.cli.main, komorebi_math.coordinates, komorebi_math.coordinates.calculus, komorebi_math.coordinates.classical, komorebi_math.coordinates.operators, and 7 more in the machine-readable ledger. Examples (3): block19_measure_integration_worked_example.py, classical_coordinate_families.py, coordinate_calculus.py. Tests or test support files (5): test_block19_integration.py, test_coordinates_general_calculus.py, test_coordinates_general_classical.py, test_coordinates_operators.py, test_coordinates_polynomials.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.11.19 komorebi_math.coordinates.portable

Purpose and source. Trusted portable codecs for finite coordinate-calculus records. It is an ordinary module in the coordinates family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/coordinates/portable.py contains 722 lines and 23,549 bytes. Source SHA-256 begins 9951aea34982c110.

Surface and ownership. API classification: API-manifested implementation owner. It owns 9 manifest cards: 0 stable, 0 beta, and 9 research. Linked cards: load_coordinate_object, save_coordinate_object, load_coordinate_object, load_coordinate_record, load_transform, register_coordinate_codecs, save_coordinate_object, save_coordinate_record, save_transform. Its __all__ declaration contains 7 names: load_coordinate_object, load_coordinate_record, load_transform, register_coordinate_codecs, save_coordinate_object, save_coordinate_record, save_transform.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (17): komorebi_math.coordinates.affine, komorebi_math.coordinates.analysis_synthesis, komorebi_math.coordinates.basis, komorebi_math.coordinates.budget, komorebi_math.coordinates.calculus, komorebi_math.coordinates.exact, and 11 more in the machine-readable ledger. Reverse package dependents (4): komorebi_math, komorebi_math.coordinates, komorebi_math.export.persistence, komorebi_math.verify.coordinates. Examples (0): none. Tests or test support files (2): test_coordinates_general_persistence.py, test_coordinates_normalized_persistence.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.11.20 komorebi_math.coordinates.realization

Purpose and source. Finite-dimensional realizations, observability, and intertwiners. It is an ordinary module in the coordinates family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/coordinates/realization.py contains 372 lines and 13,139 bytes. Source SHA-256 begins c120c7b5c75b6614.

Surface and ownership. API classification: API-manifested implementation owner. It owns 15 manifest cards: 0 stable, 0 beta, and 15 research. Linked cards: intertwiner_space, IntertwinerSpace, LinearRealization, observability_matrix, recover_initial_state, StateRecovery, cyclic_krylov_matrix, intertwiner_space, IntertwinerSpace, LinearRealization, and 5 more in the machine-readable ledger. Its __all__ declaration contains 9 names: cyclic_krylov_matrix, intertwiner_space, IntertwinerSpace, LinearRealization, observability_matrix, realization_defect, recover_initial_state, similarity_from_cyclic_vectors, StateRecovery.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (5): komorebi_math.certificates, komorebi_math.coordinates.budget, komorebi_math.coordinates.exact, komorebi_math.coordinates.recurrence, komorebi_math.exceptions. Reverse package dependents (4): komorebi_math, komorebi_math.coordinates, komorebi_math.coordinates.portable, komorebi_math.verify.coordinates. Examples (0): none. Tests or test support files (1): test_coordinates_realization.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.11.21 komorebi_math.coordinates.recurrence

Purpose and source. Universal defect coordinates for finite companion recurrences. It is an ordinary module in the coordinates family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/coordinates/recurrence.py contains 386 lines and 12,796 bytes. Source SHA-256 begins 38fe5570bf0b632e.

Surface and ownership. API classification: API-manifested implementation owner. It owns 20 manifest cards: 0 stable, 0 beta, and 20 research. Linked cards: analyze_recurrence_coordinates, companion_matrix, coordinate_change_between_recurrences, euler_fibonacci_coordinates, RecurrenceCoordinateRecord, variation_of_constants_prefix, analyze_recurrence_coordinates, apply_recurrence_defect, companion_matrix, coordinate_change_between_recurrences, and 10 more in the machine-readable ledger. Its __all__ declaration contains 15 names: analyze_recurrence_coordinates, apply_recurrence_defect, companion_matrix, coordinate_change_between_recurrences, defect_green_weights, euler_fibonacci_coordinates, fibonacci_numbers, recurrence_analysis_transform, recurrence_polynomial, recurrence_synthesis_transform, and 5 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (7): komorebi_math.arithmetic.recurrence, komorebi_math.certificates, komorebi_math.coordinates.budget, komorebi_math.coordinates.exact, komorebi_math.coordinates.normalized, komorebi_math.coordinates.transform, and 1 more in the machine-readable ledger. Reverse package dependents (8): komorebi_math, komorebi_math.cli.main, komorebi_math.coordinates, komorebi_math.coordinates.families, komorebi_math.coordinates.portable, komorebi_math.coordinates.realization, and 2 more in the machine-readable ledger. Examples (3): block21_public_capability_tour.py, coordinate_calculus.py, euler_to_fibonacci_coordinates.py. Tests or test support files (4): test_coordinates_normalized_persistence.py, test_coordinates_polynomials.py, test_coordinates_realization.py, test_coordinates_recurrence.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.11.22 komorebi_math.coordinates.state_space

Purpose and source. Exact state-space coordinate changes and canonical finite realizations. It is an ordinary module in the coordinates family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/coordinates/state_space.py contains 340 lines and 12,616 bytes. Source SHA-256 begins dbcf5a293e5a4dd0.

Surface and ownership. API classification: API-manifested implementation owner. It owns 10 manifest cards: 0 stable, 0 beta, and 10 research. Linked cards: StateSpaceModel, certify_state_coordinate_change, controllability_basis, controllability_coordinate_transform, controllability_matrix, observability_block_matrix, observability_coordinate_transform, siso_state_space, StateSpaceCoordinateCertificate, StateSpaceModel. Its __all__ declaration contains 9 names: certify_state_coordinate_change, controllability_basis, controllability_coordinate_transform, controllability_matrix, observability_block_matrix, observability_coordinate_transform, siso_state_space, StateSpaceCoordinateCertificate, StateSpaceModel.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (6): komorebi_math.certificates, komorebi_math.coordinates.basis, komorebi_math.coordinates.exact, komorebi_math.coordinates.tensor, komorebi_math.coordinates.transform, komorebi_math.exceptions. Reverse package dependents (4): komorebi_math, komorebi_math.coordinates, komorebi_math.coordinates.portable, komorebi_math.verify.coordinates. Examples (1): recurrence_state_coordinate_atlas.py. Tests or test support files (2): test_coordinates_general_families_state.py, test_coordinates_general_persistence.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.11.23 komorebi_math.coordinates.tensor

Purpose and source. Transport of operators, forms, and tensor components between bases. It is an ordinary module in the coordinates family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/coordinates/tensor.py contains 229 lines and 7,624 bytes. Source SHA-256 begins 2b882d2e43168d65.

Surface and ownership. API classification: API-manifested implementation owner. It owns 12 manifest cards: 0 stable, 0 beta, and 12 research. Linked cards: tensor_coordinate_transform, contragredient_transform, coordinate_pairing, direct_sum_transform, pairing_invariant, tensor_coordinate_transform, tensor_product_transform, transport_bilinear_form, transport_covector, transport_operator, and 2 more in the machine-readable ledger. Its __

all__ declaration contains 12 names: contragredient_transform, coordinate_pairing, direct_sum_transform, pairing_invariant, tensor_coordinate_transform, tensor_product_transform, transport_bilinear_form, transport_covector, transport_operator, transport_rectangular_operator, and 2 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (3): komorebi_math.coordinates.exact, komorebi_math.coordinates.transform, komorebi_math.exceptions. Reverse package dependents (5): komorebi_math, komorebi_math.coordinates, komorebi_math.coordinates.calculus, komorebi_math.coordinates.state_space, komorebi_math.verify.coordinates. Examples (0): none. Tests or test support files (1): test_coordinates_general_tensor_modular.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.11.24 komorebi_math.coordinates.transform

Purpose and source. Composable exact coordinate transformations. It is an ordinary module in the coordinates family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/coordinates/transform.py contains 154 lines and 5,382 bytes. Source SHA-256 begins 1e02b12f4797006d.

Surface and ownership. API classification: API-manifested implementation owner. It owns 5 manifest cards: 2 stable, 0 beta, and 3 research. Linked cards: CoordinateTransform, ExactCoordinateTransform, CoordinateTransform, CoordinateTransform, ExactCoordinateTransform. Its __all__ declaration contains 2 names: CoordinateTransform, ExactCoordinateTransform.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (3): komorebi_math.certificates, komorebi_math.coordinates.exact, komorebi_math.exceptions. Reverse package dependents (17): komorebi_math, komorebi_math.coordinates, komorebi_math.coordinates.basis, komorebi_math.coordinates.classical, komorebi_math.coordinates.combinatorial, komorebi_math.coordinates.families, and 11 more in the machine-readable ledger. Examples (0): none. Tests or test support files (5): test_block14_cli_api.py, test_block16_analysis_synthesis.py, test_coordinates_general_basis.py, test_coordinates_general_persistence.py, test_coordinates_general_tensor_modular.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.12 Generating: 22 modules

D.2.12.1 komorebi_math.generating

Purpose and source. Normalized generating functions, local masks, and arithmetic compilers. It is the package initializer for the generating family. The ledger classifies it as a canonical API namespace facade. Source src/komorebi_math/generating/__init__.py contains 451 lines and 12,061 bytes. Source SHA-256 begins b936240b6714f457.

Surface and ownership. API classification: canonical public namespace without implementation ownership. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its __all__ declaration contains 181 names: additive_series, algebraic_function, AlgebraicGFGuess, analytic_mahler_product, AnalyticMaskApproximation, apery_numbers, audit_apery_supercongruence, audit_factorial_power_digit_algebra, audit_factorial_power_packet_convolution, audit_generalized_bell_digit_sum, and 171 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (21): komorebi_math.generating.adapters, komorebi_math.generating.basis, komorebi_math.generating.bell, komorebi_math.generating.budget, komorebi_math.generating.cartier, komorebi_math.generating.closed_forms, and 15 more in the machine-readable ledger. Reverse package dependents (6): komorebi_math, komorebi_math.cli.main, komorebi_math.coordinates.normalized, komorebi_math.golden, komorebi_math.stable, komorebi_math.verify.generating. Examples (3): custom_generating_functions.py, mahler_cartier_compilers.py, prime_power_generating.py. Tests or test support files (6): test_block13_cli_api.py, test_block19_integration.py, test_generating_block13.py, test_generating_cartier_padic.py, test_generating_domains.py, and 1 more in the machine-readable ledger. Documentation files with literal module or public-route references (6): API_COMPATIBILITY_0.13.md, API_INVENTORY.md, BLOCK_13_RELEASE.md, GENERAL_COORDINATE_ATLAS.md, NORMALIZED_GENERATI

NG_FUNCTIONS.md, and 1 more in the machine-readable ledger. AST parsing and runtime import both passed in the recorded module audit.

D.2.12.2 komorebi_math.generating.adapters

Purpose and source. Explicit bridges between formal generating functions and finite signals. It is an ordinary module in the generating family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/generating/adapters.py contains 155 lines and 5,553 bytes. Source SHA-256 begins 8a0a1f837ec254ad.

Surface and ownership. API classification: API-manifested implementation owner. It owns 4 manifest cards: 0 stable, 0 beta, and 4 research. Linked cards: fir_taps_to_series, FIRSeriesAdapter, series_to_fir_adapter, series_to_fir_taps. Its __all__ declaration contains 4 names: fir_taps_to_series, FIRSeriesAdapter, series_to_fir_adapter, series_to_fir_taps.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (5): komorebi_math.certificates, komorebi_math.exceptions, komorebi_math.generating.basis, komorebi_math.generating.series, komorebi_math.signals.filtering. Reverse package dependents (2): komorebi_math.generating, komorebi_math.verify.generating. Examples (0): none. Tests or test support files (1): test_generating_block13.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.12.3 komorebi_math.generating.basis

Purpose and source. Exact normalized monomial bases for formal generating functions. It is an ordinary module in the generating family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/generating/basis.py contains 448 lines and 15,462 bytes. Source SHA-256 begins 09829d076c6b799c.

Surface and ownership. API classification: API-manifested implementation owner. It owns 23 manifest cards: 2 stable, 0 beta, and 21 research. Linked cards: factorial_power_basis, ordinary_generating_basis, factorial_power_basis, NormalizedBasis, ordinary_basis, WeightedMonomialBasis, exponential_basis, fibonacci_factorial_basis, NormalizedBasis, ordinary_basis, and 13 more in the machine-readable ledger. Its __all__ declaration contains 11 names: basis_from_descriptor, basis_from_values, BasisCertificate, BasisDescriptor, exponential_basis, factorial_power_basis, fibonacci_factorial_basis, ordinary_basis, pochhammer_basis, q_factorial_basis, and 1 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (0): none. Reverse package dependents (15): komorebi_math, komorebi_math.coordinates.normalized, komorebi_math.generating, komorebi_math.generating.adapters, komorebi_math.generating.congruences, komorebi_math.generating.finite_series, and 9 more in the machine-readable ledger. Examples (1): custom_generating_functions.py. Tests or test support files (5): test_block19_integration.py, test_coordinates_normalized_persistence.py, test_generating_block13.py, test_generating_domains.py, test_generating_foundation.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.12.4 komorebi_math.generating.bell

Purpose and source. Factorial-power Bell recurrences and digit-sum compilers. It is an ordinary module in the generating family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/generating/bell.py contains 409 lines and 13,426 bytes. Source SHA-256 begins 4c5a85c09768939d.

Surface and ownership. API classification: API-manifested implementation owner. It owns 13 manifest cards: 0 stable, 0 beta, and 13 research. Linked cards: BellParityAutomaton, BellResidueAutomaton, generalized_bell_digit_reduction, bell_period_bound, BellDigitReduction, BellParityAutomaton, BellResidueAutomaton, certify_pointed_subindex_identity, generalized_bell_digit_reduction, generalized_bell_parity, and 3 more in the machine-readable ledger. Its __all__ declaration contains 10 names: bell_period_bound, BellDigitReduction, BellParityAutomaton, BellResidueAutomaton, certify_pointed_subindex_identity, generalized_bell_digit_reduction, generalized_bell_parity, pointed_subindex_polynomial, PointedSubindexCertificate, weighted_bell_digit_reduction.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (5): komorebi_math.certificates, komorebi_math.exceptio

ns, komorebi_math.generating.budget, komorebi_math.generating.congruences, komorebi_math.generating.sequences. Reverse package dependents (4): komorebi_math, komorebi_math.cli.main, komorebi_math.generating, komorebi_math.verify.generating. Examples (1): prime_power_generating.py. Tests or test support files (2): test_block13_cli_api.py, test_generating_domains.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.12.5 komorebi_math.generating.budget

Purpose and source. Explicit resource contracts for generating-function compilation. It is an ordinary module in the generating family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/generating/budget.py contains 147 lines and 5,519 bytes. Source SHA-256 begins 10277f2d7fe126db.

Surface and ownership. API classification: API-manifested implementation owner. It owns 3 manifest cards: 0 stable, 0 beta, and 3 research. Linked cards: GeneratingBudget, DEFAULT_GENERATING_BUDGET, GeneratingBudget. Its __all__ declaration contains 2 names: DEFAULT_GENERATING_BUDGET, GeneratingBudget.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (1): komorebi_math.exceptions. Reverse package dependents (7): komorebi_math, komorebi_math.generating, komorebi_math.generating.bell, komorebi_math.generating.cartier, komorebi_math.generating.finite_series, komorebi_math.generating.masks, and 1 more in the machine-readable ledger. Examples (0): none. Tests or test support files (3): test_block13_cli_api.py, test_generating_block13.py, test_generating_domains.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.12.6 komorebi_math.generating.cartier

Purpose and source. Cartier-section compilers for finite and rational Mahler masks. It is an ordinary module in the generating family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/generating/cartier.py contains 524 lines and 17,934 bytes. Source SHA-256 begins 73ec9bbb8ef2b8d7.

Surface and ownership. API classification: API-manifested implementation owner. It owns 15 manifest cards: 0 stable, 0 beta, and 15 research. Linked cards: compile_rational_mask_automaton, AutomatonCertificate, cartier_degree_bound, CartierAutomaton, CartierCarryEquivalence, CartierLinearRepresentation, certify_cartier_carry_equivalence, compile_cartier_linear_representation, compile_rational_mask_automaton, finite_mask_carry_matrices, and 5 more in the machine-readable ledger. Static analysis found 14 top-level public definitions: AutomatonCertificate, cartier_degree_bound, CartierAutomaton, CartierCarryEquivalence, CartierLinearRepresentation, certify_cartier_carry_equivalence, compile_cartier_linear_representation, compile_rational_mask_automaton, finite_mask_carry_matrices, minimize_cartier_automaton, and 4 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (2): komorebi_math.exceptions, komorebi_math.generating.budget. Reverse package dependents (6): komorebi_math, komorebi_math.cli.main, komorebi_math.generating, komorebi_math.generating.masks, komorebi_math.generating.portable, komorebi_math.verify.generating. Examples (1): mahler_cartier_compilers.py. Tests or test support files (3): test_generating_block13.py, test_generating_cartier_padic.py, test_generating_domains.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.12.7 komorebi_math.generating.closed_forms

Purpose and source. Exact finite-prefix closed-form and recurrence recognition tools. It is an ordinary module in the generating family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/generating/closed_forms.py contains 482 lines and 17,781 bytes. Source SHA-256 begins 129b948581a0799a.

Surface and ownership. API classification: API-manifested implementation owner. It owns 12 manifest cards: 0 stable, 0 beta, and 12 research. Linked cards: AlgebraicGFGuess, characteristic_closed_form, guess_algebraic_gf, guess_hypergeometric_ratio, guess_linear_recurrence, guess_p_recursive, guess_quasipolynomial, HypergeometricGuess, LinearRecurrenceGuess, PRecursiveGuess, and 2 more in the machine-readable ledger. Its __all__ declaration contains 12 names: AlgebraicGFGuess, characteristic_closed_form, guess_algebraic_gf, guess_hypergeometric_ratio, guess_linear_recurrence, guess_p_recursive, guess_quasipolynomial, HypergeometricGuess, LinearRecurrenceGuess, PRecursiveGuess, and 2 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (1): `komorebi_math.exceptions`. Reverse package dependents (1): `komorebi_math.generating`. Examples (0): none. Tests or test support files (1): `test_generating_foundation.py`. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.12.8 `komorebi_math.generating.congruences`

Purpose and source. Finite congruence discovery and exact verification tools. It is an ordinary module in the `generating` family. The ledger classifies it as a public implementation owner. Source `src/komorebi_math/generating/congruences.py` contains 667 lines and 22,248 bytes. Source SHA-256 begins `db2e2d5a38d8ba6a`.

Surface and ownership. API classification: API-manifested implementation owner. It owns 16 manifest cards: 0 stable, 0 beta, and 16 research. Linked cards: audit_apery_supercongruence, audit_factorial_power_packet_convolution, audit_generalized_bell_digit_sum, audit_generalized_lucas, audit_partition_lane, audit_q_lucas, audit_touchard, audit_weighted_generalized_bell_digit_sum, base_digit_sum, berlekamp_massey, and 6 more in the machine-readable ledger. Its `__all__` declaration contains 31 names: `audit_apery_supercongruence`, `audit_factorial_power_packet_convolution`, `audit_generalized_bell_digit_sum`, `audit_generalized_lucas`, `audit_partition_lane`, `audit_q_lucas`, `audit_touchard`, `audit_weighted_generalized_bell_digit_sum`, `base_digit_sum`, `berlekamp_massey`, and 21 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (2): `komorebi_math.generating.basis`, `komorebi_math.generating.sequences`. Reverse package dependents (2): `komorebi_math.generating`, `komorebi_math.generating.bell`. Examples (0): none. Tests or test support files (1): `test_generating_foundation.py`. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.12.9 `komorebi_math.generating.cyclotomic`

Purpose and source. Cyclotomic orbit tests for exact Mahler-product collapse. It is an ordinary module in the `generating` family. The ledger classifies it as a public implementation owner. Source `src/komorebi_math/generating/cyclotomic.py` contains 163 lines and 5,335 bytes. Source SHA-256 begins `4410f5e15f14de2b`.

Surface and ownership. API classification: API-manifested implementation owner. It owns 6 manifest cards: 0 stable, 0 beta, and 6 research. Linked cards: cyclotomic_coboundary, CyclotomicCollapse, normalized_cyclotomic_expression, p_free_orbit, verify_cyclotomic_coboundary, verify_cyclotomic_exponent_relation. Static analysis found 6 top-level public definitions: `cyclotomic_coboundary`, `CyclotomicCollapse`, `normalized_cyclotomic_expression`, `p_free_orbit`, `verify_cyclotomic_coboundary`, `verify_cyclotomic_exponent_relation`.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (1): `komorebi_math.exceptions`. Reverse package dependents (1): `komorebi_math.generating`. Examples (0): none. Tests or test support files (1): `test_generating_cartier_padic.py`. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.12.10 `komorebi_math.generating.digit_algebra`

Purpose and source. Finite digit-kernel convolution laws for modular generating-function products. It is an ordinary module in the `generating` family. The ledger classifies it as a public implementation owner. Source `src/komorebi_math/generating/digit_algebra.py` contains 245 lines and 8,549 bytes. Source SHA-256 begins `858b577e7a69049e`.

Surface and ownership. API classification: API-manifested implementation owner. It owns 6 manifest cards: 0 stable, 0 beta, and 6 research. Linked cards: audit_factorial_power_digit_algebra, base_digits, DigitAlgebraAudit, DigitKernel, factorial_power_digit_kernel, lift_digit_sequence. Its `__all__` declaration contains 6 names: `audit_factorial_power_digit_algebra`, `base_digits`, `DigitAlgebraAudit`, `DigitKernel`, `factorial_power_digit_kernel`, `lift_digit_sequence`.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (0): none. Reverse package dependents (1): `komorebi_`

math.generating. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.12.11 komorebi_math.generating.domains

Purpose and source. Exact coefficient domains for normalized generating functions. It is an ordinary module in the generating family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/generating/domains.py contains 318 lines and 10,294 bytes. Source SHA-256 begins 225007918e1337aa.

Surface and ownership. API classification: API-manifested implementation owner. It owns 9 manifest cards: 0 stable, 0 beta, and 9 research. Linked cards: CoefficientDomain, domain_from_descriptor, INTEGER_DOMAIN, IntegerDomain, PrimeFieldDomain, RATIONAL_DOMAIN, RationalDomain, ResidueRingDomain, TruncatedPadicDomain. Its __all__ declaration contains 9 names: CoefficientDomain, domain_from_descriptor, INTEGER_DOMAIN, IntegerDomain, PrimeFieldDomain, RATIONAL_DOMAIN, RationalDomain, ResidueRingDomain, TruncatedPadicDomain.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (1): komorebi_math.exceptions. Reverse package dependents (4): komorebi_math.generating, komorebi_math.generating.finite_series, komorebi_math.generating.portable, komorebi_math.verify.generating. Examples (1): custom_generating_functions.py. Tests or test support files (1): test_generating_domains.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.12.12 komorebi_math.generating.finite_series

Purpose and source. Immutable finite normalized series over explicit exact coefficient domains. It is an ordinary module in the generating family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/generating/finite_series.py contains 451 lines and 17,732 bytes. Source SHA-256 begins b31dab73da218f69.

Surface and ownership. API classification: API-manifested implementation owner. It owns 3 manifest cards: 0 stable, 0 beta, and 3 research. Linked cards: FiniteNormalizedSeries, FiniteNormalizedSeries, SeriesTransferCertificate. Its __all__ declaration contains 2 names: FiniteNormalizedSeries, SeriesTransferCertificate.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (5): komorebi_math.certificates, komorebi_math.exceptions, komorebi_math.generating.basis, komorebi_math.generating.budget, komorebi_math.generating.domains. Reverse package dependents (4): komorebi_math, komorebi_math.generating, komorebi_math.generating.portable, komorebi_math.verify.generating. Examples (1): custom_generating_functions.py. Tests or test support files (2): test_block13_cli_api.py, test_generating_domains.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.12.13 komorebi_math.generating.functions

Purpose and source. Declarative custom generating-function specifications for exact finite computation. It is an ordinary module in the generating family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/generating/functions.py contains 346 lines and 12,170 bytes. Source SHA-256 begins 3adfb83e7be377a3.

Surface and ownership. API classification: API-manifested implementation owner. It owns 13 manifest cards: 0 stable, 0 beta, and 13 research. Linked cards: CustomFunctionSpec, FunctionRegistry, algebraic_function, compose_functions, CustomFunctionSpec, euler_product_function, explicit_function, factorial_power_bell_function, FunctionRegistry, hypergeometric_function, and 3 more in the machine-readable ledger. Its __all__ declaration contains 11 names: algebraic_function, compose_functions, CustomFunctionSpec, euler_product_function, explicit_function, factorial_power_bell_function, FunctionRegistry, hypergeometric_function, linear_recurrence_function, rational_function, and 1 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (3): komorebi_math.generating.basis, komorebi_math.generating.sequences, komorebi_math.generating.series. Reverse package dependents (3): komorebi_math, komorebi_math.generating, komorebi_math.generating.portable. Examples (1): custom_generating_functions.py. Tests or test support files (2): test_generating_block13.py, test_generating_foundation.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.12.14 komorebi_math.generating.masks

Purpose and source. Local Mahler masks, exact carry compilers, and prime-power support layers. It is an ordinary module in the generating family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/generating/masks.py contains 1,083 lines and 37,475 bytes. Source SHA-256 begins cd0f5beac88643f1.

Surface and ownership. API classification: API-manifested implementation owner. It owns 31 manifest cards: 0 stable, 0 beta, and 31 research. Linked cards: compile_finite_mask_carry_plan, FiniteMaskCarryPlan, mahler_mask_inverse, mahler_mask_product, analytic_mahler_product, AnalyticMaskApproximation, CarryLayerConvolution, certify_mahler_functional_equation, certify_rational_mask_algebraic_relation, compile_digit_cauchy_product, and 21 more in the machine-readable ledger. Its __all__ declaration contains 27 names: analytic_mahler_product, AnalyticMaskApproximation, CarryLayerConvolution, certify_mahler_functional_equation, certify_rational_mask_algebraic_relation, compile_digit_cauchy_product, compile_finite_mask_carry_plan, DigitCauchyProductPlan, euler_product_collapse, EulerProductCollapse, and 17 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (5): komorebi_math.certificates, komorebi_math.exceptions, komorebi_math.generating.budget, komorebi_math.generating.cartier, komorebi_math.generating.padic_units. Reverse package dependents (6): komorebi_math, komorebi_math.cli.main, komorebi_math.generating, komorebi_math.generating.portable, komorebi_math.golden, komorebi_math.verify.generating. Examples (1): mahler_cartier_compilers.py. Tests or test support files (1): test_generating_block13.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.12.15 komorebi_math.generating.monoid

Purpose and source. Finite additive and multiplicative monoid series. It is an ordinary module in the generating family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/generating/monoid.py contains 102 lines and 3,811 bytes. Source SHA-256 begins 6912891418926c36.

Surface and ownership. API classification: API-manifested implementation owner. It owns 5 manifest cards: 0 stable, 0 beta, and 5 research. Linked cards: additive_series, dirichlet_convolution, FiniteMonoidSeries, multiplicative_series, prime_valuation_packets. Its __all__ declaration contains 5 names: additive_series, dirichlet_convolution, FiniteMonoidSeries, multiplicative_series, prime_valuation_packets.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (0): none. Reverse package dependents (1): komorebi_math.generating. Examples (0): none. Tests or test support files (1): test_generating_foundation.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.12.16 komorebi_math.generating.normalization

Purpose and source. Carry-linear valuation profiles and digit-local unit twists. It is an ordinary module in the generating family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/generating/normalization.py contains 194 lines and 6,413 bytes. Source SHA-256 begins c862b1945b622a1a.

Surface and ownership. API classification: API-manifested implementation owner. It owns 6 manifest cards: 0 stable, 0 beta, and 6 research. Linked cards: CarryLinearCertificate, CarryLinearValuation, classify_carry_linear_valuation, digit_factorized_normalized_kernel, digit_unit, local_normalized_kernel. Static analysis found 7 top-level public definitions: base_digits, CarryLinearCertificate, CarryLinearValuation, classify_carry_linear_valuation, digit_factorized_normalized_kernel, digit_unit, local_normalized_kernel.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (2): komorebi_math.exceptions, komorebi_math.generating.padic_units. Reverse package dependents (1): komorebi_math.generating. Examples (0): none. Tests or test support files (1): test_generating_cartier_padic.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.12.17 komorebi_math.generating.padic_units

Purpose and source. Exact prime-power unit-factorial and custom kernel compilers. It is an ordinary module in the generating family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/generating/padic_units.py contains 301 lines and 9,487 bytes. Source SHA-256 begins 3c4e77f1eb3a55f6.

Surface and ownership. API classification: API-manifested implementation owner. It owns 9 manifest cards: 0 stable, 0 beta, and 9 research. Linked cards: PrimePowerKernelCompiler, binomial_valuation, borrow_count, factorial_valuation, integer_valuation, NormalizedKernelResidue, PrimePowerBinomial, PrimePowerKernelCompiler, PrimePowerUnitCompiler. Static analysis found 9 top-level public definitions: binomial_valuation, borrow_count, constant_unit_sequence, factorial_valuation, integer_valuation, NormalizedKernelResidue, PrimePowerBinomial, PrimePowerKernelCompiler, PrimePowerUnitCompiler.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (0): none. Reverse package dependents (8): komorebi_math, komorebi_math.cli.main, komorebi_math.generating, komorebi_math.generating.masks, komorebi_math.generating.normalization, komorebi_math.generating.q_kummer, and 2 more in the machine-readable ledger. Examples (1): prime_power_generating.py. Tests or test support files (2): test_generating_block13.py, test_generating_cartier_padic.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.12.18 komorebi_math.generating.portable

Purpose and source. Trusted finite snapshots and portable codecs for generating-function objects. It is an ordinary module in the generating family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/generating/portable.py contains 495 lines and 18,857 bytes. Source SHA-256 begins 6a6fdf84655de482.

Surface and ownership. API classification: API-manifested implementation owner. It owns 5 manifest cards: 0 stable, 0 beta, and 5 research. Linked cards: freeze_custom_function, FrozenGeneratingFunction, freeze_custom_function, FrozenGeneratingFunction, register_generating_codecs. Its __all__ declaration contains 3 names: freeze_custom_function, FrozenGeneratingFunction, register_generating_codecs.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (10): komorebi_math.exceptions, komorebi_math.export.persistence, komorebi_math.generating.basis, komorebi_math.generating.budget, komorebi_math.generating.cartier, komorebi_math.generating.domains, and 4 more in the machine-readable ledger. Reverse package dependents (3): komorebi_math, komorebi_math.export.persistence, komorebi_math.generating. Examples (1): custom_generating_functions.py. Tests or test support files (1): test_generating_block13.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.12.19 komorebi_math.generating.q_kummer

Purpose and source. Mixed-radix Kummer formulas for integer Gaussian binomial coefficients. It is an ordinary module in the generating family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/generating/q_kummer.py contains 212 lines and 6,258 bytes. Source SHA-256 begins 6160a345c7538e85.

Surface and ownership. API classification: API-manifested implementation owner. It owns 9 manifest cards: 0 stable, 0 beta, and 9 research. Linked cards: q_binomial_valuation_decomposition, gaussian_binomial, multiplicative_order_mod_prime, q_binomial_support, q_binomial_valuation_decomposition, q_binomial_valuation_direct, q_factorial, q_integer, QKummerDecomposition. Static analysis found 8 top-level public definitions: gaussian_binomial, multiplicative_order_mod_prime, q_binomial_support, q_binomial_valuation_decomposition, q_binomial_valuation_direct, q_factorial, q_integer, QKummerDecomposition.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (1): komorebi_math.generating.padic_units. Reverse package dependents (3): komorebi_math, komorebi_math.generating, komorebi_math.verify.generating. Examples (1): prime_power_generating.py. Tests or test support files (1): test_generating_cartier_padic.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.12.20 komorebi_math.generating.sections

Purpose and source. Universal custom exponential sections under the basis lowering operator. It is an ordinary module in the generating family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/generating/sections.py contains 123 lines and 3,590 bytes. Source SHA-256 begins 4b8f10c0f695fa95.

Surface and ownership. API classification: API-manifested implementation owner. It owns 5 manifest cards: 0 stable, 0 beta, and 5 research. Linked cards: audit_section_cycle, custom_exponential, custom_section, root_filter_value, SectionCycleAudit. Its __all__ declaration contains 5 names: audit_section_cycle, custom_exponential, custom_section, root_filter_value, SectionCycleAudit.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (2): komorebi_math.generating.basis, komorebi_math.generating.series. Reverse package dependents (1): komorebi_math.generating. Examples (0): none. Tests or test support files (1): test_generating_foundation.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.12.21 komorebi_math.generating.sequences

Purpose and source. Exact benchmark and custom sequence families. It is an ordinary module in the generating family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/generating/sequences.py contains 268 lines and 8,468 bytes. Source SHA-256 begins f7865546be191e8e.

Surface and ownership. API classification: API-manifested implementation owner. It owns 12 manifest cards: 0 stable, 0 beta, and 12 research. Linked cards: apery_numbers, bell_numbers, bernoulli_numbers, catalan_numbers, central_binomial_numbers, fibonacci_numbers, generalized_bell_numbers, generalized_bell_polynomials, partition_numbers, signed_euler_numbers, and 2 more in the machine-readable ledger. Its __all__ declaration contains 14 names: apery_numbers, bell_numbers, bernoulli_numbers, catalan_numbers, central_binomial_numbers, fibonacci_numbers, finite_differences, generalized_bell_hypergeometric_inner, generalized_bell_numbers, generalized_bell_polynomials, and 4 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (0): none. Reverse package dependents (5): komorebi_math.generating, komorebi_math.generating.bell, komorebi_math.generating.congruences, komorebi_math.generating.functions, komorebi_math.verify.generating. Examples (0): none. Tests or test support files (2): test_generating_domains.py, test_generating_foundation.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.12.22 komorebi_math.generating.series

Purpose and source. Exact truncated formal-series operations in a custom normalized basis. It is an ordinary module in the generating family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/generating/series.py contains 324 lines and 13,500 bytes. Source SHA-256 begins e53da8fbe75432cf.

Surface and ownership. API classification: API-manifested implementation owner. It owns 4 manifest cards: 1 stable, 0 beta, and 3 research. Linked cards: FormalSeries, FormalSeries, FormalSeries, PacketIdentity. Its __all__ declaration contains 2 names: FormalSeries, PacketIdentity.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (1): komorebi_math.generating.basis. Reverse package dependents (11): komorebi_math, komorebi_math.generating, komorebi_math.generating.adapters, komorebi_math.generating.functions, komorebi_math.generating.portable, komorebi_math.generating.sections, and 5 more in the machine-readable ledger. Examples (0): none. Tests or test support files (3): test_block19_integration.py, test_generating_block13.py, test_generating_foundation.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.13 Signals: 21 modules

D.2.13.1 komorebi_math.signals

Purpose and source. Executable multiresolution signal calculus and singular packet tools. It is the package initializer for the signals family. The ledger classifies it as a canonical API namespace facade and owner. Source src/komorebi_math/signals/__init__.py contains 379 lines and 9,229 bytes. Source SHA-256 begins 6f57c41ee2ae91f3.

Surface and ownership. API classification: API-manifested implementation owner. It owns 1 manifest card: 0 stable, 0 beta, and 1 research. Linked cards: status. Its __all__ declaration contains 171 names: AdaptiveFilterResult, AdaptiveMethod, best_basis, BestBasisResult, BiquadKind, BoundaryMode, BreakpointBandResult, canonical_dual_window, certified_overlap_add_convolve, coefficient_packets, and 161 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (22): komorebi_math._planned, komorebi_math.registry, komorebi_math.signals.adaptive, komorebi_math.signals.approximation, komorebi_math.signals.convolution, komorebi_math.signals.denoise, and 16 more in the machine-readable ledger. Reverse package dependents (5): komorebi_math, komorebi_math.qualification, komorebi_math.verify.dsp, komorebi_math.verify.signals, komorebi_math.verify.stabilization. Examples (5): block15_polyphase_refinement.py, block21_public_capability_tour.py, polyphase_filterbank.py, production_dsp.py, singular_packet_recovery.py. Tests or test support files (13): test_block12_cli_api.py, test_block12_denoise_streaming.py, test_block12_recursive_fft.py, test_block12_resampling_adaptive.py, test_block12_spectral_filtering.py, and 8 more in the machine-readable ledger. Documentation files with literal module or public-route references (8): API_COMPATIBILITY_0.12.md, API_COMPATIBILITY_0.13.md, API_INVENTORY.md, BLOCK_12_RELEASE.md, BLOCK_4_RELEASE.md, and 3 more in the machine-readable ledger. AST parsing and runtime import both passed in the recorded module audit.

D.2.13.2 komorebi_math.signals.adaptive

Purpose and source. Finite LMS, normalized LMS, and recursive least-squares adaptation. It is an ordinary module in the signals family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/signals/adaptive.py contains 313 lines and 11,711 bytes. Source SHA-256 begins 2266adb261be7ece.

Surface and ownership. API classification: API-manifested implementation owner. It owns 4 manifest cards: 0 stable, 0 beta, and 4 research. Linked cards: AdaptiveFilterResult, lms_filter, nlms_filter, rls_filter. Its __all__ declaration contains 5 names: AdaptiveFilterResult, AdaptiveMethod, lms_filter, nlms_filter, rls_filter.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (3): komorebi_math.certificates, komorebi_math.exceptions, komorebi_math.provenance. Reverse package dependents (2): komorebi_math.signals, komorebi_math.verify.dsp. Examples (1): production_dsp.py. Tests or test support files (1): test_block12_resampling_adaptive.py. Documentation files with literal module or public-route references (1): BLOCK_12_RELEASE.md. AST parsing and runtime import both passed in the recorded module audit.

D.2.13.3 komorebi_math.signals.approximation

Purpose and source. Fourier approximation and deliberately narrow packet estimators. It is an ordinary module in the signals family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/signals/approximation.py contains 121 lines and 4,692 bytes. Source SHA-256 begins 639eb8b4061599fb.

Surface and ownership. API classification: API-manifested implementation owner. It owns 5 manifest cards: 0 stable, 0 beta, and 5 research. Linked cards: estimate_positive_packet_1d, fft_lowpass_1d, fft_lowpass_2d, otsu_threshold, PacketEstimate1D. Its __all__ declaration contains 5 names: estimate_positive_packet_1d, fft_lowpass_1d, fft_lowpass_2d, otsu_threshold, PacketEstimate1D.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (3): komorebi_math.certificates, komorebi_math.exceptions, komorebi_math.provenance. Reverse package dependents (1): komorebi_math.signals. Examples (0): none. Tests or test support files (1): test_signals_extended.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.13.4 komorebi_math.signals.convolution

Purpose and source. Finite convolution operators and their exact polyphase blocks. It is an ordinary module in the signals family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/signals/convolution.py contains 257 lines and 8,846 bytes. Source SHA-256 begins 2260b1f8345213ad.

Surface and ownership. API classification: API-manifested implementation owner. It owns 9 manifest cards: 0 stable, 0 beta, and 9 research. Linked cards: convolution_matrix, convolution_output_length, convolve_signal, finite_polyphase_blocks, finite_polyphase_filter_kernels, FinitePolyphaseKernel, polyphase_convolution, PolyphaseBlockMatrix, PolyphaseConvolution. Its __all__ declaration contains 8 names: convolution_matrix, convolution_output_length, ConvolutionMode, convolve_signal, finite_polyphase_blocks, polyphase_convolution, PolyphaseBlockMatrix, PolyphaseConvolution.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (4): komorebi_math.certificates, komorebi_math.exceptions, komorebi_math.provenance, komorebi_math.signals.polyphase. Reverse package dependents (3): komorebi_math.signals, komorebi_math.signals.filterbanks, komorebi_math.signals.polyphase. Examples (1): polyphase_filterbank.py. Tests or test support files (2): test_block4_hardening.py, test_signals_polyphase_filterbanks.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.13.5 komorebi_math.signals.denoise

Purpose and source. Finite Haar shrinkage and threshold selection for deterministic denoising. It is an ordinary module in the signals family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/signals/denoise.py contains 373 lines and 15,173 bytes. Source SHA-256 begins f949f18afa533173.

Surface and ownership. API classification: API-manifested implementation owner. It owns 10 manifest cards: 0 stable, 0 beta, and 10 research. Linked cards: denoising_risk_certificate, estimate_white_noise_sigma, haar_decompose, HaarDecomposition, median_absolute_deviation, sure_threshold, threshold_coefficients, universal_threshold, wavelet_denoise, WaveletDenoiseResult. Its __all__ declaration contains 12 names: denoising_risk_certificate, estimate_white_noise_sigma, haar_decompose, HaarDecomposition, median_absolute_deviation, sure_threshold, threshold_coefficients, ThresholdMode, ThresholdRule, universal_threshold, and 2 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (4): komorebi_math.certificates, komorebi_math.exceptions, komorebi_math.provenance, komorebi_math.signals.filterbanks. Reverse package dependents (2): komorebi_math.signals, komorebi_math.verify.dsp. Examples (1): production_dsp.py. Tests or test support files (1): test_block12_denoise_streaming.py. Documentation files with literal module or public-route references (1): BLOCK_12_RELEASE.md. AST parsing and runtime import both passed in the recorded module audit.

D.2.13.6 komorebi_math.signals.edge_recovery

Purpose and source. Certified recovery of separated jumps from finite samples. It is an ordinary module in the signals family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/signals/edge_recovery.py contains 273 lines and 10,275 bytes. Source SHA-256 begins a0d9a34fa46230b7.

Surface and ownership. API classification: API-manifested implementation owner. It owns 6 manifest cards: 0 stable, 0 beta, and 6 research. Linked cards: half_shift_distance, recover_vector_jumps, recover_vector_jumps_subgaussian, smooth_haar_coefficient_bound, step_haar_coefficient, VectorJumpRecoveryResult. Its __all__ declaration contains 6 names: half_shift_distance, JumpRecoveryResult, recover_jumps, recover_jumps_subgaussian, smooth_haar_coefficient_bound, step_haar_coefficient.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (4): komorebi_math.certificates, komorebi_math.exceptions, komorebi_math.provenance, komorebi_math.signals.singular_packets. Reverse package dependents (1): komorebi_math.signals. Examples (0): none. Tests or test support files (1): test_block4_hardening.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.13.7 komorebi_math.signals.fft_convolution

Purpose and source. FFT convolution and bounded-buffer overlap-add execution. It is an ordinary module in the signals family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/signals/fft_convolution.py contains 253 lines and 10,317 bytes. Source SHA-256 begins 3f88cfd9bd289139.

Surface and ownership. API classification: API-manifested implementation owner. It owns 4 manifest cards: 0 stable, 0 beta, and 4 research. Linked cards: certified_overlap_add_convolve, fft_convolve, overlap_add_convolve, OverlapAddConvolver. Its __all__ declaration contains 4 names: certified_overlap_add_convolve, fft_convolve, overlap_add_convolve, OverlapAddConvolver.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (4): komorebi_math.certificates, komorebi_math.exceptions, komorebi_math.provenance, komorebi_math.signals.filtering. Reverse package dependents (2): komorebi_math.signals, komorebi_math.verify.dsp. Examples (1): production_dsp.py. Tests or test support files (1): test_block12_recursive_fft.py. Documentation files with literal module or public-route references (1): BLOCK_12_RELEASE.md. AST parsing and runtime import both passed in the recorded module audit.

D.2.13.8 komorebi_math.signals.filterbanks

Purpose and source. Finite periodic filter banks, frames, and lifting schemes. It is an ordinary module in the signals family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/signals/filterbanks.py contains 444 lines and 18,907 bytes. Source SHA-256 begins 41c241aeaf678fd2.

Surface and ownership. API classification: API-manifested implementation owner. It owns 12 manifest cards: 0 stable, 0 beta, and 12 research. Linked cards: PeriodicFilterBank, FilterBankReport, FiniteFilterBank, haar_analyze, haar_filter_bank, haar_matrix, haar_synthesize, HaarFilterBank, periodic_five_three_lifting, PeriodicFilterBank, and 2 more in the machine-readable ledger. Its __all__ declaration contains 11 names: FilterBankReport, FiniteFilterBank, haar_analyze, haar_filter_bank, haar_matrix, haar_synthesize, HaarFilterBank, periodic_five_three_lifting, PeriodicFilterBank, PeriodicLiftingScheme, and 1 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (4): komorebi_math.certificates, komorebi_math.exceptions, komorebi_math.provenance, komorebi_math.signals.convolution. Reverse package dependents (5): komorebi_math, komorebi_math.signals, komorebi_math.signals.denoise, komorebi_math.signals.wavelets, komorebi_math.verify.signals. Examples (1): polyphase_filterbank.py. Tests or test support files (2): test_signals_extended.py, test_signals_polyphase_filterbanks.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.13.9 komorebi_math.signals.filtering

Purpose and source. Finite FIR design, direct filtering, and stateful streaming execution. It is an ordinary module in the signals family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/signals/filtering.py contains 451 lines and 16,677 bytes. Source SHA-256 begins f694446c17d6f8ff.

Surface and ownership. API classification: API-manifested implementation owner. It owns 11 manifest cards: 0 stable, 0 beta, and 11 research. Linked cards: design_bandpass, design_bandstop, design_fir, design_highpass, design_lowpass, fir_filter, fir_frequency_response, FIRFilter, FrequencyResponse, stream_fir_filter, and 1 more in the machine-readable ledger. Its __all__ declaration contains 13 names: design_bandpass, design_bandstop, design_fir, design_highpass, design_lowpass, fir_filter, fir_frequency_response, FIRFilter, FIRKind, FIRMode, and 3 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (4): komorebi_math.certificates, komorebi_math.exceptions, komorebi_math.provenance, komorebi_math.signals.windows. Reverse package dependents (5): komorebi_math.generating.adapters, komorebi_math.signals, komorebi_math.signals.fft_convolution, komorebi_math.signals.recursive, komorebi_math.verify.dsp. Examples (1): production_dsp.py. Tests or test support files (2): test_block12_resampling_adaptive.py, test_block12_spectral_filtering.py. Documentation files with literal module or public-route references (1): BLOCK_12_RELEASE.md. AST parsing and runtime import both passed in the recorded module audit.

D.2.13.10 komorebi_math.signals.nonlinear

Purpose and source. Nonlinear residue packet identities for finite signal sequences. It is an ordinary module in the signals family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/signals/nonlinear.py contains 186 lines and 8,235 bytes. Source SHA-256 begins 711b02775de75032.

Surface and ownership. API classification: API-manifested implementation owner. It owns 3 manifest cards: 0 stable, 0 beta, and 3 research. Linked cards: coefficient_packets, nonlinear_shift_filter, packet_product. Its __all__ declaration contains 3 names: coefficient_packets, nonlinear_shift_filter, packet_product.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (3): komorebi_math.certificates, komorebi_math.exceptions, komorebi_math.provenance. Reverse package dependents (2): komorebi_math.signals, komorebi_math.verify.signals. Examples (0): none. Tests or test support files (2): test_block4_hardening.py, test_signals_extended.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.13.11 komorebi_math.signals.polyphase

Purpose and source. Finite polyphase coordinates, residue projectors, and transfer kernels. It is an ordinary module in the signals family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/signals/polyphase.py contains 485 lines and 19,299 bytes. Source SHA-256 begins bcfda9ec1c68b4cc.

Surface and ownership. API classification: API-manifested implementation owner. It owns 26 manifest cards: 10 stable, 0 beta, and 16 research. Linked cards: lattice_polyphase_analyze, lattice_polyphase_synthesize, LatticePolyphaseCoordinates, polyphase_roundtrip_certificate, PolyphaseCoordinates, PolyphaseSystem, refinement_children, residue_project, signal_polyphase_analyze, signal_polyphase_synthesize, and 16 more in the machine-readable ledger. Its __all__ declaration contains 15 names: lattice_polyphase_analyze, lattice_polyphase_synthesize, LatticePolyphaseCoordinates, mixed_radix_address, polyphase_analyze, polyphase_convolution, polyphase_filter_kernels, polyphase_roundtrip_certificate, polyphase_synthesize, PolyphaseCoordinates, and 5 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (5): komorebi_math.certificates, komorebi_math.exceptions, komorebi_math.provenance, komorebi_math.refinement.polyphase, komorebi_math.signals.convolution. Reverse package dependents (10): komorebi_math, komorebi_math.qualification, komorebi_math.refinement.polyphase, komorebi_math.refinement.portable, komorebi_math.signals, komorebi_math.signals.convolution, and 4 more in the machine-readable ledger. Examples (3): block15_polyphase_refinement.py, block21_public_capability_tour.py, polyphase_filterbank.py. Tests or test support files (5): test_block15_persistence.py, test_block15_polyphase_refinement.py, test_block4_hardening.py, test_signals_extended.py, test_signals_polyphase_filterbanks.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.13.12 komorebi_math.signals.recovery

Purpose and source. Stable recovery of separated jumps and piecewise polynomial breakpoints. It is an ordinary module in the signals family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/signals/recovery.py contains 301 lines and 11,712 bytes. Source SHA-256 begins be8e22ea519e686a.

Surface and ownership. API classification: API-manifested implementation owner. It owns 11 manifest cards: 0 stable, 0 beta, and 11 research. Linked cards: recover_separated_jumps, BreakpointBandResult, detect_piecewise_polynomial_breaks, finite_difference, haar_lipschitz_bound, haar_step_coefficient, JumpDetection, JumpRecoveryResult, recover_jumps_subgaussian, recover_separated_jumps, and 1 more in the machine-readable ledger. Its __all__ declaration contains 10 names: BreakpointBandResult, detect_piecewise_polynomial_breaks, finite_difference, haar_lipschitz_bound, haar_step_coefficient, JumpDetection, JumpRecoveryResult, recover_jumps_subgaussian, recover_separated_jumps, subgaussian_noise_bound.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (3): komorebi_math.certificates, komorebi_math.exceptions, komorebi_math.provenance. Reverse package dependents (3): komorebi_math, komorebi_math.signals, komorebi_math.verify.signals. Examples (1): singular_packet_recovery.py. Tests or test support files (1): test_signals_recovery.py. Documentation files with literal module or

public-route references (1): PUBLIC_API.md. AST parsing and runtime import both passed in the recorded module audit.

D.2.13.13 komorebi_math.signals.recursive

Purpose and source. Finite recursive digital filters, biquad design, and chunked state transport. It is an ordinary module in the signals family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/signals/recursive.py contains 312 lines and 11,845 bytes. Source SHA-256 begins 1282eca34e17ca83.

Surface and ownership. API classification: API-manifested implementation owner. It owns 6 manifest cards: 0 stable, 0 beta, and 6 research. Linked cards: design_biquad, FilterOutput, lfilter, LinearFilter, stream_lfilter, StreamingLinearFilter. Its __all__ declaration contains 7 names: BiquadKind, design_biquad, FilterOutput, lfilter, LinearFilter, stream_lfilter, StreamingLinearFilter.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (4): komorebi_math.certificates, komorebi_math.exceptions, komorebi_math.provenance, komorebi_math.signals.filtering. Reverse package dependents (2): komorebi_math.signals, komorebi_math.verify.dsp. Examples (1): production_dsp.py. Tests or test support files (1): test_block12_recursive_fft.py. Documentation files with literal module or public-route references (1): BLOCK_12_RELEASE.md. AST parsing and runtime import both passed in the recorded module audit.

D.2.13.14 komorebi_math.signals.resampling

Purpose and source. Finite rational resampling with explicit polyphase execution. It is an ordinary module in the signals family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/signals/resampling.py contains 305 lines and 11,036 bytes. Source SHA-256 begins 051a0a75295f6df6.

Surface and ownership. API classification: API-manifested implementation owner. It owns 10 manifest cards: 0 stable, 0 beta, and 10 research. Linked cards: decimate, design_resample_filter, downsample, interpolate, RationalResamplingPlan, resample_poly, resample_poly_result, resampled_length, ResamplingResult, upsample. Its __all__ declaration contains 10 names: decimate, design_resample_filter, downsample, interpolate, RationalResamplingPlan, resample_poly, resample_poly_result, resampled_length, ResamplingResult, upsample.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (4): komorebi_math.certificates, komorebi_math.exceptions, komorebi_math.provenance, komorebi_math.signals.windows. Reverse package dependents (2): komorebi_math.signals, komorebi_math.verify.dsp. Examples (1): production_dsp.py. Tests or test support files (1): test_block12_resampling_adaptive.py. Documentation files with literal module or public-route references (1): BLOCK_12_RELEASE.md. AST parsing and runtime import both passed in the recorded module audit.

D.2.13.15 komorebi_math.signals.singular_packets

Purpose and source. One dimensional singular packets and exact reconstruction identities. It is an ordinary module in the signals family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/signals/singular_packets.py contains 531 lines and 20,253 bytes. Source SHA-256 begins 856d0f0739890bcb.

Surface and ownership. API classification: API-manifested implementation owner. It owns 16 manifest cards: 0 stable, 0 beta, and 16 research. Linked cards: retained_packet_reconstruct, SingularPacket1D, difference_noise_bound, estimated_packet_reconstruct, overlap_matrix_error, overlap_partition_lp_error, retained_packet_reconstruct, retained_packet_reconstruction, RetainedPacketReconstruction, RetainedPacketResult, and 6 more in the machine-readable ledger. Its __all__ declaration contains 15 names: difference_noise_bound, estimated_packet_reconstruct, finite_difference, overlap_matrix_error, overlap_partition_lp_error, retained_packet_reconstruct, retained_packet_reconstruction, RetainedPacketReconstruction, RetainedPacketResult, scalar_region_error, and 5 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (3): komorebi_math.certificates, komorebi_math.exceptions, komorebi_math.provenance. Reverse package dependents (4): komorebi_math, komorebi_math.signals, komorebi_math.signals.edge_recovery, komorebi_math.verify.signals. Examples (1): singular_packet_recovery.py. Tests or test support files (3): test_block4_hardening.py, test_signals_

extended.py, test_signals_wavelets_singular.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.13.16 komorebi_math.signals.smith

Purpose and source. Cyclic Cayley coordinates used as a secondary signal realization. It is an ordinary module in the signals family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/signals/smith.py contains 55 lines and 2,190 bytes. Source SHA-256 begins 2590d54d968ad654.

Surface and ownership. API classification: API-manifested implementation owner. It owns 4 manifest cards: 0 stable, 0 beta, and 4 research. Linked cards: gamma_from_impedance, impedance_from_gamma, phase_mobius, smith_sensitivity. Its __all__ declaration contains 4 names: gamma_from_impedance, impedance_from_gamma, phase_mobius, smith_sensitivity.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (1): komorebi_math.exceptions. Reverse package dependents (1): komorebi_math.signals. Examples (0): none. Tests or test support files (1): test_signals_extended.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.13.17 komorebi_math.signals.spectral

Purpose and source. Finite Fourier spectra, Welch estimates, cross spectra, and coherence. It is an ordinary module in the signals family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/signals/spectral.py contains 438 lines and 17,134 bytes. Source SHA-256 begins 3368f345b9aaa55e.

Surface and ownership. API classification: API-manifested implementation owner. It owns 6 manifest cards: 0 stable, 0 beta, and 6 research. Linked cards: cross_spectrum, CrossSpectrum, parseval_certificate, periodogram, PowerSpectrum, welch_psd. Its __all__ declaration contains 8 names: cross_spectrum, CrossSpectrum, parseval_certificate, periodogram, PowerSpectrum, SpectralAverage, SpectralScaling, welch_psd.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (5): komorebi_math.certificates, komorebi_math.exceptions, komorebi_math.provenance, komorebi_math.signals.time_frequency, komorebi_math.signals.windows. Reverse package dependents (2): komorebi_math.signals, komorebi_math.verify.dsp. Examples (1): production_dsp.py. Tests or test support files (1): test_block12_spectral_filtering.py. Documentation files with literal module or public-route references (1): BLOCK_12_RELEASE.md. AST parsing and runtime import both passed in the recorded module audit.

D.2.13.18 komorebi_math.signals.streaming

Purpose and source. Stateful chunked short-time Fourier analysis with batch equivalence certificates. It is an ordinary module in the signals family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/signals/streaming.py contains 346 lines and 13,441 bytes. Source SHA-256 begins 906f29ed7a2828ee.

Surface and ownership. API classification: API-manifested implementation owner. It owns 3 manifest cards: 0 stable, 0 beta, and 3 research. Linked cards: stream_stft, StreamingSTFT, StreamingSTFTBatch. Its __all__ declaration contains 3 names: stream_stft, StreamingSTFT, StreamingSTFTBatch.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (5): komorebi_math.certificates, komorebi_math.exceptions, komorebi_math.provenance, komorebi_math.signals.time_frequency, komorebi_math.signals.windows. Reverse package dependents (2): komorebi_math.signals, komorebi_math.verify.dsp. Examples (1): production_dsp.py. Tests or test support files (1): test_block12_denoise_streaming.py. Documentation files with literal module or public-route references (1): BLOCK_12_RELEASE.md. AST parsing and runtime import both passed in the recorded module audit.

D.2.13.19 komorebi_math.signals.time_frequency

Purpose and source. Finite framing, short-time Fourier transforms, and overlap-add synthesis. It is an ordinary module in the signals family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/signals/time_frequency.py contains 493 lines and 18,516 bytes. Source SHA-256 begins e127fadcb3d83e9c.

Surface and ownership. API classification: API-manifested implementation owner. It owns 8 manifest cards: 0 stable, 0 beta, and 8 research. Linked cards: frame_signal, FrameBatch, istft, overlap_add, spectrogram, SpectrogramResult, stft, STFTResult. Its __all__ declaration contains 11 names: BoundaryMode, DetrendMode, frame_signal, FrameBatch, istft, overlap_add, spectrogram, SpectrogramResult, SpectrogramScaling, stft, and 1 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (4): komorebi_math.certificates, komorebi_math.exceptions, komorebi_math.provenance, komorebi_math.signals.windows. Reverse package dependents (4): komorebi_math.signals, komorebi_math.signals.spectral, komorebi_math.signals.streaming, komorebi_math.verify.dsp. Examples (1): production_dsp.py. Tests or test support files (1): test_block12_windows_time_frequency.py. Documentation files with literal module or public-route references (1): BLOCK_12_RELEASE.md. AST parsing and runtime import both passed in the recorded module audit.

D.2.13.20 komorebi_math.signals.wavelets

Purpose and source. Finite Haar packet trees, admissible prunings, and exact best bases. It is an ordinary module in the signals family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/signals/wavelets.py contains 387 lines and 14,348 bytes. Source SHA-256 begins 418c10491f59da24.

Surface and ownership. API classification: API-manifested implementation owner. It owns 11 manifest cards: 0 stable, 0 beta, and 11 research. Linked cards: HaarPacketTree, best_basis, BestBasisResult, entropy_cost, haar_packet_analyze, haar_packet_synthesize, HaarPacketTree, lp_cost, packet_energy, PrunedPacketCoefficients, and 1 more in the machine-readable ledger. Its __all__ declaration contains 12 names: best_basis, BestBasisResult, CostFunction, entropy_cost, haar_packet_analyze, haar_packet_synthesize, HaarPacketTree, lp_cost, packet_energy, PacketPath, and 2 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (4): komorebi_math.certificates, komorebi_math.exceptions, komorebi_math.provenance, komorebi_math.signals.filterbanks. Reverse package dependents (3): komorebi_math, komorebi_math.signals, komorebi_math.verify.signals. Examples (1): polyphase_filterbank.py. Tests or test support files (1): test_signals_wavelets_singular.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.13.21 komorebi_math.signals.windows

Purpose and source. Deterministic finite windows and overlap-add diagnostics. It is an ordinary module in the signals family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/signals/windows.py contains 317 lines and 11,919 bytes. Source SHA-256 begins fa18b3d5c0aa1747.

Surface and ownership. API classification: API-manifested implementation owner. It owns 6 manifest cards: 0 stable, 0 beta, and 6 research. Linked cards: canonical_dual_window, get_window, overlap_add_certificate, overlap_profile, window_metrics, WindowMetrics. Its __all__ declaration contains 8 names: canonical_dual_window, get_window, overlap_add_certificate, overlap_profile, window_metrics, WindowMetrics, WindowName, WindowSpec.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (3): komorebi_math.certificates, komorebi_math.exceptions, komorebi_math.provenance. Reverse package dependents (7): komorebi_math.signals, komorebi_math.signals.filtering, komorebi_math.signals.resampling, komorebi_math.signals.spectral, komorebi_math.signals.streaming, komorebi_math.signals.time_frequency, and 1 more in the machine-readable ledger. Examples (0): none. Tests or test support files (1): test_block12_windows_time_frequency.py. Documentation files with literal module or public-route references (1): BLOCK_12_RELEASE.md. AST parsing and runtime import both passed in the recorded module audit.

D.2.14 Nonlinear: 22 modules

D.2.14.1 komorebi_math.nonlinear

Purpose and source. Finite nonlinear packet calculus, obstruction theory, and smooth transport. It is the package initializer for the nonlinear family. The ledger classifies it as a canonical API namespace facade and owner. Source src/komorebi_math/nonlinear/__init__.py contains 347 lines and 9,344 bytes. Source SHA-256 begins 2d8d2b30 b1dace1f.

Surface and ownership. API classification: API-manifested implementation owner. It owns 2 manifest cards: 0 stable, 0 beta, and 2 research. Linked cards: require, status. Its __all__ declaration contains 149 names: autodiff_provider, automatic_derivative_provider, AutomaticDifferentiationProvider, available_ad_frameworks, available_autodiff_backends, build_lift_tree, CallableDerivativeProvider, capture_packet_jet_stream, catalan_number, compile_packet_jet_plan, and 139 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (22): komorebi_math.nonlinear.autodiff, komorebi_math.nonlinear.composition, komorebi_math.nonlinear.coordinates, komorebi_math.nonlinear.correction, komorebi_math.nonlinear.defects, komorebi_math.nonlinear.derivatives, and 16 more in the machine-readable ledger. Reverse package dependents (7): komorebi_math, komorebi_math.cli.main, komorebi_math.langlands.local.bridge, komorebi_math.verify.nonlinear, komorebi_math.verify.nonlinear_advanced, komorebi_math.verify.scalable_nonlinear, and 1 more in the machine-readable ledger. Examples (9): block11_semantic_profile.py, filtered_transport.py, nonlinear_jets_composition.py, nonlinear_trees_elimination.py, obstruction_lifting.py, and 4 more in the machine-readable ledger. Tests or test support files (20): optional_backend_support.py, test_block10_execution_persistence.py, test_block10_sparse_autodiff.py, test_block11_cli.py, test_block11_golden_profiling_beta.py, and 15 more in the machine-readable ledger. Documentation files with literal module or public-route references (11): API_COMPATIBILITY_0.11.md, API_COMPATIBILITY_0.12.md, API_COMPATIBILITY_0.13.md, API_INVENTORY.md, API_STABILITY_REVIEW.md, and 6 more in the machine-readable ledger. AST parsing and runtime import both passed in the recorded module audit.

D.2.14.2 komorebi_math.nonlinear.autodiff

Purpose and source. Optional automatic-differentiation adapters for directional derivatives. It is an ordinary module in the nonlinear family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/nonlinear/autodiff.py contains 728 lines and 26,563 bytes. Source SHA-256 begins dae6aa596e657ae5.

Surface and ownership. API classification: API-manifested implementation owner. It owns 16 manifest cards: 0 stable, 0 beta, and 16 research. Linked cards: autodiff_provider, automatic_derivative_provider, AutomaticDifferentiationProvider, available_ad_frameworks, available_autodiff_backends, DirectionalJVP, JaxDerivativeProvider, TorchDerivativeProvider, autodiff_provider, automatic_derivative_provider, and 6 more in the machine-readable ledger. Its __all__ declaration contains 9 names: ad_framework_diagnostics, autodiff_provider, automatic_derivative_provider, AutomaticDifferentiationProvider, available_ad_frameworks, available_autodiff_backends, DirectionalJVP, JaxDerivativeProvider, TorchDerivativeProvider.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (3): komorebi_math.certificates, komorebi_math.exceptions, komorebi_math.provenance. Reverse package dependents (5): komorebi_math, komorebi_math.nonlinear, komorebi_math.qualification, komorebi_math.support, komorebi_math.verify.scalable_nonlinear. Examples (0): none. Tests or test support files (4): optional_backend_support.py, test_block10_sparse_autodiff.py, test_nonlinear_sparse_autodiff.py, test_optional_backend_probe.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.14.3 komorebi_math.nonlinear.composition

Purpose and source. Higher chain rules and packet Faà di Bruno composition. It is an ordinary module in the nonlinear family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/nonlinear/composition.py contains 447 lines and 18,130 bytes. Source SHA-256 begins 56bf1bc5b8f5a2b6.

Surface and ownership. API classification: API-manifested implementation owner. It owns 11 manifest cards: 0 stable, 0 beta, and 11 research. Linked cards: ComposedDerivativeProvider, faa_di_bruno, packet_faa_di_bruno, ComposedDerivativeProvider, faa_di_bruno, FaaDiBrunoResult, FaaDiBrunoTerm, labeled_composition_term_count, packet_faa_di_bruno, set_partitions, and 1 more in the machine-readable ledger. Its __all__ declaration contains 9 names: ComposedDerivativeProvider, faa_di_bruno, FaaDiBrunoResult, FaaDiBrunoTerm, labeled_composition_term_count, packet_faa_di_bruno, Partition, set_partitions, stirling_second.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (6): komorebi_math.certificates, komorebi_math.core.packets, komorebi_math.exceptions, komorebi_math.nonlinear.derivatives, komorebi_math.nonlinear.jets, komorebi_math.provenance. Reverse package dependents (3): komorebi_math, komorebi_math.nonlinear, komorebi_math.verify.nonlinear_advanced. Examples (1): nonlinear_jets_composition.py. Tests or test support files (1): test_nonlinear_jets_composition.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.14.4 komorebi_math.nonlinear.coordinates

Purpose and source. Exact coordinate conjugation for nonlinear maps. It is an ordinary module in the nonlinear family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/nonlinear/coordinates.py contains 110 lines and 5,073 bytes. Source SHA-256 begins b99aba8af7fd53d3.

Surface and ownership. API classification: API-manifested implementation owner. It owns 2 manifest cards: 0 stable, 0 beta, and 2 research. Linked cards: NonlinearPacketMap, NonlinearPacketMap. Its __all__ declaration contains 1 name: NonlinearPacketMap.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (4): komorebi_math.certificates, komorebi_math.core.packets, komorebi_math.exceptions, komorebi_math.provenance. Reverse package dependents (3): komorebi_math, komorebi_math.nonlinear, komorebi_math.verify.nonlinear. Examples (0): none. Tests or test support files (1): test_nonlinear_coordinates_defects.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.14.5 komorebi_math.nonlinear.correction

Purpose and source. Quantitative nonlinear correction and finite Lyapunov-Schmidt reduction. It is an ordinary module in the nonlinear family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/nonlinear/correction.py contains 538 lines and 24,456 bytes. Source SHA-256 begins 6ae92fabdd5d87fc.

Surface and ownership. API classification: API-manifested implementation owner. It owns 10 manifest cards: 0 stable, 0 beta, and 10 research. Linked cards: HilbertObstructionProjector, LyapunovSchmidtReduction, RightInverseCorrectionProblem, HilbertObstructionProjector, HilbertObstructionResult, LyapunovSchmidtDecomposition, LyapunovSchmidtPoint, LyapunovSchmidtReduction, RightInverseCorrectionProblem, RightInverseCorrectionResult. Its __all__ declaration contains 7 names: HilbertObstructionProjector, HilbertObstructionResult, LyapunovSchmidtDecomposition, LyapunovSchmidtPoint, LyapunovSchmidtReduction, RightInverseCorrectionProblem, RightInverseCorrectionResult.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (4): komorebi_math.certificates, komorebi_math.exceptions, komorebi_math.nonlinear.derivatives, komorebi_math.provenance. Reverse package dependents (3): komorebi_math, komorebi_math.nonlinear, komorebi_math.verify.nonlinear_advanced. Examples (1): nonlinear_trees_elimination.py. Tests or test support files (1): test_nonlinear_correction_elimination.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.14.6 komorebi_math.nonlinear.defects

Purpose and source. Nonlinear refinement defects and stability bounds. It is an ordinary module in the nonlinear family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/nonlinear/defects.py contains 131 lines and 5,032 bytes. Source SHA-256 begins c035ec4f6ea1dcee.

Surface and ownership. API classification: API-manifested implementation owner. It owns 2 manifest cards: 0 stable, 0 beta, and 2 research. Linked cards: NonlinearRefinementDefect, NonlinearRefinementDefect. Its __all__ declaration contains 1 name: NonlinearRefinementDefect.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (3): komorebi_math.certificates, komorebi_math.exceptions, komorebi_math.provenance. Reverse package dependents (3): komorebi_math, komorebi_math.nonlinear, komorebi_math.verify.nonlinear. Examples (0): none. Tests or test support files (1): test_nonlinear_coordinates_defects.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.14.7 komorebi_math.nonlinear.derivatives

Purpose and source. Finite-dimensional derivative providers for nonlinear packet calculus. It is an ordinary module in the nonlinear family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/nonlinear/derivatives.py contains 549 lines and 24,044 bytes. Source SHA-256 begins 2d2084098d5a2cb3.

Surface and ownership. API classification: API-manifested implementation owner. It owns 13 manifest cards: 0 stable, 4 beta, and 9 research. Linked cards: CallableDerivativeProvider, DerivativeProvider, FiniteDifferenceDerivativeProvider, PolynomialDerivativeProvider, CallableDerivativeProvider, FiniteDifferenceDerivativeProvider, PolynomialDerivativeProvider, SympyDerivativeProvider, CallableDerivativeProvider, DerivativeProvider, and 3 more in the machine-readable ledger. Its __all__ declaration contains 6 names: CallableDerivativeProvider, DerivativeProvider, DirectionalDerivative, FiniteDifferenceDerivativeProvider, PolynomialDerivativeProvider, SympyDerivativeProvider.

Dependencies and evidence. Direct dependency profile: optional exact extra, required NumPy dependency. Direct package dependencies (3): komorebi_math.certificates, komorebi_math.exceptions, komorebi_math.provenance. Reverse package dependents (13): komorebi_math, komorebi_math.beta, komorebi_math.golden, komorebi_math.nonlinear, komorebi_math.nonlinear.composition, komorebi_math.nonlinear.correction, and 7 more in the machine-readable ledger. Examples (7): block11_semantic_profile.py, nonlinear_jets_composition.py, nonlinear_trees_elimination.py, portable_nonlinear_state.py, resumable_packet_jet_stream.py, and 2 more in the machine-readable ledger. Tests or test support files (10): test_block10_execution_persistence.py, test_block11_cli.py, test_block11_golden_profiling_beta.py, test_block11_streaming_migrations.py, test_nonlinear_correction_elimination.py, and 5 more in the machine-readable ledger. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.14.8 komorebi_math.nonlinear.elimination

Purpose and source. Finite nonlinear Schur elimination through second order. It is an ordinary module in the nonlinear family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/nonlinear/elimination.py contains 496 lines and 20,155 bytes. Source SHA-256 begins eb257c3e02a55333.

Surface and ownership. API classification: API-manifested implementation owner. It owns 5 manifest cards: 0 stable, 0 beta, and 5 research. Linked cards: NonlinearSchurElimination, HiddenSolveResult, NonlinearSchurElimination, SchurDerivativeResult, SchurSecondDerivativeResult. Its __all__ declaration contains 4 names: HiddenSolveResult, NonlinearSchurElimination, SchurDerivativeResult, SchurSecondDerivativeResult.

Dependencies and evidence. Direct dependency profile: optional exact extra, required NumPy dependency. Direct package dependencies (4): komorebi_math.certificates, komorebi_math.exceptions, komorebi_math.nonlinear.derivatives, komorebi_math.provenance. Reverse package dependents (3): komorebi_math, komorebi_math.nonlinear, komorebi_math.verify.nonlinear_advanced. Examples (1): nonlinear_trees_elimination.py. Tests or test support files (1): test_nonlinear_correction_elimination.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.14.9 komorebi_math.nonlinear.execution

Purpose and source. Projection-cached packet-jet execution and optional CuPy kernels. It is an ordinary module in the nonlinear family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/nonlinear/execution.py contains 696 lines and 27,704 bytes. Source SHA-256 begins 474902031dcfa406.

Surface and ownership. API classification: API-manifested implementation owner. It owns 29 manifest cards: 0 stable, 7 beta, and 22 research. Linked cards: compile_packet_jet_plan, execute_packet_jet_plan, execute_packet_jets, PacketJetBatchResult, PacketJetEvaluation, PacketJetExecutionPlan, PacketJetInputGroup, compile_packet_jet_plan, CuPyDeviceDerivativeProvider, CuPyPacketJetExecutor, and 19 more in the machine-readable ledger. Its __all__ declaration contains 15 names: compile_packet_jet_plan, CuPyDeviceDerivativeProvider, CupyDeviceDerivativeProvider, CupyPacketJetExecutor, CuPyPacketJetExecutor, CuPyPolynomialDerivativeProvider, CupyPolynomialDerivativeProvider, execute_packet_jet_plan, execute_packet_jets, nonlinear_cupy_available, and 5 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: optional CuPy accelerator outside the declared project extras, required NumPy dependency. Direct package dependencies (6): komorebi_math.algorithms.benchmarks, komorebi_math.certificates, komorebi_math.exceptions, komorebi_math.nonlinear.derivatives, komorebi_math.nonlinear.jets, komorebi_math.provenance. Reverse package dependents (6): komorebi_math, komorebi_math.beta, komorebi_math.golden, komorebi_math.nonlinear, komorebi_math.nonlinear.profiling, komorebi_math.verify.scalable_nonlinear. Examples (2): scalable_nonlinear.py, scalable_nonlinear_runtime.py. Tests or test support files (2): test_block10_execution_persistence.py, test_nonlinear_execution.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.14.10 komorebi_math.nonlinear.filtered

Purpose and source. Finite filtered obstructions, lift completion, and spectral-sequence records. It is an ordinary module in the nonlinear family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/nonlinear/filtered.py contains 844 lines and 36,672 bytes. Source SHA-256 begins 9209ff4d05afcaf0.

Surface and ownership. API classification: API-manifested implementation owner. It owns 16 manifest cards: 0 stable, 0 beta, and 16 research. Linked cards: conditional_projective_limit_completion, FilteredCorrectionProblem, FilteredVectorSpace, FiniteFilteredCochainComplex, verify_finite_filtered_branch, conditional_projective_limit_completion, DepthObstructionResult, FilteredCorrectionProblem, FilteredSpectralSequenceResult, FilteredVectorSpace, and 6 more in the machine-readable ledger. Its __all__ declaration contains 11 names: conditional_projective_limit_completion, DepthObstructionResult, FilteredCorrectionProblem, FilteredSpectralSequenceResult, FilteredVectorSpace, FiniteFilteredBranchResult, FiniteFilteredCochainComplex, ProjectiveLimitRecord, SpectralSequenceEntry, SpectralSequencePage, and 1 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (5): komorebi_math._number_theory, komorebi_math.certificates, komorebi_math.exceptions, komorebi_math.nonlinear.lifting, komorebi_math.provenance. Reverse package dependents (4): komorebi_math, komorebi_math.export.persistence, komorebi_math.nonlinear, komorebi_math.verify.nonlinear_advanced. Examples (1): filtered_transport.py. Tests or test support files (3): test_block10_execution_persistence.py, test_nonlinear_filtered.py, test_nonlinear_snapshots_persistence.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.14.11 komorebi_math.nonlinear.gpu

Purpose and source. Optional CuPy execution for persisted packet-jet snapshots. It is an ordinary module in the nonlinear family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/nonlinear/gpu.py contains 205 lines and 7,670 bytes. Source SHA-256 begins 2a4ffa8606b9b8ea.

Surface and ownership. API classification: API-manifested implementation owner. It owns 12 manifest cards: 0 stable, 0 beta, and 12 research. Linked cards: cupy_runtime_status, CuPyPacketJetSnapshotExecutor, CuPyRuntimeStatus, CuPySnapshotExecutor, CuPyValidationResult, validate_cupy_snapshot, cupy_runtime_status, CuPyPacketJetSnapshotExecutor, CuPyRuntimeStatus, CuPySnapshotExecutor, and 2 more in the machine-readable ledger. Its __all__ declaration contains 6 names: cupy_runtime_status, CuPyPacketJetSnapshotExecutor, CuPyRuntimeStatus, CuPySnapshotExecutor, CuPyValidationResult, validate_cupy_snapshot.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (4): `komorebi_math.certificates`, `komorebi_math.exceptions`, `komorebi_math.nonlinear.snapshots`, `komorebi_math.provenance`. Reverse package dependents (3): `komorebi_math`, `komorebi_math.nonlinear`, `komorebi_math.verify.scalable_nonlinear`. Examples (1): `scalable_nonlinear.py`. Tests or test support files (2): `test_block10_execution_persistence.py`, `test_nonlinear_execution.py`. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.14.12 `komorebi_math.nonlinear.graded`

Purpose and source. Finite graded nonlinear operations and packet Maurer-Cartan equations. It is an ordinary module in the `nonlinear` family. The ledger classifies it as a public implementation owner. Source `src/komorebi_math/nonlinear/graded.py` contains 633 lines and 25,862 bytes. Source SHA-256 begins `f751b044f309565a`.

Surface and ownership. API classification: API-manifested implementation owner. It owns 16 manifest cards: 0 stable, 0 beta, and 16 research. Linked cards: divided_square_packets, FiniteAbelianGroup, GradedPacketOperation, higher_maurer_cartan_packets, maurer_cartan_packets, divided_square_packets, DividedSquareResult, FiniteAbelianGroup, GradedConvolutionResult, GradedConvolutionTerm, and 6 more in the machine-readable ledger. Its `__all__` declaration contains 11 names: `divided_square_packets`, `DividedSquareResult`, `FiniteAbelianGroup`, `GradedConvolutionResult`, `GradedConvolutionTerm`, `GradedPacketOperation`, `GradingCertificate`, `higher_maurer_cartan_packets`, `HigherMaurerCartanResult`, `maurer_cartan_packets`, and 1 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (4): `komorebi_math.certificates`, `komorebi_math.core.packets`, `komorebi_math.exceptions`, `komorebi_math.provenance`. Reverse package dependents (3): `komorebi_math`, `komorebi_math.nonlinear`, `komorebi_math.verify.nonlinear_advanced`. Examples (0): none. Tests or test support files (1): `test_nonlinear_graded.py`. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.14.13 `komorebi_math.nonlinear.holonomy`

Purpose and source. Finite matrix transport, gauges, and holonomy certificates. It is an ordinary module in the `nonlinear` family. The ledger classifies it as a public implementation owner. Source `src/komorebi_math/nonlinear/holonomy.py` contains 329 lines and 14,597 bytes. Source SHA-256 begins `0cc4194ed31c8f57`.

Surface and ownership. API classification: API-manifested implementation owner. It owns 6 manifest cards: 0 stable, 0 beta, and 6 research. Linked cards: TransportEdge, TransportGraph, FundamentalHolonomy, IntertwiningResult, TransportEdge, TransportGraph. Its `__all__` declaration contains 4 names: `FundamentalHolonomy`, `IntertwiningResult`, `TransportEdge`, `TransportGraph`.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (4): `komorebi_math.backends`, `komorebi_math.certificates`, `komorebi_math.exceptions`, `komorebi_math.provenance`. Reverse package dependents (4): `komorebi_math`, `komorebi_math.langlands.local.bridge`, `komorebi_math.nonlinear`, `komorebi_math.verify.nonlinear`. Examples (1): `holonomy.py`. Tests or test support files (1): `test_holonomy.py`. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.14.14 `komorebi_math.nonlinear.jets`

Purpose and source. Packet jets, derivative reconstruction, support, and Taylor data. It is an ordinary module in the `nonlinear` family. The ledger classifies it as a public implementation owner. Source `src/komorebi_math/nonlinear/jets.py` contains 515 lines and 21,521 bytes. Source SHA-256 begins `913c9ede0ff13b43`.

Surface and ownership. API classification: API-manifested implementation owner. It owns 16 manifest cards: 0 stable, 5 beta, and 11 research. Linked cards: packet_jet_support, PacketJet, PacketJetAddress, PacketJetFamily, PacketJetSupport, packet_jet_support, packet_taylor, PacketJet, PacketJetFamily, packet_jet_support, and 6 more in the machine-readable ledger. Its `__all__` declaration contains 8 names: `packet_jet_support`, `packet_taylor`, `PacketJet`, `PacketJetAddress`, `PacketJetFamily`, `PacketJetSupport`, `PacketLabel`, `PacketTaylorResult`.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (5): `komorebi_math.certificates`, `komorebi_math.core.packets`, `komorebi_math.exceptions`, `komorebi_math.nonlinea`

r.derivatives, komorebi_math.provenance. Reverse package dependents (14): komorebi_math, komorebi_math.beta, komorebi_math.export.persistence, komorebi_math.golden, komorebi_math.nonlinear, komorebi_math.nonlinear.composition, and 8 more in the machine-readable ledger. Examples (6): block11_semantic_profile.py, nonlinear_jets_composition.py, portable_nonlinear_state.py, resumable_packet_jet_stream.py, scalable_nonlinear.py, and 1 more in the machine-readable ledger. Tests or test support files (8): test_block10_execution_persistence.py, test_block11_cli.py, test_block11_golden_profiling_beta.py, test_block11_streaming_migrations.py, test_nonlinear_execution.py, and 3 more in the machine-readable ledger. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.14.15 komorebi_math.nonlinear.lifting

Purpose and source. Exact finite lifting over prime-power rings. It is an ordinary module in the nonlinear family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/nonlinear/lifting.py contains 480 lines and 18,927 bytes. Source SHA-256 begins d28ba65c3473b900.

Surface and ownership. API classification: API-manifested implementation owner. It owns 13 manifest cards: 0 stable, 0 beta, and 13 research. Linked cards: build_lift_tree, FiniteLiftTree, LinearLiftProblem, PolynomialLiftProblem, solve_mod_prime, build_lift_tree, FiniteLiftTree, LiftStepResult, LiftTreeNode, LinearLiftProblem, and 3 more in the machine-readable ledger. Its __all__ declaration contains 8 names: build_lift_tree, FiniteLiftTree, LiftStepResult, LiftTreeNode, LinearLiftProblem, ModularLinearSolution, PolynomialLiftProblem, solve_mod_prime.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (4): komorebi_math._number_theory, komorebi_math.certificates, komorebi_math.exceptions, komorebi_math.provenance. Reverse package dependents (6): komorebi_math, komorebi_math.export.persistence, komorebi_math.langlands.local.symplectic, komorebi_math.nonlinear, komorebi_math.nonlinear.filtered, komorebi_math.verify.nonlinear. Examples (1): obstruction_lifting.py. Tests or test support files (2): test_block10_execution_persistence.py, test_lifting.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.14.16 komorebi_math.nonlinear.obstruction

Purpose and source. Finite-dimensional cokernel obstruction computations. It is an ordinary module in the nonlinear family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/nonlinear/obstruction.py contains 196 lines and 8,115 bytes. Source SHA-256 begins ed11236a44f5f571.

Surface and ownership. API classification: API-manifested implementation owner. It owns 4 manifest cards: 0 stable, 0 beta, and 4 research. Linked cards: LinearObstructionProblem, LinearObstructionProblem, ObstructionResult, ObstructionSquare. Its __all__ declaration contains 3 names: LinearObstructionProblem, ObstructionResult, ObstructionSquare.

Dependencies and evidence. Direct dependency profile: optional exact extra, required NumPy dependency. Direct package dependencies (4): komorebi_math.backends, komorebi_math.certificates, komorebi_math.exceptions, komorebi_math.provenance. Reverse package dependents (4): komorebi_math, komorebi_math.nonlinear, komorebi_math.unified, komorebi_math.verify.nonlinear. Examples (1): obstruction_lifting.py. Tests or test support files (1): test_nonlinear_obstruction.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.14.17 komorebi_math.nonlinear.profiling

Purpose and source. Operation counts and observational profiles for finite packet-jet runtimes. It is an ordinary module in the nonlinear family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/nonlinear/profiling.py contains 467 lines and 16,848 bytes. Source SHA-256 begins 034aa8498ddafce5.

Surface and ownership. API classification: API-manifested implementation owner. It owns 21 manifest cards: 0 stable, 7 beta, and 14 research. Linked cards: estimate_packet_jet_complexity, PacketJetComplexityEstimate, PacketJetProfileReport, profile_operation, profile_packet_jet_runtime, RuntimeProfileSample, RuntimeStageProfile, estimate_packet_jet_complexity, PacketJetComplexityEstimate, PacketJetProfileReport, and 11 more in the machine-readable ledger. Its __all__ declaration contains 7 names: estimate_packet_jet_complexity, PacketJetComplexityEstimate, PacketJetProfileReport, profile_operation, profile_packet_jet_runtime, RuntimeProfileSample, RuntimeStageProfile.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (8): `komorebi_math.certificates`, `komorebi_math.exceptions`, `komorebi_math.export.persistence`, `komorebi_math.nonlinear.execution`, `komorebi_math.nonlinear.jets`, `komorebi_math.nonlinear.snapshots`, and 2 more in the machine-readable ledger. Reverse package dependents (4): `komorebi_math`, `komorebi_math.beta`, `komorebi_math.nonlinear`, `komorebi_math.verify.streaming_beta`. Examples (1): `block11_semantic_profile.py`. Tests or test support files (1): `test_block11_golden_profiling_beta.py`. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.14.18 `komorebi_math.nonlinear.snapshots`

Purpose and source. Portable finite snapshots of packet-jet blocks. It is an ordinary module in the `nonlinear` family. The ledger classifies it as a public implementation owner. Source `src/komorebi_math/nonlinear/snapshots.py` contains 375 lines and 15,205 bytes. Source SHA-256 begins `bd7a755c4b145b2f`.

Surface and ownership. API classification: API-manifested implementation owner. It owns 8 manifest cards: 0 stable, 3 beta, and 5 research. Linked cards: PacketJetCoefficient, PacketJetSnapshot, snapshot_packet_jet, PacketJetSnapshot, snapshot_packet_jet, PacketJetCoefficient, PacketJetSnapshot, snapshot_packet_jet. Its `__all__` declaration contains 3 names: `PacketJetCoefficient`, `PacketJetSnapshot`, `snapshot_packet_jet`.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (4): `komorebi_math.certificates`, `komorebi_math.exceptions`, `komorebi_math.nonlinear.jets`, `komorebi_math.provenance`. Reverse package dependents (10): `komorebi_math`, `komorebi_math.beta`, `komorebi_math.export.persistence`, `komorebi_math.golden`, `komorebi_math.nonlinear`, `komorebi_math.nonlinear.gpu`, and 4 more in the machine-readable ledger. Examples (2): `portable_nonlinear_state.py`, `scalable_nonlinear.py`. Tests or test support files (4): `test_block10_execution_persistence.py`, `test_block11_cli.py`, `test_block11_streaming_migrations.py`, `test_nonlinear_snapshots_persistence.py`. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.14.19 `komorebi_math.nonlinear.sparse`

Purpose and source. Sparse symmetric derivative tensors and polynomial derivative providers. It is an ordinary module in the `nonlinear` family. The ledger classifies it as a public implementation owner. Source `src/komorebi_math/nonlinear/sparse.py` contains 660 lines and 28,243 bytes. Source SHA-256 begins `f4d37624c5797b83`.

Surface and ownership. API classification: API-manifested implementation owner. It owns 9 manifest cards: 0 stable, 3 beta, and 6 research. Linked cards: SparsePolynomialDerivativeProvider, SparseSymmetricTensor, SparseTensorEntry, SparsePolynomialDerivativeProvider, SparseSymmetricTensor, SparseTensorEntry, SparsePolynomialDerivativeProvider, SparseSymmetricTensor, SparseTensorEntry. Its `__all__` declaration contains 3 names: `SparsePolynomialDerivativeProvider`, `SparseSymmetricTensor`, `SparseTensorEntry`.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (4): `komorebi_math.certificates`, `komorebi_math.exceptions`, `komorebi_math.nonlinear.derivatives`, `komorebi_math.provenance`. Reverse package dependents (6): `komorebi_math`, `komorebi_math.beta`, `komorebi_math.export.persistence`, `komorebi_math.golden`, `komorebi_math.nonlinear`, `komorebi_math.verify.scalable_nonlinear`. Examples (2): `scalable_nonlinear.py`, `scalable_nonlinear_runtime.py`. Tests or test support files (4): `test_block10_execution_persistence.py`, `test_block10_sparse_autodiff.py`, `test_nonlinear_snapshots_persistence.py`, `test_nonlinear_sparse_autodiff.py`. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.14.20 `komorebi_math.nonlinear.streaming`

Purpose and source. Chunked, resumable persistence for finite packet-jet snapshots. It is an ordinary module in the `nonlinear` family. The ledger classifies it as a public implementation owner. Source `src/komorebi_math/nonlinear/streaming.py` contains 1,664 lines and 65,296 bytes. Source SHA-256 begins `b4db5f4d1493421b`.

Surface and ownership. API classification: API-manifested implementation owner. It owns 30 manifest cards: 0 stable, 10 beta, and 20 research. Linked cards: capture_packet_jet_stream, load_packet_jet_snapshot_stream, open_packet_jet_snapshot_stream, PacketJetSnapshotStreamReader, PacketJetStreamChunkRecord, PacketJetStreamManifest, PacketJetStreamSelection, PacketJetStreamWriteResult, save_packet_jet_snapshot_stream,

StreamedPacketJetEvaluation, and 20 more in the machine-readable ledger. Its `__all__` declaration contains 14 names: `capture_packet_jet_stream`, `load_packet_jet_snapshot_stream`, `open_packet_jet_snapshot_stream`, `PACKET_JET_CHUNK_SCHEMA`, `PACKET_JET_CHUNK_SCHEMA_VERSION`, `PACKET_JET_STREAM_SCHEMA`, `PACKET_JET_STREAM_SCHEMA_VERSION`, `PacketJetSnapshotStreamReader`, `PacketJetStreamChunkRecord`, `PacketJetStreamManifest`, and 4 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (7): `komorebi_math._version`, `komorebi_math.certificates`, `komorebi_math.exceptions`, `komorebi_math.export.persistence`, `komorebi_math.nonlinear.jets`, `komorebi_math.nonlinear.snapshots`, and 1 more in the machine-readable ledger. Reverse package dependents (7): `komorebi_math`, `komorebi_math.beta`, `komorebi_math.cli.main`, `komorebi_math.golden`, `komorebi_math.nonlinear`, `komorebi_math.nonlinear.profiling`, and 1 more in the machine-readable ledger. Examples (1): `resumable_packet_jet_stream.py`. Tests or test support files (2): `test_block11_cli.py`, `test_block11_streaming_migrations.py`. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.14.21 `komorebi_math.nonlinear.transport`

Purpose and source. Smooth transport for finite moving orthogonal packet systems. It is an ordinary module in the `nonlinear` family. The ledger classifies it as a public implementation owner. Source `src/komorebi_math/nonlinear/transport.py` contains 472 lines and 19,763 bytes. Source SHA-256 begins `8b11db4ddeca0bd0`.

Surface and ownership. API classification: API-manifested implementation owner. It owns 9 manifest cards: 0 stable, 0 beta, and 9 research. Linked cards: direct_packet_rotation, KatoTransport, MovingPacketSystem, direct_packet_rotation, DirectRotationResult, KatoTransport, MovingPacketSystem, PacketDerivativeResult, TransportRefinementResult. Its `__all__` declaration contains 6 names: `direct_packet_rotation`, `DirectRotationResult`, `KatoTransport`, `MovingPacketSystem`, `PacketDerivativeResult`, `TransportRefinementResult`.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (4): `komorebi_math.certificates`, `komorebi_math.dynamics._linalg`, `komorebi_math.exceptions`, `komorebi_math.provenance`. Reverse package dependents (4): `komorebi_math`, `komorebi_math.nonlinear`, `komorebi_math.quantum.kato`, `komorebi_math.verify.nonlinear_advanced`. Examples (1): `filtered_transport.py`. Tests or test support files (1): `test_nonlinear_transport.py`. Documentation files with literal module or public-route references (2): `API_STABILITY_REVIEW.md`, `BLOCK_9_RELEASE.md`. AST parsing and runtime import both passed in the recorded module audit.

D.2.14.22 `komorebi_math.nonlinear.trees`

Purpose and source. Ordered rooted-tree expansions for polynomial nonlinear fixed points. It is an ordinary module in the `nonlinear` family. The ledger classifies it as a public implementation owner. Source `src/komorebi_math/nonlinear/trees.py` contains 739 lines and 27,451 bytes. Source SHA-256 begins `94ce7611a8552386`.

Surface and ownership. API classification: API-manifested implementation owner. It owns 18 manifest cards: 0 stable, 0 beta, and 18 research. Linked cards: majorant_convergence, OrderedRootedTree, packet_tree_reconstruction, PolynomialFixedPointProblem, quadratic_catalan_majorant, catalan_number, majorant_convergence, MajorantConvergence, ordered_rooted_trees, ordered_tree_count, and 8 more in the machine-readable ledger. Its `__all__` declaration contains 14 names: `catalan_number`, `majorant_convergence`, `MajorantConvergence`, `MultilinearOperation`, `ordered_rooted_trees`, `ordered_tree_count`, `ordered_trees_with_leaves`, `OrderedRootedTree`, `packet_tree_reconstruction`, `PacketTreeResult`, and 4 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (5): `komorebi_math.certificates`, `komorebi_math.core.packets`, `komorebi_math.exceptions`, `komorebi_math.nonlinear.jets`, `komorebi_math.provenance`. Reverse package dependents (4): `komorebi_math`, `komorebi_math.export.persistence`, `komorebi_math.nonlinear`, `komorebi_math.verify.nonlinear_advanced`. Examples (1): `nonlinear_trees_elimination.py`. Tests or test support files (3): `test_block10_execution_persistence.py`, `test_nonlinear_snapshots_persistence.py`, `test_nonlinear_trees.py`. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.15 Integration: 8 modules

D.2.15.1 `komorebi_math.integration`

Purpose and source. General finite measure integration for the research API. It is the package initializer for the `integration` family. The ledger classifies it as an importable package facade outside API manifests. Source `src/komorebi_math/integration/__init__.py` contains 89 lines and 2,385 bytes. Source SHA-256 begins `43776914f55055c7`.

Surface and ownership. API classification: importable package module outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its `__all__` declaration contains 38 names: `affine_interval_change_of_variables`, `all_fubini_orders`, `analysis_synthesis_integration_certificate`, `AtomicIntegratedDerivativeProvider`, `AtomicMeasure`, `CompositeSimpson`, `coordinate_fundamental_theorem`, `coordinate_transport_identity`, `DerivativeBound`, `ErrorGuarantee`, and 28 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (7): `komorebi_math.integration.adapters`, `komorebi_math.integration.coordinates`, `komorebi_math.integration.identities`, `komorebi_math.integration.measures`, `komorebi_math.integration.polynomial`, `komorebi_math.integration.quadrature`, and 1 more in the machine-readable ledger. Reverse package dependents (2): `komorebi_math`, `komorebi_math.verify.integration`. Examples (1): `block19_measure_integration_worked_example.py`. Tests or test support files (1): `test_block19_integration.py`. Documentation files with literal module or public-route references (1): `PUBLIC_API.md`. AST parsing and runtime import both passed in the recorded module audit.

D.2.15.2 `komorebi_math.integration.adapters`

Purpose and source. Adapters from existing coordinate, series, derivative, and frame calculi. It is an ordinary module in the `integration` family. The ledger classifies it as a public implementation owner. Source `src/komorebi_math/integration/adapters.py` contains 408 lines and 16,614 bytes. Source SHA-256 begins `9b8a3651c643a7a2`.

Surface and ownership. API classification: API-manifested implementation owner. It owns 6 manifest cards: 0 stable, 0 beta, and 6 research. Linked cards: analysis_synthesis_integration_certificate, AtomicIntegratedDerivativeProvider, integrate_formal_series, integrate_polynomial_coordinates, polynomial_from_family_coordinates, polynomial_from_formal_series. Its `__all__` declaration contains 6 names: `analysis_synthesis_integration_certificate`, `AtomicIntegratedDerivativeProvider`, `integrate_formal_series`, `integrate_polynomial_coordinates`, `polynomial_from_family_coordinates`, `polynomial_from_formal_series`.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (8): `komorebi_math.certificates`, `komorebi_math.coordinates.analysis_synthesis`, `komorebi_math.coordinates.polynomials`, `komorebi_math.exceptions`, `komorebi_math.generating.series`, `komorebi_math.integration.measures`, and 2 more in the machine-readable ledger. Reverse package dependents (1): `komorebi_math.integration`. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.15.3 `komorebi_math.integration.coordinates`

Purpose and source. Integration covectors in arbitrary exact polynomial coordinates. It is an ordinary module in the `integration` family. The ledger classifies it as a public implementation owner. Source `src/komorebi_math/integration/coordinates.py` contains 371 lines and 13,498 bytes. Source SHA-256 begins `687601788d64e15f`.

Surface and ownership. API classification: API-manifested implementation owner. It owns 4 manifest cards: 0 stable, 0 beta, and 4 research. Linked cards: coordinate_fundamental_theorem, coordinate_transport_identity, polynomial_integration_covector, PolynomialIntegrationCovector. Its `__all__` declaration contains 4 names: `coordinate_fundamental_theorem`, `coordinate_transport_identity`, `polynomial_integration_covector`, `PolynomialIntegrationCovector`.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (8): `komorebi_math.certificates`, `komorebi_math.coordinates.calculus`, `komorebi_math.coordinates.polynomials`, `komorebi_math.exceptions`, `komorebi_math.integration.identities`, `komorebi_math.integration.measures`, and 2 more in the machine-readable ledger. Reverse package dependents

(1): `komorebi_math.integration`. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.15.4 `komorebi_math.integration.identities`

Purpose and source. Exact finite integral identities with replayable witness data. It is an ordinary module in the `integration` family. The ledger classifies it as a public implementation owner. Source `src/komorebi_math/integration/identities.py` contains 341 lines and 13,394 bytes. Source SHA-256 begins `5a323effecd17cde`.

Surface and ownership. API classification: API-manifested implementation owner. It owns 7 manifest cards: 0 stable, 0 beta, and 7 research. Linked cards: affine_interval_change_of_variables, all_fubini_orders, fubini_identity, fundamental_theorem, IntegralIdentity, integration_by_parts, pushforward_identity. Its `__all__` declaration contains 7 names: `affine_interval_change_of_variables`, `all_fubini_orders`, `fubini_identity`, `fundamental_theorem`, `IntegralIdentity`, `integration_by_parts`, `pushforward_identity`.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (5): `komorebi_math.certificates`, `komorebi_math.exceptions`, `komorebi_math.integration.measures`, `komorebi_math.integration.polynomial`, `komorebi_math.integration.quadrature`. Reverse package dependents (2): `komorebi_math.integration`, `komorebi_math.integration.coordinates`. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.15.5 `komorebi_math.integration.measures`

Purpose and source. Finite measure objects and exact pushforward semantics. It is an ordinary module in the `integration` family. The ledger classifies it as a public implementation owner. Source `src/komorebi_math/integration/measures.py` contains 1,090 lines and 44,458 bytes. Source SHA-256 begins `8afd4b87c10b7309`.

Surface and ownership. API classification: API-manifested implementation owner. It owns 8 manifest cards: 0 stable, 0 beta, and 8 research. Linked cards: AtomicMeasure, ExactAffineMap, IntervalMeasure, Measure, product_measure, ProductMeasure, pushforward_measure, PushforwardMeasure. Its `__all__` declaration contains 9 names: `AtomicMeasure`, `ExactAffineMap`, `IntervalMeasure`, `Measure`, `product_measure`, `ProductMeasure`, `pushforward_measure`, `PushforwardMeasure`, `RealScalar`.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (5): `komorebi_math.certificates`, `komorebi_math.coordinates.exact`, `komorebi_math.exceptions`, `komorebi_math.integration.polynomial`, `komorebi_math.integration.quadrature`. Reverse package dependents (6): `komorebi_math.integration`, `komorebi_math.integration.adapters`, `komorebi_math.integration.coordinates`, `komorebi_math.integration.identities`, `komorebi_math.integration.quadrature`, `komorebi_math.integration.rules`. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.15.6 `komorebi_math.integration.polynomial`

Purpose and source. Exact finite multivariate polynomials for measure integration. It is an ordinary module in the `integration` family. The ledger classifies it as a public implementation owner. Source `src/komorebi_math/integration/polynomial.py` contains 373 lines and 15,731 bytes. Source SHA-256 begins `f4f739bafd6ab4da`.

Surface and ownership. API classification: API-manifested implementation owner. It owns 1 manifest card: 0 stable, 0 beta, and 1 research. Linked cards: ExactPolynomial. Its `__all__` declaration contains 4 names: `ExactPolynomial`, `Exponent`, `Rational`, `Term`.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (2): `komorebi_math.certificates`, `komorebi_math.exceptions`. Reverse package dependents (7): `komorebi_math.integration`, `komorebi_math.integration.adapters`, `komorebi_math.integration.coordinates`, `komorebi_math.integration.identities`, `komorebi_math.integration.measures`, `komorebi_math.integration.quadrature`, and 1 more in the machine-readable ledger. Examples (0): none. Tests or test support files

(0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.15.7 `komorebi_math.integration.quadrature`

Purpose and source. Generic integration dispatch and honest quadrature error semantics. It is an ordinary module in the `integration` family. The ledger classifies it as a public implementation owner. Source `src/komorebi_math/integration/quadrature.py` contains 1,261 lines and 49,338 bytes. Source SHA-256 begins `0b48e7ee5eb5b42c`.

Surface and ownership. API classification: API-manifested implementation owner. It owns 9 manifest cards: 0 stable, 0 beta, and 9 research. Linked cards: CompositeSimpson, DerivativeBound, ErrorGuarantee, IntegralResult, integrate, integrate_derivative_provider, integrate_polynomial_oriented, IntegrationError, polynomial_derivative_bound. Its `__all__` declaration contains 9 names: `CompositeSimpson`, `DerivativeBound`, `ErrorGuarantee`, `IntegralResult`, `integrate`, `integrate_derivative_provider`, `integrate_polynomial_oriented`, `IntegrationError`, `polynomial_derivative_bound`.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (4): `komorebi_math.certificates`, `komorebi_math.exceptions`, `komorebi_math.integration.measures`, `komorebi_math.integration.polynomial`. Reverse package dependents (6): `komorebi_math.integration`, `komorebi_math.integration.adapters`, `komorebi_math.integration.coordinates`, `komorebi_math.integration.identities`, `komorebi_math.integration.measures`, `komorebi_math.integration.rules`. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.15.8 `komorebi_math.integration.rules`

Purpose and source. Signed quadrature functionals kept distinct from positive measures. It is an ordinary module in the `integration` family. The ledger classifies it as a public implementation owner. Source `src/komorebi_math/integration/rules.py` contains 526 lines and 21,223 bytes. Source SHA-256 begins `ba0e8d7ad4154999`.

Surface and ownership. API classification: API-manifested implementation owner. It owns 3 manifest cards: 0 stable, 0 beta, and 3 research. Linked cards: ExactQuadratureRule, QuadratureExactness, QuadratureRule. Its `__all__` declaration contains 3 names: `ExactQuadratureRule`, `QuadratureExactness`, `QuadratureRule`.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (5): `komorebi_math.certificates`, `komorebi_math.exceptions`, `komorebi_math.integration.measures`, `komorebi_math.integration.polynomial`, `komorebi_math.integration.quadrature`. Reverse package dependents (1): `komorebi_math.integration`. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.16 Dynamics: 12 modules

D.2.16.1 `komorebi_math.dynamics`

Purpose and source. Semigroups, probabilistic refinement, fractional operators, and memory. It is the package initializer for the `dynamics` family. The ledger classifies it as an importable package facade outside API manifests. Source `src/komorebi_math/dynamics/__init__.py` contains 112 lines and 2,815 bytes. Source SHA-256 begins `e4c6b8fdf58de468`.

Surface and ownership. API classification: importable package module outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its `__all__` declaration contains 43 names: `aggregation_matrix`, `AggregationMap`, `approximate_likelihood_partition`, `BayesianFlow`, `BayesianPartition`, `BayesianUpdate`, `coarsest_sufficient_partition`, `ConditionalExpectation`, `ConditionalExpectationTower`, `ContinuousTimeMarkovChain`, and 33 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (12): `komorebi_math._planned`, `komorebi_math.dynamics.bayes`, `komorebi_math.dynamics.feshbach`, `komorebi_math.dynamics.feynman_kac`, `komorebi_math.dynamics.fractional`, `komorebi_math.dynamics.green`, and 6 more in the machine-readable ledger. Reverse package dependents

(4): komorebi_math, komorebi_math.quantum.effective, komorebi_math.verify.dynamics, komorebi_math.verify.quantum. Examples (5): feshbach_memory.py, filtered_transport.py, fractional_survival.py, kato_feshbach.py, markov_bayes.py. Tests or test support files (7): test_block4_hardening.py, test_dynamics_bayes_martingales.py, test_dynamics_feshbach.py, test_dynamics_fractional_green_feynman_survival.py, test_dynamics_semigroups_markov.py, and 2 more in the machine-readable ledger. Documentation files with literal module or public-route references (2): API_INVENTORY.md, PUBLIC_API.md. AST parsing and runtime import both passed in the recorded module audit.

D.2.16.2 komorebi_math.dynamics._linalg

Purpose and source. Small matrix functions used by the finite dynamics layer. It is an ordinary module in the dynamics family. The ledger classifies it as an internal support module. Source src/komorebi_math/dynamics/_linalg.py contains 117 lines and 4,672 bytes. Source SHA-256 begins 3b75d5e7183b8b78.

Surface and ownership. API classification: internal by module name. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its __all__ declaration contains 4 names: as_square_matrix, block_partition, hermitian_fractional_power, matrix_exponential.

Dependencies and evidence. Direct dependency profile: optional scientific extra, required NumPy dependency. Direct package dependencies (1): komorebi_math.exceptions. Reverse package dependents (12): komorebi_math.analytic.sections, komorebi_math.dynamics.feshbach, komorebi_math.dynamics.feynman_kac, komorebi_math.dynamics.markov, komorebi_math.dynamics.memory, komorebi_math.dynamics.semigroups, and 6 more in the machine-readable ledger. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.16.3 komorebi_math.dynamics.bayes

Purpose and source. Finite Bayesian updates, sufficient partitions, flows, and coarse filters. It is an ordinary module in the dynamics family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/dynamics/bayes.py contains 448 lines and 18,979 bytes. Source SHA-256 begins da5215d0b6a8be7c.

Surface and ownership. API classification: API-manifested implementation owner. It owns 1 manifest card: 0 stable, 0 beta, and 1 research. Linked cards: BayesianUpdate. Its __all__ declaration contains 8 names: approximate_likelihood_partition, BayesianFlow, BayesianPartition, BayesianUpdate, coarsest_sufficient_partition, HiddenMarkovFilter, relative_entropy, relative_entropy_decomposition.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (4): komorebi_math.certificates, komorebi_math.dynamics.markov, komorebi_math.exceptions, komorebi_math.provenance. Reverse package dependents (1): komorebi_math.dynamics. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.16.4 komorebi_math.dynamics.feshbach

Purpose and source. Finite Schur Feshbach reduction and exact hidden state memory. It is an ordinary module in the dynamics family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/dynamics/feshbach.py contains 196 lines and 7,882 bytes. Source SHA-256 begins b011baa1ebef8aa2.

Surface and ownership. API classification: API-manifested implementation owner. It owns 1 manifest card: 0 stable, 0 beta, and 1 research. Linked cards: FeshbachReduction. Its __all__ declaration contains 1 name: FeshbachReduction.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (5): komorebi_math.certificates, komorebi_math.dynamics._linalg, komorebi_math.dynamics.memory, komorebi_math.exceptions, komorebi_math.provenance. Reverse package dependents (1): komorebi_math.dynamics. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.16.5 komorebi_math.dynamics.feynman_kac

Purpose and source. Finite-state Feynman-Kac transfer operators. It is an ordinary module in the dynamics family. The ledger classifies it as unmanifested implementation support. Source src/komorebi_math/dynamics/feynman_kac.py contains 185 lines and 8,371 bytes. Source SHA-256 begins 49c17e4210904781.

Surface and ownership. API classification: importable package module outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its __all__ declaration contains 2 names: FeynmanKacKind, FiniteFeynmanKac.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (5): komorebi_math.certificates, komorebi_math.dynamics._linalg, komorebi_math.dynamics.markov, komorebi_math.exceptions, komorebi_math.provenance. Reverse package dependents (1): komorebi_math.dynamics. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.16.6 komorebi_math.dynamics.fractional

Purpose and source. Finite positive fractional operators and refinement certificates. It is an ordinary module in the dynamics family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/dynamics/fractional.py contains 184 lines and 7,094 bytes. Source SHA-256 begins 90b600829c896cdb.

Surface and ownership. API classification: API-manifested implementation owner. It owns 1 manifest card: 0 stable, 0 beta, and 1 research. Linked cards: fractional_power. Its __all__ declaration contains 5 names: fractional_defect, fractional_naturality_certificate, fractional_power, fractional_semigroup, FractionalDefectResult.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (3): komorebi_math.certificates, komorebi_math.exceptions, komorebi_math.provenance. Reverse package dependents (1): komorebi_math.dynamics. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.16.7 komorebi_math.dynamics.green

Purpose and source. Exact one-dimensional finite-difference Green operators. It is an ordinary module in the dynamics family. The ledger classifies it as unmanifested implementation support. Source src/komorebi_math/dynamics/green.py contains 88 lines and 3,171 bytes. Source SHA-256 begins b757cff0bfdfaf24.

Surface and ownership. API classification: importable package module outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its __all__ declaration contains 1 name: DirichletGreen1D.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (3): komorebi_math.certificates, komorebi_math.exceptions, komorebi_math.provenance. Reverse package dependents (1): komorebi_math.dynamics. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.16.8 komorebi_math.dynamics.markov

Purpose and source. Finite Markov operators, aggregation maps, and exact lumping diagnostics. It is an ordinary module in the dynamics family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/dynamics/markov.py contains 425 lines and 18,121 bytes. Source SHA-256 begins adc03fae56e58430.

Surface and ownership. API classification: API-manifested implementation owner. It owns 1 manifest card: 0 stable, 0 beta, and 1 research. Linked cards: MarkovLumping. Its __all__ declaration contains 8 names: aggregation_matrix, AggregationMap, ContinuousTimeMarkovChain, coordinate_lumpability_residual, FiniteMarkovChain, MarkovKind, MarkovLumping, normalize_partition.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (4): komorebi_math.certificates, komorebi_math.dynamics._linalg, komorebi_math.exceptions, komorebi_math.

provenance. Reverse package dependents (3): komorebi_math.dynamics, komorebi_math.dynamics.bayes, komorebi_math.dynamics.feynman_kac. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.16.9 komorebi_math.dynamics.martingales

Purpose and source. Finite conditional expectations and martingale packet towers. It is an ordinary module in the dynamics family. The ledger classifies it as unmanifested implementation support. Source src/komorebi_math/dynamics/martingales.py contains 234 lines and 9,795 bytes. Source SHA-256 begins afabfcd820079bb9.

Surface and ownership. API classification: importable package module outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its __all__ declaration contains 5 names: blocks_from_labels, ConditionalExpectation, ConditionalExpectationTower, MartingaleTower, partition_refines.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (3): komorebi_math.certificates, komorebi_math.exceptions, komorebi_math.provenance. Reverse package dependents (1): komorebi_math.dynamics. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.16.10 komorebi_math.dynamics.memory

Purpose and source. Finite rational memory kernels, auxiliary realizations, and stability bounds. It is an ordinary module in the dynamics family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/dynamics/memory.py contains 320 lines and 12,890 bytes. Source SHA-256 begins 9a63a0123887def3.

Surface and ownership. API classification: API-manifested implementation owner. It owns 1 manifest card: 0 stable, 0 beta, and 1 research. Linked cards: MemoryKernel. Its __all__ declaration contains 6 names: kernel_l1_error, memory_kernel_error_bound, MemoryKernel, RationalMemoryKernel, RationalMemoryRealization, volterra_trajectory_bound.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (4): komorebi_math.certificates, komorebi_math.dynamics._linalg, komorebi_math.exceptions, komorebi_math.provenance. Reverse package dependents (2): komorebi_math.dynamics, komorebi_math.dynamics.feshbach. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.16.11 komorebi_math.dynamics.semigroups

Purpose and source. Finite matrix semigroups, packet blocks, and refinement defects. It is an ordinary module in the dynamics family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/dynamics/semigroups.py contains 342 lines and 14,304 bytes. Source SHA-256 begins 3e905fdef4dc9eec.

Surface and ownership. API classification: API-manifested implementation owner. It owns 1 manifest card: 0 stable, 0 beta, and 1 research. Linked cards: MatrixSemigroup. Its __all__ declaration contains 7 names: FiniteSemigroupTower, matrix_exponential, MatrixSemigroup, PacketSemigroup, ProjectiveSemigroupTower, semigroup_refinement_defect, SemigroupRefinement.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (5): komorebi_math.certificates, komorebi_math.core, komorebi_math.dynamics._linalg, komorebi_math.exceptions, komorebi_math.provenance. Reverse package dependents (2): komorebi_math.dynamics, komorebi_math.dynamics.survival. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.16.12 komorebi_math.dynamics.survival

Purpose and source. Finite phase-type survival and competing risk packets. It is an ordinary module in the dynamics family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/dynamics/survival.py contains 193 lines and 8,097 bytes. Source SHA-256 begins a02759b3a1b01c4c.

Surface and ownership. API classification: API-manifested implementation owner. It owns 1 manifest card: 0 stable, 0 beta, and 1 research. Linked cards: PhaseTypeSurvival. Its __all__ declaration contains 1 name: PhaseTypeSurvival.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (4): komorebi_math.certificates, komorebi_math.dynamics.semigroups, komorebi_math.exceptions, komorebi_math.provenance. Reverse package dependents (1): komorebi_math.dynamics. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.17 Quantum: 15 modules

D.2.17.1 komorebi_math.quantum

Purpose and source. Finite quantum packet calculus, channels, transport, and diagnostics. It is the package initializer for the quantum family. The ledger classifies it as an importable package facade outside API manifests. Source src/komorebi_math/quantum/__init__.py contains 107 lines and 3,031 bytes. Source SHA-256 begins 03d21a689ca36e84.

Surface and ownership. API classification: importable package module outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its __all__ declaration contains 46 names: amplitude_damping, BoundedHamiltonianTower, ChannelPacketBlock, charge_certificate, charge_selection_certificate, CircuitPacketExpansion, CircuitPathAmplitude, computational_basis_generation, constraint_closure, ConstraintClosureReport, and 36 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (15): komorebi_math._planned, komorebi_math.quantum.channels, komorebi_math.quantum.circuits, komorebi_math.quantum.constraints, komorebi_math.quantum.effective, komorebi_math.quantum.error_correction, and 9 more in the machine-readable ledger. Reverse package dependents (2): komorebi_math, komorebi_math.verify.quantum. Examples (4): euclidean_spectral.py, kato_feshbach.py, quantum_channel_qec.py, quantum_symmetry_circuit.py. Tests or test support files (4): test_nonlinear_transport.py, test_quantum_channels_qec.py, test_quantum_symmetry_circuits.py, test_quantum_transport_effective_spectral.py. Documentation files with literal module or public-route references (5): API_INVENTORY.md, API_STABILITY_REVIEW.md, BLOCK_9_RELEASE.md, NONLINEAR_CALCULUS.md, PUBLIC_API.md. AST parsing and runtime import both passed in the recorded module audit.

D.2.17.2 komorebi_math.quantum._validation

Purpose and source. Validation helpers for finite quantum models. It is an ordinary module in the quantum family. The ledger classifies it as an internal support module. Source src/komorebi_math/quantum/_validation.py contains 126 lines and 4,812 bytes. Source SHA-256 begins 76c9e6aa914ad834.

Surface and ownership. API classification: internal by module name. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its __all__ declaration contains 8 names: as_matrix, as_square_matrix, as_vector, density_matrix, orthogonal_packet_family, require_hermitian, require_projector, require_unitary.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (1): komorebi_math.exceptions. Reverse package dependents (12): komorebi_math.quantum.channels, komorebi_math.quantum.circuits, komorebi_math.quantum.constraints, komorebi_math.quantum.effective, komorebi_math.quantum.error_correction, komorebi_math.quantum.euclidean, and 6 more in the machine-readable ledger. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.17.3 komorebi_math.quantum.channels

Purpose and source. Finite quantum channels, operator packets, Choi matrices, and covariance. It is an ordinary module in the quantum family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/quantum/channels.py contains 269 lines and 11,511 bytes. Source SHA-256 begins 956acd703044a0ba.

Surface and ownership. API classification: API-manifested implementation owner. It owns 1 manifest card: 0 stable, 0 beta, and 1 research. Linked cards: QuantumChannel. Its __all__ declaration contains 6 names: amplitude_damping, ChannelPacketBlock, dephase, operator_packets, QuantumChannel, reconstruct_operator_packets.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (5): komorebi_math.certificates, komorebi_math.core, komorebi_math.exceptions, komorebi_math.provenance, komorebi_math.quantum._validation. Reverse package dependents (2): komorebi_math.quantum, komorebi_math.quantum.error_correction. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.17.4 komorebi_math.quantum.circuits

Purpose and source. Exact finite quantum circuit packet path expansions. It is an ordinary module in the quantum family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/quantum/circuits.py contains 199 lines and 8,725 bytes. Source SHA-256 begins 49c75d8b5fb23c57.

Surface and ownership. API classification: API-manifested implementation owner. It owns 1 manifest card: 0 stable, 0 beta, and 1 research. Linked cards: CircuitPacketExpansion. Its __all__ declaration contains 3 names: CircuitPacketExpansion, CircuitPathAmplitude, InterferenceReport.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (5): komorebi_math.certificates, komorebi_math.core, komorebi_math.exceptions, komorebi_math.provenance, komorebi_math.quantum._validation. Reverse package dependents (1): komorebi_math.quantum. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.17.5 komorebi_math.quantum.constraints

Purpose and source. Finite constraint closure and refinement anomaly diagnostics. It is an ordinary module in the quantum family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/quantum/constraints.py contains 102 lines and 4,294 bytes. Source SHA-256 begins 4709fae78ee0879a.

Surface and ownership. API classification: API-manifested implementation owner. It owns 1 manifest card: 0 stable, 0 beta, and 1 research. Linked cards: constraint_closure. Its __all__ declaration contains 3 names: constraint_closure, ConstraintClosureReport, transported_constraint_defect.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (4): komorebi_math.certificates, komorebi_math.exceptions, komorebi_math.provenance, komorebi_math.quantum._validation. Reverse package dependents (1): komorebi_math.quantum. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.17.6 komorebi_math.quantum.effective

Purpose and source. Hermitian Feshbach reduction and hidden quantum memory. It is an ordinary module in the quantum family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/quantum/effective.py contains 98 lines and 3,958 bytes. Source SHA-256 begins bcc39f27082afbce.

Surface and ownership. API classification: API-manifested implementation owner. It owns 1 manifest card: 0 stable, 0 beta, and 1 research. Linked cards: QuantumFeshbachReduction. Its __all__ declaration contains 1 name: QuantumFeshbachReduction.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (5): komorebi_math.certificates, komorebi_math.dynamics, komorebi_math.exceptions, komorebi_math.provenance, komorebi_math.quantum._validation. Reverse package dependents (1): komorebi_math.quantum. Examples (0): none.

Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.17.7 komorebi_math.quantum.error_correction

Purpose and source. Finite quantum error correction through Knill-Laflamme compression. It is an ordinary module in the quantum family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/quantum/error_correction.py contains 162 lines and 7,675 bytes. Source SHA-256 begins 63929c14114a1271.

Surface and ownership. API classification: API-manifested implementation owner. It owns 1 manifest card: 0 stable, 0 beta, and 1 research. Linked cards: QuantumCode. Its __all__ declaration contains 2 names: KnillLaflammeReport, QuantumCode.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (5): komorebi_math.certificates, komorebi_math.exceptions, komorebi_math.provenance, komorebi_math.quantum._validation, komorebi_math.quantum.channels. Reverse package dependents (1): komorebi_math.quantum. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.17.8 komorebi_math.quantum.euclidean

Purpose and source. Finite Euclidean transfer operators and reflection positivity. It is an ordinary module in the quantum family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/quantum/euclidean.py contains 130 lines and 5,771 bytes. Source SHA-256 begins b93ca15e938e2775.

Surface and ownership. API classification: API-manifested implementation owner. It owns 1 manifest card: 0 stable, 0 beta, and 1 research. Linked cards: EuclideanTransfer. Its __all__ declaration contains 1 name: EuclideanTransfer.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (5): komorebi_math.certificates, komorebi_math.dynamics._linalg, komorebi_math.exceptions, komorebi_math.provenance, komorebi_math.quantum._validation. Reverse package dependents (1): komorebi_math.quantum. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.17.9 komorebi_math.quantum.kato

Purpose and source. Backward-compatible quantum exports for shared smooth packet transport. It is an ordinary module in the quantum family. The ledger classifies it as unmanifested implementation support. Source src/komorebi_math/quantum/kato.py contains 19 lines and 456 bytes. Source SHA-256 begins 5358211f57124e3f.

Surface and ownership. API classification: importable package module outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its __all__ declaration contains 6 names: direct_packet_rotation, DirectRotationResult, KatoTransport, MovingPacketSystem, PacketDerivativeResult, TransportRefinementResult.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (1): komorebi_math.nonlinear.transport. Reverse package dependents (2): komorebi_math.quantum, komorebi_math.quantum.leakage. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (1): API_STABILITY_REVIEW.md. AST parsing and runtime import both passed in the recorded module audit.

D.2.17.10 komorebi_math.quantum.leakage

Purpose and source. Fixed and moving packet leakage bounds. It is an ordinary module in the quantum family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/quantum/leakage.py contains 136 lines and 4,765 bytes. Source SHA-256 begins 5f29ee27743076ec.

Surface and ownership. API classification: API-manifested implementation owner. It owns 1 manifest card: 0 stable, 0 beta, and 1 research. Linked cards: fixed_packet_leakage. Its __all__ declaration contains 4 names: fixed_packet_leakage, LeakageResult, moving_geometric_coupling, moving_packet_leakage_bound.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (6): komorebi_math.certificates, komorebi_math.dynamics._linalg, komorebi_math.exceptions, komorebi_math.

provenance, komorebi_math.quantum._validation, komorebi_math.quantum.kato. Reverse package dependents (1): komorebi_math.quantum. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.17.11 komorebi_math.quantum.lindblad

Purpose and source. Finite Lindblad generators and cyclic coherence packets. It is an ordinary module in the quantum family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/quantum/lindblad.py contains 121 lines and 5,251 bytes. Source SHA-256 begins 27480c2a9ab227d0.

Surface and ownership. API classification: API-manifested implementation owner. It owns 1 manifest card: 0 stable, 0 beta, and 1 research. Linked cards: LindbladGenerator. Its __all__ declaration contains 1 name: LindbladGenerator.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (5): komorebi_math.certificates, komorebi_math.dynamics._linalg, komorebi_math.exceptions, komorebi_math.provenance, komorebi_math.quantum._validation. Reverse package dependents (1): komorebi_math.quantum. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.17.12 komorebi_math.quantum.seeds

Purpose and source. Finite quantum seed systems and local orbit dimensions. It is an ordinary module in the quantum family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/quantum/seeds.py contains 168 lines and 5,861 bytes. Source SHA-256 begins 81fdd926c0415333.

Surface and ownership. API classification: API-manifested implementation owner. It owns 1 manifest card: 0 stable, 0 beta, and 1 research. Linked cards: QuantumSeedSystem. Its __all__ declaration contains 5 names: computational_basis_generation, local_orbit_dimension, observable_seed_rank, QuantumSeedSystem, schmidt_rank.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (5): komorebi_math.certificates, komorebi_math.core, komorebi_math.exceptions, komorebi_math.provenance, komorebi_math.quantum._validation. Reverse package dependents (1): komorebi_math.quantum. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.17.13 komorebi_math.quantum.spectral

Purpose and source. Finite spectral dimension and periodic lattice diagnostics. It is an ordinary module in the quantum family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/quantum/spectral.py contains 143 lines and 5,815 bytes. Source SHA-256 begins 1b6f2b6f4c6b1664.

Surface and ownership. API classification: API-manifested implementation owner. It owns 2 manifest cards: 0 stable, 0 beta, and 2 research. Linked cards: periodic_laplacian, spectral_dimension. Its __all__ declaration contains 5 names: heat_trace, low_mode_dispersion_bound, periodic_laplacian, periodic_lattice_eigenvalues, spectral_dimension.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (3): komorebi_math.exceptions, komorebi_math.provenance, komorebi_math.quantum._validation. Reverse package dependents (1): komorebi_math.quantum. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.17.14 komorebi_math.quantum.symmetry

Purpose and source. Finite group, cyclic charge, and tensor symmetry packets. It is an ordinary module in the quantum family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/quantum/symmetry.py contains 188 lines and 6,896 bytes. Source SHA-256 begins 3696b4f2db5045b2.

Surface and ownership. API classification: API-manifested implementation owner. It owns 1 manifest card: 0 stable, 0 beta, and 1 research. Linked cards: cyclic_symmetry_projectors. Its __all__ declaration contains 6 names: charge_certificate, charge_selection_certificate, cyclic_symmetry_projectors, finite_group_projector, symmetry_projector_certificate, total_charge_packets.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (5): komorebi_math.certificates, komorebi_math.core, komorebi_math.exceptions, komorebi_math.provenance, komorebi_math.quantum._validation. Reverse package dependents (1): komorebi_math.quantum. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.17.15 komorebi_math.quantum.towers

Purpose and source. Finite bounded Hamiltonian towers and compatibility diagnostics. It is an ordinary module in the quantum family. The ledger classifies it as unmanifested implementation support. Source src/komorebi_math/quantum/towers.py contains 80 lines and 3,363 bytes. Source SHA-256 begins 5236b88b39304556.

Surface and ownership. API classification: importable package module outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its __all__ declaration contains 2 names: BoundedHamiltonianTower, HamiltonianTowerReport.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (4): komorebi_math.certificates, komorebi_math.dynamics._linalg, komorebi_math.exceptions, komorebi_math.quantum._validation. Reverse package dependents (1): komorebi_math.quantum. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.18 Deformation: 7 modules

D.2.18.1 komorebi_math.deformation

Purpose and source. Finite residual deformation, support, and companion models. It is the package initializer for the deformation family. The ledger classifies it as an importable package facade outside API manifests. Source src/komorebi_math/deformation/__init__.py contains 85 lines and 2,107 bytes. Source SHA-256 begins c7d52eec665158f0.

Surface and ownership. API classification: importable package module outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its __all__ declaration contains 32 names: CohomologyData, CompanionForest, CompatibleCarrier, diagonal_nilpotent_count, FEATURE_KEY, FiberSupportResult, finite_frobenius_separator, FiniteCochainComplex, FiniteLiftForest, FittingGenerators, and 22 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (8): komorebi_math._planned, komorebi_math.deformation.companions, komorebi_math.deformation.complexes, komorebi_math.deformation.counting, komorebi_math.deformation.forests, komorebi_math.deformation.support, and 2 more in the machine-readable ledger. Reverse package dependents (3): komorebi_math, komorebi_math.cli.main, komorebi_math.verify.deformation. Examples (2): deformation_support.py, frobenius_companions.py. Tests or test support files (4): test_block10_execution_persistence.py, test_deformation_complex_support_companions.py, test_deformation_forests_visibility_counting.py, test_nonlinear_snapshots_persistence.py. Documentation files with literal module or public-route references (1): PUBLIC_API.md. AST parsing and runtime import both passed in the recorded module audit.

D.2.18.2 komorebi_math.deformation.companions

Purpose and source. Finite companion forests, Frobenius spines, and trace separation. It is an ordinary module in the deformation family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/deformation/companions.py contains 276 lines and 11,291 bytes. Source SHA-256 begins d0e1870a622976a6.

Surface and ownership. API classification: API-manifested implementation owner. It owns 2 manifest cards: 0 stable, 0 beta, and 2 research. Linked cards: finite_frobenius_separator, FrobeniusSpine. Its __all__ declaration contains 5 names: CompanionForest, CompatibleCarrier, finite_frobenius_separator, FrobeniusSeparationResult, FrobeniusSpine.

Dependencies and evidence. Direct dependency profile: optional exact extra. Direct package dependencies (4): komorebi_math._number_theory, komorebi_math.certificates, komorebi_math.deformation.forests, komorebi_math.exceptions. Reverse package dependents (1): komorebi_math.deformation. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.18.3 komorebi_math.deformation.complexes

Purpose and source. Finite cochain complexes and packetwise cohomology over prime fields. It is an ordinary module in the deformation family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/deformation/complexes.py contains 234 lines and 9,961 bytes. Source SHA-256 begins 436b06399cb0b4e7.

Surface and ownership. API classification: API-manifested implementation owner. It owns 1 manifest card: 0 stable, 0 beta, and 1 research. Linked cards: FiniteCochainComplex. Its __all__ declaration contains 3 names: CohomologyData, FiniteCochainComplex, row_rank_mod.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (3): komorebi_math._number_theory, komorebi_math.certificates, komorebi_math.exceptions. Reverse package dependents (4): komorebi_math.deformation, komorebi_math.deformation.support, komorebi_math.langlands.local.bridge, komorebi_math.langlands.local.symplectic. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.18.4 komorebi_math.deformation.counting

Purpose and source. Exact finite-level counts for basic deformation charts. It is an ordinary module in the deformation family. The ledger classifies it as unmanifested implementation support. Source src/komorebi_math/deformation/counting.py contains 143 lines and 5,074 bytes. Source SHA-256 begins 5f9171cbf1899f54.

Surface and ownership. API classification: importable package module outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its __all__ declaration contains 9 names: diagonal_nilpotent_count, nilpotent_chart_count, nilpotent_chart_series, normal_crossing_count, normal_crossing_series, RationalCountingSeries, smooth_chart_count, smooth_chart_series, weighted_chart_count.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (2): komorebi_math._number_theory, komorebi_math.exceptions. Reverse package dependents (1): komorebi_math.deformation. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.18.5 komorebi_math.deformation.forests

Purpose and source. Finite residual lift forests and stable inverse-system cores. It is an ordinary module in the deformation family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/deformation/forests.py contains 226 lines and 8,897 bytes. Source SHA-256 begins 3721563aea386fe8.

Surface and ownership. API classification: API-manifested implementation owner. It owns 1 manifest card: 0 stable, 0 beta, and 1 research. Linked cards: FiniteLiftForest. Its __all__ declaration contains 4 names: FiniteLiftForest, LiftBranch, LiftForestReport, residue_forest.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (3): `komorebi_math._number_theory`, `komorebi_math.certificates`, `komorebi_math.exceptions`. Reverse package dependents (3): `komorebi_math.deformation`, `komorebi_math.deformation.companions`, `komorebi_math.export.persistence`. Examples (0): none. Tests or test support files (1): `test_nonlinear_snapshots_persistence.py`. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.18.6 `komorebi_math.deformation.support`

Purpose and source. Finite presentation, Fitting, support, and occupancy calculations. It is an ordinary module in the `deformation` family. The ledger classifies it as a public implementation owner. Source `src/komorebi_math/deformation/support.py` contains 240 lines and 9,423 bytes. Source SHA-256 begins `be6bffe1f6a8cbb4`.

Surface and ownership. API classification: API-manifested implementation owner. It owns 1 manifest card: 0 stable, 0 beta, and 1 research. Linked cards: PresentedModule. Its `__all__` declaration contains 5 names: `FiberSupportResult`, `FittingGenerators`, `nested_subspace_occupancy`, `NestedOccupancyResult`, `PresentedModule`.

Dependencies and evidence. Direct dependency profile: optional `exact` extra, required NumPy dependency. Direct package dependencies (4): `komorebi_math._number_theory`, `komorebi_math.certificates`, `komorebi_math.deformation.complexes`, `komorebi_math.exceptions`. Reverse package dependents (1): `komorebi_math.deformation`. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.18.7 `komorebi_math.deformation.visibility`

Purpose and source. Prime-adic visibility partitions and cluster trees. It is an ordinary module in the `deformation` family. The ledger classifies it as a public implementation owner. Source `src/komorebi_math/deformation/visibility.py` contains 121 lines and 4,386 bytes. Source SHA-256 begins `3b61d0116228a469`.

Surface and ownership. API classification: API-manifested implementation owner. It owns 1 manifest card: 0 stable, 0 beta, and 1 research. Linked cards: VisibilityTree. Its `__all__` declaration contains 3 names: `rational_valuation`, `VisibilityPair`, `VisibilityTree`.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (2): `komorebi_math._number_theory`, `komorebi_math.exceptions`. Reverse package dependents (1): `komorebi_math.deformation`. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.19 Langlands: 24 modules

D.2.19.1 `komorebi_math.langlands`

Purpose and source. Finite local, global diagnostic, finite image, ramification, and completion models. It is the package initializer for the `langlands` family. The ledger classifies it as an importable package facade outside API manifests. Source `src/komorebi_math/langlands/__init__.py` contains 5 lines and 238 bytes. Source SHA-256 begins `13d0d7c50d0fdb80`.

Surface and ownership. API classification: importable package module outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its `__all__` declaration contains 5 names: `completion`, `finite_image`, `global_`, `local`, `ramification`.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (5): `komorebi_math.langlands.completion`, `komorebi_math.langlands.finite_image`, `komorebi_math.langlands.global_`, `komorebi_math.langlands.local`, `komorebi_math.langlands.ramification`. Reverse package dependents (0): none. Examples (3): `finite_image_clifford.py`, `ramification_completion.py`, `tame_heisenberg.py`. Tests or test support files (4): `test_langlands_finite_image.py`, `test_langlands_local_fields_tame_gauss.py`, `test_langlands_local_heisenberg_symplectic.py`, `test_langlands_ramification_global_`

completion.py. Documentation files with literal module or public-route references (1): PUBLIC_API.md. AST parsing and runtime import both passed in the recorded module audit.

D.2.19.2 komorebi_math.langlands.completion

Purpose and source. Gate-based conditional Langlands completion reports. It is the package initializer for the langlands family. The ledger classifies it as an importable package facade outside API manifests. Source src/komorebi_math/langlands/completion/__init__.py contains 38 lines and 786 bytes. Source SHA-256 begins cf6c28c2492cd2c3.

Surface and ownership. API classification: importable package module outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its __all__ declaration contains 7 names: AutomorphicCompletionReport, CompatiblePropagationReport, CompletionGate, FEATURE_KEY, GateStatus, require, status.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (3): komorebi_math._planned, komorebi_math.langlands.completion.gates, komorebi_math.registry. Reverse package dependents (4): komorebi_math, komorebi_math.cli.main, komorebi_math.langlands, komorebi_math.verify.langlands. Examples (1): ramification_completion.py. Tests or test support files (1): test_langlands_ramification_global_completion.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.19.3 komorebi_math.langlands.completion.gates

Purpose and source. Explicit gate reports for conditional Langlands completion. It is an ordinary module in the langlands family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/langlands/completion/gates.py contains 119 lines and 4,369 bytes. Source SHA-256 begins 9eae5b42d05d6d49.

Surface and ownership. API classification: API-manifested implementation owner. It owns 4 manifest cards: 0 stable, 0 beta, and 4 research. Linked cards: AutomorphicCompletionReport, CompatiblePropagationReport, CompletionGate, GateStatus. Its __all__ declaration contains 4 names: AutomorphicCompletionReport, CompatiblePropagationReport, CompletionGate, GateStatus.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (2): komorebi_math.certificates, komorebi_math.exceptions. Reverse package dependents (1): komorebi_math.langlands.completion. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.19.4 komorebi_math.langlands.finite_image

Purpose and source. Finite group, monomial packet, Mackey, and Clifford execution models. It is the package initializer for the langlands family. The ledger classifies it as an importable package facade outside API manifests. Source src/komorebi_math/langlands/finite_image/__init__.py contains 61 lines and 1,566 bytes. Source SHA-256 begins 6fa36cd624baf74d.

Surface and ownership. API classification: importable package module outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its __all__ declaration contains 23 names: AutomorphicInductionPlan, CliffordMode, CliffordStep, CliffordTower, CliffordTowerReport, compose_permutations, cyclic_group, FEATURE_KEY, FiniteGroup, FiniteSubgroup, and 13 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (8): komorebi_math._planned, komorebi_math.langlands.finite_image.characters, komorebi_math.langlands.finite_image.clifford, komorebi_math.langlands.finite_image.groups, komorebi_math.langlands.finite_image.mackey, komorebi_math.langlands.finite_image.orbits, and 2 more in the machine-readable ledger. Reverse package dependents (3): komorebi_math, komorebi_math.langlands, komorebi_math.verify.langlands. Examples (1): finite_image_clifford.py. Tests or test support files (1): test_langlands_finite_image.py. Documentation files with literal module or public-route references (1): PUBLIC_API.md. AST parsing and runtime import both passed in the recorded module audit.

D.2.19.5 komorebi_math.langlands.finite_image.characters

Purpose and source. Linear characters and induced finite group representations. It is an ordinary module in the langlands family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/langlands/finite_image/characters.py contains 116 lines and 5,172 bytes. Source SHA-256 begins e0d6d96f440ba0a1.

Surface and ownership. API classification: API-manifested implementation owner. It owns 2 manifest cards: 0 stable, 0 beta, and 2 research. Linked cards: InducedRepresentation, LinearCharacter. Its __all__ declaration contains 2 names: InducedRepresentation, LinearCharacter.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (3): komorebi_math.certificates, komorebi_math.exceptions, komorebi_math.langlands.finite_image.groups. Reverse package dependents (4): komorebi_math.langlands.finite_image, komorebi_math.langlands.finite_image.mackey, komorebi_math.langlands.finite_image.orbits, komorebi_math.langlands.finite_image.plans. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.19.6 komorebi_math.langlands.finite_image.clifford

Purpose and source. Finite Clifford tower words, depth, and conditional descent plans. It is an ordinary module in the langlands family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/langlands/finite_image/clifford.py contains 202 lines and 8,052 bytes. Source SHA-256 begins 4c808f92dc3ed8fe.

Surface and ownership. API classification: API-manifested implementation owner. It owns 2 manifest cards: 0 stable, 0 beta, and 2 research. Linked cards: CliffordStep, CliffordTower. Its __all__ declaration contains 6 names: CliffordMode, CliffordStep, CliffordTower, CliffordTowerReport, VirtualCharacterResolution, VirtualCharacterTerm.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (3): komorebi_math._number_theory, komorebi_math.certificates, komorebi_math.exceptions. Reverse package dependents (1): komorebi_math.langlands.finite_image. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.19.7 komorebi_math.langlands.finite_image.groups

Purpose and source. Small exact finite groups, subgroups, and permutation constructors. It is an ordinary module in the langlands family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/langlands/finite_image/groups.py contains 278 lines and 11,548 bytes. Source SHA-256 begins 46953d9eed271410.

Surface and ownership. API classification: API-manifested implementation owner. It owns 2 manifest cards: 0 stable, 0 beta, and 2 research. Linked cards: cyclic_group, FiniteGroup. Its __all__ declaration contains 5 names: compose_permutations, cyclic_group, FiniteGroup, FiniteSubgroup, symmetric_group.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (3): komorebi_math._number_theory, komorebi_math.certificates, komorebi_math.exceptions. Reverse package dependents (3): komorebi_math.langlands.finite_image, komorebi_math.langlands.finite_image.characters, komorebi_math.langlands.finite_image.mackey. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.19.8 komorebi_math.langlands.finite_image.mackey

Purpose and source. Finite Mackey double-coset packet data. It is an ordinary module in the langlands family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/langlands/finite_image/mackey.py contains 71 lines and 2,707 bytes. Source SHA-256 begins 2804cd9bed595227.

Surface and ownership. API classification: API-manifested implementation owner. It owns 1 manifest card: 0 stable, 0 beta, and 1 research. Linked cards: mackey_tensor_packets. Its __all__ declaration contains 3 names: mackey_tensor_packets, MackeyDecomposition, MackeyPacket.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (3): komorebi_math.certificates, komorebi_math.langlands.

finite_image.characters, komorebi_math.langlands.finite_image.groups. Reverse package dependents (1): komorebi_math.langlands.finite_image. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.19.9 komorebi_math.langlands.finite_image.orbits

Purpose and source. Orbit packet decompositions for monomial tensor operations. It is an ordinary module in the langlands family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/langlands/finite_image/orbits.py contains 167 lines and 7,474 bytes. Source SHA-256 begins 828529d2ffa213f7.

Surface and ownership. API classification: API-manifested implementation owner. It owns 1 manifest card: 0 stable, 0 beta, and 1 research. Linked cards: MonomialAction. Its __all__ declaration contains 3 names: MonomialAction, OrbitPacket, weak_compositions.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (4): komorebi_math._number_theory, komorebi_math.certificates, komorebi_math.exceptions, komorebi_math.langlands.finite_image.characters. Reverse package dependents (1): komorebi_math.langlands.finite_image. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.19.10 komorebi_math.langlands.finite_image.plans

Purpose and source. Symbolic automorphic induction plans for finite monomial parameters. It is an ordinary module in the langlands family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/langlands/finite_image/plans.py contains 47 lines and 1,949 bytes. Source SHA-256 begins 9004a8c55d653ba1.

Surface and ownership. API classification: API-manifested implementation owner. It owns 1 manifest card: 0 stable, 0 beta, and 1 research. Linked cards: AutomorphicInductionPlan. Its __all__ declaration contains 1 name: AutomorphicInductionPlan.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (2): komorebi_math.certificates, komorebi_math.langlands.finite_image.characters. Reverse package dependents (1): komorebi_math.langlands.finite_image. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.19.11 komorebi_math.langlands.global_

Purpose and source. Conditional global support, Hecke packet, and patching models. It is the package initializer for the langlands family. The ledger classifies it as an importable package facade outside API manifests. Source src/komorebi_math/langlands/global_/__init__.py contains 37 lines and 895 bytes. Source SHA-256 begins 767c34a5eb0c1b3b.

Surface and ownership. API classification: importable package module outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its __all__ declaration contains 9 names: dyadic_sign_diagnostic, DyadicSignDiagnostic, FEATURE_KEY, FiniteHeckePackets, HeckePacket, PatchingSupportReport, require, status, SupportComparison.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (5): komorebi_math._planned, komorebi_math.langlands.global_.dyadic, komorebi_math.langlands.global_.hecke, komorebi_math.langlands.global_.patching, komorebi_math.registry. Reverse package dependents (3): komorebi_math, komorebi_math.langlands, komorebi_math.verify.langlands. Examples (0): none. Tests or test support files (1): test_langlands_ramification_global_completion.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.19.12 komorebi_math.langlands.global_.dyadic

Purpose and source. Coefficient-prime-two group algebra diagnostics. It is an ordinary module in the langlands family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/langlands/global_/dyadic.py contains 42 lines and 1,383 bytes. Source SHA-256 begins 9bacd8ed4d652c2d.

Surface and ownership. API classification: API-manifested implementation owner. It owns 1 manifest card: 0 stable, 0 beta, and 1 research. Linked cards: dyadic_sign_diagnostic. Its __all__ declaration contains 2 names: dyadic_sign_diagnostic, DyadicSignDiagnostic.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (1): komorebi_math.certificates. Reverse package dependents (1): komorebi_math.langlands.global_. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.19.13 komorebi_math.langlands.global_.hecke

Purpose and source. Finite Hecke packet decompositions and support transport models. It is an ordinary module in the langlands family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/langlands/global_/hecke.py contains 137 lines and 5,783 bytes. Source SHA-256 begins 147eea0cf592a9b9.

Surface and ownership. API classification: API-manifested implementation owner. It owns 2 manifest cards: 0 stable, 0 beta, and 2 research. Linked cards: FiniteHeckePackets, SupportComparison. Its __all__ declaration contains 3 names: FiniteHeckePackets, HeckePacket, SupportComparison.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (2): komorebi_math.certificates, komorebi_math.exceptions. Reverse package dependents (1): komorebi_math.langlands.global_. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.19.14 komorebi_math.langlands.global_.patching

Purpose and source. Finite gate models for patched support and characteristic-zero fibers. It is an ordinary module in the langlands family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/langlands/global_/patching.py contains 69 lines and 2,657 bytes. Source SHA-256 begins 6bb134c099c63852.

Surface and ownership. API classification: API-manifested implementation owner. It owns 1 manifest card: 0 stable, 0 beta, and 1 research. Linked cards: PatchingSupportReport. Its __all__ declaration contains 1 name: PatchingSupportReport.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (1): komorebi_math.certificates. Reverse package dependents (1): komorebi_math.langlands.global_. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.19.15 komorebi_math.langlands.local

Purpose and source. Executable finite models for tame and wild local packet geometry. It is the package initializer for the langlands family. The ledger classifies it as an importable package facade outside API manifests. Source src/komorebi_math/langlands/local/__init__.py contains 63 lines and 1,682 bytes. Source SHA-256 begins cce8b2f7cd922550.

Surface and ownership. API classification: importable package module outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its __all__ declaration contains 24 names: additive_character, AlternatingForm, EnvelopeRecovery, FEATURE_KEY, FiniteField, FiniteFieldElement, FrobeniusOrbit, gauss_sum, GaussSumResult, HeisenbergElement, and 14 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (9): komorebi_math._planned, komorebi_math.langlands.local.bridge, komorebi_math.langlands.local.envelopes, komorebi_math.langlands.local.finite_fields, komorebi_

math.langlands.local.gauss, komorebi_math.langlands.local.heisenberg, and 3 more in the machine-readable ledger. Reverse package dependents (5): komorebi_math, komorebi_math.cli.main, komorebi_math.langlands, komorebi_math.langlands.ramification.towers, komorebi_math.verify.langlands. Examples (2): ramification_completion.py, tame_heisenberg.py. Tests or test support files (3): test_langlands_local_fields_tame_gauss.py, test_langlands_local_heisenberg_symplectic.py, test_langlands_ramification_global_completion.py. Documentation files with literal module or public-route references (1): PUBLIC_API.md. AST parsing and runtime import both passed in the recorded module audit.

D.2.19.16 komorebi_math.langlands.local.bridge

Purpose and source. Finite packet bridge and holonomy diagnostics. It is an ordinary module in the langlands family. The ledger classifies it as unmanifested implementation support. Source src/komorebi_math/langlands/local/bridge.py contains 74 lines and 2,683 bytes. Source SHA-256 begins d602ecb7891d84c6.

Surface and ownership. API classification: importable package module outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its __all__ declaration contains 2 names: LocalPacketBridge, SymplecticBridge.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (6): komorebi_math.certificates, komorebi_math.deformation.complexes, komorebi_math.exceptions, komorebi_math.langlands.local.symplectic, komorebi_math.nonlinear, komorebi_math.nonlinear.holonomy. Reverse package dependents (1): komorebi_math.langlands.local. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.19.17 komorebi_math.langlands.local.envelopes

Purpose and source. Projective intertwiner envelopes and commutator forms. It is an ordinary module in the langlands family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/langlands/local/envelopes.py contains 104 lines and 4,663 bytes. Source SHA-256 begins 4b6469345802df9f.

Surface and ownership. API classification: API-manifested implementation owner. It owns 1 manifest card: 0 stable, 0 beta, and 1 research. Linked cards: ProjectiveIntertwinerEnvelope. Its __all__ declaration contains 2 names: EnvelopeRecovery, ProjectiveIntertwinerEnvelope.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (4): komorebi_math._number_theory, komorebi_math.certificates, komorebi_math.exceptions, komorebi_math.langlands.local.symplectic. Reverse package dependents (1): komorebi_math.langlands.local. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.19.18 komorebi_math.langlands.local.finite_fields

Purpose and source. Small exact finite fields for local packet computations. It is an ordinary module in the langlands family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/langlands/local/finite_fields.py contains 365 lines and 13,731 bytes. Source SHA-256 begins 078140eafd796083.

Surface and ownership. API classification: API-manifested implementation owner. It owns 1 manifest card: 0 stable, 0 beta, and 1 research. Linked cards: FiniteField. Its __all__ declaration contains 3 names: FiniteField, FiniteFieldElement, is_irreducible_polynomial.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (2): komorebi_math._number_theory, komorebi_math.exceptions. Reverse package dependents (2): komorebi_math.langlands.local, komorebi_math.langlands.local.gauss. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.19.19 `komorebi_math.langlands.local.gauss`

Purpose and source. Finite field Gauss sums and phase certificates. It is an ordinary module in the `langlands` family. The ledger classifies it as a public implementation owner. Source `src/komorebi_math/langlands/local/gauss.py` contains 79 lines and 2,595 bytes. Source SHA-256 begins `a09ddf5c664e7017`.

Surface and ownership. API classification: API-manifested implementation owner. It owns 1 manifest card: 0 stable, 0 beta, and 1 research. Linked cards: gauss_sum. Its `__all__` declaration contains 4 names: `additive_character`, `gauss_sum`, `GaussSumResult`, `multiplicative_character`.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (3): `komorebi_math.certificates`, `komorebi_math.exceptions`, `komorebi_math.langlands.local.finite_fields`. Reverse package dependents (1): `komorebi_math.langlands.local`. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.19.20 `komorebi_math.langlands.local.heisenberg`

Purpose and source. Finite Heisenberg representations and isotropic selectors. It is an ordinary module in the `langlands` family. The ledger classifies it as a public implementation owner. Source `src/komorebi_math/langlands/local/heisenberg.py` contains 175 lines and 7,849 bytes. Source SHA-256 begins `489d9922de7922c5`.

Surface and ownership. API classification: API-manifested implementation owner. It owns 1 manifest card: 0 stable, 0 beta, and 1 research. Linked cards: HeisenbergModel. Its `__all__` declaration contains 2 names: `HeisenbergElement`, `HeisenbergModel`.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (3): `komorebi_math._number_theory`, `komorebi_math.certificates`, `komorebi_math.exceptions`. Reverse package dependents (1): `komorebi_math.langlands.local`. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.19.21 `komorebi_math.langlands.local.symplectic`

Purpose and source. Finite alternating forms, radicals, and dyadic quadratic refinements. It is an ordinary module in the `langlands` family. The ledger classifies it as a public implementation owner. Source `src/komorebi_math/langlands/local/symplectic.py` contains 174 lines and 7,028 bytes. Source SHA-256 begins `83e286a4dadaf4d2`.

Surface and ownership. API classification: API-manifested implementation owner. It owns 2 manifest cards: 0 stable, 0 beta, and 2 research. Linked cards: AlternatingForm, QuadraticRefinement. Its `__all__` declaration contains 2 names: `AlternatingForm`, `QuadraticRefinement`.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (5): `komorebi_math._number_theory`, `komorebi_math.certificates`, `komorebi_math.deformation.complexes`, `komorebi_math.exceptions`, `komorebi_math.nonlinear.lifting`. Reverse package dependents (3): `komorebi_math.langlands.local`, `komorebi_math.langlands.local.bridge`, `komorebi_math.langlands.local.envelopes`. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.19.22 `komorebi_math.langlands.local.tame`

Purpose and source. Finite tame Frobenius packets and monodromy chains. It is an ordinary module in the `langlands` family. The ledger classifies it as a public implementation owner. Source `src/komorebi_math/langlands/local/tame.py` contains 277 lines and 9,448 bytes. Source SHA-256 begins `63ba77e6127a5e13`.

Surface and ownership. API classification: API-manifested implementation owner. It owns 2 manifest cards: 0 stable, 0 beta, and 2 research. Linked cards: FrobeniusOrbit, TameWeilPacket. Its `__all__` declaration contains 6 names: `FrobeniusOrbit`, `regular_orbit_count`, `SpecialMonodromyBlock`, `tame_tensor_refinement`, `TameWeilPacket`, `TensorRefinement`.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (3): `komorebi_math._number_theory`, `komorebi_math.certificates`, `komorebi_math.exceptions`. Reverse package

dependents (1): komorebi_math.langlands.local. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.19.23 komorebi_math.langlands.ramification

Purpose and source. Finite ramification Clifford tower diagnostics. It is the package initializer for the langlands family. The ledger classifies it as an importable package facade outside API manifests. Source src/komorebi_math/langlands/ramification/__init__.py contains 38 lines and 806 bytes. Source SHA-256 begins de860a4740cd27ed.

Surface and ownership. API classification: importable package module outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its __all__ declaration contains 7 names: FEATURE_KEY, ProjectiveMultiplicityPacket, RamificationCliffordTower, RamificationStage, RamificationTowerReport, require, status.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (3): komorebi_math._planned, komorebi_math.langlands.ramification.towers, komorebi_math.registry. Reverse package dependents (3): komorebi_math, komorebi_math.langlands, komorebi_math.verify.langlands. Examples (1): ramification_completion.py. Tests or test support files (1): test_langlands_ramification_global_completion.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.19.24 komorebi_math.langlands.ramification.towers

Purpose and source. Finite ramification packet and Clifford tower diagnostics. It is an ordinary module in the langlands family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/langlands/ramification/towers.py contains 188 lines and 8,364 bytes. Source SHA-256 begins 9efdf00432bb3b62.

Surface and ownership. API classification: API-manifested implementation owner. It owns 3 manifest cards: 0 stable, 0 beta, and 3 research. Linked cards: ProjectiveMultiplicityPacket, RamificationCliffordTower, RamificationStage. Its __all__ declaration contains 4 names: ProjectiveMultiplicityPacket, RamificationCliffordTower, RamificationStage, RamificationTowerReport.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (4): komorebi_math._number_theory, komorebi_math.certificates, komorebi_math.exceptions, komorebi_math.langlands.local. Reverse package dependents (1): komorebi_math.langlands.ramification. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.20 Expression: 7 modules

D.2.20.1 komorebi_math.expression

Purpose and source. Research-tier typed finite expression calculus. It is the package initializer for the expression family. The ledger classifies it as an importable package facade outside API manifests. Source src/komorebi_math/expression/__init__.py contains 187 lines and 3,921 bytes. Source SHA-256 begins a4526fb71a56d603.

Surface and ownership. API classification: importable package module outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its __all__ declaration contains 83 names: add, affine_substitution, antiderivative, apply, Apply, boundary_functional, check_expression, CheckedExpression, compile_expression, compose, and 73 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (6): komorebi_math.expression.certification, komorebi_math.expression.engine, komorebi_math.expression.model, komorebi_math.expression.planning, komorebi_math.expression.polynomial, komorebi_math.expression.portable. Reverse package dependents (2): komorebi_math, komorebi_math.verify.expression. Examples (1): block20_typed_expression_worked_example.py. Tests or test support files (1): test_block20_expression_calculus.py. Documentation files with literal module or public-route references (1): PUBLIC_API.md. AST parsing and runtime import both passed in the recorded module audit.

D.2.20.2 komorebi_math.expression.certification

Purpose and source. Sound certificate composition for the research expression calculus. It is an ordinary module in the expression family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/expression/certification.py contains 313 lines and 11,084 bytes. Source SHA-256 begins 4348615486a61a71.

Surface and ownership. API classification: API-manifested implementation owner. It owns 7 manifest cards: 0 stable, 0 beta, and 7 research. Linked cards: compose_certificates, exact_internal_certificate, exactness_strength, ExpressionBinding, NormalForm, RewriteStep, weakest_exactness. Its __all__ declaration contains 7 names: compose_certificates, exact_internal_certificate, exactness_strength, ExpressionBinding, NormalForm, RewriteStep, weakest_exactness.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (2): komorebi_math.certificates, komorebi_math.exceptions. Reverse package dependents (2): komorebi_math.expression, komorebi_math.expression.engine. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.20.3 komorebi_math.expression.engine

Purpose and source. Checking, exact normalization, certified rewriting, and execution plans. It is an ordinary module in the expression family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/expression/engine.py contains 1,744 lines and 71,584 bytes. Source SHA-256 begins 5ae39e027b7f9c33.

Surface and ownership. API classification: API-manifested implementation owner. It owns 15 manifest cards: 0 stable, 0 beta, and 15 research. Linked cards: check_expression, CheckedExpression, compile_expression, EquivalenceResult, equivalent, evaluate_expression, exact_binding, execute_plan, ExpressionPlan, ExpressionResult, and 5 more in the machine-readable ledger. Its __all__ declaration contains 15 names: check_expression, CheckedExpression, compile_expression, EquivalenceResult, equivalent, evaluate_expression, exact_binding, execute_plan, ExpressionPlan, ExpressionResult, and 5 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (8): komorebi_math.certificates, komorebi_math.coordinates.exact, komorebi_math.exceptions, komorebi_math.expression.certification, komorebi_math.expression.model, komorebi_math.expression.planning, and 2 more in the machine-readable ledger. Reverse package dependents (2): komorebi_math.expression, komorebi_math.expression.portable. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.20.4 komorebi_math.expression.model

Purpose and source. Typed immutable expressions for finite exact rational computations. It is an ordinary module in the expression family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/expression/model.py contains 712 lines and 24,960 bytes. Source SHA-256 begins e01883acc993da14.

Surface and ownership. API classification: API-manifested implementation owner. It owns 28 manifest cards: 0 stable, 0 beta, and 28 research. Linked cards: add, Apply, apply, Compose, compose, exact_map, exact_value, ExactMapLiteral, ExactValueLiteral, Expression, and 18 more in the machine-readable ledger. Its __all__ declaration contains 30 names: add, Apply, apply, compose, Compose, CoordinateSpace, exact_map, exact_value, ExactMapLiteral, ExactValueLiteral, and 20 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (2): komorebi_math.coordinates.exact, komorebi_math.exceptions. Reverse package dependents (4): komorebi_math.expression, komorebi_math.expression.engine, komorebi_math.expression.polynomial, komorebi_math.expression.portable. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.20.5 komorebi_math.expression.planning

Purpose and source. Exact dense matrix-chain planning independent of expression evaluation. It is an ordinary module in the expression family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/expression/planning.py contains 400 lines and 15,157 bytes. Source SHA-256 begins cfa5341a696cabed.

Surface and ownership. API classification: API-manifested implementation owner. It owns 5 manifest cards: 0 stable, 0 beta, and 5 research. Linked cards: matrix_chain_multiply, matrix_chain_parenthesization, matrix_chain_plan, MatrixChainPlan, MatrixChainTree. Its __all__ declaration contains 5 names: matrix_chain_multiply, matrix_chain_parenthesization, matrix_chain_plan, MatrixChainPlan, MatrixChainTree.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (3): komorebi_math.certificates, komorebi_math.coordinates.exact, komorebi_math.exceptions. Reverse package dependents (2): komorebi_math.expression, komorebi_math.expression.engine. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.20.6 komorebi_math.expression.polynomial

Purpose and source. Replayable exact polynomial primitives for the expression calculus. It is an ordinary module in the expression family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/expression/polynomial.py contains 624 lines and 21,239 bytes. Source SHA-256 begins 73a4a594e4c0f26c.

Surface and ownership. API classification: API-manifested implementation owner. It owns 15 manifest cards: 0 stable, 0 beta, and 15 research. Linked cards: affine_substitution, antiderivative, boundary_functional, constant_injection, coordinate_change, derivative, evaluation, finite_difference, interval_integration, PolynomialPrimitive, and 5 more in the machine-readable ledger. Its __all__ declaration contains 15 names: affine_substitution, antiderivative, boundary_functional, constant_injection, coordinate_change, derivative, evaluation, finite_difference, interval_integration, PolynomialPrimitive, and 5 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (4): komorebi_math.certificates, komorebi_math.coordinates.exact, komorebi_math.exceptions, komorebi_math.expression.model. Reverse package dependents (3): komorebi_math.expression, komorebi_math.expression.engine, komorebi_math.expression.portable. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.20.7 komorebi_math.expression.portable

Purpose and source. Strict portable JSON for finite typed expressions. It is an ordinary module in the expression family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/expression/portable.py contains 1,025 lines and 36,065 bytes. Source SHA-256 begins 366c25d81842e91c.

Surface and ownership. API classification: API-manifested implementation owner. It owns 7 manifest cards: 0 stable, 0 beta, and 7 research. Linked cards: expression_digest, expression_from_dict, expression_from_json, expression_to_dict, expression_to_json, type_from_dict, type_to_dict. Its __all__ declaration contains 11 names: expression_digest, expression_from_dict, expression_from_json, EXPRESSION_NODE_VERSION, EXPRESSION_SCHEMA, EXPRESSION_SCHEMA_VERSION, expression_to_dict, expression_to_json, EXPRESSION_TYPE_VERSION, type_from_dict, and 1 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (4): komorebi_math.exceptions, komorebi_math.expression.engine, komorebi_math.expression.model, komorebi_math.expression.polynomial. Reverse package dependents (2): komorebi_math.expression, komorebi_math.expression.engine. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.21 Export: 8 modules

D.2.21.1 komorebi_math.export

Purpose and source. Portable data, report, LaTeX, and PDF export tools. It is the package initializer for the export family. The ledger classifies it as a canonical API namespace facade. Source src/komorebi_math/export/__init__.py contains 131 lines and 2,743 bytes. Source SHA-256 begins 46407a9f8b1ddb00.

Surface and ownership. API classification: canonical public namespace without implementation ownership. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its __all__ declaration contains 58 names: available_export_formats, available_latex_engines, certificate_document, CodeBlock, compile_latex, document_from_data, DocumentMetadata, dumps_portable, Equation, export_adapters, and 48 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (7): komorebi_math.export.latex, komorebi_math.export.model, komorebi_math.export.persistence, komorebi_math.export.renderers, komorebi_math.export.report, komorebi_math.export.serialization, and 1 more in the machine-readable ledger. Reverse package dependents (8): komorebi_math, komorebi_math.cli.main, komorebi_math.stable, komorebi_math.verify.coordinates, komorebi_math.verify.export, komorebi_math.verify.generating, and 2 more in the machine-readable ledger. Examples (5): block21_public_capability_tour.py, custom_generating_functions.py, export_reports.py, portable_nonlinear_state.py, scalable_nonlinear.py. Tests or test support files (16): test_block10_execution_persistence.py, test_block11_streaming_migrations.py, test_block15_persistence.py, test_block15_polyphase_refinement.py, test_block15_stable_api.py, and 11 more in the machine-readable ledger. Documentation files with literal module or public-route references (8): API_COMPATIBILITY_0.11.md, API_COMPATIBILITY_0.12.md, API_COMPATIBILITY_0.13.md, BLOCK_10_RELEASE.md, EXPORT_AND_LATEX.md, and 3 more in the machine-readable ledger. AST parsing and runtime import both passed in the recorded module audit.

D.2.21.2 komorebi_math.export.latex

Purpose and source. LaTeX compilation with explicit engine discovery. It is an ordinary module in the export family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/export/latex.py contains 267 lines and 8,921 bytes. Source SHA-256 begins 6732665ea6282003.

Surface and ownership. API classification: API-manifested implementation owner. It owns 10 manifest cards: 0 stable, 0 beta, and 10 research. Linked cards: available_latex_engines, compile_latex, LatexCompileOptions, LatexEngine, available_latex_engines, compile_latex, LatexCompilation, LatexCompileOptions, LatexEngine, resolve_latex_engine. Its __all__ declaration contains 6 names: available_latex_engines, compile_latex, LatexCompilation, LatexCompileOptions, LatexEngine, resolve_latex_engine.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (2): komorebi_math.certificates, komorebi_math.exceptions. Reverse package dependents (5): komorebi_math, komorebi_math.cli.main, komorebi_math.export, komorebi_math.export.writers, komorebi_math.verify.export. Examples (1): export_reports.py. Tests or test support files (2): test_export_cli.py, test_export_files.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.21.3 komorebi_math.export.model

Purpose and source. Document objects for portable mathematical exports. It is an ordinary module in the export family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/export/model.py contains 244 lines and 6,917 bytes. Source SHA-256 begins 14821695790ebcf9.

Surface and ownership. API classification: API-manifested implementation owner. It owns 18 manifest cards: 4 stable, 0 beta, and 14 research. Linked cards: DocumentMetadata, Equation, MathDocument, Paragraph, DocumentMetadata, Equation, MathDocument, Table, CodeBlock, DocumentMetadata, and 8 more in the machine-readable ledger. Its __all__ declaration contains 11 names: CodeBlock, DocumentBlock, DocumentMetadata, Equation, Figure, Heading, ListBlock, MathDocument, Paragraph, RawLatex, and 1 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (1): `komorebi_math.exceptions`. Reverse package dependents (8): `komorebi_math`, `komorebi_math.export`, `komorebi_math.export.renderers`, `komorebi_math.export.report`, `komorebi_math.export.writers`, `komorebi_math.stable`, and 2 more in the machine-readable ledger. Examples (1): `export_reports.py`. Tests or test support files (2): `test_export_files.py`, `test_export_renderers.py`. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.21.4 `komorebi_math.export.persistence`

Purpose and source. Versioned, digest-checked persistence for finite Komorebi objects. It is an ordinary module in the `export` family. The ledger classifies it as a public implementation owner. Source `src/komorebi_math/export/persistence.py` contains 887 lines and 32,742 bytes. Source SHA-256 begins `0044e9c7eb95677d`.

Surface and ownership. API classification: API-manifested implementation owner. It owns 48 manifest cards: 11 stable, 13 beta, and 24 research. Linked cards: dumps_portable, from_portable_data, load_portable, loads_portable, portable_codecs, portable_digest, portable_migrations, PortableCodec, PortableMigration, save_portable, and 38 more in the machine-readable ledger. Its `__all__` declaration contains 15 names: `dumps_portable`, `from_portable_data`, `load_portable`, `loads_portable`, `portable_codecs`, `portable_digest`, `portable_migrations`, `PORTABLE_SCHEMA`, `PORTABLE_SCHEMA_VERSION`, `PortableCodec`, and 5 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (13): `komorebi_math._version`, `komorebi_math.certificates`, `komorebi_math.coordinates.portable`, `komorebi_math.deformation.forests`, `komorebi_math.exceptions`, `komorebi_math.generating.portable`, and 7 more in the machine-readable ledger. Reverse package dependents (19): `komorebi_math`, `komorebi_math.beta`, `komorebi_math.coordinates.portable`, `komorebi_math.export`, `komorebi_math.generating.portable`, `komorebi_math.golden`, and 13 more in the machine-readable ledger. Examples (5): `block16_analysis_synthesis.py`, `block21_public_capability_tour.py`, `custom_generating_functions.py`, `portable_nonlinear_state.py`, `scalable_nonlinear.py`. Tests or test support files (12): `test_block10_execution_persistence.py`, `test_block11_streaming_migrations.py`, `test_block15_persistence.py`, `test_block15_polyphase_refinement.py`, `test_block15_stable_api.py`, and 7 more in the machine-readable ledger. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.21.5 `komorebi_math.export.renderers`

Purpose and source. Text, Markdown, HTML, and LaTeX renderers. It is an ordinary module in the `export` family. The ledger classifies it as a public implementation owner. Source `src/komorebi_math/export/renderers.py` contains 402 lines and 16,487 bytes. Source SHA-256 begins `1a86b12145e02c30`.

Surface and ownership. API classification: API-manifested implementation owner. It owns 12 manifest cards: 2 stable, 0 beta, and 10 research. Linked cards: render_latex, render_markdown, LatexRenderOptions, render_latex, HtmlRenderOptions, latex_escape, latex_label, LatexRenderOptions, render_html, render_latex, and 2 more in the machine-readable ledger. Its `__all__` declaration contains 8 names: `HtmlRenderOptions`, `latex_escape`, `latex_label`, `LatexRenderOptions`, `render_html`, `render_latex`, `render_markdown`, `render_text`.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (1): `komorebi_math.export.model`. Reverse package dependents (7): `komorebi_math`, `komorebi_math.cli.main`, `komorebi_math.export`, `komorebi_math.export.writers`, `komorebi_math.stable`, `komorebi_math.stable_semantics`, and 1 more in the machine-readable ledger. Examples (0): none. Tests or test support files (1): `test_export_renderers.py`. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.21.6 komorebi_math.export.report

Purpose and source. Adapters that turn mathematical values into readable documents. It is an ordinary module in the export family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/export/report.py contains 142 lines and 5,310 bytes. Source SHA-256 begins f303c23f5f9c027f.

Surface and ownership. API classification: API-manifested implementation owner. It owns 5 manifest cards: 0 stable, 0 beta, and 5 research. Linked cards: document_from_data, sequence_document, certificate_document, document_from_data, sequence_document. Its __all__ declaration contains 3 names: certificate_document, document_from_data, sequence_document.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (3): komorebi_math.certificates, komorebi_math.export.model, komorebi_math.export.serialization. Reverse package dependents (4): komorebi_math, komorebi_math.cli.main, komorebi_math.export, komorebi_math.export.writers. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.21.7 komorebi_math.export.serialization

Purpose and source. Normalization helpers for portable data export. It is an ordinary module in the export family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/export/serialization.py contains 351 lines and 12,158 bytes. Source SHA-256 begins 20d4f7cf7754519d.

Surface and ownership. API classification: API-manifested implementation owner. It owns 8 manifest cards: 0 stable, 0 beta, and 8 research. Linked cards: from_serializable, to_serializable, export_adapters, Exportable, from_serializable, register_export_adapter, table_rows, to_serializable. Its __all__ declaration contains 6 names: export_adapters, Exportable, from_serializable, register_export_adapter, table_rows, to_serializable.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (2): komorebi_math.certificates, komorebi_math.exceptions. Reverse package dependents (6): komorebi_math, komorebi_math.cli.main, komorebi_math.export, komorebi_math.export.report, komorebi_math.export.writers, komorebi_math.verify.export. Examples (0): none. Tests or test support files (3): test_block10_execution_persistence.py, test_export_serialization.py, test_nonlinear_snapshots_persistence.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.21.8 komorebi_math.export.writers

Purpose and source. File writers for data, reports, LaTeX, and compiled PDFs. It is an ordinary module in the export family. The ledger classifies it as a public implementation owner. Source src/komorebi_math/export/writers.py contains 518 lines and 18,038 bytes. Source SHA-256 begins 4ac13bea553cd032.

Surface and ownership. API classification: API-manifested implementation owner. It owns 19 manifest cards: 0 stable, 0 beta, and 19 research. Linked cards: available_export_formats, export_bundle, export_data, export_document, export_many, ExportBundleResult, ExportFormat, ExportOptions, ExportResult, available_export_formats, and 9 more in the machine-readable ledger. Its __all__ declaration contains 10 names: available_export_formats, export_bundle, export_data, export_document, export_many, ExportBundleResult, ExportFormat, ExportOptions, ExportResult, infer_export_format.

Dependencies and evidence. Direct dependency profile: optional yaml extra, required NumPy dependency. Direct package dependencies (7): komorebi_math.certificates, komorebi_math.exceptions, komorebi_math.export.latex, komorebi_math.export.model, komorebi_math.export.renderers, komorebi_math.export.report, and 1 more in the machine-readable ledger. Reverse package dependents (4): komorebi_math, komorebi_math.cli.main, komorebi_math.export, komorebi_math.verify.export. Examples (1): export_reports.py. Tests or test support files (1): test_export_files.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.22 Extensions: 3 modules

D.2.22.1 `komorebi_math.extensions`

Purpose and source. Extension API and runtime discovery. It is the package initializer for the `extensions` family. The ledger classifies it as an extension infrastructure module. Source `src/komorebi_math/extensions/__init__.py` contains 6 lines and 254 bytes. Source SHA-256 begins `f58b0afe8ae4a746`.

Surface and ownership. API classification: extension infrastructure outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its `__all__` declaration contains 4 names: `discover_extensions`, `ENTRY_POINT_GROUP`, `ExtensionLoadResult`, `ExtensionSpec`.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (2): `komorebi_math.extensions.api`, `komorebi_math.extensions.discovery`. Reverse package dependents (1): `komorebi_math.cli.main`. Examples (0): none. Tests or test support files (1): `test_unified_extensions_cli.py`. Documentation files with literal module or public-route references (2): `EXTENSION_API.md`, `PUBLIC_API.md`. AST parsing and runtime import both passed in the recorded module audit.

D.2.22.2 `komorebi_math.extensions.api`

Purpose and source. Public extension contract. It is an ordinary module in the `extensions` family. The ledger classifies it as an extension infrastructure module. Source `src/komorebi_math/extensions/api.py` contains 26 lines and 687 bytes. Source SHA-256 begins `e0a5f52cd0e4dbd4`.

Surface and ownership. API classification: extension infrastructure outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Static analysis found 1 top-level public definition: `ExtensionSpec`.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (0): none. Reverse package dependents (2): `komorebi_math.extensions`, `komorebi_math.extensions.discovery`. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.22.3 `komorebi_math.extensions.discovery`

Purpose and source. Extension discovery through standard Python entry points. It is an ordinary module in the `extensions` family. The ledger classifies it as an extension infrastructure module. Source `src/komorebi_math/extensions/discovery.py` contains 41 lines and 1,475 bytes. Source SHA-256 begins `93b3ea91e482d33d`.

Surface and ownership. API classification: extension infrastructure outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Static analysis found 2 top-level public definitions: `discover_extensions`, `ExtensionLoadResult`.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (1): `komorebi_math.extensions.api`. Reverse package dependents (1): `komorebi_math.extensions`. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.23 Cli: 3 modules

D.2.23.1 komorebi_math.cli

Purpose and source. Command-line package. It is the package initializer for the cli family. The ledger classifies it as a command line implementation module. Source src/komorebi_math/cli/__init__.py contains 1 lines and 28 bytes. Source SHA-256 begins d79725da2dfaf0df.

Surface and ownership. API classification: CLI implementation outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Static analysis found no top-level public definitions.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (0): none. Reverse package dependents (0): none. Examples (8): Test-KomorebiBlock10.ps1, Test-KomorebiBlock11.ps1, Test-KomorebiBlock12.ps1, Test-KomorebiBlock13.ps1, Test-KomorebiBlock14.ps1, and 3 more in the machine-readable ledger. Tests or test support files (10): test_block10_cli.py, test_block11_cli.py, test_block12_cli_api.py, test_block13_cli_api.py, test_block14_cli_api.py, and 5 more in the machine-readable ledger. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.23.2 komorebi_math.cli.contracts

Purpose and source. Frozen command-line names covered by the release contract. It is an ordinary module in the cli family. The ledger classifies it as a command line implementation module. Source src/komorebi_math/cli/contracts.py contains 59 lines and 1,051 bytes. Source SHA-256 begins 39011c0addae62b5.

Surface and ownership. API classification: CLI implementation outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its __all__ declaration contains 2 names: API_CONTRACT_MODES, VERIFICATION_SUITE_NAMES.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (0): none. Reverse package dependents (2): komorebi_math.cli.main, komorebi_math.release.contract. Examples (0): none. Tests or test support files (1): test_block16_release_contract.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.23.3 komorebi_math.cli.main

Purpose and source. Command-line interface for package inspection and verification. It is an ordinary module in the cli family. The ledger classifies it as a command line implementation module. Source src/komorebi_math/cli/main.py contains 1,651 lines and 67,103 bytes. Source SHA-256 begins a61a4e5d81a838e7.

Surface and ownership. API classification: CLI implementation outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Static analysis found 48 top-level public definitions: build_parser, command_api_check, command_api_snapshot, command_completion, command_coordinates_basis, command_coordinates_catalog, command_coordinates_combinatorial, command_coordinates_euler_fibonacci, command_coordinates_formal, command_coordinates_model, and 38 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (39): komorebi_math.arithmetic, komorebi_math.backends, komorebi_math.cli.contracts, komorebi_math.compatibility, komorebi_math.coordinates, komorebi_math.coordinates.basis, and 33 more in the machine-readable ledger. Reverse package dependents (1): komorebi_math.__main__. Examples (8): Test-KomorebiBlock10.ps1, Test-KomorebiBlock11.ps1, Test-KomorebiBlock12.ps1, Test-KomorebiBlock13.ps1, Test-KomorebiBlock14.ps1, and 3 more in the machine-readable ledger. Tests or test support files (9): test_block10_cli.py, test_block11_cli.py, test_block12_cli_api.py, test_block13_cli_api.py, test_block14_cli_api.py, and 4 more in the machine-readable ledger. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.24 Release: 8 modules

D.2.24.1 `komorebi_math.release`

Purpose and source. Release, upgrade, evidence, documentation, readiness, and GPU gates. It is the package initializer for the `release` family. The ledger classifies it as a release infrastructure module. Source `src/komorebi_math/release/__init__.py` contains 12 lines and 670 bytes. Source SHA-256 begins `93d7ade753bb79de`.

Surface and ownership. API classification: release tooling outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its `__all__` declaration contains 41 names: `ApiTier`, `BetaReadinessReport`, `check_beta_readiness`, `check_release_contract`, `check_stable_documentation`, `check_upgrade_contract`, `check_validation_evidence`, `ContractArtifact`, `create_stable_documentation_contract`, `current_runtime_record`, and 31 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (6): `komorebi_math.release.contract`, `komorebi_math.release.documentation`, `komorebi_math.release.evidence`, `komorebi_math.release.gpu`, `komorebi_math.release.readiness`, `komorebi_math.release.upgrade`. Reverse package dependents (4): `komorebi_math.cli.main`, `komorebi_math.qualification`, `komorebi_math.verify.beta`, `komorebi_math.verify.release`. Examples (0): none. Tests or test support files (6): `test_block16_gpu_qualification.py`, `test_block16_release_contract.py`, `test_block17_documentation.py`, `test_block17_evidence.py`, `test_block17_release_contract.py`, and 1 more in the machine-readable ledger. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.24.2 `komorebi_math.release._gpu_worker`

Purpose and source. Private isolated worker for `komorebi_math.release.gpu`. It is an ordinary module in the `release` family. The ledger classifies it as an internal support module. Source `src/komorebi_math/release/_gpu_worker.py` contains 112 lines and 5,194 bytes. Source SHA-256 begins `45d51b0da5eeadce`.

Surface and ownership. API classification: internal by module name. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Static analysis found 1 top-level public definition: `main`.

Dependencies and evidence. Direct dependency profile: optional CuPy accelerator outside the declared project extras, optional `torch` or `autodiff` extra, required NumPy dependency. Direct package dependencies (1): `komorebi_math.algorithms`. Reverse package dependents (0): none. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.24.3 `komorebi_math.release.contract`

Purpose and source. Machine-readable contract for the reference Komorebi Math beta. It is an ordinary module in the `release` family. The ledger classifies it as a release infrastructure module. Source `src/komorebi_math/release/contract.py` contains 94 lines and 10,362 bytes. Source SHA-256 begins `80f205ddc875538f`.

Surface and ownership. API classification: release tooling outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its `__all__` declaration contains 11 names: `ApiTier`, `check_release_contract`, `ContractArtifact`, `current_runtime_record`, `DEFAULT_RELEASE_CONTRACT`, `load_release_contract`, `QualificationCheck`, `RELEASE_CONTRACT_SCHEMA`, `RELEASE_CONTRACT_SCHEMA_VERSION`, `ReleaseContract`, and 1 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (11): `komorebi_math._version`, `komorebi_math.certificates`, `komorebi_math.cli.contracts`, `komorebi_math.compatibility`, `komorebi_math.exceptions`, `komorebi_math.persistence_contract`, and 5 more in the machine-readable ledger. Reverse package dependents (2): `komorebi_math.release`, `komorebi_math.release.readiness`. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.24.4 `komorebi_math.release.documentation`

Purpose and source. Exact documentation coverage for the frozen stable namespace. It is an ordinary module in the `release` family. The ledger classifies it as a release infrastructure module. Source `src/komorebi_math/release/documentation.py` contains 342 lines and 12,229 bytes. Source SHA-256 begins `9158244b43886cd5`.

Surface and ownership. API classification: release tooling outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its `__all__` declaration contains 9 names: `check_stable_documentation`, `create_stable_documentation_contract`, `DEFAULT_STABLE_DOCUMENTATION`, `load_stable_documentation`, `STABLE_DOCUMENTATION_SCHEMA`, `STABLE_DOCUMENTATION_SCHEMA_VERSION`, `StableDocumentationContract`, `StableDocumentationEntry`, `StableDocumentationReport`.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (4): `komorebi_math._version`, `komorebi_math.certificates`, `komorebi_math.compatibility`, `komorebi_math.exceptions`. Reverse package dependents (3): `komorebi_math.release`, `komorebi_math.release.contract`, `komorebi_math.release.readiness`. Examples (0): none. Tests or test support files (1): `test_block17_documentation.py`. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.24.5 `komorebi_math.release.evidence`

Purpose and source. Digest-checked executed-platform evidence for the Komorebi Math 1.0 beta. It is an ordinary module in the `release` family. The ledger classifies it as a release infrastructure module. Source `src/komorebi_math/release/evidence.py` contains 341 lines and 12,955 bytes. Source SHA-256 begins `c471af87d4ef017e`.

Surface and ownership. API classification: release tooling outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its `__all__` declaration contains 8 names: `check_validation_evidence`, `DEFAULT_VALIDATION_EVIDENCE`, `load_validation_evidence`, `VALIDATION_EVIDENCE_SCHEMA`, `VALIDATION_EVIDENCE_SCHEMA_VERSION`, `ValidationEvidence`, `ValidationEvidenceRecord`, `ValidationEvidenceReport`.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (4): `komorebi_math._version`, `komorebi_math.certificates`, `komorebi_math.exceptions`, `komorebi_math.release.upgrade`. Reverse package dependents (3): `komorebi_math.release`, `komorebi_math.release.contract`, `komorebi_math.release.readiness`. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.24.6 `komorebi_math.release.gpu`

Purpose and source. Crash-isolated CUDA qualification for optional PyTorch and CuPy paths. It is an ordinary module in the `release` family. The ledger classifies it as a release infrastructure module. Source `src/komorebi_math/release/gpu.py` contains 178 lines and 5,658 bytes. Source SHA-256 begins `4cbc2f52cec7a18f`.

Surface and ownership. API classification: release tooling outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its `__all__` declaration contains 3 names: `GpuBackendResult`, `GpuQualificationReport`, `qualify_gpu`.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (2): `komorebi_math.certificates`, `komorebi_math.exceptions`. Reverse package dependents (2): `komorebi_math.release`, `komorebi_math.release.readiness`. Examples (0): none. Tests or test support files (1): `test_block16_gpu_qualification.py`. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.24.7 komorebi_math.release.readiness

Purpose and source. Aggregate public-beta and release-candidate readiness gates. It is an ordinary module in the release family. The ledger classifies it as a release infrastructure module. Source src/komorebi_math/release/readiness.py contains 44 lines and 3,702 bytes. Source SHA-256 begins 4b1c4c219082e6d0.

Surface and ownership. API classification: release tooling outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its __all__ declaration contains 2 names: BetaReadinessReport, check_beta_readiness.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (13): komorebi_math._version, komorebi_math.api, komorebi_math.api.v1, komorebi_math.certificates, komorebi_math.release.contract, komorebi_math.release.documentation, and 7 more in the machine-readable ledger. Reverse package dependents (1): komorebi_math.release. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.24.8 komorebi_math.release.upgrade

Purpose and source. Cross-version upgrade contract for the Komorebi Math 1.0 line. It is an ordinary module in the release family. The ledger classifies it as a release infrastructure module. Source src/komorebi_math/release/upgrade.py contains 137 lines and 9,819 bytes. Source SHA-256 begins 8fce6479bd8875e4.

Surface and ownership. API classification: release tooling outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its __all__ declaration contains 8 names: check_upgrade_contract, DEFAULT_UPGRADE_CONTRACT, load_upgrade_contract, UpgradeArtifact, UpgradeCheck, UpgradeContract, UpgradeContractReport, UpgradeFixture.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (7): komorebi_math._version, komorebi_math.certificates, komorebi_math.compatibility, komorebi_math.exceptions, komorebi_math.export.persistence, komorebi_math.persistence_contract, and 1 more in the machine-readable ledger. Reverse package dependents (4): komorebi_math.release, komorebi_math.release.contract, komorebi_math.release.evidence, komorebi_math.release.readiness. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.25 Verify: 28 modules

D.2.25.1 komorebi_math.verify

Purpose and source. Installed package verification entry points. It is the package initializer for the verify family. The ledger classifies it as an installed verification module. Source src/komorebi_math/verify/__init__.py contains 249 lines and 8,639 bytes. Source SHA-256 begins 569fa314a0d4a8d7.

Surface and ownership. API classification: installed verifier outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its __all__ declaration contains 56 names: run_advanced_nonlinear_verification, run_algorithms_verification, run_analytic_verification, run_arithmetic_verification, run_beta_readiness_verification, run_block10_verification, run_block11_verification, run_block12_verification, run_block13_verification, run_block14_verification, and 46 more in the machine-readable ledger.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (26): komorebi_math.verify.algorithms, komorebi_math.verify.analytic, komorebi_math.verify.arithmetic, komorebi_math.verify.beta, komorebi_math.verify.coordinates, komorebi_math.verify.core, and 20 more in the machine-readable ledger. Reverse package dependents (1): komorebi_math.cli.main. Examples (0): none. Tests or test support files (3): test_block16_release_qualification.py, test_block17_release_contract.py, test_unified_extensions_cli.py. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.25.2 komorebi_math.verify.algorithms

Purpose and source. Installed package verification for structured algorithms. It is an ordinary module in the verify family. The ledger classifies it as an installed verification module. Source src/komorebi_math/verify/algorithms.py contains 150 lines and 4,882 bytes. Source SHA-256 begins b9ca065d0d4af8a4.

Surface and ownership. API classification: installed verifier outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Static analysis found 1 top-level public definition: run_algorithms_verification.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (2): komorebi_math.algorithms, komorebi_math.verify.core. Reverse package dependents (1): komorebi_math.verify. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.25.3 komorebi_math.verify.analytic

Purpose and source. Installed verification for cyclic analytic functions. It is an ordinary module in the verify family. The ledger classifies it as an installed verification module. Source src/komorebi_math/verify/analytic.py contains 104 lines and 3,721 bytes. Source SHA-256 begins d2ccc2ee0e4ae07a.

Surface and ownership. API classification: installed verifier outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its __all__ declaration contains 1 name: run_analytic_verification.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (2): komorebi_math.analytic, komorebi_math.verify.core. Reverse package dependents (1): komorebi_math.verify. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.25.4 komorebi_math.verify.arithmetic

Purpose and source. Installed verification for local arithmetic packets. It is an ordinary module in the verify family. The ledger classifies it as an installed verification module. Source src/komorebi_math/verify/arithmetic.py contains 101 lines and 3,975 bytes. Source SHA-256 begins 7f4960cd813a8e2a.

Surface and ownership. API classification: installed verifier outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its __all__ declaration contains 1 name: run_arithmetic_verification.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (2): komorebi_math.arithmetic, komorebi_math.verify.core. Reverse package dependents (1): komorebi_math.verify. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.25.5 komorebi_math.verify.beta

Purpose and source. Installed verification for the Komorebi Math 1.0 public beta. It is an ordinary module in the verify family. The ledger classifies it as an installed verification module. Source src/komorebi_math/verify/beta.py contains 8 lines and 1,036 bytes. Source SHA-256 begins f1579f4095f62875.

Surface and ownership. API classification: installed verifier outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its __all__ declaration contains 4 names: run_beta_readiness_verification, run_documentation_verification, run_evidence_verification, run_upgrade_verification.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (2): komorebi_math.release, komorebi_math.verify.core. Reverse package dependents (1): komorebi_math.verify. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.25.6 komorebi_math.verify.coordinates

Purpose and source. Installed-package verification for the exact coordinate calculus. It is an ordinary module in the verify family. The ledger classifies it as an installed verification module. Source src/komorebi_math/verify/coordinates.py contains 447 lines and 19,322 bytes. Source SHA-256 begins 82e37ae57211e3d6.

Surface and ownership. API classification: installed verifier outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its __all__ declaration contains 1 name: run_coordinates_verification.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (27): komorebi_math.coordinates, komorebi_math.coordinates.affine, komorebi_math.coordinates.basis, komorebi_math.coordinates.budget, komorebi_math.coordinates.calculus, komorebi_math.coordinates.classical, and 21 more in the machine-readable ledger. Reverse package dependents (1): komorebi_math.verify. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.25.7 komorebi_math.verify.core

Purpose and source. Lightweight installed package verification for the finite core. It is an ordinary module in the verify family. The ledger classifies it as an installed verification module. Source src/komorebi_math/verify/core.py contains 144 lines and 4,892 bytes. Source SHA-256 begins d673945ade70d3cb.

Surface and ownership. API classification: installed verifier outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Static analysis found 3 top-level public definitions: run_core_verification, VerificationCheck, VerificationReport.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (2): komorebi_math.certificates, komorebi_math.core. Reverse package dependents (26): komorebi_math.verify, komorebi_math.verify.algorithms, komorebi_math.verify.analytic, komorebi_math.verify.arithmetic, komorebi_math.verify.beta, komorebi_math.verify.coordinates, and 20 more in the machine-readable ledger. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.25.8 komorebi_math.verify.deformation

Purpose and source. Installed verification for finite deformation and support models. It is an ordinary module in the verify family. The ledger classifies it as an installed verification module. Source src/komorebi_math/verify/deformation.py contains 75 lines and 2,650 bytes. Source SHA-256 begins f5a9e5a74933e080.

Surface and ownership. API classification: installed verifier outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its __all__ declaration contains 1 name: run_deformation_verification.

Dependencies and evidence. Direct dependency profile: optional exact extra, required NumPy dependency. Direct package dependencies (2): komorebi_math.deformation, komorebi_math.verify.core. Reverse package dependents (1): komorebi_math.verify. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.25.9 komorebi_math.verify.dsp

Purpose and source. Installed-package verification for the DSP implementation. It is an ordinary module in the verify family. The ledger classifies it as an installed verification module. Source src/komorebi_math/verify/dsp.py contains 172 lines and 6,115 bytes. Source SHA-256 begins 688c3f4d5b39eb23.

Surface and ownership. API classification: installed verifier outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its __all__ declaration contains 1 name: run_dsp_verification.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (12): `komorebi_math.signals`, `komorebi_math.signals.adaptive`, `komorebi_math.signals.denoise`, `komorebi_math.signals.fft_convolution`, `komorebi_math.signals.filtering`, `komorebi_math.signals.recursive`, and 6 more in the machine-readable ledger. Reverse package dependents (1): `komorebi_math.verify`. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.25.10 `komorebi_math.verify.dynamics`

Purpose and source. Installed package verification for semigroups, probability, and memory. It is an ordinary module in the `verify` family. The ledger classifies it as an installed verification module. Source `src/komorebi_math/verify/dynamics.py` contains 142 lines and 5,020 bytes. Source SHA-256 begins `e8f740da54c4c4be`.

Surface and ownership. API classification: installed verifier outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its `__all__` declaration contains 1 name: `run_dynamics_verification`.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (2): `komorebi_math.dynamics`, `komorebi_math.verify.core`. Reverse package dependents (1): `komorebi_math.verify`. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.25.11 `komorebi_math.verify.export`

Purpose and source. Installed verification suite for portable exports and LaTeX output. It is an ordinary module in the `verify` family. The ledger classifies it as an installed verification module. Source `src/komorebi_math/verify/export.py` contains 118 lines and 4,497 bytes. Source SHA-256 begins `9d6757c71e0df993`.

Surface and ownership. API classification: installed verifier outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its `__all__` declaration contains 1 name: `run_export_verification`.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (7): `komorebi_math.export`, `komorebi_math.export.latex`, `komorebi_math.export.model`, `komorebi_math.export.renderers`, `komorebi_math.export.serialization`, `komorebi_math.export.writers`, and 1 more in the machine-readable ledger. Reverse package dependents (1): `komorebi_math.verify`. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.25.12 `komorebi_math.verify.expression`

Purpose and source. Installed verification for the finite typed expression calculus. It is an ordinary module in the `verify` family. The ledger classifies it as an installed verification module. Source `src/komorebi_math/verify/expression.py` contains 559 lines and 20,753 bytes. Source SHA-256 begins `fee8884d61def5d8`.

Surface and ownership. API classification: installed verifier outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its `__all__` declaration contains 1 name: `run_expression_verification`.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (7): `komorebi_math.certificates`, `komorebi_math.coordinates`, `komorebi_math.coordinates.exact`, `komorebi_math.coordinates.polynomials`, `komorebi_math.exceptions`, `komorebi_math.expression`, and 1 more in the machine-readable ledger. Reverse package dependents (1): `komorebi_math.verify`. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.25.13 komorebi_math.verify.frame_algebra

Purpose and source. Installed verification for redundant-frame algebra. It is an ordinary module in the verify family. The ledger classifies it as an installed verification module. Source src/komorebi_math/verify/frame_algebra.py contains 125 lines and 4,628 bytes. Source SHA-256 begins 017efd3c54ca263c.

Surface and ownership. API classification: installed verifier outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its __all__ declaration contains 1 name: run_frame_algebra_verification.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (6): komorebi_math.coordinates, komorebi_math.coordinates.analysis_synthesis, komorebi_math.coordinates.exact, komorebi_math.coordinates.frame_algebra, komorebi_math.exceptions, komorebi_math.verify.core. Reverse package dependents (1): komorebi_math.verify. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.25.14 komorebi_math.verify.generating

Purpose and source. Installed-package verification for normalized generating functions. It is an ordinary module in the verify family. The ledger classifies it as an installed verification module. Source src/komorebi_math/verify/generating.py contains 199 lines and 7,112 bytes. Source SHA-256 begins d02a650e9c627111.

Surface and ownership. API classification: installed verifier outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its __all__ declaration contains 1 name: run_generating_verification.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (15): komorebi_math.export, komorebi_math.export.persistence, komorebi_math.generating, komorebi_math.generating.adapters, komorebi_math.generating.basis, komorebi_math.generating.bell, and 9 more in the machine-readable ledger. Reverse package dependents (1): komorebi_math.verify. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.25.15 komorebi_math.verify.gpu

Purpose and source. Crash-isolated hardware-backed CUDA verification. It is an ordinary module in the verify family. The ledger classifies it as an installed verification module. Source src/komorebi_math/verify/gpu.py contains 42 lines and 1,405 bytes. Source SHA-256 begins e5234f735f8cfc2a.

Surface and ownership. API classification: installed verifier outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its __all__ declaration contains 1 name: run_gpu_verification.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (2): komorebi_math.qualification, komorebi_math.verify.core. Reverse package dependents (1): komorebi_math.verify. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.25.16 komorebi_math.verify.hardening

Purpose and source. Compatibility import for the scalable nonlinear verification suite. It is an ordinary module in the verify family. The ledger classifies it as an installed verification module. Source src/komorebi_math/verify/hardening.py contains 5 lines and 203 bytes. Source SHA-256 begins 5bd44d51e6397c7a.

Surface and ownership. API classification: installed verifier outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its __all__ declaration contains 1 name: run_scalable_nonlinear_verification.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (1): komorebi_math.verify.scalable_nonlinear. Reverse

package dependents (0): none. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.25.17 `komorebi_math.verify.integration`

Purpose and source. Installed verification for finite measure integration. It is an ordinary module in the `verify` family. The ledger classifies it as an installed verification module. Source `src/komorebi_math/verify/integration.py` contains 255 lines and 10,325 bytes. Source SHA-256 begins `43fa1d21f1e086af`.

Surface and ownership. API classification: installed verifier outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its `__all__` declaration contains 1 name: `run_integration_verification`.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (8): `komorebi_math.certificates`, `komorebi_math.coordinates`, `komorebi_math.coordinates.analysis_synthesis`, `komorebi_math.coordinates.classical`, `komorebi_math.coordinates.polynomials`, `komorebi_math.exceptions`, and 2 more in the machine-readable ledger. Reverse package dependents (1): `komorebi_math.verify`. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.25.18 `komorebi_math.verify.langlands`

Purpose and source. Installed verification for finite local, finite image, and completion models. It is an ordinary module in the `verify` family. The ledger classifies it as an installed verification module. Source `src/komorebi_math/verify/langlands.py` contains 168 lines and 6,308 bytes. Source SHA-256 begins `b2a1616fec75cf0d`.

Surface and ownership. API classification: installed verifier outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its `__all__` declaration contains 4 names: `run_langlands_completion_verification`, `run_langlands_finite_image_verification`, `run_langlands_local_verification`, `run_langlands_verification`.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (6): `komorebi_math.langlands.completion`, `komorebi_math.langlands.finite_image`, `komorebi_math.langlands.global_`, `komorebi_math.langlands.local`, `komorebi_math.langlands.ramification`, `komorebi_math.verify.core`. Reverse package dependents (1): `komorebi_math.verify`. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.25.19 `komorebi_math.verify.nonlinear`

Purpose and source. Installed verification suite for the nonlinear foundation. It is an ordinary module in the `verify` family. The ledger classifies it as an installed verification module. Source `src/komorebi_math/verify/nonlinear.py` contains 92 lines and 3,428 bytes. Source SHA-256 begins `2807f35643e010dd`.

Surface and ownership. API classification: installed verifier outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its `__all__` declaration contains 2 names: `run_nonlinear_foundation_verification`, `run_nonlinear_verification`.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (9): `komorebi_math.core`, `komorebi_math.nonlinear`, `komorebi_math.nonlinear.coordinates`, `komorebi_math.nonlinear.defects`, `komorebi_math.nonlinear.holonomy`, `komorebi_math.nonlinear.lifting`, and 3 more in the machine-readable ledger. Reverse package dependents (1): `komorebi_math.verify`. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.25.20 komorebi_math.verify.nonlinear_advanced

Purpose and source. Installed verification suite for advanced nonlinear calculus. It is an ordinary module in the verify family. The ledger classifies it as an installed verification module. Source src/komorebi_math/verify/nonlinear_advanced.py contains 231 lines and 8,865 bytes. Source SHA-256 begins b8b7d3b65fe11a92.

Surface and ownership. API classification: installed verifier outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its __all__ declaration contains 1 name: run_advanced_nonlinear_verification.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (14): komorebi_math.certificates, komorebi_math.core, komorebi_math.dynamics._linalg, komorebi_math.nonlinear, komorebi_math.nonlinear.composition, komorebi_math.nonlinear.correction, and 8 more in the machine-readable ledger. Reverse package dependents (2): komorebi_math.verify, komorebi_math.verify.nonlinear. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.25.21 komorebi_math.verify.quantum

Purpose and source. Installed verification for finite quantum packet calculus. It is an ordinary module in the verify family. The ledger classifies it as an installed verification module. Source src/komorebi_math/verify/quantum.py contains 150 lines and 5,972 bytes. Source SHA-256 begins 8af5b89ad28093b6.

Surface and ownership. API classification: installed verifier outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its __all__ declaration contains 1 name: run_quantum_verification.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (3): komorebi_math.dynamics, komorebi_math.quantum, komorebi_math.verify.core. Reverse package dependents (1): komorebi_math.verify. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.25.22 komorebi_math.verify.refinement

Purpose and source. Installed verification suite for refinement and filtered transfer. It is an ordinary module in the verify family. The ledger classifies it as an installed verification module. Source src/komorebi_math/verify/refinement.py contains 88 lines and 3,372 bytes. Source SHA-256 begins e008926e1df92a77.

Surface and ownership. API classification: installed verifier outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Static analysis found 1 top-level public definition: run_refinement_verification.

Dependencies and evidence. Direct dependency profile: optional exact extra, required NumPy dependency. Direct package dependencies (3): komorebi_math.core, komorebi_math.refinement, komorebi_math.verify.core. Reverse package dependents (1): komorebi_math.verify. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.25.23 komorebi_math.verify.release

Purpose and source. Installed release-qualification checks for the path to 1.0. It is an ordinary module in the verify family. The ledger classifies it as an installed verification module. Source src/komorebi_math/verify/release.py contains 95 lines and 3,972 bytes. Source SHA-256 begins 47400964ee98a885.

Surface and ownership. API classification: installed verifier outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its __all__ declaration contains 1 name: run_release_qualification_verification.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (7): komorebi_math.compatibility, komorebi_math.persistence_contract, komorebi_math.release, komorebi_math.stable, komorebi_math.stable_semantics, komorebi_math.

support, and 1 more in the machine-readable ledger. Reverse package dependents (1): komorebi_math.verify. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.25.24 komorebi_math.verify.scalable_nonlinear

Purpose and source. Installed verification for the scalable nonlinear runtime. It is an ordinary module in the verify family. The ledger classifies it as an installed verification module. Source src/komorebi_math/verify/scalable_nonlinear.py contains 159 lines and 6,084 bytes. Source SHA-256 begins cccac56a9186bea0.

Surface and ownership. API classification: installed verifier outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its __all__ declaration contains 1 name: run_scalable_nonlinear_verification.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (15): komorebi_math.certificates, komorebi_math.compatibility, komorebi_math.core, komorebi_math.exceptions, komorebi_math.export, komorebi_math.export.persistence, and 9 more in the machine-readable ledger. Reverse package dependents (2): komorebi_math.verify, komorebi_math.verify.hardening. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.25.25 komorebi_math.verify.sequences

Purpose and source. Installed verification for lazy exact sequence execution. It is an ordinary module in the verify family. The ledger classifies it as an installed verification module. Source src/komorebi_math/verify/sequences.py contains 77 lines and 2,812 bytes. Source SHA-256 begins 4f7ed71444419308.

Surface and ownership. API classification: installed verifier outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its __all__ declaration contains 1 name: run_sequences_verification.

Dependencies and evidence. Direct dependency profile: no direct third-party import beyond the standard library and package-local modules. Direct package dependencies (2): komorebi_math.sequences, komorebi_math.verify.core. Reverse package dependents (1): komorebi_math.verify. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.25.26 komorebi_math.verify.signals

Purpose and source. Installed package verification for the finite signal domain. It is an ordinary module in the verify family. The ledger classifies it as an installed verification module. Source src/komorebi_math/verify/signals.py contains 141 lines and 5,210 bytes. Source SHA-256 begins c981e9878071c41a.

Surface and ownership. API classification: installed verifier outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its __all__ declaration contains 1 name: run_signals_verification.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (8): komorebi_math.signals, komorebi_math.signals.filterbanks, komorebi_math.signals.nonlinear, komorebi_math.signals.polyphase, komorebi_math.signals.recovery, komorebi_math.signals.singular_packets, and 2 more in the machine-readable ledger. Reverse package dependents (1): komorebi_math.verify. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.25.27 `komorebi_math.verify.stabilization`

Purpose and source. Installed verification for polyphase stabilization and 1.0 contracts. It is an ordinary module in the `verify` family. The ledger classifies it as an installed verification module. Source `src/komorebi_math/verify/stabilization.py` contains 175 lines and 6,665 bytes. Source SHA-256 begins `9a969f6fbd408f50`.

Surface and ownership. API classification: installed verifier outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its `__all__` declaration contains 2 names: `run_polyphase_stabilization_verification`, `run_stable_contract_verification`.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (15): `komorebi_math.api`, `komorebi_math.api.v1`, `komorebi_math.arithmetic`, `komorebi_math.compatibility`, `komorebi_math.core`, `komorebi_math.export.persistence`, and 9 more in the machine-readable ledger. Reverse package dependents (1): `komorebi_math.verify`. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.2.25.28 `komorebi_math.verify.streaming_beta`

Purpose and source. Installed verification for persistence and compatibility boundaries. It is an ordinary module in the `verify` family. The ledger classifies it as an installed verification module. Source `src/komorebi_math/verify/streaming_beta.py` contains 191 lines and 7,319 bytes. Source SHA-256 begins `fae5f92bedb4f7d8`.

Surface and ownership. API classification: installed verifier outside the API manifests. No stable, beta, or research card names this module as its implementation owner. Its importability or declared names therefore do not create a manifest compatibility promise. Its `__all__` declaration contains 1 name: `run_streaming_beta_verification`.

Dependencies and evidence. Direct dependency profile: required NumPy dependency. Direct package dependencies (13): `komorebi_math.compatibility`, `komorebi_math.core`, `komorebi_math.exceptions`, `komorebi_math.export`, `komorebi_math.export.persistence`, `komorebi_math.golden`, and 7 more in the machine-readable ledger. Reverse package dependents (1): `komorebi_math.verify`. Examples (0): none. Tests or test support files (0): none. Documentation files with literal module or public-route references (0): none. AST parsing and runtime import both passed in the recorded module audit.

D.3 External implementation-owner rows

Five manifest owner strings resolve to Python or typing support modules rather than Komorebi source files. They own data aliases, protocols, paths, or runtime types that are exposed through Komorebi namespaces. These rows close the implementation-owner crosswalk without presenting the standard library as Komorebi implementation code.

D.3.1 `builtins`

Owner scope. External Python or typing support owner. It owns 48 cards: 9 stable, 10 beta, and 29 research. Linked cards: **version**, PORTABLE_CODEC_CONTRACT_SCHEMA, PORTABLE_CODEC_CONTRACT_SCHEMA_VERSION, PORTABLE_SCHEMA, PORTABLE_SCHEMA_VERSION, STABLE_API_STABILITY_LEVEL, STABLE_PUBLIC_NAMESPACES, STABLE_SEMANTICS_SCHEMA, STABLE_SEMANTICS_SCHEMA_VERSION, **version**, BEHAVIORAL_GOLDEN_SCHEMA, BEHAVIORAL_GOLDEN_SCHEMA_VERSION, and 36 more in the machine-readable ledger.

Evidence. Examples (1): `block21_public_capability_tour.py`. Tests or support files (4): `test_block15_stable_api.py`, `test_block16_analysis_synthesis.py`, `test_block16_release_contract.py`, `test_block19_integration.py`. Documentation files with literal public-route references (0): none. No Komorebi source path or source digest is assigned to this row.

D.3.2 `collections.abc`

Owner scope. External Python or typing support owner. It owns 3 cards: 0 stable, 0 beta, and 3 research. Linked cards: MultilinearOperation, DirectionalDerivative, MultilinearOperation.

Evidence. Examples (0): none. Tests or support files (0): none. Documentation files with literal public-route references (0): none. No Komorebi source path or source digest is assigned to this row.

D.3.3 `pathlib`

Owner scope. External Python or typing support owner. It owns 4 cards: 2 stable, 0 beta, and 2 research. Linked cards: DEFAULT_PORTABLE_CODEC_CONTRACT, DEFAULT_STABLE_SEMANTICS_FIXTURE, DEFAULT_BEHAVIORAL_GOLDEN_FIXTURE, DEFAULT_BETA_API_MANIFEST.

Evidence. Examples (0): none. Tests or support files (0): none. Documentation files with literal public-route references (0): none. No Komorebi source path or source digest is assigned to this row.

D.3.4 `types`

Owner scope. External Python or typing support owner. It owns 6 cards: 1 stable, 0 beta, and 5 research. Linked cards: PolyphaseCoordinateRecord, CoordinateSpace, ExpressionType, Scalar, PacketLabel, PacketPath.

Evidence. Examples (0): none. Tests or support files (1): `test_block15_stable_api.py`. Documentation files with literal public-route references (0): none. No Komorebi source path or source digest is assigned to this row.

D.3.5 `typing`

Owner scope. External Python or typing support owner. It owns 16 cards: 0 stable, 0 beta, and 16 research. Linked cards: RecurrenceLike, Variance, AdaptiveMethod, BiquadKind, BoundaryMode, ConvolutionMode, DetrendMode, FIRKind, FIRMode, SpectralAverage, SpectralScaling, SpectrogramScaling, and 4 more in the machine-readable ledger.

Evidence. Examples (0): none. Tests or support files (0): none. Documentation files with literal public-route references (0): none. No Komorebi source path or source digest is assigned to this row.

D.4 Machine-readable crosswalk and closure

The machine-readable ledger contains the complete arrays abbreviated by the printed cards. `Block36_Module_Reference_Ledger.json` is the canonical module inventory. `Block36_Module_Reference_Ledger.csv` provides one row per package module. The family summary, external-owner crosswalk, internal dependency edge list, and evidence edge list are emitted as separate CSV files.

Crosswalk	Rows
Package module cards	300
External owner rows	5
Direct package dependency edges	1591
Module to example edges	268
Module to test or support-file edges	668
Module to documentation edges	165

Module-reference coverage. All 300 shipped Python source modules have one module card and one unique anchor. All 202 package implementation-owner modules and all 5 external owner strings are represented. The 201 research owner strings from Appendix C are fully joined to package or external rows. This establishes the module-reference coverage. Execution records, command documentation, and release qualification remain separate evidence layers.

Appendix E. Executable-example records

Choosing and running an example. Use the record index to choose a mathematical task, then read the linked chapter before adapting the script. For a first packet-based program, start with EX-E-043 and Chapter 4. For a broader orientation, EX-E-007 is the public capability tour. The remaining records are independent entry points into particular topics; their order is not a required curriculum.

Each record separates the ingredients of a reproducible example:

Field	Read it as
Input (`I`)	The concrete data used by this example.
Hypotheses (`H`)	Conditions that must hold before interpreting the result.
Actions (`A`)	The sequence of operations performed by the script.
Expected result (`O`)	The values, relationships, or messages to compare with your run.
Certificates (`C`)	The evidence produced and the scope of the claim.
Failure boundary (`F`)	An invalid case or limit that helps explain the contract.
Reproducibility (`R`)	The interpreter, dependencies, invocation, and associated execution context.

Before running a record, check its language, dependencies, working directory, and file effects. Use the complete source script identified in the record: the prose and displayed output are a guide to that script, not a substitute for its imports and setup. Run the unchanged example first, inspect its result, then change one input or hypothesis at a time. Keep the original working example available for comparison.

The replay controller described below is a source-bundle tool, not an additional command installed by the wheel. A wheel-only installation does not supply every repository script or historical replay fixture. Obtain the matching source material before using those paths, and keep reference replay separate from a run against a newer installed package. For an ordinary individual run, use the entry point recorded on that example's card.

Interpreting a different result. First compare the installed package and dependency versions, then the input, working directory, backend, seed, and resource settings. Exact discrete results and explicit numerical tolerances are the meaningful comparison targets. Wall-clock timings, allocation peaks, device availability, and display formatting can vary. Read the record's expected-result and failure fields before deciding that a printed `False`, an expected exception, or an unavailable backend is an error in the example.

An expected-rejection example succeeds by detecting the declared invalid case. It does not establish that the rejected mathematical or hardware condition has become valid. This distinction is particularly useful for tampered persistence data, incomplete streams, unmet completion assumptions, and GPU qualification. For recovery steps, follow Appendix I and the relevant domain chapter rather than disabling the check that exposed the problem.

E.0 Scope, evidence and replay contract

All observations in this appendix belong to the recorded reference baseline and are bound by its source and wheel hashes. They are not a new replay of the final package.

The reference inventory contains exactly the **57 Python and eight PowerShell scripts** in `komorebi-math/examples`. Each script has one record identifier and one unique anchor. A record is a documentation unit, not a new theorem. Its local claim identifier ends in `CONTRACT`; mathematical theorem identifiers, where observed, are copied from runtime certificates.

The record schema retains the guide's original contract:

$$\mathcal{E} = (I, H, A, O, C, F, R).$$

Input and hypotheses are separated from the action and expected result. The machine-readable ledger adds source path, language, imports, public symbols, dependencies, platform, runtime class, seed, working directory, artifact hashes, cleanup, related tests and historical/current status. Exact input literals and helper functions remain available in the source file bound by each SHA-256 digest; the prose describes them rather than replacing them with an incomplete paraphrase.

Record completeness and runtime qualification are separate. All 65 records are filled. All 57 Python default invocations passed against the installed reference wheel. Seven reference PowerShell workflows await native qualification; the eighth is a historical `0.16.0a2` aggregate runner and is not applicable to the reference release. An unavailable or historical runner never contributes a runtime pass.

The Python replay. The controller checks the installed package version and all 366 package-file hashes, verifies the source-record inventory, copies only the example scripts to an isolated run location, disables optimization and runs each script in a bounded child process. Its own mathematical assertions remain active. The worker observes base `Certificate.require` calls without changing their return values or rejection behavior and rejects source-checkout imports. Each run must exit zero, satisfy its declared output/artifact predicates and leave no unexpected stderr.

The recorded run passed **190 predicates across 57 scripts**. It observed **841 base-certificate require calls across 104 distinct identifier strings**, including the possible no-ID category. These are repeated runtime observations, not 841 distinct theorems or new formal proofs. Certificates of other classes and Python assertions remain active but are not counted by this base-class observer.

Reference environment. CPython 3.13.5, Linux, NumPy 2.3.5, SciPy 1.17.0 and SymPy 1.14.0. Dependencies were inherited by the selected virtual environment. All package modules came from the separately installed wheel. Timings were measured with three workers and one BLAS thread per child, so they are diagnostic observations, not comparative performance claims.

Output matching. Exact discrete fields, stable text markers and selected JSON fields are checked explicitly. Declared numerical checks use tolerances. Profiling timings, Python allocation peaks and device availability are not equality targets. Some stable numeric text remains a reference-environment display check; cross-version formatting portability is not claimed. Raw stdout, stderr, module origins, certificate samples and artifact hashes are retained separately. The formatted output excerpts below may wrap long lines and are not raw-byte snapshots.

Failure variants. Each record names a relevant failure boundary. The shipped scripts execute some negative controls, such as payload tampering, broken holonomy and false quadrature exactness. Other variants identify hypotheses or required rejection behavior and are not automatically claimed as newly injected tests. The 134 example-record tests validate record/source correspondence, replay artifacts, missing-output rejection, tolerance behavior and corruption controls. Related package tests are a direct-shared-import crosswalk, not a theorem-coverage proof.

Reproduce from the delivery root. Select a Python interpreter in which the reference wheel and its dependencies are already installed. The controller does not install packages or use the network.

```
python tools/block37/replay_examples.py \
  --python /path/to/installed/python \
  --output /path/to/new-evidence-directory
```

The Python-scope exit code is zero only when all 57 contracts pass. Adding `--require-all-platforms` deliberately returns a nonzero result while native PowerShell evidence is missing. On Windows the same arguments may be written on one line; the POSIX line-continuation marks above are not PowerShell syntax.

E.1 Record index

Record	Example topic	Runtime scope
EX-E-001	Automatic and holonomic sequences	Python passed
EX-E-002	Semantic counts and observational profiling	Python passed
EX-E-003	Polyphase refinement coherence	Python passed
EX-E-004	Redundant analysis and operator blocks	Python passed
EX-E-005	Exact change of variables and a false-exactness adversary	Python passed
EX-E-006	Typed differentiation and integration through coordinates	Python passed
EX-E-007	Integrated public capability tour	Python passed
EX-E-008	Legendre, Chebyshev and derivative transport	Python passed
EX-E-009	Recurrence coordinates and naturality	Python passed
EX-E-010	Custom rational and algebraic finite series	Python passed
EX-E-011	Cyclic projection, reciprocal coefficients and zeros	Python passed
EX-E-012	Finite residue forest and occupancy	Python passed
EX-E-013	Finite Euclidean transfer and lattice spectrum	Python passed
EX-E-014	Euler data in Fibonacci defect coordinates	Python passed
EX-E-015	Symbolic backend and seed rank	Python passed
EX-E-016	Lattice gluing and Smith congruences	Python passed
EX-E-017	Multi-format reports and export artifacts	Python passed
EX-E-018	Feshbach self-energy and nonlocal memory	Python passed
EX-E-019	Filtered pages and moving-projector transport	Python passed
EX-E-020	Induced finite representation and unresolved descent	Python passed
EX-E-021	Affine, barycentric and formal changes	Python passed
EX-E-022	Fractional transfer and phase-type survival	Python passed
EX-E-023	Finite Frobenius separator and carrier multiplicities	Python passed
EX-E-024	Three-basis exact atlas	Python passed
EX-E-025	Gauge-invariant gluing with a broken-cycle control	Python passed
EX-E-026	Quantum transport and Feshbach Laplace identity	Python passed
EX-E-027	Dyadic and odd-prime Euler truncations	Python passed
EX-E-028	Low-rank structured solve and recompression	Python passed
EX-E-029	Finite-mask carry and rational-mask Mahler compilation	Python passed
EX-E-030	Lumping and Bayesian sufficiency	Python passed
EX-E-031	Faà di Bruno and sparse packet support	Python passed
EX-E-032	Tree expansion, Schur elimination and obstruction	Python passed
EX-E-033	Cokernel obstruction and prime-power branches	Python passed
EX-E-034	Finite candidate-tree optimization	Python passed

Record	Example topic	Runtime scope
EX-E-035	Structured packet intermediate representation	Python passed
EX-E-036	Polyphase and Haar reconstruction	Python passed
EX-E-037	Portable jet snapshot and tamper rejection	Python passed
EX-E-038	Discrete precision and task scheduling	Python passed
EX-E-039	Prime-power kernels and valuations	Python passed
EX-E-040	Finite DSP pipeline with fixed seed	Python passed
EX-E-041	Amplitude damping and finite error correction	Python passed
EX-E-042	Interfering symmetry-resolved circuit paths	Python passed
EX-E-043	First packet-resolved operator program	Python passed
EX-E-044	Finite ramification with an unmet realization gate	Python passed
EX-E-045	Recurrence lanes and modular large-index terms	Python passed
EX-E-046	Recurrence bridge and state observability	Python passed
EX-E-047	Packet refinement, CRT and weighted transfer	Python passed
EX-E-048	Pause, resume and verify a finite jet stream	Python passed
EX-E-049	Sparse derivative plan with backend disclosure	Python passed
EX-E-050	Sparse packet-jet runtime counts	Python passed
EX-E-051	Declared sequence models and compiler strategies	Python passed
EX-E-052	Separated jumps and retained reconstruction	Python passed
EX-E-053	Fourier, Haar and exact residue ordering	Python passed
EX-E-054	Standard-window and affine Mahler refinement	Python passed
EX-E-055	Preconditioned conjugate gradient	Python passed
EX-E-056	Finite tame and Heisenberg packets	Python passed
EX-E-057	Typed operator relation certificate	Python passed
EX-E-058	Scalable nonlinear workflow	Native run pending
EX-E-059	Streaming and semantic workflow	Native run pending
EX-E-060	DSP workflow	Native run pending
EX-E-061	Generating-function workflow	Native run pending
EX-E-062	Coordinate workflow	Native run pending
EX-E-063	Nonlinear verification workflow	Native run pending
EX-E-064	Historical aggregate runner	Historical
EX-E-065	Windows wheel installation and export workflow	Native run pending

E.2 Python execution records

EX-E-001. Automatic and holonomic sequences

Source and placement. `komorebi-math/examples/automatic_holonomic.py`. Chapter 26, exact sequence execution. Local claim: `EX-E-001-CONTRACT`.

Input (I). Thue-Morse residue power 4, address 7, length 20; Apery prefix length 10.

Hypotheses and evidence class (H). EXACT finite sequences. The source fixture fixes all remaining literals and dimensions. The first observed certificate for each identifier, including checked and unchecked assumptions, is retained in the execution record.

Actions (A). Require the automaton certificate at 256 terms and the holonomic certificate at 15 terms, then evaluate the two prefixes.

Expected result (O). The residue starts 1,0,0,1; the Apery prefix starts 1,5,73,1445. These are finite prefix checks, not evidence about all automatic or holonomic models.

Certificate and replay checks (C). The source contains 0 explicit `assert` statements and 2 syntactic `.require()` call sites. These are source counts, not dynamic proof counts. The installed-wheel run passed 2 output/artifact predicates with exit code zero and 244 package modules checked for import origin.

Observed base-certificate identifiers include `PSC-SEQ-AUTOMATIC`, `PSC-SEQ-HOLONOMIC`.

Failure boundary (F). Invalid residue addresses or negative prefix lengths must be rejected rather than silently wrapped.

Source API routes. `komorebi_math.sequences.apery_recurrence`, `komorebi_math.sequences.thue_morse_automaton`.

Reproducibility (R). `python examples/automatic_holonomic.py`. Reference wheel, NumPy, and the logged validation environment with SciPy, SymPy and mpmath. Direct imports are listed separately; this profile is not a proof of minimal transitive dependencies. No random input in the shipped fixture.

Runtime and filesystem. Recorded Linux replay: 0.902 seconds, measured under the shared reference profile; this is not a performance promise. No persistent output files are declared by the example. Replay stdout/stderr and metadata are retained in the selected evidence directory.

Related package tests. `tests/test_sequences_models.py`. These files were included in the separate 1,203-test source run. Shared imports alone do not prove full behavioral coverage.

Source SHA-256. `399b420ae1364f5dcc84a73f8693b95e813f154e65d1e48839e084576c167081`. Full source and replay hashes are also stored in the JSON ledger and per-record result files.

Observed output excerpt. Selected fields or wrapped lines only. Full stdout is preserved in the replay directory.

```
Thue Morse residue (1, 0, 0, 1, 0, 1, 1, 0, 0, 1, 1, 0, 1, 0, 0, 1, 0, 1, 1,
 0)
Apery (Fraction(1, 1), Fraction(5, 1), Fraction(73, 1), Fraction(1445, 1),
Fraction(33001, 1), Fraction(819005, 1), Fraction(21460825, 1),
Fraction(584307365, 1), Fraction(16367912425, 1), Fraction(468690849005, 1))
```

EX-E-002. Semantic counts and observational profiling

Source and placement. `komorebi-math/examples/block11_semantic_profile.py`. Chapter 72, semantic contracts. Local claim: `EX-E-002-CONTRACT`.

Input (I). A two-dimensional order-2 tensor has diagonal entries 2 and -3. Use x/y coordinate packets, directions (2,3),(-1,4), two repetitions and one warmup.

Hypotheses and evidence class (H). EXACT counts / NUMERICAL evaluation / OBSERVATIONAL timing. The source fixture fixes all remaining literals and dimensions. The first observed certificate for each identifier, including checked and unchecked assumptions, is retained in the execution record.

Actions (A). Construct the support, profile the declared jet, capture stream-writing stages, and require both the profile and golden-fixture certificates.

Expected result (O). Eight family addresses reduce to two active addresses and six pruned addresses; two planned derivative calls replace eight naive calls. Fifteen golden cases are present. Timings and allocation peaks are observations, not portable equality targets.

Certificate and replay checks (C). The source contains 0 explicit `assert` statements and 2 syntactic `.require()` call sites. These are source counts, not dynamic proof counts. The installed-wheel run passed 6 output/artifact predicates with exit code zero and 245 package modules checked for import origin.

Observed base-certificate identifiers include `KM-GOLDEN-011`, `PSC-I-T20`, `PSC-III-A10`, `PSC-III-A61`, `PSC-III-C08` (the complete set is in the execution record).

Failure boundary (F). Changing an exact count must fail the contract even when timing changes are allowed.

Source API routes. `komorebi_math.PacketSystem`, `komorebi_math.golden.run_behavioral_golden_fixture`, `komorebi_math.nonlinear.PacketJetFamily`, `komorebi_math.nonlinear.PolynomialDerivativeProvider`, `komorebi_math.nonlinear.packet_jet_support` and 1 more route in the JSON ledger.

Reproducibility (R). `python examples/block11_semantic_profile.py`. Reference wheel, NumPy, and the logged validation environment with SciPy, SymPy and mpmath. Direct imports are listed separately; this profile is not a proof of minimal transitive dependencies. No random input in the shipped fixture.

Runtime and filesystem. Recorded Linux replay: 1.107 seconds, measured under the shared reference profile; this is not a performance promise. No persistent output files are declared by the example. Replay stdout/stderr and metadata are retained in the selected evidence directory.

Related package tests. `tests/test_block10_sparse_autodiff.py`, `tests/test_operators_transfer.py`, `tests/test_nonlinear_trees.py` and 16 more files in the shared-import crosswalk. These files were included in the separate 1,203-test source run. Shared imports alone do not prove full behavioral coverage.

Source SHA-256. `7c41b8bc55b671b6bca2a3c03e28bc4e9d5903037a8c07619e775e0b1355d2ab`. Full source and replay hashes are also stored in the JSON ledger and per-record result files.

Observed output excerpt. Selected fields or wrapped lines only. Full stdout is preserved in the replay directory.


```
{
  "golden_cases": 15,
  "active_addresses": 2,
  "pruned_addresses": 6,
  "planned_derivative_calls": 2
}
```


EX-E-003. Polyphase refinement coherence

Source and placement. `komorebi-math/examples/block15_polyphase_refinement.py`. Chapters 12 and 14, refinement and polyphase. Local claim: `EX-E-003-CONTRACT`.

Input (I). The integer signal arange(29), a 4-to-12 refinement, and the tower (2,6,12).

Hypotheses and evidence class (H). EXACT finite indexing. The source fixture fixes all remaining literals and dimensions. The first observed certificate for each identifier, including checked and unchecked assumptions, is retained in the execution record.

Actions (A). Refine and coarsen the finite signal, require reconstruction and tower-coherence certificates, and check the residue children of lane 1.

Expected result (O). The children are (1,5,9); both reconstructions equal the original signal exactly. The success line is emitted only after the explicit assertions.

Certificate and replay checks (C). The source contains 3 explicit `assert` statements and 2 syntactic `.require()` call sites. These are source counts, not dynamic proof counts. The installed-wheel run passed 1 output/artifact predicate with exit code zero and 244 package modules checked for import origin.

Observed base-certificate identifiers include `KM-B15-T01`, `KM-B15-T02`.

Failure boundary (F). An order that is not divisible by the coarse order is not a valid refinement.

Source API routes. `komorebi_math.refinement.PolyphaseRefinement`, `komorebi_math.refinement.PolyphaseRefinementTower`, `komorebi_math.signals.polyphase_analyze`.

Reproducibility (R). `python examples/block15_polyphase_refinement.py`. Reference wheel, NumPy, and the logged validation environment with SciPy, SymPy and mpmath. Direct imports are listed separately; this profile is not a proof of minimal transitive dependencies. No random input in the shipped fixture.

Runtime and filesystem. Recorded Linux replay: 0.828 seconds, measured under the shared reference profile; this is not a performance promise. No persistent output files are declared by the example. Replay stdout/stderr and metadata are retained in the selected evidence directory.

Related package tests. `tests/test_block16_release_qualification.py`, `tests/test_block17_upgrade_contract.py`, `tests/test_signals_polyphase_filterbanks.py` and 4 more files in the shared-import crosswalk. These files were included in the separate 1,203-test source run. Shared imports alone do not prove full behavioral coverage.

Source SHA-256. `796d8e1a42fef3960f23d6af470ce7b616daacc35aeb933ebf5c944096b0b75f`. Full source and replay hashes are also stored in the JSON ledger and per-record result files.

Observed output excerpt. Selected fields or wrapped lines only. Full stdout is preserved in the replay directory.

```
Block 15 polyphase refinement verified
```

EX-E-004. Redundant analysis and operator blocks

Source and placement. `komorebi-math/examples/block16_analysis_synthesis.py`. Chapter 20, analysis-synthesis. Local claim: `EX-E-004-CONTRACT`.

Input (I). Analysis rows (1,0),(0,1),(1,1),(1,-1); signal (3,-2); operator [[2,-1],[3,4]]; coordinate groups (0,2),(1,3).

Hypotheses and evidence class (H). NUMERICAL, declared certificate tolerances. The source fixture fixes all remaining literals and dimensions. The first observed certificate for each identifier, including checked and unchecked assumptions, is retained in the execution record.

Actions (A). Build the canonical dual, reconstruct signal and operator blocks, then portable-serialize and restore the analysis system.

Expected result (O). Both reconstructions satisfy their numerical certificates; the signal is (3,-2). Redundancy is four coordinates for a two-dimensional signal, not an invertible square change of basis.

Certificate and replay checks (C). The source contains 3 explicit `assert` statements and 3 syntactic `.require()` call sites. These are source counts, not dynamic proof counts. The installed-wheel run passed 4 output/artifact predicates with exit code zero and 249 package modules checked for import origin.

Observed base-certificate identifiers include `KM-AS-01`, `KM-AS-02`.

Failure boundary (F). A rank-deficient analysis matrix or coordinate groups that do not partition the coordinates must not be certified.

Source API routes. `komorebi_math.stable.AnalysisSynthesisSystem`, `komorebi_math.stable.dumps_portable`, `komorebi_math.stable.loads_portable`.

Reproducibility (R). `python examples/block16_analysis_synthesis.py`. Reference wheel, NumPy, and the logged validation environment with SciPy, SymPy and mpmath. Direct imports are listed separately; this profile is not a proof of minimal transitive dependencies. No random input in the shipped fixture.

Runtime and filesystem. Recorded Linux replay: 0.820 seconds, measured under the shared reference profile; this is not a performance promise. No persistent output files are declared by the example. Replay stdout/stderr and metadata are retained in the selected evidence directory.

Source SHA-256. `4403a6ef5db1aeb5e94840618ca5926b7a46f89840f1d7f81bd86285f53787d2`. Full source and replay hashes are also stored in the JSON ledger and per-record result files.

Observed output excerpt. Selected fields or wrapped lines only. Full stdout is preserved in the replay directory.

```
analysis-synthesis certificate {'valid': True, 'exactness': 'numerical',
'residuals': {'analysis_synthesis_left_inverse': 0.0}, 'tolerance': 1e-12,
'assumptions_checked': ('finite matrices', 'declared matrix dimensions',
'full source-space rank'), 'assumptions_unchecked': (), 'diagnostics':
{'source_dimension': 2, 'coordinate_dimension': 4, 'analysis_rank': 2,
'square': False, 'redundant': True}, 'theorem_id': 'KM-AS-01', 'message':
'analysis--synthesis reconstruction verified'}
operator-block certificate {'valid': True, 'exactness': 'numerical',
'residuals': {'operator_block_reconstruction': 1.1102230246251565e-15},
'tolerance': 1e-12, 'assumptions_checked': ('finite analysis and synthesis
matrices', 'coordinate groups form a partition', 'analysis--synthesis left
inverse'), 'assumptions_unchecked': (), 'diagnostics': {}, 'theorem_id':
'KM-AS-02', 'message': 'analysis--synthesis operator blocks reconstruct'}
reconstructed [ 3. -2.]
```

EX-E-005. Exact change of variables and a false-exactness adversary

Source and placement. `komorebi-math/examples/block19_measure_integration_worked_example.py`. Chapters 47 through 49, exact integration. Local claim: `EX-E-005-CONTRACT`.

Input (I). Lebesgue measure on [0,1], y=2x+1, and f(y)=y^4-2y+1. Simpson uses 16 panels. The adversarial polynomial is defined in the shipped run() function.

Hypotheses and evidence class (H). EXACT rational identities / RIGOROUS finite quadrature bound. The source fixture fixes all remaining literals and dimensions. The first observed certificate for each identifier, including checked and unchecked assumptions, is retained in the execution record.

Actions (A). Compare pushforward, density and pullback integrals, pair three polynomial coordinate families, bound Simpson error, and test the node-vanishing adversary.

Expected result (O). Every exact integral and coordinate pairing is 106/5. The adversary samples to zero but integrates to 1/840, and its false exactness certificate is blocked.

Certificate and replay checks (C). The source contains 7 explicit `assert` statements and 9 syntactic `.require()` call sites. These are source counts, not dynamic proof counts. The installed-wheel run passed 5 output/artifact predicates with exit code zero and 244 package modules checked for import origin.

Observed base-certificate identifiers include `INT.COORD.TRANSPORT`, `INT.EVAL.POLY.MOMENTS`, `INT.ID.COV.AFFINE`, `INT.ID.FTC`, `INT.ID.PUSHFORWARD` (the complete set is in the execution record).

Failure boundary (F). The shipped adversarial node polynomial is an executed negative control, not merely a suggested future test.

Source API routes. `komorebi_math.coordinates.connection_transform`, `komorebi_math.coordinates.euler_polynomial_family`, `komorebi_math.coordinates.fibonacci_polynomial_family`, `komorebi_math.coordinates.monomial_polynomial_family`, `komorebi_math.exceptions.ValidationError` and 13 more routes in the JSON ledger.

Reproducibility (R). `python examples/block19_measure_integration_worked_example.py`. Reference wheel, NumPy, and the logged validation environment with SciPy, SymPy and mpmath. Direct imports are listed separately; this profile is not a proof of minimal transitive dependencies. No random input in the shipped fixture.

Runtime and filesystem. Recorded Linux replay: 0.902 seconds, measured under the shared reference profile; this is not a performance promise. No persistent output files are declared by the example. Replay stdout/stderr and metadata are retained in the selected evidence directory.

Related package tests. `tests/test_generating_block13.py`, `tests/test_certificates_registry.py`, `tests/test_algorithms_complexity_benchmarks.py` and 67 more files in the shared-import crosswalk. These files were included in the separate 1,203-test source run. Shared imports alone do not prove full behavioral coverage.

Source SHA-256. `3c013d95c1be0d73b6bc4884d8bce2178f5766687d7ad5489e518544b7587480`. Full source and replay hashes are also stored in the JSON ledger and per-record result files.

Observed output excerpt. Selected fields or wrapped lines only. Full stdout is preserved in the replay directory.

```
Block 19 worked example
  chart: y = 2x + 1, x in [0,1]
  pushed measure: (1/2) dy on [1,3]
  integrand: f(y) = y^4 - 2y + 1
  pullback: f(2x+1) = 16x^4 + 32x^3 + 24x^2 + 4x
  exact integral: 106/5
  monomials pairing: 106/5
  Euler polynomials pairing: 106/5
```

EX-E-006. Typed differentiation and integration through coordinates

Source and placement. `komorebi-math/examples/block20_typed_expression_worked_example.py`. Chapters 50 through 52, typed expressions. Local claim: `EX-E-006-CONTRACT`.

Input (I). G3(x)=x^3+2x in Fibonacci coordinates, degree-3 and degree-4 spaces, Euler coordinates, and integration over [0,1].

Hypotheses and evidence class (H). EXACT finite typed expressions. The source fixture fixes all remaining literals and dimensions. The first observed certificate for each identifier, including checked and unchecked assumptions, is retained in the execution record.

Actions (A). Compose antiderivative, coordinate changes and derivative; normalize, simplify, compile an optimal matrix chain, and round-trip the portable expression.

Expected result (O). The pipeline is I4, the integral is 5/4, and the declared chain costs are 244 versus 260. The affine weighted regression returns 106/5.

Certificate and replay checks (C). The source contains 12 explicit `assert` statements and 0 syntactic `.require()` call sites. These are source counts, not dynamic proof counts. The installed-wheel run passed 4 output/artifact predicates with exit code zero and 244 package modules checked for import origin.

Failure boundary (F). Composition across incompatible spaces must fail type validation. A portable digest alone is not a mathematical identity proof.

Source API routes. `komorebi_math.expression.IdentityMap`, `komorebi_math.expression.MatrixChainPlan`, `komorebi_math.expression.PolynomialCoordinateSpace`, `komorebi_math.expression.affine_substitution`, `komorebi_math.expression.antiderivative` and 15 more routes in the JSON ledger.

Reproducibility (R). `python examples/block20_typed_expression_worked_example.py`. Reference wheel, NumPy, and the logged validation environment with SciPy, SymPy and mpmath. Direct imports are listed separately; this profile is not a proof of minimal transitive dependencies. No random input in the shipped fixture.

Runtime and filesystem. Recorded Linux replay: 1.069 seconds, measured under the shared reference profile; this is not a performance promise. No persistent output files are declared by the example. Replay stdout/stderr and metadata are retained in the selected evidence directory.

Related package tests. `tests/test_block20_expression_calculus.py`. These files were included in the separate 1,203-test source run. Shared imports alone do not prove full behavioral coverage.

Source SHA-256. `4e816b9e809d993ef141f011b4e9d800d432be59c5494ade945275c80290d2d7`. Full source and replay hashes are also stored in the JSON ledger and per-record result files.

Observed output excerpt. Selected fields or wrapped lines only. Full stdout is preserved in the replay directory.

```
Block 20 typed finite expression worked example
Input: G_3(x) = x^3 + 2x
  J_0_in_Fibonacci: (-1/2, 0, 1/4, 0, 1/4)
  Fibonacci4_to_Euler4: (5/8, 3/2, 7/4, 1/2, 1/4)
  D_in_Euler: (3/2, 7/2, 3/2, 1)
  Euler3_to_Fibonacci3: (0, 0, 0, 1)
Integration covector: (1, 1/2, 4/3, 5/4)
Compiled pipeline normal form: I4
```

EX-E-007. Integrated public capability tour

Source and placement. `komorebi-math/examples/block21_public_capability_tour.py`. Chapter 74, publication capability tour. Local claim: `EX-E-007-CONTRACT`.

Input (I). Use the built-in polyphase, analysis-synthesis, integration, and typed-expression fixtures, eighteen Euler coefficients, and the 131-entry stable manifest.

Hypotheses and evidence class (H). MIXED exact and numerical finite checks. The source fixture fixes all remaining literals and dimensions. The first observed certificate for each identifier, including checked and unchecked assumptions, is retained in the execution record.

Actions (A). Call sibling integration/expression examples and combine refinement, reconstruction, serialization and stable-contract checks in a JSON record.

Expected result (O). passed is true; stable symbol count is 131; the exact integral fields are 106/5 and 5/4. The record digest binds the canonical record, not external publication.

Certificate and replay checks (C). The source contains 12 explicit `assert` statements and 4 syntactic `.require()` call sites. These are source counts, not dynamic proof counts. The installed-wheel run passed 7 output/artifact predicates with exit code zero and 249 package modules checked for import origin.

Observed base-certificate identifiers include `CC.R1`, `INT.COORD.TRANSPORT`, `INT.EVAL.POLY.MOMENTS`, `INT.ID.COV.AFFINE`, `INT.ID.FTC` (the complete set is in the execution record).

Failure boundary (F). Retain the sibling example files. Copying this script alone causes an import failure; changing a capability field must fail the replay contract.

Source API routes. `komorebi_math.__version__`, `komorebi_math.coordinates.AnalysisSynthesisSystem`, `komorebi_math.coordinates.euler_fibonacci_coordinates`, `komorebi_math.export.persistence.dumps_portable`, `komorebi_math.export.persistence.loads_portable` and 3 more routes in the JSON ledger.

Reproducibility (R). `python examples/block21_public_capability_tour.py`. Reference wheel, NumPy, and the logged validation environment with SciPy, SymPy and mpmath. Direct imports are listed separately; this profile is not a proof of minimal transitive dependencies. No random input in the shipped fixture.

Runtime and filesystem. Recorded Linux replay: 1.052 seconds, measured under the shared reference profile; this is not a performance promise. Default invocation writes only stdout. The optional –output PATH creates directories and writes JSON; that alternate path is not part of this default replay.

Related package tests. `tests/test_block16_release_qualification.py`, `tests/test_coordinates_normalized_persistence.py`, `tests/test_block17_upgrade_contract.py` and 12 more files in the shared-import crosswalk. These files were included in the separate 1,203-test source run. Shared imports alone do not prove full behavioral coverage.

Source SHA-256. `07e0a014e14aa62afbbf9bd5dc6464f534121caa215e84da10ff7e162422ddda`. Full source and replay hashes are also stored in the JSON ledger and per-record result files.

Observed output excerpt. Selected fields or wrapped lines only. Full stdout is preserved in the replay directory.

```
{
  "passed": true,
  "package_version": "1.0.0b4",
  "stable_symbol_count": 131,
  "finite_integral": "106/5",
  "typed_integral": "5/4"
}
```

EX-E-008. Legendre, Chebyshev and derivative transport

Source and placement. `komorebi-math/examples/classical_coordinate_families.py`. Chapter 17, classical coordinate families. Local claim: `EX-E-008-CONTRACT`.

Input (I). Degree 8; Legendre, Chebyshev-T and monomial families; exact coefficient vector (1,2,…,9).

Hypotheses and evidence class (H). EXACT rational coordinate algebra. The source fixture fixes all remaining literals and dimensions. The first observed certificate for each identifier, including checked and unchecked assumptions, is retained in the execution record.

Actions (A). Require the connection certificate, invert the transformed coefficients, and compare transported differentiation with direct family differentiation.

Expected result (O). The exact coefficient round-trip holds and derivative matrices agree. The connection determinant is 21091995799875/2251799813685248.

Certificate and replay checks (C). The source contains 2 explicit `assert` statements and 1 syntactic `.require()` call site. These are source counts, not dynamic proof counts. The installed-wheel run passed 2 output/artifact predicates with exit code zero and 244 package modules checked for import origin.

Observed base-certificate identifiers include `CC.P1`.

Failure boundary (F). A coefficient vector of the wrong degree must be rejected; a singular basis cannot supply an inverse.

Source API routes. `komorebi_math.coordinates.connection_transform`, `komorebi_math.coordinates.derivative_coordinate_operator`, `komorebi_math.coordinates.get_polynomial_family`, `komorebi_math.coordinates.transport_polynomial_operator`.

Reproducibility (R). `python examples/classical_coordinate_families.py`. Reference wheel, NumPy, and the logged validation environment with SciPy, SymPy and mpmath. Direct imports are listed separately; this profile is not a proof of minimal transitive dependencies. No random input in the shipped fixture.

Runtime and filesystem. Recorded Linux replay: 0.890 seconds, measured under the shared reference profile; this is not a performance promise. No persistent output files are declared by the example. Replay stdout/stderr and metadata are retained in the selected evidence directory.

Related package tests. `tests/test_coordinates_general_classical.py`, `tests/test_coordinates_polynomials.py`. These files were included in the separate 1,203-test source run. Shared imports alone do not prove full behavioral coverage.

Source SHA-256. `bd85e6b2ff7d9b5ced8a07e46e65be7dc8d01bebd4b37c564b78a23e13f79fd3`. Full source and replay hashes are also stored in the JSON ledger and per-record result files.

Observed output excerpt. Selected fields or wrapped lines only. Full stdout is preserved in the replay directory.

```
Legendre to Chebyshev determinant: 21091995799875/2251799813685248
Chebyshev coordinates: (Fraction(62417, 16384), Fraction(803, 128),
Fraction(13583, 2048), Fraction(719, 128), Fraction(24493, 4096),
Fraction(609, 128), Fraction(10329, 2048), Fraction(429, 128),
Fraction(57915, 16384))
```

EX-E-009. Recurrence coordinates and naturality

Source and placement. `komorebi-math/examples/coordinate_calculus.py`. Chapter 16, coordinate calculus. Local claim: `EX-E-009-CONTRACT`.

Input (I). Values 1 through 8; recurrence coefficients (1,1) and (2,-1,3); degree-10 Euler and Fibonacci polynomial systems.

Hypotheses and evidence class (H). EXACT finite coordinate algebra. The source fixture fixes all remaining literals and dimensions. The first observed certificate for each identifier, including checked and unchecked assumptions, is retained in the execution record.

Actions (A). Compare coordinate conversion with independent analysis in the target recurrence, then transport differentiation through the polynomial transform.

Expected result (O). Both displayed determinants are 1 and both explicit coordinate/naturality assertions pass.

Certificate and replay checks (C). The source contains 2 explicit `assert` statements and 0 syntactic `.require()` call sites. These are source counts, not dynamic proof counts. The installed-wheel run passed 2 output/artifact predicates with exit code zero and 244 package modules checked for import origin.

Failure boundary (F). Do not confuse a finite defect-coordinate conversion with a claim that the input obeys either homogeneous recurrence.

Source API routes. `komorebi_math.coordinates.analyze_recurrence_coordinates`, `komorebi_math.coordinates.coordinate_change_between_recurrences`, `komorebi_math.coordinates.derivative_operator_in_family`, `komorebi_math.coordina

tes.euler_polynomial_family, komorebi_math.coordinates.euler_to_fibonacci_transform and 2 more routes in the JSON ledger.

Reproducibility (R). `python examples/coordinate_calculus.py`. Reference wheel, NumPy, and the logged validation environment with SciPy, SymPy and mpmath. Direct imports are listed separately; this profile is not a proof of minimal transitive dependencies. No random input in the shipped fixture.

Runtime and filesystem. Recorded Linux replay: 0.932 seconds, measured under the shared reference profile; this is not a performance promise. No persistent output files are declared by the example. Replay stdout/stderr and metadata are retained in the selected evidence directory.

Related package tests. `tests/test_coordinates_recurrence.py`, `tests/test_coordinates_polynomials.py`, `tests/test_block19_integration.py`. These files were included in the separate 1,203-test source run. Shared imports alone do not prove full behavioral coverage.

Source SHA-256. `7085836c27f4dbd4f87db3c8b1f3c0d9c465a970564fc41b69608fcd0f3bd3fe`. Full source and replay hashes are also stored in the JSON ledger and per-record result files.

Observed output excerpt. Selected fields or wrapped lines only. Full stdout is preserved in the replay directory.

```
recurrence determinant 1
polynomial determinant 1
```

EX-E-010. Custom rational and algebraic finite series

Source and placement. `komorebi-math/examples/custom_generating_functions.py`. Chapters 21 through 24, generating functions. Local claim: `EX-E-010-CONTRACT`.

Input (I). Fibonacci rational function z/(1-z-z^2), the Catalan equation 1-y+zy^2=0 with y0=1, and a factorial-power basis of order 2.

Hypotheses and evidence class (H). EXACT finite prefixes. The source fixture fixes all remaining literals and dimensions. The first observed certificate for each identifier, including checked and unchecked assumptions, is retained in the execution record.

Actions (A). Register both functions, evaluate finite prefixes, multiply normalized series and serialize a frozen degree-12 Catalan prefix.

Expected result (O). Fibonacci coefficient 12 is 144; Catalan coefficient 8 is 1430; the product coefficients are (2,3,-10,109,-512,2000). The frozen portable prefix round-trips.

Certificate and replay checks (C). The source contains 3 explicit `assert` statements and 1 syntactic `.require()` call site. These are source counts, not dynamic proof counts. The installed-wheel run passed 3 output/artifact predicates with exit code zero and 245 package modules checked for import origin.

Observed base-certificate identifiers include `NGF.FINITE_NORMALIZED_SERIES`.

Failure boundary (F). A duplicate registry name or a non-simple algebraic branch must not be accepted as a new unambiguous function.

Source API routes. `komorebi_math.export.dumps_portable`, `komorebi_math.export.loads_portable`, `komorebi_math.generating.FiniteNormalizedSeries`, `komorebi_math.generating.FunctionRegistry`, `komorebi_math.generating.RationalDomain` and 4 more routes in the JSON ledger.

Reproducibility (R). `python examples/custom_generating_functions.py`. Reference wheel, NumPy, and the logged validation environment with SciPy, SymPy and mpmath. Direct imports are listed separately; this profile is not a proof of minimal transitive dependencies. No random input in the shipped fixture.

Runtime and filesystem. Recorded Linux replay: 0.815 seconds, measured under the shared reference profile; this is not a performance promise. No persistent output files are declared by the example. Replay stdout/stderr and metadata are retained in the selected evidence directory.

Related package tests. `tests/test_generating_block13.py`, `tests/test_block10_execution_persistence.py`, `tests/test_generating_foundation.py` and 2 more files in the shared-import crosswalk. These files were included in the separate 1,203-test source run. Shared imports alone do not prove full behavioral coverage.

Source SHA-256. `d42f374925bd24119a5177760718ad7d1f87f64fdd9d7eb0054f0129f9cd61a5`. Full source and replay hashes are also stored in the JSON ledger and per-record result files.

Observed output excerpt. Selected fields or wrapped lines only. Full stdout is preserved in the replay directory.

```
{'registry': ('catalan', 'fibonacci'), 'catalan_8': '1430', 'product': ('2',
 '3', '-10', '109', '-512', '2000'), 'portable_degree': 12}
```

EX-E-011. Cyclic projection, reciprocal coefficients and zeros

Source and placement. `komorebi-math/examples/cyclic_analytic.py`. Chapter 27, cyclic analytic systems. Local claim: `EX-E-011-CONTRACT`.

Input (I). Coefficients 0 through 14, cyclic order 4, reciprocal family order 2 and parameter 1, and the first four positive order-4 zeros.

Hypotheses and evidence class (H). EXACT coefficients / NUMERICAL roots. The source fixture fixes all remaining literals and dimensions. The first observed certificate for each identifier, including checked and unchecked assumptions, is retained in the execution record.

Actions (A). Require the finite projection certificate, check secant coefficients and two theta-polynomial roots, and require the numerical zero certificate.

Expected result (O). Secant coefficients begin (1,1,5,61); the first zero is approximately 2.2214414691. Numerical root certificates do not make the roots exact algebraic values.

Certificate and replay checks (C). The source contains 3 explicit `assert` statements and 2 syntactic `.require()` call sites. These are source counts, not dynamic proof counts. The installed-wheel run passed 2 output/artifact predicates with exit code zero and 244 package modules checked for import origin.

Observed base-certificate identifiers include `PSC-IV-T01`, `PSC-IV-T54`.

Failure boundary (F). Invalid cyclic orders or insufficient root-search conditions must fail rather than be represented as an exact zero list.

Source API routes. `komorebi_math.analytic.CyclicDenominator`, `komorebi_math.analytic.cyclic_projection_certificate`, `komorebi_math.analytic.reciprocal_coefficients`, `komorebi_math.analytic.theta_polynomial`.

Reproducibility (R). `python examples/cyclic_analytic.py`. Reference wheel, NumPy, and the logged validation environment with SciPy, SymPy and mpmath. Direct imports are listed separately; this profile is not a proof of minimal transitive dependencies. No random input in the shipped fixture.

Runtime and filesystem. Recorded Linux replay: 0.934 seconds, measured under the shared reference profile; this is not a performance promise. No persistent output files are declared by the example. Replay stdout/stderr and metadata are retained in the selected evidence directory.

Related package tests. `tests/test_analytic_appell_zeros.py`, `tests/test_analytic_cyclic_reciprocal.py`. These files were included in the separate 1,203-test source run. Shared imports alone do not prove full behavioral coverage.

Source SHA-256. `bc1eab12edd47ced72ea5669cc18b864a4d96a739a54d930cdc6391657b42e1e`. Full source and replay hashes are also stored in the JSON ledger and per-record result files.

Observed output excerpt. Selected fields or wrapped lines only. Full stdout is preserved in the replay directory.

```
cyclic analytic example passed (2.221441469079183, 6.66432440723755,
11.107207345395917, 15.550090283554283)
```

EX-E-012. Finite residue forest and occupancy

Source and placement. `komorebi-math/examples/deformation_support.py`. Chapter 56, deformation support. Local claim: `EX-E-012-CONTRACT`.

Input (I). Prime 2; residue levels {0,1}, {0,1,2,3}, {0,1,4,5}; nested coordinate subspaces of dimensions 3,2,1.

Hypotheses and evidence class (H). EXACT combinatorics / NUMERICAL subspaces. The source fixture fixes all remaining literals and dimensions. The first observed certificate for each identifier, including checked and unchecked assumptions, is retained in the execution record.

Actions (A). Build the finite forest and nested occupancy report and require their certificates; evaluate the finite normal-crossing count at q=3, depth=2.

Expected result (O). Level sizes are (2,4,4), stable-core sizes (2,2,4), branch count 4, intersection dimension 1 and normal-crossing count 21.

Certificate and replay checks (C). The source contains 0 explicit `assert` statements and 2 syntactic `.require()` call sites. These are source counts, not dynamic proof counts. The installed-wheel run passed 4 output/artifact predicates with exit code zero and 244 package modules checked for import origin.

Observed base-certificate identifiers include `PSC-X-FINITE-OCCUPANCY`, `PSC-X-LIFT-FOREST`.

Failure boundary (F). This finite residue forest does not certify infinite persistence or an arithmetic deformation theorem.

Source API routes. `komorebi_math.deformation.nested_subspace_occupancy`, `komorebi_math.deformation.normal_crossing_count`, `komorebi_math.deformation.residue_forest`.

Reproducibility (R). `python examples/deformation_support.py`. Reference wheel, NumPy, and the logged validation environment with SciPy, SymPy and mpmath. Direct imports are listed separately; this profile is not a proof of minimal transitive dependencies. No random input in the shipped fixture.

Runtime and filesystem. Recorded Linux replay: 0.823 seconds, measured under the shared reference profile; this is not a performance promise. No persistent output files are declared by the example. Replay stdout/stderr and metadata are retained in the selected evidence directory.

Related package tests. `tests/test_deformation_complex_support_companions.py`, `tests/test_deformation_forests_visibility_counting.py`. These files were included in the separate 1,203-test source run. Shared imports alone do not prove full behavioral coverage.

Source SHA-256. `bf7389f272d9d7caab6da2a3fe084b9ad8d86daae85f23b039adb3dfd17e5a11`. Full source and replay hashes are also stored in the JSON ledger and per-record result files.

Observed output excerpt. Selected fields or wrapped lines only. Full stdout is preserved in the replay directory.

```
{'level_sizes': (2, 4, 4), 'stable_core_sizes': (2, 2, 4), 'branch_count':
4, 'intersection_dimension': 1, 'normal_crossing_count_q3_depth2': 21}
```

EX-E-013. Finite Euclidean transfer and lattice spectrum

Source and placement. `komorebi-math/examples/euclidean_spectral.py`. Chapter 53, Euclidean quantum spectral systems. Local claim: `EX-E-013-CONTRACT`.

Input (I). Diagonal Hamiltonian (0,1,2), time step 0.2, a periodic 4-by-5 lattice and three explicitly bounded 2-by-2 Hamiltonians.

Hypotheses and evidence class (H). NUMERICAL finite-dimensional. The source fixture fixes all remaining literals and dimensions. The first observed certificate for each identifier, including checked and unchecked assumptions, is retained in the execution record.

Actions (A). Check reflection positivity for the supplied vectors/times, compare dense eigenvalues with the lattice formula and require the finite tower report.

Expected result (O). The lattice spectra agree to numerical tolerance and the spectral-dimension value is nonnegative. The success line follows all checks.

Certificate and replay checks (C). The source contains 2 explicit `assert` statements and 2 syntactic `.require()` call sites. These are source counts, not dynamic proof counts. The installed-wheel run passed 1 output/artifact predicate with exit code zero and 244 package modules checked for import origin.

Observed base-certificate identifiers include `PSC-VII-T56`, `PSC-VII-T70`.

Failure boundary (F). No continuum reflection-positivity, spectral-dimension limit or quantum field existence statement follows from this finite example.

Source API routes. `komorebi_math.quantum.BoundedHamiltonianTower`, `komorebi_math.quantum.EuclideanTransfer`, `komorebi_math.quantum.periodic_laplacian`, `komorebi_math.quantum.periodic_lattice_eigenvalues`, `komorebi_math.quantum.spectral_dimension`.

Reproducibility (R). `python examples/euclidean_spectral.py`. Reference wheel, NumPy, and the logged validation environment with SciPy, SymPy and mpmath. Direct imports are listed separately; this profile is not a proof of minimal transitive dependencies. No random input in the shipped fixture.

Runtime and filesystem. Recorded Linux replay: 1.226 seconds, measured under the shared reference profile; this is not a performance promise. No persistent output files are declared by the example. Replay stdout/stderr and metadata are retained in the selected evidence directory.

Related package tests. `tests/test_quantum_transport_effective_spectral.py`. These files were included in the separate 1,203-test source run. Shared imports alone do not prove full behavioral coverage.

Source SHA-256. `bdb08297db34b86e1d9c2205b924a38d035782b83b2512a48e3bbb611bf37de1`. Full source and replay hashes are also stored in the JSON ledger and per-record result files.

Observed output excerpt. Selected fields or wrapped lines only. Full stdout is preserved in the replay directory.

```
Euclidean and spectral example passed
```

EX-E-014. Euler data in Fibonacci defect coordinates

Source and placement. `komorebi-math/examples/euler_to_fibonacci_coordinates.py`. Chapters 17 and 65, classical-family conversion. Local claim: `EX-E-014-CONTRACT`.

Input (I). Eighteen Euler-sequence values generated by the named coordinate constructor.

Hypotheses and evidence class (H). EXACT finite recurrence coordinates. The source fixture fixes all remaining literals and dimensions. The first observed certificate for each identifier, including checked and unchecked assumptions, is retained in the execution record.

Actions (A). Require the coordinate certificate and compare direct reconstruction with the variation-of-constants prefix.

Expected result (O). Initial data are (1,0); the reconstructed prefix begins 1,0,-1,0,5,0,-61,0. Sixteen defect entries accompany the two initial values.

Certificate and replay checks (C). The source contains 1 explicit `assert` statement and 1 syntactic `.require()` call site. These are source counts, not dynamic proof counts. The installed-wheel run passed 2 output/artifact predicates with exit code zero and 244 package modules checked for import origin.

Observed base-certificate identifiers include `CC.R1`.

Failure boundary (F). Dropping the defects would incorrectly claim a homogeneous Fibonacci recurrence for the Euler sequence.

Source API routes. `komorebi_math.coordinates.euler_fibonacci_coordinates`, `komorebi_math.coordinates.variation_of_constants_prefix`.

Reproducibility (R). `python examples/euler_to_fibonacci_coordinates.py`. Reference wheel, NumPy, and the logged validation environment with SciPy, SymPy and mpmath. Direct imports are listed separately; this profile is not a proof of minimal transitive dependencies. No random input in the shipped fixture.

Runtime and filesystem. Recorded Linux replay: 0.884 seconds, measured under the shared reference profile; this is not a performance promise. No persistent output files are declared by the example. Replay stdout/stderr and metadata are retained in the selected evidence directory.

Related package tests. `tests/test_coordinates_normalized_persistence.py`, `tests/test_coordinates_recurrence.py`, `tests/test_coordinates_realization.py`. These files were included in the separate 1,203-test source run. Shared imports alone do not prove full behavioral coverage.

Source SHA-256. `5b362d07c86d368036865082633d60ccc6a6f300a8a5dfa3b5ff668740b59ddd`. Full source and replay hashes are also stored in the JSON ledger and per-record result files.

Observed output excerpt. Selected fields or wrapped lines only. Full stdout is preserved in the replay directory.

```
initial ('1', '0')
defects ('-2', '1', '6', '-5', '-66', '61', '1446', '-1385', '-51906',
'50521', '2753286', '-2702765', '-202063746', '199360981', '19590873126',
'-19391512145')
reconstructed ('1', '0', '-1', '0', '5', '0', '-61', '0', '1385', '0',
'-50521', '0', '2702765', '0', '-199360981', '0', '19391512145', '0')
```

EX-E-015. Symbolic backend and seed rank

Source and placement. `komorebi-math/examples/exact_backend.py`. Chapter 8, exact scalar backends. Local claim: `EX-E-015-CONTRACT`.

Input (I). SymPy nilpotent matrix [[0,1],[0,0]], two coordinate projectors and the direct sum of a size-2 Jordan block at 4 with a size-1 block at 4.

Hypotheses and evidence class (H). EXACT SymPy linear algebra. The source fixture fixes all remaining literals and dimensions. The first observed certificate for each identifier, including checked and unchecked assumptions, is retained in the execution record.

Actions (A). Construct the operator algebra and packet system with backend=sympy; print the packet certificate and compute the single-operator seed rank.

Expected result (O). Algebra dimension and seed rank are both 2. The printed packet certificate is valid and tagged exact.

Certificate and replay checks (C). The source contains 0 explicit `assert` statements and 0 syntactic `.require()` call sites. These are source counts, not dynamic proof counts. The installed-wheel run passed 4 output/artifact predicates with exit code zero and 245 package modules checked for import origin.

Observed base-certificate identifiers include `PSC-I-T20`.

Failure boundary (F). This example requires SymPy. A missing optional symbolic backend is a dependency failure, not a successful NumPy substitute.

Source API routes. `komorebi_math.OperatorAlgebra`, `komorebi_math.PacketSystem`, `komorebi_math.single_operator_seed_rank`.

Reproducibility (R). `python examples/exact_backend.py`. Reference wheel, NumPy, and the logged validation environment with SciPy, SymPy and mpmath. SymPy is imported directly and is required for this fixture. No random input in the shipped fixture.

Runtime and filesystem. Recorded Linux replay: 1.448 seconds, measured under the shared reference profile; this is not a performance promise. No persistent output files are declared by the example. Replay stdout/stderr and metadata are retained in the selected evidence directory.

Related package tests. `tests/test_operators_transfer.py`, `tests/test_nonlinear_trees.py`, `tests/test_morphisms.py` and 16 more files in the shared-import crosswalk. These files were included in the separate 1,203-test source run. Shared imports alone do not prove full behavioral coverage.

Source SHA-256. `28d479db2e1a20588c940020c513b88b7e9b119d6a7bae8c8edde5c3796aaa08`. Full source and replay hashes are also stored in the JSON ledger and per-record result files.

Observed output excerpt. Selected fields or wrapped lines only. Full stdout is preserved in the replay directory.

```
algebra dimension: 2
{
  "assumptions_checked": [
    "common dimension",
    "idempotence",
    "annihilation",
    "completeness"
  ],
```

EX-E-016. Lattice gluing and Smith congruences

Source and placement. `komorebi-math/examples/exact_lattice_separation.py`. Chapter 13, lattice separation. Local claim: `EX-E-016-CONTRACT`.

Input (I). P0=[[1,1/4],[0,0]], P1=I-P0, split basis [[1,-1],[0,4]], prime 2; congruence diag(2,4) on three variables at depth 3.

Hypotheses and evidence class (H). EXACT rational/integer lattice algebra. The source fixture fixes all remaining literals and dimensions. The first observed certificate for each identifier, including checked and unchecked assumptions, is retained in the execution record.

Actions (A). Require the gluing-weight certificate and solve the exact prime-power congruence.

Expected result (O). Projector weights (2,2), gluing invariant factors (1,4), and exactly 64 solutions.

Certificate and replay checks (C). The source contains 3 explicit `assert` statements and 2 syntactic `.require()` call sites. These are source counts, not dynamic proof counts. The installed-wheel run passed 3 output/artifact predicates with exit code zero and 244 package modules checked for import origin.

Observed base-certificate identifiers include `PSC-II-D12`, `PSC-II-SNF`, `PSC-II-T13`, `PSC-II-T18`.

Failure boundary (F). A composite prime parameter, invalid split basis or inconsistent congruence must not yield a valid solution certificate.

Source API routes. `komorebi_math.LatticePacketData`, `komorebi_math.SmithCongruenceProblem`.

Reproducibility (R). `python examples/exact_lattice_separation.py`. Reference wheel, NumPy, and the logged validation environment with SciPy, SymPy and mpmath. SymPy is imported directly and is required for this fixture. No random input in the shipped fixture.

Runtime and filesystem. Recorded Linux replay: 1.344 seconds, measured under the shared reference profile; this is not a performance promise. No persistent output files are declared by the example. Replay stdout/stderr and metadata are retained in the selected evidence directory.

Source SHA-256. `9b79fbd8bf454b9a68237a62d54873fc906bcfc165354f9758b95ad8db1d2fe3`. Full source and replay hashes are also stored in the JSON ledger and per-record result files.

Observed output excerpt. Selected fields or wrapped lines only. Full stdout is preserved in the replay directory.

```
projector weights: (2, 2)
gluing factors: (1, 4)
congruence solutions: 64
```

EX-E-017. Multi-format reports and export artifacts

Source and placement. `komorebi-math/examples/export_reports.py`. Chapters 59 through 60 and 70, reports and rendering. Local claim: `EX-E-017-CONTRACT`.

Input (I). Twenty-four Fibonacci values and the certified Euler coordinate at index 20, depth 2. Defaults: output=komorebi_export_demo, engine=auto.

Hypotheses and evidence class (H). EXACT data / FORMAT and compiler checks. The source fixture fixes all remaining literals and dimensions. The first observed certificate for each identifier, including checked and unchecked assumptions, is retained in the execution record.

Actions (A). Build a MathDocument, export text/Markdown/HTML/TeX and sequence CSV/JSON, optionally compile PDFs, then build an export ZIP.

Expected result (O). The JSON and CSV contain the exact 24 indexed Fibonacci values. All declared files are nonempty; export certificates pass. PDF is conditional on an available LaTeX engine.

Certificate and replay checks (C). The source contains 0 explicit `assert` statements and 6 syntactic `.require()` call sites. These are source counts, not dynamic proof counts. The installed-wheel run passed 1 output/artifact predicate with exit code zero and 244 package modules checked for import origin.

Observed base-certificate identifiers include `PSC-IX-T19`.

Failure boundary (F). The script uses overwrite=True. Run only in a disposable output directory. Missing LaTeX must be distinguished from a failed available compiler.

Source API routes. `komorebi_math.EulerLocalCoordinates`, `komorebi_math.export.DocumentMetadata`, `komorebi_math.export.ExportOptions`, `komorebi_math.export.LatexCompileOptions`, `komorebi_math.export.MathDocument` and 5 more routes in the JSON ledger.

Reproducibility (R). `python examples/export_reports.py`. Reference wheel, NumPy, and the logged validation environment with SciPy, SymPy and mpmath. Direct imports are listed separately; this profile is not a proof of minimal transitive dependencies. No random input in the shipped fixture.

Runtime and filesystem. Recorded Linux replay: 8.433 seconds, measured under the shared reference profile; this is not a performance promise. Writes komorebi_export_demo under the replay working directory with overwrite=True. The runner retains all exports and their hashes. Delete only the disposable run directory after preserving evidence.

Related package tests. `tests/test_export_renderers.py`, `tests/test_export_cli.py`, `tests/test_sequences_execution.py` and 2 more files in the shared-import crosswalk. These files were included in the separate 1,203-test source run. Shared imports alone do not prove full behavioral coverage.

Source SHA-256. `ea0aeca271247086dd1b8993c23b303254e62e27c130e2eee5c2774a1995b893`. Full source and replay hashes are also stored in the JSON ledger and per-record result files.

Observed output excerpt. Selected fields or wrapped lines only. Full stdout is preserved in the replay directory.

```
No stdout. Verified artifact files:
komorebi_export_demo/fibonacci.csv
komorebi_export_demo/fibonacci.json
komorebi_export_demo/fibonacci_bundle.zip
komorebi_export_demo/fibonacci_bundle.zip.manifest
komorebi_export_demo/report.html
komorebi_export_demo/report.komorebi.log
komorebi_export_demo/report.md
komorebi_export_demo/report.pdf
komorebi_export_demo/report.tex
komorebi_export_demo/report.txt
```

EX-E-018. Feshbach self-energy and nonlocal memory

Source and placement. `komorebi-math/examples/feshbach_memory.py`. Chapter 46, Feshbach memory kernels. Local claim: `EX-E-018-CONTRACT`.

Input (I). The shipped 4-by-4 dissipative operator, retained indices (0,1), resolvent points 2+0.3i and 2+0.2i, horizon 15 and 3000 quadrature points.

Hypotheses and evidence class (H). NUMERICAL finite resolvent and quadrature. The source fixture fixes all remaining literals and dimensions. The first observed certificate for each identifier, including checked and unchecked assumptions, is retained in the execution record.

Actions (A). Require resolvent and memory-kernel certificates, then inspect the zero-time kernel and local-closure diagnostic.

Expected result (O). The zero-time kernel is diag(0.12,-0.1). Exact local closure is False by design: memory remains present.

Certificate and replay checks (C). The source contains 0 explicit `assert` statements and 2 syntactic `.require()` call sites. These are source counts, not dynamic proof counts. The installed-wheel run passed 3 output/artifact predicates with exit code zero and 244 package modules checked for import origin.

Observed base-certificate identifiers include `PSC-VI-T28`, `PSC-VI-T30`.

Failure boundary (F). Requiring every printed Boolean to be true would be wrong. The negative local-closure diagnostic is the intended result.

Source API routes. `komorebi_math.dynamics.FeshbachReduction`.

Reproducibility (R). `python examples/feshbach_memory.py`. Reference wheel, NumPy, and the logged validation environment with SciPy, SymPy and mpmath. Direct imports are listed separately; this profile is not a proof of minimal transitive dependencies. No random input in the shipped fixture.

Runtime and filesystem. Recorded Linux replay: 1.201 seconds, measured under the shared reference profile; this is not a performance promise. No persistent output files are declared by the example. Replay stdout/stderr and metadata are retained in the selected evidence directory.

Related package tests. `tests/test_dynamics_feshbach.py`. These files were included in the separate 1,203-test source run. Shared imports alone do not prove full behavioral coverage.

Source SHA-256. `cd45ded4bbf8c931b7f186384781ad8839c4a9073744ee6ce118c544d48de346`. Full source and replay hashes are also stored in the JSON ledger and per-record result files.

Observed output excerpt. Selected fields or wrapped lines only. Full stdout is preserved in the replay directory.

```
self energy at z=2+0.3i:
[[ 0.03960396-0.0039604j   0.00143203-0.00027196j]
 [-0.        +0.j         -0.02918455+0.00257511j]]
memory kernel at zero:
[[ 0.12  0.  ]
 [ 0.   -0.1 ]]
exact local closure: False
```

EX-E-019. Filtered pages and moving-projector transport

Source and placement. `komorebi-math/examples/filtered_transport.py`. Chapters 42 and 44, filtered transport. Local claim: `EX-E-019-CONTRACT`.

Input (I). A one-dimensional cochain map over prime 5 with shifted filtration; a 2-by-2 rotation generator of speed 0.4 and 32 coarse time steps.

Hypotheses and evidence class (H). EXACT finite filtration / NUMERICAL transport. The source fixture fixes all remaining literals and dimensions. The first observed certificate for each identifier, including checked and unchecked assumptions, is retained in the execution record.

Actions (A). Require the finite spectral-sequence certificate and compare coarse/fine Kato transport after a derivative check.

Expected result (O). Page dimensions/ranks are (2,0),(2,1),(0,0); the coarse/fine transport difference must be at most 1e-6.

Certificate and replay checks (C). The source contains 0 explicit `assert` statements and 3 syntactic `.require()` call sites. These are source counts, not dynamic proof counts. The installed-wheel run passed 4 output/artifact predicates with exit code zero and 244 package modules checked for import origin.

Observed base-certificate identifiers include `PSC-III-C60`, `PSC-III-D56`, `PSC-III-S54`.

Failure boundary (F). A filtration that the differential does not preserve must not be certified. Coarse/fine agreement is not a global error theorem.

Source API routes. `komorebi_math.dynamics.matrix_exponential`, `komorebi_math.nonlinear.FilteredVectorSpace`, `komorebi_math.nonlinear.FiniteFilteredCochainComplex`, `komorebi_math.nonlinear.KatoTransport`, `komorebi_math.nonlinear.MovingPacketSystem`.

Reproducibility (R). `python examples/filtered_transport.py`. Reference wheel, NumPy, and the logged validation environment with SciPy, SymPy and mpmath. Direct imports are listed separately; this profile is not a proof of minimal transitive dependencies. No random input in the shipped fixture.

Runtime and filesystem. Recorded Linux replay: 1.288 seconds, measured under the shared reference profile; this is not a performance promise. No persistent output files are declared by the example. Replay stdout/stderr and metadata are retained in the selected evidence directory.

Related package tests. `tests/test_block4_hardening.py`, `tests/test_nonlinear_transport.py`, `tests/test_quantum_transport_effective_spectral.py` and 1 more file in the shared-import crosswalk. These files were included in the separate 1,203-test source run. Shared imports alone do not prove full behavioral coverage.

Source SHA-256. `ecd83b558196743d21639f48cafc0f111a552ed9e421f63023126fddf3acd2bb`. Full source and replay hashes are also stored in the JSON ledger and per-record result files.

Observed output excerpt. Selected fields or wrapped lines only. Full stdout is preserved in the replay directory.

```
page 0 dimension 2 rank 0
page 1 dimension 2 rank 1
page 2 dimension 0 rank 0
transport coarse-fine difference: 4.649628121515814e-15
```

EX-E-020. Induced finite representation and unresolved descent

Source and placement. `komorebi-math/examples/finite_image_clifford.py`. Chapter 58, finite-image and Clifford models. Local claim: `EX-E-020-CONTRACT`.

Input (I). C4 with subgroup C2 and character (1,-1), symmetric degree 3, and three supplied Clifford steps forming EIE.

Hypotheses and evidence class (H). FINITE representation model / CONDITIONAL descent. The source fixture fixes all remaining literals and dimensions. The first observed certificate for each identifier, including checked and unchecked assumptions, is retained in the execution record.

Actions (A). Require the induced representation, packet-dimension and finite Clifford report certificates.

Expected result (O). Induced dimension 2, symmetric packet dimensions (2,2), word EIE, depth 1 and unresolved extension step (0,).

Certificate and replay checks (C). The source contains 0 explicit `assert` statements and 3 syntactic `.require()` call sites. These are source counts, not dynamic proof counts. The installed-wheel run passed 4 output/artifact predicates with exit code zero and 244 package modules checked for import origin.

Observed base-certificate identifiers include `KM-INDUCED-REPRESENTATION`, `PSC-LANG-CLIFFORD-DEPTH`, `PSC-LANG-MONOMIAL-ORBITS`.

Failure boundary (F). The unresolved extension must remain visible. A finite Clifford word is not evidence that an unattached descent has been constructed.

Source API routes. `komorebi_math.langlands.finite_image.CliffordStep`, `komorebi_math.langlands.finite_image.CliffordTower`, `komorebi_math.langlands.finite_image.InducedRepresentation`, `komorebi_math.langlands.finite_image.LinearCharacter`, `komorebi_math.langlands.finite_image.MonomialAction` and 1 more route in the JSON ledger.

Reproducibility (R). `python examples/finite_image_clifford.py`. Reference wheel, NumPy, and the logged validation environment with SciPy, SymPy and mpmath. Direct imports are listed separately; this profile is not a proof of minimal transitive dependencies. No random input in the shipped fixture.

Runtime and filesystem. Recorded Linux replay: 0.867 seconds, measured under the shared reference profile; this is not a performance promise. No persistent output files are declared by the example. Replay stdout/stderr and metadata are retained in the selected evidence directory.

Related package tests. `tests/test_langlands_finite_image.py`. These files were included in the separate 1,203-test source run. Shared imports alone do not prove full behavioral coverage.

Source SHA-256. `c7876aa3661747d437d26b4174acf8b3186017c9fd2cbd2ca94d871e50f74f16`. Full source and replay hashes are also stored in the JSON ledger and per-record result files.

Observed output excerpt. Selected fields or wrapped lines only. Full stdout is preserved in the replay directory.

```
{'induced_dimension': 2, 'symmetric_packet_dimensions': (2, 2),
'clifford_word': 'EIE', 'clifford_depth': 1, 'unresolved_extension_steps':
(0,)}
```

EX-E-021. Affine, barycentric and formal changes

Source and placement. `komorebi-math/examples/formal_affine_coordinate_changes.py`. Chapters 15 through 16, formal affine changes. Local claim: `EX-E-021-CONTRACT`.

Input (I). Origin (3,-2), basis ((1,1),(1,-1)), local point (5/2,-3/2); triangle ((0,0),(2,0),(0,3)); formal coefficients (0,1,1,-2,3).

Hypotheses and evidence class (H). EXACT finite rational algebra. The source fixture fixes all remaining literals and dimensions. The first observed certificate for each identifier, including checked and unchecked assumptions, is retained in the execution record.

Actions (A). Invert affine and barycentric coordinates and pull/push a finite series through the certified formal coordinate change.

Expected result (O). Cartesian point (4,2), barycentric image (2/3,3/2), and formal pullback (3,4,3,-8,26); all finite round-trips hold.

Certificate and replay checks (C). The source contains 3 explicit `assert` statements and 1 syntactic `.require()` call site. These are source counts, not dynamic proof counts. The installed-wheel run passed 3 output/artifact predicates with exit code zero and 244 package modules checked for import origin.

Observed base-certificate identifiers include `CC.J1`.

Failure boundary (F). A singular affine basis, degenerate simplex or zero formal linear coefficient must not be treated as invertible.

Source API routes. `komorebi_math.coordinates.BarycentricCoordinateSystem`, `komorebi_math.coordinates.FormalCoordinateChange`, `komorebi_math.coordinates.affine_basis_transform`.

Reproducibility (R). `python examples/formal_affine_coordinate_changes.py`. Reference wheel, NumPy, and the logged validation environment with SciPy, SymPy and mpmath. Direct imports are listed separately; this profile is not a proof of minimal transitive dependencies. No random input in the shipped fixture.

Runtime and filesystem. Recorded Linux replay: 0.837 seconds, measured under the shared reference profile; this is not a performance promise. No persistent output files are declared by the example. Replay stdout/stderr and metadata are retained in the selected evidence directory.

Related package tests. `tests/test_coordinates_general_persistence.py`. These files were included in the separate 1,203-test source run. Shared imports alone do not prove full behavioral coverage.

Source SHA-256. `e5503b36732eb502882387df8b314cf8672b2d3178721a9895b6e3fea4831e8b`. Full source and replay hashes are also stored in the JSON ledger and per-record result files.

Observed output excerpt. Selected fields or wrapped lines only. Full stdout is preserved in the replay directory.

```
Cartesian point: (Fraction(4, 1), Fraction(2, 1))
barycentric point: (Fraction(2, 3), Fraction(3, 2))
formal pullback: (Fraction(3, 1), Fraction(4, 1), Fraction(3, 1),
Fraction(-8, 1), Fraction(26, 1))
```

EX-E-022. Fractional transfer and phase-type survival

Source and placement. `komorebi-math/examples/fractional_survival.py`. Chapter 46, fractional survival. Local claim: `EX-E-022-CONTRACT`.

Input (I). Positive diagonal operator (1,4,9), exponent 1/2, time 0.7; two-state subgenerator [[-0.7,0.2],[0.3,-0.6]] and initial mass (1,0).

Hypotheses and evidence class (H). NUMERICAL finite-dimensional. The source fixture fixes all remaining literals and dimensions. The first observed certificate for each identifier, including checked and unchecked assumptions, is retained in the execution record.

Actions (A). Compute the spectral square root and transfer, then require the survival and identity certificates.

Expected result (O). Square-root spectrum (1,2,3), survival at time 1 approximately 0.6703200460 and mean lifetime 2.5.

Certificate and replay checks (C). The source contains 0 explicit `assert` statements and 2 syntactic `.require()` call sites. These are source counts, not dynamic proof counts. The installed-wheel run passed 3 output/artifact predicates with exit code zero and 244 package modules checked for import origin.

Observed base-certificate identifiers include `PSC-VI-T32`, `PSC-VI-T33`.

Failure boundary (F). Invalid probability mass or a non-transient subgenerator must not be accepted as a normalized survival model.

Source API routes. `komorebi_math.dynamics.PhaseTypeSurvival`, `komorebi_math.dynamics.fractional_power`, `komorebi_math.dynamics.fractional_semigroup`.

Reproducibility (R). `python examples/fractional_survival.py`. Reference wheel, NumPy, and the logged validation environment with SciPy, SymPy and mpmath. Direct imports are listed separately; this profile is not a proof of minimal transitive dependencies. No random input in the shipped fixture.

Runtime and filesystem. Recorded Linux replay: 1.206 seconds, measured under the shared reference profile; this is not a performance promise. No persistent output files are declared by the example. Replay stdout/stderr and metadata are retained in the selected evidence directory.

Related package tests. `tests/test_block4_hardening.py`, `tests/test_dynamics_fractional_green_feynman_survival.py`. These files were included in the separate 1,203-test source run. Shared imports alone do not prove full behavioral coverage.

Source SHA-256. `bd0e7f3364aaa98a964a6dbbd3358f970b2f020699599fb8aa74ceee58b97a90`. Full source and replay hashes are also stored in the JSON ledger and per-record result files.

Observed output excerpt. Selected fields or wrapped lines only. Full stdout is preserved in the replay directory.

```
square root spectrum: [1. 2. 3.]
fractional transfer diagonal: [0.4965853  0.24659696 0.12245643]
survival at t=1: 0.6703200460356394
mean lifetime: 2.5
```

EX-E-023. Finite Frobenius separator and carrier multiplicities

Source and placement. `komorebi-math/examples/frobenius_companions.py`. Chapter 56, Frobenius companions. Local claim: `EX-E-023-CONTRACT`.

Input (I). Frobenius polynomials at 2 and 3; trace rows (1,1),(1,-1),(2,0); observed trace (5,1); carrier capacities (4,2).

Hypotheses and evidence class (H). EXACT finite separator / CONDITIONAL carrier input. The source fixture fixes all remaining literals and dimensions. The first observed certificate for each identifier, including checked and unchecked assumptions, is retained in the execution record.

Actions (A). Build the finite separator, require its certificate, recover multiplicities and select within the declared carrier.

Expected result (O). Rank 2, separating places (2,3), multiplicities (3,2), selected rho1=3 and rho2=2.

Certificate and replay checks (C). The source contains 0 explicit `assert` statements and 1 syntactic `.require()` call site. These are source counts, not dynamic proof counts. The installed-wheel run passed 3 output/artifact predicates with exit code zero and 244 package modules checked for import origin.

Observed base-certificate identifiers include `PSC-X-FROBENIUS-SEPARATOR`.

Failure boundary (F). This verifies supplied finite traces and capacities, not the existence of compatible global Galois or automorphic objects.

Source API routes. `komorebi_math.deformation.CompatibleCarrier`, `komorebi_math.deformation.FrobeniusSpine`, `komorebi_math.deformation.finite_frobenius_separator`.

Reproducibility (R). `python examples/frobenius_companions.py`. Reference wheel, NumPy, and the logged validation environment with SciPy, SymPy and mpmath. Direct imports are listed separately; this profile is not a proof of minimal transitive dependencies. No random input in the shipped fixture.

Runtime and filesystem. Recorded Linux replay: 0.962 seconds, measured under the shared reference profile; this is not a performance promise. No persistent output files are declared by the example. Replay stdout/stderr and metadata are retained in the selected evidence directory.

Related package tests. `tests/test_deformation_complex_support_companions.py`. These files were included in the separate 1,203-test source run. Shared imports alone do not prove full behavioral coverage.

Source SHA-256. `1bd76f1683ad2a7827e1d018c229ebae503dad726467e125f683ba39cd144b14`. Full source and replay hashes are also stored in the JSON ledger and per-record result files.

Observed output excerpt. Selected fields or wrapped lines only. Full stdout is preserved in the replay directory.

```
{'spine_rank': 2, 'separating_places': (2, 3), 'multiplicities': (3, 2),
'selected_carrier': {'rho1': 3, 'rho2': 2}}
```

EX-E-024. Three-basis exact atlas

Source and placement. `komorebi-math/examples/general_coordinate_atlas.py`. Chapters 15 through 19, coordinate atlas. Local claim: `EX-E-024-CONTRACT`.

Input (I). Standard basis in dimension 3, upper-unitriangular shear, diagonal scaling (2,3,5), and vector (7,-3,11).

Hypotheses and evidence class (H). EXACT finite rational algebra. The source fixture fixes all remaining literals and dimensions. The first observed certificate for each identifier, including checked and unchecked assumptions, is retained in the execution record.

Actions (A). Register direct and inverse changes, compare the direct and two-step paths, then synthesize from scaled coordinates.

Expected result (O). The direct route is standard to scaled; exact coordinates are (7/2,-1,11/5) and reconstruct the vector.

Certificate and replay checks (C). The source contains 1 explicit `assert` statement and 1 syntactic `.require()` call site. These are source counts, not dynamic proof counts. The installed-wheel run passed 2 output/artifact predicates with exit code zero and 244 package modules checked for import origin.

Observed base-certificate identifiers include `CC.A2`.

Failure boundary (F). A conflicting direct edge must produce a noncommuting-diagram diagnostic, not silently replace the disagreement.

Source API routes. `komorebi_math.coordinates.CoordinateAtlas`, `komorebi_math.coordinates.CoordinateSystem`, `komorebi_math.coordinates.basis_from_matrix`, `komorebi_math.coordinates.standard_basis`.

Reproducibility (R). `python examples/general_coordinate_atlas.py`. Reference wheel, NumPy, and the logged validation environment with SciPy, SymPy and mpmath. Direct imports are listed separately; this profile is not a proof of minimal transitive dependencies. No random input in the shipped fixture.

Runtime and filesystem. Recorded Linux replay: 0.890 seconds, measured under the shared reference profile; this is not a performance promise. No persistent output files are declared by the example. Replay stdout/stderr and metadata are retained in the selected evidence directory.

Related package tests. `tests/test_coordinates_general_persistence.py`. These files were included in the separate 1,203-test source run. Shared imports alone do not prove full behavioral coverage.

Source SHA-256. `9f5fd4259beacf4b50f3b7867b0c81174a69219db6965ca5b2c60232ac8f3779`. Full source and replay hashes are also stored in the JSON ledger and per-record result files.

Observed output excerpt. Selected fields or wrapped lines only. Full stdout is preserved in the replay directory.

```
path: ('standard', 'scaled')
scaled coordinates: (Fraction(7, 2), Fraction(-1, 1), Fraction(11, 5))
```

EX-E-025. Gauge-invariant gluing with a broken-cycle control

Source and placement. `komorebi-math/examples/holonomy.py`. Chapter 40, holonomy. Local claim: `EX-E-025-CONTRACT`.

Input (I). Two shear matrices A,B, direct edge C=B A, three vertices and invertible diagonal gauges. The negative graph uses direct edge 2C.

Hypotheses and evidence class (H). NUMERICAL finite matrix relations. The source fixture fixes all remaining literals and dimensions. The first observed certificate for each identifier, including checked and unchecked assumptions, is retained in the execution record.

Actions (A). Require gluing before and after gauge change and assert that the deliberately broken graph is invalid.

Expected result (O). One fundamental cycle, gauge-invariant gluing True and broken cycle detected True.

Certificate and replay checks (C). The source contains 1 explicit `assert` statement and 2 syntactic `.require()` call sites. These are source counts, not dynamic proof counts. The installed-wheel run passed 3 output/artifact predicates with exit code zero and 244 package modules checked for import origin.

Observed base-certificate identifiers include `PSC-III-T40`.

Failure boundary (F). The shipped 2C edge is an executed negative control. Flatness is only asserted for this finite graph.

Source API routes. `komorebi_math.TransportEdge`, `komorebi_math.TransportGraph`.

Reproducibility (R). `python examples/holonomy.py`. Reference wheel, NumPy, and the logged validation environment with SciPy, SymPy and mpmath. Direct imports are listed separately; this profile is not a proof of minimal transitive dependencies. No random input in the shipped fixture.

Runtime and filesystem. Recorded Linux replay: 0.875 seconds, measured under the shared reference profile; this is not a performance promise. No persistent output files are declared by the example. Replay stdout/stderr and metadata are retained in the selected evidence directory.

Source SHA-256. `c8eb15199c3e3b812f13152a5fefa2b4b0a91a4a50412e84487db91a3eec5487`. Full source and replay hashes are also stored in the JSON ledger and per-record result files.

Observed output excerpt. Selected fields or wrapped lines only. Full stdout is preserved in the replay directory.

```
fundamental cycles: 1
gauge invariant gluing: True
broken cycle detected: True
```

EX-E-026. Quantum transport and Feshbach Laplace identity

Source and placement. `komorebi-math/examples/kato_feshbach.py`. Chapters 45 through 46, Kato-Feshbach dynamics. Local claim: `EX-E-026-CONTRACT`.

Input (I). Rotation generator speed 0.3, 160 time steps, Hamiltonian [[0.2,0.3],[0.3,1.4]], retained coordinate 0 and z=2.1+0.4i.

Hypotheses and evidence class (H). NUMERICAL finite quantum model. The source fixture fixes all remaining literals and dimensions. The first observed certificate for each identifier, including checked and unchecked assumptions, is retained in the execution record.

Actions (A). Require transport and resolvent certificates and compare kernel.laplace(-i z) with i times self-energy.

Expected result (O). The explicit Laplace identity holds with atol=1e-8 and the success line follows the checks.

Certificate and replay checks (C). The source contains 1 explicit `assert` statement and 2 syntactic `.require()` call sites. These are source counts, not dynamic proof counts. The installed-wheel run passed 1 output/artifact predicate with exit code zero and 244 package modules checked for import origin.

Observed base-certificate identifiers include `PSC-III-T58`, `PSC-VI-T28`.

Failure boundary (F). A singular resolvent or a non-Hermitian input outside the constructor contract must not receive the same certificate.

Source API routes. `komorebi_math.dynamics.matrix_exponential`, `komorebi_math.quantum.KatoTransport`, `komorebi_math.quantum.MovingPacketSystem`, `komorebi_math.quantum.QuantumFeshbachReduction`.

Reproducibility (R). `python examples/kato_feshbach.py`. Reference wheel, NumPy, and the logged validation environment with SciPy, SymPy and mpmath. Direct imports are listed separately; this profile is not a proof of minimal transitive dependencies. No random input in the shipped fixture.

Runtime and filesystem. Recorded Linux replay: 1.306 seconds, measured under the shared reference profile; this is not a performance promise. No persistent output files are declared by the example. Replay stdout/stderr and metadata are retained in the selected evidence directory.

Related package tests. `tests/test_block4_hardening.py`, `tests/test_nonlinear_transport.py`, `tests/test_quantum_transport_effective_spectral.py` and 1 more file in the shared-import crosswalk. These files were included in the separate 1,203-test source run. Shared imports alone do not prove full behavioral coverage.

Source SHA-256. `a7bf14e656dab624dcee56b49bf5238a79e054d5ae16572e8c1a8f934cb55576`. Full source and replay hashes are also stored in the JSON ledger and per-record result files.

Observed output excerpt. Selected fields or wrapped lines only. Full stdout is preserved in the replay directory.

```
Kato and Feshbach example passed
```

EX-E-027. Dyadic and odd-prime Euler truncations

Source and placement. `komorebi-math/examples/local_euler_coordinates.py`. Chapters 17 and 25, local Euler coordinates. Local claim: `EX-E-027-CONTRACT`.

Input (I). Euler index 20 and depth 2 for primes 2 and 5.

Hypotheses and evidence class (H). EXACT rational / prime-power congruence. The source fixture fixes all remaining literals and dimensions. The first observed certificate for each identifier, including checked and unchecked assumptions, is retained in the execution record.

Actions (A). Compute each local truncation, require its certificate and print exact, truncated, modulus and error fields.

Expected result (O). The exact Euler value is 370371188237525 for both primes. The certified moduli are 2048 and 15625; the truncations are p-adic approximations, not small real errors.

Certificate and replay checks (C). The source contains 0 explicit `assert` statements and 1 syntactic `.require()` call site. These are source counts, not dynamic proof counts. The installed-wheel run passed 5 output/artifact predicates with exit code zero and 244 package modules checked for import origin.

Observed base-certificate identifiers include `PSC-IX-T19`.

Failure boundary (F). Do not interpret a large real error as failed p-adic accuracy. Prime/compression hypotheses still apply.

Source API routes. `komorebi_math.arithmetic.EulerLocalCoordinates`.

Reproducibility (R). `python examples/local_euler_coordinates.py`. Reference wheel, NumPy, and the logged validation environment with SciPy, SymPy and mpmath. Direct imports are listed separately; this profile is not a proof of minimal transitive dependencies. No random input in the shipped fixture.

Runtime and filesystem. Recorded Linux replay: 0.815 seconds, measured under the shared reference profile; this is not a performance promise. No persistent output files are declared by the example. Replay stdout/stderr and metadata are retained in the selected evidence directory.

Related package tests. `tests/test_arithmetic_euler.py`. These files were included in the separate 1,203-test source run. Shared imports alone do not prove full behavioral coverage.

Source SHA-256. `89ae0fd16f1960e0ae167bb68024f3634ce5dfacaad81203cd12878034811979`. Full source and replay hashes are also stored in the JSON ledger and per-record result files.

Observed output excerpt. Selected fields or wrapped lines only. Full stdout is preserved in the replay directory.

```
{'prime': 2, 'exact': Fraction(370371188237525, 1), 'truncated':
'8804179016755/231', 'modulus': 2048, 'error': '85546940303851520/231'}
{'prime': 5, 'exact': Fraction(370371188237525, 1), 'truncated':
'86777696087933558237525', 'modulus': 15625, 'error':
'-8677769571756237000000'}
```

EX-E-028. Low-rank structured solve and recompression

Source and placement. `komorebi-math/examples/low_rank_recompression.py`. Chapter 29, low-rank recompression. Local claim: `EX-E-028-CONTRACT`.

Input (I). NumPy generator seed 7; diagonal blocks 2I2 and 3I2; two small 4-by-2 factors and a generated right-hand side.

Hypotheses and evidence class (H). NUMERICAL structured algebra. The source fixture fixes all remaining literals and dimensions. The first observed certificate for each identifier, including checked and unchecked assumptions, is retained in the execution record.

Actions (A). Apply and invert the structured operator, form an exact structured product and recompress it to rank 2.

Expected result (O). Solve residual is below 1e-10, product rank bound 4, compressed rank 2; spectral compression error is finite and nonzero for this fixture.

Certificate and replay checks (C). The source contains 0 explicit `assert` statements and 1 syntactic `.require()` call site. These are source counts, not dynamic proof counts. The installed-wheel run passed 4 output/artifact predicates with exit code zero and 244 package modules checked for import origin.

Observed base-certificate identifiers include `PSC-VIII-T12`.

Failure boundary (F). Recompression is an approximation. Rank reduction does not imply zero spectral error or a universal speedup.

Source API routes. `komorebi_math.algorithms.BlockDiagonalLowRankOperator`, `komorebi_math.algorithms.BlockDiagonalOperator`.

Reproducibility (R). `python examples/low_rank_recompression.py`. Reference wheel, NumPy, and the logged validation environment with SciPy, SymPy and mpmath. Direct imports are listed separately; this profile is not a proof of minimal transitive dependencies. NumPy default_rng(7).

Runtime and filesystem. Recorded Linux replay: 0.870 seconds, measured under the shared reference profile; this is not a performance promise. No persistent output files are declared by the example. Replay stdout/stderr and metadata are retained in the selected evidence directory.

Related package tests. `tests/test_algorithms_structured.py`, `tests/test_algorithms_low_rank.py`. These files were included in the separate 1,203-test source run. Shared imports alone do not prove full behavioral coverage.

Source SHA-256. `83fb6064816002498a5eeab4e7b39af492d6e16f0b2c840af6a2a2672fb4fdd8`. Full source and replay hashes are also stored in the JSON ledger and per-record result files.

Observed output excerpt. Selected fields or wrapped lines only. Full stdout is preserved in the replay directory.

```
solve residual 0.0
product rank bound 4
compressed rank 2
compression spectral error 0.00024475675260188207
```

EX-E-029. Finite-mask carry and rational-mask Mahler compilation

Source and placement. `komorebi-math/examples/mahler_cartier_compilers.py`. Chapters 23, 25, and 26, Mahler-Cartier execution. Local claim: `EX-E-029-CONTRACT`.

Input (I). Mask (1,1,1), radix 2; A=(1,1), B=(1,-1), prime 5; coefficient index 10^18+12345; degree-4 coboundary problem.

Hypotheses and evidence class (H). EXACT finite-state / modular arithmetic. The source fixture fixes all remaining literals and dimensions. The first observed certificate for each identifier, including checked and unchecked assumptions, is retained in the execution record.

Actions (A). Certify 500 Stern-shift coefficients, evaluate a bounded-dimensional Cartier state and solve the exact polynomial coboundary.

Expected result (O). Cartier dimension 2, large-index residue 0 and coboundary (1,2). The Cartier model solves B(z)F(z)=A(z)F(z^5), F(0)=1, not the ordinary rational series A(z)/B(z).

Certificate and replay checks (C). The source contains 1 explicit `assert` statement and 2 syntactic `.require()` call sites. These are source counts, not dynamic proof counts. The installed-wheel run passed 3 output/artifact predicates with exit code zero and 244 package modules checked for import origin.

Observed base-certificate identifiers include `GF-FINITE-MASK-CARRY`.

Failure boundary (F). Conflating a rational mask with a rational generating function would give the wrong coefficient expectation. Composite primes must be rejected.

Source API routes. `komorebi_math.generating.compile_cartier_linear_representation`, `komorebi_math.generating.compile_finite_mask_carry_plan`, `komorebi_math.generating.solve_polynomial_coboundary`.

Reproducibility (R). `python examples/mahler_cartier_compilers.py`. Reference wheel, NumPy, and the logged validation environment with SciPy, SymPy and mpmath. Direct imports are listed separately; this profile is not a proof of minimal transitive dependencies. No random input in the shipped fixture.

Runtime and filesystem. Recorded Linux replay: 0.973 seconds, measured under the shared reference profile; this is not a performance promise. No persistent output files are declared by the example. Replay stdout/stderr and metadata are retained in the selected evidence directory.

Related package tests. `tests/test_generating_block13.py`, `tests/test_generating_cartier_padic.py`, `tests/test_generating_domains.py`. These files were included in the separate 1,203-test source run. Shared imports alone do not prove full behavioral coverage.

Source SHA-256. `b66b364ec0168205a96288549652ff7d3316b9b264e1fc14d5de853ee4974088`. Full source and replay hashes are also stored in the JSON ledger and per-record result files.

Observed output excerpt. Selected fields or wrapped lines only. Full stdout is preserved in the replay directory.

```
{'stern_shift_prefix': (Fraction(1, 1), Fraction(1, 1), Fraction(2, 1),
Fraction(1, 1), Fraction(3, 1), Fraction(2, 1), Fraction(3, 1), Fraction(1,
1), Fraction(4, 1), Fraction(3, 1), Fraction(5, 1), Fraction(2, 1),
Fraction(5, 1), Fraction(3, 1), Fraction(4, 1), Fraction(1, 1), Fraction(5,
1), Fraction(4, 1), Fraction(7, 1), Fraction(3, 1)), 'cartier_dimension': 2,
 'large_index_value_mod_5': 0, 'coboundary': (Fraction(1, 1), Fraction(2,
1))}
```

EX-E-030. Lumping and Bayesian sufficiency

Source and placement. `komorebi-math/examples/markov_bayes.py`. Chapter 45, Markov and Bayesian dynamics. Local claim: `EX-E-030-CONTRACT`.

Input (I). The shipped 4-by-4 column-conservative generator, partition (0,0,1,1), prior (0.1,0.2,0.3,0.4) and likelihood (2,2,0.5,0.5).

Hypotheses and evidence class (H). NUMERICAL finite stochastic model. The source fixture fixes all remaining literals and dimensions. The first observed certificate for each identifier, including checked and unchecked assumptions, is retained in the execution record.

Actions (A). Require lumpability and coarse-posterior certificates, then inspect fixed sufficiency on the supplied likelihood.

Expected result (O). Coarse generator [[-0.1,0],[0.1,0]], coarse posterior (12/19,7/19) numerically, and fixed sufficiency True.

Certificate and replay checks (C). The source contains 0 explicit `assert` statements and 2 syntactic `.require()` call sites. These are source counts, not dynamic proof counts. The installed-wheel run passed 2 output/artifact predicates with exit code zero and 244 package modules checked for import origin.

Observed base-certificate identifiers include `PSC-VI-T21`, `PSC-VI-T36`.

Failure boundary (F). A likelihood that varies within a block need not be fixed sufficient; the example is not a universal sufficiency claim.

Source API routes. `komorebi_math.dynamics.BayesianPartition`, `komorebi_math.dynamics.MarkovLumping`.

Reproducibility (R). `python examples/markov_bayes.py`. Reference wheel, NumPy, and the logged validation environment with SciPy, SymPy and mpmath. Direct imports are listed separately; this profile is not a proof of minimal transitive dependencies. No random input in the shipped fixture.

Runtime and filesystem. Recorded Linux replay: 0.841 seconds, measured under the shared reference profile; this is not a performance promise. No persistent output files are declared by the example. Replay stdout/stderr and metadata are retained in the selected evidence directory.

Related package tests. `tests/test_dynamics_bayes_martingales.py`, `tests/test_dynamics_semigroups_markov.py`. These files were included in the separate 1,203-test source run. Shared imports alone do not prove full behavioral coverage.

Source SHA-256. `7a4361d727bfbb5bedbc707377777f3475032df09e3ef1c5c8643ccdab64c591`. Full source and replay hashes are also stored in the JSON ledger and per-record result files.

Observed output excerpt. Selected fields or wrapped lines only. Full stdout is preserved in the replay directory.

```
coarse generator:
[[-0.1  0. ]
 [ 0.1  0. ]]
coarse posterior: [0.63157895 0.36842105]
fixed sufficient: True
```

EX-E-031. Faà di Bruno and sparse packet support

Source and placement. `komorebi-math/examples/nonlinear_jets_composition.py`. Chapter 38, nonlinear jet composition. Local claim: `EX-E-031-CONTRACT`.

Input (I). Scalar outer quadratic and inner x+x^2 at x=2 with directions 3,4; then a two-output diagonal quadratic tensor.

Hypotheses and evidence class (H). EXACT scalar example / NUMERICAL packet example. The source fixture fixes all remaining literals and dimensions. The first observed certificate for each identifier, including checked and unchecked assumptions, is retained in the execution record.

Actions (A). Require the composed directional-derivative certificate and reconstruct the two-dimensional packet-jet family from its support.

Expected result (O). The scalar second directional derivative is 888 with two set-partition terms; only the x/xx and y/yy packet jets are nonzero.

Certificate and replay checks (C). The source contains 0 explicit `assert` statements and 3 syntactic `.require()` call sites. These are source counts, not dynamic proof counts. The installed-wheel run passed 4 output/artifact predicates with exit code zero and 244 package modules checked for import origin.

Observed base-certificate identifiers include `PSC-I-T20`, `PSC-III-C08`, `PSC-III-T07`, `PSC-III-T10`.

Failure boundary (F). Wrong derivative order, direction count or dimension must fail rather than fabricate missing derivatives.

Source API routes. `komorebi_math.Exactness`, `komorebi_math.PacketSystem`, `komorebi_math.nonlinear.PacketJetFamily`, `komorebi_math.nonlinear.PolynomialDerivativeProvider`, `komorebi_math.nonlinear.faa_di_bruno` and 1 more route in the JSON ledger.

Reproducibility (R). `python examples/nonlinear_jets_composition.py`. Reference wheel, NumPy, and the logged validation environment with SciPy, SymPy and mpmath. Direct imports are listed separately; this profile is not a proof of minimal transitive dependencies. No random input in the shipped fixture.

Runtime and filesystem. Recorded Linux replay: 0.884 seconds, measured under the shared reference profile; this is not a performance promise. No persistent output files are declared by the example. Replay stdout/stderr and metadata are retained in the selected evidence directory.

Related package tests. `tests/test_nonlinear_correction_elimination.py`, `tests/test_block10_sparse_autodiff.py`, `tests/test_operators_transfer.py` and 19 more files in the shared-import crosswalk. These files were included in the separate 1,203-test source run. Shared imports alone do not prove full behavioral coverage.

Source SHA-256. `9d5465383605a4ef8878f6ba72c0fc9b77c6fe3f46e47623fa001c88db382655`. Full source and replay hashes are also stored in the JSON ledger and per-record result files.

Observed output excerpt. Selected fields or wrapped lines only. Full stdout is preserved in the replay directory.

```
second directional derivative: 888
set-partition terms: 2
nonzero packet jets: (PacketJetAddress(output='x', inputs=('x', 'x')),
PacketJetAddress(output='y', inputs=('y', 'y')))
```

EX-E-032. Tree expansion, Schur elimination and obstruction

Source and placement. `komorebi-math/examples/nonlinear_trees_elimination.py`. Chapters 39 and 41, tree elimination. Local claim: `EX-E-032-CONTRACT`.

Input (I). Fixed-point seed 0.1 with quadratic nonlinearity, depth 4 and radius 0.2; retained map 3x+2y, hidden map y-4x; singular map (y,x^2).

Hypotheses and evidence class (H). NUMERICAL finite expansion / stated majorant bound. The source fixture fixes all remaining literals and dimensions. The first observed certificate for each identifier, including checked and unchecked assumptions, is retained in the execution record.

Actions (A). Require tree and majorant certificates, compute implicit/reduced derivatives, and solve the Lyapunov-Schmidt range equation at x=0.3.

Expected result (O). Finite tree sum 0.11256641, implicit derivative 4, reduced derivative 11 and reduced obstruction 0.09. The nonzero obstruction is not a solution.

Certificate and replay checks (C). The source contains 0 explicit `assert` statements and 4 syntactic `.require()` call sites. These are source counts, not dynamic proof counts. The installed-wheel run passed 4 output/artifact predicates with exit code zero and 244 package modules checked for import origin.

Observed base-certificate identifiers include `PSC-III-T14`, `PSC-III-T14A`, `PSC-III-T33`, `PSC-III-T41`.

Failure boundary (F). A nonzero reduced obstruction must not be reported as a root of the full nonlinear system.

Source API routes. `komorebi_math.nonlinear.CallableDerivativeProvider`, `komorebi_math.nonlinear.LyapunovSchmidtReduction`, `komorebi_math.nonlinear.NonlinearSchurElimination`, `komorebi_math.nonlinear.PolynomialFixedPointProblem`, `komorebi_math.nonlinear.majorant_convergence`.

Reproducibility (R). `python examples/nonlinear_trees_elimination.py`. Reference wheel, NumPy, and the logged validation environment with SciPy, SymPy and mpmath. Direct imports are listed separately; this profile is not a proof of minimal transitive dependencies. No random input in the shipped fixture.

Runtime and filesystem. Recorded Linux replay: 0.854 seconds, measured under the shared reference profile; this is not a performance promise. No persistent output files are declared by the example. Replay stdout/stderr and metadata are retained in the selected evidence directory.

Source SHA-256. `2aaf93d61b99676bc6725ff4adfc2bbc066f9ce9bf07c9e817a7e56a1e771cd5`. Full source and replay hashes are also stored in the JSON ledger and per-record result files.

Observed output excerpt. Selected fields or wrapped lines only. Full stdout is preserved in the replay directory.

```
finite tree sum: 0.11256641000000002
Picard tail bound at depth 4: 0.004266666666666668
implicit derivative: [[4.-0.j]]
reduced derivative: [[11.+0.j]]
reduced obstruction: [0.09]
```

EX-E-033. Cokernel obstruction and prime-power branches

Source and placement. `komorebi-math/examples/obstruction_lifting.py`. Chapter 40, obstruction lifting. Local claim: `EX-E-033-CONTRACT`.

Input (I). Exact matrix diag(1,0), target (0,1); polynomial x^2-1 over prime 3, roots 1 and 2 mod 3, depths 1 through 5.

Hypotheses and evidence class (H). EXACT finite prime-power algebra. The source fixture fixes all remaining literals and dimensions. The first observed certificate for each identifier, including checked and unchecked assumptions, is retained in the execution record.

Actions (A). Solve the linear obstruction and construct the finite lift tree from the two roots.

Expected result (O). The cokernel obstruction is nonzero with dimension 1; there are five levels and two terminal branches of length five.

Certificate and replay checks (C). The source contains 4 explicit `assert` statements and 0 syntactic `.require()` call sites. These are source counts, not dynamic proof counts. The installed-wheel run passed 3 output/artifact predicates with exit code zero and 245 package modules checked for import origin.

Observed base-certificate identifiers include `PSC-III-T28`.

Failure boundary (F). Nonvanishing obstruction is intentional. Five verified depths alone are not an unqualified infinite lifting statement.

Source API routes. `komorebi_math.LinearObstructionProblem`, `komorebi_math.PolynomialLiftProblem`, `komorebi_math.nonlinear.build_lift_tree`.

Reproducibility (R). `python examples/obstruction_lifting.py`. Reference wheel, NumPy, and the logged validation environment with SciPy, SymPy and mpmath. SymPy is imported directly and is required for this fixture. No random input in the shipped fixture.

Runtime and filesystem. Recorded Linux replay: 1.440 seconds, measured under the shared reference profile; this is not a performance promise. No persistent output files are declared by the example. Replay stdout/stderr and metadata are retained in the selected evidence directory.

Related package tests. `tests/test_lifting.py`. These files were included in the separate 1,203-test source run. Shared imports alone do not prove full behavioral coverage.

Source SHA-256. `123e7c4c7bc12fef7295c39c68d5c96f10bf37fee9c6920811e66a30cd9d1316`. Full source and replay hashes are also stored in the JSON ledger and per-record result files.

Observed output excerpt. Selected fields or wrapped lines only. Full stdout is preserved in the replay directory.

```
obstruction coordinates: (1,)
lift levels: 5
terminal branches: 2
```

EX-E-034. Finite candidate-tree optimization

Source and placement. `komorebi-math/examples/packet_compiler.py`. Chapters 28 through 30, packet compilation. Local claim: `EX-E-034-CONTRACT`.

Input (I). The shipped five-node tree with keep costs/errors, split overheads 0.5 and 0.2, Lagrangian penalty 4, error budget 6 and error unit 0.1.

Hypotheses and evidence class (H). FINITE combinatorial optimization / floating input costs. The source fixture fixes all remaining literals and dimensions. The first observed certificate for each identifier, including checked and unchecked assumptions, is retained in the execution record.

Actions (A). Compute Lagrangian and budget plans and enumerate the finite Pareto frontier.

Expected result (O). Both selected leaves are (left,right); the frontier has the root plan and two split plans. The budget is the declared quantized error budget, not a cost cap of six.

Certificate and replay checks (C). The source contains 0 explicit `assert` statements and 0 syntactic `.require()` call sites. These are source counts, not dynamic proof counts. The installed-wheel run passed 3 output/artifact predicates with exit code zero and 244 package modules checked for import origin.

Failure boundary (F). An infeasible budget returns no plan. Optimality is relative to the supplied candidate tree, not all possible representations.

Source API routes. `komorebi_math.algorithms.TreeNode`, `komorebi_math.algorithms.compile_tree_budget`, `komorebi_math.algorithms.compile_tree_lagrangian`, `komorebi_math.algorithms.pareto_frontier`.

Reproducibility (R). `python examples/packet_compiler.py`. Reference wheel, NumPy, and the logged validation environment with SciPy, SymPy and mpmath. Direct imports are listed separately; this profile is not a proof of minimal transitive dependencies. No random input in the shipped fixture.

Runtime and filesystem. Recorded Linux replay: 0.925 seconds, measured under the shared reference profile; this is not a performance promise. No persistent output files are declared by the example. Replay stdout/stderr and metadata are retained in the selected evidence directory.

Related package tests. `tests/test_algorithms_compiler.py`. These files were included in the separate 1,203-test source run. Shared imports alone do not prove full behavioral coverage.

Source SHA-256. `a6509e6230b080d88a75c22c7b37d54f102de1a9d54b57711899bdf001134511`. Full source and replay hashes are also stored in the JSON ledger and per-record result files.

Observed output excerpt. Selected fields or wrapped lines only. Full stdout is preserved in the replay directory.

```
Lagrangian leaves ('left', 'right')
budget leaves ('left', 'right')
Pareto points [(9.0, 0.0, ('root',)), (7.5, 0.30000000000000004, ('left',
'right')), (5.2, 0.8999999999999999, ('left', 'right_low', 'right_high'))]
```

EX-E-035. Structured packet intermediate representation

Source and placement. `komorebi-math/examples/packet_ir.py`. Chapter 28, packet intermediate representation. Local claim: `EX-E-035-CONTRACT`.

Input (I). Layout (2,1), matrix [[2,1,0],[0,3,0],[4,0,5]], vector (1,2,3).

Hypotheses and evidence class (H). NUMERICAL apply / EXACT layout counts. The source fixture fixes all remaining literals and dimensions. The first observed certificate for each identifier, including checked and unchecked assumptions, is retained in the execution record.

Actions (A). Create a PacketIR, require its certificate, apply it and evaluate the declared cost model.

Expected result (O). Result (4,6,19), support {(0,0),(1,0),(1,1)}, seven stored entries and fourteen modeled flops.

Certificate and replay checks (C). The source contains 0 explicit `assert` statements and 1 syntactic `.require()` call site. These are source counts, not dynamic proof counts. The installed-wheel run passed 4 output/artifact predicates with exit code zero and 244 package modules checked for import origin.

Observed base-certificate identifiers include `PSC-VIII-T01`.

Failure boundary (F). Modeled flops are not measured runtime. An incompatible vector or layout must be rejected.

Source API routes. `komorebi_math.algorithms.PacketIR`, `komorebi_math.algorithms.PacketLayout`, `komorebi_math.algorithms.packet_cost_estimate`.

Reproducibility (R). `python examples/packet_ir.py`. Reference wheel, NumPy, and the logged validation environment with SciPy, SymPy and mpmath. Direct imports are listed separately; this profile is not a proof of minimal transitive dependencies. No random input in the shipped fixture.

Runtime and filesystem. Recorded Linux replay: 0.963 seconds, measured under the shared reference profile; this is not a performance promise. No persistent output files are declared by the example. Replay stdout/stderr and metadata are retained in the selected evidence directory.

Related package tests. `tests/test_algorithms_complexity_benchmarks.py`, `tests/test_algorithms_discovery_gpu.py`, `tests/test_algorithms_krylov.py` and 1 more file in the shared-import crosswalk. These files were included in the separate 1,203-test source run. Shared imports alone do not prove full behavioral coverage.

Source SHA-256. `977ea6b7e2944b44fb12e46529d74959cbc21b7570fba73ff5783a50738718bf`. Full source and replay hashes are also stored in the JSON ledger and per-record result files.

Observed output excerpt. Selected fields or wrapped lines only. Full stdout is preserved in the replay directory.

```
support [(0, 0), (1, 0), (1, 1)]
result [ 4.  6. 19.]
stored entries 7
modeled flops 14
```

EX-E-036. Polyphase and Haar reconstruction

Source and placement. `komorebi-math/examples/polyphase_filterbank.py`. Chapters 14 and 35, polyphase filter banks. Local claim: `EX-E-036-CONTRACT`.

Input (I). Signal arange(16), modulus 4, FIR taps (1/4,1/2,1/4), Haar depth 3 and pruning leaves (0,10,110,111).

Hypotheses and evidence class (H). EXACT indexing / NUMERICAL transforms. The source fixture fixes all remaining literals and dimensions. The first observed certificate for each identifier, including checked and unchecked assumptions, is retained in the execution record.

Actions (A). Require polyphase/convolution certificates and verify Haar and pruned-tree reconstruction.

Expected result (O). Four channels of length four, filtered length 18 and fifteen packet-tree nodes; both reconstruction assertions pass.

Certificate and replay checks (C). The source contains 2 explicit `assert` statements and 3 syntactic `.require()` call sites. These are source counts, not dynamic proof counts. The installed-wheel run passed 3 output/artifact predicates with exit code zero and 244 package modules checked for import origin.

Observed base-certificate identifiers include `PSC-V-T02`, `PSC-V-T07`, `PSC-V-T13`.

Failure boundary (F). An invalid pruning cover must not masquerade as a complete reconstruction.

Source API routes. `komorebi_math.signals.HaarPacketTree`, `komorebi_math.signals.haar_analyze`, `komorebi_math.signals.haar_synthesize`, `komorebi_math.signals.polyphase_analyze`, `komorebi_math.signals.polyphase_convolution`.

Reproducibility (R). `python examples/polyphase_filterbank.py`. Reference wheel, NumPy, and the logged validation environment with SciPy, SymPy and mpmath. Direct imports are listed separately; this profile is not a proof of minimal transitive dependencies. No random input in the shipped fixture.

Runtime and filesystem. Recorded Linux replay: 0.842 seconds, measured under the shared reference profile; this is not a performance promise. No persistent output files are declared by the example. Replay stdout/stderr and metadata are retained in the selected evidence directory.

Related package tests. `tests/test_signals_polyphase_filterbanks.py`, `tests/test_block4_hardening.py`, `tests/test_block15_polyphase_refinement.py` and 2 more files in the shared-import crosswalk. These files were included in the separate 1,203-test source run. Shared imports alone do not prove full behavioral coverage.

Source SHA-256. `70c0560838804d81cfc22326f2b4ef37d611aba24d57c4a3709197b860e050ab`. Full source and replay hashes are also stored in the JSON ledger and per-record result files.

Observed output excerpt. Selected fields or wrapped lines only. Full stdout is preserved in the replay directory.

```
polyphase channels: (4, 4, 4, 4)
filtered length: 18
packet tree nodes: 15
```

EX-E-037. Portable jet snapshot and tamper rejection

Source and placement. `komorebi-math/examples/portable_nonlinear_state.py`. Chapters 43 and 61, portable nonlinear state. Local claim: `EX-E-037-CONTRACT`.

Input (I). Diagonal order-2 tensor entries 3,-2; x/xx jet; directions (2,5),(-1,4).

Hypotheses and evidence class (H). NUMERICAL snapshot / INTEGRITY check. The source fixture fixes all remaining literals and dimensions. The first observed certificate for each identifier, including checked and unchecked assumptions, is retained in the execution record.

Actions (A). Capture and serialize a jet, restore it, require a comparison certificate, then replace the payload digest with 64 zeroes.

Expected result (O). One stored coefficient; evaluation (-6,0); the tampered payload is rejected with ValidationError.

Certificate and replay checks (C). The source contains 0 explicit `assert` statements and 2 syntactic `.require()` call sites. These are source counts, not dynamic proof counts. The installed-wheel run passed 3 output/artifact predicates with exit code zero and 245 package modules checked for import origin.

Observed base-certificate identifiers include `PSC-I-T20`, `PSC-III-D58`.

Failure boundary (F). The shipped digest replacement is an executed negative control. Digest integrity does not establish callback semantics outside the captured finite jet.

Source API routes. `komorebi_math.PacketSystem`, `komorebi_math.exceptions.ValidationError`, `komorebi_math.export.dumps_portable`, `komorebi_math.export.loads_portable`, `komorebi_math.export.portable_digest` and 3 more routes in the JSON ledger.

Reproducibility (R). `python examples/portable_nonlinear_state.py`. Reference wheel, NumPy, and the logged validation environment with SciPy, SymPy and mpmath. Direct imports are listed separately; this profile is not a proof of minimal transitive dependencies. No random input in the shipped fixture.

Runtime and filesystem. Recorded Linux replay: 0.806 seconds, measured under the shared reference profile; this is not a performance promise. No persistent output files are declared by the example. Replay stdout/stderr and metadata are retained in the selected evidence directory.

Related package tests. `tests/test_generating_block13.py`, `tests/test_certificates_registry.py`, `tests/test_algorithms_complexity_benchmarks.py` and 71 more file in the shared-import crosswalk. These files were included in the separate 1,203-test source run. Shared imports alone do not prove full behavioral coverage.

Source SHA-256. `72aaebad93767495ffcb5bf2f22f97de2222e5a81b2daa0918fa56160bf96469`. Full source and replay hashes are also stored in the JSON ledger and per-record result files.

Observed output excerpt. Selected fields or wrapped lines only. Full stdout is preserved in the replay directory.

```
snapshot address: PacketJetAddress(output='x', inputs=('x', 'x'))
stored coefficients: 1
portable digest:
dae6206f9b99acfa72649825253fd0b748c1e62236109f0a120872767ad97033
evaluation: [(-6+0j), 0j]
tampered payload: rejected
```

EX-E-038. Discrete precision and task scheduling

Source and placement. `komorebi-math/examples/precision_schedule.py`. Chapter 30, precision scheduling. Local claim: `EX-E-038-CONTRACT`.

Input (I). Two two-choice dtype groups with costs 1 or 3 and quantized errors (3,0),(2,0); budget 3; analyze/low/high/synthesize task DAG.

Hypotheses and evidence class (H). EXACT finite model / supplied duration assumptions. The source fixture fixes all remaining literals and dimensions. The first observed certificate for each identifier, including checked and unchecked assumptions, is retained in the execution record.

Actions (A). Require the chosen-precision certificate, then compute an unlimited-resource schedule for the fixed dependencies and durations.

Expected result (O). Choose (low64,high32), cost 4, makespan 5 and critical path (analyze,high,synthesize).

Certificate and replay checks (C). The source contains 0 explicit `assert` statements and 1 syntactic `.require()` call site. These are source counts, not dynamic proof counts. The installed-wheel run passed 4 output/artifact predicates with exit code zero and 244 package modules checked for import origin.

Observed base-certificate identifiers include `PSC-VIII-T28`.

Failure boundary (F). A cyclic dependency graph or infeasible precision budget is not a valid schedule. This is not a measured parallel speedup.

Source API routes. `komorebi_math.algorithms.DTypeChoice`, `komorebi_math.algorithms.Task`, `komorebi_math.algorithms.assign_discrete_precisions`, `komorebi_math.algorithms.schedule_unlimited`.

Reproducibility (R). `python examples/precision_schedule.py`. Reference wheel, NumPy, and the logged validation environment with SciPy, SymPy and mpmath. Direct imports are listed separately; this profile is not a proof of minimal transitive dependencies. No random input in the shipped fixture.

Runtime and filesystem. Recorded Linux replay: 0.808 seconds, measured under the shared reference profile; this is not a performance promise. No persistent output files are declared by the example. Replay stdout/stderr and metadata are retained in the selected evidence directory.

Related package tests. `tests/test_algorithms_precision.py`, `tests/test_algorithms_scheduling.py`. These files were included in the separate 1,203-test source run. Shared imports alone do not prove full behavioral coverage.

Source SHA-256. `fb4ee131cb7b5081a0984974d4d51010824312bfbafbd2e52f36da44a5f6e27a`. Full source and replay hashes are also stored in the JSON ledger and per-record result files.

Observed output excerpt. Selected fields or wrapped lines only. Full stdout is preserved in the replay directory.

```
precision choices ('low64', 'high32')
total cost 4.0
schedule makespan 5.0
critical path ('analyze', 'high', 'synthesize')
```

EX-E-039. Prime-power kernels and valuations

Source and placement. `komorebi-math/examples/prime_power_generating.py`. Chapter 24, prime-power generating systems. Local claim: `EX-E-039-CONTRACT`.

Input (I). Prime 3, exponent 5, power 2, binomial indices 137 and 52; Bell input (137,5,9); q-binomial input (48,22,2,5).

Hypotheses and evidence class (H). EXACT integer/modular arithmetic. The source fixture fixes all remaining literals and dimensions. The first observed certificate for each identifier, including checked and unchecked assumptions, is retained in the execution record.

Actions (A). Compare the kernel residue with Python integer comb modulo 3^5, require the Bell reduction and compute the q-valuation.

Expected result (O). Kernel residue 81, valuation 4, Bell digit sum 5 and q-binomial valuation 1.

Certificate and replay checks (C). The source contains 1 explicit `assert` statement and 1 syntactic `.require()` call site. These are source counts, not dynamic proof counts. The installed-wheel run passed 4 output/artifact predicates with exit code zero and 244 package modules checked for import origin.

Failure boundary (F). Invalid primes or negative exponents must be rejected; residues do not identify the full integer value.

Source API routes. `komorebi_math.generating.PrimePowerKernelCompiler`, `komorebi_math.generating.generalized_bell_digit_reduction`, `komorebi_math.generating.q_binomial_valuation_decomposition`.

Reproducibility (R). `python examples/prime_power_generating.py`. Reference wheel, NumPy, and the logged validation environment with SciPy, SymPy and mpmath. Direct imports are listed separately; this profile is not a proof of minimal transitive dependencies. No random input in the shipped fixture.

Runtime and filesystem. Recorded Linux replay: 2.057 seconds, measured under the shared reference profile; this is not a performance promise. No persistent output files are declared by the example. Replay stdout/stderr and metadata are retained in the selected evidence directory.

Related package tests. `tests/test_generating_cartier_padic.py`, `tests/test_generating_domains.py`. These files were included in the separate 1,203-test source run. Shared imports alone do not prove full behavioral coverage.

Source SHA-256. `992082ab5f90e3f77cfc7b899e10844e7fea09cc5519e8a9588c75136b70423b`. Full source and replay hashes are also stored in the JSON ledger and per-record result files.

Observed output excerpt. Selected fields or wrapped lines only. Full stdout is preserved in the replay directory.

```
{'kernel_residue': 81, 'kernel_valuation': 4, 'bell_digit_sum': 5,
'q_binomial_valuation': 1}
```

EX-E-040. Finite DSP pipeline with fixed seed

Source and placement. `komorebi-math/examples/production_dsp.py`. Chapters 31 through 36 and 66, production DSP. Local claim: `EX-E-040-CONTRACT`.

Input (I). Seed 1212; 1024 samples at 1024 Hz with a 73 Hz sinusoid; finite FIR, resampling 5/4, NLMS and wavelet fixtures.

Hypotheses and evidence class (H). NUMERICAL finite DSP / OBSERVATIONAL risk. The source fixture fixes all remaining literals and dimensions. The first observed certificate for each identifier, including checked and unchecked assumptions, is retained in the execution record.

Actions (A). Require STFT, streaming filters, resampling, adaptive and wavelet certificates, then report finite output sizes and an observed denoising comparison.

Expected result (O). Dominant frequency 73 Hz; 1280 resampled values, 1027 convolution values, learned weights approximately (0.7,-0.35,0.15). Denoising improves on this seeded fixture only.

Certificate and replay checks (C). The source contains 0 explicit `assert` statements and 11 syntactic `.require()` call site. These are source counts, not dynamic proof counts. The installed-wheel run passed 12 output/artifact predicates with exit code zero and 244 package modules checked for import origin.

Observed base-certificate identifiers include `KM-DSP-C02`, `KM-DSP-C04`, `KM-DSP-D02`, `KM-DSP-D07`, `KM-DSP-D10` (the complete set is in the execution record).

Failure boundary (F). Do not equate batch and streaming frame counts with different framing contracts: this fixture prints 33 and 29 respectively.

Source API routes. `komorebi_math.signals.certified_overlap_add_convolve`, `komorebi_math.signals.cross_spectrum`, `komorebi_math.signals.denoising_risk_certificate`, `komorebi_math.signals.design_biquad`, `komorebi_math.signals.design_lowpass` and 10 more routes in the JSON ledger.

Reproducibility (R). `python examples/production_dsp.py`. Reference wheel, NumPy, and the logged validation environment with SciPy, SymPy and mpmath. Direct imports are listed separately; this profile is not a proof of minimal transitive dependencies. NumPy default_rng(1212).

Runtime and filesystem. Recorded Linux replay: 0.949 seconds, measured under the shared reference profile; this is not a performance promise. No persistent output files are declared by the example. Replay stdout/stderr and metadata are retained in the selected evidence directory.

Related package tests. `tests/test_block12_recursive_fft.py`, `tests/test_block12_windows_time_frequency.py`, `tests/test_block12_denoise_streaming.py` and 2 more files in the shared-import crosswalk. These files were included in the separate 1,203-test source run. Shared imports alone do not prove full behavioral coverage.

Source SHA-256. `4a0928bb7048e62417efbba97f90fd0f4ba78c4fd4018338b23ee0adfb1bf376`. Full source and replay hashes are also stored in the JSON ledger and per-record result files.

Observed output excerpt. Selected fields or wrapped lines only. Full stdout is preserved in the replay directory.


```
{
  "adaptive_final_weights": [
    0.7,
    -0.35,
    0.15000000000000002
  ],
  "denoising_observed_risk_improved": true,
  "dominant_frequency_hz": 73.0,
```


EX-E-041. Amplitude damping and finite error correction

Source and placement. `komorebi-math/examples/quantum_channel_qec.py`. Chapters 54 through 55, channels and error correction. Local claim: `EX-E-041-CONTRACT`.

Input (I). Damping parameter 0.2; excited qubit; the three-qubit repetition-code projector and identity plus three single-X errors.

Hypotheses and evidence class (H). NUMERICAL finite quantum algebra. The source fixture fixes all remaining literals and dimensions. The first observed certificate for each identifier, including checked and unchecked assumptions, is retained in the execution record.

Actions (A). Require the channel certificate, compare the output density diagonal, then require Knill-Laflamme and recovered-channel certificates.

Expected result (O). The excited-state diagonal becomes (0.2,0.8); the declared finite error set is correctable by the supplied recovery.

Certificate and replay checks (C). The source contains 1 explicit `assert` statement and 3 syntactic `.require()` call sites. These are source counts, not dynamic proof counts. The installed-wheel run passed 1 output/artifact predicate with exit code zero and 244 package modules checked for import origin.

Observed base-certificate identifiers include `PSC-VII-T31`, `PSC-VII-T42`, `PSC-VII-T44`.

Failure boundary (F). Correcting this error set does not certify arbitrary errors. Parameters outside the channel domain must fail.

Source API routes. `komorebi_math.quantum.QuantumCode`, `komorebi_math.quantum.amplitude_damping`.

Reproducibility (R). `python examples/quantum_channel_qec.py`. Reference wheel, NumPy, and the logged validation environment with SciPy, SymPy and mpmath. Direct imports are listed separately; this profile is not a proof of minimal transitive dependencies. No random input in the shipped fixture.

Runtime and filesystem. Recorded Linux replay: 0.873 seconds, measured under the shared reference profile; this is not a performance promise. No persistent output files are declared by the example. Replay stdout/stderr and metadata are retained in the selected evidence directory.

Related package tests. `tests/test_quantum_channels_qec.py`. These files were included in the separate 1,203-test source run. Shared imports alone do not prove full behavioral coverage.

Source SHA-256. `c7d4831611607d0e208ba02f1422ee08dfa3100c8a50c64dec975bb13cdc6e6a`. Full source and replay hashes are also stored in the JSON ledger and per-record result files.

Observed output excerpt. Selected fields or wrapped lines only. Full stdout is preserved in the replay directory.

```
quantum channel and error correction example passed
```

EX-E-042. Interfering symmetry-resolved circuit paths

Source and placement. `komorebi-math/examples/quantum_symmetry_circuit.py`. Chapters 53 through 54, symmetry circuits. Local claim: `EX-E-042-CONTRACT`.

Input (I). Z symmetry, cyclic order 2, three Hadamard gates and four matching packet systems; initial state (1,0).

Hypotheses and evidence class (H). NUMERICAL finite quantum circuit. The source fixture fixes all remaining literals and dimensions. The first observed certificate for each identifier, including checked and unchecked assumptions, is retained in the execution record.

Actions (A). Require reconstruction and block-unitarity certificates and evaluate the declared packet interference report.

Expected result (O). Target probability is 1/2 within numerical tolerance. Coherent path amplitudes, not independent path probabilities, are combined.

Certificate and replay checks (C). The source contains 0 explicit `assert` statements and 3 syntactic `.require()` call sites. These are source counts, not dynamic proof counts. The installed-wheel run passed 2 output/artifact predicates with exit code zero and 244 package modules checked for import origin.

Observed base-certificate identifiers include `PSC-I-T20`, `PSC-VII-C17`, `PSC-VII-T14`, `PSC-VII-T15`.

Failure boundary (F). Replacing coherent addition with a probability sum would discard interference and is not the implemented contract.

Source API routes. `komorebi_math.quantum.CircuitPacketExpansion`, `komorebi_math.quantum.cyclic_symmetry_projectors`.

Reproducibility (R). `python examples/quantum_symmetry_circuit.py`. Reference wheel, NumPy, and the logged validation environment with SciPy, SymPy and mpmath. Direct imports are listed separately; this profile is not a proof of minimal transitive dependencies. No random input in the shipped fixture.

Runtime and filesystem. Recorded Linux replay: 0.855 seconds, measured under the shared reference profile; this is not a performance promise. No persistent output files are declared by the example. Replay stdout/stderr and metadata are retained in the selected evidence directory.

Related package tests. `tests/test_quantum_symmetry_circuits.py`, `tests/test_quantum_channels_qec.py`. These files were included in the separate 1,203-test source run. Shared imports alone do not prove full behavioral coverage.

Source SHA-256. `7f5e976cd4255999aace0e6edd359120b57c0d0249b82c17989152404056f8a7`. Full source and replay hashes are also stored in the JSON ledger and per-record result files.

Observed output excerpt. Selected fields or wrapped lines only. Full stdout is preserved in the replay directory.

```
quantum circuit example passed 0.4999999999999998
```

EX-E-043. First packet-resolved operator program

Source and placement. `komorebi-math/examples/quickstart.py`. Chapter 4, first verified program. Local claim: `EX-E-043-CONTRACT`.

Input (I). Four-cycle shift matrix, seed e0, even/odd diagonal projectors and a UnifiedSystem.

Hypotheses and evidence class (H). NUMERICAL finite matrix algebra. The source fixture fixes all remaining literals and dimensions. The first observed certificate for each identifier, including checked and unchecked assumptions, is retained in the execution record.

Actions (A). Construct seed and packet systems, resolve operator blocks and print the system summary and block norms.

Expected result (O). The summary is valid, the seed hull dimension is 4 and the off-diagonal block norms are sqrt(2); diagonal block norms vanish.

Certificate and replay checks (C). The source contains 0 explicit `assert` statements and 0 syntactic `.require()` call sites. These are source counts, not dynamic proof counts. The installed-wheel run passed 2 output/artifact predicates with exit code zero and 244 package modules checked for import origin.

Observed base-certificate identifiers include `PSC-I-T20`.

Failure boundary (F). The seed hull is this finite generated span; it is not a universal claim of minimal generators for arbitrary algebras.

Source API routes. `komorebi_math.PacketOperator`, `komorebi_math.PacketSystem`, `komorebi_math.SeedSystem`, `komorebi_math.UnifiedSystem`.

Reproducibility (R). `python examples/quickstart.py`. Reference wheel, NumPy, and the logged validation environment with SciPy, SymPy and mpmath. Direct imports are listed separately; this profile is not a proof of minimal transitive dependencies. No random input in the shipped fixture.

Runtime and filesystem. Recorded Linux replay: 0.795 seconds, measured under the shared reference profile; this is not a performance promise. No persistent output files are declared by the example. Replay stdout/stderr and metadata are retained in the selected evidence directory.

Related package tests. `tests/test_operators_transfer.py`, `tests/test_nonlinear_trees.py`, `tests/test_morphisms.py` and 16 more files in the shared-import crosswalk. These files were included in the separate 1,203-test source run. Shared imports alone do not prove full behavioral coverage.

Source SHA-256. `7d844580b57e0fccd5a923a78570d7265797c8dcbf39ad5bc31ce81b5f9e844a`. Full source and replay hashes are also stored in the JSON ledger and per-record result files.

Observed output excerpt. Selected fields or wrapped lines only. Full stdout is preserved in the replay directory.

```
{'valid': True, 'exactness': 'numerical', 'grammar_vertices': (),
'seed_dimension': 4, 'packet_names': ('even', 'odd'), 'operators':
('shift',), 'refinement_levels': 0, 'metadata': {}}
hull dimension: 4
block norms: ((0.0, 1.4142135623730951), (1.4142135623730951, 0.0))
```

EX-E-044. Finite ramification with an unmet realization gate

Source and placement. `komorebi-math/examples/ramification_completion.py`. Chapter 58, ramification and completion. Local claim: `EX-E-044-CONTRACT`.

Input (I). Prime-2 alternating form and quadratic refinement, two declared ramification stages, and five Boolean completion gates with realization=False.

Hypotheses and evidence class (H). EXACT finite data / CONDITIONAL external gates. The source fixture fixes all remaining literals and dimensions. The first observed certificate for each identifier, including checked and unchecked assumptions, is retained in the execution record.

Actions (A). Build the ramification report and inspect the conditional completion result.

Expected result (O). Breaks (0,1), dimensions (4,2), Arf invariant 0, completion_ready=False and unmet gate realization.

Certificate and replay checks (C). The source contains 0 explicit `assert` statements and 0 syntactic `.require()` call sites. These are source counts, not dynamic proof counts. The installed-wheel run passed 4 output/artifact predicates with exit code zero and 244 package modules checked for import origin.

Failure boundary (F). A successful script must retain completion_ready=False. The four supplied true gates are assumptions, not newly proven global theorems.

Source API routes. `komorebi_math.langlands.completion.AutomorphicCompletionReport`, `komorebi_math.langlands.completion.CompletionGate`, `komorebi_math.langlands.completion.GateStatus`, `komorebi_math.langlands.local.AlternatingForm`, `komorebi_math.langlands.local.QuadraticRefinement` and 3 more routes in the JSON ledger.

Reproducibility (R). `python examples/ramification_completion.py`. Reference wheel, NumPy, and the logged validation environment with SciPy, SymPy and mpmath. Direct imports are listed separately; this profile is not a proof of minimal transitive dependencies. No random input in the shipped fixture.

Runtime and filesystem. Recorded Linux replay: 0.940 seconds, measured under the shared reference profile; this is not a performance promise. No persistent output files are declared by the example. Replay stdout/stderr and metadata are retained in the selected evidence directory.

Related package tests. `tests/test_langlands_ramification_global_completion.py`, `tests/test_langlands_local_heisenberg_symplectic.py`. These files were included in the separate 1,203-test source run. Shared imports alone do not prove full behavioral coverage.

Source SHA-256. `3d7343488c3c849ddcd61bfb02c531a3a6b07ad0c13ab1d4e989073d0045d279`. Full source and replay hashes are also stored in the JSON ledger and per-record result files.

Observed output excerpt. Selected fields or wrapped lines only. Full stdout is preserved in the replay directory.

```
{'ramification_breaks': (0, 1), 'ramification_dimensions': (4, 2),
'arf_invariant': 0, 'completion_ready': False, 'unmet_gates':
('realization',)}
```

EX-E-045. Recurrence lanes and modular large-index terms

Source and placement. `komorebi-math/examples/recurrence_packets.py`. Chapters 18 and 26, recurrence packets. Local claim: `EX-E-045-CONTRACT`.

Input (I). Fibonacci recurrence, modulus-7 lane at address 3, twelve lane terms; index 1,000,000 modulo 1,000,003; period modulo 32.

Hypotheses and evidence class (H). EXACT integer/modular recurrence. The source fixture fixes all remaining literals and dimensions. The first observed certificate for each identifier, including checked and unchecked assumptions, is retained in the execution record.

Actions (A). Require the finite lane certificate, then evaluate a modular term and finite period.

Expected result (O). The lane begins 2,55,1597; the large-index residue is 3 and the period modulo 32 is 48.

Certificate and replay checks (C). The source contains 0 explicit `assert` statements and 1 syntactic `.require()` call site. These are source counts, not dynamic proof counts. The installed-wheel run passed 3 output/artifact predicates with exit code zero and 244 package modules checked for import origin.

Observed base-certificate identifiers include `PSC-IX-T43`.

Failure boundary (F). The lane modulus and the arithmetic reduction modulus have different roles and must not be interchanged.

Source API routes. `komorebi_math.arithmetic.fibonacci_recurrence`.

Reproducibility (R). `python examples/recurrence_packets.py`. Reference wheel, NumPy, and the logged validation environment with SciPy, SymPy and mpmath. Direct imports are listed separately; this profile is not a proof of minimal transitive dependencies. No random input in the shipped fixture.

Runtime and filesystem. Recorded Linux replay: 0.878 seconds, measured under the shared reference profile; this is not a performance promise. No persistent output files are declared by the example. Replay stdout/stderr and metadata are retained in the selected evidence directory.

Related package tests. `tests/test_arithmetic_recurrence.py`. These files were included in the separate 1,203-test source run. Shared imports alone do not prove full behavioral coverage.

Source SHA-256. `6c4852b544e9aabb96d69e94125e593d7a4b662493593f4f0129bb1ef062a344`. Full source and replay hashes are also stored in the JSON ledger and per-record result files.

Observed output excerpt. Selected fields or wrapped lines only. Full stdout is preserved in the replay directory.

```
packet (2, 55, 1597, 46368, 1346269, 39088169, 1134903170, 32951280099,
956722026041, 27777890035288, 806515533049393, 23416728348467685)
F(1_000_000) mod 1_000_003 3
period mod 32 48
```

EX-E-046. Recurrence bridge and state observability

Source and placement. `komorebi-math/examples/recurrence_state_coordinate_atlas.py`. Chapter 18, recurrence state coordinates. Local claim: `EX-E-046-CONTRACT`.

Input (I). Sixteen Pell values mapped to Tribonacci defect coordinates; a third-order SISO system with the shipped exact A,B,C,D and state (2,-3,5).

Hypotheses and evidence class (H). EXACT finite rational state-space algebra. The source fixture fixes all remaining literals and dimensions. The first observed certificate for each identifier, including checked and unchecked assumptions, is retained in the execution record.

Actions (A). Check the sequence-coordinate bridge, require controllability-coordinate equivalence and invert the observability map.

Expected result (O). The bridge preserves finite coordinates and the observable state is (-9,-15,-6).

Certificate and replay checks (C). The source contains 2 explicit `assert` statements and 1 syntactic `.require()` call site. These are source counts, not dynamic proof counts. The installed-wheel run passed 2 output/artifact predicates with exit code zero and 244 package modules checked for import origin.

Observed base-certificate identifiers include `CC.R3`.

Failure boundary (F). Finite defect coordinates do not assert that the Pell sequence obeys the homogeneous Tribonacci recurrence.

Source API routes. `komorebi_math.coordinates.certify_state_coordinate_change`, `komorebi_math.coordinates.controllability_coordinate_transform`, `komorebi_math.coordinates.get_recurrence_model`, `komorebi_math.coordinates.observability_coordinate_transform`, `komorebi_math.coordinates.sequence_coordinates_between_models` and 1 more route in the JSON ledger.

Reproducibility (R). `python examples/recurrence_state_coordinate_atlas.py`. Reference wheel, NumPy, and the logged validation environment with SciPy, SymPy and mpmath. Direct imports are listed separately; this profile is not a proof of minimal transitive dependencies. No random input in the shipped fixture.

Runtime and filesystem. Recorded Linux replay: 0.919 seconds, measured under the shared reference profile; this is not a performance promise. No persistent output files are declared by the example. Replay stdout/stderr and metadata are retained in the selected evidence directory.

Source SHA-256. `f54912ebd67030bf03f20630f33de3f9fcedd5cda7135679a0f5b2c37bc1fa5d`. Full source and replay hashes are also stored in the JSON ledger and per-record result files.

Observed output excerpt. Selected fields or wrapped lines only. Full stdout is preserved in the replay directory.

```
recurrence bridge: Pell recurrence coordinates of length 16 -> Tribonacci
recurrence coordinates of length 16
observable state coordinates: (Fraction(-9, 1), Fraction(-15, 1),
Fraction(-6, 1))
```

EX-E-047. Packet refinement, CRT and weighted transfer

Source and placement. `komorebi-math/examples/refinement_pipeline.py`. Chapters 12 through 14, refinement pipeline. Local claim: `EX-E-047-CONTRACT`.

Input (I). Two coarse and four fine coordinate projectors, vector (1,2,3,4), residue modulus 3 and CRT pairs (1 mod 2,2 mod 3,4 mod 5).

Hypotheses and evidence class (H). EXACT indexing/weights / NUMERICAL inversion. The source fixture fixes all remaining literals and dimensions. The first observed certificate for each identifier, including checked and unchecked assumptions, is retained in the execution record.

Actions (A). Refine/coarsen, reconstruct residue data, solve CRT and verify the acyclic weighted-transfer inverse against a dense inverse.

Expected result (O). Two refinement levels, CRT residue 29 and minimum path weight 3 from packet 0 to 2.

Certificate and replay checks (C). The source contains 5 explicit `assert` statements and 1 syntactic `.require()` call site. These are source counts, not dynamic proof counts. The installed-wheel run passed 3 output/artifact predicates with exit code zero and 244 package modules checked for import origin.

Observed base-certificate identifiers include `PSC-I-T20`, `PSC-II-C11`, `PSC-II-T03`.

Failure boundary (F). Cyclic transfer graphs do not automatically admit the same finite acyclic inverse formula.

Source API routes. `komorebi_math.PacketRefinement`, `komorebi_math.PacketSystem`, `komorebi_math.WeightedTransferEdge`, `komorebi_math.WeightedTransferGraph`, `komorebi_math.refinement.RefinementTower` and 2 more routes in the JSON ledger.

Reproducibility (R). `python examples/refinement_pipeline.py`. Reference wheel, NumPy, and the logged validation environment with SciPy, SymPy and mpmath. Direct imports are listed separately; this profile is not a proof of minimal transitive dependencies. No random input in the shipped fixture.

Runtime and filesystem. Recorded Linux replay: 0.831 seconds, measured under the shared reference profile; this is not a performance promise. No persistent output files are declared by the example. Replay stdout/stderr and metadata are retained in the selected evidence directory.

Related package tests. `tests/test_operators_transfer.py`, `tests/test_nonlinear_trees.py`, `tests/test_morphisms.py` and 18 more files in the shared-import crosswalk. These files were included in the separate 1,203-test source run. Shared imports alone do not prove full behavioral coverage.

Source SHA-256. `122868a33c99ee2b334f141e876540d07ff40b71354f649e16523a77ea1115ce`. Full source and replay hashes are also stored in the JSON ledger and per-record result files.

Observed output excerpt. Selected fields or wrapped lines only. Full stdout is preserved in the replay directory.

```
refinement levels: 2
CRT address: 29
minimum transfer weight 0 to 2: 3
```

EX-E-048. Pause, resume and verify a finite jet stream

Source and placement. `komorebi-math/examples/resumable_packet_jet_stream.py`. Chapter 43, resumable jet streaming. Local claim: `EX-E-048-CONTRACT`.

Input (I). Order-2 tensor [[[1,2],[3,4]]], full source/target packets, directions (2,3),(-1,4), chunk size 1 and initial evaluation budget 2.

Hypotheses and evidence class (H). NUMERICAL finite stream / INTEGRITY checks. The source fixture fixes all remaining literals and dimensions. The first observed certificate for each identifier, including checked and unchecked assumptions, is retained in the execution record.

Actions (A). Capture a partial stream, resume it, verify chunk digests and require the streamed evaluation certificate inside a temporary directory.

Expected result (O). Pause after two evaluations, resume=True, four chunks, four stored coefficients and value 53. The temporary stream directory is deleted on exit.

Certificate and replay checks (C). The source contains 0 explicit `assert` statements and 1 syntactic `.require()` call site. These are source counts, not dynamic proof counts. The installed-wheel run passed 5 output/artifact predicates with exit code zero and 245 package modules checked for import origin.

Observed base-certificate identifiers include `PSC-I-T20`, `PSC-III-D60`.

Failure boundary (F). Tampered or incompatible stream chunks must be rejected. The captured finite snapshot is not a serialized arbitrary callback.

Source API routes. `komorebi_math.PacketSystem`, `komorebi_math.nonlinear.PacketJetFamily`, `komorebi_math.nonlinear.PolynomialDerivativeProvider`, `komorebi_math.nonlinear.capture_packet_jet_stream`, `komorebi_math.nonlinear.open_packet_jet_snapshot_stream`.

Reproducibility (R). `python examples/resumable_packet_jet_stream.py`. Reference wheel, NumPy, and the logged validation environment with SciPy, SymPy and mpmath. Direct imports are listed separately; this profile is not a proof of minimal transitive dependencies. No random input in the shipped fixture.

Runtime and filesystem. Recorded Linux replay: 0.926 seconds, measured under the shared reference profile; this is not a performance promise. The script owns a TemporaryDirectory for its kmstream and deletes it on normal exit. The runner retains logs and metadata, not the temporary stream.

Related package tests. `tests/test_block10_sparse_autodiff.py`, `tests/test_operators_transfer.py`, `tests/test_nonlinear_trees.py` and 16 more files in the shared-import crosswalk. These files were included in the separate 1,203-test source run. Shared imports alone do not prove full behavioral coverage.

Source SHA-256. `15c78001b6938920535c006a8c968dd8cda75a49ae029ea673acd80aca276cc8`. Full source and replay hashes are also stored in the JSON ledger and per-record result files.

Observed output excerpt. Selected fields or wrapped lines only. Full stdout is preserved in the replay directory.


```
{
  "paused_after": 2,
  "resumed": true,
  "chunks": 4,
  "stored_coefficients": 4,
  "stream_sha256":
"d635f63647540dd0c21604cf09bd7863ff47dab8e86e46b21482449e61c9f4ba",
  "value": [
    53.0
```


EX-E-049. Sparse derivative plan with backend disclosure

Source and placement. `komorebi-math/examples/scalable_nonlinear.py`. Chapters 37 through 43, scalable nonlinear execution. Local claim: `EX-E-049-CONTRACT`.

Input (I). Exact rational two-output derivative data with four stored entries versus ten dense entries, x/y packets and fixed rational directions.

Hypotheses and evidence class (H). EXACT declared input / NUMERICAL packet realization. The source fixture fixes all remaining literals and dimensions. The first observed certificate for each identifier, including checked and unchecked assumptions, is retained in the execution record.

Actions (A). Compile and execute sparse support, round-trip a jet snapshot and inspect CuPy availability without demanding a GPU execution.

Expected result (O). Two active and six pruned blocks, two derivative calls and reconstructed derivative (-4,-36). GPU status is diagnostic and must not be counted as a CUDA pass.

Certificate and replay checks (C). The source contains 0 explicit `assert` statements and 4 syntactic `.require()` call sites. These are source counts, not dynamic proof counts. The installed-wheel run passed 6 output/artifact predicates with exit code zero and 245 package modules checked for import origin.

Observed base-certificate identifiers include `PSC-I-T20`, `PSC-III-A10`, `PSC-III-C08`, `PSC-III-D58`.

Failure boundary (F). Exact coefficient inputs do not by themselves imply that all intermediate NumPy packet operations remain exact.

Source API routes. `komorebi_math.Exactness`, `komorebi_math.PacketSystem`, `komorebi_math.export.dumps_portable`, `komorebi_math.export.loads_portable`, `komorebi_math.nonlinear.PacketJetFamily` and 7 more routes in the JSON ledger.

Reproducibility (R). `python examples/scalable_nonlinear.py`. Reference wheel, NumPy, and the logged validation environment with SciPy, SymPy and mpmath. Direct imports are listed separately; this profile is not a proof of minimal transitive dependencies. No random input in the shipped fixture.

Runtime and filesystem. Recorded Linux replay: 0.787 seconds, measured under the shared reference profile; this is not a performance promise. No persistent output files are declared by the example. Replay stdout/stderr and metadata are retained in the selected evidence directory.

Related package tests. `tests/test_nonlinear_correction_elimination.py`, `tests/test_generating_block13.py`, `tests/test_block10_sparse_autodiff.py` and 21 more file in the shared-import crosswalk. These files were included in the separate 1,203-test source run. Shared imports alone do not prove full behavioral coverage.

Source SHA-256. `3279dfc545c0828f2ac84306e9db3ecb35277c3371f8518ac7d7b068d204c687`. Full source and replay hashes are also stored in the JSON ledger and per-record result files.

Observed output excerpt. Selected fields or wrapped lines only. Full stdout is preserved in the replay directory.

```
sparse entries: 4
dense entries: 10
active packet blocks: 2
pruned packet blocks: 6
provider derivative calls: 2
reconstructed derivative: (np.complex128(-4+0j), np.complex128(-36+0j))
snapshot coefficients: 1
portable bytes: 6806
```

EX-E-050. Sparse packet-jet runtime counts

Source and placement. `komorebi-math/examples/scalable_nonlinear_runtime.py`. Chapter 43, nonlinear runtime. Local claim: `EX-E-050-CONTRACT`.

Input (I). Three-dimensional tensor with x^2, xz and z^2 entries; three coordinate packets; directions (2,-1,1/2),(-1/4,3,4).

Hypotheses and evidence class (H). NUMERICAL derivative evaluation / EXACT counts. The source fixture fixes all remaining literals and dimensions. The first observed certificate for each identifier, including checked and unchecked assumptions, is retained in the execution record.

Actions (A). Require sparse-storage, support, plan and evaluation certificates and report the declared finite execution counts.

Expected result (O). Three stored versus 27 dense entries; four active and 23 pruned blocks; four planned versus 27 naive calls; result (-1,-11.8125,8).

Certificate and replay checks (C). The source contains 0 explicit `assert` statements and 4 syntactic `.require()` call sites. These are source counts, not dynamic proof counts. The installed-wheel run passed 7 output/artifact predicates with exit code zero and 244 package modules checked for import origin.

Observed base-certificate identifiers include `PSC-I-T20`, `PSC-III-A10`, `PSC-III-C08`, `PSC-III-D04P`.

Failure boundary (F). Pruning requires complete certified support, not merely absence from a sampled address list.

Source API routes. `komorebi_math.PacketSystem`, `komorebi_math.nonlinear.PacketJetFamily`, `komorebi_math.nonlinear.PolynomialDerivativeProvider`, `komorebi_math.nonlinear.SparsePolynomialDerivativeProvider`, `komorebi_math.nonlinear.compile_packet_jet_plan` and 2 more routes in the JSON ledger.

Reproducibility (R). `python examples/scalable_nonlinear_runtime.py`. Reference wheel, NumPy, and the logged validation environment with SciPy, SymPy and mpmath. Direct imports are listed separately; this profile is not a proof of minimal transitive dependencies. No random input in the shipped fixture.

Runtime and filesystem. Recorded Linux replay: 0.822 seconds, measured under the shared reference profile; this is not a performance promise. No persistent output files are declared by the example. Replay stdout/stderr and metadata are retained in the selected evidence directory.

Related package tests. `tests/test_block10_sparse_autodiff.py`, `tests/test_operators_transfer.py`, `tests/test_nonlinear_trees.py` and 16 more files in the shared-import crosswalk. These files were included in the separate 1,203-test source run. Shared imports alone do not prove full behavioral coverage.

Source SHA-256. `78874af6b3b667359f0482e72731c929dc8d8d2461a188cf8989afb9f846777c`. Full source and replay hashes are also stored in the JSON ledger and per-record result files.

Observed output excerpt. Selected fields or wrapped lines only. Full stdout is preserved in the replay directory.

```
stored derivative entries: 3
dense derivative entries: 27
active packet blocks: 4
pruned packet blocks: 23
planned derivative calls: 4
naive derivative calls: 27
reconstruction: [(-1+0j), (-11.8125+0j), (8+0j)]
```

EX-E-051. Declared sequence models and compiler strategies

Source and placement. `komorebi-math/examples/sequence_compiler.py`. Chapter 26, sequence compiler. Local claim: `EX-E-051-CONTRACT`.

Input (I). Built-in Fibonacci, Thue-Morse and Apery sequence specifications; twelve terms each.

Hypotheses and evidence class (H). EXACT finite sequence execution. The source fixture fixes all remaining literals and dimensions. The first observed certificate for each identifier, including checked and unchecked assumptions, is retained in the execution record.

Actions (A). Compile each declared model, require the compilation certificate and print its strategy and prefix.

Expected result (O). Strategies are declared linear recurrence, declared finite automaton and declared holonomic recurrence. Fibonacci ends the 12-term prefix at 89.

Certificate and replay checks (C). The source contains 0 explicit `assert` statements and 1 syntactic `.require()` call site. These are source counts, not dynamic proof counts. The installed-wheel run passed 3 output/artifact predicates with exit code zero and 244 package modules checked for import origin.

Failure boundary (F). The compiler uses supplied models here. This is not an experiment in autonomous discovery of an unknown recurrence.

Source API routes. `komorebi_math.sequences.apery_sequence`, `komorebi_math.sequences.compile_sequence`, `komorebi_math.sequences.fibonacci_sequence`, `komorebi_math.sequences.thue_morse_sequence`.

Reproducibility (R). `python examples/sequence_compiler.py`. Reference wheel, NumPy, and the logged validation environment with SciPy, SymPy and mpmath. Direct imports are listed separately; this profile is not a proof of minimal transitive dependencies. No random input in the shipped fixture.

Runtime and filesystem. Recorded Linux replay: 0.936 seconds, measured under the shared reference profile; this is not a performance promise. No persistent output files are declared by the example. Replay stdout/stderr and metadata are retained in the selected evidence directory.

Related package tests. `tests/test_sequences_execution.py`, `tests/test_sequences_models.py`. These files were included in the separate 1,203-test source run. Shared imports alone do not prove full behavioral coverage.

Source SHA-256. `10fe1ba2019775f48f935659f010283b161557f80e63cd783cbaf7004869a143`. Full source and replay hashes are also stored in the JSON ledger and per-record result files.

Observed output excerpt. Selected fields or wrapped lines only. Full stdout is preserved in the replay directory.

```
Fibonacci declared linear recurrence (0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55,
89)
Thue Morse declared finite automaton (0, 1, 1, 0, 1, 0, 0, 1, 1, 0, 0, 1)
Apery declared holonomic recurrence (Fraction(1, 1), Fraction(5, 1),
Fraction(73, 1), Fraction(1445, 1), Fraction(33001, 1), Fraction(819005, 1),
 Fraction(21460825, 1), Fraction(584307365, 1), Fraction(16367912425, 1),
Fraction(468690849005, 1), Fraction(13657436403073, 1),
Fraction(403676083788125, 1))
```

EX-E-052. Separated jumps and retained reconstruction

Source and placement. `komorebi-math/examples/singular_packet_recovery.py`. Chapter 36, singular packet recovery. Local claim: `EX-E-052-CONTRACT`.

Input (I). 513 samples on a 512-cell grid; jumps at 0.23,0.71 with amplitudes 0.45,-0.35; sinusoidal background with declared Lipschitz bound 0.08*pi and zero noise.

Hypotheses and evidence class (H). NUMERICAL finite sampling / stated separation hypotheses. The source fixture fixes all remaining literals and dimensions. The first observed certificate for each identifier, including checked and unchecked assumptions, is retained in the execution record.

Actions (A). Require separated-jump recovery, build the singular packet and certify an identity retained-reconstruction operator.

Expected result (O). Detected cells are (117,363), estimated locations approximately (0.22949219,0.70996094). Location errors are within one grid cell; amplitude errors obey the finite smooth-background bound.

Certificate and replay checks (C). The source contains 0 explicit `assert` statements and 2 syntactic `.require()` call sites. These are source counts, not dynamic proof counts. The installed-wheel run passed 1 output/artifact predicate with exit code zero and 244 package modules checked for import origin.

Observed base-certificate identifiers include `PSC-V-T18`, `PSC-V-T36`.

Failure boundary (F). The Lipschitz and separation hypotheses are supplied. The identity reconstruction is not evidence of compression or recovery under arbitrary noise.

Source API routes. `komorebi_math.signals.SingularPacket1D`, `komorebi_math.signals.recover_separated_jumps`, `komorebi_math.signals.retained_packet_reconstruction`.

Reproducibility (R). `python examples/singular_packet_recovery.py`. Reference wheel, NumPy, and the logged validation environment with SciPy, SymPy and mpmath. Direct imports are listed separately; this profile is not a proof of minimal transitive dependencies. No random input in the shipped fixture.

Runtime and filesystem. Recorded Linux replay: 0.825 seconds, measured under the shared reference profile; this is not a performance promise. No persistent output files are declared by the example. Replay stdout/stderr and metadata are retained in the selected evidence directory.

Related package tests. `tests/test_signals_recovery.py`, `tests/test_signals_wavelets_singular.py`. These files were included in the separate 1,203-test source run. Shared imports alone do not prove full behavioral coverage.

Source SHA-256. `5f11bb3142a0b715562bbbd8d07935885214ee42b60d204b717d53faa3655f3c`. Full source and replay hashes are also stored in the JSON ledger and per-record result files.

Observed output excerpt. Selected fields or wrapped lines only. Full stdout is preserved in the replay directory.

```
detected cells: (117, 363)
estimated locations: [0.22949219 0.70996094]
estimated amplitudes: [ 0.45006308 -0.35012219]
```

EX-E-053. Fourier, Haar and exact residue ordering

Source and placement. `komorebi-math/examples/spectral_packet_coordinates.py`. Chapters 19 and 32, spectral packet coordinates. Local claim: `EX-E-053-CONTRACT`.

Input (I). Eight samples arange(8), normalized DFT and Haar transforms, and an exact modulus-2 residue permutation.

Hypotheses and evidence class (H). NUMERICAL spectral transforms / EXACT permutation. The source fixture fixes all remaining literals and dimensions. The first observed certificate for each identifier, including checked and unchecked assumptions, is retained in the execution record.

Actions (A). Require spectral transform certificates, invert both numerical transforms and round-trip the exact residue ordering.

Expected result (O). The residue order is (0,2,4,6,1,3,5,7); Fourier and Haar reconstruct the input to tolerance.

Certificate and replay checks (C). The source contains 3 explicit `assert` statements and 2 syntactic `.require()` call sites. These are source counts, not dynamic proof counts. The installed-wheel run passed 1 output/artifact predicate with exit code zero and 244 package modules checked for import origin.

Observed base-certificate identifiers include `CC.S1`, `CC.S4`.

Failure boundary (F). Tiny complex residuals in Fourier coordinates are numerical roundoff, not nonzero exact invariants.

Source API routes. `komorebi_math.coordinates.discrete_fourier_transform`, `komorebi_math.coordinates.haar_wavelet_transform`, `komorebi_math.coordinates.residue_class_transform`.

Reproducibility (R). `python examples/spectral_packet_coordinates.py`. Reference wheel, NumPy, and the logged validation environment with SciPy, SymPy and mpmath. Direct imports are listed separately; this profile is not a proof of minimal transitive dependencies. No random input in the shipped fixture.

Runtime and filesystem. Recorded Linux replay: 0.933 seconds, measured under the shared reference profile; this is not a performance promise. No persistent output files are declared by the example. Replay stdout/stderr and metadata are retained in the selected evidence directory.

Related package tests. `tests/test_coordinates_general_persistence.py`. These files were included in the separate 1,203-test source run. Shared imports alone do not prove full behavioral coverage.

Source SHA-256. `d72f667e04cca2940e0fcc064faf36a01e98f58c373d3f32e18148a85bf38be4`. Full source and replay hashes are also stored in the JSON ledger and per-record result files.

Observed output excerpt. Selected fields or wrapped lines only. Full stdout is preserved in the replay directory.

```
Fourier coordinates: [ 9.89949494+0.00000000e+00j
-1.41421356+3.41421356e+00j
 -1.41421356+1.41421356e+00j -1.41421356+5.85786438e-01j
 -1.41421356-1.21233848e-15j -1.41421356-5.85786438e-01j
 -1.41421356-1.41421356e+00j -1.41421356-3.41421356e+00j]
Haar coordinates: [ 9.89949494 -5.65685425 -2.         -2.
-0.70710678 -0.70710678
 -0.70710678 -0.70710678]
residue-lane order: (Fraction(0, 1), Fraction(2, 1), Fraction(4, 1),
Fraction(6, 1), Fraction(1, 1), Fraction(3, 1), Fraction(5, 1), Fraction(7,
1))
```

EX-E-054. Standard-window and affine Mahler refinement

Source and placement. `komorebi-math/examples/standard_window_mahler.py`. Chapter 25, standard-window Mahler models. Local claim: `EX-E-054-CONTRACT`.

Input (I). Prime 7, compression 2, address 3 and degree 12; values n^4+2n+1 for n=0,…,8; affine change n -> 3n+2.

Hypotheses and evidence class (H). EXACT polynomial/Mahler algebra. The source fixture fixes all remaining literals and dimensions. The first observed certificate for each identifier, including checked and unchecked assumptions, is retained in the execution record.

Actions (A). Require the finite standard-window certificate, form a Mahler series from the exact values and evaluate eight refined terms.

Expected result (O). Window period 3; refined values start 21,636,4113,14664 and end at 279888 for the eight-term prefix.

Certificate and replay checks (C). The source contains 0 explicit `assert` statements and 1 syntactic `.require()` call site. These are source counts, not dynamic proof counts. The installed-wheel run passed 3 output/artifact predicates with exit code zero and 244 package modules checked for import origin.

Observed base-certificate identifiers include `PSC-IX-T09`.

Failure boundary (F). A finite interpolation model is not an arbitrary infinite-sequence continuation without its polynomial hypothesis.

Source API routes. `komorebi_math.arithmetic.MahlerSeries`, `komorebi_math.arithmetic.StandardWindow`.

Reproducibility (R). `python examples/standard_window_mahler.py`. Reference wheel, NumPy, and the logged validation environment with SciPy, SymPy and mpmath. Direct imports are listed separately; this profile is not a proof of minimal transitive dependencies. No random input in the shipped fixture.

Runtime and filesystem. Recorded Linux replay: 0.786 seconds, measured under the shared reference profile; this is not a performance promise. No persistent output files are declared by the example. Replay stdout/stderr and metadata are retained in the selected evidence directory.

Related package tests. `tests/test_arithmetic_hurwitz_windows.py`, `tests/test_arithmetic_mahler_polyphase.py`. These files were included in the separate 1,203-test source run. Shared imports alone do not prove full behavioral coverage.

Source SHA-256. `3ae534dd9fa2f50a2d9d4b6b79bd2cdc91c12f266acd683d948f51a32c4c87ca`. Full source and replay hashes are also stored in the JSON ledger and per-record result files.

Observed output excerpt. Selected fields or wrapped lines only. Full stdout is preserved in the replay directory.

```
period 3
window (1, 6, 5, 2, 6, 5, 2, 6, 5, 2, 6, 5, 2)
refined (Fraction(21, 1), Fraction(636, 1), Fraction(4113, 1),
Fraction(14664, 1), Fraction(38445, 1), Fraction(83556, 1), Fraction(160041,
 1), Fraction(279888, 1))
```

EX-E-055. Preconditioned conjugate gradient

Source and placement. `komorebi-math/examples/structured_krylov.py`. Chapters 29 through 30, structured Krylov methods. Local claim: `EX-E-055-CONTRACT`.

Input (I). The shipped symmetric positive-definite 4-by-4 matrix, right-hand side (1,2,3,4), layout (2,2) and relative tolerance 1e-12.

Hypotheses and evidence class (H). NUMERICAL linear solve. The source fixture fixes all remaining literals and dimensions. The first observed certificate for each identifier, including checked and unchecked assumptions, is retained in the execution record.

Actions (A). Require the block-Jacobi certificate, solve with preconditioned CG and require its convergence certificate.

Expected result (O). Four iterations on this fixture and a residual below 1e-10.

Certificate and replay checks (C). The source contains 0 explicit `assert` statements and 2 syntactic `.require()` call sites. These are source counts, not dynamic proof counts. The installed-wheel run passed 2 output/artifact predicates with exit code zero and 244 package modules checked for import origin.

Observed base-certificate identifiers include `PSC-VIII-T22`, `PSC-VIII-T24`.

Failure boundary (F). A convergence certificate must not be inferred from an iteration cap alone; indefinite matrices violate CG assumptions.

Source API routes. `komorebi_math.algorithms.PacketLayout`, `komorebi_math.algorithms.block_jacobi_certificate`, `komorebi_math.algorithms.block_jacobi_preconditioner`, `komorebi_math.algorithms.conjugate_gradient`.

Reproducibility (R). `python examples/structured_krylov.py`. Reference wheel, NumPy, and the logged validation environment with SciPy, SymPy and mpmath. Direct imports are listed separately; this profile is not a proof of minimal transitive dependencies. No random input in the shipped fixture.

Runtime and filesystem. Recorded Linux replay: 0.940 seconds, measured under the shared reference profile; this is not a performance promise. No persistent output files are declared by the example. Replay stdout/stderr and metadata are retained in the selected evidence directory.

Related package tests. `tests/test_algorithms_complexity_benchmarks.py`, `tests/test_algorithms_discovery_gpu.py`, `tests/test_algorithms_krylov.py` and 1 more file in the shared-import crosswalk. These files were included in the separate 1,203-test source run. Shared imports alone do not prove full behavioral coverage.

Source SHA-256. `7b1259b89df76c0498535fc3dfba37ba5c8166cbff8e4424e021d366aa354f3b`. Full source and replay hashes are also stored in the JSON ledger and per-record result files.

Observed output excerpt. Selected fields or wrapped lines only. Full stdout is preserved in the replay directory.

```
iterations 4
residual 9.930136612989092e-16
```

EX-E-056. Finite tame and Heisenberg packets

Source and placement. `komorebi-math/examples/tame_heisenberg.py`. Chapter 57, tame Heisenberg packets. Local claim: `EX-E-056-CONTRACT`.

Input (I). Frobenius q=2, degree 3, exponent 1; Heisenberg prime 3 and rank 1; two explicitly given group elements.

Hypotheses and evidence class (H). FINITE exact group data / NUMERICAL matrix representation. The source fixture fixes all remaining literals and dimensions. The first observed certificate for each identifier, including checked and unchecked assumptions, is retained in the execution record.

Actions (A). Require orbit, tame relation, group-law and selector certificates.

Expected result (O). Orbit (1,2,4), tame and Heisenberg dimensions 3, determinant orientation 1 and selector rank 1.

Certificate and replay checks (C). The source contains 0 explicit `assert` statements and 4 syntactic `.require()` call sites. These are source counts, not dynamic proof counts. The installed-wheel run passed 4 output/artifact predicates with exit code zero and 244 package modules checked for import origin.

Observed base-certificate identifiers include `PSC-XI-HEISENBERG-LAW`, `PSC-XI-ISOTROPIC-SELECTOR`, `PSC-XI-TAME-ATOM`, `PSC-XI-TAME-ORBIT`.

Failure boundary (F). Finite packet relations do not establish a global Langlands correspondence.

Source API routes. `komorebi_math.langlands.local.FrobeniusOrbit`, `komorebi_math.langlands.local.HeisenbergModel`, `komorebi_math.langlands.local.TameWeilPacket`.

Reproducibility (R). `python examples/tame_heisenberg.py`. Reference wheel, NumPy, and the logged validation environment with SciPy, SymPy and mpmath. Direct imports are listed separately; this profile is not a proof of minimal transitive dependencies. No random input in the shipped fixture.

Runtime and filesystem. Recorded Linux replay: 0.780 seconds, measured under the shared reference profile; this is not a performance promise. No persistent output files are declared by the example. Replay stdout/stderr and metadata are retained in the selected evidence directory.

Related package tests. `tests/test_langlands_local_fields_tame_gauss.py`, `tests/test_langlands_local_heisenberg_symplectic.py`. These files were included in the separate 1,203-test source run. Shared imports alone do not prove full behavioral coverage.

Source SHA-256. `319d3d92a3187ec81723b6a2d80f0a0a3fc6d2b83f9dd6c53b04251fb966b0a2`. Full source and replay hashes are also stored in the JSON ledger and per-record result files.

Observed output excerpt. Selected fields or wrapped lines only. Full stdout is preserved in the replay directory.

```
{'frobenius_orbit': (1, 2, 4), 'tame_dimension': 3,
'determinant_orientation': (1+0j), 'heisenberg_dimension': 3,
'selector_rank': 1}
```

EX-E-057. Typed operator relation certificate

Source and placement. `komorebi-math/examples/typed_grammar.py`. Chapters 9 and 50, typed grammars. Local claim: `EX-E-057-CONTRACT`.

Input (I). One object X of dimension 3, generator T and relation T^3-T=0; realization T=diag(-1,0,1).

Hypotheses and evidence class (H). NUMERICAL finite matrix relation. The source fixture fixes all remaining literals and dimensions. The first observed certificate for each identifier, including checked and unchecked assumptions, is retained in the execution record.

Actions (A). Build the typed relation and print its numerical certificate.

Expected result (O). Certificate valid=True, relation count 1, cubic residual 0 and theorem identifier PSC-I-T01.

Certificate and replay checks (C). The source contains 0 explicit `assert` statements and 0 syntactic `.require()` call sites. These are source counts, not dynamic proof counts. The installed-wheel run passed 4 output/artifact predicates with exit code zero and 244 package modules checked for import origin.

Failure boundary (F). A dimension mismatch or realization that violates T^3=T must not receive a valid relation certificate.

Source API routes. `komorebi_math.OperatorGrammar`, `komorebi_math.OperatorRealization`.

Reproducibility (R). `python examples/typed_grammar.py`. Reference wheel, NumPy, and the logged validation environment with SciPy, SymPy and mpmath. Direct imports are listed separately; this profile is not a proof of minimal transitive dependencies. No random input in the shipped fixture.

Runtime and filesystem. Recorded Linux replay: 0.791 seconds, measured under the shared reference profile; this is not a performance promise. No persistent output files are declared by the example. Replay stdout/stderr and metadata are retained in the selected evidence directory.

Related package tests. `tests/test_unified_extensions_cli.py`, `tests/test_grammar_realization.py`. These files were included in the separate 1,203-test source run. Shared imports alone do not prove full behavioral coverage.

Source SHA-256. `92242b96f29d7e06fc35ad7059e03d9567bb20743940154189c0bc91db3a4df0`. Full source and replay hashes are also stored in the JSON ledger and per-record result files.

Observed output excerpt. Selected fields or wrapped lines only. Full stdout is preserved in the replay directory.

```
{
  "assumptions_checked": [
    "operator shapes",
    "typed path composition",
    "declared relations"
  ],
  "assumptions_unchecked": [],
  "diagnostics": {
```

E.3 PowerShell and historical records

EX-E-058. Scalable nonlinear workflow

Source and placement. `komorebi-math/examples/Test-KomorebiBlock10.ps1`. Chapters 63 and 71, scalable nonlinear CLI verification. Local claim: `EX-E-058-CONTRACT`.

Input (I). -Python (default python). Root is resolved from PSScriptRoot.

Hypotheses and evidence class (H). ORCHESTRATION evidence, not a mathematical exactness class. Requires the native PowerShell interpreter and the paths, dependencies and source release explicitly named by the script. A Linux reconstruction of child commands is not a native PowerShell qualification.

Actions (A). Runs sparse runtime and portable-state examples, strict research API validation, verify block10 and its deep audit.

Expected result (O). Both examples and all verifier children must report success.

Certificate and replay checks (C). Legacy workflow for the reference package; native PowerShell runtime has not been tested. Native acceptance requires individual child exit codes, expected outputs/artifacts and a deliberate failing-child control. No success is claimed by the static inventory.

Failure boundary (F). The script does not explicitly test LASTEXITCODE after every native command. ErrorActionPreference alone is not treated here as a fail-closed native-child gate. Inspect each child result; inject a nonzero Python exit during native qualification.

Source API routes. `CLI api check --strict-signatures`, `CLI verify block10`.

Reproducibility (R). `powershell -NoProfile -File examples/Test-KomorebiBlock10.ps1`. Native PowerShell, installed Python and required package dependencies. Export additionally expects Windows Scripts/python.exe and Script-

s/komorebi.exe; optional LaTeX enables PDFs. No top-level deterministic seed. Child scripts retain their own declared seeds.

Runtime and filesystem. Not measured; workflow may run multiple verification suites or build/install operations. komorebi-math repository directory. Export output must be disposable; historical Everything must use a separate matching 0.16.0a2 archive. The individual verification wrappers primarily produce console output. Export retains its environment and exports. Historical Everything owns a results/work directory and applies its own cleanup policy; it is not executed here.

Source SHA-256. `bdedbcdf85ef86b804c67d58728879ee1f68066af9f50fa1a85e09bb189f81d7`. Full source and replay hashes are also stored in the JSON ledger and per-record result files.

EX-E-059. Streaming and semantic workflow

Source and placement. `komorebi-math/examples/Test-KomorebiBlock11.ps1`. Chapters 63, 71, and 72, streaming and semantic verification. Local claim: `EX-E-059-CONTRACT`.

Input (I). -Python (default python). Root is resolved from PSScriptRoot.

Hypotheses and evidence class (H). ORCHESTRATION evidence, not a mathematical exactness class. Requires the native PowerShell interpreter and the paths, dependencies and source release explicitly named by the script. A Linux reconstruction of child commands is not a native PowerShell qualification.

Actions (A). Runs resumable streams, semantic profiling, golden fixtures, API checks, verify block11 and its deep audit.

Expected result (O). The stream resumes correctly, semantic counts match, and all verifier children succeed.

Certificate and replay checks (C). Legacy workflow for the reference package; native PowerShell runtime has not been tested. Native acceptance requires individual child exit codes, expected outputs/artifacts and a deliberate failing-child control. No success is claimed by the static inventory.

Failure boundary (F). The script does not explicitly test LASTEXITCODE after every native command. ErrorActionPreference alone is not treated here as a fail-closed native-child gate. Inspect each child result; inject a nonzero Python exit during native qualification.

Source API routes. `CLI golden`, `CLI verify block11`.

Reproducibility (R). `powershell -NoProfile -File examples/Test-KomorebiBlock11.ps1`. Native PowerShell, installed Python and required package dependencies. Export additionally expects Windows Scripts/python.exe and Scripts/komorebi.exe; optional LaTeX enables PDFs. No top-level deterministic seed. Child scripts retain their own declared seeds.

Runtime and filesystem. Not measured; workflow may run multiple verification suites or build/install operations. komorebi-math repository directory. Export output must be disposable; historical Everything must use a separate matching 0.16.0a2 archive. The individual verification wrappers primarily produce console output. Export retains its environment and exports. Historical Everything owns a results/work directory and applies its own cleanup policy; it is not executed here.

Source SHA-256. `a8e89805f07e3e2c94082e908ee4d62666c34d1bfba3d6edc3ae875ca108222b`. Full source and replay hashes are also stored in the JSON ledger and per-record result files.

EX-E-060. DSP workflow

Source and placement. `komorebi-math/examples/Test-KomorebiBlock12.ps1`. Chapters 63 and 71, DSP CLI verification. Local claim: `EX-E-060-CONTRACT`.

Input (I). No parameters. python must be on PATH. Run from the komorebi-math directory because the example path is relative.

Hypotheses and evidence class (H). ORCHESTRATION evidence, not a mathematical exactness class. Requires the native PowerShell interpreter and the paths, dependencies and source release explicitly named by the script. A Linux reconstruction of child commands is not a native PowerShell qualification.

Actions (A). Runs DSP verification and its aggregate suite, strict/beta API checks and production_dsp.py.

Expected result (O). Five child invocations must succeed; the DSP record must match its finite fixture contract.

Certificate and replay checks (C). Legacy workflow for the reference package; native PowerShell runtime has not been tested. Native acceptance requires individual child exit codes, expected outputs/artifacts and a deliberate failing-child control. No success is claimed by the static inventory.

Failure boundary (F). The script does not explicitly test LASTEXITCODE after every native command. ErrorActionPreference alone is not treated here as a fail-closed native-child gate. Inspect each child result; inject a nonzero Python exit during native qualification.

Source API routes. `CLI verify dsp`, `CLI verify block12`.

Reproducibility (R). `powershell -NoProfile -File examples/Test-KomorebiBlock12.ps1`. Native PowerShell, installed Python and required package dependencies. Export additionally expects Windows Scripts/python.exe and Scripts/komorebi.exe; optional LaTeX enables PDFs. No top-level deterministic seed. Child scripts retain their own declared seeds.

Runtime and filesystem. Not measured; workflow may run multiple verification suites or build/install operations. komorebi-math repository directory. Export output must be disposable; historical Everything must use a separate matching 0.16.0a2 archive. The individual verification wrappers primarily produce console output. Export retains its environment and exports. Historical Everything owns a results/work directory and applies its own cleanup policy; it is not executed here.

Source SHA-256. `6609830bf08e1ea2d5bd19369be9a3139a4cdecc5c6ff86e988c8f69b9ca0729`. Full source and replay hashes are also stored in the JSON ledger and per-record result files.

EX-E-061. Generating-function workflow

Source and placement. `komorebi-math/examples/Test-KomorebiBlock13.ps1`. Chapters 63 and 71, generating-function CLI verification. Local claim: `EX-E-061-CONTRACT`.

Input (I). No parameters. python on PATH; run from komorebi-math. The script hardcodes finite mask, Cartier, kernel and Bell parameters.

Hypotheses and evidence class (H). ORCHESTRATION evidence, not a mathematical exactness class. Requires the native PowerShell interpreter and the paths, dependencies and source release explicitly named by the script. A Linux reconstruction of child commands is not a native PowerShell qualification.

Actions (A). Runs generating-function verification and its aggregate suite, API checks, four generating commands and three generating examples.

Expected result (O). All verifier, CLI and example children must succeed. Cartier coefficients refer to a rational-mask Mahler series.

Certificate and replay checks (C). Legacy workflow for the reference package; native PowerShell runtime has not been tested. Native acceptance requires individual child exit codes, expected outputs/artifacts and a deliberate failing-child control. No success is claimed by the static inventory.

Failure boundary (F). The script does not explicitly test LASTEXITCODE after every native command. ErrorActionPreference alone is not treated here as a fail-closed native-child gate. Inspect each child result; inject a nonzero Python exit during native qualification.

Source API routes. `CLI verify generating`, `CLI generating mask`, `CLI generating cartier`, `CLI generating kernel`, `CLI generating bell`.

Reproducibility (R). `powershell -NoProfile -File examples/Test-KomorebiBlock13.ps1`. Native PowerShell, installed Python and required package dependencies. Export additionally expects Windows Scripts/python.exe and Scripts/komorebi.exe; optional LaTeX enables PDFs. No top-level deterministic seed. Child scripts retain their own declared seeds.

Runtime and filesystem. Not measured; workflow may run multiple verification suites or build/install operations. komorebi-math repository directory. Export output must be disposable; historical Everything must use a separate matching 0.16.0a2 archive. The individual verification wrappers primarily produce console output. Export retains its environment and exports. Historical Everything owns a results/work directory and applies its own cleanup policy; it is not executed here.

Source SHA-256. `ea48d1edaec34a55d9c18cec97320577fa2a1634f4200ce001d458ff03ba8012`. Full source and replay hashes are also stored in the JSON ledger and per-record result files.

EX-E-062. Coordinate workflow

Source and placement. `komorebi-math/examples/Test-KomorebiBlock14.ps1`. Chapters 63 and 74A, coordinate example with `--length` syntax. Local claim: `EX-E-062-CONTRACT`.

Input (I). -Python (default python); named coordinate fixtures and packet –length 12 –modulus 3.

Hypotheses and evidence class (H). ORCHESTRATION evidence, not a mathematical exactness class. Requires the native PowerShell interpreter and the paths, dependencies and source release explicitly named by the script. A Linux reconstruction of child commands is not a native PowerShell qualification.

Actions (A). Runs coordinate verification and its aggregate suite, coordinate catalog and eight coordinate operations, golden fixtures and API checks.

Expected result (O). All child commands must succeed. The packet invocation uses the corrected –length option, not –size.

Certificate and replay checks (C). Legacy workflow for the reference package; native PowerShell runtime has not been tested. Native acceptance requires individual child exit codes, expected outputs/artifacts and a deliberate failing-child control. No success is claimed by the static inventory.

Failure boundary (F). The script does not explicitly test LASTEXITCODE after every native command. ErrorActionPreference alone is not treated here as a fail-closed native-child gate. Inspect each child result; inject a nonzero Python exit during native qualification.

Source API routes. `CLI verify coordinates`, `CLI coordinates packet`, `CLI coordinates model`, `CLI coordinates formal`.

Reproducibility (R). `powershell -NoProfile -File examples/Test-KomorebiBlock14.ps1`. Native PowerShell, installed Python and required package dependencies. Export additionally expects Windows Scripts/python.exe and Scripts/komorebi.exe; optional LaTeX enables PDFs. No top-level deterministic seed. Child scripts retain their own declared seeds.

Runtime and filesystem. Not measured; workflow may run multiple verification suites or build/install operations. komorebi-math repository directory. Export output must be disposable; historical Everything must use a separate matching 0.16.0a2 archive. The individual verification wrappers primarily produce console output. Export retains its environment and exports. Historical Everything owns a results/work directory and applies its own cleanup policy; it is not executed here.

Source SHA-256. `9fc18b1b97995c2864b1ba6c2cd7124b4413d715ad73d4680168f31e1b9e77c7`. Full source and replay hashes are also stored in the JSON ledger and per-record result files.

EX-E-063. Nonlinear verification workflow

Source and placement. `komorebi-math/examples/Test-KomorebiBlock9.ps1`. Chapters 63 and 71, nonlinear CLI verification. Local claim: `EX-E-063-CONTRACT`.

Input (I). -Python (default python). Root is resolved from PSScriptRoot.

Hypotheses and evidence class (H). ORCHESTRATION evidence, not a mathematical exactness class. Requires the native PowerShell interpreter and the paths, dependencies and source release explicitly named by the script. A Linux reconstruction of child commands is not a native PowerShell qualification.

Actions (A). Runs jets, tree elimination, filtered transport, verify block9 and the nonlinear deep audit.

Expected result (O). All five child commands must succeed independently.

Certificate and replay checks (C). Legacy workflow for the reference package; native PowerShell runtime has not been tested. Native acceptance requires individual child exit codes, expected outputs/artifacts and a deliberate failing-child control. No success is claimed by the static inventory.

Failure boundary (F). The script does not explicitly test LASTEXITCODE after every native command. ErrorActionPreference alone is not treated here as a fail-closed native-child gate. Inspect each child result; inject a nonzero Python exit during native qualification.

Source API routes. `CLI verify block9`, `verification/block9_deep_audit.py`.

Reproducibility (R). `powershell -NoProfile -File examples/Test-KomorebiBlock9.ps1`. Native PowerShell, installed Python and required package dependencies. Export additionally expects Windows Scripts/python.exe and Script-

s/komorebi.exe; optional LaTeX enables PDFs. No top-level deterministic seed. Child scripts retain their own declared seeds.

Runtime and filesystem. Not measured; workflow may run multiple verification suites or build/install operations. komorebi-math repository directory. Export output must be disposable; historical Everything must use a separate matching 0.16.0a2 archive. The individual verification wrappers primarily produce console output. Export retains its environment and exports. Historical Everything owns a results/work directory and applies its own cleanup policy; it is not executed here.

Source SHA-256. `a53bcd528b9293e2c5f840f150d0886beca7501b4ba90578e2dcaca4873c6623`. Full source and replay hashes are also stored in the JSON ledger and per-record result files.

EX-E-064. Historical aggregate runner

Source and placement. `komorebi-math/examples/Test-KomorebiEverything.ps1`. Chapters 63 and 71, historical aggregate verification. Local claim: `EX-E-064-CONTRACT`.

Input (I). Modes Quick/Full/Maximum; -Python defaults to py. Other switches control system-site dependencies, offline wheelhouse/network, optional AD/CUDA/build checks, retained work and results directory.

Hypotheses and evidence class (H). ORCHESTRATION evidence, not a mathematical exactness class. Requires the native PowerShell interpreter and the paths, dependencies and source release explicitly named by the script. A Linux reconstruction of child commands is not a native PowerShell qualification.

Actions (A). Searches specifically for the 0.16.0a2 wheel/source distribution and optional historical Git bundle, then orchestrates that release workflow.

Expected result (O). In the reference delivery, the required historical artifacts are absent. This script is not the reference release gate. The reference delivery uses VERIFY_REPAIR.ps1.

Certificate and replay checks (C). Historical, not applicable as a reference release check; not executed. Native acceptance requires individual child exit codes, expected outputs/artifacts and a deliberate failing-child control. No success is claimed by the static inventory.

Failure boundary (F). The fixed 0.16.0a2 artifact lookup prevents use as a reference release gate. Do not remove this version guard merely to obtain a green exit.

Source API routes. `historical CLI verify block16`, `historical release 0.16.0a2`.

Reproducibility (R). `Historical record only: do not run as the R2 release gate.`. Native PowerShell, installed Python and required package dependencies. Export additionally expects Windows Scripts/python.exe and Scripts/komorebi.exe; optional LaTeX enables PDFs. No top-level deterministic seed. Child scripts retain their own declared seeds.

Runtime and filesystem. Not measured; workflow may run multiple verification suites or build/install operations. komorebi-math repository directory. Export output must be disposable; historical Everything must use a separate matching 0.16.0a2 archive. The individual verification wrappers primarily produce console output. Export retains its environment and exports. Historical Everything owns a results/work directory and applies its own cleanup policy; it is not executed here.

Source SHA-256. `178283fd02310695b18ad69733b8b36f381bc08d8c15977eeafdce23bdbcad4b`. Full source and replay hashes are also stored in the JSON ledger and per-record result files.

EX-E-065. Windows wheel installation and export workflow

Source and placement. `komorebi-math/examples/Test-KomorebiExport.ps1`. Chapters 59 through 60 and 63, export CLI verification. Local claim: `EX-E-065-CONTRACT`.

Input (I). The `-Wheel` argument is required. `-Python` defaults to `python`, and `-OutputDirectory` defaults to `.\komorebi_export_test`. The optional switches are `UseSystemSitePackages`, `NoDependencies` and `SkipPdf`.

Hypotheses and evidence class (H). ORCHESTRATION evidence, not a mathematical exactness class. Requires the native PowerShell interpreter and the paths, dependencies and source release explicitly named by the script. A Linux reconstruction of child commands is not a native PowerShell qualification.

Actions (A). Creates a Windows-layout virtual environment, installs the supplied wheel, runs export/LaTeX CLIs, writes sequence and sample reports and prints SHA-256 hashes.

Expected result (O). Current wheel installation plus nonempty matching JSON/CSV/Markdown/HTML/TeX and ZIP outputs are required. PDF paths are conditional on SkipPdf and compiler availability.

Certificate and replay checks (C). Windows-specific virtual-environment paths; native runtime not executed. Native acceptance requires individual child exit codes, expected outputs/artifacts and a deliberate failing-child control. No success is claimed by the static inventory.

Failure boundary (F). The script does not explicitly test LASTEXITCODE after every native command. ErrorActionPreference alone is not treated here as a fail-closed native-child gate. Inspect each child result; inject a nonzero Python exit during native qualification.

Source API routes. `CLI export sequence`, `CLI export json`, `CLI export bundle`, `CLI latex compile`, `komorebi_math.export.export_data`.

Reproducibility (R). `powershell -NoProfile -File examples/Test-KomorebiExport.ps1 -Wheel release/dist/komorebi_math-1.0.0b4-2-py3-none-any.whl`. Native PowerShell, installed Python and required package dependencies. Export additionally expects Windows Scripts/python.exe and Scripts/komorebi.exe; optional LaTeX enables PDFs. No top-level deterministic seed. Child scripts retain their own declared seeds.

Runtime and filesystem. Not measured; workflow may run multiple verification suites or build/install operations. komorebi-math repository directory. Export output must be disposable; historical Everything must use a separate matching 0.16.0a2 archive. The individual verification wrappers primarily produce console output. Export retains its environment and exports. Historical Everything owns a results/work directory and applies its own cleanup policy; it is not executed here.

Source SHA-256. `ae4682c36e744a1b4f3d90be8df0f2d2e99859c1bb9ed395c27b852b317c83a3`. Full source and replay hashes are also stored in the JSON ledger and per-record result files.

E.4 Coverage and qualification boundaries

The record/source bijection covers **65 of 65 scripts** and all record anchors are unique. The recorded Python replay covers **57 of 57 default invocations**. The recorded full-source suite passed 1,203 tests and the separate record/contract suite passed 134 tests. These results describe the unchanged reference package and scripts.

Appendix E's documentation-record layer is complete. Its all-platform execution predicate is not: seven reference native PowerShell workflows remain unexecuted, and the historical aggregate is explicitly not applicable to the reference package. Optional GPU/device observations do not establish CUDA qualification. The replay checks default invocations, not every possible argument combination, input size or backend.

Appendix F provides the complete CLI option matrix for this reference baseline. Strict HTML validation, native Windows qualification, and the final publication predicate require their own evidence.

Appendix F. Complete CLI option matrix

Using the command reference. Begin with the command record for your task, read its argument table, and then inspect its output, file effects, and failure contract. The smallest invocation is a starting point using the prepared fixtures; it may depend on files created in Section F.1. For a conceptual introduction to command-line use, read Chapter 63 first.

A short command workflow.

1. Select the Python interpreter containing the package and dependencies you intend to use. The `python -m komorebi_math` form makes that selection explicit.
2. Inspect help for the exact command or subcommand you need. Compare it with this version-bound record before reusing a script with another release.
3. Read the argument types and defaults. Distinguish a switch from an option that consumes a value, and distinguish an input path from an output path.
4. Prepare the stated input files and choose a new output location when the command writes files. The replacement rules differ by command.
5. After the command finishes, inspect the exit status and diagnostics as well as any result or certificate. A file can exist after a failed run.

The fixture helper in F.1 is distributed with the source material used for the recorded replay. It is not an installed CLI subcommand. If you have only the wheel, obtain the corresponding source fixtures or prepare equivalent inputs according to the selected command's contract. Do not copy a fixture pathname without first creating the file it names.

Common command-line difficulties.

Symptom	First check
An option is reported as unrecognized	Confirm the command group, the installed version, and the exact full option spelling in that command's help. Options are not automatically global.
A negative list is treated as another option	Use the attached-value form illustrated in F.0, such as `--values=-1,0,1`.
A matrix argument is split by the shell	Quote the complete value, including semicolon-separated rows.
Parsing succeeds but the operation rejects the input	Read the semantic boundary: parser types do not enforce every mathematical precondition.
An output path already exists	Check this command's replacement policy and select a fresh path before requesting an overwrite.
A JSON response or output file exists but the run failed	Inspect the exit status, stderr, and the response's validation or certificate fields.
A verification command reports an unavailable GPU or unmet gate	Read the qualification requirement and the recorded scope; an expected rejection is not a qualified platform.

Use the suite-dispatch table in F.2 when choosing verification. The name `all` has a specific recorded dispatch target and should not be read as every possible test in the repository. Similarly, shared export flags can have different effects on different formats. The per-command semantic boundary explains what is actually consumed and produced.

F.0 Scope, syntax and evidence

The parser, commands, source locations, and outcomes below are the recorded reference-baseline inventory. They preserve the complete documented interface and do not assert that later package revisions expose identical commands or have been replayed here.

This reference covers every leaf reachable from the reference package's installed `komorebi` parser: **46 commands, 201 non-help argument records and 51 verification-suite names**. There are **57 parser nodes**, including the root and command groups. The 201 count repeats shared export options at each consuming command. It is not a count of 201 distinct flag spellings. With each leaf's help action included, the ledger has 247 argument records.

The installed console command `komorebi` and `python -m komorebi_math` dispatch to the same CLI. The latter is used below to select the intended interpreter. The only root option is `-h` / `--help`; there is no global `--version`, `--json` or `--force`. Run `info` for the version. Every command and group supports help. A bare root or group requires a subcommand and exits with code 2.

Defaults and token rules. Not supplied means the Python default is `None`, not the string `None`. Boolean store-true flags default to false and consume no following value. The reference parser permits unambiguous long-option abbreviations and repeated scalar options take their last value; use full flags in maintained scripts. Parser type conversion is not mathematical domain validation. An accepted integer can still be rejected by the implementation.

Shell quoting. Quote comma-separated values and especially semicolon-separated matrix rows in both shells. Attach a leading-negative list to its option, for example `--values=-1,0,1`, because quoting alone does not stop argparse from treating a leading hyphen as an option. Commands shown below use one-line syntax valid in either POSIX or PowerShell. Native PowerShell execution is not claimed. Inspect `$LASTEXITCODE` in PowerShell and `$?` immediately after a POSIX command. Do not infer success from the presence of a JSON file.

Exit and failure contract. Help exits 0. Missing arguments, unknown options, conflicting tiers and invalid parser types/choices exit 2 with usage on stderr. Domain handlers typically return 0 for success and 1 for rejected validation or failed certificates. Some commands explicitly return 2 for semantic errors. There is no global exception-to-JSON adapter: uncaught errors can terminate with code 1 and a traceback. The individual records name their boundaries. Filesystem permissions and unexpected backend failures are not promised to have a uniform JSON schema.

Evidence classes. Parser reflection establishes syntax coverage. CLI replay establishes the recorded finite behavior. A certificate's exactness field still controls its mathematical scope. Caller-supplied completion gates remain assumptions. A successful expected-rejection test is never counted as successful platform qualification.

Traceability. `Block38_CLI_Inventory.json` contains the full parser tree, per-action metadata, mutually exclusive groups, handler line ranges and source hashes. `Block38_CLI_Records.json` adds semantics. `Block38_Runtime_Selectors.json` binds runtime aliases. `Block38_CLI_Output_Shapes.json` retains observed nested field/type structures, not a universal or closed JSON Schema for every possible result.

F.1 Local fixture preparation

Use a scratch directory outside the source tree. The following helper creates the small JSON, array, TeX and stream fixtures used by the examples, and refuses existing fixture paths unless explicitly forced. It also creates a deliberately corrupted stream for rejection tests. No network access is used.

```
python tools/block38/fixtures.py --output-dir cli-scratch
cd cli-scratch
```

Run the helper from the extracted bundle root, with the reference wheel installed in that interpreter. `input.json` contains columns x=[1,2] and y=[3,4]. `array.json` contains [1,2,3]. `small.tex` is a minimal local document. `complete.kmstream` is a four-coefficient finite quadratic stream. `partial.kmstream` contains its committed prefix. `bad.kmstream` deliberately fails chunk integrity. Commands with output paths assume those paths do not already exist.

Optional YAML export requires PyYAML. PDF commands require a working local LaTeX installation. NumPy binary export requires a suitable numeric representation: the reference NPY command uses `array.json`, not the column dictionary. Optional GPU checks cannot qualify CUDA on a CPU-only host. Run only trusted TeX and installed plugins: disabling shell escape or detecting an extension is not a general execution sandbox.

F.2 Runtime selectors and the verification dispatch matrix

`verify` accepts the following exact suite strings. The second column is the function selected by the reference dispatcher. The result is the recorded host exit, not an all-input proof. Several suites aggregate or repeat earlier suites, so check totals must not be summed as unique mathematics.

Suite	Dispatched function	Observed result
`core`	`run_core_verification`	exit 0
`refinement`	`run_refinement_verification`	exit 0
`nonlinear`	`run_nonlinear_verification`	exit 0
`block2`	`run_block2_verification`	exit 0
`algorithms`	`run_algorithms_verification`	exit 0
`block3`	`run_block3_verification`	exit 0
`signals`	`run_signals_verification`	exit 0
`dynamics`	`run_dynamics_verification`	exit 0
`block4`	`run_block4_verification`	exit 0
`analytic`	`run_analytic_verification`	exit 0
`quantum`	`run_quantum_verification`	exit 0
`block5`	`run_block5_verification`	exit 0
`arithmetic`	`run_arithmetic_verification`	exit 0
`sequences`	`run_sequences_verification`	exit 0
`block6`	`run_block6_verification`	exit 0
`deformation`	`run_deformation_verification`	exit 0
`langlands-local`	`run_langlands_local_verification`	exit 0
`langlands-finite-image`	`run_langlands_finite_image_verification`	exit 0
`langlands-completion`	`run_langlands_completion_verification`	exit 0
`langlands`	`run_langlands_verification`	exit 0
`block7`	`run_block7_verification`	exit 0
`export`	`run_export_verification`	exit 0
`block8`	`run_block8_verification`	exit 0
`block9`	`run_block9_verification`	exit 0
`hardening`	`run_scalable_nonlinear_verification`	exit 0
`block10`	`run_block10_verification`	exit 0
`streaming-beta`	`run_streaming_beta_verification`	exit 0
`block11`	`run_block11_verification`	exit 0
`dsp`	`run_dsp_verification`	exit 0
`block12`	`run_block12_verification`	exit 0
`generating`	`run_generating_verification`	exit 0
`block13`	`run_block13_verification`	exit 0
`coordinates`	`run_coordinates_verification`	exit 0
`block14`	`run_block14_verification`	exit 0
`block15`	`run_block15_verification`	exit 0
`stable`	`run_stable_verification`	exit 0
`release`	`run_release_qualification_verification`	exit 0
`gpu`	`run_gpu_verification`	exit 1; CUDA unqualified
`block16`	`run_block16_verification`	exit 0
`upgrade`	`run_upgrade_verification`	exit 0
`evidence`	`run_evidence_verification`	exit 0
`documentation`	`run_documentation_verification`	exit 0
`beta`	`run_beta_readiness_verification`	exit 0
`block17`	`run_block17_verification`	exit 0
`frame-algebra`	`run_frame_algebra_verification`	exit 0

Suite	Dispatched function	Observed result
`block18`	`run_block18_verification`	exit 0
`integration`	`run_integration_verification`	exit 0
`block19`	`run_block19_verification`	exit 0
`expression`	`run_expression_verification`	exit 0
`block20`	`run_block20_verification`	exit 0
`all`	`run_block17_verification`	exit 0

Reference aggregate-verification alias. `verify all` is exactly `run_block17_verification`. It does not select the source pytest suite or automatically include frame-algebra, integration, expression or their later aggregate extensions. To check those installed extensions explicitly, use:

```
python -m komorebi_math verify block18
python -m komorebi_math verify block19
python -m komorebi_math verify block20
```

Sequence names. `apery`, `bell`, `bernoulli`, `catalan`, `euler`, `fibonacci`, `lucas`, `motzkin`, `partition`, `thue-morse`, `zigzag`. Strip whitespace, convert to lowercase, replace underscores with hyphens, then apply the alias map. Aliases: `apery-zeta3` -> `apery`, `euler-zigzag` -> `zigzag`, `signed-euler` -> `euler`, `thue-morse` -> `thue-morse`, `thuemorse` -> `thue-morse`.

Polynomial families. `monomials`, `Euler polynomials`, `Fibonacci polynomials`, `falling factorials`, `rising factorials`, `binomial polynomials`, `Bernoulli polynomials`, `Lucas polynomials`, `Pell polynomials`, `Chebyshev T polynomials`, `Chebyshev U polynomials`, `Legendre polynomials`, `probabilists Hermite polynomials`, `physicists Hermite polynomials`, `Laguerre polynomials`. Convert to lowercase, retain only alphanumeric characters, then apply the alias map. Short aliases map to normalized catalog keys:

Alias	Normalized target
`bernoulli`	`bernoullipolynomials`
`binomial`	`binomialpolynomials`
`chebyshevt`	`chebyshevtpolynomials`
`chebyshevu`	`chebyshevupolynomials`
`euler`	`eulerpolynomials`
`falling`	`fallingfactorials`
`fibonacci`	`fibonaccipolynomials`
`hermitephysicists`	`physicistshermitepolynomials`
`hermiteprobabilists`	`probabilistshermitepolynomials`
`laguerre`	`laguerrepolynomials`
`legendre`	`legendrepolynomials`
`lucas`	`lucaspolynomials`
`monomial`	`monomials`
`pell`	`pellpolynomials`
`rising`	`risingfactorials`

Recurrence models. `Fibonacci`, `Lucas`, `Pell`, `Jacobsthal`, `Tribonacci`, `Padovan`. Convert to lowercase, retain only alphanumeric characters, and match the six catalog names.

Completion gate names. `residual`, `patching`, `comparison`, `fiber`, `realization`. `--satisfied` uses a comma-separated list. Supplying all five marks external flags as supplied, not proved. Unknown names return 2.

Export selectors. `json`, `yaml`, `csv`, `tsv`, `text`, `markdown`, `html`, `latex`, `pdf`, `npy`, `npz`. Explicit aliases: `md` -> `markdown`, `tex` -> `latex`, `txt` -> `text`, `yml` -> `yaml`.

Suffix	Inferred format
`.csv`	`csv`
`.htm`	`html`
`.html`	`html`
`.json`	`json`
`.markdown`	`markdown`
`.md`	`markdown`
`.npy`	`npy`
`.npz`	`npz`
`.pdf`	`pdf`
`.tex`	`latex`
`.tsv`	`tsv`
`.txt`	`text`
`.yaml`	`yaml`
`.yml`	`yaml`

F.3 Shared export option group

The following 16 options occur on `export sequence`, `export standard-window`, `export euler`, `export json` and `export bundle`. Each command's JSON record contains the full expanded list. The per-command tables below avoid printing these shared rows five times. Accepted options can have format-specific or handler-specific effects; an accepted flag is not necessarily consumed by every route.

Option	Type and requirement	Default	Accepted values
`--output`	str; required	`no default`	runtime domain; see semantics
`--format`	str; optional	`not supplied`	runtime domain; see semantics
`--title`	str; optional	`'Komorebi Math Export'`	runtime domain; see semantics
`--author`	str; optional	`not supplied`	runtime domain; see semantics
`--date`	str; optional	`not supplied`	runtime domain; see semantics
`--abstract`	str; optional	`not supplied`	runtime domain; see semantics
`--document-class`	str; optional	`'article'`	runtime domain; see semantics
`--toc`	switch; optional	`False`	on/off flag
`--mathjax`	switch; optional	`False`	on/off flag
`--engine`	str; optional	`'auto'`	auto, pdflatex, xelatex, lualatex, latexmk
`--passes`	int; optional	`2`	runtime domain; see semantics
`--timeout`	float; optional	`120.0`	runtime domain; see semantics
`--keep-auxiliary`	switch; optional	`False`	on/off flag
`--no-keep-tex`	switch; optional	`False`	on/off flag
`--no-type-metadata`	switch; optional	`False`	on/off flag
`--force`	switch; optional	`False`	on/off flag

`--output`. Required destination path. Parent creation and existence checks are handled by the export writer. Use a fresh path unless replacement is intended.

`--format`. Runtime format name or alias; None means infer from extension. On export bundle this singular option is unused; use –formats.

`--title`. Document title for generic data/document construction. export sequence constructs its own sequence title.

`--author`. Optional author passed to the document constructor. It is metadata, not an authenticated signature.

`--date`. Optional displayed document date, not a build timestamp attestation. Not propagated by the export sequence handler.

`--abstract`. Optional document abstract. Not propagated by the export sequence handler.

`--document-class`. LaTeX document class string, validated by LatexRenderOptions. Mainly relevant to LaTeX/PDF.

`--toc`. Request a LaTeX table of contents. No effect on non-LaTeX formats.

`--mathjax`. Include the HTML MathJax loader. Rendering that HTML can require a network-accessible script; CLI export itself writes local text.

`--engine`. Select a PDF compiler; auto resolves an available engine. Merely parsing the option does not verify a compiler exists.

`--passes`. Positive integer direct-compiler pass count; compiler options are constructed even when exporting another format.

`--timeout`. Positive per-compiler-process timeout in seconds. Does not cap sequence generation, serialization or the entire CLI run.

`--keep-auxiliary`. Keep LaTeX auxiliary files after a successful render.

`--no-keep-tex`. Do not retain the TeX companion when the exporter uses that option for PDF export.

`--no-type-metadata`. Set preserve_types=False in serialization options. This can change encoded structure and is not a portable round-trip guarantee.

`--force`. Permit replacement of an existing export destination. It does not supply automatic rollback or guarantee atomic updates.

A LaTeX document-class name must match `[A-Za-z0-9_.-]+`. Compiler passes and timeout must be positive. The CLI constructs render/compiler option objects even for non-PDF exports, so invalid option values can fail before a format-specific writer is reached. Explicit export formats override suffix inference. A requested output file stores the rendered payload, whereas stdout normally stores an export acknowledgment. `--no-type-metadata` affects the exported representation, not necessarily the acknowledgment printed by the CLI.

Overwrite caution. Most export destinations and optional release-report outputs refuse an existing path without `--force`. Companion files and multi-step operations are not a general atomic transaction. `latex compile` is the important exception: it has no force guard and can overwrite existing PDF/log artifacts. Use a new directory for repeatable runs.

F.4 Output contracts and schema boundaries

JSON records below name all emitted top-level keys seen in the bounded fixtures and give their types. Conditional fields are explained in the command-specific contract. Nested structures and variant observations are retained in the machine-readable output-shape ledger. Those observations are examples of the versioned Python serializers, not a replacement for the portable codec contract in Appendix G. Environment maps, diagnostics, source paths and residuals can vary by run.

The common certificate object has `valid:boolean`, `exactness:string`, `assumptions_checked:array`, `assumptions_unchecked:array`, `diagnostics:object`, `message:string`, `residuals:object`, `theorem_id:string or null` and `tolerance:number or null`. A certificate may be the whole stdout object or a nested field. Qualification uses `certificate.valid`, and its stdout has no top-level `passed` field. Export and LaTeX result serializers can include `__type__`. Rational values may be strings; spectral complex numbers use `{real:number, imag:number}`.

The report path acknowledgment from `api snapshot` is not the manifest itself. Similarly, export stdout is not the exported data. `sequence --index` switches to a scalar line rather than a prefix JSON object. A failed command may

emit a valid rejection report, only stderr, or a traceback. Consumers must inspect exit status and the appropriate report field together.

F.5 Complete command records

All records include handler location in `src/komorebi_math/cli/main.py`, exact argument defaults, output behavior, overwrite policy and a minimum fixture invocation. Commands that have no mathematical inputs still have records. The common `-h` / `--help` action is available on every record and is documented once in F.0.

F.5.1 `info`

Record CLI-F-001. Prints the installed distribution version and a textual capability summary.

Syntax. Parser-generated usage reflowed for print, with optional groups in square brackets. Exact help text is retained in the inventory.

```
usage: komorebi info [-h]
```

No positional arguments or command-specific options beyond help.

Output and exit contract. Human-readable text, not a stable JSON object.

Semantic boundary. The phrase implemented blocks is explanatory metadata, not a build or proof audit.

File effects and replacement. No output-file option. This command does not intentionally overwrite user data; stdout can be redirected by the shell.

Minimum invocation (POSIX and PowerShell). The same one-line command is valid in either shell; run from the scratch fixture directory with a fresh output destination.

```
python -m komorebi_math info
```

Observed reference result. Exit 0. The bounded invocation completed successfully. 1 associated replay case is retained, including shared-suite selections or variants where applicable.

Observed text excerpt. Only the first five lines are shown.

```
komorebi-math 1.0.0b4
language: seed -> address -> operate -> project -> transfer -> refine -> constrain -> complete -> reconstruct
implemented blocks: finite dimensional core through Block 17, production finite DSP, plus research-tier Blocks
   ↪ 18-20 for frame algebra, finite integration, and typed exact expressions
```

Failure handling. All parser help requests exit 0. Missing/unknown/conflicting arguments and invalid parser choices/types exit 2 with usage on stderr. No global CLI exception normalizer exists: uncaught implementation/-filesystem errors normally terminate the Python process with code 1 and a traceback. Command-specific 0/1/2 behavior is described above. Handler-local caught exception types: none explicitly caught.

Source binding. `src/komorebi_math/cli/main.py`, lines 80-84, handler `command_info`. Handler SHA-256: `48e1f485 7b08efac42af0fdf96cc1c296d8d39a32a5e01355e7ee52c220c1ac0`.

F.5.2 `doctor`

Record CLI-F-002. Inspects available backends, optional import specifications, LaTeX executables and extension entry points.

Syntax. Parser-generated usage reflowed for print, with optional groups in square brackets. Exact help text is retained in the inventory.

```
usage: komorebi doctor [-h]
```

No positional arguments or command-specific options beyond help.

Output and exit contract. Human-readable environment listing. Returns 0 when all discovered extensions load, otherwise 1.

Semantic boundary. Available or installed means detection, not numerical/device qualification. Extension discovery can import third-party plugins.

File effects and replacement. No output-file option. Plugin discovery can run third-party extension import code; no general no-side-effects promise is made for installed plugins.

Minimum invocation (POSIX and PowerShell). The same one-line command is valid in either shell; run from the scratch fixture directory with a fresh output destination.

```
python -m komorebi_math doctor
```

Observed reference result. Exit 0. The bounded invocation completed successfully. 1 associated replay case is retained, including shared-suite selections or variants where applicable.

Observed text excerpt. Only the first five lines are shown.

```
Python: 3.13.5
Platform: Linux-6.18.35-x86_64-with-glibc2.41
Backends:
  numpy    available
  sympy    available
```

Failure handling. All parser help requests exit 0. Missing/unknown/conflicting arguments and invalid parser choices/types exit 2 with usage on stderr. No global CLI exception normalizer exists: uncaught implementation/-filesystem errors normally terminate the Python process with code 1 and a traceback. Command-specific 0/1/2 behavior is described above. Handler-local caught exception types: none explicitly caught.

Source binding. `src/komorebi_math/cli/main.py`, lines 87-106, handler `command_doctor`. Handler SHA-256: `4b7395aa701ef6c667695555c1c70a437ba027b7d58c8e959d2a96f8630f5544`.

F.5.3 features

Record CLI-F-003. Prints registered feature keys, statuses and summaries.

Syntax. Parser-generated usage reflowed for print, with optional groups in square brackets. Exact help text is retained in the inventory.

```
usage: komorebi features [-h]
```

No positional arguments or command-specific options beyond help.

Output and exit contract. Aligned text rows: feature key, status value and summary.

Semantic boundary. Planned and implemented status labels are retained. Listing is not a runtime test.

File effects and replacement. No output-file option. This command does not intentionally overwrite user data; stdout can be redirected by the shell.

Minimum invocation (POSIX and PowerShell). The same one-line command is valid in either shell; run from the scratch fixture directory with a fresh output destination.

```
python -m komorebi_math features
```

Observed reference result. Exit 0. The bounded invocation completed successfully. 1 associated replay case is retained, including shared-suite selections or variants where applicable.

Observed text excerpt. Only the first five lines are shown.

```
algorithms              implemented               Packet IR, structured operators, low rank algebra, support
    ↪ propagation, Krylov methods, precision planning, exact tree compilation, scheduling, restricted discovery,
    ↪ and optional CuPy execution.
analytic                implemented               Cyclic projections, transfer and signed sections, reciprocal and
    ↪ Appell families, numerical zero strings, Hadamard products, and zeta diagnostics.
arithmetic              implemented               Modular domains, Hurwitz modes, standard windows, odd and dyadic
    ↪ Euler coordinates, Mahler transfer, polyphase refinement, local unit gains, and recurrence packets.
backends                partial                    NumPy and optional exact SymPy backends are implemented. A CuPy
    ↪ packet executor is available when CuPy is installed.
coordinates             implemented               Arbitrary exact bases and atlases, checked finite
    ↪ analysis--synthesis systems, normalized-series changes, recurrence defects, companion and state-space
    ↪ coordinates, classical polynomial-family connections, combinatorial transforms, affine and formal charts,
    ↪ tensor and operator transport, finite-field maps, spectral transforms, packet permutations, trusted
    ↪ persistence, and named bridges such as Euler-to-Fibonacci.
```

Failure handling. All parser help requests exit 0. Missing/unknown/conflicting arguments and invalid parser choices/types exit 2 with usage on stderr. No global CLI exception normalizer exists: uncaught implementation/-filesystem errors normally terminate the Python process with code 1 and a traceback. Command-specific 0/1/2 behavior is described above. Handler-local caught exception types: none explicitly caught.

Source binding. `src/komorebi_math/cli/main.py`, lines 109-111, handler `command_features`. Handler SHA-256: `6c054eaab8e6f1fe796725a05601870dd0d156f1ad8d9d24e06a03e1bc6befd0`.

F.5.4 `verify`

Record CLI-F-004. Invokes exactly one installed verification-suite function and prints each check plus a summary.

Syntax. Parser-generated usage reflowed for print, with optional groups in square brackets. Exact help text is retained in the inventory.

```
usage: komorebi verify [-h]
    {core, refinement, nonlinear, block2, algorithms, block3, signals, dynamics, block4,
    analytic, quantum, block5, arithmetic, sequences, block6, deformation,
    langlands-local, langlands-finite-image, langlands-completion, langlands, block7,
    export, block8, block9, hardening, block10, streaming-beta, block11, dsp, block12,
    generating, block13, coordinates, block14, block15, stable, release, gpu, block16,
    upgrade, evidence, documentation, beta, block17, frame-algebra, block18, integration,
    block19, expression, block20, all}
```

Argument	Type and requirement	Default	Accepted values
`suite`	str; required	`no default`	51 exact suite names; see F.2

Output and exit contract. Text lines [PASS] or [FAIL], optional detail, then the certificate message. 0 means the selected report passed, 1 means it failed.

Semantic boundary. The all choice dispatches to run_block17_verification, not the full pytest suite or the later redundant-frame, finite-integration, and typed-expression aggregate suites. Suite names are listed in F.2.

File effects and replacement. No persistent output-file option. Selected suites may use temporary files or optional backend probes.

Minimum invocation (POSIX and PowerShell). The same one-line command is valid in either shell; run from the scratch fixture directory with a fresh output destination.

```
python -m komorebi_math verify core
```

Observed reference result. Exit 0. The bounded invocation completed successfully. 52 associated replay cases are retained, including shared-suite selections or variants where applicable.

Observed text excerpt. Only the first five lines are shown.

```
[PASS] typed grammar relations
[PASS] finite seed hull
[PASS] packet analysis and synthesis
[PASS] packet operator reconstruction
[PASS] packet path expansion
```

Failure handling. All parser help requests exit 0. Missing/unknown/conflicting arguments and invalid parser choices/types exit 2 with usage on stderr. No global CLI exception normalizer exists: uncaught implementation/-filesystem errors normally terminate the Python process with code 1 and a traceback. Command-specific 0/1/2 behavior is described above. Handler-local caught exception types: `KeyError`.

Source binding. `src/komorebi_math/cli/main.py`, lines 114-178, handler `command_verify`. Handler SHA-256: `64ca9c5c0367b49a2748ab739ba90389b5d93ee5064e4ecf8bdbcde3524843f8`.

F.5.5 `api check`

Record CLI-F-005. Compares a supplied or committed manifest against the installed research, beta or stable API.

Syntax. Parser-generated usage reflowed for print, with optional groups in square brackets. Exact help text is retained in the inventory.

```
usage: komorebi api check [-h] [--manifest MANIFEST] [--strict-signatures]
    [--beta | --stable]
```

Argument	Type and requirement	Default	Accepted values
`--manifest`	str; optional	`not supplied`	runtime domain; see semantics
`--strict-signatures`	switch; optional	`False`	on/off flag
`--beta`	switch; optional	`False`	on/off flag
`--stable`	switch; optional	`False`	on/off flag

Mutually exclusive flags. `--beta` and `--stable` cannot be supplied together.

Output and exit contract. API compatibility JSON report including certificate, compatible, added, removed, kind_changed, signature_changed, reference_digest, current_digest and strict_signatures. 0 means compatible, 1 means incompatible or handled validation failure.

Semantic boundary. Default tier is research. –beta and –stable are mutually exclusive. –strict-signatures affects the research route; beta/stable use their own strict policies. –manifest is an input, never an output.

File effects and replacement. No output-file option. This command does not intentionally overwrite user data; stdout can be redirected by the shell.

Minimum invocation (POSIX and PowerShell). The same one-line command is valid in either shell; run from the scratch fixture directory with a fresh output destination.

```
python -m komorebi_math api check
```

Observed reference result. Exit 0. The bounded invocation completed successfully. 4 associated replay cases are retained, including shared-suite selections or variants where applicable.

Reference stdout field/type matrix.

Field	Type at this invocation
`added`	array (empty in this fixture)
`certificate`	object
`compatible`	boolean
`current_digest`	string
`kind_changed`	array (empty in this fixture)
`reference_digest`	string
`removed`	array (empty in this fixture)
`signature_changed`	array (empty in this fixture)
`strict_signatures`	boolean

Failure handling. All parser help requests exit 0. Missing/unknown/conflicting arguments and invalid parser choices/types exit 2 with usage on stderr. No global CLI exception normalizer exists: uncaught implementation/-filesystem errors normally terminate the Python process with code 1 and a traceback. Command-specific 0/1/2 behavior is described above. Handler-local caught exception types: `ValidationError`.

Source binding. `src/komorebi_math/cli/main.py`, lines 181-200, handler `command_api_check`. Handler SHA-256: `174af0d3e1428da632539550eaf1b7761028e8b47393353bf91a1a826202496a`.

F.5.6 `api snapshot`

Record CLI-F-006. Creates a current API manifest for the selected tier and writes it to –output.

Syntax. Parser-generated usage reflowed for print, with optional groups in square brackets. Exact help text is retained in the inventory.

```
usage: komorebi api snapshot [-h] --output OUTPUT [--force] [--beta |
    --stable]
```

Argument	Type and requirement	Default	Accepted values
`--output`	str; required	`no default`	runtime domain; see semantics
`--force`	switch; optional	`False`	on/off flag
`--beta`	switch; optional	`False`	on/off flag
`--stable`	switch; optional	`False`	on/off flag

Mutually exclusive flags. `--beta` and `--stable` cannot be supplied together.

Output and exit contract. JSON acknowledgment with path:string and digest:string. The file contains the API manifest, not this acknowledgment.

Semantic boundary. Default tier is research. –beta and –stable cannot be combined. Refuses an existing path unless –force is present.

File effects and replacement. No destination is written unless the command has a required output or –output is supplied. Existing destination is rejected unless –force. Failure can leave partial artifacts; transactional rollback is not promised.

Minimum invocation (POSIX and PowerShell). The same one-line command is valid in either shell; run from the scratch fixture directory with a fresh output destination.

```
python -m komorebi_math api snapshot --output api.json
```

Observed reference result. Exit 0. The bounded invocation completed successfully. 5 associated replay cases are retained, including shared-suite selections or variants where applicable.

Reference stdout field/type matrix.

Field	Type at this invocation
`digest`	string
`path`	string

Failure handling. All parser help requests exit 0. Missing/unknown/conflicting arguments and invalid parser choices/types exit 2 with usage on stderr. No global CLI exception normalizer exists: uncaught implementation/-filesystem errors normally terminate the Python process with code 1 and a traceback. Command-specific 0/1/2 behavior is described above. Handler-local caught exception types: `ValidationError`.

Source binding. `src/komorebi_math/cli/main.py`, lines 203-225, handler `command_api_snapshot`. Handler SHA-256: `3e22829fdb745d15031c840ef3f3514c4506375ca872b47ae06f89fd61087f02`.

F.5.7 `golden`

Record CLI-F-007. Executes the committed finite behavioral fixture.

Syntax. Parser-generated usage reflowed for print, with optional groups in square brackets. Exact help text is retained in the inventory.

```
usage: komorebi golden [-h]
```

No positional arguments or command-specific options beyond help.

Output and exit contract. JSON object containing cases/results and certificate. 0 means all fixture checks passed, otherwise 1.

Semantic boundary. A finite behavioral fixture is not a claim that all inputs or all theorems are verified.

File effects and replacement. No output-file option. This command does not intentionally overwrite user data; stdout can be redirected by the shell.

Minimum invocation (POSIX and PowerShell). The same one-line command is valid in either shell; run from the scratch fixture directory with a fresh output destination.

```
python -m komorebi_math golden
```

Observed reference result. Exit 0. The bounded invocation completed successfully. 1 associated replay case is retained, including shared-suite selections or variants where applicable.

Reference stdout field/type matrix.

Field	Type at this invocation
`certificate`	object
`fixture_digest`	string
`fixture_path`	string
`results`	array[object

Field	Type at this invocation

Failure handling. All parser help requests exit 0. Missing/unknown/conflicting arguments and invalid parser choices/types exit 2 with usage on stderr. No global CLI exception normalizer exists: uncaught implementation/-filesystem errors normally terminate the Python process with code 1 and a traceback. Command-specific 0/1/2 behavior is described above. Handler-local caught exception types: `ValidationError`.

Source binding. `src/komorebi_math/cli/main.py`, lines 228-239, handler `command_golden`. Handler SHA-256: `59c88298c63c8fe9d336046071b896dd01708f27172c58cb733a7023aa2621e7`.

F.5.8 `release contract`

Record CLI-F-008. Checks the committed current-beta release contract.

Syntax. Parser-generated usage reflowed for print, with optional groups in square brackets. Exact help text is retained in the inventory.

```
usage: komorebi release contract [-h]
```

No positional arguments or command-specific options beyond help.

Output and exit contract. JSON release report with checks, passed and certificate. 0/1 follows report.passed.

Semantic boundary. Checks an installed frozen contract. It does not certify the outer documentation ZIP or its editorial edition.

File effects and replacement. No output-file option. This command does not intentionally overwrite user data; stdout can be redirected by the shell.

Minimum invocation (POSIX and PowerShell). The same one-line command is valid in either shell; run from the scratch fixture directory with a fresh output destination.

```
python -m komorebi_math release contract
```

Observed reference result. Exit 0. The bounded invocation completed successfully. 1 associated replay case is retained, including shared-suite selections or variants where applicable.

Reference stdout field/type matrix.

Field	Type at this invocation
`certificate`	object
`checks`	array[object]
`contract`	object
`contract_digest`	string
`passed`	boolean
`runtime`	object

Failure handling. All parser help requests exit 0. Missing/unknown/conflicting arguments and invalid parser choices/types exit 2 with usage on stderr. No global CLI exception normalizer exists: uncaught implementation/-filesystem errors normally terminate the Python process with code 1 and a traceback. Command-specific 0/1/2 behavior is described above. Handler-local caught exception types: none explicitly caught.

Source binding. `src/komorebi_math/cli/main.py`, lines 242-249, handler `command_release_contract`. Handler SHA-256: `7def743048b70cf34a9264d94d8e1f0a72ba1471abca4c3b66e7bb772142c0e8`.

F.5.9 `release environment`

Record CLI-F-009. Prints the current finite runtime record.

Syntax. Parser-generated usage reflowed for print, with optional groups in square brackets. Exact help text is retained in the inventory.

```
usage: komorebi release environment [-h]
```

No positional arguments or command-specific options beyond help.

Output and exit contract. JSON environment object. Recorded keys and nested types are retained in the output-shape ledger.

Semantic boundary. A runtime record is descriptive; exit 0 does not mean all optional packages, platforms or hardware passed qualification.

File effects and replacement. No output-file option. This command does not intentionally overwrite user data; stdout can be redirected by the shell.

Minimum invocation (POSIX and PowerShell). The same one-line command is valid in either shell; run from the scratch fixture directory with a fresh output destination.

```
python -m komorebi_math release environment
```

Observed reference result. Exit 0. The bounded invocation completed successfully. 1 associated replay case is retained, including shared-suite selections or variants where applicable.

Reference stdout field/type matrix.

Field	Type at this invocation
`environment`	object
`package_version`	string
`platform`	string
`python`	string
`python_executable`	string

Failure handling. All parser help requests exit 0. Missing/unknown/conflicting arguments and invalid parser choices/types exit 2 with usage on stderr. No global CLI exception normalizer exists: uncaught implementation/-filesystem errors normally terminate the Python process with code 1 and a traceback. Command-specific 0/1/2 behavior is described above. Handler-local caught exception types: none explicitly caught.

Source binding. `src/komorebi_math/cli/main.py`, lines 252-258, handler `command_release_environment`. Handler SHA-256: `482c880da9c20aa759ce7013e2860b02569a20966a3b0dd83d4edb411d7cb64e`.

F.5.10 `release qualify`

Record CLI-F-010. Runs installed freeze-candidate qualification; optionally writes the full report.

Syntax. Parser-generated usage reflowed for print, with optional groups in square brackets. Exact help text is retained in the inventory.

```
usage: komorebi release qualify [-h] [--gpu] [--require-gpu] [--output OUTPUT]
    [--force]
```

Argument	Type and requirement	Default	Accepted values
`--gpu`	switch; optional	`False`	on/off flag
`--require-gpu`	switch; optional	`False`	on/off flag

Argument	Type and requirement	Default	Accepted values
`--output`	str; optional	`not supplied`	runtime domain; see semantics
`--force`	switch; optional	`False`	on/off flag

Output and exit contract. JSON qualification report with checks, gpu, gpu_required, optional_backends, runtime, schema, schema_version, version and certificate. There is no top-level passed field; use certificate.valid. The same full report is written when –output is set.

Semantic boundary. –require-gpu implies a GPU probe and requires success; –gpu alone probes opportunistically. Output overwrite rejection is raised by report.write and is not caught at the CLI boundary.

File effects and replacement. No destination is written unless the command has a required output or –output is supplied. Existing destination is rejected unless –force. Failure can leave partial artifacts; transactional rollback is not promised.

Minimum invocation (POSIX and PowerShell). The same one-line command is valid in either shell; run from the scratch fixture directory with a fresh output destination.

```
python -m komorebi_math release qualify
```

Observed reference result. Exit 0. The bounded invocation completed successfully. 4 associated replay cases are retained, including shared-suite selections or variants where applicable.

Reference stdout field/type matrix.

Field	Type at this invocation
`certificate`	object
`checks`	array[object]
`gpu`	null
`gpu_required`	boolean
`optional_backends`	object
`runtime`	object
`schema`	string
`schema_version`	integer
`version`	string

Failure handling. All parser help requests exit 0. Missing/unknown/conflicting arguments and invalid parser choices/types exit 2 with usage on stderr. No global CLI exception normalizer exists: uncaught implementation/-filesystem errors normally terminate the Python process with code 1 and a traceback. Command-specific 0/1/2 behavior is described above. Handler-local caught exception types: none explicitly caught.

Source binding. `src/komorebi_math/cli/main.py`, lines 261-273, handler `command_release_qualify`. Handler SHA-256: `d08d543517da3278c8561dc6be69e9415defabde14a0c9caaaac9c52aac98faa`.

F.5.11 `release upgrade`

Record CLI-F-011. Checks predecessor API, semantic, codec and portable-record upgrade contracts.

Syntax. Parser-generated usage reflowed for print, with optional groups in square brackets. Exact help text is retained in the inventory.

```
usage: komorebi release upgrade [-h]
```

No positional arguments or command-specific options beyond help.

Output and exit contract. JSON report with checks, passed and certificate. Returns 0/1 according to report.passed.

Semantic boundary. Only the committed predecessor contracts and records are covered. This is not an automatic source or installed-package upgrade.

File effects and replacement. No output-file option. This command does not intentionally overwrite user data; stdout can be redirected by the shell.

Minimum invocation (POSIX and PowerShell). The same one-line command is valid in either shell; run from the scratch fixture directory with a fresh output destination.

```
python -m komorebi_math release upgrade
```

Observed reference result. Exit 0. The bounded invocation completed successfully. 1 associated replay case is retained, including shared-suite selections or variants where applicable.

Reference stdout field/type matrix.

Field	Type at this invocation
`certificate`	object
`checks`	array[object]
`contract`	object
`contract_digest`	string
`loaded_fixtures`	array[string]
`passed`	boolean

Failure handling. All parser help requests exit 0. Missing/unknown/conflicting arguments and invalid parser choices/types exit 2 with usage on stderr. No global CLI exception normalizer exists: uncaught implementation/-filesystem errors normally terminate the Python process with code 1 and a traceback. Command-specific 0/1/2 behavior is described above. Handler-local caught exception types: none explicitly caught.

Source binding. `src/komorebi_math/cli/main.py`, lines 292-299, handler `command_release_upgrade`. Handler SHA-256: `3df02aa760167edb3b3c0c4a1d93e45b2c9c2d9affa697f3dd5a3a910c7d7332`.

F.5.12 `release evidence`

Record CLI-F-012. Checks digest-bound platform evidence and can write a copy of the evidence report.

Syntax. Parser-generated usage reflowed for print, with optional groups in square brackets. Exact help text is retained in the inventory.

```
usage: komorebi release evidence [-h] [--current] [--output OUTPUT] [--force]
```

Argument	Type and requirement	Default	Accepted values
`--current`	switch; optional	`False`	on/off flag
`--output`	str; optional	`not supplied`	runtime domain; see semantics
`--force`	switch; optional	`False`	on/off flag

Output and exit contract. JSON platform-evidence report with passed, certificate and coverage/missing-platform fields. A handled output-path rejection prints a diagnostic to stderr and returns 1.

Semantic boundary. –current requires current-version evidence across every declared platform family. Baseline evidence passing does not imply –current passes.

File effects and replacement. No destination is written unless the command has a required output or –output is supplied. Existing destination is rejected unless –force. Failure can leave partial artifacts; transactional rollback is not promised.

Minimum invocation (POSIX and PowerShell). The same one-line command is valid in either shell; run from the scratch fixture directory with a fresh output destination.

```
python -m komorebi_math release evidence
```

Observed reference result. Exit 0. The bounded invocation completed successfully. 4 associated replay cases are retained, including shared-suite selections or variants where applicable.

Reference stdout field/type matrix.

Field	Type at this invocation
`certificate`	object
`covered_platforms`	array[string]
`current_platforms`	array[string]
`evidence`	object
`evidence_digest`	string
`inherited_records`	array[string]
`missing_current_platforms`	array[string]
`missing_platforms`	array (empty in this fixture)
`passed`	boolean
`release_candidate_ready`	boolean
`require_current_all_platforms`	boolean

Failure handling. All parser help requests exit 0. Missing/unknown/conflicting arguments and invalid parser choices/types exit 2 with usage on stderr. No global CLI exception normalizer exists: uncaught implementation/-filesystem errors normally terminate the Python process with code 1 and a traceback. Command-specific 0/1/2 behavior is described above. Handler-local caught exception types: `ValidationError`.

Source binding. `src/komorebi_math/cli/main.py`, lines 302-317, handler `command_release_evidence`. Handler SHA-256: `bd6a100a37964c562fb5ec28a25e6aaf74ef2f8027c4626d6b984e35fb681029`.

F.5.13 `release documentation`

Record CLI-F-013. Checks exact stable-namespace documentation coverage in the installed package.

Syntax. Parser-generated usage reflowed for print, with optional groups in square brackets. Exact help text is retained in the inventory.

```
usage: komorebi release documentation [-h]
```

No positional arguments or command-specific options beyond help.

Output and exit contract. JSON documentation report with changed, deprecated, extra, missing, current_digest, reference_digest, passed and certificate. Returns 0/1 by report.passed.

Semantic boundary. This is the stable package documentation gate, not a claim that the full manual, its supplementary crosswalks, or a strict MkDocs build is complete.

File effects and replacement. No output-file option. This command does not intentionally overwrite user data; stdout can be redirected by the shell.

Minimum invocation (POSIX and PowerShell). The same one-line command is valid in either shell; run from the scratch fixture directory with a fresh output destination.

```
python -m komorebi_math release documentation
```

Observed reference result. Exit 0. The bounded invocation completed successfully. 1 associated replay case is retained, including shared-suite selections or variants where applicable.

Reference stdout field/type matrix.

Field	Type at this invocation
`certificate`	object
`changed`	array (empty in this fixture)
`current_digest`	string
`deprecated`	array (empty in this fixture)
`extra`	array (empty in this fixture)
`missing`	array (empty in this fixture)
`passed`	boolean
`reference_digest`	string

Failure handling. All parser help requests exit 0. Missing/unknown/conflicting arguments and invalid parser choices/types exit 2 with usage on stderr. No global CLI exception normalizer exists: uncaught implementation/-filesystem errors normally terminate the Python process with code 1 and a traceback. Command-specific 0/1/2 behavior is described above. Handler-local caught exception types: none explicitly caught.

Source binding. `src/komorebi_math/cli/main.py`, lines 320-327, handler `command_release_documentation`. Handler SHA-256: `65935e554a078b3237618e38da25f6683127a1d1c17adbeb6763cc3fc702cd1e`.

F.5.14 `release readiness`

Record CLI-F-014. Evaluates public-beta readiness or the stricter final/current-platform gate.

Syntax. Parser-generated usage reflowed for print, with optional groups in square brackets. Exact help text is retained in the inventory.

```
usage: komorebi release readiness [-h] [--final] [--gpu] [--require-gpu]
    [--output OUTPUT] [--force]
```

Argument	Type and requirement	Default	Accepted values
`--final`	switch; optional	`False`	on/off flag
`--gpu`	switch; optional	`False`	on/off flag
`--require-gpu`	switch; optional	`False`	on/off flag
`--output`	str; optional	`not supplied`	runtime domain; see semantics
`--force`	switch; optional	`False`	on/off flag

Output and exit contract. JSON readiness report with checks, passed and certificate, optionally copied to –output.

Semantic boundary. –final requires current-version platform evidence. –require-gpu implies –gpu and requires the hardware gate. A passing baseline is not a final-platform qualification.

File effects and replacement. No destination is written unless the command has a required output or –output is supplied. Existing destination is rejected unless –force. Failure can leave partial artifacts; transactional rollback is not promised.

Minimum invocation (POSIX and PowerShell). The same one-line command is valid in either shell; run from the scratch fixture directory with a fresh output destination.

```
python -m komorebi_math release readiness
```

Observed reference result. Exit 0. The bounded invocation completed successfully. 4 associated replay cases are retained, including shared-suite selections or variants where applicable.

Reference stdout field/type matrix.

Field	Type at this invocation
`certificate`	object
`checks`	array[object]
`gpu`	null
`mode`	string
`passed`	boolean
`runtime`	object
`version`	string

Failure handling. All parser help requests exit 0. Missing/unknown/conflicting arguments and invalid parser choices/types exit 2 with usage on stderr. No global CLI exception normalizer exists: uncaught implementation/-filesystem errors normally terminate the Python process with code 1 and a traceback. Command-specific 0/1/2 behavior is described above. Handler-local caught exception types: `ValidationError`.

Source binding. `src/komorebi_math/cli/main.py`, lines 330-347, handler `command_release_readiness`. Handler SHA-256: `8313a6c12b50403c4cd9b4e96606ccb5733f34534349ec5262e441f78e4392fc`.

F.5.15 `stream inspect`

Record CLI-F-015. Reads a packet-jet stream directory and emits its manifest.

Syntax. Parser-generated usage reflowed for print, with optional groups in square brackets. Exact help text is retained in the inventory.

```
usage: komorebi stream inspect [-h] [--allow-incomplete] [--verify-chunks]
    path
```

Argument	Type and requirement	Default	Accepted values
`path`	str; required	`no default`	runtime domain; see semantics
`--allow-incomplete`	switch; optional	`False`	on/off flag
`--verify-chunks`	switch; optional	`False`	on/off flag

Output and exit contract. JSON manifest, including state, completed coefficient count, chunks and stream digest. A rejected stream prints stderr and returns 1.

Semantic boundary. –allow-incomplete permits opening a partial stream; it does not declare completion. –verify-chunks adds chunk validation before printing. Inspection does not write or repair the stream.

File effects and replacement. No output-file option. This command does not intentionally overwrite user data; stdout can be redirected by the shell.

Minimum invocation (POSIX and PowerShell). The same one-line command is valid in either shell; run from the scratch fixture directory with a fresh output destination.

```
python -m komorebi_math stream inspect complete.kmstream
```

Observed reference result. Exit 0. The bounded invocation completed successfully. 5 associated replay cases are retained, including shared-suite selections or variants where applicable.

Reference stdout field/type matrix.

Field	Type at this invocation
`capture_certificate`	object
`chunk_size`	integer
`chunks`	array[object]
`coefficients_completed`	integer
`evaluations_completed`	integer
`maximum_omitted_basis_norm`	number
`metadata`	object
`metadata_sha256`	string
`producer_version`	string
`state`	string
`stream_sha256`	string

Failure handling. All parser help requests exit 0. Missing/unknown/conflicting arguments and invalid parser choices/types exit 2 with usage on stderr. No global CLI exception normalizer exists: uncaught implementation/-filesystem errors normally terminate the Python process with code 1 and a traceback. Command-specific 0/1/2 behavior is described above. Handler-local caught exception types: `ValidationError`.

Source binding. `src/komorebi_math/cli/main.py`, lines 350-365, handler `command_stream_inspect`. Handler SHA-256: `aedc002a5cf0894af36728376237503c5627925be7143fbae437c84d56000397`.

F.5.16 `stream verify`

Record CLI-F-016. Opens a packet-jet stream and verifies every committed chunk.

Syntax. Parser-generated usage reflowed for print, with optional groups in square brackets. Exact help text is retained in the inventory.

```
usage: komorebi stream verify [-h] [--allow-incomplete] path
```

Argument	Type and requirement	Default	Accepted values
`path`	str; required	`no default`	runtime domain; see semantics
`--allow-incomplete`	switch; optional	`False`	on/off flag

Output and exit contract. Certificate JSON. Exit 0/1 follows certificate.valid or a handled stream validation error.

Semantic boundary. –allow-incomplete permits partial streams, so a valid committed prefix is not a completed full stream. No repair or write is performed.

File effects and replacement. No output-file option. This command does not intentionally overwrite user data; stdout can be redirected by the shell.

Minimum invocation (POSIX and PowerShell). The same one-line command is valid in either shell; run from the scratch fixture directory with a fresh output destination.

```
python -m komorebi_math stream verify complete.kmstream
```

Observed reference result. Exit 0. The bounded invocation completed successfully. 3 associated replay cases are retained, including shared-suite selections or variants where applicable.

Reference stdout field/type matrix.

Field	Type at this invocation
`assumptions_checked`	array[string]
`assumptions_unchecked`	array[string]
`diagnostics`	object
`exactness`	string
`message`	string
`residuals`	object
`theorem_id`	string
`tolerance`	null
`valid`	boolean

Failure handling. All parser help requests exit 0. Missing/unknown/conflicting arguments and invalid parser choices/types exit 2 with usage on stderr. No global CLI exception normalizer exists: uncaught implementation/-filesystem errors normally terminate the Python process with code 1 and a traceback. Command-specific 0/1/2 behavior is described above. Handler-local caught exception types: `ValidationError`.

Source binding. `src/komorebi_math/cli/main.py`, lines 368-383, handler `command_stream_verify`. Handler SHA-256: `4b4b90a6a920fedfce4799a420885f4702218d56772aa719ae2462594436f254`.

F.5.17 `sequence`

Record CLI-F-017. Computes a finite prefix or one indexed exact sequence term.

Syntax. Parser-generated usage reflowed for print, with optional groups in square brackets. Exact help text is retained in the inventory.

```
usage: komorebi sequence [-h] [--terms TERMS] [--index INDEX]
    [--modulus MODULUS] [--compile]
    name
```

Argument	Type and requirement	Default	Accepted values
`name`	str; required	`no default`	runtime domain; see semantics
`--terms`	int; optional	`20`	runtime domain; see semantics
`--index`	int; optional	`not supplied`	runtime domain; see semantics
`--modulus`	int; optional	`not supplied`	runtime domain; see semantics
`--compile`	switch; optional	`False`	on/off flag

Output and exit contract. Prefix: JSON name:string, terms:array of strings; –compile adds strategy and exactness. With –index: a scalar text line, not the prefix object.

Semantic boundary. –index takes precedence over –terms. –modulus uses a compiled model even without –compile. Sequence names are normalized and aliases appear in F.2. Invalid names or domains can raise an uncaught exception.

File effects and replacement. No output-file option. This command does not intentionally overwrite user data; stdout can be redirected by the shell.

Minimum invocation (POSIX and PowerShell). The same one-line command is valid in either shell; run from the scratch fixture directory with a fresh output destination.

```
python -m komorebi_math sequence fibonacci --terms 8
```

Observed reference result. Exit 0. The bounded invocation completed successfully. 7 associated replay cases are retained, including shared-suite selections or variants where applicable.

Reference stdout field/type matrix.

Field	Type at this invocation
`name`	string
`terms`	array[string]

Failure handling. All parser help requests exit 0. Missing/unknown/conflicting arguments and invalid parser choices/types exit 2 with usage on stderr. No global CLI exception normalizer exists: uncaught implementation/-filesystem errors normally terminate the Python process with code 1 and a traceback. Command-specific 0/1/2 behavior is described above. Handler-local caught exception types: none explicitly caught.

Source binding. `src/komorebi_math/cli/main.py`, lines 396-419, handler `command_sequence`. Handler SHA-256: `713abaf159e30cb450b7b82774c3339bcb98f6dfd969e5d0f622763109cc3bda`.

F.5.18 `coordinates euler-fibonacci`

Record CLI-F-018. Constructs the exact signed-Euler recurrence-defect coordinates and their reconstruction.

Syntax. Parser-generated usage reflowed for print, with optional groups in square brackets. Exact help text is retained in the inventory.

```
usage: komorebi coordinates euler-fibonacci [-h] [--count COUNT]
```

Argument	Type and requirement	Default	Accepted values
`--count`	int; optional	`20`	runtime domain; see semantics

Output and exit contract. JSON system, count, initial, defects, reconstructed, homogeneous and defect_egf.

Semantic boundary. Exact quantities are rendered as rational strings where the serializer receives Fraction values. The finite count controls this calculation, not an infinite identity proof.

File effects and replacement. No output-file option. This command does not intentionally overwrite user data; stdout can be redirected by the shell.

Minimum invocation (POSIX and PowerShell). The same one-line command is valid in either shell; run from the scratch fixture directory with a fresh output destination.

```
python -m komorebi_math coordinates euler-fibonacci --count 8
```

Observed reference result. Exit 0. The bounded invocation completed successfully. 1 associated replay case is retained, including shared-suite selections or variants where applicable.

Reference stdout field/type matrix.

Field	Type at this invocation
`count`	integer
`defect_egf`	string
`defects`	array[string]
`homogeneous`	boolean
`initial`	array[string]
`reconstructed`	array[string]
`system`	string

Failure handling. All parser help requests exit 0. Missing/unknown/conflicting arguments and invalid parser choices/types exit 2 with usage on stderr. No global CLI exception normalizer exists: uncaught implementation/-filesystem errors normally terminate the Python process with code 1 and a traceback. Command-specific 0/1/2 behavior is described above. Handler-local caught exception types: `ValidationError`.

Source binding. `src/komorebi_math/cli/main.py`, lines 651-673, handler `command_coordinates_euler_fibonacci`. Handler SHA-256: `22c8daeeaf360fb2efa181517fb30d1f5c94ef36c3759c6e796ce20694c621f0`.

F.5.19 `coordinates recurrence`

Record CLI-F-019. Changes a supplied finite prefix between two exact recurrence-defect coordinate systems.

Syntax. Parser-generated usage reflowed for print, with optional groups in square brackets. Exact help text is retained in the inventory.

```
usage: komorebi coordinates recurrence [-h] --values VALUES --source SOURCE
    --target TARGET
    [--source-name SOURCE_NAME]
    [--target-name TARGET_NAME]
```

Argument	Type and requirement	Default	Accepted values
`--values`	str; required	`no default`	runtime domain; see semantics
`--source`	str; required	`no default`	runtime domain; see semantics
`--target`	str; required	`no default`	runtime domain; see semantics
`--source-name`	str; optional	`'source recurrence'`	runtime domain; see semantics
`--target-name`	str; optional	`'target recurrence'`	runtime domain; see semantics

Output and exit contract. JSON source_coordinates, target_coordinates, matrix, determinant:string and verified:boolean.

Semantic boundary. All three coefficient lists use comma-separated rational values. The mapped source coordinates are checked against independent target analysis before verified is emitted.

File effects and replacement. No output-file option. This command does not intentionally overwrite user data; stdout can be redirected by the shell.

Minimum invocation (POSIX and PowerShell). The same one-line command is valid in either shell; run from the scratch fixture directory with a fresh output destination.

```
python -m komorebi_math coordinates recurrence --values '0,1,1,2,3,5' --source '1,1' --target '2,1'
```

Observed reference result. Exit 0. The bounded invocation completed successfully. 1 associated replay case is retained, including shared-suite selections or variants where applicable.

Reference stdout field/type matrix.

Field	Type at this invocation
`determinant`	string
`matrix`	array[array[string]]
`source_coordinates`	array[string]
`target_coordinates`	array[string]
`verified`	boolean

Failure handling. All parser help requests exit 0. Missing/unknown/conflicting arguments and invalid parser choices/types exit 2 with usage on stderr. No global CLI exception normalizer exists: uncaught implementation/-filesystem errors normally terminate the Python process with code 1 and a traceback. Command-specific 0/1/2 behavior is described above. Handler-local caught exception types: `(ValidationError, ValueError)`.

Source binding. `src/komorebi_math/cli/main.py`, lines 676-715, handler `command_coordinates_recurrence`. Handler SHA-256: `f695ca61720e3e1b5aec31b39f70f5e19313435a2db6583a4bc93dc2e8ed17e6`.

F.5.20 `coordinates catalog`

Record CLI-F-020. Lists built-in coordinate families, recurrence models and transform categories.

Syntax. Parser-generated usage reflowed for print, with optional groups in square brackets. Exact help text is retained in the inventory.

```
usage: komorebi coordinates catalog [-h]
```

No positional arguments or command-specific options beyond help.

Output and exit contract. JSON category names mapped to arrays of descriptive names.

Semantic boundary. Use these names for runtime selectors where applicable; not every descriptive category is a direct CLI subcommand.

File effects and replacement. No output-file option. This command does not intentionally overwrite user data; stdout can be redirected by the shell.

Minimum invocation (POSIX and PowerShell). The same one-line command is valid in either shell; run from the scratch fixture directory with a fresh output destination.

```
python -m komorebi_math coordinates catalog
```

Observed reference result. Exit 0. The bounded invocation completed successfully. 1 associated replay case is retained, including shared-suite selections or variants where applicable.

Reference stdout field/type matrix.

Field	Type at this invocation
`affine_and_formal`	array[string]
`arithmetic`	array[string]
`combinatorial`	array[string]
`exact_linear`	array[string]

Field	Type at this invocation
`packet`	array[string]
`polynomial_families`	array[string]
`recurrence_models`	array[string]
`spectral`	array[string]
`state_space`	array[string]

Failure handling. All parser help requests exit 0. Missing/unknown/conflicting arguments and invalid parser choices/types exit 2 with usage on stderr. No global CLI exception normalizer exists: uncaught implementation/-filesystem errors normally terminate the Python process with code 1 and a traceback. Command-specific 0/1/2 behavior is described above. Handler-local caught exception types: none explicitly caught.

Source binding. `src/komorebi_math/cli/main.py`, lines 743-785, handler `command_coordinates_catalog`. Handler SHA-256: `7ff9cc720866a2783e7ea47f2d04ec53af7aa5a6ecec3ccc67ee2049e5ef711e`.

F.5.21 `coordinates basis`

Record CLI-F-021. Connects exact finite bases supplied as matrices and optionally analyzes an ambient vector.

Syntax. Parser-generated usage reflowed for print, with optional groups in square brackets. Exact help text is retained in the inventory.

```
usage: komorebi coordinates basis [-h] --source-matrix SOURCE_MATRIX
    --target-matrix TARGET_MATRIX
    [--source-name SOURCE_NAME]
    [--target-name TARGET_NAME]
    [--vector VECTOR]
```

Argument	Type and requirement	Default	Accepted values
`--source-matrix`	str; required	`no default`	runtime domain; see semantics
`--target-matrix`	str; required	`no default`	runtime domain; see semantics
`--source-name`	str; optional	`'source basis'`	runtime domain; see semantics
`--target-name`	str; optional	`'target basis'`	runtime domain; see semantics
`--vector`	str; optional	`not supplied`	runtime domain; see semantics

Output and exit contract. JSON source, target, ambient_dimension, dimension, matrix, determinant, verified; –vector adds ambient_vector and both coordinate arrays.

Semantic boundary. Rows are separated by semicolons and entries by commas, but the basis vectors are columns. Quote the matrix token in both shells. Singular/incompatible bases and malformed rows are rejected.

File effects and replacement. No output-file option. This command does not intentionally overwrite user data; stdout can be redirected by the shell.

Minimum invocation (POSIX and PowerShell). The same one-line command is valid in either shell; run from the scratch fixture directory with a fresh output destination.

```
python -m komorebi_math coordinates basis --source-matrix '1,0;0,1' --target-matrix '1,1;0,1' --vector '3,2'
```

Observed reference result. Exit 0. The bounded invocation completed successfully. 3 associated replay cases are retained, including shared-suite selections or variants where applicable.

Reference stdout field/type matrix.

Field	Type at this invocation
`ambient_dimension`	integer
`ambient_vector`	array[string]
`determinant`	string
`dimension`	integer
`matrix`	array[array[string]]
`source`	string
`source_coordinates`	array[string]
`target`	string
`target_coordinates`	array[string]
`verified`	boolean

Failure handling. All parser help requests exit 0. Missing/unknown/conflicting arguments and invalid parser choices/types exit 2 with usage on stderr. No global CLI exception normalizer exists: uncaught implementation/-filesystem errors normally terminate the Python process with code 1 and a traceback. Command-specific 0/1/2 behavior is described above. Handler-local caught exception types: `(ValidationError, ValueError)`.

Source binding. `src/komorebi_math/cli/main.py`, lines 788-821, handler `command_coordinates_basis`. Handler SHA-256: `56efd2a713babb6fe33f960f2ad3e1635fc94f95caa3657763d4d6838c8aa05c`.

F.5.22 `coordinates model`

Record CLI-F-022. Maps a sequence between named recurrence-model coordinates.

Syntax. Parser-generated usage reflowed for print, with optional groups in square brackets. Exact help text is retained in the inventory.

```
usage: komorebi coordinates model [-h] --source SOURCE --target TARGET
    [--count COUNT] [--values VALUES]
```

Argument	Type and requirement	Default	Accepted values
`--source`	str; required	`no default`	runtime domain; see semantics
`--target`	str; required	`no default`	runtime domain; see semantics
`--count`	int; optional	`12`	runtime domain; see semantics
`--values`	str; optional	`not supplied`	runtime domain; see semantics

Output and exit contract. JSON values, source, target, source_coordinates, target_coordinates, matrix and verified.

Semantic boundary. Without –values, generates –count values from the source model. With –values, that list is used and –count is ignored. Matching model names ignores case and nonalphanumeric characters.

File effects and replacement. No output-file option. This command does not intentionally overwrite user data; stdout can be redirected by the shell.

Minimum invocation (POSIX and PowerShell). The same one-line command is valid in either shell; run from the scratch fixture directory with a fresh output destination.

```
python -m komorebi_math coordinates model --source Fibonacci --target Pell --count 6
```

Observed reference result. Exit 0. The bounded invocation completed successfully. 2 associated replay cases are retained, including shared-suite selections or variants where applicable.

Reference stdout field/type matrix.

Field	Type at this invocation
`matrix`	array[array[string]]
`source`	string
`source_coordinates`	array[string]
`target`	string
`target_coordinates`	array[string]
`values`	array[string]
`verified`	boolean

Failure handling. All parser help requests exit 0. Missing/unknown/conflicting arguments and invalid parser choices/types exit 2 with usage on stderr. No global CLI exception normalizer exists: uncaught implementation/-filesystem errors normally terminate the Python process with code 1 and a traceback. Command-specific 0/1/2 behavior is described above. Handler-local caught exception types: `(ValidationError, ValueError)`.

Source binding. `src/komorebi_math/cli/main.py`, lines 824-852, handler `command_coordinates_model`. Handler SHA-256: `397e5daecd5663df8537a4cea573b6c7a2c92a382160916a1941b8d1a1b3e531`.

F.5.23 `coordinates polynomial`

Record CLI-F-023. Computes the exact connection matrix between graded polynomial families through a finite degree.

Syntax. Parser-generated usage reflowed for print, with optional groups in square brackets. Exact help text is retained in the inventory.

```
usage: komorebi coordinates polynomial [-h] [--source SOURCE]
    [--target TARGET] [--degree DEGREE]
```

Argument	Type and requirement	Default	Accepted values
`--source`	str; optional	`'euler'`	runtime domain; see semantics
`--target`	str; optional	`'fibonacci'`	runtime domain; see semantics
`--degree`	int; optional	8	runtime domain; see semantics

Output and exit contract. JSON degree, source, target, matrix, determinant:string and verified:boolean.

Semantic boundary. Runtime family names and aliases, not parser choices, determine acceptance. Degree controls a finite matrix. Unknown family or domain validation returns a handled failure.

File effects and replacement. No output-file option. This command does not intentionally overwrite user data; stdout can be redirected by the shell.

Minimum invocation (POSIX and PowerShell). The same one-line command is valid in either shell; run from the scratch fixture directory with a fresh output destination.

```
python -m komorebi_math coordinates polynomial --degree 3
```

Observed reference result. Exit 0. The bounded invocation completed successfully. 1 associated replay case is retained, including shared-suite selections or variants where applicable.

Reference stdout field/type matrix.

Field	Type at this invocation
`degree`	integer
`determinant`	string
`matrix`	array[array[string]]
`source`	string
`target`	string
`verified`	boolean

Failure handling. All parser help requests exit 0. Missing/unknown/conflicting arguments and invalid parser choices/types exit 2 with usage on stderr. No global CLI exception normalizer exists: uncaught implementation/-filesystem errors normally terminate the Python process with code 1 and a traceback. Command-specific 0/1/2 behavior is described above. Handler-local caught exception types: `ValidationError`.

Source binding. `src/komorebi_math/cli/main.py`, lines 718-740, handler `command_coordinates_polynomial`. Handler SHA-256: `c9c9aa21a244e2f79d4052dd687a69cf2f2faad988360e1f553897b3f4633a99`.

F.5.24 `coordinates combinatorial`

Record CLI-F-024. Builds one exact binomial, factorial or forward-difference transform.

Syntax. Parser-generated usage reflowed for print, with optional groups in square brackets. Exact help text is retained in the inventory.

```
usage: komorebi coordinates combinatorial [-h]
    --kind {binomial, inverse-binomial, forward-difference, monomial-to-falling,
    falling-to-monomial, monomial-to-rising, rising-to-monomial, rising-to-falling}
    --size SIZE [--values VALUES]
```

Argument	Type and requirement	Default	Accepted values
`--kind`	str; required	`no default`	binomial, inverse-binomial, forward-difference, monomial-to-falling, falling-to-monomial, monomial-to-rising, rising-to-monomial, rising-to-falling
`--size`	int; required	`no default`	runtime domain; see semantics
`--values`	str; optional	`not supplied`	runtime domain; see semantics

Output and exit contract. JSON kind, source, target, matrix, output:null or array and verified.

Semantic boundary. –size controls the transform size. Supplied –values must have that length. The eight parser choices are exhaustive for this command, not for the whole Python namespace.

File effects and replacement. No output-file option. This command does not intentionally overwrite user data; stdout can be redirected by the shell.

Minimum invocation (POSIX and PowerShell). The same one-line command is valid in either shell; run from the scratch fixture directory with a fresh output destination.

```
python -m komorebi_math coordinates combinatorial --kind binomial --size 4 --values '1,2,3,4'
```

Observed reference result. Exit 0. The bounded invocation completed successfully. 8 associated replay cases are retained, including shared-suite selections or variants where applicable.

Reference stdout field/type matrix.

Field	Type at this invocation
`kind`	string
`matrix`	array[array[string]]
`output`	array[string]
`source`	string
`target`	string
`verified`	boolean

Failure handling. All parser help requests exit 0. Missing/unknown/conflicting arguments and invalid parser choices/types exit 2 with usage on stderr. No global CLI exception normalizer exists: uncaught implementation/-filesystem errors normally terminate the Python process with code 1 and a traceback. Command-specific 0/1/2 behavior is described above. Handler-local caught exception types: `(KeyError, ValidationError, ValueError)`.

Source binding. `src/komorebi_math/cli/main.py`, lines 855-896, handler `command_coordinates_combinatorial`. Handler SHA-256: `1760a255db3450ddb72a839f39517d2aa87680a747124dc8447072d3664c8652`.

F.5.25 `coordinates spectral`

Record CLI-F-025. Builds a finite Fourier, cosine, Hadamard or Haar coordinate transform.

Syntax. Parser-generated usage reflowed for print, with optional groups in square brackets. Exact help text is retained in the inventory.

```
usage: komorebi coordinates spectral [-h] --kind {dft, dct, hadamard, haar}
    --size SIZE [--values VALUES]
```

Argument	Type and requirement	Default	Accepted values
`--kind`	str; required	`no default`	dft, dct, hadamard, haar
`--size`	int; required	`no default`	runtime domain; see semantics
`--values`	str; optional	`not supplied`	runtime domain; see semantics

Output and exit contract. JSON kind, source, target, condition_number:number, matrix, output:null or array and verified. Complex entries are objects with real and imag numbers.

Semantic boundary. –values accepts real floating-point comma-separated data, not general rational fractions or complex tokens. Hadamard/Haar size restrictions are enforced by their constructors. Valid means numerical tolerance checks, not exact symbolic equality.

File effects and replacement. No output-file option. This command does not intentionally overwrite user data; stdout can be redirected by the shell.

Minimum invocation (POSIX and PowerShell). The same one-line command is valid in either shell; run from the scratch fixture directory with a fresh output destination.

```
python -m komorebi_math coordinates spectral --kind dft --size 4 --values '1,2,3,4'
```

Observed reference result. Exit 0. The bounded invocation completed successfully. 4 associated replay cases are retained, including shared-suite selections or variants where applicable.

Reference stdout field/type matrix.

Field	Type at this invocation
`condition_number`	number
`kind`	string
`matrix`	array[array[object]]
`output`	array[object]
`source`	string
`target`	string
`verified`	boolean

Failure handling. All parser help requests exit 0. Missing/unknown/conflicting arguments and invalid parser choices/types exit 2 with usage on stderr. No global CLI exception normalizer exists: uncaught implementation/-filesystem errors normally terminate the Python process with code 1 and a traceback. Command-specific 0/1/2 behavior is described above. Handler-local caught exception types: `(KeyError, ValidationError, ValueError)`.

Source binding. `src/komorebi_math/cli/main.py`, lines 899-934, handler `command_coordinates_spectral`. Handler SHA-256: `b7169d9d9ad9b55968e3b10377bd25b92dc8742b4195e0905c4a4dee517fffdb`.

F.5.26 `coordinates packet`

Record CLI-F-026. Constructs an exact residue-lane, digit-reversal or Chinese-remainder permutation.

Syntax. Parser-generated usage reflowed for print, with optional groups in square brackets. Exact help text is retained in the inventory.

```
usage: komorebi coordinates packet [-h] --kind {residue, digit-reversal, crt}
    [--length LENGTH] [--modulus MODULUS]
    [--radix RADIX] [--moduli MODULI]
```

Argument	Type and requirement	Default	Accepted values
`--kind`	str; required	`no default`	residue, digit-reversal, crt
`--length`	int; optional	`8`	runtime domain; see semantics
`--modulus`	int; optional	`2`	runtime domain; see semantics
`--radix`	int; optional	`2`	runtime domain; see semantics
`--moduli`	str; optional	`'2,3'`	runtime domain; see semantics

Output and exit contract. JSON kind, source, target, matrix and verified.

Semantic boundary. residue uses –length/–modulus. digit-reversal uses –length/–radix. crt uses –moduli and ignores –length, –modulus and –radix. CRT dimension is derived from the moduli.

File effects and replacement. No output-file option. This command does not intentionally overwrite user data; stdout can be redirected by the shell.

Minimum invocation (POSIX and PowerShell). The same one-line command is valid in either shell; run from the scratch fixture directory with a fresh output destination.

```
python -m komorebi_math coordinates packet --kind residue
```

Observed reference result. Exit 0. The bounded invocation completed successfully. 3 associated replay cases are retained, including shared-suite selections or variants where applicable.

Reference stdout field/type matrix.

Field	Type at this invocation
`kind`	string
`matrix`	array[array[string]]
`source`	string
`target`	string
`verified`	boolean

Failure handling. All parser help requests exit 0. Missing/unknown/conflicting arguments and invalid parser choices/types exit 2 with usage on stderr. No global CLI exception normalizer exists: uncaught implementation/-filesystem errors normally terminate the Python process with code 1 and a traceback. Command-specific 0/1/2 behavior is described above. Handler-local caught exception types: `(ValidationError, ValueError)`.

Source binding. `src/komorebi_math/cli/main.py`, lines 937-965, handler `command_coordinates_packet`. Handler SHA-256: `f5404e9104d03ec87a54cde4d8c40c714d32a607a11f200607a8965e40702146`.

F.5.27 `coordinates formal`

Record CLI-F-027. Builds a truncated invertible formal change of variable and its inverse.

Syntax. Parser-generated usage reflowed for print, with optional groups in square brackets. Exact help text is retained in the inventory.

```
usage: komorebi coordinates formal [-h] --coordinate COORDINATE
    [--series SERIES] [--name NAME]
    [--source SOURCE] [--target TARGET]
```

Argument	Type and requirement	Default	Accepted values
`--coordinate`	str; required	`no default`	runtime domain; see semantics
`--series`	str; optional	`not supplied`	runtime domain; see semantics
`--name`	str; optional	`'formal coordinate change'`	runtime domain; see semantics
`--source`	str; optional	`'source formal coordinate'`	runtime domain; see semantics
`--target`	str; optional	`'target formal coordinate'`	runtime domain; see semantics

Output and exit contract. JSON name, source, target, coefficients, inverse, substitution_matrix, verified; –series adds pullback and roundtrip.

Semantic boundary. Use comma-separated rational coefficients. The coordinate must satisfy the constructor/inversion conditions, including the origin and invertible linear term. Labels do not change coefficients.

File effects and replacement. No output-file option. This command does not intentionally overwrite user data; stdout can be redirected by the shell.

Minimum invocation (POSIX and PowerShell). The same one-line command is valid in either shell; run from the scratch fixture directory with a fresh output destination.

```
python -m komorebi_math coordinates formal --coordinate '0,1,1,0' --series '1,2,3,4'
```

Observed reference result. Exit 0. The bounded invocation completed successfully. 3 associated replay cases are retained, including shared-suite selections or variants where applicable.

Reference stdout field/type matrix.

Field	Type at this invocation
`coefficients`	array[string]
`inverse`	array[string]
`name`	string
`pullback`	array[string]
`roundtrip`	array[string]
`source`	string
`substitution_matrix`	array[array[string]]
`target`	string
`verified`	boolean

Failure handling. All parser help requests exit 0. Missing/unknown/conflicting arguments and invalid parser choices/types exit 2 with usage on stderr. No global CLI exception normalizer exists: uncaught implementation/-filesystem errors normally terminate the Python process with code 1 and a traceback. Command-specific 0/1/2 behavior is described above. Handler-local caught exception types: `(ValidationError, ValueError)`.

Source binding. `src/komorebi_math/cli/main.py`, lines 968-1001, handler `command_coordinates_formal`. Handler SHA-256: `533d3f900f0ceac1c263481ff83f0ad20e8136ecaf5a5d9d91d67e77f211397e`.

F.5.28 `generating mask`

Record CLI-F-028. Compiles a finite Mahler mask into a carry plan and emits a finite sequence.

Syntax. Parser-generated usage reflowed for print, with optional groups in square brackets. Exact help text is retained in the inventory.

```
usage: komorebi generating mask [-h] --mask MASK --radix RADIX [--count COUNT]
```

Argument	Type and requirement	Default	Accepted values
`--mask`	str; required	`no default`	runtime domain; see semantics
`--radix`	int; required	`no default`	runtime domain; see semantics
`--count`	int; optional	`20`	runtime domain; see semantics

Output and exit contract. JSON radix, mask:array of strings, carry_bound, dimension, coefficients:array of strings and verified.

Semantic boundary. Mask entries are exact rationals and radix/count domains are checked by the compiler and certificate. A leading negative list should use –mask=-1,1 syntax so argparse receives one value.

File effects and replacement. No output-file option. This command does not intentionally overwrite user data; stdout can be redirected by the shell.

Minimum invocation (POSIX and PowerShell). The same one-line command is valid in either shell; run from the scratch fixture directory with a fresh output destination.

```
python -m komorebi_math generating mask --mask '1,1' --radix 2 --count 8
```

Observed reference result. Exit 0. The bounded invocation completed successfully. 1 associated replay case is retained, including shared-suite selections or variants where applicable.

Reference stdout field/type matrix.

Field	Type at this invocation
`carry_bound`	integer
`coefficients`	array[string]
`dimension`	integer
`mask`	array[string]
`radix`	integer
`verified`	boolean

Failure handling. All parser help requests exit 0. Missing/unknown/conflicting arguments and invalid parser choices/types exit 2 with usage on stderr. No global CLI exception normalizer exists: uncaught implementation/-filesystem errors normally terminate the Python process with code 1 and a traceback. Command-specific 0/1/2 behavior is described above. Handler-local caught exception types: `(ValidationError, ValueError)`.

Source binding. `src/komorebi_math/cli/main.py`, lines 1004-1025, handler `command_generating_mask`. Handler SHA-256: `4c9b57c71ac7477d11fb9226c8e23edf907e00f5e797d088bf6e7330ca45033b`.

F.5.29 `generating cartier`

Record CLI-F-029. Compiles a rational Mahler mask modulo a prime into a Cartier linear representation.

Syntax. Parser-generated usage reflowed for print, with optional groups in square brackets. Exact help text is retained in the inventory.

```
usage: komorebi generating cartier [-h] --numerator NUMERATOR
    --denominator DENOMINATOR --prime PRIME
    [--count COUNT]
```

Argument	Type and requirement	Default	Accepted values
`--numerator`	str; required	`no default`	runtime domain; see semantics
`--denominator`	str; required	`no default`	runtime domain; see semantics
`--prime`	int; required	`no default`	runtime domain; see semantics
`--count`	int; optional	`20`	runtime domain; see semantics

Output and exit contract. JSON prime, degree_bound, dimension and coefficients:array of integers.

Semantic boundary. Numerator/denominator lists contain integers, not rational strings. This describes the Mahler-mask functional equation, not automatically the ordinary Taylor coefficients of the rational function with those coefficients.

File effects and replacement. No output-file option. This command does not intentionally overwrite user data; stdout can be redirected by the shell.

Minimum invocation (POSIX and PowerShell). The same one-line command is valid in either shell; run from the scratch fixture directory with a fresh output destination.

```
python -m komorebi_math generating cartier --numerator 1 --denominator '1,-1' --prime 3 --count 8
```

Observed reference result. Exit 0. The bounded invocation completed successfully. 1 associated replay case is retained, including shared-suite selections or variants where applicable.

Reference stdout field/type matrix.

Field	Type at this invocation
`coefficients`	array[integer]
`degree_bound`	integer
`dimension`	integer
`prime`	integer

Failure handling. All parser help requests exit 0. Missing/unknown/conflicting arguments and invalid parser choices/types exit 2 with usage on stderr. No global CLI exception normalizer exists: uncaught implementation/-filesystem errors normally terminate the Python process with code 1 and a traceback. Command-specific 0/1/2 behavior is described above. Handler-local caught exception types: `(ValidationError, ValueError, RuntimeError)`.

Source binding. `src/komorebi_math/cli/main.py`, lines 1028-1049, handler `command_generating_cartier`. Handler SHA-256: `28538bbd947b0ea4f0ec0cef38e6f2fcca7c57fb32b2616e24e33117685acd0b`.

F.5.30 `generating kernel`

Record CLI-F-030. Evaluates a factorial-power kernel modulo a prime power.

Syntax. Parser-generated usage reflowed for print, with optional groups in square brackets. Exact help text is retained in the inventory.

```
usage: komorebi generating kernel [-h] --index INDEX --subindex SUBINDEX
    --prime PRIME --exponent EXPONENT
    --power POWER
```

Argument	Type and requirement	Default	Accepted values
`--index`	int; required	`no default`	runtime domain; see semantics
`--subindex`	int; required	`no default`	runtime domain; see semantics
`--prime`	int; required	`no default`	runtime domain; see semantics
`--exponent`	int; required	`no default`	runtime domain; see semantics
`--power`	int; required	`no default`	runtime domain; see semantics

Output and exit contract. JSON index, subindex, prime, exponent, factorial_power, valuation, unit, residue and survives:boolean.

Semantic boundary. The five required integer arguments are parser-typed only. Prime, exponent, power and index relations receive semantic validation in the kernel compiler.

File effects and replacement. No output-file option. This command does not intentionally overwrite user data; stdout can be redirected by the shell.

Minimum invocation (POSIX and PowerShell). The same one-line command is valid in either shell; run from the scratch fixture directory with a fresh output destination.

```
python -m komorebi_math generating kernel --index 5 --subindex 2 --prime 3 --exponent 3 --power 2
```

Observed reference result. Exit 0. The bounded invocation completed successfully. 2 associated replay cases are retained, including shared-suite selections or variants where applicable.

Reference stdout field/type matrix.

Field	Type at this invocation
`exponent`	integer
`factorial_power`	integer
`index`	integer
`prime`	integer
`residue`	integer
`subindex`	integer
`survives`	boolean
`unit`	integer
`valuation`	integer

Failure handling. All parser help requests exit 0. Missing/unknown/conflicting arguments and invalid parser choices/types exit 2 with usage on stderr. No global CLI exception normalizer exists: uncaught implementation/-filesystem errors normally terminate the Python process with code 1 and a traceback. Command-specific 0/1/2 behavior is described above. Handler-local caught exception types: `(ValidationError, ValueError)`.

Source binding. `src/komorebi_math/cli/main.py`, lines 1052-1078, handler `command_generating_kernel`. Handler SHA-256: `6281298785cd107525c13c381db67ed0d7dfdf4308a2312c9fe6c849c903cfb3`.

F.5.31 `generating bell`

Record CLI-F-031. Checks one generalized Bell digit-sum reduction.

Syntax. Parser-generated usage reflowed for print, with optional groups in square brackets. Exact help text is retained in the inventory.

```
usage: komorebi generating bell [-h] --index INDEX --prime PRIME --power POWER
```

Argument	Type and requirement	Default	Accepted values
`--index`	int; required	`no default`	runtime domain; see semantics
`--prime`	int; required	`no default`	runtime domain; see semantics
`--power`	int; required	`no default`	runtime domain; see semantics

Output and exit contract. JSON index, prime, factorial_power, digit_sum, high_polynomial, reduced_polynomial and valid:boolean.

Semantic boundary. Power must be at least two and congruent to one modulo prime-1. The returned reduction certificate is required before a success payload is emitted. This is a specified finite modular identity, not an unrestricted convergence claim.

File effects and replacement. No output-file option. This command does not intentionally overwrite user data; stdout can be redirected by the shell.

Minimum invocation (POSIX and PowerShell). The same one-line command is valid in either shell; run from the scratch fixture directory with a fresh output destination.

```
python -m komorebi_math generating bell --index 8 --prime 3 --power 3
```

Observed reference result. Exit 0. The bounded invocation completed successfully. 1 associated replay case is retained, including shared-suite selections or variants where applicable.

Reference stdout field/type matrix.

Field	Type at this invocation
`digit_sum`	integer
`factorial_power`	integer
`high_polynomial`	array[integer]
`index`	integer
`prime`	integer
`reduced_polynomial`	array[integer]
`valid`	boolean

Failure handling. All parser help requests exit 0. Missing/unknown/conflicting arguments and invalid parser choices/types exit 2 with usage on stderr. No global CLI exception normalizer exists: uncaught implementation/-filesystem errors normally terminate the Python process with code 1 and a traceback. Command-specific 0/1/2 behavior is described above. Handler-local caught exception types: `(ValidationError, ValueError)`.

Source binding. `src/komorebi_math/cli/main.py`, lines 1081-1105, handler `command_generating_bell`. Handler SHA-256: `40984ce5570a711f7b99458feac393dccb0bccf61df9eda523079caa2b99411f`.

F.5.32 `arithmetic standard-window`

Record CLI-F-032. Computes a residual standard window and requires its finite certificate.

Syntax. Parser-generated usage reflowed for print, with optional groups in square brackets. Exact help text is retained in the inventory.

```
usage: komorebi arithmetic standard-window [-h] --prime PRIME
    --compression COMPRESSION
    [--address ADDRESS] [--count COUNT]
```

Argument	Type and requirement	Default	Accepted values
`--prime`	int; required	`no default`	runtime domain; see semantics
`--compression`	int; required	`no default`	runtime domain; see semantics
`--address`	int; optional	`0`	runtime domain; see semantics
`--count`	int; optional	`20`	runtime domain; see semantics

Output and exit contract. JSON prime, compression, address, period, minimum_period and coefficients.

Semantic boundary. Prime/compression/address compatibility is validated by StandardWindow. –count controls the number of emitted coefficients; zero emits an empty prefix. No file is written.

File effects and replacement. No output-file option. This command does not intentionally overwrite user data; stdout can be redirected by the shell.

Minimum invocation (POSIX and PowerShell). The same one-line command is valid in either shell; run from the scratch fixture directory with a fresh output destination.

```
python -m komorebi_math arithmetic standard-window --prime 5 --compression 2 --count 8
```

Observed reference result. Exit 0. The bounded invocation completed successfully. 2 associated replay cases are retained, including shared-suite selections or variants where applicable.

Reference stdout field/type matrix.

Field	Type at this invocation
`address`	integer
`coefficients`	array[integer]
`compression`	integer
`minimum_period`	integer
`period`	integer
`prime`	integer

Failure handling. All parser help requests exit 0. Missing/unknown/conflicting arguments and invalid parser choices/types exit 2 with usage on stderr. No global CLI exception normalizer exists: uncaught implementation/-filesystem errors normally terminate the Python process with code 1 and a traceback. Command-specific 0/1/2 behavior is described above. Handler-local caught exception types: none explicitly caught.

Source binding. `src/komorebi_math/cli/main.py`, lines 422-439, handler `command_standard_window`. Handler SHA-256: `a5265e63fec3036a05eab6c1b52aa29373b4aaa2fbab5b7ed3367be40b951861`.

F.5.33 `arithmetic euler`

Record CLI-F-033. Reconstructs one local Euler value exactly or computes a depth-truncated record.

Syntax. Parser-generated usage reflowed for print, with optional groups in square brackets. Exact help text is retained in the inventory.

```
usage: komorebi arithmetic euler [-h] --prime PRIME --index INDEX
    [--depth DEPTH]
```

Argument	Type and requirement	Default	Accepted values
`--prime`	int; required	`no default`	runtime domain; see semantics
`--index`	int; required	`no default`	runtime domain; see semantics
`--depth`	int; optional	`not supplied`	runtime domain; see semantics

Output and exit contract. Without depth: prime, index, value:string, exact:true. With depth: prime, index, depth, value:string, exact_value, modulus and error:string.

Semantic boundary. Providing –depth switches both the computation and the output schema. The truncated result certificate is required; do not interpret that branch as the exact:true branch.

File effects and replacement. No output-file option. This command does not intentionally overwrite user data; stdout can be redirected by the shell.

Minimum invocation (POSIX and PowerShell). The same one-line command is valid in either shell; run from the scratch fixture directory with a fresh output destination.

```
python -m komorebi_math arithmetic euler --prime 5 --index 6
```

Observed reference result. Exit 0. The bounded invocation completed successfully. 2 associated replay cases are retained, including shared-suite selections or variants where applicable.

Reference stdout field/type matrix.

Field	Type at this invocation
`exact`	boolean
`index`	integer
`prime`	integer
`value`	string

Failure handling. All parser help requests exit 0. Missing/unknown/conflicting arguments and invalid parser choices/types exit 2 with usage on stderr. No global CLI exception normalizer exists: uncaught implementation/-filesystem errors normally terminate the Python process with code 1 and a traceback. Command-specific 0/1/2 behavior is described above. Handler-local caught exception types: none explicitly caught.

Source binding. `src/komorebi_math/cli/main.py`, lines 442-469, handler `command_euler_coordinates`. Handler SHA-256: `c3b7ca43c3066d65ad19e524961b586b44c349499c9c0c271db80bacc82836be`.

F.5.34 `deformation count`

Record CLI-F-034. Computes a finite smooth, weighted, nilpotent or normal-crossing chart count.

Syntax. Parser-generated usage reflowed for print, with optional groups in square brackets. Exact help text is retained in the inventory.

```
usage: komorebi deformation count [-h]
    --chart {smooth, weighted, nilpotent, normal-crossing}
    --q Q --depth DEPTH [--dimension DIMENSION]
    [--weights WEIGHTS] [--exponent EXPONENT]
```

Argument	Type and requirement	Default	Accepted values
`--chart`	str; required	`no default`	smooth, weighted, nilpotent, normal-crossing
`--q`	int; required	`no default`	runtime domain; see semantics
`--depth`	int; required	`no default`	runtime domain; see semantics
`--dimension`	int; optional	`1`	runtime domain; see semantics
`--weights`	str; optional	`'1'`	runtime domain; see semantics
`--exponent`	int; optional	`2`	runtime domain; see semantics

Output and exit contract. JSON chart, q, depth and count:integer.

Semantic boundary. –dimension is used only for smooth; –weights only for weighted; –exponent only for nilpotent. q is constructor-validated. Parser acceptance of an integer is not the mathematical domain check.

File effects and replacement. No output-file option. This command does not intentionally overwrite user data; stdout can be redirected by the shell.

Minimum invocation (POSIX and PowerShell). The same one-line command is valid in either shell; run from the scratch fixture directory with a fresh output destination.

```
python -m komorebi_math deformation count --chart smooth --q 3 --depth 2
```

Observed reference result. Exit 0. The bounded invocation completed successfully. 4 associated replay cases are retained, including shared-suite selections or variants where applicable.

Reference stdout field/type matrix.

Field	Type at this invocation
`chart`	string
`count`	integer
`depth`	integer
`q`	integer

Failure handling. All parser help requests exit 0. Missing/unknown/conflicting arguments and invalid parser choices/types exit 2 with usage on stderr. No global CLI exception normalizer exists: uncaught implementation/-filesystem errors normally terminate the Python process with code 1 and a traceback. Command-specific 0/1/2 behavior is described above. Handler-local caught exception types: none explicitly caught.

Source binding. `src/komorebi_math/cli/main.py`, lines 472-492, handler `command_deformation_count`. Handler SHA-256: `ee39bb86ba97b2b25d8067a48f670c9b9ee76e9ef783d92306f8e029d6093e95`.

F.5.35 `langlands tame-orbit`

Record CLI-F-035. Inspects a finite tame Frobenius orbit and its regularity/self-duality.

Syntax. Parser-generated usage reflowed for print, with optional groups in square brackets. Exact help text is retained in the inventory.

```
usage: komorebi langlands tame-orbit [-h] --q Q --degree DEGREE
    --exponent EXPONENT
```

Argument	Type and requirement	Default	Accepted values
`--q`	int; required	`no default`	runtime domain; see semantics
`--degree`	int; required	`no default`	runtime domain; see semantics
`--exponent`	int; required	`no default`	runtime domain; see semantics

Output and exit contract. JSON q, degree, exponent, orbit, regular, self_dual and regular_orbit_count.

Semantic boundary. The finite orbit certificate is required. Parameters are not a certificate of a general local-global correspondence.

File effects and replacement. No output-file option. This command does not intentionally overwrite user data; stdout can be redirected by the shell.

Minimum invocation (POSIX and PowerShell). The same one-line command is valid in either shell; run from the scratch fixture directory with a fresh output destination.

```
python -m komorebi_math langlands tame-orbit --q 3 --degree 2 --exponent 1
```

Observed reference result. Exit 0. The bounded invocation completed successfully. 1 associated replay case is retained, including shared-suite selections or variants where applicable.

Reference stdout field/type matrix.

Field	Type at this invocation
`degree`	integer
`exponent`	integer
`orbit`	array[integer]
`q`	integer
`regular`	boolean
`regular_orbit_count`	integer
`self_dual`	boolean

Failure handling. All parser help requests exit 0. Missing/unknown/conflicting arguments and invalid parser choices/types exit 2 with usage on stderr. No global CLI exception normalizer exists: uncaught implementation/-filesystem errors normally terminate the Python process with code 1 and a traceback. Command-specific 0/1/2 behavior is described above. Handler-local caught exception types: none explicitly caught.

Source binding. `src/komorebi_math/cli/main.py`, lines 495-511, handler `command_tame_orbit`. Handler SHA-256: `c3b7117ba92f41d9600415139cfd10efc0f95ef3fdea28e8100cabe759bf2ab4`.

F.5.36 `langlands heisenberg`

Record CLI-F-036. Builds and checks one finite Schrodinger Heisenberg model.

Syntax. Parser-generated usage reflowed for print, with optional groups in square brackets. Exact help text is retained in the inventory.

```
usage: komorebi langlands heisenberg [-h] --prime PRIME --rank RANK
```

Argument	Type and requirement	Default	Accepted values
`--prime`	int; required	`no default`	runtime domain; see semantics
`--rank`	int; required	`no default`	runtime domain; see semantics

Output and exit contract. JSON prime, rank, dimension and selector_rank.

Semantic boundary. Trace-support and selector certificates are required. Dimension grows with the parameters; the small reference command is not a large-scale resource promise.

File effects and replacement. No output-file option. This command does not intentionally overwrite user data; stdout can be redirected by the shell.

Minimum invocation (POSIX and PowerShell). The same one-line command is valid in either shell; run from the scratch fixture directory with a fresh output destination.

```
python -m komorebi_math langlands heisenberg --prime 3 --rank 1
```

Observed reference result. Exit 0. The bounded invocation completed successfully. 1 associated replay case is retained, including shared-suite selections or variants where applicable.

Reference stdout field/type matrix.

Field	Type at this invocation
`dimension`	integer
`prime`	integer
`rank`	integer
`selector_rank`	integer

Failure handling. All parser help requests exit 0. Missing/unknown/conflicting arguments and invalid parser choices/types exit 2 with usage on stderr. No global CLI exception normalizer exists: uncaught implementation/-filesystem errors normally terminate the Python process with code 1 and a traceback. Command-specific 0/1/2 behavior is described above. Handler-local caught exception types: none explicitly caught.

Source binding. `src/komorebi_math/cli/main.py`, lines 514-528, handler `command_heisenberg`. Handler SHA-256: `2adc69327aa3933f57adb01e41ea0f1b0614ef6920809f773fc5c94f256e40ac`.

F.5.37 `langlands completion`

Record CLI-F-037. Evaluates five flags supplied by the caller as a conditional completion report.

Syntax. Parser-generated usage reflowed for print, with optional groups in square brackets. Exact help text is retained in the inventory.

```
usage: komorebi langlands completion [-h] [--satisfied SATISFIED]
```

Argument	Type and requirement	Default	Accepted values
`--satisfied`	str; optional	`''`	runtime domain; see semantics

Output and exit contract. Certificate JSON with exactness=conditional. Empty/partial gates return 1; all five supplied return 0; unknown gate names return 2 and stderr.

Semantic boundary. The CLI does not prove the supplied gates. Passing –satisfied for all names asserts external assumptions, not a mathematical proof or unconditional automorphy result.

File effects and replacement. No output-file option. This command does not intentionally overwrite user data; stdout can be redirected by the shell.

Minimum invocation (POSIX and PowerShell). The same one-line command is valid in either shell; run from the scratch fixture directory with a fresh output destination.

```
python -m komorebi_math langlands completion
```

Observed reference result. Exit 1. This is the expected unmet-gate rejection, not completion. 3 associated replay cases are retained, including shared-suite selections or variants where applicable.

Reference stdout field/type matrix.

Field	Type at this invocation
`assumptions_checked`	array (empty in this fixture)
`assumptions_unchecked`	array[string]
`diagnostics`	object
`exactness`	string
`message`	string
`residuals`	object
`theorem_id`	string
`tolerance`	null
`valid`	boolean

Failure handling. All parser help requests exit 0. Missing/unknown/conflicting arguments and invalid parser choices/types exit 2 with usage on stderr. No global CLI exception normalizer exists: uncaught implementation/-filesystem errors normally terminate the Python process with code 1 and a traceback. Command-specific 0/1/2 behavior is described above. Handler-local caught exception types: none explicitly caught.

Source binding. `src/komorebi_math/cli/main.py`, lines 531-547, handler `command_completion`. Handler SHA-256: `23f991ded93a18786d71eddcddb4bacb8aa59b1311b8a199dc48fc910fbe8202`.

F.5.38 `export formats`

Record CLI-F-038. Lists supported export format names.

Syntax. Parser-generated usage reflowed for print, with optional groups in square brackets. Exact help text is retained in the inventory.

```
usage: komorebi export formats [-h]
```

No positional arguments or command-specific options beyond help.

Output and exit contract. JSON formats:array of strings.

Semantic boundary. The list has 11 format names. The YAML backend or PDF engine may still be absent. A format name does not mean every source object is exportable to that format.

File effects and replacement. No output-file option. This command does not intentionally overwrite user data; stdout can be redirected by the shell.

Minimum invocation (POSIX and PowerShell). The same one-line command is valid in either shell; run from the scratch fixture directory with a fresh output destination.

```
python -m komorebi_math export formats
```

Observed reference result. Exit 0. The bounded invocation completed successfully. 1 associated replay case is retained, including shared-suite selections or variants where applicable.

Reference stdout field/type matrix.

Field	Type at this invocation
`formats`	array[string]

Failure handling. All parser help requests exit 0. Missing/unknown/conflicting arguments and invalid parser choices/types exit 2 with usage on stderr. No global CLI exception normalizer exists: uncaught implementation/-filesystem errors normally terminate the Python process with code 1 and a traceback. Command-specific 0/1/2 behavior is described above. Handler-local caught exception types: none explicitly caught.

Source binding. `src/komorebi_math/cli/main.py`, lines 1108-1114, handler `command_export_formats`. Handler SHA-256: `8c04dd63cce128a4f0ace6776a7d3b601688d1f55f8d564bf792caaa18f109f6`.

F.5.39 `export sequence`

Record CLI-F-039. Computes a sequence prefix and exports a mathematical document.

Syntax. Parser-generated usage reflowed for print, with optional groups in square brackets. Exact help text is retained in the inventory.

```
usage: komorebi export sequence [-h] [--terms TERMS] [--modulus MODULUS]
    [--compile] --output OUTPUT [--format FORMAT]
    [--title TITLE] [--author AUTHOR]
    [--date DATE] [--abstract ABSTRACT]
    [--document-class DOCUMENT_CLASS] [--toc]
    [--mathjax]
    [--engine {auto, pdflatex, xelatex, lualatex, latexmk}]
    [--passes PASSES] [--timeout TIMEOUT]
    [--keep-auxiliary] [--no-keep-tex]
    [--no-type-metadata] [--force]
    name
```

Argument	Type and requirement	Default	Accepted values
`name`	str; required	`no default`	runtime domain; see semantics
`--terms`	int; optional	`20`	runtime domain; see semantics
`--modulus`	int; optional	`not supplied`	runtime domain; see semantics
`--compile`	switch; optional	`False`	on/off flag

Also accepts all 16 shared export options from F.3, including required `--output`. The expanded machine-readable record contains every action.

Output and exit contract. JSON acknowledgment path, format, bytes, companion_files and certificate. The destination stores the rendered document/data, not this acknowledgment.

Semantic boundary. There is no –index here. –modulus can use compilation even without –compile. The sequence_ document supplies the title; common –title, –date and –abstract are not propagated to that document by this handler.

File effects and replacement. No destination is written unless the command has a required output or –output is supplied. Existing destination is rejected unless –force. Failure can leave partial artifacts; transactional rollback is not promised.

Minimum invocation (POSIX and PowerShell). The same one-line command is valid in either shell; run from the scratch fixture directory with a fresh output destination.

```
python -m komorebi_math export sequence fibonacci --terms 8 --output sequence.json
```

Observed reference result. Exit 0. The bounded invocation completed successfully. 4 associated replay cases are retained, including shared-suite selections or variants where applicable.

Reference stdout field/type matrix.

Field	Type at this invocation
`bytes`	integer
`certificate`	object
`companion_files`	array (empty in this fixture)
`format`	string
`path`	string

Failure handling. All parser help requests exit 0. Missing/unknown/conflicting arguments and invalid parser choices/types exit 2 with usage on stderr. No global CLI exception normalizer exists: uncaught implementation/-filesystem errors normally terminate the Python process with code 1 and a traceback. Command-specific 0/1/2 behavior is described above. Handler-local caught exception types: none explicitly caught.

Source binding. `src/komorebi_math/cli/main.py`, lines 1117-1139, handler `command_export_sequence`. Handler SHA-256: `ae14bb99e194db72eb67c0ffe73b8b97d1ebb0da0aac142964fa0b9c018706cf`.

F.5.40 `export standard-window`

Record CLI-F-040. Computes and exports a standard-window report with its certificate.

Syntax. Parser-generated usage reflowed for print, with optional groups in square brackets. Exact help text is retained in the inventory.

```
usage: komorebi export standard-window [-h] --prime PRIME
    --compression COMPRESSION
    [--address ADDRESS] [--count COUNT]
    --output OUTPUT [--format FORMAT]
    [--title TITLE] [--author AUTHOR]
    [--date DATE] [--abstract ABSTRACT]
    [--document-class DOCUMENT_CLASS]
    [--toc] [--mathjax]
    [--engine {auto, pdflatex, xelatex, lualatex, latexmk}]
    [--passes PASSES] [--timeout TIMEOUT]
    [--keep-auxiliary] [--no-keep-tex]
    [--no-type-metadata] [--force]
```

Argument	Type and requirement	Default	Accepted values
`--prime`	int; required	`no default`	runtime domain; see semantics
`--compression`	int; required	`no default`	runtime domain; see semantics
`--address`	int; optional	`0`	runtime domain; see semantics
`--count`	int; optional	`20`	runtime domain; see semantics

Also accepts all 16 shared export options from F.3, including required `--output`. The expanded machine-readable record contains every action.

Output and exit contract. JSON acknowledgment path, format, bytes, companion_files and certificate.

Semantic boundary. The exported document includes the certificate. –format, if supplied, selects the format rather than solely relying on the extension.

File effects and replacement. No destination is written unless the command has a required output or –output is supplied. Existing destination is rejected unless –force. Failure can leave partial artifacts; transactional rollback is not promised.

Minimum invocation (POSIX and PowerShell). The same one-line command is valid in either shell; run from the scratch fixture directory with a fresh output destination.

```
python -m komorebi_math export standard-window --prime 5 --compression 2 --count 8 --output window.json
```

Observed reference result. Exit 0. The bounded invocation completed successfully. 3 associated replay cases are retained, including shared-suite selections or variants where applicable.

Reference stdout field/type matrix.

Field	Type at this invocation
`bytes`	integer
`certificate`	object
`companion_files`	array (empty in this fixture)
`format`	string
`path`	string

Failure handling. All parser help requests exit 0. Missing/unknown/conflicting arguments and invalid parser choices/types exit 2 with usage on stderr. No global CLI exception normalizer exists: uncaught implementation/-filesystem errors normally terminate the Python process with code 1 and a traceback. Command-specific 0/1/2 behavior is described above. Handler-local caught exception types: none explicitly caught.

Source binding. `src/komorebi_math/cli/main.py`, lines 1142-1169, handler `command_export_standard_window`. Handler SHA-256: `3d33ed28773e3b27742d8bef9b3b5b6c7b58970569abed2bfe2c905d4bd29590`.

F.5.41 `export euler`

Record CLI-F-041. Exports an exact or depth-truncated local Euler report.

Syntax. Parser-generated usage reflowed for print, with optional groups in square brackets. Exact help text is retained in the inventory.

```
usage: komorebi export euler [-h] --prime PRIME --index INDEX [--depth DEPTH]
    --output OUTPUT [--format FORMAT] [--title TITLE]
    [--author AUTHOR] [--date DATE]
    [--abstract ABSTRACT]
    [--document-class DOCUMENT_CLASS] [--toc]
    [--mathjax]
    [--engine {auto, pdflatex, xelatex, lualatex, latexmk}]
    [--passes PASSES] [--timeout TIMEOUT]
    [--keep-auxiliary] [--no-keep-tex]
    [--no-type-metadata] [--force]
```

Argument	Type and requirement	Default	Accepted values
`--prime`	int; required	`no default`	runtime domain; see semantics
`--index`	int; required	`no default`	runtime domain; see semantics
`--depth`	int; optional	`not supplied`	runtime domain; see semantics

Also accepts all 16 shared export options from F.3, including required `--output`. The expanded machine-readable record contains every action.

Output and exit contract. JSON acknowledgment path, format, bytes, companion_files and certificate.

Semantic boundary. –depth switches the mathematical report payload. An export certificate describes export success; distinguish it from the embedded arithmetic certificate.

File effects and replacement. No destination is written unless the command has a required output or –output is supplied. Existing destination is rejected unless –force. Failure can leave partial artifacts; transactional rollback is not promised.

Minimum invocation (POSIX and PowerShell). The same one-line command is valid in either shell; run from the scratch fixture directory with a fresh output destination.

```
python -m komorebi_math export euler --prime 5 --index 6 --output euler.json
```

Observed reference result. Exit 0. The bounded invocation completed successfully. 4 associated replay cases are retained, including shared-suite selections or variants where applicable.

Reference stdout field/type matrix.

Field	Type at this invocation
`bytes`	integer
`certificate`	object
`companion_files`	array (empty in this fixture)
`format`	string
`path`	string

Failure handling. All parser help requests exit 0. Missing/unknown/conflicting arguments and invalid parser choices/types exit 2 with usage on stderr. No global CLI exception normalizer exists: uncaught implementation/-filesystem errors normally terminate the Python process with code 1 and a traceback. Command-specific 0/1/2 behavior is described above. Handler-local caught exception types: none explicitly caught.

Source binding. `src/komorebi_math/cli/main.py`, lines 1172-1208, handler `command_export_euler`. Handler SHA-256: `26c402020d83ace71227723a5d69b6aef249ff07b2c0493c02ec0eaa52c93718`.

F.5.42 `export json`

Record CLI-F-042. Reads UTF-8/UTF-8-BOM JSON from –input and renders it in another supported format.

Syntax. Parser-generated usage reflowed for print, with optional groups in square brackets. Exact help text is retained in the inventory.

```
usage: komorebi export json [-h] --input INPUT --output OUTPUT
    [--format FORMAT] [--title TITLE]
    [--author AUTHOR] [--date DATE]
    [--abstract ABSTRACT]
    [--document-class DOCUMENT_CLASS] [--toc]
    [--mathjax]
    [--engine {auto, pdflatex, xelatex, lualatex, latexmk}]
    [--passes PASSES] [--timeout TIMEOUT]
    [--keep-auxiliary] [--no-keep-tex]
    [--no-type-metadata] [--force]
```

Argument	Type and requirement	Default	Accepted values
`--input`	str; required	`no default`	runtime domain; see semantics

Also accepts all 16 shared export options from F.3, including required `--output`. The expanded machine-readable record contains every action.

Output and exit contract. JSON acknowledgment path, format, bytes, companion_files and certificate.

Semantic boundary. Input parsing, unsupported data shapes and filesystem errors can raise exceptions. –input and –output are not mutually exclusive paths, so use distinct files and avoid –force on the input.

File effects and replacement. No destination is written unless the command has a required output or –output is supplied. Existing destination is rejected unless –force. Failure can leave partial artifacts; transactional rollback is not promised.

Minimum invocation (POSIX and PowerShell). The same one-line command is valid in either shell; run from the scratch fixture directory with a fresh output destination.

```
python -m komorebi_math export json --input input.json --output table.csv
```

Observed reference result. Exit 0. The bounded invocation completed successfully. 19 associated replay cases are retained, including shared-suite selections or variants where applicable.

Reference stdout field/type matrix.

Field	Type at this invocation
`bytes`	integer
`certificate`	object
`companion_files`	array (empty in this fixture)
`format`	string
`path`	string

Failure handling. All parser help requests exit 0. Missing/unknown/conflicting arguments and invalid parser choices/types exit 2 with usage on stderr. No global CLI exception normalizer exists: uncaught implementation/-filesystem errors normally terminate the Python process with code 1 and a traceback. Command-specific 0/1/2 behavior is described above. Handler-local caught exception types: none explicitly caught.

Source binding. `src/komorebi_math/cli/main.py`, lines 1211-1220, handler `command_export_json`. Handler SHA-256: `440c38bd62ec7594efbb9c2969b106f4e4da5c8879ab35d01d559f1d82991cf7`.

F.5.43 `export bundle`

Record CLI-F-043. Reads JSON and creates a ZIP containing the requested output formats.

Syntax. Parser-generated usage reflowed for print, with optional groups in square brackets. Exact help text is retained in the inventory.

```
usage: komorebi export bundle [-h] --input INPUT [--base-name BASE_NAME]
    [--formats FORMATS] --output OUTPUT
    [--format FORMAT] [--title TITLE]
    [--author AUTHOR] [--date DATE]
    [--abstract ABSTRACT]
    [--document-class DOCUMENT_CLASS] [--toc]
    [--mathjax]
    [--engine {auto, pdflatex, xelatex, lualatex, latexmk}]
    [--passes PASSES] [--timeout TIMEOUT]
    [--keep-auxiliary] [--no-keep-tex]
    [--no-type-metadata] [--force]
```

Argument	Type and requirement	Default	Accepted values
`--input`	str; required	`no default`	runtime domain; see semantics
`--base-name`	str; optional	`'komorebi_export'`	runtime domain; see semantics
`--formats`	str; optional	`'json,markdown,latex'`	runtime domain; see semantics

Also accepts all 16 shared export options from F.3, including required `--output`. The expanded machine-readable record contains every action.

Output and exit contract. Serialized ExportBundleResult JSON, including destination/member information and certificate; exact fields appear in the observation ledger.

Semantic boundary. –formats is the comma-separated member-format selector. The inherited singular –format is accepted by argparse but ignored by this handler. –base-name governs member names. This is a data-export ZIP, not the library release archive.

File effects and replacement. No destination is written unless the command has a required output or –output is supplied. Existing destination is rejected unless –force. Failure can leave partial artifacts; transactional rollback is not promised.

Minimum invocation (POSIX and PowerShell). The same one-line command is valid in either shell; run from the scratch fixture directory with a fresh output destination.

```
python -m komorebi_math export bundle --input input.json --output bundle.zip
```

Observed reference result. Exit 0. The bounded invocation completed successfully. 5 associated replay cases are retained, including shared-suite selections or variants where applicable.

Reference stdout field/type matrix.

Field	Type at this invocation
`__type__`	string
`certificate`	object
`formats`	array[string]
`manifest_path`	string
`members`	array[string]
`path`	string

Failure handling. All parser help requests exit 0. Missing/unknown/conflicting arguments and invalid parser choices/types exit 2 with usage on stderr. No global CLI exception normalizer exists: uncaught implementation/-filesystem errors normally terminate the Python process with code 1 and a traceback. Command-specific 0/1/2 behavior is described above. Handler-local caught exception types: none explicitly caught.

Source binding. `src/komorebi_math/cli/main.py`, lines 1223-1240, handler `command_export_bundle`. Handler SHA-256: `3f29595b3c3295d6397f6e84e59d8b43a62b80a1f2c9882226953d923dd6fc8d`.

F.5.44 `latex engines`

Record CLI-F-044. Checks for local LaTeX compiler executables.

Syntax. Parser-generated usage reflowed for print, with optional groups in square brackets. Exact help text is retained in the inventory.

```
usage: komorebi latex engines [-h]
```

No positional arguments or command-specific options beyond help.

Output and exit contract. JSON pdflatex, xelatex, lualatex and latexmk, each boolean.

Semantic boundary. A detected binary is not a successful render, package-availability check or security sandbox.

File effects and replacement. No output-file option. This command does not intentionally overwrite user data; stdout can be redirected by the shell.

Minimum invocation (POSIX and PowerShell). The same one-line command is valid in either shell; run from the scratch fixture directory with a fresh output destination.

```
python -m komorebi_math latex engines
```

Observed reference result. Exit 0. The bounded invocation completed successfully. 1 associated replay case is retained, including shared-suite selections or variants where applicable.

Reference stdout field/type matrix.

Field	Type at this invocation
`latexmk`	boolean
`lualatex`	boolean
`pdflatex`	boolean
`xelatex`	boolean

Failure handling. All parser help requests exit 0. Missing/unknown/conflicting arguments and invalid parser choices/types exit 2 with usage on stderr. No global CLI exception normalizer exists: uncaught implementation/-filesystem errors normally terminate the Python process with code 1 and a traceback. Command-specific 0/1/2 behavior is described above. Handler-local caught exception types: none explicitly caught.

Source binding. `src/komorebi_math/cli/main.py`, lines 1243-1249, handler `command_latex_engines`. Handler SHA-256: `156cceb70723cf26b6ec70dddacbb923fad03603d50f32a66a944868575a5186`.

F.5.45 `latex compile`

Record CLI-F-045. Compiles an existing TeX source into a local PDF and records compiler output.

Syntax. Parser-generated usage reflowed for print, with optional groups in square brackets. Exact help text is retained in the inventory.

```
usage: komorebi latex compile [-h] [--output OUTPUT]
    [--engine {auto, pdflatex, xelatex, lualatex, latexmk}]
    [--passes PASSES] [--timeout TIMEOUT]
    [--keep-auxiliary]
    source
```

Argument	Type and requirement	Default	Accepted values
`source`	str; required	`no default`	runtime domain; see semantics
`--output`	str; optional	`not supplied`	runtime domain; see semantics
`--engine`	str; optional	`'auto'`	auto, pdflatex, xelatex, lualatex, latexmk
`--passes`	int; optional	`2`	runtime domain; see semantics
`--timeout`	float; optional	`120.0`	runtime domain; see semantics
`--keep-auxiliary`	switch; optional	`False`	on/off flag

Output and exit contract. Serialized LatexCompilation JSON: source/pdf/log paths, engine, commands, return_ codes and certificate. 0 means valid compilation, 1 failure; a non-.pdf output is explicitly rejected with 2.

Semantic boundary. Without –output, uses the source path with .pdf suffix. There is no –force and existing PDF/log paths can be overwritten. Compiler timeout is per subprocess, not the total command budget. Shell escape is disabled by default, but arbitrary TeX is not made safe by that flag.

File effects and replacement. Writes PDF, compiler log and auxiliaries. No refusal guard/–force. Existing target artifacts may be replaced; use a fresh directory. Failed compilation may leave files.

Minimum invocation (POSIX and PowerShell). The same one-line command is valid in either shell; run from the scratch fixture directory with a fresh output destination.

```
python -m komorebi_math latex compile small.tex --output compiled.pdf --passes 1
```

Observed reference result. Exit 0. The bounded invocation completed successfully. 6 associated replay cases are retained, including shared-suite selections or variants where applicable.

Reference stdout field/type matrix.

Field	Type at this invocation
`__type__`	string
`certificate`	object
`commands`	array[array[string]]
`engine`	string
`log_path`	string
`pdf_path`	string
`return_codes`	array[integer]
`source_path`	string

Failure handling. All parser help requests exit 0. Missing/unknown/conflicting arguments and invalid parser choices/types exit 2 with usage on stderr. No global CLI exception normalizer exists: uncaught implementation/-filesystem errors normally terminate the Python process with code 1 and a traceback. Command-specific 0/1/2 behavior is described above. Handler-local caught exception types: none explicitly caught.

Source binding. `src/komorebi_math/cli/main.py`, lines 1252-1273, handler `command_latex_compile`. Handler SHA-256: `5a9195661dcae58c3d32090a196ca969e8a24fd25e898848e4f5694a98c45fd1`.

F.5.46 `describe`

Record CLI-F-046. Looks up registered mathematical provenance for a fully qualified object name.

Syntax. Parser-generated usage reflowed for print, with optional groups in square brackets. Exact help text is retained in the inventory.

```
usage: komorebi describe [-h] object
```

Argument	Type and requirement	Default	Accepted values
`object`	str; required	`no default`	runtime domain; see semantics

Output and exit contract. Provenance-record JSON on success. Missing record: stderr text and exit 1.

Semantic boundary. This is a provenance registry lookup, not arbitrary Python evaluation or proof verification.

File effects and replacement. No output-file option. This command does not intentionally overwrite user data; stdout can be redirected by the shell.

Minimum invocation (POSIX and PowerShell). The same one-line command is valid in either shell; run from the scratch fixture directory with a fresh output destination.

```
python -m komorebi_math describe komorebi_math.PacketSystem
```

Observed reference result. Exit 0. The bounded invocation completed successfully. 2 associated replay cases are retained, including shared-suite selections or variants where applicable.

Reference stdout field/type matrix.

Field	Type at this invocation
`assumptions`	array[string]
`identifier`	string
`limitations`	array (empty in this fixture)
`source`	string
`status`	string
`title`	string
`verification_suite`	string

Failure handling. All parser help requests exit 0. Missing/unknown/conflicting arguments and invalid parser choices/types exit 2 with usage on stderr. No global CLI exception normalizer exists: uncaught implementation/-filesystem errors normally terminate the Python process with code 1 and a traceback. Command-specific 0/1/2 behavior is described above. Handler-local caught exception types: none explicitly caught.

Source binding. `src/komorebi_math/cli/main.py`, lines 386-392, handler `command_describe`. Handler SHA-256: `3bdefc2c3242360f900eb71cab98a1d6aed1aaa1ff8a2b1565e5e26f7c2cef09`.

F.6 Recorded replay outcome and boundaries

The documentation bijection covers all 46 leaf commands. All 57 help nodes and all 201 non-help argument records are checked. The installed-wheel reference replay completed **243 cases** with all declared outcome predicates satisfied. That includes intentional nonzero exits for incomplete streams, corrupted chunks, invalid arguments/data, protected output paths and unavailable qualification gates. It does not mean every invoked verification gate passed.

The installed verification dispatch selections produced **50 successful suites and one failed GPU suite**. Four CUDA checks within that failed suite remained negative. Current-platform evidence, final readiness and required-GPU qualification were also negative. A separate validator invoked with `--require-all-gates` refuses to turn these observations into final qualification.

The recorded source suite passed 1,203 tests. The separate parser/ledger suite passed 509 tests. The replay executes the 46 default/minimal commands plus variants, so command, help, suite and test counts are overlapping evidence classes. These observations bind the unchanged mathematical reference package, its tests, and its distributions. The generated manual source, standalone appendix, selector ledger, inventory and replay validation scripts are included for reproducibility.

The replay uses a valid positive Bell power and the qualification report's actual field structure. It retains the expected GPU rejection and the rejection of dictionary input for NPY. The initial failed fixture records are preserved separately. These distinctions belong to the fixture contract and do not indicate a change to the package implementation.

Appendix F is complete as a version-bound CLI reference with finite Linux execution evidence. Native PowerShell, Windows, other Python versions, CUDA, strict MkDocs and fully dependency-isolated installation remain unqualified. Appendix G describes portable schemas and migrations. The subsequent appendices retain the separate supporting reference layers.

Appendix G. Portable schemas and migrations

Start with the file you need to recover. This appendix helps you choose a storage representation, understand its fields, and diagnose a failed load. For a first ordinary save-and-load example, go directly to G.9, then return to G.1 to understand the representation it uses.

Your task	Read first	Keep in view
Save one supported object and load it later	G.1, G.4 and G.9	The portable envelope and the decoded object are different layers.
Inspect a particular saved type	Its G.7 codec record	Match the tag and codec version before interpreting the fields.
Read an older packet-jet snapshot	G.5	Preserve the original and let the registered migration reconstruct the current object.
Resume or inspect a chunked snapshot	G.8	Check the stream's completion state as well as its chunk integrity.

Read a codec record from the outside inward. First identify the envelope, then the tagged payload, then the decoded fields, and finally the reconstructed Python object. The field table describes the decoded data supplied to reconstruction. For example, a matching payload digest shows agreement with the stored checksum; the codec's own paragraph explains which structural or mathematical checks follow. Keeping those steps separate makes a loading error much easier to locate.

G.0 Scope, status and how to use this reference

This appendix describes the executable persistence layer shipped in Komorebi Math 1.0.0b4, Repair R2, wheel build tag 2. It covers **all 51 frozen mathematical codecs, their 214 encoder fields, the single committed adjacent migration, canonical payloads, digest boundaries and resource limits**. Nine built-in scalar/container tags and 26 finite primitive fixtures are documented separately from the 51 mathematical registry entries. The stream directory and codec compatibility contract are separate schemas, not additional entries in that registry.

The observations in this reference describe the behavior of the stated source baseline. This reference does not silently tighten a decoder, change a schema version, certify arbitrary stored claims or qualify another platform. The fixture collection is deliberately small and deterministic. Its completeness is a coverage statement about the frozen registry, not exhaustive testing of every possible input value.

Each G-C record gives the codec tag and version, represented Python type, the complete decoded encoder-field table, a concrete canonical JSON fixture, payload digest, migration path, decoder behavior and source binding. **Field types in the table describe the displayed fixture.** They do not assert a formal JSON Schema, a universal type checker, or every possible union accepted by a decoder. The encoded data are recursively tagged, so a decoded field table is not a literal untagged JSON record.

The machine-readable reference retains full encoder, decoder and class-source locations and SHA-256 digests. Full pretty and compact canonical UTF-8 envelopes live in `manual/current/block39-fixtures`. The source-bound fixture generator, stream replay, mutation tests and archive verifier are included under `tools/block39` and at the delivery root. Reproduction should be performed in a working copy, not by changing files in an attested extraction.

G.1 Choose the representation before writing a file

Representation	Intended use	Reconstruction and integrity boundary
`to_serializable` / `from_serializable`	Broad presentation and export	Built-in markers can recover selected scalars/arrays. Unknown `__type__` records remain dictionaries. This is not the portable envelope.
`to_portable_data` / `from_portable_data`	An in-memory encoded payload tree	Uses explicit tags and versions. There is no envelope digest at this layer.
`dumps_portable` / `loads_portable`	One complete UTF-8 JSON document	Requires the exact portable envelope and verifies its payload digest before decoding.
`save_portable` / `load_portable`	One disk file	Adds file I/O and guarded replacement. It does not add authentication or a global size cap.
Packet-jet stream directory	Chunked fixed finite jet data	Uses distinct manifest and chunk envelopes, progress records, independent digests and explicit incomplete-state semantics.
Portable codec compatibility contract	Frozen release inventory	Compares tag/type/version identities and migration edges. It is not the serialized mathematical object itself.

Converting an ordinary export report into a portable record requires deliberate reconstruction of a supported object and then portable encoding. Renaming `__type__` to `__komorebi_type__`, inserting a version, or copying a CLI report into an envelope is not a supported migration. Likewise, a `.json` extension does not identify which schema it contains.

G.2 The exact single-document envelope

A newly emitted envelope has exactly five keys: `schema`, `schema_version`, `producer_version`, `payload` and `payload_sha256`. The schema string is `komorebi-math.portable`, and its current schema version is 1. The producer is the package version string, not the wheel build tag. The current implementation requires the producer field to be present but does not validate its type or value.

Only the **payload** participates in `payload_sha256`. Producer metadata is not bound by that digest. Changing the payload without changing its digest is rejected. Changing both together can produce another accepted document. This is corruption detection, not authentication, authorization, confidentiality or an independent mathematical certificate.

The canonical byte calculation is the implementation's exact JSON serialization:

```
canonical_payload = json.dumps(
    payload,
    sort_keys=True,
    separators=(",", ":"),
    ensure_ascii=False,
    allow_nan=False,
).encode("utf-8")
payload_sha256 = hashlib.sha256(canonical_payload).hexdigest()
```

This is the package's canonicalization rule, not a claim of conformity to a separate cross-language canonical-JSON standard. Key sorting, retained Unicode, finite numbers and separators matter. Whitespace in a loaded envelope need not match the writer. Duplicate JSON object keys are rejected by the parser hook, as are nonstandard `NaN` and infinity constants. The parser validates UTF-8 byte input and requires a JSON-object root.

A tagged value has exactly `__komorebi_type__`, `__version__` and `data`. The tag resolves only through known built-ins or the active trusted registry. No serialized import path selects an arbitrary Python class. Unknown tags and

future codec versions are rejected. Registered decoder and migrator callbacks are trusted application code, not code sandboxed by this format.

Observed permissive version boundary. The outer schema version is compared by equality with 1, so JSON `true` and `1.0` are accepted there. The codec version validator rejects booleans but accepts an integral float such as `1.0`. The canonical writer emits integers. Applications requiring strict untrusted-input typing must apply a stricter policy before relying on these fields.

G.3 Scalar, container and ndarray forms

Native null, booleans, integers, strings and finite floats pass through without a type tag. A list is recursively encoded as a JSON array. NumPy scalar inputs normalize through `.item()` rather than retaining a NumPy scalar class. Generic supported sequences can normalize to tuples. The format is not a general Python object graph serializer: aliases, shared references, arbitrary classes, open files, generators and callback code are not automatically preserved.

Built-in tag	Version	Literal `data` representation
`python.fraction`	1	Object containing `numerator` and `denominator`. The writer emits integer values.
`python.complex`	1	Object containing finite `real` and `imag` values.
`python.path`	1	Path spelling as a string. Loading creates a Path, not a file read.
`python.bytes`	1	Validated base64 string.
`python.tuple`	1	Array of recursively encoded members.
`python.set`	1	Array sorted by canonical encoded member spelling.
`python.frozenset`	1	The same array rule, restoring an immutable set.
`python.mapping`	1	Array of two-element arrays, sorted by the canonical encoded key.
`numpy.ndarray`	1	Exact keys `dtype`, `shape`, `data`; flat recursively encoded values and an integer shape array.

Mappings can preserve heterogeneous hashable keys, including fractions and tuples. The loader rejects duplicate decoded keys, including a collision between integer 1 and boolean true. Unhashable mapping keys and set items are rejected. Duplicate set entries, by contrast, are coalesced by set construction rather than rejected.

An untagged JSON object without `__komorebi_type__` is accepted by the low-level decoder as a mapping. Re-encoding it uses the tagged mapping form. Acceptance therefore does not imply that the input was an encoder-canonical payload.

For ndarrays the writer flattens values with `reshape(-1).tolist()`. The decoder restores a NumPy dtype, checks nonnegative dimensions and the product of shape against the flat length, then reconstructs the array. A scalar shape needs one flat entry, and a shape with a zero dimension needs none. Object-array elements are recursively encoded, not pickled. This still does not preserve memory strides, views, aliasing or device placement.

Array scope. Finite integer, floating, complex, boolean, Unicode, empty, scalar and exact-fraction object arrays have explicit fixtures. Not every NumPy dtype is a supported round-trip. A structured dtype is an observed counterexample: encoding succeeds but reconstruction rejects its dtype spelling. Do not generalize a successful numeric-array test to all NumPy dtypes.

Coercion scope. Fraction numerator/denominator and ndarray dimensions are converted with `int`. The decoder can accept fractional numeric inputs by truncating them. Examples include numerator 1.9 becoming 1 and dimension 2.9 becoming 2. These are tested acceptance boundaries, not endorsed canonical inputs. Canonical fixtures use integers in those fields.

G.4 Current portable API, file effects and errors

API	Defaults and return	Important boundary
`to_portable_data(value, *, max_depth=128)`	Encoded payload tree	No envelope or integrity check is added.
`from_portable_data(value, *, max_depth=128)`	Reconstructed value	Input must use supported nodes and tags; some noncanonical coercible data are accepted.
`portable_digest(value, *, max_depth=128)`	Lowercase payload SHA-256	Hashes the encoded object, not an arbitrary caller-supplied envelope.
`dumps_portable(value, *, indent=2, max_depth=128)`	JSON text ending with newline	Writer emits finite JSON and all five envelope keys.
`loads_portable(text, *, max_depth=128)`	Reconstructed value	Accepts text or UTF-8 bytes, checks envelope and payload digest, then decodes.
`save_portable(value, path, *, overwrite=False, indent=2, max_depth=128)`	Destination Path	Creates parents, writes a temporary file, flushes and fsyncs it, then replaces the destination.
`load_portable(path, *, max_depth=128)`	Reconstructed value	Reads the complete file before decoding. A missing/unreadable file raises ValidationError.
`register_portable_codec(type_, *, tag, version, encoder, decoder, replace=False)`	None	Type, callable, tag and version validation; deliberate replacement changes the active registry.
`register_portable_migration(*, tag, from_version, to_version, migrator, replace=False)`	None	An adjacent increment only. The parameter is `migrator`, not `migrate`.
`portable_codecs()` / `portable_migrations()`	Sorted tuples	Trigger built-in registration; extension-owned entries can exist in the active registry.

`save_portable` refuses an already existing destination unless overwrite is explicit. The test injects a replacement failure and verifies the original file and temporary-file cleanup. These are finite local checks of the implemented failure path. They are not a claim of crash durability of every filesystem, protection against concurrent writers, or a cross-platform file-locking guarantee. The existence check and the eventual replacement are separate operations.

Most decode failures are ValidationError. Registered decoder exceptions other than ValidationError are wrapped with the codec tag. Encoder exceptions are not uniformly wrapped: an opaque callback basis raises ValueError, and an operating-system write failure can propagate OSError. Applications should not assume every failure has one exception type.

G.5 Migration, compatibility and trust

A migration acts on **already recursively decoded codec data**, not on raw JSON text and not on the whole envelope. The loader validates the old payload digest before applying the migration. The reconstructed object re-encodes at the current codec version and generally has a new payload digest. Keep the original old envelope intact when retaining provenance.

Only one migration ships in the frozen baseline:

The `nonlinear.packet_jet_snapshot` codec migrates **version 1 to version 2**. It renames decoded `provider_name` to `provider`, introduces `coefficient_ordering` with value `"lexicographic"`, and carries the other snapshot fields unchanged.

Version 1 cannot be interpreted as version 2 by merely changing the version number. The legacy fixture is a genuine v1 field layout with a recomputed v1 payload digest. Loading it traverses the registered migration and reproduces the current finite snapshot. A future v3 payload is rejected.

For an extension, each edge must be `n -> n+1`. A missing intermediate edge, a future version, an invalid version, or a failing migrator raises an error. Duplicate edges require an explicit `replace=True`. Tests exercise a two-step custom chain, a missing link, duplicate registration, a nonadjacent edge and a failing callback. Registration requires a callable but does **not** prove that the callback is deterministic, pure, terminating or mathematically equivalent. Those remain the extension author's obligations.

```
from komorebi_math.export import register_portable_migration

register_portable_migration(
    tag="my_project.record",
    from_version=1,
    to_version=2,
    migrator=lambda data: {**data, "schema_name": "record-v2"},
)
```

This registration example alone does not install the extension's codec. The corresponding trusted type, encoder and decoder must be registered before such records can be reconstructed. Built-in registration is lazy and uses deliberate replacement for its own entries. Call `portable_codecs()` before introducing controlled extension registrations, and do not use replacement as an ordinary way to override package contracts.

The committed compatibility contract uses schema `komorebi-math.portable-codec-contract`, version 1, with exact top-level keys `schema`, `schema_version`, `package_version`, `entries`. Each entry has `tag`, `version`, `implementation_type`. Entries are deterministically ordered and tags are unique. The frozen package-owned inventory is 51 entries. The comparator rejects removed tags, changed represented types, decreased versions and increases without every adjacent edge. Additions are permitted. The comparator checks registry structure, not migrator semantics or every possible object representation.

The ordinary contract-file loader uses its own JSON path, not the portable envelope. The contract format does not gain payload authentication merely because it names codecs. Custom extension codecs are excluded from the package-owned snapshot unless their represented Python type claims a `komorebi_math.` module prefix. Type ownership here is a naming convention, not an authentication system.

G.6 Resource policy and integrity limits

`max_depth=128` is the generic portable traversal limit, starting at depth zero. It is not a file-byte, array-element, CPU-time or memory cap. `load_portable` reads the whole file, `loads_portable` parses JSON before the portable walk, and digest canonicalization materializes JSON text before reconstruction. Large valid-looking inputs can therefore consume resources before the traversal bound is reached. Extremely deep inputs may encounter Python's own parser or recursion limits first.

Component	Enforced or represented limit	What it does not automatically limit
Generic portable walk	Configured recursive depth	File bytes, integer digit count, overall node count, allocation size, decoder runtime
ndarray reconstruction	Nonnegative converted shape and matching flat length	A global maximum number of elements or total bytes
GeneratingBudget	Eight declared per-operation limits	Global JSON loading or every decoder merely because the budget is nested in a record
CoordinateBudget	Four declared per-operation limits	Generic persistence memory or all coordinate operations without explicit budget use
Finite frozen function	Captured degree	The original callback or any extension beyond that finite prefix
Stream selection/load	Explicit coefficient selection/materialization cap	The size of an arbitrary chunk file before it is parsed
Direct jet capture	Explicit evaluation budget per invocation	The exponential total basis count of the underlying finite jet

Component	Enforced or represented limit	What it does not automatically limit

Default GeneratingBudget fields are maximum_degree=100000, maximum_coefficient_bits=4000000, maximum_matrix_dimension=4096, maximum_reachable_states=100000, maximum_local_combinations=1000000, maximum_modulus=2000000, maximum_serialized_bytes=67108864 and maximum_recursion_depth=1000. CoordinateBudget defaults are maximum_matrix_dimension=512, maximum_polynomial_degree=512, maximum_horizon=4096 and maximum_exact_entries=1000000. These values are also source-bound in their record fixtures.

A record with `maximum_serialized_bytes=1` still serializes and loads through the generic API. That test demonstrates the distinction between a stored policy and an enforced universal cap. For untrusted data, callers should bound input bytes, nesting, element counts and trusted callback execution before treating this format as a service boundary. This appendix does not certify the current decoder as a strict hostile-input validator.

A certificate's stored `valid` field is data. Many result codecs lack the original mathematical inputs needed to re-run the computation. Some specialized decoders do perform finite checks, such as Kummer count recomputation, analysis/synthesis certification and mask-plan compilation. Their stronger checks are identified in the corresponding records. Do not promote every successfully decoded object to the same evidentiary status.

G.7 Complete mathematical codec records

The 51 records below match the committed registry one for one. Encoder-field tables are complete for the pinned version. Every field was removed individually in a malformed fixture and rejected. Extra decoded data fields were rejected by 39 codecs but ignored by 12 core/nonlinear/deformation codecs. The outer tagged wrapper rejects extra keys for all 51. The three facts refer to different layers and must not be conflated.

G.7.1 `arithmetic.polyphase_coordinates`

Record G-C01. Codec version 1. Represented type: `komorebi_math.arithmetic.polyphase.ArithmeticPolyphase`.

Exact arithmetic lanes use a positive modulus, tuple channels and a finite sequence length. Reconstructing the fixture recovers its rational sequence. The record stores lanes, not an infinite sequence rule. Rational arithmetic and finite shape consistency are distinct from any analytic continuation claim.

Complete decoded encoder data. The fixture forms below are examples, not a closed type specification for arbitrary inputs. Each row is a required data key.

Field	Form in the finite fixture
`channels`	`tuple[tuple; length=3]`
`length`	`int`
`modulus`	`int`

Read policy. Extra decoded data fields are rejected by the decoder. Exact field coverage does not imply strict numeric or boolean type checking. Missing fields were individually rejected in the version-bound tests. Extra outer tagged-wrapper fields and future versions were also rejected.

Canonical JSON and digest. Complete compact UTF-8 envelope: `block39-fixtures/arithmetic.polyphase_coordinates.canonical.json`. The pretty fixture has the same payload. Payload SHA-256: `d7ced187ff65537988308ba70e360db24e37b314d590ad7494b7a8248765163b`. Both forms reconstruct the represented type; the finite fixture re-encodes identically.

Migration. No older-version migration ships for this tag. Current version 1 is the supported frozen form; a future version is rejected.

Source binding. Encoder: `src/komorebi_math/refinement/portable.py`, lines 137-141. Decoder: `src/komorebi_math/refinement/portable.py`, lines 63-70. Represented class: `src/komorebi_math/arithmetic/polyphase.py`, lines 14-57. Full source and fragment hashes are in `Block39_Portable_Schema_Records.json`.

G.7.2 `coordinates.affine_transform`

Record G-C02. Codec version 1. Represented type: `komorebi_math.coordinates.affine.AffineCoordinateTransform`.

The decoder converts the matrix and offset through exact rational conversion, then constructs an affine transform. Source, target, theorem and description are labels carried with the data. Invertibility or a separately asserted theorem is not established merely by serializing those labels.

Complete decoded encoder data. The fixture forms below are examples, not a closed type specification for arbitrary inputs. Each row is a required data key.

Field	Form in the finite fixture
`description`	`str`
`matrix`	`tuple[tuple; length=2]`
`offset`	`tuple[Fraction; length=2]`
`source`	`str`
`target`	`str`
`theorem`	`str`

Read policy. Extra decoded data fields are rejected by the decoder. Exact field coverage does not imply strict numeric or boolean type checking. Missing fields were individually rejected in the version-bound tests. Extra outer tagged-wrapper fields and future versions were also rejected.

Canonical JSON and digest. Complete compact UTF-8 envelope: `block39-fixtures/coordinates.affine_transform.canonical.json`. The pretty fixture has the same payload. Payload SHA-256: `8e4e1529aa5acbd40f6de85eb969431867e89642bd02ef0c2bd7f5c7448a61a0`. Both forms reconstruct the represented type; the finite fixture re-encodes identically.

Migration. No older-version migration ships for this tag. Current version 1 is the supported frozen form; a future version is rejected.

Source binding. Encoder: `src/komorebi_math/coordinates/portable.py`, lines 464-471. Decoder: `src/komorebi_math/coordinates/portable.py`, lines 176-186. Represented class: `src/komorebi_math/coordinates/affine.py`, lines 27-148. Full source and fragment hashes are in `Block39_Portable_Schema_Records.json`.

G.7.3 `coordinates.analysis_synthesis`

Record G-C03. Codec version 1. Represented type: `komorebi_math.coordinates.analysis_synthesis.AnalysisSynthesisSystem`.

Analysis and synthesis remain NumPy arrays with their declared dtypes. The decoder reconstructs the system and requires its certificate. This finite numerical check uses the stored tolerance. It is not an exact symbolic proof or independent validation that a caller-selected tolerance is appropriate.

Complete decoded encoder data. The fixture forms below are examples, not a closed type specification for arbitrary inputs. Each row is a required data key.

Field	Form in the finite fixture
`analysis`	`ndarray[float64; shape=(2, 2)]`
`coordinates`	`str`
`source`	`str`
`synthesis`	`ndarray[float64; shape=(2, 2)]`
`theorem`	`str`
`tolerance`	`float`

Read policy. Extra decoded data fields are rejected by the decoder. Exact field coverage does not imply strict numeric or boolean type checking. Missing fields were individually rejected in the version-bound tests. Extra outer tagged-wrapper fields and future versions were also rejected.

Canonical JSON and digest. Complete compact UTF-8 envelope: `block39-fixtures/coordinates.analysis_synthesis.canonical.json`. The pretty fixture has the same payload. Payload SHA-256: `b777f8217c51a107c59748c1e6bec1cd736168b80c2abc01f317f2aa904ee557`. Both forms reconstruct the represented type; the finite fixture re-encodes identically.

Migration. No older-version migration ships for this tag. Current version 1 is the supported frozen form; a future version is rejected.

Source binding. Encoder: `src/komorebi_math/coordinates/portable.py`, lines 518-525. Decoder: `src/komorebi_math/coordinates/portable.py`, lines 224-236. Represented class: `src/komorebi_math/coordinates/analysis_synthesis.py`, lines 127-381. Full source and fragment hashes are in `Block39_Portable_Schema_Records.json`.

G.7.4 `coordinates.atlas`

Record G-C04. Codec version 1. Represented type: `komorebi_math.coordinates.basis.CoordinateAtlas`.

Systems and transforms are nested portable records. The decoder requires CoordinateSystem nodes and CoordinateTransform edges, registers them in the reconstructed atlas, and uses include_inverse=False while replaying stored edges. Inverses present in the saved graph remain explicit, rather than being silently regenerated.

Complete decoded encoder data. The fixture forms below are examples, not a closed type specification for arbitrary inputs. Each row is a required data key.

Field	Form in the finite fixture
`name`	`str`
`systems`	`tuple[CoordinateSystem; length=2]`
`transforms`	`tuple[CoordinateTransform; length=2]`

Read policy. Extra decoded data fields are rejected by the decoder. Exact field coverage does not imply strict numeric or boolean type checking. Missing fields were individually rejected in the version-bound tests. Extra outer tagged-wrapper fields and future versions were also rejected.

Canonical JSON and digest. Complete compact UTF-8 envelope: `block39-fixtures/coordinates.atlas.canonical.json`. The pretty fixture has the same payload. Payload SHA-256: `a417ec64d549f062a4c06b84e3f73f10fcfcbcf6ad9a07afaba5ecf35f865acf`. Both forms reconstruct the represented type; the finite fixture re-encodes identically.

Migration. No older-version migration ships for this tag. Current version 1 is the supported frozen form; a future version is rejected.

Source binding. Encoder: `src/komorebi_math/coordinates/portable.py`, lines 439-443. Decoder: `src/komorebi_math/coordinates/portable.py`, lines 150-162. Represented class: `src/komorebi_math/coordinates/basis.py`, lines 260-418. Full source and fragment hashes are in `Block39_Portable_Schema_Records.json`.

G.7.5 `coordinates.barycentric_system`

Record G-C05. Codec version 1. Represented type: `komorebi_math.coordinates.affine.BarycentricCoordinateSystem`.

Vertices are converted row by row to exact vectors and passed to BarycentricCoordinateSystem. The name and finite vertices are the complete encoder fields. This record does not capture a global geometric atlas or a triangulation algorithm.

Complete decoded encoder data. The fixture forms below are examples, not a closed type specification for arbitrary inputs. Each row is a required data key.

Field	Form in the finite fixture
`name`	`str`
`vertices`	`tuple[tuple; length=3]`

Read policy. Extra decoded data fields are rejected by the decoder. Exact field coverage does not imply strict numeric or boolean type checking. Missing fields were individually rejected in the version-bound tests. Extra outer tagged-wrapper fields and future versions were also rejected.

Canonical JSON and digest. Complete compact UTF-8 envelope: `block39-fixtures/coordinates.barycentric_system.canonical.json`. The pretty fixture has the same payload. Payload SHA-256: `85b32c5e4fecc0d5f15a125d5a7d6c6dfb4029f3e40889e506fbfe6779cc40d9`. Both forms reconstruct the represented type; the finite fixture re-encodes identically.

Migration. No older-version migration ships for this tag. Current version 1 is the supported frozen form; a future version is rejected.

Source binding. Encoder: `src/komorebi_math/coordinates/portable.py`, lines 479-482. Decoder: `src/komorebi_math/coordinates/portable.py`, lines 189-195. Represented class: `src/komorebi_math/coordinates/affine.py`, lines 151-231. Full source and fragment hashes are in `Block39_Portable_Schema_Records.json`.

G.7.6 `coordinates.bezout_defect_identity`

Record G-C06. Codec version 1. Represented type: `komorebi_math.coordinates.operators.BezoutDefectIdentity`.

All five polynomial vectors pass through exact conversion. The valid field is converted with bool. When that converted flag is true, the decoder requires the reconstructed result certificate. A false flag is retained without that require call. This is narrower than re-proving every possible interpretation of the stored multipliers.

Complete decoded encoder data. The fixture forms below are examples, not a closed type specification for arbitrary inputs. Each row is a required data key.

Field	Form in the finite fixture
`expected`	`tuple[Fraction; length=1]`
`gcd`	`tuple[Fraction; length=1]`
`left_multiplier`	`tuple[Fraction; length=1]`
`reconstructed`	`tuple[Fraction; length=1]`
`right_multiplier`	`tuple[Fraction; length=1]`
`valid`	`bool`

Read policy. Extra decoded data fields are rejected by the decoder. Exact field coverage does not imply strict numeric or boolean type checking. Missing fields were individually rejected in the version-bound tests. Extra outer tagged-wrapper fields and future versions were also rejected.

Canonical JSON and digest. Complete compact UTF-8 envelope: `block39-fixtures/coordinates.bezout_defect_identity.canonical.json`. The pretty fixture has the same payload. Payload SHA-256: `175235855ca6d89d4ea5fa15bc7ee480e989801d790a506788fd9ad734b018d9`. Both forms reconstruct the represented type; the finite fixture re-encodes identically.

Migration. No older-version migration ships for this tag. Current version 1 is the supported frozen form; a future version is rejected.

Source binding. Encoder: `src/komorebi_math/coordinates/portable.py`, lines 410-417. Decoder: `src/komorebi_math/coordinates/portable.py`, lines 115-135. Represented class: `src/komorebi_math/coordinates/operators.py`, lines 76-97. Full source and fragment hashes are in `Block39_Portable_Schema_Records.json`.

G.7.7 `coordinates.budget`

Record G-C07. Codec version 1. Represented type: `komorebi_math.coordinates.budget.CoordinateBudget`.

The four limits are converted to integers and stored in CoordinateBudget. They describe a resource policy for coordinate operations that explicitly accept that policy. A serialized budget is not a process-wide limit on JSON parsing, decoder memory or other objects in the same document.

Complete decoded encoder data. The fixture forms below are examples, not a closed type specification for arbitrary inputs. Each row is a required data key.

Field	Form in the finite fixture
`maximum_exact_entries`	`int`
`maximum_horizon`	`int`
`maximum_matrix_dimension`	`int`
`maximum_polynomial_degree`	`int`

Read policy. Extra decoded data fields are rejected by the decoder. Exact field coverage does not imply strict numeric or boolean type checking. Missing fields were individually rejected in the version-bound tests. Extra outer tagged-wrapper fields and future versions were also rejected.

Canonical JSON and digest. Complete compact UTF-8 envelope: `block39-fixtures/coordinates.budget.canonical.json`. The pretty fixture has the same payload. Payload SHA-256: `31472fd64d7de117fe8f92b2e3b7da7d5b08c3c745b3d72f2bdd1646222996e8`. Both forms reconstruct the represented type; the finite fixture re-encodes identically.

Migration. No older-version migration ships for this tag. Current version 1 is the supported frozen form; a future version is rejected.

Source binding. Encoder: `src/komorebi_math/coordinates/portable.py`, lines 318-323. Decoder: `src/komorebi_math/coordinates/portable.py`, lines 37-45. Represented class: `src/komorebi_math/coordinates/budget.py`, lines 10-65. Full source and fragment hashes are in `Block39_Portable_Schema_Records.json`.

G.7.8 `coordinates.finite_basis`

Record G-C08. Codec version 1. Represented type: `komorebi_math.coordinates.basis.FiniteBasis`.

Vectors pass through exact rational conversion and labels through string conversion. The declared basis includes its name, labels and description. The constructor remains the authority on finite consistency; persistence does not extend the basis beyond the stored vectors.

Complete decoded encoder data. The fixture forms below are examples, not a closed type specification for arbitrary inputs. Each row is a required data key.

Field	Form in the finite fixture
`description`	`str`
`labels`	`tuple[str; length=2]`
`name`	`str`
`vectors`	`tuple[tuple; length=2]`

Read policy. Extra decoded data fields are rejected by the decoder. Exact field coverage does not imply strict numeric or boolean type checking. Missing fields were individually rejected in the version-bound tests. Extra outer tagged-wrapper fields and future versions were also rejected.

Canonical JSON and digest. Complete compact UTF-8 envelope: `block39-fixtures/coordinates.finite_basis.canonical.json`. The pretty fixture has the same payload. Payload SHA-256: `6594540e52c27746385bd0cdc815eeef5ccfc9ec0ea1474341c5f8d1299646e4`. Both forms reconstruct the represented type; the finite fixture re-encodes identically.

Migration. No older-version migration ships for this tag. Current version 1 is the supported frozen form; a future version is rejected.

Source binding. Encoder: `src/komorebi_math/coordinates/portable.py`, lines 451-456. Decoder: `src/komorebi_math/coordinates/portable.py`, lines 165-173. Represented class: `src/komorebi_math/coordinates/basis.py`, lines 54-217. Full source and fragment hashes are in `Block39_Portable_Schema_Records.json`.

G.7.9 `coordinates.finite_field_transform`

Record G-C09. Codec version 1. Represented type: `komorebi_math.coordinates.modular.FiniteFieldCoordinateTransform`.

Matrix entries and prime are converted with int before construction. The constructor controls the finite-field transform contract. The loader does not preserve the original lexical representation of numeric inputs, and coercible inputs should not be confused with strictly typed JSON integers.

Complete decoded encoder data. The fixture forms below are examples, not a closed type specification for arbitrary inputs. Each row is a required data key.

Field	Form in the finite fixture
`description`	`str`
`matrix`	`tuple[tuple; length=2]`
`prime`	`int`
`source`	`str`
`target`	`str`
`theorem`	`str`

Read policy. Extra decoded data fields are rejected by the decoder. Exact field coverage does not imply strict numeric or boolean type checking. Missing fields were individually rejected in the version-bound tests. Extra outer tagged-wrapper fields and future versions were also rejected.

Canonical JSON and digest. Complete compact UTF-8 envelope: `block39-fixtures/coordinates.finite_field_transform.canonical.json`. The pretty fixture has the same payload. Payload SHA-256: `cbe38e903bff429a00f3e8251da0a748d592210083470422e3ced5342e969afe`. Both forms reconstruct the represented type; the finite fixture re-encodes identically.

Migration. No older-version migration ships for this tag. Current version 1 is the supported frozen form; a future version is rejected.

Source binding. Encoder: `src/komorebi_math/coordinates/portable.py`, lines 503-510. Decoder: `src/komorebi_math/coordinates/portable.py`, lines 209-219. Represented class: `src/komorebi_math/coordinates/modular.py`, lines 85-210. Full source and fragment hashes are in `Block39_Portable_Schema_Records.json`.

G.7.10 `coordinates.formal_change`

Record G-C10. Codec version 1. Represented type: `komorebi_math.coordinates.formal.FormalCoordinateChange`.

The finite coefficient vector is converted to exact rationals. Source and target coordinate names are retained. Only the supplied formal prefix is stored, not an analytic map defined on a global domain or a proof of convergence.

Complete decoded encoder data. The fixture forms below are examples, not a closed type specification for arbitrary inputs. Each row is a required data key.

Field	Form in the finite fixture
`coefficients`	`tuple[Fraction; length=3]`
`name`	`str`
`source`	`str`
`target`	`str`

Read policy. Extra decoded data fields are rejected by the decoder. Exact field coverage does not imply strict numeric or boolean type checking. Missing fields were individually rejected in the version-bound tests. Extra outer tagged-wrapper fields and future versions were also rejected.

Canonical JSON and digest. Complete compact UTF-8 envelope: `block39-fixtures/coordinates.formal_change.canonical.json`. The pretty fixture has the same payload. Payload SHA-256: `a81d12bb6b719f1c8d2ab4ced797a2c6`

`c1df7cd7ca0180fc265181131f77d76b`. Both forms reconstruct the represented type; the finite fixture re-encodes identically.

Migration. No older-version migration ships for this tag. Current version 1 is the supported frozen form; a future version is rejected.

Source binding. Encoder: `src/komorebi_math/coordinates/portable.py`, lines 490-495. Decoder: `src/komorebi_math/coordinates/portable.py`, lines 198-206. Represented class: `src/komorebi_math/coordinates/formal.py`, lines 54-241. Full source and fragment hashes are in `Block39_Portable_Schema_Records.json`.

G.7.11 `coordinates.intertwiner_space`

Record G-C11. Codec version 1. Represented type: `komorebi_math.coordinates.realization.IntertwinerSpace`.

The decoder converts dimensions to integers and each stored basis matrix to exact form. It reconstructs a basis record, but the original pair of source and target operators is not among its fields. Therefore decoding alone cannot independently re-solve the original intertwining problem.

Complete decoded encoder data. The fixture forms below are examples, not a closed type specification for arbitrary inputs. Each row is a required data key.

Field	Form in the finite fixture
`basis`	`tuple[tuple; length=2]`
`source_dimension`	`int`
`target_dimension`	`int`

Read policy. Extra decoded data fields are rejected by the decoder. Exact field coverage does not imply strict numeric or boolean type checking. Missing fields were individually rejected in the version-bound tests. Extra outer tagged-wrapper fields and future versions were also rejected.

Canonical JSON and digest. Complete compact UTF-8 envelope: `block39-fixtures/coordinates.intertwiner_space.canonical.json`. The pretty fixture has the same payload. Payload SHA-256: `82e065b2557027228b80e0ba715786a055dae32527f15af0afc05812dbd7629b`. Both forms reconstruct the represented type; the finite fixture re-encodes identically.

Migration. No older-version migration ships for this tag. Current version 1 is the supported frozen form; a future version is rejected.

Source binding. Encoder: `src/komorebi_math/coordinates/portable.py`, lines 387-391. Decoder: `src/komorebi_math/coordinates/portable.py`, lines 100-107. Represented class: `src/komorebi_math/coordinates/realization.py`, lines 124-166. Full source and fragment hashes are in `Block39_Portable_Schema_Records.json`.

G.7.12 `coordinates.linear_realization`

Record G-C12. Codec version 1. Represented type: `komorebi_math.coordinates.realization.LinearRealization`.

Transition, initial and output are exact matrices/vectors after decoding. They define the stored finite state realization with its name. Equivalent realizations are not automatically identified or minimized by the persistence layer.

Complete decoded encoder data. The fixture forms below are examples, not a closed type specification for arbitrary inputs. Each row is a required data key.

Field	Form in the finite fixture
`initial`	`tuple[Fraction; length=2]`
`name`	`str`
`output`	`tuple[Fraction; length=2]`
`transition`	`tuple[tuple; length=2]`

Read policy. Extra decoded data fields are rejected by the decoder. Exact field coverage does not imply strict numeric or boolean type checking. Missing fields were individually rejected in the version-bound tests. Extra outer tagged-wrapper fields and future versions were also rejected.

Canonical JSON and digest. Complete compact UTF-8 envelope: `block39-fixtures/coordinates.linear_realization.canonical.json`. The pretty fixture has the same payload. Payload SHA-256: `01ae83c639d05c400f1ed9c137f9d64dce80a351c2be604a15202be8c029fdce`. Both forms reconstruct the represented type; the finite fixture re-encodes identically.

Migration. No older-version migration ships for this tag. Current version 1 is the supported frozen form; a future version is rejected.

Source binding. Encoder: `src/komorebi_math/coordinates/portable.py`, lines 358-363. Decoder: `src/komorebi_math/coordinates/portable.py`, lines 73-81. Represented class: `src/komorebi_math/coordinates/realization.py`, lines 37-86. Full source and fragment hashes are in `Block39_Portable_Schema_Records.json`.

G.7.13 `coordinates.modal_system`

Record G-C13. Codec version 1. Represented type: `komorebi_math.coordinates.numeric.ModalCoordinateSystem`.

The decoder requires both nested transforms to decode as NumericCoordinateTransform, and converts diagonal_residual to float. Eigenvalues and the two transforms are restored. The original operator is not encoded here, so its eigenproblem is not independently recomputed during decoding.

Complete decoded encoder data. The fixture forms below are examples, not a closed type specification for arbitrary inputs. Each row is a required data key.

Field	Form in the finite fixture
`analysis`	`komorebi_math.coordinates.numeric.NumericCoordinateTransform`
`diagonal_residual`	`float`
`eigenvalues`	`ndarray[float64; shape=(2,)]`
`synthesis`	`komorebi_math.coordinates.numeric.NumericCoordinateTransform`

Read policy. Extra decoded data fields are rejected by the decoder. Exact field coverage does not imply strict numeric or boolean type checking. Missing fields were individually rejected in the version-bound tests. Extra outer tagged-wrapper fields and future versions were also rejected.

Canonical JSON and digest. Complete compact UTF-8 envelope: `block39-fixtures/coordinates.modal_system.canonical.json`. The pretty fixture has the same payload. Payload SHA-256: `691b0ba59605bae84cd4a23b00d37df774d146d301f92b39c428ce9ed89aeef1`. Both forms reconstruct the represented type; the finite fixture re-encodes identically.

Migration. No older-version migration ships for this tag. Current version 1 is the supported frozen form; a future version is rejected.

Source binding. Encoder: `src/komorebi_math/coordinates/portable.py`, lines 548-553. Decoder: `src/komorebi_math/coordinates/portable.py`, lines 251-263. Represented class: `src/komorebi_math/coordinates/numeric.py`, lines 287-312. Full source and fragment hashes are in `Block39_Portable_Schema_Records.json`.

G.7.14 `coordinates.numeric_transform`

Record G-C14. Codec version 1. Represented type: `komorebi_math.coordinates.numeric.NumericCoordinateTransform`.

The matrix keeps its ndarray representation, and tolerance is converted to float. Source, target, theorem and description are descriptive fields. The numerical contract must still be interpreted at the declared tolerance and dtype.

Complete decoded encoder data. The fixture forms below are examples, not a closed type specification for arbitrary inputs. Each row is a required data key.

Field	Form in the finite fixture
description	str
matrix	ndarray[complex128; shape=(3, 3)]
source	str
target	str
theorem	str
tolerance	float

Read policy. Extra decoded data fields are rejected by the decoder. Exact field coverage does not imply strict numeric or boolean type checking. Missing fields were individually rejected in the version-bound tests. Extra outer tagged-wrapper fields and future versions were also rejected.

Canonical JSON and digest. Complete compact UTF-8 envelope: `block39-fixtures/coordinates.numeric_transform.canonical.json`. The pretty fixture has the same payload. Payload SHA-256: `2ba0fb2d2ba3c6eeb44247c6e1618e03ff21fc9a28afbc87817f352aeee231ab`. Both forms reconstruct the represented type; the finite fixture re-encodes identically.

Migration. No older-version migration ships for this tag. Current version 1 is the supported frozen form; a future version is rejected.

Source binding. Encoder: `src/komorebi_math/coordinates/portable.py`, lines 533-540. Decoder: `src/komorebi_math/coordinates/portable.py`, lines 238-248. Represented class: `src/komorebi_math/coordinates/numeric.py`, lines 26-164. Full source and fragment hashes are in `Block39_Portable_Schema_Records.json`.

G.7.15 `coordinates.polynomial_operator`

Record G-C15. Codec version 1. Represented type: `komorebi_math.coordinates.calculus.PolynomialCoordinateOperator`.

The finite matrix, source/target degrees, family labels, name and theorem label are stored. Exact conversion reconstructs the matrix. Loading this record does not verify an infinite polynomial-family identity beyond the represented finite operator.

Complete decoded encoder data. The fixture forms below are examples, not a closed type specification for arbitrary inputs. Each row is a required data key.

Field	Form in the finite fixture
matrix	tuple[tuple; length=3]
name	str
source_degree	int
source_family	str
target_degree	int
target_family	str
theorem	str

Read policy. Extra decoded data fields are rejected by the decoder. Exact field coverage does not imply strict numeric or boolean type checking. Missing fields were individually rejected in the version-bound tests. Extra outer tagged-wrapper fields and future versions were also rejected.

Canonical JSON and digest. Complete compact UTF-8 envelope: `block39-fixtures/coordinates.polynomial_operator.canonical.json`. The pretty fixture has the same payload. Payload SHA-256: `b7a355fa665d9ddbdbb85183d05b77951b533d3049cd52e0941f49c997ff3c68`. Both forms reconstruct the represented type; the finite fixture re-encodes identically.

Migration. No older-version migration ships for this tag. Current version 1 is the supported frozen form; a future version is rejected.

Source binding. Encoder: `src/komorebi_math/coordinates/portable.py`, lines 588-596. Decoder: `src/komorebi_math/coordinates/portable.py`, lines 289-308. Represented class: `src/komorebi_math/coordinates/calculus.py`, lines 32-102. Full source and fragment hashes are in `Block39_Portable_Schema_Records.json`.

G.7.16 `coordinates.recurrence_model`

Record G-C16. Codec version 1. Represented type: `komorebi_math.coordinates.families.RecurrenceModel`.

The coefficients and initial values are converted to exact vectors. The decoder uses the four-argument current constructor, including description. The recorded baseline includes the constructor correction, so the earlier wheel defect is absent from this implementation.

Complete decoded encoder data. The fixture forms below are examples, not a closed type specification for arbitrary inputs. Each row is a required data key.

Field	Form in the finite fixture
`coefficients`	`tuple[Fraction; length=2]`
`description`	`str`
`initial_values`	`tuple[Fraction; length=2]`
`name`	`str`

Read policy. Extra decoded data fields are rejected by the decoder. Exact field coverage does not imply strict numeric or boolean type checking. Missing fields were individually rejected in the version-bound tests. Extra outer tagged-wrapper fields and future versions were also rejected.

Canonical JSON and digest. Complete compact UTF-8 envelope: `block39-fixtures/coordinates.recurrence_model.canonical.json`. The pretty fixture has the same payload. Payload SHA-256: `92e8e314ebcc20d24a6f81623057fe63c8e7871d06b7f37c9d9679996fe3e85c`. Both forms reconstruct the represented type; the finite fixture re-encodes identically.

Migration. No older-version migration ships for this tag. Current version 1 is the supported frozen form; a future version is rejected.

Source binding. Encoder: `src/komorebi_math/coordinates/portable.py`, lines 561-566. Decoder: `src/komorebi_math/coordinates/portable.py`, lines 266-274. Represented class: `src/komorebi_math/coordinates/families.py`, lines 22-136. Full source and fragment hashes are in `Block39_Portable_Schema_Records.json`.

G.7.17 `coordinates.recurrence_record`

Record G-C17. Codec version 1. Represented type: `komorebi_math.coordinates.recurrence.RecurrenceCoordinateRecord`.

The decoder reconstructs exact recurrence coefficients, initial values and defects, then requires the record certificate. The finite reconstruction contract is checked. No statement about arbitrary later terms is inferred from a finite defect list.

Complete decoded encoder data. The fixture forms below are examples, not a closed type specification for arbitrary inputs. Each row is a required data key.

Field	Form in the finite fixture
`coefficients`	`tuple[Fraction; length=2]`
`defects`	`tuple[Fraction; length=6]`
`initial`	`tuple[Fraction; length=2]`
`name`	`str`

Read policy. Extra decoded data fields are rejected by the decoder. Exact field coverage does not imply strict numeric or boolean type checking. Missing fields were individually rejected in the version-bound tests. Extra outer tagged-wrapper fields and future versions were also rejected.

Canonical JSON and digest. Complete compact UTF-8 envelope: `block39-fixtures/coordinates.recurrence_record.canonical.json`. The pretty fixture has the same payload. Payload SHA-256: `85b1158e9f3817e02d9a3a53cb44b697280a7a4f4053d9d0dfb85409df4c9256`. Both forms reconstruct the represented type; the finite fixture re-encodes identically.

Migration. No older-version migration ships for this tag. Current version 1 is the supported frozen form; a future version is rejected.

Source binding. Encoder: `src/komorebi_math/coordinates/portable.py`, lines 345-350. Decoder: `src/komorebi_math/coordinates/portable.py`, lines 60-70. Represented class: `src/komorebi_math/coordinates/recurrence.py`, lines 127-208. Full source and fragment hashes are in `Block39_Portable_Schema_Records.json`.

G.7.18 `coordinates.shift_polynomial`

Record G-C18. Codec version 1. Represented type: `komorebi_math.coordinates.operators.ShiftPolynomial`.

Coefficients are converted to an exact vector and the ShiftPolynomial constructor establishes its normalized representation. The record includes the optional name. A noncanonical integer input can therefore re-encode differently after exact normalization.

Complete decoded encoder data. The fixture forms below are examples, not a closed type specification for arbitrary inputs. Each row is a required data key.

Field	Form in the finite fixture
`coefficients`	`tuple[Fraction; length=3]`
`name`	`str`

Read policy. Extra decoded data fields are rejected by the decoder. Exact field coverage does not imply strict numeric or boolean type checking. Missing fields were individually rejected in the version-bound tests. Extra outer tagged-wrapper fields and future versions were also rejected.

Canonical JSON and digest. Complete compact UTF-8 envelope: `block39-fixtures/coordinates.shift_polynomial.canonical.json`. The pretty fixture has the same payload. Payload SHA-256: `d99bd2d700e30c355061cdc09b38bc1d6baa722631c6c4d23c2c3b6f33fc78a2`. Both forms reconstruct the represented type; the finite fixture re-encodes identically.

Migration. No older-version migration ships for this tag. Current version 1 is the supported frozen form; a future version is rejected.

Source binding. Encoder: `src/komorebi_math/coordinates/portable.py`, lines 399-402. Decoder: `src/komorebi_math/coordinates/portable.py`, lines 110-112. Represented class: `src/komorebi_math/coordinates/operators.py`, lines 45-73. Full source and fragment hashes are in `Block39_Portable_Schema_Records.json`.

G.7.19 `coordinates.state_recovery`

Record G-C19. Codec version 1. Represented type: `komorebi_math.coordinates.realization.StateRecovery`.

The decoder applies bool to exact and unique, int to rank and horizon, and exact vector conversion to state, predicted and residual. The record does not include the original system and observations needed to independently redo the recovery. Stored flags are not fresh mathematical evidence.

Complete decoded encoder data. The fixture forms below are examples, not a closed type specification for arbitrary inputs. Each row is a required data key.

Field	Form in the finite fixture
`exact`	`bool`
`horizon`	`int`
`predicted`	`tuple[Fraction; length=2]`
`rank`	`int`
`residual`	`tuple[Fraction; length=2]`

Field	Form in the finite fixture
`state`	`tuple[Fraction; length=2]`
`unique`	`bool`

Read policy. Extra decoded data fields are rejected by the decoder. Exact field coverage does not imply strict numeric or boolean type checking. Missing fields were individually rejected in the version-bound tests. Extra outer tagged-wrapper fields and future versions were also rejected.

Canonical JSON and digest. Complete compact UTF-8 envelope: `block39-fixtures/coordinates.state_recovery.canonical.json`. The pretty fixture has the same payload. Payload SHA-256: `3b11526f93fdb34c57d86f73b0adaff3e57212e4c5b45d9c59f7d35b51a9db2b`. Both forms reconstruct the represented type; the finite fixture re-encodes identically.

Migration. No older-version migration ships for this tag. Current version 1 is the supported frozen form; a future version is rejected.

Source binding. Encoder: `src/komorebi_math/coordinates/portable.py`, lines 371-379. Decoder: `src/komorebi_math/coordinates/portable.py`, lines 84-97. Represented class: `src/komorebi_math/coordinates/realization.py`, lines 89-121. Full source and fragment hashes are in `Block39_Portable_Schema_Records.json`.

G.7.20 `coordinates.state_space_model`

Record G-C20. Codec version 1. Represented type: `komorebi_math.coordinates.state_space.StateSpaceModel`.

Transition, input, output and feedthrough matrices use exact conversion. The record describes a finite state-space model, not its execution history. File reconstruction is separate from controllability, observability or stability analysis.

Complete decoded encoder data. The fixture forms below are examples, not a closed type specification for arbitrary inputs. Each row is a required data key.

Field	Form in the finite fixture
`feedthrough`	`tuple[tuple; length=1]`
`input_matrix`	`tuple[tuple; length=2]`
`name`	`str`
`output_matrix`	`tuple[tuple; length=1]`
`transition`	`tuple[tuple; length=2]`

Read policy. Extra decoded data fields are rejected by the decoder. Exact field coverage does not imply strict numeric or boolean type checking. Missing fields were individually rejected in the version-bound tests. Extra outer tagged-wrapper fields and future versions were also rejected.

Canonical JSON and digest. Complete compact UTF-8 envelope: `block39-fixtures/coordinates.state_space_model.canonical.json`. The pretty fixture has the same payload. Payload SHA-256: `6e55b2b3961ef6f9db7476f6ba25554b4ef1521ca04903b68bddedcf46f7b6ca`. Both forms reconstruct the represented type; the finite fixture re-encodes identically.

Migration. No older-version migration ships for this tag. Current version 1 is the supported frozen form; a future version is rejected.

Source binding. Encoder: `src/komorebi_math/coordinates/portable.py`, lines 574-580. Decoder: `src/komorebi_math/coordinates/portable.py`, lines 277-286. Represented class: `src/komorebi_math/coordinates/state_space.py`, lines 31-131. Full source and fragment hashes are in `Block39_Portable_Schema_Records.json`.

G.7.21 `coordinates.system`

Record G-C21. Codec version 1. Represented type: `komorebi_math.coordinates.basis.CoordinateSystem`.

Name, kind and description pass through string conversion, while dimension passes through int. This is a coordinate-system identity and dimension record. No actual basis matrix is included in this tag.

Complete decoded encoder data. The fixture forms below are examples, not a closed type specification for arbitrary inputs. Each row is a required data key.

Field	Form in the finite fixture
`description`	`str`
`dimension`	`int`
`kind`	`str`
`name`	`str`

Read policy. Extra decoded data fields are rejected by the decoder. Exact field coverage does not imply strict numeric or boolean type checking. Missing fields were individually rejected in the version-bound tests. Extra outer tagged-wrapper fields and future versions were also rejected.

Canonical JSON and digest. Complete compact UTF-8 envelope: `block39-fixtures/coordinates.system.canonical.json`. The pretty fixture has the same payload. Payload SHA-256: `c82e71385cafdc6288a8153da292341d1af7fa63ed92893b5ee29f6c68a1669a`. Both forms reconstruct the represented type; the finite fixture re-encodes identically.

Migration. No older-version migration ships for this tag. Current version 1 is the supported frozen form; a future version is rejected.

Source binding. Encoder: `src/komorebi_math/coordinates/portable.py`, lines 426-431. Decoder: `src/komorebi_math/coordinates/portable.py`, lines 138-146. Represented class: `src/komorebi_math/coordinates/basis.py`, lines 34-51. Full source and fragment hashes are in `Block39_Portable_Schema_Records.json`.

G.7.22 `coordinates.transform`

Record G-C22. Codec version 1. Represented type: `komorebi_math.coordinates.transform.CoordinateTransform`.

The five fields reconstruct an exact CoordinateTransform. Matrix values use as_matrix and labels use str. The theorem string is carried metadata rather than a pointer to an executable proof checker.

Complete decoded encoder data. The fixture forms below are examples, not a closed type specification for arbitrary inputs. Each row is a required data key.

Field	Form in the finite fixture
`description`	`str`
`matrix`	`tuple[tuple; length=2]`
`source`	`str`
`target`	`str`
`theorem`	`str`

Read policy. Extra decoded data fields are rejected by the decoder. Exact field coverage does not imply strict numeric or boolean type checking. Missing fields were individually rejected in the version-bound tests. Extra outer tagged-wrapper fields and future versions were also rejected.

Canonical JSON and digest. Complete compact UTF-8 envelope: `block39-fixtures/coordinates.transform.canonical.json`. The pretty fixture has the same payload. Payload SHA-256: `20bf225a216c6ccbf8f71f3a1e5db64cf3cf621b8ac1dd47d41862f20dd364c3`. Both forms reconstruct the represented type; the finite fixture re-encodes identically.

Migration. No older-version migration ships for this tag. Current version 1 is the supported frozen form; a future version is rejected.

Source binding. Encoder: `src/komorebi_math/coordinates/portable.py`, lines 331-337. Decoder: `src/komorebi_math/coordinates/portable.py`, lines 48-57. Represented class: `src/komorebi_math/coordinates/transform.py`, lines 25-148. Full source and fragment hashes are in `Block39_Portable_Schema_Records.json`.

G.7.23 `core.certificate`

Record G-C23. Codec version 1. Represented type: `komorebi_math.certificates.Certificate`.

The loader copies a certificate record, converts valid with bool, and converts residuals and tolerance to floats. It does not re-run the underlying theorem or require the certificate. A stored false certificate round-trips as false. A malformed nonempty string such as "false" converts to true; the boundary test documents this permissive behavior rather than treating it as safe input validation.

Complete decoded encoder data. The fixture forms below are examples, not a closed type specification for arbitrary inputs. Each row is a required data key.

Field	Form in the finite fixture
`assumptions_checked`	`tuple[empty; length=0]`
`assumptions_unchecked`	`tuple[str; length=1]`
`diagnostics`	`mapping[0 entries]`
`exactness`	`str`
`message`	`str`
`residuals`	`mapping[1 entries]`
`theorem_id`	`str`
`tolerance`	`null`
`valid`	`bool`

Read policy. Extra decoded data fields are accepted and discarded. This codec does not check an exact data-key set. Missing fields were individually rejected in the version-bound tests. Extra outer tagged-wrapper fields and future versions were also rejected.

Canonical JSON and digest. Complete compact UTF-8 envelope: `block39-fixtures/core.certificate.canonical.json`. The pretty fixture has the same payload. Payload SHA-256: `ff8eb4772ed098b67bc2fc7d3e85d19a89382f8610fce3adc5a596b8fb7ffb19`. Both forms reconstruct the represented type; the finite fixture re-encodes identically.

Migration. No older-version migration ships for this tag. Current version 1 is the supported frozen form; a future version is rejected.

Source binding. Encoder: `src/komorebi_math/export/persistence.py`, lines 205-216. Decoder: `src/komorebi_math/export/persistence.py`, lines 219-230. Represented class: `src/komorebi_math/certificates.py`, lines 27-54. Full source and fragment hashes are in `Block39_Portable_Schema_Records.json`.

G.7.24 `deformation.lift_branch`

Record G-C24. Codec version 1. Represented type: `komorebi_math.deformation.forests.LiftBranch`.

Only the finite sequence of nodes is encoded. The decoder wraps it in LiftBranch. Extra data fields are ignored by this codec, and persistence does not construct an infinite lift or supply an omitted convergence argument.

Complete decoded encoder data. The fixture forms below are examples, not a closed type specification for arbitrary inputs. Each row is a required data key.

Field	Form in the finite fixture
`nodes`	`tuple[tuple; length=2]`

Read policy. Extra decoded data fields are accepted and discarded. This codec does not check an exact data-key set. Missing fields were individually rejected in the version-bound tests. Extra outer tagged-wrapper fields and future versions were also rejected.

Canonical JSON and digest. Complete compact UTF-8 envelope: `block39-fixtures/deformation.lift_branch.canonical.json`. The pretty fixture has the same payload. Payload SHA-256: `108f4c92d4aff3172b2c0be15721440822d90e1407e8b9ac6d1e39841f23a19a`. Both forms reconstruct the represented type; the finite fixture re-encodes identically.

Migration. No older-version migration ships for this tag. Current version 1 is the supported frozen form; a future version is rejected.

Source binding. Encoder: `src/komorebi_math/export/persistence.py`, lines 373-373. Decoder: `src/komorebi_math/export/persistence.py`, lines 374-374. Represented class: `src/komorebi_math/deformation/forests.py`, lines 16-26. Full source and fragment hashes are in `Block39_Portable_Schema_Records.json`.

G.7.25 `generating.basis_descriptor`

Record G-C25. Codec version 1. Represented type: `komorebi_math.generating.basis.BasisDescriptor`.

The decoder enforces exactly name, kind and parameters, requires a mapping of parameters, and calls BasisDescriptor.create. The descriptor names a controlled declarative basis form. Arbitrary serialized callback code is not supported.

Complete decoded encoder data. The fixture forms below are examples, not a closed type specification for arbitrary inputs. Each row is a required data key.

Field	Form in the finite fixture
`kind`	`str`
`name`	`str`
`parameters`	`mapping[0 entries]`

Read policy. Extra decoded data fields are rejected by the decoder. Exact field coverage does not imply strict numeric or boolean type checking. Missing fields were individually rejected in the version-bound tests. Extra outer tagged-wrapper fields and future versions were also rejected.

Canonical JSON and digest. Complete compact UTF-8 envelope: `block39-fixtures/generating.basis_descriptor.canonical.json`. The pretty fixture has the same payload. Payload SHA-256: `93ae0a2a81c50e563cb78fdd6f4af259ea6bae6c4aec3676a6d58c02d8e5601f`. Both forms reconstruct the represented type; the finite fixture re-encodes identically.

Migration. No older-version migration ships for this tag. Current version 1 is the supported frozen form; a future version is rejected.

Source binding. Encoder: `src/komorebi_math/generating/portable.py`, lines 346-350. Decoder: `src/komorebi_math/generating/portable.py`, lines 130-136. Represented class: `src/komorebi_math/generating/basis.py`, lines 22-62. Full source and fragment hashes are in `Block39_Portable_Schema_Records.json`.

G.7.26 `generating.budget`

Record G-C26. Codec version 1. Represented type: `komorebi_math.generating.budget.GeneratingBudget`.

All eight fields pass through int into GeneratingBudget. These stored limits become effective only where an operation explicitly checks them. In particular, maximum_serialized_bytes does not impose a global input-size limit on dumps_portable or loads_portable.

Complete decoded encoder data. The fixture forms below are examples, not a closed type specification for arbitrary inputs. Each row is a required data key.

Field	Form in the finite fixture
`maximum_coefficient_bits`	`int`
`maximum_degree`	`int`
`maximum_local_combinations`	`int`
`maximum_matrix_dimension`	`int`

Field	Form in the finite fixture
`maximum_modulus`	`int`
`maximum_reachable_states`	`int`
`maximum_recursion_depth`	`int`
`maximum_serialized_bytes`	`int`

Read policy. Extra decoded data fields are rejected by the decoder. Exact field coverage does not imply strict numeric or boolean type checking. Missing fields were individually rejected in the version-bound tests. Extra outer tagged-wrapper fields and future versions were also rejected.

Canonical JSON and digest. Complete compact UTF-8 envelope: `block39-fixtures/generating.budget.canonical.json`. The pretty fixture has the same payload. Payload SHA-256: `61397a505f1c39f55f5e49d4bb3223aef3e867968d84320e56b747da8980c093`. Both forms reconstruct the represented type; the finite fixture re-encodes identically.

Migration. No older-version migration ships for this tag. Current version 1 is the supported frozen form; a future version is rejected.

Source binding. Encoder: `src/komorebi_math/generating/portable.py`, lines 366-375. Decoder: `src/komorebi_math/generating/portable.py`, lines 145-158. Represented class: `src/komorebi_math/generating/budget.py`, lines 16-141. Full source and fragment hashes are in `Block39_Portable_Schema_Records.json`.

G.7.27 `generating.carry_layer_convolution`

Record G-C27. Codec version 1. Represented type: `komorebi_math.generating.masks.CarryLayerConvolution`.

The decoder enforces all six fields, checks modulus equals prime raised to exponent, and checks every layer sum is a canonical residue. The input sequences that produced the sums are absent. The decoder therefore does not independently recompute that convolution.

Complete decoded encoder data. The fixture forms below are examples, not a closed type specification for arbitrary inputs. Each row is a required data key.

Field	Form in the finite fixture
`exponent`	`int`
`factorial_power`	`int`
`index`	`int`
`layer_sums`	`tuple[int; length=2]`
`modulus`	`int`
`prime`	`int`

Read policy. Extra decoded data fields are rejected by the decoder. Exact field coverage does not imply strict numeric or boolean type checking. Missing fields were individually rejected in the version-bound tests. Extra outer tagged-wrapper fields and future versions were also rejected.

Canonical JSON and digest. Complete compact UTF-8 envelope: `block39-fixtures/generating.carry_layer_convolution.canonical.json`. The pretty fixture has the same payload. Payload SHA-256: `64dc2c8dd47ecf44daee6718837250508b902a92e4081e8f2e9be2d89d53fe7c`. Both forms reconstruct the represented type; the finite fixture re-encodes identically.

Migration. No older-version migration ships for this tag. Current version 1 is the supported frozen form; a future version is rejected.

Source binding. Encoder: `src/komorebi_math/generating/portable.py`, lines 477-484. Decoder: `src/komorebi_math/generating/portable.py`, lines 313-336. Represented class: `src/komorebi_math/generating/masks.py`, lines 516-529. Full source and fragment hashes are in `Block39_Portable_Schema_Records.json`.

G.7.28 `generating.cartier_automaton`

Record G-C28. Codec version 1. Represented type: `komorebi_math.generating.cartier.CartierAutomaton`.

After exact field matching and integer conversion, the decoder checks a nonempty transition table, valid start state, output count, digit-row lengths and in-range transition targets. These are finite structural checks. The defining rational-mask relation is not fully recertified by this decoder; run the appropriate automaton certificate separately.

Complete decoded encoder data. The fixture forms below are examples, not a closed type specification for arbitrary inputs. Each row is a required data key.

Field	Form in the finite fixture
`degree_bound`	`int`
`denominator`	`tuple[int; length=2]`
`numerator`	`tuple[int; length=2]`
`outputs`	`tuple[int; length=5]`
`prime`	`int`
`start_state`	`int`
`state_members`	`tuple[tuple; length=5]`
`state_numerators`	`tuple[tuple; length=5]`
`transitions`	`tuple[tuple; length=5]`

Read policy. Extra decoded data fields are rejected by the decoder. Exact field coverage does not imply strict numeric or boolean type checking. Missing fields were individually rejected in the version-bound tests. Extra outer tagged-wrapper fields and future versions were also rejected.

Canonical JSON and digest. Complete compact UTF-8 envelope: `block39-fixtures/generating.cartier_automaton.canonical.json`. The pretty fixture has the same payload. Payload SHA-256: `1a96f38dbf1b620b5ad522ed084fce2ebec408d4c4dca01438f953582dc9fd44`. Both forms reconstruct the represented type; the finite fixture re-encodes identically.

Migration. No older-version migration ships for this tag. Current version 1 is the supported frozen form; a future version is rejected.

Source binding. Encoder: `src/komorebi_math/generating/portable.py`, lines 430-440. Decoder: `src/komorebi_math/generating/portable.py`, lines 214-255. Represented class: `src/komorebi_math/generating/cartier.py`, lines 155-211. Full source and fragment hashes are in `Block39_Portable_Schema_Records.json`.

G.7.29 `generating.cartier_linear_representation`

Record G-C29. Codec version 1. Represented type: `komorebi_math.generating.cartier.CartierLinearRepresentation`.

The decoder checks exact fields, the start-vector dimension, one transition matrix per radix digit and square matrix dimensions. It preserves the compiled representation. These checks alone are not an independent derivation of the matrices from numerator and denominator.

Complete decoded encoder data. The fixture forms below are examples, not a closed type specification for arbitrary inputs. Each row is a required data key.

Field	Form in the finite fixture
`degree_bound`	`int`
`denominator`	`tuple[int; length=2]`
`numerator`	`tuple[int; length=2]`
`output_scale`	`int`
`prime`	`int`
`start_vector`	`tuple[int; length=2]`
`transition_matrices`	`tuple[tuple; length=3]`

Read policy. Extra decoded data fields are rejected by the decoder. Exact field coverage does not imply strict numeric or boolean type checking. Missing fields were individually rejected in the version-bound tests. Extra outer tagged-wrapper fields and future versions were also rejected.

Canonical JSON and digest. Complete compact UTF-8 envelope: `block39-fixtures/generating.cartier_linear_representation.canonical.json`. The pretty fixture has the same payload. Payload SHA-256: `b01776df61e88540ef7624c0537e4433f36ac8e4be68e627166e3ed1eee12419`. Both forms reconstruct the represented type; the finite fixture re-encodes identically.

Migration. No older-version migration ships for this tag. Current version 1 is the supported frozen form; a future version is rejected.

Source binding. Encoder: `src/komorebi_math/generating/portable.py`, lines 449-457. Decoder: `src/komorebi_math/generating/portable.py`, lines 259-296. Represented class: `src/komorebi_math/generating/cartier.py`, lines 432-481. Full source and fragment hashes are in `Block39_Portable_Schema_Records.json`.

G.7.30 `generating.finite_mask_carry_plan`

Record G-C30. Codec version 1. Represented type: `komorebi_math.generating.masks.FiniteMaskCarryPlan`.

Only radix and mask are serialized. The carry bound and matrices are deliberately not encoded. The decoder re-runs compile_finite_mask_carry_plan, regenerating derived state using the implementation defaults. This is a reconstruction-by-compilation codec, not a field-for-field dump of the Python object.

Complete decoded encoder data. The fixture forms below are examples, not a closed type specification for arbitrary inputs. Each row is a required data key.

Field	Form in the finite fixture
`mask`	`tuple[Fraction; length=3]`
`radix`	`int`

Read policy. Extra decoded data fields are rejected by the decoder. Exact field coverage does not imply strict numeric or boolean type checking. Missing fields were individually rejected in the version-bound tests. Extra outer tagged-wrapper fields and future versions were also rejected.

Canonical JSON and digest. Complete compact UTF-8 envelope: `block39-fixtures/generating.finite_mask_carry_plan.canonical.json`. The pretty fixture has the same payload. Payload SHA-256: `32b3461540b5d8e5b7cd208ece5b047941afdf635dc583a92bf41f88ff3774d9`. Both forms reconstruct the represented type; the finite fixture re-encodes identically.

Migration. No older-version migration ships for this tag. Current version 1 is the supported frozen form; a future version is rejected.

Source binding. Encoder: `src/komorebi_math/generating/portable.py`, lines 422-422. Decoder: `src/komorebi_math/generating/portable.py`, lines 298-301. Represented class: `src/komorebi_math/generating/masks.py`, lines 195-270. Full source and fragment hashes are in `Block39_Portable_Schema_Records.json`.

G.7.31 `generating.finite_normalized_series`

Record G-C31. Codec version 1. Represented type: `komorebi_math.generating.finite_series.FiniteNormalizedSeries`.

Basis must be a BasisDescriptor, domain a descriptor mapping and budget a GeneratingBudget. The decoder reconstructs the declared coefficient domain and finite series. Domain normalization and series construction remain responsible for coefficient validity and relevant resource checks.

Complete decoded encoder data. The fixture forms below are examples, not a closed type specification for arbitrary inputs. Each row is a required data key.

Field	Form in the finite fixture
`basis`	`komorebi_math.generating.basis.BasisDescriptor`

Field	Form in the finite fixture
budget	komorebi_math.generating.budget.GeneratingBudget
coefficients	tuple[int; length=3]
domain	mapping[2 entries]

Read policy. Extra decoded data fields are rejected by the decoder. Exact field coverage does not imply strict numeric or boolean type checking. Missing fields were individually rejected in the version-bound tests. Extra outer tagged-wrapper fields and future versions were also rejected.

Canonical JSON and digest. Complete compact UTF-8 envelope: `block39-fixtures/generating.finite_normalized_series.canonical.json`. The pretty fixture has the same payload. Payload SHA-256: `54dc118e4a9cb6c12f62824bd3981e749fa846c18b9caa57d514a3dbffe0b8c4`. Both forms reconstruct the represented type; the finite fixture re-encodes identically.

Migration. No older-version migration ships for this tag. Current version 1 is the supported frozen form; a future version is rejected.

Source binding. Encoder: `src/komorebi_math/generating/portable.py`, lines 394-399. Decoder: `src/komorebi_math/generating/portable.py`, lines 172-189. Represented class: `src/komorebi_math/generating/finite_series.py`, lines 93-448. Full source and fragment hashes are in `Block39_Portable_Schema_Records.json`.

G.7.32 `generating.formal_series`

Record G-C32. Codec version 1. Represented type: `komorebi_math.generating.series.FormalSeries`.

The decoder requires a basis descriptor and a nonempty exact coefficient list. A callback-defined basis embedded in a finite series can be replaced by a finite table descriptor. This captures only the requested degree prefix, not the original callable.

Complete decoded encoder data. The fixture forms below are examples, not a closed type specification for arbitrary inputs. Each row is a required data key.

Field	Form in the finite fixture
basis	komorebi_math.generating.basis.BasisDescriptor
coefficients	tuple[Fraction; length=3]

Read policy. Extra decoded data fields are rejected by the decoder. Exact field coverage does not imply strict numeric or boolean type checking. Missing fields were individually rejected in the version-bound tests. Extra outer tagged-wrapper fields and future versions were also rejected.

Canonical JSON and digest. Complete compact UTF-8 envelope: `block39-fixtures/generating.formal_series.canonical.json`. The pretty fixture has the same payload. Payload SHA-256: `5972ed8219f7240c1d59ea68e001fee138bbfd5c033749493d7259b426a9a867`. Both forms reconstruct the represented type; the finite fixture re-encodes identically.

Migration. No older-version migration ships for this tag. Current version 1 is the supported frozen form; a future version is rejected.

Source binding. Encoder: `src/komorebi_math/generating/portable.py`, lines 383-386. Decoder: `src/komorebi_math/generating/portable.py`, lines 161-169. Represented class: `src/komorebi_math/generating/series.py`, lines 52-321. Full source and fragment hashes are in `Block39_Portable_Schema_Records.json`.

G.7.33 `generating.frozen_function`

Record G-C33. Codec version 1. Represented type: `komorebi_math.generating.portable.FrozenGeneratingFunction`.

The frozen function stores a name, basis name, finite normalization, finite coefficients, source kind and portable metadata. Constructor checks include equal lengths, a leading normalization of one and nonzero normalization values. Its series method rejects degrees beyond the capture. The original provider callback is not restored.

Complete decoded encoder data. The fixture forms below are examples, not a closed type specification for arbitrary inputs. Each row is a required data key.

Field	Form in the finite fixture
`basis_name`	`str`
`coefficients`	`tuple[Fraction; length=5]`
`metadata`	`mapping[2 entries]`
`name`	`str`
`normalization`	`tuple[Fraction; length=5]`
`source_kind`	`str`

Read policy. Extra decoded data fields are rejected by the decoder. Exact field coverage does not imply strict numeric or boolean type checking. Missing fields were individually rejected in the version-bound tests. Extra outer tagged-wrapper fields and future versions were also rejected.

Canonical JSON and digest. Complete compact UTF-8 envelope: `block39-fixtures/generating.frozen_function.canonical.json`. The pretty fixture has the same payload. Payload SHA-256: `2b0e4024ae31f72fa91e09de6ea6d2828fda8259eb5592c1702e42f48f893a63`. Both forms reconstruct the represented type; the finite fixture re-encodes identically.

Migration. No older-version migration ships for this tag. Current version 1 is the supported frozen form; a future version is rejected.

Source binding. Encoder: `src/komorebi_math/generating/portable.py`, lines 407-414. Decoder: `src/komorebi_math/generating/portable.py`, lines 191-211. Represented class: `src/komorebi_math/generating/portable.py`, lines 32-87. Full source and fragment hashes are in `Block39_Portable_Schema_Records.json`.

G.7.34 `generating.kummer_layer_distribution`

Record G-C34. Codec version 1. Represented type: `komorebi_math.generating.masks.KummerLayerDistribution`.

The decoder recomputes kummer_layer_distribution from the stored index and prime, then compares the supplied counts against that result. This is a stronger finite check than merely accepting the stored count vector.

Complete decoded encoder data. The fixture forms below are examples, not a closed type specification for arbitrary inputs. Each row is a required data key.

Field	Form in the finite fixture
`counts`	`tuple[int; length=1]`
`index`	`int`
`prime`	`int`

Read policy. Extra decoded data fields are rejected by the decoder. Exact field coverage does not imply strict numeric or boolean type checking. Missing fields were individually rejected in the version-bound tests. Extra outer tagged-wrapper fields and future versions were also rejected.

Canonical JSON and digest. Complete compact UTF-8 envelope: `block39-fixtures/generating.kummer_layer_distribution.canonical.json`. The pretty fixture has the same payload. Payload SHA-256: `746bba8b87dd1a8aca5fcc8f101c063889255a4a1fb14bb0a74e74507267f324`. Both forms reconstruct the represented type; the finite fixture re-encodes identically.

Migration. No older-version migration ships for this tag. Current version 1 is the supported frozen form; a future version is rejected.

Source binding. Encoder: `src/komorebi_math/generating/portable.py`, lines 465-469. Decoder: `src/komorebi_math/generating/portable.py`, lines 304-310. Represented class: `src/komorebi_math/generating/masks.py`, lines 384-419. Full source and fragment hashes are in `Block39_Portable_Schema_Records.json`.

G.7.35 `generating.weighted_monomial_basis`

Record G-C35. Codec version 1. Represented type: `komorebi_math.generating.basis.WeightedMonomialBasis`.

Only a BasisDescriptor is encoded. A raw callback basis without a descriptor is rejected by require_descriptor with ValueError. A declared finite table descriptor or supported declarative basis is reconstructed through the closed basis factory.

Complete decoded encoder data. The fixture forms below are examples, not a closed type specification for arbitrary inputs. Each row is a required data key.

Field	Form in the finite fixture
`descriptor`	`komorebi_math.generating.basis.BasisDescriptor`

Read policy. Extra decoded data fields are rejected by the decoder. Exact field coverage does not imply strict numeric or boolean type checking. Missing fields were individually rejected in the version-bound tests. Extra outer tagged-wrapper fields and future versions were also rejected.

Canonical JSON and digest. Complete compact UTF-8 envelope: `block39-fixtures/generating.weighted_monomial_basis.canonical.json`. The pretty fixture has the same payload. Payload SHA-256: `2f0dd6f843b4b22ca0e1586676c09e7436c98890dcdf37d4bb0b4b5a9b927ecc`. Both forms reconstruct the represented type; the finite fixture re-encodes identically.

Migration. No older-version migration ships for this tag. Current version 1 is the supported frozen form; a future version is rejected.

Source binding. Encoder: `src/komorebi_math/generating/portable.py`, lines 358-358. Decoder: `src/komorebi_math/generating/portable.py`, lines 139-142. Represented class: `src/komorebi_math/generating/basis.py`, lines 77-245. Full source and fragment hashes are in `Block39_Portable_Schema_Records.json`.

G.7.36 `nonlinear.filtered_spectral_sequence_result`

Record G-C36. Codec version 1. Represented type: `komorebi_math.nonlinear.filtered.FilteredSpectralSequenceResult`.

Pages are nested records. The two degree-to-dimension mappings have integer keys and values after decoding, and the certificate is retained. The original filtered complex and differential are not stored in this result codec, so reconstruction is not a new spectral-sequence computation.

Complete decoded encoder data. The fixture forms below are examples, not a closed type specification for arbitrary inputs. Each row is a required data key.

Field	Form in the finite fixture
`certificate`	`komorebi_math.certificates.Certificate`
`limiting_dimensions`	`mapping[1 entries]`
`pages`	`tuple[SpectralSequencePage; length=1]`
`total_cohomology_dimensions`	`mapping[1 entries]`

Read policy. Extra decoded data fields are accepted and discarded. This codec does not check an exact data-key set. Missing fields were individually rejected in the version-bound tests. Extra outer tagged-wrapper fields and future versions were also rejected.

Canonical JSON and digest. Complete compact UTF-8 envelope: `block39-fixtures/nonlinear.filtered_spectral_sequence_result.canonical.json`. The pretty fixture has the same payload. Payload SHA-256: `8502c4c637162187bdf7165e7ab6ec8dfe9d6a9edc1269f01c8567af8c3d9866`. Both forms reconstruct the represented type; the finite fixture re-encodes identically.

Migration. No older-version migration ships for this tag. Current version 1 is the supported frozen form; a future version is rejected.

Source binding. Encoder: `src/komorebi_math/export/persistence.py`, lines 449-454. Decoder: `src/komorebi_math/export/persistence.py`, lines 455-460. Represented class: `src/komorebi_math/nonlinear/filtered.py`, lines 599-607. Full source and fragment hashes are in `Block39_Portable_Schema_Records.json`.

G.7.37 `nonlinear.finite_lift_tree`

Record G-C37. Codec version 1. Represented type: `komorebi_math.nonlinear.lifting.FiniteLiftTree`.

Prime and finite levels are retained; nodes are nested LiftTreeNode records. The result is a finite tree, not an infinite compatible lifting tower. Extra fields are ignored by this decoder.

Complete decoded encoder data. The fixture forms below are examples, not a closed type specification for arbitrary inputs. Each row is a required data key.

Field	Form in the finite fixture
`levels`	`tuple[tuple; length=2]`
`prime`	`int`

Read policy. Extra decoded data fields are accepted and discarded. This codec does not check an exact data-key set. Missing fields were individually rejected in the version-bound tests. Extra outer tagged-wrapper fields and future versions were also rejected.

Canonical JSON and digest. Complete compact UTF-8 envelope: `block39-fixtures/nonlinear.finite_lift_tree.canonical.json`. The pretty fixture has the same payload. Payload SHA-256: `86abe3f2781a838b6708638842bfc441544377b669824a744e3179460a05705a`. Both forms reconstruct the represented type; the finite fixture re-encodes identically.

Migration. No older-version migration ships for this tag. Current version 1 is the supported frozen form; a future version is rejected.

Source binding. Encoder: `src/komorebi_math/export/persistence.py`, lines 397-397. Decoder: `src/komorebi_math/export/persistence.py`, lines 398-398. Represented class: `src/komorebi_math/nonlinear/lifting.py`, lines 385-417. Full source and fragment hashes are in `Block39_Portable_Schema_Records.json`.

G.7.38 `nonlinear.lift_tree_node`

Record G-C38. Codec version 1. Represented type: `komorebi_math.nonlinear.lifting.LiftTreeNode`.

Value coordinates, depth and a non-null parent_index are converted to integers. The parent may be null for a root. This record alone has no parent tree against which every link can be checked.

Complete decoded encoder data. The fixture forms below are examples, not a closed type specification for arbitrary inputs. Each row is a required data key.

Field	Form in the finite fixture
`depth`	`int`
`parent_index`	`null`
`value`	`tuple[int; length=1]`

Read policy. Extra decoded data fields are accepted and discarded. This codec does not check an exact data-key set. Missing fields were individually rejected in the version-bound tests. Extra outer tagged-wrapper fields and future versions were also rejected.

Canonical JSON and digest. Complete compact UTF-8 envelope: `block39-fixtures/nonlinear.lift_tree_node.canonical.json`. The pretty fixture has the same payload. Payload SHA-256: `61b4dfc45b61f1c49cb642a6977db5808e43ad017862b08dd4df18e655b2150f`. Both forms reconstruct the represented type; the finite fixture re-encodes identically.

Migration. No older-version migration ships for this tag. Current version 1 is the supported frozen form; a future version is rejected.

Source binding. Encoder: `src/komorebi_math/export/persistence.py`, lines 381-385. Decoder: `src/komorebi_math/export/persistence.py`, lines 386-390. Represented class: `src/komorebi_math/nonlinear/lifting.py`, lines 376-382. Full source and fragment hashes are in `Block39_Portable_Schema_Records.json`.

G.7.39 `nonlinear.ordered_rooted_tree`

Record G-C39. Codec version 1. Represented type: `komorebi_math.nonlinear.trees.OrderedRootedTree`.

Children are recursively encoded tree records and reconstructed as a tuple of children. This represents a finite ordered rooted tree. The generic recursion-depth limit still applies to deeply nested trees.

Complete decoded encoder data. The fixture forms below are examples, not a closed type specification for arbitrary inputs. Each row is a required data key.

Field	Form in the finite fixture
`children`	`tuple[OrderedRootedTree; length=2]`

Read policy. Extra decoded data fields are accepted and discarded. This codec does not check an exact data-key set. Missing fields were individually rejected in the version-bound tests. Extra outer tagged-wrapper fields and future versions were also rejected.

Canonical JSON and digest. Complete compact UTF-8 envelope: `block39-fixtures/nonlinear.ordered_rooted_tree.canonical.json`. The pretty fixture has the same payload. Payload SHA-256: `d91880c6db4ef74758d79b474ea8866ac3fe110591544b5d2fe32ab564cf6533`. Both forms reconstruct the represented type; the finite fixture re-encodes identically.

Migration. No older-version migration ships for this tag. Current version 1 is the supported frozen form; a future version is rejected.

Source binding. Encoder: `src/komorebi_math/export/persistence.py`, lines 282-282. Decoder: `src/komorebi_math/export/persistence.py`, lines 283-283. Represented class: `src/komorebi_math/nonlinear/trees.py`, lines 97-174. Full source and fragment hashes are in `Block39_Portable_Schema_Records.json`.

G.7.40 `nonlinear.packet_jet_address`

Record G-C40. Codec version 1. Represented type: `komorebi_math.nonlinear.jets.PacketJetAddress`.

The output label is converted to str and input labels to a tuple. The address describes an ordered packet block. It contains neither projection matrices nor a derivative provider.

Complete decoded encoder data. The fixture forms below are examples, not a closed type specification for arbitrary inputs. Each row is a required data key.

Field	Form in the finite fixture
`inputs`	`tuple[str; length=2]`
`output`	`str`

Read policy. Extra decoded data fields are accepted and discarded. This codec does not check an exact data-key set. Missing fields were individually rejected in the version-bound tests. Extra outer tagged-wrapper fields and future versions were also rejected.

Canonical JSON and digest. Complete compact UTF-8 envelope: `block39-fixtures/nonlinear.packet_jet_address.canonical.json`. The pretty fixture has the same payload. Payload SHA-256: `2860344375e9f1f64972222b4afb6359499de8ff630886ffcd6d7ad994cfb29b`. Both forms reconstruct the represented type; the finite fixture re-encodes identically.

Migration. No older-version migration ships for this tag. Current version 1 is the supported frozen form; a future version is rejected.

Source binding. Encoder: `src/komorebi_math/export/persistence.py`, lines 290-290. Decoder: `src/komorebi_math/export/persistence.py`, lines 291-291. Represented class: `src/komorebi_math/nonlinear/jets.py`, lines 55-67. Full source and fragment hashes are in `Block39_Portable_Schema_Records.json`.

G.7.41 `nonlinear.packet_jet_coefficient`

Record G-C41. Codec version 1. Represented type: `komorebi_math.nonlinear.snapshots.PacketJetCoefficient`.

Indices and value become tuples. Their association with a source dimension, target dimension and jet order is checked only when an enclosing snapshot or stream supplies that context. A standalone coefficient record is not that enclosing contract.

Complete decoded encoder data. The fixture forms below are examples, not a closed type specification for arbitrary inputs. Each row is a required data key.

Field	Form in the finite fixture
`indices`	`tuple[int; length=2]`
`value`	`tuple[Fraction; length=1]`

Read policy. Extra decoded data fields are accepted and discarded. This codec does not check an exact data-key set. Missing fields were individually rejected in the version-bound tests. Extra outer tagged-wrapper fields and future versions were also rejected.

Canonical JSON and digest. Complete compact UTF-8 envelope: `block39-fixtures/nonlinear.packet_jet_coefficient.canonical.json`. The pretty fixture has the same payload. Payload SHA-256: `33d3bf06c16b86763678827acfc5deafb304d324f9dc661a1365f6f141daedf3`. Both forms reconstruct the represented type; the finite fixture re-encodes identically.

Migration. No older-version migration ships for this tag. Current version 1 is the supported frozen form; a future version is rejected.

Source binding. Encoder: `src/komorebi_math/export/persistence.py`, lines 298-298. Decoder: `src/komorebi_math/export/persistence.py`, lines 299-299. Represented class: `src/komorebi_math/nonlinear/snapshots.py`, lines 71-76. Full source and fragment hashes are in `Block39_Portable_Schema_Records.json`.

G.7.42 `nonlinear.packet_jet_snapshot`

Record G-C42. Codec version 2. Represented type: `komorebi_math.nonlinear.snapshots.PacketJetSnapshot`.

Version 2 stores provider under the key provider and requires coefficient_ordering to equal lexicographic. Address, dimensions, point, sparse coefficients, exactness and capture_certificate define a fixed finite snapshot. Version 1 used provider_name and no ordering field; the sole built-in migration supplies the current representation. The captured certificate is retained, not independently re-proved.

Complete decoded encoder data. The fixture forms below are examples, not a closed type specification for arbitrary inputs. Each row is a required data key.

Field	Form in the finite fixture
`address`	`komorebi_math.nonlinear.jets.PacketJetAddress`

Field	Form in the finite fixture
capture_certificate	komorebi_math.certificates.Certificate
coefficient_ordering	str
coefficients	tuple[PacketJetCoefficient; length=1]
exactness	str
point	tuple[int; length=2]
provider	str
source_dimension	int
target_dimension	int

Read policy. Extra decoded data fields are accepted and discarded. This codec does not check an exact data-key set. Missing fields were individually rejected in the version-bound tests. Extra outer tagged-wrapper fields and future versions were also rejected.

Canonical JSON and digest. Complete compact UTF-8 envelope: `block39-fixtures/nonlinear.packet_jet_snapshot.canonical.json`. The pretty fixture has the same payload. Payload SHA-256: `aeec4f5e898932b67a9bb04f3d1fb99d088baef324db101554efcb68eb2cb559`. Both forms reconstruct the represented type; the finite fixture re-encodes identically.

Migration. Version 1 is accepted only through the committed 1-to-2 migration described in G.5. Re-encoding uses version 2.

Source binding. Encoder: `src/komorebi_math/export/persistence.py`, lines 306-316. Decoder: `src/komorebi_math/export/persistence.py`, lines 233-250. Represented class: `src/komorebi_math/nonlinear/snapshots.py`, lines 79-277. Full source and fragment hashes are in `Block39_Portable_Schema_Records.json`.

G.7.43 `nonlinear.sparse_symmetric_tensor`

Record G-C43. Codec version 1. Represented type: `komorebi_math.nonlinear.sparse.SparseSymmetricTensor`.

The tensor stores order, dimensions and finite entries. Each entry carries output, inputs and value. Index fields are converted to integers before construction. It is a finite sparse tensor, not a callback or an unbounded multilinear operator.

Complete decoded encoder data. The fixture forms below are examples, not a closed type specification for arbitrary inputs. Each row is a required data key.

Field	Form in the finite fixture
entries	tuple[dict; length=1]
order	int
source_dimension	int
target_dimension	int

Read policy. Extra decoded data fields are accepted and discarded. This codec does not check an exact data-key set. Missing fields were individually rejected in the version-bound tests. Extra outer tagged-wrapper fields and future versions were also rejected.

Canonical JSON and digest. Complete compact UTF-8 envelope: `block39-fixtures/nonlinear.sparse_symmetric_tensor.canonical.json`. The pretty fixture has the same payload. Payload SHA-256: `a3558542088fbbfc0efb3caa9924e49715ac58916467f7962844dc23ee5ec354`. Both forms reconstruct the represented type; the finite fixture re-encodes identically.

Migration. No older-version migration ships for this tag. Current version 1 is the supported frozen form; a future version is rejected.

Source binding. Encoder: `src/komorebi_math/export/persistence.py`, lines 341-353. Decoder: `src/komorebi_math/export/persistence.py`, lines 354-366. Represented class: `src/komorebi_math/nonlinear/sparse.py`, lines 98-341. Full source and fragment hashes are in `Block39_Portable_Schema_Records.json`.

G.7.44 `nonlinear.spectral_sequence_entry`

Record G-C44. Codec version 1. Represented type: `komorebi_math.nonlinear.filtered.SpectralSequenceEntry`.

All seven page/degree/dimension/rank fields are converted to integers. The record is the reported entry of a finite page. It does not contain differentials needed to recompute ranks.

Complete decoded encoder data. The fixture forms below are examples, not a closed type specification for arbitrary inputs. Each row is a required data key.

Field	Form in the finite fixture
`complementary_degree`	`int`
`dimension`	`int`
`filtration_degree`	`int`
`incoming_rank`	`int`
`outgoing_rank`	`int`
`page`	`int`
`total_degree`	`int`

Read policy. Extra decoded data fields are accepted and discarded. This codec does not check an exact data-key set. Missing fields were individually rejected in the version-bound tests. Extra outer tagged-wrapper fields and future versions were also rejected.

Canonical JSON and digest. Complete compact UTF-8 envelope: `block39-fixtures/nonlinear.spectral_sequence_entry.canonical.json`. The pretty fixture has the same payload. Payload SHA-256: `471129223f38760b451ee61f80ce0b80c997e00d79a0ebb15a392908fa1e361a`. Both forms reconstruct the represented type; the finite fixture re-encodes identically.

Migration. No older-version migration ships for this tag. Current version 1 is the supported frozen form; a future version is rejected.

Source binding. Encoder: `src/komorebi_math/export/persistence.py`, lines 405-413. Decoder: `src/komorebi_math/export/persistence.py`, lines 414-422. Represented class: `src/komorebi_math/nonlinear/filtered.py`, lines 559-570. Full source and fragment hashes are in `Block39_Portable_Schema_Records.json`.

G.7.45 `nonlinear.spectral_sequence_page`

Record G-C45. Codec version 1. Represented type: `komorebi_math.nonlinear.filtered.SpectralSequencePage`.

The page number, total dimension and total differential rank become integers; entries and certificate are nested records. Loading does not rebuild the original filtered complex. Additional data fields are ignored.

Complete decoded encoder data. The fixture forms below are examples, not a closed type specification for arbitrary inputs. Each row is a required data key.

Field	Form in the finite fixture
`certificate`	`komorebi_math.certificates.Certificate`
`entries`	`tuple[SpectralSequenceEntry; length=1]`
`page`	`int`
`total_differential_rank`	`int`
`total_dimension`	`int`

Read policy. Extra decoded data fields are accepted and discarded. This codec does not check an exact data-key set. Missing fields were individually rejected in the version-bound tests. Extra outer tagged-wrapper fields and future versions were also rejected.

Canonical JSON and digest. Complete compact UTF-8 envelope: `block39-fixtures/nonlinear.spectral_sequence_page.canonical.json`. The pretty fixture has the same payload. Payload SHA-256: `8fd4ea1f41e81e28038cce207cff3023`

82fac32fba4c50c9418817f073784e7c. Both forms reconstruct the represented type; the finite fixture re-encodes identically.

Migration. No older-version migration ships for this tag. Current version 1 is the supported frozen form; a future version is rejected.

Source binding. Encoder: src/komorebi_math/export/persistence.py, lines 429-435. Decoder: src/komorebi_math/export/persistence.py, lines 436-442. Represented class: src/komorebi_math/nonlinear/filtered.py, lines 573-596. Full source and fragment hashes are in Block39_Portable_Schema_Records.json.

G.7.46 refinement.lattice_polyphase

Record G-C46. Codec version 1. Represented type: komorebi_math.refinement.polyphase.LatticePolyphaseRefinement.

Coarse and fine stride tuples are converted to integers. The constructor enforces the finite refinement relation. Persisting the refinement records an address-system relationship rather than materializing all refined signal data.

Complete decoded encoder data. The fixture forms below are examples, not a closed type specification for arbitrary inputs. Each row is a required data key.

Field	Form in the finite fixture
coarse_strides	tuple[int; length=2]
fine_strides	tuple[int; length=2]

Read policy. Extra decoded data fields are rejected by the decoder. Exact field coverage does not imply strict numeric or boolean type checking. Missing fields were individually rejected in the version-bound tests. Extra outer tagged-wrapper fields and future versions were also rejected.

Canonical JSON and digest. Complete compact UTF-8 envelope: block39-fixtures/refinement.lattice_polyphase.canonical.json. The pretty fixture has the same payload. Payload SHA-256: 91a98988d0e04d16294337da933f4f77b46f693b0641295c33871b249a9a8d22. Both forms reconstruct the represented type; the finite fixture re-encodes identically.

Migration. No older-version migration ships for this tag. Current version 1 is the supported frozen form; a future version is rejected.

Source binding. Encoder: src/komorebi_math/refinement/portable.py, lines 168-171. Decoder: src/komorebi_math/refinement/portable.py, lines 84-90. Represented class: src/komorebi_math/refinement/polyphase.py, lines 670-914. Full source and fragment hashes are in Block39_Portable_Schema_Records.json.

G.7.47 refinement.polyphase

Record G-C47. Codec version 1. Represented type: komorebi_math.refinement.polyphase.PolyphaseRefinement.

The two orders define the finite coarse-to-fine refinement, with divisibility and positive-order conditions delegated to the constructor. No signal values are included in this tag.

Complete decoded encoder data. The fixture forms below are examples, not a closed type specification for arbitrary inputs. Each row is a required data key.

Field	Form in the finite fixture
coarse_order	int
fine_order	int

Read policy. Extra decoded data fields are rejected by the decoder. Exact field coverage does not imply strict numeric or boolean type checking. Missing fields were individually rejected in the version-bound tests. Extra outer tagged-wrapper fields and future versions were also rejected.

Canonical JSON and digest. Complete compact UTF-8 envelope: `block39-fixtures/refinement.polyphase.canonical.json`. The pretty fixture has the same payload. Payload SHA-256: `40542e7310863ff33eda20906a31533e0483d24a5363f18ff2c64c847dee0e8f`. Both forms reconstruct the represented type; the finite fixture re-encodes identically.

Migration. No older-version migration ships for this tag. Current version 1 is the supported frozen form; a future version is rejected.

Source binding. Encoder: `src/komorebi_math/refinement/portable.py`, lines 149-152. Decoder: `src/komorebi_math/refinement/portable.py`, lines 73-76. Represented class: `src/komorebi_math/refinement/polyphase.py`, lines 129-494. Full source and fragment hashes are in `Block39_Portable_Schema_Records.json`.

G.7.48 `refinement.polyphase_tower`

Record G-C48. Codec version 1. Represented type: `komorebi_math.refinement.polyphase.PolyphaseRefinementTower`.

The ordered refinement levels are an integer tuple. The tower is finite and does not assert an infinite limiting refinement. Its ordering and adjacent relation are constructor-level concerns.

Complete decoded encoder data. The fixture forms below are examples, not a closed type specification for arbitrary inputs. Each row is a required data key.

Field	Form in the finite fixture
`orders`	`tuple[int; length=3]`

Read policy. Extra decoded data fields are rejected by the decoder. Exact field coverage does not imply strict numeric or boolean type checking. Missing fields were individually rejected in the version-bound tests. Extra outer tagged-wrapper fields and future versions were also rejected.

Canonical JSON and digest. Complete compact UTF-8 envelope: `block39-fixtures/refinement.polyphase_tower.canonical.json`. The pretty fixture has the same payload. Payload SHA-256: `b1112a304e55a98af630be3169e2b53846f362d4bb4e8ce8d64643896b79598f`. Both forms reconstruct the represented type; the finite fixture re-encodes identically.

Migration. No older-version migration ships for this tag. Current version 1 is the supported frozen form; a future version is rejected.

Source binding. Encoder: `src/komorebi_math/refinement/portable.py`, lines 160-160. Decoder: `src/komorebi_math/refinement/portable.py`, lines 79-81. Represented class: `src/komorebi_math/refinement/polyphase.py`, lines 497-667. Full source and fragment hashes are in `Block39_Portable_Schema_Records.json`.

G.7.49 `refinement.residue_coordinates`

Record G-C49. Codec version 1. Represented type: `komorebi_math.refinement.residue.ResidueCoordinates`.

The modulus and finite length become integers; channels remain portable numeric arrays. The represented finite sequence can be reconstructed from those lanes. Original array strides, memory sharing and device placement are not persistence guarantees.

Complete decoded encoder data. The fixture forms below are examples, not a closed type specification for arbitrary inputs. Each row is a required data key.

Field	Form in the finite fixture
`channels`	`tuple[ndarray; length=3]`
`length`	`int`
`modulus`	`int`

Read policy. Extra decoded data fields are rejected by the decoder. Exact field coverage does not imply strict numeric or boolean type checking. Missing fields were individually rejected in the version-bound tests. Extra outer tagged-wrapper fields and future versions were also rejected.

Canonical JSON and digest. Complete compact UTF-8 envelope: `block39-fixtures/refinement.residue_coordinates.canonical.json`. The pretty fixture has the same payload. Payload SHA-256: `6b8b954c099b99c53e0bb4508a1e0ae2a1644d2ec0cce90bd9cbaef62cf88b16`. Both forms reconstruct the represented type; the finite fixture re-encodes identically.

Migration. No older-version migration ships for this tag. Current version 1 is the supported frozen form; a future version is rejected.

Source binding. Encoder: `src/komorebi_math/refinement/portable.py`, lines 125-129. Decoder: `src/komorebi_math/refinement/portable.py`, lines 53-60. Represented class: `src/komorebi_math/refinement/residue.py`, lines 55-102. Full source and fragment hashes are in `Block39_Portable_Schema_Records.json`.

G.7.50 `signals.lattice_polyphase_coordinates`

Record G-C50. Codec version 1. Represented type: `komorebi_math.signals.polyphase.LatticePolyphaseCoordinates`.

Channels are a mapping keyed by integer address tuples. Strides and original shape are converted to integer tuples. Reconstruction is finite; sparse addresses, shapes and channel values are interpreted by LatticePolyphaseCoordinates, not by an implicit external lattice source.

Complete decoded encoder data. The fixture forms below are examples, not a closed type specification for arbitrary inputs. Each row is a required data key.

Field	Form in the finite fixture
`channels`	`mapping[4 entries]`
`original_shape`	`tuple[int; length=2]`
`strides`	`tuple[int; length=2]`

Read policy. Extra decoded data fields are rejected by the decoder. Exact field coverage does not imply strict numeric or boolean type checking. Missing fields were individually rejected in the version-bound tests. Extra outer tagged-wrapper fields and future versions were also rejected.

Canonical JSON and digest. Complete compact UTF-8 envelope: `block39-fixtures/signals.lattice_polyphase_coordinates.canonical.json`. The pretty fixture has the same payload. Payload SHA-256: `9243e01023a8aeaeed53a18111765666a8665f371376cca2fe1c037e7493cfcc`. Both forms reconstruct the represented type; the finite fixture re-encodes identically.

Migration. No older-version migration ships for this tag. Current version 1 is the supported frozen form; a future version is rejected.

Source binding. Encoder: `src/komorebi_math/refinement/portable.py`, lines 113-117. Decoder: `src/komorebi_math/refinement/portable.py`, lines 36-50. Represented class: `src/komorebi_math/signals/polyphase.py`, lines 293-377. Full source and fragment hashes are in `Block39_Portable_Schema_Records.json`.

G.7.51 `signals.polyphase_coordinates`

Record G-C51. Codec version 1. Represented type: `komorebi_math.signals.polyphase.PolyphaseCoordinates`.

Channels are tuples of ndarrays, with integer order and axis and a finite original_shape. The record preserves finite channel values and array dtypes. It does not promise preservation of memory views, contiguity, original strides or a CUDA allocation.

Complete decoded encoder data. The fixture forms below are examples, not a closed type specification for arbitrary inputs. Each row is a required data key.

Field	Form in the finite fixture
`axis`	`int`
`channels`	`tuple[ndarray; length=3]`
`order`	`int`
`original_shape`	`tuple[int; length=2]`

Field	Form in the finite fixture

Read policy. Extra decoded data fields are rejected by the decoder. Exact field coverage does not imply strict numeric or boolean type checking. Missing fields were individually rejected in the version-bound tests. Extra outer tagged-wrapper fields and future versions were also rejected.

Canonical JSON and digest. Complete compact UTF-8 envelope: `block39-fixtures/signals.polyphase_coordinates.canonical.json`. The pretty fixture has the same payload. Payload SHA-256: `dd353f482582ebce5c370de8bebae72a97f3eab5f9060262bb3c138d35b3c37b`. Both forms reconstruct the represented type; the finite fixture re-encodes identically.

Migration. No older-version migration ships for this tag. Current version 1 is the supported frozen form; a future version is rejected.

Source binding. Encoder: `src/komorebi_math/refinement/portable.py`, lines 100-105. Decoder: `src/komorebi_math/refinement/portable.py`, lines 25-33. Represented class: `src/komorebi_math/signals/polyphase.py`, lines 37-129. Full source and fragment hashes are in `Block39_Portable_Schema_Records.json`.

G.8 Packet-jet stream directory schemas

The directory format is separate from `komorebi-math.portable`. It has a `manifest.json`, a `chunks` directory, and canonical chunk filenames such as `chunk-00000000.json`. Incomplete writes use `<destination>.partial`. A completed, verified staging directory is promoted to the requested destination. A crash can leave uncommitted files; resume verifies the committed prefix before continuing. This is not a database transaction or a guarantee against arbitrary concurrent filesystem modification.

The manifest schema is `komorebi-math.packet-jet-stream`, version 1. The chunk schema is `komorebi-math.packet-jet-stream.chunk`, version 1. Both envelopes have **four** keys: `schema`, `schema_version`, `payload_sha256`, `payload`. Unlike the single-document portable envelope, they do not have a producer field at the envelope level. The manifest payload carries `producer_version`.

Manifest payload field matrix.

Field	Meaning
`state`	`in_progress` or `complete`.
`producer_version`	Recorded package producer.
`metadata`	Address, dimensions, provider, exactness, point, tolerance, dense count, capture mode and source digest.
`metadata_sha256`	Canonical digest of the metadata object.
`chunk_size`	Positive configured chunk size. Its meaning differs between direct and snapshot capture.
`evaluations_completed`	Covered dense-basis progress.
`coefficients_completed`	Number of stored finite coefficient records.
`maximum_omitted_basis_norm`	Nonnegative bound retained from capture.
`chunks`	Ordered index records with contiguous basis and coefficient ranges.
`capture_certificate`	Null during an incomplete capture; nested portable certificate when complete.
`stream_sha256`	Content digest binding the declared stream composition.

Metadata field matrix.

Field	Meaning
`address`	Nested portable PacketJetAddress.
`source_dimension` / `target_dimension`	Positive finite dimensions.

Field	Meaning
`provider_name`	Descriptive provider label. This differs from the v2 single-snapshot codec key `provider`.
`exactness`	Exactness label retained from the capture.
`point`	Portable tuple whose length matches the source dimension.
`tolerance`	Finite nonnegative capture tolerance.
`dense_coefficient_count`	Source dimension raised to address order.
`capture_mode`	`direct` or `snapshot`.
`source_digest`	Source identity digest used for resume matching.

Each chunk index record has exactly `index`, `filename`, `basis_start`, `basis_stop`, `coefficient_start`, `coefficient_stop`, `payload_sha256`, `byte_count`. The filename must be precisely the zero-padded spelling for that index. Noncanonical filenames, including traversal paths, are rejected. Basis and coefficient intervals are nondecreasing and must form contiguous committed progress. Digests must have the expected lowercase hexadecimal form.

A chunk payload has exactly `index`, `basis_start`, `basis_stop`, `coefficient_start`, `coefficient_stop`, `coefficients`. Each coefficient has exactly `indices`, `value`: indices are a JSON array and the value is a portable-encoded tuple. Decoding checks order, source-index range and target-vector length. Chunk envelopes and the manifest's chunk records are cross-checked, not interpreted as unrelated JSON files.

Progress semantics. Direct capture counts dense-basis evaluations, even where a resulting coefficient is omitted as zero. Its chunks cover basis ranges. Snapshot capture writes already selected coefficients; incomplete snapshot chunks do not yet claim dense-basis completion, and its final chunk closes that coverage. A complete manifest requires the full dense count and a capture certificate. An in-progress manifest cannot claim a final certificate.

`open_packet_jet_snapshot_stream` rejects incomplete input by default. `allow_incomplete=True` makes that state explicit. A partial range evaluation remains partial even if its local integrity check succeeds. Selection and full snapshot materialization can impose a maximum coefficient count. They are not equivalent to limiting an arbitrary input file's size before parsing it.

Executed finite example. The stream replay uses a two-dimensional quadratic jet with one-dimensional output. It pauses and resumes both capture modes, verifies chunks, loads a full snapshot, and compares streamed evaluation with the live jet. Splitting the stored coefficient range yields two explicitly partial evaluations whose sum matches the full result. The replay covers ten named checks. Corruption, missing chunks, noncanonical filenames, changed resume metadata, missing required fields and exhausted materialization bounds are separate rejection tests.

The source dimension raised to derivative order is still the dense-basis count. Chunking bounds the working set for normal generated chunks but does not remove that combinatorial cost or establish a hostile-input resource boundary. All stream observations here are finite CPU tests, not CUDA or native Windows qualification.

G.9 Reproducible recipes and reference files

From a working copy of the delivered root, install the pinned wheel into a separate environment with its required dependencies. The examples below use the active interpreter; they do not download dependencies and do not silently select the source checkout.

```
python tools/block39/inventory.py
python tools/block39/stream_lab.py --output scratch-stream-replay
python -I -m pytest tools/block39 -p no:cacheprovider
python tools/block39/generate_manual.py
python tools/block39/build_pdfs.py
```

The inventory generator refuses a source-checkout import and compares every package-source/data file with the installed package. Use a new stream output directory for each active replay; existing evidence is not silently overwritten. Test, PDF and manifest receipts are bound to file hashes. Regenerating a document or running a new test command in a copy does not update an already sealed delivery's validation claim.

A minimal safe round-trip for trusted locally created finite data is:

```
from fractions import Fraction
from komorebi_math.export import save_portable, load_portable

value = {"ratio": Fraction(2, 7), "prefix": (1, 1, 2, 3)}
save_portable(value, "example.portable.json")
restored = load_portable("example.portable.json")
assert restored == value
```

An existing destination is rejected unless overwrite is explicitly allowed. This recipe checks value preservation, not a mathematical assertion embedded in the data. For migrations, retain the original file, decode through the registered chain, and write the current result to a new destination. A migration cannot be authenticated by the same unkeyed digest it can replace.

The reference ledger includes all 51 canonical envelope fixtures and their 51 pretty equivalents, 26 primitive/container/array fixtures, and one legacy snapshot-v1 fixture. Complete stream fixtures and paused manifests live with the replay evidence. JSON files are data, not executable proof certificates. Source locators and hashes provide reproducible traceability to this exact release.

G.10 Corrections, known limits and closure

The inherited package document `docs/PERSISTENCE_SCHEMAS.md` is narrower and older than this appendix. Its migration example uses the unsupported keyword `migrate`; the actual API is `migrator`. Its abbreviated envelope omits the mandatory `producer_version`. Its enumerated mathematical codec list covers only the early core codecs, not all 51 frozen entries. This appendix explicitly supersedes those points while retaining the unchanged R2 package document as historical release content.

Several input-validation weaknesses were reproduced and documented rather than silently changed: producer metadata is unchecked; outer schema-version equality accepts a boolean; integral floats can pass codec-version checks; integer conversion can truncate fraction data and array dimensions; certificate flags use truthiness; twelve codecs ignore extra data fields; and not every ndarray dtype round-trips. The generic loader has no universal byte/allocation cap. These are remaining hardening opportunities, not successful security qualifications.

Do not treat a passing schema-reference test for a permissive behavior as an endorsement of that behavior. The expected-outcome tests make the limitation visible so a later implementation repair cannot silently change the documented contract. A stricter future release should define its compatibility policy, update the relevant fields/versions where appropriate, and rerun malformed-input and legacy migration tests.

Executed checks for this edition. The unchanged package source suite passed 1,203 tests. The installed-wheel schema-reference suite passed 663 tests, including all codec routes, required-field removal cases, future-version rejections, scalar/container fixtures, migration cases and stream boundaries. The ten stream replay checks repeat part of this behavioral area and are not ten additional mathematical theorems. All 366 installed package files match the stored source. The reference does not turn serialized claims into Lean proofs.

Reference boundaries. This source-bound portable-schema and migration reference does not certify every dtype, custom codec, input size, malicious payload or platform. The recorded audit did not qualify native Windows/PowerShell, other Python versions, CUDA, strict MkDocs, fully dependency-isolated installation or final publication. Appendix O distinguishes the final source's retained execution evidence from this predecessor audit.

Appendix H. Certificate and theorem-ID index

Start with the result you have. If a computation returned a certificate, read H.2 and the matching type in H.4 before looking up its theorem label. If you already have a label and need its producer, use H.7, then follow the symbol into H.8. H.9 helps when the certificate is nested inside a larger result object.

A practical reading order. Identify the result type; check whether its reported validity meets your task; read its exactness class and unchecked assumptions; then inspect the residuals, tolerance and finite range used by the producer. Preserve those details when recording the answer. A useful report says what was computed, on which inputs and domain, and under which assumptions, rather than quoting only a Boolean.

For example, the finite-prefix certificates described below carry an audit degree or a last checked index. Include that boundary in your description of a successful result. If the certificate is conditional, the unchecked assumptions remain part of the result even when its validity check succeeds. H.3 explains how those assumptions propagate through a composed computation.

H.1 Purpose and scope

This appendix connects a recorded theorem label to its implementation symbol, declared hypotheses, certificate fields and example evidence. It covers the pinned `1.0.0b4-block36-r2` source, not all possible downstream extensions. Its source vocabulary is finite; the runtime label namespace is not. A caller can supply a new string, compose existing strings, or load one from a portable file.

The word *certificate* is not a uniform proof protocol in this release. The base `Certificate` is a stored validation record. Some specialized records compute a Boolean from stored sides or a failure list. Others wrap a base record, and `Basis Certificate` supplies diagnostics without `valid` or `require`. The type inventory below preserves those differences.

Three axes must be read independently:

Axis	Question answered	What it does not establish
Runtime validity	Did the producer or stored field report success?	The validity flag is not independently recomputed by the base `require`.
`Exactness`	How did the producer classify its calculation or assumptions?	An exact finite calculation does not establish an unbounded theorem.
`ClaimStatus` provenance	How is the attached source claim classified by its author?	A `PROVED` record is metadata, not evidence of a Lean check in this reference.

API stability from Appendices A-C is a fourth, separate axis. A stable callable can produce a numerical certificate. A research callable can perform exact rational arithmetic. A label containing `T` is not a theorem-proof count, and the occurrence of a source-paper name is not a checked cross-reference to the current separate PRSC paper series.

All source locators are relative to the delivered root. Constructor references `H-C...` address the construction-site matrix and JSON ledger. Label references `H-T...` are local index keys, not new theorem names. Their numbering belongs to this edition. Use the literal theorem label together with the qualified symbol and pinned source digest for durable identification. `EX-E-...` identifies the unchanged example record in Appendix E.

The JSON index contains the complete expressions, source fragments, hashes, constructor fields, specialized types, typed carriers, provenance records and literal references. Printed cells abbreviate long code expressions for legibility. Ellipses in a printed expression never replace the complete machine-readable record. The installed-wheel crosswalk is a separate observation ledger so a static reference cannot accidentally become an execution claim.

H.2 The base record and result wrappers

`Certificate` has nine fields: `valid`, `exactness`, `residuals`, `tolerance`, `assumptions_checked`, `assumptions_unchecked`, `diagnostics`, `theorem_id` and `message`. Only the first two are required by its generated initializer. Type annotations describe the intended interface; the base dataclass has no post-initialization validator enforcing every field's type or mathematical consistency.

The implementation of `Certificate.require()` is a truthiness check. It raises `ValidationError` when `valid` is false and otherwise returns the same record. It does not compare residuals against tolerance, recompute a witness, reject unchecked assumptions, require exact arithmetic, look up the theorem label, or invoke a proof kernel. A manually supplied truthy string such as `"false"` can therefore pass this method. These are reproduced interface limits, not a recommendation to construct such records.

The dataclass is frozen only at the outer field-assignment level. Its residual and diagnostic dictionaries remain mutable. `to_dict()` constructs a dataclass-derived representation and emits the enum value. `to_json()` uses a presentation serialization with a string fallback. That presentation is not the portable persistence envelope from Appendix G and is not a signed proof artifact.

`ComputationResult.require()` delegates to its certificate and returns the *value*, not the record. Its `valid` property mirrors the validity flag. Its `exact` property tests whether the status is `Exactness.EXACT`; it can therefore be true for an invalid record. Always inspect validity separately. A successful conditional result can also pass `require()` while retaining unchecked assumptions.

`ConditionalResult` enforces that its certificate's exactness is the conditional enum member. It does not, by that check alone, require validity, prove assumptions or enforce a nonempty unchecked list. Treat the wrapper as a representation constraint rather than an admission gate.

A normal local inspection pattern is:

```
certificate = result.certificate
certificate.require()
print(certificate.exactness.value)
print(certificate.assumptions_unchecked)
print(certificate.residuals, certificate.tolerance)
print(certificate.theorem_id)
```

The producer's input validation, arithmetic backend, finite range and witness computation still determine the conclusion. Reading these fields is not a substitute for those checks.

H.3 Composition, assumptions and numerical evidence

The expression subsystem supplies a stricter *composition* operation, not a general proof checker. Its declared weakening order is exact, numerical, conditional, heuristic, then planned. `weakest_exactness` requires enum instances, selects the weakest supplied status, and returns exact for an empty input by convention. This is an implementation ordering for evidence aggregation, not a claim that all notions of evidence form a universal mathematical hierarchy.

`compose_certificates` requires base certificate instances. It preserves child residuals under `child0:`, `child1:` and subsequent prefixes, separates child/local diagnostics, retains child theorem IDs in the diagnostic record, and uses the maximum supplied finite nonnegative tolerance. It checks residuals for finite real values. It does not independently retest each residual against a tolerance or establish that the arithmetic behind a child was correct.

Unchecked assumptions dominate checked assumptions with the same text. A nonempty unchecked set downgrades otherwise exact or numerical evidence to conditional. A false child flag makes the aggregate invalid. The empty composition is a valid exact record with zero children by convention; it is not a new proof of any caller-chosen theorem label. Labels passed to this composition helper must be nonempty strings when supplied, unlike the unconstrained base initializer.

`exact_internal_certificate` records an internal operation declared exact by construction. It validates the description, label and diagnostic form. It does not execute or verify an arbitrary mathematical proposition supplied as text. The operation that calls it remains responsible for its finite construction.

For numerical producers, inspect the actual residual norm, scaling factor, rank threshold and input domain. Two producers can share a theorem label yet check different residuals. Backend-dependent producers can emit exact or numerical status through the same construction site. A zero floating residual is not automatically an exact symbolic identity, and a tolerance is not automatically a theorem-level error bound.

H.4 Specialized certificate records

The following cards give the full declared field lists. Method descriptions distinguish stored flags, computed comparisons and nested base records. Their source fragments, including the exact `valid`, `require` and `certificate` methods where present, are retained in the JSON index.

BlockJacobiCertificate

`komorebi_math.algorithms.krylov.BlockJacobiCertificate`. Source: `algorithms/krylov.py:396`.

A result carrier for the packet diagonal, normalized coupling, eta and an optional condition bound. Inspect its nested `certificate`. It does not expose the base require protocol directly. The producer checks positive definiteness and coupling conditions; the record itself is not a new proof of those inputs.

Field	Annotation and default
`diagonal`	`BlockDiagonalOperator; [required]`
`normalized_coupling`	`Array; [required]`
`eta`	`float; [required]`
`condition_bound`	`float | None; [required]`
`certificate`	`Certificate; [required]`

Certificate

`komorebi_math.certificates.Certificate`. Source: `certificates.py:28`.

The base record described in H.2. Its require method returns itself, and its validity field is stored rather than recomputed.

Field	Annotation and default
`valid`	`bool; [required]`
`exactness`	`Exactness; [required]`
`residuals`	`dict[str, float]; field(default_factory=dict)`
`tolerance`	`float | None; None`
`assumptions_checked`	`tuple[str, ...]; ()`
`assumptions_unchecked`	`tuple[str, ...]; ()`
`diagnostics`	`dict[str, Any]; field(default_factory=dict)`
`theorem_id`	`str | None; None`
`message`	`str; ''`

StateSpaceCoordinateCertificate

`komorebi_math.coordinates.state_space.StateSpaceCoordinateCertificate`. Source: `coordinates/state_space.py:259`.

Stores a validity flag along with systems, a state transform and a finite Markov-parameter count. Its certificate method wraps that flag with label `CC.R3`. Calling the wrapper does not rerun the original transfer-invariance computation.

Field	Annotation and default
`source`	`StateSpaceModel; [required]`

Field	Annotation and default
`target`	`StateSpaceModel; [required]`
`transform`	`CoordinateTransform; [required]`
`markov_count`	`int; [required]`
`valid`	`bool; [required]`

BasisCertificate

`komorebi_math.generating.basis.BasisCertificate`. Source: `generating/basis.py:66`.

A finite basis diagnostic with two separate Boolean diagnostics, denominator and bit-size observations. There is no generic valid property and no require method. Do not coerce all fields into an invented all-purpose pass/fail result.

Field	Annotation and default
`name`	`str; [required]`
`degree`	`int; [required]`
`nonzero_normalization`	`bool; [required]`
`integral_structure_constants`	`bool; [required]`
`maximum_denominator`	`int; [required]`
`maximum_numerator_bits`	`int; [required]`

PointedSubindexCertificate

`komorebi_math.generating.bell.PointedSubindexCertificate`. Source: `generating/bell.py:115`.

Computes valid by comparing the stored left and right sides. Require returns None on success and raises on a mismatch. The certificate method emits a base record with `NGF.POINTED_SUBINDEX`. Direct construction with equal sides does not independently verify primality or regenerate the combinatorial sides.

Field	Annotation and default
`index`	`int; [required]`
`prime`	`int; [required]`
`left`	`tuple[int, ...]; [required]`
`right`	`tuple[int, ...]; [required]`

AutomatonCertificate

`komorebi_math.generating.cartier.AutomatonCertificate`. Source: `generating/cartier.py:136`.

Computes valid from an empty finite discrepancy list. Require returns None or raises on the first stored discrepancy. The compared_terms field limits the recorded audit; it is not an all-indices automaton proof.

Field	Annotation and default
`compared_terms`	`int; [required]`
`failures`	`tuple[tuple[int, int, int], ...]; [required]`

SeriesTransferCertificate

`komorebi_math.generating.finite_series.SeriesTransferCertificate`. Source: `generating/finite_series.py:62`.

Uses its stored valid flag. Require returns None or raises, and certificate emits `NGF.UNIVERSAL_PRODUCT_AND_TRANSFER`. A manually constructed inconsistent flag/failures pair is not reconciled by these wrappers.

Field	Annotation and default
`valid`	`bool; [required]`
`degree`	`int; [required]`
`source_basis`	`str; [required]`
`target_basis`	`str; [required]`
`domain`	`str; [required]`
`failures`	`tuple[int, ...]; ()`

RationalMaskAlgebraicCertificate

`komorebi_math.generating.masks.RationalMaskAlgebraicCertificate`. Source: `generating/masks.py:754`.

Computes valid from the finite failure list, with prime and degree retained. Require returns None or raises on the first failure. Its base certificate label is `GF-RATIONAL-MASK-ALGEBRAIC`. The finite degree remains part of the conclusion.

Field	Annotation and default
`prime`	`int; [required]`
`degree`	`int; [required]`
`failures`	`tuple[tuple[int, int], ...]; [required]`

CarryLinearCertificate

`komorebi_math.generating.normalization.CarryLinearCertificate`. Source: `generating/normalization.py:56`.

Computes valid from a finite failure list for a supplied valuation callback. Require returns None or raises. The producer recovers slope/carry parameters and audits through checked_through; the unbounded classification is not proved by that finite prefix.

Field	Annotation and default
`prime`	`int; [required]`
`linear_slope`	`int; [required]`
`carry_weight`	`int; [required]`
`checked_through`	`int; [required]`
`failures`	`tuple[tuple[int, int, int, int], ...]; [required]`

GradingCertificate

`komorebi_math.nonlinear.graded.GradingCertificate`. Source: `nonlinear/graded.py:193`.

A carrier for arity, tested labels, a forbidden residual and a nested base certificate. Use the nested record. The finite tests and any assumptions on the supplied multilinear operation must be kept visible.

Field	Annotation and default
`arity`	`int; [required]`
`tested_input_labels`	`int; [required]`
`forbidden_residual`	`float; [required]`

Field	Annotation and default
`certificate`	`Certificate; [required]`

H.5 Reproduced boundaries and safe interpretation

Disabled construction validation. A packet system created from `2 * I` with `validate=False` fails its packet-axiom certificate but passes the separate self-adjointness check in `hilbert_certificate()`. The latter's checked-assumption text includes packet axioms without recomputing them in that method. The example deliberately demonstrates the caller's ability to bypass a prerequisite, not a valid packet system. The reference records that behavior without altering the frozen package bytes.

For that interface, explicitly check the packet axioms before the Hilbert refinement:

```
import numpy as np
from komorebi_math import PacketSystem

system = PacketSystem([np.diag([1, 0]), np.diag([0, 1])])
system.certificate().require()
system.hilbert_certificate().require()
```

The second call refines an already validated system. It must not be read as a replacement for the first on an unchecked object. This is one examined producer boundary, not a claim that every constructor-bypass path in the library was audited.

Stored claims and portable data. Appendix G's decoder can preserve a supplied label and use truthiness for a supplied validity flag. Neither a correct payload digest nor successful decoding authenticates the writer or re-proves the mathematical result. Rebuild a result from its mathematical inputs with the relevant producer when fresh verification is required.

Finite witnesses. Automaton, carry-linear, grading and finite-series certificates retain finite audit parameters. A finite check may validate exactly the stated finite instance while leaving a global statement unresolved. Do not remove its degree, tested range, arithmetic domain, backend or unchecked assumptions when quoting a conclusion.

Expected negative conclusions. A test can pass by correctly observing an invalid certificate. The replay includes nontrivial holonomy, hidden-feedback memory and unmet automorphic completion gates. Those outcomes remain negative conclusions for their stated checks, not successful theorem qualifications.

H.6 Provenance registry and non-unique labels

The provenance registry is keyed by qualified Python names, not by theorem label. `documented` attaches a record to an object and registers its qualified name. `register_record` replaces an existing entry under the same qualified-name key; it does not enforce globally unique theorem IDs. `records()` returns a dictionary snapshot. `describe()` retrieves attached or registered metadata, not a checked proof object.

The seven repeated labels below are retained as distinct symbol records. Reuse does not by itself establish that either source claim is false, but it makes an ID-only citation ambiguous. The canonical citation tuple for this index is `(theorem label, qualified symbol, pinned source hash)`. Metadata statuses are reproduced exactly rather than promoted or demoted by this reference.

Reused label	Separate declared titles
`KM-DSP-D05`	Finite FIR frequency response / Finite modified periodogram
`KM-DSP-D06`	Finite Welch power spectral estimate / Windowed-sinc finite FIR design
`KM-DSP-T06`	Finite centered polyphase rational-resampling identity / Finite multilevel Haar reconstruction

Reused label	Separate declared titles
PSC-II-T13	Exact separation weight from the lattice action / Gluing module and exact separation weight
PSC-III-D13	Finite ordered rooted tree / Linear correction datum and cokernel obstruction
PSC-III-T14	Cokernel obstruction criterion / Polynomial rooted-tree convergence certificate
PSC-III-T41	Finite nonlinear Schur elimination / Fundamental holonomy intertwining criterion

The loaded registry contains 198 qualified-name records and 191 distinct registered labels. Declared status totals are 28 CLASSICAL, 1 CONDITIONAL, 40 DEFINITION, 11 FINITE_CERTIFICATE, 3 HEURISTIC, 115 PROVED. These totals reproduce metadata only. They are not a count of Lean theorems or newly verified mathematical proofs.

The registry uses 99 distinct verification-suite strings. These strings are not literal CLI choices from Appendix F. They are provenance navigation labels, not automatically executable `komorebi verify` arguments. The index prints them as declared; consult the CLI matrix for actual commands.

H.7 Theorem-label and symbol index

The finite source vocabulary has **488 labels**, including definitions, diagnostic labels, verification fixtures and engineering gates. It includes explicit theorem arguments/defaults, provenance identifiers, and additional source literals in the project ID families, including table-selected and rewrite-return labels. A label mentioned only by a verification fixture remains a reference label, not an invented production theorem.

Each card distinguishes direct constructor sites from other declarations. The checked and unchecked expressions shown for direct sites are source expressions, not proof that all branches execute those checks. Full expressions are in the `constructor_sites` ledger. The absence of a runtime example observation is explicitly recorded. Literal mentions in test files are navigation, not theorem coverage.

H-T001 / CC.A0

Source symbol(s): `komorebi_math.coordinates.basis.CoordinateAtlas.resolve`. No attached provenance record under this label.

Locations: `coordinates/basis.py:348`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T002 / CC.A1

Source symbol(s): `komorebi_math.coordinates.basis.CoordinateAtlas.resolve`, `komorebi_math.coordinates.basis.CoordinateAtlas.compose_path`. No attached provenance record under this label.

Locations: `coordinates/basis.py:376`, `coordinates/basis.py:395`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** `test_coordinates_general_persistence.py`.

H-T003 / CC.A2

Source symbol(s): `komorebi_math.coordinates.basis.CommutingDiagramCheck.certificate`. No attached provenance record under this label.

Locations: `coordinates/basis.py:246`. **Direct base sites:** `H-C043`.

`H-C043`: status `Exactness.EXACT`; checked `('common source', 'common target', 'exact matrix products')`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: `EX-E-024`. **Literal test mentions:** none recorded.

H-T004 / CC.B0

Source symbol(s): `komorebi_math.coordinates.basis.FiniteBasis.certificate`. No attached provenance record under this label.

Locations: `coordinates/basis.py:205`. **Direct base sites:** `H-C042`.

`H-C042`: status `Exactness.EXACT`; checked `('finite rational vectors', 'exact row reduction')`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T005 / CC.B1

Source symbol(s): `komorebi_math.coordinates.basis.FiniteBasis`. No attached provenance record under this label.

Locations: `coordinates/basis.py:146`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T006 / CC.B2

Source symbol(s): `komorebi_math.coordinates.basis.permutation_transform`. No attached provenance record under this label.

Locations: `coordinates/basis.py:500`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T007 / CC.B3

Source symbol(s): `komorebi_math.coordinates.basis.scaling_transform`. No attached provenance record under this label.

Locations: `coordinates/basis.py:520`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T008 / CC.C1

Source symbol(s): `komorebi_math.coordinates.combinatorial.monomial_to_falling_factorial_transform`. No attached provenance record under this label.

Locations: `coordinates/combinatorial.py:101`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T009 / CC.C10

Source symbol(s): `komorebi_math.coordinates.combinatorial.divisor_zeta_transform`. No attached provenance record under this label.

Locations: `coordinates/combinatorial.py:306`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T010 / CC.C11

Source symbol(s): `komorebi_math.coordinates.combinatorial.divisor_zeta_transform`. No attached provenance record under this label.

Locations: `coordinates/combinatorial.py:299`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T011 / CC.C12

Source symbol(s): `komorebi_math.coordinates.combinatorial.subset_zeta_transform`. No attached provenance record under this label.

Locations: `coordinates/combinatorial.py:344`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T012 / CC.C13

Source symbol(s): `komorebi_math.coordinates.combinatorial.subset_zeta_transform`. No attached provenance record under this label.

Locations: `coordinates/combinatorial.py:337`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T013 / CC.C2

Source symbol(s): `komorebi_math.coordinates.combinatorial.falling_factorial_to_monomial_transform`. No attached provenance record under this label.

Locations: `coordinates/combinatorial.py:119`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T014 / CC.C3

Source symbol(s): `komorebi_math.coordinates.combinatorial.monomial_to_rising_factorial_transform`. No attached provenance record under this label.

Locations: `coordinates/combinatorial.py:141`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T015 / CC.C4

Source symbol(s): `komorebi_math.coordinates.combinatorial.rising_factorial_to_monomial_transform`. No attached provenance record under this label.

Locations: `coordinates/combinatorial.py:159`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T016 / CC.C5

Source symbol(s): `komorebi_math.coordinates.combinatorial.rising_to_falling_factorial_transform`. No attached provenance record under this label.

Locations: `coordinates/combinatorial.py:177`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T017 / CC.C6

Source symbol(s): `komorebi_math.coordinates.combinatorial.binomial_transform`. No attached provenance record under this label.

Locations: `coordinates/combinatorial.py:212`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T018 / CC.C7

Source symbol(s): `komorebi_math.coordinates.combinatorial.binomial_transform`. No attached provenance record under this label.

Locations: `coordinates/combinatorial.py:211`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T019 / CC.C8

Source symbol(s): `komorebi_math.coordinates.combinatorial.forward_difference_transform`. No attached provenance record under this label.

Locations: `coordinates/combinatorial.py:233`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T020 / CC.C9

Source symbol(s): `komorebi_math.coordinates.combinatorial.newton_evaluation_transform`. No attached provenance record under this label.

Locations: `coordinates/combinatorial.py:251`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T021 / CC.F1

Source symbol(s): `komorebi_math.coordinates.affine.AffineCoordinateTransform`. No attached provenance record under this label.

Locations: `coordinates/affine.py:35`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T022 / CC.F2

Source symbol(s): `komorebi_math.coordinates.affine.affine_basis_transform`. No attached provenance record under this label.

Locations: `coordinates/affine.py:249`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T023 / CC.F3

Source symbol(s): `komorebi_math.coordinates.affine.BarycentricCoordinateSystem.certificate`. No attached provenance record under this label.

Locations: `coordinates/affine.py:224`. **Direct base sites:** `H-C038`.

`H-C038`: status `Exactness.EXACT`; checked `('affine independence', 'exact augmented solve')`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T024 / CC.F4

Source symbol(s): `komorebi_math.coordinates.affine.translation_transform`. No attached provenance record under this label.

Locations: `coordinates/affine.py:270`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T025 / CC.J1

Source symbol(s): `komorebi_math.coordinates.formal.FormalCoordinateChange.certificate`. No attached provenance record under this label.

Locations: `coordinates/formal.py:229`. **Direct base sites:** `H-C045`.

`H-C045`: status `Exactness.EXACT`; checked `('origin fixed', 'nonzero linear coefficient', 'finite jet truncation')`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: `EX-E-002`, `EX-E-021`. **Literal test mentions:** none recorded.

H-T026 / CC.J2

Source symbol(s): `komorebi_math.coordinates.formal.FormalCoordinateChange.substitution_transform`. No attached provenance record under this label.

Locations: `coordinates/formal.py:213`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T027 / CC.M1

Source symbol(s): `komorebi_math.coordinates.modular.FiniteFieldCoordinateTransform`. No attached provenance record under this label.

Locations: `coordinates/modular.py:93`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** `test_coordinates_general_persistence.py`.

H-T028 / CC.M2

Source symbol(s): `komorebi_math.coordinates.modular.reduce_exact_transform_mod_prime`. No attached provenance record under this label.

Locations: `coordinates/modular.py:234`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T029 / CC.N1

Source symbol(s): `komorebi_math.coordinates.normalized.normalization_change_transform`. No attached provenance record under this label.

Locations: `coordinates/normalized.py:54`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T030 / CC.NM1

Source symbol(s): `komorebi_math.coordinates.numeric.NumericCoordinateTransform`. No attached provenance record under this label.

Locations: `coordinates/numeric.py:33`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T031 / CC.O1

Source symbol(s): `komorebi_math.coordinates.calculus.PolynomialCoordinateOperator`, `komorebi_math.coordinates.calculus`. No attached provenance record under this label.

Locations: `coordinates/calculus.py:42`, `coordinates/calculus.py:105`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T032 / CC.O2

Source symbol(s): `komorebi_math.coordinates.calculus.derivative_coordinate_operator`, `komorebi_math.coordinates.operators.BezoutDefectIdentity.certificate`. No attached provenance record under this label.

Locations: `coordinates/calculus.py:146`, `coordinates/operators.py:90`. **Direct base sites:** `H-C058`.

`H-C058`: status `Exactness.EXACT`; checked `('coprime shift polynomials', 'finite sequence prefix')`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T033 / CC.O3

Source symbol(s): `komorebi_math.coordinates.calculus.integral_coordinate_operator`. No attached provenance record under this label.

Locations: `coordinates/calculus.py:166`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T034 / CC.O4

Source symbol(s): `komorebi_math.coordinates.calculus.multiplication_coordinate_operator`. No attached provenance record under this label.

Locations: `coordinates/calculus.py:191`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T035 / CC.O5

Source symbol(s): `komorebi_math.coordinates.calculus.affine_substitution_coordinate_operator`. No attached provenance record under this label.

Locations: `coordinates/calculus.py:213`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T036 / CC.O6

Source symbol(s): `komorebi_math.coordinates.calculus.finite_difference_coordinate_operator`. No attached provenance record under this label.

Locations: `coordinates/calculus.py:237`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T037 / CC.O7

Source symbol(s): `komorebi_math.coordinates.calculus.transport_polynomial_operator`. No attached provenance record under this label.

Locations: `coordinates/calculus.py:276`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T038 / CC.P1

Source symbol(s): `komorebi_math.coordinates.polynomials.connection_transform`. No attached provenance record under this label.

Locations: `coordinates/polynomials.py:210`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: `EX-E-008`. **Literal test mentions:** `test_coordinates_general_persistence.py`.

H-T039 / CC.P3

Source symbol(s): `komorebi_math.coordinates.polynomials.euler_to_fibonacci_transform`. No attached provenance record under this label.

Locations: `coordinates/polynomials.py:276`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T040 / CC.P4

Source symbol(s): `komorebi_math.coordinates.classical.evaluation_transform`. No attached provenance record under this label.

Locations: `coordinates/classical.py:460`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T041 / CC.P5

Source symbol(s): `komorebi_math.coordinates.classical.interpolation_transform`. No attached provenance record under this label.

Locations: `coordinates/classical.py:480`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T042 / CC.P6

Source symbol(s): `komorebi_math.coordinates.classical.affine_substitution_transform`. No attached provenance record under this label.

Locations: `coordinates/classical.py:532`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T043 / CC.Q1

Source symbol(s): `komorebi_math.coordinates.families.RecurrenceModel.coordinate_transform_to`. No attached provenance record under this label.

Locations: `coordinates/families.py:114`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T044 / CC.R1

Source symbol(s): `komorebi_math.coordinates.recurrence.recurrence_analysis_transform`, `komorebi_math.coordinates.recurrence.recurrence_synthesis_transform`, `komorebi_math.coordinates.recurrence.RecurrenceCoordinateRecord.certificate`, `komorebi_math.coordinates.state_space.controllability_coordinate_transform`. No attached provenance record under this label.

Locations: `coordinates/recurrence.py:101`, `coordinates/recurrence.py:119`, `coordinates/recurrence.py:189`, `coordinates/state_space.py:217`, `coordinates/state_space.py:224`. **Direct base sites:** `H-C061`.

`H-C061`: status `Exactness.EXACT`; checked `('finite sequence prefix', 'constant coefficient recurrence')`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: `EX-E-007`, `EX-E-014`. **Literal test mentions:** none recorded.

H-T045 / CC.R2

Source symbol(s): `komorebi_math.coordinates.recurrence.coordinate_change_between_recurrences`, `komorebi_math.coordinates.state_space.observability_coordinate_transform`. No attached provenance record under this label.

Locations: `coordinates/recurrence.py:275`, `coordinates/state_space.py:249`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T046 / CC.R3

Source symbol(s): `komorebi_math.coordinates.state_space.StateSpaceCoordinateCertificate.certificate`. No attached provenance record under this label.

Locations: `coordinates/state_space.py:271`. **Direct base sites:** `H-C062`.

`H-C062`: status `Exactness.EXACT`; checked `('invertible state transform', 'exact Markov parameters')`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: `EX-E-046`. **Literal test mentions:** none recorded.

H-T047 / CC.S1

Source symbol(s): `komorebi_math.coordinates.numeric.discrete_fourier_transform`. No attached provenance record under this label.

Locations: `coordinates/numeric.py:188`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: `EX-E-053`. **Literal test mentions:** none recorded.

H-T048 / CC.S2

Source symbol(s): `komorebi_math.coordinates.numeric.discrete_cosine_transform`, `komorebi_math.coordinates.realization.StateRecovery.certificate`. No attached provenance record under this label.

Locations: `coordinates/numeric.py:215`, `coordinates/realization.py:105`. **Direct base sites:** `H-C059`.

`H-C059`: status `Exactness.EXACT`; checked `('finite rational realization', 'declared output horizon')`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T049 / CC.S3

Source symbol(s): `komorebi_math.coordinates.numeric.walsh_hadamard_transform`, `komorebi_math.coordinates.realization.IntertwinerSpace.certificate`. No attached provenance record under this label.

Locations: `coordinates/numeric.py:240`, `coordinates/realization.py:154`. **Direct base sites:** `H-C060`.

`H-C060`: status `Exactness.EXACT`; checked `('finite rational transition matrices',)`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T050 / CC.S4

Source symbol(s): `komorebi_math.coordinates.numeric.haar_wavelet_transform`. No attached provenance record under this label.

Locations: `coordinates/numeric.py:277`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: `EX-E-053`. **Literal test mentions:** none recorded.

H-T051 / CC.S5

Source symbol(s): `komorebi_math.coordinates.numeric.ModalCoordinateSystem.certificate`, `komorebi_math.coordinates.numeric.modal_coordinate_system`. No attached provenance record under this label.

Locations: `coordinates/numeric.py:300`, `coordinates/numeric.py:330`. **Direct base sites:** `H-C057`.

`H-C057`: status `Exactness.NUMERICAL`; checked `('square matrix', 'complete numerical eigenvector basis')`; unchecked `()`. Residual keys: `diagonalization`.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T052 / CC.T

Source symbol(s): `komorebi_math.verify.coordinates.run_coordinates_verification.tensor_and_operator_transport`. No attached provenance record under this label.

Locations: `verify/coordinates.py:322`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T053 / CC.T1

Source symbol(s): `komorebi_math.coordinates.tensor.contragredient_transform`. No attached provenance record under this label.

Locations: `coordinates/tensor.py:34`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T054 / CC.T2

Source symbol(s): `komorebi_math.coordinates.tensor.tensor_coordinate_transform`. No attached provenance record under this label.

Locations: `coordinates/tensor.py:129`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T055 / CC.T3

Source symbol(s): `komorebi_math.coordinates.tensor.tensor_product_transform`. No attached provenance record under this label.

Locations: `coordinates/tensor.py:154`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T056 / CC.T4

Source symbol(s): `komorebi_math.coordinates.tensor.direct_sum_transform`. No attached provenance record under this label.

Locations: `coordinates/tensor.py:181`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T057 / EXPR.BINDING.EXACT

Source symbol(s): `komorebi_math.expression.engine`. No attached provenance record under this label.

Locations: `expression/engine.py:325`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T058 / EXPR.EQUALITY.DECIDABLE

Source symbol(s): `komorebi_math.expression.engine.equivalent`. No attached provenance record under this label.

Locations: `expression/engine.py:821`, `expression/engine.py:830`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T059 / EXPR.EVALUATE.DUAL_PATH

Source symbol(s): `komorebi_math.expression.engine.evaluate_expression`. No attached provenance record under this label.

Locations: `expression/engine.py:1713`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: `EX-E-006`, `EX-E-007`. **Literal test mentions:** none recorded.

H-T060 / EXPR.NORMAL_FORM.APPLY

Source symbol(s): `komorebi_math.expression.engine._normalize_node_uncached`. No attached provenance record under this label.

Locations: `expression/engine.py:742`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: `EX-E-006`, `EX-E-007`. **Literal test mentions:** none recorded.

H-T061 / EXPR.NORMAL_FORM.COMPOSE

Source symbol(s): `komorebi_math.expression.engine._normalize_node_uncached`. No attached provenance record under this label.

Locations: `expression/engine.py:708`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: `EX-E-006`, `EX-E-007`. **Literal test mentions:** none recorded.

H-T062 / EXPR.NORMAL_FORM.EXACT

Source symbol(s): `komorebi_math.expression.engine._literal_certificate`. No attached provenance record under this label.

Locations: `expression/engine.py:525`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: `EX-E-006`, `EX-E-007`. **Literal test mentions:** none recorded.

H-T063 / EXPR.NORMAL_FORM.LET

Source symbol(s): `komorebi_math.expression.engine._normalize_node_uncached`. No attached provenance record under this label.

Locations: `expression/engine.py:688`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T064 / EXPR.NORMAL_FORM.SCALE

Source symbol(s): `komorebi_math.expression.engine._normalize_node_uncached`. No attached provenance record under this label.

Locations: `expression/engine.py:732`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T065 / EXPR.NORMAL_FORM.SUM

Source symbol(s): `komorebi_math.expression.engine._normalize_node_uncached`. No attached provenance record under this label.

Locations: `expression/engine.py:723`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T066 / EXPR.NORMAL_FORM.UNIQUE

Source symbol(s): `komorebi_math.expression.engine.normalize_expression`. No attached provenance record under this label.

Locations: `expression/engine.py:776`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: `EX-E-006`, `EX-E-007`. **Literal test mentions:** none recorded.

H-T067 / EXPR.PLAN.ADD

Source symbol(s): `komorebi_math.expression.engine._PlanBuilder._compile_uncached`. No attached provenance record under this label.

Locations: `expression/engine.py:1487`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T068 / EXPR.PLAN.APPLY

Source symbol(s): `komorebi_math.expression.engine._PlanBuilder._compile_uncached`. No attached provenance record under this label.

Locations: `expression/engine.py:1513`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: `EX-E-006`, `EX-E-007`. **Literal test mentions:** none recorded.

H-T069 / EXPR.PLAN.COMPOSE

Source symbol(s): `komorebi_math.expression.engine._PlanBuilder._compile_uncached`. No attached provenance record under this label.

Locations: `expression/engine.py:1531`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: `EX-E-006`, `EX-E-007`. **Literal test mentions:** none recorded.

H-T070 / EXPR.PLAN.DENOTATION

Source symbol(s): `komorebi_math.expression.engine.compile_expression`. No attached provenance record under this label.

Locations: `expression/engine.py:1649`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: `EX-E-006`, `EX-E-007`. **Literal test mentions:** none recorded.

H-T071 / EXPR.PLAN.EXECUTE

Source symbol(s): `komorebi_math.expression.engine.execute_plan`. No attached provenance record under this label.

Locations: `expression/engine.py:1614`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: `EX-E-006`, `EX-E-007`. **Literal test mentions:** none recorded.

H-T072 / EXPR.PLAN.MATRIX_CHAIN_DP

Source symbol(s): `komorebi_math.expression.planning`, `komorebi_math.expression.planning.matrix_chain_plan`. No attached provenance record under this label.

Locations: `expression/planning.py:22`, `expression/planning.py:212`. **Direct base sites:** `H-C114`.

`H-C114`: status `Exactness.EXACT`; checked `('finite order-preserving matrix chain', 'positive integer matrix dimensions', 'all binary parenthesizations compared by dynamic programming', 'classica...`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: `EX-E-006`, `EX-E-007`. **Literal test mentions:** none recorded.

H-T073 / EXPR.PLAN.SCALE

Source symbol(s): `komorebi_math.expression.engine._PlanBuilder._compile_uncached`. No attached provenance record under this label.

Locations: `expression/engine.py:1501`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T074 / EXPR.POLY.AFFINE

Source symbol(s): `komorebi_math.expression.polynomial`. No attached provenance record under this label.

Locations: `expression/polynomial.py:176`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: `EX-E-006`, `EX-E-007`. **Literal test mentions:** none recorded.

H-T075 / EXPR.POLY.ANTIDERIVATIVE

Source symbol(s): `komorebi_math.expression.polynomial`. No attached provenance record under this label.

Locations: `expression/polynomial.py:173`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: `EX-E-006`, `EX-E-007`. **Literal test mentions:** none recorded.

H-T076 / EXPR.POLY.BOUNDARY

Source symbol(s): `komorebi_math.expression.polynomial`. No attached provenance record under this label.

Locations: `expression/polynomial.py:178`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T077 / EXPR.POLY.CONSTANT

Source symbol(s): `komorebi_math.expression.polynomial`. No attached provenance record under this label.

Locations: `expression/polynomial.py:179`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T078 / EXPR.POLY.COORDINATE

Source symbol(s): `komorebi_math.expression.polynomial`, `komorebi_math.verify.expression.run_expression_verification.primitive_replay`. No attached provenance record under this label.

Locations: `expression/polynomial.py:171`, `verify/expression.py:164`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: `EX-E-006`, `EX-E-007`. **Literal test mentions:** none recorded.

H-T079 / EXPR.POLY.DERIVATIVE

Source symbol(s): `komorebi_math.expression.polynomial`. No attached provenance record under this label.

Locations: `expression/polynomial.py:172`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: `EX-E-006`, `EX-E-007`. **Literal test mentions:** none recorded.

H-T080 / EXPR.POLY.DIFFERENCE

Source symbol(s): `komorebi_math.expression.polynomial`. No attached provenance record under this label.

Locations: `expression/polynomial.py:177`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T081 / EXPR.POLY.EVALUATION

Source symbol(s): `komorebi_math.expression.polynomial`. No attached provenance record under this label.

Locations: `expression/polynomial.py:174`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T082 / EXPR.POLY.INTEGRATION

Source symbol(s): `komorebi_math.expression.polynomial`. No attached provenance record under this label.

Locations: `expression/polynomial.py:175`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: `EX-E-006`, `EX-E-007`. **Literal test mentions:** none recorded.

H-T083 / EXPR.REWRITE.ADD_ASSOC

Source symbol(s): `komorebi_math.expression.engine._local_rewrite`. No attached provenance record under this label.

Locations: `expression/engine.py:989`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T084 / EXPR.REWRITE.ADD_ZERO

Source symbol(s): `komorebi_math.expression.engine._local_rewrite`. No attached provenance record under this label.

Locations: `expression/engine.py:985`, `expression/engine.py:987`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T085 / EXPR.REWRITE.APPLY_IDENTITY

Source symbol(s): `komorebi_math.expression.engine._local_rewrite`. No attached provenance record under this label.

Locations: `expression/engine.py:1011`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T086 / EXPR.REWRITE.APPLY_LITERAL

Source symbol(s): `komorebi_math.expression.engine._local_rewrite`. No attached provenance record under this label.

Locations: `expression/engine.py:1016`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T087 / EXPR.REWRITE.APPLY_ZERO

Source symbol(s): `komorebi_math.expression.engine._local_rewrite`. No attached provenance record under this label.

Locations: `expression/engine.py:1013`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T088 / EXPR.REWRITE.BOUNDARY_TRANSPORT

Source symbol(s): `komorebi_math.expression.engine._primitive_pair_rewrite`. No attached provenance record under this label.

Locations: `expression/engine.py:959`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T089 / EXPR.REWRITE.COMPOSE_ASSOC

Source symbol(s): `komorebi_math.expression.engine._local_rewrite`. No attached provenance record under this label.

Locations: `expression/engine.py:1001`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: `EX-E-006`, `EX-E-007`. **Literal test mentions:** none recorded.

H-T090 / EXPR.REWRITE.COMPOSE_IDENTITY

Source symbol(s): `komorebi_math.expression.engine._local_rewrite`. No attached provenance record under this label.

Locations: `expression/engine.py:997`, `expression/engine.py:999`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T091 / EXPR.REWRITE.COMPOSE_ZERO

Source symbol(s): `komorebi_math.expression.engine._local_rewrite`. No attached provenance record under this label.

Locations: `expression/engine.py:994`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T092 / EXPR.REWRITE.COORDINATE_COCYCLE

Source symbol(s): `komorebi_math.expression.engine._primitive_pair_rewrite`. No attached provenance record under this label.

Locations: `expression/engine.py:888`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: `EX-E-006`, `EX-E-007`. **Literal test mentions:** none recorded.

H-T093 / EXPR.REWRITE.COORDINATE_IDENTITY

Source symbol(s): `komorebi_math.expression.engine._local_rewrite`. No attached provenance record under this label.

Locations: `expression/engine.py:1020`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T094 / EXPR.REWRITE.DENOTATION

Source symbol(s): `komorebi_math.expression.engine.simplify_expression`. No attached provenance record under this label.

Locations: `expression/engine.py:1167`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: `EX-E-006`, `EX-E-007`. **Literal test mentions:** none recorded.

H-T095 / EXPR.REWRITE.DERIVATIVE_NATURALITY

Source symbol(s): `komorebi_math.expression.engine._primitive_pair_rewrite`. No attached provenance record under this label.

Locations: `expression/engine.py:914`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: `EX-E-006`, `EX-E-007`. **Literal test mentions:** none recorded.

H-T096 / EXPR.REWRITE.DJ

Source symbol(s): `komorebi_math.expression.engine._primitive_pair_rewrite`, `komorebi_math.verify.expression.run_expression_verification.simplifier_preservation`. No attached provenance record under this label.

Locations: `expression/engine.py:892`, `verify/expression.py:309`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: `EX-E-006`, `EX-E-007`. **Literal test mentions:** none recorded.

H-T097 / EXPR.REWRITE.EVALUATION_TRANSPORT

Source symbol(s): `komorebi_math.expression.engine._primitive_pair_rewrite`. No attached provenance record under this label.

Locations: `expression/engine.py:951`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T098 / EXPR.REWRITE.FTC

Source symbol(s): `komorebi_math.expression.engine._primitive_pair_rewrite`. No attached provenance record under this label.

Locations: `expression/engine.py:937`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** `test_block20_expression_calculus.py`.

H-T099 / EXPR.REWRITE.INTEGRATION_TRANSPORT

Source symbol(s): `komorebi_math.expression.engine._primitive_pair_rewrite`. No attached provenance record under this label.

Locations: `expression/engine.py:955`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T100 / EXPR.REWRITE.JD_DEFECT

Source symbol(s): `komorebi_math.expression.engine._primitive_pair_rewrite`. No attached provenance record under this label.

Locations: `expression/engine.py:926`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** `test_block20_expression_calculus.py`.

H-T101 / EXPR.REWRITE.SCALE_ASSOC

Source symbol(s): `komorebi_math.expression.engine._local_rewrite`. No attached provenance record under this label.

Locations: `expression/engine.py:971`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T102 / EXPR.REWRITE.SCALE_ONE

Source symbol(s): `komorebi_math.expression.engine._local_rewrite`. No attached provenance record under this label.

Locations: `expression/engine.py:969`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T103 / EXPR.REWRITE.SCALE_ZERO

Source symbol(s): `komorebi_math.expression.engine._local_rewrite`. No attached provenance record under this label.

Locations: `expression/engine.py:967`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T104 / EXPR.TEST.CONDITIONAL_BINDING

Source symbol(s): `komorebi_math.verify.expression.run_expression_verification.binding_certificate_propagation`. No attached provenance record under this label.

Locations: `verify/expression.py:418`. **Direct base sites:** `H-C367`.

`H-C367`: status `Exactness.CONDITIONAL`; checked `()`; unchecked `(assumption,)`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T105 / EXPR.TYPE.FINITE

Source symbol(s): `komorebi_math.expression.engine.check_expression`. No attached provenance record under this label.

Locations: `expression/engine.py:506`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: `EX-E-006`, `EX-E-007`. **Literal test mentions:** none recorded.

H-T106 / GF-ANALYTIC-MASK-REALIZATION

Source symbol(s): `komorebi_math.generating.masks.AnalyticMaskApproximation.certificate`. No attached provenance record under this label.

Locations: `generating/masks.py:654`. **Direct base sites:** `H-C130`.

`H-C130`: status `Exactness.NUMERICAL`; checked `('polynomial mask has constant coefficient one', 'evaluation point lies inside the unit disk')`; unchecked `()`. Residual keys: `absolute_tail_bound`.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T107 / GF-FINITE-MASK-CARRY

Source symbol(s): `komorebi_math.generating.masks.FiniteMaskCarryPlan.certificate`. No attached provenance record under this label.

Locations: `generating/masks.py:255`. **Direct base sites:** `H-C128`.

`H-C128`: status `Exactness.EXACT`; checked `('finite mask', 'exact rational arithmetic')`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: `EX-E-029`. **Literal test mentions:** none recorded.

H-T108 / GF-KUMMER-LAYERS

Source symbol(s): `komorebi_math.generating.masks.KummerLayerDistribution.certificate`. No attached provenance record under this label.

Locations: `generating/masks.py:408`. **Direct base sites:** `H-C129`.

`H-C129`: status `Exactness.EXACT`; checked `('prime radix', 'finite borrow automaton')`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T109 / GF-MASK-INVERSE

Source symbol(s): `komorebi_math.generating.masks.certify_mahler_functional_equation`. No attached provenance record under this label.

Locations: `generating/masks.py:178`. **Direct base sites:** `H-C127`.

`H-C127`: status `Exactness.EXACT`; checked `('constant coefficients equal one', 'finite formal truncation')`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T110 / GF-POLYNOMIAL-COBOUNDARY

Source symbol(s): `komorebi_math.generating.masks.PolynomialCoboundaryResult.certificate`. No attached provenance record under this label.

Locations: `generating/masks.py:907`. **Direct base sites:** `H-C133`.

`H-C133`: status `Exactness.EXACT`; checked `('finite degree bound', 'unit rational mask')`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T111 / GF-RATIONAL-MASK-ALGEBRAIC

Source symbol(s): `komorebi_math.generating.masks.RationalMaskAlgebraicCertificate.certificate`. No attached provenance record under this label.

Locations: `generating/masks.py:772`. **Direct base sites:** `H-C131`.

`H-C131`: status `Exactness.EXACT`; checked `('prime characteristic', 'unit rational mask')`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T112 / GF-SIGNAL-FIR-BRIDGE

Source symbol(s): `komorebi_math.generating.adapters.FIRSeriesAdapter.certificate`. No attached provenance record under this label.

Locations: `generating/adapters.py:61`. **Direct base sites:** `H-C116`.

`H-C116`: status `Exactness.NUMERICAL`; checked `('formal z is interpreted as one sample of delay',)`; unchecked `()`. Residual keys: `coefficient_to_tap`.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T113 / INT.ADAPTER.DERIVATIVE

Source symbol(s): `komorebi_math.integration.adapters.AtomicIntegratedDerivativeProvider.certificate`. No attached provenance record under this label.

Locations: `integration/adapters.py:247`. **Direct base sites:** `H-C138`.

`H-C138`: status `self.exactness`; checked `('finite fixed weights', 'common provider source and target dimensions', 'linearity of finite differentiation')`; unchecked `('providers represent globally correct derivatives',)`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T114 / INT.ADAPTER.FRAME

Source symbol(s): `komorebi_math.integration.adapters.analysis_synthesis_integration_certificate`. No attached provenance record under this label.

Locations: `integration/adapters.py:372`. **Direct base sites:** `H-C139`.

`H-C139`: status `_weakest_exactness(Exactness.NUMERICAL, system_certificate.exactness, measure_certificate.exactness)`; checked `_merged_strings(system_certificate.assumptions_checked, measure_certificate.assumptions_checked, ('analysis-synthesis system certificate incorporated', ...)`; unchecked `_merged_strings(system_certificate.assumptions_unchecked, measure_certificate.assumptions_unchecked)`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T115 / INT.COORD.COVECTOR

Source symbol(s): `komorebi_math.integration.coordinates.PolynomialIntegrationCovector.certificate`. No attached provenance record under this label.

Locations: `integration/coordinates.py:183`, `integration/coordinates.py:202`. **Direct base sites:** `H-C140`, `H-C141`.

`H-C140`: status `status`; checked `('exact rational interval data', 'finite rational covector', 'coordinate dimension') + tuple(structure.assumptions_checked)`; unchecked `tuple(dict.fromkeys(unchecked))`. Residual keys: none recorded (computed/default when not literal). `H-C141`: status `Exactness.EXACT`; checked `('replayed exact graded polynomial basis witness', 'exact rational interval moments', 'finite coordinate truncation')`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: `EX-E-005`, `EX-E-007`. **Literal test mentions:** none recorded.

H-T116 / INT.COORD.FTC

Source symbol(s): `komorebi_math.integration.coordinates.coordinate_fundamental_theorem`. No attached provenance record under this label.

Locations: `integration/coordinates.py:297`. **Direct base sites:** `H-C142`.

`H-C142`: status `Exactness.EXACT`; checked `('exact derivative coordinate operator', 'exact integration covector', 'exact endpoint evaluation covectors')`; unchecked `()`. Residual keys: `coordinate_ftc`.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T117 / INT.COORD.TRANSPORT

Source symbol(s): `komorebi_math.integration.coordinates.coordinate_transport_identity`. No attached provenance record under this label.

Locations: `integration/coordinates.py:340`. **Direct base sites:** `H-C143`.

`H-C143`: status `Exactness.EXACT`; checked `('exact polynomial connection transform', 'contragredient covector action', 'common physical interval measure')`; unchecked `()`. Residual keys: `covector_transport`.

Observed examples: `EX-E-005`, `EX-E-007`. **Literal test mentions:** none recorded.

H-T118 / INT.EVAL.ATOM

Source symbol(s): `komorebi_math.integration.quadrature._atomic_integral`. No attached provenance record under this label.

Locations: `integration/quadrature.py:425`. **Direct base sites:** `H-C161`.

`H-C161`: status `Exactness.EXACT if exact else Exactness.NUMERICAL`; checked `('finite weighted sum over every atom', )`; unchecked `() if exact else ('floating-point roundoff is not enclosed',)`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T119 / INT.EVAL.ATOM.MEASURE

Source symbol(s): `komorebi_math.integration.quadrature.integrate`. No attached provenance record under this label.

Locations: `integration/quadrature.py:1122`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T120 / INT.EVAL.ATOMIC.PRODUCT

Source symbol(s): `komorebi_math.integration.quadrature._product_integral`. No attached provenance record under this label.

Locations: `integration/quadrature.py:863`. **Direct base sites:** `H-C166`.

`H-C166`: status `result_exactness`; checked `('one finite weighted sum over product support',) + tuple(measure_certificate.assumptions_checked)`; unchecked `unchecked`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T121 / INT.EVAL.ORIENTED.POLY

Source symbol(s): `komorebi_math.integration.quadrature.integrate_polynomial_oriented`. No attached provenance record under this label.

Locations: `integration/quadrature.py:1195`. **Direct base sites:** `H-C171`.

`H-C171`: status `Exactness.EXACT`; checked `('exact rational polynomial', 'oriented endpoint convention', 'nonnegative constant density')`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T122 / INT.EVAL.POLY.MOMENTS

Source symbol(s): `komorebi_math.integration.quadrature._exact_polynomial_integral`. No attached provenance record under this label.

Locations: `integration/quadrature.py:1066`. **Direct base sites:** `H-C170`.

`H-C170`: status `exactness`; checked `('finite sparse polynomial', 'measure monomial moments') + checked`; unchecked `unchecked`. Residual keys: none recorded (computed/default when not literal).

Observed examples: `EX-E-005`, `EX-E-007`. **Literal test mentions:** none recorded.

H-T123 / INT.EVAL.PUSHFORWARD.ATOMIC.CALLABLE

Source symbol(s): `komorebi_math.integration.quadrature.integrate`. No attached provenance record under this label.

Locations: `integration/quadrature.py:1142`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T124 / INT.EVAL.PUSHFORWARD.CALLABLE

Source symbol(s): `komorebi_math.integration.quadrature.integrate`. No attached provenance record under this label.

Locations: `integration/quadrature.py:1158`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T125 / INT.EVAL.ZERO.MEASURE

Source symbol(s): `komorebi_math.integration.quadrature._zero_interval_result`. No attached provenance record under this label.

Locations: `integration/quadrature.py:558`. **Direct base sites:** `H-C163`.

`H-C163`: status `Exactness.EXACT`; checked `('structurally zero finite measure',)`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T126 / INT.ID.COV.AFFINE

Source symbol(s): `komorebi_math.integration.identities.affine_interval_change_of_variables`. No attached provenance record under this label.

Locations: `integration/identities.py:314`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: `EX-E-005`, `EX-E-007`. **Literal test mentions:** none recorded.

H-T127 / INT.ID.FTC

Source symbol(s): `komorebi_math.integration.identities.fundamental_theorem`. No attached provenance record under this label.

Locations: `integration/identities.py:87`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: `EX-E-005`, `EX-E-007`. **Literal test mentions:** none recorded.

H-T128 / INT.ID.FUBINI

Source symbol(s): `komorebi_math.integration.identities.fubini_identity`. No attached provenance record under this label.

Locations: `integration/identities.py:214`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T129 / INT.ID.IBP

Source symbol(s): `komorebi_math.integration.identities.integration_by_parts`. No attached provenance record under this label.

Locations: `integration/identities.py:127`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T130 / INT.ID.PUSHFORWARD

Source symbol(s): `komorebi_math.integration.identities.pushforward_identity`. No attached provenance record under this label.

Locations: `integration/identities.py:272`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: `EX-E-005`, `EX-E-007`. **Literal test mentions:** none recorded.

H-T131 / INT.MAP.1

Source symbol(s): `komorebi_math.integration.measures.ExactAffineMap.certificate`. No attached provenance record under this label.

Locations: `integration/measures.py:288`. **Direct base sites:** `H-C145`.

`H-C145`: status `Exactness.EXACT`; checked `('exact rational matrix', 'exact rational shift', 'rectangular affine typing')`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: `EX-E-005`, `EX-E-007`. **Literal test mentions:** none recorded.

H-T132 / INT.MEASURE.ATOM.1

Source symbol(s): `komorebi_math.integration.measures.AtomicMeasure.certificate`. No attached provenance record under this label.

Locations: `integration/measures.py:471`. **Direct base sites:** `H-C148`.

`H-C148`: status `self.exactness`; checked `('finite support', 'finite coordinates', 'nonnegative weights')`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T133 / INT.MEASURE.ATOM.MASS

Source symbol(s): `komorebi_math.integration.measures.AtomicMeasure.mass`. No attached provenance record under this label.

Locations: `integration/measures.py:364`. **Direct base sites:** `H-C146`.

`H-C146`: status `self.exactness`; checked `('finite positive atomic weights',)`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T134 / INT.MEASURE.ATOM.MOMENT

Source symbol(s): `komorebi_math.integration.measures.AtomicMeasure.moment`. No attached provenance record under this label.

Locations: `integration/measures.py:410`. **Direct base sites:** `H-C147`.

`H-C147`: status `self.exactness`; checked `('finite atomic sum',)`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T135 / INT.MEASURE.INTERVAL.1

Source symbol(s): `komorebi_math.integration.measures.IntervalMeasure.certificate`. No attached provenance record under this label.

Locations: `integration/measures.py:678`. **Direct base sites:** `H-C151`.

`H-C151`: status `self.exactness`; checked `('ordered finite endpoints', 'constant nonnegative density')`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: `EX-E-005`, `EX-E-007`. **Literal test mentions:** none recorded.

H-T136 / INT.MEASURE.INTERVAL.MASS

Source symbol(s): `komorebi_math.integration.measures.IntervalMeasure.mass`. No attached provenance record under this label.

Locations: `integration/measures.py:549`. **Direct base sites:** `H-C149`.

`H-C149`: status `self.exactness`; checked `('bounded interval', 'constant nonnegative density')`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: `EX-E-005`, `EX-E-007`. **Literal test mentions:** none recorded.

H-T137 / INT.MEASURE.INTERVAL.MOMENT

Source symbol(s): `komorebi_math.integration.measures.IntervalMeasure.moment`. No attached provenance record under this label.

Locations: `integration/measures.py:585`. **Direct base sites:** `H-C150`.

`H-C150`: status `self.exactness`; checked `('closed-form monomial antiderivative',)`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: `EX-E-005`, `EX-E-007`. **Literal test mentions:** none recorded.

H-T138 / INT.MEASURE.PRODUCT.1

Source symbol(s): `komorebi_math.integration.measures.ProductMeasure.certificate`. No attached provenance record under this label.

Locations: `integration/measures.py:829`. **Direct base sites:** `H-C154`.

`H-C154`: status `_aggregate_exactness((self.exactness, *(certificate.exactness for certificate in certificates)))`; checked `_unique_assumptions(('typed finite measure factors', 'finite Cartesian product'), *(certificate.assumptions_checked for certificate in certificates))`; unchecked `_unique_assumptions(*(certificate.assumptions_unchecked for certificate in certificates))`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T139 / INT.MEASURE.PRODUCT.MASS

Source symbol(s): `komorebi_math.integration.measures.ProductMeasure.mass`. No attached provenance record under this label.

Locations: `integration/measures.py:742`. **Direct base sites:** `H-C152`.

`H-C152`: status `exactness`; checked `_unique_assumptions(('finite product mass',), *(certificate.assumptions_checked for certificate in certificates))`; unchecked `_unique_assumptions(*(certificate.assumptions_unchecked for certificate in certificates))`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T140 / INT.MEASURE.PRODUCT.MOMENT

Source symbol(s): `komorebi_math.integration.measures.ProductMeasure.moment`. No attached provenance record under this label.

Locations: `integration/measures.py:794`. **Direct base sites:** `H-C153`.

`H-C153`: status `exactness`; checked `_unique_assumptions(('factorization of product-measure monomial moments',), *(certificate.assumptions_checked for certificate in certificates))`; unchecked `_unique_assumptions(*(certificate.assumptions_unchecked for certificate in certificates))`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T141 / INT.MEASURE.PUSH.1

Source symbol(s): `komorebi_math.integration.measures.PushforwardMeasure.certificate`. No attached provenance record under this label.

Locations: `integration/measures.py:1024`. **Direct base sites:** `H-C157`.

`H-C157`: status `_aggregate_exactness((self.exactness, *(certificate.exactness for certificate in certificates)))`; checked `_unique_assumptions(('finite base measure', 'typed affine mapping' if exact_map else 'pushforward defined by precomposition'), *(certificate.assumptions...`; unchecked `_unique_assumptions(*(certificate.assumptions_unchecked for certificate in certificates), () if exact_map else _CALLA...`. Residual keys: none recorded (computed/default when not literal).

Observed examples: `EX-E-005`, `EX-E-007`. **Literal test mentions:** none recorded.

H-T142 / INT.MEASURE.PUSH.MASS

Source symbol(s): `komorebi_math.integration.measures.PushforwardMeasure.mass`. No attached provenance record under this label.

Locations: `integration/measures.py:917`. **Direct base sites:** `H-C155`.

`H-C155`: status `_aggregate_exactness((self.exactness, *(certificate.exactness for certificate in certificates)))`; checked `_unique_assumptions(('pushforward preserves total mass',), *(certificate.assumptions_checked for certificate in certificates))`; unchecked `_unique_assumptions(*(certificate.assumptions_unchecked for certificate in certificates), () if exact_map else _CALLA...`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T143 / INT.MEASURE.PUSH.MOMENT

Source symbol(s): `komorebi_math.integration.measures.PushforwardMeasure.moment`. No attached provenance record under this label.

Locations: `integration/measures.py:991`. **Direct base sites:** `H-C156`.

`H-C156`: status `_aggregate_exactness((certificate.exactness for certificate in certificates))`; checked `_unique_assumptions(('pushforward moment equals the pulled-back monomial integral',), *(certificate.assumptions_checked for certificate in certificates))`; unchecked `_unique_assumptions(*(certificate.assumptions_unchecked for certificate in certificates))`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T144 / INT.POLY.1

Source symbol(s): `komorebi_math.integration.polynomial.ExactPolynomial.certificate`. No attached provenance record under this label.

Locations: `integration/polynomial.py:344`. **Direct base sites:** `H-C158`.

`H-C158`: status `Exactness.EXACT`; checked `('positive ambient dimension', 'nonnegative integer exponents', 'exact rational coefficients', 'canonical sparse representation')`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T145 / INT.PUSH.ATOMIC.SNAPSHOT.POLY

Source symbol(s): `komorebi_math.integration.quadrature._exact_polynomial_integral`. No attached provenance record under this label.

Locations: `integration/quadrature.py:937`. **Direct base sites:** `H-C168`.

`H-C168`: status `Exactness.CONDITIONAL if unchecked else result.certificate.exactness`; checked `result.certificate.assumptions_checked + ('callable pushforward support materialized once',)`; unchecked `result.certificate.assumptions_unchecked + unchecked`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T146 / INT.PUSH.ATOMIC.SNAPSHOT.POLY.MEASURE

Source symbol(s): `komorebi_math.integration.quadrature._exact_polynomial_integral`. No attached provenance record under this label.

Locations: `integration/quadrature.py:950`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T147 / INT.PUSH.POLY

Source symbol(s): `komorebi_math.integration.quadrature._exact_polynomial_integral`. No attached provenance record under this label.

Locations: `integration/quadrature.py:970`. **Direct base sites:** `H-C169`.

`H-C169`: status `base.certificate.exactness`; checked `base.certificate.assumptions_checked + ('pushforward integral equals pullback integral by definition',)`; unchecked `base.certificate.assumptions_unchecked`. Residual keys: none recorded (computed/default when not literal).

Observed examples: `EX-E-005`, `EX-E-007`. **Literal test mentions:** none recorded.

H-T148 / INT.PUSH.POLY.MEASURE

Source symbol(s): `komorebi_math.integration.quadrature._exact_polynomial_integral`. No attached provenance record under this label.

Locations: `integration/quadrature.py:985`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: `EX-E-005`, `EX-E-007`. **Literal test mentions:** none recorded.

H-T149 / INT.QUAD.BOUND.ASSUMED

Source symbol(s): `komorebi_math.integration.quadrature.DerivativeBound.assumed`. No attached provenance record under this label.

Locations: `integration/quadrature.py:205`. **Direct base sites:** `H-C159`.

`H-C159`: status `Exactness.CONDITIONAL`; checked `()`; unchecked `(description,)`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T150 / INT.QUAD.POLY.BOUND

Source symbol(s): `komorebi_math.integration.quadrature.polynomial_derivative_bound`. No attached provenance record under this label.

Locations: `integration/quadrature.py:269`. **Direct base sites:** `H-C160`.

`H-C160`: status `Exactness.EXACT`; checked `('exact polynomial differentiation', 'coefficient triangle inequality', 'rational bounded interval') + tuple(structure.assumptions_checked)`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: `EX-E-005`, `EX-E-007`. **Literal test mentions:** none recorded.

H-T151 / INT.QUAD.RULE.1

Source symbol(s): `komorebi_math.integration.rules.QuadratureRule.certificate`. No attached provenance record under this label.

Locations: `integration/rules.py:492`. **Direct base sites:** `H-C176`.

`H-C176`: status `Exactness.EXACT if self.exact_data else Exactness.NUMERICAL`; checked `('finite nodes', 'finite signed weights', 'common node dimension', 'signed weights are not treated as a positive measure')`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T152 / INT.QUAD.RULE.APPLY

Source symbol(s): `komorebi_math.integration.rules.QuadratureRule.apply`. No attached provenance record under this label.

Locations: `integration/rules.py:243`. **Direct base sites:** `H-C172`.

`H-C172`: status `Exactness.EXACT if exact_evaluation else Exactness.NUMERICAL`; checked `('finite quadrature functional evaluation', 'no target-integral equality inferred')`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T153 / INT.QUAD.RULE.CERTIFIED.POLY

Source symbol(s): `komorebi_math.integration.rules.QuadratureRule.certify_polynomial_integral`. No attached provenance record under this label.

Locations: `integration/rules.py:470`. **Direct base sites:** `H-C175`.

`H-C175`: status `Exactness.EXACT`; checked `('exact monomial moments through polynomial degree', 'linearity of finite quadrature functional')`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T154 / INT.QUAD.RULE.DEGREE

Source symbol(s): `komorebi_math.integration.rules.QuadratureRule.degree_of_exactness`. No attached provenance record under this label.

Locations: `integration/rules.py:435`. **Direct base sites:** `H-C174`.

`H-C174`: status `certificate_exactness`; checked `('monomial moments compared consecutively from degree zero',) + tuple(measure_certificate.assumptions_checked)`; unchecked `() if exact_comparison else tuple(dict.fromkeys(inherited_unchecked)) + ('floating moment agreement implies exact alg...`. Residual keys: none recorded (computed/default when not literal).

Observed examples: `EX-E-005`, `EX-E-007`. **Literal test mentions:** none recorded.

H-T155 / INT.QUAD.RULE.POLY

Source symbol(s): `komorebi_math.integration.rules.QuadratureRule.integrate_polynomial`. No attached provenance record under this label.

Locations: `integration/rules.py:311`. **Direct base sites:** `H-C173`.

`H-C173`: status `Exactness.EXACT if exact_evaluation else Exactness.NUMERICAL`; checked `('finite polynomial', 'finite quadrature moments', 'no target-integral equality inferred')`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: `EX-E-005`, `EX-E-007`. **Literal test mentions:** none recorded.

H-T156 / INT.QUAD.SIMPSON

Source symbol(s): `komorebi_math.integration.quadrature._simpson_integral`. No attached provenance record under this label.

Locations: `integration/quadrature.py:646`. **Direct base sites:** `H-C164`.

`H-C164`: status `exactness`; checked `tuple(checked)`; unchecked `unchecked`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T157 / INT.QUAD.SIMPSON.EXACT.POLY

Source symbol(s): `komorebi_math.integration.quadrature._simpson_polynomial_integral`. No attached provenance record under this label.

Locations: `integration/quadrature.py:738`. **Direct base sites:** `H-C165`.

`H-C165`: status `exactness`; checked `checked`; unchecked `unchecked`. Residual keys: `richardson_difference`.

Observed examples: `EX-E-005`, `EX-E-007`. **Literal test mentions:** none recorded.

H-T158 / INT.QUAD.SIMPSON.EXACT.POLY.MEASURE

Source symbol(s): `komorebi_math.integration.quadrature.integrate`. No attached provenance record under this label.

Locations: `integration/quadrature.py:1112`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: `EX-E-005`, `EX-E-007`. **Literal test mentions:** none recorded.

H-T159 / INT.QUAD.SIMPSON.MEASURE

Source symbol(s): `komorebi_math.integration.quadrature.integrate`. No attached provenance record under this label.

Locations: `integration/quadrature.py:1128`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T160 / INT.QUAD.TENSOR.SIMPSON

Source symbol(s): `komorebi_math.integration.quadrature._product_integral`. No attached provenance record under this label.

Locations: `integration/quadrature.py:893`. **Direct base sites:** `H-C167`.

`H-C167`: status `Exactness.HEURISTIC`; checked `('finite tensor-product Simpson rule', 'finite integrand samples')`; unchecked `('tensor refinement estimate bounds the true multidimensional quadrature error',)`. Residual keys: `tensor_refinement_difference`.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T161 / INT.QUAD.TENSOR.SIMPSON.MEASURE

Source symbol(s): `komorebi_math.integration.quadrature._product_integral`. No attached provenance record under this label.

Locations: `integration/quadrature.py:917`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T162 / KM-API-010

Source symbol(s): `komorebi_math.compatibility.compare_public_api`. No attached provenance record under this label.

Locations: `compatibility.py:479`. **Direct base sites:** `H-C036`.

`H-C036`: status `Exactness.EXACT`; checked `('selected namespaces define explicit __all__ exports', 'export presence and broad symbol kind', 'deterministic callable signature text')`; unchecked `('behavioral and mathematical semantic equivalence', 'compatibility of undocumented implementation details')`. Residual keys: `removed_exports`, `kind_changes`, `signature_changes`.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T163 / KM-AS-01

Source symbol(s): `komorebi_math.coordinates.analysis_synthesis.AnalysisSynthesisSystem`. No attached provenance record under this label.

Locations: `coordinates/analysis_synthesis.py:142`, `coordinates/analysis_synthesis.py:164`, `coordinates/analysis_synthesis.py:185`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: `EX-E-004`, `EX-E-007`. **Literal test mentions:** none recorded.

H-T164 / KM-AS-02

Source symbol(s): `komorebi_math.coordinates.analysis_synthesis.AnalysisSynthesisBlocks.certificate`. No attached provenance record under this label.

Locations: `coordinates/analysis_synthesis.py:108`. **Direct base sites:** `H-C039`.

`H-C039`: status `Exactness.NUMERICAL`; checked `('finite analysis and synthesis matrices', 'coordinate groups form a partition', 'analysis--synthesis left inverse')`; unchecked `()`. Residual keys: `operator_block_reconstruction`.

Observed examples: `EX-E-004`. **Literal test mentions:** none recorded.

H-T165 / KM-AS-BLOCK-01

Source symbol(s): `komorebi_math.coordinates.frame_algebra.AnalysisSynthesisCoefficientBlocks.certificate`. No attached provenance record under this label.

Locations: `coordinates/frame_algebra.py:569`. **Direct base sites:** `H-C048`.

`H-C048`: status `Exactness.NUMERICAL`; checked `('source coefficient partition', 'target coefficient partition', 'canonical coefficient lift')`; unchecked `()`. Residual keys: `coefficient_block_assembly`.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T166 / KM-AS-EXACT-01

Source symbol(s): `komorebi_math.coordinates.frame_algebra.ExactAnalysisSynthesisSystem`. No attached provenance record under this label.

Locations: `coordinates/frame_algebra.py:654`, `coordinates/frame_algebra.py:679`, `coordinates/frame_algebra.py:694`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T167 / KM-AS-EXACT-DUAL-01

Source symbol(s): `komorebi_math.coordinates.frame_algebra.ExactAnalysisSynthesisSystem.dual`. No attached provenance record under this label.

Locations: `coordinates/frame_algebra.py:779`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T168 / KM-AS-EXACT-MAP-01

Source symbol(s): `komorebi_math.coordinates.frame_algebra.ExactAnalysisSynthesisMap`. No attached provenance record under this label.

Locations: `coordinates/frame_algebra.py:862`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T169 / KM-AS-EXACT-MAP-ADD-01

Source symbol(s): `komorebi_math.coordinates.frame_algebra.ExactAnalysisSynthesisMap.add`. No attached provenance record under this label.

Locations: `coordinates/frame_algebra.py:958`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T170 / KM-AS-EXACT-MAP-ADJOINT-01

Source symbol(s): `komorebi_math.coordinates.frame_algebra.ExactAnalysisSynthesisMap.adjoint`. No attached provenance record under this label.

Locations: `coordinates/frame_algebra.py:927`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T171 / KM-AS-EXACT-MAP-COMPOSE-01

Source symbol(s): `komorebi_math.coordinates.frame_algebra.ExactAnalysisSynthesisMap.then`. No attached provenance record under this label.

Locations: `coordinates/frame_algebra.py:913`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T172 / KM-AS-EXACT-MAP-DUAL-ADJOINT-01

Source symbol(s): `komorebi_math.coordinates.frame_algebra.ExactAnalysisSynthesisMap.dual_adjoint`. No attached provenance record under this label.

Locations: `coordinates/frame_algebra.py:935`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T173 / KM-AS-EXACT-MAP-SCALE-01

Source symbol(s): `komorebi_math.coordinates.frame_algebra.ExactAnalysisSynthesisMap.scaled`. No attached provenance record under this label.

Locations: `coordinates/frame_algebra.py:944`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T174 / KM-AS-EXACT-SUM-01

Source symbol(s): `komorebi_math.coordinates.frame_algebra.ExactAnalysisSynthesisSystem.direct_sum`. No attached provenance record under this label.

Locations: `coordinates/frame_algebra.py:804`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T175 / KM-AS-EXACT-TENSOR-01

Source symbol(s): `komorebi_math.coordinates.frame_algebra.ExactAnalysisSynthesisSystem.tensor_product`. No attached provenance record under this label.

Locations: `coordinates/frame_algebra.py:818`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T176 / KM-AS-GAUGE-01

Source symbol(s): `komorebi_math.coordinates.frame_algebra.AnalysisSynthesisGauge.certificate`. No attached provenance record under this label.

Locations: `coordinates/frame_algebra.py:311`. **Direct base sites:** `H-C046`.

`H-C046`: status `Exactness.NUMERICAL`; checked `('finite analysis--synthesis presentation', 'analysis--synthesis left inverse', 'coefficient quotient by synthesis kernel')`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T177 / KM-AS-GAUGE-DUAL-01

Source symbol(s): `komorebi_math.coordinates.frame_algebra.AnalysisSynthesisGauge.dual`. No attached provenance record under this label.

Locations: `coordinates/frame_algebra.py:232`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T178 / KM-AS-GAUGE-SUM-01

Source symbol(s): `komorebi_math.coordinates.frame_algebra.AnalysisSynthesisGauge.direct_sum`. No attached provenance record under this label.

Locations: `coordinates/frame_algebra.py:270`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T179 / KM-AS-GAUGE-TENSOR-01

Source symbol(s): `komorebi_math.coordinates.frame_algebra.AnalysisSynthesisGauge.tensor_product`. No attached provenance record under this label.

Locations: `coordinates/frame_algebra.py:283`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T180 / KM-AS-MAP-01

Source symbol(s): `komorebi_math.coordinates.frame_algebra.AnalysisSynthesisMap`. No attached provenance record under this label.

Locations: `coordinates/frame_algebra.py:338`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T181 / KM-AS-MAP-ADD-01

Source symbol(s): `komorebi_math.coordinates.frame_algebra.AnalysisSynthesisMap.add`. No attached provenance record under this label.

Locations: `coordinates/frame_algebra.py:439`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T182 / KM-AS-MAP-ADJOINT-01

Source symbol(s): `komorebi_math.coordinates.frame_algebra.AnalysisSynthesisMap.adjoint`. No attached provenance record under this label.

Locations: `coordinates/frame_algebra.py:409`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T183 / KM-AS-MAP-COMPOSE-01

Source symbol(s): `komorebi_math.coordinates.frame_algebra.AnalysisSynthesisMap.then`. No attached provenance record under this label.

Locations: `coordinates/frame_algebra.py:393`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T184 / KM-AS-MAP-DUAL-ADJOINT-01

Source symbol(s): `komorebi_math.coordinates.frame_algebra.AnalysisSynthesisMap.dual_adjoint`. No attached provenance record under this label.

Locations: `coordinates/frame_algebra.py:417`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T185 / KM-AS-MAP-ID-01

Source symbol(s): `komorebi_math.coordinates.frame_algebra.AnalysisSynthesisMap.identity`. No attached provenance record under this label.

Locations: `coordinates/frame_algebra.py:358`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T186 / KM-AS-MAP-SCALE-01

Source symbol(s): `komorebi_math.coordinates.frame_algebra.AnalysisSynthesisMap.scaled`. No attached provenance record under this label.

Locations: `coordinates/frame_algebra.py:428`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T187 / KM-B15-T01

Source symbol(s): `komorebi_math.refinement.polyphase.PolyphaseRefinement.certificate`. No attached provenance record under this label.

Locations: `refinement/polyphase.py:276`. **Direct base sites:** `H-C303`.

`H-C303`: status `exactness`; checked `('positive divisible packet orders', 'finite polyphase coordinate record', 'mixed-radix address law a = r + M s', 'coarsen after refine identity', 'nest...`; unchecked `()`. Residual keys: `coarsen_after_refine`, `direct_fine_analysis`.

Observed examples: `EX-E-003`, `EX-E-007`. **Literal test mentions:** none recorded.

H-T188 / KM-B15-T02

Source symbol(s): `komorebi_math.refinement.polyphase.PolyphaseRefinementTower.coherence_certificate`. No attached provenance record under this label.

Locations: `refinement/polyphase.py:626`. **Direct base sites:** `H-C304`.

`H-C304`: status `Exactness.EXACT if exact else Exactness.NUMERICAL`; checked `('nested divisible refinement tower', 'associativity of mixed-radix address composition')`; unchecked `()`. Residual keys: `sequential_vs_direct_refinement`.

Observed examples: `EX-E-003`. **Literal test mentions:** none recorded.

H-T189 / KM-B15-T03

Source symbol(s): `komorebi_math.refinement.polyphase.LatticePolyphaseRefinement.certificate`. No attached provenance record under this label.

Locations: `refinement/polyphase.py:857`. **Direct base sites:** `H-C305`.

`H-C305`: status `Exactness.EXACT if exact else Exactness.NUMERICAL`; checked `('componentwise divisible rectangular strides', 'finite rectangular lattice polyphase coordinates', 'componentwise address law a = r + M*s')`; unchecked `()`. Residual keys: `coarsen_after_refine`, `direct_fine_analysis`.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T190 / KM-DOCS-017

Source symbol(s): `komorebi_math.release.documentation.check_stable_documentation`. No attached provenance record under this label.

Locations: `release/documentation.py:296`. **Direct base sites:** `H-C313`.

`H-C313`: status `Exactness.EXACT`; checked `('one documentation record per stable export', 'exact callable signatures and first-line summaries', 'no deprecated stable export')`; unchecked `('narrative completeness beyond the committed stable reference',)`. Residual keys: `missing`, `extra`, `changed`, `deprecated`.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T191 / KM-DSP-C01

Declared claim: Finite lattice overlap-add certificate; `FINITE_CERTIFICATE`. Symbol: `komorebi_math.signals.windows.overlap_add_certificate`.

Declared hypotheses: uniform integer hop; infinite shift lattice away from finite boundaries. Suite label: `signals.windows`.

Locations: `signals/windows.py:213`, `signals/windows.py:243`. **Direct base sites:** `H-C361`.

`H-C361`: status `Exactness.NUMERICAL`; checked `('finite window', 'uniform integer hop')`; unchecked `('finite-record boundary coverage',)`. Residual keys: `overlap_spread`, `minimum_overlap`.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T192 / KM-DSP-C02

Source symbol(s): `komorebi_math.signals.filtering.FIRFilter.certificate`. No attached provenance record under this label.

Locations: `signals/filtering.py:181`. **Direct base sites:** `H-C336`.

`H-C336`: status `Exactness.NUMERICAL`; checked `('finite real linear-phase design', 'declared gain normalization')`; unchecked `()`. Residual keys: `tap_symmetry`, `reference_gain`.

Observed examples: `EX-E-040`. **Literal test mentions:** none recorded.

H-T193 / KM-DSP-C03

Source symbol(s): `komorebi_math.signals.denoise.denoising_risk_certificate`. No attached provenance record under this label.

Locations: `signals/denoise.py:349`. **Direct base sites:** `H-C328`.

`H-C328`: status `Exactness.NUMERICAL`; checked `('supplied clean finite reference',)`; unchecked `()`. Residual keys: `noisy_mse`, `denoised_mse`.

Observed examples: `EX-E-040`. **Literal test mentions:** none recorded.

H-T194 / KM-DSP-C04

Source symbol(s): `komorebi_math.signals.recursive.LinearFilter.stability_certificate`. No attached provenance record under this label.

Locations: `signals/recursive.py:160`. **Direct base sites:** `H-C345`.

`H-C345`: status `Exactness.NUMERICAL`; checked `('finite rational transfer function',)`; unchecked `()`. Residual keys: `maximum_pole_radius`, `stability_margin`.

Observed examples: `EX-E-040`. **Literal test mentions:** none recorded.

H-T195 / KM-DSP-D01

Declared claim: Finite deterministic analysis window; `CLASSICAL`. Symbol: `komorebi_math.signals.windows.get_window`.

Declared hypotheses: positive finite window length. Suite label: `signals.windows`.

Locations: `signals/windows.py:48`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T196 / KM-DSP-D02

Declared claim: Finite uniform signal framing; `DEFINITION`. Symbol: `komorebi_math.signals.time_frequency.frame_signal`.

Declared hypotheses: finite one dimensional signal; positive frame and hop lengths. Suite label: `signals.time_frequency`.

Locations: `signals/time_frequency.py:83`, `signals/time_frequency.py:94`. **Direct base sites:** `H-C356`.

`H-C356`: status `Exactness.EXACT`; checked `('complete equal-length frames', 'uniform integer hop')`; unchecked `()`. Residual keys: `uncovered_tail`.

Observed examples: `EX-E-040`. **Literal test mentions:** none recorded.

H-T197 / KM-DSP-D03

Declared claim: Finite short-time Fourier transform; `CLASSICAL`. Symbol: `komorebi_math.signals.time_frequency.stft`.

Declared hypotheses: uniform finite framing; declared window and Fourier length. Suite label: `signals.time_frequency`.

Locations: `signals/time_frequency.py:247`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T198 / KM-DSP-D04

Declared claim: Finite spectrogram scaling; `CLASSICAL`. Symbol: `komorebi_math.signals.time_frequency.spectrogram`.

Declared hypotheses: finite STFT; declared spectral scaling. Suite label: `signals.time_frequency`.

Locations: `signals/time_frequency.py:432`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T199 / KM-DSP-D05

Declared claim: Finite FIR frequency response; `CLASSICAL`. Symbol: `komorebi_math.signals.filtering.fir_frequency_response`.

Declared hypotheses: finite impulse response; positive sample rate. Suite label: `signals.filtering`.

Declared claim: Finite modified periodogram; `CLASSICAL`. Symbol: `komorebi_math.signals.spectral.periodogram`.

Declared hypotheses: finite signal; declared window and scaling. Suite label: `signals.spectral`.

Locations: `signals/filtering.py:95`, `signals/spectral.py:157`, `signals/spectral.py:192`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: `EX-E-040`. **Literal test mentions:** none recorded.

H-T200 / KM-DSP-D06

Declared claim: Windowed-sinc finite FIR design; `CLASSICAL`. Symbol: `komorebi_math.signals.filtering.design_fir`.

Declared hypotheses: cutoffs strictly inside Nyquist; finite tap count. Suite label: `signals.filtering`.

Declared claim: Finite Welch power spectral estimate; `CLASSICAL`. Symbol: `komorebi_math.signals.spectral.welch_psd`.

Declared hypotheses: complete uniform finite segments; declared averaging rule. Suite label: `signals.spectral`.

Locations: `signals/filtering.py:207`, `signals/spectral.py:237`, `signals/spectral.py:279`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: `EX-E-040`. **Literal test mentions:** none recorded.

H-T201 / KM-DSP-D07

Declared claim: Finite Welch cross spectrum and magnitude-squared coherence; `CLASSICAL`. Symbol: `komorebi_math.signals.spectral.cross_spectrum`.

Declared hypotheses: paired finite segments; nonzero auto spectra for interpreted transfer bins. Suite label: `signals.spectral`.

Locations: `signals/spectral.py:297`, `signals/spectral.py:369`. **Direct base sites:** `H-C353`.

`H-C353`: status `Exactness.NUMERICAL`; checked `('paired complete frames', 'finite DFT scaling')`; unchecked `('linear time-invariant data-generating model', 'statistical consistency')`. Residual keys: `coherence_bound`.

Observed examples: `EX-E-040`. **Literal test mentions:** none recorded.

H-T202 / KM-DSP-D10

Declared claim: Finite Haar coefficient shrinkage; `CLASSICAL`. Symbol: `komorebi_math.signals.denoise.wavelet_denoise`.

Declared hypotheses: finite dyadic Haar decomposition; declared threshold rule. **Declared limits:** noise-model optimality is not inferred from one finite signal. Suite label: `signals.denoise`.

Locations: `signals/denoise.py:241`, `signals/denoise.py:299`. **Direct base sites:** `H-C327`.

`H-C327`: status `Exactness.NUMERICAL`; checked `('finite threshold recurrence', 'complete Haar synthesis')`; unchecked `('Gaussian white-noise interpretation' if sigma is None else 'declared noise scale matches the application', 'thresho...`. Residual keys: `shrinkage_monotonicity`.

Observed examples: `EX-E-040`. **Literal test mentions:** none recorded.

H-T203 / KM-DSP-D11

Declared claim: Windowed finite rational-resampling anti-alias filter; `CLASSICAL`. Symbol: `komorebi_math.signals.resampling.design_resample_filter`.

Declared hypotheses: positive reduced rational rate; odd finite tap count. **Declared limits:** zero-extension boundary convention; finite-window approximation. Suite label: `signals.resampling`.

Locations: `signals/resampling.py:71`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T204 / KM-DSP-D12

Declared claim: Finite least-mean-squares adaptation; `CLASSICAL`. Symbol: `komorebi_math.signals.adaptive.lms_filter`.

Declared hypotheses: finite paired signals; positive step size. **Declared limits:** the finite routine does not assert stochastic convergence. Suite label: `signals.adaptive`.

Locations: `signals/adaptive.py:99`, `signals/adaptive.py:121`. **Direct base sites:** `H-C323`.

`H-C323`: status `Exactness.NUMERICAL`; checked `('finite recursion', 'declared parameter ranges')`; unchecked `('persistent excitation', 'stochastic independence assumptions', 'convergence for the supplied step or forgetting fac...`. Residual keys: `nonfinite_entries`.

Observed examples: `EX-E-040`. **Literal test mentions:** none recorded.

H-T205 / KM-DSP-D13

Declared claim: Finite normalized least-mean-squares adaptation; `CLASSICAL`. Symbol: `komorebi_math.signals.adaptive.nlms_filter`.

Declared hypotheses: finite paired signals; positive normalization floor. **Declared limits:** the finite routine does not assert stochastic convergence. Suite label: `signals.adaptive`.

Locations: `signals/adaptive.py:174`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T206 / KM-DSP-D14

Declared claim: Finite recursive least-squares adaptation; `CLASSICAL`. Symbol: `komorebi_math.signals.adaptive.rls_filter`.

Declared hypotheses: finite paired signals; positive inverse-covariance initialization; forgetting factor in (0, 1]. **Declared limits:** the finite routine does not assert a stochastic identification theorem. Suite label: `signals.adaptive`.

Locations: `signals/adaptive.py:237`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T207 / KM-DSP-D26

Declared claim: Direct-form II transposed finite recursion; `CLASSICAL`. Symbol: `komorebi_math.signals.recursive.lfilter`.

Declared hypotheses: finite normalized transfer coefficients. Suite label: `signals.recursive`.

Locations: `signals/recursive.py:172`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T208 / KM-DSP-D27

Declared claim: Normalized audio-EQ biquad construction; `CLASSICAL`. Symbol: `komorebi_math.signals.recursive.design_biquad`.

Declared hypotheses: frequency strictly inside Nyquist; positive Q. Suite label: `signals.recursive`.

Locations: `signals/recursive.py:194`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T209 / KM-DSP-D28

Declared claim: Zero-padded finite FFT convolution; `CLASSICAL`. Symbol: `komorebi_math.signals.fft_convolution.fft_convolve`.

Declared hypotheses: finite one-dimensional inputs; zero padding to linear-convolution length. Suite label: `signals.fft_convolution`.

Locations: `signals/fft_convolution.py:48`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T210 / KM-DSP-T01

Declared claim: Finite window gain and equivalent noise bandwidth; `CLASSICAL`. Symbol: `komorebi_math.signals.windows.window_metrics`.

Declared hypotheses: finite nonzero coherent gain; positive sample rate. Suite label: `signals.windows`.

Locations: `signals/windows.py:155`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T211 / KM-DSP-T02

Declared claim: Canonical finite lattice dual window; `PROVED`. Symbol: `komorebi_math.signals.windows.canonical_dual_window`.

Declared hypotheses: nonzero squared overlap profile; uniform integer hop. Suite label: `signals.windows`.

Locations: `signals/windows.py:262`, `signals/windows.py:295`. **Direct base sites:** `H-C362`.

`H-C362`: status `Exactness.NUMERICAL`; checked `('uniform integer hop',)`; unchecked `('finite-record boundary coverage',)`. Residual keys: `dual_overlap`, `minimum_squared_overlap`.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T212 / KM-DSP-T03

Declared claim: Normalized finite STFT overlap-add reconstruction; `PROVED`. Symbol: `komorebi_math.signals.time_frequency.istft`.

Declared hypotheses: no destructive detrending; nonzero finite overlap denominator. Suite label: `signals.time_frequency`.

Locations: `signals/time_frequency.py:235`, `signals/time_frequency.py:327`, `signals/time_frequency.py:390`. **Direct base sites:** `H-C357`, `H-C358`.

H-C357: status Exactness.NUMERICAL; checked ('matching finite source', 'normalized overlap-add'); unchecked (). Residual keys: stft_roundtrip. H-C358: status Exactness.NUMERICAL; checked ('finite Fourier inversion', 'finite overlap normalization'); unchecked ('source agreement requires a separate roundtrip comparison',). Residual keys: normalized_overlap_identity, minimum_overlap.

Observed examples: EX-E-040. **Literal test mentions:** none recorded.

H-T213 / KM-DSP-T04

Declared claim: Finite discrete Parseval identity; PROVED. Symbol: komorebi_math.signals.spectral.parseval_certificate.

Declared hypotheses: NumPy DFT normalization convention; finite one dimensional signal. Suite label: signals.spectral.

Locations: signals/spectral.py:397, signals/spectral.py:418. **Direct base sites:** H-C354.

H-C354: status Exactness.NUMERICAL; checked ('finite DFT pair',); unchecked (). Residual keys: parseval.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T214 / KM-DSP-T05

Declared claim: Chunk-invariant finite FIR execution; PROVED. Symbol: komorebi_math.signals.filtering.stream_fir_filter.

Declared hypotheses: ordered nonoverlapping chunks; common FIR taps. Suite label: signals.filtering.

Locations: signals/filtering.py:390, signals/filtering.py:425. **Direct base sites:** H-C337.

H-C337: status Exactness.NUMERICAL; checked ('ordered complete chunk partition', 'finite causal FIR'); unchecked (). Residual keys: streaming_batch_equivalence.

Observed examples: EX-E-040. **Literal test mentions:** none recorded.

H-T215 / KM-DSP-T06

Declared claim: Finite multilevel Haar reconstruction; PROVED. Symbol: komorebi_math.signals.denoise.haar_decompose.

Declared hypotheses: signal length divisible by the requested dyadic depth. Suite label: signals.denoise.

Declared claim: Finite centered polyphase rational-resampling identity; PROVED. Symbol: komorebi_math.signals.resampling.resample_poly_result.

Declared hypotheses: zero-extension boundary convention; odd centered FIR kernel. Suite label: signals.resampling.

Locations: signals/denoise.py:178, signals/denoise.py:190, signals/resampling.py:186, signals/resampling.py:215. **Direct base sites:** H-C326, H-C347.

H-C326: status Exactness.NUMERICAL; checked ('complete finite Haar decomposition',); unchecked (). Residual keys: haar_reconstruction, haar_energy. H-C347: status Exactness.NUMERICAL; checked ('centered zero-extension convention', 'reduced rational rate'); unchecked ('application-specific alias rejection', 'boundary-transient acceptability'). Residual keys: polyphase_reference, dc_gain.

Observed examples: EX-E-040. **Literal test mentions:** none recorded.

H-T216 / KM-DSP-T07

Declared claim: Chunk-invariant finite STFT framing; PROVED. Symbol: komorebi_math.signals.streaming.stream_stft.

Declared hypotheses: contiguous complete chunk partition; zero-padded terminal framing. Suite label: signals.streaming.

Locations: signals/streaming.py:263, signals/streaming.py:330. **Direct base sites:** H-C355.

H-C355: status Exactness.NUMERICAL; checked ('contiguous declared chunks', 'matching finite STFT parameters'); unchecked (). Residual keys: stream_batch_coefficients, stream_batch_times.

Observed examples: EX-E-040. **Literal test mentions:** none recorded.

H-T217 / KM-DSP-T09

Source symbol(s): komorebi_math.signals.recursive.stream_lfilter. No attached provenance record under this label.

Locations: signals/recursive.py:292. **Direct base sites:** H-C346.

H-C346: status Exactness.NUMERICAL; checked ('zero initial state', 'ordered contiguous chunks'); unchecked (). Residual keys: streaming_batch_max_abs.

Observed examples: EX-E-040. **Literal test mentions:** none recorded.

H-T218 / KM-DSP-T10

Declared claim: Finite overlap-add convolution identity; PROVED. Symbol: komorebi_math.signals.fft_convolution.overlap_add_convolve.

Declared hypotheses: ordered contiguous chunks; zero-padded FFT blocks. Suite label: signals.fft_convolution.

Locations: signals/fft_convolution.py:192, signals/fft_convolution.py:236. **Direct base sites:** H-C331.

H-C331: status Exactness.NUMERICAL; checked ('finite ordered input', 'linear zero-padded convolution'); unchecked (). Residual keys: overlap_add_direct_max_abs.

Observed examples: EX-E-040. **Literal test mentions:** none recorded.

H-T219 / KM-EVIDENCE-017

Source symbol(s): komorebi_math.release.evidence.check_validation_evidence. No attached provenance record under this label.

Locations: release/evidence.py:286. **Direct base sites:** H-C314.

H-C314: status Exactness.EXACT; checked ('current-version execution exists', 'required platform families are covered', 'inherited evidence is authorized by the upgrade contract', 'evidence sou...; unchecked () if require_current_all_platforms else ('release candidate requires current-version evidence on every platform fami.... Residual keys: missing_platforms, missing_current_platforms, invalid_records, invalid_sources.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T220 / KM-FINITE-GROUP

Source symbol(s): komorebi_math.langlands.finite_image.groups.FiniteGroup.certificate. No attached provenance record under this label.

Locations: langlands/finite_image/groups.py:158. **Direct base sites:** H-C182.

H-C182: status Exactness.EXACT; checked ('finite explicit group table',); unchecked (). Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T221 / KM-GOLDEN-011

Source symbol(s): komorebi_math.golden._exact_snapshot, komorebi_math.golden._result, komorebi_math.golden.run_behavioral_golden_fixture. No attached provenance record under this label.

Locations: golden.py:104, golden.py:228, golden.py:590. **Direct base sites:** H-C135, H-C136, H-C137.

H-C135: status Exactness.EXACT; checked ('committed finite golden coefficient table',); unchecked (). Residual keys: none recorded (computed/default when not literal). H-C136: status exactness; checked ('closed built-in golden case evaluator', 'committed explicit inputs and expected output'); unchecked (). Residual keys: golden_residual. H-C137: status Exactness.EXACT; checked ('fixture schema and digest', 'closed case-kind registry', 'deterministic committed expected values'); unchecked ('the fixture set is representative rather than exhaustive',). Residual keys: failed_cases.

Observed examples: EX-E-002. **Literal test mentions:** none recorded.

H-T222 / KM-GPU-016

Source symbol(s): `komorebi_math.release.gpu.qualify_gpu`. No attached provenance record under this label.

Locations: `release/gpu.py:152`. **Direct base sites:** `H-C315`.

`H-C315`: status `Exactness.NUMERICAL`; checked `tuple((f'isolated {name} CUDA execution' for name in passed_required))`; unchecked `tuple((f'{result.backend} was optional for this invocation' for result in results if not required[result.backend]))`. Residual keys: `failed_required_backends`.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T223 / KM-INDUCED-REPRESENTATION

Source symbol(s): `komorebi_math.langlands.finite_image.characters.InducedRepresentation.certificate`. No attached provenance record under this label.

Locations: `langlands/finite_image/characters.py:104`. **Direct base sites:** `H-C179`.

`H-C179`: status `Exactness.NUMERICAL`; checked `('finite explicit group', 'multiplicative inducing character')`; unchecked `()`. Residual keys: `representation_law`.

Observed examples: `EX-E-020`. **Literal test mentions:** none recorded.

H-T224 / KM-PERSIST-016

Source symbol(s): `komorebi_math.persistence_contract.compare_portable_codec_contract`. No attached provenance record under this label.

Locations: `persistence_contract.py:268`. **Direct base sites:** `H-C273`.

`H-C273`: status `Exactness.EXACT`; checked `('package-owned portable codec tags', 'codec implementation type identity', 'monotone codec versions', 'adjacent migration path for each version increase')`; unchecked `('semantic equivalence of extension-owned codecs', 'compatibility of undocumented serialized implementation details')`. Residual keys: `removed_tags`, `type_changes`, `version_decreases`, `missing_migrations`.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T225 / KM-READINESS-017

Source symbol(s): `komorebi_math.release.readiness.check_beta_readiness`. No attached provenance record under this label.

Locations: `release/readiness.py:42`. **Direct base sites:** `H-C316`.

`H-C316`: status `Exactness.EXACT`; checked `tuple((x.name for x in checks if x.passed))`; unchecked `tuple(unchecked)`. Residual keys: `failed_checks`.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T226 / KM-REL-16

Source symbol(s): `komorebi_math.qualification.ReleaseQualificationReport.certificate`. No attached provenance record under this label.

Locations: `qualification.py:308`. **Direct base sites:** `H-C274`.

`H-C274`: status `Exactness.NUMERICAL`; checked `tuple((check.name for check in self.verification.checks if check.passed))`; unchecked `() if self.gpu is not None else ('hardware-backed CUDA qualification not requested',)`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T227 / KM-RELEASE-CONTRACT-021

Source symbol(s): `komorebi_math.release.contract.check_release_contract`. No attached provenance record under this label.

Locations: `release/contract.py:91`. **Direct base sites:** `H-C312`.

`H-C312`: status `Exactness.EXACT`; checked `tuple((x.name for x in checks if x.passed))`; unchecked `('current-version execution on all platforms is enforced by RC readiness', 'hardware performance is not semantic comp...`. Residual keys: `failed_checks`.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T228 / KM-STABLE-CONTRACT-016

Source symbol(s): `komorebi_math.stable_contract.check_stable_contract`. No attached provenance record under this label.

Locations: `stable_contract.py:53`. **Direct base sites:** `H-C363`.

`H-C363`: status `Exactness.EXACT`; checked `('stable exports and signatures', 'committed stable mathematical behavior', 'portable codec identity and migration paths')`; unchecked `('performance stability across hardware', 'compatibility of research and experimental namespaces')`. Residual keys: `api_failures`, `semantic_failures`, `persistence_failures`.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T229 / KM-STABLE-SEM-016

Source symbol(s): `komorebi_math.stable_semantics.run_stable_semantics_fixture`. No attached provenance record under this label.

Locations: `stable_semantics.py:415`. **Direct base sites:** `H-C364`.

`H-C364`: status `Exactness.EXACT`; checked `('fixture schema and canonical digest', 'closed stable semantic case registry', 'exact committed expected values', 'major stable API families represented')`; unchecked `('fixture coverage is representative rather than exhaustive', 'optional hardware backend equivalence')`. Residual keys: `failed_cases`.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T230 / KM-UPGRADE-017

Source symbol(s): `komorebi_math.release.upgrade.check_upgrade_contract`. No attached provenance record under this label.

Locations: `release/upgrade.py:134`. **Direct base sites:** `H-C317`.

`H-C317`: status `Exactness.EXACT`; checked `('stable API predecessor manifests', 'closed stable semantics', 'portable codec continuity', 'digest-checked predecessor records')`; unchecked `('undocumented implementation objects', 'pickle payloads', 'third-party codecs')`. Residual keys: `failed_checks`.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T231 / NGF.BELL_DIGIT_SUM_REDUCTION

Source symbol(s): `komorebi_math.generating.bell.BellDigitReduction.certificate`. No attached provenance record under this label.

Locations: `generating/bell.py:187`. **Direct base sites:** `H-C120`.

`H-C120`: status `Exactness.EXACT`; checked `('prime modulus', 'factorial_power >= 2', 'prime-1 divides factorial_power-1')`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T232 / NGF.BELL_RESIDUE_AUTOMATON

Source symbol(s): `komorebi_math.generating.bell.BellResidueAutomaton.certificate`. No attached provenance record under this label.

Locations: `generating/bell.py:380`. **Direct base sites:** `H-C121`.

`H-C121`: status `Exactness.EXACT`; checked `('prime modulus', 'factorial_power >= 2', 'prime-1 divides factorial_power-1')`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T233 / NGF.FINITE_NORMALIZED_SERIES

Source symbol(s): `komorebi_math.generating.finite_series.FiniteNormalizedSeries.certificate`. No attached provenance record under this label.

Locations: `generating/finite_series.py:423`. **Direct base sites:** `H-C126`.

`H-C126`: status `Exactness.EXACT`; checked `('finite degree', 'explicit coefficient domain')`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: `EX-E-010`. **Literal test mentions:** none recorded.

H-T234 / NGF.POINTED_SUBINDEX

Source symbol(s): `komorebi_math.generating.bell.PointedSubindexCertificate.certificate`. No attached provenance record under this label.

Locations: `generating/bell.py:132`. **Direct base sites:** `H-C118`.

`H-C118`: status `Exactness.EXACT`; checked `('prime modulus', 'positive index')`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T235 / NGF.UNIVERSAL_PRODUCT_AND_TRANSFER

Source symbol(s): `komorebi_math.generating.finite_series.SeriesTransferCertificate.certificate`. No attached provenance record under this label.

Locations: `generating/finite_series.py:77`. **Direct base sites:** `H-C123`.

`H-C123`: status `Exactness.EXACT`; checked `('finite truncation', 'exact coefficient domain')`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T236 / PSC-I-D01

Declared claim: Typed operator grammar; `DEFINITION`; Paper I. Symbol: `komorebi_math.core.grammar.OperatorGrammar`.

Declared hypotheses: finite vertex set; typed arrows. **Declared limits:** relations are finite linear combinations of typed paths. Suite label: `core.grammar`.

Locations: `core/grammar.py:66`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T237 / PSC-I-D05

Declared claim: Realized operator algebra; DEFINITION; Paper I. Symbol: komorebi_math.core.algebra.OperatorAlgebra.

Declared hypotheses: finite matrix generators. Suite label: core.algebra.

Locations: core/algebra.py:30, core/algebra.py:138. **Direct base sites:** H-C065.

H-C065: status Exactness.EXACT if self.backend.exact else Exactness.NUMERICAL; checked ('generator dimensions', 'linear independence', 'multiplicative closure'); unchecked (). Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T238 / PSC-I-T01

Declared claim: Typed realization and quotient path algebra correspondence; PROVED; Paper I. Symbol: komorebi_math.core.realization.OperatorRealization.

Declared hypotheses: finite dimensional matrix realization. Suite label: core.realization.

Locations: core/realization.py:24, core/realization.py:109. **Direct base sites:** H-C073.

H-C073: status exactness; checked ('operator shapes', 'typed path composition', 'declared relations'); unchecked (). Residual keys: none recorded (computed/default when not literal).

Observed examples: EX-E-057. **Literal test mentions:** none recorded.

H-T239 / PSC-I-T02

Declared claim: Finite seed hull generation; PROVED; Paper I. Symbol: komorebi_math.core.seeds.SeedSystem.

Declared hypotheses: finite dimensional realized operator family. **Declared limits:** numerical rank depends on the declared tolerance. Suite label: core.seeds.

Locations: core/seeds.py:43, core/seeds.py:147. **Direct base sites:** H-C074.

H-C074: status Exactness.EXACT if self.backend.exact else Exactness.NUMERICAL; checked ('operator dimensions', 'seed dimensions', 'finite orbit closure'); unchecked (). Residual keys: none recorded (computed/default when not literal).

Observed examples: EX-E-043. **Literal test mentions:** none recorded.

H-T240 / PSC-I-T10

Declared claim: Semisimple seed rank formula; PROVED; Paper I. Symbol: komorebi_math.core.seeds.semisimple_seed_rank.

Declared hypotheses: semisimple Artinian acting algebra. Suite label: core.seed_rank.

Locations: core/seeds.py:269. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T241 / PSC-I-T13

Declared claim: Single operator primary seed rank formula; PROVED; Paper I. Symbol: komorebi_math.core.seeds.single_operator_seed_rank.

Declared hypotheses: exact matrix over the rational field. **Declared limits:** the current executable factorization path is implemented over rational entries. Suite label: core.seed_rank.

Locations: core/seeds.py:311. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T242 / PSC-I-T20

Declared claim: Exact finite packet coordinates; `PROVED`; Paper I. Symbol: `komorebi_math.core.packets.PacketSystem`.

Declared hypotheses: finite complete family of pairwise annihilating idempotents. Suite label: `core.packets`.

Locations: `core/packets.py:42`, `core/packets.py:119`, `core/packets.py:144`. **Direct base sites:** `H-C071`, `H-C072`.

`H-C071`: status `Exactness.EXACT if self.backend.exact else Exactness.NUMERICAL`; checked `('common dimension', 'idempotence', 'annihilation', 'completeness')`; unchecked `()`. Residual keys: `idempotence`, `annihilation`, `completeness`. `H-C072`: status `Exactness.EXACT if self.backend.exact else Exactness.NUMERICAL`; checked `('packet system axioms',)`; unchecked `()`. Residual keys: `self_adjointness`.

Observed examples: `EX-E-002`, `EX-E-015`, `EX-E-031`, `EX-E-037`, `EX-E-042`, `EX-E-043`, `EX-E-047`, `EX-E-048`, `EX-E-049`, `EX-E-050`. **Literal test mentions:** `test_certificates_registry.py`, `test_unified_extensions_cli.py`.

H-T243 / PSC-I-T27

Declared claim: Transfer block algebra; `PROVED`; Paper I. Symbol: `komorebi_math.core.operators.PacketOperator`.

Declared hypotheses: finite source and target packet systems. Suite label: `core.operators`.

Locations: `core/operators.py:15`, `core/operators.py:93`. **Direct base sites:** `H-C070`.

`H-C070`: status `Exactness.EXACT if self.backend.exact else Exactness.NUMERICAL`; checked `('source packet axioms', 'target packet axioms', 'block shapes')`; unchecked `()`. Residual keys: `block_reprojection`.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T244 / PSC-I-T29

Source symbol(s): `komorebi_math.core.transfer.block_from_paths`. No attached provenance record under this label.

Locations: `core/transfer.py:76`. **Direct base sites:** `H-C076`.

`H-C076`: status `Exactness.EXACT if operator.backend.exact else Exactness.NUMERICAL`; checked `('packet endomorphism', 'fixed path length')`; unchecked `()`. Residual keys: `path_sum`.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T245 / PSC-I-T40

Declared claim: Packet transfer graph; `DEFINITION`; Paper I. Symbol: `komorebi_math.core.graphs.TransferGraph`.

Declared hypotheses: none listed. Suite label: `core.graphs`.

Locations: `core/graphs.py:15`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T246 / PSC-I-T44

Source symbol(s): `komorebi_math.core.graphs.TransferGraph.source_component_certificate`. No attached provenance record under this label.

Locations: `core/graphs.py:195`. **Direct base sites:** `H-C066`.

`H-C066`: status `Exactness.CONDITIONAL`; checked `('finite transfer graph', 'source strongly connected components')`; unchecked `('edge maps are surjective on the declared packet spaces', 'each source component is generated by one packet pure seed')`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T247 / PSC-I-T50

Declared claim: Naturality of seed hulls and packet blocks; `PROVED`; Paper I. Symbol: `komorebi_math.core.morphisms.OperatorMorphism`.

Declared hypotheses: declared intertwining equations. Suite label: `core.morphisms`.

Locations: `core/morphisms.py:17`, `core/morphisms.py:72`, `core/morphisms.py:173`. **Direct base sites:** `H-C067`, `H-C069`.

`H-C067`: status `Exactness.EXACT if self.backend.exact else Exactness.NUMERICAL`; checked `('operator dimensions', 'declared operator map')`; unchecked `()`. Residual keys: none recorded (computed/default when not literal). `H-C069`: status `Exactness.EXACT if backend.exact else Exactness.NUMERICAL`; checked `('operator intertwining', 'declared seed images')`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T248 / PSC-I-T52

Source symbol(s): `komorebi_math.core.morphisms.PacketMorphism.certificate`. No attached provenance record under this label.

Locations: `core/morphisms.py:122`. **Direct base sites:** `H-C068`.

`H-C068`: status `Exactness.EXACT if backend.exact else Exactness.NUMERICAL`; checked `('source packets', 'target packets', 'declared aggregation map')`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T249 / PSC-II-C09

Declared claim: Finite coprime residue factorization; `PROVED`; Paper II. Symbol: `komorebi_math.refinement.crt.crt_many`.

Declared hypotheses: pairwise coprime positive moduli. Suite label: `refinement.crt`.

Locations: `refinement/crt.py:67`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T250 / PSC-II-C11

Source symbol(s): `komorebi_math.refinement.weighted.WeightedTransferGraph.exact_inverse_if_acyclic`. No attached provenance record under this label.

Locations: `refinement/weighted.py:341`. **Direct base sites:** `H-C311`.

`H-C311`: status `Exactness.EXACT if self.backend.exact else Exactness.NUMERICAL`; checked `('acyclic transfer graph', 'invertible packet diagonal')`; unchecked `()`. Residual keys: `inverse_residual`.

Observed examples: `EX-E-047`. **Literal test mentions:** none recorded.

H-T251 / PSC-II-C21

Source symbol(s): `komorebi_math.refinement.defects.LinearRefinementDefect.polynomial`. No attached provenance record under this label.

Locations: `refinement/defects.py:122`. **Direct base sites:** `H-C296`.

`H-C296`: status `Exactness.EXACT if self.backend.exact else Exactness.NUMERICAL`; checked `('finite polynomial',)`; unchecked `()`. Residual keys: `polynomial_identity`.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T252 / PSC-II-D02

Declared claim: Packet refinement; `DEFINITION`; Paper II. Symbol: `komorebi_math.refinement.towers.PacketRefinement`.

Declared hypotheses: finite packet systems on one ambient space. Suite label: `refinement.towers`.

Locations: `refinement/towers.py:16`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T253 / PSC-II-D05

Declared claim: Residue packet system; `DEFINITION`; Paper II. Symbol: `komorebi_math.refinement.residue.residue_packet_system`.

Declared hypotheses: finite coordinate length; positive modulus. Suite label: `refinement.residue`.

Locations: `refinement/residue.py:17`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T254 / PSC-II-D12

Declared claim: Lattice packet data; `DEFINITION`; Paper II. Symbol: `komorebi_math.refinement.lattices.LatticePacketData`.

Declared hypotheses: rational packet projectors; full lattice basis. Suite label: `refinement.lattices`.

Locations: `refinement/lattices.py:77`, `refinement/lattices.py:140`. **Direct base sites:** `H-C300`.

`H-C300`: status `Exactness.EXACT`; checked `('rational packet axioms', 'full lattice basis') + (('integral split inclusion',) if self.split_basis is not None else ())`; unchecked `()`. Residual keys: `packet_axioms`.

Observed examples: `EX-E-016`. **Literal test mentions:** none recorded.

H-T255 / PSC-II-D19

Declared claim: Linear refinement defect; `DEFINITION`; Paper II. Symbol: `komorebi_math.refinement.defects.LinearRefinementDefect`.

Declared hypotheses: linear fine and coarse operators; linear coarsening map. Suite label: `refinement.defects`.

Locations: `refinement/defects.py:25`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T256 / PSC-II-SNF

Declared claim: Smith normal form over the integers; `CLASSICAL`; Paper II. Symbol: `komorebi_math.refinement.smith.smith_decomposition`.

Declared hypotheses: integer matrix. Suite label: `refinement.smith`.

Locations: `refinement/smith.py:67`, `refinement/smith.py:78`. **Direct base sites:** `H-C306`.

`H-C306`: status `Exactness.EXACT`; checked `('integer matrix', 'unimodular transforms', 'diagonal divisibility')`; unchecked `()`. Residual keys: `smith_identity`.

Observed examples: `EX-E-016`. **Literal test mentions:** none recorded.

H-T257 / PSC-II-T03

Source symbol(s): `komorebi_math.refinement.towers.PacketRefinement.certificate`. No attached provenance record under this label.

Locations: `refinement/towers.py:72`. **Direct base sites:** `H-C308`.

`H-C308`: status `Exactness.EXACT if self.backend.exact else Exactness.NUMERICAL`; checked `('packet axioms', 'one parent per fine packet', 'coarse projector equals child sum')`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: `EX-E-047`. **Literal test mentions:** none recorded.

H-T258 / PSC-II-T04

Declared claim: Exact coordinate change across one refinement; `PROVED`; Paper II. Symbol: `komorebi_math.refinement.towers.PacketRefinement.coarsen`.

Declared hypotheses: exact packet refinement. Suite label: `refinement.towers`.

Locations: `refinement/towers.py:83`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T259 / PSC-II-T05

Declared claim: Coherence of finite refinement towers; `PROVED`; Paper II. Symbol: `komorebi_math.refinement.towers.RefinementTower`.

Declared hypotheses: adjacent exact packet refinements. Suite label: `refinement.towers`.

Locations: `refinement/towers.py:143`, `refinement/towers.py:201`. **Direct base sites:** `H-C309`.

`H-C309`: status `Exactness.EXACT if all((certificate.exactness is Exactness.EXACT for certificate in certificates)) els...`; checked `('all adjacent refinements', 'level coherence')`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T260 / PSC-II-T06

Declared claim: Exact residue analysis and synthesis; `PROVED`; Paper II. Symbol: `komorebi_math.refinement.residue.residue_analyze`.

Declared hypotheses: finite sequence; positive modulus. Suite label: `refinement.residue`.

Locations: `refinement/residue.py:106`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T261 / PSC-II-T07

Declared claim: Nested residue refinement; `PROVED`; Paper II. Symbol: `komorebi_math.refinement.residue.nested_residue_address`.

Declared hypotheses: positive outer and inner moduli. Suite label: `refinement.residue`.

Locations: `refinement/residue.py:142`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T262 / PSC-II-T08

Declared claim: Coprime factorization of residue addresses; `PROVED`; Paper II. Symbol: `komorebi_math.refinement.crt.crt_pair`.

Declared hypotheses: pairwise coprime positive moduli. Suite label: `refinement.crt`.

Locations: `refinement/crt.py:37`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T263 / PSC-II-T10

Declared claim: Weighted inverse expansion; `PROVED`; Paper II. Symbol: `komorebi_math.refinement.weighted.WeightedTransferGraph`.

Declared hypotheses: invertible packet diagonal; positive declared transfer weights. Suite label: `refinement.weighted`.

Locations: `refinement/weighted.py:65`, `refinement/weighted.py:296`. **Direct base sites:** `H-C310`.

`H-C310`: status `exactness`; checked `('packet support', 'positive path weights', 'invertible packet diagonal')`; unchecked `assumptions_unchecked`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T264 / PSC-II-T13

Declared claim: Gluing module and exact separation weight; `PROVED`; Paper II. Symbol: `komorebi_math.refinement.lattices.LatticePacketData.gluing_module`.

Declared hypotheses: basis of the split lattice is supplied. Suite label: `refinement.lattices`.

Declared claim: Exact separation weight from the lattice action; `PROVED`; Paper II. Symbol: `komorebi_math.refinement.lattices.LatticePacketData.projector_weights`.

Declared hypotheses: rational projectors; discrete valuation at the declared prime. Suite label: `refinement.lattices`.

Locations: `refinement/lattices.py:154`, `refinement/lattices.py:192`, `refinement/lattices.py:212`. **Direct base sites:** `H-C301`.

`H-C301`: status `Exactness.EXACT`; checked `('Smith invariants', 'projector denominator valuations')`; unchecked `()`. Residual keys: `weight_difference`.

Observed examples: `EX-E-016`. **Literal test mentions:** none recorded.

H-T265 / PSC-II-T15

Source symbol(s): `komorebi_math.refinement.lattices.refinement_weight_monotonicity`. No attached provenance record under this label.

Locations: `refinement/lattices.py:274`. **Direct base sites:** `H-C302`.

`H-C302`: status `Exactness.EXACT`; checked `('common lattice', 'exact packet refinement')`; unchecked `()`. Residual keys: `monotonicity_gap`.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T266 / PSC-II-T18

Declared claim: Smith weighted congruence solve; PROVED; Paper II. Symbol: komorebi_math.refinement.smith.SmithCongruenceProblem.

Declared hypotheses: integer matrix; prime power modulus. Suite label: refinement.smith.

Locations: refinement/smith.py:140, refinement/smith.py:236. **Direct base sites:** H-C307.

H-C307: status Exactness.EXACT; checked ('Smith decomposition', 'prime power modulus', 'coordinate divisibility tests'); unchecked (). Residual keys: modular_residual.

Observed examples: EX-E-016. **Literal test mentions:** none recorded.

H-T267 / PSC-II-T20

Declared claim: Power defect identity; PROVED; Paper II. Symbol: komorebi_math.refinement.defects.LinearRefinementDefect.power.

Declared hypotheses: positive integer power. Suite label: refinement.defects.

Locations: refinement/defects.py:65, refinement/defects.py:93. **Direct base sites:** H-C295.

H-C295: status Exactness.EXACT if self.backend.exact else Exactness.NUMERICAL; checked ('linear defect definition', 'positive exponent'); unchecked (). Residual keys: power_identity.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T268 / PSC-II-T22

Declared claim: Resolvent defect identity; PROVED; Paper II. Symbol: komorebi_math.refinement.defects.LinearRefinementDefect.resolvent.

Declared hypotheses: both resolvents exist. Suite label: refinement.defects.

Locations: refinement/defects.py:134, refinement/defects.py:157. **Direct base sites:** H-C297.

H-C297: status Exactness.EXACT if self.backend.exact else Exactness.NUMERICAL; checked ('fine resolvent exists', 'coarse resolvent exists'); unchecked (). Residual keys: resolvent_identity.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T269 / PSC-II-T24

Declared claim: Exponential defect identity; PROVED; Paper II. Symbol: komorebi_math.refinement.defects.LinearRefinementDefect.exponential.

Declared hypotheses: finite matrices. Suite label: refinement.defects.

Locations: refinement/defects.py:169, refinement/defects.py:210. **Direct base sites:** H-C298.

H-C298: status Exactness.EXACT if self.backend.exact else Exactness.NUMERICAL; checked ('matrix exponentials exist', 'block Duhamel identity'); unchecked (). Residual keys: exponential_identity.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T270 / PSC-II-T25

Declared claim: Multilevel defect decomposition; PROVED; Paper II. Symbol: komorebi_math.refinement.defects.multilevel_defect.

Declared hypotheses: finite chain of compatible dimensions. Suite label: refinement.defects.

Locations: refinement/defects.py:244, refinement/defects.py:290. **Direct base sites:** H-C299.

H-C299: status Exactness.EXACT if algebra.exact else Exactness.NUMERICAL; checked ('adjacent dimensions', 'finite refinement chain'); unchecked (). Residual keys: multilevel_identity.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T271 / PSC-III-A10

Declared claim: Projection-cached packet-jet execution plan; PROVED; Paper III scalable runtime. Symbol: komorebi_math.nonlinear.execution.PacketJetExecutionPlan.

Declared hypotheses: finite packet systems; declared zero packet blocks are certified. Suite label: nonlinear.execution.

Locations: nonlinear/execution.py:48, nonlinear/execution.py:166, nonlinear/execution.py:314. **Direct base sites:** H-C223, H-C224.

H-C223: status family.exactness; checked ('all packet-jet addresses were enumerated', 'only certified zero addresses were pruned', 'active addresses were grouped by ordered input packet tuple'); unchecked ('support classification is incomplete, so untested blocks remain active',) if support is not None and (not support.c.... Residual keys: address_partition. H-C224: status family.exactness; checked ('source directions projected once per packet', 'one derivative call per active input address', 'target packets reconstruct the retained outputs', 'prun...; unchecked ('global correctness of the derivative provider',). Residual keys: planned_reconstruction.

Observed examples: EX-E-002, EX-E-049, EX-E-050. **Literal test mentions:** none recorded.

H-T272 / PSC-III-A10G

Source symbol(s): komorebi_math.nonlinear.execution.CupyPacketJetExecutor.execute, komorebi_math.nonlinear.execution.CupyPacketJetExecutor.hardware_certificate. No attached provenance record under this label.

Locations: nonlinear/execution.py:579, nonlinear/execution.py:628. **Direct base sites:** H-C225, H-C226.

H-C225: status Exactness.NUMERICAL; checked ('CuPy installed', 'packet projectors transferred to the active device', 'device derivative provider used without host round trips inside the kernel', '...; unchecked ('cross-backend agreement with a CPU reference', 'performance comparison under a matched synchronized timing contract'). Residual keys: device_reconstruction. H-C226: status Exactness.NUMERICAL; checked ('CuPy runtime loaded', 'active CUDA device queried'); unchecked ('matched performance benchmark',). Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T273 / PSC-III-A61

Declared claim: Finite packet-jet execution profile; HEURISTIC; Block 11 runtime. Symbol: komorebi_math.nonlinear.profiling.profile_packet_jet_runtime.

Declared hypotheses: fixed local process and supplied finite workload. **Declared limits:** timings are environment dependent; tracemalloc observes Python-managed allocations rather than total device or process memory. Suite label: nonlinear.profiling.

Locations: nonlinear/profiling.py:214, nonlinear/profiling.py:308, nonlinear/profiling.py:430. **Direct base sites:** H-C252, H-C253.

H-C252: status Exactness.EXACT; checked ('finite packet counts and derivative order', 'complete family address enumeration', 'execution-plan grouping and pruning certificate', 'ambient standar...; unchecked ('wall-clock cost per derivative call', 'backend-specific kernel fusion, caching, and transfer overhead'). Residual keys: address_count_mismatch. H-C253: status Exactness.HEURISTIC; checked ('all exact finite operation counts passed their structural certificate', 'every stage used the same supplied family and direction tuple', 'timing and a...; unchecked ('scheduler isolation, fixed CPU frequency, cache state, and thermal stability', 'cross-platform or cross-device comp.... Residual keys: none recorded (computed/default when not literal).

Observed examples: EX-E-002. **Literal test mentions:** none recorded.

H-T274 / PSC-III-C03

Source symbol(s): `komorebi_math.nonlinear.coordinates.NonlinearPacketMap.fixed_point_residual`. No attached provenance record under this label.

Locations: `nonlinear/coordinates.py:99`. **Direct base sites:** `H-C208`.

`H-C208`: status `Exactness.EXACT if self.source.backend.exact else Exactness.NUMERICAL`; checked `('source and target packet equivalence',)`; unchecked `()`. Residual keys: `fixed_point_residual_norm`.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T275 / PSC-III-C08

Declared claim: Finite packet jet support test; `FINITE_CERTIFICATE`; Paper III runtime. Symbol: `komorebi_math.nonlinear.jets.packet_jet_support`.

Declared hypotheses: finite dimensional packet ranges; basis test of a multilinear map. Suite label: `nonlinear.jets`.

Locations: `nonlinear/jets.py:343`, `nonlinear/jets.py:408`. **Direct base sites:** `H-C245`.

`H-C245`: status `exactness`; checked `('packet range bases', 'multilinearity', 'tested packet addresses')`; unchecked `('untested addresses due to evaluation limit',) if untested else ()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: `EX-E-002`, `EX-E-031`, `EX-E-049`, `EX-E-050`. **Literal test mentions:** none recorded.

H-T276 / PSC-III-C09

Declared claim: Finite packet Taylor reconstruction; `FINITE_CERTIFICATE`; Paper III. Symbol: `komorebi_math.nonlinear.jets.packet_taylor`.

Declared hypotheses: declared derivatives through the truncation order. **Declared limits:** no automatic bound on the omitted Taylor remainder. Suite label: `nonlinear.jets`.

Locations: `nonlinear/jets.py:440`, `nonlinear/jets.py:492`. **Direct base sites:** `H-C246`.

`H-C246`: status `exactness`; checked `('packet reconstruction of every retained derivative order',)`; unchecked `('Taylor remainder beyond the retained order', 'global correctness of user supplied derivatives')`. Residual keys: `observed_truncation_residual`.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T277 / PSC-III-C15

Source symbol(s): `komorebi_math.nonlinear.trees.quadratic_catalan_majorant`. No attached provenance record under this label.

Locations: `nonlinear/trees.py:421`. **Direct base sites:** `H-C270`.

`H-C270`: status `Exactness.NUMERICAL`; checked `('nonnegative scalar norms',)`; unchecked `result.certificate.assumptions_unchecked`. Residual keys: `catalan_discriminant_defect`.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T278 / PSC-III-C34

Source symbol(s): `komorebi_math.nonlinear.correction.LyapunovSchmidtReduction.reduced_second_derivative`. No attached provenance record under this label.

Locations: `nonlinear/correction.py:517`. **Direct base sites:** `H-C215`.

`H-C215`: status `Exactness.NUMERICAL`; checked `('kernel input projection', 'cokernel output projection')`; unchecked `('declared second derivative is globally correct',)`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T279 / PSC-III-C60

Source symbol(s): `komorebi_math.nonlinear.transport.KatoTransport.refinement_certificate`. No attached provenance record under this label.

Locations: `nonlinear/transport.py:451`. **Direct base sites:** `H-C268`.

`H-C268`: status `Exactness.NUMERICAL`; checked `('nested midpoint discretizations', 'endpoint residual comparison')`; unchecked `('asymptotic convergence outside the two sampled resolutions',)`. Residual keys: `coarse_fine_difference`, `endpoint_intertwining`.

Observed examples: `EX-E-019`. **Literal test mentions:** none recorded.

H-T280 / PSC-III-D01

Declared claim: Nonlinear packet coordinate map; `DEFINITION`; Paper III. Symbol: `komorebi_math.nonlinear.coordinates.NonlinearPacketMap`.

Declared hypotheses: finite source and target packet systems. Suite label: `nonlinear.coordinates`.

Locations: `nonlinear/coordinates.py:15`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T281 / PSC-III-D02

Declared claim: Declared finite dimensional derivative provider; `DEFINITION`; Paper III runtime. Symbol: `komorebi_math.nonlinear.derivatives.CallableDerivativeProvider`.

Declared hypotheses: declared directional derivatives. Suite label: `nonlinear.derivatives`.

Locations: `nonlinear/derivatives.py:96`, `nonlinear/derivatives.py:227`. **Direct base sites:** `H-C217`.

`H-C217`: status `self.exactness`; checked `('source and target dimensions', 'finite derivative symmetry probes')`; unchecked `tuple(unchecked)`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T282 / PSC-III-D04

Declared claim: Symmetric polynomial derivative tensors; `DEFINITION`; Paper III runtime. Symbol: `komorebi_math.nonlinear.derivatives.PolynomialDerivativeProvider`.

Declared hypotheses: declared symmetric coefficient tensors. Suite label: `nonlinear.derivatives`.

Locations: `nonlinear/derivatives.py:317`, `nonlinear/derivatives.py:464`. **Direct base sites:** `H-C218`.

`H-C218`: status `self.exactness`; checked `('tensor dimensions', 'adjacent source-axis symmetry')`; unchecked `()`. Residual keys: `tensor_symmetry`.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T283 / PSC-III-D04P

Declared claim: Sparse polynomial derivative provider; `DEFINITION`; Paper III scalable runtime. Symbol: `komorebi_math.nonlinear.sparse.SparsePolynomialDerivativeProvider`.

Declared hypotheses: finite sparse symmetric coefficient tensors. Suite label: `nonlinear.sparse`.

Locations: `nonlinear/sparse.py:345`, `nonlinear/sparse.py:619`. **Direct base sites:** `H-C257`.

`H-C257`: status `Exactness.EXACT if exact else Exactness.NUMERICAL`; checked `('symmetric orbit expansion', 'finite value probes', 'finite directional derivative probes')`; unchecked `('global equality beyond the supplied probes',) if probes else ()`. Residual keys: `sparse_dense_equivalence`.

Observed examples: `EX-E-050`. **Literal test mentions:** none recorded.

H-T284 / PSC-III-D04S

Declared claim: Sparse symmetric derivative tensor; `DEFINITION`; Paper III scalable runtime. Symbol: `komorebi_math.nonlinear.sparse.SparseSymmetricTensor`.

Declared hypotheses: finite dimensions; symmetric source axes. Suite label: `nonlinear.sparse`.

Locations: `nonlinear/sparse.py:99`, `nonlinear/sparse.py:313`. **Direct base sites:** `H-C256`.

`H-C256`: status `exactness`; checked `('canonical symmetric orbit keys', 'dense expansion', 'dense recompression')`; unchecked `()`. Residual keys: `dense_roundtrip`.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T285 / PSC-III-D06

Declared claim: Finite packet jet; `DEFINITION`; Paper III. Symbol: `komorebi_math.nonlinear.jets.PacketJet`.

Declared hypotheses: finite packet systems; declared finite dimensional derivative provider. Suite label: `nonlinear.jets`.

Locations: `nonlinear/jets.py:71`, `nonlinear/jets.py:180`. **Direct base sites:** `H-C243`.

`H-C243`: status `self.exactness`; checked `('symmetric declared derivative', 'corresponding packet address permutation')`; unchecked `('global correctness of user supplied derivatives',)`. Residual keys: `jet_symmetry`.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T286 / PSC-III-D10

Declared claim: Nonlinear refinement defect; `DEFINITION`; Paper III. Symbol: `komorebi_math.nonlinear.defects.NonlinearRefinementDefect`.

Declared hypotheses: fine and coarse maps; linear coarsening maps. Suite label: `nonlinear.defects`.

Locations: `nonlinear/defects.py:20`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T287 / PSC-III-D12

Declared claim: Composed derivative provider; `DEFINITION`; Paper III runtime. Symbol: `komorebi_math.nonlinear.composition.ComposedDerivativeProvider`.

Declared hypotheses: compatible finite dimensional derivative providers. Suite label: `nonlinear.composition`.

Locations: `nonlinear/composition.py:390`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T288 / PSC-III-D13

Declared claim: Linear correction datum and cokernel obstruction; `DEFINITION`; Paper III. Symbol: `komorebi_math.nonlinear.obstruction.LinearObstructionProblem`.

Declared hypotheses: finite dimensional linear correction map. Suite label: `nonlinear.obstruction`.

Declared claim: Finite ordered rooted tree; `DEFINITION`; Paper III. Symbol: `komorebi_math.nonlinear.trees.OrderedRootedTree`.

Declared hypotheses: ordered children; internal arity at least two. Suite label: `nonlinear.trees`.

Locations: `nonlinear/obstruction.py:36`, `nonlinear/trees.py:98`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T289 / PSC-III-D17

Declared claim: Finite abelian grading group; `DEFINITION`; Paper III runtime. Symbol: `komorebi_math.nonlinear.graded.FiniteAbelianGroup`.

Declared hypotheses: finite explicit operation table. Suite label: `nonlinear.graded`.

Locations: `nonlinear/graded.py:75`, `nonlinear/graded.py:165`. **Direct base sites:** `H-C234`.

`H-C234`: status `Exactness.EXACT`; checked `('closure', 'identity', 'inverses', 'commutativity', 'associativity')`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T290 / PSC-III-D38

Declared claim: Finite matrix transport system; `DEFINITION`; Paper III. Symbol: `komorebi_math.nonlinear.holonomy.TransportGraph`.

Declared hypotheses: connected finite graph; invertible edge transports; one model fiber dimension. Suite label: `nonlinear.holonomy`.

Locations: `nonlinear/holonomy.py:48`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T291 / PSC-III-D50

Declared claim: Finite decreasing filtration over a prime field; `DEFINITION`; Paper III runtime. Symbol: `komorebi_math.nonlinear.filtered.FilteredVectorSpace`.

Declared hypotheses: explicit nested finite-dimensional subspaces. Suite label: `nonlinear.filtered`.

Locations: `nonlinear/filtered.py:179`, `nonlinear/filtered.py:286`. **Direct base sites:** `H-C227`.

`H-C227`: status `Exactness.EXACT`; checked `('full F^0', 'nested levels', 'terminal zero level')`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T292 / PSC-III-D56

Declared claim: Smooth finite packet system; `DEFINITION`; Paper III shared runtime. Symbol: `komorebi_math.nonlinear.transport.MovingPacketSystem`.

Declared hypotheses: declared differentiable complete orthogonal packet family. Suite label: `nonlinear.transport`.

Locations: `nonlinear/transport.py:149`, `nonlinear/transport.py:247`. **Direct base sites:** `H-C264`.

`H-C264`: status `Exactness.NUMERICAL`; checked `('centered finite-difference comparison', 'differentiated projector identity', 'differentiated partition of unity')`; unchecked `('analytic differentiability beyond the sampled point',)`. Residual keys: `finite_difference`, `projector_tangent`, `partition_derivative`.

Observed examples: `EX-E-019`. **Literal test mentions:** none recorded.

H-T293 / PSC-III-D58

Declared claim: Finite packet-jet snapshot; FINITE_CERTIFICATE; Block 10 runtime. Symbol: komorebi_math.nonlinear.snapshots.PacketJetSnapshot.

Declared hypotheses: finite dimensional multilinear packet jet. **Declared limits:** snapshot represents one fixed order, address, and base point. Suite label: nonlinear.snapshots.

Locations: nonlinear/snapshots.py:80, nonlinear/snapshots.py:246, nonlinear/snapshots.py:332. **Direct base sites:** H-C254, H-C255.

H-C254: status self.exactness; checked ('matching packet address', 'matching finite dimensions'); unchecked ('probes not supplied to this finite comparison',). Residual keys: snapshot_reconstruction. H-C255: status jet.exactness; checked ('complete ambient standard basis', 'finite multilinearity', 'fixed packet address and base point'); unchecked ('global correctness of the underlying derivative provider',). Residual keys: maximum_omitted_basis_norm.

Observed examples: EX-E-002, EX-E-037, EX-E-049. **Literal test mentions:** none recorded.

H-T294 / PSC-III-D60

Declared claim: Chunked resumable packet-jet capture; FINITE_CERTIFICATE; Block 11 runtime. Symbol: komorebi_math.nonlinear.streaming.capture_packet_jet_stream.

Declared hypotheses: finite-dimensional multilinear packet jet. **Declared limits:** integrity digests are not signatures; resuming assumes the provider behavior still agrees with its recorded identity. Suite label: nonlinear.streaming.

Locations: nonlinear/streaming.py:966, nonlinear/streaming.py:997, nonlinear/streaming.py:1131, nonlinear/streaming.py:1441, nonlinear/streaming.py:1508, nonlinear/streaming.py:1584. **Direct base sites:** H-C258, H-C259, H-C260, H-C261, H-C262.

H-C258: status manifest.exactness if completed else Exactness.CONDITIONAL; checked ('every committed chunk has an independent SHA-256 payload digest', 'manifest progress is updated only after an atomic chunk write', 'source metadata ma...; unchecked unchecked. Residual keys: remaining_basis_evaluations, incomplete_stream. H-C259: status jet.exactness; checked ('complete ambient standard basis', 'finite multilinearity', 'fixed packet address and base point', 'each committed chunk passed atomic write and digest...; unchecked ('global correctness of the underlying derivative provider',). Residual keys: maximum_omitted_basis_norm. H-C260: status self.manifest.exactness; checked ('selected chunks passed envelope and manifest digest validation', 'coefficient ordinal bounds were enforced'); unchecked () if complete_snapshot else ('coefficients outside the selected ordinal interval were not loaded',). Residual keys: missing_selected_coefficients. H-C261: status self.manifest.exactness; checked ('each contributing chunk passed digest validation', 'ordered multilinear coefficient contraction'); unchecked () if complete_snapshot else ('the returned value is only the selected coefficient partial sum',). Residual keys: unprocessed_selected_coefficients. H-C262: status Exactness.EXACT; checked ('manifest envelope digest', 'metadata digest', 'stream content digest', 'every chunk envelope digest and manifest cross-reference', 'contiguous basis a... ; unchecked ('SHA-256 integrity does not establish authenticity or authorship',). Residual keys: coefficient_count_mismatch, basis_count_mismatch, coefficient_ordering_violations.

Observed examples: EX-E-002, EX-E-048. **Literal test mentions:** none recorded.

H-T295 / PSC-III-E05

Declared claim: Exact symbolic directional derivatives; FINITE_CERTIFICATE; SymPy adapter. Symbol: komorebi_math.nonlinear.derivatives.SympyDerivativeProvider.

Declared hypotheses: SymPy expressions and substitution values are exact. Suite label: nonlinear.derivatives.

Locations: nonlinear/derivatives.py:476. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T296 / PSC-III-LIN-FP

Declared claim: Linear correction space over a prime field; `CLASSICAL`; Paper III. Symbol: `komorebi_math.nonlinear.lifting.solve_mod_prime`.

Declared hypotheses: prime modulus. Suite label: `nonlinear.lifting`.

Locations: `nonlinear/lifting.py:82`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T297 / PSC-III-N03

Declared claim: Recursive central directional differences; `FINITE_CERTIFICATE`; Numerical runtime. Symbol: `komorebi_math.nonlinear.derivatives.FiniteDifferenceDerivativeProvider`.

Declared hypotheses: sufficient smoothness near the evaluation point; finite difference scale resolves the derivative. **Declared limits:** roundoff and truncation error grow with derivative order. Suite label: `nonlinear.derivatives`.

Locations: `nonlinear/derivatives.py:241`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T298 / PSC-III-N04A

Declared claim: Directional automatic-differentiation adapter; `DEFINITION`; Paper III scalable runtime. Symbol: `komorebi_math.nonlinear.autodiff.AutomaticDifferentiationProvider`.

Declared hypotheses: declared forward-mode JVP primitive. Suite label: `nonlinear.autodiff`.

Locations: `nonlinear/autodiff.py:227`, `nonlinear/autodiff.py:354`, `nonlinear/autodiff.py:448`. **Direct base sites:** `H-C203`, `H-C204`.

`H-C203`: status `self.exactness`; checked `('finite source and target dimensions', 'permutation symmetry probes')`; unchecked `('global correctness of the supplied JVP primitive', 'framework tracing outside the tested points')`. Residual keys: `directional_symmetry`. `H-C204`: status `Exactness.NUMERICAL`; checked `('finite source and target dimensions', 'permutation symmetry probes')`; unchecked `('global correctness of the external automatic-differentiation runtime', 'framework tracing outside the tested points')`. Residual keys: `directional_symmetry`.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T299 / PSC-III-N04J

Declared claim: Lazy JAX derivative adapter; `FINITE_CERTIFICATE`; External automatic-differentiation adapter. Symbol: `komorebi_math.nonlinear.autodiff.JaxDerivativeProvider`.

Declared hypotheses: compatible JAX installation; JAX-traceable function. **Declared limits:** device and precision behavior are controlled by the external JAX runtime. Suite label: `nonlinear.autodiff`.

Locations: `nonlinear/autodiff.py:472`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T300 / PSC-III-N04T

Declared claim: Lazy PyTorch derivative adapter; `FINITE_CERTIFICATE`; External automatic-differentiation adapter. Symbol: `komorebi_math.nonlinear.autodiff.TorchDerivativeProvider`.

Declared hypotheses: compatible PyTorch installation; torch.func.jvp-compatible function. **Declared limits:** device and precision behavior are controlled by the external PyTorch runtime. Suite label: `nonlinear.autodiff`.

Locations: `nonlinear/autodiff.py:571`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T301 / PSC-III-N59

Declared claim: Sparse CuPy packet-jet execution; `FINITE_CERTIFICATE`; Block 10 runtime. Symbol: `komorebi_math.nonlinear.gpu.CuPySnapshotExecutor`.

Declared hypotheses: captured finite packet jet; working CuPy CUDA runtime. **Declared limits:** no performance claim without matched hardware benchmarking. Suite label: `nonlinear.gpu`.

Locations: `nonlinear/gpu.py:64`, `nonlinear/gpu.py:165`. **Direct base sites:** `H-C233`.

`H-C233`: status `Exactness.NUMERICAL`; checked `('matched snapshot coefficients', 'matched probe directions', 'actual CuPy CUDA execution')`; unchecked `('performance comparison and timing-scope equivalence', 'behavior outside the supplied finite probes')`. Residual keys: `maximum_cpu_gpu_residual`.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T302 / PSC-III-P20

Declared claim: Finite truncated packet equations for higher operations; `PROVED`; Paper III. Symbol: `komorebi_math.nonlinear.graded.higher_maurer_cartan_packets`.

Declared hypotheses: finite truncation; grade-respecting multilinear operations; factorials invertible. **Declared limits:** does not certify convergence of an infinite Maurer–Cartan series. Suite label: `nonlinear.graded`.

Locations: `nonlinear/graded.py:481`, `nonlinear/graded.py:540`. **Direct base sites:** `H-C239`.

`H-C239`: status `exactness`; checked `('finite higher-operation truncation', 'label convolution')`; unchecked `('operations are multilinear and grade respecting', 'convergence of any omitted infinite tail')`. Residual keys: `higher_packet_reconstruction`.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T303 / PSC-III-P21

Source symbol(s): `komorebi_math.nonlinear.graded.divided_square_packets`. No attached provenance record under this label.

Locations: `nonlinear/graded.py:608`. **Direct base sites:** `H-C240`.

`H-C240`: status `exactness`; checked `('ordered cross-label pairs', 'finite label convolution')`; unchecked `('declared divided-square polarization identity', 'grading compatibility')`. Residual keys: `divided_square_packet_reconstruction`.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T304 / PSC-III-S54

Declared claim: Spectral sequence of a finite filtered complex; CLASSICAL; Paper III. Symbol: komorebi_math.nonlinear.filtered.FiniteFilteredCochainComplex.

Declared hypotheses: bounded finite cochain complex; finite decreasing filtration by subcomplexes. Suite label: nonlinear.filtered.

Locations: nonlinear/filtered.py:611, nonlinear/filtered.py:767, nonlinear/filtered.py:809. **Direct base sites:** H-C231, H-C232.

H-C231: status Exactness.EXACT; checked ('Z_r and B_r subquotients', 'induced differential ranks'); unchecked (). Residual keys: page_cohomology_relation_failures. H-C232: status Exactness.EXACT; checked ('finite filtration by subcomplexes', 'page-to-page cohomology identities', 'limiting dimensions equal total cohomology dimensions'); unchecked (). Residual keys: convergence_dimension_mismatches, limiting_differential_rank.

Observed examples: EX-E-019. **Literal test mentions:** none recorded.

H-T305 / PSC-III-T02

Declared claim: Exact nonlinear packet coordinate theorem; PROVED; Paper III. Symbol: komorebi_math.nonlinear.coordinates.NonlinearPacketMap.certificate.

Declared hypotheses: exact packet analysis and synthesis. Suite label: nonlinear.coordinates.

Locations: nonlinear/coordinates.py:56, nonlinear/coordinates.py:72. **Direct base sites:** H-C207.

H-C207: status Exactness.EXACT if self.source.backend.exact else Exactness.NUMERICAL; checked ('source reconstruction', 'target analysis', 'function evaluation'); unchecked (). Residual keys: coordinate_conjugacy.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T306 / PSC-III-T07

Declared claim: Exact reconstruction of finite packet derivatives; PROVED; Paper III. Symbol: komorebi_math.nonlinear.jets.PacketJetFamily.

Declared hypotheses: complete finite packet systems; multilinear derivative. Suite label: nonlinear.jets.

Locations: nonlinear/jets.py:202, nonlinear/jets.py:295. **Direct base sites:** H-C244.

H-C244: status self.exactness; checked ('complete input packets', 'complete output packets', 'multilinearity'); unchecked ('global correctness of user supplied derivatives',). Residual keys: derivative_reconstruction.

Observed examples: EX-E-031. **Literal test mentions:** none recorded.

H-T307 / PSC-III-T10

Declared claim: Finite dimensional Faà di Bruno formula; CLASSICAL; Classical higher chain rule used in Paper III. Symbol: komorebi_math.nonlinear.composition.faa_di_bruno.

Declared hypotheses: declared derivatives through the requested order. Suite label: nonlinear.composition.

Locations: nonlinear/composition.py:134, nonlinear/composition.py:204. **Direct base sites:** H-C205.

H-C205: status exactness; checked ('set partition enumeration', 'higher chain rule assembly'); unchecked tuple(assumptions_unchecked). Residual keys: none recorded (computed/default when not literal).

Observed examples: EX-E-031. **Literal test mentions:** none recorded.

H-T308 / PSC-III-T11

Declared claim: Exact nonlinear defect under composition; PROVED; Paper III. Symbol: komorebi_math.nonlinear.defects.NonlinearRefinementDefect.composition_identity.

Declared hypotheses: composable fine and coarse maps. Suite label: nonlinear.defects.

Locations: nonlinear/defects.py:47, nonlinear/defects.py:78. **Direct base sites:** H-C216.

H-C216: status Exactness.NUMERICAL; checked ('pointwise map evaluation',); unchecked (). Residual keys: composition_identity.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T309 / PSC-III-T11A

Declared claim: Packet Faà di Bruno formula; PROVED; Paper III. Symbol: komorebi_math.nonlinear.composition.packet_faa_di_bruno.

Declared hypotheses: complete finite packet systems; declared higher derivatives. Suite label: nonlinear.composition.

Locations: nonlinear/composition.py:231, nonlinear/composition.py:364. **Direct base sites:** H-C206.

H-C206: status exactness; checked ('set partition enumeration', 'complete intermediate packet identity', 'input and output packet projection', 'declared zero supports used only for pruni...; unchecked ('global correctness of declared derivative providers',). Residual keys: packet_composition_reconstruction.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T310 / PSC-III-T14

Declared claim: Cokernel obstruction criterion; PROVED; Paper III. Symbol: komorebi_math.nonlinear.obstruction.LinearObstructionProblem.solve.

Declared hypotheses: finite dimensional correction problem. Suite label: nonlinear.obstruction.

Declared claim: Polynomial rooted-tree convergence certificate; PROVED; Paper III, polynomial tree expansion theorem. Symbol: komorebi_math.nonlinear.trees.majorant_convergence.

Declared hypotheses: declared multilinear norm bounds; closed Banach ball; majorant contraction. Suite label: nonlinear.trees.

Locations: nonlinear/obstruction.py:64, nonlinear/obstruction.py:99, nonlinear/obstruction.py:130, nonlinear/trees.py:327, nonlinear/trees.py:370. **Direct base sites:** H-C249, H-C250, H-C269.

H-C249: status Exactness.EXACT; checked ('exact rank comparison', 'left nullspace quotient coordinates', 'kernel basis'); unchecked (). Residual keys: correction_residual. H-C250: status Exactness.NUMERICAL; checked ('least squares correction', 'orthogonal cokernel coordinates', 'numerical rank'); unchecked (). Residual keys: correction_residual, obstruction_coordinate_norm. H-C269: status Exactness.NUMERICAL; checked ('nonnegative declared norm bounds', 'majorant maps declared ball into itself', 'majorant derivative is strictly below one'); unchecked ('declared operation bounds dominate the actual multilinear norms', 'ambient space is complete'). Residual keys: ball_image_excess, contraction_excess.

Observed examples: EX-E-032, EX-E-033. **Literal test mentions:** none recorded.

H-T311 / PSC-III-T14A

Declared claim: Finite polynomial rooted-tree expansion; FINITE_CERTIFICATE; Paper III. Symbol: komorebi_math.nonlinear.trees.PolynomialFixedPointProblem.

Declared hypotheses: finite-dimensional declared multilinear operations. Suite label: nonlinear.trees.

Locations: nonlinear/trees.py:459, nonlinear/trees.py:577. **Direct base sites:** H-C271.

H-C271: status self.exactness; checked ('ordered tree enumeration', 'recursive multilinear evaluation'); unchecked ('convergence beyond the finite depth',). Residual keys: tree_sum_minus_picard.

Observed examples: EX-E-032. **Literal test mentions:** none recorded.

H-T312 / PSC-III-T15

Source symbol(s): komorebi_math.nonlinear.obstruction.ObstructionSquare.certificate. No attached provenance record under this label.

Locations: nonlinear/obstruction.py:180. **Direct base sites:** H-C251.

H-C251: status Exactness.EXACT if algebra.exact else Exactness.NUMERICAL; checked ('linear square dimensions',); unchecked (). Residual keys: commuting_square.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T313 / PSC-III-T16

Declared claim: Packet-labeled rooted-tree reconstruction; PROVED; Paper III. Symbol: komorebi_math.nonlinear.trees.packet_tree_reconstruction.

Declared hypotheses: complete finite packet system; declared multilinear vertex maps. Suite label: nonlinear.trees.

Locations: nonlinear/trees.py:627, nonlinear/trees.py:701. **Direct base sites:** H-C272.

H-C272: status exactness; checked ('complete packet identity at every input and output slot', 'finite recursive tree evaluation', 'declared zero blocks used only for pruning'); unchecked ('global correctness of declared multilinear operations',). Residual keys: packet_tree_reconstruction.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T314 / PSC-III-T18

Declared claim: Graded packet convolution; PROVED; Paper III. Symbol: komorebi_math.nonlinear.graded.GradedPacketOperation.

Declared hypotheses: finite group grading; grade-respecting multilinear operation. Suite label: nonlinear.graded.

Locations: nonlinear/graded.py:204, nonlinear/graded.py:306, nonlinear/graded.py:362. **Direct base sites:** H-C236, H-C237.

H-C236: status exactness; checked ('packet-range bases', 'forbidden output packets', 'tested label tuples'); unchecked ('unseen label tuples due to evaluation limit',) if not complete else (). Residual keys: forbidden_packet_output. H-C237: status exactness; checked ('complete input packets', 'output label sum enumeration'); unchecked ('operation respects the grading globally',). Residual keys: graded_convolution_reconstruction.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T315 / PSC-III-T19

Declared claim: Packet Maurer–Cartan equations; PROVED; Paper III. Symbol: komorebi_math.nonlinear.graded.maurer_cartan_packets.

Declared hypotheses: grading-preserving differential; grade-additive bracket; two invertible. Suite label: nonlinear.graded.

Locations: nonlinear/graded.py:409, nonlinear/graded.py:454. **Direct base sites:** H-C238.

H-C238: status exactness; checked ('finite group label convolution', 'packet equation reconstruction at supplied components'); unchecked ('differential is linear', 'bracket is bilinear and grade respecting'). Residual keys: packet_maurer_cartan_reconstruction.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T316 / PSC-III-T28

Declared claim: One step lifting criterion; PROVED; Paper III. Symbol: komorebi_math.nonlinear.lifting.LinearLiftProblem.

Declared hypotheses: current point solves through the declared depth; linearized correction law. Suite label: nonlinear.lifting.

Locations: nonlinear/lifting.py:173, nonlinear/lifting.py:214, nonlinear/lifting.py:339. **Direct base sites:** H-C247, H-C248.

H-C247: status Exactness.EXACT; checked ('current congruence', 'associated graded correction equation'); unchecked (). Residual keys: child_verification. H-C248: status exactness; checked ('current polynomial congruence', 'all computed children') + (('Jacobian linearization over every residual correction',) if linearization_verified else ()); unchecked assumptions_unchecked. Residual keys: child_verification.

Observed examples: EX-E-033. **Literal test mentions:** none recorded.

H-T317 / PSC-III-T30

Declared claim: Finite lift tree construction; PROVED; Paper III. Symbol: komorebi_math.nonlinear.lifting.build_lift_tree.

Declared hypotheses: finite residue field; finite correction spaces. Suite label: nonlinear.lifting.

Locations: nonlinear/lifting.py:421. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T318 / PSC-III-T31

Declared claim: Orthogonal Hilbert obstruction projector; CLASSICAL; Paper III. Symbol: komorebi_math.nonlinear.correction.HilbertObstructionProjector.

Declared hypotheses: finite-dimensional Hilbert spaces; numerically resolved singular-value rank. Suite label: nonlinear.correction.

Locations: nonlinear/correction.py:57, nonlinear/correction.py:112. **Direct base sites:** H-C209.

H-C209: status Exactness.NUMERICAL; checked ('singular-value rank', 'Moore--Penrose pseudoinverse', 'orthogonal image and cokernel projectors'); unchecked ('rank stability below the singular-value threshold',). Residual keys: correction_residual, distance_identity.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T319 / PSC-III-T32

Declared claim: Right-inverse nonlinear correction; PROVED; Paper III. Symbol: komorebi_math.nonlinear.correction.RightInverseCorrectionProblem.

Declared hypotheses: declared right inverse; declared Lipschitz bound; invariant contraction ball. Suite label: nonlinear.correction.

Locations: nonlinear/correction.py:152, nonlinear/correction.py:213, nonlinear/correction.py:266. **Direct base sites:** H-C210, H-C211.

H-C210: status Exactness.NUMERICAL; checked ('D R equals identity numerically', 'declared contraction factor below one', 'declared ball invariance inequality'); unchecked ('declared Lipschitz bound holds throughout the ball',). Residual keys: right_inverse, invariant_ball_excess. H-C211: status Exactness.NUMERICAL; checked hypothesis.assumptions_checked + ('fixed-point convergence in the declared ball',); unchecked hypothesis.assumptions_unchecked. Residual keys: equation, fixed_point.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T320 / PSC-III-T33

Declared claim: Finite local Lyapunov–Schmidt reduction; CLASSICAL; Paper III. Symbol: komorebi_math.nonlinear.correction.LyapunovSchmidtReduction.

Declared hypotheses: C1 finite-dimensional map; constant resolved rank near the base point; local implicit-function neighborhood. Suite label: nonlinear.correction.

Locations: nonlinear/correction.py:326, nonlinear/correction.py:384, nonlinear/correction.py:467, nonlinear/correction.py:491. **Direct base sites:** H-C212, H-C213, H-C214.

H-C212: status Exactness.NUMERICAL; checked ('SVD kernel/range decomposition', 'invertibility on the resolved complement'); unchecked ('rank is stable in a neighborhood of the base point',). Residual keys: restricted_inverse. H-C213: status Exactness.NUMERICAL; checked ('local range equation solved numerically',); unchecked ('the selected point remains in an implicit-function neighborhood', 'local solution branch is the unique branch repre... Residual keys: range_equation. H-C214: status Exactness.NUMERICAL; checked ('kernel and cokernel SVD identities',); unchecked ('local differentiability of the selected implicit branch',). Residual keys: Dpsi_at_base, Dkappa_at_base.

Observed examples: EX-E-032. **Literal test mentions:** none recorded.

H-T321 / PSC-III-T39

Declared claim: Spanning tree normalization; PROVED; Paper III. Symbol: komorebi_math.nonlinear.holonomy.TransportGraph.tree_gauge.

Declared hypotheses: connected transport graph. Suite label: nonlinear.holonomy.

Locations: nonlinear/holonomy.py:165. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T322 / PSC-III-T40

Declared claim: Fundamental cycle holonomy criterion; PROVED; Paper III. Symbol: komorebi_math.nonlinear.holonomy.TransportGraph.fundamental_holonomies.

Declared hypotheses: connected finite graph. Suite label: nonlinear.holonomy.

Locations: nonlinear/elimination.py:445, nonlinear/elimination.py:477, nonlinear/holonomy.py:197, nonlinear/holonomy.py:234. **Direct base sites:** H-C221, H-C222, H-C241.

H-C221: status Exactness.NUMERICAL; checked ('damped Newton solution of the hidden equation',); unchecked ('the computed branch remains in the local implicit-function neighborhood', 'no other remote hidden solution is repre... Residual keys: hidden_equation. H-C222: status Exactness.NUMERICAL; checked ('hidden equation solved', 'retained equation evaluated on the branch'); unchecked solve.certificate.assumptions_unchecked. Residual keys: hidden_equation. H-C241: status Exactness.EXACT if self.backend.exact else Exactness.NUMERICAL; checked ('spanning tree normalization', 'fundamental cycle basis'); unchecked (). Residual keys: none recorded (computed/default when not literal).

Observed examples: EX-E-025. **Literal test mentions:** none recorded.

H-T323 / PSC-III-T41

Declared claim: Finite nonlinear Schur elimination; PROVED; Paper III. Symbol: komorebi_math.nonlinear.elimination.NonlinearSchurElimination.

Declared hypotheses: C2 split equation; invertible hidden derivative; local implicit-function branch. Suite label: nonlinear.elimination.

Declared claim: Fundamental holonomy intertwining criterion; PROVED; Paper III. Symbol: komorebi_math.nonlinear.holonomy.TransportGraph.intertwine.

Declared hypotheses: two transport systems on the same graph; root intertwiner. Suite label: nonlinear.holonomy.

Locations: nonlinear/elimination.py:176, nonlinear/elimination.py:280, nonlinear/holonomy.py:246, nonlinear/holonomy.py:308. **Direct base sites:** H-C219, H-C242.

H-C219: status self.exactness; checked ('hidden derivative inversion', 'first-order Schur formula'); unchecked ('existence of a local implicit-function neighborhood',). Residual keys: hidden_inverse. H-C242: status Exactness.EXACT if self.backend.exact else Exactness.NUMERICAL; checked ('common graph', 'tree propagated vertex intertwiners', 'fundamental cycle tests'); unchecked (). Residual keys: none recorded (computed/default when not literal).

Observed examples: EX-E-032. **Literal test mentions:** none recorded.

H-T324 / PSC-III-T42

Source symbol(s): `komorebi_math.nonlinear.elimination.NonlinearSchurElimination.second_derivative`. No attached provenance record under this label.

Locations: `nonlinear/elimination.py:348`. **Direct base sites:** `H-C220`.

`H-C220`: status `self.exactness`; checked `('lifted retained directions', 'second-order hidden feedback')`; unchecked `('existence and C2 regularity of the local implicit branch',)`. Residual keys: `second_schur_formula`.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T325 / PSC-III-T51

Declared claim: One-step filtered lifting criterion; `PROVED`; Paper III. Symbol: `komorebi_math.nonlinear.filtered.FilteredCorrectionProblem`.

Declared hypotheses: filtration-preserving correction map at the requested depth. Suite label: `nonlinear.filtered`.

Locations: `nonlinear/filtered.py:320`, `nonlinear/filtered.py:397`. **Direct base sites:** `H-C228`.

`H-C228`: status `Exactness.EXACT`; checked `('filtration preservation at consecutive levels', 'associated graded correction equation', 'all enumerated corrections improve the defect by one level')`; unchecked `()`. Residual keys: `child_verification`.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T326 / PSC-III-T52

Source symbol(s): `komorebi_math.nonlinear.filtered.verify_finite_filtered_branch`. No attached provenance record under this label.

Locations: `nonlinear/filtered.py:476`. **Direct base sites:** `H-C229`.

`H-C229`: status `Exactness.EXACT`; checked `('successive congruence compatibility', 'defect depth at every finite level')`; unchecked `() if terminal_reached else ('continuation beyond the verified finite prefix',)`. Residual keys: `compatibility_failures`, `defect_depth_failures`, `terminal_defect`.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T327 / PSC-III-T53

Source symbol(s): `komorebi_math.nonlinear.filtered.conditional_projective_limit_completion`. No attached provenance record under this label.

Locations: `nonlinear/filtered.py:546`. **Direct base sites:** `H-C230`.

`H-C230`: status `Exactness.CONDITIONAL`; checked `tuple((name.replace('_', ' ') for name, enabled in flags.items() if enabled))`; unchecked `unchecked`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T328 / PSC-III-T55

Declared claim: Direct rotation of finite packet systems; `PROVED`; Paper III shared runtime. Symbol: `komorebi_math.nonlinear.transport.direct_packet_rotation`.

Declared hypotheses: two complete orthogonal packet systems; invertible overlap sum. Suite label: `nonlinear.transport`.

Locations: `nonlinear/transport.py:98`, `nonlinear/transport.py:136`. **Direct base sites:** `H-C263`.

`H-C263`: status `Exactness.NUMERICAL`; checked `('complete orthogonal packets', 'invertible overlap sum')`; unchecked `()`. Residual keys: `unitarity`, `intertwining`.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T329 / PSC-III-T57

Source symbol(s): `komorebi_math.nonlinear.transport.MovingPacketSystem.generator_certificate`. No attached provenance record under this label.

Locations: `nonlinear/transport.py:289`. **Direct base sites:** `H-C265`.

`H-C265`: status `Exactness.NUMERICAL`; checked `('declared differentiable packet family',)`; unchecked `()`. Residual keys: `skew_adjoint`, `projector_commutator`.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T330 / PSC-III-T58

Declared claim: Finite smooth packet transport; `PROVED`; Paper III shared runtime. Symbol: `komorebi_math.nonlinear.transport.KatoTransport`.

Declared hypotheses: differentiable complete orthogonal packet family. Suite label: `nonlinear.transport`.

Locations: `nonlinear/transport.py:301`, `nonlinear/transport.py:356`. **Direct base sites:** `H-C266`.

`H-C266`: status `Exactness.NUMERICAL`; checked `('midpoint integration of the sampled canonical generator',)`; unchecked `('global higher-order integration error beyond residual checks',)`. Residual keys: `intertwining`, `unitarity`, `generator_failures`.

Observed examples: `EX-E-019`, `EX-E-026`. **Literal test mentions:** none recorded.

H-T331 / PSC-III-T59

Source symbol(s): `komorebi_math.nonlinear.transport.KatoTransport.direct_rotation_product`. No attached provenance record under this label.

Locations: `nonlinear/transport.py:399`. **Direct base sites:** `H-C267`.

`H-C267`: status `Exactness.NUMERICAL`; checked `('invertible overlap at every partition step',)`; unchecked `()`. Residual keys: `endpoint_intertwining`.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T332 / PSC-IV-T01

Declared claim: Cyclic coefficient projection; `PROVED`; Paper IV. Symbol: `komorebi_math.analytic.cyclic.cyclic_project_coefficients`.

Declared hypotheses: finite or convergent coefficient sequence. Suite label: `analytic.cyclic`.

Locations: `analytic/cyclic.py:27`, `analytic/cyclic.py:121`. **Direct base sites:** `H-C018`.

`H-C018`: status `Exactness.EXACT`; checked `('finite coefficient vector', 'complete residue family')`; unchecked `()`. Residual keys: `reconstruction`.

Observed examples: `EX-E-011`. **Literal test mentions:** none recorded.

H-T333 / PSC-IV-T05

Declared claim: Transfer exponential section; `PROVED`; Paper IV. Symbol: `komorebi_math.analytic.sections.transfer_section`.

Declared hypotheses: finite order; convergent entire series. Suite label: `analytic.sections`.

Locations: `analytic/sections.py:55`, `analytic/sections.py:224`. **Direct base sites:** `H-C020`.

`H-C020`: status `Exactness.NUMERICAL`; checked `('finite root system', 'entire section series')`; unchecked `()`. Residual keys: `root_sum`.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T334 / PSC-IV-T09

Declared claim: Signed cyclic section; PROVED; Paper IV. Symbol: komorebi_math.analytic.sections.signed_section.

Declared hypotheses: finite order; convergent entire series. Suite label: analytic.sections.

Locations: analytic/sections.py:87, analytic/sections.py:224. **Direct base sites:** H-C020.

H-C020: status Exactness.NUMERICAL; checked ('finite root system', 'entire section series'); unchecked (). Residual keys: root_sum.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T335 / PSC-IV-T15

Declared claim: Reciprocal power coefficients; PROVED; Paper IV. Symbol: komorebi_math.analytic.reciprocal.reciprocal_coefficients.

Declared hypotheses: finite cyclic order; declared reciprocal parameter. Suite label: analytic.reciprocal.

Locations: analytic/reciprocal.py:45. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T336 / PSC-IV-T17

Declared claim: Logarithmic reciprocal coefficients; PROVED; Paper IV. Symbol: komorebi_math.analytic.reciprocal.logarithmic_coefficients.

Declared hypotheses: finite cyclic order. Suite label: analytic.reciprocal.

Locations: analytic/reciprocal.py:26. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T337 / PSC-IV-T18

Source symbol(s): komorebi_math.analytic.reciprocal.reciprocal_semigroup_certificate. No attached provenance record under this label.

Locations: analytic/reciprocal.py:150. **Direct base sites:** H-C019.

H-C019: status Exactness.EXACT if exact and maximum == 0 else Exactness.NUMERICAL; checked ('finite reciprocal coefficient system',); unchecked (). Residual keys: maximum_coefficient.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T338 / PSC-IV-T28

Declared claim: Congruential triangular inverse; PROVED; Paper IV. Symbol: komorebi_math.analytic.appell.congruential_inverse.

Declared hypotheses: finite cyclic order; valid residue address. Suite label: analytic.appell.

Locations: analytic/appell.py:42. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T339 / PSC-IV-T54

Declared claim: Large cyclic zero strings; PROVED; Paper IV. Symbol: komorebi_math.analytic.zeros.positive_zero_string.

Declared hypotheses: cyclic order at least two; numerical Newton convergence for requested representatives. **Declared limits:** the executable routine certifies residuals, not the full asymptotic theorem. Suite label: analytic.zeros.

Locations: analytic/zeros.py:85, analytic/zeros.py:127. **Direct base sites:** H-C021.

H-C021: status Exactness.NUMERICAL; checked ('positive asymptotic seeds', 'Newton residuals', 'strict ordering'); unchecked ('exhaustion of every large zero outside the computed range',). Residual keys: maximum_zero_residual.

Observed examples: EX-E-011. **Literal test mentions:** none recorded.

H-T340 / PSC-IV-T68

Source symbol(s): komorebi_math.analytic.zeros.reduced_zero_zeta. No attached provenance record under this label.

Locations: analytic/zeros.py:205, analytic/zeros.py:224. **Direct base sites:** H-C022, H-C023.

H-C022: status Exactness.NUMERICAL; checked ('finite computed zero string',); unchecked ('zeta tail beyond the computed zeros',). Residual keys: maximum_zero_residual. H-C023: status Exactness.CONDITIONAL; checked ('Hurwitz zeta leading model',); unchecked ('entire correction term',). Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T341 / PSC-IX-D01

Declared claim: Exact prime valuation; DEFINITION; Paper IX. Symbol: komorebi_math.arithmetic.valuation.padic_valuation.

Declared hypotheses: rational input; prime valuation base. Suite label: arithmetic.valuation.

Locations: arithmetic/valuation.py:37. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T342 / PSC-IX-D02

Declared claim: Finite Hurwitz series; DEFINITION; Paper IX. Symbol: komorebi_math.arithmetic.hurwitz.HurwitzSeries.

Declared hypotheses: finite coefficient truncation. Suite label: arithmetic.hurwitz.

Locations: arithmetic/hurwitz.py:92. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T343 / PSC-IX-P18

Declared claim: Binomial carry valuation; CLASSICAL; Paper IX. Symbol: komorebi_math.arithmetic.valuation.binomial_valuation.

Declared hypotheses: prime valuation base; valid binomial indices. Suite label: arithmetic.valuation.

Locations: arithmetic/valuation.py:86. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T344 / PSC-IX-T03

Declared claim: Finite field subgroup selector; `PROVED`; Paper IX. Symbol: `komorebi_math.arithmetic.hurwitz.subgroup_selector`.

Declared hypotheses: prime field; subgroup order divides prime minus one. Suite label: `arithmetic.hurwitz`.

Locations: `arithmetic/hurwitz.py:204`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T345 / PSC-IX-T09

Declared claim: Uniform standard window period; `PROVED`; Paper IX. Symbol: `komorebi_math.arithmetic.standard_windows.StandardWindow`.

Declared hypotheses: odd prime; compression divides prime minus one; standard address. **Declared limits:** period is proved modulo the residue prime. Suite label: `arithmetic.standard_windows`.

Locations: `arithmetic/standard_windows.py:67`, `arithmetic/standard_windows.py:176`. **Direct base sites:** `H-C035`.

`H-C035`: status `Exactness.EXACT`; checked `('prime field arithmetic', 'standard address', 'compression divides prime minus one')`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: `EX-E-054`. **Literal test mentions:** none recorded.

H-T346 / PSC-IX-T19

Declared claim: Unified local Euler transfer; `PROVED`; Paper IX. Symbol: `komorebi_math.arithmetic.euler.EulerLocalCoordinates`.

Declared hypotheses: prime local chart; even Euler index. **Declared limits:** odd and dyadic integral charts remain distinct. Suite label: `arithmetic.euler`.

Locations: `arithmetic/euler.py:139`, `arithmetic/euler.py:272`, `arithmetic/euler.py:304`. **Direct base sites:** `H-C024`, `H-C025`.

`H-C024`: status `Exactness.EXACT`; checked `('prime local chart', 'even Euler index', 'finite exact arithmetic')`; unchecked `()`. Residual keys: none recorded (computed/default when not literal). `H-C025`: status `Exactness.EXACT`; checked `('prime local chart', 'finite exact coefficient range')`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: `EX-E-017`, `EX-E-027`. **Literal test mentions:** none recorded.

H-T347 / PSC-IX-T25

Source symbol(s): `komorebi_math.arithmetic.mahler.UnitModeMahler.certificate`. No attached provenance record under this label.

Locations: `arithmetic/mahler.py:356`. **Direct base sites:** `H-C031`.

`H-C031`: status `Exactness.EXACT`; checked `('nonnegative integer evaluation range',)`; unchecked `('a finite truncation is exact beyond its Mahler degree only when omitted terms vanish',) if truncation < nmax else ()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T348 / PSC-IX-T26

Source symbol(s): `komorebi_math.arithmetic.local_modes.simultaneous_unit_difference`. No attached provenance record under this label.

Locations: `arithmetic/local_modes.py:133`. **Direct base sites:** `H-C027`.

`H-C027`: status `Exactness.EXACT`; checked `('base is a unit at every declared prime', 'common Carmichael jump')`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T349 / PSC-IX-T28

Declared claim: Finite Mahler reconstruction; `PROVED`; Paper IX. Symbol: `komorebi_math.arithmetic.mahler.MahlerSeries`.

Declared hypotheses: finite consecutive value prefix. Suite label: `arithmetic.mahler`.

Locations: `arithmetic/mahler.py:64`, `arithmetic/mahler.py:161`. **Direct base sites:** `H-C028`.

`H-C028`: status `Exactness.EXACT if exact else Exactness.NUMERICAL`; checked `('finite Mahler polynomial',)`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T350 / PSC-IX-T34

Source symbol(s): `komorebi_math.arithmetic.mahler.affine_kernel_composition_certificate`. No attached provenance record under this label.

Locations: `arithmetic/mahler.py:228`. **Direct base sites:** `H-C029`.

`H-C029`: status `Exactness.EXACT`; checked `('finite affine Mahler degree',)`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T351 / PSC-IX-T36

Source symbol(s): `komorebi_math.arithmetic.mahler.prime_power_kernel_certificate`. No attached provenance record under this label.

Locations: `arithmetic/mahler.py:280`. **Direct base sites:** `H-C030`.

`H-C030`: status `Exactness.EXACT`; checked `('prime power multiplier', 'finite kernel range')`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T352 / PSC-IX-T43

Declared claim: Companion jump recurrence; `PROVED`; Paper IX. Symbol: `komorebi_math.arithmetic.recurrence.LinearRecurrence`.

Declared hypotheses: finite order linear recurrence; positive residue modulus. Suite label: `arithmetic.recurrence`.

Locations: `arithmetic/recurrence.py:110`, `arithmetic/recurrence.py:313`, `arithmetic/recurrence.py:384`. **Direct base sites:** `H-C033`, `H-C034`.

`H-C033`: status `Exactness.EXACT if exact else Exactness.NUMERICAL`; checked `('finite recurrence prefix',)`; unchecked `()`. Residual keys: none recorded (computed/default when not literal). `H-C034`: status `Exactness.EXACT if exact else Exactness.NUMERICAL`; checked `('finite residue packet',)`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: `EX-E-045`. **Literal test mentions:** none recorded.

H-T353 / PSC-IX-T47

Source symbol(s): `komorebi_math.arithmetic.recurrence.LinearRecurrence.period_mod`. No attached provenance record under this label.

Locations: `arithmetic/recurrence.py:257`. **Direct base sites:** `H-C032`.

`H-C032`: status `Exactness.EXACT`; checked `('finite modular state space',)`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: `EX-E-045`. **Literal test mentions:** none recorded.

H-T354 / PSC-LANG-AUTOMORPHIC-INDUCTION-PLAN

Source symbol(s): `komorebi_math.langlands.finite_image.plans.AutomorphicInductionPlan.operations`. No attached provenance record under this label.

Locations: `langlands/finite_image/plans.py:32`. **Direct base sites:** `H-C185`.

`H-C185`: status `Exactness.CONDITIONAL`; checked `('finite induced representation', 'declared field and character labels')`; unchecked `('the declared inducing character is algebraic when required', 'the declared field tower satisfies the external autom...`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T355 / PSC-LANG-CLIFFORD-DEPTH

Source symbol(s): `komorebi_math.langlands.finite_image.clifford.CliffordTower.report`. No attached provenance record under this label.

Locations: `langlands/finite_image/clifford.py:121`. **Direct base sites:** `H-C180`.

`H-C180`: status `Exactness.EXACT`; checked `('declared prime-cyclic order chain', 'declared Clifford degree alternative')`; unchecked `('existence of the declared representation restriction data',)`. Residual keys: none recorded (computed/default when not literal).

Observed examples: `EX-E-020`. **Literal test mentions:** none recorded.

H-T356 / PSC-LANG-MACKEY-TENSOR

Source symbol(s): `komorebi_math.langlands.finite_image.mackey.mackey_tensor_packets`. No attached provenance record under this label.

Locations: `langlands/finite_image/mackey.py:60`. **Direct base sites:** `H-C183`.

`H-C183`: status `Exactness.EXACT`; checked `('finite group', 'linear inducing characters', 'double coset representatives')`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T357 / PSC-LANG-MONOMIAL-ORBITS

Source symbol(s): `komorebi_math.langlands.finite_image.orbits.MonomialAction.packet_certificate`. No attached provenance record under this label.

Locations: `langlands/finite_image/orbits.py:157`. **Direct base sites:** `H-C184`.

`H-C184`: status `Exactness.EXACT`; checked `('finite monomial basis action', 'orbit stabilizer theorem')`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: `EX-E-020`. **Literal test mentions:** none recorded.

H-T358 / PSC-LANG-RAMIFICATION-TOWER

Source symbol(s): `komorebi_math.langlands.ramification.towers.RamificationCliffordTower.report`. No attached provenance record under this label.

Locations: `langlands/ramification/towers.py:143`. **Direct base sites:** `H-C202`.

`H-C202`: status `Exactness.CONDITIONAL`; checked `('finite ramification order chain', 'declared Clifford orbit dimensions')`; unchecked `tuple((f'canonical phase at ramification stage {index}' for index in unresolved))`. Residual keys: none recorded (computed/default when not literal).

Observed examples: `EX-E-044`. **Literal test mentions:** none recorded.

H-T359 / PSC-LANG-SOLVABLE-COMPLETION

Source symbol(s): `komorebi_math.langlands.finite_image.clifford.CliffordTower.automorphic_plan`. No attached provenance record under this label.

Locations: `langlands/finite_image/clifford.py:147`. **Direct base sites:** `H-C181`.

`H-C181`: status `Exactness.CONDITIONAL`; checked `('finite Clifford word', 'cyclic induction steps', 'rank-one extension steps')`; unchecked `tuple(unchecked)`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T360 / PSC-SEQ-AUTOMATIC

Source symbol(s): `komorebi_math.sequences.automatic.AutomaticSequence.certificate`. No attached provenance record under this label.

Locations: `sequences/automatic.py:159`. **Direct base sites:** `H-C318`.

`H-C318`: status `Exactness.EXACT`; checked `('finite deterministic transition table', 'numeric output table')`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: `EX-E-001`. **Literal test mentions:** none recorded.

H-T361 / PSC-SEQ-HOLONOMIC

Source symbol(s): `komorebi_math.sequences.holonomic.HolonomicRecurrence.certificate`. No attached provenance record under this label.

Locations: `sequences/holonomic.py:139`. **Direct base sites:** `H-C322`.

`H-C322`: status `Exactness.EXACT`; checked `('exact rational polynomial recurrence', 'nonvanishing executed leading coefficients')`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: `EX-E-001`. **Literal test mentions:** none recorded.

H-T362 / PSC-V-C37

Source symbol(s): `komorebi_math.signals.recovery.recover_jumps_subgaussian`. No attached provenance record under this label.

Locations: `signals/recovery.py:182`. **Direct base sites:** `H-C343`.

`H-C343`: status `Exactness.CONDITIONAL`; checked `result.certificate.assumptions_checked + ('declared sub Gaussian confidence calculation',)`; unchecked `('sample noises are independent centered sigma sub Gaussian variables',) + result.certificate.assumptions_unchecked[1:]`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T363 / PSC-V-D01

Declared claim: One dimensional polyphase coordinates; `DEFINITION`; Paper V. Symbol: `komorebi_math.signals.polyphase.PolyphaseCoordinates`.

Declared hypotheses: finite array; positive packet order. Suite label: `signals.polyphase`.

Locations: `signals/polyphase.py:38`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T364 / PSC-V-D03

Declared claim: Rectangular lattice packet coordinates; `DEFINITION`; Paper V. Symbol: `komorebi_math.signals.polyphase.LatticePolyphaseCoordinates`.

Declared hypotheses: finite array; positive stride in every dimension. Suite label: `signals.polyphase`.

Locations: `signals/polyphase.py:294`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T365 / PSC-V-D06

Declared claim: Finite polyphase convolution transfer matrix; `DEFINITION`; Paper V. Symbol: `komorebi_math.signals.convolution.PolyphaseConvolution`.

Declared hypotheses: finite convolution matrix; common packet order. Suite label: `signals.convolution`.

Locations: `signals/convolution.py:98`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T366 / PSC-V-D08

Declared claim: Finite periodic filter bank; `DEFINITION`; Paper V. Symbol: `komorebi_math.signals.filterbanks.PeriodicFilterBank`.

Declared hypotheses: finite periodic signal; length divisible by decimation. Suite label: `signals.filterbanks`.

Locations: `signals/filterbanks.py:38`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T367 / PSC-V-D12

Declared claim: Finite Haar packet tree; `DEFINITION`; Paper V. Symbol: `komorebi_math.signals.wavelets.HaarPacketTree`.

Declared hypotheses: finite signal length divisible by two to the tree depth. Suite label: `signals.wavelets`.

Locations: `signals/wavelets.py:52`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T368 / PSC-V-D15

Declared claim: Truncated power singular packet; `DEFINITION`; Paper V. Symbol: `komorebi_math.signals.singular_packets.TruncatedPowerTerm`.

Declared hypotheses: nonnegative derivative order. Suite label: `signals.singular_packets`.

Locations: `signals/singular_packets.py:33`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T369 / PSC-V-D16

Declared claim: Finite one dimensional singular packet; `DEFINITION`; Paper V. Symbol: `komorebi_math.signals.singular_packets.SingularPacket1D`.

Declared hypotheses: finite packet family. Suite label: `signals.singular_packets`.

Locations: `signals/singular_packets.py:108`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T370 / PSC-V-H01

Declared claim: Positive contrast two class packet estimator; `HEURISTIC`; Paper V. Symbol: `komorebi_math.signals.approximation.estimate_positive_packet_1d`.

Declared hypotheses: one dominant positive contrast region. **Declared limits:** not a universal edge detector. Suite label: `signals.approximation`.

Locations: `signals/approximation.py:78`, `signals/approximation.py:104`. **Direct base sites:** `H-C324`.

`H-C324`: status `Exactness.HEURISTIC`; checked `()`; unchecked `('two class positive contrast model', 'topological cleanup is unnecessary')`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T371 / PSC-V-T02

Declared claim: Exact finite polyphase decomposition; `PROVED`; Paper V. Symbol: `komorebi_math.signals.polyphase.polyphase_analyze`.

Declared hypotheses: finite array; positive packet order. Suite label: `signals.polyphase`.

Locations: `signals/polyphase.py:121`, `signals/polyphase.py:157`. **Direct base sites:** `H-C340`.

`H-C340`: status `Exactness.NUMERICAL`; checked `('finite residue partition', 'declared original shape')`; unchecked `()`. Residual keys: `polyphase_reconstruction`.

Observed examples: `EX-E-036`. **Literal test mentions:** none recorded.

H-T372 / PSC-V-T04

Declared claim: Exact rectangular lattice polyphase decomposition; PROVED; Paper V. Symbol: komorebi_math.signals.polyphase.lattice_polyphase_analyze.

Declared hypotheses: finite multidimensional array; positive rectangular strides. Suite label: signals.polyphase.

Locations: signals/polyphase.py:369, signals/polyphase.py:381. **Direct base sites:** H-C341.

H-C341: status Exactness.NUMERICAL; checked ('rectangular lattice residue partition',); unchecked (). Residual keys: lattice_polyphase_reconstruction.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T373 / PSC-V-T05

Declared claim: Finite convolution matrix; PROVED; Paper V. Symbol: komorebi_math.signals.convolution.convolution_matrix.

Declared hypotheses: finite one dimensional signal; declared boundary mode. Suite label: signals.convolution.

Locations: signals/convolution.py:44. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T374 / PSC-V-T07

Declared claim: Exact finite convolution transfer law; PROVED; Paper V. Symbol: komorebi_math.signals.convolution.PolyphaseConvolution.certificate.

Declared hypotheses: finite convolution; residue packet coordinates. Suite label: signals.convolution.

Locations: signals/convolution.py:182, signals/convolution.py:199. **Direct base sites:** H-C325.

H-C325: status Exactness.NUMERICAL; checked ('finite convolution matrix', 'exact residue partition'); unchecked (). Residual keys: transfer_law.

Observed examples: EX-E-036. **Literal test mentions:** none recorded.

H-T375 / PSC-V-T09

Declared claim: Finite perfect reconstruction criterion; PROVED; Paper V. Symbol: komorebi_math.signals.filterbanks.PeriodicFilterBank.perfect_reconstruction_certificate.

Declared hypotheses: finite periodic bank; matching analysis and synthesis matrices. Suite label: signals.filterbanks.

Locations: signals/filterbanks.py:114, signals/filterbanks.py:132, signals/filterbanks.py:353. **Direct base sites:** H-C332, H-C335.

H-C332: status Exactness.NUMERICAL; checked ('finite periodic convention', 'matching channel count'); unchecked (). Residual keys: reconstruction_operator. H-C335: status Exactness.NUMERICAL; checked ('finite analysis matrix', 'declared synthesis matrix'); unchecked (). Residual keys: reconstruction.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T376 / PSC-V-T10

Declared claim: Orthonormal Haar reconstruction; PROVED; Paper V. Symbol: komorebi_math.signals.filterbanks.HaarFilterBank.

Declared hypotheses: even finite signal length. Suite label: signals.filterbanks.

Locations: signals/filterbanks.py:161, signals/filterbanks.py:206. **Direct base sites:** H-C333.

H-C333: status Exactness.NUMERICAL; checked ('even finite signal',); unchecked (). Residual keys: reconstruction, energy.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T377 / PSC-V-T11

Declared claim: Exact periodic lifting update; `PROVED`; Paper V. Symbol: `komorebi_math.signals.filterbanks.PeriodicLiftingScheme`.

Declared hypotheses: even finite periodic signal. Suite label: `signals.filterbanks`.

Locations: `signals/filterbanks.py:226`, `signals/filterbanks.py:281`. **Direct base sites:** `H-C334`.

`H-C334`: status `Exactness.NUMERICAL`; checked `('periodic boundary convention', 'even finite signal')`; unchecked `()`. Residual keys: `lifting_roundtrip`.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T378 / PSC-V-T13

Declared claim: Exact reconstruction on a pruned Haar packet tree; `PROVED`; Paper V. Symbol: `komorebi_math.signals.wavelets.HaarPacketTree.reconstruct`.

Declared hypotheses: complete finite pruning; Haar local splits. Suite label: `signals.wavelets`.

Locations: `signals/wavelets.py:152`, `signals/wavelets.py:188`. **Direct base sites:** `H-C359`.

`H-C359`: status `Exactness.NUMERICAL`; checked `('complete pruning', 'finite Haar tree')`; unchecked `()`. Residual keys: `reconstruction`, `energy`.

Observed examples: `EX-E-036`. **Literal test mentions:** none recorded.

H-T379 / PSC-V-T14

Declared claim: Exact additive best basis recursion; `PROVED`; Paper V. Symbol: `komorebi_math.signals.wavelets.best_basis`.

Declared hypotheses: finite packet tree; additive node cost. Suite label: `signals.wavelets`.

Locations: `signals/wavelets.py:264`, `signals/wavelets.py:302`. **Direct base sites:** `H-C360`.

`H-C360`: status `Exactness.EXACT`; checked `('finite binary tree', 'additive local costs')`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T380 / PSC-V-T17

Declared claim: Exact retained packet reconstruction identity; `PROVED`; Paper V. Symbol: `komorebi_math.signals.singular_packets.retained_packet_reconstruct`.

Declared hypotheses: signal equals smooth component plus retained packet. Suite label: `signals.singular_packets`.

Locations: `signals/singular_packets.py:206`, `signals/singular_packets.py:242`. **Direct base sites:** `H-C348`.

`H-C348`: status `Exactness.NUMERICAL`; checked `('declared packet subtraction', 'shape preserving smooth approximator')`; unchecked `()`. Residual keys: `retained_packet_identity`.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T381 / PSC-V-T18

Source symbol(s): `komorebi_math.signals.singular_packets.retained_packet_reconstruction`. No attached provenance record under this label.

Locations: `signals/singular_packets.py:304`. **Direct base sites:** `H-C349`.

`H-C349`: status `Exactness.NUMERICAL`; checked `tuple(assumptions_checked)`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: `EX-E-052`. **Literal test mentions:** none recorded.

H-T382 / PSC-V-T19

Declared claim: Deterministic separated jump cell recovery; PROVED; Paper V. Symbol: komorebi_math.signals.edge_recovery.recover_jumps.

Declared hypotheses: one jump in every active sample cell; Lipschitz smooth component; uniform sample noise bound; minimum jump magnitude exceeds twice the background bound. **Declared limits:** does not certify unknown smoothness or unknown minimum jump size from the samples alone. Suite label: signals.edge_recovery.

Locations: signals/edge_recovery.py:54, signals/edge_recovery.py:129. **Direct base sites:** H-C329.

H-C329: status exactness; checked ('positive finite spacing', 'minimum jump exceeds twice the declared background bound' if theorem_separation else 'minimum jump separation failed', 'thr...; unchecked unchecked. Residual keys: background_bound, threshold_lower_margin, threshold_upper_margin.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T383 / PSC-V-T20

Declared claim: Sub Gaussian jump recovery bound; PROVED; Paper V. Symbol: komorebi_math.signals.edge_recovery.recover_jumps_subgaussian.

Declared hypotheses: independent centered sub Gaussian sample noise; one separated jump per active cell; known Lipschitz and minimum jump bounds. Suite label: signals.edge_recovery.

Locations: signals/edge_recovery.py:151, signals/edge_recovery.py:195. **Direct base sites:** H-C330.

H-C330: status Exactness.CONDITIONAL; checked result.certificate.assumptions_checked + (f'declared probability at least {1.0 - delta:.12g}',); unchecked ('sample noise coordinates are independent and centered', f'every coordinate obeys the declared sub Gaussian scale {s.... Residual keys: coordinate_noise_bound, simultaneous_noise_difference_bound.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T384 / PSC-V-T21

Source symbol(s): komorebi_math.signals.singular_packets.scalar_region_error. No attached provenance record under this label.

Locations: signals/singular_packets.py:438. **Direct base sites:** H-C350.

H-C350: status Exactness.NUMERICAL; checked ('finite equal shaped masks',); unchecked (). Residual keys: region_overlap_formula.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T385 / PSC-V-T22

Source symbol(s): komorebi_math.signals.singular_packets.overlap_matrix_error. No attached provenance record under this label.

Locations: signals/singular_packets.py:481. **Direct base sites:** H-C351.

H-C351: status Exactness.NUMERICAL; checked ('finite labeled partitions',); unchecked (). Residual keys: overlap_matrix_formula.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T386 / PSC-V-T36

Declared claim: Stable recovery of separated one dimensional jumps; CONDITIONAL; Paper V. Symbol: komorebi_math.signals.recovery.recover_separated_jumps.

Declared hypotheses: smooth part has the declared Lipschitz bound; sample noise has the declared uniform bound; every jump magnitude is at least the declared minimum; at most one jump lies in each sample cell. **Declared limits:** one dimensional sampled signals; separated value jumps. Suite label: signals.recovery.

Locations: signals/recovery.py:52, signals/recovery.py:117. **Direct base sites:** H-C342.

H-C342: status Exactness.CONDITIONAL; checked ('positive sample spacing', 'valid threshold gap', 'declared minimum separation is at least one cell' if minimum_separation is not None else 'no finite ...; unchecked ('unknown smooth part obeys the declared Lipschitz bound', 'unknown sample noise obeys the declared uniform bound', '.... Residual keys: none recorded (computed/default when not literal).

Observed examples: EX-E-052. **Literal test mentions:** none recorded.

H-T387 / PSC-V-T43

Source symbol(s): komorebi_math.signals.recovery.detect_piecewise_polynomial_breaks. No attached provenance record under this label.

Locations: signals/recovery.py:253. **Direct base sites:** H-C344.

H-C344: status Exactness.CONDITIONAL; checked ('finite difference order', 'threshold exceeds declared noise amplification'); unchecked ('the noiseless signal is polynomial of the declared degree on every segment', 'sample noise obeys the declared unifo.... Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T388 / PSC-V-T51

Declared claim: Cyclic product packet law; PROVED; Paper V. Symbol: komorebi_math.signals.nonlinear.packet_product.

Declared hypotheses: finite coefficient sequences or explicit residue channels. Suite label: signals.nonlinear.

Locations: signals/nonlinear.py:55, signals/nonlinear.py:112, signals/nonlinear.py:141. **Direct base sites:** H-C338, H-C339.

H-C338: status Exactness.NUMERICAL; checked ('explicit compact residue channels', 'radix carry propagation'); unchecked (). Residual keys: cyclic_channel_law. H-C339: status Exactness.NUMERICAL; checked ('finite Cauchy product',); unchecked (). Residual keys: product_reconstruction.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T389 / PSC-VI-C22

Source symbol(s): komorebi_math.dynamics.markov.FiniteMarkovChain.stationary_distribution. No attached provenance record under this label.

Locations: dynamics/markov.py:225. **Direct base sites:** H-C093.

H-C093: status Exactness.NUMERICAL; checked ('finite Markov operator',); unchecked (). Residual keys: stationarity, mass, negativity.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T390 / PSC-VI-C57

Source symbol(s): komorebi_math.dynamics.feynman_kac.FiniteFeynmanKac.coarse_model. No attached provenance record under this label.

Locations: dynamics/feynman_kac.py:154, dynamics/feynman_kac.py:173. **Direct base sites:** H-C088, H-C089.

H-C088: status Exactness.CONDITIONAL; checked ('finite partition',); unchecked ('exact Markov lumping', 'block constant potential'). Residual keys: lumping, potential_variation. H-C089: status Exactness.NUMERICAL; checked ('exact Markov lumping', 'block constant potential'); unchecked (). Residual keys: lumping, potential_variation.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T391 / PSC-VI-D01

Declared claim: Finite matrix semigroup; `DEFINITION`; Paper VI. Symbol: `komorebi_math.dynamics.semigroups.MatrixSemigroup`.

Declared hypotheses: finite square generator. Suite label: `dynamics.semigroups`.

Locations: `dynamics/semigroups.py:25`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T392 / PSC-VI-D07

Declared claim: Semigroup refinement defect; `DEFINITION`; Paper VI. Symbol: `komorebi_math.dynamics.semigroups.SemigroupRefinement`.

Declared hypotheses: finite fine and coarse generators; linear coarsening map. Suite label: `dynamics.semigroups`.

Locations: `dynamics/semigroups.py:149`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T393 / PSC-VI-D18

Declared claim: Finite Markov operator; `DEFINITION`; Paper VI. Symbol: `komorebi_math.dynamics.markov.FiniteMarkovChain`.

Declared hypotheses: column probability convention. Suite label: `dynamics.markov`.

Locations: `dynamics/markov.py:150`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T394 / PSC-VI-D27

Declared claim: Finite Feshbach decomposition; `DEFINITION`; Paper VI. Symbol: `komorebi_math.dynamics.feshbach.FeshbachReduction`.

Declared hypotheses: finite matrix; declared retained coordinates. Suite label: `dynamics.feshbach`.

Locations: `dynamics/feshbach.py:27`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T395 / PSC-VI-D33

Declared claim: Finite Bayesian update; `DEFINITION`; Paper VI. Symbol: `komorebi_math.dynamics.bayes.BayesianUpdate`.

Declared hypotheses: finite prior; nonnegative likelihood with positive evidence. Suite label: `dynamics.bayes`.

Locations: `dynamics/bayes.py:48`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T396 / PSC-VI-T01

Source symbol(s): `komorebi_math.dynamics.semigroups.MatrixSemigroup.semigroup_certificate`. No attached provenance record under this label.

Locations: `dynamics/semigroups.py:76`. **Direct base sites:** `H-C101`.

`H-C101`: status `Exactness.NUMERICAL`; checked `('finite matrix generator',)`; unchecked `()`. Residual keys: `semigroup_law`.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T397 / PSC-VI-T02

Declared claim: Packet semigroup convolution; `PROVED`; Paper VI. Symbol: `komorebi_math.dynamics.semigroups.PacketSemigroup`.

Declared hypotheses: finite complete packet family. Suite label: `dynamics.semigroups`.

Locations: `dynamics/semigroups.py:88`, `dynamics/semigroups.py:128`. **Direct base sites:** `H-C102`.

`H-C102`: status `Exactness.NUMERICAL`; checked `('complete packet family', 'finite semigroup')`; unchecked `()`. Residual keys: `packet_convolution`.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T398 / PSC-VI-T08

Source symbol(s): `komorebi_math.dynamics.semigroups.SemigroupRefinement.duhamel`. No attached provenance record under this label.

Locations: `dynamics/semigroups.py:211`. **Direct base sites:** `H-C103`.

`H-C103`: status `Exactness.NUMERICAL`; checked `('finite generators', 'Gauss Legendre quadrature')`; unchecked `()`. Residual keys: `duhamel_identity`.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T399 / PSC-VI-T10

Source symbol(s): `komorebi_math.dynamics.semigroups.SemigroupRefinement.resolvent_defect`. No attached provenance record under this label.

Locations: `dynamics/semigroups.py:240`. **Direct base sites:** `H-C104`.

`H-C104`: status `Exactness.NUMERICAL`; checked `('value lies in both resolvent sets',)`; unchecked `()`. Residual keys: `resolvent_identity`.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T400 / PSC-VI-T12

Source symbol(s): `komorebi_math.dynamics.semigroups.FiniteSemigroupTower.certificate`. No attached provenance record under this label.

Locations: `dynamics/semigroups.py:300`. **Direct base sites:** `H-C105`.

`H-C105`: status `Exactness.NUMERICAL`; checked `('finite tower dimensions',)`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T401 / PSC-VI-T17

Declared claim: Finite weighted conditional expectation; PROVED; Paper VI. Symbol: komorebi_math.dynamics.martingales.ConditionalExpectation.

Declared hypotheses: positive finite probability weights; finite partition. Suite label: dynamics.martingales.

Locations: dynamics/martingales.py:26, dynamics/martingales.py:81. **Direct base sites:** H-C095.

H-C095: status Exactness.NUMERICAL; checked ('positive normalized weights', 'finite partition'); unchecked (). Residual keys: idempotence, weighted_self_adjointness.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T402 / PSC-VI-T18

Declared claim: Finite martingale packet tower; PROVED; Paper VI. Symbol: komorebi_math.dynamics.martingales.MartingaleTower.

Declared hypotheses: nested finite partitions; common positive probability weights. Suite label: dynamics.martingales.

Locations: dynamics/martingales.py:103, dynamics/martingales.py:152, dynamics/martingales.py:172. **Direct base sites:** H-C096, H-C097.

H-C096: status Exactness.NUMERICAL; checked ('nested finite partitions', 'positive weights'); unchecked (). Residual keys: energy_decomposition. H-C097: status Exactness.NUMERICAL; checked ('nested partition validation',); unchecked (). Residual keys: tower_law.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T403 / PSC-VI-T21

Declared claim: Exact Markov lumpability criterion; PROVED; Paper VI. Symbol: komorebi_math.dynamics.markov.MarkovLumping.

Declared hypotheses: finite Markov operator; fixed state partition. Suite label: dynamics.markov.

Locations: dynamics/markov.py:283, dynamics/markov.py:381. **Direct base sites:** H-C094.

H-C094: status Exactness.NUMERICAL; checked ('column Markov convention', 'finite partition'); unchecked (). Residual keys: kernel_obstruction, coordinate_residual.

Observed examples: EX-E-030. **Literal test mentions:** none recorded.

H-T404 / PSC-VI-T24

Declared claim: Finite spectral fractional operator; PROVED; Paper VI. Symbol: komorebi_math.dynamics.fractional.fractional_power.

Declared hypotheses: finite positive semidefinite Hermitian operator; positive exponent. Suite label: dynamics.fractional.

Locations: dynamics/fractional.py:30. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T405 / PSC-VI-T25

Declared claim: Exact finite fractional naturality; PROVED; Paper VI. Symbol: komorebi_math.dynamics.fractional.fractional_naturality_certificate.

Declared hypotheses: positive self adjoint fine and coarse operators; exact intertwining. Suite label: dynamics.fractional.

Locations: dynamics/fractional.py:70, dynamics/fractional.py:102. **Direct base sites:** H-C090.

H-C090: status Exactness.NUMERICAL; checked ('finite positive Hermitian operators',); unchecked (). Residual keys: generator_intertwining, fractional_intertwining.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T406 / PSC-VI-T26

Declared claim: Finite fractional defect bound; PROVED; Paper VI. Symbol: komorebi_math.dynamics.fractional.FractionalDefectResult.

Declared hypotheses: exponent strictly between zero and one; common positive spectral gap. Suite label: dynamics.fractional.

Locations: dynamics/fractional.py:114, dynamics/fractional.py:166. **Direct base sites:** H-C091.

H-C091: status Exactness.NUMERICAL; checked ('positive Hermitian operators', 'strict positive common gap', 'zero to one exponent'); unchecked (). Residual keys: actual_defect, theorem_bound, bound_excess.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T407 / PSC-VI-T28

Declared claim: Finite Schur Feshbach resolvent; PROVED; Paper VI. Symbol: komorebi_math.dynamics.feshbach.FeshbachReduction.resolvent_certificate.

Declared hypotheses: full and hidden resolvents exist. Suite label: dynamics.feshbach.

Locations: dynamics/feshbach.py:114, dynamics/feshbach.py:136. **Direct base sites:** H-C086.

H-C086: status Exactness.NUMERICAL; checked ('hidden resolvent exists', 'full resolvent exists'); unchecked (). Residual keys: feshbach_resolvent.

Observed examples: EX-E-018, EX-E-026. **Literal test mentions:** none recorded.

H-T408 / PSC-VI-T29

Declared claim: Finite exact local closure criterion; PROVED; Paper VI. Symbol: komorebi_math.dynamics.feshbach.FeshbachReduction.local_closure_certificate.

Declared hypotheses: finite hidden state space. Suite label: dynamics.feshbach.

Locations: dynamics/feshbach.py:172, dynamics/feshbach.py:185. **Direct base sites:** H-C087.

H-C087: status Exactness.NUMERICAL; checked ('finite hidden dimension', 'Cayley Hamilton moment test'); unchecked (). Residual keys: none recorded (computed/default when not literal).

Observed examples: EX-E-018. **Literal test mentions:** none recorded.

H-T409 / PSC-VI-T30

Declared claim: Finite rational memory realization; PROVED; Paper VI. Symbol: komorebi_math.dynamics.memory.MemoryKernel.

Declared hypotheses: finite auxiliary state realization. Suite label: dynamics.memory.

Locations: dynamics/memory.py:32, dynamics/memory.py:118. **Direct base sites:** H-C098.

H-C098: status Exactness.NUMERICAL; checked ('finite auxiliary realization', 'hidden resolvent exists'); unchecked (). Residual keys: laplace_decomposition.

Observed examples: EX-E-018. **Literal test mentions:** none recorded.

H-T410 / PSC-VI-T31

Declared claim: Finite horizon Volterra kernel stability bound; PROVED; Paper VI. Symbol: komorebi_math.dynamics.memory.volterra_trajectory_bound.

Declared hypotheses: finite horizon; declared trajectory and kernel bounds. Suite label: dynamics.memory.

Locations: dynamics/memory.py:241, dynamics/memory.py:261, dynamics/memory.py:275. **Direct base sites:** H-C099, H-C100.

H-C099: status Exactness.NUMERICAL; checked ('finite horizon', 'finite rational memory kernels', 'operator norm exponential and derivative bounds'); unchecked (). Residual keys: quadrature_estimate, quadrature_allowance, separate_exponential_bound. H-C100: status Exactness.CONDITIONAL; checked ('finite horizon', 'trapezoidal operator norm quadrature'); unchecked ('quadrature estimate upper bounds the true kernel integral',). Residual keys: quadrature_estimate.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T411 / PSC-VI-T32

Declared claim: Finite state dependent killing survival model; PROVED; Paper VI. Symbol: komorebi_math.dynamics.survival.PhaseTypeSurvival.

Declared hypotheses: finite transient column subgenerator. Suite label: dynamics.survival.

Locations: dynamics/survival.py:30, dynamics/survival.py:101. **Direct base sites:** H-C106.

H-C106: status Exactness.NUMERICAL; checked ('column transient convention', 'nonnegative off diagonal rates', 'strictly stable transient generator'); unchecked (). Residual keys: negative_off_diagonal, negative_absorption_rate, cause_rate_sum, spectral_abscissa_failure.

Observed examples: EX-E-022. **Literal test mentions:** none recorded.

H-T412 / PSC-VI-T33

Source symbol(s): komorebi_math.dynamics.survival.PhaseTypeSurvival.identities_certificate. No attached provenance record under this label.

Locations: dynamics/survival.py:182. **Direct base sites:** H-C107.

H-C107: status Exactness.NUMERICAL; checked ('finite difference derivative check', 'valid survival model'); unchecked (). Residual keys: density_derivative, cause_sum.

Observed examples: EX-E-022. **Literal test mentions:** none recorded.

H-T413 / PSC-VI-T36

Declared claim: Exact two stage Bayesian coarsening; PROVED; Paper VI. Symbol: komorebi_math.dynamics.bayes.BayesianPartition.coarse_posterior.

Declared hypotheses: finite partition; positive evidence. Suite label: dynamics.bayes.

Locations: dynamics/bayes.py:131, dynamics/bayes.py:152. **Direct base sites:** H-C082.

H-C082: status Exactness.NUMERICAL; checked ('finite partition', 'positive evidence'); unchecked (). Residual keys: two_stage_update.

Observed examples: EX-E-030. **Literal test mentions:** none recorded.

H-T414 / PSC-VI-T41

Source symbol(s): komorebi_math.dynamics.bayes.relative_entropy_decomposition. No attached provenance record under this label.

Locations: dynamics/bayes.py:313. **Direct base sites:** H-C083.

H-C083: status Exactness.NUMERICAL; checked ('finite probability vectors', 'fixed partition'); unchecked (). Residual keys: relative_entropy_decomposition.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T415 / PSC-VI-T46

Source symbol(s): `komorebi_math.dynamics.bayes.HiddenMarkovFilter.coarse_step`. No attached provenance record under this label.

Locations: `dynamics/bayes.py:404`, `dynamics/bayes.py:427`. **Direct base sites:** `H-C084`, `H-C085`.

`H-C084`: status `Exactness.CONDITIONAL`; checked `('finite filter step',)`; unchecked `tuple((condition for condition, satisfied in (('transition is exactly lumpable', exact_markov), ('likelihood is block...` Residual keys: none recorded (computed/default when not literal). `H-C085`: status `Exactness.NUMERICAL`; checked `('exact Markov lumping', 'block constant likelihood')`; unchecked `()`. Residual keys: `coarse_filter`.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T416 / PSC-VI-T49

Declared claim: Exact discrete Dirichlet Green reconstruction; `PROVED`; Paper VI. Symbol: `komorebi_math.dynamics.green.DirichletGreen1D`.

Declared hypotheses: uniform one dimensional Dirichlet grid. Suite label: `dynamics.green`.

Locations: `dynamics/green.py:17`, `dynamics/green.py:77`. **Direct base sites:** `H-C092`.

`H-C092`: status `Exactness.NUMERICAL`; checked `('uniform Dirichlet grid',)`; unchecked `()`. Residual keys: `inverse_identity`.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T417 / PSC-VI-T55

Declared claim: Finite state Feynman Kac transfer; `PROVED`; Paper VI. Symbol: `komorebi_math.dynamics.feynman_kac.FiniteFeynmanKac`.

Declared hypotheses: finite column Markov model; nonnegative killing potential. Suite label: `dynamics.feynman_kac`.

Locations: `dynamics/feynman_kac.py:41`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T418 / PSC-VII-C03

Declared claim: Local orbit and Schmidt rank; `PROVED`; Paper VII. Symbol: `komorebi_math.quantum.seeds.local_orbit_dimension`.

Declared hypotheses: finite bipartite pure state. Suite label: `quantum.seeds`.

Locations: `quantum/seeds.py:67`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T419 / PSC-VII-C17

Source symbol(s): `komorebi_math.quantum.circuits.CircuitPacketExpansion.interference`. No attached provenance record under this label.

Locations: `quantum/circuits.py:161`. **Direct base sites:** `H-C278`.

`H-C278`: status `Exactness.NUMERICAL`; checked `('source packet projection', 'coherent path amplitudes')`; unchecked `()`. Residual keys: `interference_identity`.

Observed examples: `EX-E-042`. **Literal test mentions:** none recorded.

H-T420 / PSC-VII-D59

Declared claim: Finite spectral dimension diagnostic; `DEFINITION`; Paper VII. Symbol: `komorebi_math.quantum.spectral.spectral_dimension`.

Declared hypotheses: finite nonnegative Hermitian operator; positive diffusion time. **Declared limits:** finite dimension forces zero limits at both time endpoints. Suite label: `quantum.spectral`.

Current-source correction. The large-time statement in the historical declaration above is too broad. The limit is zero when the nonnegative operator has a zero eigenvalue. For a positive-definite operator, the diagnostic instead grows asymptotically linearly with time. The zero-time limit remains zero. Section 53.3 gives the corrected interpretation, consistent with the current source at commit `5f2b7ddc879c12d2bb00cbe5c487aa0bc7709222`.

Locations: `quantum/spectral.py:61`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T421 / PSC-VII-P49

Source symbol(s): `komorebi_math.quantum.effective.QuantumFeshbachReduction.upper_half_plane_sign_certificate`. No attached provenance record under this label.

Locations: `quantum/effective.py:64`. **Direct base sites:** `H-C281`.

`H-C281`: status `Exactness.NUMERICAL`; checked `('Hermitian hidden Hamiltonian', 'upper half plane spectral parameter')`; unchecked `()`. Residual keys: `positive_imaginary_part`.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T422 / PSC-VII-T01

Declared claim: Finite quantum seed orbit; `PROVED`; Paper VII. Symbol: `komorebi_math.quantum.seeds.QuantumSeedSystem`.

Declared hypotheses: finite observable family; finite state vector. Suite label: `quantum.seeds`.

Locations: `quantum/seeds.py:18`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T423 / PSC-VII-T04

Source symbol(s): `komorebi_math.quantum.seeds.computational_basis_generation`. No attached provenance record under this label.

Locations: `quantum/seeds.py:150`. **Direct base sites:** `H-C291`.

`H-C291`: status `Exactness.NUMERICAL`; checked `('Pauli X address family',)`; unchecked `()`. Residual keys: `basis_orthogonality`.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T424 / PSC-VII-T08

Declared claim: Finite group isotypic projector; `CLASSICAL`; Paper VII. Symbol: `komorebi_math.quantum.symmetry.finite_group_projector`.

Declared hypotheses: finite unitary representation; irreducible character values. Suite label: `quantum.symmetry`.

Locations: `quantum/symmetry.py:20`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T425 / PSC-VII-T09

Declared claim: Cyclic character projectors; `PROVED`; Paper VII. Symbol: `komorebi_math.quantum.symmetry.cyclic_symmetry_projectors`.

Declared hypotheses: unitary symmetry of finite order. Suite label: `quantum.symmetry`.

Locations: `quantum/symmetry.py:49`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T426 / PSC-VII-T11

Source symbol(s): `komorebi_math.quantum.symmetry.charge_certificate`, `komorebi_math.quantum.symmetry.charge_selection_certificate`. No attached provenance record under this label.

Locations: `quantum/symmetry.py:106`, `quantum/symmetry.py:137`. **Direct base sites:** `H-C292`, `H-C293`.

`H-C292`: status `Exactness.NUMERICAL`; checked `('unitary cyclic symmetry',)`; unchecked `()`. Residual keys: `charge_relation`. `H-C293`: status `Exactness.NUMERICAL`; checked `('complete cyclic packet family',)`; unchecked `()`. Residual keys: `forbidden_block`.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T427 / PSC-VII-T14

Declared claim: Circuit packet path expansion; `PROVED`; Paper VII. Symbol: `komorebi_math.quantum.circuits.CircuitPacketExpansion`.

Declared hypotheses: finite unitary circuit; complete packet system at every circuit depth. Suite label: `quantum.circuits`.

Locations: `quantum/circuits.py:38`, `quantum/circuits.py:129`. **Direct base sites:** `H-C277`.

`H-C277`: status `Exactness.NUMERICAL`; checked `('complete packet resolutions', 'unitary circuit gates')`; unchecked `()`. Residual keys: `circuit_reconstruction`.

Observed examples: `EX-E-042`. **Literal test mentions:** none recorded.

H-T428 / PSC-VII-T15

Source symbol(s): `komorebi_math.quantum.circuits.CircuitPacketExpansion.block_unitarity_certificate`. No attached provenance record under this label.

Locations: `quantum/circuits.py:188`. **Direct base sites:** `H-C279`.

`H-C279`: status `Exactness.NUMERICAL`; checked `('unitary circuit', 'complete source and target packets')`; unchecked `()`. Residual keys: `column_unitarity`.

Observed examples: `EX-E-042`. **Literal test mentions:** none recorded.

H-T429 / PSC-VII-T18

Declared claim: Fixed packet leakage bound; PROVED; Paper VII. Symbol: komorebi_math.quantum.leakage.fixed_packet_leakage.

Declared hypotheses: finite Hermitian Hamiltonian; orthogonal packet projector. Suite label: quantum.leakage.

Locations: quantum/leakage.py:29, quantum/leakage.py:63. **Direct base sites:** H-C287.

H-C287: status Exactness.NUMERICAL; checked ('Hermitian Hamiltonian', 'orthogonal packet projector'); unchecked (). Residual keys: bound_violation.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T430 / PSC-VII-T22

Source symbol(s): komorebi_math.quantum.leakage.moving_packet_leakage_bound. No attached provenance record under this label.

Locations: quantum/leakage.py:124. **Direct base sites:** H-C288.

H-C288: status Exactness.NUMERICAL; checked ('sampled moving geometric coupling',); unchecked ('quadrature error between samples',). Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T431 / PSC-VII-T31

Declared claim: Finite Kraus representation; CLASSICAL; Paper VII. Symbol: komorebi_math.quantum.channels.QuantumChannel.

Declared hypotheses: finite Kraus family. Suite label: quantum.channels.

Locations: quantum/channels.py:62, quantum/channels.py:138. **Direct base sites:** H-C275.

H-C275: status Exactness.NUMERICAL; checked ('finite Kraus representation',); unchecked (). Residual keys: trace_preservation, negative_choi_part.

Observed examples: EX-E-041. **Literal test mentions:** none recorded.

H-T432 / PSC-VII-T35

Source symbol(s): komorebi_math.quantum.channels.QuantumChannel.covariance_certificate. No attached provenance record under this label.

Locations: quantum/channels.py:240. **Direct base sites:** H-C276.

H-C276: status Exactness.NUMERICAL; checked ('unitary input and output symmetries',); unchecked (). Residual keys: covariance.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T433 / PSC-VII-T37

Source symbol(s): komorebi_math.quantum.lindblad.LindbladGenerator.trace_preservation_certificate. No attached provenance record under this label.

Locations: quantum/lindblad.py:84. **Direct base sites:** H-C289.

H-C289: status Exactness.NUMERICAL; checked ('finite Lindblad form',); unchecked (). Residual keys: trace_derivative.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T434 / PSC-VII-T38

Declared claim: Lindblad coherence packet preservation; `PROVED`; Paper VII. Symbol: `komorebi_math.quantum.lindblad.LindbladGenerator`.

Declared hypotheses: finite Lindblad generator; cyclic Hamiltonian symmetry; charged jump operators. Suite label: `quantum.lindblad`.

Locations: `quantum/lindblad.py:17`, `quantum/lindblad.py:110`. **Direct base sites:** `H-C290`.

`H-C290`: status `Exactness.NUMERICAL`; checked `('finite cyclic symmetry',)`; unchecked `()`. Residual keys: `coherence_commutator`.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T435 / PSC-VII-T42

Declared claim: Knill Laflamme compression criterion; `CLASSICAL`; Paper VII. Symbol: `komorebi_math.quantum.error_correction.QuantumCode`.

Declared hypotheses: finite orthogonal code projector; finite error family. Suite label: `quantum.error_correction`.

Locations: `quantum/error_correction.py:28`, `quantum/error_correction.py:74`. **Direct base sites:** `H-C283`.

`H-C283`: status `Exactness.NUMERICAL`; checked `('orthogonal code projector', 'finite error family')`; unchecked `()`. Residual keys: `compression_obstruction`.

Observed examples: `EX-E-041`. **Literal test mentions:** none recorded.

H-T436 / PSC-VII-T44

Source symbol(s): `komorebi_math.quantum.error_correction.QuantumCode.recovered_channel_certificate`. No attached provenance record under this label.

Locations: `quantum/error_correction.py:123`, `quantum/error_correction.py:141`. **Direct base sites:** `H-C284`, `H-C285`.

`H-C284`: status `Exactness.NUMERICAL`; checked `('Knill Laflamme compression',)`; unchecked `('equal weight error family is trace preserving',)`. Residual keys: `noise_trace_preservation`. `H-C285`: status `Exactness.NUMERICAL`; checked `('Knill Laflamme compression', 'equal weight error channel')`; unchecked `()`. Residual keys: `recovered_code_matrix_units`.

Observed examples: `EX-E-041`. **Literal test mentions:** none recorded.

H-T437 / PSC-VII-T47

Declared claim: Quantum Feshbach reduction; `PROVED`; Paper VII. Symbol: `komorebi_math.quantum.effective.QuantumFeshbachReduction`.

Declared hypotheses: finite Hermitian Hamiltonian; declared retained sector. Suite label: `quantum.effective`.

Locations: `quantum/effective.py:17`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T438 / PSC-VII-T53

Source symbol(s): `komorebi_math.quantum.effective.QuantumFeshbachReduction.bare_closure_certificate`. No attached provenance record under this label.

Locations: `quantum/effective.py:87`. **Direct base sites:** `H-C282`.

`H-C282`: status `Exactness.NUMERICAL`; checked `('finite Hermitian block decomposition',)`; unchecked `()`. Residual keys: `retained_hidden_coupling`.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T439 / PSC-VII-T56

Declared claim: Finite reflection positive transfer; PROVED; Paper VII. Symbol: komorebi_math.quantum.euclidean.EuclideanTransfer.

Declared hypotheses: finite nonnegative Hamiltonian after ground shift. Suite label: quantum.euclidean.

Locations: quantum/euclidean.py:18, quantum/euclidean.py:98. **Direct base sites:** H-C286.

H-C286: status Exactness.NUMERICAL; checked ('positive Euclidean transfer operator',); unchecked (). Residual keys: negative_eigenvalue.

Observed examples: EX-E-013. **Literal test mentions:** none recorded.

H-T440 / PSC-VII-T64

Declared claim: Constraint quotient obstruction; PROVED; Paper VII. Symbol: komorebi_math.quantum.constraints.constraint_closure.

Declared hypotheses: finite declared constraint span. Suite label: quantum.constraints.

Locations: quantum/constraints.py:27, quantum/constraints.py:67. **Direct base sites:** H-C280.

H-C280: status Exactness.NUMERICAL; checked ('finite complex linear constraint span',); unchecked (). Residual keys: closure_obstruction.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T441 / PSC-VII-T70

Source symbol(s): komorebi_math.quantum.towers.BoundedHamiltonianTower.report. No attached provenance record under this label.

Locations: quantum/towers.py:61. **Direct base sites:** H-C294.

H-C294: status Exactness.NUMERICAL; checked ('finite bounded Hermitian tower',); unchecked ('infinite tail summability beyond supplied levels',). Residual keys: unitary_bound_violation.

Observed examples: EX-E-013. **Literal test mentions:** none recorded.

H-T442 / PSC-VIII-C01

Source symbol(s): komorebi_math.algorithms.gpu.NumpyPacketExecutor.apply. No attached provenance record under this label.

Locations: algorithms/gpu.py:41. **Direct base sites:** H-C005.

H-C005: status Exactness.NUMERICAL; checked ('NumPy packet execution',); unchecked (). Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T443 / PSC-VIII-C02

Source symbol(s): komorebi_math.algorithms.gpu.CupyPacketExecutor.apply. No attached provenance record under this label.

Locations: algorithms/gpu.py:99. **Direct base sites:** H-C006.

H-C006: status Exactness.NUMERICAL; checked ('CuPy installed', 'packet blocks transferred to device'); unchecked ('device execution has not been synchronized for timing',). Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T444 / PSC-VIII-D01

Declared claim: Finite packet layout; `DEFINITION`; Paper VIII. Symbol: `komorebi_math.algorithms.layout.PacketLayout`.

Declared hypotheses: positive finite packet sizes. Suite label: `algorithms.layout`.

Locations: `algorithms/layout.py:16`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T445 / PSC-VIII-D02

Declared claim: Dense operator contract; `DEFINITION`; Paper VIII. Symbol: `komorebi_math.algorithms.structured.DenseOperator`.

Declared hypotheses: finite matrix. Suite label: `algorithms.structured`.

Locations: `algorithms/structured.py:78`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T446 / PSC-VIII-H01

Declared claim: Threshold packet support discovery; `HEURISTIC`; Paper VIII. Symbol: `komorebi_math.algorithms.discovery.discover_packet_support`.

Declared hypotheses: fixed packet layout; declared norm threshold. **Declared limits:** a thresholded zero is not an exact algebraic zero. Suite label: `algorithms.discovery`.

Locations: `algorithms/discovery.py:30`, `algorithms/discovery.py:81`. **Direct base sites:** `H-C003`.

`H-C003`: status `Exactness.HEURISTIC`; checked `('fixed packet layout', 'declared threshold')`; unchecked `('discarded blocks are mathematically zero or negligible for the application',)`. Residual keys: `discarded_spectral_norm`.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T447 / PSC-VIII-T01

Declared claim: Exact packet operator intermediate representation; `PROVED`; Paper VIII. Symbol: `komorebi_math.algorithms.ir.PacketIR`.

Declared hypotheses: finite source and target packet layouts; declared block dimensions. Suite label: `algorithms.ir`.

Locations: `algorithms/ir.py:22`, `algorithms/ir.py:259`. **Direct base sites:** `H-C007`.

`H-C007`: status `Exactness.NUMERICAL`; checked `('block dimensions', 'packet address ranges')`; unchecked `()`. Residual keys: `apply_materialize`.

Observed examples: `EX-E-035`. **Literal test mentions:** none recorded.

H-T448 / PSC-VIII-T03

Declared claim: Block diagonal packet operator; `PROVED`; Paper VIII. Symbol: `komorebi_math.algorithms.structured.BlockDiagonalOperator`.

Declared hypotheses: finite square packet blocks. Suite label: `algorithms.structured`.

Locations: `algorithms/structured.py:219`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T449 / PSC-VIII-T04

Declared claim: Packet apply arithmetic count; `PROVED`; Paper VIII. Symbol: `komorebi_math.algorithms.complexity.packet_apply_flops`.

Declared hypotheses: dense active packet blocks; classical matrix multiplication count. Suite label: `algorithms.complexity`.

Locations: `algorithms/complexity.py:46`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T450 / PSC-VIII-T07

Declared claim: Kronecker modewise application; `PROVED`; Paper VIII. Symbol: `komorebi_math.algorithms.structured.KroneckerOperator`.

Declared hypotheses: two finite matrix factors. Suite label: `algorithms.structured`.

Locations: `algorithms/structured.py:412`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T451 / PSC-VIII-T10

Declared claim: Block diagonal plus low rank algebra; `PROVED`; Paper VIII. Symbol: `komorebi_math.algorithms.low_rank.BlockDiagonalLowRankOperator`.

Declared hypotheses: finite square block diagonal part; finite low rank correction. Suite label: `algorithms.low_rank`.

Locations: `algorithms/low_rank.py:214`. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T452 / PSC-VIII-T12

Declared claim: Optimal small core recompression; `PROVED`; Paper VIII. Symbol: `komorebi_math.algorithms.low_rank.recompress_core`.

Declared hypotheses: finite factor matrices; rank constrained spectral or Frobenius approximation. Suite label: `algorithms.low_rank`.

Locations: `algorithms/low_rank.py:98`, `algorithms/low_rank.py:137`, `algorithms/low_rank.py:166`. **Direct base sites:** `H-C014`, `H-C015`.

`H-C014`: status `Exactness.NUMERICAL`; checked `('empty low rank correction',)`; unchecked `()`. Residual keys: `spectral_error`, `frobenius_error`. `H-C015`: status `Exactness.NUMERICAL`; checked `('reduced QR', 'small core singular value decomposition')`; unchecked `()`. Residual keys: `actual_spectral_error`, `certified_spectral_error`, `certified_frobenius_error`.

Observed examples: `EX-E-028`. **Literal test mentions:** none recorded.

H-T453 / PSC-VIII-T13

Declared claim: Optimal fixed layout block diagonal plus low rank approximation; PROVED; Paper VIII. Symbol: komorebi_math.algorithms.discovery.discover_block_diagonal_low_rank.

Declared hypotheses: fixed packet layout; rank constrained approximation of the off diagonal residual. Suite label: algorithms.discovery.

Locations: algorithms/discovery.py:107, algorithms/discovery.py:149. **Direct base sites:** H-C004.

H-C004: status Exactness.NUMERICAL; checked ('fixed packet diagonal projection', 'singular value truncation of residual'); unchecked (). Residual keys: actual_spectral_error, optimal_spectral_error, optimal_frobenius_error.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T454 / PSC-VIII-T15

Declared claim: Packet support multiplication; PROVED; Paper VIII. Symbol: komorebi_math.algorithms.support.support_product.

Declared hypotheses: finite packet support relations. Suite label: algorithms.support.

Locations: algorithms/support.py:48. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T455 / PSC-VIII-T18

Declared claim: Schur complement fill support; PROVED; Paper VIII. Symbol: komorebi_math.algorithms.support.schur_fill_support.

Declared hypotheses: finite retained and hidden packet partition. **Declared limits:** support is an inclusion because algebraic cancellation may remove permitted blocks. Suite label: algorithms.support.

Locations: algorithms/support.py:146. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T456 / PSC-VIII-T20

Declared claim: Finite Krylov packet decomposition; PROVED; Paper VIII. Symbol: komorebi_math.algorithms.krylov.krylov_basis.

Declared hypotheses: finite linear operator; declared start vector. Suite label: algorithms.krylov.

Locations: algorithms/krylov.py:32, algorithms/krylov.py:75. **Direct base sites:** H-C008.

H-C008: status Exactness.NUMERICAL; checked ('finite dimension', 'modified Gram Schmidt'); unchecked (). Residual keys: orthogonality.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T457 / PSC-VIII-T22

Declared claim: Preconditioned conjugate gradient certificate; CLASSICAL; Paper VIII. Symbol: komorebi_math.algorithms.krylov.conjugate_gradient.

Declared hypotheses: Hermitian positive definite operator; positive definite preconditioner. Suite label: algorithms.krylov.

Locations: algorithms/krylov.py:100, algorithms/krylov.py:183, algorithms/krylov.py:259. **Direct base sites:** H-C009, H-C010.

H-C009: status Exactness.NUMERICAL; checked ('initial residual tolerance',); unchecked (). Residual keys: final_residual. H-C010: status Exactness.NUMERICAL; checked ('positive curvature on generated directions',); unchecked ('global Hermitian positive definiteness',). Residual keys: final_residual, recursive_final_residual.

Observed examples: EX-E-055. **Literal test mentions:** none recorded.

H-T458 / PSC-VIII-T23

Source symbol(s): komorebi_math.algorithms.krylov.lanczos_tridiagonal. No attached provenance record under this label.

Locations: algorithms/krylov.py:362. **Direct base sites:** H-C011.

H-C011: status Exactness.NUMERICAL; checked ('Hermitian behavior on generated basis', 'full reorthogonalization'); unchecked ('global Hermitian operator',). Residual keys: orthogonality, projected_tridiagonal.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T459 / PSC-VIII-T24

Declared claim: Packet block Jacobi condition bound; PROVED; Paper VIII. Symbol: komorebi_math.algorithms.krylov.block_jacobi_certificate.

Declared hypotheses: Hermitian positive definite packet diagonal; normalized coupling norm below one. Suite label: algorithms.krylov.

Locations: algorithms/krylov.py:407, algorithms/krylov.py:456. **Direct base sites:** H-C012.

H-C012: status Exactness.NUMERICAL; checked ('Hermitian matrix', 'positive definite packet diagonal'); unchecked () if eta < 1 else ('normalized coupling norm below one',). Residual keys: hermitian, normalized_coupling, actual_preconditioned_condition.

Observed examples: EX-E-055. **Literal test mentions:** none recorded.

H-T460 / PSC-VIII-T25

Source symbol(s): komorebi_math.algorithms.precision.precision_certificate. No attached provenance record under this label.

Locations: algorithms/precision.py:269. **Direct base sites:** H-C017.

H-C017: status Exactness.NUMERICAL; checked ('packetwise operator perturbation norms', 'source packet norms'); unchecked (). Residual keys: observed_error, certified_bound.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T461 / PSC-VIII-T26

Declared claim: Continuous packet precision allocation; PROVED; Paper VIII. Symbol: komorebi_math.algorithms.precision.optimal_precision_tolerances.

Declared hypotheses: positive sensitivities; positive logarithmic costs; positive error budget. Suite label: algorithms.precision.

Locations: algorithms/precision.py:37. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T462 / PSC-VIII-T28

Declared claim: Exact discrete precision assignment; PROVED; Paper VIII. Symbol: komorebi_math.algorithms.precision.assign_discrete_precisions.

Declared hypotheses: finite option list for every packet; integer error budget. Suite label: algorithms.precision.

Locations: algorithms/precision.py:158, algorithms/precision.py:201. **Direct base sites:** H-C016.

H-C016: status Exactness.EXACT; checked ('finite option lists', 'integer additive error model'); unchecked (). Residual keys: none recorded (computed/default when not literal).

Observed examples: EX-E-038. **Literal test mentions:** none recorded.

H-T463 / PSC-VIII-T30

Declared claim: Exact additive tree compiler; PROVED; Paper VIII. Symbol: komorebi_math.algorithms.compiler.compile_tree_lagrangian.

Declared hypotheses: finite rooted tree; additive keep and split models. Suite label: algorithms.compiler.

Locations: algorithms/compiler.py:76, algorithms/compiler.py:119. **Direct base sites:** H-C001.

H-C001: status Exactness.EXACT; checked ('finite tree', 'additive Lagrangian objective'); unchecked (). Residual keys: none recorded (computed/default when not literal).

Observed examples: EX-E-034. **Literal test mentions:** none recorded.

H-T464 / PSC-VIII-T33

Declared claim: Exact quantized budget compiler; PROVED; Paper VIII. Symbol: komorebi_math.algorithms.compiler.compile_tree_budget.

Declared hypotheses: finite rooted tree; integer quantized additive error budget. Suite label: algorithms.compiler.

Locations: algorithms/compiler.py:131, algorithms/compiler.py:231. **Direct base sites:** H-C002.

H-C002: status Exactness.EXACT; checked ('finite tree', 'quantized additive error model'); unchecked (). Residual keys: none recorded (computed/default when not literal).

Observed examples: EX-E-034. **Literal test mentions:** none recorded.

H-T465 / PSC-VIII-T38

Declared claim: Unlimited concurrency task graph schedule; PROVED; Paper VIII. Symbol: komorebi_math.algorithms.scheduling.schedule_unlimited.

Declared hypotheses: finite acyclic task graph; declared task durations; unlimited concurrency. Suite label: algorithms.scheduling.

Locations: algorithms/scheduling.py:64. **Direct base sites:** none recorded.

Certificate fields: no direct literal-ID base constructor is assigned by this static link. Helper-selected IDs and caller-supplied labels are traced through their declarations and the observation ledger; do not infer a per-ID field contract from spelling alone.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T466 / PSC-X-COCHAIN-COMPLEX

Source symbol(s): komorebi_math.deformation.complexes.FiniteCochainComplex.certificate. No attached provenance record under this label.

Locations: deformation/complexes.py:139. **Direct base sites:** H-C078.

H-C078: status Exactness.EXACT; checked ('prime field', 'finite bounded complex', 'successive differentials'); unchecked (). Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T467 / PSC-X-FINITE-OCCUPANCY

Source symbol(s): `komorebi_math.deformation.support.nested_subspace_occupancy`. No attached provenance record under this label.

Locations: `deformation/support.py:222`. **Direct base sites:** `H-C081`.

`H-C081`: status `Exactness.NUMERICAL`; checked `('finite-dimensional column spaces', 'numerical principal-angle intersection')`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: `EX-E-012`. **Literal test mentions:** none recorded.

H-T468 / PSC-X-FITTING-SUPPORT

Source symbol(s): `komorebi_math.deformation.support.PresentedModule.fiber`. No attached provenance record under this label.

Locations: `deformation/support.py:145`. **Direct base sites:** `H-C080`.

`H-C080`: status `exactness`; checked `assumptions + ('zeroth Fitting minors',)`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T469 / PSC-X-FROBENIUS-SEPARATOR

Source symbol(s): `komorebi_math.deformation.companions.finite_frobenius_separator`. No attached provenance record under this label.

Locations: `deformation/companions.py:230`. **Direct base sites:** `H-C077`.

`H-C077`: status `Exactness.EXACT`; checked `('finite supplied Frobenius trace table', 'exact column rank')`; unchecked `('Chebotarev density beyond the supplied table',)`. Residual keys: none recorded (computed/default when not literal).

Observed examples: `EX-E-023`. **Literal test mentions:** none recorded.

H-T470 / PSC-X-LIFT-FOREST

Source symbol(s): `komorebi_math.deformation.forests.FiniteLiftForest.report`. No attached provenance record under this label.

Locations: `deformation/forests.py:182`. **Direct base sites:** `H-C079`.

`H-C079`: status `Exactness.EXACT`; checked `('finite consecutive levels', 'compatible reduction maps', 'finite stable core calculation')`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: `EX-E-012`. **Literal test mentions:** none recorded.

H-T471 / PSC-XI-GAUSS

Source symbol(s): `komorebi_math.langlands.local.gauss.gauss_sum`. No attached provenance record under this label.

Locations: `langlands/local/gauss.py:67`. **Direct base sites:** `H-C192`.

`H-C192`: status `Exactness.NUMERICAL`; checked `('explicit finite field', 'nontrivial additive character')`; unchecked `()`. Residual keys: `magnitude`.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T472 / PSC-XI-HEISENBERG-LAW

Source symbol(s): `komorebi_math.langlands.local.heisenberg.HeisenbergModel.group_law_certificate`. No attached provenance record under this label.

Locations: `langlands/local/heisenberg.py:115`. **Direct base sites:** `H-C193`.

`H-C193`: status `Exactness.NUMERICAL`; checked `('finite Schrödinger representation',)`; unchecked `()`. Residual keys: `group_law`.

Observed examples: `EX-E-056`. **Literal test mentions:** none recorded.

H-T473 / PSC-XI-HEISENBERG-TRACE

Source symbol(s): `komorebi_math.langlands.local.heisenberg.HeisenbergModel.trace_support_certificate`. No attached provenance record under this label.

Locations: `langlands/local/heisenberg.py:139`. **Direct base sites:** `H-C194`.

`H-C194`: status `Exactness.NUMERICAL`; checked `('finite exhaustive phase space',)`; unchecked `()`. Residual keys: `off_center_trace`.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T474 / PSC-XI-ISOTROPIC-SELECTOR

Source symbol(s): `komorebi_math.langlands.local.heisenberg.HeisenbergModel.selector_certificate`. No attached provenance record under this label.

Locations: `langlands/local/heisenberg.py:163`. **Direct base sites:** `H-C195`.

`H-C195`: status `Exactness.NUMERICAL`; checked `('standard translation Lagrangian',)`; unchecked `()`. Residual keys: `idempotence`, `self_adjoint`.

Observed examples: `EX-E-056`. **Literal test mentions:** none recorded.

H-T475 / PSC-XI-MONODROMY

Source symbol(s): `komorebi_math.langlands.local.tame.SpecialMonodromyBlock.certificate`. No attached provenance record under this label.

Locations: `langlands/local/tame.py:259`. **Direct base sites:** `H-C201`.

`H-C201`: status `Exactness.NUMERICAL`; checked `('finite monodromy chain',)`; unchecked `()`. Residual keys: `monodromy_relation`, `nilpotence`.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T476 / PSC-XI-PROJECTIVE-ENVELOPE

Source symbol(s): `komorebi_math.langlands.local.envelopes.ProjectiveIntertwinerEnvelope.recover`. No attached provenance record under this label.

Locations: `langlands/local/envelopes.py:91`. **Direct base sites:** `H-C191`.

`H-C191`: status `Exactness.NUMERICAL`; checked `('projective basis intertwiners', 'scalar commutators')`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T477 / PSC-XI-QUADRATIC-REFINEMENT

Source symbol(s): `komorebi_math.langlands.local.symplectic.QuadraticRefinement.polar_certificate`. No attached provenance record under this label.

Locations: `langlands/local/symplectic.py:143`. **Direct base sites:** `H-C197`.

`H-C197`: status `Exactness.EXACT`; checked `('finite exhaustive dyadic vectors',)`; unchecked `()`. Residual keys: `polar_mismatch`.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T478 / PSC-XI-RESTRICTED-BRIDGE

Source symbol(s): `komorebi_math.langlands.local.bridge.SymplecticBridge.construct`. No attached provenance record under this label.

Locations: `langlands/local/bridge.py:50`. **Direct base sites:** `H-C190`.

`H-C190`: status `Exactness.EXACT`; checked `('finite prime field symplectic forms',)`; unchecked `()`. Residual keys: `symplectic_mismatch`.

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T479 / PSC-XI-SYMPLECTIC-ENVELOPE

Source symbol(s): `komorebi_math.langlands.local.symplectic.AlternatingForm.certificate`. No attached provenance record under this label.

Locations: `langlands/local/symplectic.py:92`. **Direct base sites:** `H-C196`.

`H-C196`: status `Exactness.EXACT`; checked `('prime field', 'alternating matrix')`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T480 / PSC-XI-TAME-ATOM

Source symbol(s): `komorebi_math.langlands.local.tame.TameWeilPacket.relation_certificate`. No attached provenance record under this label.

Locations: `langlands/local/tame.py:178`. **Direct base sites:** `H-C199`.

`H-C199`: status `Exactness.NUMERICAL`; checked `('regular tame orbit', 'declared Frobenius cycle')`; unchecked `()`. Residual keys: `tame_conjugacy`.

Observed examples: `EX-E-056`. **Literal test mentions:** none recorded.

H-T481 / PSC-XI-TAME-ORBIT

Source symbol(s): `komorebi_math.langlands.local.tame.FrobeniusOrbit.certificate`. No attached provenance record under this label.

Locations: `langlands/local/tame.py:109`. **Direct base sites:** `H-C198`.

`H-C198`: status `Exactness.EXACT`; checked `('finite exponent orbit',)`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: `EX-E-056`. **Literal test mentions:** none recorded.

H-T482 / PSC-XI-TAME-TENSOR

Source symbol(s): `komorebi_math.langlands.local.tame.tame_tensor_refinement`. No attached provenance record under this label.

Locations: `langlands/local/tame.py:212`. **Direct base sites:** `H-C200`.

`H-C200`: status `Exactness.EXACT`; checked `('two cyclic Frobenius orbits',)`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T483 / PSC-XII-DYADIC-SIGN

Source symbol(s): `komorebi_math.langlands.global_.dyadic.dyadic_sign_diagnostic`. No attached provenance record under this label.

Locations: `langlands/global_/dyadic.py:32`. **Direct base sites:** `H-C186`.

`H-C186`: status `Exactness.EXACT`; checked `('group algebra of C2 over F2',)`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T484 / PSC-XII-FIVE-GATE-COMPLETION

Source symbol(s): `komorebi_math.langlands.completion.gates.AutomorphicCompletionReport.result`. No attached provenance record under this label.

Locations: `langlands/completion/gates.py:69`. **Direct base sites:** `H-C177`.

`H-C177`: status `Exactness.CONDITIONAL`; checked `checked`; unchecked `unchecked`. Residual keys: none recorded (computed/default when not literal).

Observed examples: `EX-E-044`. **Literal test mentions:** none recorded.

H-T485 / PSC-XII-HECKE-PACKETS

Source symbol(s): `komorebi_math.langlands.global_.hecke.FiniteHeckePackets.certificate`. No attached provenance record under this label.

Locations: `langlands/global_/hecke.py:87`. **Direct base sites:** `H-C187`.

`H-C187`: status `Exactness.NUMERICAL`; checked `('finite central idempotents', 'complete orthogonal decomposition')`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T486 / PSC-XII-ONE-MEMBER-PROPAGATION

Source symbol(s): `komorebi_math.langlands.completion.gates.CompatiblePropagationReport.result`. No attached provenance record under this label.

Locations: `langlands/completion/gates.py:103`. **Direct base sites:** `H-C178`.

`H-C178`: status `Exactness.CONDITIONAL`; checked `tuple((name for name, value in values.items() if value))`; unchecked `tuple((name for name, value in values.items() if not value))`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T487 / PSC-XII-PATCHING-SUPPORT

Source symbol(s): `komorebi_math.langlands.global_.patching.PatchingSupportReport.certificate`. No attached provenance record under this label.

Locations: `langlands/global_/patching.py:58`. **Direct base sites:** `H-C189`.

`H-C189`: status `Exactness.CONDITIONAL`; checked `tuple((label for value, label in statuses if value))`; unchecked `tuple((label for value, label in statuses if not value))`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H-T488 / PSC-XII-SUPPORT-COMPARISON

Source symbol(s): `komorebi_math.langlands.global_.hecke.SupportComparison.construct`. No attached provenance record under this label.

Locations: `langlands/global_/hecke.py:126`. **Direct base sites:** `H-C188`.

`H-C188`: status `Exactness.EXACT`; checked `('finite declared component sets',)`; unchecked `()`. Residual keys: none recorded (computed/default when not literal).

Observed examples: none recorded. **Literal test mentions:** none recorded.

H.8 Certificate construction-site matrix

The static scan identifies 357 base constructions and 10 specialized/wrapper constructions. Every construction has its complete argument expressions and source fragment in the JSON ledger. This matrix provides the callable, locus, validity/status rule and cross-reference label. It does not claim that every site was executed. The 57-example replay observed 139 base sites; 218 base sites were not observed by that instrumentation.

Construction group: algorithms

H-C001 `komorebi_math.algorithms.compiler.compile_tree_lagrangian`; `algorithms/compiler.py:119`.

Validity `True`; status `Exactness.EXACT`; tolerance `None`. Label expression `'PSC-VIII-T30'`. Observed examples `EX-E-034`.

H-C002 `komorebi_math.algorithms.compiler.compile_tree_budget`; `algorithms/compiler.py:231`.

Validity `int(plan.quantized_error) <= int(budget)`; status `Exactness.EXACT`; tolerance `None`. Label expression `'PSC-VIII-T33'`. Observed examples `EX-E-034`.

H-C003 `komorebi_math.algorithms.discovery.discover_packet_support`; `algorithms/discovery.py:81`.

Validity `True`; status `Exactness.HEURISTIC`; tolerance `threshold`. Label expression `'PSC-VIII-H01'`. Observed examples none recorded.

H-C004 `komorebi_math.algorithms.discovery.discover_block_diagonal_low_rank`; `algorithms/discovery.py:149`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `tolerance`. Label expression `'PSC-VIII-T13'`. Observed examples none recorded.

H-C005 `komorebi_math.algorithms.gpu.NumpyPacketExecutor.apply`; `algorithms/gpu.py:41`.

Validity `True`; status `Exactness.NUMERICAL`; tolerance `None`. Label expression `'PSC-VIII-C01'`. Observed examples none recorded.

H-C006 `komorebi_math.algorithms.gpu.CupyPacketExecutor.apply`; `algorithms/gpu.py:99`.

Validity `True`; status `Exactness.NUMERICAL`; tolerance `None`. Label expression `'PSC-VIII-C02'`. Observed examples none recorded.

H-C007 `komorebi_math.algorithms.ir.PacketIR.certificate`; `algorithms/ir.py:259`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `tolerance`. Label expression `'PSC-VIII-T01'`. Observed examples `EX-E-035`.

H-C008 `komorebi_math.algorithms.krylov.krylov_basis`; `algorithms/krylov.py:75`.

Validity `orthogonality <= 100 * tolerance_value`; status `Exactness.NUMERICAL`; tolerance `100 * tolerance_value`. Label expression `'PSC-VIII-T20'`. Observed examples none recorded.

H-C009 `komorebi_math.algorithms.krylov.conjugate_gradient`; `algorithms/krylov.py:183`.

Validity `True`; status `Exactness.NUMERICAL`; tolerance `threshold`. Label expression `'PSC-VIII-T22'`. Observed examples none recorded.

H-C010 `komorebi_math.algorithms.krylov.conjugate_gradient`; `algorithms/krylov.py:259`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `threshold`. Label expression `'PSC-VIII-T22'`. Observed examples `EX-E-055`.

H-C011 `komorebi_math.algorithms.krylov.lanczos_tridiagonal`; `algorithms/krylov.py:362`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `1000 * tolerance_value`. Label expression `'PSC-VIII-T23'`. Observed examples none recorded.

H-C012 `komorebi_math.algorithms.krylov.block_jacobi_certificate`; `algorithms/krylov.py:456`.

Validity `bound_valid`; status `Exactness.NUMERICAL`; tolerance `tolerance_value`. Label expression `'PSC-VIII-T24'`. Observed examples `EX-E-055`.

H-C013 `komorebi_math.algorithms.krylov.block_jacobi_certificate`; `algorithms/krylov.py:478`.

Specialized/wrapper construction `komorebi_math.algorithms.krylov.BlockJacobiCertificate`. Fields: `certificate`, `condition_bound`, `diagonal`, `eta`, `normalized_coupling`. No base-constructor execution claim is inferred for this wrapper.

H-C014 `komorebi_math.algorithms.low_rank.recompress_core`; `algorithms/low_rank.py:137`.

Validity `True`; status `Exactness.NUMERICAL`; tolerance `tolerance_value`. Label expression `'PSC-VIII-T12'`. Observed examples none recorded.

H-C015 `komorebi_math.algorithms.low_rank.recompress_core`; `algorithms/low_rank.py:166`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `tolerance_value`. Label expression `'PSC-VIII-T12'`. Observed examples `EX-E-028`.

H-C016 `komorebi_math.algorithms.precision.assign_discrete_precisions`; `algorithms/precision.py:201`.

Validity `error_units <= budget`; status `Exactness.EXACT`; tolerance `None`. Label expression `'PSC-VIII-T28'`. Observed examples `EX-E-038`.

H-C017 `komorebi_math.algorithms.precision.precision_certificate`; `algorithms/precision.py:269`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `tolerance_value`. Label expression `'PSC-VIII-T25'`. Observed examples none recorded.

Construction group: analytic

H-C018 `komorebi_math.analytic.cyclic.cyclic_projection_certificate`; `analytic/cyclic.py:121`.

Validity `residual <= tolerance`; status `Exactness.EXACT`; tolerance `tolerance`. Label expression `'PSC-IV-T01'`. Observed examples `EX-E-011`.

H-C019 `komorebi_math.analytic.reciprocal.reciprocal_semigroup_certificate`; `analytic/reciprocal.py:150`.

Validity `maximum <= tolerance`; status `Exactness.EXACT if exact and maximum == 0 else Exactness.NUMERICAL`; tolerance `None if exact and maximum == 0 else tolerance`. Label expression `'PSC-IV-T18'`. Observed examples none recorded.

H-C020 `komorebi_math.analytic.sections.section_certificate`; `analytic/sections.py:224`.

Validity `residual <= tolerance`; status `Exactness.NUMERICAL`; tolerance `tolerance`. Label expression `'PSC-IV-T05' if family == 'transfer' else 'PSC-IV-T09'`. Observed examples none recorded.

H-C021 `komorebi_math.analytic.zeros.positive_zero_string`; `analytic/zeros.py:127`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `tolerance * 20`. Label expression `'PSC-IV-T54'`. Observed examples `EX-E-011`.

H-C022 `komorebi_math.analytic.zeros.reduced_zero_zeta`; `analytic/zeros.py:205`.

Validity `True`; status `Exactness.NUMERICAL`; tolerance `None`. Label expression `'PSC-IV-T68'`. Observed examples none recorded.

H-C023 `komorebi_math.analytic.zeros.reduced_zero_zeta`; `analytic/zeros.py:224`.

Validity `True`; status `Exactness.CONDITIONAL`; tolerance `None`. Label expression `'PSC-IV-T68'`. Observed examples none recorded.

Construction group: arithmetic

H-C024 `komorebi_math.arithmetic.euler.EulerLocalCoordinates.truncate`; `arithmetic/euler.py:272`.

Validity `valid`; status `Exactness.EXACT`; tolerance `None`. Label expression `'PSC-IX-T19'`. Observed examples `EX-E-017`, `EX-E-027`.

H-C025 `komorebi_math.arithmetic.euler.EulerLocalCoordinates.certificate`; `arithmetic/euler.py:304`.

Validity `valid and inverse_valid and weight_valid`; status `Exactness.EXACT`; tolerance `None`. Label expression `'PSC-IX-T19'`. Observed examples none recorded.

H-C026 `komorebi_math.arithmetic.local_modes.certify_difference_congruence`; `arithmetic/local_modes.py:81`.

Validity `valid`; status `Exactness.EXACT`; tolerance `None`. Label expression `None`. Observed examples none recorded.

H-C027 `komorebi_math.arithmetic.local_modes.simultaneous_unit_difference`; `arithmetic/local_modes.py:133`.

Validity `valid`; status `Exactness.EXACT`; tolerance `None`. Label expression `'PSC-IX-T26'`. Observed examples none recorded.

H-C028 `komorebi_math.arithmetic.mahler.MahlerSeries.reconstruction_certificate`; `arithmetic/mahler.py:161`.

Validity `valid`; status `Exactness.EXACT if exact else Exactness.NUMERICAL`; tolerance `None`. Label expression `'PSC-IX-T28'`. Observed examples none recorded.

H-C029 `komorebi_math.arithmetic.mahler.affine_kernel_composition_certificate`; `arithmetic/mahler.py:228`.

Validity `valid`; status `Exactness.EXACT`; tolerance `None`. Label expression `'PSC-IX-T34'`. Observed examples none recorded.

H-C030 `komorebi_math.arithmetic.mahler.prime_power_kernel_certificate`; `arithmetic/mahler.py:280`.

Validity `valid`; status `Exactness.EXACT`; tolerance `None`. Label expression `'PSC-IX-T36'`. Observed examples none recorded.

H-C031 `komorebi_math.arithmetic.mahler.UnitModeMahler.certificate`; `arithmetic/mahler.py:356`.

Validity `valid`; status `Exactness.EXACT`; tolerance `None`. Label expression `'PSC-IX-T25'`. Observed examples none recorded.

H-C032 `komorebi_math.arithmetic.recurrence.LinearRecurrence.period_mod`; `arithmetic/recurrence.py:257`.

Validity `valid`; status `Exactness.EXACT`; tolerance `None`. Label expression `'PSC-IX-T47'`. Observed examples `EX-E-045`.

H-C033 `komorebi_math.arithmetic.recurrence.LinearRecurrence.certificate`; `arithmetic/recurrence.py:313`.

Validity `recurrence_valid and state_valid`; status `Exactness.EXACT if exact else Exactness.NUMERICAL`; tolerance `None`. Label expression `'PSC-IX-T43'`. Observed examples none recorded.

H-C034 `komorebi_math.arithmetic.recurrence.RecurrencePacket.certificate`; `arithmetic/recurrence.py:384`.

Validity `valid`; status `Exactness.EXACT if exact else Exactness.NUMERICAL`; tolerance `None`. Label expression `'PSC-IX-T43'`. Observed examples `EX-E-045`.

H-C035 `komorebi_math.arithmetic.standard_windows.StandardWindow.certificate`; `arithmetic/standard_windows.py:176`.

Validity `valid`; status `Exactness.EXACT`; tolerance `None`. Label expression `'PSC-IX-T09'`. Observed examples `EX-E-054`.

Construction group: compatibility.py

H-C036 `komorebi_math.compatibility.compare_public_api`; `compatibility.py:479`.

Validity `valid`; status `Exactness.EXACT`; tolerance `None`. Label expression `'KM-API-010'`. Observed examples none recorded.

Construction group: coordinates

H-C037 `komorebi_math.coordinates.affine.AffineCoordinateTransform.certificate`; `coordinates/affine.py:141`.

Validity `valid`; status `Exactness.EXACT`; tolerance `None`. Label expression `self.theorem`. Observed examples none recorded.

H-C038 `komorebi_math.coordinates.affine.BarycentricCoordinateSystem.certificate`; `coordinates/affine.py:224`.

Validity `recovered == expected_columns`; status `Exactness.EXACT`; tolerance `None`. Label expression `'CC.F3'`. Observed examples none recorded.

H-C039 `komorebi_math.coordinates.analysis_synthesis.AnalysisSynthesisBlocks.certificate`; `coordinates/analysis_synthesis.py:108`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `tolerance`. Label expression `'KM-AS-02'`. Observed examples `EX-E-004`.

H-C040 `komorebi_math.coordinates.analysis_synthesis.AnalysisSynthesisSystem.certificate`; `coordinates/analysis_synthesis.py:336`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `self.tolerance`. Label expression `self.theorem or None`. Observed examples `EX-E-004`, `EX-E-007`.

H-C041 `komorebi_math.coordinates.analysis_synthesis.AnalysisSynthesisSystem.reconstruction_certificate`; `coordinates/analysis_synthesis.py:369`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `self.tolerance`. Label expression `self.theorem or None`. Observed examples none recorded.

H-C042 `komorebi_math.coordinates.basis.FiniteBasis.certificate`; `coordinates/basis.py:205`.

Validity `rank == self.dimension`; status `Exactness.EXACT`; tolerance `None`. Label expression `'CC.B0'`. Observed examples none recorded.

H-C043 `komorebi_math.coordinates.basis.CommutingDiagramCheck.certificate`; `coordinates/basis.py:246`.

Validity `self.valid`; status `Exactness.EXACT`; tolerance `None`. Label expression `'CC.A2'`. Observed examples `EX-E-024`.

H-C044 `komorebi_math.coordinates.calculus.PolynomialCoordinateOperator.certificate`; `coordinates/calculus.py:91`.

Validity `rows == self.target_degree + 1 and columns == self.source_degree + 1`; status `Exactness.EXACT`; tolerance `None`. Label expression `self.theorem`. Observed examples none recorded.

H-C045 `komorebi_math.coordinates.formal.FormalCoordinateChange.certificate`; `coordinates/formal.py:229`.

Validity `valid`; status `Exactness.EXACT`; tolerance `None`. Label expression `'CC.J1'`. Observed examples `EX-E-002`, `EX-E-021`.

H-C046 `komorebi_math.coordinates.frame_algebra.AnalysisSynthesisGauge.certificate`; `coordinates/frame_algebra.py:311`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `self.tolerance`. Label expression `'KM-AS-GAUGE-01'`. Observed examples none recorded.

H-C047 `komorebi_math.coordinates.frame_algebra.AnalysisSynthesisMap.certificate`; `coordinates/frame_algebra.py:468`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `self.tolerance`. Label expression `self.theorem or None`. Observed examples none recorded.

H-C048 `komorebi_math.coordinates.frame_algebra.AnalysisSynthesisCoefficientBlocks.certificate`; `coordinates/frame_algebra.py:569`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `self.mapping.tolerance`. Label expression `'KM-AS-BLOCK-01'`. Observed examples none recorded.

H-C049 `komorebi_math.coordinates.frame_algebra.ExactAnalysisSynthesisSystem.certificate`; `coordinates/frame_algebra.py:835`.

Validity `valid`; status `Exactness.EXACT`; tolerance `None`. Label expression `self.theorem or None`. Observed examples none recorded.

H-C050 `komorebi_math.coordinates.frame_algebra.ExactAnalysisSynthesisMap.certificate`; `coordinates/frame_algebra.py:996`.

Validity `valid`; status `Exactness.EXACT`; tolerance `None`. Label expression `self.theorem or None`. Observed examples none recorded.

H-C051 `komorebi_math.coordinates.modular.FiniteFieldCoordinateTransform.certificate`; `coordinates/modular.py:174`.

Validity `True`; status `Exactness.EXACT`; tolerance `None`. Label expression `self.theorem`. Observed examples none recorded.

H-C052 `komorebi_math.coordinates.modular.FiniteFieldCoordinateTransform.certificate`; `coordinates/modular.py:190`.

Validity `False`; status `Exactness.EXACT`; tolerance `None`. Label expression `self.theorem`. Observed examples none recorded.

H-C053 `komorebi_math.coordinates.modular.FiniteFieldCoordinateTransform.certificate`; `coordinates/modular.py:203`.

Validity `valid`; status `Exactness.EXACT`; tolerance `None`. Label expression `self.theorem`. Observed examples none recorded.

H-C054 `komorebi_math.coordinates.numeric.NumericCoordinateTransform.certificate`; `coordinates/numeric.py:124`.

Validity `True`; status `Exactness.NUMERICAL`; tolerance `None`. Label expression `self.theorem`. Observed examples none recorded.

H-C055 `komorebi_math.coordinates.numeric.NumericCoordinateTransform.certificate`; `coordinates/numeric.py:139`.

Validity `False`; status `Exactness.NUMERICAL`; tolerance `None`. Label expression `self.theorem`. Observed examples none recorded.

H-C056 `komorebi_math.coordinates.numeric.NumericCoordinateTransform.certificate`; `coordinates/numeric.py:151`.

Validity `residual <= self.tolerance * max(1.0, self.condition_number())`; status `Exactness.NUMERICAL`; tolerance `self.tolerance`. Label expression `self.theorem`. Observed examples `EX-E-053`.

H-C057 `komorebi_math.coordinates.numeric.ModalCoordinateSystem.certificate`; `coordinates/numeric.py:300`.

Validity `self.diagonal_residual <= tolerance * max(1.0, self.synthesis.condition_number())`; status `Exactness.NUMERICAL`; tolerance `tolerance`. Label expression `'CC.S5'`. Observed examples none recorded.

H-C058 `komorebi_math.coordinates.operators.BezoutDefectIdentity.certificate`; `coordinates/operators.py:90`.

Validity `self.valid and self.reconstructed == self.expected`; status `Exactness.EXACT`; tolerance `None`. Label expression `'CC.O2'`. Observed examples none recorded.

H-C059 `komorebi_math.coordinates.realization.StateRecovery.certificate`; `coordinates/realization.py:105`.

Validity `valid`; status `Exactness.EXACT`; tolerance `None`. Label expression `'CC.S2'`. Observed examples none recorded.

H-C060 `komorebi_math.coordinates.realization.IntertwinerSpace.certificate`; `coordinates/realization.py:154`.

Validity `valid`; status `Exactness.EXACT`; tolerance `None`. Label expression `'CC.S3'`. Observed examples none recorded.

H-C061 `komorebi_math.coordinates.recurrence.RecurrenceCoordinateRecord.certificate`; `coordinates/recurrence.py:189`.

Validity `valid`; status `Exactness.EXACT`; tolerance `None`. Label expression `'CC.R1'`. Observed examples `EX-E-007`, `EX-E-014`.

H-C062 `komorebi_math.coordinates.state_space.StateSpaceCoordinateCertificate.certificate`; `coordinates/state_space.py:271`.

Validity `self.valid`; status `Exactness.EXACT`; tolerance `None`. Label expression `'CC.R3'`. Observed examples `EX-E-046`.

H-C063 `komorebi_math.coordinates.state_space.certify_state_coordinate_change`; `coordinates/state_space.py:300`.

Specialized/wrapper construction `komorebi_math.coordinates.state_space.StateSpaceCoordinateCertificate`. Fields: `markov_count`, `source`, `target`, `transform`, `valid`. No base-constructor execution claim is inferred for this wrapper.

H-C064 `komorebi_math.coordinates.transform.CoordinateTransform.certificate`; `coordinates/transform.py:135`.

Validity `valid`; status `Exactness.EXACT`; tolerance `None`. Label expression `self.theorem or None`. Observed examples `EX-E-008`.

Construction group: core

H-C065 `komorebi_math.core.algebra.OperatorAlgebra.certificate`; `core/algebra.py:138`.

Validity `valid`; status `Exactness.EXACT if self.backend.exact else Exactness.NUMERICAL`; tolerance `None if self.backend.exact else self.tolerance`. Label expression `'PSC-I-D05'`. Observed examples none recorded.

H-C066 `komorebi_math.core.graphs.TransferGraph.source_component_certificate`; `core/graphs.py:195`.

Validity `True`; status `Exactness.CONDITIONAL`; tolerance `None`. Label expression `'PSC-I-T44'`. Observed examples none recorded.

H-C067 `komorebi_math.core.morphisms.OperatorMorphism.certificate`; `core/morphisms.py:72`.

Validity `valid`; status `Exactness.EXACT if self.backend.exact else Exactness.NUMERICAL`; tolerance `None if self.backend.exact else self.tolerance`. Label expression `'PSC-I-T50'`. Observed examples none recorded.

H-C068 `komorebi_math.core.morphisms.PacketMorphism.certificate`; `core/morphisms.py:122`.

Validity `valid`; status `Exactness.EXACT if backend.exact else Exactness.NUMERICAL`; tolerance `None if backend.exact else tolerance`. Label expression `'PSC-I-T52'`. Observed examples none recorded.

H-C069 `komorebi_math.core.morphisms.SeedMorphism.certificate`; `core/morphisms.py:173`.

Validity `valid`; status `Exactness.EXACT if backend.exact else Exactness.NUMERICAL`; tolerance `None if backend.exact else tolerance`. Label expression `'PSC-I-T50'`. Observed examples none recorded.

H-C070 `komorebi_math.core.operators.PacketOperator.certificate`; `core/operators.py:93`.

Validity `valid`; status `Exactness.EXACT if self.backend.exact else Exactness.NUMERICAL`; tolerance `None if self.backend.exact else tolerance`. Label expression `'PSC-I-T27'`. Observed examples none recorded.

H-C071 `komorebi_math.core.packets.PacketSystem.certificate`; `core/packets.py:119`.

Validity `valid`; status `Exactness.EXACT if self.backend.exact else Exactness.NUMERICAL`; tolerance `None if self.backend.exact else self.tolerance`. Label expression `'PSC-I-T20'`. Observed examples `EX-E-002`, `EX-E-015`, `EX-E-031`, `EX-E-037`, `EX-E-042`, `EX-E-043`, `EX-E-047`, `EX-E-048`, `EX-E-049`, `EX-E-050`.

H-C072 `komorebi_math.core.packets.PacketSystem.hilbert_certificate`; `core/packets.py:144`.

Validity `valid`; status `Exactness.EXACT if self.backend.exact else Exactness.NUMERICAL`; tolerance `None if self.backend.exact else self.tolerance`. Label expression `'PSC-I-T20'`. Observed examples none recorded.

H-C073 `komorebi_math.core.realization.OperatorRealization.relation_certificate`; `core/realization.py:109`.

Validity `valid`; status `exactness`; tolerance `None if self.backend.exact else self.tolerance`. Label expression `'PSC-I-T01'`. Observed examples `EX-E-057`.

H-C074 `komorebi_math.core.seeds.SeedSystem.generation_certificate`; `core/seeds.py:147`.

Validity `valid`; status `Exactness.EXACT if self.backend.exact else Exactness.NUMERICAL`; tolerance `None if self.backend.exact else self.tolerance`. Label expression `'PSC-I-T02'`. Observed examples `EX-E-043`.

H-C075 `komorebi_math.core.seeds.SeedSystem.minimum_standard_basis_seeds`; `core/seeds.py:258`.

Validity `False`; status `Exactness.EXACT if self.backend.exact else Exactness.NUMERICAL`; tolerance `None if self.backend.exact else self.tolerance`. Label expression `None`. Observed examples none recorded.

H-C076 `komorebi_math.core.transfer.block_from_paths`; `core/transfer.py:76`.

Validity `valid`; status `Exactness.EXACT if operator.backend.exact else Exactness.NUMERICAL`; tolerance `None if operator.backend.exact else tol`. Label expression `'PSC-I-T29'`. Observed examples none recorded.

Construction group: deformation

H-C077 `komorebi_math.deformation.companions.finite_frobenius_separator`; `deformation/companions.py:230`.

Validity `True`; status `Exactness.EXACT`; tolerance `None`. Label expression `'PSC-X-FROBENIUS-SEPARATOR'`. Observed examples `EX-E-023`.

H-C078 `komorebi_math.deformation.complexes.FiniteCochainComplex.certificate`; `deformation/complexes.py:139`.

Validity `valid`; status `Exactness.EXACT`; tolerance `None`. Label expression `'PSC-X-COCHAIN-COMPLEX'`. Observed examples none recorded.

H-C079 `komorebi_math.deformation.forests.FiniteLiftForest.report`; `deformation/forests.py:182`.

Validity `valid`; status `Exactness.EXACT`; tolerance `None`. Label expression `'PSC-X-LIFT-FOREST'`. Observed examples `EX-E-012`.

H-C080 `komorebi_math.deformation.support.PresentedModule.fiber`; `deformation/support.py:145`.

Validity `valid`; status `exactness`; tolerance `None`. Label expression `'PSC-X-FITTING-SUPPORT'`. Observed examples none recorded.

H-C081 `komorebi_math.deformation.support.nested_subspace_occupancy`; `deformation/support.py:222`.

Validity `nested`; status `Exactness.NUMERICAL`; tolerance `tolerance`. Label expression `'PSC-X-FINITE-OCCUPANCY'`. Observed examples `EX-E-012`.

Construction group: dynamics

H-C082 `komorebi_math.dynamics.bayes.BayesianPartition.coarse_posterior`; `dynamics/bayes.py:152`.

Validity `residual <= 1e-12`; status `Exactness.NUMERICAL`; tolerance `1e-12`. Label expression `'PSC-VI-T36'`. Observed examples `EX-E-030`.

H-C083 `komorebi_math.dynamics.bayes.relative_entropy_decomposition`; `dynamics/bayes.py:313`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `1e-12`. Label expression `'PSC-VI-T41'`. Observed examples none recorded.

H-C084 `komorebi_math.dynamics.bayes.HiddenMarkovFilter.coarse_step`; `dynamics/bayes.py:404`.

Validity `False`; status `Exactness.CONDITIONAL`; tolerance `None`. Label expression `'PSC-VI-T46'`. Observed examples none recorded.

H-C085 `komorebi_math.dynamics.bayes.HiddenMarkovFilter.coarse_step`; `dynamics/bayes.py:427`.

Validity `residual <= tolerance`; status `Exactness.NUMERICAL`; tolerance `tolerance`. Label expression `'PSC-VI-T46'`. Observed examples none recorded.

H-C086 `komorebi_math.dynamics.feshbach.FeshbachReduction.resolvent_certificate`; `dynamics/feshbach.py:136`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `tolerance`. Label expression `'PSC-VI-T28'`. Observed examples `EX-E-018`, `EX-E-026`.

H-C087 `komorebi_math.dynamics.feshbach.FeshbachReduction.local_closure_certificate`; `dynamics/feshbach.py:185`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `tolerance`. Label expression `'PSC-VI-T29'`. Observed examples `EX-E-018`.

H-C088 `komorebi_math.dynamics.feynman_kac.FiniteFeynmanKac.coarse_model`; `dynamics/feynman_kac.py:154`.

Validity `False`; status `Exactness.CONDITIONAL`; tolerance `tolerance`. Label expression `'PSC-VI-C57'`. Observed examples none recorded.

H-C089 `komorebi_math.dynamics.feynman_kac.FiniteFeynmanKac.coarse_model`; `dynamics/feynman_kac.py:173`.

Validity `True`; status `Exactness.NUMERICAL`; tolerance `tolerance`. Label expression `'PSC-VI-C57'`. Observed examples none recorded.

H-C090 `komorebi_math.dynamics.fractional.fractional_naturality_certificate`; `dynamics/fractional.py:102`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `tolerance`. Label expression `'PSC-VI-T25'`. Observed examples none recorded.

H-C091 `komorebi_math.dynamics.fractional.fractional_defect`; `dynamics/fractional.py:166`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `max(tolerance, 1e-10 * max(1.0, bound))`. Label expression `'PSC-VI-T26'`. Observed examples none recorded.

H-C092 `komorebi_math.dynamics.green.DirichletGreen1D.certificate`; `dynamics/green.py:77`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `tolerance`. Label expression `'PSC-VI-T49'`. Observed examples none recorded.

H-C093 `komorebi_math.dynamics.markov.FiniteMarkovChain.stationary_distribution`; `dynamics/markov.py:225`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `tolerance`. Label expression `'PSC-VI-C22'`. Observed examples none recorded.

H-C094 `komorebi_math.dynamics.markov.MarkovLumping.certificate`; `dynamics/markov.py:381`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `tolerance`. Label expression `'PSC-VI-T21'`. Observed examples `EX-E-030`.

H-C095 `komorebi_math.dynamics.martingales.ConditionalExpectation.certificate`; `dynamics/martingales.py:81`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `tolerance`. Label expression `'PSC-VI-T17'`. Observed examples none recorded.

H-C096 `komorebi_math.dynamics.martingales.MartingaleTower.energy_decomposition`; `dynamics/martingales.py:152`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `1e-10`. Label expression `'PSC-VI-T18'`. Observed examples none recorded.

H-C097 `komorebi_math.dynamics.martingales.MartingaleTower.tower_certificate`; `dynamics/martingales.py:172`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `tolerance`. Label expression `'PSC-VI-T18'`. Observed examples none recorded.

H-C098 `komorebi_math.dynamics.memory.MemoryKernel.certificate`; `dynamics/memory.py:118`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `tolerance`. Label expression `'PSC-VI-T30'`. Observed examples `EX-E-018`.

H-C099 `komorebi_math.dynamics.memory.memory_kernel_error_bound`; `dynamics/memory.py:241`.

Validity `np.isfinite(value) and value >= estimate - 1e-12`; status `Exactness.NUMERICAL`; tolerance `None`. Label expression `'PSC-VI-T31'`. Observed examples none recorded.

H-C100 `komorebi_math.dynamics.memory.memory_kernel_error_bound`; `dynamics/memory.py:261`.

Validity `np.isfinite(estimate) and estimate >= 0`; status `Exactness.CONDITIONAL`; tolerance `None`. Label expression `'PSC-VI-T31'`. Observed examples none recorded.

H-C101 `komorebi_math.dynamics.semigroups.MatrixSemigroup.semigroup_certificate`; `dynamics/semigroups.py:76`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `tolerance`. Label expression `'PSC-VI-T01'`. Observed examples none recorded.

H-C102 `komorebi_math.dynamics.semigroups.PacketSemigroup.convolution_certificate`; `dynamics/semigroups.py:128`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `tolerance`. Label expression `'PSC-VI-T02'`. Observed examples none recorded.

H-C103 `komorebi_math.dynamics.semigroups.SemigroupRefinement.duhamel`; `dynamics/semigroups.py:211`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `tolerance`. Label expression `'PSC-VI-T08'`. Observed examples none recorded.

H-C104 `komorebi_math.dynamics.semigroups.SemigroupRefinement.resolvent_defect`; `dynamics/semigroups.py:240`.

Validity `residual <= tolerance`; status `Exactness.NUMERICAL`; tolerance `tolerance`. Label expression `'PSC-VI-T10'`. Observed examples none recorded.

H-C105 `komorebi_math.dynamics.semigroups.FiniteSemigroupTower.certificate`; `dynamics/semigroups.py:300`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `tolerance`. Label expression `'PSC-VI-T12'`. Observed examples none recorded.

H-C106 `komorebi_math.dynamics.survival.PhaseTypeSurvival.certificate`; `dynamics/survival.py:101`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `self.tolerance`. Label expression `'PSC-VI-T32'`. Observed examples `EX-E-022`.

H-C107 `komorebi_math.dynamics.survival.PhaseTypeSurvival.identities_certificate`; `dynamics/survival.py:182`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `1e-05`. Label expression `'PSC-VI-T33'`. Observed examples `EX-E-022`.

Construction group: export

H-C108 `komorebi_math.export.latex.compile_latex`; `export/latex.py:222`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `None`. Label expression `None`. Observed examples `EX-E-017`.

H-C109 `komorebi_math.export.persistence._certificate_from_data`; `export/persistence.py:220`.

Validity `bool(data['valid'])`; status `Exactness(data['exactness'])`; tolerance `None if data['tolerance'] is None else float(data['tolerance'])`. Label expression `None if data['theorem_id'] is None else str(data['theorem_id'])`. Observed examples `EX-E-002`, `EX-E-037`, `EX-E-048`, `EX-E-049`.

H-C110 `komorebi_math.export.writers.export_data`; `export/writers.py:362`.

Validity `target.is_file() and target.stat().st_size == size and (size > 0)`; status `Exactness.EXACT`; tolerance `None`. Label expression `None`. Observed examples `EX-E-017`.

H-C111 `komorebi_math.export.writers.export_bundle`; `export/writers.py:482`.

Validity `target.is_file() and target.stat().st_size > 0`; status `Exactness.EXACT`; tolerance `None`. Label expression `None`. Observed examples `EX-E-017`.

Construction group: expression

H-C112 `komorebi_math.expression.certification.compose_certificates`; `expression/certification.py:189`.

Validity `valid`; status `exactness`; tolerance `max(tolerances) if tolerances else None`. Label expression `theorem_id`. Observed examples `EX-E-006`, `EX-E-007`.

H-C113 `komorebi_math.expression.certification.exact_internal_certificate`; `expression/certification.py:228`.

Validity `True`; status `Exactness.EXACT`; tolerance `None`. Label expression `theorem_id`. Observed examples `EX-E-006`, `EX-E-007`.

H-C114 `komorebi_math.expression.planning.matrix_chain_plan`; `expression/planning.py:212`.

Validity `True`; status `Exactness.EXACT`; tolerance `None`. Label expression `_THEOREM_ID`. Observed examples `EX-E-006`, `EX-E-007`.

H-C115 `komorebi_math.expression.polynomial.primitive_certificate`; `expression/polynomial.py:587`.

Validity `valid`; status `Exactness.EXACT`; tolerance `0.0`. Label expression `primitive.theorem_id`. Observed examples `EX-E-006`, `EX-E-007`.

Construction group: generating

H-C116 `komorebi_math.generating.adapters.FIRSeriesAdapter.certificate`; `generating/adapters.py:61`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `tolerance`. Label expression `'GF-SIGNAL-FIR-BRIDGE'`. Observed examples none recorded.

H-C117 `komorebi_math.generating.basis.WeightedMonomialBasis.finite_certificate`; `generating/basis.py:219`.

Specialized/wrapper construction `komorebi_math.generating.basis.BasisCertificate`. Fields: `degree`, `integral_structure_constants`, `maximum_denominator`, `maximum_numerator_bits`, `name`, `nonzero_normalization`. No base-constructor execution claim is inferred for this wrapper.

H-C118 `komorebi_math.generating.bell.PointedSubindexCertificate.certificate`; `generating/bell.py:132`.

Validity `self.valid`; status `Exactness.EXACT`; tolerance `None`. Label expression `'NGF.POINTED_SUBINDEX'`. Observed examples none recorded.

H-C119 `komorebi_math.generating.bell.certify_pointed_subindex_identity`; `generating/bell.py:157`.

Specialized/wrapper construction `komorebi_math.generating.bell.PointedSubindexCertificate`. Fields: `index`, `left`, `prime`, `right`. No base-constructor execution claim is inferred for this wrapper.

H-C120 `komorebi_math.generating.bell.BellDigitReduction.certificate`; `generating/bell.py:187`.

Validity `self.valid`; status `Exactness.EXACT`; tolerance `None`. Label expression `'NGF.BELL_DIGIT_SUM_REDUCTION'`. Observed examples none recorded.

H-C121 `komorebi_math.generating.bell.BellResidueAutomaton.certificate`; `generating/bell.py:380`.

Validity `not failures`; status `Exactness.EXACT`; tolerance `None`. Label expression `'NGF.BELL_RESIDUE_AUTOMATON'`. Observed examples none recorded.

H-C122 `komorebi_math.generating.cartier.CartierAutomaton.certificate`; `generating/cartier.py:211`.

Specialized/wrapper construction `komorebi_math.generating.cartier.AutomatonCertificate`. Fields: `compared_terms`, `failures`. No base-constructor execution claim is inferred for this wrapper.

H-C123 `komorebi_math.generating.finite_series.SeriesTransferCertificate.certificate`; `generating/finite_series.py:77`.

Validity `self.valid`; status `Exactness.EXACT`; tolerance `None`. Label expression `'NGF.UNIVERSAL_PRODUCT_AND_TRANSFER'`. Observed examples none recorded.

H-C124 `komorebi_math.generating.finite_series.FiniteNormalizedSeries.convert`; `generating/finite_series.py:170`.

Specialized/wrapper construction `komorebi_math.generating.finite_series.SeriesTransferCertificate`. Fields: `degree`, `domain`, `source_basis`, `target_basis`, `valid`. No base-constructor execution claim is inferred for this wrapper.

H-C125 `komorebi_math.generating.finite_series.FiniteNormalizedSeries.convert`; `generating/finite_series.py:196`.

Specialized/wrapper construction `komorebi_math.generating.finite_series.SeriesTransferCertificate`. Fields: `degree`, `domain`, `failures`, `source_basis`, `target_basis`, `valid`. No base-constructor execution claim is inferred for this wrapper.

H-C126 `komorebi_math.generating.finite_series.FiniteNormalizedSeries.certificate`; `generating/finite_series.py:423`.

Validity `True`; status `Exactness.EXACT`; tolerance `None`. Label expression `'NGF.FINITE_NORMALIZED_SERIES'`. Observed examples `EX-E-010`.

H-C127 `komorebi_math.generating.masks.certify_mahler_functional_equation`; `generating/masks.py:178`.

Validity `not failures`; status `Exactness.EXACT`; tolerance `None`. Label expression `'GF-MASK-INVERSE'`. Observed examples none recorded.

H-C128 `komorebi_math.generating.masks.FiniteMaskCarryPlan.certificate`; `generating/masks.py:255`.

Validity `not failures`; status `Exactness.EXACT`; tolerance `None`. Label expression `'GF-FINITE-MASK-CARRY'`. Observed examples `EX-E-029`.

H-C129 `komorebi_math.generating.masks.KummerLayerDistribution.certificate`; `generating/masks.py:408`.

Validity `valid`; status `Exactness.EXACT`; tolerance `None`. Label expression `'GF-KUMMER-LAYERS'`. Observed examples none recorded.

H-C130 `komorebi_math.generating.masks.AnalyticMaskApproximation.certificate`; `generating/masks.py:654`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `self.absolute_error_bound`. Label expression `'GF-ANALYTIC-MASK-REALIZATION'`. Observed examples none recorded.

H-C131 `komorebi_math.generating.masks.RationalMaskAlgebraicCertificate.certificate`; `generating/masks.py:772`.

Validity `self.valid`; status `Exactness.EXACT`; tolerance `None`. Label expression `'GF-RATIONAL-MASK-ALGEBRAIC'`. Observed examples none recorded.

H-C132 `komorebi_math.generating.masks.certify_rational_mask_algebraic_relation`; `generating/masks.py:834`.

Specialized/wrapper construction `komorebi_math.generating.masks.RationalMaskAlgebraicCertificate`. Fields: `degree`, `failures`, `prime`. No base-constructor execution claim is inferred for this wrapper.

H-C133 `komorebi_math.generating.masks.PolynomialCoboundaryResult.certificate`; `generating/masks.py:907`.

Validity `self.valid`; status `Exactness.EXACT`; tolerance `None`. Label expression `'GF-POLYNOMIAL-COBOUNDARY'`. Observed examples none recorded.

H-C134 `komorebi_math.generating.normalization.classify_carry_linear_valuation`; `generating/normalization.py:108`.

Specialized/wrapper construction `komorebi_math.generating.normalization.CarryLinearCertificate`. Fields: `carry_weight`, `checked_through`, `failures`, `linear_slope`, `prime`. No base-constructor execution claim is inferred for this wrapper.

Construction group: golden.py

H-C135 `komorebi_math.golden._exact_snapshot`; `golden.py:104`.

Validity `True`; status `Exactness.EXACT`; tolerance `None`. Label expression `'KM-GOLDEN-011'`. Observed examples `EX-E-002`.

H-C136 `komorebi_math.golden._result`; `golden.py:228`.

Validity `passed`; status `exactness`; tolerance `tolerance`. Label expression `'KM-GOLDEN-011'`. Observed examples `EX-E-002`.

H-C137 `komorebi_math.golden.run_behavioral_golden_fixture`; `golden.py:590`.

Validity `not failed`; status `Exactness.EXACT`; tolerance `None`. Label expression `'KM-GOLDEN-011'`. Observed examples `EX-E-002`.

Construction group: integration

H-C138 `komorebi_math.integration.adapters.AtomicIntegratedDerivativeProvider.certificate`; `integration/adapters.py:247`.

Validity `True`; status `self.exactness`; tolerance `None`. Label expression `'INT.ADAPTER.DERIVATIVE'`. Observed examples none recorded.

H-C139 `komorebi_math.integration.adapters.analysis_synthesis_integration_certificate`; `integration/adapters.py:372`.

Validity `valid`; status `_weakest_exactness(Exactness.NUMERICAL, system_certificate.exactness, measure_certificate.exactness)`; tolerance `threshold`. Label expression `'INT.ADAPTER.FRAME'`. Observed examples none recorded.

H-C140 `komorebi_math.integration.coordinates.PolynomialIntegrationCovector.certificate`; `integration/coordinates.py:183`.

Validity `structure.valid`; status `status`; tolerance `None`. Label expression `'INT.COORD.COVECTOR'`. Observed examples none recorded.

H-C141 `komorebi_math.integration.coordinates.PolynomialIntegrationCovector.certificate`; `integration/coordinates.py:202`.

Validity `True`; status `Exactness.EXACT`; tolerance `None`. Label expression `'INT.COORD.COVECTOR'`. Observed examples `EX-E-005`, `EX-E-007`.

H-C142 `komorebi_math.integration.coordinates.coordinate_fundamental_theorem`; `integration/coordinates.py:297`.

Validity `valid`; status `Exactness.EXACT`; tolerance `0.0`. Label expression `'INT.COORD.FTC'`. Observed examples none recorded.

H-C143 `komorebi_math.integration.coordinates.coordinate_transport_identity`; `integration/coordinates.py:340`.

Validity `valid`; status `Exactness.EXACT`; tolerance `0.0`. Label expression `'INT.COORD.TRANSPORT'`. Observed examples `EX-E-005`, `EX-E-007`.

H-C144 `komorebi_math.integration.identities._identity_certificate`; `integration/identities.py:59`.

Validity `valid`; status `Exactness.EXACT`; tolerance `0.0`. Label expression `theorem_id`. Observed examples `EX-E-005`, `EX-E-007`.

H-C145 `komorebi_math.integration.measures.ExactAffineMap.certificate`; `integration/measures.py:288`.

Validity `True`; status `Exactness.EXACT`; tolerance `None`. Label expression `'INT.MAP.1'`. Observed examples `EX-E-005`, `EX-E-007`.

H-C146 `komorebi_math.integration.measures.AtomicMeasure.mass`; `integration/measures.py:364`.

Validity `True`; status `self.exactness`; tolerance `None`. Label expression `'INT.MEASURE.ATOM.MASS'`. Observed examples none recorded.

H-C147 `komorebi_math.integration.measures.AtomicMeasure.moment`; `integration/measures.py:410`.

Validity `True`; status `self.exactness`; tolerance `None`. Label expression `'INT.MEASURE.ATOM.MOMENT'`. Observed examples none recorded.

H-C148 `komorebi_math.integration.measures.AtomicMeasure.certificate`; `integration/measures.py:471`.

Validity `True`; status `self.exactness`; tolerance `None`. Label expression `'INT.MEASURE.ATOM.1'`. Observed examples none recorded.

H-C149 `komorebi_math.integration.measures.IntervalMeasure.mass`; `integration/measures.py:549`.

Validity `True`; status `self.exactness`; tolerance `None`. Label expression `'INT.MEASURE.INTERVAL.MASS'`. Observed examples `EX-E-005`, `EX-E-007`.

H-C150 `komorebi_math.integration.measures.IntervalMeasure.moment`; `integration/measures.py:585`.

Validity `True`; status `self.exactness`; tolerance `None`. Label expression `'INT.MEASURE.INTERVAL.MOMENT'`. Observed examples `EX-E-005`, `EX-E-007`.

H-C151 `komorebi_math.integration.measures.IntervalMeasure.certificate`; `integration/measures.py:678`.

Validity `True`; status `self.exactness`; tolerance `None`. Label expression `'INT.MEASURE.INTERVAL.1'`. Observed examples `EX-E-005`, `EX-E-007`.

H-C152 `komorebi_math.integration.measures.ProductMeasure.mass`; `integration/measures.py:742`.

Validity `valid`; status `exactness`; tolerance `None`. Label expression `'INT.MEASURE.PRODUCT.MASS'`. Observed examples none recorded.

H-C153 `komorebi_math.integration.measures.ProductMeasure.moment`; `integration/measures.py:794`.

Validity `valid`; status `exactness`; tolerance `None`. Label expression `'INT.MEASURE.PRODUCT.MOMENT'`. Observed examples none recorded.

H-C154 `komorebi_math.integration.measures.ProductMeasure.certificate`; `integration/measures.py:829`.

Validity `valid`; status `_aggregate_exactness((self.exactness, *(certificate.exactness for certificate in certificates)))`; tolerance `None`. Label expression `'INT.MEASURE.PRODUCT.1'`. Observed examples none recorded.

H-C155 `komorebi_math.integration.measures.PushforwardMeasure.mass`; `integration/measures.py:917`.

Validity `valid`; status `_aggregate_exactness((self.exactness, *(certificate.exactness for certificate in certificates)))`; tolerance `None`. Label expression `'INT.MEASURE.PUSH.MASS'`. Observed examples none recorded.

H-C156 `komorebi_math.integration.measures.PushforwardMeasure.moment`; `integration/measures.py:991`.

Validity `valid`; status `_aggregate_exactness((certificate.exactness for certificate in certificates))`; tolerance `None`. Label expression `'INT.MEASURE.PUSH.MOMENT'`. Observed examples none recorded.

H-C157 `komorebi_math.integration.measures.PushforwardMeasure.certificate`; `integration/measures.py:1024`.

Validity `valid`; status `_aggregate_exactness((self.exactness, *(certificate.exactness for certificate in certificates)))`; tolerance `None`. Label expression `'INT.MEASURE.PUSH.1'`. Observed examples `EX-E-005`, `EX-E-007`.

H-C158 `komorebi_math.integration.polynomial.ExactPolynomial.certificate`; `integration/polynomial.py:344`.

Validity `True`; status `Exactness.EXACT`; tolerance `None`. Label expression `'INT.POLY.1'`. Observed examples none recorded.

H-C159 `komorebi_math.integration.quadrature.DerivativeBound.assumed`; `integration/quadrature.py:205`.

Validity `True`; status `Exactness.CONDITIONAL`; tolerance `None`. Label expression `'INT.QUAD.BOUND.ASSUMED'`. Observed examples none recorded.

H-C160 `komorebi_math.integration.quadrature.polynomial_derivative_bound`; `integration/quadrature.py:269`.

Validity `True`; status `Exactness.EXACT`; tolerance `None`. Label expression `'INT.QUAD.POLY.BOUND'`. Observed examples `EX-E-005`, `EX-E-007`.

H-C161 `komorebi_math.integration.quadrature._atomic_integral`; `integration/quadrature.py:425`.

Validity `True`; status `Exactness.EXACT if exact else Exactness.NUMERICAL`; tolerance `None`. Label expression `'INT.EVAL.ATOM'`. Observed examples none recorded.

H-C162 `komorebi_math.integration.quadrature._inherit_measure_assumptions`; `integration/quadrature.py:485`.

Validity `result.valid and structure.valid`; status `exactness`; tolerance `result.certificate.tolerance`. Label expression `theorem_id`. Observed examples `EX-E-005`, `EX-E-007`.

H-C163 `komorebi_math.integration.quadrature._zero_interval_result`; `integration/quadrature.py:558`.

Validity `True`; status `Exactness.EXACT`; tolerance `None`. Label expression `'INT.EVAL.ZERO.MEASURE'`. Observed examples none recorded.

H-C164 `komorebi_math.integration.quadrature._simpson_integral`; `integration/quadrature.py:646`.

Validity `True`; status `exactness`; tolerance `None`. Label expression `'INT.QUAD.SIMPSON'`. Observed examples none recorded.

H-C165 `komorebi_math.integration.quadrature._simpson_polynomial_integral`; `integration/quadrature.py:738`.

Validity `True`; status `exactness`; tolerance `None`. Label expression `'INT.QUAD.SIMPSON.EXACT.POLY'`. Observed examples `EX-E-005`, `EX-E-007`.

H-C166 `komorebi_math.integration.quadrature._product_integral`; `integration/quadrature.py:863`.

Validity `True`; status `result_exactness`; tolerance `None`. Label expression `'INT.EVAL.ATOMIC.PRODUCT'`. Observed examples none recorded.

H-C167 `komorebi_math.integration.quadrature._product_integral`; `integration/quadrature.py:893`.

Validity `True`; status `Exactness.HEURISTIC`; tolerance `None`. Label expression `'INT.QUAD.TENSOR.SIMPSON'`. Observed examples none recorded.

H-C168 `komorebi_math.integration.quadrature._exact_polynomial_integral`; `integration/quadrature.py:937`.

Validity `result.valid`; status `Exactness.CONDITIONAL if unchecked else result.certificate.exactness`; tolerance `None`. Label expression `'INT.PUSH.ATOMIC.SNAPSHOT.POLY'`. Observed examples none recorded.

H-C169 `komorebi_math.integration.quadrature._exact_polynomial_integral`; `integration/quadrature.py:970`.

Validity `base.valid`; status `base.certificate.exactness`; tolerance `None`. Label expression `'INT.PUSH.POLY'`. Observed examples `EX-E-005`, `EX-E-007`.

H-C170 `komorebi_math.integration.quadrature._exact_polynomial_integral`; `integration/quadrature.py:1066`.

Validity `True`; status `exactness`; tolerance `None`. Label expression `'INT.EVAL.POLY.MOMENTS'`. Observed examples `EX-E-005`, `EX-E-007`.

H-C171 `komorebi_math.integration.quadrature.integrate_polynomial_oriented`; `integration/quadrature.py:1195`.

Validity `True`; status `Exactness.EXACT`; tolerance `None`. Label expression `'INT.EVAL.ORIENTED.POLY'`. Observed examples none recorded.

H-C172 `komorebi_math.integration.rules.QuadratureRule.apply`; `integration/rules.py:243`.

Validity `True`; status `Exactness.EXACT if exact_evaluation else Exactness.NUMERICAL`; tolerance `None`. Label expression `'INT.QUAD.RULE.APPLY'`. Observed examples none recorded.

H-C173 `komorebi_math.integration.rules.QuadratureRule.integrate_polynomial`; `integration/rules.py:311`.

Validity `True`; status `Exactness.EXACT if exact_evaluation else Exactness.NUMERICAL`; tolerance `None`. Label expression `'INT.QUAD.RULE.POLY'`. Observed examples `EX-E-005`, `EX-E-007`.

H-C174 `komorebi_math.integration.rules.QuadratureRule.degree_of_exactness`; `integration/rules.py:435`.

Validity `True`; status `certificate_exactness`; tolerance `0.0 if exact_comparison else threshold`. Label expression `'INT.QUAD.RULE.DEGREE'`. Observed examples `EX-E-005`, `EX-E-007`.

H-C175 `komorebi_math.integration.rules.QuadratureRule.certify_polynomial_integral`; `integration/rules.py:470`.

Validity `True`; status `Exactness.EXACT`; tolerance `0.0`. Label expression `'INT.QUAD.RULE.CERTIFIED.POLY'`. Observed examples none recorded.

H-C176 `komorebi_math.integration.rules.QuadratureRule.certificate`; `integration/rules.py:492`.

Validity `True`; status `Exactness.EXACT if self.exact_data else Exactness.NUMERICAL`; tolerance `None`. Label expression `'INT.QUAD.RULE.1'`. Observed examples none recorded.

Construction group: langlands

H-C177 `komorebi_math.langlands.completion.gates.AutomorphicCompletionReport.result`; `langlands/completion/gates.py:69`.

Validity `self.complete`; status `Exactness.CONDITIONAL`; tolerance `None`. Label expression `'PSC-XII-FIVE-GATE-COMPLETION'`. Observed examples `EX-E-044`.

H-C178 `komorebi_math.langlands.completion.gates.CompatiblePropagationReport.result`; `langlands/completion/gates.py:103`.

Validity `complete`; status `Exactness.CONDITIONAL`; tolerance `None`. Label expression `'PSC-XII-ONE-MEMBER-PROPAGATION'`. Observed examples none recorded.

H-C179 `komorebi_math.langlands.finite_image.characters.InducedRepresentation.certificate`; `langlands/finite_image/characters.py:104`.

Validity `maximum <= tolerance`; status `Exactness.NUMERICAL`; tolerance `tolerance`. Label expression `'KM-INDUCED-REPRESENTATION'`. Observed examples `EX-E-020`.

H-C180 `komorebi_math.langlands.finite_image.clifford.CliffordTower.report`; `langlands/finite_image/clifford.py:121`.

Validity `True`; status `Exactness.EXACT`; tolerance `None`. Label expression `'PSC-LANG-CLIFFORD-DEPTH'`. Observed examples `EX-E-020`.

H-C181 `komorebi_math.langlands.finite_image.clifford.CliffordTower.automorphic_plan`; `langlands/finite_image/clifford.py:147`.

Validity `not unchecked`; status `Exactness.CONDITIONAL`; tolerance `None`. Label expression `'PSC-LANG-SOLVABLE-COMPLETION'`. Observed examples none recorded.

H-C182 `komorebi_math.langlands.finite_image.groups.FiniteGroup.certificate`; `langlands/finite_image/groups.py:158`.

Validity `True`; status `Exactness.EXACT`; tolerance `None`. Label expression `'KM-FINITE-GROUP'`. Observed examples none recorded.

H-C183 `komorebi_math.langlands.finite_image.mackey.mackey_tensor_packets`; `langlands/finite_image/mackey.py:60`.

Validity `total == expected`; status `Exactness.EXACT`; tolerance `None`. Label expression `'PSC-LANG-MACKEY-TENSOR'`. Observed examples none recorded.

H-C184 `komorebi_math.langlands.finite_image.orbits.MonomialAction.packet_certificate`; `langlands/finite_image/orbits.py:157`.

Validity `valid`; status `Exactness.EXACT`; tolerance `None`. Label expression `'PSC-LANG-MONOMIAL-ORBITS'`. Observed examples `EX-E-020`.

H-C185 `komorebi_math.langlands.finite_image.plans.AutomorphicInductionPlan.operations`; `langlands/finite_image/plans.py:32`.

Validity `True`; status `Exactness.CONDITIONAL`; tolerance `None`. Label expression `'PSC-LANG-AUTOMORPHIC-INDUCTION-PLAN'`. Observed examples none recorded.

H-C186 `komorebi_math.langlands.global_.dyadic.dyadic_sign_diagnostic`; `langlands/global_/dyadic.py:32`.

Validity `square_zero`; status `Exactness.EXACT`; tolerance `None`. Label expression `'PSC-XII-DYADIC-SIGN'`. Observed examples none recorded.

H-C187 `komorebi_math.langlands.global_.hecke.FiniteHeckePackets.certificate`; `langlands/global_/hecke.py:87`.

Validity `sum(ranks.values()) == self.dimension`; status `Exactness.NUMERICAL`; tolerance `self.tolerance`. Label expression `'PSC-XII-HECKE-PACKETS'`. Observed examples none recorded.

H-C188 `komorebi_math.langlands.global_.hecke.SupportComparison.construct`; `langlands/global_/hecke.py:126`.

Validity `valid`; status `Exactness.EXACT`; tolerance `None`. Label expression `'PSC-XII-SUPPORT-COMPARISON'`. Observed examples none recorded.

H-C189 `komorebi_math.langlands.global_.patching.PatchingSupportReport.certificate`; `langlands/global_/patching.py:58`.

Validity `self.complete`; status `Exactness.CONDITIONAL`; tolerance `None`. Label expression `'PSC-XII-PATCHING-SUPPORT'`. Observed examples none recorded.

H-C190 `komorebi_math.langlands.local.bridge.SymplecticBridge.construct`; `langlands/local/bridge.py:50`.

Validity `valid`; status `Exactness.EXACT`; tolerance `None`. Label expression `'PSC-XI-RESTRICTED-BRIDGE'`. Observed examples none recorded.

H-C191 `komorebi_math.langlands.local.envelopes.ProjectiveIntertwinerEnvelope.recover`; `langlands/local/envelopes.py:91`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `self.tolerance`. Label expression `'PSC-XI-PROJECTIVE-ENVELOPE'`. Observed examples none recorded.

H-C192 `komorebi_math.langlands.local.gauss.gauss_sum`; `langlands/local/gauss.py:67`.

Validity `residual <= tolerance`; status `Exactness.NUMERICAL`; tolerance `tolerance`. Label expression `'PSC-XI-GAUSS'`. Observed examples none recorded.

H-C193 `komorebi_math.langlands.local.heisenberg.HeisenbergModel.group_law_certificate`; `langlands/local/heisenberg.py:115`.

Validity `residual <= tolerance`; status `Exactness.NUMERICAL`; tolerance `tolerance`. Label expression `'PSC-XI-HEISENBERG-LAW'`. Observed examples `EX-E-056`.

H-C194 `komorebi_math.langlands.local.heisenberg.HeisenbergModel.trace_support_certificate`; `langlands/local/heisenberg.py:139`.

Validity `maximum <= tolerance`; status `Exactness.NUMERICAL`; tolerance `tolerance`. Label expression `'PSC-XI-HEISENBERG-TRACE'`. Observed examples none recorded.

H-C195 `komorebi_math.langlands.local.heisenberg.HeisenbergModel.selector_certificate`; `langlands/local/heisenberg.py:163`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `tolerance`. Label expression `'PSC-XI-ISOTROPIC-SELECTOR'`. Observed examples `EX-E-056`.

H-C196 `komorebi_math.langlands.local.symplectic.AlternatingForm.certificate`; `langlands/local/symplectic.py:92`.

Validity `valid`; status `Exactness.EXACT`; tolerance `None`. Label expression `'PSC-XI-SYMPLECTIC-ENVELOPE'`. Observed examples none recorded.

H-C197 `komorebi_math.langlands.local.symplectic.QuadraticRefinement.polar_certificate`; `langlands/local/symplectic.py:143`.

Validity `maximum == 0`; status `Exactness.EXACT`; tolerance `None`. Label expression `'PSC-XI-QUADRATIC-REFINEMENT'`. Observed examples none recorded.

H-C198 `komorebi_math.langlands.local.tame.FrobeniusOrbit.certificate`; `langlands/local/tame.py:109`.

Validity `valid`; status `Exactness.EXACT`; tolerance `None`. Label expression `'PSC-XI-TAME-ORBIT'`. Observed examples `EX-E-056`.

H-C199 `komorebi_math.langlands.local.tame.TameWeilPacket.relation_certificate`; `langlands/local/tame.py:178`.

Validity `residual <= tolerance`; status `Exactness.NUMERICAL`; tolerance `tolerance`. Label expression `'PSC-XI-TAME-ATOM'`. Observed examples `EX-E-056`.

H-C200 `komorebi_math.langlands.local.tame.tame_tensor_refinement`; `langlands/local/tame.py:212`.

Validity `count * depth == n * m`; status `Exactness.EXACT`; tolerance `None`. Label expression `'PSC-XI-TAME-TENSOR'`. Observed examples none recorded.

H-C201 `komorebi_math.langlands.local.tame.SpecialMonodromyBlock.certificate`; `langlands/local/tame.py:259`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `tolerance`. Label expression `'PSC-XI-MONODROMY'`. Observed examples none recorded.

H-C202 `komorebi_math.langlands.ramification.towers.RamificationCliffordTower.report`; `langlands/ramification/towers.py:143`.

Validity `True`; status `Exactness.CONDITIONAL`; tolerance `None`. Label expression `'PSC-LANG-RAMIFICATION-TOWER'`. Observed examples `EX-E-044`.

Construction group: nonlinear

H-C203 `komorebi_math.nonlinear.autodiff.AutomaticDifferentiationProvider.validation_certificate`; `nonlinear/autodiff.py:354`.

Validity `valid`; status `self.exactness`; tolerance `None if self.exactness is Exactness.EXACT else threshold`. Label expression `'PSC-III-N04A'`. Observed examples none recorded.

H-C204 `komorebi_math.nonlinear.autodiff._FrameworkProviderBase.validation_certificate`; `nonlinear/autodiff.py:448`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `threshold`. Label expression `'PSC-III-N04A'`. Observed examples none recorded.

H-C205 `komorebi_math.nonlinear.composition.faa_di_bruno`; `nonlinear/composition.py:204`.

Validity `valid`; status `exactness`; tolerance `None if exactness is Exactness.EXACT else tolerance`. Label expression `'PSC-III-T10'`. Observed examples `EX-E-031`.

H-C206 `komorebi_math.nonlinear.composition.packet_faa_di_bruno`; `nonlinear/composition.py:364`.

Validity `valid`; status `exactness`; tolerance `None if exactness is Exactness.EXACT else tol`. Label expression `'PSC-III-T11A'`. Observed examples none recorded.

H-C207 `komorebi_math.nonlinear.coordinates.NonlinearPacketMap.certificate`; `nonlinear/coordinates.py:72`.

Validity `valid`; status `Exactness.EXACT if self.source.backend.exact else Exactness.NUMERICAL`; tolerance `None if self.source.backend.exact else tolerance`. Label expression `'PSC-III-T02'`. Observed examples none recorded.

H-C208 `komorebi_math.nonlinear.coordinates.NonlinearPacketMap.fixed_point_residual`; `nonlinear/coordinates.py:99`.

Validity `True`; status `Exactness.EXACT if self.source.backend.exact else Exactness.NUMERICAL`; tolerance `None`. Label expression `'PSC-III-C03'`. Observed examples none recorded.

H-C209 `komorebi_math.nonlinear.correction.HilbertObstructionProjector.solve`; `nonlinear/correction.py:112`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `self.tolerance * 10`. Label expression `'PSC-III-T31'`. Observed examples none recorded.

H-C210 `komorebi_math.nonlinear.correction.RightInverseCorrectionProblem.hypothesis_certificate`; `nonlinear/correction.py:213`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `self.tolerance * 10`. Label expression `'PSC-III-T32'`. Observed examples none recorded.

H-C211 `komorebi_math.nonlinear.correction.RightInverseCorrectionProblem.solve`; `nonlinear/correction.py:266`.

Validity `converged`; status `Exactness.NUMERICAL`; tolerance `self.tolerance * 20`. Label expression `'PSC-III-T32'`. Observed examples none recorded.

H-C212 `komorebi_math.nonlinear.correction.LyapunovSchmidtReduction.__init__`; `nonlinear/correction.py:384`.

Validity `restricted_inverse_residual <= self.tolerance * 10`; status `Exactness.NUMERICAL`; tolerance `self.tolerance * 10`. Label expression `'PSC-III-T33'`. Observed examples `EX-E-032`.

H-C213 `komorebi_math.nonlinear.correction.LyapunovSchmidtReduction.solve_range`; `nonlinear/correction.py:467`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `self.tolerance * 10`. Label expression `'PSC-III-T33'`. Observed examples `EX-E-032`.

H-C214 `komorebi_math.nonlinear.correction.LyapunovSchmidtReduction.base_derivative_certificate`; `nonlinear/correction.py:491`.

Validity `residual <= self.tolerance * 10`; status `Exactness.NUMERICAL`; tolerance `self.tolerance * 10`. Label expression `'PSC-III-T33'`. Observed examples none recorded.

H-C215 `komorebi_math.nonlinear.correction.LyapunovSchmidtReduction.reduced_second_derivative`; `nonlinear/correction.py:517`.

Validity `True`; status `Exactness.NUMERICAL`; tolerance `self.tolerance`. Label expression `'PSC-III-C34'`. Observed examples none recorded.

H-C216 `komorebi_math.nonlinear.defects.NonlinearRefinementDefect.composition_identity`; `nonlinear/defects.py:78`.

Validity `residual <= 1e-10`; status `Exactness.NUMERICAL`; tolerance `1e-10`. Label expression `'PSC-III-T11'`. Observed examples none recorded.

H-C217 `komorebi_math.nonlinear.derivatives.CallableDerivativeProvider.validation_certificate`; `nonlinear/derivatives.py:227`.

Validity `valid`; status `self.exactness`; tolerance `None if self.exactness is Exactness.EXACT else tolerance`. Label expression `'PSC-III-D02'`. Observed examples none recorded.

H-C218 `komorebi_math.nonlinear.derivatives.PolynomialDerivativeProvider.symmetry_certificate`; `nonlinear/derivatives.py:464`.

Validity `valid`; status `self.exactness`; tolerance `None if self.exactness is Exactness.EXACT else self.tolerance`. Label expression `'PSC-III-D04'`. Observed examples none recorded.

H-C219 `komorebi_math.nonlinear.elimination.NonlinearSchurElimination.first_derivatives`; `nonlinear/elimination.py:280`.

Validity `valid`; status `self.exactness`; tolerance `None if self.exact else self.tolerance * 10`. Label expression `'PSC-III-T41'`. Observed examples `EX-E-032`.

H-C220 `komorebi_math.nonlinear.elimination.NonlinearSchurElimination.second_derivative`; `nonlinear/elimination.py:348`.

Validity `valid`; status `self.exactness`; tolerance `None if self.exact else self.tolerance * 10`. Label expression `'PSC-III-T42'`. Observed examples none recorded.

H-C221 `komorebi_math.nonlinear.elimination.NonlinearSchurElimination.solve_hidden`; `nonlinear/elimination.py:445`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `self.tolerance * 10`. Label expression `'PSC-III-T40'`. Observed examples none recorded.

H-C222 `komorebi_math.nonlinear.elimination.NonlinearSchurElimination.reduced_value`; `nonlinear/elimination.py:477`.

Validity `True`; status `Exactness.NUMERICAL`; tolerance `self.tolerance * 10`. Label expression `'PSC-III-T40'`. Observed examples none recorded.

H-C223 `komorebi_math.nonlinear.execution.compile_packet_jet_plan`; `nonlinear/execution.py:166`.

Validity `valid`; status `family.exactness`; tolerance `None if family.exactness is Exactness.EXACT else max(family.source.tolerance, family.ta...`. Label expression `'PSC-III-A10'`. Observed examples `EX-E-002`, `EX-E-049`, `EX-E-050`.

H-C224 `komorebi_math.nonlinear.execution.execute_packet_jet_plan`; `nonlinear/execution.py:314`.

Validity `valid`; status `family.exactness`; tolerance `None if family.exactness is Exactness.EXACT else tol`. Label expression `'PSC-III-A10'`. Observed examples `EX-E-002`, `EX-E-049`, `EX-E-050`.

H-C225 `komorebi_math.nonlinear.execution.CupyPacketJetExecutor.execute`; `nonlinear/execution.py:579`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `tol`. Label expression `'PSC-III-A10G'`. Observed examples none recorded.

H-C226 `komorebi_math.nonlinear.execution.CupyPacketJetExecutor.hardware_certificate`; `nonlinear/execution.py:628`.

Validity `True`; status `Exactness.NUMERICAL`; tolerance `None`. Label expression `'PSC-III-A10G'`. Observed examples none recorded.

H-C227 `komorebi_math.nonlinear.filtered.FilteredVectorSpace.certificate`; `nonlinear/filtered.py:286`.

Validity `True`; status `Exactness.EXACT`; tolerance `None`. Label expression `'PSC-III-D50'`. Observed examples none recorded.

H-C228 `komorebi_math.nonlinear.filtered.FilteredCorrectionProblem.solve`; `nonlinear/filtered.py:397`.

Validity `valid`; status `Exactness.EXACT`; tolerance `None`. Label expression `'PSC-III-T51'`. Observed examples none recorded.

H-C229 `komorebi_math.nonlinear.filtered.verify_finite_filtered_branch`; `nonlinear/filtered.py:476`.

Validity `valid`; status `Exactness.EXACT`; tolerance `None`. Label expression `'PSC-III-T52'`. Observed examples none recorded.

H-C230 `komorebi_math.nonlinear.filtered.conditional_projective_limit_completion`; `nonlinear/filtered.py:546`.

Validity `valid`; status `Exactness.CONDITIONAL`; tolerance `None`. Label expression `'PSC-III-T53'`. Observed examples none recorded.

H-C231 `komorebi_math.nonlinear.filtered.FiniteFilteredCochainComplex.page`; `nonlinear/filtered.py:767`.

Validity `relation_failures == 0`; status `Exactness.EXACT`; tolerance `None`. Label expression `'PSC-III-S54'`. Observed examples `EX-E-019`.

H-C232 `komorebi_math.nonlinear.filtered.FiniteFilteredCochainComplex.spectral_sequence`; `nonlinear/filtered.py:809`.

Validity `valid`; status `Exactness.EXACT`; tolerance `None`. Label expression `'PSC-III-S54'`. Observed examples `EX-E-019`.

H-C233 `komorebi_math.nonlinear.gpu.validate_cupy_snapshot`; `nonlinear/gpu.py:165`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `tol`. Label expression `'PSC-III-N59'`. Observed examples none recorded.

H-C234 `komorebi_math.nonlinear.graded.FiniteAbelianGroup.certificate`; `nonlinear/graded.py:165`.

Validity `valid`; status `Exactness.EXACT`; tolerance `None`. Label expression `'PSC-III-D17'`. Observed examples none recorded.

H-C235 `komorebi_math.nonlinear.graded.GradedPacketOperation.grading_certificate`; `nonlinear/graded.py:302`.

Specialized/wrapper construction `komorebi_math.nonlinear.graded.GradingCertificate`. Fields: `arity`, `certificate`, `forbidden_residual`, `tested_input_labels`. No base-constructor execution claim is inferred for this wrapper.

H-C236 `komorebi_math.nonlinear.graded.GradedPacketOperation.grading_certificate`; `nonlinear/graded.py:306`.

Validity `valid`; status `exactness`; tolerance `None if exactness is Exactness.EXACT else self.tolerance`. Label expression `'PSC-III-T18'`. Observed examples none recorded.

H-C237 `komorebi_math.nonlinear.graded.GradedPacketOperation.packet_convolution`; `nonlinear/graded.py:362`.

Validity `valid`; status `exactness`; tolerance `None if exactness is Exactness.EXACT else self.tolerance`. Label expression `'PSC-III-T18'`. Observed examples none recorded.

H-C238 `komorebi_math.nonlinear.graded.maurer_cartan_packets`; `nonlinear/graded.py:454`.

Validity `valid`; status `exactness`; tolerance `None if exact else tolerance`. Label expression `'PSC-III-T19'`. Observed examples none recorded.

H-C239 `komorebi_math.nonlinear.graded.higher_maurer_cartan_packets`; `nonlinear/graded.py:540`.

Validity `valid`; status `exactness`; tolerance `None if exact else tolerance`. Label expression `'PSC-III-P20'`. Observed examples none recorded.

H-C240 `komorebi_math.nonlinear.graded.divided_square_packets`; `nonlinear/graded.py:608`.

Validity `valid`; status `exactness`; tolerance `None if exact else tolerance`. Label expression `'PSC-III-P21'`. Observed examples none recorded.

H-C241 `komorebi_math.nonlinear.holonomy.TransportGraph.gluing_certificate`; `nonlinear/holonomy.py:234`.

Validity `bool(valid)`; status `Exactness.EXACT if self.backend.exact else Exactness.NUMERICAL`; tolerance `None if self.backend.exact else self.tolerance`. Label expression `'PSC-III-T40'`. Observed examples `EX-E-025`.

H-C242 `komorebi_math.nonlinear.holonomy.TransportGraph.intertwine`; `nonlinear/holonomy.py:308`.

Validity `bool(valid)`; status `Exactness.EXACT if self.backend.exact else Exactness.NUMERICAL`; tolerance `None if self.backend.exact else self.tolerance`. Label expression `'PSC-III-T41'`. Observed examples none recorded.

H-C243 `komorebi_math.nonlinear.jets.PacketJet.symmetry_certificate`; `nonlinear/jets.py:180`.

Validity `valid`; status `self.exactness`; tolerance `None if self.exactness is Exactness.EXACT else tol`. Label expression `'PSC-III-D06'`. Observed examples none recorded.

H-C244 `komorebi_math.nonlinear.jets.PacketJetFamily.reconstruction_certificate`; `nonlinear/jets.py:295`.

Validity `valid`; status `self.exactness`; tolerance `None if self.exactness is Exactness.EXACT else tol`. Label expression `'PSC-III-T07'`. Observed examples `EX-E-031`.

H-C245 `komorebi_math.nonlinear.jets.packet_jet_support`; `nonlinear/jets.py:408`.

Validity `True`; status `exactness`; tolerance `None if family.exactness is Exactness.EXACT else tol`. Label expression `'PSC-III-C08'`. Observed examples `EX-E-002`, `EX-E-031`, `EX-E-049`, `EX-E-050`.

H-C246 `komorebi_math.nonlinear.jets.packet_taylor`; `nonlinear/jets.py:492`.

Validity `representation_valid`; status `exactness`; tolerance `None if exactness is Exactness.EXACT else max(source.tolerance, target.tolerance) if to...`. Label expression `'PSC-III-C09'`. Observed examples none recorded.

H-C247 `komorebi_math.nonlinear.lifting.LinearLiftProblem.step`; `nonlinear/lifting.py:214`.

Validity `bool(valid)`; status `Exactness.EXACT`; tolerance `None`. Label expression `'PSC-III-T28'`. Observed examples none recorded.

H-C248 `komorebi_math.nonlinear.lifting.PolynomialLiftProblem.step`; `nonlinear/lifting.py:339`.

Validity `bool(children_valid)`; status `exactness`; tolerance `None`. Label expression `'PSC-III-T28'`. Observed examples `EX-E-033`.

H-C249 `komorebi_math.nonlinear.obstruction.LinearObstructionProblem._solve_exact`; `nonlinear/obstruction.py:99`.

Validity `bool(valid and residual == 0.0)`; status `Exactness.EXACT`; tolerance `None`. Label expression `'PSC-III-T14'`. Observed examples `EX-E-033`.

H-C250 `komorebi_math.nonlinear.obstruction.LinearObstructionProblem._solve_numerical`; `nonlinear/obstruction.py:130`.

Validity `bool(valid)`; status `Exactness.NUMERICAL`; tolerance `self.tolerance`. Label expression `'PSC-III-T14'`. Observed examples none recorded.

H-C251 `komorebi_math.nonlinear.obstruction.ObstructionSquare.certificate`; `nonlinear/obstruction.py:180`.

Validity `valid`; status `Exactness.EXACT if algebra.exact else Exactness.NUMERICAL`; tolerance `None if algebra.exact else self.tolerance`. Label expression `'PSC-III-T15'`. Observed examples none recorded.

H-C252 `komorebi_math.nonlinear.profiling.estimate_packet_jet_complexity`; `nonlinear/profiling.py:214`.

Validity `valid`; status `Exactness.EXACT`; tolerance `None`. Label expression `'PSC-III-A61'`. Observed examples `EX-E-002`.

H-C253 `komorebi_math.nonlinear.profiling.profile_packet_jet_runtime`; `nonlinear/profiling.py:430`.

Validity `True`; status `Exactness.HEURISTIC`; tolerance `None`. Label expression `'PSC-III-A61'`. Observed examples `EX-E-002`.

H-C254 `komorebi_math.nonlinear.snapshots.PacketJetSnapshot.comparison_certificate`; `nonlinear/snapshots.py:246`.

Validity `valid`; status `self.exactness`; tolerance `None if self.exactness is Exactness.EXACT else tol`. Label expression `'PSC-III-D58'`. Observed examples `EX-E-037`, `EX-E-049`.

H-C255 `komorebi_math.nonlinear.snapshots.snapshot_packet_jet`; `nonlinear/snapshots.py:332`.

Validity `True`; status `jet.exactness`; tolerance `None if jet.exactness is Exactness.EXACT else tol`. Label expression `'PSC-III-D58'`. Observed examples `EX-E-002`, `EX-E-037`, `EX-E-049`.

H-C256 `komorebi_math.nonlinear.sparse.SparseSymmetricTensor.certificate`; `nonlinear/sparse.py:313`.

Validity `valid`; status `exactness`; tolerance `None if exactness is Exactness.EXACT else tol`. Label expression `'PSC-III-D04S'`. Observed examples none recorded.

H-C257 `komorebi_math.nonlinear.sparse.SparsePolynomialDerivativeProvider.storage_certificate`; `nonlinear/sparse.py:619`.

Validity `valid`; status `Exactness.EXACT if exact else Exactness.NUMERICAL`; tolerance `None if exact else tol`. Label expression `'PSC-III-D04P'`. Observed examples `EX-E-050`.

H-C258 `komorebi_math.nonlinear.streaming._write_result_certificate`; `nonlinear/streaming.py:966`.

Validity `True`; status `manifest.exactness if completed else Exactness.CONDITIONAL`; tolerance `None if manifest.exactness is Exactness.EXACT else manifest.tolerance`. Label expression `'PSC-III-D60'`. Observed examples `EX-E-002`, `EX-E-048`.

H-C259 `komorebi_math.nonlinear.streaming.capture_packet_jet_stream`; `nonlinear/streaming.py:1131`.

Validity `True`; status `jet.exactness`; tolerance `None if jet.exactness is Exactness.EXACT else tol`. Label expression `'PSC-III-D60'`. Observed examples `EX-E-048`.

H-C260 `komorebi_math.nonlinear.streaming.PacketJetSnapshotStreamReader.select`; `nonlinear/streaming.py:1441`.

Validity `len(coefficients) == upper - lower`; status `self.manifest.exactness`; tolerance `None if self.manifest.exactness is Exactness.EXACT else self.manifest.tolerance`. Label expression `'PSC-III-D60'`. Observed examples none recorded.

H-C261 `komorebi_math.nonlinear.streaming.PacketJetSnapshotStreamReader.evaluate`; `nonlinear/streaming.py:1508`.

Validity `processed == upper - lower`; status `self.manifest.exactness`; tolerance `None if self.manifest.exactness is Exactness.EXACT else self.manifest.tolerance`. Label expression `'PSC-III-D60'`. Observed examples `EX-E-002`, `EX-E-048`.

H-C262 `komorebi_math.nonlinear.streaming.PacketJetSnapshotStreamReader.verify`; `nonlinear/streaming.py:1584`.

Validity `valid`; status `Exactness.EXACT`; tolerance `None`. Label expression `'PSC-III-D60'`. Observed examples `EX-E-002`, `EX-E-048`.

H-C263 `komorebi_math.nonlinear.transport.direct_packet_rotation`; `nonlinear/transport.py:136`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `threshold`. Label expression `'PSC-III-T55'`. Observed examples none recorded.

H-C264 `komorebi_math.nonlinear.transport.MovingPacketSystem.derivative_certificate`; `nonlinear/transport.py:247`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `threshold`. Label expression `'PSC-III-D56'`. Observed examples `EX-E-019`.

H-C265 `komorebi_math.nonlinear.transport.MovingPacketSystem.generator_certificate`; `nonlinear/transport.py:289`.

Validity `max(skew, commutator) <= threshold`; status `Exactness.NUMERICAL`; tolerance `threshold`. Label expression `'PSC-III-T57'`. Observed examples none recorded.

H-C266 `komorebi_math.nonlinear.transport.KatoTransport.evolve`; `nonlinear/transport.py:356`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `tolerance`. Label expression `'PSC-III-T58'`. Observed examples `EX-E-019`, `EX-E-026`.

H-C267 `komorebi_math.nonlinear.transport.KatoTransport.direct_rotation_product`; `nonlinear/transport.py:399`.

Validity `residual <= threshold`; status `Exactness.NUMERICAL`; tolerance `threshold`. Label expression `'PSC-III-T59'`. Observed examples none recorded.

H-C268 `komorebi_math.nonlinear.transport.KatoTransport.refinement_certificate`; `nonlinear/transport.py:451`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `threshold`. Label expression `'PSC-III-C60'`. Observed examples `EX-E-019`.

H-C269 `komorebi_math.nonlinear.trees.majorant_convergence`; `nonlinear/trees.py:370`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `tol`. Label expression `'PSC-III-T14'`. Observed examples `EX-E-032`.

H-C270 `komorebi_math.nonlinear.trees.quadratic_catalan_majorant`; `nonlinear/trees.py:421`.

Validity `False`; status `Exactness.NUMERICAL`; tolerance `tolerance`. Label expression `'PSC-III-C15'`. Observed examples none recorded.

H-C271 `komorebi_math.nonlinear.trees.PolynomialFixedPointProblem.tree_expansion`; `nonlinear/trees.py:577`.

Validity `valid`; status `self.exactness`; tolerance `None if self.exactness is Exactness.EXACT else self.tolerance`. Label expression `'PSC-III-T14A'`. Observed examples `EX-E-032`.

H-C272 `komorebi_math.nonlinear.trees.packet_tree_reconstruction`; `nonlinear/trees.py:701`.

Validity `valid`; status `exactness`; tolerance `None if exactness is Exactness.EXACT else tol`. Label expression `'PSC-III-T16'`. Observed examples none recorded.

Construction group: persistence_contract.py

H-C273 `komorebi_math.persistence_contract.compare_portable_codec_contract`; `persistence_contract.py:268`.

Validity `valid`; status `Exactness.EXACT`; tolerance `None`. Label expression `'KM-PERSIST-016'`. Observed examples none recorded.

Construction group: qualification.py

H-C274 `komorebi_math.qualification.ReleaseQualificationReport.certificate`; `qualification.py:308`.

Validity `self.passed`; status `Exactness.NUMERICAL`; tolerance `None`. Label expression `'KM-REL-16'`. Observed examples none recorded.

Construction group: quantum

H-C275 `komorebi_math.quantum.channels.QuantumChannel.certificate`; `quantum/channels.py:138`.

Validity `residual <= self.tolerance and choi_eigenvalue >= -self.tolerance`; status `Exactness.NUMERICAL`; tolerance `self.tolerance`. Label expression `'PSC-VII-T31'`. Observed examples `EX-E-041`.

H-C276 `komorebi_math.quantum.channels.QuantumChannel.covariance_certificate`; `quantum/channels.py:240`.

Validity `residual <= self.tolerance`; status `Exactness.NUMERICAL`; tolerance `self.tolerance`. Label expression `'PSC-VII-T35'`. Observed examples none recorded.

H-C277 `komorebi_math.quantum.circuits.CircuitPacketExpansion.reconstruction_certificate`; `quantum/circuits.py:129`.

Validity `residual <= self.tolerance`; status `Exactness.NUMERICAL`; tolerance `self.tolerance`. Label expression `'PSC-VII-T14'`. Observed examples `EX-E-042`.

H-C278 `komorebi_math.quantum.circuits.CircuitPacketExpansion.interference`; `quantum/circuits.py:161`.

Validity `residual <= self.tolerance * max(1.0, probability)`; status `Exactness.NUMERICAL`; tolerance `self.tolerance * max(1.0, probability)`. Label expression `'PSC-VII-C17'`. Observed examples `EX-E-042`.

H-C279 `komorebi_math.quantum.circuits.CircuitPacketExpansion.block_unitarity_certificate`; `quantum/circuits.py:188`.

Validity `residual <= self.tolerance`; status `Exactness.NUMERICAL`; tolerance `self.tolerance`. Label expression `'PSC-VII-T15'`. Observed examples `EX-E-042`.

H-C280 `komorebi_math.quantum.constraints.constraint_closure`; `quantum/constraints.py:67`.

Validity `maximum <= tolerance * scale`; status `Exactness.NUMERICAL`; tolerance `tolerance * scale`. Label expression `'PSC-VII-T64'`. Observed examples none recorded.

H-C281 `komorebi_math.quantum.effective.QuantumFeshbachReduction.upper_half_plane_sign_certificate`; `quantum/effective.py:64`.

Validity `maximum <= self.tolerance`; status `Exactness.NUMERICAL`; tolerance `self.tolerance`. Label expression `'PSC-VII-P49'`. Observed examples none recorded.

H-C282 `komorebi_math.quantum.effective.QuantumFeshbachReduction.bare_closure_certificate; quantum/effective.py:87`.

Validity `coupling <= self.tolerance`; status `Exactness.NUMERICAL`; tolerance `self.tolerance`. Label expression `'PSC-VII-T53'`. Observed examples none recorded.

H-C283 `komorebi_math.quantum.error_correction.QuantumCode.knill_laflamme_report; quantum/error_correction.py:74`.

Validity `maximum <= self.tolerance * scale`; status `Exactness.NUMERICAL`; tolerance `self.tolerance * scale`. Label expression `'PSC-VII-T42'`. Observed examples `EX-E-041`.

H-C284 `komorebi_math.quantum.error_correction.QuantumCode.recovered_channel_certificate; quantum/error_correction.py:123`.

Validity `False`; status `Exactness.NUMERICAL`; tolerance `noise_certificate.tolerance`. Label expression `'PSC-VII-T44'`. Observed examples none recorded.

H-C285 `komorebi_math.quantum.error_correction.QuantumCode.recovered_channel_certificate; quantum/error_correction.py:141`.

Validity `residual <= max(self.tolerance * 100, 1e-08)`; status `Exactness.NUMERICAL`; tolerance `max(self.tolerance * 100, 1e-08)`. Label expression `'PSC-VII-T44'`. Observed examples `EX-E-041`.

H-C286 `komorebi_math.quantum.euclidean.EuclideanTransfer.reflection_certificate; quantum/euclidean.py:98`.

Validity `minimum >= -self.tolerance`; status `Exactness.NUMERICAL`; tolerance `self.tolerance`. Label expression `'PSC-VII-T56'`. Observed examples `EX-E-013`.

H-C287 `komorebi_math.quantum.leakage.fixed_packet_leakage; quantum/leakage.py:63`.

Validity `residual <= tolerance * max(1.0, observed, bound)`; status `Exactness.NUMERICAL`; tolerance `tolerance * max(1.0, observed, bound)`. Label expression `'PSC-VII-T18'`. Observed examples none recorded.

H-C288 `komorebi_math.quantum.leakage.moving_packet_leakage_bound; quantum/leakage.py:124`.

Validity `np.isfinite(value) and value >= 0`; status `Exactness.NUMERICAL`; tolerance `None`. Label expression `'PSC-VII-T22'`. Observed examples none recorded.

H-C289 `komorebi_math.quantum.lindblad.LindbladGenerator.trace_preservation_certificate; quantum/lindblad.py:84`.

Validity `residual <= self.tolerance`; status `Exactness.NUMERICAL`; tolerance `self.tolerance`. Label expression `'PSC-VII-T37'`. Observed examples none recorded.

H-C290 `komorebi_math.quantum.lindblad.LindbladGenerator.coherence_preservation_certificate; quantum/lindblad.py:110`.

Validity `residual <= self.tolerance * max(1.0, float(np.linalg.norm(generator, ord=2)))`; status `Exactness.NUMERICAL`; tolerance `self.tolerance * max(1.0, float(np.linalg.norm(generator, ord=2)))`. Label expression `'PSC-VII-T38'`. Observed examples none recorded.

H-C291 `komorebi_math.quantum.seeds.computational_basis_generation; quantum/seeds.py:150`.

Validity `residual <= 1e-12`; status `Exactness.NUMERICAL`; tolerance `1e-12`. Label expression `'PSC-VII-T04'`. Observed examples none recorded.

H-C292 `komorebi_math.quantum.symmetry.charge_certificate; quantum/symmetry.py:106`.

Validity `residual <= tolerance * scale`; status `Exactness.NUMERICAL`; tolerance `tolerance * scale`. Label expression `'PSC-VII-T11'`. Observed examples none recorded.

H-C293 `komorebi_math.quantum.symmetry.charge_selection_certificate; quantum/symmetry.py:137`.

Validity `residual <= tolerance * scale`; status `Exactness.NUMERICAL`; tolerance `tolerance * scale`. Label expression `'PSC-VII-T11'`. Observed examples none recorded.

H-C294 `komorebi_math.quantum.towers.BoundedHamiltonianTower.report; quantum/towers.py:61`.

Validity `violation <= self.tolerance * 20`; status `Exactness.NUMERICAL`; tolerance `self.tolerance * 20`. Label expression `'PSC-VII-T70'`. Observed examples `EX-E-013`.

Construction group: refinement

H-C295 `komorebi_math.refinement.defects.LinearRefinementDefect.power`; `refinement/defects.py:93`.

Validity `valid`; status `Exactness.EXACT if self.backend.exact else Exactness.NUMERICAL`; tolerance `None if self.backend.exact else tolerance`. Label expression `'PSC-II-T20'`. Observed examples none recorded.

H-C296 `komorebi_math.refinement.defects.LinearRefinementDefect.polynomial`; `refinement/defects.py:122`.

Validity `valid`; status `Exactness.EXACT if self.backend.exact else Exactness.NUMERICAL`; tolerance `None if self.backend.exact else 1e-10`. Label expression `'PSC-II-C21'`. Observed examples none recorded.

H-C297 `komorebi_math.refinement.defects.LinearRefinementDefect.resolvent`; `refinement/defects.py:157`.

Validity `valid`; status `Exactness.EXACT if self.backend.exact else Exactness.NUMERICAL`; tolerance `None if self.backend.exact else 1e-09`. Label expression `'PSC-II-T22'`. Observed examples none recorded.

H-C298 `komorebi_math.refinement.defects.LinearRefinementDefect.exponential`; `refinement/defects.py:210`.

Validity `valid`; status `Exactness.EXACT if self.backend.exact else Exactness.NUMERICAL`; tolerance `None if self.backend.exact else 1e-08`. Label expression `'PSC-II-T24'`. Observed examples none recorded.

H-C299 `komorebi_math.refinement.defects.multilevel_defect`; `refinement/defects.py:290`.

Validity `valid`; status `Exactness.EXACT if algebra.exact else Exactness.NUMERICAL`; tolerance `None if algebra.exact else 1e-09`. Label expression `'PSC-II-T25'`. Observed examples none recorded.

H-C300 `komorebi_math.refinement.lattices.LatticePacketData.certificate`; `refinement/lattices.py:140`.

Validity `bool(valid)`; status `Exactness.EXACT`; tolerance `None`. Label expression `'PSC-II-D12'`. Observed examples `EX-E-016`.

H-C301 `komorebi_math.refinement.lattices.LatticePacketData.gluing_weight_certificate`; `refinement/lattices.py:212`.

Validity `valid`; status `Exactness.EXACT`; tolerance `None`. Label expression `'PSC-II-T13'`. Observed examples `EX-E-016`.

H-C302 `komorebi_math.refinement.lattices.refinement_weight_monotonicity`; `refinement/lattices.py:274`.

Validity `valid`; status `Exactness.EXACT`; tolerance `None`. Label expression `'PSC-II-T15'`. Observed examples none recorded.

H-C303 `komorebi_math.refinement.polyphase.PolyphaseRefinement.certificate`; `refinement/polyphase.py:276`.

Validity `valid`; status `exactness`; tolerance `None if exact else threshold`. Label expression `'KM-B15-T01'`. Observed examples `EX-E-003`, `EX-E-007`.

H-C304 `komorebi_math.refinement.polyphase.PolyphaseRefinementTower.coherence_certificate`; `refinement/polyphase.py:626`.

Validity `valid`; status `Exactness.EXACT if exact else Exactness.NUMERICAL`; tolerance `None if exact else float(tolerance)`. Label expression `'KM-B15-T02'`. Observed examples `EX-E-003`.

H-C305 `komorebi_math.refinement.polyphase.LatticePolyphaseRefinement.certificate`; `refinement/polyphase.py:857`.

Validity `valid`; status `Exactness.EXACT if exact else Exactness.NUMERICAL`; tolerance `None if exact else float(tolerance)`. Label expression `'KM-B15-T03'`. Observed examples none recorded.

H-C306 `komorebi_math.refinement.smith.SmithDecomposition.certificate`; `refinement/smith.py:67`.

Validity `bool(valid)`; status `Exactness.EXACT`; tolerance `None`. Label expression `'PSC-II-SNF'`. Observed examples `EX-E-016`.

H-C307 `komorebi_math.refinement.smith.SmithCongruenceProblem.solve`; `refinement/smith.py:236`.

Validity `bool(valid)`; status `Exactness.EXACT`; tolerance `None`. Label expression `'PSC-II-T18'`. Observed examples `EX-E-016`.

H-C308 `komorebi_math.refinement.towers.PacketRefinement.certificate`; `refinement/towers.py:72`.

Validity `bool(valid)`; status `Exactness.EXACT if self.backend.exact else Exactness.NUMERICAL`; tolerance `None if self.backend.exact else max(self.coarse.tolerance, self.fine.tolerance)`. Label expression `'PSC-II-T03'`. Observed examples `EX-E-047`.

H-C309 `komorebi_math.refinement.towers.RefinementTower.certificate`; `refinement/towers.py:201`.

Validity `all((certificate.valid for certificate in certificates))`; status `Exactness.EXACT if all((certificate.exactness is Exactness.EXACT for certificate in certificates)) else Exactnes...`; tolerance `None if all((certificate.exactness is Exactness.EXACT for certificate in certificates))...`. Label expression `'PSC-II-T05'`. Observed examples none recorded.

H-C310 `komorebi_math.refinement.weighted.WeightedTransferGraph.truncated_inverse`; `refinement/weighted.py:296`.

Validity `bool(valid)`; status `exactness`; tolerance `None`. Label expression `'PSC-II-T10'`. Observed examples none recorded.

H-C311 `komorebi_math.refinement.weighted.WeightedTransferGraph.exact_inverse_if_acyclic`; `refinement/weighted.py:341`.

Validity `valid`; status `Exactness.EXACT if self.backend.exact else Exactness.NUMERICAL`; tolerance `None if self.backend.exact else self.packet_system.tolerance`. Label expression `'PSC-II-C11'`. Observed examples `EX-E-047`.

Construction group: release

H-C312 `komorebi_math.release.contract.check_release_contract`; `release/contract.py:91`.

Validity `not failed`; status `Exactness.EXACT`; tolerance `None`. Label expression `'KM-RELEASE-CONTRACT-021'`. Observed examples none recorded.

H-C313 `komorebi_math.release.documentation.check_stable_documentation`; `release/documentation.py:296`.

Validity `valid`; status `Exactness.EXACT`; tolerance `None`. Label expression `'KM-DOCS-017'`. Observed examples none recorded.

H-C314 `komorebi_math.release.evidence.check_validation_evidence`; `release/evidence.py:286`.

Validity `valid`; status `Exactness.EXACT`; tolerance `None`. Label expression `'KM-EVIDENCE-017'`. Observed examples none recorded.

H-C315 `komorebi_math.release.gpu.qualify_gpu`; `release/gpu.py:152`.

Validity `not failures`; status `Exactness.NUMERICAL`; tolerance `1e-10`. Label expression `'KM-GPU-016'`. Observed examples none recorded.

H-C316 `komorebi_math.release.readiness.check_beta_readiness`; `release/readiness.py:42`.

Validity `not failed`; status `Exactness.EXACT`; tolerance `None`. Label expression `'KM-READINESS-017'`. Observed examples none recorded.

H-C317 `komorebi_math.release.upgrade.check_upgrade_contract`; `release/upgrade.py:134`.

Validity `not failed`; status `Exactness.EXACT`; tolerance `None`. Label expression `'KM-UPGRADE-017'`. Observed examples none recorded.

Construction group: sequences

H-C318 `komorebi_math.sequences.automatic.AutomaticSequence.certificate`; `sequences/automatic.py:159`.

Validity `valid`; status `Exactness.EXACT`; tolerance `None`. Label expression `'PSC-SEQ-AUTOMATIC'`. Observed examples `EX-E-001`.

H-C319 `komorebi_math.sequences.compiler._model_certificate`; `sequences/compiler.py:190`.

Validity `valid`; status `Exactness.EXACT if exact else Exactness.CONDITIONAL`; tolerance `None`. Label expression `None`. Observed examples `EX-E-017`, `EX-E-051`.

H-C320 `komorebi_math.sequences.compiler.compile_sequence`; `sequences/compiler.py:265`.

Validity `True`; status `Exactness.CONDITIONAL`; tolerance `None`. Label expression `None`. Observed examples none recorded.

H-C321 `komorebi_math.sequences.compiler.compile_sequence`; `sequences/compiler.py:278`.

Validity `True`; status `Exactness.EXACT if sequence.exact else Exactness.NUMERICAL`; tolerance `None`. Label expression `None`. Observed examples none recorded.

H-C322 `komorebi_math.sequences.holonomic.HolonomicRecurrence.certificate`; `sequences/holonomic.py:139`.

Validity `valid`; status `Exactness.EXACT`; tolerance `None`. Label expression `'PSC-SEQ-HOLONOMIC'`. Observed examples `EX-E-001`.

Construction group: signals

H-C323 `komorebi_math.signals.adaptive._result`; `signals/adaptive.py:99`.

Validity `finite`; status `Exactness.NUMERICAL`; tolerance `None`. Label expression `'KM-DSP-D12'`. Observed examples `EX-E-040`.

H-C324 `komorebi_math.signals.approximation.estimate_positive_packet_1d`; `signals/approximation.py:104`.

Validity `True`; status `Exactness.HEURISTIC`; tolerance `None`. Label expression `'PSC-V-H01'`. Observed examples none recorded.

H-C325 `komorebi_math.signals.convolution.PolyphaseConvolution.certificate`; `signals/convolution.py:199`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `tolerance`. Label expression `'PSC-V-T07'`. Observed examples `EX-E-036`.

H-C326 `komorebi_math.signals.denoise.HaarDecomposition.certificate`; `signals/denoise.py:178`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `tolerance`. Label expression `'KM-DSP-T06'`. Observed examples `EX-E-040`.

H-C327 `komorebi_math.signals.denoise.wavelet_denoise`; `signals/denoise.py:299`.

Validity `finite and shrinkage_residual <= 1e-12`; status `Exactness.NUMERICAL`; tolerance `1e-12`. Label expression `'KM-DSP-D10'`. Observed examples `EX-E-040`.

H-C328 `komorebi_math.signals.denoise.denoising_risk_certificate`; `signals/denoise.py:349`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `allowed`. Label expression `'KM-DSP-C03'`. Observed examples `EX-E-040`.

H-C329 `komorebi_math.signals.edge_recovery.recover_jumps`; `signals/edge_recovery.py:129`.

Validity `valid`; status `exactness`; tolerance `None`. Label expression `'PSC-V-T19'`. Observed examples none recorded.

H-C330 `komorebi_math.signals.edge_recovery.recover_jumps_subgaussian`; `signals/edge_recovery.py:195`.

Validity `result.certificate.valid`; status `Exactness.CONDITIONAL`; tolerance `None`. Label expression `'PSC-V-T20'`. Observed examples none recorded.

H-C331 `komorebi_math.signals.fft_convolution.certified_overlap_add_convolve`; `signals/fft_convolution.py:236`.

Validity `bool(value.shape == reference.shape and residual <= tolerance * scale)`; status `Exactness.NUMERICAL`; tolerance `tolerance`. Label expression `'KM-DSP-T10'`. Observed examples `EX-E-040`.

H-C332 `komorebi_math.signals.filterbanks.PeriodicFilterBank.perfect_reconstruction_certificate`; `signals/filterbanks.py:132`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `tolerance`. Label expression `'PSC-V-T09'`. Observed examples none recorded.

H-C333 `komorebi_math.signals.filterbanks.HaarFilterBank.certificate`; `signals/filterbanks.py:206`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `tolerance`. Label expression `'PSC-V-T10'`. Observed examples none recorded.

H-C334 `komorebi_math.signals.filterbanks.PeriodicLiftingScheme.certificate`; `signals/filterbanks.py:281`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `tolerance`. Label expression `'PSC-V-T11'`. Observed examples none recorded.

H-C335 `komorebi_math.signals.filterbanks.FiniteFilterBank.report`; `signals/filterbanks.py:353`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `tolerance`. Label expression `'PSC-V-T09'`. Observed examples none recorded.

H-C336 `komorebi_math.signals.filtering.FIRFilter.certificate`; `signals/filtering.py:181`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `tolerance`. Label expression `'KM-DSP-C02'`. Observed examples `EX-E-040`.

H-C337 `komorebi_math.signals.filtering.stream_fir_filter`; `signals/filtering.py:425`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `tolerance`. Label expression `'KM-DSP-T05'`. Observed examples `EX-E-040`.

H-C338 `komorebi_math.signals.nonlinear.packet_product`; `signals/nonlinear.py:112`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `1e-10`. Label expression `'PSC-V-T51'`. Observed examples none recorded.

H-C339 `komorebi_math.signals.nonlinear.packet_product`; `signals/nonlinear.py:141`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `1e-10`. Label expression `'PSC-V-T51'`. Observed examples none recorded.

H-C340 `komorebi_math.signals.polyphase.PolyphaseCoordinates.certificate`; `signals/polyphase.py:121`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `float(tolerance)`. Label expression `'PSC-V-T02'`. Observed examples `EX-E-036`.

H-C341 `komorebi_math.signals.polyphase.LatticePolyphaseCoordinates.certificate`; `signals/polyphase.py:369`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `tolerance`. Label expression `'PSC-V-T04'`. Observed examples none recorded.

H-C342 `komorebi_math.signals.recovery.recover_separated_jumps`; `signals/recovery.py:117`.

Validity `True`; status `Exactness.CONDITIONAL`; tolerance `None`. Label expression `'PSC-V-T36'`. Observed examples `EX-E-052`.

H-C343 `komorebi_math.signals.recovery.recover_jumps_subgaussian`; `signals/recovery.py:182`.

Validity `result.certificate.valid`; status `Exactness.CONDITIONAL`; tolerance `None`. Label expression `'PSC-V-C37'`. Observed examples none recorded.

H-C344 `komorebi_math.signals.recovery.detect_piecewise_polynomial_breaks`; `signals/recovery.py:253`.

Validity `True`; status `Exactness.CONDITIONAL`; tolerance `None`. Label expression `'PSC-V-T43'`. Observed examples none recorded.

H-C345 `komorebi_math.signals.recursive.LinearFilter.stability_certificate`; `signals/recursive.py:160`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `allowed`. Label expression `'KM-DSP-C04'`. Observed examples `EX-E-040`.

H-C346 `komorebi_math.signals.recursive.stream_lfilter`; `signals/recursive.py:292`.

Validity `bool(streamed.shape == reference.shape and residual <= tolerance * scale)`; status `Exactness.NUMERICAL`; tolerance `tolerance`. Label expression `'KM-DSP-T09'`. Observed examples `EX-E-040`.

H-C347 `komorebi_math.signals.resampling.RationalResamplingPlan.certificate`; `signals/resampling.py:186`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `tolerance`. Label expression `'KM-DSP-T06'`. Observed examples `EX-E-040`.

H-C348 `komorebi_math.signals.singular_packets.retained_packet_reconstruct`; `signals/singular_packets.py:242`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `tolerance`. Label expression `'PSC-V-T17'`. Observed examples none recorded.

H-C349 `komorebi_math.signals.singular_packets.retained_packet_reconstruction`; `signals/singular_packets.py:304`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `tolerance`. Label expression `'PSC-V-T18'`. Observed examples `EX-E-052`.

H-C350 `komorebi_math.signals.singular_packets.scalar_region_error`; `signals/singular_packets.py:438`.

Validity `residual <= tolerance`; status `Exactness.NUMERICAL`; tolerance `tolerance`. Label expression `'PSC-V-T21'`. Observed examples none recorded.

H-C351 `komorebi_math.signals.singular_packets.overlap_matrix_error`; `signals/singular_packets.py:481`.

Validity `residual <= tolerance`; status `Exactness.NUMERICAL`; tolerance `tolerance`. Label expression `'PSC-V-T22'`. Observed examples none recorded.

H-C352 `komorebi_math.signals.spectral._power_certificate`; `signals/spectral.py:144`.

Validity `bool(valid)`; status `Exactness.NUMERICAL`; tolerance `tolerance`. Label expression `theorem_id`. Observed examples `EX-E-040`.

H-C353 `komorebi_math.signals.spectral.cross_spectrum`; `signals/spectral.py:369`.

Validity `finite and bound_residual <= 1e-12`; status `Exactness.NUMERICAL`; tolerance `1e-12`. Label expression `'KM-DSP-D07'`. Observed examples `EX-E-040`.

H-C354 `komorebi_math.signals.spectral.parseval_certificate`; `signals/spectral.py:418`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `tolerance`. Label expression `'KM-DSP-T04'`. Observed examples none recorded.

H-C355 `komorebi_math.signals.streaming.stream_stft`; `signals/streaming.py:330`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `tolerance`. Label expression `'KM-DSP-T07'`. Observed examples `EX-E-040`.

H-C356 `komorebi_math.signals.time_frequency.FrameBatch.certificate`; `signals/time_frequency.py:83`.

Validity `valid`; status `Exactness.EXACT`; tolerance `None`. Label expression `'KM-DSP-D02'`. Observed examples `EX-E-040`.

H-C357 `komorebi_math.signals.time_frequency.STFTResult.roundtrip_certificate`; `signals/time_frequency.py:235`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `tolerance`. Label expression `'KM-DSP-T03'`. Observed examples `EX-E-040`.

H-C358 `komorebi_math.signals.time_frequency.istft`; `signals/time_frequency.py:390`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `tolerance`. Label expression `'KM-DSP-T03'`. Observed examples `EX-E-040`.

H-C359 `komorebi_math.signals.wavelets.HaarPacketTree.reconstruction_certificate`; `signals/wavelets.py:188`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `tolerance`. Label expression `'PSC-V-T13'`. Observed examples `EX-E-036`.

H-C360 `komorebi_math.signals.wavelets.best_basis`; `signals/wavelets.py:302`.

Validity `True`; status `Exactness.EXACT`; tolerance `None`. Label expression `'PSC-V-T14'`. Observed examples none recorded.

H-C361 `komorebi_math.signals.windows.overlap_add_certificate`; `signals/windows.py:243`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `tolerance`. Label expression `'KM-DSP-C01'`. Observed examples none recorded.

H-C362 `komorebi_math.signals.windows.canonical_dual_window`; `signals/windows.py:295`.

Validity `valid`; status `Exactness.NUMERICAL`; tolerance `tolerance`. Label expression `'KM-DSP-T02'`. Observed examples none recorded.

Construction group: stable_contract.py

H-C363 `komorebi_math.stable_contract.check_stable_contract`; `stable_contract.py:53`.

Validity `valid`; status `Exactness.EXACT`; tolerance `None`. Label expression `'KM-STABLE-CONTRACT-016'`. Observed examples none recorded.

Construction group: stable_semantics.py

H-C364 `komorebi_math.stable_semantics.run_stable_semantics_fixture`; `stable_semantics.py:415`.

Validity `not failed`; status `Exactness.EXACT`; tolerance `None`. Label expression `'KM-STABLE-SEM-016'`. Observed examples none recorded.

Construction group: unified.py

H-C365 `komorebi_math.unified.UnifiedSystem.validate`; `unified.py:126`.

Validity `valid`; status `Exactness.EXACT if exact else Exactness.NUMERICAL`; tolerance `None if exact else max(1e-10, 0.0 if self.seed_system is None else self.seed_system.tol...`. Label expression `None`. Observed examples `EX-E-043`.

Construction group: verify

H-C366 `komorebi_math.verify.core.VerificationReport.certificate`; `verify/core.py:52`.

Validity `self.passed`; status `Exactness.NUMERICAL`; tolerance `None`. Label expression `None`. Observed examples none recorded.

H-C367 `komorebi_math.verify.expression.run_expression_verification.binding_certificate_propagation`; `verify/expression.py:418`.

Validity `True`; status `Exactness.CONDITIONAL`; tolerance `None`. Label expression `'EXPR.TEST.CONDITIONAL_BINDING'`. Observed examples none recorded.

H.9 Dynamic labels and typed carriers

Forty-two dynamic declaration sites accept, propagate, decode or compose labels. The finite catalog does not enumerate arbitrary user strings or all compositions. Each expression below is retained exactly in the ledger. A supplied label never substitutes for its mathematical assumptions.

Source	Dynamic label expression
coordinates/affine.py:87	f'inverse({self.theorem})'
coordinates/affine.py:111	f'{self.theorem} then {following.theorem}'
coordinates/affine.py:141	self.theorem
coordinates/analysis_synthesis.py:336	self.theorem or None
coordinates/analysis_synthesis.py:369	self.theorem or None
coordinates/basis.py:171	theorem
coordinates/basis.py:316	f'inverse({transform.theorem})'
coordinates/calculus.py:77	f'{self.theorem} then {following.theorem}'
coordinates/calculus.py:91	self.theorem
coordinates/calculus.py:127	theorem
coordinates/frame_algebra.py:468	self.theorem or None
coordinates/frame_algebra.py:835	self.theorem or None
coordinates/frame_algebra.py:996	self.theorem or None
coordinates/modular.py:143	f'inverse({self.theorem})'
coordinates/modular.py:161	f'{self.theorem} then {following.theorem}'
coordinates/modular.py:174	self.theorem
coordinates/modular.py:190	self.theorem
coordinates/modular.py:203	self.theorem
coordinates/numeric.py:88	f'inverse({self.theorem})'
coordinates/numeric.py:106	f'{self.theorem} then {following.theorem}'
coordinates/numeric.py:124	self.theorem
coordinates/numeric.py:139	self.theorem
coordinates/numeric.py:151	self.theorem
coordinates/portable.py:51	str(data['theorem'])
coordinates/portable.py:179	str(data['theorem'])
coordinates/portable.py:212	str(data['theorem'])
coordinates/portable.py:227	str(data['theorem'])
coordinates/portable.py:241	str(data['theorem'])
coordinates/portable.py:300	str(data['theorem'])
coordinates/transform.py:84	theorem or self.theorem
coordinates/transform.py:106	theorem or f'{self.theorem} then {following.theorem}'
coordinates/transform.py:135	self.theorem or None
export/persistence.py:220	None if data['theorem_id'] is None else str(data['theorem_id'])
expression/certification.py:189	theorem_id
expression/certification.py:228	theorem_id
expression/engine.py:337	theorem_id

Source	Dynamic label expression
expression/engine.py:1103	theorem_id
expression/engine.py:1113	theorem_id
expression/polynomial.py:587	primitive.theorem_id
integration/identities.py:59	theorem_id
integration/quadrature.py:485	theorem_id
signals/spectral.py:144	theorem_id

There are 119 class declarations with a certificate-typed field, including value/result carriers. They are not 119 additional certificate protocols or theorem proofs. The field crosswalk below states where to look rather than inventing a generic require method on each carrier.

Carrier	Certificate field(s)	Source
algorithms.compiler.TreePlan	certificate	algorithms/compiler.py:60
algorithms.discovery.BDLRDiscovery Result	certificate	algorithms/discovery.py:96
algorithms.discovery.Support DiscoveryResult	certificate	algorithms/discovery.py:20
algorithms.gpu.DeviceExecution Result	certificate	algorithms/gpu.py:22
algorithms.krylov.BlockJacobi Certificate	certificate	algorithms/krylov.py:396
algorithms.krylov.ConjugateGradient Result	certificate	algorithms/krylov.py:89
algorithms.krylov.KrylovBasisResult	certificate	algorithms/krylov.py:22
algorithms.krylov.LanczosResult	certificate	algorithms/krylov.py:282
algorithms.low_rank.Recompression Result	certificate	algorithms/low_rank.py:75
algorithms.precision.Precision Assignment	certificate	algorithms/precision.py:148
analytic.zeros.ZeroStringResult	certificate	analytic/zeros.py:20
arithmetic.euler.EulerTruncation	certificate	arithmetic/euler.py:125
arithmetic.local_modes.Difference Congruence	certificate	arithmetic/local_modes.py:17
arithmetic.recurrence.OrbitPeriod	certificate	arithmetic/recurrence.py:98
certificates.ComputationResult	certificate	certificates.py:58
certificates.ConditionalResult	certificate	certificates.py:78
compatibility.ApiCompatibility Report	certificate	compatibility.py:185
core.algebra.AlgebraBasisResult	certificate	core/algebra.py:23
core.seeds.StandardSeedSearchResult	certificate	core/seeds.py:36
deformation.companions.Frobenius SeparationResult	certificate	deformation/companions.py:159
deformation.forests.LiftForest Report	certificate	deformation/forests.py:30
deformation.support.FiberSupport Result	certificate	deformation/support.py:42
deformation.support.NestedOccupancy Result	certificate	deformation/support.py:172
dynamics.fractional.Fractional DefectResult	certificate	dynamics/fractional.py:124
export.latex.LatexCompilation	certificate	export/latex.py:51
export.writers.ExportBundleResult	certificate	export/writers.py:125
export.writers.ExportResult	certificate	export/writers.py:104

Carrier	Certificate field(s)	Source
`expression.certification.ExpressionBinding`	`certificate`	`expression/certification.py:291`
`expression.certification.NormalForm`	`certificate`	`expression/certification.py:275`
`expression.certification.RewriteStep`	`certificate`	`expression/certification.py:239`
`expression.engine.CheckedExpression`	`certificate`	`expression/engine.py:394`
`expression.engine.EquivalenceResult`	`certificate`	`expression/engine.py:793`
`expression.engine.ExpressionPlan`	`certificate`	`expression/engine.py:1262`
`expression.engine.ExpressionResult`	`certificate`	`expression/engine.py:1679`
`expression.engine.PlanInstruction`	`certificate`	`expression/engine.py:1177`
`expression.engine.SimplificationResult`	`certificate`	`expression/engine.py:1134`
`expression.planning.MatrixChainPlan`	`certificate`	`expression/planning.py:158`
`golden.BehavioralGoldenCaseResult`	`certificate`	`golden.py:127`
`golden.BehavioralGoldenReport`	`certificate`	`golden.py:151`
`integration.identities.IntegralIdentity`	`certificate`	`integration/identities.py:29`
`integration.quadrature.DerivativeBound`	`certificate`	`integration/quadrature.py:109`
`integration.quadrature.IntegralResult`	`certificate`	`integration/quadrature.py:86`
`integration.rules.QuadratureExactness`	`certificate`	`integration/rules.py:136`
`langlands.finite_image.clifford.CliffordTowerReport`	`certificate`	`langlands/finite_image/clifford.py:69`
`langlands.finite_image.mackey.MackeyDecomposition`	`certificate`	`langlands/finite_image/mackey.py:26`
`langlands.global_.dyadic.DyadicSignDiagnostic`	`certificate`	`langlands/global_/dyadic.py:13`
`langlands.global_.hecke.SupportComparison`	`certificate`	`langlands/global_/hecke.py:99`
`langlands.local.bridge.SymplecticBridge`	`certificate`	`langlands/local/bridge.py:14`
`langlands.local.envelopes.EnvelopeRecovery`	`certificate`	`langlands/local/envelopes.py:17`
`langlands.local.gauss.GaussSumResult`	`certificate`	`langlands/local/gauss.py:14`
`langlands.local.tame.TensorRefinement`	`certificate`	`langlands/local/tame.py:190`
`langlands.ramification.towers.RamificationTowerReport`	`certificate`	`langlands/ramification/towers.py:106`
`nonlinear.composition.FaaDiBrunoResult`	`certificate`	`nonlinear/composition.py:122`
`nonlinear.correction.HilbertObstructionResult`	`certificate`	`nonlinear/correction.py:44`
`nonlinear.correction.LyapunovSchmidtDecomposition`	`certificate`	`nonlinear/correction.py:298`
`nonlinear.correction.LyapunovSchmidtPoint`	`certificate`	`nonlinear/correction.py:312`
`nonlinear.correction.RightInverseCorrectionResult`	`certificate`	`nonlinear/correction.py:139`
`nonlinear.elimination.HiddenSolveResult`	`certificate`	`nonlinear/elimination.py:164`
`nonlinear.elimination.SchurDerivativeResult`	`certificate`	`nonlinear/elimination.py:140`

Carrier	Certificate field(s)	Source
nonlinear.elimination.SchurSecondDerivativeResult	certificate	nonlinear/elimination.py:152
nonlinear.execution.PacketJetBatchResult	certificate	nonlinear/execution.py:216
nonlinear.execution.PacketJetExecutionPlan	certificate	nonlinear/execution.py:58
nonlinear.filtered.DepthObstructionResult	certificate	nonlinear/filtered.py:303
nonlinear.filtered.FilteredSpectralSequenceResult	certificate	nonlinear/filtered.py:600
nonlinear.filtered.FiniteFilteredBranchResult	certificate	nonlinear/filtered.py:419
nonlinear.filtered.SpectralSequencePage	certificate	nonlinear/filtered.py:574
nonlinear.gpu.CuPyValidationResult	certificate	nonlinear/gpu.py:40
nonlinear.graded.DividedSquareResult	certificate	nonlinear/graded.py:558
nonlinear.graded.GradedConvolutionResult	certificate	nonlinear/graded.py:386
nonlinear.graded.GradingCertificate	certificate	nonlinear/graded.py:193
nonlinear.graded.HigherMaurerCartanResult	certificate	nonlinear/graded.py:469
nonlinear.graded.MaurerCartanResult	certificate	nonlinear/graded.py:398
nonlinear.holonomy.IntertwiningResult	certificate	nonlinear/holonomy.py:38
nonlinear.jets.PacketJetSupport	certificate	nonlinear/jets.py:323
nonlinear.jets.PacketTaylorResult	order_certificates, certificate	nonlinear/jets.py:429
nonlinear.lifting.LiftStepResult	certificate	nonlinear/lifting.py:156
nonlinear.obstruction.ObstructionResult	certificate	nonlinear/obstruction.py:17
nonlinear.profiling.PacketJetComplexityEstimate	certificate	nonlinear/profiling.py:61
nonlinear.profiling.PacketJetProfileReport	certificate	nonlinear/profiling.py:176
nonlinear.snapshots.PacketJetSnapshot	capture_certificate	nonlinear/snapshots.py:91
nonlinear.streaming.PacketJetStreamManifest	capture_certificate	nonlinear/streaming.py:395
nonlinear.streaming.PacketJetStreamSelection	certificate	nonlinear/streaming.py:654
nonlinear.streaming.PacketJetStreamWriteResult	certificate	nonlinear/streaming.py:639
nonlinear.streaming.StreamedPacketJetEvaluation	certificate	nonlinear/streaming.py:665
nonlinear.transport.DirectRotationResult	certificate	nonlinear/transport.py:64
nonlinear.transport.PacketDerivativeResult	certificate	nonlinear/transport.py:73
nonlinear.transport.TransportRefinementResult	certificate	nonlinear/transport.py:85
nonlinear.trees.MajorantConvergence	certificate	nonlinear/trees.py:291
nonlinear.trees.PacketTreeResult	certificate	nonlinear/trees.py:606
nonlinear.trees.TreeExpansionResult	certificate	nonlinear/trees.py:447
persistence_contract.PortableCodecCompatibilityReport	certificate	persistence_contract.py:122
quantum.circuits.InterferenceReport	certificate	quantum/circuits.py:27

Carrier	Certificate field(s)	Source
`quantum.constraints.ConstraintClosureReport`	`certificate`	`quantum/constraints.py:17`
`quantum.error_correction.KnillLaflammeReport`	`certificate`	`quantum/error_correction.py:18`
`quantum.leakage.LeakageResult`	`certificate`	`quantum/leakage.py:19`
`quantum.towers.HamiltonianTowerReport`	`certificate`	`quantum/towers.py:17`
`refinement.smith.SmithCongruenceSolution`	`certificate`	`refinement/smith.py:115`
`refinement.weighted.FilteredInverseExpansion`	`certificate`	`refinement/weighted.py:50`
`release.contract.ReleaseContractReport`	`certificate`	`release/contract.py:59`
`release.documentation.StableDocumentationReport`	`certificate`	`release/documentation.py:180`
`release.evidence.ValidationEvidenceReport`	`certificate`	`release/evidence.py:176`
`release.gpu.GpuQualificationReport`	`certificate`	`release/gpu.py:45`
`release.readiness.BetaReadinessReport`	`certificate`	`release/readiness.py:17`
`release.upgrade.UpgradeContractReport`	`certificate`	`release/upgrade.py:76`
`sequences.compiler.CompiledSequence`	`certificate`	`sequences/compiler.py:122`
`signals.adaptive.AdaptiveFilterResult`	`certificate`	`signals/adaptive.py:54`
`signals.approximation.PacketEstimate1D`	`certificate`	`signals/approximation.py:68`
`signals.denoise.WaveletDenoiseResult`	`certificate`	`signals/denoise.py:216`
`signals.edge_recovery.JumpRecoveryResult`	`certificate`	`signals/edge_recovery.py:34`
`signals.filterbanks.FilterBankReport`	`certificate`	`signals/filterbanks.py:294`
`signals.recovery.BreakpointBandResult`	`certificate`	`signals/recovery.py:222`
`signals.recovery.JumpRecoveryResult`	`certificate`	`signals/recovery.py:28`
`signals.resampling.ResamplingResult`	`certificate`	`signals/resampling.py:205`
`signals.singular_packets.RetainedPacketResult`	`certificate`	`signals/singular_packets.py:190`
`signals.spectral.CrossSpectrum`	`certificate`	`signals/spectral.py:121`
`signals.spectral.PowerSpectrum`	`certificate`	`signals/spectral.py:81`
`signals.wavelets.BestBasisResult`	`certificate`	`signals/wavelets.py:232`
`stable_contract.StableContractReport`	`certificate`	`stable_contract.py:19`
`stable_semantics.StableSemanticReport`	`certificate`	`stable_semantics.py:90`

H.10 Executed evidence and index integrity

Check	Observed result
Pinned package source tests	1,203
New index/interface tests	1,261
Replayed Python examples	57
Output predicates	190
Base construction invocations	1,842
Base require invocations	841
Observed distinct nonempty labels	142
Observed base construction sites	139
Unobserved base construction sites	218
Source/installed package files compared	366

The construction and require counts overlap, and multiple invocations can represent the same finite example or label. A construction can hold an invalid or conditional result. The example contracts check expected output and preserved negative conclusions, not unconditional validity of every certificate. The source suite passes as a suite; its pass count is not a per-theorem proof mapping.

All 300 Python package source files are hashed in the index. Each constructor, type, carrier and provenance row has a source path, line range and full SHA-256. The index records the frozen mathematical implementation and distribution without modifying them.

H.11 Reproduction, source identity and closure

The working tools under `tools/block40` regenerate the static inventory, validate the separately installed wheel against all 366 package files, replay the 57 Python example contracts, build the observed crosswalk, execute the source/index and behavioral tests, and generate the cumulative and standalone documentation. The source tests are a separate package suite. The new reference tests test the index and deliberately chosen interface boundaries; they are not additional mathematical theorems.

Use a fresh output directory for the example replay. It does not overwrite an earlier evidence run. The replay delegates the original base constructor and require method, records observations in the example interpreter, then restores the methods. Independent specialized-only certificates and separately launched child interpreters are outside that instrumentation boundary. The original script assertions, output predicates and package-origin checks remain active.

The directory verifier binds every delivered file and executable mode. The build gate requires current source/test/replay/document hashes. These are local integrity and execution records, not authenticated signatures. Reproducing Markdown, JSON and LaTeX is a separate claim from binary reproducibility of independently rebuilt PDFs. The final ZIP's own hash belongs in the external delivery record to avoid circular self-attestation.

Reference boundaries. This source-bound certificate and theorem-label index does not resolve producer-hardening issues, establish a bijection between labels and mathematical theorems, or verify correspondence with external Lean declarations. Appendix I provides the separate exception and diagnostic reference.

The recorded audit did not qualify native Windows/PowerShell, other Python versions, CUDA, strict MkDocs, fully dependency-isolated installation or final publication. Prior GPU and platform failures remain negative observations. It included no new Lean proof audit. Appendix O identifies the separate final-source qualification records.

Appendix I. Exceptions and diagnostic cookbook

Start with the symptom. This cookbook is intended for use beside a failing program. Keep the smallest input that still reproduces the problem and use I.2 to record the run before changing it.

What you observe	First place to look	Next question
A Python exception or traceback	I.3 and the matching recipe	Which input contract or dependency produced the refusal?
A returned object whose result is unusable	I.4 and Appendix H	Is the certificate invalid, conditional, or numerical when exact data are required?
A warning, or a warning treated as an exception	I.4 and Appendix L	Which warning filter is active, and did the function body run?
A failed command-line run	I.4 and Appendix F	Did argument parsing fail, or did the selected handler execute and refuse the task?

Use the recovery as a comparison. Run a cookbook recipe's setup before its trigger, and run its recovery separately if the trigger raises. Compare the failing input with the control input to identify the changed condition. Then repair that condition in your own problem and rerun its original check. This makes the recipe a diagnostic example rather than a replacement for your data.

I.1 Scope and evidence channels

This appendix answers a practical question: when a Komorebi Math computation does not produce the intended usable result, what exactly failed, where is that behavior defined, and what is a justified next action? It is a reference to **Komorebi Math 1.0.0b4, Repair R2, wheel build tag 2**. All runtime observations are bound to that source baseline.

Four channels must be read separately. An exception interrupts the Python call. A returned certificate can report an invalid or conditional result without interrupting it. A warning reports a condition through Python's warning policy and may leave the return value unchanged. A command-line process has an exit status plus separate stdout and stderr streams. A zero exit, an exactness label, a nonempty output file, and a favorable stored validity flag are not interchangeable evidence.

The static census includes every explicit `raise`, `except`, and `assert` statement and the recognized `warnings.warn` call in all 300 shipped Python package source files. It includes the package's verification modules. It does not include the test tree, third-party source, operating-system internals, or arbitrary user callbacks. A function can propagate a dependency error without containing any explicit raise. A syntax census is therefore **not a complete set of all runtime exceptions**.

Each electronic site record contains a qualified owner, source path, exact line range, source-file SHA-256, complete statement, exception or handler expression, and syntactic ancestor guards. A guard is a description of the source branch, not a proof that the predicate is sufficient, reachable, or true for every caller. Bare re-raises have no invented exception type. Handler records distinguish syntactic return, pass, continue, raise and re-raise actions, without claiming a full control-flow analysis.

The cookbook executes representative failing inputs and paired valid or diagnostic-control inputs in a separately installed wheel. It does not execute every one of the thousands of static branches. Missing-dependency discovery, a compiler timeout, and the planned-feature registry guard use explicitly labeled controlled fixtures. They are not claims that a native dependency was absent, a real compiler hung, or an existing feature was implemented during the test.

I.2 Triage before changing the mathematics

Capture the channel and preserve the original input. Record the exact command or API call, the last exception and its cause chain, exit status, stdout, stderr, certificate fields and generated-file names. Keep a copy of failed persisted inputs before regeneration. Messages can contain paths or data, so review them before sharing a report externally.

Confirm the executed artifact. Record `komorebi_math.__file__`, the package version, Python executable, optional backend selection, source checksum and wheel checksum. The public version alone cannot distinguish the stale pre-repair distribution from the repaired build-2 wheel. Use the R2 release parity checks rather than assuming that an import from a neighboring source checkout validates the installed artifact.

Locate the actual refusal. Look up the message or symbol in `Block41_Raise_Site_Lookup.tsv` or the JSON inventory. Retain `__cause__`: a `ValidationError` around a portable load can originate in a missing file, invalid UTF-8, malformed JSON, a schema mismatch, a checksum mismatch or a type-specific constructor. Re-running the mathematical calculation does not repair all of those causes.

Correct the violated contract. Shape errors call for compatible spaces and dimensions. Relation errors call for typed parallel paths. Positive-evidence failures call for a justified probability model. Exact-route errors call for exact inputs, not a cast of already rounded data. Resource refusals call for a smaller request or a justified explicit budget, not an unbounded retry. A returned negative certificate calls for inspection of its producer and prerequisites.

Recheck, do not relabel. Do not suppress a failure with `validate=False`, overwrite a certificate's validity, remove an unchecked hypothesis, enlarge a tolerance without justification, or recompute a checksum merely to conceal a damaged artifact. The positive recipe inputs are independent small fixtures that satisfy the contract; they are not permission to replace the user's real data with arbitrary convenient values.

I.3 Public exception hierarchy and catch boundaries

The nine package exception classes are defined in `komorebi_math.exceptions`. The project warning class is defined separately in `komorebi_math.compatibility`. This reference does not classify the `IntegrationError` data record or the `ErrorGuarantee` enum as exception classes merely because their names contain the word Error.

`ValidationError` inherits both `KomorebiMathError` and `ValueError`. `DimensionError`, `RelationError`, and `ResourceLimitError` inherit from it. Catching `ValueError` before a more specific package subtype can hide whether the caller violated a geometry contract or exceeded a resource bound. `FeatureNotImplementedError` also inherits `NotImplementedError`; its multiple inheritance should be considered when ordering handlers.

`BackendUnavailableError`, `ExactnessError`, and `AssumptionError` are package exceptions but not ValidationError subtypes. Ordinary `ValueError`, `TypeError`, `KeyError`, `OSError` and dependency exceptions can also propagate. Catching only `KomorebiMathError` is not a universal application boundary. Conversely, catching all `Exception` and returning success destroys the distinction between an expected refusal and an implementation defect.

`ExactnessError` is a declared public type with **no explicit raise site** in this pinned source census. Its name does not imply universal exactness enforcement. The exact-expression literal route instead rejects floating and boolean coefficients with `ValidationError`; the numerical packet route can return a valid `NUMERICAL` certificate without raising at all.

Class	Direct base(s)	Explicit package raise sites
`KomorebiDeprecationWarning`	`DeprecationWarning`	0
`AssumptionError`	`KomorebiMathError`	17
`BackendUnavailableError`	`KomorebiMathError`	31
`DimensionError`	`ValidationError`	686
`ExactnessError`	`KomorebiMathError`	0
`FeatureNotImplementedError`	`NotImplementedError`, `KomorebiMathError`	1
`KomorebiMathError`	`Exception`	0
`RelationError`	`ValidationError`	7
`ResourceLimitError`	`ValidationError`	63
`ValidationError`	`KomorebiMathError`, `ValueError`	2204

Class	Direct base(s)	Explicit package raise sites

A zero in the last column means no explicit `raise Class(...)` was found, not that callers cannot instantiate or raise the class. The warning class is emitted through `warnings.warn`, so its explicit-raise count is also zero.

KomorebiDeprecationWarning

Warning category for explicit Komorebi Math API deprecations.

Source: `komorebi_math/compatibility.py`, lines 229-230. Bases: `builtins.DeprecationWarning`. Source SHA-256: `9422447e4d0c996c6474c93c326722ad2e449ceb35f52211c2eb70fae12543b7`.

AssumptionError

Raised when a required mathematical assumption is not satisfied.

Source: `komorebi_math/exceptions.py`, lines 30-31. Bases: `komorebi_math.exceptions.KomorebiMathError`. Source SHA-256: `65aee29fea129eec941c2a3667dd3d5dff67680cafb50e37674bb475fa21a1bb`.

BackendUnavailableError

Raised when an optional computation backend is not installed.

Source: `komorebi_math/exceptions.py`, lines 22-23. Bases: `komorebi_math.exceptions.KomorebiMathError`. Source SHA-256: `65aee29fea129eec941c2a3667dd3d5dff67680cafb50e37674bb475fa21a1bb`.

DimensionError

Raised when matrix, vector, or packet dimensions do not agree.

Source: `komorebi_math/exceptions.py`, lines 14-15. Bases: `komorebi_math.exceptions.ValidationError`. Source SHA-256: `65aee29fea129eec941c2a3667dd3d5dff67680cafb50e37674bb475fa21a1bb`.

ExactnessError

Raised when an exact operation is requested from an inexact backend.

Source: `komorebi_math/exceptions.py`, lines 26-27. Bases: `komorebi_math.exceptions.KomorebiMathError`. Source SHA-256: `65aee29fea129eec941c2a3667dd3d5dff67680cafb50e37674bb475fa21a1bb`.

FeatureNotImplementedError

Raised by a deliberately declared placeholder feature.

Source: `komorebi_math/exceptions.py`, lines 38-39. Bases: `builtins.NotImplementedError`, `komorebi_math.exceptions.KomorebiMathError`. Source SHA-256: `65aee29fea129eec941c2a3667dd3d5dff67680cafb50e37674bb475fa21a1bb`.

KomorebiMathError

Base class for all public package exceptions.

Source: `komorebi_math/exceptions.py`, lines 6-7. Bases: `builtins.Exception`. Source SHA-256: `65aee29fea129eec941c2a3667dd3d5dff67680cafb50e37674bb475fa21a1bb`.

RelationError

Raised when a grammar relation is malformed or violated.

Source: `komorebi_math/exceptions.py`, lines 18-19. Bases: `komorebi_math.exceptions.ValidationError`. Source SHA-256: `65aee29fea129eec941c2a3667dd3d5dff67680cafb50e37674bb475fa21a1bb`.

ResourceLimitError

Raised before a computation exceeds an explicit resource contract.

Source: `komorebi_math/exceptions.py`, lines 34-35. Bases: `komorebi_math.exceptions.ValidationError`. Source SHA-256: `65aee29fea129eec941c2a3667dd3d5dff67680cafb50e37674bb475fa21a1bb`.

ValidationError

Raised when a mathematical object fails a declared validation rule.

Source: `komorebi_math/exceptions.py`, lines 10-11. Bases: `komorebi_math.exceptions.KomorebiMathError`, `builtins.ValueError`. Source SHA-256: `65aee29fea129eec941c2a3667dd3d5dff67680cafb50e37674bb475fa21a1bb`.

I.4 Exceptions, certificates, warnings and exit codes

The base `Certificate.require()` tests the truthiness of its stored `valid` field and raises `ValidationError` when false. It does not recompute residuals, validate a witness, verify a theorem ID or discharge unchecked assumptions. Appendix H describes specialized protocols, which must not be guessed from a common attribute name. The supplied diagnostic helper observes base-certificate fields without changing them, records whether `valid` is an actual bool, and always leaves `proof_admitted` false.

The packet-validation bypass from Appendix H remains present: a self-adjoint matrix that is not a projector can receive a favorable Hilbert flag after `validate=False`, while its base packet-axiom certificate fails. The cookbook repeats the discrepancy as a regression boundary. The recovery is to require the prerequisite packet axioms, not to promote the specialized flag into a proof.

`KomorebiDeprecationWarning` is a `DeprecationWarning` subclass. One package warning-emission site constructs its message from the API name, since version, optional replacement, removal version and reason. The cookbook captures that category explicitly and also promotes it to an exception using a local warning filter. Those two outcomes are consequences of policy applied to the same source behavior, not two different mathematical failures.

Python assert statements form a separate verification channel. They can be disabled by optimized execution. The reference and source test runs keep assertions active. The 411 indexed assertions include internal and verification assertions; they are not all user-input guards or independently proved invariants. Never treat an optimized run that omitted assertions as equivalent validation.

The pinned CLI parses arguments and invokes the handler without one universal exception-to-JSON wrapper. Parser usage failures normally exit 2. An uncaught handler exception normally produces a traceback and exit 1. A handler can also intentionally return 1 with a negative certificate, or return 2 for its own semantic selector checks. Read the output and handler record before assigning a meaning to the number. `langlands completion` with unsatisfied assumptions remains an intentional negative result. Displaying help is navigation, not completion of those assumptions.

A compiler child return code 124 in a `LatexCompilation` record is the wrapper's timeout convention. It is not a mathematical error class. An outer process exit code must be recorded separately. Native launcher and PowerShell exit propagation remain pending platform work; no Linux recipe is silently counted as a native Windows acceptance test.

I.5 Cookbook execution contract

Each recipe prints its setup, failing or diagnostic trigger, expected observed outcome, explanation and valid recovery/control. The blocks share one fresh workspace named `work`, supplied as a `pathlib.Path` by the runner. To reproduce a file-producing recipe manually, open a fresh temporary directory first:

```
from pathlib import Path
from tempfile import TemporaryDirectory

workspace = TemporaryDirectory(prefix="km-diagnostic-")
work = Path(workspace.name)
# Run the recipe setup, then its trigger under the shown exception policy.
# Run its recovery/control and inspect the outcome before cleanup.
workspace.cleanup()
```

Do not concatenate an uncaught failing block with its recovery and expect execution to continue. The provided runner checks the exact exception class name, message fragment, cause chain and source frames, then executes the separate recovery block in the same child. A recipe marked observation uses explicit assertions about the returned negative status or warning; it is not expected to throw immediately.

Every child has a bounded execution timeout. All mathematical imports must resolve to the separate pinned wheel. The seven CLI recipes invoke the actual isolated entry point in an additional child with no source `PYTHONPATH`; those nested CLI children do not have a per-module origin observer. Their origin basis is the selected virtual-environment executable, isolated mode and the parent package-parity preflight. This distinction is retained in the replay metadata.

The portable-file recipes test preservation of existing sentinel files on overwrite refusal. The compiler recipes use temporary directories and shell escape remains disabled. A deliberately invalid small TeX input is compiled by the installed engine, and an independent valid TeX fixture is compiled successfully. Discovery absence and timeout behavior are controlled separately using mocks, as labeled. These tests do not qualify compilation of arbitrary hostile TeX.

Mathematical validation

XI-D001 / No projectors supplied

Meaning. An empty family cannot resolve the ambient space.

Recovery decision. Provide a nonempty complete projector family.

Evidence mode: native finite input. **Observed exception:** `komorebi_math.exceptions.ValidationError`. Expected message fragment: `at least one`.

Setup

```
from komorebi_math import PacketSystem
import numpy as np
```

Trigger

```
PacketSystem([])
```

Valid recovery or diagnostic control

```
p = PacketSystem([np.eye(2)])
assert p.certificate().require().valid
```

Observed explicit source site(s): `XI-R0777` at `core/packets.py:64`.

Implementation lookup: `komorebi_math.core.packets.PacketSystem`. **Executed receipt:** `diagnostic_replay/XI-D001/receipt.json`. Its hash is `c288e100c0ba248e91c20b9e0f57ae5ec056b3c0eaf104e47dfbe6a2de370f42`. The source hashes and complete traceback are retained in the crosswalk and receipt.

XI-D002 / Nonsquare projector

Meaning. The projector is not an endomorphism of one finite space.

Recovery decision. Correct the matrix geometry before checking the projector axioms.

Evidence mode: native finite input. **Observed exception:** `komorebi_math.exceptions.DimensionError`. Expected message fragment: `square`.

Setup

```
from komorebi_math import PacketSystem
import numpy as np
```

Trigger

```
PacketSystem([np.ones((2, 3))])
```

Valid recovery or diagnostic control

```
p = PacketSystem([np.eye(2)])
assert p.dimension == 2
```

Observed explicit source site(s): `XI-R0778` at `core/packets.py:70`.

Implementation lookup: `komorebi_math.core.packets.PacketSystem`. **Executed receipt:** `diagnostic_replay/XI-D002/receipt.json`. Its hash is `bd77dfdfe6c178e9e6801b9cbeef149d8896e8d62390c726a161ff4fe80a4f77`. The source hashes and complete traceback are retained in the crosswalk and receipt.

XI-D003 / Projector dimensions disagree

Meaning. Every projector must act on the same ambient dimension.

Recovery decision. Use compatible blocks in one declared coordinate system.

Evidence mode: native finite input. **Observed exception:** `komorebi_math.exceptions.DimensionError`. Expected message fragment: `common size`.

Setup

```
from komorebi_math import PacketSystem
import numpy as np
```

Trigger

```
PacketSystem([np.eye(2), np.eye(3)])
```

Valid recovery or diagnostic control

```
p = PacketSystem([np.diag([1.0, 0.0]), np.diag([0.0, 1.0])])
p.certificate().require()
```

Observed explicit source site(s): `XI-R0779` at `core/packets.py:72`.

Implementation lookup: `komorebi_math.core.packets.PacketSystem`. **Executed receipt:** `diagnostic_replay/XI-D003/receipt.json`. Its hash is `7bfbb2e1ff92cc1933690986f2a0628eef827d69cdfa30431072bfed9ae5a15e`. The source hashes and complete traceback are retained in the crosswalk and receipt.

XI-D004 / Name count mismatch

Meaning. Names and projectors have different cardinalities.

Recovery decision. Provide exactly one name per projector.

Evidence mode: native finite input. **Observed exception:** `komorebi_math.exceptions.ValidationError`. Expected message fragment: `same length`.

Setup

```
from komorebi_math import PacketSystem
import numpy as np
```

Trigger

```
PacketSystem([np.eye(2)], names=['a', 'b'])
```

Valid recovery or diagnostic control

```
p = PacketSystem([np.eye(2)], names=['all'])
assert p.index('all') == 0
```

Observed explicit source site(s): `XI-R0780` at `core/packets.py:79`.

Implementation lookup: `komorebi_math.core.packets.PacketSystem`. **Executed receipt:** `diagnostic_replay/XI-D004/receipt.json`. Its hash is `f06004a6c77e4ad0be5a26a998a9c3c79ba3e98634409702a530e8f5b3cec6b5`. The source hashes and complete traceback are retained in the crosswalk and receipt.

XI-D005 / Duplicate packet names

Meaning. Name lookup would be ambiguous.

Recovery decision. Assign distinct labels without changing the projector meaning.

Evidence mode: native finite input. **Observed exception:** `komorebi_math.exceptions.ValidationError`. Expected message fragment: `unique`.

Setup

```
from komorebi_math import PacketSystem
import numpy as np
```

Trigger

```
PacketSystem([np.diag([1.0, 0.0]), np.diag([0.0, 1.0])], names=['a', 'a'])
```

Valid recovery or diagnostic control

```
p = PacketSystem([np.diag([1.0, 0.0]), np.diag([0.0, 1.0])], names=['a', 'b'])
assert p.names == ('a', 'b')
```

Observed explicit source site(s): `XI-R0781` at `core/packets.py:81`.

Implementation lookup: `komorebi_math.core.packets.PacketSystem`. **Executed receipt:** `diagnostic_replay/XI-D005/receipt.json`. Its hash is `10ed80edd352948841571845c3c2cc779b4fddf9f06ca25805987dc056449d6e`. The source hashes and complete traceback are retained in the crosswalk and receipt.

XI-D006 / Vector belongs to another dimension

Meaning. The input vector is not in the declared ambient space.

Recovery decision. Reconcile the data dimension or rebuild the correct packet system.

Evidence mode: native finite input. **Observed exception:** `komorebi_math.exceptions.DimensionError`. Expected message fragment: `vector dimension`.

Setup

```
from komorebi_math import PacketSystem
import numpy as np
```

Trigger

```
PacketSystem([np.eye(2)]).analyze([1, 2, 3])
```

Valid recovery or diagnostic control

```
p = PacketSystem([np.eye(2)])
assert np.allclose(p.analyze([1, 2]).reconstruct(), [1, 2])
```

Observed explicit source site(s): `XI-R0783` at `core/packets.py:157`.

Implementation lookup: `komorebi_math.core.packets.PacketSystem`. **Executed receipt:** `diagnostic_replay/XI-D006/receipt.json`. Its hash is `18a6507e4ef7bbf77455aeef3198812fb04d4e81d32bc63026ab5512787b2346`. The source hashes and complete traceback are retained in the crosswalk and receipt.

XI-D007 / Projector axioms fail

Meaning. Construction calls the packet certificate and refuses failed axioms.

Recovery decision. Correct the projectors. Do not use validate=False or enlarge tolerance merely to suppress failure.

Evidence mode: native finite input. **Observed exception:** `komorebi_math.exceptions.ValidationError`. Expected message fragment: `complete packet`.

Setup

```
from komorebi_math import PacketSystem
import numpy as np
```

Trigger

```
PacketSystem([2 * np.eye(2)])
```

Valid recovery or diagnostic control

```
p = PacketSystem([np.eye(2)])
p.certificate().require()
```

Observed explicit source site(s): `XI-R0334` at `certificates.py:45`.

Implementation lookup: `komorebi_math.core.packets.PacketSystem`. **Executed receipt:** `diagnostic_replay/XI-D007/receipt.json`. Its hash is `6267c0fa27bdaa297df1fa256da1d1bee6e542b481afa32d39737805c7fa91cf`. The source hashes and complete traceback are retained in the crosswalk and receipt.

XI-D008 / Unknown packet name

Meaning. The name does not occur in the stored packet name tuple.

Recovery decision. Inspect p.names and request a declared name. The ValueError cause is preserved.

Evidence mode: native finite input. **Observed exception:** `builtins.KeyError`. Expected message fragment: `missing`. **Explicit cause:** `builtins.ValueError`.

Setup

```
from komorebi_math import PacketSystem
import numpy as np
```

Trigger

```
PacketSystem([np.eye(2)], names=['all']).index('missing')
```

Valid recovery or diagnostic control

```
p = PacketSystem([np.eye(2)], names=['all'])
assert p.index('all') == 0
```

Observed explicit source site(s): `XI-R0782` at `core/packets.py:90`.

Implementation lookup: `komorebi_math.core.packets.PacketSystem`. **Executed receipt:** `diagnostic_replay/XI-D008/receipt.json`. Its hash is `9db75c5258467903d44682837dcafce638ebc958f9f551a1a31b8fe069523e66`. The source hashes and complete traceback are retained in the crosswalk and receipt.

Dimension and relation mismatch

XI-D009 / Unnamed relation

Meaning. Every relation needs an identity for registration.

Recovery decision. Correct the typed relation, including its endpoints, before registration.

Evidence mode: native finite input. **Observed exception:** `komorebi_math.exceptions.RelationError`. Expected message fragment: `requires a name`.

Setup

```
from komorebi_math.core.grammar import OperatorRelation, OperatorWord, OperatorGrammar
w = OperatorWord((), 'X', 'X')
```

Trigger

```
OperatorRelation('', ((1, w),))
```

Valid recovery or diagnostic control

```
r = OperatorRelation('identity-cancel', ((1, w), (-1, w)))
assert r.source == r.target == 'X'
```

Observed explicit source site(s): `XI-R0729` at `core/grammar.py:48`.

Implementation lookup: `komorebi_math.core.grammar.OperatorRelation`. **Executed receipt:** `diagnostic_replay/XI-D009/receipt.json`. Its hash is `c0d8637ea4e1e2c00bf608a2ac6ad9ebd1074a52297e7b37ecdead64096c98f0`. The source hashes and complete traceback are retained in the crosswalk and receipt.

XI-D010 / Empty relation

Meaning. An empty list is not a declared linear relation.

Recovery decision. Correct the typed relation, including its endpoints, before registration.

Evidence mode: native finite input. **Observed exception:** `komorebi_math.exceptions.RelationError`. Expected message fragment: `at least one term`.

Setup

```
from komorebi_math.core.grammar import OperatorRelation, OperatorWord, OperatorGrammar
w = OperatorWord((), 'X', 'X')
```

Trigger

```
OperatorRelation('r', ())
```

Valid recovery or diagnostic control

```
r = OperatorRelation('identity-cancel', ((1, w), (-1, w)))
assert r.source == r.target == 'X'
```

Observed explicit source site(s): `XI-R0730` at `core/grammar.py:50`.

Implementation lookup: `komorebi_math.core.grammar.OperatorRelation`. **Executed receipt:** `diagnostic_replay/XI-D010/receipt.json`. Its hash is `bba551ca59ab76cb45e593e2bf1afdb01748a7d46f80d506bcafc66bec9c4c41`. The source hashes and complete traceback are retained in the crosswalk and receipt.

XI-D011 / Nonparallel relation terms

Meaning. A relation must compare parallel typed paths.

Recovery decision. Correct the typed relation, including its endpoints, before registration.

Evidence mode: native finite input. **Observed exception:** `komorebi_math.exceptions.RelationError`. Expected message fragment: `same source and target`.

Setup

```
from komorebi_math.core.grammar import OperatorRelation, OperatorWord, OperatorGrammar
w = OperatorWord((), 'X', 'X')
```

Trigger

```
OperatorRelation('r', ((1, w), (1, OperatorWord((), 'Y', 'Y'))))
```

Valid recovery or diagnostic control

```
r = OperatorRelation('identity-cancel', ((1, w), (-1, w)))
assert r.source == r.target == 'X'
```

Observed explicit source site(s): `XI-R0731` at `core/grammar.py:54`.

Implementation lookup: `komorebi_math.core.grammar.OperatorRelation`. **Executed receipt:** `diagnostic_replay/XI-D011/receipt.json`. Its hash is `997c3fce9c67fff895fe132c20e0adac17046b0fbf1b2ae262b7d4bacecd4cc7`. The source hashes and complete traceback are retained in the crosswalk and receipt.

XI-D012 / Arrow references an unknown vertex

Meaning. An endpoint has no declared vertex.

Recovery decision. Declare the intended vertex or correct the arrow.

Evidence mode: native finite input. **Observed exception:** `komorebi_math.exceptions.ValidationError`. Expected message fragment: `unknown vertex`.

Setup

```
from komorebi_math.core.grammar import OperatorRelation, OperatorWord, OperatorGrammar
w = OperatorWord((), 'X', 'X')
```

Trigger

```
OperatorGrammar(['X'], {'a': ('X', 'Y')})
```

Valid recovery or diagnostic control

```
g = OperatorGrammar(['X', 'Y'], {'a': ('X', 'Y')})
assert g.word('a').target == 'Y'
```

Observed explicit source site(s): `XI-R0736` at `core/grammar.py:111`.

Implementation lookup: `komorebi_math.core.grammar.OperatorGrammar`. **Executed receipt:** `diagnostic_replay/XI-D012/receipt.json`. Its hash is `6948218604648d3dc1b219c021cc1ec72e0744016f68f4235472c454b0a53f01`. The source hashes and complete traceback are retained in the crosswalk and receipt.

Unavailable backend

XI-D013 / Unknown backend name

Meaning. Backend selection is explicit and unknown names do not silently fall back.

Recovery decision. Check the registered selector. NumPy is a numerical replacement only when that meets the task.

Evidence mode: native finite input. **Observed exception:** `komorebi_math.exceptions.BackendUnavailableError`. Expected message fragment: `unknown or unavailable`. **Explicit cause:** `builtins.KeyError`.

Setup

```
from komorebi_math.backends import get_backend, register_backend
from komorebi_math.backends.numpy import NumpyBackend
import numpy as np
```

Trigger

```
get_backend('not-a-backend')
```

Valid recovery or diagnostic control

```
assert get_backend('numpy').name == 'numpy'
```

Observed explicit source site(s): `XI-R0328` at `backends/__init__.py:36`.

Implementation lookup: `komorebi_math.backends.get_backend`. **Executed receipt:** `diagnostic_replay/XI-D013/receipt.json`. Its hash is `023edcec2085dc133f53193d4fe25a4e6e53bd09a0b0b0486b02c7704272fcb8`. The source hashes and complete traceback are retained in the crosswalk and receipt.

XI-D014 / Duplicate backend registration

Meaning. Registration is protected by default.

Recovery decision. Reuse the registered backend. Replace only as a deliberate extension change.

Evidence mode: native finite input. **Observed exception:** `builtins.ValueError`. Expected message fragment: `already registered`.

Setup

```
from komorebi_math.backends import get_backend, register_backend
from komorebi_math.backends.numpy import NumpyBackend
import numpy as np
```

Trigger

```
register_backend(NumpyBackend())
```

Valid recovery or diagnostic control

```
assert get_backend('numpy').name == 'numpy'
```

Observed explicit source site(s): `XI-R0326` at `backends/__init__.py:17`.

Implementation lookup: `komorebi_math.backends.register_backend`. **Executed receipt:** `diagnostic_replay/XI-D014/receipt.json`. Its hash is `14d60876574ce74be29456051cc04927dfdf9c5149362d1464de71d8699551bd`. The source hashes and complete traceback are retained in the crosswalk and receipt.

XI-D015 / Missing exact dependency (controlled)

Meaning. The missing-import branch is exercised with controlled discovery, not by uninstalling SymPy.

Recovery decision. Use the declared exact dependency when exact arithmetic is required. This numerical control is not an exactness-preserving substitution.

Evidence mode: mocked discovery only; not actual missing installation. **Observed exception:** `komorebi_math.exceptions.BackendUnavailableError`. Expected message fragment: `exact`.

Setup

```
from komorebi_math.backends import get_backend, register_backend
from komorebi_math.backends.numpy import NumpyBackend
import numpy as np
from unittest.mock import patch
import komorebi_math.backends as b
b._BACKENDS.pop('sympy', None)
```

Trigger

```
with patch.object(b, 'find_spec', return_value=None):
    get_backend('sympy')
```

Valid recovery or diagnostic control

```
assert get_backend('numpy').exact is False
```

Observed explicit source site(s): `XI-R0327` at `backends/__init__.py:27`.

Implementation lookup: `komorebi_math.backends.get_backend`. **Executed receipt:** `diagnostic_replay/XI-D015/receipt.json`. Its hash is `77cd9fe177c8109616694463d85e7110484cc32019b5e1c780477296c0e81e4c`. The source hashes and complete traceback are retained in the crosswalk and receipt.

XI-D016 / Singular numerical inverse

Meaning. A dependency exception propagates through the numerical backend.

Recovery decision. Verify invertibility. Use another mathematical problem formulation only when justified.

Evidence mode: native finite input. **Observed exception:** `numpy.linalg.LinAlgError`. Expected message fragment: `Singular matrix`.

Setup

```
from komorebi_math.backends import get_backend, register_backend
from komorebi_math.backends.numpy import NumpyBackend
import numpy as np
```

Trigger

```
get_backend('numpy').inverse(np.zeros((2, 2)))
```

Valid recovery or diagnostic control

```
assert np.allclose(get_backend('numpy').inverse(np.eye(2)), np.eye(2))
```

No explicit package raise site is attributed from this recipe's terminal traceback. This is expected for returned observations, warnings emitted through the warning machinery, dependency-only raises and nested CLI processes.

Implementation lookup: `komorebi_math.backends.numpy.NumpyBackend.inverse`. **Executed receipt:** `diagnostic_replay/XI-D016/receipt.json`. Its hash is `68324d58b2438afbbeb13ccb840449ed03b16dc1e5da1b7e16d27afc8284a86f`. The source hashes and complete traceback are retained in the crosswalk and receipt.

Exactness loss

XI-D017 / Floating value on an exact expression route

Meaning. This exact route rejects values outside int/Fraction. It raises ValidationError, not the declared ExactnessError.

Recovery decision. Provide an exact rational witness. Fraction(1,10) expresses one tenth; constructing a Fraction from an already rounded float does not recover intended decimal semantics.

Evidence mode: native finite input. **Observed exception:** `komorebi_math.exceptions.ValidationError`. Expected message fragment: `integer or Fraction`.

Setup

```
from fractions import Fraction
from komorebi_math.expression.model import ExactValueLiteral, RATIONAL_SPACE
```

Trigger

```
ExactValueLiteral(RATIONAL_SPACE, (0.1,))
```

Valid recovery or diagnostic control

```
x = ExactValueLiteral(RATIONAL_SPACE, (Fraction(1, 10),))
assert x.entries == (Fraction(1, 10),)
```

Observed explicit source site(s): `XI-R1276` at `expression/model.py:39`.

Implementation lookup: `komorebi_math.expression.model._fraction`, `komorebi_math.expression.model.ExactValueLiteral`. **Executed receipt:** `diagnostic_replay/XI-D017/receipt.json`. Its hash is `cd3f2f669fdfbf26c140656d255ecd9017b9415975ee5b01aa7ebfbb6f7f6a7d`. The source hashes and complete traceback are retained in the crosswalk and receipt.

XI-D018 / Boolean on an exact expression route

Meaning. This exact route rejects values outside int/Fraction. It raises ValidationError, not the declared ExactnessError.

Recovery decision. Provide an exact rational witness. Fraction(1,10) expresses one tenth; constructing a Fraction from an already rounded float does not recover intended decimal semantics.

Evidence mode: native finite input. **Observed exception:** `komorebi_math.exceptions.ValidationError`. Expected message fragment: `integer or Fraction`.

Setup

```
from fractions import Fraction
from komorebi_math.expression.model import ExactValueLiteral, RATIONAL_SPACE
```

Trigger

```
ExactValueLiteral(RATIONAL_SPACE, (True,))
```

Valid recovery or diagnostic control

```
x = ExactValueLiteral(RATIONAL_SPACE, (Fraction(1, 10),))
assert x.entries == (Fraction(1, 10),)
```

Observed explicit source site(s): `XI-R1276` at `expression/model.py:39`.

Implementation lookup: `komorebi_math.expression.model._fraction`, `komorebi_math.expression.model.ExactValueLiteral`. **Executed receipt:** `diagnostic_replay/XI-D018/receipt.json`. Its hash is `265f248be52f0004437b8253720a29e0f61cd113492cf0307dac04bad5f50eab`. The source hashes and complete traceback are retained in the crosswalk and receipt.

XI-D019 / Conditional wrapper given a different status

Meaning. The wrapper checks status membership, not a theorem witness.

Recovery decision. Use the correct result type and retain unproved hypotheses instead of relabeling them.

Evidence mode: native finite input. **Observed exception:** `builtins.ValueError`. Expected message fragment: `conditional certificate`.

Setup

```
from komorebi_math.certificates import Certificate, Exactness, ConditionalResult
from komorebi_math.expression.certification import compose_certificates
```

Trigger

```
ConditionalResult(7, Certificate(True, Exactness.EXACT))
```

Valid recovery or diagnostic control

```
r = ConditionalResult(7, Certificate(True, Exactness.CONDITIONAL, assumptions_unchecked=('H',)))
assert r.certificate.exactness is Exactness.CONDITIONAL
```

Observed explicit source site(s): `XI-R0335` at `certificates.py:86`.

Implementation lookup: `komorebi_math.certificates.ConditionalResult`. **Executed receipt:** `diagnostic_replay/XI-D019/receipt.json`. Its hash is `9ef6f60afeb4b88cb3f862a800f43324bd0e3780edc41a082a15c8a544b3b049`. The source hashes and complete traceback are retained in the crosswalk and receipt.

XI-D020 / Nonfinite residual in certificate composition

Meaning. The composition layer refuses a nonfinite residual.

Recovery decision. Trace the numerical or producer failure rather than replacing a NaN with a fabricated passing residual. The zero example is an independent valid input.

Evidence mode: native finite input. **Observed exception:** `komorebi_math.exceptions.ValidationError`. Expected message fragment: `finite`.

Setup

```
from komorebi_math.certificates import Certificate, Exactness, ConditionalResult
from komorebi_math.expression.certification import compose_certificates
```

Trigger

```
compose_certificates([Certificate(True, Exactness.EXACT, residuals={'r': float('nan')})])
```

Valid recovery or diagnostic control

```
c = compose_certificates([Certificate(True, Exactness.EXACT, residuals={'r': 0.0})])
assert c.valid
```

Observed explicit source site(s): `XI-R1157` at `expression/certification.py:86`.

Implementation lookup: `komorebi_math.expression.certification._finite_float`. **Executed receipt:** `diagnostic_replay/XI-D020/receipt.json`. Its hash is `a8787f4728f6eada689cb75ce9b8935ae35dc07d1f94db1bf3fa7a96ebcfa6c6`. The source hashes and complete traceback are retained in the crosswalk and receipt.

XI-D021 / Numerical status without ExactnessError

Meaning. A successful numerical computation does not automatically raise ExactnessError when the caller hoped for exactness.

Recovery decision. Inspect the result evidence class and select a genuinely exact route when required.

Evidence mode: native finite input. No immediate exception is required. Observed fields: `{"exactness": "numerical", "valid": true}`.

Setup

```
from komorebi_math import PacketSystem
import numpy as np
from komorebi_math.certificates import Exactness
```

Trigger

```
c = PacketSystem([np.eye(2)]).certificate()
assert c.valid and c.exactness is Exactness.NUMERICAL
observation = {'valid': bool(c.valid), 'exactness': c.exactness.value}
```

Valid recovery or diagnostic control

```
assert c.exactness is not Exactness.EXACT
```

No explicit package raise site is attributed from this recipe's terminal traceback. This is expected for returned observations, warnings emitted through the warning machinery, dependency-only raises and nested CLI processes.

Implementation lookup: `komorebi_math.core.packets.PacketSystem.certificate`, `komorebi_math.exceptions.ExactnessError`. **Executed receipt:** `diagnostic_replay/XI-D021/receipt.json`. Its hash is `75f4c5cdd2b643eb659935dc8726a1971b1e39f3f930d1eddd6d565a2ea81b4f`. The source hashes and complete traceback are retained in the crosswalk and receipt.

Failed assumptions

XI-D022 / Negative prior mass

Meaning. Input probability mass must be finite and nonnegative.

Recovery decision. Reconcile the likelihood, state space and prior from the actual model. Do not add unsupported mass solely to pass the check.

Evidence mode: native finite input. **Observed exception:** `komorebi_math.exceptions.ValidationError`. Expected message fragment: `nonnegative`.

Setup

```
from komorebi_math.dynamics.bayes import BayesianUpdate
import numpy as np
```

Trigger

```
BayesianUpdate([-1, 2], [1, 1])
```

Valid recovery or diagnostic control

```
u = BayesianUpdate([0.5, 0.5], [1, 2])
assert np.allclose(u.posterior, [1 / 3, 2 / 3])
```

Observed explicit source site(s): `XI-R0906` at `dynamics/bayes.py:21`.

Implementation lookup: `komorebi_math.dynamics.bayes.BayesianUpdate`, `komorebi_math.dynamics.bayes._probability`, `komorebi_math.dynamics.bayes._likelihood`. **Executed receipt:** `diagnostic_replay/XI-D022/receipt.json`. Its hash is `be958a22ff31d659bc9a8f09bbce1328dab0a53c0e6cc83cf34d2876dbf9cf1f`. The source hashes and complete traceback are retained in the crosswalk and receipt.

XI-D023 / Zero evidence despite nonzero likelihood

Meaning. The likelihood is nonzero only where the prior is zero. Construction is possible; evaluating posterior enforces positive evidence.

Recovery decision. Reconcile the likelihood, state space and prior from the actual model. Do not add unsupported mass solely to pass the check.

Evidence mode: native finite input. **Observed exception:** `komorebi_math.exceptions.AssumptionError`. Expected message fragment: `zero evidence`.

Setup

```
from komorebi_math.dynamics.bayes import BayesianUpdate
import numpy as np
```

Trigger

```
BayesianUpdate([1, 0], [0, 1]).posterior
```

Valid recovery or diagnostic control

```
u = BayesianUpdate([0.5, 0.5], [1, 2])
assert np.allclose(u.posterior, [1 / 3, 2 / 3])
```

Observed explicit source site(s): `XI-R0912` at `dynamics/bayes.py:78`.

Implementation lookup: `komorebi_math.dynamics.bayes.BayesianUpdate`, `komorebi_math.dynamics.bayes._probability`, `komorebi_math.dynamics.bayes._likelihood`. **Executed receipt:** `diagnostic_replay/XI-D023/receipt.json`. Its hash is

`99dc3d089cacb780408165e77fffcfc372e215291ef217f74ab930b7e9946714`. The source hashes and complete traceback are retained in the crosswalk and receipt.

XI-D024 / Likelihood dimension mismatch

Meaning. Likelihood and prior must refer to the same state space.

Recovery decision. Reconcile the likelihood, state space and prior from the actual model. Do not add unsupported mass solely to pass the check.

Evidence mode: native finite input. **Observed exception:** `komorebi_math.exceptions.DimensionError`. Expected message fragment: `dimension`.

Setup

```
from komorebi_math.dynamics.bayes import BayesianUpdate
import numpy as np
```

Trigger

```
BayesianUpdate([0.5, 0.5], [1])
```

Valid recovery or diagnostic control

```
u = BayesianUpdate([0.5, 0.5], [1, 2])
assert np.allclose(u.posterior, [1 / 3, 2 / 3])
```

Observed explicit source site(s): `XI-R0908` at `dynamics/bayes.py:31`.

Implementation lookup: `komorebi_math.dynamics.bayes.BayesianUpdate`, `komorebi_math.dynamics.bayes._probability`, `komorebi_math.dynamics.bayes._likelihood`. **Executed receipt:** `diagnostic_replay/XI-D024/receipt.json`. Its hash is `25fb13ed46caec70cdd6c000ddddfab6e7b7e5794383c22bbe5527bbfeb0ca69`. The source hashes and complete traceback are retained in the crosswalk and receipt.

XI-D025 / All-zero likelihood

Meaning. The constructor rejects an everywhere-zero likelihood before posterior evaluation.

Recovery decision. Reconcile the likelihood, state space and prior from the actual model. Do not add unsupported mass solely to pass the check.

Evidence mode: native finite input. **Observed exception:** `komorebi_math.exceptions.ValidationError`. Expected message fragment: `positive on at least one`.

Setup

```
from komorebi_math.dynamics.bayes import BayesianUpdate
import numpy as np
```

Trigger

```
BayesianUpdate([0.5, 0.5], [0, 0])
```

Valid recovery or diagnostic control

```
u = BayesianUpdate([0.5, 0.5], [1, 2])
assert np.allclose(u.posterior, [1 / 3, 2 / 3])
```

Observed explicit source site(s): `XI-R0910` at `dynamics/bayes.py:35`.

Implementation lookup: `komorebi_math.dynamics.bayes.BayesianUpdate`, `komorebi_math.dynamics.bayes._probability`, `komorebi_math.dynamics.bayes._likelihood`. **Executed receipt:** `diagnostic_replay/XI-D025/receipt.json`. Its hash is

`2db8a7cfcc3f88f8ca2c4ec999fbe612bb82ee65d0f0449a1fe1c60c2f1c0a70`. The source hashes and complete traceback are retained in the crosswalk and receipt.

Resource refusal

XI-D026 / Budget refusal: maximum_degree

Meaning. The explicit request exceeds this local budget field before the guarded work.

Recovery decision. Reduce the requested work or justify a measured larger limit. A larger cap is not evidence that memory is available.

Evidence mode: native finite input. **Observed exception:** `komorebi_math.exceptions.ResourceLimitError`. Expected message fragment: `maximum_degree`.

Setup

```
from komorebi_math.generating.budget import GeneratingBudget
b = GeneratingBudget(maximum_degree=4)
```

Trigger

```
b.require_degree(5)
```

Valid recovery or diagnostic control

```
assert b.require_degree(4) == 4
```

Observed explicit source site(s): `XI-R1526` at `generating/budget.py:52`.

Implementation lookup: `komorebi_math.generating.budget.GeneratingBudget.require_degree`. **Executed receipt:** `diagnostic_replay/XI-D026/receipt.json`. Its hash is `ccde16281aa5affffbfd2aa71b1f9ec48fae119def7b1c1bd486af82b04ad7c4`. The source hashes and complete traceback are retained in the crosswalk and receipt.

XI-D027 / Budget refusal: maximum_local_combinations

Meaning. The explicit request exceeds this local budget field before the guarded work.

Recovery decision. Reduce the requested work or justify a measured larger limit. A larger cap is not evidence that memory is available.

Evidence mode: native finite input. **Observed exception:** `komorebi_math.exceptions.ResourceLimitError`. Expected message fragment: `maximum_local_combinations`.

Setup

```
from komorebi_math.generating.budget import GeneratingBudget
b = GeneratingBudget(maximum_local_combinations=4)
```

Trigger

```
b.require_terms(5)
```

Valid recovery or diagnostic control

```
assert b.require_terms(4) == 4
```

Observed explicit source site(s): `XI-R1528` at `generating/budget.py:64`.

Implementation lookup: `komorebi_math.generating.budget.GeneratingBudget.require_terms`. **Executed receipt:** `diagnostic_replay/XI-D027/receipt.json`. Its hash is `b67accf1f331f622b1232f688f9198eb592e5ec00394e38272e96a47caf32158`. The source hashes and complete traceback are retained in the crosswalk and receipt.

XI-D028 / Budget refusal: maximum_matrix_dimension

Meaning. The explicit request exceeds this local budget field before the guarded work.

Recovery decision. Reduce the requested work or justify a measured larger limit. A larger cap is not evidence that memory is available.

Evidence mode: native finite input. **Observed exception:** `komorebi_math.exceptions.ResourceLimitError`. Expected message fragment: `maximum_matrix_dimension`.

Setup

```
from komorebi_math.generating.budget import GeneratingBudget
b = GeneratingBudget(maximum_matrix_dimension=4)
```

Trigger

```
b.require_matrix_dimension(5)
```

Valid recovery or diagnostic control

```
assert b.require_matrix_dimension(4) == 4
```

Observed explicit source site(s): `XI-R1529` at `generating/budget.py:81`.

Implementation lookup: `komorebi_math.generating.budget.GeneratingBudget.require_matrix_dimension`. **Executed receipt:** `diagnostic_replay/XI-D028/receipt.json`. Its hash is `050fed0bd969dd0d2e3245126d97371a241c64f7e786596da5a02ba5bfdcc332`. The source hashes and complete traceback are retained in the crosswalk and receipt.

XI-D029 / Budget refusal: maximum_reachable_states

Meaning. The explicit request exceeds this local budget field before the guarded work.

Recovery decision. Reduce the requested work or justify a measured larger limit. A larger cap is not evidence that memory is available.

Evidence mode: native finite input. **Observed exception:** `komorebi_math.exceptions.ResourceLimitError`. Expected message fragment: `maximum_reachable_states`.

Setup

```
from komorebi_math.generating.budget import GeneratingBudget
b = GeneratingBudget(maximum_reachable_states=4)
```

Trigger

```
b.require_states(5)
```

Valid recovery or diagnostic control

```
assert b.require_states(4) == 4
```

Observed explicit source site(s): `XI-R1530` at `generating/budget.py:92`.

Implementation lookup: `komorebi_math.generating.budget.GeneratingBudget.require_states`. **Executed receipt:** `diagnostic_replay/XI-D029/receipt.json`. Its hash is `448519ba87a85f93b611d082a7d1239eaf53ff45c3d55f274adda2ad25fb1d74`. The source hashes and complete traceback are retained in the crosswalk and receipt.

XI-D030 / Budget refusal: maximum_local_combinations

Meaning. The explicit request exceeds this local budget field before the guarded work.

Recovery decision. Reduce the requested work or justify a measured larger limit. A larger cap is not evidence that memory is available.

Evidence mode: native finite input. **Observed exception:** `komorebi_math.exceptions.ResourceLimitError`. Expected message fragment: `maximum_local_combinations`.

Setup

```
from komorebi_math.generating.budget import GeneratingBudget
b = GeneratingBudget(maximum_local_combinations=4)
```

Trigger

```
b.require_local_combinations(5)
```

Valid recovery or diagnostic control

```
assert b.require_local_combinations(4) == 4
```

Observed explicit source site(s): `XI-R1531` at `generating/budget.py:103`.

Implementation lookup: `komorebi_math.generating.budget.GeneratingBudget.require_local_combinations`. **Executed receipt:** `diagnostic_replay/XI-D030/receipt.json`. Its hash is `7b0362cf51b3ecaa60198e65be854855066fccc53f3a618a7f88c1e7401ea10f`. The source hashes and complete traceback are retained in the crosswalk and receipt.

XI-D031 / Budget refusal: maximum_modulus

Meaning. The explicit request exceeds this local budget field before the guarded work.

Recovery decision. Reduce the requested work or justify a measured larger limit. A larger cap is not evidence that memory is available.

Evidence mode: native finite input. **Observed exception:** `komorebi_math.exceptions.ResourceLimitError`. Expected message fragment: `maximum_modulus`.

Setup

```
from komorebi_math.generating.budget import GeneratingBudget
b = GeneratingBudget(maximum_modulus=4)
```

Trigger

```
b.require_modulus(5)
```

Valid recovery or diagnostic control

```
assert b.require_modulus(4) == 4
```

Observed explicit source site(s): `XI-R1532` at `generating/budget.py:114`.

Implementation lookup: `komorebi_math.generating.budget.GeneratingBudget.require_modulus`. **Executed receipt:** `diagnostic_replay/XI-D031/receipt.json`. Its hash is `f0e7defdf230501571b6a900470c9ef93519c791042116b129184d48131cec73`. The source hashes and complete traceback are retained in the crosswalk and receipt.

XI-D032 / Budget refusal: maximum_serialized_bytes

Meaning. The explicit request exceeds this local budget field before the guarded work.

Recovery decision. Reduce the requested work or justify a measured larger limit. A larger cap is not evidence that memory is available.

Evidence mode: native finite input. **Observed exception:** `komorebi_math.exceptions.ResourceLimitError`. Expected message fragment: `maximum_serialized_bytes`.

Setup

```
from komorebi_math.generating.budget import GeneratingBudget
b = GeneratingBudget(maximum_serialized_bytes=4)
```

Trigger

```
b.require_serialized_bytes(5)
```

Valid recovery or diagnostic control

```
assert b.require_serialized_bytes(4) == 4
```

Observed explicit source site(s): `XI-R1534` at `generating/budget.py:125`.

Implementation lookup: `komorebi_math.generating.budget.GeneratingBudget.require_serialized_bytes`. **Executed receipt:** `diagnostic_replay/XI-D032/receipt.json`. Its hash is `62475796e4e28dad65f1562576675faf2981e36c27285ee109b8942be65e6095`. The source hashes and complete traceback are retained in the crosswalk and receipt.

XI-D033 / Budget refusal: maximum_coefficient_bits

Meaning. The explicit request exceeds this local budget field before the guarded work.

Recovery decision. Reduce the requested work or justify a measured larger limit. A larger cap is not evidence that memory is available.

Evidence mode: native finite input. **Observed exception:** `komorebi_math.exceptions.ResourceLimitError`. Expected message fragment: `maximum_coefficient_bits`.

Setup

```
from komorebi_math.generating.budget import GeneratingBudget
b = GeneratingBudget(maximum_coefficient_bits=4)
```

Trigger

```
b.require_coefficient_bits(5)
```

Valid recovery or diagnostic control

```
assert b.require_coefficient_bits(4) == 4
```

Observed explicit source site(s): `XI-R1536` at `generating/budget.py:137`.

Implementation lookup: `komorebi_math.generating.budget.GeneratingBudget.require_coefficient_bits`. **Executed receipt:** `diagnostic_replay/XI-D033/receipt.json`. Its hash is `c987ecb4ced07d7beb1bba4a0d29584144bbc0d9741131bc15ca5ff238e22952`. The source hashes and complete traceback are retained in the crosswalk and receipt.

XI-D034 / Boolean is not a truncation degree

Meaning. The budget route deliberately excludes bool even though it is an int subclass.

Recovery decision. Pass an explicit nonnegative integer request.

Evidence mode: native finite input. **Observed exception:** `komorebi_math.exceptions.ValidationError`. Expected message fragment: `nonnegative integer`.

Setup

```
from komorebi_math.generating.budget import GeneratingBudget
b = GeneratingBudget()
```

Trigger

```
b.require_degree(True)
```

Valid recovery or diagnostic control

```
assert b.require_degree(0) == 0
```

Observed explicit source site(s): `XI-R1525` at `generating/budget.py:50`.

Implementation lookup: `komorebi_math.generating.budget.GeneratingBudget.require_degree`. **Executed receipt:** `diagnostic_replay/XI-D034/receipt.json`. Its hash is `1f303ce5e4947f2d5fb0d3b51b9c422c2eb1510dac20488050c179c5a2a80422`. The source hashes and complete traceback are retained in the crosswalk and receipt.

XI-D035 / Invalid budget declaration

Meaning. Limit fields must be positive integers.

Recovery decision. Set a positive bound. A zero request and a zero configured ceiling are different contracts.

Evidence mode: native finite input. **Observed exception:** `komorebi_math.exceptions.ValidationError`. Expected message fragment: `positive integer`.

Setup

```
from komorebi_math.generating.budget import GeneratingBudget
```

Trigger

```
GeneratingBudget(maximum_degree=0)
```

Valid recovery or diagnostic control

```
assert GeneratingBudget(maximum_degree=4).maximum_degree == 4
```

Observed explicit source site(s): `XI-R1524` at `generating/budget.py:12`.

Implementation lookup: `komorebi_math.generating.budget._positive`. **Executed receipt:** `diagnostic_replay/XI-D035/receipt.json`. Its hash is `84754ec75029a59751d82b18a6e853f973c5dc359aaccb59b02849ba80df299d`. The source hashes and complete traceback are retained in the crosswalk and receipt.

Planned feature

XI-D036 / Declared planned feature (controlled registry fixture)

Meaning. The reserved-namespace guard raises for a controlled planned registry entry. No existing feature is claimed to be planned in R2.

Recovery decision. Use an implemented capability or leave the task unavailable. Changing fixture metadata exercises the guard only; it does not implement a real feature.

Evidence mode: synthetic registry entry in a fresh child process. **Observed exception:** `komorebi_math.exceptions.FeatureNotImplementedError`. Expected message fragment: `declared as planned`.

Setup

```
from komorebi_math.registry import FeatureSpec, FeatureStatus, register_feature
from komorebi_math._planned import require_implemented
register_feature(FeatureSpec('block41-demo', 'example.namespace', 'Controlled planned feature',
    ↪  FeatureStatus.PLANNED, 'test fixture'))
```

Trigger

```
require_implemented('block41-demo')
```

Valid recovery or diagnostic control

```
register_feature(FeatureSpec('block41-demo', 'example.namespace', 'Controlled implemented fixture',
    ↪  FeatureStatus.IMPLEMENTED, 'test fixture'), replace=True)
assert require_implemented('block41-demo') is None
```

Observed explicit source site(s): `XI-R0004` at `_planned.py:17`.

Implementation lookup: `komorebi_math._planned.require_implemented`. **Executed receipt:** `diagnostic_replay/XI-D036/receipt.json`. Its hash is `958e42c960e557adac172616012b432e189526211f952c30e0cd5503694d2220`. The source hashes and complete traceback are retained in the crosswalk and receipt.

Persistence, schema and tamper failure

XI-D037 / Malformed JSON

Meaning. The JSON parser failure is wrapped as ValidationError with its cause.

Recovery decision. Keep the failing artifact for diagnosis. Restore or regenerate from a trusted producer and supported schema; do not patch a digest merely to conceal corruption. The successful control uses the unchanged valid envelope.

Evidence mode: native finite input. **Observed exception:** `komorebi_math.exceptions.ValidationError`. Expected message fragment: `valid UTF-8 JSON`. **Explicit cause:** `json.decoder.JSONDecodeError`.

Setup

```
import json
from fractions import Fraction
from komorebi_math.export.persistence import dumps_portable, loads_portable, to_portable_data, from_portable_data,
    ↪  save_portable, load_portable
good = dumps_portable(Fraction(1, 3))
```

Trigger

```
loads_portable('{')
```

Valid recovery or diagnostic control

```
assert loads_portable(good) == Fraction(1, 3)
```

Observed explicit source site(s): `XI-R1116` at `export/persistence.py:805`.

Implementation lookup: `komorebi_math.export.persistence`. **Executed receipt:** `diagnostic_replay/XI-D037/receipt.json`. Its hash is `04f6926a86370fe19b8e6e6b7e5adb8e756022c0311eac53f0ac5653f157bdc0`. The source hashes and complete traceback are retained in the crosswalk and receipt.

XI-D038 / Invalid UTF-8

Meaning. Byte decoding fails before portable envelope validation.

Recovery decision. Keep the failing artifact for diagnosis. Restore or regenerate from a trusted producer and supported schema; do not patch a digest merely to conceal corruption. The successful control uses the unchanged valid envelope.

Evidence mode: native finite input. **Observed exception:** `komorebi_math.exceptions.ValidationError`. Expected message fragment: `valid UTF-8 JSON`. **Explicit cause:** `builtins.UnicodeDecodeError`.

Setup

```
import json
from fractions import Fraction
from komorebi_math.export.persistence import dumps_portable, loads_portable, to_portable_data, from_portable_data,
    ↪  save_portable, load_portable
good = dumps_portable(Fraction(1, 3))
```

Trigger

```
loads_portable(b'\xff')
```

Valid recovery or diagnostic control

```
assert loads_portable(good) == Fraction(1, 3)
```

Observed explicit source site(s): `XI-R1116` at `export/persistence.py:805`.

Implementation lookup: `komorebi_math.export.persistence`. **Executed receipt:** `diagnostic_replay/XI-D038/receipt.json`. Its hash is `76df98947003d3f7fb862ef52ac02146e96e2e758f4434bc2fb5a284b32967bd`. The source hashes and complete traceback are retained in the crosswalk and receipt.

XI-D039 / Duplicate JSON object key

Meaning. Duplicate keys are rejected before envelope comparison.

Recovery decision. Keep the failing artifact for diagnosis. Restore or regenerate from a trusted producer and supported schema; do not patch a digest merely to conceal corruption. The successful control uses the unchanged valid envelope.

Evidence mode: native finite input. **Observed exception:** `komorebi_math.exceptions.ValidationError`. Expected message fragment: `duplicate object key`.

Setup

```
import json
from fractions import Fraction
from komorebi_math.export.persistence import dumps_portable, loads_portable, to_portable_data, from_portable_data,
    ↪  save_portable, load_portable
good = dumps_portable(Fraction(1, 3))
```

Trigger

```
loads_portable('{"x":1,"x":2}')
```

Valid recovery or diagnostic control

```
assert loads_portable(good) == Fraction(1, 3)
```

Observed explicit source site(s): `XI-R1076` at `export/persistence.py:509`.

Implementation lookup: `komorebi_math.export.persistence`. **Executed receipt:** `diagnostic_replay/XI-D039/receipt.json`. Its hash is `3c7a169658b858983b8e38a5bf9026ec91c0ef8bb8f0633d8723824755b62a9b`. The source hashes and complete traceback are retained in the crosswalk and receipt.

XI-D040 / Nonfinite JSON constant

Meaning. Nonstandard nonfinite numeric tokens are rejected.

Recovery decision. Keep the failing artifact for diagnosis. Restore or regenerate from a trusted producer and supported schema; do not patch a digest merely to conceal corruption. The successful control uses the unchanged valid envelope.

Evidence mode: native finite input. **Observed exception:** `komorebi_math.exceptions.ValidationError`. Expected message fragment: `nonfinite constant`.

Setup

```
import json
from fractions import Fraction
from komorebi_math.export.persistence import dumps_portable, loads_portable, to_portable_data, from_portable_data,
    ↪  save_portable, load_portable
good = dumps_portable(Fraction(1, 3))
```

Trigger

```
loads_portable('{"x":NaN}')
```

Valid recovery or diagnostic control

```
assert loads_portable(good) == Fraction(1, 3)
```

Observed explicit source site(s): `XI-R1077` at `export/persistence.py:517`.

Implementation lookup: `komorebi_math.export.persistence`. **Executed receipt:** `diagnostic_replay/XI-D040/receipt.json`. Its hash is `be19394b3a81984ebc9fc535be6b10693c682c124b7c114ab6550b268368e508`. The source hashes and complete traceback are retained in the crosswalk and receipt.

XI-D041 / Missing envelope field

Meaning. Every required outer field must be present.

Recovery decision. Keep the failing artifact for diagnosis. Restore or regenerate from a trusted producer and supported schema; do not patch a digest merely to conceal corruption. The successful control uses the unchanged valid envelope.

Evidence mode: native finite input. **Observed exception:** `komorebi_math.exceptions.ValidationError`. Expected message fragment: `fields do not match`.

Setup

```
import json
from fractions import Fraction
from komorebi_math.export.persistence import dumps_portable, loads_portable, to_portable_data, from_portable_data,
    ↪  save_portable, load_portable
good = dumps_portable(Fraction(1, 3))
e = json.loads(good)
e.pop('producer_version')
```

Trigger

```
loads_portable(json.dumps(e))
```

Valid recovery or diagnostic control

```
assert loads_portable(good) == Fraction(1, 3)
```

Observed explicit source site(s): `XI-R1118` at `export/persistence.py:810`.

Implementation lookup: `komorebi_math.export.persistence`. **Executed receipt:** `diagnostic_replay/XI-D041/receipt.json`. Its hash is `e311b01f44d27334eb2a2e6ce817bc09d94d188b34f0d9483bc7d3f8454a13b9`. The source hashes and complete traceback are retained in the crosswalk and receipt.

XI-D042 / Unknown outer schema

Meaning. The outer schema identifies the decoding contract.

Recovery decision. Keep the failing artifact for diagnosis. Restore or regenerate from a trusted producer and supported schema; do not patch a digest merely to conceal corruption. The successful control uses the unchanged valid envelope.

Evidence mode: native finite input. **Observed exception:** `komorebi_math.exceptions.ValidationError`. Expected message fragment: `unknown schema`.

Setup

```
import json
from fractions import Fraction
from komorebi_math.export.persistence import dumps_portable, loads_portable, to_portable_data, from_portable_data,
    ↪  save_portable, load_portable
good = dumps_portable(Fraction(1, 3))
e = json.loads(good)
e['schema'] = 'other'
```

Trigger

```
loads_portable(json.dumps(e))
```

Valid recovery or diagnostic control

```
assert loads_portable(good) == Fraction(1, 3)
```

Observed explicit source site(s): `XI-R1119` at `export/persistence.py:812`.

Implementation lookup: `komorebi_math.export.persistence`. **Executed receipt:** `diagnostic_replay/XI-D042/receipt.json`. Its hash is `58bd00880133b40b4404cbf6378fe09b8c81b99c910643f467b14fee04a08dba`. The source hashes and complete traceback are retained in the crosswalk and receipt.

XI-D043 / Unsupported outer version

Meaning. This reader does not advertise arbitrary forward compatibility.

Recovery decision. Keep the failing artifact for diagnosis. Restore or regenerate from a trusted producer and supported schema; do not patch a digest merely to conceal corruption. The successful control uses the unchanged valid envelope.

Evidence mode: native finite input. **Observed exception:** `komorebi_math.exceptions.ValidationError`. Expected message fragment: `version is unsupported`.

Setup

```
import json
from fractions import Fraction
from komorebi_math.export.persistence import dumps_portable, loads_portable, to_portable_data, from_portable_data,
    ↪  save_portable, load_portable
good = dumps_portable(Fraction(1, 3))
e = json.loads(good)
e['schema_version'] = 99
```

Trigger

```
loads_portable(json.dumps(e))
```

Valid recovery or diagnostic control

```
assert loads_portable(good) == Fraction(1, 3)
```

Observed explicit source site(s): `XI-R1120` at `export/persistence.py:814`.

Implementation lookup: `komorebi_math.export.persistence`. **Executed receipt:** `diagnostic_replay/XI-D043/receipt.json`. Its hash is `dc93356969cba0e6767effd19fd8104845e4d35ac0383ab888592bb239719070`. The source hashes and complete traceback are retained in the crosswalk and receipt.

XI-D044 / Payload checksum mismatch

Meaning. The decoded payload no longer matches its recorded digest.

Recovery decision. Keep the failing artifact for diagnosis. Restore or regenerate from a trusted producer and supported schema; do not patch a digest merely to conceal corruption. The successful control uses the unchanged valid envelope.

Evidence mode: native finite input. **Observed exception:** `komorebi_math.exceptions.ValidationError`. Expected message fragment: `SHA256 mismatch`.

Setup

```
import json
from fractions import Fraction
from komorebi_math.export.persistence import dumps_portable, loads_portable, to_portable_data, from_portable_data,
    ↪  save_portable, load_portable
good = dumps_portable(Fraction(1, 3))
e = json.loads(good)
e['payload'] = 123
```

Trigger

```
loads_portable(json.dumps(e))
```

Valid recovery or diagnostic control

```
assert loads_portable(good) == Fraction(1, 3)
```

Observed explicit source site(s): `XI-R1122` at `export/persistence.py:823`.

Implementation lookup: `komorebi_math.export.persistence`. **Executed receipt:** `diagnostic_replay/XI-D044/receipt.json`. Its hash is `38df18d64ee4bbf4cf4434e89c41f0b7dc39da4c6615ca9bc1f1c02759e880b3`. The source hashes and complete traceback are retained in the crosswalk and receipt.

XI-D045 / Unknown portable tag

Meaning. Low-level tagged decoding rejects an unregistered tag.

Recovery decision. Keep the failing artifact for diagnosis. Restore or regenerate from a trusted producer and supported schema; do not patch a digest merely to conceal corruption. The successful control uses the unchanged valid envelope.

Evidence mode: native finite input. **Observed exception:** `komorebi_math.exceptions.ValidationError`. Expected message fragment: `unknown portable codec`. **Explicit cause:** `builtins.KeyError`.

Setup

```
import json
from fractions import Fraction
from komorebi_math.export.persistence import dumps_portable, loads_portable, to_portable_data, from_portable_data,
    ↪  save_portable, load_portable
good = dumps_portable(Fraction(1, 3))
```

Trigger

```
from_portable_data({'__komorebi_type__': 'missing.tag', '__version__': 1, 'data': {}})
```

Valid recovery or diagnostic control

```
assert loads_portable(good) == Fraction(1, 3)
```

Observed explicit source site(s): `XI-R1113` at `export/persistence.py:735`.

Implementation lookup: `komorebi_math.export.persistence`. **Executed receipt:** `diagnostic_replay/XI-D045/receipt.json`. Its hash is `30cff219e5e2db50294fde3b8df32c15f87f59589f0cbd018fdec3f6ee9da5c9`. The source hashes and complete traceback are retained in the crosswalk and receipt.

XI-D046 / Unsupported future codec version

Meaning. A supported type is not permission to accept unknown versions.

Recovery decision. Keep the failing artifact for diagnosis. Restore or regenerate from a trusted producer and supported schema; do not patch a digest merely to conceal corruption. The successful control uses the unchanged valid envelope.

Evidence mode: native finite input. **Observed exception:** `komorebi_math.exceptions.ValidationError`. Expected message fragment: `unsupported Fraction codec version`.

Setup

```
import json
from fractions import Fraction
from komorebi_math.export.persistence import dumps_portable, loads_portable, to_portable_data, from_portable_data,
    ↪  save_portable, load_portable
good = dumps_portable(Fraction(1, 3))
d = to_portable_data(Fraction(1, 3))
d['__version__'] = 99
```

Trigger

```
from_portable_data(d)
```

Valid recovery or diagnostic control

```
assert loads_portable(good) == Fraction(1, 3)
```

Observed explicit source site(s): `XI-R1086` at `export/persistence.py:639`.

Implementation lookup: `komorebi_math.export.persistence`. **Executed receipt:** `diagnostic_replay/XI-D046/receipt.json`. Its hash is `be014735a817e304a5f0fbc6ccacf4b7782488ec6656ad6934102a0cc6e04cb8`. The source hashes and complete traceback are retained in the crosswalk and receipt.

XI-D047 / Invalid rational denominator

Meaning. Rational reconstruction rejects a zero denominator and preserves its cause.

Recovery decision. Keep the failing artifact for diagnosis. Restore or regenerate from a trusted producer and supported schema; do not patch a digest merely to conceal corruption. The successful control uses the unchanged valid envelope.

Evidence mode: native finite input. **Observed exception:** `komorebi_math.exceptions.ValidationError`. Expected message fragment: `Fraction data is invalid`. **Explicit cause:** `builtins.ZeroDivisionError`.

Setup

```
import json
from fractions import Fraction
from komorebi_math.export.persistence import dumps_portable, loads_portable, to_portable_data, from_portable_data,
    ↪  save_portable, load_portable
good = dumps_portable(Fraction(1, 3))
d = to_portable_data(Fraction(1, 3))
d['data']['denominator'] = 0
```

Trigger

```
from_portable_data(d)
```

Valid recovery or diagnostic control

```
assert loads_portable(good) == Fraction(1, 3)
```

Observed explicit source site(s): `XI-R1088` at `export/persistence.py:645`.

Implementation lookup: `komorebi_math.export.persistence`. **Executed receipt:** `diagnostic_replay/XI-D047/receipt.json`. Its hash is `f1edd884252ae290d77de13ec2566ab823697f61137169bbade6451dd093f78d`. The source hashes and complete traceback are retained in the crosswalk and receipt.

XI-D048 / Serialization depth exceeded

Meaning. The local recursion limit is checked by the encoder.

Recovery decision. Keep the failing artifact for diagnosis. Restore or regenerate from a trusted producer and supported schema; do not patch a digest merely to conceal corruption. The successful control uses the unchanged valid envelope.

Evidence mode: native finite input. **Observed exception:** `komorebi_math.exceptions.ValidationError`. Expected message fragment: `serialization depth exceeded`.

Setup

```
import json
from fractions import Fraction
from komorebi_math.export.persistence import dumps_portable, loads_portable, to_portable_data, from_portable_data,
    ↪  save_portable, load_portable
good = dumps_portable(Fraction(1, 3))
```

Trigger

```
to_portable_data([[[1]]], max_depth=1)
```

Valid recovery or diagnostic control

```
assert loads_portable(good) == Fraction(1, 3)
```

Observed explicit source site(s): `XI-R1078` at `export/persistence.py:522`.

Implementation lookup: `komorebi_math.export.persistence`. **Executed receipt:** `diagnostic_replay/XI-D048/receipt.json`. Its hash is `5ea3ea69112e3df0599c82e6f96e612fc3bcc494d2db804f1818681d6b643dd4`. The source hashes and complete traceback are retained in the crosswalk and receipt.

XI-D049 / Deserialization depth exceeded

Meaning. The decoder rejects the nested tree at its local depth limit.

Recovery decision. Keep the failing artifact for diagnosis. Restore or regenerate from a trusted producer and supported schema; do not patch a digest merely to conceal corruption. The successful control uses the unchanged valid envelope.

Evidence mode: native finite input. **Observed exception:** `komorebi_math.exceptions.ValidationError`. Expected message fragment: `deserialization depth exceeded`.

Setup

```
import json
from fractions import Fraction
from komorebi_math.export.persistence import dumps_portable, loads_portable, to_portable_data, from_portable_data,
    ↪  save_portable, load_portable
good = dumps_portable(Fraction(1, 3))
```

Trigger

```
from_portable_data([[[1]]], max_depth=1)
```

Valid recovery or diagnostic control

```
assert loads_portable(good) == Fraction(1, 3)
```

Observed explicit source site(s): `XI-R1082` at `export/persistence.py:614`.

Implementation lookup: `komorebi_math.export.persistence`. **Executed receipt:** `diagnostic_replay/XI-D049/receipt.json`. Its hash is `43bb8d56f5e4bff16f232502517bd5351da31abfc179373bc613e100d36d8e7a`. The source hashes and complete traceback are retained in the crosswalk and receipt.

XI-D050 / Nonfinite float serialization

Meaning. The writer refuses nonfinite scalar values.

Recovery decision. Keep the failing artifact for diagnosis. Restore or regenerate from a trusted producer and supported schema; do not patch a digest merely to conceal corruption. The successful control uses the unchanged valid envelope.

Evidence mode: native finite input. **Observed exception:** `komorebi_math.exceptions.ValidationError`. Expected message fragment: `nonfinite floats`.

Setup

```
import json
from fractions import Fraction
from komorebi_math.export.persistence import dumps_portable, loads_portable, to_portable_data, from_portable_data,
    ↪  save_portable, load_portable
good = dumps_portable(Fraction(1, 3))
```

Trigger

```
to_portable_data(float('nan'))
```

Valid recovery or diagnostic control

```
assert loads_portable(good) == Fraction(1, 3)
```

Observed explicit source site(s): `XI-R1079` at `export/persistence.py:537`.

Implementation lookup: `komorebi_math.export.persistence`. **Executed receipt:** `diagnostic_replay/XI-D050/receipt.json`. Its hash is `71b024d24d7e6dfeb981ec7b1d9778e250a59fada145dbf5d200fc0322585fb4`. The source hashes and complete traceback are retained in the crosswalk and receipt.

XI-D051 / Unregistered object serialization

Meaning. Only supported values and explicitly registered codecs are encoded.

Recovery decision. Keep the failing artifact for diagnosis. Restore or regenerate from a trusted producer and supported schema; do not patch a digest merely to conceal corruption. The successful control uses the unchanged valid envelope.

Evidence mode: native finite input. **Observed exception:** `komorebi_math.exceptions.ValidationError`. Expected message fragment: `portable`.

Setup

```
import json
from fractions import Fraction
from komorebi_math.export.persistence import dumps_portable, loads_portable, to_portable_data, from_portable_data,
    ↪ save_portable, load_portable
good = dumps_portable(Fraction(1, 3))
```

Trigger

```
to_portable_data(object())
```

Valid recovery or diagnostic control

```
assert loads_portable(good) == Fraction(1, 3)
```

Observed explicit source site(s): `XI-R1081` at `export/persistence.py:607`.

Implementation lookup: `komorebi_math.export.persistence`. **Executed receipt:** `diagnostic_replay/XI-D051/receipt.json`. Its hash is `1f55aa5f66b856d76bf73c68e7ad707e731a81321111acf5a786cec70b1bf708`. The source hashes and complete traceback are retained in the crosswalk and receipt.

XI-D052 / Portable file cannot be read

Meaning. File I/O errors are translated to ValidationError and retain an OSError cause.

Recovery decision. Check the actual path and read permissions, retaining the cause. A parse fix will not repair a missing file.

Evidence mode: native finite input. **Observed exception:** `komorebi_math.exceptions.ValidationError`. Expected message fragment: `could not read`. **Explicit cause:** `builtins.FileNotFoundError`.

Setup

```
import json
from fractions import Fraction
from komorebi_math.export.persistence import dumps_portable, loads_portable, to_portable_data, from_portable_data,
    ↪ save_portable, load_portable
good = dumps_portable(Fraction(1, 3))
```

Trigger

```
load_portable(work / 'missing.json')
```

Valid recovery or diagnostic control

```
save_portable(Fraction(1, 3), work / 'valid.json')
assert load_portable(work / 'valid.json') == Fraction(1, 3)
```

Observed explicit source site(s): `XI-R1125` at `export/persistence.py:867`.

Implementation lookup: `komorebi_math.export.persistence.load_portable`. **Executed receipt:** `diagnostic_replay/XI-D052/receipt.json`. Its hash is `7b6bf722a83bdca62cb880444bc86f7e47efe79f029f694a70f380d2bcc6a9b7`. The source hashes and complete traceback are retained in the crosswalk and receipt.

XI-D053 / Protected portable destination

Meaning. The existing file is protected before encoding and replacement.

Recovery decision. Choose a new destination, or make an explicit reviewed overwrite decision after preserving the old file.

Evidence mode: native finite input. **Observed exception:** `komorebi_math.exceptions.ValidationError`. Expected message fragment: `refusing to overwrite`.

Setup

```
import json
from fractions import Fraction
from komorebi_math.export.persistence import dumps_portable, loads_portable, to_portable_data, from_portable_data,
    ↪  save_portable, load_portable
good = dumps_portable(Fraction(1, 3))
p = work / 'existing.json'
p.write_text('sentinel')
```

Trigger

```
save_portable(Fraction(1, 3), p)
```

Valid recovery or diagnostic control

```
assert p.read_text() == 'sentinel'
save_portable(Fraction(1, 3), work / 'new.json')
assert load_portable(work / 'new.json') == Fraction(1, 3)
```

Observed explicit source site(s): `XI-R1123` at `export/persistence.py:839`.

Implementation lookup: `komorebi_math.export.persistence.save_portable`. **Executed receipt:** `diagnostic_replay/XI-D053/receipt.json`. Its hash is `d0b54ee586b6ebb827cfe7b6ed0ef647f95db2bcdc493b10a2d98a43f096c8ad`. The source hashes and complete traceback are retained in the crosswalk and receipt.

Export, compiler and path failure

XI-D054 / Unknown export suffix

Meaning. The suffix has no implicit format mapping.

Recovery decision. Use a supported suffix or explicitly choose a supported format.

Evidence mode: native finite input. **Observed exception:** `komorebi_math.exceptions.ValidationError`. Expected message fragment: `cannot infer`. **Explicit cause:** `builtins.KeyError`.

Setup

```
from komorebi_math.export import export_data, ExportOptions
import json
```

Trigger

```
export_data([1, 2], work / 'result.unknown')
```

Valid recovery or diagnostic control

```
r = export_data([1, 2], work / 'result.json')
assert r.require().is_file()
```

Observed explicit source site(s): `XI-R1141` at `export/writers.py:160`.

Implementation lookup: `komorebi_math.export.writers.infer_export_format`. **Executed receipt:** `diagnostic_replay/XI-D054/receipt.json`. Its hash is `5908c6528497adc5d452b05597c8df67d34619bead268d5688a6f110110ef885`. The source hashes and complete traceback are retained in the crosswalk and receipt.

XI-D055 / Unknown explicit export format

Meaning. The format enum raises ordinary ValueError rather than ValidationError.

Recovery decision. Check format names separately from path permissions and mathematical data.

Evidence mode: native finite input. **Observed exception:** `builtins.ValueError`. Expected message fragment: `not-a-format`.

Setup

```
from komorebi_math.export import export_data, ExportOptions
import json
```

Trigger

```
export_data([1, 2], work / 'result.json', format='not-a-format')
```

Valid recovery or diagnostic control

```
r = export_data([1, 2], work / 'result.json', format='json')
assert r.valid
```

No explicit package raise site is attributed from this recipe's terminal traceback. This is expected for returned observations, warnings emitted through the warning machinery, dependency-only raises and nested CLI processes.

Implementation lookup: `komorebi_math.export.writers._resolve_format`. **Executed receipt:** `diagnostic_replay/XI-D055/receipt.json`. Its hash is `64417ebc670575e3ecc82ff671164164a6c65a79a9cc5b385323fd30548bd4a4`. The source hashes and complete traceback are retained in the crosswalk and receipt.

XI-D056 / Protected ordinary export destination

Meaning. The writer refuses replacement without explicit permission.

Recovery decision. Prefer a new output path; use overwrite only after review.

Evidence mode: native finite input. **Observed exception:** `komorebi_math.exceptions.ValidationError`. Expected message fragment: `already exists`.

Setup

```
from komorebi_math.export import export_data, ExportOptions
import json
p = work / 'existing.json'
p.write_text('sentinel')
```

Trigger

```
export_data([1, 2], p)
```

Valid recovery or diagnostic control

```
assert p.read_text() == 'sentinel'
r = export_data([1, 2], work / 'new.json')
assert r.valid
```

Observed explicit source site(s): `XI-R1142` at `export/writers.py:176`.

Implementation lookup: `komorebi_math.export.writers._prepare_path`. **Executed receipt:** `diagnostic_replay/XI-D056/receipt.json`. Its hash is `09c2ec07ed9afd7769b844826def33955393c280f187fe525dd43656f5a6df74`. The source hashes and complete traceback are retained in the crosswalk and receipt.

XI-D057 / Parent directory absent with creation disabled

Meaning. The requested create_parents=False contract forbids implicit creation.

Recovery decision. Create the intended parent or explicitly enable creation.

Evidence mode: native finite input. **Observed exception:** `komorebi_math.exceptions.ValidationError`. Expected message fragment: `parent directory does not exist`.

Setup

```
from komorebi_math.export import export_data, ExportOptions
import json
```

Trigger

```
export_data([1], work / 'absent' / 'value.json', options=ExportOptions(create_parents=False))
```

Valid recovery or diagnostic control

```
r = export_data([1], work / 'created' / 'value.json')
assert r.valid
```

Observed explicit source site(s): `XI-R1143` at `export/writers.py:180`.

Implementation lookup: `komorebi_math.export.writers._prepare_path`. **Executed receipt:** `diagnostic_replay/XI-D057/receipt.json`. Its hash is `d1f7c7c69339341e9312f5af90b75ee6a5a02ca287631dcd514e525c806f2c74`. The source hashes and complete traceback are retained in the crosswalk and receipt.

XI-D058 / Invalid compiler pass count

Meaning. No zero-pass compilation is defined.

Recovery decision. Choose a positive bounded pass count.

Evidence mode: native finite input. **Observed exception:** `komorebi_math.exceptions.ValidationError`. Expected message fragment: `passes must be positive`.

Setup

```
from pathlib import Path
from komorebi_math.export.latex import LatexCompileOptions, compile_latex, resolve_latex_engine
```

Trigger

```
LatexCompileOptions(passes=0)
```

Valid recovery or diagnostic control

```
assert LatexCompileOptions(passes=1).passes == 1
```

Observed explicit source site(s): `XI-R1044` at `export/latex.py:42`.

Implementation lookup: `komorebi_math.export.latex.LatexCompileOptions`. **Executed receipt:** `diagnostic_replay/XI-D058/receipt.json`. Its hash is `c5e04783b0797e013a2ebad505d2f1a59732de4241be8bce0a2df4d5ecde1935`. The source hashes and complete traceback are retained in the crosswalk and receipt.

XI-D059 / Invalid compiler time limit

Meaning. Compilation requires a positive timeout value.

Recovery decision. Choose a finite operational budget and inspect failure logs rather than repeating without bounds.

Evidence mode: native finite input. **Observed exception:** `komorebi_math.exceptions.ValidationError`. Expected message fragment: `timeout must be positive`.

Setup

```
from pathlib import Path
from komorebi_math.export.latex import LatexCompileOptions, compile_latex, resolve_latex_engine
```

Trigger

```
LatexCompileOptions(timeout=0)
```

Valid recovery or diagnostic control

```
assert LatexCompileOptions(timeout=5).timeout == 5
```

Observed explicit source site(s): `XI-R1045` at `export/latex.py:44`.

Implementation lookup: `komorebi_math.export.latex.LatexCompileOptions`. **Executed receipt:** `diagnostic_replay/XI-D059/receipt.json`. Its hash is `13a2aa78ba394cbf183dd446f9e4a13c61ac88e245fb13ea08103938d759e4b1`. The source hashes and complete traceback are retained in the crosswalk and receipt.

XI-D060 / Unsafe compiler job name

Meaning. Job names cannot contain path separators.

Recovery decision. Use a simple basename and control the output directory separately.

Evidence mode: native finite input. **Observed exception:** `komorebi_math.exceptions.ValidationError`. Expected message fragment: `simple file name`.

Setup

```
from pathlib import Path
from komorebi_math.export.latex import LatexCompileOptions, compile_latex, resolve_latex_engine
```

Trigger

```
LatexCompileOptions(jobname='../escape')
```

Valid recovery or diagnostic control

```
assert LatexCompileOptions(jobname='report').jobname == 'report'
```

Observed explicit source site(s): `XI-R1046` at `export/latex.py:47`.

Implementation lookup: `komorebi_math.export.latex.LatexCompileOptions`. **Executed receipt:** `diagnostic_replay/XI-D060/receipt.json`. Its hash is `2ffed8ebef58d7e9c48231888eee23fcc3a0668aaa21b31259268999621c337d`. The source hashes and complete traceback are retained in the crosswalk and receipt.

XI-D061 / Missing compiler (controlled PATH discovery)

Meaning. The discovery refusal is exercised with a controlled unavailable engine.

Recovery decision. Resolve a real compiler before invoking it. Constructing options alone does not establish availability.

Evidence mode: mocked compiler discovery, not native absence. **Observed exception:** `komorebi_math.exceptions.BackendUnavailableError`. Expected message fragment: `not installed`.

Setup

```
from pathlib import Path
from komorebi_math.export.latex import LatexCompileOptions, compile_latex, resolve_latex_engine
from unittest.mock import patch
```

Trigger

```
with patch('komorebi_math.export.latex.shutil.which', return_value=None):
    resolve_latex_engine('pdflatex')
```

Valid recovery or diagnostic control

```
assert LatexCompileOptions(engine='pdflatex').engine.value == 'pdflatex'
```

Observed explicit source site(s): `XI-R1048` at `export/latex.py:87`.

Implementation lookup: `komorebi_math.export.latex.resolve_latex_engine`. **Executed receipt:** `diagnostic_replay/XI-D061/receipt.json`. Its hash is `94d17393ad54ba1bbd13f717062bc370bc44a78c721a5691e540306065edd301`. The source hashes and complete traceback are retained in the crosswalk and receipt.

XI-D062 / Missing LaTeX Path object

Meaning. After engine resolution, an explicit Path must point to a file. A nonexistent .tex string is instead treated as literal TeX by this implementation.

Recovery decision. Use a Path for an existing source file and validate it before compiling.

Evidence mode: native finite input. **Observed exception:** `komorebi_math.exceptions.ValidationError`. Expected message fragment: `source file does not exist`.

Setup

```
from pathlib import Path
from komorebi_math.export.latex import LatexCompileOptions, compile_latex, resolve_latex_engine
```

Trigger

```
compile_latex(work / 'missing.tex', work / 'out', options=LatexCompileOptions(engine='pdflatex', passes=1))
```

Valid recovery or diagnostic control

```
p = work / 'actual.tex'
p.write_text('\\documentclass{article}\\begin{document}Valid fixture\\end{document}')
r = compile_latex(p, work / 'valid', options=LatexCompileOptions(engine='pdflatex', passes=1))
assert r.require().is_file()
```

Observed explicit source site(s): `XI-R1050` at `export/latex.py:166`.

Implementation lookup: `komorebi_math.export.latex.compile_latex`. **Executed receipt:** `diagnostic_replay/XI-D062/receipt.json`. Its hash is `9b46db700f71773653260b64b1370ff93c4270605bc8fcde3cbda6d1fefd97e2`. The source hashes and complete traceback are retained in the crosswalk and receipt.

XI-D063 / Compiler failure is a returned negative result

Meaning. A compiler nonzero exit returns a LatexCompilation with a false certificate, rather than raising immediately.

Recovery decision. Inspect valid, return_codes and the retained compiler log. Require success before publishing or consuming the PDF.

Evidence mode: native finite input. No immediate exception is required. Observed fields: `{"log_exists": true, "return_codes": [1], "valid": false}`.

Setup

```
from pathlib import Path
from komorebi_math.export.latex import LatexCompileOptions, compile_latex, resolve_latex_engine
```

Trigger

```
r = compile_latex('\\documentclass{article}\\begin{document}\\unknownBlockFortyOneCommand\\end{document}', work /
    ↪  'bad', options=LatexCompileOptions(engine='pdflatex', passes=1))
assert not r.valid
observation = {'valid': r.valid, 'return_codes': list(r.return_codes), 'log_exists': r.log_path.is_file()}
```

Valid recovery or diagnostic control

```
r.certificate.valid is False
try:
    r.require()
except Exception as exc:
    assert type(exc).__name__ == 'ValidationError'
else:
    raise AssertionError('negative compiler certificate admitted')
s = compile_latex('\\documentclass{article}\\begin{document}Repaired fixture\\end{document}', work / 'good',
    ↪  options=LatexCompileOptions(engine='pdflatex', passes=1))
assert s.require().is_file()
```

No explicit package raise site is attributed from this recipe's terminal traceback. This is expected for returned observations, warnings emitted through the warning machinery, dependency-only raises and nested CLI processes.

Implementation lookup: `komorebi_math.export.latex.compile_latex`, `komorebi_math.export.latex.LatexCompilation.require`. **Executed receipt:** `diagnostic_replay/XI-D063/receipt.json`. Its hash is `2124fd3abe7110714e0f342d92d367b13ba28f087f3677e1bdca858bf016ed6b`. The source hashes and complete traceback are retained in the crosswalk and receipt.

XI-D064 / Compiler timeout (controlled subprocess)

Meaning. The wrapper converts TimeoutExpired into return code 124 and a negative certificate.

Recovery decision. Keep timeout and compiler diagnostics separate from an invalid mathematical input. Retry only after fixing complexity, resources or the timeout rationale.

Evidence mode: mocked subprocess timeout; no real hanging compiler. No immediate exception is required. Observed fields: `{"return_codes": [124], "valid": false}`.

Setup

```
from pathlib import Path
from komorebi_math.export.latex import LatexCompileOptions, compile_latex, resolve_latex_engine
import subprocess
from unittest.mock import patch
```

Trigger

```
with patch('komorebi_math.export.latex.subprocess.run', side_effect=subprocess.TimeoutExpired(['pdflatex'], 1)):
    r = compile_latex('fixture', work / 'timed', options=LatexCompileOptions(engine='pdflatex', passes=1))
assert not r.valid and r.return_codes == (124,)
observation = {'valid': r.valid, 'return_codes': list(r.return_codes)}
```

Valid recovery or diagnostic control

```
assert 'timed out' in r.log_path.read_text()
```

No explicit package raise site is attributed from this recipe's terminal traceback. This is expected for returned observations, warnings emitted through the warning machinery, dependency-only raises and nested CLI processes.

Implementation lookup: `komorebi_math.export.latex.compile_latex`. **Executed receipt:** `diagnostic_replay/XI-D064/receipt.json`. Its hash is `78f99e2b74ea0aa05d14896b21c5ab7173a7823f3691e26c822c4aa70525815e`. The source hashes and complete traceback are retained in the crosswalk and receipt.

Warnings

XI-D065 / Deprecation warning without function failure

Meaning. Warnings have a distinct control channel and the wrapped function still runs. Capture the package warning category explicitly.

Recovery decision. Plan migration to the documented replacement. Avoid turning all warnings off as a repair.

Evidence mode: native finite input. No immediate exception is required. Observed fields: `{"category": "KomorebiDeprecationWarning", "message": "old_api is deprecated since 1.0; use new_api instead", "value": 7}`.

Setup

```
import warnings
from komorebi_math.compatibility import deprecated, KomorebiDeprecationWarning

@deprecated(since='1.0', replacement='new_api')
def old_api():
    return 7
```

Trigger

```
with warnings.catch_warnings(record=True) as seen:
    warnings.simplefilter('always', KomorebiDeprecationWarning)
    value = old_api()
assert value == 7 and len(seen) == 1 and (seen[0].category is KomorebiDeprecationWarning)
observation = {'category': seen[0].category.__name__, 'message': str(seen[0].message), 'value': value}
```

Valid recovery or diagnostic control

```
assert 'new_api' in str(seen[0].message)
```

No explicit package raise site is attributed from this recipe's terminal traceback. This is expected for returned observations, warnings emitted through the warning machinery, dependency-only raises and nested CLI processes.

Implementation lookup: `komorebi_math.compatibility.warn_deprecated`, `komorebi_math.compatibility.deprecated`. **Executed receipt:** `diagnostic_replay/XI-D065/receipt.json`. Its hash is `4bfbb554cda1eabdb63aea373d06c34fc1c19421463c8002daca644e7a728cc8`. The source hashes and complete traceback are retained in the crosswalk and receipt.

XI-D066 / Warnings promoted to errors

Meaning. A warning filter can turn the same call into a raised warning exception.

Recovery decision. Distinguish a chosen warning policy from a mathematical computation failure; migrate the deprecated call.

Evidence mode: native finite input. **Observed exception:** `komorebi_math.compatibility.KomorebiDeprecationWarning`. Expected message fragment: `new_api`.

Setup

```
import warnings
from komorebi_math.compatibility import deprecated, KomorebiDeprecationWarning

@deprecated(since='1.0', replacement='new_api')
def old_api():
    return 7
```

Trigger

```
with warnings.catch_warnings():
    warnings.simplefilter('error', KomorebiDeprecationWarning)
    old_api()
```

Valid recovery or diagnostic control

```
with warnings.catch_warnings(record=True) as seen:
    warnings.simplefilter('always', KomorebiDeprecationWarning)
    assert old_api() == 7
assert len(seen) == 1
```

No explicit package raise site is attributed from this recipe's terminal traceback. This is expected for returned observations, warnings emitted through the warning machinery, dependency-only raises and nested CLI processes.

Implementation lookup: `komorebi_math.compatibility.warn_deprecated`. **Executed receipt:** `diagnostic_replay/XI-D066/receipt.json`. Its hash is `c7e44131b4ca58bd26afb01c044b8ebdfb19b434224e2043cb314b0684dc104b`. The source hashes and complete traceback are retained in the crosswalk and receipt.

Returned certificates

XI-D067 / Negative stored certificate

Meaning. require raises only because the stored flag is false. The positive control is not a repair of an invalid mathematical witness.

Recovery decision. Inspect the original producer, residuals and assumptions. Never edit valid=True as a substitute for fixing the producer.

Evidence mode: native finite input. **Observed exception:** `komorebi_math.exceptions.ValidationError`. Expected message fragment: `fixture failed`.

Setup

```
from komorebi_math.certificates import Certificate, Exactness, ConditionalResult
from komorebi_math.expression.certification import compose_certificates
```

Trigger

```
Certificate(False, Exactness.NUMERICAL, message='fixture failed').require()
```

Valid recovery or diagnostic control

```
c = Certificate(True, Exactness.NUMERICAL, message='independent positive control')
assert c.require() is c
```

Observed explicit source site(s): `XI-R0334` at `certificates.py:45`.

Implementation lookup: `komorebi_math.certificates.Certificate.require`. **Executed receipt:** `diagnostic_replay/XI-D067/receipt.json`. Its hash is `84c3092591a4abaf1eb74b69d4bd7203f72a06b80780a6553e1ac10774dedf69`. The source hashes and complete traceback are retained in the crosswalk and receipt.

XI-D068 / Bypassed packet axioms are not repaired by Hilbert status

Meaning. A previously documented producer boundary is reproduced. hilbert_certificate does not rerun all packet axioms after construction validation was bypassed.

Recovery decision. Require the base packet-axiom certificate as well as any specialized certificate. Do not use the favorable specialized flag to override a failed prerequisite.

Evidence mode: native finite input. No immediate exception is required. Observed fields: `{"hilbert_flag": true, "packet_axioms": false}`.

Setup

```
from komorebi_math import PacketSystem
import numpy as np
```

Trigger

```
p = PacketSystem([2 * np.eye(2)], validate=False)
assert not p.certificate().valid and p.hilbert_certificate().valid
observation = {'packet_axioms': bool(p.certificate().valid), 'hilbert_flag': bool(p.hilbert_certificate().valid)}
```

Valid recovery or diagnostic control

```
q = PacketSystem([np.eye(2)])
assert q.certificate().valid and q.hilbert_certificate().valid
```

No explicit package raise site is attributed from this recipe's terminal traceback. This is expected for returned observations, warnings emitted through the warning machinery, dependency-only raises and nested CLI processes.

Implementation lookup: `komorebi_math.core.packets.PacketSystem.hilbert_certificate`, `komorebi_math.core.packets.PacketSystem.certificate`. **Executed receipt:** `diagnostic_replay/XI-D068/receipt.json`. Its hash is `19945b4c88a60b23e8de85b04086936b78f71188e8e63fbc25bfad4a54393cf6`. The source hashes and complete traceback are retained in the crosswalk and receipt.

Cli exit and qualification

XI-D069 / Missing top-level command

Meaning. argparse rejects a missing required command before dispatch.

Recovery decision. Check syntax first; status 2 is not a theorem failure.

Evidence mode: actual isolated CLI child interpreter. The failure invocation returned 2 and the recovery/navigation invocation returned 0.

Run through `python -I -m komorebi_math` with these argument vectors.

```
Failure argv: []
Recovery argv: ["--help"]
```

No explicit package raise site is attributed from this recipe's terminal traceback. This is expected for returned observations, warnings emitted through the warning machinery, dependency-only raises and nested CLI processes.

Implementation lookup: `komorebi_math.cli.main.main`. **Executed receipt:** `diagnostic_replay/XI-D069/receipt.json`. Its hash is `fa32e3196f6d84edcabf39985fcdd0c419836912bd54984541e986dd4a994295`. The source hashes and complete traceback are retained in the crosswalk and receipt.

XI-D070 / Unknown verification suite

Meaning. The suite selector is parser-constrained.

Recovery decision. Use the explicit suite list. The all selector is historical, not exhaustive.

Evidence mode: actual isolated CLI child interpreter. The failure invocation returned 2 and the recovery/navigation invocation returned 0.

Run through `python -I -m komorebi_math` with these argument vectors.

```
Failure argv: ["verify", "not-a-suite"]
Recovery argv: ["verify", "stable"]
```

No explicit package raise site is attributed from this recipe's terminal traceback. This is expected for returned observations, warnings emitted through the warning machinery, dependency-only raises and nested CLI processes.

Implementation lookup: `komorebi_math.cli.main.main`. **Executed receipt:** `diagnostic_replay/XI-D070/receipt.json`. Its hash is `0484c107f2fc509a769347f5fca8a6267fd641f1f8307f4f6f69d0ff88194538`. The source hashes and complete traceback are retained in the crosswalk and receipt.

XI-D071 / Missing required export destination

Meaning. The required output argument is absent.

Recovery decision. Supply an intended destination after reviewing overwrite policy.

Evidence mode: actual isolated CLI child interpreter. The failure invocation returned 2 and the recovery/navigation invocation returned 0.

Run through `python -I -m komorebi_math` with these argument vectors.

```
Failure argv: ["export", "sequence", "fibonacci"]
Recovery argv: ["export", "formats"]
```

No explicit package raise site is attributed from this recipe's terminal traceback. This is expected for returned observations, warnings emitted through the warning machinery, dependency-only raises and nested CLI processes.

Implementation lookup: `komorebi_math.cli.main.main`. **Executed receipt:** `diagnostic_replay/XI-D071/receipt.json`. Its hash is `678d37646295cb2d916846bf8ece195b2cdad110cc7c0e5112375046fdb79302`. The source hashes and complete traceback are retained in the crosswalk and receipt.

XI-D072 / Unknown root version flag

Meaning. There is no global –version flag in the pinned parser.

Recovery decision. Use info or the installed package metadata, rather than assuming a conventional flag exists.

Evidence mode: actual isolated CLI child interpreter. The failure invocation returned 2 and the recovery/navigation invocation returned 0.

Run through `python -I -m komorebi_math` with these argument vectors.

```
Failure argv: ["--version"]
Recovery argv: ["info"]
```

No explicit package raise site is attributed from this recipe's terminal traceback. This is expected for returned observations, warnings emitted through the warning machinery, dependency-only raises and nested CLI processes.

Implementation lookup: `komorebi_math.cli.main.main`. **Executed receipt:** `diagnostic_replay/XI-D072/receipt.json`. Its hash is `4ba8e62825c8ee6fe48d804479d8242ed7db71552ff68bb62382f07fe9a42edf`. The source hashes and complete traceback are retained in the crosswalk and receipt.

XI-D073 / Malformed numeric parser argument

Meaning. Argument coercion fails before mathematical validation.

Recovery decision. Supply a correctly typed integer, then check its mathematical admissibility.

Evidence mode: actual isolated CLI child interpreter. The failure invocation returned 2 and the recovery/navigation invocation returned 0.

Run through `python -I -m komorebi_math` with these argument vectors.

```
Failure argv: ["arithmetic", "euler", "--prime", "x", "--index", "2"]
Recovery argv: ["arithmetic", "euler", "--prime", "3", "--index", "2"]
```

No explicit package raise site is attributed from this recipe's terminal traceback. This is expected for returned observations, warnings emitted through the warning machinery, dependency-only raises and nested CLI processes.

Implementation lookup: `komorebi_math.cli.main.main`. **Executed receipt:** `diagnostic_replay/XI-D073/receipt.json`. Its hash is `8309af91f6f5909147d42e9895eec044d60dba7dd078bb4da34dc56f1dbaea04`. The source hashes and complete traceback are retained in the crosswalk and receipt.

XI-D074 / Mathematically invalid prime argument

Meaning. A syntactically valid integer fails the handler mathematical contract; the traceback is not wrapped by a universal CLI catch.

Recovery decision. Correct the actual mathematical parameter. Do not relabel a composite as prime.

Evidence mode: actual isolated CLI child interpreter. The failure invocation returned 1 and the recovery/navigation invocation returned 0.

Run through `python -I -m komorebi_math` with these argument vectors.

```
Failure argv: ["arithmetic", "euler", "--prime", "4", "--index", "2"]
Recovery argv: ["arithmetic", "euler", "--prime", "3", "--index", "2"]
```

No explicit package raise site is attributed from this recipe's terminal traceback. This is expected for returned observations, warnings emitted through the warning machinery, dependency-only raises and nested CLI processes.

Implementation lookup: `komorebi_math.cli.main.main`. **Executed receipt:** `diagnostic_replay/XI-D074/receipt.json`. Its hash is `c23fd886992603808b05111c1692e4ceffa5ef2999367cbdc34c14d35946c932`. The source hashes and complete traceback are retained in the crosswalk and receipt.

XI-D075 / Unmet completion assumptions

Meaning. The handler emits a negative completion result; this is not a parser failure.

Recovery decision. Inspect the unmet gates. The recovery is navigation only, not a proof that the assumptions hold.

Evidence mode: actual isolated CLI child interpreter. The failure invocation returned 1 and the recovery/navigation invocation returned 0.

Run through `python -I -m komorebi_math` with these argument vectors.

```
Failure argv: ["langlands", "completion"]
Recovery argv: ["--help"]
```

No explicit package raise site is attributed from this recipe's terminal traceback. This is expected for returned observations, warnings emitted through the warning machinery, dependency-only raises and nested CLI processes.

Implementation lookup: `komorebi_math.cli.main.main`. **Executed receipt:** `diagnostic_replay/XI-D075/receipt.json`. Its hash is `f35b74c15ec42ea0620f2e9ef45170a63fa50093b9f99cc295946a19d82c5759`. The source hashes and complete traceback are retained in the crosswalk and receipt.

I.6 Complete electronic site index and module census

The JSON ledger `Block41_Exception_Diagnostic_Index.json` contains all 3408 explicit raise statements, including 6 bare re-raises, 301 handlers, 411 assertions and 1 direct warnings.warn call. The census spans 230 modules containing explicit raises. The remaining source files are still hashed and checked. Column offsets distinguish multiple statements on one source line.

The companion TSV is an exact-message-expression lookup, not an abbreviated sample. Query the inventory by a literal message, exception name, module or owner. For example, from the extracted bundle root:

```
python tools/block41/inventory.py --query "zero evidence"
python tools/block41/inventory.py --query ResourceLimitError --limit 100
```

The lookup reports matching source records and leaves the package unchanged. A dynamic message expression is preserved as source rather than guessed into every possible runtime string. Use a source location and hash together; message text alone is not a stable unique identifier.

Resolved raise expression	Sites
`<reraise>`	6

Resolved raise expression	Sites
`builtins.ArithmeticError`	30
`builtins.AssertionError`	8
`builtins.FileExistsError`	1
`builtins.IndexError`	10
`builtins.KeyError`	8
`builtins.NotImplementedError`	1
`builtins.RuntimeError`	6
`builtins.SystemExit`	3
`builtins.TypeError`	95
`builtins.ValueError`	219
`builtins.ZeroDivisionError`	10
`komorebi_math.exceptions.AssumptionError`	17
`komorebi_math.exceptions.BackendUnavailableError`	31
`komorebi_math.exceptions.DimensionError`	686
`komorebi_math.exceptions.FeatureNotImplementedError`	1
`komorebi_math.exceptions.RelationError`	7
`komorebi_math.exceptions.ResourceLimitError`	63
`komorebi_math.exceptions.ValidationError`	2204
`numpy.linalg.LinAlgError`	2

The following table covers every module with at least one inventoried diagnostic statement. The complete 300-file hash manifest also includes modules with no such statements. R, C, A and W mean explicit raises, catch clauses, assertions and warning calls. These are syntax counts, not newly proved theorems or dynamically exercised branches.

Module below komorebi_math	R	C	A	W
`__main__`	1	0	0	0
`_number_theory`	2	1	0	0
`_planned`	1	0	0	0
`algorithms._validation`	7	2	0	0
`algorithms.benchmarks`	4	0	0	0
`algorithms.compiler`	9	0	0	0
`algorithms.complexity`	8	1	0	0
`algorithms.discovery`	5	0	0	0
`algorithms.gpu`	3	0	0	0
`algorithms.ir`	13	1	0	0
`algorithms.krylov`	20	0	1	0
`algorithms.layout`	20	6	0	0
`algorithms.low_rank`	15	0	0	0
`algorithms.precision`	12	2	0	0
`algorithms.scheduling`	9	1	0	0
`algorithms.structured`	14	0	0	0
`algorithms.support`	12	2	0	0
`analytic._series`	5	0	0	0
`analytic.appell`	4	2	0	0
`analytic.cyclic`	6	0	0	0
`analytic.reciprocal`	4	1	0	0
`analytic.sections`	8	1	0	0
`analytic.zeros`	9	1	0	0
`arithmetic.domains`	7	0	0	0
`arithmetic.euler`	10	0	0	0
`arithmetic.hurwitz`	19	0	0	0
`arithmetic.local_modes`	9	0	0	0

Module below komorebi_math	R	C	A	W
arithmetic.mahler	27	0	0	0
arithmetic.polyphase	11	0	0	0
arithmetic.recurrence	29	1	0	0
arithmetic.standard_windows	10	0	0	0
arithmetic.valuation	12	0	0	0
backends	3	1	0	0
backends.numpy	2	0	0	0
backends.sympy	3	1	0	0
certificates	2	0	0	0
cli.main	10	21	0	0
compatibility	30	3	0	1
coordinates.affine	19	2	0	0
coordinates.analysis_synthesis	21	1	0	0
coordinates.basis	40	1	0	0
coordinates.budget	8	0	0	0
coordinates.calculus	10	0	0	0
coordinates.classical	14	0	0	0
coordinates.combinatorial	5	0	0	0
coordinates.exact	18	1	0	0
coordinates.families	10	0	0	0
coordinates.formal	19	0	0	0
coordinates.frame_algebra	55	6	0	0
coordinates.modular	14	1	0	0
coordinates.normalized	2	0	0	0
coordinates.numeric	18	2	0	0
coordinates.operators	2	0	0	0
coordinates.packet	6	0	0	0
coordinates.polynomials	15	0	0	0
coordinates.portable	10	0	0	0
coordinates.realization	15	0	0	0
coordinates.recurrence	12	0	0	0
coordinates.state_space	19	0	0	0
coordinates.tensor	9	0	0	0
coordinates.transform	7	0	0	0
core.algebra	5	0	0	0
core.grammar	17	1	0	0
core.graphs	11	0	0	0
core.morphisms	9	0	0	0
core.operators	11	0	0	0
core.packets	24	1	0	0
core.realization	6	0	0	0
core.seeds	15	1	0	0
core.transfer	1	0	0	0
deformation.companions	20	3	0	0
deformation.complexes	19	0	0	0
deformation.counting	6	0	0	0
deformation.forests	15	2	0	0
deformation.support	7	1	0	0
deformation.visibility	4	0	0	0
dynamics._linalg	11	1	0	0
dynamics.bayes	29	0	0	0
dynamics.feshbach	6	0	0	0
dynamics.feynman_kac	18	0	0	0

Module below komorebi_math	R	C	A	W
`dynamics.fractional`	10	0	0	0
`dynamics.green`	3	0	0	0
`dynamics.markov`	28	1	0	0
`dynamics.martingales`	9	0	0	0
`dynamics.memory`	14	0	0	0
`dynamics.semigroups`	9	0	0	0
`dynamics.survival`	13	0	0	0
`export.latex`	8	1	0	0
`export.model`	10	0	0	0
`export.persistence`	64	18	0	0
`export.renderers`	2	0	0	0
`export.serialization`	13	4	0	0
`export.writers`	9	2	0	0
`expression.certification`	22	1	0	0
`expression.engine`	101	9	8	0
`expression.model`	51	7	2	0
`expression.planning`	32	5	1	0
`expression.polynomial`	35	3	0	0
`expression.portable`	79	11	0	0
`extensions.discovery`	1	1	0	0
`generating.adapters`	11	1	0	0
`generating.basis`	28	0	0	0
`generating.bell`	14	0	0	0
`generating.budget`	13	0	0	0
`generating.cartier`	25	0	0	0
`generating.closed_forms`	2	3	0	0
`generating.congruences`	17	0	0	0
`generating.cyclotomic`	6	1	0	0
`generating.digit_algebra`	15	0	0	0
`generating.domains`	15	1	0	0
`generating.finite_series`	13	0	0	0
`generating.functions`	12	0	0	0
`generating.masks`	32	0	0	0
`generating.monoid`	3	0	0	0
`generating.normalization`	14	0	0	0
`generating.padic_units`	16	0	0	0
`generating.portable`	36	1	0	0
`generating.q_kummer`	13	0	0	0
`generating.sections`	2	0	0	0
`generating.sequences`	14	0	0	0
`generating.series`	11	0	0	0
`golden`	16	3	0	0
`integration.adapters`	33	8	0	0
`integration.coordinates`	16	2	0	0
`integration.identities`	24	0	0	0
`integration.measures`	63	14	0	0
`integration.polynomial`	27	3	0	0
`integration.quadrature`	67	9	0	0
`integration.rules`	36	12	0	0
`langlands.completion.gates`	1	0	0	0
`langlands.finite_image.characters`	6	1	0	0
`langlands.finite_image.clifford`	7	0	0	0
`langlands.finite_image.groups`	20	0	0	0

Module below komorebi_math	R	C	A	W
langlands.finite_image.orbits	8	0	0	0
langlands.global_.hecke	10	0	0	0
langlands.local.bridge	2	0	0	0
langlands.local.envelopes	4	0	0	0
langlands.local.finite_fields	15	0	0	0
langlands.local.gauss	1	0	0	0
langlands.local.heisenberg	3	0	0	0
langlands.local.symplectic	11	0	0	0
langlands.local.tame	6	0	0	0
langlands.ramification.towers	9	0	0	0
nonlinear.autodiff	27	10	0	0
nonlinear.composition	19	2	0	0
nonlinear.coordinates	8	0	0	0
nonlinear.correction	23	1	0	0
nonlinear.defects	5	0	0	0
nonlinear.derivatives	40	6	0	0
nonlinear.elimination	18	3	0	0
nonlinear.execution	23	2	0	0
nonlinear.filtered	40	0	0	0
nonlinear.gpu	5	1	0	0
nonlinear.graded	21	5	0	0
nonlinear.holonomy	15	0	0	0
nonlinear.jets	22	0	0	0
nonlinear.lifting	22	0	0	0
nonlinear.obstruction	4	0	0	0
nonlinear.profiling	9	2	0	0
nonlinear.snapshots	17	3	0	0
nonlinear.sparse	32	3	0	0
nonlinear.streaming	87	8	0	0
nonlinear.transport	23	0	0	0
nonlinear.trees	32	4	0	0
persistence_contract	18	2	0	0
provenance	0	1	0	0
qualification	1	2	28	0
quantum._validation	17	0	0	0
quantum.channels	13	0	0	0
quantum.circuits	7	0	0	0
quantum.constraints	6	0	0	0
quantum.effective	1	0	0	0
quantum.error_correction	4	0	0	0
quantum.euclidean	8	0	0	0
quantum.leakage	6	0	0	0
quantum.lindblad	4	0	0	0
quantum.seeds	5	0	0	0
quantum.spectral	12	0	0	0
quantum.symmetry	9	0	0	0
quantum.towers	3	0	0	0
refinement.crt	16	0	0	0
refinement.defects	5	1	0	0
refinement.lattices	16	1	0	0
refinement.polyphase	59	5	4	0
refinement.portable	2	0	0	0
refinement.residue	14	0	0	0

Module below komorebi_math	R	C	A	W
`refinement.smith`	10	1	0	0
`refinement.towers`	19	0	0	0
`refinement.weighted`	17	1	0	0
`registry`	1	0	0	0
`release._gpu_worker`	1	2	0	0
`release.contract`	10	2	4	0
`release.documentation`	12	3	0	0
`release.evidence`	15	2	0	0
`release.gpu`	2	3	0	0
`release.readiness`	0	1	4	0
`release.upgrade`	21	3	0	0
`sequences.automatic`	14	0	0	0
`sequences.base`	11	0	0	0
`sequences.catalog`	1	0	0	0
`sequences.compiler`	12	0	0	0
`sequences.holonomic`	10	0	0	0
`sequences.transforms`	1	0	0	0
`signals.adaptive`	15	0	0	0
`signals.approximation`	7	0	0	0
`signals.convolution`	11	0	0	0
`signals.denoise`	28	0	0	0
`signals.edge_recovery`	14	0	0	0
`signals.fft_convolution`	6	0	0	0
`signals.filterbanks`	23	0	0	0
`signals.filtering`	19	0	0	0
`signals.nonlinear`	9	1	0	0
`signals.polyphase`	31	1	0	0
`signals.recovery`	18	0	0	0
`signals.recursive`	14	0	0	0
`signals.resampling`	9	0	0	0
`signals.singular_packets`	27	0	0	0
`signals.smith`	5	0	0	0
`signals.spectral`	18	0	0	0
`signals.streaming`	13	0	0	0
`signals.time_frequency`	25	0	0	0
`signals.wavelets`	17	2	0	0
`signals.windows`	16	0	0	0
`stable_semantics`	13	4	0	0
`support`	12	3	0	0
`unified`	9	1	0	0
`verify.algorithms`	0	1	11	0
`verify.analytic`	0	0	11	0
`verify.arithmetic`	0	0	12	0
`verify.coordinates`	2	3	45	0
`verify.core`	0	1	7	0
`verify.deformation`	0	0	10	0
`verify.dsp`	0	1	6	0
`verify.dynamics`	0	1	11	0
`verify.export`	0	1	11	0
`verify.expression`	1	1	56	0
`verify.frame_algebra`	1	1	10	0
`verify.generating`	0	1	13	0
`verify.integration`	1	1	24	0

Module below komorebi_math	R	C	A	W
verify.langlands	0	0	10	0
verify.nonlinear	0	0	4	0
verify.nonlinear_advanced	0	0	15	0
verify.quantum	0	0	7	0
verify.refinement	0	0	5	0
verify.release	0	0	21	0
verify.scalable_nonlinear	1	1	14	0
verify.sequences	0	0	12	0
verify.signals	0	1	8	0
verify.stabilization	0	0	20	0
verify.streaming_beta	1	1	16	0

I.7 Executed outcomes and coverage boundary

Executed check	Result
Unchanged source tests	1,203
Source/index/helper/receipt reference tests	432
Diagnostic recipes and paired controls	75
Recipes expecting an exception	63
Returned-status or warning observations	5
Actual CLI paired recipes	7
Distinct explicit raise sites seen in terminal tracebacks	57
Source raise sites not observed by this finite cookbook	3,351
Observed package-module union in recipe workers	245

All recorded recipe expectations and their recovery/control assertions passed. These are finite examples. A passing negative-case expectation means the refusal happened as specified, not that the denied operation succeeded. The cookbook's unobserved sites may still be exercised by the separate source suite. The source, receipt and recipe counts overlap and must not be added as distinct mathematical coverage totals.

The source and installed package match in all 366 package files, and all 901 library files remain identical to R2. The inherited dependencies are recorded in the environment receipt. Separate installation and origin checks do not establish fully dependency-isolated acceptance.

I.8 Reading handlers and reconstructing a failure chain

The electronic handler ledger records all 301 package `except` clauses. Some translate a dependency failure into a package exception with `raise ... from exc`; some produce a negative result or fall back; others clean up a temporary artifact and re-raise. An empty handler action list means none of the limited syntactic actions was found, not that the handler is harmless or unreachable. Bare re-raises preserve an active exception and do not define an independent error type.

A useful support record names both the final surface exception and its cause. For example, a missing portable file reaches the caller as ValidationError but has a FileNotFoundError cause; an unknown packet name reaches the caller as KeyError with a ValueError cause. The cause is essential for selecting a recovery. The helper `tools/block41/diagnostics.py` retains explicit and unsuppressed implicit chains, stops on cycles, and does not change or suppress the original exception.

For callers that need to log and rethrow, the control pattern is:

```
from diagnostics import exception_record

try:
    result = operation()  # the caller's actual operation
except Exception as exc:
    record = exception_record(exc, include_traceback=True)
    persist_diagnostic(record)  # the caller's reviewed logging implementation
    raise
```

Here `operation` and `persist_diagnostic` are application integration points, not exported Komorebi APIs. A logging failure must not be converted into mathematical success. KeyboardInterrupt and SystemExit are not caught by this pattern. The helper can record them when explicitly supplied, but an application should make interruption handling an intentional policy.

For base certificates, use `certificate_record` to retain exactness, residuals, tolerance, checked and unchecked assumptions, and validity type. It never coerces a truthy string into an actual bool. It also never claims that a favorable stored flag proves the producer was correctly executed. Use the protocol appropriate to specialized certificates rather than passing them through an invented common interface.

I.9 Resource, persistence and compiler boundaries

The eight generating-budget fields are local resource contracts, not a process-wide memory governor. They cover degree, coefficient bits, matrix dimension, reachable states, local combinations, modulus, serialized bytes and recursion depth. The cookbook actively exercises eight public require methods plus invalid declaration/type cases. It does not prove every producer calls every relevant guard. The full site inventory locates the other resource refusals for targeted follow-up.

Portable loading validates a specific schema and detects payload corruption, but successful loading is not proof admission or authenticated origin. Appendix G's permissive field/version conversions, payload-only digest scope, unsupported structured-array behavior and lack of a universal hostile-input budget remain in force. JSON parsing happens before all low-level decoder checks. A user should not treat a stored GeneratingBudget object as a security limit on an arbitrary JSON input.

The LaTeX wrapper resolves an engine before checking the source. When passed a `Path`, it insists on an existing file. A short string ending in `.tex` is treated as a filename only if that path exists; otherwise it becomes literal TeX source. Prefer an explicit Path for files and actual TeX text for in-memory source, rather than relying on that heuristic. Invalid TeX can return a negative compilation record with a log rather than raise. Require the result before treating a produced file as a successful delivery.

The compile route has no general overwrite guard like the ordinary export writers. Use a dedicated output directory for a fresh job and preserve any prior outputs. A favorable filesystem existence check is not a semantic document check. Compiler timeouts, operating-system permissions, unavailable executables and returned negative certificates are separate diagnostic categories.

I.10 Reproduction and completion boundary

Run `inventory.py` to recreate the JSON census and TSV lookup. `generate_recipes.py` recreates the reviewed recipe data. `replay_diagnostics.py` actively executes the cases in fresh child workspaces. `crosswalk.py` relates observed exception tracebacks to explicit source sites. `run_reference_tests.py` checks the source census, hierarchy, diagnostic helper and bound replay receipts. `generate_manual.py` reproduces the cumulative and standalone Markdown and LaTeX files. PDF rendering and visual inspection remain distinct from textual-source reproducibility.

A single recipe can be replayed with the isolated worker launcher documented in the delivery README. The recipe JSON contains trusted executable fixture code; never run a modified recipe file received from an untrusted source merely because it has a familiar schema name. Hashes detect change, not authorship. The current outer manifest binds the delivered payload independently of the historical source and certificate-index manifests.

Appendix I is complete as a source-bound exception reference and finite diagnostic cookbook for this R2 snapshot. It is not an exhaustive dynamic test of all input sizes, all error branches, all optional dependencies or all operating systems. The 75 cases overlap with source tests and existing appendices; none is a new mathematical theorem or Lean proof. Passing a controlled negative-case expectation does not make the underlying denied operation successful.

Reference boundaries. The recorded diagnostic audit did not qualify native Windows/PowerShell, other Python versions, CUDA, strict MkDocs, fully dependency-isolated installation or final publication. The validation and hardening limitations in Appendices G and H remain relevant to that baseline. Appendix J addresses backend behavior, and Appendix O identifies the separate final-source execution records.

Appendix J. Backend and dependency matrix

Reference baseline: Komorebi Math 1.0.0b4, Repair R2, wheel build tag 2. This appendix records declared requirements, the actual integration contracts, and finite execution evidence. It is not a claim that every dependency version, platform, device or option combination has been tested.

Choose the operation before the dependency. The library has several integration points, and they use different selectors. Core linear algebra, automatic differentiation, packet execution and document compilation do not share one universal backend switch. J.3 is the decision point; J.2 tells you which extra declares the required Python distribution.

You want to do	Use the J.3 entry for	Then inspect
Work with numerical or symbolic matrices	NumPy or SymPy finite linear algebra	Vector shape, scalar conversion and zero-testing behavior
Differentiate a supported callable	JAX or PyTorch directional autodiff	Callable compatibility, derivative order, dtype and device
Execute a packet plan on CPU or GPU	NumPy and CuPy PacketIR execution	Setup, transfers and the returned execution certificate
Compile an exported document	External LaTeX engines	Executable discovery, required packages and the compilation result

For an unfamiliar environment, first reproduce a small operation from the relevant entry with the backend you intend to use. Record the selected backend and the result, not just whether an import succeeded. Use J.4 to understand the recorded environment, and Appendix O for the separate final-source qualification status. The historical environment tables here are evidence about those particular runs, rather than a prescription to copy every listed dependency.

J.1 Reading the qualification levels

Read five distinct levels: a distribution is **declared** in metadata; an import or executable is **discoverable**; its import and required capability may be **usable**; a particular operation can **execute successfully**; and a specified platform or release gate can be **qualified**. No arrow between these levels is automatic. An installed package can lack a usable device, and a favorable optional-gate aggregate can coexist with zero successful GPU executions.

This reference binds all 300 Python package source files, 191 explicit external import/discovery sites, three imports inside literal child-program strings, and seven retained dynamic expressions. The 19-method linear algebra protocol, ten dependency extras, all supplied environment constraints and both workflow definitions are indexed. Dynamic user-provided imports, extension names and transitive third-party implementations remain outside an exhaustive package-source census.

J.2 Declared installation and build contract

Field	Pinned source/wheel declaration
Distribution / namespace	`komorebi-math` / `komorebi_math`
Package / build	`1.0.0b4` / wheel build `2`
Python requirement	`>=3.11`
Wheel tag	`py3-none-any`
Required runtime dependency	`numpy>=1.26`

Field	Pinned source/wheel declaration
Build backend	`setuptools.build_meta`
Build-system requirements	`setuptools>=77.0.3`, `wheel`

The pure-Python wheel tag describes the Komorebi wheel, not the binary portability of NumPy, SciPy, JAX, PyTorch or CuPy. The NumPy dependency has no upper bound in project/wheel metadata. The separately supplied base constraint files add `<3`, but a constraint file only participates when the caller explicitly supplies it. Neither an absent upper bound nor a classifier certifies future Python or dependency versions.

Extra	Complete declared requirement list
`all`	`sympy>=1.12`, `scipy>=1.11`, `mpmath>=1.3`, `matplotlib>=3.8`, `PyYAML>=6.0`
`autodiff`	`jax>=0.4`, `torch>=2.1`
`dev`	`pytest>=8`, `build>=1.2`, `mypy>=1.10`, `ruff>=0.6`, `mkdocs>=1.6`
`exact`	`sympy>=1.12`
`jax`	`jax>=0.4`
`plot`	`matplotlib>=3.8`
`scientific`	`scipy>=1.11`, `mpmath>=1.3`
`test`	`pytest>=8`
`torch`	`torch>=2.1`
`yaml`	`PyYAML>=6.0`

The `all` extra is not universal. It combines SymPy, SciPy, mpmath, Matplotlib and PyYAML. It does not include JAX, PyTorch, CuPy, pytest or the development tools. `autodiff` combines JAX and PyTorch, but neither that extra nor the separate `jax` and `torch` extras promise CUDA support. No CuPy extra is declared. CUDA-specific distributions and runtime setup are separate environment choices. The import namespace `yaml` comes from the `PyYAML` distribution; the `cupy` namespace can be supplied by a CUDA-specific CuPy distribution.

The observed install used the shipped build-2 wheel, not an assumed public package-index release. Its digest is `c150c2f5b25551aebff7ccde8b2e57d03f1ffcbc569ce24cf7625cb2f607115e`. The exact `Requires-Dist` marker expansion is tested against every source extra and the base requirement. The recorded audit made no dependency downloads.

J.3 Selectors, protocols and integration boundaries

Interface	Built-in choices / meaning
`get_backend`	NumPy default; lazy SymPy adapter; caller registration is separate
`available_backends`	NumPy, SymPy, SciPy, mpmath and CuPy discovery flags
`backend_name(value)`	A type-module heuristic, not a general device/protocol registry
`packet_executor`	NumPy or CuPy finite packet execution
`autodiff_provider`	JAX or PyTorch derivative adapters
`cupy_runtime_status`	Module, device count, CUDA runtime and error record
`qualify_gpu`	Separate Torch/CuPy workers and required-backend flags
`resolve_latex_engine`	External compiler selection, independent of Python extras

Discoverable SciPy, mpmath or CuPy entries do not become core linear algebra backend names: the shipped `get_backend` rejects those names until an application explicitly registers a suitable implementation. JAX and PyTorch are separate derivative providers. Backend registration checks duplicate names but is not a full semantic protocol-

validation suite. A caller-supplied backend remains responsible for satisfying the abstract interface and its mathematical contracts.

The 19 abstract operations are: `as_matrix`, `as_vector`, `shape`, `zeros`, `eye`, `matmul`, `add`, `subtract`, `scale`, `transpose`, `adjoint`, `inverse`, `rank`, `norm`, `is_zero`, `column_stack`, `nullspace`, `kron`, `copy`. Both built-in implementations were exercised on small finite fixtures for each method. NumPy vectors are one-dimensional; SymPy vectors are column matrices. The same interface does not make their shapes, scalar types or zero-testing rules identical.

NumPy finite linear algebra

Selector or entry: `get_backend(None), get_backend('numpy')`. **Relevant distribution(s):** `numpy`.

Contract. The default core backend implements 19 abstract operations. Matrix conversion defaults to complex128. Vectors become one-dimensional arrays. Rank, nullspace, zero tests and norms are numerical and tolerance-dependent.

Failure and fallback behavior. Conversion rejects incompatible dimensions. Singular inverse errors can come from NumPy. A missing NumPy prevents root import; there is no whole-package standard-library-only mode.

Qualification boundary. Core exact=False is not a guarantee of error bounds. Small fixtures do not cover conditioning, overflow, all dtypes or large allocations. Recorded probes: `J01`, `J02`, `J03`.

Source binding: `backends/__init__.py`, `backends/numpy.py`. Full file digests are retained in `Block42_Backend_Capability_Matrix.json`.

SymPy finite linear algebra

Selector or entry: `get_backend('sympy')`. **Relevant distribution(s):** `sympy`.

Contract. Lazy exact-extra adapter. Vectors become column matrices. Rank, nullspace and is_zero ignore the numerical tolerance argument; norm requests a float from a symbolic expression.

Failure and fallback behavior. Discovery failure yields BackendUnavailableError. Unresolved symbolic norm conversion can raise TypeError. Float inputs remain SymPy Float rather than being silently converted to exact rationals.

Qualification boundary. The exact=True flag describes the adapter, not proof of exact provenance for every supplied scalar. Symbolic equality and numerical diagnostics are distinct. Recorded probes: `J04`, `J14`, `J17`, `J18`.

Source binding: `backends/__init__.py`, `backends/sympy.py`. Full file digests are retained in `Block42_Backend_Capability_Matrix.json`.

Exact finite Fraction and integer arithmetic

Selector or entry: `FormalSeries and finite coordinate/generating constructors`. **Relevant distribution(s):** `numpy`.

Contract. Many finite algebra paths use Python int and Fraction without requiring the SymPy backend. Root import still requires NumPy. The executed example multiplies two rational formal series exactly.

Failure and fallback behavior. Individual constructors enforce their own integral/rational/budget contracts. This entry is not a claim that every symbol in the exact research namespace is SymPy-free.

Qualification boundary. Large exact coefficient growth and the placement of local budgets are separate resource questions. Recorded probes: `J06`.

Source binding: `generating/series.py`, `generating/basis.py`. Full file digests are retained in `Block42_Backend_Capability_Matrix.json`.

Symbolic algebra, Smith and elimination paths

Selector or entry: `smith_decomposition and symbolic generating/deformation/elimination APIs`. **Relevant distribution(s):** `sympy`.

Contract. Operation-local lazy imports provide exact symbolic expressions, integer Smith transforms, lattices, polynomial elimination and closed forms. `smith_decomposition` directly imports `sympy.polys.matrices.normalforms.smith_normal_decomp`.

Failure and fallback behavior. Missing SymPy generally yields an exact-extra refusal. Missing sub-APIs can propagate import errors. The declared sympy>=1.12 metadata is not fresh proof that every operation works at that lower bound.

Qualification boundary. The recorded audit executed only the installed SymPy 1.14.0 capability. Do not reinterpret this as a sweep of historical SymPy versions. Recorded probes: `J15`, `J16`.

Source binding: `refinement/smith.py`, `refinement/lattices.py`, `generating/cyclotomic.py`, `generating/closed_forms.py`, `deformation/companions.py`, `nonlinear/elimination.py`. Full file digests are retained in `Block42_Backend_Capability_Matrix.json`.

SciPy matrix exponentials with operation-specific fallback

Selector or entry: `matrix_exponential(method='auto'\|'scipy'\|'taylor')`. **Relevant distribution(s):** `scipy`.

Contract. The dynamics auto route and cyclic fundamental_matrix can fall back to a NumPy scaling/Taylor path on ImportError. Explicit scipy mode requires SciPy. Numerical LinearRefinementDefect.exponential also requires SciPy; its SymPy route uses symbolic matrix.exp.

Failure and fallback behavior. Fallback is not universal. ImportError handling does not imply recovery from arbitrary binary-loader/runtime errors, failed convergence or invalid data.

Qualification boundary. Both SciPy and Taylor results are numerical. Symbolic defect validation still has its own assumptions and residual diagnostics. Recorded probes: `J07`, `J08`, `J09`, `J10`, `J11`, `J12`, `J13`.

Source binding: `dynamics/_linalg.py`, `analytic/sections.py`, `refinement/defects.py`. Full file digests are retained in `Block42_Backend_Capability_Matrix.json`.

mpmath Hurwitz-zeta leading model

Selector or entry: `reduced_zero_zeta(..., use_computed_zeros=False)`. **Relevant distribution(s):** `mpmath`.

Contract. The leading Hurwitz model imports mpmath lazily. Its certificate remains conditional because the entire correction is not supplied. The computed-zero branch follows a separate finite numerical route.

Failure and fallback behavior. The leading-model branch refuses an absent mpmath. Returned complex values do not turn the conditional model into a proved full formula.

Qualification boundary. Availability is not error control for all arguments or precision settings. The model result is converted to a Python complex. Recorded probes: `J21`, `J22`.

Source binding: `analytic/zeros.py`. Full file digests are retained in `Block42_Backend_Capability_Matrix.json`.

JAX directional autodiff

Selector or entry: `autodiff_provider('jax', ...) or JaxDerivativeProvider`. **Relevant distribution(s):** `jax`.

Contract. Lazy nested jax.jvp adapter. Discovery/import/capability probing runs in a child interpreter, caches two framework results, and exposes refresh=True diagnostics. Actual function evaluation returns NumPy host arrays through the provider validation path.

Failure and fallback behavior. An importable framework is not proof that a user function traces or a device operation succeeds. Dtype availability follows JAX configuration. Unsupported derivative order and incompatible dimensions are rejected.

Qualification boundary. The recorded audit executed CPU derivatives of a cubic through order three with x64 enabled. It did not qualify JAX CUDA or every compatible function, order, precision or version. Recorded probes: `J23`, `J24`, `J26`.

Source binding: `nonlinear/autodiff.py`. Full file digests are retained in `Block42_Backend_Capability_Matrix.json`.

PyTorch directional autodiff

Selector or entry: `autodiff_provider('torch', ...) or TorchDerivativeProvider`. **Relevant distribution(s):** `torch`.

Contract. Lazy nested torch.func.jvp adapter with explicit dtype/device options. The dtype defaults to float64 and device to None, leaving placement to the framework; the recorded audit explicitly selected cpu. Returned values detach and transfer to CPU NumPy arrays.

Failure and fallback behavior. Missing torch.func.jvp or an unusable import produces BackendUnavailableError. Unknown dtype produces ValidationError. Native operation errors may remain external errors.

Qualification boundary. Only the observed CPU build and cubic fixture are qualified here. A successful CPU JVP does not establish CUDA availability or zero-copy output. Recorded probes: `J23`, `J25`, `J26`.

Source binding: `nonlinear/autodiff.py`. Full file digests are retained in `Block42_Backend_Capability_Matrix.json`.

NumPy and CuPy PacketIR execution

Selector or entry: `packet_executor(ir, 'numpy'\|'cupy')`. **Relevant distribution(s):** `numpy`, `cupy`.

Contract. A selector separate from get_backend. CuPy materializes and uploads block matrices at setup. apply accepts return_host and input_on_device; synchronize is explicit. NumPy returns host data.

Failure and fallback behavior. Explicit CuPy selection refuses missing CuPy rather than silently choosing CPU. Discovery alone does not verify a CUDA driver or device.

Qualification boundary. Certificates report numerical execution. Transfer/setup cost, synchronization and timing must be accounted for separately. No GPU speed claim is made. Recorded probes: `J27`, `J29`, `J30`.

Source binding: `algorithms/gpu.py`. Full file digests are retained in `Block42_Backend_Capability_Matrix.json`.

CuPy persisted snapshot and polynomial execution

Selector or entry: `CuPySnapshotExecutor, CupyPacketJetExecutor and CupyPolynomialDerivativeProvider`. **Relevant distribution(s):** `cupy`.

Contract. Separate finite GPU executors for persisted packet-jet snapshots and polynomial plans. cupy_runtime_ status distinguishes installed module, runtime availability, device count, runtime version and error. Validation compares finite CPU/device values only when hardware is usable.

Failure and fallback behavior. Missing import, absent CUDA devices and external runtime errors are different states. The status helper catches ordinary exceptions but is not a shield against a native process abort.

Qualification boundary. All successful GPU execution remains unqualified on this host. General Python callbacks are not automatically made GPU-traceable by installing CuPy. Recorded probes: `J28`.

Source binding: `nonlinear/gpu.py`, `nonlinear/execution.py`. Full file digests are retained in `Block42_Backend_Capability_Matrix.json`.

CUDA release qualification

Selector or entry: `qualify_gpu(require_torch=True, require_cupy=True)`. **Relevant distribution(s):** `torch`, `cupy`.

Contract. Both backend workers execute in separate interpreters. Torch tests CUDA JVP and a matrix product; CuPy tests the Komorebi packet executor. Required flags control which failed backends invalidate the aggregate certificate.

Failure and fallback behavior. The required gate fails on this host. Making both backends optional yields a valid aggregate with zero backend passes, so inspect per-backend records and required flags.

Qualification boundary. An empty requirement set is not GPU evidence. This matrix retains raw negative qualification results. Recorded probes: `J30`, `J31`.

Source binding: `release/gpu.py`, `release/_gpu_worker.py`. Full file digests are retained in `Block42_Backend_Capability_Matrix.json`.

PyYAML presentation export

Selector or entry: `export_data(value, path ending in .yaml)`. **Relevant distribution(s):** `PyYAML`.

Contract. Lazy yaml.safe_dump of presentation-normalized values. The yaml extra supplies PyYAML; the import namespace is yaml.

Failure and fallback behavior. No PyYAML means BackendUnavailableError. Presentation YAML is not the portable round-trip object schema catalog of Appendix G.

Qualification boundary. Round-trip fixture testing is not untrusted-file security qualification. Recorded probes: `J19`, `J20`.

Source binding: `export/writers.py`. Full file digests are retained in `Block42_Backend_Capability_Matrix.json`.

Matplotlib optional plotting dependency

Selector or entry: `No package backend selector or direct package import in this census.` **Relevant distribution(s):** `matplotlib`.

Contract. The plot extra declares matplotlib>=3.8. The 300-file package scan has no direct Matplotlib import or literal dynamic import. A separate Agg rendering smoke test was executed for the dependency.

Failure and fallback behavior. A declared extra is not evidence that a public Komorebi plotting adapter exists.

Qualification boundary. The standalone offscreen render is not a package plotting-API acceptance test and does not test interactive displays. Recorded probes: `J32`.

This is a dependency-only entry. No package implementation file is invented to fill the source column.

External LaTeX engines

Selector or entry: `compile_latex(..., engine='auto'\|'pdflatex'\|'xelatex'\|'lualatex'\|'latexmk')`. **Relevant distribution(s):** external executables, not pip requirements.

Contract. Four PATH-discovered external commands, not pip extras. Auto tries lualatex, xelatex, pdflatex, then latexmk. Compilation returns a record whose validity and PDF path must be checked. Shell escape defaults off.

Failure and fallback behavior. No executable on PATH refuses engine selection. A discovered compiler can still fail because packages, fonts or source are invalid. Ordinary output overwrite guards are not universal compiler guards.

Qualification boundary. Each installed engine compiled one minimal document. Full manual builds separately require the retained TeX package set and Pandoc for source generation. Recorded probes: `J33`, `J34`, `J35`, `J36`, `J37`, `J38`.

Source binding: `export/latex.py`. Full file digests are retained in `Block42_Backend_Capability_Matrix.json`.

J.4 Observed versions and the platform matrix

The active replay used `3.13.5 (main, Jun 25 2025, 18:55:22) [GCC 14.2.0]`, platform `linux`. The package was installed separately and every source package file was compared with that installation. Most workers inherited host dependencies. Restricted profiles instead removed site initialization and exposed selected already-installed bytes. Neither method is a fresh dependency-isolated installation from a verified wheelhouse.

Distribution	Observed installed version
`PyYAML`	`6.0.3`
`build`	**Absent**
`cupy`	**Absent**
`cupy-cuda12x`	**Absent**
`jax`	`0.9.0.1`
`jaxlib`	`0.9.0.1`
`llvmlite`	`0.47.0`
`matplotlib`	`3.10.8`
`mkdocs`	**Absent**
`mpmath`	`1.3.0`

Distribution	Observed installed version
`mypy`	**Absent**
`numba`	`0.65.1`
`numpy`	`2.3.5`
`pytest`	`9.0.2`
`ruff`	**Absent**
`scipy`	`1.17.0`
`setuptools`	`82.0.1`
`sympy`	`1.14.0`
`torch`	`2.10.0+cpu`
`wheel`	`0.46.1`

A version entry is a metadata observation, not an execution qualification by itself. NumPy, SymPy, SciPy, mpmath and PyYAML have actual package-operation fixtures. JAX and PyTorch additionally ran actual CPU derivatives. Matplotlib rendered a standalone Agg figure, which is not a Komorebi plotting-API claim. NumPy/SciPy/AD results must not be equated with exact arithmetic merely because the arithmetic inputs were simple integers.

Target / evidence class	Current status
Linux, CPython 3.13.5	Source suite and this reference executed
NumPy 2.3.5 core	Nineteen operation fixtures and NumPy-only import-path profile pass
SymPy 1.14.0	Core interface, exact defect and Smith capability fixtures pass
SciPy 1.17.0 / mpmath 1.3.0	Listed finite operation fixtures pass
JAX 0.9.0.1 / PyTorch 2.10.0+cpu	Cubic CPU JVPs through order three pass
CUDA	Required gate fails; no backend execution qualified
Windows / PowerShell	No native execution in the recorded audit
CPython 3.11 / 3.12	Declared/CI-configured, not executed here
Other Python versions and operating systems	Not qualified by the recorded audit
Every declared dependency lower bound	Not tested as a version sweep
Strict MkDocs	Executable/module unavailable; not executed
Fresh dependency-isolated installation	Still pending

The Smith path directly requires `sympy.polys.matrices.normalforms.smith_normal_decomp`. The successful fixture proves this capability for the installed SymPy, not for every release admitted by `sympy>=1.12`. Similar callable checks in autodiff do not establish traceability or binary compatibility for every admitted framework build. This appendix therefore reports declared minima without labeling them tested minima.

J.5 Isolation, numeric policy and resource interpretation

The **base** profile launches the selected interpreter with `-I -S`, inserts the documentation worker path, and exposes only the pinned package and existing NumPy bytes through a temporary import directory. The **none** profile removes NumPy as well. Actual `find_spec` calls see the optional packages as absent: no mock backend replaces their functionality. The **base-empty-path** profile additionally clears PATH to exercise actual compiler-discovery refusal. These temporary profiles are local POSIX link-based tooling; native Windows link permissions and execution remain unqualified.

The **host** profile uses a separately installed pinned wheel with inherited dependencies. All package origins in the reference tests and worker receipts are checked against the installation rather than the source checkout. Crash-sensitive native operations execute in bounded child interpreters, so a child exit, signal or timeout becomes a recorded failure instead of silently disappearing. This is engineering containment, not a hostile-input sandbox or a memory quota.

The replay set `OMP_NUM_THREADS`, `OPENBLAS_NUM_THREADS` and `MKL_NUM_THREADS` to 1, selected `JAX_PLATFORMS=cpu`, and selected Agg for plotting. The JAX derivative fixture explicitly enabled 64-bit mode; the Torch fixture explicitly chose

CPU and float64. These settings document this experiment. They are not package-wide guarantees, thread-safety certification, performance benchmarks or universal numerical-accuracy defaults.

Input exactness must be explicit. NumPy defaults to complex128 in its core adapter. SymPy may preserve an input Float even though its backend flag says exact=True. SymPy norm evaluation can fail on an unresolved symbol, and a caller-supplied tolerance is ignored by its exact zero/rank/nullspace methods. AD providers report numerical results. GPU executors have setup/transfer/synchronization costs and may return host arrays. Do not infer speedups, device residence or proof status from a backend name alone.

J.6 Executed acceptance and refusal probes

The following 38 source-bound cases are deliberately finite. "Expectation passed" means the recorded operation, refusal or boundary observation matched its stated condition. In particular, J30 retains a failed required CUDA gate, and J31 demonstrates that requiring no backend can produce an aggregate pass with zero backend passes. Those two outcomes do not contradict one another.

J01 / Base import without optional distributions

Profile: `base`. **Evidence kind:** `operation`. **Expectation:** passed. NumPy-only import-path exposure works. Existing dependency bytes are reused. This is not a fresh dependency-isolated installation.

```
import importlib.util,sys
import komorebi_math
from komorebi_math.backends import available_backends
missing=['sympy','scipy','mpmath','yaml','matplotlib','jax','torch','cupy']
assert all(importlib.util.find_spec(n) is None for n in missing)
assert all(n not in sys.modules for n in missing)
assert available_backends()=={'numpy':True,'sympy':False,'scipy':False,'mpmath':False,'cupy':False}
observation={'base_import':True,'absent_optional_roots':missing,'discovery':available_backends()}
```

Observed output:


```
{
  "absent_optional_roots": [
    "sympy",
    "scipy",
    "mpmath",
    "yaml",
    "matplotlib",
    "jax",
    "torch",
    "cupy"
  ],
  "base_import": true,
  "discovery": {
    "cupy": false,
    "mpmath": false,
    "numpy": true,
    "scipy": false,
    "sympy": false
  }
}
```


The recipe runner supplies `work` as a fresh directory; `backend_contracts.py` is the supplied finite-interface helper. Complete stdout, stderr, generated files, module origins and the receipt are under `backend_replay/J01`. Receipt SHA-256: `a037ce982a1a74049bf98a0ff4d06667a6c169c69d3435da49c2f7219d247762`.

Source files: `backends/__init__.py`, `__init__.py`. Per-file hashes are retained in the matrix.

J02 / Root import actually requires NumPy

Profile: `none`. **Evidence kind:** `expected-refusal`. **Expectation:** passed. No NumPy-free package import is established, even for exact submodules.

```
try:
 import komorebi_math
except ModuleNotFoundError as e:
 assert e.name=='numpy';observation={'refused':True,'missing':e.name}
else:raise AssertionError('Import unexpectedly worked without NumPy')
```

Observed output:

```
{
  "missing": "numpy",
  "refused": true
}
```

The recipe runner supplies `work` as a fresh directory; `backend_contracts.py` is the supplied finite-interface helper. Complete stdout, stderr, generated files, module origins and the receipt are under `backend_replay/J02`. Receipt SHA-256: `d5cf4b4d18041e4ecc2a1e8b5ea619dea508e83509424b06c56d0591b9f226f4`.

Source files: `__init__.py`, `backends/numpy.py`. Per-file hashes are retained in the matrix.

J03 / All 19 NumPy backend operations

Profile: `base`. **Evidence kind:** `operation`. **Expectation:** passed. Nineteen small numerical fixtures, not exhaustive linear algebra validation.

```
from backend_contracts import METHODS,exercise
observation={'methods':[exercise('numpy',m) for m in METHODS]}
```

Observed output:

```
{
  "all_finite_checks_passed": true,
  "backend": "numpy",
  "methods_exercised": 19
}
```

The recipe runner supplies `work` as a fresh directory; `backend_contracts.py` is the supplied finite-interface helper. Complete stdout, stderr, generated files, module origins and the receipt are under `backend_replay/J03`. Receipt SHA-256: `53b225a6a108f886d1fb135f32144e77999f3c7a5fa0d254b2f973394ed28934`.

Source files: `backends/base.py`, `backends/numpy.py`. Per-file hashes are retained in the matrix.

J04 / All 19 SymPy backend operations

Profile: `host`. **Evidence kind:** `operation`. **Expectation:** passed. Nineteen finite SymPy fixtures at the observed version, not at every declared version.

```
from backend_contracts import METHODS,exercise
observation={'methods':[exercise('sympy',m) for m in METHODS]}
```

Observed output:

```
{
  "all_finite_checks_passed": true,
  "backend": "sympy",
  "methods_exercised": 19
}
```

The recipe runner supplies `work` as a fresh directory; `backend_contracts.py` is the supplied finite-interface helper. Complete stdout, stderr, generated files, module origins and the receipt are under `backend_replay/J04`. Receipt SHA-256: `a952c88a0b3cc9a35077eb1a04ded245641829aac85301823350b85db5dce28e`.

Source files: `backends/base.py`, `backends/sympy.py`. Per-file hashes are retained in the matrix.

J05 / Discovery is different from core selection

Profile: `host`. **Evidence kind:** `expected-refusal`. **Expectation:** passed. Discovery entries for SciPy, mpmath and CuPy are not built-in core backend implementations.

```
from komorebi_math.backends import available_backends,get_backend
from komorebi_math.exceptions import BackendUnavailableError
seen=available_backends();refused=[]
for n in ('scipy','mpmath','cupy','jax','torch'):
 try:get_backend(n)
 except BackendUnavailableError:refused.append(n)
 else:raise AssertionError(n)
assert len(refused)==5 and get_backend().name=='numpy'
observation={'discovery':seen,'unsupported_core_names':refused}
```

Observed output:

```
{
  "discovery": {
    "cupy": false,
    "mpmath": true,
    "numpy": true,
    "scipy": true,
    "sympy": true
  },
  "unsupported_core_names": [
    "scipy",
    "mpmath",
    "cupy",
    "jax",
    "torch"
  ]
}
```

The recipe runner supplies `work` as a fresh directory; `backend_contracts.py` is the supplied finite-interface helper. Complete stdout, stderr, generated files, module origins and the receipt are under `backend_replay/J05`. Receipt SHA-256: `2337525a818810d39b5b13c69d8e5f78aa28cd742a5b294f4beeab40c266e5bb`.

Source files: `backends/__init__.py`. Per-file hashes are retained in the matrix.

J06 / Exact fractions without SymPy

Profile: `base`. **Evidence kind:** `operation`. **Expectation:** passed. Exact finite rational arithmetic does not universally require the SymPy adapter.

```
from fractions import Fraction
from komorebi_math.generating import FormalSeries,ordinary_basis
x=FormalSeries(ordinary_basis(),[Fraction(1,2),Fraction(1,3)])
y=x.multiply(x,degree=2)
assert y.coefficients==(Fraction(1,4),Fraction(1,3),Fraction(1,9))
observation={'coefficients':[str(v) for v in y.coefficients],'sympy_required':False}
```

Observed output:

```
{
  "coefficients": [
    "1/4",
    "1/3",
    "1/9"
  ],
  "sympy_required": false
}
```

The recipe runner supplies `work` as a fresh directory; `backend_contracts.py` is the supplied finite-interface helper. Complete stdout, stderr, generated files, module origins and the receipt are under `backend_replay/J06`. Receipt SHA-256: `ceedb5f3ad2e08c612048e4ce84309933fa1b5cbdef2eb07bb0e6c3abcfc4983`.

Source files: `generating/series.py`. Per-file hashes are retained in the matrix.

J07 / Auto exponential fallback with SciPy absent

Profile: `base`. **Evidence kind:** `operation`. **Expectation:** passed. The auto path has a numerical Taylor fallback on ImportError, not a blanket fallback for all SciPy failures.

```
import numpy as np
from komorebi_math.dynamics import matrix_exponential
m=np.array([[0.,1.],[0.,0.]])
a=matrix_exponential(m,method='auto');b=matrix_exponential(m,method='taylor')
assert np.allclose(a,[[1,1],[0,1]],atol=1e-12) and np.allclose(a,b,atol=1e-12)
observation={'auto':np.real(a).tolist(),'explicit_taylor_matches':True}
```

Observed output:

```
{
  "auto": [
    [
      1.0,
      1.0
    ],
    [
      0.0,
      1.0
    ]
  ],
  "explicit_taylor_matches": true
}
```

The recipe runner supplies `work` as a fresh directory; `backend_contracts.py` is the supplied finite-interface helper. Complete stdout, stderr, generated files, module origins and the receipt are under `backend_replay/J07`. Receipt SHA-256: `cf53e026c6a96296769f178967ba99348a8ef8b9bf0d18db0017436af71a1109`.

Source files: `dynamics/_linalg.py`. Per-file hashes are retained in the matrix.

J08 / Explicit SciPy mode refuses a missing SciPy

Profile: `base`. **Evidence kind:** `expected-refusal`. **Expectation:** passed. Explicit method selection preserves its dependency requirement.

```
from komorebi_math.dynamics import matrix_exponential
from komorebi_math.exceptions import BackendUnavailableError
try:matrix_exponential([[0.,1.],[0.,0.]],method='scipy')
except BackendUnavailableError as e:observation={'refused':True,'message':str(e)}
else:raise AssertionError('Required SciPy not enforced')
```

Observed output:

```
{
  "message": "SciPy is required for method='scipy'",
  "refused": true
}
```

The recipe runner supplies `work` as a fresh directory; `backend_contracts.py` is the supplied finite-interface helper. Complete stdout, stderr, generated files, module origins and the receipt are under `backend_replay/J08`. Receipt SHA-256: `95cfb303c6d69789c05a5110508a3b97e7002f960a55fa2bc866e7e1c09b9501`.

Source files: `dynamics/_linalg.py`. Per-file hashes are retained in the matrix.

J09 / Cyclic fundamental matrix fallback

Profile: `base`. **Evidence kind:** `operation`. **Expectation:** passed. The cyclic section route can use the core numerical exponential fallback.

```
import numpy as np
from komorebi_math.analytic.sections import CyclicSectionFamily
x=CyclicSectionFamily(2)
a=x.fundamental_matrix(0.2)
b=np.array([[np.cos(0.2),-np.sin(0.2)],[np.sin(0.2),np.cos(0.2)]])
assert np.allclose(a,b,atol=1e-12) or np.allclose(a,b.T,atol=1e-12)
assert np.allclose(a@x.fundamental_matrix(-0.2),np.eye(2),atol=1e-12)
observation={'matrix':np.real(a).tolist(),'inverse_law':True}
```

Observed output:

```
{
  "inverse_law": true,
  "matrix": [
    [
      0.9800665778412416,
      -0.19866933079506124
    ],
    [
      0.19866933079506124,
      0.9800665778412416
    ]
  ]
}
```

The recipe runner supplies `work` as a fresh directory; `backend_contracts.py` is the supplied finite-interface helper. Complete stdout, stderr, generated files, module origins and the receipt are under `backend_replay/J09`. Receipt SHA-256: `94ce3a9a6383163bdbd0b7951a37cc0622eca8f6c3be40187931cf56d48d1d1d`.

Source files: `analytic/sections.py`, `dynamics/_linalg.py`. Per-file hashes are retained in the matrix.

J10 / Refinement exponential has no NumPy-only fallback

Profile: `base`. **Evidence kind:** `expected-refusal`. **Expectation:** passed. Fallback availability is operation-specific, not package-wide.

```
from komorebi_math.refinement.defects import LinearRefinementDefect
from komorebi_math.exceptions import BackendUnavailableError
x=LinearRefinementDefect([[1]],[[1]],[[0]])
try:x.exponential(0.1)
except BackendUnavailableError as e:observation={'refused':True,'message':str(e)}
else:raise AssertionError('Expected scientific-extra refusal')
```

Observed output:

```
{
  "message": "matrix exponential defects require the 'scientific' SciPy extra",
  "refused": true
}
```

The recipe runner supplies `work` as a fresh directory; `backend_contracts.py` is the supplied finite-interface helper. Complete stdout, stderr, generated files, module origins and the receipt are under `backend_replay/J10`. Receipt SHA-256: `612af7f42c80aafcde5769e3a3a18d67192999712fb4f66082554fca8286f307`.

Source files: `refinement/defects.py`. Per-file hashes are retained in the matrix.

J11 / SciPy exponential matches the Taylor path

Profile: `host`. **Evidence kind:** `operation`. **Expectation:** passed. Independent finite numerical path comparison at installed SciPy.

```
import numpy as np
from komorebi_math.dynamics import matrix_exponential
m=np.array([[0.,-1.],[1.,0.]])
a=matrix_exponential(m,method='scipy');b=matrix_exponential(m,method='taylor')
res=float(np.linalg.norm(a-b));assert res<1e-12
observation={'residual':res,'finite_matrix':m.tolist()}
```

Observed output:

```
{
  "finite_matrix": [
    [
      0.0,
      -1.0
    ],
    [
      1.0,
      0.0
    ]
  ],
  "residual": 1.9229626863835638e-16
}
```

The recipe runner supplies `work` as a fresh directory; `backend_contracts.py` is the supplied finite-interface helper. Complete stdout, stderr, generated files, module origins and the receipt are under `backend_replay/J11`. Receipt SHA-256: `cd422d86ee95d3a196914f72e63de9ca6646a5af4aa71a6620863eb4fdac94a4`.

Source files: `dynamics/_linalg.py`. Per-file hashes are retained in the matrix.

J12 / Scientific refinement exponential

Profile: `host`. **Evidence kind:** `operation`. **Expectation:** passed. Numerical block-exponential identity for a scalar fixture, not exact arithmetic.

```
import numpy as np
from komorebi_math.refinement.defects import LinearRefinementDefect
r=LinearRefinementDefect([[1]],[[1]],[[0]]).exponential(0.1)
r.certificate.require();assert np.allclose(r.value,[[np.exp(.1)-1]],atol=1e-12)
observation={'valid':bool(r.certificate.valid),'exactness':str(r.certificate.exactness),'value'
    ↪ :np.real(r.value).tolist()}
```

Observed output:

```
{
  "exactness": "numerical",
  "valid": true,
  "value": [
    [
      0.10517091807564763
    ]
  ]
}
```

The recipe runner supplies `work` as a fresh directory; `backend_contracts.py` is the supplied finite-interface helper. Complete stdout, stderr, generated files, module origins and the receipt are under `backend_replay/J12`. Receipt SHA-256: `d4d894e1e12a4bc27bfaa3ba0c4d10b36f3e4b1c0c2f06fd635c5877ed10bf6e`.

Source files: `refinement/defects.py`. Per-file hashes are retained in the matrix.

J13 / SymPy refinement exponential

Profile: `host`. **Evidence kind:** `operation`. **Expectation:** passed. The symbolic route uses SymPy matrix.exp, not SciPy.

```
import sympy as sp
from komorebi_math.refinement.defects import LinearRefinementDefect
r=LinearRefinementDefect([[1]],[[1]],[[0]],backend='sympy').exponential(sp.Rational(1,10))
r.certificate.require();assert sp.simplify(r.value[0,0]-(sp.exp(sp.Rational(1,10))-1))==0
observation={'valid':bool(r.certificate.valid),'value':str(r.value),'exactness':str(r.certificate.exactness)}
```

Observed output:

```
{
  "exactness": "exact",
  "valid": true,
  "value": "Matrix([[-1 + exp(1/10)]])"
}
```

The recipe runner supplies `work` as a fresh directory; `backend_contracts.py` is the supplied finite-interface helper. Complete stdout, stderr, generated files, module origins and the receipt are under `backend_replay/J13`. Receipt SHA-256: `223dcfa5e58bd27b86664a7dfdf34595f94568284eaa84684ac3c3d6aeccb5ac`.

Source files: `refinement/defects.py`. Per-file hashes are retained in the matrix.

J14 / Missing SymPy adapter refusal

Profile: `base`. **Evidence kind:** `expected-refusal`. **Expectation:** passed. The exact extra is required for the SymPy core adapter.

```
from komorebi_math.backends import get_backend
from komorebi_math.exceptions import BackendUnavailableError
try:get_backend('sympy')
except BackendUnavailableError as e:observation={'refused':True,'message':str(e)}
else:raise AssertionError('SymPy should be absent')
```

Observed output:

```
{
  "message": "the SymPy backend requires the 'exact' optional dependency",
  "refused": true
}
```

The recipe runner supplies `work` as a fresh directory; `backend_contracts.py` is the supplied finite-interface helper. Complete stdout, stderr, generated files, module origins and the receipt are under `backend_replay/J14`. Receipt SHA-256: `edec6322efccd0f7aa9ad168f936f2f5e028b69001485f2a0128f01bad5f0a60`.

Source files: `backends/__init__.py`. Per-file hashes are retained in the matrix.

J15 / Smith normal decomposition capability

Profile: `host`. **Evidence kind:** `operation`. **Expectation:** passed. The installed release supplies the required Smith API. The declared lower bound is not thereby qualified.

```
from komorebi_math.refinement.smith import smith_decomposition
r=smith_decomposition([[2,4],[0,6]]);r.certificate().require()
assert r.left*r.matrix*r.right==r.diagonal
observation={'invariant_factors':r.invariant_factors,'valid':bool(r.certificate().valid),'required_symbol':
    ↪ 'sympy.polys.matrices.normalforms.smith_normal_decomp'}
```

Observed output:

```
{
  "invariant_factors": [
    2,
    6
  ],
  "required_symbol": "sympy.polys.matrices.normalforms.smith_normal_decomp",
  "valid": true
}
```

The recipe runner supplies `work` as a fresh directory; `backend_contracts.py` is the supplied finite-interface helper. Complete stdout, stderr, generated files, module origins and the receipt are under `backend_replay/J15`. Receipt SHA-256: `31953284d6d54315b2c829fe8be1eba98e7468ff5e750d30a440866e8538a82b`.

Source files: `refinement/smith.py`. Per-file hashes are retained in the matrix.

J16 / Missing SymPy Smith refusal

Profile: `base`. **Evidence kind:** `expected-refusal`. **Expectation:** passed. No silent numeric substitution for an exact Smith operation.

```
from komorebi_math.refinement.smith import smith_decomposition
from komorebi_math.exceptions import BackendUnavailableError
try:smith_decomposition([[2]])
except BackendUnavailableError as e:observation={'refused':True,'message':str(e)}
else:raise AssertionError('SymPy should be absent')
```

Observed output:

```
{
  "message": "Smith coordinate operations require the 'exact' SymPy extra",
  "refused": true
}
```

The recipe runner supplies `work` as a fresh directory; `backend_contracts.py` is the supplied finite-interface helper. Complete stdout, stderr, generated files, module origins and the receipt are under `backend_replay/J16`. Receipt SHA-256: `50756c355a46f0b9e91214007ac1024a02d4ec05835ddb2f49a74fdaa4e0fa3b`.

Source files: `refinement/smith.py`. Per-file hashes are retained in the matrix.

J17 / SymPy exact flag does not rationalize float inputs

Profile: `host`. **Evidence kind:** `boundary-observation`. **Expectation:** passed. Choose exact input types explicitly. The backend flag is not input-provenance validation.

```
import sympy as sp
from komorebi_math.backends import get_backend
b=get_backend('sympy');m=b.as_matrix([[0.1]])
assert b.exact is True and isinstance(m[0,0],sp.Float)
observation={'backend_exact_flag':b.exact,'stored_scalar_type':type(m[0,0]).__name__,'value':str(m[0,0])}
```

Observed output:

```
{
  "backend_exact_flag": true,
  "stored_scalar_type": "Float",
  "value": "0.100000000000000"
}
```

The recipe runner supplies `work` as a fresh directory; `backend_contracts.py` is the supplied finite-interface helper. Complete stdout, stderr, generated files, module origins and the receipt are under `backend_replay/J17`. Receipt SHA-256: `bb450a5a8682335d5062d8341bf875982987fe20d8ec33c80e033695c31e22ca`.

Source files: `backends/sympy.py`. Per-file hashes are retained in the matrix.

J18 / SymPy norm can reject unresolved symbols

Profile: `host`. **Evidence kind:** `expected-refusal`. **Expectation:** passed. A symbolic matrix type does not guarantee every numerical diagnostic can evaluate it.

```
import sympy as sp
from komorebi_math.backends import get_backend
b=get_backend('sympy')
try:b.norm(b.as_matrix([[sp.Symbol('x')]]))
except TypeError as e:observation={'refused':True,'message':str(e)}
else:raise AssertionError('Unresolved symbol unexpectedly produced a floating norm')
```

Observed output:

```
{
  "message": "Cannot convert expression to float",
  "refused": true
}
```

The recipe runner supplies `work` as a fresh directory; `backend_contracts.py` is the supplied finite-interface helper. Complete stdout, stderr, generated files, module origins and the receipt are under `backend_replay/J18`. Receipt SHA-256: `3bd74a783d1dc6e6eabdc2ee7a10bebdb69740a565f9593c9ab4ce4380f9f613`.

Source files: `backends/sympy.py`. Per-file hashes are retained in the matrix.

J19 / YAML export and parse-back

Profile: `host`. **Evidence kind:** `operation`. **Expectation:** passed. PyYAML supplies presentation export; this is not the portable-object codec.

```
import yaml
from komorebi_math.export import export_data
p=work/'record.yaml';export_data({'a':[1,2],'ok':True},p)
a=yaml.safe_load(p.read_text());assert a=={'a':[1,2],'ok':True}
observation={'parsed':a,'output':p.name}
```

Observed output:

```
{
  "output": "record.yaml",
  "parsed": {
    "a": [
      1,
      2
    ],
    "ok": true
  }
}
```

The recipe runner supplies `work` as a fresh directory; `backend_contracts.py` is the supplied finite-interface helper. Complete stdout, stderr, generated files, module origins and the receipt are under `backend_replay/J19`. Receipt SHA-256: `c55dd5eec4e2347b202b7a9bc6b8070d38b1b75445d3c7c6c3112a7d7c3a4f0c`.

Source files: `export/writers.py`. Per-file hashes are retained in the matrix.

J20 / YAML export absent-dependency refusal

Profile: `base`. **Evidence kind:** `expected-refusal`. **Expectation:** passed. YAML is optional and no handwritten YAML fallback is substituted.

```
from komorebi_math.export import export_data
from komorebi_math.exceptions import BackendUnavailableError
try:export_data({'a':1},work/'record.yaml')
except BackendUnavailableError as e:observation={'refused':True,'message':str(e)}
else:raise AssertionError('PyYAML should be absent')
```

Observed output:

```
{
  "message": "YAML export requires PyYAML, install komorebi-math[yaml]",
  "refused": true
}
```

The recipe runner supplies `work` as a fresh directory; `backend_contracts.py` is the supplied finite-interface helper. Complete stdout, stderr, generated files, module origins and the receipt are under `backend_replay/J20`. Receipt SHA-256: `19fd94c90eea59ba21a830eb8ca62cabae4ea97f7acd28bd12dadfb899fb3bf6`.

Source files: `export/writers.py`. Per-file hashes are retained in the matrix.

J21 / mpmath leading zero-zeta model

Profile: `host`. **Evidence kind:** `operation`. **Expectation:** passed. The leading Hurwitz model remains conditional. mpmath availability does not supply the omitted correction.

```
import numpy as np
from komorebi_math.analytic.zeros import reduced_zero_zeta
r=reduced_zero_zeta(2,1,use_computed_zeros=False)
assert abs(r.value-.5)<1e-12
observation={'value_real':r.value.real,'value_imag':r.value.imag,'exactness':str(r.certificate.exactness),
    ↪ 'unchecked':r.certificate.assumptions_unchecked}
```

Observed output:

```
{
  "exactness": "conditional",
  "unchecked": [
    "entire correction term"
  ],
  "value_imag": 0.0,
  "value_real": 0.5
}
```

The recipe runner supplies `work` as a fresh directory; `backend_contracts.py` is the supplied finite-interface helper. Complete stdout, stderr, generated files, module origins and the receipt are under `backend_replay/J21`. Receipt SHA-256: `68d3c784d9c7addf028b2555090f8c0f4956793298be9111aa85cb4b33f08004`.

Source files: `analytic/zeros.py`. Per-file hashes are retained in the matrix.

J22 / Missing mpmath leading-model refusal

Profile: `base`. **Evidence kind:** `expected-refusal`. **Expectation:** passed. The scientific extra is required for this branch.

```
from komorebi_math.analytic.zeros import reduced_zero_zeta
from komorebi_math.exceptions import BackendUnavailableError
try:reduced_zero_zeta(2,1,use_computed_zeros=False)
except BackendUnavailableError as e:observation={'refused':True,'message':str(e)}
else:raise AssertionError('mpmath should be absent')
```

Observed output:

```
{
  "message": "the Hurwitz zeta model requires the scientific extra",
  "refused": true
}
```

The recipe runner supplies `work` as a fresh directory; `backend_contracts.py` is the supplied finite-interface helper. Complete stdout, stderr, generated files, module origins and the receipt are under `backend_replay/J22`. Receipt SHA-256: `f38f5e04a049654839f52d6a80c5203bebb74fc01573bc8f0b30b8c860319895`.

Source files: `analytic/zeros.py`. Per-file hashes are retained in the matrix.

J23 / Native AD discovery and capability report

Profile: `host`. **Evidence kind:** `capability-observation`. **Expectation:** passed. A child import plus callable capability check is weaker than actually evaluating a derivative.

```
from komorebi_math.nonlinear.autodiff import ad_framework_diagnostics
observation=ad_framework_diagnostics(refresh=True)
assert set(observation)=={'jax','torch'}
for r in observation.values():assert isinstance(r['available'],bool) and isinstance(r['discovered'],bool)
```

Observed output:

```
{
  "jax": {
    "available": true,
    "discovered": true,
    "returncode": 0,
    "version": "0.9.0.1"
  },
  "torch": {
    "available": true,
    "discovered": true,
    "returncode": 0,
    "version": "2.10.0+cpu"
  }
}
```

The recipe runner supplies `work` as a fresh directory; `backend_contracts.py` is the supplied finite-interface helper. Complete stdout, stderr, generated files, module origins and the receipt are under `backend_replay/J23`. Receipt SHA-256: `c01f21d40b0abe854ecea698a24d599eab4e4f12339ef73148f2b654b03933f7`.

Source files: `nonlinear/autodiff.py`. Per-file hashes are retained in the matrix.

J24 / JAX CPU derivatives through order three

Profile: `host`. **Evidence kind:** `operation`. **Expectation:** passed. Actual CPU derivative execution. No CUDA, transfer-cost or performance qualification is implied.

```
import numpy as np
import jax,jax.numpy as jnp
from komorebi_math.nonlinear.autodiff import JaxDerivativeProvider
jax.config.update('jax_enable_x64',True)
p=JaxDerivativeProvider(lambda x:jnp.stack((x[0]**3+x[0]*x[1],x[1]**3)),2,2,max_order=3,dtype='float64')
point=[2.,3.];directions=([1.,0.],)*3
assert np.allclose(p.value(point),[14.,27.],atol=1e-12)
actual=[p.derivative(n,point,directions[:n]).tolist() for n in (1,2,3)]
assert np.allclose(actual,[[15,0],[12,0],[6,0]],atol=1e-12)
observation={'version':jax.__version__,'device_platform':jax.default_backend(),'jax_enable_x64':bool
    ↪ (jax.config.jax_enable_x64),'derivatives':actual,'certificate_exactness':str(p.exactness)}
```

Observed output:

```
{
  "certificate_exactness": "numerical",
  "derivatives": [
    [
      15.0,
      0.0
    ],
    [
      12.0,
      0.0
    ],
    [
      6.0,
      0.0
    ]
  ],
  "device_platform": "cpu",
  "jax_enable_x64": true,
  "version": "0.9.0.1"
}
```

The recipe runner supplies `work` as a fresh directory; `backend_contracts.py` is the supplied finite-interface helper. Complete stdout, stderr, generated files, module origins and the receipt are under `backend_replay/J24`. Receipt SHA-256: `94e03f7efcc28a73704dd09b07a6e79242a6a71c03e8e27ff128c84bd1c16f67`.

Source files: `nonlinear/autodiff.py`. Per-file hashes are retained in the matrix.

J25 / TORCH CPU derivatives through order three

Profile: `host`. **Evidence kind:** `operation`. **Expectation:** passed. Actual CPU derivative execution. No CUDA, transfer-cost or performance qualification is implied.

```
import numpy as np
import torch
from komorebi_math.nonlinear.autodiff import TorchDerivativeProvider
p=TorchDerivativeProvider(lambda
    ↪ x:torch.stack((x[0]**3+x[0]*x[1],x[1]**3)),2,2,max_order=3,dtype='float64',device='cpu')
point=[2.,3.];directions=([1.,0.],)*3
assert np.allclose(p.value(point),[14.,27.],atol=1e-12)
actual=[p.derivative(n,point,directions[:n]).tolist() for n in (1,2,3)]
```

```
assert np.allclose(actual,[[15,0],[12,0],[6,0]],atol=1e-12)
observation={'version':torch.__version__,'device':'cpu','dtype':p.dtype,'cuda_available'
    ↪ :torch.cuda.is_available(),'compiled_cuda':torch.version.cuda,'derivatives':actual,'certificate_exactness'
    ↪ :str(p.exactness)}
```

Observed output:

```
{
  "certificate_exactness": "numerical",
  "compiled_cuda": null,
  "cuda_available": false,
  "derivatives": [
    [
      15.0,
      0.0
    ],
    [
      12.0,
      0.0
    ],
    [
      6.0,
      0.0
    ]
  ],
  "device": "cpu",
  "dtype": "float64",
  "version": "2.10.0+cpu"
}
```

The recipe runner supplies `work` as a fresh directory; `backend_contracts.py` is the supplied finite-interface helper. Complete stdout, stderr, generated files, module origins and the receipt are under `backend_replay/J25`. Receipt SHA-256: `f6b09e1c001dd388e06ae74e66ec372e67c19754b4afa882b0583c68911c347e`.

Source files: `nonlinear/autodiff.py`. Per-file hashes are retained in the matrix.

J26 / AD discovery with frameworks absent

Profile: `base`. **Evidence kind:** `expected-refusal`. **Expectation:** passed. Import-path isolation makes the frameworks absent without substituting a fake provider.

```
from komorebi_math.nonlinear.autodiff import available_ad_frameworks,ad_framework_diagnostics
assert available_ad_frameworks()=={'jax':False,'torch':False}
observation=ad_framework_diagnostics()
```

Observed output:

```
{
  "jax": {
    "available": false,
    "detail": "jax is not installed in the active Python environment",
    "discovered": false,
    "framework": "jax",
    "returncode": null,
    "version": null
  },
  "torch": {
    "available": false,
    "detail": "torch is not installed in the active Python environment",
    "discovered": false,
    "framework": "torch",
```

```
    "returncode": null,
    "version": null
  }
}
```

The recipe runner supplies `work` as a fresh directory; `backend_contracts.py` is the supplied finite-interface helper. Complete stdout, stderr, generated files, module origins and the receipt are under `backend_replay/J26`. Receipt SHA-256: `a483915b8ade1af71b0d8ffc03961590bfaf17e39ac1e8ed7707dedb8d8e6e99`.

Source files: `nonlinear/autodiff.py`. Per-file hashes are retained in the matrix.

J27 / NumPy packet execution

Profile: `base`. **Evidence kind:** `operation`. **Expectation:** passed. The packet-executor selector is separate from the core linear algebra selector.

```
import numpy as np
from komorebi_math.algorithms import PacketIR,PacketLayout,packet_executor
ir=PacketIR.square(PacketLayout((2,2)),{(0,0):np.eye(2),(1,1):2*np.eye(2)})
r=packet_executor(ir,'numpy').apply(np.arange(4.))
assert np.allclose(r.value,[0,1,4,6]) and r.backend=='numpy' and r.returned_to_host
observation={'value':r.value.tolist(),'backend':r.backend,'returned_to_host':r.returned_to_host}
```

Observed output:

```
{
  "backend": "numpy",
  "returned_to_host": true,
  "value": [
    0.0,
    1.0,
    4.0,
    6.0
  ]
}
```

The recipe runner supplies `work` as a fresh directory; `backend_contracts.py` is the supplied finite-interface helper. Complete stdout, stderr, generated files, module origins and the receipt are under `backend_replay/J27`. Receipt SHA-256: `df77c0dd0ba642c5cddf352c0076529f95c82b592af10b743031f803cf1a3ac1`.

Source files: `algorithms/gpu.py`. Per-file hashes are retained in the matrix.

J28 / CuPy runtime report on this host

Profile: `host`. **Evidence kind:** `hardware-unavailable`. **Expectation:** passed. CuPy is genuinely absent on the audited host. This is a negative hardware observation.

```
from dataclasses import asdict
from komorebi_math.nonlinear.gpu import cupy_runtime_status
observation=asdict(cupy_runtime_status())
assert observation['installed'] is False and observation['runtime_available'] is False
```

Observed output:

```
{
  "device_count": 0,
  "error": "CuPy is not installed",
  "installed": false,
  "runtime_available": false,
```

```
  "runtime_version": null
}
```

The recipe runner supplies `work` as a fresh directory; `backend_contracts.py` is the supplied finite-interface helper. Complete stdout, stderr, generated files, module origins and the receipt are under `backend_replay/J28`. Receipt SHA-256: `ef13a71567b4cdee5639f1164d4a32c0528eb1f3afaf42e43ac0cbcfa3d46dcb`.

Source files: `nonlinear/gpu.py`. Per-file hashes are retained in the matrix.

J29 / Missing CuPy packet-executor refusal

Profile: `base`. **Evidence kind:** `expected-refusal`. **Expectation:** passed. There is no silent CPU fallback when CuPy is explicitly selected.

```
import numpy as np
from komorebi_math.algorithms import PacketIR,PacketLayout,packet_executor
from komorebi_math.exceptions import BackendUnavailableError
ir=PacketIR.square(PacketLayout((1,)),{(0,0):np.eye(1)})
try:packet_executor(ir,'cupy')
except BackendUnavailableError as e:observation={'refused':True,'message':str(e)}
else:raise AssertionError('CuPy should be absent')
```

Observed output:

```
{
  "message": "CuPy is not installed",
  "refused": true
}
```

The recipe runner supplies `work` as a fresh directory; `backend_contracts.py` is the supplied finite-interface helper. Complete stdout, stderr, generated files, module origins and the receipt are under `backend_replay/J29`. Receipt SHA-256: `c39c7ffb8ececf9b4eb5254f9c667cddd235e92750b98e7ca9b620a34944fa26`.

Source files: `algorithms/gpu.py`. Per-file hashes are retained in the matrix.

J30 / Required CUDA qualification remains negative

Profile: `host`. **Evidence kind:** `hardware-unavailable`. **Expectation:** passed. Actual required GPU gate failure is retained separately from successful observation of that failure.

```
from komorebi_math.release.gpu import qualify_gpu
r=qualify_gpu(timeout=60)
assert not r.certificate.valid and all(not b.passed for b in r.backends)
observation={'qualification_passed':bool(r.certificate.valid),'backends':[{'backend':b.backend,'discovered'
    ↪ :b.discovered,'available':b.available,'passed':b.passed,'version':b.version,'detail':b.detail,'returncode'
    ↪ :b.returncode} for b in
    ↪ r.backends]}
```

Observed output:

```
{
  "backends": [
    {
      "available": false,
      "backend": "torch",
      "detail": "torch.cuda.is_available() is false",
      "discovered": true,
      "passed": false,
      "returncode": 3,
      "version": "2.10.0+cpu"
```

```
    },
    {
      "available": false,
      "backend": "cupy",
      "detail": "cupy is not installed",
      "discovered": false,
      "passed": false,
      "returncode": 2,
      "version": null
    }
  ],
  "qualification_passed": false
}
```


The recipe runner supplies `work` as a fresh directory; `backend_contracts.py` is the supplied finite-interface helper. Complete stdout, stderr, generated files, module origins and the receipt are under `backend_replay/J30`. Receipt SHA-256: `0b0feb81d6533f901aecfb02026caa8ba78af9e175d9980fc067e37fa4e19f75`.

Source files: `release/gpu.py`, `release/_gpu_worker.py`. Per-file hashes are retained in the matrix.

J31 / Optional GPU flags do not prove GPU execution

Profile: `host`. **Evidence kind:** `boundary-observation`. **Expectation:** passed. An empty required-backend set can pass with zero successful GPU executions.

```
from komorebi_math.release.gpu import qualify_gpu
r=qualify_gpu(require_torch=False,require_cupy=False,timeout=60)
assert r.certificate.valid and all(not b.passed for b in r.backends)
observation={'qualification_passed':bool(r.certificate.valid),'required_backends':[],'actual_backend_passes':0,
    ↪ 'unchecked':r.certificate.assumptions_unchecked}
```

Observed output:


```
{
  "actual_backend_passes": 0,
  "qualification_passed": true,
  "required_backends": [],
  "unchecked": [
    "torch was optional for this invocation",
    "cupy was optional for this invocation"
  ]
}
```


The recipe runner supplies `work` as a fresh directory; `backend_contracts.py` is the supplied finite-interface helper. Complete stdout, stderr, generated files, module origins and the receipt are under `backend_replay/J31`. Receipt SHA-256: `8c10abae2a0a1b0099ae9ec93a84ecc14fc5ac7ee13a8667db9097251ffc1400`.

Source files: `release/gpu.py`. Per-file hashes are retained in the matrix.

J32 / Declared plotting extra can render offscreen

Profile: `host`. **Evidence kind:** `operation`. **Expectation:** passed. A standalone dependency smoke test, not evidence of a plotting backend API in Komorebi Math.

```
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
fig,ax=plt.subplots();ax.plot([0,1,2],[0,1,4]);ax.set(xlabel='x',ylabel='x
    ↪ squared');p=work/'plot.png';fig.savefig(p);plt.close(fig)
assert p.read_bytes().startswith(b'\x89PNG\r\n\x1a\n')
observation={'version':matplotlib.__version__,'backend':matplotlib.get_backend(),'output_bytes':p.stat().st_size,
    ↪ 'package_plotting_api_claim':False}
```

Observed output:

```
{
  "backend": "Agg",
  "output_bytes": 21139,
  "package_plotting_api_claim": false,
  "version": "3.10.8"
}
```

The recipe runner supplies `work` as a fresh directory; `backend_contracts.py` is the supplied finite-interface helper. Complete stdout, stderr, generated files, module origins and the receipt are under `backend_replay/J32`. Receipt SHA-256: `7bdb0f90525268ef01b104eb4089f13ea0dee6ba6ab3656141fa3fc348af0fe9`.

J33 / Native pdflatex PDF compilation

Profile: `host`. **Evidence kind:** `operation`. **Expectation:** passed. Actual compiler execution on a minimal TeX fixture. It does not qualify every document or font.

```
from komorebi_math.export.latex import compile_latex,LatexCompileOptions
text=r'\documentclass{article}\begin{document}Komorebi P. $1+1=2$.\end{document}'
r=compile_latex(text,work/'tex',options=LatexCompileOptions(engine='pdflatex',passes=1,timeout=45))
p=r.require();assert p.read_bytes().startswith(b'%PDF-')
observation={'engine':str(r.engine),'valid':bool(r.valid),'pdf_bytes':p.stat().st_size,'return_codes'
    ↪ :r.return_codes,'shell_escape':False}
```

Observed output:

```
{
  "engine": "pdflatex",
  "pdf_bytes": 13505,
  "return_codes": [
    0
  ],
  "shell_escape": false,
  "valid": true
}
```

The recipe runner supplies `work` as a fresh directory; `backend_contracts.py` is the supplied finite-interface helper. Complete stdout, stderr, generated files, module origins and the receipt are under `backend_replay/J33`. Receipt SHA-256: `f4c44f902ba5fe01e088377aaaf8f3099476a5a45e95c18229584b3351198248`.

Source files: `export/latex.py`. Per-file hashes are retained in the matrix.

J34 / Native xelatex PDF compilation

Profile: `host`. **Evidence kind:** `operation`. **Expectation:** passed. Actual compiler execution on a minimal TeX fixture. It does not qualify every document or font.

```
from komorebi_math.export.latex import compile_latex,LatexCompileOptions
text=r'\documentclass{article}\begin{document}Komorebi P. $1+1=2$.\end{document}'
r=compile_latex(text,work/'tex',options=LatexCompileOptions(engine='xelatex',passes=1,timeout=45))
p=r.require();assert p.read_bytes().startswith(b'%PDF-')
observation={'engine':str(r.engine),'valid':bool(r.valid),'pdf_bytes':p.stat().st_size,'return_codes'
    ↪ :r.return_codes,'shell_escape':False}
```

Observed output:

```
{
  "engine": "xelatex",
  "pdf_bytes": 4306,
  "return_codes": [
    0
  ],
  "shell_escape": false,
  "valid": true
}
```

The recipe runner supplies `work` as a fresh directory; `backend_contracts.py` is the supplied finite-interface helper. Complete stdout, stderr, generated files, module origins and the receipt are under `backend_replay/J34`. Receipt SHA-256: `ef268524c4f01d3e6caad26d6eee81da991c151146e2e641d9d00402a82c13ed`.

Source files: `export/latex.py`. Per-file hashes are retained in the matrix.

J35 / Native lualatex PDF compilation

Profile: `host`. **Evidence kind:** `operation`. **Expectation:** passed. Actual compiler execution on a minimal TeX fixture. It does not qualify every document or font.

```
from komorebi_math.export.latex import compile_latex,LatexCompileOptions
text=r'\documentclass{article}\begin{document}Komorebi P. $1+1=2$.\end{document}'
r=compile_latex(text,work/'tex',options=LatexCompileOptions(engine='lualatex',passes=1,timeout=45))
p=r.require();assert p.read_bytes().startswith(b'%PDF-')
observation={'engine':str(r.engine),'valid':bool(r.valid),'pdf_bytes':p.stat().st_size,'return_codes'
    ↪ :r.return_codes,'shell_escape':False}
```

Observed output:

```
{
  "engine": "lualatex",
  "pdf_bytes": 11570,
  "return_codes": [
    0
  ],
  "shell_escape": false,
  "valid": true
}
```

The recipe runner supplies `work` as a fresh directory; `backend_contracts.py` is the supplied finite-interface helper. Complete stdout, stderr, generated files, module origins and the receipt are under `backend_replay/J35`. Receipt SHA-256: `037c0cdf2fbd42b51aab3b4b22e87edd2bfce8a076d5a08cd6dd0aa3ab94d124`.

Source files: `export/latex.py`. Per-file hashes are retained in the matrix.

J36 / Native latexmk PDF compilation

Profile: `host`. **Evidence kind:** `operation`. **Expectation:** passed. Actual compiler execution on a minimal TeX fixture. It does not qualify every document or font.

```
from komorebi_math.export.latex import compile_latex,LatexCompileOptions
text=r'\documentclass{article}\begin{document}Komorebi P. $1+1=2$.\end{document}'
r=compile_latex(text,work/'tex',options=LatexCompileOptions(engine='latexmk',passes=1,timeout=45))
p=r.require();assert p.read_bytes().startswith(b'%PDF-')
observation={'engine':str(r.engine),'valid':bool(r.valid),'pdf_bytes':p.stat().st_size,'return_codes'
    ↪ :r.return_codes,'shell_escape':False}
```

Observed output:

```
{
  "engine": "latexmk",
  "pdf_bytes": 13505,
  "return_codes": [
    0
  ],
  "shell_escape": false,
  "valid": true
}
```

The recipe runner supplies `work` as a fresh directory; `backend_contracts.py` is the supplied finite-interface helper. Complete stdout, stderr, generated files, module origins and the receipt are under `backend_replay/J36`. Receipt SHA-256: `f9453561232428a361adbd172c8de535b04d66a5c7b18e81292e9e84bb0eaf8b`.

Source files: `export/latex.py`. Per-file hashes are retained in the matrix.

J37 / No compiler on PATH

Profile: `base-empty-path`. **Evidence kind:** `expected-refusal`. **Expectation:** passed. An empty child PATH tests real executable discovery refusal, not mocked shutil.

```
from komorebi_math.export.latex import resolve_latex_engine,available_latex_engines
from komorebi_math.exceptions import BackendUnavailableError
assert not any(available_latex_engines().values())
try:resolve_latex_engine('auto')
except BackendUnavailableError as e:observation={'refused':True,'message':str(e)}
else:raise AssertionError('No compiler should be discoverable')
```

Observed output:

```
{
  "message": "no supported LaTeX engine was found on PATH",
  "refused": true
}
```

The recipe runner supplies `work` as a fresh directory; `backend_contracts.py` is the supplied finite-interface helper. Complete stdout, stderr, generated files, module origins and the receipt are under `backend_replay/J37`. Receipt SHA-256: `07f95b95dafb64ffa19d55d058e877899b568bd7f0b9538bee4ce10504a27e2b`.

Source files: `export/latex.py`. Per-file hashes are retained in the matrix.

J38 / Latex engine discovery and default ordering

Profile: `host`. **Evidence kind:** `capability-observation`. **Expectation:** passed. Executable discovery is not proof a particular document will build.

```
from komorebi_math.export.latex import resolve_latex_engine,available_latex_engines
observation={'discovered':available_latex_engines(),'selected_auto':str(resolve_latex_engine('auto'))}
assert observation['discovered'][observation['selected_auto']]
```

Observed output:

```
{
  "discovered": {
    "latexmk": true,
    "lualatex": true,
    "pdflatex": true,
    "xelatex": true
  },
  "selected_auto": "lualatex"
}
```

The recipe runner supplies `work` as a fresh directory; `backend_contracts.py` is the supplied finite-interface helper. Complete stdout, stderr, generated files, module origins and the receipt are under `backend_replay/J38`. Receipt SHA-256: `bd835c211527fe7c631785db02d2183aa0997be665221144808d7854b00c1720`.

Source files: `export/latex.py`. Per-file hashes are retained in the matrix.

J.7 Complete electronic dependency-site census

External import root	Explicit package import/discovery sites
`cupy`	11
`jax`	2
`mpmath`	2
`numpy`	135
`scipy`	4
`sympy`	33
`torch`	3
`yaml`	1

The totals count source sites, including verification modules and repeated uses. They are not the number of required installations or mathematical algorithms. `Block42_Backend_Dependency_Index.json` retains source lines, columns, owners, exact statements, ancestor conditions and source hashes for every site; `Block42_Dependency_Source_Lookup.tsv` supplies all 191 lookup rows. The 135 NumPy sites remain in that full electronic inventory rather than being hidden by the shorter optional-path table below.

Optional root	Package source and line	Site kind
`cupy`	`algorithms/gpu.py:18`	`discovery`
`cupy`	`algorithms/gpu.py:65`	`import`
`sympy`	`analytic/appell.py:93`	`import`
`sympy`	`analytic/appell.py:136`	`import`
`sympy`	`analytic/reciprocal.py:83`	`import`
`scipy`	`analytic/sections.py:266`	`from`
`mpmath`	`analytic/zeros.py:217`	`import`
`sympy`	`arithmetic/recurrence.py:286`	`import`
`sympy`	`backends/__init__.py:26`	`discovery`

Optional root	Package source and line	Site kind
sympy	backends/__init__.py:42	discovery
scipy	backends/__init__.py:43	discovery
mpmath	backends/__init__.py:44	discovery
cupy	backends/__init__.py:45	discovery
sympy	backends/sympy.py:13	import
sympy	core/seeds.py:292	import
sympy	deformation/companions.py:18	import
sympy	deformation/support.py:19	import
scipy	dynamics/_linalg.py:50	from
yaml	export/writers.py:214	import
sympy	generating/closed_forms.py:23	dynamic-import
sympy	generating/cyclotomic.py:15	dynamic-import
jax	nonlinear/autodiff.py:509	dynamic-import
jax	nonlinear/autodiff.py:510	dynamic-import
torch	nonlinear/autodiff.py:609	dynamic-import
sympy	nonlinear/derivatives.py:492	import
sympy	nonlinear/elimination.py:41	import
sympy	nonlinear/elimination.py:64	import
sympy	nonlinear/elimination.py:72	import
sympy	nonlinear/elimination.py:84	import
sympy	nonlinear/elimination.py:125	import
sympy	nonlinear/elimination.py:133	import
cupy	nonlinear/execution.py:24	discovery
cupy	nonlinear/execution.py:408	import
cupy	nonlinear/execution.py:513	import
cupy	nonlinear/gpu.py:52	discovery
cupy	nonlinear/gpu.py:55	dynamic-import
cupy	nonlinear/gpu.py:84	dynamic-import
sympy	nonlinear/obstruction.py:79	import
scipy	refinement/defects.py:18	from
sympy	refinement/defects.py:190	import
sympy	refinement/lattices.py:18	import
sympy	refinement/residue.py:44	import
sympy	refinement/smith.py:18	import
sympy	refinement/smith.py:91	from
sympy	refinement/smith.py:92	from
sympy	refinement/smith.py:93	from
sympy	refinement/weighted.py:374	import
sympy	refinement/weighted.py:390	import
torch	release/_gpu_worker.py:18	discovery
torch	release/_gpu_worker.py:21	import
cupy	release/_gpu_worker.py:57	discovery
cupy	release/_gpu_worker.py:60	import
sympy	verify/deformation.py:52	import
sympy	verify/deformation.py:60	discovery
sympy	verify/refinement.py:73	discovery
sympy	verify/refinement.py:75	import

Literal JAX/Torch child programs contain three additional imports. Seven dynamic expressions are retained without pretending their runtime strings form a closed external dependency set. Examples include extension/module names selected at runtime and compiler choices held in variables. Matplotlib, Numba and llvmlite have no direct import in the scanned package statements; this does not mean external dependencies never import them transitively.

J.8 Constraints, historical environments and CI declarations

The supplied Python 3.11, 3.12 and 3.13 base constraints all state `numpy>=1.26,<3`. The development constraints add bounded pytest/build and development tools. The Windows CUDA files are retained recorded environment profiles, not the active Linux environment and not independently rerun validations. Their filenames containing “validated” do not qualify R2 on this host.

base-py311.txt

Source `komorebi-math/constraints/base-py311.txt`. SHA-256: `99f43573522565d7c480efd1f954892948033915df6a4e685327e995d3396eb6`.

```
numpy>=1.26,<3
```

base-py312.txt

Source `komorebi-math/constraints/base-py312.txt`. SHA-256: `99f43573522565d7c480efd1f954892948033915df6a4e685327e995d3396eb6`.

```
numpy>=1.26,<3
```

base-py313.txt

Source `komorebi-math/constraints/base-py313.txt`. SHA-256: `99f43573522565d7c480efd1f954892948033915df6a4e685327e995d3396eb6`.

```
numpy>=1.26,<3
```

development.txt

Source `komorebi-math/constraints/development.txt`. SHA-256: `947e4674f7c9eee54e394a62632540d3927fcd8dbb48e0a2e51b10b5b4f2665d`.

```
numpy>=1.26,<3
pytest>=8,<10
build>=1.2,<2
ruff>=0.6
mypy>=1.10
mkdocs>=1.6
```

windows-cuda126-validated.txt

Source `komorebi-math/constraints/windows-cuda126-validated.txt`. SHA-256: `9f88b6da39ab93d0dc6e022c659c7af6c3a903ca61f6a9701a060b3c9d707506`.

```
# Recorded user validation environment. PyTorch requires the cu126 index shown in docs/GPU_INSTALLATION.md.
numpy==2.5.2
jax==0.11.1
cupy-cuda12x==14.1.1
numba==0.67.0
llvmlite>=0.49,<0.50
pytest==9.1.1
```

windows-cuda126.txt

Source `komorebi-math/constraints/windows-cuda126.txt`. SHA-256: `22c7a06cc921fb45c64cee96541844ee01d7dbb403316926f80a8286f407ea27`.

```
numpy==2.5.2
jax==0.11.1
cupy-cuda12x==14.1.1
numba==0.67.0
llvmlite>=0.49,<0.50
torch==2.12.1+cu126
torchvision==0.27.1+cu126
```

windows-cuda126.yml

Source `komorebi-math/environment/windows-cuda126.yml`. SHA-256: `8045ec74bab8e2ce38a82449446f4e1c99c030c49245056ceebde986429f2709`.

```
name: komorebi-full
channels:
  - conda-forge
dependencies:
  - python=3.13
  - pip
  - pip:
      - numpy==2.5.2
      - jax==0.11.1
      - cupy-cuda12x==14.1.1
      - numba==0.67.0
      - llvmlite>=0.49,<0.50
      - pytest==9.1.1
      # Install PyTorch separately from the official cu126 index:
      # python -m pip install torch==2.12.1 torchvision==0.27.1 --index-url https://download.pytorch.org/whl/cu126
```

The CUDA constraint file containing Torch names pins `torch==2.12.1+cu126` and `torchvision==0.27.1+cu126`; the "validated" companion omits those lines and points to separate installation instructions. The environment YAML similarly leaves Torch installation to a separate commented command. These are historical source contents. The recorded audit neither fetched those releases nor established that the listed versions form a resolvable environment now. Numba/llvmlite pins are environment context, not declared Komorebi package requirements.

The CPU workflow declares a Linux/Windows by Python-3.11/3.12/3.13 job matrix, plus Linux documentation and distribution jobs. The tagged workflow checks the repaired release identity. Their definitions request package tests, API checks, distribution rebuilding, strict MkDocs and clean artifact checks, but the recorded audit executed no hosted workflow. The complete files and hashes remain in the JSON index. A configured job is not a passing job receipt.

J.9 Reproduction and acceptance checklist

From a fresh working copy, select an interpreter where the unchanged build-2 wheel is installed and the needed test dependencies are available. The runtime probe tool checks all 366 package-file hashes and refuses a source-checkout import. It makes no dependency installations or network requests. Run data generation and tests in a working copy because regenerated files or receipts invalidate the outer archive hashes.

```
python tools/block42/inventory.py
python tools/block42/run_probes.py --output manual/verification/block42-appendix-j/backend_replay
python tools/block42/generate_matrix.py
python tools/block42/run_reference_tests.py
python tools/block42/run_source_tests.py
python tools/block42/generate_manual.py
```

```
python tools/block42/build_pdfs.py
python VERIFY_BLOCK42_PACKAGE.py
```

The reference tests require pytest and packaging in addition to the selected numerical environment. PDF regeneration uses Pandoc, pdflatex and the inherited TeX packages; PDF inspection uses PyMuPDF and Pillow. These are documentation tools, not additions to the mathematical library runtime requirements. The probe suite is pinned to this audited CPU host profile. A different hardware configuration needs reviewed expectations rather than relabeling an unexpected GPU success or failure.

Executed check	Result
Complete unchanged source tests	1,203 passed
Reference/source/metadata/interface tests	618 passed
Actual probe expectations	38/38 satisfied
Finite native LaTeX engine fixtures	4/4 produced PDFs
Built-in linear algebra methods exercised	19 per backend
Required GPU qualification	Failed, retained as negative
Library files preserved against R2	901/901 identical
Package files versus installation	366/366 identical

The source suite, method fixtures, source-index tests and saved-receipt checks overlap. They are not additive counts of independent mathematical theorems. The 618 reference tests include per-file and per-site checks, not that many independent backend workloads. Test receipts must be read with their current input bindings, not only with an old passing count.

J.10 Completion boundary and next appendix

Reference boundaries. This version-bound backend and dependency reference records the listed finite Linux observations. That audit did not qualify native Windows/PowerShell, other Python versions, dependency lower bounds, CUDA, strict MkDocs, fresh dependency-isolated installation or final publication. Portable-decoder and certificate-validation limitations remain separate. No new Lean proof audit formed part of those observations; Appendix O identifies the final-source qualification records.

Resource models and guards are described in Appendix K. Changing a documentation file alone does not change the installed mathematical package or its qualification.

Appendix K. Complexity, memory, and resource-budget tables

Reference baseline: Komorebi Math 1.0.0b4, Repair R2, wheel build tag 2. This appendix gives source-derived operation and storage models, the complete explicitly defined budget fields, a lexical parameter and guard census, and bounded execution evidence. It is not a proof that every operation has a budget, that every cap runs before allocation, or that the process is protected against arbitrary untrusted input.

Estimate the request before increasing a limit. Start with the operation profile in K.4, check when its guard runs in K.3, and use the worksheet in K.10 before allocating a larger problem. K.5 translates several common shapes into payload sizes; K.9 explains why a long-lived process can retain more memory than one result suggests.

For each planned run, write down four quantities: the number of output entries, their scalar representation, the additional work needed to produce them, and the point where the selected limit is checked. This separates a manageable output from an expensive construction. A small sparse result, for example, does not by itself show that discovering its support was cheap.

A small reading example. For a dense packet tensor, use the source dimension, target dimension and derivative order in the shape formula given below. Compare the resulting cell count with the element cap first. Next convert cells to bytes using the actual dtype, and allow separately for inputs and temporaries. For exact objects, include the represented integers or fractions as well as array slots. The resource record should retain these assumptions so that a later change of dtype or order does not reuse an obsolete estimate.

K.1 Units, parameters, and what a bound establishes

Use four distinct ledgers. **Cardinality** counts entries, states, addresses, paths or callback invocations. **Arithmetic work** counts scalar operations under a stated model. **Bit work** additionally accounts for the sizes of exact numerators, denominators and intermediate values. **Observed resources** are time and allocations measured in one execution. A bound in one ledger does not automatically bound the others.

Symbol	Meaning in this appendix
`n, m`	Ambient source and target dimension
`p, q`	Number of source and target packets
`r`	Derivative order or explicitly named lattice rank
`N`	Requested truncation degree, hence `N+1` coefficients
`k`	Recurrence order or explicitly named inner matrix dimension
`V, E`	Unique expression nodes and traversed child edges
`B`	Actual JSON bytes, or explicitly named quantized error allowance
`L, S`	Visited-state count or reached-automaton states, as stated locally
`w`	Fixed-width element size in bytes

A dense fixed-width array with shape `(d0,...,dt)` has logical data-buffer size `w*product(di)`. For object arrays, this formula counts pointer slots, not the referenced Python objects. Views can share an underlying allocation. Python container headers, tuple pointers, integer payload, allocation overhead, temporaries, retained caches, native libraries and device memory must be considered separately. A sum of object sizes can double-count shared references and is still not peak process resident memory.

Every big-O statement below is a reviewed model of the specified implementation, with its source location and checksum in the ledger. It is not an independently verified asymptotic theorem. Classical dense matrix multiplica-

tion is modeled by cubic scalar work; no claim is made that every backend uses exactly that algorithm or the same workspace. For exact arithmetic, replace constant scalar cost by the appropriate cost at the actual intermediate bit sizes. User callbacks and basis normalization functions have separate cost contracts.

K.2 All three explicit budget objects

The package declares three named budget dataclasses with **20 fields**. All fields require positive integer values and reject booleans in construction. The guards on the operations they govern differ. The objects are local arguments or stored records, not process-wide controllers. Defaults below are pinned source values, not recommended maximum workloads.

GeneratingBudget

Field	Exact default
`maximum_degree`	100,000
`maximum_coefficient_bits`	4,000,000
`maximum_matrix_dimension`	4,096
`maximum_reachable_states`	100,000
`maximum_local_combinations`	1,000,000
`maximum_modulus`	2,000,000
`maximum_serialized_bytes`	67,108,864
`maximum_recursion_depth`	1,000

`require_degree`, `require_terms`, `require_serialized_bytes` and `require_coefficient_bits` permit zero requests; dimensions, states, local combinations and modulus require positive requests. Equality with the cap is accepted, one above is refused with `ResourceLimitError`. `require_terms` and `require_local_combinations` use the same field but differ on zero. `max_modulus_table` is a compatibility property for `maximum_modulus`.

Enforcement is not automatic. The eight standalone guard methods are callable contracts. This snapshot contains no explicit package call to `require_serialized_bytes`, and `maximum_recursion_depth` has no associated guard method. These fields are preserved by serialization, but the generic portable codec does not adopt them as its own policy. With `maximum_serialized_bytes=1`, a finite-series object still serializes to a larger envelope. Setting `maximum_recursion_depth=1` does not replace the codec argument `max_depth`. The source census is lexical, not a proof that an arbitrary caller could never invoke a guard dynamically.

`FiniteNormalizedSeries` checks degree and recognized coefficient bit sizes, but first consumes and coerces the entire supplied iterable. It checks output coefficient bits when constructing results, not after every intermediate addition or multiplication. The helper recognizes `int` and `Fraction`; other domain representations can have a helper bit count of zero. The unbudgeted `FormalSeries` is a separate class. A permitted degree of 100,000 can request **5,000,150,001 convolution terms** in the triangular implementation; the default is not a promise that such a run is practical.

Source: `generating/budget.py`. Complete field expressions and source SHA-256 values are in `Block43_Resource_Complexity_Index.json`.

CoordinateBudget

Field	Exact default
`maximum_matrix_dimension`	512
`maximum_polynomial_degree`	512
`maximum_horizon`	4,096
`maximum_exact_entries`	1,000,000

`require_matrix(rows, columns, operation=...)` applies both per-dimension and entry-product limits. Degree and horizon have separate methods. In the dense Boolean-lattice transform, the matrix guard precedes the comprehension. At default dimension 512, rank 9 reaches the dimension boundary and rank 10 is refused. This statement is arithmetic and source inspection; no default-maximum rank-9 benchmark was run.

The low-level matrix guard checks signs and comparisons but does not strictly enforce integer row/column types. Positive fractional dimensions and booleans can pass the guard itself; a producer may impose its own stronger type validation. The constructor of the budget and the degree/horizon methods are stricter. Low-level exact matrix helpers do not automatically receive this budget, and `maximum_exact_entries` counts entries rather than their full rational payload.

Source: `coordinates/budget.py`. Complete field expressions and source SHA-256 values are in `Block43_Resource_Complexity_Index.json`.

ExpressionBudget

Field	Exact default
`max_nodes`	10,000
`max_depth`	1,000
`max_steps`	50,000
`max_degree`	512
`max_entries`	1,000,000
`max_coefficient_bits`	1,000,000
`max_serialized_bytes`	64,000,000
`max_symbol_length`	1,024

`preflight` performs a color-map traversal, rejects cycles and builds postorder. It checks `max_nodes` and `max_steps` only after this graph traversal; `max_steps` is `nodes+edges`, not executed CPU instructions or the cost of later matrix operations. Shared nodes count once, while repeated incoming edges still count. Depth, exact entries, polynomial degree, coefficient bits and symbol length are checked in subsequent structural passes.

`serialized_bytes` in `ExpressionStats` is deterministic structural accounting, not the size of a portable JSON envelope. The expression JSON reader separately measures actual UTF-8 bytes before `json.loads`. Obtaining, decoding and re-encoding the input text already takes memory. Expression normalization and recursive serialization have their own code paths and cannot be assumed safe at every permitted default depth merely because preflight itself is iterative.

Source: `expression/model.py`. Complete field expressions and source SHA-256 values are in `Block43_Resource_Complexity_Index.json`.

K.3 When the checks happen

Operation	Check timing
Finite-series constructor	After full iterable coercion
Finite-series output bit size	At result construction
Dense subset transform	Before matrix comprehension
Expression node/step counts	After graph traversal
Expression JSON byte length	Before JSON parsing, after text conversion
Packet-jet plan addresses	After complete enumeration
Snapshot basis evaluations	Before basis/evaluation loop
Snapshot dense elements	Before `np.zeros`
Stream selection	Before selected tuple construction
Height-bounded trees	After exact pre-count, before family generation
Leaf-bounded trees	After cached family generation
Pareto tree candidates	After child frontiers, before product plans
Weighted paths	After next path construction

Operation	Check timing
Modular recurrence period	Loop-bound guard

These distinctions are implementation facts, not reasons to remove guards. The safe response to a late check is to compute a cheap count before the call where possible, supply bounded inputs, retain smaller explicit caps, and keep untrusted or unpredictable callbacks in a separately controlled execution environment. Do not use `validate=False`, a truthy stored certificate flag, or a recomputed checksum as a resource or correctness workaround.

K.4 Source-derived complexity and storage models

The following **26 profiles** cover the principal dense, exact, packet, expression, enumeration and streaming families. The named-parameter census in K.8 is exhaustive under its stated pattern; these 26 reviewed algorithmic profiles are not an assertion that every one of the 1,526 API reference cards has a new complexity proof. Exact source fragments are preserved electronically rather than repeating implementation bodies in print.

K01 / Exact matrix product

Work model. Exactly a*b*c scalar product terms for (a,b) by (b,c); O(a*b*c) rational operations.

Storage model. a*c result entries plus input normalization copies; rational object payload is additional.

Derivation from source. Three nested row/column/inner-index traversals construct the result. Summing each dot product also performs b additions.

Guard and boundary. The low-level helper has shape validation but no CoordinateBudget argument. Exact scalar cost grows with numerator/denominator bit length.

Implementation: `coordinates/exact.py:matrix_multiply`, lines 97-114. SHA-256 `730ffa5075436ef26d9d65167c43073edd96971d14fe258fa94aef14556f8b0e`.

K02 / Exact row reduction and inverse

Work model. O(a*b*min(a,b)) rational elimination operations; square inverse O(n^3).

Storage model. O(a*b) rational entries; inverse uses an n by 2n augmented matrix.

Derivation from source. For each pivot, scan rows, normalize a row and eliminate other rows across the columns.

Guard and boundary. No global coefficient-bit or allocation budget in the low-level reducer. Intermediate fractions can be much larger than input fractions.

Implementation: `coordinates/exact.py:rref`, lines 157-184. SHA-256 `730ffa5075436ef26d9d65167c43073edd96971d14fe258fa94aef14556f8b0e`.

K03 / Exact matrix powers

Work model. O(log(e+1)) square matrix multiplications, hence O(n^3*log(e+1)) arithmetic work.

Storage model. O(n^2) live matrix entries up to constant-factor temporaries.

Derivation from source. Repeated squaring shifts the nonnegative exponent and multiplies the accumulating result on set bits.

Guard and boundary. This is an arithmetic-operation model. Integer/fraction growth can dominate; no claim of logarithmic bit complexity or tiny memory.

Implementation: `coordinates/exact.py:matrix_power`, lines 130-146. SHA-256 `730ffa5075436ef26d9d65167c43073edd96971d14fe258fa94aef14556f8b0e`.

K04 / Dense Boolean-lattice coordinates

Work model. 4^r matrix cells for rank r, before downstream exact matrix work.

Storage model. 4^r Fraction entries, stored in 2^r rows of length 2^r.

Derivation from source. The implementation builds every row/column pair; it is not an in-place fast subset transform.

Guard and boundary. CoordinateBudget checks dimension and entry product before the matrix comprehension. The default dimension 512 permits rank 9 but rejects rank 10; rank 9 is not benchmark-qualified here.

Implementation: `coordinates/combinatorial.py:subset_zeta_transform`, lines 315-350. SHA-256 `bb0a73e44deecfd2c64ea6c1418c38be7792d09d3106b22facda9dbf72c1dbc1`.

K05 / Formal-series convolution

Work model. (N+1)(N+2)/2 coefficient product terms through degree N; O(N^2) scalar operations.

Storage model. O(N) output and padded coefficient arrays, plus normalization/basis caches.

Derivation from source. For each n=0..N, every k=0..n contributes a structure constant times a[k] times b[n-k].

Guard and boundary. FormalSeries has no GeneratingBudget argument. A degree guard must be applied explicitly by the caller before invoking an expensive operation.

Implementation: `generating/series.py:FormalSeries.multiply`, lines 123-134. SHA-256 `e53da8fbe75432cffb8739880e1db7df027ce5abf9112f30b49afdbb5f2160af`.

K06 / Budgeted finite-series construction

Work model. O(L) domain coercions and coefficient bit checks for L supplied values.

Storage model. All L coerced coefficients are materialized before degree refusal.

Derivation from source. The tuple(domain.coerce(value) for value in coefficients) is constructed before require_degree(len(values)-1).

Guard and boundary. A small maximum_degree does not stop an infinite or excessively long input iterator. Supply an independently bounded iterable. Coefficient bits are recognized only for int/Fraction by the helper.

Implementation: `generating/finite_series.py:FiniteNormalizedSeries.__init__`, lines 102-125. SHA-256 `b31dab73da218f69ad5017f534ab75f632baa1198b953a688e8afca4b5397bb4`.

K07 / Finite normalized-series convolution

Work model. (N+1)(N+2)/2 terms, each two domain multiplications and one addition, excluding conversion.

Storage model. O(N) main arrays; output coefficient size is checked when the result is constructed.

Derivation from source. Nested n/k loops fill a finite output. Compatibility and basis conversion precede truncation guards.

Guard and boundary. Input/output degree caps do not bound intermediate scalar arithmetic. Other.convert can do work before an explicitly over-limit requested degree is rejected.

Implementation: `generating/finite_series.py:FiniteNormalizedSeries.multiply`, lines 280-308. SHA-256 `b31dab73da218f69ad5017f534ab75f632baa1198b953a688e8afca4b5397bb4`.

K08 / Formal composition, reciprocal and integer powers

Work model. Dense composition O(N^3); reciprocal O(N^2); repeated-squaring integer power O(N^2*log(e+1)).

Storage model. O(N) live coefficient buffers for each operation, excluding callback caches and scalar payload.

Derivation from source. Composition makes N+1 ordinary truncated convolutions of degree N; reciprocal uses triangular recurrence.

Guard and boundary. Zero entries can reduce convolution work but do not establish a subcubic worst-case composition algorithm. Basis gamma/structure-constant cost is separate.

Implementation: `generating/series.py:FormalSeries.compose`, lines 171-192. SHA-256 `e53da8fbe75432cffb8739880e1db7df027ce5abf9112f30b49afdbb5f2160af`.

K09 / Recurrence cache growth

Work model. From a cold cache, O(n*k) scalar work to index n for order k; repeated cached lookup O(1).

Storage model. O(n) retained terms after the highest requested index.

Derivation from source. The while loop appends every missing predecessor term before returning the requested index.

Guard and boundary. This term method is not logarithmic jumping. State/jump APIs are separate. Exact value sizes and an indefinitely retained cache matter.

Implementation: `arithmetic/recurrence.py:LinearRecurrence.term`, lines 163-173. SHA-256 `3374acced7d63e284f8b3ba7da6314bf44439ac7ea6ceea234ab006fd0185cbd`.

K10 / Finite modular recurrence period

Work model. For L nonrepeated states, O(L*k^2) matrix-vector work plus final determinant computation.

Storage model. O(L*k) state entries and O(k^2) companion entries.

Derivation from source. Each new state is stored in seen and advanced through a dense object matrix. The loop is range(limit+1).

Guard and boundary. Default max_steps is modulus^k+1, a mathematical finite bound, not a practical memory cap. Supply a smaller explicit max_steps for exploration.

Implementation: `arithmetic/recurrence.py:LinearRecurrence.period_mod`, lines 230-275. SHA-256 `3374acced7d63e284f8b3ba7da6314bf44439ac7ea6ceea234ab006fd0185cbd`.

K11 / Cartier automaton compile and lookup

Work model. S*p polynomial transitions for S reached states and prime p; scalar transition cost depends on degree. Query uses the base-p digits of n.

Storage model. O(S*p) transition indices plus O(S*D) stored state coefficients of width D.

Derivation from source. Every newly reached state gets one row of p transitions. Query follows a single precomputed state per digit.

Guard and boundary. The new-state guard runs after computing its candidate polynomial. Compilation, minimization and large-integer digit division are separate from query transition count.

Implementation: `generating/cartier.py:_reachable_automaton`, lines 214-268. SHA-256 `73ec9bbb8ef2b8d7e51e72d37757f1c9c63073a165b1e05e1b9a01ea7712ee56`.

K12 / Packet-axiom certificate

Work model. Exactly p^2 backend matrix products for p projectors; O(p^2*n^3) under a classical dense-product model.

Storage model. p*n^2 input entries and O(n^2) main temporary residuals.

Derivation from source. p idempotence products plus p*(p-1) cross-annihilation products.

Guard and boundary. Backend multiplication, norm and exactness rules determine actual cost. Certificate generation is work and can dominate cheap downstream operations.

Implementation: `core/packets.py:PacketSystem.certificate`, lines 92-136. SHA-256 `02427ff27728818c95e2e0d4d6d0f8dcdb3c8821a17fcf5cc63a312c4ad7cf31`.

K13 / Packet analysis

Work model. p dense matrix-vector applications; O(p*n^2) under the dense model.

Storage model. p*n ambient output entries in addition to* pn^2 stored projectors.

Derivation from source. Each projector is applied to the full input vector and all projected vectors are retained.

Guard and boundary. Packet decomposition does not automatically imply compressed storage. There is no per-call memory budget here.

Implementation: `core/packets.py:PacketSystem.analyze`, lines 154-159. SHA-256 `02427ff27728818c95e2e0d4d6d0f8dcdb3c8821a17fcf5cc63a312c4ad7cf31`.

K14 / Tensor products of packet systems

Work model. p*q Kronecker products, followed by validation of the resulting* pq projectors.

Storage model. p*q*(n*m)^2 scalar projector entries.

Derivation from source. The two projector loops form ambient Kronecker matrices and then invoke default PacketSystem construction.

Guard and boundary. Account for the additional (p*q)^2 products in validation. No tensor-specific allocation cap is applied before constructing the matrices.

Implementation: `core/packets.py:PacketSystem.tensor`, lines 220-234. SHA-256 `02427ff27728818c95e2e0d4d6d0f8dcdb3c8821a17fcf5cc63a312c4ad7cf31`.

K15 / Packet-jet family addresses and plan

Work model. A=q*p*^*r full addresses at order r; O(A*r) address payload before name lookup/grouping costs.

Storage model. O(A*r) active/pruned address records plus groups; retaining pruned records still costs memory.

Derivation from source. family.addresses() materializes all addresses before len(all_addresses)>max_addresses is tested. Grouping then retains active/pruned tuples.

Guard and boundary. max_addresses is a post-enumeration guard in R2. Name lookup, sorting and support partition checks add work. Predict q*p^r before calling.

Implementation: `nonlinear/execution.py:compile_packet_jet_plan`, lines 124-204. SHA-256 `474902031dcfa4066f4cc60b957489510b95d4d5ca9848ea9bc58844639f2158`.

K16 / Projection-cached packet-jet execution

Work model. Counts: r*p input projections, G derivative calls, A_active output projections, plus verification/direct work of the executor.

Storage model. Projected direction cache plus returned evaluations; backend/derivative workspace is extra.

Derivation from source. Grouping identical ordered input-packet tuples shares derivative evaluation across output packets. The built-in estimator reports G and A_active.

Guard and boundary. A call count is not a FLOP count. The derivative provider can be expensive or allocate its own tensor. The estimator compiles a plan unless one is supplied, so it is not a cheap universal preflight.

Implementation: `nonlinear/profiling.py:estimate_packet_jet_complexity`, lines 193-260. SHA-256 `034aa8498ddafce588627e5219ba80b7ee5950b512548f09503eff07263ab0de`.

K17 / Ambient snapshot capture

Work model. n^r jet evaluations of one fixed block, regardless of how many coefficients survive.

Storage model. O(n^2) ambient basis plus up to n^r records of r indices and m output scalars.

Derivation from source. The evaluator traverses the complete ambient Cartesian basis and omits coefficients only after evaluation and zero testing.

Guard and boundary. max_evaluations is checked before creating the basis and entering the evaluation loop. Sparse stored output does not mean sparse capture work.

Implementation: `nonlinear/snapshots.py:snapshot_packet_jet`, lines 280-368. SHA-256 `bd7a755c4b145b2fea23aeab89a1fa81e2d1fdcbe7117b365c0152b5e970d5d2`.

K18 / Snapshot dense expansion

Work model. Initialize m*n^r scalar cells and write all retained coefficient vectors.

Storage model. m*n^r scalar cells; numerical route uses complex dtype, exact route object pointers plus referenced objects.

Derivation from source. Shape is (target_dimension, source_dimension repeated order times).

Guard and boundary. max_elements is checked before np.zeros. Counting object array .nbytes is not the total memory of exact coefficients. Numerical default dtype is complex, not float64.

Implementation: `nonlinear/snapshots.py:PacketJetSnapshot.to_dense`, lines 205-222. SHA-256 `bd7a755c4b145b2fea23aeab89a1fa81e2d1fdcbe7117b365c0152b5e970d5d2`.

K19 / Streaming capture and materialization

Work model. Up to n^r evaluations, bounded per call by evaluation_budget; chunk I/O and manifest updates add work.

Storage model. Chunk coefficient buffers plus an expanding manifest, ambient basis, provider state and parsed JSON.

Derivation from source. Each loop commits at most chunk_size evaluations, then records progress. A pause is a valid incomplete stream.

Guard and boundary. chunk_size counts evaluations for capture, not bytes. Rewriting cumulative metadata can create superlinear metadata I/O as chunk count grows. max_coefficients guards selection/materialization, not arbitrary chunk byte size.

Implementation: `nonlinear/streaming.py:capture_packet_jet_stream`, lines 1010-1188. SHA-256 `b4db5f4d1493421b3b011167915735145c41bdbc5e8cdd64f63b6552bb6d0b90`.

K20 / Expression structural preflight

Work model. O(V+E+C) structural visits plus semantic-space and scalar text costs for a DAG.

Storage model. O(V+E) traversal state and coefficient/space witness temporaries.

Derivation from source. Color-map DFS builds postorder; node and step totals are checked only after traversal. Edges are counted even when child nodes are shared.

Guard and boundary. max_nodes is not an early traversal stop. max_steps is nodes+edges, not a CPU instruction budget. Structural serialized_bytes is not the final JSON envelope size.

Implementation: `expression/model.py:ExpressionBudget.preflight`, lines 580-676. SHA-256 `e01883acc993da14266dff0fd2518d8e9f045ef615a1094f3fbecc46b1ce4649`.

K21 / Expression JSON transport

Work model. O(B) byte counting/parse work plus decoded-tree and expression validation, where B is actual JSON bytes.

Storage model. Full input string/bytes, parse tree and expression objects may coexist.

Derivation from source. The UTF-8 size is checked before json.loads, but bytes decoding and text re-encoding already take memory.

Guard and boundary. Stricter than generic portable loads, yet not a process sandbox. A positive limit cannot constrain memory spent obtaining the input or allocations made before the length check.

Implementation: `expression/portable.py:expression_from_json`, lines 972-1010. SHA-256 `366c25d81842e91cbf228f84b10ad6626493d48eef3acad1129439cbdaf0e9d0`.

K22 / Height-bounded ordered trees

Work model. Exact count N_0=1, N_h=1+sum_{a in arities} N_(h-1)^a. Enumeration is output-sensitive.

Storage model. All generated tree objects/tuples are retained by unbounded lru_cache decorators.

Derivation from source. The count recurrence runs before tree generation and max_trees rejection for this height-based API.

Guard and boundary. Counting very large integers still consumes resources. max_trees is per requested family, not a cumulative cache cap. The separate leaf-count API checks after generation.

Implementation: `nonlinear/trees.py:ordered_rooted_trees`, lines 208-225. SHA-256 `94ce7611a8552386e3e8328bd9a9937a46efae730acf270feb99b9c37cb3aad4`.

K23 / Tree optimization versus resource budget

Work model. At most (B+1)^2 state-pair candidates per child merge, before plan-copy costs.

Storage model. Up to B+1 best plans per table; each plan retains leaves and decisions.

Derivation from source. B is quantized additive error allowance. Nested prior/child state loops enumerate feasible summed weights.

Guard and boundary. B is not a byte cap. O(V*B^2) is only the scalar DP-transition model; plan tuple copying can add up to O(V) per candidate. Lagrangian tree copying also prevents an unconditional linear-time Python claim.

Implementation: `algorithms/compiler.py:compile_tree_budget`, lines 140-243. SHA-256 `2bf95f61c84f3fd3f2256aaa3d02581c50cd5a81dab5c2a78565e0471621579f`.

K24 / Pareto tree enumeration

Work model. At a node, product of child frontier sizes candidates; pairwise frontier pruning can be quadratic in candidate count.

Storage model. All split plans plus keep plan and temporary sorted/frontier lists.

Derivation from source. max_plans is checked against the Cartesian product before materializing split plans, but after child frontiers exist.

Guard and boundary. The returned candidate set can include an extra keep plan. The guard is local, not total tree work or total byte accounting. Plan payload lengths matter.

Implementation: `algorithms/compiler.py:pareto_frontier`, lines 252-311. SHA-256 `2bf95f61c84f3fd3f2256aaa3d02581c50cd5a81dab5c2a78565e0471621579f`.

K25 / Krylov basis with orthogonality check

Work model. At most s operator applications; O(n*s^2) orthogonalization and final Gram check.

Storage model. O(n*s+s^2) numeric entries plus operator/provider workspace.

Derivation from source. Each new vector is projected against the existing basis. The final certificate forms Q*Q and an identity matrix.

Guard and boundary. The steps bound is 1..n, not a byte budget. Matrix-free input avoids storing a dense operator only if the callback does so.

Implementation: `algorithms/krylov.py:krylov_basis`, lines 41-85. SHA-256 `d6c5ae7cdd4ac9beabcda0fbdd4a16392b2d0f444145ecd9a3b64c2f4cda725d`.

K26 / Weighted-path enumeration

Work model. Output-sensitive generation with path-tuple copying, followed by sorting retained paths.

Storage model. Generated stack paths and retained output paths, each carrying all vertices and edges.

Derivation from source. The generated counter increments before the target-output filter and is checked after constructing the next path.

Guard and boundary. max_paths counts generated accepted-weight prefixes, not only paths ending at the requested target; None explicitly disables it. Positive weights control depth but not a polynomial count in precision.

Implementation: `refinement/weighted.py:WeightedTransferGraph.paths_below_weight,` lines 155-200. SHA-256 `94c13dd4ed26909c8cafffd2fcc88069d968ff38e5c7726b593316934553a012`.

K.5 Capacity tables without large allocations

The following figures are computed from shapes and element widths. They do **not** allocate the forecast objects and do not include extra working memory. One MiB is `2**20` bytes and one GiB is `2**30` bytes.

Dense shape	float64 payload	complex128 payload
`512 by 512`	2.0 MiB	4.0 MiB
`1024 by 1024`	8.0 MiB	16.0 MiB
`4096 by 4096`	128.0 MiB	256.0 MiB
`8192 by 8192`	512.0 MiB	1,024.0 MiB

A factorization, residual certificate, input conversion or temporary product can hold additional full matrices. These entries are payload sizes, not permission to fill the available memory completely. Exact object arrays have the same slot-count formula but a different referent cost.

Source n / target m / order r	Basis evaluations n^r	Dense cells m*n^r	complex128 payload
`4 / 4 / 4`	256	1,024	0.016 MiB
`8 / 8 / 4`	4,096	32,768	0.500 MiB
`32 / 32 / 4`	1,048,576	33,554,432	512.000 MiB
`64 / 64 / 4`	16,777,216	1,073,741,824	16,384.000 MiB

These are ambient tensor sizes, not packet-address counts. The default snapshot dense cap is ten million elements and would refuse the last two rows. Its numerical route uses complex cells. A higher explicit cap does not create memory or make the derivative provider cheaper.

Source packets p / target packets q / order r	Full family addresses q*p^r
`2 / 2 / 2`	8
`4 / 4 / 4`	1,024
`8 / 8 / 5`	262,144
`16 / 16 / 5`	16,777,216

Each address carries an ordered tuple and labels, not just a single integer. A predicted family exceeding the selected address cap should be rejected before calling the R2 plan compiler, whose own check follows enumeration. Even a pruned plan retains its pruned-address records. Support discovery and numerical zero testing are separate work, not free inputs to a sparsity claim.

K.6 Observed finite resource samples

The local profile uses Linux and `3.13.5 (main, Jun 25 2025, 18:55:22) [GCC 14.2.0]`, with NumPy `2.3.5`. Each record has one warmup and three measured repetitions, with thread-count environment variables set to one by the launcher. Inputs are constructed before profiling. Compilation/import time, cold caches and input construction are excluded. Timings include tracing overhead. No performance threshold is used to declare correctness.

Operation	Size	Median time (ms)	Median traced peak (bytes)
series	8	0.586215	3,520
series	16	1.609129	5,248
series	32	5.925964	8,808
exact matrix	4	0.247243	3,392
exact matrix	8	1.325661	6,208
exact matrix	12	4.028080	11,056
numpy matrix	16	0.027742	2,696
numpy matrix	32	0.025418	8,840
numpy matrix	64	0.057316	33,416

These nine workload records contain 27 timed samples. They are not nine asymptotic experiments or proof of an optimization. NumPy timings at these sizes can be nonmonotone because fixed costs and measurement noise dominate. The complete individual nanosecond and allocation samples, scope strings and environment are retained in `resource_replay/R36/result.json`. The final sealed-delivery replay records a new set of observations; it need not reproduce these measured values.

The library profiler uses `tracemalloc`, not peak process RSS or GPU memory. Native allocations may be outside its tracing scope, and an input created before tracing is not newly allocated inside the measurement. The profiler does not add general GPU synchronization. If a tracer is already active, it resets that tracer's peak and subtracts the starting baseline; that changes the outer peak statistic. Choose a separate profiling process when peak-statistic ownership matters. Object-buffer and referent sizes are demonstrated by R35.

K.7 Executed boundary probes and observations

All **36 probe expectations** below passed in bounded child processes against the separately installed build-2 wheel. Each process has active assertions and a 90-second wall timeout. Selected probes use counting iterators, wrappers, a parser sentinel, cache introspection or a custom finite expression node to observe order/counts. Such instrumentation is identified in the records and is not independent hardware qualification. No intentionally infinite iterable, huge allocation or uncontrolled recursion experiment is run. A passing probe may establish that a documented limit is permissive or late, not that the library is hardened.

R01 / Generating guard: require_degree

The standalone method accepts the exact limit and rejects limit+1. This alone does not prove all producers call it.

At limit: `4`.

Boolean: `message=degree must be a nonnegative integer; type=ValidationError`.

Field: `maximum_degree`.

Limit: `4`.

Over: `message=degree 5 exceeds maximum_degree=4; type=ResourceLimitError`.

Zero allowed: `True`.

Reproduce: from a working copy with the pinned wheel installed, use `python tools/block43/run_probes.py --worker R01 --output <fresh-existing-scratch-directory>`. The all-probe launcher creates fresh directories and verifies package parity. Receipt: `manual/verification/block43-appendix-k/resource_replay/R01/result.json`.

R02 / Generating guard: require_coefficient_bits

The standalone method accepts the exact limit and rejects limit+1. This alone does not prove all producers call it.

At limit: `4`.

Boolean: `message=coefficient bit length must be nonnegative; type=ValidationError`.

Field: `maximum_coefficient_bits`.

Limit: `4`.

Over: `message=coefficient bit length 5 exceeds maximum_coefficient_bits=4; type=ResourceLimitError`.

Zero allowed: `True`.

Reproduce: from a working copy with the pinned wheel installed, use `python tools/block43/run_probes.py --worker R02 --output <fresh-existing-scratch-directory>`. The all-probe launcher creates fresh directories and verifies package parity. Receipt: `manual/verification/block43-appendix-k/resource_replay/R02/result.json`.

R03 / Generating guard: require_matrix_dimension

The standalone method accepts the exact limit and rejects limit+1. This alone does not prove all producers call it.

At limit: `4`.

Boolean: `message=dimension must be a positive integer; type=ValidationError`.

Field: `maximum_matrix_dimension`.

Limit: `4`.

Over: `message=matrix dimension 5 exceeds maximum_matrix_dimension=4; type=ResourceLimitError`.

Zero allowed: `False`.

Reproduce: from a working copy with the pinned wheel installed, use `python tools/block43/run_probes.py --worker R03 --output <fresh-existing-scratch-directory>`. The all-probe launcher creates fresh directories and verifies package parity. Receipt: `manual/verification/block43-appendix-k/resource_replay/R03/result.json`.

R04 / Generating guard: require_states

The standalone method accepts the exact limit and rejects limit+1. This alone does not prove all producers call it.

At limit: `4`.

Boolean: `message=states must be a positive integer; type=ValidationError`.

Field: `maximum_reachable_states`.

Limit: `4`.

Over: `message=state request 5 exceeds maximum_reachable_states=4; type=ResourceLimitError`.

Zero allowed: `False`.

Reproduce: from a working copy with the pinned wheel installed, use `python tools/block43/run_probes.py --worker R04 --output <fresh-existing-scratch-directory>`. The all-probe launcher creates fresh directories and verifies package parity. Receipt: `manual/verification/block43-appendix-k/resource_replay/R04/result.json`.

R05 / Generating guard: require_local_combinations

The standalone method accepts the exact limit and rejects limit+1. This alone does not prove all producers call it.

At limit: `4`.

Boolean: `message=combinations must be a positive integer; type=ValidationError`.

Field: `maximum_local_combinations`.

Limit: `4`.

Over: `message=local enumeration requires 5 combinations, exceeding maximum_local_combinations=4; type=ResourceLimitError`.

Zero allowed: `False`.

Reproduce: from a working copy with the pinned wheel installed, use `python tools/block43/run_probes.py --worker R05 --output <fresh-existing-scratch-directory>`. The all-probe launcher creates fresh directories and verifies package parity. Receipt: `manual/verification/block43-appendix-k/resource_replay/R05/result.json`.

R06 / Generating guard: require_modulus

The standalone method accepts the exact limit and rejects limit+1. This alone does not prove all producers call it.

At limit: `4`.

Boolean: `message=modulus must be a positive integer; type=ValidationError`.

Field: `maximum_modulus`.

Limit: `4`.

Over: `message=modulus 5 exceeds maximum_modulus=4; type=ResourceLimitError`.

Zero allowed: `False`.

Reproduce: from a working copy with the pinned wheel installed, use `python tools/block43/run_probes.py --worker R06 --output <fresh-existing-scratch-directory>`. The all-probe launcher creates fresh directories and verifies package parity. Receipt: `manual/verification/block43-appendix-k/resource_replay/R06/result.json`.

R07 / Generating guard: require_serialized_bytes

The standalone method accepts the exact limit and rejects limit+1. This alone does not prove all producers call it.

At limit: `4`.

Boolean: `message=serialized size must be a nonnegative integer; type=ValidationError`.

Field: `maximum_serialized_bytes`.

Limit: `4`.

Over: `message=serialized payload has 5 bytes, exceeding maximum_serialized_bytes=4; type=ResourceLimitError`.

Zero allowed: `True`.

Reproduce: from a working copy with the pinned wheel installed, use `python tools/block43/run_probes.py --worker R07 --output <fresh-existing-scratch-directory>`. The all-probe launcher creates fresh directories and verifies package parity. Receipt: `manual/verification/block43-appendix-k/resource_replay/R07/result.json`.

R08 / Generating guard: require_terms

The standalone method accepts the exact limit and rejects limit+1. This alone does not prove all producers call it.

At limit: `4`.

Boolean: `message=term count must be a nonnegative integer; type=ValidationError`.

Field: `maximum_local_combinations`.

Limit: `4`.

Over: `message=term count 5 exceeds maximum_local_combinations=4; type=ResourceLimitError`.

Zero allowed: `True`.

Reproduce: from a working copy with the pinned wheel installed, use `python tools/block43/run_probes.py --worker R08 --output <fresh-existing-scratch-directory>`. The all-probe launcher creates fresh directories and verifies package parity. Receipt: `manual/verification/block43-appendix-k/resource_replay/R08/result.json`.

R09 / Coordinate dimension boundary

A matrix cap is tested before a coordinate producer allocates its dense subset table.

Columns: `4`.

Refusal: `message=subset zeta transform requires a 8 by 8 matrix, exceeding the coordinate dimension limit 4; type=ResourceLimitError`.

Rows: `4`.

Reproduce: from a working copy with the pinned wheel installed, use `python tools/block43/run_probes.py --worker R09 --output <fresh-existing-scratch-directory>`. The all-probe launcher creates fresh directories and verifies package parity. Receipt: `manual/verification/block43-appendix-k/resource_replay/R09/result.json`.

R10 / Coordinate entry-product boundary

Rows and columns can satisfy their separate caps while the entry-product cap still refuses the matrix.

At limit entries: `6`.

Refusal: `message=probe requires 9 exact entries, exceeding the coordinate entry limit 6; type=ResourceLimitError`.

Reproduce: from a working copy with the pinned wheel installed, use `python tools/block43/run_probes.py --worker R10 --output <fresh-existing-scratch-directory>`. The all-probe launcher creates fresh directories and verifies package parity. Receipt: `manual/verification/block43-appendix-k/resource_replay/R10/result.json`.

R11 / Coordinate degree and horizon

Both guards accept equality and reject one above their declared caps.

Degree: `message=probe degree 4 exceeds the coordinate degree limit 3; type=ResourceLimitError`.

Horizon: `message=probe horizon 5 exceeds the coordinate horizon limit 4; type=ResourceLimitError`.

Reproduce: from a working copy with the pinned wheel installed, use `python tools/block43/run_probes.py --worker R11 --output <fresh-existing-scratch-directory>`. The all-probe launcher creates fresh directories and verifies package parity. Receipt: `manual/verification/block43-appendix-k/resource_replay/R11/result.json`.

R12 / Coordinate matrix argument permissiveness

The low-level require_matrix method accepts positive floats and booleans; do not interpret it as a strict integer validator.

Boolean rows accepted: `True`.

Float rows accepted: `True`.

Negative: `message=matrix dimensions must be nonnegative; type=ValidationError`.

Reproduce: from a working copy with the pinned wheel installed, use `python tools/block43/run_probes.py --worker R12 --output <fresh-existing-scratch-directory>`. The all-probe launcher creates fresh directories and verifies package parity. Receipt: `manual/verification/block43-appendix-k/resource_replay/R12/result.json`.

R13 / Finite constructor consumes before degree guard

A finite counting iterator is exhausted before its over-degree construction is rejected. No unbounded iterator is run.

Consumed before refusal: `9`.

Degree limit: `3`.

Refusal: `message=degree 8 exceeds maximum_degree=3; type=ResourceLimitError`.

Reproduce: from a working copy with the pinned wheel installed, use `python tools/block43/run_probes.py --worker R13 --output <fresh-existing-scratch-directory>`. The all-probe launcher creates fresh directories and verifies package parity. Receipt: `manual/verification/block43-appendix-k/resource_replay/R13/result.json`.

R14 / Finite coefficient bit boundary

Fraction numerator and denominator bit lengths are capped on stored values, not on every arithmetic temporary.

Denominator: `message=coefficient bit length 5 exceeds maximum_coefficient_bits=4; type=ResourceLimitError`.

Numerator: `message=coefficient bit length 5 exceeds maximum_coefficient_bits=4; type=ResourceLimitError`.

Reproduce: from a working copy with the pinned wheel installed, use `python tools/block43/run_probes.py --worker R14 --output <fresh-existing-scratch-directory>`. The all-probe launcher creates fresh directories and verifies package parity. Receipt: `manual/verification/block43-appendix-k/resource_replay/R14/result.json`.

R15 / Stored generating byte cap is not generic codec enforcement

A generating budget survives serialization as data, while its tiny serialized-byte field does not automatically govern dumps_portable.

Actual envelope bytes: `3214`.

Declared bytes: `1`.

Roundtrip: `True`.

Reproduce: from a working copy with the pinned wheel installed, use `python tools/block43/run_probes.py --worker R15 --output <fresh-existing-scratch-directory>`. The all-probe launcher creates fresh directories and verifies package parity. Receipt: `manual/verification/block43-appendix-k/resource_replay/R15/result.json`.

R16 / Stored generating recursion cap is not generic codec depth

The field maximum_recursion_depth is retained; generic portable depth uses its own argument.

Default codec roundtrip: `True`.

Stored maximum recursion depth: `1`.

Reproduce: from a working copy with the pinned wheel installed, use `python tools/block43/run_probes.py --worker R16 --output <fresh-existing-scratch-directory>`. The all-probe launcher creates fresh directories and verifies package parity. Receipt: `manual/verification/block43-appendix-k/resource_replay/R16/result.json`.

R17 / Shared expression DAG count

Identity sharing lowers node count but repeated incoming edges still contribute to max_steps.

Refusal: `message=expression graph traversal exceeds max_steps; type=ResourceLimitError`.

Stats: `depth=2; exact_entries=1; maximum_coefficient_bits=1; maximum_degree=0; nodes=2; serialized_bytes=32; steps=4`.

Reproduce: from a working copy with the pinned wheel installed, use `python tools/block43/run_probes.py --worker R17 --output <fresh-existing-scratch-directory>`. The all-probe launcher creates fresh directories and verifies package parity. Receipt: `manual/verification/block43-appendix-k/resource_replay/R17/result.json`.

R18 / Expression node cap is checked after graph traversal

Instrumented reads of child nodes confirm that a small over-limit DAG is still traversed before the node-count refusal.

Declared max nodes: `1`.

Refusal: `message=expression node count exceeds max_nodes; type=ResourceLimitError`.

Unique nodes visited: `9`.

Instrumentation: custom finite Expression children property; no patch to guard logic.

Reproduce: from a working copy with the pinned wheel installed, use `python tools/block43/run_probes.py --worker R18 --output <fresh-existing-scratch-directory>`. The all-probe launcher creates fresh directories and verifies package parity. Receipt: `manual/verification/block43-appendix-k/resource_replay/R18/result.json`.

R19 / Expression depth, symbol and coefficient guards

Small valid inputs and one-over inputs exercise separate structural limits.

Bits: `message=expression coefficient size exceeds max_coefficient_bits; type=ResourceLimitError`.

Depth: `message=expression depth exceeds max_depth; type=ResourceLimitError`.

Symbol: `message=symbol name exceeds max_symbol_length; type=ResourceLimitError`.

Reproduce: from a working copy with the pinned wheel installed, use `python tools/block43/run_probes.py --worker R19 --output <fresh-existing-scratch-directory>`. The all-probe launcher creates fresh directories and verifies package parity. Receipt: `manual/verification/block43-appendix-k/resource_replay/R19/result.json`.

R20 / Expression JSON bytes checked before parsing

A patched parser sentinel is not invoked when the actual input JSON exceeds the byte cap; UTF-8 counting still precedes that check.

Actual bytes: `282`.

Parser called: `False`.

Refusal: `message=expression JSON exceeds max_serialized_bytes; type=ResourceLimitError`.

Instrumentation: parser sentinel only.

Reproduce: from a working copy with the pinned wheel installed, use `python tools/block43/run_probes.py --worker R20 --output <fresh-existing-scratch-directory>`. The all-probe launcher creates fresh directories and verifies package parity. Receipt: `manual/verification/block43-appendix-k/resource_replay/R20/result.json`.

R21 / Formal multiply triangular operation count

Wrapping the real structure-constant method counts every emitted convolution term on a small dense input.

Degree: `7`.

Formula: `36`.

Product terms: `36`.

Instrumentation: method wrapper delegates to original.

Reproduce: from a working copy with the pinned wheel installed, use `python tools/block43/run_probes.py --worker R21 --output <fresh-existing-scratch-directory>`. The all-probe launcher creates fresh directories and verifies package parity. Receipt: `manual/verification/block43-appendix-k/resource_replay/R21/result.json`.

R22 / Finite multiplication output bit refusal

Small permitted input coefficients can produce an over-budget result, rejected only when the result constructor checks its bits.

Input bits: `3`.

Mathematical result: `49`.

Refusal: `message=coefficient bit length 6 exceeds maximum_coefficient_bits=3; type=ResourceLimitError`.

Result bits: `6`.

Reproduce: from a working copy with the pinned wheel installed, use `python tools/block43/run_probes.py --worker R22 --output <fresh-existing-scratch-directory>`. The all-probe launcher creates fresh directories and verifies package parity. Receipt: `manual/verification/block43-appendix-k/resource_replay/R22/result.json`.

R23 / Packet certificate counts all pair products

The actual NumPy backend wrapper records p squared matrix products during the axiom certificate.

Matrix products: `4`.

Packets: `2`.

Instrumentation: backend method wrapper delegates to original.

Reproduce: from a working copy with the pinned wheel installed, use `python tools/block43/run_probes.py --worker R23 --output <fresh-existing-scratch-directory>`. The all-probe launcher creates fresh directories and verifies package parity. Receipt: `manual/verification/block43-appendix-k/resource_replay/R23/result.json`.

R24 / Plan address cap is post-enumeration

Wrapping the actual address generator records all eight returned addresses before a limit of one rejects the plan.

Addresses materialized: `8`.

Refusal: `message=packet-jet family has 8 addresses above max_addresses=1; type=ValidationError`.

Refusal limit: `1`.

Instrumentation: wrapper delegates to original address enumeration.

Reproduce: from a working copy with the pinned wheel installed, use `python tools/block43/run_probes.py --worker R24 --output <fresh-existing-scratch-directory>`. The all-probe launcher creates fresh directories and verifies package parity. Receipt: `manual/verification/block43-appendix-k/resource_replay/R24/result.json`.

R25 / Packet-jet count estimator and pruning

Source-derived counts match the finite grouped plan. Counts do not include a derivative callback complexity model.

Unpruned addresses: `8`.

Unpruned derivative calls: `4`.

Pruned active addresses: `2`.

Pruned derivative calls: `2`.

Reproduce: from a working copy with the pinned wheel installed, use `python tools/block43/run_probes.py --worker R25 --output <fresh-existing-scratch-directory>`. The all-probe launcher creates fresh directories and verifies package parity. Receipt: `manual/verification/block43-appendix-k/resource_replay/R25/result.json`.

R26 / Snapshot evaluation cap precedes evaluation

A tiny snapshot refuses below n to the r evaluations and passes exactly at the bound.

Evaluations: `4`.

Refusal: `message=packet-jet snapshot requires 4 evaluations, exceeding 3; type=ValidationError`.

Stored coefficients: `4`.

Reproduce: from a working copy with the pinned wheel installed, use `python tools/block43/run_probes.py --worker R26 --output <fresh-existing-scratch-directory>`. The all-probe launcher creates fresh directories and verifies package parity. Receipt: `manual/verification/block43-appendix-k/resource_replay/R26/result.json`.

R27 / Dense expansion count and dtype

The numerical snapshot dense route uses complex cells and guards element count, not bytes.

Dtype: `complex128`.

Itemsize: `16`.

Nbytes: `64`.

Refusal: `message=dense packet-jet expansion requires 4 elements, exceeding 3; type=ValidationError`.

Shape: `[1, 2, 2]`.

Reproduce: from a working copy with the pinned wheel installed, use `python tools/block43/run_probes.py --worker R27 --output <fresh-existing-scratch-directory>`. The all-probe launcher creates fresh directories and verifies package parity. Receipt: `manual/verification/block43-appendix-k/resource_replay/R27/result.json`.

R28 / Stream pause, resume and coefficient materialization cap

Per-call work budgets produce incomplete valid streams; selection caps count entries, not file bytes.

Chunks: `3`.

Coefficients: `4`.

Complete: `True`.

Initial evaluations: `1`.

Refusal: `message=coefficient selection requires 4 entries, exceeding 3; type=ValidationError`.

Reproduce: from a working copy with the pinned wheel installed, use `python tools/block43/run_probes.py --worker R28 --output <fresh-existing-scratch-directory>`. The all-probe launcher creates fresh directories and verifies package parity. Receipt: `manual/verification/block43-appendix-k/resource_replay/R28/result.json`.

R29 / Height-tree pre-count refusal

A small exact recurrence predicts five trees at height two for binary arity. A cap of four is refused, while a cap of five succeeds.

Count: `5`.

Refusal: `message=tree family has 5 members, above max_trees=4; lower the height or raise the explicit limit; type=ValidationError`.

Reproduce: from a working copy with the pinned wheel installed, use `python tools/block43/run_probes.py --worker R29 --output <fresh-existing-scratch-directory>`. The all-probe launcher creates fresh directories and verifies package parity. Receipt: `manual/verification/block43-appendix-k/resource_replay/R29/result.json`.

R30 / Leaf-tree refusal leaves a populated cache

The leaf-count API builds the cached family before testing its length. Small binary trees expose that late check without large allocation.

Cache entries after: `4`.

Cache entries before: `0`.

Cached family size: `5`.

Refusal: `message=tree family has 5 members, above max_trees=1; lower the leaf count or raise the explicit limit; type=ValidationError`.

Reproduce: from a working copy with the pinned wheel installed, use `python tools/block43/run_probes.py --worker R30 --output <fresh-existing-scratch-directory>`. The all-probe launcher creates fresh directories and verifies package parity. Receipt: `manual/verification/block43-appendix-k/resource_replay/R30/result.json`.

R31 / Recurrence term retains predecessors

A single cold request to term(30) builds every preceding term; this API is not logarithmic random access.

Cache entries: `31`.

Index: `30`.

Value: `832040`.

Reproduce: from a working copy with the pinned wheel installed, use `python tools/block43/run_probes.py --worker R31 --output <fresh-existing-scratch-directory>`. The all-probe launcher creates fresh directories and verifies package parity. Receipt: `manual/verification/block43-appendix-k/resource_replay/R31/result.json`.

R32 / Finite modular period search refusal

The explicit step budget can stop a finite search; its default state-space bound is not a memory promise.

Period: `3`.

Refusal: `message=period search exceeded max_steps before a state repeated; type=ValidationError`.

States checked: `3`.

Reproduce: from a working copy with the pinned wheel installed, use `python tools/block43/run_probes.py --worker R32 --output <fresh-existing-scratch-directory>`. The all-probe launcher creates fresh directories and verifies package parity. Receipt: `manual/verification/block43-appendix-k/resource_replay/R32/result.json`.

R33 / Quantized tree budget is error allowance

Larger B can enable a cheaper plan. B is not a memory resource limit.

Two error cost: `1.0`.

Used units: `2`.

Zero error cost: `4.0`.

Reproduce: from a working copy with the pinned wheel installed, use `python tools/block43/run_probes.py --worker R33 --output <fresh-existing-scratch-directory>`. The all-probe launcher creates fresh directories and verifies package parity. Receipt: `manual/verification/block43-appendix-k/resource_replay/R33/result.json`.

R34 / Pareto local combination limit

Two child frontiers create four candidate combinations; local max_plans=3 refuses before their product is materialized.

Combinations: `4`.

Frontier size: `4`.

Refusal: `message=Pareto enumeration would create 4 combinations, above max_plans; type=ValidationError`.

Reproduce: from a working copy with the pinned wheel installed, use `python tools/block43/run_probes.py --worker R34 --output <fresh-existing-scratch-directory>`. The all-probe launcher creates fresh directories and verifies package parity. Receipt: `manual/verification/block43-appendix-k/resource_replay/R34/result.json`.

R35 / Object-array buffer does not include exact integers

Two equally shaped object arrays have equal pointer-buffer bytes despite different integer payload sizes.

Big referent sum: `352`.

Pointer bytes: `64`.

Small referent sum: `224`.

Reproduce: from a working copy with the pinned wheel installed, use `python tools/block43/run_probes.py --worker R35 --output <fresh-existing-scratch-directory>`. The all-probe launcher creates fresh directories and verifies package parity. Receipt: `manual/verification/block43-appendix-k/resource_replay/R35/result.json`.

R36 / Finite observed scaling samples

Warm local timings and traced allocation peaks are observations, not asymptotic proofs, speedups or peak RSS.

Workload records: `9`.

Repetitions each: `3`.

Warmup each: `1`.

Reproduce: from a working copy with the pinned wheel installed, use `python tools/block43/run_probes.py --worker R36 --output <fresh-existing-scratch-directory>`. The all-probe launcher creates fresh directories and verifies package parity. Receipt: `manual/verification/block43-appendix-k/resource_replay/R36/result.json`.

K.8 Complete lexical parameter lookup

The index records **196 parameter declaration sites** matching `^(?:max_\|maximum_)\|(?:budget\|limit\|timeout)$\|^chunk_size$`. It scans all 300 package Python files, including verification helpers. Defaults are source expressions, not automatically resolved promises. Repeated `budget` arguments are distinct call surfaces. The list deliberately retains audit horizons, approximation denominator bounds, quantized error allowances and formatting limits; these are named limit candidates, not all memory caps. A size-controlling argument named only `degree`, `order`, `steps` or `shape` lies outside this syntactic census unless described in a reviewed profile.

algorithms/compiler.py

Callable / parameter	Declared default	Source line
`compile_tree_budget / budget`	`(required)`	140
`pareto_frontier / max_plans`	`100000`	252

algorithms/krylov.py

Callable / parameter	Declared default	Source line
`conjugate_gradient / max_iterations`	`None`	109

algorithms/precision.py

Callable / parameter	Declared default	Source line
`assign_discrete_precisions / budget`	`(required)`	167

analytic/sections.py

Callable / parameter	Declared default	Source line
`_section_sum / max_terms`	`(required)`	18
`transfer_section / max_terms`	`512`	64
`signed_section / max_terms`	`512`	96

analytic/zeros.py

Callable / parameter	Declared default	Source line
`_newton_positive_zero / max_iterations`	`(required)`	60
`positive_zero_string / max_iterations`	`64`	95

arithmetic/euler.py

Callable / parameter	Declared default	Source line
`EulerLocalCoordinates.certificate / maximum_index`	`20`	287

arithmetic/mahler.py

Callable / parameter	Declared default	Source line
`UnitModeMahler.certificate / maximum_value`	`(required)`	342

arithmetic/recurrence.py

Callable / parameter	Declared default	Source line
`LinearRecurrence.period_mod / max_steps`	`None`	230

coordinates/basis.py

Callable / parameter	Declared default	Source line
`FiniteBasis.connection_to / budget`	`DEFAULT_COORDINATE_BUDGET`	146

coordinates/calculus.py

Callable / parameter	Declared default	Source line
`polynomial_operator / budget`	`DEFAULT_COORDINATE_BUDGET`	105
`derivative_coordinate_operator / budget`	`DEFAULT_COORDINATE_BUDGET`	138
`integral_coordinate_operator / budget`	`DEFAULT_COORDINATE_BUDGET`	158
`multiplication_coordinate_operator / budget`	`DEFAULT_COORDINATE_BUDGET`	178
`affine_substitution_coordinate_operator / budget`	`DEFAULT_COORDINATE_BUDGET`	203
`finite_difference_coordinate_operator / budget`	`DEFAULT_COORDINATE_BUDGET`	225
`transport_polynomial_operator / budget`	`DEFAULT_COORDINATE_BUDGET`	252

coordinates/classical.py

Callable / parameter	Declared default	Source line
`evaluation_transform / budget`	`DEFAULT_COORDINATE_BUDGET`	443
`interpolation_transform / budget`	`DEFAULT_COORDINATE_BUDGET`	472
`operator_matrix_between_families / budget`	`DEFAULT_COORDINATE_BUDGET`	483
`affine_substitution_transform / budget`	`DEFAULT_COORDINATE_BUDGET`	512

coordinates/combinatorial.py

Callable / parameter	Declared default	Source line
`monomial_to_falling_factorial_transform / budget`	`DEFAULT_COORDINATE_BUDGET`	92
`falling_factorial_to_monomial_transform / budget`	`DEFAULT_COORDINATE_BUDGET`	110
`monomial_to_rising_factorial_transform / budget`	`DEFAULT_COORDINATE_BUDGET`	128
`rising_factorial_to_monomial_transform / budget`	`DEFAULT_COORDINATE_BUDGET`	150
`rising_to_falling_factorial_transform / budget`	`DEFAULT_COORDINATE_BUDGET`	168
`binomial_transform / budget`	`DEFAULT_COORDINATE_BUDGET`	186
`forward_difference_transform / budget`	`DEFAULT_COORDINATE_BUDGET`	215
`newton_evaluation_transform / budget`	`DEFAULT_COORDINATE_BUDGET`	242
`divisor_zeta_transform / budget`	`DEFAULT_COORDINATE_BUDGET`	276
`subset_zeta_transform / budget`	`DEFAULT_COORDINATE_BUDGET`	315

coordinates/formal.py

Callable / parameter	Declared default	Source line
`FormalCoordinateChange.substitution_transform / budget`	`DEFAULT_COORDINATE_BUDGET`	195

coordinates/normalized.py

Callable / parameter	Declared default	Source line
`normalization_change_transform / budget`	`DEFAULT_COORDINATE_BUDGET`	22
`convert_normalized_coefficients / budget`	`DEFAULT_COORDINATE_BUDGET`	63

coordinates/polynomials.py

Callable / parameter	Declared default	Source line
`connection_transform / budget`	`DEFAULT_COORDINATE_BUDGET`	190
`euler_to_fibonacci_transform / budget`	`DEFAULT_COORDINATE_BUDGET`	257

coordinates/realization.py

Callable / parameter	Declared default	Source line
`observability_matrix / budget`	`DEFAULT_COORDINATE_BUDGET`	169
`recover_initial_state / budget`	`DEFAULT_COORDINATE_BUDGET`	210

coordinates/recurrence.py

Callable / parameter	Declared default	Source line
`recurrence_analysis_transform / budget`	`DEFAULT_COORDINATE_BUDGET`	72
`recurrence_synthesis_transform / budget`	`DEFAULT_COORDINATE_BUDGET`	110
`coordinate_change_between_recurrences / budget`	`DEFAULT_COORDINATE_BUDGET`	251

core/grammar.py

Callable / parameter	Declared default	Source line
`OperatorGrammar.paths / max_length`	`1`	180

core/realization.py

Callable / parameter	Declared default	Source line
`OperatorRealization.path_operators / max_length`	`1`	120
`OperatorRealization.loop_operators / max_length`	`(required)`	138

core/seeds.py

Callable / parameter	Declared default	Source line
`SeedSystem.hull_vectors / max_rounds`	`None`	97
`SeedSystem.hull_basis / max_rounds`	`None`	132
`SeedSystem.hull_dimension / max_rounds`	`None`	138
`SeedSystem.orbit_layers / max_rounds`	`None`	163
`SeedSystem.minimum_standard_basis_seeds / max_dimension`	`12`	231

deformation/forests.py

Callable / parameter	Declared default	Source line
`FiniteLiftForest.branches / limit`	`100000`	151
`FiniteLiftForest.report / branch_limit`	`100000`	172

deformation/visibility.py

Callable / parameter	Declared default	Source line
`VisibilityTree.refinement_chain / max_depth`	`(required)`	100

dynamics/_linalg.py

Callable / parameter	Declared default	Source line
`matrix_exponential / max_terms`	`256`	24

export/persistence.py

Callable / parameter	Declared default	Source line
`_encode / max_depth`	`(required)`	520
`_decode / max_depth`	`(required)`	612
`to_portable_data / max_depth`	`128`	752
`from_portable_data / max_depth`	`128`	759
`portable_digest / max_depth`	`128`	766
`dumps_portable / max_depth`	`128`	773
`loads_portable / max_depth`	`128`	794
`save_portable / max_depth`	`128`	827
`load_portable / max_depth`	`128`	860

export/serialization.py

Callable / parameter	Declared default	Source line
`to_serializable / max_depth`	`64`	53
`from_serializable / max_depth`	`64`	225

expression/engine.py

Callable / parameter	Declared default	Source line
`_replay_graph_invariants / budget`	`(required)`	234
`check_expression / budget`	`None`	447
`_normal_form / budget`	`None`	528
`_validate_data_budget / budget`	`(required)`	562

Callable / parameter	Declared default	Source line
`_normalize_node / budget`	`(required)`	593
`_normalize_node_uncached / budget`	`(required)`	621
`normalize_expression / budget`	`None`	751
`equivalent / budget`	`None`	806
`_rebuild_with_simplified_children / budget`	`(required)`	1024
`_expression_digest / budget`	`(required)`	1064
`_simplify_node / budget`	`(required)`	1070
`simplify_expression / budget`	`None`	1142
`_PlanBuilder.__init__ / budget`	`(required)`	1387
`execute_plan / budget`	`None`	1548
`compile_expression / budget`	`None`	1630
`evaluate_expression / budget`	`None`	1700

expression/portable.py

Callable / parameter	Declared default	Source line
`_preflight / budget`	`(required)`	59
`_validate_json_tree / budget`	`(required)`	146
`_check_encoded_size / budget`	`(required)`	183
`type_to_dict / budget`	`None`	351
`type_from_dict / budget`	`None`	367
`_expression_to_payload / budget`	`(required)`	401
`_semantic_payload / budget`	`(required)`	839
`expression_digest / budget`	`None`	853
`expression_to_dict / budget`	`None`	871
`expression_from_dict / budget`	`None`	916
`expression_to_json / budget`	`None`	960
`expression_from_json / budget`	`None`	972

generating/adapters.py

Callable / parameter	Declared default	Source line
`fir_taps_to_series / max_denominator`	`None`	111

generating/bell.py

Callable / parameter	Declared default	Source line
`generalized_bell_digit_reduction / budget`	`DEFAULT_GENERATING_BUDGET`	206
`weighted_bell_digit_reduction / budget`	`DEFAULT_GENERATING_BUDGET`	248
`BellResidueAutomaton.compile / budget`	`DEFAULT_GENERATING_BUDGET`	335

generating/cartier.py

Callable / parameter	Declared default	Source line
`_reachable_automaton / max_states`	`(required)`	214
`compile_rational_mask_automaton / budget`	`DEFAULT_GENERATING_BUDGET`	336
`compile_rational_mask_automaton / max_states`	`100000`	336
`finite_mask_carry_matrices / budget`	`DEFAULT_GENERATING_BUDGET`	358
`certify_cartier_carry_equivalence / budget`	`DEFAULT_GENERATING_BUDGET`	405
`compile_cartier_linear_representation / budget`	`DEFAULT_GENERATING_BUDGET`	484

generating/closed_forms.py

Callable / parameter	Declared default	Source line
`guess_linear_recurrence / max_order`	`12`	141
`guess_hypergeometric_ratio / max_denominator_degree`	`4`	208
`guess_hypergeometric_ratio / max_numerator_degree`	`4`	208
`guess_p_recursive / max_degree`	`5`	265
`guess_p_recursive / max_order`	`4`	265
`guess_algebraic_gf / max_x_degree`	`4`	329
`guess_algebraic_gf / max_y_degree`	`4`	329
`guess_quasipolynomial / max_degree`	`6`	397
`guess_quasipolynomial / max_period`	`12`	397

generating/congruences.py

Callable / parameter	Declared default	Source line
`difference_valuation_profile / max_order`	`(required)`	126
`minimal_prefix_period / preperiod_limit`	`0`	186
`search_decimation_relation / max_order`	`8`	291
`audit_generalized_lucas / high_limit`	`12`	334
`audit_q_lucas / high_limit`	`10`	442

generating/digit_algebra.py

Callable / parameter	Declared default	Source line
`audit_factorial_power_digit_algebra / max_digits`	`3`	180

generating/finite_series.py

Callable / parameter	Declared default	Source line
`FiniteNormalizedSeries.__init__ / budget`	`DEFAULT_GENERATING_BUDGET`	102
`FiniteNormalizedSeries.from_ordinary_coefficients / budget`	`DEFAULT_GENERATING_BUDGET`	215

generating/masks.py

Callable / parameter	Declared default	Source line
`mahler_mask_product / budget`	`DEFAULT_GENERATING_BUDGET`	93
`mahler_mask_inverse / budget`	`DEFAULT_GENERATING_BUDGET`	124
`certify_mahler_functional_equation / budget`	`DEFAULT_GENERATING_BUDGET`	152
`FiniteMaskCarryPlan.sequence / budget`	`DEFAULT_GENERATING_BUDGET`	224
`FiniteMaskCarryPlan.certificate / budget`	`DEFAULT_GENERATING_BUDGET`	235
`compile_finite_mask_carry_plan / budget`	`DEFAULT_GENERATING_BUDGET`	273
`DigitCauchyProductPlan.sequence / budget`	`DEFAULT_GENERATING_BUDGET`	334
`compile_digit_cauchy_product / budget`	`DEFAULT_GENERATING_BUDGET`	344
`KummerLayerDistribution.support_count / maximum_borrows`	`(required)`	401
`factorial_power_support_indices / budget`	`DEFAULT_GENERATING_BUDGET`	495
`factorial_power_convolution_layers / budget`	`DEFAULT_GENERATING_BUDGET`	532
`frobenius_defect_mask / budget`	`DEFAULT_GENERATING_BUDGET`	578
`certify_rational_mask_algebraic_relation / budget`	`DEFAULT_GENERATING_BUDGET`	790
`solve_polynomial_coboundary / budget`	`DEFAULT_GENERATING_BUDGET`	926

integration/identities.py

Callable / parameter	Declared default	Source line
`all_fubini_orders / max_dimensions`	`7`	233

integration/measures.py

Callable / parameter	Declared default	Source line
`ExactAffineMap.pullback / max_terms`	`100000`	275

integration/polynomial.py

Callable / parameter	Declared default	Source line
`ExactPolynomial.multiply / max_terms`	`100000`	165
`ExactPolynomial.power / max_terms`	`100000`	192
`ExactPolynomial.pullback_affine / max_terms`	`100000`	264

integration/rules.py

Callable / parameter	Declared default	Source line
`QuadratureRule.degree_of_exactness / max_degree`	`(required)`	352

langlands/finite_image/groups.py

Callable / parameter	Declared default	Source line
`FiniteGroup.__init__ / associativity_limit`	`128`	19
`symmetric_group / max_order`	`40320`	259

langlands/local/heisenberg.py

Callable / parameter	Declared default	Source line
`HeisenbergModel.__init__ / max_dimension`	`4096`	39

nonlinear/autodiff.py

Callable / parameter	Declared default	Source line
`_short_probe_detail / limit`	`600`	119
`autodiff_provider / max_order`	`4`	682

nonlinear/composition.py

Callable / parameter	Declared default	Source line
`packet_faa_di_bruno / max_terms`	`1000000`	240

nonlinear/correction.py

Callable / parameter	Declared default	Source line
`RightInverseCorrectionProblem.solve / max_iterations`	`10000`	238
`LyapunovSchmidtReduction.solve_range / max_iterations`	`50`	405

nonlinear/elimination.py

Callable / parameter	Declared default	Source line
`NonlinearSchurElimination.solve_hidden / max_iterations`	`50`	360
`NonlinearSchurElimination.reduced_value / max_iterations`	`50`	461

nonlinear/execution.py

Callable / parameter	Declared default	Source line
`compile_packet_jet_plan / max_addresses`	`1000000`	124
`execute_packet_jets / max_addresses`	`1000000`	354

nonlinear/filtered.py

Callable / parameter	Declared default	Source line
`FilteredCorrectionProblem.solve / enumerate_limit`	`100000`	374

nonlinear/graded.py

Callable / parameter	Declared default	Source line
`GradedPacketOperation.grading_certificate / max_evaluations`	`100000`	257

nonlinear/jets.py

Callable / parameter	Declared default	Source line
`packet_jet_support / max_evaluations`	`100000`	352

nonlinear/lifting.py

Callable / parameter	Declared default	Source line
`ModularLinearSolution.enumerate / limit`	`100000`	64
`LinearLiftProblem.step / enumerate_limit`	`100000`	197
`PolynomialLiftProblem.step / enumerate_limit`	`100000`	259
`PolynomialLiftProblem.step / linearization_limit`	`100000`	259
`build_lift_tree / max_depth`	`(required)`	430
`build_lift_tree / max_nodes`	`100000`	430

nonlinear/profiling.py

Callable / parameter	Declared default	Source line
`estimate_packet_jet_complexity / max_addresses`	`1000000`	193
`profile_packet_jet_runtime / max_addresses`	`1000000`	321
`profile_packet_jet_runtime / max_snapshot_evaluations`	`100000`	321

nonlinear/snapshots.py

Callable / parameter	Declared default	Source line
`PacketJetSnapshot.to_dense / max_elements`	`10000000`	205
`snapshot_packet_jet / max_evaluations`	`100000`	280

nonlinear/sparse.py

Callable / parameter	Declared default	Source line
`SparsePolynomialDerivativeProvider.storage_certificate / max_order`	`None`	575

nonlinear/streaming.py

Callable / parameter	Declared default	Source line
`_manifest_payload / chunk_size`	`(required)`	676
`_manifest_payload / maximum_omitted_basis_norm`	`(required)`	676
`_prepare_staging / chunk_size`	`(required)`	856
`capture_packet_jet_stream / chunk_size`	`1024`	1010
`capture_packet_jet_stream / evaluation_budget`	`None`	1010
`capture_packet_jet_stream / max_evaluations`	`10000000`	1010

Callable / parameter	Declared default	Source line
`save_packet_jet_snapshot_stream / chunk_size`	`1024`	1191
`save_packet_jet_snapshot_stream / coefficient_budget`	`None`	1191
`PacketJetSnapshotStreamReader.select / max_coefficients`	`100000`	1416
`PacketJetSnapshotStreamReader.load_snapshot / max_coefficients`	`1000000`	1544
`load_packet_jet_snapshot_stream / max_coefficients`	`1000000`	1637

nonlinear/trees.py

Callable / parameter	Declared default	Source line
`ordered_tree_count / max_height`	`(required)`	177
`_ordered_rooted_trees_cached / max_height`	`(required)`	195
`ordered_rooted_trees / max_height`	`(required)`	208
`ordered_rooted_trees / max_trees`	`100000`	208
`ordered_trees_with_leaves / max_trees`	`100000`	259
`PolynomialFixedPointProblem.tree_expansion / max_trees`	`100000`	541

qualification.py

Callable / parameter	Declared default	Source line
`gpu_diagnostics / timeout`	`120.0`	262

quantum/circuits.py

Callable / parameter	Declared default	Source line
`CircuitPacketExpansion.__init__ / max_paths`	`1000000`	50

refinement/smith.py

Callable / parameter	Declared default	Source line
`SmithCongruenceProblem.enumerate_solutions / limit`	`100000`	272

refinement/weighted.py

Callable / parameter	Declared default	Source line
`WeightedTransferGraph.paths_below_weight / max_paths`	`1000000`	155
`WeightedTransferGraph.truncated_inverse / max_paths`	`1000000`	238
`WeightedTransferGraph.exact_inverse_if_acyclic / max_paths`	`1000000`	315

release/gpu.py

Callable / parameter	Declared default	Source line
`_worker_result / timeout`	`(required)`	62
`qualify_gpu / timeout`	`180.0`	122

sequences/compiler.py

Callable / parameter	Declared default	Source line
`detect_linear_recurrence / max_order`	`None`	68
`compile_sequence / max_order`	`8`	200

The same source index retains 148 conditional-statement candidates mentioning these names, 58 named budget-method calls and 63 explicit `ResourceLimitError` raise sites. Exact fragments and coordinates are retained. Condition mentions are not control-flow proofs; variable names can be aliased or interpreted differently. A limit can raise `ValidationError`, `ValueError` or `RuntimeError` instead, or return an incomplete/negative record. Appendix I remains the exception reference. Source fragments are matched at line and column granularity, avoiding conflation of statements on one physical line.

K.9 Retained caches and cumulative memory

Source callable	Decorator
`generating/basis.py:WeightedMonomialBasis.gamma`	`lru_cache(maxsize=None)`
`generating/basis.py:q_factorial_basis.gamma`	`lru_cache(maxsize=None)`
`generating/basis.py:fibonacci_factorial_basis.fibonacci`	`lru_cache(maxsize=None)`
`generating/basis.py:fibonacci_factorial_basis.gamma`	`lru_cache(maxsize=None)`
`generating/basis.py:pochhammer_basis.gamma`	`lru_cache(maxsize=None)`
`nonlinear/autodiff.py:_probe_ad_framework`	`lru_cache(maxsize=2)`
`nonlinear/composition.py:set_partitions`	`lru_cache(maxsize=None)`
`nonlinear/sparse.py:_unique_permutations`	`lru_cache(maxsize=None)`
`nonlinear/trees.py:_ordered_rooted_trees_cached`	`lru_cache(maxsize=None)`
`nonlinear/trees.py:_positive_compositions`	`lru_cache(maxsize=None)`
`nonlinear/trees.py:_trees_with_leaves_cached`	`lru_cache(maxsize=None)`

Ten of the eleven explicit cache decorators use `maxsize=None`; the autodiff discovery probe uses two entries. No eviction bound is implied by the former. Some basis caches are attached to methods or closures and can retain arguments or objects through cache references. Tree/composition caches can retain combinatorial families even after a local result is discarded or a post-generation limit rejects it. Other ordinary mutable caches, such as the recurrence term list, are not decorator sites and are documented separately. Cache decorators are not an exhaustive inventory of every possible retained object.

Do not clear global/private caches blindly during another computation. Reproducible short-lived worker processes provide a clear lifetime for exploratory allocations. The leaf-tree probe clears only its dedicated worker's small cache before instrumentation. A shared long-lived production process needs an application-owned lifecycle policy and measurement; this manual does not introduce a new cache-management API.

K.10 A finite preflight worksheet

Before an expensive call, name its exact object, output cardinality, scalar type, possible temporaries, and limit enforcement phase. Calculate dimensions using Python integers before constructing arrays. For dense packet tensors compare `m*n**r` with the selected element limit and then multiply by the actual numerical dtype width. For packet plans compare `q*p**r` before plan compilation. For Boolean-lattice coordinates compare both `2**rank` and `4**rank` to the coordinate budget. For exact arrays reserve for coefficients and their intermediate growth, not just pointer buffers.

```
from math import prod

def dense_payload_bytes(shape, itemsize):
    shape = tuple(shape)
    if not isinstance(itemsize, int) or isinstance(itemsize, bool) or itemsize < 1:
        raise ValueError("itemsize must be a positive integer")
    if any(not isinstance(d, int) or isinstance(d, bool) or d < 0 for d in shape):
        raise ValueError("shape must contain nonnegative integers")
    return prod(shape) * itemsize

# A count only, not an allocation or a complete peak-memory estimate.
source_dimension, target_dimension, order = 32, 32, 4
shape = (target_dimension,) + (source_dimension,) * order
print(dense_payload_bytes(shape, 16))  # 536870912
```

When streaming, choose chunk size and per-call work budget separately. Verify the explicit completion flag before calling complete-snapshot operations. A coefficient cap is not a cap on input byte size, manifest size, individual exact values or decompression. Generic portable loaders parse the full envelope before their decoder depth checks; they do not inherit an embedded GeneratingBudget as a security policy. Consult Appendix G before reading external files.

A timeout is an observation/control on elapsed subprocess work, not a mathematical resource certificate. It does not retroactively prevent allocation, authenticate input, or necessarily terminate every external descendant. None of the recorded fixtures deliberately exhausts memory or proves behavior under memory exhaustion. Production RSS, allocator, filesystem and device limits must be configured outside these local mathematical budget dataclasses.

K.11 Reproduction, evidence, and completion boundary

The library source and both distributions remain exactly R2. The source suite passed 1,203 tests; the new source/-census/budget/reference suite passed 1,021 tests. All 36 active resource probe expectations passed. Those classes overlap and are not additive theorem counts. The 26 cost models are reviewed source-based derivations, not new Lean proofs. The separately installed wheel contains the same 366 package files; the full 901-file library payload is unchanged.

```
python tools/block43/inventory.py
python tools/block43/profiles.py
python tools/block43/run_probes.py --output <fresh-replay-directory>
```

```
python tools/block43/run_reference_tests.py
python tools/block43/run_source_tests.py
python tools/block43/generate_manual.py
python tools/block43/build_pdfs.py
python VERIFY_BLOCK43_PACKAGE.py
```

Regenerate only in a working copy. The default replay path is the recorded resource-evidence directory; move or archive old evidence first rather than overwriting its historical receipt. Tests and document generation require the active default replay to be present. The tools make no dependency downloads and reject source-checkout leakage for installed reference tests. Dependencies were inherited from the existing host environment, not installed into a fully dependency-isolated environment. Rebuilding or revalidating changes evidence and invalidates the outer archive hashes until a new reviewed delivery is made.

Reference boundaries. This version-bound resource reference supplies the stated lexical census, reviewed cost profiles and finite probes. It is not a universal resource-safety certificate, an exhaustive input or worst-case benchmark, a source-code repair or an end-to-end hardware qualification. The recorded audit did not qualify Windows/PowerShell, other Python versions, CUDA, strict MkDocs, fresh dependency-isolated installation or final publication. Portable-reader and certificate-validation limitations are documented separately. Appendix O identifies the final-source qualification records.

Appendix L covers migration and deprecation. The resource observations retain their original source, runtime and input bindings.

Appendix L. Migration and deprecation tables

This appendix is the migration reference for **Komorebi Math 1.0.0b4, Repair R2, wheel build 2**. It distinguishes a retained spelling, a delegating wrapper, an explicitly deprecated callable, a manifest compatibility result, a portable-codec migration and an executed package replacement. None of those categories by itself proves mathematical equivalence or qualifies every platform.

Reference scope. All 300 Python package source files are hashed. The index compares all 30 retained API-manifest files over each manifest's own namespace tuple and scans all 1,526 current export records for deprecation metadata. It records 22 direct function/class assignment aliases, five explicitly documented wrapper aliases, three literal selector-alias maps and 131 stable export identities in two versioned facades. The 1,526 count includes repeated objects exported at different routes. The alias census deliberately excludes coincidental constant equality and arbitrary caller-defined aliases.

Execution scope. Five stored predecessor wheels were genuinely installed one at a time in disposable environments. Each wrote two portable records, then was replaced by the pinned R2 wheel. Both records loaded unchanged, the complete fifteen-check upgrade contract passed and all ten stable semantic cases executed for each path. Thirty additional migration/deprecation probe expectations and 572 reference tests passed. Dependencies were inherited, not freshly resolved in a clean environment.

Identify what is being migrated. An import spelling, a function call, an installed package and a saved object require different checks. Use the route below before making a replacement.

Change you are considering	Read	What to compare
Adopt a clearer name for an existing object	L.6 and L.7	Object identity versus wrapper behavior, including keyword names
Move an import to a stable namespace	L.2 and Appendix A	Whether that exact export belongs to the stable tier
Upgrade the installed package	L.3, L.11 and L.12	Actual distribution, environment, application results and retained rollback inputs
Load an older portable object	L.10 and Appendix G	Codec tag, adjacent migration path and reconstructed value

Make one kind of change at a time when diagnosing compatibility. Keep a small application example with its expected values, shapes and certificate interpretation, and run it in a fresh process after replacing the package. A manifest comparison helps locate interface differences; the application example checks the particular behavior you depend on. Keep the original saved data until the migrated copy has passed the checks appropriate to that object.

L.1 Status vocabulary and admission rules

Record class	Meaning in this release	What it does not establish
Identity alias	Two exported Python names refer to the same object.	A removal date, deprecation warning or alternative implementation.
Delegating wrapper	A function/property forwards to another implementation.	Identical Python keyword names or object identity.
Explicit deprecation	A callable carries `__komorebi_deprecation__` metadata and a warning wrapper.	Automatic removal, a validated version ordering or visible warnings under every filter.

Record class	Meaning in this release	What it does not establish
Structural comparison	Selected names, broad kinds and optionally signature text match policy.	Equivalent implementations or all application behavior.
Exact stable freeze	The selected upgrade path also has no added exports or unapproved signature drift.	Future indefinite support or native-platform qualification.
Portable migration	An adjacent versioned decoder-data transformation exists.	Pickle compatibility, downgrade capability, proof checking or malicious-input hardening.
Executed replacement	A predecessor install was actually replaced and finite records were replayed.	All old scripts, all dependency versions or every application dataset.

No declared export in the reference baseline has a `__komorebi_deprecation__` record. No @deprecated application appears in the scanned package source. The one direct warn_deprecated call is inside the generic decorator wrapper itself. The utility exists, but this snapshot does not contain a removal schedule for a concrete current export. The zero-metadata census records this baseline; it is not a promise about future releases.

L.2 Select the API tier before migrating

Tier	Current export records	Pinned policy
Stable	131	Frozen signatures, explicit semantic fixtures and persistence/upgrade contracts.
Beta	83	Curated additive surface with strict signature checks.
Research	1312	Broad research surface with an additive compatibility checker; request strict signatures explicitly.

Moving a name from a broad namespace to stable is appropriate only when the name is actually exported there. A broad research name is not promoted by changing an import string. Keep experimental code explicitly tied to its package snapshot and verify its inputs and outputs. The current stable surface is available through three import styles:

```
from komorebi_math import stable
from komorebi_math import v1
from komorebi_math.api import v1 as api_v1

assert set(stable.__all__) == set(v1.__all__) == set(api_v1.__all__)
assert all(getattr(stable, name) is getattr(v1, name)
           is getattr(api_v1, name) for name in stable.__all__)
```

The identity assertion is about exported values, not module object identity. Choose one style consistently. A successful reflection check does not authorize changing persisted Python implementation paths by hand.

L.3 Predecessor versions and exact stable deltas

The shipped contract targets 1.0.0b4 and names **0.15.0a2, 0.16.0a2, 1.0.0b1, 1.0.0b2 and 1.0.0b3**. The old UPGRADE_CONTRACT.md prose lists only the first two and describes a 1.0.0b1 target. This appendix supersedes that outdated coverage statement without rewriting historical library documentation.

Predecessor	Stable exports	Added	Signatures changed	Contract rule
0.15.0a2	72	59	1	Allowed named migration
0.16.0a2	131	0	0	Exact unchanged freeze
1.0.0b1	131	0	0	Exact unchanged freeze
1.0.0b2	131	0	0	Exact unchanged freeze
1.0.0b3	131	0	0	Exact unchanged freeze

There are no removed exports or broad kind changes in these five stable comparisons. The 0.15.0a2 path adds 59 names and changes create_stable_api_manifest from a baked-in package-version default to package_version=None. Its single signature change is explicitly allowed by the upgrade checker. The other four stable baselines are exactly unchanged at 131 names. A raw strict comparator will still reject the 0.15 signature difference; that is expected and distinct from the explicit upgrade allowance.

Stable additions relative to 0.15.0a2. These are additions, not replacements for removed names. Appendix A gives their full API records.

Added stable export	Added stable export
`AnalysisSynthesisBlocks`	`AnalysisSynthesisSystem`
`Arrow`	`CoordinateAtlas`
`CoordinateTransform`	`DEFAULT_PORTABLE_CODEC_CONTRACT`
`DEFAULT_STABLE_SEMANTICS_FIXTURE`	`DocumentMetadata`
`Equation`	`ExactCoordinateTransform`
`FiniteBasis`	`FormalCoordinateChange`
`FormalSeries`	`MathDocument`
`NumericCoordinateTransform`	`OperatorGrammar`
`OperatorRealization`	`OperatorRelation`
`OperatorWord`	`PORTABLE_CODEC_CONTRACT_SCHEMA`
`PORTABLE_CODEC_CONTRACT_SCHEMA_VERSION`	`PacketOperator`
`Paragraph`	`PortableCodecCompatibilityReport`
`PortableCodecContract`	`PortableCodecContractEntry`
`RelationResidual`	`STABLE_SEMANTICS_SCHEMA`
`STABLE_SEMANTICS_SCHEMA_VERSION`	`SeedSystem`
`Settings`	`StableContractReport`
`StableSemanticCaseResult`	`StableSemanticReport`
`TransferGraph`	`TransferPath`
`basis_from_matrix`	`block_from_paths`
`check_portable_codec_contract`	`check_stable_contract`
`compare_portable_codec_contract`	`configure`
`connection_transform`	`create_portable_codec_contract`
`factorial_power_basis`	`formal_coordinate_from_iterable`
`forward_difference_transform`	`get_polynomial_family`
`get_settings`	`load_portable_codec_contract`
`load_stable_semantics_fixture`	`ordinary_generating_basis`
`path_contributions`	`render_latex`
`render_markdown`	`run_stable_semantics_fixture`
`save_portable_codec_contract`	`settings_context`
`standard_basis`	

L.4 All retained API-manifest comparisons

The table covers every retained API-manifest JSON file, including unversioned current copies that duplicate versioned snapshots. The electronic index identifies exact filenames and SHA-256 digests; these thirty files do not represent thirty distinct releases. A historical research manifest may cover fewer namespaces than the current research manifest. Each comparison reflects the recorded package over exactly the old tuple and never mixes unmatched scopes.

ID	Version	Tier	Old / current	Added / removed / signature	Strict
A01	0.10.0a1	research	464 / 723	259 / 0 / 2	no
A02	0.11.0a1	research	528 / 723	195 / 0 / 3	no
A03	0.12.0a1	research	699 / 894	195 / 0 / 3	no
A04	0.13.0a1	research	901 / 1075	174 / 0 / 3	no
A05	0.14.0a2	research	1175 / 1312	137 / 0 / 8	no
A06	0.15.0a2	research	1175 / 1312	137 / 0 / 8	no
A07	0.16.0a2	research	1179 / 1312	133 / 0 / 6	no
A08	1.0.0b4	research	1312 / 1312	0 / 0 / 0	yes
A09	1.0.0b1	research	1179 / 1312	133 / 0 / 6	no
A10	1.0.0b2	research	1191 / 1312	121 / 0 / 0	yes
A11	1.0.0b3	research	1229 / 1312	83 / 0 / 0	yes
A12	1.0.0b4	research	1312 / 1312	0 / 0 / 0	yes
A13	0.11.0a1	beta	83 / 83	0 / 0 / 1	no
A14	0.12.0a1	beta	83 / 83	0 / 0 / 1	no
A15	0.13.0a1	beta	83 / 83	0 / 0 / 1	no
A16	0.14.0a2	beta	83 / 83	0 / 0 / 1	no
A17	0.15.0a2	beta	83 / 83	0 / 0 / 1	no
A18	0.16.0a2	beta	83 / 83	0 / 0 / 0	yes
A19	1.0.0b4	beta	83 / 83	0 / 0 / 0	yes
A20	1.0.0b1	beta	83 / 83	0 / 0 / 0	yes
A21	1.0.0b2	beta	83 / 83	0 / 0 / 0	yes
A22	1.0.0b3	beta	83 / 83	0 / 0 / 0	yes
A23	1.0.0b4	beta	83 / 83	0 / 0 / 0	yes
A24	0.15.0a2	stable	72 / 131	59 / 0 / 1	no
A25	0.16.0a2	stable	131 / 131	0 / 0 / 0	yes
A26	1.0.0b4	stable	131 / 131	0 / 0 / 0	yes
A27	1.0.0b1	stable	131 / 131	0 / 0 / 0	yes
A28	1.0.0b2	stable	131 / 131	0 / 0 / 0	yes
A29	1.0.0b3	stable	131 / 131	0 / 0 / 0	yes
A30	1.0.0b4	stable	131 / 131	0 / 0 / 0	yes

All thirty comparisons have zero removals and zero broad kind changes. Strict failures record real signature-text differences, not necessarily broken calls. Early research manifests retain old namespace defaults and version defaults for manifest builders. Six tuple-based public type aliases changed recorded signature text from (iterable=(), /) to (*args, **kwargs) under the package's portable normalization. That change is not evidence of a different mathematical representation. Do not edit historical snapshots merely to remove a reported difference.

The full row-by-row additions, removals, kind differences and old/new signature text are in `Block44_API_Migration_Differences.tsv`. It contains 1671 comparison rows, including repeated changes against different historical baselines. Appendix C remains the semantic/API reference for current research symbols.

L.5 Signature changes that require review

Manifest builders now use package_version=None to select the running package version. Supplying an explicit version labels a manifest; it does not install that version or prove its behavior. The broad builder's default namespace tuple expanded as the library grew. Pass an explicit tuple when comparing a historical research surface. Stable and beta builders retain their dedicated namespace contracts.

Route family	Historical difference	Migration action
create_public_api_manifest	Default namespaces expanded; literal version default became None.	Pass the exact desired namespaces and run a separately selected strict policy.
create_beta_api_manifest	Literal version default became None.	Allow runtime version selection, or provide an intentional metadata label.
create_stable_api_manifest	0.15 literal version default became None.	Use the named 0.15 upgrade allowance; retain semantic and persistence checks.
Partition; Matrix; ModularMatrix; ModularVector; Polynomial; Vector	Tuple-based generic alias signature normalization changed.	Treat these as shared type aliases, not automatically as custom domain constructors. Test the actual values your code uses.
shannon_entropy_cost to entropy_cost	Wrapper second keyword is path, canonical second keyword is _.	Use positional second arguments or adjust the keyword intentionally.
projector to component	Wrapper name can overstate idempotence for redundant systems.	Use component terminology and independently establish any required projector laws.

L.6 Explicit identity-alias table

Every following assignment is source-bound and object-identical at runtime. The arrow follows the assignment direction, not a chronology of removal. Some aliases are present in multiple public namespaces. None is decorated as deprecated. The complete source file, line range and checksum are in the electronic record.

L-A01 / NormalizedBasis

`komorebi_math.coordinates.normalized.NormalizedBasis = komorebi_math.coordinates.normalized.WeightedMonomialBasis`. Identity checked in the separately installed R2 package. Source: `komorebi_math/coordinates/normalized.py` line 19; SHA-256 prefix `e92de85740e2`.

L-A02 / ExactCoordinateTransform

`komorebi_math.coordinates.transform.ExactCoordinateTransform = komorebi_math.coordinates.transform.CoordinateTransform`. Identity checked in the separately installed R2 package. Source: `komorebi_math/coordinates/transform.py` line 151; SHA-256 prefix `1e02b12f4797`.

L-A03 / RationalMemoryKernel

`komorebi_math.dynamics.memory.RationalMemoryKernel = komorebi_math.dynamics.memory.MemoryKernel`. Identity checked in the separately installed R2 package. Source: `komorebi_math/dynamics/memory.py` line 130; SHA-256 prefix `9a63a0123887`.

L-A04 / ProjectiveSemigroupTower

`komorebi_math.dynamics.semigroups.ProjectiveSemigroupTower = komorebi_math.dynamics.semigroups.FiniteSemigroupTower`. Identity checked in the separately installed R2 package. Source: `komorebi_math/dynamics/semigroups.py` line 311; SHA-256 prefix `3e905fdef4dc`.

L-A05 / replay_matrix

`komorebi_math.expression.polynomial.replay_matrix` = `komorebi_math.expression.polynomial.replay`. Identity checked in the separately installed R2 package. Source: `komorebi_math/expression/polynomial.py` line 547; SHA-256 prefix `73a4a594e4c0`.

L-A06 / NormalizedBasis

`komorebi_math.generating.NormalizedBasis` = `komorebi_math.generating.WeightedMonomialBasis`. Identity checked in the separately installed R2 package. Source: `komorebi_math/generating/__init__.py` line 384; SHA-256 prefix `b936240b6714`.

L-A07 / NormalizedMonomialBasis

`komorebi_math.generating.NormalizedMonomialBasis` = `komorebi_math.generating.WeightedMonomialBasis`. Identity checked in the separately installed R2 package. Source: `komorebi_math/generating/__init__.py` line 385; SHA-256 prefix `b936240b6714`.

L-A08 / NormalizedMaskPlan

`komorebi_math.generating.masks.NormalizedMaskPlan` = `komorebi_math.generating.masks.FiniteMaskCarryPlan`. Identity checked in the separately installed R2 package. Source: `komorebi_math/generating/masks.py` line 1052; SHA-256 prefix `cd0f5beac886`.

L-A09 / available_autodiff_backends

`komorebi_math.nonlinear.autodiff.available_autodiff_backends` = `komorebi_math.nonlinear.autodiff.available_ad_frameworks`. Identity checked in the separately installed R2 package. Source: `komorebi_math/nonlinear/autodiff.py` line 714; SHA-256 prefix `dae6aa596e65`.

L-A10 / automatic_derivative_provider

`komorebi_math.nonlinear.autodiff.automatic_derivative_provider` = `komorebi_math.nonlinear.autodiff.autodiff_provider`. Identity checked in the separately installed R2 package. Source: `komorebi_math/nonlinear/autodiff.py` line 715; SHA-256 prefix `dae6aa596e65`.

L-A11 / CuPyDeviceDerivativeProvider

`komorebi_math.nonlinear.execution.CuPyDeviceDerivativeProvider` = `komorebi_math.nonlinear.execution.CupyDeviceDerivativeProvider`. Identity checked in the separately installed R2 package. Source: `komorebi_math/nonlinear/execution.py` line 675; SHA-256 prefix `474902031dcf`.

L-A12 / CuPyPolynomialDerivativeProvider

`komorebi_math.nonlinear.execution.CuPyPolynomialDerivativeProvider` = `komorebi_math.nonlinear.execution.CupyPolynomialDerivativeProvider`. Identity checked in the separately installed R2 package. Source: `komorebi_math/nonlinear/execution.py` line 676; SHA-256 prefix `474902031dcf`.

L-A13 / CuPyPacketJetExecutor

`komorebi_math.nonlinear.execution.CuPyPacketJetExecutor` = `komorebi_math.nonlinear.execution.CupyPacketJetExecutor`. Identity checked in the separately installed R2 package. Source: `komorebi_math/nonlinear/execution.py` line 677; SHA-256 prefix `474902031dcf`.

L-A14 / CuPyPacketJetSnapshotExecutor

`komorebi_math.nonlinear.gpu.CuPyPacketJetSnapshotExecutor = komorebi_math.nonlinear.gpu.CuPySnapshotExecutor`. Identity checked in the separately installed R2 package. Source: `komorebi_math/nonlinear/gpu.py` line 195; SHA-256 prefix `2a4ffa8606b9`.

L-A15 / FinitePolyphaseKernel

`komorebi_math.signals.FinitePolyphaseKernel = komorebi_math.signals.PolyphaseBlockMatrix`. Identity checked in the separately installed R2 package. Source: `komorebi_math/signals/__init__.py` line 195; SHA-256 prefix `6f57c41ee2ae`.

L-A16 / finite_polyphase_filter_kernels

`komorebi_math.signals.finite_polyphase_filter_kernels = komorebi_math.signals.finite_polyphase_blocks`. Identity checked in the separately installed R2 package. Source: `komorebi_math/signals/__init__.py` line 196; SHA-256 prefix `6f57c41ee2ae`.

L-A17 / TruncatedPowerPacket

`komorebi_math.signals.singular_packets.TruncatedPowerPacket = komorebi_math.signals.singular_packets.TruncatedPowerTerm`. Identity checked in the separately installed R2 package. Source: `komorebi_math/signals/singular_packets.py` line 104; SHA-256 prefix `856d0f073989`.

L-A18 / RetainedPacketReconstruction

`komorebi_math.signals.singular_packets.RetainedPacketReconstruction = komorebi_math.signals.singular_packets.RetainedPacketResult`. Identity checked in the separately installed R2 package. Source: `komorebi_math/signals/singular_packets.py` line 202; SHA-256 prefix `856d0f073989`.

L-A19 / polyphase_refine

`komorebi_math.stable.polyphase_refine = komorebi_math.stable.refine_polyphase`. Identity checked in the separately installed R2 package. Source: `komorebi_math/stable.py` line 161; SHA-256 prefix `8fc11ed02b89`.

L-A20 / polyphase_coarsen

`komorebi_math.stable.polyphase_coarsen = komorebi_math.stable.coarsen_polyphase`. Identity checked in the separately installed R2 package. Source: `komorebi_math/stable.py` line 162; SHA-256 prefix `8fc11ed02b89`.

L-A21 / lattice_polyphase_refine

`komorebi_math.stable.lattice_polyphase_refine = komorebi_math.stable.refine_lattice_polyphase`. Identity checked in the separately installed R2 package. Source: `komorebi_math/stable.py` line 163; SHA-256 prefix `8fc11ed02b89`.

L-A22 / lattice_polyphase_coarsen

`komorebi_math.stable.lattice_polyphase_coarsen = komorebi_math.stable.coarsen_lattice_polyphase`. Identity checked in the separately installed R2 package. Source: `komorebi_math/stable.py` line 164; SHA-256 prefix `8fc11ed02b89`.

The CuPy capitalization and descriptive aliases can be checked without allocating a GPU object. This establishes only Python identity. It is not evidence that CuPy is installed, a driver works, or a CUDA computation passed. The original Cupy spellings are retained; replacing their imports is optional readability work, not an announced forced migration.

L.7 Delegating wrappers and semantic naming

Retained entry point	Current destination	Boundary
AnalysisSynthesisSystem.projector	component	Value delegation, not independent proof of idempotence.
AnalysisSynthesisSystem.projectors	components	Returns additive reconstruction maps for redundant frames.
HaarPacketTree.decompose	analyze	Classmethod delegation with the same finite-input conditions.
HaarPacketTree.coefficients	nodes	Shallow mapping copy; contained arrays are not newly deep-copied.
shannon_entropy_cost	entropy_cost	Positional semantics agree; second keyword spelling differs.

For the analysis matrix A with rows (1,0), (0,1), (1,1), the canonical dual constructs a valid redundant reconstruction system. Its components sum to I, yet its first component is not idempotent. Both the old projector spelling and the preferred component spelling return that same map. A name change does not repair a missing mathematical hypothesis.

```
import numpy as np
from komorebi_math.coordinates.analysis_synthesis import AnalysisSynthesisSystem

A = np.array([[1., 0.], [0., 1.], [1., 1.]])
system = AnalysisSynthesisSystem.canonical_dual(A)
part = system.component(0)
assert np.allclose(part, system.projector(0))
assert np.allclose(sum(system.components()), np.eye(2))
assert not np.allclose(part @ part, part)
```

This is a finite numerical fixture at the recorded tolerance. The check is not a theorem for all floating inputs. Use Appendix H to understand the separate certificate interfaces, and do not bypass validation to make a migration appear successful.

L.8 Textual selector aliases

Selector spellings are not Python-object aliases or deprecations. Three source-local alias dictionaries retain the following spellings. Their surrounding normalization is part of the contract: sequence/window selectors strip whitespace, lowercase and replace underscores with hyphens; polynomial-family selection uses its separately normalized short-name lookup. The table is the exact literal-map census, not every normalized spelling a user can construct.

Selector owner: `komorebi_math.coordinates.classical.PolynomialFamilyCatalog.get`.

Accepted alias	Destination key
bernoulli	bernoullipolynomials
binomial	binomialpolynomials
chebyshevt	chebyshevtpolynomials
chebyshevu	chebyshevupolynomials
euler	eulerpolynomials
falling	fallingfactorials
fibonacci	fibonaccipolynomials
hermitephysicists	physicistshermitepolynomials
hermiteprobabilists	probabilistshermitepolynomials
laguerre	laguerrepolynomials
legendre	legendrepolynomials
lucas	lucaspolynomials

Accepted alias	Destination key
`monomial`	`monomials`
`pell`	`pellpolynomials`
`rising`	`risingfactorials`

Selector owner: `komorebi_math.sequences.catalog.get_sequence`.

Accepted alias	Destination key
`apery-zeta3`	`apery`
`euler-zigzag`	`zigzag`
`signed-euler`	`euler`
`thue-morse`	`thue-morse`
`thuemorse`	`thue-morse`

Selector owner: `komorebi_math.signals.windows._canonical_name`.

Accepted alias	Destination key
`hanning`	`hann`
`ones`	`boxcar`
`rect`	`boxcar`
`rectangle`	`boxcar`
`rectangular`	`boxcar`
`triang`	`bartlett`
`triangle`	`bartlett`

The thue-morse self-map is an explicit dictionary entry, not a new function. Finite probes compare each sequence alias on the first five terms, polynomial aliases by selected family identity/name, and window aliases by canonical selector. These checks do not make window boundary conventions or polynomial normalization interchangeable across different families.

L.9 Deprecation utility protocol

Field	Meaning	Enforcement
since	First announced deprecation version label.	Stripped and required to be nonempty; not parsed as a release version.
replacement	Suggested alternate callable name.	Optional text; existence or equivalence is not checked.
remove_in	Planned removal label.	Metadata only; no automatic removal.
reason	Explanation of the change.	Optional text added to warning output.

The wrapper exposes `__komorebi_deprecation__`, preserves `__wrapped__` and uses functools.wraps. A normal finite callable retains its reflected signature and delegates arguments and return value. The wrapper does not implement a version clock. Do not rely on it to disable dangerous behavior or execute a removal policy automatically. The generic helper accepts any callable, but the finite tests do not establish transparent class, generator, descriptor or async transformation for all callable forms.

```
import warnings
from komorebi_math import deprecated, KomorebiDeprecationWarning

# Synthetic example, not a newly deprecated library export.
@deprecated(since="0.10", replacement="new_function", remove_in="0.11")
def old_function(value: int) -> int:
    return value + 1

with warnings.catch_warnings(record=True) as caught:
    warnings.simplefilter("always", KomorebiDeprecationWarning)
    assert old_function(2) == 3
    assert len(caught) == 1
```

Warning filters materially affect control flow. Under ignore, the body runs without a recorded warning. Under error, the warning becomes an exception before the body runs. The decorator's observed stacklevel points into compatibility.py, not the external caller. A direct call to warn_deprecated has different stack depth. Test warning category and metadata rather than assuming every displayed filename is the application call site.

A deprecation decision still needs a named symbol, reason/replacement, first warning label, announced removal label when known, persistence and CLI impact, test evidence and manifest differences. The retained policy permits exceptions for correctness/security/packaging defects, but this appendix does not create such a removal or change licensing permissions.

L.10 Portable version migration and file preservation

Python import names, package versions, outer envelope schema versions, individual codec versions and wheel build tags are independent identities. Changing one does not migrate the others. Appendix G provides the complete codec-field reference. This appendix supplies the operational upgrade direction and rollback boundaries.

Format or record	Migration status	Action
nonlinear.packet_jet_snapshot, codec 1 to 2	Single built-in adjacent migration.	Rename provider_name to provider, set coefficient_ordering to lexicographic and retain the other declared values through decoding.
Seven committed 0.15/0.16 portable fixtures	Current reader reconstructs all seven with bound type and payload digest.	Retain producer_version and original bytes.
Ten newly generated records from five predecessor installs	All loaded after actual replacement with R2.	Keep originals separately; newly saved envelopes identify the current producer.
Caller-defined codec with multiple older versions	Every adjacent forward edge is required.	Register migrator= functions and test missing/broken/future edges.
Pickle, undocumented implementation objects, third-party codecs	Excluded from shipped upgrade contract.	No generic upgrade or reverse compatibility claim.

The built-in snapshot edge acts on already decoded codec data. The valid registration keyword is **migrator=**, not migrate=. A direct edge that skips a version is rejected. Missing or failing migrations return failures; future versions are not automatically accepted. No built-in downgrade edge is present. A v2 record cannot be made v1 merely by editing `__version__`.

Keep original files immutable and write migrated output into a new destination. Save/load overwrite checks and payload hashes are useful corruption controls, not authenticity guarantees. The payload digest excludes producer metadata, and loading stored certificates is not proof validation. Neither this appendix nor a successful upgrade fixes the permissive decoder and certificate boundaries recorded in Appendices G-H.

L.11 Executed wheel-to-wheel upgrades

Installed predecessor	New portable records	Replacement target	Observed result
0.15.0a2	2	1.0.0b4 / build 2	Load originals + 15 upgrade checks + 10 semantic cases
0.16.0a2	2	1.0.0b4 / build 2	Load originals + 15 upgrade checks + 10 semantic cases
1.0.0b1	2	1.0.0b4 / build 2	Load originals + 15 upgrade checks + 10 semantic cases
1.0.0b2	2	1.0.0b4 / build 2	Load originals + 15 upgrade checks + 10 semantic cases
1.0.0b3	2	1.0.0b4 / build 2	Load originals + 15 upgrade checks + 10 semantic cases

Each row used a newly created temporary environment. The predecessor wheel was installed with –no-index –no-deps, its imported version and metadata were checked, and every installed package file was compared against that wheel. It then created PolyphaseRefinement(4,12) and PolyphaseRefinementTower((2,6,24)) portable files. After a real pip replacement, a fresh Python process verified R2 version and package-byte parity, loaded both original records and executed the current contract and stable semantic cases. Both before/after workers recorded package-module origins.

The refinement children for residue 1 are (1,5,9), and the tower orders remain (2,6,24). Those finite values and original envelope bytes matched across all five paths. The ten files are newly generated upgrade observations, not ten distinct mathematical algorithms. The seven committed historical fixtures are additional inputs already inside the fifteen-check contract; do not add repeated checks as independent proofs.

The environments inherited existing external dependency paths. Thus these are actual package replacements with package-byte/origin checks, not a newly resolved dependency matrix. Only Linux and the recorded CPython environment were exercised. The stored predecessor wheels remain byte-for-byte unchanged and their archival hashes are included in the migration index.

L.12 Same-version wheel builds, process restart and rollback

The public version remains 1.0.0b4, while the repaired distribution filename carries wheel build tag 2 and the release identity is 1.0.0b4-block36-r2. Version text alone cannot distinguish the stale original b4 wheel from R2. Pin the actual artifact hash and use an explicit replacement when an old same-version wheel may already be installed.

```
python -m pip install --no-index --no-deps --force-reinstall \
  release/dist/komorebi_math-1.0.0b4-2-py3-none-any.whl
```

The command assumes the required dependencies are already available. On PowerShell, enter it as one line rather than copying the POSIX continuation backslash. Native PowerShell execution is not claimed. Perform the replacement in the intended environment, then restart the Python process or notebook kernel before importing the new code. Replacing files on disk is not a supported way to migrate objects already retained in a running interpreter.

For rollback, retain the predecessor wheel, its hashes, environment description, application requirements and original portable records outside the upgrade environment. Recreate a separate prior environment and read original records there. The recorded forward tests do not establish that a predecessor can read a newly written R2 envelope or a newly introduced codec. Do not overwrite an original with an unqualified downgrade. Avoid mixing old and new object instances in one process.

L.13 Contract checks and caller-controlled coverage

A custom path accepted by check_upgrade_contract changes the set of checks performed. The reduced-contract probe deliberately supplied no fixtures and an empty predecessor list; the remaining checks passed. That is not evidence for the omitted records. A consumer requiring the shipped contract must independently pin its file digest and require five predecessor names, seven fixture records, five stable baselines and fifteen executed checks.

The ordinary strict API comparator is additive: additions are allowed, while removed exports, broad kind changes and signature changes fail. The stable predecessor upgrade branch imposes the stronger no-additions freeze for every predecessor except the specifically allowed 0.15 migration. Changing implementation_module or qualname alone does not fail compare_public_api. Stable semantic fixtures and your own regression examples therefore remain essential.

```
from komorebi_math.release import check_upgrade_contract
from komorebi_math.stable_semantics import run_stable_semantics_fixture

report = check_upgrade_contract()
assert report.contract.target_version == "1.0.0b4"
assert len(report.contract.predecessor_versions) == 5
assert len(report.contract.fixtures) == 7
assert len(report.checks) == 15
assert report.passed
assert run_stable_semantics_fixture().certificate.valid
```

These assertions expose the intended inventory but are not an authenticated trust anchor. Use a trusted release checksum for the contract and wheel, and preserve the actual result record. A checksum published by the same untrusted writer is not authentication. The report's stored validity flag is not a Lean proof.

L.14 Complete stable-facade identity ledger

For every name below, both versioned namespaces refer to the same value as komorebi_math.stable. The table is a complete 131-name identity ledger. It does not expand the frozen stable API with research-only symbols.

Stable export	Both versioned facades
AnalysisSynthesisBlocks	Object-identical
AnalysisSynthesisSystem	Object-identical
ApiCompatibilityReport	Object-identical
ArithmeticPolyphase	Object-identical
Arrow	Object-identical
AssumptionError	Object-identical
BackendUnavailableError	Object-identical
Certificate	Object-identical
ComputationResult	Object-identical
ConditionalResult	Object-identical
CoordinateAtlas	Object-identical
CoordinateTransform	Object-identical
DEFAULT_PORTABLE_CODEC_CONTRACT	Object-identical
DEFAULT_STABLE_SEMANTICS_FIXTURE	Object-identical
DimensionError	Object-identical
DocumentMetadata	Object-identical
Equation	Object-identical
ExactCoordinateTransform	Object-identical
Exactness	Object-identical
ExactnessError	Object-identical
FiniteBasis	Object-identical
FormalCoordinateChange	Object-identical
FormalSeries	Object-identical

Stable export	Both versioned facades
`KomorebiMathError`	Object-identical
`LatticePolyphaseCoordinates`	Object-identical
`LatticePolyphaseRefinement`	Object-identical
`MathDocument`	Object-identical
`NumericCoordinateTransform`	Object-identical
`OperatorGrammar`	Object-identical
`OperatorRealization`	Object-identical
`OperatorRelation`	Object-identical
`OperatorWord`	Object-identical
`PORTABLE_CODEC_CONTRACT_SCHEMA`	Object-identical
`PORTABLE_CODEC_CONTRACT_SCHEMA_VERSION`	Object-identical
`PORTABLE_SCHEMA`	Object-identical
`PORTABLE_SCHEMA_VERSION`	Object-identical
`PacketCoordinates`	Object-identical
`PacketOperator`	Object-identical
`PacketRefinement`	Object-identical
`PacketSystem`	Object-identical
`Paragraph`	Object-identical
`PolyphaseCoordinateRecord`	Object-identical
`PolyphaseCoordinates`	Object-identical
`PolyphaseRefinement`	Object-identical
`PolyphaseRefinementTower`	Object-identical
`PolyphaseSystem`	Object-identical
`PortableCodec`	Object-identical
`PortableCodecCompatibilityReport`	Object-identical
`PortableCodecContract`	Object-identical
`PortableCodecContractEntry`	Object-identical
`PortableMigration`	Object-identical
`PublicApiManifest`	Object-identical
`PublicApiSymbol`	Object-identical
`RefinementTower`	Object-identical
`RelationResidual`	Object-identical
`ResidueCoordinates`	Object-identical
`ResourceLimitError`	Object-identical
`STABLE_API_STABILITY_LEVEL`	Object-identical
`STABLE_PUBLIC_NAMESPACES`	Object-identical
`STABLE_SEMANTICS_SCHEMA`	Object-identical
`STABLE_SEMANTICS_SCHEMA_VERSION`	Object-identical
`SeedSystem`	Object-identical
`Settings`	Object-identical
`StableContractReport`	Object-identical
`StableSemanticCaseResult`	Object-identical
`StableSemanticReport`	Object-identical
`TransferGraph`	Object-identical
`TransferPath`	Object-identical
`ValidationError`	Object-identical
`__version__`	Object-identical
`arithmetic_mixed_radix_address`	Object-identical
`arithmetic_mixed_radix_digits`	Object-identical
`arithmetic_polyphase_analyze`	Object-identical
`arithmetic_polyphase_synthesize`	Object-identical
`basis_from_matrix`	Object-identical
`block_from_paths`	Object-identical

Stable export	Both versioned facades
`check_portable_codec_contract`	Object-identical
`check_stable_api`	Object-identical
`check_stable_contract`	Object-identical
`coarsen_lattice_polyphase`	Object-identical
`coarsen_polyphase`	Object-identical
`common_refinement`	Object-identical
`compare_portable_codec_contract`	Object-identical
`configure`	Object-identical
`connection_transform`	Object-identical
`create_portable_codec_contract`	Object-identical
`create_stable_api_manifest`	Object-identical
`dumps_portable`	Object-identical
`factorial_power_basis`	Object-identical
`formal_coordinate_from_iterable`	Object-identical
`forward_difference_transform`	Object-identical
`from_portable_data`	Object-identical
`get_polynomial_family`	Object-identical
`get_settings`	Object-identical
`lattice_polyphase_analyze`	Object-identical
`lattice_polyphase_coarsen`	Object-identical
`lattice_polyphase_refine`	Object-identical
`lattice_polyphase_synthesize`	Object-identical
`load_portable`	Object-identical
`load_portable_codec_contract`	Object-identical
`load_stable_api_manifest`	Object-identical
`load_stable_semantics_fixture`	Object-identical
`loads_portable`	Object-identical
`mahler_polyphase`	Object-identical
`nested_residue_address`	Object-identical
`nested_residue_analysis`	Object-identical
`ordinary_generating_basis`	Object-identical
`path_contributions`	Object-identical
`polyphase_coarsen`	Object-identical
`polyphase_refine`	Object-identical
`polyphase_refinement_certificate`	Object-identical
`polyphase_roundtrip_certificate`	Object-identical
`portable_codecs`	Object-identical
`portable_digest`	Object-identical
`portable_migrations`	Object-identical
`refine_lattice_polyphase`	Object-identical
`refine_polyphase`	Object-identical
`refinement_children`	Object-identical
`render_latex`	Object-identical
`render_markdown`	Object-identical
`residue_analyze`	Object-identical
`residue_project`	Object-identical
`residue_synthesize`	Object-identical
`run_stable_semantics_fixture`	Object-identical
`save_portable`	Object-identical
`save_portable_codec_contract`	Object-identical
`settings_context`	Object-identical
`signal_polyphase_analyze`	Object-identical
`signal_polyphase_synthesize`	Object-identical

Stable export	Both versioned facades
`standard_basis`	Object-identical
`to_portable_data`	Object-identical

L.15 Executable migration and deprecation recipes

All thirty cases execute in fresh bounded subprocesses against the pinned installed wheel. The cases include expected exceptions, permissive-boundary observations and synthetic metadata controls. A passed expectation is not always a positive compatibility or security qualification. Full inputs, result records, tracebacks for failed attempts and module-origin records are retained in the verification directory.

M01 / Complete shipped upgrade contract

Runs the complete shipped contract: five stable manifest baselines, one semantic continuity check, one codec continuity check, seven digest-bound portable fixtures and target identity. Fifteen checks pass. Read the inventory and unchecked assumptions, not only the Boolean result.

Executed evidence: `migration_replay/M01/result.json`; expectation passed, worker exit 0. Reproduce with `python tools/block44/run_probes.py --output <fresh-directory>`. Each worker has a 90-second timeout; no fixture attempts uncontrolled memory exhaustion or native GPU execution.

M02 / Execute the ten stable semantic fixtures

Actively computes all ten current stable semantic cases. This is separate from the upgrade checker, whose semantic continuity step compares stored fixture content and digests. The active finite cases are not a proof of every input.

Executed evidence: `migration_replay/M02/result.json`; expectation passed, worker exit 0. Reproduce with `python tools/block44/run_probes.py --output <fresh-directory>`. Each worker has a 90-second timeout; no fixture attempts uncontrolled memory exhaustion or native GPU execution.

M03 / Reject a wrong target version

Changing only the target version produces a failed target-identity check. The fixture does not alter the installed package version to make a stale contract appear current.

Executed evidence: `migration_replay/M03/result.json`; expectation passed, worker exit 0. Reproduce with `python tools/block44/run_probes.py --output <fresh-directory>`. Each worker has a 90-second timeout; no fixture attempts uncontrolled memory exhaustion or native GPU execution.

M04 / Reject a changed fixture checksum

A changed expected source-file checksum is rejected even when the actual fixture is otherwise valid. Restore an independently retained correct original instead of updating the checksum to conceal corruption.

Executed evidence: `migration_replay/M04/result.json`; expectation passed, worker exit 0. Reproduce with `python tools/block44/run_probes.py --output <fresh-directory>`. Each worker has a 90-second timeout; no fixture attempts uncontrolled memory exhaustion or native GPU execution.

M05 / Reject a wrong reconstructed type

A deliberately wrong reconstructed Python type is rejected. Portable equality, type identity and producer metadata are separate checks.

Executed evidence: `migration_replay/M05/result.json`; expectation passed, worker exit 0. Reproduce with `python tools/block44/run_probes.py --output <fresh-directory>`. Each worker has a 90-second timeout; no fixture attempts uncontrolled memory exhaustion or native GPU execution.

M06 / Reduced caller contract can pass with less coverage

A caller-supplied contract with an empty predecessor list and no portable fixtures still passes its remaining checks. Passing does not enforce that the caller selected the shipped coverage. Pin the contract file hash and require the expected five predecessors, seven fixtures and fifteen checks.

Executed evidence: `migration_replay/M06/result.json`; expectation passed, worker exit 0. Reproduce with `python tools/block44/run_probes.py --output <fresh-directory>`. Each worker has a 90-second timeout; no fixture attempts uncontrolled memory exhaustion or native GPU execution.

M07 / Reject removed exports

Removing a stable export in a synthetic manifest fails both ordinary and strict compatibility. This tests the comparator, not a modified installation.

Executed evidence: `migration_replay/M07/result.json`; expectation passed, worker exit 0. Reproduce with `python tools/block44/run_probes.py --output <fresh-directory>`. Each worker has a 90-second timeout; no fixture attempts uncontrolled memory exhaustion or native GPU execution.

M08 / Reject broad kind changes

Changing a broad symbol kind fails both policies. The fixture changes metadata only and does not claim to exercise an alternative implementation.

Executed evidence: `migration_replay/M08/result.json`; expectation passed, worker exit 0. Reproduce with `python tools/block44/run_probes.py --output <fresh-directory>`. Each worker has a 90-second timeout; no fixture attempts uncontrolled memory exhaustion or native GPU execution.

M09 / Signature strictness is explicit

A changed signature is reported by both policies but fails only with strict_signatures=True. The ordinary additive certificate can be true with signature differences.

Executed evidence: `migration_replay/M09/result.json`; expectation passed, worker exit 0. Reproduce with `python tools/block44/run_probes.py --output <fresh-directory>`. Each worker has a 90-second timeout; no fixture attempts uncontrolled memory exhaustion or native GPU execution.

M10 / Implementation identity is not a compatibility check

Changing only implementation_module and qualname in a synthetic manifest leaves strict compatibility true. The comparator does not inspect implementation equivalence. Use semantic fixtures and application tests separately.

Executed evidence: `migration_replay/M10/result.json`; expectation passed, worker exit 0. Reproduce with `python tools/block44/run_probes.py --output <fresh-directory>`. Each worker has a 90-second timeout; no fixture attempts uncontrolled memory exhaustion or native GPU execution.

M11 / Additive symbols remain structurally compatible

Adding a new exported symbol passes ordinary and strict additive comparison. A stronger exact-freeze gate must additionally require an empty added set.

Executed evidence: `migration_replay/M11/result.json`; expectation passed, worker exit 0. Reproduce with `python tools/block44/run_probes.py --output <fresh-directory>`. Each worker has a 90-second timeout; no fixture attempts uncontrolled memory exhaustion or native GPU execution.

M12 / Decorated call retains arguments and metadata

A synthetic deprecated function preserves its inspect.signature, `__wrapped__`, arguments, return value and deprecation record. The always warning filter records the warning. This is a fixture, not a newly deprecated library export.

Executed evidence: `migration_replay/M12/result.json`; expectation passed, worker exit 0. Reproduce with `python tools/block44/run_probes.py --output <fresh-directory>`. Each worker has a 90-second timeout; no fixture attempts uncontrolled memory exhaustion or native GPU execution.

M13 / Warnings-as-errors blocks the body

The error warning filter turns the warning into an exception before the wrapped function body runs. A test that elevates warnings can therefore change control flow.

Executed evidence: `migration_replay/M13/result.json`; expectation passed, worker exit 0. Reproduce with `python tools/block44/run_probes.py --output <fresh-directory>`. Each worker has a 90-second timeout; no fixture attempts uncontrolled memory exhaustion or native GPU execution.

M14 / Warning filters may suppress output

The ignore filter suppresses the warning while the wrapped function continues to run. Absence of visible warnings is not evidence that deprecation metadata is absent.

Executed evidence: `migration_replay/M14/result.json`; expectation passed, worker exit 0. Reproduce with `python tools/block44/run_probes.py --output <fresh-directory>`. Each worker has a 90-second timeout; no fixture attempts uncontrolled memory exhaustion or native GPU execution.

M15 / Removal metadata and warning source location

The recorded remove_in version does not disable the callable. The decorator uses stacklevel=2 inside its wrapper, and the observed warning points into compatibility.py rather than the external call site. Do not treat that location as the application caller.

Executed evidence: `migration_replay/M15/result.json`; expectation passed, worker exit 0. Reproduce with `python tools/block44/run_probes.py --output <fresh-directory>`. Each worker has a 90-second timeout; no fixture attempts uncontrolled memory exhaustion or native GPU execution.

M16 / Reject malformed deprecation setup

Blank since text, a noncallable decoration target and a blank direct warning name are rejected. These are small utility validation checks, not a complete semantic version parser.

Executed evidence: `migration_replay/M16/result.json`; expectation passed, worker exit 0. Reproduce with `python tools/block44/run_probes.py --output <fresh-directory>`. Each worker has a 90-second timeout; no fixture attempts uncontrolled memory exhaustion or native GPU execution.

M17 / Version labels are not a scheduler

Nonempty non-version strings are accepted as since and remove_in labels. The utility does not order versions, validate a release calendar, or enforce a removal deadline.

Executed evidence: `migration_replay/M17/result.json`; expectation passed, worker exit 0. Reproduce with `python tools/block44/run_probes.py --output <fresh-directory>`. Each worker has a 90-second timeout; no fixture attempts uncontrolled memory exhaustion or native GPU execution.

M18 / Redundant projector wrapper is not idempotent

The redundant canonical-dual fixture returns equal values from projector and component, but the component is not idempotent. Components sum to the identity. No deprecation warning is emitted. Use component terminology unless idempotence is established independently.

Executed evidence: `migration_replay/M18/result.json`; expectation passed, worker exit 0. Reproduce with `python tools/block44/run_probes.py --output <fresh-directory>`. Each worker has a 90-second timeout; no fixture attempts uncontrolled memory exhaustion or native GPU execution.

M19 / Wavelet wrapper and keyword boundary

HaarPacketTree.decompose delegates to analyze, coefficients exposes the nodes mapping, and shannon_entropy_cost matches entropy_cost positionally. The wrapper accepts path= while the canonical entropy_cost second parameter is named _. A blind keyword-preserving rename can fail.

Executed evidence: `migration_replay/M19/result.json`; expectation passed, worker exit 0. Reproduce with `python tools/block44/run_probes.py --output <fresh-directory>`. Each worker has a 90-second timeout; no fixture attempts uncontrolled memory exhaustion or native GPU execution.

M20 / All explicit object aliases

Every one of the twenty-two explicit module-level function/class assignment aliases is object-identical to its source name. Class identity does not require constructing a GPU executor and does not qualify CUDA.

Executed evidence: `migration_replay/M20/result.json`; expectation passed, worker exit 0. Reproduce with `python tools/block44/run_probes.py --output <fresh-directory>`. Each worker has a 90-second timeout; no fixture attempts uncontrolled memory exhaustion or native GPU execution.

M21 / All versioned stable aliases

All 131 stable exports are identical to their counterparts in both komorebi_math.v1 and komorebi_math.api.v1. Namespaces themselves remain different module objects.

Executed evidence: `migration_replay/M21/result.json`; expectation passed, worker exit 0. Reproduce with `python tools/block44/run_probes.py --output <fresh-directory>`. Each worker has a 90-second timeout; no fixture attempts uncontrolled memory exhaustion or native GPU execution.

M22 / Actual snapshot v1 to v2 migration

Loads the genuine version-1 packet-jet snapshot fixture retained in Appendix G and re-encodes it as codec version 2. Original data stays immutable; the new record supplies provider and lexicographic ordering.

Executed evidence: `migration_replay/M22/result.json`; expectation passed, worker exit 0. Reproduce with `python tools/block44/run_probes.py --output <fresh-directory>`. Each worker has a 90-second timeout; no fixture attempts uncontrolled memory exhaustion or native GPU execution.

M23 / Missing custom migration chain is refused

A custom version-3 codec cannot decode version-1 data when the adjacent migration chain is missing. This synthetic codec is local to a disposable probe process.

Executed evidence: `migration_replay/M23/result.json`; expectation passed, worker exit 0. Reproduce with `python tools/block44/run_probes.py --output <fresh-directory>`. Each worker has a 90-second timeout; no fixture attempts uncontrolled memory exhaustion or native GPU execution.

M24 / Future codec version is refused

A future codec version is refused. There is no automatic reverse migration or undocumented future-version acceptance.

Executed evidence: `migration_replay/M24/result.json`; expectation passed, worker exit 0. Reproduce with `python tools/block44/run_probes.py --output <fresh-directory>`. Each worker has a 90-second timeout; no fixture attempts uncontrolled memory exhaustion or native GPU execution.

M25 / Broken migrator is wrapped

A migrator that raises RuntimeError produces a wrapped ValidationError. The failure is preserved rather than turning a partly migrated value into a successful load.

Executed evidence: `migration_replay/M25/result.json`; expectation passed, worker exit 0. Reproduce with `python tools/block44/run_probes.py --output <fresh-directory>`. Each worker has a 90-second timeout; no fixture attempts uncontrolled memory exhaustion or native GPU execution.

M26 / Successful custom adjacent chain

A disposable custom 1-to-2-to-3 migration chain produces the expected finite value. This is evidence for the extension protocol, not two additional built-in migration edges.

Executed evidence: `migration_replay/M26/result.json`; expectation passed, worker exit 0. Reproduce with `python tools/block44/run_probes.py --output <fresh-directory>`. Each worker has a 90-second timeout; no fixture attempts uncontrolled memory exhaustion or native GPU execution.

M27 / Adjacent edges and the migrator keyword

A nonadjacent edge is rejected. The accepted keyword is migrator=; the obsolete migrate= spelling raises TypeError. Update the call rather than patching a persisted version tag.

Executed evidence: `migration_replay/M27/result.json`; expectation passed, worker exit 0. Reproduce with `python tools/block44/run_probes.py --output <fresh-directory>`. Each worker has a 90-second timeout; no fixture attempts uncontrolled memory exhaustion or native GPU execution.

M28 / Preserve originals and reject a damaged envelope

Saving without explicit overwrite refuses replacement and leaves the original bytes unchanged. A separately damaged envelope digest is rejected. Checksums are corruption checks, not authentication.

Executed evidence: `migration_replay/M28/result.json`; expectation passed, worker exit 0. Reproduce with `python tools/block44/run_probes.py --output <fresh-directory>`. Each worker has a 90-second timeout; no fixture attempts uncontrolled memory exhaustion or native GPU execution.

M29 / Read all seven committed predecessor records

Reconstructs all seven committed predecessor records and compares their expected types, producers and payload digests. These are stored historical fixtures, distinct from the ten records newly written by actual predecessor installations.

Executed evidence: `migration_replay/M29/result.json`; expectation passed, worker exit 0. Reproduce with `python tools/block44/run_probes.py --output <fresh-directory>`. Each worker has a 90-second timeout; no fixture attempts uncontrolled memory exhaustion or native GPU execution.

M30 / All three current API contracts

Current stable, beta and research manifests pass their selected strict checks. Those positive structural checks coexist with the separately documented certificate, decoder and resource limitations.

Executed evidence: `migration_replay/M30/result.json`; expectation passed, worker exit 0. Reproduce with `python tools/block44/run_probes.py --output <fresh-directory>`. Each worker has a 90-second timeout; no fixture attempts uncontrolled memory exhaustion or native GPU execution.

L.16 Source mapping, reproducibility and remaining work

Primary sources are compatibility.py, release/upgrade.py, the thirty retained API manifests, the shipped upgrade contract and its seven fixtures, the single portable migration registration, the explicit alias/wrapper implementations, and the deprecation/migration policy documents. Each electronic record includes a source-file SHA-256; AST records retain source ranges and exact text. No external website or later repository state was substituted for the pinned source.

The full source suite passed 1,203 tests. The new reference suite passed 572 tests, including per-file census checks, all historical comparisons, all stable/explicit aliases, selector routes, fixture identities and executed receipts. Thirty probe expectations and five genuine package-upgrade paths passed. The counts overlap. They are neither new theorem counts nor universal semantic equivalence claims.

```
python tools/block44/inventory.py
python tools/block44/upgrade_lab.py --output <fresh-upgrade-directory>
```

```
python tools/block44/run_probes.py --output <fresh-probe-directory>
python tools/block44/run_reference_tests.py
python tools/block44/run_source_tests.py
python tools/block44/generate_manual.py
python tools/block44/build_pdfs.py
python VERIFY_BLOCK44_PACKAGE.py
```

Use a separate working copy for regeneration. The installed-reference tools require the separately installed pinned wheel, not a source checkout on PYTHONPATH. They do not download dependencies. Default evidence directories are immutable replay inputs after review; archive earlier receipts before replacing them. Re-running tools changes evidence hashes and requires a newly reviewed outer archive.

Reference boundaries. This migration and deprecation reference covers the stated alias census and finite upgrade observations for the frozen source. The 901 library files matched that baseline. The observations do not modify package, example or distribution bytes.

The recorded audit did not qualify native Windows/PowerShell, other Python/dependency versions, CUDA, strict MkDocs, fully dependency-isolated installation or final publication. Certificate, portable-reader and resource limitations remain separate from finite upgrade success. The audit included no new Lean proof qualification. Appendix O identifies the final-source execution records.

Appendix M. Glossary and notation index

This is the version-bound terminology reference for **Komorebi Math 1.0.0b4, Repair R2, wheel build 2**. It separates the ordinary mathematical or technical meaning of a term from the behavior of this implementation. A mathematical definition, a stored status flag, a source label and a qualified software artifact answer different questions.

Use this appendix while reading, not as a prerequisite. Search M.4 for an unfamiliar word and M.5 for a symbol. Read the general meaning first, then the "In this library" paragraph, and return to the cited chapter for the full construction or example. M.2 includes alternate search wording when the name you remember differs from the printed term.

If a familiar term seems inconsistent with a nearby formula, check its local meaning before changing the formula. M.3 collects common collisions such as signal frames versus redundant frames, packet projectors versus reconstruction components, and API stability versus a stable core. For a symbol, check the surrounding space, domain and indices as well as its glyph. The same printed symbol can denote different objects in different chapters.

A useful lookup habit. Keep the chapter name with a copied definition or notation card. That context is often enough to distinguish two otherwise identical search results. Follow "See also" links when the distinction concerns the relationship between two objects, such as analysis and synthesis, rather than just an unfamiliar word.

M.1 Scope and reading rules

Reference surface	Included
Reviewed glossary concepts	223
Locally scoped notation senses	66
Distinct displayed notation strings	65
Navigation redirects	28
Explicit terminology collision groups	16
Source chapters/appendices supporting glossary terms	69
Bound Python source files	300
Formula occurrences in the frozen reader source	6411
Finite semantic probe cases	24

The glossary covers the **declared 223-entry concept catalog** and its 66 notation records. It is not an assertion that every ordinary word, every exported method name or every locally bound dummy variable in this 2,000-plus-page manual has a separate semantic definition. Appendices A-D retain exhaustive API/module references. The electronic formula concordance supplies every mathematical span recognized by the delimiter rule below, including occurrences in the historical planning material; it does not infer meanings from glyph frequency.

Each glossary record contains the term, general definition, package-specific interpretation, classical/general sense, explanatory source anchor, related concepts and notation links. Each notation record contains its displayed glyph, spoken form, local definition, explanatory source anchor, domain, collisions and related glossary entry. These implement the retained glossary/notation record shapes at the end of Chapter 88.

Source precedence. Definitions summarize the pinned reader source. Later source-backed Appendix G-H-I-K-L qualifications take precedence over an older unqualified sentence where the two differ. No independent current-library website or later repository revision is substituted. The glossary's frozen Markdown and TeX source remain the authority for its recorded line numbers and hashes. Typeset pagination does not replace those source coordinates.

First anchor. This field means the explanatory chapter for the indexed local sense. It is not a claim about the earliest printed appearance of the same glyph anywhere in the manual. Some displays normalize the source wording into

a convenient expression. Every notation card identifies whether its exact displayed spelling was found in its source chapter. A normalized display is not attributed as a verbatim source formula.

Status separation. General mathematics describes a property under hypotheses. A local implementation decides finite predicates. A certificate stores a producer's report. A portable codec reconstructs that report. Provenance declares a claim class. Release qualification accepts a specified execution profile. None of those operations silently promotes the others into a new proof.

M.2 Fast lookup and ambiguity policy

Alphabetical cards appear in M.4. M.5 is the notation register and M.6 contains executable meaning checks. The electronic lookup tool never chooses one overloaded sense merely because it appears first. An exact collision-group query expands to its distinct concepts. Chapter filtering narrows context; it does not merge meanings.

```
python tools/block45/lookup.py "frame" --kind term
python tools/block45/lookup.py "P_i" --kind notation
python tools/block45/lookup.py "NM-001" --strict-unique
python tools/block45/lookup.py "P_i" --strict-unique
```

The last invocation intentionally exits nonzero because two local records share that glyph. Queries emit JSON. A query with no matches returns an empty result and a nonzero exit status. It does not invent a definition. Search redirects below are navigation aids, not assertions that two concepts or Python objects are identical.

Search wording	Navigate to
adjoint matrix	Adjoint
API stability	API tier
automatic differentiation	Autodiff adapter
basis change	Coordinate transform
Chinese remainder theorem coordinates	CRT coordinates
coefficient gauge freedom	Coefficient gauge
congruence lane	Lane
CRC and SHA-256 checks	Source checksum
CRT	CRT coordinates
cyclic seed	Cyclic vector
deprecation warning	Deprecation
Faà di Bruno formula	Faà di Bruno composition
finite-field Frobenius	Frobenius orbit
formal power series	Formal series
frozen provider	Frozen record
jet	Packet jet
kernel modulo image	Cohomology
linear map realization	Operator realization
normalization weight	Weighted monomial basis
orthogonal packet	Orthogonal projector
packet block	Transfer block
portable envelope	Portable record
power spectral density	Power spectral density
prime-adic valuation	Valuation
PSD	Power spectral density
release acceptance	Qualification
right-hand-side batching	Structured operator
short-time Fourier transform	STFT

M.3 Words whose meanings must stay separate

M.C01. frame

A signal frame is a windowed segment. A redundant frame is an analysis/synthesis presentation.

Compare: Frame, Analysis-synthesis system.

M.C02. frame operator

The package method returns SA; conventional frame notation often uses A* A.

Compare: Frame operator, Analysis-synthesis system.

M.C03. projector

Reconstruction components need not be idempotent; coefficient projectors and physical packet projectors live in different spaces.

Compare: Projector, Reconstruction component, Coefficient projector.

M.C04. exact / valid / proved

Arithmetic/evidence class, stored validity and author-supplied provenance are independent axes.

Compare: Exactness, Validity flag, Claim status.

M.C05. stable

Compatibility tier and surviving candidates in a finite forest are different meanings.

Compare: Stable API, Stable core.

M.C06. Frobenius

The field power map is not a matrix norm.

Compare: Frobenius orbit, Frobenius norm.

M.C07. refinement

Ambient-space packet refinement is not sample-rate conversion.

Compare: Packet refinement, Polyphase refinement, Resampling.

M.C08. support

Boolean transfer support, retained jet addresses and algebraic support/fibers are distinct.

Compare: Transfer graph, Jet support, Fitting fiber.

M.C09. tree

Nonlinear composition trees and signal basis trees have different node semantics.

Compare: Rooted tree, Wavelet packet tree.

M.C10. version

Package, envelope and codec versions advance under separate contracts.

Compare: Codec, Migration, Package upgrade.

M.C11. kernel

A memory kernel, an arithmetic coefficient kernel and the kernel of a linear map are different objects.

Compare: Memory kernel, Digit kernel, Cohomology.

M.C12. gate

Release requirement selection is not proof of an external mathematical completion condition.

Compare: Required gate, Completion gate.

M.C13. backend

Core selection, dependency discovery and an autodiff interface are not interchangeable capabilities.

Compare: Backend, Backend discovery, Autodiff adapter.

M.C14. convolution

Linear, circular and binomially weighted products are different operations on stored data.

Compare: Convolution, Circular convolution, Hurwitz series.

M.C15. residual / defect

A discrepancy is not automatically numerical noise, and a finite obstruction is not automatically a universal no-go theorem.

Compare: Residual, Defect, Obstruction.

M.C16. immutable

Frozen attributes, read-only array buffers and deep immutability are not equivalent.

Compare: Frozen record, Certificate, Snapshot.

M.4 Alphabetical glossary

General meaning includes ordinary technical vocabulary where a term has no independent classical-mathematics sense. The "In this library" paragraph controls implementation interpretation. Source locations support navigation; they are not certificates of the truth of every nearby statement.

Adaptive filter

General meaning. A filter whose coefficients are updated using an observed signal and a declared update rule.

In this library. Finite coefficient updates do not establish universal convergence or learning performance.

Source. Chapter 34 (explanatory anchor `resampling-and-adaptive-filters`); frozen reference-source Markdown lines 29105-29109.

See also. Filter state, Resource budget.

Adjoint

General meaning. The transpose-conjugate map in the global Euclidean/Hermitian convention, or a metric-dependent adjoint when explicitly declared.

In this library. A transpose alone omits conjugation. Frame coefficient and physical adjoints can use different metrics.

Source. Chapter 1 (explanatory anchor `editorial-and-symbolic-grammar`); frozen reference-source Markdown lines 341-347.

See also. Orthogonal projector, Frame operator. **Notation:** NM-007.

Alias

General meaning. A retained name that refers to the same object, or a separately documented delegating compatibility route.

In this library. Identity aliases and wrappers are distinct. A retained spelling is not automatically deprecated.

Source. Appendix L (explanatory anchor `appendix-l-migrations`); frozen reference-source Markdown lines 109674-109680.

See also. Deprecation, Qualified name.

Alpha equivalence

General meaning. Equality of binding structure up to consistent renaming of bound variables.

In this library. It is not arbitrary renaming of free variables and not by itself equality of every numerical expression.

Source. Chapter 50 (explanatory anchor `typed-finite-expression-model`); frozen reference-source Markdown lines 43318-43324.

See also. Lexical binding, Decidable equality. **Notation:** NM-063.

Analysis map

General meaning. A map from physical vectors to their coefficient representation.

In this library. In a redundant presentation the coefficient dimension can exceed the physical dimension.

Source. Chapter 20 (explanatory anchor `analysis-synthesis-and-redundant-frames`); frozen reference-source Markdown lines 15519-15525.

See also. Synthesis map, Analysis-synthesis system. **Notation:** NM-028.

Analysis-synthesis system

General meaning. A pair of compatible analysis and synthesis maps forming a left-inverse presentation when verified.

In this library. Bare numerical construction can store a proposal with a failed left-inverse certificate. Construction alone does not establish reconstruction.

Source. Chapter 20 (explanatory anchor `analysis-synthesis-and-redundant-frames`); frozen reference-source Markdown lines 15505-15511.

See also. Analysis map, Synthesis map, Frame operator. **Notation:** NM-030.

API tier

General meaning. A compatibility-policy category attached to an interface.

In this library. Stable, beta and research describe the import surface, not mathematical truth or numerical accuracy.

Source. Chapter 7 (explanatory anchor `public-api-tiers`); frozen reference-source Markdown lines 2599-2605.

See also. Exactness, Stable API, Research API.

Appell family

General meaning. A polynomial family satisfying the stated derivative-lowering relation.

In this library. Exact symbolic finite identities do not certify a global zero pattern or convergence of a truncated product.

Source. Chapter 27 (explanatory anchor `cyclic-analytic-systems`); frozen reference-source Markdown lines 24949-24955.

See also. Cyclic analytic packet, Zero certificate.

Approximation budget

General meaning. A declared allowance for a finite approximation or retained model.

In this library. Approximation error, arithmetic roundoff and process memory are different budgets.

Source. Chapter 36A (explanatory anchor `a.-approximation-nonlinear-packet-products-and-smith-charts`); frozen reference-source Markdown lines 31454-31458.

See also. Resource budget, Numerical result.

Arrow

General meaning. A directed operator symbol with a source type and a target type.

In this library. Grammar legality is established before a realization supplies matrices.

Source. Chapter 9 (explanatory anchor `typed-operator-grammars`); frozen reference-source Markdown lines 3802-3808.

See also. Operator word, Operator grammar.

Assumption

General meaning. A prerequisite on which a stated conclusion depends.

In this library. Checked and unchecked assumptions are separate fields. Declaring a hypothesis does not demonstrate it.

Source. Chapter 5 (explanatory anchor `certificates-and-result-types`); frozen reference-source Markdown lines 1545-1551.

See also. Conditional result, Certificate.

Autodiff adapter

General meaning. A bridge from a supported automatic-differentiation framework into the derivative-provider interface.

In this library. Device, dtype and supported derivative order are explicit. CPU execution does not qualify CUDA.

Source. Chapter 37 (explanatory anchor `derivative-providers-and-automatic-differentiation`); frozen reference-source Markdown lines 31948-31952.

See also. Derivative provider, Backend.

Automatic sequence

General meaning. A sequence evaluated from a finite-state process over radix digits.

In this library. The declared radix and digit direction matter. Fitting a finite sample is not a proof that the whole sequence is automatic.

Source. Chapter 26 (explanatory anchor `exact-sequence-execution`); frozen reference-source Markdown lines 23439-23445.

See also. Cartier automaton, Sequence model.

Backend

General meaning. An implementation of a particular execution or algebra interface.

In this library. Core linear algebra, autodiff adapters and GPU executors are separate interfaces. Discovery does not imply usability or qualification.

Source. Chapter 8 (explanatory anchor `backends-scalar-domains-and-coercion`); frozen reference-source Markdown lines 3185-3189.

See also. Scalar domain, Backend discovery, Autodiff adapter.

Backend discovery

General meaning. A check that a dependency or implementation can be located.

In this library. The discoverable set is broader than names accepted by the core selector. Successful discovery is not a successful device operation.

Source. Chapter J (explanatory anchor `appendix-j-backends-dependencies`); frozen reference-source Markdown lines 106772-106778.

See also. Backend, Qualification.

Basis witness

General meaning. A finite semantic description of a basis used to interpret typed data.

In this library. Names, dimensions and matrix shape alone do not replace the witness that brands polynomial primitives.

Source. Chapter 50 (explanatory anchor `typed-finite-expression-model`); frozen reference-source Markdown lines 43059-43065.

See also. Coordinate system, Typed expression. **Notation:** NM-057.

Bell digit reduction

General meaning. A finite reduction rule relating generalized Bell data to radix-digit structure.

In this library. The certificate checks the stated factorial-power construction and finite coefficient identity, not every Bell-like sequence.

Source. Chapter 24 (explanatory anchor `prime-power-bell-cyclotomic-and-q-kummer-systems`); frozen reference-source Markdown lines 20578-20582.

See also. Digit kernel, Prime-power kernel.

Best basis

General meaning. A basis selected by optimizing a stated cost over the allowed tree choices.

In this library. Best is relative to the finite candidate family and cost, not an absolute optimal representation for every task.

Source. Chapter 35 (explanatory anchor `filter-banks-wavelets-and-packet-trees`); frozen reference-source Markdown lines 30145-30151.

See also. Wavelet packet tree, Pareto frontier.

Beta API

General meaning. A curated interface still governed by beta-stage compatibility rules.

In this library. Beta is not a weaker kind of arithmetic and is not synonymous with a failed result.

Source. Chapter 7 (explanatory anchor `public-api-tiers`); frozen reference-source Markdown lines 2604-2610.

See also. API tier, Stable API.

Canonical dual

General meaning. The dual synthesis associated with an analysis map using the appropriate inverse or pseudoinverse.

In this library. Rank, tolerance and the selected metric govern the construction. Redundant components obtained from it need not be idempotent.

Source. Chapter 20 (explanatory anchor `analysis-synthesis-and-redundant-frames`); frozen reference-source Markdown lines 15504-15510.

See also. Analysis-synthesis system, Reconstruction component.

Canonical JSON

General meaning. A deterministic JSON serialization used for the specified content digest.

In this library. Canonical byte equality is a representation property. The payload digest does not authenticate producer metadata or prove the encoded mathematics.

Source. Chapter G (explanatory anchor `appendix-g-portable-schemas`); frozen reference-source Markdown lines 95096-95102.

See also. Source checksum, Portable record.

Carry matrix

General meaning. A finite transition matrix tracking carry states while processing radix digits.

In this library. Digit order and the finite state construction are part of the evaluation contract.

Source. Chapter 23 (explanatory anchor `masks-cartier-operators-and-digit-algebra`); frozen reference-source Markdown lines 19324-19330.

See also. Mahler mask, Automatic sequence.

Cartier automaton

General meaning. A finite state machine obtained by closing the relevant Cartier sections.

In this library. Its existence and state budget are tied to the compiler's finite algebraic model, not an arbitrary sequence prefix.

Source. Chapter 23 (explanatory anchor `masks-cartier-operators-and-digit-algebra`); frozen reference-source Markdown lines 19336-19342.

See also. Cartier operator, Automatic sequence.

Cartier operator

General meaning. A residue-section operator that selects coefficients at indices in a fixed radix class.

In this library. Polynomial, rational-mask and finite-field implementations have distinct finite domains and compiler assumptions.

Source. Chapter 23 (explanatory anchor `masks-cartier-operators-and-digit-algebra`); frozen reference-source Markdown lines 19323-19327.

See also. Section operator, Cartier automaton.

Certificate

General meaning. A record describing a claim and the evidence or diagnostics supplied for it.

In this library. The base require method checks the stored validity flag. It does not re-run a proof, a residual test or a witness-producing algorithm. Specialized certificates have different protocols.

Source. Chapter H (explanatory anchor `appendix-h-certificate-theorem-index`); frozen reference-source Markdown lines 96539-96545.

See also. Validity flag, Finite certificate, Provenance.

Certified rewrite

General meaning. A recorded transformation whose relevant finite expression identity is checked under its assumptions.

In this library. A rewrite's evidence is local to the supported types and primitive semantics, not an unrestricted symbolic law.

Source. Chapter 51 (explanatory anchor `expression-evaluation-and-certification`); frozen reference-source Markdown lines 43678-43684.

See also. Normal form, Certificate.

Choi matrix

General meaning. A matrix representation of a linear quantum map under a specified vectorization convention.

In this library. Positivity, trace conditions and reshape ordering are separate parts of the finite channel contract.

Source. Chapter 54 (explanatory anchor `chapter-54.-circuits-channels-and-open-dynamics`); frozen reference-source Markdown lines 44922-44928.

See also. Quantum channel, Kraus operator.

Circular convolution

General meaning. Convolution with periodic index wrap on a fixed finite length.

In this library. It is not equal to unpadded linear convolution at arbitrary boundaries.

Source. Chapter 33 (explanatory anchor `filtering-and-convolution`); frozen reference-source Markdown lines 28430-28436.

See also. Convolution, Polyphase decomposition.

Claim status

General meaning. A taxonomy such as definition, classical input, conditional theorem or finite certificate.

In this library. This taxonomy is not the ordered Exactness ranking used by expression composition.

Source. Chapter 62 (explanatory anchor `chapter-62.-provenance-registries-and-extensions`); frozen reference-source Markdown lines 46342-46348.

See also. Provenance, Exactness.

Codec

General meaning. A registered tag, version and pair of encoding/decoding procedures for a represented object.

In this library. The package version, envelope schema version and codec version are separate identifiers.

Source. Chapter G (explanatory anchor `appendix-g-portable-schemas`); frozen reference-source Markdown lines 95096-95102.

See also. Migration, Portable record.

Coefficient gauge

General meaning. A change of coefficients that does not change the synthesized physical vector.

In this library. Gauge directions lie in the kernel of synthesis. Different coefficient vectors can therefore encode the same physical vector.

Source. Chapter 20 (explanatory anchor `analysis-synthesis-and-redundant-frames`); frozen reference-source Markdown lines 15501-15507.

See also. Coefficient projector, Quotient metric. **Notation:** NM-032.

Coefficient projector

General meaning. The idempotent analysis-after-synthesis map on coefficient space when the left-inverse relation holds.

In this library. It is generally not the full coefficient identity in a redundant presentation.

Source. Chapter 20 (explanatory anchor `analysis-synthesis-and-redundant-frames`); frozen reference-source Markdown lines 15500-15506.

See also. Synthesis map, Coefficient gauge. **Notation:** NM-031.

Coercion

General meaning. Conversion into a selected scalar or array representation.

In this library. Symbolic backends do not automatically rationalize a floating input. Converting a float to Fraction preserves its binary value.

Source. Chapter 8 (explanatory anchor `backends-scalar-domains-and-coercion`); frozen reference-source Markdown lines 3185-3189.

See also. Scalar domain, Exact arithmetic.

Coherence

General meaning. A normalized cross-spectral relationship between two finite signals.

In this library. Interpretation depends on cross-spectrum, averaging and zero-power conventions; it is not causality.

Source. Chapter 32 (explanatory anchor `time-frequency-and-spectral-analysis`); frozen reference-source Markdown lines 28007-28013.

See also. Welch PSD, Residual.

Cohomology

General meaning. Kernel modulo image for consecutive differentials in a declared complex.

In this library. The finite complex must satisfy the differential-composition condition; dimension counts are not arbitrary obstruction labels.

Source. Chapter 56 (explanatory anchor `chapter-56.-finite-deformation-systems`); frozen reference-source Markdown lines 45204-45210.

See also. Finite spectral sequence, Obstruction.

Common refinement

General meaning. A fine decomposition resolving two compatible decompositions at once.

In this library. The supplied construction requires the relevant packet systems to commute. It is not an arbitrary joint refinement of noncommuting projectors.

Source. Chapter 12 (explanatory anchor `refinement-towers`); frozen reference-source Markdown lines 6759-6765.

See also. Packet refinement, Projector.

Commuting diagram

General meaning. Two composable routes with the same endpoints and equal induced maps.

In this library. The atlas compares the actual composed matrices and records their difference. Endpoint equality alone is insufficient.

Source. Chapter 15 (explanatory anchor `coordinate-objects`); frozen reference-source Markdown lines 9414-9420.

See also. Coordinate atlas, Residual.

Completion gate

General meaning. A named required condition in the conditional completion model.

In this library. Supplying a gate name as satisfied records an assumption. It does not prove the missing realization or correspondence theorem.

Source. Chapter 58 (explanatory anchor `chapter-58.-finite-image-ramification-and-completion-models`); frozen reference-source Markdown lines 45602-45606.

See also. Conditional result, Required gate.

Conditional result

General meaning. A conclusion whose use depends on assumptions not all established by the computation.

In this library. The wrapper records the dependency on those assumptions. It is not a proof-admission mechanism.

Source. Chapter 5 (explanatory anchor `certificates-and-result-types`); frozen reference-source Markdown lines 1539-1545.

See also. Assumption, Exactness, Completion gate. **Notation:** NM-015.

Conjugate gradients

General meaning. An iterative linear solver whose standard guarantees require the appropriate positive-definite Hermitian setting.

In this library. Local numerical checks and convergence on a fixture do not certify every operator supplied as a callback.

Source. Chapter 30 (explanatory anchor `krylov-precision-compilation-and-scheduling`); frozen reference-source Markdown lines 26688-26694.

See also. Krylov subspace, Residual.

Convolution

General meaning. A sum combining shifted products under a specified finite or periodic indexing rule.

In this library. Signal routines are signal-first. Full, same, valid and circular modes have different lengths and boundary meanings.

Source. Chapter 33 (explanatory anchor `filtering-and-convolution`); frozen reference-source Markdown lines 28426-28430.

See also. Circular convolution, Hurwitz series.

Coordinate atlas

General meaning. A finite graph of named coordinate systems and registered transformations.

In this library. Route resolution follows registered edges. Commutativity means equality of composed matrices, not merely matching endpoints.

Source. Chapter 15 (explanatory anchor `coordinate-objects`); frozen reference-source Markdown lines 9154-9160.

See also. Coordinate transform, Commuting diagram.

Coordinate covector

General meaning. A linear functional expressed as a row of coefficients dual to a coordinate basis.

In this library. Under coordinate change the covector must transform contragrediently to preserve its pairing with vectors.

Source. Chapter 49 (explanatory anchor `integral-identities-and-coordinate-covectors`); frozen reference-source Markdown lines 42488-42492.

See also. Coordinate vector, Integral identity.

Coordinate system

General meaning. A named finite vector-space presentation with a dimension and basis convention.

In this library. Coordinate-system names and dimensions participate in typed transport. Equal array shapes do not make different named systems identical.

Source. Chapter 15 (explanatory anchor `coordinate-objects`); frozen reference-source Markdown lines 9171-9177.

See also. Coordinate transform, Basis witness.

Coordinate transform

General meaning. A linear change from source coordinates to target coordinates.

In this library. Exact transforms use rational matrices. Composition applies the source transform first and multiplies the following matrix on the left.

Source. Chapter 16 (explanatory anchor `exact-and-numerical-coordinate-transforms`); frozen reference-source Markdown lines 9775-9781.

See also. Coordinate system, Coordinate atlas. **Notation:** NM-011, NM-012.

Coordinate vector

General meaning. Coefficients interpreted in a specified coordinate system.

In this library. The low-level Vector alias is a raw tuple of Fraction values; higher-level objects supply system context. The same tuple in another basis can denote a different vector.

Source. Chapter 15 (explanatory anchor `coordinate-objects`); frozen reference-source Markdown lines 9155-9161.

See also. Coordinate system, Coordinate covector.

Correction

General meaning. A finite adjustment intended to reduce a defect under a specified solver or update rule.

In this library. Successful local correction is not a general convergence or global existence theorem.

Source. Chapter 41 (explanatory anchor `correction-and-elimination`); frozen reference-source Markdown lines 35474-35478.

See also. Defect, Elimination.

Coverage contract

General meaning. A declared inventory and acceptance rule for what a reference promises to document.

In this library. Counts describe named surfaces, not an independent proof of every implementation or a platform qualification.

Source. Chapter 0 (explanatory anchor `coverage-contract`); frozen reference-source Markdown lines 22-26.

See also. Symbol card, Source checksum. **Notation:** NM-061, NM-062.

CRT coordinates

General meaning. Coordinates using residues in a Chinese-remainder decomposition.

In this library. Invertible reconstruction requires the stated coprimality hypotheses. It is not simply digit concatenation.

Source. Chapter 13 (explanatory anchor `residue-crt-lattice-and-smith-coordinates`); frozen reference-source Markdown lines 8022-8028.

See also. Residue coordinate, Mixed-radix address.

Cyclic analytic packet

General meaning. A residue-class decomposition used in the finite cyclic analytic constructions.

In this library. Exact coefficient identities coexist with numerical or conditional zero and product diagnostics.

Source. Chapter 27 (explanatory anchor `cyclic-analytic-systems`); frozen reference-source Markdown lines 24948-24954.

See also. Appell family, Zero certificate.

Cyclic vector

General meaning. A vector whose algebra orbit spans the whole representation.

In this library. Existence and a supplied vector's generation certificate are separate claims.

Source. Chapter 10 (explanatory anchor `operator-algebras-and-seed-systems`); frozen reference-source Markdown lines 4853-4859.

See also. Seed system, Orbit hull.

Cyclotomic coboundary

General meaning. A mask represented as a ratio induced by a cyclotomic factor construction under scaling.

In this library. The finite procedure returns exponents or obstruction orbits. It is not a generic analytic convergence certificate.

Source. Chapter 24 (explanatory anchor `prime-power-bell-cyclotomic-and-q-kummer-systems`); frozen reference-source Markdown lines 20591-20597.

See also. Mahler mask, Obstruction.

Decidable equality

General meaning. Equality determined by a terminating procedure in a specified finite language and domain.

In this library. The expression normalizer's scope is not all mathematical functions, arbitrary callbacks or infinite analytic objects.

Source. Chapter 51 (explanatory anchor `expression-evaluation-and-certification`); frozen reference-source Markdown lines 43544-43550.

See also. Normal form, Alpha equivalence.

Defect

General meaning. A computed discrepancy from a declared compatibility, invariance or lifting identity.

In this library. It can be an exact obstruction signal or a numerical residual. Its class does not automatically diagnose roundoff.

Source. Chapter 40 (explanatory anchor `defects-obstruction-lifting-and-holonomy`); frozen reference-source Markdown lines 34282-34286.

See also. Obstruction, Refinement defect.

Deprecation

General meaning. A compatibility notice indicating that a route should be replaced under a declared policy.

In this library. The generic decorator records metadata and emits a warning. Its remove_in field does not automatically disable a callable at a version boundary.

Source. Appendix L (explanatory anchor `appendix-l-migrations`); frozen reference-source Markdown lines 109991-109997.

See also. Alias, Package upgrade.

Derivative provider

General meaning. An interface supplying finite-order derivative evaluations with declared dimensions and evidence class.

In this library. A callback's declared multilinearity or smoothness is not proved solely by the interface.

Source. Chapter 37 (explanatory anchor `derivative-providers-and-automatic-differentiation`); frozen reference-source Markdown lines 31979-31985.

See also. Autodiff adapter, Packet jet. **Notation:** NM-046.

Digit kernel

General meaning. A local coefficient rule combined across radix digits.

In this library. Lucas and no-borrow behavior are model-dependent and must not be inferred for an arbitrary normalized kernel.

Source. Chapter 23 (explanatory anchor `masks-cartier-operators-and-digit-algebra`); frozen reference-source Markdown lines 19326-19332.

See also. Carry matrix, Kummer valuation.

Document model

General meaning. A typed representation of metadata and an ordered buffer of mathematical document blocks.

In this library. Renderers share this semantic source while producing different text formats. Rendered bytes are not the portable reconstruction format.

Source. Chapter 59 (explanatory anchor `chapter-59.-mathematical-document-model`); frozen reference-source Markdown lines 45895-45901.

See also. Renderer, Portable record. **Notation:** NM-059.

Elimination

General meaning. Removal of selected variables or sectors through a stated algebraic procedure.

In this library. Inverse and rank assumptions control validity, and eliminated dynamics can retain memory.

Source. Chapter 41 (explanatory anchor `correction-and-elimination`); frozen reference-source Markdown lines 35474-35478.

See also. Feshbach reduction, Correction.

Euler coordinates

General meaning. A finite coordinate system for the specified odd-prime or dyadic Euler arithmetic model.

In this library. The applicable prime, window and truncation conditions are part of the reconstruction claim.

Source. Chapter 25 (explanatory anchor `local-arithmetic-models`); frozen reference-source Markdown lines 21820-21826.

See also. Residue coordinate, Truncation.

Exact arithmetic

General meaning. Arithmetic in a declared exact domain rather than tolerance-based floating comparison.

In this library. Rational operations can be exact while a finite prefix, caller hypothesis or certificate producer still limits the conclusion.

Source. Chapter 21 (explanatory anchor `formal-series-and-weighted-bases`); frozen reference-source Markdown lines 16943-16949.

See also. Scalar domain, Formal series, Exactness. **Notation:** NM-002.

Exactness

General meaning. The kind of evidence or arithmetic attributed to a result.

In this library. Exactness is distinct from certificate validity and provenance status. An enum value does not independently validate a computation.

Source. Chapter 5 (explanatory anchor `certificates-and-result-types`); frozen reference-source Markdown lines 1547-1553.

See also. Certificate, Numerical result, Conditional result. **Notation:** NM-013.

Execution plan

General meaning. A finite representation of selected work, addresses and execution options.

In this library. Plan integrity and plan resource estimates are separate from a successful execution or device qualification.

Source. Chapter 43 (explanatory anchor `nonlinear-execution-streaming-profiling-and-gpu-paths`); frozen reference-source Markdown lines 37424-37430.

See also. Resource budget, Packet IR.

Faà di Bruno composition

General meaning. A derivative-composition formula organized by partitions of input directions.

In this library. The finite formula assumes the relevant derivative-provider contracts. Partitions here differ from integer partitions or signal packet trees.

Source. Chapter 38 (explanatory anchor `packet-jets-and-fauxe0-di-bruno-composition`); frozen reference-source Markdown lines 32528-32532.

See also. Set partition, Derivative provider.

Feature registry

General meaning. An index of implemented, conditional or planned domain capabilities.

In this library. A registered name and its status do not supply a proof or guarantee that every optional backend is installed.

Source. Chapter 62 (explanatory anchor `chapter-62.-provenance-registries-and-extensions`); frozen reference-source Markdown lines 46334-46340.

See also. Planned feature, Provenance.

Feshbach reduction

General meaning. Elimination of a complementary sector with an effective operator depending on a resolvent or memory term.

In this library. The required hidden-sector inverse must exist. Reduced evolution is not generally local or Markovian.

Source. Chapter 46 (explanatory anchor `feshbach-reduction-memory-and-survival`); frozen reference-source Markdown lines 40656-40660.

See also. Memory kernel, Elimination.

Filter bank

General meaning. A family of analysis/synthesis filters with explicit sampling and reconstruction structure.

In this library. Perfect reconstruction is a property to check under the selected finite boundary and phase conventions.

Source. Chapter 35 (explanatory anchor `filter-banks-wavelets-and-packet-trees`); frozen reference-source Markdown lines 29632-29636.

See also. Wavelet packet tree, Polyphase decomposition.

Filter state

General meaning. Retained information needed to continue a streaming or recursive filter.

In this library. Restarting with an empty state can change boundary outputs even when filter coefficients are identical.

Source. Chapter 33 (explanatory anchor `filtering-and-convolution`); frozen reference-source Markdown lines 28602-28608.

See also. Convolution, Adaptive filter.

Filtered lift

General meaning. A compatible extension through represented filtration levels.

In this library. Finite level compatibility does not automatically produce an infinite lift.

Source. Chapter 42 (explanatory anchor `filtered-lifting-and-finite-spectral-sequences`); frozen reference-source Markdown lines 36452-36456.

See also. Obstruction, Finite spectral sequence.

Finite certificate

General meaning. Evidence for a specified finite calculation under stated assumptions.

In this library. Its scope is the checked dimensions, inputs, domains and predicates. It is not automatically an infinite theorem or a Lean proof.

Source. Chapter 5 (explanatory anchor `certificates-and-result-types`); frozen reference-source Markdown lines 1553-1559.

See also. Certificate, Exactness, Assumption.

Finite field

General meaning. A field with finitely many elements represented through a prime field and an extension construction.

In this library. The supplied modulus polynomial, coefficient ordering and irreducibility conditions govern valid arithmetic.

Source. Chapter 57 (explanatory anchor `chapter-57.-local-finite-langlands-packets`); frozen reference-source Markdown lines 45436-45442.

See also. Prime field, Frobenius orbit. **Notation:** NM-056.

Finite Heisenberg model

General meaning. A finite algebraic representation capturing the declared Heisenberg commutation structure.

In this library. The model supplies finite matrices and diagnostics, not the local Langlands correspondence.

Source. Chapter 57 (explanatory anchor `chapter-57.-local-finite-langlands-packets`); frozen reference-source Markdown lines 45430-45436.

See also. Symmetry packet, Completion gate.

Finite lift forest

General meaning. Finite candidate levels connected by explicit adjacent reduction maps.

In this library. A surviving branch is compatible through represented levels only. It is not an automatically proved infinite inverse-limit branch.

Source. Chapter 56 (explanatory anchor `chapter-56.-finite-deformation-systems`); frozen reference-source Markdown lines 45209-45215.

See also. Stable core, Filtered lift.

Finite measure

General meaning. A measure model explicitly represented on a finite domain or by specified finite data.

In this library. Exact integration claims depend on the measure normalization and integrand representation.

Source. Chapter 47 (explanatory anchor `measures-and-exact-polynomials`); frozen reference-source Markdown lines 41459-41463.

See also. Integration provider, Moment.

Finite spectral sequence

General meaning. Page-wise algebraic bookkeeping for a represented finite filtered complex.

In this library. Finite computed pages and differentials are not a claim of unbounded convergence beyond the supplied model.

Source. Chapter 42 (explanatory anchor `filtered-lifting-and-finite-spectral-sequences`); frozen reference-source Markdown lines 36452-36456.

See also. Cohomology, Filtered lift.

Fitting fiber

General meaning. A finite support/fiber diagnostic constructed from the declared presentation data.

In this library. It is algebraic support information, not automatically a Hilbert-space spectral projector.

Source. Chapter 56 (explanatory anchor `chapter-56.-finite-deformation-systems`); frozen reference-source Markdown lines 45306-45312.

See also. Cohomology, Symmetry packet.

Formal series

General meaning. A coefficient sequence treated algebraically without an analytic convergence requirement.

In this library. The package stores a finite prefix in a declared scalar domain and normalized basis. A prefix is not an infinite lazy series.

Source. Chapter 21 (explanatory anchor `formal-series-and-weighted-bases`); frozen reference-source Markdown lines 16908-16914.

See also. Weighted monomial basis, Truncation.

Frame

General meaning. A finite windowed segment in the signal-framing sense.

In this library. This is not the redundant linear-algebra meaning of a frame. Arrays place frame number and within-frame sample on separate axes.

Source. Chapter 31 (explanatory anchor `signal-conventions-windows-and-framing`); frozen reference-source Markdown lines 27208-27214.

See also. Analysis-synthesis system, Window, STFT.

Frame operator

General meaning. In conventional frame notation the analysis Gram action on physical space is often written as the adjoint-analysis product.

In this library. The frozen package method instead returns synthesis times analysis, the physical reconstruction operator. Do not replace it blindly by the conventional expression.

Source. Chapter 20 (explanatory anchor `analysis-synthesis-and-redundant-frames`); frozen reference-source Markdown lines 15580-15586.

See also. Analysis-synthesis system, Adjoint.

Frobenius norm

General meaning. The square root of the sum of squared absolute matrix entries.

In this library. It is one specific residual norm, distinct from an arithmetic Frobenius map or an operator spectral norm.

Source. Chapter 20 (explanatory anchor `analysis-synthesis-and-redundant-frames`); frozen reference-source Markdown lines 15615-15621.

See also. Frobenius orbit, Residual. **Notation:** NM-033.

Frobenius orbit

General meaning. An orbit under the characteristic-prime power map in a finite-field model.

In this library. Frobenius here is an arithmetic map, not the Frobenius matrix norm.

Source. Chapter 57 (explanatory anchor `chapter-57.-local-finite-langlands-packets`); frozen reference-source Markdown lines 45429-45435.

See also. Finite field, Frobenius norm.

Frozen record

General meaning. A finite data capture replacing an executable or evolving source with a portable representation.

In this library. Freezing removes reliance on a live provider where supported. Frozen dataclass syntax alone does not guarantee deep immutability of nested dictionaries.

Source. Chapter 22 (explanatory anchor `normalized-generating-functions`); frozen reference-source Markdown lines 17860-17866.

See also. Snapshot, Portable record.

Fundamental matrix

General meaning. A matrix-valued evolution operator for a declared finite linear differential system.

In this library. Numerical discretization, exact model identities and user-supplied generator assumptions remain distinct.

Source. Chapter 27 (explanatory anchor `cyclic-analytic-systems`); frozen reference-source Markdown lines 25201-25207.

See also. Matrix dynamics, Backend.

Generated operator algebra

General meaning. The unital matrix algebra generated by a finite family of square operators.

In this library. The implementation builds a finite span. Numerical independence depends on tolerance, whereas the exact backend uses exact decisions.

Source. Chapter 10 (explanatory anchor `operator-algebras-and-seed-systems`); frozen reference-source Markdown lines 4724-4730.

See also. Seed system, Orbit hull. **Notation:** NM-022.

Generating provider

General meaning. A rule or finite specification supplying coefficients for a generating function.

In this library. A mathematical infinite definition is executed only through a requested finite prefix. Opaque callbacks are not automatically portable.

Source. Chapter 22 (explanatory anchor `normalized-generating-functions`); frozen reference-source Markdown lines 17857-17863.

See also. Formal series, Frozen record.

Heuristic

General meaning. A discovery method or estimate without the complete guarantee of the stronger claimed property.

In this library. A good finite fit or small residual alone does not promote the label to an exact or global theorem.

Source. Chapter 1 (explanatory anchor `editorial-and-symbolic-grammar`); frozen reference-source Markdown lines 240-246.

See also. Planned feature, Finite certificate. **Notation:** NM-016.

Holonomic recurrence

General meaning. A linear recurrence with polynomial coefficients in the index.

In this library. A vanishing leading coefficient can block forward evaluation at an index. Initial data and singular steps must be inspected.

Source. Chapter 26 (explanatory anchor `exact-sequence-execution`); frozen reference-source Markdown lines 23466-23472.

See also. Linear recurrence, Sequence model.

Holonomy

General meaning. The net transport obtained around a closed route.

In this library. Trivial endpoints alone do not make the composed transport the identity. Finite holonomy diagnostics require the actual route maps.

Source. Chapter 40 (explanatory anchor `defects-obstruction-lifting-and-holonomy`); frozen reference-source Markdown lines 34282-34286.

See also. Coordinate atlas, Commuting diagram.

Hurwitz series

General meaning. A coefficient algebra using the binomially weighted product associated with exponential normalization.

In this library. Finite truncation and scalar-domain assumptions remain explicit. Its product is not ordinary unweighted convolution on stored coefficients.

Source. Chapter 25 (explanatory anchor `local-arithmetic-models`); frozen reference-source Markdown lines 21845-21851.

See also. Normalized coefficient, Convolution. **Notation:** NM-037.

Integral identity

General meaning. An equality between specified integral or functional expressions.

In this library. The certificate records the finite basis and measure conditions actually checked, not an unrestricted analytic identity.

Source. Chapter 49 (explanatory anchor `integral-identities-and-coordinate-covectors`); frozen reference-source Markdown lines 42489-42495.

See also. Coordinate covector, Integration provider.

Integration provider

General meaning. An interface supplying an integral and its stated evidence guarantee.

In this library. Exact finite polynomial integration, numerical quadrature and conditional user providers are different contracts.

Source. Chapter 48 (explanatory anchor `integration-providers-and-quadrature`); frozen reference-source Markdown lines 41957-41961.

See also. Quadrature, Coordinate covector.

Inverse STFT

General meaning. Reconstruction from compatible time-frequency frames.

In this library. Inversion relies on the recorded framing/window/boundary conventions and overlap support, not just matching array dimensions.

Source. Chapter 32 (explanatory anchor `time-frequency-and-spectral-analysis`); frozen reference-source Markdown lines 27660-27666.

See also. STFT, Overlap-add.

Jet address

General meaning. One output label together with an ordered tuple of input packet labels.

In this library. Derivative order is the input tuple length. Output-major and input-lexicographic ordering are part of the record contract.

Source. Chapter 38 (explanatory anchor `packet-jets-and-fauxe0-di-bruno-composition`); frozen reference-source Markdown lines 32540-32546.

See also. Packet jet, Multi-index. **Notation:** NM-047, NM-048.

Jet support

General meaning. A selection or diagnostic of packet-jet addresses retained as active.

In this library. Numerical support and a complete exact address enumeration have different evidence. Omitting a small block is not exact vanishing.

Source. Chapter 38 (explanatory anchor `packet-jets-and-fauxe0-di-bruno-composition`); frozen reference-source Markdown lines 32540-32546.

See also. Packet jet, Transfer graph.

Kato transport

General meaning. Transport adapted to moving projector sectors.

In this library. The finite implementation uses the specified generator and numerical or exact path assumptions, not a universal adiabatic theorem.

Source. Chapter 54 (explanatory anchor `chapter-54.-circuits-channels-and-open-dynamics`); frozen reference-source Markdown lines 44832-44838.

See also. Smooth packet transport, Orthogonal projector.

Knill-Laflamme condition

General meaning. The condition that each compressed error-product operator acts as a scalar on the code subspace.

In this library. Numerical satisfaction is relative to the declared tolerance and supplied error set. It is not protection against unspecified errors.

Source. Chapter 55 (explanatory anchor `chapter-55.-error-correction-and-effective-dynamics`); frozen reference-source Markdown lines 45083-45089.

See also. Quantum code, Orthogonal projector.

Kraus operator

General meaning. One operator in an operator-sum representation of a quantum map.

In this library. A supplied error family or arbitrary operator list need not already define a trace-preserving channel.

Source. Chapter 54 (explanatory anchor `chapter-54.-circuits-channels-and-open-dynamics`); frozen reference-source Markdown lines 44911-44917.

See also. Quantum channel, Quantum code.

Kronecker product

General meaning. The matrix representation of a tensor product in a specified factor ordering.

In this library. Apply cost and storage depend on the structured path versus dense materialization.

Source. Chapter 29 (explanatory anchor `structured-and-low-rank-operators`); frozen reference-source Markdown lines 26422-26428.

See also. Tensor coordinate transport, Structured operator.

Krylov subspace

General meaning. The span of a starting vector and its successive images under an operator.

In this library. Finite callable probes do not prove global linearity, Hermitian symmetry or positive definiteness.

Source. Chapter 30 (explanatory anchor `krylov-precision-compilation-and-scheduling`); frozen reference-source Markdown lines 26697-26703.

See also. Conjugate gradients, Seed system. **Notation:** NM-039.

Kummer valuation

General meaning. A prime-adic valuation of a combinatorial coefficient expressed through carry or digit data.

In this library. Valuation and unit part are separate. The implemented branch and modulus determine the recovered residue.

Source. Chapter 24 (explanatory anchor `prime-power-bell-cyclotomic-and-q-kummer-systems`); frozen reference-source Markdown lines 20579-20585.

See also. Unit part, Prime-power kernel.

Lane

General meaning. A residue-indexed channel in an arithmetic or coefficient decomposition.

In this library. It is a mathematical indexing convention here, not a CPU SIMD lane or an independently learned latent unit.

Source. Chapter 23 (explanatory anchor `masks-cartier-operators-and-digit-algebra`); frozen reference-source Markdown lines 19325-19331.

See also. Residue coordinate, Polyphase decomposition.

Lattice quotient

General meaning. Cosets of an integer lattice modulo a declared sublattice.

In this library. Finite address counts depend on the lattice index and the nonsingularity hypotheses used by the constructor.

Source. Chapter 13 (explanatory anchor `residue-crt-lattice-and-smith-coordinates`); frozen reference-source Markdown lines 8019-8023.

See also. Smith coordinates, Residue coordinate.

Leakage

General meaning. Transfer from a chosen sector into its complement.

In this library. The norm and packet selection define the diagnostic. Small finite leakage is not exact invariant-subspace closure.

Source. Chapter 53 (explanatory anchor `chapter-53.-finite-quantum-foundations`); frozen reference-source Markdown lines 44601-44607.

See also. Survival operator, Transfer block.

Lexical binding

General meaning. Association of a variable occurrence with its enclosing binder.

In this library. Binding scope matters to evaluation and equality; matching displayed names is insufficient.

Source. Chapter 50 (explanatory anchor `typed-finite-expression-model`); frozen reference-source Markdown lines 43257-43263.

See also. Alpha equivalence, Typed expression.

Lindblad generator

General meaning. A generator of the stated finite open-system evolution with Hamiltonian and dissipative terms.

In this library. A finite matrix construction does not assert all infinite-dimensional semigroup hypotheses.

Source. Chapter 54 (explanatory anchor `chapter-54.-circuits-channels-and-open-dynamics`); frozen reference-source Markdown lines 44832-44838.

See also. Matrix dynamics, Quantum channel.

Linear recurrence

General meaning. A relation determining future terms from a finite set of predecessors.

In this library. Declared constant recurrences and polynomial-coefficient holonomic recurrences have different execution and singularity conditions.

Source. Chapter 26 (explanatory anchor `exact-sequence-execution`); frozen reference-source Markdown lines 23432-23438.

See also. Holonomic recurrence, Sequence model.

Low-rank factorization

General meaning. Representation of a matrix as a product through a lower-dimensional intermediate space.

In this library. Numerical recompression uses a declared approximation criterion and is not an exact rank theorem for noisy data.

Source. Chapter 29 (explanatory anchor `structured-and-low-rank-operators`); frozen reference-source Markdown lines 26372-26376.

See also. Structured operator, Residual.

Mahler mask

General meaning. A finite mask defining a radix scale equation for a generating function.

In this library. Coefficients are interpreted through that scale equation, not automatically through an ordinary rational function with the same finite data.

Source. Chapter 23 (explanatory anchor `masks-cartier-operators-and-digit-algebra`); frozen reference-source Markdown lines 19324-19330.

See also. Carry matrix, Cartier operator.

Matrix dynamics

General meaning. Finite evolution specified by a matrix generator or sequence of matrices.

In this library. Semigroup, Markov, Bayesian and related finite models have different assumptions and backend paths.

Source. Chapter 45 (explanatory anchor `matrix-dynamics`); frozen reference-source Markdown lines 39356-39360.

See also. Fundamental matrix, Feshbach reduction. **Notation:** NM-052.

Matrix-chain plan

General meaning. A parenthesization chosen to optimize a stated dense matrix-composition cost.

In this library. Equal mathematical products can have different finite arithmetic costs. The plan digest is distinct from the expression's semantic digest.

Source. Chapter 52 (explanatory anchor `expression-planning-and-persistence`); frozen reference-source Markdown lines 44004-44010.

See also. Execution plan, Canonical JSON.

Memory estimate

General meaning. A model of storage for a specified representation.

In this library. Array nbytes omits referenced object payloads and some temporaries. Traced allocations are not total resident or GPU memory.

Source. Chapter K (explanatory anchor `appendix-k-resource-budgets`); frozen reference-source Markdown lines 108451-108457.

See also. Resource budget, Exact arithmetic. **Notation:** NM-066.

Memory kernel

General meaning. A kernel carrying feedback from eliminated history into the retained sector.

In this library. Nonlocal memory is an expected structural outcome, not a solver error to relabel as Markovian closure.

Source. Chapter 46 (explanatory anchor `feshbach-reduction-memory-and-survival`); frozen reference-source Markdown lines 40656-40660.

See also. Feshbach reduction, Survival operator.

Migration

General meaning. A declared transformation between adjacent versions of a serialized representation.

In this library. The built-in snapshot edge is one concrete migration. No generic downgrade or arbitrary future-version support is implied.

Source. Appendix L (explanatory anchor `appendix-l-migrations`); frozen reference-source Markdown lines 110024-110030.

See also. Codec, Package upgrade.

Mixed-radix address

General meaning. A positional address whose digit places have specified possibly different radices.

In this library. Radix order and residue conventions are part of the coordinate contract. It is not interchangeable with CRT reconstruction.

Source. Chapter 13 (explanatory anchor `residue-crt-lattice-and-smith-coordinates`); frozen reference-source Markdown lines 8021-8027.

See also. CRT coordinates, Lane. **Notation:** NM-027.

Moment

General meaning. An integral of a declared basis monomial or polynomial against a specified measure.

In this library. A finite list of moments is not automatically a uniquely determined measure or all-order reconstruction.

Source. Chapter 47 (explanatory anchor `measures-and-exact-polynomials`); frozen reference-source Markdown lines 41461-41467.

See also. Finite measure, Quadrature.

Monoid series

General meaning. A finite coefficient object whose multiplication follows the declared index monoid.

In this library. Changing the monoid changes the convolution law. It is not merely another name for ordinary power-series multiplication.

Source. Chapter 22 (explanatory anchor `normalized-generating-functions`); frozen reference-source Markdown lines 17858-17864.

See also. Formal series, Convolution.

Multi-index

General meaning. A finite tuple indexing several positions, directions or powers.

In this library. Ordered jet packet labels are not automatically commuting polynomial exponents. Read the local address interpretation.

Source. Chapter 38 (explanatory anchor `packet-jets-and-fauxe0-di-bruno-composition`); frozen reference-source Markdown lines 32546-32552.

See also. Jet address, Set partition.

Normal form

General meaning. A canonical evaluated representation used to compare the supported finite expressions.

In this library. Closed value terms normalize to exact vectors and map terms to exact matrices under the declared type semantics.

Source. Chapter 51 (explanatory anchor `expression-evaluation-and-certification`); frozen reference-source Markdown lines 43518-43524.

See also. Decidable equality, Typed expression. **Notation:** NM-058.

Normalized coefficient

General meaning. A coefficient measured relative to a chosen normalized basis.

In this library. With basis z^n/gamma_n the ordinary coefficient is a_n/gamma_n. Changing normalization changes stored values, not just display.

Source. Chapter 22 (explanatory anchor `normalized-generating-functions`); frozen reference-source Markdown lines 17856-17862.

See also. Weighted monomial basis, Hurwitz series. **Notation:** NM-036.

Numerical result

General meaning. A result obtained through floating computation and interpreted using a declared error model.

In this library. Residuals and tolerances must be read explicitly. Decreasing a tolerance does not convert floating work into exact arithmetic.

Source. Chapter 5 (explanatory anchor `certificates-and-result-types`); frozen reference-source Markdown lines 1554-1560.

See also. Residual, Tolerance, Exact arithmetic. **Notation:** NM-003, NM-014.

Observable

General meaning. An operator representing a measured quantity in a specified quantum model.

In this library. The foundation uses finite matrices with explicit Hermiticity and algebra hypotheses. It does not infer an infinite physical theory.

Source. Chapter 53 (explanatory anchor `chapter-53.-finite-quantum-foundations`); frozen reference-source Markdown lines 44599-44605.

See also. Quantum channel, Symmetry packet. **Notation:** NM-053.

Obstruction

General meaning. A quantity or class preventing a proposed lift or compatibility condition from being satisfied.

In this library. Vanishing of the checked finite obstruction is only as strong as the model's stated sufficiency assumptions.

Source. Chapter 40 (explanatory anchor `defects-obstruction-lifting-and-holonomy`); frozen reference-source Markdown lines 34282-34286.

See also. Defect, Filtered lift.

Operator conjugation

General meaning. Transport of an operator into another coordinate presentation.

In this library. For one change of basis the transformed matrix uses the change and its inverse in the documented order. Source and target changes may differ.

Source. Chapter 19 (explanatory anchor `spectral-packet-tensor-and-operator-transport`); frozen reference-source Markdown lines 14173-14179.

See also. Coordinate transform, Transfer block.

Operator grammar

General meaning. A typed directed multigraph with declared linear relations among parallel paths.

In this library. It describes allowed syntax. A separate realization determines whether the relations hold for actual matrices.

Source. Chapter 9 (explanatory anchor `typed-operator-grammars`); frozen reference-source Markdown lines 3823-3829.

See also. Operator realization, Operator relation. **Notation:** NM-021.

Operator realization

General meaning. Assignment of finite spaces and matrices to a typed grammar.

In this library. Relation residuals are checked in the chosen backend. A declaration and a verified realization are different objects.

Source. Chapter 9 (explanatory anchor `typed-operator-grammars`); frozen reference-source Markdown lines 3792-3798.

See also. Operator grammar, Backend.

Operator relation

General meaning. A finite linear combination of parallel paths declared equal to zero.

In this library. Equal endpoints are a type requirement, not evidence that a chosen realization satisfies the relation.

Source. Chapter 9 (explanatory anchor `typed-operator-grammars`); frozen reference-source Markdown lines 3825-3831.

See also. Operator word, Residual.

Operator word

General meaning. A composable finite path of operator symbols.

In this library. Its written/execution order must be distinguished from the right-to-left order in the resulting matrix product.

Source. Chapter 9 (explanatory anchor `typed-operator-grammars`); frozen reference-source Markdown lines 3826-3832.

See also. Arrow, Operator relation, Operator realization. **Notation:** NM-065.

Orbit hull

General meaning. The span of vectors reachable by applying the algebra to supplied seeds.

In this library. Its dimension is bounded by the ambient finite vector-space dimension. Numerical rank is tolerance-dependent.

Source. Chapter 10 (explanatory anchor `operator-algebras-and-seed-systems`); frozen reference-source Markdown lines 4728-4734.

See also. Seed system, Generated operator algebra.

Orthogonal projector

General meaning. An idempotent that is also self-adjoint for the stated inner product.

In this library. Algebraic annihilation of distinct packets is not by itself Hilbert orthogonality. The pinned hilbert check has a validation-bypass boundary documented in H.

Source. Chapter 38 (explanatory anchor `packet-jets-and-fauxe0-di-bruno-composition`); frozen reference-source Markdown lines 32538-32544.

See also. Projector, Adjoint, Packet system.

Overlap-add

General meaning. Reconstruction by adding shifted, weighted finite frames.

In this library. Covered samples and the analysis/synthesis normalization must satisfy the stated nonzero overlap conditions.

Source. Chapter 31 (explanatory anchor `signal-conventions-windows-and-framing`); frozen reference-source Markdown lines 27219-27225.

See also. Window, STFT.

Package upgrade

General meaning. Replacement of an installed distribution with a later tested artifact.

In this library. Passing finite predecessor fixtures does not establish every historical behavior, dependency combination or downgrade path.

Source. Appendix L (explanatory anchor `appendix-l-migrations`); frozen reference-source Markdown lines 109676-109682.

See also. Migration, Qualification.

Packet coordinates

General meaning. A vector represented by its projected packet components.

In this library. General packet components can remain full ambient vectors. They are not automatically compressed scalar coefficients.

Source. Chapter 11 (explanatory anchor `packet-systems-and-transfer-blocks`); frozen reference-source Markdown lines 5651-5657.

See also. Packet system, Coordinate system.

Packet IR

General meaning. An intermediate representation of a finite operator through addressed blocks.

In this library. Layouts, support and execution schedules are data for the finite computation, not proofs of a universal complexity advantage.

Source. Chapter 28 (explanatory anchor `packet-intermediate-representation`); frozen reference-source Markdown lines 26037-26043.

See also. Packet layout, Structured operator.

Packet jet

General meaning. An output-projected multilinear derivative with each input direction projected into a named packet.

In this library. Finite jet reconstruction requires complete packet decompositions and the provider hypotheses. It does not establish convergence of an infinite Taylor series.

Source. Chapter 38 (explanatory anchor `packet-jets-and-fauxe0-di-bruno-composition`); frozen reference-source Markdown lines 32540-32546.

See also. Jet address, Jet support, Taylor truncation. **Notation:** NM-049.

Packet layout

General meaning. A finite division of coordinates into named contiguous blocks with explicit sizes.

In this library. It is an array layout, not an arbitrary family of ambient projectors. Target/source block order remains explicit.

Source. Chapter 28 (explanatory anchor `packet-intermediate-representation`); frozen reference-source Markdown lines 26051-26057.

See also. Packet IR, Packet system.

Packet operator

General meaning. An operator represented through all compatible packet transfer blocks.

In this library. Reconstruction and composition use the complete source and target decompositions under their checked hypotheses.

Source. Chapter 11 (explanatory anchor `packet-systems-and-transfer-blocks`); frozen reference-source Markdown lines 5653-5659.

See also. Transfer block, Packet coordinates.

Packet refinement

General meaning. A fine packet system with a parent map whose children sum to each coarse packet.

In this library. Both levels act on the same ambient space. Refinement is not resampling or an increase in vector dimension.

Source. Chapter 12 (explanatory anchor `refinement-towers`); frozen reference-source Markdown lines 6756-6762.

See also. Refinement tower, Common refinement. **Notation:** NM-024, NM-025.

Packet system

General meaning. A finite complete family of pairwise-annihilating idempotent maps.

In this library. General packets need not be self-adjoint. Hilbert orthogonality is a separate condition, and bypassing construction validation leaves prerequisites unchecked.

Source. Chapter 11 (explanatory anchor `packet-systems-and-transfer-blocks`); frozen reference-source Markdown lines 5654-5660.

See also. Projector, Orthogonal projector, Transfer block. **Notation:** NM-009.

Pareto frontier

General meaning. Nondominated choices under several explicit optimization criteria.

In this library. Nondominance is relative to the candidates, costs and constraints actually enumerated.

Source. Chapter 30 (explanatory anchor `krylov-precision-compilation-and-scheduling`); frozen reference-source Markdown lines 26961-26967.

See also. Precision allocation, Execution plan.

Periodogram

General meaning. A finite spectral estimator obtained from a transformed signal and normalization.

In this library. Spectrum versus density scaling and one-sided doubling are explicit choices.

Source. Chapter 32 (explanatory anchor `time-frequency-and-spectral-analysis`); frozen reference-source Markdown lines 27890-27896.

See also. Welch PSD, Sample rate.

Planned feature

General meaning. A named but unimplemented or reserved capability.

In this library. Registry availability and a documented name must not be mistaken for an executable implementation.

Source. Chapter 6 (explanatory anchor `errors-assumptions-and-resource-guards`); frozen reference-source Markdown lines 2150-2156.

See also. Feature registry, Heuristic. **Notation:** NM-017.

Polynomial basis

General meaning. A specified ordered basis for a finite polynomial space.

In this library. Coefficients are meaningful only with that basis and truncation degree. Classical family parameters form part of the contract.

Source. Chapter 17 (explanatory anchor `polynomial-and-classical-coordinate-families`); frozen reference-source Markdown lines 10819-10823.

See also. Normalized coefficient, Basis witness.

Polyphase decomposition

General meaning. Separation of a finite sequence into residue-indexed phases.

In this library. Phase ordering, lengths and synthesis conventions are explicit. Exact reconstruction of phases does not prove an infinite filter identity.

Source. Chapter 14 (explanatory anchor `polyphase-systems-and-refinement`); frozen reference-source Markdown lines 8703-8707.

See also. Lane, Polyphase refinement.

Polyphase refinement

General meaning. Refinement of residue phases into finer phases with a declared reconstruction map.

In this library. This arithmetic address refinement is distinct from changing a signal's sample rate.

Source. Chapter 14 (explanatory anchor `polyphase-systems-and-refinement`); frozen reference-source Markdown lines 8805-8811.

See also. Polyphase decomposition, Resampling.

Portable record

General meaning. A finite data representation decoded through the admitted structural grammar and registered codecs.

In this library. Decoding reconstructs stored information, not an independently verified mathematical witness. Decoder strictness varies by codec.

Source. Chapter G (explanatory anchor `appendix-g-portable-schemas`); frozen reference-source Markdown lines 95098-95104.

See also. Codec, Certificate, Canonical JSON.

Power spectral density

General meaning. A spectral power representation normalized per unit of frequency.

In this library. It differs from a raw squared FFT or power spectrum. Sample rate, window normalization and one-sided scaling are part of the finite estimator.

Source. Chapter 32 (explanatory anchor `time-frequency-and-spectral-analysis`); frozen reference-source Markdown lines 27848-27854.

See also. Periodogram, Welch PSD, Sample rate.

Precision allocation

General meaning. Selection of computation precision under a stated error-and-cost model.

In this library. A model's bound and a measured machine runtime are different quantities.

Source. Chapter 30 (explanatory anchor `krylov-precision-compilation-and-scheduling`); frozen reference-source Markdown lines 26683-26687.

See also. Resource budget, Pareto frontier.

Prime field

General meaning. The field of residues modulo a prime.

In this library. The constructor's prime validation is part of the contract. A residue-ring instance with arbitrary modulus is not interchangeable.

Source. Chapter 21 (explanatory anchor `formal-series-and-weighted-bases`); frozen reference-source Markdown lines 16932-16938.

See also. Residue ring, Finite field. **Notation:** NM-005.

Prime-power kernel

General meaning. A finite combinatorial coefficient rule evaluated modulo a declared prime power.

In this library. The compiler records valuation, unit, residue and support. It does not construct a complete p-adic field.

Source. Chapter 24 (explanatory anchor `prime-power-bell-cyclotomic-and-q-kummer-systems`); frozen reference-source Markdown lines 20578-20582.

See also. Kummer valuation, Residue ring. **Notation:** NM-038.

Projector

General meaning. An endomorphism satisfying its idempotence identity in the local scalar domain.

In this library. The guide reserves the word for idempotent maps. A redundant reconstruction component is not a projector merely because its components sum to identity.

Source. Chapter 1 (explanatory anchor `editorial-and-symbolic-grammar`); frozen reference-source Markdown lines 356-362.

See also. Packet system, Reconstruction component. **Notation:** NM-008.

Provenance

General meaning. Information about the origin and declared classification of a mathematical claim.

In this library. Qualified-name metadata and author-supplied claim status are not a fresh proof audit.

Source. Chapter 62 (explanatory anchor `chapter-62.-provenance-registries-and-extensions`); frozen reference-source Markdown lines 46334-46338.

See also. Claim status, Qualified name, Source checksum.

q-Kummer decomposition

General meaning. A branch-sensitive valuation and unit analysis for Gaussian-binomial data.

In this library. Ordinary prime valuation, cyclotomic order and the value of q must be distinguished.

Source. Chapter 24 (explanatory anchor `prime-power-bell-cyclotomic-and-q-kummer-systems`); frozen reference-source Markdown lines 20578-20582.

See also. Kummer valuation, Unit part.

Quadrature

General meaning. A weighted finite evaluation rule approximating or exactly integrating a stated function class.

In this library. Polynomial exactness degree is not exactness for arbitrary functions, and a numerical rule needs its error assumptions.

Source. Chapter 48 (explanatory anchor `integration-providers-and-quadrature`); frozen reference-source Markdown lines 41957-41961.

See also. Integration provider, Moment.

Qualification

General meaning. Acceptance against an explicitly named set of release or platform gates.

In this library. An aggregate pass can depend on which gates were required. It must be read with coverage and environment, not as universal support.

Source. Chapter 73 (explanatory anchor `platform-and-release-qualification`); frozen reference-source Markdown lines 47793-47797.

See also. Backend, Verification suite, Required gate.

Qualified name

General meaning. An object's name together with the namespace that identifies its route.

In this library. Reused theorem identifiers need qualified owner and source identity for disambiguation. Import aliases can refer to one implementation.

Source. Chapter 62 (explanatory anchor `chapter-62.-provenance-registries-and-extensions`); frozen reference-source Markdown lines 46335-46341.

See also. Provenance, Alias, Theorem identifier.

Quantum channel

General meaning. A linear map on operators satisfying the required positivity and normalization conditions in the intended model.

In this library. Kraus/Choi representations and the finite checks must be interpreted with their conventions and tolerances.

Source. Chapter 54 (explanatory anchor `chapter-54.-circuits-channels-and-open-dynamics`); frozen reference-source Markdown lines 44831-44837.

See also. Choi matrix, Kraus operator. **Notation:** NM-054.

Quantum code

General meaning. A code subspace together with a declared finite error family.

In this library. The constructor validates the projector and errors, while correctability is checked by the separate finite criterion.

Source. Chapter 55 (explanatory anchor `chapter-55.-error-correction-and-effective-dynamics`); frozen reference-source Markdown lines 45069-45075.

See also. Knill-Laflamme condition, Kraus operator. **Notation:** NM-055.

Quotient metric

General meaning. A metric interpreted on coefficient classes after identifying gauge-equivalent representatives.

In this library. A coefficient-space expression is not automatically a positive definite metric on all redundant coordinates.

Source. Chapter 20 (explanatory anchor `analysis-synthesis-and-redundant-frames`); frozen reference-source Markdown lines 15507-15513.

See also. Coefficient gauge, Analysis-synthesis system.

Ramification tower

General meaning. A represented sequence of local ramification data and compatibility records.

In this library. External descent, phase or realization assumptions remain explicit where not proved by the finite construction.

Source. Chapter 58 (explanatory anchor `chapter-58.-finite-image-ramification-and-completion-models`); frozen reference-source Markdown lines 45602-45606.

See also. Conditional result, Completion gate.

Reconstruction component

General meaning. One contribution to a physical reconstruction through selected coefficient coordinates.

In this library. Components sum to the reconstruction operator, but individual redundant components need not square to themselves.

Source. Chapter 20 (explanatory anchor `analysis-synthesis-and-redundant-frames`); frozen reference-source Markdown lines 15792-15798.

See also. Projector, Canonical dual. **Notation:** NM-010.

Recovery

General meaning. Reconstruction of missing or transformed information under a stated finite model.

In this library. Identifiability assumptions and failure diagnostics remain necessary. A returned candidate need not imply unique recovery.

Source. Chapter 36 (explanatory anchor `singular-packets-and-recovery`); frozen reference-source Markdown lines 30764-30768.

See also. Singular packet, Residual.

Recurrence coordinates

General meaning. Coordinates built from a finite recurrence or state-space realization.

In this library. Recurrence order, seeds and reconstruction map must match. This is not a proof of an empirically fitted global sequence rule.

Source. Chapter 18 (explanatory anchor `combinatorial-recurrence-and-state-space-coordinates`); frozen reference-source Markdown lines 12605-12609.

See also. Linear recurrence, State-space realization.

Refinement defect

General meaning. A discrepancy measuring failure of an operator relation across a refinement map.

In this library. It is not the same as numerical roundoff: a nonzero exact defect can record a genuine obstruction.

Source. Chapter 12 (explanatory anchor `refinement-towers`); frozen reference-source Markdown lines 6761-6767.

See also. Defect, Packet refinement.

Refinement tower

General meaning. A finite chain of compatible adjacent packet refinements.

In this library. Stored finite levels do not establish the existence or convergence of an infinite inverse limit.

Source. Chapter 12 (explanatory anchor `refinement-towers`); frozen reference-source Markdown lines 6758-6764.

See also. Packet refinement, Finite lift forest.

Renderer

General meaning. A transformation of a document model into presentation text.

In this library. Rendering is distinct from a writer's path, overwrite and asset policy, and from mathematical object deserialization.

Source. Chapter 60 (explanatory anchor `chapter-60.-renderers-and-writers`); frozen reference-source Markdown lines 46031-46035.

See also. Document model, Portable record.

Reproducibility

General meaning. The ability to repeat a specified construction from recorded inputs and an environment.

In this library. Byte reproduction, semantic replay, package isolation and dependency isolation are distinct claims.

Source. Chapter 70 (explanatory anchor `chapter-70.-export-and-reproducibility-recipes`); frozen reference-source Markdown lines 47369-47373.

See also. Source checksum, Qualification.

Required gate

General meaning. An acceptance condition selected as mandatory by a qualification profile.

In this library. An empty required set can produce a favorable aggregate with no successful GPU execution. Inspect the actual backend records.

Source. Chapter J (explanatory anchor `appendix-j-backends-dependencies`); frozen reference-source Markdown lines 106481-106487.

See also. Qualification, Completion gate.

Resampling

General meaning. Conversion between declared sampling grids using specified interpolation/filtering rules.

In this library. Sample-rate conversion is distinct from a packet refinement that keeps the same ambient vector space.

Source. Chapter 34 (explanatory anchor `resampling-and-adaptive-filters`); frozen reference-source Markdown lines 29105-29109.

See also. Polyphase refinement, Signal.

Research API

General meaning. An exploratory interface surface with its own additive compatibility policy.

In this library. Research exports can implement exact finite identities. Their tier does not certify a new theorem.

Source. Chapter 7 (explanatory anchor `public-api-tiers`); frozen reference-source Markdown lines 2616-2622.

See also. API tier, Exactness, Provenance.

Residual

General meaning. A discrepancy between the compared quantities in a stated norm or algebra.

In this library. Interpret each named residual locally. A zero residual for one fixture does not establish all hypotheses of an operator identity.

Source. Chapter 5 (explanatory anchor `certificates-and-result-types`); frozen reference-source Markdown lines 1545-1551.

See also. Tolerance, Numerical result, Defect. **Notation:** NM-050.

Residue coordinate

General meaning. A finite address classified by its congruence class modulo a chosen integer.

In this library. Residue lanes and their quotient indices preserve a declared finite indexing convention.

Source. Chapter 13 (explanatory anchor `residue-crt-lattice-and-smith-coordinates`); frozen reference-source Markdown lines 8019-8023.

See also. Lane, CRT coordinates, Polyphase decomposition. **Notation:** NM-026.

Residue ring

General meaning. Integers modulo a positive modulus with modular arithmetic.

In this library. Not every nonzero element is invertible when the modulus is composite.

Source. Chapter 21 (explanatory anchor `formal-series-and-weighted-bases`); frozen reference-source Markdown lines 16973-16979.

See also. Unit, Prime field.

Resource budget

General meaning. A set of declared local bounds on selected operations or representations.

In this library. Some checks occur after traversal or allocation. A stored budget is not a universal parser, process-memory or hostile-input limit.

Source. Chapter K (explanatory anchor `appendix-k-resource-budgets`); frozen reference-source Markdown lines 108159-108165.

See also. Approximation budget, Memory estimate.

Rooted tree

General meaning. A finite rooted combinatorial structure used to organize nonlinear compositions.

In this library. Ordered nonlinear trees, leaf counts and heights have their own conventions. They are not interchangeable with wavelet packet trees.

Source. Chapter 39 (explanatory anchor `nonlinear-trees-and-graded-operations`); frozen reference-source Markdown lines 33404-33408.

See also. Tree height, Wavelet packet tree.

Sample rate

General meaning. The rate connecting sample index with physical time or frequency units.

In this library. FFT-bin indices are not physical frequencies without the corresponding rate and transform length.

Source. Chapter 31 (explanatory anchor `signal-conventions-windows-and-framing`); frozen reference-source Markdown lines 27208-27214.

See also. Signal, Periodogram.

Scalar domain

General meaning. The set of scalars and permitted arithmetic operations used by a construction.

In this library. Integer, rational, residue, prime-field and truncated prime-power domains have different units and inverses.

Source. Chapter 21 (explanatory anchor `formal-series-and-weighted-bases`); frozen reference-source Markdown lines 16961-16967.

See also. Unit, Exact arithmetic, Backend. **Notation:** NM-001, NM-004, NM-020.

Section operator

General meaning. Selection of a residue lane from a coefficient sequence.

In this library. Lane index, radix and normalization determine which coefficients and weights are retained.

Source. Chapter 22 (explanatory anchor `normalized-generating-functions`); frozen reference-source Markdown lines 17858-17864.

See also. Cartier operator, Lane.

Seed system

General meaning. A family of vectors used to generate a representation under an operator algebra.

In this library. Generation refers to the finite orbit hull, not arbitrary learning or discovery of generators.

Source. Chapter 10 (explanatory anchor `operator-algebras-and-seed-systems`); frozen reference-source Markdown lines 4725-4731.

See also. Orbit hull, Cyclic vector.

Sequence

General meaning. An indexed rule assigning values to natural-number indices.

In this library. The package evaluates finite requests and may cache values. Its exact metadata does not inspect or prove the evaluator's arithmetic.

Source. Chapter 26 (explanatory anchor `exact-sequence-execution`); frozen reference-source Markdown lines 23427-23431.

See also. Linear recurrence, Automatic sequence.

Sequence model

General meaning. A finite declared or inferred representation used for sequence evaluation.

In this library. A recurrence fitted to a finite prefix is extrapolation unless an independent justification establishes the rule globally.

Source. Chapter 26 (explanatory anchor `exact-sequence-execution`); frozen reference-source Markdown lines 23434-23440.

See also. Linear recurrence, Heuristic.

Set partition

General meaning. A division of a finite set of direction positions into disjoint nonempty blocks.

In this library. Block ordering conventions used for deterministic enumeration do not change the underlying partition, but affect stored addresses.

Source. Chapter 38 (explanatory anchor `packet-jets-and-fauxe0-di-bruno-composition`); frozen reference-source Markdown lines 32713-32719.

See also. Faà di Bruno composition, Multi-index. **Notation:** NM-051.

Signal

General meaning. A finite array interpreted with a declared sampling and axis convention.

In this library. Time is axis zero in the documented signal layer. Boundary extension and dtype are part of each operation.

Source. Chapter 31 (explanatory anchor `signal-conventions-windows-and-framing`); frozen reference-source Markdown lines 27201-27207.

See also. Frame, Sample rate. **Notation:** NM-040.

Singular packet

General meaning. A packet associated with the declared singularity or recovery model.

In this library. Its meaning depends on the specified finite transform and diagnostic, not simply a numerically singular matrix.

Source. Chapter 36 (explanatory anchor `singular-packets-and-recovery`); frozen reference-source Markdown lines 30764-30768.

See also. Recovery, Packet system.

Smith coordinates

General meaning. Integer-lattice coordinates obtained using a Smith normal form and unimodular changes of basis.

In this library. The diagonal invariant factors and basis changes carry the meaning, not a floating eigenvalue decomposition.

Source. Chapter 13 (explanatory anchor `residue-crt-lattice-and-smith-coordinates`); frozen reference-source Markdown lines 8019-8023.

See also. Unimodular transform, Lattice quotient.

Smooth packet transport

General meaning. Transport of a moving packet decomposition under a declared finite differentiable path.

In this library. Smoothness and derivative assumptions belong to the input model. Sampled checks do not prove arbitrary global regularity.

Source. Chapter 44 (explanatory anchor `smooth-packet-transport`); frozen reference-source Markdown lines 38677-38681.

See also. Kato transport, Packet jet.

Snapshot

General meaning. A finite stored capture of coefficients or state at a specified point in an execution.

In this library. A snapshot is not a live callback and may cover only a declared finite order or prefix.

Source. Chapter 43 (explanatory anchor `nonlinear-execution-streaming-profiling-and-gpu-paths`); frozen reference-source Markdown lines 37417-37423.

See also. Frozen record, Stream manifest.

Source checksum

General meaning. A content fingerprint computed over a specified byte or canonical-data representation.

In this library. A matching digest binds content identity, not semantic truth, authorship or authenticity.

Source. Chapter 75 (explanatory anchor `freeze-and-finalization-protocol`); frozen reference-source Markdown lines 110456-110462.

See also. Canonical JSON, Reproducibility. **Notation:** NM-060.

Spectrogram

General meaning. A time-frequency display derived from STFT values, often by magnitude or power.

In this library. A power spectrogram is not automatically a power spectral density with physical units.

Source. Chapter 32 (explanatory anchor `time-frequency-and-spectral-analysis`); frozen reference-source Markdown lines 27842-27848.

See also. STFT, Periodogram.

Stable API

General meaning. An interface governed by a declared compatibility contract.

In this library. The stable namespace is a selected surface. A stable symbol can perform numerical or conditional work.

Source. Chapter 7 (explanatory anchor `public-api-tiers`); frozen reference-source Markdown lines 2599-2605.

See also. API tier, Beta API, Research API.

Stable core

General meaning. Candidates that survive every represented compatibility level in a finite lift forest.

In this library. Stable here concerns the finite forest, not the stable API tier or asymptotic dynamical stability.

Source. Chapter 56 (explanatory anchor `chapter-56.-finite-deformation-systems`); frozen reference-source Markdown lines 45202-45208.

See also. Finite lift forest, Stable API.

State-space realization

General meaning. A finite state update and observation representation of a system or sequence.

In this library. The selected coordinates and state dimension are explicit, and minimality is not implied merely by successful construction.

Source. Chapter 18 (explanatory anchor `combinatorial-recurrence-and-state-space-coordinates`); frozen reference-source Markdown lines 12605-12609.

See also. Recurrence coordinates, Coordinate transform.

STFT

General meaning. A short-time Fourier transform of framed signal data.

In this library. The implementation uses local-frame Fourier coordinates. FFT length, hop, one/two-sided output and phase convention must be read together.

Source. Chapter 32 (explanatory anchor `time-frequency-and-spectral-analysis`); frozen reference-source Markdown lines 27654-27660.

See also. Frame, Spectrogram, Inverse STFT. **Notation:** NM-041, NM-042, NM-043, NM-044, NM-045.

Stream manifest

General meaning. Metadata describing a chunked finite stream, its progress and content identities.

In this library. Paused, complete and loadable states must be distinguished. A valid partial stream is not a completed reconstruction.

Source. Chapter 43 (explanatory anchor `nonlinear-execution-streaming-profiling-and-gpu-paths`); frozen reference-source Markdown lines 37929-37935.

See also. Snapshot, Source checksum.

Structured operator

General meaning. An operator stored in a representation exploiting a stated algebraic structure.

In this library. Apply, materialize, adjoint, storage and cost models are separate interfaces. Dense materialization can erase the storage advantage.

Source. Chapter 29 (explanatory anchor `structured-and-low-rank-operators`); frozen reference-source Markdown lines 26375-26381.

See also. Low-rank factorization, Kronecker product.

Supported lift

General meaning. A coefficient-space map representing a physical map on the supported coefficient subspaces.

In this library. Support projectors and possible transition bridges are required for the correct composition law.

Source. Chapter 20 (explanatory anchor `analysis-synthesis-and-redundant-frames`); frozen reference-source Markdown lines 15501-15507.

See also. Coefficient projector, Transition bridge.

Survival operator

General meaning. Evolution restricted or projected to a chosen retained sector.

In this library. Loss of norm or leakage can reflect coupling to excluded sectors rather than failure of the full evolution law.

Source. Chapter 46 (explanatory anchor `feshbach-reduction-memory-and-survival`); frozen reference-source Markdown lines 40656-40660.

See also. Memory kernel, Leakage.

Symbol card

General meaning. A structured reference record for one public symbol, its type, assumptions, behavior and evidence.

In this library. Canonical placement is separate from the number of aliases exposing the same object.

Source. Chapter 1 (explanatory anchor `editorial-and-symbolic-grammar`); frozen reference-source Markdown lines 253-259.

See also. Qualified name, API tier. **Notation:** NM-018.

Symmetry packet

General meaning. A sector resolved by projectors associated with declared finite symmetry data.

In this library. Sector labels do not prove a global symmetry or continuum spectral theorem.

Source. Chapter 53 (explanatory anchor `chapter-53.-finite-quantum-foundations`); frozen reference-source Markdown lines 44600-44606.

See also. Packet system, Observable.

Synthesis map

General meaning. A map from coefficients back to physical vectors.

In this library. Reconstruction requires synthesis after analysis to equal the physical identity. Analysis after synthesis need only be a projection.

Source. Chapter 20 (explanatory anchor `analysis-synthesis-and-redundant-frames`); frozen reference-source Markdown lines 15521-15527.

See also. Analysis map, Coefficient projector. **Notation:** NM-029.

Taylor truncation

General meaning. A finite derivative expansion through a declared order.

In this library. Exact computation of its coefficients does not by itself provide an analytic remainder bound or infinite-series convergence.

Source. Chapter 38 (explanatory anchor `packet-jets-and-fauxe0-di-bruno-composition`); frozen reference-source Markdown lines 32538-32544.

See also. Packet jet, Truncation.

Tensor coordinate transport

General meaning. Coordinate conversion induced on a tensor product of factor spaces.

In this library. Factor order and flattening conventions matter. A Kronecker product with reversed factors generally represents another map.

Source. Chapter 19 (explanatory anchor `spectral-packet-tensor-and-operator-transport`); frozen reference-source Markdown lines 14168-14172.

See also. Kronecker product, Coordinate transform. **Notation:** NM-064.

Theorem identifier

General meaning. A source label used to navigate a stated claim or construction.

In this library. The finite label index includes definitions, engineering checks and conditional records. IDs can be reused and are not globally unique proof keys.

Source. Chapter H (explanatory anchor `appendix-h-certificate-theorem-index`); frozen reference-source Markdown lines 96760-96766.

See also. Qualified name, Certificate.

Tolerance

General meaning. A threshold used for specified numerical comparisons.

In this library. It belongs to a particular norm, scale and procedure. It neither changes exact equality nor supplies a universal error bound.

Source. Chapter 1 (explanatory anchor `editorial-and-symbolic-grammar`); frozen reference-source Markdown lines 346-352.

See also. Residual, Exact arithmetic.

Transfer block

General meaning. The target-by-source component of an operator after output and input projection.

In this library. The index order is target then source. The block contains coefficients, while a transfer graph keeps only support information.

Source. Chapter 11 (explanatory anchor `packet-systems-and-transfer-blocks`); frozen reference-source Markdown lines 5637-5643.

See also. Packet operator, Transfer graph. **Notation:** NM-023.

Transfer graph

General meaning. A directed graph recording active source-to-target transfers.

In this library. Numerical support is relative to the active tolerance. Boolean support forgets weights and possible cancellation between paths.

Source. Chapter 11 (explanatory anchor `packet-systems-and-transfer-blocks`); frozen reference-source Markdown lines 5657-5663.

See also. Transfer block, Transfer path.

Transfer path

General meaning. A legal sequence of packet transitions through a finite operator product.

In this library. Its contribution is an ordered block product. Existence of a graph path alone does not ensure a nonzero total contribution.

Source. Chapter 11 (explanatory anchor `packet-systems-and-transfer-blocks`); frozen reference-source Markdown lines 5664-5670.

See also. Transfer graph, Packet operator.

Transition bridge

General meaning. A map reconciling different intermediate coefficient presentations during composition.

In this library. Omitting it can produce the wrong coefficient product even when the underlying physical maps compose.

Source. Chapter 20 (explanatory anchor `analysis-synthesis-and-redundant-frames`); frozen reference-source Markdown lines 15502-15508.

See also. Supported lift, Coordinate transform.

Tree height

General meaning. A depth measure defined by the selected rooted-tree convention.

In this library. Use the library's leaf/root convention before interpreting height-bounded enumeration and its resource guard.

Source. Chapter 39 (explanatory anchor `nonlinear-trees-and-graded-operations`); frozen reference-source Markdown lines 33476-33482.

See also. Rooted tree, Resource budget.

Truncated p-adic value

General meaning. A finite-precision residue model modulo a prime power.

In this library. Increasing a requested precision cannot recover information that was already discarded by reduction.

Source. Chapter 25 (explanatory anchor `local-arithmetic-models`); frozen reference-source Markdown lines 21835-21841.

See also. Valuation, Truncation. **Notation:** NM-006.

Truncation

General meaning. Restriction to a declared finite degree, order, length or precision.

In this library. Its meaning is domain-specific. Discarded high coefficients and discarded modular information are different operations.

Source. Chapter 21 (explanatory anchor `formal-series-and-weighted-bases`); frozen reference-source Markdown lines 16885-16891.

See also. Formal series, Truncated p-adic value.

Typed expression

General meaning. A finite symbolic value or map carrying domain, codomain and basis semantics.

In this library. Shape agreement alone does not establish compatible types. Opaque callbacks are not substituted for branded exact primitives.

Source. Chapter 50 (explanatory anchor `typed-finite-expression-model`); frozen reference-source Markdown lines 42982-42986.

See also. Basis witness, Normal form. **Notation:** NM-019.

Unimodular transform

General meaning. An integer matrix invertible over the integers, with determinant a unit.

In this library. It preserves the integer lattice rather than merely preserving a real vector space.

Source. Chapter 13 (explanatory anchor `residue-crt-lattice-and-smith-coordinates`); frozen reference-source Markdown lines 8378-8384.

See also. Smith coordinates, Lattice quotient.

Unit

General meaning. A scalar with a multiplicative inverse in its declared domain.

In this library. A nonzero residue modulo a composite modulus can be a nonunit. Rational nonzero values and prime-field nonzero values have different domain rules.

Source. Chapter 21 (explanatory anchor `formal-series-and-weighted-bases`); frozen reference-source Markdown lines 17012-17018.

See also. Scalar domain, Residue ring.

Unit part

General meaning. The factor remaining after removing the relevant prime-power valuation.

In this library. It is evaluated in the declared finite precision, and must not be conflated with the full residue.

Source. Chapter 24 (explanatory anchor `prime-power-bell-cyclotomic-and-q-kummer-systems`); frozen reference-source Markdown lines 20580-20586.

See also. Valuation, Prime-power kernel.

Validity flag

General meaning. A stored indication that a producer regarded its local predicate as satisfied.

In this library. The base certificate currently accepts truthy values. Stored validity is not independent witness checking, and loading it does not prove the claim.

Source. Chapter H (explanatory anchor `appendix-h-certificate-theorem-index`); frozen reference-source Markdown lines 96539-96545.

See also. Certificate, Portable record.

Valuation

General meaning. A measure of divisibility by a chosen prime, or the corresponding local order.

In this library. Zero conventions differ by API: some valuation methods return infinity and others reject zero. The local method contract controls.

Source. Chapter 25 (explanatory anchor `local-arithmetic-models`); frozen reference-source Markdown lines 21819-21825.

See also. Unit part, Truncated p-adic value.

Verification suite

General meaning. A named collection of finite checks.

In this library. Suite names can dispatch to historical aggregates. A suite is not automatically the full source test run.

Source. Chapter 71 (explanatory anchor `verification-suites`); frozen reference-source Markdown lines 47529-47533.

See also. Qualification, Finite certificate.

Wavelet packet tree

General meaning. A hierarchy of recursively split wavelet/filter-bank coefficient subspaces.

In this library. The finite tree, boundary rule and selected basis determine reconstruction. It is not the nonlinear rooted-tree algebra.

Source. Chapter 35 (explanatory anchor `filter-banks-wavelets-and-packet-trees`); frozen reference-source Markdown lines 29632-29636.

See also. Best basis, Rooted tree.

Weighted monomial basis

General meaning. A basis with degree-dependent monomial normalization.

In this library. The basis element is z to the n divided by gamma_n. The stored coefficient is not generally the ordinary monomial coefficient.

Source. Chapter 21 (explanatory anchor `formal-series-and-weighted-bases`); frozen reference-source Markdown lines 16910-16916.

See also. Normalized coefficient, Formal series. **Notation:** NM-034, NM-035.

Weighted transfer graph

General meaning. A transfer graph with weights representing additional arithmetic or operator data.

In this library. Weight meaning is supplied by the local model, not by the Boolean graph alone.

Source. Chapter 12 (explanatory anchor `refinement-towers`); frozen reference-source Markdown lines 6761-6767.

See also. Transfer graph, Valuation.

Welch PSD

General meaning. An averaged, windowed finite power spectral density estimate.

In this library. Averaging parameters, segment lengths and scaling are part of the estimate. The result is not an exact underlying stochastic spectrum.

Source. Chapter 32 (explanatory anchor `time-frequency-and-spectral-analysis`); frozen reference-source Markdown lines 27890-27896.

See also. Periodogram, Coherence.

Window

General meaning. A finite weighting sequence applied to a signal segment.

In this library. Window metrics, overlap support and reconstruction conditions are distinct; a named window alone does not ensure invertible framing.

Source. Chapter 31 (explanatory anchor `signal-conventions-windows-and-framing`); frozen reference-source Markdown lines 27201-27205.

See also. Frame, Overlap-add.

Zero certificate

General meaning. A diagnostic record for the supplied or computed zero representatives.

In this library. Small residuals and ordering do not establish that every zero has been found or prove a global counting theorem.

Source. Chapter 27 (explanatory anchor `cyclic-analytic-systems`); frozen reference-source Markdown lines 24957-24963.

See also. Residual, Conditional result.

M.5 Notation register: glyph, speech and local sense

A symbol is identified by **display + local domain + explanatory anchor**, not by typography alone. The collision-family key supports lookup and does not claim that all strings in that family are literally identical. In particular, plain, calligraphic, bold and sans-serif versions can denote different objects.

NM-001. integers

Display: $\mathbb{Z}$.

Local meaning. Integer scalar domain. A quotient-ring notation must include its modulus.

Domain and source. Chapter 1 (explanatory anchor `editorial-and-symbolic-grammar`); frozen reference-source Markdown lines 338-344.

Spelling status. The exact display string occurs in the cited source chapter.

Concept: Scalar domain. **Other senses in this notation family:** NM-002, NM-003, NM-004.

NM-002. rationals

Display: $\mathbb{Q}$.

Local meaning. Rational scalars. The implemented exact coordinate layer uses finite rational tuples.

Domain and source. Chapter 1 (explanatory anchor `editorial-and-symbolic-grammar`); frozen reference-source Markdown lines 338-344.

Spelling status. The exact display string occurs in the cited source chapter.

Concept: Exact arithmetic. **Other senses in this notation family:** NM-001, NM-003, NM-004.

NM-003. reals

Display: $\mathbb{R}$.

Local meaning. Mathematical real scalars; floating arrays approximate real arithmetic rather than realize all real numbers exactly.

Domain and source. Chapter 1 (explanatory anchor `editorial-and-symbolic-grammar`); frozen reference-source Markdown lines 338-344.

Spelling status. The exact display string occurs in the cited source chapter.

Concept: Numerical result. **Other senses in this notation family:** NM-001, NM-002, NM-004.

NM-004. complex numbers

Display: $\mathbb{C}$.

Local meaning. Mathematical complex scalars. Transpose and conjugate transpose differ on these scalars.

Domain and source. Chapter 1 (explanatory anchor `editorial-and-symbolic-grammar`); frozen reference-source Markdown lines 338-344.

Spelling status. The exact display string occurs in the cited source chapter.

Concept: Scalar domain. **Other senses in this notation family:** NM-001, NM-002, NM-003.

NM-005. F p

Display: $\mathbb{F}_p$.

Local meaning. Residues in the prime field of characteristic p. Primality is part of the intended domain.

Domain and source. Chapter 1 (explanatory anchor `editorial-and-symbolic-grammar`); frozen reference-source Markdown lines 338-344.

Spelling status. The exact display string occurs in the cited source chapter.

Concept: Prime field. **Other senses in this notation family:** NM-006, NM-024, NM-025, NM-056.

NM-006. integers modulo p to the k

Display: $\mathbb{Z}/p^k\mathbb{Z}$.

Local meaning. A finite prime-power residue domain, not the complete p-adic field.

Domain and source. Chapter 1 (explanatory anchor `editorial-and-symbolic-grammar`); frozen reference-source Markdown lines 338-344.

Spelling status. The exact display string occurs in the cited source chapter.

Concept: Truncated p-adic value. **Other senses in this notation family:** NM-005, NM-024, NM-025, NM-056.

NM-007. A adjoint

Display: A^*.

Local meaning. Conjugate transpose under the global matrix convention. A different metric must be stated explicitly.

Domain and source. Chapter 1 (explanatory anchor `editorial-and-symbolic-grammar`); frozen reference-source Markdown lines 341-347.

Spelling status. The exact display string occurs in the cited source chapter.

Concept: Adjoint. **Other senses in this notation family:** NM-059.

NM-008. identity on X

Display: I_X.

Local meaning. Identity map on the indicated space X. Its dimensions follow the type, not a context-free scalar one.

Domain and source. Chapter 1 (explanatory anchor `editorial-and-symbolic-grammar`); frozen reference-source Markdown lines 342-348.

Spelling status. The exact display string occurs in the cited source chapter.

Concept: Projector. **Other senses in this notation family:** NM-030.

NM-009. P sub i, packet projector

Display: P_i.

Local meaning. A packet projector only where the local chapter has declared that interpretation.

Domain and source. Chapter 1 (explanatory anchor `editorial-and-symbolic-grammar`); frozen reference-source Markdown lines 343-349.

Spelling status. The exact display string occurs in the cited source chapter.

Concept: Packet system. **Other senses in this notation family:** NM-010, NM-017, NM-057.

NM-010. P sub i, coordinate selector

Display: P_i.

Local meaning. A local coordinate selector is not automatically a physical reconstruction component or the same map as a packet projector.

Domain and source. Chapter 1 (explanatory anchor `editorial-and-symbolic-grammar`); frozen reference-source Markdown lines 343-349.

Spelling status. The exact display string occurs in the cited source chapter.

Concept: Reconstruction component. **Other senses in this notation family:** NM-009, NM-017, NM-057.

NM-011. T from A coordinates to B

Display: $T_{B\leftarrow A}$.

Local meaning. A map converting A coordinates to B coordinates. The arrow names source and target.

Domain and source. Chapter 1 (explanatory anchor `editorial-and-symbolic-grammar`); frozen reference-source Markdown lines 348-354.

Spelling status. The exact display string occurs in the cited source chapter.

Concept: Coordinate transform. **Other senses in this notation family:** NM-012, NM-023, NM-052.

NM-012. T from B to C times T from A to B

Display: $T_{C\leftarrow B}T_{B\leftarrow A}$.

Local meaning. Composition applies A-to-B first. The following transformation multiplies on the left.

Domain and source. Chapter 1 (explanatory anchor `editorial-and-symbolic-grammar`); frozen reference-source Markdown lines 352-358.

Spelling status. The exact display string occurs in the cited source chapter.

Concept: Coordinate transform. **Other senses in this notation family:** NM-011, NM-023, NM-052.

NM-013. exact evidence E

Display: E.

Local meaning. Editorial exact finite evidence class. It is not a coordinate space or a function expectation.

Domain and source. Chapter 1 (explanatory anchor `editorial-and-symbolic-grammar`); frozen reference-source Markdown lines 414-420.

Spelling status. The exact display string occurs in the cited source chapter.

Concept: Exactness. **Other senses in this notation family:** NM-018, NM-055.

NM-014. numerical evidence N

Display: N.

Local meaning. Editorial numerical evidence class, not a signal length or truncation degree.

Domain and source. Chapter 1 (explanatory anchor `editorial-and-symbolic-grammar`); frozen reference-source Markdown lines 414-420.

Spelling status. The exact display string occurs in the cited source chapter.

Concept: Numerical result. **Other senses in this notation family:** NM-040, NM-061.

NM-015. conditional evidence C

Display: C.

Local meaning. Editorial conditional evidence class, not the coefficient space C or complex scalars.

Domain and source. Chapter 1 (explanatory anchor `editorial-and-symbolic-grammar`); frozen reference-source Markdown lines 414-420.

Spelling status. The exact display string occurs in the cited source chapter.

Concept: Conditional result. **Other senses in this notation family:** NM-045.

NM-016. heuristic evidence H

Display: H.

Local meaning. Editorial heuristic class, not a signal hop or a Hilbert space.

Domain and source. Chapter 1 (explanatory anchor `editorial-and-symbolic-grammar`); frozen reference-source Markdown lines 414-420.

Spelling status. The exact display string occurs in the cited source chapter.

Concept: Heuristic. **Other senses in this notation family:** NM-041, NM-053.

NM-017. planned evidence P

Display: P.

Local meaning. Editorial planned class, not a packet projector or polynomial coordinate space.

Domain and source. Chapter 1 (explanatory anchor `editorial-and-symbolic-grammar`); frozen reference-source Markdown lines 414-420.

Spelling status. The exact display string occurs in the cited source chapter.

Concept: Planned feature. **Other senses in this notation family:** NM-009, NM-010, NM-057.

NM-018. example record E

Display: $\mathfrak{E}$.

Local meaning. Input/hypothesis/action/output/certificate/failure/reproducibility example record. This is an editorial record, not an operator.

Domain and source. Chapter 1 (explanatory anchor `editorial-and-symbolic-grammar`); frozen reference-source Markdown lines 317-323.

Spelling status. The exact display string occurs in the cited source chapter.

Concept: Symbol card. **Other senses in this notation family:** NM-013, NM-055.

NM-019. f maps X to Y

Display: $f : X \to Y$.

Local meaning. Typed map with stated domain and codomain. Equal dimensions alone do not identify the two named spaces.

Domain and source. Chapter 1 (explanatory anchor `editorial-and-symbolic-grammar`); frozen reference-source Markdown lines 274-280.

Spelling status. The exact display string occurs in the cited source chapter.

Concept: Typed expression. No other notation record is assigned to this collision family.

NM-020. R m by n

Display: $R^{m\times n}$.

Local meaning. Matrices over the locally named scalar domain R. It does not force R to be the real numbers.

Domain and source. Chapter 1 (explanatory anchor `editorial-and-symbolic-grammar`); frozen reference-source Markdown lines 509-515.

Spelling status. The exact display string occurs in the cited source chapter.

Concept: Scalar domain. No other notation record is assigned to this collision family.

NM-021. operator grammar G

Display: $\mathsf{G} = (V, A, s, t)$.

Local meaning. Vertices, arrows and source/target maps of the typed operator quiver. Here A is an arrow set, not an analysis matrix.

Domain and source. Chapter 9 (explanatory anchor `typed-operator-grammars`); frozen reference-source Markdown lines 3800-3806.

Spelling status. The exact display string occurs in the cited source chapter.

Concept: Operator grammar. **Other senses in this notation family:** NM-022, NM-028, NM-031, NM-047.

NM-022. generated algebra A

Display: $\mathcal{A}$.

Local meaning. The algebra generated by the declared operator family. Finite rank tests depend on backend and tolerance.

Domain and source. Chapter 10 (explanatory anchor `operator-algebras-and-seed-systems`); frozen reference-source Markdown lines 4714-4718.

Spelling status. The exact display string occurs in the cited source chapter.

Concept: Generated operator algebra. **Other senses in this notation family:** NM-021, NM-028, NM-031, NM-047.

NM-023. T j i equals Q j T P i

Display: $T_{ji} = Q_j T P_i$.

Local meaning. Transfer from input packet i to output packet j. The first block index is the target.

Domain and source. Chapter 11 (explanatory anchor `packet-systems-and-transfer-blocks`); frozen reference-source Markdown lines 5637-5643.

Spelling status. The exact display string occurs in the cited source chapter.

Concept: Transfer block. **Other senses in this notation family:** NM-011, NM-012, NM-052.

NM-024. coarse P a is the sum of its children

Display: $P_a = \sum_{b:p(b)=a} Q_b$.

Local meaning. Refinement compatibility on the same ambient space, with parent map p.

Domain and source. Chapter 12 (explanatory anchor `refinement-towers`); frozen reference-source Markdown lines 6751-6757.

Spelling status. The exact display string occurs in the cited source chapter.

Concept: Packet refinement. **Other senses in this notation family:** NM-005, NM-006, NM-025, NM-056.

NM-025. parent map p

Display: $p : B \to A$.

Local meaning. The parent of a fine packet label. This p is not an arithmetic prime.

Domain and source. Chapter 12 (explanatory anchor `refinement-towers`); frozen reference-source Markdown lines 6745-6751.

Spelling status. Normalized editorial display for the cited sense; not asserted to be a verbatim source formula.

Concept: Packet refinement. **Other senses in this notation family:** NM-005, NM-006, NM-024, NM-056.

NM-026. residue r channel at k

Display: $x_r[k] = x[r + Mk]$.

Local meaning. Residue-lane indexing under modulus M. Ragged channel lengths depend on the original finite length.

Domain and source. Chapter 13 (explanatory anchor `residue-crt-lattice-and-smith-coordinates`); frozen reference-source Markdown lines 8054-8060.

Spelling status. The exact display string occurs in the cited source chapter.

Concept: Residue coordinate. **Other senses in this notation family:** NM-027, NM-043.

NM-027. nested address r plus M s

Display: $r + Ms$.

Local meaning. Positional nested address. It is not generally the Chinese-remainder representative.

Domain and source. Chapter 13 (explanatory anchor `residue-crt-lattice-and-smith-coordinates`); frozen reference-source Markdown lines 8028-8034.

Spelling status. The exact display string occurs in the cited source chapter.

Concept: Mixed-radix address. **Other senses in this notation family:** NM-026, NM-043.

NM-028. analysis A from X to C

Display: $A : X \to C$.

Local meaning. Analysis from physical space to coefficient space. Here C names a space and does not mean the complex field.

Domain and source. Chapter 20 (explanatory anchor `analysis-synthesis-and-redundant-frames`); frozen reference-source Markdown lines 15519-15525.

Spelling status. The exact display string occurs in the cited source chapter.

Concept: Analysis map. **Other senses in this notation family:** NM-021, NM-022, NM-031, NM-047.

NM-029. synthesis S from C to X

Display: $S : C \to X$.

Local meaning. Synthesis from coefficients to physical vectors.

Domain and source. Chapter 20 (explanatory anchor `analysis-synthesis-and-redundant-frames`); frozen reference-source Markdown lines 15521-15527.

Spelling status. The exact display string occurs in the cited source chapter.

Concept: Synthesis map. No other notation record is assigned to this collision family.

NM-030. S A equals physical identity

Display: $SA = I_X$.

Local meaning. Verified left-inverse relation for the physical presentation. Bare construction can store a proposal that fails it.

Domain and source. Chapter 20 (explanatory anchor `analysis-synthesis-and-redundant-frames`); frozen reference-source Markdown lines 15523-15529.

Spelling status. The exact display string occurs in the cited source chapter.

Concept: Analysis-synthesis system. **Other senses in this notation family:** NM-008.

NM-031. A S, coefficient projector

Display: AS.

Local meaning. Analysis after synthesis is idempotent when SA is identity, but is not generally the full coefficient identity.

Domain and source. Chapter 20 (explanatory anchor `analysis-synthesis-and-redundant-frames`); frozen reference-source Markdown lines 15500-15506.

Spelling status. The exact display string occurs in the cited source chapter.

Concept: Coefficient projector. **Other senses in this notation family:** NM-021, NM-022, NM-028, NM-047.

NM-032. kernel of synthesis S

Display: ker S.

Local meaning. Coefficient changes invisible after synthesis. Gauge directions are not necessarily zero coefficient vectors.

Domain and source. Chapter 20 (explanatory anchor `analysis-synthesis-and-redundant-frames`); frozen reference-source Markdown lines 15501-15507.

Spelling status. The exact display string occurs in the cited source chapter.

Concept: Coefficient gauge. No other notation record is assigned to this collision family.

NM-033. Frobenius norm of reconstruction defect

Display: $\|SA - I_n\|_F$.

Local meaning. The stated left-inverse residual. The subscript F denotes a norm, not a finite-field Frobenius map.

Domain and source. Chapter 20 (explanatory anchor `analysis-synthesis-and-redundant-frames`); frozen reference-source Markdown lines 15615-15621.

Spelling status. The exact display string occurs in the cited source chapter.

Concept: Frobenius norm. **Other senses in this notation family:** NM-044, NM-060.

NM-034. gamma sub n

Display: γ_n.

Local meaning. Nonzero degree-n normalization denominator with gamma_0 equal to one.

Domain and source. Chapter 21 (explanatory anchor `formal-series-and-weighted-bases`); frozen reference-source Markdown lines 17069-17075.

Spelling status. The exact display string occurs in the cited source chapter.

Concept: Weighted monomial basis. No other notation record is assigned to this collision family.

NM-035. w n equals inverse gamma n

Display: $w_n = \gamma_n^{-1}$.

Local meaning. Monomial weight reciprocal to the normalization denominator. Do not confuse it with a signal window.

Domain and source. Chapter 21 (explanatory anchor `formal-series-and-weighted-bases`); frozen reference-source Markdown lines 17076-17082.

Spelling status. The exact display string occurs in the cited source chapter.

Concept: Weighted monomial basis. No other notation record is assigned to this collision family.

NM-036. ordinary coefficient c n

Display: $c_n = a_n/\gamma_n$.

Local meaning. Conversion from stored normalized coefficient to ordinary monomial coefficient.

Domain and source. Chapter 22 (explanatory anchor `normalized-generating-functions`); frozen reference-source Markdown lines 17856-17862.

Spelling status. The exact display string occurs in the cited source chapter.

Concept: Normalized coefficient. No other notation record is assigned to this collision family.

NM-037. z power n divided by n factorial

Display: $z^n/n!$.

Local meaning. Exponential normalization. Multiplication on stored coefficients carries binomial factors.

Domain and source. Chapter 21 (explanatory anchor `formal-series-and-weighted-bases`); frozen reference-source Markdown lines 17206-17212.

Spelling status. The exact display string occurs in the cited source chapter.

Concept: Hurwitz series. **Other senses in this notation family:** NM-051.

NM-038. p to precision r

Display: p^r.

Local meaning. Prime-power modulus in the specified finite arithmetic branch. r here is precision, not a residue label or derivative order.

Domain and source. Chapter 24 (explanatory anchor `prime-power-bell-cyclotomic-and-q-kummer-systems`); frozen reference-source Markdown lines 20589-20595.

Spelling status. The exact display string occurs in the cited source chapter.

Concept: Prime-power kernel. **Other senses in this notation family:** NM-046.

NM-039. Krylov space K k of A and b

Display: $\mathcal{K}_k(A, b)$.

Local meaning. Span of b and its first k-1 iterated images under A. The callback hypotheses remain explicit.

Domain and source. Chapter 30 (explanatory anchor `krylov-precision-compilation-and-scheduling`); frozen reference-source Markdown lines 26693-26699.

Spelling status. The exact display string occurs in the cited source chapter.

Concept: Krylov subspace. **Other senses in this notation family:** NM-042.

NM-040. length N signal x

Display: $x \in \mathbb{K}^N$.

Local meaning. Finite real or complex signal. N is a sample count in this sense.

Domain and source. Chapter 31 (explanatory anchor `signal-conventions-windows-and-framing`); frozen reference-source Markdown lines 27204-27210.

Spelling status. Normalized editorial display for the cited sense; not asserted to be a verbatim source formula.

Concept: Signal. **Other senses in this notation family:** NM-014, NM-061.

NM-041. hop H

Display: H.

Local meaning. Number of sample positions between successive frames. It is not a Hamiltonian or heuristic evidence class.

Domain and source. Chapter 32 (explanatory anchor `time-frequency-and-spectral-analysis`); frozen reference-source Markdown lines 27644-27650.

Spelling status. The exact display string occurs in the cited source chapter.

Concept: STFT. **Other senses in this notation family:** NM-016, NM-053.

NM-042. FFT length K

Display: K.

Local meaning. Transform length, at least the stated frame length. It need not equal the original signal length.

Domain and source. Chapter 32 (explanatory anchor `time-frequency-and-spectral-analysis`); frozen reference-source Markdown lines 27644-27650.

Spelling status. The exact display string occurs in the cited source chapter.

Concept: STFT. **Other senses in this notation family:** NM-039.

NM-043. frame count M

Display: M.

Local meaning. Number of frames in the returned array. Elsewhere M can name an arithmetic modulus or memory estimate.

Domain and source. Chapter 32 (explanatory anchor `time-frequency-and-spectral-analysis`); frozen reference-source Markdown lines 27644-27650.

Spelling status. The exact display string occurs in the cited source chapter.

Concept: STFT. **Other senses in this notation family:** NM-026, NM-027.

NM-044. frequency-bin count F

Display: F.

Local meaning. Count determined by one-sided or two-sided transform conventions. This F is not a nonlinear map.

Domain and source. Chapter 32 (explanatory anchor `time-frequency-and-spectral-analysis`); frozen reference-source Markdown lines 27644-27650.

Spelling status. The exact display string occurs in the cited source chapter.

Concept: STFT. **Other senses in this notation family:** NM-033, NM-060.

NM-045. complex frame by frequency array

Display: $\mathbb{C}^{M \times F}$.

Local meaning. STFT output with frame axis first and frequency axis second.

Domain and source. Chapter 32 (explanatory anchor `time-frequency-and-spectral-analysis`); frozen reference-source Markdown lines 27657-27663.

Spelling status. The exact display string occurs in the cited source chapter.

Concept: STFT. **Other senses in this notation family:** NM-015.

NM-046. r-th derivative of F at x

Display: $D^r F(x)$.

Local meaning. An r-linear derivative evaluation in the declared provider model. The index r is derivative order here.

Domain and source. Chapter 38 (explanatory anchor `packet-jets-and-fauxe0-di-bruno-composition`); frozen reference-source Markdown lines 32593-32599.

Spelling status. The exact display string occurs in the cited source chapter.

Concept: Derivative provider. **Other senses in this notation family:** NM-038.

NM-047. order r address space

Display: $\mathcal{A}_r = B \times A^r$.

Local meaning. One output packet label and an ordered r-tuple of input labels. Here A and B are label sets.

Domain and source. Chapter 38 (explanatory anchor `packet-jets-and-fauxe0-di-bruno-composition`); frozen reference-source Markdown lines 32582-32588.

Spelling status. The exact display string occurs in the cited source chapter.

Concept: Jet address. **Other senses in this notation family:** NM-021, NM-022, NM-028, NM-031.

NM-048. alpha, output and input packet address

Display: $\alpha = (b; a_1, \dots, a_r)$.

Local meaning. A packet-jet address, not automatically a polynomial-exponent multi-index.

Domain and source. Chapter 38 (explanatory anchor `packet-jets-and-fauxe0-di-bruno-composition`); frozen reference-source Markdown lines 32586-32592.

Spelling status. The exact display string occurs in the cited source chapter.

Concept: Jet address. No other notation record is assigned to this collision family.

NM-049. addressed order r packet jet

Display: $J^{(r)}_{b;a_1,\dots,a_r}(x)$.

Local meaning. The output projection of a derivative evaluated on projected input directions.

Domain and source. Chapter 38 (explanatory anchor `packet-jets-and-fauxe0-di-bruno-composition`); frozen reference-source Markdown lines 32592-32598.

Spelling status. The exact display string occurs in the cited source chapter.

Concept: Packet jet. No other notation record is assigned to this collision family.

NM-050. rho rec, reconstruction residual

Display: ρ_{rec}.

Local meaning. Norm of direct derivative evaluation minus the complete addressed reconstruction. It is not a density matrix.

Domain and source. Chapter 38 (explanatory anchor `packet-jets-and-fauxe0-di-bruno-composition`); frozen reference-source Markdown lines 32661-32667.

Spelling status. The exact display string occurs in the cited source chapter.

Concept: Residual. **Other senses in this notation family:** NM-054.

NM-051. r factorial

Display: $r!$.

Local meaning. Number of paired permutations used by the finite symmetry check, not a claim that every jet algorithm has this cost.

Domain and source. Chapter 38 (explanatory anchor `packet-jets-and-fauxe0-di-bruno-composition`); frozen reference-source Markdown lines 32628-32634.

Spelling status. The exact display string occurs in the cited source chapter.

Concept: Set partition. **Other senses in this notation family:** NM-037.

NM-052. matrix semigroup T at t

Display: $T(t) = e^{tA}$.

Local meaning. Autonomous finite matrix evolution. It is not a coordinate-change matrix merely because it is called T.

Domain and source. Chapter 45 (explanatory anchor `matrix-dynamics`); frozen reference-source Markdown lines 39419-39425.

Spelling status. The exact display string occurs in the cited source chapter.

Concept: Matrix dynamics. **Other senses in this notation family:** NM-011, NM-012, NM-023.

NM-053. Hilbert space H

Display: $\mathcal{H}$.

Local meaning. Finite complex Hilbert space of the quantum model. Distinct from hop H and heuristic class H.

Domain and source. Chapter 53 (explanatory anchor `chapter-53.-finite-quantum-foundations`); frozen reference-source Markdown lines 44607-44613.

Spelling status. The exact display string occurs in the cited source chapter.

Concept: Observable. **Other senses in this notation family:** NM-016, NM-041.

NM-054. density rho

Display: ρ.

Local meaning. Density operator in the finite open-system model. This is not the packet reconstruction residual.

Domain and source. Chapter 54 (explanatory anchor `chapter-54.-circuits-channels-and-open-dynamics`); frozen reference-source Markdown lines 44832-44838.

Spelling status. Normalized editorial display for the cited sense; not asserted to be a verbatim source formula.

Concept: Quantum channel. **Other senses in this notation family:** NM-050.

NM-055. error E i

Display: E_i.

Local meaning. A supplied error operator. E here is not the exactness class or editorial example record.

Domain and source. Chapter 55 (explanatory anchor `chapter-55.-error-correction-and-effective-dynamics`); frozen reference-source Markdown lines 45070-45076.

Spelling status. The exact display string occurs in the cited source chapter.

Concept: Quantum code. **Other senses in this notation family:** NM-013, NM-018.

NM-056. finite field p to n

Display: $\mathbb{F}_{p^n}$.

Local meaning. Extension field represented by a supplied polynomial basis and valid modulus polynomial.

Domain and source. Chapter 57 (explanatory anchor `chapter-57.-local-finite-langlands-packets`); frozen reference-source Markdown lines 45439-45445.

Spelling status. The exact display string occurs in the cited source chapter.

Concept: Finite field. **Other senses in this notation family:** NM-005, NM-006, NM-024, NM-025.

NM-057. polynomial coordinate space P n B

Display: P_n^B.

Local meaning. Degree-through-n polynomial coordinate space in an explicit graded basis B. Not a packet projector.

Domain and source. Chapter 50 (explanatory anchor `typed-finite-expression-model`); frozen reference-source Markdown lines 43003-43009.

Spelling status. The exact display string occurs in the cited source chapter.

Concept: Basis witness. **Other senses in this notation family:** NM-009, NM-010, NM-017.

NM-058. normal form of expression e

Display: $\mathrm{NF}(e)$.

Local meaning. Exact vector or matrix denotation of an accepted closed typed expression.

Domain and source. Chapter 51 (explanatory anchor `expression-evaluation-and-certification`); frozen reference-source Markdown lines 43523-43529.

Spelling status. The exact display string occurs in the cited source chapter.

Concept: Normal form. No other notation record is assigned to this collision family.

NM-059. document metadata and block sequence

Display: $\mathcal{D} = \mathcal{M} \times \mathcal{B}^*$.

Local meaning. A document's metadata and finite ordered block buffer. Star here means finite sequence construction, not adjoint.

Domain and source. Chapter 59 (explanatory anchor `chapter-59.-mathematical-document-model`); frozen reference-source Markdown lines 45886-45892.

Spelling status. Normalized editorial display for the cited sense; not asserted to be a verbatim source formula.

Concept: Document model. **Other senses in this notation family:** NM-007.

NM-060. freeze record F

Display: $F = (v, \mathcal{I}, h, \mathcal{R})$.

Local meaning. Version, inventory, digests and reconciliation items of a documentation freeze.

Domain and source. Chapter 75 (explanatory anchor `freeze-and-finalization-protocol`); frozen reference-source Markdown lines 110452-110458.

Spelling status. The exact display string occurs in the cited source chapter.

Concept: Source checksum. **Other senses in this notation family:** NM-033, NM-044.

NM-061. notation record N

Display: $N = (glyph, spoken, definition, first_anchor, domain, collisions, see_also)$.

Local meaning. The retained editorial schema for a notation entry. It is not a numerical dimension.

Domain and source. Chapter 88 (explanatory anchor `export-persistence-cli-recipes-verification-and-publication`); frozen reference-source Markdown lines 117957-117963.

Spelling status. The exact display string occurs in the cited source chapter.

Concept: Coverage contract. **Other senses in this notation family:** NM-014, NM-040.

NM-062. glossary record G

Display: $G = (term, definition, package_meaning, classical_meaning, anchors, symbols)$.

Local meaning. The retained editorial glossary schema. It is not the operator quiver G.

Domain and source. Chapter 88 (explanatory anchor `export-persistence-cli-recipes-verification-and-publication`); frozen reference-source Markdown lines 117958-117964.

Spelling status. The exact display string occurs in the cited source chapter.

Concept: Coverage contract. No other notation record is assigned to this collision family.

NM-063. equivalent under a local relation

Display: $\sim$.

Local meaning. Read the relation's domain and binder or gauge convention. A generic equivalence glyph alone does not assert object identity.

Domain and source. Chapter 50 (explanatory anchor `typed-finite-expression-model`); frozen reference-source Markdown lines 43318-43324.

Spelling status. Normalized editorial display for the cited sense; not asserted to be a verbatim source formula.

Concept: Alpha equivalence. No other notation record is assigned to this collision family.

NM-064. tensor product

Display: $\otimes$.

Local meaning. Tensor product with declared factor order and coordinate flattening. Distinct from ordinary matrix multiplication.

Domain and source. Chapter 19 (explanatory anchor `spectral-packet-tensor-and-operator-transport`); frozen reference-source Markdown lines 14168-14172.

Spelling status. The exact display string occurs in the cited source chapter.

Concept: Tensor coordinate transport. **Other senses in this notation family:** NM-065.

NM-065. composition

Display: $\circ$.

Local meaning. Composition uses the declared source and target types and execution order. Written path order and matrix order must be reconciled.

Domain and source. Chapter 9 (explanatory anchor `typed-operator-grammars`); frozen reference-source Markdown lines 3989-3995.

Spelling status. The exact display string occurs in the cited source chapter.

Concept: Operator word. **Other senses in this notation family:** NM-064.

NM-066. big O operation bound

Display: $O(\cdot)$.

Local meaning. An implementation cost model under stated arithmetic assumptions, not a measured runtime or total memory guarantee.

Domain and source. Chapter K (explanatory anchor `appendix-k-resource-budgets`); frozen reference-source Markdown lines 108137-108143.

Spelling status. Normalized editorial display for the cited sense; not asserted to be a verbatim source formula.

Concept: Memory estimate. No other notation record is assigned to this collision family.

M.6 Executable meaning checks

All 24 examples below executed against a separately installed copy of the pinned wheel. They use small finite fixtures with assertions active. Several intentionally reproduce a negative or permissive implementation boundary. A satisfied expectation is not a newly successful soundness, security, platform or theorem qualification.

GM-P01. Exactness does not imply a valid certificate

Concepts: Exactness, Certificate, Validity flag.

Observed result.

```
{
  "exactness": "exact",
  "refusal": "ValidationError"
}
```

Expectation satisfied: yes. The exact executable function and its checked assertions are retained in `tools/block45/probes.py`. Runtime observations use tagged real/imaginary fields for complex numbers when necessary. These tags are only receipt formatting, not an additional portable mathematical codec.

GM-P02. Stored truthiness is not witness verification

Concepts: Validity flag, Certificate.

Observed result.

```
{
  "require_returned": true,
  "soundness_qualification": false,
  "stored_validity_type": "str"
}
```

Expectation satisfied: yes. The exact executable function and its checked assertions are retained in `tools/block45/probes.py`. Runtime observations use tagged real/imaginary fields for complex numbers when necessary. These tags are only receipt formatting, not an additional portable mathematical codec.

GM-P03. General packets can be oblique

Concepts: Packet system, Orthogonal projector, Projector.

Observed result.

```
{
  "hilbert_valid": false,
  "packet_valid": true
}
```

Expectation satisfied: yes. The exact executable function and its checked assertions are retained in `tools/block45/probes.py`. Runtime observations use tagged real/imaginary fields for complex numbers when necessary. These tags are only receipt formatting, not an additional portable mathematical codec.

GM-P04. Packet validation bypass remains visible

Concepts: Packet system, Orthogonal projector.

Observed result.


```
{
  "default_refuses": true,
  "repair_claim": false,
  "separate_hilbert_flag": true,
  "unchecked_packet_valid": false
}
```


Expectation satisfied: yes. The exact executable function and its checked assertions are retained in `tools/block45/probes.py`. Runtime observations use tagged real/imaginary fields for complex numbers when necessary. These tags are only receipt formatting, not an additional portable mathematical codec.

GM-P05. Package frame operator is SA, not A* A

Concepts: Frame operator, Analysis-synthesis system.

Observed result.


```
{
  "analysis_gram": [
    [
      2.0,
      1.0
    ],
    [
      1.0,
      2.0
    ]
  ],
  "reconstruction": [
    [
      0.9999999999999996,
      1.6653345369377348e-16
    ],
    [
      5.551115123125783e-17,
      1.0
    ]
  ]
}
```


Expectation satisfied: yes. The exact executable function and its checked assertions are retained in `tools/block45/probes.py`. Runtime observations use tagged real/imaginary fields for complex numbers when necessary. These tags are only receipt formatting, not an additional portable mathematical codec.

GM-P06. Coefficient projection is not the coefficient identity

Concepts: Coefficient projector, Synthesis map.

Observed result.


```
{
  "coefficient_dimension": 3,
  "full_coefficient_identity": false,
  "idempotent": true,
  "physical_dimension": 2
}
```


Expectation satisfied: yes. The exact executable function and its checked assertions are retained in `tools/block45/probes.py`. Runtime observations use tagged real/imaginary fields for complex numbers when necessary. These tags are only receipt formatting, not an additional portable mathematical codec.

GM-P07. Gauge changes preserve the synthesized vector

Concepts: Coefficient gauge, Coordinate vector.

Observed result.


```
{
  "different_coefficients_same_physical_vector": true,
  "kernel_vector": [
    -1.0,
    -1.0,
    1.0
  ]
}
```


Expectation satisfied: yes. The exact executable function and its checked assertions are retained in `tools/block45/probes.py`. Runtime observations use tagged real/imaginary fields for complex numbers when necessary. These tags are only receipt formatting, not an additional portable mathematical codec.

GM-P08. A reconstruction component need not be a projector

Concepts: Reconstruction component, Projector.

Observed result.


```
{
  "component": [
    [
      0.6666666666666663,
      0.0
    ],
    [
      -0.33333333333333326,
      0.0
    ]
  ],
  "components_reconstruct": true,
  "idempotent": false
}
```


Expectation satisfied: yes. The exact executable function and its checked assertions are retained in `tools/block45/probes.py`. Runtime observations use tagged real/imaginary fields for complex numbers when necessary. These tags are only receipt formatting, not an additional portable mathematical codec.

GM-P09. Coordinate composition multiplies the following map on the left

Concepts: Coordinate transform, Operator word.

Observed result.


```
{
  "matrix": [
    [
      "1",
      "1"
    ],
    [
      "1",
      "2"
    ]
  ],
  "source": "A",
  "target": "C"
}
```


Expectation satisfied: yes. The exact executable function and its checked assertions are retained in `tools/block45/probes.py`. Runtime observations use tagged real/imaginary fields for complex numbers when necessary. These tags are only receipt formatting, not an additional portable mathematical codec.

GM-P10. Transfer blocks use target then source

Concepts: Transfer block, Packet operator.

Observed result.


```
{
  "target_1_source_0": [
    [
      {
        "imag": 0.0,
        "real": 0.0
      },
      {
        "imag": 0.0,
        "real": 0.0
      }
    ],
    [
      {
        "imag": 0.0,
        "real": 3.0
      },
      {
        "imag": 0.0,
        "real": 0.0
      }
    ]
  ]
}
```


Expectation satisfied: yes. The exact executable function and its checked assertions are retained in `tools/block45/probes.py`. Runtime observations use tagged real/imaginary fields for complex numbers when necessary. These tags are only receipt formatting, not an additional portable mathematical codec.

GM-P11. Stored normalized and monomial coefficients differ

Concepts: Normalized coefficient, Weighted monomial basis.

Observed result.

```
{
  "ordinary": [
    "1",
    "2",
    "3"
  ],
  "stored": [
    "1",
    "2",
    "6"
  ]
}
```

Expectation satisfied: yes. The exact executable function and its checked assertions are retained in `tools/block45/probes.py`. Runtime observations use tagged real/imaginary fields for complex numbers when necessary. These tags are only receipt formatting, not an additional portable mathematical codec.

GM-P12. Exponential and ordinary products use different coefficient laws

Concepts: Hurwitz series, Convolution, Formal series.

Observed result.

```
{
  "exponential_stored": [
    "1",
    "2",
    "2"
  ],
  "ordinary_stored": [
    "1",
    "2",
    "1"
  ]
}
```

Expectation satisfied: yes. The exact executable function and its checked assertions are retained in `tools/block45/probes.py`. Runtime observations use tagged real/imaginary fields for complex numbers when necessary. These tags are only receipt formatting, not an additional portable mathematical codec.

GM-P13. Nonzero need not mean unit in a composite residue ring

Concepts: Residue ring, Unit, Prime field.

Observed result.

```
{
  "inverse_two_mod_seven": 4,
  "two_is_unit_mod_six": false
}
```

Expectation satisfied: yes. The exact executable function and its checked assertions are retained in `tools/block45/probes.py`. Runtime observations use tagged real/imaginary fields for complex numbers when necessary. These tags are only receipt formatting, not an additional portable mathematical codec.

GM-P14. Exact binary float conversion is not intended decimal rationalization

Concepts: Coercion, Exact arithmetic.

Observed result.


```
{
  "binary_exact_fraction": "3602879701896397/36028797018963968",
  "equal_one_tenth": false
}
```


Expectation satisfied: yes. The exact executable function and its checked assertions are retained in `tools/block45/probes.py`. Runtime observations use tagged real/imaginary fields for complex numbers when necessary. These tags are only receipt formatting, not an additional portable mathematical codec.

GM-P15. Zero valuation follows the selected finite-domain convention

Concepts: Valuation, Truncated p-adic value.

Observed result.


```
{
  "valuation_27": 3,
  "valuation_zero": "infinity"
}
```


Expectation satisfied: yes. The exact executable function and its checked assertions are retained in `tools/block45/probes.py`. Runtime observations use tagged real/imaginary fields for complex numbers when necessary. These tags are only receipt formatting, not an additional portable mathematical codec.

GM-P16. Residue lanes can have unequal lengths

Concepts: Lane, Residue coordinate.

Observed result.


```
{
  "channel_lengths": [
    3,
    2,
    2
  ],
  "reconstructed": true
}
```


Expectation satisfied: yes. The exact executable function and its checked assertions are retained in `tools/block45/probes.py`. Runtime observations use tagged real/imaginary fields for complex numbers when necessary. These tags are only receipt formatting, not an additional portable mathematical codec.

GM-P17. Nested positional addressing is not CRT reconstruction

Concepts: Mixed-radix address, CRT coordinates.

Observed result.


```
{
  "CRT_residue": 1,
  "common_modulus": 6,
  "nested_address": 3
}
```


Expectation satisfied: yes. The exact executable function and its checked assertions are retained in `tools/block45/probes.py`. Runtime observations use tagged real/imaginary fields for complex numbers when necessary. These tags are only receipt formatting, not an additional portable mathematical codec.

GM-P18. Typed exact identity has a finite matrix normal form

Concepts: Normal form, Basis witness, Typed expression.

Observed result.


```
{
  "certificate_valid": true,
  "normal_form": [
    [
      "1",
      "0"
    ],
    [
      "0",
      "1"
    ]
  ]
}
```


Expectation satisfied: yes. The exact executable function and its checked assertions are retained in `tools/block45/probes.py`. Runtime observations use tagged real/imaginary fields for complex numbers when necessary. These tags are only receipt formatting, not an additional portable mathematical codec.

GM-P19. Portable loading preserves an invalid stored certificate

Concepts: Portable record, Certificate.

Observed result.


```
{
  "certificate_valid": false,
  "roundtrip_succeeded": true
}
```


Expectation satisfied: yes. The exact executable function and its checked assertions are retained in `tools/block45/probes.py`. Runtime observations use tagged real/imaginary fields for complex numbers when necessary. These tags are only receipt formatting, not an additional portable mathematical codec.

GM-P20. Package and codec versions are different fields

Concepts: Codec, Migration, Package upgrade.

Observed result.


```
{
  "codec": 1,
  "package": "1.0.0b4",
  "tag": "python.fraction"
}
```


Expectation satisfied: yes. The exact executable function and its checked assertions are retained in `tools/block45/probes.py`. Runtime observations use tagged real/imaginary fields for complex numbers when necessary. These tags are only receipt formatting, not an additional portable mathematical codec.

GM-P21. Adjoint is not transpose on complex data

Concepts: Adjoint, Orthogonal projector.

Observed result.

```
{
  "transpose_equals_adjoint": false
}
```

Expectation satisfied: yes. The exact executable function and its checked assertions are retained in `tools/block45/probes.py`. Runtime observations use tagged real/imaginary fields for complex numbers when necessary. These tags are only receipt formatting, not an additional portable mathematical codec.

GM-P22. Stable facade identity is not an exactness claim

Concepts: Stable API, Alias.

Observed result.

```
{
  "finite_frame_exactness": "numerical",
  "identical_facade_class": true
}
```

Expectation satisfied: yes. The exact executable function and its checked assertions are retained in `tools/block45/probes.py`. Runtime observations use tagged real/imaginary fields for complex numbers when necessary. These tags are only receipt formatting, not an additional portable mathematical codec.

GM-P23. A zero generator gives finite semigroup identity

Concepts: Matrix dynamics, Fundamental matrix.

Observed result.

```
{
  "dimension": 2,
  "finite_times": [
    0.2,
    0.3,
    0.5
  ],
  "infinite_model_claim": false
}
```

Expectation satisfied: yes. The exact executable function and its checked assertions are retained in `tools/block45/probes.py`. Runtime observations use tagged real/imaginary fields for complex numbers when necessary. These tags are only receipt formatting, not an additional portable mathematical codec.

GM-P24. Object-array buffer size is not complete exact-object memory

Concepts: Memory estimate, Resource budget.

Observed result.

```
{
  "buffer_bytes": 16,
  "pointer_size": 8,
  "scalar_payload_in_nbytes": false
}
```

Expectation satisfied: yes. The exact executable function and its checked assertions are retained in `tools/block45/probes.py`. Runtime observations use tagged real/imaginary fields for complex numbers when necessary. These tags are only receipt formatting, not an additional portable mathematical codec.

M.7 Complete lexical formula concordance

`Block45_Math_Formula_Concordance.jsonl` contains **6,411 occurrence records** from the complete frozen glossary-source Markdown. Each record stores the exact delimited LaTeX span, byte-independent Python string offsets, 1-based source line range, enclosing heading and SHA-256 of the UTF-8 span. Repeated formulas remain separate occurrences so that different contexts are not collapsed.

The scanner excludes fenced code, inline backtick code and HTML comments, preserving their offsets as blanks. Its documented delimiters are backslash-parentheses, backslash-brackets, paired double dollars and single-line paired single dollars. This is a lexical concordance, not a full TeX parser, proof-term inventory or claim to recognize formulas without those delimiters. A table cell or historical planning equation is indexed when it satisfies the same rule. Individual bare variable mentions and code identifiers are outside this formula-spanning census.

For formula lookup, inspect the JSONL record's `start_line` and `end_line` in `Komorebi_Math_User_Guide_Block44_Appendix_L.md`. Do not apply those source offsets to a later edited edition. The index binds both reader-source and TeX checksums. The two smaller TSV files provide term and notation lookup without loading the full concordance.

M.8 Validation, regeneration and closure

The unchanged source suite passed **1,203 tests**. The installed glossary/reference suite passed **675 tests** and checked all source entries, links, collision groups, semantic examples and formula offsets/hashes. These counts overlap; the 24 semantic cases are also included in the reference suite. No count is a number of new mathematical theorems.

```
python tools/block45/inventory.py
python tools/block45/probes.py --output <fresh-replay-directory>
python tools/block45/run_reference_tests.py
python tools/block45/run_source_tests.py
python tools/block45/generate_manual.py
python tools/block45/build_pdfs.py
python VERIFY_BLOCK45_PACKAGE.py
```

Regenerate in a working copy. The reference runner checks that imported package bytes equal the pinned source and originate outside the checkout. Dependencies remain inherited, not freshly resolved in a dependency-isolated environment. Changing source, editorial data, index or receipts invalidates the corresponding evidence and requires new validation. A lookup-only use does not alter the artifacts.

Reference boundaries. The terminology catalog contains the stated 223 terms and 66 notation senses, together with its lexical concordance. These records do not repair certificate, portable-decoder, resource-limit, GPU or platform behavior. The recorded audit did not qualify Windows/PowerShell, other Python/dependency versions, CUDA, strict MkDocs, fully dependency-isolated installation, final publication or new Lean proofs. Appendix N provides the documentation-source crosswalk, and Appendix O identifies the separate final-source execution records.

Appendix N. Documentation-source crosswalk

The crosswalk is a reading and source-navigation contract for **Komorebi Math 1.0.0b4, Repair R2, wheel build 2**. It names every source in the configured documentation directory, identifies the current place to read its subject, preserves the historical role of older text, and records specific corrections without changing the pinned mathematical library.

Use the crosswalk when two documents seem to disagree. Find the file in N.3, read its status and resolution, then follow the listed destination. A "corrected" record also points to a specific explanation in N.4. This route lets you use an older document without guessing which of its statements still applies.

Read a record in this order: source path, reading status, destination, resolution, then source identity. The path tells you which text is being discussed; the status explains how to read it; the destination supplies the integrated explanation. The counts at the end help locate material within the source and need not be studied before following the reading route.

For example, the persistence source has explicit corrections for its envelope and migration keyword. Follow its correction records to Appendix G before adapting the old example. For a question about tests in the final source, go to Appendix O instead of treating an older documentation test count as a current execution result.

N.1 Coverage and source precedence

Surface	Recorded coverage
Markdown sources under configured docs directory	71
Additional repository-root Markdown sources	16
Unique source rows	87
Configured navigation entries / distinct targets	57 / 57
Site sources not in navigation	14
Explicit reviewed correction records	13
Export / module / command / example joins	1,526 / 300 / 46 / 65
Code blocks retained verbatim	301
Direct command-line occurrences	236
Lexical assertion candidates	716

The primary invariant is one and only one crosswalk row for each Markdown source below the configured `docs_dir`. This includes unlisted pages: navigation placement is not the document census. Sixteen repository-root Markdown files are an explicitly separate supplement. Release-directory reports, arbitrary third-party documents and all historical manual snapshots are not silently included as new primary rows.

Each record implements the retained design tuple: path, snapshot version, reading status, target anchors, source claims, commands, stale selectors and resolution. It additionally binds source lines, SHA-256, headings, link paths, imports and code blocks. Claims are **source assertion candidates**, recognized by a declared lexical rule. They are neither endorsed truth nor a theorem-to-proof relation. Test and verification attribution remains Appendix O (anchor `appendix-o-test-verification-crosswalk`).

Reading status	Meaning	Rows
current	Retained current policy or entry point; execution claims still need dated evidence	7
merged	Subject is integrated into the linked manual destinations	18
historical	Statements are scoped to the original named release or recorded event	45
superseded	Another pinned reference controls the present interface or workflow	4
corrected	Read with the explicit correction or precedence instruction in this crosswalk	13

Corrected is a reading instruction, not a claim that the retained source file was edited. Every R2 library byte is unchanged. The current R2 repair notice controls artifact selection, the source and manifests control interface identity, and current hash-bound execution records control local qualification. A historical positive count is not refreshed merely because the same file ships with a newer manual.

Line numbers identify a fixed source. Line-bound destinations refer to the frozen predecessor Markdown and LaTeX in the reference bundle. Page positions in the typeset book do not replace the recorded source offsets. Cumulative PDF references resolve to the current integrated chapters; the standalone volume names those destinations in the cumulative manual. Full source hashes are in the JSON; abbreviated hashes below are navigation aids, not complete integrity checks.

N.2 Navigation and implementation joins

All 57 configured navigation targets exist. The following 14 additional pages are indexed even though they are not listed in `mkdocs.yml`:

Unlisted source	Reading status
`docs/API_INVENTORY.md`	superseded
`docs/BLOCK_1_RELEASE.md`	historical
`docs/BLOCK_2_RELEASE.md`	historical
`docs/BLOCK_3_RELEASE.md`	historical
`docs/BLOCK_4_RELEASE.md`	historical
`docs/BLOCK_5_RELEASE.md`	historical
`docs/BLOCK_6_RELEASE.md`	historical
`docs/BLOCK_7_RELEASE.md`	historical
`docs/BLOCK_8_RELEASE.md`	historical
`docs/EXTENSION_API.md`	corrected
`docs/NINTH_MAJOR_BLOCK_REPORT.md`	historical
`docs/OPTIONAL_BACKEND_ISOLATION.md`	merged
`docs/POWERSHELL_TESTS.md`	superseded
`docs/STABLE_API.md`	superseded

Unlisted does not mean intentionally unpublished or unbuilt. That behavior belongs to the site generator. The recorded crosswalk audit did not run strict MkDocs and does not infer a production-site accessibility guarantee from this static inventory.

The API join retains all 131 stable, 83 beta and 1,312 research export records. Each export keeps its tier, public route, reported defining owner, source identity where package-owned, and its existing unique A-C anchor. The 300 module rows join package source files to Appendix D. The 46 command rows join handler sites to Appendix F, and the 65 example rows join script hashes to Appendix E. These joins are checked against actual separately installed objects and retained source, not guessed from matching display names.

Do not merge constants by their reflected type. A data export reported under `builtins.str`, `builtins.int` or another external type owner is still keyed by its public export route. Its reported owner does not mean the package constant was defined in that external module. Aliases, same-name concepts and data-type ownership are different relations. Lexical import matches provide navigation only and do not prove that a source page explains every behavior of that object.

N.3 Source record catalog

Every primary and supplemental source appears once below. Target references identify the authoritative reading destinations, not an assertion that every sentence was copied verbatim. Historical release cards keep their original chronology. Source assertion candidates and complete command lines are preserved electronically to avoid replacing them with an unsupported summary of proof coverage.

DN-001. `CHANGELOG.md`

Status: historical. **Scope:** repository-prose. **Navigation:** not listed.

Destinations: Chapter 72 (anchor `api-semantic-persistence-and-upgrade-contracts`), Chapter 74 (anchor `block-21-publication-handoff`), Appendix L (anchor `appendix-l-migrations`).

Resolution. Chronological changes are history. Each version heading scopes its statements; do not read every old current/count token as a present claim.

Source record: 396 lines; SHA-256 prefix `4d9e2ef5698cec59`. 81 lexical assertion candidates, 0 indexed command occurrences, 0 code blocks and 5 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-002. `CONTRIBUTING.md`

Status: current. **Scope:** repository-prose. **Navigation:** not listed.

Destinations: Chapter 62 (anchor `chapter-62.-provenance-registries-and-extensions`), Chapter 74 (anchor `block-21-publication-handoff`), Chapter 82 (anchor `final-guide-publication-gate-and-retained-architecture`).

Resolution. Repository contribution policy is retained; no new public mutation or workflow execution is authorized by indexing it.

Source record: 26 lines; SHA-256 prefix `317c4476106460e2`. 1 lexical assertion candidate, 0 indexed command occurrences, 2 code blocks and 3 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-003. `LICENSE.md`

Status: current. **Scope:** repository-prose. **Navigation:** not listed.

Destinations: Chapter 74 (anchor `block-21-publication-handoff`), Chapter 82 (anchor `final-guide-publication-gate-and-retained-architecture`).

Resolution. License text is preserved verbatim and remains controlling. No legal interpretation or new license permission is supplied by this reference.

Source record: 7 lines; SHA-256 prefix `1a18635621db5832`. 0 lexical assertion candidates, 0 indexed command occurrences, 0 code blocks and 0 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-004. `PACKAGE_AUDIT.md`

Status: historical. **Scope:** repository-prose. **Navigation:** not listed.

Destinations: Chapter 73 (anchor `platform-and-release-qualification`), Chapter 74 (anchor `block-21-publication-handoff`), Appendix L (anchor `appendix-l-migrations`).

Resolution. Recorded package-audit counts and test results are retained historical evidence, not the current source-suite totals.

Source record: 31 lines; SHA-256 prefix `c69a08734e1c90df`. 2 lexical assertion candidates, 0 indexed command occurrences, 0 code blocks and 0 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-005. `README.md`

Status: corrected. **Scope:** repository-prose. **Navigation:** not listed.

Destinations: Chapter 2 (anchor `what-komorebi-math-is`), Chapter 3 (anchor `installation-and-environment-profiles`), Chapter 7 (anchor `public-api-tiers`), Appendix L (anchor `appendix-l-migrations`).

Resolution. The leading R2 repair banner controls artifact selection. Lower untagged wheel examples and alpha-era material remain historical.

Source record: 765 lines; SHA-256 prefix `4fbe47f0df7d9365`. 42 lexical assertion candidates, 62 indexed command occurrences, 38 code blocks and 75 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

Specific corrections: CN-13.

DN-006. `README_BLOCK16.md`

Status: historical. **Scope:** repository-prose. **Navigation:** not listed.

Destinations: Chapter 7 (anchor `public-api-tiers`), Chapter 20 (anchor `analysis-synthesis-and-redundant-frames`), Chapter 72 (anchor `api-semantic-persistence-and-upgrade-contracts`), Chapter 73 (anchor `platform-and-release-qualification`), Appendix L (anchor `appendix-l-migrations`).

Resolution. Retained block handoff describes that numbered revision. R2 repair instructions control the current artifact selection.

Source record: 46 lines; SHA-256 prefix `95c04507dfe6828c`. 2 lexical assertion candidates, 7 indexed command occurrences, 4 code blocks and 0 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-007. `README_BLOCK17.md`

Status: historical. **Scope:** repository-prose. **Navigation:** not listed.

Destinations: Chapter 7 (anchor `public-api-tiers`), Chapter 72 (anchor `api-semantic-persistence-and-upgrade-contracts`), Chapter 73 (anchor `platform-and-release-qualification`), Appendix L (anchor `appendix-l-migrations`).

Resolution. Retained block handoff describes that numbered revision. R2 repair instructions control the current artifact selection.

Source record: 172 lines; SHA-256 prefix `c500851084c3df4b`. 10 lexical assertion candidates, 12 indexed command occurrences, 15 code blocks and 5 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-008. `README_BLOCK18.md`

Status: historical. **Scope:** repository-prose. **Navigation:** not listed.

Destinations: Chapter 20 (anchor `analysis-synthesis-and-redundant-frames`), Appendix L (anchor `appendix-l-migrations`).

Resolution. Retained block handoff describes that numbered revision. R2 repair instructions control the current artifact selection.

Source record: 32 lines; SHA-256 prefix `09105a93e5eea3e9`. 1 lexical assertion candidate, 0 indexed command occurrences, 0 code blocks and 0 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-009. `README_BLOCK19.md`

Status: historical. **Scope:** repository-prose. **Navigation:** not listed.

Destinations: Chapter 47 (anchor `measures-and-exact-polynomials`), Chapter 48 (anchor `integration-providers-and-quadrature`), Chapter 49 (anchor `integral-identities-and-coordinate-covectors`), Appendix L (anchor `appendix-l-migrations`).

Resolution. Retained block handoff describes that numbered revision. R2 repair instructions control the current artifact selection.

Source record: 38 lines; SHA-256 prefix `2d5bcb4762ebffe9`. 5 lexical assertion candidates, 2 indexed command occurrences, 2 code blocks and 0 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-010. `README_BLOCK20.md`

Status: historical. **Scope:** repository-prose. **Navigation:** not listed.

Destinations: Chapter 50 (anchor `typed-finite-expression-model`), Chapter 51 (anchor `expression-evaluation-and-certification`), Chapter 52 (anchor `expression-planning-and-persistence`), Appendix L (anchor `appendix-l-migrations`).

Resolution. Retained block handoff describes that numbered revision. R2 repair instructions control the current artifact selection.

Source record: 84 lines; SHA-256 prefix `4ef56d3b562351a2`. 12 lexical assertion candidates, 0 indexed command occurrences, 1 code block and 0 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-011. `README_BLOCK21.md`

Status: historical. **Scope:** repository-prose. **Navigation:** not listed.

Destinations: Chapter 74 (anchor `block-21-publication-handoff`), Appendix L (anchor `appendix-l-migrations`).

Resolution. Retained block handoff describes that numbered revision. R2 repair instructions control the current artifact selection.

Source record: 36 lines; SHA-256 prefix `d7fd766f23b73f2b`. 1 lexical assertion candidate, 0 indexed command occurrences, 3 code blocks and 0 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-012. `RELEASE_REPAIR_R1.md`

Status: superseded. **Scope:** repository-prose. **Navigation:** not listed.

Destinations: Chapter 74 (anchor `block-21-publication-handoff`), Appendix L (anchor `appendix-l-migrations`).

Resolution. R1 is superseded by R2. Retain its diagnosis but do not install build 1 as the current artifact.

Source record: 28 lines; SHA-256 prefix `0f7e278a7f03bb29`. 4 lexical assertion candidates, 0 indexed command occurrences, 1 code block and 0 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-013. `RELEASE_REPAIR_R2.md`

Status: current. **Scope:** repository-prose. **Navigation:** not listed.

Destinations: Chapter 74 (anchor `block-21-publication-handoff`), Appendix L (anchor `appendix-l-migrations`).

Resolution. Authoritative retained repair identity and build-tag-2 artifact instructions. Old positive records do not qualify later platform executions.

Source record: 50 lines; SHA-256 prefix `811343fe6e7a2cd3`. 7 lexical assertion candidates, 0 indexed command occurrences, 0 code blocks and 0 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-014. `REPRODUCIBILITY.md`

Status: current. **Scope:** repository-prose. **Navigation:** not listed.

Destinations: Chapter 70 (anchor `chapter-70.-export-and-reproducibility-recipes`), Chapter 74 (anchor `block-21-publication-handoff`), Chapter 81 (anchor `site-and-pdf-build-closure`).

Resolution. Reproducibility instructions are a workflow, not a claim that the crosswalk audit executed every gate. The package and platform boundaries remain explicit.

Source record: 59 lines; SHA-256 prefix `309431e1eb915086`. 4 lexical assertion candidates, 0 indexed command occurrences, 6 code blocks and 0 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-015. `SECURITY.md`

Status: current. **Scope:** repository-prose. **Navigation:** not listed.

Destinations: Chapter 61 (anchor `chapter-61.-portable-persistence`), Chapter 62 (anchor `chapter-62.-provenance-registries-and-extensions`), Appendix G (anchor `appendix-g-portable-schemas`), Appendix H (anchor `appendix-h-certificate-theorem-index`), Appendix I (anchor `appendix-i-exceptions-diagnostics`).

Resolution. Security-reporting policy is retained verbatim. This crosswalk neither hardens the reader nor changes disclosure permissions.

Source record: 17 lines; SHA-256 prefix `48dd3cbf84e6482e`. 2 lexical assertion candidates, 0 indexed command occurrences, 0 code blocks and 0 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-016. `STATIC_AUDIT.md`

Status: historical. **Scope:** repository-prose. **Navigation:** not listed.

Destinations: Chapter 73 (anchor `platform-and-release-qualification`), Chapter 74 (anchor `block-21-publication-handoff`), Appendix L (anchor `appendix-l-migrations`).

Resolution. Recorded static-audit results are historical. Hash-bound current checks govern the delivered snapshot.

Source record: 20 lines; SHA-256 prefix `85670e693e47cb79`. 0 lexical assertion candidates, 0 indexed command occurrences, 0 code blocks and 0 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-017. `docs/ANALYSIS_SYNTHESIS_SYSTEMS.md`

Status: merged. **Scope:** mkdocs-source. **Navigation:** listed.

Destinations: Chapter 20 (anchor `analysis-synthesis-and-redundant-frames`), Appendix H (anchor `appendix-h-certificate-theorem-index`), Appendix M (anchor `appendix-m-glossary-notation`).

Resolution. Finite analysis/synthesis exposition is integrated in Chapter 20; H and M clarify the actual certificate and frame-operator conventions.

Source record: 88 lines; SHA-256 prefix `df2f841e733979a5`. 1 lexical assertion candidate, 0 indexed command occurrences, 4 code blocks and 4 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-018. `docs/API_COMPATIBILITY_0.10.md`

Status: historical. **Scope:** mkdocs-source. **Navigation:** listed.

Destinations: Chapter 7 (anchor `public-api-tiers`), Chapter 72 (anchor `api-semantic-persistence-and-upgrade-contracts`), Appendix L (anchor `appendix-l-migrations`).

Resolution. Version-bound structural policy is retained history. Current export, signature and semantic contracts are separately indexed.

Source record: 51 lines; SHA-256 prefix `5b59d694fb781bc6`. 3 lexical assertion candidates, 3 indexed command occurrences, 1 code block and 0 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-019. `docs/API_COMPATIBILITY_0.11.md`

Status: historical. **Scope:** mkdocs-source. **Navigation:** listed.

Destinations: Chapter 7 (anchor `public-api-tiers`), Chapter 72 (anchor `api-semantic-persistence-and-upgrade-contracts`), Appendix L (anchor `appendix-l-migrations`).

Resolution. Version-bound structural policy is retained history. Current export, signature and semantic contracts are separately indexed.

Source record: 92 lines; SHA-256 prefix `7a5499d26b868372`. 5 lexical assertion candidates, 5 indexed command occurrences, 4 code blocks and 3 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-020. `docs/API_COMPATIBILITY_0.12.md`

Status: historical. **Scope:** mkdocs-source. **Navigation:** listed.

Destinations: Chapter 7 (anchor `public-api-tiers`), Chapter 72 (anchor `api-semantic-persistence-and-upgrade-contracts`), Appendix L (anchor `appendix-l-migrations`).

Resolution. Version-bound structural policy is retained history. Current export, signature and semantic contracts are separately indexed.

Source record: 24 lines; SHA-256 prefix `2e64f013a202d543`. 1 lexical assertion candidate, 2 indexed command occurrences, 2 code blocks and 4 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-021. `docs/API_COMPATIBILITY_0.13.md`

Status: historical. **Scope:** mkdocs-source. **Navigation:** listed.

Destinations: Chapter 7 (anchor `public-api-tiers`), Chapter 72 (anchor `api-semantic-persistence-and-upgrade-contracts`), Appendix L (anchor `appendix-l-migrations`).

Resolution. Version-bound structural policy is retained history. Current export, signature and semantic contracts are separately indexed.

Source record: 22 lines; SHA-256 prefix `7550dfac3da7fad6`. 1 lexical assertion candidate, 2 indexed command occurrences, 1 code block and 5 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-022. `docs/API_COMPATIBILITY_0.14.md`

Status: historical. **Scope:** mkdocs-source. **Navigation:** listed.

Destinations: Chapter 7 (anchor `public-api-tiers`), Chapter 72 (anchor `api-semantic-persistence-and-upgrade-contracts`), Appendix L (anchor `appendix-l-migrations`).

Resolution. Version-bound structural policy is retained history. Current export, signature and semantic contracts are separately indexed.

Source record: 9 lines; SHA-256 prefix `ccd29a6da86730ad`. 0 lexical assertion candidates, 0 indexed command occurrences, 0 code blocks and 2 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-023. docs/API_COMPATIBILITY_0.15.md

Status: historical. **Scope:** mkdocs-source. **Navigation:** listed.

Destinations: Chapter 7 (anchor `public-api-tiers`), Chapter 72 (anchor `api-semantic-persistence-and-upgrade-contracts`), Appendix L (anchor `appendix-l-migrations`).

Resolution. Version-bound structural policy is retained history. Current export, signature and semantic contracts are separately indexed.

Source record: 32 lines; SHA-256 prefix `80781528b1197af1`. 2 lexical assertion candidates, 0 indexed command occurrences, 0 code blocks and 3 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-024. docs/API_INVENTORY.md

Status: superseded. **Scope:** mkdocs-source. **Navigation:** not listed.

Destinations: Appendix A (anchor `appendix-a.-stable-api-131-entry-frozen-reference`), Appendix B (anchor `appendix-b.-beta-api-83-entry-reference`), Appendix C (anchor `appendix-c.-research-api-1312-entry-reference`), Appendix D (anchor `appendix-d.-module-reference-ledger`).

Resolution. The thematic inventory is not the current exhaustive contract. Follow the source-bound 1,526 export records and module ledger.

Source record: 537 lines; SHA-256 prefix `344dcf267775330f`. 7 lexical assertion candidates, 0 indexed command occurrences, 0 code blocks and 14 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-025. docs/API_STABILITY_REVIEW.md

Status: historical. **Scope:** mkdocs-source. **Navigation:** listed.

Destinations: Chapter 7 (anchor `public-api-tiers`), Chapter 37 (anchor `derivative-providers-and-automatic-differentiation`), Appendix L (anchor `appendix-l-migrations`).

Resolution. Retain the explicitly historical review. Do not use its successor link to 0.10 as the latest API contract.

Source record: 90 lines; SHA-256 prefix `975dc242cd54254f`. 2 lexical assertion candidates, 0 indexed command occurrences, 2 code blocks and 3 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-026. docs/BEHAVIORAL_GOLDEN_AND_PROFILING.md

Status: merged. **Scope:** mkdocs-source. **Navigation:** listed.

Destinations: Chapter 43 (anchor `nonlinear-execution-streaming-profiling-and-gpu-paths`), Chapter 72 (anchor `api-semantic-persistence-and-upgrade-contracts`), Appendix K (anchor `appendix-k-resource-budgets`).

Resolution. Closed behavioral fixtures and finite profiling are integrated. Sampling, traced allocation and complete process memory remain different measures.

Source record: 70 lines; SHA-256 prefix `4f66f14481d9df56`. 4 lexical assertion candidates, 0 indexed command occurrences, 3 code blocks and 2 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-027. docs/BETA_TO_RC.md

Status: corrected. **Scope:** mkdocs-source. **Navigation:** listed.

Destinations: Chapter 73 (anchor `platform-and-release-qualification`), Appendix J (anchor `appendix-j-backends-dependencies`), Appendix L (anchor `appendix-l-migrations`).

Resolution. Retain the gate distinction; the sentence naming 1.0.0b1 is a historical example. Apply current evidence to the exact R2 artifact.

Source record: 30 lines; SHA-256 prefix `03630d50ccdbbd85`. 4 lexical assertion candidates, 2 indexed command occurrences, 3 code blocks and 0 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

Specific corrections: CN-11.

DN-028. `docs/BLOCK_10_RELEASE.md`

Status: historical. **Scope:** mkdocs-source. **Navigation:** listed.

Destinations: Chapter 37 (anchor `derivative-providers-and-automatic-differentiation`), Chapter 38 (anchor `packet-jets-and-fauxe0-di-bruno-composition`), Chapter 43 (anchor `nonlinear-execution-streaming-profiling-and-gpu-paths`), Chapter 61 (anchor `chapter-61.-portable-persistence`), Appendix L (anchor `appendix-l-migrations`).

Resolution. Release-era scope and counts are retained history. Follow the current linked domain chapters and compatibility records.

Source record: 123 lines; SHA-256 prefix `bd30ed6232943eaf`. 9 lexical assertion candidates, 5 indexed command occurrences, 4 code blocks and 2 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-029. `docs/BLOCK_11_RELEASE.md`

Status: historical. **Scope:** mkdocs-source. **Navigation:** listed.

Destinations: Chapter 43 (anchor `nonlinear-execution-streaming-profiling-and-gpu-paths`), Chapter 61 (anchor `chapter-61.-portable-persistence`), Chapter 72 (anchor `api-semantic-persistence-and-upgrade-contracts`), Appendix L (anchor `appendix-l-migrations`).

Resolution. Release-era scope and counts are retained history. Follow the current linked domain chapters and compatibility records.

Source record: 42 lines; SHA-256 prefix `9f930c82081776ae`. 2 lexical assertion candidates, 0 indexed command occurrences, 0 code blocks and 1 qualified lexical/import reference. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-030. `docs/BLOCK_12_RELEASE.md`

Status: historical. **Scope:** mkdocs-source. **Navigation:** listed.

Destinations: Chapter 31 (anchor `signal-conventions-windows-and-framing`), Chapter 32 (anchor `time-frequency-and-spectral-analysis`), Chapter 33 (anchor `filtering-and-convolution`), Chapter 34 (anchor `resampling-and-adaptive-filters`), Chapter 35 (anchor `filter-banks-wavelets-and-packet-trees`), Appendix L (anchor `appendix-l-migrations`).

Resolution. Release-era scope and counts are retained history. Follow the current linked domain chapters and compatibility records.

Source record: 42 lines; SHA-256 prefix `81b1e54792735e8b`. 1 lexical assertion candidate, 4 indexed command occurrences, 1 code block and 11 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-031. `docs/BLOCK_13_RELEASE.md`

Status: historical. **Scope:** mkdocs-source. **Navigation:** listed.

Destinations: Chapter 21 (anchor `formal-series-and-weighted-bases`), Chapter 22 (anchor `normalized-generating-functions`), Chapter 23 (anchor `masks-cartier-operators-and-digit-algebra`), Chapter 24 (anchor `prime-power-bell-cyclotomic-and-q-kummer-systems`), Appendix L (anchor `appendix-l-migrations`).

Resolution. Release-era scope and counts are retained history. Follow the current linked domain chapters and compatibility records.

Source record: 56 lines; SHA-256 prefix `73d2295c2a25e4dc`. 6 lexical assertion candidates, 7 indexed command occurrences, 2 code blocks and 1 qualified lexical/import reference. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-032. `docs/BLOCK_14_RELEASE.md`

Status: historical. **Scope:** mkdocs-source. **Navigation:** listed.

Destinations: Chapter 15 (anchor `coordinate-objects`), Chapter 16 (anchor `exact-and-numerical-coordinate-transforms`), Chapter 17 (anchor `polynomial-and-classical-coordinate-families`), Chapter 18 (anchor `combinatorial-recurrence-and-state-space-coordinates`), Chapter 19 (anchor `spectral-packet-tensor-and-operator-transport`), Appendix L (anchor `appendix-l-migrations`).

Resolution. Release-era scope and counts are retained history. Follow the current linked domain chapters and compatibility records.

Source record: 41 lines; SHA-256 prefix `37adc614be29f184`. 1 lexical assertion candidate, 12 indexed command occurrences, 3 code blocks and 1 qualified lexical/import reference. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-033. `docs/BLOCK_15_RELEASE.md`

Status: historical. **Scope:** mkdocs-source. **Navigation:** listed.

Destinations: Chapter 14 (anchor `polyphase-systems-and-refinement`), Chapter 72 (anchor `api-semantic-persistence-and-upgrade-contracts`), Appendix L (anchor `appendix-l-migrations`).

Resolution. Release-era scope and counts are retained history. Follow the current linked domain chapters and compatibility records.

Source record: 81 lines; SHA-256 prefix `122f41023d2c4432`. 2 lexical assertion candidates, 0 indexed command occurrences, 1 code block and 4 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-034. `docs/BLOCK_16_RELEASE.md`

Status: historical. **Scope:** mkdocs-source. **Navigation:** listed.

Destinations: Chapter 7 (anchor `public-api-tiers`), Chapter 20 (anchor `analysis-synthesis-and-redundant-frames`), Chapter 72 (anchor `api-semantic-persistence-and-upgrade-contracts`), Chapter 73 (anchor `platform-and-release-qualification`), Appendix L (anchor `appendix-l-migrations`).

Resolution. Release-era scope and counts are retained history. Follow the current linked domain chapters and compatibility records.

Source record: 199 lines; SHA-256 prefix `96883b27a024fd2d`. 10 lexical assertion candidates, 7 indexed command occurrences, 5 code blocks and 3 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-035. `docs/BLOCK_17_RELEASE.md`

Status: historical. **Scope:** mkdocs-source. **Navigation:** listed.

Destinations: Chapter 7 (anchor `public-api-tiers`), Chapter 72 (anchor `api-semantic-persistence-and-upgrade-contracts`), Chapter 73 (anchor `platform-and-release-qualification`), Appendix L (anchor `appendix-l-migrations`).

Resolution. Release-era scope and counts are retained history. Follow the current linked domain chapters and compatibility records.

Source record: 42 lines; SHA-256 prefix `54c0b7073d095eb1`. 4 lexical assertion candidates, 0 indexed command occurrences, 1 code block and 0 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-036. docs/BLOCK_18_RELEASE.md

Status: historical. **Scope:** mkdocs-source. **Navigation:** listed.

Destinations: Chapter 20 (anchor `analysis-synthesis-and-redundant-frames`), Appendix L (anchor `appendix-l-migrations`).

Resolution. Release-era scope and counts are retained history. Follow the current linked domain chapters and compatibility records.

Source record: 24 lines; SHA-256 prefix `599fca1dd8939338`. 3 lexical assertion candidates, 0 indexed command occurrences, 0 code blocks and 0 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-037. docs/BLOCK_19_RELEASE.md

Status: historical. **Scope:** mkdocs-source. **Navigation:** listed.

Destinations: Chapter 47 (anchor `measures-and-exact-polynomials`), Chapter 48 (anchor `integration-providers-and-quadrature`), Chapter 49 (anchor `integral-identities-and-coordinate-covectors`), Appendix L (anchor `appendix-l-migrations`).

Resolution. Release-era scope and counts are retained history. Follow the current linked domain chapters and compatibility records.

Source record: 116 lines; SHA-256 prefix `66632da57ccafac2`. 16 lexical assertion candidates, 3 indexed command occurrences, 3 code blocks and 0 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-038. docs/BLOCK_19_WORKED_EXAMPLE.md

Status: merged. **Scope:** mkdocs-source. **Navigation:** listed.

Destinations: Chapter 47 (anchor `measures-and-exact-polynomials`), Chapter 48 (anchor `integration-providers-and-quadrature`), Chapter 49 (anchor `integral-identities-and-coordinate-covectors`), Chapter 68 (anchor `chapter-68.-integration-and-expression-recipes`), Appendix E (anchor `appendix-e-executable-example-records`).

Resolution. Finite integration example is integrated; its exact/conditional/error classes remain local to the integrand and provider.

Source record: 144 lines; SHA-256 prefix `853ea9d60518c610`. 13 lexical assertion candidates, 0 indexed command occurrences, 2 code blocks and 0 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-039. docs/BLOCK_1_RELEASE.md

Status: historical. **Scope:** mkdocs-source. **Navigation:** not listed.

Destinations: Chapter 9 (anchor `typed-operator-grammars`), Chapter 10 (anchor `operator-algebras-and-seed-systems`), Chapter 11 (anchor `packet-systems-and-transfer-blocks`), Appendix L (anchor `appendix-l-migrations`).

Resolution. Release-era scope and counts are retained history. Follow the current linked domain chapters and compatibility records.

Source record: 110 lines; SHA-256 prefix `9a6da3c01a7c3b71`. 9 lexical assertion candidates, 0 indexed command occurrences, 1 code block and 0 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-040. docs/BLOCK_20_RELEASE.md

Status: historical. **Scope:** mkdocs-source. **Navigation:** listed.

Destinations: Chapter 50 (anchor `typed-finite-expression-model`), Chapter 51 (anchor `expression-evaluation-and-certification`), Chapter 52 (anchor `expression-planning-and-persistence`), Appendix L (anchor `appendix-l-migrations`).

Resolution. Release-era scope and counts are retained history. Follow the current linked domain chapters and compatibility records.

Source record: 183 lines; SHA-256 prefix `f3eda735cc995c81`. 19 lexical assertion candidates, 0 indexed command occurrences, 1 code block and 0 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-041. `docs/BLOCK_20_WORKED_EXAMPLE.md`

Status: merged. **Scope:** mkdocs-source. **Navigation:** listed.

Destinations: Chapter 50 (anchor `typed-finite-expression-model`), Chapter 51 (anchor `expression-evaluation-and-certification`), Chapter 52 (anchor `expression-planning-and-persistence`), Chapter 68 (anchor `chapter-68.-integration-and-expression-recipes`), Appendix E (anchor `appendix-e-executable-example-records`).

Resolution. Typed finite expression example is integrated without promoting research names into the stable API.

Source record: 400 lines; SHA-256 prefix `14ea7f7a2e2cf189`. 12 lexical assertion candidates, 0 indexed command occurrences, 2 code blocks and 0 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-042. `docs/BLOCK_21_PUBLIC_REPRODUCIBILITY.md`

Status: corrected. **Scope:** mkdocs-source. **Navigation:** listed.

Destinations: Chapter 74 (anchor `block-21-publication-handoff`), Appendix E (anchor `appendix-e-executable-example-records`), Appendix F (anchor `appendix-f-cli-option-matrix`), Appendix L (anchor `appendix-l-migrations`).

Resolution. Use the R2 build-before-qualification workflow and build-tag-2 artifacts. Earlier handoff evidence remains historical.

Source record: 62 lines; SHA-256 prefix `a59d903635c87443`. 6 lexical assertion candidates, 0 indexed command occurrences, 4 code blocks and 0 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-043. `docs/BLOCK_2_RELEASE.md`

Status: historical. **Scope:** mkdocs-source. **Navigation:** not listed.

Destinations: Chapter 12 (anchor `refinement-towers`), Chapter 36 (anchor `singular-packets-and-recovery`), Chapter 40 (anchor `defects-obstruction-lifting-and-holonomy`), Chapter 41 (anchor `correction-and-elimination`), Appendix L (anchor `appendix-l-migrations`).

Resolution. Release-era scope and counts are retained history. Follow the current linked domain chapters and compatibility records.

Source record: 219 lines; SHA-256 prefix `2f07ca3449c4ff0e`. 19 lexical assertion candidates, 0 indexed command occurrences, 9 code blocks and 0 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-044. `docs/BLOCK_3_RELEASE.md`

Status: historical. **Scope:** mkdocs-source. **Navigation:** not listed.

Destinations: Chapter 28 (anchor `packet-intermediate-representation`), Chapter 29 (anchor `structured-and-low-rank-operators`), Chapter 30 (anchor `krylov-precision-compilation-and-scheduling`), Appendix L (anchor `appendix-l-migrations`).

Resolution. Release-era scope and counts are retained history. Follow the current linked domain chapters and compatibility records.

Source record: 183 lines; SHA-256 prefix `f39d088e75cd2421`. 9 lexical assertion candidates, 0 indexed command occurrences, 4 code blocks and 0 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-045. docs/BLOCK_4_RELEASE.md

Status: historical. **Scope:** mkdocs-source. **Navigation:** not listed.

Destinations: Chapter 31 (anchor `signal-conventions-windows-and-framing`), Chapter 35 (anchor `filter-banks-wavelets-and-packet-trees`), Chapter 45 (anchor `matrix-dynamics`), Chapter 46 (anchor `feshbach-reduction-memory-and-survival`), Appendix L (anchor `appendix-l-migrations`).

Resolution. Release-era scope and counts are retained history. Follow the current linked domain chapters and compatibility records.

Source record: 179 lines; SHA-256 prefix `8f80a9b94fc6ee1c`. 20 lexical assertion candidates, 0 indexed command occurrences, 7 code blocks and 3 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-046. docs/BLOCK_5_RELEASE.md

Status: historical. **Scope:** mkdocs-source. **Navigation:** not listed.

Destinations: Chapter 27 (anchor `cyclic-analytic-systems`), Chapter 53 (anchor `chapter-53.-finite-quantum-foundations`), Chapter 54 (anchor `chapter-54.-circuits-channels-and-open-dynamics`), Chapter 55 (anchor `chapter-55.-error-correction-and-effective-dynamics`), Appendix L (anchor `appendix-l-migrations`).

Resolution. Release-era scope and counts are retained history. Follow the current linked domain chapters and compatibility records.

Source record: 69 lines; SHA-256 prefix `1be16cbb47b44a35`. 7 lexical assertion candidates, 4 indexed command occurrences, 1 code block and 0 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-047. docs/BLOCK_6_RELEASE.md

Status: historical. **Scope:** mkdocs-source. **Navigation:** not listed.

Destinations: Chapter 25 (anchor `local-arithmetic-models`), Chapter 26 (anchor `exact-sequence-execution`), Appendix L (anchor `appendix-l-migrations`).

Resolution. Release-era scope and counts are retained history. Follow the current linked domain chapters and compatibility records.

Source record: 31 lines; SHA-256 prefix `e2510bbf397603c0`. 6 lexical assertion candidates, 0 indexed command occurrences, 0 code blocks and 0 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-048. docs/BLOCK_7_RELEASE.md

Status: historical. **Scope:** mkdocs-source. **Navigation:** not listed.

Destinations: Chapter 56 (anchor `chapter-56.-finite-deformation-systems`), Chapter 57 (anchor `chapter-57.-local-finite-langlands-packets`), Chapter 58 (anchor `chapter-58.-finite-image-ramification-and-completion-models`), Appendix L (anchor `appendix-l-migrations`).

Resolution. Release-era scope and counts are retained history. Follow the current linked domain chapters and compatibility records.

Source record: 41 lines; SHA-256 prefix `759a8b8466ee958a`. 1 lexical assertion candidate, 0 indexed command occurrences, 1 code block and 0 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-049. docs/BLOCK_8_RELEASE.md

Status: historical. **Scope:** mkdocs-source. **Navigation:** not listed.

Destinations: Chapter 59 (anchor `chapter-59.-mathematical-document-model`), Chapter 60 (anchor `chapter-60.-renderers-and-writers`), Chapter 61 (anchor `chapter-61.-portable-persistence`), Chapter 70 (anchor `chapter-70.-export-and-reproducibility-recipes`), Appendix L (anchor `appendix-l-migrations`).

Resolution. Release-era scope and counts are retained history. Follow the current linked domain chapters and compatibility records.

Source record: 23 lines; SHA-256 prefix `fbd58928ab4d2804`. 2 lexical assertion candidates, 0 indexed command occurrences, 0 code blocks and 0 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-050. docs/BLOCK_9_RELEASE.md

Status: historical. **Scope:** mkdocs-source. **Navigation:** listed.

Destinations: Chapter 37 (anchor `derivative-providers-and-automatic-differentiation`), Chapter 38 (anchor `packet-jets-and-fauxe0-di-bruno-composition`), Chapter 39 (anchor `nonlinear-trees-and-graded-operations`), Chapter 40 (anchor `defects-obstruction-lifting-and-holonomy`), Chapter 41 (anchor `correction-and-elimination`), Chapter 42 (anchor `filtered-lifting-and-finite-spectral-sequences`), Chapter 44 (anchor `smooth-packet-transport`), Appendix L (anchor `appendix-l-migrations`).

Resolution. Release-era scope and counts are retained history. Follow the current linked domain chapters and compatibility records.

Source record: 149 lines; SHA-256 prefix `674458831a50c29f`. 14 lexical assertion candidates, 0 indexed command occurrences, 2 code blocks and 7 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-051. docs/COMPANION_COORDINATE_CALCULUS.md

Status: merged. **Scope:** mkdocs-source. **Navigation:** listed.

Destinations: Chapter 13 (anchor `residue-crt-lattice-and-smith-coordinates`), Chapter 15 (anchor `coordinate-objects`), Chapter 16 (anchor `exact-and-numerical-coordinate-transforms`), Chapter 17 (anchor `polynomial-and-classical-coordinate-families`), Chapter 18 (anchor `combinatorial-recurrence-and-state-space-coordinates`), Chapter 19 (anchor `spectral-packet-tensor-and-operator-transport`).

Resolution. Companion-derived finite coordinate examples have implementation-specific homes in Chapters 13 and 15-19. External companion claims are not re-proved by this crosswalk.

Source record: 13 lines; SHA-256 prefix `7e934e39ca92f3bc`. 1 lexical assertion candidate, 0 indexed command occurrences, 0 code blocks and 1 qualified lexical/import reference. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-052. docs/COMPANION_C_INTEGRATION.md

Status: historical. **Scope:** mkdocs-source. **Navigation:** listed.

Destinations: Chapter 11 (anchor `packet-systems-and-transfer-blocks`), Chapter 14 (anchor `polyphase-systems-and-refinement`), Chapter 20 (anchor `analysis-synthesis-and-redundant-frames`), Chapter 72 (anchor `api-semantic-persistence-and-upgrade-contracts`).

Resolution. Retained reference-baseline adoption map, not a claim that every external Companion C construction is implemented or independently checked here.

Source record: 68 lines; SHA-256 prefix `f0966232553946c7`. 3 lexical assertion candidates, 0 indexed command occurrences, 0 code blocks and 0 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-053. docs/DEPRECATION_POLICY.md

Status: merged. **Scope:** mkdocs-source. **Navigation:** listed.

Destinations: Chapter 7 (anchor `public-api-tiers`), Chapter 72 (anchor `api-semantic-persistence-and-upgrade-contracts`), Appendix L (anchor `appendix-l-migrations`).

Resolution. Policy is integrated with actual decorator behavior. No removal schedule is inferred for unmarked retained aliases.

Source record: 45 lines; SHA-256 prefix `da3e4a5b82a7956a`. 1 lexical assertion candidate, 0 indexed command occurrences, 1 code block and 1 qualified lexical/import reference. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-054. docs/ELEVENTH_MAJOR_BLOCK_REPORT.md

Status: historical. **Scope:** mkdocs-source. **Navigation:** listed.

Destinations: Chapter 43 (anchor `nonlinear-execution-streaming-profiling-and-gpu-paths`), Chapter 61 (anchor `chapter-61.-portable-persistence`), Chapter 72 (anchor `api-semantic-persistence-and-upgrade-contracts`), Appendix L (anchor `appendix-l-migrations`).

Resolution. Named development-block report is historical. Its implementation themes are integrated in the linked chapters; old execution counts are not current tests.

Source record: 134 lines; SHA-256 prefix `371687b2788b1e3e`. 16 lexical assertion candidates, 0 indexed command occurrences, 0 code blocks and 1 qualified lexical/import reference. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-055. docs/EXPORT_AND_LATEX.md

Status: corrected. **Scope:** mkdocs-source. **Navigation:** listed.

Destinations: Chapter 59 (anchor `chapter-59.-mathematical-document-model`), Chapter 60 (anchor `chapter-60.-renderers-and-writers`), Chapter 70 (anchor `chapter-70.-export-and-reproducibility-recipes`), Appendix F (anchor `appendix-f-cli-option-matrix`), Appendix J (anchor `appendix-j-backends-dependencies`).

Resolution. Use F for command-specific overwrite behavior, including latex compile, and J for tool qualification. No common force flag is assumed across all paths.

Source record: 138 lines; SHA-256 prefix `cc509bc2df52d51b`. 3 lexical assertion candidates, 0 indexed command occurrences, 8 code blocks and 9 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-056. docs/EXTENSION_API.md

Status: corrected. **Scope:** mkdocs-source. **Navigation:** not listed.

Destinations: Chapter 62 (anchor `chapter-62.-provenance-registries-and-extensions`), Appendix I (anchor `appendix-i-exceptions-diagnostics`), Appendix J (anchor `appendix-j-backends-dependencies`).

Resolution. Extension entry points execute trusted Python in the caller. Catching ordinary exceptions is not child-process isolation or sandboxing; doctor discovers with activate=False.

Source record: 39 lines; SHA-256 prefix `6faa47f3b5232058`. 1 lexical assertion candidate, 1 indexed command occurrence, 3 code blocks and 2 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

Specific corrections: CN-08.

DN-057. docs/FINITE_MEASURE_INTEGRATION.md

Status: merged. **Scope:** mkdocs-source. **Navigation:** listed.

Destinations: Chapter 47 (anchor `measures-and-exact-polynomials`), Chapter 48 (anchor `integration-providers-and-quadrature`), Chapter 49 (anchor `integral-identities-and-coordinate-covectors`), Chapter 68 (anchor `chapter-68.-integration-and-expression-recipes`).

Resolution. Finite exact measures, providers and covectors are integrated. Scope and quadrature error strength remain explicit.

Source record: 117 lines; SHA-256 prefix `7c33409a3174379f`. 18 lexical assertion candidates, 0 indexed command occurrences, 1 code block and 0 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-058. docs/FOURTEENTH_MAJOR_BLOCK_REPORT.md

Status: historical. **Scope:** mkdocs-source. **Navigation:** listed.

Destinations: Chapter 15 (anchor `coordinate-objects`), Chapter 16 (anchor `exact-and-numerical-coordinate-transforms`), Chapter 17 (anchor `polynomial-and-classical-coordinate-families`), Chapter 18 (anchor `combinatorial-recurrence-and-state-space-coordinates`), Chapter 19 (anchor `spectral-packet-tensor-and-operator-transport`), Appendix L (anchor `appendix-l-migrations`).

Resolution. Named development-block report is historical. Its implementation themes are integrated in the linked chapters; old execution counts are not current tests.

Source record: 69 lines; SHA-256 prefix `a9879ab74341d78f`. 6 lexical assertion candidates, 0 indexed command occurrences, 0 code blocks and 0 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-059. docs/GENERAL_COORDINATE_ATLAS.md

Status: merged. **Scope:** mkdocs-source. **Navigation:** listed.

Destinations: Chapter 15 (anchor `coordinate-objects`), Chapter 16 (anchor `exact-and-numerical-coordinate-transforms`), Chapter 17 (anchor `polynomial-and-classical-coordinate-families`), Chapter 18 (anchor `combinatorial-recurrence-and-state-space-coordinates`), Chapter 19 (anchor `spectral-packet-tensor-and-operator-transport`), Chapter 64 (anchor `chapter-64.-exact-algebra-and-coordinate-recipes`).

Resolution. General finite coordinate atlas is integrated. Euler-Fibonacci is one case, not the namespace definition.

Source record: 257 lines; SHA-256 prefix `24990dd2b3a21622`. 14 lexical assertion candidates, 10 indexed command occurrences, 5 code blocks and 9 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-060. docs/GPU_INSTALLATION.md

Status: corrected. **Scope:** mkdocs-source. **Navigation:** listed.

Destinations: Chapter 3 (anchor `installation-and-environment-profiles`), Chapter 73 (anchor `platform-and-release-qualification`), Appendix J (anchor `appendix-j-backends-dependencies`), Appendix L (anchor `appendix-l-migrations`).

Resolution. Treat the hardware stack as historical recorded context, not current R2 CUDA qualification. Replace the untagged b4 wheel path with the shipped build-2 artifact.

Source record: 30 lines; SHA-256 prefix `12df1bf3f72830e3`. 2 lexical assertion candidates, 0 indexed command occurrences, 3 code blocks and 0 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

Specific corrections: CN-09.

DN-061. `docs/GPU_VALIDATION.md`

Status: historical. **Scope:** mkdocs-source. **Navigation:** listed.

Destinations: Chapter 73 (anchor `platform-and-release-qualification`), Appendix J (anchor `appendix-j-backends-dependencies`).

Resolution. The file labels its Windows results as a recorded configuration. Do not transfer those results to the current CPU-only run or claim current CUDA success.

Source record: 48 lines; SHA-256 prefix `48d795cb61e10737`. 2 lexical assertion candidates, 1 indexed command occurrence, 2 code blocks and 0 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-062. `docs/IMPLEMENTATION_ROADMAP.md`

Status: historical. **Scope:** mkdocs-source. **Navigation:** listed.

Destinations: Chapter 0 (anchor `coverage-contract`), Chapter 74 (anchor `block-21-publication-handoff`), Chapter 82 (anchor `final-guide-publication-gate-and-retained-architecture`), Appendix L (anchor `appendix-l-migrations`).

Resolution. Roadmap stages and future release objectives are retained plans, not evidence of a completed rc1 or 1.0.0 release.

Source record: 428 lines; SHA-256 prefix `4e1b7148c5364a16`. 55 lexical assertion candidates, 0 indexed command occurrences, 0 code blocks and 4 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-063. `docs/LOCAL_VALIDATION.md`

Status: corrected. **Scope:** mkdocs-source. **Navigation:** listed.

Destinations: Chapter 3 (anchor `installation-and-environment-profiles`), Chapter 73 (anchor `platform-and-release-qualification`), Chapter 74 (anchor `block-21-publication-handoff`), Appendix F (anchor `appendix-f-cli-option-matrix`), Appendix J (anchor `appendix-j-backends-dependencies`), Appendix L (anchor `appendix-l-migrations`).

Resolution. The WindowsHotfix1 folder name is historical. Run the explicit current R2 launchers from the current checkout, and keep native-platform and MkDocs gates unclaimed here.

Source record: 62 lines; SHA-256 prefix `8a493de9655eecdf`. 6 lexical assertion candidates, 8 indexed command occurrences, 3 code blocks and 0 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

Specific corrections: CN-10.

DN-064. `docs/MIGRATION_TO_1_0.md`

Status: merged. **Scope:** mkdocs-source. **Navigation:** listed.

Destinations: Chapter 7 (anchor `public-api-tiers`), Chapter 72 (anchor `api-semantic-persistence-and-upgrade-contracts`), Appendix L (anchor `appendix-l-migrations`).

Resolution. Migration guidance is integrated with the full five-predecessor contract and actually tested finite forward upgrades.

Source record: 56 lines; SHA-256 prefix `4ea4cbb8f9d34609`. 0 lexical assertion candidates, 5 indexed command occurrences, 5 code blocks and 8 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-065. `docs/NINTH_MAJOR_BLOCK_REPORT.md`

Status: historical. **Scope:** mkdocs-source. **Navigation:** not listed.

Destinations: Chapter 37 (anchor `derivative-providers-and-automatic-differentiation`), Chapter 38 (anchor `packet-jets-and-fauxe0-di-bruno-composition`), Chapter 39 (anchor `nonlinear-trees-and-graded-operations`), Chapter 40 (anchor `defects-obstruction-lifting-and-holonomy`), Chapter 41 (anchor `correction-and-elimination`), Chapter 42 (anchor `filtered-lifting-and-finite-spectral-sequences`), Chapter 44 (anchor `smooth-packet-transport`), Appendix L (anchor `appendix-l-migrations`).

Resolution. Named development-block report is historical. Its implementation themes are integrated in the linked chapters; old execution counts are not current tests.

Source record: 116 lines; SHA-256 prefix `433c342f1f3584d9`. 6 lexical assertion candidates, 0 indexed command occurrences, 0 code blocks and 0 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-066. `docs/NONLINEAR_CALCULUS.md`

Status: merged. **Scope:** mkdocs-source. **Navigation:** listed.

Destinations: Chapter 37 (anchor `derivative-providers-and-automatic-differentiation`), Chapter 38 (anchor `packet-jets-and-fauxe0-di-bruno-composition`), Chapter 39 (anchor `nonlinear-trees-and-graded-operations`), Chapter 40 (anchor `defects-obstruction-lifting-and-holonomy`), Chapter 41 (anchor `correction-and-elimination`), Chapter 42 (anchor `filtered-lifting-and-finite-spectral-sequences`), Chapter 44 (anchor `smooth-packet-transport`).

Resolution. Higher nonlinear calculus is integrated with finite-dimensional and provider-specific boundaries preserved.

Source record: 133 lines; SHA-256 prefix `583f7a1daa3395ee`. 8 lexical assertion candidates, 0 indexed command occurrences, 9 code blocks and 3 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-067. `docs/NORMALIZED_GENERATING_FUNCTIONS.md`

Status: merged. **Scope:** mkdocs-source. **Navigation:** listed.

Destinations: Chapter 21 (anchor `formal-series-and-weighted-bases`), Chapter 22 (anchor `normalized-generating-functions`), Chapter 23 (anchor `masks-cartier-operators-and-digit-algebra`), Chapter 24 (anchor `prime-power-bell-cyclotomic-and-q-kummer-systems`).

Resolution. Generating bases, transfer and arithmetic compilers are integrated. Use K for exact-scalar growth and local budget behavior.

Source record: 181 lines; SHA-256 prefix `9ae326da05e48905`. 12 lexical assertion candidates, 0 indexed command occurrences, 4 code blocks and 9 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-068. `docs/OPTIONAL_BACKEND_ISOLATION.md`

Status: merged. **Scope:** mkdocs-source. **Navigation:** not listed.

Destinations: Chapter 3 (anchor `installation-and-environment-profiles`), Chapter 37 (anchor `derivative-providers-and-automatic-differentiation`), Appendix J (anchor `appendix-j-backends-dependencies`).

Resolution. The bounded child probes for optional autodiff are real and distinct from in-process extension discovery. Finite CPU evidence is not GPU qualification.

Source record: 47 lines; SHA-256 prefix `ba57434ff8161f3e`. 1 lexical assertion candidate, 0 indexed command occurrences, 2 code blocks and 0 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-069. `docs/PERSISTENCE_SCHEMAS.md`

Status: corrected. **Scope:** mkdocs-source. **Navigation:** listed.

Destinations: Chapter 61 (anchor `chapter-61.-portable-persistence`), Appendix G (anchor `appendix-g-portable-schemas`), Appendix L (anchor `appendix-l-migrations`).

Resolution. Use the five-field envelope including producer_version and migrator=, not migrate=. The short codec list is partial and current permissive-reader limits are explicit in G.

Source record: 123 lines; SHA-256 prefix `6cc7466fc378ef99`. 4 lexical assertion candidates, 0 indexed command occurrences, 4 code blocks and 3 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

Specific corrections: CN-01, CN-02, CN-03, CN-04.

DN-070. docs/POWERSHELL_TESTS.md

Status: superseded. **Scope:** mkdocs-source. **Navigation:** not listed.

Destinations: Chapter 3 (anchor `installation-and-environment-profiles`), Chapter 70 (anchor `chapter-70.-export-and-reproducibility-recipes`), Chapter 74 (anchor `block-21-publication-handoff`), Appendix E (anchor `appendix-e-executable-example-records`), Appendix F (anchor `appendix-f-cli-option-matrix`), Appendix J (anchor `appendix-j-backends-dependencies`).

Resolution. The 0.11.0a1 installation recipe is historical. Use current build-2 installation instructions and E for pending native PowerShell execution.

Source record: 157 lines; SHA-256 prefix `983e529df7e9ad56`. 1 lexical assertion candidate, 26 indexed command occurrences, 13 code blocks and 1 qualified lexical/import reference. These counts are navigation metadata, not new tests or semantic completeness claims.

Specific corrections: CN-12.

DN-071. docs/PRODUCTION_SIGNAL_PROCESSING.md

Status: merged. **Scope:** mkdocs-source. **Navigation:** listed.

Destinations: Chapter 31 (anchor `signal-conventions-windows-and-framing`), Chapter 32 (anchor `time-frequency-and-spectral-analysis`), Chapter 33 (anchor `filtering-and-convolution`), Chapter 34 (anchor `resampling-and-adaptive-filters`), Chapter 35 (anchor `filter-banks-wavelets-and-packet-trees`), Chapter 36 (anchor `singular-packets-and-recovery`), Chapter 66 (anchor `chapter-66.-signal-processing-recipes`).

Resolution. Production finite signal routines are integrated with transform normalization, framing and boundary conditions.

Source record: 346 lines; SHA-256 prefix `f395154c31b2bccc`. 16 lexical assertion candidates, 2 indexed command occurrences, 14 code blocks and 20 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-072. docs/PUBLIC_API.md

Status: corrected. **Scope:** mkdocs-source. **Navigation:** listed.

Destinations: Chapter 7 (anchor `public-api-tiers`), Appendix A (anchor `appendix-a.-stable-api-131-entry-frozen-reference`), Appendix B (anchor `appendix-b.-beta-api-83-entry-reference`), Appendix C (anchor `appendix-c.-research-api-1312-entry-reference`), Appendix L (anchor `appendix-l-migrations`).

Resolution. Use the complete A-C manifests. The historical sentence calling the whole package alpha is historical, not the current b4 release status.

Source record: 795 lines; SHA-256 prefix `0e87971221560e21`. 28 lexical assertion candidates, 29 indexed command occurrences, 34 code blocks and 110 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

Specific corrections: CN-07.

DN-073. docs/REDUNDANT_FRAME_ALGEBRA.md

Status: merged. **Scope:** mkdocs-source. **Navigation:** listed.

Destinations: Chapter 20 (anchor `analysis-synthesis-and-redundant-frames`), Appendix M (anchor `appendix-m-glossary-notation`).

Resolution. Redundant coefficient-space algebra is integrated. Physical reconstruction components and coefficient projectors remain distinct.

Source record: 130 lines; SHA-256 prefix `429783554f95dd1b`. 4 lexical assertion candidates, 0 indexed command occurrences, 0 code blocks and 0 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-074. `docs/RELEASE_PROCESS.md`

Status: corrected. **Scope:** mkdocs-source. **Navigation:** listed.

Destinations: Chapter 73 (anchor `platform-and-release-qualification`), Chapter 74 (anchor `block-21-publication-handoff`), Chapter 81 (anchor `site-and-pdf-build-closure`), Chapter 82 (anchor `final-guide-publication-gate-and-retained-architecture`), Appendix L (anchor `appendix-l-migrations`).

Resolution. Read through the current R2 repair note. Build artifacts before qualification and use the build-2 wheel; a workflow declaration is not an executed hosted run.

Source record: 119 lines; SHA-256 prefix `996e47749965e57c`. 7 lexical assertion candidates, 0 indexed command occurrences, 4 code blocks and 0 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-075. `docs/RELEASE_QUALIFICATION.md`

Status: corrected. **Scope:** mkdocs-source. **Navigation:** listed.

Destinations: Chapter 73 (anchor `platform-and-release-qualification`), Chapter 74 (anchor `block-21-publication-handoff`), Appendix F (anchor `appendix-f-cli-option-matrix`), Appendix J (anchor `appendix-j-backends-dependencies`), Appendix L (anchor `appendix-l-migrations`).

Resolution. The current R2 launcher builds before qualification and binds installed-artifact checks. Native-platform and strict documentation gates remain separate from local source checks.

Source record: 63 lines; SHA-256 prefix `6019ab8d88c39c06`. 5 lexical assertion candidates, 8 indexed command occurrences, 5 code blocks and 0 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-076. `docs/SCALABLE_NONLINEAR_EXECUTION.md`

Status: merged. **Scope:** mkdocs-source. **Navigation:** listed.

Destinations: Chapter 37 (anchor `derivative-providers-and-automatic-differentiation`), Chapter 38 (anchor `packet-jets-and-fauxe0-di-bruno-composition`), Chapter 43 (anchor `nonlinear-execution-streaming-profiling-and-gpu-paths`), Appendix K (anchor `appendix-k-resource-budgets`).

Resolution. Finite plans, caches, snapshots and device paths are integrated. Check K for late guards and actual representation costs.

Source record: 146 lines; SHA-256 prefix `dc93a741e005ad74`. 5 lexical assertion candidates, 0 indexed command occurrences, 7 code blocks and 5 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-077. `docs/STABLE_API.md`

Status: superseded. **Scope:** mkdocs-source. **Navigation:** not listed.

Destinations: Chapter 7 (anchor `public-api-tiers`), Appendix A (anchor `appendix-a.-stable-api-131-entry-frozen-reference`), Appendix L (anchor `appendix-l-migrations`).

Resolution. Use Appendix A for the frozen 131-name b4 stable surface. Candidate-era naming is retained history rather than a different current API.

Source record: 39 lines; SHA-256 prefix `23754cecb960a1f5`. 2 lexical assertion candidates, 2 indexed command occurrences, 2 code blocks and 5 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-078. `docs/STABLE_API_1_0_CANDIDATE.md`

Status: corrected. **Scope:** mkdocs-source. **Navigation:** listed.

Destinations: Chapter 7 (anchor `public-api-tiers`), Appendix A (anchor `appendix-a.-stable-api-131-entry-frozen-reference`), Appendix B (anchor `appendix-b.-beta-api-83-entry-reference`), Appendix C (anchor `appendix-c.-research-api-1312-entry-reference`), Appendix L (anchor `appendix-l-migrations`).

Resolution. The 1,179 research-name count is historical. The pinned research manifest now has 1,312 records; stable remains 131 and beta remains 83.

Source record: 98 lines; SHA-256 prefix `f815eae16d91bcec`. 4 lexical assertion candidates, 3 indexed command occurrences, 2 code blocks and 12 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

Specific corrections: CN-06.

DN-079. `docs/STREAMING_PACKET_JETS.md`

Status: merged. **Scope:** mkdocs-source. **Navigation:** listed.

Destinations: Chapter 43 (anchor `nonlinear-execution-streaming-profiling-and-gpu-paths`), Chapter 61 (anchor `chapter-61.-portable-persistence`), Appendix G (anchor `appendix-g-portable-schemas`), Appendix K (anchor `appendix-k-resource-budgets`).

Resolution. Chunked stream and resumability records are integrated. Partial evaluation does not imply a complete materialized snapshot.

Source record: 168 lines; SHA-256 prefix `02cb2fe400a6dfb8`. 7 lexical assertion candidates, 2 indexed command occurrences, 7 code blocks and 4 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-080. `docs/SUPPORTED_PLATFORMS.md`

Status: current. **Scope:** mkdocs-source. **Navigation:** listed.

Destinations: Chapter 3 (anchor `installation-and-environment-profiles`), Chapter 8 (anchor `backends-scalar-domains-and-coercion`), Chapter 73 (anchor `platform-and-release-qualification`), Appendix J (anchor `appendix-j-backends-dependencies`).

Resolution. Declared Python/dependency support is explicitly distinct from recorded execution. No new platform support is inferred from this crosswalk.

Source record: 25 lines; SHA-256 prefix `ca089e55c98cb10a`. 2 lexical assertion candidates, 0 indexed command occurrences, 0 code blocks and 0 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-081. `docs/TENTH_MAJOR_BLOCK_REPORT.md`

Status: historical. **Scope:** mkdocs-source. **Navigation:** listed.

Destinations: Chapter 37 (anchor `derivative-providers-and-automatic-differentiation`), Chapter 38 (anchor `packet-jets-and-fauxe0-di-bruno-composition`), Chapter 43 (anchor `nonlinear-execution-streaming-profiling-and-gpu-paths`), Chapter 61 (anchor `chapter-61.-portable-persistence`), Appendix L (anchor `appendix-l-migrations`).

Resolution. Named development-block report is historical. Its implementation themes are integrated in the linked chapters; old execution counts are not current tests.

Source record: 131 lines; SHA-256 prefix `4c20dcb814d2b497`. 9 lexical assertion candidates, 0 indexed command occurrences, 0 code blocks and 0 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-082. `docs/THIRTEENTH_MAJOR_BLOCK_REPORT.md`

Status: historical. **Scope:** mkdocs-source. **Navigation:** listed.

Destinations: Chapter 21 (anchor `formal-series-and-weighted-bases`), Chapter 22 (anchor `normalized-generating-functions`), Chapter 23 (anchor `masks-cartier-operators-and-digit-algebra`), Chapter 24 (anchor `prime-power-bell-cyclotomic-and-q-kummer-systems`), Appendix L (anchor `appendix-l-migrations`).

Resolution. Named development-block report is historical. Its implementation themes are integrated in the linked chapters; old execution counts are not current tests.

Source record: 68 lines; SHA-256 prefix `c62c83149a73659f`. 7 lexical assertion candidates, 0 indexed command occurrences, 0 code blocks and 0 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-083. `docs/TWELFTH_MAJOR_BLOCK_REPORT.md`

Status: historical. **Scope:** mkdocs-source. **Navigation:** listed.

Destinations: Chapter 31 (anchor `signal-conventions-windows-and-framing`), Chapter 32 (anchor `time-frequency-and-spectral-analysis`), Chapter 33 (anchor `filtering-and-convolution`), Chapter 34 (anchor `resampling-and-adaptive-filters`), Chapter 35 (anchor `filter-banks-wavelets-and-packet-trees`), Appendix L (anchor `appendix-l-migrations`).

Resolution. Named development-block report is historical. Its implementation themes are integrated in the linked chapters; old execution counts are not current tests.

Source record: 87 lines; SHA-256 prefix `fd44fa290ed58b0f`. 5 lexical assertion candidates, 0 indexed command occurrences, 0 code blocks and 0 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-084. `docs/TYPED_FINITE_EXPRESSION_CALCULUS.md`

Status: merged. **Scope:** mkdocs-source. **Navigation:** listed.

Destinations: Chapter 50 (anchor `typed-finite-expression-model`), Chapter 51 (anchor `expression-evaluation-and-certification`), Chapter 52 (anchor `expression-planning-and-persistence`), Chapter 68 (anchor `chapter-68.-integration-and-expression-recipes`).

Resolution. Finite rational typed language is integrated. The normal-form equality statement stays within its declared finite domain.

Source record: 320 lines; SHA-256 prefix `8c47d1cb4d09d88f`. 32 lexical assertion candidates, 0 indexed command occurrences, 0 code blocks and 0 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-085. `docs/UNIFIED_LANGUAGE.md`

Status: merged. **Scope:** mkdocs-source. **Navigation:** listed.

Destinations: Chapter 2 (anchor `what-komorebi-math-is`), Chapter 9 (anchor `typed-operator-grammars`), Chapter 11 (anchor `packet-systems-and-transfer-blocks`), Chapter 28 (anchor `packet-intermediate-representation`), Chapter 50 (anchor `typed-finite-expression-model`).

Resolution. Unified language is an organizing interface across finite domains, not an assertion that every planned completion exists.

Source record: 277 lines; SHA-256 prefix `1dcf70b9988707b7`. 21 lexical assertion candidates, 0 indexed command occurrences, 16 code blocks and 0 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

DN-086. `docs/UPGRADE_CONTRACT.md`

Status: corrected. **Scope:** mkdocs-source. **Navigation:** listed.

Destinations: Chapter 72 (anchor `api-semantic-persistence-and-upgrade-contracts`), Appendix L (anchor `appendix-l-migrations`).

Resolution. Two listed alpha predecessors and the b1 target are historical. The current contract covers five predecessor wheels and the b4 reader.

Source record: 33 lines; SHA-256 prefix `4c2b76e06be9196d`. 3 lexical assertion candidates, 0 indexed command occurrences, 1 code block and 0 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

Specific corrections: CN-05.

DN-087. `docs/index.md`

Status: current. **Scope:** mkdocs-source. **Navigation:** listed.

Destinations: Chapter 2 (anchor `what-komorebi-math-is`), Chapter 7 (anchor `public-api-tiers`), Chapter 74 (anchor `block-21-publication-handoff`).

Resolution. Current site landing page. Read the R2 repair note before selecting an installable artifact.

Source record: 50 lines; SHA-256 prefix `f1732465c164d55c`. 4 lexical assertion candidates, 0 indexed command occurrences, 0 code blocks and 0 qualified lexical/import references. These counts are navigation metadata, not new tests or semantic completeness claims.

N.4 Concrete correction and precedence records

The following selectors locate the exact retained text. A source file may have more than one correction. Conversely, a reading-status correction can be a workflow precedence rule rather than a single obsolete token. A literal version-shaped string is not automatically stale: older releases, dependency versions and numeric examples can all legitimately occur in current prose.

CN-01. `PERSISTENCE_SCHEMAS.md`

Retained source: `komorebi-math/docs/PERSISTENCE_SCHEMAS.md`, line 16. **Exact selector:** `"schema_version": 1`.

Resolution. The displayed four-field envelope omits required producer_version. Use dumps_portable and preserve all five envelope fields. Read Appendix G (anchor `appendix-g-portable-schemas`).

CN-02. `PERSISTENCE_SCHEMAS.md`

Retained source: `komorebi-math/docs/PERSISTENCE_SCHEMAS.md`, line 84. **Exact selector:** `migrate=lambda`.

Resolution. register_portable_migration accepts migrator=; migrate= raises TypeError. The corrected custom registration is executed in the recorded correction probe. Read Appendix G (anchor `appendix-g-portable-schemas`).

CN-03. `PERSISTENCE_SCHEMAS.md`

Retained source: `komorebi-math/docs/PERSISTENCE_SCHEMAS.md`, line 58. **Exact selector:** `The current registry includes codecs for:.`

Resolution. This is a partial early list, not an exhaustive current registry. The pinned registry has 51 mathematical codecs. Read Appendix G (anchor `appendix-g-portable-schemas`).

CN-04. PERSISTENCE_SCHEMAS.md

Retained source: komorebi-math/docs/PERSISTENCE_SCHEMAS.md, line 54. **Exact selector:** `The decoder validates dimensions and recursion depth before reconstruction.`.

Resolution. Do not infer strict universal types or hostile-input resource safety from this sentence. G records permissive conversions and parsing before the depth guard. Read Appendix G (anchor `appendix-g-portable-schemas`).

CN-05. UPGRADE_CONTRACT.md

Retained source: komorebi-math/docs/UPGRADE_CONTRACT.md, line 8. **Exact selector:** `0.15.0a2`.

Resolution. The two-entry predecessor listing is incomplete for the current five-predecessor contract. No downgrade or all-object migration guarantee follows. Read Appendix L (anchor `appendix-l-migrations`).

CN-06. STABLE_API_1_0_CANDIDATE.md

Retained source: komorebi-math/docs/STABLE_API_1_0_CANDIDATE.md, line 83. **Exact selector:** `1,179`.

Resolution. The research count is 1,312 in the pinned manifests. Stable 131 and beta 83 are different tier inventories. Read Appendix C (anchor `appendix-c.-research-api-1312-entry-reference`).

CN-07. PUBLIC_API.md

Retained source: komorebi-math/docs/PUBLIC_API.md, line 735. **Exact selector:** `The complete package remains alpha.`.

Resolution. This sentence belongs to the historical source baseline. The actual pinned package version is 1.0.0b4, Repair R2 build 2. Read Appendix L (anchor `appendix-l-migrations`).

CN-08. EXTENSION_API.md

Retained source: komorebi-math/docs/EXTENSION_API.md, line 37. **Exact selector:** `loads each extension in isolation.`

Resolution. Discovery uses point.load and any callable factory in the same interpreter; ordinary Exception handling is not a subprocess boundary. doctor does not request activation. Read Appendix J (anchor `appendix-j-backends-dependencies`).

CN-09. GPU_INSTALLATION.md

Retained source: komorebi-math/docs/GPU_INSTALLATION.md, line 13. **Exact selector:** `komorebi_math-1.0.0b4-py3-none-any.whl`.

Resolution. The retained untagged filename is not the shipped R2 artifact. Use komorebi_math-1.0.0b4-2-py3-none-any.whl and verify its hash. Read Appendix L (anchor `appendix-l-migrations`).

CN-10. LOCAL_VALIDATION.md

Retained source: komorebi-math/docs/LOCAL_VALIDATION.md, line 13. **Exact selector:** `Block21_WindowsHotfix1`.

Resolution. This folder name is historical. Current launchers must run from the current R2 checkout; an old folder name is not release identity. Read Appendix L (anchor `appendix-l-migrations`).

CN-11. BETA_TO_RC.md

Retained source: komorebi-math/docs/BETA_TO_RC.md, line 26. **Exact selector:** `For 1.0.0b1`.

Resolution. This is a historical beta example. Current platform acceptance must be tied to the exact current build and evidence. Read Appendix J (anchor `appendix-j-backends-dependencies`).

CN-12. POWERSHELL_TESTS.md

Retained source: komorebi-math/docs/POWERSHELL_TESTS.md, line 6. **Exact selector:** komorebi_math-0.11.0a1-py3-none-any.whl.

Resolution. Old PowerShell install recipe is superseded for this delivery. Native current PowerShell replay remains pending, not certified by Linux execution. Read Appendix E (anchor appendix-e-executable-example-records).

CN-13. README.md

Retained source: komorebi-math/README.md, line 43. **Exact selector:** komorebi_math-1.0.0b4-py3-none-any.whl.

Resolution. Follow the leading R2 repair notice over the lower inherited untagged wheel example. Build 2 is the retained current artifact. Read Appendix L (anchor appendix-l-migrations).

Extension isolation is especially easy to misread. The pinned implementation calls point.load() and a callable factory in the same Python interpreter. It catches ordinary Exception and stores a failure record, but SystemExit propagates. Discovery defaults to activate=False; that does not prevent import-time or factory code from running. Two controlled synthetic-entry-point probes confirm this behavior. They do not install an external plugin, simulate a native abort or qualify a sandbox.

N.5 Links, snippets and command interpretation

Static observation	Result
CommonMark link occurrences	75
Ordinary local file links resolving to retained paths	74
Math-markup link candidates with no matching file	1
Direct command lines recognized in the current command census	236
Argument-parser acceptance in the recorded normalization profile	236 / 236
Indexed Python code-block syntax	All Python-tagged blocks parse; this is not execution

The one ambiguous link is in docs/NORMALIZED_GENERATING_FUNCTIONS.md: the local-mask formula containing R](z) is parsed as a link to z by the CommonMark inventory parser. No docs/z exists. The source location and parser interpretation remain visible in the ledger; the record is not discarded to report zero unresolved links. A mathematics-aware site build should establish its actual rendering. **No strict MkDocs result is claimed**, and the CommonMark observation is not presented as a tested MkDocs failure.

Only file paths are checked here. Markdown fragments are retained but not resolved through an actual MkDocs renderer; external URLs are not fetched. Manual destination anchors are a separate relation and are checked for uniqueness in the pinned TeX. Heading line numbers and source fragments are bound to checksums, so an old anchor cannot silently be applied to changed prose.

The code-block inventory retains 301 fenced or indented blocks with language labels and exact source spans. Python-tagged blocks are parsed syntactically without execution. The CLI scanner recognizes direct komorebi, python -m komorebi_math and & $Komorebi lines inside these blocks. It does not expand arbitrary shell variables, execute installations, run compound shell programs or infer commands from un-delimited prose.

The 236 parser observations normalize quoting with the declared local tokenization profile and invoke only build_parser().parse_args. **No command handler is executed by this replay.** Valid option spelling does not establish that an input file exists, a mathematical assumption holds, a backend works or the command returns a positive qualification. For real finite command execution, use the retained evidence in Appendix F (anchor appendix-f-cli-option-matrix) and its explicit platform limits.

N.6 Executable correction observations and lookup

Executed check	Recorded result
Unchanged full source suite	1,203 passed
Reference/source/anchor/identity suite	2357 passed
Finite correction probes	8 / 8 expectations satisfied
Actual lookup CLI scenarios	8 / 8 expectations satisfied
Source-to-installed-package byte parity	366 / 366
Argument-parser observations	236 accepted, zero handlers executed

ND-P01. Reflect stable, beta and research inventories as 131, 83 and 1,312 records.

Observed expectation: satisfied. Full output and input hashes are in `correction_probe_execution.json`. This is a finite implementation observation, not a new theorem or universal migration guarantee.

ND-P02. Load a genuine five-field portable envelope and reject the same envelope after removing producer_version.

Observed expectation: satisfied. Full output and input hashes are in `correction_probe_execution.json`. This is a finite implementation observation, not a new theorem or universal migration guarantee.

ND-P03. Reject migrate=, register the correct migrator= callable, and apply it to one finite dictionary.

Observed expectation: satisfied. Full output and input hashes are in `correction_probe_execution.json`. This is a finite implementation observation, not a new theorem or universal migration guarantee.

ND-P04. Count the actual installed registry as 51 mathematical codecs, not the short early prose list.

Observed expectation: satisfied. Full output and input hashes are in `correction_probe_execution.json`. This is a finite implementation observation, not a new theorem or universal migration guarantee.

ND-P05. Observe synthetic entry-point loading and factory execution in the caller PID; distinguish default discovery from explicit activation.

Observed expectation: satisfied. Full output and input hashes are in `correction_probe_execution.json`. This is a finite implementation observation, not a new theorem or universal migration guarantee.

ND-P06. Record an ordinary extension exception and observe that a controlled SystemExit propagates.

Observed expectation: satisfied. Full output and input hashes are in `correction_probe_execution.json`. This is a finite implementation observation, not a new theorem or universal migration guarantee.

ND-P07. Find the shipped build-2 wheel and confirm the inherited untagged current-wheel path is absent.

Observed expectation: satisfied. Full output and input hashes are in `correction_probe_execution.json`. This is a finite implementation observation, not a new theorem or universal migration guarantee.

ND-P08. Read the actual predecessor_versions contract field and verify its five declared versions.

Observed expectation: satisfied. Full output and input hashes are in `correction_probe_execution.json`. This is a finite implementation observation, not a new theorem or universal migration guarantee.

The actual lookup tool validates source bindings before answering. Examples, from the delivery root:

```
python tools/block46/lookup.py \
  --source docs/PERSISTENCE_SCHEMAS.md --unique
python tools/block46/lookup.py --target G
python tools/block46/lookup.py \
  --symbol komorebi_math.PacketSystem --unique
```

The eight command scenarios include a single-source lookup, a target reverse lookup, an exact API-route lookup, a deliberately ambiguous historical-name query, no-match and incompatible-filter refusals, the current repair note, and a source omitted from navigation. Exit 0 means matches, 1 means no match, 2 means strict uniqueness was not satisfied, and 3 means validation found stale inputs. The tool does not silently select one of several historical matches.

N.7 Reconstruction, boundaries and next work

`inventory.py` regenerates the JSON crosswalk, source lookup TSV and complete code-block JSONL. `generate_manual.py` reconstructs both Markdown and both LaTeX sources. `reproduce_sources.py` checks all seven outputs in a separate temporary working tree. `validate.py` rejects missing rows, changed source bytes, ambiguous manual anchors and unreviewed non-mathematical link failures. Build and delivery tools independently bind test logs, content digests and reviewed PDF bytes.

The eight correction probes, 236 parser observations and 2357 reference tests overlap in scope. They must not be summed as independent mathematical proofs. The eight actual lookup processes do not execute the mathematical commands being described. All installed-package observations use a separate build-2 installation with inherited host dependencies, not a successful fully dependency-isolated environment.

Reference boundaries. This crosswalk covers the configured documentation-source census and the stated API, module, command and example joins. The recorded audit did not qualify native Windows/PowerShell, other Python/dependency versions, CUDA, strict MkDocs, fully dependency-isolated installation, final publication or new Lean proofs. Certificate, portable-reader and resource-budget limitations remain separate. Appendix O supplies the final source's static test and verification crosswalk and identifies its retained execution evidence.

Appendix O. Test and verification crosswalk

Use this appendix to find the relevant tests. Search for a module, function or class name, then open the matching source record. O.4 lists test modules, O.5 installed verification modules, and O.6 repository verification programs. O.3 identifies the retained execution reports when you need to know what was actually run.

Read one card in three steps. First check its source path and pinned revision. Next use its import targets to locate the code under examination. Finally read the linked test or verification program to see its setup, input values and assertions. Those details tell you whether it exercises the case you care about. A file with no literal Komorebi import may still reach the library through a helper or subprocess, so inspect its entry point before ruling it out.

Keep two questions separate as you navigate: “Where is this behavior checked?” and “Which run demonstrated it in my intended environment?” The index helps answer the first; the source and execution receipt together answer the second. If you add a regression case, retain the command, selected tests, environment and actual result alongside the source revision, so the next reader can follow the same route.

O.1 Scope and interpretation

This crosswalk identifies the test modules and verification programs in the final Python source. It connects each file to literal Python import statements and records its syntactic test definitions, assertions and source locations. These are navigation records. An import does not establish that every imported object was called, that a branch was exercised, or that a mathematical claim was proved.

The inventory is produced by parsing Python source with `ast`. It does not import the package, run pytest collection, expand parametrized cases, execute a verification program, or measure statement or branch coverage. A function whose name begins with `test_` is counted as a syntactic definition, including nested definitions. Its presence does not guarantee that pytest collects or executes it. Conditional imports are retained even when their branch might not execute. Imports made through runtime strings and attribute access through a package alias are not resolved into an invented call graph.

The complete JSON record retains every import name, alias and source line, all test-definition names and line ranges, top-level function names, per-file SHA-256 digests and the bound evidence receipts. Printed cards list literal Komorebi import targets. Files without such an import can still exercise the package through a subprocess, helper or dynamic route; inspect their source and execution evidence.

O.2 Frozen source identity

The source commit is `5f2b7ddc879c12d2bb00cbe5c487aa0bc7709222` and its repository tree is `7c9b4caee01fdc0bb64149e27ce46926241e21f2`. This current-source inventory is distinct from the recorded predecessor observations in Appendices A through N. The original test and diagnostic identifiers in those appendices remain locators for their stated source baseline.

The canonical path-and-SHA-256 manifest for these 328 files has digest `f19784e2b48a750161d9a3cc7576af477f5f58dc6747d9b5dd384cb099ed7d6e`. The full structured record inventory has digest `4bf60c79f3ab92132d5e1933f76cffaa68e8d1cf84fcebb153dcb08c3edd18b0`. Both digests use UTF-8 JSON with sorted keys and compact separators. The machine-readable inventory and the generator accompany the editable manual.

Source set	Files	Syntactic test definitions
Test modules	194	1655
Installed verification modules	28	0
Repository verification programs	106	0

These definition counts are not expected to equal a pytest result count. Parametrization can produce several runtime cases from one function, while selection rules can exclude a definition. Verification programs also contain helpers and entry points that are not pytest tests.

O.3 Recorded execution evidence

The retained release status records **2,752 passing source tests with seven optional-backend skips**. A separate CPU campaign records all seven of those cases passing, with no failures, errors or skips. Its input-binding receipt confirms that the campaign inputs match the final source; its source-parity receipt records only workflow-file differences outside that campaign. The installed direct wheel and the wheel rebuilt from the source distribution each record 1,556 passing tests. These reports predate this editorial pass and were not rerun to generate the crosswalk.

The release records also retain 84 executable documentation examples, 351 base stress checks and 11 resource controls. These groups overlap and must not be added together as independent mathematical proofs. This appendix does not promote a successful finite test into a Lean proof or an unbounded theorem.

Fresh hosted Linux/Windows qualification did not start because the provider reported an account payment or spending-limit restriction. Real NVIDIA qualification and a qualified tagged release-candidate predecessor remain publication prerequisites in the retained status. A prepared distribution and a configured workflow are not completed publication or hardware qualification.

Retained evidence file	SHA-256
`FINAL_RELEASE_STATUS.json`	`6e8e7f266a85bc1daaad17a66425fe6ec5683c62cdfa956c29cc42fa3a359b86`
`QUALIFICATION_SUMMARY.json`	`e5fe48201038f0f8b240db1fc2635817e21fd46c40399e8882c7cc1c8ef0f363`
`ORIGINAL_SKIP_COVERAGE.json`	`a17a5dd1835ba5ea4325bae9d2fed3126cf762dd0be3db09d504b2e47b98975c`
`CURRENT_SOURCE_PARITY.json`	`29af576652f77ffef79ea4fbc22b832dd00c40115cdd7fb919ae2e360250e3d6`

O.4 Test-module index

O-T001: `tests/test_algebra_quotients.py`

Source: lines 1 through 79. **Test definitions:** 8. **Assert statements:** 17.

File SHA-256: `87e0b1ef1b12ff54d1b29611d144a659bb2676a1ce6991395edc3ba0c7337b11`.

Literal Komorebi import targets: `komorebi_math.OperatorAlgebra`, `komorebi_math.PacketSystem`, `komorebi_math.independent_direct_sum_seed_rank`, `komorebi_math.matrix_units`, `komorebi_math.shared_direct_sum_seed_bounds`, `komorebi_math.tensor_seed_rank_upper_bound`.

O-T002: `tests/test_algorithms_compiler.py`

Source: lines 1 through 145. **Test definitions:** 9. **Assert statements:** 14.

File SHA-256: `67d1d01af3baa1175db0e12037405f24fc122d7ce1da0a52ad11e85f1a181c31`.

Literal Komorebi import targets: `komorebi_math.algorithms.TreeNode`, `komorebi_math.algorithms.combine_tree_objectives`, `komorebi_math.algorithms.compile_tree_budget`, `komorebi_math.algorithms.compile_tree_lagrangian`, `komorebi_math.algorithms.decision_margins`, `komorebi_math.algorithms.pareto_frontier`, `komorebi_math.algorithms.replace_tree_nodes`, `komorebi_math.exceptions.ValidationError`.

O-T003: `tests/test_algorithms_complexity_benchmarks.py`

Source: lines 1 through 95. **Test definitions:** 7. **Assert statements:** 12.

File SHA-256: `1a01d224db057a3ed7ed8f2a504bff828a86c46a0bf902fece7ddd15e8c68b85`.

Literal Komorebi import targets: `komorebi_math.algorithms.BenchmarkContract`, `komorebi_math.algorithms.PacketLayout`, `komorebi_math.algorithms.TimingScope`, `komorebi_math.algorithms.arithmetic_intensity`, `komorebi_math.algorithms.benchmark_callable`, `komorebi_math.algorithms.compare_benchmarks`, `komorebi_math.algorithms.dense_apply_flops`, `komorebi_math.algorithms.packet_apply_flops`, `komorebi_math.algorithms.packet_cost_estimate`, `komorebi_math.algorithms.roofline_time_lower_bound`, `komorebi_math.exceptions.ValidationError`.

O-T004: `tests/test_algorithms_discovery_gpu.py`

Source: lines 1 through 70. **Test definitions:** 5. **Assert statements:** 5.

File SHA-256: `ed549b27d30def1913ab25fdd648bae9fc36bdccbb2424b0f7aa786f25d900a0`.

Literal Komorebi import targets: `komorebi_math.algorithms.NumpyPacketExecutor`, `komorebi_math.algorithms.PacketIR`, `komorebi_math.algorithms.PacketLayout`, `komorebi_math.algorithms.cupy_available`, `komorebi_math.algorithms.discover_block_diagonal_low_rank`, `komorebi_math.algorithms.discover_packet_support`, `komorebi_math.algorithms.packet_executor`, `komorebi_math.exceptions.BackendUnavailableError`, `komorebi_math.exceptions.ValidationError`.

O-T005: `tests/test_algorithms_krylov.py`

Source: lines 1 through 125. **Test definitions:** 10. **Assert statements:** 19.

File SHA-256: `82c15d31ac5df3180ca28e84f68eb207110175969551d35a661d97c3ea1a6885`.

Literal Komorebi import targets: `komorebi_math.algorithms.PacketLayout`, `komorebi_math.algorithms.block_jacobi_certificate`, `komorebi_math.algorithms.block_jacobi_preconditioner`, `komorebi_math.algorithms.conjugate_gradient`, `komorebi_math.algorithms.krylov_basis`, `komorebi_math.algorithms.lanczos_tridiagonal`, `komorebi_math.algorithms.power_perturbation_bound`, `komorebi_math.algorithms.preconditioned_condition_bound`, `komorebi_math.exceptions.AssumptionError`, `komorebi_math.exceptions.ValidationError`.

O-T006: `tests/test_algorithms_layout_ir.py`

Source: lines 1 through 129. **Test definitions:** 12. **Assert statements:** 15.

File SHA-256: `c7c6c66d8446f6d1fe21b433f80c018fecb6a92829e52440ce06bca9a0dad0be`.

Literal Komorebi import targets: `komorebi_math.algorithms.DenseOperator`, `komorebi_math.algorithms.PacketIR`, `komorebi_math.algorithms.PacketLayout`, `komorebi_math.exceptions.DimensionError`, `komorebi_math.exceptions.ValidationError`.

O-T007: `tests/test_algorithms_low_rank.py`

Source: lines 1 through 110. **Test definitions:** 8. **Assert statements:** 11.

File SHA-256: `9d38da540075d8eefb44176c3ab2845725b7d16d15479057353f44f150f2d9bd`.

Literal Komorebi import targets: `komorebi_math.algorithms.BlockDiagonalLowRankOperator`, `komorebi_math.algorithms.BlockDiagonalOperator`, `komorebi_math.algorithms.LowRankOperator`, `komorebi_math.algorithms.accumulated_recompression_bound`, `komorebi_math.algorithms.product_chain_error_bound`, `komorebi_math.algorithms.recompress_core`, `komorebi_math.algorithms.recompress_factors`, `komorebi_math.exceptions.DimensionError`, `komorebi_math.exceptions.ValidationError`.

O-T008: `tests/test_algorithms_precision.py`

Source: lines 1 through 99. **Test definitions:** 7. **Assert statements:** 13.

File SHA-256: `361497947acbab852295aaf10d37c465b8a8202394f48615a1201cd55c702160`.

Literal Komorebi import targets: `komorebi_math.algorithms.DTypeChoice`, `komorebi_math.algorithms.assign_discrete_precisions`, `komorebi_math.algorithms.clipped_precision_allocation`, `komorebi_math.algorithms.optimal_precision_tolerances`, `komorebi_math.algorithms.packet_error_bound`, `komorebi_math.algorithms.precision_certific

ate, komorebi_math.algorithms.quantization_error, komorebi_math.algorithms.quantize_array, komorebi_math.algorithms.quantize_by_packet, komorebi_math.exceptions.ValidationError.

O-T009: tests/test_algorithms_scheduling.py

Source: lines 1 through 69. **Test definitions:** 7. **Assert statements:** 11.

File SHA-256: df06fc42c2288b7ace55021d764dbdd74e8b3e0cdc64e0392d361bf51062a0b5.

Literal Komorebi import targets: komorebi_math.algorithms.Task, komorebi_math.algorithms.amortization_threshold, komorebi_math.algorithms.batched_launch_count, komorebi_math.algorithms.break_even_analysis, komorebi_math.algorithms.schedule_unlimited, komorebi_math.algorithms.shape_classes, komorebi_math.algorithms.verify_schedule, komorebi_math.exceptions.ValidationError.

O-T010: tests/test_algorithms_structured.py

Source: lines 1 through 94. **Test definitions:** 7. **Assert statements:** 19.

File SHA-256: 578c4dc02420d573f210fa0916468b096ca720da20cd0b85e42fea595f1a9877.

Literal Komorebi import targets: komorebi_math.algorithms.BlockDiagonalOperator, komorebi_math.algorithms.CirculantOperator, komorebi_math.algorithms.DenseOperator, komorebi_math.algorithms.DiagonalOperator, komorebi_math.algorithms.IdentityOperator, komorebi_math.algorithms.KroneckerOperator, komorebi_math.algorithms.ToeplitzOperator, komorebi_math.algorithms.WalshHadamardOperator, komorebi_math.exceptions.ValidationError.

O-T011: tests/test_algorithms_support.py

Source: lines 1 through 78. **Test definitions:** 7. **Assert statements:** 13.

File SHA-256: 363774d256ce266afab4d193b24d2ecb6795b0af714417917820be42a0237ac0.

Literal Komorebi import targets: komorebi_math.algorithms.SupportGraph, komorebi_math.algorithms.inverse_support_if_acyclic, komorebi_math.algorithms.longest_path_vertices, komorebi_math.algorithms.schur_fill_support, komorebi_math.algorithms.support_polynomial, komorebi_math.algorithms.support_power, komorebi_math.algorithms.support_product, komorebi_math.algorithms.topological_levels, komorebi_math.algorithms.transitive_closure, komorebi_math.exceptions.ValidationError.

O-T012: tests/test_analytic_appell_zeros.py

Source: lines 1 through 122. **Test definitions:** 10. **Assert statements:** 20.

File SHA-256: c13bd23fdba66dfb85bf8a38efc828642c79d58a35458ad36908f3a48c2fbc33.

Literal Komorebi import targets: komorebi_math.analytic.CyclicDenominator, komorebi_math.analytic.appell_coefficients, komorebi_math.analytic.appell_polynomial, komorebi_math.analytic.appell_polynomial_symbolic, komorebi_math.analytic.congruential_inverse, komorebi_math.analytic.congruential_reciprocal_coefficients, komorebi_math.analytic.hadamard_partial_product, komorebi_math.analytic.positive_zero_string, komorebi_math.analytic.reciprocal_coefficient_from_zeros, komorebi_math.analytic.reciprocal_coefficients, komorebi_math.analytic.reduced_zero_zeta, komorebi_math.analytic.theta_polynomial, komorebi_math.analytic.theta_polynomial_symbolic, komorebi_math.analytic.zero_count_asymptotic, komorebi_math.certificates.Exactness, komorebi_math.exceptions.ValidationError.

O-T013: tests/test_analytic_cyclic_reciprocal.py

Source: lines 1 through 110. **Test definitions:** 9. **Assert statements:** 13.

File SHA-256: 991ad61bc8335078f70468c38e9116a860d50107f50f41a2d0bbe63a02ee6d8a.

Literal Komorebi import targets: komorebi_math.analytic.CyclicSectionFamily, komorebi_math.analytic.cyclic_packets, komorebi_math.analytic.cyclic_product_packets, komorebi_math.analytic.cyclic_project_coefficients, komorebi_math.analytic.cyclic_project_value, komorebi_math.analytic.cyclic_projection_certificate, komorebi_math.analytic.cyclic_reconstruct, komorebi_math.analytic.denominator, komorebi_math.analytic.denominator_root_sum, komorebi_math.analytic.logarithmic_coefficients, komorebi_math.analytic.reciprocal_coefficients, komorebi_math.analytic.reciprocal_polynomial, komorebi_math.analytic.reciprocal_semigroup_certificate, komorebi_math.

analytic.section_certificate, komorebi_math.analytic.signed_section, komorebi_math.analytic.signed_shift_matrix, komorebi_math.analytic.transfer_section, komorebi_math.analytic.transfer_tangent, komorebi_math.exceptions.ValidationError.

O-T014: tests/test_arithmetic_domains_valuation.py

Source: lines 1 through 72. **Test definitions:** 4. **Assert statements:** 26.

File SHA-256: 03e9e283989893ff2ca65cefffab54ee0e0bf8567d60b54ba6b141dacd769e87.

Literal Komorebi import targets: komorebi_math.arithmetic.ModularRing, komorebi_math.arithmetic.PrimeField, komorebi_math.arithmetic.TruncatedPadic, komorebi_math.arithmetic.binomial_valuation, komorebi_math.arithmetic.binomial_valuation_from_digits, komorebi_math.arithmetic.carmichael_exponent_prime_power, komorebi_math.arithmetic.digit_sum, komorebi_math.arithmetic.factorial_valuation, komorebi_math.arithmetic.falling_factorial, komorebi_math.arithmetic.padic_valuation, komorebi_math.arithmetic.unit_difference_valuation, komorebi_math.arithmetic.unit_difference_value, komorebi_math.exceptions.ValidationError.

O-T015: tests/test_arithmetic_euler.py

Source: lines 1 through 66. **Test definitions:** 4. **Assert statements:** 12.

File SHA-256: 1662441fdce46273b5e5fda9d6ed3ce1ed25f9f67225427e34b955a441d2d03f.

Literal Komorebi import targets: komorebi_math.arithmetic.EulerLocalCoordinates, komorebi_math.arithmetic.bernoulli_numbers, komorebi_math.arithmetic.dyadic_beta, komorebi_math.arithmetic.dyadic_kernel_valuation, komorebi_math.arithmetic.dyadic_row_weight, komorebi_math.arithmetic.dyadic_source, komorebi_math.arithmetic.euler_zigzag_numbers, komorebi_math.arithmetic.padic_valuation, komorebi_math.arithmetic.signed_euler_numbers.

O-T016: tests/test_arithmetic_hurwitz_windows.py

Source: lines 1 through 89. **Test definitions:** 8. **Assert statements:** 13.

File SHA-256: 9b69c3fe11d60aa90ee465bc5ce307231e1fac96dea5c8ac470c9564000c11cb.

Literal Komorebi import targets: komorebi_math.arithmetic.HurwitzSeries, komorebi_math.arithmetic.StandardWindow, komorebi_math.arithmetic.compressed_hurwitz_inverse, komorebi_math.arithmetic.hurwitz_inverse, komorebi_math.arithmetic.hurwitz_product, komorebi_math.arithmetic.subgroup_roots, komorebi_math.arithmetic.subgroup_selector, komorebi_math.exceptions.ValidationError.

O-T017: tests/test_arithmetic_mahler_polyphase.py

Source: lines 1 through 80. **Test definitions:** 7. **Assert statements:** 11.

File SHA-256: 29c6c63a7af89d34bb9f55d127865b8dea6fe3359c9d980f4f093b9b9c4002a4.

Literal Komorebi import targets: komorebi_math.arithmetic.MahlerSeries, komorebi_math.arithmetic.UnitModeMahler, komorebi_math.arithmetic.affine_kernel_composition_certificate, komorebi_math.arithmetic.affine_mahler_kernel, komorebi_math.arithmetic.evaluate_matrix_mahler, komorebi_math.arithmetic.mahler_polyphase, komorebi_math.arithmetic.matrix_mahler_coefficients, komorebi_math.arithmetic.mixed_radix_address, komorebi_math.arithmetic.mixed_radix_digits, komorebi_math.arithmetic.polyphase_analyze, komorebi_math.arithmetic.polyphase_synthesize, komorebi_math.arithmetic.prime_power_kernel_certificate.

O-T018: tests/test_arithmetic_recurrence.py

Source: lines 1 through 87. **Test definitions:** 8. **Assert statements:** 12.

File SHA-256: 3bd0baeea8248f862cd5c96d0e81b38582b44d7ef5e07af7043d70ccf706078e.

Literal Komorebi import targets: komorebi_math.arithmetic.LinearRecurrence, komorebi_math.arithmetic.fibonacci_recurrence, komorebi_math.arithmetic.lucas_recurrence, komorebi_math.arithmetic.matrix_power_exact, komorebi_math.arithmetic.matrix_power_mod, komorebi_math.arithmetic.second_order_jump, komorebi_math.exceptions.ValidationError.

O-T019: tests/test_artifact_gate_hardening.py

Source: lines 1 through 95. **Test definitions:** 9. **Assert statements:** 9.

File SHA-256: 459241e556c48d102fa06fda194f81e8bbcce4088086ef9d7b8ac48f3278b9e9.

Literal Komorebi import targets: none. This syntax result does not rule out subprocess, helper-mediated or dynamic package use.

O-T020: tests/test_backends.py

Source: lines 1 through 41. **Test definitions:** 4. **Assert statements:** 14.

File SHA-256: c6fdd33d15bc9fe56ed8e538beb86e53675cdfa533fe1238a1062650ed8604e2.

Literal Komorebi import targets: komorebi_math.backends.available_backends, komorebi_math.backends.get_backend.

O-T021: tests/test_block10_cli.py

Source: lines 1 through 50. **Test definitions:** 5. **Assert statements:** 18.

File SHA-256: 805edea456ebc671b687f4e291278f4d102b83ae36cb29e3ef0ab51867e3784c.

Literal Komorebi import targets: komorebi_math.cli.main.main, komorebi_math.compatibility.check_public_api.

O-T022: tests/test_block10_compatibility.py

Source: lines 1 through 134. **Test definitions:** 7. **Assert statements:** 20.

File SHA-256: d9a941e176c87c9e4087d747bd644e32d0f84cc339e5ed105d68c65b40bb0463.

Literal Komorebi import targets: komorebi_math.compatibility.ApiCompatibilityReport, komorebi_math.compatibility.KomorebiDeprecationWarning, komorebi_math.compatibility.PublicApiManifest, komorebi_math.compatibility.PublicApiSymbol, komorebi_math.compatibility.check_public_api, komorebi_math.compatibility.compare_public_api, komorebi_math.compatibility.create_public_api_manifest, komorebi_math.compatibility.deprecated, komorebi_math.compatibility.load_public_api_manifest, komorebi_math.compatibility.save_public_api_manifest, komorebi_math.exceptions.ValidationError.

O-T023: tests/test_block10_execution_persistence.py

Source: lines 1 through 267. **Test definitions:** 17. **Assert statements:** 29.

File SHA-256: 665eab4e55e8155121d1120d966ea077beaae29cf56308f6af53915c56d41df7.

Literal Komorebi import targets: komorebi_math.Exactness, komorebi_math.PacketSystem, komorebi_math.certificates.Certificate, komorebi_math.deformation.LiftBranch, komorebi_math.exceptions.BackendUnavailableError, komorebi_math.exceptions.ValidationError, komorebi_math.export.dumps_portable, komorebi_math.export.from_serializable, komorebi_math.export.load_portable, komorebi_math.export.loads_portable, komorebi_math.export.portable_digest, komorebi_math.export.register_portable_codec, komorebi_math.export.save_portable, komorebi_math.export.to_serializable, komorebi_math.nonlinear.CuPyPacketJetExecutor, komorebi_math.nonlinear.CuPyPacketJetSnapshotExecutor, komorebi_math.nonlinear.FiniteLiftTree, komorebi_math.nonlinear.LiftTreeNode, komorebi_math.nonlinear.OrderedRootedTree, komorebi_math.nonlinear.PacketJetFamily, komorebi_math.nonlinear.PolynomialDerivativeProvider, komorebi_math.nonlinear.SparseSymmetricTensor, komorebi_math.nonlinear.compile_packet_jet_plan, komorebi_math.nonlinear.cupy_runtime_status, komorebi_math.nonlinear.execute_packet_jet_plan, komorebi_math.nonlinear.filtered.FilteredSpectralSequenceResult, komorebi_math.nonlinear.filtered.SpectralSequenceEntry, komorebi_math.nonlinear.filtered.SpectralSequencePage, komorebi_math.nonlinear.packet_jet_support, komorebi_math.nonlinear.snapshot_packet_jet.

O-T024: `tests/test_block10_sparse_autodiff.py`

Source: lines 1 through 194. **Test definitions:** 11. **Assert statements:** 19.

File SHA-256: `85864e6b4a47fbd8521238765ae9fe97cc38f17a47e2bc5dcecafe70b012df99`.

Literal Komorebi import targets: `komorebi_math.Exactness`, `komorebi_math.exceptions.BackendUnavailableError`, `komorebi_math.exceptions.ValidationError`, `komorebi_math.nonlinear.AutomaticDifferentiationProvider`, `komorebi_math.nonlinear.JaxDerivativeProvider`, `komorebi_math.nonlinear.PolynomialDerivativeProvider`, `komorebi_math.nonlinear.SparsePolynomialDerivativeProvider`, `komorebi_math.nonlinear.SparseSymmetricTensor`, `komorebi_math.nonlinear.TorchDerivativeProvider`, `komorebi_math.nonlinear.autodiff_provider`, `komorebi_math.nonlinear.available_ad_frameworks`.

O-T025: `tests/test_block11_cli.py`

Source: lines 1 through 72. **Test definitions:** 4. **Assert statements:** 19.

File SHA-256: `8c692d9126aec1d007934e8c591fdd82a5bb941bd73fcf301585229916075a59`.

Literal Komorebi import targets: `komorebi_math.PacketSystem`, `komorebi_math.cli.main.main`, `komorebi_math.nonlinear.PacketJetFamily`, `komorebi_math.nonlinear.PolynomialDerivativeProvider`, `komorebi_math.nonlinear.save_packet_jet_snapshot_stream`, `komorebi_math.nonlinear.snapshot_packet_jet`.

O-T026: `tests/test_block11_golden_profiling_beta.py`

Source: lines 1 through 187. **Test definitions:** 8. **Assert statements:** 38.

File SHA-256: `6ed1804bd473f364f9f0dd534f1ab8cc457345d3e92b8db8a1c6a64b1eb47bd0`.

Literal Komorebi import targets: `komorebi_math.PacketSystem`, `komorebi_math.beta`, `komorebi_math.compatibility.BETA_API_STABILITY_LEVEL`, `komorebi_math.compatibility.BETA_PUBLIC_NAMESPACES`, `komorebi_math.compatibility.PublicApiManifest`, `komorebi_math.compatibility.PublicApiSymbol`, `komorebi_math.compatibility.check_beta_api`, `komorebi_math.compatibility.compare_public_api`, `komorebi_math.compatibility.create_beta_api_manifest`, `komorebi_math.compatibility.load_beta_api_manifest`, `komorebi_math.compatibility.save_public_api_manifest`, `komorebi_math.exceptions.ValidationError`, `komorebi_math.golden.load_behavioral_golden_fixture`, `komorebi_math.golden.run_behavioral_golden_fixture`, `komorebi_math.nonlinear.PacketJetFamily`, `komorebi_math.nonlinear.PolynomialDerivativeProvider`, `komorebi_math.nonlinear.estimate_packet_jet_complexity`, `komorebi_math.nonlinear.packet_jet_support`, `komorebi_math.nonlinear.profile_operation`, `komorebi_math.nonlinear.profile_packet_jet_runtime`.

O-T027: `tests/test_block11_streaming_migrations.py`

Source: lines 1 through 361. **Test definitions:** 10. **Assert statements:** 31.

File SHA-256: `5a2a13ded37264006dbfaefae55051544aee0895cb1ff8cc6cfd767b44595eb7`.

Literal Komorebi import targets: `komorebi_math.Exactness`, `komorebi_math.PacketSystem`, `komorebi_math.certificates.Certificate`, `komorebi_math.exceptions.ValidationError`, `komorebi_math.export.dumps_portable`, `komorebi_math.export.loads_portable`, `komorebi_math.export.register_portable_codec`, `komorebi_math.export.register_portable_migration`, `komorebi_math.export.to_portable_data`, `komorebi_math.nonlinear.PacketJetAddress`, `komorebi_math.nonlinear.PacketJetFamily`, `komorebi_math.nonlinear.PacketJetSnapshot`, `komorebi_math.nonlinear.PolynomialDerivativeProvider`, `komorebi_math.nonlinear.capture_packet_jet_stream`, `komorebi_math.nonlinear.load_packet_jet_snapshot_stream`, `komorebi_math.nonlinear.open_packet_jet_snapshot_stream`, `komorebi_math.nonlinear.save_packet_jet_snapshot_stream`, `komorebi_math.nonlinear.snapshot_packet_jet`.

O-T028: `tests/test_block12_cli_api.py`

Source: lines 1 through 52. **Test definitions:** 5. **Assert statements:** 15.

File SHA-256: `308dafe17b037b91cd107337b9888cbf93b96e120d4f73f99d06543fe2d31217`.

Literal Komorebi import targets: `komorebi_math.__version__`, `komorebi_math.cli.main.main`, `komorebi_math.compatibility.BETA_PUBLIC_NAMESPACES`, `komorebi_math.compatibility.DEFAULT_PUBLIC_NAMESPACES`, `komorebi_math.compatibility.check_beta_api`, `komorebi_math.compatibility.check_public_api`, `komorebi_math.compatibility.create_public_api_manifest`.

O-T029: `tests/test_block12_denoise_streaming.py`

Source: lines 1 through 179. **Test definitions:** 13. **Assert statements:** 21.

File SHA-256: `d15bf3106c148976800f6ca134a79d5da7bedf20fb43fd43ae231dbf6dfd089a`.

Literal Komorebi import targets: `komorebi_math.exceptions.DimensionError`, `komorebi_math.exceptions.ValidationError`, `komorebi_math.signals.StreamingSTFT`, `komorebi_math.signals.denoising_risk_certificate`, `komorebi_math.signals.estimate_white_noise_sigma`, `komorebi_math.signals.haar_decompose`, `komorebi_math.signals.median_absolute_deviation`, `komorebi_math.signals.stream_stft`, `komorebi_math.signals.sure_threshold`, `komorebi_math.signals.threshold_coefficients`, `komorebi_math.signals.universal_threshold`, `komorebi_math.signals.wavelet_denoise`.

O-T030: `tests/test_block12_recursive_fft.py`

Source: lines 1 through 207. **Test definitions:** 14. **Assert statements:** 27.

File SHA-256: `72990e461008b30b481ad91be1d37469ceef49839ef0806278b43606eab199b5`.

Literal Komorebi import targets: `komorebi_math.exceptions.DimensionError`, `komorebi_math.exceptions.ValidationError`, `komorebi_math.signals.LinearFilter`, `komorebi_math.signals.OverlapAddConvolver`, `komorebi_math.signals.StreamingLinearFilter`, `komorebi_math.signals.certified_overlap_add_convolve`, `komorebi_math.signals.design_biquad`, `komorebi_math.signals.fft_convolve`, `komorebi_math.signals.lfilter`, `komorebi_math.signals.overlap_add_convolve`, `komorebi_math.signals.stream_lfilter`.

O-T031: `tests/test_block12_resampling_adaptive.py`

Source: lines 1 through 164. **Test definitions:** 16. **Assert statements:** 28.

File SHA-256: `77c99acac918ceb6524cff3a8acc24be5dc33cb6fdbaff7b3b86ccafb9883600`.

Literal Komorebi import targets: `komorebi_math.exceptions.DimensionError`, `komorebi_math.exceptions.ValidationError`, `komorebi_math.signals.RationalResamplingPlan`, `komorebi_math.signals.decimate`, `komorebi_math.signals.design_resample_filter`, `komorebi_math.signals.downsample`, `komorebi_math.signals.fir_filter`, `komorebi_math.signals.interpolate`, `komorebi_math.signals.lms_filter`, `komorebi_math.signals.nlms_filter`, `komorebi_math.signals.resample_poly`, `komorebi_math.signals.resample_poly_result`, `komorebi_math.signals.resampled_length`, `komorebi_math.signals.rls_filter`, `komorebi_math.signals.upsample`.

O-T032: `tests/test_block12_spectral_filtering.py`

Source: lines 1 through 194. **Test definitions:** 16. **Assert statements:** 28.

File SHA-256: `a4be856cc734902db527a3ba8349ef16c1eea4050151ccec4102f89d3c111045`.

Literal Komorebi import targets: `komorebi_math.exceptions.DimensionError`, `komorebi_math.exceptions.ValidationError`, `komorebi_math.signals.StreamingFIR`, `komorebi_math.signals.cross_spectrum`, `komorebi_math.signals.design_bandpass`, `komorebi_math.signals.design_bandstop`, `komorebi_math.signals.design_fir`, `komorebi_math.signals.design_highpass`, `komorebi_math.signals.design_lowpass`, `komorebi_math.signals.fir_filter`, `komorebi_math.signals.fir_frequency_response`, `komorebi_math.signals.parseval_certificate`, `komorebi_math.signals.periodogram`, `komorebi_math.signals.stream_fir_filter`, `komorebi_math.signals.welch_psd`.

O-T033: `tests/test_block12_windows_time_frequency.py`

Source: lines 1 through 173. **Test definitions:** 17. **Assert statements:** 36.

File SHA-256: `13bd62fad94e6be6e973aff3148682e78cbc528bea1c0cc8adb0a37ae08a4db0`.

Literal Komorebi import targets: `komorebi_math.exceptions.AssumptionError`, `komorebi_math.exceptions.DimensionError`, `komorebi_math.exceptions.ValidationError`, `komorebi_math.signals.canonical_dual_window`, `komorebi_math.signals.frame_signal`, `komorebi_math.signals.get_window`, `komorebi_math.signals.istft`, `komorebi_math.signals.overlap_add`, `komorebi_math.signals.overlap_add_certificate`, `komorebi_math.signals.overlap_profile`, `komorebi_math.signals.spectrogram`, `komorebi_math.signals.stft`, `komorebi_math.signals.window_metrics`.

O-T034: `tests/test_block13_cli_api.py`

Source: lines 1 through 90. **Test definitions:** 5. **Assert statements:** 21.

File SHA-256: `eee90f4b0b8ea37ede417e3ce3eb16c24a4247e867e8d2b3d447c5fa8ccbce52`.

Literal Komorebi import targets: `komorebi_math.FiniteNormalizedSeries`, `komorebi_math.cli.main.main`, `komorebi_math.compatibility.DEFAULT_PUBLIC_NAMESPACES`, `komorebi_math.compatibility.check_beta_api`, `komorebi_math.compatibility.check_public_api`, `komorebi_math.compatibility.create_public_api_manifest`, `komorebi_math.feature`, `komorebi_math.generating.BellResidueAutomaton`, `komorebi_math.generating.GeneratingBudget`.

O-T035: `tests/test_block14_cli_api.py`

Source: lines 1 through 125. **Test definitions:** 5. **Assert statements:** 38.

File SHA-256: `d0af2f00cc7ebaca41744f9fe9409ac08e4b6eb09e3ee764ce4dd15610fb8554`.

Literal Komorebi import targets: `komorebi_math.CoordinateTransform`, `komorebi_math.cli.main.main`, `komorebi_math.compatibility.DEFAULT_PUBLIC_NAMESPACES`, `komorebi_math.compatibility.check_beta_api`, `komorebi_math.compatibility.check_public_api`, `komorebi_math.compatibility.create_public_api_manifest`, `komorebi_math.feature`.

O-T036: `tests/test_block15_persistence.py`

Source: lines 1 through 106. **Test definitions:** 10. **Assert statements:** 18.

File SHA-256: `fdcc7603d590640a027f206162c600728f1adab247a8d9f1478100da1165402f`.

Literal Komorebi import targets: `komorebi_math.arithmetic.polyphase_analyze`, `komorebi_math.exceptions.ValidationError`, `komorebi_math.export.persistence.dumps_portable`, `komorebi_math.export.persistence.from_portable_data`, `komorebi_math.export.persistence.load_portable`, `komorebi_math.export.persistence.loads_portable`, `komorebi_math.export.persistence.portable_digest`, `komorebi_math.export.persistence.save_portable`, `komorebi_math.export.persistence.to_portable_data`, `komorebi_math.refinement.LatticePolyphaseRefinement`, `komorebi_math.refinement.PolyphaseRefinement`, `komorebi_math.refinement.PolyphaseRefinementTower`, `komorebi_math.refinement.residue_analyze`, `komorebi_math.signals.lattice_polyphase_analyze`, `komorebi_math.signals.polyphase_analyze`.

O-T037: `tests/test_block15_polyphase_refinement.py`

Source: lines 1 through 246. **Test definitions:** 14. **Assert statements:** 45.

File SHA-256: `26e277967a58ee361d0d814682bf5a75ff36772d217e326cdf477e0d237f523e`.

Literal Komorebi import targets: `komorebi_math.api.v1`, `komorebi_math.arithmetic.polyphase.polyphase_analyze`, `komorebi_math.compatibility.check_stable_api`, `komorebi_math.exceptions.ValidationError`, `komorebi_math.export.persistence.dumps_portable`, `komorebi_math.export.persistence.loads_portable`, `komorebi_math.refinement.LatticePolyphaseRefinement`, `komorebi_math.refinement.PolyphaseRefinement`, `komorebi_math.refinement.PolyphaseRefinementTower`, `komorebi_math.refinement.coarsen_lattice_polyphase`, `komorebi_math.refinement.coarsen_polyphase`, `komorebi_math.refinement.polyphase_refinement_certificate`, `komorebi_math.refinement.refine_lattice_polyphase`, `komorebi_math.refinement.refine_polyphase`, `komorebi_math.refinement.residue_analyze`, `komorebi_math.signals.lattice_polyphase_analyze`, `komorebi_math.signals.polyphase_analyze`, `komorebi_math.stable`, `komorebi_math.v1`.

O-T038: `tests/test_block15_stable_api.py`

Source: lines 1 through 87. **Test definitions:** 9. **Assert statements:** 22.

File SHA-256: `e5820f018f15cf3eaac54a072f4e8a2d7edc914d8b7efa84a4bf014ac91b1690`.

Literal Komorebi import targets: `komorebi_math`, `komorebi_math.api.v1`, `komorebi_math.compatibility.STABLE_API_STABILITY_LEVEL`, `komorebi_math.compatibility.STABLE_PUBLIC_NAMESPACES`, `komorebi_math.compatibility._public_symbol`, `komorebi_math.compatibility.check_stable_api`, `komorebi_math.compatibility.create_stable_api_manifest`, `komorebi_math.compatibility.load_stable_api_manifest`, `komorebi_math.export.persistence.portable_codecs`, `komorebi_math.refinement.PolyphaseRefinement`, `komorebi_math.stable`, `komorebi_math.v1`.

O-T039: `tests/test_block16_analysis_synthesis.py`

Source: lines 1 through 87. **Test definitions:** 9. **Assert statements:** 19.

File SHA-256: `5ec653b61ccd825ffd8787a61528b76f3bb81c98e97389a0d463714e3e2fc183`.

Literal Komorebi import targets: `komorebi_math`, `komorebi_math.api.v1`, `komorebi_math.compatibility.load_stable_api_manifest`, `komorebi_math.coordinates.AnalysisSynthesisBlocks`, `komorebi_math.coordinates.AnalysisSynthesisSystem`, `komorebi_math.exceptions.DimensionError`, `komorebi_math.exceptions.ValidationError`, `komorebi_math.export.persistence.dumps_portable`, `komorebi_math.export.persistence.loads_portable`, `komorebi_math.export.persistence.portable_codecs`, `komorebi_math.stable`, `komorebi_math.v1`.

O-T040: `tests/test_block16_cli.py`

Source: lines 1 through 45. **Test definitions:** 4. **Assert statements:** 11.

File SHA-256: `80ab4f7db297ce3fe540f39fc792b2ea987607a8d9261bf72f176dc17e9379af`.

Literal Komorebi import targets: `komorebi_math.__version__`, `komorebi_math.cli.main.build_parser`, `komorebi_math.cli.main.main`.

O-T041: `tests/test_block16_companion_crosswalk.py`

Source: lines 1 through 47. **Test definitions:** 2. **Assert statements:** 12.

File SHA-256: `dc75bb6869d60b71a45bcd96e2cd562f26c73697574ab3d770999e9241d813e6`.

Literal Komorebi import targets: none. This syntax result does not rule out subprocess, helper-mediated or dynamic package use.

O-T042: `tests/test_block16_contracts.py`

Source: lines 1 through 112. **Test definitions:** 8. **Assert statements:** 15.

File SHA-256: `c1bf143637a92611de98cd5195f92ea5e98e4229527858d4527117ab4c0867b5`.

Literal Komorebi import targets: `komorebi_math.__version__`, `komorebi_math.compatibility.create_beta_api_manifest`, `komorebi_math.compatibility.create_public_api_manifest`, `komorebi_math.compatibility.create_stable_api_manifest`, `komorebi_math.coordinates.AnalysisSynthesisSystem`, `komorebi_math.exceptions.ValidationError`, `komorebi_math.persistence_contract.PortableCodecContract`, `komorebi_math.persistence_contract.PortableCodecContractEntry`, `komorebi_math.persistence_contract.compare_portable_codec_contract`, `komorebi_math.persistence_contract.create_portable_codec_contract`, `komorebi_math.stable_contract.check_stable_contract`, `komorebi_math.stable_semantics.run_stable_semantics_fixture`, `komorebi_math.support.SupportMatrix`, `komorebi_math.support.load_support_matrix`.

O-T043: `tests/test_block16_gpu_qualification.py`

Source: lines 1 through 68. **Test definitions:** 4. **Assert statements:** 6.

File SHA-256: `9caf9efdfafd533240c5dcb304d5ab8560b6ad98d2ffc46a1297d7e0182d03b6`.

Literal Komorebi import targets: `komorebi_math.exceptions.ValidationError`, `komorebi_math.release.gpu`, `komorebi_math.release.gpu.GpuBackendResult`, `komorebi_math.release.gpu.qualify_gpu`.

O-T044: `tests/test_block16_release_contract.py`

Source: lines 1 through 103. **Test definitions:** 8. **Assert statements:** 34.

File SHA-256: `126f1502ec450b4c13d12cd2f6112e007765a47f22672a7ec948858f7f8fc527`.

Literal Komorebi import targets: `komorebi_math`, `komorebi_math.__version__`, `komorebi_math.api.v1`, `komorebi_math.cli.contracts.API_CONTRACT_MODES`, `komorebi_math.cli.contracts.VERIFICATION_SUITE_NAMES`, `komorebi_math.exceptions.ValidationError`, `komorebi_math.release.ApiTier`, `komorebi_math.release.ReleaseContract`, `komorebi_math.release.check_release_contract`, `komorebi_math.release.current_runtime_record`, `komorebi_math.release.load_release_contract`, `komorebi_math.release.load_upgrade_contract`, `komorebi_math.stable`, `komorebi_math.v1`.

O-T045: `tests/test_block16_release_layout.py`

Source: lines 1 through 38. **Test definitions:** 3. **Assert statements:** 3.

File SHA-256: `a07ceeab31609cff660117cf8575299d52d8d4d4d14f6b1c131f2ad32f02fa0f`.

Literal Komorebi import targets: none. This syntax result does not rule out subprocess, helper-mediated or dynamic package use.

O-T046: `tests/test_block16_release_qualification.py`

Source: lines 1 through 85. **Test definitions:** 6. **Assert statements:** 20.

File SHA-256: `c03fd2c233fac6d6beadbf3684b8cae305107fd0378cbaa9bb9572cf843f7f04`.

Literal Komorebi import targets: `komorebi_math.__version__`, `komorebi_math.compatibility.STABLE_API_STABILITY_LEVEL`, `komorebi_math.compatibility.check_beta_api`, `komorebi_math.compatibility.check_public_api`, `komorebi_math.compatibility.check_stable_api`, `komorebi_math.compatibility.create_stable_api_manifest`, `komorebi_math.export.persistence.load_portable`, `komorebi_math.qualification.RELEASE_QUALIFICATION_SCHEMA`, `komorebi_math.qualification.run_release_qualification`, `komorebi_math.qualification.run_stable_semantic_verification`, `komorebi_math.qualification.runtime_inventory`, `komorebi_math.refinement.PolyphaseRefinement`, `komorebi_math.refinement.PolyphaseRefinementTower`, `komorebi_math.verify.run_stable_verification`.

O-T047: `tests/test_block16_static_audit.py`

Source: lines 1 through 47. **Test definitions:** 4. **Assert statements:** 13.

File SHA-256: `fc1b4aee3dd0d1d47a2a0ac4c06ba7e5ef947130221977fc2850e0334026adbf`.

Literal Komorebi import targets: none. This syntax result does not rule out subprocess, helper-mediated or dynamic package use.

O-T048: `tests/test_block17_cli.py`

Source: lines 1 through 55. **Test definitions:** 6. **Assert statements:** 19.

File SHA-256: `f2214e15bf0f6b106ba1eb0498cf1ea3aa698cd2ac3e6b0276b05f2310e48dae`.

Literal Komorebi import targets: `komorebi_math.__version__`, `komorebi_math.cli.main.build_parser`, `komorebi_math.cli.main.main`.

O-T049: `tests/test_block17_documentation.py`

Source: lines 1 through 81. **Test definitions:** 5. **Assert statements:** 14.

File SHA-256: `6378a8c631097838a076e3e2ad2605b5f8d4f8a4ef69cc72cc0107aad3549b15`.

Literal Komorebi import targets: `komorebi_math.__version__`, `komorebi_math.exceptions.ValidationError`, `komorebi_math.release.StableDocumentationContract`, `komorebi_math.release.check_stable_documentation`, `komorebi_math.release.create_stable_documentation_contract`, `komorebi_math.release.documentation`, `komorebi_math.release.load_stable_documentation`.

O-T050: `tests/test_block17_evidence.py`

Source: lines 1 through 70. **Test definitions:** 6. **Assert statements:** 17.

File SHA-256: `7853bfe44bad8c3c81a2f149783d572e6a19467d041744e8f978c9772ad678ac`.

Literal Komorebi import targets: `komorebi_math.exceptions.ValidationError`, `komorebi_math.release.ValidationEvidence`, `komorebi_math.release.check_beta_readiness`, `komorebi_math.release.check_validation_evidence`, `komorebi_math.release.evidence._check_historical_validation_evidence`, `komorebi_math.release.load_validation_evidence`.

O-T051: `tests/test_block17_package_audit.py`

Source: lines 1 through 31. **Test definitions:** 2. **Assert statements:** 13.

File SHA-256: `34ba47e54116dfee8702a75a25acc382e4bb10e73256f495f8d3fa4dd25484cf`.

Literal Komorebi import targets: `komorebi_math.__version__`, `komorebi_math.release.load_release_contract`.

O-T052: `tests/test_block17_release_contract.py`

Source: lines 1 through 42. **Test definitions:** 4. **Assert statements:** 17.

File SHA-256: `3bf8ad5c42f5ec75a7f2fbdc9bf92b3efe0617a94958e551bc3d523ba0d044cc`.

Literal Komorebi import targets: `komorebi_math.__version__`, `komorebi_math.release.check_beta_readiness`, `komorebi_math.release.check_release_contract`, `komorebi_math.release.load_release_contract`, `komorebi_math.verify.run_block17_verification`.

O-T053: `tests/test_block17_release_layout.py`

Source: lines 1 through 43. **Test definitions:** 2. **Assert statements:** 7.

File SHA-256: `8d96ecb664b031d89d81f6c62e9c4a3d09c5421cba99319e5ed45bfff87d5b78`.

Literal Komorebi import targets: none. This syntax result does not rule out subprocess, helper-mediated or dynamic package use.

O-T054: `tests/test_block17_static_audit.py`

Source: lines 1 through 8. **Test definitions:** 1. **Assert statements:** 3.

File SHA-256: `b5fa30be378dbc19d86068ebeaab91472e7bbe7281826866f8f9fc10d9ddce9c`.

Literal Komorebi import targets: none. This syntax result does not rule out subprocess, helper-mediated or dynamic package use.

O-T055: `tests/test_block17_upgrade_contract.py`

Source: lines 1 through 99. **Test definitions:** 7. **Assert statements:** 23.

File SHA-256: `05f02702d356a24e3820f84ab45e4a196b5452663d19ed319ce611ce353e6d19`.

Literal Komorebi import targets: `komorebi_math.__version__`, `komorebi_math.compatibility.compare_public_api`, `komorebi_math.compatibility.create_stable_api_manifest`, `komorebi_math.compatibility.load_public_api_manifest`, `komorebi_math.exceptions.ValidationError`, `komorebi_math.export.persistence.load_portable`, `komorebi_math.refinement.PolyphaseRefinement`, `komorebi_math.refinement.PolyphaseRefinementTower`, `komorebi_math.release.UpgradeContract`, `komorebi_math.release.check_upgrade_contract`, `komorebi_math.release.load_upgrade_contract`.

O-T056: `tests/test_block18_frame_algebra.py`

Source: lines 1 through 478. **Test definitions:** 14. **Assert statements:** 38.

File SHA-256: `7abaf8c01602c7c0288f3453e443971e8f1f16783c4ab5d9f19d31aa3b1ac3d2`.

Literal Komorebi import targets: `komorebi_math.api.v1`, `komorebi_math.compatibility.check_stable_api`, `komorebi_math.compatibility.create_stable_api_manifest`, `komorebi_math.coordinates.AnalysisSynthesisCoefficientBlocks`, `komorebi_math.coordinates.AnalysisSynthesisGauge`, `komorebi_math.coordinates.AnalysisSynthesisMap`, `komorebi_math.coordinates.AnalysisSynthesisSystem`, `komorebi_math.coordinates.CoefficientPartition`, `komorebi_math.coordinates.ExactAnalysisSynthesisMap`, `komorebi_math.coordinates.ExactAnalysisSynthesisSystem`, `komorebi_math.coordinates.exact.identity_matrix`, `komorebi_math.coordinates.exact.matrix_multiply`, `komorebi_math.exceptions.ValidationError`, `komorebi_math.stable`, `komorebi_math.stable_semantics.run_stable_semantics_fixture`, `komorebi_math.v1`.

O-T057: `tests/test_block19_integration.py`

Source: lines 1 through 888. **Test definitions:** 47. **Assert statements:** 114.

File SHA-256: `e10951338aeb2b6b800c4b5195af0a6c683eacd8a08a24dea16f8d8ebfe1d2b8`.

Literal Komorebi import targets: `komorebi_math`, `komorebi_math.beta`, `komorebi_math.certificates.Certificate`, `komorebi_math.certificates.ComputationResult`, `komorebi_math.certificates.Exactness`, `komorebi_math.compatibility.load_stable_api_manifest`, `komorebi_math.coordinates.AnalysisSynthesisSystem`, `komorebi_math.coordinates.euler_polynomial_family`, `komorebi_math.coordinates.fibonacci_polynomial_family`, `komorebi_math.coordinates.legendre_polynomial_family`, `komorebi_math.coordinates.monomial_polynomial_family`, `komorebi_math.exceptions.DimensionError`, `komorebi_math.exceptions.ValidationError`, `komorebi_math.export.portable_codecs`, `komorebi_math.generating.FormalSeries`, `komorebi_math.generating.ordinary_basis`, `komorebi_math.integration.AtomicIntegratedDerivativeProvider`, `komorebi_math.integration.AtomicMeasure`, `komorebi_math.integration.CompositeSimpson`, `komorebi_math.integration.DerivativeBound`, `komorebi_math.integration.ErrorGuarantee`, `komorebi_math.integration.ExactAffineMap`, `komorebi_math.integration.ExactPolynomial`, `komorebi_math.integration.ExactQuadratureRule`, `komorebi_math.integration.IntervalMeasure`, `komorebi_math.integration.PolynomialIntegrationCovector`, `komorebi_math.integration.ProductMeasure`, `komorebi_math.integration.PushforwardMeasure`, `komorebi_math.integration.QuadratureRule`, `komorebi_math.integration.affine_interval_change_of_variables`, `komorebi_math.integration.all_fubini_orders`, `komorebi_math.integration.analysis_synthesis_integration_certificate`, `komorebi_math.integration.coordinate_fundamental_theorem`, `komorebi_math.integration.coordinate_transport_identity`, `komorebi_math.integration.fubini_identity`, `komorebi_math.integration.fundamental_theorem`, `komorebi_math.integration.integrate`, `komorebi_math.integration.integrate_formal_series`, `komorebi_math.integration.integrate_polynomial_coordinates`, `komorebi_math.integration.integrate_polynomial_oriented`, `komorebi_math.integration.integration_by_parts`, `komorebi_math.integration.polynomial_derivative_bound`, `komorebi_math.integration.polynomial_from_family_coordinates`, `komorebi_math.integration.polynomial_integration_covector`, `komorebi_math.integration.pushforward_identity`, `komorebi_math.persistence_contract.check_portable_codec_contract`, `komorebi_math.persistence_contract.load_portable_codec_contract`, `komorebi_math.stable`.

O-T058: `tests/test_block20_expression_calculus.py`

Source: lines 1 through 807. **Test definitions:** 32. **Assert statements:** 91.

File SHA-256: `7678bfa2db0c1e4d0b558dcb826fcde322faafc5ba364cdff0ed60c2d6458245`.

Literal Komorebi import targets: `komorebi_math.certificates.Certificate`, `komorebi_math.certificates.Exactness`, `komorebi_math.exceptions.DimensionError`, `komorebi_math.exceptions.ResourceLimitError`, `komorebi_math.exceptions.ValidationError`, `komorebi_math.expression.Apply`, `komorebi_math.expression.Compose`, `komorebi_math.expression.ExactMapLiteral`, `komorebi_math.expression.ExactValueLiteral`, `komorebi_math.expression.ExpressionBinding`, `komorebi_math.expression.ExpressionBudget`, `komorebi_math.expression.IdentityMap`, `komorebi_math.expression.Let`, `komorebi_math.expression.LinearMapType`, `komorebi_math.expression.PolynomialCoordinateSpace`, `komorebi_math.expression.PolynomialPrimitive`, `komorebi_math.expression.PrimitiveKind`, `komorebi_math.expression.RATIONAL_SPACE`, `komorebi_math.expression.Scale`, `komorebi_math.expression.Sum`, `komorebi_math.expression.Symbol`, `komorebi_math.expression.ZeroMap`, `komorebi_math.expression.add`, `komorebi_math.expression.affine_substitution`, `komorebi_math.expression.antiderivative`, `komorebi_math.expression.apply`, `komorebi_math.expression.boundary_functional`, `komorebi_math.expression.check_expression`, `komorebi_math.expression.compile_expression`, `komorebi_math.expression.compose`, `komorebi_math.expression.constant_injection`, `komorebi_math.expression.coordinate_change`, `komorebi_math.expression.derivative`, `komorebi_math.expression.equivalent`, `komorebi_math.expression.evaluate_expression`, `komorebi_math.expression.evaluation`, `komorebi_math.expression.exact_binding`, `komorebi_math.expression.exact_internal_certificate`, `komorebi_math.expression.exact_map`, `komorebi_math.expression.exact_value`, `komorebi_math.expression.execute_plan`, `komorebi_math.expression.expression_digest`, `komorebi_math.expression.expression_from_dict`, `komorebi_math.expression.expression_from_json`, `komorebi_math.expression.expression_to_dict`, `komorebi_math.expression.expression_to_json`, `komorebi_math.expression.finite_difference`, `komorebi_math.expression.identity`, `komorebi_math.expression.interval_integration`, `komorebi_math.expression.let`, `komorebi_math.expression.matrix_chain_multiply`, `komorebi_math.expression.matrix_chain_parenthesization`, `komorebi_math.expression.matrix_chain_plan`, `komorebi_math.expression.normalize_expression`, `komorebi_math.expression.replay`, `komorebi_math.expression.scale`, `komorebi_math.expression.simplify_expression`, `komorebi_math.expression.symbol`, `komorebi_math.expression.type_from_dict`, `komorebi_math.expression.type_to_dict`, `komorebi_math.expression.zero_map`.

O-T059: `tests/test_block21_distribution_builder.py`

Source: lines 1 through 45. **Test definitions:** 2. **Assert statements:** 1.

File SHA-256: `81004e9b2b103567d0818c3029ceeabb9a15b2c5cfa4b9bb4f169c9dca4c28a3`.

Literal Komorebi import targets: none. This syntax result does not rule out subprocess, helper-mediated or dynamic package use.

O-T060: `tests/test_block21_publication_gate.py`

Source: lines 1 through 80. **Test definitions:** 5. **Assert statements:** 18.

File SHA-256: `8dbb0e0d9f8053d82d30354ae945aab63770ff4d4875401f76973368324f4d93`.

Literal Komorebi import targets: `komorebi_math.compatibility.check_public_api`.

O-T061: `tests/test_block4_hardening.py`

Source: lines 1 through 139. **Test definitions:** 10. **Assert statements:** 14.

File SHA-256: `8ebb3b18eb3be47966669bf764687044b85c2e471a3ee94baf6b52996935151f`.

Literal Komorebi import targets: `komorebi_math.dynamics.HiddenMarkovFilter`, `komorebi_math.dynamics.MatrixSemigroup`, `komorebi_math.dynamics.PhaseTypeSurvival`, `komorebi_math.dynamics.approximate_likelihood_partition`, `komorebi_math.dynamics.coarsest_sufficient_partition`, `komorebi_math.dynamics.matrix_exponential`, `komorebi_math.exceptions.ValidationError`, `komorebi_math.signals.TruncatedPowerTerm`, `komorebi_math.signals.convolve_signal`, `komorebi_math.signals.packet_product`, `komorebi_math.signals.polyphase_analyze`, `komorebi_math.signals.polyphase_filter_kernels`, `komorebi_math.signals.recover_vector_jumps_subgaussian`.

O-T062: `tests/test_candidate_evidence.py`

Source: lines 1 through 196. **Test definitions:** 8. **Assert statements:** 8.

File SHA-256: `1cd7781bda64695337894384045 46a0669de4284e2400209acadc758d5f35a83`.

Literal Komorebi import targets: none. This syntax result does not rule out subprocess, helper-mediated or dynamic package use.

O-T063: `tests/test_candidate_wheel_reproducibility.py`

Source: lines 1 through 103. **Test definitions:** 4. **Assert statements:** 10.

File SHA-256: `4d6ebff305d6992d60ed660546d739767077bdcd73e8eb00eb39c9bc2fdfe136`.

Literal Komorebi import targets: none. This syntax result does not rule out subprocess, helper-mediated or dynamic package use.

O-T064: `tests/test_certificate_finite_evidence.py`

Source: lines 1 through 55. **Test definitions:** 5. **Assert statements:** 12.

File SHA-256: `449fbfcd327cbd43a5f6d494f6be264c051fb13afa037aeab2e235ff8bc0917a`.

Literal Komorebi import targets: `komorebi_math.certificates.Certificate`, `komorebi_math.certificates.ComputationResult`, `komorebi_math.certificates.Exactness`, `komorebi_math.exceptions.ValidationError`, `komorebi_math.export.dumps_portable`, `komorebi_math.export.loads_portable`.

O-T065: `tests/test_certificates_registry.py`

Source: lines 1 through 89. **Test definitions:** 7. **Assert statements:** 22.

File SHA-256: `7d3c0decf3b9f6bc4a7781876ef78772b28dd9332070c8420fecf593f803e9bd`.

Literal Komorebi import targets: `komorebi_math`, `komorebi_math.analytic`, `komorebi_math.arithmetic`, `komorebi_math.certificates.Certificate`, `komorebi_math.certificates.ConditionalResult`, `komorebi_math.certificates.Exactness`, `komorebi_math.config.Settings`, `komorebi_math.config.configure`, `komorebi_math.config.get_settings`, `komorebi_math.config.settings_context`, `komorebi_math.exceptions.ValidationError`, `komorebi_math.quantum`, `komorebi_math.refinement`, `komorebi_math.registry.FeatureStatus`, `komorebi_math.sequences`.

O-T066: `tests/test_coordinates_exact.py`

Source: lines 1 through 42. **Test definitions:** 3. **Assert statements:** 13.

File SHA-256: `502b708f02b3eb97e9ea9d8e49f39efa92ea940c26828c3db00561a8c72fb382`.

Literal Komorebi import targets: `komorebi_math.coordinates.exact.determinant`, `komorebi_math.coordinates.exact.identity_matrix`, `komorebi_math.coordinates.exact.inverse_matrix`, `komorebi_math.coordinates.exact.matrix_multiply`, `komorebi_math.coordinates.exact.matrix_power`, `komorebi_math.coordinates.exact.matrix_rank`, `komorebi_math.coordinates.exact.matrix_vector_multiply`, `komorebi_math.coordinates.exact.nullspace`, `komorebi_math.coordinates.exact.solve_linear_system`.

O-T067: `tests/test_coordinates_general_affine_formal.py`

Source: lines 1 through 55. **Test definitions:** 3. **Assert statements:** 6.

File SHA-256: `e8082128ed806194155dba9adaeba0eb280d592a29138a008fcc23351d9fe582`.

Literal Komorebi import targets: `komorebi_math.coordinates.affine.BarycentricCoordinateSystem`, `komorebi_math.coordinates.affine.affine_basis_transform`, `komorebi_math.coordinates.affine.translation_transform`, `komorebi_math.coordinates.exact.identity_matrix`, `komorebi_math.coordinates.exact.matrix_multiply`, `komorebi_math.coordinates.formal.FormalCoordinateChange`, `komorebi_math.coordinates.formal.identity_formal_coordinate`.

O-T068: `tests/test_coordinates_general_basis.py`

Source: lines 1 through 62. **Test definitions:** 4. **Assert statements:** 17.

File SHA-256: `107c052ede542e2cdb6ac290392d3584aaf92057a9046c6a9c72a535e1f8f0d9`.

Literal Komorebi import targets: `komorebi_math.coordinates.basis.CoordinateAtlas`, `komorebi_math.coordinates.basis.CoordinateSystem`, `komorebi_math.coordinates.basis.basis_from_matrix`, `komorebi_math.coordinates.basis.direct_sum_basis`, `komorebi_math.coordinates.basis.standard_basis`, `komorebi_math.coordinates.basis.tensor_product_basis`, `komorebi_math.coordinates.transform.CoordinateTransform`.

O-T069: `tests/test_coordinates_general_calculus.py`

Source: lines 1 through 62. **Test definitions:** 3. **Assert statements:** 7.

File SHA-256: `cb4512c9ce0cf58c20d55db18c95a65b8d6a0bd9ecbe10f0a739ffdc21083f98`.

Literal Komorebi import targets: `komorebi_math.coordinates.calculus.affine_substitution_coordinate_operator`, `komorebi_math.coordinates.calculus.derivative_coordinate_operator`, `komorebi_math.coordinates.calculus.finite_difference_coordinate_operator`, `komorebi_math.coordinates.calculus.integral_coordinate_operator`, `komorebi_math.coordinates.calculus.multiplication_coordinate_operator`, `komorebi_math.coordinates.calculus.transport_polynomial_operator`, `komorebi_math.coordinates.classical.chebyshev_t_polynomial_family`, `komorebi_math.coordinates.classical.falling_factorial_polynomial_family`, `komorebi_math.coordinates.classical.legendre_polynomial_family`, `komorebi_math.coordinates.polynomials.monomial_polynomial_family`.

O-T070: `tests/test_coordinates_general_classical.py`

Source: lines 1 through 64. **Test definitions:** 4. **Assert statements:** 13.

File SHA-256: `b7ea751ad116b8a272e08082b13934ab221819c01d594b418f5988038ec9db29`.

Literal Komorebi import targets: `komorebi_math.coordinates.classical.affine_substitution_transform`, `komorebi_math.coordinates.classical.appell_polynomial_family`, `komorebi_math.coordinates.classical.bernoulli_polynomial`, `komorebi_math.coordinates.classical.bernstein_basis`, `komorebi_math.coordinates.classical.chebyshev_t_polynomial`, `komorebi_math.coordinates.classical.classical_polynomial_catalog`, `komorebi_math.coordinates.classical.hermite_probabilists_polynomial`, `komorebi_math.coordinates.classical.lagrange_interpolation_basis`, `komorebi_math.coordinates.classical.laguerre_polynomial`, `komorebi_math.coordinates.classical.legendre_polynomial`, `komorebi_math.coordinates.classical.newton_interpolation_basis`, `komorebi_math.coordinates.connection_transform`, `komorebi_math.coordinates.exact.identity_matrix`, `komorebi_math.coordinates.exact.matrix_multiply`, `komorebi_math.coordinates.monomial_polynomial_family`, `komorebi_math.coordinates.polynomials.polynomial_derivative`, `komorebi_math.coordinates.polynomials.polynomial_evaluate`.

O-T071: `tests/test_coordinates_general_combinatorial.py`

Source: lines 1 through 52. **Test definitions:** 3. **Assert statements:** 8.

File SHA-256: `655547b3e982028889b9672403db46546a55228475b9c3c657beb596faefa5b0`.

Literal Komorebi import targets: `komorebi_math.coordinates.combinatorial.binomial_transform`, `komorebi_math.coordinates.combinatorial.divisor_zeta_transform`, `komorebi_math.coordinates.combinatorial.falling_factorial_to_monomial_transform`, `komorebi_math.coordinates.combinatorial.forward_difference_transform`, `komorebi_math.coordinates.combinatorial.monomial_to_falling_factorial_transform`, `komorebi_math.coordinates.combinatorial.monomial_to_rising_factorial_transform`, `komorebi_math.coordinates.combinatorial.newton_evaluation_transform`, `komorebi_math.coordinates.combinatorial.rising_factorial_to_monomial_transform`, `komorebi_math.coordinates.combinatorial.rising_to_falling_factorial_transform`, `komorebi_math.coordinates.combinatorial.subset_zeta_transform`, `komorebi_math.coordinates.exact.identity_matrix`, `komorebi_math.coordinates.exact.matrix_multiply`.

O-T072: `tests/test_coordinates_general_families_state.py`

Source: lines 1 through 73. **Test definitions:** 4. **Assert statements:** 16.

File SHA-256: `6316c04f9167b8763b221559c26d19ea2ae4e07175b13d4ff1d8bec99e3d0db6`.

Literal Komorebi import targets: `komorebi_math.coordinates.families.fibonacci_recurrence_model`, `komorebi_math.coordinates.families.jacobsthal_recurrence_model`, `komorebi_math.coordinates.families.lucas_recurrence_model`, `komorebi_math.coordinates.families.padovan_recurrence_model`, `komorebi_math.coordinates.families.pell_recurrence_model`, `komorebi_math.coordinates.families.recurrence_model_catalog`, `komorebi_math.coordinates.families.sequence_coordinates_between_models`, `komorebi_math.coordinates.families.tribonacci_recurrence_model`, `komorebi_math.coordinates.state_space.certify_state_coordinate_change`, `komorebi_math.coordinates.state_space.controllability_coordinate_transform`, `komorebi_math.coordinates.state_space.observability_coordinate_transform`, `komorebi_math.coordinates.state_space.siso_state_space`.

O-T073: `tests/test_coordinates_general_numeric_packet.py`

Source: lines 1 through 57. **Test definitions:** 3. **Assert statements:** 8.

File SHA-256: `722a75b84c0c9bb1f5eb7c5088f217aa2ee0f89d37c1c1cbd960983b2bdfe46c`.

Literal Komorebi import targets: `komorebi_math.coordinates.numeric.discrete_cosine_transform`, `komorebi_math.coordinates.numeric.discrete_fourier_transform`, `komorebi_math.coordinates.numeric.haar_wavelet_transform`, `komorebi_math.coordinates.numeric.modal_coordinate_system`, `komorebi_math.coordinates.numeric.walsh_hadamard_transform`, `komorebi_math.coordinates.packet.chinese_remainder_transform`, `komorebi_math.coordinates.packet.digit_reversal_transform`, `komorebi_math.coordinates.packet.residue_class_transform`, `komorebi_math.coordinates.packet.tensor_axis_permutation_transform`.

O-T074: `tests/test_coordinates_general_persistence.py`

Source: lines 1 through 103. **Test definitions:** 3. **Assert statements:** 21.

File SHA-256: `13a9808e7d1595831686918e5ddd0372927ac44a2818232964356492853b0efc`.

Literal Komorebi import targets: `komorebi_math.coordinates.AffineCoordinateTransform`, `komorebi_math.coordinates.BarycentricCoordinateSystem`, `komorebi_math.coordinates.CoordinateAtlas`, `komorebi_math.coordinates.CoordinateSystem`, `komorebi_math.coordinates.CoordinateTransform`, `komorebi_math.coordinates.FiniteBasis`, `komorebi_math.coordinates.FiniteFieldCoordinateTransform`, `komorebi_math.coordinates.FormalCoordinateChange`, `komorebi_math.coordinates.NumericCoordinateTransform`, `komorebi_math.coordinates.PolynomialCoordinateOperator`, `komorebi_math.coordinates.RecurrenceModel`, `komorebi_math.coordinates.StateSpaceModel`, `komorebi_math.coordinates.discrete_fourier_transform`, `komorebi_math.coordinates.load_coordinate_object`, `komorebi_math.coordinates.modal_coordinate_system`, `komorebi_math.coordinates.save_coordinate_object`.

O-T075: `tests/test_coordinates_general_tensor_modular.py`

Source: lines 1 through 53. **Test definitions:** 2. **Assert statements:** 7.

File SHA-256: `d21043f8ca75c7089667be7a7bb8a0921e1135db2609549c104bfc5268723042`.

Literal Komorebi import targets: `komorebi_math.coordinates.combinatorial.binomial_transform`, `komorebi_math.coordinates.exact.identity_matrix`, `komorebi_math.coordinates.exact.matrix_multiply`, `komorebi_math.coordinates.modular.reduce_exact_transform_mod_prime`, `komorebi_math.coordinates.tensor.coordinate_pairing`, `komorebi_math.coordinates.tensor.pairing_invariant`, `komorebi_math.coordinates.tensor.tensor_coordinate_transform`, `komorebi_math.coordinates.tensor.transport_bilinear_form`, `komorebi_math.coordinates.tensor.transport_operator`, `komorebi_math.coordinates.tensor.transport_tensor_components`, `komorebi_math.coordinates.transform.CoordinateTransform`.

O-T076: `tests/test_coordinates_normalized_persistence.py`

Source: lines 1 through 50. **Test definitions:** 2. **Assert statements:** 4.

File SHA-256: `ae95de1b063ffc2c4ed61dfbe5fec182bc6efdf3740ff242fa352bfb4b3402c5`.

Literal Komorebi import targets: `komorebi_math.coordinates.convert_normalized_coefficients`, `komorebi_math.coordinates.euler_fibonacci_coordinates`, `komorebi_math.coordinates.exponential_basis`, `komorebi_math.coordinates.fibonacci_factorial_basis`, `komorebi_math.coordinates.load_coordinate_record`, `komorebi_math.coordinates.load_transform`, `komorebi_math.coordinates.normalization_change_transform`, `komorebi_math.coordinates.save_coordinate_record`, `komorebi_math.coordinates.save_transform`, `komorebi_math.coordinates.signed_secant_euler_numbers`.

O-T077: `tests/test_coordinates_operators.py`

Source: lines 1 through 29. **Test definitions:** 2. **Assert statements:** 4.

File SHA-256: `110d96e5b417b61393e6ef843e75a1bb6d1c10a73e86f239700bd7d631fff72b`.

Literal Komorebi import targets: `komorebi_math.coordinates.ShiftPolynomial`, `komorebi_math.coordinates.bezout_defect_identity`, `komorebi_math.coordinates.polynomial_extended_gcd`, `komorebi_math.coordinates.polynomials.polynomial_add`, `komorebi_math.coordinates.polynomials.polynomial_multiply`.

O-T078: `tests/test_coordinates_polynomials.py`

Source: lines 1 through 89. **Test definitions:** 5. **Assert statements:** 11.

File SHA-256: `cdcea421993b7b7dd34c3edefc509a2e2744d8a2444aaeb32897a697458509e6`.

Literal Komorebi import targets: `komorebi_math.coordinates.connection_transform`, `komorebi_math.coordinates.derivative_operator_in_family`, `komorebi_math.coordinates.euler_polynomial`, `komorebi_math.coordinates.euler_to_fibonacci_coefficient`, `komorebi_math.coordinates.euler_to_fibonacci_transform`, `komorebi_math.coordinates.exact.matrix_multiply`, `komorebi_math.coordinates.fibonacci_companion_power`, `komorebi_math.coordinates.fibonacci_polynomial`, `komorebi_math.coordinates.fibonacci_polynomial_family`, `komorebi_math.coordinates.monomial_polynomial_family`, `komorebi_math.coordinates.monomial_to_fibonacci_coefficient`, `komorebi_math.coordinates.operator_change_of_coordinates`, `komorebi_math.coordinates.polynomial_evaluate`, `komorebi_math.coordinates.polynomials.euler_polynomial_family`, `komorebi_math.coordinates.secant_euler_from_fibonacci_coordinates`, `komorebi_math.coordinates.signed_secant_euler_numbers`.

O-T079: `tests/test_coordinates_realization.py`

Source: lines 1 through 58. **Test definitions:** 4. **Assert statements:** 10.

File SHA-256: `9952476609d433c9e5b867d3d641039ecaffa6b4471710b9204c1ddd80e5cd25`.

Literal Komorebi import targets: `komorebi_math.coordinates.LinearRealization`, `komorebi_math.coordinates.companion_matrix`, `komorebi_math.coordinates.cyclic_krylov_matrix`, `komorebi_math.coordinates.euler_fibonacci_coordinates`, `komorebi_math.coordinates.exact.inverse_matrix`, `komorebi_math.coordinates.exact.matrix_multiply`, `komorebi_math.coordinates.exact.matrix_vector_multiply`, `komorebi_math.coordinates.intertwiner_space`, `komorebi_math.coordinates.observability_matrix`, `komorebi_math.coordinates.realization_defect`, `komorebi_math.coordinates.recover_initial_state`, `komorebi_math.coordinates.signed_secant_euler_numbers`, `komorebi_math.coordinates.similarity_from_cyclic_vectors`.

O-T080: `tests/test_coordinates_recurrence.py`

Source: lines 1 through 73. **Test definitions:** 6. **Assert statements:** 15.

File SHA-256: `699dc6d010d2af712f7932d7146ca28417efb32051eec0ccbf21c48459afe030`.

Literal Komorebi import targets: `komorebi_math.coordinates.analyze_recurrence_coordinates`, `komorebi_math.coordinates.companion_matrix`, `komorebi_math.coordinates.coordinate_change_between_recurrences`, `komorebi_math.coordinates.defect_green_weights`, `komorebi_math.coordinates.euler_fibonacci_coordinates`, `komorebi_math.coordinates.exact.identity_matrix`, `komorebi_math.coordinates.exact.matrix_multiply`, `komorebi_math.coordinates.fibonacci_numbers`, `komorebi_math.coordinates.recurrence_analysis_transform`, `komorebi_math.coordinates.recurrence_synthesis_transform`, `komorebi_math.coordinates.signed_secant_euler_numbers`, `komorebi_math.coordinates.variation_of_constants_prefix`.

O-T081: `tests/test_deep_analysis_synthesis_soundness.py`

Source: lines 1 through 55. **Test definitions:** 6. **Assert statements:** 8.

File SHA-256: `734e328f3ef7c73023fb31aab3107fa1f01dee2863998d3dfa75b760ace93b5f`.

Literal Komorebi import targets: `komorebi_math.coordinates.AnalysisSynthesisBlocks`, `komorebi_math.coordinates.AnalysisSynthesisSystem`, `komorebi_math.exceptions.ValidationError`.

O-T082: `tests/test_deep_exact_core_soundness.py`

Source: lines 1 through 130. **Test definitions:** 13. **Assert statements:** 16.

File SHA-256: `7b8530a2f8b24c1dc3f35df31f80e95d2e18b7406c495d5e3f81c47ec8ab9ed4`.

Literal Komorebi import targets: `komorebi_math.core.seeds.semisimple_seed_rank`, `komorebi_math.exceptions.ValidationError`, `komorebi_math.refinement.LatticePacketData`, `komorebi_math.refinement.SmithCongruenceProblem`, `komorebi_math.refinement.SmithDecomposition`, `komorebi_math.refinement.refinement_weight_monotonicity`.

O-T083: `tests/test_deep_route_packet_exactness.py`

Source: lines 1 through 45. **Test definitions:** 3. **Assert statements:** 6.

File SHA-256: `a12e816b3e213250d0bb94fd8333bef1151d400a301f27aacde3ad41b98b6501`.

Literal Komorebi import targets: `komorebi_math.signals.route_packets.PolyphaseRoutePacket`.

O-T084: `tests/test_deep_signal_spectral_tf.py`

Source: lines 1 through 140. **Test definitions:** 16. **Assert statements:** 22.

File SHA-256: `74519cdc8fd950d8cb1e4a88e7109adea44d20ed1882327402a9e7554b930bbd`.

Literal Komorebi import targets: `komorebi_math.exceptions.ValidationError`, `komorebi_math.signals.approximation.estimate_positive_packet_1d`, `komorebi_math.signals.approximation.fft_lowpass_1d`, `komorebi_math.signals.approximation.otsu_threshold`, `komorebi_math.signals.canonical_dual_window`, `komorebi_math.signals.cross_spectrum`, `komorebi_math.signals.get_window`, `komorebi_math.signals.overlap_add_certificate`, `komorebi_math.signals.parseval_certificate`, `komorebi_math.signals.periodogram`, `komorebi_math.signals.spectrogram`, `komorebi_math.signals.stft`.

O-T085: `tests/test_deep_signal_states_and_frames.py`

Source: lines 1 through 159. **Test definitions:** 18. **Assert statements:** 16.

File SHA-256: `a65ba6029cc9868ac0d63ee17d44434684216b377eeb7ee8f28641e806bdcb05`.

Literal Komorebi import targets: `komorebi_math.exceptions.DimensionError`, `komorebi_math.exceptions.ValidationError`, `komorebi_math.signals.denoise.haar_decompose`, `komorebi_math.signals.edge_recovery.recover_jumps`, `komorebi_math.signals.edge_recovery.recover_jumps_subgaussian`, `komorebi_math.signals.filterbanks.FiniteFilterBank`, `komorebi_math.signals.filterbanks.HaarFilterBank`, `komorebi_math.signals.filterbanks.PeriodicFilterBank`, `komorebi_math.signals.filtering.StreamingFIR`, `komorebi_math.signals.filtering.fir_filter`, `komorebi_math.signals.recovery.detect_piecewise_polynomial_breaks`, `komorebi_math.signals.recovery.recover_separated_jumps`, `komorebi_math.signals.recursive.LinearFilter`, `komorebi_math.signals.recursive.StreamingLinearFilter`, `komorebi_math.signals.wavelets.HaarPacketTree`.

O-T086: `tests/test_deformation_complex_support_companions.py`

Source: lines 1 through 111. **Test definitions:** 8. **Assert statements:** 28.

File SHA-256: `497426375bf7c0d69010665aef9b2ae66b10db6dd40af08d9d7e820dca96b3f2`.

Literal Komorebi import targets: `komorebi_math.deformation.CompatibleCarrier`, `komorebi_math.deformation.FiniteCochainComplex`, `komorebi_math.deformation.FrobeniusSpine`, `komorebi_math.deformation.PresentedModule`, `komorebi_math.deformation.finite_frobenius_separator`, `komorebi_math.deformation.nested_subspace_occupancy`, `komorebi_math.exceptions.ValidationError`.

O-T087: `tests/test_deformation_forests_visibility_counting.py`

Source: lines 1 through 69. **Test definitions:** 5. **Assert statements:** 16.

File SHA-256: `5a48a6aa6eea89750d05bca66c4446dfa9c21861bc460f67f83946ddb2ee9f2a`.

Literal Komorebi import targets: `komorebi_math.deformation.FiniteLiftForest`, `komorebi_math.deformation.VisibilityTree`, `komorebi_math.deformation.nilpotent_chart_count`, `komorebi_math.deformation.nilpotent_chart_series`, `komorebi_math.deformation.normal_crossing_count`, `komorebi_math.deformation.normal_crossing_series`, `komorebi_math.deformation.residue_forest`, `komorebi_math.deformation.smooth_chart_count`, `komorebi_math.deformation.smooth_chart_series`, `komorebi_math.deformation.weighted_chart_count`, `komorebi_math.exceptions.ValidationError`.

O-T088: `tests/test_dirac_modes.py`

Source: lines 1 through 109. **Test definitions:** 9. **Assert statements:** 19.

File SHA-256: `47c3d39c2f2fd2d50f36b629a7ff0b503c77f49a1ef14080c48de3260f382d65`.

Literal Komorebi import targets: `komorebi_math.exceptions.DimensionError`, `komorebi_math.exceptions.ValidationError`, `komorebi_math.quantum.dirac.central_mode_momentum`, `komorebi_math.quantum.dirac.dirac_matrices`, `komorebi_math.quantum.dirac.dirac_propagator`, `komorebi_math.quantum.dirac.dirac_symbol`.

O-T089: `tests/test_dynamics_bayes_martingales.py`

Source: lines 1 through 78. **Test definitions:** 6. **Assert statements:** 6.

File SHA-256: `760190081ffd23f0a8913634a065b2c6619bb18d4882cf35221360f5ce6d02e5`.

Literal Komorebi import targets: `komorebi_math.dynamics.BayesianFlow`, `komorebi_math.dynamics.BayesianPartition`, `komorebi_math.dynamics.BayesianUpdate`, `komorebi_math.dynamics.ConditionalExpectationTower`, `komorebi_math.dynamics.HiddenMarkovFilter`, `komorebi_math.dynamics.coarsest_sufficient_partition`, `komorebi_math.dynamics.relative_entropy_decomposition`.

O-T090: `tests/test_dynamics_feshbach.py`

Source: lines 1 through 66. **Test definitions:** 4. **Assert statements:** 5.

File SHA-256: `e94c8b345c6d798043064748487 76f136202817e0c5cc145c6b3ea1c73bf556b`.

Literal Komorebi import targets: `komorebi_math.dynamics.FeshbachReduction`, `komorebi_math.dynamics.MemoryKernel`, `komorebi_math.dynamics.RationalMemoryRealization`, `komorebi_math.dynamics.memory_kernel_error_bound`, `komorebi_math.dynamics.volterra_trajectory_bound`.

O-T091: `tests/test_dynamics_fractional_green_feynman_survival.py`

Source: lines 1 through 105. **Test definitions:** 8. **Assert statements:** 17.

File SHA-256: `79fa6bee072420c594d394b6d0cd2c17bcc147f5d2bd823a516398fe2c1f12e0`.

Literal Komorebi import targets: `komorebi_math.dynamics.DirichletGreen1D`, `komorebi_math.dynamics.FiniteFeynmanKac`, `komorebi_math.dynamics.PhaseTypeSurvival`, `komorebi_math.dynamics.fractional_defect`, `komorebi_math.dynamics.fractional_naturality_certificate`, `komorebi_math.dynamics.fractional_power`, `komorebi_math.dynamics.fractional_semigroup`.

O-T092: `tests/test_dynamics_semigroups_markov.py`

Source: lines 1 through 74. **Test definitions:** 6. **Assert statements:** 7.

File SHA-256: `63498936d1d7935700fd7ab248bed0d4beaa5c579efa669d8c39a59660dc4a77`.

Literal Komorebi import targets: `komorebi_math.PacketSystem`, `komorebi_math.dynamics.ContinuousTimeMarkovChain`, `komorebi_math.dynamics.FiniteSemigroupTower`, `komorebi_math.dynamics.MarkovLumping`, `komorebi_math.dynamics.MatrixSemigroup`, `komorebi_math.dynamics.PacketSemigroup`, `komorebi_math.dynamics.SemigroupRefinement`, `komorebi_math.dynamics.matrix_exponential`.

O-T093: `tests/test_enum_signature_compatibility.py`

Source: lines 1 through 77. **Test definitions:** 4. **Assert statements:** 6.

File SHA-256: `307c038b32d8822efb9e781f63fee16e216fdd3fb0a22e12bd3c0b635a0bc3a6`.

Literal Komorebi import targets: `komorebi_math.certificates.Exactness`, `komorebi_math.compatibility`.

O-T094: `tests/test_export_cli.py`

Source: lines 1 through 77. **Test definitions:** 5. **Assert statements:** 14.

File SHA-256: `cf6fb8067dd4043e749bbeeecf96e1c8839edd4886ea96adc37539cfb282f88e`.

Literal Komorebi import targets: `komorebi_math.cli.main.main`, `komorebi_math.export.available_latex_engines`.

O-T095: `tests/test_export_files.py`

Source: lines 1 through 174. **Test definitions:** 9. **Assert statements:** 22.

File SHA-256: `10fb2f85f901d55ac9eb5d608da646860e9311cf8c2f20e08035566f22a7400c`.

Literal Komorebi import targets: `komorebi_math.exceptions.BackendUnavailableError`, `komorebi_math.exceptions.ValidationError`, `komorebi_math.export.DocumentMetadata`, `komorebi_math.export.ExportOptions`, `komorebi_math.export.MathDocument`, `komorebi_math.export.available_latex_engines`, `komorebi_math.export.compile_latex`, `komorebi_math.export.export_bundle`, `komorebi_math.export.export_data`, `komorebi_math.export.export_many`.

O-T096: `tests/test_export_renderers.py`

Source: lines 1 through 74. **Test definitions:** 5. **Assert statements:** 13.

File SHA-256: `017d8e014684a5599558ed750836f5dfb11c506c5ac86d5654bdd3fa257863b6`.

Literal Komorebi import targets: `komorebi_math.export.DocumentMetadata`, `komorebi_math.export.HtmlRenderOptions`, `komorebi_math.export.LatexRenderOptions`, `komorebi_math.export.MathDocument`, `komorebi_math.export.latex_escape`, `komorebi_math.export.render_html`, `komorebi_math.export.render_latex`, `komorebi_math.export.render_markdown`, `komorebi_math.export.render_text`.

O-T097: `tests/test_export_serialization.py`

Source: lines 1 through 76. **Test definitions:** 5. **Assert statements:** 13.

File SHA-256: `5ebb9ff64b8bab5d242da75ea58d677eaf27474a8c53d2c6b4397e47d4135900`.

Literal Komorebi import targets: `komorebi_math.Certificate`, `komorebi_math.ComputationResult`, `komorebi_math.Exactness`, `komorebi_math.exceptions.ValidationError`, `komorebi_math.export.register_export_adapter`, `komorebi_math.export.table_rows`, `komorebi_math.export.to_serializable`.

O-T098: `tests/test_filterbank_pseudoinverse_compatibility.py`

Source: lines 1 through 47. **Test definitions:** 4. **Assert statements:** 0.

File SHA-256: `eafb10e6d33fca9bbaa4be591bfa6c59c96634af25f5df3d3f4d08ea04c34c3c`.

Literal Komorebi import targets: `komorebi_math.signals.FiniteFilterBank`, `komorebi_math.signals.PeriodicFilterBank`.

O-T099: `tests/test_final_cli.py`

Source: lines 1 through 68. **Test definitions:** 5. **Assert statements:** 27.

File SHA-256: `e98efb5b837b346db6564be654e145998b36959a07d6d660a902b22118aacbab`.

Literal Komorebi import targets: `komorebi_math.__version__`, `komorebi_math.cli.main.build_parser`, `komorebi_math.cli.main.main`, `komorebi_math.qualification`, `komorebi_math.release.check_validation_evidence`.

O-T100: `tests/test_final_distribution_hygiene.py`

Source: lines 1 through 153. **Test definitions:** 9. **Assert statements:** 7.

File SHA-256: `2ae3a1df555817d0141b1e18898371b9aed82101b8fdd3cf5d76b73f5d009029`.

Literal Komorebi import targets: none. This syntax result does not rule out subprocess, helper-mediated or dynamic package use.

O-T101: `tests/test_finite_join_semilattice.py`

Source: lines 1 through 136. **Test definitions:** 11. **Assert statements:** 17.

File SHA-256: `707f247bbd1716f024ece766b58c12d63a908caf663c3ac15db2cdd666ea9617`.

Literal Komorebi import targets: `komorebi_math.certificates.Exactness`, `komorebi_math.core.semilattice.FiniteJoinSemilattice`, `komorebi_math.core.semilattice.SemilatticeQuotient`, `komorebi_math.exceptions.DimensionError`, `komorebi_math.exceptions.ValidationError`.

O-T102: `tests/test_fourier_recovery.py`

Source: lines 1 through 173. **Test definitions:** 17. **Assert statements:** 14.

File SHA-256: `3a1efffdbee7943b75d75cfc047951c3f1ff7624fed07b529fc0b8f8e756e2bc`.

Literal Komorebi import targets: `komorebi_math.exceptions.AssumptionError`, `komorebi_math.exceptions.DimensionError`, `komorebi_math.exceptions.ValidationError`, `komorebi_math.signals.fourier_recovery.FourierJumpRecovery`, `komorebi_math.signals.fourier_recovery.PeriodicJumpPacket`, `komorebi_math.signals.fourier_recovery.edge_moments_from_fourier`, `komorebi_math.signals.fourier_recovery.hankel_moment_matrix`, `komorebi_math.signals.fourier_recovery.recover_fourier_jumps`.

O-T103: `tests/test_generating_block13.py`

Source: lines 1 through 265. **Test definitions:** 20. **Assert statements:** 33.

File SHA-256: `a0a6d7c2deaa1444773bcb9ae6ba5fe37fef681ec4ffb6b38aa1070f49e1fee7`.

Literal Komorebi import targets: `komorebi_math.exceptions.BackendUnavailableError`, `komorebi_math.exceptions.ValidationError`, `komorebi_math.export.dumps_portable`, `komorebi_math.export.loads_portable`, `komorebi_math.generating.FiniteMaskCarryPlan`, `komorebi_math.generating.FormalSeries`, `komorebi_math.generating.GeneratingBudget`, `komorebi_math.generating.analytic_mahler_product`, `komorebi_math.generating.certify_rational_mask_algebraic_relation`, `komorebi_math.generating.compile_cartier_linear_representation`, `komorebi_math.generating.compile_digit_cauchy_product`, `komorebi_math.generating.compile_finite_mask_carry_plan`, `komorebi_math.generating.euler_product_collapse`, `komorebi_math.generating.exponential_basis`, `komorebi_math.generating.factorial_power_bell_function`, `komorebi_math.generating.factorial_power_convolution_layers`, `komorebi_math.generating.factorial_power_support_count`, `komorebi_math.generating.factorial_power_support_indices`, `komorebi_math.generating.fir_taps_to_series`, `komorebi_math.generating.first_frobenius_lift`, `komorebi_math.generating.freeze_custom_function`, `komorebi_math.generating.frobenius_defect_mask`, `komorebi_math.generating.kummer_layer_distribution`, `komorebi_math.generating.mahler_mask_inverse`, `komorebi_math.generating.mahler_mask_product`, `komorebi_math.generating.one_borrow_count`, `komorebi_math.generating.ordinary_basis`, `komorebi_math.generating.padic_units.borrow_count`, `komorebi_math.generating.polynomial_mask_coboundary`, `komorebi_math.generating.series_to_fir_adapter`, `komorebi_math.generating.series_to_fir_taps`, `komorebi_math.generating.solve_polynomial_coboundary`, `komorebi_math.generating.zero_borrow_count`.

O-T104: `tests/test_generating_cartier_padic.py`

Source: lines 1 through 256. **Test definitions:** 19. **Assert statements:** 25.

File SHA-256: `bff8efd42d1223452fa90872fc9d0e463d9ac8bc2f9f73527156d4281606ebcd`.

Literal Komorebi import targets: `komorebi_math.generating.CarryLinearValuation`, `komorebi_math.generating.PrimePowerKernelCompiler`, `komorebi_math.generating.PrimePowerUnitCompiler`, `komorebi_math.generating.binomial_valuation`, `komorebi_math.generating.borrow_count`, `komorebi_math.generating.certify_cartier_carry_equivalence`, `komorebi_math.generating.classify_carry_linear_valuation`, `komorebi_math.generating.compile_cartier_linear_representation`, `komorebi_math.generating.compile_rational_mask_automaton`, `komorebi_math.generating.cyclotomic_coboundary`, `komorebi_math.generating.digit_factorized_normalized_kernel`, `komorebi_math.generating.digit_unit`, `komorebi_math.generating.finite_mask_carry_matrices`, `komorebi_math.generating.gaussian_binomial`, `komorebi_math.generating.q_binomial_support`, `komorebi_math.generating.q_binomial_valuation_decomposition`, `komorebi_math.generating.q_binomial_valuation_direct`, `komorebi_math.generating.rational_mask_coefficients_mod_prime`, `komorebi_math.generating.verify_cyclotomic_coboundary`, `komorebi_math.generating.verify_cyclotomic_exponent_relation`.

O-T105: `tests/test_generating_domains.py`

Source: lines 1 through 223. **Test definitions:** 15. **Assert statements:** 26.

File SHA-256: `12cf3f950c5ec8e5c808bed040828472c23d081132d085202d07355e5e2efc42`.

Literal Komorebi import targets: `komorebi_math.exceptions.ResourceLimitError`, `komorebi_math.exceptions.ValidationError`, `komorebi_math.export.dumps_portable`, `komorebi_math.export.loads_portable`, `komorebi_math.generating.BellParityAutomaton`, `komorebi_math.generating.BellResidueAutomaton`, `komorebi_math.generating.FiniteNormalizedSeries`, `komorebi_math.generating.FormalSeries`, `komorebi_math.generating.GeneratingBudget`, `komorebi_math.generating.IntegerDomain`, `komorebi_math.generating.PrimeFieldDomain`, `komorebi_math.generating.RationalDomain`, `komorebi_math.generating.ResidueRingDomain`, `komorebi_math.generating.TruncatedPadicDomain`, `komorebi_math.generating.WeightedMonomialBasis`, `komorebi_math.generating.basis_from_descriptor`, `komorebi_math.generating.basis_from_values`, `komorebi_math.generating.bell_numbers`, `komorebi_math.generating.certify_pointed_subindex_identity`, `komorebi_math.generating.compile_cartier_linear_representation`, `komorebi_math.generating.exponential_basis`, `komorebi_math.generating.factorial_power_basis`, `komorebi_math.generating.generalized_bell_digit_reduction`, `komorebi_math.generating.generalized_bell_numbers`, `komorebi_math.generating.ordinary_basis`, `komorebi_math.generating.weighted_bell_digit_reduction`.

O-T106: `tests/test_generating_foundation.py`

Source: lines 1 through 280. **Test definitions:** 26. **Assert statements:** 48.

File SHA-256: `fd7fca313208302af2c54b89bcb700ddee1c1566aed17cf14fac7684a4849a6c`.

Literal Komorebi import targets: `komorebi_math.generating.FormalSeries`, `komorebi_math.generating.FunctionRegistry`, `komorebi_math.generating.algebraic_function`, `komorebi_math.generating.apery_numbers`, `komorebi_math.generating.audit_apery_supercongruence`, `komorebi_math.generating.audit_factorial_power_digit_algebra`, `komorebi_math.generating.audit_factorial_power_packet_convolution`, `komorebi_math.generating.audit_generalized_bell_digit_sum`, `komorebi_math.generating.audit_generalized_lucas`, `komorebi_math.generating.audit_partition_lane`, `komorebi_math.generating.audit_q_lucas`, `komorebi_math.generating.audit_section_cycle`, `komorebi_math.generating.audit_touchard`, `komorebi_math.generating.audit_weighted_generalized_bell_digit_sum`, `komorebi_math.generating.bell_numbers`, `komorebi_math.generating.catalan_numbers`, `komorebi_math.generating.central_binomial_numbers`, `komorebi_math.generating.characteristic_closed_form`, `komorebi_math.generating.dirichlet_convolution`, `komorebi_math.generating.euler_product_function`, `komorebi_math.generating.exponential_basis`, `komorebi_math.generating.factorial_power_basis`, `komorebi_math.generating.factorial_power_bell_function`, `komorebi_math.generating.fibonacci_factorial_basis`, `komorebi_math.generating.fibonacci_numbers`, `komorebi_math.generating.generalized_bell_numbers`, `komorebi_math.generating.generalized_bell_polynomials`, `komorebi_math.generating.guess_algebraic_gf`, `komorebi_math.generating.guess_hypergeometric_ratio`, `komorebi_math.generating.guess_linear_recurrence`, `komorebi_math.generating.guess_p_recursive`, `komorebi_math.generating.ordinary_basis`, `komorebi_math.generating.partition_numbers`, `komorebi_math.generating.pochhammer_basis`, `komorebi_math.generating.q_factorial_basis`, `komorebi_math.generating.rational_gf_from_recurrence`, `komorebi_math.generating.weighted_factorial_power_bell_function`.

O-T107: `tests/test_grammar_realization.py`

Source: lines 1 through 69. **Test definitions:** 7. **Assert statements:** 9.

File SHA-256: `fff10e756e2a9275ac098b289f9f60fda3af441f57c529d36381805e2d3a34d0`.

Literal Komorebi import targets: `komorebi_math.OperatorGrammar`, `komorebi_math.OperatorRealization`, `komorebi_math.exceptions.RelationError`, `komorebi_math.exceptions.ValidationError`.

O-T108: `tests/test_graphs.py`

Source: lines 1 through 71. **Test definitions:** 7. **Assert statements:** 11.

File SHA-256: `4de3f9a00ec0dab693370a3437ab2ff7fb45b40c2a78d79233c5b8f348ae8132`.

Literal Komorebi import targets: `komorebi_math.SupportPreorder`, `komorebi_math.TransferGraph`, `komorebi_math.exceptions.ValidationError`.

O-T109: `tests/test_hardware_qualification.py`

Source: lines 1 through 186. **Test definitions:** 10. **Assert statements:** 21.

File SHA-256: `1ec23b9414b79ef20445e1551c920964f652aacc767f6660edf1c5d2282fd47c`.

Literal Komorebi import targets: none. This syntax result does not rule out subprocess, helper-mediated or dynamic package use.

O-T110: `tests/test_holonomy.py`

Source: lines 1 through 93. **Test definitions:** 8. **Assert statements:** 10.

File SHA-256: `3d04261f4a9f7a2f5b694bf9ac9243dfc32bf72968035abd705fc16687d24859`.

Literal Komorebi import targets: `komorebi_math.exceptions.DimensionError`, `komorebi_math.nonlinear.TransportEdge`, `komorebi_math.nonlinear.TransportGraph`.

O-T111: `tests/test_langlands_finite_image.py`

Source: lines 1 through 99. **Test definitions:** 6. **Assert statements:** 15.

File SHA-256: `5d8de26c68712791b3858bbe9745ceac37fca606c7b821d505c8530617277583`.

Literal Komorebi import targets: `komorebi_math.langlands.finite_image.AutomorphicInductionPlan`, `komorebi_math.langlands.finite_image.CliffordStep`, `komorebi_math.langlands.finite_image.CliffordTower`, `komorebi_math.langlands.finite_image.InducedRepresentation`, `komorebi_math.langlands.finite_image.LinearCharacter`, `komorebi_math.langlands.finite_image.MonomialAction`, `komorebi_math.langlands.finite_image.VirtualCharacterResolution`, `komorebi_math.langlands.finite_image.VirtualCharacterTerm`, `komorebi_math.langlands.finite_image.cyclic_group`, `komorebi_math.langlands.finite_image.mackey_tensor_packets`, `komorebi_math.langlands.finite_image.symmetric_group`.

O-T112: `tests/test_langlands_local_fields_tame_gauss.py`

Source: lines 1 through 68. **Test definitions:** 7. **Assert statements:** 15.

File SHA-256: `ced2ea1e1b22e11b4ef18c5437f378ed576828c235cf73296bbdcb6c3797c0d9`.

Literal Komorebi import targets: `komorebi_math.exceptions.ValidationError`, `komorebi_math.langlands.local.FiniteField`, `komorebi_math.langlands.local.FrobeniusOrbit`, `komorebi_math.langlands.local.SpecialMonodromyBlock`, `komorebi_math.langlands.local.TameWeilPacket`, `komorebi_math.langlands.local.gauss_sum`, `komorebi_math.langlands.local.is_irreducible_polynomial`, `komorebi_math.langlands.local.regular_orbit_count`, `komorebi_math.langlands.local.tame_tensor_refinement`.

O-T113: `tests/test_langlands_local_heisenberg_symplectic.py`

Source: lines 1 through 51. **Test definitions:** 4. **Assert statements:** 6.

File SHA-256: `ae1aad13b85a725a86db06864ba6117f17bda6539f688cf404c35b8ea80f3b77`.

Literal Komorebi import targets: `komorebi_math.langlands.local.AlternatingForm`, `komorebi_math.langlands.local.HeisenbergModel`, `komorebi_math.langlands.local.ProjectiveIntertwinerEnvelope`, `komorebi_math.langlands.local.QuadraticRefinement`, `komorebi_math.langlands.local.SymplecticBridge`.

O-T114: `tests/test_langlands_ramification_global_completion.py`

Source: lines 1 through 92. **Test definitions:** 6. **Assert statements:** 15.

File SHA-256: `bab8b2737fa8d7fa957c9eaad80dde16df4580cfa2c189d4dd44de6fcd55e0d7`.

Literal Komorebi import targets: `komorebi_math.langlands.completion.AutomorphicCompletionReport`, `komorebi_math.langlands.completion.CompatiblePropagationReport`, `komorebi_math.langlands.completion.CompletionGate`, `komorebi_math.langlands.completion.GateStatus`, `komorebi_math.langlands.global_.FiniteHeckePackets`, `komorebi_math.langlands.global_.PatchingSupportReport`, `komorebi_math.langlands.global_.SupportComparison`, `komorebi_math.langlands.global_.dyadic_sign_diagnostic`, `komorebi_math.langlands.local.AlternatingForm`, `komorebi_math.langlands.local.QuadraticRefinement`, `komorebi_math.langlands.ramification.ProjectiveMultiplicityPacket`, `komorebi_math.langlands.ramification.RamificationCliffordTower`, `komorebi_math.langlands.ramification.RamificationStage`.

O-T115: `tests/test_lattice_smith.py`

Source: lines 1 through 107. **Test definitions:** 10. **Assert statements:** 21.

File SHA-256: `69c5acf9d60ce520515e12b33c2b6580abdaa6058506e92839895f2139a789d6`.

Literal Komorebi import targets: `komorebi_math.exceptions.ValidationError`, `komorebi_math.refinement.LatticePacketData`, `komorebi_math.refinement.SmithCongruenceProblem`, `komorebi_math.refinement.padic_valuation`, `komorebi_math.refinement.rational_padic_valuation`, `komorebi_math.refinement.refinement_weight_monotonicity`, `komorebi_math.refinement.smith_decomposition`.

O-T116: `tests/test_lifting.py`

Source: lines 1 through 103. **Test definitions:** 10. **Assert statements:** 18.

File SHA-256: `611bc7dfcf5917204b12bbf51df631386343d743ab441fd0e8e91fd087f899ab`.

Literal Komorebi import targets: `komorebi_math.exceptions.ValidationError`, `komorebi_math.nonlinear.LinearLiftProblem`, `komorebi_math.nonlinear.PolynomialLiftProblem`, `komorebi_math.nonlinear.build_lift_tree`, `komorebi_math.nonlinear.solve_mod_prime`.

O-T117: `tests/test_linear_defects.py`

Source: lines 1 through 50. **Test definitions:** 5. **Assert statements:** 4.

File SHA-256: `c4726faaf1c1106124ed6d41d3cef10eabc85b28a85bdcaa0c4a9632f1a6b48c`.

Literal Komorebi import targets: `komorebi_math.refinement.LinearRefinementDefect`, `komorebi_math.refinement.multilevel_defect`.

O-T118: `tests/test_manual_release_regressions.py`

Source: lines 1 through 171. **Test definitions:** 13. **Assert statements:** 24.

File SHA-256: `774aa276c90e44b9ab5ec16e9ab603be1e5d3ce83ad24e691f80cb37e48974d0`.

Literal Komorebi import targets: `komorebi_math.arithmetic.LinearRecurrence`, `komorebi_math.arithmetic.local_modes.simultaneous_unit_difference`, `komorebi_math.coordinates.formal.FormalCoordinateChange`, `komorebi_math.coordinates.operators.bezout_defect_identity`, `komorebi_math.coordinates.operators.polynomial_extended_gcd`, `komorebi_math.coordinates.packet.digit_reversal_order`, `komorebi_math.exceptions.ValidationError`, `komorebi_math.generating.closed_forms.guess_algebraic_gf`, `komorebi_math.generating.closed_forms.guess_hypergeometric_ratio`, `komorebi_math.generating.closed_forms.guess_linear_recurrence`, `komorebi_math.generating.closed_forms.guess_p_`

recursive, komorebi_math.generating.closed_forms.guess_quasipolynomial, komorebi_math.generating.digit_algebra.factorial_power_digit_kernel, komorebi_math.generating.functions.algebraic_function, komorebi_math.generating.normalization.digit_factorized_normalized_kernel, komorebi_math.generating.normalization.local_normalized_kernel, komorebi_math.generating.q_kummer.q_factorial, komorebi_math.sequences.Sequence, komorebi_math.sequences.compile_sequence, komorebi_math.sequences.detect_linear_recurrence.

O-T119: tests/test_memory_laplace_domains.py

Source: lines 1 through 70. **Test definitions:** 7. **Assert statements:** 8.

File SHA-256: 030a94c39066e055bf3a91f7299a7b89af073f080ce4dc92dd40c40126c4878d.

Literal Komorebi import targets: komorebi_math.dynamics.MemoryKernel, komorebi_math.exceptions.ValidationError.

O-T120: tests/test_minimum_dependency_capabilities.py

Source: lines 1 through 49. **Test definitions:** 2. **Assert statements:** 2.

File SHA-256: 95dd62ceff3056a1f79c125c3f08fa1c43828d108585ae6e9a2ddf952304e9b6.

Literal Komorebi import targets: komorebi_math.exceptions.BackendUnavailableError, komorebi_math.quantum.MovingPacketSystem, komorebi_math.quantum.moving_packet_leakage_bound, komorebi_math.refinement.SmithCongruenceProblem, komorebi_math.refinement.smith_decomposition.

O-T121: tests/test_morphisms.py

Source: lines 1 through 64. **Test definitions:** 4. **Assert statements:** 4.

File SHA-256: d05bca8a0d24dd3ae561c3fc8b0a780b4448d0377a6f9ed464ed961e80d1b1a9.

Literal Komorebi import targets: komorebi_math.OperatorMorphism, komorebi_math.PacketMorphism, komorebi_math.PacketSystem, komorebi_math.SeedMorphism, komorebi_math.SeedSystem, komorebi_math.backends.get_backend.

O-T122: tests/test_nonlinear_coordinates_defects.py

Source: lines 1 through 79. **Test definitions:** 6. **Assert statements:** 5.

File SHA-256: ade9bc91d6bd8d6218f687c54f1f52f97514d836ae474cdc88ef995c408a024e.

Literal Komorebi import targets: komorebi_math.PacketSystem, komorebi_math.exceptions.ValidationError, komorebi_math.nonlinear.NonlinearPacketMap, komorebi_math.nonlinear.NonlinearRefinementDefect.

O-T123: tests/test_nonlinear_correction_elimination.py

Source: lines 1 through 149. **Test definitions:** 5. **Assert statements:** 22.

File SHA-256: 384937220abf662c7be8ce43fa16b58ab426e924f454d11a93d03d93a2f5c468.

Literal Komorebi import targets: komorebi_math.Exactness, komorebi_math.nonlinear.correction.HilbertObstructionProjector, komorebi_math.nonlinear.correction.LyapunovSchmidtReduction, komorebi_math.nonlinear.correction.RightInverseCorrectionProblem, komorebi_math.nonlinear.derivatives.CallableDerivativeProvider, komorebi_math.nonlinear.derivatives.SympyDerivativeProvider, komorebi_math.nonlinear.elimination.NonlinearSchurElimination.

O-T124: tests/test_nonlinear_execution.py

Source: lines 1 through 133. **Test definitions:** 6. **Assert statements:** 28.

File SHA-256: c063dc8f1bed70580f17cc99bf274cee40aa1d206d259045272b62ca047293b9.

Literal Komorebi import targets: komorebi_math.PacketSystem, komorebi_math.exceptions.BackendUnavailableError, komorebi_math.exceptions.ValidationError, komorebi_math.nonlinear.derivatives.PolynomialDerivativeProvider, komorebi_math.nonlinear.execution.CuPyPacketJetExecutor, komorebi_math.nonlinear.execution.CuPyPolynomialDerivativeProvider, komorebi_math.nonlinear.execution.compile_packet_jet_plan, komorebi_math.nonlinear.execution.execute_packet_jet_plan, komorebi_math.nonlinear.execution.execute_packet_jets, komorebi_math.nonlinear.execution.nonlinear_cupy_available, komorebi_math.nonlinear.execution.packet_jet_benchmark_contract, komorebi_math.nonlinear.gpu.cupy_runtime_status, komorebi_math.nonlinear.jets.PacketJetAddress, komorebi_math.nonlinear.jets.PacketJetFamily, komorebi_math.nonlinear.jets.packet_jet_support.

O-T125: tests/test_nonlinear_filtered.py

Source: lines 1 through 150. **Test definitions:** 7. **Assert statements:** 37.

File SHA-256: 700ab5332e39b0dc21d23ff8495e3ecb90ac854e8a93d2d9399a026a4530b6be.

Literal Komorebi import targets: komorebi_math.exceptions.ValidationError, komorebi_math.nonlinear.filtered.FilteredCorrectionProblem, komorebi_math.nonlinear.filtered.FilteredVectorSpace, komorebi_math.nonlinear.filtered.FiniteFilteredCochainComplex, komorebi_math.nonlinear.filtered.conditional_projective_limit_completion, komorebi_math.nonlinear.filtered.verify_finite_filtered_branch.

O-T126: tests/test_nonlinear_graded.py

Source: lines 1 through 114. **Test definitions:** 5. **Assert statements:** 10.

File SHA-256: 290618d697e7f0c1f5c6e8b91d7507f24f912b3e2e4688fc41db25f9fff541ed.

Literal Komorebi import targets: komorebi_math.Exactness, komorebi_math.PacketSystem, komorebi_math.exceptions.ValidationError, komorebi_math.nonlinear.graded.FiniteAbelianGroup, komorebi_math.nonlinear.graded.GradedPacketOperation, komorebi_math.nonlinear.graded.divided_square_packets, komorebi_math.nonlinear.graded.higher_maurer_cartan_packets, komorebi_math.nonlinear.graded.maurer_cartan_packets.

O-T127: tests/test_nonlinear_jets_composition.py

Source: lines 1 through 225. **Test definitions:** 8. **Assert statements:** 22.

File SHA-256: 0fefc36b7202604ced03bdacf6d60d66ad585afc7e5fe9c915eae17139f8dc68.

Literal Komorebi import targets: komorebi_math.Exactness, komorebi_math.PacketSystem, komorebi_math.exceptions.ValidationError, komorebi_math.nonlinear.composition.ComposedDerivativeProvider, komorebi_math.nonlinear.composition.faa_di_bruno, komorebi_math.nonlinear.composition.labeled_composition_term_count, komorebi_math.nonlinear.composition.packet_faa_di_bruno, komorebi_math.nonlinear.composition.set_partitions, komorebi_math.nonlinear.composition.stirling_second, komorebi_math.nonlinear.derivatives.CallableDerivativeProvider, komorebi_math.nonlinear.derivatives.FiniteDifferenceDerivativeProvider, komorebi_math.nonlinear.derivatives.PolynomialDerivativeProvider, komorebi_math.nonlinear.derivatives.SympyDerivativeProvider, komorebi_math.nonlinear.jets.PacketJetAddress, komorebi_math.nonlinear.jets.PacketJetFamily, komorebi_math.nonlinear.jets.packet_jet_support, komorebi_math.nonlinear.jets.packet_taylor.

O-T128: tests/test_nonlinear_obstruction.py

Source: lines 1 through 63. **Test definitions:** 6. **Assert statements:** 11.

File SHA-256: de347c398420228421865a70869c167f4bda6b09a2fe87510f927540940d761a.

Literal Komorebi import targets: komorebi_math.nonlinear.LinearObstructionProblem, komorebi_math.nonlinear.ObstructionSquare.

O-T129: tests/test_nonlinear_snapshots_persistence.py

Source: lines 1 through 143. **Test definitions:** 5. **Assert statements:** 20.

File SHA-256: 6c8b134121fd08f17f98b5d7879f9ce3a2d7cc6ba764d912c67789d9e8b541d5.

Literal Komorebi import targets: komorebi_math.Exactness, komorebi_math.PacketSystem, komorebi_math.deformation.forests.LiftBranch, komorebi_math.exceptions.ValidationError, komorebi_math.export.persistence.dumps_portable, komorebi_math.export.persistence.load_portable, komorebi_math.export.persistence.loads_portable, komorebi_math.export.persistence.portable_digest, komorebi_math.export.persistence.save_portable, komorebi_math.export.serialization.from_serializable, komorebi_math.export.serialization.to_serializable, komorebi_math.nonlinear.derivatives.PolynomialDerivativeProvider, komorebi_math.nonlinear.filtered.FilteredVectorSpace, komorebi_math.nonlinear.filtered.FiniteFilteredCochainComplex, komorebi_math.nonlinear.jets.PacketJetFamily, komorebi_math.nonlinear.snapshots.PacketJetSnapshot, komorebi_math.nonlinear.snapshots.snapshot_packet_jet, komorebi_math.nonlinear.sparse.SparseSymmetricTensor, komorebi_math.nonlinear.trees.OrderedRootedTree.

O-T130: `tests/test_nonlinear_sparse_autodiff.py`

Source: lines 1 through 229. **Test definitions:** 10. **Assert statements:** 21.

File SHA-256: `76d2cc4583a7d2bec2d38f5b15d0ed3b86931ae5a4b568fd281b1483f927d93a`.

Literal Komorebi import targets: `komorebi_math.Exactness`, `komorebi_math.exceptions.DimensionError`, `komorebi_math.exceptions.ValidationError`, `komorebi_math.nonlinear.autodiff.AutomaticDifferentiationProvider`, `komorebi_math.nonlinear.autodiff.JaxDerivativeProvider`, `komorebi_math.nonlinear.autodiff.TorchDerivativeProvider`, `komorebi_math.nonlinear.autodiff.autodiff_provider`, `komorebi_math.nonlinear.autodiff.available_ad_frameworks`, `komorebi_math.nonlinear.derivatives.PolynomialDerivativeProvider`, `komorebi_math.nonlinear.sparse.SparsePolynomialDerivativeProvider`, `komorebi_math.nonlinear.sparse.SparseSymmetricTensor`, `komorebi_math.nonlinear.sparse.SparseTensorEntry`.

O-T131: `tests/test_nonlinear_transport.py`

Source: lines 1 through 65. **Test definitions:** 4. **Assert statements:** 5.

File SHA-256: `0960cd1ee2498c27e9eef22710516c32b843d189b295973b18897e2e7a092cd4`.

Literal Komorebi import targets: `komorebi_math.dynamics.matrix_exponential`, `komorebi_math.exceptions.ValidationError`, `komorebi_math.nonlinear.transport.KatoTransport`, `komorebi_math.nonlinear.transport.MovingPacketSystem`, `komorebi_math.nonlinear.transport.direct_packet_rotation`, `komorebi_math.quantum.KatoTransport`.

O-T132: `tests/test_nonlinear_trees.py`

Source: lines 1 through 112. **Test definitions:** 6. **Assert statements:** 25.

File SHA-256: `2e255467f8bc9e9276f81761c3a54c01060714db28eaeeabf54f8103179418b5`.

Literal Komorebi import targets: `komorebi_math.Exactness`, `komorebi_math.PacketSystem`, `komorebi_math.exceptions.ValidationError`, `komorebi_math.nonlinear.derivatives.SympyDerivativeProvider`, `komorebi_math.nonlinear.jets.PacketJetFamily`, `komorebi_math.nonlinear.jets.packet_jet_support`, `komorebi_math.nonlinear.trees.OrderedRootedTree`, `komorebi_math.nonlinear.trees.PolynomialFixedPointProblem`, `komorebi_math.nonlinear.trees.catalan_number`, `komorebi_math.nonlinear.trees.majorant_convergence`, `komorebi_math.nonlinear.trees.ordered_rooted_trees`, `komorebi_math.nonlinear.trees.ordered_tree_count`, `komorebi_math.nonlinear.trees.ordered_trees_with_leaves`, `komorebi_math.nonlinear.trees.packet_tree_reconstruction`, `komorebi_math.nonlinear.trees.quadratic_catalan_majorant`.

O-T133: `tests/test_numpy_signature_compatibility.py`

Source: lines 1 through 34. **Test definitions:** 2. **Assert statements:** 6.

File SHA-256: `b065dc159a53316b1800f57c74c2cf79977cb2133b161533867ea80309b90dfc`.

Literal Komorebi import targets: `komorebi_math.compatibility`, `komorebi_math.signals.design_biquad`.

O-T134: `tests/test_operators_transfer.py`

Source: lines 1 through 79. **Test definitions:** 7. **Assert statements:** 15.

File SHA-256: `64febe449b0da96ead3fc163ddb6c913d8b464830c4c6525a94be97f4a023e4a`.

Literal Komorebi import targets: `komorebi_math.PacketOperator`, `komorebi_math.PacketSystem`, `komorebi_math.block_from_paths`, `komorebi_math.path_contributions`.

O-T135: `tests/test_optional_backend_probe.py`

Source: lines 1 through 91. **Test definitions:** 4. **Assert statements:** 10.

File SHA-256: `99a7ab3e55872a3da9dd563401b5e661984fb344960f20c3a019d5f966fec59f`.

Literal Komorebi import targets: `komorebi_math.exceptions.BackendUnavailableError`, `komorebi_math.nonlinear.autodiff`, `komorebi_math.nonlinear.autodiff.TorchDerivativeProvider`.

O-T136: `tests/test_packets.py`

Source: lines 1 through 76. **Test definitions:** 8. **Assert statements:** 13.

File SHA-256: `01b5f2f427adc62e3b14f57ab6c1257c4cafdf9ce84aa2ba7622ba27bac5fd09`.

Literal Komorebi import targets: `komorebi_math.PacketSystem`, `komorebi_math.exceptions.ValidationError`.

O-T137: `tests/test_polyphase_certificate_extremes.py`

Source: lines 1 through 69. **Test definitions:** 6. **Assert statements:** 12.

File SHA-256: `c1fa88bfe80678dd8ab591ad56742f091d974d8ef90a69407b1ada652a383521`.

Literal Komorebi import targets: `komorebi_math.signals.polyphase.lattice_polyphase_analyze`, `komorebi_math.signals.polyphase.polyphase_analyze`.

O-T138: `tests/test_quadratic_forms.py`

Source: lines 1 through 238. **Test definitions:** 19. **Assert statements:** 25.

File SHA-256: `7eefe72b33024e7f6ad0705889d1f35892b48b275f4d1c5740deece5cdd24d23`.

Literal Komorebi import targets: `komorebi_math.exceptions.BackendUnavailableError`, `komorebi_math.exceptions.ValidationError`, `komorebi_math.quadratic_forms`.

O-T139: `tests/test_quadratic_forms_source.py`

Source: lines 1 through 151. **Test definitions:** 10. **Assert statements:** 0.

File SHA-256: `b85b4f4b0905c487ca0ab716ee53ceda287781b161433161a87be436586c0f6b`.

Literal Komorebi import targets: `komorebi_math.quadratic_forms._accuracy.certified_positive_logsf`, `komorebi_math.quadratic_forms._directed_accuracy.directed_relative_accuracy`, `komorebi_math.quadratic_forms._directed_accuracy.interval_endpoint_strings`, `komorebi_math.quadratic_forms._directed_accuracy.outward_interval_string`, `komorebi_math.quadratic_forms._directed_accuracy.scaled_accuracy_budget`, `komorebi_math.quadratic_forms._interval.interval_positive_logsf`, `komorebi_math.quadratic_forms._spectrum.certified_float64_factor_logsf`.

O-T140: `tests/test_quantum_channels_qec.py`

Source: lines 1 through 107. **Test definitions:** 7. **Assert statements:** 13.

File SHA-256: `659393bccf285330874497b375de5ac94e0829002c965502c074a8cbba5711a8`.

Literal Komorebi import targets: `komorebi_math.exceptions.ValidationError`, `komorebi_math.quantum.LindbladGenerator`, `komorebi_math.quantum.QuantumChannel`, `komorebi_math.quantum.QuantumCode`, `komorebi_math.quantum.amplitude_damping`, `komorebi_math.quantum.cyclic_symmetry_projectors`, `komorebi_math.quantum.dephase`, `komorebi_math.quantum.operator_packets`, `komorebi_math.quantum.reconstruct_operator_packets`.

O-T141: `tests/test_quantum_numerics_adversarial.py`

Source: lines 1 through 116. **Test definitions:** 11. **Assert statements:** 14.

File SHA-256: `bb906181fc59e8df9e95cbe61bdb27183f6a5c813c00a3dc614ecf2450c77984`.

Literal Komorebi import targets: `komorebi_math.dynamics.RationalMemoryRealization`, `komorebi_math.dynamics.matrix_exponential`, `komorebi_math.exceptions.ValidationError`, `komorebi_math.quantum.LindbladGenerator`, `komorebi_math.quantum.QuantumFeshbachReduction`, `komorebi_math.quantum._validation.density_matrix`, `komorebi_math.quantum.constraint_closure`, `komorebi_math.quantum.spectral_dimension`.

O-T142: `tests/test_quantum_symmetry_circuits.py`

Source: lines 1 through 99. **Test definitions:** 9. **Assert statements:** 12.

File SHA-256: `d3ecdc6991ef1a45cbca35705ec6d1f18f6aee3589f33134483e4976e8f30fef`.

Literal Komorebi import targets: `komorebi_math.core.PacketSystem`, `komorebi_math.exceptions.ValidationError`, `komorebi_math.quantum.CircuitPacketExpansion`, `komorebi_math.quantum.QuantumSeedSystem`, `komorebi_math.quantum.charge_certificate`, `komorebi_math.quantum.charge_selection_certificate`, `komorebi_math.quantum.computational_basis_generation`, `komorebi_math.quantum.cyclic_symmetry_projectors`, `komorebi_math.quantum.finite_group_projector`, `komorebi_math.quantum.local_orbit_dimension`, `komorebi_math.quantum.observable_seed_rank`, `komorebi_math.quantum.schmidt_rank`, `komorebi_math.quantum.total_charge_packets`.

O-T143: `tests/test_quantum_transport_effective_spectral.py`

Source: lines 1 through 145. **Test definitions:** 10. **Assert statements:** 12.

File SHA-256: `5351651fb9d84fbf8344166e63f430aacbd476c84646317b26a1c66a405f85d1`.

Literal Komorebi import targets: `komorebi_math.dynamics.matrix_exponential`, `komorebi_math.exceptions.ValidationError`, `komorebi_math.quantum.BoundedHamiltonianTower`, `komorebi_math.quantum.EuclideanTransfer`, `komorebi_math.quantum.KatoTransport`, `komorebi_math.quantum.MovingPacketSystem`, `komorebi_math.quantum.QuantumFeshbachReduction`, `komorebi_math.quantum.constraint_closure`, `komorebi_math.quantum.direct_packet_rotation`, `komorebi_math.quantum.fixed_packet_leakage`, `komorebi_math.quantum.heat_trace`, `komorebi_math.quantum.low_mode_dispersion_bound`, `komorebi_math.quantum.moving_geometric_coupling`, `komorebi_math.quantum.periodic_laplacian`, `komorebi_math.quantum.periodic_lattice_eigenvalues`, `komorebi_math.quantum.spectral_dimension`, `komorebi_math.quantum.transported_constraint_defect`.

O-T144: `tests/test_r4_block10_distortion_cocycle.py`

Source: lines 1 through 100. **Test definitions:** 5. **Assert statements:** 17.

File SHA-256: `45a50f4a9d4f12f6dcc5f7eb22c81f68faa27458be4c23fb3f29f15a96ce9012`.

Literal Komorebi import targets: `komorebi_math.nonlinear.distortion.compose_distortion_bound`, `komorebi_math.nonlinear.distortion.determinant_normalized`, `komorebi_math.nonlinear.distortion.metric_distortion_packet`, `komorebi_math.nonlinear.distortion.spectral_distortion_lower_bound`.

O-T145: `tests/test_r4_block11_convex_packets.py`

Source: lines 1 through 84. **Test definitions:** 5. **Assert statements:** 17.

File SHA-256: `e762386805db5091c3755d74bc3a02a019995d72ff2bc18d0293b4b3947b8cb4`.

Literal Komorebi import targets: `komorebi_math.nonlinear.convex_packets.joint_operator_bound`, `komorebi_math.nonlinear.convex_packets.parallelotope_norm_packet`, `komorebi_math.nonlinear.convex_packets.quadratic_operator_norm`.

O-T146: `tests/test_r4_block12_support_packets.py`

Source: lines 1 through 162. **Test definitions:** 8. **Assert statements:** 17.

File SHA-256: `da52f001b00bb5c628687586a50ef75359ce83b610bfdeb3bd2454baef892d5e`.

Literal Komorebi import targets: `komorebi_math.nonlinear.convex_packets.polyhedral_support_packet`.

O-T147: `tests/test_r4_block13_provenance_packets.py`

Source: lines 1 through 303. **Test definitions:** 12. **Assert statements:** 31.

File SHA-256: `32dca22ee29270a6ee9cf95edc441efdf3bbddf1070694babe5ca64dba841d6a`.

Literal Komorebi import targets: `komorebi_math.nonlinear.convex_packets.polyhedral_support_packet`, `komorebi_math.nonlinear.provenance_packets.compile_observer_convex_packet`, `komorebi_math.nonlinear.provenance_packets.compile_policy_packet`, `komorebi_math.nonlinear.provenance_packets.finite_provenance_observer`, `komorebi_math.nonlinear.provenance_packets.observer_convex_packet`.

O-T148: `tests/test_r4_block14_polyphase_route_packets.py`

Source: lines 1 through 73. **Test definitions:** 5. **Assert statements:** 11.

File SHA-256: `759458467d0180daeda5283c9e2c9edf564df2b32da032acc027715d5569f120`.

Literal Komorebi import targets: `komorebi_math.signals.filterbanks.HaarFilterBank`, `komorebi_math.signals.route_packets.PolyphaseRoutePacket`, `komorebi_math.signals.route_packets.polyphase_route_packet`.

O-T149: `tests/test_r4_block15_carrier_change.py`

Source: lines 1 through 256. **Test definitions:** 18. **Assert statements:** 65.

File SHA-256: `16b2255998f01d54147c1ec4e599e7e56924671cab8343b7b1c3bf5f71cd041f`.

Literal Komorebi import targets: `komorebi_math.core.carrier_change.SignedMassPacket`, `komorebi_math.core.carrier_change.TropicalResidualPacket`, `komorebi_math.core.carrier_change.boolean_support_nonnegative`, `komorebi_math.core.carrier_change.finite_factorization_audit`, `komorebi_math.exceptions.ValidationError`.

O-T150: `tests/test_r4_block16_hankel_falsification.py`

Source: lines 1 through 129. **Test definitions:** 4. **Assert statements:** 9.

File SHA-256: `fb9ff17f245eb198f68ea9f77c38f11c951e5cae68c5e8089b932cc792e05f92`.

Literal Komorebi import targets: `komorebi_math.arithmetic.recurrence.LinearRecurrence`, `komorebi_math.signals.route_packets.PolyphaseRoutePacket`.

O-T151: `tests/test_r4_block17_quantum_kraus_gauge.py`

Source: lines 1 through 131. **Test definitions:** 5. **Assert statements:** 17.

File SHA-256: `694b23f48d1df0d64458535082ee4230cb5849978f7bcdfbd993d377786f35b7`.

Literal Komorebi import targets: `komorebi_math.quantum.QuantumChannel`, `komorebi_math.quantum.QuantumCode`, `komorebi_math.quantum.amplitude_damping`.

O-T152: `tests/test_r4_block18_lindblad_gauge.py`

Source: lines 1 through 79. **Test definitions:** 4. **Assert statements:** 16.

File SHA-256: `74a1f0ca09458d392b3f55590876b4b19a050b4efa7047fd85b96d387d0c49a9`.

Literal Komorebi import targets: `komorebi_math.quantum.LindbladGenerator`, `komorebi_math.quantum.charge_certificate`.

O-T153: `tests/test_r4_block19_arithmetic_semantic_repair.py`

Source: lines 1 through 58. **Test definitions:** 5. **Assert statements:** 18.

File SHA-256: `88c8ccb3ee79b3cb5f6f7a10e6ceb3c0a8eaf5f400d0137cc0ff267834bac065`.

Literal Komorebi import targets: `komorebi_math.arithmetic.MahlerSeries`, `komorebi_math.arithmetic.TruncatedPadic`, `komorebi_math.arithmetic.UnitModeMahler`, `komorebi_math.generating.TruncatedPadicDomain`.

O-T154: `tests/test_r4_block20_deformation_separator_repair.py`

Source: lines 1 through 50. **Test definitions:** 4. **Assert statements:** 11.

File SHA-256: `54145d608c4c1d88d34135af9551370882129d1b7b2d1fff7494538cdbf440d1`.

Literal Komorebi import targets: `komorebi_math.deformation.finite_frobenius_separator`, `komorebi_math.exceptions.ValidationError`.

O-T155: `tests/test_r4_block21_certificate_consistency.py`

Source: lines 1 through 106. **Test definitions:** 6. **Assert statements:** 25.

File SHA-256: `4e2a50c080adfa8042e873f9320d852bc2e66f536f05d92e1e6e844c4ee42c39`.

Literal Komorebi import targets: `komorebi_math.algorithms.compiler.TreeNode`, `komorebi_math.algorithms.compiler.compile_tree_budget`, `komorebi_math.algorithms.compiler.compile_tree_lagrangian`, `komorebi_math.algorithms.precision.DTypeChoice`, `komorebi_math.algorithms.precision.assign_discrete_precisions`, `komorebi_math.certificates.Exactness`, `komorebi_math.exceptions.ValidationError`, `komorebi_math.langlands.finite_image.groups.FiniteGroup`, `komorebi_math.signals.wavelets.best_basis`, `komorebi_math.signals.wavelets.haar_packet_analyze`.

O-T156: `tests/test_r4_block9_frame_metric_packet.py`

Source: lines 1 through 176. **Test definitions:** 6. **Assert statements:** 6.

File SHA-256: `7f57cad10ff2c519ed18122762aa449893c4b48ce128d22557c5f4a073392ce9`.

Literal Komorebi import targets: `komorebi_math.coordinates.AnalysisSynthesisGauge`, `komorebi_math.coordinates.AnalysisSynthesisSystem`, `komorebi_math.coordinates.ExactAnalysisSynthesisSystem`, `komorebi_math.coordinates.exact.matrix_multiply`, `komorebi_math.coordinates.exact.transpose`, `komorebi_math.exceptions.ValidationError`, `komorebi_math.nonlinear.TransportEdge`, `komorebi_math.nonlinear.TransportGraph`.

O-T157: `tests/test_refinement_towers.py`

Source: lines 1 through 89. **Test definitions:** 7. **Assert statements:** 17.

File SHA-256: `53a9d52f458c5a1c604d4d3b7d73656cf1ce8ca599e66b0c4bd2cd051fcccdab`.

Literal Komorebi import targets: `komorebi_math.PacketSystem`, `komorebi_math.exceptions.ValidationError`, `komorebi_math.refinement.PacketRefinement`, `komorebi_math.refinement.RefinementTower`, `komorebi_math.refinement.common_refinement`.

O-T158: `tests/test_release_closeout_contracts.py`

Source: lines 1 through 126. **Test definitions:** 5. **Assert statements:** 22.

File SHA-256: `6e91867e724133bc6d27fbc7034ddb8df46f32746bbf3a6055be6b4ec6f655a6`.

Literal Komorebi import targets: `komorebi_math.certificates.Certificate`, `komorebi_math.certificates.Exactness`, `komorebi_math.exceptions.ValidationError`, `komorebi_math.export.persistence.dumps_portable`, `komorebi_math.export.persistence.from_portable_data`, `komorebi_math.export.persistence.loads_portable`, `komorebi_math.export.persistence.portable_digest`, `komorebi_math.export.persistence.to_portable_data`, `komorebi_math.nonlinear.autodiff.AutomaticDifferentiationProvider`.

O-T159: `tests/test_release_closeout_qualification.py`

Source: lines 1 through 144. **Test definitions:** 6. **Assert statements:** 23.

File SHA-256: `acf3b9e62f9c98b9f1cf1f77317a80af0312560e6d11f50ee84c6bf83e6838c1`.

Literal Komorebi import targets: none. This syntax result does not rule out subprocess, helper-mediated or dynamic package use.

O-T160: `tests/test_release_closeout_tools.py`

Source: lines 1 through 228. **Test definitions:** 14. **Assert statements:** 29.

File SHA-256: `7b9720f1b8c8b81dfe4940c24631c942676019473263dd60607a98f5f9849df2`.

Literal Komorebi import targets: none. This syntax result does not rule out subprocess, helper-mediated or dynamic package use.

O-T161: tests/test_release_metadata.py

Source: lines 1 through 88. **Test definitions:** 8. **Assert statements:** 10.

File SHA-256: ac6646e9998788eb986dc8ae8019fbdbd629fb0b0a9357bbfd44cc21a0eb5a81.

Literal Komorebi import targets: none. This syntax result does not rule out subprocess, helper-mediated or dynamic package use.

O-T162: tests/test_release_promotion.py

Source: lines 1 through 357. **Test definitions:** 15. **Assert statements:** 24.

File SHA-256: 95ea796b7cb5ee42fc6e7a24ce7db1f64fd6561abdb9d4d66b9de8bd38c1e05d.

Literal Komorebi import targets: komorebi_math.release.evidence, komorebi_math.release.load_release_contract, komorebi_math.release.readiness.

O-T163: tests/test_release_repair.py

Source: lines 1 through 252. **Test definitions:** 20. **Assert statements:** 34.

File SHA-256: 8b1e8a603799b8abd17b8a7c93226fde6d542d64d49341d787a930870d460963.

Literal Komorebi import targets: none. This syntax result does not rule out subprocess, helper-mediated or dynamic package use.

O-T164: tests/test_repair_r3_hardening.py

Source: lines 1 through 282. **Test definitions:** 28. **Assert statements:** 23.

File SHA-256: b8bfbdf3d0357c24a9c604db403abdfe555dfe3558791ce5e8aa0ae9449f85b7.

Literal Komorebi import targets: komorebi_math.certificates.Certificate, komorebi_math.certificates.Exactness, komorebi_math.coordinates.budget.CoordinateBudget, komorebi_math.core.packets.PacketSystem, komorebi_math.exceptions.ResourceLimitError, komorebi_math.exceptions.ValidationError, komorebi_math.export.persistence.dumps_portable, komorebi_math.export.persistence.from_portable_data, komorebi_math.export.persistence.loads_portable, komorebi_math.export.persistence.to_portable_data, komorebi_math.expression.model.Expression, komorebi_math.expression.model.ExpressionBudget, komorebi_math.generating.basis.ordinary_basis, komorebi_math.generating.budget.GeneratingBudget, komorebi_math.generating.domains.RationalDomain, komorebi_math.generating.finite_series.FiniteNormalizedSeries, komorebi_math.nonlinear.derivatives.PolynomialDerivativeProvider, komorebi_math.nonlinear.execution.compile_packet_jet_plan, komorebi_math.nonlinear.jets.PacketJetFamily, komorebi_math.nonlinear.trees.

O-T165: tests/test_repair_r4_block1.py

Source: lines 1 through 274. **Test definitions:** 21. **Assert statements:** 62.

File SHA-256: 4aee95384fcb182d2d26e66a62251e2c08606bef21458975f3415deb6b3b11a7.

Literal Komorebi import targets: komorebi_math.OperatorMorphism, komorebi_math.PacketMorphism, komorebi_math.PacketOperator, komorebi_math.PacketSystem, komorebi_math.SeedMorphism, komorebi_math.SeedSystem, komorebi_math.backends.get_backend, komorebi_math.config.Settings, komorebi_math.config.configure, komorebi_math.config.get_settings, komorebi_math.config.settings_context, komorebi_math.exceptions.ValidationError, komorebi_math.nonlinear.trees.majorant_convergence, komorebi_math.nonlinear.trees.quadratic_catalan_majorant, komorebi_math.refinement.towers.PacketRefinement.

O-T166: tests/test_repair_r4_block2.py

Source: lines 1 through 375. **Test definitions:** 36. **Assert statements:** 58.

File SHA-256: 2c17a2e248be2da45d2acb6ce43e12d7a9f1825326937d56bf5949a7523abb29.

Literal Komorebi import targets: komorebi_math._residuals.value_audit, komorebi_math.certificates.Exactness, komorebi_math.core.operators.PacketOperator, komorebi_math.core.packets.PacketSystem, komorebi_math.core.transfer.block_from_paths, komorebi_math.deformation.complexes.FiniteCochainComplex, komorebi_math.deformat

ion.complexes.row_rank_mod, komorebi_math.exceptions.DimensionError, komorebi_math.exceptions.ValidationError, komorebi_math.nonlinear.composition.faa_di_bruno, komorebi_math.nonlinear.composition.packet_faa_di_bruno, komorebi_math.nonlinear.coordinates.NonlinearPacketMap, komorebi_math.nonlinear.derivatives.CallableDerivativeProvider, komorebi_math.nonlinear.graded.FiniteAbelianGroup, komorebi_math.nonlinear.graded.GradedPacketOperation, komorebi_math.nonlinear.graded.divided_square_packets, komorebi_math.nonlinear.graded.higher_maurer_cartan_packets, komorebi_math.nonlinear.graded.maurer_cartan_packets, komorebi_math.nonlinear.holonomy.TransportEdge, komorebi_math.nonlinear.holonomy.TransportGraph, komorebi_math.nonlinear.jets.PacketJetFamily, komorebi_math.nonlinear.jets.packet_jet_support, komorebi_math.refinement.defects.LinearRefinementDefect, komorebi_math.refinement.defects.multilevel_defect, komorebi_math.refinement.weighted.WeightedTransferGraph.

O-T167: `tests/test_repair_r4_block3.py`

Source: lines 1 through 262. **Test definitions:** 31. **Assert statements:** 46.

File SHA-256: `257bd1eadfda2d2c44e1acb354b369197d5ba9614688e7da795dcf5dbabb6b51`.

Literal Komorebi import targets: komorebi_math.Exactness, komorebi_math.PacketSystem, komorebi_math._residuals.value_audit, komorebi_math.exceptions.ValidationError, komorebi_math.nonlinear.correction.HilbertObstructionProjector, komorebi_math.nonlinear.derivatives.CallableDerivativeProvider, komorebi_math.nonlinear.derivatives.PolynomialDerivativeProvider, komorebi_math.nonlinear.execution.execute_packet_jets, komorebi_math.nonlinear.jets.PacketJetFamily, komorebi_math.nonlinear.obstruction.ObstructionSquare, komorebi_math.nonlinear.snapshots.PacketJetCoefficient, komorebi_math.nonlinear.snapshots.snapshot_packet_jet, komorebi_math.nonlinear.sparse.SparsePolynomialDerivativeProvider, komorebi_math.nonlinear.sparse.SparseSymmetricTensor, komorebi_math.nonlinear.streaming.capture_packet_jet_stream, komorebi_math.nonlinear.streaming.open_packet_jet_snapshot_stream, komorebi_math.nonlinear.streaming.save_packet_jet_snapshot_stream, komorebi_math.nonlinear.trees.OrderedRootedTree, komorebi_math.nonlinear.trees.PolynomialFixedPointProblem, komorebi_math.nonlinear.trees.packet_tree_reconstruction.

O-T168: `tests/test_repair_r4_block4.py`

Source: lines 1 through 169. **Test definitions:** 16. **Assert statements:** 17.

File SHA-256: `1d34c5b57b7c19fa28a7c0897d547ae797adfc4e9feaeb399dd3bde433df6895`.

Literal Komorebi import targets: komorebi_math.Exactness, komorebi_math.exceptions.ValidationError, komorebi_math.nonlinear.correction.LyapunovSchmidtReduction, komorebi_math.nonlinear.correction.RightInverseCorrectionProblem, komorebi_math.nonlinear.derivatives.CallableDerivativeProvider, komorebi_math.nonlinear.derivatives.SympyDerivativeProvider, komorebi_math.nonlinear.elimination.NonlinearSchurElimination, komorebi_math.nonlinear.trees.PolynomialFixedPointProblem.

O-T169: `tests/test_repair_r4_block5.py`

Source: lines 1 through 136. **Test definitions:** 16. **Assert statements:** 18.

File SHA-256: `743548619e1f0063648f4628386f55f4eb5db74436ca6770a410bcd92a940002`.

Literal Komorebi import targets: komorebi_math.Exactness, komorebi_math.PacketSystem, komorebi_math.deformation.companions.finite_frobenius_separator, komorebi_math.nonlinear.composition.packet_faa_di_bruno, komorebi_math.nonlinear.coordinates.NonlinearPacketMap, komorebi_math.nonlinear.derivatives.CallableDerivativeProvider, komorebi_math.nonlinear.execution.execute_packet_jets, komorebi_math.nonlinear.graded.FiniteAbelianGroup, komorebi_math.nonlinear.graded.GradedPacketOperation, komorebi_math.nonlinear.jets.PacketJetFamily, komorebi_math.nonlinear.jets.packet_jet_support, komorebi_math.nonlinear.jets.packet_taylor, komorebi_math.nonlinear.transport.KatoTransport, komorebi_math.nonlinear.transport.MovingPacketSystem, komorebi_math.nonlinear.trees.OrderedRootedTree, komorebi_math.nonlinear.trees.PolynomialFixedPointProblem, komorebi_math.nonlinear.trees.packet_tree_reconstruction.

O-T170: `tests/test_repair_r4_block6.py`

Source: lines 1 through 141. **Test definitions:** 10. **Assert statements:** 25.

File SHA-256: `cb81bd910c6b1defb9eee9a2ef49ded5b7bb7c45560c47742bf0318a92c5532d`.

Literal Komorebi import targets: `komorebi_math.OperatorGrammar`, `komorebi_math.OperatorRealization`, `komorebi_math.PacketOperator`, `komorebi_math.PacketSystem`, `komorebi_math.coordinates.analysis_synthesis.AnalysisSynthesisSystem`, `komorebi_math.coordinates.numeric.NumericCoordinateTransform`, `komorebi_math.nonlinear.correction.HilbertObstructionProjector`, `komorebi_math.nonlinear.correction.LyapunovSchmidtReduction`, `komorebi_math.nonlinear.correction.RightInverseCorrectionProblem`, `komorebi_math.nonlinear.derivatives.CallableDerivativeProvider`, `komorebi_math.refinement.WeightedTransferEdge`, `komorebi_math.refinement.WeightedTransferGraph`.

O-T171: `tests/test_repair_r4_block7.py`

Source: lines 1 through 173. **Test definitions:** 11. **Assert statements:** 29.

File SHA-256: `9cbc6f5428fcb5d2a4e86a3e674249ee441e96f58d8958f706df79ca9c62dc92`.

Literal Komorebi import targets: `komorebi_math.Certificate`, `komorebi_math.Exactness`, `komorebi_math.PacketSystem`, `komorebi_math.exceptions.ValidationError`, `komorebi_math.export.persistence.dumps_portable`, `komorebi_math.export.persistence.loads_portable`, `komorebi_math.nonlinear.derivatives.PolynomialDerivativeProvider`, `komorebi_math.nonlinear.jets.PacketJetFamily`, `komorebi_math.nonlinear.snapshots.PacketJetSnapshot`, `komorebi_math.nonlinear.snapshots.snapshot_packet_jet`, `komorebi_math.nonlinear.streaming.load_packet_jet_snapshot_stream`, `komorebi_math.nonlinear.streaming.open_packet_jet_snapshot_stream`, `komorebi_math.nonlinear.streaming.save_packet_jet_snapshot_stream`.

O-T172: `tests/test_repair_r4_block8.py`

Source: lines 1 through 101. **Test definitions:** 7. **Assert statements:** 17.

File SHA-256: `551f4046d337caca24982cda3acf2a51f8329fd9c8195fd3246f39aaaa1a2562`.

Literal Komorebi import targets: `komorebi_math.exceptions.ValidationError`, `komorebi_math.nonlinear.TransportEdge`, `komorebi_math.nonlinear.TransportGraph`.

O-T173: `tests/test_residue_crt.py`

Source: lines 1 through 63. **Test definitions:** 6. **Assert statements:** 11.

File SHA-256: `33c6a98ff699bf43e3e167ee11c74283f9436f771df24821a76eebb00be62536`.

Literal Komorebi import targets: `komorebi_math.refinement.crt_many`, `komorebi_math.refinement.crt_pair`, `komorebi_math.refinement.dyadic_odd_factor`, `komorebi_math.refinement.factor_residue`, `komorebi_math.refinement.nested_residue_address`, `komorebi_math.refinement.nested_residue_analysis`, `komorebi_math.refinement.residue_analyze`, `komorebi_math.refinement.residue_packet_system`.

O-T174: `tests/test_seeds.py`

Source: lines 1 through 78. **Test definitions:** 9. **Assert statements:** 14.

File SHA-256: `a83bbf3b8f0c001f6e6afa9446ce5e32946e9d022d5a55a6d7df3d75fc18e6bf`.

Literal Komorebi import targets: `komorebi_math.SeedSystem`, `komorebi_math.exceptions.ValidationError`, `komorebi_math.semisimple_seed_rank`, `komorebi_math.single_operator_seed_rank`.

O-T175: `tests/test_sequences_execution.py`

Source: lines 1 through 91. **Test definitions:** 8. **Assert statements:** 21.

File SHA-256: `0b292eb415ab89c675c052f8ea569d1000c0586b1a5fd6c3f0b81d5a90c01f7a`.

Literal Komorebi import targets: `komorebi_math.arithmetic.LinearRecurrence`, `komorebi_math.exceptions.ValidationError`, `komorebi_math.sequences.Sequence`, `komorebi_math.sequences.apery_sequence`, `komorebi_math.sequences.bell_sequence`, `komorebi_math.sequences.binomial_transform_values`, `komorebi_math.sequences.catalan_sequence`, `komorebi_math.sequences.compile_sequence`, `komorebi_math.sequences.detect_linear_recurrence`, `komorebi_math.sequences.`

fibonacci_sequence, komorebi_math.sequences.finite_difference, komorebi_math.sequences.get_sequence, komorebi_math.sequences.inverse_binomial_transform_values, komorebi_math.sequences.motzkin_sequence, komorebi_math.sequences.partition_sequence, komorebi_math.sequences.thue_morse_sequence.

O-T176: tests/test_sequences_models.py

Source: lines 1 through 69. **Test definitions:** 5. **Assert statements:** 14.

File SHA-256: 08362783f399ce9debdc569194fe981f55696a17fe7709e8651c38997727139f.

Literal Komorebi import targets: komorebi_math.exceptions.ValidationError, komorebi_math.sequences.AutomaticSequence, komorebi_math.sequences.Sequence, komorebi_math.sequences.apery_recurrence, komorebi_math.sequences.apery_sequence, komorebi_math.sequences.base_digits, komorebi_math.sequences.compile_sequence, komorebi_math.sequences.motzkin_recurrence, komorebi_math.sequences.motzkin_sequence, komorebi_math.sequences.thue_morse_automaton, komorebi_math.sequences.thue_morse_sequence.

O-T177: tests/test_sequences_oeis_families.py

Source: lines 1 through 90. **Test definitions:** 8. **Assert statements:** 15.

File SHA-256: 8792dbfbbaf17f2ae2b8768d990fac9cd19ed66fc643849e50fc90d947ac6efe.

Literal Komorebi import targets: komorebi_math.arithmetic.hurwitz.hurwitz_inverse, komorebi_math.arithmetic.hurwitz.hurwitz_product, komorebi_math.exceptions.ValidationError, komorebi_math.sequences.derangement_sequence, komorebi_math.sequences.fubini_sequence, komorebi_math.sequences.kbonacci_closed_form, komorebi_math.sequences.kbonacci_sequence, komorebi_math.sequences.reciprocal_coefficients, komorebi_math.sequences.reciprocal_sequence, komorebi_math.sequences.universal_denominator_coefficients.

O-T178: tests/test_sequences_oeis_matching.py

Source: lines 1 through 137. **Test definitions:** 14. **Assert statements:** 34.

File SHA-256: 86ad9c49570a728b7e53b4e3031669de46cf1a8f8572f86198399474ce16602d.

Literal Komorebi import targets: komorebi_math.exceptions.ValidationError, komorebi_math.sequences.LocalOEISCorpus, komorebi_math.sequences.OEISPrefixMatch, komorebi_math.sequences.OEISRecord, komorebi_math.sequences.compare_oeis_prefix, komorebi_math.sequences.find_local_oeis_matches, komorebi_math.sequences.load_local_oeis, komorebi_math.sequences.parse_oeis_internal, komorebi_math.sequences.parse_oeis_stripped.

O-T179: tests/test_signals_extended.py

Source: lines 1 through 74. **Test definitions:** 7. **Assert statements:** 15.

File SHA-256: 0bde278d058f53dfb614f6175b8b4336d48631897a214998fdeadebf847f8ed6.

Literal Komorebi import targets: komorebi_math.certificates.Exactness, komorebi_math.signals.PeriodicFilterBank, komorebi_math.signals.estimate_positive_packet_1d, komorebi_math.signals.gamma_from_impedance, komorebi_math.signals.impedance_from_gamma, komorebi_math.signals.lattice_polyphase_analyze, komorebi_math.signals.nonlinear_shift_filter, komorebi_math.signals.packet_product, komorebi_math.signals.phase_mobius, komorebi_math.signals.polyphase_filter_kernels, komorebi_math.signals.residue_project, komorebi_math.signals.scalar_region_lp_error, komorebi_math.signals.smith_sensitivity.

O-T180: tests/test_signals_polyphase_filterbanks.py

Source: lines 1 through 93. **Test definitions:** 7. **Assert statements:** 12.

File SHA-256: 8e65145ccdda13f672454fb36808d2d1364cbf3cd4a0c05248afaad57c577db7.

Literal Komorebi import targets: komorebi_math.signals.FiniteFilterBank, komorebi_math.signals.PolyphaseSystem, komorebi_math.signals.haar_analyze, komorebi_math.signals.haar_filter_bank, komorebi_math.signals.haar_synthesize, komorebi_math.signals.mixed_radix_address, komorebi_math.signals.periodic_five_three_lifting, komorebi_math.signals.polyphase_analyze, komorebi_math.signals.polyphase_convolution, komorebi_math.signals.refinement_children, komorebi_math.signals.split_mixed_radix_address.

O-T181: `tests/test_signals_recovery.py`

Source: lines 1 through 105. **Test definitions:** 6. **Assert statements:** 11.

File SHA-256: `b4a92ddda5a84305a00c013b3ee0bc907b698d18e53b8d4fff1eb27e24ccbb48`.

Literal Komorebi import targets: `komorebi_math.exceptions.ValidationError`, `komorebi_math.signals.detect_piecewise_polynomial_breaks`, `komorebi_math.signals.finite_difference`, `komorebi_math.signals.haar_lipschitz_bound`, `komorebi_math.signals.haar_step_coefficient`, `komorebi_math.signals.recover_jumps_subgaussian`, `komorebi_math.signals.recover_separated_jumps`, `komorebi_math.signals.recover_vector_jumps`.

O-T182: `tests/test_signals_wavelets_singular.py`

Source: lines 1 through 82. **Test definitions:** 5. **Assert statements:** 7.

File SHA-256: `a641231d371a10fe15873892472fce59371b038eaf72af12b854eeabb5a5fc31`.

Literal Komorebi import targets: `komorebi_math.signals.HaarPacketTree`, `komorebi_math.signals.SingularPacket1D`, `komorebi_math.signals.TruncatedPowerTerm`, `komorebi_math.signals.best_basis`, `komorebi_math.signals.entropy_cost`, `komorebi_math.signals.overlap_matrix_error`, `komorebi_math.signals.retained_packet_reconstruction`, `komorebi_math.signals.scalar_region_error`.

O-T183: `tests/test_smith_certificate_soundness.py`

Source: lines 1 through 73. **Test definitions:** 3. **Assert statements:** 6.

File SHA-256: `4a755540be8858ee99dad4a2821303d7edfb9a1b7bf4db687f6e39fe417e001d`.

Literal Komorebi import targets: `komorebi_math.exceptions.ValidationError`, `komorebi_math.refinement.SmithDecomposition`, `komorebi_math.refinement.smith_decomposition`.

O-T184: `tests/test_special_functions_bernoulli.py`

Source: lines 1 through 74. **Test definitions:** 7. **Assert statements:** 11.

File SHA-256: `9919d95152a9dce8abeb5e481ccd184a865bbb751880f7eb1c70eb3ad58bc78a`.

Literal Komorebi import targets: `komorebi_math.exceptions.ValidationError`, `komorebi_math.special_functions.bernoulli_hurwitz`, `komorebi_math.special_functions.bernoulli_hurwitz_packet`, `komorebi_math.special_functions.bernoulli_packet_integer`.

O-T185: `tests/test_special_functions_shape.py`

Source: lines 1 through 141. **Test definitions:** 11. **Assert statements:** 6.

File SHA-256: `9ecef47b2779ee46b54ed0f1ccc1596ba929f389809d95cdb684ed57ba86afdc`.

Literal Komorebi import targets: `komorebi_math.exceptions.DimensionError`, `komorebi_math.exceptions.ValidationError`, `komorebi_math.special_functions.beta_binomial_refinement`, `komorebi_math.special_functions.beta_logit_density`, `komorebi_math.special_functions.gamma_sech_ambiguity`, `komorebi_math.special_functions.log_gamma_fourier`, `komorebi_math.special_functions.log_gamma_window`, `komorebi_math.special_functions.meixner_characteristic`, `komorebi_math.special_functions.meixner_density`, `komorebi_math.special_functions.meixner_pollaczek`, `komorebi_math.special_functions.shape_affinity`, `komorebi_math.special_functions.shape_refinement_coefficient`, `komorebi_math.special_functions.shape_transfer_singular_value`.

O-T186: `tests/test_special_functions_terminating.py`

Source: lines 1 through 82. **Test definitions:** 8. **Assert statements:** 10.

File SHA-256: `a5b476c9b6293091390f1183c4536c619b87d50b3b0e8e3f8ee3516f83b121a3`.

Literal Komorebi import targets: `komorebi_math.exceptions.ValidationError`, `komorebi_math.special_functions.q_pochhammer`, `komorebi_math.special_functions.q_racah_lattice`, `komorebi_math.special_functions.q_racah_scalar`, `komorebi_math.special_functions.racah_scalar`.

O-T187: `tests/test_stress_autodiff_contracts.py`

Source: lines 1 through 112. **Test definitions:** 6. **Assert statements:** 6.

File SHA-256: `8d9bc6218f9a3c6f5d6d8e243cce566783d206a9d6a72f27a17e034dbc2ac73c`.

Literal Komorebi import targets: `komorebi_math.certificates.Exactness`, `komorebi_math.exceptions.ValidationError`, `komorebi_math.nonlinear.autodiff.AutomaticDifferentiationProvider`, `komorebi_math.nonlinear.autodiff.JaxDerivativeProvider`, `komorebi_math.nonlinear.autodiff.TorchDerivativeProvider`.

O-T188: `tests/test_stress_certificates.py`

Source: lines 1 through 432. **Test definitions:** 17. **Assert statements:** 44.

File SHA-256: `9ba361e16bfd329fc0688d34b061ebc9a16574735ca050e83fea8717db53a313`.

Literal Komorebi import targets: `komorebi_math.certificates.Certificate`, `komorebi_math.certificates.ComputationResult`, `komorebi_math.certificates.Exactness`, `komorebi_math.core.semilattice.FiniteJoinSemilattice`, `komorebi_math.core.semilattice.SemilatticeQuotient`, `komorebi_math.exceptions.ValidationError`, `komorebi_math.export.persistence`, `komorebi_math.persistence_contract.PortableCodecContract`, `komorebi_math.persistence_contract.PortableCodecContractEntry`.

O-T189: `tests/test_stress_directed_tails.py`

Source: lines 1 through 196. **Test definitions:** 12. **Assert statements:** 33.

File SHA-256: `65d27ce700821d777b938a91c1e8f911b6adc5aad6cfc31dae54c748967becaa`.

Literal Komorebi import targets: `komorebi_math.exceptions.ValidationError`, `komorebi_math.quadratic_forms`.

O-T190: `tests/test_stress_exact_arithmetic.py`

Source: lines 1 through 314. **Test definitions:** 15. **Assert statements:** 56.

File SHA-256: `c8420926ec0bdfde15dbb95160c8133be622cc3a1930080fe1bd49c188cd4e2c`.

Literal Komorebi import targets: `komorebi_math.arithmetic.hurwitz.HurwitzSeries`, `komorebi_math.arithmetic.hurwitz.compressed_hurwitz_inverse`, `komorebi_math.arithmetic.hurwitz.hurwitz_inverse`, `komorebi_math.arithmetic.hurwitz.hurwitz_product`, `komorebi_math.arithmetic.hurwitz.subgroup_selector`, `komorebi_math.arithmetic.recurrence.LinearRecurrence`, `komorebi_math.arithmetic.recurrence.matrix_power_exact`, `komorebi_math.arithmetic.recurrence.matrix_power_mod`, `komorebi_math.coordinates.exact.as_matrix`, `komorebi_math.coordinates.exact.matrix_power`, `komorebi_math.coordinates.packet.digit_reversal_order`, `komorebi_math.coordinates.packet.residue_class_order`, `komorebi_math.coordinates.packet.tensor_axis_permutation_order`, `komorebi_math.coordinates.recurrence.analyze_recurrence_coordinates`, `komorebi_math.coordinates.recurrence.variation_of_constants_prefix`, `komorebi_math.exceptions.DimensionError`, `komorebi_math.exceptions.ResourceLimitError`, `komorebi_math.exceptions.ValidationError`, `komorebi_math.generating.FiniteNormalizedSeries`, `komorebi_math.generating.FormalSeries`, `komorebi_math.generating.GeneratingBudget`, `komorebi_math.generating.IntegerDomain`, `komorebi_math.generating.RationalDomain`, `komorebi_math.generating.ResidueRingDomain`, `komorebi_math.generating.WeightedMonomialBasis`, `komorebi_math.generating.basis_from_values`, `komorebi_math.generating.exponential_basis`, `komorebi_math.generating.factorial_power_basis`, `komorebi_math.generating.fibonacci_factorial_basis`, `komorebi_math.generating.ordinary_basis`, `komorebi_math.generating.pochhammer_basis`, `komorebi_math.generating.q_factorial_basis`, `komorebi_math.sequences.base.Sequence`, `komorebi_math.sequences.oeis.OEISRecord`, `komorebi_math.sequences.oeis.compare_oeis_prefix`, `komorebi_math.sequences.transforms.binomial_transform_values`, `komorebi_math.sequences.transforms.inverse_binomial_transform_values`, `komorebi_math.sequences.transforms.partial_sums`.

O-T191: `tests/test_stress_numerical.py`

Source: lines 1 through 209. **Test definitions:** 19. **Assert statements:** 8.

File SHA-256: `90f45a170a43c61572149de8491ed30caa695fcaf2b5329ddac75f3d5e334955`.

Literal Komorebi import targets: `komorebi_math.exceptions.DimensionError`, `komorebi_math.exceptions.ResourceLimitError`, `komorebi_math.exceptions.ValidationError`, `komorebi_math.quantum.channels.QuantumChannel`, `komorebi_math.quantum.dirac.central_mode_momentum`, `komorebi_math.quantum.lindblad.LindbladGenerator`, `komorebi_math.`

quantum.spectral.heat_trace, komorebi_math.quantum.spectral.low_mode_dispersion_bound, komorebi_math.quantum.spectral.periodic_laplacian, komorebi_math.quantum.spectral.periodic_lattice_eigenvalues, komorebi_math.special_functions.shape.beta_binomial_refinement, komorebi_math.special_functions.shape.gamma_sech_ambiguity, komorebi_math.special_functions.shape.log_gamma_window, komorebi_math.special_functions.shape.meixner_characteristic, komorebi_math.special_functions.shape.meixner_density, komorebi_math.special_functions.shape.shape_affinity, komorebi_math.special_functions.shape.shape_refinement_coefficient, komorebi_math.special_functions.shape.shape_transfer_singular_value.

O-T192: tests/test_stress_optional_backends.py

Source: lines 1 through 167. **Test definitions:** 2. **Assert statements:** 7.

File SHA-256: d6d4e4537fe7484c13cc73404e51d436bc66563cccd6516b9c0f36a9f810817b.

Literal Komorebi import targets: komorebi_math.nonlinear.autodiff.ad_framework_diagnostics.

O-T193: tests/test_unified_extensions_cli.py

Source: lines 1 through 86. **Test definitions:** 8. **Assert statements:** 17.

File SHA-256: ed34ace5df11a369fe28befa193b0d4f76464a7c8c1997128bd37d3635ba456d.

Literal Komorebi import targets: komorebi_math.OperatorGrammar, komorebi_math.OperatorRealization, komorebi_math.PacketSystem, komorebi_math.SeedSystem, komorebi_math.UnifiedSystem, komorebi_math.cli.main.main, komorebi_math.extensions.discover_extensions, komorebi_math.verify.run_core_verification.

O-T194: tests/test_weighted_transfer.py

Source: lines 1 through 111. **Test definitions:** 9. **Assert statements:** 15.

File SHA-256: 0693af2ea38aae6b4ee4b2e2203a3cfb45fb280709bd7b4f3a4b34177d62362f.

Literal Komorebi import targets: komorebi_math.PacketSystem, komorebi_math.exceptions.ValidationError, komorebi_math.refinement.WeightedTransferEdge, komorebi_math.refinement.WeightedTransferGraph.

O.5 Installed verification-module index

O-V001: src/komorebi_math/verify/__init__.py

Source: lines 1 through 249. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: a96ec56129d61cd76bcb3fd996963d310594112e8e8a04e2d8e420405aa0ac9f.

Literal Komorebi import targets: komorebi_math.verify.algorithms.run_algorithms_verification, komorebi_math.verify.analytic.run_analytic_verification, komorebi_math.verify.arithmetic.run_arithmetic_verification, komorebi_math.verify.beta.run_beta_readiness_verification, komorebi_math.verify.beta.run_documentation_verification, komorebi_math.verify.beta.run_evidence_verification, komorebi_math.verify.beta.run_upgrade_verification, komorebi_math.verify.coordinates.run_coordinates_verification, komorebi_math.verify.core.VerificationCheck, komorebi_math.verify.core.VerificationReport, komorebi_math.verify.core.run_core_verification, komorebi_math.verify.deformation.run_deformation_verification, komorebi_math.verify.dsp.run_dsp_verification, komorebi_math.verify.dynamics.run_dynamics_verification, komorebi_math.verify.export.run_export_verification, komorebi_math.verify.expression.run_expression_verification, komorebi_math.verify.frame_algebra.run_frame_algebra_verification, komorebi_math.verify.generating.run_generating_verification, komorebi_math.verify.gpu.run_gpu_verification, komorebi_math.verify.integration.run_integration_verification, komorebi_math.verify.langlands.run_langlands_completion_verification, komorebi_math.verify.langlands.run_langlands_finite_image_verification, komorebi_math.verify.langlands.run_langlands_local_verification, komorebi_math.verify.langlands.run_langlands_verification, komorebi_math.verify.nonlinear.run_nonlinear_foundation_verification, komorebi_math.verify.nonlinear.run_nonlinear_verification, komorebi_math.verify.nonlinear_advanced.run_advanced_nonlinear_verification, komorebi_math.verify.quantum.run_quantum_verification, komorebi_math.verify.refinement.run_refinement_verification, komorebi_math.verify.release.run_release_qualification_verification, komorebi_math.verify.scalable_nonlinear.run_scalable_nonlinear_verification, komorebi_math.verify.sequences.run_sequences_verification, komorebi_math.verify.signals.run_signals_verification, komorebi_math.verify.stabilization.run_polyphase_stabilization_

verification, komorebi_math.verify.stabilization.run_stable_contract_verification, komorebi_math.verify.streaming_beta.run_streaming_beta_verification.

O-V002: src/komorebi_math/verify/algorithms.py

Source: lines 1 through 150. **Test definitions:** 0. **Assert statements:** 11.

File SHA-256: b9ca065d0d4af8a436b45976b936b671714050f70a818e570a2ff597f10b4ada.

Literal Komorebi import targets: komorebi_math.algorithms.BlockDiagonalLowRankOperator, komorebi_math.algorithms.BlockDiagonalOperator, komorebi_math.algorithms.DTypeChoice, komorebi_math.algorithms.PacketIR, komorebi_math.algorithms.PacketLayout, komorebi_math.algorithms.Task, komorebi_math.algorithms.TreeNode, komorebi_math.algorithms.assign_discrete_precisions, komorebi_math.algorithms.compile_tree_lagrangian, komorebi_math.algorithms.conjugate_gradient, komorebi_math.algorithms.discover_block_diagonal_low_rank, komorebi_math.algorithms.packet_executor, komorebi_math.algorithms.recompress_factors, komorebi_math.algorithms.schedule_unlimited, komorebi_math.algorithms.schur_fill_support, komorebi_math.verify.core.VerificationCheck, komorebi_math.verify.core.VerificationReport.

O-V003: src/komorebi_math/verify/analytic.py

Source: lines 1 through 104. **Test definitions:** 0. **Assert statements:** 11.

File SHA-256: d2ccc2ee0e4ae07aeb746edcbe9c91f6b7fbace219da4b287b20799db9d12136.

Literal Komorebi import targets: komorebi_math.analytic.CyclicDenominator, komorebi_math.analytic.appell_polynomial, komorebi_math.analytic.congruential_inverse, komorebi_math.analytic.cyclic_projection_certificate, komorebi_math.analytic.denominator, komorebi_math.analytic.denominator_root_sum, komorebi_math.analytic.positive_zero_string, komorebi_math.analytic.reciprocal_coefficients, komorebi_math.analytic.reciprocal_semigroup_certificate, komorebi_math.analytic.section_certificate, komorebi_math.analytic.theta_polynomial, komorebi_math.verify.core.VerificationCheck, komorebi_math.verify.core.VerificationReport, komorebi_math.verify.core._capture.

O-V004: src/komorebi_math/verify/arithmetic.py

Source: lines 1 through 101. **Test definitions:** 0. **Assert statements:** 12.

File SHA-256: 7f4960cd813a8e2a549556b963bf7695c9fa2d34889f6a46bf5747007ccaab03.

Literal Komorebi import targets: komorebi_math.arithmetic.EulerLocalCoordinates, komorebi_math.arithmetic.HurwitzSeries, komorebi_math.arithmetic.MahlerSeries, komorebi_math.arithmetic.StandardWindow, komorebi_math.arithmetic.affine_kernel_composition_certificate, komorebi_math.arithmetic.fibonacci_recurrence, komorebi_math.arithmetic.prime_power_kernel_certificate, komorebi_math.arithmetic.simultaneous_unit_difference, komorebi_math.verify.core.VerificationCheck, komorebi_math.verify.core.VerificationReport, komorebi_math.verify.core._capture.

O-V005: src/komorebi_math/verify/beta.py

Source: lines 1 through 8. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: f1579f4095f62875db3c1e171ca44c2ce5937643d0d4dc905631202d5b65255b.

Literal Komorebi import targets: komorebi_math.release.check_beta_readiness, komorebi_math.release.check_stable_documentation, komorebi_math.release.check_upgrade_contract, komorebi_math.release.check_validation_evidence, komorebi_math.verify.core.VerificationReport, komorebi_math.verify.core._capture.

O-V006: src/komorebi_math/verify/coordinates.py

Source: lines 1 through 447. **Test definitions:** 0. **Assert statements:** 45.

File SHA-256: 82e37ae57211e3d6c6592fb9e84bb8b628092f05043035c2f46ac5c974556460.

Literal Komorebi import targets: komorebi_math.coordinates.BarycentricCoordinateSystem, komorebi_math.coordinates.CoordinateAtlas, komorebi_math.coordinates.CoordinateBudget, komorebi_math.coordinates.CoordinateSystem, komorebi_math.coordinates.CoordinateTransform, komorebi_math.coordinates.FormalCoordinateChange, komorebi_math.coordinates.LinearRealization, komorebi_math.coordinates.affine_basis_transform, komorebi_math.coordinates.

analyze_recurrence_coordinates, komorebi_math.coordinates.basis_from_matrix, komorebi_math.coordinates.bezout_defect_identity, komorebi_math.coordinates.binomial_transform, komorebi_math.coordinates.certify_state_coordinate_change, komorebi_math.coordinates.chinese_remainder_transform, komorebi_math.coordinates.companion_matrix, komorebi_math.coordinates.connection_transform, komorebi_math.coordinates.controllability_coordinate_transform, komorebi_math.coordinates.coordinate_change_between_recurrences, komorebi_math.coordinates.coordinate_pairing, komorebi_math.coordinates.derivative_coordinate_operator, komorebi_math.coordinates.derivative_operator_in_family, komorebi_math.coordinates.digit_reversal_transform, komorebi_math.coordinates.discrete_cosine_transform, komorebi_math.coordinates.discrete_fourier_transform, komorebi_math.coordinates.euler_fibonacci_coordinates, komorebi_math.coordinates.euler_polynomial_family, komorebi_math.coordinates.euler_to_fibonacci_transform, komorebi_math.coordinates.exponential_basis, komorebi_math.coordinates.falling_factorial_to_monomial_transform, komorebi_math.coordinates.fibonacci_factorial_basis, komorebi_math.coordinates.fibonacci_polynomial_family, komorebi_math.coordinates.finite_difference_coordinate_operator, komorebi_math.coordinates.forward_difference_transform, komorebi_math.coordinates.get_polynomial_family, komorebi_math.coordinates.get_recurrence_model, komorebi_math.coordinates.haar_wavelet_transform, komorebi_math.coordinates.intertwiner_space, komorebi_math.coordinates.inverse_matrix, komorebi_math.coordinates.load_coordinate_object, komorebi_math.coordinates.load_coordinate_record, komorebi_math.coordinates.load_transform, komorebi_math.coordinates.matrix_multiply, komorebi_math.coordinates.monomial_to_falling_factorial_transform, komorebi_math.coordinates.newton_evaluation_transform, komorebi_math.coordinates.normalization_change_transform, komorebi_math.coordinates.observability_coordinate_transform, komorebi_math.coordinates.operator_change_of_coordinates, komorebi_math.coordinates.pairing_invariant, komorebi_math.coordinates.recover_initial_state, komorebi_math.coordinates.recurrence_model_catalog, komorebi_math.coordinates.reduce_exact_transform_mod_prime, komorebi_math.coordinates.residue_class_transform, komorebi_math.coordinates.save_coordinate_object, komorebi_math.coordinates.save_coordinate_record, komorebi_math.coordinates.save_transform, komorebi_math.coordinates.sequence_coordinates_between_models, komorebi_math.coordinates.signed_secant_euler_numbers, komorebi_math.coordinates.siso_state_space, komorebi_math.coordinates.standard_basis, komorebi_math.coordinates.tensor_coordinate_transform, komorebi_math.coordinates.transport_operator, komorebi_math.coordinates.transport_polynomial_operator, komorebi_math.coordinates.transport_tensor_components, komorebi_math.coordinates.variation_of_constants_prefix, komorebi_math.coordinates.walsh_hadamard_transform, komorebi_math.exceptions.ResourceLimitError, komorebi_math.exceptions.ValidationError, komorebi_math.export.dumps_portable, komorebi_math.export.loads_portable, komorebi_math.verify.core.VerificationCheck, komorebi_math.verify.core.VerificationReport.

O-V007: `src/komorebi_math/verify/core.py`

Source: lines 1 through 144. **Test definitions:** 0. **Assert statements:** 7.

File SHA-256: `d673945ade70d3cba6cfb013f567cd6e535d20a5c0edfd6e76efe1447fdeeb83`.

Literal Komorebi import targets: komorebi_math.certificates.Certificate, komorebi_math.certificates.Exactness, komorebi_math.core.OperatorGrammar, komorebi_math.core.OperatorRealization, komorebi_math.core.PacketOperator, komorebi_math.core.PacketSystem, komorebi_math.core.SeedSystem, komorebi_math.core.TransferGraph, komorebi_math.core.block_from_paths.

O-V008: `src/komorebi_math/verify/deformation.py`

Source: lines 1 through 78. **Test definitions:** 0. **Assert statements:** 12.

File SHA-256: `15d2a260a18c34941f46f1b57f7a8cb32850ca693eabd40a1813f9918cdf474d`.

Literal Komorebi import targets: komorebi_math.deformation.FiniteCochainComplex, komorebi_math.deformation.FrobeniusSpine, komorebi_math.deformation.PresentedModule, komorebi_math.deformation.VisibilityTree, komorebi_math.deformation.finite_frobenius_separator, komorebi_math.deformation.normal_crossing_count, komorebi_math.deformation.normal_crossing_series, komorebi_math.deformation.residue_forest, komorebi_math.verify.core.VerificationCheck, komorebi_math.verify.core.VerificationReport, komorebi_math.verify.core._capture.

O-V009: `src/komorebi_math/verify/dsp.py`

Source: lines 1 through 172. **Test definitions:** 0. **Assert statements:** 6.

File SHA-256: `2d3f4c2575a507106ea380376a483db9362321046618bbfae72d49cc1a87e28f`.

Literal Komorebi import targets: `komorebi_math.signals.canonical_dual_window`, `komorebi_math.signals.certified_overlap_add_convolve`, `komorebi_math.signals.cross_spectrum`, `komorebi_math.signals.denoising_risk_certificate`, `komorebi_math.signals.design_biquad`, `komorebi_math.signals.design_lowpass`, `komorebi_math.signals.fir_filter`, `komorebi_math.signals.get_window`, `komorebi_math.signals.nlms_filter`, `komorebi_math.signals.overlap_add_certificate`, `komorebi_math.signals.parseval_certificate`, `komorebi_math.signals.periodogram`, `komorebi_math.signals.resample_poly_result`, `komorebi_math.signals.stft`, `komorebi_math.signals.stream_fir_filter`, `komorebi_math.signals.stream_lfilter`, `komorebi_math.signals.stream_stft`, `komorebi_math.signals.wavelet_denoise`, `komorebi_math.signals.welch_psd`, `komorebi_math.verify.core.VerificationCheck`, `komorebi_math.verify.core.VerificationReport`.

O-V010: `src/komorebi_math/verify/dynamics.py`

Source: lines 1 through 142. **Test definitions:** 0. **Assert statements:** 11.

File SHA-256: `e8f740da54c4c4be92512d15131e64e1c71a4855bfe08ce00ae6de63bfb84553`.

Literal Komorebi import targets: `komorebi_math.dynamics.BayesianUpdate`, `komorebi_math.dynamics.ConditionalExpectationTower`, `komorebi_math.dynamics.DirichletGreen1D`, `komorebi_math.dynamics.FeshbachReduction`, `komorebi_math.dynamics.FiniteFeynmanKac`, `komorebi_math.dynamics.HiddenMarkovFilter`, `komorebi_math.dynamics.MarkovLumping`, `komorebi_math.dynamics.MatrixSemigroup`, `komorebi_math.dynamics.PhaseTypeSurvival`, `komorebi_math.dynamics.fractional_power`, `komorebi_math.dynamics.matrix_exponential`, `komorebi_math.verify.core.VerificationCheck`, `komorebi_math.verify.core.VerificationReport`.

O-V011: `src/komorebi_math/verify/export.py`

Source: lines 1 through 118. **Test definitions:** 0. **Assert statements:** 11.

File SHA-256: `9d6757c71e0df993000b7acd8e9d4562dca0357044707f3ea6ec3bb5af29c787`.

Literal Komorebi import targets: `komorebi_math.export.DocumentMetadata`, `komorebi_math.export.ExportOptions`, `komorebi_math.export.MathDocument`, `komorebi_math.export.available_latex_engines`, `komorebi_math.export.compile_latex`, `komorebi_math.export.export_bundle`, `komorebi_math.export.export_data`, `komorebi_math.export.latex_escape`, `komorebi_math.export.render_latex`, `komorebi_math.export.to_serializable`, `komorebi_math.verify.core.VerificationCheck`, `komorebi_math.verify.core.VerificationReport`.

O-V012: `src/komorebi_math/verify/expression.py`

Source: lines 1 through 559. **Test definitions:** 0. **Assert statements:** 56.

File SHA-256: `fee8884d61def5d82cf0bb650329c710de81a8ca346737672950ebf99139b29e`.

Literal Komorebi import targets: `komorebi_math.certificates.Certificate`, `komorebi_math.certificates.Exactness`, `komorebi_math.coordinates.PolynomialFamily`, `komorebi_math.coordinates.euler_polynomial_family`, `komorebi_math.coordinates.exact.identity_matrix`, `komorebi_math.coordinates.exact.matrix_multiply`, `komorebi_math.coordinates.exact.matrix_vector_multiply`, `komorebi_math.coordinates.fibonacci_polynomial_family`, `komorebi_math.coordinates.monomial_polynomial_family`, `komorebi_math.exceptions.DimensionError`, `komorebi_math.exceptions.ResourceLimitError`, `komorebi_math.exceptions.ValidationError`, `komorebi_math.expression.Expression`, `komorebi_math.expression.ExpressionBinding`, `komorebi_math.expression.ExpressionBudget`, `komorebi_math.expression.LinearMapType`, `komorebi_math.expression.PolynomialCoordinateSpace`, `komorebi_math.expression.PolynomialPrimitive`, `komorebi_math.expression.PrimitiveKind`, `komorebi_math.expression.RATIONAL_SPACE`, `komorebi_math.expression.RationalSpace`, `komorebi_math.expression.add`, `komorebi_math.expression.affine_substitution`, `komorebi_math.expression.antiderivative`, `komorebi_math.expression.apply`, `komorebi_math.expression.boundary_functional`, `komorebi_math.expression.check_expression`, `komorebi_math.expression.compile_expression`, `komorebi_math.expression.compose`, `komorebi_math.expression.constant_injection`, `komorebi_math.expression.coordinate_change`, `komorebi_math.expression.derivative`, `komorebi_math.expression.equivalent`, `komorebi_math.expression.evaluate_expression`, `komorebi_math.expression.evaluation`, `komorebi_math.expression.exact_map`, `komorebi_math.expression.exact_value`, `komorebi_math.expression.execute_plan`, `komorebi_math.expression.expression_digest`, `komorebi_math.expression.expression_from_dict`, `komorebi_`

math.expression.expression_from_json, komorebi_math.expression.expression_to_dict, komorebi_math.expression.expression_to_json, komorebi_math.expression.finite_difference, komorebi_math.expression.identity, komorebi_math.expression.interval_integration, komorebi_math.expression.let, komorebi_math.expression.matrix_chain_multiply, komorebi_math.expression.matrix_chain_parenthesization, komorebi_math.expression.matrix_chain_plan, komorebi_math.expression.normalize_expression, komorebi_math.expression.primitive_certificate, komorebi_math.expression.replay, komorebi_math.expression.scale, komorebi_math.expression.simplify_expression, komorebi_math.expression.symbol, komorebi_math.expression.then, komorebi_math.verify.core.VerificationCheck, komorebi_math.verify.core.VerificationReport, komorebi_math.verify.core._capture.

O-V013: src/komorebi_math/verify/frame_algebra.py

Source: lines 1 through 146. **Test definitions:** 0. **Assert statements:** 11.

File SHA-256: 08c89e1d444c14a17ca2894064e3b6d5b205725509a91017c821e58b6717ba2d.

Literal Komorebi import targets: komorebi_math.coordinates.AnalysisSynthesisGauge, komorebi_math.coordinates.AnalysisSynthesisMap, komorebi_math.coordinates.AnalysisSynthesisSystem, komorebi_math.coordinates.CoefficientPartition, komorebi_math.coordinates.ExactAnalysisSynthesisMap, komorebi_math.coordinates.ExactAnalysisSynthesisSystem, komorebi_math.coordinates.exact.matrix_multiply, komorebi_math.exceptions.ValidationError, komorebi_math.verify.core.VerificationCheck, komorebi_math.verify.core.VerificationReport, komorebi_math.verify.core._capture.

O-V014: src/komorebi_math/verify/generating.py

Source: lines 1 through 199. **Test definitions:** 0. **Assert statements:** 13.

File SHA-256: d02a650e9c6271115faf19767c220227f47c13f0e19588dc8a3981c0806301cd.

Literal Komorebi import targets: komorebi_math.export.dumps_portable, komorebi_math.export.loads_portable, komorebi_math.generating.BellParityAutomaton, komorebi_math.generating.FiniteNormalizedSeries, komorebi_math.generating.FormalSeries, komorebi_math.generating.PrimeFieldDomain, komorebi_math.generating.PrimePowerUnitCompiler, komorebi_math.generating.RationalDomain, komorebi_math.generating.analytic_mahler_product, komorebi_math.generating.certify_mahler_functional_equation, komorebi_math.generating.certify_rational_mask_algebraic_relation, komorebi_math.generating.compile_cartier_linear_representation, komorebi_math.generating.compile_finite_mask_carry_plan, komorebi_math.generating.euler_product_collapse, komorebi_math.generating.exponential_basis, komorebi_math.generating.factorial_power_basis, komorebi_math.generating.factorial_power_support_count, komorebi_math.generating.fir_taps_to_series, komorebi_math.generating.generalized_bell_numbers, komorebi_math.generating.kummer_layer_distribution, komorebi_math.generating.mahler_mask_inverse, komorebi_math.generating.mahler_mask_product, komorebi_math.generating.q_binomial_valuation_decomposition, komorebi_math.generating.q_binomial_valuation_direct, komorebi_math.generating.rational_mask_coefficients_mod_prime, komorebi_math.generating.series_to_fir_adapter, komorebi_math.generating.solve_polynomial_coboundary, komorebi_math.verify.core.VerificationCheck, komorebi_math.verify.core.VerificationReport.

O-V015: src/komorebi_math/verify/gpu.py

Source: lines 1 through 42. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: e5234f735f8cfc2a9a7580a582973aec832b7a6fc40878117f338192c9b8d096.

Literal Komorebi import targets: komorebi_math.qualification.gpu_diagnostics, komorebi_math.verify.core.VerificationCheck, komorebi_math.verify.core.VerificationReport.

O-V016: src/komorebi_math/verify/hardening.py

Source: lines 1 through 5. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: b6e7b09f6c7c79a3b9227a9d9073a273b4ceb250908e5cf0f2be43726d6edda2.

Literal Komorebi import targets: komorebi_math.verify.scalable_nonlinear.run_scalable_nonlinear_verification.

O-V017: `src/komorebi_math/verify/integration.py`

Source: lines 1 through 255. **Test definitions:** 0. **Assert statements:** 24.

File SHA-256: `3b82f52f14aa45a1912ac17a6655ae9df26c90998a17aa216cc31a3f267fc8e5`.

Literal Komorebi import targets: `komorebi_math.certificates.Exactness`, `komorebi_math.coordinates.AnalysisSynthesisSystem`, `komorebi_math.coordinates.euler_polynomial_family`, `komorebi_math.coordinates.fibonacci_polynomial_family`, `komorebi_math.coordinates.legendre_polynomial_family`, `komorebi_math.coordinates.monomial_polynomial_family`, `komorebi_math.exceptions.ValidationError`, `komorebi_math.integration.AtomicIntegratedDerivativeProvider`, `komorebi_math.integration.AtomicMeasure`, `komorebi_math.integration.CompositeSimpson`, `komorebi_math.integration.ErrorGuarantee`, `komorebi_math.integration.ExactAffineMap`, `komorebi_math.integration.ExactPolynomial`, `komorebi_math.integration.ExactQuadratureRule`, `komorebi_math.integration.IntervalMeasure`, `komorebi_math.integration.ProductMeasure`, `komorebi_math.integration.PushforwardMeasure`, `komorebi_math.integration.affine_interval_change_of_variables`, `komorebi_math.integration.all_fubini_orders`, `komorebi_math.integration.analysis_synthesis_integration_certificate`, `komorebi_math.integration.coordinate_fundamental_theorem`, `komorebi_math.integration.coordinate_transport_identity`, `komorebi_math.integration.fundamental_theorem`, `komorebi_math.integration.integrate`, `komorebi_math.integration.integrate_polynomial_oriented`, `komorebi_math.integration.integration_by_parts`, `komorebi_math.integration.polynomial_derivative_bound`, `komorebi_math.verify.core.VerificationCheck`, `komorebi_math.verify.core.VerificationReport`, `komorebi_math.verify.core._capture`.

O-V018: `src/komorebi_math/verify/langlands.py`

Source: lines 1 through 168. **Test definitions:** 0. **Assert statements:** 10.

File SHA-256: `b2a1616fec75cf0d7f39df0ab8ad12096593f6cbd7395712a0ff5fe8374d50e2`.

Literal Komorebi import targets: `komorebi_math.langlands.completion.AutomorphicCompletionReport`, `komorebi_math.langlands.completion.CompletionGate`, `komorebi_math.langlands.completion.GateStatus`, `komorebi_math.langlands.finite_image.CliffordStep`, `komorebi_math.langlands.finite_image.CliffordTower`, `komorebi_math.langlands.finite_image.InducedRepresentation`, `komorebi_math.langlands.finite_image.LinearCharacter`, `komorebi_math.langlands.finite_image.MonomialAction`, `komorebi_math.langlands.finite_image.cyclic_group`, `komorebi_math.langlands.finite_image.mackey_tensor_packets`, `komorebi_math.langlands.global_.FiniteHeckePackets`, `komorebi_math.langlands.global_.PatchingSupportReport`, `komorebi_math.langlands.global_.dyadic_sign_diagnostic`, `komorebi_math.langlands.local.AlternatingForm`, `komorebi_math.langlands.local.FiniteField`, `komorebi_math.langlands.local.FrobeniusOrbit`, `komorebi_math.langlands.local.HeisenbergModel`, `komorebi_math.langlands.local.ProjectiveIntertwinerEnvelope`, `komorebi_math.langlands.local.QuadraticRefinement`, `komorebi_math.langlands.local.TameWeilPacket`, `komorebi_math.langlands.local.gauss_sum`, `komorebi_math.langlands.ramification.ProjectiveMultiplicityPacket`, `komorebi_math.langlands.ramification.RamificationCliffordTower`, `komorebi_math.langlands.ramification.RamificationStage`, `komorebi_math.verify.core.VerificationCheck`, `komorebi_math.verify.core.VerificationReport`, `komorebi_math.verify.core._capture`.

O-V019: `src/komorebi_math/verify/nonlinear.py`

Source: lines 1 through 113. **Test definitions:** 0. **Assert statements:** 5.

File SHA-256: `d829cec4bd763d28cc5ff0b1d4608327ba5106537793d771518d2b0a37c66215`.

Literal Komorebi import targets: `komorebi_math.core.PacketSystem`, `komorebi_math.nonlinear.LinearLiftProblem`, `komorebi_math.nonlinear.LinearObstructionProblem`, `komorebi_math.nonlinear.NonlinearPacketMap`, `komorebi_math.nonlinear.NonlinearRefinementDefect`, `komorebi_math.nonlinear.PolynomialLiftProblem`, `komorebi_math.nonlinear.TransportEdge`, `komorebi_math.nonlinear.TransportGraph`, `komorebi_math.verify.core.VerificationCheck`, `komorebi_math.verify.core.VerificationReport`, `komorebi_math.verify.core._capture`, `komorebi_math.verify.nonlinear_advanced.run_advanced_nonlinear_verification`.

O-V020: `src/komorebi_math/verify/nonlinear_advanced.py`

Source: lines 1 through 231. **Test definitions:** 0. **Assert statements:** 15.

File SHA-256: `cc889679abf3565940d335541be90d230c01d91e880b45bf0863b50acad63ae7`.

Literal Komorebi import targets: `komorebi_math.certificates.Exactness`, `komorebi_math.core.PacketSystem`, `komorebi_math.dynamics._linalg.matrix_exponential`, `komorebi_math.nonlinear.CallableDerivativeProvider`, `komorebi_math.nonlinear.FilteredCorrectionProblem`, `komorebi_math.nonlinear.FilteredVectorSpace`, `komorebi_math.nonlinear.FiniteAbelianGroup`, `komorebi_math.nonlinear.FiniteFilteredCochainComplex`, `komorebi_math.nonlinear.HilbertObstructionProjector`, `komorebi_math.nonlinear.KatoTransport`, `komorebi_math.nonlinear.LyapunovSchmidtReduction`, `komorebi_math.nonlinear.MovingPacketSystem`, `komorebi_math.nonlinear.NonlinearSchurElimination`, `komorebi_math.nonlinear.PacketJetAddress`, `komorebi_math.nonlinear.PacketJetFamily`, `komorebi_math.nonlinear.PolynomialDerivativeProvider`, `komorebi_math.nonlinear.PolynomialFixedPointProblem`, `komorebi_math.nonlinear.divided_square_packets`, `komorebi_math.nonlinear.faa_di_bruno`, `komorebi_math.nonlinear.majorant_convergence`, `komorebi_math.nonlinear.packet_jet_support`, `komorebi_math.nonlinear.packet_taylor`, `komorebi_math.nonlinear.set_partitions`, `komorebi_math.nonlinear.verify_finite_filtered_branch`, `komorebi_math.verify.core.VerificationCheck`, `komorebi_math.verify.core.VerificationReport`, `komorebi_math.verify.core._capture`.

O-V021: `src/komorebi_math/verify/quantum.py`

Source: lines 1 through 150. **Test definitions:** 0. **Assert statements:** 7.

File SHA-256: `8af5b89ad28093b64450a747f46897b8cc79ca819d4ba80ca8961cef4c8db813`.

Literal Komorebi import targets: `komorebi_math.dynamics.matrix_exponential`, `komorebi_math.quantum.BoundedHamiltonianTower`, `komorebi_math.quantum.CircuitPacketExpansion`, `komorebi_math.quantum.EuclideanTransfer`, `komorebi_math.quantum.LindbladGenerator`, `komorebi_math.quantum.MovingPacketSystem`, `komorebi_math.quantum.QuantumCode`, `komorebi_math.quantum.QuantumFeshbachReduction`, `komorebi_math.quantum.amplitude_damping`, `komorebi_math.quantum.computational_basis_generation`, `komorebi_math.quantum.constraint_closure`, `komorebi_math.quantum.cyclic_symmetry_projectors`, `komorebi_math.quantum.direct_packet_rotation`, `komorebi_math.quantum.fixed_packet_leakage`, `komorebi_math.quantum.local_orbit_dimension`, `komorebi_math.quantum.periodic_laplacian`, `komorebi_math.quantum.periodic_lattice_eigenvalues`, `komorebi_math.quantum.spectral_dimension`, `komorebi_math.verify.core.VerificationCheck`, `komorebi_math.verify.core.VerificationReport`, `komorebi_math.verify.core._capture`.

O-V022: `src/komorebi_math/verify/refinement.py`

Source: lines 1 through 88. **Test definitions:** 0. **Assert statements:** 5.

File SHA-256: `e008926e1df92a77310e3dd921c6bcb0423237435e40b7c113f3a62d473fe82d`.

Literal Komorebi import targets: `komorebi_math.core.PacketSystem`, `komorebi_math.refinement.LatticePacketData`, `komorebi_math.refinement.LinearRefinementDefect`, `komorebi_math.refinement.PacketRefinement`, `komorebi_math.refinement.SmithCongruenceProblem`, `komorebi_math.refinement.WeightedTransferEdge`, `komorebi_math.refinement.WeightedTransferGraph`, `komorebi_math.refinement.crt_pair`, `komorebi_math.refinement.nested_residue_analysis`, `komorebi_math.refinement.residue_analyze`, `komorebi_math.verify.core.VerificationCheck`, `komorebi_math.verify.core.VerificationReport`, `komorebi_math.verify.core._capture`.

O-V023: `src/komorebi_math/verify/release.py`

Source: lines 1 through 96. **Test definitions:** 0. **Assert statements:** 21.

File SHA-256: `d07309fe5666b04bec15a9519a3b053c4cc8ce3b4bc2cb39d2c18946a49b902c`.

Literal Komorebi import targets: `komorebi_math.__version__`, `komorebi_math.compatibility.create_beta_api_manifest`, `komorebi_math.compatibility.create_public_api_manifest`, `komorebi_math.compatibility.create_stable_api_manifest`, `komorebi_math.persistence_contract.load_portable_codec_contract`, `komorebi_math.release.check_release_contract`, `komorebi_math.release.check_stable_documentation`, `komorebi_math.release.check_upgrade_contract`, `komorebi_math.release.evidence._check_historical_validation_evidence`, `komorebi_math.stable`, `komorebi_math.stable_semantics.load_stable_semantics_fixture`, `komorebi_math.support.load_support_matrix`, `komorebi_math.support.runtime_environment`, `komorebi_math.verify.core.VerificationCheck`, `komorebi_math.verify.core.VerificationReport`, `komorebi_math.verify.core._capture`.

O-V024: `src/komorebi_math/verify/scalable_nonlinear.py`

Source: lines 1 through 159. **Test definitions:** 0. **Assert statements:** 14.

File SHA-256: `5fcecfd53307b1734bd5b009466b3410b49cffab91fea02c072475a5a068aa27`.

Literal Komorebi import targets: `komorebi_math.certificates.Exactness`, `komorebi_math.compatibility.check_public_api`, `komorebi_math.core.PacketSystem`, `komorebi_math.exceptions.ValidationError`, `komorebi_math.export.dumps_portable`, `komorebi_math.export.loads_portable`, `komorebi_math.nonlinear.AutomaticDifferentiationProvider`, `komorebi_math.nonlinear.PacketJetFamily`, `komorebi_math.nonlinear.PolynomialDerivativeProvider`, `komorebi_math.nonlinear.SparsePolynomialDerivativeProvider`, `komorebi_math.nonlinear.SparseSymmetricTensor`, `komorebi_math.nonlinear.compile_packet_jet_plan`, `komorebi_math.nonlinear.cupy_runtime_status`, `komorebi_math.nonlinear.execute_packet_jet_plan`, `komorebi_math.nonlinear.packet_jet_support`, `komorebi_math.nonlinear.snapshot_packet_jet`, `komorebi_math.verify.core.VerificationCheck`, `komorebi_math.verify.core.VerificationReport`, `komorebi_math.verify.core._capture`.

O-V025: `src/komorebi_math/verify/sequences.py`

Source: lines 1 through 77. **Test definitions:** 0. **Assert statements:** 12.

File SHA-256: `4f7ed714444193082285649e1aa90b26a8b168e99a06a38591bba3bd92b20027`.

Literal Komorebi import targets: `komorebi_math.sequences.apery_recurrence`, `komorebi_math.sequences.apery_sequence`, `komorebi_math.sequences.bell_sequence`, `komorebi_math.sequences.binomial_transform_values`, `komorebi_math.sequences.catalan_sequence`, `komorebi_math.sequences.compile_sequence`, `komorebi_math.sequences.fibonacci_sequence`, `komorebi_math.sequences.inverse_binomial_transform_values`, `komorebi_math.sequences.motzkin_recurrence`, `komorebi_math.sequences.partition_sequence`, `komorebi_math.sequences.thue_morse_automaton`, `komorebi_math.sequences.thue_morse_sequence`, `komorebi_math.verify.core.VerificationCheck`, `komorebi_math.verify.core.VerificationReport`, `komorebi_math.verify.core._capture`.

O-V026: `src/komorebi_math/verify/signals.py`

Source: lines 1 through 141. **Test definitions:** 0. **Assert statements:** 8.

File SHA-256: `c981e9878071c41aed43742626d89b111e6211283cb7da5215520b8bee59d8d5`.

Literal Komorebi import targets: `komorebi_math.signals.HaarPacketTree`, `komorebi_math.signals.PeriodicFilterBank`, `komorebi_math.signals.SingularPacket1D`, `komorebi_math.signals.lattice_polyphase_analyze`, `komorebi_math.signals.packet_product`, `komorebi_math.signals.polyphase_analyze`, `komorebi_math.signals.polyphase_filter_kernels`, `komorebi_math.signals.recover_separated_jumps`, `komorebi_math.signals.retained_packet_reconstruction`, `komorebi_math.verify.core.VerificationCheck`, `komorebi_math.verify.core.VerificationReport`.

O-V027: `src/komorebi_math/verify/stabilization.py`

Source: lines 1 through 175. **Test definitions:** 0. **Assert statements:** 20.

File SHA-256: `d2588c622a594c833ac92680f2121788dbbd5c72d0dbdfc0167534c89a80a1f4`.

Literal Komorebi import targets: `komorebi_math.api.v1`, `komorebi_math.arithmetic.polyphase_analyze`, `komorebi_math.compatibility.check_stable_api`, `komorebi_math.compatibility.create_stable_api_manifest`, `komorebi_math.compatibility.load_stable_api_manifest`, `komorebi_math.core.PacketSystem`, `komorebi_math.export.persistence.dumps_portable`, `komorebi_math.export.persistence.loads_portable`, `komorebi_math.persistence_contract.check_portable_codec_contract`, `komorebi_math.refinement.LatticePolyphaseRefinement`, `komorebi_math.refinement.PacketRefinement`, `komorebi_math.refinement.PolyphaseRefinement`, `komorebi_math.refinement.PolyphaseRefinementTower`, `komorebi_math.signals.lattice_polyphase_analyze`, `komorebi_math.signals.polyphase_analyze`, `komorebi_math.stable`, `komorebi_math.stable_contract.check_stable_contract`, `komorebi_math.stable_semantics.run_stable_semantics_fixture`, `komorebi_math.v1`, `komorebi_math.verify.core.VerificationCheck`, `komorebi_math.verify.core.VerificationReport`, `komorebi_math.verify.core._capture`.

O-V028: `src/komorebi_math/verify/streaming_beta.py`

Source: lines 1 through 191. **Test definitions:** 0. **Assert statements:** 16.

File SHA-256: `5382cb363d211209440cece1a41ef6a20ee7ada207b98e740a519a4fe0489ce3`.

Literal Komorebi import targets: `komorebi_math.compatibility.check_beta_api`, `komorebi_math.compatibility.check_public_api`, `komorebi_math.core.PacketSystem`, `komorebi_math.exceptions.ValidationError`, `komorebi_math.export.dumps_portable`, `komorebi_math.export.from_portable_data`, `komorebi_math.export.loads_portable`, `komorebi_math.export.to_portable_data`, `komorebi_math.golden.run_behavioral_golden_fixture`, `komorebi_math.nonlinear.PacketJetFamily`, `komorebi_math.nonlinear.PacketJetSnapshot`, `komorebi_math.nonlinear.PolynomialDerivativeProvider`, `komorebi_math.nonlinear.capture_packet_jet_stream`, `komorebi_math.nonlinear.estimate_packet_jet_complexity`, `komorebi_math.nonlinear.open_packet_jet_snapshot_stream`, `komorebi_math.nonlinear.packet_jet_support`, `komorebi_math.nonlinear.profile_operation`, `komorebi_math.nonlinear.save_packet_jet_snapshot_stream`, `komorebi_math.nonlinear.snapshot_packet_jet`, `komorebi_math.verify.core.VerificationCheck`, `komorebi_math.verify.core.VerificationReport`, `komorebi_math.verify.core._capture`.

O.6 Repository verification-program index

O-R001: `verification/block10_deep_audit.py`

Source: lines 1 through 522. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `78e8ae87aeecd2fe69291be7a9c1bdcbb5133a551bed022da7ad2a277aa3af20`.

Literal Komorebi import targets: `komorebi_math`, `komorebi_math.Exactness`, `komorebi_math.PacketSystem`, `komorebi_math.compatibility.check_public_api`, `komorebi_math.compatibility.create_public_api_manifest`, `komorebi_math.compatibility.load_public_api_manifest`, `komorebi_math.exceptions.BackendUnavailableError`, `komorebi_math.exceptions.ValidationError`, `komorebi_math.export.dumps_portable`, `komorebi_math.export.loads_portable`, `komorebi_math.export.portable_digest`, `komorebi_math.nonlinear.AutomaticDifferentiationProvider`, `komorebi_math.nonlinear.CuPyPolynomialDerivativeProvider`, `komorebi_math.nonlinear.JaxDerivativeProvider`, `komorebi_math.nonlinear.PacketJetFamily`, `komorebi_math.nonlinear.PolynomialDerivativeProvider`, `komorebi_math.nonlinear.SparsePolynomialDerivativeProvider`, `komorebi_math.nonlinear.SparseSymmetricTensor`, `komorebi_math.nonlinear.TorchDerivativeProvider`, `komorebi_math.nonlinear.available_ad_frameworks`, `komorebi_math.nonlinear.compile_packet_jet_plan`, `komorebi_math.nonlinear.cupy_runtime_status`, `komorebi_math.nonlinear.execute_packet_jet_plan`, `komorebi_math.nonlinear.nonlinear_cupy_available`, `komorebi_math.nonlinear.packet_jet_support`, `komorebi_math.nonlinear.snapshot_packet_jet`.

O-R002: `verification/block11_deep_audit.py`

Source: lines 1 through 525. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `359125cf08bc75a1bd19d39f8a998e2021551c42d1a679ccb3d3938834198f29`.

Literal Komorebi import targets: `komorebi_math`, `komorebi_math.Exactness`, `komorebi_math.PacketSystem`, `komorebi_math.backends.numpy.NumpyBackend`, `komorebi_math.compatibility.check_beta_api`, `komorebi_math.compatibility.check_public_api`, `komorebi_math.compatibility.create_beta_api_manifest`, `komorebi_math.compatibility.create_public_api_manifest`, `komorebi_math.compatibility.load_beta_api_manifest`, `komorebi_math.compatibility.load_public_api_manifest`, `komorebi_math.exceptions.ValidationError`, `komorebi_math.export.dumps_portable`, `komorebi_math.export.from_portable_data`, `komorebi_math.export.loads_portable`, `komorebi_math.export.to_portable_data`, `komorebi_math.golden.run_behavioral_golden_fixture`, `komorebi_math.nonlinear.PacketJetFamily`, `komorebi_math.nonlinear.PolynomialDerivativeProvider`, `komorebi_math.nonlinear.capture_packet_jet_stream`, `komorebi_math.nonlinear.cupy_runtime_status`, `komorebi_math.nonlinear.estimate_packet_jet_complexity`, `komorebi_math.nonlinear.open_packet_jet_snapshot_stream`, `komorebi_math.nonlinear.packet_jet_support`, `komorebi_math.nonlinear.profile_operation`, `komorebi_math.nonlinear.save_packet_jet_snapshot_stream`, `komorebi_math.nonlinear.snapshot_packet_jet`.

O-R003: `verification/block11_device_validation.py`

Source: lines 1 through 122. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `904978ff14722d884a42bd6fc7985e8084653921eb9c0f4678608b64deafe3fe`.

Literal Komorebi import targets: `komorebi_math`, `komorebi_math.PacketSystem`, `komorebi_math.nonlinear.PacketJetFamily`, `komorebi_math.nonlinear.PolynomialDerivativeProvider`, `komorebi_math.nonlinear.cupy_runtime_status`, `komorebi_math.nonlinear.snapshot_packet_jet`, `komorebi_math.nonlinear.validate_cupy_snapshot`.

O-R004: `verification/block11_environment_audit.py`

Source: lines 1 through 129. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `579b95ccdd769f86057c50e02ff45ac49a2267ea2694411a2fddc7f6c933425f`.

Literal Komorebi import targets: `komorebi_math`, `komorebi_math.nonlinear.cupy_runtime_status`.

O-R005: `verification/block12_deep_audit.py`

Source: lines 1 through 500. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `95cfa538657370620319c017a6f8f8eb260fdfaa3e38403b833a092838cd444a`.

Literal Komorebi import targets: `komorebi_math.__version__`, `komorebi_math.signals.LinearFilter`, `komorebi_math.signals.OverlapAddConvolver`, `komorebi_math.signals.canonical_dual_window`, `komorebi_math.signals.certified_overlap_add_convolve`, `komorebi_math.signals.cross_spectrum`, `komorebi_math.signals.denoising_risk_certificate`, `komorebi_math.signals.design_bandpass`, `komorebi_math.signals.design_biquad`, `komorebi_math.signals.design_highpass`, `komorebi_math.signals.design_lowpass`, `komorebi_math.signals.fft_convolve`, `komorebi_math.signals.fir_filter`, `komorebi_math.signals.get_window`, `komorebi_math.signals.haar_decompose`, `komorebi_math.signals.nlms_filter`, `komorebi_math.signals.overlap_add_certificate`, `komorebi_math.signals.parseval_certificate`, `komorebi_math.signals.periodogram`, `komorebi_math.signals.resample_poly_result`, `komorebi_math.signals.rls_filter`, `komorebi_math.signals.stft`, `komorebi_math.signals.stream_fir_filter`, `komorebi_math.signals.stream_lfilter`, `komorebi_math.signals.stream_stft`, `komorebi_math.signals.threshold_coefficients`, `komorebi_math.signals.wavelet_denoise`, `komorebi_math.signals.welch_psd`.

O-R006: `verification/block12_scipy_reference_audit.py`

Source: lines 1 through 306. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `c59d569b4a2f28cbf9386d64f381bd93bdf76352c3a7736ecc3315695a08cdd6`.

Literal Komorebi import targets: `komorebi_math.__version__`, `komorebi_math.signals.design_bandpass`, `komorebi_math.signals.design_bandstop`, `komorebi_math.signals.design_highpass`, `komorebi_math.signals.design_lowpass`, `komorebi_math.signals.design_resample_filter`, `komorebi_math.signals.fft_convolve`, `komorebi_math.signals.get_window`, `komorebi_math.signals.lfilter`, `komorebi_math.signals.overlap_add_convolve`, `komorebi_math.signals.periodogram`, `komorebi_math.signals.resample_poly`, `komorebi_math.signals.welch_psd`.

O-R007: `verification/block13_deep_audit.py`

Source: lines 1 through 327. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `02e1af271b540e9420809cf24d3991d9c335a30421f21a6b5dc0bdd6f185b139`.

Literal Komorebi import targets: `komorebi_math.__version__`, `komorebi_math.export.dumps_portable`, `komorebi_math.export.loads_portable`, `komorebi_math.generating.BellParityAutomaton`, `komorebi_math.generating.BellResidueAutomaton`, `komorebi_math.generating.FiniteNormalizedSeries`, `komorebi_math.generating.FormalSeries`, `komorebi_math.generating.GeneratingBudget`, `komorebi_math.generating.PrimeFieldDomain`, `komorebi_math.generating.PrimePowerKernelCompiler`, `komorebi_math.generating.PrimePowerUnitCompiler`, `komorebi_math.generating.RationalDomain`, `komorebi_math.generating.analytic_mahler_product`, `komorebi_math.generating.bell_numbers`, `komorebi_math.generating.certify_cartier_carry_equivalence`, `komorebi_math.generating.compile_cartier_linear_representation`, `komorebi_math.generating.compile_digit_cauchy_product`, `komorebi_math.generating.compile_finite_mask_carry_plan`, `komorebi_math.generating.compile_rational_mask_automaton`, `komorebi_math.generating.exponential_basis`, `komorebi_math.generating.factorial_power_basis`, `komorebi_math.generating.factorial_power_support_indices`, `komorebi_math.`

generating.fir_taps_to_series, komorebi_math.generating.generalized_bell_numbers, komorebi_math.generating.kummer_layer_distribution, komorebi_math.generating.mahler_mask_inverse, komorebi_math.generating.mahler_mask_product, komorebi_math.generating.ordinary_basis, komorebi_math.generating.padic_units.borrow_count, komorebi_math.generating.q_binomial_valuation_decomposition, komorebi_math.generating.q_binomial_valuation_direct, komorebi_math.generating.rational_mask_coefficients_mod_prime, komorebi_math.generating.series_to_fir_adapter.

O-R008: verification/block14_deep_audit.py

Source: lines 1 through 190. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: 7e37700d6a77896343c9916ec5d6d876362dae8d26b04d94fdf4ca297c6885a3.

Literal Komorebi import targets: komorebi_math.coordinates.

O-R009: verification/block14_general_coordinate_audit.py

Source: lines 1 through 239. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: a8524fb77cd310279f58412292283af04b3038def9068a3e935f0bbb65026fb0.

Literal Komorebi import targets: komorebi_math, komorebi_math.coordinates.

O-R010: verification/block14_reference_audit.py

Source: lines 1 through 257. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: aee311f0705747a9180e1bdda9d8d0f8bb1da07c0b3067f8434490ada42922f5.

Literal Komorebi import targets: komorebi_math.coordinates.LinearRealization, komorebi_math.coordinates.analyze_recurrence_coordinates, komorebi_math.coordinates.bezout_defect_identity, komorebi_math.coordinates.companion_matrix, komorebi_math.coordinates.connection_transform, komorebi_math.coordinates.convert_normalized_coefficients, komorebi_math.coordinates.coordinate_change_between_recurrences, komorebi_math.coordinates.cyclic_krylov_matrix, komorebi_math.coordinates.derivative_operator_in_family, komorebi_math.coordinates.euler_fibonacci_coordinates, komorebi_math.coordinates.euler_polynomial, komorebi_math.coordinates.euler_polynomial_family, komorebi_math.coordinates.euler_to_fibonacci_coefficient, komorebi_math.coordinates.euler_to_fibonacci_transform, komorebi_math.coordinates.exponential_basis, komorebi_math.coordinates.fibonacci_companion_power, komorebi_math.coordinates.fibonacci_factorial_basis, komorebi_math.coordinates.fibonacci_polynomial, komorebi_math.coordinates.fibonacci_polynomial_family, komorebi_math.coordinates.identity_matrix, komorebi_math.coordinates.intertwiner_space, komorebi_math.coordinates.inverse_matrix, komorebi_math.coordinates.matrix_multiply, komorebi_math.coordinates.matrix_vector_multiply, komorebi_math.coordinates.monomial_polynomial_family, komorebi_math.coordinates.monomial_to_fibonacci_coefficient, komorebi_math.coordinates.normalization_change_transform, komorebi_math.coordinates.operator_change_of_coordinates, komorebi_math.coordinates.polynomial_add, komorebi_math.coordinates.polynomial_extended_gcd, komorebi_math.coordinates.polynomial_multiply, komorebi_math.coordinates.polynomial_scale, komorebi_math.coordinates.realization_defect, komorebi_math.coordinates.recover_initial_state, komorebi_math.coordinates.secant_euler_from_fibonacci_coordinates, komorebi_math.coordinates.signed_secant_euler_numbers, komorebi_math.coordinates.similarity_from_cyclic_vectors, komorebi_math.coordinates.variation_of_constants_prefix.

O-R011: verification/block15_refinement_audit.py

Source: lines 1 through 140. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: d305a13431ebf054eb95558b6927dd32559997b2a2076483f3d871772211a3f3.

Literal Komorebi import targets: komorebi_math.arithmetic.polyphase_analyze, komorebi_math.refinement.LatticePolyphaseRefinement, komorebi_math.refinement.PolyphaseRefinement, komorebi_math.refinement.PolyphaseRefinementTower, komorebi_math.refinement.residue_analyze, komorebi_math.signals.lattice_polyphase_analyze, komorebi_math.signals.polyphase_analyze.

O-R012: `verification/block16_distribution_audit.py`

Source: lines 1 through 222. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `d556f96ebcdf9c63c1523f321cdead19727c194d62cb94ef367f7783e7b611ed`.

Literal Komorebi import targets: none. This syntax result does not rule out subprocess, helper-mediated or dynamic package use.

O-R013: `verification/block16_package_audit.py`

Source: lines 1 through 299. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `b5d5d3eab7fbca55c593c489275344b53bd625b7d472ab1c8db695421d6e1fb7`.

Literal Komorebi import targets: `komorebi_math`, `komorebi_math.api.v1`, `komorebi_math.stable`, `komorebi_math.stable_contract.check_stable_contract`, `komorebi_math.v1`.

O-R014: `verification/block16_static_audit.py`

Source: lines 1 through 10. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `bac060588c993583e6c6e885898ea7275e544f8666649d666b6f6861a466cdfe`.

Literal Komorebi import targets: none. This syntax result does not rule out subprocess, helper-mediated or dynamic package use.

O-R015: `verification/block17_distribution_audit.py`

Source: lines 1 through 244. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `6e686792c19a89bb1b5d05f51315a686d31b2e17195d446479ab1db79763c966`.

Literal Komorebi import targets: none. This syntax result does not rule out subprocess, helper-mediated or dynamic package use.

O-R016: `verification/block17_package_audit.py`

Source: lines 1 through 8. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `f75939ac91342685e84ab879e73fb8c80105986f00889a5d8f9364ddca62756f`.

Literal Komorebi import targets: none. This syntax result does not rule out subprocess, helper-mediated or dynamic package use.

O-R017: `verification/block17_static_audit.py`

Source: lines 1 through 10. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `bac060588c993583e6c6e885898ea7275e544f8666649d666b6f6861a466cdfe`.

Literal Komorebi import targets: none. This syntax result does not rule out subprocess, helper-mediated or dynamic package use.

O-R018: `verification/block18_adversarial_runner.py`

Source: lines 1 through 82. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `8dec7fd754de9b04285fa921bcebc992b5381f7c1925494aad4e9446afd9f6c6`.

Literal Komorebi import targets: none. This syntax result does not rule out subprocess, helper-mediated or dynamic package use.

O-R019: verification/block18_math_verifier.py

Source: lines 1 through 300. **Test definitions:** 0. **Assert statements:** 25.

File SHA-256: `80fedc66de26f92d5990709352752e80bdb83f762ee097a11d3de811cc8e8369`.

Literal Komorebi import targets: `komorebi_math`, `komorebi_math.beta`, `komorebi_math.compatibility.compare_public_api`, `komorebi_math.compatibility.load_public_api_manifest`, `komorebi_math.coordinates.AnalysisSynthesisGauge`, `komorebi_math.coordinates.AnalysisSynthesisMap`, `komorebi_math.coordinates.AnalysisSynthesisSystem`, `komorebi_math.coordinates.CoefficientPartition`, `komorebi_math.coordinates.ExactAnalysisSynthesisMap`, `komorebi_math.coordinates.ExactAnalysisSynthesisSystem`, `komorebi_math.coordinates.exact.identity_matrix`, `komorebi_math.coordinates.exact.matrix_multiply`, `komorebi_math.exceptions.ValidationError`, `komorebi_math.stable`, `komorebi_math.stable_semantics.run_stable_semantics_fixture`.

O-R020: verification/block19_adversarial_runner.py

Source: lines 1 through 87. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `2704ec19f01a7978bc6ad626502c9da5129002182953bf4d5965d4bcef3c08cd`.

Literal Komorebi import targets: none. This syntax result does not rule out subprocess, helper-mediated or dynamic package use.

O-R021: verification/block19_distribution_audit.py

Source: lines 1 through 321. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `ae2de6588abbefc350f7bd415b6387e4b2956ddfbe3e19082de82b7bae6ab13c`.

Literal Komorebi import targets: none. This syntax result does not rule out subprocess, helper-mediated or dynamic package use.

O-R022: verification/block19_math_verifier.py

Source: lines 1 through 10. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `6cd6921f1aeace48401ca8685819b3fd1064440a4fc94b80dafb7d46ac93b758`.

Literal Komorebi import targets: none. This syntax result does not rule out subprocess, helper-mediated or dynamic package use.

O-R023: verification/block20_adversarial_runner.py

Source: lines 1 through 795. **Test definitions:** 0. **Assert statements:** 65.

File SHA-256: `0ca3407ae6b6d7337c6eb2e08b91489968755b3dafafbc533288e5ba6b40379e`.

Literal Komorebi import targets: `komorebi_math.certificates.Certificate`, `komorebi_math.certificates.Exactness`, `komorebi_math.exceptions.DimensionError`, `komorebi_math.exceptions.ResourceLimitError`, `komorebi_math.exceptions.ValidationError`, `komorebi_math.expression.Compose`, `komorebi_math.expression.ExpressionBinding`, `komorebi_math.expression.ExpressionBudget`, `komorebi_math.expression.IdentityMap`, `komorebi_math.expression.LinearMapType`, `komorebi_math.expression.PolynomialCoordinateSpace`, `komorebi_math.expression.PrimitiveKind`, `komorebi_math.expression.RATIONAL_SPACE`, `komorebi_math.expression.Scale`, `komorebi_math.expression.Sum`, `komorebi_math.expression.add`, `komorebi_math.expression.affine_substitution`, `komorebi_math.expression.antiderivative`, `komorebi_math.expression.apply`, `komorebi_math.expression.boundary_functional`, `komorebi_math.expression.check_expression`, `komorebi_math.expression.compile_expression`, `komorebi_math.expression.compose`, `komorebi_math.expression.constant_injection`, `komorebi_math.expression.coordinate_change`, `komorebi_math.expression.derivative`, `komorebi_math.expression.equivalent`, `komorebi_math.expression.evaluate_expression`, `komorebi_math.expression.evaluation`, `komorebi_math.expression.exact_binding`, `komorebi_math.expression.exact_internal_certificate`, `komorebi_math.expression.exact_map`, `komorebi_math.expression.exact_value`, `komorebi_math.expression.execute_plan`, `komorebi_math.expression.expression_digest`, `komorebi_math.expression.expression_from_dict`, `komorebi_math.expression.expression_from_json`, `komorebi_math.expression.expression_to_dict`, `komorebi_math.expression.expression_to_json`, `komorebi_math.expression.finite_difference`, `komorebi_math.expression.identity`, `komorebi_math.expression.interval_integration`, `komorebi_math.expression.let`, `komorebi_math.expression.matrix_chain_multiply`, `komorebi_math.expression.matrix_`

chain_parenthesization, komorebi_math.expression.matrix_chain_plan, komorebi_math.expression.normalize_expression, komorebi_math.expression.replay, komorebi_math.expression.scale, komorebi_math.expression.simplify_expression, komorebi_math.expression.symbol, komorebi_math.expression.type_from_dict, komorebi_math.expression.type_to_dict, komorebi_math.expression.zero_map.

O-R024: verification/block20_distribution_audit.py

Source: lines 1 through 1133. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: ba5f9cf2dd3b6638699a8441afafaa1965ce33fd8c4cdc66eb03e0b99e8b7b59.

Literal Komorebi import targets: none. This syntax result does not rule out subprocess, helper-mediated or dynamic package use.

O-R025: verification/block20_math_verifier.py

Source: lines 1 through 10. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: 0693857996c9ead825df446d8e429df1fff18bd791764f39223b693bfed3f8f9.

Literal Komorebi import targets: none. This syntax result does not rule out subprocess, helper-mediated or dynamic package use.

O-R026: verification/block20_source_audit.py

Source: lines 1 through 1028. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: eb69c98db9f5c28deda0c78b5c4f868d0b0ac48fb2172ace569503ed0d64b43e.

Literal Komorebi import targets: none. This syntax result does not rule out subprocess, helper-mediated or dynamic package use.

O-R027: verification/block21_publication_gate.py

Source: lines 1 through 12. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: 271a06c18d14b60462159a7fbd3e26030f4a390f4460630789164849bb30a67a.

Literal Komorebi import targets: none. This syntax result does not rule out subprocess, helper-mediated or dynamic package use.

O-R028: verification/block2_deep_audit.py

Source: lines 1 through 322. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: eb5cced18f1f2298d67509e3a9173f19d7f6b03251ff704a5fed185f43e51686.

Literal Komorebi import targets: komorebi_math.PacketSystem, komorebi_math.nonlinear.LinearLiftProblem, komorebi_math.nonlinear.LinearObstructionProblem, komorebi_math.nonlinear.NonlinearPacketMap, komorebi_math.nonlinear.PolynomialLiftProblem, komorebi_math.nonlinear.TransportEdge, komorebi_math.nonlinear.TransportGraph, komorebi_math.refinement.SmithCongruenceProblem, komorebi_math.refinement.WeightedTransferEdge, komorebi_math.refinement.WeightedTransferGraph, komorebi_math.refinement.crt_many, komorebi_math.refinement.nested_residue_analysis, komorebi_math.refinement.residue_analyze, komorebi_math.refinement.residue_synthesize.

O-R029: verification/block3_deep_audit.py

Source: lines 1 through 362. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: a3b695d21347843c263b31f6ba6ac1e77dc53fc6b393251e554bb79e24d6a0e4.

Literal Komorebi import targets: komorebi_math.algorithms.BlockDiagonalLowRankOperator, komorebi_math.algorithms.BlockDiagonalOperator, komorebi_math.algorithms.CirculantOperator, komorebi_math.algorithms.DTypeChoice, komorebi_math.algorithms.KroneckerOperator, komorebi_math.algorithms.PacketIR, komorebi_math.algorithms.PacketLayout, komorebi_math.algorithms.Task, komorebi_math.algorithms.ToeplitzOperator, komorebi_math.algorithms.TreeNode, komorebi_math.algorithms.WalshHadamardOperator, komorebi_math.algorithms.assign_discrete_precisions, komorebi_math.algorithms.block_jacobi_certificate, komorebi_math.algorithms.clipped_precision_allocation,

komorebi_math.algorithms.compile_tree_budget, komorebi_math.algorithms.compile_tree_lagrangian, komorebi_math.algorithms.conjugate_gradient, komorebi_math.algorithms.discover_block_diagonal_low_rank, komorebi_math.algorithms.discover_packet_support, komorebi_math.algorithms.optimal_precision_tolerances, komorebi_math.algorithms.pareto_frontier, komorebi_math.algorithms.recompress_factors, komorebi_math.algorithms.schedule_unlimited, komorebi_math.algorithms.schur_fill_support, komorebi_math.algorithms.support_product, komorebi_math.algorithms.transitive_closure, komorebi_math.algorithms.verify_schedule.

O-R030: `verification/block4_deep_audit.py`

Source: lines 1 through 557. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `9b8b7262eda5a4c032ab7eb796306917a4a6b269f2be26d14577494b21bf3322`.

Literal Komorebi import targets: komorebi_math.certificates.Exactness, komorebi_math.dynamics.BayesianPartition, komorebi_math.dynamics.BayesianUpdate, komorebi_math.dynamics.ConditionalExpectationTower, komorebi_math.dynamics.DirichletGreen1D, komorebi_math.dynamics.FeshbachReduction, komorebi_math.dynamics.FiniteFeynmanKac, komorebi_math.dynamics.HiddenMarkovFilter, komorebi_math.dynamics.MarkovLumping, komorebi_math.dynamics.MatrixSemigroup, komorebi_math.dynamics.MemoryKernel, komorebi_math.dynamics.PhaseTypeSurvival, komorebi_math.dynamics.SemigroupRefinement, komorebi_math.dynamics.approximate_likelihood_partition, komorebi_math.dynamics.coarsest_sufficient_partition, komorebi_math.dynamics.fractional_defect, komorebi_math.dynamics.fractional_naturality_certificate, komorebi_math.dynamics.fractional_power, komorebi_math.dynamics.matrix_exponential, komorebi_math.dynamics.memory_kernel_error_bound, komorebi_math.dynamics.relative_entropy, komorebi_math.dynamics.relative_entropy_decomposition, komorebi_math.signals.HaarPacketTree, komorebi_math.signals.PeriodicFilterBank, komorebi_math.signals.SingularPacket1D, komorebi_math.signals.TruncatedPowerTerm, komorebi_math.signals.best_basis, komorebi_math.signals.convolution_matrix, komorebi_math.signals.convolve_signal, komorebi_math.signals.finite_difference, komorebi_math.signals.gamma_from_impedance, komorebi_math.signals.impedance_from_gamma, komorebi_math.signals.lattice_polyphase_analyze, komorebi_math.signals.mixed_radix_address, komorebi_math.signals.nonlinear_shift_filter, komorebi_math.signals.packet_product, komorebi_math.signals.periodic_five_three_lifting, komorebi_math.signals.phase_mobius, komorebi_math.signals.polyphase_analyze, komorebi_math.signals.polyphase_convolution, komorebi_math.signals.polyphase_filter_kernels, komorebi_math.signals.recover_separated_jumps, komorebi_math.signals.recover_vector_jumps_subgaussian, komorebi_math.signals.retained_packet_reconstruction, komorebi_math.signals.scalar_region_lp_error, komorebi_math.signals.split_mixed_radix_address.

O-R031: `verification/block5_deep_audit.py`

Source: lines 1 through 471. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `7beb2f035b6274a2449d5ccc846442ab021229b83a3b3c46f4ef8b2939bd72ac`.

Literal Komorebi import targets: komorebi_math.analytic.CyclicDenominator, komorebi_math.analytic.CyclicSectionFamily, komorebi_math.analytic.appell_coefficients, komorebi_math.analytic.appell_polynomial, komorebi_math.analytic.congruential_inverse, komorebi_math.analytic.cyclic_addition, komorebi_math.analytic.cyclic_packets, komorebi_math.analytic.cyclic_product_packets, komorebi_math.analytic.cyclic_project_coefficients, komorebi_math.analytic.cyclic_project_value, komorebi_math.analytic.cyclic_reconstruct, komorebi_math.analytic.denominator, komorebi_math.analytic.denominator_root_sum, komorebi_math.analytic.logarithmic_coefficients, komorebi_math.analytic.positive_zero_string, komorebi_math.analytic.reciprocal_coefficients, komorebi_math.analytic.reciprocal_semigroup_certificate, komorebi_math.analytic.signed_sections, komorebi_math.analytic.signed_shift_matrix, komorebi_math.analytic.theta_polynomial, komorebi_math.analytic.transfer_sections, komorebi_math.analytic.transfer_shift_matrix, komorebi_math.analytic.transfer_tangent, komorebi_math.core.PacketSystem, komorebi_math.dynamics.matrix_exponential, komorebi_math.quantum.BoundedHamiltonianTower, komorebi_math.quantum.CircuitPacketExpansion, komorebi_math.quantum.EuclideanTransfer, komorebi_math.quantum.KatoTransport, komorebi_math.quantum.LindbladGenerator, komorebi_math.quantum.MovingPacketSystem, komorebi_math.quantum.QuantumChannel, komorebi_math.quantum.QuantumCode, komorebi_math.quantum.QuantumFeshbachReduction, komorebi_math.quantum.QuantumSeedSystem, komorebi_math.quantum.amplitude_damping, komorebi_math.quantum.charge_certificate, komorebi_math.quantum.charge_selection_certificate, komorebi_math.quantum.computational_basis_generation, komorebi_math.quantum.constraint_closure, komorebi_math.quantum.cyclic_symmetry_projectors, komorebi_math.quantum.dephase, komorebi_math.quantum.direct_packet_rotation, komorebi_math.quantum.fixed_packet_leakage, komorebi_math.quantum.heat_trace, komorebi_math.quantum.local_orbit_dimension, komorebi_math.quantum.low_mode_dispersion_bound, komorebi_math.quantum.

observable_seed_rank, komorebi_math.quantum.operator_packets, komorebi_math.quantum.periodic_laplacian, komorebi_math.quantum.periodic_lattice_eigenvalues, komorebi_math.quantum.reconstruct_operator_packets, komorebi_math.quantum.schmidt_rank, komorebi_math.quantum.spectral_dimension, komorebi_math.quantum.total_charge_packets, komorebi_math.quantum.transported_constraint_defect.

O-R032: verification/block6_deep_audit.py

Source: lines 1 through 372. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: 077504497551379eac1ce429dd6f21bc6a889bf18cdae1fe4cc0c98d9fe11090.

Literal Komorebi import targets: komorebi_math.arithmetic.EulerLocalCoordinates, komorebi_math.arithmetic.HurwitzSeries, komorebi_math.arithmetic.LinearRecurrence, komorebi_math.arithmetic.MahlerSeries, komorebi_math.arithmetic.StandardWindow, komorebi_math.arithmetic.TruncatedPadic, komorebi_math.arithmetic.affine_kernel_composition_certificate, komorebi_math.arithmetic.affine_mahler_kernel, komorebi_math.arithmetic.binomial_valuation, komorebi_math.arithmetic.binomial_valuation_from_digits, komorebi_math.arithmetic.carmichael_exponent_prime_power, komorebi_math.arithmetic.compressed_hurwitz_inverse, komorebi_math.arithmetic.digit_sum, komorebi_math.arithmetic.dyadic_kernel_valuation, komorebi_math.arithmetic.evaluate_matrix_mahler, komorebi_math.arithmetic.finite_difference_value, komorebi_math.arithmetic.hurwitz_inverse, komorebi_math.arithmetic.hurwitz_product, komorebi_math.arithmetic.local_unit_jump, komorebi_math.arithmetic.mahler_polyphase, komorebi_math.arithmetic.matrix_mahler_coefficients, komorebi_math.arithmetic.padic_valuation, komorebi_math.arithmetic.polyphase_analyze, komorebi_math.arithmetic.polyphase_synthesize, komorebi_math.arithmetic.prime_power_kernel_certificate, komorebi_math.arithmetic.signed_euler_number, komorebi_math.arithmetic.simultaneous_unit_difference, komorebi_math.arithmetic.subgroup_selector, komorebi_math.sequences.Sequence, komorebi_math.sequences.apery_recurrence, komorebi_math.sequences.apery_sequence, komorebi_math.sequences.bell_sequence, komorebi_math.sequences.binomial_transform_values, komorebi_math.sequences.catalan_sequence, komorebi_math.sequences.compile_sequence, komorebi_math.sequences.detect_linear_recurrence, komorebi_math.sequences.fibonacci_sequence, komorebi_math.sequences.inverse_binomial_transform_values, komorebi_math.sequences.motzkin_recurrence, komorebi_math.sequences.partition_sequence, komorebi_math.sequences.thue_morse_automaton, komorebi_math.sequences.thue_morse_sequence.

O-R033: verification/block7_deep_audit.py

Source: lines 1 through 328. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: 21187710aafb9226372e02b512bd06f401d6580c89cd33aecea7ccaec223c0af.

Literal Komorebi import targets: komorebi_math.deformation.FiniteCochainComplex, komorebi_math.deformation.PresentedModule, komorebi_math.deformation.VisibilityTree, komorebi_math.deformation.finite_frobenius_separator, komorebi_math.deformation.nilpotent_chart_count, komorebi_math.deformation.normal_crossing_count, komorebi_math.deformation.residue_forest, komorebi_math.langlands.completion.AutomorphicCompletionReport, komorebi_math.langlands.completion.CompletionGate, komorebi_math.langlands.completion.GateStatus, komorebi_math.langlands.finite_image.CliffordStep, komorebi_math.langlands.finite_image.CliffordTower, komorebi_math.langlands.finite_image.InducedRepresentation, komorebi_math.langlands.finite_image.LinearCharacter, komorebi_math.langlands.finite_image.MonomialAction, komorebi_math.langlands.finite_image.cyclic_group, komorebi_math.langlands.finite_image.mackey_tensor_packets, komorebi_math.langlands.global_.FiniteHeckePackets, komorebi_math.langlands.global_.PatchingSupportReport, komorebi_math.langlands.global_.SupportComparison, komorebi_math.langlands.local.AlternatingForm, komorebi_math.langlands.local.FiniteField, komorebi_math.langlands.local.FrobeniusOrbit, komorebi_math.langlands.local.HeisenbergModel, komorebi_math.langlands.local.ProjectiveIntertwinerEnvelope, komorebi_math.langlands.local.QuadraticRefinement, komorebi_math.langlands.local.SpecialMonodromyBlock, komorebi_math.langlands.local.TameWeilPacket, komorebi_math.langlands.local.gauss_sum, komorebi_math.langlands.local.regular_orbit_count, komorebi_math.langlands.local.tame_tensor_refinement, komorebi_math.langlands.ramification.ProjectiveMultiplicityPacket, komorebi_math.langlands.ramification.RamificationCliffordTower, komorebi_math.langlands.ramification.RamificationStage.

O-R034: `verification/block8_deep_audit.py`

Source: lines 1 through 231. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `cec5ecdb1924544056852c1d361f84f86069614db5afb14a5d7668d300ed6bae`.

Literal Komorebi import targets: `komorebi_math`, `komorebi_math.Certificate`, `komorebi_math.ComputationResult`, `komorebi_math.Exactness`, `komorebi_math.export`, `komorebi_math.export.DocumentMetadata`, `komorebi_math.export.ExportOptions`, `komorebi_math.export.MathDocument`, `komorebi_math.export.available_export_formats`, `komorebi_math.export.available_latex_engines`, `komorebi_math.export.export_bundle`, `komorebi_math.export.export_data`, `komorebi_math.export.latex_escape`, `komorebi_math.export.render_html`, `komorebi_math.export.render_latex`, `komorebi_math.export.render_markdown`, `komorebi_math.export.table_rows`, `komorebi_math.export.to_serializable`.

O-R035: `verification/block9_deep_audit.py`

Source: lines 1 through 425. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `4dcebea3960c55028c33012e69c226a0a80c12af1dc83a20d45fcba21846198c`.

Literal Komorebi import targets: `komorebi_math`, `komorebi_math.Exactness`, `komorebi_math.PacketSystem`, `komorebi_math.dynamics.matrix_exponential`, `komorebi_math.nonlinear`, `komorebi_math.nonlinear.CallableDerivativeProvider`, `komorebi_math.nonlinear.FilteredCorrectionProblem`, `komorebi_math.nonlinear.FilteredVectorSpace`, `komorebi_math.nonlinear.FiniteAbelianGroup`, `komorebi_math.nonlinear.FiniteFilteredCochainComplex`, `komorebi_math.nonlinear.KatoTransport`, `komorebi_math.nonlinear.MovingPacketSystem`, `komorebi_math.nonlinear.NonlinearSchurElimination`, `komorebi_math.nonlinear.PacketJetFamily`, `komorebi_math.nonlinear.PolynomialDerivativeProvider`, `komorebi_math.nonlinear.PolynomialFixedPointProblem`, `komorebi_math.nonlinear.catalan_number`, `komorebi_math.nonlinear.faa_di_bruno`, `komorebi_math.nonlinear.higher_maurer_cartan_packets`, `komorebi_math.nonlinear.majorant_convergence`, `komorebi_math.nonlinear.ordered_tree_count`, `komorebi_math.nonlinear.ordered_trees_with_leaves`, `komorebi_math.nonlinear.set_partitions`, `komorebi_math.nonlinear.stirling_second`, `komorebi_math.quantum`.

O-R036: `verification/build_block15_release_records.py`

Source: lines 1 through 294. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `0751d12950eacafd7b69a5647d22530f331a4865855e7c611239bb6c56707df2`.

Literal Komorebi import targets: none. This syntax result does not rule out subprocess, helper-mediated or dynamic package use.

O-R037: `verification/build_block16_distributions.py`

Source: lines 1 through 65. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `770132dbdbaaec8742249083b1c958f82df7bb698e22019fc86df26c0d786f0a`.

Literal Komorebi import targets: none. This syntax result does not rule out subprocess, helper-mediated or dynamic package use.

O-R038: `verification/build_block17_distributions.py`

Source: lines 1 through 65. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `90b3e1d0e250b3026b91fc5fc383af4a26866d0a31cc2fceedaecfbf256ccd6b`.

Literal Komorebi import targets: none. This syntax result does not rule out subprocess, helper-mediated or dynamic package use.

O-R039: `verification/build_block20_handoff.py`

Source: lines 1 through 677. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `9bb6f8649d1a266f4be9906df107b76573915484c49c705bc28385a904f6de25`.

Literal Komorebi import targets: none. This syntax result does not rule out subprocess, helper-mediated or dynamic package use.

O-R040: `verification/build_block21_distributions.py`

Source: lines 1 through 13. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `d39a946374db84a0285ca1e27dd74bd7647bc041c4d5e718eb25c18286d510c3`.

Literal Komorebi import targets: none. This syntax result does not rule out subprocess, helper-mediated or dynamic package use.

O-R041: `verification/build_block21_handoff.py`

Source: lines 1 through 13. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `6750dd7e26a0c736f708496129347109e10b972c63ab13a29296d76917078010`.

Literal Komorebi import targets: none. This syntax result does not rule out subprocess, helper-mediated or dynamic package use.

O-R042: `verification/build_distributions.py`

Source: lines 1 through 264. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `0ca25123c69d2593aecbe3b4f2c3fe4e9304f4c3877132e630dc9b15253aac90`.

Literal Komorebi import targets: none. This syntax result does not rule out subprocess, helper-mediated or dynamic package use.

O-R043: `verification/build_r4_block15_distributions.py`

Source: lines 1 through 109. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `e4de1968ba8f2780809b8243e715f5729aeede41adfcbb16637f602e1675e4f9`.

Literal Komorebi import targets: none. This syntax result does not rule out subprocess, helper-mediated or dynamic package use.

O-R044: `verification/build_r4_block16_distributions.py`

Source: lines 1 through 109. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `08f320a8f54271efea329a287eb1ef65f65d91de03851ebbafcdcf22656a7b4a`.

Literal Komorebi import targets: none. This syntax result does not rule out subprocess, helper-mediated or dynamic package use.

O-R045: `verification/build_r4_block17_distributions.py`

Source: lines 1 through 109. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `1575b6d75a84ffd46fdeff955580e2a629ec32665d2a258d78af88572077c087`.

Literal Komorebi import targets: none. This syntax result does not rule out subprocess, helper-mediated or dynamic package use.

O-R046: `verification/build_r4_block18_distributions.py`

Source: lines 1 through 109. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `2e7e00547145e1f3dd347350111d3c6db934e3610f58f0427bd7e609c232875b`.

Literal Komorebi import targets: none. This syntax result does not rule out subprocess, helper-mediated or dynamic package use.

O-R047: `verification/build_r4_block19_distributions.py`

Source: lines 1 through 109. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `f9d2bf291e1423f23282b9d31412a85cf1bc05fd7c938bad5846a9e76504bb11`.

Literal Komorebi import targets: none. This syntax result does not rule out subprocess, helper-mediated or dynamic package use.

O-R048: `verification/build_r4_block20_distributions.py`

Source: lines 1 through 109. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `62532890d64dd1b3c2cc596815b56d5bd06e7d98196b496944ade51868b2eda6`.

Literal Komorebi import targets: none. This syntax result does not rule out subprocess, helper-mediated or dynamic package use.

O-R049: `verification/build_r4_block21_distributions.py`

Source: lines 1 through 109. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `774f086d74f4e16c9d0ea71daa3ef170c50c815af6f2c2d839bef4425e23c36b`.

Literal Komorebi import targets: none. This syntax result does not rule out subprocess, helper-mediated or dynamic package use.

O-R050: `verification/build_source_release.py`

Source: lines 1 through 447. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `6d6aff204ac522762b8b12d1dc18d8632bf205a5abded27dd7edde32dddb5cb5`.

Literal Komorebi import targets: none. This syntax result does not rule out subprocess, helper-mediated or dynamic package use.

O-R051: `verification/candidate_evidence.py`

Source: lines 1 through 365. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `be040f10cd6f92c3e2508c7fa9dc2b824b9c6f890d661f66e621f819dc6c5882`.

Literal Komorebi import targets: none. This syntax result does not rule out subprocess, helper-mediated or dynamic package use.

O-R052: `verification/canonicalize_block20_sdist.py`

Source: lines 1 through 13. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `c376c9c8b44ec5042d2485d0fae537f03513813c981872e381bda9bd78a700c0`.

Literal Komorebi import targets: none. This syntax result does not rule out subprocess, helper-mediated or dynamic package use.

O-R053: `verification/canonicalize_sdist.py`

Source: lines 1 through 126. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `bf42891bab3b4cbea409f80c2db54306a7fee484a900d0c1bfcd41c1b8e0be76`.

Literal Komorebi import targets: none. This syntax result does not rule out subprocess, helper-mediated or dynamic package use.

O-R054: `verification/check_math_docs.py`

Source: lines 1 through 149. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `4b6d1b83e28fe2cc618bf28ca5c6a34e1bd88d3e7ec672366733b461229eb15e`.

Literal Komorebi import targets: none. This syntax result does not rule out subprocess, helper-mediated or dynamic package use.

O-R055: `verification/check_release.py`

Source: lines 1 through 546. **Test definitions:** 0. **Assert statements:** 5.

File SHA-256: `b529b6254746bc0a214f596e9da11795d7cb6919030314907eff6b8a5f194708`.

Literal Komorebi import targets: `komorebi_math`, `komorebi_math.compatibility.check_beta_api`, `komorebi_math.compatibility.check_public_api`, `komorebi_math.compatibility.check_stable_api`, `komorebi_math.release.check_beta_readiness`, `komorebi_math.release.check_release_contract`, `komorebi_math.release.check_upgrade_contract`, `komorebi_math.release.check_validation_evidence`, `komorebi_math.release.contract._version_stage`, `komorebi_math.release.evidence._check_historical_validation_evidence`, `komorebi_math.verify.run_block20_verification`, `komorebi_math.verify.run_stable_verification`.

O-R056: `verification/check_release_manual.py`

Source: lines 1 through 163. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `714f01054919d28388213a385c9f27a9b4ccd50c695cf74be762d592d7e4c777`.

Literal Komorebi import targets: none. This syntax result does not rule out subprocess, helper-mediated or dynamic package use.

O-R057: `verification/deep_audit_regressions.py`

Source: lines 1 through 149. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `b5cf3cdaa51c01e8a5ff350e5e38557ff6b28fd2cc9b128392f272b3cc3c0d7d`.

Literal Komorebi import targets: none. This syntax result does not rule out subprocess, helper-mediated or dynamic package use.

O-R058: `verification/episode004_conditioning_stress.py`

Source: lines 1 through 65. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `ef3a0b3b2252c0000b0d44035c8f57f756b38b1480e5333fc2f32a0b4f89ab11`.

Literal Komorebi import targets: none. This syntax result does not rule out subprocess, helper-mediated or dynamic package use.

O-R059: `verification/episode005_metric_stress.py`

Source: lines 1 through 138. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `aa61ffb95d9cc4617a1407fdeaffff4c4ee9a123bd83b7b22d78848b2310e179`.

Literal Komorebi import targets: `komorebi_math.coordinates.AnalysisSynthesisGauge`, `komorebi_math.coordinates.AnalysisSynthesisSystem`, `komorebi_math.exceptions.ValidationError`, `komorebi_math.nonlinear.TransportEdge`, `komorebi_math.nonlinear.TransportGraph`.

O-R060: `verification/episode006_distortion_probe.py`

Source: lines 1 through 144. **Test definitions:** 0. **Assert statements:** 1.

File SHA-256: `e66212dba3613b8b00076e690a492a98aeb1a51c305599f37c71a4449fb5be2b`.

Literal Komorebi import targets: `komorebi_math.nonlinear.distortion.metric_distortion_packet`, `komorebi_math.nonlinear.distortion.spectral_distortion_lower_bound`.

O-R061: `verification/episode007_convex_probe.py`

Source: lines 1 through 122. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `cfd1562eefb7472f731d948210296b9b5a93a4b9f4dd00023109a70aafcf5629`.

Literal Komorebi import targets: `komorebi_math.nonlinear.convex_packets.parallelotope_norm_packet`, `komorebi_math.nonlinear.convex_packets.quadratic_operator_norm`.

O-R062: `verification/episode008_support_packet_probe.py`

Source: lines 1 through 167. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `7cd01157b26e79264defd302671d4508eb81e4b37ed9899fd08896a0420be096`.

Literal Komorebi import targets: `komorebi_math.nonlinear.convex_packets.polyhedral_support_packet`.

O-R063: `verification/episode009_batch_qualification.py`

Source: lines 1 through 32. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `b9254c92bad70969b2f7bf9e2bf8556514c594a844d91e12ff9303b86a8af8a7`.

Literal Komorebi import targets: none. This syntax result does not rule out subprocess, helper-mediated or dynamic package use.

O-R064: `verification/episode009_provenance_probe.py`

Source: lines 1 through 198. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `8a06902695d59d9c94a055aae7677cfc7dfb70750dd9c0d8cf7bb579eab2ec15`.

Literal Komorebi import targets: `komorebi_math.nonlinear.convex_packets.polyhedral_support_packet`, `komorebi_math.nonlinear.provenance_packets.compile_observer_convex_packet`, `komorebi_math.nonlinear.provenance_packets.compile_policy_packet`, `komorebi_math.nonlinear.provenance_packets.finite_provenance_observer`.

O-R065: `verification/episode010_polyphase_route_probe.py`

Source: lines 1 through 191. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `54ca10ff7b34fae205a1ed29a6a8d163e6df12b1c9d45f61d75f88656305f040`.

Literal Komorebi import targets: `komorebi_math.signals.filterbanks.HaarFilterBank`, `komorebi_math.signals.route_packets.PolyphaseRoutePacket`.

O-R066: `verification/episode011_carrier_change_probe.py`

Source: lines 1 through 159. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `d4657513fa72455fbf1edfa3c329212c14e028a264d83495c85558df91d95c08`.

Literal Komorebi import targets: `komorebi_math.core.carrier_change.SignedMassPacket`, `komorebi_math.core.carrier_change.TropicalResidualPacket`, `komorebi_math.core.carrier_change.boolean_support_nonnegative`.

O-R067: `verification/episode012_hankel_probe.py`

Source: lines 1 through 220. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `6264e69cf39561aaf2b869feff270e0ce05a1b40cfddd0873914d914d36b6b5b`.

Literal Komorebi import targets: `komorebi_math.arithmetic.recurrence.LinearRecurrence`, `komorebi_math.signals.route_packets.PolyphaseRoutePacket`.

O-R068: `verification/episode013_quantum_gauge_probe.py`

Source: lines 1 through 228. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `5a87253ff3dc678497f783a671fc1ebe39f9da7f0fc3b261e2cf0208c6f16012`.

Literal Komorebi import targets: `komorebi_math.quantum.QuantumChannel`, `komorebi_math.quantum.QuantumCode`.

O-R069: `verification/episode014_lindblad_gauge_probe.py`

Source: lines 1 through 172. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `40971dd5ae95e46bcce79c2314d288d625a69467c6a624992b9fa6f8a8b9b71f`.

Literal Komorebi import targets: `komorebi_math.quantum.LindbladGenerator`, `komorebi_math.quantum.charge_certificate`.

O-R070: `verification/episode015_arithmetic_semantic_probe.py`

Source: lines 1 through 132. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `e66371952b9999c4da073761654c7193540a5e51446ff18b14f84172cf330f06`.

Literal Komorebi import targets: `komorebi_math.arithmetic.MahlerSeries`, `komorebi_math.arithmetic.TruncatedPadic`, `komorebi_math.arithmetic.UnitModeMahler`, `komorebi_math.arithmetic.padic_valuation`, `komorebi_math.generating.TruncatedPadicDomain`.

O-R071: `verification/episode017_deformation_separator_probe.py`

Source: lines 1 through 120. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `6905e95dc432d035547044a6fad4e0f4555722f2ab4e355e776e7c229b2d115d`.

Literal Komorebi import targets: `komorebi_math.deformation.finite_frobenius_separator`, `komorebi_math.exceptions.ValidationError`.

O-R072: `verification/episode018_certificate_consistency_probe.py`

Source: lines 1 through 316. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `a8dfdecced47d790e5273dcae76a4571a994703730a9045ab283fd4069ad6f8b`.

Literal Komorebi import targets: `komorebi_math.algorithms.compiler.TreeNode`, `komorebi_math.algorithms.compiler.compile_tree_budget`, `komorebi_math.algorithms.compiler.compile_tree_lagrangian`, `komorebi_math.algorithms.precision.DTypeChoice`, `komorebi_math.algorithms.precision.assign_discrete_precisions`, `komorebi_math.core.packets.PacketSystem`, `komorebi_math.dynamics.survival.PhaseTypeSurvival`, `komorebi_math.langlands.finite_image.groups.FiniteGroup`, `komorebi_math.nonlinear.graded.FiniteAbelianGroup`, `komorebi_math.signals.wavelets.best_basis`, `komorebi_math.signals.wavelets.haar_packet_analyze`.

O-R073: `verification/hardware_qualification.py`

Source: lines 1 through 276. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `7e075daaae65a7b525e4ac7f80092d82e9849fdc5de90722e120492ead214d53`.

Literal Komorebi import targets: `komorebi_math`, `komorebi_math.release.gpu.qualify_gpu`, `komorebi_math.verify.gpu.run_gpu_verification`.

O-R074: `verification/installed_artifact_regressions.py`

Source: lines 1 through 58. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `2fe61d4ee13ed7092644817c30511bdc40e0474dc4a405dc5e577a49bd9ac90b`.

Literal Komorebi import targets: `komorebi_math`, `komorebi_math.coordinates.families.fibonacci_recurrence_model`, `komorebi_math.export.persistence.from_portable_data`, `komorebi_math.export.persistence.to_portable_data`.

O-R075: `verification/installed_block12_probe.py`

Source: lines 1 through 32. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `3ab8d83ea5bf18c95f79ee1c6f0e4cf3d11cbcec8d7a78b71406fe2204112897`.

Literal Komorebi import targets: `komorebi_math.nonlinear.convex_packets.polyhedral_support_packet`.

O-R076: `verification/installed_block13_probe.py`

Source: lines 1 through 39. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `ecb2702dcca6a1556936ff2c9583f5a9dc43272245d37a1cc0e6c35e70d1a6bf`.

Literal Komorebi import targets: `komorebi_math.nonlinear.convex_packets.polyhedral_support_packet`, `komorebi_math.nonlinear.provenance_packets.compile_observer_convex_packet`, `komorebi_math.nonlinear.provenance_packets.compile_policy_packet`, `komorebi_math.nonlinear.provenance_packets.finite_provenance_observer`.

O-R077: `verification/installed_block16_probe.py`

Source: lines 1 through 54. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `909cd533917a9d9272922f2a8e281f1d1b1f95f2484ca49e7fbc19f17262e376`.

Literal Komorebi import targets: `komorebi_math`, `komorebi_math.arithmetic.recurrence.LinearRecurrence`, `komorebi_math.signals.route_packets.PolyphaseRoutePacket`.

O-R078: `verification/installed_block17_probe.py`

Source: lines 1 through 80. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `36ecc1c228e9af60a9fea26876d2955c1e55fa7b0d33b7f42887a95142a50516`.

Literal Komorebi import targets: `komorebi_math`, `komorebi_math.quantum.QuantumCode`.

O-R079: `verification/installed_block18_probe.py`

Source: lines 1 through 72. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `d5a5ff5b65b36dbd0365fad6255614069177e45c0245f7b0e5825d5695a4a6ee`.

Literal Komorebi import targets: `komorebi_math`, `komorebi_math.quantum.LindbladGenerator`, `komorebi_math.quantum.charge_certificate`.

O-R080: `verification/installed_block19_probe.py`

Source: lines 1 through 25. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `84bcd6ae18dd28c171d6b5349189e50daf316d6b0fa4a1ab60a8ff13cf86fcc6`.

Literal Komorebi import targets: `komorebi_math.arithmetic.TruncatedPadic`, `komorebi_math.arithmetic.UnitModeMahler`, `komorebi_math.generating.TruncatedPadicDomain`.

O-R081: `verification/installed_block20_probe.py`

Source: lines 1 through 54. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `e027f19389b1a832fd24d4f94871dcad11333f5c47ad99d84a5ae34c38f8440f`.

Literal Komorebi import targets: `komorebi_math`, `komorebi_math.deformation.finite_frobenius_separator`, `komorebi_math.exceptions.ValidationError`.

O-R082: `verification/installed_runtime_suite.py`

Source: lines 1 through 102. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `d0581325ff02470a334e7603e6228aeae448a6ffc417add9a20742d7757751c9`.

Literal Komorebi import targets: `komorebi_math`.

O-R083: `verification/optional_ad_backend_smoke.py`

Source: lines 1 through 170. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `f18a8c3654128f9c81c8a2899bacac58ea905c79eb8371e16b1c48d6a8421c01`.

Literal Komorebi import targets: `komorebi_math.nonlinear.autodiff.JaxDerivativeProvider`, `komorebi_math.nonlinear.autodiff.TorchDerivativeProvider`.

O-R084: `verification/optional_ad_backend_status.py`

Source: lines 1 through 28. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `c43435ebcf9e8bde4d36b50cbf5d6c9221b16df55ab2d7ac23f83e0dc0f1228b`.

Literal Komorebi import targets: `komorebi_math.nonlinear.autodiff.ad_framework_diagnostics`.

O-R085: `verification/package_audit.py`

Source: lines 1 through 290. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `929454bd3a1ea173cf829f01e57aca2c86dc26884b59380d093826965f0920a8`.

Literal Komorebi import targets: `komorebi_math`, `komorebi_math.qualification.run_release_qualification`, `komorebi_math.release.check_release_contract`, `komorebi_math.release.check_stable_documentation`, `komorebi_math.release.check_upgrade_contract`, `komorebi_math.release.check_validation_evidence`, `komorebi_math.release.evidence._check_historical_validation_evidence`.

O-R086: `verification/release_artifact_gate.py`

Source: lines 1 through 207. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `633a0412ffcf40009b4d48e247ebe20066a0db20df47aca920b32d11811022bf`.

Literal Komorebi import targets: none. This syntax result does not rule out subprocess, helper-mediated or dynamic package use.

O-R087: `verification/release_integrity.py`

Source: lines 1 through 322. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `f3e9c8d2b611c20ff3199d9f8a1d8354c75055b9e306e77e176d68e4a0f441e8`.

Literal Komorebi import targets: none. This syntax result does not rule out subprocess, helper-mediated or dynamic package use.

O-R088: `verification/release_metadata.py`

Source: lines 1 through 135. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `4f0f717231f2f961be2542c589376ec76914ac03ee6580915fdc27ac086bab30`.

Literal Komorebi import targets: none. This syntax result does not rule out subprocess, helper-mediated or dynamic package use.

O-R089: `verification/release_promotion.py`

Source: lines 1 through 446. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `794d40d0ae3ec5d8b5ca4aaaeca817450169ade84c010a8cbb2a0ae70a0d0ceb`.

Literal Komorebi import targets: none. This syntax result does not rule out subprocess, helper-mediated or dynamic package use.

O-R090: `verification/repair_r4_block6_mutation_audit.py`

Source: lines 1 through 64. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `5f3c02fcea8d01e6b2d5b9f8fcf6b7ef08089981f256de34171bcbbd9d3dbc4b`.

Literal Komorebi import targets: `komorebi_math.OperatorGrammar`, `komorebi_math.OperatorRealization`, `komorebi_math.PacketOperator`, `komorebi_math.PacketSystem`, `komorebi_math.coordinates.analysis_synthesis.AnalysisSynthesisSystem`, `komorebi_math.coordinates.numeric.NumericCoordinateTransform`, `komorebi_math.nonlinear.correction.HilbertObstructionProjector`, `komorebi_math.nonlinear.correction.LyapunovSchmidtReduction`, `komorebi_math.nonlinear.correction.RightInverseCorrectionProblem`, `komorebi_math.nonlinear.derivatives.CallableDerivativeProvider`, `komorebi_math.refinement.WeightedTransferEdge`, `komorebi_math.refinement.WeightedTransferGraph`.

O-R091: `verification/repair_r4_block7_persistence_audit.py`

Source: lines 1 through 170. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `df9f39e4f159e23aae9982a8160a1e22386641746c1dda2b445be1235ccaa3e3`.

Literal Komorebi import targets: `komorebi_math`, `komorebi_math.Certificate`, `komorebi_math.Exactness`, `komorebi_math.PacketSystem`, `komorebi_math.exceptions.ValidationError`, `komorebi_math.export.persistence.dumps_portable`, `komorebi_math.export.persistence.loads_portable`, `komorebi_math.nonlinear.derivatives.PolynomialDerivativeProvider`, `komorebi_math.nonlinear.jets.PacketJetFamily`, `komorebi_math.nonlinear.snapshots.snapshot_packet_jet`, `komorebi_math.nonlinear.streaming.load_packet_jet_snapshot_stream`, `komorebi_math.nonlinear.streaming.open_packet_jet_snapshot_stream`, `komorebi_math.nonlinear.streaming.save_packet_jet_snapshot_stream`.

O-R092: `verification/repair_r4_block8_holonomy_gauge_audit.py`

Source: lines 1 through 110. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `aad2e37651ad208f87589157d521429316b82c9fda75dc26e9d7731f40b6fb63`.

Literal Komorebi import targets: `komorebi_math`, `komorebi_math.exceptions.ValidationError`, `komorebi_math.nonlinear.TransportEdge`, `komorebi_math.nonlinear.TransportGraph`.

O-R093: `verification/repair_r4_block9_metric_packet_audit.py`

Source: lines 1 through 129. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `d40b48ae2fe5ee879b3197bdb8c047d7ec090d2ff43f3061d237dc269ab5ab6e`.

Literal Komorebi import targets: `komorebi_math`, `komorebi_math.coordinates.AnalysisSynthesisGauge`, `komorebi_math.coordinates.AnalysisSynthesisSystem`, `komorebi_math.exceptions.ValidationError`, `komorebi_math.nonlinear.TransportEdge`, `komorebi_math.nonlinear.TransportGraph`.

O-R094: `verification/reproducible_wheel.py`

Source: lines 1 through 44. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `12edcbe6a137c5d18164fa90ab28b7cb10de6673abb5d672b3d06fe944f1e8df`.

Literal Komorebi import targets: none. This syntax result does not rule out subprocess, helper-mediated or dynamic package use.

O-R095: `verification/run_block14_test_batches.py`

Source: lines 1 through 244. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `e7b07fe91204a9b84fad04f09f2def68eafa7e65720e308789f0de9a256254c9`.

Literal Komorebi import targets: none. This syntax result does not rule out subprocess, helper-mediated or dynamic package use.

O-R096: `verification/run_block14a2_examples.py`

Source: lines 1 through 62. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `ad1eba9e18d21e6ddf34ce62d725abb23d515dddc2ff30dbc80563c92dd6c198`.

Literal Komorebi import targets: none. This syntax result does not rule out subprocess, helper-mediated or dynamic package use.

O-R097: `verification/run_block15_test_batches.py`

Source: lines 1 through 311. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `c30a096e27e15c4745849c48b6792b7e121c069e6c8bbb6cac5a768d5c87a0e7`.

Literal Komorebi import targets: none. This syntax result does not rule out subprocess, helper-mediated or dynamic package use.

O-R098: `verification/run_block16_test_batches.py`

Source: lines 1 through 125. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `23b7e04d81148b187849de8f1545de1ec3df7385bef7db75212bef6d339bafd2`.

Literal Komorebi import targets: none. This syntax result does not rule out subprocess, helper-mediated or dynamic package use.

O-R099: `verification/run_block17_upgrade_validation.py`

Source: lines 1 through 175. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `225bede1f8473f655895b11a57b386ca15253da28da9a9576ff92e28a7f43cc6`.

Literal Komorebi import targets: none. This syntax result does not rule out subprocess, helper-mediated or dynamic package use.

O-R100: `verification/run_isolated_test_batches.py`

Source: lines 1 through 121. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `893bcb29b43c385ce1933bc7dda3f659efa52c197d2b291ed639806118489730`.

Literal Komorebi import targets: none. This syntax result does not rule out subprocess, helper-mediated or dynamic package use.

O-R101: `verification/source_audit.py`

Source: lines 1 through 288. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `94e69a52166cf55329a572dd50c21eb9b41cc1c4a8a8b5cda93346633a0979e1`.

Literal Komorebi import targets: `komorebi_math`, `komorebi_math.stable`.

O-R102: `verification/source_layout.py`

Source: lines 1 through 116. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `5176b64fb55d2571bf62fdd07a802bca6cc9066081420cffba69d16bcff11e22`.

Literal Komorebi import targets: none. This syntax result does not rule out subprocess, helper-mediated or dynamic package use.

O-R103: `verification/stress_release_candidate.py`

Source: lines 1 through 238. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `3a80ca718146d998d8b0c4f9567e64374c37b7b945ffa2b73142bfc333aa0991`.

Literal Komorebi import targets: none. This syntax result does not rule out subprocess, helper-mediated or dynamic package use.

O-R104: `verification/stress_resource_probe.py`

Source: lines 1 through 90. **Test definitions:** 0. **Assert statements:** 6.

File SHA-256: `48f7e4166bb749237895c14206f1acb1d29ce4a3157c0465b93566a9af0fc3cc`.

Literal Komorebi import targets: `komorebi_math.core.semilattice.FiniteJoinSemilattice`, `komorebi_math.quadratic_forms.certified_positive_logsf`, `komorebi_math.quadratic_forms.certify_float64_factor_spectrum`.

O-R105: `verification/verify_expression_math.py`

Source: lines 1 through 798. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `737b461ffe875d4f0e2262ccc4610071ca8b1b66038e9a905d19a466206dcf60`.

Literal Komorebi import targets: none. This syntax result does not rule out subprocess, helper-mediated or dynamic package use.

O-R106: `verification/verify_integration_math.py`

Source: lines 1 through 481. **Test definitions:** 0. **Assert statements:** 0.

File SHA-256: `7e098327a81e167d7f8d92a4698eccdde51139b734f9bbb45d24d6ee3559d308`.

Literal Komorebi import targets: `komorebi_math`, `komorebi_math.certificates.Exactness`, `komorebi_math.coordinates.AnalysisSynthesisSystem`, `komorebi_math.coordinates.euler_polynomial_family`, `komorebi_math.coordinates.fibonacci_polynomial_family`, `komorebi_math.coordinates.legendre_polynomial_family`, `komorebi_math.coordinates.monomial_polynomial_family`, `komorebi_math.exceptions.ValidationError`, `komorebi_math.integration.AtomicIntegratedDerivativeProvider`, `komorebi_math.integration.AtomicMeasure`, `komorebi_math.integration.CompositeSimpson`, `komorebi_math.integration.ErrorGuarantee`, `komorebi_math.integration.ExactAffineMap`, `komorebi_math.integration.ExactPolynomial`, `komorebi_math.integration.ExactQuadratureRule`, `komorebi_math.integration.IntervalMeasure`, `komorebi_math.integration.ProductMeasure`, `komorebi_math.integration.PushforwardMeasure`, `komorebi_math.integration.affine_interval_change_of_variables`, `komorebi_math.integration.all_fubini_orders`, `komorebi_math.integration.analysis_synthesis_integration_certificate`, `komorebi_math.integration.coordinate_fundamental_theorem`, `komorebi_math.integration.coordinate_transport_identity`, `komorebi_math.integration.fundamental_theorem`, `komorebi_math.integration.integrate`, `komorebi_math.integration.integrate_polynomial_oriented`, `komorebi_math.integration.integration_by_parts`, `komorebi_math.integration.polynomial_derivative_bound`, `komorebi_math.integration.pushforward_identity`.

O.7 Reproduction and limits

Run the bundled `build_test_crosswalk.py` against the frozen repository source and the retained release-evidence directory. The generator writes the JSON inventory and this appendix from the same parsed records. A source-file change changes the manifest binding even when its printed test count stays the same. The generator intentionally performs no package import or test execution.

For a dynamic coverage claim, run the chosen test selection with an explicitly configured coverage tool, retain the exact source and environment, and distinguish statement, branch and mathematical-condition coverage. For a new qualification claim, retain its actual execution receipt. The static relationships printed here provide routes to inspection and reproduction; they do not substitute for either measurement.

Part XIV. Documentation standards and design specifications

How to use this reference part. Chapters 75 through 82 describe the standards for maintaining a reliable manual and its supporting records. Chapters 83 through 88 collect detailed design specifications for the earlier mathematical and computational chapters. These are useful to contributors, implementers, and readers checking a precise contract. They are not prerequisites for using the library.

Start with the explanatory chapter when learning a feature. Use its API card for the import, signature, accepted inputs, and result. Consult the matching specification here when you need the compact list of equations, hypotheses, edge cases, and example obligations. Where a specification identifies its explanatory chapter as authoritative, read that chapter's more detailed qualification before applying the compact statement.

Several terms recur in this part. A *gate* is a named acceptance check. A *ledger* is a structured inventory with evidence or status attached to each item. A *fixture* is a committed input and expected result used for a repeatable check. *Closure* means that the stated inventory has been accounted for under its stated rule. None of these terms enlarges the mathematical scope of a calculation.

Requirements and observations have different roles. An example checklist says what a complete example should demonstrate. A recorded result says what a specified run actually demonstrated. A target completion predicate states the conditions to be met; it does not assert that every condition has been met. Historical inventories and release observations retain their declared source baseline. Use the current package records for a new release decision.

75. Freeze and finalization protocol

The production freeze converts a moving source tree into a reviewable documentation snapshot. It freezes **evidence identity**, not future project development. A later release may add symbols, examples, or workflows, but it must do so under a new versioned inventory instead of changing the meaning of the declared reference baseline.

75.1 What is frozen

A usable freeze record has four layers:

$$F = (v, \mathcal{I}, h, \mathcal{R}),$$

where v is the package and handoff identity, $\mathcal{I}$ is the measured inventory, h is the set of content digests, and $\mathcal{R}$ records open reconciliation items. A count without the files that produced it is not a freeze, and a digest without a semantic inventory is not a coverage claim.

The reference source snapshot resolves to these measured sets.

Source surface	Frozen observation
Stable API	131 exports: 58 classes, 61 functions, 12 data symbols
Beta API	83 exports: 45 classes, 28 functions, 10 data symbols
Research API	1,312 exports: 518 classes, 733 functions, 61 data symbols. All 1,312 symbol cards are complete: 516 root and the complete coordinate, generating, signal, nonlinear, and export namespaces
Research implementation-owner strings	201: 196 package owners and 5 standard-library owners
Python implementation modules	300
Shipped examples	65: 57 Python and 8 PowerShell
Root workflow wrappers	42: 21 PowerShell and 21 POSIX shell
CLI leaf commands	46
Installed verification-suite names	51
Verification modules	50
Test inventory	130 by guide convention: 129 `test_*.py` modules plus `optional_backend_support.py`
Frozen portable codecs	51 codecs and one adjacent migration
Export formats	11
Documentation Markdown files	71: 57 in MkDocs navigation and 14 outside it

The five standard-library implementation owners are `builtins`, `collections.abc`, `pathlib`, `types`, and `typing`. They are reflected owners, not five missing package modules. Likewise, the 201 research owner strings are not interchangeable with the 300 Python source modules. Keeping those meanings separate prevents a numerically plausible but semantically false reconciliation.

The canonical JSON digests bind the three public manifests:

Manifest	Canonical SHA-256
Stable	`23cd9e47a1663f2d4b3c0fef50f9f7d8c88d4af2f8bbc57e7ba76461e6e69276`
Beta	`311f2bf7cbcf0a45bcbe2af433d73a8f597c0fbf7133750882314d6e0edb16b2`
Research	`473c73e1a0563fe0b2569c8be384f7c9f947570bea2f300ef611db351136847a`

Manifest	Canonical SHA-256

Canonical JSON means UTF-8 JSON serialized with sorted keys and compact separators. A raw-file checksum is also useful, but it changes when formatting changes. A canonical digest binds the decoded contract.

75.2 Source precedence

Source precedence is claim-specific. Export names, kinds, owners, and signatures come from live reflection reconciled with the committed manifest. Stable summaries come from `stable_documentation_1_0.json` and its release loader. Mathematical meaning comes from implementation, frozen semantic cases, and tests. Persistence claims come from the 51-codec contract and its one adjacent migration. Example existence comes from the filesystem. Execution claims require a fresh result tied to an environment and source digest. The cumulative Markdown is the reader source for this manual, while the repository MkDocs corpus is a separate companion site source.

These commands inspect the current public contracts:

```
python -m komorebi_math api check --strict-signatures
python -m komorebi_math api check --beta
python -m komorebi_math api check --stable
python -m komorebi_math verify stable
python -m komorebi_math release documentation
python -m komorebi_math release contract
```

The stable verifier additionally requires exact committed-snapshot equality. An isolated additive-compatibility comparison may tolerate a new name, but the frozen stable snapshot does not accept an uncommitted addition.

75.3 Alias and identity rules

The canonicalization map from Chapter 0 remains the governing rule. Classes and functions join by defining module and qualified name. Data exports remain keyed by exported namespace and name unless the source explicitly declares an alias. That distinction prevents equal or interned constants from being collapsed.

Under this rule the 1,312 research exports form 1,092 canonical identities: a 220-entry alias surplus distributed across 207 multi-export groups. These numbers help detect drift. They do **not** reduce the obligation to give every exported name a searchable path. The stable tier similarly has 131 names across each of the three versioned facades, for 393 name-preserving routes, while Appendix A intentionally retains one anchor per stable exported name.

Five deliberate cross-name stable pairs receive explicit links: `CoordinateTransform` / `ExactCoordinateTransform`, `refine_polyphase` / `polyphase_refine`, `coarsen_polyphase` / `polyphase_coarsen`, `refine_lattice_polyphase` / `lattice_polyphase_refine`, and `coarsen_lattice_polyphase` / `lattice_polyphase_coarsen`. Root or domain re-export alone does not grant stable status.

The final ledger therefore stores both maps:

$$\text{exported name} \longrightarrow \text{canonical identity}, \qquad \text{canonical identity} \longrightarrow \text{all public routes}.$$

75.4 Freeze procedure

Use this order so that prose never becomes the source of truth for an inventory.

1. Record the exact repository, package version, and release-contract identity.
2. Parse the committed API, codec, CLI, verification, documentation, and release inventories directly from source.
3. Apply the alias rule above and retain every exported route.
4. Compare observed counts and canonical digests with the prior checkpoint.
5. Classify each difference as an intended source change, a documentation correction, historical evidence, or an unresolved inconsistency.

6. Freeze only after every difference has an owner and disposition. An open disposition may remain, but it must remain visible.

Before describing a domain as complete, identify its source set, required records, and unresolved items. Prose cannot substitute for missing evidence.

75.5 Recorded freeze result

The stable manifest and all 131 Appendix A cards reconcile exactly. The beta manifest and all 83 Appendix B cards also reconcile exactly, including signatures, anchors, aliases, and strict compatibility. Appendix C reconciles all 1,312 research cards with four derived indexes built from the same source set.

C.7 accounts for all 201 research implementation owners. C.8 covers all 1,093 distinct research names and all 1,312 research namespace and name pairs. C.9 covers all 923 exact research signature values. C.10 assigns all 1,526 stable, beta, and research card routes to one route-identity and stability group each. Same-spelling functions that resolve to different objects remain separate.

The reference-baseline negative controls below retain their declared failure outcomes. They illustrate that reference coverage and platform qualification are separate predicates:

```
python -m komorebi_math release evidence --current
python -m komorebi_math release readiness --final
```

The failure records concern the declared reference baseline. They do not determine the readiness of another package version or platform.

> **Chapter 75 boundary.** A freeze binds source identity, precedence, inventories, canonicalization rules, and reconciliation decisions. It does not establish unexecuted platform or publication claims.

76. Symbol-card completion contract

A symbol card is the smallest independently reviewable API explanation. It must let a reader identify the exported object, decide whether it applies, predict its result and failures, choose dependencies and resources, and reach the evidence that supports the claim. A signature and a one-line summary alone are an index entry, not a complete card.

76.1 Card anatomy

For exported record s, define

$$\mathrm{Card}(s) = \{id, q, a, t, k, s, D, C, H, R, X, E, P, B, K, V, L\}.$$

Field	Required meaning
`id`	exported namespace and name
`q`	canonical implementation identity
`a`	aliases and supported import routes
`t`	stability tier and version
`k`	manifest kind
`s`	exact manifest signature, including literal `null`
`D`	mathematical or operational definition
`C`	domain, codomain, types, shapes, and units
`H`	hypotheses and preconditions
`R`	return contract and result variants
`X`	exceptions, diagnostics, and invalid inputs
`E`	exactness, tolerance, and determinism class
`P`	dependency and platform profile
`B`	backend behavior and fallbacks
`K`	complexity, memory, and resource limits
`V`	executable verification or certificate evidence
`L`	guide, example, CLI, schema, and test links

A field may say “not applicable,” “not measured,” or “not supported” when that is the true contract. It may not disappear. Literal `null` signatures in the frozen manifest must remain `null`. Inferring a constructor would manufacture an API promise that the manifest does not make. Where no tighter public complexity bound exists, the card says so instead of inventing a benchmark.

76.2 Field provenance and scope

Identity fields are copied from the manifest and checked against runtime reflection. Definition, hypotheses, result, exactness, and failure fields come from implementation, committed contracts, and tests. Dependency, backend, complexity, and evidence fields cite the corresponding source record rather than relying on convention. Reader cross-links are generated last so they cannot alter the semantic fields.

A card covers the exported symbol. It does not promise a separate exhaustive card for every public method and property on an exported class. Structural compatibility checks likewise do not certify every mathematical statement a user might form with the object.

76.3 Completeness and aliases

Let $A(s)$ be the unique anchor for exported name s. Then

$$\text{CompleteCard}(s) \iff \text{FieldsPresent}(s) \wedge \text{SourceMatch}(s) \wedge |A(s)| = 1 \wedge \text{LinksResolve}(s).$$

Callable aliases may share a definition, but each public name still gets an anchor and points to all supported routes. Data records remain name-keyed under the Chapter 75 rule. A profile may reduce drafting duplication, but every rendered card repeats the operational fields needed to use that exported name. A card is rejected if its prose silently widens exactness, platform support, dependencies, or theorem scope beyond the source.

76.4 Appendix A worked closure

Appendix A is the worked implementation of this contract. Its 131 cards match the stable name set: 58 classes, 61 functions, and 12 data exports. They contain 112 exact non-null signatures and 19 literal `null` signatures and expose every stable name through `komorebi_math.stable`, `komorebi_math.v1`, and `komorebi_math.api.v1`, covering all 393 name-preserving routes. The stable documentation comparison reports zero missing, extra, changed, or deprecated records.

The reference bundle verifier reparses Appendices A through C instead of trusting carried counts. It requires exact name-set equality, canonical anchors, section counts, signature equality, implementation identities, operational field labels, and the committed stable, beta, and research-manifest evidence.

76.5 Tier checkpoint and review algorithm

Tier	Inventory	Complete cards in this edition	Status
Stable	131	131	closed in Appendix A
Beta	83	83	closed in Appendix B
Research	1,312	1,312	closed in Appendix C, including C.7 through C.10

The research row records full symbol-card and derived-index closure. Every research namespace and name pair has one complete card. The implementation, alphabetical, signature, and alias ledgers reconcile with those cards and the committed manifests.

The appendix sequence separates module, example, command-line, schema, certificate, exception, backend, complexity, migration, glossary, documentation, and verification records. Complete operational coverage requires the corresponding joins and replay evidence. It cannot be inferred from the presence of an outline or an API inventory.

> **Chapter 76 boundary.** The card schema, provenance rules, alias treatment, explicit non-support vocabulary, parser rules, and tier accounting are complete. Appendix A closes 131 stable cards, Appendix B closes 83 beta cards, and Appendix C closes all 1,312 research cards and all four derived indexes.

77. Executable-example review

Examples are evidence only when the guide records what was executed, what outcome was expected, and what environment made the result meaningful. The shipped inventory has 65 paths: 57 Python programs and 8 PowerShell programs. The Python count and the cross-platform total describe different source sets.

77.1 Exact inventory and evidence status

An example inventory, guide placement, historical replay, representative capability tour, exhaustive Python replay, and complete platform replay are different evidence classes. Each record binds its own source version, file set, environment, and execution scope.

The five later Python examples are `block15_polyphase_refinement.py`, `block16_analysis_synthesis.py`, `block19_measure_integration_worked_example.py`, `block20_typed_expression_worked_example.py`, and `block21_public_capability_tour.py`. Historical PowerShell workflows remain useful examples, but their block names and predecessor wheel references keep them from becoming current release qualification by implication.

77.2 Replay-record contract

For each example path e, the replay ledger stores at least

$$N_e = (id, source, v, env, c, \kappa, r, o_x, o_e, A, h_A, p, d, status),$$

where `source` binds the file digest, v is the package version, `env` the interpreter and environment identity, c the exact command, κ the expected-outcome class, r the observed return code, o_x and o_e the normalized stdout and stderr digests, A the emitted artifacts, h_A their digests, p the platform, and d the dependency profile.

The permitted outcome classes are:

Class	Passing observation
success	return code zero and required assertions or certificate checks pass
expected rejection	declared nonzero code or exception and matching diagnostic
expected unavailability	missing optional backend is reported through the documented route
platform-inapplicable	the record names the unsupported platform and portable alternative

Therefore the example gate is not $r = 0$ for every path. It is

$$G_{\text{example}} = \bigwedge_{e \in \mathcal{E}} \text{Observed}(N_e) \wedge \text{MatchesExpected}(N_e, \kappa_e).$$

An adversarial example that accepts invalid input has failed even if it exits zero. Conversely, a rejection example has passed when it returns the declared nonzero code and diagnostic.

77.3 Review procedure

The authoritative replay uses a clean workspace and the bundled wheel rather than an import from the repository checkout.

1. Create an isolated environment for the example's declared Python version.
2. Install the exact wheel and only the named dependency profile.
3. Invoke a Python example with `python <example-path>` or a PowerShell example with `powershell.exe -NoProfile -ExecutionPolicy Bypass -File <example-path>`.
4. Capture output, return code, report or certificate fields, platform, interpreter, dependency versions, source digest, and artifact digests.
5. Normalize only declared nondeterministic fields such as temporary paths or elapsed time. Never normalize a mathematical value merely because it drifted.
6. Compare the observation with the expected class and store the verdict.
7. Repeat platform-sensitive examples on every claimed platform or keep the missing evidence open.

Randomized examples must record a seed or validate an invariant independent of a particular draw. Numerical examples state tolerances. Exact examples compare exact objects. Certificate examples inspect the claimed identifier, hypotheses, residual, and verification flag rather than merely printing the object.

77.4 Reference commands and mapping boundary

The representative capability tour is:

```
python examples/block21_public_capability_tour.py --json
```

Its committed capability digest is `0e20e32f4ad9004d5248d258813b6acfec7b9e96cc30c001aa6865aba9039fdf`. An interim source sweep can use:

```
python verification/run_block14a2_examples.py --start 0 --end 57 --output <record.json>
```

That runner's historical schema lacks clean-wheel identity, complete environment and source hashes, and PowerShell coverage, so even a successful 57-file run would not close the 65-example gate.

Appendix E supplies the example-record index. The command-line, schema, certificate, failure, dependency, resource, and verification joins have separate evidence requirements. Placement must not be reported as exhaustive execution validation.

> **Chapter 77 boundary.** The 65-example inventory, evidence-status ledger, expected-outcome model, replay record, and review procedure are complete. All 65 paths are placed, but a current exhaustive replay ledger and current Windows evidence are not claimed.

78. Connected tutorial review

Chapters 64 through 70 provide seven workflows and 34 connected subrecipes spanning certificate-checked algebra, arithmetic, signal processing, nonlinear methods, integration and typed expressions, finite quantum and deformation models, document export, and reproducibility. The criteria below distinguish readable explanation from complete execution evidence.

78.1 Tutorial families

Chapter	Connected family	Representative source joins
64	algebra and coordinates	quickstart, coordinate, and analysis/synthesis examples
65	arithmetic and refinement	sequence, Mahler-Cartier, prime-power, and polyphase examples
66	signal workflows	production DSP, filter bank, and singular-recovery examples
67	nonlinear and dynamics	jets, trees, transport, Markov, and Feshbach examples
68	integration and typed expressions	measure/integration and typed-expression examples
69	advanced finite systems	quantum, deformation, finite-image, and finite Langlands examples
70	export and reproducibility	report export, PowerShell export, and repository handoff routes

Each family includes representative output and explicit failure boundaries. Qualification additionally requires a fresh replay record for every declared profile and platform.

78.2 Tutorial record

A connected tutorial T is a sequence of checkpoints rather than one long transcript:

$$T = (goal, prerequisites, objects, steps, checkpoints, claim, \\ replay, extensions, platform, profile, expected, evidence).$$

Every checkpoint identifies the command or code cell, its assertion, normalized observation, evidence record, and the earlier artifact it consumes. This makes a tutorial restartable and reveals accidental dependence on a developer tree, ambient cache, or undeclared optional package.

Each review asks:

- Can a reader install the declared profile from the delivered wheel?
- Do commands use current filenames, options, and result types?
- Are exact, numerical, heuristic, and certificate-backed claims labeled?
- Do expected failures check their diagnostics?
- Are intermediate files written to explicit, disposable locations?
- Does the Windows route use PowerShell-safe quoting and the POSIX route use shell-safe quoting without pretending the syntaxes are identical?
- Can each checkpoint be rerun without hidden state or destructive overwrite?

78.3 Clean-wheel and platform matrix

Narrative completion and replay completion are separate states.

Evidence level	Meaning
prose	steps, boundaries, and expected results are complete
source-linked	symbols, examples, CLI leaves, and schemas have explicit joins
representative	the current capability tour exercises a connected route
clean-wheel	every tutorial passes from an isolated wheel install
cross-platform	every claimed Windows and POSIX route has current evidence
reproducible	reruns match declared exact, tolerance, and digest rules

Clean-wheel means imports resolve from installed `site-packages`, source `PYTHONPATH` is removed, the working directory and dependency profile are declared, artifacts and cleanup are recorded, and restart points consume only their declared state. Optional-backend branches may be conditionally closed: the tutorial shows the success branch when installed and the documented unavailability branch when absent. Shell-sensitive workflows receive both PowerShell and POSIX evidence. Platform-neutral Python is not duplicated merely for cosmetic symmetry.

78.4 Review algorithm and evidence boundary

1. Build or select the exact wheel named by the freeze record.
2. Create one clean environment per dependency profile and claimed platform.
3. Execute each checkpoint in order, then replay restartable checkpoints from their declared saved state.
4. Match structured results before comparing presentation text.
5. Run the tutorial's expected-failure and resource-limit branches.
6. Store the command, platform, dependencies, source digest, observation, and verdict for every checkpoint.
7. Reject stale wheel names, obsolete CLI flags, developer-checkout imports, undeclared files, or claims whose certificates do not match the observation.

Complete tutorial qualification requires replay records tied to the exact wheel, dependency profiles, and claimed platforms. A finished explanation does not supply that operational evidence by itself.

> **Chapter 78 boundary.** A connected tutorial records inputs, checkpoints, certificates, expected failures, dependencies, and restart state. Claims about clean installation or platform parity require separate replay evidence.

79. Warning, limitation, and drift audit

A closing audit must distinguish failure, warning, limitation, skip, and drift. Collapsing them into a single "passed" flag makes a report easy to read and unsafe to trust.

79.1 Status vocabulary

Class	Meaning	Gate effect
passed	a required check ran and matched its contract	contributes to the named gate only
failed	a required check ran and contradicted its contract	closes the gate as failed
warning	a reviewable condition outside the check's default fail set	non-gating only when policy says so
limitation	the implementation or evidence intentionally does not claim a capability	remains visible and may keep a broader gate open
not executed	a required or optional check did not run	never counts as a pass
expected failure	a negative control returned its declared rejection	validates that rejection, not the missing positive claim
historical	an internally valid record for an older source or platform	informative, never current execution
planned	a required artifact or check has a contract but does not yet exist	open
drift	two supposed current sources disagree	blocks publication until reconciled

Warnings are not intrinsically harmless. The current quick publication report can pass while warning that full tests were not requested and MkDocs was skipped. That report is valid for its 14-check quick contract, but it cannot satisfy the strict full-guide gate.

79.2 Coverage versus placement

For surface S, use three separate ratios:

$$I(S) = \frac{\text{inventoried}}{|S|}, \qquad P(S) = \frac{\text{placed}}{|S|}, \qquad C(S) = \frac{\text{complete records}}{|S|}.$$

An item may be inventoried and placed without having a complete card, replay, or crosswalk. Each surface requires its own inventory, completion, and qualification record.

Coverage is evaluated separately for API tiers, commands, examples, codecs, verification suites, tests, documentation, and implementation modules.

`Placed(201/201)` and `Placed(65/65)` are outline-placement results. They do not by themselves establish complete module cards or executable-example replay.

79.3 Warning and limitation records

A warning record identifies the named check, invocation mode, source, environment, observation, and gate effect. Historical, skipped, and selectively inspected evidence must retain those distinctions.

79.4 Recording drift corrections

A drift record binds both the superseded value and the accepted replacement to their source identities. Canonical payload digests and raw-file checksums remain different evidence types.

79.5 Drift-audit procedure

1. Recompute inventories and canonical digests from decoded source contracts.
2. Recompute raw checksums for exact distributable files.
3. Search current prose, commands, and support files for predecessor names, deprecated options, obsolete digests, and unconditional platform claims.
4. Compare every generated report with the source identity it claims to cover.
5. Require an explicit disposition for every mismatch: update the current statement, mark the record historical, or leave a blocking issue.
6. Reject a broad success claim derived only from aggregate `passed: true` when a required named check is absent or skipped.

The disposition log is itself review evidence: it names the conflicting sources, the chosen authority, the reason for the choice, and the first edition whose published text contains the correction.

> **Chapter 79 boundary.** The status vocabulary, per-surface checkpoint, current warning ledger, digest corrections, and audit algorithm are complete. The audit reports open work instead of converting limitations and skipped checks into passes.

80. Coverage ledger and symbol index

Human-readable appendices answer “what does this do?” The coverage ledger and symbol index answer “is every source item accounted for, and where?” They are different deterministic artifacts and neither can be reconstructed reliably by scraping the rendered PDF.

80.1 Coverage is a vector

For every source set T, distinguish

$$\mathrm{Placed}(T), \qquad \mathrm{Documented}(T), \qquad \mathrm{Qualified}(T).$$

Placement means an item has a guide destination. Documentation means its required record is complete. Qualification means the relevant examples, links, tests, and platform predicates have current evidence. These values may differ without inconsistency, but a publisher must never substitute the larger one for the smaller.

80.2 `coverage.json`

The coverage-ledger specification records one row per source surface and one disposition per item. Its minimum envelope is:

```
{
  "schema": "komorebi-math.guide-coverage",
  "schema_version": 1,
  "package_version": "1.0.0b4",
  "manual_block": 16,
  "source_sha256": {},
  "surfaces": [],
  "payload_sha256": "<canonical digest>"
}
```

A surface row contains `domain`, `source`, `expected`, `placed`, `complete`, `validated`, `missing`, and `status`. Each item stores its source key, canonical identity, guide anchor, completion state, evidence links, and open reason. Counts are derived from item rows. They are never hand-entered independently.

80.3 `symbol_index.json`

The symbol-index specification is search-oriented. Each exported-name row contains `qualified_name`, `tier`, `kind`, `signature`, `implementation_module`, `qualname`, `canonical_anchor`, `aliases`, `guide_locations`, and `status`. Evidence joins add examples, CLI leaves, schemas, certificates, exceptions, dependencies, tests, exactness, and backend labels.

Class and function aliases may point to one canonical definition. Data exports remain independently keyed unless source declares an alias. Index generation must therefore use the same π rule as the coverage ledger and Appendix A.

80.4 Deterministic generation

The generation sequence is fixed:

$$\text{source parse} \to \text{canonicalize} \to \text{join guide records} \to \text{validate} \to \text{canonical JSON} \to \text{digest}.$$

Rows are sorted by explicit stable keys, paths use forward slashes, text is UTF-8, object keys are sorted for the payload digest, and no timestamp or absolute workspace path enters the hashed payload. The validator recomputes surface counts, checks unique anchors, rejects unresolved internal targets, and requires the coverage and index records to agree on every exported name.

A row marked complete must resolve to exactly one current anchor. The final anchor gate checks fragments, not merely destination files, and the index gate rejects duplicate qualified names, missing source items, extra current items, and conflicting aliases.

80.5 Ledger and index requirements

The ledger accounts for stable, beta, and research exports, defining owners, CLI leaves, examples, verification suites, codecs and migrations, documentation, and tests. Every completion claim resolves to its own evidence rows rather than inheriting a larger aggregate count.

> **Chapter 80 boundary.** The ledger and index are explicit data contracts. A schema description does not establish that a concrete record was generated, validated, or qualified on a particular platform.

81. Site and PDF build closure

The companion documentation site and the cumulative technical manual are independent publication graphs. The site validates curated navigation, links, and browser presentation. The manual validates the full reader sequence, print pagination, mathematical composition, and offline readability. Success in one format never substitutes for the other.

81.1 Independent publication graphs

The repository site consumes its MkDocs configuration and documentation corpus:

$$(\texttt{mkdocs.yml}, 71\ \texttt{docs/*.md}) \longrightarrow \texttt{site/}.$$

The technical manual consumes the cumulative manual Markdown plus its metadata, header, and two Lua filters:

$$(\text{manual Markdown}, \text{build assets}) \longrightarrow \text{standalone LaTeX} \longrightarrow \text{PDF}.$$

The 57 configured navigation entries leave 14 documentation files outside the curated navigation. No configured target is missing, but the final navigation policy must give those 14 files an explicit disposition. The site is therefore a companion, not an HTML rendering of this thousand-page manual.

81.2 Strict MkDocs and full-test gate

From the repository root, the normative commands are:

```
python -m mkdocs build --strict
python -m pytest -q --tb=short tests
```

The final source gate requires both named checks:

$$G_{\text{strict}} = \text{PresentPass}(\text{full test suite}, r = 0) \wedge \text{PresentPass}(\text{strict MkDocs build}, r = 0).$$

In the reference publication report, `mode == "full"` means full pytest was requested. It does not prove MkDocs ran. The handoff loader currently checks report schema, aggregate pass, full mode, and pytest return code, but does not itself require a passed named strict-MkDocs check. The final documentation publisher must enforce the stronger predicate above rather than attribute it to the current loader.

An environment record must distinguish tools installed on the build host from checks actually executed. A carried full-test report with strict documentation skipped cannot establish a fresh strict site build.

81.3 Reference technical-manual build path

The reference baseline uses this build path from its `manual` directory:

```
pandoc .\Komorebi_Math_User_Guide_Progressive_Edition_Block16.md `
  --from=markdown+tex_math_single_backslash `
  --to=latex --standalone --top-level-division=part `
  --metadata-file=.\build-assets\komorebi_block16_technical_manual_metadata.yaml `
  --include-in-header=.\build-assets\komorebi_block16_technical_manual_header.tex `
  --lua-filter=.\build-assets\komorebi_technical_manual_layout.lua `
  --lua-filter=.\build-assets\komorebi_content_filter.lua `
  --highlight-style=pygments `
  --output=.\Komorebi_Math_User_Guide_Technical_Manual_Edition_Block16.tex

latexmk -pdf -interaction=nonstopmode -halt-on-error -file-line-error `
  .\Komorebi_Math_User_Guide_Technical_Manual_Edition_Block16.tex
```

PDF closure requires more than a successful TeX exit code. The validation record checks page size and count, final EOF, Ghostscript traversal, embedded fonts with no Type 3 font, unresolved-reference and rerun messages, new overfull boxes, extracted heading presence, and visual rendering of the changed chapters and transitions. Inherited warnings remain identified by location. A new chapter may not hide behind them.

81.4 Warning and visual-QA policy

The PDF gate does not require a literally empty warning log. It requires no fatal diagnostic, no unresolved reference, no unexplained new overfull warning in the changed span, and no material clipping or collision. Visual inspection covers every changed physical page plus the transition into and out of the closing block. A selective inspection is reported as selective, not upgraded to an all-page visual review.

PDF reproducibility is likewise an independent claim. The deterministic wheel, source distribution, and outer ZIP do not prove two independent PDF compiles are byte-identical. The build record states which artifacts have replay or byte reproducibility evidence.

81.5 Reproducible build record and current boundary

The generic `build_record.json` contract binds source digests, exact commands, tool versions, dependency identity, named-check results, artifact sizes and SHA-256 values, accepted warnings, platform evidence, and clean-replay result. It distinguishes a documented non-material warning from a skipped required check. The final site, `coverage.json`, `symbol_index.json`, PDF, and sources all become named artifacts in that record.

A bounded manual build does not by itself establish a strict companion site, full source-test execution, generated coverage and symbol indexes, or current platform qualification. Each requires its own named evidence.

> **Chapter 81 boundary.** Site validation, PDF layout, source testing, structured records, reproducibility, and platform execution are distinct predicates. A publication record must identify which predicates were tested.

82. Publication requirements and reference architecture

This chapter defines publication requirements separately from editorial completion. Finished prose does not establish unexecuted software, platform, or external-publication claims.

82.1 Three-layer source model

The manual separates reader explanations, exhaustive reference records, and retained design specifications:

$$\text{GuideSource} = \text{Reader}_{0:82,A:B} \bowtie \text{Pending}_{C:O} \bowtie \text{Regression}_{83:88}\,.$$

Reader explanations present the mathematical constructions and operational contracts. Reference appendices organize the detailed source records. The design specifications preserve equations, hypotheses, failures, examples, and source obligations for independent review. A specification does not override a more precise implemented contract.

When layers disagree, first consult the live source and committed contracts for the declared version, then the reader explanations and their checks, and finally the retained design specifications. Historical evidence remains tied to its source version and is identified as historical.

82.2 Manual-completion predicate

Let R_{16} mean that each primary Chapter 75 through 82 occurs once in order, contains its required boundary and operational fields, matches the live counts and digests, preserves the stable Appendix A contract, and states the open work without a false whole-guide claim. Let P_{16} mean that the canonical Markdown was rendered, structurally and visually validated, archived twice with byte identity, safely extracted, and verified again.

$$\text{ReaderReady}_{16} = R_{16}, \qquad \text{BoundedBuild}_{16} = R_{16} \wedge P_{16}.$$

The reference verifier evaluates R_{16} separately from stable Appendix A closure and bundle integrity. This makes the reader boundary machine-visible without substituting editorial completeness for operational evidence.

82.3 Appendix gate matrix

Appendix	Reference obligation
A	131 frozen stable cards
B	83 beta cards
C	1,312 research cards and four derived indexes
D	300 package-module cards and 5 external owner rows
E	complete example records and replay joins
F	46-leaf CLI reference and recipe crosswalk
G	51 codec records and adjacent migration
H	certificate and theorem-ID index
I	exceptions and diagnostic cookbook
J	backend and dependency matrix
K	complexity, memory, and budget tables
L	migration and deprecation tables
M	glossary and notation index
N	71-file documentation crosswalk and navigation policy

Appendix	Reference obligation
O	test and verification crosswalk

82.4 Whole-guide definition of done

Let $G_A, \dots, G_O$ be the appendix gates, G_{ex} the complete example and tutorial replay gate, G_{strict} the named full-test and strict-MkDocs gate, G_{pdf} the final PDF gate, G_{json} the coverage, index, and build-record gate, $G_{platform}$ the current evidence required by the release claim, and G_{drift} the zero-unresolved-current-drift gate. Then

$$\text{CompleteGuide} \iff \left(\bigwedge_{X=A}^{O} G_X \right) \wedge G_{ex} \wedge G_{strict} \wedge G_{pdf} \wedge G_{json} \wedge G_{platform} \wedge G_{drift}.$$

This predicate requires complete stable, beta, research, CLI, example, codec, suite, module, documentation, and test coverage. It also requires every snippet to match its expected outcome, every certificate claim to match its record, all local links and fragments to resolve, and every required artifact to be bound by the final build record.

82.5 Evaluating the publication predicate

Evaluate each conjunct against named source, build, replay, and platform records. A passing bounded reference check does not fill an absent example replay, test result, site check, coverage ledger, or external publication record.

An all-one target vector retained later in the document is normative, not an observation. `Placed(201/201)` and `Placed(65/65)` similarly describe outline placement, not complete module cards or example replay.

82.6 Publication decision and maintainer handoff

A distribution title must match its demonstrated scope. A bounded reference release may be described as such when its identity, validation, and replay records are explicit. A whole-guide completion claim requires every conjunct of the stated predicate.

The local Windows qualification entry point is:

```
powershell.exe -NoProfile -ExecutionPolicy Bypass -File .\Run_Block16_Full_Verification.ps1
```

The underlying source gate may be invoked with:

```
powershell.exe -NoProfile -ExecutionPolicy Bypass -File .\Run_Block21_Publication_Gate.ps1 -Python
    ↪ "$env:CONDA_PREFIX\python.exe"
```

The final gate rejects predecessor wheel names, obsolete CLI options, stale support-matrix filenames, historical aggregate verifiers presented as current, required checks reported only as skipped warnings, and artifacts not bound to their source digests.

82.7 Preserved chapter contract

For regression review, each retained chapter card instantiates

$$\mathfrak{C} = (\Sigma, \mathcal{T}, \mathcal{M}, \mathcal{H}, \mathcal{I}, \mathcal{R}, \mathcal{F}, \mathcal{E}, \mathcal{Q})$$

using implemented objects and current contracts. These tuples remain useful as checklists even after their reader counterparts are complete. They never override live source and they never create a public claim by themselves.

82.8 Design-specification crosswalk

Specification family	Reader chapters
Foundations	0 through 20
Discrete systems	21 through 27
Algorithms and signals	28 through 36A
Nonlinear systems and dynamics	37 through 46
Advanced finite systems	47 through 58
Interfaces and publication	59 through 82 and Appendices A through O
Design specifications	83 through 88

82.9 Retained-specification acceptance predicate

Let $B = \{B_f, B_d, B_n, B_{nl}, B_a, B_i\}$ denote the six marked regression blocks in Chapters 83 through 88. Their preservation gate is

$$\mathrm{Accept}_{\mathrm{reg}} = \bigwedge_{b \in B} \left(P_b \wedge M_b \wedge R_b \wedge S_b \wedge X_b\right),$$

where P_b is standalone and integrated parity, M_b balanced mathematical markup, R_b balanced markers and retained numbering, S_b source faithfulness, and X_b explicit exactness, failure, and example coverage. This is a preservation predicate, not `CompleteGuide`.

> **Chapter 82 boundary.** Reference coverage, replay, site and PDF quality, source testing, platform qualification, and external publication remain distinct claims. The design specifications preserve their requirements without asserting that an absent record has been supplied.

83. Foundations, refinement, coordinates, and analysis-synthesis

Specifications for foundations and coordinate systems

The sections below deepen the existing topology without creating new public claims. A displayed equation is either an implemented finite rule, a validation invariant, or a marked example target. Numerical equality always means a residual comparison at an explicit tolerance.

Chapter 0: Coverage contract

0.A Notation and typed sets

- Public surfaces:
$$\mathcal{S}_s, \mathcal{S}_b, \mathcal{S}_r, \mathcal{C}, \mathcal{E}, \mathcal{V}.$$
- Canonical symbol record:
$$s = (N, n, k, m, q, \sigma, \ell),$$
where N is export namespace, n name, k kind, $m.q$ implementation identity, σ reflected signature, and ℓ stability label.
- Canonicalization map:
$$\pi : s \mapsto (m, q), \qquad \text{Aliases}(m, q) = \pi^{-1}(m, q).$$
- Guide anchor map:
$$\alpha : \mathcal{S}_s \cup \mathcal{S}_b \cup \mathcal{S}_r \to \mathcal{A}.$$

0.B Defining audit algorithm

1. Read each frozen API manifest and each defining module.
2. Normalize aliases by π, retaining every exported path.
3. Assign one canonical anchor $\alpha(s)$ per implementation identity.
4. Attach signatures, examples, CLI routes, tests, theorem identifiers, codecs, and failure contracts as edges in a documentation graph.
5. Reject the build if an exported symbol has no anchor or if one alias resolves to incompatible cards.

0.C Hypotheses and boundaries

- Manifest membership is version-specific: $v = 1.0.0b4$.
- Private names are excluded unless they define behavior observable through a public symbol.
- Historical contracts are evidence, not current API definitions.
- A finite certificate is not promoted to a continuum or asymptotic theorem.

0.D Coverage invariants and certificate target

- Unique placement:
$$\forall s, \quad |\alpha(s)| = 1.$$
- Example placement and CLI exercise:
$$\forall e \in \mathcal{E} : \ |\,\text{GuideMap}(e)| \geq 1, \qquad \forall c \in \mathcal{C} : \ |\,\text{Recipe}(c)| \geq 1.$$
- Recorded inventory target: 131 stable, 83 beta, 1,312 research entries. 201 public implementation owners. 65 shipped executable examples.
- `[PLANNED]` A later guide build emits a machine-readable coverage ledger L_{guide} and digest $h(L_{guide})$.

0.E Failure and edge cases

- Alias collision: equal export path, unequal implementation identity.
- Signature drift: manifest $\sigma_m \neq \sigma_{runtime}$.
- Orphan example, command, codec, theorem identifier, or public module.
- Duplicate canonical anchors or an anchor that silently merges semantically distinct overloads.

0.F Example specifications

- **E0: canonical alias:** input `stable.CoordinateTransform`, `v1.CoordinateTransform`, `api.v1.CoordinateTransform`. Assert one implementation card and three aliases.
- **E1: coverage row:** choose s. Resolve $s \to \alpha(s) \to \{\sigma, H, C, E, T\}$. Assert all required fields are populated.
- **E2: deliberate drift:** perturb one reflected parameter. Expect manifest comparison failure with the exact symbol path.

Chapter 1: Editorial and symbolic grammar

1.A Notation and type grammar

- Exact scalar and finite containers:
$$q \in \mathbb{Q}, \quad v \in \mathbb{Q}^n, \quad A \in \mathbb{Q}^{m \times n}.$$
- Numerical scalar and arrays:
$$z \in \mathbb{R} \text{ or } \mathbb{C}, \quad A \in \mathbb{F}^{m \times n}, \quad \mathbb{F} \in \{\mathbb{R}, \mathbb{C}\}.$$
- Symbol card type:
$$\mathfrak{S} = (\text{id}, \sigma, X, Y, f, H, I, \chi, R, F, P, B, K, E).$$
- Example card type:
$$\mathfrak{E} = (I, H, A, O, C, F, R), \qquad E_j \in \{E_0, E_1, E_2, E_3, E_4\}.$$

1.B Defining editorial rules

- Every callable is displayed as a typed map $f : X \to Y$, then as its exact Python signature.
- Every class separates constructor invariants, immutable/public fields, properties, methods, and returned record types.
- Every equality is tagged `[EXACT]`, `[NUMERICAL]`, `[CONDITIONAL]`, `[HEURISTIC]`, or `[PLANNED]`.
- `Certificate.require()` is shown once in every certificate-producing family.
- Complexity is stated only for implemented finite algorithms and includes explicit resource guards when present.

1.C Hypotheses and conventions

- Vectors are columns. Matrices act on the left. A^* is conjugate transpose.
- Polynomial tuples use ascending powers.
- Signal convolution is signal-first.
- Exact rational objects are not silently converted into numerical proof records.
- Numerical tolerances bound residuals. They do not redefine exact equality.

1.D Documentation invariants

- Domain/codomain dimensions in prose match runtime dimension checks.
- All unchecked assumptions occur inside the card and certificate discussion.
- Each example identifies expected exactness and backend.
- Portable objects list tag, schema version, fields, and rejection rules.

1.E Failure and edge cases

- Ambiguous use of "projector" for a redundant reconstruction component.
- Reversing transform composition or path application order.
- Omitting the distinction between transpose and adjoint.
- Calling a conditional result exact because its numerical residual is small.

1.F Example specifications

- **E0: exact card:** $f : \mathbb{Q}^2 \to \mathbb{Q}^2$, $f(v) = Av$. Show hypotheses, inverse invariant, exact return.
- **E1: numerical card:** $f : \mathbb{C}^n \to \mathbb{C}^n$. Show $\|f^{-1}f - I\| \le \tau$ and condition number.
- **E2: conditional card:** value x, checked finite identities, nonempty $H_?$. Assert the conclusion remains conditional.

Part I: Orientation and first execution

Chapter 2: What Komorebi Math is

2.A Notation and typed pipeline

- Unified stages:

$$\mathsf{U} = \{\text{seed}, \text{address}, \text{operate}, \text{project}, \text{transfer}, \text{refine}, \text{constrain}, \text{complete}, \text{reconstruct}\}.$$

- Finite stage chain:

$$S \to W \to \rho(W) \to (P_\alpha x)_\alpha \to (Q_\beta T P_\alpha)_{\beta,\alpha} \to \mathcal{R} \to \Delta \to x.$$

- `UnifiedSystem` data:

$$U = (G, \rho, \Sigma, P, \{T_i\}, \mathcal{T}, M),$$

  where fields may be absent and are validated only when present.

2.B Implemented maps

- `validate: UnifiedSystem -> Certificate`.
- `analyze: X -> PacketCoordinates`, requiring a packet system.
- `block_operator: name -> PacketOperator`, requiring the named ambient operator.
- `refine_coordinates, coarsen_coordinates: (c,i,j) -> c'`, requiring a refinement tower.
- `obstruction: (L,d) -> LinearObstructionProblem`.
- `summary: UnifiedSystem -> dict[str, Any]`.

2.C Defining equations and composition boundaries

- Grammar relation check: $\rho(\sum_w c_w w) = 0$.
- Seed condition: $\text{span}\{as : a \in \mathcal{A}, s \in S\} = X$.
- Packet condition: $P_\alpha^2 = P_\alpha$, $P_\alpha P_\beta = 0$, $\sum_\alpha P_\alpha = I$.
- Transfer block: $T_{\beta\alpha} = Q_\beta T P_\alpha$.
- The `COMPLETE` stage is explicitly marked `implemented_in_alpha=False`. The guide must not imply `UnifiedSystem` executes a general inverse-limit completion.

2.D Hypotheses and certificates

- Realization grammar identity must be object-consistent with the declared grammar.
- Seed and packet ambient dimensions must agree.
- Named operators must be square on the packet ambient space.
- Declared packet system must equal a level of the refinement tower up to `PacketSystem.equivalent`.
- `validate()` aggregates relation, seed, packet, and tower residuals. Exactness becomes numerical if any constituent certificate is numerical.

2.E Failure and edge cases

- Missing packet system for analysis or block resolution.
- Unknown operator name.
- Mismatched grammar/realization or ambient dimension.
- Treating the common language as a claim that domain-specific mathematical meanings are identical.

2.F Example specifications

- **E0: minimal finite system:** $G = \langle a : X \to X \mid a^2 - a = 0 \rangle$. One seed. Two packets. Validate and summarize.
- **E1: full finite path:** $x \mapsto (P_0 x, P_1 x) \mapsto (T_{ij}) \mapsto \mathcal{R}_{0\to 1} \mapsto x$.
- **E2: incomplete system:** instantiate metadata only. Show which summary fields are empty without claiming validation of absent structures.
- **E3: incompatible tower:** choose P not equivalent to any tower level. Expect `DimensionError`.

Chapter 3: Installation and environment profiles

3.A Environment objects and dependency lattice

- Runtime floor: Python ≥ 3.11.
- Core environment:

$$\mathcal{P}_0 = \{\texttt{komorebi-math}, \texttt{numpy}\}.$$

- Selected optional-dependency sets (not a complete ledger of the declared `plot`, `jax`, `torch`, `test`, `docs`, `build`, `dev`, and `all` extras):

$$\begin{aligned}
\mathcal{P}_{exact} &= \{\texttt{sympy}\},\\
\mathcal{P}_{scientific} &= \{\texttt{scipy}, \texttt{mpmath}\},\\
\mathcal{P}_{yaml} &= \{\texttt{PyYAML}\},\\
\mathcal{P}_{autodiff} &= \{\texttt{jax}, \texttt{torch}\},\\
\mathcal{P}_{publication} &= \{\texttt{pytest}, \texttt{build}, \texttt{setuptools}, \texttt{wheel}, \texttt{mkdocs}, \texttt{sympy}, \texttt{scipy}, \texttt{mpmath}, \texttt{PyYAML}\}.
\end{aligned}$$

3.B Settings maps

- `Settings = (backend: str, tolerance: float, validate_on_construction: bool)`.
- Default:
$$s_0 = (\texttt{"numpy"}, 10^{-10}, \text{True}).$$
- `get_settings: 1 -> Settings`.
- `configure(*, backend: str | None = None, tolerance: float | None = None, validate_on_construction: bool | None = None) -> Settings` mutates the current context.
- `settings_context(**changes: object) -> Iterator[Settings]` is the decorated context-manager signature. Entering yields the changed settings and exit restores the prior context token.

3.C Installation and probe algorithm

1. Install wheel, editable source, or rebuilt sdist into a chosen interpreter.
2. Verify `python -c "import komorebi_math"` and package identity.
3. Evaluate `available_backends()` without importing absent optional packages.
4. Select a backend only at the first operation requiring it.
5. Run the smallest relevant certificate before a full publication gate.

3.D Hypotheses and invariants

- `numpy` is mandatory. Other dependencies are optional and feature-scoped.
- `settings_context` restores the exact prior context even after an exception.
- Backend discovery reports presence, not hardware qualification.
- CUDA/CuPy presence does not certify a CUDA execution path or numerical equivalence.

3.E Failure and edge cases

- Missing optional backend produces `BackendUnavailableError`, not a core import failure.
- Unknown backend name is rejected.
- Environment contains incompatible third-party versions even if import succeeds. Use `pip check` in qualification.
- A PowerShell prompt can display a duplicated conda prefix without implying two active interpreters. Record the resolved `python.exe` path.

3.F Example specifications

- **E0: core:** clean environment $\mathcal{P}_0$. Import, print version, perform one NumPy certificate.
- **E1: exact:** install `.[exact]`. `get_backend("sympy").exact=True`. Certify one rational packet system.
- **E2: scoped settings:** outer s_0, inner tolerance 10^{-14}, forced exception, assert post-context state s_0.
- **E3: absent optional:** probe `sympy=False`. Request it. Assert isolated `BackendUnavailableError` while NumPy remains usable.

Chapter 4: First verified program

4.A Minimal data and maps

- Vector $x \in \mathbb{F}^n$.
- Complete packet system $P = (P_0, \dots, P_{r-1})$.
- Analysis and synthesis:

$$\mathcal{A}_P(x) = (P_\alpha x)_\alpha, \qquad \mathcal{S}_P((x_\alpha)) = \sum_\alpha x_\alpha.$$

- Exact basis transform $T_{B \leftarrow A} \in \mathrm{GL}_n(\mathbb{Q})$.

4.B Execution algorithm

1. Query package identity and enabled feature records through the CLI.
2. Construct two coordinate projectors and validate `PacketSystem.certificate().require()`.
3. Analyze x, reconstruct $\hat{x}$, assert $\hat{x} = x$ or residual $\leq \tau$.
4. Apply a rational coordinate transform, invert it, and assert exact roundtrip.
5. Inspect `valid`, `exactness`, `residuals`, `assumptions_checked`, `assumptions_unchecked`, and `theorem_id`.

4.C Hypotheses and invariants

- Projectors share one backend and ambient dimension.
- Exact coordinate transform is square and invertible.
- Packet coordinates are bound to their originating `PacketSystem`.
- The public capability tour is deterministic for its declared finite inputs. Its recorded digest is part of release evidence, not a mathematical theorem.

4.D Failure and edge cases

- Invalid projector family fails construction when validation is enabled.
- Supplying coordinates from another packet system fails synthesis/application compatibility.
- A singular exact transform cannot produce an inverse.
- `komorebi features` reports feature status. It does not install optional dependencies.

4.E Example specifications

- **E0: packet roundtrip:** $P_0 = \mathrm{diag}(1, 0)$, $P_1 = \mathrm{diag}(0, 1)$, $x = (2, 3)^T$. Expect packets $(2, 0)^T, (0, 3)^T$.
- **E1: exact shear:** $T = \begin{bmatrix} 1 & 1/2 \\ 0 & 1 \end{bmatrix}$. Check $T^{-1}Tv = v$ over $\mathbb{Q}^2$.
- **E2: certificate anatomy:** perturb P_1. Capture invalid residual rather than calling `.require()` first.
- **E3: capability tour:** run shipped `block21_public_capability_tour.py`. Compare the structured record to its committed finite assertions.

Part II: Trust, types, and execution semantics

Chapter 5: Certificates and result types

> **Reference scope.** Chapter 5 is authoritative for explanation and examples. This retained card is the source-based specification. It uses the certificate normal form introduced in Chapter 1 rather than defining a competing tuple notation.

5.A Runtime records and exact field map

- `Exactness` is a `StrEnum` with the five values

$$\mathsf{X} = \{\text{exact}, \text{numerical}, \text{conditional}, \text{heuristic}, \text{planned}\}.$$

  The runtime enum has no ordering or promotion operation. Chapter 1's strongest-to-weakest ordering is an editorial reading rule only.
- `Certificate` is a frozen, slotted dataclass with the runtime fields

$$\mathfrak{K} = (v, e, r, \tau, h_c, h_u, d, t, m),$$

  mapped exactly as follows:

Symbol	Runtime field	Runtime annotation/default
v	`valid`	`bool`
e	`exactness`	`Exactness`
r	`residuals`	`dict[str, float]`, default `{}`
τ	`tolerance`	`float | None`, default `None`
h_c	`assumptions_checked`	`tuple[str, ...]`, default `()`
h_u	`assumptions_unchecked`	`tuple[str, ...]`, default `()`
d	`diagnostics`	`dict[str, Any]`, default `{}`
t	`theorem_id`	`str | None`, default `None`
m	`message`	`str`, default `""`

- `ComputationResult[T]` is a separate frozen, slotted dataclass with exactly `value: T` and `certificate: Certificate`. It adds the properties `valid` and `exact`, plus `require()`.
- `ConditionalResult[T]` is another frozen, slotted dataclass with exactly `value: T` and `certificate: Certificate`. Its only custom runtime operation is `__post_init__`, which requires `certificate.exactness is Exactness.CONDITIONAL`.
- `ConditionalResult` is not a subclass of `ComputationResult`. In this release it does **not** provide `.valid`, `.exact`, `.require()`, `.to_dict()`, or `.to_json()`.

5.B Operational semantics

- Certificate acceptance is

$$\text{Certificate.require}(\mathfrak{K}) = \begin{cases} \mathfrak{K}, & v = \mathsf{True}, \\ \text{ValidationError}(m), & v = \mathsf{False},\ m \neq \texttt{""}, \\ \text{ValidationError}(\texttt{"certificate validation failed"}), & v = \mathsf{False},\ m = \texttt{""}. \end{cases}$$

- `ComputationResult.valid` returns `certificate.valid`.
- `ComputationResult.exact` is the identity test `certificate.exactness is Exactness.EXACT`. It does not also test validity.
- `ComputationResult.require()` first calls `certificate.require()` and then returns the stored value.
- `Certificate.to_dict()` converts the dataclass recursively and replaces the enum object with `exactness.value`.
- `Certificate.to_json(indent=2)` serializes that mapping with sorted keys and `default=str`. These serialization methods belong to `Certificate`, not to either result wrapper.

5.C Validity, exactness, and assumptions

- `Certificate` stores an API-supplied Boolean. The record class does not recompute `valid` from residuals, tolerance, assumptions, theorem ID, or diagnostics.
- The dataclass does not enforce cross-field implications. In particular:

$$v \not\Rightarrow e = \text{exact}, \qquad e = \text{exact} \not\Rightarrow v, \qquad r = 0 \not\Rightarrow e = \text{exact}.$$

- An identity-checking exact producer normally sets validity from exact finite equality. A numerical producer normally sets validity from a documented scaled residual bound. Those are producer contracts, not behavior supplied by the record class.
- `ConditionalResult` enforces only the exactness category. It does not require `certificate.valid` to be true and does not require a nonempty `assumptions_unchecked` tuple. Producing APIs remain responsible for those stronger domain-specific invariants.
- A valid conditional certificate means that the implemented finite construction passed under its declared record. It does not establish the truth of unchecked assumptions.

5.D Hypotheses and invariants

- Residuals are annotated as machine-readable floats, including exact-zero residuals represented as `0.0`. Runtime construction does not independently validate every dictionary value against that annotation.
- Exactness is determined by the executed operation and scalar/backend path, not by a user's desired conclusion.
- A theorem ID names an implemented finite theorem or contract when a producer supplies one. The field may be absent.
- Diagnostics are explanatory data. They neither override `valid` nor promote `exactness`.
- JSON serialization of arbitrary diagnostic objects uses string fallback and is not a general object-reconstruction protocol.

5.E Failure and edge cases

- Invalid certificate with an empty message uses the standard failure detail.
- Constructing `ConditionalResult` from exact, numerical, heuristic, or planned exactness raises the built-in `ValueError`, not `ValidationError`.
- `ComputationResult.exact` may be true even when its certificate is invalid. Use `.valid` or `.require()` for acceptance.
- Calling `.require()` is an explicit control-flow boundary. Inspecting a certificate without calling it leaves the caller responsible for the next action.
- Portable persistence of domain records is a separate codec contract. A certificate JSON rendering alone is not that persistence guarantee.

5.F Example checklist

- **E0: exact record:** valid exact certificate. `.require()` returns the same certificate object.
- **E1: numerical record:** residual below an explicit tolerance. Validity is true while exactness remains numerical.
- **E2: rejected record:** residual-producing API returns `valid=False`. Both certificate and `ComputationResult` acceptance raise `ValidationError`.
- **E3: conditional record:** construct `ConditionalResult` from a conditional certificate with visible unchecked assumptions, then contrast its intentionally smaller interface with `ComputationResult`.
- **E4: serialization boundary:** show stable field ordering in JSON without claiming arbitrary diagnostic reconstruction.

Chapter 6: Errors, assumptions, and resource guards

> **Reference scope.** Chapter 6 is authoritative. This checklist distinguishes the frozen stable exception aliases from additional implementation-module classes and from built-in or third-party exceptions that remain observable on particular paths.

6.A Exception tiers and inheritance

- The frozen stable surface re-exports these seven package-specific classes: `KomorebiMathError`, `ValidationError`, `DimensionError`, `BackendUnavailableError`, `ExactnessError`, `AssumptionError`, and `ResourceLimitError`.
- `komorebi_math.exceptions` additionally defines `RelationError` and `FeatureNotImplementedError`. They belong to the implementation-module hierarchy but are not members of the frozen stable `__all__` list.
- The source inheritance graph is

  `KomorebiMathError`,

  `ValidationError` $\supset$ {`DimensionError`, `RelationError`, `ResourceLimitError`},

  `BackendUnavailableError`, `ExactnessError`, `AssumptionError`, `FeatureNotImplementedError`.

- `ValidationError` also inherits from the built-in `ValueError`. `FeatureNotImplementedError` also inherits from built-in `NotImplementedError`.
- Catching `KomorebiMathError` catches this package-specific hierarchy. It does not catch every exception observable from every public call.

6.B Guard maps and `CoordinateBudget`

- The guide uses these editorial guard classes:

$$
\begin{aligned}
&V : X \to X \cup \texttt{ValidationError},\\
&D : (\text{observed shape}, \text{required shape}) \to \{\checkmark, \texttt{DimensionError}\},\\
&H : (x, h) \to \{\checkmark, \texttt{AssumptionError}\},\\
&B : (request, limit) \to \{\checkmark, \texttt{ResourceLimitError}\}.
\end{aligned}
$$

- `[RESEARCH]` `CoordinateBudget` is a frozen, slotted record with defaults

$$(d_{matrix}, d_{poly}, h, n_{exact}) = (512, 512, 4096, 1{,}000{,}000).$$

- Its constructor rejects Boolean, non-integer, or nonpositive limits with `ValidationError`.
- `require_matrix(rows, columns, operation=...)` rejects negative dimensions with `ValidationError`. It raises `ResourceLimitError` when either dimension or the exact-entry product exceeds the declared budget.
- `require_degree(...)` and `require_horizon(...)` reject Boolean, non-integer, or negative requests with `ValidationError`, then use `ResourceLimitError` for an otherwise legal request above the configured maximum.

6.C Failure discipline

The decision rule is:

1. Reject malformed syntax, invalid scalar categories, and incompatible shapes before expensive work where the implementation supplies those guards.
2. Reject unmet preconditions with the most specific documented exception.
3. Apply an explicit resource budget before materializing a large exact object.
4. Return an invalid certificate when a legal computation completes but its post-computation identity or residual test fails.
5. Treat each source path as authoritative: not every public API normalizes every failure into this idealized sequence.

Resource limits are safety contracts. They are not numerical tolerances and do not authorize an approximation when a request is rejected.

6.D Construction-validation boundary

- `Settings.validate_on_construction` is a context-local default consulted by constructors that explicitly read it. It is not a universal switch applied automatically to every class in the package.
- Some constructors also expose a per-call `validate` argument. The function card for that constructor determines precedence and behavior.
- Disabling or deferring construction validation may make a failed certificate inspectable. It does not make an invalid mathematical relation valid, bypass later explicit `.require()` calls, or suppress unrelated Python exceptions.
- `FeatureNotImplementedError` identifies the package's deliberate planned placeholder path. An absent optional dependency instead uses `BackendUnavailableError` where the public adapter translates that condition.

6.E Built-in, third-party, and edge failures

- `ConditionalResult` constructed with nonconditional exactness raises the built-in `ValueError`.
- `register_backend` given an already registered name without `replace=True` raises the built-in `ValueError`.
- NumPy operations such as singular inversion may expose `numpy.linalg.LinAlgError` when the calling wrapper does not translate it.
- A third-party import or runtime may expose its own documented exception. The final function card records the actual boundary rather than promising a package-wide translation layer that does not exist.
- Negative sizes, ragged matrices, zero moduli, singular transforms, incompatible packet labels, and unavailable optional backends are distinct failures and require distinct examples.

6.F Example checklist

- **E0: dimensions:** legal matrix object plus incompatible vector shape. Observe `DimensionError` on the documented operation.
- **E1: assumptions:** execute one public precondition failure and preserve its actual exception class and message.
- **E2: resource:** budget dimension four, request a five-by-five exact matrix. Observe pre-materialization `ResourceLimitError`.
- **E3: certificate failure:** legal input reaches a residual check, returns an invalid certificate, and raises `ValidationError` only when acceptance is required.
- **E4: untranslated boundary:** singular NumPy inversion demonstrates why `KomorebiMathError` is not a universal catch-all.

Chapter 7: Public API tiers

Reference scope. Chapter 7 is authoritative. This checklist keeps structural API manifests, semantic fixtures, persistence contracts, and versioned-alias identity as separate evidence layers.

7.A Surfaces and source-faithful manifest records

- The research surface consists of the root and audited public domain namespaces. There is no required literal `komorebi_math.research` namespace.
- The curated beta surface is exported through `komorebi_math.beta`.
- The frozen compatibility surface is exported through `komorebi_math.stable`, with versioned aliases `komorebi_math.v1` and `komorebi_math.api.v1`.
- `PublicApiSymbol` has exactly the fields

$$M_{api} = (namespace, name, kind, implementation_module, qualname, signature).$$

- `PublicApiManifest` has the fields `package_version`, `namespaces`, `symbols`, `schema`, `schema_version`, and `stability_level`. Its digest is the SHA-256 of canonical JSON derived from those fields.
- Semantic fixtures are **not** fields of `PublicApiSymbol` or `PublicApiManifest`. Stable behavior is recorded in a separate committed semantic-fixture artifact.
- Portable codec identity and migrations are recorded in a separate persistence contract.

7.B Tier semantics

- `[STABLE]`: the frozen 1.x compatibility surface. Its contract covers the declared names and signatures plus separately qualified semantic and persistence behavior.
- `[BETA]`: a curated additive beta surface with strict signature comparison for its committed manifest. Evolution remains possible under beta release policy.
- `[RESEARCH]`: the broad current surface. Documentation remains source-faithful without silently promoting every symbol into a compatibility promise.
- API tier and mathematical evidence class are orthogonal. A stable function may return a numerical or conditional result. A research function may execute exact finite algebra.
- Re-export aliases receive one canonical implementation explanation and retain alias-specific import examples.

7.C Distinct verification layers

1. `check_stable_api()` loads the committed stable manifest, creates the current `komorebi_math.stable` manifest, and performs strict signature comparison.
2. `run_stable_semantics_fixture()` executes the separate committed stable behavior fixture.
3. `check_portable_codec_contract()` checks the separate codec and migration contract.
4. `check_stable_contract()` aggregates those structural, semantic, and persistence reports into one `StableContractReport`.
5. `run_stable_contract_verification()` additionally checks deterministic manifest regeneration and versioned stable namespace identity.

The stable manifest itself covers `komorebi_math.stable`. It does not by itself prove that `stable`, `v1`, and `api.v1` expose identical objects.

Versioned-alias evidence is the separate finite check

$$\begin{aligned} &\texttt{stable.__all__} = \texttt{v1.__all__} = \texttt{api.v1.__all__}, \\ \forall n: \quad &\texttt{getattr(stable,n)} \ \textbf{is} \ \texttt{getattr(v1,n)}, \\ &\texttt{getattr(v1,n)} \ \textbf{is} \ \texttt{getattr(api.v1,n)}. \end{aligned}$$

7.D Hypotheses and invariants

- Structural compatibility certifies the selected exported names, kinds, and signature text under the supported reflection policy. Its own certificate explicitly leaves behavioral and mathematical equivalence unchecked.
- Signature normalization absorbs supported interpreter representation differences without authorizing a callable-semantic change.
- A data symbol has no callable signature. Its value/type stability belongs to its documented semantic or persistence contract.
- Historical manifests remain migration evidence. They must not be described as the live `1.0.0b4` manifest.
- Stable alias identity is an API fact, not a claim that differently named mathematical concepts are interchangeable.

7.E Failure and edge cases

- Assuming every research symbol is beta or stable.
- Calling the root surface a literal `komorebi_math.research` namespace.
- Claiming semantic-fixture execution is part of a `PublicApiManifest` record.
- Claiming `api check --stable` alone proves semantic behavior, portable persistence, or versioned-alias identity.
- Duplicating prose for each stable alias and allowing those pages to drift.
- Treating historical version assertions or predecessor fixtures as current.
- Treating `__all__` order as mathematical meaning beyond an explicitly frozen compatibility record.

7.F Example checklist

- **E0: tri-alias identity:** import the frozen surface through all three paths. Compare `__all__`, object identity, and representative signatures.
- **E1: tier contrast:** place one stable symbol beside one research-only symbol while keeping mathematical exactness labels independent.
- **E2: structural check:** run `api check --stable` and inspect its manifest digests and zero drift fields.
- **E3: aggregate check:** run `verify stable` and distinguish structural, semantic, persistence, aggregate, and alias-identity checks.
- **E4: drift fixture:** compare a controlled mock manifest with one changed signature and report the exact failed field without mutating the shipped contract.

Chapter 8: Backends, scalar domains, and coercion

> **Reference scope.** Chapter 8 is authoritative. This checklist separates import presence, backend registry membership, backend selection, scalar-domain semantics, and hardware/runtime qualification.

8.A Core linear-algebra interface

- `LinearAlgebraBackend` is an abstract base class with data attributes `name` and `exact`, and these abstract methods: `as_matrix`, `as_vector`, `shape`, `zeros`, `eye`, `matmul`, `add`, `subtract`, `scale`, `transpose`, `adjoint`, `inverse`, `rank`, `norm`, `is_zero`, `column_stack`, `nullspace`, `kron`, and `copy`.
- Selection and discovery functions are

$$\begin{aligned}
\texttt{get_backend} &: \texttt{str | LinearAlgebraBackend | None} \to \texttt{LinearAlgebraBackend},\\
\texttt{register_backend} &: (\texttt{LinearAlgebraBackend}, \texttt{replace: bool}) \to \texttt{None},\\
\texttt{available_backends} &: () \to \texttt{dict[str,bool]},\\
\texttt{backend_name} &: \texttt{Any} \to \texttt{str}.
\end{aligned}$$

- The registry initially contains one `NumpyBackend()` under the name `"numpy"`.
- The annotation on `register_backend` is the public contract. Registration checks name collision but is not an exhaustive runtime conformance audit of every backend method.

8.B Presence, registry, selection, and qualification

- `get_backend(None)` selects NumPy. It does not consult `get_settings().backend` in this release.
- Passing an existing `LinearAlgebraBackend` instance to `get_backend` returns that instance.
- Requesting `"sympy"` lazily imports and registers `SympyBackend` when SymPy is installed. Otherwise it raises `BackendUnavailableError`.
- An unknown or currently unregistered name raises `BackendUnavailableError`.
- `register_backend` requires a unique name unless `replace=True`. A duplicate name raises built-in `ValueError`.
- `available_backends()` is a mixed import-presence probe with exactly these keys in this release:

$$\{\texttt{numpy}, \texttt{sympy}, \texttt{scipy}, \texttt{mpmath}, \texttt{cupy}\}.$$

  It is not a list of registered `LinearAlgebraBackend` instances. It does not probe JAX or PyTorch.
- `backend_name(value)` returns `"sympy"` only when the root defining module of `type(value)` is `sympy`. Every other value is classified as `"numpy"`. This is a lightweight heuristic, not universal dispatch.
- Import presence does not establish a working device, compatible runtime, numerical equivalence, or hardware qualification.

8.C Scalar-domain realizations and optional feature adapters

- `NumpyBackend` defaults to `dtype=np.complex128` for `as_matrix`, `as_vector`, `zeros`, and `eye`. This is a converter/constructor default, not a claim that every arbitrary ndarray supplied to every primitive is silently recast first.
- NumPy coercion of rational Python objects into the default array domain loses exact rational representation.
- `SympyBackend` uses SymPy matrices for exact symbolic rank, nullspace, inverse, and structural zero comparison.
- The `Fraction`-based exact coordinate layer used by `ExactCoordinateTransform` is independent of SymPy and remains available in the core installation.
- SciPy and mpmath are optional scientific/analytic dependencies, not core `LinearAlgebraBackend` implementations.
- JAX and PyTorch are optional directional-differentiation providers. They are not reported by `available_backends()` and are not core linear-algebra backend names.
- CuPy is an optional GPU execution dependency used by selected algorithm paths. Presence alone neither registers it as the core backend nor proves a working CUDA path.

8.D Numerical and exact operation rules

- NumPy zero testing uses

$$\text{is_zero}(A;\tau) \iff \text{np.linalg.norm}(A) \le \tau, \qquad \tau = 10^{-10} \text{ when omitted},$$

  where `np.linalg.norm` gives the Euclidean norm for a vector and the Frobenius norm for a matrix when no `ord` is supplied.
- NumPy `rank` delegates to `np.linalg.matrix_rank`. NumPy `nullspace` performs an SVD and, when tolerance is omitted, uses shape, floating epsilon, and the largest singular value to determine numerical rank.
- NumPy `transpose(A)` is `A.T`. `adjoint(A)` is `A.conj().T`.
- SymPy rank, nullspace, and zero testing ignore the numerical tolerance argument. Zero testing is structural equality with a zero matrix and does not promise a separate global simplification theorem.
- `SympyBackend.norm()` returns a Python float after numerical evaluation for a nonzero matrix, even though the backend's algebraic identity checks are exact.
- Core finite algebra, realization, seed, packet, operator, transfer, and morphism certificates use `backend.exact` to choose exact versus numerical certificate labeling on those implemented paths. Other domains retain their own producer-specific rules.

8.E Failure and edge cases

- SymPy absent. Unknown backend name. Duplicate registration.
- Malformed custom backend accepted by a non-exhaustive registry boundary and failing only when a missing or incompatible operation is called.
- Exact rational input coerced into NumPy and becoming numerical complex data.
- Near-singular numerical rank/nullspace changing with the tolerance rule.
- Singular NumPy inverse exposing `numpy.linalg.LinAlgError`.
- Symbolically equivalent but not structurally identical SymPy expressions requiring domain-appropriate simplification before a structural zero check.
- `backend_name` classifying an unfamiliar array/object type as NumPy.
- CuPy import succeeding while CUDA execution or device qualification fails.

8.F Example checklist

- **E0: NumPy core path:** select the default backend, materialize a matrix, inspect dtype, rank, adjoint, nullspace, and one numerical zero residual.
- **E1: exact/numerical mirror:** use the same rational nilpotent matrix through NumPy and optional SymPy. Preserve numerical versus exact certificate labels.
- **E2: rank boundary:** $A_\varepsilon = \operatorname{diag}(1, \varepsilon)$. Vary the numerical tolerance without claiming exact singularity.
- **E3: adjoint boundary:** complex matrix with $A^T \neq A^* = \overline{A}^T$.
- **E4: lazy optional failure:** request SymPy only after inspecting presence. If absent, retain `BackendUnavailable Error` while proving NumPy remains usable.
- **E5: feature-adapter boundary:** distinguish SciPy/mpmath/JAX/Torch/CuPy feature availability from the core linear-algebra registry and from hardware qualification.

Part III: Operator, packet, and refinement foundations

Chapter 9: Typed operator grammars

> **Reference scope.** Chapter 9 is authoritative for construction, composition, realization, relation validation, executable examples, and invalid paths. This retained card is the source-based specification. Paths remain formal typed words rather than quotient classes. Arrow names must be unique, while repeated vertex inputs are normalized by first occurrence. `Relation Residual` is available from the core and stable namespaces but is not a package-root export.

9.A Notation, maps, and definitions

- `Arrow(name,source,target)` realizes $a : X \to Y$. `OperatorWord(arrows,source,target)` stores arrows in application order.
- `OperatorGrammar.word(self, *arrow_names: str, vertex: str | None = None) -> OperatorWord` realizes $(a_1, \dots, a_k) \mapsto w : X_0 \to X_k$, valid when $t(a_j) = s(a_{j+1})$. `vertex` specifies the empty word's object.
- Identity is the empty word 1_X. A relation is `OperatorRelation(name, terms)`:

$$R = \sum_w c_w w : X \to Y.$$

- `OperatorRealization` supplies dimensions d_X and matrices $\rho(a) \in \mathbb{F}^{d_{t(a)} \times d_{s(a)}}$:

$$\rho(a_1 \cdots a_k) = \rho(a_k) \cdots \rho(a_1), \qquad \rho(1_X) = I_{d_X}.$$

- Reference cards: grammar properties and `arrow`, `identity`, `word`, `relation`, `add_relation`, `paths`. Realization `evaluate`, `relation_matrix`, `relation_residuals`, `relation_certificate`, `path_operators`, `loop_operators`. `RelationResidual`.

9.B Hypotheses, invariants, and certificates

- Vertex and arrow names are nonempty. Repeated vertex inputs are silently deduplicated in first-occurrence order, while arrow names must be unique. Relation terms are nonempty and parallel.
- Realization space keys equal grammar vertices. Operator keys equal arrows. Every matrix has the typed shape.
- `[EXACT]` relation iff $\rho(R) = 0$. `[NUMERICAL]` relation iff $\|\rho(R)\| \le \tau$.
- `relation_certificate()` records theorem `PSC-I-T01`. Constructor can raise `RelationError` immediately when `validate_relations=True`.
- `paths(max_length=L)` enumerates typed paths and does not quotient by declared relations.

9.C Failure and edge cases

- Empty word without `vertex`, noncomposable arrows, nonparallel relation, missing/extra realization key, wrong matrix shape, failed declared relation.
- `word(a,b)` realizes as BA, not AB.

9.D Example checklist

- **E0:** $X \xrightarrow{a} Y \xrightarrow{b} Z$. Assert $\rho(ab) = BA$, enumerate paths through length two.
- **E1:** one loop T with $T^3 - T = 0$. Inspect zero relation matrix and `PSC-I-T01`.
- **E2:** exact nilpotent $N^2 = 0$ under SymPy and numerical mirror under NumPy.
- **E3:** attempt $b + a$ with unequal endpoints. Expect `RelationError`.

Chapter 10: Operator algebras and seed systems

> **Reference scope.** Chapter 10 is authoritative for finite algebra closure, seed hulls, presentations, bounded standard-seed search, and seed-rank formulas. This retained card is the source-based specification. `OperatorAlgebra` is unital by default but supports `include_identity=False`. `SeedSystem` requires at least one seed but permits an empty operator family. `AlgebraBasisResult` is a public record in this release, but no documented routine is presented as returning it.

10.A Notation, maps, and algorithms

- Generated algebra:
$$\mathcal{A} = \operatorname{span}\{I, A_i, A_iA_j, ...\} \subseteq M_n(\mathbb{F}), \qquad \dim \mathcal{A} \le n^2.$$
- `OperatorAlgebra(generators, *, backend=None, tolerance=None, include_identity=True)` closes a rank-increasing basis under pairwise multiplication. `basis`, `dimension`, `contains`, `certificate`, `seed_system`, `transfer_graph`.
- `matrix_units(n)` returns E_{ij} in row-major order with $E_{ij}E_{k\ell} = \delta_{jk}E_{i\ell}$.
- For seeds S, `SeedSystem` computes breadth-first rank-increasing orbit closure:
$$\mathcal{H}(S) = \operatorname{span}\{As : A \in \mathcal{A}, s \in S\}.$$
- Cover `hull_vectors`, `hull_basis`, `hull_dimension`, `is_generating`, `generation_certificate`, `orbit_layers`, `presentation`, `minimum_standard_basis_seeds`, `cyclic_span`, and rank-bound functions.

10.B Hypotheses, invariants, and certificates

- Generators are nonempty same-size square matrices. Seeds share ambient dimension.
- Algebra `PSC-I-D05` checks generator inclusion, basis independence, and multiplicative closure. It checks identity inclusion only when `include_identity=True`.
- Seed `PSC-I-T02` is valid iff $\dim \mathcal{H}(S) = n$. `FreePresentation` records evaluation map, kernel, rank, domain/-codomain dimensions.
- Exact formulas:
$$r_{ss} = \max_i \lceil m_i/d_i \rceil, \quad \max_i r_i \le r_{shared} \le \sum_i r_i, \quad r(V \otimes W) \le r(V)r(W).$$

- `single_operator_seed_rank` requires the `exact` SymPy dependency and a square matrix whose entries convert into `sympy.QQ`. It computes $\max_p \operatorname{nullity}(p(T))/\deg p$.

10.C Failure and edge cases

- Non-square/mixed-size generators. Empty family. Incompatible seed dimension. Missing SymPy or single-operator entries that do not convert into `sympy.QQ`.
- Standard-basis subset search is exponential and guarded above dimension 12 unless `force=True`.

10.D Example checklist

- **E0:** construct `matrix_units(2, backend="sympy")` and `OperatorAlgebra(..., backend="sympy")`. $\{E_{ij}\}_{i,j<2}$ generates M_2, with dimension four and exact closure. With the default NumPy backend the same closure certificate is numerical.
- **E1:** a 3×3 Jordan block generates span$\{I, J, J^2\}$. One cyclic seed yields three orbit layers.
- **E2:** diagonal action with a coordinate seed. Compute positive hull codimension and failed generation certificate.
- **E3:** compare shared direct-sum bounds, independent sum rank, and tensor upper bound for $(r_1, r_2) = (2, 3)$.

Chapter 11: Packet systems and transfer blocks

Reference scope. Chapter 11 is authoritative for packet coordinates, ambient transfer blocks, graph/path orientation, support preorders, and validated morphisms. This retained card is the source-based specification. Blocks use target-by-source indexing while graph adjacency uses source-by-target indexing. Algebraic and Hilbert certificates remain separate. `TransferPath` is available from its domain and stable namespaces rather than the package root.

11.A Notation, maps, and defining equations

- `PacketSystem(P_0,...,P_{r-1})` and embedded `PacketCoordinates`:

$$P_i^2 = P_i, \quad P_i P_j = 0\ (i \neq j), \quad \sum_i P_i = I, \quad \mathcal{A}_P(x) = (P_i x)_i, \quad \mathcal{S}_P(c) = \sum_i c_i.$$

- Cover packet `analyze`, `synthesize`, `support`, `packet_dimensions`, `aggregate`, `tensor`, `dual`, `equivalent`, `transport`, `restrict`, `quotient`, `block_diagonal_projection` and coordinate access/reconstruction.
- `PacketOperator.from_operator(operator, source, target=None) -> PacketOperator` stores ambient blocks (with `target=source` when omitted)

$$T_{ji} = Q_j T P_i, \qquad T = \sum_{j,i} T_{ji}, \qquad (ST)_{ki} = \sum_j S_{kj} T_{ji}.$$

- Cover reconstruct/apply/coordinate apply, compose, adjoint, tensor, power, norms, diagonal test, support mask.
- `TransferGraph` uses adjacency `[source][target]`. `TransferPath` and

$$(T^\ell)_{ba} = \sum_{\pi: a \to b, |\pi| = \ell} T_\pi$$

  are implemented by `path_contributions` and `block_from_paths`.
- `SupportPreorder` requires a nonempty square reflexive-transitive relation. It computes mutual-reachability classes, the strict class graph, and the corresponding strict-ideal nilpotency bound.

11.B Hypotheses, invariants, and certificates

- Algebraic packet `PSC-I-T20` checks idempotence, annihilation, completeness. `hilbert_certificate` separately checks $P_i = P_i^*$.
- Packet-operator `PSC-I-T27` checks that reprojected reconstruction equals every stored block.
- Path sum returns `ComputationResult` with `PSC-I-T29` exact/numerical comparison to `power(length)`.
- Aggregation groups partition indices exactly once. Restriction requires $RJ = I$ and invariance. Quotient requires $QS = I$ and compatibility.
- Validated operator, packet, and seed morphism factory routes enforce $\Phi A = A'\Phi$ and the declared packet/seed image laws. Direct dataclass construction does not perform those checks.

11.C Failure and edge cases

- Oblique packets can pass algebraic certification and fail Hilbert certification.
- `PacketSystem.dual()` transposes packet matrices. It is not the conjugate-adjoint operation used by Hilbert certification.
- Coordinates tied to another system. Nonpartition aggregation. Nonendomorphism powers/paths. Negative power/length. Support-mask orientation reversal.

11.D Example checklist

- **E0:** three coordinate packets, sparse T, reconstruct and apply via blocks.
- **E1:** diamond graph with two length-two paths. Show both ordered products sum to $(T^2)_{30}$.
- **E2:** aggregate $1 + 1 + 1$ packets to $2 + 1$. Reflect support and certify packet morphism.
- **E3:** construct an exact oblique decomposition $P_0 + P_1 = I$ with `backend="sympy"`. Contrast `certificate()` with failed `hilbert_certificate()`.

Chapter 12: Refinement towers

> **Reference scope.** Chapter 12 is authoritative for adjacent and multilevel refinement, common refinements, weighted transfer expansions, and linear refinement defects. This retained card is the source-based specification. Weighted paths are retained only when their total weight is strictly below the requested precision. Exact p-adic qualification and conditional numerical qualification are intentionally distinct.

12.A Notation, maps, and algorithms

- `PacketRefinement(coarse=P, fine=Q, parent_of_fine=p)` encodes

$$P_a = \sum_{b:p(b)=a} Q_b.$$

- Coordinate maps:

$$c_a = \sum_{b:p(b)=a} f_b, \qquad f_b = Q_b \sum_a c_a.$$

- Cover `children`, `parent`, `certificate`, `coarsen`, `refine`, operator conversions, and `support_reflection`.
- `RefinementTower(R_0,...,R_{L-1})` supplies ordered `levels`, multilevel refine/coarsen, `descendant_address`, and the adjacent-level tower certificate.
- `common_refinement(P,Q)` uses products of commuting projectors $R_{ab} = P_a Q_b$, removes zero packets, and returns refinements back to both parents.
- `LinearRefinementDefect` records $\Delta = CT_f - T_c C$ and implements power, polynomial, resolvent, exponential/-Duhamel, and multilevel propagation identities.

- `WeightedTransferGraph` stores $T = D + \sum_e E_e$, positive weights $w(e)$, supported edges, weighted paths, and the filtered inverse

$$T^{-1} = D^{-1} + \sum_{\pi \neq \emptyset} (-1)^{|\pi|} A_\pi D^{-1}, \qquad A_e = D^{-1} E_e,$$

truncated to $w(\pi) < q$. Reference `WeightedTransferEdge`, `WeightedPath`, `FilteredInverseExpansion`, path/minimum-weight methods, acyclic exact inverse, and transfer-series matrices.

12.B Hypotheses, invariants, and certificates

- Coarse/fine systems share backend and ambient dimension. Each fine packet has one valid parent. Every coarse packet has a child.
- Adjacent tower endpoints are equivalent. Each local refinement has `PSC-II-T03`. Coordinate maps and tower checks retain exact/numerical status.
- Common refinement requires commuting projectors, checked exactly or at tolerance.
- Defect identities compare finite matrix expressions. Zero defect means exact intertwining $CT_f = T_c C$, not equality of the spaces.
- Weighted transfer requires packet-diagonal invertible D, exact source/target edge support, and positive weights. Enumeration retains paths with $w(\pi) < q$ and enforces the declared `max_paths` guard. With exact SymPy plus prime p, the omitted inverse error is certified by p-adic valuation. Otherwise the filtration conclusion remains conditional, even when the evaluated operator happens to be nonsingular.

12.C Failure and edge cases

- Missing child, invalid parent index, endpoint mismatch, coordinates from wrong level, reverse level order, non-commuting common-refinement inputs, invalid edge support, singular diagonal, or weighted-path explosion beyond `max_paths`.
- A zero product packet is omitted. Generated common-refinement names therefore need not form a full Cartesian list.

12.D Example checklist

- **E0:** four fine coordinate packets grouped into two coarse packets. Refine/coarsen one vector and reflect support.
- **E1:** three levels $P^{(0)} \prec P^{(1)} \prec P^{(2)}$. Compare tower refinement with direct finest-level analysis of the reconstructed vector.
- **E2:** acyclic weighted chain $0 \to 1 \to 2$ on the SymPy exact backend. Compare the finite path inverse with the ordinary exact inverse and inspect omitted weight at precision q.
- **E3:** choose C, T_c, T_f with nonzero Δ. Verify the implemented $k = 2$ propagation identity.

Chapter 13: Residue, CRT, lattice, and Smith coordinates

Reference scope. Chapter 13 is authoritative for the finite residue, nested-address, CRT, rational lattice, gluing, valuation-delay, and Smith-congruence contracts. This retained card is a source-verified checklist. It preserves the SymPy-only lattice/Smith boundary and the narrower portable-codec surface.

13.A Notation, maps, and defining equations

- Residue analysis:
$$x_r[k] = x[r + Mk], \qquad x[n] = x_{n \bmod M}[\lfloor n/M \rfloor].$$
Cover `ResidueCoordinates`, analyze/synthesize/nested/refine/coarsen, residue packet systems, nested address $a = r + Ms$, and packet-coordinate bridge.
- CRT address for pairwise-coprime (m_i):
$$x \equiv r_i \pmod{m_i}, \qquad 0 \le x < \prod_i m_i.$$
Cover `CRTAddress`, `crt_pair`, `crt_many`, `factor_residue`, `dyadic_odd_factor`.
- `LatticePacketData(lattice_basis, projectors, *, prime: int, split_basis=None)` moves rational projectors into lattice coordinates $B^{-1}P_\alpha B$. Denominator weights use
$$w_\alpha = \max(0, -\min_{ij} v_p((B^{-1}P_\alpha B)_{ij})).$$
- Cover `GluingModule`, `PrecisionDelay`, integral separation, gluing quotient invariants, refinement parent data, and `refinement_weight_monotonicity`.
- `SmithDecomposition` and `SmithCongruenceProblem(matrix, rhs, *, prime: int, depth: int)` reduce $Ax \equiv b \pmod{p^k}$ through Smith coordinates. Cover solution obstruction, count, particular solution, and enumeration guard.

13.B Hypotheses, invariants, and certificates

- Residue modulus is positive. Channels may be ragged but must have exact expected lengths.
- CRT component moduli are positive and pairwise coprime. Nested residue composition does not require coprimality and is a different construction.
- Lattice basis is invertible over $\mathbb{Q}$. Prime arguments are prime. Valuation uses $v_p(0) = \infty$.
- Smith decomposition supplies unimodular transforms and diagonal divisibility through the SymPy reduction. Its certificate directly checks (LAR=D). Congruence solution records solvability, obstructions, solution count, and exact modular verification.
- Portable refinement records reject extra/missing fields. Codec envelopes reject malformed data and digest tampering.

13.C Failure and edge cases

- Noncoprime CRT moduli, zero/negative modulus, bad channel length, singular lattice basis, composite "prime", large exact-arithmetic cost, inconsistent congruence, and guarded or expensive enumeration.
- Signed Smith diagonal entries must be normalized correctly. Preserve the regression $-2x = 2 \pmod 8 \Rightarrow x \in \{3, 7\}$.

13.D Example specifications

Chapter 13 presents these four worked examples. Their symbolic statements remain here as compact reference requirements.

- **E0:** split `range(17)` modulo five, display ragged lengths, synthesize exactly.
- **E1:** `crt_pair(1,4,2,3)` gives 5 mod 12. Contrast nested address with the same integers.
- **E2:** compute $v_2(3/8) = -3$, projector weights, and a precision-delay record.
- **E3:** $A = \mathrm{diag}(2, 4, 0)$, modulus 8. Derive transformed divisibility conditions and solution count.

Chapter 14: Polyphase systems and refinement

Reference scope. Chapter 14 is authoritative for signal, residue, arithmetic, tower, and lattice polyphase refinement. This retained card remains the finite-law and example checklist. It does not introduce a lattice tower or an infinite refinement limit.

14.A Notation, maps, and algorithm

- `PolyphaseCoordinateRecord` is `PolyphaseCoordinates | ResidueCoordinates | ArithmeticPolyphase`.
- `PolyphaseRefinement(M,N)` requires $M \mid N$, $K = N/M$, and implements

$$a = r + Ms, \quad r = a \bmod M, \quad s = \lfloor a/M \rfloor, \quad (x_r^{(M)})_s^{(K)} = x_{r+Ms}^{(N)}.$$

- Refinement slices parent channel `r` by `s::K` without reconstructing the ambient signal. Coarsening interleaves children.
- Cover address/split/parent/child/children/parent-map, refine/coarsen, `certified_refine`, certificate, compose, local JSON, and convenience functions.
- `PolyphaseRefinementTower(orders: tuple[int, ...])`, with $M_0 \mid \cdots \mid M_L$, implements

$$a_L = r + \sum_{j=0}^{L-1} M_j s_j$$

  plus descendants, path address, direct refinement, and sequential/direct coherence.
- `LatticePolyphaseRefinement(coarse_strides: tuple[int, ...], fine_strides: tuple[int, ...])` applies the same law componentwise, with Cartesian child digits and channel counts $\prod_j M_j, \prod_j N_j$.

14.B Hypotheses, invariants, and certificates

- Orders/strides are positive integers, `bool` rejected. Dimensions agree. Every coarse component divides its fine component.
- `KM-B15-T01` checks coarsen-after-refine and equality with direct fine analysis. Exact integer/rational paths are exact. Floating/complex paths use a scaled tolerance.
- `KM-B15-T02` checks tower sequential versus direct refinement. `KM-B15-T03` is the lattice analogue.
- Composition requires matching middle order/strides. Local schemas contain exact field sets and version 1.

14.C Failure and edge cases

- Nondivisible orders, malformed digits/addresses, wrong record order, inconsistent child lengths/shapes, mismatched tower level, extra schema field.
- `copy=False` may permit sliced NumPy views but does not promise view semantics for coarsening allocations.

14.D Example specifications

Chapter 14 presents the four examples through address tables, direct-fine comparisons, a tower certificate, and a lattice roundtrip.

- **E0:** Ref$(2, 8)$, $(r, s) = (1, 3) \leftrightarrow a = 7$. Print full address table.
- **E1:** integer residue data length 31, $4 \to 12$. Compare refined channels to direct modulus-12 analysis and coarsen exactly.
- **E2:** tower $(2, 6, 24)$. Compare adjacent execution and direct address map under `KM-B15-T02`.
- **E3:** lattice array shape $(6, 9)$, strides $(2, 3) \to (4, 9)$. Certify componentwise roundtrip.

Part IV: Coordinate calculus and analysis-synthesis systems

Chapter 15: Coordinate objects

> **Reference scope.** Chapter 15 is authoritative for dense exact rational conventions, finite bases, atlas routing, explicit budgets, exact solves, and affine and barycentric objects. The retained card is the source-based specification.

15.A Notation, types, and definitions

- Exact layer: `Scalar=int|Fraction`, `Vector=tuple[Fraction,...]`, `Matrix=tuple[tuple[Fraction,...],...]`, `Polynomial=tuple[Fraction,...]` with ascending powers.
- `FiniteBasis(B)` has columns b_j, synthesis $x = Bc$, exact analysis by solve, Gram $G = B^T B$, square-basis dual rows $L = B^{-1}$, and the rectangular-span projector $P = B(B^T B)^{-1} B^T$.
- `CoordinateSystem(name,dimension,kind,description)` and directed `CoordinateAtlas` with shortest resolved path transform. `AtlasPath.length`. `CommutingDiagramCheck` compares products.
- Exact linear-algebra cards cover normalization, shape, zero and identity constructors, transpose and column operations, addition, subtraction, multiplication, vector multiplication, powers, Kronecker products, RREF, rank, determinants, inversion, solving, nullspaces and `ExactLinearSystem`.
- Basis constructors: standard, matrix columns, direct sum, tensor product, permutation, scaling. `CoordinateBudget.require_*` guards dimension, degree, horizon, and exact-entry count.

15.B Hypotheses, invariants, and failures

- Rows are rectangular and dimensions legal. Basis vectors are independent. Square analysis requires invertibility. Rectangular analysis requires exact membership.
- `FiniteBasis.certificate()` proves independence and stated finite basis properties. Atlas path validity requires matching endpoint labels and dimensions. Path comparison certificate records the exact matrix difference.
- Reject ragged/empty-invalid matrices, singular inverse, inconsistent system, vector outside a rectangular span, duplicate atlas system/edge without replacement, disconnected route, and exceeded budget.

15.C Example specifications

E0 and E1 are executed exactly in Chapter 15. E2 is presented as a derived guide example built from the shipped atlas API. Its perturbed edge is not a separate shipped example file.

- **E0:** solve underdetermined $Ax = b$. Display particular x_0, nullspace N, and $x = x_0 + Nz$. Then show an inconsistent system residual.
- **E1:** plane basis in $\mathbb{Q}^3$. Compute Gram matrix and P, verify $P^2 = P$, test membership, and show that `dual_rows()` is square-basis-only.
- **E2:** atlas $A \to B \to C$ versus $A \to D \to C$. Compare products, then perturb one edge for a noncommuting diagnostic.

Chapter 16: Exact and numerical coordinate transforms

> **Reference scope.** Chapter 16 is authoritative for the exact, numerical, affine, formal, normalized, polynomial-operator, shift, realization, modular, persistence, and stable numerical analysis/synthesis contracts. This card remains the compact completion checklist.

16.A Typed maps and equations

- `CoordinateTransform(source: str, target: str, matrix: Matrix, theorem: str, description: str = '')` and its `ExactCoordinateTransform` alias implement $c_t = Tc_s$, inverse T^{-1} when available, and `then(U)` as UT. `certificate()` validates the finite rational transform object and reports square/determinant/invertibility diagnostics. `roundtrip_valid()` separately checks exact two-sided inversion.
- `NumericCoordinateTransform` mirrors the map over NumPy with tolerance and condition number. `FiniteFieldCoordinateTransform` acts in $\mathbb{F}_p$. Rational reduction sends $a/b \mapsto ab^{-1} \pmod p$ only when $p \nmid b$.
- Affine maps: $y = Ax + b$, $(B, c) \circ (A, b) = (BA, Bb + c)$, homogeneous matrix $\begin{bmatrix} A & b \\ 0 & 1 \end{bmatrix}$. Barycentric coordinates satisfy $x = \sum_i \lambda_i v_i$, $\sum_i \lambda_i = 1$.
- Formal jet $\phi(t) = a_1 t + \cdots + a_N t^N$ has $a_1 \neq 0$. Cover truncated composition, inversion, pullback/pushforward, and substitution transform.
- Coordinate calculus compiles polynomial rules columnwise. Include derivative, integral, multiplication, affine substitution, finite difference, transport, composition, and $[A, B] = AB - BA$.
- Normalization sends $F = \sum a_n z^n/\gamma_n = \sum b_n z^n/\eta_n$ to $b_n = (\eta_n/\gamma_n)a_n$. Shift-polynomial Bézout uses $u(L)p(L) + v(L)q(L) = I$ on the common finite interior.

16.B Hypotheses, invariants, and failures

- Labels/dimensions compose. Inverse availability requires a square nonsingular matrix. Numerical validity uses residual/tolerance and records conditioning. Modular prime must be prime and denominators invertible.
- Affine matrix is square, simplex vertices affinely independent, formal constant term is zero and linear coefficient nonzero, polynomial degree/horizon remains within budget.
- Singular `CoordinateTransform`, `NumericCoordinateTransform`, and `FiniteFieldCoordinateTransform` objects are representable, but inverse calls fail and square-map diagnostics/certificates report noninvertibility or invalidity as defined by each class. `AffineCoordinateTransform` rejects a singular linear part at construction. Also reject ill-shaped inputs, label mismatch, degenerate simplex, noninvertible modular denominator, composite modulus, noncomposable jets, over-truncation, noncoprime shift polynomials, and finite-horizon boundary misuse.

16.C Example specifications

The exact composition, barycentric, finite polynomial-calculus, and modular obstruction targets are all resolved in the published chapter. The chapter also records advanced edge cases that the four compact example specifications do not enumerate.

- **E0:** exact $A \to B \to C$. Verify `then` order and $(UT)^{-1} = T^{-1}U^{-1}$.
- **E1:** triangle barycentrics including one point with a negative weight. Distinguish affine validity from convex membership.
- **E2:** transport D and M_x into falling-factorial coordinates. Verify $[D, M_x] = I$ only on a safe truncated subspace.
- **E3:** reduce rational shear modulo seven, then choose a denominator divisible by seven and expect obstruction.

Chapter 17: Polynomial and classical coordinate families

> **Reference scope.** Chapter 17 is authoritative for exact polynomial storage, graded and finite bases, the classical catalog, Bell/Touchard ownership boundaries, connection maps, budgets, persistence, failures, examples, and current verification evidence. This retained card is the compact contract checklist.

17.A Families, maps, and equations

- `PolynomialFamily(name,rule)` supplies $P_n(x)$ and coefficient matrix $M_P = [P_0 \cdots P_N]$. Connection:

$$C_{Q\leftarrow P} = M_Q^{-1} M_P, \qquad p_n = \sum_k (C_{Q\leftarrow P})_{kn} q_k.$$

- Cover exact polynomial add/scale/shift/multiply/evaluate/derivative, monomial/Fibonacci/Euler constructors, Euler-to-Fibonacci coefficients/transform, companion powers, derivative operators, operator conjugation, and secant-Euler recovery.
- Catalog cards cover falling/rising factorial, binomial, Bernoulli, Euler, Fibonacci, Lucas, Pell, Chebyshev T/U, Legendre, Hermite variants, Laguerre, custom Appell, Bernstein, Newton, and Lagrange.
- Interpolation maps: evaluation $c \mapsto (p(x_i))_i$, inverse interpolation when nodes are distinct. Affine substitution sends $p(x) \mapsto p(ax+b)$.

17.B Hypotheses, invariants, and failures

- A degree-N family must yield exactly graded independent columns through N. Connections are exact and invertible within the finite degree budget.
- Operator transport obeys $L_Q = CL_PC^{-1}$. Bernstein basis obeys $\sum_{k=0}^{n} B_{n,k}(x) = 1$.
- Family connections return `CoordinateTransform` and its exact certificate. Elementary polynomial/scalar helpers return raw exact tuples or integers rather than separate certificates.
- Reject negative degree/index, index beyond Bernstein degree, duplicate interpolation nodes, insufficient Appell initial values, nongraded custom family, singular connection, and exceeded polynomial budget.

17.C Example checklist

Chapter 17 presents these four targets, each as an executable guide path or a source-verified contract example with its exactness boundary stated.

- **E0:** expand $E_n(x)$ in graded Fibonacci polynomials through $n = 6$. Reconstruct every polynomial exactly.
- **E1:** Legendre $\leftrightarrow$ Chebyshev connection and transported derivative matrix.
- **E2:** interpolate rational samples in Newton and Lagrange bases. Synthesize the same polynomial.
- **E3:** verify Bernstein partition of unity and reject repeated interpolation nodes.

Chapter 18: Combinatorial, recurrence, and state-space coordinates

> **Reference scope.** Chapter 18 is authoritative for exact combinatorial maps, forward differences, recurrence defect coordinates, realization and state-space transport, persistence, failures, examples, and verification evidence. It also records the shipped `RecurrenceModel.synthesize()` convenience-method defect without hiding the fully operational lower-level synthesis path.

18.A Typed transforms and defining equations

- Triangular transforms include Stirling/Lah, binomial, forward difference, divisor/subset zeta and Möbius inverses:

$$\Delta^n a_0 = \sum_{k=0}^{n} (-1)^{n-k} \binom{n}{k} a_k, \quad b_n = \sum_{d|n} a_d, \quad b_S = \sum_{T\subseteq S} a_T.$$

- Recurrence coordinates for $a_{n+d} = \sum_{i<d} c_i a_{n+i} + \delta_n$ are $(a_0, \ldots, a_{d-1}, \delta_0, \ldots)$. Analysis is unit lower triangular and synthesis is exact.
- Cover recurrence polynomial, companion, defect, analysis/synthesis transforms, records, cross-model change, Green weights, variation of constants, named models/catalog, and Euler-to-Fibonacci path.
- Realization $x_n = A^n x_0$, $a_n = c^T x_n$. Observability $O_h = [c^T; c^T A; \ldots]$, Krylov $K(A, v) = [v, Av, \ldots]$, intertwiners solve $BS = SA$.
- State space: $x_{k+1} = Ax_k + Bu_k$, $y_k = Cx_k + Du_k$, and under $z = Tx$, $(A', B', C', D') = (TAT^{-1}, TB, CT^{-1}, D)$.

18.B Hypotheses, invariants, and failures

- Transform sizes and recurrence horizons respect budget. Recurrence prefix length is at least the order. State recovery uniqueness requires rank $O_h = d$.
- State-coordinate certificate compares Markov parameters $H_0 = D$, $H_k = CA^{k-1}B$. Cyclic similarity requires invertible Krylov matrices.
- Subset zeta grows as 2^r. Reject undersized prefixes, unknown model, nonobservable/noncontrollable canonical transform, noncyclic vectors, dimension mismatch, and budget overflow.

18.C Example checklist

Chapter 18 presents these four targets. They remain here as reference requirements.

- **E0:** squares $(0, 1, 4, 9, ...)$ to forward differences. Recover by inverse Newton evaluation.
- **E1:** arbitrary length-18 sequence in Fibonacci defect coordinates. Reconstruct and inspect homogeneous versus defect parts.
- **E2:** Euler prefix analyzed by Fibonacci recurrence, yielding the shipped exact reconstruction path.
- **E3:** SISO companion model before/after T. Assert identical finite Markov parameters and show nonunique recovery at insufficient horizon.

Chapter 19: Spectral, packet, tensor, and operator transport

> **Reference scope.** Chapter 19 is authoritative for numerical spectral systems, exact packet permutations, tensor and variance-aware transport, certificates, persistence, dense cost models, worked paths, and failures. Its digit-reversal contract explicitly requires radix at least two because the shipped radix-one path does not terminate for lengths greater than one.

19.A Maps and equations

- Numeric transforms: unitary DFT $F_{kn} = N^{-1/2}e^{-2\pi ikn/N}$, orthonormal DCT-II, Sylvester Walsh-Hadamard, dyadic Haar, and modal $V^{-1}AV \approx \mathrm{diag}(\lambda)$.
- Exact packet permutations cover residue-class order, fixed-width digit reversal, tensor-axis permutation, CRT tuple order, and stable key sorting. Each order induces a permutation `CoordinateTransform`.
- Variance-aware transport for $y = Tx$:

$$v' = Tv, \quad \phi' = T^{-T}\phi, \quad A' = TAT^{-1}, \quad G' = T^{-T}GT^{-1}.$$

- Rectangular maps use $A' = T_c A T_d^{-1}$. Tensor and direct-sum transforms use Kronecker and block diagonal products. Pairing invariant: $\phi'(v') = \phi(v)$.

19.B Hypotheses, invariants, and failures

- Numeric matrices have finite entries and compatible dimensions. Walsh/Haar size requirements are enforced by constructors. Modal coordinates require a numerically complete eigenvector basis under `np.linalg.matrix_rank(eigenvectors, tol=tolerance)`.
- Permutations cover each index exactly once. CRT moduli are pairwise coprime. Digit-reversal length has the required radix width. Tensor axes are a permutation.
- Variance list length equals tensor rank. Exact tensor/variance helpers operate on rational `CoordinateTransform` objects and return raw exact `Vector` or `Matrix` values. `pairing_invariant` returns `bool`. Transform-object certificates come from `CoordinateTransform.certificate()` or `NumericCoordinateTransform.certificate()`. The raw transport helpers return no separate certificate.
- Reject rank-deficient modal eigenvector systems at the declared tolerance, nonfinite input, invalid radix length, noncoprime CRT moduli, repeated axis, covariance mismatch, or a noninvertible transform where an inverse is required. The modal synthesis condition number is diagnostic rather than a separate constructor rejection threshold.

19.C Example checklist

Chapter 19 presents these four targets. They remain here as reference requirements.

- **E0:** DFT and Haar roundtrips. Compare $\|F^*F - I\|$ and energy residuals.
- **E1:** bit reversal and residue grouping for length eight. Display order and exact permutation inverse.
- **E2:** transport vector/covector/operator/form by one rational T. Certify pairing and quadratic-form invariance.
- **E3:** diagonalizable $A = V\Lambda V^{-1}$ versus a defective Jordan block rejection.

Chapter 20: Analysis-synthesis and redundant frames

> **Reference scope.** Chapter 20 is authoritative for numerical and exact analysis/synthesis systems, canonical duals, redundant components, gauges, lifts, bridges, coefficient blocks, adjoints, persistence, costs, failures, examples, and current verification evidence. The retained card is the finite contract checklist.

20.A Typed systems, maps, and equations

- A mathematical analysis/synthesis system has

$$A : X \to K, \quad S : K \to X, \quad SA = I_X.$$

  `ExactAnalysisSynthesisSystem(A,S)` enforces this identity during construction. Bare numerical `AnalysisSynthesisSystem(A,S)` validates shape, finiteness, labels, and tolerance but may be constructed with $SA \neq I_X$. Its `certificate()` then has `valid=False`. Numerical `canonical_dual` first applies a relative-SVD full-column-rank test, computes `np.linalg.pinv(A, rcond=tolerance)`, constructs the system, and requires its certificate. The exact finite identity is $S = (A^*A)^{-1}A^*$ for full-column-rank A.
- Coordinate selection E_I yields physical component $C_I = SE_IA$, with $\sum_I C_I = I$ for a partition. Operator blocks are $C_\beta T C_\alpha$.
- Gauge algebra: $P = AS$, $Q = I - P$, $c = Pc + Qc$, $Qc \in \ker S$, and $c \sim d \iff S(c - d) = 0$.
- Physical map lift $\widehat{T} = A_Y T S_X$ satisfies $P_Y \widehat{T} P_X = \widehat{T}$ and $S_Y \widehat{T} A_X = T$. Mismatched composition uses bridge $R = A_{Y,in} S_{Y,out}$.
- Reference all methods of `AnalysisSynthesisSystem`, `AnalysisSynthesisBlocks`, `CoefficientPartition`, numerical/exact gauge systems, maps, coefficient blocks, direct sums, tensor products, adjoints, dual adjoints, reblocking, and physical-block recovery.

20.B Hypotheses, invariants, and certificates

- Shapes are $A \in \mathbb{F}^{m\times n}$, $S \in \mathbb{F}^{n\times m}$, $m \geq n$ for canonical dual. Partitions cover coefficient indices exactly once.
- System certificates check $SA = I$. Block certificates check reconstruction. Map certificates check lift support and physical reconstruction. Exact layer uses rational matrices. Numerical layer records tolerance. Numerical canonical-dual, gauge, and map paths require a valid system certificate where their construction depends on the left-inverse law.
- In redundancy $m > n$, C_i need not be idempotent and coefficient blocks $E_\beta \widehat{T} E_\alpha$ are not physical blocks $C_\beta T C_\alpha$.

20.C Failure and edge cases

- Rank-deficient analysis is rejected by `canonical_dual`. A bare numerical system can instead construct with a failed left-inverse certificate. Other failures include incompatible shapes, malformed partition, incompatible composition, confusing raw adjoint with dual adjoint, or treating gauge-equivalent coefficient vectors as equal arrays.
- Portable roundtrip covers supported systems but never reconstructs arbitrary callable family rules.

20.D Example checklist

Chapter 20 presents these four targets. They remain here as reference requirements.

- **E0:** redundant $A : \mathbb{R}^2 \to \mathbb{R}^4$, canonical dual S. Certify $SA = I_2$, analyze/reconstruct x.
- **E1:** choose g with $\|Sg\|$ within the numerical system tolerance. Show $c \sim c+g$, synthesis residual within tolerance, distinct coefficient arrays, and gauge seminorm within tolerance. Use the exact gauge counterpart when claiming literal equality and an exactly zero seminorm.
- **E2:** lift $T : X \to Y$, reconstruct it, partition coefficient blocks, and compare one coefficient block with its physical block.
- **E3:** compose through mismatched middle presentations using transition bridge. Compare raw adjoint and dual adjoint on an oblique frame.

Parts I through IV source and example crosswalk

- Each chapter above expands into symbol cards using the master card schema and keeps the displayed equations as the finite contract spine.
- Existing executable anchors: `typed_grammar.py`, `quickstart.py`, `refinement_pipeline.py`, `exact_lattice_separation.py`, `block15_polyphase_refinement.py`, `general_coordinate_atlas.py`, `classical_coordinate_families.py`, `coordinate_calculus.py`, `formal_affine_coordinate_changes.py`, `recurrence_state_coordinate_atlas.py`, `spectral_packet_coordinates.py`, `euler_to_fibonacci_coordinates.py`, and `block16_analysis_synthesis.py`.
- Example specifications remain planned until implemented and asserted. They must not be represented as shipped scripts. The specification prohibits unlabeled claims and invented executable anchors.

84. Generating functions, arithmetic, sequences, and analytic systems

Specifications for generating functions, arithmetic, sequences, and analytic packets

These specifications supplement the Part V explanations. Every displayed assertion is an implemented definition, a finite certificate contract, or an explicitly labeled numerical, conditional, heuristic, or planned statement.

Part-wide symbolic conventions

Coefficient universes

Let

$$\mathcal{R} \in \{\mathbb{Z},\ \mathbb{Q},\ \mathbb{Z}/M\mathbb{Z},\ \mathbb{F}_p,\ \mathbb{Z}/p^r\mathbb{Z}\}.$$

The exact scalar universes exposed by the Python API are:

$$\texttt{int} \leftrightarrow \mathbb{Z}, \qquad \texttt{Fraction} \leftrightarrow \mathbb{Q}, \qquad \texttt{int in } [0, M-1] \leftrightarrow \mathbb{Z}/M\mathbb{Z}.$$

Booleans are not silently accepted as integers where strict integer validation is part of the contract. Numerical analytic values live in $\mathbb{C}$ with floating residuals and declared tolerances. Several generic containers also accept arbitrary `numbers.Number` values: `HurwitzSeries` without a modulus, `MahlerSeries`, `LinearRecurrence`, `RecurrencePacket`, and `Sequence`. Their algebra is exact for exact `int` or `Fraction` data and numerical for floating or complex data. `Sequence.exact` is declared metadata, not a runtime proof that every returned scalar is exact.

Finite normalized-series universe

For a nonzero weight sequence $\gamma = (\gamma_n)_{n\ge 0}$, $\gamma_0 = 1$, define

$$e_n^{(\gamma)}(z) = \frac{z^n}{\gamma_n}, \qquad \mathcal{R}_{\gamma,N}[z] = \left\{ \sum_{n=0}^{N} a_n e_n^{(\gamma)}(z) : a_n \in \mathcal{R} \right\}.$$

All formal-series operations in this part are finite:

$$\operatorname{trunc}_N : \mathcal{R}[[z]] \longrightarrow \mathcal{R}[z]/(z^{N+1}).$$

No finite prefix is to be described as a proof of an unrestricted infinite identity unless the API supplies a separate exact structural argument.

Truth-status tags for every reference card

- **E**: exact finite computation or exact finite certificate.
- **N**: numerical result with an explicit residual/tolerance contract.
- **C**: conclusion retaining named unchecked assumptions.
- **H**: heuristic, estimator, or bounded-data discovery claim.
- **P**: planned or unimplemented claim.
- **O**: orthogonal optional-backend qualifier. Name the backend without changing the evidence class. Every subsection below has the editorial slots

$$(\text{domain}, \text{map}, \text{hypotheses}, \text{invariant}, \text{certificate}, \text{failure}, \text{example}).$$

Public constants, aliases, and feature keys

The exhaustive ledger must include the singleton exact domains `INTEGER_DOMAIN` and `RATIONAL_DOMAIN`, the default resource object `DEFAULT_GENERATING_BUDGET`, and the compatibility aliases

$$\texttt{NormalizedBasis} = \texttt{NormalizedMonomialBasis} = \texttt{WeightedMonomialBasis}.$$

The package status keys are `arithmetic.FEATURE_KEY = "arithmetic"`, `sequences.FEATURE_KEY = "sequences"`, and `analytic.FEATURE_KEY = "analytic"`. They select feature-status metadata and do not prove a mathematical capability.

Optional-backend ledger

SymPy is required by `guess_linear_recurrence`, `guess_hypergeometric_ratio`, `guess_p_recursive`, `guess_algebraic_gf`, `characteristic_closed_form`, the symbolic-rendering methods on the corresponding guess records, `normalized_cyclotomic_expression`, `verify_cyclotomic_coboundary`, `LinearRecurrence.jump_characteristic_polynomial`, `reciprocal_polynomial`, `appell_polynomial_symbolic`, and `theta_polynomial_symbolic`. SymPy is **not** required by `guess_quasipolynomial`, `rational_gf_from_recurrence`, or `verify_cyclotomic_exponent_relation`.

`reduced_zero_zeta(..., use_computed_zeros=False)` requires `mpmath`. `CyclicSectionFamily.fundamental_matrix` uses SciPy when available and an internal matrix-exponential fallback otherwise. Every function card must repeat its own backend branch rather than applying one package-wide rule.

21. Formal series and weighted bases

21.1 Coefficient-domain protocol

21.1.1 Abstract map signature

Document `CoefficientDomain` (generic form `CoefficientDomain[E]`) as the exact algebraic interface

$$(\mathcal{R}, 0, 1, +, -, \cdot, \mathcal{R}^{\times}, {}^{-1}) \longrightarrow \text{finite normalized-series arithmetic.}$$

Its operations have symbolic signatures

$$\begin{aligned} \text{coerce} &: X \to \mathcal{R}, \\ \text{add}, \text{subtract}, \text{multiply} &: \mathcal{R}^2 \to \mathcal{R}, \\ \text{is_unit} &: \mathcal{R} \to \{\mathrm{T}, \mathrm{F}\}, \\ \text{inverse} &: \mathcal{R}^{\times} \to \mathcal{R}^{\times}, \\ \text{encode} &: \mathcal{R} \to \mathcal{J}, \\ \text{decode} &: \mathcal{J} \to \mathcal{R}. \end{aligned}$$

Here $\mathcal{J}$ is the trusted data-only serialization domain.

21.1.2 Concrete domains and unit hypotheses

Class	Carrier	Unit condition	Inverse failure
`IntegerDomain`	$\mathbb{Z}$	$x = \pm 1$	$\lvert x\rvert \neq 1$
`RationalDomain`	$\mathbb{Q}$	$x \neq 0$	$x = 0$
`ResidueRingDomain`	$\mathbb{Z}/M\mathbb{Z}$	$\gcd(x, M) = 1$	nonunit residue
`PrimeFieldDomain`	$\mathbb{F}_p$	$x \neq 0$	zero residue
`TruncatedPadicDomain`	$\mathbb{Z}/p^r\mathbb{Z}$	$p \nmid x$	positive valuation

`domain_from_descriptor` implements descriptor restoration as the partial map

$$\text{domain_from_descriptor} : \mathcal{J} \rightharpoonup \{\text{known exact domains}\}.$$

Failure branches to exhibit symbolically: malformed descriptors, composite `PrimeFieldDomain` moduli, nonpositive precision, modulus below two, and division by a nonunit.

21.1.3 Symbolic example slot

Over $\mathbb{Z}/8\mathbb{Z}$, compare

$$3^{-1} = 3, \qquad 2 \notin (\mathbb{Z}/8\mathbb{Z})^{\times}.$$

The example must show successful multiplication by the unit inverse and the explicit exception raised for the zero divisor.

21.2 Weighted monomial bases

21.2.1 Basis object and structural constants

Document `BasisDescriptor`, `BasisCertificate`, and `WeightedMonomialBasis` through

$$e_n(z) = z^n/\gamma_n, \qquad C_\gamma(n,k) = \frac{\gamma_n}{\gamma_k \gamma_{n-k}}.$$

The basis maps are

$$\gamma : \mathbb{N} \to \mathbb{Q}^{\times}, \qquad C_\gamma : \{(n,k) : 0 \le k \le n\} \to \mathbb{Q}.$$

Normalized multiplication is

$$\mu_{\gamma,N} : \mathbb{Q}_{\gamma,N}[z]^2 \to \mathbb{Q}_{\gamma,N}[z], \qquad c_n = \sum_{k=0}^{n} C_\gamma(n,k) a_k b_{n-k}.$$

The custom lowering operator is defined by

$$D_\gamma e_n = e_{n-1}, \quad n \ge 1, \qquad D_\gamma e_0 = 0.$$

21.2.2 Basis constructors to expand into individual cards

- `ordinary_basis`: $\gamma_n = 1$.
- `exponential_basis`: $\gamma_n = n!$.
- `factorial_power_basis`(s): $\gamma_n = (n!)^s$.
- `q_factorial_basis`(q): $\gamma_n = [n]_q!$, with `q=1` permitted.
- `fibonacci_factorial_basis`: $\gamma_n = \prod_{j=1}^{n} F_j$.
- `pochhammer_basis`(α): $\gamma_n = (\alpha)_n$.
- `basis_from_values`: finite portable table basis.
- `basis_from_descriptor`: closed descriptor reconstruction. For each constructor, record the admissible parameter set, exact scalar type, whether every γ_n is nonzero through degree N, and whether the basis is portable or only live in memory. Class-member cards must also cover `BasisDescriptor.parameter_map`, `BasisDescriptor.create`, `WeightedMonomialBasis.require_descriptor`, `semantically_equivalent`, `equivalent_through`, `lowering_weight`, `structure_constant`, and `packet_transfer_weight`.

21.2.3 Packet transfer weight

For residue $r \in \{0, \dots, m-1\}$ and packet basis η_q, write

$$P_{m,r} F(z) = z^r G(z^m), \qquad b_q = \frac{\eta_q}{\gamma_{mq+r}} a_{mq+r}.$$

`PacketIdentity` stores exactly `modulus`, `residue`, `source_basis`, `target_basis`, `source_degree`, `packet_degree`, and `valid`. It compares the reconstructed and projected ordinary coefficient vectors. It records basis names, not the normalization tables γ, η, or callable identities.

21.2.4 Certificate and failure ledger

`BasisCertificate` is the finite diagnostics record

$$\left(\begin{array}{c}\texttt{name}, \texttt{degree}, \texttt{nonzero_normalization},\\ \texttt{integral_structure_constants}, \texttt{maximum_denominator},\\ \texttt{maximum_numerator_bits}\end{array}\right).$$

It has no `valid`, `require`, or descriptor-validity field. Descriptor portability is a separate property of `WeightedMonomialBasis`. `WeightedMonomialBasis.finite_certificate` produces this record through the requested finite degree. Failure examples: zero normalization factor, invalid factorial power, unsupported descriptor kind, malformed finite table, and attempted portable restoration of an unrecorded callback.

21.2.5 Symbolic examples

1. Compare the same polynomial in ordinary and exponential coordinates:

$$1 + 2z + 3z^2 = 1\, e_0^{(\exp)} + 2\, e_1^{(\exp)} + 6\, e_2^{(\exp)}.$$

2. Derive the EGF product rule

$$c_n = \sum_{k=0}^{n} \binom{n}{k} a_k b_{n-k}$$

from $C_\gamma(n,k)$ with $\gamma_n = n!$.

21.3 `FormalSeries`: rational normalized calculus

21.3.1 Object signature

Treat

$$\texttt{FormalSeries}(\gamma, (a_0, \dots, a_N)) \in \mathbb{Q}_{\gamma,N}[z].$$

Public operations to receive separate symbolic cards: `degree`, `truncate`, `ordinary_coefficients`, `from_ordinary_coefficients`, `convert`, `add`, `subtract`, `scale`, `multiply`, `hadamard`, `ordinary_derivative`, `ordinary_integral`, `custom_lower`, `reciprocal`, `compose`, `exp`, `log`, `power`, `project`, `packet`, `packet_identity`, and `evaluate`.

21.3.2 Core map signatures

$$\begin{aligned}
\text{convert}_{\gamma\to\eta} &: \mathbb{Q}_{\gamma,N}[z] \to \mathbb{Q}_{\eta,N}[z],\\
\text{multiply} &: \mathbb{Q}_{\gamma,N}[z]^2 \to \mathbb{Q}_{\gamma,N}[z],\\
\text{reciprocal} &: \{F : a_0 \neq 0\} \to \mathbb{Q}_{\gamma,N}[z],\\
\text{compose} &: \{(F,G) : G(0) = 0\} \to \mathbb{Q}_{\gamma,N}[z],\\
\text{project}_{m,r} &: \mathbb{Q}_{\gamma,N}[z] \to \mathbb{Q}_{\gamma,N}[z].
\end{aligned}$$

Basis conversion must preserve the ordinary polynomial:

$$\sum_{n=0}^{N} a_n \frac{z^n}{\gamma_n} = \sum_{n=0}^{N} b_n \frac{z^n}{\eta_n}.$$

Projection is

$$(P_{m,r}F)_n = \mathbf{1}_{n\equiv r \pmod m} a_n.$$

21.3.3 Unit-series branches

The guide must state these preconditions beside the methods:

operation	required constant coefficient
F^{-1}	$F(0) \neq 0$
$\log F$	$F(0) = 1$
$\exp F$	$F(0) = 0$
$F \circ G$	$G(0) = 0$

All equalities are modulo z^{N+1}. `PacketIdentity` records exact finite reconstruction of a projected/interleaved coefficient vector.

21.3.4 Symbolic example

Use

$$F(z) = 1 - z - z^2, \qquad F(z)^{-1} = 1 + z + 2z^2 + 3z^3 + 5z^4 + \cdots \pmod{z^{N+1}}$$

to connect reciprocal series, Fibonacci coefficients, truncation, and exact packet reconstruction.

21.4 `FiniteNormalizedSeries`: domain-aware calculus

21.4.1 Object and operation domain

$$\texttt{FiniteNormalizedSeries}[\mathcal{R}] \in \mathcal{R}_{\gamma,N}[z].$$

Give individual cards for `degree`, `coefficient`, `truncate`, `convert`, `ordinary_coefficients`, `from_ordinary_coefficients`, `add`, `subtract`, `scale`, `multiply`, `reciprocal`, `compose`, `exp`, `log`, `project`, `custom_lower`, and `certificate`.

21.4.2 Transfer certificate

`SeriesTransferCertificate` should be written as the finite proposition

$$\forall n \le N: \quad \operatorname{ordinary}_n(F_\gamma) = \operatorname{ordinary}_n(F_\eta),$$

together with the normalization-factor comparisons used to obtain the target coefficients.

21.4.3 Nonfield failure cases

The guide must explicitly construct a reciprocal that exists over $\mathbb{Q}$ but fails over $\mathbb{Z}$, and a modular reciprocal whose constant term is a zero divisor. No fallback to floating division is permitted. For `exp` and `log`, the constant-term guards are necessary but not sufficient over a general `CoefficientDomain`. Through requested degree N, every embedded integer $1, \dots, N$ that the recurrence divides by must be a unit:

$$f_n = n^{-1} \sum_{k=1}^{n} k g_k f_{n-k}, \qquad (\log F)_n = n^{-1}[z^{n-1}]\, F'/F.$$

Thus degree $N \ge p$ can fail in characteristic p. Basis conversion and ordinary-coordinate access also fail when a required rational normalization factor cannot be coerced into the domain.

21.4.4 Symbolic example

Multiply two factorial-power series over $\mathbb{Q}$, convert the product to ordinary coordinates, convert it back, and require the transfer certificate.

22. Normalized generating functions

22.1 Declarative coefficient sources

22.1.1 `CustomFunctionSpec`

Represent a declarative source as

$$\mathfrak{F} = (\text{name}, \gamma, a : \mathbb{N} \to \mathbb{Q}, \text{source kind}, \text{metadata}).$$

Its finite realization is

$$\text{series}_N(\mathfrak{F}) = \sum_{n=0}^{N} a_n e_n^{(\gamma)}.$$

The reference card must separate the live evaluator from the finite frozen record.

22.1.2 Constructor equations

- `explicit_function`: supplied map $n \mapsto a_n$.
- `linear_recurrence_function`:

$$a_{n+d} = \sum_{j=0}^{d-1} c_j a_{n+j}.$$

- `hypergeometric_function`:

$$a_{n+1} = R(n) a_n.$$

- `rational_function`:

$$F(z) = \frac{P(z)}{Q(z)}, \qquad Q(0) \neq 0.$$

- `algebraic_function`:

$$P(z, F(z)) = 0, \quad P(0, y_0) = 0, \quad P_y(0, y_0) \neq 0.$$

- `euler_product_function`:

$$F(z) = \prod_{m \geq 1} (1 - z^m)^{-c_m}.$$

- `factorial_power_bell_function` and `weighted_factorial_power_bell_function`:

$$\exp\left(x \sum_{m \geq 1} c_m \frac{z^m}{(m!)^s} \right) = \sum_{n \geq 0} P_n(x) \frac{z^n}{(n!)^s}.$$

- `compose_functions`:

$$H(z) = F(G(z)), \qquad G(0) = 0.$$

22.1.3 Registry behavior and failures

`FunctionRegistry.register` always rejects a duplicate stripped/lowercased name with `KeyError`. No replacement option exists. Document `FunctionRegistry.get` as the corresponding normalized-name lookup. The guide must also demonstrate invalid recurrence seeds, a singular rational denominator, non-simple algebraic initial roots, and illegal composition with $G(0) \neq 0$.

22.1.4 Symbolic examples

$$F_{\text{Fib}}(z) = \frac{z}{1 - z - z^2}, \qquad C(z) = 1 + zC(z)^2.$$

Generate the same finite prefixes by declarative recurrence, rational, and algebraic sources, while preserving the different source metadata.

22.2 Eager coefficient families

22.2.1 Exact maps

Each function is a finite map $N \mapsto (a_0, \dots, a_{N-1})$:

- `fibonacci_numbers`, `central_binomial_numbers`, `catalan_numbers`, and `bell_numbers`.
- `partition_numbers`, `apery_numbers`, `signed_euler_numbers`, and `bernoulli_numbers`.
- `generalized_bell_numbers`, `generalized_bell_polynomials`, `weighted_generalized_bell_polynomials`, and `weighted_generalized_bell_numbers`.
- `generalized_bell_hypergeometric_inner` and `finite_differences`.

22.2.2 Identities requiring symbolic derivation

$$A_n = \sum_{k=0}^{n} \binom{n}{k}^2 \binom{n+k}{k}^2,$$

$$B_n^{(s)} = \sum_{m=1}^{n} \binom{n-1}{m-1} \binom{n}{m}^{s-1} B_{n-m}^{(s)},$$

$$P_n(x) = x \sum_{m=1}^{n} c_m \binom{n-1}{m-1} \binom{n}{m}^{s-1} P_{n-m}(x).$$

`finite_differences` should display the full triangular table

$$\Delta^{r+1} a_n = \Delta^r a_{n+1} - \Delta^r a_n.$$

22.2.3 Example and boundary cases

Compare $s = 1$ and $s \geq 2$ generalized Bell coefficients. Reject negative counts and invalid factorial powers. Preserve `Fraction` output for Bernoulli numbers and any rational weighted family.

22.3 Bounded closed-form recognition

22.3.1 Search families

The candidate maps are:

$$a_{n+r} = \sum_{j<r} c_j a_{n+j}, \qquad \frac{a_{n+1}}{a_n} = \frac{P(n)}{Q(n)},$$

$$\sum_{j=0}^{r} P_j(n) a_{n+j} = 0, \qquad P(z, F(z)) = 0,$$

and residue-lane quasipolynomials

$$a_{mq+r} = Q_r(q).$$

22.3.2 Public record/function ledger

Records: `LinearRecurrenceGuess`, `HypergeometricGuess`, `PRecursiveGuess`, `AlgebraicGFGuess`, `QuasiPolynomialGuess`. Search/render functions: `guess_linear_recurrence`, `rational_gf_from_recurrence`, `guess_hypergeometric_ratio`, `guess_p_recursive`, `guess_algebraic_gf`, `guess_quasipolynomial`, `characteristic_closed_form`, plus record methods `equation`, `ratio_expression`, and `expression` where defined.

22.3.3 Scope proposition

Every returned guess has the bounded claim

$$\forall n \in I_{\text{fit}} \cup I_{\text{heldout}} : \quad \mathcal{E}(a, n) = 0.$$

It does **not** imply

$$\forall n \geq 0 : \mathcal{E}(a, n) = 0.$$

The guide must label optional SymPy absence, singular fitting systems, too-short prefixes, zero denominators in a ratio, and no candidate inside the requested degree/order bounds.

22.4 Sections, monoids, adapters, and portable records

22.4.1 Exponential sections

For `custom_exponential`, `custom_section`, and `root_filter_value`, define

$$E_\gamma(z) = \sum_{n \geq 0} \frac{z^n}{\gamma_n}, \qquad P_{m,r} E_\gamma(z) = \sum_{q \geq 0} \frac{z^{mq+r}}{\gamma_{mq+r}},$$

$$P_{m,r} E_\gamma(z) = \frac{1}{m} \sum_{j=0}^{m-1} \omega^{-rj} E_\gamma(\omega^j z).$$

`SectionCycleAudit` and `audit_section_cycle` must separately record the finite lowering cycle and numerical root-filter residual.

22.4.2 Finite monoid convolution

For `FiniteMonoidSeries` (generic form `FiniteMonoidSeries[M]`),

$$(a * b)(n) = \sum_{x \circ y = n} a(x) b(y).$$

Document `additive_series`, `multiplicative_series`, `dirichlet_convolution`, and `prime_valuation_packets`. Addition gives the Cauchy product. Multiplication gives

$$(a * b)(n) = \sum_{d \mid n} a(d) b(n/d).$$

22.4.3 Formal-series/FIR adapter

The convention is

$$\sum_{n=0}^{N} h_n z^n \longleftrightarrow H(z) = \sum_{n=0}^{N} h_n z^{-n}.$$

Document `FIRSeriesAdapter`, `series_to_fir_taps`, `series_to_fir_adapter`, `fir_taps_to_series`, and the numerical method `FIRSeriesAdapter.response`. Its certificate is numerical coefficient/tap agreement, not a proof of filter stability or causality beyond the finite convention.

22.4.4 Portable frozen source

`freeze_custom_function` maps a live source to `FrozenGeneratingFunction`,

$$\texttt{FrozenGeneratingFunction} = (\text{name}, \text{basis snapshot}, (a_0, \dots, a_N), \text{source kind}, \text{closed metadata}).$$

Restoration must execute no callback and import no arbitrary class. Document `degree`, `series`, and `register_generating_codecs`.

22.5 Resource budgets as preconditions

22.5.1 `GeneratingBudget`

Each expansive map is partial:

$$T : B \times X \rightharpoonup Y, \qquad T(B, x) \text{ defined only if } \operatorname{cost}(x) \preceq B.$$

Document `require_degree`, `require_terms`, `require_matrix_dimension`, `require_states`, `require_local_combinations`, `require_modulus`, `require_serialized_bytes`, `require_coefficient_bits`, and the `GeneratingBudget.max_modulus_table` property.

22.5.2 Default-bound table

quantity	default maximum
degree	100,000
coefficient bits	4,000,000
matrix dimension	4,096
reachable states	100,000
local combinations	1,000,000
modulus table	2,000,000
serialized bytes	64 MiB
recursion depth	1,000

Each bound needs one accepted boundary example and one rejected request.

23. Masks, Cartier operators, and digit algebra

23.1 Mahler masks and functional equations

23.1.1 Product and inverse

For radix $b \geq 2$ and unit mask $R(0) = 1$,

$$\mathcal{M}_b[R](z) = \prod_{j \geq 0} R(z^{b^j}), \qquad F(z) = R(z)F(z^b).$$

Document `mahler_mask_product`, `mahler_mask_inverse`, and `certify_mahler_functional_equation` as truncated maps in $\mathbb{Q}[z]/(z^{N+1})$. Failures: $b < 2$, nonunit constant, negative degree, budget excess, and a functional-equation residual that is nonzero in the tested range.

23.1.2 Finite carry compiler

For mask coefficients r_j, carry-state transition weights have the form

$$T_d(q, q') = r_{d+bq'-q}.$$

Document `FiniteMaskCarryPlan`, alias `NormalizedMaskPlan`, `compile_finite_mask_carry_plan`, `DigitCauchyProductPlan`, and `compile_digit_cauchy_product`, including state set, digit order, transitions, evaluation, finite certificate, and `DigitCauchyProductPlan.factor_count`.

23.1.3 Symbolic example

Compile the binary mask $R(z) = 1 + z + z^2$, write all nonzero entries of T_0, T_1, and trace one least-significant-first digit word.

23.2 Carry layers and factorial-power support

23.2.1 Kummer layer polynomial

For $0 \le k \le n$, define

$$L_{n,p}(t) = \sum_{k=0}^{n} t^{v_p\binom{n}{k}}.$$

`KummerLayerDistribution` and `kummer_layer_distribution` store the finite coefficient counts. Also document `zero_borrow_count`, `one_borrow_count`, `factorial_power_support_count`, `factorial_power_support_indices`, `CarryLayerConvolution`, and `factorial_power_convolution_layers`. The record cards must include `KummerLayerDistribution.total` and `KummerLayerDistribution.support_count`.

23.2.2 Survival predicate

For factorial power s modulo p^r,

$$\binom{n}{k}^s \not\equiv 0 \pmod{p^r} \iff s\, v_p\binom{n}{k} < r.$$

The guide must show the strict inequality and test both boundary cases $sv = r-1$ and $sv = r$.

23.3 Frobenius lifts and mask coboundaries

23.3.1 Defect maps

Document `frobenius_defect_mask` and `first_frobenius_lift` through

$$\mathfrak{D}_p R(z) = \frac{R(z^p)}{R(z)^p}, \qquad \mathfrak{L}_p R(z) = \frac{\mathfrak{D}_p R(z) - 1}{p} \pmod{p}.$$

The divisibility by p required by the second expression must be checked, not presumed.

23.3.2 Polynomial coboundary

`polynomial_mask_coboundary`, `PolynomialCoboundaryResult`, and `solve_polynomial_coboundary` concern

$$B(z)Q(z) = A(z)Q(z^b), \qquad Q(0) = 1.$$

Include the convenience property `PolynomialCoboundaryResult.found`. The result must distinguish solved, inconsistent, and underdetermined bounded linear systems. A finite-degree solution is not automatically an infinite analytic factorization.

23.3.3 Analytic and algebraic mask certificates

For `analytic_mahler_product` and `AnalyticMaskApproximation`, require $|z| < 1$ and report a finite product plus conservative tail bound. Both are ordinary floating-complex computations with `Exactness.NUMERICAL`. The bound is conservative but explicitly **not interval-rigorous**. For `certify_rational_mask_algebraic_relation` and `RationalMaskAlgebraicCertificate`, check

$$A(z)F(z)^{p-1} = B(z) \pmod{(p, z^{N+1})}.$$

Document `euler_product_collapse`, `EulerProductCollapse`, and its `collapses` property as finite exponent bookkeeping, with no unrestricted convergence claim.

23.4 Cartier sections and automata

23.4.1 Polynomial maps

Document `polynomial_multiply_mod`, `polynomial_power_mod`, `polynomial_cartier`, `rational_mask_coefficients_mod_prime`, and `cartier_degree_bound` over $\mathbb{F}_p[z]$. The Cartier operator is

$$\Lambda_d\left(\sum_{n\ge 0} a_n z^n\right) = \sum_{n\ge 0} a_{pn+d} z^n, \qquad 0 \le d < p.$$

23.4.2 Automaton compiler

Given

$$B(z)F(z) = A(z)F(z^p) \quad \text{in } \mathbb{F}_p[[z]],$$

document `CartierAutomaton`, `AutomatonCertificate`, `compile_rational_mask_automaton`, and `minimize_cartier_automaton`. Digits are read least-significant first. Evaluation has finite transition cost

$$O(\lfloor \log_p N \rfloor + 1).$$

`AutomatonCertificate` stores only `compared_terms` and the finite-prefix `failures`. Reachable states, transition closure, outputs, and degree bounds belong to `CartierAutomaton`. Document its `number_states` and `theoretical_state_bound` properties separately.

23.4.3 Carry/Cartier equivalence and linear representation

Document `finite_mask_carry_matrices`, `CartierCarryEquivalence`, `certify_cartier_carry_equivalence`, `CartierLinear Representation`, and `compile_cartier_linear_representation`. The finite claim is entrywise equality between the finite-mask carry matrices and Cartier multiplication matrices on the declared state space, not a conjugacy statement about an arbitrary external automaton.

23.4.4 Symbolic example

Evaluate a compiled representation at

$$N = 10^{18} + 12345$$

modulo 5 by displaying its base-five digit word and the corresponding ordered matrix product.

23.5 Lucas-local digit algebra

23.5.1 Local and global kernels

For `DigitKernel`, define

$$K(d,e) = \binom{d}{e}^s \pmod p, \qquad K_{\text{global}}(n,k) = \prod_i K(n_i,k_i).$$

Document `base_digits`, `factorial_power_digit_kernel`, `lift_digit_sequence`, `DigitAlgebraAudit`, and `audit_factorial_power_digit_algebra`. Member cards must cover `DigitKernel.local_weight`, `global_weight`, `local_multiply`, and `global_multiply`. The Lucas theorem used by the audit assumes that p is prime. The current `factorial_power_digit_kernel` runtime guard checks only `p >= 2`. Document that weaker guard separately so a composite input is not presented as a certified Lucas setting.

23.5.2 Exhaustive finite audit

For declared digit depth D, the certificate checks

$$\forall 0 \le n < p^D: \quad (a \star_s b)_n \equiv \prod_i (a \star_s b)_{n_i} \pmod p.$$

It must not be promoted to a universal statement for an arbitrary user kernel. Include one perturbed local weight that the audit rejects.

23.6 Congruence discovery and finite audits

23.6.1 Scalar and profile maps

Inventory `is_prime`, `p_adic_valuation_int`, `p_adic_valuation_fraction`, `rational_mod`, `base_digit_sum`, `mahler_coefficients`, `step_differences`, `DifferenceValuationProfile`, `difference_valuation_profile`, `ResidueLaneFinding`, `scan_residue_lanes`, `minimal_prefix_period`, `berlekamp_massey`, `verify_modular_recurrence`, `DecimationRelation`, and `search_decimation_relation`. Every profile is a finite map from supplied values and search bounds to a record. Infinity may occur only as the valuation of zero.

23.6.2 Audit identities

Place these beside their record classes, with tested ranges printed:

$$C_\gamma(pn+r,pk+i)\equiv C_\gamma(n,k)C_\gamma(r,i)\pmod p,$$

$$a_{pn+r}\equiv\sum_j c_j a_{n+j}\pmod p,$$

$$\begin{bmatrix}ad+b\\ rd+s\end{bmatrix}_q\equiv\binom{a}{r}\begin{bmatrix}b\\ s\end{bmatrix}_q\pmod p,\quad d=\operatorname{ord}_p(q),$$

$$B_{n+p}\equiv B_{n+1}+B_n\pmod p.$$

Record/function families: `LucasBasisAudit`, `audit_generalized_lucas`, `ConvolutionPacketAudit`, `audit_factorial_power_packet_convolution`, `QLucasAudit`, `multiplicative_order_mod`, `audit_q_lucas`, `DigitSumCongruenceAudit`, `audit_generalized_bell_digit_sum`, `WeightedDigitSumCongruenceAudit`, `audit_weighted_generalized_bell_digit_sum`, `RelationAudit`, `audit_touchard`, `audit_apery_supercongruence`, and `audit_partition_lane`. Each card needs `tested_cases`, `failure_count`, first counterexample where available, and a false-relation control.

24. Prime-power, Bell, cyclotomic, and q-Kummer systems

24.1 Exact prime-power binomial compiler

24.1.1 Valuation/unit decomposition

For prime p, precision $r\ge 1$, and $0\le k\le n$,

$$\binom{n}{k}=p^v u,\qquad p\nmid u,\qquad v=v_p\binom{n}{k}.$$

Document `factorial_valuation`, `integer_valuation`, `binomial_valuation`, `borrow_count`, `PrimePowerBinomial`, and `PrimePowerUnitCompiler`. The compiler card must include `PrimePowerUnitCompiler.unit_factorial` and `PrimePowerUnitCompiler.binomial_data`. The compiled result stores the exact valuation, unit part, and residue modulo p^r. The survival predicate is

$$\texttt{survives}\iff v<r.$$

24.1.2 Normalized prime-power kernel

For

$$\gamma_n=c^n(n!)^s u_n,$$

document `NormalizedKernelResidue`, `constant_unit_sequence`, and `PrimePowerKernelCompiler` through

$$C_\gamma(n,k)=\frac{\gamma_n}{\gamma_k\gamma_{n-k}}pmodp^r.$$

The compiler must separate the carry valuation from the unit ratio. Strict guards: prime modulus base, positive precision, nonnegative indices, $k\le n$, and unit-valued twist data.

24.1.3 Cost and example

Prefix storage is $O(p^r)$. A memoized query uses $O(\log_p n)$ scale steps. Check

$$\binom{137}{52}^2 \pmod{3^5}$$

against direct exact arithmetic and include the valuation-boundary branch.

24.2 Carry-linear normalization

24.2.1 Classification target

Document `CarryLinearValuation`, `CarryLinearCertificate`, and `classify_carry_linear_valuation` for a supplied valuation callback ν:

$$\nu(n) = an + s\, v_p(n!).$$

Include `CarryLinearValuation.defect`, the pointwise difference between the observed valuation and this affine/factorial model. Then

$$\nu(n) - \nu(k) - \nu(n-k) = s\, v_p\binom{n}{k}.$$

This is a finite classification certificate over the declared prefix, not a proof for an arbitrary callback outside that prefix.

24.2.2 Unit twists

Document `base_digits`, `digit_unit`, `local_normalized_kernel`, and `digit_factorized_normalized_kernel` with

$$u(n) = \prod_i w_{n_i} \pmod{p}.$$

Failure cases: wrong weight-table length, zero/nonunit digit weight where a unit is required, composite p, and index/radix violations.

24.3 Bell digit reductions

24.3.1 Pointed subindex identity

Document `bell_period_bound`, `pointed_subindex_polynomial`, `PointedSubindexCertificate`, and `certify_pointed_subindex_identity` through the finite congruence

$$P_{n,p}(u) \equiv (1+u)^{s_p(n)-1} \pmod{p}.$$

The exact meaning and treatment of the $s_p(n) = 0$ endpoint must follow the implementation rather than an informal negative exponent.

24.3.2 Generalized Bell reduction

Under $s \geq 2$ and $p-1 \mid s-1$,

$$B_n^{(s)}(x) \equiv B_{s_p(n)}^{(1)}(x) \pmod{p}, \qquad s \geq 2, \qquad p-1 \mid s-1.$$

Here $B_j^{(1)}(x)$ is the ordinary Bell polynomial. The implementation constructs the reduced family with power 1. It does not reuse power s on the right-hand side. Document `BellDigitReduction`, `generalized_bell_digit_reduction`, `weighted_bell_digit_reduction`, `generalized_bell_parity`, `BellParityAutomaton`, and `BellResidueAutomaton`. The finite period bound is

$$1 + p + \cdots + p^{p-1} = \frac{p^p - 1}{p-1}.$$

24.3.3 Symbolic example

Use $(n, p, s) = (137, 5, 9)$. Display the base-five digits, digit sum, reduced index, direct residue, compiled residue, and certificate fields.

24.4 q-Kummer valuation branches

24.4.1 Exact Gaussian objects

Document `multiplicative_order_mod_prime`, `q_integer`, `q_factorial`, and `gaussian_binomial`:

$$[n]_q = \frac{q^n - 1}{q - 1}, \quad [n]_q! = \prod_{j=1}^{n} [j]_q, \quad \begin{bmatrix} n \\ k \end{bmatrix}_q = \frac{[n]_q!}{[k]_q![n-k]_q!}.$$

24.4.2 Decomposition map

`q_binomial_valuation_decomposition` has the partial map signature

$$(n, k, q, p) \longmapsto (\text{block length}, \text{quotient carries}, \text{residue carry}, \text{endpoint term}, v_p\left(\begin{bmatrix} n \\ k \end{bmatrix}_q\right)).$$

Document all branches exactly: 1. $p \mid q$: the Gaussian binomial is a p-adic unit. 2. Odd $p \nmid q$: $d = \operatorname{ord}_p(q)$. 3. $p = 2$, odd q: block length 2 and lift constant $v_2(q + 1)$. Also document `QKummerDecomposition`, `q_binomial_valuation_direct`, and `q_binomial_support`, with

$$\begin{bmatrix} n \\ k \end{bmatrix}_q^s \not\equiv 0 \pmod{p^r} \iff s\, v_p\left(\begin{bmatrix} n \\ k \end{bmatrix}_q\right) < r.$$

24.4.3 Adversarial examples

- Main example: $(n, k, q, p) = (48, 22, 2, 5)$.
- Endpoint checks: $k = 0$, $k = n$, and $k > n$.
- Branch checks: $p \mid q$, odd-prime unit base, and dyadic odd base.
- Record method-specific invalid-index behavior: `q_binomial_valuation_decomposition` rejects $k < 0$ or $k > n$, while `gaussian_binomial` returns zero for those subindices. `q_integer` and `q_factorial` reject negative indices. `q_binomial_support` currently returns an empty tuple for a negative total index rather than raising.
- The integer q-Kummer maps that evaluate q-integers require $q > 1$, unlike `q_factorial_basis`, which permits `q=1`. Reject nonprime p and nonpositive power or precision on the branches that accept those parameters.

24.5 Cyclotomic collapse

24.5.1 Orbit decomposition

Write every index as

$$m = up^j, \qquad p \nmid u.$$

Document `p_free_orbit`, `CyclotomicCollapse`, `cyclotomic_coboundary`, `normalized_cyclotomic_expression`, `verify_cyclotomic_coboundary`, and `verify_cyclotomic_exponent_relation`. Include the boolean property `CyclotomicCollapse.collapses`.

24.5.2 Exact obstruction

For normalized cyclotomic exponents, solve

$$R(z) = \frac{Q(z)}{Q(z^p)}.$$

Writing the exponent at up^j as $e_{u,j}$, the implementation reports a coboundary obstruction exactly when

$$e_{u,0} \neq 0$$

for a p-free orbit. This differs from `euler_product_collapse`, whose obstruction is a nonzero terminal orbit sum. Optional SymPy verifies the symbolic expression. The divisor-lattice verifier remains exact without SymPy.

24.5.3 Positive and negative examples

Show one exponent ledger that telescopes along every orbit and one with a single uncancelled orbit residual. Print both the candidate Q and the obstruction map.

25. Local arithmetic models

25.1 Exact local domains and valuations

25.1.1 Ring maps

Document `ModularRing`, `PrimeField`, and `TruncatedPadic` as

$$\mathbb{Z} \to \mathbb{Z}/M\mathbb{Z}, \quad \mathbb{Z} \to \mathbb{F}_p, \quad \mathbb{Z} \to \mathbb{Z}/p^r\mathbb{Z}.$$

Include all public operations: `normalize`, `add`, `subtract`, `multiply`, `power`, `is_unit`, `inverse`, `prime`, `modulus`, `ring`, `valuation`, `refine`, and `coarsen`. Refinement changes the visible modulus, not the mathematical knowledge encoded by an existing coarse residue. Coarsening is canonical reduction.

25.1.2 Valuation identities

Inventory `digit_sum`, `padic_valuation`, `factorial_valuation`, `binomial_valuation`, `binomial_valuation_from_digits`, `falling_factorial`, `carmichael_exponent_prime_power`, `unit_difference_value`, `unit_difference_valuation`, `verify_binomial_digit_formula`, and `exact_binomial`.

$$v_p(n!) = \sum_{j \geq 1} \left\lfloor \frac{n}{p^j} \right\rfloor,$$

$$v_p\binom{n}{k} = \frac{s_p(k) + s_p(n-k) - s_p(n)}{p-1}.$$

Use $v_p(0) = +\infty$. Reject nonprime p, negative factorial indices, and invalid binomial subindices according to each function's source contract.

25.2 Hurwitz-series algebra

25.2.1 Product and inverse

For

$$A(z) = \sum_{n=0}^{N} a_n \frac{z^n}{n!}, \qquad B(z) = \sum_{n=0}^{N} b_n \frac{z^n}{n!},$$

document `hurwitz_product` and `hurwitz_inverse` through

$$(A \star_H B)_n = \sum_{k=0}^{n} \binom{n}{k} a_k b_{n-k}, \qquad (A \star_H A^{-1})_n = \delta_{n0}.$$

`HurwitzSeries` cards: `degree`, `coefficient`, `truncate`, `multiply`, `inverse`, `evaluate_egf`, `identity`, and `exponential_mode`. Without a modulus, coefficients may be arbitrary `Number` values: `int` and `Fraction` inputs give exact finite algebra, whereas floating or complex inputs give numerical algebra. Modular mode retains exact residue arithmetic.

25.2.2 Subgroup selector and compressed inverse

For odd prime p and $m \mid p-1$,

$$\frac{1}{m} \sum_{\omega \in \mu_m} \omega^n = \mathbf{1}_{m \mid n} \quad \text{in } \mathbb{F}_p.$$

Document `subgroup_roots`, `subgroup_selector`, `compressed_hurwitz_inverse`, and `compressed_period`, including

$$u_0 = 1, \qquad u_n = -\sum_{q<n} \binom{mn}{mq} u_q \pmod{p}, \qquad h = \frac{p-1}{m}.$$

The positive-tail period statement must not be rewritten as a period including the exceptional constant term without verification.

25.3 Standard residual windows

Document `PeriodicGeneratingFunction`, `StandardWindow`, `standard_window_inverse`, `standard_window_period`, and `subgroup_standard_window_inverse`. The residual OGF has the form

$$u_0 + \frac{P(z)}{1 - z^h}, \qquad h = (p-1)/m.$$

The uniform positive-tail period and residual OGF require

$$p \text{ odd prime}, \qquad m \mid p-1, \qquad h = (p-1)/m.$$

`StandardWindow(2,1)` can be constructed, but its `period` and `rational_generating_function` reject this exceptional prime. The periodic claim starts at coefficient 1. The constant term is exceptional. `StandardWindow.certificate` must check address independence, recurrence, claimed positive-tail period, and minimum observed period. Add member cards for `has_uniform_period`, `reduced_inverse`, `exact_coefficients`, `residues`, `rational_generating_function`, and `minimum_period`. The worked example uses $p = 7, m = 2$ and address 3.

25.4 Odd and dyadic Euler coordinates

25.4.1 Source sequences and kernels

Inventory `bernoulli_numbers`, `signed_euler_numbers`, `signed_euler_number`, `euler_zigzag_numbers`, `euler_zigzag_number`, `odd_power_sum`, `dyadic_source`, `dyadic_beta`, `dyadic_kernel`, `odd_kernel`, `EulerTruncation`, `EulerLocalCoordinates`, `dyadic_kernel_valuation`, and `dyadic_row_weight`.

$$\operatorname{sech} t = \sum_{n \geq 0} E_n \frac{t^n}{n!}, \qquad \sec t + \tan t = \sum_{n \geq 0} A_n \frac{t^n}{n!}.$$

For even n,

$$E_n = \sum_{r=0}^{n/2} \binom{n}{2r} \kappa_{p,r} A_{p,n-2r}.$$

25.4.2 Dyadic and odd branches

$$A_{2,n} = \frac{3^{n+2} - 1}{4(n+1)(n+2)},$$

$$\kappa_{2,0} = 1, \qquad \kappa_{2,r} = -2^{4r}(2r-1)B_{2r} \quad (r > 0),$$

$$\kappa_{p,r} = (-1)^r A_{2r+1} p^{2r} \quad (p \text{ odd}).$$

For $r > 0$,

$$v_2(\kappa_{2,r}) = 4r - 1.$$

Truncation at depth d uses the exact moduli 2^{4d+3} and p^{2d+2} in their respective branches.

25.4.3 Certificate anatomy

`EulerTruncation` stores truncated value, exact Euler value, error, modulus, and congruence status. `EulerLocalCoordinates.certificate` checks forward and inverse reconstruction and the finite row-valuation ledger. The example must print every retained row and the first discarded row. The `EulerLocalCoordinates` ledger must also name `is_dyadic`, `inverse_kernel`, `reconstruct`, `inverse_reconstruct_source`, `kernel_valuation`, `row_weight`, `formula_row_weight`, and `sharp_modulus`.

25.5 Mahler transfer and arithmetic polyphase

25.5.1 Newton/Mahler coordinates

Document `generalized_binomial`, `difference_table`, `mahler_coefficients`, and `MahlerSeries`:

$$f(x) = \sum_{r=0}^{d} \Delta^r f(0) \binom{x}{r}.$$

Cards: `degree`, `from_values`, `evaluate`, `first`, `difference`, `affine_subsequence`, `valuation_ledger`, and `reconstruction_certificate`. `MahlerSeries` accepts general numeric values. Its finite identities and certificate are exact for exact data and numerical for floating or complex data. The reconstruction certificate selects that status from the value types.

25.5.2 Affine kernel

For

$$\binom{mq+s}{r} = \sum_{k} K_{m,s}(r,k) \binom{q}{k},$$

$$K_{m,s}(r,k) = \sum_{j=0}^{k} (-1)^{k-j} \binom{k}{j} \binom{mj+s}{r}.$$

Document `affine_mahler_kernel`, `affine_kernel_matrix`, `affine_kernel_composition_certificate`, `prime_power_kernel_residue`, and `prime_power_kernel_certificate`, including

$$K_{m,s} K_{n,t} = K_{mn,mt+s},$$

$$K_{p^a,s}(r,k) \equiv \binom{s}{r-p^a k} \pmod{p}.$$

25.5.3 Unit and matrix modes

Document `UnitModeMahler`, `matrix_mahler_coefficients`, and `evaluate_matrix_mahler` through

$$u^{hx+j} = u^j \sum_{r \geq 0} (u^h - 1)^r \binom{x}{r},$$

with $u^h - 1$ replaced by $C^h - I$ for matrix modes. Include `UnitModeMahler.increment` and `UnitModeMahler.valuation_slope`.

25.5.4 Polyphase maps

Document `ArithmeticPolyphase`, `polyphase_analyze`, `polyphase_synthesize`, `mixed_radix_address`, `mixed_radix_digits`, and `mahler_polyphase`:

$$a \mapsto ((a_{mq+r})_{q \geq 0})_{r=0}^{m-1}.$$

Include `ArithmeticPolyphase.reconstruct` as the class-level synthesis map. Nested digits are least-significant first:

$$j = d_0 + r_0 d_1 + r_0 r_1 d_2 + \cdots.$$

Examples must verify analysis/synthesis identity, nested/direct address agreement, and refine/coarsen equality.

25.6 Recurrence packets

25.6.1 Exact state evolution

Document `matrix_power_exact`, `matrix_power_mod`, `determinant_integer`, `OrbitPeriod`, `LinearRecurrence`, `RecurrencePacket`, `second_order_trace`, `second_order_jump`, `fibonacci_recurrence`, and `lucas_recurrence`.

$$x_{n+d} = \sum_{i=0}^{d-1} c_i x_{n+i}, \qquad X_{n+1} = CX_n.$$

The residue packet obeys

$$X_{mk+r} = (C^m)^k X_r.$$

`LinearRecurrence` and `RecurrencePacket` accept general numeric coefficients and initial values. Their evolution is exact for exact data and numerical for floating or complex data. Their certificates select `EXACT` versus `NUMERICAL` from those types.

25.6.2 Public-member cards

For `LinearRecurrence`: `order`, `companion_matrix`, `initial_state`, `term`, `term_mod`, `first`, `state`, `jump_matrix`, `residue_packet`, `period_mod`, `characteristic_polynomial`, `jump_characteristic_polynomial`, and `certificate`. For `RecurrencePacket`: `jump_matrix`, `initial_state`, `state`, `term`, `term_mod`, `first`, and `certificate`.

25.6.3 Finite-state and symbolic boundaries

Period classification modulo M is exact in a finite state space. If the companion matrix is invertible modulo M, preperiod must be zero. Symbolic characteristic-polynomial rendering requires optional SymPy. Example: compute $F_{1{,}000{,}000}$ mod $1{,}000{,}003$ without constructing the full integer, then classify the Fibonacci orbit modulo 32.

25.7 Carmichael jumps and simultaneous congruences

Document `DifferenceCongruence`, `local_unit_jump`, `unit_mode_for_precision`, `finite_difference_value`, `certify_difference_congruence`, and `simultaneous_unit_difference`.

$$\Delta_h^r f(n) = \sum_{k=0}^{r} (-1)^{r-k} \binom{r}{k} f(n + kh).$$

For a local unit u, $h = \lambda(p^d)$ gives

$$v_p(\Delta_h^r u^n) \geq dr.$$

For primes p_i, use

$$h = \operatorname{lcm}_i \lambda(p_i^{d_i}).$$

The example should certify the 2-, 3-, and 5-adic lower bounds and include a nonunit-base rejection.

26. Exact sequence execution

26.1 Lazy sequence protocol

26.1.1 `Sequence`

Model

$$a : \mathbb{N} \to \mathcal{R}$$

with cached numeric evaluation carrying an explicit exactness flag. `Sequence.exact` is constructor metadata rather than an enforced proof of scalar arithmetic. Document `term`, `first`, `residue`, `modulo`, `map`, finite slicing, and lazy iteration.

$$\text{residue}_{m,r}(a)(k) = a_{mk+r}, \qquad \text{modulo}_M(a)(k) = a_k \bmod M.$$

Transforms propagate the exactness flag by default. `map(..., exact=...)` may override it explicitly. `modulo` requires integral rational terms. Slices require a finite stop. Infinite iteration is intentionally unbounded.

26.2 Exact catalog

26.2.1 Constructors

Document `fibonacci_sequence`, `lucas_sequence`, `signed_euler_sequence`, `euler_zigzag_sequence`, `bernoulli_sequence`, `catalan_sequence`, `bell_sequence`, `motzkin_sequence`, `partition_sequence`, `apery_sequence`, `thue_morse_sequence`, `sequence_catalog`, and `get_sequence`.

26.2.2 Defining equations

$$F_{n+2} = F_{n+1} + F_n, \quad (F_0, F_1) = (0, 1),$$
$$L_{n+2} = L_{n+1} + L_n, \quad (L_0, L_1) = (2, 1),$$
$$C_n = \frac{1}{n+1}\binom{2n}{n}, \qquad B_{n+1} = \sum_{k=0}^{n} \binom{n}{k} B_k,$$
$$n\,p(n) = \sum_{k=1}^{n} \sigma(k) p(n-k),$$
$$t_{2n} = t_n, \qquad t_{2n+1} = 1 - t_n.$$

Print aliases `apery-zeta3`, `euler-zigzag`, `signed-euler`, `thue-morse`, and `thuemorse`. Unknown names must fail explicitly.

26.3 Automatic sequences

26.3.1 DFAO model

Document `base_digits`, `AutomaticSequence`, and `thue_morse_automaton`. For base b, states Q, transition δ, initial state q_0, and output τ,

$$a_n = \tau(\delta^*(q_0, \text{digits}_b(n))).$$

Digits are most-significant first in this namespace. The initial state must have a zero loop so canonical leading zeros do not change the value.

26.3.2 Cards and certificate

Document `state_count`, `state`, `term`, `first`, `residue_model`, `to_sequence`, and `certificate`. For $0 \le r < b^e$,

$$\text{residue_model}(e, r)(n) = a_{b^e n + r}.$$

The example must display the two-state Thue-Morse transition graph, certify the first 1,000 terms, and compile all residue models below 2^5.

26.4 Holonomic recurrences

26.4.1 Model

Document type alias `Polynomial`, `evaluate_polynomial`, `HolonomicRecurrence`, `motzkin_recurrence`, and `apery_recurrence`:

$$\sum_{j=0}^{r} P_j(n) a_{n+j} = 0.$$

The final coefficient $P_r(n)$ must be nonzero at every executed index. Terms and polynomial coefficients are exact rationals.

26.4.2 Apéry recurrence

$$(n+1)^3 A_n - \left(34(n+1)^3 + 51(n+1)^2 + 27(n+1) + 5\right) A_{n+1} + (n+2)^3 A_{n+2} = 0,$$
$$(A_0, A_1) = (1, 5).$$

The finished guide must print the Motzkin coefficient tuples directly from the source and derive its first terms, avoiding hand-copied polynomial errors.

26.4.3 Certificate/failure split

Document `order`, `coefficient`, `term`, `first`, `to_sequence`, and `certificate`. Fail on insufficient initial values, zero leading coefficient at an executed index, invalid index, or malformed polynomial coefficients.

26.5 Sequence compiler and strongest-model selection

26.5.1 Selection order

`compile_sequence` follows this symbolic decision relation:

$$\begin{aligned}\text{declared constant recurrence} \succ \text{declared DFAO} \succ \text{declared holonomic model}\\ \succ \text{prefix-fitted recurrence} \succ \text{direct evaluator}.\end{aligned}$$

Document `detect_linear_recurrence`, `recurrence_generating_function`, `CompiledSequence`, and `compile_sequence`.

26.5.2 Recurrence OGF

For

$$a_{n+d} = \sum_{j=0}^{d-1} c_j a_{n+j},$$

$$Q(z) = 1 - c_{d-1} z - c_{d-2} z^2 - \cdots - c_0 z^d, \qquad F(z) = \frac{P(z)}{Q(z)}.$$

Cards for `CompiledSequence`: `term`, `first`, `modulo`, and `generating_function`.

26.5.3 Truth-status boundary

A detected relation gives

$$\forall n \in I_{\text{sample}} : \mathcal{R}(a, n) = 0,$$

not an unconditional infinite recurrence. The compiled record must retain the strategy and certificate provenance.

26.6 Exact transforms

Document `binomial_transform_values`, `binomial_transform`, `inverse_binomial_transform_values`, `inverse_binomial_transform`, `finite_difference`, `partial_sums`, and `cauchy_product`:

$$b_n = \sum_{k=0}^{n} \binom{n}{k} a_k, \qquad a_n = \sum_{k=0}^{n} (-1)^{n-k} \binom{n}{k} b_k,$$

$$\Delta_h^r a_n = \sum_{k=0}^{r} (-1)^{r-k} \binom{r}{k} a_{n+kh},$$

$$s_n = \sum_{k=0}^{n} a_k, \qquad c_n = \sum_{k=0}^{n} a_k b_{n-k}.$$

Examples: exact transform/inverse identity. Fifth difference of a quartic is zero. Cauchy coefficients equal ordinary-GF multiplication coefficients.

27. Cyclic analytic systems

27.1 Exact/numerical boundary

The chapter opens with the type split

$$\text{finite coefficient algebra over } \mathbb{Q} \quad \oplus \quad \text{numerical entire-function evaluation over } \mathbb{C}.$$

`analytic.status()` and `analytic.require()` report feature availability only. They do not certify a user-supplied analytic claim.

27.2 Cyclic packets and projections

27.2.1 Coefficient projector

Document `cyclic_project_coefficients`, `cyclic_packets`, `cyclic_reconstruct`, `cyclic_project_value`, `cyclic_product_packets`, and `cyclic_projection_certificate`.

$$(P_{m,r}a)_n = \mathbf{1}_{n \equiv r \pmod m} a_n,$$

$$P_{m,r} f(z) = \frac{1}{m} \sum_{\ell=0}^{m-1} \omega^{-r\ell} f(\omega^\ell z), \qquad \omega = e^{2\pi i/m}.$$

Packet multiplication is cyclic convolution in the residue address:

$$(AB)_t = \sum_{r+s \equiv t \pmod m} A_r B_s.$$

The finite coefficient reconstruction is exact. The floating norm in its certificate is a diagnostic encoding of that reconstruction residual.

27.3 Transfer and signed section families

27.3.1 Entire sections

Document `transfer_section`, `signed_section`, `transfer_sections`, `signed_sections`, `denominator`, `denominator_root_sum`, `denominator_derivative`, `transfer_shift_matrix`, `signed_shift_matrix`, `cyclic_addition`, `section_certificate`, and `CyclicSectionFamily`.

$$T_{m,r}(z;c)=\sum_{n\ge 0}\frac{(cz)^{mn+r}}{(mn+r)!},$$

$$S_{m,r}(z)=\sum_{n\ge 0}\frac{(-1)^n z^{mn+r}}{(mn+r)!},\qquad D_m(z)=S_{m,0}(z).$$

27.3.2 Root sum and finite dynamics

For

$$\eta_k=e^{(2k+1)\pi i/m},$$

$$D_m(z)=\frac{1}{m}\sum_{k=0}^{m-1}e^{\eta_k z},\qquad D_m^{(r)}(z)=\frac{1}{m}\sum_k \eta_k^r e^{\eta_k z}.$$

The shift matrices satisfy

$$J_T(c)^m=c^m I,\qquad J_S^m=-I.$$

Only the unit-scale transfer matrix has $J_T(1)^m=I$. The signed constructor is unscaled. The generator used by a signed `CyclicSectionFamily` with scale c is cJ_S, so

$$(cJ_S)^m=-c^m I.$$

`cyclic_addition` is finite coordinate convolution for adding arguments. `section_certificate` is numerical agreement between series and root-sum representations, with truncation/tolerance stated.

27.3.3 Class cards

For `CyclicSectionFamily`, document `evaluate`, `vector`, and `fundamental_matrix`, including order, residue, scale, derivative index, dtype, and error conditions.

27.4 Reciprocal and Appell families

27.4.1 Reciprocal powers

Document `logarithmic_coefficients`, `reciprocal_coefficients`, `reciprocal_polynomial`, `reciprocal_value`, `transfer_tangent`, `transfer_tangent_coefficients`, and `reciprocal_semigroup_certificate`.

$$D_m(z)^{-r}=\sum_{n\ge 0}P_{m,n}(r)\frac{z^{mn}}{(mn)!},$$

$$-\frac{D_m'(z)}{D_m(z)}$$

is the transfer tangent. Parameter addition gives the exact finite identity

$$P_{m,n}(r+s)=\sum_{q=0}^{n}\binom{mn}{mq}P_{m,q}(r)P_{m,n-q}(s).$$

Rational parameters remain exact. Complex parameters are numerical. `reciprocal_value` uses the principal complex branch and rejects $D_m(z)=0$. Symbolic polynomial construction is optional-SymPy.

27.4.2 Congruential Appell families

Document `congruential_reciprocal_coefficients`, `congruential_inverse`, `appell_polynomial`, `appell_coefficients`, `appell_polynomial_symbolic`, `theta_polynomial`, and `theta_polynomial_symbolic`.

$$\Phi_{m,j}(w) = j! \sum_{k\ge 0} \frac{w^k}{(mk+j)!},$$

with reciprocal coefficients from $\Phi_{m,j}(w)^{-r}$. Then

$$A_n^{(m,j,r)}(x) = \sum_{k=0}^{\lfloor n/m \rfloor} \binom{n}{mk} C_k^{(m,j,r)} x^{n-mk}.$$

The theta polynomial is $(mn)!$ times the coefficient of w^n in

$$\left(\sum_{k\ge 0} \frac{u^{mk} w^k}{(mk)!} \right)^{-1} \left(\sum_{k\ge 0} \frac{w^k}{(mk)!} \right)^{-u}.$$

The guide should connect `congruential_inverse` to the standard-window chapter without conflating rational exact coefficients with modular residual windows.

27.5 Internal finite-series helpers

Document these as advanced module API rather than recommended top-level imports: `validate_order`, `validate_degree`, `multiply_series`, `inverse_series`, `logarithmic_coefficients`, `power_series_from_log`, `signed_denominator_coefficients`, and `positive_section_coefficients`. Their finite identities include

$$-\log A(w) = \sum_{n\ge 1} \kappa_n \frac{w^n}{n},$$

$$\exp\left(r \sum_{n\ge 1} \kappa_n \frac{w^n}{n} \right),$$

$$D_m(w) = \sum_{n\ge 0} \frac{(-1)^n w^n}{(mn)!}, \qquad \Phi_{m,j}(w) = j! \sum_{n\ge 0} \frac{w^n}{(mn+j)!}.$$

All are exact through finite degree for `Fraction` inputs. The guide must avoid presenting an internal helper as a stronger public stability promise.

27.6 Positive zero strings and finite zero diagnostics

27.6.1 Public maps

Document these public maps and records:

- `ZeroStringResult`, `asymptotic_positive_zero`, and `positive_zero_string`.
- `zero_count_asymptotic`, `hadamard_partial_product`, and `reciprocal_coefficient_from_zeros`.
- `reduced_zero_zeta` and `CyclicDenominator`.

Class cards cover `ZeroStringResult.rotational_orbits` and `CyclicDenominator.derivative`, `positive_zeros`, `zero_count`, and callable evaluation.

27.6.2 Numerical models

The positive asymptotic seed is

$$\rho_k^{(0)} = \frac{\pi(k+\frac{1}{2})}{\sin(\pi/m)}.$$

Leading count models are

$$N_m(R) \sim \frac{m\sin(\pi/m)}{\pi}R, \qquad N_m^{\mathrm{red}}(R) \sim \frac{\sin(\pi/m)}{\pi}R^{1/m}.$$

The finite products and sums are

$$H_K(z) = \prod_{k=0}^{K-1}\left(1-\frac{z^m}{\rho_k^m}\right), \qquad Z_K(s) = \sum_{k=0}^{K-1}\rho_k^{-ms}.$$

27.6.3 Mandatory scope box

`positive_zero_string` certifies only the returned numerical roots:

$$\widetilde{D}_m(x) = e^{-x\cos(\pi/m)}D_m(x) = \frac{1}{m}\sum_{j=0}^{m-1} e^{(\eta_j-\cos(\pi/m))x},$$

$$|\widetilde{D}_m(\rho_k)| \le 20\tau, \qquad 0<\rho_0<\rho_1<\cdots<\rho_{K-1},$$

where τ is the requested tolerance. `ZeroStringResult.residuals` stores this scaled residual, not $|D_m(\rho_k)|$. It does not certify global exhaustion. `reduced_zero_zeta` omits an unchecked tail. Its Hurwitz-zeta leading model has an unchecked correction and is conditional. `reciprocal_coefficient_from_zeros` is a finite approximation, not an exact infinite-product coefficient identity.

27.6.4 Symbolic/numerical example

For $m = 4$: 1. form the exact finite secant reciprocal coefficients $(1, 1, 5, 61, ...)$. 2. Compute four positive zero representatives numerically. 3. Print each Newton residual and ordering gap. 4. Compare a finite Hadamard coefficient with the exact reciprocal coefficient. 5. Label the discrepancy as truncation error, not theorem failure.

Cross-chapter dependency and example matrix

Formal dependencies

$$\begin{gathered}\text{domains} \to \text{weighted bases} \to \text{finite series} \to \text{declarative functions} \\ \to \text{mask/Cartier compilers} \to \text{prime-power systems.}\end{gathered}$$

$$\begin{gathered}\text{local arithmetic} \to \text{recurrence packets} \to \text{lazy sequences} \\ \to \text{compiled execution.}\end{gathered}$$

$$\begin{gathered}\text{cyclic projections} \to \text{entire sections} \to \text{reciprocal/Appell families} \\ \to \text{finite zero diagnostics.}\end{gathered}$$

Existing runnable example anchors

- `custom_generating_functions.py`: normalized series, rational/algebraic sources, basis conversion, frozen records.
- `mahler_cartier_compilers.py`: masks, carry plans, Cartier automata, coboundaries, large-index evaluation.
- `prime_power_generating.py`: prime-power binomials, Bell digit reduction, q-Kummer decomposition.
- `standard_window_mahler.py`: standard windows and affine Mahler transfer.
- `local_euler_coordinates.py`: odd/dyadic Euler branches and truncation certificates.
- `recurrence_packets.py`: exact/modular recurrence execution and periods.
- `sequence_compiler.py`: declared, detected, and fallback sequence models.
- `cyclic_analytic.py`: cyclic sections, reciprocal/Appell coefficients, numerical zero records.

Adversarial example requirements

1. Reciprocal over a nonfield with a nonunit constant coefficient.
2. Composition with nonzero inner constant.
3. False finite-prefix recurrence that passes fit data but fails held-out data.
4. Perturbed Lucas-local digit kernel rejected by exhaustive finite audit.
5. Prime-power support case at the exact valuation cutoff.
6. q-Kummer input routed through each valuation branch.
7. Cyclotomic exponent ledger with one surviving orbit obstruction.
8. Analytic zero list with an intentionally insufficient Newton tolerance.

Per-card completion invariant

No symbol card in Chapters 21 through 27 is complete until it contains

$$\boxed{\begin{gathered}\text{import path} + \text{Python signature} + \text{mathematical map} \\ +\text{domain} + \text{hypotheses} + \text{truth status} \\ +\text{certificate fields} + \text{failure branches} + \text{example}\end{gathered}}.$$

The reader explanations and these specifications must agree without weakening or broadening any recorded claim.

85. Algorithms and signals

Specifications for structured algorithms and finite signals

These specifications supplement Chapters 28 through 36A. Each item records a definition, API contract, derivation, example requirement, certificate interpretation, or failure boundary.

VI.0 Common symbols, claim labels, and result types

VI.0.1 Scalar and array domains

- Scalar field: $\mathbb{K} \in \{\mathbb{R}, \mathbb{C}\}$, with selected routines also accepting exact object scalars.
- Dense vector: $x \in \mathbb{K}^n$.
- Batched right-hand side: $X \in \mathbb{K}^{n \times r}$, $r \geq 1$.
- Matrix: $A \in \mathbb{K}^{m \times n}$.
- Finite signal: $x = (x_0, \dots, x_{N-1}) \in \mathbb{K}^N$.
- Sample rate: $f_s \in \mathbb{R}_{>0}$.
- Norms: $\|\cdot\|_2$ for vectors and spectral matrix norm unless another norm is displayed.
- Adjoint: $A^* = \overline{A}^{\mathsf{T}}$.
- Machine comparison scale: $s(a, b) = \max(1, \|a\|, \|b\|)$.
- Numerical acceptance predicate:

$$\operatorname{close}_\tau(a, b) \iff \|a - b\| \leq \tau\, s(a, b), \qquad \tau \geq 0.$$

- `close_tau` is guide notation only, not a universal implementation comparator. Each API card must reproduce the source comparator's exact norm, scaling, strictness, multiplier, fixed internal tolerance, and validation behavior. Implementations include scaled and absolute residuals, strict lower bounds, fixed internal tolerances, and branches that return an invalid certificate rather than applying one uniform tolerance rule.

VI.0.2 Claim labels

- **E**: exact finite combinatorics, indexing, shape law, or algebra over exact inputs.
- **N**: finite numerical result checked against an explicit tolerance.
- **C**: conditional mathematical statement with user-declared assumptions.
- **H**: heuristic, estimator, or data-dependent discovery rule.
- **M**: declared cost or traffic model, not an observed runtime theorem.
- **O**: optional backend whose import, runtime, and device execution are separate facts.
- **I**: integrity or deterministic identity record.

VI.0.3 Canonical return distinction

- Plain value: $y = f(x)$.
- Certified value: `ComputationResult[T] = (value: T, certificate: Certificate)`.
- Diagnostic record: finite observations without a theorem claim.
- Conditional certificate:

$$\mathcal{C} = (v, e, R, \tau, A_\checkmark, A_?, D, T, m),$$

where v is validity, e exactness, R residuals, τ tolerance, $A_\checkmark$ checked assumptions, $A_?$ unchecked assumptions, D diagnostics, T theorem identifier, and m message.

VI.0.4 Coverage routing

- Chapters 28 through 30: every public symbol in `algorithms.layout`, `ir`, `structured`, `low_rank`, `support`, `krylov`, `precision`, `compiler`, `scheduling`, `complexity`, `discovery`, `gpu`, and `benchmarks`.
- Chapters 31 through 36A: every public symbol in the twenty `signals` modules listed in the source inventory.
- Compatibility aliases are documented at the canonical implementation card and indexed separately.
- Internal `_validation` helpers are contributor material, not supported user imports.

VI.0.5 Package controls and compatibility aliases

- Algorithms namespace controls: `algorithms.FEATURE_KEY = "algorithms"`. `algorithms.status()` returns that registered feature specification. `algorithms.require()` is the implemented-namespace compatibility no-op and returns `None`.
- Signals namespace controls: `signals.FEATURE_KEY = "signals"`. `signals.status()` returns that registered feature specification.
- Exact compatibility aliases:

$$\texttt{FinitePolyphaseKernel} = \texttt{PolyphaseBlockMatrix},$$
$$\texttt{finite_polyphase_filter_kernels} = \texttt{finite_polyphase_blocks}.$$

- Recovery routing aliases: `VectorJumpRecoveryResult` is `signals.edge_recovery.JumpRecoveryResult`. `recover_vector_jumps` is `signals.edge_recovery.recover_jumps`. `recover_vector_jumps_subgaussian` is `signals.edge_recovery.recover_jumps_subgaussian`.
- Singular-packet aliases:

$$\texttt{RetainedPacketReconstruction} = \texttt{RetainedPacketResult},$$
$$\texttt{TruncatedPowerPacket} = \texttt{TruncatedPowerTerm}.$$

- `shannon_entropy_cost(coefficients, path="")` is a compatibility wrapper that calls `entropy_cost(coefficients, path)`. Document behavioral equivalence without claiming object identity.

VI.0.6 Duplicate-name import resolution

- Package import `komorebi_math.signals.polyphase_convolution` resolves to `signals.convolution.polyphase_convolution(signal, kernel, order, *, mode="full", tolerance=1e-11)`, the finite convolution-mode API.
- Module import `komorebi_math.signals.polyphase.polyphase_convolution(signal, kernel, order)` is a distinct full-convolution coordinate function and is not separately re-exported under another package-level name.
- Package import `komorebi_math.signals.finite_difference` resolves to `signals.recovery.finite_difference(values, *, order=1, step=1)`.
- `komorebi_math.signals.singular_packets.finite_difference(signal, order=1)` remains module-public but is not package-re-exported.
- Package `haar_step_coefficient(amplitude, support_length, relative_location)` comes from `signals.recovery`.
- Package `step_haar_coefficient(amplitude, relative_location, interval_length)` comes from `signals.edge_recovery`. The distinct names preserve the distinct argument orders.

---

28. Packet intermediate representation

28.0 Chapter contract

- Goal: encode $A : X \to Y$ by finite packet blocks with explicit layout, support, storage, arithmetic, and execution semantics.
- Exact layer: packet names, sizes, slices, addresses, graph relations, and structural composition.
- Numerical layer: block values, thresholded omission, materialized equality, and floating certificates.
- Central warning: structural support is an upper inclusion because different block paths may cancel numerically.

28.1 `PacketLayout`: typed packet coordinate spaces

Let

$$X = \bigoplus_{s=0}^{p-1} X_s, \qquad \dim X_s = n_s > 0, \qquad n = \sum_{s=0}^{p-1} n_s.$$

- Constructor data: `sizes = (n_0,...,n_{p-1})`, optional unique nonempty `names`. - Derived offsets:

$$o_0 = 0, \qquad o_{s+1} = o_s + n_s, \qquad I_s = [o_s, o_{s+1}).$$

- `dimension = n`, `offsets = (o_0,...,o_p)`, `slices = (I_0,...,I_{p-1})`. - `index : (int | str) -> {0,...,p-1}`. - `packet_slice(s) = I_s`. `packet_size(s)=n_s`. - Split and merge:

$$\mathrm{split}_X : \mathbb{K}^{n \times r} \to \prod_s \mathbb{K}^{n_s \times r}, \qquad \mathrm{merge}_X((x_s)_s) = \mathrm{concat}_s\, x_s.$$

- Tensor crossing with $Y = \bigoplus_t Y_t$:

$$X \boxtimes Y = \bigoplus_{s,t} (X_s \otimes Y_t), \qquad \dim(X_s \otimes Y_t) = n_s m_t.$$

- Packet permutation $\pi \in S_p$ induces coordinate index vector

$$P_\pi = (I_{\pi(0)}, I_{\pi(1)}, \dots, I_{\pi(p-1)}).$$

- Failure cards: empty layout, noninteger or nonpositive size, duplicate or empty name, missing names, invalid packet key, invalid permutation, wrong split leading dimension, mixed vector/matrix packets, incompatible RHS counts.

28.2 `PacketIR`: block-typed finite operators

For source layout $X = \bigoplus_s X_s$ and target layout $Y = \bigoplus_t Y_t$, define

$$A_{ts} = Q_t A P_s \in \mathbb{K}^{m_t \times n_s}, \qquad (t,s) \in \Sigma(A).$$

- Address order is always `(target, source)`. - Type:

$$\mathrm{PacketIR}[X,Y] = \{A_{ts} : X_s \to Y_t\}_{(t,s)\in\Sigma(A)}.$$

- Shape: $(\sum_t m_t, \sum_s n_s)$. - Dtype: result type of active block dtypes, or floating default for empty support. - Support: $\Sigma(A) = \{(t,s) : A_{ts} \text{ stored}\}$. - Storage:

$$S(A) = \sum_{(t,s)\in\Sigma(A)} S(A_{ts}).$$

- Batched application:

$$y_t = \sum_{s:(t,s)\in\Sigma(A)} A_{ts} x_s, \qquad y = \mathrm{merge}_Y((y_t)_t).$$

- Materialization:

$$\mathrm{mat}(A)[I_t, I_s] = A_{ts},$$

with addition if a representation ever contributes more than once to a dense region. - `from_dense` threshold:

$$\theta = \tau_{\mathrm{abs}} + \tau_{\mathrm{rel}} \|A\|_2, \qquad (t,s) \text{ retained} \iff \|A[I_t, I_s]\|_2 > \theta,$$

unless `keep_zeros=True`. - Adjoint:

$$(A^*)_{st} = A^*_{ts}, \qquad \Sigma(A^*) = \{(s,t) : (t,s) \in \Sigma(A)\}.$$

- Composition $C = A \circ B$:

$$C_{ts} = \sum_u A_{tu} B_{us}, \qquad \Sigma(C) \subseteq \Sigma(A) \circ \Sigma(B).$$

- Composition pruning is numerical when `atol > 0`:

$$C_{ts} \text{ stored} \iff \|C_{ts}\|_2 > \tau_{\mathrm{compose}}.$$

- Powers by binary exponentiation, with $A^0 = I_X$. - Tensor:

$$(A \otimes B)_{(t,u),(s,v)} = A_{ts} \otimes B_{uv}.$$

- Packet diagonal:

$$D(A) = \bigoplus_s A_{ss},$$

defined only when source and target layouts agree. - Certificate residual:

$$r_{\mathrm{IR}} = \|A(I_n) - \mathrm{mat}(A)\|, \qquad r_{\mathrm{IR}} \le \tau.$$

- Failure cards: out-of-range address, malformed address, block shape mismatch, incompatible composition layouts, non-square power, negative exponent, negative/nonfinite tolerance, incompatible RHS.

28.3 Structural support algebra

- Relation product:

$$L \circ R = \{(t,s) : \exists u\ (t,u) \in L \wedge (u,s) \in R\}.$$

- `support_power` has no ambient `vertex_count`. Its zero power is

$$\mathrm{support_power}(\Sigma, 0) = \{(v,v) : v \text{ occurs in an edge of } \Sigma\}.$$

  Thus empty support has empty zero power. For $k \ge 1$, `support_power` recursively forms $\Sigma^k = \Sigma \circ \Sigma^{k-1}$.

- In contrast, `support_polynomial(..., vertex_count=q)` inserts the full q-vertex identity when the constant coefficient is nonzero, and `PacketIR.power(0)` constructs the full identity on its packet layout.
- Polynomial inclusion for $p(A) = \sum_{k=0}^{d} c_k A^k$:

$$\Sigma(p(A)) \subseteq \bigcup_{k:c_k \neq 0} \Sigma(A)^k.$$

- Transitive closure: $\Sigma^* = I \cup \Sigma \cup \Sigma^2 \cup \cdots$ on a finite vertex set.
- Acyclic unit-diagonal inverse inclusion:

$$(I - N)^{-1} = I + N + \cdots + N^{q-1}, \qquad \Sigma((I - N)^{-1}) \subseteq \Sigma(N)^*.$$

- Schur fill for retained R and hidden H: retain direct $R \to R$ edges and every path $R \to H \to \cdots \to H \to R$.
- `topological_levels` returns executable DAG layers $(L_0, \ldots, L_d)$.
- `longest_path_vertices` returns the maximum vertex count on a directed path.
- `SupportGraph.targets`, `sources`, `reachable`, `is_acyclic`, and `closure` receive separate mini-cards.
- Failure cards: vertex outside range, retained-hidden overlap, noncontiguous partition, cyclic inverse request, cyclic topological request.

28.4 Cost and memory symbols

For dense active block support Σ, batch size r, and scalar size b bytes:

$$F_{\text{packet}} = 2r \sum_{(t,s)\in\Sigma} m_t n_s, \qquad S_{\text{packet}} = \sum_{(t,s)\in\Sigma} m_t n_s.$$

One-pass no-cache-reuse traffic model:

$$B_{\text{stream}} = b \sum_{(t,s)\in\Sigma} (m_t n_s + n_s r + 2 m_t r).$$

Arithmetic intensity and elementary roofline lower bound:

$$I = F/B, \qquad T_{\min} = \max(F/P_{\max}, B/W_{\max}).$$

- Status: counts are **E/M** under the declared dense-block model. The roofline number is **M**, not measured latency. - `CostEstimate` stores flops, scalar storage, traffic, intensity, optional roofline seconds, and metadata. - Failure cards: negative/nonfinite rates, one missing device rate, invalid support, nonpositive item size or RHS count.

28.5 Benchmark contract and observational timing

- Timing scopes: kernel, device-resident, end-to-end.
- Matched contract:

$$\mathfrak{B} = (\text{case signature}, \text{output contract}, \text{scope}, \text{backend}).$$

- Comparison is permitted only if signatures, output contracts, and scopes match. Backend equality is optional by policy.
- Observations:

$$\tilde{t} = \text{median}(t_1, \ldots, t_R), \qquad \text{speedup} = \tilde{t}_{\text{baseline}}/\tilde{t}_{\text{candidate}}.$$

- Synchronization occurs before and after timed device calls when supplied.
- Status: all timing claims are **H/O** and environment-bound.

28.6 Example specifications

EX-28-A: layout roundtrip

- Input: $n = (2, 1, 3)$, names (a, b, c), $X \in \mathbb{R}^{6\times 2}$. Construct: `PacketLayout`, `split`, `merge`, `crossed`, `permutation`. Assert: $\text{merge}(\text{split}(X)) = X$. Negative case: duplicate name or wrong leading dimension.

EX-28-B: IR execution and support cancellation

- Input: two compatible packet layouts and blocks A_{tu}, B_{us}. Compute: $C = A \circ B$, materialized reference $\widehat{C} = \text{mat}(A)\,\text{mat}(B)$. Verify: $\|\text{mat}(C) - \widehat{C}\| \le \tau$. Adversary: choose two paths with opposite products so the structural edge exists but the dense block vanishes.

EX-28-C: modeled versus observed performance

- Build: one sparse packet IR and its dense materialization. Report: $(F, S, B, I, T_{\text{min}})$ separately from benchmark samples. Refuse: compare timing records with different transfer/setup scopes.

---

29. Structured and low-rank operators

29.0 Common structured operator protocol

Every `StructuredOperator` card specifies

$$A : \mathbb{K}^n \to \mathbb{K}^m, \quad A : \mathbb{K}^{n\times r} \to \mathbb{K}^{m\times r},$$

plus `shape`, `dtype`, `apply`, `materialize`, `adjoint`, `storage_entries`, `flop_count`, optional `solve`, and `certificate_residual`. - Default solve is dense and square:

$$x = \text{solve}(\text{mat}(A), b).$$

- The protocol preserves vector-versus-matrix RHS rank. - A cost method is a declared model and must not be described as a benchmark.

29.1 Dense, identity, diagonal, and block diagonal

- `DenseOperator(M)`: $Ax = Mx$, storage mn, modeled flops $2mnr$.
- `IdentityOperator(n,c)`:

$$A = cI_n, \quad S = 1, \quad F = nr, \quad A^{-1} = c^{-1}I_n \ (c \neq 0).$$

- `DiagonalOperator(d)`:

$$Ax = d \odot x, \quad S = n, \quad F = nr, \quad A^{-1}x = d^{-1} \odot x.$$

- `BlockDiagonalOperator(A_0,...,A_{p-1})`:

$$A = \bigoplus_s A_s, \quad Ax = \bigoplus_s A_s x_s, \quad A^{-1} = \bigoplus_s A_s^{-1}.$$

- Failure cards: nonmatrix dense input, zero scalar solve, zero diagonal entry, empty block list, nonsquare block, incompatible RHS.

29.2 Circulant, Toeplitz, Kronecker, and Walsh-Hadamard

- Circulant first column $c \in \mathbb{K}^n$:

$$C_{ij} = c_{(i-j) \bmod n}, \qquad Cx = \mathcal{F}_n^{-1}(\mathcal{F}_n(c) \odot \mathcal{F}_n(x)).$$

- Modeled circulant work: $10n\lceil \log_2 \max(2, n)\rceil r$. Storage n.
- Rectangular Toeplitz $T \in \mathbb{K}^{m\times n}$ is specified by first column and row with common corner.
- `ToeplitzOperator` accepts the common corner through NumPy `allclose`, so constructor acceptance is a numerical corner-consistency guard rather than exact scalar equality.

- Toeplitz application uses a circulant embedding of length $m+n$, with storage $m+n-1$.
- Kronecker factors $L \in \mathbb{K}^{a\times b}$, $R \in \mathbb{K}^{c\times d}$:

$$(L \otimes R)\,\mathrm{vec}_F(X) = \mathrm{vec}_F(RXL^{\mathsf{T}}), \quad X \in \mathbb{K}^{d\times b}.$$

- Walsh-Hadamard dimension $n = 2^q$: butterfly recursion

$$(u, v) \mapsto (u+v, u-v), \qquad H_n^{-1} = n^{-1}H_n,$$

  or $H_n^{-1} = H_n$ in normalized mode.
- Failure cards: empty first column, Toeplitz corner mismatch, nonmatrix Kronecker factor, non-power-of-two Hadamard dimension.

29.3 Low-rank operators and recompression

- `LowRankOperator(U,V)`:

$$A = UV^*, \quad U \in \mathbb{K}^{m\times k}, \quad V \in \mathbb{K}^{n\times k}, \quad \mathrm{rank}(A) \le k.$$

- Apply:

$$Ax = U(V^*x), \quad S = k(m+n), \quad F = 2kr(m+n).$$

- Core recompression input:

$$A = LCR^*, \quad L \in \mathbb{K}^{m\times p}, \quad C \in \mathbb{K}^{p\times q}, \quad R \in \mathbb{K}^{n\times q}.$$

- Reduced QR and small SVD:

$$L = Q_L R_L, \quad R = Q_R R_R, \quad R_L C R_R^* = U\Sigma V^*.$$

- Rank-k factors:

$$\widetilde{U} = Q_L U_k \Sigma_k^{1/2}, \qquad \widetilde{V} = Q_R V_k \Sigma_k^{1/2}.$$

- In exact arithmetic, the Eckart-Young rank-truncation optimum has tail diagnostics

$$\|A - \widetilde{U}\widetilde{V}^*\|_2 = \sigma_{k+1}, \qquad \|A - \widetilde{U}\widetilde{V}^*\|_F = \Big(\sum_{j>k} \sigma_j^2\Big)^{1/2}.$$

- The returned singular values, factors, and residual comparison are NumPy numerical realizations, and the source certificate has `Exactness.NUMERICAL`.
- Numerical certificate checks actual spectral residual against the SVD tail plus scaled tolerance.
- Empty-factor branch returns a valid rank-zero correction.

29.4 Block diagonal plus low rank and Woodbury

Let

$$A = D + UV^*, \qquad D = \bigoplus_s D_s.$$

- Storage model:

$$S = \sum_s n_s^2 + 2nk.$$

- Apply model:

$$F = 2r\sum_s n_s^2 + 4nkr.$$

- Woodbury solve:

$$(D+UV^*)^{-1}b = D^{-1}b - D^{-1}U(I + V^*D^{-1}U)^{-1}V^*D^{-1}b.$$

- Preconditions: each block of D solvable and small middle matrix nonsingular. - Exact product representation is formed before optional recompression. - Repeated local recompression recurrence:

$$E_j \le \|A_j\|E_{j-1} + e_j.$$

- Telescoping product bound for exact A_j and approximate $\widetilde{A}_j$:

$$\Big\|\widetilde{A}_{q-1}\cdots\widetilde{A}_0 - A_{q-1}\cdots A_0\Big\| \le \sum_{j=0}^{q-1}\Big(\prod_{i>j}\|\widetilde{A}_i\|\Big)\|\widetilde{A}_j - A_j\|\Big(\prod_{i<j}\|A_i\|\Big).$$

29.5 Restricted structure discovery

- Packet threshold discovery in a fixed layout:

$$\theta = \tau_a + \tau_r \|A\|_2, \qquad \widehat{\Sigma} = \{(t,s) : \|A_{ts}\|_2 > \theta\}.$$

- Status: **H/N**. A discarded block is not proved algebraically zero.
- Diagnostic: $\|A - \widehat{A}\|_2$.
- BDLR discovery:

$$D = \text{blockdiag}(A), \qquad R = A - D, \qquad R_k = \text{SVD}_k(R), \qquad \widehat{A} = D + R_k.$$

- In exact arithmetic, fixed-layout BDLR discovery inherits fixed-rank SVD optimality for the residual. Returned factors, singular values, and error diagnostics are NumPy numerical realizations. Status is **N**.
- Failure cards: layout mismatch, invalid rank, nonfinite threshold, instability when $\|A_{ts}\| \approx \theta$.

29.6 Example specifications

EX-29-A: structured operator gallery

- Choose one vector x and construct dense, diagonal, circulant, Toeplitz, Kronecker, and Hadamard operators. Compare `apply(x)` with `materialize() @ x`. Tabulate shape, stored entries, modeled flops, and numerical residual separately.

EX-29-B: Woodbury and recompression

- Input: invertible block diagonal D, factors U, V, RHS b. Verify: $\|(D+UV^*)x - b\| \le \tau$. Compress: rank $k \to r < k$. Report σ_{r+1}, Frobenius tail, actual residual. Negative case: singular diagonal block or singular Woodbury middle matrix.

EX-29-C: threshold sensitivity

- Sweep $\theta_- < \|A_{ts}\| < \theta_+$. Display support discontinuity and continuous discarded norm. State explicitly that discovery is heuristic even when its finite residual is certified.

---

30. Krylov, precision, compilation, and scheduling

30.0 Krylov operator protocol

- Matrix-free callable:

$$\mathcal{A} : \mathbb{K}^n \to \mathbb{K}^n.$$

- Callable shape preservation is checked on every generated vector.
- Global Hermitian or positive-definite behavior is not inferred from finitely many directions.

30.1 Krylov bases

For nonzero $b \in \mathbb{K}^n$:

$$\mathcal{K}_k(A, b) = \text{span}\{b, Ab, \dots, A^{k-1}b\}.$$

- Modified Gram-Schmidt constructs $Q_k \in \mathbb{C}^{n \times d}$, $d \le k$. - Breakdown:

$$\|v_j - \sum_{i<j} q_i (q_i^* v_j)\| \le \tau_{\text{krylov}}.$$

- Certificate:

$$r_{\text{orth}} = \|Q_k^* Q_k - I_d\|, \qquad r_{\text{orth}} \le 100 \tau_{\text{krylov}}.$$

- Failure cards: zero start relative to tolerance, steps outside $[1, n]$, changed callable output dimension.

30.2 Preconditioned conjugate gradients

Declared system:

$$Ax = b, \qquad A = A^* > 0, \qquad M = M^* > 0.$$

- Stop threshold:

$$\tau_{\text{stop}} = \max(\tau_{\text{abs}}, \tau_{\text{rel}}\|b\|_2).$$

- Initialization:

$$r_0 = b - Ax_0, \quad z_0 = M^{-1}r_0, \quad p_0 = z_0.$$

- Recurrence:

$$\alpha_k = \frac{r_k^* z_k}{p_k^* A p_k}, \quad x_{k+1} = x_k + \alpha_k p_k, \quad r_{k+1} = r_k - \alpha_k A p_k,$$

$$z_{k+1} = M^{-1}r_{k+1}, \quad \beta_k = \frac{r_{k+1}^* z_{k+1}}{r_k^* z_k}, \quad p_{k+1} = z_{k+1} + \beta_k p_k.$$

- Generated-direction checks: near-real pairings and positive curvature above numerical floor. - Certificate reports true final residual and recursive residual. - Unchecked assumption: global Hermitian positive definiteness. - Failure cards: non-Hermitian generated pairing, negative curvature, nonpositive preconditioner pairing, nonfinite recurrence, iteration limit.

30.3 Lanczos and Ritz data

- Fully reorthogonalized recurrence:

$$Aq_j = \beta_{j-1}q_{j-1} + \alpha_j q_j + \beta_j q_{j+1}.$$

- Projected tridiagonal:

$$T_k = Q_k^* A Q_k = \text{tridiag}(\beta_{j-1}, \alpha_j, \beta_j).$$

- Certificate residuals:

$$r_{\text{orth}} = \|Q_k^* Q_k - I\|, \qquad r_{\text{tri}} = \|Q_k^* A Q_k - T_k\|.$$

- Ritz values are $\lambda(T_k)$. No unqualified global eigenvalue accuracy claim.
- Failure card: materially complex $q_j^* A q_j$ signals non-Hermitian behavior.

30.4 Block Jacobi certificate

Let $D = \text{blockdiag}(A)$ and

$$E = D^{-1/2}(A - D)D^{-1/2}, \qquad \eta = \|E\|_2.$$

If $A = A^*$, every diagonal block is positive definite, and $\eta < 1$, then

$$\kappa(D^{-1/2}AD^{-1/2}) \le \frac{1+\eta}{1-\eta}.$$

- Record: `BlockJacobiCertificate(diagonal, normalized_coupling, eta, condition_bound, certificate).` - `block_jacobi_preconditioner(D)` is the exact packet-block solve callable. - Power perturbation:

$$\|A^k - B^k\| \le k \max(\|A\|, \|B\|)^{k-1}\|A - B\|.$$

- Failure cards: non-Hermitian input, nonpositive packet block, $\eta \ge 1$, shape/layout mismatch.

30.5 Continuous precision allocation

For sensitivities $s_i > 0$, costs $c_i > 0$, and total error budget $T > 0$:

$$\min_{\varepsilon_i > 0} \sum_i c_i \log(1/\varepsilon_i) \quad \text{s.t.} \quad \sum_i s_i \varepsilon_i \le T.$$

- Unclipped solution:

$$\varepsilon_i^* = \frac{T c_i}{s_i \sum_j c_j}.$$

- With maxima $0 < \varepsilon_i \le u_i$:

$$\varepsilon_i(\lambda) = \min\left(u_i, \frac{c_i}{\lambda s_i}\right), \qquad \sum_i s_i \varepsilon_i(\lambda) = T.$$

- Packet perturbation bound:

$$\|\Delta y\| \le \sum_i \|\Delta A_i\| \|x_i\|.$$

- Certificate compares observed error with the bound plus τ.

30.6 Discrete precision and quantization

- Packet i has finite options $d \in \mathcal{D}_i$ with cost c_{id} and integer error units e_{id}.
- Exact finite dynamic program:

$$\min_{d_i \in \mathcal{D}_i} \sum_i c_{i,d_i} \quad \text{s.t.} \quad \sum_i e_{i,d_i} \le B.$$

- Return `None` for infeasible budget, not a false certificate.
- Complex arrays cast to the complex partner of a real target dtype.
- Quantization diagnostic:

$$e_q(x, d) = \| \operatorname{cast}_d(x) - x \|_p.$$

- Failure cards: empty option list, duplicate option names, invalid dtype, negative error units/cost/budget, value/dtype list mismatch.

30.7 Exact tree compilation

At node v: keep pair (c_v, e_v), split overhead s_v, children $\mathrm{ch}(v)$. - Lagrangian recursion:

$$J_v^{\text{keep}} = c_v + \lambda e_v, \qquad J_v^{\text{split}} = s_v + \sum_{u \in \mathrm{ch}(v)} J_u^*, \qquad J_v^* = \min(J_v^{\text{keep}}, J_v^{\text{split}}).$$

- Budget compiler quantizes each selected leaf independently:

$$q_v = \lceil e_v / u \rceil, \qquad \sum_{v \in L} q_v \le B.$$

- Pareto frontier contains nondominated (c, e) plans. Positive tolerance alters tie semantics. - Enumeration cap `max_plans` is a resource guard. - Decision margin:

$$\Delta_v = |J_v^{\text{keep}} - J_v^{\text{split}}|.$$

- Multiple objectives with shared named topology:

$$(c_v, e_v, s_v) = \sum_j w_j (c_{v,j}, e_{v,j}, s_{v,j}).$$

- Failure cards: duplicate node names, topology mismatch, unknown update, negative/nonfinite model value, frontier explosion.

30.8 Scheduling and amortization

- Task DAG $G = (V, E)$, duration $d_v \geq 0$.
- Unlimited-concurrency earliest start:

$$t_v^{\text{start}} = \max_{u \to v} t_u^{\text{finish}}, \qquad t_v^{\text{finish}} = t_v^{\text{start}} + d_v.$$

- Makespan is weighted critical-path length.
- Shape batching ideal launch count:

$$L_{\text{batch}} = |\{\text{shape keys}\}|, \quad L_{\text{no batch}} = |V|.$$

- Setup amortization with setup S, baseline per-use b, candidate per-use c:

$$q_{\min} = \left\lfloor \frac{S}{b - c} \right\rfloor + 1 \quad \text{when } b > c,$$

  otherwise no finite break-even count.
- Status: schedule is **E** for declared durations. Performance meaning is **H/M**.
- Failure cards: unknown dependency, duplicate task, self-dependency, directed cycle, unhashable shape key.

30.9 NumPy and optional CuPy execution

- `NumpyPacketExecutor`: host input, host output, PacketIR semantics.
- `CupyPacketExecutor`: blocks materialized and transferred once at construction.
- Device apply options distinguish `input_on_device` and `return_host`.
- `cupy_available()` means module discoverability only.
- Hardware qualification requires import success, CUDA runtime, device count, synchronized execution, CPU parity, and environment record.
- GPU certificate does not claim synchronized timing unless synchronization is explicitly in the benchmark contract.
- Failure cards: CuPy absent, runtime/device error, unknown backend, wrong RHS shape.

30.10 Example specifications

EX-30-A: block-Jacobi CG

- Construct HPD A, packet layout, D, and η. Solve with and without preconditioning. Report residual histories and bound $(1 + \eta)/(1 - \eta)$. Negative case: perturb a generated direction to negative curvature.

EX-30-B: precision pipeline

- Input: sensitivities s_i, costs c_i, maxima u_i, dtypes $\mathcal{D}_i$, budget B. Compare continuous allocation, clipped allocation, exact discrete assignment, and observed quantization error. Negative case: infeasible integer budget.

EX-30-C: compiler-to-schedule

- Compile a finite packet tree, turn selected leaves into a task DAG, schedule it, group shape keys, and compute setup break-even. Preserve the distinction between exact decisions and modeled durations.

EX-30-D: CPU/GPU matched contract

- Create identical case signatures and output contracts. Execute CPU always. Execute GPU only if runtime-qualified. Verify numerical parity before presenting timing. Negative case: refuse kernel-only versus end-to-end comparison.

---

31. Signal conventions, windows, and framing

31.0 Stable signal conventions

- Signal-first convention: first convolution argument is signal, second is kernel.
- Time axis is the only axis for one-dimensional DSP routines.
- Nonemptiness, dimensionality, and finiteness checks are transcribed per function. The package does not impose one universal finite-entry guard across all signal APIs.
- Spectral, filtering, recursive, FFT-convolution, resampling, adaptive, and streaming paths explicitly reject nonfinite samples. Framing/STFT, direct convolution, several filter-bank paths, polyphase paths, and recovery paths do not uniformly call `isfinite` on every signal entry.
- Real input may produce complex transforms. Inverse routines use `real_if_close` only after computation.
- `same` output always has signal length N, beginning at full-convolution index $\lfloor (K-1)/2 \rfloor$, even when $K > N$.
- Boundary choices are mathematical data, not formatting options.

31.1 Deterministic windows

For $n = 0, \dots, N-1$: - Boxcar: $w_n = 1$. - Hann: $w_n = \frac{1}{2} - \frac{1}{2}\cos(2\pi n/(N-1))$. - Hamming: $w_n = 0.54 - 0.46\cos(2\pi n/(N-1))$. - Blackman:

$$w_n = 0.42 - 0.5\cos\theta_n + 0.08\cos(2\theta_n), \quad \theta_n = 2\pi n/(N-1).$$

- Bartlett, Kaiser $w(\beta)$, and Tukey $w(\alpha)$ receive formula boxes. - Periodic convention constructs the symmetric length $N+1$ window and drops the last sample. - Explicit array windows must already have length N and finite entries.

31.2 Window metrics

For $w \in \mathbb{K}^N$, $W_0 = \sum_n w_n \neq 0$:

$$G_c = |W_0|/N, \qquad G_p = \frac{1}{N}\sum_n |w_n|^2,$$

$$\mathrm{ENBW}_{\mathrm{bins}} = \frac{N\sum_n |w_n|^2}{|W_0|^2}, \qquad \mathrm{ENBW}_{\mathrm{Hz}} = \frac{f_s}{N}\,\mathrm{ENBW}_{\mathrm{bins}}\,.$$

- Failure cards: zero coherent gain for ENBW, nonfinite samples, nonpositive sample rate.

31.3 Overlap profiles and canonical duals

For hop $H \leq N$ and residue $r \in \{0, \dots, H-1\}$:

$$p_r^{(2)} = \sum_{n \equiv r\ (H)} |w_n|^2, \qquad p_r^{(1)} = \sum_{n \equiv r\ (H)} w_n.$$

- NOLA: $\min_r |p_r^{(2)}| > \tau \max(1, \max_r |p_r^{(2)}|)$. - COLA: $\max_r |p_r - p_0| \leq \tau \max(1, \max_r |p_r|)$. - Condition diagnostic:

$$\kappa_{\mathrm{overlap}} = \max_r |p_r| / \min_r |p_r|.$$

- Canonical dual:

$$\widetilde{w}_n = \frac{\overline{w_n}}{p^{(2)}_{n \bmod H}}, \qquad \sum_k w_{n-kH}\widetilde{w}_{n-kH} = 1.$$

- Infinite-lattice certificate leaves finite-record boundary coverage unchecked.

31.4 Deterministic framing

Given signal length N, frame length L, hop $H \le L$:

$$s_m = mH, \qquad F_{m,n} = x^{\text{pad}}_{s_m+n}, \quad 0 \le n < L.$$

- Centering adds $\lfloor L/2 \rfloor$ samples on both sides. - `pad_end=False` retains complete frames only. - `pad_end=True` appends the minimum right padding needed for the last complete frame. - Boundary modes: zeros, reflect, edge, none. - Frame record shapes:

$$F \in \mathbb{K}^{M \times L}, \quad s \in \mathbb{Z}^M, \quad s_m = mH.$$

- Covered length: $(M-1)H + L$. Padded length: $N + p_L + p_R$. - Exact framing certificate checks shape, starts, and coverage inclusion.

31.5 Overlap-add synthesis primitive

For frames F_m and optional synthesis window v:

$$y[n] = \sum_m F_m[n - mH]v[n - mH].$$

- Output length: $(M-1)H + L$. - Optional `length` may crop but not exceed represented range. - This primitive does not normalize by an analysis/synthesis denominator. Normalized STFT inversion is Chapter 32.

31.6 Example specification

EX-31-A: window/framing truth table

- Evaluate seven window families under symmetric and periodic conventions. Tabulate $G_c, G_p, \text{ENBW}, p^{(1)}, p^{(2)}$. Frame one signal under every boundary choice and show exact starts/padding. Negative cases: reflect with one sample, `boundary='none'` when padding is required, hop greater than frame length, zero-overlap dual request.

---

32. Time-frequency and spectral analysis

32.0 STFT typed map

For $x \in \mathbb{K}^N$, frame count M, FFT length $K \ge L$:

$$\text{STFT}_{w,H,K} : \mathbb{K}^N \to \mathbb{C}^{M \times F},$$

where $F = K$ for two-sided output and $F = \lfloor K/2 \rfloor + 1$ for real one-sided output.

32.1 STFT definition and coordinates

$$X[m,k] = \sum_{n=0}^{L-1} x^{\text{pad}}[mH + n]w[n]e^{-2\pi ikn/K}.$$

- Default hop: $H = \max(1, \lfloor L/4 \rfloor)$. - Physical frequencies: $f_k = kf_s/K$ in one-sided order or NumPy signed order. - Frame-center times:

$$t_m = \frac{mH + L/2 - p_L}{f_s}.$$

- `detrend='constant'` subtracts each frame mean before windowing. - One-sided mode rejects complex input. - Metadata is sufficient for normalized inversion.

32.2 Normalized inverse STFT

Let v be the synthesis window and $\widehat{F}_m$ the inverse FFT frame. Define

$$a[n] = \sum_m \widehat{F}_m[n - mH]v[n - mH],$$

$$d[n] = \sum_m w[n - mH]v[n - mH], \qquad \hat{x}[n] = a[n]/d[n].$$

- Validity on the requested unpadded interval requires

$$\min_n |d[n]| > \tau \max(1, \max_n |d[n]|).$$

- Certificate residual:

$$r_{\text{norm}} = \max_n |\hat{x}[n]d[n] - a[n]|.$$

- Source roundtrip is a separate certificate:

$$r_{\text{rt}} = \|\hat{x} - x\|_2 \le \tau \max(1, \|x\|_2).$$

- Failure cards: destructive constant detrending, zero local overlap denominator, requested output beyond original or represented range.

32.3 Spectrogram scalings

For $Z = |X|$: - Magnitude: $S_{\text{mag}} = Z$. - Power: $S_{\text{power}} = Z^2$. - Density:

$$S_{\text{density}} = \frac{Z^2}{f_s \sum_n |w_n|^2}.$$

- Spectrum:

$$S_{\text{spectrum}} = \frac{Z^2}{|\sum_n w_n|^2}.$$

- One-sided power-like scalings double interior bins, excluding DC and even-length Nyquist. - Failure card: spectrum scaling with zero coherent gain.

32.4 Periodogram and Welch PSD

- Modified periodogram:

$$P_{xx}[k] = \frac{|\mathcal{F}_K((x - \text{trend}(x))w)[k]|^2}{D_w},$$

  where $D_w = f_s \sum |w|^2$ for density and $|\sum w|^2$ for spectrum.
- `periodogram` constructs named windows with `get_window(..., periodic=False)`.
- Welch:

$$\widehat{P}_{xx}[k] = \text{Avg}_{m=0}^{M-1} P_{xx}^{(m)}[k],$$

  with mean or median averaging. `welch_psd` constructs named windows with `get_window(..., periodic=True)`.
- `cross_spectrum` also uses `periodic=True`. Its complex median is formed by taking separate medians of real and imaginary parts, with no statistical median-bias correction.
- Frequency resolution:

$$\Delta f = f_s/K.$$

- Integrated density: $\sum_k P[k]\Delta f$. Integrated spectrum: $\sum_k P[k]$.
- Status: finite computations **N**, stochastic consistency and variance claims **H/C**.

32.5 Cross spectra, coherence, and H1 transfer

For paired frame transforms $X_m[k], Y_m[k]$:

$$P_{xx} = \operatorname{Avg} |X_m|^2 / D_w, \quad P_{yy} = \operatorname{Avg} |Y_m|^2 / D_w,$$

$$P_{xy} = \operatorname{Avg}(\overline{X_m} Y_m) / D_w.$$

With ridge $\rho \geq 0$:

$$\gamma^2[k] = \frac{|P_{xy}[k]|^2}{(P_{xx}[k] + \rho)(P_{yy}[k] + \rho)}, \qquad H_1[k] = \frac{P_{xy}[k]}{P_{xx}[k] + \rho}.$$

- Returned coherence is clipped to $[0, 1]$, while the certificate records pre-clipping bound excess. - Unchecked: linear time-invariant data generation and statistical consistency.

32.6 Parseval certificate

Under NumPy DFT normalization:

$$\sum_{n=0}^{N-1} |x_n|^2 = \frac{1}{N} \sum_{k=0}^{N-1} |X_k|^2.$$

- Residual uses scaled tolerance. - Both real and complex signals must be examples.

32.7 Streaming STFT

- Mutable state: bounded buffer, buffer start, next frame start, total accepted samples, closed flag.
- `process(chunk)` emits all newly complete frames.
- `flush(pad_end=True)` emits at most the terminal zero-padded frame and closes.
- Post-flush processing is invalid. Repeated flush emits an empty batch.
- Chunk-invariance theorem:

  $$\operatorname{concat}_j \operatorname{streamSTFT}(x^{(j)}) = \operatorname{STFT}(x), \qquad x = x^{(0)} \| \cdots \| x^{(q-1)},$$

  for matching uncentered, zero-terminal-padding conventions.
- Certificate separately records coefficient and timestamp residuals.

32.8 Example specifications

EX-32-A: invertible chirp STFT

- Input: deterministic chirp x, Hann w, $H = L/4$, centered frames. Compute: STFT, canonical or default dual, inverse, spectrogram. Verify: local denominator, normalized identity, source roundtrip. Negative case: constant detrending followed by inverse request.

EX-32-B: spectral transfer experiment

- Generate $y = h * x + \varepsilon$ with fixed seed. Estimate $P_{xx}, P_{yy}, P_{xy}, \gamma^2, H_1$. Compare estimated H_1 with known FIR response only as a finite diagnostic. Show low-input-power bins and ridge sensitivity.

EX-32-C: adversarial chunks

- Partition the same signal into one-sample, prime-length, empty-process, and final partial chunks. Verify concatenated streaming frames equal batch frames. Negative case: chunk sizes do not sum to signal length.

---

33. Filtering and convolution

33.0 Finite convolution conventions

For signal $x \in \mathbb{K}^N$ and kernel $h \in \mathbb{K}^K$:

$$(h * x)[n] = \sum_{j=0}^{K-1} h[j]x[n-j].$$

- The public mode aliases are distinct:

$$\texttt{ConvolutionMode} = \{\texttt{full}, \texttt{same}, \texttt{valid}, \texttt{circular}\},$$

$$\texttt{FIRMode} = \{\texttt{causal}, \texttt{full}, \texttt{same}, \texttt{valid}\}.$$

- Full, common to both: length $N + K - 1$. - Causal, FIR-only: length N, indices $0{:}N$ of full output. - Same, common to both: length N, start $\lfloor (K-1)/2 \rfloor$. - Valid, common to both: length $\max(N - K + 1, 0)$, start $K - 1$. - Circular, convolution-only: length N, with source index $(n - j)$ mod N. - `convolution_matrix`, `convolution_output_length`, `convolve_signal`, and finite `PolyphaseConvolution` consume `ConvolutionMode`. They do not accept `causal`. - `fir_filter`, `fft_convolve`, `overlap_add_convolve`, and `certified_overlap_add_convolve` consume `FIRMode`. They do not accept `circular`.

33.1 Convolution matrices and finite polyphase blocks

- Matrix type:

$$H_h^{\text{mode}} \in \mathbb{K}^{M \times N}, \qquad y = H_h^{\text{mode}} x.$$

- Residue source and target indices for order q:

$$I_s^X = \{s, s+q, s+2q, \ldots\} \cap [0, N),$$

$$I_t^Y = \{t, t+q, t+2q, \ldots\} \cap [0, M).$$

- Finite transfer blocks:

$$H_{ts} = H_h^{\text{mode}}[I_t^Y, I_s^X].$$

- Coordinate application:

$$y_t = \sum_s H_{ts} x_s.$$

- Certificate compares reconstructed packet output with $H_h x$.
- Infinite causal zero-extended kernels $h[qm + t - s]$ must not be substituted for finite boundary blocks without proving boundary equivalence.

33.2 FIR response and windowed-sinc design

- FIR transfer:

$$H(e^{i\omega}) = \sum_{n=0}^{K-1} h_n e^{-i\omega n}.$$

- Ideal low-pass seed at cutoff f_c:

$$h_n^{\text{ideal}} = 2\frac{f_c}{f_s}\,\text{sinc}\left(2\frac{f_c}{f_s}\left[n - \frac{K-1}{2}\right]\right).$$

- High-pass and band-stop use spectral inversion. Band-pass is a low-pass difference.
- Final taps: $h_n = h_n^{\text{ideal}} w_n$, optionally normalized at the declared reference frequency.
- Odd tap count is required by this implementation's linear-phase construction.
- Group delay: $(K-1)/2$ samples.
- FIR certificate:

$$r_{\text{sym}} = \max_n |h_n - h_{K-1-n}|, \qquad r_{\text{gain}} = \left||H(f_{\text{ref}})| - g_{\text{ref}}\right|.$$

- Failure cards: cutoff outside $(0, f_s/2)$, unordered band, wrong cutoff arity, even tap count, zero normalization gain.

33.3 Streaming FIR

- State contains the most recent $K-1$ input samples.
- `process` emits one causal output for every accepted sample.
- `flush` emits the remaining $K-1$ full-convolution samples exactly once.
- `reset` restores zero history and reopens the stream.
- Chunk-invariance certificate:

$$\operatorname{concat}_j \operatorname{process}(x^{(j)}) = \operatorname{fir_filter}(x, h, \operatorname{causal}).$$

- The wrapper certificate excludes the full tail unless `flush` is included explicitly in the example.

33.4 Recursive FIR/IIR filters

Normalized rational transfer:

$$H(z) = \frac{\sum_{j=0}^{q} b_j z^{-j}}{1 + \sum_{j=1}^{q} a_j z^{-j}}.$$

- Direct-form II transposed state $s \in \mathbb{K}^q$:

$$y_n = b_0 x_n + s_0,$$

$$s_j \leftarrow s_{j+1} + b_{j+1} x_n - a_{j+1} y_n, \quad s_{q-1} \leftarrow b_q x_n - a_q y_n.$$

- Constructor divides numerator and denominator by a_0, then zero-pads to common order. - Zeros and poles are roots of numerator and denominator coefficient polynomials. - Strict BIBO pole certificate:

$$r_{\max} = \max_{\lambda \in \text{poles}} |\lambda|, \qquad 1 - r_{\max} > \tau.$$

- Biquad parameters:

$$\omega_0 = 2\pi f_0 / f_s, \qquad \alpha = \sin \omega_0 / (2Q),$$

with low-pass, high-pass, band-pass, notch, and all-pass numerator tables. - Streaming recursion transports the final state exactly between chunks and certifies batch equality. - Failure cards: $a_0 = 0$, state dimension mismatch, singular response denominator, invalid f_0 or Q, unstable poles.

33.5 FFT and overlap-add convolution

- Zero-padded FFT length:

$$L_{\text{FFT}} = 2^{\lceil \log_2 (N+K-1) \rceil}.$$

- Linear convolution:

$$y = \mathcal{F}^{-1}_{L_{\text{FFT}}} \left(\mathcal{F}(x) \odot \mathcal{F}(h) \right)_{0:N+K-1}.$$

- Overlap-add block length B uses FFT size

$$L_B = 2^{\lceil \log_2 (B+K-1) \rceil}.$$

- Mutable state: precomputed kernel spectrum, untransformed input buffer, $K-1$ overlap samples, complex-mode flag, closed flag.
- Certificate:

$$r_{\text{OLA}} = \|y_{\text{OLA}} - y_{\text{direct}}\|_\infty \le \tau \max(1, \|y_{\text{direct}}\|_\infty).$$

- NumPy FFT is the implemented backend here. GPU claims belong only to separately qualified executor paths.

33.6 Example specifications

EX-33-A: all convolution modes

- Choose $K > N$ intentionally. Print $H_h^{\text{full,same,valid,circular}}$ shapes and outputs. Verify matrix, direct finite, FFT, overlap-add, and polyphase implementations under matching modes. Highlight the stable signal-length `same` convention.

EX-33-B: FIR and IIR streaming

- Design one FIR and one stable biquad. Process identical irregular chunks, retain state, flush where applicable. Verify batch equivalence and plot response metadata later. Negative cases: processing after FIR flush and an unstable IIR denominator.

---

34. Resampling and adaptive filters

34.0 Rational rate normalization

For requested integers $u, d > 0$:

$$g = \gcd(u, d), \qquad U = u/g, \qquad D = d/g.$$

- All plan metadata uses the reduced ratio U/D. - Output length:

$$N_{\text{out}} = \left\lceil \frac{NU}{D} \right\rceil = \left\lfloor \frac{NU + D - 1}{D} \right\rfloor.$$

34.1 Primitive upsampling and downsampling

- Upsampling:

$$(\uparrow_U x)[n] = \begin{cases} x[n/U], & n \equiv 0 \pmod U, \\ 0, & \text{otherwise.} \end{cases}$$

- Downsampling with phase r:

$$(\downarrow_{D,r} x)[m] = x[r + mD], \qquad 0 \le r < D.$$

- `interpolate` is filtered $(U, 1)$. `decimate` is filtered $(1, D)$.

34.2 Centered rational resampling plan

- Default odd tap count: $K = 20 \max(U, D) + 1$.
- Upsampled-grid cutoff:

$$f_c = \frac{1}{2 \max(U, D)} \quad \text{cycles per high-rate sample.}$$

- Windowed sinc taps are scaled to

$$\sum_{j=0}^{K-1} h_j = U.$$

- Center delay: $\delta = (K - 1)/2$.
- Polyphase components: $h^{(r)} = (h_r, h_{r+U}, \ldots)$, $0 \le r < U$.
- Direct plan formula for output m:

$$q_m = \delta + mD, \qquad r_m = q_m \bmod U,$$

$$y_m = \sum_{j \equiv r_m \ (U)} h_j x_{(q_m - j)/U},$$

where out-of-range source indices are omitted by zero extension.

- Explicit reference: zero insertion, full convolution, remove delay, decimate by D.
- Certificate residuals:

$$r_{\text{poly}} = \|y_{\text{plan}} - y_{\text{explicit}}\|_\infty, \qquad r_{\text{DC}} = |\sum_j h_j - U|.$$

- Unchecked: application-specific alias rejection and boundary transient acceptability.
- Failure cards: even tap count, invalid phase, zero DC gain, nonpositive ratio, nonfinite signal.

34.3 Adaptive filtering state model

Given input x_n, desired d_n, and tap count p:

$$u_n = (x_n, x_{n-1}, \dots, x_{n-p+1})^{\mathsf{T}}, \qquad y_n = w_n^* u_n, \qquad e_n = d_n - y_n.$$

- Output shapes:

$$y, e \in \mathbb{K}^N, \quad w_N \in \mathbb{K}^p, \quad W_{\text{hist}} \in \mathbb{K}^{(N+1)\times p}.$$

- Result diagnostics:

$$\text{MSE} = N^{-1} \sum_n |e_n|^2, \qquad \text{tailMSE}_\rho = |I_\rho|^{-1} \sum_{n \in I_\rho} |e_n|^2.$$

34.4 LMS and NLMS

- Leaky LMS:

$$w_{n+1} = (1 - \mu\lambda) w_n + \mu u_n \overline{e_n}, \qquad \mu > 0,\ \lambda \geq 0.$$

- NLMS denominator and update:

$$g_n = \varepsilon + u_n^* u_n, \qquad w_{n+1} = \left(1 - \frac{\mu\lambda}{g_n}\right) w_n + \frac{\mu}{g_n} u_n \overline{e_n},$$

  with $\varepsilon > 0$.
- Finite certificate asserts only finite recursion and parameter ranges.
- Unchecked: persistent excitation, independence model, and convergence for the chosen step.

34.5 RLS

With forgetting $0 < \gamma \leq 1$ and initial inverse covariance $P_0 = \delta I$, $\delta > 0$:

$$k_n = \frac{P_n u_n}{\gamma + u_n^* P_n u_n},$$

$$w_{n+1} = w_n + k_n \overline{e_n}, \qquad P_{n+1} = \gamma^{-1} \left(P_n - k_n u_n^* P_n\right).$$

- Implementation symmetrizes $P_{n+1} \leftarrow (P_{n+1} + P_{n+1}^*)/2$. - Diagnostic records maximum Hermitian residual. - `AdaptiveFilterResult.certificate.theorem_id` is currently `KM-DSP-D12` for LMS, NLMS, and RLS. This is the field actually returned, although function provenance identifiers are respectively `KM-DSP-D12`, `KM-DSP-D13`, and `KM-DSP-D14`. Flag the latter two as a source metadata inconsistency rather than silently rewriting the certificate. - Failure cards: vanished gain denominator, invalid forgetting factor, nonpositive initialization, unequal input/desired length.

34.6 Chunk invariance matrix

The guide should cross-reference these state laws: - FIR: $K - 1$ input-history state. - IIR: direct-form state $s \in \mathbb{K}^q$. - STFT: overlap/unframed sample buffer. - Overlap-add: transform buffer plus convolution overlap. - Rational resampler: currently a finite batch plan, not a mutable streaming API. - Adaptive filters: currently finite batch recurrences with complete weight history, not a chunked state class. This distinction prevents examples from promising streaming APIs that are not implemented.

34.7 Example specifications

EX-34-A: coprime and reducible resampling

- Compare requests $(u, d) = (6, 4)$ and reduced $(3, 2)$. Verify identical plan ratio, output length, polyphase components, explicit reference, and DC gain. Inspect a tone only as a finite alias diagnostic.

EX-34-B: system identification

- Generate fixed-seed input and desired output from known w_*. Run LMS, NLMS, RLS from common zero weights. Report $\|w_N - w_*\|$, MSE, tail MSE, and certificate assumptions. Negative cases: no excitation, excessive step, and forgetting outside $(0, 1]$.

---

35. Filter banks, wavelets, and packet trees

35.0 Periodic analysis/synthesis matrices

For periodic length N divisible by decimation D, channel filters h_c, g_c:

$$A_c = \downarrow_D C_{h_c} \in \mathbb{K}^{(N/D)\times N}, \qquad S_c = C_{g_c} \uparrow_D \in \mathbb{K}^{N\times(N/D)}.$$

- Stacked analysis and synthesis:

$$A = \begin{bmatrix} A_0 \\ \vdots \\ A_{C-1} \end{bmatrix}, \qquad S = \begin{bmatrix} S_0 & \cdots & S_{C-1} \end{bmatrix}.$$

- Reconstruction:

$$\hat{x} = SAx.$$

- Perfect reconstruction certificate:

$$r_{\text{PR}} = \|SA - I_N\|_2 \le \tau.$$

- Frame bounds from singular values of A:

$$\alpha = \sigma_{\min}(A)^2, \qquad \beta = \sigma_{\max}(A)^2.$$

- The source reports the square of NumPy's smallest returned singular value. It is a genuine positive lower frame bound on $\mathbb{K}^N$ only when A has full column rank, hence its row count is at least N. For an undercomplete A, the mathematical lower frame bound is zero even when NumPy's smallest returned nonzero singular value is positive. - `PeriodicFilterBank.frame_bounds` and `FiniteFilterBank.report` do not append the missing zero singular values in the undercomplete case. - Canonical synthesis: $S_{\text{can}} = A^\dagger$. `PeriodicFilterBank.canonical_synthesis_matrix` returns the pseudoinverse without separately enforcing perfect reconstruction.

35.1 General finite filter banks

For analysis $E \in \mathbb{K}^{m\times n}$ and synthesis $R \in \mathbb{K}^{n\times m}$: - `analyze(x)=Ex`. `synthesize(c)=Rc`. Reconstruction RE. - Canonical dual requires $\operatorname{rank}(E) = n$, then $R = E^\dagger$. - Thus `FiniteFilterBank.canonical_dual` enforces full column rank, while the singular-value output in `FiniteFilterBank.report` is only the source's reported diagnostic until that rank condition is established. - Perturbation bound for $\widetilde{E} = E + \Delta E$, $\widetilde{R} = R + \Delta R$:

$$\|\widetilde{R}\widetilde{E} - I\| \le \|RE - I\| + \|\Delta R\|\|E\| + \|R\|\|\Delta E\| + \|\Delta R\|\|\Delta E\|.$$

- Failure cards: incompatible matrix shapes, rank-deficient canonical-dual request, mismatched perturbation shape.

35.2 Haar analysis and synthesis

For even/odd pair (x_{2j}, x_{2j+1}):

$$a_j = \frac{x_{2j} + x_{2j+1}}{c}, \qquad d_j = \frac{x_{2j} - x_{2j+1}}{c},$$

with $c = \sqrt{2}$ normalized and $c = 1$ unnormalized. - Synthesis uses denominator $\sqrt{2}$ normalized or 2 unnormalized. - Normalized energy identity:

$$\|x\|_2^2 = \|a\|_2^2 + \|d\|_2^2.$$

- Failure card: odd signal length or unequal channel shape.

35.3 Periodic lifting

With circular predict P and update U:

$$d = x_o - Px_e, \qquad s = x_e + Ud.$$

- Exact inverse algebra:

$$x_e = s - Ud, \qquad x_o = d + Px_e.$$

- Periodic 5/3 floating scheme uses predict $(1/2, 1/2)$ and update $(1/4, 1/4)$. - Certificate checks finite roundtrip under the periodic boundary convention.

35.4 Haar packet trees

- Path alphabet: $\{0, 1\}$, root ϵ, depth J.
- Node coefficient c_p splits into c_{p0}, c_{p1}.
- Signal length requirement: $2^J \mid N$.
- A pruning L is valid iff it is prefix-free and every full-depth path has exactly one selected ancestor.
- Reconstruction recursively merges selected leaves.
- Normalized pruning energy:

$$\|x\|_2^2 = \sum_{p \in L} \|c_p\|_2^2.$$

- Failure cards: invalid path character, ancestor and descendant both selected, incomplete cover, excessive depth.

35.5 Exact additive best basis

For node cost $C(p, c_p)$:

$$B(p) = \min\left(C(p, c_p), B(p0) + B(p1)\right).$$

- Tie policy: keep node when costs are equal. - Margin:

$$\Delta_p = |C(p, c_p) - B(p0) - B(p1)|.$$

- Entropy cost:

$$C_{\text{ent}}(c) = -\sum_{j : q_j > 0} q_j \log q_j, \qquad q_j = |c_j|^2 / \sum_{\ell} |c_\ell|^2.$$

- Positive-power cost:

$$C_p(c) = \sum_j |c_j|^p, \qquad p > 0.$$

- Optimization is **E** for the supplied finite costs. Floating cost evaluation remains numerical data.

35.6 Multilevel Haar denoising

- Decomposition depth J requires $2^J \mid N$.
- MAD and Gaussian scale estimate:

$$\mathrm{MAD}(d) = \mathrm{median}\,|d - \mathrm{center}(d)|, \qquad \hat{\sigma} = \mathrm{MAD}(d_1)/0.6744897501960817.$$

- Universal threshold:

$$\lambda = \hat{\sigma}\sqrt{2\log n}.$$

- SURE chooses among observed coefficient magnitudes using its finite risk array.
- Shrinkage maps:

$$T_{\mathrm{hard}}(z) = z\mathbf{1}_{|z|\geq\lambda},$$

$$T_{\mathrm{soft}}(z) = z\frac{\max(|z| - \lambda, 0)}{|z|},$$

$$T_{\mathrm{garrote}}(z) = z\frac{\max(|z|^2 - \lambda^2, 0)}{|z|^2},$$

  with zero-safe denominators.
- Finite certificate checks reconstruction finiteness and shrinkage magnitude monotonicity.
- Noise-model optimality remains unchecked.
- Observed risk certificate with known clean reference:

$$\mathrm{MSE}(\hat{x}, x) \leq \mathrm{MSE}(y, x) + \tau.$$

  This is a finite comparison, not a population risk theorem.

35.7 Example specifications

EX-35-A: redundant canonical dual

- Construct tall analysis $E \in \mathbb{R}^{m\times n}$, $m > n$. Compute frame bounds, canonical dual, PR residual, and perturbation bound. Negative case: remove rank and reject the canonical dual.

EX-35-B: packet best basis

- Build depth-J Haar packet tree. Compare full leaves, entropy best basis, and ℓ^p best basis. Verify pruning, reconstruction, energy, decisions, and margins.

EX-35-C: denoising claims ladder

- Fixed seed, clean piecewise-smooth signal, additive noise. Compare threshold rules and modes. Separate: decomposition certificate, shrinkage certificate, observed clean-reference risk, and unproved stochastic generalization.

---

36. Singular packets and recovery

36.0 Singular packet model

For finite components (ξ_j, a_j, r_j), $r_j \in \mathbb{Z}_{\geq 0}$:

$$j(x) = \sum_{j=1}^{J} a_j \frac{(x-\xi_j)_+^{r_j}}{r_j!}.$$

- `TruncatedPowerTerm(location, amplitude, order)` is one component. - `SingularPacket1D` is a finite sum. - At order zero, the implementation uses $\mathbf{1}_{x\geq\xi}$. - `trace_jumps` groups derivative-jump amplitudes by location and order. - `remove(f, x)=f-j(x). restore(g,x)=g+j(x).`

36.1 Location perturbation bounds

- Equal-amplitude steps:

$$\|a\mathbf{1}_{x\geq\xi} - a\mathbf{1}_{x\geq\hat{\xi}}\|_{L^p} = |a|\,|\xi-\hat{\xi}|^{1/p},$$

  and $|a|$ in L^∞ when displacement is nonzero.
- Positive-order mean-value bound on interval $[L, R]$:

$$\|a(\cdot-\xi)_+^r/r! - a(\cdot-\hat{\xi})_+^r/r!\|_{L^p} \leq \frac{|a|S^{r-1}}{(r-1)!}|\xi-\hat{\xi}|(R-L)^{1/p},$$

  with $S = \max(0, R - \min(\xi, \hat{\xi}))$.

36.2 Deterministic separated-jump recovery

Samples:

$$y_i = g(x_i) + j(x_i) + \eta_i, \qquad x_i = x_0 + ih.$$

Declared bounds:

$$\mathrm{Lip}(g) \leq M, \qquad |\eta_i| \leq \varepsilon, \qquad |a_j| \geq a_{\min}.$$

- Adjacent contrast background:

$$B = Mh + 2\varepsilon.$$

- Required separation:

$$a_{\min} > 2B.$$

- Valid threshold interval:

$$B < \theta < a_{\min} - B.$$

- Detect cell i when $|y_{i+1} - y_i| > \theta$. - Report midpoint location $x_i + h/2$, contrast amplitude, location error bound $h/2$, amplitude error bound B. - Scalar and vector forms use absolute value or Euclidean contrast norm. - Conditional assumptions remain unchecked because samples alone do not prove true smoothness, noise bounds, minimum jump, or one jump per cell.

36.2.1 Scalar and scalar/vector API split

- `signals.recovery.JumpRecoveryResult`, exported at package level as `JumpRecoveryResult`, stores `detections`, `threshold`, `background_bound`, `location_error_bound`, `amplitude_error_bound`, and `certificate`. `.cells`, `.locations`, and `.amplitudes` are derived properties.
- `recover_separated_jumps(samples, *, spacing, lipschitz_bound, noise_bound, minimum_jump, threshold=None, start=0.0, minimum_separation=None)` is the scalar API. It raises `AssumptionError` when the finite minimum-jump gap, threshold interval, or declared minimum-separation check fails.
- Its probabilistic wrapper is package-level `recover_jumps_subgaussian(samples, *, spacing, lipschitz_bound, sigma, delta, minimum_jump, threshold=None, start=0.0, minimum_separation=None)`.
- `signals.edge_recovery.JumpRecoveryResult`, exported at package level as `VectorJumpRecoveryResult`, stores `cells`, `locations`, `amplitudes`, `contrasts`, `threshold`, `background_bound`, `location_error_bound`, `amplitude_error_bound`, and `certificate` directly. `.packet()` is defined only when the amplitude array is one-dimensional. Vector amplitudes raise `ValidationError`.

- `signals.edge_recovery.recover_jumps`, exported as `recover_vector_jumps(samples, *, spacing, lipschitz_bound, noise_bound, minimum_jump, origin=0.0, threshold=None)`, accepts scalar or (N, D) sample arrays. A failed finite theorem gap returns a conditional certificate with `valid=False` rather than raising solely for that gap.
- Its probabilistic package alias is `recover_vector_jumps_subgaussian(samples, *, spacing, lipschitz_bound, sigma, minimum_jump, failure_probability, origin=0.0)` and applies to scalar or vector sample matrices.

36.3 Sub-Gaussian recovery

- Scalar simultaneous contrast scale:

$$B_\eta = 2\sigma\sqrt{\log(2N/\delta)}.$$

- Vector coordinate union-bound scale with D coordinates and $N-1$ cells:

$$b_{\text{coord}} = 2\sigma\sqrt{\log(2\max(N-1,1)D/\delta)},$$

$$B_\eta = \sqrt{D}\, b_{\text{coord}}.$$

- Claimed probability is at least $1-\delta$ only under independent centered sub-Gaussian coordinate assumptions.
- Certificate must display the probability model among unchecked assumptions.

36.4 Haar coefficient diagnostics

- Smooth Lipschitz contribution over support length L:

$$|\langle g, \psi_L\rangle| \le ML^{3/2}/4.$$

- Step of amplitude a at relative position $u \in [0,1]$:

$$|\langle a\mathbf{1}_{x\ge\xi}, \psi_L\rangle| = |a|\sqrt{L}\min(u, 1-u).$$

- Half-shift helper compares a point with grid and half-grid boundaries. Its exact implementation returns the larger of the two nearest-boundary distances.
- These formulas motivate localization but do not replace the recovery hypotheses.

36.5 Piecewise-polynomial breakpoint bands

- Forward difference with step s:

$$(\Delta_s x)_n = x_{n+s} - x_n, \qquad \Delta_s^q = \Delta_s \circ \cdots \circ \Delta_s.$$

- Degree-d segments vanish under Δ^{d+1} away from breakpoints.
- Uniform sample noise ε amplifies to

$$B_{d+1} = 2^{d+1}\varepsilon.$$

- Detection threshold must satisfy $\theta \ge B_{d+1}$.
- Output is a band of response indices, not automatically a unique breakpoint location.

36.6 Retained and estimated reconstruction

For exact retained packet j, smooth part $g = f - j$, and approximator Q:

$$\hat{f} = Q(f-j) + j = Qg + j,$$

$$f - \hat{f} = g - Qg.$$

For estimated packet $\hat{j}$ and linear matrix Q:

$$\hat{f} = Q(f-\hat{j}) + \hat{j}, \qquad f - \hat{f} = (I-Q)(f-\hat{j}).$$

If $f = g + j$:

$$f - \hat{f} = (I-Q)g + (I-Q)(j-\hat{j}).$$

- Certificates check these algebraic decompositions numerically. - Supplying only one of `true_smooth` and `true_packet` is invalid.

36.7 Exact region-error formulas

For true scalar value a, estimate b, overlap measure μ_I, true-only μ_T, estimate-only μ_E:

$$\|a\mathbf{1}_T - b\mathbf{1}_E\|_p^p = |a-b|^p\mu_I + |a|^p\mu_T + |b|^p\mu_E.$$

For labeled partitions with overlap matrix $M_{ij} = \mu(T_i \cap E_j)$:

$$\|f - \hat{f}\|_p = \left(\sum_{i,j} M_{ij}|a_i - b_j|^p\right)^{1/p}.$$

- $p = \infty$ takes the largest active mismatch. - Empirical mask/label wrappers compare formula against direct finite-array norm. - Failure cards: negative overlap, incompatible labels, value index outside table, invalid p.

36.8 Failure cases that must remain conditional

- Two jumps inside one cell can cancel an adjacent contrast.
- Adjacent active cells violate the separated-cell hypothesis.
- A wrong Lipschitz or noise bound invalidates theorem interpretation even if code executes.
- Low-amplitude jumps below the contrast gap are not recoverable by this theorem.
- A breakpoint band need not identify an exact physical breakpoint.
- Packet estimation error can dominate smooth approximation error when $(I-Q)(j-\hat{j})$ is large.

36.9 Example specifications

EX-36-A: scalar recovery with full claim ledger

- Construct g with known Lipschitz constant, separated steps, and bounded deterministic noise. Compute B, threshold interval, cells, midpoint locations, amplitudes, and bounds. Verify true locations/amplitudes only because synthetic truth is available. Negative cases: $a_{\min} = 2B$, adjacent steps, two cancelling steps in one cell.

EX-36-B: retained reconstruction identity

- Choose exact j, estimated $\hat{j}$, and projection Q. Display smooth error and packet error terms separately. Verify both reconstruction identities and an exact step-location L^p error.

36A. Approximation, nonlinear packet products, and Smith charts

36A.1 Fourier low-pass approximation

- One-dimensional integer mode mask:

$$\hat{x}_k^{(M)} = \hat{x}_k\mathbf{1}_{|k|\le M}, \qquad x^{(M)} = \mathcal{F}^{-1}(\hat{x}^{(M)}).$$

- Two-dimensional rectangular centered mask:

$$\widehat{X}_{k,\ell}^{(M)} = \widehat{X}_{k,\ell}\mathbf{1}_{|k-k_0|\le M}\mathbf{1}_{|\ell-\ell_0|\le M}.$$

- Status: deterministic finite approximation **N**.
- A discontinuous input example must show Gibbs ringing rather than imply universal edge preservation.

36A.2 Otsu and positive-packet estimation

For histogram class probability $\omega(t)$ and cumulative mean $\mu(t)$:

$$\sigma_B^2(t) = \frac{(\mu_T\omega(t) - \mu(t))^2}{\omega(t)(1-\omega(t))}.$$

- Otsu selects a finite bin center maximizing σ_B^2. - Positive packet mask: $M_i = \mathbf{1}_{x_i > t}$. - Amplitude heuristic: median selected minus median unselected. - Status: **H**. Model assumes one dominant positive contrast. No universal edge theorem. - Counterexamples: negative contrast, multiple regions, heavy gradient, and histogram-bin instability.

36A.3 Coefficient packets and cyclic products

- Residue-supported sequence packets:

$$a_n^{(r)} = a_n \mathbf{1}_{n \equiv r \pmod q}, \qquad a = \sum_{r=0}^{q-1} a^{(r)}.$$

- Product routing:

$$c^{(t)} = \sum_{r+s \equiv t \ (q)} a^{(r)} * b^{(s)}.$$

- Compact channels require carry

$$t = (r+s) \bmod q, \qquad \kappa = \lfloor (r+s)/q \rfloor,$$

  which shifts the coarse convolution by κ.
- Certificate reconstructs the direct Cauchy product.
- Mixed flat/compact representations are rejected.

36A.4 Nonlinear shift filters

- Typed implementation map:

$$F : \mathbb{R}^N \to \mathbb{R}^N.$$

- `nonlinear_shift_filter` coerces the signal with `dtype=float`. Generalized shift indices and coefficients are converted to Python numeric scalars, so this API is real-only rather than a generic $\mathbb{K}$-valued transform.
- Legacy map:

$$y = x + \alpha x \odot R_s x.$$

- Generalized map:

$$y = x + \alpha x \odot \left(\sum_j c_j R_{s_j} x \right),$$

  where R_s is cyclic `roll`.
- Status: finite numerical polynomial transform. No stability theorem is attached.
- `packet_product`, by contrast, supports complex coefficient arrays.
- Failure cards: mismatched or empty shift/coefficient lists, non-one-dimensional input.

36A.5 Smith/Cayley coordinates

- Normalized impedance to reflection coefficient:

$$\Gamma = \frac{Z-1}{Z+1}, \qquad Z \neq -1.$$

- Inverse:

$$Z = \frac{1+\Gamma}{1-\Gamma}, \qquad \Gamma \neq 1.$$

- Cyclic phase $\zeta = e^{2\pi ip/q}$ acts as disk rotation $\Gamma \mapsto \zeta\Gamma$, pulled back to

$$Z \mapsto \frac{(1+\zeta)Z + (1-\zeta)}{(1-\zeta)Z + (1+\zeta)}.$$

- Regularized sensitivity diagnostic:

$$S_\rho(\Gamma) = \frac{1}{|1-\Gamma|^2 + \rho} + \frac{1}{|1+\Gamma|^2 + \rho}, \qquad \rho \geq 0.$$

- The exact exclusions $Z = -1$, $\Gamma = 1$, and a vanishing phase-map denominator describe the rational formulas. The implemented practical domains are narrower because `gamma_from_impedance`, `impedance_from_gamma`, and `phase_mobius` reject denominators that NumPy `isclose` regards as zero.
- `smith_sensitivity` rejects negative regularization, but does not explicitly reject nonfinite regularization. Preserve this source behavior in the failure card rather than implying a stronger finite-value guard.
- Singular and near-singular numerical chart boundaries must be shown explicitly.

36A.6 Example specifications

EX-36A-A: approximation versus retained packets

- Apply Fourier low-pass directly to a step signal. Estimate or supply a singular packet, remove it, low-pass the remainder, and restore it. Compare direct approximation error, retained-packet error, and packet-estimation term without claiming universal superiority.

EX-36A-B: cyclic product and Smith action

- Verify compact packet carry routing against direct polynomial multiplication. Roundtrip several complex impedances through $Z \leftrightarrow \Gamma$. Apply all q cyclic phases and inspect sensitivity near $\Gamma = \pm 1$.

VI.R Exhaustive reference-card and example requirements

VI.R.1 Per-symbol card schema

Every public symbol covered by this expansion receives: 1. Canonical import and API tier. 2. Typed signature with scalar field and array shapes. 3. Definition or recurrence. 4. Preconditions and dimension constraints. 5. Boundary, axis, normalization, and dtype conventions. 6. Exact, numerical, conditional, heuristic, model, optional, or integrity label. 7. Tolerance scaling and certificate residuals. 8. Return record fields, properties, and mutable state where relevant. 9. Failure modes and exact exception family. 10. Backend behavior and optional-dependency isolation. 11. Storage, arithmetic, traffic, or state budget when defined. 12. Minimal success example, adversarial failure example, and related symbols.

VI.R.2 Algorithms module ledger

- `algorithms` package controls: `FEATURE_KEY`, `status()`, and `require()`.
- `algorithms.layout`: `PacketLayout` and every public constructor, property, and transform.
- `algorithms.ir`: `Address`, `PacketIR`, dense import, block access, execution, adjoint, composition, power, tensor, diagonal, certificate.
- `algorithms.structured`: protocol plus dense, identity, diagonal, block-diagonal, circulant, Toeplitz, Kronecker, Walsh-Hadamard, adapter.
- `algorithms.low_rank`: low-rank factors, recompression records/functions, BDLR algebra, Woodbury solve, product and accumulated error bounds.
- `algorithms.support`: relation product/power/polynomial, closure, acyclic inverse, Schur fill, levels, longest path, `SupportGraph`.
- `algorithms.krylov`: basis, CG, Lanczos, block-Jacobi certificate/preconditioner, condition and power bounds.
- `algorithms.precision`: continuous/clipped allocation, discrete options/assignment, quantization, error/certificate.

- `algorithms.compiler`: nodes/plans, two exact compilers, Pareto frontier, combined objectives, margins, immutable update.
- `algorithms.scheduling`: task records, unlimited schedule, shape grouping, launch count, break-even, verification.
- `algorithms.complexity`: every arithmetic, storage, traffic, intensity, roofline, and aggregate estimate function.
- `algorithms.discovery`: support and BDLR discovery result cards and the heuristic-versus-optimal distinction.
- `algorithms.gpu`: availability, execution results, CPU/CuPy executors, synchronization, backend factory.
- `algorithms.benchmarks`: timing scope, contract, result, comparison, measurement, matched comparison.

VI.R.3 Signals module ledger

- `signals` package controls: `FEATURE_KEY` and `status()`. Package aliases and duplicate-name routing follow §§VI.0.5 through VI.0.6.
- `signals.windows`: all window specifications, metrics, profiles, NOLA/COLA certificate, canonical dual.
- `signals.time_frequency`: boundary/detrend/scaling aliases, frame records, framing, overlap-add, STFT/inverse, spectrogram records/functions.
- `signals.streaming`: streaming STFT batch/state/wrapper and all public counters/properties.
- `signals.spectral`: power/cross records, periodogram, Welch, cross spectrum, Parseval.
- `signals.filtering`: FIR aliases, response, designs, application, streaming state/wrapper.
- `signals.recursive`: biquad alias, output/filter/state records, recursion, design, streaming wrapper.
- `signals.convolution`: modes, lengths, matrices, direct application, finite polyphase records/functions.
- `signals.fft_convolution`: FFT convolution, overlap-add state, batch and certified wrappers.
- `signals.resampling`: lengths, primitive samplers, design, plan/result records, rational/interpolation/decimation functions.
- `signals.adaptive`: method alias, result diagnostics, LMS, NLMS, RLS.
- `signals.filterbanks`: periodic/Haar/lifting/general banks, reports, matrices, convenience constructors, perturbation and reconstruction tools.
- `signals.wavelets`: path/cost aliases, complete/pruned tree records, best basis, costs, analysis/synthesis/energy.
- `signals.denoise`: threshold aliases, MAD/noise/threshold functions, decomposition/result records, denoiser, observed risk certificate.
- `signals.recovery`: scalar detections/results, separated/sub-Gaussian recovery, differences, breakpoint records, Haar formulas.
- `signals.edge_recovery`: scalar/vector result, recovery functions, Haar and half-shift helpers.
- `signals.singular_packets`: canonical classes and aliases, reconstruction records/functions, region/overlap errors, differences/noise bound.
- `signals.approximation`: 1-D/2-D Fourier truncation, Otsu, positive-packet record/estimator.
- `signals.nonlinear`: coefficient packets, cyclic product, nonlinear shift filter.
- `signals.smith`: Cayley maps, phase Möbius map, sensitivity.
- `signals.polyphase`: referenced finite coordinate, lattice, kernel, convolution, mixed-radix, and roundtrip cards required by Chapters 33 and 35.

VI.R.4 Required connected tutorials

- `T-VI-1`: dense matrix → layout → PacketIR → support graph → cost estimate.
- `T-VI-2`: structure discovery → BDLR approximation → recompression → Woodbury solve.
- `T-VI-3`: block-Jacobi certificate → CG → precision plan → task schedule.
- `T-VI-4`: window → frame → STFT → inverse → streaming equivalence.
- `T-VI-5`: FIR design → batch/stream/FFT/OLA/polyphase execution → spectral verification.
- `T-VI-6`: rational resampling → adaptive identification → finite diagnostic comparison.
- `T-VI-7`: filter bank → Haar packet tree → best basis → denoising.
- `T-VI-8`: jump recovery → singular packet → retained approximation → exact region-error decomposition.
- `T-VI-9`: negative-example atlas covering every assumption, shape, tolerance, boundary, singularity, unavailable backend, and resource-limit branch above.

VI.R.5 Editorial completion gates

- No timing claim without a benchmark contract and recorded environment.
- No GPU claim from module discovery alone.
- No exact label on FFT, SVD, eigensolve, pseudoinverse, or floating reconstruction.
- No theorem claim from a heuristic threshold or empirical risk comparison.
- No streaming claim for a batch-only API.
- No infinite convolution law substituted for a finite boundary convention.
- No conditional recovery guarantee without displaying unchecked hypotheses.
- Every worked example prints or explains its certificate rather than only asserting `.valid`.
- Every negative example identifies whether failure is a validation error, dimension error, assumption error, backend-unavailable error, zero division, or linear-algebra failure.
- Every prose chapter is cross-linked to the exhaustive API appendix and at least one executable example.

86. Nonlinear systems and dynamics

Specifications for nonlinear calculus and finite dynamics

> These specifications supplement Chapters 37 through 46. Every identity retains its exact, numerical, conditional, heuristic, optional, or integrity-only classification.

VII.0 Common notation and writing contract

Let

$$F : X \to Y, \qquad X \cong \mathbb{K}^{d_X}, \qquad Y \cong \mathbb{K}^{d_Y}, \qquad D^r F(x)[v_1, \dots, v_r] \in Y.$$

Let finite packet resolutions be

$$I_X = \sum_{a \in A} P_a, \qquad I_Y = \sum_{b \in B} Q_b.$$

Claim labels used throughout:

- **E**: exact finite algebra or combinatorics.
- **N**: numerical finite-dimensional identity with tolerance.
- **C**: conditional conclusion with visible unchecked assumptions.
- **H**: heuristic, estimator, or observational profile.
- **O**: optional framework or hardware path.
- **I**: integrity or reproducibility, not authenticity.

Every final API card must distinguish

`value`, `ComputationResult[value]`, `Certificate`, `ConditionalResult[value]`.

Every example has the record $\mathcal{E} = (I, H, A, O, C, F, R)$: input, hypotheses, algorithm, output, certificate, deliberate failure, and reproducibility metadata.

Source modules covered here are all modules under `komorebi_math.nonlinear` and `komorebi_math.dynamics`. Package control cards must also cover each namespace's `FEATURE_KEY` and `status()`.

---

37. Derivative providers and automatic differentiation

37.1 Provider protocol

For every provider $\mathfrak{D}_F$, document (d_X, d_Y, e_F, n_F), where e_F is exactness and n_F is the provider name.

$$\mathfrak{D}_F.\,\text{value}(x) = F(x), \qquad \mathfrak{D}_F.\,\text{derivative}(r, x, V) = D^r F(x)[V].$$

Public protocol symbols: `DirectionalDerivative`, `DerivativeProvider`.

Universal guards:

- $x, v_j \in \mathbb{K}^{d_X}$ and output in $\mathbb{K}^{d_Y}$.
- r is a positive supported integer and exactly r directions are supplied.
- A finite probe never establishes global derivative correctness.

37.2 Declared callable derivatives

Public symbol: `CallableDerivativeProvider`.

$$\mathcal{D} = \{r \mapsto \mathcal{D}_r\}, \qquad \mathcal{D}_r(x; v_1, \dots, v_r) \overset{\text{declared}}{=} D^r F(x)[v_1, \dots, v_r].$$

Finite symmetry probe:

$$\rho_{\text{sym}} = \max_{r, \sigma \in S_r} \|\mathcal{D}_r(x; v_{\sigma(1)}, \dots, v_{\sigma(r)}) - \mathcal{D}_r(x; v_1, \dots, v_r)\|.$$

Optional Taylor probe:

$$T_R(x; h) = F(x) + \sum_{r \in R} \mathcal{D}_r(x; h, \dots, h)/r!, \qquad \rho_T = \|F(x + h) - T_R(x; h)\|.$$

- `[CERT]` Validity is gated only by the selected finite permutation-symmetry residuals. - `[DIAG]` The Taylor residual is recorded when requested but does not change `certificate.valid`. - `[UNCHECKED]` Global validity and the omitted Taylor remainder. - `[FAIL]` Missing order, asymmetric declaration, bad dimensions.

37.3 Polynomial tensors

Public symbol: `PolynomialDerivativeProvider`.

Factorial convention:

$$F(x) = \sum_{m=0}^{M} \frac{1}{m!} T_m[x^m], \qquad T_m \in Y \otimes (X^*)^{\otimes m}.$$

Directional derivative:

$$D^r F(x)[v_1, \dots, v_r] = \sum_{m=r}^{M} \frac{1}{(m-r)!} T_m[v_1, \dots, v_r, x^{m-r}].$$

Symmetry residual:

$$\rho_m = \max_{1 \le j < m} \|T_m - \text{swap}_{j,j+1} T_m\|.$$

- `symmetry_certificate()` passes when exact object tensors have $\rho_m = 0$, or numerical tensors have $\rho_m \le \varepsilon$. Construction validates shapes but does not itself require symmetry. - `[FAIL]` Tensor rank not $m + 1$, unequal source axes, inconsistent target dimension.

37.4 Recursive central differences

Public symbol: `FiniteDifferenceDerivativeProvider`.

$$\eta(v) = h/\max(1, \|v\|), \qquad \Delta_v F(x) = \frac{F(x+\eta v) - F(x-\eta v)}{2\eta}.$$

$$\Delta_{v_1,\dots,v_r} F = \Delta_{v_r}(\Delta_{v_1,\dots,v_{r-1}} F).$$

- **N**: error depends on h, smoothness, scale, roundoff, and order. - `[FAIL]` Nonpositive step, nonfinite output, order above `max_order`. - `[EX]` Compare $h = 10^{-2}, \dots, 10^{-8}$ with an exact polynomial oracle.

37.5 SymPy derivatives

Public symbol: `SympyDerivativeProvider`.

$$\nabla_{v_r} \cdots \nabla_{v_1} F = \sum_{i_1,\dots,i_r} v_{1,i_1} \cdots v_{r,i_r} \frac{\partial^r F}{\partial z_{i_r} \cdots \partial z_{i_1}}.$$

- **E** when expressions, substitutions, and directions remain exact SymPy objects. - `[FAIL]` Empty expressions/variables, repeated variables, bad dimensions.

37.6 Automatic differentiation

Public symbols: `DirectionalJVP`, `AutomaticDifferentiationProvider`, `JaxDerivativeProvider`, `TorchDerivativeProvider`, `autodiff_provider`, `automatic_derivative_provider`, `available_ad_frameworks`, `available_autodiff_backends`, `ad_framework_diagnostics`.

JVP primitive:

$$\mathrm{jvp}(G, x, v) = (G(x), DG(x)[v]).$$

Nested construction:

$$G_0 = F, \qquad G_j(x) = DG_{j-1}(x)[v_j], \qquad G_r(x) = D^r F(x)[v_1, \dots, v_r].$$

Availability states must remain distinct:

$$\text{installed} \Rightarrow \text{?importable} \Rightarrow \text{?JVP-capable} \Rightarrow \text{?function traceable}.$$

- JAX and PyTorch load lazily. Probes run in isolated child interpreters. - No silent numerical replacement after an AD failure. - `[FAIL]` NumPy escape, mutation, unsupported traced control flow, bad device/dtype.

37.7 Comparison example

Use

$$F(x_1, x_2) = (x_1^3 + x_1 x_2,\ x_2^2)$$

through all providers and tabulate $\rho_r^{p,q} = \|D_p^r F(x)[V] - D_q^r F(x)[V]\|$ for $r = 1, 2, 3$. Use an asymmetric wrong second derivative to force an invalid symmetry certificate, and a symmetric wrong derivative to exhibit a large non-gating Taylor diagnostic. Include an optional-backend unavailable record.

---

38. Packet jets and Faà di Bruno composition

38.1 Jet addresses and blocks

Public symbols: `PacketLabel`, `PacketJetAddress`, `PacketJet`, `PacketJetFamily`.

$$\mathcal{A}_r = B \times A^r, \qquad |\mathcal{A}_r| = |B||A|^r, \qquad \alpha = (b; a_1, \dots, a_r).$$

$$J^{(r)}_{b;a_1,\dots,a_r}(x)[v_1, \dots, v_r] = Q_b D^r F(x)[P_{a_1} v_1, \dots, P_{a_r} v_r].$$

Norm bound:

$$\|J^{(r)}_{b;\mathbf{a}}\| \le \|Q_b\| \, \|D^r F(x)\| \prod_{j=1}^{r} \|P_{a_j}\|.$$

Paired permutation symmetry:

$$J_{b;a_{\sigma(1)},\dots,a_{\sigma(r)}}[v_{\sigma(1)}, \dots, v_{\sigma(r)}] = J_{b;a_1,\dots,a_r}[v_1, \dots, v_r].$$

38.2 Reconstruction theorem

$$D^r F(x)[V] = \sum_{b \in B} \sum_{\mathbf{a} \in A^r} J^{(r)}_{b;\mathbf{a}}(x)[V].$$

$$\rho_{\mathrm{rec}} = \left\| D^r F(x)[V] - \sum_{\alpha \in \mathcal{A}_r} J_\alpha(x)[V] \right\|.$$

- `[HYP]` Complete source and target packets and a multilinear derivative. - `[FAIL]` Provider dimensions disagree or packet backends differ.

38.3 Support discovery

Public symbols: `PacketJetSupport`, `packet_jet_support`.

For packet-range bases $\mathcal{B}_a$, classify a block by exhaustive tests

$$J_\alpha[e_1, \dots, e_r] = 0 \quad \forall (e_1, \dots, e_r) \in \mathcal{B}_{a_1} \times \cdots \times \mathcal{B}_{a_r}.$$

$$\mathcal{A}_r = \mathcal{A}_r^{\neq 0} \sqcup \mathcal{A}_r^{0} \sqcup \mathcal{A}_r^{?}.$$

- Exact exhaustive scan: **E**. Tolerance scan: **N**. - Exhausted `max_evaluations`: preserve explicit `untested` addresses and mark **C**. - Untested addresses never justify pruning.

38.4 Packet Taylor polynomial

Public symbols: `PacketTaylorResult`, `packet_taylor`.

$$T_n F(x; h) = F(x) + \sum_{r=1}^{n} \frac{1}{r!} \sum_{\alpha \in \mathcal{A}_r} J_\alpha(x)[h^r].$$

$$\rho_n(x, h) = \|F(x + h) - T_n F(x; h)\|.$$

- `[CERT]` Reconstruction of every retained derivative order. - `[DIAG]` `certificate.valid` does not depend on the observed truncation residual. - `[UNCHECKED]` No automatic omitted-remainder bound.

38.5 Faà di Bruno formula

Public symbols: `Partition`, `set_partitions`, `stirling_second`, `labeled_composition_term_count`, `FaaDiBrunoTerm`, `FaaDiBrunoResult`, `faa_di_bruno`, `packet_faa_di_bruno`, `ComposedDerivativeProvider`.

For $H = F \circ G$,

$$D^n H(x)[V] = \sum_{\pi \in \Pi_n} D^{|\pi|} F(G(x)) \left[D^{|B|} G(x)[v_i]_{i \in B} \right]_{B \in \pi}.$$

Counts:

$$|\Pi_n| = \sum_{k=0}^{n} S(n,k), \qquad N_{\text{labeled}}(n,m) = \sum_{k=1}^{n} S(n,k) m^k.$$

Packet decoration for output c, inputs a_i, and intermediate label map $\lambda : \pi \to M$:

$$Q_c D^n (F \circ G)(x)[P_{a_1} v_1, \dots, P_{a_n} v_n]$$

$$= \sum_{\pi \in \Pi_n} \sum_{\lambda : \pi \to M} J^F_{c;(\lambda(B))_{B \in \pi}} \left[J^G_{\lambda(B);(a_i)_{i \in B}} [v_i]_{i \in B} \right]_{B \in \pi}.$$

Pruning rules:

- Prune when the outer jet has certified-zero support.
- Prune when any required inner jet has certified-zero support.
- Incomplete support cannot prove a zero.
- Reject a raw term count above `max_terms` before support pruning. Support cannot rescue an already over-budget raw count.
- Without `direct_provider`, a valid classical Faà di Bruno certificate records finite partition assembly, not an independent derivative comparison.

38.6 Sparse symmetric tensors

Public symbols: `SparseTensorEntry`, `SparseSymmetricTensor`, `SparsePolynomialDerivativeProvider`.

Store one coefficient per nondecreasing orbit $i_1 \le \cdots \le i_r$. Contract via distinct permutations:

$$T[v_1, \dots, v_r]_o = \sum_{(o,\mathbf{i},c)} c \sum_{\sigma \in \mathrm{Orb}(\mathbf{i})} \prod_{j=1}^{r} (v_j)_{i_{\sigma(j)}}.$$

$$N_{\text{dense}} = d_Y d_X^r, \qquad \rho_{\text{storage}} = N_{\text{stored}} / N_{\text{dense}}.$$

- `[CERT]` Sparse/dense contraction and provider parity. - `[FAIL]` Decreasing orbit, duplicate orbit, asymmetric dense source.

38.7 Finite snapshots

Public symbols: `PacketJetCoefficient`, `PacketJetSnapshot`, `snapshot_packet_jet`.

$$c_{o;i_1,\dots,i_r} = (J[e_{i_1}, \dots, e_{i_r}])_o,$$

$$J[v_1, \dots, v_r]_o = \sum_{i_1,\dots,i_r} c_{o;i_1,\dots,i_r} \prod_{j=1}^{r} (v_j)_{i_j}.$$

- Exact zero omission is **E**. Tolerance omission is **N** and records the largest omitted norm. - Digest identity is **I**, not authorship.

38.8 Example ladder

- Reproduce `nonlinear_jets_composition.py`. Print order-one through order-four partitions.
- Force partial support and retain `untested` blocks.
- Compare exact dense/sparse `Fraction` tensors.
- Snapshot, portable roundtrip, digest, and tamper rejection.

---

39. Nonlinear trees and graded operations

39.1 Ordered rooted trees

Public symbols: `MultilinearOperation`, `OrderedRootedTree`, `ordered_tree_count`, `ordered_rooted_trees`, `ordered_trees_with_leaves`, `catalan_number`.

$$\tau ::= \bullet \mid B_r(\tau_1, \dots, \tau_r), \qquad r \in \mathcal{R} \subset \{2, 3, \dots\}.$$

Record $h(\tau), L(\tau), I(\tau), V(\tau)$, arity profile, prefix form, and serialization.

$$N_0 = 1, \qquad N_h = 1 + \sum_{r \in \mathcal{R}} N_{h-1}^r,$$

$$C_n = \frac{1}{n+1}\binom{2n}{n}.$$

- `[FAIL]` Arity below two, negative height, or family above `max_trees`.

39.2 Fixed-point tree expansion

Public symbols: `TreeTerm`, `TreeExpansionResult`, `PolynomialFixedPointProblem`.

$$u = s + \sum_{r \in \mathcal{R}} B_r(u^r) =: \Phi(u), \qquad u_0 = 0, \quad u_{m+1} = \Phi(u_m).$$

$$\operatorname{val}(\bullet) = s, \quad \operatorname{val}(B_r(\tau_1, \dots, \tau_r)) = B_r(\operatorname{val}\tau_1, \dots, \operatorname{val}\tau_r).$$

Finite identity:

$$u_m = \sum_{h(\tau) < m} \operatorname{val}(\tau).$$

- `[CERT]` Tree sum equals matching Picard iterate. - `[UNCHECKED]` Convergence as $m \to \infty$.

39.3 Majorant convergence

Public symbols: `MajorantConvergence`, `majorant_convergence`, `quadratic_catalan_majorant`.

Assume $\|B_r(x_1, \dots, x_r)\| \le b_r \prod_j \|x_j\|$, define

$$\phi(t) = a + \sum_r b_r t^r, \qquad a \ge \|s\|.$$

Certificate conditions and tail:

$$\phi(R) \le R, \qquad q = \sum_r r b_r R^{r-1} < 1, \qquad \|u - u_m\| \le \frac{q^m a}{1-q}.$$

Quadratic small root:

$$4ab < 1, \qquad R = \frac{1 - \sqrt{1 - 4ab}}{2b}.$$

- **C**: operation bounds and completeness are caller hypotheses. - `[FAIL]` $4ab \ge 1$, $\phi(R) > R$, or $q \ge 1$.

39.4 Packet-labeled trees

Public symbols: `PacketTreeResult`, `packet_tree_reconstruction`.

Insert $I = \sum_a P_a$ on every edge:

$$\mathrm{val}(\tau) = \sum_{\text{packet labelings } \ell} \mathrm{val}(\tau, \ell).$$

- Only certified-zero jet blocks prune labelings. - Record evaluated/pruned counts and direct reconstruction residual.

39.5 Finite abelian gradings

Public symbols: `FiniteAbelianGroup`, `GradingCertificate`, `GradedPacketOperation`, `GradedConvolutionTerm`, `GradedConvolutionResult`. Module-visible callable aliases: `GradedOperationCallable`, `LinearOperation`, `BracketOperation`.

$$X = \bigoplus_{g \in \Lambda} X_g, \qquad \ell_r(X_{g_1}, \dots, X_{g_r}) \subseteq X_{g_1 + \cdots + g_r}.$$

$$(\ell_r \star x)_g = \sum_{g_1 + \cdots + g_r = g} \ell_r(x_{g_1}, \dots, x_{g_r}).$$

39.6 Maurer-Cartan packets

Public symbols: `MaurerCartanResult`, `maurer_cartan_packets`, `HigherMaurerCartanResult`, `higher_maurer_cartan_packets`, `DividedSquareResult`, `divided_square_packets`.

$$\mathrm{MC}(\eta) = d\eta + \tfrac{1}{2}[\eta, \eta], \qquad \mathrm{MC}_g = d\eta_g + \tfrac{1}{2} \sum_{a+b=g} [\eta_a, \eta_b].$$

$$\mathrm{MC}^{(R)}(\eta) = \sum_{r=1}^{R} \ell_r(\eta^r)/r!.$$

Characteristic-sensitive alternative:

$$d\eta + q(\eta), \qquad q(x+y) = q(x) + q(y) + [x, y].$$

- `[CERT]` Finite packet reconstruction. - `[UNCHECKED]` Declared multilinearity, grading, and any infinite tail. - `[EX]` Reproduce `nonlinear_trees_elimination.py`. Add Catalan counts, majorant failure, packet pruning, cyclic grading, and characteristic two.

---

40. Defects, obstruction, lifting, and holonomy

40.1 Nonlinear coordinates and refinement defect

Public symbols: `NonlinearPacketMap`, `NonlinearRefinementDefect`.

Packet conjugation:

$$\widehat{F} = A_Y \circ F \circ S_X.$$

`NonlinearPacketMap.fixed_point_residual()` certifies that the residual was computed. `certificate.valid=True` does not assert that the residual vanishes or that the input is fixed.

Refinement defect:

$$\delta_{C_X, C_Y}(F)(x) = C_Y F_f(x) - F_c(C_X x).$$

For the implemented orbit and fixed-point bounds, declare endomaps and a common coarsening $C_X = C_Y =: C$, and abbreviate $\delta_C := \delta_{C,C}$.

If F_c is L-Lipschitz and $\varepsilon_j = \|\delta_C(F)(F_f^j(x))\|$,

$$\|CF_f^n(x) - F_c^n(Cx)\| \le \sum_{j=0}^{n-1} L^{n-1-j}\varepsilon_j.$$

If x_f is fine-fixed and F_c contracts by $q < 1$,

$$\|Cx_f - x_c\| \le \|\delta_C(F)(x_f)\|/(1-q).$$

- **C**: Lipschitz/contraction constants are declared. - `[FAIL]` $q \ge 1$ or supplied fine point is not fixed.

40.2 Linear obstruction

Public symbols: `ObstructionResult`, `LinearObstructionProblem`, `ObstructionSquare`.

$$Du = -\delta, \qquad \mathrm{ob}(\delta) = \pi_{\mathrm{coker}\, D}(\delta).$$

$$\mathrm{ob}(\delta) = 0 \iff \delta \in \mathrm{im}\, D, \qquad u = u_0 + \ker D.$$

For a commuting correction square,

$$C_Y D_X = D_Y C_X, \qquad [\delta] \mapsto [C_Y \delta], \qquad \rho_\square = \|C_Y D_X - D_Y C_X\|.$$

- Exact quotient arithmetic where supported. Numerical rank/residual otherwise. - `require_correction()` rejects a nonzero obstruction.

40.3 Exact linear lifting

Public symbols: `ModularLinearSolution`, `solve_mod_prime`, `LiftStepResult`, `LinearLiftProblem`.

$$Au = b \pmod p, \qquad u = u_0 + \sum_{j=1}^{\nu} c_j k_j, \qquad N_{\mathrm{sol}} = p^\nu.$$

For $Ax_k = b \pmod{p^k}$, let

$$x_{k+1} = x_k + p^k u, \qquad r_k = (Ax_k - b)/p^k \pmod p, \qquad Au = -r_k \pmod p.$$

40.4 Polynomial lifting and finite trees

Public symbols: `PolynomialLiftProblem`, `LiftTreeNode`, `FiniteLiftTree`, `build_lift_tree`.

$$F(x_k + p^k u) \equiv F(x_k) + p^k J_F(x_k) u \pmod{p^{k+1}},$$

$$J_F(x_k) u = -F(x_k)/p^k \pmod p.$$

- Exhaustively validate the Jacobian law over all p^{d_X} residual corrections: **E**. - Above `linearization_limit`, require explicit trust and mark **C**. - Verify every child directly modulo p^{k+1}. - `max_nodes` and enumeration limits bound branch explosion. - A finite nonempty tree does not prove an infinite p-adic branch.

40.5 Finite graph holonomy

Public symbols: `TransportEdge`, `FundamentalHolonomy`, `IntertwiningResult`, `TransportGraph`.

For edge $e : v \to w$, let $T_e : X_v \to X_w$ be invertible. Choose root r and tree transports $S_v : X_r \to X_v$. For each non-tree edge,

$$H_e = S_w^{-1} T_e S_v.$$

Gluing criterion and gauge action:

$$H_e = I \ \forall e \notin T, \qquad T_e \mapsto G_w T_e G_v^{-1}, \qquad \beta_1 = |E| - |V| + 1.$$

For two systems and root intertwiner R,

$$R_v = S_v' R S_v^{-1}, \qquad R_w T_e = T_e' R_v, \qquad R H_e = H_e' R.$$

40.6 Example ladder

- Reproduce `obstruction_lifting.py`: solvable, obstructed, unique, branching, and terminating cases.
- Force the conditional trusted-Jacobian path.
- Reproduce `holonomy.py`. Show gauge invariance and a broken cycle.

---

41. Correction and elimination

41.1 Hilbert obstruction projector

Public symbols: `HilbertObstructionResult`, `HilbertObstructionProjector`.

For SVD-resolved $D : X \to Y$,

$$\Pi_{\text{im}} = U_r U_r^*, \qquad \Pi_{\text{coker}} = I - \Pi_{\text{im}}, \qquad o = \Pi_{\text{coker}}\delta.$$

$$\|o\| = \operatorname{dist}(-\delta, \operatorname{im} D), \qquad u_* = -D^+\delta \quad (\|o\| \le \varepsilon).$$

- `[CERT]` Orthogonal decomposition, distance identity, least-norm correction residual. - `certificate.valid` can be true while `vanishes=False`. Solvability is represented by `vanishes`, not certificate validity alone. - `[UNCHECKED]` Rank stability below the singular-value threshold.

41.2 Right-inverse correction

Public symbols: `RightInverseCorrectionResult`, `RightInverseCorrectionProblem`.

Solve $Dx + N(x) = y$ by

$$\Phi(x) = R(y - N(x)), \qquad x_{m+1} = \Phi(x_m), \qquad x_0 = 0.$$

Hypotheses:

$$DR = I, \qquad q = \|R\| L_N < 1, \qquad \|R(y - N(0))\| \le (1 - q) R_0.$$

Stability:

$$\|x(y) - x(\tilde{y})\| \le \frac{\|R\|}{1 - q} \|y - \tilde{y}\|.$$

- **C**: declared Lipschitz bound throughout the ball. - `[FAIL]` $q \ge 1$, failed ball invariance, iterate exits ball, iteration cap.

41.3 Lyapunov-Schmidt reduction

Public symbols: `LyapunovSchmidtDecomposition`, `LyapunovSchmidtPoint`, `LyapunovSchmidtReduction`.

At $F(x_0) = 0$, $L = DF(x_0)$, resolve

$$X = \ker L \oplus X_1, \qquad Y = \operatorname{im} L \oplus C.$$

For $x = x_0 + K\kappa + X_1\xi$, solve

$$\Pi_{\text{im}} F(x_0 + K\kappa + X_1\xi) = 0$$

for $\xi = \psi(\kappa)$, then

$$\kappa_{\text{red}}(\kappa) = \Pi_C F(x_0 + K\kappa + X_1\psi(\kappa)).$$

$$D\psi(0) = 0, \qquad D\kappa_{\text{red}}(0) = 0.$$

- SVD resolves spaces. Damped Newton solves the range equation. - **C**: local rank constancy and branch uniqueness. - `[FAIL]` Base is not a root or range Jacobian becomes singular.

41.4 Nonlinear Schur elimination

Public symbols: `SchurDerivativeResult`, `SchurSecondDerivativeResult`, `HiddenSolveResult`, `NonlinearSchurElimination`.

Split $F = (F_P, F_Q)$ on retained u and hidden v. Assume

$$F_Q(u_0, v_0) = 0, \qquad D_v F_Q(u_0, v_0) \text{ invertible.}$$

$$Dv = -(D_v F_Q)^{-1} D_u F_Q, \qquad F_{\mathrm{red}}(u) = F_P(u, v(u)).$$

$$DF_{\mathrm{red}} = D_u F_P + D_v F_P Dv.$$

For directions h, k,

$$D^2 v[h,k] = -(D_v F_Q)^{-1} D^2 F_Q[(h, Dv[h]), (k, Dv[k])],$$

$$D^2 F_{\mathrm{red}}[h,k] = D^2 F_P[(h, Dv[h]), (k, Dv[k])] + D_v F_P D^2 v[h,k].$$

- `[CERT]` Hidden residual and first/second implicit derivatives. - **C**: local invertibility and convergence. - `[EX]` Extend `nonlinear_trees_elimination.py` with contraction failures and finite-difference derivative checks.

---

42. Filtered lifting and finite spectral sequences

42.1 Finite decreasing filtration

Public symbol: `FilteredVectorSpace`.

$$V = F^0 V \supseteq F^1 V \supseteq \cdots \supseteq F^N V = 0, \qquad \mathrm{gr}^k V = F^k V / F^{k+1} V.$$

For coordinate weights $w_i \geq 0$,

$$F^k V = \mathrm{span}\{e_i : w_i \geq k\}.$$

Document $\dim F^k V$, $\dim \mathrm{gr}^k V$, graded coordinates, lifts, and certificate.

- **E**: RREF, subspaces, quotients, and coordinates over $\mathbf{F}_p$.
- `[FAIL]` Nonprime modulus, noncontiguous/nonnested levels, nonzero terminal level.

42.2 One-depth correction

Public symbols: `DepthObstructionResult`, `FilteredCorrectionProblem`.

For filtration-preserving $D : V \to W$ and $\delta \in F^k W$, solve

$$\mathrm{gr}^k D([u]) = -[\delta], \qquad \delta + Du \in F^{k+1} W.$$

- `[CERT]` Preservation at $k, k+1$, graded equation, and every lifted child. - `[FAIL]` Defect at wrong depth or nonpreserving map.

42.3 Finite compatible branches

Public symbols: `FiniteFilteredBranchResult`, `verify_finite_filtered_branch`.

$$F(x_k) \in F^k W, \qquad x_{k+1} - x_k \in F^k V.$$

If terminal depth N is reached, $F^N W = 0$ forces $F(x_N) = 0$.

- Prefix ending before N: continuation remains unchecked.
- Terminal branch with nonzero defect fails.

42.4 Conditional inverse limit

Public symbols: `ProjectiveLimitRecord`, `conditional_projective_limit_completion`.

Require external declarations

$$\bigcap_k F^k V = 0, \qquad V \cong \varprojlim V/F^k V, \qquad F \text{ continuous}, \qquad (x_k)_k \text{ compatible and infinite.}$$

Only then conditionally conclude

$$x = \lim_k x_k, \qquad F(x) = 0.$$

- **C**: runtime records but does not establish infinite/topological assumptions. - Never promote a finite prefix to infinite existence.

42.5 Filtered cochain complex

Public symbols: `SpectralSequenceEntry`, `SpectralSequencePage`, `FilteredSpectralSequenceResult`, `FiniteFilteredCochainComplex`.

$$\cdots \xrightarrow{d^{n-1}} C^n \xrightarrow{d^n} C^{n+1} \to \cdots, \qquad d^{n+1}d^n = 0, \qquad d(F^pC^n) \subseteq F^pC^{n+1}.$$

$$Z_r^{p,n} = \{x \in F^pC^n : dx \in F^{p+r}C^{n+1}\}, \qquad B_r^{p,n} = F^pC^n \cap d(F^{p-r}C^{n-1}).$$

$$E_0^{p,q} = F^pC^{p+q}/F^{p+1}C^{p+q}.$$

For $r \geq 1$, use the implemented finite subquotient

$$E_r^{p,q} = Z_r^{p,p+q}/(Z_{r-1}^{p+1,p+q} + B_{r-1}^{p,p+q}).$$

$$d_r : E_r^{p,q} \to E_r^{p+r,q-r+1}.$$

Page relation:

$$\dim E_{r+1}^{p,q} = \dim E_r^{p,q} - \operatorname{rank} d_r^{p,q} - \operatorname{rank} d_r^{p-r,q+r-1}.$$

Finite terminal convergence:

$$\sum_{p+q=n} \dim E_\infty^{p,q} = \dim H^n(C), \qquad \operatorname{rank} d_\infty = 0.$$

- **E** over $\mathbf{F}_p$. - `[FAIL]` $d^2 \neq 0$, inconsistent dimensions, or failed filtration preservation. - `[EX]` Extend `filtered_transport.py` with a failed depth obstruction and inverse-limit truth table.

---

43. Nonlinear execution, streaming, profiling, and GPU paths

43.1 Projection-cached plan

Public symbols: `PacketJetInputGroup`, `PacketJetExecutionPlan`, `compile_packet_jet_plan`, `PacketJetEvaluation`, `PacketJetBatchResult`, `execute_packet_jet_plan`, `execute_packet_jets`.

$$N_{\text{addr}} = |B||A|^r.$$

Group active addresses by ordered input tuple $\mathbf{a} = (a_1, \dots, a_r)$, evaluate once

$$w_{\mathbf{a}} = D^r F(x)[P_{a_1} v_1, \dots, P_{a_r} v_r],$$

and reuse $J_{b;\mathbf{a}} = Q_b w_{\mathbf{a}}$. Hence

$$N_{\text{calls}} = |\{\mathbf{a} : \exists b \ (b; \mathbf{a}) \text{ active}\}|.$$

Conservative pruning:

$$\mathcal{A}_{\text{pruned}} \subseteq \mathcal{A}_r^0$$

only for certified zeros.

- `[CERT]` Reconstructed batch equals direct derivative.
- `plan.matches(family)` compares only derivative order plus source and target packet names.
- `[FAIL]` Order/name mismatch or address count above `max_addresses`.
- `[UNCHECKED]` Provider identity, base point, projector matrices, tolerances, and the support proof behind pruned addresses are not plan fingerprints.

43.2 Complexity records

Public symbols: `PacketJetComplexityEstimate`, `estimate_packet_jet_complexity`.

Record

$$(N_{\text{full}}, N_{\text{active}}, N_{\text{pruned}}, N_{\text{groups}}, N_{\text{calls}}^{\text{planned}}, N_{\text{calls}}^{\text{naive}}, d_X^r).$$

- Counts are **E** for a finite plan, not asymptotic speed guarantees.

43.3 Snapshot boundary

$$\text{live provider} \xrightarrow{\text{basis capture}} \text{sparse coefficient record}, \qquad N_{\text{basis}} = d_X^r, \quad N_{\text{scalars}} = d_Y d_X^r.$$

- Snapshot stores coefficients, not executable provider semantics. - `[FAIL]` Capture above `max_evaluations` or dense conversion above `max_elements`. - `[CERT]` Snapshot/live parity on selected probes.

43.4 Resumable stream

Public symbols: `PACKET_JET_STREAM_SCHEMA`, `PACKET_JET_STREAM_SCHEMA_VERSION`, `PACKET_JET_CHUNK_SCHEMA`, `PACKET_JET_CHUNK_SCHEMA_VERSION`, `StreamState`, `CaptureMode`, `PacketJetStreamChunkRecord`, `PacketJetStreamManifest`, `PacketJetStreamWriteResult`, `PacketJetStreamSelection`, `StreamedPacketJetEvaluation`, `capture_packet_jet_stream`, `save_packet_jet_snapshot_stream`, `PacketJetSnapshotStreamReader`, `open_packet_jet_snapshot_stream`, `load_packet_jet_snapshot_stream`. Module-visible layout constants: `STREAM_MANIFEST_NAME`, `STREAM_CHUNK_DIRECTORY`, `STREAM_PARTIAL_SUFFIX`.

State machine:

$$\text{absent} \to \texttt{destination.partial} \to \text{verified staging} \to \texttt{destination}.$$

Manifest/chunk records:

$$M = (\text{metadata}, \text{progress}, (h_j)_j, \text{capture certificate}, h_M),$$

$$C_j = (\text{schema}, \text{version}, \text{payload}, \text{SHA256}(\text{payload})).$$

Direct resume matches

$$(\alpha, d_X, d_Y, n_F, e_F, x, \varepsilon, \text{chunk size}).$$

- A live callable has no portable semantic hash. Direct resume assumes unchanged behavior. - Snapshot resume also matches a complete coefficient fingerprint. - Iterators retain at most one decoded chunk at a time. - A selected range $[a, b)$ is $J_{[a,b)}[V] = \sum_{k=a}^{b-1} c_k m_k(V)$, a partial sum unless full. - `[FAIL]` Incomplete-as-complete, corrupt chunk, prefix mismatch, selection/materialization guard.

Integrity boundary:

$$\text{SHA256 consistency} \not\Rightarrow \text{signature, authorship, or trusted provenance.}$$

43.5 Profiling

Public symbols: `RuntimeProfileSample`, `RuntimeStageProfile`, `PacketJetProfileReport`, `profile_operation`, `profile_packet_jet_runtime`, `packet_jet_benchmark_contract`.

`T` is a module-level typing `TypeVar`, not a package-namespace runtime operation.

For each named stage record timings (t_i), minimum, median, maximum, Python allocation peaks, warmups, repetitions, scope, and synchronization policy.

- Timings and allocations are **H** and environment-dependent.
- `tracemalloc` does not measure total process/device memory.
- A benchmark contract does not itself execute or certify speedup.

43.6 Optional CuPy provider execution

Public symbols: `nonlinear_cupy_available`, `CupyDeviceDerivativeProvider`, `CupyPolynomialDerivativeProvider`, `CupyPacketJetExecutor`, plus aliases `CuPyDeviceDerivativeProvider`, `CuPyPolynomialDerivativeProvider`, `CuPyPacketJetExecutor`.

$$\text{host directions} \xrightarrow{\text{optional transfer}} \text{device derivative/projections} \xrightarrow{\text{optional return}} \text{host result}.$$

- `nonlinear_cupy_available()` checks only whether Python can discover a `cupy` module. It does not establish import, runtime, or device availability. - No implicit CPU fallback for a class promising CuPy execution. - Hardware certificate records the actual device/runtime boundary. - `[FAIL]` Installed package but unusable CUDA runtime. Host/device flag mismatch.

43.7 Optional CuPy snapshot execution

Public symbols: `CuPyRuntimeStatus`, `CuPyValidationResult`, `cupy_runtime_status`, `CuPySnapshotExecutor`, `CuPyPacketJetSnapshotExecutor`, `validate_cupy_snapshot`.

$$s_{\mathrm{GPU}} = (\text{installed}, \text{runtime available}, N_{\text{devices}}, \text{runtime version}, \text{error}).$$

$$\rho_{\mathrm{GPU}} = \max_j \|J_{\mathrm{GPU}}[V_j] - J_{\mathrm{CPU}}[V_j]\|.$$

- **O/N**: sampled parity only. - CPU-only systems emit unavailable/skip, never a false hardware pass.

43.8 Example ladder

- Reproduce `scalable_nonlinear.py` and `scalable_nonlinear_runtime.py`. Compare naive/planned calls, reject an order/name mismatch, and demonstrate that same-name semantic staleness is not fingerprinted by `matches`.
- Reproduce `portable_nonlinear_state.py` and `resumable_packet_jet_stream.py`.
- Interrupt, resume, select a partial range, corrupt a chunk, and test the guards.
- Emit CPU/CuPy parity or an explicit no-runtime record.

---

44. Smooth packet transport

44.1 Moving orthogonal packets

Public symbols: `PacketDerivativeResult`, `MovingPacketSystem`.

$$P_a^* = P_a, \qquad P_a^2 = P_a, \qquad P_a P_b = 0 \ (a \neq b), \qquad \sum_a P_a = I.$$

Differentiated identities:

$$\dot{P}_a = \dot{P}_a P_a + P_a \dot{P}_a, \qquad \sum_a \dot{P}_a = 0.$$

Derivative residuals:

$$\rho_{\mathrm{FD}} = \max_a \left\| \dot{P}_a(t) - \frac{P_a(t+h) - P_a(t-h)}{2h} \right\|,$$

$$\rho_{\tan} = \max_a \|\dot{P}_a - \dot{P}_a P_a - P_a \dot{P}_a\|, \qquad \rho_{\mathrm{sum}} = \|\sum_a \dot{P}_a\|.$$

- **N**: sampled evidence, not global differentiability.

44.2 Direct polar rotation

Public symbols: `DirectRotationResult`, `direct_packet_rotation`.

$$S = \sum_a Q_a P_a, \qquad U = S(S^*S)^{-1/2}, \qquad Q_a U = U P_a.$$

Record $\sigma_{\min}(S)$, $\|U^*U - I\|$, and $\max_a \|Q_a U - U P_a\|$.

- `[FAIL]` Singular or unresolved overlap sum.

44.3 Kato generator and transport

Public symbols: `KatoTransport`, `TransportRefinementResult`.

For a complete orthogonal family,

$$K(t) = \sum_a \dot{P}_a P_a = \frac{1}{2} \sum_a [\dot{P}_a, P_a], \qquad K^* = -K, \qquad \dot{P}_a = [K, P_a].$$

$$\dot{U}(t) = K(t)U(t), \qquad U(t_0) = I.$$

Midpoint step:

$$U_{j+1} = \exp(\Delta K(t_j + \Delta/2))U_j.$$

Endpoint checks:

$$\rho_U = \|U_N^* U_N - I\|, \qquad \rho_{\text{int}} = \max_a \|P_a(t_1)U_N - U_N P_a(t_0)\|.$$

Direct-rotation product on $t_0 < \cdots < t_N$:

$$U_{\text{rot}} = U(P_{N-1}, P_N) \cdots U(P_0, P_1).$$

Mesh comparison for refinement factor $q \geq 2$:

$$\rho_{\text{mesh}} = \|U_{qN} - U_N\|.$$

- **N**: a posteriori two-mesh agreement. Asymptotic convergence remains unchecked. - `[EX]` Extend `filtered_transport.py` and `kato_feshbach.py` with mesh sweep, direct rotations, and singular-overlap failure.

---

45. Matrix dynamics, Markov systems, and Bayesian flow

45.1 Matrix and packet semigroups

Public symbols: `matrix_exponential`, `MatrixSemigroup`, `PacketSemigroup`. Implementation appendix symbols: `as_square_matrix`, `hermitian_fractional_power`, `block_partition`. State their validation role and SciPy-or-Taylor exponential boundary. The semigroup identities below are exact finite mathematics. Matrix exponentials, resolvents, and certificates are **N** at runtime.

$$T(t) = e^{tA}, \qquad T(s+t) = T(s)T(t), \qquad R(z; A) = (zI - A)^{-1}.$$

$$T_{ba}(t) = Q_b T(t) P_a, \qquad T_{ca}(s+t) = \sum_b T_{cb}(s) T_{ba}(t).$$

Transfer-graph paths give minimum generator-block transfer order.

45.2 Semigroup refinement

Public symbols: `SemigroupRefinement`, `FiniteSemigroupTower`, `ProjectiveSemigroupTower`, `semigroup_refinement_defect`.

$$E = CA_f - A_cC, \qquad \Delta(t) = Ce^{tA_f} - e^{tA_c}C.$$

$$\Delta(t) = \int_0^t e^{(t-s)A_c} E e^{sA_f}\, ds.$$

$$\|\Delta(t)\| \le te^{t\max(\|A_f\|,\|A_c\|)}\|E\|.$$

$$CR_f(z) - R_c(z)C = R_c(z)ER_f(z).$$

- Duhamel uses Gauss-Legendre quadrature and checks against the direct defect. - `[FAIL]` Negative time or parameter outside a resolvent set.

45.3 Markov convention and lumping

Public symbols: `MarkovKind`, `normalize_partition`, `AggregationMap`, `aggregation_matrix`, `FiniteMarkovChain`, `ContinuousTimeMarkovChain`, `coordinate_lumpability_residual`, `MarkovLumping`.

Column conventions:

$$Q_{ij} \ge 0\ (i \ne j), \quad \mathbf{1}^T Q = 0; \qquad P_{ij} \ge 0, \quad \mathbf{1}^T P = \mathbf{1}^T.$$

$$p(t) = e^{tQ}p_0, \qquad p_n = P^n p_0.$$

Let C aggregate a partition, S be its uniform section, and N_C be the implemented basis matrix for $\ker C$. Kernel obstruction and approximate coarse operator:

$$\mathcal{O} = CAN_C, \qquad \widetilde{A}_c = CAS.$$

Exact lumpability requires $CA = A_cC$ and equal coarse rates within each source block.

$$\Delta(t) = Ce^{tA} - e^{t\widetilde{A}_c}C,$$

$$\|\Delta(t)\| \le te^{t\max(\|A\|,\|\widetilde{A}_c\|)}\|CA - \widetilde{A}_cC\|.$$

- Exact coarse construction rejects a nonlumpable partition. - Structural lumpability is exact mathematics. Source certificates and stationary/evolution calculations are **N** tolerance-based computations. - `[FAIL]` Invalid column sums, negative entries, noninteger discrete step.

45.4 Conditional expectations and martingales

Public symbols: `ConditionalExpectation`, `partition_refines`, `blocks_from_labels`, `MartingaleTower`, `ConditionalExpectationTower`.

$$(E_{\mathcal{P}} f)_i = \frac{\sum_{j\in B(i)} w_j f_j}{\sum_{j\in B(i)} w_j}, \qquad \langle f, g\rangle_w = \sum_i w_i \overline{f}_i g_i.$$

For nested partitions,

$$E_iE_j = E_{\min(i,j)}, \qquad d_0 = E_0 f, \qquad d_j = E_j f - E_{j-1} f.$$

$$\|E_m f\|_w^2 = \sum_{j=0}^{m} \|d_j\|_w^2.$$

- `[FAIL]` Nonpositive weights or nonnested partitions. - Runtime projector, tower, and energy certificates are **N** even though the finite identities are exact.

45.5 Bayesian update and sufficiency

Public symbols: `BayesianUpdate`, `BayesianPartition`, `coarsest_sufficient_partition`, `approximate_likelihood_partitio n`.

$$Z = L^T p, \qquad p'_i = L_i p_i / Z.$$

$$\|p'_L - p'_{\widetilde{L}}\|_1 \le \frac{2\sum_i p_i |L_i - \widetilde{L}_i|}{\max(Z_L, Z_{\widetilde{L}})}.$$

For block B, use the source's zero-mass convention:

$$\bar{p}_B = \sum_{i \in B} p_i, \qquad L_B^{\text{eff}} = \begin{cases} \dfrac{\sum_{i \in B} L_i p_i}{\bar{p}_B}, & \bar{p}_B > 0, \\ 0, & \bar{p}_B = 0. \end{cases}$$

$$\text{agg}(p') = \text{Bayes}(\text{agg}(p), L^{\text{eff}}).$$

Fixed sufficiency requires $L_i = L_j$ within every block for every supplied likelihood.

- `coarsest_sufficient_partition` requires exact profile equality and rejects positive tolerance.
- `approximate_likelihood_partition` uses tolerance-graph components and is **H**, not exact sufficiency.
- Normalization and posterior/coarse-posterior certificates are **N**.
- `[FAIL]` Zero evidence.

45.6 Relative entropy

Public symbols: `relative_entropy`, `relative_entropy_decomposition`.

$$D(p\|q) = \sum_{i : p_i > 0} p_i \log(p_i / q_i).$$

$$D(p\|q) = D(\bar{p}\|\bar{q}) + \sum_B \bar{p}_B D(p(\cdot \mid B)\|q(\cdot \mid B)).$$

Preserve $+\infty$ when $p_i > 0, q_i = 0$.

45.7 Bayesian flow and HMM filtering

Public symbols: `BayesianFlow`, `HiddenMarkovFilter`.

$$p_i(t) = \frac{p_i(0) e^{t s_i}}{\sum_j p_j(0) e^{t s_j}}, \qquad \dot{p}_i = p_i(s_i - \mathbb{E}_p s).$$

$$\frac{d}{dt} \mathbb{E}_{p(t)} s = \text{Var}_{p(t)}\, s.$$

$$\hat{p} = Pp, \qquad p^+ = \text{Bayes}(\hat{p}, L).$$

Exact coarse HMM filtering requires both exact transition lumpability and block-constant likelihood. Otherwise return the directly aggregated fine value with a conditional failed certificate.

45.8 Fractional dynamics

Public symbols: `fractional_power`, `fractional_semigroup`, `fractional_naturality_certificate`, `FractionalDefectResult`, `fractional_defect`.

For Hermitian $A = U\,\text{diag}(\lambda_i) U^* \succeq 0$,

$$A^\alpha = U\,\text{diag}(\lambda_i^\alpha) U^*, \qquad e^{-tA^\alpha} = U\,\text{diag}(e^{-t\lambda_i^\alpha}) U^*.$$

$$CA_f = A_c C \Longrightarrow CA_f^\alpha = A_c^\alpha C.$$

For $0 < \alpha < 1$ and common gap $\gamma > 0$,

$$\|CA_f^\alpha - A_c^\alpha C\| \le \alpha \gamma^{\alpha - 1} \|CA_f - A_c C\|.$$

- The finite spectral identities are exact theorems. Eigendecomposition and residual/bound certificates are **N**. - `[FAIL]` Non-Hermitian, negative spectrum, nonpositive exponent, or missing positive gap.

45.9 Dirichlet Green operator

Public symbol: `DirichletGreen1D`.

$$h = (N+1)^{-1}, \qquad A_h = h^{-2}\,\mathrm{tridiag}(-1,2,-1), \qquad G_h = A_h^{-1}.$$

$$G(x,y) = \min(x,y)(1-\max(x,y)), \qquad G_h = h\,[G(x_i,x_j)]_{i,j}.$$

Compare inverse identity, sampled continuum kernel, discrete eigenvalues, and implemented $O(h^2)$ mode bound. The uniform-grid formula is exact. The floating inverse certificate is **N**.

45.10 Finite Feynman-Kac

Public symbols: `FeynmanKacKind`, `FiniteFeynmanKac`.

$$L_V = Q^T - \mathrm{diag}(V), \qquad K_V(t) = e^{tL_V}.$$

Discrete step:

$$K_V = \mathrm{diag}(e^{-V})P^T.$$

$$\|K_V(t) - K_{\widetilde{V}}(t)\| \le te^{t(\|Q^T\|+\max(\|V\|_\infty,\|\widetilde{V}\|_\infty))}\|V-\widetilde{V}\|_\infty.$$

Exact coarsening requires a lumpable Markov operator and block-constant V. Propagation and perturbation certificates are **N**. A failed coarse model is returned with conditional exactness **C**.

45.11 Example ladder

- Extend `markov_bayes.py` with nonlumpable perturbation, martingale energy, exact/approximate likelihood partitions, KL chain, replicator flow, and HMM.
- Extend `fractional_survival.py` with naturality, defect bounds, and invalid PSD/gap cases.
- Add Green grid convergence and continuous/discrete Feynman-Kac coarsening pass/fail.

---

46. Feshbach reduction, memory, and survival

46.1 Block decomposition and resolvent

Public symbol: `FeshbachReduction`.

$$A = \begin{bmatrix} A_{PP} & A_{PQ} \\ A_{QP} & A_{QQ} \end{bmatrix}.$$

At least one retained and one hidden coordinate are required. `retained_to_hidden` is the historical name for top-right A_{PQ}. `hidden_to_retained` is A_{QP}.

Using the code convention $(zI-A)^{-1}$,

$$\Sigma(z) = A_{PQ}(zI_Q - A_{QQ})^{-1}A_{QP}, \qquad A_{\mathrm{eff}}(z) = A_{PP} + \Sigma(z).$$

$$P(zI-A)^{-1}P = (zI_P - A_{\mathrm{eff}}(z))^{-1}.$$

$$\rho_{\mathrm{Fesh}} = \|(zI_P - A_{\mathrm{eff}})^{-1} - P(zI-A)^{-1}P\|.$$

- `[HYP]` Full and hidden resolvents exist.
- The Schur identity is exact finite algebra. The source certificate is **N**.
- `[FAIL]` Singular hidden or full resolvent.

46.2 Hidden-state memory

Eliminate y from

$$\dot{x} = A_{PP}x + A_{PQ}y, \qquad \dot{y} = A_{QP}x + A_{QQ}y.$$

$$\dot{x}(t) = A_{PP}x(t) + A_{PQ}e^{tA_{QQ}}y(0) + \int_0^t K(t-s)x(s)\,ds,$$

$$K(t) = A_{PQ}e^{tA_{QQ}}A_{QP}, \qquad f_Q(t) = A_{PQ}e^{tA_{QQ}}y(0).$$

46.3 Rational memory realization

Public symbols: `MemoryKernel`, `RationalMemoryKernel`, `RationalMemoryRealization`.

$$K(t) = Be^{tD}C, \qquad \widehat{K}(z) = B(zI - D)^{-1}C.$$

$$\widehat{K}_T(z) = B(zI - D)^{-1}(I - e^{(D-zI)T})C.$$

$$\widehat{K}(z) - \widehat{K}_T(z) = B(zI - D)^{-1}e^{(D-zI)T}C.$$

`MemoryKernel.certificate(..., points=...)` records the requested point count but performs no quadrature and does not use `points` to determine its value or validity.

Auxiliary system:

$$\frac{d}{dt}\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} A & B \\ C & D \end{bmatrix}\begin{bmatrix} x \\ y \end{bmatrix}.$$

Its retained component realizes the Volterra memory equation.

46.4 Finite local closure

Cayley-Hamilton reduces closure to

$$M_j = A_{PQ}A_{QQ}^j A_{QP}, \qquad j = 0, \dots, d_Q - 1.$$

$$M_j = 0\ \forall j < d_Q \iff \text{hidden feedback closes locally;}$$

otherwise it produces memory. The equivalence is exact. `local_closure_certificate` is **N** and accepts when $\max_j \|M_j\|_2 \le \varepsilon$.

46.5 Kernel and trajectory bounds

Public symbols: `kernel_l1_error`, `memory_kernel_error_bound`, `volterra_trajectory_bound`.

$$\delta_K = \int_0^T \|K_1(t) - K_2(t)\|\,dt.$$

Rational kernels combine exponential norm bounds with sampled quadrature plus derivative allowance. Arbitrary callables receive only a conditional trapezoidal estimate.

$$\|x_1 - x_2\|_{[0,T]} \le MT\delta_K \exp((a+\kappa)T).$$

- **C**: M, a, κ are declared trajectory/local/kernel bounds.
- `[FAIL]` Negative/nonfinite data or missing declared bounds.

46.6 Phase-type survival

Public symbol: `PhaseTypeSurvival`.

For stable transient column subgenerator Q, the constructor normalizes its supplied initial vector. Write $p_0 = p_{\text{supplied}}/(\mathbf{1}^T p_{\text{supplied}})$. Then

$$p(t) = e^{tQ}p_0, \qquad S(t) = \mathbf{1}^T p(t), \qquad a = -\mathbf{1}^T Q.$$
$$f(t) = a\,p(t), \qquad h(t) = f(t)/S(t).$$

Model conditions:

$$Q_{ij} \geq 0\ (i \neq j), \qquad a_j = -\sum_i Q_{ij} \geq 0, \qquad \max \operatorname{Re}\sigma(Q) < 0.$$

For cause rates $a^{(c)}$, require $\sum_c a_j^{(c)} = a_j$.

Transform and moments:

$$\mathbb{E}[e^{-sT}] = a(sI - Q)^{-1}p_0,$$
$$\mathbb{E}[T^m] = m!\,\mathbf{1}^T(-Q)^{-m}p_0.$$
$$f_c(t) = a^{(c)}p(t), \qquad \sum_c f_c(t) = f(t), \qquad p_t^{\text{res}} = p(t)/S(t).$$

Identity certificate checks $-S'(t) = f(t)$ by centered difference and the cause sum.

- Model admissibility, transforms, moments, and identity checks are **N**. Stability is required below `-tolerance`, and the centered-difference identity certificate uses a fixed 10^{-5} threshold.
- `[FAIL]` Unstable generator, negative rate, inconsistent causes, or restart at zero survival.

46.7 Example ladder

- Extend `feshbach_memory.py` with safe resolvent sweep, local-closure/nonclosure pair, rational realization, and finite-horizon bounds.
- Extend `kato_feshbach.py` with explicit sign-convention comparison.
- Extend `fractional_survival.py` with competing risks, residual lifetime, transform/moment checks, and invalid models.
- Do not identify a generic memory kernel with survival unless Markov subgenerator hypotheses are independently certified.

VII.A Cross-chapter case studies

VII.A.1 Nonlinear coarse-model pipeline

$$\text{provider} \to \text{packet jets} \to \text{support} \to \text{plan} \to \text{snapshot/stream} \to \delta_C(F) \to \text{obstruction/correction}.$$

Record

$$(\rho_{\text{derivative}}, \rho_{\text{packet}}, N_{\text{active}}, h_{\text{stream}}, \|\delta_C(F)\|, \text{ob}(\delta), \rho_{\text{corrected}}).$$

VII.A.2 Finite-to-infinite boundary

Keep separate

$$F(x_k) = 0 \pmod{p^k} \quad \forall k \leq N \quad \text{(E)},$$

from

$$\exists x \in \varprojlim X/p^k X :\ F(x) = 0 \quad \text{(C; external infinite hypotheses)}.$$

VII.A.3 Dynamic memory pipeline

$$e^{tA} \to \text{packet blocks} \to \text{coarse defect} \to \text{Feshbach split} \to K(t) \to \text{rational realization}.$$

Do not let a small finite residual become an unqualified infinite-dimensional claim.

VII.B Certificate and failure schema

Every final table must include: object, claim class, residual formula, tolerance, checked hypotheses, unchecked hypotheses, failure behavior, and theorem identifier.

Failure families:

- Shape: dimension, tensor rank, packet count, retained/hidden block mismatch.
- Algebra: nonprime modulus, singular Jacobian/resolvent, noncommuting square, nontrivial holonomy, $d^2 \neq 0$.
- Hypothesis: failed contraction, filtration, lumpability, sufficiency, positivity, gap, or survival stability.
- Resource: `max_terms`, `max_trees`, `max_evaluations`, `max_nodes`, dense/stream materialization guards.
- Optional backend: not installed, import/JVP failure, no CUDA runtime/device, CPU/GPU mismatch.
- Integrity: payload/chunk digest mismatch, incomplete stream, tampered portable envelope, digest without authentication.

VII.C Source example crosswalk

Guide locus	Existing executable seed	Symbolic extension
37 through 38	`nonlinear_jets_composition.py`	All providers, orders 1 through 4, partial support, failure probes
38, 43	`scalable_nonlinear.py`	Exact sparse parity, plan counts, snapshot integrity
43	`scalable_nonlinear_runtime.py`	Stage profiles, order/name mismatch, same-name staleness caveat, optional GPU parity
38, 43	`portable_nonlinear_state.py`	Dense guard, multiple probes, tamper taxonomy
43	`resumable_packet_jet_stream.py`	Interrupted resume, partial range, corrupt chunk
39, 41	`nonlinear_trees_elimination.py`	Majorant failure, second derivatives, Newton failures
40	`obstruction_lifting.py`	Solvable/obstructed cases and all lift branch types
40	`holonomy.py`	Intertwiners, exact backend, nontrivial cycle
42, 44	`filtered_transport.py`	Depth obstruction, inverse-limit flags, mesh sweep
44, 46	`kato_feshbach.py`	Rotation product, refinement, resolvent relation
45	`markov_bayes.py`	Nonlumpable case, KL, HMM, martingale tower
45 through 46	`fractional_survival.py`	Defect bound, competing risks, invalid models
46	`feshbach_memory.py`	Closure pair, realization, callable comparison

VII.D Documentation requirements

- Every public class receives a constructor card.
- Every public method/property receives a symbolic contract.
- Compatibility aliases point to canonical symbols.
- Sign, packet, and Markov column conventions are explicit.
- Conditional conclusions list unchecked assumptions.
- Optional backends have deterministic unavailable paths.
- Integrity statements distinguish digest from signature.
- Combinatorial routines expose resource guards.
- Every chapter contains an invalid-certificate case and an exception case.
- Examples print expected values, residuals, and metadata.
- Theorem identifiers are reconciled with the final source manifest.
- PDF rendering preserves matrices, long symbols, links, and cross-references.

87. Integration, expressions, quantum, deformation, and Langlands

Source clarifications for the Part VIII specifications. The reader chapters supply four implementation details: `Exact Polynomial.evaluate` is exact only at exact coordinates. `max_terms` is enforced while sparse expansions grow rather than wholly before expansion. The displayed `ErrorGuarantee` chain is a conceptual evidence hierarchy, not Python enum ordering, and the integration-coordinate FTC uses a square degree-`n` derivative operator, while the tight degree-lowering derivative belongs to the Chapter 51 expression primitive.

Specifications for integration, expressions, quantum, deformation, and Langlands models

These specifications supplement Chapters 47 through 58. Every displayed relation retains its intended evidence class:

- **E**: finite exact arithmetic under checked hypotheses.
- **N**: finite numerical computation with a stated residual and tolerance.
- **C**: conditional result retaining named unchecked hypotheses.
- **H**: heuristic or estimated diagnostic.
- **P**: planned exposition or reserved extension, not a shipped theorem.

The final guide must not remove these tags until the accompanying certificate, hypotheses, and failure behavior have been written beside the formula.

Shared symbolic grammar for Chapters 47 through 58

Typed objects

Use this object-level notation consistently:

$$\begin{aligned}
\mathsf{Poly}_d(\mathbb{Q}) &= \{p : \mathbb{Q}^d \to \mathbb{Q} \text{ with finite support}\},\\
\mathsf{Meas}_d^{+} &= \{\text{implemented finite positive measures of dimension } d\},\\
\mathsf{Quad}_d &= \{\text{finite signed quadrature functionals of dimension } d\},\\
\mathsf{P}_n^B &= \operatorname{span}_{\mathbb{Q}}\{b_0, \dots, b_n\},\\
\mathsf{Lin}(X, Y) &= \operatorname{Hom}_{\mathbb{Q}}(X, Y),\\
\mathsf{M}_d(\mathbb{C}) &= \text{finite complex } d \times d \text{ matrices},\\
\mathsf{F}_p^n &= \text{finite } n\text{-dimensional vector space over } \mathbb{F}_p.
\end{aligned}$$

Certificate record used in the prose

For an operation $F : X \to Y$, display its evidence as

$$\mathrm{Cert}(F, x) = (\tau, \mathrm{id}, H_{\checkmark}, H_{?}, r, \varepsilon, D),$$

where:

- $\tau \in \{\mathrm{E}, \mathrm{N}, \mathrm{C}, \mathrm{H}, \mathrm{P}\}$.
- id is the theorem or operation identifier.
- $H_{\checkmark}$ is the finite list of replayed hypotheses.
- $H_{?}$ is the list of assumptions not established internally.
- r is an exact discrepancy or numerical residual.
- ε is the comparison tolerance when numerical.
- D is deterministic diagnostic data.

Certificate composition uses the public strongest-to-weakest order

$$\mathrm{E} \prec \mathrm{N} \prec \mathrm{C} \prec \mathrm{H} \prec \mathrm{P},$$

and, for $m \geq 1$, obeys

$$H_{?}^{\mathrm{out}} = \bigcup_{i=1}^{m} H_{?}(C_i),$$

$$S_H = \begin{cases} \{\mathrm{C}\}, & H_{?}^{\mathrm{out}} \neq \emptyset, \\ \emptyset, & H_{?}^{\mathrm{out}} = \emptyset, \end{cases} \qquad \tau_{\mathrm{out}} = \max_{\prec}(\{\tau(C_i) : 1 \leq i \leq m\} \cup S_H)\,.$$

The empty composition has status **E**. Thus newly introduced unchecked assumptions impose a conditional floor even when every child is exact. No downstream operation may silently strengthen a child certificate.

Standard symbolic example frame

Every worked example in these chapters should be outlined as

$$\mathcal{E} = (I, H, A, O, C, F, R),$$

with input I, hypotheses H, algorithm A, output O, certificate C, deliberate failure F, and related symbols R.

Part VIII. Exact finite integration and typed expressions

47. Measures and exact polynomials

47.0 Scope boundary

Implemented scope:

$$\text{finite sparse rational polynomials} \quad + \quad \text{explicit finite-dimensional measures.}$$

Excluded from the chapter's claims:

$$\text{general Lebesgue theory}, \quad \text{improper integrals}, \quad \text{distributions},$$
$$\text{infinite product measures}, \quad \text{stochastic integration}.$$

47.1 `ExactPolynomial` as a typed sparse object

Define

$$p(x) = \sum_{\alpha \in S} c_\alpha x^\alpha, \qquad S \subset \mathbb{N}^d \text{ finite}, \qquad c_\alpha \in \mathbb{Q}.$$

Type its public constructors as

$$\begin{aligned} \text{zero}_d &: 1 \to \mathsf{Poly}_d(\mathbb{Q}), \\ \text{constant}_d &: \mathbb{Q} \to \mathsf{Poly}_d(\mathbb{Q}), \\ \text{monomial}_d &: \mathbb{Q} \times \mathbb{N}^d \to \mathsf{Poly}_d(\mathbb{Q}), \\ \text{univariate} &: \mathbb{Q}^{n+1} \to \mathsf{Poly}_1(\mathbb{Q}). \end{aligned}$$

The public monomial argument order is `(coefficient, exponents)`. Record the invariants

$$\dim p = d, \qquad \operatorname{supp}(p) = \{\alpha : c_\alpha \neq 0\}, \qquad \deg p = \max_{\alpha \in \operatorname{supp}(p)} |\alpha|.$$

Public inspection slots:

- `degree` and componentwise `degrees`.
- `term_count`.
- `coefficient(exponent)`.
- Exact `evaluate(point)`.
- `to_univariate_coefficients()`.
- `certificate()` and `to_dict()`.

47.2 Exact polynomial algebra

Type the algebra maps:

$$\begin{aligned} +, - &: \mathsf{Poly}_d \times \mathsf{Poly}_d \to \mathsf{Poly}_d, \\ \text{scale} &: \mathbb{Q} \times \mathsf{Poly}_d \to \mathsf{Poly}_d, \\ \cdot &: \mathsf{Poly}_d \times \mathsf{Poly}_d \to \mathsf{Poly}_d, \\ (\cdot)^k &: \mathsf{Poly}_d \times \mathbb{N} \to \mathsf{Poly}_d, \\ \partial_i &: \mathsf{Poly}_d \to \mathsf{Poly}_d, \\ J_i &: \mathsf{Poly}_d \to \mathsf{Poly}_d. \end{aligned}$$

Specify

$$\partial_i x^\alpha = \alpha_i x^{\alpha - e_i}, \qquad J_i x^\alpha = \frac{x^{\alpha + e_i}}{\alpha_i + 1}.$$

For `integrate_out(i,a,b,density=rho)`, reserve the exact rule

$$\int_a^b c_\alpha x^\alpha \rho \, dx_i = \rho c_\alpha \frac{b^{\alpha_i + 1} - a^{\alpha_i + 1}}{\alpha_i + 1} x^{\alpha - \alpha_i e_i}.$$

The result is a `Fraction` for a one-dimensional source and an `ExactPolynomial` of dimension $d - 1$ otherwise. The prose must show that multiplication, powers, and affine pullback enforce `max_terms` while sparse expansions grow.

47.3 Exact affine pullback

For

$$T : \mathbb{Q}^m \to \mathbb{Q}^d, \qquad T(y) = Ay + b,$$

type

$$T^* : \mathsf{Poly}_d(\mathbb{Q}) \to \mathsf{Poly}_m(\mathbb{Q}), \qquad T^*p = p \circ T.$$

Certificate schema:

$$C_{T^*p} = (\mathrm{E}, \texttt{exact-affine-pullback}, \{A, b \text{ rational}, \text{ shape valid}, \text{ budget met}\}, \emptyset, 0, 0, D).$$

Failure slots:

- incompatible source dimension.
- Negative exponent.
- Nonrational coefficient.
- Expansion beyond `max_terms`.
- Malformed point passed to `evaluate`.

47.4 Positive measure protocol

Type the protocol

$$\mu \in \mathsf{Meas}_d^+ \quad \Longrightarrow \quad \begin{cases} \mathrm{mass}(\mu) \in \mathbb{R}_{\geq 0}, \\ m_\alpha(\mu) = \int x^\alpha \, d\mu, \\ \mathrm{Cert}(\mu). \end{cases}$$

The guide must separate structural positivity from signed numerical functionals.

47.5 Atomic measures

Define

$$\mu = \sum_{j=1}^{N} w_j \delta_{x_j}, \qquad w_j \geq 0, \qquad x_j \in \mathbb{Q}^d \text{ or } \mathbb{R}^d.$$

Then

$$\mu(1) = \sum_j w_j, \qquad m_\alpha(\mu) = \sum_j w_j x_j^\alpha, \qquad \int f \, d\mu = \sum_j w_j f(x_j).$$

Method outline:

- `atom_count`.
- `mass()` and `moment(alpha)`.
- `normalized()` when mass is positive.
- `coalesced()` for equal atoms.
- `certificate()`.

Failure slots: negative weight, inconsistent point dimension, zero-mass normalization, nonfinite numerical data.

47.6 Interval, product, and pushforward measures

For an interval measure,

$$d\mu(x) = \rho \, dx\big|_{[a,b]}, \qquad \rho \geq 0, \qquad a \leq b,$$

derive

$$m_k(\mu) = \rho \frac{b^{k+1} - a^{k+1}}{k+1}.$$

Document `structurally_zero`, `normalized`, and `pushforward_affine`.

For products, let $d_j = \dim \mu_j$ and split $\alpha = (\alpha^{(1)}, \dots, \alpha^{(r)})$ with $\alpha^{(j)} \in \mathbb{N}^{d_j}$. Then

$$\mu = \bigotimes_{j=1}^{r} \mu_j, \qquad m_\alpha(\mu) = \prod_{j=1}^{r} m_{\alpha^{(j)}}(\mu_j).$$

For `ExactAffineMap`, expose

$$T(x) = Ax + b, \qquad S \circ T(x) = B(Ax + b) + c, \qquad \det T = \det A \text{ when square.}$$

For pushforward,

$$T_{\#} : \mathsf{Meas}_d^+ \to \mathsf{Meas}_m^+, \qquad \int f \, d(T_{\#}\mu) = \int f \circ T \, d\mu.$$

Distinguish exact affine maps from arbitrary callables. Only the former can support exact symbolic moment replay in general.

47.7 Symbolic example ladder

1. **E0:** $p(x) = 1 - 2x + x^4$, compute support, degree, derivative, and primitive.
2. **E1:** $\mu = \frac{1}{3}\delta_0 + \frac{2}{3}\delta_1$, compute mass and moments.
3. **E1:** $d\mu = dx$ on $[0, 1]$, derive $m_k = 1/(k + 1)$.
4. **E2:** $(dx|_{[0,1]})^{\otimes 2}$, integrate $x^2 y + 3y$.
5. **E2:** $T(x) = 2x + 1$, materialize the atomic pushforward.
6. **F:** attempt to treat weights $(1, -1)$ as an `AtomicMeasure`.

48. Integration providers and quadrature

48.0 Integration dispatcher as a typed case split

Outline `integrate(integrand, measure, provider=None)` as

$$\text{integrate} : (\mathsf{ExactPolynomial}_d \cup \mathsf{Callable}_d) \times \mathsf{Measure}_d \times (\mathsf{CompositeSimpson} \cup \{\emptyset\}) \to \mathsf{IntegralResult}.$$

The `provider` argument is keyword-only. Derivative-provider objects use the separate public function `integrate_derivative_provider(...)`. They are not an `integrand` branch of `integrate(...)`. The final prose must display which branch is exact, which is numerical, and which requires an explicit quadrature rule.

48.1 Signed quadrature functionals

Define

$$Q[f] = \sum_{i=1}^{N} w_i f(x_i), \qquad w_i \in \mathbb{Q} \text{ or } \mathbb{R}.$$

Public maps:

$$Q : f \mapsto Q[f],$$
$$Q^{(1)} \otimes Q^{(2)} : f(x, y) \mapsto \sum_{i,j} w_i^{(1)} w_j^{(2)} f(x_i, y_j),$$
$$Q[x^\alpha] : \alpha \mapsto \text{quadrature moment.}$$

Cover `node_count`, `exact_data`, `positive`, `apply`, `moment`, `integrate_polynomial`, `tensor_product`, `as_atomic_measure`, `degree_of_exactness`, `certify_polynomial_integral`, and `certificate`.

The conversion

$$Q \mapsto \sum_i w_i \delta_{x_i}$$

is allowed only when every $w_i \geq 0$.

48.2 Exactness through moments

The shipped certification methods are one-dimensional. Define degree r exactness by

$$Q[x^k] = \int x^k \, d\mu \quad (0 \le k \le r), \qquad \dim Q = \dim \mu = 1.$$

Type the result

$$\text{degree_of_exactness}(Q, \mu, R) \to \mathsf{QuadratureExactness}(r, R, e, C).$$

An exact certificate for $Q[p] = \int p \, d\mu$ requires

$$r \ge \deg p.$$

Do not infer exactness above the maximum tested degree. Multivariate tensor moments may be compared manually, but the public method does not issue a multivariate degree certificate.

48.3 `IntegralResult` and `IntegrationError`

Outline

$$\mathsf{IntegralResult} = (\text{value}, \text{certificate}, \text{error}, \text{method}, \text{evaluations}).$$

The separate `ErrorGuarantee` enum has strongest-to-weakest order

$$\texttt{exact} > \texttt{rigorous} > \texttt{conditional} > \texttt{estimated} > \texttt{unknown}.$$

This is not the `Exactness` order. Explain `valid`, `exact`, and `require()` separately.

48.4 Composite Simpson branches

For $N \ge 4$ with $N \equiv 0 \pmod 4$, $h = (b-a)/N$, write

$$S_N(f) = \frac{h}{3}\left[f(a) + f(b) + 4\sum_{j \text{ odd}} f(a+jh) + 2\sum_{j \text{ even},\ 0<j<N} f(a+jh)\right].$$

If $M_4 \ge \sup_{[a,b]} |f^{(4)}|$, then

$$|I(f) - S_N(f)| \le \frac{b-a}{180} h^4 M_4.$$

Write five principal integration/error paths:

1. exact polynomial with no explicit Simpson provider: `Exactness.EXACT / ErrorGuarantee.EXACT` moment contraction.
2. Explicit polynomial Simpson with a provenance-matched derivative bound: `Exactness.EXACT / ErrorGuarantee.RIGOROUS`, meaning exact rational arithmetic computes an approximation with a generally nonzero rigorous truncation bound.
3. Explicit polynomial Simpson without that bound: `Exactness.HEURISTIC / ErrorGuarantee.ESTIMATED` from the exact rational fine/coarse difference.
4. Callable Simpson with an assumed bound: `Exactness.CONDITIONAL / ErrorGuarantee.CONDITIONAL`.
5. Callable Simpson without a bound: `Exactness.HEURISTIC / ErrorGuarantee.ESTIMATED`, because the fine/coarse Richardson difference is always computed.

`ErrorGuarantee.UNKNOWN` belongs to a bare `QuadratureRule.apply()` or `integrate_polynomial()` without a target-integral theorem, not to a Simpson branch that omitted its fine/coarse comparison.

48.5 Derivative providers

For fixed atoms and weights, define

$$F(x) = \sum_j w_j F_j(x), \qquad D^r F(x) = \sum_j w_j D^r F_j(x).$$

Type `AtomicIntegratedDerivativeProvider` as

$$(F_j, w_j)_{j=1}^N \mapsto F, \qquad \mathsf{Provider}(d_s, d_t, r) \to \mathsf{Provider}(d_s, d_t, r).$$

Reject parameter-dependent $w_j(x)$ because

$$D \sum_j w_j(x) F_j(x) = \sum_j Dw_j(x) F_j(x) + \sum_j w_j(x) DF_j(x).$$

The unimplemented first term must not be dropped.

48.6 Symbolic example ladder

1. certify a two-node exact rule against moments through degree one.
2. Tensor two exact rules and verify bivariate moments.
3. Integrate $x^4 - 2x + 1$ exactly and by Simpson.
4. Show a matched `DerivativeBound` and an assumed one side by side.
5. Fail on any Simpson count not divisible by four, including 2, 6, and odd counts.
6. Fail `as_atomic_measure()` for a negative quadrature weight.

49. Integral identities and coordinate covectors

49.1 Fundamental theorem and integration by parts

For $p \in \mathsf{Poly}_1(\mathbb{Q})$ and $d\mu = \rho\, dx|_{[a,b]}$, define $P' = p$ and certify

$$\int_a^b p(x)\rho\, dx = \rho(P(b) - P(a)).$$

For $u, v \in \mathsf{Poly}_1(\mathbb{Q})$, certify

$$\int_a^b uv'\rho\, dx = \rho[uv]_a^b - \int_a^b u'v\rho\, dx.$$

Both are finite polynomial identities, not claims about weak derivatives or general function spaces.

49.2 Finite Fubini

For $p \in \mathsf{Poly}_d$ and $\mu = \mu_1 \otimes \cdots \otimes \mu_d$, outline

$$I_\sigma(p) = \int \cdots \int p(x_1, \ldots, x_d)\, d\mu_{\sigma(1)} \cdots d\mu_{\sigma(d)}.$$

The exact finite claim is

$$I_\sigma(p) = I_\tau(p) \quad \forall \sigma, \tau \in S_d.$$

`all_fubini_orders` must document the factorial enumeration budget and `max_dimensions=7` guard.

49.3 Pushforward and oriented change of variables

For exact affine T, certify

$$\int_Y f(y)\, d(T_{\#}\mu)(y) = \int_X f(Tx)\, d\mu(x).$$

For $T(x) = sx + t$, $s \neq 0$, distinguish:

$$T_{\#}(\rho\, dx|_{[a,b]}) = \frac{\rho}{|s|}\, dy|_{[\min T(a,b), \max T(a,b)]}$$

from the oriented substitution identity, where the sign of s remains in the directed endpoints.

49.4 Coordinate integration covectors

Let

$$B_n = (b_0, \dots, b_n), \qquad p = \sum_{k=0}^{n} c_k b_k.$$

Define

$$\ell_{B,\mu} = \left(\int b_0\, d\mu, \dots, \int b_n\, d\mu \right), \qquad \int p\, d\mu = \ell_{B,\mu} c.$$

Type

$$\ell_{B,\mu} : \mathsf{P}_n^B \to \mathbb{Q}.$$

For a basis change $C_{S\to T}$, require

$$\ell_T C_{S\to T} = \ell_S.$$

For the derivative matrix $D_n : \mathsf{P}_n \to \mathsf{P}_{n-1}$, require

$$\ell_{n-1} D_n = \rho(e_b - e_a).$$

A bare weight vector is only a finite functional until its triangular basis witness is replayed.

49.5 Analysis-synthesis commutation

For atomic samples $f = (f(x_1), \dots, f(x_N))$, weights w, analysis A, and synthesis S, outline the diagram

$$f@ > A >> c$$

$$@Vw^T VV@VVw^T SV$$

$$\mathbb{K}@ = \mathbb{K}$$

with the finite check

$$w^T f = w^T S A f.$$

The certificate must incorporate the frame-system certificate. Zero-valued samples must not allow an invalid SA to pass vacuously.

49.6 Canonical worked identity

Reserve the full derivation

$$f(y) = y^4 - 2y + 1, \qquad y = 2x + 1, \qquad x \in [0, 1],$$

so that

$$\int_0^1 f(2x+1)\, dx = \frac{106}{5}.$$

Show the same scalar via direct pullback, pushforward measure, monomial covector, and one nonmonomial coordinate family.

50. Typed finite expression model

50.0 Semantic universe

Restrict the model to

$$\mathbb{Q}, \qquad \mathsf{P}_n^B, \qquad \mathsf{Lin}(X, Y).$$

Explicitly exclude arbitrary CAS terms, higher-order functions, analytic identities, nonlinear maps, and infinite-dimensional operators.

50.1 Type witnesses

Type records:

$$\begin{aligned} \mathsf{RationalSpace} &: \dim = 1, \\ \mathsf{PolynomialCoordinateSpace}(x, n, B) &: \dim = n + 1, \\ \mathsf{LinearMapType}(X, Y) &: X \to Y. \end{aligned}$$

Semantic identity is

$$\mathrm{key}(\mathsf{P}_n^B) = (x, n, B),$$

not a mutable display label.

50.2 Expression grammar

Give the formation rules:

$$\frac{v \in X}{\mathrm{Value}(v) : X}, \qquad \frac{A \in \mathsf{Lin}(X, Y)}{\mathrm{Map}(A) : X \to Y},$$

$$\frac{A : X \to Y \quad B : Y \to Z}{B \circ A : X \to Z}, \qquad \frac{A, B : X \to Y}{A + B : X \to Y},$$

$$\frac{q : \mathbb{Q} \quad A : X \to Y}{qA : X \to Y}, \qquad \frac{A : X \to Y \quad x : X}{A(x) : Y}.$$

Cover nodes `ExactValueLiteral`, `ExactMapLiteral`, `Symbol`, `Let`, `IdentityMap`, `ZeroMap`, `Compose`, `Sum`, `Scale`, and `Apply`.

Clarify ordering:

$$\mathrm{compose}(B, A) = B \circ A, \qquad \mathrm{then}(A, B) = B \circ A.$$

50.3 Lexical binding

For

$$\text{let } x = e_1 \text{ in } e_2,$$

record

$$\mathrm{type}(e_1) = \mathrm{declared}(x), \qquad \mathrm{FV}(e_2) \setminus \{x\}.$$

Binder identity is object identity, not merely string equality. Plan paired examples for shadowing, same-name free symbols, and alpha-equivalent binders.

50.4 Budgets and structural safety

Outline `ExpressionStats` as

$$(N_{nodes}, D, N_{steps}, N_{entries}, n_{max}, b_{max}, B_{json}).$$

Outline `ExpressionBudget` inequalities

$$N_{nodes} \le N_*, \quad D \le D_*, \quad n_{max} \le n_*, \quad b_{max} \le b_*, \quad B_{json} \le B_*.$$

Cycle rejection and maximum symbol length belong in the construction chapter, not only the persistence chapter.

50.5 Failure matrix

Include one symbolic failure for each:

- vector length mismatch.
- Matrix shape mismatch.
- Composition endpoint mismatch.
- Sum type mismatch.
- Application source mismatch.
- Let declaration mismatch.
- Cyclic object graph.
- Oversized coefficient or serialized payload.

51. Expression evaluation and certification

51.1 Basis-aware primitive maps

For source basis S and target basis T, write

$$C^{(n)}_{S\to T} = B^{-1}_{T,n} B_{S,n}.$$

The primitive ledger must include:

$$\begin{aligned}
C_{S\to T} &: \mathsf{P}^S_n \to \mathsf{P}^T_n,\\
D_B &: \mathsf{P}^B_n \to \mathsf{P}^B_{n-1} \quad (n>0),\\
D_B &: \mathsf{P}^B_0 \to \mathsf{P}^B_0 \quad (n=0),\\
J_{a,B} &: \mathsf{P}^B_{n-1} \to \mathsf{P}^B_n,\\
e_{a,B} &: \mathsf{P}^B_n \to \mathbb{Q},\\
\ell_{B,[a,b],\rho} &: \mathsf{P}^B_n \to \mathbb{Q},\\
S_{s,t} &: \mathsf{P}^B_n \to \mathsf{P}^B_n,\\
\Delta_h &: \mathsf{P}^B_n \to \mathsf{P}^B_n,\\
\partial_{a,b} &: \mathsf{P}^B_n \to \mathbb{Q},\\
\iota &: \mathbb{Q} \to \mathsf{P}^B_n.
\end{aligned}$$

The finite-difference image has degree at most $n-1$, but the shipped primitive re-encodes it in the original P^B_n coordinate space. The degree-zero derivative is the zero endomorphism. Match these with `PrimitiveKind`: coordinate change, derivative, antiderivative, evaluation, interval integration, affine substitution, finite difference, boundary functional, and constant injection.

51.2 Replay identities

Reserve exact derivations of

$$\begin{gathered}
C_{T\to U}C_{S\to T} = C_{S\to U}, \qquad C_{T\to S}C_{S\to T} = I,\\
D_T C^{(n)}_{S\to T} = C^{(n-1)}_{S\to T} D_S,\\
J_{a,T} C^{(n-1)}_{S\to T} = C^{(n)}_{S\to T} J_{a,S},\\
D_B J_{a,B} = I, \qquad J_{a,B} D_B = I - u_B e_{a,B},\\
e_{a,T}C_{S\to T} = e_{a,S}, \qquad \ell_T C_{S\to T} = \ell_S,\\
\ell_{n-1} D_n = \rho(e_b - e_a).
\end{gathered}$$

`replay_matrix` must recompute the matrix from its witness. A caller-provided label or certificate text is never sufficient.

51.3 Checking, normalization, and equality

Type the checking phase

$$\text{check} : e \mapsto (\text{type}(e), \text{stats}(e), \text{FV}(e), C_e).$$

For a closed term, define

$$\text{NF}(e) \in \mathbb{Q} \cup \mathbb{Q}^n \cup \mathbb{Q}^{m\times n}.$$

Decidable equality is

$$e_1 \equiv e_2 \iff \text{type}(e_1) = \text{type}(e_2) \ \wedge \ \text{NF}(e_1) = \text{NF}(e_2).$$

This is entrywise exact rational equality. For open terms, distinguish structural equality from a universally quantified identity.

51.4 Conservative simplification

For each `RewriteStep`, display

$$(r, \text{id}, d_{before}, d_{after}, H_{side}, C_r).$$

Candidate rules to explain symbolically:

$$I \circ A \to A, \quad A \circ I \to A, \quad A + 0 \to A, \quad 1A \to A, \quad 0A \to 0,$$

only when the type witnesses and side conditions replay.

51.5 Certificate propagation adversaries

Construct three paired examples:

1. exact parent plus invalid child ⇒ invalid result.
2. Exact parent plus numerical child ⇒ no exact upgrade.
3. Exact matrices plus unchecked side condition ⇒ conditional.

52. Expression planning and persistence

52.1 Matrix-chain dynamic program

For dimensions $(p_0, \dots, p_n)$, let $A_i \in \mathbb{Q}^{p_{i-1}\times p_i}$. Define

$$m[i,i] = 0,$$

$$m[i,j] = \min_{i\le k<j} \{m[i,k] + m[k+1,j] + p_{i-1}p_k p_j\}.$$

Record split $s[i,j]$ and reconstruct a `MatrixChainTree`.

The theorem is narrow:

optimal among binary parenthesizations under dense scalar cost.

Do not claim optimization of sparsity, rank, memory, transfer, kernel launch, or numerical stability.

52.2 Expression plan IR

Outline each instruction as

$$I_k = (\text{opcode}, \text{inputs}, \text{type}, \text{payload}, c_k, C_k).$$

Then

$$P = (d_e, \text{type}(e), (I_k), k_{out}, \sum c_k, d_P, C_P).$$

`compile_expression` must emit and replay the plan. `execute_plan` must validate d_P before execution. `evaluate_expression` compares

$$\text{NF}_{direct}(e) = \text{NF}_{plan}(e).$$

52.3 Strict portable schema

Record constants:

- `EXPRESSION_SCHEMA = "komorebi_math.expression.v1".`
- `EXPRESSION_SCHEMA_VERSION = 1.`
- `EXPRESSION_NODE_VERSION = 1.`
- `EXPRESSION_TYPE_VERSION = 1.`

Canonical digest:

$$d(e) = \mathrm{SHA256}(\mathrm{CanonicalUTF8NFCJSON}(\mathrm{record}(e))).$$

The record must include the complete type witness. Rationals are reduced integer pairs, never binary floats.

52.4 Decoder threat model

Explicitly reject:

- duplicate JSON keys.
- Floats and noncanonical rational tokens.
- Unknown schema, node, or type versions.
- Digest mismatch.
- Unknown node tags.
- Excessive depth, nodes, entries, coefficient bits, or bytes.
- Cycles in reconstructed objects.
- Arbitrary class imports or function execution.

Because serialized JSON expands DAG sharing, the encoder must replay the expanded record against the same budget.

52.5 Canonical typed example

Reserve an end-to-end expression

$$e = \ell_T \, C_{S\to T} \, c_S$$

and a map identity pipeline with

$$\mathrm{NF}(e) = \frac{5}{4}, \qquad \mathrm{NF}(P) = I_4.$$

Show construction, checking, simplification, compilation, direct/plan replay, JSON roundtrip, digest equality, and one-byte tamper rejection.

Part IX. Quantum, deformation, and finite Langlands models

53. Finite quantum foundations

53.0 Scope and feature status

Domain:

finite-dimensional complex matrices and finite packet families.

Certificates are ordinarily **N**, with residual and tolerance. Exclude continuum QFT, infinite volume, Lorentzian reconstruction, anomaly cancellation, and quantum gravity conclusions.

53.1 Seed-orbit systems

For observables $A_1, \dots, A_r \in \mathsf{M}_d(\mathbb{C})$ and seeds $\psi_1, \dots, \psi_s$, define

$$\mathcal{H}_{seed} = \operatorname{span}\{A_{i_k} \cdots A_{i_1} \psi_j : k \geq 0\}.$$

`QuantumSeedSystem` exposes `dimension`, `orbit_basis`, `orbit_dimension`, `is_generating`, and `certificate`.

Rank diagnostics:

$$\operatorname{SchmidtRank}(\psi) = \operatorname{rank}(\operatorname{reshape}_{d_L \times d_R} \psi),$$

$$\text{local_orbit_dimension} = d_L \operatorname{SchmidtRank}(\psi).$$

State the singular-value threshold in every numerical rank example.

53.2 Finite symmetry projectors

For a finite representation U_g and character χ, define

$$P_\chi = \frac{d_\chi}{|G|} \sum_{g \in G} \overline{\chi(g)} U_g.$$

For $U^n = I$, define cyclic packets

$$P_q = \frac{1}{n} \sum_{r=0}^{n-1} \omega^{-qr} U^r.$$

Certificate residuals must include

$$\max_q \|P_q^2 - P_q\|, \quad \max_{q \neq r} \|P_q P_r\|, \quad \left\| \sum_q P_q - I \right\|.$$

For charge a, check

$$UAU^* = \omega^a A, \qquad P_b A P_c = 0 \text{ unless } b = c + a \pmod{n}.$$

53.3 Euclidean and spectral diagnostics

For Hermitian H, spacing $a > 0$, and reference energy E_0, define

$$T = e^{-a(H - E_0 I)}.$$

For test vectors v_i and times t_i, form

$$M_{ij} = \langle v_i, T^{t_i + t_j} v_j \rangle.$$

`reflection_certificate` checks finite positive semidefiniteness of M.

Spectral functions:

$$K(t) = \operatorname{Tr}(e^{-tL}), \qquad d_s(t) = 2t \frac{\operatorname{Tr}(L e^{-tL})}{\operatorname{Tr}(e^{-tL})}.$$

For a periodic lattice,

$$\lambda_k = \frac{4}{h^2} \sum_j \sin^2\left(\frac{\pi k_j}{N_j}\right).$$

`spectral_dimension` is a finite diagnostic, not a continuum-dimension theorem.

53.4 Constraints and anomaly residuals

Fit

$$[C_i, C_j] \approx \sum_k f_{ij}^{\ k} C_k$$

and record the quotient residual. For a coarsening map R, compute

$$\Delta_i = RC_i^{fine} - C_i^{coarse} R.$$

Call Δ_i a transported constraint defect, not proof of a continuum anomaly.

53.5 Symbolic examples and failures

1. two-qubit computational-basis generation.
2. Non-generating seed with deficient orbit rank.
3. C_n projectors and charged Pauli-style operator.
4. Periodic Laplacian and low-mode dispersion bound.
5. Nearly dependent constraints with tolerance-sensitive rank.

54. Circuits, channels, and open dynamics

54.1 Packet-expanded circuits

For gates $U_1, \dots, U_m$ and packet systems $P_a^{(j)}$, define a path block

$$B_{a_m \leftarrow a_0}^{a_1,\dots,a_{m-1}} = P_{a_m}^{(m)} U_m P_{a_{m-1}}^{(m-1)} \cdots U_1 P_{a_0}^{(0)}.$$

The reconstruction certificate checks

$$\left\| U_m \cdots U_1 - \sum_{a_0,\dots,a_m} B_a \right\| \le \varepsilon.$$

For path vectors v_γ, decompose

$$\left\| \sum_\gamma v_\gamma \right\|^2 = \sum_\gamma \|v_\gamma\|^2 + 2\,\mathrm{Re} \sum_{\gamma<\eta} \langle v_\gamma, v_\eta \rangle.$$

The constructor's path guard is

$$\prod_{j=1}^{m-1} |P^{(j)}| \le \texttt{max_paths}.$$

It bounds internal paths for one fixed source/target packet pair. The outer source and target loops used by complete reconstruction are not included.

54.2 Quantum channels

Define

$$\Phi(X) = \sum_j K_j X K_j^*, \qquad \sum_j K_j^* K_j = I \quad \text{for TP.}$$

Method map:

- `apply` and `apply_density`.
- `adjoint`.
- `is_trace_preserving` and `is_unital`.
- `superoperator` and `choi_matrix`.
- `compose` and `tensor`.
- `packet_blocks` and `covariance_certificate`.

Certificate residuals:

$$r_{TP} = \left\| \sum_j K_j^* K_j - I \right\|, \qquad r_{CP} = \max(0, -\lambda_{min}(J_\Phi)).$$

For amplitude damping,

$$K_0 = \begin{pmatrix} 1 & 0 \\ 0 & \sqrt{1-\gamma} \end{pmatrix}, \qquad K_1 = \begin{pmatrix} 0 & \sqrt{\gamma} \\ 0 & 0 \end{pmatrix}, \qquad 0 \le \gamma \le 1.$$

54.3 Lindblad generators

Define

$$\mathcal{L}(\rho) = -i[H, \rho] + \sum_j \left(L_j \rho L_j^* - \frac{1}{2}\{L_j^* L_j, \rho\} \right).$$

Expose `apply`, `superoperator`, `evolve`, `trace_preservation_certificate`, and `coherence_preservation_certificate`.

The trace certificate checks

$$\operatorname{Tr} \mathcal{L}(X) = 0$$

on the finite matrix representation. It does not establish properties of an unbounded infinite-dimensional generator.

54.4 Kato transport

For differentiable packets $P_a(t)$, define

$$K(t) = \sum_a \dot{P}_a(t) P_a(t), \qquad \dot{P}_a = [K, P_a], \qquad K^* = -K.$$

Transport solves

$$\dot{U}(t) = K(t) U(t), \qquad P_a(t) U(t) = U(t) P_a(0).$$

Document `direct_packet_rotation`, `MovingPacketSystem`, `KatoTransport`, derivative certificates, refinement certificates, and endpoint intertwining.

Finite-difference derivative checks are **N**, not proofs of differentiability.

54.5 Leakage

For fixed P, $Q = I - P$, certify

$$\|Q e^{-itH} P\| \le |t| \, \|QHP\|.$$

For moving packets, define

$$G(t) = Q(t)(H(t) - iK(t))P(t), \qquad B = \int_{t_0}^{t_1} \|G(t)\| \, dt.$$

The shipped result is **N** (`Exactness.NUMERICAL`) and carries the explicit unchecked assumption `quadrature error between samples`. It does not issue a stronger continuum error bound.

55. Error correction and effective dynamics

55.1 Quantum codes

For code projector P and errors E_a, define

$$R_{ab} = PE_a^* E_b P - \alpha_{ab} P, \qquad \alpha_{ab} = \frac{\operatorname{Tr}(PE_a^* E_b P)}{\operatorname{rank} P}.$$

Knill-Laflamme is the finite criterion

$$\max_{a,b} \|R_{ab}\| \le \varepsilon.$$

Separate three claims:

1. criterion satisfied for supplied P, E_a.
2. Recovery map constructed.
3. Chosen error mixture is trace preserving.

One does not imply the other two without their checks.

55.2 Feshbach reduction

For

$$H = \begin{pmatrix} H_{PP} & H_{PQ} \\ H_{QP} & H_{QQ} \end{pmatrix},$$

define

$$\Sigma(z) = H_{PQ}(z - H_{QQ})^{-1} H_{QP}, \qquad H_{eff}(z) = H_{PP} + \Sigma(z).$$

Reserve derivations of

$$P(z - H)^{-1} P = (z - H_{eff}(z))^{-1},$$

and, for $\operatorname{Im} z > 0$,

$$\operatorname{Im} \Sigma(z) \preceq 0.$$

The memory kernel is finite hidden-sector dynamics. Bare Markovian closure is certified only when

$$\|H_{PQ}\| \le \varepsilon.$$

55.3 Hamiltonian towers

For finite $H_0, \dots, H_N$, compute

$$\delta_n = \|H_{n+1} - H_n\|, \qquad V_N = \sum_{n=0}^{N-1} \delta_n,$$

and bound

$$\|e^{-itH_{n+1}} - e^{-itH_n}\| \le |t| \delta_n.$$

The report is finite in scope and **N**: it numerically checks the supplied adjacent norms and Duhamel bounds. Infinite-tail summability remains unchecked.

55.4 Example and failure sequence

1. exact code projector plus Pauli error set.
2. Near-miss error set with visible residual matrix.
3. Recovery-channel check independent of KL check.
4. Two-block Feshbach Hamiltonian and projected resolvent.
5. Coupled hidden sector showing nonzero memory.
6. Finite tower whose observed differences decrease but whose infinite tail is not claimed summable.

56. Finite deformation systems

56.0 Scope boundary

The implemented objects are finite lift levels, finite-field complexes, finite support computations, finite trace tables, and finite occupancy diagnostics. No finite report proves existence of an infinite deformation or inverse-limit branch.

56.1 Lift forests and visibility

For levels $X_0, \dots, X_N$ and reductions $r_m : X_m \to X_{m-1}$, define

$$\mathrm{Core}_m = X_m \cap \bigcap_{n=m+1}^{N} \mathrm{im}(r_{n\to m}).$$

`FiniteLiftForest` exposes levels, children, images, stable cores, finite branches, and report.

For rational x_i, define

$$d_{ij} = v_p(x_i - x_j) + 1,$$

and verify nesting through

$$d_{ik} \geq \min(d_{ij}, d_{jk}).$$

`VisibilityTree` exposes separation depths, partitions, refinement chains, and `ultrametric_certificate()`, which is a plain boolean exhaustive finite check rather than a `Certificate` object.

56.2 Exact chart counts

List and derive

$$N_{smooth}(q, d, m) = q^{d(m-1)},$$
$$N_{weighted}(q, w, m) = q^{\sum_i \max(m-w_i,0)},$$
$$N_{x^e=0}(q, e, m) = q^{m-\lceil m/e \rceil},$$
$$N_{xy=0}(q, m) = q^m + m(q-1)q^{m-1}.$$

For

$$R(T) = \frac{A(T)}{B(T)} = \sum_{m\geq 0} a_m T^m,$$

show how `RationalCountingSeries.coefficient` uses the exact denominator recurrence. Compare the first coefficients against direct count functions.

56.3 Finite cochain complexes

For

$$C^0 \xrightarrow{d^0} C^1 \xrightarrow{d^1} \cdots, \qquad d^{k+1}d^k = 0 \pmod{p},$$

compute

$$\dim H^k = \dim C^k - \operatorname{rank} d^k - \operatorname{rank} d^{k-1}.$$

For packetwise cohomology require

$$P_a^{k+1} d^k = d^k P_a^k, \qquad \sum_a P_a^k = I, \qquad P_a^k P_b^k = \delta_{ab} P_a^k.$$

All ranks here are exact over $\mathbb{F}_p$.

56.4 Fitting support and occupancy

For a presentation $M = \operatorname{coker} A$, define the fiber dimension

$$\dim M_x = (\#\text{generators}) - \operatorname{rank} A(x).$$

Outline `PresentedModule.fitting_generators`, `evaluate`, `fiber`, and `finite_support`. Distinguish characteristic zero from $\mathbb{F}_p$, and require denominators to remain units after reduction.

For numerical subspaces $V_1, \dots, V_r$, record

$$d_j = \dim(V_1 \cap \cdots \cap V_j), \qquad d_{j+1} \le d_j.$$

`nested_subspace_occupancy` must report principal-angle tolerance and a near-threshold failure example.

56.5 Frobenius companion systems

For a finite trace table $T = (t_{v,i})$, seek a row set S such that

$$\operatorname{rank}(t_{v,i})_{v \in S} = r.$$

Then recover multiplicities from

$$T_S m = y.$$

`finite_frobenius_separator` is exact for the supplied table, but diagnostic validity and actual separation are distinct: its certificate remains valid for a rank-deficient table, `.separating` records full column rank, and `.recover(...)` raises unless `.separating` is true. Chebotarev density or separation at unlisted places remains outside the certificate.

56.6 Deformation example ladder

1. three-level residue forest and finite complete branches.
2. Branch that terminates at the deepest supplied level.
3. p-adic visibility partition chain.
4. Direct versus rational-series chart counts.
5. Packetwise cohomology over $\mathbb{F}_p$.
6. Fitting fibers with and without optional SymPy.
7. Exact finite Frobenius multiplicity recovery.

57. Local finite Langlands packets

57.0 Status boundary

The chapter covers explicit finite fields, tame packets, Gauss sums, Heisenberg models, symplectic diagnostics, and finite bridges. These do not constitute an unrestricted local Langlands correspondence.

57.1 Explicit finite fields

Represent

$$\mathbb{F}_{p^n} = \mathbb{F}_p[t]/(f(t)), \qquad f \text{ monic irreducible of degree } n.$$

Type operations

$$+, -, \cdot : \mathbb{F}_{p^n}^2 \to \mathbb{F}_{p^n}, \qquad (\cdot)^{-1} : \mathbb{F}_{p^n}^\times \to \mathbb{F}_{p^n}^\times.$$

Define

$$\operatorname{Tr}(x) = \sum_{j=0}^{n-1} x^{p^j}, \qquad \operatorname{Nm}(x) = \prod_{j=0}^{n-1} x^{p^j}.$$

Cover encoding, decoding, coercion, enumeration, primitive generator, logarithm table, and discrete logarithm. Fail on reducible modulus, cross-field arithmetic, and inversion of zero.

57.2 Tame Frobenius orbits and packets

For $a \in \mathbb{Z}/(q^n - 1)$, define

$$\mathcal{O}(a) = \{a, qa, q^2a, \ldots\}.$$

Record orbit length, regularity, dual exponent, and self-duality. The implementation checks only that q is an integer at least two. Its interpretation as a residue cardinality is a declared hypothesis. No prime-power test is performed.

For `TameWeilPacket`, require

$$FTF^{-1} = T^q.$$

Tensor refinement uses

$$g = \gcd(n, m), \qquad \ell = \operatorname{lcm}(n, m), \qquad g\ell = nm.$$

For a special monodromy block,

$$FNF^{-1} = qN, \qquad N^\ell = 0.$$

Here `SpecialMonodromyBlock.q` accepts any positive finite float, and the relation and nilpotence certificate is **N**, not exact finite-field evidence.

57.3 Finite character sums

Define additive and multiplicative characters by

$$\psi_b(x) = \exp\left(\frac{2\pi i}{p}\operatorname{Tr}(bx)\right),$$

$$\chi_k(g^j) = \exp\left(\frac{2\pi i k j}{q-1}\right).$$

Then

$$G(\chi, \psi) = \sum_{x \in \mathbb{F}_q^\times} \chi(x)\psi(x).$$

The magnitude check is numerical even though the finite enumeration is exact at the combinatorial level.

57.4 Heisenberg models

For $x, \xi, z \in \mathbb{F}_p$, define the chosen finite cocycle and

$$\rho(x, \xi, z) = \omega^z M_\xi T_x.$$

Outline exact modular coordinate multiplication separately from the numerical complex-matrix check returned by `group_law_certificate()`. Also cover trace support on the center and the rank-one isotropic translation selector.

57.5 Symplectic and quadratic data

For an alternating matrix B, require

$$B^T = -B \pmod p, \qquad v^T B v = 0.$$

Compute rank, radical, and nondegeneracy exactly over $\mathbb{F}_p$.

At $p = 2$, a quadratic refinement satisfies

$$q(x + y) - q(x) - q(y) = B(x, y).$$

Document exhaustive enumeration limit `max_dimension=20`, and state the nondegenerate even-dimensional hypotheses for Arf and normalized Gauss sign.

57.6 Projective envelopes and bridges

Recover a form from projective commutators

$$U_i U_j U_i^{-1} U_j^{-1} \approx \omega^{B_{ij}} I.$$

Report both scalar residual and nearest prime-phase residual.

A symplectic bridge M must satisfy

$$M^T B_{target} M = B_{source} \pmod{p}, \qquad \det M \neq 0 \pmod{p}.$$

For a graph of packet bridges, certify finite cycle holonomy. Do not infer a global arithmetic correspondence from graph consistency.

58. Finite-image, ramification, and completion models

58.0 Three-layer status matrix

Keep the implementation statuses visible:

Layer	Runtime status	Mathematical status
finite image	implemented	explicit finite groups and representations
ramification	partial	finite tower diagnostics, phases may be external
global support	partial	finite Hecke/support checks only
completion gates	implemented reporter	conclusion always conditional

58.1 Finite groups and induced characters

For `FiniteGroup`, outline checks of closure, identity, inverse, and associativity. They run only when `validate=True`, and associativity is exhaustive only when `order <= associativity_limit`. Above that limit it is not checked. `FiniteGroup.certificate()` does not replay the axioms and currently returns an exact valid record even for `validate=False`. It must not be presented alone as an exhaustive group-law proof.

For $H \leq G$ and a linear character $\chi : H \to \mathbb{C}^\times$, define

$$\rho = \operatorname{Ind}_H^G \chi, \qquad \dim \rho = [G : H].$$

The induced representation certificate checks

$$\rho(g)\rho(h) = \rho(gh)$$

on the finite table within tolerance.

58.2 Orbit packets and Mackey decomposition

For a monomial action, require orbit-stabilizer

$$|\operatorname{Orb}(v)|\,|\operatorname{Stab}(v)| = |G|.$$

Dimension checks:

$$\dim \operatorname{Sym}^m V = \binom{d+m-1}{m}, \qquad \dim \bigwedge^r V = \binom{d}{r}.$$

For double cosets $H\backslash G/K$, define Mackey intersections

$$H \cap gKg^{-1}$$

and verify total induced dimension conservation.

58.3 Clifford towers and virtual boundaries

For a prime quotient $r = [G : H]$, encode steps

$$\mathrm{I} : d_G = r d_H, \qquad \mathrm{E} : d_G = d_H.$$

`CliffordTowerReport` records the I/E word, depth, and unresolved extension steps. It does not establish existence of every restriction or extension.

For a virtual character

$$\Theta = \sum_i c_i \Theta_i,$$

the symbolic L-expression is

$$L(\Theta, s) = \prod_i L(\Theta_i, s)^{c_i}.$$

Negative c_i mark a virtual boundary, not a positive automorphic realization.

58.4 Ramification towers

For an alternating quotient form B, define

$$d_{Heis} = p^{\mathrm{rank}(B)/2}.$$

The public `.heisenberg_dimension` property first requires the form rank to be even and raises otherwise. A stage must satisfy

$$\text{orbit_size} = \frac{\text{parent_order}}{\text{child_order}},$$

with strictly increasing breaks and a valid order chain.

At $p = 2$, `phase_diagnostic()` exposes Arf and Gauss-sign data only when a nondegenerate quadratic refinement was supplied. The current tower reporter has a narrower implementation check: it marks a dyadic stage as not unresolved whenever any quadratic-refinement object is present, without independently rechecking nondegeneracy. Preserve this distinction unless the reporter is repaired.

58.5 Finite Hecke and support diagnostics

For projectors e_i and commuting operators T_j, check

$$e_i^2 = e_i, \quad e_i e_k = 0 \ (i \neq k), \quad \sum_i e_i = I, \quad [e_i, T_j] = 0.$$

`SupportComparison` requires each declared source component to occur in the declared image. This is a finite support statement only.

The dyadic diagnostic records

$$X = g - 1 \in \mathbb{F}_2[C_2], \qquad X^2 = 0,$$

so rational sign projectors merge after residual reduction. Treat this as a finite algebra calculation, not a global modularity theorem.

58.6 Patching and completion gates

Define the five mandatory gates

$$G = \{R_{res}, R_{patch}, R_{comp}, R_{fiber}, R_{real}\}.$$

The conceptual exposition order is

$$R_{res} \to R_{patch} \to R_{comp} \to R_{fiber} \to R_{real}.$$

The runtime report does not enforce this dependency graph: it requires an unordered mandatory-key map and computes the conjunction of all five values. For statuses $s_g \in \{0, 1\}$, define

$$\text{complete} = \bigwedge_{g \in G} s_g, \qquad \text{unmet} = \{g : s_g = 0\}.$$

`AutomorphicCompletionReport.result()` remains a `ConditionalResult` even if all $s_g = 1$. It records that external inputs were supplied, not that the package independently proved them.

For compatible propagation, require

$$S_{common} \wedge S_{ss} \wedge S_{aut} \wedge S_{lg}.$$

No missing conjunct may be inferred from the others.

58.7 Patching plans

`AutomorphicInductionPlan` produces a sequence of formal operations from a base field, inducing field, inducing character, and optional cyclic tower.

Its shipped certificate records

$$H_? = \left\{ \begin{array}{l} \text{inducing character algebraic when required,} \\ \text{field tower satisfies external automorphic-induction hypotheses} \end{array} \right\}.$$

"Class field theory for …" is an operation string, not a separate current `assumptions_unchecked` entry. The prose may still flag it as a guide-level mathematical warning. This object is a plan, not an execution engine for those theorems.

58.8 Symbolic example ladder

1. cyclic group, subgroup, cosets, and induced character.
2. Symmetric/exterior orbit packet dimension checks.
3. Mackey packets over explicit double cosets.
4. Clifford word with one unresolved extension.
5. Ramification stage with and without dyadic phase data.
6. Finite Hecke packets and a support-losing comparison.
7. Completion report failing `REALIZATION`.
8. All five gates supplied, with result still explicitly conditional.
9. Compatible propagation with one intentionally false conjunct.

Cross-chapter symbolic capstones

A. Exact-to-conditional status preservation

Construct

$$c_S \xrightarrow{C_{S\to T}} c_T \xrightarrow{\ell_T} q \in \mathbb{Q} \xrightarrow{\text{declared input}} \mathcal{D}_{finite} \xrightarrow{G} \mathsf{ConditionalResult}.$$

Even if the first three arrows are **E**, the last output is **C** whenever $H_?(G) \neq \emptyset$.

B. Finite versus asymptotic boundary table

Finite shipped statement	Forbidden silent promotion
all supplied Fubini orders agree	general infinite product Fubini theorem
exact finite expression normal form	general theorem proving or CAS equality
finite channel Choi matrix is positive	arbitrary infinite-dimensional complete positivity
finite Hamiltonian differences are bounded	convergence of an infinite tower
finite lift branch reaches level N	existence of an inverse-limit point
supplied Frobenius rows separate constituents	Chebotarev separation theorem
finite tame packet relation holds	unrestricted local Langlands correspondence
all completion booleans were supplied true	independent proof of automorphy

C. Required certificate displays

The finished prose must print, not merely mention:

1. exact theorem identifier and empty unchecked-assumption set.
2. Numerical residual, tolerance, norm, and scale.
3. Conditional unchecked-assumption list.
4. Resource limit that bounded the finite search.
5. Deliberate invalid example and its exception or failed certificate.
6. Deterministic serialization digest where persistence is involved.

D. Source and verification anchors

Integration chapters must absorb:

- `examples/block19_measure_integration_worked_example.py`.
- Integration portion of `examples/block21_public_capability_tour.py`.
- `docs/FINITE_MEASURE_INTEGRATION.md`.
- `run_integration_verification` and `run_block19_verification`.

Expression chapters must absorb:

- `examples/block20_typed_expression_worked_example.py`.
- Expression portion of `examples/block21_public_capability_tour.py`.
- `docs/TYPED_FINITE_EXPRESSION_CALCULUS.md`.
- `run_expression_verification` and `run_block20_verification`.

Quantum chapters must absorb:

- `examples/quantum_symmetry_circuit.py`.
- `examples/quantum_channel_qec.py`.
- `examples/kato_feshbach.py`.
- `examples/euclidean_spectral.py`.
- `run_quantum_verification` and `run_block5_verification`.

Deformation and Langlands chapters must absorb:

- `examples/deformation_support.py`.
- `examples/frobenius_companions.py`.
- `examples/tame_heisenberg.py`.
- `examples/finite_image_clifford.py`.
- `examples/ramification_completion.py`.
- `run_deformation_verification`.
- All `run_langlands_*_verification` entry points.
- `run_block7_verification`.

E. Documentation requirements for these chapters

- Every public class has a typed constructor and method/property ledger.
- Every function has explicit domain, codomain, and shape constraints.
- Every exact identity states finite rational or finite-field hypotheses.
- Every numerical check states norm, tolerance, and scale.
- Every conditional result exposes all unchecked assumptions.
- Every optional SymPy path has an installation and fallback note.
- Every factorial, path, term, group-order, dimension, and sampling guard is visible.
- Every example includes a deliberate failure or adversarial variant.
- No finite diagnostic is narrated as an asymptotic or global theorem.
- No completion gate reporter is narrated as proving its external inputs.

- All code snippets are regenerated against package version `1.0.0b4`.
- All example outputs are captured from the final qualified environment.

88. Export, persistence, CLI, recipes, verification, and publication

Specifications for interfaces, verification, and publication

Scope: Chapters 59 through 81 and Appendices A through O.

Source basis:

- Komorebi Math $1.0.0b4$, archived source handoff
- reference package source, CLI parser, manifests, schemas, examples, and release scripts
- `outline_parts/interfaces.md`

These specifications state the schemas, workflow requirements, symbol contracts, example obligations, and publication checks for Parts X through XIV. They supplement the reader explanations.

Interface notation

$$\begin{aligned} \mathbb{S} &= \text{finite Unicode strings}, & \mathbb{B} &= \{\bot, \top\}, \\ \mathbb{N}_0 &= \{0, 1, 2, ...\}, & \mathbb{P} &= \text{normalized paths}, \\ \mathbb{J} &= \text{finite JSON values}, & \mathbb{H} &= \{0, \dots, 9, a, \dots, f\}^{64}, \\ \mathbb{V} &= \text{supported in-memory values}, & \mathbb{A} &= \text{finite byte strings}. \end{aligned}$$

$$\begin{aligned} \text{Exists}(p) &\iff p \text{ names a regular file}, \\ \text{Nonempty}(p) &\iff \text{size}(p) > 0, \\ \text{SafePath}(p) &\iff p \text{ is relative, normalized, and traversal-free}, \\ \text{SHA}(b) &= \text{SHA256}(b), \\ \text{Canon}(j) &= \text{canonical finite JSON encoding of } j, \\ \text{ExactBytes}(p, q) &\iff \text{bytes}(p) = \text{bytes}(q). \end{aligned}$$

Guide cross-reference identifiers use

$$\text{Anchor}(k, n) = \text{ug-}k\text{-}n.$$

Part X. Export, persistence, extension, and command-line use

59. Mathematical document model

59.0 Chapter contract

$$\mathcal{D} = \mathcal{M} \times \mathcal{B}^*,$$

where $\mathcal{M}$ is metadata and $\mathcal{B}^*$ is a finite ordered block sequence.

Required outputs:

- one type grammar for every public document block
- one constructor invariant per block
- one fluent-construction trace
- one data-to-document trace
- one invalid-document counterexample
- one renderer-neutrality statement

59.1 Metadata schema

$$M = (t, a, d, u, k)$$

with

$$t \in \mathbb{S} \setminus \{\epsilon\}, \quad a, d, u \in \mathbb{S} \cup \{\emptyset\}, \quad k \in \mathbb{S}^*.$$

Symbol	Field	Constraint
t	`title`	$\mathrm{strip}(t) \neq \epsilon$
a	`author`	optional
d	`date`	optional presentation string
u	`abstract`	optional
k	`keywords`	finite tuple

$$M_0 = (\text{Komorebi Math Export}, \emptyset, \emptyset, \emptyset, ()).$$

59.2 Block sum type

$$\mathcal{B} = \mathsf{Heading} \sqcup \mathsf{Paragraph} \sqcup \mathsf{Equation} \sqcup \mathsf{Table} \sqcup \mathsf{Code} \sqcup \mathsf{List} \sqcup \mathsf{Figure} \sqcup \mathsf{RawLatex}.$$

$$\begin{aligned}
\mathsf{Heading} &= (x, \ell, \lambda), && x \neq \epsilon,\ 1 \leq \ell \leq 6,\\
\mathsf{Paragraph} &= (x), && x \in \mathbb{S},\\
\mathsf{Equation} &= (q, \lambda, n, e), && q \neq \epsilon,\\
\mathsf{Table} &= (h, r, c, \lambda, \alpha), && |h| > 0,\ \forall row \in r:\ |row| = |h|,\\
\mathsf{Code} &= (x, g, c), && x, g \in \mathbb{S},\\
\mathsf{List} &= (x, o), && |x| > 0,\ o \in \mathbb{B},\\
\mathsf{Figure} &= (p, c, \lambda, w), && p \in \mathbb{P},\ w \neq \epsilon,\\
\mathsf{RawLatex} &= (x), && x \in \mathbb{S}.
\end{aligned}$$

$$\alpha = \emptyset \quad \vee \quad \alpha \in \{l, c, r\}^{|h|}.$$

59.3 `MathDocument` state machine

$$D_i = (M, (b_1, \dots, b_i)), \qquad D_i \xrightarrow{\text{add}(b_{i+1})} D_{i+1}.$$

$$\begin{aligned}
\text{section} &: \mathbb{S} \times \{1, \dots, 6\} \to \textsf{Heading},\\
\text{paragraph} &: \mathbb{S} \to \textsf{Paragraph},\\
\text{equation} &: \mathbb{S} \to \textsf{Equation},\\
\text{table} &: \mathbb{S}^m \times \mathbb{V}^{n\times m} \to \textsf{Table},\\
\text{code} &: \mathbb{S} \times \mathbb{S} \to \textsf{Code},\\
\text{bullet_list} &: \mathbb{S}^n \to \textsf{List},\\
\text{figure} &: \mathbb{P} \to \textsf{Figure}.
\end{aligned}$$

Mutation boundary:

- metadata and block records are frozen
- `MathDocument.blocks` is an ordered construction buffer
- rendering must not mutate D

59.4 Report-builder morphisms

$$\begin{aligned}
\text{document_from_data} &: \mathbb{V} \to \mathcal{D},\\
\text{sequence_document} &: \mathbb{V}^* \to \mathcal{D},\\
\text{certificate_document} &: \textsf{Certificate} \to \mathcal{D}.
\end{aligned}$$

Conceptual commutation:

$$v \to D(v) \to R_{\text{markdown}}(D) \quad \sim_{\text{presentation}} \quad v \to S_{\text{presentation}}(v) \to \text{tabulate}(v).$$

This is semantic presentation equivalence, not byte identity.

59.5 Document validity

$$\text{ValidDocument}(D) \iff \text{ValidMetadata}(M) \wedge \bigwedge_{b \in B} \text{ValidBlock}(b).$$

$$\text{PortableMeaning}(\textsf{RawLatex}) = \bot.$$

`RawLatex` is trusted renderer-specific material.

60. Renderers and writers

60.0 Rendering algebra

$$R_f : \mathcal{D} \times O_f \to \mathbb{S}, \quad f \in \{\text{text}, \text{markdown}, \text{html}, \text{latex}\}.$$

$$D = D' \implies R_f(D, O) = R_f(D', O).$$

Every block/renderer pair receives one explicit coverage row.

60.1 Renderer options and escaping

$$O_H = (\text{include_mathjax}, \text{title}),$$

$$O_T = (\text{document_class}, \text{class_options}, \text{packages}, \text{include_table_of_contents}, \text{number_sections}, \text{geometry}).$$

Standalone HTML structure and CSS are fixed renderer behavior, not `HtmlRenderOptions` fields.

$$\text{latex_escape} : \mathbb{S} \to \mathbb{S}, \qquad \text{latex_label} : \mathbb{S} \to \mathbb{S}.$$

Document exact option fields from source in symbol cards.

60.2 Export format universe

$$\mathcal{F} = \{\text{json}, \text{yaml}, \text{csv}, \text{tsv}, \text{text}, \text{markdown}, \text{html}, \text{latex}, \text{pdf}, \text{npy}, \text{npz}\}.$$

$$\iota : \text{Suffix} \rightharpoonup \mathcal{F}.$$

$$\text{md} \mapsto \text{markdown}, \quad \text{tex} \mapsto \text{latex}, \quad \text{txt} \mapsto \text{text}, \quad \text{yml} \mapsto \text{yaml}.$$

Undefined suffixes raise `ValidationError`.

60.3 Export option tuple

$$O_E = (t, a, d, u, w, c, p, i, z, k, h, \ell).$$

- t, a, d, u: document metadata
- w: overwrite permission
- c: create-parent permission
- p: preserve presentation type metadata
- i: JSON indentation
- z: include certificate
- k: retain generated TeX
- h: HTML options
- ℓ: LaTeX render/compiler options

$$\text{Writable}(p, O_E) \iff (\neg \text{Exists}(p) \vee w) \wedge (\text{Exists}(\text{parent}(p)) \vee c).$$

60.4 Writer morphisms

$$\begin{aligned}
\text{export_data} &: \mathbb{V} \times \mathbb{P} \times \mathcal{F} \times O_E \to \mathsf{ExportResult}, \\
\text{export_document} &: \mathcal{D} \times \mathbb{P} \times \mathcal{F} \times O_E \to \mathsf{ExportResult}, \\
\text{export_many} &: \mathbb{V} \times \mathbb{P}_{\text{directory}} \times \mathbb{S}_{\text{base}} \times \mathcal{F}^* \times O_E \to \mathsf{ExportResult}^*, \\
\text{export_bundle} &: \mathbb{V} \times \mathbb{P}_{\text{zip}} \times \mathbb{S}_{\text{base}} \times \mathcal{F}^* \times O_E \to \mathsf{ExportBundleResult}.
\end{aligned}$$

$$E = (p, f, n, C, \gamma).$$

$$\text{ValidExport}(E) \iff \gamma.\text{valid} \wedge \text{Exists}(p) \wedge \text{size}(p) = n > 0.$$

60.5 Format-specific contracts

JSON :	$v \to S_{\text{presentation}}(v)$	sorted keys, finite JSON,
YAML :	$v \to \text{safe_dump}(v)$	requires PyYAML,
CSV/TSV :	$v \to (h, r)$	rectangular table,
TEXT :	$v \to D \to R_{text}(D)$	human presentation,
MD/HTML/TEX :	$v \to D \to R_f(D)$	localized figures,
PDF :	$v \to D \to \text{TEX} \to \text{PDF}$	external compiler,
NPY :	$v \to \text{asarray}(v)$	pickle disabled,
NPZ :	$v \to \{name \mapsto array\}$	compressed, pickle-free.

60.6 Figure localization

$$a_i = \text{stem}(t) + \texttt{_asset_} + \text{pad}_3(i) + \text{suffix}(p_i).$$

$$\text{LocalizeSafe}(p_i, a_i, w) \iff \text{Exists}(p_i) \wedge (\neg\,\text{Exists}(a_i) \vee \text{ExactBytes}(p_i, a_i) \vee w).$$

60.7 LaTeX compilation state machine

$$\mathcal{L} = \{\text{auto}, \text{pdflatex}, \text{xelatex}, \text{lualatex}, \text{latexmk}\}.$$

$$\text{source.tex} \xrightarrow{\text{resolve_latex_engine}} e \xrightarrow{\text{compile_latex}} (p_{pdf}, p_{log}, r, \gamma).$$

$$\text{ValidCompilation} \iff r = 0 \wedge \text{Exists}(p_{pdf}) \wedge \text{Nonempty}(p_{pdf}) \wedge \gamma.\text{valid}.$$

Required guards:

- shell escape disabled by default
- finite positive timeout
- return code checked
- requested engine available
- auxiliary retention explicit
- compiler log retained
- `latexmk` must not re-enable shell escape

60.8 Manifested bundle

$$B = (p, M, F, p_m, \gamma).$$

$$\mathcal{M}_B = \{(m, \text{size}(m), \text{SHA}(\text{bytes}(m))) : m \in M\}.$$

$$\text{ValidBundle}(B) \iff \gamma.\text{valid} \wedge \forall m \in M : \text{SafePath}(m) \wedge \text{ManifestMatches}(B).$$

`ExportBundleResult.valid` itself is only γ.valid. The stronger predicate above is a guide/release replay requirement, not a guarantee currently established by that result certificate. The ZIP contains `MANIFEST.sha256`, while p_m names the external `<zip>.manifest` companion.

60.9 Export adversaries

- unknown suffix
- existing output without overwrite
- absent parent with creation disabled
- nonrectangular table input
- missing PyYAML
- missing LaTeX engine
- compiler timeout
- missing figure asset
- conflicting localized asset
- unsafe or colliding bundle member

61. Portable persistence

61.0 Separation of serialization layers

$$S_{\text{presentation}} : \mathbb{V} \to \mathbb{J}, \qquad S_{\text{portable}} : \mathbb{V}_{\text{registered}} \to \mathbb{J}_{\text{versioned}}.$$

$$S_{\text{presentation}} \not\equiv S_{\text{portable}}.$$

The first optimizes export. The second promises strict versioned reconstruction.

61.1 Portable value grammar

$$\begin{aligned} v ::= {} & \text{null} \mid \text{bool} \mid \text{int} \mid \text{finite float} \mid \text{string} \\ & \mid [v_1, \dots, v_n] \\ & \mid \{\text{type} : t, \text{version} : n, \text{data} : v\}. \end{aligned}$$

Built-in structural tags:

$$\begin{aligned} &\{\text{python.fraction}, \text{python.complex}, \text{python.path}, \text{python.bytes}, \text{python.tuple}, \\ &\ \text{python.set}, \text{python.frozenset}, \text{python.mapping}, \text{numpy.ndarray}\}. \end{aligned}$$

$$\text{keys}(v) = \{\text{type}, \text{version}, \text{data}\}.$$

61.2 Codec and migration schemas

$$K = (T, t, n, e, d),$$

where T is represented type, t immutable tag, n current version, $e : T \to \mathbb{V}$, and $d : \mathbb{V} \to T$.

$$\mu = (t, n, n+1, m), \qquad m : \mathbb{V}_n \to \mathbb{V}_{n+1}.$$

$$\text{UniqueTag}(t) \land \text{UniqueType}(T), \qquad \text{Adjacent}(\mu) \iff \mu.to = \mu.from + 1.$$

61.3 Canonical encoding and digest

$$E(v) = \text{to_portable_data}(v), \qquad C(v) = \text{Canon}(E(v)),$$

$$d(v) = \text{SHA}(\text{UTF8}(C(v))).$$

$$v_1 = v_2 \implies d(v_1) = d(v_2)$$

for deterministically encoded values.

61.4 Envelope schema

$$P(v) = \begin{cases} \texttt{schema}: & \texttt{komorebi-math.portable}, \\ \texttt{schema_version}: & 1, \\ \texttt{producer_version}: & \texttt{1.0.0b4}, \\ \texttt{payload_sha256}: & d(v), \\ \texttt{payload}: & E(v). \end{cases}$$

No additional or missing envelope field is accepted.

61.5 Decode relation

$$\text{loads_portable}(P) = D(\text{VerifyDigest}(P)).$$

$$\begin{aligned} \text{UTF8} &\to \text{JSON} \to \text{UniqueKeys} \to \text{ExactEnvelopeSchema} \\ &\to \text{FinitePayload} \to \text{DigestMatch} \to \text{KnownTag} \\ &\to \text{MigrationPath} \to \text{Decoder}. \end{aligned}$$

$$\text{DecodeAuthority} = \text{RegisteredCodecTable}\,.$$

No arbitrary class import or instantiation is permitted.

61.6 Resource guards

$$\delta(v) \le d_{\max}, \qquad d_{\max} = 128 \text{ by default.}$$

Reject excessive depth, duplicate keys, nonfinite numbers, invalid base64, unhashable or duplicate mapping keys, negative ndarray dimensions, shape mismatch, unknown tags, unsupported versions, missing migrations, and decoder failures.

61.7 Atomic file persistence

$$v \xrightarrow{\text{dumps_portable}} s \xrightarrow{\text{write+flush+fsync}} p_{tmp} \xrightarrow{\text{replace}} p.$$

$$\text{load_portable}(\text{save_portable}(v, p)) \cong v.$$

$$\text{FailureBeforeReplace} \implies \text{OriginalDestinationUnchanged}\,.$$

61.8 Frozen codec contract

$$K_c = (s, n, p, (k_1, \dots, k_{51}))$$

with schema `komorebi-math.portable-codec-contract`, version 1.

$$\begin{aligned} K_{old} \preceq K_{new} \iff\ & \forall k \in K_{old} : \text{tag}(k) \in K_{new} \\ & \wedge \text{type}_{new}(k) = \text{type}_{old}(k) \\ & \wedge \text{version}_{new}(k) \ge \text{version}_{old}(k) \\ & \wedge \forall i \in [n_{old}, n_{new} - 1] : (k, i, i+1) \in \mathcal{M}. \end{aligned}$$

$$|K| = 51, \qquad |\mathcal{M}| = 1.$$

`nonlinear.packet_jet_snapshot` is version 2 with migration $1 \to 2$.

61.9 Persistence example ladder

- E_0: `Fraction` roundtrip
- E_1: ndarray shape/dtype roundtrip
- E_2: registered Komorebi object
- E_3: nested heterogeneous state
- E_4: packet-jet snapshot migration
- E_A: payload tamper rejection
- E_B: unknown-tag rejection
- E_C: depth-limit rejection
- E_D: atomic overwrite policy

62. Provenance, registries, and extensions

62.1 Claim-status alphabet

$$\mathcal{Q} = \{\text{definition}, \text{proved}, \text{classical input}, \text{conditional theorem}, \text{finite certificate}, \text{heuristic}, \text{conjecture}, \text{planned}\}.$$

This is the public `ClaimStatus` alphabet, not `Exactness`. Do not impose a false total order: `ClaimStatus` identifies claim kind, not scalar confidence.

62.2 Provenance record

$$R_p = (i, t, q, s, A, L, V).$$

- i: immutable identifier
- t: title
- $q \in \mathcal{Q}$: status
- s: source
- A: assumptions
- L: limitations
- V: verification suite

$$\text{describe} : \text{Object} \rightharpoonup R_p.$$

$$\mathsf{ClaimBox} = (i, q, A, L, V, \text{ObservedEvidence}).$$

$$q = \text{finite certificate} \not\Rightarrow q = \text{proved global theorem}.$$

62.3 Feature registry

$$F = (\text{key}, \text{module}, \text{title}, \text{status}, \text{summary}, \text{papers}, \text{dependencies}, \text{public_objects}), \qquad \text{key}_i = \text{key}_j \implies i = j.$$

$$\text{status} \in \{\text{implemented}, \text{partial}, \text{experimental}, \text{planned}, \text{external theorem required}\}.$$

unless explicit replacement authority is supplied.

$$\text{register_feature}, \quad \text{feature}, \quad \text{features}.$$

62.4 Extension schema

$$X = (n, v, a, M, D, d, r).$$

- n: name
- v: extension version
- a: extension API version
- M: contributed modules
- D: optional dependencies
- d: description
- r: registration callback

$$X.\,\text{activate}() : r();\ \text{return}\ \emptyset.$$

$$\mathcal{E}_P = \text{entry_points}(\texttt{komorebi_math.extensions}).$$

$$L = (\text{entry_point}, \text{loaded}, \text{extension}, \text{error}).$$

With `discover_extensions(activate=True)`, successful activation is implicit in `loaded=True`. An activation exception is captured as `loaded=False` plus `error`.

62.5 Extension isolation

$$\begin{aligned}
\text{FailureIsolated}(L_i) &\iff \neg L_i.\mathit{loaded} \implies \forall j \neq i : \text{DiscoveryContinues}(L_j),\\
\text{CoreStableUnchanged}(X) &\iff \mathcal{S}^{\mathit{after}}_{\mathit{stable}} = \mathcal{S}^{\mathit{before}}_{\mathit{stable}},\\
\text{CodecNotImplicitlyStable}(X) &= \top.
\end{aligned}$$

Required examples:

- installable miniature distribution
- discovery without activation
- explicit activation
- failing import isolated into a diagnostic
- explicit warning for extension codec contract scope

63. Complete CLI reference

63.0 Invocation equivalence

$$\text{exec}(\texttt{komorebi}\ a) = \text{exec}(\texttt{python}\ {-}\texttt{m}\ \texttt{komorebi_math}\ a).$$

Equality means the same parser and semantics under one interpreter/environment.

63.1 Command contract schema

$$C_c = (P_c, O_c, D_c, I_c, J_c, X_c, W_c, E_c).$$

- P_c: positional grammar
- O_c: option grammar
- D_c: defaults
- I_c: input decoder
- J_c: stdout/stderr or JSON schema
- X_c: exit-code map
- W_c: filesystem writes and overwrite policy
- E_c: executable examples

$$\forall c \in \mathcal{C}_{CLI} : \exists!\ \text{Anchor}(c).$$

63.2 Exit semantics

$$\text{exit}(c) = \begin{cases} 0, & \text{RequestedPredicate}(c) = \top, \\ 1, & \text{ValidationOrRequiredCapability}(c) = \bot, \\ 2, & \text{UsageOrChoiceError}(c). \end{cases}$$

63.3 Output and write discipline

$$\text{stdout}(c) = \text{JSON}(R_c) \oplus \backslash\text{n}$$

when JSON is promised.

$$\text{MayWrite}(c, p) \iff p \in W_c \wedge (\neg\,\text{Exists}(p) \vee \text{–force}).$$

63.4 Identity, API, and verification grammar

$$\begin{aligned} \text{info} &: () \to R_{identity}, \\ \text{doctor} &: () \to R_{environment}, \\ \text{features} &: () \to F^*, \\ \text{describe OBJECT} &: \mathbb{S} \to R_p \cup \{\emptyset\}, \\ \text{api check} &: (m, s, t) \to R_{api}, \\ \text{api snapshot} &: (p, w, t) \to p, \\ \text{golden} &: () \to R_{golden}, \\ \text{verify SUITE} &: \mathcal{V}_{51} \to R_{suite}. \end{aligned}$$

$$t \in \{\text{research}, \text{beta}, \text{stable}\}.$$

63.5 Release grammar

$$\begin{aligned} \text{release contract} &: () \to R_{contract}, \\ \text{release environment} &: () \to R_{env}, \\ \text{release qualify} &: (g, r, p, w) \to R_{qual}, \\ \text{release upgrade} &: () \to R_{upgrade}, \\ \text{release evidence} &: (current, p, w) \to R_{evidence}, \\ \text{release documentation} &: () \to R_{docs}, \\ \text{release readiness} &: (final, g, r, p, w) \to R_{ready}. \end{aligned}$$

$$\text{–gpu} = \text{execute optional probes}, \qquad \text{–require-gpu} = \text{make GPU success release-blocking}.$$

63.6 Domain-command grammar

$$\begin{aligned} \text{stream inspect PATH} &: (p, a, v) \to R_{stream}, \\ \text{stream verify PATH} &: (p, a) \to R_{verify}, \\ \text{sequence NAME} &: (n, t, i, m, c) \to R_{sequence}. \end{aligned}$$

$$\mathcal{C}_{coord} = \{\text{catalog}, \text{euler}-\text{fibonacci}, \text{recurrence}, \text{basis}, \text{model}, \text{polynomial}, \text{combinatorial}, \text{spectral}, \text{packet}, \text{formal}\}.$$

$$\mathcal{C}_{gen} = \{\text{mask}, \text{cartier}, \text{kernel}, \text{bell}\}, \quad \mathcal{C}_{arith} = \{\text{standard}-\text{window}, \text{euler}\}.$$

$$\mathcal{C}_{finite} = \{\text{deformation count}, \text{langlands tame}-\text{orbit}, \text{langlands heisenberg}, \text{langlands completion}\}.$$

Each subcommand receives input shape, accepted values, decoder, output schema, and malformed input.

63.7 Export and LaTeX CLI

$$\mathcal{C}_{export} = \{\text{formats}, \text{sequence}, \text{standard}-\text{window}, \text{euler}, \text{json}, \text{bundle}\}.$$

$$O_{cli-export} = (p, f, t, a, d, u, c, m, e, n, x, k, q, w).$$

$$\begin{aligned} \text{latex engines} &: () \to \{\text{pdflatex}, \text{xelatex}, \text{lualatex}, \text{latexmk}\} \to \mathbb{B}, \\ \text{latex compile SOURCE} &: (s, p, e, n, t, k) \to R_{compile}. \end{aligned}$$

63.8 Legacy-alias warning

$$\text{komorebi verify all} = \text{historical Block 17 aggregate}.$$

$$\text{verify all} \neq \text{verify block20} \neq \text{full pytest} \neq \text{Block 21 gate}.$$

Part XI. Connected recipes

Recipe calculus

Every recipe is

$$\mathcal{E} = (I, H, A, O, C, F, R),$$

where I is input, H hypotheses, $A = (a_1, \dots, a_n)$ actions, O expected output, C certificate predicates, F adversarial variants, and R the reproducibility record.

$$\text{ValidRecipe}(\mathcal{E}) \iff H(I) \wedge A(I) \Downarrow O \wedge \bigwedge_{c \in C} c(O) \wedge \text{Replay}(R) = O.$$

Example depth:

$$E_0 \subset E_1 \subset E_2 \subset E_3 \subset E_4,$$

representing signature, minimal value, certificate inspection, connected workflow, and persistence/export replay.

64. Exact algebra and coordinate arrows

$$\text{grammar} \to \text{operator word} \to \text{realization} \to \text{seed hull} \to \gamma_{closure}.$$

$$x \xrightarrow{A} (x_r)_r \xrightarrow{T} (y_s)_s \xrightarrow{S} \hat{x}, \qquad \gamma : \hat{x} = x.$$

$$B_s \xrightarrow{B_t^{-1} B_s} C_{s \to t} \xrightarrow{c_t = C_{s \to t} c_s} B_t c_t = B_s c_s.$$

$$(E_n)_{n<N} \to c_F \to \widehat{E} \to \gamma_{rec}.$$

$$x \xrightarrow{A} c \xrightarrow{S^\dagger} x, \qquad c \sim c + z,\ z \in \ker S.$$

65. Arithmetic and sequence arrows

$$\text{name} \to \text{SequenceSpec} \to \text{strongest model} \to (a_n)_{n<N} \to \gamma.$$

$$f(x) \to \text{Mahler mask} \to \text{Cartier sections} \to \text{finite automaton}.$$

$$(n, p, k) \to B_n \bmod p^k \to v_p(\Delta) \to \gamma_{cong}.$$

$$(i, j, p, e, w) \to K_{i,j}^{(p^e)}(w) \to \text{valuation record}.$$

$$a \to (a_{qn+r})_{r=0}^{q-1} \to \hat{a} \to \gamma_{roundtrip}.$$

66. Signal-processing arrows

$$x \to \mathrm{STFT}_{w,h}(x) \to \mathrm{ISTFT}_{w,h}(X) \to \hat{x}, \quad \|x - \hat{x}\| \le \varepsilon.$$

$$\text{filter spec} \to h \to \text{stateful filter} \to (y_b)_b \to y.$$

$$x \xrightarrow{\uparrow L} x_L \xrightarrow{h} z \xrightarrow{\downarrow M} y \to \gamma_{rate}.$$

$$x \to \text{packet tree} \to \text{costs} \to \text{best leaves} \to \hat{x}.$$

$$y = x + \epsilon \to c \to \eta_\lambda(c) \to \hat{x} \to \gamma_{risk}.$$

$$y \to \text{candidate packets} \to \widehat{J} \to \hat{x} \to (H, \gamma_{recovery}).$$

67. Nonlinear and dynamics arrows

$$F, G \to J^m F, J^m G \to J^m(F \circ G) \to \gamma_{\text{Faà di Bruno}}.$$

$$F(x) = x \to \text{rooted trees} \to \text{majorant} \to \text{radius bound}.$$

$$x_k \to \text{defect} \to \mathrm{coker}(D) \to \mathrm{branches}_{k+1} \to \text{lift tree}.$$

$$(C, F^\bullet, d) \to E_r \to d_r \to E_{r+1} \to E_\infty.$$

$$P \to \text{partition} \to \bar{P} \to \text{Bayes update} \to \gamma_{sufficiency}.$$

$$H = \begin{bmatrix} A & B \\ C & D \end{bmatrix} \to A - B(D - zI)^{-1}C \to \text{memory realization}.$$

68. Integration and expression arrows

$$p(x), [a,b] \to \mu_{[a,b]} \to \int p \, d\mu \in \mathbb{Q} \to \gamma_{exact}.$$

$$f, [a,b], H_4 \to Q_S(f) \pm B(H_4) \to \gamma_{bound}.$$

$$T(x) = Ax + b, \ \mu \to T_\# \mu, \qquad \int f \, d(T_\# \mu) = \int f \circ T \, d\mu.$$

$$e : \tau \to \mathrm{NF}(e) : \tau \to [e_1 = e_2].$$

$$(A_1, \dots, A_n) \to m[i,j] \to \text{plan} \to A_1 \cdots A_n.$$

$$e \to P(e) \to d(e) \to \text{loads}(P(e)) \to e' \to [e' \equiv e].$$

69. Quantum, deformation, and Langlands arrows

$$G \curvearrowright \mathcal{H} \to (P_\rho)_\rho \to \text{packet circuit} \to \gamma_{decomposition}.$$

$$\Phi, \mathcal{C}, \{E_a\} \to (P E_a^* E_b P)_{a,b} \to \gamma_{KL}.$$

$$\text{residual forest} \to \text{visible branches} \to \text{support packet}.$$

$$\text{Frobenius orbit} \to \text{tame packet} \to \text{Heisenberg model} \to \text{Clifford tower} \to \text{completion report}.$$

70. Export and reproducibility arrows

$$v \to D(v) \to \{R_f(D)\}_{f \in F} \to \text{bundle} \to \text{manifest check}.$$

$$\text{source} \to \text{gate} \to \{\text{wheel}, \text{sdist}, \text{zip}\} \to \text{clean extraction} \to \text{gate}.$$

Part XII. Verification, release, and reproducibility

71. Verification suites

Suite record:

$$V = (n, S, H, C, O, r), \qquad \mathrm{Pass}(V) \iff r = 0 \wedge \bigwedge_{c \in C} c = \top.$$

$$|\mathcal{V}| = 51.$$

Domain suite family:

$$\begin{aligned}\mathcal{V}_D = \{&\mathrm{core}, \mathrm{refinement}, \mathrm{nonlinear}, \mathrm{algorithms}, \mathrm{signals},\\ &\mathrm{dynamics}, \mathrm{analytic}, \mathrm{quantum}, \mathrm{arithmetic}, \mathrm{sequences},\\ &\mathrm{deformation}, \mathrm{langlands}-\mathrm{local}, \mathrm{langlands}-\mathrm{finite}-\mathrm{image},\\ &\mathrm{langlands}-\mathrm{completion}, \mathrm{langlands}, \mathrm{export}, \mathrm{generating},\\ &\mathrm{coordinates}, \mathrm{frame}-\mathrm{algebra}, \mathrm{integration}, \mathrm{expression},\\ &\mathrm{dsp}, \mathrm{hardening}, \mathrm{streaming}-\mathrm{beta}\}.\end{aligned}$$

Contract suite family:

$$\mathcal{V}_R = \{\mathrm{stable}, \mathrm{release}, \mathrm{gpu}, \mathrm{upgrade}, \mathrm{evidence}, \mathrm{documentation}, \mathrm{beta}\}.$$

Cumulative suites are the accepted `block2` through `block20` names plus `all`, enumerated exactly from `cli.contracts` in Appendix O.

Normalize checks and reports:

$$o = (n, p, d), \quad R = (s, v, (o_i)_i, p, d_R), \quad \lambda(R) = \{(o_i.n, o_i.d) : o_i.p = \bot\}.$$

Failure classes: assertion, validation exception, optional backend absence, required capability absence, resource refusal, crash, timeout, stale digest, and environment mismatch.

72. API, semantic, persistence, and upgrade contracts

72.1 API manifest

$$M_t = \{(q, k, s, o) : q \in \mathbb{S}\}, \quad t \in \{research, beta, stable\}.$$

$$\begin{aligned}\mathrm{NameExact}(M_t, S_t) &\iff \mathrm{names}(M_t) = S_t,\\ \mathrm{SignatureExact}(M_t, S_t) &\iff \forall q : \ s_M(q) = s_R(q),\\ \mathrm{OwnerResolvable}(M_t) &\iff \forall q : \mathrm{import}(o(q)) \Downarrow .\end{aligned}$$

72.2 Stable alias identity

$$\forall q \in \mathcal{S}_{stable} : \mathrm{id}(stable.q) = \mathrm{id}(v1.q) = \mathrm{id}(api.v1.q).$$

72.3 Closed semantic fixture

$$G = (v, s, \{(i, x_i, y_i, d_i)\}_{i=1}^{m}).$$

$$\mathrm{SemanticPass}(G) \iff \forall i : \mathrm{Canon}(f_i(x_i)) = y_i \wedge \mathrm{SHA}(y_i) = d_i.$$

72.4 Aggregate stable contract

$$\text{StablePass} = \text{ApiPass} \wedge \text{SemanticPass} \wedge \text{PersistencePass}\,.$$

72.5 Release and upgrade contracts

Artifact rows have the exact live shape

$$A_c = (\text{filename}, \text{sha256}, \text{count}).$$

The release contract is

$$\begin{aligned} R_c = (&\text{package_version}, \text{predecessor_version}, \text{release_line}, \text{stage}, \text{python_requires},\\ &\text{declared_python}, \text{required_platforms}, \text{stable_api}, \text{beta_api}, \text{research_api},\\ &\text{semantics}, \text{persistence}, \text{upgrade}, \text{evidence}, \text{documentation},\\ &\text{stable_aliases}, \text{required_cli_suites}, \text{schema}, \text{schema_version}). \end{aligned}$$

Its verifier binds artifact filenames, digests, and counts to the live package:

$$\begin{aligned} \text{ReleaseContractPass} \iff\ &\text{ArtifactIdentity} \wedge \text{LiveApi} \wedge \text{LiveSemantics} \wedge \text{LivePersistence}\\ &\wedge \text{Upgrade} \wedge \text{Evidence} \wedge \text{Documentation} \wedge \text{AliasIdentity}\\ &\wedge \text{CLIRoutes} \wedge \text{SupportPolicy}\,. \end{aligned}$$

The accepted predecessor set and exact upgrade-record shape are

$$P = \{0.15.0a2, 0.16.0a2, 1.0.0b1, 1.0.0b2, 1.0.0b3\},$$

$$U = (\text{target_version}, P, \text{stable_api}^*, \text{stable_semantics}, \text{portable_codecs}, \text{fixtures}^*, \text{schema}, \text{schema_version}).$$

The committed contract contains five stable-API baselines, one `0.16.0a2` semantic baseline, one `0.16.0a2` 51-codec baseline, and seven digest-bound portable fixtures. It does not assign a separate semantic or codec baseline to every predecessor.

$$\text{UpgradePass}(U) \iff \text{ArtifactIntegrity} \wedge \text{ApiRelations} \wedge \text{FixtureLoadable} \wedge \text{SemanticReplay} \wedge K_{0.16.0a2} \preceq K_{current}.$$

73. Platform and release qualification

Environment and evidence:

$$E = (python, implementation, executable, platform, machine, packages, backends, hardware, time),$$

$$\begin{aligned} E_v = (&\text{identifier}, \text{platform_key}, \text{platform}, \text{package_version}, \text{python}, \text{numpy},\\ &\text{source}, \text{source_sha256}, \text{inherited_via_upgrade_contract}, \text{results}, \text{optional}). \end{aligned}$$

$$\text{DeclaredSupported}(E) \not\equiv \text{ExecutedSuccessfully}(E, R) \not\equiv \text{HardwareQualified}(E, H).$$

Implemented currentness is package-version based, while source integrity is checked separately:

$$\text{CurrentPlatform}(E_v) \iff E_v.\text{package_version} = __version__,$$

$$\text{SHA256}(\text{data}/E_v.\text{source}) = E_v.\text{source_sha256}.$$

For the archived reference handoff,

$$\text{current} = \{\text{linux–!x86_64}\}, \qquad \text{missing_current} = \{\text{windows–!x86_64}\}.$$

Thus `check_validation_evidence()` passes the beta policy, while `check_validation_evidence(require_current_all_platforms=True)` fails.

Qualification:

$$Q = Q_{alias} \wedge Q_{api} \wedge Q_{semantic} \wedge Q_{analysis/synthesis} \wedge Q_{transfer} \wedge Q_{polyphase} \wedge Q_{arithmetic} \wedge Q_{persistence} \wedge Q_{settings} \wedge Q_{release} \wedge Q_{upgrade} \wedge Q_{evidence} \wedge Q_{support} \wedge Q_{python} \wedge Q_{version}.$$

$$Q(g, r) = Q \wedge (g \Rightarrow Q_{gpu_observed}) \wedge (r \Rightarrow Q_{gpu_passed}).$$

$$B_{ready} = Q \wedge Q_{documentation} \wedge Q_{beta_version} \wedge Q_{contracts}.$$

Windows qualification transcript recorded in the archived source:

$$N_{pass} = 1148, \qquad N_{skip} = 4, \qquad N_{fail} = 0,$$

$$\text{StrictMkDocs} = \top, \quad d_R = \text{4aed4f725e7592d0783ede586b7940dc80374fbe67123bf1acd507520c26e2e3}.$$

The four skips are optional JAX/PyTorch cases. This transcript was supplied from an external Windows replay. Its JSON record was not committed in the archived handoff. The cited `BLOCK21_WINDOWS_HOTFIX1_PROVENANCE.json` was also absent from that tree. Its committed evidence contract therefore inherited Windows evidence from `0.16.0a2`. Current-platform evidence requires importing and digest-binding the actual Windows JSON, then regenerating the evidence contract, release-contract artifact, and dependent handoff records. The separate committed Linux gate has report digest `9ece0eef69115b4519cfcc266eacb16d0881bfb3d3bc4b3b4d5c36e900ef46d5`.

74. Source publication handoff

74.1 Publication gate

The checks always appended form

$$G_{base} = G_{identity} \wedge G_{layout} \wedge G_{paths} \wedge G_{research\ api} \wedge G_{beta\ api} \wedge G_{stable\ api} \wedge G_{stable\ verify} \wedge G_{block20} \wedge G_{release} \wedge G_{upgrade} \wedge G_{evidence} \wedge G_{readiness} \wedge G_{tour} \wedge G_{links}.$$

For `full_tests` flag f and MkDocs mode $m \in \{\text{required}, \text{auto}, \text{skipped}\}$,

$$G_{21}(f, m) = G_{base} \wedge (f \Rightarrow G_{tests}) \wedge (m = \text{required} \Rightarrow G_{mkdocs}).$$

In auto mode, MkDocs becomes a check only when importable. Otherwise the report records a warning. The strict handoff prerequisite is distinct:

$$G_{21}^{strict} = G_{base} \wedge G_{tests} \wedge G_{mkdocs}.$$

74.2 Capability record

$$T = (schema, version, package, capabilities, passed, digest),$$

with polyphase refinement, analysis/synthesis, Euler/Fibonacci coordinates, finite integration, typed expression, and stable contract keys.

$$\text{SHA}(\text{Canon}(T \setminus \{digest\})) = T.digest.$$

74.3 Reproducible distributions and handoff

$$\text{Build}(S, N)_1 = (W_1, D_1), \quad \text{Build}(S, N)_2 = (W_2, D_2),$$

$$G_{repro-dist} = \text{ExactBytes}(W_1, W_2) \wedge \text{ExactBytes}(D_1, D_2).$$

Payload ledger:

$$L_S = \{(p, m, n, d) : p \in \text{Payload}(S)\}.$$

$$G_{archive} = G_{relative} \wedge G_{no-traversal} \wedge G_{no-symlink} \wedge G_{regular-only} \wedge G_{NFC} \wedge G_{casefold-unique} \wedge G_{excluded-local-state}.$$

$$G_{repro-zip} = \text{ExactBytes}(Z_1, Z_2).$$

74.4 Clean replay and wrapper arrows

$$Z \xrightarrow{\text{extract_safe}} S' \xrightarrow{G_{21}^{strict}} R'.$$

$$\begin{aligned} G_{replay} = G_{archive} &\wedge \text{Full}(R') \wedge \text{NamedChecks}(R') \wedge \text{SelfDigest}(R') \\ &\wedge \text{VersionMatch}(R') \wedge \text{InventoryMatch}(S', L_S) \wedge \text{PayloadHashes}(S', L_S) \\ &\wedge \text{EncodedModes}(S', L_S) \wedge \text{EmbeddedManifestEqual}(S', L_S). \end{aligned}$$

$$Z \to \text{venv} \to \text{wheel[publication]} \to \text{pip check} \to \text{preflight} \to \text{MkDocs} \to G_{21}^{strict}.$$

$$S \to G_{21}^{strict} \to G_{repro-dist} \to G_{repro-zip} \to G_{replay}.$$

74.5 Publication boundary

Qualified artifacts do not imply

$$\text{PyPIPublished} \vee \text{GitHubReleaseCreated} \vee \text{AuthenticatedTagCreated}\,.$$

74A. Source reconciliation

Forbidden in current instructions:

- predecessor wheel `0.16.0a2`
- a superseded wrapper as the canonical publication gate
- stale `support_matrix_0_16.json`
- `coordinates packet --size`
- `verify all` described as complete
- MkDocs described as PDF compilation
- historical evidence labeled current
- a license description inconsistent with the declared release license

$$G_{reconcile} = \forall f \in F_{stale} : f \notin \text{CurrentInstructions}\,.$$

Historical reproduction requires

$$\text{HistoricalLabel} = (version, date, purpose, replacement).$$

Part XIII. Exhaustive reference appendices

Appendices A through C. Stable, beta, and research symbol ledgers

Canonical symbol row:

$$S = (id, q, a, t, k, s, D, C, H, R, X, E, P, B, K, V, L).$$

Field	Required content
id	stable guide anchor
q	fully qualified public name
a	exported aliases
t	API tiers
k	class, function, data, enum, or protocol
s	exact normalized signature
D, C	mathematical domain and codomain
H	hypotheses and validation constraints

Field	Required content
R	return/result schema
X	exceptions and failure set
E	exactness and certificate semantics
P	persistence/export support
B	backend/dependency profile
K	complexity/resource notes
V	verifying tests and suites
L	related symbols and recipes

$$|S_{stable}| = 131, \qquad |S_{beta}| = 83, \qquad |S_{research}| = 1312.$$

$$\mathrm{Cov}(T) = \frac{|\{q \in T : \exists\, \mathrm{CompleteSymbolRow}(q)\}|}{|T|}.$$

$$\mathrm{Cov}_{Block14}(stable) = 1, \qquad \mathrm{Cov}_{Block14}(beta), \mathrm{Cov}_{Block14}(research) \text{ are not yet closed.}$$

The all-three equality is a normative coverage requirement, not evidence that a particular execution supplied every record.

Alias quotient:

$$q_1 \sim q_2 \iff \mathrm{id}(\mathrm{resolve}(q_1)) = \mathrm{id}(\mathrm{resolve}(q_2)).$$

$$\pi : S_{exports} \to S_{exports}/\sim .$$

Alias row:

$$A = (alias, canonical, identity_equal, tiers, deprecated, replacement).$$

Appendix D. Module reference ledger

$$M = (module, public, owner, domain, exports, tiers, optional_deps, examples, tests, docs, status).$$

$$|M_{implementation}| = 201, \qquad \forall m \in M_{implementation} : \exists!\, \mathrm{ModuleAnchor}(m).$$

Internal modules receive maintainer rows but no public cards.

Appendix E. Executable-example crosswalk

$$E = (path, language, anchor, symbols, imports, H, I, A, O, C, F, R, D, T).$$

Add dependency profile, platform, runtime class, deterministic seed, working directory, artifact hashes, cleanup behavior, tests, and historical/current status.

$$|E_{python}| = 57, \qquad |E_{powershell}| = 8.$$

$$\begin{aligned} G_{example-anchor} &= \forall e : \exists! a(e), \\ G_{example-symbol} &= \forall e : \mathrm{symbols}(e) \neq \emptyset, \\ G_{example-test} &= \forall e : \exists t(e) \text{ or documented manual gate}, \\ G_{example-replay} &= \forall e : \mathrm{Replay}(e) = \mathrm{Expected}(e). \end{aligned}$$

Source of truth:

$$\mathrm{paths}(E) = \mathrm{Files}(\text{examples/*.py}) \cup \mathrm{Files}(\text{examples/*.ps1}).$$

Appendix F. Complete CLI option matrix

$$C = (path, positionals, options, defaults, choices, decoder, output, exit, writes, force, ps, posix, tests).$$

$$\mathcal{C}_{CLI} = \text{WalkSubparsers}(\text{build_parser}()), \qquad \text{paths}(C) = \mathcal{C}_{CLI}.$$

Every row includes one successful invocation and one invalid-input invocation.

Appendix G. Portable schemas and migrations

Codec row:

$$K = (tag, version, type, encoder_shape, decoder_shape, migrations, digest, limits, examples, tests).$$

Migration row:

$$M = (tag, from, to, input_schema, output_schema, total, lossless, fixture, test).$$

$$|K| = 51, \quad |M| = 1, \quad \text{tags}(K) = \text{portable_codecs}().tags.$$

Appendix H. Certificate and theorem-ID index

$$T = (theorem_id, symbol, status, H, checked, unchecked, residuals, diagnostics, example, suite).$$

$$\forall id : \exists! T(id), \qquad \forall \gamma.theorem_id \neq \emptyset : \exists T(\gamma.theorem_id).$$

Appendix I. Exceptions and diagnostics

$$X = (exception, base, predicate, meaning, recovery, example, cli_exit, tests).$$

Required families:

- mathematical validation
- dimension/relation mismatch
- unavailable backend
- exactness loss
- failed assumptions
- resource refusal
- planned feature
- persistence/schema/tamper failure
- export/compiler/path failure

Appendix J. Backend and dependency matrix

$$D = (capability, base, extra, package, min_version, native, platform, probe, fallback, required_by).$$

$$B = (name, operations, exactness, dtypes, devices, availability, selection, tests, limits).$$

$$\text{installed} \neq \text{importable} \neq \text{device available} \neq \text{qualified}.$$

Appendix K. Complexity and resource budgets

$$K = (symbol, input_parameters, time, memory, exact_growth, backend, guard, empirical_fixture).$$

$$T(n, m, p), \quad M(n, m, p), \quad B_{depth}, \quad B_{nodes}, \quad B_{bytes}.$$

Unmeasured asymptotics remain symbolic and are not presented as benchmarks.

Appendix L. Migration and deprecation

$$L = (old, new, introduced, deprecated, removed, adapter, semantic_change, recipe, test).$$

$$L = L_{current} \sqcup L_{historical}.$$

Appendix M. Glossary and notation

$$N = (glyph, spoken, definition, first_anchor, domain, collisions, see_also).$$

$$G = (term, definition, package_meaning, classical_meaning, anchors, symbols).$$

Appendix N. Documentation-source crosswalk

$$D = (path, version, status, target_anchors, claims, commands, stale_tokens, resolution).$$

$$\text{status}(D) \in \{current, merged, historical, superseded, corrected\}.$$

$$\forall d \in \text{MkDocsSources} : \exists! \, \text{CrosswalkRow}(d).$$

Appendix O. Test and verification crosswalk

$$T = (nodeid, module, symbols, claims, examples, dependencies, platform, markers, result, suite).$$

$$V = (name, runner, child_suites, tests, claims, expected_count, current, status).$$

$$\text{TestsSymbol}(t, s), \quad \text{TestsExample}(t, e), \quad \text{VerifiesClaim}(t, c), \quad \text{BelongsToSuite}(t, v).$$

$$U = \{s \in S_{public} : \nexists t : \text{TestsSymbol}(t, s)\}.$$

Release acceptance requires $U = \emptyset$ or an explicit justified documentation-only class.

Part XIV. Documentation standards and design specifications

75. Stage 1: symbolic architecture

$$I_1 = (source, manifests, cli, examples, tests, docs, schemas, release\ artifacts).$$

$$I_1 \to (L_{symbols}, L_{modules}, L_{cli}, L_{examples}, L_{schemas}, L_{claims}, L_{tests}).$$

$$G_1 = G_{counts} \wedge G_{aliases} \wedge G_{anchors} \wedge G_{crosswalks} \wedge G_{reconcile}.$$

No prose drafting begins for an incomplete domain ledger.

76. Stage 2: function cards

$$(manifest, object, source, docs, tests) \to \text{SymbolCard}.$$

$$\text{CompleteCard}(s) \iff \bigwedge_{f \in \{q,t,k,s,D,C,H,R,X,E,P,B,K,V,L\}} f(s) \neq \text{missing}.$$

Explicit `not applicable`, `not measured`, or `not supported` is allowed. Silent blanks are not.

$$G_2 = \forall s \in S_{public} : \text{CompleteCard}(s).$$

77. Stage 3: executable examples

$$N = (id, source, environment, command, stdout, stderr, artifacts, returncode, digest).$$

$$\text{Run}(N, E) = (o, e, a, r, t, d).$$

$$G_3 = \forall N : r = 0 \wedge \text{ExpectedOutput}(N, o) \wedge \text{ExpectedArtifacts}(N, a).$$

Platform-specific snippets carry a platform predicate rather than a silent skip.

$$G_{adv} = \forall f \in \text{InterfaceFamilies} : |A_f| \geq 1.$$

78. Stage 4: connected tutorials

$$T = (goal, prerequisites, objects, steps, checkpoints, final\ claim, replay, extensions).$$

$$G_4 = \forall T : \text{CleanWheelReplay}(T) \wedge \text{ClaimMatchesCertificate}(T).$$

Provide $T_{Windows}$ and T_{POSIX} for shell-sensitive workflows.

79. Stage 5: reference validation

Coverage vector:

$$\mathbf{c} = (c_s, c_b, c_r, c_{cli}, c_e, c_k, c_m, c_d, c_t), \qquad \mathbf{c} = \mathbf{1}.$$

$$G_{links} = \forall \ell \in \text{LocalLinks} : \text{TargetExists}(\ell).$$

$$G_{anchors} = \text{Unique}(\text{Anchors}) \wedge \forall r \in \text{Refs} : r \in \text{Anchors}\,.$$

$$G_{claims} = \forall c : \text{Status}(c) \wedge \text{Hypotheses}(c) \wedge \text{Limitations}(c) \wedge \text{VerificationRef}(c).$$

$$G_{drift} = L_{committed} = \text{GenerateLedgers}(source).$$

80. Stage 6: publication formats

80.1 Canonical-source graph

$$\text{Markdown}_{canonical} \to \begin{cases} \text{MkDocs HTML}, \\ \text{LaTeX}_{curated} \to \text{PDF}, \\ \text{coverage.json}, \\ \text{symbol_index.json}. \end{cases}$$

80.2 HTML gate

$$G_{html} = G_{mkdocs-strict} \wedge G_{links} \wedge G_{assets} \wedge G_{navigation} \wedge G_{search-index}.$$

Observe return code 0, resolved navigation, resolved links, present assets, preserved mathematics, and searchable symbol anchors.

80.3 PDF gate

$$G_{pdf} = G_{latex-render} \wedge G_{compile} \wedge G_{nonempty} \wedge G_{fonts} \wedge G_{figures} \wedge G_{tables} \wedge G_{toc} \wedge G_{index} \wedge G_{visual-QA}.$$

$$Q_{pdf} = (pages, rendered\ images, overflow, clipping, orphans, figure\ scale, table\ fit, pass).$$

Before the PDF stage, reflow every long tuple and conjunction with `aligned`, `gathered`, or `multline`. Symbolic delimiter balance and successful compilation do not imply that display boxes fit the page. The rendered-page audit must observe zero material overflow or clipping.

$$G_{html} \not\Rightarrow G_{pdf}, \qquad G_{pdf} \not\Rightarrow G_{html}.$$

80.4 Build record and reproducibility

$$P_g = (schema, version, source\ digest, commit, toolchain, counts, gates, artifacts, time).$$

$$A = (name, path, media\ type, size, sha256, reproducible).$$

$$\text{GuideBuild}(S, N)_1 = \text{GuideBuild}(S, N)_2$$

when toolchains permit byte normalization. Otherwise require semantic and rendered-page identity with a documented normalization boundary.

81. Definition of done

$$\begin{aligned} G_{guide} = {} & G_1 \wedge G_2 \wedge G_3 \wedge G_4 \wedge G_{links} \wedge G_{anchors} \wedge G_{claims} \wedge G_{drift} \\ & \wedge G_{html} \wedge G_{pdf} \wedge G_{reconcile} \wedge G_{example-replay} \wedge G_{cli} \wedge G_{schema} \wedge G_{tests}. \end{aligned}$$

$$\begin{gathered} \text{Cov}(\mathcal{S}_{stable}) = 1, \qquad \text{Cov}(\mathcal{S}_{beta}) = 1, \quad \text{Cov}(\mathcal{S}_{research}) = 1, \\ \text{Cov}(\mathcal{C}_{CLI}) = 1, \quad \text{Cov}(\mathcal{E}_{examples}) = 1, \quad \text{Cov}(\mathcal{K}_{codecs}) = 1, \\ \text{Cov}(\mathcal{V}_{suites}) = 1, \quad \text{Cov}(\mathcal{D}_{sources}) = 1. \end{gathered}$$

$$\forall n \in \text{Snippets} : \text{Run}(n, \text{DeclaredEnvironment}(n)).passed.$$

$$\forall c \in \text{Claims} : \text{WordingStrength}(c) \leq \text{EvidenceStrength}(c).$$

Finite certificates remain finite. Numerical residuals remain tolerance-qualified. Optional backend skips remain skips. Historical evidence remains historical.

$$\mathcal{A}_{guide} = \{\text{source.md}, \text{site/}, \text{user_guide.pdf}, \text{coverage.json}, \text{symbol_index.json}, \text{build_record.json}, \text{SHA256SUMS}\}.$$

$$\boxed{\text{CompleteGuide} \iff G_{guide} \wedge \forall a \in \mathcal{A}_{guide} : \text{Exists}(a) \wedge \text{Nonempty}(a) \wedge \text{DigestRecorded}(a)}$$

Specification boundary

These specifications preserve object schemas, domains and codomains, workflow arrows, resource and validity invariants, verification predicates, coverage ledgers, example fields, and independent HTML/PDF requirements. Explanations and observed outputs must agree with this architecture and the more precise implemented contracts.